CSS

MDN Web Docs (Mozilla) — English

Fonte oficial: https://developer.mozilla.org/en-US/docs/Web/CSS
Gerado para o projeto doc-frog

CSS: Cascading Style Sheets

Cascading Style Sheets (**CSS**) is a stylesheet language used to describe the presentation of a document written in HTML or XML (including XML dialects such as SVG, MathML or XHTML). CSS describes how elements should be rendered on screen, on paper, in speech, or on other media.

CSS is among the core languages of the **open web** and is standardized across web browsers according to W3C specifications. Previously, the development of various parts of CSS specification was done synchronously, which allowed the versioning of the latest recommendations. You might have heard about CSS1, CSS2.1, or even CSS3. There will never be a CSS3 or a CSS4; rather, everything is now just "CSS" with individual CSS modules having version numbers.

After CSS 2.1, the scope of the specification increased significantly and the progress on different CSS modules started to differ so much, that it became more effective to develop and release recommendations separately per module. Instead of versioning the CSS specification, W3C now periodically takes a snapshot of the latest stable state of the CSS specification and individual modules progress. CSS modules now have version numbers, or levels, such as CSS Color Module Level 5.

Beginner's tutorials

Our learn web development core modules contain modern, up-to-date tutorials covering CSS fundamentals.

Your first website: Styling the content

> This article provides a brief tour of what CSS is and how to use it, aimed at people who are completely new to web development.

CSS styling basics

> This module provides all the CSS fundamentals you'll need to start learning the technology effectively, including syntax, features, and techniques.

CSS text styling

> Here we look at CSS text fundamentals including setting font, boldness, italics, line and letter spacing, and drop shadows. We round off the module by looking at applying custom fonts to your page, and styling lists and links.

CSS layout

> This module looks at floats, positioning, other modern layout tools, and building responsive designs that will adapt to different devices, screen sizes, and resolutions.

Guides

CSS guides are organized by modules and are there to help you learn what you can achieve using CSS. Browse the complete list at CSS guides, which includes topics such as:

- CSS syntax including declarations and rulesets.
- Specificity, inheritance, and cascade
- Nesting, scoping and shadow parts
- Media and container queries
- Numeric and textual data types
- Box model and margin collapse
- Containing block
- Stacking and block-formatting contexts
- Property value processing
- Shorthand properties
- Flexible box, multi-column and grid layouts
- Animations, transitions, and transforms

How to

CSS layout cookbook

> Recipes for common layout patterns you might need to implement in your sites. These recipes provide code you can use as a starting point in your projects. These recipes also highlight the different ways layout specifications can be used and the choices you can make as a developer.

Tools

Border-image generator

> Generate CSS `border-image` values.

Border-radius generator

> Generate CSS `border-radius` effects.

Box-shadow generator

> Add `box-shadow` effects to your CSS objects.

Color format converter

> Enter or pick a color and copy its corresponding value in any CSS color format.

Color mixer

> Mix two colors in any color space using the `color-mix()` function and copy the resulting color in any CSS color format.

Shape generator

> Define coordinates and syntax for `<basic-shape>` features.

You can also use the following resources:

- W3C CSS Validation Service: To check if your CSS is valid. This is an invaluable debugging tool.
- Firefox Developer Tools: To view and edit a page's live CSS via the Inspector and the Style Editor tools.
- Web Developer extension: To track and edit live CSS on websites in Firefox.

Reference

Browse the complete CSS reference documentation.

CSS properties

> Reference for all CSS properties.

CSS selectors

Reference for CSS selectors, combinators, pseudo-classes, and pseudo-elements.

CSS at-rules

Reference for CSS at-rules including media queries.

CSS values

Reference for CSS keywords, data types, and functions.

See also

- Web languages to which CSS is often applied: HTML, SVG, MathML, XHTML, and XML.
- Stack Overflow questions about CSS

CSS guides

This page lists CSS modules, which reflect how CSS specifications are structured. Each module provides an overview of the specification, lists the features it provides (such as properties, at-rules, and data types), and shows how they relate to other parts of CSS and the web platform. You can use the module pages as a starting point to navigate through the related guides and dive deeper into reference documentation for specific features.

To access the individual guides within each module, click on a module name from the following list to visit its landing page, or use the sidebar to navigate directly to a topic and discover all related guides.

CSS anchor positioning

The **CSS anchor positioning** module defines features that allow you to tether elements together. Certain elements are defined as **anchor elements**; **anchor-positioned elements** can then have their size and position set based on the size and location of the anchor elements to which they are bound.

CSS animation triggers

The **CSS animation triggers** module provides functionality to trigger standard time-based CSS animations when a specific timeline trigger occurs.

CSS animations

The **CSS animations** module lets you animate the values of CSS properties, such as background-position and transform, over time by using keyframes. Each keyframe describes how the animated element should render at a given time during the animation sequence. You can use the properties in the animations module to control the duration, number of repetitions, delayed start, and other aspects of an animation.

CSS backgrounds and borders

The **CSS backgrounds and borders** module provides properties for adding backgrounds, borders, rounded corners, and box shadows to elements.

CSS basic user interface

The **CSS basic user interface** module allows you to define the rendering and functionality of features related to the user interface, including outline properties, visual feedback for pointing devices and keyboards, and modifying the default appearance of UI widgets.

CSS borders and box decorations

The **CSS borders and box decorations** module provides properties for adding borders, shaped corners, and box shadows to elements. This module extends borders and box decorations introduced in the CSS backgrounds and borders module, adding `corner-shape` and `border-shape` properties, logical `border-radius` properties, longhand properties for the `box-shadow` property, and properties to create partial borders.

CSS box alignment

The **CSS box alignment** module specifies CSS features relating to the alignment of boxes within their containers. It defines the alignment of the various CSS box layout models including block layout, table layout, flexible box layout (flexbox), and grid layout, creating a consistent alignment method across all of CSS.

CSS box model

The **CSS box model** module defines the `margin` and `padding` properties, which along with the height, width and border properties, make up the CSS box model.

CSS box sizing

The **CSS box sizing** module enables you to specify how elements fit their content or fit into a particular layout context. It defines sizing, minimum sizing, and maximum sizing properties, and also extends the CSS sizing properties with keywords that represent content-based intrinsic size and context-based extrinsic size.

CSS cascading and inheritance

The **CSS cascading and inheritance** module defines the rules for assigning values to properties by way of cascading and inheritance. This module specifies the rules for finding the specified value for all properties on all elements.

CSS color adjustment

The **CSS color adjustment** module provides a model and controls automatic color adjustment by the user agent to handle user preferences, such as "Dark Mode", contrast adjustment, and other color scheme preferences.

CSS colors

The **CSS colors** module defines colors, color types, color blending, opacity, and how you can apply these colors and effects to HTML content.

CSS compositing and blending

The **CSS compositing and blending** module defines how an element's background layers can be blended together, how an element can be blended with its container, and whether the element must create a new stacking context.

CSS conditional rules

The **CSS conditional rules** module defines CSS media and support queries, enabling you to define styles that are only applied if specific conditions are met. The conditional rules defined in this module are based on device, user-agent, and viewport capabilities. With conditional rules, you can target CSS styles based on query values or browser and device features, independent of the document being rendered.

CSS containment

The **CSS containment** module defines containment and container queries.

CSS counter styles

The **CSS counter styles** module lets you define your own counter styles to manage the appearance of markers in lists and counters in generated content. It also enables you to extend native browser list styles with your own customizations.

CSS custom functions and mixins

The **CSS custom functions and mixins** module allows developers to create reusable blocks of CSS code that can accept arguments, contain complex logic (defined using features such as CSS `if()` functions and `@media` at-rules), and return values based on that logic.

CSS custom highlight API

The **CSS custom highlight API** module provides a programmatic way to target specific ranges of text defined by range objects, without affecting the underlying DOM structure. The range objects can then be selected via `::highlight()` pseudo-elements, and have highlight styles added and removed. The features of this module can create highlight effects similar to how text editors highlight spelling or grammar errors, and code editors highlight syntax errors.

CSS custom properties for cascading variables

The **CSS custom properties for cascading variables** module adds support for cascading variables in CSS properties and lets you create custom properties to define these variables along with the mechanisms to use custom properties as the values for other CSS properties.

CSS display

The **CSS display** module defines how the CSS formatting box tree is generated from the document element tree and defines properties controlling it.

CSS easing functions

The **CSS easing functions** module defines easing functions, which provide a means to control the transformation of values. The functions defined include linear, cubic bezier, and step easing functions. These easing functions can be applied to animations and transitions.

CSS environment variables

The **CSS environment variables** module defines the concept of environment variables and the `env` function. Environment variables work similarly to custom properties and the `var` function, except that they are globally defined; they are global variables scoped to the entire document. They are user agent values, provided by the browser or operating system, you can access using the `env` function enabling you to adapt your styles to the user's device or context.

CSS filter effects

The properties in the **CSS filter effects** module let you define a way of processing an element's rendering before the element is displayed in the document. Examples of such effects include blurring and changing the intensity of the color of an element.

CSS flexible box layout

The **CSS flexible box layout** module defines a CSS box model optimized for user interface design, and the layout of items in one dimension. In the flex layout model, the children of a flex container can be laid out in any direction, and can "flex" their sizes, either growing to fill unused space or shrinking to avoid overflowing the parent. Both horizontal and vertical alignment of the children can be easily manipulated.

CSS font loading

The **CSS font loading** module describes events and interfaces used for dynamically loading font resources.

CSS fonts

The **CSS fonts** module defines font-related properties and how font resources are loaded. It lets you define the style of a font, such as its family, size and weight, and the glyph variants to use when multiple are available for a single character.

CSS fragmentation

The **CSS fragmentation** module defines how content is displayed when it is broken (fragmented) and flows across multiple pages, regions, or columns. This module defines features for pagination, breaking variable fragment size and orientation, widows and orphans.

CSS gaps

The **CSS gaps** module lets you specify spacing, or "gaps", between items in multi-column, flexbox, and grid layouts. The CSS multi-column layout module originally defined gaps and rules that control the spacing between columns in multi-column containers. This module expands on these visible separators, called *gap decorations*, extending it to grid and flexbox.

CSS generated content

The **CSS generated content** module defines how an element's content can be replaced and content can be added to a document with CSS.

CSS grid layout

The **CSS grid layout** module excels at dividing a page into major regions or defining the relationship in terms of size, position, and layering between parts of a control built from HTML primitives.

CSS images

The **CSS images** module defines the types of images that can be used (the `image` type, containing URLs, gradients and other types of images), how to resize them and how they, and other replaced content, interact with the different layout models.

CSS inline layout

The **CSS inline layout** module defines the block-axis alignment and sizing of inline-level content and adds a special layout mode for drop-caps. It describes the CSS formatting model for a flow of elements and text inside a container to be wrapped across multiple lines.

CSS lists and counters

The **CSS lists and counters** module enables styling and positioning of list item bullets and manipulating their values with a combination of strings, counters, and other features.

CSS logical properties and values

The **CSS logical properties and values** module defines logical properties and values that can control layout through logical rather than physical direction and dimension mappings. Logical properties define direction-relative equivalents to their corresponding physical properties.

CSS masking

The **CSS masking** module defines masking and clipping, two different graphical operations that are used to partially or fully hide portions of visual elements.

CSS media queries

The **CSS media queries** module enables testing and querying of viewport values and browser or device features, to conditionally apply CSS styles based on the current user environment. Media queries are used in the CSS `@media` rule and other contexts and languages such as HTML and JavaScript.

CSS motion path

The **CSS motion path** module allows authors to animate any graphical object along a custom path.

CSS multi-column layout

The **CSS multi-column layout** module lets you divide content across multiple columns. By using the properties in this module, you can define the preferred number and width of columns, the gap size between columns, and the visual appearance of the optional column dividing lines (known as column rules). You can also define how content should flow from column to column and how to break content

between columns.

CSS namespaces

The **CSS namespaces** module defines the syntax for using namespaces in CSS.

CSS nesting

The **CSS nesting** module defines a syntax for nesting selectors, providing the ability to nest one style rule inside another, with the selector of the child rule relative to the selector of the parent rule.

CSS overflow

The **CSS overflow** module properties enable you to handle scrollable overflow in visual media.

CSS overscroll behavior

The **CSS overscroll behavior** module provides properties to control the behavior of a scroll container when its scroll position reaches the scroll boundary. Controlling this aspect is particularly useful in scenarios where embedded scrollable areas should not trigger scrolling of the parent container.

CSS paged media

The **CSS paged media** module defines the properties that control the presentation of content for print or any other media that splits content into discrete pages. It allows you to set page breaks, control printable areas, and style left and right pages differently.

CSS positioned layout

The **CSS positioned layout** module defines the coordinate-based positioning and offsetting schemes available in CSS and the properties used to position and stack elements on a web page. The module is best known for defining the basic positioning methods, including relative positioning, sticky positioning, absolute positioning, and fixed positioning. It also defines how positioned elements are painted and layered, clarifying stacking behavior and visual order.

CSS properties and values API

The **CSS properties and values API** module defines a method for registering new CSS properties, defining the property's data type, inheritance behavior, and, optionally, an initial value. This API expands on CSS custom properties for cascading variables module, which allows authors to define custom properties in CSS using two dash syntax (`--`). The CSS properties and values API is part of the CSS Houdini umbrella of APIs.

CSS pseudo-elements

The **CSS pseudo-element** module defines abstract elements that are not directly present in the document tree. These abstract elements, called pseudo-elements, represent portions of the render tree that can be selected and styled. Pseudo-elements are used to create abstractions about the document tree beyond those provided by the document tree.

CSS round display

The **CSS round display** module defines CSS extensions to support a round display, such as wrist-watches, to help developers build web pages suitable for those devices.

CSS ruby layout

The **CSS ruby layout** module provides the rendering model and formatting controls related to the display of ruby annotations. Ruby annotations are a form of interlinear annotation, consisting of short runs of text alongside the base text. They are typically used in East Asian documents to indicate pronunciation or define meaning.

CSS scoping

The **CSS scoping** module defines the CSS scoping and encapsulation mechanisms, focusing on the Shadow DOM scoping mechanism.

CSS scroll anchoring

The **CSS scroll anchoring** module defines a mechanism to prevent page movement due to DOM changes above the visible region of a scrolling box while the user is consuming the visible content.

CSS scroll snap

The **CSS scroll snap** module provides properties that let you control the panning and scrolling behavior by defining snap positions. Content can be snapped into position as the user scrolls overflowing content within a scroll container, providing paging and scroll positioning.

CSS scroll-driven animations

The **CSS scroll-driven animations** module provides functionality that builds on the CSS animations module and Web Animations API. It allows you to animate property values along a scroll-based timeline rather than the default time-based document timeline. This means that you can animate an element by scrolling the element, its scroll container, or its root element, rather than just by the passing of time.

CSS scrollbars styling

The **CSS scrollbars styling** module defines properties that you can use for visual styling of scrollbars. You can customize the scrollbar width as needed. You can also customize the scrollbar *track* color (the background) and the scrollbar *thumb* color (the draggable handle).

CSS selectors

The **CSS selectors** module defines the patterns to select elements to which a set of CSS rules are then applied along with their specificity. The CSS selectors module provides us with more than 60 selectors and five combinators. Other modules provide additional pseudo-class selectors and pseudo-elements.

CSS shadow parts

The **CSS shadow parts** module defines the `::part()` pseudo-element that can be set on a shadow host. Using this pseudo-element, you can enable shadow hosts to expose the selected element in the shadow tree to the outside page for styling purposes.

CSS shapes

The **CSS shapes** module describes geometric shapes. It also defines CSS properties that can use the shapes to control the geometry of an element's float area; this area can then be applied to exclusions, or specify an element's content area.

CSS syntax

The **CSS syntax** module describes, in general terms, the structure and syntax of cascading stylesheets, or CSS. It defines CSS as the language for describing the rendering of structured documents (such as HTML and XML), on the web and elsewhere.

CSS table

The **CSS table** module helps you define how to lay out table data.

CSS text

The **CSS text** module defines how to perform text manipulation, like line breaking, justification and alignment, white space handling, and text transformation.

CSS text decoration

The **CSS text decoration** module defines features relating to text decoration, such as underlines, text shadows, and emphasis marks. Text decoration features can provide visual cues for spelling errors, grammar issues, and links. These features can help improve the usability, accessibility, functionality, and aesthetics of your text.

CSS transforms

The **CSS transforms** module defines how elements styled with CSS can be transformed in two-dimensional or three-dimensional space.

CSS transitions

The **CSS transitions** module specifies functionality for creating gradual transitions between different CSS property values. The behavior of these transitions can be controlled by specifying their easing function, duration, and other values.

CSS values and units

Every CSS declaration consists of a property/value pair. The value can take various forms depending on the property, such as a single integer, keyword, function, or a combination of different items; some values have units, while others do not. Every property also accepts the CSS-wide values. The CSS values and units module defines the data types — values and units — that CSS properties accept. This module also defines the CSS value definition syntax, or formal grammar, used to define the set of valid values for every CSS property and function.

CSS view transitions

The **CSS view transitions** module defines the behavior of the View Transition API, which allows developers to create animated transitions between different states within a document and across documents. This module also defines the CSS properties and pseudo-elements for styling these transitions.

CSS viewport

The **CSS viewport** module enables specifying the size, zoom factor, and orientation of the user-agent's initial containing block, or *viewport*.

CSS will-change

The **CSS will change** module provides a hint to the browser to enable optimizations that can improve rendering performance by doing potentially expensive work before it is required via the `will-change` property.

CSS writing modes

The **CSS writing modes** module defines support for various international writing modes and their combinations, including left-to-right and right-to-left text ordering as well as horizontal and vertical orientations.

CSSOM view

The **CSSOM view** module lets you manipulate the visual view of a document, including getting the position of element layout boxes, obtaining the width or height of the viewport through script, and also scrolling an element.

WebXR DOM overlays

The **WebXR DOM overlays** module, an immersive web specification, expands the WebXR Device API to allow HTML content — such as text and controls — to be displayed during an immersive virtual reality (VR) or augmented reality (AR) session.

See also

- CSS reference
- Learn CSS
- Layout cookbook

CSS anchor positioning

The **CSS anchor positioning** module defines features that allow you to tether elements together. Certain elements are defined as **anchor elements**; **anchor-positioned elements** can then have their size and position set based on the size and location of the anchor elements to which they are bound.

In addition, the specification provides CSS-only mechanisms to:

- Specify a set of alternative positions for an anchored element; when the default rendering position causes it to overflow its containing block and/or be rendered offscreen, the browser will try rendering the anchored element in the alternative positions instead.
- Declare conditions under which anchor-positioned elements should be hidden, in situations where it is not appropriate to tether them to anchor elements.

Reference

Properties

- `anchor-scope`
- `anchor-name`
- `position-anchor`
- `position-area`
- `position-try-fallbacks`
- `position-try-order`
- `position-try` shorthand
- `position-visibility`

At-rules and descriptors

- `@position-try`

Functions

- `anchor()`
- `anchor-size()`

Data types and values

- `anchor-center`
- `<anchor-side>`
- `<anchor-size>`
- `<position-area>`
- `<try-size>`
- `<try-tactic>`

HTML attributes

- `anchor`

Interfaces

- `CSSPositionTryDescriptors`
- `CSSPositionTryRule`
- `HTMLElement.anchorElement`

Guides

Using CSS anchor positioning

An introductory guide to fundamental anchor positioning concepts, including associating, positioning, and sizing elements relative to their anchor.

Fallback options and conditional hiding for overflow

A guide to the mechanisms CSS anchor positioning provides to prevent anchor-positioned elements from overflowing their containing elements or the viewport, including position try fallback options and conditionally hiding elements.

Using anchored container queries

Explains how to use anchored container queries to conditionally apply styles to anchor-positioned elements depending on what position try fallback options are active on them.

Related concepts

- CSS logical properties and values module:
  - `inset-block-start`
  - `inset-block-end`
  - `inset-inline-start`
  - `inset-inline-end`
  - `inset-block`
  - `inset-inline`
  - `inset` shorthand
  - `inline-size`
  - `min-block-size`
  - `min-inline-size`
  - `block-size`
  - `max-block-size`
  - `max-inline-size`
  - `margin-block`
  - `margin-block-end`
  - `margin-block-start`
  - `margin-inline`
  - `margin-inline-end`
  - `margin-inline-start`
  - Inset properties glossary term
- CSS positioned layout module:
  - `top`
  - `left`
  - `bottom`
  - `right`
- CSS box model module:
  - `width`
  - `height`
  - `min-width`
  - `min-height`
  - `max-width`
  - `max-height`
  - `margin`
  - `margin-bottom`
  - `margin-left`
  - `margin-right`
  - `margin-top`
- CSS box alignment module:
  - `align-items`
  - `align-self`
  - `justify-items`
  - `justify-self`
  - `place-items`
  - `place-self`

See also

- CSS scroll anchoring module
- Learn: CSS positioning
- CSS logical properties and values module
- Learn: Sizing items in CSS

Using anchored container queries

CSS anchor positioning includes mechanisms for providing fallback options. These are alternative positions that the browser can try placing an anchor-positioned element in, relative to its anchor, to put it back on-screen if the positioned element starts to overflow the viewport.

Anchored container queries further increase the usefulness of anchor positioning fallback options by enabling different styling of the anchor-positioned element depending on which fallback position it is placed in. This guide shows how to use anchored container queries and provides a couple of examples.

> **Note:** For information on the basic fundamentals of CSS anchor positioning, see Using CSS anchor positioning.

Feature summary

When positioning a tooltip relative to a UI element using anchor positioning, it is useful to provide `position-try` fallback options, via the `position-try-fallbacks` property. These can be used to ensure that the tooltip is displayed on-screen for as long as possible.

For example, if the tooltip is placed above the UI element it is anchored to by default, if the user scrolls upward you can use fallbacks to move the tooltip below the element just before the tooltip goes off-screen.

One problem this doesn't solve on its own is updating the styling of the anchor-positioned element to suit the different fallback options. For example, it is common to include a small arrow on the tooltip that points to the anchor element it is associated with, improving UX by making the visual association clearer. When the tooltip moves to a different position, the position and orientation of the arrow must also change, otherwise it will look wrong.

To solve this problem, you can use anchored container queries. These extend the functionality of CSS container queries to enable you to detect when a specific fallback option is applied to an anchor-positioned element, and apply CSS to its descendants as a result. Specifically, anchored container queries rely on two features:

- The `container-type` property `anchored` value: Apply this to the anchor-positioned element to start detecting when different fallback options are applied to it.
- The `@container` at-rule `anchored()` function: This is given a `fallback` descriptor as an argument. The descriptor's value is a `position-try-fallbacks` value.

For example, let's say we have a tooltip element that is positioned above its anchor by default via a `position-area` value of `top`, but has a `position-try-fallbacks` value of `flip-block` specified. This will cause the tooltip to flip in the block direction to the bottom of its anchor when it starts to overflow the top of the viewport. If we want to detect when the fallback is applied to the tooltip, we first need to set `container-type: anchored` on it to turn it into an anchored query container.

```
.tooltip {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: top;
  position-try-fallbacks: flip-block;
  container-type: anchored;
}
```

With this in place, we can now write a container query like so:

```
@container anchored(fallback: flip-block) {
  /* Descendant styles here */
}
```

The query test — `anchored(fallback: flip-block)` — will return true when the `flip-block` fallback option is applied to the tooltip, in which case the styles specified within the `@container` block will be applied. You might for example want to change the position and orientation of an arrow so that it points upwards rather than downwards, or change the direction of a gradient.

> **Note:** Bear in mind that, as with all container queries, the applied styles can only affect descendants of the container, not the container itself. This might require you to apply some of your positioned element styles to a wrapper element inside it, rather than to the element itself, as demonstrated in Multiple fallbacks example.

Basic usage example

This example includes an anchor element that has an infobox positioned relative to it. Initially, the infobox is positioned above the anchor and includes an arrow pointing down towards the anchor. We include a position try fallback so that the infobox moves below the anchor when the content scrolls up enough that the infobox starts to scroll off the top of the viewport. In addition, we use an anchored container query to change styles once the fallback kicks in, moving the arrow and pointing it upwards instead.

The anchor and infobox are represented by two `<div>` elements, as shown below. They are surrounded by text content in the final rendering to cause the page to scroll, but we've hidden it for brevity:

```
<div class="anchor">⚓</div>

<div class="infobox">Infobox</div>
```

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<div class="anchor">⚓</div>

<div class="infobox">Infobox</div>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

In our CSS, we start by specifying the `anchor` `<div>` as an anchor element by giving it an `anchor-name` of `--my-anchor`.

```
* {
  box-sizing: border-box;
}

html {
  font-family: sans-serif;
}

body {
  width: 80%;
  max-width: 600px;
  margin: 0 auto;
}

p {
  font-size: 1.4em;
  line-height: 1.5;
}

.anchor {
  font-size: 2em;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: blue;
  width: fit-content;
  padding: 5px 10px;
}

.infobox {
  color: white;
  background-color: black;
  font-size: 1.4em;
  padding: 10px;
  margin: 1rem;
  border-radius: 10px;
}

.infobox::before {
  color: black;
  font-size: 1rem;
  margin: 0;
  line-height: 0.5;
  left: 0;
  width: 100%;
  text-align: center;
}

@supports not (container-type: anchored) {
  body::before {
    content: "Your browser does not support anchored container queries.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
.anchor {
  anchor-name: --my-anchor;
}
```

Next, we give the `infobox` `<div>` a `position` value of `fixed` and a `position-anchor` value of `--my-anchor` to associate it with the anchor element. We then give the infobox a `position-area` value of `top` to position it above the anchor element and a `position-try-fallbacks` value of `bottom` so that the infobox will be moved below the anchor when it starts to overflow the top of the viewport as the content is scrolled upwards.

Finally, we set a `container-type` value of `anchored` on the infobox to designate it as an anchored query container, meaning that we can now detect when different `position-try-fallbacks` are active on the infobox via `@container` at-rules, and update styles on its descendants as a result.

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top;
  position-try-fallbacks: bottom;
  container-type: anchored;
}
```

Now we'll add the arrow to the infobox using generated content on its `::before` pseudo-element. We set the pseudo-element's `content` property to a suitable down arrow icon, position it absolutely, and set its `top` property to `105%` to position it at the bottom of the infobox (we set it to more than `100%` so that it visually matches up with the position of the corresponding up arrow).

```
.infobox::before {
  content: "▼";
  position: absolute;
  top: 105%;
}
```

Next we add the anchored container query. We include a `@container` at-rule with its test defined as `anchored(fallback: bottom)`. This means that when the `bottom` position-try fallback is applied to the infobox, the CSS inside the at-rule is applied to the document. Inside, we define alternative styling for the infobox `::before` pseudo-element that swaps out the down arrow icon for an up arrow and positions it at the top of the infobox.

```
@container anchored(fallback: bottom) {
  .infobox::before {
    content: "▲";
    bottom: 100%;
    top: auto;
  }
}
```

> **Note:** There is more CSS included in this example to handle the basic styling of all the elements, but we've only shown you the parts relevant to anchored container queries. To see the full code, open the example in the MDN Playground by pressing the "Play" button on one of the code blocks or the live rendering.

This example renders like so:

Try scrolling the demo so that the anchor moves near to the top of the viewport and note how, not only does the infobox move below the anchor to remain on the screen, but the styling also updates so that the arrow icon still works for the new infobox position.

If you scroll the anchor back down towards the bottom of the viewport, the infobox will move back up above it again. We don't need to specify an additional `position-try-fallbacks` value of `top` to achieve this because `position-area: top` is the infobox's default provided position. If the provided fallbacks don't stop the anchor-positioned element from overflowing, the browser will revert to its default position.

Multiple fallbacks example

This example shows multiple position-try fallbacks and anchored container queries in action, and also addresses the problem of what to do if you want to use anchored container queries to set styles on the anchor-positioned element itself, rather than its descendants, using an extra wrapper element. The example also includes some JavaScript that allows you to move the anchor element around the screen using the mouse or the keyboard to check out the different fallbacks.

The HTML for this example includes two `<div>` elements to represent the anchor and infobox. The `anchor` `<div>` includes a `tabindex` attribute to make it keyboard-focusable, while the `infobox` `<div>` includes an extra wrapper `<div>` to apply the infobox styles to, so we can style it via `@container` at-rules.

```
<div class="anchor" tabindex="0">⚓</div>

<div class="infobox">
  <div>Infobox</div>
</div>
```

We start off our styles by specifying the `anchor` `<div>` as an anchor element, again by giving it an `anchor-name` of `--my-anchor`. We also absolutely position it so that we can move it around by setting different inset property values on it via JavaScript.

```
* {
  box-sizing: border-box;
}

html {
  font-family: sans-serif;
  height: 100%;
}

body {
  height: inherit;
}

p {
  font-size: 1.4em;
  line-height: 1.5;
}

.anchor {
  font-size: 2em;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: blue;
  width: fit-content;
  padding: 5px 10px;
}

@supports not (container-type: anchored) {
  body::before {
    content: "Your browser does not support anchored container queries.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
.anchor {
  anchor-name: --my-anchor;
  position: absolute;
}
```

Next, we position our infobox relative to our anchor by absolutely positioning it and giving it a `position-anchor` value of `--my-anchor`. This time we position it at the top-left of the anchor with a `position-area` value of `top left`. We then set three `position-try-fallbacks` — `flip-block`, `flip-inline`, and `flip-block flip-inline` — this causes the infobox to flip its position along its block axis, inline axis, or both, to stay on-screen when the anchor gets near to the different edges of the viewport.

Finally, we turn the infobox into an anchored query container by setting `container-type: anchored`.

```
.infobox {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: top left;
  position-try-fallbacks:
    flip-block,
    flip-inline,
    flip-block flip-inline;
  container-type: anchored;
}
```

At this point, we will show you the basic visual styles set on the infobox, to illustrate the fact that, in this case, we are setting these styles on the wrapper `<div>` inside the infobox rather than the infobox itself. As mentioned earlier, we are doing this so we can manipulate these styles via anchored container queries. This wouldn't be possible if they were set directly on the infobox, as it is the anchored query container.

Most notably, here we are setting a `border-radius` value that creates a rounded corner on every corner of the infobox except for the bottom-right corner. Since the infobox is positioned to the top-left of the anchor, this corner acts as an arrow, pointing at the anchor.

```
.infobox div {
  color: white;
```

```
  background-color: black;
  font-size: 1.4em;
  padding: 10px;
  margin: 1px;

  border-radius: 10px 10px 0 10px;
}
```

Finally, we define an anchored container query for each position-try fallback that may be applied to the infobox using `@container` at-rules. In each case, we alter the rounded corners applied to the infobox wrapper `<div>` so that the nearest corner to the anchor is always not rounded.

```
@container anchored(fallback: flip-block) {
  .infobox div {
    border-radius: 10px 0 10px 10px;
  }
}

@container anchored(fallback: flip-inline) {
  .infobox div {
    border-radius: 10px 10px 10px 0;
  }
}

@container anchored(fallback: flip-block flip-inline) {
  .infobox div {
    border-radius: 0 10px 10px 10px;
  }
}
```

> **Note:** Again, we've hidden most of the basic styling for brevity, as well as the JavaScript that provides the movement controls (this isn't relevant to what we are trying to demonstrate here). To see the full code, open the example in the MDN Playground by pressing the "Play" button on one of the code blocks or the live rendering.

```
const anchorDiv = document.querySelector(".anchor");

let xPos = 250;
let yPos = 120;

function setPos() {
  const maxX = document.body.clientWidth - anchorDiv.clientWidth - 25;
  const maxY = document.body.clientHeight - anchorDiv.clientHeight - 25;

  if (xPos < 25) {
    xPos = 25;
  }

  if (xPos > maxX) {
    xPos = maxX;
  }

  if (yPos < 25) {
    yPos = 25;
  }

  if (yPos > maxY) {
    yPos = maxY;
  }

  anchorDiv.style.left = `${xPos}px`;
  anchorDiv.style.top = `${yPos}px`;
}

setPos();

document.body.addEventListener("keydown", (e) => {
  if (e.key === "w") {
    yPos -= 25;
  } else if (e.key === "d") {
    xPos += 25;
  } else if (e.key === "s") {
    yPos += 25;
  } else if (e.key === "a") {
    xPos -= 25;
  }

  setPos();
});

document.body.addEventListener("click", (e) => {
  xPos = e.clientX;
  yPos = e.clientY;

  setPos();
});
```

This example renders like so:

Try moving the anchor element around the viewport by:

- Clicking the mouse (or tapping the screen if you're on a touchscreen device) in the position you want to move the anchor to.
- Using the `W`, `A`, `S`, and `D` keys to move the anchor up, left, down, and right, respectively.

When you move the anchor element near the edges of the screen, note how the infobox moves to different positions around it to stay on-screen, and also how the `border-radius` set on the infobox changes so that the non-rounded corner is always pointing at the anchor. Moving the anchor into the different corners is guaranteed to show you the different effects.

See also

- CSS container queries
- Using container size and style queries
- Using container scroll-state queries

- CSS anchor positioning module
- Learn: CSS positioning

Fallback options and conditional hiding for overflow

When using CSS anchor positioning, an important consideration is ensuring that anchor-positioned elements always appear in a convenient place for the user to interact with them, if at all possible, regardless of where the anchor is positioned. For example, when you scroll the page, anchors and their associated positioned elements will move toward the edge of the viewport. When a positioned element starts to overflow the viewport, you will want to change its position to put it back on the screen again, for example on the opposite side of the anchor.

Alternatively, in some situations it may be preferable to just hide overflowing positioned elements — for example, if their anchors are off-screen their content might not make sense.

This guide explains how to use CSS anchor positioning mechanisms to manage these issues — **position-try fallback options** and **conditional hiding**. Position-try fallback options provide alternative positions for the browser to try placing positioned elements in as they start to overflow, to keep them on-screen. Conditional hiding allows conditions to be specified under which the anchor or a positioned element will be hidden.

> **Note:** For information on the basic fundamentals of CSS anchor positioning, see Using CSS anchor positioning.

Feature summary

If a tooltip is fixed to the top-right of a UI element, when the user scrolls the content so that the UI feature is in the top-right corner of the viewport, that UI feature's tooltip will have scrolled off the screen. CSS anchor positioning solves such problems. The module's `position-try-fallbacks` property specifies one or more alternative position-try fallback options for the browser to try to prevent the positioned element from overflowing.

Position-try fallback options can be specified using:

- Predefined fallback options.
- `position-area` values.
- Custom options defined using the `@position-try` at-rule.

In addition, the `position-try-order` property allows you to specify various options that result in an available position try option being set in preference to the element's initial positioning when it first renders. For example, you might want to initially display the element in a space that has more available height or width.

The shorthand property `position-try` can be used to specify `position-try-order` and `position-try-fallbacks` values in a single declaration.

In some situations, anchor-positioned content does not make sense if the anchor is off-screen, or vice-versa. For example, you might have an anchor containing a quiz question, and answers contained in associated positioned elements, and wish to show them both together or not at all. This can be achieved with conditional hiding, which is managed via the `position-visibility` property. This property takes various values that define conditions under which overflowing elements will be hidden.

Predefined fallback options

The predefined fallback option values of the `position-try-fallbacks` property (defined as `<try-tactic>`s in the spec) will "flip" the position of the anchor-positioned element across one or both axes if the element would otherwise overflow.

The element can be set to flip across the block axis (`flip-block`), the inline axis (`flip-inline`), or diagonally across an imaginary line drawn from a corner of the anchor through its center to its opposite corner (`flip-start`). These three values flip the element, mirroring its position on an opposite side for the first two values, and an adjacent side for `flip-start`. For example, if an element positioned `10px` above its anchor starts to overflow at the top of the anchor, the `flip-block` value would flip the positioned element to be 10px below its anchor.

In this example, we include two `<div>` elements. The first one will be our anchor element, and the second one will be positioned relative to the anchor:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

We style the `<body>` element to be larger than the viewport so that we can scroll the anchor and the positioned element around in the viewport, both horizontally and vertically:

```
body {
  width: 1500px;
  height: 500px;
}
```

For illustrative purposes, we absolutely position the anchor so that it appears near the center of the initial `<body>` rendering:

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
  position: absolute;
  top: 100px;
  left: 45%;
}
```

The anchor-positioned element is given fixed positioning and tethered to the anchor's top-left corner using a `position-area`. It is given `position-try-fallbacks: flip-block, flip-inline;` to provide it with some fallback options for moving the positioned element to stop it from overflowing when the anchor gets near the edge of the viewport.

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top left;
  position-try-fallbacks: flip-block, flip-inline;
}
```

> **Note:** When multiple position try fallback options are specified, they are separated by commas, and tried in the order they are specified.

Try scrolling the demo so that the anchor starts to get near the edges:

- Move the anchor to the top of the viewport. The positioned element flips to the bottom-left of the anchor to avoid overflowing.
- Move the anchor to the left of the viewport. The positioned element flips to the top-right of the anchor to avoid overflowing.

If you move the anchor towards the top-left corner of the viewport, you'll notice a problem — as the positioned element starts to overflow in the block and inline direction, it flips back to its default top-left position and overflows in both directions, which is not what we want.

This happens because we only gave the browser position options of `flip-block` *or* `flip-inline`. We didn't give it the option of trying both at the same time. The browser tries the fallback options, looking for one that causes the positioned element to be rendered completely inside the viewport or containing block. If it doesn't find one, it renders the positioned element in its originally-defined rendering position, with no position fallback options applied.

The next section demonstrates how to fix this issue.

Combining multiple values into one option

It is possible to put multiple predefined try fallback options or custom try option names into a single space-separated try fallback option value within the comma-separated `position-try-fallbacks` list. When trying to apply these fallback options, the browser will combine the individual effects into a single combined fallback option.

Let's use a combined try fallback option to fix the problem we found with the previous demo. The HTML and CSS in this demo are the same, except for the infobox positioning styles. In this case, it is given a third position try fallback option: `flip-block flip-inline`:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

```
body {
  width: 1500px;
  height: 500px;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
  position: absolute;
  top: 100px;
  left: 45%;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top left;
  position-try-fallbacks:
    flip-block,
    flip-inline,
    flip-block flip-inline;
}
```

This means that the browser will first try `flip-block` and then try `flip-inline` to avoid overflow. If those fallback options both fail, it will then try combining the two, flipping the element's position in the block *and* inline directions at the same time. Now when you scroll the anchor towards the top *and* left edges of the viewport, the positioned element will flip over to the bottom-right.

Using <code>position-area</code> try fallback options

The predefined `<try-tactic>` try fallback options are useful but limited, as they only allow positioned element placement to be flipped across axes. What if you had an anchor-positioned element positioned to the top left of its anchor, and wanted to change its position to directly below the anchor if it started to overflow?

To achieve this, you can use a `position-area` value as a position-try fallback option, including it in the `position-try-fallbacks` list. This automatically creates a try fallback option based on that position area. In effect, it is a shortcut for creating a custom position option that contains only that `position-area` property value.

The following example shows `position-area` position try fallback options in use. We use the same HTML and CSS, except for the infobox positioning. In this case, our position-try fallback options are `position-area` values — `top`, `top-right`, `right`, `bottom-right`, `bottom`, `bottom-left`, and `left`. The positioned element will be reasonably positioned no matter which viewport edge the anchor approaches. This verbose approach is more granular and flexible than the predefined values approach.

```
<div class="anchor">⚓</div>

<div class="infobox">
```

```
  <p>This is an information box.</p>
</div>
```

```
body {
  width: 1500px;
  height: 500px;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
  position: absolute;
  top: 100px;
  left: 45%;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top left;
  position-try-fallbacks:
    top, top right, right,
    bottom right, bottom,
    bottom left, left;
}
```

> **Note:** You can't add `position-area` try fallback options into a space-separated combined position option within a position-try fallback list.

Scroll the page and check out the effect of these position-try fallback options as the anchor nears the edge of the viewport:

Custom fallback options

To use custom position fallback options that aren't available via the above mechanisms, you can create your own with the `@position-try` at-rule. The syntax is:

```
@position-try --try-fallback-name {
  descriptor-list
}
```

The `--try-fallback-name` is a developer-defined name for the position try fallback option. This name can then be specified within the comma-separated list of try fallback options within the `position-try-fallbacks` property value. If multiple `@position-try` rules have the same name, the last one in the document order overrides the others. Avoid using the same name for your try fallback options *and* your anchor or custom property names; it doesn't invalidate the at-rule, but it will make your CSS very difficult to follow.

The `descriptor-list` defines the property values for that individual custom try fallback option, including how the positioned element should be placed and sized, and any margins. The limited list of property descriptors allowed includes:

- `position-area`
- Inset properties
- Margin properties (e.g., `margin-left`, `margin-block-start`)
- self-alignment properties
- Sizing properties (`width`, `block-size`, etc.)
- `position-anchor`

The values you include in the at-rule get applied to the positioned element if the named custom-try fallback option gets applied. If any of the properties were previously set on the positioned element, those property values get overridden by the descriptor values. If the user scrolls, causing a different try fallback option or no try fallback option to be applied, the values from the previously-applied try fallback option are unset.

In this example, we set up and use several custom try fallback options. We use the same base HTML and CSS code as in the previous examples.

We start by defining four custom try fallback options using `@position-try`:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

```
body {
  width: 1500px;
  height: 500px;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
```

```
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
  position: absolute;
  top: 100px;
  left: 45%;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
@position-try --custom-left {
  position-area: left;
  width: 100px;
  margin-right: 10px;
}

@position-try --custom-bottom {
  position-area: bottom;
  margin-top: 10px;
}

@position-try --custom-right {
  position-area: right;
  width: 100px;
  margin-left: 10px;
}

@position-try --custom-bottom-right {
  position-area: bottom right;
  margin: 10px 0 0 10px;
}
```

Once our custom try fallback options are created, we can include them in the position list by referencing their names:

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top;
  width: 200px;
  margin-bottom: 10px;
  position-try-fallbacks:
    --custom-left, --custom-bottom, --custom-right, --custom-bottom-right;
}
```

Note that our default position is defined by `position-area: top`. When the infobox isn't overflowing the page in any direction, the infobox sits above the anchor, and the position-try fallback options set in the `position-try-fallbacks` property are ignored. Also, note that the infobox has a fixed width and bottom margin set. These values will change as different position-try fallback options are applied.

If the infobox starts to overflow, the browser tries the position options listed in the `position-try-fallbacks` property:

- The browser first tries the `--custom-left` fallback position. This moves the infobox to the left of the anchor, adjusts the margin to suit, and also gives the infobox a different width.
- Next, the browser tries the `--custom-bottom` position. This moves the infobox to the bottom of the anchor, and sets an appropriate margin. It doesn't include a `width` descriptor, so the infobox returns to its default width of `200px` set by the `width` property.
- The browser next tries the `--custom-right` position. This works much the same as the `--custom-left` position, with the same `width` descriptor value applied, but the `position-area` and `margin` values are mirrored to place the infobox appropriately to the right.
- If none of the other fallbacks succeed in stopping the positioned element from overflowing, the browser tries the `--custom-bottom-right` position as a last resort. This works in much the same way as the other fallback options, but it places the positioned element to the bottom-right of the anchor.

If none of the fallbacks succeed in stopping the positioned element from overflowing, the position will revert to the initial `position-area: top;` value.

> **Note:** When a position try fallback option is applied, its values will override the default values set on the positioned element. For example, the default `width` set on the positioned element is `200px`, but when the `--custom-right` position try fallback option is applied, its width is set to `100px`.

Scroll the page and check out the effect of these position-try fallback options as the anchor nears the edge of the viewport:

Styling anchor-positioned elements based on active fallback

One problem the above functionality doesn't solve is updating the styling of an anchor-positioned element to suit its different fallback options. For example, it is common to include a small arrow on a tooltip that points to the anchor element it is associated with, improving UX by making the visual association clearer. When the tooltip moves to a different position, you'll need to change the position and orientation of the arrow, otherwise it will look wrong.

To solve this problem, you can use anchored container queries. These extend the functionality of CSS container queries to enable you to detect when a specific fallback option is applied to an anchor-positioned element, and apply CSS to its descendants as a result. Specifically, anchored container queries rely on two features:

- The `container-type` property `anchored` value: Apply this to the anchor-positioned element to start detecting when different fallback options are applied to it.
- The `@container` at-rule `anchored` keyword: This is followed by a set of parentheses inside which the `fallback` descriptor is included. The descriptor's value is a `position-try-fallbacks` value.

For example, let's say we have an anchor-positioned tooltip element that is positioned above its anchor by default via a `position-area` value of `top`, but has a `position-try-fallbacks` value of `flip-block` specified. This will cause the tooltip to flip in the block direction to the bottom of its anchor when it starts to overflow the top of the viewport. If we want to detect when the fallback is applied to the tooltip, we first need to set `container-type: anchored` on it to turn it into an anchored query container.

```
.tooltip {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: top;
  position-try-fallbacks: flip-block;
```

```
  container-type: anchored;
}
```

With this in place, we can now write a container query like so:

```
@container anchored(fallback: flip-block) {
  /* Descendant styles here */
}
```

The query test — `anchored(fallback: flip-block)` — will return true when the `flip-block` fallback option is applied to the tooltip, in which case the styles specified within the `@container` block will be applied. You might for example want to change the position and orientation of the arrow icon so that it continues to point towards the anchor, change the direction of a gradient, etc.

For more information on anchored container queries and some examples, see Using anchored container queries.

Using <code>position-try-order</code>

The `position-try-order` property has a slightly different focus from the rest of the position-try features, in that it influences which position-try fallback option is applied when the positioned element is first displayed, rather than when it is being scrolled. For example, you might want to initially display the element in a space that has more available height or width than the default initial position.

The browser tests the available `position-try-fallbacks` to find which one gives the anchor-positioned element the most space in the specified direction. It will then apply that option, overriding the element's initial styling when the page first renders.

Check out the `position-try-order` reference page for more information and a live example that demonstrates the property's effect.

Conditionally hiding anchor-positioned elements

In some situations, you might want to hide an anchor-positioned element. For example, if the anchor element is clipped because it's too close to the edge of the viewport, you might want to just hide its associated element altogether. The `position-visibility` property allows you to specify conditions under which positioned elements are hidden.

By default, the positioned element is `always` displayed. The `no-overflow` value will **strongly hide** the positioned element if it starts to overflow its containing element or the viewport.

The `anchors-visible` value, on the other hand, strongly hides the positioned element if its associated anchor(s) are *completely* hidden, either by overflowing its containing element (or the viewport) or being covered by other elements. The positioned element will be visible if any part of the anchor(s) is still visible.

A strongly hidden element acts as though it and its descendant elements have a `visibility` value of `hidden` set, regardless of what their actual `visibility` value is.

Let's see this property in action.

This example uses the same HTML and CSS as in the previous examples, with the infobox tethered to the anchor's bottom edge. The infobox is given `position-visibility: no-overflow;` to hide it completely when it is scrolled upwards to the point where it starts to overflow the viewport.

```
<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique.
</p>

<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
}

body {
  width: 50%;
  margin: 0 auto;
}
```

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
```

```
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  margin-bottom: 5px;
  position-area: top span-all;
  position-visibility: no-overflow;
}
```

Scroll down the page and note how the positioned element is hidden once it reaches the top of the viewport:

See also

- CSS anchor positioning module
- Using CSS anchor positioning
- Learn: CSS positioning
- CSS logical properties and values module
- Learn: Sizing items in CSS

Using CSS anchor positioning

The **CSS anchor positioning** module defines features that allow you to tether elements together. Elements can be defined as **anchor elements** and **anchor-positioned elements**. Anchor-positioned elements can be bound to anchor elements. The anchor-positioned elements can then have their size and position set relative to the size and location of the anchor elements to which they are bound.

CSS anchor positioning also provides CSS-only mechanisms for specifying multiple alternative positions for an anchor-positioned element. For example, if a tooltip is anchored to a form field but the tooltip would otherwise be rendered offscreen in its default position settings, the browser can try rendering it in a different suggested position so it is placed onscreen, or, alternatively, hide it altogether if desired.

This article explains the fundamental anchor positioning concepts, and how to use the module's association, positioning, and sizing features at a basic level. We've included links to reference pages with additional examples and syntax details for each concept discussed below. For information on specifying alternative positions and hiding anchor-positioned elements, see the Fallback options and conditional hiding for overflow guide.

Fundamental concepts

It's very common to want to tether, or bind, one element to another. For example:

- Error messages that appear alongside form controls.
- Tooltips or infoboxes that pop up next to a UI element to provide more information about it.
- Settings or options dialogs that can be accessed to quickly configure UI elements.
- Drop-down or popover menus that appear next to an associated navigation bar or button.

Modern interfaces frequently call for some content — often reusable and dynamically-generated — to be situated relative to an anchor element. Creating such use cases would be fairly straightforward if the element to tether to (aka the **anchor element**) was always in the same place in the UI and the tethered element (aka the **anchor-positioned element**, or just **positioned element**) could always be placed immediately before or after it in the source order. However, things are rarely that simple.

The location of positioned elements relative to their anchor element needs to be maintained and adjusted as the anchor element moves or otherwise changes configuration (e.g., by scrolling, changing the viewport size, drag and drop, etc.). For example, if an element such as a form field gets close to the edge of the viewport, its tooltip may end up offscreen. Generally, you want to bind the tooltip to its form control and ensure the tooltip is kept fully visible on-screen as long as the form field is visible, automatically moving the tooltip if needed. You may have noticed this as the default behavior in your operating system when you right-click (`Ctrl` + click) context menus on your desktop or laptop.

Historically, associating an element with another element and dynamically changing a positioned element's location and size based on an anchor's position required JavaScript, which added complexity and performance issues. It also wasn't guaranteed to work in all situations. The features defined in the CSS anchor positioning module enable implementing such use cases performantly and declaratively with CSS (and HTML) instead of JavaScript.

Associating anchor and positioned elements

To associate an element with an anchor, you need to first declare which element is the anchor, and then specify which positioned element(s) to associate with that anchor. This creates an anchor reference between the two. This association can be created explicitly via CSS or implicitly.

Explicit CSS anchor association

To declare an element as an anchor with CSS, you need to set an anchor name on it via the `anchor-name` property. The anchor name needs to be a `<dashed-ident>`. In this example, we also set the anchor's `width` to `fit-content` to get a small square anchor, which better demonstrates the anchoring effect.

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
.anchor {
  anchor-name: --my-anchor;
  width: fit-content;
}
```

Converting an element to an anchor-positioned element requires two steps: It needs to be absolutely or fixed positioned using the `position` property. The positioned element then has its `position-anchor` property set to the value of the anchor element's `anchor-name` property to associate the two together:

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
}
```

We'll apply the above CSS to the following HTML:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

This will render as follows:

The anchor and infobox are now associated, but for the moment you'll have to trust us on this. They are not tethered to each other yet — if you were to position the anchor and move it somewhere else on the page, it would move on its own, leaving the infobox in the same place. You'll see the actual tethering in action when we look at positioning elements based on anchor position.

Implicit anchor association

In some cases, an implicit anchor reference will be made between two elements, due to the semantic nature of their relationship:

- When using the Popover API to associate a popover with a control, an implicit anchor reference is made between the two. This can occur when:
  - Declaratively associating a popover with a control using the `popovertarget` and `id` attributes or the `commandfor` and `id` attributes.
  - Programmatically associating a popover action such as `showPopover()` with a control using the `source` option.
- A `<select>` element and its dropdown picker are opted into customizable select element functionality via the `appearance` property `base-select` value. In this case, an implicit popover-invoker relationship is created between the two, which also means they'll have an implicit anchor reference.

> **Note:** The methods above associate an anchor with an element, but they are not yet tethered. To tether them together, the positioned element needs to be positioned relative to its anchor, which is done with CSS.

Removing an anchor association

If you wish to remove an explicit anchor association previously made between an anchor element and a positioned element, you can do one of the following:

1. Set the anchor's `anchor-name` property value to `none`, or to a different `<dashed-ident>`, if you want a different element to be anchored to it.
2. Set the `position-anchor` property of the positioned element to `none`, or to an anchor name that doesn't exist in the current document, such as `--not-an-anchor-name`.

In the case of implicit anchor associations, you'll need to use the second method — the first method doesn't work. This is because the association is controlled internally, and you can't remove the `anchor-name` via CSS.

For example, to stop a customizable `<select>` element's picker from being anchored to the `<select>` element itself, you could use the following rule:

```
::picker(select) {
  position-anchor: none;
}
```

Anchor scoping

When multiple anchor elements are given the same `anchor-name` value and a positioned element has that name as its `position-anchor` property value, the positioned element will be associated with the *last* anchor element in the source order with that `anchor-name` value.

For example, if a document contains multiple repeated components, each with a positioned element tethered to an anchor, all the positioned elements will be anchored to the last anchor on the page unless each component uses a different anchor name. This is likely not the desired behavior.

The `anchor-scope` property can fix this problem by limiting the visibility, or "scope", of an `anchor-name` value to a specific subtree. The result is that each positioned element can only be anchored to an element within the same subtree of the element that has the scope set on it.

- `anchor-scope: all` sets the scope so that *any* `anchor-name` values set in the subtree can only be bound to by positioned elements in the same subtree.
- `anchor-scope: --my-anchor, --my-anchor2` sets the scope so that the specified `anchor-name` values, when set in the subtree, can only be bound to by positioned elements in the same subtree.
- `anchor-scope: none` is the default value; it specifies that no anchor scoping is set.

For example, let's say you have multiple anchors and anchor-positioned `<div>` elements inside `<section>` containers:

```
<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 1</div>
</section>

<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 2</div>
</section>

<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 3</div>
</section>
```

We turn each `anchor` `<div>` into an anchor element by giving them an `anchor-name` of `--my-anchor`. We then position each `positioned` `<div>` relative to an element with the `--my-anchor` anchor name by giving them absolute positioning, a `position-anchor` value of `--my-anchor`, and a `position-area` value of `right`. Finally, we set the anchor scope of each `<section>` container using `anchor-scope: --my-anchor`:

```
html {
  height: 100%;
}

body {
  height: inherit;
  display: flex;
  justify-content: space-evenly;
  align-items: center;
}

.scoped {
  padding: 20px;
  background: #eeeeee;
}

.anchor {
  font-size: 1.8rem;
  color: white;
```

```
  text-shadow: 1px 1px 1px black;
  background-color: blue;
  width: fit-content;
  padding: 3px;
}

.positioned {
  background: orange;
  width: fit-content;
  padding: 3px;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}

.positioned {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: right;
}

.scoped {
  anchor-scope: --my-anchor;
}
```

This results in the following positioning behavior:

Each positioned element is positioned relative to the anchor inside the same `<section>` element. This is because each `<section>` element has an `anchor-scope` of `--my-anchor` set on it; positioned elements inside each scoped container can therefore only be positioned relative to `my-anchor` anchors inside the same container.

If we didn't set `anchor-scope: --my-anchor` on the containers, all of the positioned elements would be positioned relative to the last anchor on the page.

Positioning elements relative to their anchor

As we saw earlier, associating a positioned element with an anchor is not really much use on its own. Our goal is to place the positioned element relative to its associated anchor element. This is done either by setting a CSS `anchor()` function value on an inset property, specifying a `position-area`, or centering the positioned element with the `anchor-center` placement value.

> **Note:** CSS anchor positioning also provides mechanisms for specifying fallback positions if the positioned element's default position causes it to overflow the viewport. See the Fallback options and conditional hiding guide for details.

> **Note:** The anchor element must be a visible DOM node for the association and positioning to work. If it is hidden (for example via `display: none`), the positioned element will be positioned relative to its nearest positioned ancestor. We discuss how to hide an anchor-positioned element when its anchor disappears in Conditional hiding using `position-visibility`.

Using inset properties with <code>anchor()</code> function values

Conventional absolutely and fixed positioned elements are explicitly positioned by setting `<length>` or `<percentage>` values on inset properties. With `position: absolute`, this inset position value is an absolute distance relative to the edges of the nearest positioned ancestor. With `position: fixed`, the inset position value is an absolute distance relative to the viewport.

CSS anchor positioning changes this paradigm, enabling anchor-positioned elements to be placed relative to the edges of their associated anchor(s). The module defines the `anchor()` function, which is a valid value for each of the inset properties. When used, the function sets the inset position value as an absolute distance relative to the anchor element by defining the anchor element, the side of the anchor element the positioned element is being positioned relative to, and the distance from that side.

The function components look like this:

```
anchor(<anchor-name> <anchor-side>, <fallback>)
```

`<anchor-name>`

The `anchor-name` property value of the anchor element you want to position the element's side relative to. This is a `<dashed-ident>` value. If omitted, the element's **default anchor** is used. This is the anchor referenced in its `position-anchor` property, or associated with the element via the non-standard `anchor` HTML attribute.

> **Note:** Specifying an `<anchor-name>` positions the element relative to that anchor, but does not provide element association. While you can position an element's sides relative to multiple anchors by specifying different `<anchor-name>` values inside different `anchor()` functions on the same element, the positioned element is only associated with a single anchor.

`<anchor-side>`

Specifies the position relative to a side, or sides, of the anchor. Valid values include the `center` of the anchor, physical (`top`, `left`, etc.) or logical (`start`, `self-end`, etc.) sides of the anchor, or a `<percentage>` between the start (`0%`) and end (`100%`) of the axis of the inset property `anchor()` is set on. If a value is used that is not compatible with the inset property on which the `anchor()` function is set, the fallback value is used.

`<fallback>`

A `<length-percentage>` defining the distance to use as a fallback value if the element is not absolutely or fixed positioned, if the `<anchor-side>` value used is not compatible with the inset property on which the `anchor()` function is set, or if the anchor element doesn't exist.

The return value of the `anchor()` function is a length value calculated based on the position of the anchor. If you set a length or percentage directly on an anchor-positioned element's inset property, it is positioned as if it were not bound to the anchor element. This is the same behavior seen if the `<anchor-side>` value is incompatible with the inset property on which it is set and the fallback is used. These two declarations are equivalent:

```
bottom: anchor(right, 50px);
bottom: 50px;
```

Both will place the positioned element `50px` above the bottom of the element's closest positioned ancestor (if any) or of the initial containing block.

The most common `anchor()` parameters you'll use will refer to a side of the default anchor. You will also often either add a `margin` to create spacing between the edge of the anchor and positioned element or use `anchor()` within a `calc()` function to add that spacing.

For example, this rule positions the left edge of the positioned element flush to the anchor element's right edge, then adds some `margin-left` to make some space between the edges:

```
.positionedElement {
  left: anchor(right);
```

```
  margin-left: 10px;
}
```

The return value of an `anchor()` function is a length. This means you can use it within a `calc()` function. This rule positions the positioned element's logical block end edge `10px` from the anchor element's logical block start edge, adding the spacing using the `calc()` function so we don't need to add a margin:

```
.positionedElement {
  inset-block-end: calc(anchor(start) + 10px);
}
```

`anchor()` example

Let's look at an example of `anchor()` in action. We've used the same HTML as in the previous examples, but with some filler text placed below and above it to cause the content to overflow its container and scroll. We'll also give the anchor element the same `anchor-name` as in the previous examples:

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

body {
  width: 50%;
  margin: 0 auto;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}
```

The infobox is associated with the anchor via the anchor name and given fixed positioning. By including the `inset-block-start` and `inset-inline-start` properties (which are equivalent to `top` and `left` in horizontal left-to-right writing modes) we have tethered it to the anchor. We add a `margin` to the infobox to add space between the positioned element and its anchor:

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position-anchor: --my-anchor;
  position: fixed;
  inset-block-start: anchor(end);
  inset-inline-start: anchor(self-end);
  margin: 5px 0 0 5px;
}
```

Let's look at the inset property positioning declarations in more detail:

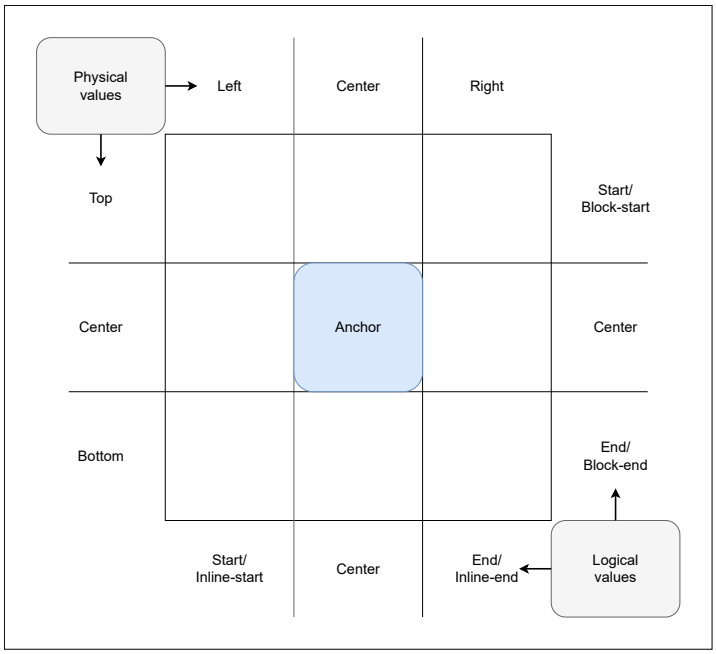
- `inset-block-start: anchor(end)`: This sets the positioned element's block start edge to the anchor's block end edge, calculated using the `anchor(end)` function.
- `inset-inline-start: anchor(self-end)`: This sets the positioned element's inline start edge to the anchor's inline end edge, calculated using the `anchor(self-end)` function.

This gives us the following result:

The positioned element is `5px` below and `5px` to the right of the anchor element. If you scroll the document up and down, the positioned element maintains its position relative to the anchor element — it is fixed to the anchor element, not the viewport.

Setting a <code>position-area</code>

The `position-area` property provides an alternative to the `anchor()` function for positioning elements relative to anchors. The `position-area` property works on the concept of a 3x3 grid of tiles, with the anchor element being the center tile. The `position-area` property can be used to position the anchor positioned element in any of the nine tiles, or have it span across two or three tiles.



The grid tiles are broken up into rows and columns:

- The three rows are represented by the physical values `top`, `center`, and `bottom`. They also have logical equivalents such as `start`, `center`, and `end`, and coordinate equivalents such as `y-start`, `center`, and `y-end`.
- The three columns are represented by the physical values `left`, `center`, and `right`. They also have logical equivalents such as `start`, `center`, and `end`, and coordinate equivalents such as `x-start`, `center`, and `x-end`.

The dimensions of the center tile are defined by the containing block of the anchor element, while the distance between the center tile and the grid's outer edge is defined by the positioned element's containing block.

`position-area` property values are composed of one or two values based on the row and column values described above, with spanning options available to define the region of the grid where the element should be positioned.

For example:

You can specify two values to place the positioned element in a specific grid square. For example:

- `top left` (logical equivalent `start start`) will place the positioned element in the top-left square.
- `bottom center` (logical equivalent `end center`) will place the positioned element in the bottom center square.

You can specify a row or column value plus a `span-*` value. The first value specifies the row or column to place the positioned element in, placing it initially in the center, and the other one specifies the amount of that column to span. For example:

- `top span-left` causes the positioned element to be placed in the top row, and span across the center and left tiles of that row.
- `y-end span-x-end` causes the positioned element to be placed in the end of the y column, and span across the center and x-end tiles of that column.
- `block-end span-all` causes the positioned element to be placed in the block end row, and span across the inline-start, center, and inline-end tiles of that row.

If you only specify one value, the effect is different depending on which value is set:

- A physical side value (`top`, `bottom`, `left`, or `right`) or coordinate value (`y-start`, `y-end`, `x-start`, `x-end`) acts as if the other value is `span-all`. For example, `top` gives the same effect as `top span-all`.
- A logical side value (`start` or `end`) acts as if the other value is set to the same value; for example `start` gives the same effect as `start start`.
- A value of `center` acts as if both values are set to `center` (so, `center center`).

> **Note:** See the `<position-area>` value reference page for a detailed description of all the available values. Mixing a logical value with a physical value will invalidate the declaration.

Let's demonstrate some of these values; this example uses the same HTML and base CSS styles as the previous example, except that we've included a `<select>` element to enable changing the positioned element's `position-area` value.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

```
<form>
  <label for="position-area-select">Choose a position-area:</label>
  <select id="position-area-select" name="position-area-select">
    <option>top</option>
    <option>bottom</option>
    <option>left</option>
    <option>right</option>
    <option>start</option>
    <option>end</option>
    <option>top left</option>
    <option>top right</option>
    <option>bottom left</option>
    <option>bottom right</option>
    <option>top span-left</option>
    <option>bottom span-right</option>
    <option>start span-start</option>
    <option>end span-end</option>
    <option>center</option>
    <option>center span-all</option>
    <option>bottom center</option>
    <option>end span-all</option>
  </select>
</form>
```

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
}

body {
  width: 50%;
  margin: 0 auto;
}

form {
  background: white;
  border: 1px solid black;
  padding: 5px;
}

select {
  display: block;
  margin-top: 5px;
}

form {
  position: fixed;
  top: 0;
  right: 2px;
}
```

The infobox is given fixed positioning and associated with the anchor using CSS. When loaded, it is set to be tethered to the anchor with `position-area: top;`, which causes it to be positioned at the top of the position-area grid. This will be overridden once you select different values from the `<select>` menu.

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top;
}
```

We also include a short script to apply new `position-area` values chosen from the `<select>` menu to the infobox:

```
const infobox = document.querySelector(".infobox");
const selectElem = document.querySelector("select");

selectElem.addEventListener("change", () => {
  const area = selectElem.value;

  // Set the position-area to the value chosen in the select box
  infobox.style.positionArea = area;
});
```

Try selecting new `position-area` values from the `<select>` menu to see the effect they have on the position of the infobox:

Positioned element width

In the above example, we did not explicitly size the positioned element in either dimension. We deliberately omitted sizing to allow you to observe the behavior this causes.

When a positioned element is placed into `position-area` grid cells without explicit sizing, it aligns with the grid area specified and behaves as if `width` were set to `max-content`. It is sized according to its containing block size, which is the width of its content. This size was imposed by setting `position: fixed`. Auto-sized absolutely and fixed-positioned elements are automatically sized, stretching as wide as needed to fit the

text content, while constrained by the edge of the viewport. In this case, when placed on the left side of the grid with any `left` or `inline-start` value, the text wraps. If the anchored element's `max-content` size is narrower or shorter than its anchor, they won't grow to match the size of the anchor.

If the positioned element is centered vertically, such as with `position-area: bottom center`, it will align with the grid cell specified and the width will be the same as the anchor element. In this case, its minimum height is the anchor element's containing block size. It will not overflow, as the `min-width` is `min-content`, meaning it will be at least as wide as its longest word.

Centering on the anchor using <code>anchor-center</code>

While you can center the anchor-positioned element using `position-area`'s `center` values, inset properties combined with the `anchor()` function provide more control over the exact position. CSS anchor positioning provides a way to center an anchor-positioned element relative to its anchor when inset properties, rather than `position-area`, are used to tether.

The properties `justify-self`, `align-self`, `justify-items`, and `align-items` (and their `place-items` and `place-self` shorthands) exist to allow developers to easily align elements in the inline or block direction inside various layout systems, for example along the main or cross axis in the case of flex children. CSS anchor positioning provides an additional value for these properties, `anchor-center`, which aligns a positioned element with the center of its default anchor.

This example uses the same HTML and base CSS as the previous example. The infobox is given fixed positioning and tethered to the anchor's bottom edge. `justify-self: anchor-center` is then used to make sure it is centered horizontally on the anchor's center:

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
  anchor-name: --my-anchor;
}

body {
  width: 50%;
  margin: 0 auto;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  top: calc(anchor(bottom) + 5px);
  justify-self: anchor-center;
}
```

This centers the anchor-positioned element at the bottom of its anchor:

Sizing elements based on anchor size

As well as positioning an element relative to its anchor's position, you can also size an element relative to its anchor's size using the `anchor-size()` function within a sizing property value.

Sizing properties that can accept an `anchor-size()` value include:

- `width`
- `height`
- `min-width`
- `min-height`
- `max-width`
- `max-height`
- `block-size`
- `inline-size`
- `min-block-size`
- `min-inline-size`
- `max-block-size`
- `max-inline-size`

`anchor-size()` functions resolve to `<length>` values. Their syntax looks like this:

```
anchor-size(<anchor-name> <anchor-size>, <length-percentage>)
```

`<anchor-name>`

The `<dashed-ident>` name set as the value of the `anchor-name` property of the anchor element you want to size the element relative to. If omitted, the element's **default anchor**, which is the anchor referenced in the `position-anchor` property, is used.

`<anchor-size>`

Specifies the dimension of the anchor element that the positioned element will be sized relative to. This can be expressed using physical (`width` or `height`) or logical (`inline`, `block`, `self-inline`, or `self-block`) values.

`<length-percentage>`

Specifies the size to use as a fallback value if the element is not absolutely or fixed positioned, or the anchor element doesn't exist.

The most common `anchor-size()` functions you'll use will just refer to a dimension of the default anchor. You can also use them inside `calc()` functions, to modify the size applied to the positioned element.

For example, this rule sizes the positioned element's width equal to the default anchor element's width:

```
.elem {
  width: anchor-size(width);
}
```

This rule sizes the positioned element's inline size to 4 times the anchor element's inline size, with the multiplication being done inside a `calc()` function:

```
.elem {
  inline-size: calc(anchor-size(self-inline) * 4);
}
```

Let's look at an example. The HTML and base CSS are the same as in the previous examples, except that the anchor element is given a `tabindex="0"` attribute to make it focusable. The infobox is given fixed positioning and associated with the anchor in the same way as before. However, this time around we tether it to the right of the anchor using a `position-area`, and give it a width five times the width of the anchor's width:

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor" tabindex="0">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
}

body {
  width: 50%;
  margin: 0 auto;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: right;
  margin-left: 5px;
  width: calc(anchor-size(width) * 5);
}
```

In addition, we increase the anchor element's `width` on `:hover` and `:focus`, and give it a `transition` so that it animates when the state changes.

```
.anchor {
  text-align: center;
  width: 30px;
  transition: 1s width;
}

.anchor:hover,
.anchor:focus {
  width: 50px;
}
```

Hover over or tab to the anchor element — the positioned element grows as the anchor grows, demonstrating that the anchor-positioned element's size is relative to its anchor:

Other uses of <code>anchor-size()</code>

You can also use `anchor-size()` in physical and logical inset and margin properties. The sections below explore these uses in more detail, before providing a usage example.

Setting element position based on anchor size

You can use the `anchor-size()` function within an inset property value to position elements based on their anchor element's size, for example:

```
left: anchor-size(width);
inset-inline-end: anchor-size(--my-anchor height, 100px);
```

This doesn't position an element relative to the position of its anchor like the `anchor()` function or `position-area` property do (see Positioning elements relative to their anchor, above); the element won't change its position when its anchor does. Instead, the element will be positioned according to the normal rules of `absolute` or `fixed` positioning.

This can be useful in some situations. For example, if your anchor element can only move vertically, and always remains next to the edge of its closest positioned ancestor horizontally, you could use `left: anchor-size(width)` to cause the anchor-positioned element to always be positioned to the right of its anchor, even if the anchor width changes.

Setting element margin based on anchor size

You can use the `anchor-size()` function within a `margin-*` property value to set element margins based on their anchor element's size, for example:

```
margin-left: calc(anchor-size(width) / 4);
margin-block-start: anchor-size(--my-anchor self-block, 20px);
```

This can be useful in cases where you want to set an anchor-positioned element's margin to be always equal to the same percentage of the anchor element's width, even when the width changes.

<code>anchor-size()</code> position and margin example

Let's look at an example where we set an anchor-positioned element's margin and position relative to the anchor element's width.

In the HTML, we specify two `<div>` elements, one `anchor` element and one `infobox` element that we'll position relative to the anchor. We give the anchor element a `tabindex` attribute so that it can be focused via the keyboard. We also include filler text to make the `<body>` tall enough to require scrolling, but this has been hidden for the sake of brevity.

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar.
</p>
```

```
<div class="anchor" tabindex="0">⚓</div>

<div class="infobox">
  <p>Infobox.</p>
</div>
```

```
<p>Vulputate ut pharetra sit amet aliquam.</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

In the CSS, we first declare the `anchor` `<div>` as an anchor element by giving it an `anchor-name`. The positioned element has its `position` property set to `absolute`, and is associated with the anchor element via its `position-anchor` property. We also set absolute `height` and `width` dimensions on the anchor and infobox, and include a `transition` on the anchor so that width changes are smoothly animated when its state changes:

```
.anchor {
  font-size: 2rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  text-align: center;
  align-content: center;
  outline: 1px solid black;
}

body {
  width: 80%;
  margin: 0 auto;
  position: relative;
}

.infobox {
  align-content: center;
  color: darkblue;
  background-color: azure;
  outline: 1px solid #dddddd;
  font-size: 1rem;
```

```
  text-align: center;
}
```

```
.anchor {
  anchor-name: --my-anchor;
  width: 100px;
  height: 100px;
  transition: 1s all;
}

.infobox {
  position-anchor: --my-anchor;
  position: absolute;
  height: 100px;
  width: 100px;
}
```

Now onto the most interesting part. Here we set the anchor's `width` to `300px` when it is hovered or focused. We then set the infobox's:

- `top` value to `anchor(top)`. This causes the top of the infobox to always stay in line with the top of the anchor.
- `left` value to `anchor-size(width)`. This causes the left of the infobox to be positioned the specified distance away from the left edge of its nearest positioned ancestor. In this case, the specified distance is equal to the anchor element's width and the nearest positioned ancestor is the `<body>` element, so the infobox appears to the right of the anchor.
- `margin-left` value to `calc(anchor-size(width)/4)`. This cases the infobox to always have a left margin separating it and the anchor, equal to a quarter of the anchor's width.

```
.anchor:hover,
.anchor:focus {
  width: 300px;
}

.infobox {
  top: anchor(top);
  left: anchor-size(width);
  margin-left: calc(anchor-size(width) / 4);
}
```

The rendered result is as follows:

Try tabbing to the anchor or hovering over it with the mouse pointer, and note how the infobox's position and left margin grow in proportion to the anchor element's width.

See also

- CSS anchor positioning module
- Fallback options and conditional hiding for overflow guide
- Learn: Positioning
- CSS logical properties and values module
- Learn: Sizing items in CSS

CSS animation triggers

The **CSS animation triggers** module provides functionality to trigger standard time-based CSS animations when a specific timeline trigger occurs.

Scroll-triggered animations allow you to control when a regular time-based animation starts, pauses, or stops based on when a trigger activates or deactivates, without using JavaScript. This could include when a scrolling element enters or leaves a timeline range.

These ranges normally come from view progress timelines, enabling, for example, an animation to start when an element enters a scrollport and pause when it leaves the scrollport. Properties let you change the timeline range and control the active and activation ranges. Triggers can specify different actions when entering versus exiting the timeline range, allowing control over animation playback.

The animation triggers module also defines **event triggers**. When supported, these will activate timeline-based animations when specific DOM events occur.

Animation triggers in action

This example demonstrates scroll-triggered animations. Scroll the box up and down — the appearance of the "bouncer" text will trigger the ball's animation, and its disappearance will trigger the same animation in reverse. Scroll further, and the same process will repeat when the "another bouncer" text appears and then disappears from the scrollport.

```
<section>
  <div>
    <p>scroll down</p>
    <p id="trigger">bouncer</p>
    <p>keep scrolling</p>
    <p>keep scrolling</p>
    <p>keep scrolling</p>
    <p id="trigger2">another bouncer</p>
    <p>scroll up</p>
  </div>
</section>

<span id="ball"><span></span></span>
```

```
#ball,
#ball span {
  animation:
    move-right 2s 1 ease-out both,
    move-right 2s 1 ease-out forwards;
  animation-trigger:
    --t play-forwards play-backwards,
    --t2 play-forwards play-backwards;
}
#ball span {
  animation-name: bounce, bounce;
}

#trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
#trigger2 {
  timeline-trigger-name: --t2;
  timeline-trigger-source: view();
}
```

```
html {
  font-family: sans-serif;
  font-size: 1.3rem;
}

* {
  box-sizing: border-box;
}

@layer scroller {
  section {
    border: solid;
    width: 200px;
    height: 250px;
    margin-top: 50px;
    overflow: scroll;
  }
  p {
    text-align: center;
    font-size: 1.5rem;
    margin: 100% 0;
  }
  #trigger,
  #trigger2 {
    padding: 0;
    color: red;
  }
}

@layer animation-setup {
  #ball {
    position: fixed;
    top: 96vh;
  }
  #ball span {
    background: red;
    border-radius: 50%;
    height: 5vw;
    display: block;
    aspect-ratio: 1/1;
  }

  @keyframes move-right {
    from {
      transform: translateX(0);
    }
    to {
      transform: translateX(90vw);
    }
  }

  @keyframes bounce {
    9%,
    24%,
    35%,
    44%,
    51%,
    58%,
    63%,
    68%,
    72%,
    76%,
    100% {
      transform: translateY(0);
      animation-timing-function: ease-out;
    }
    0%,
    17%,
    30%,
    40%,
    48%,
    55%,
    61%,
    66%,
    70%,
    74% {
      animation-timing-function: ease-in;
    }
    0% {
      transform: translateY(-96vh);
    }
    17% {
      transform: translateY(-57.6vh);
    }
    30% {
      transform: translateY(-34.56vh);
    }
    40% {
      transform: translateY(-20.74vh);
    }
    48% {
      transform: translateY(-12.44vh);
    }
    55% {
      transform: translateY(-7.46vh);
    }
    61% {
      transform: translateY(-4.48vh);
    }
    66% {
      transform: translateY(-2.69vh);
    }
```

```
    70% {
      transform: translateY(-1.61vh);
    }
    74% {
      transform: translateY(-0.97vh);
    }
  }
}
```

```
@supports not (timeline-trigger-source: view()) {
  body::before {
    content: "Your browser does not support scroll-triggered animations.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

We defined the animation triggers by specifying a `timeline-trigger-name` and `timeline-trigger-source` on the "bouncer" elements. The ball has a bouncing `animation` set on it twice. It also has an `animation-trigger` property set: this references the names of both triggers and the actions to perform when the animations are activated and deactivated. Activation and deactivation are caused by the "bouncer" elements coming into and out of view.

Reference

Properties

- `animation-trigger`
- `timeline-trigger` shorthand
- `timeline-trigger-activation-range` shorthand
  - `timeline-trigger-activation-range-end`
  - `timeline-trigger-activation-range-start`
- `timeline-trigger-active-range` shorthand
  - `timeline-trigger-active-range-end`
  - `timeline-trigger-active-range-start`
- `timeline-trigger-name`
- `timeline-trigger-source`
- `trigger-scope`

The CSS animation triggers module also introduces the `event-trigger`, `event-trigger-name`, and `event-trigger-source` properties. Currently, no browsers support these features.

Data types and values

- `<animation-action>`

Guides

Using CSS scroll-triggered animations

An introduction to implementing CSS scroll-triggered animations.

Timeline range names

The `<timeline-range-name>` data type: Understanding the various timeline range names.

Understanding timeline insets

Understanding the animation attachment range syntax used to inset scroll-driven and scroll-triggered animations.

Related concepts

- CSS animations module
  - `animation-timeline`
  - `@keyframes` at-rule
  - `<keyframe-selector>`
- CSS scroll-driven animations module
  - `animation-range` shorthand
    - `animation-range-end`
    - `animation-range-start`
  - `scroll-timeline` shorthand
    - `scroll-timeline-axis`
    - `scroll-timeline-name`
  - `timeline-scope`
  - `view-timeline` shorthand
    - `view-timeline-axis`
    - `view-timeline-inset`
    - `view-timeline-name`

See also

- Using CSS animations
- Using scroll driven animations
- CSS scroll-driven animation timelines
- CSS scroll-triggered animations are coming! on developer.chrome.com (2025)

Using CSS scroll-triggered animations

CSS **scroll-triggered animations** provide a declarative mechanism to start, pause, stop, or reverse an element's `DocumentTimeline`-based CSS animation when the user scrolls it (or a different element) to a specified offset within a scrollport.

This article covers how to create CSS scroll-triggered animations.

Scroll-triggered animation concepts

A common UI pattern involves triggering animations on a web page when the user scrolls to a certain place in the content, for example, to pull in additional UI elements or draw the user's attention to certain details.

CSS scroll-triggered animations enable defining scroll-based triggers that start and stop regular time-based CSS animations. You can define trigger positions inside a scroll container so that, when a tracked element reaches those positions within the scrollport, they toggle the play state of an animation applied to that element, or to a completely different element.

> **Note:** Scroll-triggered animations provide an alternative to using JavaScript features — such as frameworks or the Intersection Observer API — to trigger animations on scroll. CSS scroll-triggered animations are more performant and, arguably, simpler to implement.

Scroll-triggered versus scroll-driven animations

Scroll-triggered animations are similar to CSS scroll-driven animations, but they are different:

- Scroll-triggered animations are regular time-based animations that play when a trigger becomes active; they observe the `animation-delay` each time the animation starts, and always complete each iteration in the amount of time defined by the `animation-duration`, regardless of how fast the user scrolls.
- With scroll-driven animations, the normal, time-based animation timeline is replaced by a scroll-based timeline, meaning that the animation progresses forwards and backwards as you scroll towards the start and end of the content, respectively, with faster scrolling resulting in a faster animation. Scroll-driven animations ignore the `animation-duration` and `animation-delay` properties.

Scroll-triggered animation basics

Let's walk through a basic example to show you how a scroll-triggered animation works. An image caption will fade in and out when the image it is captioning is scrolled into and out of view. In this case:

- The `<figcaption>` element has a `@keyframes` animation set on it: a fade-in effect. This animation is the *triggered animation*.
- For the *animation actions*, we specify that the animation should play forwards when the trigger is *activated*, fading the caption in, and play backwards when the trigger is *deactivated*, fading the caption out.
- The *animation triggers* are defined on the `<img>` element. The activation is triggered when the `<img>` starts entering the scrollport and the deactivation occurs when the `<img>` fully exits the scrollport, meaning the full scrollport is the *timeline range*, the `<img>` is the *tracked element*, and the `<figcaption>` is the animated element.
- We'll set an anonymous view progress timeline as the *trigger source* on the `<img>` element, created using the `view()` function.

As the content is scrolled up and down, the caption's animation will start as soon as the `<img>` begins to appear inside the scrollport, with the animation reversing when the `<img>` exits the scrollport. This basic example doesn't create the effect we want, but it's a good start that we can improve on as we learn about more features.

The HTML features several paragraphs of content with a `<figure>` element included in the middle of them, which contains the `<img>` and `<figcaption>`. For the sake of brevity, we are not showing the full source.

```
...

<p>...</p>

<figure>
  <img
    src="jungle-coast.jpg"
    alt="A view across some trees towards a rocky coast" />
  <figcaption>A view of the Jungle coast</figcaption>
</figure>

<p>...</p>

...
```

```
<h1>Information about Cairns</h1>

<p>
  The countryside surrounding Cairns, located in eastern Australia, is a
  breathtakingly beautiful region characterized by diverse landscapes, lush
  greenery, and unique natural wonders.
</p>

<p>
  Nestled between the Great Dividing Range and the sparkling Coral Sea, this
  area offers a stunning blend of tropical rainforests, pristine beaches, and
  majestic mountain ranges.
</p>

<p>
  As you venture away from the city, you'll be greeted by the verdant
  rainforests of the Wet Tropics World Heritage Area. Towering ancient trees,
  vibrant ferns, and cascading waterfalls create a serene and magical
  atmosphere.
</p>
<p>
  Exploring the lush undergrowth, you may encounter unique wildlife such as
  colorful birds, tree-dwelling mammals, and rare reptiles. The sounds of
  chirping birds and rushing water add to the symphony of nature, providing an
  immersive experience in this captivating wilderness.
</p>

<p>
  Traveling further, you'll discover the Atherton Tablelands, a plateau renowned
  for its fertile farmlands, rolling hills, and picturesque lakes. This
  agricultural heartland is dotted with quaint rural towns and farms producing
  an array of fresh produce.
</p>
<p>
  Fields of sugar cane, banana plantations, and dairy farms stretch as far as
  the eye can see, creating a patchwork of vibrant green hues against the
  backdrop of distant mountains. The region is also famous for its waterfalls,
  such as Millaa Millaa Falls, where crystal-clear water tumbles down verdant
  cliffs, offering a refreshing retreat.
</p>

<figure>
  <img
    src="https://mdn.github.io/shared-assets/images/examples/learn/gallery/pic5.jpg"
    alt="A butterfly with red, white, and gold wing sections, sitting in a leaf" />
  <figcaption>A beautiful butterfly seen in the Jungle near Cairns</figcaption>
</figure>
```

```
<p>
  No visit to the countryside surrounding Cairns is complete without exploring
  the stunning beaches that line the Coral Sea. From Palm Cove to Mission Beach,
  the coastline is adorned with golden sands, swaying palm trees, and azure
  waters. These idyllic beaches provide the perfect setting for relaxation,
  swimming, and water sports.
</p>
<p>
  Snorkeling enthusiasts can also discover the wonders of the Great Barrier
  Reef, a UNESCO World Heritage Site, which lies just off the coast. This
  vibrant underwater ecosystem teems with an incredible diversity of marine
  life, including colorful coral formations, tropical fish, and sea turtles.
</p>

<p>
  Lastly, the surrounding countryside is home to an impressive array of national
  parks and mountains. Barron Gorge National Park, for instance, showcases
  rugged terrain, deep gorges, and thundering waterfalls.
</p>
<p>
  You can embark on hiking trails that lead to breathtaking viewpoints, offering
  panoramic vistas of the surrounding rainforest-clad mountains and valleys.
  Further west, the towering peaks of the Great Dividing Range present
  opportunities for adventurous hiking and exploring scenic vistas, including
  the iconic Walshs Pyramid near Gordonvale.
</p>
```

We'll start by defining `@keyframes` for the `fade-in` animation we'll apply to our `<figcaption>`.

```
@keyframes fade-in {
  from {
    opacity: 0;
  }

  to {
    opacity: 1;
  }
}
```

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

figure {
  display: block;
  margin: 0 auto;
  max-width: 60%;
  position: relative;
}

img {
  max-width: 100%;
  border: 3px solid black;
}

h1 {
  font-size: 3rem;
}

p {
  line-height: 1.5;
}

figcaption {
  font-size: 1rem;
  padding: 5px;
  position: absolute;
  top: 5px;
  left: 5px;
  border: 3px solid black;
  background: white;
}
```

The first declaration block applies the animation and an animation trigger, along with the animation actions, to the `<figcaption>` element:

1. We use the `animation` shorthand to apply the `fade-in` animation to `<figcaption>`. By itself, without triggers, this would cause the `<figcaption>` to fade into view as soon as the page loads.
2. We use the `animation-trigger` property to delay the start of the animation until the `<img>` element is scrolled into view by identifying which element will provide the triggers, and what those trigger's actions will be. The `animation-trigger` property value components include:
    - A `<dashed-ident>`, `--t`, which is the identifier set as the value of the `timeline-trigger-name` property on the triggering element.
    - Two `<animation-action>` values, which specify how the animation should behave when the trigger is activated and deactivated (the animation actions). When the trigger is activated, we set the `<figcaption>` element's animation to play forwards. When the trigger is deactivated, we set the `<figcaption>` element's animation to play backwards.

```
figcaption {
  animation: fade-in 1s ease-in both;
  animation-trigger: --t play-forwards play-backwards;
}
```

The second declaration block creates the animation trigger:

1. We use the `timeline-trigger-name` property to give the `<img>` element an identifying name for triggers created on it. This is the same dashed identifier as the trigger name referenced in the `<figcaption>` element's `animation-trigger` property value.
2. We use the `timeline-trigger-source` property to create the animation trigger type. Specifying the `view()` function means our trigger type is an anonymous view progress timeline, which has a default activation range equal to the `cover` timeline range.

```
img {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
```

By default, the `ViewTimeline` created by the `view()` function tracks the position of the `<img>` element across the block-axis of the nearest parent scroller. The element that is tracked, in this case the `<img>`, is the **subject** or the **tracked element**.

By default, the triggers are activated and deactivated when the tracked element is scrolled to the start and end of the timeline range respectively, in the block direction, causing the `<figcaption>` animation to play forwards and play backwards. Activation occurs when the tracked element enters the **activation range**, and deactivation occurs when the tracked element leaves the **active range**. When the `timeline-trigger-source` is set to `view()`, the default activation and active ranges resolve to `cover`, which is from the point when the tracked element's start border edge starts to enter the scrollport to the point when the tracked element's end border edge completely exits the scrollport.

The example renders like so:

Note how the caption starts to fade in as soon as any part of the image becomes visible in the scrollport, whether you are moving it in from the bottom or the top. It doesn't fade out again until the entire image has moved out of the scrollport, so you won't be able to see the fading out effect; if you scroll the image back into view, the caption will fade in again.

Creating the trigger on the same element

In the previous example, the trigger was defined on the `<img>` element, and the `<figcaption>` was animated. It is possible to define the trigger on the animated element itself. Let's modify the previous example to create the trigger on the animated `<figcaption>` element.

The HTML is identical to the previous example. The CSS differs only in where the `timeline-trigger-*` properties are set.

This time, the `animation`, `animation-trigger`, `timeline-trigger-name`, and `timeline-trigger-source` properties are all set on the `<figcaption>` element — it will animate when it appears in the scrollport. In the previous example, the `<figcaption>` was the animated element and the `<img>` was the tracked element. Now the caption plays both roles.

```
figcaption {
  animation: fade-in 1s ease-in both;
  animation-trigger: --t play-forwards play-backwards;
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
```

The updated rendering looks like this:

In this case, the `<figcaption>` fades into view when it, rather than the image, first enters into the scrollport.

Adjusting the trigger activation range

In the previous examples, the trigger activates (`fade-in` starts) as soon as a block edge of the tracked element enters the scrollport at one edge, and deactivates (fade-out starts: `fade-in` is played backwards) when the tracked element has finished exiting the scrollport at the opposite edge. As a result, the fade out is never visible. This is because the default activation and active ranges (see `<timeline-range-name>`) when using `view()` as the `timeline-trigger-source` are both `cover`.

To make the fade out animation visible, we can offset the start and end of the activation range using the `timeline-trigger-activation-range-start` and `timeline-trigger-activation-range-end` properties, respectively, or the `timeline-trigger-activation-range` shorthand to set both values in a single declaration. Each of these properties can take the following values:

- The `normal` default value.
- A `<length-percentage>` value to specify a point along the default range.
- A `<timeline-range-name>` keyword specifying a named range.
- A `timeline-range-name` and a `<length-percentage>` to specify a point along the named range.

Percentages are relative to the length of the `<timeline-range-name>`, which resolves to `cover` for our view progress timeline. Had we set `scroll()` as our `timeline-trigger-source`, the default `<timeline-range-name>` would have resolved to `scroll`. See Timeline range names to learn about the `<timeline-range-name>` values.

The following example will cause the trigger to activate `50%` of the way through the `entry` range (when `50%` of the tracked element has entered the scrollport via one of the scrollport's block edges) and deactivate `0%` of the way through the `exit` range (when `50%` of the tracked element has exited the scrollport's opposite block edge).

```
timeline-trigger-activation-range: entry 50% exit 0%;
```

Let's apply this to our first example so you can see what the effect is. Our `img` declaration block is updated to the following:

```
img {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: entry 50% exit 0%;
}
```

The updated rendering looks like this:

The animation of the `<figcaption>` is now a bit more useful — it only starts to fade into view when a significant portion of the `<img>` has entered the scrollport at its end edge, and it starts to fade out when the `<img>` has started to exit the scrollport at its start edge. When you scroll the content back down again, the trigger reactivates and the fade-in occurs again at the scrollport's start edge, and deactivation occurs again at the scrollport's end edge.

Setting a custom active range

The **active range** is the range within which a trigger will remain activated once activation has occurred. By default, the active range is the same as the activation range; therefore, in the examples we've seen so far, the range for activation and deactivation has been the same.

It is possible to set a different active range from the activation range using the `timeline-trigger-active-range-start` and `timeline-trigger-active-range-end` properties, or the `timeline-trigger-active-range` shorthand to set both values in a single declaration.

You might want to do this to extend the time an animation has to complete — for example, if you want the animation trigger to activate only within a small range but stay active over a larger range. Only when the tracked element moves out of the active range does the trigger become inactive; after that, you can activate it again by moving the subject back into the activation range.

Let's build on our previous examples to demonstrate the effect of the active range. The HTML is the same, except we've included two identical `<figure>` elements with classes of `.one` and `.two`, placed next to one another using flexbox, meaning they'll come in and go out of view at the same time as each other. In each case, the `<img>` will be the tracked element for its sibling animated `<figcaption>`.

```
<div class="figure-wrapper">
  <figure class="one">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/learn/gallery/pic5.jpg"
      alt="A butterfly with red, white, and gold wing sections, sitting in a leaf" />
    <figcaption>
      A beautiful butterfly seen in the Jungle near Cairns
```

```
    </figcaption>
  </figure>

  <figure class="two">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/learn/gallery/pic5.jpg"
      alt="A butterfly with red, white, and gold wing sections, sitting in a leaf" />
    <figcaption>
      A beautiful butterfly seen in the Jungle near Cairns
    </figcaption>
  </figure>
</div>
```

```
<h1>Information about Cairns</h1>

<p>
  The countryside surrounding Cairns, located in eastern Australia, is a
  breathtakingly beautiful region characterized by diverse landscapes, lush
  greenery, and unique natural wonders.
</p>

<p>
  Nestled between the Great Dividing Range and the sparkling Coral Sea, this
  area offers a stunning blend of tropical rainforests, pristine beaches, and
  majestic mountain ranges.
</p>

<p>
  As you venture away from the city, you'll be greeted by the verdant
  rainforests of the Wet Tropics World Heritage Area. Towering ancient trees,
  vibrant ferns, and cascading waterfalls create a serene and magical
  atmosphere.
</p>
<p>
  Exploring the lush undergrowth, you may encounter unique wildlife such as
  colorful birds, tree-dwelling mammals, and rare reptiles. The sounds of
  chirping birds and rushing water add to the symphony of nature, providing an
  immersive experience in this captivating wilderness.
</p>

<p>
  Traveling further, you'll discover the Atherton Tablelands, a plateau renowned
  for its fertile farmlands, rolling hills, and picturesque lakes. This
  agricultural heartland is dotted with quaint rural towns and farms producing
  an array of fresh produce.
</p>
<p>
  Fields of sugar cane, banana plantations, and dairy farms stretch as far as
  the eye can see, creating a patchwork of vibrant green hues against the
  backdrop of distant mountains. The region is also famous for its waterfalls,
  such as Millaa Millaa Falls, where crystal-clear water tumbles down verdant
  cliffs, offering a refreshing retreat.
</p>

<div class="figure-wrapper">
  <figure class="one">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/learn/gallery/pic5.jpg"
      alt="A butterfly with red, white, and gold wing sections, sitting in a leaf" />
    <figcaption>
      A beautiful butterfly seen in the Jungle near Cairns
    </figcaption>
  </figure>

  <figure class="two">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/learn/gallery/pic5.jpg"
      alt="A butterfly with red, white, and gold wing sections, sitting in a leaf" />
    <figcaption>
      A beautiful butterfly seen in the Jungle near Cairns
    </figcaption>
  </figure>
</div>

<p>
  No visit to the countryside surrounding Cairns is complete without exploring
  the stunning beaches that line the Coral Sea. From Palm Cove to Mission Beach,
  the coastline is adorned with golden sands, swaying palm trees, and azure
  waters. These idyllic beaches provide the perfect setting for relaxation,
  swimming, and water sports.
</p>
<p>
  Snorkeling enthusiasts can also discover the wonders of the Great Barrier
  Reef, a UNESCO World Heritage Site, which lies just off the coast. This
  vibrant underwater ecosystem teems with an incredible diversity of marine
  life, including colorful coral formations, tropical fish, and sea turtles.
</p>

<p>
  Lastly, the surrounding countryside is home to an impressive array of national
  parks and mountains. Barron Gorge National Park, for instance, showcases
  rugged terrain, deep gorges, and thundering waterfalls.
</p>
<p>
  You can embark on hiking trails that lead to breathtaking viewpoints, offering
  panoramic vistas of the surrounding rainforest-clad mountains and valleys.
  Further west, the towering peaks of the Great Dividing Range present
  opportunities for adventurous hiking and exploring scenic vistas, including
  the iconic Walshs Pyramid near Gordonvale.
</p>
```

We apply the same `animation` to both `<figcaption>` elements as in previous examples, but their `animation-trigger` property values reference two different `timeline-trigger-name` values.

```
figcaption {
  animation: fade-in 0.4s ease-in both;
}

.one figcaption {
  animation-trigger: --t1 play-forwards play-backwards;
}

.two figcaption {
  animation-trigger: --t2 play-forwards play-backwards;
}
```

We set the same `timeline-trigger-source` and the same `timeline-trigger-activation-range` on both `<img>` elements. The `timeline-trigger-name` for each `<img>` element reference the two different dashed identifiers in the previous code block, which means the trigger created on each `<img>` acts as the trigger for the animation on its own sibling `<figcaption>`.

The `timeline-trigger-activation-range: contain 40% contain 60%` declaration means that the trigger activates — and thus the animation starts playing — when the tracked element reaches a narrow range, the middle `20%` of the scrollport, and deactivates when the subject exits that narrow range.

By default, the activation range is the same as the active range. However, we also set a `timeline-trigger-active-range` range of `entry 50% exit 100%` on the second `<img>`. This means that, once faded in, the second `<figcaption>` will only fade out again when the second `<img>` scrolls to `exit 100%`, which is when it has completely left the scrollport.

```
img {
  timeline-trigger-source: view();
  timeline-trigger-activation-range: contain 40% contain 60%;
}

.one img {
  timeline-trigger-name: --t1;
}

.two img {
  timeline-trigger-name: --t2;
  timeline-trigger-active-range: entry 50% exit 100%;
}
```

> **Note:** To have an effect, the `timeline-trigger-active-range` must be set to a larger range than the `timeline-trigger-activation-range`.

```
.figure-wrapper {
  display: flex;
  gap: 20px;
}

figcaption {
  left: 5px;
  right: -1px;
}
```

This example renders like so:

Scroll the images into view, and then scroll them carefully up and down. Note how both captions fade into view at the same time, at a point roughly one third up the embedded page. The first caption fades out slightly further up, whereas the second caption doesn't fade out until it has been moved completely out of the scrollport. This is because both `<img>` triggers have the same *activation* range, while the second one has a much larger *active* range applied to it than the activation range.

The timeline-trigger shorthand

So far, we've written all the CSS for our scroll-triggered animation as a mixture of shorthand and longhand properties, to effectively explain each of the properties and their values. However, this is cumbersome and wordy. Now that you've understood the concepts, we can use the `timeline-trigger` shorthand to create the shortest possible equivalent. You'll likely opt for this shorthand in your future projects.

Taking these declarations as an example:

```
img {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: contain 25% contain 75%;
  timeline-trigger-active-range: entry 0% exit 100%;
}
```

We can rewrite these in a single line of CSS using the `timeline-trigger` shorthand:

```
img {
  timeline-trigger: --t view() contain 25% contain 75% / entry 0% exit 100%;
}
```

Adjusting the animation's action

All the examples so far in this guide have triggered the `fade-in` animation, causing the caption to fade in and fade out. The forwards and backwards play is controlled by the `animation-trigger` declaration set on the animated element:

```
animation-trigger: --t play-forwards play-backwards;
```

The `<animation-action>` values `play-forwards` and `play-backwards` specify that the animation plays forwards when the trigger activates, and backwards when the trigger deactivates. The first value is the animation activation action, and the second value is the animation deactivation action.

If we set the following `animation` declaration on the same element:

```
animation: fade-in 1s ease-in both;
```

The animation only happens once when the trigger is activated, and once in reverse when it is deactivated: we didn't specify an `animation-iteration-count` in our `animation` shorthand, therefore the default value, `1`, is used.

There are other `animation-action` values that can be set to produce different effects. For example:

- `play-once` causes the animation to play only once. Once finished, it won't play again on subsequent activations or deactivations.

- `play` causes the animation to play, in whatever direction it was previously playing in. In contrast, `play-forwards` and `play-backwards` interact with the animation's `playbackRate`, setting it to its absolute positive value or its absolute positive value multiplied by `-1`, respectively. This results in the animation playing forwards or backwards. Note that the `animation-direction` value is not affected.
- `pause` causes the animation to pause. For example, you could set an animation's iteration count to `infinite`, and set the corresponding `animation-trigger` to `--t play pause`: This would cause the animation to play when the trigger is activated, and pause when it deactivates.
- `reset` has the same effect as `pause`, except that additionally it sets the animation progress back to `0`.

Some of these values are designed to be used together. For example, `play-forwards play-backwards` is intended for use in cases where you want to alternate the direction of play in the animation's end visual effect, causing a UI element to "animate in" when it appears on-screen and then "animate out" again when it goes off-screen. On the other hand, `play pause` is common for animating an element as it appears, then pausing the animation as it starts to go off-screen.

Let's look at a brief example — we'll take our first example and change it so that our `<figure>` fades in only once, when has completely entered the scrollport, and doesn't fade out or animate again until the page is reloaded.

We set the `animation-trigger` property's `animation-action` to `play-once` so that the animation only plays once when the `<figure>` first enters the activation range. We've also set the `timeline-trigger-activation-range` to `contain` so that the animation only plays when the `<figure>` is completely on-screen — it only plays once, so we don't want you to miss it.

```
figure {
  animation: fade-in 1s ease-in both;
  animation-trigger: --t play-once;

  timeline-trigger: --t view() contain;
}
```

This example renders like so:

When you first scroll the `<figure>` on-screen, it will fade in. After that, it will stay at `100%` opacity regardless of how many times you scroll it up and down the scrollport. You can only get it to fade in again by refreshing the page (or reloading the embedded example's `<iframe>`).

Trigger scope

If multiple triggers use the same `timeline-trigger-name`, because of how triggers are determined by the browser by default, the triggers will be associated with the last element in the HTML source order that has that `timeline-trigger-name` value. This is likely not the desired behavior.

For example, if a document contains multiple repeated components, each containing a scroll-triggered animation where the animated element and tracked element are different elements, all of the animated elements will have their animations controlled by the last component's trigger, unless you use a different `timeline-trigger-name` in each component, or scope the name by subtree.

The `trigger-scope` property limits the visibility, or "scope", of a `timeline-trigger-name` value to a specific subtree. The result is that each animated element can only have its animation triggered by a trigger created within the same scoped subtree. See the `trigger-scope` reference page for details of how this works and a `trigger-scope` example.

Multiple scroll-triggered animations

In previous examples, we set only a single scroll-triggered animation on an element; however, all of the `animation-*` and `timeline-trigger-*` properties discussed in this guide accept a comma separated list of values to enable triggering multiple animations from multiple triggers. In this section we'll build a slightly more complex example with multiple scroll-triggered animations on the same element.

The `animation-trigger` property works in exactly the same way as the `animation` shorthand property and the other animation longhand properties with regards to setting multiple values. If multiple `animation-name` values are set, but only a single `animation-trigger` value is set, the `animation-trigger` will apply to all the animations. If two `animation-trigger` values are set, they will cycle between the animations until all of them have an `animation-trigger` value set. And so on.

This example progressively animates an element — applying further animations when new triggers are activated as the page scrolls. As the user scrolls, the element first slides in from the right of the screen, then reveals its contents, then slides down the screen and changes its background color.

The HTML is similar to previous examples except that we have included a `<section>` element at the top containing some highlighted content, and some empty `<div>` elements interspersed throughout our main content that will have triggers defined on them to trigger our animations.

```
<section>
  <h2>This content is animated!</h2>

  <p>
    The countryside surrounding Cairns, located in eastern Australia, is a
    breathtakingly beautiful region characterized by diverse landscapes, lush
    greenery, and unique natural wonders.
  </p>
</section>

<h1>Information about Cairns</h1>

...
```

```
<section>
  <h2>This content is animated!</h2>

  <p>
    The countryside surrounding Cairns, located in eastern Australia, is a
    breathtakingly beautiful region characterized by diverse landscapes, lush
    greenery, and unique natural wonders.
  </p>
</section>

<h1>Information about Cairns</h1>

<p>
  Nestled between the Great Dividing Range and the sparkling Coral Sea, this
  area offers a stunning blend of tropical rainforests, pristine beaches, and
  majestic mountain ranges.
</p>

<p>
  As you venture away from the city, you'll be greeted by the verdant
  rainforests of the Wet Tropics World Heritage Area. Towering ancient trees,
  vibrant ferns, and cascading waterfalls create a serene and magical
  atmosphere.
</p>
<p>
  Exploring the lush undergrowth, you may encounter unique wildlife such as
  colorful birds, tree-dwelling mammals, and rare reptiles. The sounds of
  chirping birds and rushing water add to the symphony of nature, providing an
  immersive experience in this captivating wilderness.
</p>
```

```
<p>
  Traveling further, you'll discover the Atherton Tablelands, a plateau renowned
  for its fertile farmlands, rolling hills, and picturesque lakes. This
  agricultural heartland is dotted with quaint rural towns and farms producing
  an array of fresh produce.
</p>

<div id="one"></div>

<p>
  Fields of sugar cane, banana plantations, and dairy farms stretch as far as
  the eye can see, creating a patchwork of vibrant green hues against the
  backdrop of distant mountains. The region is also famous for its waterfalls,
  such as Millaa Millaa Falls, where crystal-clear water tumbles down verdant
  cliffs, offering a refreshing retreat.
</p>

<p>
  No visit to the countryside surrounding Cairns is complete without exploring
  the stunning beaches that line the Coral Sea. From Palm Cove to Mission Beach,
  the coastline is adorned with golden sands, swaying palm trees, and azure
  waters. These idyllic beaches provide the perfect setting for relaxation,
  swimming, and water sports.
</p>

<p>
  Snorkeling enthusiasts can also discover the wonders of the Great Barrier
  Reef, a UNESCO World Heritage Site, which lies just off the coast. This
  vibrant underwater ecosystem teems with an incredible diversity of marine
  life, including colorful coral formations, tropical fish, and sea turtles.
</p>

<div id="two"></div>

<p>
  Lastly, the surrounding countryside is home to an impressive array of national
  parks and mountains. Barron Gorge National Park, for instance, showcases
  rugged terrain, deep gorges, and thundering waterfalls.
</p>

<p>
  You can embark on hiking trails that lead to breathtaking viewpoints, offering
  panoramic vistas of the surrounding rainforest-clad mountains and valleys.
  Further west, the towering peaks of the Great Dividing Range present
  opportunities for adventurous hiking and exploring scenic vistas, including
  the iconic Walshs Pyramid near Gordonvale.
</p>

<p>
  Lastly, the surrounding countryside is home to an impressive array of national
  parks and mountains. Barron Gorge National Park, for instance, showcases
  rugged terrain, deep gorges, and thundering waterfalls.
</p>

<p>
  You can embark on hiking trails that lead to breathtaking viewpoints, offering
  panoramic vistas of the surrounding rainforest-clad mountains and valleys.
  Further west, the towering peaks of the Great Dividing Range present
  opportunities for adventurous hiking and exploring scenic vistas, including
  the iconic Walshs Pyramid near Gordonvale.
</p>

<div id="three"></div>

<p>
  Walshs Pyramid is located within Wooroonooran National Park south of Cairns,
  Queensland, Australia. An annual footrace to its summit is held on the third
  Saturday in August. For experienced hikers, the ascent and descent can take 4
  to 6 hours. The vegetation on the mountain is fairly dense with exposed rocks
  which can make the surface very slippery after rain.
</p>
```

Initially, the highlighted content `<section>` is hidden off-screen. Our CSS starts by styling the `<section>` element, giving it a `position` of `fixed` and positioning it near the top-left of the scrollport. We also define the initial styles that we will be animating from and returning to. We then set three `animation` values, which will make the `<section>` element `slide-from-right`, then `reveal` its contents, then `slide-down` the screen and change its background color. We also set an `animation-trigger` for each one of the animations so that they will be triggered by different triggers being activated.

We want the finished state of each animation to apply throughout, after it is reached, therefore it is important to set appropriate `animation-fill-mode` values on the animations and `<animation-action>` values on the `animation-trigger` values to achieve this. We had to set an `animation-fill-mode` of `forwards` rather than `both` on the last animation because there is no `from` keyframe.

```
body {
  overflow-x: hidden;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
  width: 80%;
  margin: 0 auto;
}

h1 {
  font-size: 3rem;
}

h2 {
  font-size: 2rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}

section {
  background: red;
```

```
  color: #ffffff00;
  padding: 10px;
  width: 400px;
  height: 50px;
}

section p {
  font-size: 1.2rem;
}
```

```
section {
  position: fixed;
  left: 1em;
  top: 1em;
  height: 240px;
  background: red;
  width: 400px;
  transform-origin: top;

  animation:
    slide-from-right 1s both,
    reveal 1s both,
    slide-down 1s forwards;

  animation-trigger:
    --t1 play-forwards pause,
    --t2 play-forwards pause,
    --t3 play-forwards pause;
}
```

Next, we create triggers on the `<div>` elements, setting their `timeline-trigger-name` equal to the identifiers specified in the `<section>` elements' `animation-trigger` property values. This means that as the user scrolls, when each tracked `<div>` enters the scrollport, a different animation is activated. In this case, our tracked elements are invisible — they don't contain any useful content and are only here to create the triggers.

```
#one {
  timeline-trigger: --t1 view();
}

#two {
  timeline-trigger: --t2 view();
}

#three {
  timeline-trigger: --t3 view();
}
```

Finally, we define the animation `@keyframes` referenced in the `<section>` element's `animation` property earlier on.

```
@keyframes slide-from-right {
  from {
    translate: 400%;
  }
  to {
    translate: 0;
  }
}

@keyframes reveal {
  from {
    color: #ffffff00;
    transform: scaleY(0.2);
  }
  to {
    color: white;
    transform: scaleY(1);
  }
}

@keyframes slide-down {
  to {
    translate: 0 100%;
    background: blue;
  }
}
```

```
@supports not (timeline-trigger-name: --t) {
  body::before {
    content: "Your browser does not support scroll-triggered animations.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

This example renders as follows:

Try carefully scrolling the example, and note how each animation is applied to the `<section>` when each `<div>` is scrolled to.

Multiple triggers for the same animation

If you have an animated element, and you want to define triggers on multiple different elements that all trigger the same animation, you need to specify the same named animation multiple times on the animated element, and then give each instance of that animation a different trigger. See Multiple triggers for the same animation for more information.

See also

- CSS animation triggers module

- CSS animations module
- CSS scroll-driven animations module
- Using the Web Animations API
- CSS scroll-triggered animations are coming! on developer.chrome.com (2025)

CSS animations

The **CSS animations** module lets you animate the values of CSS properties, such as background-position and transform, over time by using keyframes. Each keyframe describes how the animated element should render at a given time during the animation sequence. You can use the properties in the animations module to control the duration, number of repetitions, delayed start, and other aspects of an animation.

Animations in action

To view the animation in the box below, click the checkbox 'Play the animation' or hover the cursor over the box. When the animation is active, the cloud at the top changes shape, snowflakes fall, and the snow level at the bottom rises. To pause the animation, uncheck the checkbox or move your cursor away from the box.

```
<!-- See aria-label: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Attributes/aria-label -->
<input
  type="checkbox"
  id="animate"
  aria-label="Toggle the play state of the animation" />
<label for="animate">the animation</label>
<div class="root">
  <i></i><i></i><i></i>
```

```
}
i:nth-of-type(5n + 4) {
  animation-timing-function: linear;
}
i:nth-of-type(11n) {
  animation-timing-function: cubic-bezier(0.2, 0.3, 0.8, 0.9);
}
i:nth-of-type(7n) {
  opacity: 0.5;
}
i:nth-of-type(7n + 2) {
  opacity: 0.3;
}
i:nth-of-type(7n + 4) {
  opacity: 0.7;
}
i:nth-of-type(7n + 6) {
  opacity: 0.6;
  animation-timing-function: ease-in;
  transform-origin: left 10px;
}
i:nth-of-type(7n + 1) {
  opacity: 0.8;
}

.root {
  height: 580px;
  background-color: skyblue;
  border: 1px solid darkgrey;
  position: relative;
  overflow: hidden;
}
.ground,
.cloud {
  position: absolute;
  top: 0;
  right: 0;
  left: 0;
  background-repeat: no-repeat;
}
.cloud {
  width: 100%;
  height: 150px;
  background: white;
  border-radius: 0 0 90px 33% / 0 0 45px 50px;
  box-shadow:
    5px 15px 15px white,
    -5px 15px 15px white,
    0 20px 20px rgb(125 125 125 / 0.5);
  animation:
    clouds ease 5s alternate infinite 0.2s,
    wind ease-out 4s alternate infinite;
}
.ground {
  bottom: 0;
  background-image: linear-gradient(to top, white 97%, 99%, #bbbbbb 100%);
  background-position: center 580px;
  animation: snowfall linear 300s forwards;
  border: 1px solid grey;
  /* Put the ground into a 3D rendering context (because the snow flakes are in a 3d rendering context) */
  transform: translate3d(0, 0, 0);
}

@keyframes snowfall {
  from {
    background-position: center 580px;
  }
  to {
    background-position: center 280px;
  }
}

@keyframes clouds {
  from {
    border-radius: 0 0 90px 33% / 0 0 45px 50px;
  }
  to {
    border-radius: 0 0 40px 50% / 0 0 55px 80px;
  }
}

@keyframes wind {
  from {
    height: 150px;
  }
  to {
    height: 100px;
  }
}

@keyframes falling {
  from {
    transform: translate(0, -50px) rotate(0deg) scale(0.9, 0.9);
  }
  to {
    transform: translate(30px, 600px) rotate(360deg) scale(1.1, 1.1);
  }
}

/* By default, the animations are paused. */
i,
div[class] {
  animation-play-state: paused;
```

```
}
/* When the div is hovered, the animation plays. Also,
when the input is checked, the animation coming after the checked checkbox plays */
div:hover *,
input:checked ~ div * {
  animation-play-state: running;
}

/* Change the content of the label that comes right after the input. Included aria-label on the label to improve accessibility. */
input + label::before {
  content: "Play ";
}
input:checked + label::before {
  content: "Pause ";
}
```

This sample animation uses `animation-iteration-count` to make the flakes fall repeatedly, `animation-direction` to make the cloud move back and forth, `animation-fill-mode` to raise the snow level in response to the cloud movement, and `animation-play-state` to pause the animation.

Click "Play" in the example above to see or edit the code for the animation in the MDN Playground.

Reference

Properties

- `animation` shorthand
- `animation-composition`
- `animation-delay`
- `animation-direction`
- `animation-duration`
- `animation-fill-mode`
- `animation-iteration-count`
- `animation-name`
- `animation-play-state`
- `animation-timeline`
- `animation-timing-function`

At-rules and descriptors

- `@keyframes`
- `<keyframe-selector>`

Events

All animations, even those with 0 seconds duration, throw animation events.

- `animationstart`
- `animationend`
- `animationcancel`
- `animationiteration`

Interfaces

- Web Animations API
- `AnimationEvent`
- `CSSAnimation`
- `CSSKeyframeRule`
- `CSSKeyframesRule`

Guides

Using CSS animations

Step-by-step tutorial on how to create animations using CSS. This article describes the animation-related CSS properties and at-rule and how they interact with each other.

Animatable CSS properties

Overview of how different CSS properties can be animated, including their animation types and interpolation methods.

Using the Web Animations API

Common animation requirements that can be solved with a few lines of JavaScript.

Related concepts

- CSS animation triggers module
  - `animation-trigger`
  - `timeline-trigger` shorthand
  - `timeline-trigger-activation-range` shorthand
    - `timeline-trigger-activation-range-end`
    - `timeline-trigger-activation-range-start`
  - `timeline-trigger-active-range` shorthand
    - `timeline-trigger-active-range-end`
    - `timeline-trigger-active-range-start`
  - `timeline-trigger-name`
  - `timeline-trigger-source`
  - `trigger-scope`
- CSS scroll-driven animations module
  - `animation-range` shorthand
    - `animation-range-end`
    - `animation-range-start`
  - `scroll-timeline` shorthand
    - `scroll-timeline-axis`
    - `scroll-timeline-name`
  - `timeline-scope`

    - `view-timeline` shorthand
        - `view-timeline-axis`
        - `view-timeline-inset`
        - `view-timeline-name`
- `will-change` CSS property
- `<easing-function>` data type
- `prefers-reduced-motion` media query
- Bezier curve glossary term

See also

- CSS scroll-driven animations module
- CSS animation triggers module
- CSS transitions module
- The `interpolate-size` property and `calc-size()` function for enabling animations to and from intrinsic size values.
- The `<canvas>` HTML element along with canvas API and WebGL API to draw graphics and animations.
- The `SVGAnimationElement` interface for all the animation-related element interfaces, including `SVGAnimateElement`, `SVGSetElement`, `SVGAnimateColorElement`, `SVGAnimateMotionElement`, and `SVGAnimateTransformElement`.

Animatable CSS properties

CSS Animations and Transitions rely on the concept of **animatable** properties, and all CSS properties are animatable unless otherwise specified. Each property's *animation type* determines how values combine - interpolate, add, or accumulate - for this property. Transitions only involve interpolation, whereas animations may use all three combination methods.

> **Note:** The animation type for each CSS property is listed in its "Formal definition" table (E.g., `color`).

> **Note:** The interpolation method for each CSS data type is described in its "Interpolation" section (E.g., `<length>`).

Animation types

There are mainly four animation types as defined in the Web Animations specification:

Not animatable

: The property is not animatable. It is not processed when listed in an animation keyframe and is unaffected by transitions.

> **Note:** An animation effect targeting only properties that are not animatable will still exhibit the usual behavior for an animation effect (E.g., firing the `animationstart` event).

Discrete

: The property's values are not additive, and interpolation swaps from the start value to the end value at `50%`. Specifically, denoting by `p` the progress value:

- If `p < 0.5`, then `V_result = V_start`;
- If `p ≥ 0.5`, then `V_result = V_end`.

By computed value

: Corresponding individual components of the computed values are combined using the indicated procedure for that value type. If the number of components or the types of corresponding components do not match, or if any component value uses discrete animation and the two corresponding values do not match, then the property values combine as discrete.

Repeatable list

: Same as by computed value except that if the two lists have differing numbers of items, they are first repeated to the least common multiple numbers of items. Each item is then combined by computed value. If a pair of values cannot be combined or any component value uses discrete animation, then the property values combine as discrete.

Some properties have specific interpolation behavior not covered by these four types. In this case, refer to the property's "Interpolation" section (E.g., `visibility`).

Animating custom properties

For custom properties registered using the `registerProperty()` method, the animation type is by computed value, with the computed value type determined by the property's syntax definition.

For unregistered custom properties, the animation type is discrete.

See also

- `transition-behavior`
- `@starting-style`
- CSS animations module
- Using CSS Animations
- Using CSS Transitions

Using CSS animations

CSS animations make it possible to animate transitions from one CSS style configuration to another. Animations consist of two components: a style describing the CSS animation and a set of keyframes that indicate the start and end states of the animation's style, as well as possible intermediate waypoints.

There are three key advantages to CSS animations over traditional script-driven animation techniques:

1. You can create basic animations with a few lines of CSS; no JavaScript required.
2. The animations run well, even under moderate system load. Simple animations can often perform poorly in JavaScript. The rendering engine can use frame-skipping and other techniques to keep the performance as smooth as possible.
3. Letting the browser control the animation sequence lets the browser optimize performance and efficiency by, for example, reducing the update frequency of animations running in tabs that aren't currently visible.

Configuring an animation

To create a CSS animation sequence, you style the element you want to animate with the `animation` property or its sub-properties. This lets you configure the timing, duration, and other details of how the animation sequence should progress. This does **not** configure the actual appearance of the animation, which is done using the `@keyframes` at-rule as described in the Defining animation sequence using keyframes section below.

The sub-properties of the `animation` property are:

`animation-composition`

Specifies the composite operation to use when multiple animations affect the same property simultaneously. This property is not part of the `animation` shorthand property.

`animation-delay`

Specifies the delay between an element loading and the start of an animation sequence and whether the animation should start immediately from its beginning or partway through the animation.

`animation-direction`

Specifies whether an animation's first iteration should be forward or backward and whether subsequent iterations should alternate direction on each run through the sequence or reset to the start point and repeat.

`animation-duration`

Specifies the length of time in which an animation completes one cycle.

`animation-fill-mode`

Specifies how an animation applies styles to its target before and after it runs.

> **Note:** In the case of animation forwards fill mode, animated properties behave as if included in a set `will-change` property value. If a new stacking context was created during the animation, the target element retains the stacking context after the animation has finished.

`animation-iteration-count`

Specifies the number of times an animation should repeat.

`animation-name`

Specifies the name of the `@keyframes` at-rule describing an animation's keyframes.

`animation-play-state`

Specifies whether to pause or play an animation sequence.

`animation-timeline`

Specifies the timeline that is used to control the progress of a CSS animation.

`animation-timing-function`

Specifies how an animation transitions through keyframes by establishing acceleration curves.

Defining an animation sequence using keyframes

After you've configured the animation's timing, you need to define the appearance of the animation. This is done by establishing one or more keyframes using the `@keyframes` at-rule. Each keyframe describes how the animated element should render at a given time during the animation sequence.

Since the timing of the animation is defined in the CSS style that configures the animation, keyframes use a `<percentage>` to indicate the time during the animation sequence at which they take place. 0% indicates the first moment of the animation sequence, while 100% indicates the final state of the animation. Because these two times are so important, they have special aliases: `from` and `to`. Both are optional. If `from`/`0%` or `to`/`100%` is not specified, the browser starts or finishes the animation using the computed values of all attributes.

You can optionally include additional keyframes that describe intermediate steps between the start and end of the animation.

Using the animation shorthand

The `animation` shorthand is useful for saving space. As an example, some of the rules we've been using through this article:

```
p {
  animation-duration: 3s;
  animation-name: slide-in;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}
```

...could be replaced by using the `animation` shorthand.

```
p {
  animation: 3s infinite alternate slide-in;
}
```

To learn more about the sequence in which different animation property values can be specified using the `animation` shorthand, see the `animation` reference page.

Setting multiple animation property values

The CSS animation longhand properties can accept multiple values, separated by commas. This feature can be used when you want to apply multiple animations in a single rule and set different durations, iteration counts, etc., for each of the animations. Let's look at some quick examples to explain the different permutations.

In this first example, there are three duration and three iteration count values. So each animation is assigned a value of duration and iteration count with the same position as the animation name. The `fadeInOut` animation is assigned a duration of `2.5s` and an iteration count of `2`, and the `bounce` animation is assigned a duration of `1s` and an iteration count of `5`.

```
animation-name: fadeInOut, moveLeft300px, bounce;
animation-duration: 2.5s, 5s, 1s;
animation-iteration-count: 2, 1, 5;
```

In this second example, three animation names are set, but there's only one duration and iteration count. In this case, all three animations are given the same duration and iteration count.

```
animation-name: fadeInOut, moveLeft300px, bounce;
animation-duration: 3s;
animation-iteration-count: 1;
```

In this third example, three animations are specified, but only two durations and iteration counts. In such cases where there are not enough values in the list to assign a separate one to each animation, the value assignment cycles from the first to the last item in the available list and then cycles back to the first item. So, `fadeInOut` gets a duration of `2.5s`, and `moveLeft300px` gets a duration of `5s`, which is the last value in the list of duration values. The duration value assignment now resets to the first value; `bounce`, therefore, gets a duration of `2.5s`. The iteration count values (and any other property values you specify) will be assigned in the same way.

```
animation-name: fadeInOut, moveLeft300px, bounce;
animation-duration: 2.5s, 5s;
animation-iteration-count: 2, 1;
```

If the mismatch in the number of animations and animation property values is inverted, say there are five `animation-duration` values for three `animation-name` values, then the extra or unused animation property values, in this case, two `animation-duration` values, don't apply to any animation and are ignored.

Examples

Making text slide across the browser window

This basic example styles a `<p>` element using the `translate` and `scale` transition properties so that the text slides in from off the right edge of the browser window.

```
p {
  animation-duration: 3s;
  animation-name: slide-in;
}

@keyframes slide-in {
  from {
    translate: 150vw 0;
    scale: 200% 1;
  }

  to {
    translate: 0 0;
    scale: 100% 1;
  }
}
```

In this example, the style for the `<p>` element specifies that the animation should take 3 seconds to execute from start to finish, using the `animation-duration` property and that the name of the `@keyframes` at-rule defining the keyframes for the animation sequence is `slide-in`.

In this case, we have just two keyframes. The first occurs at `0%` (using the alias `from`). Here, we configure the `translate` property of the element to be at `150vw` (that is, beyond the far right edge of the containing element), and the `scale` of the element to be 200% (or two times its default inline size), causing the paragraph to be twice as wide as its `<body>` containing block. This causes the first frame of the animation to have the header drawn off the right edge of the browser window.

The second keyframe occurs at `100%` (using the alias `to`). The `translate` property is set to `0%` and the `scale` of the element is set to `1`, which is `100%`. This causes the header to finish its animation in its default state, flush against the left edge of the content area.

```
<p>
  The Caterpillar and Alice looked at each other for some time in silence: at
  last the Caterpillar took the hookah out of its mouth, and addressed her in a
  languid, sleepy voice.
</p>
```

> **Note:** Reload page to see the animation.

Adding another keyframe animation

Let's add another keyframe to the previous example's animation. Let's say we want Alice's name to turn pink and grow and then shrink back to its original size and color as it moves from right to left. While we could change the `font-size`, changing any properties that impact the box model negatively impacts performance. Instead, we wrap her name in a `<span>` and then scale and assign a color to that separately. That requires adding a second animation impacting only the `<span>`:

```
@keyframes grow-shrink {
  25%,
  75% {
    scale: 100%;
  }

  50% {
    scale: 200%;
    color: magenta;
  }
}
```

The full code now looks like this:

```
p {
  animation-duration: 3s;
  animation-name: slide-in;
}
p span {
  display: inline-block;
  animation-duration: 3s;
  animation-name: grow-shrink;
}

@keyframes slide-in {
  from {
    translate: 150vw 0;
    scale: 200% 1;
  }

  to {
    translate: 0 0;
    scale: 100% 1;
  }
}

@keyframes grow-shrink {
  25%,
  75% {
    scale: 100%;
  }

  50% {
```

```
    scale: 200%;
    color: magenta;
  }
}
```

We've added a `<span>` around "Alice":

```
<p>
  The Caterpillar and <span>Alice</span> looked at each other for some time in
  silence: at last the Caterpillar took the hookah out of its mouth, and
  addressed her in a languid, sleepy voice.
</p>
```

This tells the browser the name should be normal for the first and last 25% of the animation, but turn pink while being scaled up and back again in the middle. We set the spans's `display` property to `inline-block` as the `transform` properties do not affect non-replaced inline-level content.

> **Note:** Reload page to see the animation.

Repeating the animation

To make the animation repeat itself, use the `animation-iteration-count` property to indicate how many times to repeat the animation. In this case, let's use `infinite` to have the animation repeat indefinitely:

```
p {
  animation-duration: 3s;
  animation-name: slide-in;
  animation-iteration-count: infinite;
}
```

```
@keyframes slide-in {
  from {
    translate: 150vw 0;
    scale: 200% 1;
  }

  to {
    translate: 0 0;
    scale: 100% 1;
  }
}
```

```
<p>
  The Caterpillar and Alice looked at each other for some time in silence: at
  last the Caterpillar took the hookah out of its mouth, and addressed her in a
  languid, sleepy voice.
</p>
```

Making the animation move back and forth

That made it repeat, but it's very odd having it jump back to the start each time it begins animating. What we really want is for it to move back and forth across the screen. That's easily accomplished by setting `animation-direction` to `alternate`:

```
p {
  animation-duration: 3s;
  animation-name: slide-in;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}
```

```
@keyframes slide-in {
  from {
    translate: 150vw 0;
    scale: 200% 1;
  }

  to {
    translate: 0 0;
    scale: 100% 1;
  }
}
```

```
<p>
  The Caterpillar and Alice looked at each other for some time in silence: at
  last the Caterpillar took the hookah out of its mouth, and addressed her in a
  languid, sleepy voice.
</p>
```

Using animation events

You can get additional control over animations — as well as useful information about them — by making use of animation events. These events, represented by the `AnimationEvent` object, can be used to detect when animations start, finish, and begin a new iteration. Each event includes the time at which it occurred as well as the name of the animation that triggered the event.

We'll modify the sliding text example to output some information about each animation event when it occurs, so we can get a look at how they work.

We've included the same keyframe animation as the previous example. This animation will last 3 seconds, be called "slide-in", repeat 3 times, and travel in an alternate direction each time. In the `@keyframes`, the scale and translation are manipulated along the x-axis to make the element slide across the screen.

```
.slide-in {
  animation-duration: 3s;
  animation-name: slide-in;
  animation-iteration-count: 3;
  animation-direction: alternate;
}
```

```
@keyframes slide-in {
  from {
```

```
    translate: 150vw 0;
    scale: 200% 1;
  }

  to {
    translate: 0 0;
    scale: 100% 1;
  }
}
```

Adding the animation event listeners

We'll use JavaScript code to listen for all three possible animation events. This code configures our event listeners; we call it when the document is first loaded in order to set things up.

```
const element = document.getElementById("watch-me");
element.addEventListener("animationstart", listener);
element.addEventListener("animationend", listener);
element.addEventListener("animationiteration", listener);

element.className = "slide-in";
```

This is pretty standard code; you can get details on how it works in the documentation for `eventTarget.addEventListener()`. The last thing this code does is set the `class` on the element we'll be animating to "slide-in"; we do this to start the animation.

Why? Because the `animationstart` event fires as soon as the animation starts, and in our case, that happens before our code runs. So we'll start the animation ourselves by setting the class of the element to the style that gets animated after the fact.

Receiving the events

The events get delivered to the `listener()` function, which is shown below.

```
function listener(event) {
  const l = document.createElement("li");
  switch (event.type) {
    case "animationstart":
      l.textContent = `Started: elapsed time is ${event.elapsedTime}`;
      break;
    case "animationend":
      l.textContent = `Ended: elapsed time is ${event.elapsedTime}`;
      break;
    case "animationiteration":
      l.textContent = `New loop started at time ${event.elapsedTime}`;
      break;
  }
  document.getElementById("output").appendChild(l);
}
```

This code, too, is very simple. It looks at the `event.type` to determine which kind of animation event occurred, then adds an appropriate note to the `<ul>` (unordered list) we're using to log these events.

The output, when all is said and done, looks something like this:

- Started: elapsed time is 0
- New loop started at time 3.01200008392334
- New loop started at time 6.00600004196167
- Ended: elapsed time is 9.234000205993652

Note that the times are very close to, but not exactly, those expected given the timing established when the animation was configured. Note also that after the final iteration of the animation, the `animationiteration` event isn't sent; instead, the `animationend` event is sent.

Just for the sake of completeness, here's the HTML that displays the page content, including the list into which the script inserts information about the received events:

```
<h1 id="watch-me">Watch me move</h1>
<p>
  This example shows how to use CSS animations to make <code>H1</code>
  elements move across the page.
</p>
<p>
  In addition, we output some text each time an animation event fires, so you
  can see them in action.
</p>
<ul id="output"></ul>
```

And here's the live output.

> **Note:** Reload page to see the animation.

Animating display and content-visibility

This example demonstrates how `display` and `content-visibility` can be animated. This behavior is useful for creating entry/exit animations where you want to for example remove a container from the DOM with `display: none`, but have it fade out smoothly with `opacity` rather than disappearing immediately.

Supporting browsers animate `display` and `content-visibility` with a variation on the discrete animation type. This generally means that properties will flip between two values 50% of the way through animating between the two.

There is an exception, however, which is when animating to/from `display: none` or `content-visibility: hidden` to a visible value. In this case, the browser will flip between the two values so that the animated content is shown for the entire animation duration.

So for example:

- When animating `display` from `none` to `block` (or another visible `display` value), the value will flip to `block` at `0%` of the animation duration so it is visible throughout.
- When animating `display` from `block` (or another visible `display` value) to `none`, the value will flip to `none` at `100%` of the animation duration so it is visible throughout.

HTML

The HTML contains two `<p>` elements with a `<div>` in between that we will animate from `display` `none` to `block`.

```
<p>
  Click anywhere on the screen or press any key to toggle the
  <code>&lt;div&gt;</code> between hidden and showing.
</p>

<div>
  This is a <code>&lt;div&gt;</code> element that animates between
  <code>display: none; opacity: 0</code> and
  <code>display: block; opacity: 1</code>. Neat, huh?
</div>

<p>
  This is another paragraph to show that <code>display: none;</code> is being
  applied and removed on the above <code>&lt;div&gt; </code>. If only its
  <code>opacity</code> was being changed, it would always take up the space in
  the DOM.
</p>
```

CSS

```
html {
  height: 100vh;
}

div {
  font-size: 1.6rem;
  padding: 20px;
  border: 3px solid red;
  border-radius: 20px;
  width: 480px;
  opacity: 0;
  display: none;
}

/* Animation classes */

div.fade-in {
  display: block;
  animation: fade-in 0.7s ease-in forwards;
}

div.fade-out {
  animation: fade-out 0.7s ease-out forwards;
}

/* Animation keyframes */

@keyframes fade-in {
  0% {
    opacity: 0;
    display: none;
  }

  100% {
    opacity: 1;
    display: block;
  }
}

@keyframes fade-out {
  0% {
    opacity: 1;
    display: block;
  }

  100% {
    opacity: 0;
    display: none;
  }
}
```

Note the inclusion of the `display` property in the keyframe animations.

JavaScript

Finally, we include a bit of JavaScript to set up event listeners to trigger the animations. Specifically, we add the `fade-in` class to the `<div>` when we want it to appear, and `fade-out` when we want it to disappear.

```
const divElem = document.querySelector("div");
const htmlElem = document.querySelector(":root");

htmlElem.addEventListener("click", showHide);
document.addEventListener("keydown", showHide);

function showHide() {
  if (divElem.classList[0] === "fade-in") {
    divElem.classList.remove("fade-in");
    divElem.classList.add("fade-out");
  } else {
    divElem.classList.remove("fade-out");
    divElem.classList.add("fade-in");
  }
}
```

Result

The code renders as follows:

See also

- CSS animations module
- `AnimationEvent`

- Using CSS transitions
- Using the Web Animations API

CSS backgrounds and borders

The **CSS backgrounds and borders** module provides properties for adding backgrounds, borders, rounded corners, and box shadows to elements.

You can add different types of border styles, including borders made of images of any image type, from raster images to CSS gradients. Borders can be square or rounded, and a different radius can be set for each corner. Elements can be rounded whether or not they have a visible border.

Box shadows include inset and outset shadow, single or multiple shadows, and solid or allowed to fade to transparent. An outer box-shadow casts a shadow as if the border-box of the element were opaque. An inner box-shadow casts a shadow as if everything outside the padding edge were opaque. The shadow can be solid or include a spread distance with the shadow color transitioning to transparent.

The properties in this module also let you define whether cells inside a `<table>` should have shared or separate borders.

Backgrounds, borders, and box shadows in action

This sample of borders, backgrounds, and box shadows consists of centered background images made of linear and radial gradients. A series of box shadows make the border appear to "pop". The element on the left has a border image set. The element on the right has a rounded dotted border.

```
<article>
  <div></div>
  <div></div>
</article>
```

```
article {
  display: flex;
  gap: 10px;
}
div {
  color: #58ade3;
  height: 320px;
  width: 240px;
  padding: 20px;
  margin: 10px;
  border: dotted 15px; /* defaults to `currentColor` */
  border-radius: 100px 0;
  background-image:
    radial-gradient(
      circle,
      transparent 60%,
      currentColor 60% 70%,
      transparent 70%
    ),
    linear-gradient(45deg, currentColor, white),
    linear-gradient(transparent, transparent);
  /* the third transparent background image was added to provide space for the background color to show through */
  background-color: currentColor;
  background-position: center;
  background-size:
    60px 60px,
    120px 120px;
  background-clip: content-box, content-box, padding-box;
  box-shadow:
    inset 5px 5px 5px rgb(0 0 0 / 0.4),
    inset -5px -5px 5px rgb(0 0 0 / 0.4),
    5px 5px 5px rgb(0 0 0 / 0.4),
    -5px -5px 5px rgb(0 0 0 / 0.4);
}
div:first-of-type {
  border-radius: 0;
  border-image-source: repeating-conic-gradient(
    from 3deg at 25% 25%,
    currentColor 0 3deg,
    transparent 3deg 6deg
  );
  border-image-slice: 30;
}
```

The background images are defined with `background-image`. The images are centered with `background-position`. Different values of the `background-clip` property for the multiple background images are used to make the background images stay within the content box. The background color gets clipped to the padding box preventing the background from appearing through the transparent sections for the `border-image` and the `dotted border`. The rounded corners in the element on the right are created using the `border-radius` property. A single `box-shadow` declaration is used to set all the shadows, both inset and outset.

Reference

Properties

- `background-attachment`
- `background-clip`
- `background-color`
- `background-image`
- `background-origin`
- `background-position`
- `background-repeat-x`
- `background-repeat-y`
- `background-repeat` shorthand
- `background-size`
- `background` shorthand
- `background-position-x`
- `background-position-y`
- `border-bottom-color`
- `border-bottom-style`
- `border-bottom-width`
- `border-bottom` shorthand
- `border-left-color`
- `border-left-style`

- `border-left-width`
- `border-left` shorthand
- `border-right-color`
- `border-right-style`
- `border-right-width`
- `border-right` shorthand
- `border-top-color`
- `border-top-style`
- `border-top-width`
- `border-top` shorthand
- `border-color` shorthand
- `border-style` shorthand
- `border-width` shorthand
- `border` shorthand
- `border-bottom-left-radius`
- `border-bottom-right-radius`
- `border-top-left-radius`
- `border-top-right-radius`
- `border-radius` shorthand
- `border-image-outset`
- `border-image-repeat`
- `border-image-slice`
- `border-image-source`
- `border-image-width`
- `border-image` shorthand
- `box-shadow`

The CSS backgrounds module level 4 also introduces the `background-position-block`, `background-position-inline`, `background-repeat-block`, `background-repeat-inline`, and `background-tbd` properties. Currently, no browsers support these features.

Data types

- `<line-style>` enumerated type

Guides

Using multiple backgrounds

Setting one or more backgrounds on an element.

Resizing background images

Changing the size and repeating behavior of background images.

Scaling SVG backgrounds

How SVG aspect ratio, SVG dimension values, and the CSS `background-size` property impact the scaling of SVG background images.

Using CSS gradients

Creating CSS gradients and using them as background images.

Learn CSS: background and borders

Learn how to implement decorative images using CSS background images.

Learn CSS: the box model

Learn how borders and other box model properties impact the CSS box model.

Related concepts

- `border-block-end-color`
- `border-block-start-color`
- `border-inline-end-color`
- `border-inline-start-color`
- `border-block-end-style`
- `border-block-start-style`
- `border-inline-end-style`
- `border-inline-start-style`
- `border-block-end-width`
- `border-block-start-width`
- `border-inline-end-width`
- `border-inline-start-width`
- `border-start-start-radius`
- `border-start-end-radius`
- `border-end-start-radius`
- `border-end-end-radius`
- `box-sizing`
- `box-decoration-break`
- `text-shadow`
- `<url>` data type
- `<url>` data type
- `<image>` data type
- `position` data type
- `currentColor` keyword

See also

- `filter`
- `backdrop-filter`
- `drop-shadow()` filter function
- Applying color to HTML elements using CSS

- Tools:
  - Border-image generator
  - Border-radius generator
  - Box-shadow generator

Border-image generator

This tool can be used to generate CSS `border-image` values.

See also

- Other tools:
  - Border-radius generator
  - Box-shadow generator
- CSS backgrounds and borders module

Border-radius generator

This tool can be used to generate CSS `border-radius` effects.

See also

- Other tools:
  - Border-image generator
  - Box-shadow generator
- CSS backgrounds and borders module

Box-shadow generator

This tool lets you construct CSS `box-shadow` effects, to add box shadow effects to your CSS objects.

The box-shadow generator enables you to add one or more box shadows to an element.

On opening the tool, you'll find a rectangle in the top-right section of the tool. That's the element you're going to be applying shadows to. When this element is selected (as it is, when you first load the page) you can apply some basic styling to it:

- Set the element's `color` using the color picker tool.
- Give the element a `border` using the "border" checkbox.
- Use the sliders to set the element's `top`, `left`, `width`, and `height` properties.

To add a box shadow, click the "+" button at the top-left. This adds a shadow, and lists it in the column on the left. Now you can set the values of the new shadow:

- Set the shadow's `color` using the color picker tool.
- Set the shadow to be inset using the "inset" checkbox.
- Use the sliders to set the element's position, blur, and spread.

To add another shadow, click "+" again. Now any values you set will apply to this new shadow. Change the order in which these two shadows are applied using the ↑ and ↓ buttons at the top-left. Select the first shadow again by clicking it in column on the left. To update the element's own styles, select it by clicking the button labeled "element" along the top.

You can add `::before` and `::after` pseudo-elements to the element, and give them box shadows as well. To switch between the element and its pseudo-elements, use the buttons along the top labeled "element", "::before", and "::after".

The box at the bottom-right contains the CSS for the element and any `::before` or `::after` pseudo-elements.

See also

- Other tools:
  - Border-image generator
  - Border-radius generator
- CSS backgrounds and borders module

Resizing background images with background-size

The **`background-size`** CSS property lets you resize the background image of an element, overriding the default behavior of tiling the image at its full size by specifying the width and/or height of the image. By doing so, you can scale the image upward or downward as desired.

Tiling a large image

Let's consider a large image, a 2982x2808 Firefox logo image. We want (for some reason likely involving horrifyingly bad site design) to tile four copies of this image into a 300x300-pixel element. To do this, we can use a fixed `background-size` value of 150 pixels.

HTML

```
<div class="tiledBackground"></div>
```

CSS

```
.tiledBackground {
  background-image: url("https://www.mozilla.org/media/img/logos/firefox/logo-quantum.9c5e96634f92.png");
  background-size: 150px;
  width: 300px;
  height: 300px;
  border: 2px solid;
  color: pink;
}
```

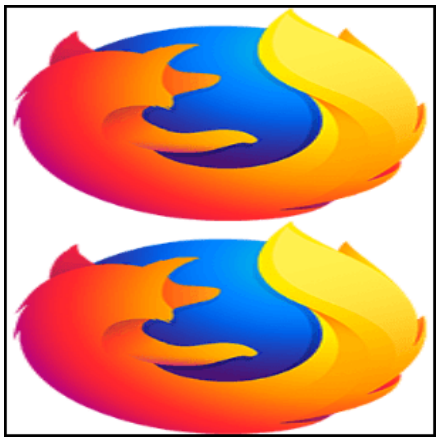
Result

Stretching an image

You can also specify both the horizontal and vertical sizes of the image, like this:

```
background-size: 300px 150px;
```

The result looks like this:



Scaling an image up

On the other end of the spectrum, you can scale an image up in the background. Here we scale a 32x32 pixel favicon to 300x300 pixels:

```
.square2 {
  background-image: url("favicon.png");
  background-size: 300px;
  width: 300px;
  height: 300px;
  border: 2px solid;
  text-shadow: white 0px 0px 2px;
  font-size: 16px;
}
```

As you can see, the CSS is actually essentially identical, save the name of the image file.

Special values: <code>contain</code> and <code>cover</code>

In addition to `<length>` values, the `background-size` CSS property offers two special size values, `contain` and `cover`. Let's take a look at these.

<code>contain</code>

The `contain` value specifies that, regardless of the size of the containing box, the background image should be scaled so that each side is as large as possible while not exceeding the length of the corresponding side of the container. Try resizing the example below to see this in action.

HTML

```
<div class="bgSizeContain">
  <p>Try resizing this element!</p>
</div>
```

CSS

```
.bgSizeContain {
  background-image: url("https://www.mozilla.org/media/img/logos/firefox/logo-quantum.9c5e96634f92.png");
  background-size: contain;
  width: 160px;
  height: 160px;
  border: 2px solid;
  resize: both;
  overflow: scroll;
}
```

Result

<code>cover</code>

The `cover` value specifies that the background image should be sized so that it is as small as possible while ensuring that both dimensions are greater than or equal to the corresponding size of the container. Try resizing the example below to see this in action.

HTML

```
<div class="bgSizeCover">
  <p>Try resizing this element!</p>
</div>
```

CSS

```
.bgSizeCover {
  background-image: url("https://www.mozilla.org/media/img/logos/firefox/logo-quantum.9c5e96634f92.png");
  background-size: cover;
  width: 160px;
  height: 160px;
  border: 2px solid;
  resize: both;
  overflow: scroll;
}
```

Result

See also

- `background-size`
- `background`
- CSS backgrounds and borders module
- Scaling SVG backgrounds

Scaling SVG backgrounds

Given the flexibility of SVG images, there's a lot to keep in mind when using them as background images with the `background-image` property, and even more to keep in mind when scaling them using the `background-size` property. This article describes how scaling of SVG images is handled when using these properties.

The background sizing algorithm

The algorithm used to determine the background size of a background image can, for the most part, be summarized by these four rules. There are some edge cases that aren't covered by these rules, but this covers the majority of cases.

1. If `background-size` specifies a fixed dimension (that is, percentages and relative units are fixed by their context), that dimension wins.
2. If the image has an intrinsic ratio (that is, its width:height ratio is constant, such as 16:9, 4:3, 2.39:1, 1:1, and so forth), the rendered size preserves that ratio.
3. If the image specifies a size, and the size isn't modified by `constrain` or `cover`, that specified size wins.
4. If none of the above cases are met, the image is rendered at the same size as the background area.

It's worth noting that the background sizing algorithm only cares about the image's dimensions and proportions, or lack thereof. An SVG image with fixed dimensions will be treated just like a raster image of the same size.

> **Note:** If you are trying to stretch your SVG to a different aspect ratio with CSS—for example in order to stretch it over the page background—make sure your SVG includes `preserveAspectRatio="none"`. Find out more about `preserveAspectRatio`.

Source image examples

Before diving in to look at the results of using different kinds of SVG source images and seeing how they look when used with `background-size`, it will be helpful to look at a few example source images that have different dimensions and sizing settings, which we will later use as our `background-image` values in our examples. The browser renders `<svg>` images as `300px` wide and `150px` tall by default.

Dimensionless and proportionless

This SVG gradient image is both dimensionless and proportionless. It doesn't care what size it is, nor does it care about remaining at a particular aspect ratio. This would make a good gradient desktop background that would work regardless of your screen size and its aspect ratio.

```
<svg>
  <title>Corner-to-corner gradient</title>
  <defs>
    <linearGradient id="g" x1="0%" x2="100%" y1="0%" y2="100%">
      <stop stop-color="pink" offset="0" />
      <stop stop-color="goldenrod" offset="1" />
    </linearGradient>
  </defs>
  <rect fill="url('#g')" width="100%" height="100%" />
</svg>
```

One specified dimension and proportionless

This image specifies a width of 100 pixels but no height or intrinsic ratio. This is, basically, a thin strip of wallpaper that could be stretched across the entire height of a block.

```
<svg width="100">
  <title>Vertical gradient, with a fixed width</title>
  <defs>
    <linearGradient id="g" x1="0%" x2="0%" y1="0%" y2="100%">
      <stop stop-color="purple" offset="0" />
      <stop stop-color="lime" offset="1" />
    </linearGradient>
  </defs>
  <rect fill="url('#g')" width="100%" height="100%" />
</svg>
```

One specified dimension with intrinsic ratio

This image specifies a 100 pixel height but no width. It also specifies an intrinsic aspect ratio of 3:4. This ensures that its width:height ratio is always 3:4, unless it's deliberately scaled to a disproportionate size (that is, by explicitly specifying both width and height that aren't of that ratio).

This is very much like specifying a specific width and height; since once you have one dimension and a ratio, the other dimension is implied.

```
<svg height="100" viewBox="0 0 3 4" preserveAspectRatio="none">
  <title>Vertical gradient, with a fixed height and intrinsic ratio</title>
  <defs>
    <linearGradient id="g" x1="0%" x2="0%" y1="0%" y2="100%">
      <stop stop-color="teal" offset="0" />
      <stop stop-color="orange" offset="1" />
    </linearGradient>
  </defs>
  <rect fill="url('#g')" width="100%" height="100%" />
</svg>
```

No width or height with intrinsic ratio

This image doesn't specify either a width or a height; instead, it specifies an intrinsic ratio of 1:1. It's always square, and is usable at any size, such as 32x32, 128x128, or 512x512, for example.

```
<svg viewBox="0 0 1 1" preserveAspectRatio="none">
  <title>Intrinsic ratio</title>
  <defs>
    <linearGradient id="g" x1="0%" x2="100%" y1="0%" y2="0%">
      <stop stop-color="navy" offset="0" />
      <stop stop-color="maroon" offset="1" />
    </linearGradient>
  </defs>
  <rect fill="url('#g')" width="100%" height="100%" />
</svg>
```

Scaling examples

Now let's see some examples of what happens as we apply different scaling to these images. In each of the examples below, the enclosing `<div>` elements are 300 pixels wide and 200 pixels tall, with a 2 pixel-wide border. To ensure we display the SVG background image only once for these demonstrations, we set `background-repeat` to `no-repeat`.

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

Specifying fixed lengths for both dimensions

If you use `background-size` to specify fixed lengths for both dimensions, those lengths are always used, per rule 1 above. In other words, the image will always get stretched to the dimensions you specify, regardless of whether or not the source image has specified its dimensions and/or aspect ratio.

No dimensions or intrinsic ratio

In this example, the image has no dimensions or intrinsic ratio set:

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-or-ratio.svg");
  background-size: 125px 175px;
}
```

One specified dimension, no intrinsic ratio

In this example, the image has one dimension specified, and no intrinsic ratio set:

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-wide-no-height-or-ratio.svg");
  background-size: 250px 150px;
}
```

One specified dimension with intrinsic ratio

```
<div></div>
```

In this example, the image has one dimension explicitly set, along with an intrinsic ratio, meaning both dimensions are effectively defined. Setting an absolute height and width for `background-size` overrides the dimensions set in the SVG:

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-height-3x4-ratio.svg");
  background-size: 275px 125px;
}
```

No specified width or height with intrinsic ratio

In this example, the image has an intrinsic ratio but no dimensions set:

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
```

```
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-1x1-ratio.svg");
  background-size: 250px 100px;
}
```

Using contain or cover

Specifying `cover` for `background-size` makes the picture as small as possible while still covering the entire background area. `contain`, on the other hand, makes the image as large as possible while not being clipped by the background area.

For an image with an intrinsic ratio, exactly one size matches the `cover`/fit criteria alone. But if there is no intrinsic ratio specified, `cover`/fit isn't sufficient, so the large/small constraints choose the resulting size.

No dimensions or intrinsic ratio

In this example, the image has no dimensions or intrinsic ratio set. If an image doesn't specify either dimensions or an intrinsic ratio, neither rule 2 nor rule 3 apply, so rule 4 takes over: the background image is rendered covering the entire background area. This satisfies the largest-or-smallest constraint.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-or-ratio.svg");
  background-size: contain;
}
```

One specified dimension, no intrinsic ratio

In this example, with the image having one dimension specified but no intrinsic ratio, rule 4 applies, and the image is scaled to cover the entire background area.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-wide-no-height-or-ratio.svg");
  background-size: contain;
}
```

One specified dimension with intrinsic ratio

In these examples, the image has one dimension explicitly set, along with an intrinsic ratio.

Things change when you specify an intrinsic ratio. In this case, rule 1 isn't relevant, so rule 2 is applied: we try to preserve any intrinsic ratio (while respecting `contain` or `cover`). For example, preserving a 3:4 intrinsic aspect ratio for a 300x200 box with `contain` means drawing a 150x200 background.

contain case

```
<div></div>
```

Given this CSS:

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-height-3x4-ratio.svg");
  background-size: contain;
}
```

Notice how the entire image is rendered, fitting as best as possible into the box without clipping any of it away.

cover case

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-height-3x4-ratio.svg");
  background-size: cover;
}
```

Here, the 3:4 ratio is preserved while still stretching the image to fill the entire box. That causes the bottom of the image to be clipped away.

No dimensions with intrinsic ratio

These example use the image with an intrinsic ratio set but no defined dimensions. When using an image with no intrinsic dimensions but an intrinsic ratio, things work similarly.

contain case

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-1x1-ratio.svg");
  background-size: contain;
}
```

Notice that the image is sized to fit the smallest dimension while preserving the 1:1 aspect ratio.

cover case

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-1x1-ratio.svg");
  background-size: cover;
}
```

Here, the image is sized so that it fills the largest dimension. The 1:1 aspect ratio has been preserved, although with this source image, that can be difficult to see.

Automatic sizing using "auto" for both dimensions

If `background-size` is set to `auto` or `auto auto`, rule 2 says that rendering must preserve any intrinsic ratio that's provided.

No dimensions or intrinsic ratio

When auto-sizing background images with no intrinsic ratio or dimensions specified, rule 4 takes effect, and the image is rendered to fill the background area.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-or-ratio.svg");
  background-size: auto auto;
}
```

One dimension and no intrinsic ratio

If no intrinsic ratio is specified, but at least one dimension is specified, rule 3 takes effect, and we render the image obeying those dimensions.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-wide-no-height-or-ratio.svg");
  background-size: auto auto;
}
```

Note here that the width, which is specified in the source SVG at 100 pixels, is obeyed, while the height fills the background area since it's not specified (either explicitly or by an intrinsic ratio).

One dimension and an intrinsic ratio

If we have an intrinsic ratio with a fixed dimension, that fixes both dimensions in place. As previously mentioned, knowing one dimension and a ratio is the same as specifying both dimensions explicitly.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-height-3x4-ratio.svg");
  background-size: auto auto;
}
```

Since this image has an explicit height of `100px`. With the 3:4 ratio set in the SVG, in the case of `auto`, the image's width is 75 pixels.

No fixed dimensions with intrinsic ratio

When an intrinsic ratio is specified, but no dimensions, rule 4 is applied — except that rule 2 also applies. The image is therefore rendered just like for the `contain` case.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-1x1-ratio.svg");
  background-size: auto auto;
}
```

Using "auto" and one specific length

Given rule 1, specified dimensions are always used, so we need to use our rules only to determine the second dimension.

No dimensions or intrinsic ratio

If the image has no dimensions or intrinsic ratio, rule 4 applies, and we use the background area's dimension to determine the value for the `auto` dimension.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-or-ratio.svg");
  background-size: auto 140px;
}
```

Here, the width is determined using the background area's width per rule 4, while the height is the `140px` specified in the CSS.

One specified dimension with no intrinsic ratio

If the image has one specified dimension but no intrinsic ratio, that specified dimension is used per rule 3 if that dimension is set to `auto` in the CSS.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-wide-no-height-or-ratio.svg");
  background-size: 200px auto;
}
```

Here, the `200px` specified in the CSS overrides the `100px` width specified in the SVG, per rule 1. Since there's no intrinsic ratio or height provided, `auto` selects the height of the background area as the height for the rendered image.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-wide-no-height-or-ratio.svg");
  background-size: auto 125px;
}
```

In this case, the width is specified as `auto` in the CSS, so the `100px` width specified in the SVG is selected, per rule 3. The height is set at `125px` in the CSS, so that is selected per rule 1.

One specified dimension with intrinsic ratio

When a dimension is specified, rule 1 applies that dimension from the SVG to the rendered background unless specifically overridden by the CSS. When an intrinsic ratio is also specified, that's used to determine the other dimension.

```
<div></div>
```

```
div {
  width: 300px;
```

```
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("100px-height-3x4-ratio.svg");
  background-size: 150px auto;
}
```

In this case, we use the width of the image specified in the CSS set at `150px`, so rule 1 is applied. The intrinsic 3:4 aspect ratio then determines the height for the `auto` case.

No specified dimensions with intrinsic ratio

If no dimensions are specified in the SVG, the specified dimension in the CSS is applied, then the intrinsic ratio is used to select the other dimension, per rule 2.

```
<div></div>
```

```
div {
  width: 300px;
  height: 200px;
  background-repeat: no-repeat;
  border: 2px solid black;
}
```

```
div {
  background-image: url("no-dimensions-1x1-ratio.svg");
  background-size: 150px auto;
}
```

The width is set by the CSS to `150px`. The `auto` value for the height is computed using that width and the 1:1 aspect ratio to be `150px` as well.

See also

- `background-size`
- CSS backgrounds and borders module

Using multiple backgrounds

You can apply **multiple backgrounds** to elements. These are layered atop one another with the first background you provide on top and the last background listed in the back. Only the last background can include a background color.

Multiple backgrounds are specified as a comma-separated list, such as `background: background1, background2, ...;`. This syntax is accepted by both the shorthand `background` property and the individual properties thereof except for `background-color`: `background-attachment`, `background-clip`, `background-image`, `background-origin`, `background-position`, `background-repeat`, `background-size`.

Example

In this example, three backgrounds are stacked: the Firefox logo, an image of bubbles, and a linear gradient:

HTML

```
<div class="multi-bg-example"></div>
```

CSS

```
.multi-bg-example {
  width: 100%;
  height: 400px;
  background-image:
    url("firefox.png"), url("bubbles.png"),
    linear-gradient(to right, rgb(30 75 115 / 100%), transparent);
  background-repeat: no-repeat, no-repeat, no-repeat;
  background-position:
    bottom right,
    left,
    right;
}
```

Result

As you can see here, the Firefox logo (listed first within `background-image`) is on top, directly above the bubbles graphic, followed by the gradient (listed last) sitting underneath all previous 'images'. Each subsequent sub-property (`background-repeat` and `background-position`) applies to the corresponding backgrounds. So the first listed value for `background-repeat` applies to the first (frontmost) background, and so forth.

Value repetition for multiple backgrounds

When using multiple backgrounds, if a background-related property is given fewer comma-separated values than the number of background layers, the user agent repeats the list of values until there are enough values for all layers.

For example:

```
.element {
  background-image: url("a.png"), url("b.png"), url("c.png");
  background-position: left top;
}
```

This is equivalent to:

```
.element {
  background-position:
    left top,
    left top,
    left top;
}
```

See also

- CSS backgrounds and borders module
- Using CSS gradients

CSS basic user interface

The **CSS basic user interface** module allows you to define the rendering and functionality of features related to the user interface, including outline properties, visual feedback for pointing devices and keyboards, and modifying the default appearance of UI widgets.

Basic user interface properties can be used to improve user experience and accessibility by providing visual cues to elements that are being interacted with, including styling mouse cursors and keyboard directional focus navigation, and styling caret cursors when an editable element has focus. The module includes features for providing outlines to focused (or unfocused) elements without impacting an element's box model dimensions and styling. This UI module also enables the styling of user interface controls.

Basic user interface in action

To view how basic user interface properties can alter the appearance of UI features, interact with the elements in this sample. Note that some features in this sample improve usability while others harm user experience.

```
<div><span contenteditable>Edit this text </span></div>
<fieldset>
  <legend>Play with these fake form controls</legend>
  <input type="checkbox" id="check" />
  <input type="radio" name="a" />
  <input type="radio" name="a" />
  <input type="range" />
  <progress></progress>
</fieldset>
<fieldset>
  <legend>Be careful not to ruin usability: try resizing these.</legend>
  <textarea>
  cursor: wait;
  </textarea>
  <textarea>
  resize: none;
  </textarea>
  <textarea>
  pointer-events: none;
  </textarea>
</fieldset>
```

```
body {
  font-family: sans-serif;
  font-size: 1.25rem;
}
[contenteditable] {
  cursor: copy;
  caret-color: magenta;
  border: 1px solid #cccccc;
}
:focus {
  outline: dashed magenta 3px;
  outline-offset: 10px;
}
* {
  accent-color: magenta;
}
div,
fieldset {
  margin: 20px;
}
textarea:nth-of-type(1) {
  cursor: wait;
}
textarea:nth-of-type(2) {
  resize: none;
}
textarea:nth-of-type(3) {
  pointer-events: none;
}
```

The CSS `outline` and `outline-offset` properties were used to provide feedback to users on which element currently has focus. An `accent-color` provides a theme color to all the form controls. The caret that appears when the text is edited has the same color thanks to the `caret-color` property. These can all be considered UI improvements.

Some features harm usability. The `cursor` property was used to change cursors from the browser default, which is confusing. The `resize` property prevents the second `<textarea>` from being resizable while the `pointer-events` property prevents the third `<textarea>` from receiving click events. It is still focusable using the keyboard.

Click "Play" in the example above to see or edit the code for the animation in the MDN Playground.

Reference

Properties

- `accent-color`
- `appearance`
- `caret-animation`
- `caret-color`
- `caret-shape`
- `cursor`
- `interest-delay`, shorthand for:
  - `interest-delay-start`
  - `interest-delay-end`
- `outline`, shorthand for:
  - `outline-color`
  - `outline-style`
  - `outline-width`
- `outline-offset`

- `pointer-events`
- `resize`
- `user-select`

The CSS basic user interface module also defines the `caret`, `nav-down`, `nav-left`, `nav-right`, and `nav-up` properties. Currently, no browsers support these features.

Guides

Learn forms: advanced form styling

Explains how `appearance` can be used to style form controls.

Related concepts

- CSS `cursor` property
- SVG `cursor` attribute
- CSS `:focus`, `:focus-within`, and `:focus-visible` pseudoclasses
- `CaretPosition` Interface

See also

- Tips for Designing Useful and Usable Focus Indicators (2016)

CSS borders and box decorations

The **CSS borders and box decorations** module provides properties for adding borders, shaped corners, and box shadows to elements. This module extends borders and box decorations introduced in the CSS backgrounds and borders module, adding `corner-shape` and `border-shape` properties, logical `border-radius` properties, longhand properties for the `box-shadow` property, and properties to create partial borders.

Borders and box decorations in action

Select a `superellipse()` value from the dropdown menu to change the border shape. Use the slider to change the border radius size. Toggle the checkbox to hide and show the box shadow.

```
<p>
  <label for="corner-shape-choice"
    >Choose a <code>superellipse()</code> value:</label
  >
  <select id="corner-shape-choice">
    <option>superellipse(infinity)</option>
    <option>superellipse(5)</option>
    <option>superellipse(3)</option>
    <option>superellipse(2)</option>
    <option>superellipse(1.5)</option>
    <option>superellipse(1)</option>
    <option>superellipse(0.5)</option>
    <option>superellipse(0)</option>
    <option>superellipse(-0.5)</option>
    <option selected>superellipse(-1)</option>
    <option>superellipse(-1.5)</option>
    <option>superellipse(-2)</option>
    <option>superellipse(-3)</option>
    <option>superellipse(-5)</option>
    <option>superellipse(-infinity)</option>
  </select>
</p>
<p>
  <label for="radius">Choose a <code>border-radius</code> value:</label>
  <input
    type="range"
    step="5"
    min="0"
    max="100"
    value="30"
    id="radius"
    list="tens" />
  <datalist id="tens">
    <option value="0" label="0"></option>
    <option value="20" label="20px"></option>
    <option value="40" label="40px"></option>
    <option value="60" label="60px"></option>
    <option value="80" label="80px"></option>
    <option value="100" label="100px"></option>
  </datalist>
</p>

<p>
  <input type="checkbox" id="check" />
  <label for="check">Toggle the box-shadow</label>
</p>
<div></div>
```

```
div {
  width: 100%;
  height: 200px;
  background-color: plum;
  background-image:
    repeating-linear-gradient(transparent 0 19px, #00000022 19px 20px),
    repeating-linear-gradient(to left, transparent 0 19px, #00000022 19px 20px);
}

div {
  box-shadow: 3px 3px 5px rgb(0 0 0 / 0.5);
  border-radius: 30px;
}
body:has(input:checked) div {
  box-shadow: none;
}

@layer page-setup {
  html {
    font-family: "Helvetica", "Arial", sans-serif;
  }
```

```
  body {
    max-width: 600px;
    min-width: fit-content;
    margin: 20px auto;
  }
  div {
    display: flex;
    justify-content: center;
    align-items: center;
    margin-top: 20px;
  }
  select {
    padding: 3px 5px;
  }
  code {
    font-weight: bolder;
  }
}
```

```
const rectangle = document.querySelector("div");
const select = document.querySelector("select");
const range = document.getElementById("radius");

function setCornerShape() {
  rectangle.style.cornerShape = select.value;
  rectangle.style.borderRadius = `${range.value}px`;
  rectangle.innerHTML = `<pre>div {
  corner-shape: ${select.value};
  border-radius: ${range.value}px;
}</pre>`;
}

select.addEventListener("change", setCornerShape);
range.addEventListener("input", setCornerShape);
setCornerShape();
```

Reference

Properties

- border-block
- border-block-color
- border-block-end
- border-block-end-color
- border-block-end-style
- border-block-end-width
- border-block-start
- border-block-start-color
- border-block-start-style
- border-block-start-width
- border-block-style
- border-block-width
- border-bottom
- border-bottom-color
- border-bottom-left-radius
- border-bottom-right-radius
- border-bottom-style
- border-bottom-width
- border-color
- border-end-end-radius
- border-end-start-radius
- border-inline
- border-inline-color
- border-inline-end
- border-inline-end-color
- border-inline-end-style
- border-inline-end-width
- border-inline-start
- border-inline-start-color
- border-inline-start-style
- border-inline-start-width
- border-inline-style
- border-inline-width
- border-left
- border-left-color
- border-left-style
- border-left-width
- border-radius
- border-right
- border-right-color
- border-right-style
- border-right-width
- border-shape
- border-start-end-radius
- border-start-start-radius
- border-top
- border-top-color
- border-top-left-radius
- border-top-right-radius
- border-top-style
- border-top-width

- box-shadow
- corner-block-end-shape
- corner-block-start-shape
- corner-bottom-left-shape
- corner-bottom-right-shape
- corner-bottom-shape
- corner-end-end-shape
- corner-end-start-shape
- corner-inline-end-shape
- corner-inline-start-shape
- corner-left-shape
- corner-right-shape
- corner-shape
- corner-start-end-shape
- corner-start-start-shape
- corner-top-left-shape
- corner-top-right-shape
- corner-top-shape

The CSS borders and box decorations module level 4 also introduces the `border-limit`, and `border-clip` properties, along with the `border-clip-bottom`, `border-clip-left`, `border-clip-right`, `border-clip-top` longhand properties. Currently, no browsers support these features. The module also introduces component properties for the well supported `border-radius` and `box-shadow` properties, including `border-block-end-radius`, `border-block-start-radius`, `border-bottom-radius`, `border-inline-end-radius`, `border-inline-start-radius`, `border-right-radius`, `border-top-radius`, `box-shadow-blur`, `box-shadow-color`, `box-shadow-offset`, `box-shadow-position`, and `box-shadow-spread`. These component properties are also as yet unsupported.

Data types

- `<corner-shape-value>`

Functions

- `superellipse()`

Guides

Learn CSS: the box model

Learn how borders and other box model properties impact the CSS box model.

Creating an irregular nav menu with border-shape

How to use the `border-shape` property to create an irregular animated navigation menu.

Related concepts

- `box-sizing` property
- `box-decoration-break` property
- `text-shadow` property
- `<url>` data type
- `<color>` data type
- `<image>` data type
- `<position>` data type
- `currentColor` keyword

CSS backgrounds and borders module

- `background-attachment`
- `background-clip`
- `background-color`
- `background-image`
- `background-origin`
- `background-position`
- `background-repeat`
- `background-size`
- `background` shorthand
- `background-position-x`
- `background-position-y`
- `border-image-outset`
- `border-image-repeat`
- `border-image-slice`
- `border-image-source`
- `border-image-width`
- `border-image` shorthand

See also

- `filter`
- `backdrop-filter`
- `drop-shadow()` filter function
- Applying color to HTML elements using CSS
- Tools:
  - Border-image generator
  - Border-radius generator

Creating an irregular nav menu with border-shape

The CSS `border-shape` property can be applied to elements to create precisely-shaped containers such as speech bubbles, abstract tooltip designs, and more, enabling many new creative ideas to be put into production without the need for workarounds or hacks. You can learn all about the property and see basic examples in action on the property reference page.

In this guide, we show you how to use `border-shape` to create an irregular animated navigation menu with each nav item shaped like a jigsaw piece.

Defining the nav menu markup

Our HTML is fairly typical for a navigation menu — a list of links.

```
<ul>
  <li><a href="#">One</a></li>
  <li><a href="#">Two</a></li>
  <li><a href="#">Three</a></li>
  <li><a href="#">Four</a></li>
</ul>
```

Setting the basic page styles

The basic page setup styles are as follows. We apply some rudimentary font styles, use `height` to cause the `<body>` to fill the viewport, and use flexbox to place the nav menu in the center of the screen.

```
* {
  box-sizing: border-box;
}

html {
  height: 100%;
  font-family: "Helvetica", "Arial";
  font-size: 1.2rem;
}

body {
  margin: 0;
  height: inherit;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Applying the general nav styles

First, we style the `<ul>` by removing the default `list-style-type` and `padding` and setting a `display` value of `flex` to lay out the contained `<li>` elements in a row. We then set a `gap` value of `0` and apply a `transition`, so that when the `<ul>` state changes, a change in the `gap` value animates smoothly.

```
ul {
  list-style-type: none;
  padding: 0;
  display: flex;
  gap: 0;
  transition: gap 0.6s;
}
```

Next, we style the `<li>` elements. We want each nav item to be square, so we set an equal `width` and `height`.

```
li {
  width: 160px;
  height: 160px;
}
```

Next, we style the `<a>` elements inside the list items. We start by removing the default `text-decoration` and setting the `color` to `black`. We then set a `width` and `height` of `100%` to make the `<a>` elements fill the full area of the `<li>` elements, then use flexbox to center their text horizontally and vertically.

We then set `box-shadow` and `text-shadow` properties on the links, plus a `transition` so that any property value changes are animated smoothly when the element's state changes.

```
a {
  text-decoration: none;
  color: black;
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;

  box-shadow:
    2px 0px 2px rgb(0 0 0 / 0.5),
    inset 3px 3px 3px rgb(255 255 255 / 0.5);
  text-shadow: 1px 1px 1px rgb(0 0 0 / 0.5);
  transition: all 0.6s;
}
```

We then give each jigsaw piece a different color:

```
li:nth-child(1) a {
  background-color: #2de1fc;
}

li:nth-child(2) a {
  background-color: #2afc98;
}

li:nth-child(3) a {
  background-color: #09e85e;
}

li:nth-child(4) a {
  background-color: #16c172;
}
```

Handling the border shape

Now it's time to set the `border-shape` for each nav item to get the jigsaw piece look we are after. For variety, we've shaped each odd-numbered `<a>` element like a downwards-pointing jigsaw-piece, and each even-numbered `<a>` element like an upwards-pointing jigsaw-piece:

```
li:nth-child(even) a {
  border-shape: shape(
      from 0% 0%,
      hline to 33%,
      arc by 33% 0% of 16% 20% small cw,
      hline to 100%,
      line to 100% 33%,
      arc by 0% 33% of 20% 16% small cw,
      line to 100% 100%,
      hline to 66%,
      arc by -33% 0% of 16% 20% small ccw,
      hline to 0%,
      line to 0% 66%,
      arc by 0% -33% of 20% 16% small ccw,
      close
    )
    content-box;
}

li:nth-child(odd) a {
  border-shape: shape(
      from 0% 0%,
      hline to 33%,
      arc by 33% 0% of 16% 20% small ccw,
      hline to 100%,
      line to 100% 33%,
      arc by 0% 33% of 20% 16% small cw,
      line to 100% 100%,
      hline to 66%,
      arc by -33% 0% of 16% 20% small cw,
      hline to 0%,
      line to 0% 66%,
      arc by 0% -33% of 20% 16% small ccw,
      close
    )
    content-box;
}
```

This immediately creates an issue — the notches on the jigsaw pieces that extend out from the original `<a>` area aren't filled by the `<a>` elements' background colors.

There is a solution to this problem. We've deliberately included the `content-box` `<geometry-box>` value after each `shape()` function in the previous two rules. This means the shapes will be drawn relative to the elements' content boxes, and any applied `padding` won't be set inside the shape. Instead, the padding will be placed outside the shape, causing it to get smaller and forcing the background color to fill up the notches.

The required `padding` is set like so:

```
a {
  padding: 24px;
}
```

> **Note:** You can see what the background problem looks like by inspecting the live example in your browser's developer tools and disabling the `padding` applied to the `<a>` elements.

The `padding` causes the jigsaw pieces to get smaller, so there are gaps between them. We want them to touch initially, so we set a large negative `margin-right` value on each list item to bring them together:

```
li {
  margin-right: -47px;
}
```

A side-effect of this `margin-right` setting is that all of the `<li>` items are moved to the right, so the nav menu is no longer horizontally centered. To fix this, we use relative positioning to move the `<ul>` back to the left:

```
ul {
  position: relative;
  right: 23.5px;
}
```

Finally, we apply some style updates on `:hover` and `:focus` that, when combined with the `transition` properties we set earlier, produce some animated effects on interaction with the nav items. We increase the `gap` set on the `<ul>` flexbox layout when it is hovered over or focused. To handle the focus state, we use the `:has` pseudo-class to select the entire `<ul>` when any `<a>` inside it is focused.

```
ul:hover,
ul:has(a:focus) {
  gap: 30px;
}
```

We then set an increased `brightness` `filter`, `scale` factor, and outer `box-shadow` on the `<a>` elements themselves when they are hovered over or focused, making them appear brighter and raised up on interaction.

```
a:hover,
a:focus {
  filter: brightness(1.2);
  scale: 1.1;
  box-shadow:
    5px 0px 10px rgb(0 0 0 / 0.5),
    inset 3px 3px 3px rgb(255 255 255 / 0.5);
}
```

Result

Hover over or focus the nav items to see the animated effects. Note how naturally the different applied effects work with the `border-shape` values.

```
@supports not (border-shape: circle(50%)) {
  body::before {
    content: "Your browser does not support the 'border-shape' property.";
    font-family: sans-serif;
    background-color: wheat;
    padding: 1rem 0;
```

```
    text-align: center;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

See also

- `border-shape`
- CSS borders and box decorations module
- CSS backgrounds and borders module
- border-shape: the future of the non-rectangular web by Una Kravets (2026)
- Get Ready For the Powerful CSS border-shape Property! on CSS Tricks (2026)

CSS box alignment

The **CSS box alignment** module specifies CSS features relating to the alignment of boxes within their containers. It defines the alignment of the various CSS box layout models including block layout, table layout, flexible box layout (flexbox), and grid layout, creating a consistent alignment method across all of CSS.

The module details alignment terminology, enabling alignment properties to be used in multiple layout modules, rather than limited to a particular layout method.

Alignment is linked to writing modes in that when we align an item we do not consider whether we are aligning it to the physical dimensions of top, right, bottom, and left. Instead, we describe alignment in terms of the start and end of the particular dimension we are working with. This ensures that alignment works in the same way whichever writing mode the document has.

The alignment of text and inline-level content is defined in CSS text module and CSS inline module, respectively.

Reference

CSS Properties

- `align-content`
- `align-items`
- `align-self`
- `justify-content`
- `justify-items`
- `justify-self`
- `place-content`
- `place-items`
- `place-self`

Data types

- `<baseline-position>`
- `<content-distribution>`
- `<content-position>`
- `<overflow-position>`
- `<self-position>`

Terms and definitions

- Alignment container
- Alignment subject
- Baseline alignment
- Distributed alignment
- Fallback alignment
- Positional alignment

Guides

Box alignment overview

Overview of the general concepts found in the CSS box alignment module.

Box alignment in flexbox

How box alignment works in the context of flexbox.

Box alignment in CSS grid layout

How box alignment works in the context of grid layout.

Box alignment in multiple-column layout

How box alignment works in the context of multi-column layout.

Box alignment for block, absolutely positioned and table layout

How box alignment works in the context of block layout, including floated, positioned, and table elements.

Related concepts

- `alignment-baseline`
- `dominant-baseline`
- `scroll-snap-align`
- SVG `dominant-baseline` attribute
- Cross axis
- Main axis

CSS gaps module

- `column-gap`
- `gap`
- `row-gap`

See also

- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS display module
- CSS flexible box layout module
- CSS grid layout module

Box alignment for block, absolutely positioned, and table layouts

The CSS box alignment module details how alignment works in various layout methods. In this guide, we explore how box alignment works in the context of block layout, including floated, positioned, and table elements. As this guide aims to detail things which are specific to block layout and box alignment, it should be read in conjunction with the box alignment guide, which details the common features of box alignment across layout methods.

align-content and justify-content

The `justify-content` property does not apply to block containers or table cells.

The `align-content` property applies to the block axis in order to align the contents of the box within its container. If a content distribution method such as `space-between`, `space-around` or `space-evenly` is requested then the fallback alignment will be used, as the content is treated as a single alignment subject.

justify-self

The `justify-self` property is used to align an item inside its containing block on the inline axis.

This property does not apply to floated elements or table cells.

align-self

The `align-self` property does not apply to block-level boxes (including floats), because there is more than one item in the block axis. It also does not apply to table cells.

Absolutely positioned elements

The alignment container is the positioned block, accounting for the offset values of top, left, bottom, and right. The normal keyword resolves to `stretch`, unless the positioned item is a replaced element, in which case it resolves to `start`.

Aligning in these layout methods today

As we do not currently have browser support for box alignment in block layout, your options for alignment are either to use one of the existing alignment methods or, to make even a single item inside a container a flex item in order to use the alignment properties as specified in flexbox.

Alignment of blocks horizontally prior to flexbox was typically achieved by way of setting auto margins on the block. A `margin` of `auto` will absorb all available space in that dimension, therefore setting a left and right margin of auto, you can push a block into the center:

```
.container {
  width: 20em;
  margin-left: auto;
  margin-right: auto;
}
```

In table layout, you have access to the `vertical-align` property to align the contents of a cell inside that cell.

For many use cases, turning the block container into a flex item will give you the alignment capability that you are looking for. In the example below, a container with a single item inside has been turned into a flex container for the purpose of being able to use the alignment properties.

```
<div class="box">
  <div></div>
</div>
```

```
.box {
  height: 300px;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
.box {
  display: flex;
  align-items: center;
  justify-content: center;
}

.box div {
  width: 100px;
  height: 100px;
}
```

See also

- CSS box alignment module
- Box alignment overview
- Box alignment in flexbox
- Box alignment in CSS grid layout
- Box alignment in multiple-column layout

Box alignment in flexbox

The box alignment module details how alignment works in various layout methods. In this guide, we explore how box alignment works in the context of flexbox. As this guide aims to detail things which are specific to flexbox and box alignment, it should be read in conjunction with the box alignment overview guide, which details the common features of box alignment across layout methods.

Basic example

In this flexbox example, three flex items are aligned on the main axis using `justify-content` and on the cross axis using `align-items`. The first item overrides the `align-items` values set on the group by setting `align-self` to `center`.

```
body {
  font-family: sans-serif;
}
.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
</div>
```

```
.box {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  border: 2px dotted rgb(96 139 168);
}

.box :first-child {
  align-self: center;
}
```

The axes and flex-direction

Flexbox respects the writing mode of the document, therefore if you are working in English and set `justify-content` to `flex-end` this will align the items to the end of the flex container. If you are working with `flex-direction` set to `row`, this alignment will be in the inline direction.

However, in flexbox you can change the main axis by setting `flex-direction` to `column`. In this case, `justify-content` will align items in the block direction. Therefore it is easiest to think about the main and cross axis when working in flexbox like so:

- The main axis = direction set by `flex-direction` = alignment via `justify-content`
- The cross axis = runs across the main axis = alignment via `align-content`, `align-self`/`align-items`

Main Axis Alignment

- `justify-content`

Cross Axis Alignment

- `align-self`
- `align-items`
- `align-content`

There is no justify-self in flexbox

On the main axis, Flexbox deals with the flex items as a group. The amount of space required to lay out the items is calculated, and the leftover space is then available for distribution. The `justify-content` property controls how that leftover space is used. Set `justify-content: flex-end` and the extra space is placed before the items, `justify-content: space-around` and it is placed either side of the item in that dimension, etc.

This means that a `justify-self` property does not make sense in flexbox as we are always dealing with moving the entire group of items around.

On the cross axis `align-self` makes sense as we potentially have additional space in the flex container in that dimension, in which a single item can be moved to the start and end.

Alignment and auto margins

There is a specific use case in flexbox where we might think that a `justify-self` property is what we need, and this is when we want to split a set of flex items, perhaps to create a split navigation pattern. For this use case, we can use an `auto` margin. A margin set to `auto` will absorb all available space in its dimension. This is how centering a block with auto margins works. By setting the left and right margin to `auto`, both sides of our block try to take up all of the available space and so push the box into the center.

By setting a `margin` of `auto` on one item in a set of flex items all aligned to start, we can create a split navigation. This works well with flexbox and the alignment properties. As soon as there is no space available for the auto margin, the item behaves in the same way as all the other flex items and shrinks to try to fit into space.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div class="push">Four</div>
  <div>Five</div>
</div>
```

```
.box {
  display: flex;
  border: 2px dotted rgb(96 139 168);
}
.push {
  margin-left: auto;
}
```

The <code>gap</code> properties

- `row-gap`
- `column-gap`
- `gap`

Creating fixed size gaps between items

On the main axis, the `column-gap` property creates fixed size gaps between adjacent items.

On the cross axis the `row-gap` property creates spacing between adjacent flex lines, therefore `flex-wrap` must also be set to `wrap` for this to have any effect.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
</div>
```

```
.box {
  width: 450px;
  display: flex;
  flex-wrap: wrap;
  row-gap: 10px;
  column-gap: 2em;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  flex: 1;
}
```

See also

- CSS box alignment module
- Box alignment overview
- Box alignment in CSS grid layout
- Box alignment in multiple-column layout
- Box alignment for block, absolutely positioned, and table layouts
- Aligning items in flex container
- Cross axis
- Main axis

Box alignment in grid layout

The CSS box alignment module details how alignment works in various layout methods. On this page, we explore how box alignment works in the context of CSS grid layout.

As this guide aims to detail things which are specific to CSS grid layout and Box Alignment, it should be read in conjunction with the box alignment overview guide, which details the common features of box alignment across layout methods.

Basic example

In this example using grid layout, there is extra space in the grid container after laying out the fixed-width tracks on the inline main axis. This space is distributed using `justify-content`. On the block cross axis the alignment of the items inside their grid areas is controlled with `align-items`. The first item overrides the `align-items` value set on the group by setting `align-self` to `center`.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
</div>
```

```
body {
  font-family: sans-serif;
}
.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

```
.box {
  display: grid;
  grid-template-columns: 120px 120px 120px;
  align-items: start;
  justify-content: space-between;
  border: 2px dotted rgb(96 139 168);
}

.box :first-child {
  align-self: center;
}
```

Grid axes

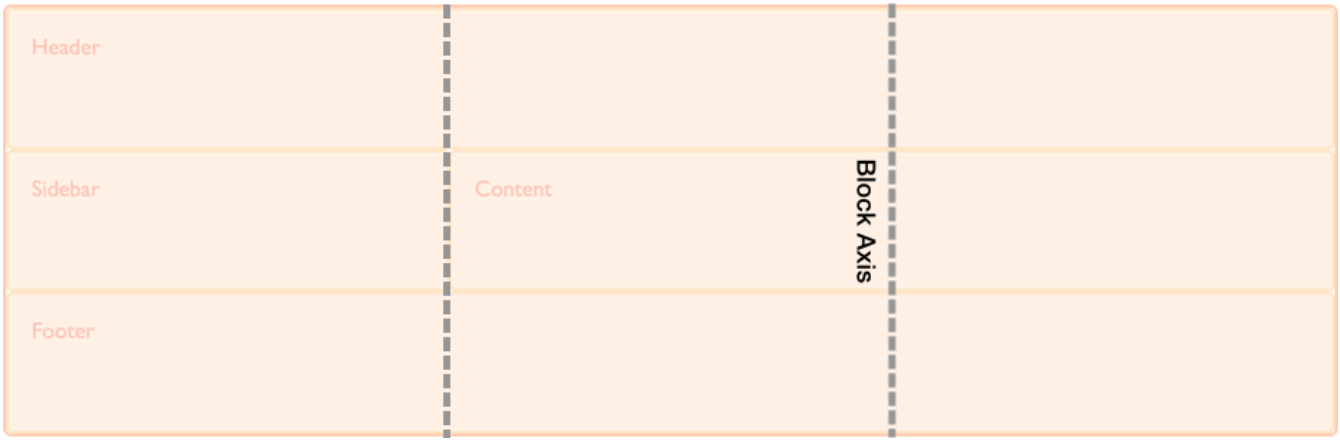
As a two-dimensional layout method, when working with grid layout we always have two axes on which to align our items. We have access to all of the box alignment properties to help us achieve this.

The inline axis is the axis that corresponds to the direction that words in a sentence would run in the writing mode used. Therefore, in a horizontal language such as English or Arabic, the inline direction runs horizontally. Should you be in a vertical writing mode, the inline axis will run vertically.

To align things on the inline axis you use the properties that start with `justify-`: `justify-content`, `justify-items` and `justify-self`.

The block axis crosses the inline axis in the direction that blocks are displayed down the page — for example, paragraphs in English are displayed one below the other vertically. This is the block dimension.

To align things on the block axis you use the properties that start with `align-`, `align-content`, `align-items` and `align-self`.



Self alignment

These properties deal with aligning the item inside the grid area it is placed into:

- `justify-self`
- `align-self`
- `place-self`
- `justify-items`
- `align-items`
- `place-items`

The `*-items` properties, `align-items` and `justify-items`, are applied to the grid container and set alignment for all grid items as a group. The `*-self` properties, `align-self` and `justify-self`, are instead set on grid items. This means that you can set alignment on all grid items, and then override any items that need a different alignment by applying the `align-self` or `justify-self` property to the rules for the individual grid items.

The initial value for `align-items` and `justify-items` is `stretch`, and the initial value for `align-self` and `justify-self` is `auto`, so the item will stretch over the entire grid area. The exception to this rule is where the item has an intrinsic aspect ratio, for example an image. In this case the item will be aligned to `start` in both dimensions in order that the image is not distorted.

Content alignment

These properties deal with aligning the tracks of the grid when there is extra space to distribute:

- `justify-content`
- `align-content`
- `place-content`

This scenario will occur if the tracks that you have defined total less than the total width of the grid container.

Gap and legacy grid-gap properties

These properties define the spacing between grid items within a grid container:

- `row-gap`
- `column-gap`
- `gap`

The grid specification originally contained the definition for the properties `grid-row-gap`, `grid-column-gap` and `grid-gap`. These have since been moved into the Box Alignment specification and aliased to `row-gap`, `column-gap`, and `gap`. This allows them to be used for other layout methods where a gap between items makes sense.

See also

- CSS box alignment module
- Box alignment overview
- Box alignment in flexbox
- Box alignment in multiple-column layout
- Box alignment for block, absolutely positioned, and table layouts
- Aligning items in CSS grid layout

Box alignment in multi-column layout

The CSS box alignment module details how alignment works in various layout methods; in this guide, we explore how box alignment works in the context of multi-column layout. As this guide aims to detail things that are specific to both modules, it should be read in conjunction with the box alignment overview guide, which details the common features of box alignment across layout methods.

In multi-column layout, the alignment container is the content box of the multicol container. The alignment subject is the column box. The properties which apply to multi-column layouts are detailed below.

align-content and justify-content

The `align-content` property applies to the block axis and `justify-content` to the inline axis. Any spacing added to the columns due to use of space distribution will be added to the gap between the columns, therefore making the gap larger than might be specified by the `column-gap` (or `gap` shorthand) property.

Using a value of `justify-content` other than `normal` or `stretch` will cause column boxes to display at the `column-width` specified on the multicol container, and the remaining space distributed according to the value of `justify-content`.

column-gap

The `column-gap` property was originally specified in the multiple-column layout specification and then later unified with the gap properties for other layout methods in box alignment. While other layout methods treat the initial value of `column-gap` as `0`, multi-column layout treats it as `1em` — you generally want a gap between columns.

See also

- CSS box alignment module
- Box alignment overview
- Box alignment in flexbox
- Box alignment in grid layout
- Box alignment for block, absolutely positioned, and table layouts

CSS box alignment overview

The CSS box alignment module specifies CSS features that relate to the alignment of boxes in the various CSS box layout models. The module aims to create a consistent method of alignment across all of CSS. The CSS box alignment properties provide full horizontal and vertical alignment capabilities.

This guide details the general concepts found in this module. Additional guides provide more information on box alignment in flexbox, grid layout, multiple-column layout, and block, absolutely positioned and table layout. Alignment of text is covered by the CSS text and CSS inline layout modules.

Key concepts and terminology

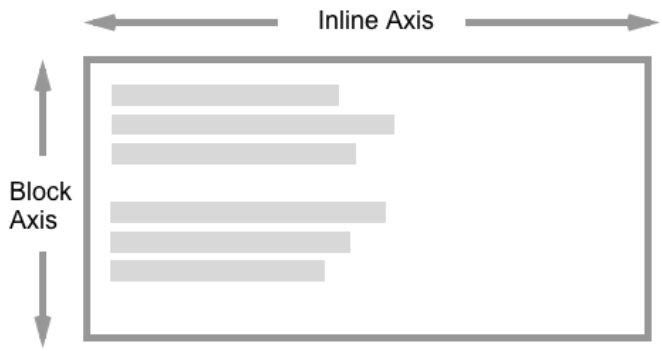
The specification details some alignment terminology to make it easier to discuss these alignment properties outside their implementation within a particular layout method. There are also some key concepts which are common to all layout methods.

Relationship to writing modes

Alignment is linked to writing modes in that when we align an item we do not consider whether we are aligning it to the physical dimensions of top, right, bottom and left. Instead we describe alignment in terms of the start and end of the particular dimension we are working with. This ensures that alignment works in the same way whichever writing mode the document has.

Two dimensions of alignment

When using the box alignment properties you will align content on one of two axes — the inline (or main) axis, and the block (or cross) axis. The inline axis is the axis along which words in a sentence flow in the writing mode being used. For English, for example, the inline axis is horizontal. The block axis is the axis along which blocks, such as paragraph elements, are laid out; it runs across the Inline axis.



When aligning items on the inline axis you will use the properties that begin with `justify-`:

- `justify-items`
- `justify-self`
- `justify-content`

When aligning items on the block axis you will use the properties that begin with `align-`:

- `align-items`
- `align-self`
- `align-content`

Flexbox adds an additional complication in that the above is true when `flex-direction` is set to `row`. The properties are swapped when flexbox is set to `column`. Therefore, when working with flexbox it is generally easier to think about the main and cross axis rather than inline and block. The `justify-` properties are always used to align on the main axis, the `align-` properties on the cross axis.

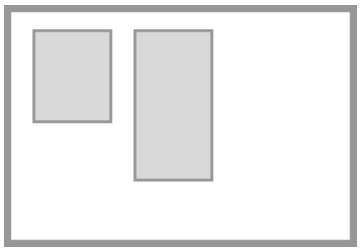
The alignment subject

The **alignment subject** is the thing that is being aligned. For `justify-self` or `align-self`, or when setting these values as a group with `justify-items` or `align-items`, this will be the margin box of the element that this property is being used on. The `justify-content` and `align-content` properties differ per layout method.

The alignment container

The **alignment container** is the box the subject is being aligned inside. This will typically be the alignment subject's containing block. An alignment container may contain one or many alignment subjects.

The below image shows an alignment container with two alignment subjects inside.



Types of alignment

There are three different types of alignment that the specification details; these use keyword values.

- Positional alignment
- Baseline alignment
- Distributed alignment

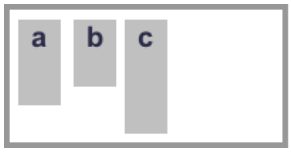
Positional alignment

Positional alignment is the position of an alignment subject with relation to its alignment container. The positional alignment keyword values are defined for positional alignment, and can be used as values for content alignment with `justify-content` and `align-content` and also for self alignment with `justify-self` and `align-self`.

- `center`
- `start`
- `end`
- `self-start`
- `self-end`
- `flex-start` for flexbox only
- `flex-end` for flexbox only
- `left`
- `right`

Other than the physical values of `left` and `right`, which relate to physical attributes of the screen, all of the other values, the `<self-position>` and `<content-position>` values, are logical values and relate to the writing mode of the content.

For example, when working in CSS grid layout, if you are working in English and set `justify-content` to `start` this will move the items in the inline dimension to the start, which will be the left as sentences in English start on the left-hand side of the page. If you were using Arabic, a right-to-left language, then the same value of `start` would result in the items moving to the right, as sentences in Arabic start on the right-hand side of the page.



Both have `justify-content: start`, but the location of the two starts is different because of the writing mode.

Baseline alignment

Baseline alignment is the relationship among the baselines of multiple alignment subjects within an alignment context. The Baseline alignment `<baseline-position>` keywords are used to align the baselines of boxes across a group of alignment subjects. They can be used as values for content alignment with `justify-content` and `align-content` and self-alignment with `justify-self` and `align-self`.

- `baseline`
- `first baseline`
- `last baseline`

Baseline content alignment — specifying a baseline alignment value for `justify-content` or `align-content` — works in layout methods that lay items out in rows. The alignment subjects are baseline aligned against each other by adding padding inside the boxes.

Baseline self-alignment shifts the boxes to align by baseline by adding a margin outside the boxes. Self-alignment is done for singular boxes using `justify-self` or `align-self`, or for groups of boxes using `justify-items` and `align-items`.

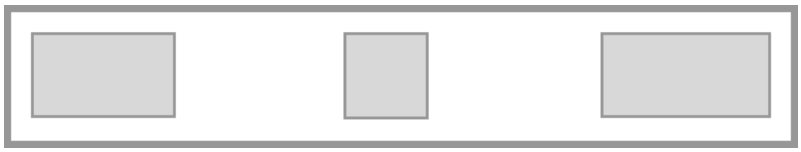
Distributed alignment

Distributed alignment defines alignment as a distribution of space among alignment subjects. The distributed alignment `<content-distribution>` keywords are used with the `align-content` and `justify-content` properties. These keywords define what happens to any additional space after alignment subjects have been displayed. The values are as follows:

- `stretch`
- `space-between`
- `space-around`
- `space-evenly`

For example, in Flex Layout items are aligned with `flex-start` initially. Working in a horizontal top-to-bottom writing mode (with a language such as English), with `flex-direction` set to `row`, the items start on the far left, and any available space after displaying the items is placed after them.

If you set `justify-content: space-between` on the flex container, the available space is now shared out and placed between the items.



For these keywords to take effect, space is required along the dimension in which you wish to align the items. With no space, there is nothing to distribute.

Basic examples

The following examples demonstrate how some of the box alignment properties are applied in Grid and Flexbox.

CSS grid layout alignment example

In this grid layout example, there is extra space in the grid container after laying out the fixed-width tracks on the inline (main) axis. This space is distributed using `justify-content`. On the block (cross) axis, the alignment of the items inside their grid areas is controlled with `align-items`. The first item overrides the `align-items` value set on the group by setting `align-self` to `center`.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box {
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

```
.box {
  display: grid;
  grid-template-columns: 120px 120px 120px;
  align-items: start;
  justify-content: space-between;
}

.box :first-child {
  align-self: center;
}
```

Flexbox Alignment Example

In this example, three flex items are aligned on the main axis using `justify-content` and on the cross axis using `align-items`. The first item overrides the `align-items` set on the group by setting `align-self` to `center`.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box {
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

```
.box {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
}

.box :first-child {
  align-self: center;
}
```

Overflow alignment

The `<overflow-position>` keywords `safe` and `unsafe` help define behavior when an alignment subject is larger than the alignment container. The `safe` keyword will align to `start` in the case of a specified alignment causing an overflow, the aim being to avoid "data loss" where part of the item is outside the boundaries of the alignment container and can't be scrolled to.

If you specify `unsafe` then the alignment will be honored even if it would cause such data loss.

Gaps between boxes

The box alignment specification also includes the `gap`, `row-gap`, and `column-gap` properties. These properties enable the setting of a consistent gap between items in a row or column, in any layout method which has items arranged in this way.

The `gap` property is a shorthand for `row-gap` and `column-gap`, which allows us to set these properties at once:

- `row-gap`
- `column-gap`
- `gap`

In the below example, a grid layout uses the `gap` shorthand to set a `10px` gap between row tracks, and a `2em` gap between column tracks.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box {
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

```
.box {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px 2em;
}

.box :first-child {
  align-self: center;
}
```

Early grid implementations included `gap` properties prefixed with `grid-`. All browsers support the unprefixed properties, though you may see the following properties in a code-base: `grid-row-gap`, `grid-column-gap`, and `grid-gap`. The prefixed versions are aliases of the unprefixed ones.

Be aware that other things may increase the visual gap displayed, for example using the space distribution keywords or adding margins to items.

Box-alignment by layout type

As the CSS box alignment properties are implemented differently depending on the specification they interact with, refer to the following guides for details of how to use the alignment properties with each layout type:

- Box alignment in flexbox
- Box alignment in CSS grid layout
- Box alignment in multiple-column layout
- Box alignment for block, absolutely positioned and table layout

See also

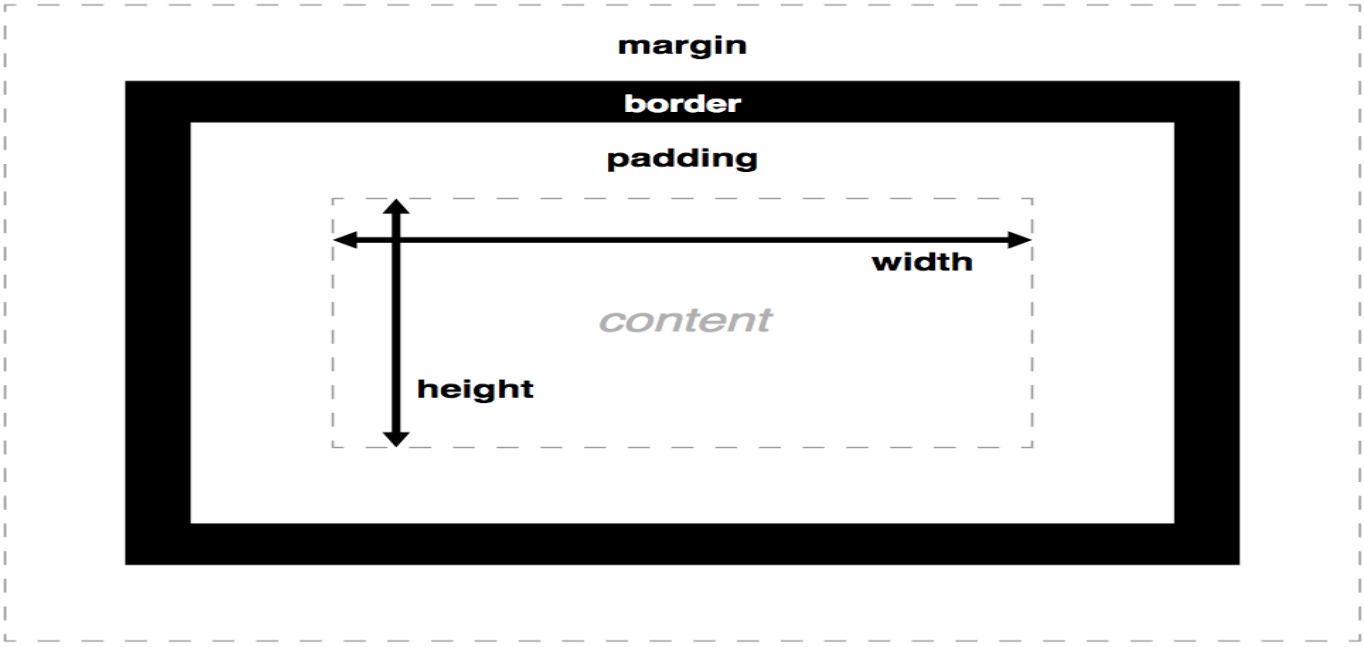
- CSS box alignment module
- Box alignment in grid layout
- CSS display module
- CSS flex layout module
- Basic concepts of flexbox
- Aligning items in a flex container
- CSS grid layout module

CSS box model

The **CSS box model** module defines the `margin` and `padding` properties, which along with the height, width and border properties, make up the CSS box model.

Every visible element on a webpage is a box laid out according to the visual formatting model. CSS properties define their size, position, and stacking level, with the box model properties (and others) defining the extrinsic size of each box, and the space around them.

Each box has a rectangular content area, inside which any text, images, and other content is displayed. The content may be surrounded by padding, a border, and a margin, on one or more sides. The padding is around the content, the border is around the padding, and the margin sits outside the border. The box model describes how these features — the content, padding, border, and margin — work together to create a box as displayed by CSS.



The CSS box model module defines physical (or "page relative") properties such as `margin-top` and `padding-top`. Flow-relative properties such as `margin-block-start` and `margin-inline-start` (which relate to text direction) are defined in Logical Properties and Values. The box model module is extended by the CSS box sizing module, which introduces the intrinsic size value and enables defining aspect ratio for elements that are auto-sized in at least one dimension.

Reference

Properties

- `margin` shorthand
- `margin-bottom`
- `margin-left`
- `margin-right`
- `margin-top`
- `margin-trim`
- `padding` shorthand
- `padding-bottom`
- `padding-left`
- `padding-right`
- `padding-top`

Data types

- `<box-edge>`
  - `<visual-box>`
  - `<layout-box>`
  - `<paint-box>`
  - `<coord-box>`
  - `<geometry-box>`

Guides

Introduction to the CSS box model

Explains one of the fundamental concepts of CSS: the box model. This model defines how CSS lays out elements, including their content, padding, border, and margin areas.

Mastering margin collapsing

Sometimes, two adjacent margins are collapsed into one. This article describes the rules that govern when and why this happens, and how to control it.

Visual formatting model

Explains the visual formatting model.

Related concepts

- CSS backgrounds and borders module
  - `border-width` shorthand
  - `border-bottom-width`
  - `border-left-width`
  - `border-right-width`
  - `border-top-width`
- CSS logical properties module
  - `block-size`
  - `inline-size`
  - `max-block-size`
  - `max-inline-size`
  - `min-block-size`
  - `min-inline-size`
  - `margin-block`
  - `margin-block-end`
  - `margin-block-start`
  - `margin-inline`

    - `margin-inline-end`
    - `margin-inline-start`
    - `padding-block`
    - `padding-block-end`
    - `padding-block-start`
    - `padding-inline`
    - `padding-inline-end`
    - `padding-inline-start`
    - `border-block`
    - `border-block-end`
    - `border-block-end-width`
    - `border-block-start`
    - `border-block-start-width`
    - `border-block-style`
    - `border-block-width`
    - `border-inline`
    - `border-inline-end`
    - `border-inline-end-width`
    - `border-inline-start`
    - `border-inline-start-width`
    - `border-inline-width`
- CSS box sizing module
    - `aspect-ratio`
    - `box-sizing`
    - `contain-intrinsic-block-size`
    - `contain-intrinsic-height`
    - `contain-intrinsic-inline-size`
    - `contain-intrinsic-size`
    - `contain-intrinsic-width`
    - `height`
    - `max-height`
    - `max-width`
    - `min-height`
    - `min-width`
    - `width`
- CSS overflow module
    - `overflow` shorthand
    - `overflow-block`
    - `overflow-clip-margin`
    - `overflow-inline`
    - `overflow-x`
    - `overflow-y`
    - `text-overflow`

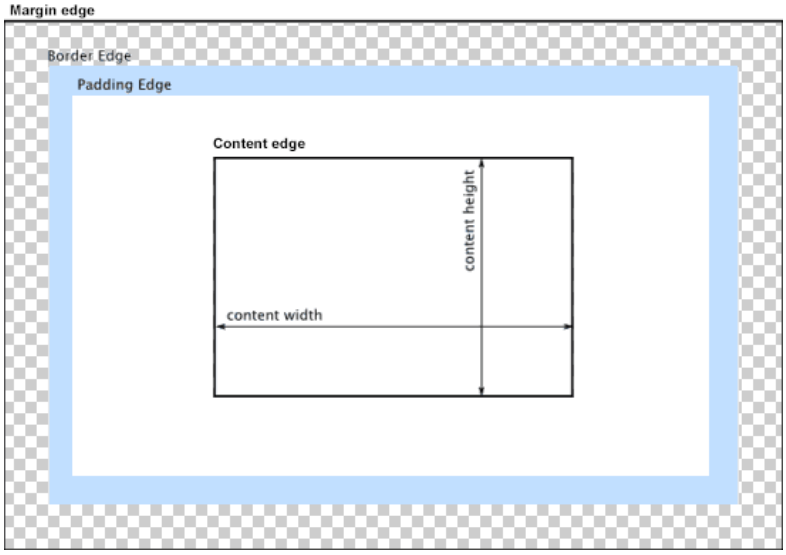
See also

- CSS display module
- CSS flex layout module
- CSS grid layout module
- CSS table module
- CSS positioned layout module
- CSS fragmentation module
- Understanding aspect ratios

Introduction to the CSS box model

When laying out a document, the browser's rendering engine represents each element as a rectangular box according to the standard **CSS basic box model**. CSS determines the size, position, and properties (color, background, border size, etc.) of these boxes.

Every box is composed of four parts (or *areas*), defined by their respective edges: the *content edge*, *padding edge*, *border edge*, and *margin edge*.



Content area

The **content area**, bounded by the content edge, contains the "real" content of the element, such as text, an image, or a video player. Its dimensions are the *content width* (or *content-box width*) and the *content height* (or *content-box height*). It often has a background color or background image.

If the `box-sizing` property is set to `content-box` (default) and if the element is a block element, the content area's size can be explicitly defined with the `width`, `min-width`, `max-width`, `height`, `min-height`, and `max-height` properties.

Padding area

The **padding area**, bounded by the padding edge, extends the content area to include the element's padding. Its dimensions are the *padding-box width* and the *padding-box height*.

The thickness of the padding is determined by the `padding-top`, `padding-right`, `padding-bottom`, `padding-left`, and shorthand `padding` properties.

Border area

The **border area**, bounded by the border edge, extends the padding area to include the element's borders. Its dimensions are the *border-box width* and the *border-box height*.

The thickness of the borders is determined by the `border-width` and shorthand `border` properties. If the `box-sizing` property is set to `border-box`, the border area's size can be explicitly defined with the `width`, `min-width`, `max-width`, `height`, `min-height`, and `max-height` properties. When there is a background (`background-color` or `background-image`) set on a box, it extends to the outer edge of the border (i.e., extends underneath the border in z-ordering). This default behavior can be altered with the `background-clip` CSS property.

Margin area

The **margin area**, bounded by the margin edge, extends the border area to include an empty area used to separate the element from its neighbors. Its dimensions are the *margin box width* and the *margin box height*.

The size of the margin area is determined by the `margin-top`, `margin-right`, `margin-bottom`, `margin-left`, and shorthand `margin` properties. When margin collapsing occurs, the margin area is not clearly defined since margins are shared between boxes.

Finally, note that for non-replaced inline elements, the amount of space taken up (the contribution to the height of the line) is determined by the `line-height` property, even though the borders and padding are still displayed around the content.

See also

- CSS box model module
- Layout and the containing block
- Introducing the CSS Cascade
- Learn: Handling conflicts
- CSS key concepts:
  - CSS syntax
  - At-rules
  - Comments
  - Specificity
  - Inheritance
  - Layout modes
  - Visual formatting model
  - Margin collapsing
  - Values:
    - Initial values
    - Computed values
    - Used values
    - Actual values
  - Value definition syntax
  - Shorthand properties
  - Replaced elements

Mastering margin collapsing

The top and bottom margins of blocks are sometimes combined (collapsed) into a single margin whose size is the largest of the individual margins (or just one of them, if they are equal), a behavior known as **margin collapsing**. Note that the margins of floating and absolutely positioned elements never collapse.

Margin collapsing occurs in three basic cases:

Adjacent siblings

> The margins of adjacent siblings are collapsed (except when the latter sibling needs to be cleared past floats).

No content separating parent and descendants

> The vertical margins between a parent block and its descendants can collapse. This happens when there is no separating content between them. Specifically, this occurs in two main cases:
>
> - The `margin-top` of a parent collapses with the `margin-top` of its first in-flow descendant unless the parent has a `border-top` , `padding-top` , contains any inline content (such as text), or has *clearance* applied.
> - The `margin-bottom` of a parent collapses with the `margin-bottom` of its last in-flow descendant unless the parent has a defined `height` or `min-height`, a `border-bottom`, or `padding-bottom`.
>
> In both cases, creating a new block formatting context on the parent will also prevent its margins from collapsing with its children.

Empty blocks

> If there is no border, padding, inline content, `height`, or `min-height` to separate a block's `margin-top` from its `margin-bottom`, then its top and bottom margins collapse.

Some things to note:

- More complex margin collapsing (of more than two margins) occurs when the above cases are combined.
- These rules apply even to margins that are zero, so the margin of a descendant ends up outside its parent (according to the rules above) whether or not the parent's margin is zero.
- When negative margins are involved, the size of the collapsed margin is the sum of the largest positive margin and the smallest (most negative) negative margin.
- When all margins are negative, the size of the collapsed margin is the smallest (most negative) margin. This applies to both adjacent elements and nested elements.
- Collapsing margins is only relevant in the vertical direction.
- Margins don't collapse in a container with `display` set to `flex` or `grid`.

Examples

HTML

```
<p>The bottom margin of this paragraph is collapsed …</p>
<p>
```

```
  … with the top margin of this paragraph, yielding a margin of
  <code>1.2rem</code> in between.
</p>

<div>
  This parent element contains two paragraphs!
  <p>
    This paragraph has a <code>.4rem</code> margin between it and the text
    above.
  </p>
  <p>
    My bottom margin collapses with my parent, yielding a bottom margin of
    <code>2rem</code>.
  </p>
</div>

<p>I am <code>2rem</code> below the element above.</p>
```

CSS

```
div {
  margin: 2rem 0;
  background: lavender;
}

p {
  margin: 0.4rem 0 1.2rem 0;
  background: yellow;
}
```

Result

See also

- CSS box model module
- Introduction to the CSS box model
- CSS key concepts:
  - CSS syntax
  - At-rules
  - Comments
  - Specificity
  - Inheritance
  - Layout modes
  - Visual formatting model
  - Values:
    - Initial values
    - Computed values
    - Used values
    - Actual values
  - Value definition syntax
  - Shorthand properties
  - Replaced elements

CSS box sizing

The **CSS box sizing** module enables you to specify how elements fit their content or fit into a particular layout context. It defines sizing, minimum sizing, and maximum sizing properties, and also extends the CSS sizing properties with keywords that represent content-based intrinsic size and context-based extrinsic size.

Elements can either be extrinsically or intrinsically sized. The CSS box model defines page-relative properties to explicitly, or "extrinsically" set an element's size, including `width`, `height`, `padding`, and `margin` properties (along with `border` properties defined in the CSS backgrounds and borders module). This CSS box sizing module extends the CSS box model module to enable an element to be sized intrinsically — setting element size based on the size of its content.

The sizing values introduced in this module allow elements with size containment to take explicit intrinsic sizes, as if their in-flow content's width and height match the specified explicit intrinsic size, rather than being sized as if they were empty.

This module also introduced the ability to define an aspect ratio for an element's box, meaning the browser can automatically adjust an element's dimensions to maintain a specified aspect ratio as long as one of the dimensions is automatically sized.

The logical properties and values module expanded the properties available in the box model and box sizing modules to include writing-mode-relative equivalents of the corresponding physical box model and intrinsic box sizing properties.

Reference

Properties

- `aspect-ratio`
- `box-sizing`
- `contain-intrinsic-block-size`
- `contain-intrinsic-height`
- `contain-intrinsic-inline-size`
- `contain-intrinsic-size`
- `contain-intrinsic-width`
- `frame-sizing`
- `height`
- `max-height`
- `max-width`
- `min-height`
- `min-width`
- `width`

The CSS box sizing module also introduces the `min-intrinsic-sizing` property. Currently, no browsers support this feature.

Data types and values

- `<ratio>` data type

- `min-content` value
- `max-content` value
- `fit-content` value

Functions

- `fit-content()`

Glossary terms

- Intrinsic size
- Extrinsic size

Guides

Understanding aspect ratios

Learn about the `aspect-ratio` property, discuss aspect ratios for replaced and non-replaced elements, and examine some common aspect ratio use cases.

Introduction to the CSS box model

Explains one of the fundamental concepts of CSS: the box model. This model defines how CSS lays out elements, including their content, padding, border, and margin areas.

Mastering margin collapsing

Sometimes, two adjacent margins are collapsed into one. This article describes the rules that govern when and why this happens, and how to control it.

Visual formatting model

Explains the visual formatting model.

Controlling ratios of flex items along the main axis

Explains intrinsic sizing as a precursor to understanding how to control the size and flexibility of flex items along the main axis using `flex-grow`, `flex-shrink`, and `flex-basis`.

Related concepts

- CSS logical properties module
  - `min-inline-size`
  - `block-size`
  - `inline-size`
  - `max-block-size`
  - `max-inline-size`
  - `min-block-size`
  - `min-inline-size`
  - `margin-block`
  - `margin-inline`
  - `padding-block`
  - `padding-inline`
  - `border-block`
  - `border-inline`
  - `contain-intrinsic-block-size`
  - `contain-intrinsic-inline-size`
  - `overflow-block`
  - `overflow-inline`
  - `overscroll-behavior-block`
  - `overscroll-behavior-inline`
- CSS box model module
  - `margin` shorthand
  - `margin-bottom`
  - `margin-left`
  - `margin-right`
  - `margin-top`
  - `margin-trim`
  - `padding` shorthand
  - `padding-bottom`
  - `padding-left`
  - `padding-right`
  - `padding-top`
- CSS backgrounds and borders module
  - `border` shorthand
  - `border-width` shorthand
  - `border-bottom-width`
  - `border-left-width`
  - `border-right-width`
  - `border-top-width`
- CSS overflow module
  - `overflow` shorthand
  - `overflow-block`
  - `overflow-clip-margin`
  - `overflow-inline`
  - `overflow-x`
  - `overflow-y`
  - `text-overflow`
- CSS grid layout module
  - `grid`
  - `grid-auto-columns`
  - `grid-auto-rows`
  - `grid-template-columns`
  - `grid-template-rows`
  - `repeat()`
  - `minmax()` function

- CSS flexible box layout module
  - `flex-basis`
  - `flex`

See also

- CSS display module
- CSS flex layout module
- CSS grid layout module
- CSS positioned layout module
- CSS fragmentation module

Understanding and setting aspect ratios

Every element rendered to the page has a height and a width, and, therefore, an aspect ratio, which is the ratio between the width and height. The natural dimensions of a media object, which are its size without any sizing, scaling, zooming, or borders applied, are known as its natural or intrinsic size. An element's intrinsic size is determined by the element itself, not by applying formatting such as box sizing or setting border, margin, or padding widths.

When developing sites, you often want to be able to set the width of an element to a percentage of the viewport or parent container size and have the height change size proportionally, thereby maintaining a specific aspect ratio depending on the size of the viewport. For replaced elements, like images and videos, maintaining a specific aspect ratio is not only necessary for creating responsive web design, but also a vital component of providing good user experience. Setting an asset's aspect ratio prevents loading jank—the layout shift that occurs when media loads after the page has already been painted, causing a reflow because the space for the asset has not been reserved.

Using CSS, you can adjust the size of replaced and non-replaced elements based on their aspect ratio. In this guide, we will learn about the `aspect-ratio` property, discuss aspect ratios for replaced and non-replaced elements, and then examine some common aspect ratio use cases.

How the <code>aspect-ratio</code> property works

The CSS `aspect-ratio` property value defines the preferred width-to-height ratio of an element's box. The value is either a `<ratio>`, the keyword `auto`, or a space-separated combination of both.

The `<ratio>` is the ratio of the width and height, in that order. It is represented by two positive `<number>` values separated by a forward slash (`/`) or a single `<number>`. When a single number is used, it is the same as writing the ratio as `<number> / 1`, which is also the width divided by the height.

The following values are all equivalent:

```
aspect-ratio: 3 / 6;
aspect-ratio: 1 / 2;
aspect-ratio: 0.5 / 1;
aspect-ratio: 0.5;
```

The following values are also all equivalent:

```
aspect-ratio: 9/6;
aspect-ratio: 3/2;
aspect-ratio: 1.5;
```

```
<div>0.5</div>
<div>1.5</div>
```

```
div {
  height: 121px;
  aspect-ratio: 0.5;
  background-color: pink;
  line-height: 100px;
  text-align: center;
  float: left;
  background-image:
    repeating-linear-gradient(to right, black 0px 1px, transparent 1px 20px),
    repeating-linear-gradient(black 0px 1px, transparent 1px 20px);
  background-size:
    181px 5px,
    5px 121px;
  background-repeat: no-repeat;
}

div + div {
  aspect-ratio: 1.5;
  background-color: lightgreen;
  margin-left: 10px;
}
```

The effect of the `auto` keyword depends on whether the element on which it is applied is a replaced element or not. For replaced elements with an intrinsic aspect ratio, `auto` means the intrinsic aspect ratio should be used. In all other instances, the `auto` value means the box has no preferred aspect ratio. In both cases, this is the default behavior as if no `aspect-ratio` property were applied.

When the value contains both the `auto` keyword and a `<ratio>` value, such as `aspect-ratio: auto 2 / 3;` or `aspect-ratio: 0.75 auto;`, the `auto` value is applied to replaced elements with a natural aspect ratio and the specified ratio of the `width / height` or `<number>` is used as the preferred aspect ratio.

You will have noticed the word "preferred" in the definitions above. The `aspect-ratio` value is not always applied when set. The `aspect-ratio` property sets a "preferred" aspect ratio, so only ever has an effect if at least one of the box's sizes is automatic.

When both the height and width or inline and block sizes are explicitly set, the `aspect-ratio` property value is ignored. In this case, no dimension is allowed to be automatically sized - the preferred sizes are explicitly set - so the `aspect-ratio` property has no effect. When you declare both the inline and block dimensions, those take precedence.

With replaced elements, if you don't explicitly set a value (other than `auto`) to either dimension, both will default to their intrinsic size (any `aspect-ratio` value isn't applied). The `aspect-ratio` will apply to non-replaced elements that don't have a dimension explicitly set, as non-replaced elements are either intrinsically or extrinsically sized, getting their size from their content, container, box model properties, etc.

When an element is rendered to the page, if no CSS is applied and no HTML sizing attributes are included, the user agent will render the object at its natural size.

Adjusting aspect ratios of replaced elements

Replaced elements like `<img>` and `<video>` are replaced with media that have set dimensions and, therefore, an intrinsic aspect ratio. Consider a raster image, such as a JPEG, PNG, or GIF. If you place an image on a page and do not set a height or width, either via `<img>` attributes or with CSS, it will be displayed at its intrinsic size.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg?image=good"
  alt="220 pixel square pride flag" />
```

This is a `220px` square image with no CSS applied; it is displayed at its intrinsic or default size.

If replaced content is auto-sized or you provide a size for only one dimension, such as setting a value for `width`, the browser will automatically resize the other dimension, in this case, the height, while maintaining the media's original aspect ratio.

In this example, only the `width` is set on the image, so the user agent preserves its aspect ratio. The same image is repeated three times, displayed at different widths: `55px`, `110px`, and at its natural size of `220px` via the `width: auto` value.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

```
img {
  width: 55px;
  margin-right: 5px;
}

img + img {
  width: 110px;
}

img + img + img {
  width: auto;
}
```

Only when you provide sizes for both dimensions is there a risk of distorting the replaced element. For example, setting `width: 100vw;` and `height: 100vh;` on an image creates a variable aspect ratio; the image will appear either stretched or squashed when the viewport's aspect ratio differs from the image's natural aspect ratio.

In this example, the same image is repeated three times, explicitly sized with the same `height` value (`110px`) but different `width` values (`55px`, `110px`, and `220px`).

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

```
img {
  width: 55px;
  height: 110px;
}

img + img {
  width: 110px;
}

img + img + img {
  width: 220px;
}
```

We have distorted the images intentionally by setting both a `height` and `width`: we've squashed the first one and stretched the third.

We could have created this same distorted effect using the CSS `aspect-ratio` property, by setting a single dimension (not both or neither) and providing a value other than `1` (or `1 / 1`). You likely don't want to do this, but it's good to know that it's possible.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

```
img {
  height: 90vh;
  aspect-ratio: 3;
}
```

We have declared a single dimension; `100vh` is the full height of the example `<iframe>` viewport. For `aspect-ratio` to apply to replaced elements, only one dimension must be set. Setting both or neither doesn't work.

Fitting replaced elements within their containers

To fit a replaced element to the dimensions of its container while maintaining its intrinsic aspect ratio, set the `object-fit` property value to `cover` or `contain`. This will resize the replaced element and either clip it to "cover" the container or display it at a smaller size, fully "contained" within it.

In this example, the square image is placed into a grid of three items, each with an aspect ratio of `5 / 2`.

To begin with, we create a container with three items, each containing one image:

```
<div class="grid">
  <div>
    <img
      src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
      alt="Pride flag" />
  </div>
  <div>
    <img
      class="cover"
      src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
      alt="Pride flag" />
  </div>
  <div>
```

```
    <img
      class="contain"
      src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
      alt="Pride flag" />
  </div>
</div>
```

Next, we designate the container as a grid, where each item has an aspect ratio of `2.5` (`5/2`) with a minimum width of `150px`. Therefore, the minimum height will be `60px`. However, the final width and height are determined by the width of the example's iframe, which will be based on your viewport size:

```
.grid {
  display: grid;
  gap: 20px;
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
  font-size: 0; /* to minimize whitespace */
}

div div {
  aspect-ratio: 5 / 2;
  background-color: #cccccc;
}
```

We then size the images and set the `object-fit` property on the last two images:

```
img {
  height: 100%;
  width: 100%;
}

.cover {
  object-fit: cover;
}

.contain {
  object-fit: contain;
}
```

Only the first image is distorted (stretched). We could have used the `fill` value of `object-fit` to create the same effect. The `cover` image spans the full width of the container, centered vertically, and clipped to fit in the container. The `contain` value ensures the image is contained within the container, centered horizontally, and shrunk to fit.

Defining aspect ratios for non-replaced elements

While the aspect ratio of a replaced element is maintained by default, adjusting the intrinsic size of a non-replaced element usually changes its aspect ratio. For example, identical content may appear as three lines on a widescreen or in a wide parent container, but as eight lines on a narrow screen or container.

In this example, the same quote is displayed in `200px` and `600px` wide containers, and a square is set with a height matching its `200px` width:

```
<p>Narrow:</p>
<blockquote>
  <q
    >When you stop living your life based on what others think of you real life
    begins. At that moment, you will finally see the door of self acceptance
    opened.</q
  >
  - Shannon L. Alder
</blockquote>
<p>Wide:</p>
<blockquote>
  <q
    >When you stop living your life based on what others think of you real life
    begins. At that moment, you will finally see the door of self acceptance
    opened.</q
  >
  - Shannon L. Alder
</blockquote>
<p>Aspect ratio with overflow:</p>
<blockquote>
  <q
    >When you stop living your life based on what others think of you real life
    begins. At that moment, you will finally see the door of self acceptance
    opened.</q
  >
  - Shannon L. Alder
</blockquote>
<p>
  <label
    ><input type="checkbox" checked /> Toggle between
    <code>overflow</code> values <code>auto</code> and
    <code>visible</code></label
  >
</p>
```

To highlight the issue with setting a non-replaced element's aspect ratio via size dimensions, toggle the `overflow` property between `auto` and `visible`.

```
blockquote {
  border: 3px dotted #cccccc;
  padding: 0 3px;
  margin: 20px 0;
  font-size: 1.25rem;
  line-height: 1.5;
}

body:has(:checked) blockquote {
  overflow: auto;
}

:checked + code,
:not(:checked) + code + code {
  outline: 1px solid green;
}
```

```
p:nth-last-of-type(n + 2) {
  font-weight: bold;
}
```

```
blockquote {
  width: 200px;
}

blockquote:nth-of-type(2) {
  width: 600px;
}

blockquote:nth-of-type(3) {
  height: 200px;
}
```

While it's possible to define an aspect ratio on non-replaced elements by setting both the dimensions and hiding overflowing content, the CSS `aspect-ratio` property provides explicit aspect ratio support. This means a specific aspect ratio can be set even when you don't know the content or screen sizes.

In the next example, we render square boxes regardless of the width of the text by setting `aspect ratio: 1` on `<blockquote>`, a non-replaced element:

```
<p>Short text:</p>
<blockquote>Breath.</blockquote>
<p>Longer text:</p>
<blockquote>You're perfect just as you are.</blockquote>
```

```
blockquote {
  inline-size: max-content;
  aspect-ratio: 1;
}
```

```
blockquote {
  border: 1px solid #cccccc;
  padding: 1px;
  margin: 20px 0;
  background-color: #ededed;
}
```

Each box has one dimension defined: the `inline-size`, which is the width in horizontal languages, is set to `max-content`, which sets the size to be as wide as it needs to be to fit the content without wrapping. The second dimension, in this case, the `block-size` or `height`, is set to be the same length as the first dimension. This is accomplished with the `aspect-ratio` property. We defined the desired width-to-height ratio of the element's box to be `1`, which is the same as `1 / 1`, a square. This sets the block direction to match the width of the element, without using the `height` or `block-size` properties.

In these examples, a size was explicitly set on the element itself. When working with non-replaced elements, aspect ratio comes into play when no size dimension is explicitly set.

Creating a circle based on the container size

The inline-size of non-replaced block-level elements is the size of their container's content box. Because they have a size by default, they don't need to have an explicit size set for the `aspect-ratio` property to work.

In this example, we have a container `<div>` that is `200px` wide, which includes `5px` of padding on each side. Therefore, the inline-size of the content box is `190px`. Without setting a height or width on the nested `<p>` element, we know its inline-size is `190px`. With `aspect-ratio: 1` set, the paragraph will be `190px` tall, unless it has visible overflowing content causing it to be taller (which it doesn't).

The height of the `<div>` element is not explicitly set, but it contains the `190px` tall paragraph, the `5px` of padding on top and bottom, and the combined heights of the default top and bottom margins of `<p>`. As a result, it is taller than it is wide. Both elements have a `border-radius` of `50%`, so the container is an oval while the child, with an `aspect-ratio` of `1` but no inline or block sizing explicitly defined, is a circle.

```
<div><p>Hello world</p></div>
```

```
div,
p {
  border-radius: 50%;
}

div {
  width: 200px;
  padding: 5px;
  border: 1px solid black;
  background-color: #66ccff;
}

p {
  aspect-ratio: 1;
  text-align: center;
  border: 10px solid white;
  background-color: #f4aab9;
}
```

To make the `<div>` a circle, we can set the `height` and `width` to the same value, or set `aspect-ratio: 1` and set the `overflow` to `auto` or `hidden`. Alternatively, we can simply remove the margins on the paragraph with `margin-block: 0`. Both these options are shown below.

```
<div><p>Hello world</p></div>
<div><p>Hello world</p></div>
```

```
div {
  width: 200px;
  padding: 5px;
  margin: 1rem;
  border: 1px solid black;
  background-color: #66ccff;
}

p {
  text-align: center;
  border: 10px solid white;
  background-color: #f4aab9;
}
```

```
div,
p {
  aspect-ratio: 1;
  border-radius: 50%;
}

div:first-of-type {
  overflow: hidden;
}

div:last-of-type p {
  margin-block: 0;
}
```

Common <code>aspect-ratio</code> use cases

Let's look at a few situations where you can use `aspect-ratio` to address some common challenges while creating responsive designs.

Making external assets responsive

All content should be responsive, even when that content is third-party embeds, such as videos from TikTok, YouTube, or Instagram. The code snippet you include to embed these external videos generally creates an `<iframe>`.

While a `<video>` element typically adopts the aspect ratio of its media file, `iframe` elements lack this capability. This poses the challenge of ensuring that the `<iframe>` is responsive while always maintaining the aspect ratio of the video it contains. One of the techniques we can use is to set the iframe's width to `100%` of its container or `100vw` to match the viewport width regardless of the viewport's size. However, setting a fixed height might stretch or squash the video. Instead, we set the `aspect-ratio` on the video's container, aligning it to be the same aspect ratio as the video. Problem solved!

For context, the standard aspect ratio of YouTube videos is 16:9 when viewed on a desktop computer or laptop, while TikTok and Instagram videos have a 9:16 aspect ratio.

```
.youtube {
  aspect-ratio: 16/9;
}

.instagram,
.tiktok {
  aspect-ratio: 9/16;
}
```

We can use the `aspect-ratio` feature within the `@media` query alongside the `aspect-ratio` property to adjust the size of both the iframe and the video it contains. This ensures that the video content is always as large as possible - taking up either the full width or height of the viewport, regardless of the viewport size - while maintaining a specific aspect ratio.

We can set the landscape-oriented YouTube videos to be as wide as the viewport and the portrait-oriented TitTok and Instagram video iframes to be as tall as the viewport. If a viewport's aspect ratio is wider than 16:9, we set the YouTube video to be the height of the viewport. If the viewport is narrower than 9:16, we set both Instagram and TikTok videos to the width of the viewport.

```
iframe.youtube {
  aspect-ratio: 16/9;
  width: 100vw;
  height: auto;
}

iframe.instagram,
iframe.tiktok {
  aspect-ratio: 9/16;
  height: 100vh;
  width: auto;
}

/* If the viewport is very wide but not very tall */
@media (aspect-ratio > 16 / 9) {
  iframe.youtube {
    width: auto;
    height: 100vh;
  }
}

/* If the viewport is very tall but not very wide */
@media (aspect-ratio < 9 / 16) {
  iframe.instagram,
  iframe.tiktok {
    height: auto;
    width: 100vw;
  }
}
```

Making grid cells square

A grid of square cells can be created by defining fixed column track sizes, ensuring each row matches the size of the column track. However, when creating responsive grids using `auto-fill` to fit as many column tracks as possible within the container, the width of each item becomes uncertain. This makes it challenging to determine the appropriate height for creating square items.

By setting an aspect ratio on the items, we can ensure when the grid items are laid out, each grid item will be as tall as it is wide, creating square grid items regardless of the container's dimensions.

In this example of square grid items, the grid tracks are auto-sized, taking their size from the items. Each item will be at least `95px` wide but could be much wider. No matter the width, each item will be a square, with the height determined by the `aspect-ratio` to match its width.

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(95px, 1fr));
}

.item {
  aspect-ratio: 1;
}
```

```
div {
  gap: 20px;
  font-size: 1.1rem;
}

div div {
```

```
  background-color: #cccccc;
  aspect-ratio: 1;
  counter-increment: items;
}

div div::after {
  content: counter(items);
}

.item::after {
  /*  hide the number if there's content  */
  position: absolute;
  color: transparent;
}
```

```
<div class="grid">
  <div></div>
  <div></div>
  <div class="item">
    This text would overflow the parent, but we've set properties to prevent it
    from doing so.
  </div>
  <div></div>
  <div></div>
  <div></div>
  <div class="item">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
      alt="Pride flag" />
  </div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

For the content of a grid item to not grow beyond the preferred height set by the `aspect-ratio`, set the `min-height` to `0` and the `overflow` to a value other than `visible`. This will work for intrinsically sized content. If you have content that is intrinsically larger than the available space, set that content to not be larger than the grid item by setting the `max-height` (or `max-width`, depending on the content) to `100%`.

```
.item {
  min-height: 0;
  overflow: auto;
}

.item > * {
  max-height: 100%;
}
```

See also

- CSS box sizing module

CSS cascading and inheritance

The **CSS cascading and inheritance** module defines the rules for assigning values to properties by way of cascading and inheritance. This module specifies the rules for finding the specified value for all properties on all elements.

One of the fundamental design principles of CSS is cascading of rules. It allows several style sheets to influence the presentation of a document. CSS property-value declarations define how a document is rendered. Multiple declarations may set different values for the same element and property combination, but only one value can be applied to any CSS property. The CSS cascade module defines how these conflicts are resolved.

The opposite also occurs. Sometimes there are no declarations defining the value of a property. The CSS cascade module defines how these missing values should be set via inheritance or from the property's initial value.

> **Note:** The rules for finding the specified values in the page context and its margin boxes are described in the CSS page module.

Reference

Properties

- `all`

At-rules and descriptors

- `@import`
- `@layer`

Keywords

- `initial`
- `inherit`
- `revert`
- `revert-layer`
- `unset`
- `!important` flag

Interfaces

- `CSSLayerBlockRule`
- `CSSGroupingRule`
- `CSSLayerStatementRule`
- `CSSRule`

Glossary terms and definitions

- Actual value
- Anonymous layer
- Author origin
- Cascade
- Computed value
- Initial value
- Named layer
- Resolved value
- Shorthand properties
- Specificity
- Specified value
- style origin
- Used value
- User origin
- User-agent origin

Guides

Introducing the CSS Cascade

Guide to the cascade algorithm that defines how user agents combine property values originating from different sources.

CSS inheritance

A guide to CSS inheritance.

Learn: Handling conflicts

The most fundamental concepts of CSS — the cascade, specificity, and inheritance — which control how CSS is applied to HTML and how conflicts are resolved.

Learn: Cascade layers

Introduction to cascade layers, a more advanced feature that builds on the fundamental concepts of the CSS cascade and CSS specificity.

Related concepts

- Element-attached styles
- Inline styles and the cascade
- Conditional rules for @imports
- Value definition syntax

See also

- CSS selectors module
- CSS pseudo-elements module
- CSS paged media module
- CSS conditional rules module
- CSS nesting module
- Shorthand properties

Inheritance

In CSS, **inheritance** controls what happens when no value is specified for a property on an element.

CSS properties can be categorized in two types:

- **inherited properties**, which by default are set to the computed value of the parent element
- **non-inherited properties**, which by default are set to initial value of the property

Refer to any CSS property definition to see whether a specific property inherits by default ("Inherited: yes") or not ("Inherited: no").

Inherited properties

When no value for an **inherited property** has been specified on an element, the element gets the computed value of that property on its parent element. Only the root element of the document gets the initial value given in the property's summary.

A typical example of an inherited property is the `color` property. Consider the following style rules and the markup:

```
p {
  color: green;
}
```

```
<p>This paragraph has <em>emphasized text</em> in it.</p>
```

The words "emphasized text" will appear green, since the `em` element has inherited the value of the `color` property from the `p` element. It does *not* get the initial value of the property (which is the color that is used for the root element when the page specifies no color).

Non-inherited properties

When no value for a **non-inherited property** has been specified on an element, the element gets the initial value of that property (as specified in the property's summary).

A typical example of a non-inherited property is the `border` property. Consider the following style rules and the markup:

```
p {
  border: medium solid;
}
```

```
<p>This paragraph has <em>emphasized text</em> in it.</p>
```

The words "emphasized text" will not have another border (since the initial value of `border-style` is `none`).

Notes

The `inherit` keyword allows authors to explicitly specify inheritance. It works on both inherited and non-inherited properties.

You can control inheritance for all properties at once using the `all` shorthand property, which applies its value to all properties. For example:

```
p {
  all: revert;
  font-size: 200%;
  font-weight: bold;
}
```

This reverts the style of the paragraphs' `font` property to the user agent's default unless a user stylesheet exists, in which case that is used instead. Then it doubles the font size and applies a `font-weight` of `"bold"`.

Overriding inheritance, an example

Using our previous example with `border`, if we explicitly set the inheritance with `inherit`, we get the following:

```
p {
  border: medium solid;
}

em {
  border: inherit;
}
```

```
<p>This paragraph has <em>emphasized text</em> in it.</p>
```

We can see here another border around the word "emphasized text".

See also

- CSS values for controlling inheritance: `inherit`, `initial`, `revert`, `revert-layer`, and `unset`
- CSS cascading and inheritance module
- CSS error handling
- Introduction to the CSS cascade
- Learn: Handling conflicts
- Learn: Cascade layers
- Introduction to CSS syntax: declarations, rulesets, and statements
- At-rules
- Values: initial, computed, used, and actual
- Value definition syntax
- CSS nesting module

Introduction to the CSS cascade

The **cascade** is an algorithm that defines how user agents combine property values originating from different sources. The cascade defines the origin and layer that takes precedence when declarations in more than one origin, cascade layer, or `@scope` block set a value for a property on an element.

The cascade lies at the core of CSS, as emphasized by the name: ***Cascading*** Style Sheets. When a selector matches an element, the property value from the origin with the highest precedence gets applied, even if the selector from a lower precedence origin or layer has greater specificity.

This article explains what the cascade is and the order in which CSS declarations cascade, covering cascade layers and origin type. Understanding origin precedence is key to understanding the cascade.

Origin types

The CSS cascade algorithm's job is to select CSS declarations in order to determine the correct values for CSS properties. CSS declarations come from different origin types: **User-agent stylesheets**, **Author stylesheets**, and **User stylesheets**.

Though stylesheets come from these different origins and can be within different layers in each of these origins, they overlap in terms of their default scope; to make this work, the cascade algorithm defines how they interact. Before addressing the interactions, we'll define some key terms in the next few sections.

User-agent stylesheets

User-agents, or browsers, have basic stylesheets that give default styles to any document. These stylesheets are named **user-agent stylesheets**. Most browsers use actual stylesheets for this purpose, while others simulate them in code. The end result is the same.

Some browsers let users modify the user-agent stylesheet, but this is rare and not something that can be controlled.

Although some constraints on user-agent stylesheets are set by the HTML specification, browsers have a lot of latitude: that means some differences exist between browsers. To simplify the development process, Web developers may use a CSS reset stylesheet, such as normalize.css, which sets common properties values to a known state for all browsers before beginning to make alterations to suit their specific needs.

Unless the user-agent stylesheet includes an `!important` next to a property, making it "important", styles declared by author styles, including a reset stylesheet, take precedence over the user-agent styles, regardless of the specificity of the associated selector.

Author stylesheets

Author stylesheets are the most common type of stylesheet; these are the styles written by web developers. These styles can reset user-agent styles, as noted above, and define the styles for the design of a given web page or application. The author, or web developer, defines the styles for the document using one or more linked or imported stylesheets, `<style>` blocks, and inline styles defined with the `style` attribute. These author styles define the look and feel of the website — its theme.

User stylesheets

In most browsers, the user (or reader) of the website can choose to override styles using a custom **user stylesheet** designed to tailor the experience to the user's wishes. Depending on the user agent, user styles can be configured directly or added via browser extensions.

Cascade layers

The cascade order is based on origin type. The cascade within each origin type is based on the declaration order of cascade layers within that type. For all origins - user-agent, author, or user - styles can be declared within or outside of named or anonymous layers. When declared using `layer`, `layer()` or `@layer`, styles are placed into the specified named layer, or into an anonymous layer if no name is provided. Styles declared outside of a layer are treated as being part of an anonymous last declared layer.

Let's take a look at cascading origin type before diving into cascade layers within each origin type.

Cascading order

The cascading algorithm determines how to find the value to apply for each property for each document element. The following steps apply to the cascading algorithm:

1. **Relevance**: It first filters all the rules from the different sources to keep only the rules that apply to a given element. That means rules whose selector matches the given element and which are part of an appropriate `media` at-rule.

2. **Origin and importance**: Then it sorts these rules according to their importance, that is, whether or not they are followed by `!important`, and by their origin. Ignoring layers for the moment, the cascade order is as follows:

Precedence Order (low to high)	Origin	Importance
1	user-agent (browser)	normal
2	user	normal
3	author (developer)	normal
4	CSS keyframe animations	
5	author (developer)	`!important`
6	user	`!important`
7	user-agent (browser)	`!important`
8	CSS transitions	

3. **Specificity**: In case of equality with an origin, the specificity of a rule is considered to choose one value or another. The specificity of the selectors are compared, and the declaration with the highest specificity wins.
4. **Scoping proximity**: When two selectors in the origin layer with precedence have the same specificity, the property value within scoped rules with the smallest number of hops up the DOM hierarchy to the scope root wins. See How `@scope` conflicts are resolved for more details and an example.
5. **Order of appearance**: In the origin with precedence, if there are competing values for a property that are in style block matching selectors of equal specificity and scoping proximity, the last declaration in the style order is applied.

The cascade is in ascending order, meaning:

- Animations take precedence over normal values, whether declared in user, author, or user-agent styles.
- Important values take precedence over animations, whether declared in user, author, or user-agent styles.
- Transitions take precedence over important values.

> **Note: Transitions and animations**
>
> Property values set by animation `@keyframes` are more important than all normal styles (those with no `!important` set).
>
> Property values being set in a `transition` take precedence over all other values set, even those marked with `!important`.

The cascade algorithm is applied *before* the specificity algorithm, meaning if `:root p { color: red;}` is declared in the user stylesheet (row 2) and a less specific `p {color: blue;}` is in the author stylesheet (row 3), the paragraphs will be blue.

Basic example

Before taking a deeper look at how cascade layers impact the cascade, let's look at an example involving multiple sources of CSS across the various origins, and work through the steps of the cascade algorithm:

Here we have a user agent stylesheet, two author stylesheets, and a user stylesheet, with no inline styles within the HTML:

User-agent CSS:

```
li {
  margin-left: 10px;
}
```

Author CSS 1:

```
li {
  margin-left: 0;
} /* This is a reset */
```

Author CSS 2:

```
@media screen {
  li {
    margin-left: 3px;
  }
}

@media print {
  li {
    margin-left: 1px;
  }
}

@layer namedLayer {
  li {
    margin-left: 5px;
  }
}
```

User CSS:

```
.specific {
  margin-left: 1em;
}
```

HTML:

```
<ul>
  <li class="specific">1<sup>st</sup></li>
  <li>2<sup>nd</sup></li>
</ul>
```

In this case, declarations inside `li` and `.specific` rules should apply.

Once again, there are five steps in the cascade algorithm, in order:

1. Relevance
2. Origin and importance
3. Specificity
4. Scoping proximity
5. Order of appearance

The `1px` is for print media. Due to lack of *relevance* based on its media type, it is removed from consideration.

No declaration is marked as `!important`, so the precedence order is author stylesheets over user stylesheets over user-agent stylesheet. Based on *origin and importance*, the `1em` from the user stylesheet and the `10px` from the user-agent stylesheet are removed from consideration.

Note that even though the user style on `.specific` of `1em` has a higher specificity, it's a normal declaration in a user stylesheet. As such, it has a lower precedence than any author styles, and gets removed by the origin and importance step of the algorithm before specificity even comes into play.

There are three declarations in author stylesheets:

```
li {
  margin-left: 0;
} /* from author css 1 */
```

```
@media screen {
  li {
    margin-left: 3px;
  }
}
```

```
@layer namedLayer {
  li {
    margin-left: 5px;
  }
}
```

The last one, the `5px` is part of a cascade layer. Normal declarations in layers have lower precedence than normal styles not in a layer within the same origin type. This is also removed by step 2 of the algorithm, *origin and importance*.

This leaves the `0` and the `3px`, which both have the same selector, hence the same *specificity*. Neither of them are inside a `@scope` block, so scoping proximity does not come into play in this example either.

We then look at *order of appearance*. The second one, the last of the two unlayered author styles, wins.

```
margin-left: 3px;
```

> **Note:** The declaration defined in the user CSS, while it may have greater specificity, is not chosen as the cascade algorithm's *origin and importance* is applied before the *specificity* algorithm. The declaration defined in a cascade layer, though it may come later in the code, will not take precedence either as normal styles in cascade layers have less precedence than normal unlayered styles. *Order of appearance* only matters when both origin, importance, and specificity are equal.

Author styles: inline styles, layers, and precedence

The table in Cascading order provided a precedence order overview. The table summarized the user-agent, user, and author origin type styles in two lines each with "origin type - normal" and "origin type - !important". The precedence within each origin type is more nuanced. Styles can be contained within layers within their origin type, and, with author styles, there is also the issue of where inline styles land in the cascade order.

The order in which layers are declared is important in determining precedence. Normal styles in a layer take precedence over styles declared in prior layers; with normal styles declared outside of any layer taking precedence over normal layered styles regardless of specificity.

In this example, the author used CSS's `@import` rule to import five external stylesheets within a `<style>` information element.

```
<style>
  @import "unlayeredStyles.css";
  @import "AStyles.css" layer(A);
  @import "moreUnlayeredStyles.css";
  @import "BStyles.css" layer(B);
  @import "CStyles.css" layer(C);
  p {
    color: red;
    padding: 1em !important;
  }
</style>
```

and then in the body of the document we have inline styles:

```
<p style="line-height: 1.6em; text-decoration: overline !important;">Hello</p>
```

In the CSS code block above, three cascade layers named "A", "B", and "C", were created, in that order. Three stylesheets were imported directly into layers and two were imported without creating or being assigned to a layer. The "All unlayered styles" in the list below (normal author style precedence - order 4) includes styles from these two stylesheets and the additional unlayered CSS style blocks. In addition, there are two inline styles, a normal `line-height` declaration and an important `text-decoration` declaration:

Precedence Order (low to high)	Author style	Importance
1	A - first layer	normal
2	B - second layer	normal
3	C - last layer	normal
4	All unlayered styles	normal
5	inline `style`	normal
6	animations	
7	All unlayered styles	`!important`
8	C - last layer	`!important`
9	B - second layer	`!important`
10	A - first layer	`!important`

Precedence Order (low to high)	Author style	Importance
11	inline `style`	`!important`
12	transitions	

In all origin types, normal styles contained in layers have the lowest precedence. In our example, the normal styles associated with the first declared layer (A) have lower precedence than normal styles in the second declared layer (B), which have lower precedence than normal styles in the third declared layer (C). These layered styles have lower precedence than all normal unlayered styles, which includes normal styles from `unlayeredStyles.css`, `moreUnlayeredStyles.css`, and the `color` of `p` in the `<style>` itself.

If any layered styles in A, B, or C have selectors with higher specificity matching an element, similar to `:root body p { color: black; }`, it doesn't matter. Those declarations are removed from consideration because of *origin*; normal layered styles have less precedence than normal unlayered styles. If, however, the more specific selector `:root body p { color: black; }` was found in `unlayeredStyles.css`, as both *origin and importance* have the same precedence, *specificity* would mean the more specific, black declaration would win.

The layer order of precedence is inverted for styles declared as `!important`. Important declarations found in a layer take precedence over important declarations found outside of a layer. Important declarations found in the first layer (A) take precedence over important declarations found in layer B, which take precedence over important declarations found in layer C, which take precedence over important declarations found outside of a layer.

Inline styles

Only relevant to author styles are inline styles, declared with the `style` attribute. Normal inline styles take precedence over any other normal author styles, no matter the specificity of the selector. If `line-height: 2;` were declared in a `:root body p` selector block in any of the five imported stylesheets, the line height would still be `1.6`. Normal inline styles do not take precedence over animated or transitioned properties.

Important inline styles take precedence over all other author styles, regardless of whether they are important, inline, or layered. Important inline styles also take precedence over animated properties, but not transitioned properties. Three things can override an important inline style:

- An important user style.
- An important user agent style.
- A transitioned property.

Importance and layers

The origin type precedence order is inverted for important styles. Important styles declared outside of any cascade layer have lower precedence than those declared as part of a layer. Important styles that come in early layers take precedence over important styles declared in subsequent cascade layers.

Take for example the following CSS:

```
p {
  color: red;
}

@layer B {
  :root p {
    color: blue;
  }
}
```

Even though the red is declared first and has a less specific selector, because unlayered CSS takes precedence over layered CSS, the paragraph will be red. Had we included an inline style on the paragraph setting it to a different color, such as `<p style="color: black">`, the paragraph would be black.

When we add `!important` to this bit of CSS, the precedence order is reversed with the stylesheet:

```
p {
  color: red !important;
}

@layer B {
  :root p {
    color: blue !important;
  }
}
```

Now the paragraph will be blue. The `!important` in the earliest declared layer takes precedence over subsequent layers and unlayered important declarations. If the inline style contained `!important`, such as `<p style="color: black !important">`, again the paragraph would be black. Inline importance does take precedence over all other author declared `!important` declarations, no matter the specificity.

> **Note:** The `!important` flag reverses the precedence of cascade layers. For this reason, try not to use `!important` to override external styles. Instead, use `@import` together with the `layer` keyword or `layer()` function to import external stylesheets (from frameworks, widget stylesheets, libraries, etc.) into layers. Importing stylesheets into a layer as the first declaration in your CSS demotes their precedence, and author-defined layers, defined later in your CSS, will have higher precedence. The `!important` flag should only be used sparingly, if ever, to guard required styles against later overrides, in the first declared layer.

Styles that are transitioning take precedence over all important styles, no matter who or how they are declared.

Complete cascade order

Now that we have a better understanding of origin type and cascade layer precedence, we realize the table in Cascading order could have more accurately been represented by the following table:

Precedence Order (low to high)	Style Origin	Importance
1	user-agent - first declared layer	normal
	user-agent - last declared layer	
	user-agent - unlayered styles	
2	user - first declared layer	normal
	user - last declared layer	
	user - unlayered styles	
3	author - first declared layer	normal
	author - last declared layer	
	author - unlayered styles	
	inline `style`	
4	animations	

Precedence Order (low to high)	Style Origin	Importance
5	author - unlayered styles	`!important`
	author - last declared layer	
	author - first declared layer	
	inline `style`	
6	user - unlayered styles	`!important`
	user - last declared layer	
	user - first declared layer	
7	user-agent - unlayered styles	`!important`
	user-agent - last declared layer	
	user-agent - first declared layer	
8	transitions	

Which CSS entities participate in the cascade

Only CSS property/value pair declarations participate in the cascade. CSS at-rule descriptors don't participate in the cascade and HTML presentational attributes are not part of the cascade.

At-rules

CSS at-rules containing entities other than declarations, such as a `@font-face` rule containing *descriptors*, don't participate in the cascade.

For the most part, the properties and descriptors defined in at-rules don't participate in the cascade. Only at-rules as a whole participate in the cascade. For example, within a `@font-face` rule, font names are identified by `font-family` descriptors. If several `@font-face` rules with the same descriptor are defined, only the most appropriate `@font-face`, as a whole, is considered. If more than one are identically appropriate, the entire `@font-face` declarations are compared using steps 1, 2, and 4 of the algorithm (there is no specificity when it comes to at-rules).

While the declarations contained in most at-rules — such as those in `@media`, `@document`, or `@supports` — participate in the cascade, the at-rule may make an entire selector not relevant, as we saw with the print style in the basic example.

Declarations in `@keyframes` don't participate in the cascade. As with `@font-face`, only the `@keyframes` as a whole is selected via the cascade algorithm. The precedence order of animation is described below.

When it comes to `@import`, the `@import` doesn't participate itself in the cascade, but all of the imported styles do participate. If the `@import` defines a named or anonymous layer, the contents of the imported stylesheet are placed into the specified layer. All other CSS imported with `@import` is treated as the last declared layer. This was discussed above.

Finally, `@charset` obeys specific algorithms and isn't affected by the cascade algorithm.

Presentational attributes

Presentational attributes are attributes in the source document that can affect styling. For example, when included, the deprecated `align` attribute sets the alignment on several HTML elements and the `fill` attribute defines the color used to paint SVG shapes and text and defines the final state for SVG animations. While they are author styles, presentational attributes do not participate in the cascade.

If the HTML presentation attribute is supported by the user agent, valid presentational attributes included in HTML and SVG, such as the `align` or `fill` attributes, are translated to the corresponding CSS rules (all SVG presentation attributes are supported as CSS properties) and inserted in the author stylesheet prior to any other styles with a specificity equal to `0`.

Presentational attributes cannot be declared `!important`.

CSS animations and the cascade

CSS animations, using `@keyframes` at-rules, define animations between states. `@keyframes` don't cascade, meaning that at any given time CSS takes values from only one single set of `@keyframes` and never mixes multiple ones. If multiple sets of `@keyframes` are defined with the same animation name, the last defined set in the origin and layer with the greatest precedence is used. Other `@keyframes` are ignored, even if they animate different properties.

```
p {
  animation: infinite 5s alternate repeatedName;
}

@keyframes repeatedName {
  from {
    font-size: 1rem;
  }
  to {
    font-size: 3rem;
  }
}

@layer A {
  @keyframes repeatedName {
    from {
      background-color: yellow;
    }
    to {
      background-color: orange;
    }
  }
}

@layer B {
  @keyframes repeatedName {
    from {
      color: white;
    }
    to {
      color: black;
    }
  }
}
```

In this example, there are three separate animation declaration named `repeatedName`. When `animation: infinite 5s alternate repeatedName` is applied to the paragraph, only one animation is applied: the keyframe animation defined in the unlayered CSS takes precedence over the layered keyframe animation declarations based on origin and layer precedence order. In this example, only the element's font size will be animated.

> **Note:** There are no important animations, as property declarations in a `@keyframes` block that contain `!important` as part of the value are ignored.

Resetting styles

After your content has finished altering styles, it may find itself in a situation where it needs to restore them to a known state. This may happen in cases of animations, theme changes, and so forth. The CSS property `all` lets you quickly set (almost) everything in CSS back to a known state.

`all` lets you opt to immediately restore all properties to any of their initial (default) state, the state inherited from the previous level of the cascade, a specific origin (the user-agent stylesheet, the author stylesheet, or the user stylesheet), or even to clear the values of the properties entirely.

See also

- CSS cascading and inheritance module
- Learn: Handling conflicts
- Learn: Cascade layers
- CSS syntax
- Specificity
- Inheritance
- At-rules
- Values: initial, computed, used, and actual

CSS property value processing

For every element in a document tree, the browser assigns a value to every CSS property that applies to that element. The rendered value of each CSS property for a given element or box is the result of a calculation based on stylesheet definitions, inheritance, the cascade, dependencies, unit conversion, and the display environment. This guide provides an overview of the processing steps applied to define how each CSS value is ultimately rendered by exploring key concepts like specified, computed, used, and actual values.

Property values

Every style applied to an element or pseudo-element is based on a single CSS property declaration. Each CSS property has only one value. The value that is applied is determined by the cascaded values of all the declarations of that property that apply to that element or pseudo-element, with the single value applied coming from the property declaration that ranks highest in the cascade sorting order based on the cascade algorithm.

When there are multiple declared values, with multiple declarations, providing the same or different property values for the same element, each property value must still come from a single property name-value pair as only a single value is applied from each property, even if the value is a comma-separated list of values.

To determine which declared value is applied, the user agent gathers and processes all the styles from different sources, such as inline styles, and internal and external stylesheets.

The cascade determines which value should be applied when multiple conflicting styles target the same element. The cascade algorithm defines how user agents combine property values originating from different sources, scopes, and/or layers. When a selector matches an element, the property's declared value from the origin with the highest precedence gets applied, even if a selector from a lower precedence origin or layers has greater specificity.

Certain properties inherit values from their parent elements unless explicitly overridden. Inheritance may occur when no style information exists for a specific property on an element. If the property is inherited, the value is set to the computed value of the parent element. If the property is not inherited, its value is set to the initial value for that element.

After applying the cascading rules and defaulting values step by step, the browser ensures the visual presentation matches the processed CSS.

Processing overview

Before diving into the individual value stages, it's important to understand the three main phases that occur in value processing; filtering, cascading, and defaulting.

Filtering

Filtering is the process of identifying all declarations that apply to each element. A declaration applies to an element only if:

- The declaration belongs to a style sheet that currently applies to this document
- Any conditional rules (like `@media` or `@supports`) that contain the declaration are currently true.
- The declaration belongs to a style rule whose selector matches the element
- The declaration is syntactically valid: the property name is recognized by the browser and the value matches the expected syntax for that property

Only valid declarations become declared values. Declarations with invalid property names or invalid values get filtered out according to CSS error handling rules.

In this example, only the `font-size` and `font-weight` declarations are processed. The CSS parser filters out errors, ignoring or "filtering" out the declaration with the invalid property name:

```
p {
  font-size: 1.25em;
  colr: blue;
  font-weight: bold;
}
```

When filtering is complete, every element has zero or more declared values for every CSS property. These declared values are the starting point for the cascading processing stage.

Cascading

Cascade resolves conflicts when multiple declarations apply to the same property on the same element. Cascade sorts declarations using the cascade sorting order algorithm.

For example, both `font-size` declarations match `<p class="large">CSS is fun!</p>`, but the second declaration gets applied because it has higher specificity. Both declarations have author origin, but the second selector has specificity of `0-1-1` while the first has `0-0-1`:

```
p {
  font-size: 1em;
}

p.large {
  font-size: 1.5em;
}
```

After cascading, the browser determines the **cascaded value** for each property on each element. This is the value used in the next processing stage; defaulting.

Defaulting

Defaulting ensures every property on every element has a value. This involves applying default property values when no CSS declarations explicitly set that property value. This involves:

- Setting **inherited values** for inherited properties
- Setting **initial values** for non-inherited properties

As a result of defaulting, every property is guaranteed to have a specified value.

Note that explicit defaulting keywords (`initial`, `inherit`, `unset`, `revert`, `revert-layer`) are also resolved to their corresponding values to determine the specified value.

Processing stages

All elements that are part of the document's flattened element tree have declared, cascaded, specified, computed, used, and actual values. For a specific property, these values may or may not be the same. For example, if your large code base includes the CSS `p { font-size: 1.25em; }` and your HTML includes `<p class="large">CSS is fun!</p>`, what size will the paragraph be? The `font-size` value moves through a few stages to go from the `em` specified value to the rendered `px` value.

The value processing stages are Declared value, Cascaded value, Specified value, Computed value, Used value, and Actual value. These values are used to determine the final rendered value.

Declared value

A **declared value** is any syntactically valid value from a declaration that applies to an element. An element can have zero or more declared values for each property. These values come from style sheets (author, user, or user-agent) and are identified during the filtering stage.

Continuing our example, in which our stylesheet includes an occurrence of `p { font-size: 1.25em; }` and the document linking to that stylesheet includes `<p class="large">CSS is fun!</p>`, there may be other `font-size` declarations that could potentially apply to the same paragraph. The user-agent stylesheet might set `font-size: 1em` for all paragraphs, while another author declaration sets `font-size: 2em` for elements with class "large":

```
/* User agent styles */
p {
  font-size: 1em;
}

/* author styles */
p {
  font-size: 1.25em;
}

.large {
  font-size: 2em;
}
```

There may be many other `font-size` declarations in our stylesheets, but only declarations whose selectors match the element become declared values. In this example, as our `<p>` element has `class="large"`, all three declarations are declared values for this element.

Cascaded value

The **cascaded value** is the declared value that wins the cascade. There is at most one cascaded value per property per element.

From our declared values, author styles win over user agent styles. Within the same origin, higher specificity styles win over lower specificity styles. In this case, the cascaded value would be `font-size: 2em`, from the author origin with specificity `0-1-1`:

```
font-size: 2em;
```

If there are no declared values for a property, there is no cascaded value, which means the specified value for that property is determined by the defaulting process.

Specified value

The **specified value** is the result of the defaulting process. It is guaranteed to exist for every property on every element. The specified value is determined as follows:

1. If there is a cascaded value, the cascaded value is the specified value.
2. If there is *no* cascaded value and the property is inherited, the specified value is the computed value of the parent element.
3. If there is *no* cascaded value and the property is *not* inherited, the specified value is the property's initial value.

In our example, since we have a cascaded value of `2em`, this becomes the specified value:

```
font-size: 2em;
```

For properties without cascaded values, the defaulting process determines the value. For example, if `color` is not specified, the `color` is inherited from the parent's computed value since it's an inherited property. If `margin` is not specified, the `initial` value of `0` is used as `margin` is not an inherited property:

```
color: inherit;
margin: 0;
```

Initial value

A property's **initial value** is the default value as listed in its definition table in the specification. The initial value is used during defaulting when:

- For inherited properties, the initial value is used on the *root element only*, which has no parent element, when no cascaded value exists.
- For non-inherited properties, the initial value is used on *all elements* when no cascaded value exists.

You can explicitly set the initial value by using the `initial` keyword.

> **Note:** The initial value can be found in the formal syntax section of each CSS property reference page. For example, the initial value of `font-size` is `medium`. The initial value should not be confused with the value specified by the browser's style sheet.

Computed value

The **computed value** of a property is the value transferred from parent to child during inheritance. It is the result after resolving things like relative units and custom properties into absolute values, but before considering layout-specific information.

The computed value is calculated from the specified value by:

1. Handling the special values `inherit`, `initial`, `revert`, `revert-layer`, and `unset`.
2. Doing the computation needed to reach the value described in the "Computed value" line in the property's definition table.

The computation needed to reach a property's computed value typically involves converting relative values (such as those in `em` units or percentages) to absolute values. For example, if an element has specified values `font-size: 16px` and `padding-top: 2em`, then the computed value of `padding-top` is `32px` (double the font size).

However, for some properties (those where percentages are relative to something that may require layout to determine, such as `width`, `margin-right`, `text-indent`, and `top`), percentage-specified values turn into percentage-computed values. Additionally, unitless numbers specified on the `line-height` property become the computed value, as specified. The relative values that remain in the computed value become absolute when the used value is determined.

Used value

The **used value** is the property's value after all calculations have been performed on the computed value and it has been refined with layout-specific details (e.g., percentages resolved to actual pixel values).

Every CSS property has a used value. The used values of dimensions (e.g., `width` or `line-height`) are in pixels. The used values of shorthand properties (e.g., `background`) are consistent with those of their component properties (e.g., `background-color` or `background-size`) and with `position` and `float`.

The used value for the `width` or `inline-size` of an element is a pixel value even if the specified value of the property was set with percentages or keyword values.

If we have three container elements with their width set as `auto`, `50%`, and `inherit`:

```
<div id="no-width">
  <p>No explicit width.</p>
  <p class="show-used-width">..</p>

  <div id="width-50">
    <p>Explicit width: 50%.</p>
    <p class="show-used-width">..</p>

    <div id="width-inherit">
      <p>Explicit width: inherit.</p>
      <p class="show-used-width">..</p>
    </div>
  </div>
</div>
```

```
#no-width {
  width: auto;
}

#width-50 {
  width: 50%;
}

#width-inherit {
  width: inherit;
}

/* Make results easier to see */
div {
  border: 1px solid red;
  padding: 8px;
}
```

```
function updateUsedWidth(id) {
  const div = document.getElementById(id);
  const par = div.querySelector(".show-used-width");
  const wid = window.getComputedStyle(div)["width"];
  par.textContent = `Used width: ${wid}.`;
}

function updateAllUsedWidths() {
  updateUsedWidth("no-width");
  updateUsedWidth("width-50");
  updateUsedWidth("width-inherit");
}

updateAllUsedWidths();
window.addEventListener("resize", updateAllUsedWidths);
```

While the three specified values, `auto`, `50%`, and `inherit`, are keyword and `<percentage>` values, retrieving the `width` using `window.getComputedStyle(el)["width"];` returns an absolute length `px` value:

Change the window size or rotate your mobile device to change the size and the used values.

Rendered values

The rendered value is called the actual value, while the value retrieved via script is called the resolved value.

Actual value

The **actual value** of a property is the used value of that property after any necessary approximations have been applied. It is the final rendered value as implemented by the browser, including adjustments for rendering quirks or limitations. For example, a user agent that can only render borders with a whole-number pixel width may round the thickness of the border to the nearest integer.

The calculation includes these steps:

1. First, the specified value is determined based on the result of cascading, inheritance, or using the initial value.
2. Next, the computed value is calculated according to the specification (for example, a `span` with `position: absolute` will have its computed `display` changed to `block`).
3. Then, layout is calculated, resulting in the used value.
4. Finally, the used value is transformed according to the limitations of the local environment, resulting in the actual value.

Resolved value

The **resolved value** of a property is the value after applying active stylesheets and resolving any basic computation those values may contain. The `getComputedStyle()` method returns a live `CSSStyleDeclaration` object containing the resolved values of all CSS properties applied to a specified element. Each resolved value is either the computed value or the used value, depending on the property.

Historically, `getComputedStyle()` returned the computed value of an element or pseudo-element. As CSS evolved, so did the concept of "computed value", but the values returned by `getComputedStyle()` had to remain the same for backward compatibility with deployed scripts. These values are the "resolved values".

For most properties, the resolved value is the computed value, but for a few legacy properties (including `width` and `height`), it is the used value. The CSSOM specification provides per-property details.

CSS 2.0 defined *computed value* as the last step in a property's calculation. CSS 2.1 introduced the distinct definition of "used value". An element could then explicitly inherit the width/height of its parent, whose computed value is a percentage. For CSS properties that don't depend on layout (e.g., `display`, `font-size`, or `line-height`), the computed values and used values are the same. The following list contains the CSS 2.1 properties that *do* depend on layout, and therefore have a different computed value and used value (taken from CSS 2.1 Changes: Specified, computed, and actual values):

- `background-position`
- `bottom`, `left`, `right`, `top`
- `height`, `width`
- `margin-bottom`, `margin-left`, `margin-right`, `margin-top`
- `min-height`, `min-width`

- `padding-bottom`, `padding-left`, `padding-right`, `padding-top`
- `text-indent`

See also

- CSS values for controlling inheritance: `inherit`, `initial`, `revert`, `revert-layer`, and `unset`
- CSS cascading and inheritance module
- CSS syntax module

Shorthand properties

Shorthand properties are CSS properties that let you set the values of multiple other CSS properties in one declaration. Using a shorthand property, you can write more concise (and often more readable) style sheets, saving time and energy.

The CSS specification defines shorthand properties to group the definition of common properties acting on the same theme. For example, the CSS `background` property is a shorthand property that's able to define the values of `background-color`, `background-image`, `background-repeat`, and `background-position`.

Tricky edge cases

There are a few edge cases to keep in mind when using shorthand properties.

Omitting properties

A value which is not specified is set to a default value defined by the shorthand, which may differ from the property's initial value.

That means that it **overrides** previously set values. For example:

```
p {
  background-color: red;
  background: url("images/bg.gif") no-repeat left top;
}
```

This will not set the color of the background to `red` but to the default value for `background-color`, which is `transparent`.

Only the individual properties values can inherit. As missing values are replaced by their initial value, it is impossible to allow inheritance of individual properties by omitting them. The keyword `inherit` can be applied to a property, but only as a whole, not as a keyword for one value or another. That means that the only way to make some specific value to be inherited is to use the longhand property with the keyword `inherit`.

Ordering properties

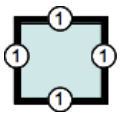
Shorthand properties try not to force a specific order for the values of the properties they replace. This works well when these properties use values of different types, as the order has no importance, but this does not work as easily when several properties can have identical values.

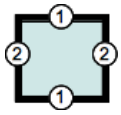
Two important cases here are:

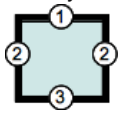
- Properties related to the sides of the box, like `border-style`, `margin` or `padding`
- Properties related to the corners of the box, like `border-radius`

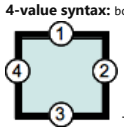
Sides of the box

Shorthands handling properties related to the sides of the box, like `border-style`, `margin` or `padding`, always use a consistent 1-to-4-value syntax representing those sides:

- **1-value syntax:** `border-width: 1em` — A single value represents all sides: 

- **2-value syntax:** `border-width: 1em 2em` — The first value represents the top and bottom sides, and the second value represents the left and right sides: 

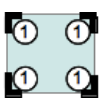
- **3-value syntax:** `border-width: 1em 2em 3em` — The first value represents the top side, the second value represents the left and right sides, and the third value represents the bottom side:


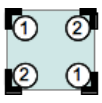
- **4-value syntax:** `border-width: 1em 2em 3em 4em` — The four values represent the top, the right, the bottom, and the left sides, respectively, always in that order; that is, clock-wise starting at the top:


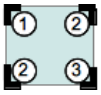
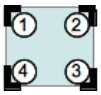
The initial letter of Top-Right-Bottom-Left matches the order of the consonant of the word *trouble*: TRBL. You can also remember it as the order that the hands would rotate on a clock: `1em` starts in the 12 o'clock position, then `2em` in the 3 o'clock position, then `3em` in the 6 o'clock position, and `4em` in the 9 o'clock position.

Corners of the box

Similarly, shorthands handling properties related to the corners of the box, like `border-radius`, always use a consistent 1-to-4-value syntax representing those corners:

- **1-value syntax:** `border-radius: 1em` — A single value represents all corners: 

- **2-value syntax:** `border-radius: 1em 2em` — The first value represents the top-left and bottom-right corners, and the second value represents the top-right and bottom-left corners: 

- **3-value syntax:** `border-radius: 1em 2em 3em` — The first value represents the top-left corner, the second value represents the top-right and bottom-left corners, and the third value represents the bottom-right corner: 
- **4-value syntax:** `border-radius: 1em 2em 3em 4em` — The four values represent the top-left, top-right, bottom-right, and bottom-left corners respectively, always in that order; that is, clock-wise starting at the top-left: 

Background properties

Consider a background with the following properties

```
background-color: black;
background-image: url("images/bg.gif");
background-repeat: no-repeat;
background-position: left top;
```

These four declarations can be shortened to just one:

```
background: black url("images/bg.gif") no-repeat left top;
```

(The shorthand form is actually the equivalent of the longhand properties above plus `background-attachment: scroll` and, in CSS3, some additional properties.)

See `background` for more detailed information, including CSS3 properties.

Font properties

Consider the following declarations:

```
font-style: italic;
font-weight: bold;
font-size: 0.8em;
line-height: 1.2;
font-family: "Arial", sans-serif;
```

These 5 statements can be shortened to the following:

```
font:
  italic bold 0.8em/1.2 "Arial",
  sans-serif;
```

This shorthand declaration is actually equivalent to the longhand declarations above plus `font-variant: normal`, `font-size-adjust: none`, and `font-stretch: normal`.

Border properties

With borders, the width, color, and style can be simplified into one declaration. For example, consider the following CSS:

```
border-width: 1px;
border-style: solid;
border-color: black;
```

It can be simplified as:

```
border: 1px solid black;
```

Margin and padding properties

Shorthand versions of margin and padding values work similarly; the margin property allows for shorthand values to be specified using one, two, three, or four values. Consider the following CSS declarations:

```
margin-top: 10px;
margin-right: 5px;
margin-bottom: 10px;
margin-left: 5px;
```

They are the same as the following declaration using the four value shorthand. Note that the values are in clockwise order, beginning at the top: top, right, bottom, then left (TRBL, the consonants in "trouble").

```
margin: 10px 5px 10px 5px;
```

Margin shorthand rules for one, two, three and four value declarations are:

- When **one** value is specified, it applies the same margin to **all four sides**.
- When **two** values are specified, the first margin applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first margin applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the margins apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Position properties

With position, the shorthand versions of top, right, bottom and left can be simplified into one declaration. For example, consider the following CSS:

```
top: 0;
right: 20px;
bottom: 0;
left: 20px;
```

It can be simplified as:

```
inset: 0 20px 0 20px;
```

Just like margins and paddings, the inset values are ordered clockwise - top, right, bottom, then left (TRBL).

The universal shorthand property

CSS provides a universal shorthand property, `all`, which applies its value to every property in the document. Its purpose is to change the properties' inheritance model.

See Handling conflicts or Introducing the CSS Cascade for more information about how inheritance works in CSS.

Shorthand properties

- `all`
- `animation`
- `animation-range`
- `background`
- `border`
- `border-block`
- `border-block-end`
- `border-block-start`
- `border-bottom`
- `border-color`
- `border-image`
- `border-inline`
- `border-inline-end`
- `border-inline-start`
- `border-left`
- `border-radius`
- `border-right`
- `border-style`
- `border-top`
- `border-width`
- `column-rule`
- `columns`
- `contain-intrinsic-size`
- `container`
- `flex`
- `flex-flow`
- `font`
- `font-synthesis`
- `font-variant`
- `gap`
- `grid`
- `grid-area`
- `grid-column`
- `grid-row`
- `grid-template`
- `inset`
- `inset-block`
- `inset-inline`
- `list-style`
- `margin`
- `margin-block`
- `margin-inline`
- `mask`
- `mask-border`
- `offset`
- `outline`
- `overflow`
- `overscroll-behavior`
- `padding`
- `padding-block`
- `padding-inline`
- `place-content`
- `place-items`
- `place-self`
- `position-try`
- `scroll-margin`
- `scroll-margin-block`
- `scroll-margin-inline`
- `scroll-padding`
- `scroll-padding-block`
- `scroll-padding-inline`
- `scroll-timeline`
- `text-box`
- `text-decoration`
- `text-emphasis`
- `text-wrap`
- `transition`
- `view-timeline`
- `-webkit-text-stroke`
- `-webkit-border-before`
- `-webkit-mask-box-image`

See also

- CSS cascading and inheritance module
- Introduction to CSS syntax: declarations, rulesets, and statements
- At-rules
- Specificity
- Inheritance

- Introduction to the CSS cascade
- Learn: Handling conflicts
- Learn: Cascade layers
- Visual formatting models
- Values: initial, computed, used, and actual
- Value definition syntax

Specificity

Specificity is the weight that browsers use in the cascade algorithm to determine the CSS declaration that is the most relevant to an element, which in turn, determines the property value to apply to the element. The specificity algorithm calculates this weight from a CSS selector, and compares the resulting values to decide which rule, from competing CSS declarations within the same origin and layer, gets applied to an element.

> **Note:** Browsers consider specificity **after** determining cascade origin and importance. In other words, for competing property declarations, specificity is relevant and compared only between selectors from the one cascade origin and layer that has precedence for the property. Scoping proximity and order of appearance become relevant when the selector specificities of the competing declarations in the cascade layer with precedence are equal.

How is specificity calculated?

Specificity is the weight that is applied to a given CSS declaration. The specificity algorithm calculates that weight based on the number of selectors of each weight category in the selector matching the element (or pseudo-element). If there are two or more declarations providing different property values for the same element, the declaration value in the style block having the matching selector with the greatest weight gets applied.

The specificity value is basically a three-column value of three categories or weights - ID, CLASS, and TYPE - corresponding to the three types of selectors. The value represents the count of selector components in each weight category and is written as *ID - CLASS - TYPE*. The three columns are created by counting the number of selector components for each selector weight category in the selectors that match the element.

Selector weight categories

The selector weight categories are listed here in the order of decreasing specificity:

ID column

> Includes only ID selectors, such as `#example`. For each ID in a matching selector, add 1-0-0 to the weight value.

CLASS column

> Includes class selectors, such as `.myClass`, attribute selectors like `[type="radio"]` and `[lang|="fr"]`, and pseudo-classes, such as `:hover`, `:nth-of-type(3n)`, and `:required`. For each class, attribute selector, or pseudo-class in a matching selector, add 0-1-0 to the weight value.

TYPE column

> Includes type selectors, such as `p`, `h1`, and `td`, and pseudo-elements like `::before`, `::placeholder`, and all other selectors with double-colon notation. For each type or pseudo-element in a matching selector, add 0-0-1 to the weight value.

No value

> The universal selector (`*`) and the pseudo-class `:where()` and its parameters aren't counted when calculating the weight so their value is 0-0-0, but they do match elements. These selectors do not impact the specificity weight value.

Combinators, such as `+`, `>`, `~`, `" "`, and `||`, may make a selector more specific in what is selected but they don't add any value to the specificity weight.

The `&` nesting combinator doesn't add specificity weight, but nested rules do. In terms of specificity, and functionality, nesting is very similar to the `:is()` pseudo-class.

Like nesting, the `:is()`, `:has()`, and negation (`:not()`) pseudo-classes themselves add no weight. The parameters in these selectors, however, do. The specificity weight of each comes from the selector parameter in the list of selectors with the highest specificity. Similarly, with nested selectors, the specificity weight added by the nested selector component is the selector in the comma-separated list of nested selectors with the highest specificity.

The `:not()`, `:is()`, `:has()` and CSS nesting exceptions are discussed below.

Matching selector

The specificity weight comes from the matching selector. Take this CSS selector with three comma-separated selectors as an example:

```
[type="password"],
input:focus,
:root #myApp input:required {
  color: blue;
}
```

The `[type="password"]` selector in the above selector list, with a specificity weight of `0-1-0`, applies the `color: blue` declaration to all password input types.

All inputs, no matter the type, when receiving focus, match the second selector in the list, `input:focus`, with a specificity weight of `0-1-1`; this weight is made up of the `:focus` pseudo-class (0-1-0) and the `input` type (0-0-1). If the password input has focus, it will match `input:focus`, and the specificity weight for the `color: blue` style declaration will be `0-1-1`. When that password doesn't have focus, the specificity weight remains at `0-1-0`.

The specificity for a required input nested in an element with attribute `id="myApp"` is `1-2-1`, based on one ID, two pseudo-classes, and one element type.

If the password input type with `required` is nested in an element with `id="myApp"` set, the specificity weight will be `1-2-1`, based on one ID, two pseudo-classes, and one element type, whether or not it has focus. Why is the specificity weight `1-2-1` rather than `0-1-1` or `0-1-0` in this case? Because the specificity weight comes from the matching selector with the greatest specificity weight. The weight is determined by comparing the values in the three columns, from left to right.

```
[type="password"] {
  /* 0-1-0 */
}
input:focus {
  /* 0-1-1 */
}
:root #myApp input:required {
  /* 1-2-1 */
}
```

Three-column comparison

Once the specificity values of the relevant selectors are determined, the number of selector components in each column is compared, from left to right.

```
#myElement {
  color: green; /* 1-0-0  - WINS!! */
}
.bodyClass .sectionClass .parentClass [id="myElement"] {
  color: yellow; /* 0-4-0 */
}
```

The first column is the value of the *ID* component, which is the number of IDs in each selector. The numbers in the *ID* columns of competing selectors are compared. The selector with the greater value in the *ID* column wins no matter what the values are in the other columns. In the above example, even though the yellow selector has more components in total, only the value of the first column matters.

If the number in the *ID* columns of competing selectors is the same, then the next column, *CLASS*, is compared, as shown below.

```
#myElement {
  color: yellow; /* 1-0-0 */
}
#myApp [id="myElement"] {
  color: green; /* 1-1-0  - WINS!! */
}
```

The *CLASS* column is the count of class names, attribute selectors, and pseudo-classes in the selector. When the *ID* column value is the same, the selector with the greater value in the *CLASS* column wins, no matter the value in the *TYPE* column. This is shown in the example below.

```
:root input {
  color: green; /* 0-1-1 - WINS because CLASS column is greater */
}
html body main input {
  color: yellow; /* 0-0-4 */
}
```

If the numbers in the *CLASS* and *ID* columns in competing selectors are the same, the *TYPE* column becomes relevant. The *TYPE* column is the number of element types and pseudo-elements in the selector. When the first two columns have the same value, the selector with the greater number in the *TYPE* column wins.

If the competing selectors have the same values in all the three columns, the proximity rule comes into play, wherein the last declared style gets precedence.

```
input.myClass {
  color: yellow; /* 0-1-1 */
}
:root input {
  color: green; /* 0-1-1 WINS because it comes later */
}
```

The <code>:is()</code>, <code>:not()</code>, <code>:has()</code> and CSS nesting exceptions

The matches-any pseudo-class `:is()`, the relational pseudo-class `:has()`, and the negation pseudo-class `:not()` are *not* considered as pseudo-classes in the specificity weight calculation. They themselves don't add any weight to the specificity equation. However, the selector parameters passed into the pseudo-class parenthesis are part of the specificity algorithm; the weight of the matches-any and negation pseudo-class in the specificity value calculation is the weight of the parameter's weight.

```
p {
  /* 0-0-1 */
}
:is(p) {
  /* 0-0-1 */
}

h2:nth-last-of-type(n + 2) {
  /* 0-1-1 */
}
h2:has(~ h2) {
  /* 0-0-2 */
}

div.outer p {
  /* 0-1-2 */
}
div:not(.inner) p {
  /* 0-1-2 */
}
```

Note that in the above CSS pairing, the specificity weight provided by the `:is()`, `:has()` and `:not()` pseudo-classes is the value of the selector parameter, not of the pseudo-class.

All three of these pseudo-classes accept complex selector lists, a list of comma-separated selectors, as a parameter. This feature can be used to increase a selector's specificity:

```
:is(p, #fakeId) {
  /* 1-0-0 */
}
h1:has(+ h2, > #fakeId) {
  /* 1-0-1 */
}
p:not(#fakeId) {
  /* 1-0-1 */
}
div:not(.inner, #fakeId) p {
  /* 1-0-2 */
}
```

In the above CSS code block, we have included `#fakeId` in the selectors. This `#fakeId` adds `1-0-0` to the specificity weight of each paragraph.

When creating complex selector lists with CSS nesting this behaves in exactly the same way as the `:is()` pseudo-class.

```
p,
#fakeId {
  span {
    /* 1-0-1 */
  }
}
```

In the above code block the complex selector `p, #fakeId` the specificity is taken from `#fakeId` and also the `span`, so this creates a specificity of `1-0-1` for both `p span` and `#fakeId span`. This is the equivalent specificity as the `:is(p, #fakeId) span` selector.

Generally, you want to keep specificity down to a minimum, but if you need to increase an element's specificity for a particular reason, these three pseudo-classes can help.

```
a:not(#fakeId#fakeId#fakeID) {
  color: blue; /* 3-0-1 */
}
```

In this example, all links will be blue, unless overridden by a link declaration with 3 or more IDs, a color value matching an `a` includes the `!important` flag, or if the link has an inline style color declaration. If you use such a technique, add a comment to explain why the hack was needed.

Inline styles

Inline styles added to an element (e.g., `style="font-weight: bold;"`) always overwrite any normal styles in author stylesheets, and therefore, can be thought of as having the highest specificity. Think of inline styles as having a specificity weight of `1-0-0-0`.

The only way to override inline styles is by using `!important`.

Many JavaScript frameworks and libraries add inline styles. Using `!important` with a very targeted selector, such as an attribute selector using the inline style, is one way to override these inline styles.

```
<p style="color: purple">…</p>
```

```
p[style*="purple"] {
  color: rebeccapurple !important;
}
```

Make sure to include a comment with every inclusion of the important flag so code maintainers understand why a CSS anti-pattern was used.

The <code>!important</code> exception

CSS declarations marked as important override any other declarations within the same cascade layer and origin. Although technically, `!important` has nothing to do with specificity, it interacts directly with specificity and the cascade. It reverses the cascade order of stylesheets.

If declarations from the same origin and cascade layer conflict and one property value has the `!important` flag set, the important declaration is applied no matter the specificity. When conflicting declarations from the same origin and cascade layer with the `!important` flag are applied to the same element, the declaration with a greater specificity is applied.

Using `!important` to override specificity is considered a **bad practice** and should be avoided for this purpose. Understanding and effectively using specificity and the cascade can remove any need for the `!important` flag.

Instead of using `!important` to override foreign CSS (from external libraries, like Bootstrap or normalize.css), import the third-party scripts directly into cascade layers. If you must use `!important` in your CSS, comment your usage so future code maintainers know why the declaration was marked important and know not to override it. But definitely, don't use `!important` when writing plugins or frameworks that other developers will need to incorporate without being able to control.

The <code>:where()</code> exception

The specificity-adjustment pseudo-class `:where()` always has its specificity replaced with zero, `0-0-0`. It enables making CSS selectors very specific in what element is targeted without any increase to specificity.

In creating third-party CSS to be used by developers who don't have access to edit your CSS, it's considered a good practice to create CSS with the lowest possible specificity. For example, if your theme includes the following CSS:

```
:where(#defaultTheme) a {
  /* 0-0-1 */
  color: red;
}
```

Then the developer implementing the widget can easily override the link color using only type selectors.

```
footer a {
  /* 0-0-2 */
  color: blue;
}
```

How <code>@scope</code> blocks affect specificity

Including a ruleset inside a `@scope` block does not affect the specificity of its selector, regardless of the selectors used inside the scope root and limit. However, if you decide to explicitly add the `:scope` pseudo-class, you'll need to factor it in when calculating their specificity. `:scope`, like all regular pseudo-classes, has a specificity of 0-1-0. For example:

```
@scope (.article-body) {
  /* :scope img has a specificity of 0-1-0 + 0-0-1 = 0-1-1 */
  :scope img {
  }
}
```

See Specificity in `@scope` for more information.

Tips for handling specificity headaches

Instead of using `!important`, consider using cascade layers and using low weight specificity throughout your CSS so that styles are easily overridden with slightly more specific rules. Using semantic HTML helps provide anchors from which to apply styling.

Making selectors specific with and without adding specificity

By indicating the section of the document you're styling before the element you're selecting, the rule becomes more specific. Depending on how you add it, you can add some, a lot, or no specificity, as shown below:

```
<main id="myContent">
  <h1>Text</h1>
</main>
```

```
#myContent h1 {
  color: green; /* 1-0-1 */
}
[id="myContent"] h1 {
  color: yellow; /* 0-1-1 */
}
```

```
:where(#myContent) h1 {
  color: blue; /* 0-0-1 */
}
```

No matter the order, the heading will be green because that rule is the most specific.

Reducing ID specificity

Specificity is based on the form of a selector. Including the `id` of an element as an attribute selector rather than an id selector is a good way to make an element more specific without adding an overabundance of specificity. In the previous example, the selector `[id="myContent"]` counts as an attribute selector for the purpose of determining the selector's specificity, even though it selects an ID.

You can also include the `id` or any part of a selector as a parameter in the `:where()` specificity-adjustment pseudo class if you need to make a selector more specific but don't want to add any specificity at all.

Increasing specificity by duplicating selector

As a special case for increasing specificity, you can duplicate weights from the *CLASS* or *ID* columns. Duplicating id, class, pseudo-class or attribute selectors within a compound selector will increase specificity when overriding very specific selectors over which you have no control.

```
#myId#myId#myId span {
  /* 3-0-1 */
}
.myClass.myClass.myClass span {
  /* 0-3-1 */
}
```

Use this sparingly, if at all. If using selector duplication, always comment your CSS.

By using `:is()` and `:not()` (and also `:has()`), you can increase specificity even if you can't add an `id` to a parent element:

```
:not(#fakeID#fakeId#fakeID) span {
  /* 3-0-1 */
}
:is(#fakeID#fakeId#fakeID, span) {
  /* 3-0-0 */
}
```

Precedence over third-party CSS

Leveraging cascade layers is the standard way of enabling one set of styles to take precedence over another set of styles; cascade layers enable this without using specificity! Normal (not important) author styles imported into cascade layers have lower precedence than unlayered author styles.

If styles are coming from a stylesheet you can't edit or don't understand and you need to override styles, a strategy is to import the styles you don't control into a cascade layer. Styles in subsequently declared layers take precedence, with unlayered styles having precedence over all layered styles from the same origin.

When two selectors from different layers match the same element, origin and importance take precedence; the specificity of the selector in the losing stylesheet is irrelevant.

```
@import "TW.css" layer();
p,
p * {
  font-size: 1rem;
}
```

In the above example, all paragraph text, including the nested content, will be `1rem` no matter how many class names the paragraphs have that match the TW stylesheet.

Avoiding and overriding <code>!important</code>

The best approach is to not use `!important`. The above explanations on specificity should be helpful in avoiding using the flag and removing it altogether when encountered.

To remove the perceived need for `!important`, you can do one of the following:

- Increase the specificity of the selector of the formerly `!important` declaration so that it is greater than other declarations
- Give it the same specificity and put it after the declaration it is meant to override
- Reduce the specificity of the selector you are trying to override.

All these methods are covered in preceding sections.

If you're unable to remove `!important` flags from an author's style sheet, the only solution to overriding the important styles is by using `!important`. Creating a cascade layer of important declaration overrides is an excellent solution. Two ways of doing this include:

Method 1

1. Create a separate, short style sheet containing only important declarations specifically overriding any important declarations you were unable to remove.
2. Import this stylesheet as the first import in your CSS using `layer()`, including the `@import` statement, before linking to other stylesheets. This is to ensure that the important overrides are imported as the first layer.

```
@import "importantOverrides.css" layer();
```

Method 2

1. At the beginning of your stylesheet declarations, create a named cascade layer, like so:

   ```
   @layer importantOverrides;
   ```

2. Each time you need to override an important declaration, declare it within the named layer. Only declare important rules within the layer.

   ```
   [id="myElement"] p {
     /* normal styles here */
   }
   @layer importantOverrides {
     [id="myElement"] p {
       /* important style here */
     }
   }
   ```

The specificity of the selector of the important style within the layer can be low, as long as it matches the element you are trying to override. Normal layers should be declared outside the layer because layered styles have lower precedence than unlayered styles.

Tree proximity ignorance

The proximity of an element to other elements that are referenced in a given selector has no impact on specificity.

```
body h1 {
  color: green;
}

html h1 {
  color: purple;
}
```

The `<h1>` elements will be purple because when declarations have the same specificity, the last declared selector has precedence.

Directly targeted elements vs. inherited styles

Styles for a directly targeted element will always take precedence over inherited styles, regardless of the specificity of the inherited rule. Given the following CSS and HTML:

```
#parent {
  color: green;
}

h1 {
  color: purple;
}
```

```
<html lang="en">
  <body id="parent">
    <h1>Here is a title!</h1>
  </body>
</html>
```

The `h1` will be purple because the `h1` selector targets the element specifically, while the green is inherited from the `#parent` declarations.

Examples

In the following CSS, we have three selectors targeting `<input>` elements to set a color. For a given input, the specificity weight of the color declaration having precedence is the matching selector with the greatest weight:

```
#myElement input.myClass {
  color: red;
} /* 1-1-1 */
input[type="password"]:required {
  color: blue;
} /* 0-2-1 */
html body main input {
  color: green;
} /* 0-0-4 */
```

If the above selectors all target the same input, the input will be red, as the first declaration has the highest value in the *ID* column.

The last selector has four *TYPE* components. While it has the highest integer value, no matter how many elements and pseudo-elements are included, even if there were 150, TYPE components never have precedence over *CLASS* components. The column values are compared starting from left to right when column values are equal.

Had we converted the id selector in the example code above to an attribute selector, the first two selectors would have the same specificity, as shown below:

```
[id="myElement"] input.myClass {
  color: red;
} /* 0-2-1 */
input[type="password"]:required {
  color: blue;
} /* 0-2-1 */
```

When multiple declarations have equal specificity, the last declaration found in the CSS is applied to the element. If both selectors match the same `<input>`, the color will be blue.

Additional notes

A few things to remember about specificity:

1. Specificity only applies when the same element is targeted by multiple declarations in the same cascade layer or origin. Specificity only matters for declarations of the same importance and same origin and cascade layer. If matching selectors are in different origins, the cascade determines which declaration takes precedence.
2. When two selectors in the same cascade layer and origin have the same specificity, scoping proximity is then calculated; the ruleset with the lowest scoping proximity wins. See How `@scope` conflicts are resolved for more details and an example.
3. If scope proximity is also the same for both selectors, source order then comes into play. When all else is equal, the last selector wins.
4. As per CSS rules, directly targeted elements will always take precedence over rules which an element inherits from its ancestor.
5. Proximity of elements in the document tree has no effect on the specificity.

See also

- CSS cascading and inheritance module
- Learn: Handling conflicts
- Learn: Cascade layers
- CSS syntax module
- Introduction to CSS syntax: declarations, rulesets, and statements
- CSS error handling
- At-rules
- Inheritance
- Values: initial, computed, used, and actual
- Value definition syntax
- CSS nesting module
- Specificity Calculator by Keegan Street: An interactive website to test and understand your own CSS rules
- SpeciFISHity on specifishity.com: A fun way to learn about CSS specificity
- *ID-CLASS-TYPE* exercise: A specificity quiz by Estelle Weyl

CSS custom properties for cascading variables

The **CSS custom properties for cascading variables** module adds support for cascading variables in CSS properties and lets you create custom properties to define these variables along with the mechanisms to use custom properties as the values for other CSS properties.

When working with CSS, you often end up reusing common project-specific values such as widths that work well with your layout, or a set of colors for your color scheme. One way of managing repetition in stylesheets is to define a value once and use it many times in other places. Custom properties let you create and define custom variables that can be reused, simplifying complex or repetitive rules and making them easier to read and maintain. For example, `--dark-grey-text` and `--dark-background` are easier to understand than hexadecimal colors such as `#323831`, and the context of how you use them is more obvious, too.

Custom properties in action

To see how custom properties can be used, move the input slider left to right.

```
<div class="container">
  <div id="color-1">--hue</div>
  <div id="color-2">--hue + 10</div>
  <div id="color-3">--hue + 20</div>
  <div id="color-4">--hue + 30</div>
  <div id="color-5">--hue + 40</div>
  <div id="color-6">--hue + 50</div>
  <div id="color-7">--hue + 60</div>
  <div id="color-8">--hue + 70</div>
</div>
<input type="range" min="0" max="360" value="0" step="0.1" id="hue" />
```

```
const hue = document.querySelector("#hue");

const updateHue = () => {
  document.documentElement.style.setProperty("--hue", hue.value);
};

hue.addEventListener("input", updateHue);
```

```
.container {
  display: grid;
  font-family: sans-serif;
  color: white;
  gap: 0.5rem;
  grid-template-columns: repeat(4, 1fr);
  margin-bottom: 1rem;
}
.container div {
  border-radius: 0.5rem;
  padding: 1rem;
}

input {
  width: 100%;
  margin: 0;
}

:root {
  --hue: 0;
}

#color-1 {
  background-color: hsl(var(--hue) 50% 50%);
}
#color-2 {
  background-color: hsl(calc(var(--hue) + 10) 50% 50%);
}
#color-3 {
  background-color: hsl(calc(var(--hue) + 20) 50% 50%);
}
#color-4 {
  background-color: hsl(calc(var(--hue) + 30) 50% 50%);
}
#color-5 {
  background-color: hsl(calc(var(--hue) + 40) 50% 50%);
}
#color-6 {
  background-color: hsl(calc(var(--hue) + 50) 50% 50%);
}
#color-7 {
  background-color: hsl(calc(var(--hue) + 60) 50% 50%);
}
#color-8 {
  background-color: hsl(calc(var(--hue) + 70) 50% 50%);
}
```

In these color swatches, the `background-color` is set using the `hsl()` `<color>` function as `hsl(var(--hue) 50% 50%)`. Each color swatch increments the `<hue>` value by 10 degrees like `calc(var(--hue) + 10)`, `calc(var(--hue) + 20)` etc. As the slider's value changes from 0 up to 360, the value of the `--hue` custom property is updated using `calc()`, and the background color of each box inside the grid is updated, also.

Reference

Properties

- `--*`

Functions

- `var()`

Guides

Using CSS custom properties (variables)

Explains how to use custom properties in CSS and JavaScript, with hints on handling undefined and invalid values, fallbacks, and inheritance.

Invalid custom properties

Explains how browsers handle property values when a custom property's value is an invalid data type for that property.

Related concepts

- CSS Properties and Values API module
  - `@property` at-rule
  - `CSS.registerProperty()` method

See also

- CSS cascading and inheritance module
- CSS `env()` function
- CSS `calc()` function
- `getPropertyValue()` method

Using CSS custom properties (variables)

Custom properties (sometimes referred to as **CSS variables** or **cascading variables**) are entities defined by CSS authors that represent specific values to be reused throughout a document. They are set using the `@property` at-rule or by custom property syntax (e.g., **`--primary-color: blue;`**). Custom properties are accessed using the CSS `var()` function (e.g., **`color: var(--primary-color);`**).

Complex websites have very large amounts of CSS, and this often results in a lot of repeated CSS values. For example, it's common to see the same color used in hundreds of different places in stylesheets. Changing a color that's been duplicated in many places requires a search and replace across all rules and CSS files. Custom properties allow a value to be defined in one place, then referenced in multiple other places so that it's easier to work with. Another benefit is readability and semantics. For example, `--main-text-color` is easier to understand than the hexadecimal color `#00ff00`, especially if the color is used in different contexts.

Custom properties defined using two dashes (`--`) are subject to the cascade and inherit their value from their parent. The `@property` at-rule allows more control over the custom property and lets you specify whether it inherits its value from a parent, what the initial value is, and the type constraints that should apply.

> **Note:** Variables do not work inside media queries and container queries. You can use the `var()` function in any part of a value in any property on an element. You cannot use `var()` for property names, selectors, or anything aside from property values, which means you can't use it in a media query or container query.

Declaring custom properties

In CSS, you can declare a custom property using two dashes as a prefix for the property name, or by using the `@property` at-rule. The following sections describe how to use these two methods.

Using a prefix of two dashes (<code>--</code>)

A custom property prefixed with two dashes begins with `--`, followed by the property name (e.g., `--my-property`), and a property value that can be any valid CSS value. Like any other property, this is written inside a ruleset. The following example shows how to create a custom property `--main-bg-color` and uses a `<named-color>` value of `brown`:

```
section {
  --main-bg-color: brown;
}
```

The selector given to the ruleset (`<section>` elements in the example above) defines the scope in which the custom property can be used. For this reason, a common practice is to define custom properties on the `:root` pseudo-class, so that it can be referenced globally:

```
:root {
  --main-bg-color: brown;
}
```

This doesn't always have to be the case: you maybe have a good reason for limiting the scope of your custom properties.

> **Note:** Custom property names are case sensitive — `--my-color` will be treated as a separate custom property to `--My-color`.

Using the <code>@property</code> at-rule

The `@property` at-rule allows you to be more expressive with the definition of a custom property with the ability to associate a type with the property, set default values, and control inheritance. The following example creates a custom property called `--logo-color` which expects a `<color>`:

```
@property --logo-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #c0ffee;
}
```

If you want to define or work with custom properties in JavaScript instead of directly in CSS, there is a corresponding API for this purpose. You can read about how this works in the CSS Properties and Values API page.

Referencing custom properties with <code>var()</code>

Regardless of which method you choose to define a custom property, you use them by referencing the property in a `var()` function in place of a standard property value:

```
details {
  background-color: var(--main-bg-color);
}
```

First steps with custom properties

Let's start with some HTML that we would like to apply some styles to. There is a `<div>` that acts as a container that includes some child elements, some with nested elements:

```
<div class="container">
  <div class="one">
    <p>One</p>
  </div>
  <div class="two">
    <p>Two</p>
    <div class="three">
      <p>Three</p>
    </div>
  </div>
  <input class="four" placeholder="Four" />
```

```
  <textarea class="five">Five</textarea>
</div>
```

We will use the following CSS to style a few different elements based on their classes (some layout rules are not shown below so we can focus on colors). Depending on their classes, we're giving elements `teal` or `pink` background colors:

```
/* Set fonts, borders and padding */
body,
textarea,
::placeholder {
  font-family: sans-serif;
  color: white;
}

div,
input,
textarea {
  border: 2px black solid;
  padding: 4px;
  margin: 4px;
}

.container {
  display: grid;
  gap: 10px;
}
```

```
/* For each class, set some colors */
.one {
  background-color: teal;
}

.two {
  color: black;
  background-color: pink;
}

.three {
  color: white;
  background-color: teal;
}

.four {
  background-color: teal;
}

.five {
  background-color: teal;
}
```

This produces the following result:

There's an opportunity to use custom properties to replace repetitive values across these rules. After defining `--main-bg-color` in the `.container` scope and referencing its value in multiple places, the updated styles look like this:

```
/* Define --main-bg-color here */
.container {
  --main-bg-color: teal;
}

/* For each class, set some colors */
.one {
  background-color: var(--main-bg-color);
}

.two {
  color: black;
  background-color: pink;
}

.three {
  color: white;
  background-color: var(--main-bg-color);
}

.four {
  background-color: var(--main-bg-color);
}

.five {
  background-color: var(--main-bg-color);
}
```

Using the :root pseudo-class

For some CSS declarations, it is possible to declare this higher in the cascade and let CSS inheritance solve this problem. For non-trivial projects, this is not always possible. By declaring a custom property on the `:root` pseudo-class and using it where needed throughout the document, a CSS author can reduce the need for repetition:

```
/* Define --main-bg-color here */
:root {
  --main-bg-color: teal;
}

/* For each class, set some colors */
.one,
.three,
.four,
.five {
  background-color: var(--main-bg-color);
}
```

```
.two {
  color: black;
  background-color: pink;
}
```

This leads to the same result as the previous example, yet allows for one canonical declaration of the desired property value (`--main-bg-color: teal;`), which is very useful if you want to change the value across the entire project later.

Inheritance of custom properties

A custom property defined using two dashes `--` instead of `@property` always inherits the value of its parent. This is demonstrated in the following example:

```
<div class="one">
  <p>One</p>
  <div class="two">
    <p>Two</p>
    <div class="three"><p>Three</p></div>
    <div class="four"><p>Four</p></div>
  </div>
</div>
```

```
div {
  color: black;
  font-family: sans-serif;
  width: 75%;
  height: 80%;
  margin: 4px;
  border: 2px black solid;
  display: inline-block;
}

p {
  margin: 0;
}

.one {
  height: 250px;
}

.two {
  color: white;
  height: 80%;
}

.three {
  color: black;
  height: 40%;
}

.four {
  color: white;
  height: 40%;
}
```

```
div {
  background-color: var(--box-color);
}

.two {
  --box-color: teal;
}

.three {
  --box-color: pink;
}
```

The results of `var(--box-color)` depending on inheritance are as follows:

- `class="one"`: *invalid value*, which is the default value of a custom property defined in this way
- `class="two"`: `teal`
- `class="three"`: `pink`
- `class="four"`: `teal` (inherited from its parent)

One aspect of custom properties that the examples above demonstrate is that they don't behave exactly like variables in other programming languages. The value is computed where it is needed, not stored and reused in other places of a stylesheet. For instance, you cannot set a property's value and expect to retrieve the value in a sibling's descendant's rule. The property is only set for the matching selector and its descendants.

Using <code>@property</code> to control inheritance

The `@property` at-rule lets you explicitly state whether the property inherits or not. The following example creates a custom property using the `@property` at-rule. Inheritance is disabled, there's a `<color>` data type defined, and an initial value of `teal`.

The parent element sets `--box-color` to a value of `green` and uses `--box-color` as a value for its background color. The child element also uses `background-color: var(--box-color)`, and we would expect it to have the color `green` if inheritance was enabled (or if it was defined using the double dash syntax).

```
<div class="parent">
  <p>Parent element</p>
  <div class="child">
    <p>Child element with inheritance disabled for --box-color.</p>
  </div>
</div>
```

```
div {
  color: white;
  font-family: sans-serif;
  width: 200px;
  height: 200px;
  margin: 4px;
```

```
  padding: 8px;
  border: 2px black solid;
  display: inline-block;
}
```

```
@property --box-color {
  syntax: "<color>";
  inherits: false;
  initial-value: teal;
}

.parent {
  --box-color: green;
  background-color: var(--box-color);
}

.child {
  width: 80%;
  height: 40%;
  background-color: var(--box-color);
}
```

Because `inherits: false;` is set in the at-rule, and a value for the `--box-color` property is not declared within the `.child` scope, the initial value of `teal` is used instead of `green` that would have been inherited from the parent:

Custom property fallback values

You can define fallback values for custom properties using the `var()` function, and the `initial-value` of the `@property` at-rule.

> **Note:** Fallback values aren't used to fix compatibility issues for when CSS custom properties are not supported, as the fallback value won't help in this case. Fallbacks cover the case where the browser supports CSS custom properties and is able to use a different value if the desired variable isn't defined yet or has an invalid value.

Defining fallbacks in the <code>var()</code> function

Using the `var()` function, you can define multiple **fallback values** when the given variable is not yet defined; this can be useful when working with Custom Elements and Shadow DOM.

The first argument to the function is the name of the custom property. The second argument to the function is an optional fallback value, which is used as the substitution value when the referenced custom property is invalid. The function accepts two parameters, assigning everything following the first comma as the second parameter. If the second parameter is invalid, the fallback will fail. For example:

```
.one {
  /* Red if --my-var is not defined */
  color: var(--my-var, red);
}

.two {
  /* pink if --my-var and --my-background are not defined */
  color: var(--my-var, var(--my-background, pink));
}

.three {
  /* Invalid: "--my-background, pink" */
  color: var(--my-var, --my-background, pink);
}
```

Including a custom property as a fallback, as seen in the second example above (`var(--my-var, var(--my-background, pink))`), is the correct way to provide more than one fallback with `var()`. You should be aware of the performance impact of this method, however, as it takes more time to parse through the nested variables.

> **Note:** The syntax of the fallback, like that of custom properties, allows commas. For example, `var(--foo, red, blue)` defines a fallback of `red, blue` — anything between the first comma and the end of the function is considered a fallback value.

Fallbacks using the <code>@property</code> initial value

Aside from using `var()`, the `initial-value` defined in the `@property` at-rule can be used as a fallback mechanism. In fact, we've already seen this in the `@property` inheritance section.

The following example sets an initial value of `--box-color` to `teal` using the `@property` at-rule. In the ruleset following the at-rule, we want to set `--box-color` to `pink`, but there's a typo in the value name. The same is true for the third `<div>` where we've used `2rem` for the custom property that's expecting a valid `<color>` value. Both `2rem` and `peenk` are invalid color values, so the initial value of `teal` is applied:

```
@property --box-color {
  syntax: "<color>";
  initial-value: teal;
  inherits: false;
}

.one {
  --box-color: pink;
  background-color: var(--box-color);
}

.two {
  --box-color: peenk;
  background-color: var(--box-color);
}

.three {
  --box-color: 2rem;
  background-color: var(--box-color);
}
```

```
div {
  color: white;
  font-family: sans-serif;
  width: 100px;
  height: 100px;
  margin: 4px;
  padding: 8px;
  border: 2px black solid;
```

```
  display: inline-block;
}

.one {
  color: black;
}
```

```
<div class="one">
  <p>One</p>
</div>
<div class="two">
  <p>Two.</p>
</div>
<div class="three">
  <p>Three.</p>
</div>
```

Invalid custom properties

Each CSS property can be assigned a defined set of values. If you try to assign a value to a property that is outside its set of valid values, it's considered *invalid*.

When the browser encounters an invalid value for a regular CSS property (for example, a value of `16px` for the `color` property), it discards the declaration, and elements are assigned the values that they would have had if the declaration did not exist. In the following example, we see what happens when a regular CSS declaration is invalid; `color: 16px;` is discarded and the previous `color: blue` rule is applied instead:

```
<p>This paragraph is initially black.</p>
```

```
p {
  font-weight: bold;
  color: blue;
}

p {
  /* oops, not a valid color */
  color: 16px;
}
```

However, when the values of custom properties are parsed, the browser doesn't yet know where they will be used, so it must consider nearly all values as *valid*. Unfortunately, these valid values can be used, via the `var()` functional notation, in a context where they might not make sense. Properties and custom variables can lead to invalid CSS statements, leading to the concept of *valid at computed time*.

When the browser encounters an invalid `var()` substitution, then the initial or inherited value of the property is used. This example is just like the last one, except we use a custom property.

The browser substitutes the value of `--text-color` in place of `var(--text-color)`, but `16px` is not a valid property value for `color`. After substitution, the property doesn't make sense, so the browser handles this situation in two steps:

1. Check if the property `color` is inheritable. It is, but this `<p>` doesn't have any parent with the `color` property set. So we move on to the next step.
2. Set the value to its **default initial value**, which is black.

```
<p>This paragraph is initially black.</p>
```

```
:root {
  --text-color: 16px;
}

p {
  font-weight: bold;
  color: blue;
}

p {
  color: var(--text-color);
}
```

For such cases, the `@property` at-rule can prevent unexpected results by allowing you to define the initial value of the property:

```
<p>This paragraph is initially black.</p>
```

```
@property --text-color {
  syntax: "<color>";
  inherits: false;
  initial-value: teal;
}

:root {
  --text-color: 16px;
}

p {
  font-weight: bold;
  color: blue;
}

p {
  color: var(--text-color);
}
```

Values in JavaScript

To use the values of custom properties in JavaScript, it is just like standard properties.

```
// get variable from inline style
element.style.getPropertyValue("--my-var");

// get variable from wherever
getComputedStyle(element).getPropertyValue("--my-var");

// set variable on inline style
element.style.setProperty("--my-var", jsVar + 4);
```

See also

- `@property` at-rule
- `var()`
- CSS custom properties for cascading variables module
- Custom property syntax
- CSS Properties and Values API

CSS color adjustment

The **CSS color adjustment** module provides a model and controls automatic color adjustment by the user agent to handle user preferences, such as "Dark Mode", contrast adjustment, and other color scheme preferences.

Together with the `@media` features `prefers-color-scheme`, `prefers-contrast` and `forced-colors`, this module defines how and when colors are automatically adjusted by the browser.

Reference

Properties

- `color-scheme`
- `forced-color-adjust`
- `print-color-adjust`

Related concepts

- `<color>` CSS data type
- Related `@media` features:
  - `prefers-color-scheme`
  - `prefers-contrast`
  - `forced-colors`
- Properties affected by forced colors mode
  - `accent-color`
  - `background-color`
  - `background-image`
  - `border-color`
  - `box-shadow`
  - `caret-color`
  - `color`
  - `color-scheme`
  - `column-rule-color`
  - `fill`
  - `flood-color`
  - `flood-opacity`
  - `lighting-color`
  - `outline-color`
  - `scrollbar-color`
  - `stop-color`
  - `stroke`
  - `text-decoration-color`
  - `text-emphasis-color`
  - `text-shadow`
  - `-webkit-tap-highlight-color`

See also

- CSS colors module

CSS colors

The **CSS colors** module defines colors, color types, color blending, opacity, and how you can apply these colors and effects to HTML content.

While this module has only two CSS properties, `color` and `opacity`, over 20 CSS and SVG properties, CSS images, at-rules, and @media rules depend on these two properties.

Colors in action

The color syntax converter below shows the values of the currently selected color in red-green-blue (RGB), hexadecimal (HEX), hue, saturation, and lightness (HSL), and hue, whiteness, and blackness (HWB) CSS color formats. All the RGB, HEX, HSL, and HWB color values here, while written differently, represent the same color value.

Selecting a color via the color picker and an opacity via the slider updates the RGB, HEX, HSL, and HWB values. When you choose a new color or opacity value, the color of the background and the slider update via the CSS properties `background-color` and `accent-color`, respectively.

To see the code for this color syntax converter, view the source on GitHub.

Reference

Properties

- `color`
- `opacity`

At-rules and descriptors

The CSS colors module also introduces the `@color-profile` at-rule, along with its `components`, `rendering-intent` and `src` descriptors. Currently, no browsers support these features.

Functions

- Color functions:
  - `rgb()`
  - `hsl()`
  - `hwb()`
  - `lab()`
  - `lch()`
  - `oklab()`

    - `oklch()`
    - `color()`
- `alpha()`
- `color-mix()`
- `contrast-color()`
- `light-dark()`

The CSS color modules also introduce the `device-cmyk()` function. Currently, no browsers support this feature.

Data types

- `<color>`
- `<color-function>`
- `<hex-color>`
- `<named-color>`
- `<alpha-value>`
- `<hue>`
- `<system-color>`
- `<colorspace-params>`

Glossary terms and keywords

- Color space
- `currentColor`
- Interpolation
- RGB
- `transparent`

Interfaces

The CSS color module also introduces the `CSSColorProfileRule` interface. Currently, no browsers support this feature.

Guides

Applying color to HTML elements using CSS

A guide to using CSS to apply color to a variety of types of content, including all CSS properties that accept `<color>` as a value.

CSS color values

An overview of color spaces and the different `<color>` functional notations available in CSS.

Using color wisely

Color theory and resources, including finding the right colors to create an accessible color palette, contrast, and printing in color.

Using relative colors

This article explains relative CSS color syntax, shows what the different options are, and looks at some illustrative examples.

Understanding color and luminance

Color perception and using colors with color insensitive (color blind) users, reduced vision users and users with vestibular disorders or other neurological disorders in mind.

WCAG 1.4.1: Color contrast

Explanation of contrast requirements between background and foreground content to ensure legibility.

CSS value serialization

How CSSOM APIs serialize color and other values into standardized string representations.

Related concepts

- CSS properties that are part of other specifications:
    - `accent-color`
    - `background-color`
    - `background-image`
    - `border-color`
    - `box-shadow`
    - `caret-color`
    - `color`
    - `color-scheme`
    - `column-rule-color`
    - `dynamic-range-limit`
    - `outline-color`
    - `scrollbar-color`
    - `text-decoration-color`
    - `text-emphasis-color`
    - `text-shadow`
    - `-webkit-tap-highlight-color`
- `dynamic-range-limit-mix()` function
- SVG color properties that are part of other specifications:
    - `fill`
    - `flood-color`
    - `lighting-color`
    - `stop-color`
    - `stroke`
- SVG `color` attribute
- Color wheel glossary term
- Interpolation glossary term
- The `@font-palette-values` at-rule `override-colors` descriptor
- The `@color-profile` at-rule
- The `color-gamut` @media feature

- The `forced-colors` @media feature

See also

- CSS color adjustment module and `print-color-adjust`
- `<gradient>` defined in CSS images module
- The `VideoColorSpace` interface
- `<feColorMatrix>` SVG element
- Canvas API: applying styles and colors
- Tools:
  - Color format converter
  - Color mixer

Applying color to HTML elements using CSS

With CSS, there are lots of ways to add color to your HTML elements to create the look you want. This guide is a primer introducing how CSS can be used to apply colors to HTML elements. This guide includes lists of the CSS properties that set color in their values and how to use colors both in stylesheets and in other ways.

> **Note:** It is important to use colors wisely. Always select appropriate colors, ensuring the contrast between text and the background is sufficient to ensure legibility, and always keep the needs of people with differing visual capabilities in mind.

To learn more about CSS colors as a data type, see the CSS `<color>` data type reference and the CSS color values guide.

Properties that can have color

At the element level, everything in HTML can have color applied to it. Let's look at the different items rendered on the page — such as text, borders, etc. We'll provide lists of the CSS properties that apply color to each.

At a fundamental level, the `color` property defines the foreground color of an HTML element's content and the `background-color` property defines the element's background color. These can be used on just about any element.

Text

Whenever an element is rendered, these properties are used to determine the color of the text, its background, and any decorations on the text.

`color`

: The color to use when drawing the text and any text decorations (such as the addition of under- or overlines, strike-through lines, and so forth).

`background-color`

: The text's background color.

`text-shadow`

: Configures a shadow effect to apply to text. Among the options for the shadow is the shadow's base color (which is then blurred and blended with the background based on the other parameters). See Text drop shadows to learn more.

`text-decoration-color`

: The default text decorations (such as underlines, strikethroughs, etc.) color is `currentColor`. This keyword represents the current value of the `color` property. However, you can override that value and use a different color for them with the `text-decoration-color` property.

`text-emphasis-color`

: The color to use when rendering emphasis symbols adjacent to each character in the text. This is used primarily when drawing text for East Asian languages.

`caret-color`

: The color to use when drawing the caret (sometimes referred to as the text input cursor) within the element. This is only useful in elements that are editable, such as `<input>` and `<textarea>` or elements whose HTML `contenteditable` attribute is set to `true`.

Boxes

Every element is a box with some sort of content, and has a background and a border in addition to whatever contents the box may have.

Borders

: See the Borders section for a list of the CSS properties you can use to set the colors of a box's borders.

`background-color`

: The background color to use in areas of the element that have no foreground content.

`box-shadow`

: Configures inset shadow and drop shadow effects on the box. Among the options for each shadow is the shadow's base color (which is then blurred and blended with any background based on the other parameters).

`column-rule-color`

: The color to use when drawing the line separating columns of text when using CSS multi-column layout.

`outline-color`

: The color to use when drawing an outline around the outside of the element. This outline is different from the border in that it doesn't get space set aside for it in the document. Outlines do not participate in the box model, overlapping other content. Outlines are generally used as focus indicators, indicating which element currently has focus and will receive keyboard input events.

Borders

Any element can have a border drawn around it. A basic element border is a line drawn around the edges of the element's content. See The box model to learn about the relationship between elements and their borders, and the article Styling borders using CSS to learn more about applying styles to borders.

You can use the `border` shorthand property, which lets you configure everything about the border in one shot (including non-color features of borders, such as its width, style (solid, dashed, etc.), and so forth.

`border-color` shorthand

: Specifies a single color to use for every side of the element's border.

`border-left-color`, `border-right-color`, `border-top-color`, and `border-bottom-color`

Lets you set the color of the corresponding side of the element's border.

`border-block-start-color` and `border-block-end-color`

With these, you can set the color used to draw the borders which are closest to the start and end of the block the border surrounds. In a left-to-right writing mode (such as the way English is written), the block start border is the top edge and the block end is the bottom. This differs from the inline start and end, which are the left and right edges (corresponding to where each line of text in the box begins and ends).

`border-inline-start-color` and `border-inline-end-color`

These let you color the edges of the border closest to the beginning and the end of the start of lines of text within the box. Which side this is will vary depending on the `writing-mode`, `direction`, and `text-orientation` properties, which are typically (but not always) used to adjust text directionality based on the language being displayed. For example, if the box's text is being rendered right-to-left, then the `border-inline-start-color` is applied to the right side of the border.

Specifying colors as values in stylesheets

Now that you know which CSS properties let you apply color to elements, you can start adding colors to your websites. Let's look at some examples of using color within a style sheet. In this example, we use several previously mentioned properties, with the concept of applying colors in CSS being the same no matter the property.

Let's look at the result first, before going on to look at the code we need to create it:

HTML

The HTML responsible for creating the above example is shown here:

```
<div class="wrapper">
  <div class="boxLeft">
    <p>This is the first box.</p>
  </div>
  <div class="boxRight">
    <p>This is the second box.</p>
  </div>
</div>
```

Here we have a wrapper `<div>` containing two child `<div>`s, each with a single child paragraph (`<p>`). Each content `<div>` is given a different look and feel.

CSS

Let's look at the CSS that creates the above result a piece at a time.

> **Note:** We are using multiple different CSS color value types in this example to demonstrate their use. This is not recommended for production code. When writing CSS, use the most intuitive value type for you and your team.

```
.wrapper {
  height: 110px;
  padding: 10px;
  display: flex;
  gap: 10px;
  text-align: center;
  font:
    28px "Marker Felt",
    "Zapfino",
    cursive;
  border: 6px solid mediumturquoise;
}

div {
  flex: 1;
}
```

The `.wrapper` class is used to assign styles to the `<div>` that encloses all of our other content. This establishes the height of the container using `height`, allowing the width of this block-level element to default to 100% of its parent. Setting the `display` to `flex` and adding a `10px` `gap` creates a flex container to lay out the children side by side with a gap between all the container's children. We use `flex` to let the flex children grow to fill the container; it doesn't affect the flex container itself.

Of more interest to our discussion here is the use of the `border` property to establish a border around the outside edge of the element. This border is a solid line, 6 pixels wide, in the named color `mediumturquoise`.

Within our wrapper, we have a left box and a right box.

```
.boxLeft {
  background-color: rgb(245 130 130);
  outline: 2px solid darkred;
}
```

The `.boxLeft` class, used to style the box on the left, sets up the color of the background and the outline:

- The box's background color is set by changing the value of the CSS `background-color` property to `rgb(245 130 130)`, using the `rgb()` functional notation.
- An outline is defined for the box. Unlike the more commonly used `border`, `outline` doesn't affect layout at all; it draws over the top of whatever may happen to be outside the element's box instead of making room as `border` does. This outline is a solid, dark red line that's two pixels thick. Note the use of the `darkred` keyword when specifying the color.
- Notice that we're not explicitly setting the text color. That means the value of `color` will be inherited from the nearest containing element that defines it. By default, that's black.

```
.boxRight {
  background-color: hwb(270deg 63% 13%);
  outline: 4px dashed #6e1478;
  color: hsl(0deg 95% 95%);
  text-decoration-line: underline;
  text-decoration-style: wavy;
  text-decoration-color: #8f8;
  text-decoration: underline wavy #8f8;
  text-shadow: 2px 2px 3px black;
}
```

> **Note:** We included the `text-decoration-*` styles separately because Safari doesn't support `text-decoration` as a shorthand property.

Finally, the `.boxRight` class sets several styles on the box that's drawn to the right. Then the following colors are established (using five different ways of declaring color values):

- The `background-color` is set using `hwb()` functional notation — `hwb(270deg 63% 13%)`. This is a medium purple color.
- The box's `outline` is used to specify that the box should be enclosed in a four-pixel thick dashed line whose color is a somewhat deeper purple using the six-digit `<hex-color>` `#6e1478`.
- The foreground (text) color is specified by setting the `color` property using `hsl()` functional notation — `hsl(0deg 95% 95%)`. This is a very light pinkish color.
- We add a green wavy line under the text with the `text-decoration` shorthand, along with the longhand component for browser compatibility. We used the 3-digit `<hex-color>` `#8f8`, which is the equivalent of `#88ff88`.
- Finally, a bit of a shadow is added to the text using `text-shadow`. Its `color` parameter is set to `black`, a `<named-color>` value.

We used five different color syntaxes to demonstrate what is possible. In the real world, you and your team will preferably pick a preferred color notation, with everyone working on a code base using the same color syntax.

Other ways to use color

CSS isn't the only web technology that supports color. Other examples include:

The HTML Canvas API

> Lets you draw 2D bitmapped graphics in a `<canvas>` element. See our Canvas tutorial to learn more.

SVG (Scalable Vector Graphics)

> Lets you create images using commands that draw specific shapes, patterns, and lines. SVG commands are formatted as XML, and can be embedded directly into a web page or placed in the page using the `<img>` element, just like any other type of image.

WebGL

> The Web Graphics Library is an OpenGL ES-based API for drawing high-performance 2D and 3D graphics on the Web. See our WebGL tutorial to find out more. Also see WebGPU, a successor to WebGL for modern GPUs.

> **Note:** A few now obsolete HTML attributes accepted colors as values, such as `bgcolor` and `vlink`. These attributes only accepted `<named-color>` and three- or six-digit `<hex-color>` values.

See also

- `<color>` data type
- CSS color values guide
- Using color wisely
- CSS color module
- Drawing graphics

Color format converter

This tool lets you enter or pick a color and copy its corresponding value in any CSS color format. You can use the generated color value anywhere the `<color>` data type is supported in CSS. This tool can also help you understand the syntax of the different color notations.

How to use the tool

Enter a color in any format or pick one in the sRGB color space. You can also adjust the opacity (alpha channel).

Click the **Copy** button next to the corresponding color format to quickly copy its value. You can also click a color format name to learn more about it.

```
<div class="container">
  <dialog open>
    <p>
      <label for="color-text">
        Enter a color value in any format or pick a color:
      </label>
      <span class="color-inputs">
        <input type="text" id="color-text" />
        <input type="color" id="color-input" />
      </span>
    </p>
    <p>
      <label for="opacity-input">Adjust the opacity: </label>
      <input
        type="range"
        id="opacity-input"
        value="1"
        min="0"
        max="1"
        step="0.01" />
    </p>
    <hr />
    <table id="output-colors">
      <caption>
        Your color in different formats:
      </caption>
      <tbody>
        <tr id="rgb-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/rgb"
              target="_blank">
              RGB
            </a>
          </th>
          <td><button id="copy-rgb-button">Copy</button><span></span></td>
        </tr>
        <tr id="hex">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/hex-color"
              target="_blank">
              HEX
            </a>
          </th>
          <td><button id="copy-hex-button">Copy</button><span></span></td>
        </tr>
```

```
        <tr id="hsl-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/hsl"
              target="_blank">
              HSL
            </a>
          </th>
          <td><button id="copy-hsl-button">Copy</button><span></span></td>
        </tr>
        <tr id="hwb-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/hwb"
              target="_blank">
              HWB
            </a>
          </th>
          <td><button id="copy-hwb-button">Copy</button><span></span></td>
        </tr>
        <tr id="color-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/color"
              target="_blank">
              color()
            </a>
          </th>
          <td><button id="copy-color-button">Copy</button><span></span></td>
        </tr>
        <tr id="lab-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/lab"
              target="_blank">
              Lab
            </a>
          </th>
          <td><button id="copy-lab-button">Copy</button><span></span></td>
        </tr>
        <tr id="lch-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/lch"
              target="_blank">
              LCH
            </a>
          </th>
          <td><button id="copy-lch-button">Copy</button><span></span></td>
        </tr>
        <tr id="oklab-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/oklab"
              target="_blank">
              Oklab
            </a>
          </th>
          <td><button id="copy-oklab-button">Copy</button><span></span></td>
        </tr>
        <tr id="oklch-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/oklch"
              target="_blank">
              OkLCh
            </a>
          </th>
          <td><button id="copy-oklch-button">Copy</button><span></span></td>
        </tr>
        <tr id="xyz-d50-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Glossary/Color_space#xyz-d50"
              target="_blank">
              XYZ D50
            </a>
          </th>
          <td><button id="copy-xyz-d50-button">Copy</button><span></span></td>
        </tr>
        <tr id="xyz-d65-function">
          <th>
            <a
              href="https://developer.mozilla.org/en-US/docs/Glossary/Color_space#xyz"
              target="_blank">
              XYZ D65
            </a>
          </th>
          <td><button id="copy-xyz-d65-button">Copy</button><span></span></td>
        </tr>
      </tbody>
    </table>
  </dialog>
</div>
```

```
dialog {
  border: 1px solid;
  border-radius: 5px;
  box-shadow: 3px 3px 10px rgb(0 0 0 / 0.2);
  background-color: white;
  font-family: "Segue", "Helvetica", "Arial", sans-serif;
  margin-top: 5vh;
  width: 550px;
```

```
}

table {
  width: 100%;
}

td,
th {
  padding: 3px 0;
}

th {
  background-color: #ededed;
}

td {
  background-color: #dedede;
  font-family: monospace;
}

body {
  background:
    linear-gradient(
      -90deg,
      transparent 0 38%,
      47%,
      rgb(0 0 0 / 0.7) 50%,
      white 50% 100%
    ),
    conic-gradient(
      black 0 90deg,
      transparent 90deg 180deg,
      black 180deg 270deg,
      transparent 270deg 360deg
    );
  background-size:
    100% 100%,
    20px 20px;
  padding: 0;
  margin: 0;
}

.container {
  width: 100vw;
  height: 100vh;
}

.color-inputs {
  display: inline-flex;
  margin-top: 0.6rem;
  margin-bottom: 1.5rem;
}

button {
  margin-right: 0.5rem;
}

input[type="text"] {
  width: 300px;
  margin-right: 0.5rem;
}

input[type="range"] {
  width: 350px;
}

label {
  user-select: none;
}

p {
  display: flex;
  flex-flow: column;
  align-items: center;
  margin: 0;
}
```

```
const background = document.querySelector("body div");
const colorText = document.getElementById("color-text");
const colorPicker = document.getElementById("color-input");
const opacityPicker = document.getElementById("opacity-input");
const hexOutput = document.querySelector("#hex td > span");
const rgbFunctionOutput = document.querySelector("#rgb-function td > span");
const colorFunctionOutput = document.querySelector("#color-function td > span");
const hslFunctionOutput = document.querySelector("#hsl-function td > span");
const hwbFunctionOutput = document.querySelector("#hwb-function td > span");
const labFunctionOutput = document.querySelector("#lab-function td > span");
const okLabFunctionOutput = document.querySelector("#oklab-function td > span");
const lchFunctionOutput = document.querySelector("#lch-function td > span");
const okLchFunctionOutput = document.querySelector("#oklch-function td > span");
const xyzD50FunctionOutput = document.querySelector(
  "#xyz-d50-function td > span",
);
const xyzD65FunctionOutput = document.querySelector(
  "#xyz-d65-function td > span",
);

const LRGB_LMS_MATRIX = [
  [0.4122214708, 0.5363325363, 0.0514459929],
  [0.2119034982, 0.6806995451, 0.1073969566],
  [0.0883024619, 0.2817188376, 0.6299787005],
];
```

```
const LMS_LAB_MATRIX = [
  [+0.2104542553, +0.793617785, -0.0040720468],
  [+1.9779984951, -2.428592205, +0.4505937099],
  [+0.0259040371, +0.7827717662, -0.808675766],
];

// srgb-linear to xyz-d50
// matrix taken from http://www.brucelindbloom.com/index.html?Eqn_RGB_to_XYZ.html
const LRGB_XYZ_D50_MATRIX = [
  [0.4360747, 0.3850649, 0.1430804],
  [0.2225045, 0.7168786, 0.0606169],
  [0.0139322, 0.0971045, 0.7141733],
];

// srgb-linear to xyz-d65
// matrix taken from http://www.brucelindbloom.com/index.html?Eqn_RGB_to_XYZ.html
const LRGB_XYZ_D65_MATRIX = [
  [0.4124564, 0.3575761, 0.1804375],
  [0.2126729, 0.7151522, 0.072175],
  [0.0193339, 0.119192, 0.9503041],
];

const currentColor = { r: 186, g: 218, b: 85, alpha: 1.0 }; // #bada55

function multiplyByMatrix(matrix, tuple) {
  let i = [0, 0, 0];
  let j = matrix.length;
  let k = matrix[0].length;
  for (let l = 0; l < j; l++)
    for (let m = 0; m < k; m++) i[l] += matrix[l][m] * tuple[m];
  return i;
}

function rgbToLinear(c) {
  return c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
}

function intToHex(i) {
  return Math.floor(i).toString(16).padStart(2, "0").toLowerCase();
}

function rgbToHEXText(c) {
  return `#${intToHex(c.r)}${intToHex(c.g)}${intToHex(c.b)}`;
}

function rgbaToHEXAText(color) {
  const hexText = rgbToHEXText(color);
  if (color.alpha === 1.0) {
    return hexText;
  }
  const alpha = intToHex(color.alpha * 255);
  return `${hexText}${alpha}`;
}

function rgbaToHSLA(color) {
  let { r, g, b, alpha } = color;
  // Let's have r, g, b in the range [0, 1]
  r /= 255;
  g /= 255;
  b /= 255;
  const min = Math.min(r, g, b);
  const max = Math.max(r, g, b);
  const delta = max - min;
  let h, s, l;

  if (delta === 0) {
    h = 0;
  } else if (max === r) {
    h = ((g - b) / delta) % 6;
  } else if (max === g) {
    h = (b - r) / delta + 2;
  } else h = (r - g) / delta + 4;
  h = Math.round(h * 60);

  // We want an angle between 0 and 360°
  if (h < 0) {
    h += 360;
  }

  l = (max + min) / 2;
  s = delta === 0 ? 0 : delta / (1 - Math.abs(2 * l - 1));
  s = Number((s * 100).toFixed(1));
  l = Number((l * 100).toFixed(1));

  return { h, s, l, alpha };
}

function toHSLAText(color) {
  const { h, s, l, alpha } = rgbaToHSLA(color);
  return `hsl(${h.toFixed(0)} ${s.toFixed(0)}% ${l.toFixed(0)}%${
    alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""
  })`;
}

function rgbaToHWBAText(color) {
  let { h, s, l, alpha } = rgbaToHSLA(color);
  const chroma = s * (1 - Math.abs(l / 50 - 1));
  let W = (l - chroma / 2).toFixed(0);
  let B = (100 - l - chroma / 2).toFixed(0);
  return `hwb(${h} ${W}% ${B}%${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function rgbaToXYZD50(color) {
```

```
  let { r, g, b, alpha } = color;
  r = rgbToLinear(r / 255) * 255;
  g = rgbToLinear(g / 255) * 255;
  b = rgbToLinear(b / 255) * 255;

  const xyz = multiplyByMatrix(LRGB_XYZ_D50_MATRIX, [r, g, b]);
  return { x: xyz[0] / 255, y: xyz[1] / 255, z: xyz[2] / 255, alpha };
}

function rgbaToXYZD50Text(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD50(color);
  return `color(xyz-d50 ${xyz.x.toFixed(5)} ${xyz.y.toFixed(5)} ${xyz.z.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function rgbaToXYZD65(color) {
  let { r, g, b, alpha } = color;
  r = rgbToLinear(r / 255) * 255;
  g = rgbToLinear(g / 255) * 255;
  b = rgbToLinear(b / 255) * 255;

  const xyz = multiplyByMatrix(LRGB_XYZ_D65_MATRIX, [r, g, b]);
  return { x: xyz[0] / 255, y: xyz[1] / 255, z: xyz[2] / 255, alpha };
}

function rgbaToXYZD65Text(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD65(color);
  return `color(xyz-d65 ${xyz.x.toFixed(5)} ${xyz.y.toFixed(5)} ${xyz.z.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

const D65 = [0.3457 / 0.3585, 1, 0.2958 / 0.3585];
function xyzToLab(color) {
  let { x, y, z, alpha } = color;
  [x, y, z] = [x, y, z].map((v, i) => {
    v /= D65[i];
    return v > 0.0088564516 ? Math.cbrt(v) : v * 903.2962962962963 + 16 / 116;
  });
  return { l: 116 * y - 16, a: 500 * (x - y), b: 200 * (y - z), alpha };
}

function rgbaToLabText(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD50(color);
  const lab = xyzToLab(xyz);
  return `lab(${lab.l.toFixed(3)} ${lab.a.toFixed(3)} ${lab.b.toFixed(3)}${
    alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""
  })`;
}

function rgbToOklab(color) {
  let { r, g, b, alpha } = color;
  r = rgbToLinear(r / 255);
  g = rgbToLinear(g / 255);
  b = rgbToLinear(b / 255);
  const lms = multiplyByMatrix(LRGB_LMS_MATRIX, [r, g, b]).map((v) =>
    Math.cbrt(v),
  );

  const oklab = multiplyByMatrix(LMS_LAB_MATRIX, lms);
  return { l: oklab[0], a: oklab[1], b: oklab[2], alpha };
}

function toOkLabText(color) {
  let { alpha } = color;
  const oklab = rgbToOklab(color);
  return `oklab(${oklab.l.toFixed(5)} ${oklab.a.toFixed(5)} ${oklab.b.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function labToLCH(color) {
  const { l, a, b, alpha } = color;
  const c = Math.sqrt(a * a + b * b);
  let h = Math.atan2(b, a) * (180 / Math.PI);
  if (h < 0) {
    h += 360;
  }
  return { l, c, h, alpha };
}

function toLCHText(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD50(color);
  const lab = xyzToLab(xyz);
  const lch = labToLCH(lab);
  return `lch(${lch.l.toFixed(3)} ${lch.c.toFixed(3)} ${lch.h.toFixed(3)}${
    alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""
  })`;
}

function rgbaToOkLCh(color) {
  const lab = rgbToOklab(color);
  const oklch = labToLCH(lab);
  return { l: oklch.l, c: oklch.c, h: oklch.h, alpha: color.alpha };
}

function toOkLChText(color) {
  let { alpha } = color;
```

```
  const oklch = rgbaToOkLCh(color);
  return `oklch(${oklch.l.toFixed(5)} ${oklch.c.toFixed(5)} ${oklch.h.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function colorToRGBA(c) {
  const ctx = new OffscreenCanvas(1, 1).getContext("2d");
  ctx.fillStyle = c;
  ctx.fillRect(0, 0, 1, 1);
  const data = ctx.getImageData(0, 0, 1, 1).data;
  return {
    r: data[0],
    g: data[1],
    b: data[2],
    alpha: data[3] / 255,
  };
}

function updateOutput(c) {
  background.style.backgroundColor = rgbaToHEXAText(c);
  hexOutput.innerText = rgbaToHEXAText(c);
  rgbFunctionOutput.innerText = `rgb(${c.r} ${c.g} ${c.b}${c.alpha < 1 ? ` / ${c.alpha.toFixed(3)}` : ""})`;
  colorFunctionOutput.innerText = `color(srgb ${(c.r / 255).toFixed(3)} ${(c.g / 255).toFixed(3)} ${(c.b / 255).toFixed(3)}${c.alpha < 1 ? ` / ${c.alpha.toFixed(3)}` : ""})`;
  hslFunctionOutput.innerText = toHSLAText(c);
  hwbFunctionOutput.innerText = rgbaToHWBAText(c);
  labFunctionOutput.innerText = rgbaToLabText(c);
  xyzD50FunctionOutput.innerText = rgbaToXYZD50Text(c);
  xyzD65FunctionOutput.innerText = rgbaToXYZD65Text(c);
  lchFunctionOutput.innerText = toLCHText(c);
  okLabFunctionOutput.innerText = toOkLabText(c);
  okLchFunctionOutput.innerText = toOkLChText(c);
}

function init() {
  colorText.addEventListener("input", (e) => {
    const color = colorToRGBA(e.target.value);
    Object.assign(currentColor, color);
    colorPicker.value = rgbToHEXText(currentColor);
    opacityPicker.value = currentColor.alpha;
    updateOutput(currentColor);
  });

  colorPicker.addEventListener("input", (e) => {
    const text = e.target.value;
    currentColor.r = parseInt(text.slice(1, 3), 16);
    currentColor.g = parseInt(text.slice(3, 5), 16);
    currentColor.b = parseInt(text.slice(5, 7), 16);
    colorText.value = rgbaToHEXAText(currentColor);
    opacityPicker.value = currentColor.alpha;
    updateOutput(currentColor);
  });

  opacityPicker.addEventListener("input", (e) => {
    const value = parseFloat(e.target.value);
    currentColor.alpha = value;
    colorText.value = rgbaToHEXAText(currentColor);
    updateOutput(currentColor);
  });
  document.querySelectorAll("#output-colors button").forEach((button) => {
    button.addEventListener("click", (e) => {
      const text = e.target.nextElementSibling.innerText;
      navigator.clipboard.writeText(text);
      e.target.innerText = "Copied!";
      setTimeout(() => {
        e.target.innerText = "Copy";
      }, 1000);
    });
  });

  colorText.value = rgbaToHEXAText(currentColor);
  colorPicker.value = rgbToHEXText(currentColor);
  opacityPicker.value = currentColor.alpha;
  updateOutput(currentColor);
}

init();
```

See also

- Applying color with CSS properties
- CSS color values
- Using color wisely
- Using relative colors
- Understanding color and luminance
- WCAG 1.4.1: Color contrast
- Learn: Styling backgrounds and borders using CSS
- Learn accessibility: Color and color contrast

Color mixer

This tool lets you mix two colors in any color space using the `color-mix()` function and copy the resulting color in any CSS color format.

The two input colors, `color-one` and `color-two`, are shown on the outside, and the mixed color returned by the function is shown in the middle. Click the outer swatches to select new colors to mix. Use the sliders to change the percentages of each input color included in the mix. Use the drop-down menu to change the color space of the function's output. The resulting color's values in different formats appear in a table following the widget and can be copied. You can also click a color format name to learn more about it.

```
<div id="color-mixer">
  <div>color-one</div>
  <div>mixed-color</div>
```

```
  <div>color-two</div>

  <div id="color-background">
    <button
      id="color-one"
      type="button"
      class="color-one"
      aria-label="Open the color one picker dialog"></button>
    <div id="mixed-color"></div>
    <button
      id="color-two"
      type="button"
      class="color-two"
      aria-label="Open the color two picker dialog"></button>
  </div>
  <div>
    <label
      id="percentage-one-label"
      for="percentage-one"
      aria-label="color one percentage">
      50%
    </label>
    <input id="percentage-one" type="range" name="percent1" step="0.01" />
  </div>
  <div>
    <label
      id="percentage-two-label"
      for="percentage-two"
      aria-label="color two percentage">
      50%
    </label>
    <input id="percentage-two" type="range" name="percent2" step="0.01" />
  </div>

  <label for="color-space">Color space: </label>
  <select id="color-space">
    <option value="srgb">sRGB</option>
    <option value="srgb-linear">sRGB-linear</option>
    <option value="hsl">HSL</option>
    <option value="hwb">HWB</option>
    <option value="lab">Lab</option>
    <option value="lch">LCH</option>
    <option value="oklab" selected>Oklab</option>
    <option value="oklch">OkLCh</option>
    <option value="xyz-d50">XYZ-D50</option>
    <option value="xyz-d65">XYZ-D65</option>
  </select>

  <label id="interpolation-method-label" for="interpolation-method">
    Hue interpolation method:
  </label>
  <select id="interpolation-method">
    <option value="shorter hue" selected>shorter hue</option>
    <option value="longer hue">longer hue</option>
    <option value="increasing hue">increasing hue</option>
    <option value="decreasing hue">decreasing hue</option>
  </select>
  <div id="mix-output-text"></div>
  <dialog id="picker-dialog">
    <div>Pick a color</div>
    <table>
      <tbody>
        <tr>
          <td><label for="color-text">Enter color:</label></td>
          <td><input type="text" id="color-text" value="#bada55" /></td>
        </tr>
        <tr>
          <td><label for="color-input">Pick color:</label></td>
          <td><input type="color" id="color-input" value="#bada55" /></td>
        </tr>
        <tr>
          <td><label for="color-opacity">Select an opacity:</label></td>
          <td>
            <input
              type="range"
              id="color-opacity"
              value="1"
              min="0"
              max="1"
              step="0.01" />
          </td>
        </tr>
      </tbody>
    </table>
    <div><button id="picker-ok-button">Ok</button></div>
  </dialog>
</div>
<hr />
<table id="output-colors">
  <caption>
    Mixed color in different formats:
  </caption>
  <tbody>
    <tr id="rgb-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/rgb"
          target="_blank">
          RGB
        </a>
      </th>
      <td><button id="copy-rgb-button">Copy</button><span></span></td>
    </tr>
```

```
    <tr id="hex">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/hex-color"
          target="_blank"
          >HEX</a
        >
      </th>
      <td><button id="copy-hex-button">Copy</button><span></span></td>
    </tr>
    <tr id="hsl-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/hsl"
          target="_blank">
          HSL
        </a>
      </th>
      <td><button id="copy-hsl-button">Copy</button><span></span></td>
    </tr>
    <tr id="hwb-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/hwb"
          target="_blank">
          HWB
        </a>
      </th>
      <td><button id="copy-hwb-button">Copy</button><span></span></td>
    </tr>
    <tr id="color-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/color"
          target="_blank">
          color()
        </a>
      </th>
      <td><button id="copy-color-button">Copy</button><span></span></td>
    </tr>
    <tr id="lab-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/lab"
          target="_blank">
          Lab
        </a>
      </th>
      <td><button id="copy-lab-button">Copy</button><span></span></td>
    </tr>
    <tr id="lch-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/lch"
          target="_blank">
          LCH
        </a>
      </th>
      <td><button id="copy-lch-button">Copy</button><span></span></td>
    </tr>
    <tr id="oklab-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/oklab"
          target="_blank">
          Oklab
        </a>
      </th>
      <td><button id="copy-oklab-button">Copy</button><span></span></td>
    </tr>
    <tr id="oklch-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/oklch"
          target="_blank">
          OkLCh
        </a>
      </th>
      <td><button id="copy-oklch-button">Copy</button><span></span></td>
    </tr>
    <tr id="xyz-d50-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Glossary/Color_space#xyz-d50"
          target="_blank">
          XYZ D50
        </a>
      </th>
      <td><button id="copy-xyz-d50-button">Copy</button><span></span></td>
    </tr>
    <tr id="xyz-d65-function">
      <th>
        <a
          href="https://developer.mozilla.org/en-US/docs/Glossary/Color_space#xyz"
          target="_blank">
          XYZ D65
        </a>
      </th>
      <td><button id="copy-xyz-d65-button">Copy</button><span></span></td>
    </tr>
  </tbody>
</table>
```

```
#picker-dialog[open] {
  position: fixed;
  top: 30%;
  z-index: 100;
  padding: 1rem;
  width: 70vw;
  background: #dddddd;
  display: flex;
  flex-flow: column;
  align-items: center;
}

#picker-dialog #color-opacity {
  width: 200px;
}

#picker-dialog > div:first-child {
  font-weight: bold;
  margin-bottom: 1rem;
}

#picker-dialog > div {
  width: max-content;
  margin: 0.4rem auto;
  white-space: nowrap;
}

#picker-dialog #color-text {
  width: 200px;
}

#picker-dialog label {
  margin: 1rem 0;
}

#picker-dialog button {
  width: 100px;
  margin-top: 1rem;
}

dialog#picker-dialog table {
  width: fit-content;
}

#color-mixer {
  margin: 0.5rem;
  font-family: sans-serif;
  display: grid;
  grid-template-columns: 150px 1fr 1fr 150px;
  grid-template-areas:
    "color-one-label   mix-label        mix-label           color-two-label"
    "color-background  color-background color-background    color-background"
    "percent-one       percent-one      percent-two         percent-two"
    "color-space-label color-space      interpolation-label interpolation"
    "mix-output-text   mix-output-text  mix-output-text     mix-output-text";
}

#color-mixer > * {
  padding: 0;
  margin: 0 auto;
}

#color-mixer > :nth-child(1) {
  grid-area: color-one-label;
}

#color-mixer > :nth-child(2) {
  grid-area: mix-label;
}

#color-mixer > :nth-child(3) {
  grid-area: color-two-label;
}

#color-mixer > :nth-child(4) {
  grid-area: color-background;
  background:
    linear-gradient(
      0deg,
      transparent 0 38%,
      47%,
      rgb(0 0 0 / 0.7) 50%,
      white 50% 100%
    ),
    conic-gradient(
      black 0 90deg,
      transparent 90deg 180deg,
      black 180deg 270deg,
      transparent 270deg 360deg
    );
  background-size:
    100% 100%,
    20px 20px;
  display: flex;
  align-items: center;
  margin: 0;
  padding: 0;
  border-radius: 10%;
}

#color-one {
  height: 100px;
  flex: 1 1 100%;
```

```
  border-radius: 20% 0 0 20%;
  cursor: pointer;
  border: none;
  z-index: 1;
  box-shadow: -5px 5px 5px grey;
  background-color: coral;
}

#mixed-color {
  flex: 2 0.7 100%;
  height: 100px;
}

#color-two {
  height: 100px;
  flex: 1 1 100%;
  border-radius: 0 20% 20% 0;
  cursor: pointer;
  border: none;
  z-index: 1;
  box-shadow: 5px 5px 5px grey;
  background-color: cyan;
}

#color-mixer > :nth-child(5) {
  grid-area: percent-one;
  width: 100%;
  display: flex;
}

#percentage-one-label,
#percentage-two-label {
  width: 2rem;
  text-align: end;
}

#color-mixer > :nth-child(6) {
  grid-area: percent-two;
  width: 100%;
  display: flex;
}

#color-mixer > :nth-child(7) {
  grid-area: color-space-label;
  margin: 0.5rem 0 0 auto;
}

#color-mixer > :nth-child(8) {
  grid-area: color-space;
  margin: 0.5rem auto 0 1rem;
}

#color-mixer > :nth-child(9) {
  grid-area: interpolation-label;
  margin: 0.5rem 0 0 auto;
}

#color-mixer > :nth-child(10) {
  grid-area: interpolation;
  margin: 0.5rem auto 0 1rem;
}

#color-mixer > :nth-child(11) {
  grid-area: mix-output-text;
  margin: 1rem 0;
  padding: 1rem 0;
  text-align: center;
  background-color: #dfdfde;
  font-family: monospace;
}

input[type="range"] {
  width: 80%;
}

label {
  margin: 1rem;
}

*:focus {
  outline: 2px dashed purple;
  outline-offset: 1px;
}

table {
  width: 100%;
}
caption {
  font-family: sans-serif;
}
th {
  padding: 5px 15px;
  background-color: #ededed;
}
td {
  background-color: #dedede;
  font-family: monospace;
  padding: 5px;
}

#output-colors th {
  padding-right: 0;
  width: 20%;
```

```
}

#output-colors td button {
  margin-right: 0.4rem;
}
```

```
const root = document.querySelector(":root");

const colorSpace = document.getElementById("color-space");
const interpolationMethod = document.getElementById("interpolation-method");
const interpolationMethodLabel = document.getElementById(
  "interpolation-method-label",
);

const colorOneButton = document.getElementById("color-one");
const colorTwoButton = document.getElementById("color-two");
const mixedColorDiv = document.getElementById("mixed-color");
const mixedOutputText = document.getElementById("mix-output-text");

const percentageOneLabel = document.getElementById("percentage-one-label");
const percentageTwoLabel = document.getElementById("percentage-two-label");
const percentageOneSlider = document.getElementById("percentage-one");
const percentageTwoSlider = document.getElementById("percentage-two");

// pop up dialog elements
const pickerDialog = document.getElementById("picker-dialog");
const colorTextInput = document.getElementById("color-text");
const colorInput = document.getElementById("color-input");
const colorOpacitySlider = document.getElementById("color-opacity");
const pickerOkButton = document.getElementById("picker-ok-button");

// color output spans
const hexOutput = document.querySelector("#hex td > span");
const colorFunctionOutput = document.querySelector("#color-function td > span");
const rgbFunctionOutput = document.querySelector("#rgb-function td > span");
const hslFunctionOutput = document.querySelector("#hsl-function td > span");
const hwbFunctionOutput = document.querySelector("#hwb-function td > span");
const labFunctionOutput = document.querySelector("#lab-function td > span");
const oklabFunctionOutput = document.querySelector("#oklab-function td > span");
const lchFunctionOutput = document.querySelector("#lch-function td > span");
const oklchFunctionOutput = document.querySelector("#oklch-function td > span");
const xyzD50FunctionOutput = document.querySelector(
  "#xyz-d50-function td > span",
);
const xyzD65FunctionOutput = document.querySelector(
  "#xyz-d65-function td > span",
);

const LRGB_LMS_MATRIX = [
  [0.4122214708, 0.5363325363, 0.0514459929],
  [0.2119034982, 0.6806995451, 0.1073969566],
  [0.0883024619, 0.2817188376, 0.6299787005],
];

const LMS_LAB_MATRIX = [
  [+0.2104542553, +0.793617785, -0.0040720468],
  [+1.9779984951, -2.428592205, +0.4505937099],
  [+0.0259040371, +0.7827717662, -0.808675766],
];

// srgb-linear to xyz-d50
// matrix taken from http://www.brucelindbloom.com/index.html?Eqn_RGB_to_XYZ.html
const LRGB_XYZ_D50_MATRIX = [
  [0.4360747, 0.3850649, 0.1430804],
  [0.2225045, 0.7168786, 0.0606169],
  [0.0139322, 0.0971045, 0.7141733],
];

// srgb-linear to xyz-d65
// matrix taken from http://www.brucelindbloom.com/index.html?Eqn_RGB_to_XYZ.html
const LRGB_XYZ_D65_MATRIX = [
  [0.4124564, 0.3575761, 0.1804375],
  [0.2126729, 0.7151522, 0.072175],
  [0.0193339, 0.119192, 0.9503041],
];

const polarColorSpaces = ["hsl", "hwb", "lch", "oklch"];
const colorOne = { r: 255, g: 127, b: 80, alpha: 1.0 };
const colorTwo = { r: 0, g: 255, b: 255, alpha: 1.0 };
let currentColor = null;

function rgbToLinear(c) {
  return c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
}

function intToHex(i) {
  return Math.floor(i).toString(16).padStart(2, "0").toLowerCase();
}

function rgbToHEXText(c) {
  return `#${intToHex(c.r)}${intToHex(c.g)}${intToHex(c.b)}`;
}

function colorToRGBA(c) {
  const ctx = new OffscreenCanvas(1, 1).getContext("2d");
  ctx.fillStyle = c;
  ctx.fillRect(0, 0, 1, 1);
  const data = ctx.getImageData(0, 0, 1, 1).data;
  return {
    r: data[0],
    g: data[1],
    b: data[2],
    alpha: data[3] / 255,
```

```
  };
}

function multiplyByMatrix(matrix, tuple) {
  let i = [0, 0, 0];
  let j = matrix.length;
  let k = matrix[0].length;
  for (let l = 0; l < j; l++)
    for (let m = 0; m < k; m++) i[l] += matrix[l][m] * tuple[m];
  return i;
}

function rgbaToHEXAText(color) {
  const hexText = rgbToHEXText(color);
  if (color.alpha === 1.0) {
    return hexText;
  }
  const alpha = intToHex(color.alpha * 255);
  return `${hexText}${alpha}`;
}

function rgbaToHSLA(color) {
  let { r, g, b, alpha } = color;
  // Let's have r, g, b in the range [0, 1]
  r /= 255;
  g /= 255;
  b /= 255;
  const min = Math.min(r, g, b);
  const max = Math.max(r, g, b);
  const delta = max - min;
  let h, s, l;

  if (delta === 0) {
    h = 0;
  } else if (max === r) {
    h = ((g - b) / delta) % 6;
  } else if (max === g) {
    h = (b - r) / delta + 2;
  } else h = (r - g) / delta + 4;
  h = Math.round(h * 60);

  // We want an angle between 0 and 360°
  if (h < 0) {
    h += 360;
  }

  l = (max + min) / 2;
  s = delta === 0 ? 0 : delta / (1 - Math.abs(2 * l - 1));
  s = Number((s * 100).toFixed(1));
  l = Number((l * 100).toFixed(1));

  return { h, s, l, alpha };
}

function toHSLAText(color) {
  const { h, s, l, alpha } = rgbaToHSLA(color);
  return `hsl(${h.toFixed(0)} ${s.toFixed(0)}% ${l.toFixed(0)}%${
    alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""
  })`;
}

function rgbaToHWBAText(color) {
  let { h, s, l, alpha } = rgbaToHSLA(color);
  const chroma = s * (1 - Math.abs(l / 50 - 1));
  let W = (l - chroma / 2).toFixed(0);
  let B = (100 - l - chroma / 2).toFixed(0);
  return `hwb(${h} ${W}% ${B}%${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function rgbaToXYZD50(color) {
  let { r, g, b, alpha } = color;
  r = rgbToLinear(r / 255) * 255;
  g = rgbToLinear(g / 255) * 255;
  b = rgbToLinear(b / 255) * 255;

  const xyz = multiplyByMatrix(LRGB_XYZ_D50_MATRIX, [r, g, b]);
  return { x: xyz[0] / 255, y: xyz[1] / 255, z: xyz[2] / 255, alpha };
}

function rgbaToXYZD50Text(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD50(color);
  return `color(xyz-d50 ${xyz.x.toFixed(5)} ${xyz.y.toFixed(5)} ${xyz.z.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function rgbaToXYZD65(color) {
  let { r, g, b, alpha } = color;
  r = rgbToLinear(r / 255) * 255;
  g = rgbToLinear(g / 255) * 255;
  b = rgbToLinear(b / 255) * 255;

  const xyz = multiplyByMatrix(LRGB_XYZ_D65_MATRIX, [r, g, b]);
  return { x: xyz[0] / 255, y: xyz[1] / 255, z: xyz[2] / 255, alpha };
}

function rgbaToXYZD65Text(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD65(color);
  return `color(xyz-d65 ${xyz.x.toFixed(5)} ${xyz.y.toFixed(5)} ${xyz.z.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
```

```
}

const D65 = [0.3457 / 0.3585, 1, 0.2958 / 0.3585];
function xyzToLab(color) {
  let { x, y, z, alpha } = color;
  [x, y, z] = [x, y, z].map((v, i) => {
    v /= D65[i];
    return v > 0.0088564516 ? Math.cbrt(v) : v * 903.2962962962963 + 16 / 116;
  });
  return { l: 116 * y - 16, a: 500 * (x - y), b: 200 * (y - z), alpha };
}

function rgbaToLabText(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD50(color);
  const lab = xyzToLab(xyz);
  return `lab(${lab.l.toFixed(3)} ${lab.a.toFixed(3)} ${lab.b.toFixed(3)}${
    alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""
  })`;
}

function rgbToOklab(color) {
  let { r, g, b, alpha } = color;
  r = rgbToLinear(r / 255);
  g = rgbToLinear(g / 255);
  b = rgbToLinear(b / 255);
  const lms = multiplyByMatrix(LRGB_LMS_MATRIX, [r, g, b]).map((v) =>
    Math.cbrt(v),
  );

  const oklab = multiplyByMatrix(LMS_LAB_MATRIX, lms);
  return { l: oklab[0], a: oklab[1], b: oklab[2], alpha };
}

function toOkLabText(color) {
  let { alpha } = color;
  const oklab = rgbToOklab(color);
  return `oklab(${oklab.l.toFixed(5)} ${oklab.a.toFixed(5)} ${oklab.b.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function labToLCH(color) {
  const { l, a, b, alpha } = color;
  const c = Math.sqrt(a * a + b * b);
  let h = Math.atan2(b, a) * (180 / Math.PI);
  if (h < 0) {
    h += 360;
  }
  return { l, c, h, alpha };
}

function toLCHText(color) {
  let { alpha } = color;
  const xyz = rgbaToXYZD50(color);
  const lab = xyzToLab(xyz);
  const lch = labToLCH(lab);
  return `lch(${lch.l.toFixed(3)} ${lch.c.toFixed(3)} ${lch.h.toFixed(3)}${
    alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""
  })`;
}

function rgbaToOkLCh(color) {
  const lab = rgbToOklab(color);
  const oklch = labToLCH(lab);
  return { l: oklch.l, c: oklch.c, h: oklch.h, alpha: color.alpha };
}

function toOkLChText(color) {
  let { alpha } = color;
  const oklch = rgbaToOkLCh(color);
  return `oklch(${oklch.l.toFixed(5)} ${oklch.c.toFixed(5)} ${oklch.h.toFixed(
    5,
  )}${alpha < 1.0 ? ` / ${alpha.toFixed(3)}` : ""})`;
}

function updateColorMix() {
  let colorMixFunction = "color-mix(in ";

  root.style.setProperty("--color-space", colorSpace.value);
  colorMixFunction += colorSpace.value;

  if (polarColorSpaces.includes(colorSpace.value)) {
    root.style.setProperty("--interpolation-method", interpolationMethod.value);
    colorMixFunction += ` ${interpolationMethod.value}, `;
  } else {
    root.style.setProperty("--interpolation-method", "");
    colorMixFunction += `, `;
  }
  root.style.setProperty(
    "--color-one",
    colorOneButton.style.getPropertyValue("background-color"),
  );
  root.style.setProperty("--percentage-one", `${percentageOneSlider.value}%`);
  colorMixFunction += `${rgbaToHEXAText(colorOne)} ${
    percentageOneSlider.value
  }%, `;

  root.style.setProperty(
    "--color-two",
    colorTwoButton.style.getPropertyValue("background-color"),
  );
  root.style.setProperty("--percentage-two", `${percentageTwoSlider.value}%`);
```

```
  colorMixFunction += `${rgbaToHEXAText(colorTwo)} ${
    percentageTwoSlider.value
  }%)`;

  mixedColorDiv.style.setProperty("background-color", colorMixFunction);

  mixedOutputText.innerText = colorMixFunction;
  displayOutputColors(window.getComputedStyle(mixedColorDiv).backgroundColor);
}

function displayOutputColors(color) {
  let rgbaColor = colorToRGBA(color);

  hexOutput.innerText = rgbaToHEXAText(rgbaColor);

  rgbFunctionOutput.innerText = `rgb(${Math.round(rgbaColor.r)} ${Math.round(
    rgbaColor.g,
  )} ${Math.round(rgbaColor.b)}`;
  if (rgbaColor.alpha && rgbaColor.alpha < 1.0) {
    rgbFunctionOutput.innerText += ` / ${rgbaColor.alpha.toFixed(3)}`;
  }
  rgbFunctionOutput.innerText += `)`;

  colorFunctionOutput.innerText = `color(srgb ${(rgbaColor.r / 255).toFixed(
    3,
  )} ${(rgbaColor.g / 255).toFixed(3)} ${(rgbaColor.b / 255).toFixed(3)}`;
  if (rgbaColor.alpha && rgbaColor.alpha < 1.0) {
    colorFunctionOutput.innerText += ` / ${rgbaColor.alpha.toFixed(3)}`;
  }
  colorFunctionOutput.innerText += `)`;

  hslFunctionOutput.innerText = toHSLAText(rgbaColor);

  hwbFunctionOutput.innerText = rgbaToHWBAText(rgbaColor);
  labFunctionOutput.innerText = rgbaToLabText(rgbaColor);
  xyzD50FunctionOutput.innerText = rgbaToXYZD50Text(rgbaColor);
  xyzD65FunctionOutput.innerText = rgbaToXYZD65Text(rgbaColor);
  lchFunctionOutput.innerText = toLCHText(rgbaColor);
  oklabFunctionOutput.innerText = toOkLabText(rgbaColor);
  oklchFunctionOutput.innerText = toOkLChText(rgbaColor);
}

function setColorToDialog() {
  colorTextInput.value = rgbaToHEXAText(currentColor);
  colorInput.value = rgbToHEXText(currentColor);
  colorOpacitySlider.value = currentColor.alpha;
}

function init() {
  percentageOneSlider.addEventListener("input", (e) => {
    percentageOneLabel.innerText = `${e.target.value}%`;
    updateColorMix();
  });

  percentageTwoSlider.addEventListener("input", (e) => {
    percentageTwoLabel.innerText = `${e.target.value}%`;
    updateColorMix();
  });

  colorSpace.addEventListener("change", (e) => {
    if (polarColorSpaces.includes(e.target.value)) {
      interpolationMethod.style.visibility = "visible";
      interpolationMethodLabel.style.visibility = "visible";
    } else {
      interpolationMethod.style.visibility = "hidden";
      interpolationMethodLabel.style.visibility = "hidden";
    }
    updateColorMix();
  });

  interpolationMethod.addEventListener("change", () => {
    updateColorMix();
  });

  interpolationMethod.style.visibility = "hidden";
  interpolationMethodLabel.style.visibility = "hidden";

  colorOneButton.addEventListener("click", () => {
    currentColor = colorOne;
    setColorToDialog();
    pickerDialog.showModal();
  });

  colorTwoButton.addEventListener("click", () => {
    currentColor = colorTwo;
    setColorToDialog();
    pickerDialog.showModal();
  });

  colorTextInput.addEventListener("input", (e) => {
    const color = colorToRGBA(e.target.value);
    Object.assign(currentColor, color);
    colorInput.value = rgbToHEXText(currentColor);
    colorOpacitySlider.value = currentColor.alpha;
  });

  colorInput.addEventListener("input", (e) => {
    const text = e.target.value;
    currentColor.r = parseInt(text.slice(1, 3), 16);
    currentColor.g = parseInt(text.slice(3, 5), 16);
    currentColor.b = parseInt(text.slice(5, 7), 16);
    colorTextInput.value = rgbaToHEXAText(currentColor);
    colorOpacitySlider.value = currentColor.alpha;
```

```
  });

  colorOpacitySlider.addEventListener("input", (e) => {
    const value = parseFloat(e.target.value);
    if (value >= 0 && value <= 1) {
      currentColor.alpha = value;
      colorTextInput.value = rgbaToHEXAText(currentColor);
    }
  });

  pickerOkButton.addEventListener("click", () => {
    colorOneButton.style.setProperty(
      "background-color",
      rgbaToHEXAText(colorOne),
    );
    colorTwoButton.style.setProperty(
      "background-color",
      rgbaToHEXAText(colorTwo),
    );
    pickerDialog.close();
    updateColorMix();
  });

  document.querySelectorAll("#output-colors button").forEach((button) => {
    button.addEventListener("click", (e) => {
      const text = e.target.nextElementSibling.innerText;
      navigator.clipboard.writeText(text);
      e.target.innerText = "Copied!";
      setTimeout(() => {
        e.target.innerText = "Copy";
      }, 1000);
    });
  });

  updateColorMix();
}

init();
```

See also

- `<color>`
- `<color-interpolation-method>`
- `<hue>`
- CSS relative colors

CSS color values

To represent a color in CSS, you have to find a way to translate the analog concept of "color" into a digital form that a computer can use. This is typically done by breaking the color down into components, such as amounts of different primary colors to mix together, or brightness and hue. Defined color models ensure that colors will appear the same no matter where they are rendered.

A color model is a mathematical model that represents colors using numeric values. Color models describe how to create the available colors within a color space. RGB was the first color model for the web. The `sRGB` color space of the RGB color model — the standard red, green, and blue color space — was created in 1996 for computer monitors and the web. A color space is a system for grouping colors so that describing any given color is consistent. If you transform a color between two different color spaces, it should look identical in both.

Originally, monitors were limited regarding how many colors they could render, and CSS colors were limited by those constraints, expanding as capabilities improved. With modern devices no longer limited to RGB, we now also have color models based on human perception instead, providing a much wider gamut of colors. We can now describe color in CSS in several ways, and the options keep expanding.

This guide introduces the different `<color>` value types. For a more detailed discussion, see the reference links provided below.

Keywords

The web defines a set of standard color names that lets you use keywords instead of numeric representations to describe colors. This is a simpler albeit more limited approach — there may not be a keyword representing the exact color you want to use.

Color keywords include standard primary and secondary colors (such as `red`, `blue`, or `orange`), shades of gray (from `black` to `white`, including colors like `darkgray` and `lightgrey`), and a variety of other blended colors, including `lightseagreen`, `cornflowerblue`, and `rebeccapurple`. Named colors use the RGB model and are associated with the sRGB (`srgb`) color space.

There are over 160 named colors. There are named colors of special interest: `transparent` sets a transparent color value, while `currentColor` sets the current value of the CSS `color` property. There are also named `<system-color>` colors, such as `accentcolortext` and `buttonface`, that reflect the default color choices made by the user, the browser, or the operating system.

All color keywords are case-insensitive. See the `<named-color>` data type for more information on color keywords.

RGB values

There are two primary ways of defining an RGB color by its red, green, and blue components in CSS — hexadecimal and `rgb()` values. Like named colors, these methods use the RGB model and are associated with the sRGB (`srgb`) color space. However, they allow a much wider range of colors to be specified.

Hexadecimal string notation

Hexadecimal (hex) string notation uses a hexadecimal value to represent each component (red, green, and blue) of an RGB color. It may also include a fourth component: the alpha channel (or opacity).

A color in hexadecimal string notation always begins with the character `"#"`. After that comes the hexadecimal digits of the color code. The string is case-insensitive.

`"#rrggbb"`

Specifies a fully opaque color whose red component is the hexadecimal number `0xrr`, green component is `0xgg`, and blue component is `0xbb`.

`"#rrggbbaa"`

Specifies a color whose red component is the hexadecimal number `0xrr`, green component is `0xgg`, and blue component is `0xbb`. The alpha channel is specified by `0xaa`; the lower this value is, the more translucent the color becomes.

`"#rgb"`

Specifies a color whose red component is the hexadecimal number `0xrr`, green component is `0xgg`, and blue component is `0xbb`.

`"#rgba"`

Specifies a color whose red component is the hexadecimal number `0xrr`, green component is `0xgg`, and blue component is `0xbb`. The alpha channel is specified by `0xaa`; the lower this value is, the more translucent the color becomes.

As shown above, the red, green, and blue color components can each be represented as a double-digit hex value representing a number between 0 (`00`) and 255 (`FF`) or a single-digit hex value (a number between 0 (`0`) and 15 (`F`)).

> **Note:** The leading `0x` in the values above indicates a hexadecimal integer literal. Hexadecimal integers can include digits (`0` - `9`) and the letters `a` – `f` and `A` – `F`. The case of a character does not change its value. Therefore: `0xa` = `0xA` = `10` and `0xf` = `0xF` = `15`.

These two hex colors are equivalent color values; they're both red:

```
color: #ff0000;
color: #f00;
```

All components *must* be specified using the same number of digits. If you use the single-digit notation, the final color is computed by using each component's digit twice; that is, `"#D"` becomes `"#DD"` when drawing.

To make the values 25% opaque, add in the alpha channel value as shown below:

```
color: #ff000044;
color: #f004;
```

See the `<hex-color>` data type for more information on hexadecimal string notation for colors.

HTML color input type

There are many situations in which your website may need to let the user select a color. Perhaps you have a customizable user interface, or you're implementing a drawing app. Maybe you have editable text and need to let the user choose the text color. Or perhaps your app lets the user assign colors to folders or items. For such use cases, the `<input>` element has a `"color"` `type`, which renders a color picker control.

This example allows you to choose a color. Once a choice is made, the `border-color` is set to that color, and the value is displayed.

```
<div id="box">
  <label for="colorPicker">Border color:</label>
  <input type="color" value="#8888ff" id="colorPicker" />
  <output></output>
</div>
```

The HTML creates a box containing a color picker control (with a label created using the `<label>` element) and an empty `<output>` element into which we'll output the color's value using JavaScript. The color input's value is always a hexadecimal string.

```
#box {
  width: 500px;
  height: 100px;
  border: 5px solid rgb(245 220 225);
  padding: 4px 6px;
  font:
    16px "Lucida Grande",
    "Helvetica",
    "Arial",
    sans-serif;
}
```

The following JavaScript updates the border's color to match the color picker's initial value, then adds two event handlers to the `<input type="color">` element to respond to changes made to its value.

```
const colorPicker = document.querySelector("#colorPicker");
const box = document.querySelector("#box");
const output = document.querySelector("output");

box.style.borderColor = colorPicker.value;

colorPicker.addEventListener("input", (event) => {
  box.style.borderColor = event.target.value;
});

colorPicker.addEventListener("change", (event) => {
  output.innerText = `${colorPicker.value}`;
});
```

The `input` event is sent every time the value of the element changes; that is, every time the user adjusts the color in the color picker. Each time this event arrives, we set the box's border color to match the color picker's current value.

The `change` event is received when the color picker's value is finalized. We respond by setting the contents of the `<output>` to the string value of the selected color.

RGB functional notation

RGB (Red/Green/Blue) functional notation, like hexadecimal string notation, represents colors using their red, green, and blue components (and, optionally, an alpha channel component for opacity). However, instead of using a string, the color is defined using the CSS function `rgb()`. This function accepts 3 or 4 input parameters — red, green, and blue component values and an optional alpha channel value.

Legal values for each of these parameters are:

`red`, `green`, and `blue`

Each must be an `<number>` value between 0 and 255 (inclusive), a `<percentage>` from 0% to 100%, or the keyword `none`, which is equal to `0` in this case.

`alpha`

The alpha channel is specified as a percentage between `0%` (fully transparent) and `100%` (fully opaque), or a number between `0.0` (equivalent to `0%`) and `1.0` (equivalent to `100%`).

For example, a bright red that's 50% opaque can be represented as `rgb(255 0 0 / 50%)` or `rgb(100% 0 0 / 0.5)`.

See the `rgb()` color function for more information on the RGB functional notation.

Color functions with a hue component

The color functions that have a `<hue>` component — an `<angle>` from that color model's color wheel — include the `srgb` color functions `hsl()` and `hwb()`, CIElab's `lch()` function, and OKLab's `oklch()` color function. These color functions are more intuitive as the hue allows us to tell the difference or similarity between colors like red, orange, yellow, green, blue, etc.

HSL functional notation

The `hsl()` CSS color function was the first hue-based color function to be supported in browsers. `hsl()` is more intuitive than `rgb()` — it is generally easier to determine the effect of varying hue (`h`), saturation (`s`), and lightness (`l`) values than it is to declare specific colors via red, green, and blue channel values. In addition, HSL is similar to the HSB (hue, saturation, and brightness) color picker in Photoshop, which made it immediately familiar to many people when first supported.

The `hsl()` and `hwb()` sRGB color functions are both cylindrical. Hue defines the color as an `<angle>` on a circular color wheel. The diagram below shows an HSL color cylinder. Saturation is a percentage that defines how far the color is along a scale between completely grayscale and having the maximum possible amount of the given hue. As the value of lightness increases, the color transitions from the darkest to the lightest possible color (from black to white).

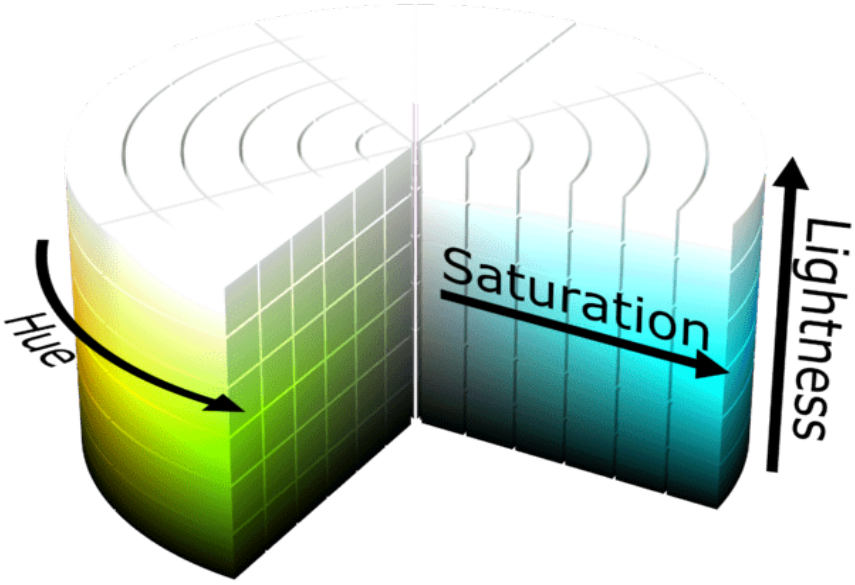


Image courtesy of user SharkD on Wikipedia, distributed under the CC BY-SA 3.0 license.

The value of the hue (`H`) component of an HSL (or HWB) color is an angle that starts at 0° as red, then moves through yellow, green, cyan, blue, and magenta, before ending up back at red again at 360°. The value can be specified in any `<angle>` unit supported by CSS, including degrees (`deg`), radians (`rad`), gradians (`grad`), or turns (`turn`). The hue value identifies what the base shade of the color is, but it doesn't control how vivid or dull, or how light or dark the color is.

The saturation (`S`) component of the color specifies the percentage of the final color comprised of the specified hue, with 100% being fully saturated and 0% being a complete lack of color (greyscale). The lightness (`L`) component specifies how light the color is along a sliding scale between completely black (`0%`) and completely white (`100%`). You can also optionally include an alpha channel, preceded by a slash (`/`) to make the color less than 100% opaque.

Here are some sample colors in HSL notation:

```
table {
  border: 1px solid black;
  font:
    16px "Open Sans",
    "Helvetica",
    "Arial",
    sans-serif;
  border-spacing: 0;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid black;
  padding: 4px 6px;
  text-align: left;
}

th {
  background-color: hsl(0 0% 75%);
}
```

```
<table>
  <thead>
    <tr>
      <th scope="col">Color in HSL notation</th>
      <th scope="col">Example</th>
    </tr>
  </thead>
  <tbody></tbody>
</table>
```

```
const colors = [
  "hsl(90deg 0% 50%)",
  "hsl(90 100% 50%)",
  "hsl(0.15turn 50% 75%)",
  "hsl(0.15turn 90% 75%)",
  "hsl(0.15turn 90% 50%)",
  "hsl(270deg 90% 50% / 50%)",
];

const tbody = document.querySelector("tbody");
for (const color of colors) {
  const tr = document.createElement("tr");
  const td1 = document.createElement("td");
```

```
  td1.appendChild(document.createElement("code")).textContent = color;
  const td2 = document.createElement("td");
  td2.style.backgroundColor = color;
  tr.appendChild(td1);
  tr.appendChild(td2);
  tbody.appendChild(tr);
}
```

The last value is semi-opaque; it includes the optional alpha value, preceded by a forward slash.

> **Note:** When you omit the hue's unit, it's assumed to be in degrees (`deg`).

HWB functional notation

The `hwb()` color function uses the same hue coordinate system as `hsl()`, with `0deg` being red. However, instead of `hsl()`'s lightness and saturation, `hwb()` functions specify whiteness (`W`) and blackness (`B`). This function is also fairly intuitive — allowing you to pick a hue and then mix in amounts of white and or black to achieve the desired color.

`W` and `B` values range from `0%` to `100%` (or `0` to `1`). If the combined value of `W` and `B` is 100% (or `1`) or greater, the color will be gray, similar to setting the `s` to `0%` with `hsl()`. As with `hsl()`, an optional alpha value can be included, preceded by a forward slash `/`.

Here are some examples of using HWB notation:

```
/* These examples all specify varying shades of a lime green. */
hwb(90 10% 10%)
hwb(90 50% 10%)
hwb(90deg 10% 10%)
hwb(1.5708rad 60% 0%)
hwb(.25turn 0% 40%)

/* Same lime green but with an alpha value */
hwb(90 10% 10% / 0.5)
hwb(90 10% 10% / 50%)
```

In the below examples, we set the same hues as in the `hsl()` examples, but we are adding whiteness and blackness to each hue via `hwb()` instead of saturation and lightness:

```
table {
  border: 1px solid black;
  font:
    16px "Open Sans",
    "Helvetica",
    "Arial",
    sans-serif;
  border-spacing: 0;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid black;
  padding: 4px 6px;
  text-align: left;
}

th {
  background-color: hwb(0 75% 25%);
}
```

```
<table>
  <thead>
    <tr>
      <th scope="col">Color in HWB notation</th>
      <th scope="col">Example</th>
    </tr>
  </thead>
  <tbody></tbody>
</table>
```

```
const colors = [
  "hwb(90deg 50% 50%)",
  "hwb(90 0% 0%)",
  "hwb(0.15turn 25% 0%)",
  "hwb(0.15turn 10% 25%)",
  "hwb(1turn 10% 65%)",
  "hwb(270deg 75% 10%)",
];

const tbody = document.querySelector("tbody");
for (const color of colors) {
  const tr = document.createElement("tr");
  const td1 = document.createElement("td");
  td1.appendChild(document.createElement("code")).textContent = color;
  const td2 = document.createElement("td");
  td2.style.backgroundColor = color;
  tr.appendChild(td1);
  tr.appendChild(td2);
  tbody.appendChild(tr);
}
```

LCH and OkLCh: CIELAB and Oklab color spaces

While `hsl()` and `hwb()` are intuitive, they have a major drawback. With these functions, every fully-saturated hue angle (`hsl(<angle> 100% 50%)` or `hwb(<angle> 0% 0%)`) has the same lightness, but that is not how human vision or monitors work. Putting white text on fully saturated blue (`hsl(240deg 100% 50%)`) is legible, but that same text on fully saturated yellow (`hsl(60deg 100% 50%)`) will not only be illegible, but may hurt your user's eyes. In these color functions, the lightness of a color is relative to other colors, not to human perception. In reality, not all hues have the same max saturation.

Wouldn't it be fantastic if you could simply change the hue channel of a color on a site without making text illegible? You can with color functions in the CIELAB and Oklab color spaces.

The CIELAB and Oklab color spaces represent the entire range of colors that humans can see. CIE Lab color functions include `lch()` and `lab()`. Oklab color functions include `oklch()` and `oklab()`. The primary purpose of these models is that they are uniform so that a given distance between any two points in the color space should appear equally different to a viewer. Oklab is a color space that uses the same model

type as CIELAB but is built using additional numerical optimization steps, so the values are considered more accurate than CIELAB. Because of this optimization, hues are more perceptually uniform.

The `lch()` and `oklch()` functions use lightness (`L`), chroma (`C`), and hue (`H`), and are discussed further in this section. The `lab()` and `oklab()` functions work differently, using lightness (`L`), red/green-ness (along the `a`-axis), and yellow/blue-ness (along the `b`-axis). These axes are referred to as rectangular coordinates. The main benefit of these color functions is that the "lightness" is perceived lightness; it is the brightness of a color as perceived by the human eye rather than the lightness as compared to other colors.

Similar to the sRGB hue color functions, the hue (`h`) value in `lch()` and `oklch()` is a number, an angle, or the keyword `none` (equivalent to `0deg`) representing the color's `<hue>` angle. However, the colors at each angle value are not the same. The angles corresponding to particular hues differ across the sRGB, CIELAB (used by `lch()`), and Oklab (used by `oklch()`) color spaces.

The following gradients demonstrate the hue colors at every angle form `0deg` to `360deg` in the sRGB, CIE Lab, and OKlab color spaces:

```
<p>sRGB (<code>hsl()</code> and <code>hwb()</code>)</p>
<div id="srgb"></div>
<p>CIE Lab (<code>lch()</code>)</p>
<div id="lch"></div>
<p>OKLab (<code>oklch()</code>)</p>
<div id="oklch"></div>
<p>
  <label><input type="checkbox" /> Toggle greyscale</label>
</p>
```

```
div:has(~ p input:checked) {
  filter: grayscale(100%);
}
p {
  margin: 0;
}
div {
  height: 50px;
  margin-bottom: 10px;
}
#srgb {
  background: linear-gradient(
    to right,
    hsl(0deg 100% 50%),
    hsl(90deg 100% 50%),
    hsl(180deg 100% 50%),
    hsl(270deg 100% 50%),
    hsl(360deg 100% 50%)
  );
}
#lch {
  background: linear-gradient(
    to right,
    lch(50% 100% 0deg),
    lch(50% 100% 90deg),
    lch(50% 100% 180deg),
    lch(50% 100% 270deg),
    lch(50% 100% 360deg)
  );
}
#oklch {
  background: linear-gradient(
    to right,
    oklch(50% 100% 0deg),
    oklch(50% 100% 90deg),
    oklch(50% 100% 180deg),
    oklch(50% 100% 270deg),
    oklch(50% 100% 360deg)
  );
}
```

You may notice how the brightness of the latter gradients is more even across the spectrum of hues than the sRGB gradient. Select the checkbox in the example above to convert the hue gradient to greyscale to make this more apparent.

Note also how the spread of blue values in CIE Lab is longer than in the other two. This is the difference between `lch()` and `oklch()`. The `lch()` blue spread is due to a bug that shifts the chroma and lightness of hue values between `270deg` and `330deg`. This was resolved in the oklab color space and therefore the `oklch()` color notation.

As discussed above, the hue (`H`) in the `lch()` and `oklch()` is an `<angle>`, `number` or the keyword `none`. The `lightness` is either a `<percentage>` or for `lch()` a number between `0` and `100` and for `oklch()` a number between `0` and `1`, with `0` or `0%` being the complete lack of lightness, which is black.

The `C` is a `<number>`, `<percentage>`, or the keyword `none` (equivalent to `0%`) is the color's chroma, or the "amount of color". This is similar to the `S` saturation value of the `hsl()` color function. The value `0` is the complete lack of chroma or saturation; resulting in a gray between white and black inclusive, depending on the lightness value. The number values are theoretically unbounded, with `100%` being equal to `150` for `lch()` and `0.4` with `oklch()`.

Like the other color functions, there is also an optional alpha transparency value, preceded by a slash (`/`).

The example below shows the effect of changing the lightness value in the `lch()` and `oklch()` functions.

```
/* Varying shades of pink */
.container {
  display: grid;
  font-family: sans-serif;
  font-size: 14px;
  color: white;
  grid-template-columns: repeat(6, 1fr);
  gap: 4px;
}

.dark-text {
  color: lch(1% 40 0deg);
}

.container div {
  border-radius: 8px;
  padding: 8px 4px;
}
```

```
<div class="container"></div>
```

```
const container = document.querySelector(".container");
for (let l = 0; l <= 100; l += 10) {
  const div = document.createElement("div");
  const usedL = l === 0 ? 1 : l === 100 ? 99 : l;
  div.textContent = div.style.backgroundColor = `lch(${usedL}% 40 0)`;
  if (usedL >= 80) div.classList.add("dark-text");
  container.appendChild(div);
}
container.appendChild(document.createElement("div"));
for (let l = 0; l <= 100; l += 10) {
  const div = document.createElement("div");
  const usedL = l === 0 ? 1 : l === 100 ? 99 : l;
  div.textContent = div.style.backgroundColor = `oklch(${usedL}% 0.12 0)`;
  if (usedL >= 80) div.classList.add("dark-text");
  container.appendChild(div);
}
```

Lab and OKLab

The `lab()` functional notation expresses a given color in the CIE L*a*b* color space. The `oklab()` function defines colors in the OKLab color space. These functions represent the entire range of colors that humans can see by specifying the color's lightness (`L`), a red/green axis value (`a`), a blue/yellow axis value (`b`), and an optional alpha transparency value.

Similar to `lch()` and `oklch()`, the `lightness` is either:

- A `<percentage>`, with `0%` being completely black and `100%` being completely white.
- A number between `0` and `100` for `lab()` and `0` and `1` for `oklab()`, where `0` is completely black and `1`/`100` is completely white.

The `a` value is `<number>` between `-125` and `125` for `lab()` or `-0.4` and `0.4` for `oklab()`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's distance along the a-axis in the color space, which defines how green (moving towards -100%) or red (moving towards +100%) the color is.

Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the ±125 or ±0.4 (±100%) limits. In practice, values cannot exceed ±160 or ±0.5, respectively.

The `b` value has the same constraints. It specifies the color's distance along the b-axis in the color space, which defines how blue (moving towards -100%) or yellow (moving towards +100%) the color is.

The following example demonstrates the effects of varying the `a` axis via a `lab()` function and the `b` axis via an `oklab()` function.

```
<div class="container"></div>
```

```
/* Varying shades of pink */
.container {
  display: grid;
  font-family: sans-serif;
  font-size: 14px;
  color: white;
  grid-template-columns: repeat(5, 1fr);
  gap: 4px;
}
.container div {
  border-radius: 8px;
  padding: 8px 4px;
}
```

```
const container = document.querySelector(".container");

for (let a = -100; a <= 100; a += 25) {
  const div = document.createElement("div");
  div.textContent = div.style.backgroundColor = `lab(50% ${a}% 0)`;
  container.appendChild(div);
}
container.appendChild(document.createElement("div"));
for (let b = -4; b <= 4; b++) {
  const div = document.createElement("div");
  div.textContent = div.style.backgroundColor = `oklab(50% 0 ${b / 10})`;
  container.appendChild(div);
}
```

Additional color functional notations

The <code>color()</code> function

If you want explicit control over color spaces when defining colors, you can use the `color()` function.

This is useful to describe a color for high-definition devices with wider color gamuts. For example, if you wanted to show the `display-p3 0 0 1` color, which is outside of the sRGB gamut, you could use a `@media color-gamut` at-rule to detect if the client's hardware supports colors in this range before trying to use it:

```
.vibrant {
  background-color: color(srgb 0 0 1);
}

@media (color-gamut: p3) {
  .vibrant {
    background-color: color(display-p3 0 0 1);
    /* Equivalent to out-of-gamut color(srgb 0 0 1.042) */
  }
}
```

Understanding `color()` is important when it comes to relative colors, discussed next. The older sRGB color notations discussed above — `hsl()`, `hwb()`, and `rgb()`— do not express the full spectrum of visible colors, while the `color()` function supports a much wider color gamut. As a result, when using the older functions types to define relative colors, the output color returned by querying `HTMLElement.style` property or the `CSSStyleDeclaration.getPropertyValue()` method will be a `color(srgb ...)` value.

To see an example of converting the `rgb()`, `hsl()`, `hwb()`, and other color formats, check out our color format converter tool.

Relative colors

Every color function listed above can be used to define **relative colors**, which allows `<color>` values to be defined relative to other existing colors, rather than defining a color value from scratch each time. This powerful feature enables the creation of complements to existing colors — such as lighter, darker, saturated, semi-transparent, or inverted variants of an original color. Relative colors provide an effective mechanism to create palettes and define color adjustments. See each color function page to learn more about their relative syntaxes.

As noted above, when using `rgb()`, `hsl()`, or `hwb()` to output a relative color, the output color will be a `color()` function in the `srgb` color space.

color-mix() function

The `color-mix()` function takes two color values of any syntax mentioned above, optionally with proportional percent values for each color, and returns the result of mixing them in a given colorspace by a given amount.

light-dark() function

The `light-dark()` function lets you specify two color values for a property intended for use in light and dark color schemes, respectively. Which one is set depends on whether the developer has set or the user has requested a light or dark color scheme. This is a shortcut function, allowing you to achieve the same results as the `prefers-color-scheme` media feature query but with less code.

See also

- Applying color to HTML elements using CSS
- Using color wisely
- Using relative colors
- Understanding color and luminance
- WCAG 1.4.1: Color contrast
- CSS color module

Using color wisely

Choosing the right colors for a website can be tricky, especially if you aren't well-grounded in art, design, or at least basic color theory. The wrong color choice can render your site unattractive, or worse, leave the content unreadable due to problems with contrast or conflicting colors. Using the wrong colors can result in your content being outright unusable by people with certain vision problems, particularly color blindness.

Finding the right colors

There are tools and processes available to help you pick a good color scheme. While they can't replace having a good designer helping you make these decisions, they can get you started.

Base color

The first step is to choose your **base color**. This color represents your website or its subject matter. Just as we associate green with the beverage Mountain Dew blue with the sky or the ocean, choosing an appropriate base color to represent your site is a good place to start. There are plenty of ways to select a base color; a few ideas include:

- A color that is naturally associated with the topic of your content, such as the existing color identified with a product or idea or a color representative of the emotion you wish to convey.
- A color that comes from imagery associated with your subject matter. If you're creating a website about a given item or product, choose a color that's physically present on that item.
- Browse websites that let you look at lots of existing color palettes and images to find inspiration.

Several useful browser extensions can help pick base colors. For example, the ColorZilla browser extension provides an eyedropper tool for picking colors from any webpage. It can also take averages of the colors of an area of a page.

An "average color" grab is useful because sometimes what may look like a solid block of color might actually be multiple related colors, such as grabbing the blue in a photograph of an ocean or the sky. A single pixel of blue selected from a photo may result in a color that looks out of place.

Fleshing out the palette

Once you have decided on your base color, the next step is to build a palette of appropriate colors to use alongside it. Several tools are available to apply color theory to your base color and output appropriate added colors. Online tools, like the free Adobe Color CC online color wheel can help you pick an accessible color palette.

Many of these tools can also apply filters to your palette so you can see what they look like to people with various forms of color blindness. See Color and accessibility for a brief explanation of why this matters.

When designing your palette, you'll probably also need to supplement it with some core neutral colors such as white (or nearly white), black (or nearly black), and one or more shades of gray.

> **Note:** Usually, you are better off using the smallest number of colors possible. Using color to highlight important content rather than adding color to everything will have more impact and your content will be more readable.

Color theory resources

A full review of color theory is beyond the scope of this article, however, there are plenty of articles about color theory available. We found the following resources particularly useful:

Color Science (Khan Academy in association with Pixar)

> An online course that introduces concepts such as what color is, how it's perceived, and how to use colors to express ideas. Presented by Pixar artists and designers.

Color theory on Wikipedia

> Wikipedia's entry on color theory has great information from a technical perspective. It likely won't help your color selection process, but is still full of useful information.

Color and accessibility

Make sure your content is accessible. There are several ways color can create an accessibility problem. Improper or careless use of color can result in a website or app that a percentage of your target audience may not be able to use adequately, resulting in lost traffic, lost business, and possibly even a public relations problem or a lawsuit. So it's important to consider your use of color carefully.

It's important to understand color and luminance and to always consider color blindness and vestibular disorders. There are several kinds; the most common is red-green color blindness, which causes people to be unable to differentiate between the colors red and green. There are others, too, ranging from an inability to tell the difference between certain colors to the total inability to see color at all. There are even color and animation combinations that can lead your photosensitive users to experience seizures.

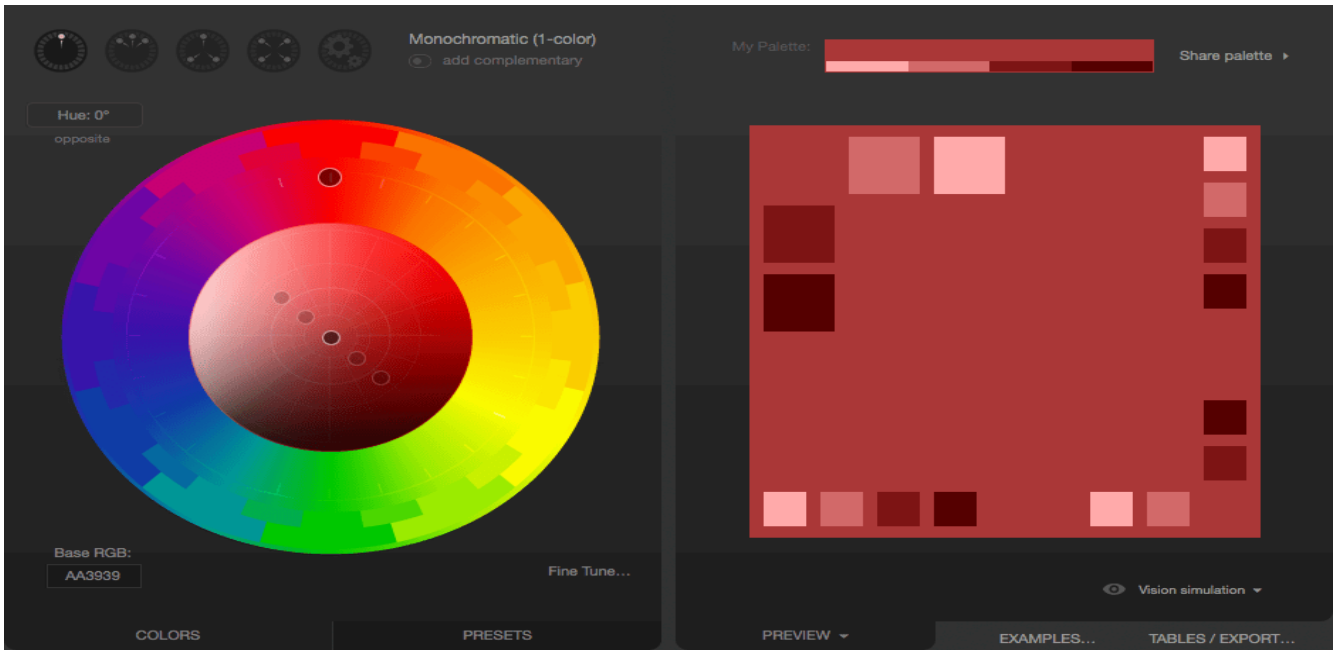
While higher color contrast is often a good thing when it comes to accessibility, when animating, especially rapidly, reducing color contrast on animating elements reduces seizure risk. If you include animations, use the `prefers-reduced-motion` `@media` query feature to reduce animations for users who have selected that preference.

That said, ensure you have enough color contrast between your background and foreground content to ensure legibility. Also, never use color as the only way to convey information. If, for example, you indicate the success of an operation with a green border around the associated UI element, and failure with a red border, users with red-green color blindness won't be able to use your site properly. Instead, use text and color indicators together to include those users. For example, a green check mark and a red cross mark would be better.

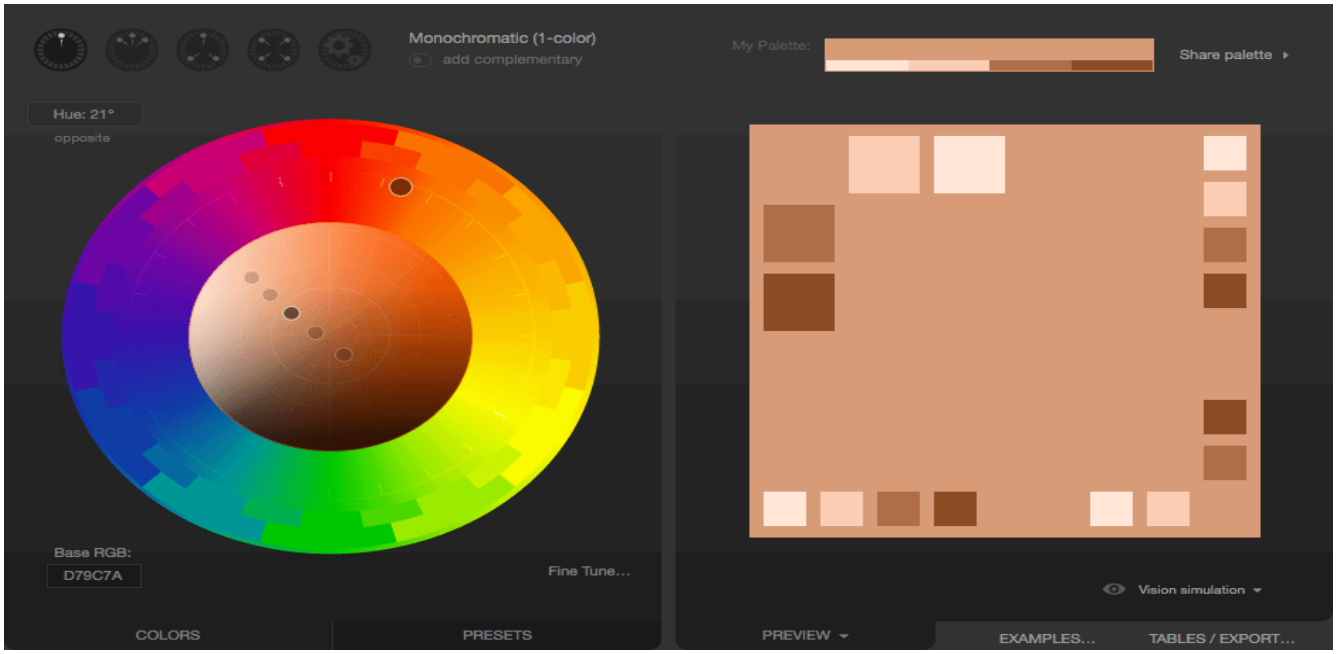
Palette design example

In this example, we will create an appropriate color palette for a website for a game that takes place on the planet Mars. A Google search for photos of Mars will output several color photos.

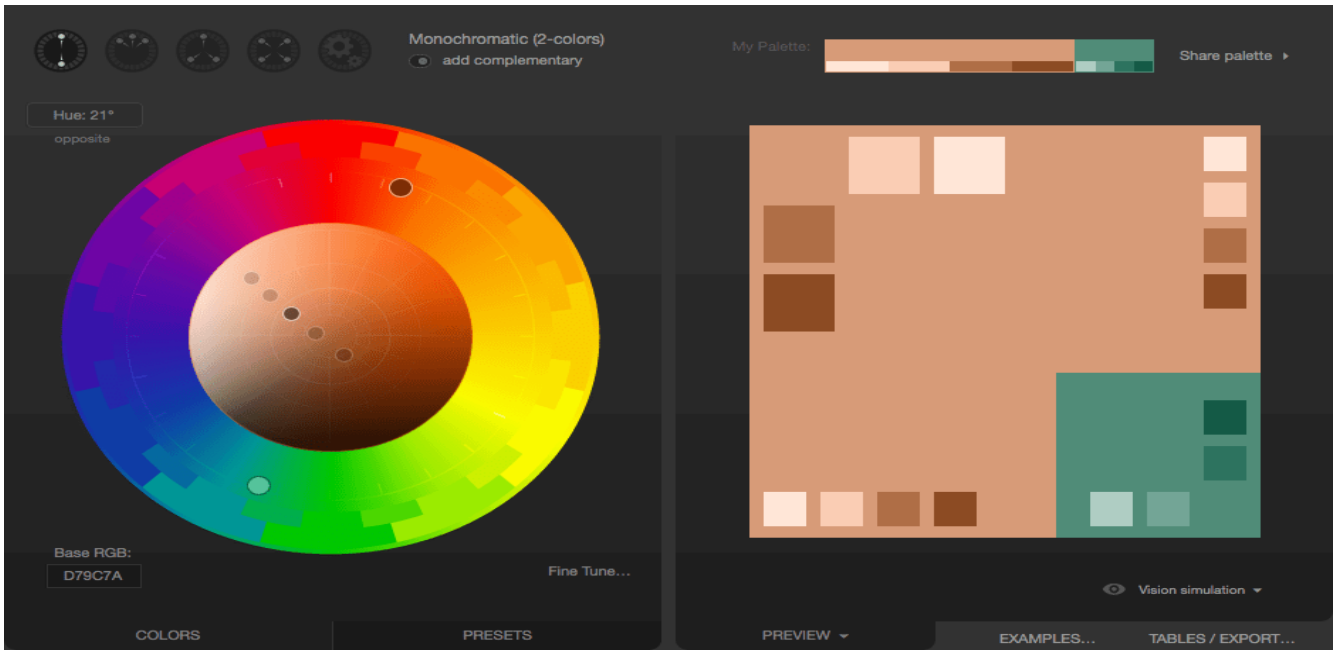
Use a color picker tool to select a color sample for the base color. For this example, we've selected `#D79C7A`, which is a rusty orange-red color. We can use Paletton to come up with the other colors for our palette. Upon opening Paletton, we see:



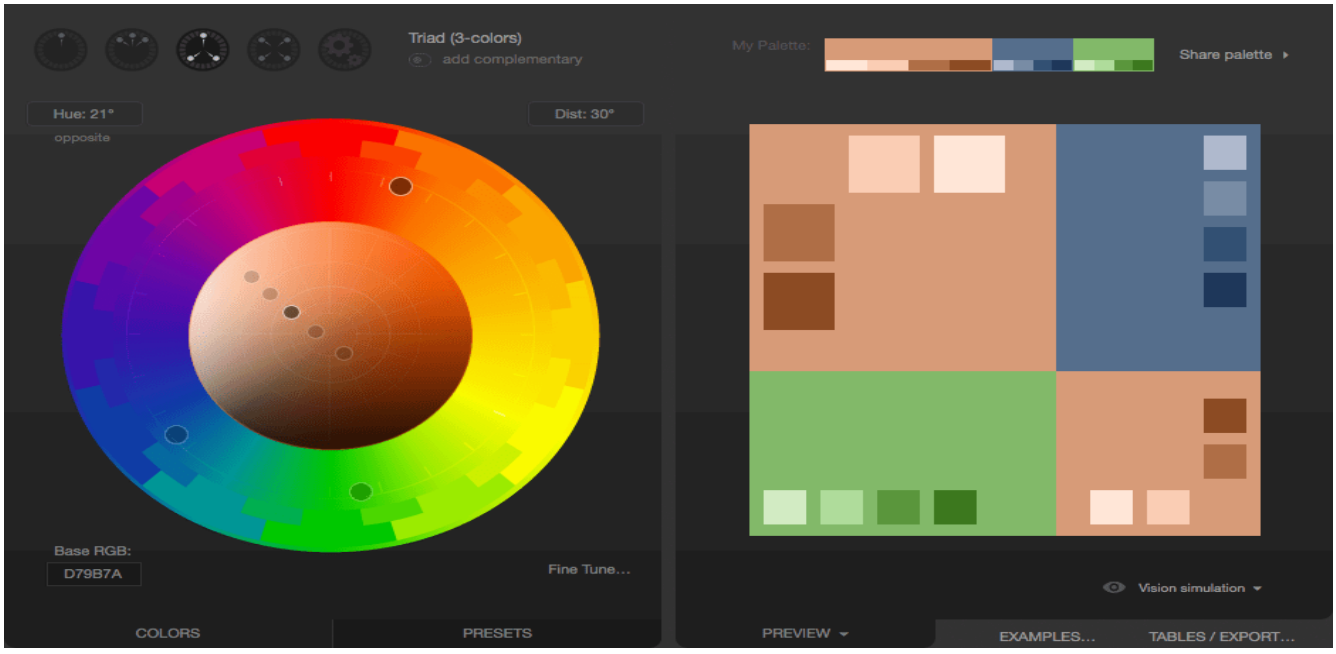
Next, we enter our color's hex code (`D79C7A`) into the "Base RGB" box at the bottom-left corner of the tool:



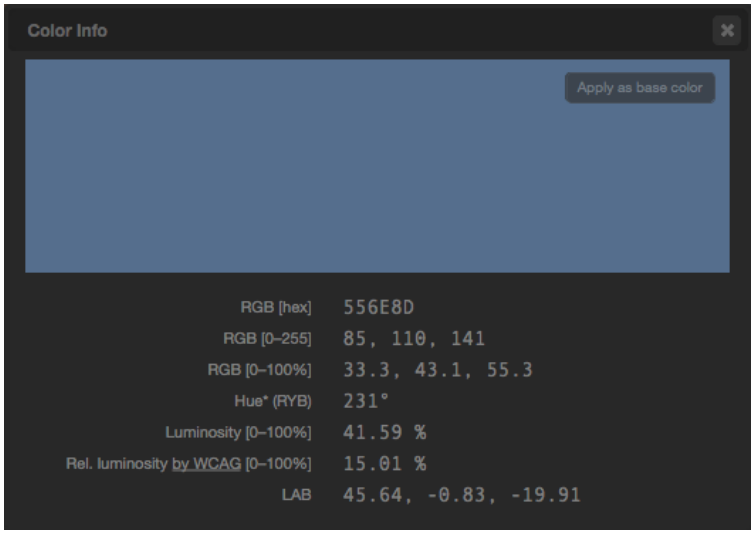
We now see a monochromatic palette based on the color we picked from the Mars photo. If you need related colors, these are likely good options. To find an accent color that pop alongside the base color, we click the "add complementary" toggle underneath the menu that lets you select the palette type. The default was "Monochromatic". Paletton computes an appropriate accent color; clicking on the accent color in the bottom-right corner tells us that this color is `#508D7C`.



If the proposed color doesn't work for your needs, you can change the color scheme. For example, if the proposed greenish-blue color doesn't work, select the Triad color scheme icon, which results in the following:



Click on the grayish blue in the top-right. The color is `#556E8D`. This can be used as an accent color to make things stand out, such as for headlines, tabs highlights, or other indicators on the site:



Now we have our base color and our accent. We also have a few complementary shades of both, which can be used to create gradients or as an accent color to indicate focus, such as for link hover states. The colors can be exported in several formats for you to use.

You should also select neutral colors. Find a color that provides enough contrast for your text to be crisp and readable while ensuring it isn't harsh for the eyes. If the contrast is too low, your text will be washed out by the background, leaving it unreadable, but if your contrast is too high, the user may find your site garish and unpleasant to look at.

Color, backgrounds, contrast, and printing

Your site may look very different when printed from what the user sees on their screen. When printing your page, the user may select to print in black and white only. Most browsers, by default, remove background colors and images when printing documents.

What matters most is usually the text itself, but if your background colors and images have been selected carefully and/or are crucial to the usefulness of the content, you can use the CSS `print-color-adjust` property to tell the browser that it should not make adjustments to the appearance of content.

The default value of `print-color-adjust: economy`, indicates that the browser is allowed to make appearance changes as it deems necessary to try to optimize the legibility and/or print economy of the content, given the type of output device the document is being drawn onto.

You can set `print-color-adjust: exact` to tell the browser that the element or elements on which you use it have been designed specifically to best work with the colors and images left as they are. With this set, the browser won't tamper with the appearance of the element on which this value is applied, and will draw it as indicated by your CSS.

> **Note:** There is no guarantee, though, that `print-color-adjust: exact` will result in your CSS being used exactly as given. If the browser provides user preferences to change the output (such as a "don't print backgrounds" checkbox in a print dialog box), that overrides the value of `print-color-adjust`.

See also

- CSS colors module
- Applying color to HTML elements using CSS
- CSS color values
- Using relative colors
- Understanding color and luminance
- WCAG 1.4.1: Color contrast
- Paletton

Using relative colors

The CSS colors module defines **relative color syntax**, which allows a CSS `<color>` value to be defined relative to another color. This is a powerful feature that enables the programmatic creation of complements to existing colors — such as lighter, darker, saturated, semi-transparent, or inverted variants — enabling more effective color palette creation.

This article explains relative color syntax, shows what the different options are, and looks at some illustrative examples.

General syntax

A relative CSS color value has the following general syntax structure:

```
color-function(from origin-color channel1 channel2 channel3)
color-function(from origin-color channel1 channel2 channel3 / alpha)

/* color space included in the case of color() functions */
color(from origin-color colorspace channel1 channel2 channel3)
color(from origin-color colorspace channel1 channel2 channel3 / alpha)
```

Relative colors are created using the same color functions as absolute colors, but with different parameters:

1. Include a basic color function (represented by *`color-function()`* above) such as `rgb()`, `hsl()`, etc. Which one you pick depends on the color model you want to use for the relative color you are creating (the **output color**).
2. Pass in the **origin color** (represented above by *`origin-color`*) your relative color will be based on, preceded by the `from` keyword. This can be any valid `<color>` value using any available color model including a color value contained in a CSS custom property, system colors, `currentColor`, or even another relative color.
3. In the case of the `color()` function, include the *`colorspace`* of the output color.
4. Provide an output value for each individual channel. The output color is defined after the origin color — represented above by the *`channel1`*, *`channel2`*, and *`channel3`* placeholders. The channels defined here depend on the color function you are using for your relative color. For example, if you are using `hsl()`, you would need to define the values for hue, saturation, and lightness. Each channel value can be a new value, the same as the original value, or a value relative to the channel value of the origin color.
5. Optionally, an `alpha` channel value of type `<alpha-value>` for the output color can be defined, preceded by a slash (`/`). If the `alpha` channel value is not explicitly specified, it defaults to the alpha channel value of the *`origin-color`* (not 100%, which is the case for absolute color values).

The browser converts the origin color to a syntax compatible with the color function then destructures it into component color channels (plus the `alpha` channel if the origin color has one). These are made available as appropriately-named values inside the color function — `r`, `g`, `b`, and `alpha` in the case of the `rgb()` function, `l`, `a`, `b`, and `alpha` in the case of the `lab()` function, `h`, `w`, `b`, and `alpha` in the case of `hwb()`, etc. — that can be used to calculate new output channel values.

Let's look at relative color syntax in action. The below CSS is used to style two `<div>` elements, one with an absolute background color — `red` — and one with a relative background color created with the `rgb()` function, based on the same `red` color value:

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
</div>
```

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
#one {
  background-color: red;
}

#two {
  background-color: rgb(from red 200 g b / alpha);
}
```

The output is as follows:

The relative color uses the `rgb()` function, which takes `red` as the origin color, converts it to an equivalent `rgb()` color (`rgb(255 0 0)`) and then defines the new color as having a red channel of value `200` and green, blue and alpha channels with a value the same as the origin color (it uses the `g` and `b` values made available inside the function by the browser, which are both equal to `0`, and the `alpha` is `100%`).

This results in an output of `rgb(200 0 0)` — a slightly darker red. If we had specified a red channel value of `255` (or just the `r` value), the resulting output color would be exactly the same as the input value. The browser's final output color (the computed value) is an sRGB `color()` value equivalent to `rgb(200 0 0)` — `color(srgb 0.784314 0 0)`.

> **Note:** As mentioned above, when calculating a relative color the first thing the browser does is to convert the provided origin color (`red` in the above example) into a value compatible with the color function being used (in this case, `rgb()`). This is done so that the browser is able to calculate the output color from the origin color. While the calculations are performed relative to the color function used, the actual output color value depends on the color's color space:
>
> - Older sRGB color functions cannot express the full spectrum of visible colors. The output colors of (`hsl()`, `hwb()`, and `rgb()`) are serialized to `color(srgb)` to avoid these limitations. That means that querying the output color value via the `HTMLElement.style` property or the `CSSStyleDeclaration.getPropertyValue()` method returns the output color as a `color(srgb ...)` value.
> - For more recent color functions (`lab()`, `oklab()`, `lch()`, and `oklch()`), relative color output values are expressed in the same syntax as the color function used. For example, if a `lab()` color function is being used, the output color will be a `lab()` value.

All the following lines produce an equivalent output color:

```
red
rgb(255 0 0)
rgb(from red 255 0 0)
rgb(from red 255 0 0 / 1)
rgb(from red 255 0 0 / 100%)

rgb(from red 255 g b)
rgb(from red r 0 0)
rgb(from red r g b / 1)
rgb(from red r g b / 100%)

rgb(from red r g b)
rgb(from red r g b / alpha)

/* With `red`, the g and b are the same, making them interchangeable */
rgb(from red r g g)
rgb(from red r b b)
rgb(from red 255 g g)
rgb(from red 255 b b)
```

Syntax flexibility

There is an important distinction to be made between the destructured origin color channel values made available in the function, and the channel values of the output color set by the developer.

To reiterate, when a relative color is defined, the channel values of the origin color are made available in the function to use when defining the output color channel values. The following example defines a relative color using an `rgb()` function and uses the origin color channel values (made available as `r`, `g`, and `b`) for the output channel values, meaning that the output color is the same as the origin color:

```
rgb(from red r g b)
```

However, when specifying the output values, you don't need to use the origin color channel values at all. You need to provide the output channel values in the right order (e.g., red, then green, then blue in the case of `rgb()`), but they can be any values you wish provided they are valid values for those channels. This gives relative CSS colors a high degree of flexibility.

For example, if you wanted to, you could specify absolute values like those shown below, transforming `red` into `blue`:

```
rgb(from red 0 0 255)
/* output color is equivalent to rgb(0 0 255), full blue */
```

> **Note:** If you are using relative color syntax but outputting the same color as the origin color or a color not based on the origin color at all, you are not really creating a relative color. You'd be unlikely to ever do this in a real codebase, and would probably just use an absolute color value instead. But, we felt it useful to explain that you *can* do this with relative color syntax, as a starting point for learning about it.

You can even mix up or repeat the provided values. The following takes a slightly darker red as an input and outputs a light gray color — the output color's `r`, `g`, and `b` channels are all set to the origin color's `r` channel value:

```
rgb(from rgb(200 0 0) r r r)
/* output color is equivalent to rgb(200 200 200), light gray */
```

The following uses the origin color's channel values for the output color's `r`, `g`, and `b` channel values, but in reverse order:

```
rgb(from rgb(200 170 0) b g r)
/* output color is equivalent to rgb(0 170 200) */
```

Color functions that support relative colors

In the section above we only saw relative colors defined via the `rgb()` function. However, relative colors can be defined using any modern CSS color function — `color()`, `hsl()`, `hwb()`, `lab()`, `lch()`, `oklab()`, `oklch()`, or `rgb()`. The general syntax structure is the same in each case, although the origin color values have different names appropriate for the function being used.

Below you can find relative color syntax examples for each color function. Each case is the simplest possible, with the output color channel values exactly matching the origin color channel values:

```
/* color() with and without alpha channel */
color(from red a98-rgb r g b)
color(from red a98-rgb r g b / alpha)

color(from red xyz-d50 x y z)
color(from red xyz-d50 x y z / alpha)

/* hsl() with and without alpha channel */
hsl(from red h s l)
hsl(from red h s l / alpha)

/* hwb() with and without alpha channel */
hwb(from red h w b)
hwb(from red h w b / alpha)

/* lab() with and without alpha channel */
lab(from red l a b)
```

```
lab(from red l a b / alpha)

/* lch() with and without alpha channel */
lch(from red l c h)
lch(from red l c h / alpha)

/* oklab() with and without alpha channel */
oklab(from red l a b)
oklab(from red l a b / alpha)

/* oklch() with and without alpha channel */
oklch(from red l c h)
oklch(from red l c h / alpha)

/* rgb() with and without alpha channel */
rgb(from red r g b)
rgb(from red r g b / alpha)
```

It is worth mentioning again that the color system of the origin color doesn't need to match the color system being used to create the output color. Again, this provides a lot of flexibility. Generally you won't be interested in and might not even know the system the origin color is defined in (you might just have a custom property value to manipulate). You'll just want to input a color and, for example, create a lighter variant of it by putting it into an `hsl()` function and varying the lightness value.

Using custom properties

When creating a relative color, you can use values defined in CSS custom properties both for the origin color and within the output color channel value definitions. Let's look at an example.

In the below CSS we define two custom properties:

- `--base-color` contains our base brand color — `purple`. Here we are using a named color keyword, but relative colors can accept any color syntax for the origin color.
- `--standard-opacity` contains the standard brand opacity value that we want to apply to semi-transparent boxes — `0.75`.

We then give two `<div>` elements a background color. One is given an absolute color — our `--base-color` brand purple. The other one is given a relative color equal to our brand purple, transformed to add an alpha channel equal to our standard opacity value.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
</div>
```

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
  background-image: repeating-linear-gradient(
    45deg,
    white,
    white 24px,
    black 25px,
    black 50px
  );
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: purple;
  --standard-opacity: 0.75;
}

#one {
  background-color: var(--base-color);
}

#two {
  background-color: hwb(from var(--base-color) h w b / var(--standard-opacity));
}
```

The output is as follows:

Using math functions

You can use CSS math functions such as `calc()` to calculate values for the output color channels. Let's look at an example.

The below CSS is used to style three `<div>` elements with different background colors. The middle one is given an unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`. These variants are defined using relative colors — the `--base-color` is passed into an `lch()` function, and the output color has its lightness channel modified to achieve the desired effect via a `calc()` function. The lightened color has 20% added to the lightness channel, and the darkened color has 20% subtracted from it.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

#one {
  background-color: lch(from var(--base-color) calc(l + 20) c h);
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: lch(from var(--base-color) calc(l - 20) c h);
}
```

The output is as follows:

Manipulating the alpha channel

This example demonstrates changing the alpha channel of a named color. Here, we have an item wrapped in a container that both have a `teal` background. To distinguish between the backgrounds, we vary the alpha channel value using the relative color feature, the `calc()` function, and a custom property.

```
<div class="container">
  <div class="item"></div>
</div>
```

```
.container {
  padding: 60px;
}

.item {
  height: 60px;
}
```

```
div {
  background-color: rgb(
    from teal r g b / calc(alpha * var(--alpha-multiplier))
  );
}

.container {
  --alpha-multiplier: 0.3;
}

.item {
  --alpha-multiplier: 1;
}
```

The alpha channel is referenced using the `alpha` keyword. In this case, the `calc(alpha * var(--alpha-multiplier))` expression modifies the alpha channel value by multiplying `alpha` with the `--alpha-multiplier` custom property value. The container gets a semitransparent background because of the multiplier of `0.3` is less than `1.0`.

The output is as follows:

Channel values resolve to <code><number></code> values

To make channel value calculations work in relative colors, all origin color channel values resolve to appropriate `<number>` values. For example, in the `lch()` examples above, we are calculating new lightness values by adding or subtracting numbers from the origin color's `l` channel value. If we tried to do `calc(l + 20%)`, that would result in an invalid color — `l` is a `<number>` and cannot have a `<percentage>` added to it.

- Channel values originally specified as a `<percentage>` resolve to a `<number>` appropriate for the output color function.
- Channel values originally specified as a `<hue>` angle resolve to a number of degrees in a range of `0` to `360`, inclusive.

Check the different color function pages for the specifics of what their origin channel values resolve to.

Origin colors outside the sRGB gamut

Converting an origin color to the output color space does not clamp its channels to that space's usual range. In the following example, `color(display-p3 1 0.5 0.5)` is inside the Display P3 gamut but outside sRGB—its red channel, expressed on the `rgb()` scale, is approximately `273.88`, which is greater than `255`. The `background-color` calculation uses that out-of-range value, so the red value ends up as `272.88`. No clamping ever happens in this calculation. Whether the final background color can be rendered accurately depends on the display's capability.

```
.very-red {
  --origin: color(display-p3 1 0.5 0.5);
  background-color: rgb(from var(--origin) calc(r - 1) g b);
}
```

The absolute `rgb()` notation does clamp its arguments, so if you write `--origin: rgb(273.88 117.95 123.15)`, then the red channel would start at `255` and the background color's red channel would be `254`.

Checking for browser support

You can check that a browser supports relative color syntax by running it through a `@supports` at-rule.

For example:

```
@supports (color: hsl(from white h s l)) {
  /* safe to use hsl() relative color syntax */
}
```

Examples

> **Note:** You can find additional examples demonstrating the use of relative color syntax in the different functional notation types on their dedicated pages: `color()`, `hsl()`, `hwb()`, `lab()`, `lch()`, `oklab()`, `oklch()`, `rgb()`.

Color palette generator

This example allows you to choose a base color and a color palette type. The browser will then show an appropriate palette of colors based on the chosen base color. The color palette choices are as follows:

- **Complementary**: Includes two colors that are at opposite sides of a color wheel, or to put it another way, *opposite hues* (see the `<hue>` data type for more information on hues and color wheels). The two colors are defined as a base color, and the base color with hue channel +180 degrees.
- **Triadic**: Includes three colors equal distances apart around the color wheel. The three colors are defined as a base color, base color with hue channel -120 degrees, and base color with hue channel +120 degrees.
- **Tetradic**: Includes four colors equal distances apart around the color wheel. The four colors are defined as a base color, and base color with hue channel +90, +180, and +270 degrees.
- **Monochrome**: Includes multiple colors with the same hue but varying lightness values. In our example we've defined five colors in a monochrome palette — base color, and base color with lightness channel -20, -10, +10, and +20.

HTML

The full HTML is included below for reference. The most interesting parts are as follows:

- The `--base-color` custom property is stored as an inline `style` on the `<div>` element with the ID of `container`. We've placed it there so we can update the value using JavaScript. We've provided an initial value of `#ff0000` (`red`) to show a color palette based on that value when the example loads. Note that normally we'd probably set this on the `<html>` element, but the MDN live sample was removing it when rendering.
- The base color picker is created using an `<input type="color">` control. When a new value is set in this control, the `--base-color` custom property is set to this value using JavaScript, which in turn generates a new color palette. All the displayed colors are relative colors based on `--base-color`.
- The set of `<input type="radio">` controls enables choosing a color palette type to generate. When a new value is chosen here, JavaScript is used to set a new class on the `container` `<div>` to represent the chosen palette. In the CSS, descendant selectors are used to target the child `<div>`s (e.g., `.comp :nth-child(1)`) so they can have the correct colors applied to them and hide the unused `<div>` nodes.
- The `container` `<div>` containing the child `<div>`s that display the colors of the generated palette. Note that an initial class of `comp` is set on it, so that the page will display a complementary color scheme when first loaded.

```
<div>
  <h1>Color palette generator</h1>
  <form>
    <div id="color-picker">
      <label for="color">Select a base color:</label>
      <input type="color" id="color" name="color" value="#ff0000" />
    </div>
    <div>
      <fieldset>
        <legend>Select a color palette type:</legend>

        <div>
          <input
            type="radio"
            id="comp"
            name="palette-type"
            value="comp"
            checked />
          <label for="comp">Complementary</label>
        </div>

        <div>
          <input
            type="radio"
            id="triadic"
            name="palette-type"
            value="triadic" />
          <label for="triadic">Triadic</label>
        </div>

        <div>
          <input
            type="radio"
            id="tetradic"
            name="palette-type"
            value="tetradic" />
          <label for="tetradic">Tetradic</label>
        </div>

        <div>
          <input
            type="radio"
            id="monochrome"
            name="palette-type"
            value="monochrome" />
          <label for="monochrome">Monochrome</label>
        </div>
      </fieldset>
    </div>
  </form>
  <div id="container" class="comp">
    <div></div>
    <div></div>
    <div></div>
    <div></div>
    <div></div>
  </div>
</div>
```

CSS

Below we are only showing the CSS that sets the palette colors. Note how, in each case, descendant selectors are used to apply the correct `background-color` to each child `<div>` for the chosen palette. We care more about the position of the `<div>`s in the source order than the type of element, so we have used `:nth-child` to target them.

In the last rule we've used the general sibling selector (`~`) to target the unused `<div>` elements in each palette type, setting `display: none` to stop them being rendered.

The colors themselves include the `--base-color`, plus relative colors derived from that `--base-color`. The relative colors use the `lch()` function — passing in the origin `--base-color` and defining an output color with an adjusted lightness or hue channel as appropriate.

```
html {
  font-family: sans-serif;
}

body {
  margin: 0;
}
```

```
h1 {
  margin-left: 16px;
}

/* Basic form styling */

#color-picker {
  margin-left: 16px;
  margin-bottom: 20px;
}

#color-picker label,
legend {
  display: block;
  font-size: 0.8rem;
  margin-bottom: 10px;
}

input[type="color"] {
  width: 200px;
  display: block;
}

fieldset {
  display: flex;
  gap: 20px;
  border: 0;
}

/* Palette container styling */

#container {
  /* Default value */
  --base-color: red;

  display: flex;
  width: 100vw;
  height: 250px;
  box-sizing: border-box;
}

#container div {
  flex: 1;
}
```

```
/* Complementary colors */
/* Base color, and base color with hue channel +180 degrees */

.comp :nth-child(1) {
  background-color: var(--base-color);
}

.comp :nth-child(2) {
  background-color: lch(from var(--base-color) l c calc(h + 180));
}

/* Use @supports to add in support old syntax that requires deg units
   to be specified in hue calculations. This is required for Safari 16.4+. */
@supports (color: lch(from red l c calc(h + 180deg))) {
  .comp :nth-child(2) {
    background-color: lch(from var(--base-color) l c calc(h + 180deg));
  }
}

/* Triadic colors */
/* Base color, base color with hue channel -120 degrees, and base color */
/* with hue channel +120 degrees */

.triadic :nth-child(1) {
  background-color: var(--base-color);
}

.triadic :nth-child(2) {
  background-color: lch(from var(--base-color) l c calc(h - 120));
}

.triadic :nth-child(3) {
  background-color: lch(from var(--base-color) l c calc(h + 120));
}

/* Use @supports to add in support old syntax that requires deg units
   to be specified in hue calculations. This is required for Safari 16.4+. */
@supports (color: lch(from red l c calc(h + 120deg))) {
  .triadic :nth-child(2) {
    background-color: lch(from var(--base-color) l c calc(h - 120deg));
  }

  .triadic :nth-child(3) {
    background-color: lch(from var(--base-color) l c calc(h + 120deg));
  }
}

/* Tetradic colors */
/* Base color, and base color with hue channel +90, +180, and +270 degrees */

.tetradic :nth-child(1) {
  background-color: var(--base-color);
}

.tetradic :nth-child(2) {
  background-color: lch(from var(--base-color) l c calc(h + 90));
}
```

```
.tetradic :nth-child(3) {
  background-color: lch(from var(--base-color) l c calc(h + 180));
}

.tetradic :nth-child(4) {
  background-color: lch(from var(--base-color) l c calc(h + 270));
}

/* Use @supports to add in support old syntax that requires deg units
   to be specified in hue calculations. This is required for Safari 16.4+. */
@supports (color: lch(from red l c calc(h + 90deg))) {
  .tetradic :nth-child(2) {
    background-color: lch(from var(--base-color) l c calc(h + 90deg));
  }

  .tetradic :nth-child(3) {
    background-color: lch(from var(--base-color) l c calc(h + 180deg));
  }

  .tetradic :nth-child(4) {
    background-color: lch(from var(--base-color) l c calc(h + 270deg));
  }
}

/* Monochrome colors */
/* Base color, and base color with lightness channel -20, -10, +10, and +20 */

.monochrome :nth-child(1) {
  background-color: lch(from var(--base-color) calc(l - 20) c h);
}

.monochrome :nth-child(2) {
  background-color: lch(from var(--base-color) calc(l - 10) c h);
}

.monochrome :nth-child(3) {
  background-color: var(--base-color);
}

.monochrome :nth-child(4) {
  background-color: lch(from var(--base-color) calc(l + 10) c h);
}

.monochrome :nth-child(5) {
  background-color: lch(from var(--base-color) calc(l + 20) c h);
}

/* Hide unused swatches for each palette type */
.comp :nth-child(2) ~ div,
.triadic :nth-child(3) ~ div,
.tetradic :nth-child(4) ~ div {
  display: none;
}
```

An aside on `@supports` testing

In the example CSS you'll notice `@supports` blocks being used to provide different `background-color` values to browsers that support a previous draft specification of the relative color syntax. These are required because Safari's initial implementation was based on an older version of the spec in which origin color channel values resolved to `<number>`s or other unit types depending on the context. This meant that values sometimes required units when performing additions and subtractions, which created confusion. In newer implementations, origin color channel values always resolve to an equivalent `<number>` value, which means calculations are always done with unitless values.

Note how the support test in each case is done using any color declaration — `color: lch(from red l c calc(h + 90deg))` for example — not necessarily the actual value that we need to vary for other browsers. When testing complex values like these, you should use the simplest possible declaration that still contains the syntactic difference you want to test for.

Including a custom property in the `@supports` test doesn't work — the test always comes back as positive regardless of what value the custom property is given. This is because a custom property value only becomes invalid when assigned to be an invalid value (or part of an invalid value) of a regular CSS property. To work around this, in each test we have replaced `var(--base-color)` with the `red` keyword.

JavaScript

In the JavaScript, we:

- Add a `change` event listener to the radio buttons so that when one is selected, the `setContainer()` function runs. This function updates the `class` value of the `<div>` with `id="container"` with the value of the selected radio button so that the correct background colors will be applied to the child `<div>`s for the chosen palette type.
- Add an `input` event listener to the color picker control so that when a new color is selected, the `setBaseColor()` function runs. This function sets the `--base-color` custom property's value to the new color.

```
const form = document.forms[0];
const radios = form.elements["palette-type"];
const colorPicker = form.elements["color"];
const containerElem = document.getElementById("container");

for (const radio of radios) {
  radio.addEventListener("change", setContainer);
}

colorPicker.addEventListener("input", setBaseColor);

function setContainer(e) {
  const palType = e.target.value;
  console.log("radio changed");
  containerElem.setAttribute("class", palType);
}

function setBaseColor(e) {
  console.log("color changed");
  containerElem.style.setProperty("--base-color", e.target.value);
}
```

Results

The output is as follows. This starts to show the power of relative CSS colors — we are defining multiple colors and generating palettes that are updated live by adjusting a single custom property.

Live UI color scheme updater

This example shows a card containing a heading and text, but with a twist — below the card is a slider (`<input type="range">`) control. When its value is changed, JavaScript is used to set a `--hue` custom property value to the new slider value.

This in turn adjusts the color scheme for the entire UI:

- The `--base-color` value is a relative color with its hue channel set to the value of `--hue`.
- The other colors used in the design are relative colors based on `--base-color`. As a result, they change when the `--base-color` changes.

HTML

The HTML for the example is shown below.

- The `<main>` element acts as an outer wrapper to contain the rest of the content, allowing the card and form to be centered vertically and horizontally inside `<main>` as one unit.
- The `<section>` element contains the `<h1>` and `<p>` elements that define the card's content.
- The `<form>` element contains the (`<input type="range">`) control and its `<label>`.

```
<main>
  <section>
    <h1>A love of colors</h1>
    <p>
      Colors, the vibrant essence of our surroundings, are truly awe-inspiring.
      From the fiery warmth of reds to the calming coolness of blues, they bring
      unparalleled richness to our world. Colors stir emotions, ignite
      creativity, and shape perceptions, acting as a universal language of
      expression. In their brilliance, colors create a visually enchanting
      tapestry that invites admiration and sparks joy.
    </p>
  </section>
  <form>
    <label for="hue-adjust">Adjust the hue:</label>
    <input
      type="range"
      name="hue-adjust"
      id="hue-adjust"
      value="240"
      min="0"
      max="360" />
  </form>
</main>
```

CSS

In the CSS the `:root` has a default `--hue` value set on it, relative `lch()` colors to define the color scheme, plus a radial gradient that fills the whole body.

The relative colors are as follows:

- `--base-color`: The base color takes an origin color of `red` (although any full color would do) and adjusts its hue value to the value set in the `--hue` custom property.
- `--bg-color`: A much lighter variant of `--base-color`, intended to be used as a background. This is created by taking an origin color of `--base-color` and adding 40 to its lightness value.
- `--complementary-color`: A complementary color 180 degrees around the color wheel from `--base-color`. This is created by taking an origin color of `--base-color` and adding 180 to its hue value.

Now have a look at the rest of the CSS and take note of all the places where these colors are used. This includes backgrounds, borders, `text-shadow`, and even the `accent-color` of the slider.

> **Note:** For brevity, only the parts of the CSS relevant to relative color usage are shown.

```
html {
  font-family: sans-serif;
}

main {
  width: 80vw;
  margin: 2rem auto;
}

h1 {
  text-align: center;
  margin: 0;
  color: black;
  border-radius: 16px 16px 0 0;
  font-size: 3rem;
  letter-spacing: -1px;
}

p {
  line-height: 1.5;
  margin: 0;
  padding: 1.2rem;
}

form {
  width: fit-content;
  display: flex;
  margin: 2rem auto;
  padding: 0.4rem;
}
```

```
:root {
  /* Default hue value */
  --hue: 240;

  /* Relative color definitions */
  --base-color: lch(from red l c var(--hue));
  --bg-color: lch(from var(--base-color) calc(l + 40) c h);
  --complementary-color: lch(from var(--base-color) l c calc(h + 180));

  background: radial-gradient(ellipse at center, white 20%, var(--base-color));
}
```

```
/* Use @supports to add in support for --complementary-color with old
   syntax that requires deg units to be specified in hue calculations.
   This is required for in Safari 16.4+. */
@supports (color: lch(from red l c calc(h + 180deg))) {
  body {
    --complementary-color: lch(from var(--base-color) l c calc(h + 180deg));
  }
}

/* Box styling */

section {
  background-color: var(--bg-color);
  border: 3px solid var(--base-color);
  border-radius: 20px;
  box-shadow: 10px 10px 30px rgb(0 0 0 / 0.5);
}

h1 {
  background-color: var(--base-color);
  text-shadow:
    1px 1px 1px var(--complementary-color),
    -1px -1px 1px var(--complementary-color),
    0 0 3px var(--complementary-color);
}

/* Range slider styling */

form {
  background-color: var(--bg-color);
  border: 3px solid var(--base-color);
}

input {
  accent-color: var(--complementary-color);
}
```

JavaScript

The JavaScript adds an `input` event listener to the slider control so that when a new value is set, the `setHue()` function runs. This function sets a new inline `--hue` custom property value on the `:root` (the `<html>` element) that overrides the original default value we set in our CSS.

```
const rootElem = document.querySelector(":root");
const slider = document.getElementById("hue-adjust");

slider.addEventListener("input", setHue);

function setHue(e) {
  rootElem.style.setProperty("--hue", e.target.value);
}
```

Results

The output is shown below. Relative CSS colors are being used here to control the color scheme of an entire UI, which can be adjusted live as a single value is modified.

See also

- The `<color>` data type
- CSS colors module
- sRGB on Wikipedia
- CIELAB on Wikipedia
- CSS relative color syntax on developer.chrome.com (2023)

CSS compositing and blending

The **CSS compositing and blending** module defines how an element's background layers can be blended together, how an element can be blended with its container, and whether the element must create a new stacking context.

The properties in this CSS module can be used to define the blending mode that should be used, if any, to blend an element's background images and colors into a single background image. This module provides 16 blending modes. You can also define how an element's borders, background, and content, including text, emojis, and images, should be blended with the background of its container.

Compositing and blending in action

In this example, each box has a border, two striped background images, and a solid color background. The common background for all the boxes contains a pattern of circles. The three boxes in the second row are set to blend with the background of the container.

```
<section>
  <div><span>Normal, with no blending</span></div>
  <div><span>Multiplies colors</span></div>
  <div><span>Multiplies based on background color</span></div>
  <div>Normal, with no blending</div>
  <div>Multiplies colors</div>
  <div>Multiplies based on background color</div>
</section>
```

```
/* Creates a div with two offset striped background images and a background color. */
div {
  width: 200px;
  height: 200px;
  background-image:
    repeating-linear-gradient(45deg, red 0 15px, pink 15px 30px),
    repeating-linear-gradient(-45deg, blue 0 15px, lightblue 15px 30px);
  background-size: 150px 150px;
  background-repeat: no-repeat;
  background-position:
    top left,
    bottom right;
  background-color: palegoldenrod;
  text-align: center;
  padding-top: 150px;
```

```
  font-family: sans-serif;
  box-sizing: border-box;
  border: 5px solid black;
}
div:nth-of-type(3n + 1) {
  background-blend-mode: normal;
}
div:nth-of-type(3n + 2) {
  background-blend-mode: multiply;
}
div:nth-of-type(3n + 3) {
  background-blend-mode: overlay;
}
div:nth-of-type(n + 4) {
  mix-blend-mode: difference;
}
/* Put a pink background with transparent round holes that covers the
  entire element, and lay the examples in two rows with three columns each */
section {
  padding: 0.75em;
  background: radial-gradient(
    circle,
    transparent 0 20px,
    rgb(255 200 200) 20px
  );
  background-size: 60px 60px;
  background-position: center;
  display: inline-grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 1em;
}
/* Make some of the text more legible */
span {
  background-color: #ffffff99;
}
```

Notice how the background, border, and the content are all impacted as a result of the blending. Click "Play" in the example above to see or edit the code for the animation in the MDN Playground.

Reference

Properties

- `background-blend-mode`
- `isolation`
- `mix-blend-mode`

Related concepts

- `<blend-mode>` data type
- `backdrop-filter` CSS property
- `filter` CSS property
- `mask-composite` CSS property
- `background-color` CSS property
- `background-image` CSS property
- stacking context glossary term
- `<feBlend>` SVG filter primitive
- `<feComposite>` SVG filter primitive

See also

- Properties in the CSS filter effects module enable applying filters effects, such as blurring and changing color intensity, to images, backgrounds, and borders.
- Compositing And Blending In CSS (2015)
- Editing Images in CSS: Blend Modes (2022)
- web.dev: blend modes (2021)

CSS conditional rules

The **CSS conditional rules** module defines CSS media and support queries, enabling you to define styles that are only applied if specific conditions are met. The conditional rules defined in this module are based on device, user-agent, and viewport capabilities. With conditional rules, you can target CSS styles based on query values or browser and device features, independent of the document being rendered.

The first CSS conditional rules were media types specifying the intended destination medium for the linked styles, for example `screen` or `print`. These were set as the value of the HTML `<link>` and `<style>` elements' `media` attributes or as a comma-separated list of media types within an `@import` statement or at-rule. The ability to conditionally apply CSS rules has been greatly expanded since the CSS 2.1 and HTML 4.01 implementations that limited conditional queries to a few media types.

CSS conditional rules now include feature queries; the `@supports` at-rule enables targeting CSS styles based on a user-agent's CSS capabilities. Additional conditions include which selector, font-formats, and font-techs are supported.

The CSS conditional rules module also expands `@media` to enable nesting at-rules, with the related CSS media queries module removing unused media types and adding many media features and conditions that can be targeted.

The CSS container queries module defines similar conditional rules, but based on an element's parent rather than the viewport.

There are plans to further extend possible queries by adding the generalized conditional rule `@when` and the chained conditional rule `@else`. These two at-rules are not yet supported.

Reference

Properties

- `container`
- `container-name`
- `container-type`

At-rules and descriptors

- `@container`
  - `aspect-ratio`
  - `block-size`
  - `fallback`

        - `height`
        - `inline-size`
        - `orientation`
        - `scrollable`
        - `snapped`
        - `stuck`
        - `width`
- `@media`
- `@supports`

The CSS conditional rules module also introduces the `@else` and `@when` at-rules. Currently, no browsers support these features.

Functions

- `anchored()`
- `style()`
- `font-tech()`
- `font-format()`
- `scroll-state()`
- `selector()`
- `supports()`

The CSS conditional rules module also introduces a `media()` CSS function. Currently, no browsers support this feature.

data types

- `<container-name>`
- `<style-feature>`
- Container relative `<length>` units
- `<media-query>`
- `<supports-condition>`
- `<supports-feature>` (see `supports()`)

Interfaces

- `CSSConditionRule`
- `CSSMediaRule`
- `CSSSupportsRule`
- `supports()` method

Terms and glossary definitions

- Media
- Supports query (See feature query)

Guides

Using CSS feature queries

Selectively applying CSS rules after checking browser support for the specified properties and values via feature queries.

Using CSS media queries

Introduces media queries, their syntax, and the operators and media features that are used to construct media query expressions.

Supporting older browsers: feature queries

How to use feature queries to target CSS based on the browser's level of support for web features.

Browser feature detection: CSS `@supports`

A look at JavaScript and CSS feature detection, including CSS `@supports`.

Using container scroll-state queries

Using container scroll-state queries, with an example of each type.

Related concepts

- CSS cascading and inheritance module
    - `@import` at-rule
- CSS media queries module
    - `<media-feature>`
    - `<media-type>`
    - `<media-condition>`
    - `<media-query-list>`
    - CSS logical operators (`not`, `or`, and `and`)
- CSSOM view module
    - `CSS` API
    - `CSSGroupingRule` API
    - `MediaQueryList` API
    - `CSSRule` API
    - `MediaList` interface
        - `MediaList.mediaText` property
- CSS syntax module
    - `@charset` declaration
    - at-rule term
    - `invalid` term
    - parse term
    - style rule term

- CSS namespaces module
    - `@namespace` at-rule
- CSS anchor positioning module
    - Using anchored container queries

See also

- CSS container queries module
- CSS media queries module
- CSS cascading and inheritance module

Using container scroll-state queries

Container scroll-state queries are a type of container query. Rather than selectively applying styles to descendant elements based on the container's size, scroll-state queries allow you to selectively apply styles to descendant elements based on the container's scroll state. This can include whether the container is partially scrolled, snapped to a scroll snap container ancestor, or positioned via `position: sticky` and stuck to a boundary of a scroll container ancestor.

This article explains how to use container scroll-state queries, walking through an example of each type. It assumes that you know the basics of container queries. If you are new to container queries, read CSS container queries before continuing.

Types of container scroll-state query

There are three `@container` descriptors you can use in a `scroll-state()` query:

- `scrollable`: Queries whether a container can be scrolled in the given direction via user-initiated scrolling (for example by dragging the scrollbar or using a trackpad gesture). In other words, is there any overflowing content in the given direction that can be scrolled to? This is useful for applying styling related to the scroll position of a scroll container. For example, you could display a hint that encourages people to scroll down and see more content when the scrollbar is up at the top, and hide it when the user has actually started scrolling.
- `scrolled`: Queries whether a container was most recently scrolled in the given direction. This allows you to selectively apply styles based on the user's scroll direction, for example a top menu bar that only displays when the user is scrolling upwards.
- `snapped`: Queries whether a container is going to be snapped to a scroll snap container ancestor along a given axis. This is useful for applying styles when an element is snapped to a scroll snap container. For example, you might want to highlight a snapped element in some way, or reveal some of its content that was previously hidden.
- `stuck`: Queries whether a container with a `position` value of `sticky` is stuck to an edge of its scroll container ancestor. This is useful for styling `position: sticky` elements differently when stuck — for example, you could give them a different color scheme or layout.

Syntax overview

To establish a container element as a scroll-state query container, set the `container-type` property on it with a value of `scroll-state`. You can optionally also give it a `container-name`, so that you can target it with a specific container query:

```
.container {
  container-type: scroll-state;
  container-name: my-container;
}
```

You can then create a `@container` block that specifies the query, the rules that are applied to children of the container if the test passes, and optionally, the `container-name` of the container(s) you want to query. If you don't specify a `container-name`, the container query will be applied to all scroll-state query containers on the page.

Here, we query only containers named `my-container` to determine whether the container can be scrolled towards its top edge:

```
@container my-container scroll-state(scrollable: top) {
  /* CSS rules go here */
}
```

> **Note:** To separate scroll-state queries from other container queries, the scroll-state descriptors and value are placed inside parentheses, preceded by `scroll-state` (`scroll-state( ... )`). These constructs look like functions, but they're not.

Using <code>scrollable</code> queries

Scroll-state `scrollable` queries, written as `scroll-state(scrollable: <keyword>)`, test whether a container's scrolling ancestor can be scrolled in the given direction via user-initiated scrolling. If not, the query returns false.

The keyword value indicates the direction you are testing for scrolling availability in, for example:

- `top`: Tests whether the container can be scrolled towards its top edge.
- `inline-end`: Tests whether the container can be scrolled towards its inline-end edge.
- `y`: Tests whether the container can be scrolled in either or both directions along its y axis.

If the test passes, the rules inside the `@container` block are applied to descendants of the matching scroll container.

Let's look at an example in which we have a scrolling container full of content, and a handy little link to scroll back to the top if wished. We will use a `scrollable` query to only show the link when the user has started to scroll down through the content.

HTML

In the HTML we have an `<article>` element containing enough content to cause the document to scroll, preceded by a back-to-top link:

```
<a class="back-to-top" href="#" aria-label="Top of page">↑</a>
<article>
  <h1>Reader with container query-controlled "back-to-top" link</h1>
  <section>
    <header>
      <h2>This first section is interesting</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
    </header>

    ...
  </section>

  ...
</article>
```

We have hidden most of the HTML for brevity.

```
<a class="back-to-top" href="#" aria-label="Scroll back to top">↑</a>
<article>
  <h1>Reader with container query-controlled "back-to-top" link</h1>
  <section>
    <header>
      <h2>This first section is interesting</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
    </header>

    <p>
      Mauris non malesuada est, sed vestibulum nibh. Duis vestibulum iaculis
      lectus, eu sagittis dolor dignissim iaculis. Nunc et orci sed sapien
      eleifend placerat. Curabitur dapibus risus eget odio sollicitudin, sit
      amet luctus justo pellentesque.
    </p>

    <p>
      <strong>Morbi non pharetra quam.</strong> Fusce vestibulum sem diam, ac
      consequat augue consectetur ut. Donec at augue viverra, tempus urna sit
      amet, porta augue.
      <em>Phasellus fringilla tincidunt sem ullamcorper varius.</em> Aenean
      gravida feugiat sem nec ultricies.
    </p>

    <img src="#" alt="Placeholder" />

    <p>
      Sed pellentesque placerat mi sed maximus. Sed vitae dui vitae mi pulvinar
      gravida sed et libero.
      <a href="#">Duis nec venenatis dolor, sed tristique felis.</a>
      Integer dapibus facilisis leo elementum vulputate. Curabitur a urna quis
      nulla vulputate tincidunt quis ac enim.
    </p>

    <p>
      Cras non elit vel leo dignissim convallis. Duis eros urna, varius sit amet
      lorem vel, feugiat euismod est.
      <strong>Aliquam ornare eu elit ut iaculis.</strong>
      Suspendisse vulputate tempor leo, non rhoncus risus aliquam vel.
    </p>
  </section>
  <section>
    <header>
      <h2>This one, not so much</h2>

      <p>Suspendisse varius est ac turpis mollis cursus.</p>
    </header>

    <p>
      <strong
        >Curabitur faucibus condimentum eros, ut auctor felis lacinia
        sed.</strong
      >
      Praesent vitae scelerisque eros.
    </p>

    <p>
      <em>Ut vitae suscipit augue.</em> Cras et orci condimentum ante dignissim
      iaculis. Sed consectetur quis est sed dignissim. Nulla egestas orci erat,
      et commodo arcu feugiat ut.
    </p>

    <img src="#" alt="Placeholder" />

    <p>
      Sed non tempor massa, at accumsan ante. Pellentesque habitant morbi
      <a href="#">tristique senectus</a> et netus et malesuada fames ac turpis
      egestas.
    </p>

    <p>
      Pellentesque placerat luctus tempor. Nunc congue dapibus eros, at
      vulputate nulla. Sed rutrum eleifend magna vel porta. Integer cursus orci
      faucibus turpis scelerisque, nec pharetra arcu molestie.
    </p>
  </section>
  <section>
    <header>
      <h2>Hopefully this section provides some clarity?</h2>

      <p>Curabitur facilisis ornare lorem et eleifend.</p>
    </header>

    <p>
      <strong>Aenean mollis non neque sed finibus.</strong> Lorem ipsum dolor
      sit amet, consectetur adipiscing elit. Suspendisse sagittis viverra urna.
      In hac habitasse platea dictumst. Vestibulum neque orci, mollis sagittis
      augue et, pharetra vehicula diam.
    </p>

    <img src="#" alt="Placeholder" />

    <p>
      <a href="#">Pellentesque sollicitudin</a> nunc quis nisl condimentum, ac
      iaculis libero feugiat.
      <strong>Nullam ultrices purus a nulla dignissim hendrerit.</strong> In
      molestie consectetur est quis pulvinar.
    </p>

    <p>
```

```
      Vivamus ac erat eu est lobortis commodo. Orci varius natoque penatibus et
      magnis dis parturient montes, nascetur ridiculus mus. In nulla turpis,
      <strong>mollis et est tempor</strong>, dignissim aliquam metus. Proin eu
      arcu quis erat mollis pulvinar. Vivamus at facilisis neque.
    </p>

    <p>
      Integer bibendum laoreet erat, quis vulputate mauris bibendum nec. Class
      aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos
      himenaeos. Nam ut est in arcu interdum hendrerit.
    </p>
  </section>
  <section>
    <header>
      <h2>A summary of sorts</h2>

      <p>Nunc facilisis augue quis ex porta aliquam.</p>
    </header>

    <img src="#" alt="Placeholder" />

    <p>
      <strong
        >Fusce nisi enim, venenatis a est vel, varius placerat lacus.</strong
      >
      Nunc tempus rutrum nisl bibendum aliquet. Pellentesque vitae nunc sed nisl
      tincidunt elementum a sit amet nisi. Morbi pretium at dolor in pulvinar.
      Curabitur dapibus eleifend accumsan.
    </p>

    <p>
      Donec rhoncus, leo vitae mollis maximus, tellus lorem interdum arcu, eu
      <em>tempor lectus libero in risus</em>. Ut sit amet magna vitae mauris
      tempor bibendum. <a href="#">Integer id mauris ut ex mattis finibus.</a>
    </p>

    <p>
      Curabitur dui felis, elementum et tellus id, blandit facilisis lorem.
      Aliquam sed posuere ligula, at auctor ipsum. Morbi dignissim accumsan
      tellus pretium iaculis.
    </p>
  </section>
</article>
```

CSS

The `.back-to-top` link is given a `position` value of `fixed`, placed at the bottom-right corner of the viewport, and moved off the viewport using a `translate` value of `80px 0`. A `transition` value will animate the `translate` and `background-color` when either value changes.

```
/* General styling */

* {
  box-sizing: border-box;
}

html {
  font-family: "Helvetica", "Arial", sans-serif;
  height: 100%;
}

body {
  height: inherit;
  width: 100%;
  max-width: 540px;
  padding: 20px;
  margin: 0 auto;
}

p {
  line-height: 1.5;
}

img {
  display: block;
  width: 90%;
  margin: 30px auto;
  padding: 20px;
  border: 2px solid gray;
  aspect-ratio: 3/2;
}
```

```
.back-to-top {
  width: 64px;
  height: 64px;
  color: white;
  text-align: center;
  position: fixed;
  bottom: 10px;
  right: 10px;
  translate: 80px 0;
  transition:
    0.4s translate,
    0.2s background-color;
}
```

```
.back-to-top {
  text-decoration: none;
  border-radius: 50%;
  border: 1px solid #00000077;
  background-color: #00000077;
  color: white;
```

```
  font-size: 3rem;
  text-shadow: 0 0 2px black;
  padding-bottom: 10px;
}

.back-to-top:hover,
.back-to-top:focus {
  background: #00000099;
}
```

The scroll container in this example is the `<html>` element itself, denoted as a scroll-state query container with a `container-type` value of `scroll-state`. The `container-name` isn't strictly necessary but is useful in cases where the code is added to a codebase with multiple scroll-state query containers targeted with different queries.

```
html {
  container-type: scroll-state;
  container-name: scroller;
}
```

Next, we define a `@container` block that sets the container name targeted by this query, and the query itself — `scrollable: top`. This query applies the rules contained inside the block only if the `<html>` element can be scrolled toward its top edge — in other words, if the container has previously been scrolled downwards. If that is the case, `translate: 0 0` is applied to the `.back-to-top` link, which transitions it back on-screen.

```
@container scroller scroll-state(scrollable: top) {
  .back-to-top {
    translate: 0 0;
  }
}
```

We've hidden the rest of the example CSS for brevity.

Result

Try scrolling the document down, and note how the "back-to-top" link appears as a result, animating smoothly from the right side of the viewport due to the `transition`. If you scroll back to the top by activating the link or manually scrolling, the "back-to-top" link transitions off-screen.

Using <code>scrolled</code> queries

Scroll-state `scrolled` queries, written as `scroll-state(scrolled: <keyword>)`, test whether a container's scrolling ancestor was most recently scrolled in the given direction. If not, the query returns false.

The keyword value indicates the direction you are testing. For example:

- `block-start`: Tests whether the container was most recently scrolled towards its block-start edge.
- `right`: Tests whether the container was most recently scrolled towards its right-hand edge.
- `y`: Tests whether the container was most recently scrolled up or down along the y axis.
- `none`: Tests whether the container is not a scroll container or has not been scrolled in any direction since rendering.

If the test returns true, the rules inside the `@container` block are applied to the descendants of the matching scroll container.

Let's look at an example of a scroll container with a `scrolled` query that displays top and bottom content "bars" only when the user is scrolling up or down, respectively.

HTML

In our HTML, we have an `<article>` element containing enough content to cause the document to scroll, preceded by two `<div>` elements that represent our top and bottom "bars":

```
<div class="bar" id="top-bar">You're currently scrolling towards the top.</div>
<div class="bar" id="bottom-bar">
  You're currently scrolling towards the bottom.
</div>
<article>
  <h1>Document with scrolled container query</h1>
  <section>
    <header>
      <h2>This first section is interesting</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
    </header>

    ...
  </section>

  ...
</article>
```

We have hidden most of the HTML for brevity.

```
<div class="bar" id="top-bar">You're currently scrolling towards the top.</div>
<div class="bar" id="bottom-bar">
  You're currently scrolling towards the bottom.
</div>
<article>
  <h1>Document with scrolled container query</h1>
  <section>
    <header>
      <h2>This first section is interesting</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
    </header>

    <p>
      Mauris non malesuada est, sed vestibulum nibh. Duis vestibulum iaculis
      lectus, eu sagittis dolor dignissim iaculis. Nunc et orci sed sapien
      eleifend placerat. Curabitur dapibus risus eget odio sollicitudin, sit
      amet luctus justo pellentesque.
    </p>

    <p>
      <strong>Morbi non pharetra quam.</strong> Fusce vestibulum sem diam, ac
      consequat augue consectetur ut. Donec at augue viverra, tempus urna sit
      amet, porta augue.
```

```
    <em>Phasellus fringilla tincidunt sem ullamcorper varius.</em> Aenean
    gravida feugiat sem nec ultricies.
  </p>

  <img src="#" alt="Placeholder" />

  <p>
    Sed pellentesque placerat mi sed maximus. Sed vitae dui vitae mi pulvinar
    gravida sed et libero.
    <a href="#">Duis nec venenatis dolor, sed tristique felis.</a>
    Integer dapibus facilisis leo elementum vulputate. Curabitur a urna quis
    nulla vulputate tincidunt quis ac enim.
  </p>

  <p>
    Cras non elit vel leo dignissim convallis. Duis eros urna, varius sit amet
    lorem vel, feugiat euismod est.
    <strong>Aliquam ornare eu elit ut iaculis.</strong>
    Suspendisse vulputate tempor leo, non rhoncus risus aliquam vel.
  </p>
</section>
<section>
  <header>
    <h2>This one, not so much</h2>

    <p>Suspendisse varius est ac turpis mollis cursus.</p>
  </header>

  <p>
    <strong
      >Curabitur faucibus condimentum eros, ut auctor felis lacinia
      sed.</strong
    >
    Praesent vitae scelerisque eros.
  </p>

  <p>
    <em>Ut vitae suscipit augue.</em> Cras et orci condimentum ante dignissim
    iaculis. Sed consectetur quis est sed dignissim. Nulla egestas orci erat,
    et commodo arcu feugiat ut.
  </p>

  <img src="#" alt="Placeholder" />

  <p>
    Sed non tempor massa, at accumsan ante. Pellentesque habitant morbi
    <a href="#">tristique senectus</a> et netus et malesuada fames ac turpis
    egestas.
  </p>

  <p>
    Pellentesque placerat luctus tempor. Nunc congue dapibus eros, at
    vulputate nulla. Sed rutrum eleifend magna vel porta. Integer cursus orci
    faucibus turpis scelerisque, nec pharetra arcu molestie.
  </p>
</section>
<section>
  <header>
    <h2>Hopefully this section provides some clarity?</h2>

    <p>Curabitur facilisis ornare lorem et eleifend.</p>
  </header>

  <p>
    <strong>Aenean mollis non neque sed finibus.</strong> Lorem ipsum dolor
    sit amet, consectetur adipiscing elit. Suspendisse sagittis viverra urna.
    In hac habitasse platea dictumst. Vestibulum neque orci, mollis sagittis
    augue et, pharetra vehicula diam.
  </p>

  <img src="#" alt="Placeholder" />

  <p>
    <a href="#">Pellentesque sollicitudin</a> nunc quis nisl condimentum, ac
    iaculis libero feugiat.
    <strong>Nullam ultrices purus a nulla dignissim hendrerit.</strong> In
    molestie consectetur est quis pulvinar.
  </p>

  <p>
    Vivamus ac erat eu est lobortis commodo. Orci varius natoque penatibus et
    magnis dis parturient montes, nascetur ridiculus mus. In nulla turpis,
    <strong>mollis et est tempor</strong>, dignissim aliquam metus. Proin eu
    arcu quis erat mollis pulvinar. Vivamus at facilisis neque.
  </p>

  <p>
    Integer bibendum laoreet erat, quis vulputate mauris bibendum nec. Class
    aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos
    himenaeos. Nam ut est in arcu interdum hendrerit.
  </p>
</section>
<section>
  <header>
    <h2>A summary of sorts</h2>

    <p>Nunc facilisis augue quis ex porta aliquam.</p>
  </header>

  <img src="#" alt="Placeholder" />

  <p>
    <strong
```

```
        >Fusce nisi enim, venenatis a est vel, varius placerat lacus.</strong
      >
      Nunc tempus rutrum nisl bibendum aliquet. Pellentesque vitae nunc sed nisl
      tincidunt elementum a sit amet nisi. Morbi pretium at dolor in pulvinar.
      Curabitur dapibus eleifend accumsan.
    </p>

    <p>
      Donec rhoncus, leo vitae mollis maximus, tellus lorem interdum arcu, eu
      <em>tempor lectus libero in risus</em>. Ut sit amet magna vitae mauris
      tempor bibendum. <a href="#">Integer id mauris ut ex mattis finibus.</a>
    </p>

    <p>
      Curabitur dui felis, elementum et tellus id, blandit facilisis lorem.
      Aliquam sed posuere ligula, at auctor ipsum. Morbi dignissim accumsan
      tellus pretium iaculis.
    </p>
  </section>
</article>
```

CSS

The "bars" are given some rudimentary styling. Most significantly, they are given a `position` value of `fixed`, which we offset from either side using `left` and `right` values.

```
/* General styling */

* {
  box-sizing: border-box;
}

html {
  font-family: "Arial", sans-serif;
  height: 100%;
}

body {
  height: inherit;
  width: 90%;
  margin: 0 auto;
}

p {
  line-height: 1.5;
}

img {
  display: block;
  width: 90%;
  margin: 30px auto;
  padding: 20px;
  border: 2px solid gray;
  aspect-ratio: 3/2;
}
```

```
.bar {
  border-radius: 10px;
  border: 1px solid black;
  background-color: #00000099;
  padding: 10px;
  color: white;
  text-shadow: 1px 1px 1px black;
  display: flex;
  justify-content: center;
  align-items: center;

  position: fixed;
  left: 5px;
  right: 5px;
}
```

Next, we set negative `top` and `bottom` length values on the top and bottom bars so they are hidden above and below the viewport by default. We add a `transition` to smoothly animate them into view when their `translate` values change.

```
#top-bar {
  top: -50px;
  transition: 0.6s translate;
}

#bottom-bar {
  bottom: -50px;
  transition: 0.6s translate;
}
```

The scroll container in this example is the `<html>` element itself, denoted as a scroll-state query container with a `container-type` value of `scroll-state`. The `container-name` isn't strictly necessary, but it is useful when a codebase has multiple scroll-state query containers targeted with different queries.

```
html {
  container-type: scroll-state;
  container-name: scroller;
}
```

Next, we define two `@container` blocks, both of which target the `scroller` container name. The first block defines a query `scrolled: block-end` and the second defines a query `scrolled: block-start`. Respectively, these queries apply the rules contained within their block only if the `<html>` element was most recently scrolled toward its block-end edge or block-start edge. In other words, when the container is scrolled down or up. When either condition becomes true, the bar referenced inside the block has a `translate` value set to cause it to transition on-screen. The bar referenced in the `@condition` that is no longer true transitions off-screen.

```
@container scroller scroll-state(scrolled: block-start) {
  #top-bar {
```

```
    translate: 0 55px;
  }
}

@container scroller scroll-state(scrolled: block-end) {
  #bottom-bar {
    translate: 0 -55px;
  }
}
```

We've hidden the rest of the example CSS for brevity.

Result

Try scrolling the document up and down, and note how the different bars appear as a result, animating smoothly on- and off-screen.

Using <code>snapped</code> queries

Relevant only when scroll snapping is implemented, scroll-state `snapped` queries, written as `scroll-state(snapped: <keyword>)`, test whether a container is going to be snapped to a scroll snap container ancestor along the given axis. If not, the query returns false.

The keyword value in this case indicates the direction you are testing the element's ability to snap in, for example:

- `x`: Tests whether the container is snapping horizontally to its scroll-snap container ancestor.
- `inline`: Tests whether the container is snapping to its scroll-snap container ancestor in the inline direction.
- `y`: Tests whether the container is snapping to its scroll-snap container ancestor in both directions.

To evaluate a container with a non-`none` `snapped` scroll-state query, it must be a container with a scroll-snap container ancestor, that is, the ancestor has a `scroll-snap-type` value other than `none`. The container query `scroll-state(snapped: none)` matches scroll-state containers that do not have a scroll container ancestor.

Evaluation will occur when the `scrollsnapchanging` event fires on the scroll snap container.

If the test passes, the rules inside the `@container` block are applied to descendants of the matching scroll snap target container.

In this example, we'll look at a scroll snap container with children that snap to it vertically and use a `snapped` query to style the children only when they are snapped or about to be snapped.

HTML

The HTML consists of a `<main>` element that will be a scroll snap container. Inside are several `<section>` elements that will be snap targets. Each `<section>` contains a wrapper `<div>` and an `<h2>` heading. The wrappers are included to create a style target as container queries enable styling a container's descendants, not the container itself.

```
<main>
  <section>
    <div class="wrapper">
      <h2>Section 1</h2>
    </div>
  </section>

  ...
</main>
```

We have hidden most of the HTML for brevity.

```
<main>
  <section>
    <div class="wrapper">
      <h2>Section 1</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 2</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 3</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 4</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 5</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 6</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 7</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 8</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 9</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
```

```
      <h2>Section 10</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 11</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 12</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 13</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 14</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 15</h2>
    </div>
  </section>
  <section>
    <div class="wrapper">
      <h2>Section 16</h2>
    </div>
  </section>
</main>
```

CSS

```
* {
  box-sizing: border-box;
}

html {
  height: 100%;
}

/* body and main sizing */

body {
  display: flex;
  align-items: center;
  justify-content: center;
  height: inherit;
}

main {
  gap: 50px;
}
```

We set an `overflow` value of `scroll` and a fixed `height` on the `<main>` element to turn it into a vertical scroll container. We also set a `scroll-snap-type` value of `y mandatory` to turn `<main>` into a scroll snap container that snap targets will snap to along the y axis; `mandatory` means that a snap target will *always* be snapped to.

```
main {
  overflow: scroll;
  scroll-snap-type: y mandatory;
  height: 450px;
  width: 250px;
  border: 3px solid black;
}
```

The `<section>` elements are designated as snap targets by setting a non-`none` `scroll-snap-align` value. The `center` value means that they will snap to the container at their center points.

```
section {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 150px;
  height: 150px;
  margin: 50px auto;

  scroll-snap-align: center;
}
```

```
.wrapper {
  width: 100%;
  height: 100%;
  border-radius: 5px;
  background: #eeeeee;
  box-shadow:
    inset 1px 1px 4px rgb(255 255 255 / 0.5),
    inset -1px -1px 4px rgb(0 0 0 / 0.5);

  display: flex;
  align-items: center;
  justify-content: center;

  transition:
    0.6s background,
    0.6s color;
}

h2 {
  font-size: 1rem;
```

```
  letter-spacing: 1px;
}
```

We want to enable the `<section>` elements to be queried. Specifically, we want to test whether the `<section>` elements are in the process of snapping to their container, so we denote them as scroll-state query containers by setting a `container-type` value of `scroll-state` on them. We also give them a `container-name`, which isn't strictly necessary, but will be useful if our code gets more complex later and we have multiple scroll-state query containers that we want to target with different queries.

```
section {
  container-type: scroll-state;
  container-name: snap-container;
}
```

Next, we define a `@container` block that sets the container name we are targeting with this query, and the query itself — `snapped: y`. This query applies the rules contained inside the block only if a `<section>` element is being snapped vertically to its container. If that is the case, we apply a new `background` and `color` to the `<section>` element's child `.wrapper` `<div>` to highlight it.

```
@container snap-container scroll-state(snapped: y) {
  .wrapper {
    background: purple;
    color: white;
  }
}
```

Result

The rendered result is shown below. Try scrolling the container up and down, and note how the `<section>` style changes when it becomes snapped to its container.

Using <code>stuck</code> queries

Scroll-state `stuck` queries, written as `scroll-state(stuck: <keyword>)`, test whether a container with a `position` value of `sticky` is stuck to an edge of its scroll container ancestor. If not, the query returns false.

The keyword value in this case indicates the scroll container edge you are testing, for example:

- `top`: Tests whether the container is stuck to the top edge of its scroll container ancestor.
- `block-end`: Tests whether the container is stuck to the block-end edge of its scroll container ancestor.
- `none`: Tests whether the container is not stuck to any edges of its scroll container ancestor. Note that `none` queries will match even if the container does not have `position: sticky` set on it.

If the query returns true, the rules inside the `@container` block are applied to descendants of the matching `position: sticky` container.

Let's look at an example where we have a scrolling container with overflowing content, in which the headings are set to `position: sticky` and stick to the top edge of the container when they scroll to that position. We will use a `stuck` scroll-state query to style the headings differently when they are stuck to the top edge.

HTML

In the HTML, we have an `<article>` element containing enough content to cause the document to scroll. It is structured using several `<section>` elements, each containing a `<header>` with nested content:

```
<article>
  <h1>Sticky reader with scroll-state container query</h1>
  <section>
    <header>
      <h2>This first section is interesting</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
    </header>

    ...
  </section>

  <section>
    <header>
      <h2>This one, not so much</h2>

      <p>Confecta res esset.</p>
    </header>

    ...
  </section>

  ...
</article>
```

We have hidden most of the HTML for brevity.

```
<article>
  <h1>Sticky reader with scroll-state container query</h1>
  <section>
    <header>
      <h2>This first section is interesting</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
    </header>

    <p>
      Id deserunt exercitation in incididunt in dolor ipsum enim occaecat quis
      officia et. In dolore Lorem anim enim. Dolor nulla deserunt laboris irure
      incididunt labore tempor amet consectetur. Commodo eu tempor qui laboris
      duis ipsum adipisicing sunt. Do anim laboris commodo aliquip consequat ut
      eiusmod mollit sint nostrud sunt exercitation do.
    </p>

    <p>
      <strong>Est proident voluptate dolore.</strong> Aute irure ea culpa mollit
      <em>anim aute proident consectetur.</em> Duis deserunt consequat elit non.
      Cupidatat et amet anim labore est. Incididunt aute incididunt ex amet
      sint.
    </p>

    <img src="#" alt="Placeholder" />

    <p>
```

```
    Ullamco duis qui amet sunt eu nulla deserunt dolore officia irure nulla.
    <a href="#">Qui enim dolore est virtus;</a>
    Erat enim consectetur.
  </p>

  <p>
    Dolor aliquip officia id esse Lorem ullamco ea veniam.
    <strong>Voluptate cillum commodo anim veniam;</strong>
    Ad nulla amet anim eu incididunt reprehenderit proident in aliqua sit
    commodo magna.
  </p>
</section>
<section>
  <header>
    <h2>This one, not so much</h2>

    <p>Confecta res esset.</p>
  </header>

  <p>
    <strong>Nam quid possums facere delius?</strong> Tu quidem reddes; Omnis
    peccant para dicts.
  </p>

  <p>
    <em>Pariatur ex cupidatat exercitation</em> reprehenderit quis culpa est
    proident pariatur ut.
  </p>

  <img src="#" alt="Placeholder" />

  <p>
    Non nulla pariatur cillum pariatur.
    <a href="#">Ipsum magna proident fugiat qui duis et.</a>
    Reprehenderit sit ipsum ut adipisicing aliqua eiusmod nulla consectetur
    deserunt dolor dolor.
  </p>

  <p>
    Cupidatat deserunt adipisicing adipisicing occaecat voluptate duis laboris
    excepteur labore et in aliquip aute. Quis aliquip reprehenderit dolor
    nostrud cupidatat nostrud nisi occaecat elit. Magna dolor ullamco eiusmod
    deserunt elit cillum qui amet. Dolor ad consequat anim commodo eu sunt do
    tempor eu velit est.
  </p>
</section>
<section>
  <header>
    <h2>Hopefully this section provides some clarity?</h2>

    <p>Dolor consectetur esse aute eiusmod ex reprehenderit cupidatat.</p>
  </header>

  <p>
    <strong>Ipsum quis sint incididunt aliquip</strong>. Duis tempor sunt enim
    Lorem ut officia cillum magna.
  </p>

  <img src="#" alt="Placeholder" />

  <p>
    <a href="#">Sint modo partes vitae beatae.</a>
    Ipsum esse nostrud incididunt ipsum ipsum esse aliquip Lorem enim.
    <strong>Sed hoc sane concedays.</strong> Voluptate qui consequat dolor
    dolor dolore ex. Enim dolor enim sit proident. Ad laborum incididunt
    dolore aliquip exercitation exercitation. Sunt cupidatat esse enim duis
    proident.
  </p>

  <p>
    Labore dolor cupidatat ut velit in minim cupidatat enim nisi proident
    eiusmod.
    <strong
      >Non nostrud consectetur est occaecat ut esse esse tempor eiusmod
      non.</strong
    >
    Cupidatat aliqua dolore est sit ad exercitation nostrud nostrud eu. Qui
    consequat et aliquip ea labore.
  </p>

  <p>
    Pariatur quis quis nostrud exercitation pariatur magna veniam Lorem
    commodo cupidatat irure reprehenderit. Nostrud pariatur est pariatur elit
    sit ea voluptate minim mollit. Aliqua pariatur cupidatat et cillum. Velit
    exercitation adipisicing laboris ullamco. Incididunt id eiusmod id cillum
    qui anim. Aliquip minim nostrud excepteur dolore deserunt pariatur eu
    eiusmod reprehenderit ullamco magna.
  </p>
</section>
<section>
  <header>
    <h2>A summary of sorts</h2>

    <p>Irure adipisicing occaecat dolore in ex voluptate.</p>
  </header>

  <img src="#" alt="Placeholder" />

  <p>
    <strong>An quidem modi?</strong>
    <strong
      >Excepteur non Lorem ex aliqua est reprehenderit incididunt sunt in
      ullamco in et;</strong
```

```
      >
      Aliqua veniam nulla magna anim labore amet proident nisi mollit non.
      Consequat incididunt exercitation nisi consequat culpa officia aliqua
      veniam. Anim aliqua consectetur pariatur mollit proident incididunt id.
    </p>

    <p>
      Non labore, inquit, de nomin. <em>Fall igniter possimus.</em>
      <a href="#">Preterits, inquit, gaud.</a>
      Est commodo adipisicing qui duis excepteur sit ea irure amet voluptate
      deserunt anim deserunt.
    </p>

    <p>
      Ad mollit laboris reprehenderit magna tempor. Consequat pariatur esse
      ipsum duis laboris.
    </p>
  </section>
</article>
```

CSS

Each `<header>` has a `position` value of `sticky` and a `top` value of `0`, which sticks them to the top edge of the scroll container. To test whether the `<header>` elements are stuck to the container top edge, they are denoted as scroll-state query containers with a `container-type` value of `scroll-state`. The `container-name` isn't strictly necessary but will be useful if this code gets added to a code base with multiple scroll-state query containers targeted with different queries.

```
/* General styling */

* {
  box-sizing: border-box;
}

html {
  font-family: "Helvetica", "Arial", sans-serif;
  height: 100%;
}

body {
  height: inherit;
  width: 100%;
  max-width: 540px;
  padding: 20px;
  margin: 0 auto;
}

p {
  line-height: 1.5;
}

img {
  display: block;
  width: 90%;
  margin: 30px auto;
  padding: 20px;
  border: 2px solid gray;
  aspect-ratio: 3/2;
}
```

```
header {
  background: white;
  position: sticky;
  top: 0;
  container-type: scroll-state;
  container-name: sticky-heading;
}
```

We also give the `<h2>` and `<p>` elements inside the `<header>` elements some basic styling, and a `transition` value so they will smoothly animate when their `background` values change.

```
h2,
header p {
  margin: 0;
  transition: 0.4s background;
}

h2 {
  padding: 20px 5px;
  margin-bottom: 10px;
}

header p {
  font-style: italic;
  padding: 10px 5px;
}
```

Next, we define a `@container` block that sets the container name we are targeting with this query, and the query itself — `stuck: top`. This query applies the rules contained inside the block only if a `<header>` element is stuck to the top of its scroll container. When that is the case, a different `background` and a `box-shadow` are applied to the contained `<h2>` and `<p>`.

```
@container sticky-heading scroll-state(stuck: top) {
  h2,
  p {
    background: #cccccc;
    box-shadow: 0 5px 2px #00000077;
  }
}
```

```
@supports not (container-type: scroll-state) {
  body::before {
    content: "Your browser does not support `scroll-state` container queries.";
    color: black;
```

```
    background-color: wheat;
    position: fixed;
    left: 0;
    right: 0;
    top: 40%;
    text-align: center;
    padding: 1rem 0;
    z-index: 1;
  }
}
```

We have hidden the rest of the CSS for brevity.

Result

Try scrolling the document down and up, and note how the `<h2>` and `<p>` elements transition to a new color scheme when they become stuck to the top of their container's top edge.

See also

- `container-name`
- `container-type`
- `position`
- `@container`
- CSS container queries
- Using container size and style queries
- CSS conditional rules module
- CSS positioning module

Using feature queries

Feature queries are conditional group rules that test whether the user agent supports or doesn't support one or more CSS features, such as CSS properties and property values. Feature queries give web developers a way to test to see if a browser has support for a certain feature, and then provide CSS that will only run based on the result of that test. In this guide, you will learn how to implement progressive enhancement using feature queries.

Feature queries are created using the CSS at-rule `@supports` (or the `supports()` function within `@import` at-rules).

Syntax

CSS feature queries are part of the CSS conditional rules module, which also defines the media query `@media` at-rule. Feature queries behave similarly to media queries. The difference is that with a media query, you are testing something about the environment in which the web page is running, whereas with feature queries you are testing browser support for CSS features.

A feature query consists of the `@supports` at-rule followed by the support condition or a `supports()` function and declaration parameter within an `@import` at-rule declaration:

```
/* `@supports` at-rule */
@supports <support-condition> {
  /* CSS rules to apply */
}

/* `supports()` function */
@import url_to_import supports(<declaration>);
```

For example, we can apply a set of styles or import an entire stylesheet if the user-agent supports `red` as a valid value for the CSS `color` property:

```
/* `@supports` at-rule */
@supports (color: red) {
  /* CSS rules to apply */
}

/* `supports()` function */
@import "/css/styles.css" supports(color: red);
```

As another example, if you want to check if a browser supports the `row-gap` property you would write the following feature query. It doesn't matter which value you use in a lot of cases: if all you want is to check that the browser supports this property, then any valid value will do.

```
<div class="box">
  If your browser supports the row-gap property, the border will be dashed and
  text will be red.
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  border: 4px solid blue;
  color: blue;
  padding: 1em;
}
@supports (row-gap: 10px) {
  .box {
    border: 4px dashed darkgreen;
    color: red;
  }
}
```

The value part of the property-value pair matters more if you are testing for new values of a particular property. All browsers support `color: red`: this dates back to CSS1. However, there are often additional values added to properties in CSS, like relative colors, which may not be supported. Feature queries enable testing property and value pairs, meaning we can detect support for values.

Expanding on the `color` property example above, here we check if the browser supports the `color: AccentColor` declaration:

```
/* `@supports` at-rule */
@supports (color: AccentColor) {
  /* CSS rules to apply */
}

/* `supports()` function */
@import "/css/styles.css" supports(color: AccentColor);
```

In these examples, we've used feature queries to check if the user-agent supports a specific value of a CSS property, listing the single declaration within parenthesis. You can test for multiple property values or the lack of support.

Testing for lack of support

In addition to asking the browser if it supports a feature, you can test for the opposite by adding in the `not` keyword:

```
/* `@supports` at-rule with `not` */
@supports not (property: value) {
  /* CSS rules to apply */
}
```

The CSS inside the following example feature query will run if the browser does not support `row-gap`.

```
<div class="box">
  If your browser does not support row-gap, the content will be darkgreen with a
  dashed border.
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  border: 4px solid blue;
  color: blue;
  padding: 1em;
}
@supports not (row-gap: 10px) {
  .box {
    border: 4px dashed darkgreen;
    color: darkgreen;
  }
}
```

Testing for more than one feature

You may need to test support for more than one property in your feature query. To do so, you can include a list of features to test for, separated by `and` keywords:

```
/* multiple feature `@supports` at-rule */
@supports (property1: value) and (property2: value) {
  /* CSS rules to apply */
}
```

For example, if the CSS you want to run requires that the browser supports CSS Shapes and CSS grid, you could create a rule that tests browser support for both of these features. The following rule will only return true if `shape-outside: circle()` and `display: grid` are both supported by the browser.

```
<div class="box">
  If your browser supports <code>display: grid</code> and
  <code>shape-outside: circle()</code>, the content will be darkgreen with a
  dashed border.
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  border: 4px solid blue;
  color: blue;
  padding: 1em;
}
@supports (display: grid) and (shape-outside: circle()) {
  .box {
    border: 4px dashed darkgreen;
    color: darkgreen;
  }
}
```

Testing for at least one of multiple features

You can also use `or` to apply CSS only if one or more declarations are supported:

```
/* any feature `@supports` at-rule */
@supports (property1: value) or (property2: value) {
  /* CSS rules to apply */
}
```

This can be particularly useful if a feature is vendor prefixed, as you can test for the standard property plus any vendor prefixes.

```
<div class="box">
  The text and border will be green if your browser supports font smoothing.
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  border: 4px solid blue;
  color: blue;
  padding: 1em;
}
@supports (font-smooth: always) or (-webkit-font-smoothing: antialiased) {
  .box {
    border: 4px dashed darkgreen;
    color: darkgreen;
  }
}
```

Additional feature query options

Feature queries are not limited to property-value pairs. You can include `font-tech()`, `font-format()`, and `selector()` functions in your feature queries to selectively apply CSS based on whether the user-agent supports a specified font technology, font format, or selector syntax, respectively.

For example, the `selector()` function can be used to import a stylesheet for browsers that support a vendor-prefixed pseudo-element:

```
/* A `selector()` query within a `supports()` function */
@import "/css/webkitShadowStyles.css"
  supports(selector(::-webkit-inner-spin-button));
```

Examples

Browser support test

In this example, we check if the browser supports the `AccentColor` `<system-color>` and use `display: none` to change the default "not supported" message to a "supported" message if the color type is supported.

HTML

```
<p class="accentcolor">
  Your browser does <span>not</span> support <code>AccentColor</code> as a color
  value.
</p>
```

CSS

```
body {
  font: 1.2em / 1.5 sans-serif;
}
p {
  padding: 1em;
}
@supports (color: AccentColor) {
  p {
    color: green;
    border: 2px solid;
  }
  span {
    display: none;
  }
}
@supports not (color: AccentColor) {
  p {
    color: red;
  }
}
```

Result

Limitations of feature queries

The `@supports` rule tests to see if browsers can parse one or more property/value pairs, and therefore if they claim to support the associated feature(s). If the property/value pairs are understood by a browser it returns a positive response. Feature queries check that declarations are considered valid by a browser, but can't be used to check if it supports a feature properly without bugs or spec violations. Feature queries cannot test for *partial implementations*.

Summary

Feature queries are a useful tool for progressively enhancing a site. They enable you to provide a good solution for all browsers, and an enhanced solution for browsers that support newer properties and values.

You don't need to use feature queries to start using new CSS features; CSS error handling means the browser simply ignores CSS it does not yet recognize. However, feature queries are a useful alternative to fallback declarations, and enable writing code once that can eventually be supported everywhere.

See also

- CSS conditional rules module
- Using CSS media queries
- Supporting older browsers: feature queries
- Browser feature detection: CSS `@supports`

CSS containment

The **CSS containment** module defines containment and container queries.

Containment enables the isolation of page subtrees from the rest of the DOM. The browser can then improve performance by optimizing the rendering of these independent parts.

Container queries are similar to dimension media queries, except that the queries are based on the dimensions of a specific container element defined as a containment context, rather than on the dimensions of the viewport. Container queries enable querying a container's size, properties, property values, or even just its assigned `container-name` to conditionally apply CSS styles. When applying these conditional styles, you can use container query length units, which specify lengths relative to the dimensions of the query container. Additional properties are defined to enable establishing a specific element as a query container and giving it a specific name.

Reference

Properties

- `contain`
- `content-visibility`

Events

- `contentvisibilityautostatechange`

Interfaces

- `ContentVisibilityAutoStateChangeEvent`
  - `skipped` property
- `CSSContainerRule`
  - `CSSContainerRule.containerName`
  - `CSSContainerRule.containerQuery`

Guides

CSS container queries

A guide to using container queries with `@container`, including naming containment contexts.

Using CSS containment

Describes the basic aims of CSS containment and how to leverage `contain` and `content-visibility` for a better user experience.

Using container size and style queries

A guide to writing container size and style queries with `@container`, including style queries for custom properties, query syntax and names, and nesting container queries.

Related concepts

- Layout and the containing block
- Block formatting context
- CSS conditional rules module
  - `@container` at-rule
  - `container` property
  - `container-name` property
  - `container-type` property
- CSS media queries module
  - `@media` at-rule
  - CSS logical operators (`not`, `or`, and `and`)
- CSS transitions module
  - `@starting-style` at-rule
  - `transition-behavior` property
- CSS box sizing module
  - `aspect-ratio` property
  - `contain-intrinsic-size` shorthand property
  - `contain-intrinsic-inline-size` property
  - `contain-intrinsic-block-size` property
  - `contain-intrinsic-width` property
  - `contain-intrinsic-height` property
- CSS counter styles module
  - Using CSS counters guide
- CSS nesting module
  - CSS nesting at-rules guide

See also

- Using feature queries
- Using CSS media queries
- Understanding aspect ratios
- `@supports` at-rule

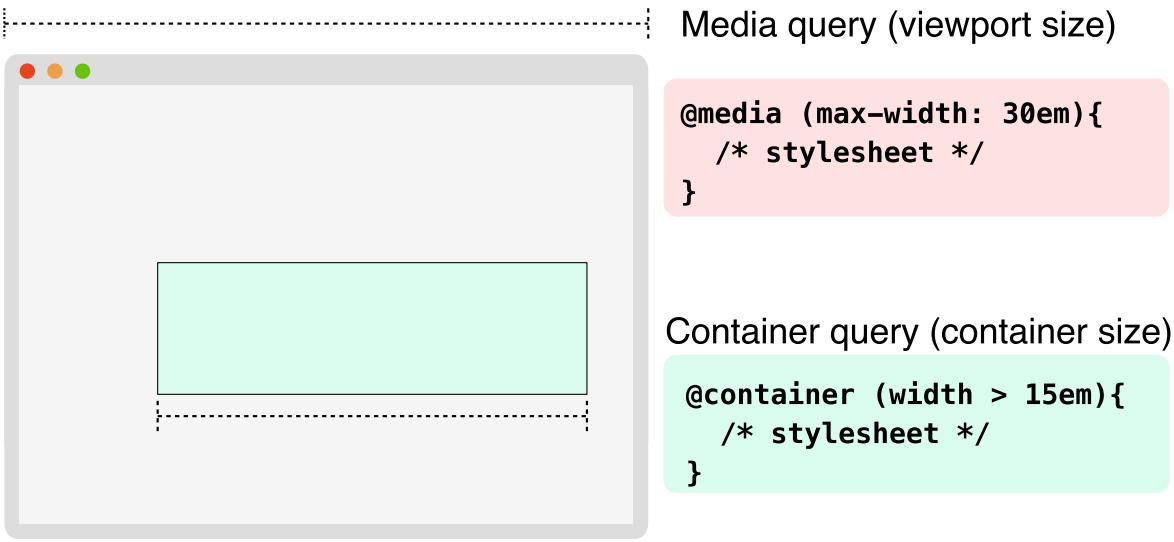
CSS container queries

Container queries enable you to apply styles to an element based on certain attributes of its container:

- The `container-name`.
- The container's size.
- Styles applied to the container.
- The container's scroll-state or that of its scrolling ancestor.
- Whether the container is anchor-positioned and has a position-try fallback option applied to it.

Container queries are an alternative to media queries, which apply styles to elements based on viewport size or other device characteristics.

This article provides an introduction to using container queries, specifically focusing on size container queries. Other guides discuss style, scroll-state, and anchored container queries in detail.



Using container size queries

While container queries apply styles based on the container name or type, container size queries apply styles specifically based on the container's dimensions. To use container size queries, you need to declare a containment context on an element so that the browser knows you might want to query the dimensions of this container later. To do this, use the `container-type` property with a value of `size`, `inline-size`, or `normal`.

These values have the following effects:

`size`

The query will be based on the inline and block dimensions of the container. Applies layout, style, and size containment to the container.

`inline-size`

The query will be based on the inline dimensions of the container. Applies layout, style, and inline-size containment to the element.

`normal`

The default value. The element is not a query container for any container size queries, but can still be used as a query container for name-only container queries or container style queries.

Consider the following example of a card component for a blog post with a title and some text:

```
<div class="post">
  <div class="card">
    <h2>Card title</h2>
    <p>Card content</p>
  </div>
</div>
```

You can create a containment context using the `container-type` property:

```
.post {
  container-type: inline-size;
}
```

Next, use the `@container` at-rule to define a container query. The query in the following example will apply styles to elements based on the size of the nearest ancestor with a containment context. Specifically, this query will apply a larger font size for the card title if the container is wider than `700px`:

```
/* Default heading styles for the card title */
.card h2 {
  font-size: 1em;
}

/* If the container is larger than 700px */
@container (width > 700px) {
  .card h2 {
    font-size: 2em;
  }
}
```

Using container queries, the card can be reused in multiple areas of a page without needing to know specifically where it will be placed each time. If the container with the card is narrower than `700px`, the font of the card title will be small, and if the card is in a container that's wider than `700px`, the font of the card title will be bigger.

For more information on the syntax of container queries, see the `@container` page.

Naming containment contexts

In the previous section, a container query applied styles based on the nearest ancestor with a containment context. It's possible to give a containment context a name using the `container-name` property. Once named, the name can be used in a `@container` query so as to target a specific container. The following example creates a containment context with the name `sidebar`:

```
.post {
  container-type: inline-size;
  container-name: sidebar;
}
```

You can then target this containment context using the `@container` at-rule:

```
@container sidebar (width > 700px) {
  .card {
    font-size: 2em;
  }
}
```

More information on naming containment contexts is available on the `container-name` page.

Name-only container queries

As well as using a `container-name` along with a `<container-query>`, you can query a container using just its name. These so-called **name-only container queries** enable selectively applying styles to elements based on whether they have an ancestor with a specific `container-name` set.

For example, consider the following HTML:

```
<div id="container">
  <p>I'm in the container.</p>
  <p>I'm also in the container.</p>
</div>
<p>I'm not in the container.</p>
```

If we assign a name to the container:

```
#container {
  container-name: my-container;
}
```

We can then selectively apply styles only to elements inside that container:

```
@container my-container {
  p {
    background-color: lime;
    font-size: 1.3rem;
    width: 50vw;
    padding: 0.5rem;
    font-family: sans-serif;
  }
}
```

In this example, the specified styles would be applied only to the first and second `<p>` elements, but not to the third.

Shorthand container syntax

The shorthand way of declaring a containment context is to use the `container` property:

```
.post {
  container: sidebar / inline-size;
}
```

For more information on this property, see the `container` reference.

Container query length units

When applying styles to the descendants of a container using size container queries (that is, its `container-type` is set to `size` or `inline-size`), you can use container query length units. These units specify a length relative to the dimensions of a query container. Components that use units of length relative to their container are more flexible to use in different containers without having to recalculate concrete length values.

If no eligible container is available for the query, the container query length unit defaults to the small viewport unit for that axis (`sv*`).

The container query length units are:

- `cqw`: 1% of a query container's width
- `cqh`: 1% of a query container's height
- `cqi`: 1% of a query container's inline size
- `cqb`: 1% of a query container's block size
- `cqmin`: The smaller value of either `cqi` or `cqb`
- `cqmax`: The larger value of either `cqi` or `cqb`

The following example uses the `cqi` unit to set the font size of a heading based on the inline size of the container:

```
@container (width > 700px) {
  .card h2 {
    font-size: max(1.5em, 1.23em + 2cqi);
  }
}
```

For more information on these units, see the Container query length units reference.

Fallbacks for container queries

For browsers that don't yet support container queries, `grid` and `flex` can be used to create a similar effect for the card component used on this page. The following example uses a `grid-template-columns` declaration to create a two-column layout for the card component.

```
.card {
  display: grid;
  grid-template-columns: 2fr 1fr;
}
```

If you want to use a single-column layout for devices with a smaller viewport, you can use a media query to change the grid template:

```
@media (width <= 700px) {
  .card {
    grid-template-columns: 1fr;
  }
}
```

See also

- Media queries
- CSS `@container` at-rule
- CSS `contain` property
- CSS `container` shorthand property
- CSS `container-name` property
- CSS `content-visibility` property
- Using container size and style queries
- Using container scroll-state queries
- Using anchored container queries
- Say Hello to CSS Container Queries by Ahmad Shadeed
- Container Queries: a Quick Start Guide
- Collection of Container Queries articles

Using container size and style queries

Container queries enable you to apply styles to elements nested within a specific container based on the features of that container. The query returns true or false depending on whether the query condition is true for the container.

Container queries are similar to media queries. The `@media` at-rule enables applying styles to elements based on viewport size or other device characteristics. Similarly, the `@container` at-rule enables applying styles to elements based on a containing element's size or other features, rather than the viewport's. Container queries have the same syntax rules and logical operators as media queries.

```
@container <container-condition># {
  /* <stylesheet> */
}
```

There are five types of container queries:

Container size queries

Size queries enable applying styles to elements based on the current size of a containing element, including the orientation and aspect ratio. The containing elements need to be explicitly declared as *size query containers*.

Container style queries

Style queries enable applying styles to elements based on a containing element's style features, where any non-empty element can be a style query container. A style feature can be a CSS property, CSS custom property, or a valid CSS declaration. This allows you to apply styles to any container element's descendants based on its style features — such as whether it has a `display: inline` flex declaration set, or the value of a custom property.

Name-only container queries

Name-only container queries enable selectively applying styles to an element based on whether it has a specific `container-name` set.

Container scroll-state queries

Scroll-state queries allow you to selectively apply CSS rules to a container's descendants based on scroll-state conditions, such as whether the queried element is partially scrolled or whether the container is snapped to a scroll snap container. The containing elements need to be explicitly declared as *scroll-state query containers*.

Anchored container queries

Anchored container queries enable querying whether the container is anchor-positioned and has a position-try fallback option applied to it.

In this guide, we learn the basics of container queries by looking at:

1. container size queries,
2. naming containers to limit their scope, and
3. using the `style()` functional notation within the `@container` at rule's `<container-condition>` to create style queries with custom properties.

Container size queries

Container size queries are filtered by a size condition. The associated styles are applied to contained elements if the container element has been declared to be a container and the container condition is true for that element. An element's size container is the nearest ancestor with containment.

Elements are declared as *size query containers* by setting their `container-type` property (or the `container` shorthand) to `size` or `inline-size`.

```
@container (orientation: landscape) {
  /* styles applied to descendants of this size container */
}

.sizeContainer {
  container-type: size;
}
```

Declaring size query containers adds containment to them. This is a performance necessity — querying the size of every element in the DOM, all the time, would be bad for performance and user experience. Additionally, if a descendant style changed the size of the container element, an infinite loop could occur.

In a container size query, the `<container-condition>` includes one or more `<size-query>`s. Each size query includes a size feature name, a comparison operator, and a value. The size features that can be queried are limited to `width`, `height`, `inline-size`, `block-size`, `aspect-ratio`, and `orientation`. The boolean syntax and logic combining one or more `<size-query>`s is the same as for `@media` size feature queries.

```
form {
  container-type: inline-size;
}

@container (10em <= width <= 20em) {
  /* styles */
}
```

The `<container-condition>` in this example contains a single `<size-query>` — `(10em <= width <= 20em)`. In this case, all `<form>` elements are potential matches for any unnamed container query. The styles declared within our container query apply to the descendants of all forms between `10em` and `30em` wide, inclusive.

Naming containers

A `<container-condition>` can include an optional case-sensitive `container-name`. A container name makes the container condition more specific — it is evaluated only against elements that have that name set in the `container-name` property.

The `container-name` property specifies a list of query `<container-name>` values that can be used in `@container` rules to target specific query containers; these are case-sensitive `<ident>` values. Without a `<container-name>`, the query matches only the nearest container ancestor, and without a `<container-query>`, the query will match elements with the specified `container-name` set on them (see Name-only container queries).

```
@container [ <container-name>? <container-query>? ]!# {
  /* <stylesheet> */
}
```

Styles inside the named `@container` at rules will be applied only to matching elements inside containers with those names set, which satisfy the container queries.

```
@container card (orientation: landscape) {
  /* styles */
}

.todo-panel > li {
  container-type: inline-size;
  container-name: card;
}
```

In the above example, the styles within the container query block will apply to the descendants of all `<li>` elements with a width that is greater than their height. Note that other elements with `container-name: card` applied to them that match the size query will also have these styles applied to their elements' descendants.

```
@container wide (width >= 20em) {
  /* styles applied to descendants of wide .sizeContainer */
}

@container narrow (width < 20em) {
  /* styles applied to descendants of narrow .sizeContainer */
}

.sizeContainer {
  container-type: size;
  container-name: wide narrow;
}
```

In the above example, the element has two container names, `wide` and `narrow`. The descendants of any elements with `class="sizeContainer"` will get the styles from the `wide` or `narrow` query applied.

The default value `container-type: normal` prevents the container from being a size container, but it can still be a style container, and it can still be targeted by a name-only container query. The default value `container-name: none` states the container has no name, but it does not prevent the element from matching unnamed queries.

Container style queries

A *container style query* is a `@container` query that evaluates computed styles of the container element as defined in one or more `style()` functional notations. The boolean syntax and logic used to combine style features into a style query are the same as in CSS feature queries. The only difference is the function name — `style()` within a `<style-feature>` as opposed to `supports()` within a `<support-condition>`:

```
@container style(<style-feature>),
    not style(<style-feature>),
    style(<style-feature>) and style(<style-feature>),
    style(<style-feature>) or style(<style-feature>) {
  /* <stylesheet> */
}
```

The parameter of each `style()` function is a single **`<style-feature>`**. Per the CSS containment specification, a `<style-feature>` can be a valid CSS declaration, a CSS property, or a `<custom-property-name>`. The only style feature currently supported is custom properties, with or without a value. See the browser compatibility table for `@container`.

If the `<style-feature>` includes a value, the style query evaluates to true if the computed value of the custom property (or, in the future, the CSS declaration) passed as the `style()` argument is true for the container being queried. Otherwise, it resolves to false. A style feature without a value evaluates to true if the computed value is different from the initial value for the given property.

In the future, we'll be able to write style queries like so:

```
@container style(color: green) and style(background-color: transparent),
    not style(background-color: red),
    style(--themeBackground),
    style(--themeColor: blue) or style(--themeColor: purple),
    (width <= 100vw) and style(max-width: 600px) {
  /* <stylesheet> */
}
```

The `style()` functional notation is used to differentiate style queries from size queries. While not yet supported, we will eventually be able to query regular CSS declarations such as `max-width: 600px`. Querying `@container (max-width: 600px)` is a size query; containment with `container-type`, or the `container` shorthand, is needed. That query will return true if the container is 600px or less. That is different from querying `@container style(max-width: 600px)`, which is a style query; when supported, this query will return true if the container has a `max-width` value of `600px`.

Until style queries for regular CSS declarations and properties are supported, we are limited to including only custom properties as the `style()` parameter, with or without a value:

```
@container style(--themeBackground),
    style(--themeColor: blue) or style(--themeColor: purple) {
  /* <stylesheet> */
}
```

A few things to note that have already been mentioned but are important to remember:

- All elements can be style query containers; setting a `container-type` is not required. When descendant styles don't impact the computed styles of an ancestor, containment is not needed.
- A `<container-condition>` can include both style and size features. If including size features in your query, make sure your container elements have a `container-type` of `size` or `inline-size` set.
- If you don't want an element to be considered as a container, ever, give it a `container-name` that will not be used. Setting `container-name: none` removes any query names associated with a container; it does not prevent the element from being a style container.
- At the time of this writing (February 2024), container style queries only work with CSS custom property values in the `style()` query.

Now, let's dive in and take a look at the different `<style-feature>` types.

Style queries for custom properties

Style queries for custom properties allow you to query the custom properties, also called "CSS variables", of a parent element. They are included within a `<style-query>` just as you would include any regular CSS property within a feature query: either with or without a value.

Standalone custom property queries

The `<style-query>` parameter of the `style()` functional notation can include just a CSS variable name; a custom property with no value. When no value is included, the query will return false if the value is the same as the value of the `initial-value` descriptor within the `@property` at-rule, if there is one. The style query will return true and match all elements that have a custom property value that differs from the `initial-value` or for all elements that have a custom property of any value if the custom property was declared without being registered.

Unregistered custom properties

When CSS variables are introduced via a CSS custom property value assignment, valueless custom property queries always return true.

```
:root {
  --theme-color: rebeccapurple;
}

@container style(--theme-color) {
  /* <stylesheet> */
}
```

In this example, the container query matches the element on which the `--theme-color` property was declared and all of its descendants. As the CSS variable `--theme-color` was declared on the `:root`, the style query `style(--theme-color)` will be true for every element within that DOM node.

Registered properties

The behavior of registered custom properties is different. When explicitly defined with the `@property` CSS at-rule or via JavaScript with `CSS.registerProperty()`, the style query `style(--theme-color)` only returns true for elements if the element's computed value for `--theme-color` is different from the `initial-value` set in the original definition of that custom property.

```
@property --theme-color {
  initial-value: rebeccapurple;
  inherits: true;
}

:root {
  --theme-color: rebeccapurple;
}

main {
  --theme-color: blue;
}

@container style(--theme-color) {
  /* <stylesheet> */
}
```

In this example, the `:root` element does NOT match the style query because the value of the custom property is the same as the `initial-value` value. The custom property value for the element (and all the elements inheriting the value) is still `rebeccapurple`. Only elements that differ from the initial value, in this case, the `<main>` and its descendants that inherit that changed value, are a match.

Custom property with a value

If a style query includes a value for the custom property, the element's computed value for that property must be an exact match, with equivalent values only being a match if the custom property was defined with a `@property` at rule (or a `CSS.registerProperty()` method call) containing a `syntax` descriptor.

```
@container style(--accent-color: blue) {
  /* <stylesheet> */
}
```

This container style query matches any element that has `blue` as the computed value of the `--accent-color` custom property.

In this case, other color values equivalent to sRGB `blue` (such as the hexadecimal code `#0000ff`) will match only if the `--accent-color` property was defined as a color with `@property` or `CSS.registerProperty()`, for example:

```
@property --accent-color {
  syntax: "<color>";
  inherits: true;
  initial-value: #0000ff;
}
```

In this case, if the value of `--accent-color` were set to `blue`, `#00f`, `#0000ff`, `rgb(0 0 255 / 1)`, or `rgb(0% 0% 100%)` it would return true for `@container style(--accent-color: blue)`.

Example

In this example, we have a `<fieldset>` with four radio buttons. The fourth option includes a text `<input>` for entering a custom color.

```
<fieldset>
  <legend>Change the value of <code>--theme</code></legend>
  <ol>
    <li>
      <input type="radio" name="selection" value="red" id="red" />
      <label for="red">--theme: red;</label>
    </li>
    <li>
      <input type="radio" name="selection" value="green" id="green" />
      <label for="green">--theme: green</label>
    </li>
    <li>
      <input type="radio" name="selection" value="blue" id="blue" />
      <label for="blue">--theme: blue</label>
    </li>
    <li>
      <input type="radio" name="selection" value="currentColor" id="other" />
      <label for="other">Other</label>
      <label for="color">color:</label>
      <input name="selection-value" value="currentColor" id="color" />
    </li>
  </ol>
</fieldset>
<output>I change colors</output>
```

JavaScript updates the value of the CSS `--theme` variable on the `<body>` element, which is an ancestor of the `<fieldset>` and `<output>` elements, whenever a radio button is selected. When the text `<input>` is updated, the `value` of the `other` radio button is updated only if the `other` radio button is checked, which in turn updates the value of `--theme`.

```
const radios = document.querySelectorAll('input[name="selection"]');
const body = document.querySelector("body");
const other = document.getElementById("other");
const color = document.getElementById("color");

for (const radio of radios) {
  radio.addEventListener("change", (e) => {
    body.style.setProperty("--theme", e.target.value);
  });
}
color.addEventListener("input", (e) => {
  other.style.setProperty("value", e.target.value);
  if (other.checked) {
    body.style.setProperty("--theme", e.target.value);
  }
});
```

We use the `@property` at-rule to define a CSS variable `--theme` to be a `<color>` value and set the `initial-value` to `red`, ensuring equivalent colors are a match regardless of what syntax is used (for example, `red` is equal to `rgb(255 0 0)`, `#ff0000`, and `#f00`).

```
@property --theme {
  syntax: "<color>";
  inherits: true;
  initial-value: red;
}
```

```
output {
  padding: 3px 5px;
  margin-top: 5px;
}
```

The first style feature query is a custom property with no value. This query type returns true when the computed value for the custom property value is different from the `initial-value` for that property. In this case, it will be true when the value of `--theme` is any value other than any syntax equivalent value of `red` (such as `#ff0000`). When true, the `<output>` will have a 5px dotted outline. The outline color is the current value of `--theme`. The default text `color` is gray.

```
@container style(--theme) {
  output {
    outline: 5px dotted var(--theme);
    color: #777777;
  }
}
```

The second and third style queries include values for the custom property. These will match if the container's `--theme` value is an equivalent color to the value listed, even if that value is the same as the `initial-value`. The first query matches elements whose `--theme` value is equivalent to `red`, `blue`, or `green`. When it is, the `color` will be the color current value of `--theme` (in the case of `blue` and `green`, overriding the gray set in the first style query).

The second style query states that when `--theme` is equivalent to `red`, the `<output>`'s contents will also be bold. We did this to better demonstrate that the container query is a match.

```
@container style(--theme: green) or style(--theme: blue) or style(--theme: red) {
  output {
    color: var(--theme);
  }
}

@container style(--theme: red) {
  output {
    font-weight: bold;
  }
}
```

Try entering different color values into the text box. You may notice that values that are sRGB equivalents of `red` will make the `<output>` red — as it matches `style(--theme: red)` — while removing the outline, because `style(--theme)` returns false if the element's value for `--theme` is the same as the initial value for `--theme` defined by the `@property` at-rule. Any non-red sRGB valid color value, including `currentColor` or `hsl(180 100% 50%)`, etc., makes the first style query return true; they are values that are different from the `initial-value`.

Because we set `syntax: "<color>";`, the CSS variable can only be assigned valid `<color>` values. Valid values for the `color` property that aren't value `<color>` values, such as `unset` or `inherit`, are invalid for this custom property, and will be ignored.

If you enter `unset` or `gibberish`, the JavaScript updates the `style` on the `<body>` to `--theme: unset` or `--theme: gibberish`. Neither of these are colors. Both are invalid and ignored. This means the initial value is inherited and unchanged, with `style(--theme)` returning false and `style(--theme: red)` returning true.

> **Note:** When declaring custom properties, consider using `@property` with the `syntax` descriptor so the browser can properly compare computed values.

Plain versus range syntax in style queries

When a `<style-feature>` includes a value, you can express the comparison in two different ways. They look similar but behave very differently, and choosing the right one matters.

The **plain syntax** uses a colon, the same syntax used in a CSS declaration:

```
@container style(--n: 3) {
  /* … */
}
```

This form is true if the computed value of the property matches the value on the right. For an unregistered custom property, the computed value is the property's value as written: the browser doesn't evaluate `calc()` or other expressions inside it. The match is essentially a comparison of the two values' tokens. To match equivalent values (such as `blue` and `#0000ff`), register the custom property with `@property` and a `syntax` descriptor.

The **range syntax** uses a comparison operator (`=`, `<`, `<=`, `>`, or `>=`):

```
@container style(--n = 3) {
  /* … */
}
```

To evaluate this form, the browser:

1. Resolves each side (custom property names are looked up as if used with `var()`).
2. Parses each side as one of `<number>`, `<percentage>`, `<length>`, `<angle>`, `<time>`, `<frequency>`, or `<resolution>`. If either side can't be parsed as one of those types, the query is false.
3. If both sides have the same type, computes each side (evaluating any `calc()` expressions) and performs the numeric comparison. Otherwise, the query is false.

Consider the following example, where `--n` is set to a `calc()` expression:

```
.box {
  --n: calc(6/2);
}

/* Evaluates to FALSE: */
/* the computed value of --n is the string `calc(6/2)`, which is */
/* not equal to the string `3`. */
@container style(--n: 3) {
  /* … */
}

/* Evaluates to TRUE: */
/* both sides are parsed as <integer>, calc(6/2) is computed to 3, */
/* and 3 = 3. */
@container style(--n = 3) {
  /* … */
}
```

The range syntax also supports a three-value form for testing whether a value falls within an interval. Both comparators must point the same way:

```
@container style(0 < --n < 10) {
  /* true when --n is greater than 0 and less than 10 */
}

@container style(100px > --width > 50px) {
  /* true when --width is less than 100px and greater than 50px */
}
```

The range syntax is also more flexible in how each side is written. Either side can be a custom property name, a `var()` reference, a literal value, or a `calc()` expression, and the operands can appear in any order. The following are all valid:

```
@container style(3 = --n) {
  /* … */
}
@container style(var(--n) = 3) {
  /* … */
}
@container style(calc(6/2) = var(--n)) {
  /* … */
}
```

The plain syntax is more restrictive: the left-hand side must be the custom property name (without `var()`), and the value goes on the right. The following are all **invalid**:

```
@container style(var(--n): 3) {
  /* … */
}
@container style(3: --n) {
  /* … */
}
```

Because the range syntax requires both sides to parse as one of the listed numeric types, it can't be used to compare keyword-like values. For example, given `--s: new`, the query `style(--s = new)` is false (because `new` isn't a number, length, etc.), while `style(--s: new)` is true.

In short:

- Use **`style(--variable: value)`** for keyword-like or string-like matching, such as `style(--stock: low)` or `style(--theme: dark)`.
- Use **`style(--variable = value)`** (or `<`, `<=`, `>`, `>=`) for numeric comparisons, such as `style(--columns >= 3)` or `style(--gap = 1rem)`.

Nested queries

Container queries can be nested within other container queries. The styles defined inside multiple nested container queries are applied when all of the wrapping container queries are true.

```
@container style(--theme: red) {
  output {
    outline: 1px dotted;
  }
  @container style(--theme: purple) {
    output {
      outline: 5px dotted;
    }
  }
}
```

In this case, the `<output>` will have a 5px dotted border if it's nested in a container where `--theme: purple` is set, and that container is nested within a container whose `--theme` value is `red`.

Style query CSS declarations and properties

Not yet supported in any browser, the `style()` functional notation can include regular CSS declarations including CSS properties and property value pairs.

```
@container style(font-weight: bold) {
  b,
  strong {
    background: yellow;
  }
}
```

When supported, this basic example will make the background color of any `<b>` and `<strong>` elements yellow when the parent is already `bold`.

The matching is done against the computed value of the parent container; if the parent's computed `font-weight` is `bold` (not `bolder` or `900`), there is a match. Just as with custom property container style queries, we did not have to define any elements as style containers as all elements are style containers by default. As long as an element doesn't have a `container-name` set, if it has `font-weight: bold` set or inherited, it

will match.

Style features that query a shorthand property will be true if the computed values match for each of its longhand properties, and false otherwise. For example, `@container style(border: 2px solid red)` will resolve to true if all 12 longhand properties (`border-bottom-style`, etc.) that make up that shorthand are set to the same equivalent values.

The global CSS values `revert` and `revert-layer` are invalid as values in a `<style-feature>` and cause the container style query to be false.

Do not apply the styles you are querying in the style query to the element you are styling with that query as this may cause an infinite loop.

It is expected that style queries will also accept properties in a boolean context. The style query will return false if the value of the property is the initial value for that property (if it has not been changed), and true otherwise.

```
@container style(font-weight) {
}
```

The above example will return true for any element that has a value for `font-weight` that differs from its initial value. User-agent stylesheets set `font-weight: bold` for heading and `<th>` elements, for example. Some browsers set `<strong>` and `<b>` to `bold`, others to `bolder`. `<optgroup>` also sometimes has a `font-weight` other than `normal` set by the user agent. As long as the element's `font-weight` is not the default value for that user-agent, the style query will return true.

These features are not yet supported in any browser.

See also

- Media queries
- CSS `@container` at-rule
- CSS `contain` property
- CSS `container` shorthand property
- CSS `container-name` property
- Using container scroll-state queries
- Understanding `aspect-ratio`
- Getting Started with Style Queries (2022)
- Style queries via una.im (2022)

Using CSS containment

CSS containment improves the performance of web pages by allowing the browser to isolate a subtree of the page from the rest of the page. If the browser knows that a part of the page is independent from the rest of the content, rendering can be optimized and performance improved.

The `contain` and `content-visibility` properties enable developers to inform user agents whether or not an element should render its contents at all, and whether it should render its contents when it is offscreen. The user agent then applies containment to elements when appropriate, potentially deferring layout and rendering until needed.

This guide describes the basic aims of CSS containment and how to leverage `contain` and `content-visibility` for a better user experience.

Basic example

Web pages often contain multiple sections which are, logically, independent of each other. CSS containment enables them to be treated truly independently from each other when it comes to rendering.

For example, blogs usually contain several articles, each containing a headline and content, as in the markup below.

```
<h1>My blog</h1>
<article>
  <h2>Heading of a nice article</h2>
  <p>Content here.</p>
</article>
<article>
  <h2>Another heading of another article</h2>
  <p>More content here.</p>
</article>
```

With CSS, we apply the `contain` property with a value of `content` to each article. The `content` value is shorthand for `contain: layout paint style`:

```
article {
  contain: content;
}
```

Logically, each article is independent of the other articles on the page. This information is something that is usually known, and likely quite obvious, to the web developer creating the page. However, browsers don't know the intent of your content and cannot assume that an article or other section of content will be entirely self-contained.

This property provides a way of explaining this to the browser and giving it explicit permission to make performance optimizations. It tells the browser that the internal layout of the element is completely separate from the rest of the page, and that everything about the element is painted inside its bounds. Nothing can visibly overflow.

By setting `contain: content` on each `<article>` we have indicated this; we have told the browser that each article is independent. The browser can then use this information to make decisions about how to render each `<article>` of content. For example, it might not render articles that are outside the viewable area.

When additional articles are appended at the end of the page, the browser does not need to recalculate layout or repaint the preceding content; it also doesn't need to touch any area outside of the containing element's subtree. If box model properties are dependent, however, the browser will need to recalculate layout and repaint. For example, if the `<article>` is styled such that its size depends on its contents (e.g., with `height: auto`), then the browser will need to account for its size changing.

Key concepts and terminology

<code>contain</code> values

There are four types of containment: layout, paint, size, and style. Use the `contain` property to specify the type or types you want to apply to an element by including any combination of these types.

Layout containment

```
article {
  contain: layout;
}
```

Layout is normally scoped to the entire document, which means that if you move one element the entire document needs to be treated as if things could have moved anywhere. By using `contain: layout` you can tell the browser it only needs to check this element — everything inside the element is scoped to that element and does not affect the rest of the page, with the containing box establishing an independent formatting context.

In addition:

- `float` layout will be performed independently inside the specified element.

- Margins won't collapse across a layout containment boundary.
- The layout container is a containing block for `absolute-` and `fixed`-positioned descendants.
- The containing box creates a stacking context, therefore `z-index` can be used.

> **Note:** The `style` and `layout` values of `contain` are automatically applied when using the `container-type` and `container-name` properties.

Paint containment

```
article {
  contain: paint;
}
```

Paint containment essentially clips the box to the padding edge of the principal box. There can be no visible overflow. The same additional notes are true for `paint` containment as `layout` containment (see above).

Another advantage is that if the element with containment applied is offscreen, the browser does not need to paint its child elements — these are also offscreen as they are contained completely by that box.

Size containment

```
article {
  contain: size;
}
```

Size containment does not offer much in the way of performance optimizations when used on its own. However, size containment means that the size of the size-contained element's children cannot affect the size of the element itself — its size is computed as if it had no children.

If you set `contain: size` on an element, you need to specify the size of the element using `contain-intrinsic-size`, or the longhand properties `contain-intrinsic-width` and `contain-intrinsic-height`, in that order. If no size is set, the element risks being zero-sized in most cases.

```
article {
  contain: size;
  contain-intrinsic-size: 100vw auto none;
}
```

Style containment

```
article {
  contain: style;
}
```

Despite the name, style containment does not provide scoped styles such as you would get with the Shadow DOM or `@scope`. The main use case for the `style` value is to prevent situations where a CSS counter could be changed in an element, which could then affect the rest of the tree.

Using `contain: style` ensures the `counter-increment` and `counter-set` properties create new counters scoped to that subtree only.

You can include more than one containment type by including multiple space-separated values, such as `contain: layout paint` or by using one of the two special values.

Special values

There are two special values of `contain` that are shorthand for the first three or all four of the containment types:

- `content`
- `strict`

We encountered the first in the example above. Using `contain: content` turns on `layout`, `paint`, and `style` containment. As it omits `size`, it is a safe value to apply widely.

The `contain: strict` declaration, which behaves the same as the declaration `contain: size layout paint style` (which includes four space-separated values), provides the most containment. It is riskier to use as it applies `size` containment; the risk exists that a box could end up zero-sized due to a reliance on the size of its children.

To remove this risk, always set a size when using `strict`:

```
article {
  contain: strict;
  contain-intrinsic-size: 80vw auto none;
}
```

The above is the same as:

```
article {
  contain: size layout paint style;
  contain-intrinsic-size: 80vw auto none;
}
```

<code>content-visibility</code>

When you have a lot of content that would benefit from heavy containment that will often be offscreen — for example if all your blog posts are viewable on the blog home pages as an infinitely scrollable blog — `content-visibility: auto` can be used to apply all of the containments at once.

The `content-visibility` property controls whether or not an element renders its contents at all, along with forcing a strong set of containments, allowing user agents to potentially omit large swathes of layout and rendering work until it becomes needed. It enables the user agent to skip an element's rendering work (including layout and painting) until it is needed — which makes the initial page load much faster.

Its possible values are:

- `visible`: The default behavior — an element's contents are laid out and rendered as normal.
- `hidden`: The element skips its contents. The skipped contents will not be accessible to user agent features such as find-in-page, tab-order navigation, etc., nor be selectable or focusable.
- `auto`: The element turns on layout containment, style containment, and paint containment, as if `contain: content` was set. If the element is not relevant to the user, it also skips its contents. Unlike `hidden`, the skipped content is still available for user interactions, remaining focusable, selectable, in regular tab order, and available to in-content search.

Relevant to the user

User-agents have a concept of content being relevant to the user. An element becomes "relevant to the user" if any of the following are true:

- The element appears in the viewport, or a user-agent-defined margin around the viewport (50% of the viewport dimensions, to give the app time to prepare for when the element visibility changes).
- The element or its contents receive focus.
- The element or its contents are selected, for example by dragging over the text with the mouse cursor or by some other highlight operation.

- The element or its contents are placed in the top layer.

When `content-visibility: auto` is set, and the browser determines that content is relevant to the user, the browser will render that content.

Skips its contents

When you set `content-visibility: hidden` on an element, you are telling the browser that it is not relevant to the user, and therefore its contents should be skipped and not rendered. This helps to improve performance.

The browser will also skip an element's contents when `content-visibility: auto` is set on it, and the browser determines that its content is *not* relevant to the user.

When an element skips its contents:

- It has layout, style, paint, and size containment turned on.
- Its contents are not painted, as if `visibility: hidden` was set on it.
- Its contents do not receive pointer events, as if `pointer-events: none` was set on it.

This happens in both the cases mentioned above, but with `content-visibility: auto` the content can be searched, receive focus, and otherwise move from not relevant to relevant. This is not the case for `content-visibility: hidden`.

> **Note:** To animate the transition from `content-visibility: hidden` to a visible value, you will need to set `transition-behavior: allow-discrete` and `@starting-style` styles. See transitioning `display` and `content-visibility` to learn more.

See also

- CSS containment module
- Learn: CSS performance optimization
- CSS container queries
- An Introduction to CSS Containment via Igalia.com (2019)
- The `contentvisibilityautostatechange` event

CSS counter styles

The **CSS counter styles** module lets you define your own counter styles to manage the appearance of markers in lists and counters in generated content. It also enables you to extend native browser list styles with your own customizations.

While we think of counters as numbers, they are actually strings with components that can be incremented. The counter styles module defines the `@counter-style` rule with ten descriptors, which enable developers to precisely define how counters are converted into strings. This module enables defining which characters to use for the counter bullets, the prefix to put before the counter and postfix that comes after, along with how to handle negative values. The descriptors can also set a range to limit the values a counter style can handle, while also providing fallback styles for when the counter value falls outside the defined range or otherwise can't render the counter value. The module also enables defining how the counter is read out loud by speech synthesizers.

Reference

Properties

No properties are defined in this module

Functions

- `symbols()`

Data types

- `<counter-style-name>`
- `<symbol>`
- `<symbols-type>`

At-rules and descriptors

- `@counter-style`
  - `system`
  - `symbols`
  - `additive-symbols`
  - `negative`
  - `prefix`
  - `suffix`
  - `range`
  - `pad`
  - `speak-as`
  - `fallback`

Interfaces

- `CSSCounterStyleRule` interface

Guides

Using CSS counters

Describes how to use counters to number any HTML element or to perform complex counting.

Related concepts

CSS lists and counters module:

- `counter-increment` property
- `counter-reset` property
- `counter-set` property
- `list-style-type` property
- `list-style` shorthand property
- `counter()` function
- `counters()` function

CSS pseudo-elements module:

- `::after` pseudo-element
- `::before` pseudo-element
- `::marker` pseudo-element

CSS generated content module:

- `content` property

See also

- CSS lists and counters module
- CSS pseudo-elements module
- CSS generated content module
- Ready-made counter styles via W3C (2023)

Using CSS counters

CSS counters let you adjust the appearance of content based on its location in a document. For example, you can use counters to automatically number the headings on a webpage or to change the numbering on ordered lists.

Counters are, in essence, variables maintained by CSS whose values may be incremented or decremented by CSS rules that track how many times they're used. The following things affect the counter values on an element:

1. Counters are inherited from the parent element or received from a previous sibling.
2. New counters are instantiated using `counter-reset` property.
3. Counters are incremented using `counter-increment` property.
4. Counters are directly set to a value using the `counter-set` property.

You can define your own named counters, and you can also manipulate the `list-item` counter that is created by default for all ordered lists.

Using counters

To use a counter, it must first be initialized to a value with the `counter-reset` property. The counter's value can be increased or decreased using the `counter-increment` property and can be directly set to a specific value using the `counter-set` property. The current value of a counter is displayed using the `counter()` or `counters()` function, typically within a pseudo-element `content` property.

Counters can only be set, reset, or incremented in elements that generate boxes. For example, if an element is set to `display: none`, then any counter operation on that element is ignored.

The properties of counters can be scoped to specific elements using style containment, which is described in more detail in the `contain` property.

Manipulating a counter's value

To use a CSS counter, it must first be initialized to a value with the `counter-reset` property. The property can also be used to change the counter value to any specific number.

The following declaration initializes a counter named `section` to the default value (`0`):

```
counter-reset: section;
```

You can also initialize multiple counters, optionally specifying an initial value for each. The following declaration initializes the `section` and `topic` counters to the default value, and the `page` counter to `3`:

```
counter-reset: section page 3 topic;
```

Once initialized, a counter's value can be increased or decreased using `counter-increment`. For example, the following declaration increments the value of the `section` counter by `1` on every `<h3>` element:

```
h3 {
  counter-increment: section;
}
```

You can specify the increment or decrement amount after the counter name. It can be a positive or negative number, but defaults to `1` if no integer is provided.

Apart from being incremented or decremented, counters can also be explicitly set to a value using the `counter-set` property.

```
.done {
  counter-set: section 20;
}
```

The counter's name must not be `none`, `inherit`, or `initial`; otherwise the declaration is ignored.

Displaying a counter

The value of a counter can be displayed using either the `counter()` or `counters()` function in a `content` property.

For example, the following declaration uses `counter()` to prefix each `h3` heading with the text `Section <number>:`, where `<number>` is the value of the count in decimal (the default display style):

This creates a counter named `section` on the `<body>` element and sets its initial value to `0`. The counter increments by `1` on every `<h3>` element. Before the content of each `<h3>` heading, `Section <n>:` is displayed, where `<n>` is the default decimal value of the `section` counter.

```
body {
  counter-reset: section;
}

h3 {
  counter-increment: section;
}

h3::before {
  content: "Section " counter(section) ": ";
}
```

Comparing <code>counter()</code> and <code>counters()</code>

The `counter()` function is used when the numbering of nesting levels does not include the context of parent levels. For example, here each nested level restarts from one:

```
1 One
  1 Nested one
  2 Nested two
2 Two
  1 Nested one
```

```
  2 Nested two
  3 Nested three
3 Three
```

The `counters()` function is used when the count for nested levels must include the count from parent levels. For example, you might use this to lay out sections as shown:

```
1 One
  1.1 Nested one
  1.2 Nested two
2 Two
  2.1 Nested one
  2.2 Nested two
  2.3 Nested three
3 Three
```

The `counter()` function has two forms: `counter(<counter-name>)` and `counter(<counter-name>, <counter-style>)`. The generated text is the value of the innermost counter of the given name in scope at the pseudo-element.

The `counters()` function also has two forms: `counters(<counter-name>, <separator>)` and `counters(<counter-name>, <separator>, <counter-style>)`. The generated text is the value of all counters with the given name in scope at the given pseudo-element, from outermost to innermost, separated by the specified string (`<separator>`).

The counter is rendered in the specified `<counter-style>` for both methods (`decimal` by default). You can use any of the `list-style-type` values or your own custom styles.

The use of the `counter()` and `counters()` functions is demonstrated later in the Basic example and Nested counters sections, respectively.

Reversed counters

A reversed counter is one that is intended to count down (decrement) rather than up (increment). Reversed counters are created using the `reversed()` function notation when naming the counter in `counter-reset`.

Reversed counters have a default initial value equal to the number of elements (unlike normal counters, which have a default value of 0). This enables implementing a counter that counts from the number of elements down to one.

For example, to create a reversed counter named `section` with a default initial value, you would use the following syntax:

```
counter-reset: reversed(section);
```

You can, of course, specify any initial value that you like.

The counter value is decreased by specifying a negative value for `counter-increment`.

> **Note:** You can also use `counter-increment` to decrement a non-reversed counter. The main benefit of using a reversed counter is the default initial value, and that the `list-item` counter automatically decrements reversed counters.

Counter inheritance and propagation

Each element or pseudo-element has a set of counters in the scope of that element. Initial counters in the set are received from the element's parent and the preceding sibling. The counter values are received from the last descendant of the previous sibling, the last sibling, or the parent.

When an element declares a counter, the counter is nested inside the counter with the same name received from the parent. If the parent doesn't have a counter with the same name, then the counter is added to the element's counters set as it is. A counter with the same name received from the previous sibling is removed from the element's set of counters.

The `counter()` function retrieves the innermost counter with the provided name. The `counters()` function retrieves the entire counter tree with the given name.

The following example demonstrates an inherited counter named `primary` and a sibling counter named `secondary`. All the `<div>` elements display their counters using the `counters()` function. Note that all the counters have been created using the `counter-reset` property, and none of the counters have been incremented.

```
<section>
  counter-reset: primary 3
  <div>A</div>
  <div>B</div>
  <div>C</div>
  <div class="same-primary-name">D</div>
  <span> counter-reset: primary 6</span>
  <div>E</div>
  <div class="new-secondary-name">F</div>
  <span> counter-reset: secondary 5</span>
  <div>G</div>
  <div>H</div>
  <div class="same-secondary-name">I </div>
  <span> counter-reset: secondary 10</span>
  <div>J </div>
  <div>K</div>
  <section></section>
</section>
```

```
.same-primary-name,
.new-secondary-name,
.same-secondary-name {
  display: inline-block;
}

@counter-style style {
  system: numeric;
  symbols: "" "1" "2" "3" "4" "5" "6" "7" "8" "9" "10";
}
```

We create the `primary` counter on divs' parent, then create a new `primary` counter on `D`. We create the `secondary` counter on `F`, then override the sibling `secondary` counter on `I`.

```
section {
  counter-reset: primary 3;
}

div::after {
  content: " ('primary' counters: " counters(primary, "-", style)
    ", 'secondary' counters: " counters(secondary, "-", style) ")";
  color: blue;
```

```
}

.same-primary-name {
  counter-reset: primary 6;
}

.new-secondary-name {
  counter-reset: secondary 5;
}

.same-secondary-name {
  counter-reset: secondary 10;
}
```

The section element initializes a counter named `primary` with value `3`, and all the child `<div>`s receive the inherited `primary` counter. The element 'D' creates a new `primary` (value `6`) counter which gets nested in the counter received from the parent, so the element has two counters named `primary` with values `3` and `6`.

The element 'F' creates the `secondary` (value `5`) counter for the first time, and it passes the counter to the next sibling 'G'. The element 'G' passes the counter to the next element 'H' and so on. Next, the element 'I' creates a new counter with the same name `secondary` (value `10`), but it drops the `secondary` (value `5`) counter received from the previous sibling 'H' and passes its own counter to 'J'.

Difference between counter-set and counter-reset

The `counter-set` property updates an existing counter. If no counter with that name exists, then a new counter is instantiated. The `counter-reset` property *always* creates a new counter.

In the following example, we have two sub-lists inside a parent list. Each list item has been numbered using a counter named 'item'. The first sub-list uses `counter-set` property and the second sub-list uses `counter-reset` property to change the 'item' counter.

```
<ul class="parent">
  <li>A</li>
  <li>B</li>
  <li>
    C (the counter updated using `counter-set`)
    <ul class="sub-list-one">
      <li>sub-A</li>
      <li>sub-B</li>
    </ul>
  </li>
  <li>D</li>
  <li>
    E (a new counter created using `counter-reset`)
    <ul class="sub-list-two">
      <li>sub-A</li>
      <li>sub-B</li>
      <li>sub-C</li>
    </ul>
  </li>
  <li>F</li>
  <li>G</li>
</ul>
```

```
ul {
  list-style: none;
}
```

We create a new counter on `parent` and increment it on each list item, then show the numbers on the list items using generated content. We update the existing counter value with `counter-set`, then reset it with `counter-reset` on `.sub-list-one` and `.sub-list-two`, respectively.

```
.parent {
  counter-reset: item 0;
}

li {
  counter-increment: item;
}

li::before {
  content: counter(item) " ";
}

.sub-list-one {
  counter-set: item 10;
}

.sub-list-two {
  counter-reset: item 0;
}
```

Notice how the first sub-list starts at `11`, and its numbering continues in the parent list. This is because the `counter-set` property updates the same `item` counter declared on `.parent`. The second sub-list starts at `1` and the parent list items after it don't carry forward the numbering. This is because the `counter-reset` property creates a new counter with the same name, so the parent list items use the original counter.

List item counters

Ordered lists, as created using `<ol>` elements, implicitly have a counter named `list-item`.

Like other counters, this has a default initial value of 0 for upward counters and "number of items" for reversed counters. Unlike author-created counters, `list-item` *automatically* increments or decrements by one for each list element, depending on whether or not the counter is reversed.

You can use the `list-item` counter to manipulate the default behavior of ordered lists with CSS. For example, you can change the default initial value, or use `counter-increment` to change the way in which the list items increment or decrement.

Examples

Basic example

This example adds "Section [the value of the counter]:" to the beginning of each heading.

HTML

We include three `<h3>` elements.

```
<h3>Introduction</h3>
<h3>Body</h3>
<h3>Conclusion</h3>
```

CSS

We create a counter named 'section', letting its initial value default to `0`. We increment the value of `section` counter by `1` at each `<h3>` and display the word "Section", the value of `section` counter, and a colon before the content of each `<h3>`.

```
body {
  counter-reset: section;
}

h3::before {
  counter-increment: section;
}
h3::before {
  content: "Section " counter(section) ": ";
}
```

Result

Reversed counter

This example uses the same HTML as the previous example but uses a reversed counter, numbering the elements from 3 to 1.

```
<h3>Introduction</h3>
<h3>Body</h3>
<h3>Conclusion</h3>
```

CSS

We use the `reversed()` function to create a counter named 'section', letting the initial value default so that when we decrement the value of `section` counter by `1` on each `<h3>`, the last element in the set will have the counter set to `1`. We use generated content to display the word 'Section ' and the value of the counter before the content of each `<h3>`.

```
body {
  counter-reset: reversed(section);
}

h3 {
  counter-increment: section -1;
}
h3::before {
  content: "Section " counter(section) ": ";
}
```

```
@supports not (counter-reset: reversed(section)) {
  body::before {
    content: "Your browser doesn't support the reversed() function.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

A more elaborate example

A counter need not necessarily be shown every time it is incremented. This example counts all links but shows the counter as generated content only when a link has no text.

HTML

We include three paragraphs, each with a link, but the first and last links contain no text.

```
<p>See <a href="https://www.mozilla.org/" aria-label="Mozilla"></a></p>
<p>Do not forget to <a href="contact-me.html">leave a message</a>!</p>
<p>See also <a href="https://developer.mozilla.org/" aria-label="MDN"></a></p>
```

CSS

We create and initialize the counter on `:root`, increment it on every link, and generate content only when the link is `:empty`.

```
:root {
  counter-reset: link;
}

a[href] {
  counter-increment: link;
}

a[href]:empty::after {
  content: "[" counter(link) "]";
}
```

Result

Nested counters

A CSS counter can be especially useful for making outlined lists, because a new instance of the counter is automatically created in child elements. Using the `counters()` function, separating text can be inserted between different levels of nested counters.

CSS

We create a new instance of the section counter with each `<ol>` element, incrementing only the current instance of the section counter on each `<li>`. Using the `counters()` function, we combine the values of all instances of the section counter, separated by a period.

```
ol {
  counter-reset: section;
  list-style-type: none;
}

li {
  counter-increment: section;
}

li::before {
  content: counters(section, ".") " ";
}
```

HTML

```
<ol>
  <li>item</li>          <!-- 1     -->
  <li>item               <!-- 2     -->
    <ol>
      <li>item</li>      <!-- 2.1   -->
      <li>item</li>      <!-- 2.2   -->
      <li>item           <!-- 2.3   -->
        <ol>
          <li>item</li>  <!-- 2.3.1 -->
          <li>item</li>  <!-- 2.3.2 -->
        </ol>
        <ol>
          <li>item</li>  <!-- 2.3.1 -->
          <li>item</li>  <!-- 2.3.2 -->
          <li>item</li>  <!-- 2.3.3 -->
        </ol>
      </li>
      <li>item</li>      <!-- 2.4   -->
    </ol>
  </li>
  <li>item</li>          <!-- 3     -->
  <li>item</li>          <!-- 4     -->
</ol>
<ol>
  <li>item</li>          <!-- 1     -->
  <li>item</li>          <!-- 2     -->
</ol>
```

Result

See also

- `contain`
- `counter-reset`
- `counter-set`
- `counter-increment`
- `@counter-style`
- CSS counter styles module
- CSS lists and counters module

CSSOM view

The **CSSOM view** module lets you manipulate the visual view of a document, including getting the position of element layout boxes, obtaining the width or height of the viewport through script, and also scrolling an element.

Reference

Events

- `Window` events:
  - `resize`
- `VisualViewport` events:
  - `resize`
  - `scroll`
  - `scrollend`
- `Document` events
  - `scroll`
  - `scrollend`
- `Element` events
  - `scroll`
  - `scrollend`
- `MediaQueryList` events
  - `change`

Glossary terms

- Viewport
- Layout viewport
- Visual viewport

API

For the JavaScript API defined by this module, see the CSSOM view API documentation.

Guides

Coordinate systems

The coordinate systems used to specify a position in a display context such as a window on a monitor, a viewport on a mobile device, or a position on a sheet of paper when printing.

Viewport concepts

The concept of the viewport — what it is, its impact in terms of CSS, SVG, and mobile devices — and the difference between the visual viewport and the layout viewport.

Related concepts

- `zoom`
- `<meta>`

Glossary terms

- CSS object model (CSSOM)
- CSS pixel
- Scroll container

See also

- CSS Object Model (CSSOM) API
- CSSOM view API
- CSS viewport module
- CSS overflow module
- CSS overscroll behavior module
- CSS scroll snap module

Viewport concepts

This article explains the concept of the viewport — what it is and its impact in terms of CSS, SVG, and mobile devices. This article defines the initial viewport and actual viewport, and differentiates between the visual viewport and the layout viewport.

What is a viewport?

A **viewport** is a user agent feature used to establish the initial containing block for continuous media.

The generic *viewport* term generally refers to the area in computer graphics being currently viewed. In web browser terms, that is generally the same as the browser window, excluding the UI, menu bar, etc. That is the part of the document you are viewing.

As a document loads, the viewport goes through two stages:

Initial viewport

> The *initial viewport* refers to the window or viewing area of the UA before user agent styles, HTML `<meta>` tags, or CSS styles have overridden its size. The initial viewport size is based on the size of the window or viewing area, and not the content. The size of a full-screen user agent's initial viewport will differ between orientations and devices, but will always be the same for the same device in the same orientation.

Actual viewport

> The *actual viewport* is the viewport you get after processing the viewport `<meta>` tag. Content designed for large viewports may exhibit a variety of bugs when viewed in smaller viewports, including unintended wrapping, clipped content, and incorrectly sized scroll containers. The viewport meta tag provides hints about the initial size of the viewport. The actual viewport is the size defined by its `content` attribute. If this tag is omitted, some mobile browsers render content using a fixed initial containing block width, typically `980px`. They set the width of the actual viewport to this value, then scale the content down to fit it, making the CSS pixel size smaller than an actual pixel.

Documents, like this article, may be very long. Your viewport is everything that is currently visible; notably, the "what is a viewport" section, and perhaps some of the navigation menu. The size of the viewport depends on the size of the screen, whether the browser is in fullscreen mode or not, and whether or not the browser is zoomed in. Content outside the viewport, such as the *See Also* section in this document, is likely to not be visible onscreen until scrolled into view.

- On larger monitors where applications aren't necessarily full screen, the viewport is the size of the browser window.
- On most mobile devices and when the browser is in fullscreen mode, the viewport is the entire screen.
- In fullscreen mode, the viewport is the device screen, the window is the browser window, which can be as big as the viewport or smaller, and the document is the website, which can be much taller or wider than the viewport.

For paged media, the initial containing block is based on the page area. The page area can be set through `@page` rules.

To recap, the viewport is basically the part of the document that is currently visible.

Viewport sizes are mutable

The width of the viewport is not always the width of the window. If you query the width or height of the window and document in Chrome or Firefox, you may get:

```
document.documentElement.clientWidth; /* 1200 */
window.innerWidth; /* 1200 */
window.outerWidth; /* 1200 */
```

```
document.documentElement.clientHeight; /* 800 */
window.innerHeight; /* 800 */
window.outerHeight; /* 900 */
```

There are several DOM properties that can help you query viewport size, and other similar lengths:

- The document element's `Element.clientWidth` is the inner width of a document in CSS pixels, including padding (but not borders, margins, or vertical scrollbars, if present). **This is the viewport width**.
- The `Window.innerWidth` is the width, in CSS pixels, of the browser window viewport including, if rendered, the vertical scrollbar.
- The `Window.outerWidth` is the width of the outside of the browser window including all the window chrome.

In an experiment with these, the `innerWidth` and `outerWidth` was seen to be the same, but the `outerHeight` was 100px taller than the `innerHeight`. This is because the `outerHeight` includes the browser chrome: measurements were taken on a browser with an address bar and bookmarks bar totaling 100px in height, but no chrome on the left or right of the window.

The area within the `innerHeight` and `innerWidth` is generally considered the **layout viewport**. The browser chrome is not considered part of the viewport.

When zoomed in, both Firefox and Chrome report the new CSS pixel size for `innerWidth` and `clientWidth`. The values returned for the `outerWidth` and `outerHeight` depend on the browser: Firefox reports the new value in CSS pixels, but Chrome returns the length in the default pixel size. When zoomed in you may get:

```
document.documentElement.clientWidth; /* 800 */
window.innerWidth; /* 800 */
window.outerWidth; /* 800 in Firefox, 1200 in chrome */
```

```
document.documentElement.clientHeight; /* 533 */
window.innerHeight; /* 533 */
window.outerHeight; /* 596 in Firefox, 900 in chrome */
```

The viewport was originally 1200 x 800 pixels. Upon zooming in, the viewport became 800 x 533 pixels. This is the *layout viewport*. Sticky headers or footers, with the following styles, will stick to the top and bottom of the *layout viewport* respectively.

```
body > header {
  position: fixed;
  top: 0;
}
body > footer {
  position: fixed;
  bottom: 0;
}
```

We got the 800 x 533 measurement when we zoomed in using the keyboard. The header and footer stayed flush against the top and bottom of the window. But what if we had pinched-zoomed on a tablet? What if a dynamic keyboard pops open on a phone?

Layout and visual viewports

The web contains two viewports, the **layout viewport** and the **visual viewport**. The visual viewport is the part of the web page that is currently visible in the browser and can change. When the user pinch-zooms the page, pops open a dynamic keyboard, or when a previously hidden address bar becomes visible, the visual viewport shrinks but the layout viewport is unchanged.

Fixed sticky headers or footers, as discussed above, stick to the top and bottom of the *layout viewport*, and therefore remain in view when we zoom in with the keyboard. If you pinch-zoom, the layout viewport may not be fully visible. If you magnify from the middle of the layout viewport, the content will expand in all four directions. If you have a sticky header or footer, they will still be stuck to the top or bottom of the layout viewport, but they may not be visible at the top and bottom of the device's screen — which is the visual viewport. The visual viewport is the currently visible portion of the layout viewport. If you scroll down, you are changing the contents of the visual viewport and bringing the bottom of the layout viewport into view, displaying the sticky footer, which will then stay stuck at the bottom.

The visual viewport is the visual portion of a screen not including on-screen keyboards, areas outside of a pinch-zoom area, or other feature that doesn't scale with the dimensions of a page. The visual viewport is the same size as the layout viewport or smaller.

For a page containing iframes, objects, or external SVG, both the containing pages and each included file has their own unique window object. Only the top-level window has a visual viewport that may be distinct from the layout viewport. For included documents, the visual viewport and layout viewport are the same.

CSS

The layout viewport and visual viewport described above are not the only viewports you will encounter. Any sub-viewport that is fully or partially displayed within the layout viewport is considered a visual viewport.

We generally think of `width` and `height` media queries as being relative to the width and height of the browser window. They are actually relative to the viewport, which is the window in the main document but is the intrinsic size of the element's parent in a nested browsing context like objects, iframes and SVG. In CSS, we also have length units based on the viewport size. A `vh` unit is 1% of the layout viewport's height. Similarly, the `vw` unit is 1% of the layout viewport's width.

`<iframe>`

Inside an `<iframe>`, the visual viewport is the size of the inner width and height of the iframe, rather than the parent document. You can set any height and width on an iframe, but the whole document may not be visible.

If you use viewport length units in your CSS within the iframe document, `1vh` will be 1% of the height of the iframe, and `1vw` will be 1% of the width of the document.

```
iframe {
  width: 50vw;
}
```

If the iframe is set to 50vw, it will be 50% of the width of the `1200px` parent document in our example above, or `600px`, with `1vw` being `6px`. When zoomed in, the iframe shrinks to `400px` and `1vw` becomes `4px`.

A width-based media query within the iframe document is relative to the iframe's viewport.

```
@media screen and (width <= 500px) {
  p {
    color: red;
  }
}
```

If the above CSS is included in the iframe, the paragraphs will become red when the user has zoomed in, but this style does not apply in the non-zoomed-in state.

SVG

In an SVG document, the viewport is the visible area of the SVG image. You can set any height and width on an `<svg>`, but the whole image might not be visible. The area that is visible is called the viewport. The size of the viewport can be defined using the width and height attributes of the `<svg>` element.

```
<svg height="300" width="400"></svg>
```

In this example, the viewport has an aspect ratio of 3:4 and is, by default, 400 by 300 units, with a unit generally being a CSS pixel.

SVG also has an internal coordinate system defined via the viewBox attribute, which is not related to this viewport discussion.

If you include an SVG file in your HTML, the viewport of the SVG is the initial containing block, or the width and height of the SVG container. Using the `@media` query in your SVG's CSS is relative to that container, not the browser.

```
@media screen and (400px <= width <= 500px) {
  /* CSS goes here */
}
```

Generally, when you write the above media query, the styles are applied if the viewport, generally the browser window, is between 400px and 500px, inclusive. The width media query in the SVG is based on the element in which the SVG is contained — the `<img>` if the source is an SVG file, the SVG itself if the SVG is included directly into the HTML, or the parent if the parent element has a width assigned and — not the viewport's width. With the above media query being in our SVG file, the CSS is applied if the SVG container is between 400px and 500px.

JavaScript

The `VisualViewport` interface provides a mechanism for querying and modifying the properties of the visual viewport.

The `Viewport` interface provides a mechanism for querying and modifying the properties of the visual viewport.

Mobile viewports

Mobile devices come in all shapes and sizes, with screens of differing device pixel ratios. The mobile browser's viewport is the area of the window in which web content can be seen, which is not necessarily the same size as the rendered page. Mobile browsers render pages in a virtual window or viewport, generally at 980px, which is usually wider than the screen, and then shrink the rendered result down so it can all be seen at once. Users can then pan and zoom to see different areas of the page. For example, if a mobile screen has a width of 320px, a website might be rendered with a virtual viewport of 980px, and then it will

be shrunk down to fit into the 320px space, which, depending on the design, is illegible for many if not everyone. To tell a mobile browser to use the viewport width instead of the default 980px as the width of the screen, developers can include a viewport meta tag, like the following:

```
<meta name="viewport" content="width=device-width" />
```

The `width` property controls the size of the viewport. It should preferably be set to `device-width`, which is the width of the screen in CSS pixels at a scale of 100%. There are other properties, including `maximum-scale`, `minimum-scale`, and `user-scalable`, which control whether users can zoom the page in or out, but the default values are the best for accessibility and user experience, so these can be omitted.

See also

- CSSOM view module
- CSSOM view API
- `<meta>`, specifically `<meta name="viewport">`
- CSS viewport module

CSS custom functions and mixins

The **CSS custom functions and mixins** module allows developers to create reusable blocks of CSS code that can accept arguments, contain complex logic (defined using features such as CSS `if()` functions and `@media` at-rules), and return values based on that logic.

CSS custom functions are defined in `@function` at-rules and called using `<dashed-function>` syntax, which looks very similar to CSS custom property syntax except that it includes parentheses at the end containing arguments (for example, `--my-function(30px, 3)`). CSS custom functions can be called within any property value, and will return a value based on the arguments passed into the function and the logic inside it.

CSS mixins are defined in `@mixin` at-rules and applied using `@apply` at-rules nested inside rulesets. CSS mixins define a set of properties that can be reused inside multiple rulesets and customized with arguments and logic.

CSS custom functions and mixins can be assigned optional data types for their arguments and return values, to restrict the values passed into them and returned from them.

> **Note:** Currently, only CSS custom functions have browser support. CSS mixins are not currently supported in any browser.

Reference

At-rules and descriptors

- `@function`
  - `result`

The CSS custom functions and mixins module also introduces the `@mixin`, `@apply`, `@contents`, and `@env` at-rules. Currently, no browsers support these features.

Data types and values

- `<dashed-function>`

Functions

- `type()`

Interfaces

- `CSSFunctionRule`
- `CSSFunctionDeclarations`
- `CSSFunctionDescriptors`

Guides

Using CSS custom functions

This guide teaches you how to use CSS custom functions and presents some typical use cases.

Related concepts

CSS values and units

- `abs()`
- `acos()`
- `asin()`
- `atan()`
- `atan2()`
- `calc()`
- `clamp()`
- `cos()`
- `exp()`
- `log()`
- `max()`
- `min()`
- `mod()`
- `pow()`
- `rem()`
- `round()`
- `sign()`
- `sin()`
- `sqrt()`
- `tan()`

See also

- `if()`
- `@media`

Using CSS custom functions

CSS custom functions enable you to create reusable blocks of CSS code that can accept arguments, contain complex logic (defined using features such as CSS `if()` functions and `@media` at-rules), and return values based on that logic. They work similarly to CSS custom properties, but provide more flexibility.

In this article, we'll show you how to use them and present some real-world examples.

Function basics

A basic CSS custom function definition looks like this:

```
@function --half-opacity() {
  result: 0.5;
}
```

After the `@function` syntax, we define a name for the function: `--half-opacity`. This must be a `<dashed-ident>` type — it must start with a double-dash, and is case-sensitive. The function name is immediately followed by a set of parentheses (`()`) and a set of curly braces (`{}`).

> **Note:** If multiple CSS functions are given the same name, the function in the stronger cascade `@layer` wins. If all of them are in the same layer, the function defined last in the source order wins.

Inside the curly braces is the function **body**, which is where the function logic is defined. This can contain multiple declarations, including custom properties (which will be locally scoped to the function body), at-rules such as `@media`, and the `result` descriptor. The value of the `result` descriptor is evaluated to determine the value returned by the function.

Here, we are setting `result` to the value `0.5`: the `--half-opacity()` function will always return `0.5`.

Why "result" and not "return"?

The `result` descriptor sounds similar in functionality to the JavaScript function `return` statement. However, `return` is not used in CSS functions. This is because, unlike JavaScript `return` statements, CSS functions do not return a value as soon as a `result` declaration is reached.

The body of a CSS function is evaluated from start to finish. If there are multiple `result` declarations included in the body, the last one in the source order overrides the earlier ones.

Calling a CSS function

A CSS function can be called in place of any suitable property value using the `<dashed-function>` syntax, which consists of the function name followed by parentheses containing the arguments to pass to the function, if any. For example, we can call our `--half-opacity()` function like this:

```
h2 {
  opacity: --half-opacity();
}
```

Since this function always returns the value `0.5`, the previous declaration is the equivalent of `opacity: 0.5`. This isn't very useful. You might as well just use a custom property, or the literal value `0.5`.

Let's move on to look at how we can use CSS functions.

Feature detecting CSS functions

One practical use of CSS functions with no parameters is in feature detection. In all of the examples we'll be looking at in this article, we define a `--supports()` function that looks like this:

```
@function --supports() {
  result: none;
}
```

You can then define a "feature not supported" banner, and set its `display` property to `--supports()`:

```
<p class="support">
  ⚠️ Your browser doesn't currently support CSS custom functions.
</p>
```

```
.support {
  /* ... */
  display: --supports();
}
```

In browsers that support custom functions, `display` will be set to `none`, and the support banner will be hidden. In non-supporting browsers, the `display: --supports()` declaration will be invalid and therefore ignored; therefore, the banner will be displayed.

Specifying function parameters

CSS function parameters are specified as comma-separated custom properties inside the parentheses following the function name. For example:

```
@function --transparent(--color, --alpha) {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

This function has a name of `--transparent` and takes two custom properties as parameters, `--color` and `--alpha`, which can be used locally inside the function body. The body contains a `result` descriptor, which uses CSS relative color syntax to convert the input `--color` value into an `oklch()` color with an alpha channel value as specified in the input `--alpha` value.

You can then call this function anywhere you want to produce a semi-transparent version of an existing color.

For example:

```
section {
  --base-color: #faa6ff;
  background-color: --transparent(var(--base-color), 0.8);
}
```

Specifying data types

It is possible to specify permitted data types for the function parameters and return value. When you don't specify these, the function will accept any type for these values.

Let's modify our previous function to provide data types:

```
@function --transparent(--color type(<color>), --alpha type(<number>)) returns
  type(<color>) {
```

```
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

The data type of each parameter is specified after the parameter name, and the data type of the `result` is specified just before the opening curly brace, preceded by the `returns` keyword. The `type()` function is used to specify a data type.

Note that, in cases where you are only specifying a single data type, you can omit the `type()` syntax and just write the type as a shorthand:

```
@function --transparent(--color <color>, --alpha <number>) returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

Now the function will only produce a valid value if the input arguments are a `<color>` and a `<number>`, respectively, and the `result` is a `<color>`. If not, for example:

```
section {
  --base-color: #faa6ff;
  background-color: --transparent(var(--base-color), 50%);
}
```

then the value will become invalid at computed-value time (since `50%` is not a `<number>` but a `<percentage>`) and the `background-color` will end up being set to `transparent`.

Specifying multiple permitted types

You can specify multiple accepted data types using the `|` symbol as a separator, for example:

```
@function --transparent(--color <color>, --alpha type(<number> | <percentage>))
  returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

In such cases, the full `type()` syntax must be used.

With this adjustment, the `--transparent(var(--base-color), 50%)` function call is now valid.

Specifying default values

You can also specify default values for parameters, after a colon at the end of their definition. For example:

```
@function --transparent(--color <color>, --alpha <number>: 0.8) returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

The `--alpha` parameter's default value is now `0.8`. If you want to use this value, you can omit the second argument when calling the function:

```
section {
  --base-color: #faa6ff;
  background-color: --transparent(var(--base-color));
}
```

> **Note:** If an invalid value is passed in as a function argument and a default value is specified in that parameter definition, the invalid value will be ignored, and the default value will be used instead.

Color adjust functions example

You can see the `--transparent()` function in action in our color-adjust-functions example (see the source code).

This example also includes functions called `--lighter()` and `--darker()` that work similarly to `--transparent()`, but return lighter and darker variants of a color, respectively:

```
@function --transparent(--color <color>, --alpha <number>: 0.8) returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}

@function --lighter(--color <color>, --lightness-adjust <number>: 0.2) returns
  <color> {
  result: oklch(from var(--color) calc(l + var(--lightness-adjust)) c h);
}

@function --darker(--color <color>, --lightness-adjust <number>: 0.2) returns
  <color> {
  result: oklch(from var(--color) calc(l - var(--lightness-adjust)) c h);
}
```

A library of functions like these can become very useful for defining color schemes based on a single color:

```
:root {
  --base-color: #faa6ff;
}

section {
  background-color: --transparent(var(--base-color));
  border: 3px solid --lighter(var(--base-color), 0.1);
  color: --darker(var(--base-color), 0.55);
}
```

Including complex logic

You can include more complex logic in functions using constructs such as `@media` at-rules and `if()` functions.

Our responsive-narrow-wide example (see the source code) features a function called `--narrow-wide()`, which can be used to provide two value options for any property. One will be set if the viewport is below a specific breakpoint, and the other will be set if it is above.

The `--narrow-wide()` function accepts two parameters, `--narrow` and `--wide`. The returned `result` is the `--wide` property, unless the viewport is less than `700px` wide, in which case `--narrow` is returned.

```
@function --narrow-wide(--narrow, --wide) {
  result: var(--wide);
  @media (width < 700px) {
```

```
    result: var(--narrow);
  }
}
```

This function can be used to provide responsive value options in multiple contexts:

```
body {
  display: grid;
  grid-template-columns: repeat(--narrow-wide(1, 3), 1fr);
  gap: --narrow-wide(0, 20px);
  padding: 0 20px;
}

h2 {
  font-size: --narrow-wide(2.5rem, 2rem);
}

p {
  font-size: --narrow-wide(1.4rem, 1rem);
  line-height: 1.5;
}
```

Using an <code>if()</code> function

We could rewrite the `--narrow-wide()` function to use an `if()` function instead:

```
@function --narrow-wide(--narrow, --wide) {
  result: if(media(width < 700px): var(--narrow) ; else: var(--wide));
}
```

Writing complex syntax once, then reusing it

A key use case for CSS functions is to define a complex section of syntax once, and be able to reuse it multiple times with a much simpler function call.

Our gradient-function example (see the source code) provides an example of this. It features a function called `--shippo-pattern()` that accepts length and color arguments, and returns a complex `background` value featuring multiple `radial-gradient()` backgrounds:

```
@function --shippo-pattern(--size <length>, --tint <color>) {
  result:
    radial-gradient(closest-side, transparent 98%, rgb(0 0 0 / 0.3) 99%) 0 0 /
      var(--size) var(--size),
    radial-gradient(closest-side, transparent 98%, rgb(0 0 0 / 0.3) 99%)
      calc(var(--size) / 2) calc(var(--size) / 2) / var(--size) var(--size)
      var(--tint);
}
```

With this function defined, we can now create variants of this background value with different color tints and circle sizes:

```
#one {
  background: --shippo-pattern(100px, #ddeeff);
}

#two {
  background: --shippo-pattern(3.5rem, lime);
}

#three {
  background: --shippo-pattern(10vw, purple);
}
```

See also

- CSS custom properties
- CSS custom functions and mixins module
- Custom CSS Functions in the Browser by Miriam Suzanne (2025)
- CSS @function + CSS if() by Bramus (2025)
- 5 Useful CSS functions using the new @function rule by Una Kravets (2025)

CSS custom highlight API

The **CSS custom highlight API** module provides a programmatic way to target specific ranges of text defined by range objects, without affecting the underlying DOM structure. The range objects can then be selected via `::highlight()` pseudo-elements, and have highlight styles added and removed. The features of this module can create highlight effects similar to how text editors highlight spelling or grammar errors, and code editors highlight syntax errors.

The CSS Custom Highlight API extends the concept of other highlight pseudo-elements such as `::selection`, `::spelling-error`, `::grammar-error`, and `::target-text` by providing a way to create arbitrary text ranges (defined as `Range` objects in JavaScript) and style them via CSS, rather than being limited to browser-defined ranges.

Custom highlight API in action

To enable styling text ranges on a webpage using the CSS Custom Highlight API, you create a `Range` object, then a `Highlight` object for the range. After registering the highlight using the `HighlightRegistry.set()` method, you can then select the range using the `::highlight()` pseudo-element. The name defined in the `set()` method is used as the parameter of the `::highlight()` pseudo-element selector to select that range. The range selected by the `::highlight()` pseudo-element can be styled using a limited number of properties.

```
<h1>Directions</h1>
<h2>Lincoln Memorial to Martin Luther King, Jr. Memorial</h2>
<ol><li>Head south on Lincoln Memorial Circle</li
  ><li>Turn right toward Independence Ave</li
  ><li>Turn left onto Independence Ave</li
  ><li>Turn right onto West Basin Dr</li
  ><li>Look up when you reach 64 Independence Ave!</li>
</ol>
<hr />
<label
  >Number of steps completed:
  <input type="number" min="0" max="5" value="0" id="currentStep" />
</label>
```

This example uses the `text-decoration` property to strike through the `steps` highlight range defined by our JavaScript:

```
::highlight(steps) {
  text-decoration: line-through;
  color: blue;
}
```

We create a `Range` with a start and end node (which is the same node in this case). We then set this range as the `Highlight` using the `set()` method of the CSS `HighlightRegistry` interface.

```
const rangeToHighlight = new Range();
const list = document.querySelector("ol");
rangeToHighlight.setStart(list, 0);
rangeToHighlight.setEnd(list, 0);

CSS.highlights.set("steps", new Highlight(rangeToHighlight));
```

An event listener updates the end of the highlighted range when the number of completed steps changes:

```
const currentPositionSlider = document.querySelector("input");
currentPositionSlider.addEventListener("change", (e) => {
  rangeToHighlight.setEnd(list, e.target.value);
});
```

Reference

Pseudo-elements

- `::highlight()`

Interfaces

- `Highlight`
- `HighlightRegistry`

Interface extensions

This module adds properties and methods to interfaces defined in other specifications.

- `CSS`
  - `CSS.highlights`

Guides

CSS custom highlight API

> The concepts and usage of the CSS custom highlight API, including creating `Range` and `Highlight` objects, registering the highlights using the `HighlightRegistry`, and styling the highlights using the `::highlight()` pseudo-element.

Related concepts

- `::grammar-error`
- `::selection`
- `::spelling-error`
- `::target-text`
- `AbstractRange` interface
- `Range` interface and `Range()` constructor
- Text fragments
- `FragmentDirective` interface

See also

- CSS pseudo-element module
- CSS Object Model (CSSOM) APIs

CSS display

The **CSS display** module defines how the CSS formatting box tree is generated from the document element tree and defines properties controlling it.

Reference

Properties

- `display`
- `order`
- `visibility`
- `reading-flow`
- `reading-order`

Data types

- `<display-outside>`
- `<display-inside>`
- `<display-listitem>`
- `<display-box>`
- `<display-internal>`
- `<display-legacy>`

Glossary terms and definitions

- Block
- Block formatting context (BFC)
- Block-level content
- Containing block
- Flow layout
- Reading order

- Replaced elements
- Ruby

Guides

Using the multi-keyword syntax with CSS display

Describes the multi-keyword syntax and compares this syntax with legacy single-keyword values.

Block and inline layout in normal flow

The basics of how block and inline elements behave when they are part of the normal flow.

Flow layout and overflow

How overflow works when working with normal flow.

Flow layout and writing modes

How flow layout behaves when used with different document writing modes.

Introduction to formatting contexts

Formatting contexts, including block, inline, and flex, their behaviors, and use.

In flow and out of flow

What takes elements out of flow, and the effect of creating new Block Formatting Contexts.

Related Concepts

Properties

- `overflow`
- `transition-behavior`

Glossary terms and definitions

- Flex
- Grid
- Inline formatting context
- Inline-level content

Guides

- CSS flexible box layout module
  - Basic concepts of flexbox
  - Aligning items in a flex container
  - Controlling ratios of flex items along the main axis
  - Mastering wrapping of flex items
  - Ordering flex items
  - Relationship of flexbox to other layout methods
  - Typical use cases of flexbox
- CSS grid layout module
  - Basic concepts of grid layout
  - Relationship of grid layout to other layout methods
  - Line-based placement
  - Grid template areas
  - Layout using named grid lines
  - Auto-placement in grid layout
  - Box alignment in grid layout
  - Grids, logical values and writing modes
  - CSS grid layout and accessibility
  - Realizing common layouts using grids

See also

- CSS lists and counters module
- CSS ruby layout module
- CSS table module
- Visual formatting model

Block and inline layout in normal flow

In this guide, we will explore the basics of how Block and Inline elements behave when they are part of the normal flow.

Normal Flow is defined in the CSS 2.1 specification, which explains that any boxes in normal flow will be part of a *formatting context*. They can be either block or inline, but not both at once. We describe block-level boxes as participating in a *block formatting context*, and inline-level boxes as participating in an *inline formatting context*.

The behavior of elements which have a block or inline formatting context is also defined in this specification. For elements with a block formatting context, the spec says:

> "In a block formatting context, boxes are laid out one after the other, vertically, beginning at the top of a containing block. The vertical distance between two sibling boxes is determined by the 'margin' properties. Vertical margins between adjacent block-level boxes in a block formatting context collapse.
> In a block formatting context, each box's left outer edge touches the left edge of the containing block (for right-to-left formatting, right edges touch)." - 9.4.1

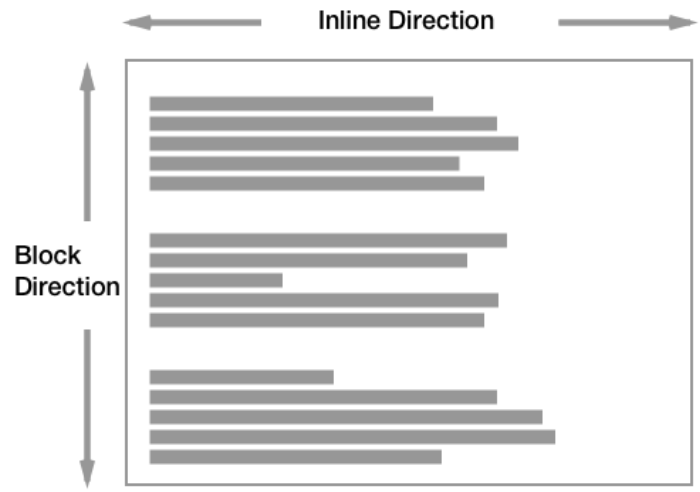
For elements with an inline formatting context:

> "In an inline formatting context, boxes are laid out horizontally, one after the other, beginning at the top of a containing block. Horizontal margins, borders, and padding are respected between these boxes. The boxes may be aligned vertically in different ways: their bottoms or tops may be aligned, or the baselines of text within them may be aligned. The rectangular area that contains the boxes that form a line is called a line box." - 9.4.2

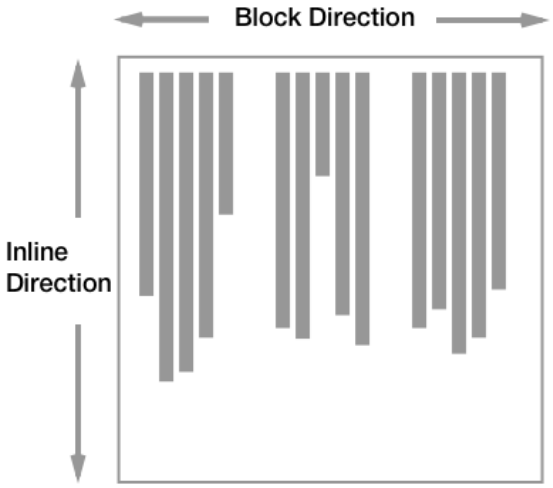
Note that the CSS 2.1 specification describes documents as being in a horizontal, top-to-bottom writing mode. For example, by describing the vertical distance between block boxes. The behavior on block and inline elements is the same when working in a vertical writing mode; we explore this in our Flow layout and writing modes guide.

Elements participating in a block formatting context

Block elements in a horizontal writing mode such as English, lay out vertically, one below the other.



In a vertical writing mode they would lay out horizontally.



In this guide, we will be working in English and therefore a horizontal writing mode. However, everything described should work in the same way if your document is in a vertical writing mode.

As defined in the specification, the margins between two block boxes are what creates separation between the elements. We can see this with the layout of two paragraphs, to which I have added a border. The default browser stylesheet adds spacing between the paragraphs by way of adding a margin to the top and bottom.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
p {
  border: 2px solid green;
}
```

If we set margins on the paragraph element to `0` then the borders will touch.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
```

```
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
p {
  border: 2px solid green;
  margin: 0;
}
```

By default block elements will consume all of the space in the inline direction, so our paragraphs spread out and get as big as they can inside their containing block. If we give them a width, they will continue to lay out one below the other - even if there would be space for them to be side by side. Each will start against the start edge of the containing block, so the place at which sentences would begin in that writing mode.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
p {
  border: 2px solid green;
  width: 40%;
}
```

Margin collapsing

The spec explains that margins between block elements *collapse*. This means that if you have an element with a top margin immediately after an element with a bottom margin, rather than the total space being the sum of these two margins, the margin collapses, and so will essentially become as large as the larger of the two margins.

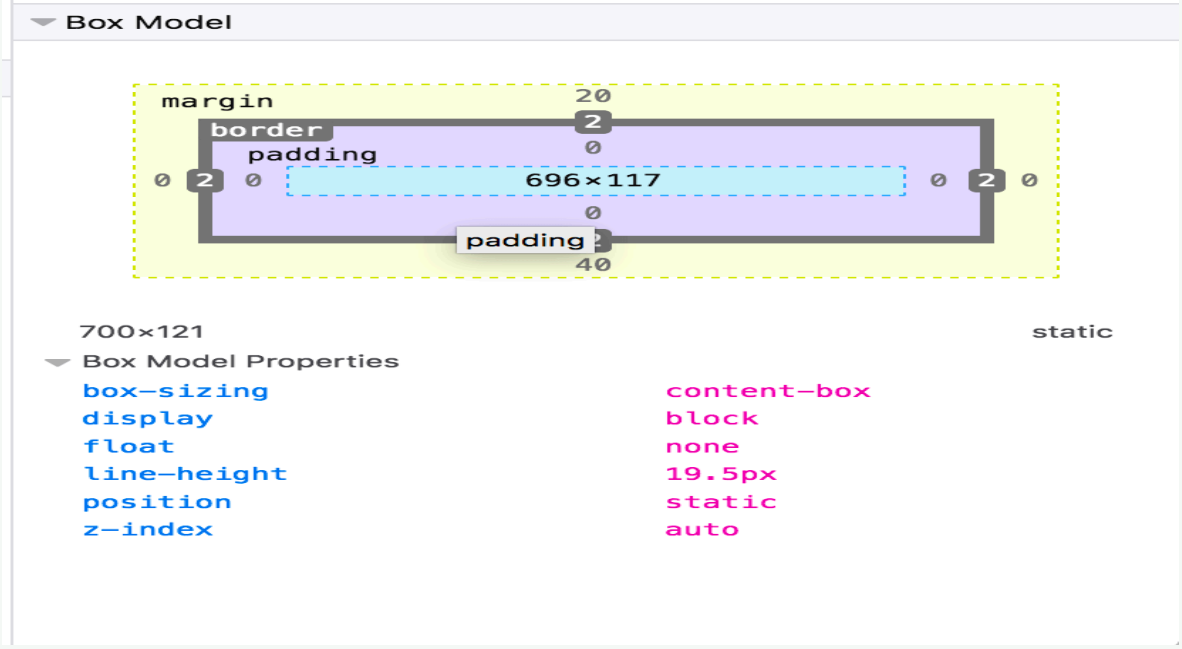
In the example below, the paragraphs have a top margin of `20px` and a bottom margin of `40px`. The size of the margin between the paragraphs is `40px` as the smaller top margin on the second paragraph has collapsed with the larger bottom margin of the first.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
p {
  border: 2px solid green;
  margin: 20px 0 40px 0;
}
```

You can read more about margin collapsing in our article Mastering Margin Collapsing.

> **Note:** If you are not sure whether margins are collapsing, check the Box Model values in your browser DevTools. This will give you the actual size of the margin which can help you to identify what is happening.



Elements participating in an inline formatting context

Inline elements display one after the other in the direction that sentences run in that particular writing mode. While we don't tend to think of inline elements as having a box, as with everything in CSS they do. These inline boxes are arranged one after the other. If there is not enough space in the containing block for all of the boxes a box can break onto a new line. The lines created are known as line boxes.

In the following example, we have three inline boxes created by a paragraph with a `<strong>` element inside it.

```
<p>
  Before that night—<strong>a memorable night</strong>, as it was to
  prove—hundreds of millions of people had watched the rising smoke-wreaths of
  their fires without drawing any special inspiration from the fact.
</p>
```

The boxes around the words before the `<strong>` element and after the `</strong>` element are referred to as anonymous boxes, boxes introduced to ensure that everything is wrapped in a box, but ones that we cannot target directly.

The line box size in the block direction (so the height when working in English) is defined by the tallest box inside it. In the next example, the `<strong>` element is 300%; since that content spans two lines, it now defines the height of the line boxes of those two lines.

```
<p>
  Before that night—<strong>a memorable night</strong>, as it was to
  prove—hundreds of millions of people had watched the rising smoke-wreaths of
  their fires without drawing any special inspiration from the fact.
</p>
```

```
strong {
  font-size: 300%;
}
```

Find out more about how block and inline boxes behave in our guide to the visual formatting model.

The display property and flow layout

In addition to the rules existing in CSS2.1, new levels of CSS further describe the behavior of block and inline boxes. The `display` property defines how a box and any boxes inside it behave. In the CSS Display Model Level 3, we can learn more about how the `display` property changes the behavior of boxes and the boxes they generate.

The display type of an element defines the outer display type; this dictates how the box displays alongside other elements in the same formatting context. It also defines the inner display type, which dictates how boxes inside this element behave. We can see this very clearly when considering a flex layout. In the example below I have a `<div>`, which I have given `display: flex`. The flex container behaves like a block element: it displays on a new line and takes up all of the space it can in the inline direction. This is the outer display type of `block`.

The flex items however are participating in a flex formatting context, because their parent is the element with `display: flex`, which has an inner display type of `flex`, establishing the flex formatting context for the direct children.

```
<div class="container">
  <div>Flex Item</div>
  <div>Flex Item</div>
  <div>
    <div>Children</div>
    <div>are in</div>
    <div>normal flow</div>
  </div>
</div>
```

```
.container {
  display: flex;
}

.container > * {
  border: 1px solid green;
}
```

Therefore you can think of every box in CSS working in this way. The box itself has an outer display type, so it knows how to behave alongside other boxes. It then has an inner display type which changes the way its children behave. Those children then have an outer and inner display type too. The flex items in the previous example become flex level boxes, so their outer display type is dictated by way of them being

part of the flex formatting context. They have an inner display type of *flow* however, meaning that their children participate in normal flow. Items nested inside our flex item lay themselves out as block and inline elements unless something changes their display type.

This concept of the outer and inner display type is important as this tells us that a container using a layout method such as flexbox (`display: flex`) and grid layout (`display: grid`) is still participating in block and inline layout, due to the outer display type of those methods being `block`.

Changing the Formatting Context an element participates in

Browsers display items in block or inline formatting contexts based on what normally makes sense for that element. For example, a `<strong>` element is used to strongly emphasize a span of content and is displayed in bold in browsers by default. It would not generally make sense for that `<strong>` element to be displayed as a block-level element, breaking onto a new line. If you did want all `<strong>` elements to display as block boxes, you could do so by setting `strong { display: block; }`. The ability to style content with CSS means you can always use the most appropriate semantic HTML elements to mark up your content and then change how they are displayed with CSS.

```
<p>
  Before that night—<strong>a memorable night</strong>, as it was to
  prove—hundreds of millions of people had watched the rising smoke-wreaths of
  their fires without drawing any special inspiration from the fact.
</p>
```

```
strong {
  display: block;
}
```

Summary

In this guide, we have looked at how elements display in normal flow, as block and inline elements. An HTML document with no CSS styling at all will display in a readable way due to this default behavior. Understanding how normal flow works is an important starting point for understanding CSS layout as a whole.

See also

- CSS Basic Box Model
- Learn: Normal Flow
- Inline-level elements
- Block-level elements

Block formatting context

A **block formatting context** (BFC) is a part of a visual CSS rendering of a web page. It's the region in which the layout of block boxes occurs and in which floats interact with other elements.

A block formatting context is created by at least one of the following:

- The root element of the document (`<html>`).
- Floats (elements where `float` isn't `none`).
- Absolutely positioned elements (elements where `position` is `absolute` or `fixed`).
- Inline-blocks (elements with `display: inline-block`). This is the default display type for `<button>` and button `<input>` elements.
- Table cells (elements with `display: table-cell`, which is the default for HTML table cells).
- Table captions (elements with `display: table-caption`, which is the default for HTML table captions).
- Anonymous table cells implicitly created by the elements with `display: table`, `table-row`, `table-row-group`, `table-header-group`, `table-footer-group` (which is the default for HTML tables, table rows, table bodies, table headers, and table footers, respectively), or `inline-table`.
- Elements with `display: flow-root`.
- Flex items (direct children of the element with `display: flex` or `inline-flex`) if they are neither flex nor grid nor table containers themselves.
- Grid items (direct children of the element with `display: grid` or `inline-grid`) if they are neither flex nor grid nor table containers themselves.
- Block elements where `overflow` has a value other than `visible` and `clip`.
- Elements with `contain: layout`, `content`, or `paint`.
- Query containers (elements where `container-type` isn't `normal`).
- Multicol containers (elements where `column-count` or `column-width` isn't `auto`, including elements with `column-count: 1`).
- `column-span: all`, even when the `column-span: all` element isn't contained by a multicol container.

Formatting contexts affect layout because an element that establishes a new block formatting context will:

- contain internal floats.
- exclude external floats.
- suppress margin collapsing.

Flex and grid containers, defined by setting an element's `display` to `flex`, `grid`, `inline-flex`, or `inline-grid`, establishes a new flex or grid formatting context. These are similar to block formatting context except there are no floating children available inside a flex or grid container, but these formatting contexts do exclude external floats and suppress margin collapsing.

Examples

Let's have a look at a couple of these in order to see the effect creating a new BFC.

Contain internal floats

In the following example, we have float content that is the same height as the content alongside it. We have a floated element inside a `<div>` with a `border` applied. The content of that `<div>` has floated alongside the floated element. As the content of the float is taller than the content alongside it, the border of the `<div>` now runs through the float. As explained in the guide to in flow and out of flow elements, the float has been taken out of flow so the `background` and `border` of the `<div>` only contain the content and not the float.

using `overflow: auto`

Setting `overflow: auto` or set other values than the initial value of `overflow: visible` created a new BFC containing the float. Our `<div>` now becomes a mini-layout inside our layout. Any child element will be contained inside it.

The problem with using `overflow` to create a new BFC is that the `overflow` property is meant for telling the browser how you want to deal with overflowing content. There are some occasions in which you will find you get unwanted scrollbars or clipped shadows when you use this property purely to create a BFC. In addition, it is potentially not readable for a future developer, as it might not be obvious why you used `overflow` for this purpose. If you use `overflow`, it is a good idea to comment the code to explain.

using `display: flow-root`

The `display: flow-root` value lets us create a new BFC without any other potentially problematic side-effects. Using `display: flow-root` on the containing block creates a new BFC.

With `display: flow-root;` on the `<div>`, everything inside that container participates in the block formatting context of that container, and floats will not poke out of the bottom of the element.

The value name of `flow-root` makes sense when you understand you are creating something that acts like the `root` element (`<html>` element in browser) in terms of how it creates a new context for the flow layout inside it.

HTML

```
<section>
  <div class="box1">
    <div class="float">I am a floated box!</div>
    <p>I am content inside the container.</p>
  </div>
</section>
<section>
  <div class="box2">
    <div class="float">I am a floated box!</div>
    <p>I am content inside the <code>overflow:auto</code> container.</p>
  </div>
</section>
<section>
  <div class="box3">
    <div class="float">I am a floated box!</div>
    <p>I am content inside the <code>display:flow-root</code> container.</p>
  </div>
</section>
```

CSS

```
section {
  height: 150px;
}
.box1 {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
}
.box2,
.box3 {
  background-color: aliceblue;
  border: 5px solid steelblue;
}
.box2 {
  overflow: auto;
}
.box3 {
  display: flow-root;
}
.float {
  float: left;
  width: 200px;
  height: 100px;
  background-color: rgb(255 255 255 / 50%);
  border: 1px solid black;
  padding: 10px;
}
```

Exclude external floats

In the following example, we are using `display: flow-root` and floats, creating two side-by-side boxes demonstrating that an element in the normal flow establishes a new BFC and does not overlap the margin box of any floats in the same block formatting context as the element itself.

HTML

```
<section>
  <div class="float">Try to resize this outer float</div>
  <div class="box"><p>Normal</p></div>
</section>
<section>
  <div class="float">Try to resize this outer float</div>
  <div class="box2">
    <p><code>display:flow-root</code></p>
  </div>
</section>
```

CSS

```
section {
  height: 150px;
}
.box {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
}
.box2 {
  background-color: aliceblue;
  border: 5px solid steelblue;
  display: flow-root;
}
.float {
  float: left;
  overflow: hidden; /* required by resize:both */
  resize: both;
  margin-right: 25px;
  width: 200px;
  height: 100px;
  background-color: rgb(255 255 255 / 75%);
  border: 1px solid black;
  padding: 10px;
}
```

Prevent margin collapsing

You can create a new BFC to avoid margin collapsing between two neighbor elements.

Margin collapsing example

In this example we have two adjacent `<div>` elements, which each have a vertical margin of `10px`. Because of margin collapsing, the vertical gap between them is `10px`, not the `20px` we might expect.

```
<div class="blue"></div>
<div class="red"></div>
```

```
.blue,
.red {
  height: 50px;
  margin: 10px 0;
}

.blue {
  background: blue;
}

.red {
  background: red;
}
```

Preventing margin collapsing

In this example, we wrap the second `<div>` in an outer `<div>`, and create a new BFC by using `overflow: hidden` on the outer `<div>`. This new BFC prevents the margins of the nested `<div>` from collapsing with those of the outer `<div>`.

```
<div class="blue"></div>
<div class="outer">
  <div class="red"></div>
</div>
```

```
.blue,
.red {
  height: 50px;
  margin: 10px 0;
}

.blue {
  background: blue;
}

.red {
  background: red;
}

.outer {
  overflow: hidden;
  background: transparent;
}
```

See also

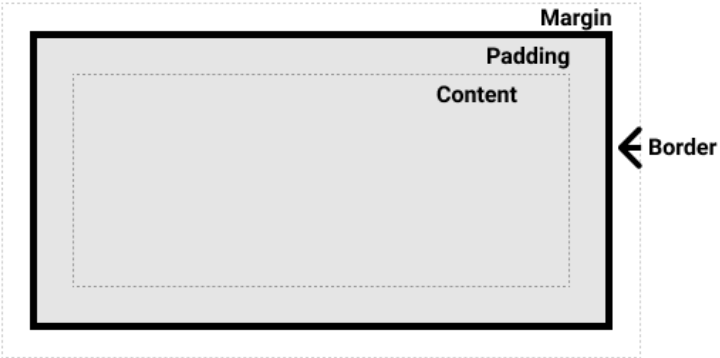
- CSS syntax
- Specificity
- Inheritance
- Box model
- Layout modes
- Visual formatting models
- Margin collapsing
- Initial, computed, used values, and actual values
- Value definition syntax
- Replaced elements

Layout and the containing block

The size and position of an element are often impacted by its **containing block**. Most often, the containing block is the content area of an element's nearest block-level ancestor, but this is not always the case. In this article, we examine the factors that determine an element's containing block.

When a user agent (such as your browser) lays out a document, it generates a box for every element. Each box is divided into four areas:

1. Content area
2. Padding area
3. Border area
4. Margin area



Many developers believe that the containing block of an element is always the content area of its parent, but that isn't necessarily true. Let's investigate the factors that determine what an element's containing block is.

Effects of the containing block

Before learning what determines the containing block of an element, it's useful to know why it matters in the first place.

The size and position of an element are often impacted by its containing block. Percentage values that are applied to the `width`, `height`, `padding`, `margin`, and offset properties of an absolutely positioned element (i.e., which has its `position` set to `absolute` or `fixed`) are computed from the element's containing block.

Identifying the containing block

The process for identifying the containing block depends entirely on the value of the element's `position` property:

1. If the `position` property is **`static`**, **`relative`**, or **`sticky`**, the containing block is formed by the edge of the *content box* of the nearest ancestor element that is either **a block container** (such as an inline-block, block, or list-item element) or **establishes a formatting context** (such as a table container, flex container, grid container, or the block container itself).
2. If the `position` property is **`absolute`**, the containing block is formed by the edge of the *padding box* of the nearest ancestor element that has a `position` value other than `static` (`fixed`, `absolute`, `relative`, or `sticky`).
3. If the `position` property is **`fixed`**, the containing block is established by the viewport (in the case of continuous media) or the page area (in the case of paged media).
4. If the `position` property is **`absolute`** or **`fixed`**, the containing block may also be formed by the edge of the *padding box* of the nearest ancestor element that has any of the following:
    - A `filter`, `backdrop-filter`, `transform`, `perspective`, `rotate`, `scale`, or `translate` value other than `none`.
    - A `contain` value of `layout`, `paint`, `strict` or `content` (e.g., `contain: paint;`).
    - A `will-change` value containing a property for which a non-initial value would form a containing block (e.g., `filter` or `transform`).
    - A `content-visibility` value of `auto`.

> **Note:** The containing block in which the root element (`<html>`) resides is a rectangle called the **initial containing block**. It has the dimensions of the viewport (for continuous media) or the page area (for paged media).

> **Note:** There are browser inconsistencies with `perspective` and `filter` contributing to containing block formation.

Calculating percentage values from the containing block

As noted above, when certain properties are given a percentage value, the computed value depends on the element's containing block. The properties that work this way are **box model properties** and **offset properties**:

1. The `height`, `top`, and `bottom` properties compute percentage values from the `height` of the containing block.
2. The `width`, `left`, `right`, `padding`, and `margin` properties compute percentage values from the `width` of the containing block.

> **Note:** A **block container** (such as an inline-block, block, or list-item element) either contains only inline-level boxes participating in an inline formatting context, or only block-level boxes participating in a block formatting context. An element is a block container only if it contains block-level or inline-level boxes.

Some examples

The HTML code for all our examples is:

```
<body>
  <section>
    <p>This is a paragraph!</p>
  </section>
</body>
```

Only the CSS is altered in each instance below.

Example 1

In this example, the paragraph is statically positioned, so its containing block is `<section>` because it's the nearest ancestor that is a block container (because of `display: block`).

```
<body>
  <section>
    <p>This is a paragraph!</p>
  </section>
</body>
```

```
body {
  background: beige;
}

section {
  display: block;
  width: 400px;
  height: 160px;
  background: lightgray;
}

p {
  width: 50%; /* == 400px * .5 = 200px */
  height: 25%; /* == 160px * .25 = 40px */
  margin: 5%; /* == 400px * .05 = 20px */
  padding: 5%; /* == 400px * .05 = 20px */
  background: cyan;
}
```

Example 2

In this example, the paragraph's containing block is the `<body>` element, because `<section>` is not a block container (because of `display: inline`) and doesn't establish a formatting context.

```
<body>
  <section>
    <p>This is a paragraph!</p>
  </section>
</body>
```

```
body {
  background: beige;
}
```

```
section {
  display: inline;
  background: lightgray;
}

p {
  width: 50%; /* == half the body's width */
  height: 200px; /* Note: a percentage would be 0 */
  background: cyan;
}
```

Example 3

In this example, the paragraph's containing block is `<section>` because the latter's `position` is `absolute`. The paragraph's percentage values are affected by the `padding` of its containing block, though if the containing block's `box-sizing` value were `border-box` this would not be the case.

```
<body>
  <section>
    <p>This is a paragraph!</p>
  </section>
</body>
```

```
body {
  background: beige;
}

section {
  position: absolute;
  left: 30px;
  top: 30px;
  width: 400px;
  height: 160px;
  padding: 30px 20px;
  background: lightgray;
}

p {
  position: absolute;
  width: 50%; /* == (400px + 20px + 20px) * .5 = 220px */
  height: 25%; /* == (160px + 30px + 30px) * .25 = 55px */
  margin: 5%; /* == (400px + 20px + 20px) * .05 = 22px */
  padding: 5%; /* == (400px + 20px + 20px) * .05 = 22px */
  background: cyan;
}
```

Example 4

In this example, the paragraph's `position` is `fixed`, so its containing block is the initial containing block (on screens, the viewport). Thus, the paragraph's dimensions change based on the size of the browser window.

```
<body>
  <section>
    <p>This is a paragraph!</p>
  </section>
</body>
```

```
body {
  background: beige;
}

section {
  width: 400px;
  height: 480px;
  margin: 30px;
  padding: 15px;
  background: lightgray;
}

p {
  position: fixed;
  width: 50%; /* == (50vw - (width of vertical scrollbar)) */
  height: 50%; /* == (50vh - (height of horizontal scrollbar)) */
  margin: 5%; /* == (5vw - (width of vertical scrollbar)) */
  padding: 5%; /* == (5vw - (width of vertical scrollbar)) */
  background: cyan;
}
```

Example 5

In this example, the paragraph's `position` is `absolute`, so its containing block is `<section>`, which is the nearest ancestor with a `transform` property that isn't `none`.

```
<body>
  <section>
    <p>This is a paragraph!</p>
  </section>
</body>
```

```
body {
  background: beige;
}

section {
  transform: rotate(0deg);
  width: 400px;
  height: 160px;
  background: lightgray;
}
```

```
p {
  position: absolute;
  left: 80px;
  top: 30px;
  width: 50%; /* == 200px */
  height: 25%; /* == 40px */
  margin: 5%; /* == 20px */
  padding: 5%; /* == 20px */
  background: cyan;
}
```

See also

- `all` property
- `contain` property
- `aspect-ratio` property
- `box-sizing` property
- `min-content` and `max-content` size values
- Learn: sizing items in CSS
- Box model
- CSS box model module
- Layout modes
- Visual formatting models
- Block formatting context
- Stacking context
- Margin collapsing
- Initial, computed, used, and actual values
- Replaced elements
- Intrinsic size

CSS flow layout

Normal Flow, or Flow Layout, is the way that block and inline elements are displayed on a page before any changes are made to their layout. The flow is essentially a set of things that are all working together and know about each other in your layout. Once something is taken *out of flow* it works independently.

In normal flow, **inline** elements display in the inline direction, that is in the direction words are displayed in a sentence according to the Writing Mode of the document. **block** elements display one after the other, as paragraphs do in the Writing Mode of that document. In English therefore, inline elements display one after the other, starting on the left, and block elements start at the top and move down the page.

Basic Example

The following example demonstrates Block and Inline Level boxes. The two paragraph elements with a green border are Block Level, displaying one under the other.

The first sentence also includes a span element with a blue background. This is inline level and therefore displays in place in the sentence.

```
<div class="box">
  <p>
    One <span>November</span> night in the year 1782, so the story runs, two
    brothers sat over their winter fire in the little French town of Annonay,
    watching the grey smoke-wreaths from the hearth curl up the wide chimney.
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

p {
  border: 2px solid green;
}
span {
  background-color: lightblue;
}
```

See also

- Block and inline layout in normal flow
- In flow and out of flow
- Formatting contexts explained
- Flow layout and writing modes
- Flow layout and overflow

Flow layout and overflow

When there is more content than can fit into a container, an overflow situation occurs. Understanding how overflow behaves is important in dealing with any element with a constrained size in CSS. This guide explains how overflow works when working with normal flow. The HTML is the same in each example, so it's visible in the first section, and hidden in others for brevity.

What is overflow?

Giving an element a fixed height and width, then adding significant content to the box, creates a basic overflow example:

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
</div>
<p>
  Their names were Stephen and Joseph Montgolfier. They were papermakers by
```

```
  trade, and were noted as possessing thoughtful minds and a deep interest in
  all scientific knowledge and new discovery.
</p>
<p>
  Before that night—a memorable night, as it was to prove—hundreds of millions
  of people had watched the rising smoke-wreaths of their fires without drawing
  any special inspiration from the fact.
</p>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.box {
  width: 300px;
  height: 100px;
  border: 5px solid rebeccapurple;
  padding: 10px;
}
```

The content goes into the box. Once it fills the box, it continues to overflow in a visible way, displaying content outside the box, potentially displaying under subsequent content. The property that controls how overflow behaves is the `overflow` property which has an initial value of `visible`. This is why we can see the overflow content.

Controlling overflow

There are other values that control how overflow content behaves. To hide overflowing content use a value of `hidden`. This may cause some of your content to not be visible.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
</div>
<p>
  Their names were Stephen and Joseph Montgolfier. They were papermakers by
  trade, and were noted as possessing thoughtful minds and a deep interest in
  all scientific knowledge and new discovery.
</p>
<p>
  Before that night—a memorable night, as it was to prove—hundreds of millions
  of people had watched the rising smoke-wreaths of their fires without drawing
  any special inspiration from the fact.
</p>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  width: 300px;
  height: 100px;
  border: 5px solid rebeccapurple;
  padding: 10px;
  overflow: hidden;
}
```

Using a value of `scroll` contains the content in its box and add scrollbars to enable viewing it. Scrollbars will be added even if the content fits in the box.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
</div>
<p>
  Their names were Stephen and Joseph Montgolfier; they were papermakers by
  trade and were noted as possessing thoughtful minds and a deep interest in all
  scientific knowledge and new discoveries.
</p>
<p>
  Before that night—a memorable night, as it was to prove—hundreds of millions
  of people had watched the rising smoke-wreaths of their fires without drawing
  any special inspiration from the fact.
</p>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  width: 300px;
  height: 100px;
  border: 5px solid rebeccapurple;
  padding: 10px;
  overflow: scroll;
}
```

Using a value of `auto` will display the content with no scrollbars if the content fits inside the box. If it doesn't fit then scrollbars will be added. Comparing the next example, you should see `overflow: scroll` example above has horizontal and vertical scrollbars even though it only needs vertical scrolling. The `auto` example below only adds the scrollbar in the direction we need to scroll.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
</div>
<p>
  Their names were Stephen and Joseph Montgolfier, they were papermakers by
```

```
  trade, and were noted as possessing thoughtful minds and a deep interest in
  all scientific knowledge and new discovery.
</p>
<p>
  Before that night—a memorable night, as it was to prove—hundreds of millions
  of people had watched the rising smoke-wreaths of their fires without drawing
  any special inspiration from the fact.
</p>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  width: 300px;
  height: 100px;
  border: 5px solid rebeccapurple;
  padding: 10px;
  overflow: auto;
}
```

As we have already learned, using any of these values, other than the default of `visible`, will create a new block formatting context.

`overflow: clip` acts like `overflow: hidden`, but it does not allow programmatic scrolling; the box becomes non-scrollable. It also does not create a block formatting context.

The overflow property is in reality a shorthand for the `overflow-x` and `overflow-y` properties. If you specify only one value for overflow, this value is used for both axes. However, you can specify both values in which case the first is used for `overflow-x` and therefore the horizontal direction, and the second for `overflow-y` and the vertical direction. In the below example, I have only specified `overflow-y: scroll` so we do not get the unwanted horizontal scrollbar.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
</div>
<p>
  Their names were Stephen and Joseph Montgolfier, they were papermakers by
  trade, and were noted as possessing thoughtful minds and a deep interest in
  all scientific knowledge and new discovery.
</p>
<p>
  Before that night—a memorable night, as it was to prove—hundreds of millions
  of people had watched the rising smoke-wreaths of their fires without drawing
  any special inspiration from the fact.
</p>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  width: 300px;
  height: 100px;
  border: 5px solid rebeccapurple;
  padding: 10px;
  overflow-y: scroll;
}
```

Flow Relative Properties

In the guide to Writing Modes and Flow Layout, we looked at the `block-size` and `inline-size` properties, which make more sense when working with different writing modes than tying our layout to the physical dimensions of the screen. The CSS overflow module also includes flow relative properties for overflow - `overflow-block` and `overflow-inline`. These correspond to `overflow-x` and `overflow-y` but the mapping depends on the writing mode of the document.

Indicating Overflow

In the CSS overflow module, there are some properties that can help improve the way content looks in an overflow situation.

Inline-Axis Overflow

The `text-overflow` property deals with text overflowing in the inline direction. It takes one of two values `clip`, in which case content is clipped when it overflows, this is the initial value and therefore the default behavior. We also have `ellipsis` which renders an ellipsis, which may be replaced with a better character for the language or writing mode in use.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>

  <p>
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.box {
  width: 300px;
  border: 5px solid rebeccapurple;
  padding: 10px;
}
```

```
.box p {
  white-space: nowrap;
  text-overflow: ellipsis;
  overflow: hidden;
}
```

Block-Axis Overflow

The Overflow Module Level 4 specification adds a `block-ellipsis` property (previously called `block-overflow`). This property enables adding an ellipsis (or custom strings) when text overflows in the block dimension, although there is no browser support for this at the time of writing.

This is useful in the situation where you have a listing of articles, for example, and you display the listings in fixed height boxes which only take a limited amount of text. It may not be obvious to the reader that there is more content to click through to when clicking the box or the title. An ellipsis indicates clearly the fact there is more content. The specification would allow a string of content or a regular ellipsis to be inserted.

Summary

Whether you are in continuous media on the web or in a Paged Media format such as print or EPUB, understanding how content overflows is useful when dealing with any layout method. Understanding how overflow works in normal flow enables a greater understanding of the implications of overflow content in layout methods such as grid and flexbox.

See also

- Overflowing content guide
- CSS overflow module
- CSS containment module

Flow layout and writing modes

The CSS 2.1 specification, which details how normal flow behaves, assumes a horizontal writing mode. Layout properties should work in the same way in vertical writing modes. In this guide, we look at how flow layout behaves when used with different document writing modes.

This is not a comprehensive guide to the use of writing modes in CSS, the aim here is to document the areas where flow layout interacts with writing modes in possibly unanticipated ways. The See also section provides links to more writing modes resources.

Writing modes specification

The CSS Writing Modes Level 3 Specification defines the impact a change the document writing mode has on flow layout. In the writing modes introduction, the specification says,

> "A writing mode in CSS is determined by the `writing-mode`, `direction`, and `text-orientation` properties. It is defined primarily in terms of its inline base direction and block flow direction."

The specification defines the *inline base direction* as the direction in which content is ordered on a line. This defines the start and end of the inline direction. The start is where sentences start and the end is where a line of text ends before it would begin to wrap onto a new line.

The *block flow direction* is the direction in which boxes, for example paragraphs, stack in that writing mode. The CSS `writing-mode` property controls the block flow direction. If you want to change your page, or part of your page, to `vertical-lr` then you can set `writing-mode: vertical-lr` on the element and this will change the direction of the blocks and therefore the inline direction as well.

While certain languages will use a particular writing mode or text direction, we can also use these properties for creative effect, such as running a heading vertically.

```
<div class="box">
  <h1>A heading</h1>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
h1 {
  writing-mode: vertical-lr;
  float: left;
}
```

Block flow direction

The `writing-mode` property accepts the values `horizontal-tb`, `vertical-rl` and `vertical-lr`. These values control the direction that blocks flow on the page. The initial value is `horizontal-tb`, which is a top to bottom block flow direction with a horizontal inline direction. Left to right languages, such as English, and Right to left languages, such as Arabic, are all `horizontal-tb`.

The following example shows blocks using the initial `horizontal-tb` value explicitly:

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  writing-mode: horizontal-tb;
}
```

Compare this with the value `vertical-rl`, which gives you a right-to-left block flow direction with a vertical inline direction, as shown in the next example.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  writing-mode: vertical-rl;
}
```

The final example demonstrates the third possible value for `writing-mode` — `vertical-lr`. This gives you a left-to-right block flow direction and a vertical inline direction.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  writing-mode: vertical-lr;
}
```

Nested writing modes

When a nested box is assigned a different writing mode to its parent, then an inline level box will display as if it has `display: inline-block`.

```
<div class="box">
  <p>
    One <span>November</span> night in the year 1782, so the story runs, two
    brothers sat over their winter fire in the little French town of Annonay,
    watching the grey smoke-wreaths from the hearth curl up the wide chimney.
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  writing-mode: vertical-rl;
}
.box span {
  writing-mode: horizontal-tb;
  padding: 10px;
  border: 1px solid rebeccapurple;
}
```

A block-level box will establish a new block formatting context (BFC), meaning that if its inner display type would be `flow`, it will get a computed display type of `flow-root`. This is shown in the next example where the box which displays as `horizontal-tb` contains a float which is contained due to its parent establishing a new BFC.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>

  <div>
    <div class="float"></div>
    This box should establish a new BFC.
  </div>

  <p>
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  writing-mode: vertical-rl;
}
.box > div {
  writing-mode: horizontal-tb;
  padding: 10px;
  border: 1px solid rebeccapurple;
}
.float {
  width: 100px;
  height: 150px;
  background-color: rebeccapurple;
  float: left;
}
```

Replaced elements

Replaced elements such as images will not change their orientation based on the `writing-mode` property. However, replaced elements such as form controls which include text, should match the writing mode in use.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>

  <img
    alt="a colorful hot air balloon against a clear sky"
    src="https://mdn.github.io/shared-assets/images/examples/balloon.jpg" />

  <p>
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  writing-mode: vertical-rl;
}
```

Logical properties and values

Once you are working in writing modes other than `horizontal-tb` many of the properties and values that are mapped to the physical dimensions of the screen seem strange. For example, if you give a box a width of 100px, in `horizontal-tb` that would control the size in the inline direction. In `vertical-lr` it controls the size in the block direction because it does not rotate with the text.

```
<div class="box">
  <div class="box1">Box 1</div>
  <div class="box2">Box 2</div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box1 {
  writing-mode: horizontal-tb;
  border: 5px solid rebeccapurple;
  width: 100px;
  margin: 10px;
}
.box2 {
  writing-mode: vertical-lr;
  border: 5px solid rebeccapurple;
  width: 100px;
  margin: 10px;
}
```

Therefore, we have the `block-size` and `inline-size` properties. If we give our block an `inline-size` of 100px, it doesn't matter whether we are in a horizontal or vertical writing mode; `inline-size` will always mean the size in the inline direction.

```
<div class="box">
  <div class="box1">Box 1</div>
  <div class="box2">Box 2</div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box1 {
  writing-mode: horizontal-tb;
  border: 5px solid rebeccapurple;
  inline-size: 100px;
  margin: 10px;
}
.box2 {
  writing-mode: vertical-lr;
  border: 5px solid rebeccapurple;
  inline-size: 100px;
  margin: 10px;
}
```

The CSS logical properties and values module includes logical versions of the properties that control margins, padding and borders as well as other mappings for things that we have typically used physical directions to specify.

Summary

In most cases, flow layout works as expected when changing the writing mode of whole documents or individual sections. This can be used to properly typeset vertical languages or for creative reasons. With CSS logical properties and values, vertical writing mode sizing can be based on an element's inline and block size. This is useful for creating components that work in different writing modes.

See also

- Writing Modes
- Writing modes and CSS layout on Smashing Magazine (2019)
- CSS writing modes on 24ways.org (2016)

Introduction to formatting contexts

This article introduces the concept of formatting contexts, of which there are several types, including block formatting contexts, inline formatting contexts, and flex formatting contexts. The basics of how they behave and how you can make use of these behaviors are also introduced.

Everything on a page is part of a **formatting context**, or an area which has been defined to lay out content in a particular way. A **block formatting context** (BFC) will lay child elements out according to block layout rules, a **flex formatting context** will lay its children out as flex items, etc. Each formatting context has specific rules about how layout behaves when in that context.

Block formatting contexts

The outermost element in a document that uses block layout rules establishes the first, or **initial block formatting context**. This means that every element inside the `<html>` element's block is laid out according to normal flow following the rules for block and inline layout. Elements participating in a BFC use the rules outlined by the CSS Box Model, which defines how an element's margins, borders, and padding interact with other blocks in the same context.

Creating a new block formatting context

The `<html>` element is not the only element capable of creating a block formatting context. Any block-level element can be made to create a BFC by the application of certain CSS properties.

A new BFC is created in the following situations:

- elements made to float using `float`
- absolutely positioned elements
- elements with `display: inline-block`
- table cells or elements with `display: table-cell`, including anonymous table cells created when using the `display: table-*` properties
- table captions or elements with `display: table-caption`
- block elements where `overflow` has a value other than `visible`
- elements with `display: flow-root` or `display: flow-root list-item`
- elements with `contain: layout`, `content`, or `strict`
- query containers (elements where `container-type` isn't `normal`)
- flex items
- grid items
- multicol containers
- elements with `column-span` set to `all`

This is useful because a new BFC will behave much like the outermost document in that it becomes a mini-layout inside the main layout. A BFC contains everything inside it, `float` and `clear` only apply to items inside the same formatting context, and margins only collapse between elements in the same formatting context.

BFC creation examples

Let's have a look at a couple of these in order to see the effect creating a new BFC.

In the example below, we have a floated element inside a `<div>` with a border applied. The content of that `<div>` has floated alongside the floated element. As the content of the float is taller than the content alongside it, the border of the `<div>` now runs through the float. As explained in the guide to in-flow and out of flow elements, the float has been taken out of flow so the background and border of the div only contain the content and not the float.

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>I am content inside the container.</p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
}

.float {
  float: left;
  width: 200px;
  height: 100px;
  background-color: white;
  border: 1px solid black;
  padding: 10px;
}
```

Creating a new BFC would contain the float. A typical way to do this in the past has been to set `overflow: auto` or set other values than the initial value of `overflow: visible`.

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>I am content inside the container.</p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
  overflow: auto;
}
```

```
.float {
  float: left;
  width: 200px;
  height: 150px;
  background-color: white;
  border: 1px solid black;
  padding: 10px;
}
```

Setting `overflow: auto` created a new BFC containing the float. Our `<div>` now becomes a mini-layout inside our layout. Any child element will be contained inside it.

The problem with using `overflow` to create a new BFC is that the `overflow` property is meant for telling the browser how you wish to deal with overflowing content. There are some occasions in which you will find you get unwanted scrollbars or clipped shadows when you use this property purely to create a BFC. In addition, it is potentially not very readable for a future developer, as it may not be obvious why you used `overflow` for this purpose. If you do this, it would be a good idea to comment the code to explain.

Explicitly creating a BFC using display: flow-root

Using `display: flow-root` (or `display: flow-root list-item`) on the containing block will create a new BFC without any other potentially problematic side-effects.

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>I am content inside the container.</p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
  display: flow-root;
}
```

```
.float {
  float: left;
  width: 200px;
  height: 100px;
  background-color: white;
  border: 1px solid black;
  padding: 10px;
}
```

With `display: flow-root` on the `<div>`, everything inside that container participates in the block formatting context of that container, and floats will not poke out of the bottom of the element.

The name of the `flow-root` keyword refers to the fact that you're creating something that serves, in essence, like a new root element (like `<html>` does), given how the new context is created and its flow layout functions.

Inline formatting contexts

Inline formatting contexts exist inside other formatting contexts and can be thought of as the context of a paragraph. The paragraph creates an inline formatting context inside which such things as `<strong>`, `<a>`, or `<span>` elements are used on text.

The box model does not fully apply to items participating in an inline formatting context. In a horizontal writing mode line, horizontal padding, borders and margin will be applied to the element and push the text away left and right. However, margins above and below the element will not be applied. Vertical padding and borders will be applied but may overlap content above and below as, in the inline formatting context, the line boxes will not be pushed apart by padding and borders.

```
<p>
  Before that night—<strong>a memorable night</strong>, as it was to
  prove—hundreds of millions of people had watched the rising smoke-wreaths of
  their fires without drawing any special inspiration from the fact.
</p>
```

```
body {
  font: 1.2em sans-serif;
}
p {
  margin-top: 2em;
}
strong {
  margin: 20px;
  padding: 20px;
  border: 5px solid rebeccapurple;
}
```

Other formatting contexts

This guide covers flow layout and is therefore not referring to other possible formatting contexts. As such, it is useful to understand that creating any kind of formatting context will change the way elements inside that formatting context behave. This behavior is always described in the specification and also here on MDN.

Summary

In this guide, we have looked in more detail at the block and Inline formatting contexts and the important subject of creating a block formatting context (BFC). In the next guide, we will find out how normal flow interacts with different writing modes.

See also

- Block formatting context
- Visual formatting model
- CSS Box Model

In flow and out of flow

The previous guide explained block and inline layout in normal flow. All elements that are in flow will be laid out using this method.

Example of elements in flow

The following example contains a heading, a paragraph, a list, and a final paragraph that contains a `strong` element. The heading and paragraphs are block level, the `strong` element inline. The list, which is displayed using flexbox to arrange the items into a row, is also participating in block and inline layout - the container has an outside `display` type of `block`.

```
<div class="box">
  <h1>A heading</h1>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>

  <ul>
    <li>One</li>
    <li>Two</li>
    <li>Three</li>
  </ul>
  <p>
    Their names were <strong>Stephen and Joseph Montgolfier</strong>, they were
    papermakers by trade, and were noted as possessing thoughtful minds and a
    deep interest in all scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box > * {
  border: 1px solid green;
}

ul {
  display: flex;
  justify-content: space-around;
  list-style: none;
  margin: 0;
}
```

All of the elements are considered "in flow"; they appear on the page in the order that they are in the source.

Taking an item out of flow

All elements are in-flow apart from:

- floated items
- items with `position: absolute` (including `position: fixed` which acts in the same way)
- the root element (`html`)

Out-of-flow items create a new Block Formatting Context (BFC), and therefore, everything inside them can be seen as a mini layout, separate from the rest of the page. The root element, therefore, is out of flow, as the container for everything in our document, and establishes the Block Formatting Context for the document.

Floated items

In this example, there is a `div` and then two paragraphs. A background color has been added to the paragraphs, and the `div` is floated left. The `div` is now out of flow.

As a float it is first laid out according to where it would be in normal flow, then taken out of flow and moved to the left as far as possible.

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
p {
  background-color: #cccccc;
}

.float {
  float: left;
  font-weight: bold;
  width: 200px;
  border: 2px dotted black;
  padding: 10px;
}
```

You can see the background color of the following paragraph running underneath, it is only the line boxes of that paragraph that have been shortened to cause the effect of wrapping content around the float. The box of our paragraph is still displaying according to the rules of normal flow. This is why, to make space around a floated item, you must add a margin to the item, in order to push the line boxes away from it. You cannot apply anything to the following in-flow content to achieve that.

Absolute positioning

Giving an item `position: absolute` or `position: fixed` removes it from flow, and any space that it would have taken up is removed. In the next example I have three paragraph elements, the second element has `position: absolute`, with offset values of `top: 30px` and `right: 30px`. It has been removed from document flow.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
  <p class="abspos">
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  width: 70%;
}
p {
  border: 2px solid green;
}

.abspos {
  position: absolute;
  background-color: green;
  color: white;
  top: 30px;
  right: 30px;
  width: 400px;
}
```

Using `position: fixed` also removes the item from flow, however the offsets are based on the viewport rather than the containing block.

When taking an item out of flow with positioning, you will need to manage the possibility of content overlapping. Out of flow essentially means that the other elements on your page no longer know that element exists so will not respond to it.

Relative positioning and flow

If you give an item relative positioning with `position: relative`, it remains in flow. However, you are then able to use the offset values to push it around. The space that it would have been placed in normal flow is reserved however, as you can see in the example below.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney.
  </p>
  <p class="relative">
    Their names were Stephen and Joseph Montgolfier, they were papermakers by
    trade, and were noted as possessing thoughtful minds and a deep interest in
    all scientific knowledge and new discovery.
  </p>
  <p>
    Before that night—a memorable night, as it was to prove—hundreds of millions
    of people had watched the rising smoke-wreaths of their fires without
    drawing any special inspiration from the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  width: 70%;
}
p {
  border: 2px solid green;
}

.relative {
  position: relative;
  background-color: green;
  color: white;
  bottom: 50px;
  left: 50px;
  width: 400px;
}
```

When you do anything to remove or shift an item from where it would be placed in normal flow, you can expect to need to do some managing of the content and the content around it to prevent overlaps. Whether that involves clearing floats, or ensuring that an element with `position: absolute` does not sit on top of some other content. For this reason methods which remove elements from being in-flow should be used with understanding of the effect that they have.

Summary

In this guide, we have explained how to take an element out of flow to achieve some very specific types of positioning. In the next guide we will look at a related issue, that of creating a Block Formatting Context, in Introduction to formatting contexts.

See also

- Learn: Positioning

Using the multi-keyword syntax with CSS display

The CSS display module defines a multi-keyword syntax for the CSS `display` property. This guide explains the multi-keyword syntax.

> **Note:** Multi-keyword syntax is also referred to as "two-value syntax" or "multi-value syntax."

What happens when we change the value of the display property?

One of the first things we learn about CSS is that some elements are block-level and some are inline-level. These are their outer display types. For example, an `<h1>` or a `<p>` are block-level by default, and a `<span>` is inline-level. Using the `display` property we can switch between block and inline. For example to make a heading inline we would use the following CSS:

```
h1 {
  display: inline;
}
```

The `display` property also lets us use CSS grid layout and Flexbox when `display: grid` or `display: flex` is set. The important concept to understand is that changing an element's `display` value can change the formatting context of its direct children. When you use `display: flex` or `display: grid`, the element's children become flex or grid items and respond to the properties in grid and flexbox specifications.

What grid and flexbox demonstrate, however, is that an element has both an **outer** and an **inner** display type. The outer display type describes whether the element is block-level or inline-level. The inner display type describes how the children of that box behave.

As an example, when we use `display: flex` we create a block-level container, with flex children. The children are described as participating in a flex formatting context. You can see this if you take a `<span>` — normally an inline-level element — and apply `display: flex` to it. The `<span>` becomes a block-level element. It behaves as block-level things do in relationship to other boxes in the layout. It's as if you had applied `display: block` to the span, however we also get the changed behavior of the children.

The live example below has a `<span>` with `display: flex` applied. It has become a block-level box taking up all available space in the inline direction. You can now use `justify-content: space-between` to put this space between the two flex items.

```
<span class="flex"> Some text <em>emphasized text</em> </span>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.flex {
  border: 5px solid #cccccc;
  display: flex;
  justify-content: space-between;
}
```

It's also possible to create inline flex containers. If you use the single value `inline-flex` you will have an inline-level box with flex children. The children behave in the same way as the flex children of a block-level container. The only thing that has changed is that the parent is now an inline-level box. It therefore behaves like other inline-level things, and doesn't take up the full width (or size in the inline dimension) that a block-level box does. This means that some following text could come up alongside the flex container.

```
<div class="flex">
  <div>One</div>
  <div>Two</div>
</div>
Text following the flex container.
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.flex > div {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.flex {
  border: 5px solid #cccccc;
  display: inline-flex;
}
```

The same is true when working with grid layout. Using `display: grid` will give you a block-level box, which creates a grid formatting context for the direct children. Using `display: inline-grid` will create an inline-level box, which creates a grid formatting context for the children.

Using the multi-keyword syntax

As you can see from the above explanation, the `display` property has considerable powers. In addition to indicating whether something is block-level or inline-level in relationship to other boxes on the page, it also indicates the formatting context inside the box it is applied to. To better describe this behavior, the `display` property allows for two values — an outer and inner value — to be set on it. The original single-value syntax is also valid.

This means that instead of setting `display: flex` to create a block-level box with flex children, we use `display: block flex`. Instead of `display: inline-flex` to create an inline-level box with flex children, we use `display: inline flex`. The example below demonstrates these values.

```
<h1>Multiple values for display</h1>

<div class="flex flex1">
  <div>Item One</div>
  <div>Item Two</div>
  <div>Item Three</div>
</div>

<p>The first example is a block element with flex children.</p>

<div class="flex flex2">
  <div>Item One</div>
  <div>Item Two</div>
  <div>Item Three</div>
</div>
The second example is an inline element with flex children.
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
```

```
.flex {
  border: 5px solid #cccccc;
  gap: 10px;
}

.flex > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.flex1 {
  display: block flex;
}

.flex2 {
  display: inline flex;
}
```

There are mappings for all of the existing values of `display`; the most common ones are listed in the table below. To see a full list take a look at the table found in the `display` property specification.

Single value	Multi value
block	block flow
flow-root	block flow-root
inline	inline flow
inline-block	inline flow-root
flex	block flex
inline-flex	inline flex
grid	block grid
inline-grid	inline grid

display: block flow-root and display: inline flow-root

Regarding how this multi-value syntax helps clarify CSS layout, we can look at some values in the table above that might be less familiar to you. The multi-keyword `display: block flow-root` maps to a single value; `display: flow-root`. This value's only purpose is to create a new Block Formatting Context (BFC). A BFC ensures that everything inside your box stays inside, and things outside the box cannot intrude into it.

In the example below, two `<p>` elements, one inside a `<div>` demonstrate how display values affect formatting contexts. The first `<div>` element with the demo controls is hidden so we can focus on the elements that follow instead. The elements that we should focus on are the "parent", "child", and "sibling" `<div>` and `<p>` elements which you can differentiate by their IDs.

What's notable about this layout is that there is no content between the parent and child elements, and the child element has a top margin applied. You might expect the top margin to effectively push the child element down within the parent element, but what happens instead is something called *margin collapse*. In this case, the margin of the child element extends well above the parent's bounding box and pushes the parent element further down the page. This is easier to see if you inspect the box model of the child element in your browser's developer tools.

Change the selected option in the `<select>` element to see the effect of different `display` values. You can use any value with `flow-root` to create a new formatting context for the parent, making the child element margin relative to its parent's outer edge and avoiding the margin collapse. Changing between `display: flow-root` and `display: block flow-root` will achieve the same effect as the single-value `flow-root` keyword.

```
const parentDiv = document.getElementById("parent");
const siblingDiv = document.getElementById("sibling");
const displayTypeSelect = document.getElementById("displayType");

function changeDisplayType() {
  parentDiv.style.display = displayTypeSelect.value;
  siblingDiv.style.display = displayTypeSelect.value;
}

displayTypeSelect.addEventListener("change", changeDisplayType);
```

```
#controls {
  padding: 1rem;
  outline: 2px dashed black;
}
body {
  margin: 10px;
  font-family: sans-serif;
}
```

```
div,
p {
  outline: 2px solid black;
  background-color: cornflowerblue;
  display: block;
  margin-bottom: 2rem;
}

#parent {
  background-color: oldlace;
  min-height: 2rem;
}

#child {
  margin-top: 4rem;
  outline: 2px dashed red;
}

#sibling {
  background-color: lavender;
}
```

```
<div id="controls">
  <label for="displayType">display:</label>
  <select id="displayType">
    <option value="block">block</option>
```

```
    <option value="flow-root">flow-root</option>
    <option value="block flow-root">block flow-root</option>
    <option value="inline">inline</option>
    <option value="inline flow-root">inline flow-root</option>
  </select>
</div>
```

```
<div id="parent">
  <p id="child">The #child paragraph (nested in #parent).</p>
</div>
<p id="sibling">The #sibling paragraph (sibling of #parent).</p>
```

The `flow-root` value makes sense if you think about block and inline layout, which is sometimes called normal flow. Our HTML page creates a new formatting context (floats and margins cannot extend out from the boundaries) and our content lays out in normal flow, using block and inline layout, unless we change the value of `display` to use some other formatting context. Creating a grid or flex container also creates a new formatting context (a grid or flex formatting context, respectively.) These also contain everything inside them. However, if you want to contain floats and margins but continue using block and inline layout, you can create a new flow root, and start over with block and inline layout. From that point downwards everything is contained inside the new flow root.

This is why `display: flow-root` can be written using the multi-keyword syntax `display: block flow-root`. You are creating a block formatting context, with a block-level box and children participating in normal flow. What about the matched pair `display: inline flow-root`? This is the current way of describing `display: inline-block`.

The value `display: inline-block` has been around since the early days of CSS. The reason we tend to use it is to allow padding to push inline items away from an element, when creating navigation items for example, or when wanting to add a background with padding to an inline element as in the example below.

```
<p>
  This paragraph has a span <span class="inline-block">with padding</span> it is
  an inline-block so the padding is contained and pushes the other line boxes
  away.
</p>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
p {
  border: 2px dashed;
  width: 300px;
}
.inline-block {
  background-color: rgb(0 0 0 / 0.4);
  color: white;
  padding: 10px;
  display: inline-block;
}
```

An element with `display: inline-block` however, will also contain floats. It contains everything inside the inline-level box. Therefore `display: inline-block` does exactly what `display: flow-root` does, but with an inline-level, rather than a block-level box. The two-value syntax accurately describes what is happening with this value. In the example above, you can change `display: inline-block` to `display: inline flow-root` and get the same result.

What about the old values of display?

The single values of `display` are described in the specification as legacy values, and currently you gain no benefit from using the multi-keyword versions, as there is a direct mapping for each multi-keyword version to a legacy version, as demonstrated in the table above.

To deal with single values of `display` the specification explains what to do if only the outer value of `block` or `inline` is used:

> "If a `<display-outside>` value is specified but `<display-inside>` is omitted, the element's inner display type defaults to flow."

This means that the behavior is exactly as it is in a single value world. If you specify `display: block` or `display: inline`, that changes the outer display value of the box but any children continue in normal flow. If only an inner value of `flex`, `grid`, or `flow-root` is specified then the specification explains that the outer value should be set to `block`:

> "If a `<display-inside>` value is specified but `<display-outside>` is omitted, the element's outer display type defaults to block—except for ruby, which defaults to inline."

Finally, we have some legacy pre-composed inline-level values of:

- `inline-block`
- `inline-table`
- `inline-flex`
- `inline-grid`

If a supporting browser comes across these as single values then it treats them the same as the multi-keyword versions:

- `inline flow-root`
- `inline table`
- `inline flex`
- `inline grid`

So all of the current situations are neatly covered, meaning that we maintain compatibility of existing and new sites that use the single values, while allowing the spec to evolve.

Visual formatting model

In CSS, the **visual formatting model** describes how user agents take the document tree, and process and display it for visual media. This includes continuous media such as a computer screen and paged media such as a book or document printed by browser print functions. Most of the information applies equally to continuous and paged media.

In the visual formatting model, each element in the document tree generates zero or more boxes according to the box model. The layout of these boxes is governed by:

- Box dimensions and type.
- Positioning scheme (normal flow, float, and absolute positioning).
- Relationships between elements in the document tree.
- External information (e.g., viewport size, intrinsic dimensions of images, etc.).

Much of the information about the visual formatting model is defined in CSS2, however, various CSS layout modules have expanded upon this information. When reading specifications you will often find references to the model as defined in CSS2, so an understanding of the model and the terms used to describe it in CSS2 is valuable when reading other layout specifications.

In this document we define the model and introduce some of the related terms and concepts, linking to more specific pages for further details.

The role of the viewport

In continuous media, the viewport is the viewing area of the browser window. User agents can change the layout of the page when the viewport size changes — for example, if you resize your window, or change the orientation of a mobile device.

If the viewport is smaller than the size of the document then the user agent needs to offer a way to scroll to the parts of the document that are not displayed. We most often see this as scrolling in the **block dimension** — vertically in a horizontal, top-to-bottom language. However, you might design something that requires scrolling in the **inline dimension** too.

Box generation

Box generation is the part of the CSS visual formatting model that creates boxes from the document's elements. Generated boxes are of different types, which affect their visual formatting. The type of the box generated depends on the value of the CSS `display` property.

Initially defined in CSS2, the `display` property was extended in the CSS display, CSS flexible box layout, CSS grid layout, and CSS ruby layout modules. In addition, some of the terminologies around the display have been updated and clarified in the years since CSS2.

CSS takes your source document and renders it onto a canvas. To do this, it generates an intermediary structure, the **box tree**, which represents the formatting structure of the rendered document. Each box in the box tree represents its corresponding element (or pseudo-element) in space and/or time on the canvas, while each text run in the box tree likewise represents the contents of its corresponding text nodes.

Then, for each element, CSS generates zero or more boxes as specified by that element's `display` property value.

> **Note:** Boxes are often referred to by their display type — e.g., a box generated by an element with `display: block` is called a "block box" or just a "block". Note however that block boxes, block-level boxes, and block containers are all subtly different; see the block boxes section below for more details.

The principal box

When an element generates one or more boxes, one of them is the **principal box**, which contains its descendant boxes and generated content in the box tree, and is also the box involved in any positioning scheme.

Some elements may generate additional boxes in addition to the principal box, for example `display: list-item` generates more than one box (e.g., a **principal block box** and a **child marker box**). And some values (such as `none` or `contents`) cause the element and/or its descendants to not generate any boxes at all.

Anonymous boxes

An **anonymous box** is created when there is not an HTML element to use for the box. This situation happens when, for example, you declare `display: flex` on a parent element, and directly inside there is a run of text not contained in another element. In order to fix the box tree, an anonymous box is created around that run of text. It will then behave as a flex item, however, it cannot be targeted and styled like a regular box because there is no element to target.

```
<div class="flex">
  I am wrapped in an anonymous box
  <p>I am in the paragraph</p>
  I am wrapped in an anonymous box.
</div>
```

```
body {
  font: 1.2em sans-serif;
  margin: 20px;
}

.flex {
  display: flex;
}

.flex > * {
  background-color: rebeccapurple;
  color: white;
}
```

The same thing happens when you have text runs interspersed with block elements. In the next example I have a string inside a `<div>`; in the middle of my string is a `<p>` element containing part of the text.

```
<div class="example">
  I am wrapped in an anonymous box
  <p>I am in the paragraph</p>
  I am wrapped in an anonymous box.
</div>
```

```
body {
  font: 1.2em sans-serif;
  margin: 20px;
}

.example > * {
  background-color: rebeccapurple;
  color: white;
}
```

The string is split into three boxes in the box tree. The part of the string before the paragraph element is wrapped in an anonymous box, then we have the `<p>`, which generates a box, and then another anonymous box.

Something to consider about these anonymous boxes is that they inherit styles from their direct parent, but you cannot change how they look by targeting the anonymous box. In my examples, I am using a direct child selector to target the children of the container. This does not change the anonymous boxes, as they are not "elements" as such.

Inline anonymous boxes are created when a string is split by an inline element, for example, a sentence that includes a section wrapped with `<em></em>`. This splits the sentence into three inline boxes — an anonymous inline box before the emphasized section, the section wrapped in the `<em>` element, then a final anonymous inline box. As with the anonymous block boxes, these anonymous inline boxes cannot be styled independently in the way the `<em>` can; they just inherit the styles of their container.

Other formatting contexts also create anonymous boxes. Grid layout behaves in the same way as the flexbox example above, turning strings of text into a grid item with an anonymous box. Multiple-column layout creates anonymous column boxes around the columns; these also cannot be styled or otherwise targeted. Table layout will add anonymous boxes to create a proper table structure — for example adding an anonymous table row — if there was no box with `display: table-row`.

Line boxes

Line boxes are the boxes that wrap each line of text. You can see the difference between line boxes and their containing block if you float an item and then follow it by a block that has a background color.

In the following example, the line boxes following the floated `<div>` are shortened to wrap around the float. The background of the box runs behind the float, as the floated item has been taken out of flow.

```
<div class="float"></div>
<p class="following">
  This text is following the float, the line boxes are shortened to make room
  for the float but the box of the element still takes position in normal flow.
</p>
```

```
body {
  font: 1.2em sans-serif;
  margin: 20px;
}

.float {
  float: left;
  width: 150px;
  height: 150px;
  background-color: rebeccapurple;
  margin: 20px;
}

.following {
  background-color: #cccccc;
}
```

Positioning schemes and in-flow and out-of-flow elements

In CSS, a box may be laid out according to three positioning schemes — **normal flow**, **floats**, or **absolute positioning**.

Normal flow

In CSS, the normal flow includes block-level formatting of block boxes, inline-level formatting of inline boxes, and also includes relative and sticky positioning of block-level and inline-level boxes.

Read more about flow layout in CSS.

Floats

In the float model, a box is first laid out according to the normal flow, then taken out of the flow and positioned, typically to the left or right. Content may flow along the side of a float.

Find out more about floats.

Absolute positioning

In the absolute positioning model (which also includes `fixed` positioning), a box is removed from the normal flow entirely and assigned a position relative to a containing block (which is the viewport in the case of fixed positioning) or to one or more anchor elements in CSS anchor positioning.

An element is called **out of flow** if it is floated, absolutely positioned, or is the root element. An element is called **in-flow** if it is not out of the flow.

Read about CSS positioned layout.

Formatting contexts and the display property

Boxes can be described as having an **outer display type**, which is `block` or `inline`. This outer display type refers to how the box behaves alongside other elements on the page.

Boxes also have an inner display type, dictating how their children behave. For normal block and inline layout, or normal flow, this display type is `flow`. This means that the child elements will also be either `block` or `inline`.

However, the inner display type might be something like `grid` or `flex`, in which case the direct children will display as a grid, or flex items. In such a case the element is described as creating a grid or flex formatting context. In many ways, this is similar to a block formatting context, however, the children behave as flex or grid items rather than items in normal flow.

The interactions between block-level and inline-level boxes are described in the `display` property reference.

In addition, the references for specific values of display explain how these formatting contexts work in terms of box layout.

- CSS grid layout module
- CSS flexible box layout module
- CSS multi-column layout module
- CSS table module
- CSS lists and counters module

Independent formatting contexts

Elements either participate in the formatting context of their containing block or establish an independent formatting context. A grid container, for example, establishes a new **grid formatting context** for its children.

Independent formatting contexts contain floats, and margins do not collapse across formatting context boundaries. Therefore, creating a new block formatting context can ensure that floats and margins remain inside a box. To do this, add `display: flow-root` to the box on which you wish to create a new block formatting context.

The following example shows the effect of `display: flow-root`. The box with the black background appears to wrap around the floated item and text. If you remove `display: flow-root`, the floated item will poke out of the bottom of the box as it is no longer contained.

```
<div class="container">
  <div class="item">Floated</div>
  <p>Text following the float.</p>
</div>
```

```
body {
  font: 1.2em sans-serif;
  margin: 20px;
}
.container {
  background-color: #333333;
  color: white;
}

.item {
  background-color: white;
  border: 1px solid #999999;
  color: #333333;
  width: 100px;
  height: 100px;
```

```
  padding: 10px;
}
```

```
.container {
  display: flow-root;
}

.item {
  margin: 10px;
  float: left;
}
```

Block boxes

In specifications, block boxes, block-level boxes, and block containers are all referred to as **block boxes** in certain places. These things are somewhat different and the term block box should only be used if there is no ambiguity.

Block containers

A **block container** either contains only inline-level boxes participating in an inline formatting context, or only block-level boxes participating in a block formatting context. For this reason, we see the behavior explained above, where anonymous boxes are introduced to ensure all of the items can participate in a block or inline formatting context. An element is a block container only if it contains block-level or inline-level boxes.

Inline-level and block-level boxes

These are the boxes contained inside the block container that are participating in inline or block layout, respectively.

Block boxes

A block box is a block-level box that is also a block container. As described in CSS `display`, a box can be a block-level box, but not also a block container (it might be a flex or grid container for example).

See also

- CSS syntax guide
- Comments
- Specificity
- Inheritance
- Stacking context
- Block formatting context
- Box model
- Layout modes
- Margin collapsing
- Replaced elements
- `VisualViewport` interface
- Scroll container

CSS easing functions

The **CSS easing functions** module defines easing functions, which provide a means to control the transformation of values. The functions defined include linear, cubic bezier, and step easing functions. These easing functions can be applied to animations and transitions.

Gradually increasing the speed at which an element moves can give the element a sense of weight as it appears to gather momentum. Easing functions enable you to control the rate of change. Cubic bezier easing functions can be used to produce intuitive user interface elements or convincing cartoon props that behave like their physical counterparts.

Alternatively, you may want an animation to move forward in distinct steps, to create robot-like movement, or the effect of a segmented wheel that rotates such that the segments always appear in the same position. For this, step easing functions can be used.

Reference

Functions

- `cubic-bezier()`
- `linear()`
- `steps()`

data types

- `<easing-function>`
- `<linear-easing-function>`
- `<cubic-bezier-easing-function>`
- `<step-easing-function>`

Terms and glossary definitions

- Bézier curve
- Interpolation

Guides

CSS value functions: easing functions

> The CSS statements that invoke special data processing or calculations to return a CSS value for a CSS property.

Related concepts

- CSS animations module
  - `animation-timing-function`
  - `animation` shorthand
- CSS transitions module
  - `transition-timing-function`
  - `transition` shorthand

See also

- CSS value functions

- `will-change` CSS property

CSS environment variables

The **CSS environment variables** module defines the concept of environment variables and the `env()` function. Environment variables work similarly to custom properties and the `var()` function, except that they are globally defined; they are global variables scoped to the entire document. They are user agent values, provided by the browser or operating system, you can access using the `env()` function enabling you to adapt your styles to the user's device or context.

Environment variables provide values that can be used on the page based on information the user agent has access to, such as the size of the titlebar, dynamic keyboard, and safe area insets. Safe area insets define a rectangle that is guaranteed to be visible on non-rectangular displays, based on its distance from the edges of the viewport. You can include the user-agent sizes in your styles, enabling you to modify the layout of essential content based on the available space inside the safe area rectangle, next to a visible dynamic keyboard, or adjacent to a visible title bar.

Reference

Functions

- `env()`

Data types

- `<environment-variable-name>`

Guides

Using environment variables

An overview of what environment variables are, browser-defined environment variables, and how to use the `env()` function.

Related concepts

- CSS media queries module
- custom properties
- `var()`
- `VirtualKeyboard` interface
- `display_override` manifest field Window Controls Overlay API and `WindowControlsOverlay` interface

See also

- CSS value functions guide
- CSS properties and values API module
- CSS custom properties for cascading variables module

Using environment variables

The CSS environment variables module introduces the concept of environment variables in CSS and defines the `env()` function to enable using environment variables. In this guide, we look at what environment variables are, the environment variables defined by the browser, and how to use environment variables with the `env()` function.

What are environment variables?

CSS environment variables are global variables; globally scoped to the entire document. They are defined by the user agent. Environment variables are special values provided by the browser or operating system that help your styles adapt to the user's device or context. They are accessed using the `env()` function.

Environment variables work similarly to custom properties and the `var()` function, but are globally defined and scoped. This means they are always scoped to the entire document, unlike custom properties, which are scoped to elements. In addition, environment variables are read-only, whereas custom properties are mutable.

Similar to custom properties, environment variables are case-sensitive. Unlike custom properties, which cannot be used outside of declarations, the `env()` function can be used in place of any part of a property value, or any part of a descriptor (for example, in Media query rules).

History

Apple first introduced environment variables in the iOS Safari browser to enable developers to optimize layouts for irregular device displays. Examples include those with notches and curved edges. The original `safe-area-inset-*` environment variables allow developers to place content in a safe area of the viewport regardless of what device or browser the user has.

Use cases

Common issues that can be solved by using environment variables include:

- Device notifications covering up sections of the app user interface.
- Handling viewport size changes when dynamic keyboards are shown and hidden.
- Positioning elements where the title bar would have been on progressive web apps (PWAs) once they are installed and ensuring that content stays clear of the window control buttons. This is especially an issue on desktop browsers.

Browser-defined environment variables

The CSS environment variables specification defines a few case-sensitive variables, including:

`preferred-text-scale`

The `preferred-text-scale` environment variable represents the user's preferred text scale factor. This is the adjustment made to the "default" font size of the operating system or user agent. On devices and browsers where `text-size-adjust` has an effect, this is the scale factor applied by `text-size-adjust: auto`. For example, if `text-size-adjust: auto` would cause the text size to double, then `env(preferred-text-scale)` would resolve to `2`.

`safe-area-inset-*`

The four safe area inset environment variables - `safe-area-inset-top`, `safe-area-inset-right`, `safe-area-inset-bottom`, and `safe-area-inset-left` - define a rectangular safe area by its top, right, bottom, and left insets from the edge of the viewport. It is safe to place content inside this area without having it cut off by the shape of a non-rectangular display. For rectangular, unobstructed viewports, such as regular desktop and laptop monitors, these four values are all equal to `0`. For non-rectangular displays — including rounded-corner devices with full-size screens and rounded or round-display smart watches — these four values, set by the user agent, form a rectangle such that all content inside the rectangle is visible and unobstructed.

`safe-area-max-inset-*`

The four safe area maximum inset environment variables - `safe-area-max-inset-top`, `safe-area-max-inset-right`, `safe-area-max-inset-bottom`, and `safe-area-max-inset-left` - each represent the static maximum value of their dynamic `safe-area-inset-*` variable counterparts. They represent the maximum value of their `safe-area-inset-*` counterpart when all dynamic user interface features are retracted. For example, on some platforms there may be a button bar displayed when scrolling up or down, changing the `safe-area-inset-*` values. While the `safe-area-inset-*` values change as the currently visible content area changes, the `safe-area-max-inset-*` values always remain the same.

`viewport-segment-*`

These variables are only relevant to devices that have multiple segments, such as foldable phones. The `viewport-segment-bottom`, `viewport-segment-left`, `viewport-segment-right`, and `viewport-segment-top` variables, along with the `viewport-segment-height` and `viewport-segment-width`, define the position and dimensions of logically separated regions of the viewport. These variables are only defined if the viewport is divided into at least two segments. They are used to place different parts of a UI comfortably into different segments of a multi-segment device and avoid having your content cut off by the fold.

Other specifications define additional environment variables.

The Window Controls Overlay API defines the `WindowControlsOverlay` interface, which exposes information about the geometry of the title bar area in progressive web applications (PWAs) installed on desktop devices. When using the `window-controls-overlay` display_override value, the following environment variables are defined:

`titlebar-area-*`

The `titlebar-area-x`, `titlebar-area-y`, `titlebar-area-width`, and `titlebar-area-height` variables define the area that would generally be occupied by the title bar in an installed web application running on a desktop environment. Use the `titlebar-area-*` variables to ensure content doesn't overlap with the window control buttons (that is, minimize, maximize, and close).

`keyboard-inset-*`

The `keyboard-inset-top`, `keyboard-inset-right`, `keyboard-inset-bottom`, `keyboard-inset-left`, `keyboard-inset-width`, and `keyboard-inset-height` variables provide information about the on-screen virtual keyboard's position and size, specifically its top, right, bottom, and left insets from the edge of the viewport (the width and height insets are calculated from the other insets). To learn more, see the VirtualKeyboard API.

You may have noticed that all the previous variable names include the physical terms of left, right, top, bottom, height, and width. Logical equivalents are not needed because the variable names refer to the physical properties of the device hardware rather than the displayed website.

The <code>env()</code> function

The `env()` function is used to insert the value of an environment variable into a CSS context. The `env()` function can be used in place of any part of a value in any property on any element, or any part of a value in any descriptor on any at-rule, including within custom property values. It can be used wherever a CSS value is allowed.

The basic syntax is as follows:

```
env( <environment-variable-name> )
env( <environment-variable-name>, <fallback-value> )
```

The function accepts a case-sensitive environment variable name and an optional, but generally recommended, fallback value.

```
line-height: env(preferred-text-scale, 2);
margin: env(safe-area-inset-top, 0) env(safe-area-inset-right, auto)
  env(safe-area-inset-bottom, 3em) env(safe-area-inset-left, auto);
```

The first argument is the name of the environment variable to be substituted. The argument after the comma, if provided, is the fallback value, which is used if the environment variable referenced in the first argument does not exist. In these examples, if the `preferred-text-scale` environment variable doesn't exist in a browser, the `line-height` will be set to `2`. And, if the browser doesn't have `safe-area-inset-*` values, the `margin` will be set to `margin: 0 auto 3em auto`.

The syntax of the fallback is similar to the custom properties syntax in that it allows for multiple commas. Anything between the first comma and the end of the function is considered the fallback value. However, if the property value or descriptor doesn't support commas, the value is not valid.

If a property or descriptor contains syntactically valid `env()` functions, it is assumed to be valid at parse time. It is only syntax-checked at computed-time, after the `env()` functions have been substituted with their browser-provided values. If the environment variable passed as the first parameter is not a recognized environment variable name, the fallback value is used. The fallback can be another environment variable, even with its own fallback. If no fallback was provided, the property or descriptor containing the `env()` function is invalid at computed-value time.

Using environment variables with <code>env()</code> example

We can use environment variables to ensure that a fixed app toolbar is not obscured by notifications appearing at the bottom of the device. In devices that display notifications at the bottom of the screen, the user agent will set the value of the `safe-area-inset-bottom` environment variable to be the distance from the top of what is blocking the viewport to the bottom of the viewport; in our example, this is likely the height of any visible notifications. On a rectangular desktop monitor, the `safe-area-inset-bottom` is generally `0`. We will use this value to create space at the bottom of the viewport for the notification to display without obscuring the content.

Our `<body>` has two children; the `<main>` contains our entire application other than the `<footer>` toolbar.

```
<body>
  <main>Application</main>
  <footer>Toolbar</footer>
</body>
```

The `<body>` is defined as a flex container that fills the height of the viewport. The `<main>` application is allowed to grow to fill any space not occupied by its sibling `<footer>`.

```
body {
  display: flex;
  flex-flow: column nowrap;
  height: 100vh;
}

main {
  flex: 1;
  padding: 1em;
  overflow-y: auto;
}
```

The `<footer>` is positioned to be stuck to the bottom of the viewport. The declaration `position: sticky` offsets the element relative to the `<body>` (its scrolling ancestor and containing block), based on the `bottom` value of `0`. We give the `<footer>` a `padding` value of `1em` on all four sides. We then add the `safe-area-inset-bottom` value to the `1em` of bottom padding, with a fallback of `1em`.

```
footer {
  position: sticky;
  bottom: 0;
  padding: 1em;
  padding-bottom: calc(1em + env(safe-area-inset-bottom, 1em));
}
```

Additional CSS is hidden for brevity.

```
main {
  background-color: palegoldenrod;
}
footer {
  background-color: black;
  color: white;
```

```
  border-top: 1px solid white;
  display: flex;
  justify-content: space-between;
}

footer::before,
footer::after {
  content: "Button" / "Fake button";
  padding: 3px 0.5em;
  background: white;
  color: black;
  border-radius: 3px;
}
```

The bottom padding of the footer will extend beyond `1em` on devices that have the `safe-area-inset-bottom` environment variable with a value greater than `0`. This CSS provides extra padding to grow the footer as needed, be it because of notifications, a device notch in the screen, or because the device doesn't have square corners.

In the future, we may see support for developer-defined environment variables, but this has not yet been defined or implemented.

See also

- `var()`
- `@media shape` descriptor
- CSS environment_variables module
- CSS round display module
- CSS custom properties for cascading variables module

CSS filter effects

The properties in the **CSS filter effects** module let you define a way of processing an element's rendering before the element is displayed in the document. Examples of such effects include blurring and changing the intensity of the color of an element.

Filter effects in action

Play with the various sliders to apply filter effects to the image below.

```
<article>
  <img
    src="https://mdn.github.io/shared-assets/images/examples/george_floyd_memorial_sm.jpg"
    width="600"
    height="400"
    alt="A colorful memorial honoring George Floyd and the Black Lives Matter movement" />
  <form>
    <fieldset>
      <legend>Select your filters</legend>
      <label>
        <input type="range" name="blur" value="0" min="0" max="1" step="0.1" />
        blur()
      </label>
      <label>
        <input
          type="range"
          name="brightness"
          value="1"
          min="0"
          max="2"
          step="0.1" />
        brightness()
      </label>
      <label>
        <input
          type="range"
          name="contrast"
          value="1"
          min="0"
          max="2"
          step="0.1" />
        contrast()
      </label>
      <label>
        <input
          type="range"
          name="dropShadow"
          value="0"
          min="-1"
          max="1"
          step="0.1" />
        drop-shadow()
      </label>
      <label>
        <input
          type="range"
          name="grayscale"
          value="0"
          min="0"
          max="1"
          step="0.1" />
        grayscale()
      </label>
      <label>
        <input
          type="range"
          name="hueRotate"
          value="0"
          min="-1"
          max="1"
          step="0.1" />
        hue-rotate()
      </label>
      <label>
        <input
```

```
          type="range"
          name="invert"
          value="0"
          min="0"
          max="1"
          step="0.1" />
        invert()
      </label>
      <label>
        <input
          type="range"
          name="opacity"
          value="1"
          min="0"
          max="1"
          step="0.1" />
        opacity()
      </label>
      <label>
        <input
          type="range"
          name="saturate"
          value="1"
          min="0"
          max="2"
          step="0.1" />
        saturate()
      </label>
      <label>
        <input type="range" name="sepia" value="0" min="0" max="1" step="0.1" />
        sepia()
      </label>
      <button type="reset">Reset</button>
    </fieldset>
  </form>
</article>

<pre><output></output></pre>

<p>Image by <cite>DigitalNomad</cite></p>
```

```
article {
  display: grid;
  grid-template-columns: minmax(200px, 800px) 15em;
  gap: 1rem;
  max-width: 100%;
}
label {
  display: block;
  font-family: sans-serif;
}
input {
  vertical-align: middle;
  margin-right: 0.25em;
  max-width: 50%;
}
output {
  white-space: normal;
  overflow-wrap: break-word;
  display: block;
  width: 100%;
}

img {
  margin: 1rem;
  width: 100%;
  object-fit: cover;
  overflow: hidden;
}
```

```
const image = document.querySelector("img");
const controls = document.querySelectorAll("input");
const output = document.querySelector("output");

for (control of controls) {
  control.addEventListener("change", () => {
    /* do function */
    changeCSS();
  });
}
document.querySelector("button").addEventListener("click", () => {
  setTimeout(() => {
    changeCSS();
  }, 50);
});

function changeCSS() {
  let currentFilter = "filter: ";
  for (const filter of [
    blur(),
    brightness(),
    contrast(),
    dropShadow(),
    grayscale(),
    hueRotate(),
    invert(),
    opacity(),
    saturate(),
    sepia(),
  ]) {
    currentFilter += filter;
  }
```

```
  currentFilter += ";";
  image.setAttribute("style", currentFilter);
  output.innerText = currentFilter;
}
function blur() {
  let blurValue = document.getElementsByName("blur")[0].value;
  return blurValue === "0" ? "" : `blur(${blurValue}rem) `;
}
function brightness() {
  let brightnessValue = document.getElementsByName("brightness")[0].value;
  return brightnessValue.toString() === "1"
    ? ""
    : `brightness(${brightnessValue}) `;
}
function contrast() {
  let contrastValue = document.getElementsByName("contrast")[0].value;
  return contrastValue === 1 ? "" : `contrast(${contrastValue}) `;
}
function dropShadow() {
  let dropShadowValue = document.getElementsByName("dropShadow")[0].value;
  return dropShadowValue === 0
    ? ""
    : `drop-shadow(${dropShadowValue}rem ${dropShadowValue}rem 0rem orange) `;
}
function grayscale() {
  let grayscaleValue = document.getElementsByName("grayscale")[0].value;
  return grayscaleValue === 0 ? "" : `grayscale(${grayscaleValue}) `;
}
function hueRotate() {
  let hueRotateValue = document.getElementsByName("hueRotate")[0].value;
  return hueRotateValue === 0 ? "" : `hue-rotate(${hueRotateValue}turn) `;
}
function invert() {
  let invertValue = document.getElementsByName("invert")[0].value;
  return invertValue === 0 ? "" : `invert(${invertValue}) `;
}
function opacity() {
  let opacityValue = document.getElementsByName("opacity")[0].value;
  return opacityValue === 1 ? "" : `opacity(${opacityValue}) `;
}
function saturate() {
  let saturateValue = document.getElementsByName("saturate")[0].value;
  return saturateValue === 1 ? "" : `saturate(${saturateValue}) `;
}
function sepia() {
  let sepiaValue = document.getElementsByName("sepia")[0].value;
  return sepiaValue === 0 ? "" : `sepia(${sepiaValue})`;
}
```

Reference

Properties

- `backdrop-filter`
- `filter`

Functions

- `blur()`
- `brightness()`
- `contrast()`
- `drop-shadow()`
- `grayscale()`
- `hue-rotate()`
- `invert()`
- `opacity()`
- `saturate()`
- `sepia()`

Guides

Using CSS filter effects

Overview of the concepts surrounding CSS filter effects, including properties, filter functions, and SVG filters, with an explanation of filter values, source order, and value interactions.

Related concepts

- `<image>` data type
- `<filter-function>` data type
- `background-image` CSS property
- `background-blend-mode` CSS property
- `mix-blend-mode` CSS property
- interpolation glossary term
- `color-interpolation-filters` SVG property

See also

- Properties in the CSS compositing and blending module enable blending an element's background layers together and blending an element with its container
- The SVG `<filter>` element and the SVG filter primitives: `<feSpotLight>`, `<feBlend>`, `<feColorMatrix>`, `<feComponentTransfer>`, `<feComposite>`, `<feConvolveMatrix>`, `<feDiffuseLighting>`, `<feDisplacementMap>`, `<feDropShadow>`, `<feFlood>`, `<feGaussianBlur>`, `<feImage>`, `<feMerge>`, `<feMorphology>`, `<feOffset>`, `<feSpecularLighting>`, `<feTile>`, `<feTurbulence>`

Using filter effects

Have you ever hovered over a black-and-white or sepia image and the full-color image came into view instantly? Have you ever encountered a background image with a small blurred-out section that makes the text on top more legible? In the past, these manipulations required image editing software, time, and additional HTTP requests.

Advantages of using CSS filter effects

The Filter effects module in CSS provides properties and functions that let you apply the visual effects described above without using Photoshop or sending extra HTTP requests. The only software required is the user's browser. Moreover, unlike pre-set image effects, CSS filter effects are responsive and animatable.

The CSS filter effects module provides the `filter` and `backdrop-filter` properties that you can use to impact the rendering of text, images, backgrounds, and borders, or any element on which you apply these properties. This module also defines the `<filter-function>` data type that lets you add graphical effects such as blurring or color shifting. Using the filter functions, you can not only alter the appearance of an element but also reference an SVG filter using a filter that you create.

Filter effect properties

The following two filter properties of the CSS filter effects module enable you to apply zero, one, or more graphical effects to an element:

- Using the `filter` property, you can apply filter effects such as blur, drop-shadow, and sepia to an element before the element is rendered. The filters effects are applied directly on the element, including the element's contents, borders, and padding.
- Using the `backdrop-filter` property, you can apply graphical effects to the area behind an element (the element's "backdrop"). The `backdrop-filter` property is often used to make the foreground content more legible, especially when the larger area on which the content is placed otherwise does not provide enough contrast for the content. The filter effects are applied only to the background of the element and not to the element's content.

The `filter` and `backdrop-filter` properties accept a space-separated list of filters, which are applied in the order declared.

Filter functions

The CSS filter effects module provides 10 `<filter-function>` functions, as well as the ability to define an almost endless array of effects using SVG filters applied via a `url()` reference.

The following table lists the 10 filter functions, along with their value types, the minimum valid value if applicable, the largest value that creates an effect, and the initial value used for interpolation.

Filter function	Parameter type	Min value	Max effect	Default value (no effect)
`blur()`	`<length>`	`0`		`0`
`brightness()`	`<number>` or `<percentage>`	`0`		`1` or `100%`
`contrast()`	`<length>`	`0`		`1` or `100%`
`drop-shadow()`	`<shadow>`			`0 0 0 currentColor`
`grayscale()`	`<number>` or `<percentage>`	`0`	`100%`	`0` or `0%`
`hue-rotate()`	`<angle>`			`0deg`
`invert()`	`<number>` or `<percentage>`	`0`	`100%`	`0` or `0%`
`opacity()`	`<number>` or `<percentage>`	`0`	`100%`	`1` or `100%`
`saturate()`	`<number>` or `<percentage>`	`0`	`100%`	`100%`
`sepia()`	`<number>` or `<percentage>`	`0`	`100%`	`0%`

The minimum value allowed is included for filter functions that have a minimum value. Including a value less than the minimum value for any filter function invalidates the entire property declaration, not just the offending filter function in the comma-separated list.

The maximum effect value can be exceeded. Including a value greater than the listed maximum value is valid, but it does not increase the effect over the listed maximum value. In other words, the effect on the element will look the same as when the maximum effect value is set. For example, setting `sepia(400%)` in the sepia example will produce the same effect as `sepia(100%)`, the maximum value.

The default value is a value that creates no effect. While these values create no effect, they provide the initial interpolation values and offer an example of how the value can be set. These default values provide a gauge between the minimum value allowed and the maximum effect value.

Applying filter effects

The `filter` and `backdrop-filter` properties accept a filter function list, which may contain one or more `<filter-function>`s, the default keyword `none`, or an SVG filter as a `url()` value.

Applying sepia filter effect

If you hover over the sepia image below, you'll see the full-color image come into view instantly.

The image is set to be sepia by specifying the value of the `filter` property as the `sepia()` filter function. The filter is removed on `:hover` and `:focus` by setting `filter: none`.

```
<img tabindex="0" alt="Four trans-people, circa 1912" src="activists.jpg" />
```

```
img {
  filter: sepia(100%);
}
img:hover,
img:focus {
  filter: none;
}
```

In the `<img>` element, `tabindex` is set to `0` to enable focus without altering the tabbing order for keyboard users because `<img>` is not an interactive element.

```
img {
  max-width: 100%;
  height: 100%;
}
```

Applying filter effects to other elements

While generally applied to images, the `filter` and `backdrop-filter` properties can be applied to any element or pseudo-element.

In this example, a glow effect is added using a `drop-shadow()` filter with a `3px` blur and `0` offset.

```
h1 {
  color: midnightblue;
  filter: drop-shadow(0 0 3px magenta);
}
```

```
h1 {
  font-family: sans-serif;
  font-size: 2rem;
}
```

```
<h1>Glow created with CSS filter</h1>
```

Applying multiple filters

While the sepia `filter` example included only a single filter function, you can set multiple filters. The `filter` and `backdrop-filter` properties accept a space-separated list of filters, which are applied in the order declared.

This example applies two filters — `hue-rotate()` and `blur()` — via the `backdrop-filter` property. The backdrop, the area behind the `<p>` element, has a color shift and a blur applied.

```
.container {
  background: url("/shared-assets/images/examples/listen_to_black_women.jpg")
    no-repeat left / contain goldenrod;
}
p {
  backdrop-filter: hue-rotate(240deg) blur(5px);
  background-color: rgb(255 255 255 / 10%);
  text-shadow: 2px 2px black;
}
```

```
.container {
  padding: 3rem;
  width: 30rem;
}
p {
  padding: 0.5rem;
  color: white;
  font-size: 2rem;
  font-family: sans-serif;
}
```

```
<div class="container">
  <p>
    Text on images can be illegible and inaccessible even with a drop shadow.
  </p>
</div>
```

Applying repeated filters

As filters are applied in sequential order, you can use filter functions more than once. In this example, the `drop-shadow()` filter has been used four times, each time with a different `<shadow>` value.

```
<img src="mandala.svg" alt="Colorful mandala" role="img" />
<img src="mandala.svg" alt="Plain mandala" role="img" />
```

```
img {
  width: 49%;
}
```

```
img {
  filter: drop-shadow(2px 2px 0 magenta) drop-shadow(-2px -2px 0 royalblue)
    drop-shadow(2px 2px 0 lime) drop-shadow(-2px -2px 0 darkorange);
}
img + img {
  filter: none;
}
```

In the first Mandala example, four drop shadows are applied to a line-drawn SVG. The same SVG, with the filter removed with `filter: none`, is included for comparison.

Specifying filter function order

When creating filter effects, the `filter` or `backdrop-filter` property is provided a space-separated list of filters. These filter effects are applied in the order in which they appear.

In this example, both `magenta` drop shadow and `180deg` hue rotation are applied on the level-one heading. The example shows the effect when these filters are applied in different orders.

```
h1 {
  color: midnightblue;
}
#hueFirst {
  filter: hue-rotate(180deg) drop-shadow(3px 3px magenta);
}
#shadowFirst {
  filter: drop-shadow(3px 3px magenta) hue-rotate(180deg);
}
```

```
h1 {
  font-family: sans-serif;
  font-size: 2rem;
}
```

```
<h1 id="hueFirst">Hue change happens before drop shadow.</h1>
<h1 id="shadowFirst">Drop shadow applied before hue change.</h1>
<h1>No filter effects applied.</h1>
```

The same filters are applied to both the lines of text but in a different order. In the first line, the hue of the text is altered before the shadow is applied, so the shadow is `magenta`. In the second line, the drop shadow is added to the dark blue text, and then the hue of both the text and the shadow are altered.

No filter effect is applied to the third line to show the original effect as a comparison. So the third line stays as `midnightblue` or `#191970`. The `hue-rotate(180deg)` filter changes the text in the first two lines to `#252500`.

> **Note:** The hexadecimal rgb color `#191970` is equal to `hsl(240deg 63.5% 26.9%)`, while `#252500` is `hsl(60deg 100% 7.3%)`. The color rotation takes place in the sRGB color space, which is why the hue has been

> changed as expected while not maintaining the same values for saturation and lightness.

Using SVG filters

In addition to the 10 defined `<filter-function>`s, the CSS filter effects support `url()`, with the parameter being an SVG filter, which may be embedded in an internal or external SVG file.

A single SVG can be used to define several filters, each with an `id`:

```
<svg role="none">
  <defs>
    <filter id="blur1">
      <feGaussianBlur stdDeviation="1" edgeMode="duplicate" />
    </filter>
    <filter id="blur3">
      <feGaussianBlur stdDeviation="3" edgeMode="duplicate" />
    </filter>
    <filter id="hue-rotate90">
      <feColorMatrix type="hueRotate" values="90" />
    </filter>
  </defs>
</svg>
```

The filter's `id` is referenced in the `url()` for both inline and external SVGs:

```
filter: url("#blur3");
filter: url("https://example.com/svg/filters.svg#blur3");
```

Blurring an image

Just like the `blur()` filter function applies a Gaussian blur to the elements on which it is applied, the SVG `<feGaussianBlur>` filter element can also be used to blur content.

In both the cases, the blur radius value, specified as a `<length>` in CSS and as a pixel equivalent `<number>` in SVG, defines the value of the standard deviation to the Gaussian function. In other words, it defines the number of pixels on the screen that blend into each other; a larger value creates more blur.

The `<filter>`'s `stdDeviation` attribute accepts up to two values enabling creating more complex blur values. To create an equivalent blur, we include one value for `stdDeviation`:

```
<table>
  <thead>
    <tr>
      <th>CSS example</th>
      <th>SVG example</th>
      <th>Original image</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <img
          class="filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag"
          class="svgFilter" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

```
<svg role="img" aria-label="Flag">
  <filter id="blur">
    <feGaussianBlur stdDeviation="3.5" edgeMode="duplicate" />
  </filter>
  <image
    xlink:href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    filter="url(#blur)" />
</svg>
```

The SVG `url()` filter value can be included as the value of the SVG `<image>` element's `filter` attribute or as part of the value of the CSS `filter` and `backdrop-filter` properties.

```
th,
td {
  padding: 5px;
}
```

```
.filter {
  filter: blur(3.5px);
}
.svgFilter {
  filter: url("#blur");
}
```

See also

- `mask`
- `background-blend-mode`, `mix-blend-mode`
- CSS filter effects
- SVG `<filter>` element, SVG `filter` attribute in SVG
- Applying SVG effects to HTML content

CSS flexible box layout

The **CSS flexible box layout** module defines a CSS box model optimized for user interface design, and the layout of items in one dimension. In the flex layout model, the children of a flex container can be laid out in any direction, and can "flex" their sizes, either growing to fill unused space or shrinking to avoid overflowing the parent. Both horizontal and vertical alignment of the children can be easily manipulated.

Flexible box layout in action

In the following example, a container has been set to `display: flex`, which means that the three child items become flex items. The value of `justify-content` has been set to `space-between` in order to space the items out evenly on the main axis. An equal amount of space is placed between each item with the left and right items being flush with the edges of the flex container. You can also see that the items are stretching on the cross axis, due to the default value of `align-items` being `stretch`. The items stretch to the height of the flex container, making them each appear as tall as the tallest item.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
</div>
```

```
body {
  font-family: sans-serif;
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  justify-content: space-between;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 1em;
}
```

Reference

Properties

- `align-content`
- `align-items`
- `align-self`
- `flex`
- `flex-basis`
- `flex-direction`
- `flex-flow`
- `flex-grow`
- `flex-line-count`
- `flex-shrink`
- `flex-wrap`
- `justify-content`

Glossary terms

- Flexbox
- Flex container
- Flex item
- Main axis
- Cross axis
- Flex

Guides

Basic concepts of flexbox

An overview of the features of Flexbox.

Relationship of flexbox to other layout methods

How Flexbox relates to other layout methods and other CSS specifications.

Aligning items in a flex container

How the box alignment properties work with Flexbox.

Ordering flex items

Explaining the different ways to change the order and direction of items, and covering the potential issues in doing so.

Controlling ratios of flex items along the main axis

Explaining the flex-grow, flex-shrink and flex-basis properties.

Mastering wrapping of flex items

How to create flex containers with multiple lines and control the display of the items in those lines.

Typical use cases of flexbox

Common design patterns that are typical Flexbox use cases.

CSS layout: flexbox

Learn how to use flexbox layout to create web layouts.

Box alignment in flexbox

Details features of CSS box alignment which are specific to flexbox.

Defining CSS gaps

Understanding and defining gaps in grid, flexbox, and multi-column layouts, including how percentage values are resolved.

Related concepts

CSS display module

- `display`
- `order`

CSS box alignment module

- `align-content`
- `align-items`
- `align-self`
- `justify-items`
- `place-content`
- `place-items`

CSS gaps module

- `column-gap`
- `column-rule`
- `gap`
- `row-gap`
- `row-rule`
- `rule`
- `rule-color`
- `rule-inset`
- `rule-overlap`
- `rule-style`
- `rule-width`

CSS box sizing module

- `aspect-ratio`
- `max-content` value
- `min-content` value
- `fit-content` value
- intrinsic size glossary term

See also

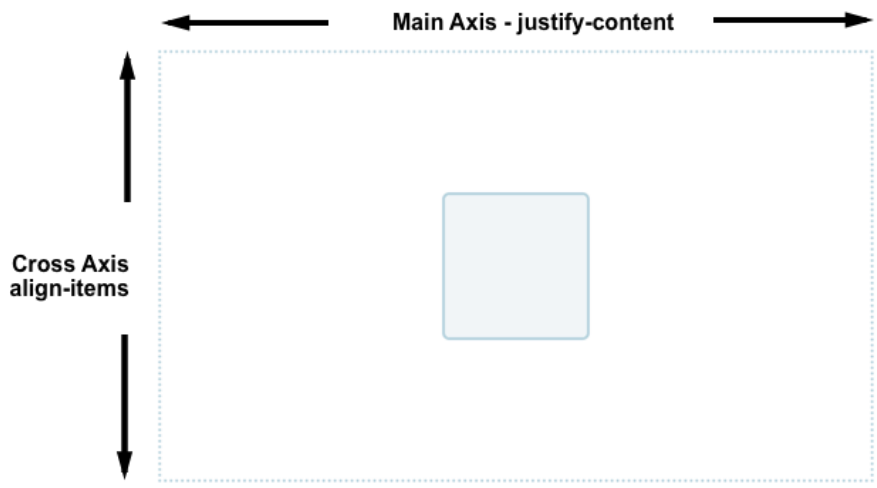
- CSS grid layout module
- CSS writing modes module
- Using the multi-keyword syntax with CSS display

Aligning items in a flex container

One of the reasons flexbox is so useful is that it enables proper alignment, including providing a quick method of vertically centering elements. In this guide, we will take a thorough look at how the alignment and justification properties work in flexbox.

Using alignment in flexbox

Flexbox provides several properties to control alignment and spacing, with `align-items` and `justify-content` being fundamental for centering elements. To center an element, we use the `align-items` property to align the item on the cross axis, which in this case is the block axis running vertically. We use `justify-content` to align the item on the main axis, which in this case is the inline axis running horizontally.



Change the size of the container or nested element in the code example below. The nested element always remains centered.

```
<div class="box">
  <div></div>
</div>
```

```
.box {
  display: flex;
  align-items: center;
  justify-content: center;
  border: 2px dotted rgb(96 139 168);
}

.box div {
```

```
  width: 100px;
  height: 100px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

Properties for controlling alignment in flexbox

The properties we will look at in this guide are as follows.

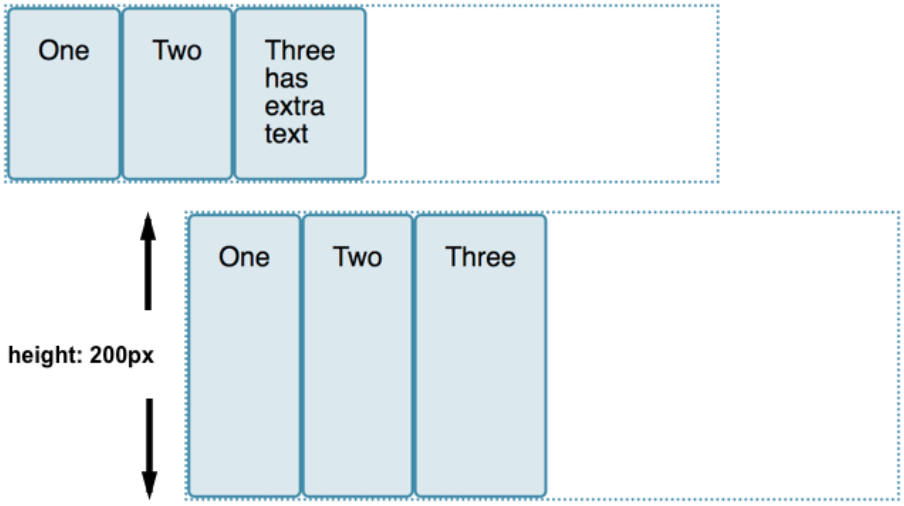
- `justify-content`: Controls the alignment of all items on the main axis.
- `align-items`: Controls the alignment of all items on the cross axis.
- `align-self`: Controls the alignment of an individual flex item on the cross axis.
- `align-content`: Controls the space between flex lines on the cross axis.
- `gap`, `column-gap`, and `row-gap`: Used to create gaps or gutters between flex items.

We will also discover how auto margins can be used for alignment in flexbox.

Aligning items on the cross axis

The `align-items` property, set on the flex container, and the `align-self` property, set on flex items, control the alignment of flex items on the cross axis. The cross axis runs down the columns if `flex-direction` is `row` and along the rows if `flex-direction` is `column`.

In this basic flex example, we're using cross-axis alignment. When we add `display: flex` to a container, the child items become flex items arranged in a row. By default, they will all stretch to match the height of the tallest item, as that item defines the height of the items on the cross axis. If the flex container has a height set, the items will stretch to that height, regardless of how much content is in each item.



The reason the items become the same height is that the initial value of `align-items`, the property that controls alignment on the cross axis, is set to `stretch`.

We can use other values to control how the items align:

- `align-items: stretch`
- `align-items: flex-start`
- `align-items: flex-end`
- `align-items: start`
- `align-items: end`
- `align-items: center`
- `align-items: baseline`
- `align-items: first baseline`
- `align-items: last baseline`

In the example below, the value of `align-items` is `stretch`. Try the other values and see how the items align against each other in the flex container.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  align-items: stretch;
}

.box div {
  width: 100px;
  background-color: rgb(96 139 168 / 0.2);
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
}
```

Aligning one item with <code>align-self</code>

The `align-items` property sets the `align-self` property on all of the flex items as a group. This means you can explicitly declare the `align-self` property to target a single item. The `align-self` property accepts all of the same values as `align-items`, plus a value of `auto`, which resets the value to that defined on the flex container.

In this next live example, the flex container has `align-items: flex-start`, which means the items are all aligned to the start of the cross axis. Using the `first-child` selector, the first item is set to `align-self: stretch`. Another item with the `selected` class has `align-self: center` set. Change the value of `align-items` or change the values of `align-self` on the individual items to see how this works.

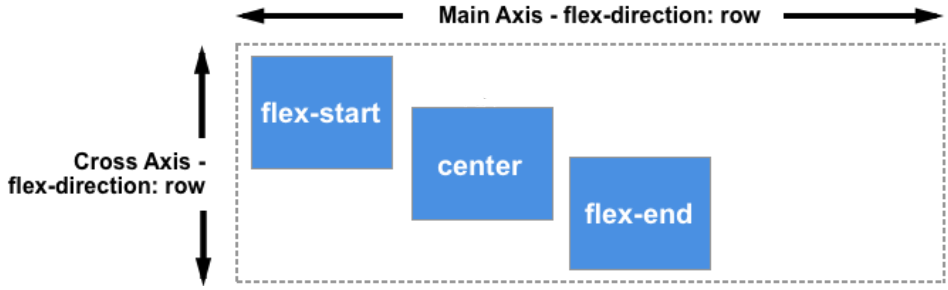
```
<div class="box">
  <div>One</div>
```

```
  <div>Two</div>
  <div class="selected">Three</div>
  <div>Four</div>
</div>
```

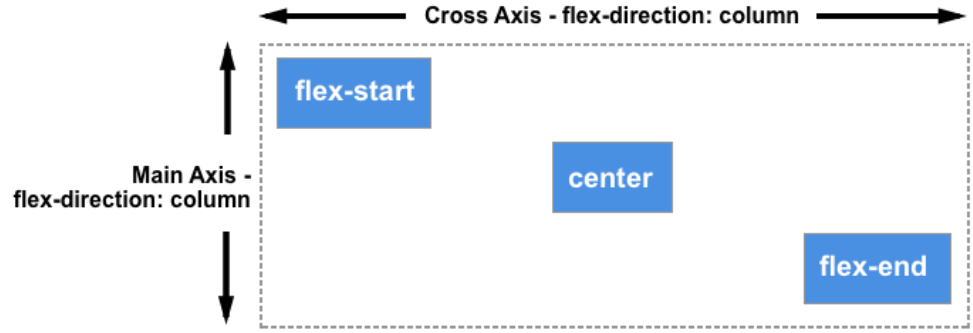
```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  align-items: flex-start;
  height: 200px;
}
.box div {
  background-color: rgb(96 139 168 / 0.2);
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  padding: 20px;
}
.box > *:first-child {
  align-self: stretch;
}
.box .selected {
  align-self: center;
}
```

Changing the main axis

Thus far, we have looked at alignment behavior when the `flex-direction` defaults to `row` while working in a language written top to bottom, with a horizontal main axis and vertical cross axis.



Keeping the same writing mode, when the `flex-direction` is changed to `column`, the `align-items` and `align-self` properties will align the items to the left and right instead of top and bottom; these properties are still aligning items along the cross axis, but the cross axis is now horizontal!



You can try this out in the example below, which has a flex container with `flex-direction: column` yet otherwise is exactly the same as the previous example.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div class="selected">Three</div>
  <div>Four</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  width: 200px;
}
.box div {
  background-color: rgb(96 139 168 / 0.2);
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  padding: 20px;
}
.box > *:first-child {
  align-self: stretch;
}
.box .selected {
  align-self: center;
}
```

Aligning content on the cross axis with the <code>align-content</code> property

So far, we have focused on aligning items or individual items inside the area defined by a flex container containing a single line of flex items. When flex items are allowed to wrap across multiple lines, the `align-content` property can be used to control the distribution of space between the lines, also known as **packing flex lines**.

For `align-content` to have an effect, the cross axis dimension (the height in this case) of the flex container must be greater than needed to display the items. It then works on all the items as a set. The `align-content` values dictate what happens with the extra available space and the alignment of the entire set of items within it.

The `align-content` property takes the following values:

- `align-content: flex-start`
- `align-content: flex-end`
- `align-content: start`
- `align-content: end`
- `align-content: center`
- `align-content: space-between`
- `align-content: space-around`
- `align-content: space-evenly`
- `align-content: stretch`
- `align-content: normal` (behaves as `stretch`)
- `align-content: baseline`
- `align-content: first baseline`
- `align-content: last baseline`

In the live example below, the flex container has a height of `400 pixels`, which is more than needed to display our items. The value of `align-content` is `space-between`, which means that the available space is shared out *between* the flex lines, which are placed flush with the start and end of the container on the cross axis.

Try out the other values to see how the `align-content` property works.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
</div>
```

```
.box {
  width: 450px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
  height: 300px;
  align-content: space-between;
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 100px;
}

.box div {
  background-color: rgb(96 139 168 / 0.2);
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  padding: 20px;
}
```

Once again we can switch our `flex-direction` to `column` in order to see how this property behaves when we are working by column. As before, we need enough space in the cross axis to have some free space after displaying all of the items.

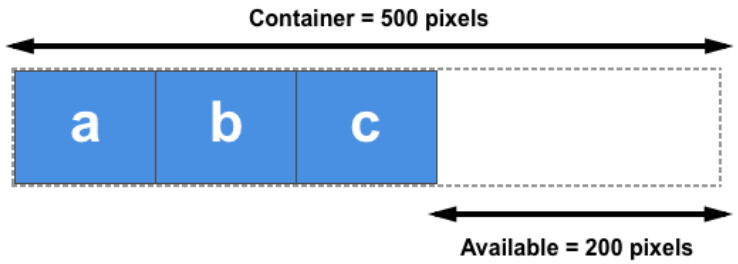
```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
</div>
```

```
.box {
  display: flex;
  flex-wrap: wrap;
  flex-direction: column;
  width: 400px;
  height: 300px;
  align-content: space-between;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 100px;
}

.box div {
  background-color: rgb(96 139 168 / 0.2);
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  padding: 20px;
}
```

Aligning content on the main axis

Now that we have seen how alignment works on the cross axis, we can take a look at the main axis. Here we only have one property available to us — `justify-content`. This is because we are only dealing with items as a group on the main axis. With `justify-content` we control what happens with available space, should there be more space than is needed to display the items.

In our initial example with `display: flex` on the container, the items display as a row and all line up at the start of the container. This is due to the initial value of `justify-content` being `normal`, which behaves as `start`. Any available space is placed at the end of the items.



The `baseline` values aren't relevant in this dimension. Otherwise, the `justify-content` property accepts the same values as `align-content`.

- `justify-content: flex-start`
- `justify-content: flex-end`
- `justify-content: start`
- `justify-content: end`
- `justify-content: left`
- `justify-content: right`
- `justify-content: center`
- `justify-content: space-between`
- `justify-content: space-around`
- `justify-content: space-evenly`
- `justify-content: stretch` (behaves as start)
- `justify-content: normal` (behaves as stretch, which behaves as start)

In the example below, the value of `justify-content` is `space-between`. The available space after displaying the items is distributed between the items. The left and right item line up flush with the start and end.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
</div>
```

```
.box {
  display: flex;
  justify-content: space-between;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

If the main axis is in the block direction because `flex-direction` is set to `column`, then `justify-content` will distribute space between items in that dimension as long as there is space in the flex container to distribute.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
</div>
```

```
.box {
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  height: 300px;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

Alignment and writing modes

Remember that with all of these alignment methods, the values of `start` and `end` are writing mode-aware. If the value of `justify-content` is `start` and the writing mode is left-to-right, as in English, the items will align starting at the left side of the container.

However if the writing mode is right-to-left as in Arabic, the items will line up starting at the right side of the container.

The live example below has the `direction` property set to `rtl` to force a right-to-left flow for our items. You can remove this, or change the values of `justify-content` to see how flexbox behaves when the start of the inline direction is on the right.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
</div>
```

```
.box {
  direction: rtl;
  display: flex;
  justify-content: flex-end;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

Alignment and <code>flex-direction</code>

The direction of `flex-start` of the line will also change if you change the `flex-direction` property — for example, using `row-reverse` instead of `row`. The directions of `start` and `end` are unaffected by `flex-direction` changes.

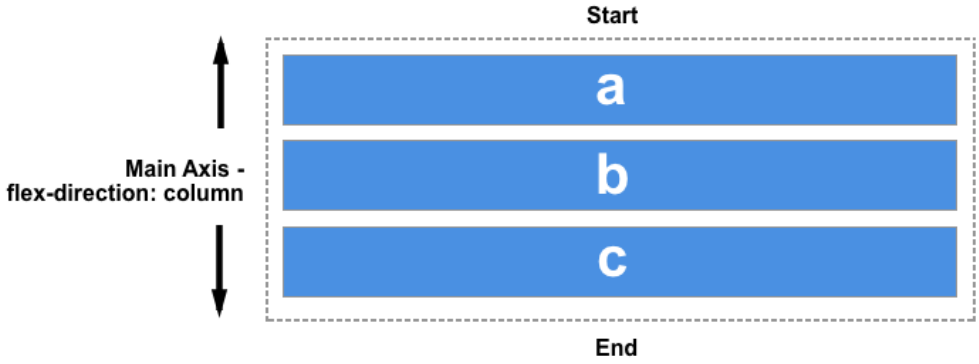
In this next example, `flex-direction: row-reverse` and `justify-content: flex-end` define the direction and location of the items within the flex container. In a left to right language, the items line up on the left. Try changing `flex-direction: row-reverse` to `flex-direction: row`. You will see that the items now move to the right-hand side, and the visual order of the items is reversed.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
</div>
```

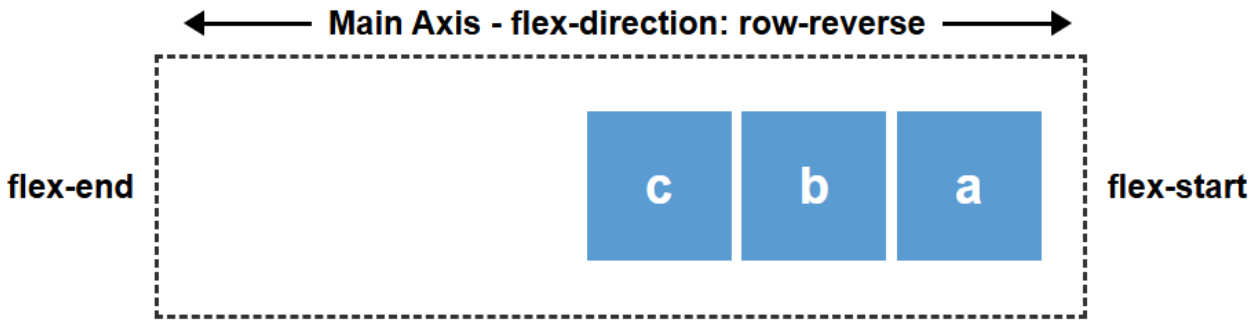
```
.box {
  display: flex;
  flex-direction: row-reverse;
  justify-content: flex-end;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

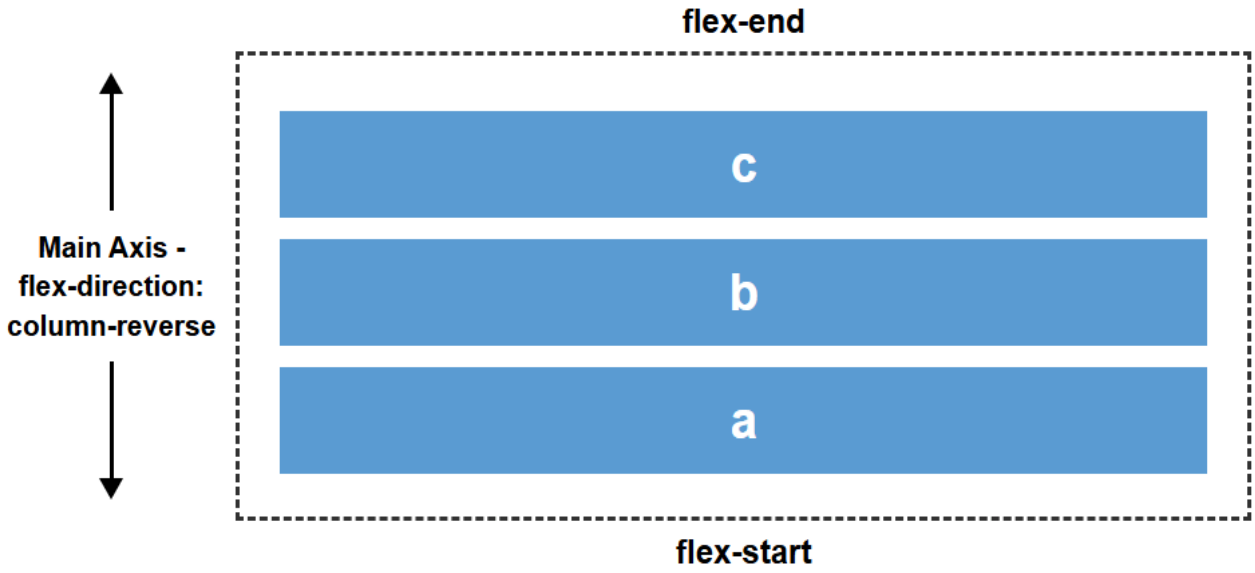
While this may all seem a little confusing, the rule to remember is that unless you do something to change it, flex items lay themselves out in the direction that words are laid out in the language of your document along the inline, row axis. `flex-start` will be where the beginning of a sentence of text would start.

You can switch them to display in the block direction for the language of your document by selecting `flex-direction: column`. Then, `start` and `flex-start` will be where the top of your first paragraph of text would start.



If you change `flex-direction` to one of the reverse values, they will lay themselves out from the end axis and in the reverse order to the way words are written in the language of your document. Then, `flex-start` will change to the end of that axis — so to the location where your lines would wrap if working in rows, or at the end of your last paragraph of text in the block direction.





Using auto margins for main axis alignment

We don't have a `justify-items` or `justify-self` property available to us on the main axis as our items are treated as a group on that axis. However it is possible to do some individual alignment in order to separate an item or a group of items from others by using auto margins along with flexbox.

A common pattern is a navigation bar where some key items are aligned to the right, with the main group on the left. You might think that this should be a use case for a `justify-self` property. However, consider the image below. As an example, take the following image with three items on one side and two on the other. If `justify-self` were to work on flex items and was set on item *d*, it would also change the alignment of item *e* that follows, which may or may not be what is intended.

Instead, the *d* item can be pushed over using CSS margins.

In this live example, item 4 is separated from the first three items by setting `margin-left` to `auto`, which consumes all the space it can in its axis. This is how centering a block with `margin` auto left and right works. Each side tries to take as much space as it can, and so the block is pushed into the middle.

In this live example, the flex items are arranged in a row with the basic flex values, and the class `push`, set on the fourth item, applies `margin-left: auto` to that item. Try removing the class on the fourth item or adding the class to a different item to see how it works.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div class="push">Four</div>
  <div>Five</div>
</div>
```

```
.box {
  display: flex;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
.push {
  margin-left: auto;
}
```

Creating gaps between items

To create a gap between flex items, use the `gap`, `column-gap`, and `row-gap` properties. The `column-gap` property creates gaps between items in a row. The `row-gap` property creates gaps between flex lines when you have `flex-wrap` set to `wrap`.

The `gap` property is a shorthand that sets both `row-gap` and `column-gap`. The gaps between flex items or flex lines depend on the direction. If the `flex-direction` property creates rows, the first value defines the gap between flex lines, and the second value defines the gap between items within each line. With columns (when `flex-direction` is set to `column` or `column-reverse`), the first value defines the gap between flex items, and the second value defines the gaps between flex lines.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
</div>
```

```
.box {
  display: flex;
  flex-wrap: wrap;
  row-gap: 10px;
  column-gap: 2em;
  border: 2px dotted rgb(96 139 168);
}

.box > * {
  flex: 1;
  padding: 20px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

See also

- CSS box alignment module
- CSS flexible box layout module
- Box alignment in flexbox
- Box alignment in grid layout

Basic concepts of flexbox

The flexible box layout module (usually referred to as flexbox) is a one-dimensional layout model for distributing space between items and includes numerous alignment capabilities. This article gives an outline of the main features of flexbox, which we will explore in more detail in the rest of these guides.

When we describe flexbox as being one-dimensional we are describing the fact that flexbox deals with layout in one dimension at a time — either as a row or as a column. This can be contrasted with the two-dimensional model of CSS Grid Layout, which controls columns and rows together.

The two axes of flexbox

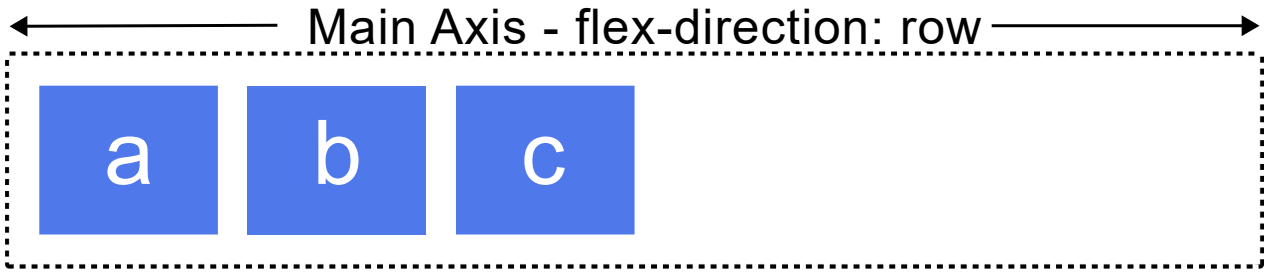
When working with flexbox you need to think in terms of two axes — the *main axis* and the *cross axis*. The main axis is defined by the `flex-direction` property, and the cross axis runs perpendicular to it. Everything we do with flexbox refers back to these axes, so it is worth understanding how they work from the outset.

The main axis

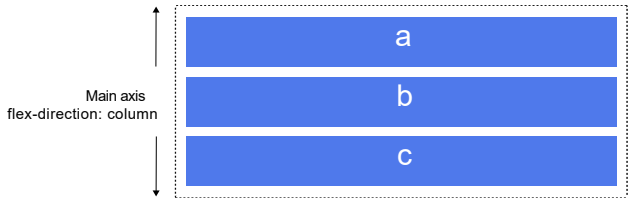
The main axis is defined by `flex-direction`, which has four possible values:

- `row`
- `row-reverse`
- `column`
- `column-reverse`

Should you choose `row` or `row-reverse`, your main axis will run along the row in the **inline direction**.



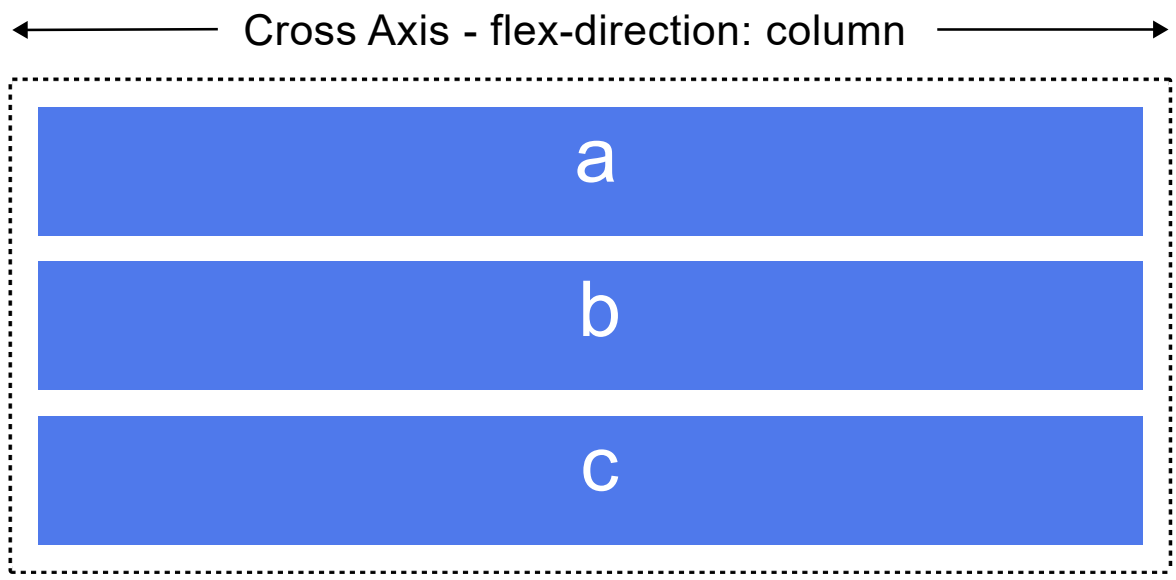
Choose `column` or `column-reverse` and your main axis will run in the **block direction**, from the top of the page to the bottom.



The cross axis

The cross axis runs perpendicular to the main axis. Therefore, if your `flex-direction` (main axis) is set to `row` or `row-reverse` the cross axis runs down the columns.

If your main axis is `column` or `column-reverse` then the cross axis runs along the rows.

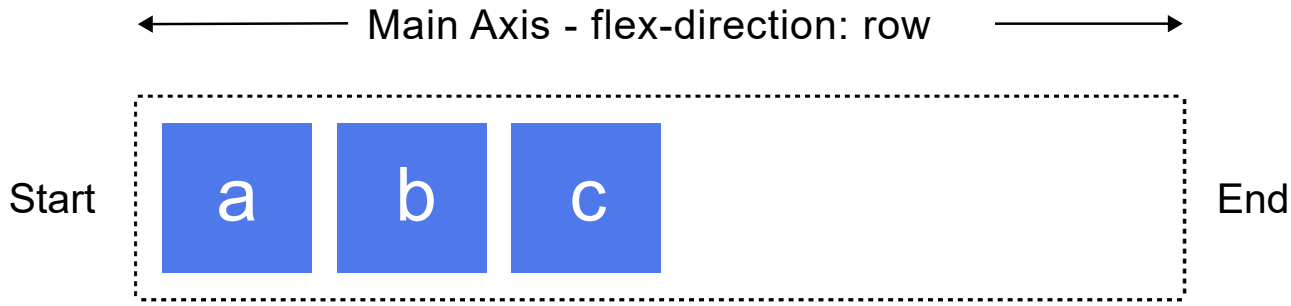


Start and end lines

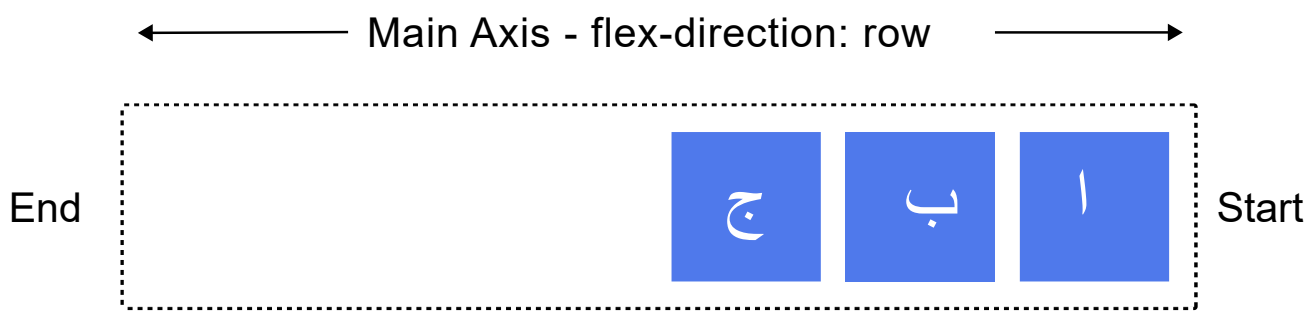
Another vital area of understanding is how flexbox makes no assumption about the writing mode of the document. Flexbox doesn't just assume that all lines of text start at the top left of a document and run towards the right-hand side, with new lines appearing one under the other. Rather, it supports all writing modes, like other logical properties and values.

You can read more about the relationship between flexbox and writing modes in a later article; however, the following description should help explain why we do not talk about left and right and top and bottom when we describe the direction that our flex items flow in.

If the `flex-direction` is `row` and I am working in English, then the start edge of the main axis will be on the left, the end edge on the right.



If I were to work in Arabic, then the start edge of my main axis would be on the right and the end edge on the left.



In both cases the start edge of the cross-axis is at the top of the flex container and the end edge at the bottom, as both languages have a horizontal writing mode.

After a while, thinking about start and end rather than left and right becomes natural, and will be useful to you when dealing with other layout methods such as CSS Grid Layout which follow the same patterns.

The flex container

An area of a document that is laid out using flexbox is called a **flex container**. To create a flex container, set the area's `display` property to `flex`. When we do this, the direct children of that container become **flex items**. You can explicitly control whether the container itself is displayed inline or in block formatting context using `inline flex` or `inline-flex` for inline flex containers or `block flex` or `flex` for block level flex containers.

Initial values

As with all properties in CSS, some initial values are defined, so the contents of a new flex container will behave in the following way:

- Items display in a row (the `flex-direction` property's default value is `row`).
- The items start from the start edge of the main axis.
- The items do not stretch on the main dimension but can shrink (a flex-item's `flex-grow` property's default value is `0` and its `flex-shrink` property's default value is `1`).
- The items will stretch to fill the size of the cross-axis (the `align-items` property's default value is `stretch`).
- The flex-item's `flex-basis` property's default value is `auto`. This means that, in each case, it will be equal to the flex item `width` in horizontal writing mode, and the flex item `height` in vertical writing mode. If the corresponding `width`/`height` is also set to `auto`, the `flex-basis` `content` value is used instead.
- All the items will be in a single row (the `flex-wrap` property's default value is `nowrap`), overflowing their container if their combined `width`/`height` exceeds the containing element `width`/`height`.

The result of this is that your items will all line up in a row, using the size of the content as their size in the main axis. If there are more items than can fit in the container, they will not wrap but will instead overflow. If some items are taller than others, all items will stretch along the full length of the cross-axis.

You can see in the live sample below how this looks. Click "Play" to open the example in the MDN Playground and edit the items or add new items to try out the initial behavior of flexbox:

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
}
```

Changing flex-direction

Adding the `flex-direction` property to the flex container allows us to change the direction in which our flex items display. Setting `flex-direction: row-reverse` will keep the items displaying along the row, however the start and end lines are switched.

If we change `flex-direction` to `column` the main axis switches and our items now display in a column. Set `column-reverse` and the start and end lines are again switched.

The live sample below has `flex-direction` set to `row-reverse`. Try the other values — `row`, `column` and `column-reverse` — to see what happens to the content.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
```

```
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-direction: row-reverse;
}
```

Multi-line flex containers with flex-wrap

While flexbox is a one dimensional model, it is possible to make flex items wrap across multiple lines. If you do this, you should consider each line as a new flex container. Any space distribution will happen across each line, without reference to the previous or subsequent lines.

To cause wrapping behavior add the property `flex-wrap` with a value of `wrap`. Now, if your items are too large to all display in one line, they will wrap onto another line. The live sample below contains items that have been given a `width`. The total width of the items is too wide for the flex container. As `flex-wrap` is set to `wrap`, the items wrap across multiple lines. If you set it to `nowrap`, which is the initial value, they will shrink to fit the container. They shrink because they are using initial flexbox values, including `flex-shrink: 1`, that allows items to shrink. Using `nowrap` would cause an overflow if the items were not able to shrink, or could not shrink small enough to fit.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  width: 200px;
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
}
```

Find out more about wrapping flex items in the guide Mastering wrapping of flex items.

The flex-flow shorthand

You can combine the two properties `flex-direction` and `flex-wrap` into the `flex-flow` shorthand.

In the live sample below, try changing the first value to one of the allowable values for `flex-direction` - `row`, `row-reverse`, `column` or `column-reverse`, and also change the second to `wrap` and `nowrap`.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  width: 200px;
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-flow: row wrap;
}
```

Properties applied to flex items

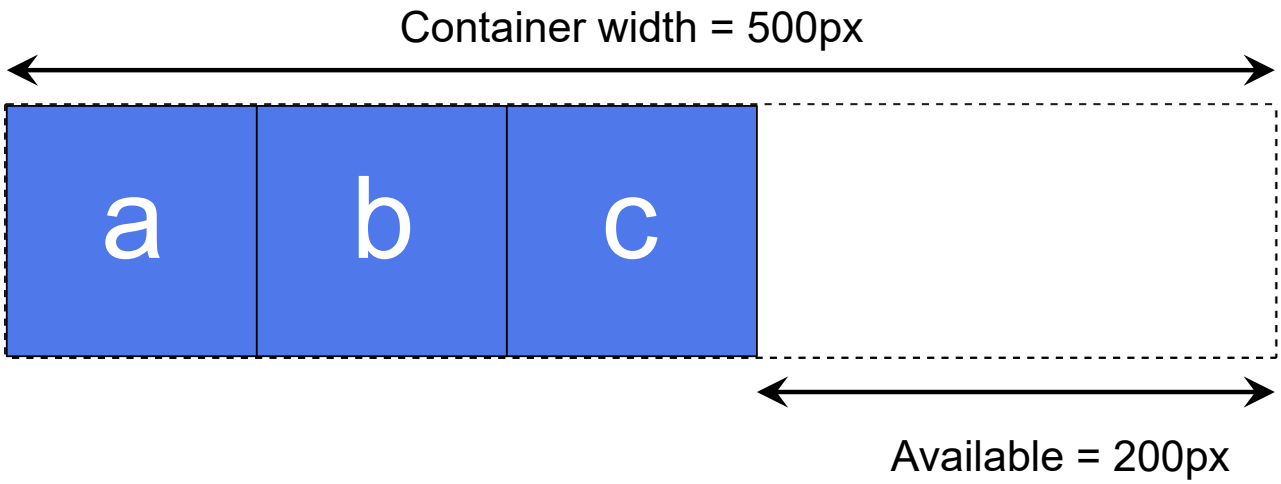
To control the inline-size of each flex item, we target them directly via three properties:

- `flex-grow`
- `flex-shrink`
- `flex-basis`

We will take a brief look at these properties below, but if you want more comprehensive information, take a look at the Controlling ratios of flex items on the main axis guide.

Before we can make sense of these properties we need to consider the concept of **available space**. What we are doing when we change the value of these flex properties is to change the way that available space is distributed among our items. This concept of available space is also important when we come to look at aligning items.

If we have three 100 pixel-wide items in a container which is 500 pixels wide, then the space we need to lay out our items is 300 pixels. This leaves 200 pixels of available space. If we don't change the initial values then flexbox will put that space after the last item.



If we instead would like the items to grow and fill the space, then we need to have a method of distributing the leftover space between the items. The `flex` properties that we apply to the items themselves, enable dictating how that available space should be distributed among the sibling flex items.

The flex-basis property

The `flex-basis` is what defines the size of that item in terms of the space it leaves as available space. The initial value of this property is `auto` — in this case the browser looks to see if the item has a size. In the example above, all of the items have a width of 100 pixels. This is used as the `flex-basis`.

If the items don't have a size then the content's size is used as the flex-basis. This is why when we just declare `display: flex` on the parent to create flex items, the items all move into a row and take only as much space as they need to display their contents.

The flex-grow property

With the `flex-grow` property set to a positive integer, if there is available space, the flex item can grow along the main axis from its `flex-basis`. Whether the item stretches to take up all the available space on that axis, or just a portion of the available space depends on if the other items are allowed to grow too and the value of their `flex-grow` properties.

Each item with a positive value consumes a portion of any available space based on their `flex-grow` value. If we gave all of our items in the example above a `flex-grow` value of 1 then the available space in the flex container would be equally shared between our items and they would stretch to fill the container on the main axis. If we give our first item a `flex-grow` value of 2, and the other items a value of 1 each, there are a total of 4 parts; 2 parts of the available space will be given to the first item (100px out of 200px in the case of the example above) and 1 part each the other two (50px each out of the 200px total).

The flex-shrink property

Where the `flex-grow` property deals with adding space in the main axis, the `flex-shrink` property controls how it is taken away. If we do not have enough space in the container to lay out our items, and `flex-shrink` is set to a positive integer, then the item can become smaller than the `flex-basis`. As with `flex-grow`, different values can be assigned in order to cause one item to shrink faster than others — an item with a higher value set for `flex-shrink` will shrink faster than its siblings that have lower values.

An item can shrink down to its `min-content` size. This minimum size is taken into account while working out the actual amount of shrinkage that will happen, which means that `flex-shrink` has the potential to appear less consistent than `flex-grow` in behavior. We'll therefore take a more detailed look at how this algorithm works in the article Controlling ratios of items along the main axis.

> **Note:** These values for `flex-grow` and `flex-shrink` are proportions. Typically if we had all of our items set to `flex: 1 1 200px` and then wanted one item to grow at twice the rate, we would set that item to `flex: 2 1 200px`. However you could also use `flex: 10 1 200px` and `flex: 20 1 200px` if you wanted.

Shorthand values for the flex properties

You will very rarely see the `flex-grow`, `flex-shrink`, and `flex-basis` properties used individually; instead they are combined into the `flex` shorthand. The `flex` shorthand allows you to set the three values in this order — `flex-grow`, `flex-shrink`, `flex-basis`.

The live sample below allows you to test out the different values of the flex shorthand; remember that the first value is `flex-grow`. Giving this a positive value means the item can grow. The second is `flex-shrink` — with a positive value the items can shrink, but only if their total values overflow the main axis. The final value is `flex-basis`; this is the value the items are using as their base value to grow and shrink from.

```
<div class="box">
  <div class="one">One</div>
  <div class="two">Two</div>
  <div class="three">Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
}

.one {
  flex: 1 1 auto;
}

.two {
  flex: 1 1 auto;
}
```

```
.three {
  flex: 1 1 auto;
}
```

There are also some predefined shorthand values which cover most of the use cases. You will often see these used in tutorials, and in many cases these are all you will need to use. The predefined values are as follows:

- `flex: initial`
- `flex: auto`
- `flex: none`
- `flex: <positive-number>`

The `initial` value is a CSS-wide keyword that represents the initial value for a property. Setting `flex: initial` resets the item to the initial values of the three longhand properties, which is the same as `flex: 0 1 auto`. The initial value of `flex-grow` is `0`, so items will not grow larger than their `flex-basis` size. The initial value of `flex-shrink` is `1`, so items can shrink if they need to rather than overflowing. The initial value of `flex-basis` is `auto`. Items will either use any size set on the item in the main dimension, or they will get their size from the content size.

Using `flex: auto` is the same as using `flex: 1 1 auto`; this is similar to `flex: initial`, except that the items can grow and fill the container as well as shrink if needed.

Using `flex: none` will create fully inflexible flex items. It is as if you wrote `flex: 0 0 auto`. The items cannot grow or shrink and will be laid out using flexbox with a `flex-basis` of `auto`.

The shorthand you often see in tutorials is `flex: 1` or `flex: 2` and so on. This is the same as writing `flex: 1 1 0` or `flex: 2 1 0` and so on, respectively. The items get minimum size due to `flex-basis: 0` and then proportionally grow to fill the available space. In this case, the `flex-shrink` value of `1` is redundant because the items start with minimum size — they're not given any size that could cause them to overflow the flex container.

Try these shorthand values in the live sample below.

```
<div class="box">
  <div class="one">One</div>
  <div class="two">Two</div>
  <div class="three">Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
}

.one {
  flex: 1;
}

.two {
  flex: 1;
}

.three {
  flex: 1;
}
```

Alignment, justification and distribution of free space between items

A key feature of flexbox is the ability to align and justify items on the main- and cross-axes, and to distribute space between flex items. Note that these properties are set on the flex container, not on the items themselves.

align-items

The `align-items` property aligns all the flex items on the cross axis.

The initial value for this property is `stretch` and is why flex items stretch to the height of the flex container by default (or the width if `flex-direction` is set to `column` or `column-reverse`). This height may come from the tallest item in the container or the size set on the flex container itself.

You could instead set `align-items` to `flex-start`, or simply `start`, in order to make the items line up at the start of the flex container, `flex-end`, or just `end`, to align them to the end, or `center` to align them in the center. Try this in the live sample — I have given the flex container a height in order that you can see how the items can be moved around inside the container. See what happens if you set the value of align-items to:

- `stretch`
- `flex-start`
- `flex-end`
- `start`
- `end`
- `center`
- `baseline`
- `last baseline`

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three <br />has <br />extra <br />text</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  width: 500px;
  height: 130px;
  border: 2px dotted rgb(96 139 168);
```

```
  display: flex;
  align-items: flex-start;
}
```

The `align-items` is set on the flex container and impacts all the flex items. If you want to align a flex item differently from others, you can set the `align-self` on the flex item.

justify-content

The `justify-content` property is used to align the items on the main axis, the direction in which `flex-direction` has set the flow. The initial value is `flex-start` which will line the items up at the start edge of the container, but you could also set the value to `flex-end` to line them up at the end, or `center` to line them up in the center.

You can also use the value `space-between` to take all the spare space after the items have been laid out, and share it out evenly between the items so there will be an equal amount of space between each item. To cause an equal amount of space on the right and left (or top and bottom for columns) of each item use the value `space-around`. With `space-around`, items have a half-size space on either end. Or, to cause items to have equal space around them use the value `space-evenly`. With `space-evenly`, items have a full-size space on either end.

Try the following values of `justify-content` in the live sample:

- `start`
- `end`
- `left`
- `right`
- `normal`
- `flex-start`
- `flex-end`
- `center`
- `space-around`
- `space-between`
- `space-evenly`
- `stretch`

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  justify-content: flex-start;
}
```

The article Aligning items in a flex container explores these properties in more depth, in order to have a better understanding of how they work. These basic examples, however, are useful in the majority of use cases.

justify-items

The `justify-items` property is ignored in flexbox layouts.

place-items and place-content

The `place-items` property is a shorthand property for `align-items` and `justify-items`. If set on a flex container, it will set the alignment but not the justification, as `justify-items` is ignored in flexbox.

There is another shorthand property, `place-content`, that defines the `align-content` and `justify-content` properties. The `align-content` property only affects flex containers that wrap, and is discussed in Aligning items in a flex container.

Next steps

After reading this article you should have an understanding of the basic features of flexbox. In the next article, we will look at how this specification relates to other parts of CSS.

Controlling ratios of flex items along the main axis

In this guide, we explore the three properties that control the size and flexibility of flex items along the main axis: `flex-grow`, `flex-shrink`, and `flex-basis`. Fully understanding how these properties work with growing and shrinking items is the key to mastering CSS flexible box layout.

A first look

Our three properties control the following aspects of a flex item's flexibility:

- `flex-grow`: How much of the positive free space does this item get?
- `flex-shrink`: How much negative free space can be removed from this item?
- `flex-basis`: What is the size of the item before growing and shrinking happens?

The properties are usually expressed using the shorthand `flex` property. The following code would set the `flex-grow` property to `2`, `flex-shrink` to `1` and `flex-basis` to `auto`.

```
.item {
  flex: 2 1 auto;
}
```

Important concepts when working on the main axis

To understand the `flex` properties, it is helpful to know the *natural size* of flex items before any growing or shrinking takes place. Additionally, it is important to understand the concept of *free space*, which is the difference between the combined natural size of all the flex items along the main axis and the size of the main axis itself.

Flex item sizing

To determine how much space is available to lay out flex items, the browser needs to know how big the item is to start with. How is this calculated for items that don't have a width or a height applied using an absolute length unit?

In CSS, the `min-content` and `max-content` keywords can be used in place of a `<length>` unit. Generally, `min-content` is the smallest size an element can be while still fitting the longest word, and `max-content` is the size the element would need to be to fit all the content without wrapping.

The example below contains two paragraph elements with different strings of text. The first paragraph has a width of `min-content`. Notice that the text has used all of the soft wrapping opportunities available to it, becoming as small as possible without overflowing. This is the `min-content` size of that string. Essentially, the longest word in the string is dictating the size.

The second paragraph, with a value of `max-content`, does the opposite. It grows as large as it needs to be to fit the content without taking soft-wrapping opportunities. It will overflow the box it is in if that container is too narrow.

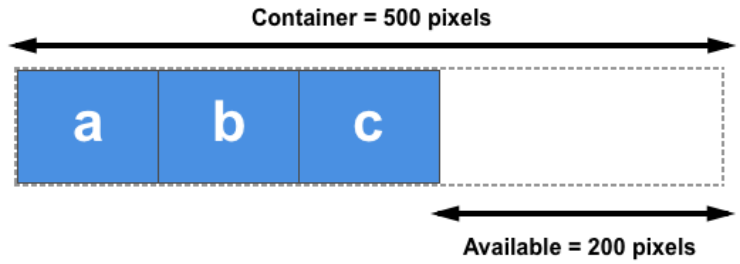
```
<p class="min-content">
  I am sized with min-content and so I will take all of the soft-wrapping
  opportunities.
</p>
<p class="max-content">
  I am sized with max-content and so I will take none of the soft-wrapping
  opportunities.
</p>
```

```
.min-content {
  width: min-content;
  border: 2px dotted rgb(96 139 168);
}
.max-content {
  width: max-content;
  border: 2px dotted rgb(96 139 168);
}
```

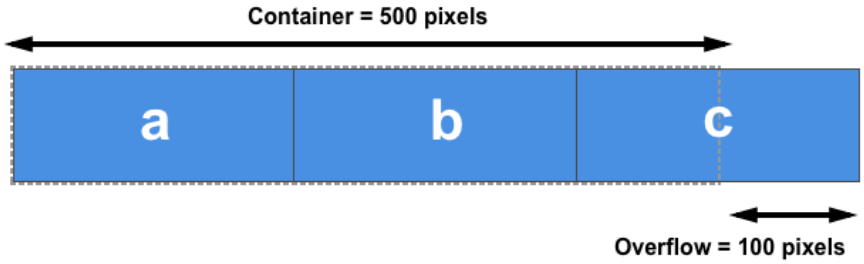
Remember this behavior and what effects `min-content` and `max-content` have as we explore `flex-grow` and `flex-shrink` later in this article.

Positive and negative free space

We also need to understand the concept of **positive and negative free space**. When a flex container has *positive free space*, it has more space than required to display the flex items inside the container. For example, a `500px`-wide container, with `flex-direction` set to `row` and containing three `100px`-wide flex items has `200px` of positive free space. This positive free space can be distributed between the items if filling the container is desired.



A flex container has *negative free space* when the combined value of the natural sizes of the flex items is larger than the available space in the flex container. If the three flex items in the `500px`-wide container example above are each `200px` wide instead of `100px`, their combined natural width is `600px`, resulting in `100px` of negative free space. This space can be removed from the items to make them fit the container, or the items will overflow.



We need to understand this distribution of positive free space and removal of negative free space to learn about the property components of the `flex` shorthand.

In the following examples, the `flex-direction` is set to `row`, so the size of items will be determined by their width. We will be calculating the positive and negative free space by comparing the total width of all the items with the container's width. You could also try out each example with `flex-direction: column`. The main axis would then be the column, and you would then compare the height of the items and their container to calculate the positive and negative free space.

The <code>flex-basis</code> property

The `flex-basis` property specifies the initial size of a flex item before any distribution of the positive or negative free space happens. The initial value for this property is `auto`. This property accepts the same values as the `width` and `height` properties, and it also accepts the `content` keyword.

If `flex-basis` is set to `auto`, the initial size of the item is the `<length-percentage>` size of the main size, if any was set. For example, if the item has `width: 200px` set, then `200px` would be the `flex-basis` for this item. Percentage values are relative to the flex container's inner main size. If `width: 50%` were set, the `flex-basis` for this item would be half of the container's content-box width. If no such size is set, meaning the item is auto-sized, then `auto` resolves to the size of its content (see the `min-` and `max-content` sizing discussion above), meaning the `flex-basis` is the item's `max-content` size.

This example contains three inflexible flex items, with both `flex-grow` and `flex-shrink` set to `0`. The first item, which has an explicit width of `150px`, takes a `flex-basis` of `150px`, whereas the other two items have no width set and so are sized according to their content width or `max-content`.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 0 0 auto;
```

```
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
}

.box :first-child {
  width: 150px;
}
```

In addition to the `auto` keyword and any other valid `width` value, you can use the `content` keyword as the `flex-basis`. This results in the `flex-basis` being based on the content size, even if there is a `width` set on the item. This creates the same effect as removing any width set and using `auto` as the `flex-basis`. While similar to setting `max-content`, the `content` value enables any `aspect-ratio` to be calculated based on the cross-axis size.

To completely ignore the size of the flex item during space distribution, set `flex-basis` to `0` and set a non-zero `flex-grow` value. Let's learn `flex-grow` before looking at this value in action.

The <code>flex-grow</code> property

The `flex-grow` property specifies the **flex grow factor**, which determines how much a flex item will grow relative to the other flex items in the flex container when positive free space is distributed.

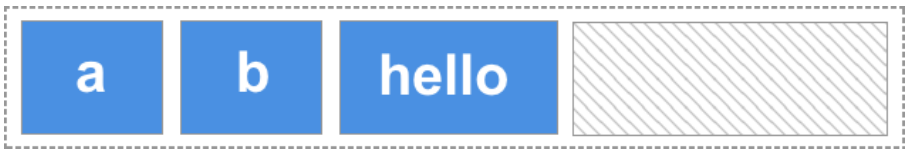
If all items have the same `flex-grow` factor, the positive free space will be distributed evenly among them. For this scenario, common practice is to set `flex-grow: 1`, but you could give them any value, such as `88`, `100`, or `1.2`; it is a ratio. If the factor is the same for all the flex items in the container and there is positive free space, that space will be distributed equally.

Combining <code>flex-grow</code> and <code>flex-basis</code>

Things can get confusing in terms of how `flex-grow` and `flex-basis` interact. Let's consider the case of three flex items of differing content lengths and the following `flex` rules applied to them:

```
.class {
  flex: 1 1 auto;
}
```

In this case, the `flex-basis` value is `auto` and the items don't have a width set, so they are auto-sized. This means the `flex-basis` used is the `max-content` size of each item. After laying out the items, there is some positive free space in the flex container, which is shown in the image below as the hatched area; the hatched area is the positive free space that will be distributed between the three items based on their `flex-grow` factors:



We are working with a `flex-basis` equal to the content size. This means the available space to distribute is subtracted from the total available space (the width of the flex container), and the remaining space is then shared equally between the three items. The biggest item remains the largest because it started from a bigger size, even though it has the same amount of spare space as the others:

To create three equally-sized items, even if the original elements have different sizes, set the `flex-basis` component to `0`:

```
.class {
  flex: 1 1 0;
}
```

Here, for the purpose of space distribution calculation, we are setting the size of each item to `0`. This means all the space is available for distribution. Since all the items have the same `flex-grow` factor, they each get an equal amount of space. This results in three equal-width flex items.

Try changing the `flex-grow` factor from 1 to 0 in this live example to see the different behavior:

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three has more content</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 0;
}

.box {
  width: 400px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
}
```

Giving items different flex-grow factors

Using `flex-grow` and `flex-basis` together enables us to control individual item sizes by setting different `flex-grow` factors. If we keep the `flex-basis` at `0` so that all the space can be distributed, we can create differently sized flex items by assigning each item a different `flex-grow` factor.

In the example below, we use `1` as the `flex-grow` factor for the first two items and double it to `2` for the third item. With `flex-basis: 0` set on all the items, the available space is distributed as follows:

1. The `flex-grow` factor values of all the sibling flex items are added together (the total is 4 in this case).
2. The positive free space in the flex container is divided by this total value.
3. The free space is distributed according to the individual values. In this case, the first item gets one part, the second one part, and the third two parts. This means that the third item is twice the size of the first and second items.

```
<div class="box">
  <div class="one">One</div>
  <div class="two">Two</div>
  <div class="three">Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 0;
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
}

.one {
  flex: 1 1 0;
}

.two {
  flex: 1 1 0;
}

.three {
  flex: 2 1 0;
}
```

Remember that you can use any positive value here. It is the ratio between the items that matters. You can use large numbers or decimals; it's up to you. To test this, change the `flex-grow` values in the above example to `.25`, `.25`, and `.50`. You should see the same result.

The <code>flex-shrink</code> property

The `flex-shrink` property specifies the **flex shrink factor**, which determines how much the flex item will shrink relative to the rest of the flex items in the flex container when negative free space is distributed.

This property deals with situations where the combined `flex-basis` value of the flex items is too large to fit in the flex container and would otherwise overflow. As long as an item's `flex-shrink` is a positive value, the item will shrink to not overflow the container.

While `flex-grow` is used to add available space to items that can grow, `flex-shrink` is used to remove space to ensure items fit in their container without overflowing.

In this example, there are three `200px`-wide flex items in a `500px`-wide container. With `flex-shrink` set to `0`, the items are not allowed to shrink, causing them to overflow the container.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three has more content</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 0 0 auto;
  width: 200px;
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
}
```

Change the `flex-shrink` value to `1`; each item will shrink by the same amount, fitting all the items into the container. The negative free space has been proportionally removed from each item, making each flex item smaller than its initial width.

Combining <code>flex-shrink</code> and <code>flex-basis</code>

It may appear that `flex-shrink` works in the same way as `flex-grow`, by shrinking rather than growing elements. However, there are some important differences to note.

The concept of flex base size affects how negative space is distributed across flex items. The flex shrink factor is multiplied by the flex base size when distributing negative space. This distributes negative space in proportion to how much the item is able to shrink. So, for example, a small item won't shrink to zero before a larger item has been noticeably reduced.

Small items will not shrink to less than their `min-content` size, which is the smallest size the element can be if it used all the available soft wrapping opportunities.

This example demonstrates `min-content` flooring, with the `flex-basis` resolving to the size of the content. If you change the width on the flex container, such as increasing it to `700px`, and then reduce the flex item width, you can see that the first two items will wrap. However, they will never become smaller than their `min-content` size. When the container gets small, the space is only removed from the third item when shrunken further.

```
<div class="box">
  <div>Item One</div>
  <div>Item Two</div>
  <div>Item Three has more content and so has a larger size</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 auto;
}

.box {
  border: 2px dotted rgb(96 139 168);
  width: 500px;
  display: flex;
}
```

In practice, this shrinking behavior provides reasonable results. It prevents content from disappearing completely and from becoming smaller than its minimum content size. The above rules are sensible for content that needs to shrink to fit its container.

Giving items different <code>flex-shrink</code> factors

In the same way as `flex-grow`, you can give flex items different `flex-shrink` factors. This can help change the default behavior if, for example, you want an item to shrink more or less rapidly than its siblings or to not shrink at all.

In this example, the first item has a `flex-shrink` factor of `1`, the second item `0` (so it won't shrink at all), and the third item `4`, making a total of `5` shrink factors. The third item, therefore, shrinks approximately four times more rapidly than the first, but neither will shrink below their `min-content` width. Play around with the different values: as with `flex-grow`, you can use decimals or larger numbers here as well.

```
<div class="box">
  <div class="one">One</div>
  <div class="two">Two</div>
  <div class="three">Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  width: 200px;
}

.box {
  display: flex;
  width: 500px;
  border: 2px dotted rgb(96 139 168);
}

.one {
  flex: 1 1 auto;
}

.two {
  flex: 1 0 auto;
}

.three {
  flex: 2 4 auto;
}
```

Mastering sizing of flex items

To understand how flex item sizing works, you need to consider the factors below, which we've discussed in these guides:

What determines the base size of an item?

- Is `flex-basis` set to `auto`, and does the item have a width set? If so, the size will be based on that width.
- Is `flex-basis` set to `auto`, but the item doesn't have a width set? If so, the size will be based on the item's content size.
- Is `flex-basis` a length or a percentage, but not zero? If so, this will be the size of the item (floored at `min-content`).
- Is `flex-basis` set to `0`? If so, the item's size will not be taken into consideration for the space-sharing calculation.

Is there available space?

Items can grow only if there is positive free space, and they won't shrink unless there is negative free space.

- If we add up the widths of all the items (or heights if working in a column), is that total **less** than the total width (or height) of the container? If so, there will be positive free space, and `flex-grow` will come into play.
- If we add up the widths of all the items (or heights if working in a column), is that total **more** than the total width (or height) of the container? If so, there will be negative free space, and `flex-shrink` will come into play.

What are the other ways to distribute space?

If you do not want space added to the items, remember that you can manage free space between or around items using the alignment properties described in the guide for aligning items in a flex container. The `justify-content` property will enable the distribution of free space between or around items. You can also use auto margins on flex items to absorb space and create gaps between items.

With all these flex properties available to you, you will find that most layout tasks are possible, although it might take a little bit of experimentation at first.

Ordering flex items

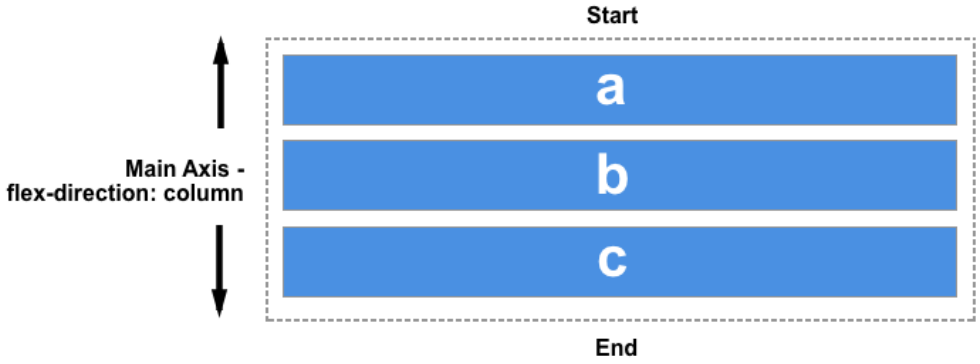
Layout methods such as flexbox and grid enable controlling the order of content. In this article, we will take a look at ways in which you can change the visual order of your content when using flexbox. We will also consider how reordering items impacts accessibility.

Reverse the display of the items

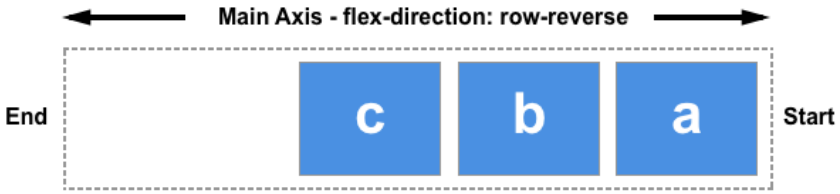
The `flex-direction` property can take one of four values:

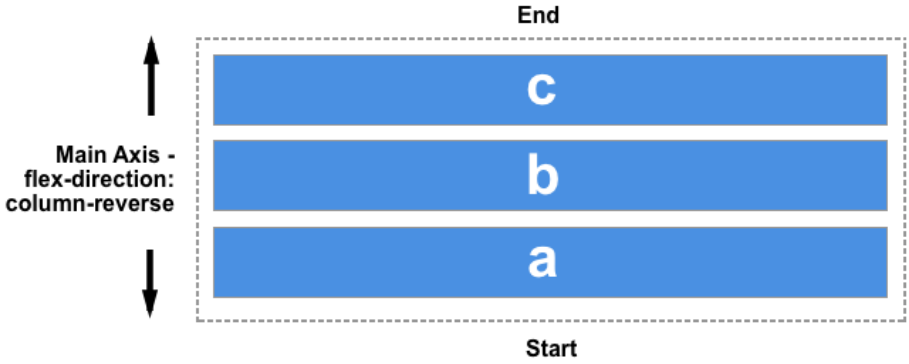
- `row`
- `column`
- `row-reverse`
- `column-reverse`

The first two values keep the items in the same order that they appear in the document source order and display them sequentially from the start line.

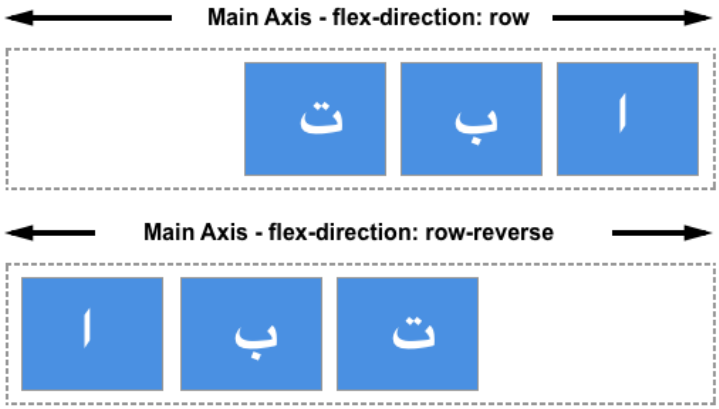


The second two values reverse the items by switching the start and end lines.





Remember that the start line relates to writing modes. The row-related examples above demonstrate how `row` and `row-reverse` work in a left-to-right language such as English. If you are working in a right-to-left language like Arabic then `row` would start on the right, `row-reverse` on the left.



This can seem like an easy way to display things in reverse order. However, you should be mindful that the items are only *visually* displayed in reverse order. The reordering capabilities of flex layout affect only the visual rendering. The tabbing order and speech order follow the order of the source code. This means that only the visual presentation changes; the source order remains the same, providing a different user experience for non-CSS UAs (think Siri or Alexa) and assistive technology users. If you change the order of a navigation bar, the tabbing order is still the document source order, not your visual order, which can be cognitively confusing.

If you are using a reverse value, or otherwise reordering your items, you should consider whether you should really be changing the logical order in the source.

The flexible box layout specification warns us not to use reordering as a way of fixing source issues:

> "Authors *must not* use order or the *-reverse values of `flex-flow`/`flex-direction` as a substitute for correct source ordering, as that can ruin the accessibility of the document."

As you tab from link to link in the live example below, the focus style is highlighted, demonstrating that changing the order of flex-items with `flex-direction` does not change the tabbing order, which will continue to follow the source code order.

```
<div class="box">
  <div><a href="#">One</a></div>
  <div><a href="#">Two</a></div>
  <div><a href="#">Three</a></div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
}

.box > * a:focus {
  background-color: yellow;
  color: black;
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-direction: row-reverse;
}
```

In the same way that changing the value of `flex-direction` does not change the tabbing order, changing this value does not change paint order. It is a visual reversal of the items only.

The order property

In addition to reversing the order in which flex items are visually displayed, you can target individual items and change where they appear in the visual order with the `order` property.

The `order` property is designed to lay the items out in *ordinal groups*. This means items are assigned an integer that represents their group. The items are then placed in the visual order according to that integer, lowest values first. If more than one item has the same integer value, then within that group the items are laid out as per source order.

As an example, five flex items are assigned `order` values as follows:

- Source item 1: `order: 2`
- Source item 2: `order: 3`
- Source item 3: `order: 1`
- Source item 4: `order: 3`
- Source item 5: `order: 1`

These items would be displayed on the page in the following order:

- Source item 3: `order: 1`
- Source item 5: `order: 1`
- Source item 1: `order: 2`
- Source item 2: `order: 3`
- Source item 4: `order: 3`

Play around with the values in this live example below and see how that changes the order. Also, try changing `flex-direction` to `row-reverse` and see what happens — the start line is switched so the ordering begins from the opposite side.

```
<div class="box">
  <div><a href="#">1</a></div>
  <div><a href="#">2</a></div>
  <div><a href="#">3</a></div>
  <div><a href="#">4</a></div>
  <div><a href="#">5</a></div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
}

.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-direction: row;
}
.box :nth-child(1) {
  order: 2;
}
.box :nth-child(2) {
  order: 3;
}
.box :nth-child(3) {
  order: 1;
}
.box :nth-child(4) {
  order: 3;
}
.box :nth-child(5) {
  order: 1;
}
```

Flex items have a default `order` value of `0`. Therefore, items with an integer value greater than `0` will be displayed after any items that have not been given an explicit `order` value.

You can also use negative values with `order`, which can be quite useful. If you want to make one item display first and leave the order of all other items unchanged, you can give that item order of `-1`. As this is lower than `0`, the item will always be displayed first.

In the live code example below, the items are laid out using flexbox. By changing which item has the class `active` assigned to it in the HTML, you can change which item displays first and therefore becomes full width at the top of the layout, with the other items displaying below it.

```
<div class="box">
  <div><a href="#">1</a></div>
  <div><a href="#">2</a></div>
  <div class="active"><a href="#">3</a></div>
  <div><a href="#">4</a></div>
  <div><a href="#">5</a></div>
</div>
```

```
* {
  box-sizing: border-box;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
  flex-direction: row;
}

.active {
  order: -1;
  flex: 1 0 100%;
}
```

The items are displayed in *order-modified document order* meaning the value of the `order` property is taken into account before the items are displayed.

`Order` also changes the paint order of the items; items with a lower value for `order` are painted first and items with a higher value for `order` painted afterwards.

The order property and accessibility

Use of the `order` property has the same implications for accessibility as changing the direction with `flex-direction`. Using `order` changes the order in which items are painted, and the order in which they appear visually. It does not change the sequential navigation order of the items. Therefore, if a user is using a keyboard to tab through the content on the page, they could find themselves jumping around your content in a very confusing way.

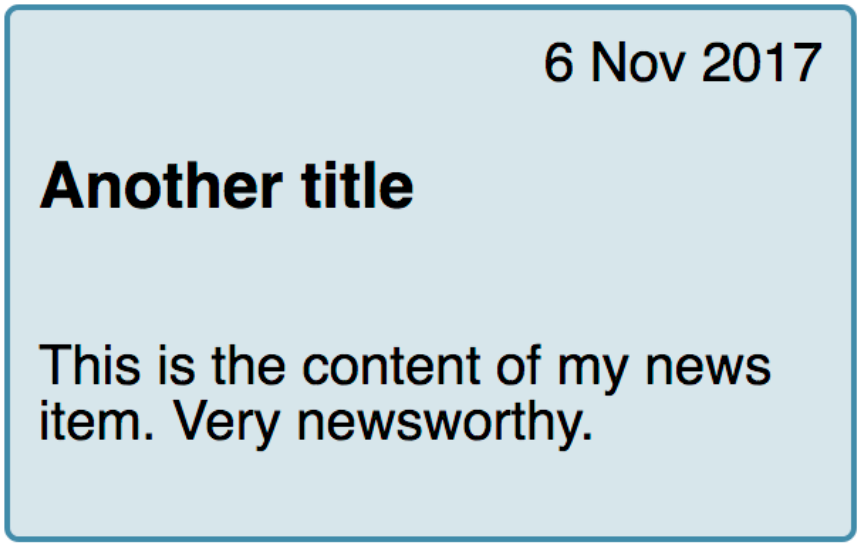
By tabbing around any of the live examples on this page, you can see how `order` is potentially creating a strange experience for anyone not using a pointing device like a mouse. To read more about this disconnect of visual order and logical order and some of the potential problems it raises for accessibility, see the following resources.

- Flexbox and the keyboard navigation disconnect via tink.uk (2016)
- HTML Source Order vs CSS Display Order via adrianroselli.com (2015)
- The Responsive Order Conflict for Keyboard Focus via alastairc.uk (2017)

Use cases for <code>order</code>

There are some use cases for which the fact that the logical and therefore reading order of flex items is separate from the visual order, is helpful. Used carefully, the `order` property can allow for some useful common patterns to be easily implemented.

You might have a design, perhaps a card that will display a news item. The heading of the news item is the key thing to highlight and would be the element that a user might jump to if they were tabbing between headings to find the content they wanted to read. The card also has a date; the finished design we want to create is something like this.



Visually, the date appears above the heading, in the source. However, if the card was read out by a screen reader I would prefer that the title was announced first and then the publication date. We can accomplish this with the `order` property.

The card is our flex container, with `flex-direction` set to `column`. We give the date an `order` of `-1`, placing it above the heading.

```
<div class="wrapper">
  <div class="card">
    <h3>News item title</h3>
    <div class="date">1 Nov 2017</div>
    <p>This is the content of my news item. Very newsworthy.</p>
  </div>
  <div class="card">
    <h3>Another title</h3>
```

```
    <div class="date">6 Nov 2017</div>
    <p>This is the content of my news item. Very newsworthy.</p>
  </div>
</div>
```

```
body {
  font-family: sans-serif;
}

.wrapper {
  display: flex;
  flex: 1 1 200px;
  gap: 1em;
}

.card {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 1em;
  display: flex;
  flex-direction: column;
}

.date {
  order: -1;
  text-align: right;
}
```

These small tweaks are the sort of cases where the `order` property makes sense. Keep the logical order the same as the reading and tab order of the document, and maintain that in the most accessible and structured fashion. Then use `order` for purely visual design tweaks. Don't reorder items that receive keyboard focus. Ensure you always test your content using only a keyboard rather than a mouse or a touchscreen; this will reveal if your development choices make it more complex to navigate the content.

See also

- Basic concepts of flexbox
- Relationship of flexbox to other layout methods
- Aligning items in a flex container
- Controlling ratios of flex items along the main axis
- Mastering wrapping of flex items
- Typical use cases of flexbox
- CSS flexible box layout module

Relationship of flexbox to other layout methods

In this article, we will take a look at how flexbox fits in with all the other CSS modules. We'll find out which specifications you also need to take notice of if you want to learn flexbox, and find out why flexbox is different to some other modules.

The box alignment module

Many people first look at flexbox when they want to properly align flex items inside a flex container. Flexbox provides access to properties that allow the alignment of items on their cross axis and justification of items on the main axis.

Flexbox was originally defined in its own flexible box layout module, but the properties and values that are common to other layout methods are defined in the CSS box alignment module. This module details how alignment, justification, gaps, and gutters work in all layout systems — not just flexbox. When a feature is defined in both specifications, note that the box alignment module supersedes the flexible box layout module.

Writing Modes

In the Basic concepts of flexbox article, it is noted that flexbox is **writing mode aware**. Writing modes are fully detailed in the CSS writing modes module, which details how CSS supports the different writing modes that exist internationally. We need to be aware of how this will impact our flex layouts as the writing mode changes the direction that blocks are laid out in our document. Understanding **block** and **inline** directions is key to new layout methods.

It is worth noting that we might want to change the writing mode of our document for reasons other than publishing content in a language that uses a different writing mode. The CSS writing modes module defines how text can be written horizontally, left-to-right and right-to-left, and vertically, top-to-bottom. This is important for internationalization and translations, but these feature can also be used for creative designs.

The writing modes

The writing modes specification defines the following values of the `writing-mode` property, which serve to change the direction that blocks are laid out on the page, to match the direction that blocks lay out when content is formatted in that particular writing mode. You can change the live example below to these modes in order to see what happens to the flex layout.

- `horizontal-tb`
- `vertical-rl`
- `vertical-lr`
- `sideways-rl`
- `sideways-lr`

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  writing-mode: horizontal-tb;
}
```

Note that you would not normally use CSS and the `writing-mode` property to change an entire document to another writing mode. This would be done via HTML, by adding a `dir` and `lang` attribute to the `<html>` element to indicate the document language and default text direction. This would mean that the document would display correctly even if CSS did not load.

Flexbox and other layout methods

Some properties were designed assuming that content is laid out using the standard block layout system, and don't apply in the context of flex layout. An element set to `display: flex` behaves in most ways like any other block-level container that establishes a containing block. Floats will not intrude, and the containers' margins will not collapse.

With regard to flex items, if an item was floated or cleared and then becomes a flex item due to the parent having `display: flex` applied, the floating and clearing will no longer happen, and the item will not be taken out of normal flow in the way that floats are. If you have used the `vertical-align` property, as used with `inline-block` or table layout for alignment, this will no longer affect the item and you can use the alignment properties of flexbox instead.

In this next live example the child elements have been floated, and then their container has had `display: flex` added. If you remove `display: flex`, you should see that the `.box` element collapses as we have no clearing applied. This demonstrates that the float is happening. Re-apply `display: flex` and the collapsing does not happen. This is because the items no longer have a float applied, as they have been transformed into flex items.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  float: left;
}
```

Flexbox and grid layout

CSS grid layout and flexbox share many properties and values. For divergent properties, if a flex item becomes a grid item, any `flex` values assigned to the child elements, such as `flex-end`, will be ignored. As noted above, values defined in the box alignment module that work across both layout methods supersede those defined in flexbox only.

Flex and grid — what's the difference?

A common question is to ask what the difference is between flexbox and CSS grid layout — why do we have two specifications that sometimes appear to be doing the same thing?

The most straightforward answer to this question is defined in the specifications themselves. Flexbox is a one-dimensional layout method whereas grid layout is a two-dimensional layout method. The example below has a flex layout. As already described in the Basic concepts article, flex items can be allowed to wrap but, once they do, each line behaves as if it were a flex container of its own. When space is distributed, flexbox does not look at the placement of items in other rows and tries to line things up with each other.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
  padding: 1em;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 1em;
  flex: 1 1 200px;
}
```

If we create a very similar layout using grid, we can control the layout in both rows and columns.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  padding: 1em;
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, auto));
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  padding: 1em;
  background-color: rgb(96 139 168 / 0.2);
}
```

These examples point to another key difference between these layout methods. In grid layout, you do the majority of sizing specification on the container, setting up tracks and then placing items into them. In flexbox, while you create a flex container and set the direction at that level, any control over item sizing needs to happen on the items themselves.

In some cases, you could use either layout method. As you become confident with both, you will find each one is better suited to specific layout needs, and you'll end up with both methods in your CSS. There is rarely a right or wrong answer.

As a general rule, if you are setting widths on flex items to make items in one row of a wrapped flex container line up with the items above them, you should instead opt for a two-dimensional grid layout.

There are no set rules like "you should use flexbox for small components and grid layout for larger ones." A tiny component can be two dimensional, and a large layout can be represented better with layout in one dimension. Try things out — you have a choice of layout methods, so take advantage of it.

For more comparisons of grid and flexbox see the article Relationship of grid layout to other layout methods. This article details many of the ways that grid layout differs from flex layout and demonstrates some of the extra functionality you get when using grid layout such as layering of items on the grid. This may also help you decide which layout method to use.

Flexbox and display: contents

The `contents` value of the `display` property is described in the spec as follows:

> "The element itself does not generate any boxes, but its children and pseudo-elements still generate boxes as normal. For the purposes of box generation and layout, the element must be treated as if it had been replaced with its children and pseudo-elements in the document tree."

This value of `display` controls box generation, and whether the element should generate a box that we can style and see on the page, or whether instead the box it would normally create should be removed and the child elements essentially moved up to participate in whatever layout method the parent would have been part of. This is easier to see with an example.

In the following live example, we have a flex container containing three flex items. One has two elements nested inside it, which would not ordinarily participate in flex layout. Flex layout only applies to the direct children of a flex container.

By adding `display: contents` to the wrapper around the nested elements, you can see that the item has disappeared from the layout, allowing the two sub-children to be laid out as if they were direct children of the flex container. You can try removing the `display: contents` line to see it return.

Note that this only removes the box from the layout; the sub-children don't become direct children in any other way. We used a direct child selector to add the background and borders to the flex items; it has not been applied to our nested children. They have been laid out as flex items, but as they are not direct children they do not get the other styling.

As the box is removed, you cannot then use it to — for example — add a background color behind the nested sub children. If you remove `display: contents` in this live example you will see that the direct child we are removing has an orange background color. This also disappears when the box disappears.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div class="nested">
    <div>Sub-item 1</div>
    <div>Sub-item 2</div>
  </div>
</div>
```

```
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  padding: 1em;
  background-color: rgb(96 139 168 / 0.2);
}

.box {
  border: 2px dotted rgb(96 139 168);
  padding: 1em;
  display: flex;
}

.nested {
  background-color: orange;
  display: contents;
}
```

> **Warning:** Some browsers incorrectly remove some elements with `display: contents` from the accessibility tree (but descendants will remain), removing those elements' semantics while maintaining their child content. This means the element itself may not be announced by screen readers. See `display: contents` and `display: contents` considered harmful.

Typical use cases of flexbox

In this guide, we will take a look at some of the common use cases for flexbox — those places where flexbox makes more sense than another layout method.

Why choose flexbox?

Flexbox is generally the right CSS layout solution when you want to lay out a collection of items in one dimension or control the spacing between items. In this guide, we'll look at some of the typical use cases of flexbox.

Navigation

A common pattern for navigation is to have a list of items displayed as a horizontal bar. It's probably the most common of flexbox examples, and could be considered an ideal flexbox use case.

When we have a set of items that we want to display horizontally, we may well end up with additional space. We need to decide what to do with that space, and have a couple of options. We either display the space outside of the items — therefore spacing them out with white space between or around them — or we absorb the extra space inside the items and therefore need a method of allowing the items to grow and take up this space.

Space distributed outside the items

To distribute the space between or around the items, we use the alignment properties in flexbox, and the `justify-content` property. You can read more about this property in Aligning items in a flex container, which deals with aligning items on the main axis.

In this example, we display the items at their natural size and use `justify-content: space-between` to space the items equally. You can change how the space is distributed using the `space-around` or `space-evenly` values. You could also use `start` to put the space at the end of the items, `end` to place it before them, or `center` to center the navigation items.

```
<nav>
  <ul>
    <li><a href="#">Page 1</a></li>
    <li><a href="#">Page 2</a></li>
    <li><a href="#">Page 3 is longer</a></li>
    <li><a href="#">Page 4</a></li>
```

```
  </ul>
</nav>
```

```
nav {
  border: 2px solid #eeeeee;
}

nav a {
  text-decoration: none;
  color: black;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  display: block;
}

nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: space-between;
}
```

Space distributed within the items

A different pattern for navigation would be to distribute the available space within the items themselves, rather than create space between them. The `flex` properties allow items to grow and shrink in proportion to one another as described in Controlling ratios of flex items along the main axis.

If you wanted to respect the size property of your navigation items but have the available space shared out equally among them, you might use `flex: auto`, which is the shorthand for `flex: 1 1 auto` — all items grow and shrink from a flex-basis of `auto`. This would mean that the longer item would have more space because it started from a larger size, even though the same amount of available space is assigned to it as the others.

In the live example below try changing `flex: auto` to `flex: 1`. This shorthand for `flex: 1 1 0` causes all of the items to become the same width, as they are working from a `flex-basis` of `0`, allowing all of the space to be distributed evenly.

```
<nav>
  <ul>
    <li><a href="#">Page 1</a></li>
    <li><a href="#">Page 2</a></li>
    <li><a href="#">Page 3 is longer</a></li>
    <li><a href="#">Page 4</a></li>
  </ul>
</nav>
```

```
nav {
  border: 2px solid #eeeeee;
}
nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
}

nav a {
  text-decoration: none;
  color: black;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  display: block;
}

nav li {
  flex: auto;
}
```

Split navigation

Another way to align items on the main axis is to use auto margins. This enables the design pattern of a navigation bar where one group of items are aligned left and another group aligned right. Here we are using the auto margins technique described in Using auto margins for main axis alignment.

The items are aligned on the main axis with `normal`, which behaves as `start`, as this is the initial behavior of flexbox. The `gap` property creates gaps between items. And we are aligning the last item to the right by giving it a `margin-left` value of `auto`. You can move the class from one item to another to change where the split happens.

```
<nav>
  <ul>
    <li><a href="#">Page 1</a></li>
    <li><a href="#">Page 2</a></li>
    <li><a href="#">Page 3 is longer</a></li>
    <li class="push-right"><a href="#">Page 4</a></li>
  </ul>
</nav>
```

```
nav {
  border: 2px solid #eeeeee;
}

nav a {
  text-decoration: none;
  color: black;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  display: block;
```

```
}

nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  gap: 20px;
}

.push-right {
  margin-left: auto;
}
```

Center item

A long-standing joke among developers is that the hardest problem in web design is vertical centering. Vertically centering content is very straightforward with flexbox alignment properties, as the following live example shows.

Click **"Play"** and try changing the alignment, for example aligning the item to the start with `start` or end with `end`:

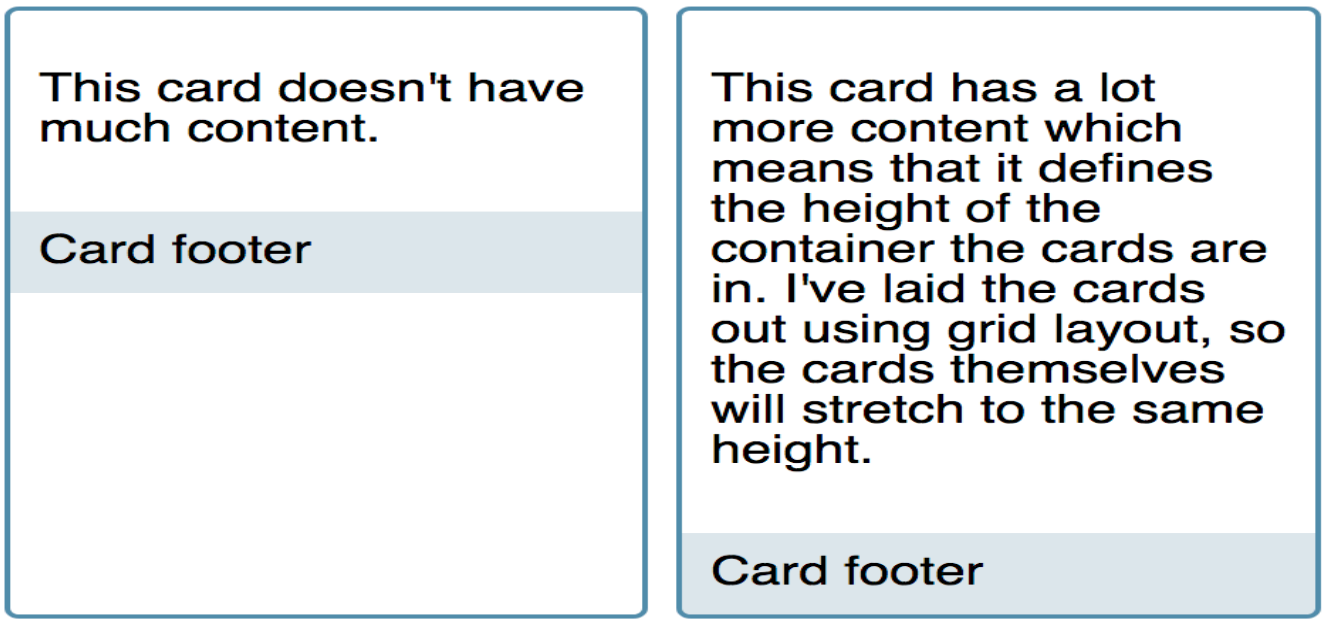
```
<div class="box">
  <div></div>
</div>
```

```
.box {
  height: 300px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  align-items: center;
  justify-content: center;
}

.box div {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  width: 100px;
  height: 100px;
}
```

With CSS box alignment properties, you can vertically center an element inside another without flexbox. In the example above, try removing the flex properties from the box and adding `align-content: center`. Then add `margin: auto` to the element you want to horizontally center.

Card layout pushing footer down

Whether you use flexbox or grid to lay out a list of card components, these layout methods only work on direct children of the flex or grid component. This means that if you have variable amounts of content, the card will stretch to the height of the grid area or flex container. Any content inside uses regular block layout, meaning that on a card with less content the footer will rise up to the bottom of the content rather than stick to the bottom of the card.



Flexbox solves this. We make the card a flex container, with `flex-direction: column`. We then set the content area to `flex: 1`, which is the shorthand for `flex: 1 1 0` — the item can grow and shrink from a flex basis of `0`. As this is the only item that can grow, it takes up all available space in the flex container and pushes the footer to the bottom. If you remove the `flex` property from the live example you will see the footer moves up to sit directly under the content.

```
<div class="cards">
  <div class="card">
    <div class="content">
      <p>This card doesn't have much content.</p>
    </div>
    <footer>Card footer</footer>
  </div>
  <div class="card">
    <div class="content">
      <p>
        This card has a lot more content which means that it defines the height
        of the container the cards are in. I've laid the cards out using grid
```

```
        layout, so the cards themselves will stretch to the same height.
      </p>
    </div>
    <footer>Card footer</footer>
  </div>
</div>
```

```
body {
  font-family: sans-serif;
}
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  grid-gap: 10px;
}

.card {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  display: flex;
  flex-direction: column;
}

.card .content {
  padding: 10px;
  flex: 1 1 auto;
}

.card footer {
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
}
```

Media objects

The media object — an image or other media element with some descriptive text side-by-side — is a common pattern in web design. Media objects should be able to be flipped — moving the image from one side to the other.

This pattern is used for comments and other places where images are placed next to their descriptions. We can use flexbox to allow the part of the media object containing the image to take its sizing information from the image with the content of the media object flexing to take up the remaining space.

In this example, the media object is aligned to `flex-start` and the `.content` is set to grow, with the grow factor set to `1`. These properties are the same as those used for our column layout card pattern above.

```
<div class="media">
  <div class="image">
    <img
      alt="A colorful balloon against a blue sky"
      src="https://mdn.github.io/shared-assets/images/examples/balloon.jpg" />
  </div>
  <div class="content">
    This is the content of my media object. Items directly inside the flex
    container will be aligned to flex-start.
  </div>
</div>
```

```
img {
  max-width: 100%;
  display: block;
}

.media {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  align-items: flex-start;
}

.media .content {
  flex: 1;
  padding: 10px;
}
```

Some things that you might want to try in this live example relate to the different ways you might want to constrain the media object in your design.

To prevent the image from growing too large, you should add a `max-width` to the image. As that side of the media object uses the initial values of flexbox, it can shrink, but not grow, and uses a `flex-basis` of auto. Any `width` or `max-width` applied to the image will become the `flex-basis`.

```
.image img {
  max-width: 100px;
}
```

You could also allow both sides to grow and shrink in proportion. If you set both sides to `flex: 1`, they will grow and shrink from a `flex-basis` of `0`, so you will end up with two equal-sized columns. You could either take the content as a guide and set both to `flex: auto`, in which case they would grow and shrink from the size of the content or any size applied directly to the flex items such as a `width` on the image.

```
.media .content {
  flex: 1;
  padding: 10px;
}

.image {
  flex: 1;
}
```

You could also give each side different `flex-grow` factors, for example setting the side with the image to `flex: 1` and the content side to `flex: 3`. This will mean they use a `flex-basis` of `0` but distribute that space at different rates according to the `flex-grow` factor you have assigned. The flex properties we use to do this are described in detail in the guide Controlling ratios of flex items along the main axis.

```
.media .content {
  flex: 3;
  padding: 10px;
}
```

```
.image {
  flex: 1;
}
```

Flipping the media object

To switch the display of the media object and have the image on the right and content on the left, we set the `flex-direction` property to `row-reverse`.

In this example, we added a `flipped` class next to the `media` class. Remove the class from the HTML to see how the display changes.

```
<div class="media flipped">
  <div class="image">
    <img
      alt="A colorful balloon against a blue sky"
      src="https://mdn.github.io/shared-assets/images/examples/balloon.jpg" />
  </div>
  <div class="content">
    This is the content of my media object. Items directly inside the flex
    container will be aligned to flex-start.
  </div>
</div>
```

```
img {
  max-width: 100%;
  display: block;
}

.media {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  align-items: flex-start;
}

.flipped {
  flex-direction: row-reverse;
}

.media .content {
  flex: 1;
  padding: 10px;
}
```

Form controls

Flexbox is particularly useful when it comes to styling form controls. Forms have several small elements that we typically want to align with each other. A common pattern is to have a `<label>` and `<input>` pair combined with a `<button>`, perhaps for a search form or a newsletter sign-up form where you want your visitor to enter their email address.

Flexbox makes this type of layout achievable with just a few declarations. The `<label>`, `<input>` and `<button>` are contained in a wrapper that is set to `display: flex`. The flex properties allow the `<input>` field to grow, while the button and label do not grow. The text input field will grow and shrink depending on the space available.

```
<div class="wrapper">
  <label for="text">Label</label>
  <input id="text" type="text" />
  <button type="button">Send</button>
</div>
```

```
* {
  font: 1.1em sans-serif;
}

.wrapper {
  display: flex;
  border: 1px solid rgb(96 139 168);
}
.wrapper > * {
  padding: 10px;
  border: none;
  color: white;
}
.wrapper > input[type="text"] {
  background-color: rgb(96 139 168 / 0.5);
  border-right: 1px solid rgb(96 139 168);
  flex: 1 1 auto;
}
.wrapper button {
  background-color: rgb(96 139 168);
  color: white;
}
.wrapper label {
  background-color: #666666;
}
```

Patterns like this can make it easier to create a library of form elements for your design, which easily accommodate additional elements being added. You are taking advantage of the flexibility of flexbox by mixing items that do not grow with those that do.

Conclusion

While exploring the above patterns, you have hopefully started to see how you can think through the best way to use flexbox to achieve the result that you want. Quite often you have more than one choice. Mix items that cannot stretch with those that can, use the content to inform the size, or allow flexbox to share out space in proportion. It's up to you.

Think about the best way to present the content that you have and then see how flexbox or other layout methods can help you achieve it.

Mastering wrapping of flex items

Flexbox was designed as a single-dimensional layout tool — it deals with laying out items as a row or column — but not both at once. It is, however, possible to wrap flex items onto new lines, creating new rows if `flex-direction` is `row` and new columns if `flex-direction` is `column`. This guide explains flexbox wrapping, what it is designed for, and what situations require CSS grid layout rather than flexbox.

Making things wrap

The initial value of the `flex-wrap` property is `nowrap`. This means if a set of flex items is too wide for their flex container, they will overflow it. To make them wrap once they are too wide, add the `flex-wrap` property with a value of `wrap`, or use the shorthand `flex-flow` with values of `row wrap` or `column wrap`. Items will then wrap onto new lines when they overflow their container.

In this example, there are ten flex items with a `flex-basis` of `160px` that can grow and shrink. Once there is not enough space to place another 160 pixel item in a row, a new flex line is created. New lines are created as needed until all of the items are placed. As the items can grow, they will expand to fill each row completely. If there is only one item on the final line it will stretch to fill the entire line.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

```
.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 160px;
}
```

The same thing happens with flex columns. To wrap and create new columns, the container needs to have a height. In the case of columns, items stretch vertically to fill each column completely.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  height: 300px;
  display: flex;
  flex-direction: column;
  flex-wrap: wrap;
}
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 80px;
}
```

Wrapping and flex-direction

Wrapping works as you might expect when combined with `flex-direction`. If `flex-direction` is set to `row-reverse` then the items will start from the end edge of the container and lay themselves out in reverse ordered lines.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
  flex-direction: row-reverse;
  width: 500px;
}
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 160px;
}
```

Note that the reversing is only happening in the inline, row direction. We start on the right then go onto the second line and again start from the right. We aren't reversing in both directions, starting from the bottom coming up the container!

Balanced wrapping

One potential issue with flexbox wrapping is that, by default, the flex items are not distributed evenly across the lines. In the previous example, we have three items on the first three lines, but only one on the last line. We can distribute the flex items more evenly across the four flex lines by including the `balance` keyword within the `flex-wrap` value, alongside the `wrap` or `wrap-reverse` keyword.

This example uses the same HTML as the previous example, and nearly the same CSS, except that the `flex-wrap` value has been changed from `wrap` to `wrap balance`.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap balance;
  flex-direction: row-reverse;
  width: 500px;
}
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 160px;
}
```

```
@supports not (flex-wrap: balance) {
  body::before {
    content: "Your browser does not support flex-wrap: balance.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

```
flex-wrap: wrap balance;
```

This line could also have been written as just `flex-wrap: balance`. If `balance` is specified as the only keyword in the `flex-wrap` value, the other keyword defaults to `wrap`. We wrote it out explicitly so it is clearer what is going on. Note that including the `balance` keyword with the `nowrap` value is invalid, and the declaration will be ignored.

The updated rendering looks like so:

Note how the items are now distributed more evenly, or "balanced," with two lines of three items and two lines of two items.

If you want to distribute your balanced flex items across a greater number of lines, you can do so using the `flex-line-count` property. This specifies a minimum number of lines. If we add the following to the previous example:

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap balance;
  flex-line-count: 5;
  flex-direction: row-reverse;
  width: 500px;
}
.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 160px;
}
```

```
flex-line-count: 5;
```

We end up with the following:

We now have five lines of two flex items.

The `flex-line-count` property has no effect on flex-containers that do not have the `balance` keyword included in their `flex-wrap` property values.

Single-dimensional layout explained

As we have seen from the above examples if our items are allowed to grow and shrink, when there are fewer items in the last row or column then those items grow to fill the available space.

There are no flexbox features to tell items in one row to line up with items in the row above — each flex line acts like a new flex container. It deals with space distribution across the main axis. If there is only one item, and that item is allowed to grow, it will fill the axis just as if you had a single item flex container. If you want layout in two dimensions then you probably want grid layout.

This example demonstrates the difference, using CSS grid layout to create a layout with as many columns of at least `160px` as will fit, distributing the extra space between all columns. We use the same HTML as the flexbox wrapped row example above but set `display: grid` on it. Instead of the `flex` shorthand, which has no effect outside of flexbox, we set the item's minimum width and ability to grow directly on the container with `grid-template-columns`. With CSS grid, the last item stays in its grid cell; grid items don't stretch when there are fewer of them in the last row.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
```

```
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));
  width: 500px;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

This is the difference between one and two-dimensional layouts. In a one-dimensional layout method like flexbox, we only control the row or column. In a two-dimensional grid layout, we control both at the same time. If you want the space distribution row by row, use Flexbox. If you don't, use CSS grid.

How do flexbox-based grid systems work?

Flexbox-based layouts can be forced to line up as grid systems, but that is not the intended purpose of flexbox. If you assign percentage widths to flex items — either using `flex-basis` or by adding a width to the item itself and leaving the value of `flex-basis` as `auto` — you can give the impression of a two-dimensional layout.

In this example, `flex-grow` and `flex-shrink` have been set to `0` to make inflexible flex items. The flexibility is controlled via percentages.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.box {
  width: 500px;
  border: 2px dotted rgb(96 139 168);
  display: flex;
  flex-wrap: wrap;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 0 0 33.3333%;
}
```

This technique allows you to line up flex items on the cross-axis. However, when you catch yourself adding widths to flex items in this way or adding empty flex items to take up space, it is a good indication you may want to switch to CSS grid layout for that component.

Creating gutters between items

To create gaps or gutters between flex items, use the gap property directly on the flex container to create a fixed space between adjacent flex items. The `gap` property is a shorthand for `row-gap` and `column-gap`. These properties specify the size of the gutters between rows and columns within grid, flex, and multi-column layouts.

The `gap` property is not the only thing that can add space between items. Margins, padding, `justify-content`, and `align-content` can also increase the size of the gutter, affecting the actual size of the gap.

To see how the `gap` property differs from `margin` in both axes, try changing the `gap` value in the container `.box` and adding a `margin` value to the `.box > *` rule in the stylesheet below. Click the "Reset" button to revert to the previous values.

```
<div class="wrapper">
  <div class="box">
    <div>One</div>
    <div>Two</div>
    <div>Three</div>
    <div>Four</div>
    <div>Five</div>
    <div>Six</div>
    <div>Seven</div>
    <div>Eight</div>
    <div>Nine</div>
    <div>Ten</div>
  </div>
</div>
```

```
.wrapper {
  border: 2px dotted rgb(96 139 168);
  width: 500px;
}
.box {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
.box > * {
  flex: 1 1 160px;
```

```
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
```

Collapsed items

The flexbox specification details what should happen if a flex item is collapsed by setting `visibility: collapse` on an item. See the MDN documentation for the `visibility` property. The specification describes the behavior as follows:

> "Specifying `visibility: collapse` on a flex item causes it to become a *collapsed flex item*, producing an effect similar to `visibility: collapse` on a table-row or table-column: the collapsed flex item is removed from rendering entirely, but leaves behind a "strut" that keeps the flex line's cross-size stable. Thus, if a flex container has only one flex line, dynamically collapsing or uncollapsing items may change the flex container's main size, but is guaranteed to have no effect on its cross size and won't cause the rest of the page's layout to "wobble". Flex line wrapping *is* re-done after collapsing, however, so the cross-size of a flex container with multiple lines might or might not change." - Collapsed items

This behavior is useful if you want to target flex items using JavaScript to show and hide content for example. The example in the specification demonstrates one such pattern.

In the following live example, the non-wrapping flex container contains a row with three flex items that are set to flex to equal sizes. The third item has multiple lines of content, growing the container. The default for `align-items` is `normal`; for flex items, `normal` behaves as `stretch`, so all the items stretch by default, filling the container's cross-size height.

The item creating the cross-size is set to `visibility: collapse`, which collapses or hides the flex item, depending on the browser. In either case, the flex container retains a *strut* of the cross-size even though it is not visible. This way, if the item is made visible, the cross-size of the single-line flex container will not change. If you remove `visibility: collapse` from the CSS or change the value to `visible`, you will see the item appear, and the main-size space is redistributed between non-collapsed items, while the cross-size remains unchanged.

> **Note:** Use Firefox for the example below as other common browsers treat `collapse` as `hidden`.

```
<p>
  <label><input type="checkbox" /> Toggle <code>visibility</code> value</label>
</p>
```

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div class="collapse">Three <br />has <br />extra <br />text</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  display: flex;
  width: 600px;
}
.box > * {
  flex: 1 1 200px;
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}
.collapse {
  visibility: collapse;
}
```

```
p:has(:checked) + div .collapse {
  visibility: visible;
}
```

The above was a single-line, non-wrapping flex container with a set size of `600px` so whether the item is visible or collapsed, the width is the same. It is important to understand that while the container retains a strut of the collapsed item's cross-size, the main size is not preserved. Multi-line flex containers rewrap their items after removing collapsed items from rendering. The new space that a collapsed item leaves in the main direction may cause non-collapsed items to be placed in a different line than if the item were not collapsed. Because each line is laid out like an independent single-line flex container and its composition may change after collapsing, its cross-axis size may change too.

The following example shows this behavior. The third flex item is collapsed, so it occupies zero space along the main axis (the inline-size is 0). When collapsed, its strut is on the first row after the fourth item, with the first row being tall enough to fit the three lines of text that the third item would have had. Then, if you uncollapse the item (e.g., by removing the `collapse` class), there is no longer enough horizontal space for the fifth item on the first row, and it moves to the second. This causes the second row to grow to fit the two lines of text of its new member, and the last flex item is pushed onto a new row. With a taller second line and a new third line, the flex container is much taller than it was before.

> **Note:** Use Firefox for the example below as other common browsers treat `collapse` as `hidden`.

```
<p>
  <label><input type="checkbox" /> Toggle <code>visibility</code> value</label>
</p>
```

```
<div class="box">
  <div>One</div>
  <div>Two is the width of this sentence.</div>
  <div class="collapse">Three <br />is <br />five <br />lines <br />tall.</div>
  <div>Four</div>
  <div>Five<br />Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
  <div>Eleven is longer</div>
</div>
```

```
.box {
  border: 2px dotted rgb(96 139 168);
  width: 500px;
  display: flex;
  flex-wrap: wrap;
}
.box > * {
  padding: 10px;
  border: 2px solid rgb(96 139 168);
```

```
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  flex: 1 1 auto;
  min-width: 50px;
}
.collapse {
  visibility: collapse;
}
```

```
p:has(:checked) + div .collapse {
  visibility: visible;
}
```

If this causes a problem for your layout, it may require a rethinking of the structure, for example, putting each row into a separate flex container so that they can't shift rows.

Using <code>visibility: hidden</code> and <code>display: none</code>

In the previous live examples, try using `visibility: hidden` or `display: none` instead of `visibility: collapse`. Using `visibility: hidden`, the item is made invisible, but the box is kept in the formatting structure, so it still behaves as if it were part of the layout. When you use `display: none`, the item is completely removed from the formatting structure. Not only is it invisible but the structure is removed as well. This means counters ignore it and things like transitions do not run.

CSS font loading

The **CSS font loading** module describes events and interfaces used for dynamically loading font resources.

Reference

Interfaces

- `fontFace` interface
  - `FontFace()` constructor
  - `fontFace.family` property
  - `fontFace.style` property
  - `fontFace.weight` property
  - `fontFace.stretch` property
  - `fontFace.unicodeRange` property
  - `fontFace.variant` property
  - `fontFace.featureSettings` property
  - `fontFace.variationSettings` property
  - `fontFace.display` property
  - `fontFace.ascentOverride` property
  - `fontFace.descentOverride` property
  - `fontFace.lineGapOverride` property
  - `fontFace.load()` method (returns a promise)
- `fontFaceSet` interface
- `fontFaceSetLoadEvent` event

Guides

CSS font loading API

Overview of the CSS Font Loading API, which provide events and interfaces for dynamically loading font resources.

Related concepts

- CSS `@font-face` at-rule
- CSS `@font-feature-values` at-rule
- `CSSFontFaceRule` interface
- Document `fonts` property (returns the `FontFaceSet` object instance)
- WorkerGlobalScope `fonts` property (returns the `FontFaceSet` object instance)
- JavaScript `Promise` object

See also

- CSS fonts module

CSS fonts

The **CSS fonts** module defines font-related properties and how font resources are loaded. It lets you define the style of a font, such as its family, size and weight, and the glyph variants to use when multiple are available for a single character.

A font is a resource file containing the visual representation of characters, mapping character codes to glyphs that represent letters, numbers, punctuation and even emojis of a typeface. A font family is a group of fonts that share common design styles and font properties, with each member of the group providing different ways of displaying the glyphs, varying by stroke weight, slant, or relative width, among other attributes. A font typically represents a single style of a typeface, such as Helvetica that is Bold and Italic. A font family is the complete set of styles. Including such a font in a document or design is done by defining a separate `@font-face` declaration for each font resource.

The properties, at-rules, and descriptors of the CSS fonts module enable the downloading of multiple variations of a font. They also define the font file to use for a particular font characteristic, along with fallback instructions in case a resource fails to load. The CSS font selection mechanism describes the process of matching a given set of CSS font properties to a single font face.

The CSS fonts module also supports variable fonts. Unlike regular fonts, where each style is implemented as a separate font file, variable fonts can contain all styles within a single file. By using a single `@font-face` declaration, you can import a variable font that includes all the styles. Depending on the font, this can include a multitude of font variants. Variable fonts are a part of the OpenType font specification.

Reference

Properties

- `font` shorthand

- `font-family`
- `font-feature-settings`
- `font-kerning`
- `font-language-override`
- `font-optical-sizing`
- `font-palette`

- `font-size`
- `font-size-adjust`
- `font-width` and its legacy alias `font-stretch`
- `font-style`
- `font-weight`
- `font-synthesis` shorthand
- `font-synthesis-position`
- `font-synthesis-small-caps`
- `font-synthesis-style`
- `font-synthesis-weight`
- `font-variant` shorthand
- `font-variant-alternates`
- `font-variant-caps`
- `font-variant-east-asian`
- `font-variant-emoji`
- `font-variant-ligatures`
- `font-variant-numeric`
- `font-variant-position`
- `font-variation-settings`

At-rules and descriptors

At-rule: `@font-face`

Descriptors:

- `ascent-override`
- `descent-override`
- `font-display`
- `font-family`
- `font-feature-settings`
- `font-width` and its legacy alias `font-stretch`
- `font-style`
- `font-variation-settings`
- `font-weight`
- `line-gap-override`
- `size-adjust`
- `src`
- `unicode-range`

The CSS fonts module also defines the `font-language-override`, `font-named-instance`, `font-size`, `subscript-position-override`, `subscript-size-override`, `superscript-position-override`, and `superscript-size-override` descriptors. Currently, no browsers support these features.

- At-rule: `@font-feature-values`

The CSS fonts module also defines the `font-display` descriptor. Currently, no browsers support this feature.

At-rule: `@font-palette-values`

Descriptors:

- `base-palette`
- `font-family`
- `override-colors`

Functions

The CSS fonts module defines the `generic()` function. Currently, no browsers support this feature.

Data types

`font-size` types:

- `<absolute-size>`
- `<relative-size>`

`font-family` type:

- `<generic-family>`

`font-feature-settings` type:

- `<feature-tag-value>`

`font-format` type:

- `<font-format>`

`font-stretch` type:

- `<font-stretch-css3>`

`font-tech` types:

- `<color-font-tech>`
- `<font-features-tech>`
- `<font-tech>`

`font-variant` types:

- `<font-variant-css2>`

- `<east-asian-variant-values>`
- `<east-asian-width-values>`

`font-variant-ligatures` types:

- `<common-lig-values>`
- `<contextual-alt-values>`
- `<discretionary-lig-values>`
- `<historical-lig-values>`

`font-variant-numeric` types:

- `<numeric-figure-values>`
- `<numeric-fraction-values>`
- `<numeric-spacing-values>`

`font-weight` type:

- `<font-weight-absolute>`

Interfaces

- `CSSFontFaceRule`
- `CSSFontFeatureValuesRule`
- `CSSFontPaletteValuesRule`

Guides

Learn: Fundamental text and font styling

> This beginner's learning article covers the basic fundamentals of text and font styling. It covers how to set the font weight, family, and style by using the `font` shorthand and how to align text and manage line and letter spacing.

Learn: Web fonts

> This beginner's learning article explains how to use custom fonts on your web page to allow for more varied and custom text styling.

OpenType font features

> Font features or variants refer to different glyphs or character styles contained within an OpenType font. These include things like ligatures (special glyphs that combine characters like 'fi' or 'ffl'), kerning (adjustments to the spacing between specific letterform pairings), fractions, numeral styles, and a number of others. These are all referred to as OpenType Features, and are made available to use on the web via specific properties and a low-level control property — `font-feature-settings`. This article provides you with all you need to know about using OpenType font features in CSS.

Web Open Font Format (WOFF)

> WOFF uses a compressed version of the same table-based `sfnt` structure used by TrueType, OpenType, and Open Font Format, but adds metadata and private-use data structures, including predefined fields allowing foundries and vendors to provide license information if desired.

Variable fonts

> This article will help you get started with using variable fonts.

Improving font performance

> This article, part of the CSS performance guide, discusses font loading, loading only the required glyphs, and defining font display behavior with the `font-display` descriptor.

Related concepts

- `letter-spacing` CSS property
- `line-height` CSS property
- `text-transform` CSS property

See also

- CSS font loading module
- CSS font loading API
- CSS text module
- CSS writing modes module

OpenType font features

Font features or variants refer to different glyphs or character styles contained within an OpenType font. These include things like ligatures (special glyphs that combine characters like 'fi' or 'ffl'), kerning (adjustments to the spacing between specific letterform pairings), fractions, numeral styles, and several others. These are all referred to as OpenType Features, and are made available to use on the web via specific properties and low-level control properties — `font-feature-settings`. This article provides you with all you need to know about using OpenType font features in CSS.

Some fonts will have one or more of these features enabled by default (kerning and default ligatures are common examples), while others are left to the designer or developer to choose to enable in specific scenarios.

In addition to broad feature sets like ligatures or lining figures (numerals that line up evenly as opposed to 'oldstyle', which look more like lower-case letters), there are also very specific ones such as stylistic sets (which might include several specific variants of glyphs meant to be used together), alternates (which might be one or more variants of the letter 'a'), or even language-specific alterations for East Asian languages. In the latter case, these alterations are actually necessary to properly express the language, so they go beyond the more stylistic preference of most other OpenType features.

> **Warning:** There are many CSS attributes defined to leverage font features, but unfortunately many are not fully implemented. They are all defined and shown here, but many will only work using the lower-level `font-feature-settings` property. It's possible to write CSS to work both ways but this can become cumbersome. The issue with using `font-feature-settings` for everything is that every time you want to change one of the individual features, you have to redefine the entire string (similar to manipulating variable fonts with `font-variation-settings`).

Discovering availability of features in fonts

This is sometimes the trickiest thing to work out if you don't have any documentation that came with the fonts (many type designers and foundries will provide sample pages and CSS just for this very reason). But there are some sites that make it easier to figure out. You can visit wakamaifondue.com, drop your font file on the circle where instructed, and in a few moments you'll have a full report on all the capabilities and features of your font. Axis-praxis.org also offers a similar capability, with the ability to click on the features to turn them on or off in a given text block.

Why you would use them?

Given that these features take a bit of work to discover and use, it may seem a fair question to ask why one would bother to use them. The answer lies in the specific features that will make a site more useful, readable, and polished:

- **Ligatures** like 'ff' or 'fi' make letter spacing and reading more even and smooth.
- **Fractions** can make home improvement and recipe sites much easier to read and understand.
- **Numerals** within paragraphs of text set as 'oldstyle' sit more comfortably between lower-case letters, and likewise setting them as 'tabular numbers' will make them line up better when setting a list of costs in a table say. 'lining' figures on the other hand sit more uniformly on their own or in front of capitalized words.

While none of these features individually will render a site useless due to their absence, each of them in turn can make a site easier to use and more memorable for its attention to detail.

> OpenType features are like secret compartments in fonts. Unlock them and you'll find ways to make fonts look and behave differently in subtle and dramatic ways. Not all OpenType features are appropriate to use all of the time, but some features are critical for great typography. -- *Tim Brown, Head of Typography at Adobe.*

Sometimes it's substance, not just style

There are some cases — like with `font-variant-east-asian` — that OpenType features are directly tied to using different forms of certain glyphs, which can impact meaning and readability. In cases such as these, it's more than just a nicety, but rather an integral part of the content itself.

The font features

There are a number of different features to consider. They are grouped and explained here according to the main attributes and options covered in the W3C specifications.

> **Note:** The examples below show the properties and some example combinations, along with the lower-level syntax equivalents. They may not match exactly due to browser implementation inconsistencies, but in many cases, the first example will match the second. The typefaces shown are Playfair Display, Source Serif Pro, IBM Plex Serif, Dancing Script, and Kokoro (all available and free to use, most are on Google Fonts and other services).

Kerning

Associated CSS property: `font-kerning`

This refers to the spacing between specific glyph pairings. This is generally on by default (as recommended by the OpenType specification). It should be noted that if `letter-spacing` is also set on your text, that is applied after kerning. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-kerning</code> property</legend>
  <div class="container container1">
    <p>Puffy Pangolins</p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Kerning active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>Puffy Pangolins</p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Kerning active</label>
</fieldset>
```

```
@font-face {
  font-family: "Plex Serif";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/plex/IBMPlexSerif-Regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/plex/IBMPlexSerif-Regular.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Plex Serif",
    "Times New Roman",
    serif;
  margin: 1rem;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
/* kerning: auto|normal|none */
.container1 * {
  font-kerning: normal;
}
.inactive.container1 * {
  font-kerning: none;
}

/* 'kern' 1|0 (on or off) */
.container2 * {
  font-feature-settings: "kern" 1;
}
.inactive.container2 * {
  font-feature-settings: "kern" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

Alternates

Associated CSS property: `font-variant-alternates`

Fonts can supply a number of different alternatives for various glyphs, such as different styles of lower case 'a' or more or less elaborate swashes in a script typeface. This property can activate an entire set of alternates or just a specific one, depending on the values supplied. The example below is showing several different aspects of working with alternate characters. Fonts with alternate glyphs can make them available across the board or individually in separate stylistic sets, or even individual characters. In this example you can see two different typefaces, and the introduction of the `@font-feature-values` at-rule. This is used to define shortcuts or named options that can be defined per font family. This way you can create a named option that applies to only a single font, or one that is shared and can be applied more generally. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-alternates</code> property</legend>
  <div class="container container1">
    <p><span class="script">My</span> Perfidious pangram</p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Alternates active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p><span class="script">My</span> Perfidious pangram</p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Alternates active</label>
</fieldset>
```

```
@font-face {
  font-family: "Plex Serif";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/plex/IBMPlexSerif-Regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/plex/IBMPlexSerif-Regular.woff2")
      format("woff2");
}

@font-face {
  font-family: "Dancing Script";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/dancing-script/dancing-script-regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/dancing-script/dancing-script-regular.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Plex Serif",
    "Times New Roman",
    serif;
  margin: 1rem;
}

.script {
  font-family: "Dancing Script", serif;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > * {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
@font-feature-values "Plex Serif" {
  @styleset {
```

```
    alt-a: 1;
    alt-g: 2;
  }
  @stylistic {
    alternates: 1;
  }
}

@font-feature-values "Dancing Script" {
  @stylistic {
    alternates: 1;
  }
}

.container1 * {
  font-variant-alternates: styleset(alt-a);
}
.container1 .script {
  font-variant-alternates: stylistic(alternates);
}
.inactive.container1 * {
  font-variant-alternates: normal;
}

.container2 * {
  font-feature-settings: "ss01";
}
.container2 .script {
  font-feature-settings: "salt";
}
.inactive.container2 * {
  font-feature-settings:
    "ss01" 0,
    "salt" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

In this case, `@stylistic(alternates)` will show all the alternate characters for either font. Applying this to just the word 'My' alters the way the 'M' renders, and applying `@styleset(alt-a)` only changes the lower case 'a'.

Try changing the line

```
font-variant-alternates: styleset(alt-a);
```

to

```
font-variant-alternates: styleset(alt-g);
```

and notice that the lower case 'a' reverts to its regular form and the lower case 'g's changes instead.

Ligatures

Associated CSS property: `font-variant-ligatures`

Ligatures are glyphs that replace two or more separate glyphs in order to represent them more smoothly (from a spacing or aesthetic perspective). Some of the most common are letters like 'fi', 'fl', or 'ffl' — but there are many other possibilities. There are the most frequent ones (referred to as common ligatures), and there are also more specialized categories like 'discretionary ligatures', 'historical ligatures', and 'contextual alternates'. While these last ones are not technically ligatures, they are generally similar in that they replace specific combinations of letters when they appear together.

While more common in script typefaces, in the below example they are used to create arrows. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-ligatures</code> property</legend>
  <div class="container container1">
    <p>Puffy Perfect -^ &lt;-&gt;</p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Ligatures active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>Puffy Perfect -^ &lt;-&gt;</p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Ligatures active</label>
</fieldset>
```

```
@font-face {
  font-family: "Playfair Display";
```

```
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Playfair Display",
    "Times New Roman",
    serif;
  margin: 1rem;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
.container1 * {
  font-variant-ligatures: common-ligatures discretionary-ligatures contextual;
}
.inactive.container1 * {
  font-variant-ligatures: none;
}

/* 'liga', 'dlig', 'hlig', 'calt' */
.container2 * {
  font-feature-settings: "dlig", "liga", "calt";
}
.inactive.container2 * {
  font-feature-settings:
    "dlig" 0,
    "liga" 0,
    "calt" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

Position

Associated CSS property: `font-variant-position`

Position variants are used to enable typographic superscript and subscript glyphs. These are designed to work with the surrounding text without altering the baseline or line spacing. This is especially useful with the `<sub>` or `<sup>` elements. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-position</code> property</legend>
  <div class="container container1">
    <p>Ups<span class="super">1</span> and downs<span class="sub">2</span></p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Position active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>Ups<span class="super">1</span> and downs<span class="sub">2</span></p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Position active</label>
</fieldset>
```

```
@font-face {
  font-family: "Playfair Display";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
```

```
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Playfair Display",
    "Times New Roman",
    serif;
  margin: 1rem;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
/* position: normal|sub|super */
.container1 .super {
  font-variant-position: super;
}
.container1 .sub {
  font-variant-position: sub;
}
.inactive.container1 * {
  font-variant-position: normal;
}

/* 'subs', 'sups' */
.container2 .super {
  font-feature-settings: "sups";
}
.container2 .sub {
  font-feature-settings: "subs";
}
.inactive.container2 * {
  font-feature-settings:
    "sups" 0,
    "subs" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

Capitals

Associated CSS property: `font-variant-caps`

One of the more common use cases for OpenType features is proper small caps. These are capital letters sized to fit better among lower case letters and are generally used for acronyms and abbreviations. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-caps</code> property</legend>
  <div class="container container1">
    <p>
      <span class="small-caps">Small Caps</span> and
      <span class="all-small-caps">All Small Caps</span>
    </p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Caps active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>
      <span class="small-caps">Small Caps</span> and
      <span class="all-small-caps">All Small Caps</span>
    </p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
```

```
  <label for="checkbox2">Caps active</label>
</fieldset>
```

```
@font-face {
  font-family: "Playfair Display";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Playfair Display",
    "Times New Roman",
    serif;
  margin: 1rem;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
/* position: normal | small-caps | all-small-caps | petite-caps | all-petite-caps | unicase | titling-caps */
.container1 .small-caps {
  font-variant-caps: small-caps;
}
.container1 .all-small-caps {
  font-variant-caps: all-small-caps;
}
.inactive.container1 * {
  font-variant-caps: normal;
}

/* 'smcp', 'c2sc' */
.container2 .small-caps {
  font-feature-settings: "smcp" 1;
}
.container2 .all-small-caps {
  font-feature-settings:
    "c2sc" 1,
    "smcp" 1;
}
.inactive.container2 * {
  font-feature-settings:
    "smcp" 0,
    "c2sc" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

Numerals

Associated CSS property: `font-variant-numeric`

There are several different styles of numerals commonly included in fonts:

- 'Lining' figures are all the same height and on the same baseline.
- 'Oldstyle' figures are mixed height and designed to have the appearance of ascenders and descenders like other lower case letters. These are designed to be used inline with a copy so the numerals visually blend with the surrounding glyphs in a similar fashion to small caps.

There is also the notion of spacing. Proportional spacing is the normal setting, whereas tabular spacing lines up numerals evenly regardless of character width, making it more appropriate for lining up tables of numbers in financial tables.

There are two types of fractions supported through this property:

- Diagonal slashed fractions.
- Vertically stacked fractions.

Ordinals are also supported (such as '1st' or '3rd'), as is a slashed zero if present in the font.

Lining and old-style figures

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-numeric</code> property</legend>
  <div class="container container1">
    <p>
      <span class="lining">6,142</span> or <span class="oldstyle">6,142</span>
    </p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Numeric active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>
      <span class="lining">6,142</span> or <span class="oldstyle">6,142</span>
    </p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Numeric active</label>
</fieldset>
```

```
@font-face {
  font-family: "Source Serif";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/source-serif/SourceSerifPro-Regular.ttf.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/source-serif/SourceSerifPro-Regular.ttf.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Source Serif",
    "Times New Roman",
    serif;
  margin: 20px;
  padding: 0;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
.container1 .lining {
  font-variant-numeric: lining-nums;
}
.container1 .oldstyle {
  font-variant-numeric: oldstyle-nums;
}
.inactive.container1 * {
  font-variant-numeric: normal;
}

.container2 .lining {
  font-feature-settings: "lnum" 1;
}
.container2 .oldstyle {
  font-feature-settings: "onum" 1;
}
.inactive.container2 * {
  font-feature-settings:
    "lnum" 0,
    "onum" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
```

```
  toggleInactive(checkBox2, container2);
});
```

Fractions, ordinals, and slashed zero

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-numeric</code> property</legend>
  <div class="container container1">
    <p>
      <span class="diagonal-fractions">3/16</span>, or
      <span class="ordinal">1st</span> of <span class="zero">0</span>
    </p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Numeric active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>
      <span class="diagonal-fractions">3/16</span>, or
      <span class="ordinal">1st</span> of <span class="zero">0</span>
    </p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Numeric active</label>
</fieldset>
```

```
@font-face {
  font-family: "Source Serif";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/source-serif/SourceSerifPro-Regular.ttf.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/source-serif/SourceSerifPro-Regular.ttf.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Source Serif",
    "Times New Roman",
    serif;
  margin: 20px;
  padding: 0;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
.container1 .diagonal-fractions {
  font-variant-numeric: diagonal-fractions;
}
.container1 .ordinal {
  font-variant-numeric: ordinal;
}
.container1 .zero {
  font-variant-numeric: slashed-zero;
}
.inactive.container1 * {
  font-variant-numeric: normal;
}

.container2 .diagonal-fractions {
  font-feature-settings: "frac" 1;
}
.container2 .ordinal {
  font-feature-settings: "ordn" 1;
}
.container2 .zero {
  font-feature-settings: "zero" 1;
}
.inactive.container2 * {
  font-feature-settings:
    "frac" 0,
    "ordn" 0,
    "zero" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
```

```
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});

checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

East Asian

Associated CSS property: `font-variant-east-asian`

This allows access to various alternate forms of glyphs within a font. The example below shows a string of normal glyphs. Uncheck the box below and you'll see characters with only the `jis78` glyphs. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant-numeric</code> property</legend>
  <div class="container container1">
    <p>唖 芦 溢 茨 鰯 嘘 欝 厩 噂</p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Features disabled</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>唖 芦 溢 茨 鰯 嘘 欝 厩 噂</p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Features disabled</label>
</fieldset>
```

```
@font-face {
  font-family: "Kokoro";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src: url("https://mdn.github.io/shared-assets/fonts/kokoro/Kokoro.woff2")
    format("woff2");
}

body {
  font:
    1.2em "Kokoro",
    "Times New Roman",
    serif;
  margin: 20px;
  padding: 0;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
.container1 * {
  font-variant-east-asian: normal;
}
.inactive.container1 * {
  font-variant-east-asian: jis78;
}

.container2 * {
  font-feature-settings: "jp78" 0;
}
.inactive.container2 * {
  font-feature-settings: "jp78";
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});
```

```
checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

> **Note:** These glyphs were copied out of a font sample and are not intended as prose.

Font variant shorthand

The `font-variant` property is the shorthand syntax for defining all of the above. Setting a value of `normal` resets all properties to their initial value. Setting a value of `none` sets `font-variant-ligatures` to none and all other properties to their initial value. Meaning that if kerning is on by default, it will still be on even with a value of `none` being supplied here. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<fieldset>
  <legend>Using <code>font-variant</code> property</legend>
  <div class="container container1">
    <p>Spiffy Plastic -> 3/4 time</p>
  </div>
  <input type="checkbox" name="checkbox1" id="checkbox1" value="on" checked />
  <label for="checkbox1">Features active</label>
</fieldset>

<fieldset>
  <legend>Using <code>font-feature-settings</code> property</legend>
  <div class="container container2">
    <p>Spiffy Plastic -> 3/4 time</p>
  </div>
  <input type="checkbox" name="checkbox2" id="checkbox2" value="on" checked />
  <label for="checkbox2">Features active</label>
</fieldset>
```

```
@font-face {
  font-family: "Playfair Display";
  font-weight: normal;
  font-style: normal;
  font-stretch: normal;
  src:
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff")
      format("woff"),
    url("https://mdn.github.io/shared-assets/fonts/playfair-display/playfair-display-regular.woff2")
      format("woff2");
}

body {
  font:
    1.2em "Playfair Display",
    "Times New Roman",
    serif;
  margin: 20px;
  padding: 0;
}

fieldset {
  margin-bottom: 1rem;
}

label {
  user-select: none;
}

.container > p {
  font-size: 4rem;
  margin: 1.5rem 0;
}
```

```
.container1 * {
  font-variant: common-ligatures discretionary-ligatures contextual
    diagonal-fractions;
}
.inactive.container1 * {
  font-variant: none;
}

.container2 * {
  font-feature-settings: "dlig", "liga", "calt", "frac";
}
.inactive.container2 * {
  font-feature-settings:
    "dlig" 0,
    "liga" 0,
    "calt" 0,
    "frac" 0;
}
```

```
const checkBox1 = document.getElementById("checkbox1");
const checkBox2 = document.getElementById("checkbox2");
const container1 = document.querySelector(".container1");
const container2 = document.querySelector(".container2");

const toggleInactive = (checkBox, container) => {
  if (checkBox.checked) {
    container.classList.remove("inactive");
  } else {
    container.classList.add("inactive");
  }
};

checkBox1.addEventListener("change", () => {
  toggleInactive(checkBox1, container1);
});
```

```
checkBox2.addEventListener("change", () => {
  toggleInactive(checkBox2, container2);
});
```

Font feature settings

`font-feature-settings` is the 'low-level syntax' that allows explicit access to every named available OpenType feature. This gives a lot of control but has some disadvantages in how it impacts inheritance and — as mentioned above — if you wish to change one setting, you have to redeclare the entire string (unless you're using CSS custom properties to set the values). Because of this, it's best to use the standard properties shown above wherever possible.

There are a huge number of possible features. You can see examples of a number of them above, and there are several resources available for finding more of them.

The general syntax looks like this:

```
.small-caps {
  font-feature-settings: "smcp", "c2sc";
}
```

According to the specification you can either supply just the 4-character feature code or supply a 1 following the code (for enabling that feature) or a 0 (zero) to disable it. This is helpful if you have a feature like ligatures enabled by default but you would like to turn them off like so:

```
.no-ligatures {
  font-feature-settings:
    "liga" 0,
    "dlig" 0;
}
```

More on font-feature-settings codes

- 'The Complete CSS Demo for OpenType Features' (can't vouch for the truth of the name, but it's pretty big)
- A list of OpenType features on Wikipedia

Using CSS feature detection for implementation

Since not all properties are evenly implemented, it's good practice to set up your CSS using feature detection to utilize the correct properties, with `font-feature-settings` as the fallback.

For example, small caps can be set several ways, but if you want to ensure that no matter what the underlying capitalization is that you end up with everything in small caps, it requires 2 settings with `font-feature-settings` versus a single property value using `font-variant-caps`.

```
.small-caps {
  font-feature-settings: "smcp", "c2sc";
}

@supports (font-variant-caps: all-small-caps) {
  .small-caps {
    font-feature-settings: normal;
    font-variant-caps: all-small-caps;
  }
}
```

See also

Demos of CSS OpenType features in CSS

- The Complete CSS Demo for OpenType Features

Web font analysis tools

- Wakamai Fondue
- Axis Praxis

W3C Specifications

- Font Feature Properties in CSS Fonts Module Level 3
- font-variant-alternatives in CSS Fonts Module Level 4

Other resources

- Using OpenType features by Tim Brown, Head of Typography, Adobe
- Adobe's Syntax for OpenType features in CSS

Variable fonts

Variable fonts are an evolution of the OpenType font specification that enables many different variations of a typeface to be incorporated into a single file, rather than having a separate font file for every width, weight, or style. They let you access all the variations contained in a given font file via CSS and a single `@font-face` reference. This article will give you all you need to know to get you started using variable fonts.

> **Note:** To use variable fonts on your operating system, you need to make sure that it is up to date. For example, Linux OSes need the latest Linux FreeType version, and macOS prior to High Sierra (10.13) does not support variable fonts. If your operating system is not up to date, you will not be able to use variable fonts in web pages or the Firefox Developer Tools.

Variable Fonts: what they are, and how they differ

To better understand what's different about variable fonts, it is worth reviewing what non-variable ones are like and how they compare.

Standard (or Static) fonts

In the past, a typeface would be produced as several individual fonts, and each font would represent one specific width/weight/style combination. So you would have separate files for 'Roboto Regular', 'Roboto Bold', and 'Roboto Bold Italic' — meaning that you could end up with 20 or 30 different font files to represent a complete typeface (it could be several times that for a large typeface that has different widths as well).

In such a scenario, to use a typeface for typical use on a site for body copy you would need at least four files: regular, italic, bold, and bold italic. If you wanted to add more weights, like a lighter one for captions or a heavier one for extra emphasis, that would mean several more files. This results in more HTTP requests, and more data being downloaded (usually around 20k or more per file).

Variable fonts

With a variable font, all of those permutations can be contained in a single file. That file would be larger than a single font, but in most cases smaller or about the same size as the 4 you might load for body copy. The advantage in choosing the variable font is that you have access to the entire range of weights, widths, and styles available, rather than being constrained to only the few that you previously would have loaded separately.

This allows for common typographic techniques such as setting different size headings in different weights for better readability at each size or using a slightly narrower width for data-dense displays. For comparison, it is typical in a typographic system for a magazine to use 10–15 or more different weight and width combinations throughout the publication — giving a much wider range of styles than currently typical on the web (or indeed practical for performance reasons alone).

A note about font families, weights, and variants

You might notice that we have been talking about having a specific font file for every weight and style (i.e., bold and italic and bold italic), rather than relying upon the browser to synthesize them. The reason for this is that most typefaces have very specific designs for bolder weights and italics that often include completely different characters (lower-case 'a' and 'g's are often quite different in italics, for example). To most accurately reflect the typeface design and avoid differences between browsers and how they may or may not synthesize the different styles, it's more accurate to load the specific font files where needed when using a non-variable font.

You may also find that some variable fonts come split into two files: one for uprights and all their variations, and one containing the italic variations. This is sometimes done to reduce the overall file size in cases where the italics aren't needed or used. In all cases, it is still possible to link them with a common `font-family` name so you can call them using the same `font-family` and appropriate `font-style`.

Introducing the 'variation axis'

The heart of the new variable fonts format is the concept of an **axis of variation** describing the allowable range of that particular aspect of the typeface design. So the 'weight axis' describes how light or how bold the letterforms can be; the 'width axis' describes how narrow or how wide they can be; the 'italic axis' describes if italic letterforms are present and can be turned on or off accordingly, etc. Note that an axis can be a range or a binary choice. Weight might range from 1–999, whereas italic might be 0 or 1 (off or on).

As defined in the specification, there are two kinds of axes: **registered** and **custom**:

- Registered axes are those that are most frequently encountered, and common enough that the authors of the specification felt it was worth standardizing. The five currently registered axes are weight, width, slant, italic, and optical size. The W3C has undertaken to map them to existing CSS attributes, and in one case introduced a new one, which you'll see below.
- Custom axes are limitless: the typeface designer can define and scope any axis they like and are just required to give it a four-letter **tag** to identify it within the font file format itself. You can use these four-letter tags in CSS to specify a point along that axis of variation, as will be shown in the code examples below.

Registered axes and existing CSS attributes

In this section we'll demonstrate the five registered axes defined with examples and the corresponding CSS. Where possible, both the standard and lower-level syntax are included. The lower-level syntax (`font-variation-settings`) was the first mechanism implemented to test the early implementations of variable font support and is necessary to utilize new or custom axes beyond the five registered ones. However, the W3C's intent was for this syntax not to be used when other attributes are available. Therefore wherever possible, the appropriate property should be used, with the lower-level syntax of `font-variation-settings` only being used to set values or axes not available otherwise.

Notes

1. When using `font-variation-settings` it is important to note that axis names are case-sensitive. The registered axis names must be in lower case, and custom axes must be in upper case. For example:

   ```
   font-variation-settings:
     "wght" 375,
     "GRAD" 88;
   ```

   `wght` (weight) is a registered axis, and `GRAD` (grade) is a custom one.

2. If you have set values using `font-variation-settings` and want to change one of those values, you must redeclare all of them (in the same way as when you set OpenType font features using `font-feature-settings`). You can work around this limitation by using CSS Custom Properties (CSS variables) for the individual values, and modifying the value of an individual custom property. Example code follows at the end of the guide.

Weight

Weight (represented by the `wght` tag) defines the design axis of how thin or thick (light or heavy, in typical typographic terms) the strokes of the letterforms can be. For a long time in CSS there has existed the ability to specify this via the `font-weight` property, which takes numeric values ranging from 100 to 900 in increments of 100, and keywords like `normal` or `bold`, which are aliases for their corresponding numeric values (400 and 700 in this case). These are still applied when dealing with non-variable or variable fonts, but with variable ones, any number from 1 to 1000 is now valid.

It should be noted that at this point there is no way in the `@font-face` declaration to 'map' a specific point on the variation axis of a variable font to the keyword `bold` (or any other keyword). This can generally be resolved fairly easily, but does require an extra step in writing your CSS:

```
font-weight: 375;

font-variation-settings: "wght" 375;
```

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS code to play with font-weight values.

```
<div>
  <p class="p1">Weight</p>
  <span>(font-weight: 625)</span>
</div>
<div>
  <p class="p2">Weight</p>
  <span>(font-variation-settings: "wght" 625)</span>
</div>
<div class="adjustable">
  <p class="p3">Weight</p>
  (font-variation-settings: "wght" <span id="angle-text">625</span>)<br />
  <label for="text-axis">Adjust Weight: </label>
  <input
    type="range"
    id="text-axis"
    name="text-axis"
    min="300"
    max="900"
    value="625" />
</div>
```

```
@font-face {
  font-family: "Amstelvar VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 35% 100%;
  font-style: normal;
  font-display: swap;
```

```
}

p {
  font:
    1.2em "Amstelvar VF",
    "Georgia",
    serif;
  font-size: 4rem;
  margin: 1rem;
  display: inline-block;
}

.adjustable {
  border: 1px dashed;
  --text-axis: 625;
}
```

```
/* weight range is 300 to 900 */
.p1 {
  font-weight: 625;
}

/* weight range is 300 to 900 */
.p2 {
  font-variation-settings: "wght" 625;
}

/* Adjust with slider & custom property */
.p3 {
  font-variation-settings: "wght" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

Width

Width (represented by the `wdth` tag) defines the design axis of how narrow or wide (condensed or extended, in typographic terms) the letterforms can be. This is typically set in CSS using the `font-stretch` property, with values expressed as a percentage above or below 'normal' (100%), any number greater than 0 is technically valid—though it is far more likely that the range would fall closer to the 100% mark, such as 75%-125%. If a number value supplied is outside the range encoded in the font, the browser should render the font at the closest value allowed.

> **Note:** The % symbol is not used when utilizing `font-variation-settings`.

```
font-stretch: 115%;

font-variation-settings: "wdth" 115;
```

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS code to play with font width values.

```
<div>
  <p class="p1">Width</p>
  <span>(font-stretch: 60%)</span>
</div>
<div>
  <p class="p2">Width</p>
  <span>(font-variation-settings: "wdth" 60)</span>
</div>
<div class="adjustable">
  <p class="p3">Width</p>
  (font-variation-settings: "wdth" <span id="angle-text">60</span>)<br />

  <label for="text-axis">Adjust Width: </label>
  <input
    type="range"
    id="text-axis"
    name="text-axis"
    min="55"
    max="100"
    value="60" />
</div>
```

```
@font-face {
  font-family: "Amstelvar VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 35% 100%;
  font-style: normal;
  font-display: swap;
}

p {
  font:
    1.2em "Amstelvar VF",
    "Georgia",
    serif;
  font-size: 4rem;
  margin: 1rem;
  display: inline-block;
}
```

```
.adjustable {
  border: 1px dashed;
  --text-axis: 60;
}
```

```
/* width range is 55% to 100% */
.p1 {
  font-stretch: 60%;
}

/* width range is an integer from 55 to 100 */
.p2 {
  font-variation-settings: "wdth" 60;
}

/* Adjust with slider & custom property */
.p3 {
  font-variation-settings: "wdth" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

Italic

The Italic (`ital`) axis can be set in the range `[0-1]`, where `0` specifies "not italic," `0.5` specifies "halfway italic," and `1` specifies "fully italic." Italic designs often include dramatically different letterforms from their upright counterparts, so in the transition from upright to italic, several glyph (or character) substitutions usually occur. Italic and oblique are often used somewhat interchangeably, but in truth are quite different. Oblique is defined in this context with the term `slant` (see the below section), and a typeface would typically have one or the other, but not both.

In CSS, both italic and oblique are applied to text using the `font-style` property. Also note the introduction of `font-synthesis: none;` — which will prevent browsers from accidentally applying the variation axis and a synthesized italic. This can be used to prevent faux-bolding as well.

```
font-style: italic;

font-variation-settings: "ital" 1;

font-synthesis: none;
```

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS code to play with font-italics.

```
<div>
  <p class="p1">Italic</p>
  <span>(font-style: italic)</span>
  <p class=".p1-no-synthesis">Italic</p>
  <span>(font-style: italic; font-synthesis: none)</span>
</div>
<div>
  <p class="p2">Italic</p>
  <span>(font-variation-settings: "ital" 1)</span>
</div>
<div class="adjustable">
  <p class="p3">Italic</p>
  (font-variation-settings: "ital" <span id="angle-text">1</span>)<br />

  <label for="slant-angle">Adjust Italic: </label>
  <input
    type="range"
    id="text-axis"
    name="text-axis"
    min="0"
    max="1"
    value="1" />
</div>
```

```
@font-face {
  font-family: "Jost VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/jost-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 75% 150%;
  font-display: swap;
}

p {
  font:
    1.2em "Jost VF",
    "Helvetica",
    "Arial",
    sans-serif;
  font-size: 4rem;
  margin: 1rem;
  display: inline-block;
}

.adjustable {
  border: 1px dashed;
  --text-axis: 1;
}
```

```
/* font-style: italic, with and without font-synthesis */
.p1 {
  font-style: italic;
```

```
}

.p1-no-synthesis {
  font-style: italic;
  font-synthesis: none;
}

/* italic range is 0 or 1 */
.p2 {
  font-variation-settings: "ital" 1;
  font-synthesis: none;
}

/* Adjust with slider & custom property */
.p3 {
  font-synthesis: none;
  font-variation-settings: "ital" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

Slant

Slant (represented by the `slnt` tag), or as it's often referred to, 'oblique' — is different from true italics in that it changes the angle of the letterforms but doesn't perform any kind of character substitution. It is also variable, in that it is expressed as a numeric range. This allows the font to be varied anywhere along the slant axis. The allowed range is from -90 to 90 degrees.

The two properties that can control the slant are `font-style` and `font-variation-settings`. The following two property declarations are the same:

```
font-style: oblique 14deg;

font-variation-settings: "slnt" -14;
```

Prefer the `font-style` property over the `font-variation-settings` property. The `deg` keyword is not used when using the `font-variation-settings` property. Also, in the case of the `font-variation-settings` property, a positive angle means a counter-clockwise slant.

In the following live example, you can adjust the slant.

```
<div>
  <p class="font-style">Slant</p>
  <span>(font-style: oblique 5deg)</span>
</div>
<div>
  <p class="font-variation">Slant</p>
  <span>(font-variation-settings: 'slnt' -5)</span>
</div>
<div class="adjustable-box">
  <p class="adjustable">Slant</p>
  (font-variation-settings: 'slnt' <span id="angle-text">-5</span>)<br />

  <label for="slant-angle">Adjust Slant: </label>
  <input
    type="range"
    name="range-slider"
    value="5"
    id="slant-angle"
    min="-15"
    max="15" />
</div>
```

```
@font-face {
  font-family: "SlantFont";
  font-style: oblique -15deg 15deg;
  src: url("https://mdn.github.io/shared-assets/fonts/font_with_slant_axis.woff2")
    format("woff2");
}

p {
  font-family: "SlantFont", sans-serif;
  display: inline-block;
  margin: 1rem;
  font-size: 4rem;
}

.adjustable-box {
  border: 1px dashed;
  --text-axis: -5;
}
```

```
.font-style {
  font-style: oblique 5deg;
}

.font-variation {
  font-variation-settings: "slnt" -5;
}

.adjustable {
  font-variation-settings: "slnt" var(--slant-angle);
}
```

```
const angle = document.querySelector("#slant-angle");
const text = document.querySelector("#angle-text");
```

```
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = -1 * e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--slant-angle", angle);
});
```

Optical size

This is something new to digital fonts and CSS, but is a centuries-old technique in designing and creating metal type. Optical sizing refers to the practice of varying the overall stroke thickness of letterforms based on physical size. If the size was very small (such as an equivalent to 10 or 12px), the characters would have an overall thicker stroke, and perhaps other small modifications to ensure that it would reproduce and be readable at a physically smaller size. Conversely, when a much larger size was being used (like 48 or 60px), there might be much greater variation in thick and thin stroke weights, showing the typeface design more in line with the original intent.

While this was originally done to compensate for the ink and paper printing process (very thin lines at small sizes often didn't print, giving the letterforms a broken appearance), it translates well to digital displays when compensating for screen quality and physical size rendering.

Optical size values are generally intended to be automatically applied corresponding to `font-size`, but can also be manipulated using the lower-level `font-variation-settings` syntax.

There is a new attribute, `font-optical-sizing`, created to support variable fonts in CSS. When using `font-optical-sizing`, the only allowed values are `auto` or `none` — so this attribute only allows for turning optical sizing on or off. However when using `font-variation-settings: 'opsz' <num>`, you can supply a numeric value. In most cases you would want to match the `font-size` (the physical size the type is being rendered) with the `opsz` value (which is how optical sizing is intended to be applied when using `auto`). The option to provide a specific value is provided so that should it be necessary to override the default — for legibility, aesthetic, or some other reason — a specific value can be applied.

```
font-optical-sizing: auto;

font-variation-settings: "opsz" 36;
```

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS code to play with optical size values.

```
<div>
  <p class="p1">Optical Size</p>
  <span>(font-optical-sizing: none)</span>
</div>
<div>
  <p class="p2">Optical Size</p>
  <span>(font-optical-sizing: auto)</span>
</div>
<div>
  <p class="p3">Optical Size</p>
  <span>(font-variation-settings: "opsz" 64)</span>
</div>

<div class="adjustable">
  <p class="p4">Optical Size</p>
  (font-variation-settings: "opsz" <span id="angle-text">-64</span>)<br />

  <label for="slant-angle">Adjust Optical Sizing: </label>
  <input
    type="range"
    name="text-axis"
    value="64"
    id="text-axis"
    min="8"
    max="144" />
</div>
```

```
@font-face {
  font-family: "Amstelvar VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 75% 150%;
  font-style: normal;
  font-display: swap;
}

p {
  font:
    1.2em "Amstelvar VF",
    "Georgia",
    serif;
  font-size: 4rem;
  margin: 1rem;
  display: inline-block;
}

.adjustable {
  border: 1px dashed;
  --text-axis: 64;
}
```

```
.p1 {
  font-optical-sizing: none;
}
/* font-optical-sizing can be auto or none */
.p2 {
  font-optical-sizing: auto;
}

/* optical range is from 8 to 144 */
.p3 {
  font-variation-settings: "opsz" 64;
}

/* Adjust with slider & custom property */
.p4 {
```

```
  font-variation-settings: "opsz" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

Custom axes

Custom axes are just that: they can be any axis of design variation that the typeface designer imagines. There may be some that become fairly common — or even become registered — but only time will tell.

Grade

Grade may become one of the more common custom axes as it has a known history in typeface design. The practice of designing different grades of a typeface was often done in reaction to intended use and printing technique. The term 'grade' refers to the relative weight or density of the typeface design, but differs from traditional 'weight' in that the physical space the text occupies does not change, so changing the text grade doesn't change the overall layout of the text or elements around it. This makes grade a useful axis of variation as it can be varied or animated without causing a reflow of the text itself.

```
font-variation-settings: "GRAD" 88;
```

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS code to play with font-grade values.

```
<div>
  <p class="p1">Grade</p>
  <span>(font-variation-settings: 'GRAD' 88)</span>
</div>

<div class="adjustable">
  <p class="p2">Grade</p>
  (font-variation-settings: 'GRAD' <span id="angle-text">88</span>)<br />

  <label for="slant-angle">Adjust Grade: </label>
  <input
    type="range"
    name="text-axis"
    value="88"
    id="text-axis"
    min="88"
    max="150" />
</div>
```

```
@font-face {
  font-family: "Amstelvar VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 75% 150%;
  font-style: normal;
  font-display: swap;
}

p {
  font:
    1.2em "Amstelvar VF",
    "Georgia",
    serif;
  font-size: 64px;
  margin: 1rem;
  display: inline-block;
}

.adjustable {
  border: 1px dashed;
  --text-axis: 88;
}
```

```
/* grade range is 88 to 150 */
.p1 {
  font-size: 64px;
  font-variation-settings: "GRAD" 88;
}

/* Adjust with slider & custom property */
.p2 {
  font-size: 64px;
  font-variation-settings: "GRAD" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

Using a variable font: @font-face changes

The syntax for loading variable fonts is very similar to any other web font, with a few notable differences, which are provided via upgrades to the traditional `@font-face` syntax now available in modern browsers.

The basic syntax is the same, but the font technology can be specified, and allowable ranges for descriptors like `font-weight` and `font-stretch` can be supplied, rather than named according to the font file being loaded.

Example for a standard upright (Roman) font

```
@font-face {
  font-family: "MyVariableFontName";
  src: url("path/to/font/file/my-variable-font.woff2")
    format("woff2-variations");
  font-weight: 125 950;
  font-stretch: 75% 125%;

  font-style: normal;
}
```

In this case, the `font-style: normal` declaration indicates that this font file should be used when `font-family` is set to `MyVariableFontName` and `font-style` is set to `normal`. As an alternative, you could use `font-style: oblique 0deg` or `font-style: oblique 0deg 20deg` to indicate that the font has normal upright glyphs (indicated by `0deg`).

Example for a font that contains only italics and no upright characters

```
@font-face {
  font-family: "MyVariableFontName";
  src: url("path/to/font/file/my-variable-font.woff2")
    format("woff2-variations");
  font-weight: 125 950;
  font-stretch: 75% 125%;

  font-style: italic;
}
```

In this case, the `font-style: italic` declaration indicates that this font file should be used when `font-family` is set to `MyVariableFontName` and `font-style` is set to `italic`. As an alternative, you could use `font-style: oblique 14deg` to indicate that the font has italic glyphs.

Example for a font that contains an oblique (slant) axis

```
@font-face {
  font-family: "MyVariableFontName";
  src: url("path/to/font/file/my-variable-font.woff2")
    format("woff2-variations");
  font-weight: 125 950;
  font-stretch: 75% 125%;

  font-style: oblique 0deg 12deg;
}
```

In this case, the `oblique 0deg 12deg` value indicates that this font file should be used when in a style rule the `font-family` property is `MyVariableFontName` and the font-style property is oblique with an angle between zero and 12 degrees inclusive.

> **Note:** Not all browsers have implemented the full syntax for font format, so test carefully. All browsers that support variable fonts will still render them if you set the format to just the file format, rather than format-variations (i.e., `woff2` instead of `woff2-variations`), but it's best to use the proper syntax if possible.

> **Note:** Supplying value ranges for `font-weight`, `font-stretch`, and `font-style` will keep the browser from attempting to render an axis outside that range if you are using the appropriate attribute (i.e., `font-weight` or `font-stretch`), but will not block you from supplying an invalid value via `font-variation-settings`, so use with care.

Working with older browsers

Variable font support can be checked with CSS Feature Queries (see `@supports`), so it's possible to use variable fonts in production and scope the CSS calling the variable fonts inside a feature query block.

```
h1 {
  font-family: some-non-variable-font-family;
}

@supports (font-variation-settings: "wdth" 115) {
  h1 {
    font-family: some-variable-font-family;
  }
}
```

Sample pages

The following example pages show two different ways to structure your CSS. The first uses the standard attributes wherever possible. The second example uses CSS Custom Properties to set values for a `font-variation-settings` string and shows how you can more easily update single variable values by overriding a single variable rather than rewriting the whole string. Note the hover effect on the `h2`, which only alters the grade axis custom property value. Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="container container1">
  <h1>Moby-Dick</h1>
  <h2>CHAPTER 1. Loomings.</h2>
  <p>
    Call me Ishmael. Some years ago—never mind how long precisely—having little
    or no money in my purse, and nothing particular to interest me on shore, I
    thought I would sail about a little and see the watery part of the world. It
    is a way I have of driving off the spleen and regulating the circulation.
    Whenever I find myself growing grim about the mouth; whenever it is a damp,
    drizzly November in my soul; whenever I find myself involuntarily pausing
    before coffin warehouses, and bringing up the rear of every funeral I meet;
    and especially whenever my hypos get such an upper hand of me, that it
    requires a strong moral principle to prevent me from deliberately stepping
    into the street, and methodically knocking people's hats off then, I account
    it high time to get to sea as soon as I can.
  </p>
</div>
<hr />
<div class="container container2 demo2">
  <h1>Moby-Dick</h1>
  <h2>CHAPTER 1. (hover here)</h2>
  <p>
    Call me Ishmael. Some years ago—never mind how long precisely—having little
    or no money in my purse, and nothing particular to interest me on shore, I
    thought I would sail about a little and see the watery part of the world. It
```

```
    is a way I have of driving off the spleen and regulating the circulation.
    Whenever I find myself growing grim about the mouth; whenever it is a damp,
    drizzly November in my soul; whenever I find myself involuntarily pausing
    before coffin warehouses, and bringing up the rear of every funeral I meet;
    and especially whenever my hypos get such an upper hand of me, that it
    requires a strong moral principle to prevent me from deliberately stepping
    into the street, and methodically knocking people's hats off then, I account
    it high time to get to sea as soon as I can.
  </p>
</div>
```

```
@font-face {
  font-family: "Amstelvar VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 75% 150%;
  font-style: normal;
  font-display: swap;
}

body {
  font:
    1.2em "Amstelvar VF",
    "Georgia",
    serif;
  margin: 20px;
  padding: 0;
}

.container * {
  margin: 0.5rem auto 1rem;
  max-width: 42rem;
}
```

```
.container1 h1 {
  font-optical-sizing: auto;
  font-size: 5rem;
  font-stretch: 85%;
  font-weight: 450;
}
.container1 h2 {
  font-optical-sizing: auto;
  font-size: 2.25rem;
  font-stretch: 90%;
  font-weight: 575;
}
.container1 p {
  font-optical-sizing: auto;
  font-size: 1rem;
  font-stretch: 100%;
  font-weight: 375;
}
.demo2 {
  --text-wght: 375;
  --text-wdth: 100;
  --text-opsz: 16;
  --text-GRAD: 88;
}
.container2 > * {
  font-size: 5rem;
  font-variation-settings:
    "wght" var(--text-wght),
    "wdth" var(--text-wdth),
    "opsz" var(--text-opsz),
    "GRAD" var(--text-GRAD);
}
.container2 h1 {
  --text-wght: 450;
  --text-wdth: 85;
  --text-opsz: 80;
  font-size: 5rem;
}
.container2 h2 {
  --text-wght: 575;
  --text-wdth: 95;
  --text-opsz: 36;
  font-size: 2.25rem;
}
.container2 h2:hover {
  --text-GRAD: 130;
}
.container2 p {
  font-size: 1rem;
}
```

Resources

- W3C CSS Fonts Module 4 Specification (editor's draft)
- W3C GitHub issue queue
- Microsoft Open Type Variations introduction
- Microsoft OpenType Design-Variation Axis Tag Registry
- Wakamai Fondue (a site that will tell you what your font can do via a drag-and-drop inspection interface)
- Axis Praxis (the original variable fonts playground site)
- V-Fonts.com (a catalog of variable fonts and where to get them)
- Font Playground (another playground for variable fonts with some very unique approaches to user interface)

Web Open Font Format (WOFF)

WOFF (the **Web Open Font Format**) is a web font format developed by Mozilla in concert with Type Supply, LettError, and other organizations. It uses a compressed version of the same table-based `sfnt` structure used by TrueType, OpenType, and Open Font Format, but adds metadata and private-use data structures, including predefined fields allowing foundries and vendors to provide license information if desired.

There are three main benefits to using WOFF:

1. The font data is compressed, so sites using WOFF will use less bandwidth and will load faster than if they used equivalent uncompressed TrueType or OpenType files.
2. Many font vendors that are unwilling to license their TrueType or OpenType format fonts for use on the web will license WOFF format fonts. This improves availability of fonts to site designers.
3. Both proprietary and free software browser vendors like the WOFF format, so it has the potential of becoming a truly universal, interoperable font format for the web, unlike other current font formats.

There are two versions of WOFF: WOFF and WOFF2. They mostly differ in regard to the compression algorithm used. In `@font-face` they are identified by the `'woff'` and `'woff2'` format descriptor respectively.

Using WOFF

You can use the `@font-face` CSS property to use WOFF fonts for text in web content. It works exactly like OpenType and TrueType format fonts do, except it will likely let your content download more efficiently due to the addition of compression.

Tools for working with WOFF fonts

- Tools for working with WOFF fonts are available. `sfnt2woff` and `woff2sfnt` convert between WOFF and OpenType.

Browser compatibility

See also

- `@font-face`

CSS fragmentation

The **CSS fragmentation** module defines how content is displayed when it is broken (fragmented) and flows across multiple pages, regions, or columns. This module defines features for pagination, breaking variable fragment size and orientation, widows and orphans.

This module explains how content breaks across fragmentation containers and how such breaks can be controlled by the author. The generic term for breaking content across containers is *fragmentation*. Fragmentation occurs when an inline box wraps onto multiple lines. This can happen when using layout features such as CSS multi-column layout: For example, when a block spans more than one column inside a column layout container, or spans a page break when printed. Each piece of the rendering for the element is called a *fragment*.

When content is physically printed or displayed as a print preview, there are page breaks. In this paged media, as opposed to continuous media, document content can be split between one or more pages, or fragments. To avoid awkward breaks, such as halfway through a line of text or with a lone word on its own page, browsers can move content that would otherwise fall across the page break in a process is called *pagination*.

Reference

Properties

- `box-decoration-break`
- `break-after`
- `break-before`
- `break-inside`
- `orphans`
- `widows`

Glossary terms

- Fragmentainer

Guides

Handling content breaks in multi-column layout

> Introduction to the fragmentation specification and how to control where column content breaks.

Related concepts

- `overflow`
- `height`, `max-height`, and `block-size` CSS properties
- `width`, `max-width`, and `inline-size` CSS properties
- Block formatting context

CSS paged media module

- `page`
- `@page`

CSS display module

- Principal box term
- Block formatting context guide

See also

- `horizontal-viewport-segments` and `vertical-viewport-segments` `@media` descriptors
- Viewport Segments API
- CSS multi-column layout module

CSS gaps

The **CSS gaps** module lets you specify spacing, or "gaps", between items in multi-column, flexbox, and grid layouts. The CSS multi-column layout module originally defined gaps and rules that control the spacing between columns in multi-column containers. This module expands on these visible separators, called *gap decorations*, extending it to grid and flexbox.

While `margin` and `padding` specify visual spacing around individual boxes, the properties in this module enable specifying the spacing between adjacent boxes within a given layout context for layouts that have gutters and gaps, when the spacing is different between sibling boxes as opposed to between the first box, last box, and the container's edge. You can show rules in every gap or in a subset of gaps, defining fully animatable rule widths, colors, and insets.

Gap decorations are painted within a gap as one or more gap decoration segments, with segments occurring between any two adjacent items. When all segments are drawn, column and row rules span the entire height and width of the container. The `rule-visibility-items` property defines whether segments are to be drawn around areas that are not occupied by items. The `rule-break` properties determine if lines break when they intersect a gap, while `rule-inset` defines where segments start and end when rules do break. If they don't break, the `rule-overlap` property defines the painting order of the rules.

Gaps in action

In this example, the 2021 poem from the USA inauguration, *The Hill We Climb*, by Amanda Gorman, is displayed across multiple columns, similar to the way articles are displayed in printed newspapers. If you have JavaScript enabled, controls enable changing the `column-gap`, `column-rule-color`, `column-rule-style` and `column-rule-width` properties, as well as the preferred number of columns and whether the title and a quote should span all of the columns.

```
<article>
  <div class="title">
    <h1>The Hill We Climb</h1>
    <p>&mdash;Amanda Gorman, 2021</p>
  </div>
  <p>
    When day comes, we ask ourselves where can we find light in this never
    ending shade? The loss we carry, a sea we must wade. We braved the belly of
    the beast.
  </p>

  <p>
    We've learned that quiet isn't always peace and the norms and notions of
    what just is, isn't always justice. And yet the dawn is hours before we knew
    it, somehow we do it, somehow we've weathered and witnessed a nation that
    isn't broken but simply unfinished.
  </p>

  <p>
    We, the successors of a country and a time, where a skinny black girl
    descended from slaves and raised by a single mother can dream of becoming
    president, only to find herself reciting for one.
  </p>

  <p>
    And yes, we are far from polished, far from pristine, but that doesn't mean
    we are striving to form a union that is perfect. We are striving to forge
    our union with purpose, to compose a country committed to all cultures,
    colors, characters, and conditions of man. And so we lift our gazes not to
    what stands between us but what stands before us. We close the divide
    because we know to put our future first. We must first put our differences
    aside.
  </p>

  <p>
    We lay down our arms so we can reach out our arms to one another We seek
    harm to none and harmony for all. Let the globe, if nothing else, say this
    is true, that even as we grieved, we grew. That even as we hurt, we hoped.
  </p>

  <p>
    That even as we tired, we tried. That we'll forever be tied together,
    victorious, not because we will never again know defeat, but because we will
    never again sow division.
  </p>

  <p>
    Scripture tells us to envision that everyone shall sit under their own vine
    and fig tree, and no one shall make them afraid.
  </p>

  <p>
    If we’re to live up to our own time, then victory won't lighten the blade
    but in all the bridges we've made, that is the promise to glade, the hill we
    climb if only we dare, it's because being American is more than a pride we
    inherit. It's the past we stepped into and how we repair it.
  </p>

  <blockquote>
    <p>
      We've seen a force that would shatter our nation rather than share it,
      would destroy our country if it meant delaying democracy.
    </p>
  </blockquote>

  <p>
    And this effort very nearly succeeded. But while democracy can be
    periodically delayed, it can never be permanently defeated. In this truth,
    in this faith, we trust. For while we have our eyes on the future, history
    has its eyes on us.
  </p>

  <p>
    This is the era of just redemption. We feared -- at its deception. We did
    not feel prepared to be the heirs of such a terrifying hour, but within it
    we found the power to author a new chapter, to offer hope and laughter to
    ourselves.
  </p>

  <p>
    So, while once we asked, "how could we possibly prevail over catastrophe?",
    now we assert, "how could catastrophe possibly prevail over us?" We will not
    march back to what was, but move to what shall be, a country that is bruised
    but whole, benevolent but bold, fierce and free. We will not be turned
    around or interrupted by intimidation.
  </p>

  <p>
    Because we know our inaction and inertia will be the inheritance of the next
    generation. Our blunders become their burdens. But one thing is certain. If
    we merge mercy with might and might with right, then love becomes our legacy
    and change, our children's birth right.
  </p>

  <p>
    So let us leave behind a country better than one we were left with, every
```

```
    breath from my bronze pounded chest, we will raise this wounded world into a
    wondrous one. We will rise through the gold-limbed hills in the west, we
    will rise from the windswept northeast where our forefathers first realized
    revolution. We will rise from the lake-rimmed cities of the Midwestern
    states.
  </p>

  <p>
    We will rise from the sun-baked South. We will rebuild, reconcile, and
    recover, in every known nook of our nation, in every corner called our
    country, our people diverse and beautiful, will emerge battered and
    beautiful.
  </p>

  <p>When day comes, we step out of the shade, aflame and unafraid.</p>

  <p>
    The new dawn blooms as we free it for there is always light if only we're
    brave enough to see it, if only we're brave enough to be it.
  </p>
</article>
<fieldset id="options" class="open">
  <legend>
    <button aria-expanded="true" aria-controls="controls">
      Column options
    </button>
  </legend>
  <div id="controls">
    <p>
      <input type="checkbox" checked id="colSpan" />
      <label for="colSpan">Byline spans all columns</label>
    </p>
    <p>
      <input type="checkbox" id="blockSpan" />
      <label for="blockSpan">Blockquote spans all columns</label>
    </p>
    <section>
      <p>
        <label for="colCount">column-count</label>
        <input type="number" min="0" max="5" value="5" id="colCount" />
      </p>
      <p>
        <label for="colHeight">column-height:</label>
        <input
          type="number"
          min="0"
          max="100"
          value="20"
          id="colHeight"
          step="5" /><label for="colHeight">vh</label>
      </p>
      <p>
        <label for="colColor">rule-color:</label>
        <input type="color" id="colColor" value="#FF0000" />
      </p>
      <p>
        <label for="columnRuleStyle">rule-style:</label>
        <select id="columnRuleStyle">
          <option>none</option>
          <option>hidden</option>
          <option>dotted</option>
          <option>dashed</option>
          <option>solid</option>
          <option selected>double</option>
          <option>groove</option>
          <option>ridge</option>
          <option>inset</option>
          <option>outset</option>
          <option></option>
        </select>
      </p>
      <p>
        <input type="range" min="0" max="4" value="1" step="0.5" id="gapSize" />
        <label for="gapSize">gap: </label
        ><output id="gap" class="output">1em</output>
      </p>
      <p>
        <input
          type="range"
          min="0"
          max="3"
          value="0.3"
          step="0.1"
          id="columnRuleWidth" />
        <label for="columnRuleWidth">rule-width: </label
        ><output id="ruleWidth" class="output">0.3em</output>
      </p>
      <p>
        <input
          type="range"
          min="-50"
          max="200"
          value="0"
          step="5"
          id="ruleInset" />
        <label for="ruleInset">rule-inset: </label
        ><output id="inset" class="output">0%</output>
      </p>
    </section>
  </div>
</fieldset>
```

```
const page = document.querySelector("article");
const title = document.querySelector(".title");
const option = document.querySelector("#options");
const legend = document.querySelector("#options > legend");
const legendBtn = document.querySelector("#options > legend > button");
const blockquote = document.getElementsByTagName("blockquote")[0];

const colCount = document.getElementById("colCount");
const colSpan = document.getElementById("colSpan");
const blockSpan = document.getElementById("blockSpan");

const gapSize = document.getElementById("gapSize");
const gap = document.getElementById("gap");
const columnRuleWidth = document.getElementById("columnRuleWidth");
const ruleWidth = document.getElementById("ruleWidth");
const columnRuleStyle = document.getElementById("columnRuleStyle");
const ruleStyle = document.getElementById("ruleStyle");
const columnRuleColor = document.getElementById("colColor");
const colHeight = document.getElementById("colHeight");
const ruleInset = document.getElementById("ruleInset");
const inset = document.getElementById("inset");

// Make options visible if JavaScript is enabled
option.style.display = "revert";

legendBtn.addEventListener("click", () => {
  showAndHideMenu();
});

colCount.addEventListener("change", () => {
  page.style.columnCount = colCount.value;
});

colHeight.addEventListener("input", () => {
  page.style.columnHeight = `${colHeight.value}vh`;
});

gapSize.addEventListener("input", () => {
  page.style.gap = `${gapSize.value}em`;
  gap.innerText = `${gapSize.value}em`;
});
ruleInset.addEventListener("input", () => {
  page.style.ruleInset = `${ruleInset.value}%`;
  inset.innerText = `${ruleInset.value}%`;
});

columnRuleWidth.addEventListener("input", () => {
  page.style.columnRuleWidth = `${columnRuleWidth.value}em`;
  page.style.ruleWidth = `${columnRuleWidth.value}em`;
  ruleWidth.innerText = `${columnRuleWidth.value}em`;
});

columnRuleStyle.addEventListener("input", () => {
  page.style.columnRuleStyle = columnRuleStyle.value;
  page.style.ruleStyle = columnRuleStyle.value;
});

colSpan.addEventListener("change", () => {
  setColSpan(colSpan, title);
});

blockSpan.addEventListener("change", () => {
  setColSpan(blockSpan, blockquote);
});

columnRuleColor.addEventListener("input", () => {
  page.style.columnRuleColor = columnRuleColor.value;
  page.style.ruleColor = columnRuleColor.value;
});

function showAndHideMenu() {
  if (legendBtn.getAttribute("aria-expanded") === "true") {
    // close it
    legendBtn.setAttribute("aria-expanded", "false");
    legend.classList.add("closed");
    legend.classList.remove("open");
  } else {
    // open it
    legendBtn.setAttribute("aria-expanded", "true");
    legend.classList.remove("closed");
    legend.classList.add("open");
  }
}

function setColSpan(control, element) {
  if (control.checked) {
    element.style.columnSpan = "all";
  } else {
    element.style.columnSpan = "none";
  }
}
```

```
article {
  column-count: 5;
  gap: 1em;
  column-rule: 0.3em double red;
  rule: 0.3em double red;
  column-height: 20vh;
}
.title {
  column-span: all;
  display: flex;
```

```
  align-items: baseline;
  gap: 1em;
  flex-wrap: wrap;
}
p {
  margin: 0 0 1em 0;
  line-height: 1.4;
}
blockquote {
  font-weight: bold;
  font-style: italic;
  margin: 0 0 0.25em 0;
}
blockquote p::before,
blockquote p::after {
  content: '"';
  vertical-align: baseline;
  color: red;
}

@layer form {
  #options {
    position: fixed;
    top: 1rem;
    right: 1rem;
    background: white;
    display: none;
    padding: 0.5em 1em;
  }
  section {
    font-family: monospace;
  }
  fieldset p {
    margin-bottom: 0.25em;
  }
  legend {
    position: relative;
    top: 0;
    transition: 200ms;
  }
  legend.closed {
    top: 0.75em;
  }
  legend.closed + #controls {
    display: none;
  }
  legend {
    background-color: #dedede;
    padding: 0.5em;
  }
  legend > button {
    all: unset;
    cursor: pointer;
  }
  legend.closed {
    margin: -1em;
    display: inline-block;
  }
  .output {
    display: inline-block;
    width: 2em;
  }
}
```

When the column rule is larger than the column gap, the decorative line appears behind the text; it doesn't change the size of the gap.

Reference

Properties

- column-gap
- column-rule
- column-rule-break
- column-rule-color
- column-rule-inset
- column-rule-inset-cap
- column-rule-inset-cap-end
- column-rule-inset-cap-start
- column-rule-inset-end
- column-rule-inset-junction
- column-rule-inset-junction-end
- column-rule-inset-junction-start
- column-rule-inset-start
- column-rule-style
- column-rule-visibility-items
- column-rule-width
- gap
- row-gap
- row-rule
- row-rule-break
- row-rule-color
- row-rule-inset
- row-rule-inset-cap
- row-rule-inset-cap-end
- row-rule-inset-cap-start
- row-rule-inset-end
- row-rule-inset-junction

- `row-rule-inset-junction-end`
- `row-rule-inset-junction-start`
- `row-rule-inset-start`
- `row-rule-style`
- `row-rule-visibility-items`
- `row-rule-width`
- `rule`
- `rule-break`
- `rule-color`
- `rule-inset`
- `rule-inset-cap`
- `rule-inset-end`
- `rule-inset-junction`
- `rule-inset-start`
- `rule-overlap`
- `rule-style`
- `rule-visibility-items`
- `rule-width`

Terms and glossary definitions

- Grid
- Grid cell
- Grid column
- Grid lines
- Grid row
- Gutters

Guides

Defining CSS gaps

Guide to understanding and defining gaps in grid, flexbox, and multi-column layouts, including how percentage values are resolved.

Defining CSS gap rules

Guide to understanding and defining CSS gap rules, including using the `repeat()` function, to define rule colors, widths, and styles.

Styling columns

Guide to styling columns and managing spacing between columns.

Handling content breaks in multi-column layout

Introduction to the fragmentation specification and how to control where column content breaks.

Box alignment guides

How box alignment works in the context of flexbox, grid layout, multiple-column layout, and for block, absolutely positioned and table layout.

Related concepts

CSS flexible box layout module

- `flex`
- `flex-basis`
- `flex-direction`
- `flex-flow`
- `flex-grow`
- `flex-shrink`
- `flex-wrap`

CSS grid layout module

- `grid`
- `grid-column`
- `grid-row`
- `repeat()`

CSS multi-column layout module

- `column-fill`
- `column-span`
- `columns` shorthand
  - `column-count`
  - `column-height`
  - `column-width`
- `column-wrap`
- `::column`

CSS box alignment

- `align-content`
- `justify-content`

CSS box sizing module

- `height`
- `max-height`
- `block-size`
- `width`
- `max-width`

CSS display module

- `display`
- Block formatting context guide

See also

- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout

Defining CSS gaps

When creating grid, flexbox, and multi-column layouts using CSS gap properties, you can define and control gaps between columns and rows.

While the `margin` and `padding` properties specify visual spacing around individual boxes, using the properties in the CSS gaps module, you can specify spacing between adjacent boxes in layouts that have gutters and gaps.

This guide explains column and row gaps in different layout types, how to define gaps, and how to use percentages as a `gap` value.

Understanding gaps

While margin and padding can specify spacing around individual boxes, it's sometimes more convenient to specify spacing between adjacent boxes within a given layout. This is particularly true when the spacing between sibling boxes differs from the spacing between the first or last box and the container's edge. The `gap` property, and its `row-gap` and `column-gap` constituent properties, provide this functionality for grid, flexbox, and multi-column layouts.

A *gap* is either a *column gap* or a *row gap*. Their definitions vary by layout type. For all layout types, any gap disappears when it coincides with a fragmentation break.

Grid container gaps

In the context of a grid container, *row gaps* and *column gaps* refer to the gutters between grid rows and grid columns, respectively. The width of the gaps makes the affected grid lines behave as though they acquired thickness: the grid track between two grid lines is the space between the gutters that represent them. By default, the width of the gap is `0` in both directions.

For the purpose of track sizing, each gutter is treated as an extra, empty, fixed-size track of the specified size. A grid item that spans multiple rows or columns also spans the gutters between them.

For example, if `gap: 20px` is set on a 4x4 grid of `100px` by `100px` boxes, the grid would be `460px` by `460px`. While each box is `100px` by `100px`, a grid item that spans two horizontal rows has a height of `220px`. If it spans three, its height is `340px`. If it spans all four, its height is `460px`.

Gutter gaps set the minimum spacing between items: additional spacing may be added by `justify-content` and `align-content` property values, increasing the size of the corresponding gaps.

Gutters appear only between tracks of the implicit grid. If a grid is fragmented between tracks, no gutter spacing is added between those tracks. There is no gutter before the first track or after the last track. If a track is collapsed, it has no gutter.

Flex container gaps

Flex containers are created by setting `display` to `flex` or `inline-flex` on an element containing multiple children. By default, flex items are laid out in a single, non-wrapping row. The default gap between adjacent flex items and, if wrapping, between adjacent columns or rows, is `0`. Whether a flex container with multiple items has columns, rows, or both depends on the flow and wrapping set using the `flex-flow` shorthand.

You can add gaps between adjacent flex items along the main axis. If the `flex-flow` property is set to `row wrap` or `row-reverse wrap`, *column gap* refers to the gutter between adjacent flex items and *row gap* refers to the gutter between flex rows. If `flex-flow` is set to `column wrap` or `column-reverse wrap`, *row gap* refers to the gutter between adjacent flex items and *column gap* refers to the gutter between flex rows.

Multi-column gaps

Multi-col containers are block-level elements with more than one column created by setting the `column-count` to a value greater than `1`. By default, columns are laid out in a single row, with a `1em` wide *column gap* between adjacent columns. The *row gap* is the gutter between rows of column boxes established by setting a `column-height` that requires columns to wrap, creating additional rows.

Using the <code>gap</code> shorthand

The `row-gap` property sets the size of the gap (gutter) between adjacent rows within a container. The `column-gap` property sets the size between a container's columns. Each property value can be specified as a `<length>`, a `<percentage>`, or the keyword `normal`. Percentage values are calculated against the content box size of the container element for that dimension.

The `gap` shorthand defines gaps between both rows and columns and accepts one or two values. The default value is `normal` for both sub-properties. If only one value is declared, it applies to both row and column gaps. If two values are declared, the first sets the `row-gap` value and the second sets the `column-gap`.

The effect of the definition depends on whether the container uses grid, flexbox, or multi-column layout.

You can add visible separators to gaps; these are called gap decorations. If you add decorative rules for gaps between columns, rows, or both, they appear in the middle of their gap but have no effect on the size of the gap, nor do they affect the size of the container. These gap decorations are added to the otherwise "empty space" by using the `rule` shorthand or its constituent properties.

Gaps in grid layouts

In grid containers, the `gap` property defines the size of the gutters between vertical and horizontal tracks. The shorthand property is specified as a value for `<'row-gap'>`, followed optionally by a value for `<'column-gap'>`. If only one value is declared, it applies to both row and column gaps.

In this example, we create a grid container with seven columns:

```
.container {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
}
```

Select different `gap` values to see how they change the spacing between rows and columns:

```
:has([value="a"]:checked) p {
  gap: 2px 10px;
}
:has([value="b"]:checked) p {
  gap: 10px 2px;
}
:has([value="c"]:checked) p {
  gap: 10px;
}
:has([value="d"]:checked) p {
  gap: 2px;
}
```

Gaps in flexbox layouts

In flex containers, the `gap` property defines the space between both flex items and flex lines. Whether the first value sets the gap between flex items or between flex lines depends on the direction in which the flex items are laid out.

Flex items are laid out in rows or columns depending on the value of the `flex-direction` property. If set to `row` or `row-reverse`, the first value defines the gap between flex lines, and the second value defines the gap between adjacent flex items within each line. If only one value is included, it applies to both gaps.

If the `flex-direction` is set to `column` or `column-reverse`, the first value defines the gap between adjacent flex items within a flex line, and the second value defines the gaps between flex lines. Again, if only one value is included, it applies to both gaps.

In this example, we create a flex container and allow the flex items to wrap:

```
.container {
  display: flex;
  flex-wrap: wrap;
  max-width: 300px;
  height: 500px;
}
```

Select different `gap` and `flex-direction` values to see how they change the spacing between flex items and flex lines:

```
i {
  flex: 0 0 18%;
}
i:nth-of-type(2n) {
  flex: 0 0 12%;
}
i:nth-of-type(3n) {
  flex: 0 0 26%;
}
i:nth-of-type(5n) {
  flex: 0 0 20%;
}
i:nth-of-type(7n) {
  flex: 0 0 34%;
}
```

```
:has([value="a"]:checked) p {
  gap: 2px 10px;
}
:has([value="b"]:checked) p {
  gap: 10px 2px;
}
:has([value="c"]:checked) p {
  gap: 10px;
}
:has([value="d"]:checked) p {
  gap: 2px;
}

:has([value="row"]:checked) p {
  flex-direction: row;
}
:has([value="rowR"]:checked) p {
  flex-direction: row-reverse;
}
:has([value="col"]:checked) p {
  flex-direction: column;
}
:has([value="colR"]:checked) p {
  flex-direction: column-reverse;
}
```

```
<fieldset>
  <legend>Select a flex direction:</legend>
  <label
    ><input type="radio" value="row" name="dir" /> flex-direction: row;</label
  >
  <label
    ><input type="radio" value="rowR" name="dir" /> flex-direction:
    row-reverse;</label
  >
  <label
    ><input type="radio" value="col" name="dir" /> flex-direction:
    column;</label
  >
  <label
    ><input type="radio" value="colR" name="dir" /> flex-direction:
    column-reverse;</label
  >
</fieldset>
```

```
<fieldset>
  <legend>Select a layout</legend>
  <label><input type="radio" value="grid" name="dir" checked />Grid</label>
  <label><input type="radio" value="flex" name="dir" />Flexbox</label>
  <label
    ><input type="radio" value="flex2" name="dir" />Flexbox (columns)</label
  >
  <label><input type="radio" value="multicol" name="dir" />Multi-col</label>
</fieldset>
```

Gaps in multi-column layouts

In CSS multi-column layouts, the `gap` property defines the gutter between columns and between rows of columns. The first value defines the gap between rows of column boxes, if multiple rows are established by the `column-height` property. The second value defines the gap between adjacent column boxes.

In this example, we create a multi-col container using the `columns` shorthand property, setting a maximum of seven columns and a minimum column width of `1em`. A column height of `2.35em` enables the creation of additional rows. We also add a thin decorative rule in the middle of the gap using the `rule` property:

```
.container {
  columns: 7 1em / 2.35em;
  width: 450px;
  rule: 1px solid #cccccc;
}
@supports not (column-height: 1em) {
  body::before {
    content: "Your browser does not support the column-height property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

By default, there is a `1em` gap between rows and columns. Change this by selecting different `gap` values:

```
:has([value="a"]:checked) p {
  gap: 0.5em 3em;
}
:has([value="b"]:checked) p {
  gap: 3em 0.5em;
}
:has([value="c"]:checked) p {
  gap: 0.5em;
}
:has([value="d"]:checked) p {
  gap: 3em;
}
```

The gutters may appear larger than the specified gap size because the letters don't fill the allotted space. The decorative rule appears in the middle of the gap, either `0.25em` or `1.5em` from the block and inline starts of the column and row's content depending on the setting selected. The extra whitespace is at block- and inline-end, making the gaps look larger than they are.

```
<fieldset>
  <legend>Select a gap value:</legend>
  <label><input type="radio" value="a" name="gap" /> gap: 2px 10px;</label>
  <label><input type="radio" value="b" name="gap" /> gap: 10px 2px;</label>
  <label><input type="radio" value="c" name="gap" /> gap: 10px;</label>
  <label><input type="radio" value="d" name="gap" /> gap: 2px;</label>
</fieldset>
```

```
<fieldset>
  <legend>Select a gap value:</legend>
  <label><input type="radio" value="a" name="gap" /> gap: 0.5em 3em;</label>
  <label><input type="radio" value="b" name="gap" /> gap: 3em 0.5em;</label>
  <label><input type="radio" value="c" name="gap" /> gap: 0.5em;</label>
  <label><input type="radio" value="d" name="gap" /> gap: 3em;</label>
</fieldset>
```

```
<fieldset>
  <legend>Select a gap value:</legend>
  <label><input type="radio" value="a" name="gap" /> gap: 1% 5%;</label>
  <label><input type="radio" value="b" name="gap" /> gap: 5% 1%;</label>
  <label><input type="radio" value="c" name="gap" /> gap: 1%;</label>
  <label><input type="radio" value="d" name="gap" /> gap: 5%;</label>
</fieldset>
```

```
<div>
  <p class="container">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </p>
</div>
```

```
.container {
  font-size: 2rem;
  font-family: monospace;
  font-weight: bold;
}
i {
  background-color: #cccccc;
  text-align: center;
```

```
}
label {
  display: block;
  font-family: monospace;
  margin: 10px;
}
```

Specifying gap values as percentages

When a container has a fixed size, column and row gaps specified as percentages are calculated relative to the width and height of the container, respectively.

In this example, the size of the container is set. Select different gap sizes as percentage values, and change the layout type to see how the gaps are calculated relative to the container's size. Column gaps of `1%` and `5%` are `3px` and `15px` wide, respectively. Row gaps of `1%` and `5%` are `6px` and `30px` tall, respectively. These gap sizes apply even if the contents overflow the container.

```
.container {
  width: 300px;
  height: 600px;
  background-color: #eeeeee;
  rule: 1px dotted #666666;
}
```

Note that if you set the `gap` property to a single percentage value, the row gap is twice the size of the column gap because the container is twice as tall as it is wide.

```
fieldset {
  width: 44%;
  float: left;
}
p {
  clear: both;
  font-size: 1.25rem;
}

:has([value="a"]:checked) p {
  gap: 1% 5%;
}
:has([value="b"]:checked) p {
  gap: 5% 1%;
}
:has([value="c"]:checked) p {
  gap: 1%;
}
:has([value="d"]:checked) p {
  gap: 5%;
}

:has([value="grid"]:checked) p {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
}
:has([value="flex"]:checked) p {
  display: flex;
  flex-flow: row wrap;
}
:has([value="flex2"]:checked) p {
  display: flex;
  flex-flow: column wrap;
}
:has([value="multicol"]:checked) p {
  columns: 5 2em / 2.35em;
}
```

If we don't have a defined height or width for the container, the behavior of percentage gap values is very different. If the container has a fixed width, percentage gap values are predictable. If the container is auto-sized, percentage gaps could create a circular dependency: the gaps depend on the container's size, but the container's size depends on the gaps. Browsers treat percentage gap values as `auto` (effectively `0`) during intrinsic sizing.

```
.container {
  background-color: #eeeeee;
  rule: 1px dotted #666666;
  height: auto;
  width: auto;
}
```

In the example, the width of the container is constrained by the containing block, while the height is not.

As percentage values are treated as `auto` during intrinsic sizing for gaps in grid layout, the gap is collapsed until the size of the container is determined. This means the size of the container is determined based solely on the dimensions of the content. If the example renders six rows of grid cells, there will be five row gaps, meaning the last row of grid items will overflow the background by either `5%` or `25%`, depending on whether the gap is set to `1%` or `5%`.

In flexbox layouts, the percentage gaps are treated as `0` or ignored during intrinsic sizing. The gap is applied only after sizing. As the container's block size is `auto`, the percentage row gaps resolve against `0`; so `1%` or `5%` of `0` is `0`. Percentage values are effectively ignored - the `row-gap` is `0` for both flexbox and multi-col layouts.

See also

- CSS gaps module
- Aligning items in a flex container
- Box alignment in grid layout

Defining CSS row and column gap rules

When creating grid, flexbox, and multi-column, you can add decorative lines, or *rules*, in the center of the gutters between rows and columns. You can show rules in every gap or a subset of gaps, and fully animate rule widths, colors, and insets. This guide explains how to specify the width, color, and style of these rules.

Overview

The CSS gaps module defines several properties that specify the appearance of lines painted behind content in a grid, flexbox, or multi-col container's gutters, or gaps. These lines don't affect the box model or layout. The size of the gutter is defined by the `gap` property, with the default value being `1em` on multi-column containers and `0` in all other contexts.

If a rule is wider than the gap, the lines are painted behind the content to avoid obscuring it. You should still ensure the color contrast of the content and both the background and the rule are sufficient to ensure legibility.

To learn more about specifying the spacing between adjacent boxes within a given layout context for layouts that have gutters and gaps, see Defining CSS gaps.

All the rule color, line-width, and line-style properties accept a comma-separated list of values, allowing for varying gap decorations within a container.

```
rule-color: blue;
rule-width: 10px, 20px;
rule-style: solid, dashed, solid, dotted;
```

You can use the `repeat()` function within the list to define a set number of repetitions, or to automatically add as many repetitions as needed to provide a value for every line drawn.

```
rule-color: blue, repeat(3, gray, red);
rule-width: 50px, repeat(10, 10px, 20px), 10px;
rule-style: solid, repeat(auto, dashed, dotted), solid;
```

The rule color, line-width, and line-style shorthand properties set the same values for both column and row rules.

The <code>repeat()</code> function

Before introducing the column and row rule properties, it is important to understand the `repeat()` function.

The `repeat()` function can be used to repeat a valid list of CSS values. The first argument specifies how many times to repeat the list of values passed as subsequent arguments. This allows color, line-width, or line-style values to repeat as many times as needed without listing them individually.

The following declarations are equivalent, with both creating a list of eight colors:

```
rule-color: blue, yellow, red, yellow, red, yellow, red, gray;
rule-color: blue, repeat(3, yellow, red), gray;
```

If the number of values exceeds the number of gaps, the excess values are ignored. For example, in a 3x3 grid container, the first row and column rules will be blue and the second yellow. The other color values in the list are unused.

If there are more gaps than values, the list is cycled. In a 10x10 grid container, there are nine gaps in each direction. In this case, the last rules are blue; the first eight row and column rules are each assigned the associated color, then the list cycles, so the ninth row and column rules get the first value in the list.

Here, with only three rows, only two values are used to color the row gaps, while the list is cycled to provide `<color>` values to each of the eleven column rules.

```
div {
  rule-color: blue, repeat(3, yellow, red), gray;
  grid-template-columns: repeat(12, 1fr);
}
```

The `repeat()` function's first parameter is either an `<integer>` of `1` or greater or the keyword `auto`. If the number is invalid (`0`, negative, missing, or a non-integer number), the entire declaration is invalid and ignored. The `auto` keyword within the `repeat()` function creates an *auto-repeater* that fills in values that would otherwise not receive values from other parts of the list, preventing the list from cycling. A property value can include, at most, one `repeat(auto, <value-list>)` auto-repeater.

By setting the following, in a grid that is 3x3 or larger, the `blue` and `gray` rules will each occur exactly once in each direction:

```
rule-color: blue, repeat(auto, yellow, red), gray;
```

In this case, `yellow` and `red` are repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value. The first rule will always be blue. If there is more than one rule, the last will always be gray. All other rules will cycle between `yellow` and `red`.

```
div {
  rule-color: blue, repeat(auto, yellow, red), gray;
  grid-template-columns: repeat(12, 1fr);
}
```

Line properties

Every visible rule has a color, width, and style. Each of these can be defined as a single value or as a list of values.

Color properties

The `column-rule-color` property specifies the color of lines drawn between columns, the `row-rule-color` property defines the color of lines drawn between rows, and the `rule-color` property sets both the `column-rule-color` and `row-rule-color` to the same value. These properties are all specified as a comma-separated list of values, which can include:

`<color>`

> A valid `<color>` value specifying the color of a line. The default is `currentcolor`.

`<repeat-line-color>`

> A `repeat()` function, with an `<integer>` as the first argument and one or more `<color>` values as subsequent arguments. The `<integer>`, with a value of `1` or more, specifies how many times the list of `<color>` values should be repeated.

`<auto-repeat-line-color>`

> A `repeat()` function, with `auto` as the first argument and one or more `<color>` values as subsequent arguments. This creates an auto-repeater that assigns `<color>` values to gap rules that would otherwise not receive values from the other components of the property value, preventing the property's comma-separated list from cycling. At most, you can include only one `<auto-repeat-line-color>` value in the list.

By using `auto` inside the `repeat()` function as part of the `rule-color` property, we can set the first and last rules to dark and all middle rules to light.

```
div {
  rule-color: black, repeat(auto, lightgray), black;
}
```

If you change the number of columns from `6` to `4` or `8`, the first and last rules in each direction will still be `black` and all the other rules `lightgray`.

The gray horizontal lines are drawn over the black vertical lines. This is because, by default, the row rules are drawn on top of the column rules. This can be changed by setting the `rule-overlap` property to `column-over-row` instead of its default `row-over-column` value.

Width properties

The `column-rule-width` property specifies the width of the lines drawn between columns, the `row-rule-width` property defines the width of the lines drawn between rows, and the `rule-width` property sets both the `column-rule-width` and `row-rule-width` to the same values. These properties are all specified as a comma-separated list of values, which can include:

`<line-width>`

> Defines the width of the line, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`, which generally resolves to `3px`. Custom properties that resolve to a `<line-width>` are also valid.

`<repeat-line-width>`

> A `repeat()` function, with an `<integer>` as the first argument and one or more `<line-width>` values as subsequent arguments. The `<integer>`, with a value of `1` or more, specifies how many times the list of `<line-width>` values should be repeated.

`<auto-repeat-line-width>`

> A `repeat()` function, with `auto` as the first argument and one or more `<line-width>` values as subsequent arguments. This creates an auto-repeater that assigns `<line-width>` values to gap rules that would otherwise not receive values from the other components of the property value, preventing the property's comma-separated list from cycling. At most, you can include only one `<auto-repeat-line-width>` value in the list.

Unlike the `gap` properties, `rule-width` properties don't affect layout. These properties only define rule width. Rules appear in the middle of the gap, have no effect on the size of the gap, and have no impact on the box model. Rules that are wider than the gap are drawn behind the content.

In this example, the `gap` is specified as `20px`. We define varying rule sizes using the `rule-width` property:

```
div {
  rule-width: 10px, repeat(auto, 20px, 30px), 40px;
}
```

The black rules are centered in the gap. This is most noticeable in the first row and column gaps, where the `10px` line is drawn in the middle of the `20px` gap. Rules are drawn behind the content. This is most noticeable when `rule-width` is larger than the `gap`, and the lines show through the semi-opaque grid items.

The grid items all have black text and a semi-transparent, almost white background. Including a `background-color` on your content helps ensure that overflowing rules don't create accessibility issues. For your content to be accessible, there must be sufficient contrast between a background, be it the background color or the rule color, and the text or other important visual information.

Line-style properties

The `column-rule-style` property specifies the style of the lines drawn between columns, the `row-rule-style` property defines the style of the lines drawn between rows, and the `rule-style` sets both the `column-rule-style` and `row-rule-style` to the same value.

These properties are all specified as a comma-separated list of values, which can include:

`<line-style>`

> A keyword — `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `none`, or `hidden` — which defines the line style of the drawn rule. The default is `none`.

`<repeat-line-style>`

> A `repeat()` function, with an `<integer>` as the first argument and one or more `<line-style>` values as subsequent arguments. The `<integer>`, with a value of `1` or more, specifies how many times the list of `<line-style>` values should be repeated.

`<auto-repeat-line-style>`

> A `repeat()` function, with `auto` as the first argument and one or more `<line-style>` values as subsequent arguments. This creates an auto-repeater that assigns `<line-style>` values to gap rules that would otherwise not receive values from the other components of the property value, preventing the property's comma-separated list from cycling. At most, you can include only one `<auto-repeat-line-style>` value in the list.

As with `border`, the default value is `none`, so a rule must have a `<line-style>` declared with a value other than `hidden` or `none` for it to be painted.

In this example, we assign a list with seven values to the `column-rule-style` property. As the eight-column grid has seven column gaps, each value is used only once. The `row-rule-style` declaration has three values for three row rules, so again, each value is used once.

```
div {
  column-rule-style: solid, dotted, dashed, double, groove, ridge, inset;
  row-rule-style: outset, none, hidden;
}
```

```
div {
  grid-template-columns: repeat(8, 1fr);
}
```

Only the `outset` rules are drawn in the row gutters, as when the `<line-style>` is set to `none` or `hidden`, no line is painted.

Shorthand properties

The `row-rule`, `column-rule`, and `rule` shorthands can be used to declare the colors, line-widths, and line-styles for one or more lines in a single declaration.

These shorthand properties are all specified as a comma-separated list of values, from the following list:

`<gap-rule>`

> Specified as one, two, or three of the values `<line-width>`, `<line-style>`, and `<color>`, in any order. Just like with `border` shorthand values, the `<line-style>` must be included for a line to be painted, as it otherwise defaults to `none`. The default `<line-width>` is `medium` and the default `<color>` is `current-color`.

`<gap-repeat-rule>`

> A `repeat()` function, with an `<integer>` as the first argument and one or more `<gap-rule>` values as subsequent arguments. The `<integer>`, with a value of `1` or more, specifies how many times the list of `<gap-rule>` values should be repeated.

`<auto-repeat-gap-rule>`

> A `repeat()` function, with `auto` as the first argument and one or more `<gap-rule>` values as subsequent arguments. This creates an auto-repeater that assigns `<gap-rule>` values to gap rules that would otherwise not receive values from the other components of the property value, preventing the property's comma-separated list from cycling. At most, you can include only one `<auto-repeat-gap-rule>` value in the list.

The shorthand property declarations are similar to the constituent property values, but each value in the comma-separated list of values is a space-separated list of up to three component values. The `repeat()` function works the same way as with constituent properties, but again, each value is a space-separated list of components.

```
div {
  rule:
    10px solid blue,
    repeat(auto, 20px dashed teal, 5px double orange),
    15px inset rebeccapurple;
}
```

The `rule` shorthand sets both the `column-rule` and `row-rule` to the same value.

```
<section>
  <div>
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
@layer hidden {
  div {
    display: grid;
    grid-template-columns: repeat(6, 1fr);
    gap: 20px;
    rule: solid 10px;
    border: 1px solid;
    overflow: visible;
    margin: 1em;
    background-color: white;
  }
  i {
    padding: 8px;
    border: 1px dashed;
    color: black;
    background-color: #ddfc;
  }
}
@supports not (row-rule-style: solid) {
  body::before {
    content: "Your browser doesn't support these features.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

See also

- Overview of CSS gap decoration properties
- Defining CSS gaps
- CSS gaps module
- CSS borders and box decorations module

CSS generated content

The **CSS generated content** module defines how an element's content can be replaced and content can be added to a document with CSS.

Generated content can be used for content replacement, in which case the content of a DOM node is replaced with a CSS `<image>`. The CSS generated content also enables generating language-specific quotes, creating custom list item numbers and bullets, and visually adding content by generating content on select pseudo-elements as anonymous replaced elements.

Generated content in action

```
<div></div>
```

```
body,
div {
  background-repeat: no-repeat;
}
body {
  background-image: linear-gradient(#3a67ab, #e8f6ff 100%);
}
div {
  position: relative;
  width: 400px;
  height: 400px;
  background-image:
```

```
    linear-gradient(
      115deg,
      transparent 48%,
      brown,
      #996600,
      brown,
      transparent 52%
    ),
    linear-gradient(
      60deg,
      transparent 48%,
      brown,
      #996600,
      brown,
      transparent 52%
    ),
    radial-gradient(
      circle 10px at 50% 50%,
      #333333 30%,
      #999999 50%,
      transparent 50%
    ),
    radial-gradient(
      circle 10px at 50% 50%,
      #333333 30%,
      #999999 50%,
      transparent 50%
    ),
    radial-gradient(circle at 50% 50%, white 30%, #eeeeee 50%, transparent 50%),
    radial-gradient(circle at 50% 50%, white 30%, #eeeeee 50%, transparent 50%),
    radial-gradient(circle at 50% 50%, white 30%, #eeeeee 50%, transparent 50%);
  background-size:
    100px 100px,
    100px 100px,
    15px 15px,
    15px 15px,
    200px 200px,
    300px 300px,
    400px 400px;
  background-position:
    95% 120px,
    5% 120px,
    46% 80px,
    54% 80px,
    50% 0,
    50% 90px,
    50% 220px;
}
div::after {
  content: "";
  border: transparent solid 4px;
  border-left: orange 30px solid;
  height: 1px;
  width: 1px;
  position: absolute;
  left: 50%;
  top: 100px;
}

div::before {
  content: "Only one <div>";
  font-size: min(6vh, 2rem);
  justify-content: center;
  display: flex;
  font-family: "Comic Sans", "Papyrus", sans-serif;
}
```

The HTML for this sample is a single, empty `<div>` inside an otherwise empty `<body>`. The snowman was created with CSS images and CSS backgrounds and borders. The carrot nose was added using generated content: an empty box with a wide orange left border added to the `::before` pseudo-element. The text is also generated content: "only one " was generated with the `content` property applied to the `::after` pseudo-element.

Click "Play" in the example above to see or edit the code in the MDN Playground.

Reference

Properties

- `content`
- `quotes`

The CSS generated content module also introduces four at-risk properties: `string-set`, `bookmark-label`, `bookmark-level`, and `bookmark-state`. Currently, no browsers support these features.

Functions

The CSS generated content module introduces six yet-to-be implemented CSS functions including `content()`, `string()`, and `leader()`, and the three `<target>` functions `target-counter()`, `target-counters()`, and `target-text()`.

Data types

- `<content-list>`
- <content-replacement> (see `<image>`)
- `<image>`
- `<counter>`
- `<quote>`
- `<target>`

Guides

"How to" guide for generated content

Learn how to add text or image content to a document using the `content` property.

Create fancy boxes with generated content

Example of styling generated content for visual effects.

Related concepts

- CSS pseudo-elements module
  - `::before` pseudo-element
  - `::after` pseudo-element
  - `::marker` pseudo-element
- CSS lists and counters module
  - `counter()` function
  - `counters()` function
  - `counter-increment` property
  - `counter-reset` property
- CSS overflow module
  - `::scroll-button()` pseudo-element
  - `::scroll-marker` pseudo-element
  - `:target-current` pseudo-class
- CSS values and units module
  - `attr()` function
  - `<string>` data type
  - `<image>` data type

See also

- CSS pseudo-elements module
- CSS lists and counters module
- Replaced elements

CSS grid layout

The **CSS grid layout** module excels at dividing a page into major regions or defining the relationship in terms of size, position, and layering between parts of a control built from HTML primitives.

Like tables, grid layout enables an author to align elements into columns and rows. However, many more layouts are either possible or easier with CSS grid than they were with tables. For example, a grid container's child elements could position themselves so they actually overlap and layer, similar to CSS positioned elements.

Grid layout in action

The example shows a three-column track grid with new rows created at a minimum of 100 pixels and a maximum of auto. Items have been placed onto the grid using line-based placement.

```
<div class="wrapper">
  <div class="one">One</div>
  <div class="two">Two</div>
  <div class="three">Three</div>
  <div class="four">Four</div>
  <div class="five">Five</div>
  <div class="six">Six</div>
</div>
```

```
* {
  box-sizing: border-box;
}
.wrapper {
  max-width: 940px;
  margin: 0 auto;
}
.wrapper > div {
  border: 2px solid rgb(233 171 88);
  border-radius: 5px;
  background-color: rgb(233 171 88 / 50%);
  padding: 1em;
  color: #d9480f;
}
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  grid-auto-rows: minmax(100px, auto);
}
.one {
  grid-column: 1 / 3;
  grid-row: 1;
}
.two {
  grid-column: 2 / 4;
  grid-row: 1 / 3;
}
.three {
  grid-column: 1;
  grid-row: 2 / 5;
}
.four {
  grid-column: 3;
  grid-row: 3;
}
.five {
  grid-column: 2;
  grid-row: 4;
}
.six {
  grid-column: 3;
```

```
  grid-row: 4;
}
```

This sample animation uses `display`, `grid-template-columns`, `grid-template-rows`, and `gap` to create the grid, and `grid-column` and `grid-row` to position items within the grid. To view and edit the HTML and CSS used, click the 'Play' at the top right of the example.

Reference

Properties

- `grid-auto-columns`
- `grid-auto-flow`
- `grid-auto-rows`
- `grid-template-columns`
- `grid-template-rows`
- `grid-template-areas`
- `grid-template` shorthand
- `grid` shorthand
- `grid-column-start`
- `grid-column-end`
- `grid-column` shorthand
- `grid-row-start`
- `grid-row-end`
- `grid-row` shorthand
- `grid-area` shorthand

Functions

- `repeat()`
- `minmax()`
- `fit-content()`

Data types and values

- `<flex>` (`fr` unit)

Terms and glossary definitions

- Grid
- Grid areas
- Grid axis
- Grid cell
- Grid column
- Grid container
- Grid lines
- Grid row
- Grid tracks
- Gutters

Guides

Basic concepts of grid layout

An overview of the various features provided in the CSS grid layout module.

Relationship of grid layout with other layout methods

How grid layout fits together with other CSS features including flexbox, absolutely positioned elements, and `display: contents`.

Grid layout using line-based placement

Grid lines and how to position items against those lines, including the `grid-area` properties, negative line numbers, spanning multiple cells, and creating grid gutters.

Grid template areas

Placing grid items using named template areas.

Grid layout using named grid lines

Combining names and track sizes; placing grid items by defining named grid lines and template areas.

Auto-placement in grid layout

How grid positions items that don't have any placement properties declared.

Aligning items in CSS grid layout

Aligning, justifying, and centering grid items along the two axes of a grid layout.

Grids, logical values, and writing modes

A look at the interaction between CSS grid layout, box alignment and writing modes, along with CSS logical and physical properties and values.

Grid layout and accessibility

A look at how CSS grid layout can both help and harm accessibility.

Realizing common layouts using grids

A few different layouts demonstrating different techniques you can use when designing with CSS grid layouts, including using `grid-template-areas`, a 12-column flexible grid system, and a product listing using auto-placement.

Subgrid

What subgrid does with use cases and design patterns that subgrid solves.

Grid lanes layout

Details what grid lanes layout is and how to use it.

Box alignment in CSS grid layout

How box alignment works in the context of grid layout.

Defining CSS gaps

Understanding and defining gaps in grid, flexbox, and multi-column layouts, including how percentage values are resolved.

Related features

CSS display module

- `display`
- `order`

CSS box alignment module

- `align-content`
- `align-items`
- `align-self`
- `justify-content`
- `justify-items`
- `justify-self`
- `place-content`
- `place-items`
- `place-self`

CSS gaps module

- `gap` shorthand
  - `column-gap`
  - `row-gap`
- `rule` shorthand
  - `column-rule`
  - `row-rule`
- `rule-break`
- `rule-color`
- `rule-inset`
- `rule-overlap`
- `rule-style`
- `rule-visibility-items`
- `rule-width`

CSS box sizing module

- `aspect-ratio`
- `box-sizing`
- `height`
- `max-height`
- `max-width`
- `min-height`
- `min-width`
- `width`
- `<ratio>` data type
- `min-content` value
- `max-content` value
- `fit-content` value
- `fit-content()` function

See also

- CSS flexible box layout module
- CSS display module
- Grid by example
- CSS grid reference via Codrops
- Firefox DevTools: grid inspector
- CSS grid playground
- CSS grid garden - A game for learning CSS grid

Grid layout and accessibility

HTML is the content layer of a website where we create semantic, well-structured documents. CSS is the presentation layer; we apply CSS to create, among other things, the desired layout for our content. Two-dimensional grid structures are defined using CSS grid layout.

Even though modern HTML and CSS are designed to enable the creation of semantic, accessible content and designs, there are ways to *create* accessibility issues with grids. This article looks at the potential issues that can arise, and how to avoid them.

Re-ordering content in CSS grid layout

We've already seen in these guides that CSS grid layout gives us the power to re-order our page content by positioning items using line-based placement of grid template areas. This placement can be done without considering the item's location in the source. There is also the `order` property, which can change how items auto-place. The `grid-auto-flow` property has a `dense` value, which can take items visually out of the DOM order.

The CSS grid layout specification includes a Reordering and Accessibility section. The introduction to that section details what browsers should do when content is visually reordered using grid layout:

> Grid layout gives authors great powers of rearrangement over the document. However, these are not a substitute for correct ordering of the document source. The `order` property and grid placement do not affect ordering in non-visual media (such as speech). Likewise, rearranging grid items visually does not affect the default traversal order of sequential navigation modes (such as cycling through links, see e.g., `tabindex`).

If you reorder things visually using grid layout, this will not change how the items are ordered if the content is being read out by a screen reader, or other text to speech user agent. In addition, the reordering will not change tab order. This means that someone navigating using the keyboard could be tabbing through links on your site and suddenly find themselves jumping from the top to the bottom of the document due to a reordered item being next in the tab order.

The specification warns authors (the CSSWG term for web developers) not to do this reordering.

> Authors must use `order` and the grid-placement properties only for visual, not logical, reordering of content. Style sheets that use these features to perform logical reordering are non-conforming.

What does this mean for designing with grid layout in practice?

Visual not logical re-ordering

Any time you reorder things with grid layout – or with flexbox – you only perform *visual reordering*. The underlying source is what controls things like text to speech, and the tab order of the document. We can see how this works with an example.

In this example, we have a grid containing five items, each containing a link. The items are placed with line-based placement properties. We've positioned box 1 on the second row of the grid so it visually appears as the fourth item in the list. If we navigate through the links with the tab key, the tab order still begins with box 1, as it comes first in the source.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column: 1;
  grid-row: 2;
}
```

```
<div class="wrapper">
  <div class="box box1"><a href="">One</a></div>
  <div class="box box2"><a href="">Two</a></div>
  <div class="box box3"><a href="">Three</a></div>
  <div class="box box4"><a href="">Four</a></div>
  <div class="box box5"><a href="">Five</a></div>
</div>
```

The specification says that in this scenario, if box 1 really makes sense logically in that position, we should go back to our HTML source and make the change there rather than reordering using grid layout. This is what is meant by visual versus logical reordering; logical ordering is important for the meaning and structure of our document and we should make sure that we preserve that structure.

How should we approach accessibility for grid layout?

From the specification we know that we need to ensure our document maintains the logical order of our content. How should we approach our development to make sure that we maintain accessibility for the different users and the ways that they interact with our pages?

Start with a structured and accessible document

> A grid layout should mean we do not need to change our document source in order to get the layout that we want. Therefore the starting point of your page should be a well structured and accessible source document. This is quite often going to give you a good structure for *your smallest screen devices too*. If a user is scrolling through a long document on mobile, the priorities for that user quite often map to what should be a priority in the source.

Create a responsive, and responsible grid

> With a solid document structure defined in your HTML, you can use CSS to add your layout on top. You will likely use media queries to make changes for different screen sizes and devices, including creating additional columns for larger screens. Grid can be very useful here. For example, elements deprioritized in the mobile source order can be moved into a sidebar in a desktop layout. The key here is to keep testing. A good test is to *tab around the document*. Does the order still make sense? Check that you do not end up leaping from the top to the bottom of the layout in a peculiar way. That is a sign that you need to address something about the layout.

Returning to the source

> If at any time in the design process you find yourself using grid to relocate the position of an element, consider whether you should return to your document and make a change to the logical order too. The nice thing about using CSS grid layout is that you should be able to move an item in the source to match the logical order, without needing to make big changes to your layout. The onus is on us as developers to remember to go back to our source and update it to maintain logical order.

Grid and the danger of markup flattening

Another issue to be aware of in CSS grid layout and to a lesser extent in CSS flexbox, is the temptation to *flatten* markup. As we have discovered, for an item to become a grid item it needs to be a direct child of the grid container. Therefore, where you have a `<ul>` element inside a grid container, *that* `ul` becomes a grid item – the child `<li>` elements do not.

The `subgrid` value of `grid-template-columns` and `grid-template-rows` solves this problem. It allows the grid to be inherited by grid items and passed down the tree. Alternatively, setting `display: contents` on a grid item causes the element's children to become grid items. If you set an item to `display: contents`, the box it would normally create disappears and the boxes of the child elements appear as if they have risen up a level.

Starting out with a well-structured document is a very good way to avoid accessibility problems.

See also

- Flexbox & the keyboard navigation disconnect and (Human After All): Accessibility remix video via Léonie Watson (2016)
- Grid, content re-ordering and accessibility via CSS-tricks (2019)
- `display: contents` is not a CSS reset via Adrian Roselli (2024)

Auto-placement in grid layout

CSS grid layout contains rules that control what happens when you create a grid and do not explicitly place some or all of the child items within the grid. When you don't need explicit control over content placement, this "auto-placement" is the simplest way of creating a grid for a set of items.

Default placement

If you don't give the items placement information, they automatically position themselves on the grid, placing one grid item in each grid cell.

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

Default rules for auto-placement

As you can see with the above example, if you create a grid without placing any items, the child items will lay themselves out, with one grid item in each grid cell in source-code order. The default flow is to arrange items by row. Grid will lay an item out into each cell of the first row. If you have created additional rows using the `grid-template-rows` property, then grid will continue placing items in these rows. If the grid does not have enough rows in the explicit grid to place all of the items new *implicit* rows will be created.

Sizing rows in the implicit grid

The default for automatically created rows in the implicit grid is for them to be *auto-sized*. This means that they will size themselves to contain the content added to them without causing an overflow.

The size of these rows can be controlled using the `grid-auto-rows` property. For example, to make all rows 100px tall, you can use `grid-auto-rows: 100px;`:

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  grid-auto-rows: 100px;
}
```

Sizing rows using minmax()

The `minmax()` function enables creating rows that have a minimum size, and can grow to fit content as needed when set as the `grid-auto-rows` value. By setting `grid-auto-rows: minmax(100px, auto);` we set each row to be at least 100px tall, while allowing each row to be as tall as needed:

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}
```

```
.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>
    Four <br />This cell <br />Has extra <br />content. <br />Max is auto
    <br />so the row expands.
  </div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  grid-auto-rows: minmax(100px, auto);
}
```

Sizing rows using a track listing

You can also pass in a track listing. This will repeat. The following track listing will create an initial implicit row track as 100 pixels and a second as `200px`. This will continue for as long as content is added to the implicit grid.

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  grid-auto-rows: 100px 200px;
}
```

Auto-placement by column

You can also ask grid to auto-place items by column. Using the property `grid-auto-flow` with a value of `column`. In this case grid will add items in rows that you have defined using `grid-template-rows`. When it fills up a column it will move onto the next explicit column, or create a new column track in the implicit grid. As with implicit row tracks, these column tracks will be auto sized. You can control the size of implicit column tracks with `grid-auto-columns`. This works in the same way as `grid-auto-rows`.

In this example, we have a grid with three 200px high row tracks. We declare `grid-auto-flow: column;` to auto-place by column. With `grid-auto-columns: 300px 100px;`, the columns created will alternate between being `300px` wide and `100px` wide until there are enough column tracks to hold all of the items.

```
.wrapper {
  display: grid;
  grid-template-rows: repeat(3, 200px);
  gap: 10px;
  grid-auto-flow: column;
  grid-auto-columns: 300px 100px;
}
```

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}
```

```
.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
</div>
```

The order of auto placed items

A grid can contain a mixture of item placements. Some of the items may have a specifically defined position on the grid, while others may be auto-placed. If you have a document order that reflects the order in which items sit on the grid, you may not need to write CSS rules to place absolutely everything. The specification contains a long section detailing the Grid item placement algorithm; however, for most of us, we just need to remember a few rules for our items.

Order modified document order

Grid places items that have not been given a grid position in what is described in the specification as "order modified document order". This means that if you have used the `order` property at all, the items will be placed by that order, not their DOM order. Otherwise they will stay by default in the order that they are entered in the document source.

Items with placement properties

The first thing grid will do is place any items that have a position. In the example below I have 12 grid items. Item 2 and item 5 have been placed using line based placement on the grid. You can see how those items are placed and the other items then auto-place in the spaces. The auto-placed items will place themselves before the placed items in DOM order, they don't start after the position of a placed item that comes before them.

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
  <div>Eleven</div>
  <div>Twelve</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 100px;
  gap: 10px;
}
.wrapper div:nth-child(2) {
  grid-column: 3;
  grid-row: 2 / 4;
}
.wrapper div:nth-child(5) {
  grid-column: 1 / 3;
  grid-row: 1 / 3;
}
```

Deal with items that span tracks

You can use placement properties while still taking advantage of auto-placement. In this next example I have added to the layout by setting items 1, 5, and 9 (4n+1) to span two tracks both for rows and columns. I do this with the `grid-column-end` and `grid-row-end` properties and setting the value of this to `span 2`. What this means is that the start line of the item will be set by auto-placement, and the end line will span two tracks.

You can see how this then leaves gaps in the grid, as for the auto-placed items if grid comes across an item that doesn't fit into a track, it will move to the next row until it finds a space the item can fit in.

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}
.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
  <div>Eleven</div>
  <div>Twelve</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 100px;
  gap: 10px;
}
.wrapper div:nth-child(4n + 1) {
  grid-column-end: span 2;
  grid-row-end: span 2;
  background-color: #ffa94d;
}
.wrapper div:nth-child(2) {
  grid-column: 3;
  grid-row: 2 / 4;
}
.wrapper div:nth-child(5) {
  grid-column: 1 / 3;
  grid-row: 1 / 3;
}
```

Filling in the gaps

So far, other than items we have specifically placed, grid is always progressing forward and keeping items in DOM order. This is generally what you want, if you are laying out a form for example you wouldn't want the labels and fields to become jumbled up in order to fill in some gap. However sometimes, we are laying things out that don't have a logical order and we would like to create a layout that doesn't have gaps in it.

To do this, add the property `grid-auto-flow` with a value of `dense` to the container. This is the same property you use to change the flow order to `column`, so if you were working in columns you would add both values `grid-auto-flow: column dense`.

Having done this, grid will now backfill the gaps, as it moves through the grid it leaves gaps as before, but then if it finds an item that will fit in a previous gap it will pick it up and take it out of DOM order to place it in the gap. As with any other reordering in grid this does not change the logical order. Tab order for example, will still follow the document order. We will take a look at the potential accessibility issues of grid layout in the Grid layout and accessibility guide, but you should take care when creating this disconnect between the visual order and display order.

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}
.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
```

```
  <div>Ten</div>
  <div>Eleven</div>
  <div>Twelve</div>
</div>
```

```
.wrapper div:nth-child(4n + 1) {
  grid-column-end: span 2;
  grid-row-end: span 2;
  background-color: #ffa94d;
}
.wrapper div:nth-child(2) {
  grid-column: 3;
  grid-row: 2 / 4;
}
.wrapper div:nth-child(5) {
  grid-column: 1 / 3;
  grid-row: 1 / 3;
}
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 100px;
  gap: 10px;
  grid-auto-flow: dense;
}
```

Anonymous grid items

There is a mention in the specification of anonymous grid items. These are created if you have a string of text inside your grid container, that is not wrapped in any other element. In the example below we have three grid items, assuming you had set the parent with a class of `grid` to `display: grid`. The first is an anonymous item as it has no enclosing markup, this item will always be dealt with via the auto-placement rules. The other two are grid items enclosed in a div, they might be auto-placed or you could place these with a positioning method onto your grid.

```
<div class="grid">
  I am a string and will become an anonymous item
  <div>A grid item</div>
  <div>A grid item</div>
</div>
```

Anonymous items are always auto-placed because there is no way to target them. Therefore if you have some unwrapped text for some reason in your grid, be aware that it might show up somewhere unexpected as it will be auto-placed according to the auto-placement rules.

Use cases for auto-placement

Auto-placement is useful whenever you have a collection of items. That could be items that do not have a logical order such as a gallery of photos, or product listing. In that case you might choose to use the dense packing mode to fill in any holes in your grid. In my image gallery example I have some landscape and some portrait images. I have set landscape images – with a class of `landscape` to span two column tracks. I then use `grid-auto-flow: dense` to create a densely packed grid.

Try removing the line `grid-auto-flow: dense` to see the content reflow to leave gaps in the layout.

```
<ul class="wrapper">
  <li>
    <img
      alt="A colorful hot air balloon against a clear sky"
      src="https://mdn.github.io/shared-assets/images/examples/balloon.jpg" />
  </li>
  <li class="landscape">
    <img
      alt="Three hot air balloons against a clear sky, as seen from the ground"
      src="https://mdn.github.io/shared-assets/images/examples/balloons-small.jpg" />
  </li>
  <li class="landscape">
    <img
      alt="Three hot air balloons against a clear sky, as seen from the ground"
      src="https://mdn.github.io/shared-assets/images/examples/balloons-small.jpg" />
  </li>
  <li class="landscape">
    <img
      alt="Three hot air balloons against a clear sky, as seen from the ground"
      src="https://mdn.github.io/shared-assets/images/examples/balloons-small.jpg" />
  </li>
  <li>
    <img
      alt="A colorful hot air balloon against a clear sky"
      src="https://mdn.github.io/shared-assets/images/examples/balloon.jpg" />
  </li>
  <li>
    <img
      alt="A colorful hot air balloon against a clear sky"
      src="https://mdn.github.io/shared-assets/images/examples/balloon.jpg" />
  </li>
</ul>
```

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}

.wrapper {
  list-style: none;
  margin: 1em auto;
  padding: 0;
  max-width: 800px;
}
.wrapper li {
  border: 1px solid #cccccc;
}

.wrapper li img {
```

```
  display: block;
  object-fit: cover;
  width: 100%;
  height: 100%;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, minmax(120px, 1fr));
  gap: 10px;
  grid-auto-flow: dense;
}

.wrapper li.landscape {
  grid-column-end: span 2;
}
```

Auto-placement can also help you lay out interface items which do have logical order. An example is the definition list in this next example. Definition lists are an interesting challenge to style as they are flat, there is nothing wrapping the groups of `dt` and `dd` items. In my example I am allowing auto-placement to place the items, however I have classes that start a `dt` in column 1, and `dd` in column 2, this ensure that terms go on one side and definitions on the other - no matter how many of each we have.

```
body {
  font: 1.2em sans-serif;
}
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}
```

```
<div class="wrapper">
  <dl>
    <dt>Mammals</dt>
    <dd>Cat</dd>
    <dd>Dog</dd>
    <dd>Mouse</dd>
    <dt>Birds</dt>
    <dd>Pied Wagtail</dd>
    <dd>Owl</dd>
    <dt>Fish</dt>
    <dd>Guppy</dd>
  </dl>
</div>
```

```
dl {
  display: grid;
  grid-template-columns: auto 1fr;
  max-width: 300px;
  margin: 1em;
  line-height: 1.4;
}
dt {
  grid-column: 1;
  font-weight: bold;
}
dd {
  grid-column: 2;
}
```

What can't we do with auto-placement (yet)?

There are a couple of things that often come up as questions. Currently we can't do things like target every other cell of the grid with our items. A related issue may have already come to mind if you followed the last guide about named lines on the grid. It would be to define a rule that said "auto-place items against the next line named `n`", and grid would then skip other lines. There is an issue raised about this on the CSSWG GitHub repository, and you would be welcome to add your own use cases to this.

It may be that you come up with your own use cases for auto-placement or any other part of grid layout. If you do, raise them as issues or add to an existing issue that could solve your use case. This will help to make future versions of the specification better.

Basic concepts of grid layout

CSS grid layout introduces a two-dimensional grid system to CSS. Grids can be used to lay out major page areas or small user interface elements. This guide introduces the CSS grid layout and the terminology that is part of the CSS grid layout specification. The features shown in this overview will then be explained in greater detail in the other guides in this series.

What is a grid?

A grid is a set of intersecting horizontal and vertical lines defining rows and columns. Elements can be placed onto the grid within these column and row lines. CSS grid layout has the following features:

Fixed and flexible track sizes

You can create a grid with fixed track sizes – using pixels for example. This sets the grid to the specified pixel which fits to the layout you desire. You can also create a grid using flexible sizes with percentages or with the `fr` unit designed for this purpose.

Item placement

You can place items into a precise location on the grid using line numbers, names or by targeting an area of the grid. Grid also contains an algorithm to control the placement of items not given an explicit position on the grid.

Creation of additional tracks to hold content

You can define an explicit grid with grid layout. The features defined by the grid layout module provide the flexibility to add additional rows and columns when needed. Features such as adding "as many columns that will fit into a container" are included.

Alignment control

CSS grid layout and CSS box alignment features enable us to control how the items align once placed into a grid area, and how the entire grid is aligned.

Control of overlapping content

More than one item can be placed into a grid cell or area and they can partially overlap each other. This layering may then be controlled with the `z-index` property.

Grid is a powerful layout method that, when combined with other parts of CSS such as flexbox, can help you create layouts that are responsive, flexible, and accessible. It all starts by creating a grid in your **grid container**.

Grid container

We create a *grid container* by declaring `display: grid` or `display: inline-grid` on an element. As soon as we do this, all *direct children* of that element become *grid items*.

In this example, we have a containing div with a class of `wrapper`. Nested inside are five child elements.

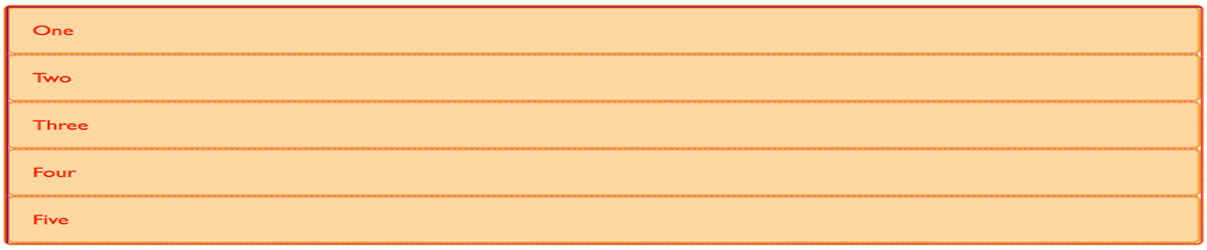
```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

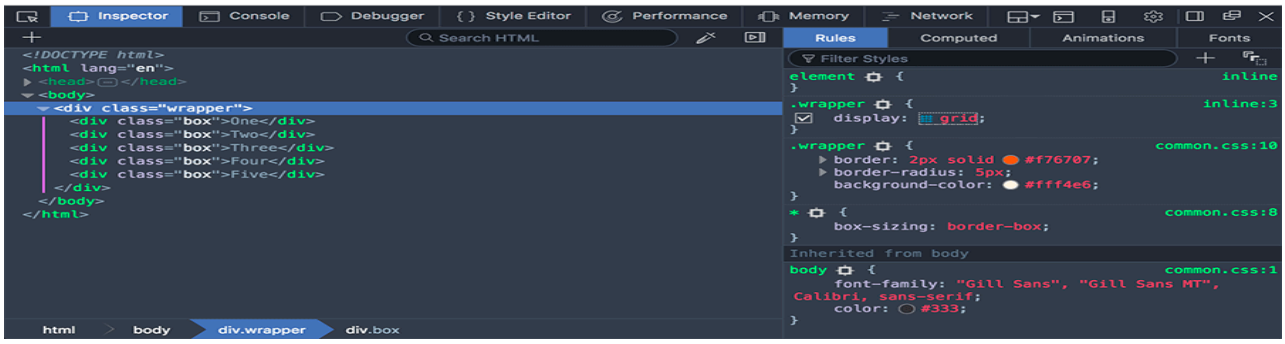
We make the `.wrapper` a grid container using `display: grid;`.

```
.wrapper {
  display: grid;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

All the direct children are now grid items. In a web browser, you won't see any difference to how these items are displayed before turning them into a grid, as grid has created a single column grid for the items. If you inspect the grid in your browsers developer tools, you may see a small icon next to the value `grid`. Click this and, in most browsers, the grid on this element will be overlaid in the browser window.



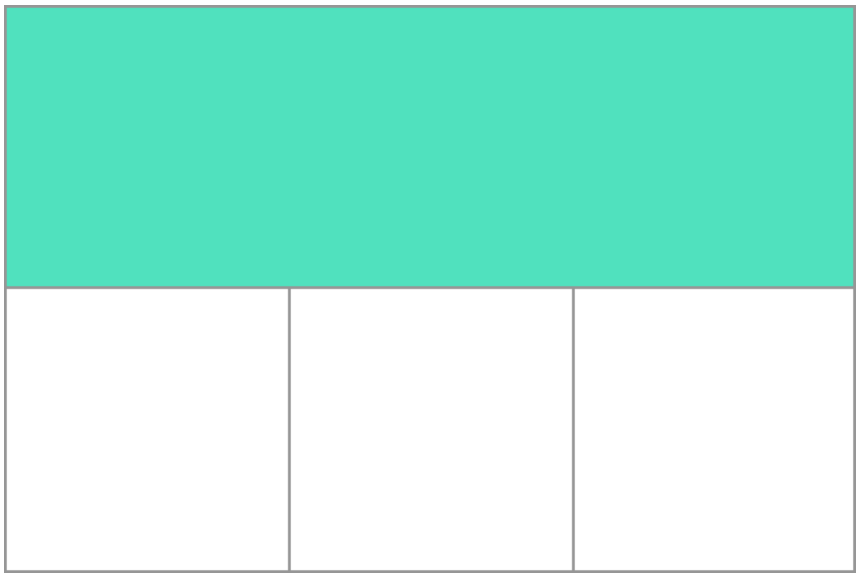


As you learn and then work with the CSS grid layout, your browser tools will give you a better idea of what is happening with your grids visually.

If we want to start making this more grid-like we need to add column tracks.

Grid tracks

We define rows and columns on our grid with the `grid-template-rows` and `grid-template-columns` properties. These define grid tracks. A *grid track* is the space between any two adjacent lines on the grid. The image below shows a highlighted track – this is the first-row track in our grid.



Grid tracks are defined in the explicit grid by using the `grid-template-columns` and `grid-template-rows` properties or the shorthand `grid` or `grid-template` properties. Tracks are also created in the implicit grid by positioning a grid item outside of the tracks created in the explicit grid.

Basic example

We can add column tracks to our earlier example by adding the `grid-template-columns` property, then defining the size of the column tracks.

We have now created a grid with three 200-pixel-wide column tracks. The child items will be laid out on this grid one in each grid cell.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 200px 200px 200px;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

The fr unit

Tracks can be defined using any length unit. Grid also introduces an additional length unit to help us create flexible grid tracks. The `fr` unit represents a fraction of the available space in the grid container. The next grid definition would create three equal width tracks that grow and shrink according to the available space.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
```

```
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Unequal sizes

In this example, we create a definition with a `2fr` track then two `1fr` tracks. The available space is split into four. Two parts are given to the first track and one part each to the next two tracks.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Mixing flexible and absolute sizes

In this final example, we mix absolute sized tracks with `fr` units. The first track is `500px`, so the fixed width is taken away from the available space. The remaining space is divided into three and assigned in proportion to the two flexible tracks.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 500px 1fr 2fr;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Track listings with repeat() notation

Large grids with many tracks can use the `repeat()` notation, to repeat all or a section of the list of grid tracks. For example the grid definition:

```
.wrapper {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Can also be written as:

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

Repeat notation can be used for a part of the list of tracks. In this example, we create an 8-column grid; the initial track is `20px`, then a repeating section of 6 `1fr` tracks, and a final `20px` track.

```
.wrapper {
  display: grid;
```

```
  grid-template-columns: 20px repeat(6, 1fr) 20px;
}
```

Repeat notation (`repeat()`) uses the track listing to create a repeating pattern of tracks. In this example, the grid will have 10 tracks; a `1fr` track is followed by a `2fr` track, with this pattern being repeated five times.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(5, 1fr 2fr);
}
```

Implicit and explicit grids

When creating our example grid, we specifically defined our column tracks with the `grid-template-columns` property, with the grid creating rows as needed to fit the content. The columns define the explicit grid while the rows are part of the implicit grid.

The *explicit grid* consists of rows and columns defined with `grid-template-columns` or `grid-template-rows`. The *implicit grid* extends the defined explicit grid when content is placed outside of that grid, such as into the rows by drawing additional grid lines.

If you place something outside of the defined grid or, due to the amount of content, more grid tracks are needed, then the grid creates rows and columns in the *implicit grid*. These implicit tracks are auto-sized by default, meaning the size of the created rows or columns is influenced by both their content and by the available free space within the grid container. The `auto` keyword allows the generated tracks to accommodate content while also sharing any remaining space.

You can also define a set size for tracks created in the implicit grid with the `grid-auto-rows` and `grid-auto-columns` properties.

In this example, we set `grid-auto-rows: 200px`, ensuring the tracks created in this implicit grid are `200px` tall.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 200px;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Track sizing and minmax

When setting up an explicit grid or defining the sizing for automatically created rows or columns we may want to give tracks a minimum size, but also ensure they expand to fit any content that is added. For example, we may want our rows to never collapse smaller than 100 pixels, but if our content stretches to 300 pixels in height, then we would like the row to stretch to that height. This is solved by the `minmax()` function.

In this example, we use `minmax()` within the `grid-auto-rows` property value. By setting `grid-auto-rows: minmax(100px, auto);`, automatically created rows will be a minimum of `100px` tall, and have a maximum of `auto`. Setting `auto` as the maximum value allows the track to grow to accommodate its content (up to its `max-content` size) while also sharing any available free space within the grid container.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
}
```

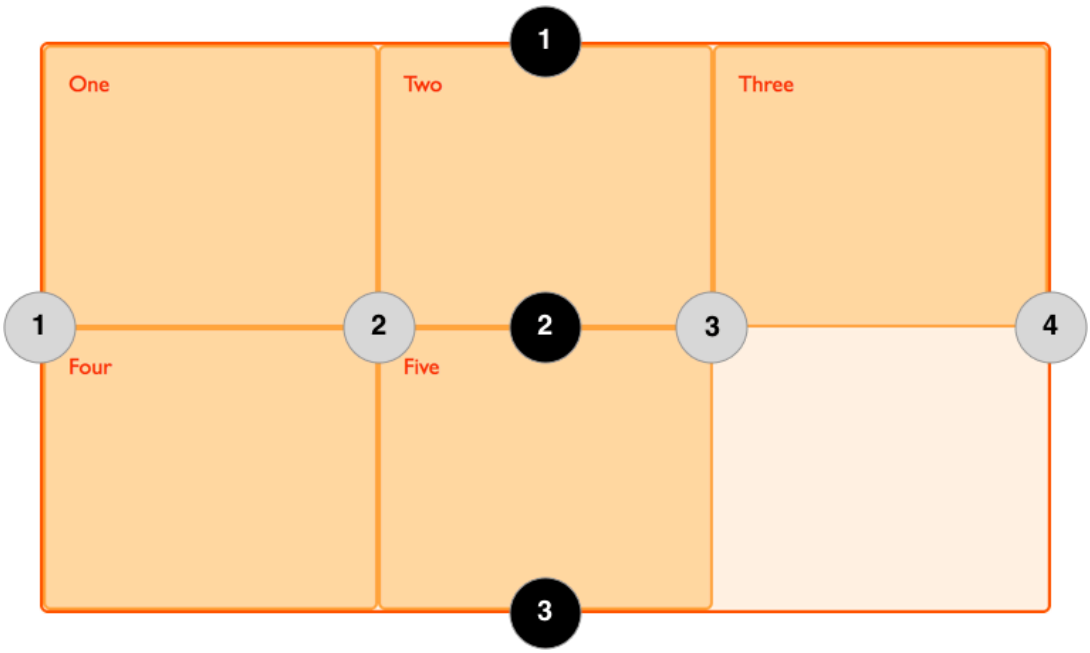
```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>
    Two
    <p>We have some more content.</p>
    <p>This makes me taller than 100 pixels.</p>
```

```
  </div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

Grid lines

It should be noted that when we define a grid we define the grid tracks, not the lines. Grid then gives us numbered lines to use when positioning items. In our three column, two row grid we have four column lines.



Lines are numbered according to the writing mode of the document. In a left-to-right language, line 1 is on the left-hand side of the grid. In a right-to-left language, it is on the right-hand side of the grid. Lines can also be named, which is discussed in the grid layout using named grid lines guide.

Positioning items against lines

The following example demonstrates basic line-based placement; when placing an item, we target the line rather than the track. We explore this in greater detail in the grid layout using line-based placement guide.

In this example, the first two items on our three column track grid are placed using the `grid-column-start`, `grid-column-end`, `grid-row-start` and `grid-row-end` properties. Working from left to right, the first item is placed against column line 1, and spans to column line 4, which in our case is the far-right line on the grid. It begins at row line 1 and ends at row line 3, therefore spanning two row tracks.

The second item starts on grid column line 1, and spans one track. This is the default, so we do not need to specify the end line. It also spans two row tracks from row line 3 to row line 5. The other items will place themselves into empty spaces on the grid.

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
  <div class="box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 3;
  grid-row-end: 5;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
```

```
  color: #d9480f;
}
```

Use the grid inspector in your developer tools to see how the items are positioned against the lines of the grid.

Line-positioning shorthands

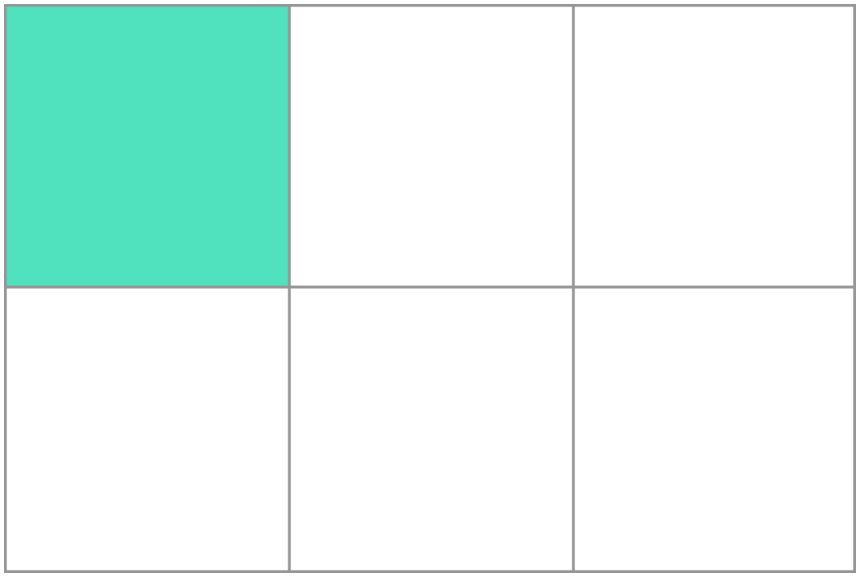
The longhand values used above can be compressed onto one line for columns with the `grid-column` shorthand, and one line for rows with the `grid-row` shorthand. The following example would give the same positioning as in the previous code, but with far less CSS. The value before the forward slash character (`/`) is the start line, the value after the end line.

You can omit the end value if the area only spans one track.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column: 1 / 4;
  grid-row: 1 / 3;
}

.box2 {
  grid-column: 1;
  grid-row: 3 / 5;
}
```

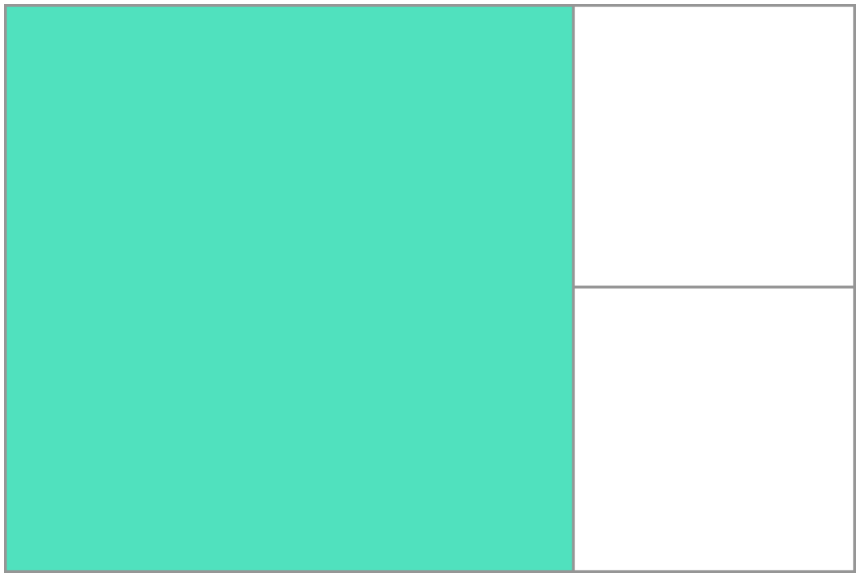
Grid cells

A *grid cell* is the smallest unit on a grid. Conceptually it is like a table cell. As we saw in our earlier examples, once a grid is defined as a parent the child items will lay themselves out in one cell each of the defined grid. In the below image, the first cell of the grid is highlighted.



Grid areas

Items can span one or more cells both by row or by column, and this creates a *grid area*. Grid areas must be rectangular – it isn't possible to create an L-shaped area for example. The highlighted grid area spans two row and two column tracks.



Gutters

Gutters or *alleys* between grid cells can be created using the `column-gap` and `row-gap` properties, or the shorthand `gap`. In the below example, we add a 10-pixel gap between columns and a `1em` gap between rows.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  column-gap: 10px;
  row-gap: 1em;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  column-gap: 10px;
  row-gap: 1em;
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Any space used by gaps will be accounted for before space is assigned to the flexible length `fr` tracks, and gaps act for sizing purposes like a regular grid track, however you cannot place anything into a gap. In terms of line-based positioning, the gap acts like a thick, transparent line.

Nesting grids

A grid item can become a grid container. In the following example, we extend the three-column grid with two positioned items seen earlier, adding sub-items to the first grid item. As these nested items are not direct children of the grid they do not participate in grid layout and so display in a normal document flow.

Nesting without subgrid

If we set `box1` to `display: grid`, we can give it a track definition and it too will become a grid. The items then lay out on this new grid.

```
<div class="wrapper">
  <div class="box box1">
    <div class="nested">a</div>
    <div class="nested">b</div>
    <div class="nested">c</div>
  </div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
```

```
  gap: 3px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

In this case the nested grid has no relationship to the parent. As you can see in the example it has not inherited the `gap` of the parent and the lines in the nested grid do not align to the lines in the parent grid.

Subgrid

In addition to regular grids, we can create *subgrid*. The `subgrid` value lets us create nested grids that use the track definition of the parent grid.

To use them, we edit the above nested grid example to change the track definition of `grid-template-columns: repeat(3, 1fr)`, to `grid-template-columns: subgrid`. The nested grid then uses the parent grid tracks to lay out items.

```
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  display: grid;
  grid-template-columns: subgrid;
}
```

Layering items with z-index

Grid items can occupy the same cell, and in this case we can use the `z-index` property to control the order in which overlapping items stack.

Overlapping without z-index

If we return to our example with items positioned by line number, we can change this to make two items overlap.

```
<div class="wrapper">
  <div class="box box1">One</div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 2;
  grid-row-end: 4;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

The item `box2` is now overlapping `box1`, it displays on top as it comes later in the source order.

Controlling the order

We can control the order in which items stack up by using the `z-index` property - just like positioned items. If we give `box2` a lower `z-index` than `box1` it will display below `box1` in the stack.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  z-index: 2;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 2;
  grid-row-end: 4;
  z-index: 1;
}
```

```
<div class="wrapper">
  <div class="box box1">One</div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Next steps

In this overview, we took a very quick look at the possibilities of grid layouts. Explore and play with the code examples, and then move on to the guide, relationship of grid layout with other layout methods, where we will really start to dig into the details of CSS grid layout.

Aligning items in CSS grid layout

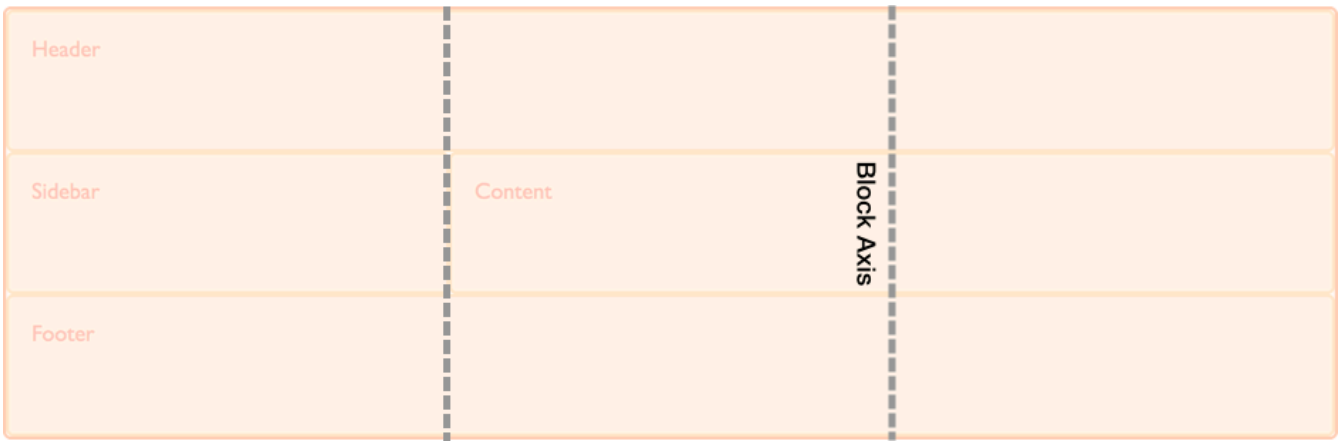
CSS grid layout implements CSS box alignment, which is the same standard flexbox uses for aligning items in its flex container. The alignment module details how alignment should work in all the layout methods.

In this guide, we look at how the box alignment properties are used to align items in grid layout.

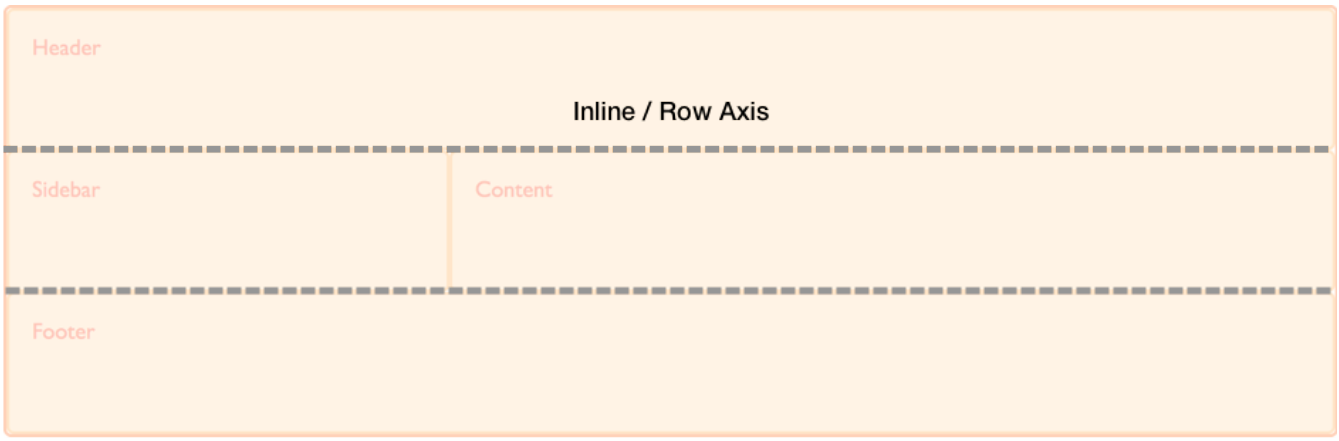
You may notice similarities to how these properties and values work in flexbox. Due to grid being two-dimensional and flexbox one-dimensional, there are some small differences that you should watch out for. For this reason, we will start by looking at the two axes that we deal with when aligning things in a grid.

The two axes of a grid layout

When working with grid layout you have two axes available to align things against – the *block axis* and the *inline axis*. The block axis is the axis upon which blocks are laid out in block layout. If you have two paragraphs on your page they display one below the other, so it is this direction we describe as the block axis.



The inline axis runs across the block axis, it is the direction in which text in regular inline flow runs.



We are able to align the content inside grid areas, and the grid tracks themselves on these two axes.

Aligning items on the block axis

The `align-self` and `align-items` properties control alignment on the block axis. When we use these properties, we are changing the alignment of the item within the grid area you have placed it.

Using align-items

In the following example, we have four grid areas within our grid. We can use the `align-items` property on the grid container, to align the items using `normal`, `stretch`, or `<self-position>` or `<baseline-position>` values:

- `normal`
- `stretch`
- `start`
- `end`
- `center`
- `baseline`
- `first baseline`
- `last baseline`
- `auto` (`align-self` only)

The default value is `normal`, which resolves to `stretch` for grid containers.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  gap: 10px;
  grid-auto-rows: 100px;
  grid-template-areas:
    "a a a a b b b b"
    "a a a a b b b b"
    "c c c c d d d d"
    "c c c c d d d d";
  align-items: start;
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
}
.item3 {
  grid-area: c;
}
.item4 {
  grid-area: d;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

Keep in mind that once you set `align-items: start`, the height of each child `<div>` will be determined by the contents of the `<div>`. This is in contrast to omitting `align-items` completely, in which case the height of each `<div>` stretches to fill its grid area.

The `align-items` property sets the `align-self` property for all of the child grid items. This means that you can set the property individually, by using `align-self` directly on a grid item.

Using align-self

In this next example, we are using the `align-self` property, to demonstrate the different alignment values. The first area is showing the default behavior of `align-self`, which in this case resolves to `stretch`. The second item, has an `align-self` value of `start`, the third `end` and the fourth `center`.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  gap: 10px;
  grid-auto-rows: 100px;
  grid-template-areas:
    "a a a a b b b b"
    "a a a a b b b b"
    "c c c c d d d d"
    "c c c c d d d d";
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
  align-self: start;
}
.item3 {
  grid-area: c;
  align-self: end;
}
.item4 {
  grid-area: d;
  align-self: center;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

Items with an intrinsic aspect ratio

The default behavior for `align-self` is to inherit from the grid container's `align-items` property, for which the `normal` default is to stretch, except for items which have an intrinsic aspect ratio, in this case they behave as `start`. The reason for this, is that if items with an aspect ratio are made to stretch, they would be distorted.

Justifying items on the inline axis

Whereas `align-items` and `align-self` align items on the block axis, `justify-items` and `justify-self` align items on the inline axis. The values you can choose from are similar to the `normal`, `stretch`, `<self-position>` and `<baseline-position>` values of the `align-self` property, along with `left` and `right`. Values include:

- `normal`
- `start`
- `end`
- `left`
- `right`
- `center`
- `stretch`
- `baseline`
- `first baseline`
- `last baseline`
- `auto` (`justify-self` only)

You can see the same example as used for `align-items`, below. This time, we are applying the `justify-self` property.

Once again, the default is `stretch` other than for items with an intrinsic aspect ratio. This means that grid items will cover their grid area by default, unless you change the alignment. In this example, the first item demonstrates the default `stretch` alignment value:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
```

```
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  gap: 10px;
  grid-auto-rows: 100px;
  grid-template-areas:
    "a a a a b b b b"
    "a a a a b b b b"
    "c c c c d d d d"
    "c c c c d d d d";
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
  justify-self: start;
}
.item3 {
  grid-area: c;
  justify-self: end;
}
.item4 {
  grid-area: d;
  justify-self: center;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

As with `align-self` and `align-items`, you can apply `justify-items` to the grid container to set a `justify-self` value for all the grid items within the container.

> **Note:** The `justify-self` and `justify-items` properties are not implemented in flexbox. This is due to the one-dimensional nature of flexbox, and that there may be multiple items along the axis, making it impossible to justify a single item. To align items along the main, inline axis in flexbox you use the `justify-content` property.

Shorthand properties

The `place-items` property is shorthand for `align-items` and `justify-items`.

The `place-self` property is shorthand for `align-self` and `justify-self`.

Center an item in the area

By combining the align and justify properties we can easily center an item inside a grid area.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
  grid-auto-rows: 200px;
  grid-template-areas:
    ". a a ."
    ". a a .";
}
.item1 {
  grid-area: a;
  align-self: center;
  justify-self: center;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
</div>
```

Aligning the grid tracks on the block axis

If you have a situation where your grid tracks use an area that is smaller than the grid container you can align the grid tracks themselves inside that container. The `align-content` aligns the tracks on the block axis and `justify-content` aligns on the inline axis. As with the `*-items` and `*-item` properties, the `place-content` property is shorthand for `align-content` and `justify-content`.

The values for `align-content`, `justify-content` and `place-content` all include the `<content-distribution>` and `<content-position>` values. The `align-content` property also accepts `<baseline-position>` values and, like the other `justify-*` properties, `justify-content` also accepts `left` and `right`.

Valid keywords for `place-content` include:

- `normal`
- `start`
- `end`
- `center`
- `stretch`
- `space-around`
- `space-between`
- `space-evenly`
- `baseline`
- `first baseline`
- `last baseline`
- `left`
- `right`

The `align-content` property is applied to the grid container as it works on the entire grid.

Default alignment

In this example, the 500px by 500px grid container has three row and three column 100px tracks with a 10px gutter. This means there is space inside the grid container in both the block and inline directions.

By default, our grid tracks are in the top left corner of the grid, aligned against the start grid lines, as the default behavior in grid layout is `start`:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(3, 100px);
  height: 500px;
  width: 500px;
  gap: 10px;
  grid-template-areas:
    "a a b"
    "a a b"
    "c d d";
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
}
.item3 {
  grid-area: c;
}
.item4 {
  grid-area: d;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

Setting align-content: end

Using the same CSS and HTML, in this example we add `align-content` with a value of `end` to the container, so the tracks all move to the end line of the grid container in the block dimension:

```
.wrapper {
  align-content: end;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
```

```
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(3, 100px);
  height: 500px;
  width: 500px;
  gap: 10px;
  grid-template-areas:
    "a a b"
    "a a b"
    "c d d";
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
}
.item3 {
  grid-area: c;
}
.item4 {
  grid-area: d;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

Setting align-content: space-between

We can also apply the `<content-distribution>` space distribution values of `space-between`, `space-around`, `space-evenly`, and `stretch`. In this example, we set `align-content`, which aligns the tracks on the block axis, to `space-between`, which spaces out the tracks:

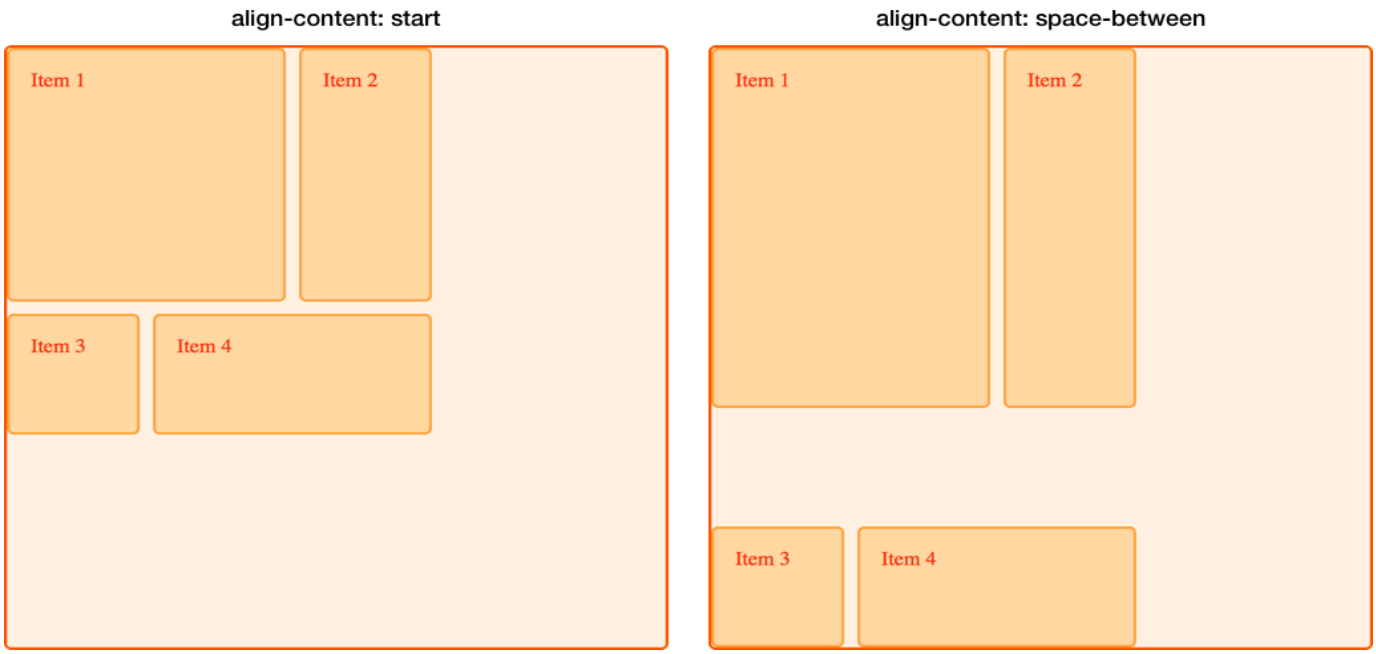
```
.wrapper {
  align-content: space-between;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(3, 100px);
  height: 500px;
  width: 500px;
  gap: 10px;
  grid-template-areas:
    "a a b"
    "a a b"
    "c d d";
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
}
.item3 {
  grid-area: c;
}
.item4 {
  grid-area: d;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

If an item spans more than one grid track, using a space distribution value will likely cause items on your grid to become larger because the space that's added between the tracks is added to the spanning item. Therefore, if you use these values, make sure the content of the tracks can cope with the extra space or that you have used alignment properties on the items, so they move to the start or end rather than stretch.

In the below image, we placed the grid with two different `align-content` values to compare `start` and `space-between`. You can see how the first two items, which span two row tracks, have taken on extra height in the `space-between` example as they gain the space that exists because of the free space that was distributed *between* the three rows:



Justifying the grid tracks on the inline axis

We can use `justify-content` to perform the same type of alignment on the inline axis that we used `align-content` for in the block axis.

Using the same example, we set `justify-content` to `space-around`. This once again causes tracks which span more than one column track to gain extra space:

```
.wrapper {
  align-content: space-between;
  justify-content: space-around;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(3, 100px);
  height: 500px;
  width: 500px;
  gap: 10px;
  grid-template-areas:
    "a a b"
    "a a b"
    "c d d";
}
.item1 {
  grid-area: a;
}
.item2 {
  grid-area: b;
}
.item3 {
  grid-area: c;
}
.item4 {
  grid-area: d;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

Alignment and auto margins

Another way to align items inside their area is to use auto margins. If you have ever centered a layout in the viewport, or any block level element within its parent, you may have done so by setting the right and left margin of element you wanted to center to `auto`. The auto margin absorbs all of the available space. Setting the margin to `auto` on both sides pushes the block-level element into the middle as both margins attempt to take all of the space.

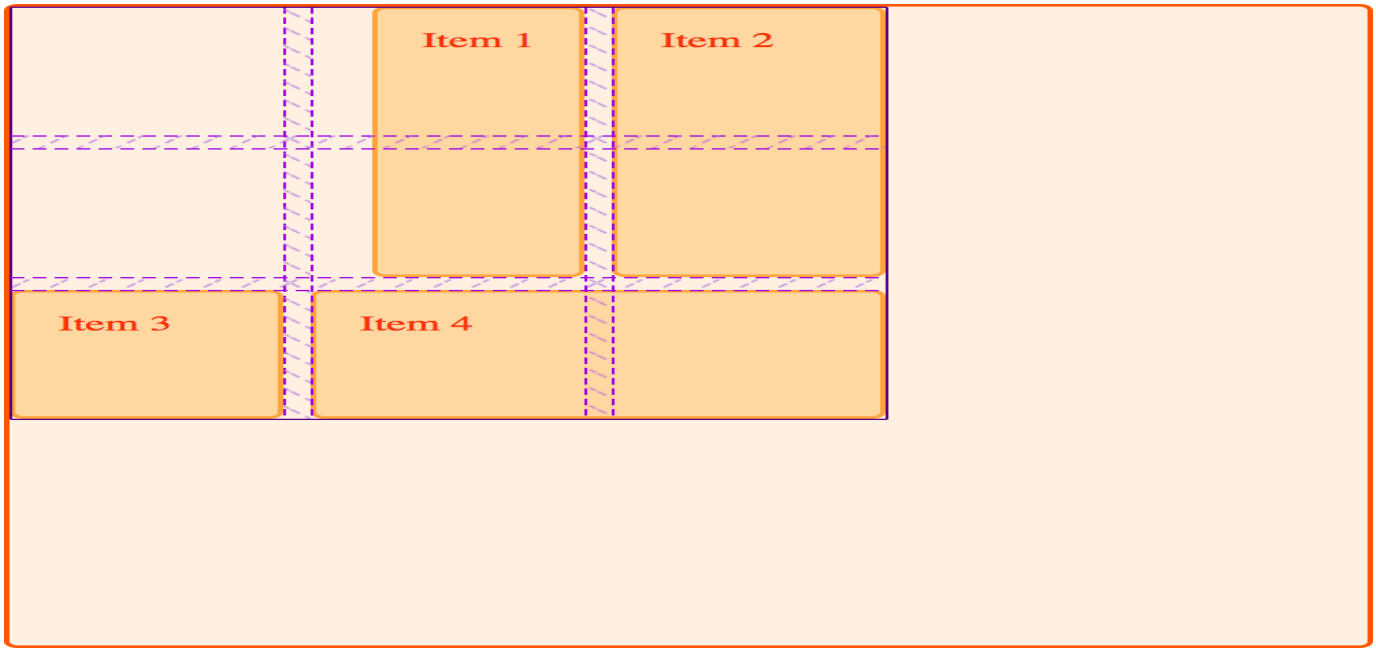
In this next example, item 1 has its `margin-left` property set to `auto`. This pushes the content over to the right side of the area, as the auto margin takes up the available space that remained after space needed for the content has been assigned:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(3, 100px);
  height: 500px;
  width: 500px;
  gap: 10px;
  grid-template-areas:
    "a a b"
    "a a b"
    "c d d";
}
.item1 {
  grid-area: a;
  margin-left: auto;
}
.item2 {
  grid-area: b;
}
.item3 {
  grid-area: c;
}
.item4 {
  grid-area: d;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
</div>
```

Use the grid inspector in your browser developer tools to see how the item is aligned:



Alignment and writing modes

All these examples were in English, a left-to-right language. This means that our start lines are top and left of our grid when thinking in physical directions.

CSS grid layout and CSS box alignment work with writing modes in CSS. When displaying a right to left language, such as Arabic, the start of the grid is the top right, so the default of `justify-content: start` would be for grid tracks to start on the right-hand side of the grid.

Setting physical properties, such as setting auto margins using `margin-right` or `margin-left`, or absolutely positioning items using the `top`, `right`, `bottom`, and `left` offsets, does not honor writing modes. In the grids, logical values, and writing modes guide, we will look further into this interaction between CSS grid layout, box alignment and writing modes. This will be important to understand, if you develop sites that are then displayed in multiple languages, or if you want to mix languages or writing modes in a design.

See also

- Basic concepts of grid layout
- Relationship of grid layout with other layout methods
- Grid layout using line-based placement
- Grid template areas
- Grid layout using named grid lines
- Auto-placement in grid layout
- Box alignment in CSS grid layout

Realizing common layouts using grids

To round off this set of CSS grid layout guides, we're going to walk through a few different layouts, which demonstrate some of the techniques you can use when designing with grid layout. We will look at an example using `grid-template-areas`, a 12-column flexible grid system, and also a product listing using auto-placement. As you can see from this set of examples, there is often more than one way to get the results you want with CSS grid layout. Choose the method you find most helpful for the problems that you are solving and the designs that you need to implement.

A responsive layout with 1 to 3 fluid columns using <code>grid-template-areas</code>

Many websites are a variation of this type of layout, with content, sidebars, a header and a footer. In a responsive design, you may want to display the layout as a single column, adding a sidebar at a certain breakpoint and then bring in a three-column layout for wider screens.

We're going to create this layout using the *named template areas* that we learned about in the Grid template areas guide.

The markup is a container with elements inside for a header, footer, main content, navigation, sidebar, and a block to place advertising.

```
* {
  box-sizing: border-box;
}
.wrapper {
  max-width: 1024px;
  margin: 0 auto;
  font:
    1.2em "Helvetica",
    "Arial",
    sans-serif;
}

.wrapper > * {
  border: 2px solid #f08c00;
  background-color: #ffec99;
  border-radius: 5px;
  padding: 10px;
}

nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
}
```

```
<div class="wrapper">
  <header class="main-head">The header</header>
  <nav class="main-nav">
    <ul>
      <li><a href="">Nav 1</a></li>
      <li><a href="">Nav 2</a></li>
      <li><a href="">Nav 3</a></li>
    </ul>
  </nav>
  <article class="content">
    <h1>Main article area</h1>
```

```
    <p>
      In this layout, we display the areas in source order for any screen less
      that 500 pixels wide. We go to a two column layout, and then to a three
      column layout by redefining the grid, and the placement of items on the
      grid.
    </p>
  </article>
  <aside class="side">Sidebar</aside>
  <div class="ad">Advertising</div>
  <footer class="main-footer">The footer</footer>
</div>
```

As we are using `grid-template-areas` to create the layout, we need to name the areas outside of any media queries. We name areas using the `grid-area` property.

```
.main-head {
  grid-area: header;
}
.content {
  grid-area: content;
}
.main-nav {
  grid-area: nav;
}
.side {
  grid-area: sidebar;
}
.ad {
  grid-area: ad;
}
.main-footer {
  grid-area: footer;
}
```

This does not create a layout. Rather, the items now have names we can use to do so. Staying outside of any media queries we're now going to set up the layout for the mobile width. Here we're keeping everything in source order to avoid any disconnect between the source and display as described in the Grid layout and accessibility guide. We've not explicitly defined any column or row tracks; this layout dictates a single column and creates rows as needed for each item in the implicit grid.

```
.wrapper {
  display: grid;
  gap: 20px;
  grid-template-areas:
    "header"
    "nav"
    "content"
    "sidebar"
    "ad"
    "footer";
}
```

With our mobile layout in place, we can now proceed to add a `@media` query to adapt this layout for bigger screens with enough real estate to display two columns.

```
@media (width >= 500px) {
  .wrapper {
    grid-template-columns: 1fr 3fr;
    grid-template-areas:
      "header  header"
      "nav     nav"
      "sidebar content"
      "ad      footer";
  }
  nav ul {
    display: flex;
    justify-content: space-between;
  }
}
```

You can see the layout taking shape in the value of `grid-template-areas`. The `header` spans over two column tracks, as does the `nav`. In the third row track, we place the `sidebar` alongside the `content`. We place the `ad` content in the fourth row track so it appears under the sidebar. The `footer` is next to it under the content. We use CSS flexible box layout on the navigation to evenly space the navigation items in a row.

We can now add a final breakpoint for wider screens able to display a three-column layout.

```
@media (width >= 700px) {
  .wrapper {
    grid-template-columns: 1fr 4fr 1fr;
    grid-template-areas:
      "header header  header"
      "nav    content sidebar"
      "nav    content ad"
      "footer footer  footer";
  }
  nav ul {
    flex-direction: column;
  }
}
```

The three-column layout has two `1fr` unit side columns and a middle column that has `4fr` as the track size. This means the available space in the container is split into six parts and assigned in proportion to our three tracks – one part each to the side columns and four parts to the center.

In this layout, we're displaying the `nav` in the left column, alongside the `content`. In the right column we have the `sidebar` and underneath it the advertisements (`ad`). The `footer` now spans across the entire bottom of the layout. Again, we use flexbox to display the navigation, but this time we display it as a column instead of a row.

This basic example demonstrates how to rearrange a grid layout across different breakpoints. In particular, we're changing the location of the `ad` block as appropriate in our different column setups. This named areas method can be very helpful, especially at the prototyping stage. You may find it easier to use names rather than numbers when playing with the location of elements on the grid.

A flexible 12-column layout

CSS frameworks and grid systems commonly use 12- or 16-column flexible grids. We can create this type of system using CSS grid layout. As an example, let's create a 12-column flexible grid with 12 `1fr`-unit column tracks, each with a start line named `col-start`. This means that we will have twelve grid lines named `col-start`.

```
.wrapper {
  max-width: 1024px;
  margin: 0 auto;
  font:
    1.2em "Helvetica",
    "Arial",
    sans-serif;
}
.wrapper > * {
  border: 2px solid #f08c00;
  background-color: #ffec99;
  border-radius: 5px;
  padding: 10px;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(12, [col-start] 1fr);
  gap: 20px;
}
```

To demonstrate how this grid system works, we have four child elements inside a wrapper.

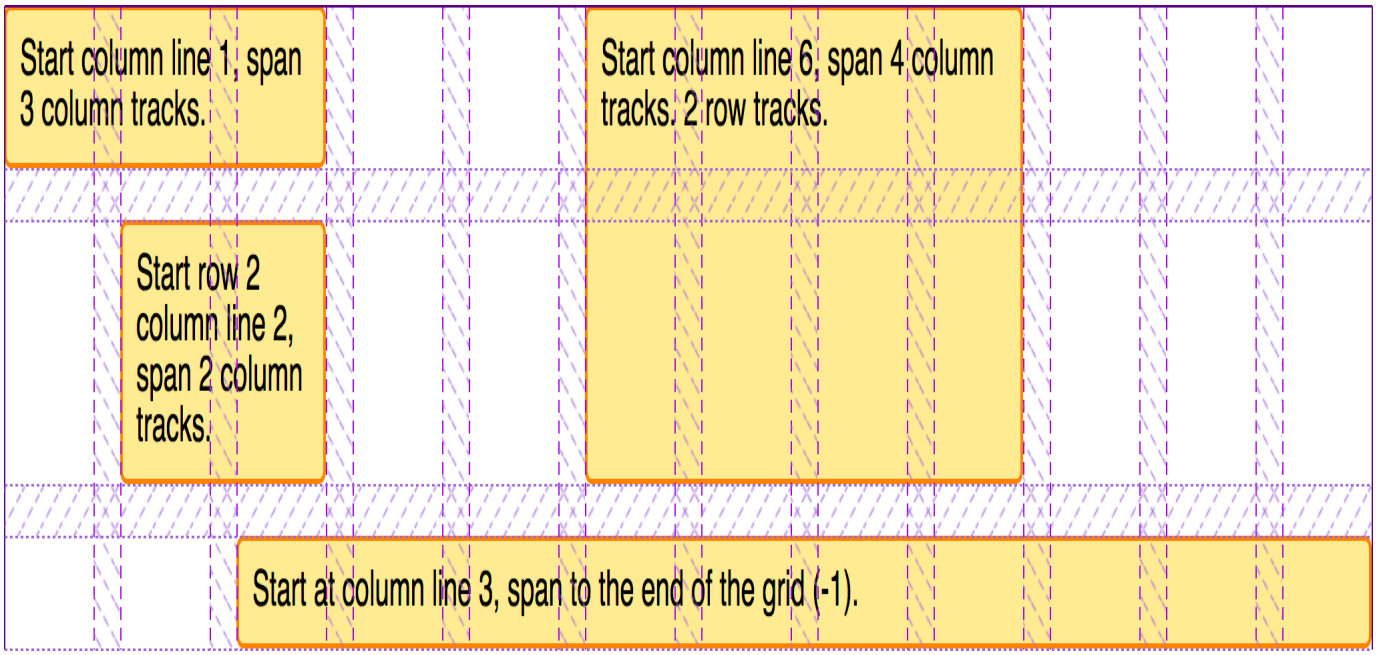
```
<div class="wrapper">
  <div class="item1">Start column line 1, span 3 column tracks.</div>
  <div class="item2">
    Start column line 6, span 4 column tracks. 2 row tracks.
  </div>
  <div class="item3">Start row 2 column line 2, span 2 column tracks.</div>
  <div class="item4">
    Start at column line 3, span to the end of the grid (-1).
  </div>
</div>
```

We can then place these on the grid using the named lines, and also the `span` keyword.

```
.item1 {
  grid-column: col-start / span 3;
}
.item2 {
  grid-column: col-start 6 / span 4;
  grid-row: 1 / 3;
}
.item3 {
  grid-column: col-start 2 / span 2;
  grid-row: 2;
}
.item4 {
  grid-column: col-start 3 / -1;
  grid-row: 3;
}
```

As described in the using named grid lines guide, we are using the named lines to place our items. As we have 12 lines all with the same name, we use the name and the index of the line. If you prefer, you can use the line index itself and avoid using named lines.

Rather than setting the end line number, we define how many tracks this element should span using the `span` keyword. When working with a multiple-column layout system, this method may be more intuitive for those who think of blocks in terms of the number of tracks of the grid they span, then adjusting for different breakpoints. To see how the blocks align themselves to the tracks, use the grid inspector in your browser developer tools; it likely clearly demonstrates how the items are placed.



We don't need to add any markup to create a row. CSS framework grid systems often did this to stop elements popping up into the row above for browsers that don't support CSS grid layout. However, this point is now moot — all modern browsers have supported CSS grid layout for a long time. CSS grids allow us to place items into rows, with no danger of them rising up into the row above if it is left empty. Due to this *strict* column and row placement, we can also easily leave white space in our layout. We also don't need special classes to indent items into the grid. All we need to do is specify the start and end line for the item.

Building a layout using the 12-column system

To see how this layout method works in practice, we can create the same layout we created with `grid-template-areas`, this time using the 12-column grid system. Let's start with the same markup as used for the grid template areas example.

```
* {
  box-sizing: border-box;
}
.wrapper {
  max-width: 1024px;
  margin: 0 auto;
  font:
    1.2em "Helvetica",
    "Arial",
    sans-serif;
}

.wrapper > * {
  border: 2px solid #f08c00;
  background-color: #ffec99;
  border-radius: 5px;
  padding: 10px;
}

nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
}
```

```
<div class="wrapper">
  <header class="main-head">The header</header>
  <nav class="main-nav">
    <ul>
      <li><a href="">Nav 1</a></li>
      <li><a href="">Nav 2</a></li>
      <li><a href="">Nav 3</a></li>
    </ul>
  </nav>
  <article class="content">
    <h1>Main article area</h1>
    <p>
      In this layout, we display the areas in source order for any screen less
      that 500 pixels wide. We go to a two column layout, and then to a three
      column layout by redefining the grid, and the placement of items on the
      grid.
    </p>
  </article>
  <aside class="side">Sidebar</aside>
  <div class="ad">Advertising</div>
  <footer class="main-footer">The footer</footer>
</div>
```

We set up our grid as we did for the 12-column layout example above.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(12, [col-start] 1fr);
  gap: 20px;
}
```

We are again going to make this a responsive layout, this time using named lines. Every breakpoint will use a 12-column grid. However, the number of tracks items will span will change depending on the size of the screen.

We start mobile first. For the narrowest screens, we want the items to remain in source order and all span across the entire grid.

```
.wrapper > * {
  grid-column: col-start / span 12;
}
```

At the next breakpoint, we want a two-column layout. Our header and navigation still span the full grid, so we do not need to specify any positioning for them. The sidebar is starting on the first column line named `col-start`, spanning 3 lines. It goes after row line 3, as the header and navigation are in the first two row tracks.

The `ad` panel is below the sidebar, starting at grid row line 4. Then we have the content and footer starting at col-start 4 and spanning nine tracks, taking both to the end of the grid.

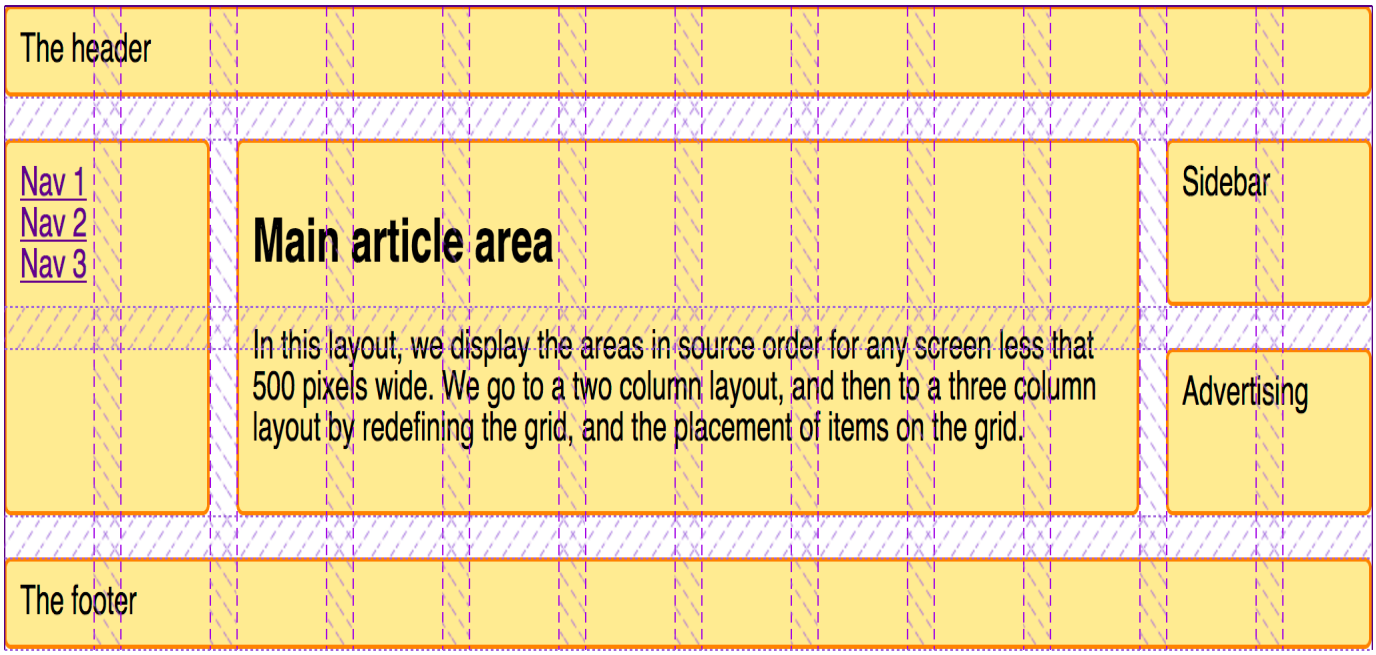
```
@media (width >= 500px) {
  .side {
    grid-column: col-start / span 3;
    grid-row: 3;
  }
  .ad {
    grid-column: col-start / span 3;
    grid-row: 4;
  }
  .content,
  .main-footer {
    grid-column: col-start 4 / span 9;
  }
  nav ul {
    display: flex;
    justify-content: space-between;
  }
}
```

Finally, for screens larger than our largest breakpoint, we define a three-column version of this layout. The header continues to span right across the grid, but now the navigation moves down to become the first sidebar, with the content and then the sidebar next to it. The footer now also spans across the full layout.

```
@media (width >= 700px) {
  .main-nav {
```

```
    grid-column: col-start / span 2;
    grid-row: 2 / 4;
  }
  .content {
    grid-column: col-start 3 / span 8;
    grid-row: 2 / 4;
  }
  .side {
    grid-column: col-start 11 / span 2;
    grid-row: 2;
  }
  .ad {
    grid-column: col-start 11 / span 2;
    grid-row: 3;
  }
  .main-footer {
    grid-column: col-start / span 12;
  }
  nav ul {
    flex-direction: column;
  }
}
```

Once again, check the grid inspector in your browser developer tools to see how the layout has taken shape.



Something to note as we created this layout is that we didn't need to explicitly position every element on the grid at each breakpoint. We inherited the placement set up for earlier breakpoints – an advantage of working "mobile first". We also took advantage of grid auto-placement. By keeping elements in a logical order, auto-placement does quite a lot of work for us in placing items onto the grid.

A product listing with auto-placement

In this last example in this guide, we create a layout that entirely relies on auto-placement.

Many layouts are essentially sets of "cards" – product listings, image galleries, and so on. A grid enables creating these listings in a way that is responsive without needing to add media queries. In this example, we combine CSS grid and flexbox layouts to make a basic product listing layout.

The markup for the listing is an unordered list of items. Each item contains a heading, some text of varying height, and a call to action link.

```
<ul class="listing">
  <li>
    <h2>Item One</h2>
    <div class="body">
      <p>The content of this listing item goes here.</p>
    </div>
    <div class="cta">
      <a href="">Call to action!</a>
    </div>
  </li>
  <li>
    <h2>Item Two</h2>
    <div class="body">
      <p>The content of this listing item goes here.</p>
    </div>
    <div class="cta">
      <a href="">Call to action!</a>
    </div>
  </li>
  <li class="wide">
    <h2>Item Three</h2>
    <div class="body">
      <p>The content of this listing item goes here.</p>
      <p>This one has more text than the other items.</p>
      <p>Quite a lot more</p>
      <p>Perhaps we could do something different with it?</p>
    </div>
    <div class="cta">
      <a href="">Call to action!</a>
    </div>
  </li>
```

```
  <li>
    <h2>Item Four</h2>
    <div class="body">
      <p>The content of this listing item goes here.</p>
    </div>
    <div class="cta">
      <a href="">Call to action!</a>
    </div>
  </li>
  <li>
    <h2>Item Five</h2>
    <div class="body">
      <p>The content of this listing item goes here.</p>
    </div>
    <div class="cta">
      <a href="">Call to action!</a>
    </div>
  </li>
</ul>
```

```
* {
  box-sizing: border-box;
}
img {
  max-width: 100%;
  display: block;
}
body {
  font:
    1.2em "Helvetica",
    "Arial",
    sans-serif;
}
a:link,
a:visited {
  text-decoration: none;
  color: #f08c00;
}

h2 {
  background-color: #f08c00;
  color: white;
  text-align: center;
  margin: 0;
  padding: 20px;
}
```

We create a grid with a flexible number of flexible columns. We want them to be at least 200 pixels wide and share any available remaining space equally – so we always get equal-width column tracks. We achieve this with the `minmax()` function in our `repeat()` notation for track sizing.

```
.listing {
  list-style: none;
  margin: 2em;
  display: grid;
  gap: 20px;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}
```

When we add this CSS, the items will be laid out as a grid. If we make the window smaller or wider, the number of column tracks changes – without media queries adding breakpoints and without needing to redefine the grid.

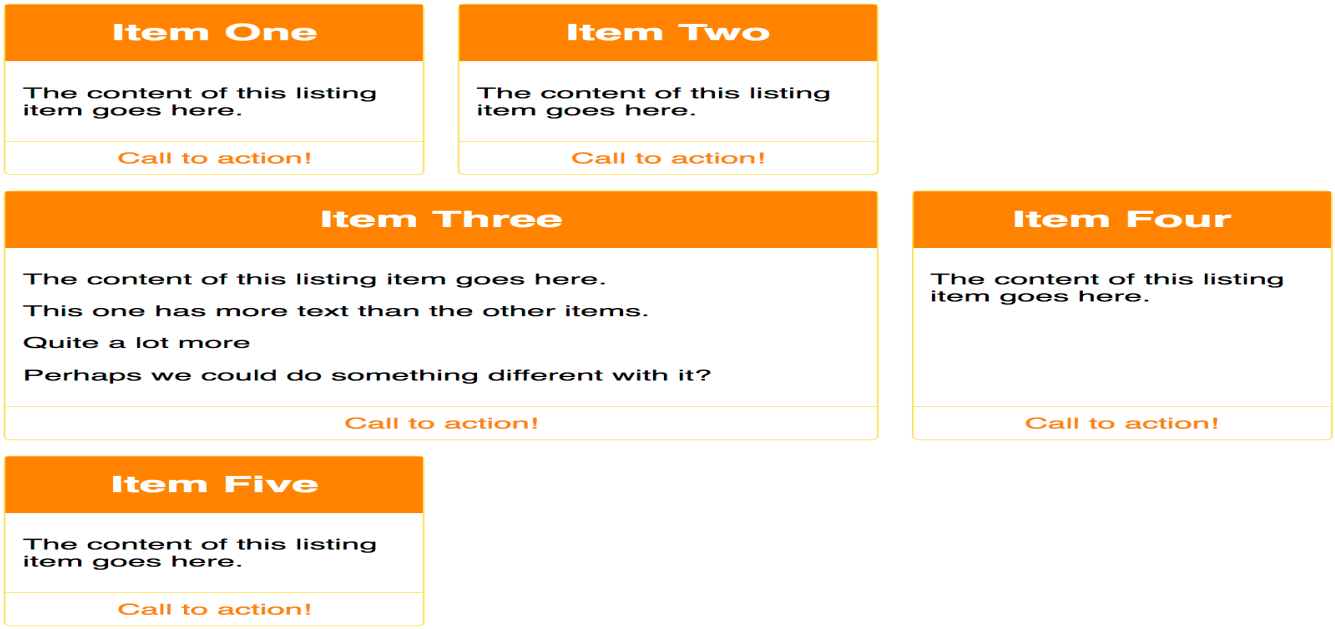
We can tidy up the internals of the boxes using a touch of flexbox. We set the list item to `display: flex` and the `flex-direction` to `column`. We can then use an auto margin on the `.cta` to push this bar down to the bottom of the box.

```
.listing li {
  border: 1px solid #ffe066;
  border-radius: 5px;
  display: flex;
  flex-direction: column;
}
.listing .cta {
  margin-block-start: auto;
  border-block-start: 1px solid #ffe066;
  padding: 10px;
  text-align: center;
}
.listing .body {
  padding: 10px;
}
```

This is one of the key reasons to use flexbox rather than CSS grid layout. If you're aligning or distributing content in a single dimension, that's a flexbox use case.

Preventing gaps with the dense keyword

This is all looking fairly complete now. However, we sometimes have cards that contain far more content than the others. It might be nice to make those span two tracks, then they won't be so tall. We add a `wide` class on the larger item, and add a rule giving it a `grid-column-end` with a value of `span 2`. When this item is encountered, it will be assigned to two tracks. This means that, at some breakpoints, we'll get a gap in the grid – where there isn't enough space to lay out a two-track item.



We can make the grid backfill those gaps by setting `grid-auto-flow: dense` on the grid container. Take care when doing this as it can cause items to be taken out of their logical source order. You should only do this if your items do not have a set order. Additionally, be aware of the accessibility and re-ordering issues resulting from the tab order following the source and not your reordered display.

```
<ul class="listing">
  <li>
    <h2>Item One</h2>
    <div class="body"><p>The content of this listing item goes here.</p></div>
    <div class="cta"><a href="">Call to action!</a></div>
  </li>
  <li>
    <h2>Item Two</h2>
    <div class="body"><p>The content of this listing item goes here.</p></div>
    <div class="cta"><a href="">Call to action!</a></div>
  </li>
  <li class="wide">
    <h2>Item Three</h2>
    <div class="body">
      <p>The content of this listing item goes here.</p>
      <p>This one has more text than the other items.</p>
      <p>Quite a lot more</p>
      <p>Perhaps we could do something different with it?</p>
    </div>
    <div class="cta"><a href="">Call to action!</a></div>
  </li>
  <li>
    <h2>Item Four</h2>
    <div class="body"><p>The content of this listing item goes here.</p></div>
    <div class="cta"><a href="">Call to action!</a></div>
  </li>
  <li>
    <h2>Item Five</h2>
    <div class="body"><p>The content of this listing item goes here.</p></div>
    <div class="cta"><a href="">Call to action!</a></div>
  </li>
</ul>
```

```
* {
  box-sizing: border-box;
}
img {
  max-width: 100%;
  display: block;
}
body {
  font:
    1.2em "Helvetica",
    "Arial",
    sans-serif;
}
a:link,
a:visited {
  text-decoration: none;
  color: #f08c00;
}

h2 {
  background-color: #f08c00;
  color: white;
  text-align: center;
  margin: 0;
  padding: 20px;
}

.listing li {
  border: 1px solid #ffe066;
  border-radius: 5px;
  display: flex;
  flex-direction: column;
```

```
}
.listing .cta {
  margin-block-start: auto;
  border-block-start: 1px solid #ffe066;
  padding: 10px;
  text-align: center;
}
.listing .body {
  padding: 10px;
}
```

```
.listing {
  list-style: none;
  margin: 2em;
  display: grid;
  gap: 20px;
  grid-auto-flow: dense;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}
.listing .wide {
  grid-column-end: span 2;
}
```

Using auto-placement with some rules applied to certain items is very useful and can help with content you can't control, such as CMS output, where you have repeated items and can use structural pseudo-classes to target them.

Further exploration

CSS grid layout provides so many possibilities. The best way to learn to use grid layout is to continue to build examples like the ones we have covered here. Pick a layout from a responsive site you like and see if you can build it using grid. You can even take inspiration from magazines or other non-web sources.

- CSS grid layout
- CSS Layout: Grids
- CSS grid layout guide on CSS-Tricks (2021)
- Grid by example
- CSS grid website layout examples on quackit.com

Grid lanes layout

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

Level 3 of the CSS grid layout specification defines **grid lanes layout**, which is accessed via the `display` values `grid-lanes` and `inline-grid-lanes`. This guide explains how grid lanes layout works and how to use it.

Grid lanes layout is a layout method where one axis uses a strict grid layout and the other uses a stacking algorithm. Each item is placed into the lane with the most available space, producing a tightly packed layout without strict tracks on the stacking axis.

Grid lanes layout is often referred to as *masonry layout*, because it resembles the way bricks are laid in a wall, filling gaps as efficiently as possible.

Creating a grid lanes layout

To create the most common grid lanes layout, where the columns are laid out in a grid and the rows are packed using the grid lanes layout algorithm, use **`display: grid-lanes`** along with `grid-template-columns`.

The child elements of this container will lay out item by item along the stacking axis according to the grid lanes layout algorithm: each row of items loads into the column with the most room, causing a tightly packed layout without strict row tracks.

```
* {
  box-sizing: border-box;
}

body {
  font: 1.2em sans-serif;
}

.grid {
  padding: 10px;
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}
```

```
.grid {
  display: grid-lanes;
  gap: 10px;
  grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));
}
```

```
<div class="grid">
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
</div>
```

```
// prettier-ignore
const itemSizes = [
  "2em", "3em", "1.6em", "4em", "3.2em",
  "3em", "4.5em", "1em", "3.5em", "2.8em",
];
const items = document.querySelectorAll(".item");
for (let i = 0; i < items.length; i++) {
  items[i].style.blockSize = itemSizes[i];
}
```

It is also possible to create a grid lanes layout with items loading into rows.

```
// prettier-ignore
const itemSizes = [
  "2em", "3em", "1.6em", "4em", "2.2em",
  "3em", "4.5em", "1em", "3.5em", "2.8em",
];
const items = document.querySelectorAll(".item");
for (let i = 0; i < items.length; i++) {
  items[i].style.inlineSize = itemSizes[i];
}
```

```
.grid {
  display: grid-lanes;
  gap: 10px;
  grid-template-rows: repeat(3, 100px);
}
```

Controlling the grid axis

On the grid axis, things will work just as you expect them to in grid layout. You can cause items to span multiple tracks while remaining in auto-placement, using the `span` keyword. Items may also be positioned using line-based positioning.

Grid lanes layout with spanning items

In this example two of the items span two tracks, and the remaining items are packed around them by the grid lanes layout algorithm.

```
<div class="grid">
  <div class="item"></div>
  <div class="item span-2"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item span-2"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
</div>
```

```
.grid {
  display: grid-lanes;
  gap: 10px;
  grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));
}

.span-2 {
  grid-column-end: span 2;
}
```

This example includes an item which has positioning for columns. Items with definite placement are placed before the grid lanes layout algorithm runs.

```
<div class="grid">
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item positioned">positioned.</div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
</div>
```

```
.grid {
  display: grid-lanes;
  gap: 10px;
  grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));
}

.positioned {
  padding: 1em;
  grid-column: 2 / 4;
}
```

Browser compatibility

See also

- `grid-auto-flow` for controlling grid auto-placement

Grid template areas

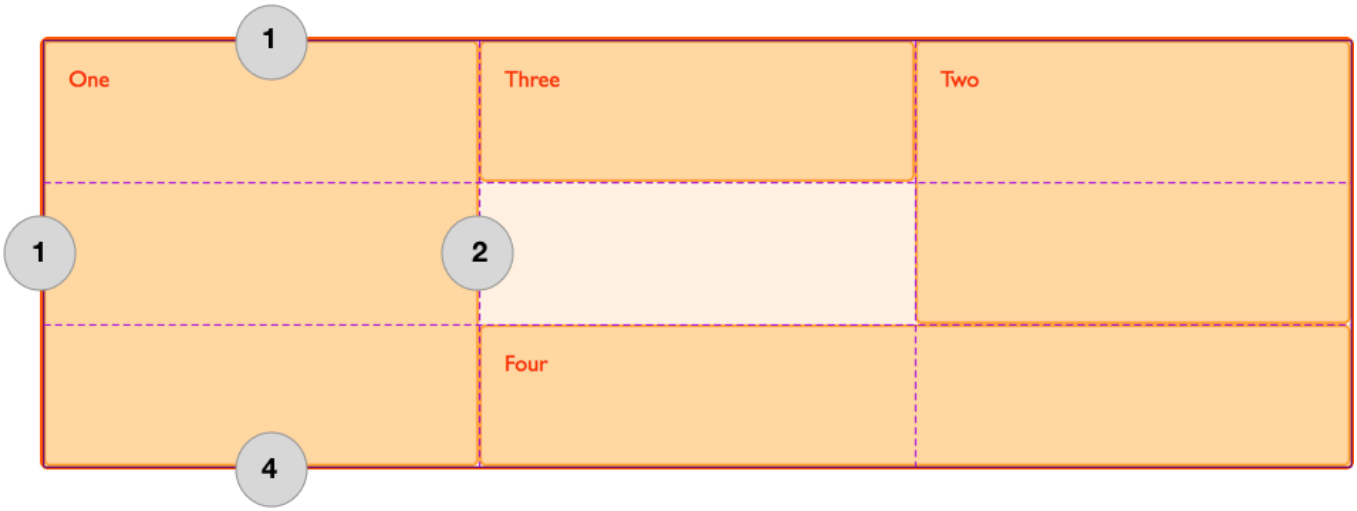
In the grid layout using line-based placement guide, we looked at grid lines and how to position items against those lines. When you use CSS grid layout, you always have lines, and this can be a straightforward way to place items on your grid. However, there is an alternate method to use for positioning items on the grid which you can use alone or in combination with line-based placement. This method involves placing our items using named template areas. You will see very quickly why we sometimes call this the ascii-art method of grid layout!

Naming a grid area

You have already encountered the `grid-area` property. This is the property that can take as a value all four of the lines used to position a grid area.

```
.box1 {
  grid-area: 1 / 1 / 4 / 2;
}
```

What we are doing here when defining all four lines, is defining the area by specifying the lines that enclose that area.



We can also define an area by giving it a name and then specify the location of that area in the value of the `grid-template-areas` property. You can choose what you would like to name your area. For example, if we wish to create the layout shown below we can identify four main areas.

- a header
- a footer
- a sidebar
- the main content

With the `grid-area` property we can assign each of these areas a name. By itself, this does not create any layout. Rather, it provides named areas to use in a layout.

```
.header {
  grid-area: hd;
}
.footer {
  grid-area: ft;
}
.content {
  grid-area: main;
}
.sidebar {
  grid-area: sd;
}
```

Having defined these names, we then create the layout. This time, instead of placing items using line numbers specified on the items themselves, we create the whole layout on the grid container. Here we create a 9-column grid and specify that the `hd` and `ft` areas span all 9 columns, while `sd` spans three and `main` spans six. Each area spans only one row.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-auto-rows: minmax(100px, auto);
  grid-template-areas:
    "hd hd hd hd   hd   hd   hd   hd   hd"
    "sd sd sd main main main main main main"
```

```
    "ft ft ft ft   ft   ft   ft   ft   ft";
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  max-width: 940px;
  margin: 0 auto;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Content</div>
  <div class="footer">Footer</div>
</div>
```

Using this method we do not need to specify anything at all on the individual grid items, everything happens on our grid container. We can see the layout described as the value of the `grid-template-areas` property.

Leaving a grid cell empty

We have completely filled our grid with areas in this example, leaving no white space. However you can leave grid cells empty with this method of layout. To leave a cell empty use the full stop character, `.`. If we want to only display the footer directly under the main content we would need to leave the three cells underneath the sidebar empty.

```
.header {
  grid-area: hd;
}
.footer {
  grid-area: ft;
}
.content {
  grid-area: main;
}
.sidebar {
  grid-area: sd;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  max-width: 940px;
  margin: 0 auto;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-auto-rows: minmax(100px, auto);
  grid-template-areas:
    "hd hd hd hd   hd   hd   hd   hd   hd"
    "sd sd sd main main main main main main"
    ".  .  .  ft   ft   ft   ft   ft   ft";
}
```

```
<div class="wrapper">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Content</div>
  <div class="footer">Footer</div>
</div>
```

In order to make the layout neater we can use multiple `.` characters. As long as there is at least one white space between the full stops it will be counted as one cell. For a complex layout there is a benefit to having the rows and columns neatly aligned. It means that you can actually see, right there in the CSS, what this layout looks like.

Spanning multiple cells

In our example, each area spans multiple grid cells and we achieve this by repeating the name of that grid area multiple times with white space between. You can add extra white space to keep your columns neatly lined up in the value of `grid-template-areas`. You can see that we have done this so that the `hd` and `ft` areas line up with `main`.

The area that you create by chaining the area names must be rectangular, at this point there is no way to create an L-shaped area. The specification does note that a future level might provide this functionality. You can however span rows just as easily as columns. For example we could make our sidebar span down to the end of the footer by replacing the `.` with `sd`.

```
.header {
  grid-area: hd;
}
.footer {
  grid-area: ft;
}
.content {
  grid-area: main;
}
.sidebar {
  grid-area: sd;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  max-width: 940px;
  margin: 0 auto;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-auto-rows: minmax(100px, auto);
  grid-template-areas:
    "hd hd hd hd   hd   hd   hd   hd   hd"
    "sd sd sd main main main main main main"
    "sd sd sd  ft  ft   ft   ft   ft   ft";
}
```

```
<div class="wrapper">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Content</div>
  <div class="footer">Footer</div>
</div>
```

The value of `grid-template-areas` must show a complete grid, otherwise it is invalid (and the property is ignored). This means that you must have the same number of cells for each row, if empty with a full stop character demonstrating that the cell is to be left empty. You will also create an invalid grid if your areas are not rectangular.

Redefining the grid using media queries

As our layout is now contained in one part of the CSS, we can make changes for different breakpoints in one location in our codebase. You can do this by redefining the grid, the position of items on the grid, or both at once.

When doing this, define the names for your areas outside of any media queries. That way the content area would always be called `main` no matter where on the grid it is placed.

For our layout above, we might like to have a very basic layout at narrow widths, defining a single column grid and stacking our four items into four rows.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  max-width: 940px;
  margin: 0 auto;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.header {
  grid-area: hd;
}
.footer {
  grid-area: ft;
}
.content {
  grid-area: main;
}
.sidebar {
  grid-area: sd;
}

.wrapper {
  display: grid;
  grid-auto-rows: minmax(100px, auto);
  grid-template-columns: 1fr;
  grid-template-areas:
```

```
    "hd"
    "main"
    "sd"
    "ft";
}
```

We can then redefine that layout inside media queries to go to our two columns layout, and perhaps take it to a three column layout if the available space is even wider. Note that for the wide layout, we keep the nine-column track grid, redefining where items are placed using `grid-template-areas`.

```
@media (width >= 30em) {
  .wrapper {
    grid-template-columns: repeat(9, 1fr);
    grid-template-areas:
      "hd hd hd hd   hd   hd   hd   hd   hd"
      "sd sd sd main main main main main main"
      "sd sd sd  ft  ft   ft   ft   ft   ft";
  }
}
@media (width >= 60em) {
  .wrapper {
    grid-template-areas:
      "hd hd hd   hd   hd   hd   hd   hd hd"
      "sd sd main main main main main ft ft";
  }
}
```

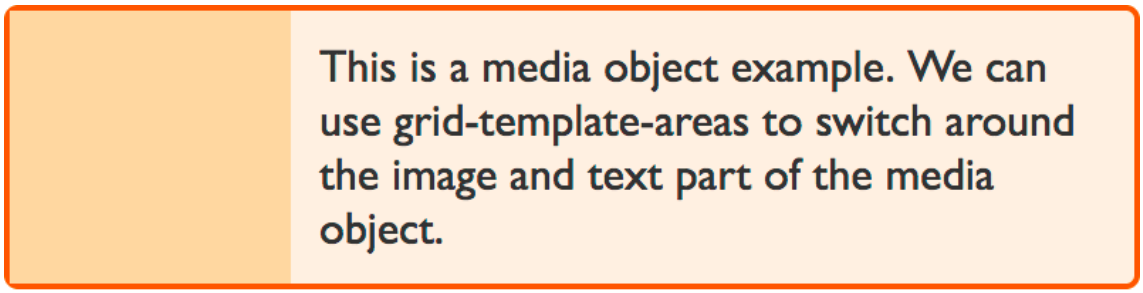
```
<div class="wrapper">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Content</div>
  <div class="footer">Footer</div>
</div>
```

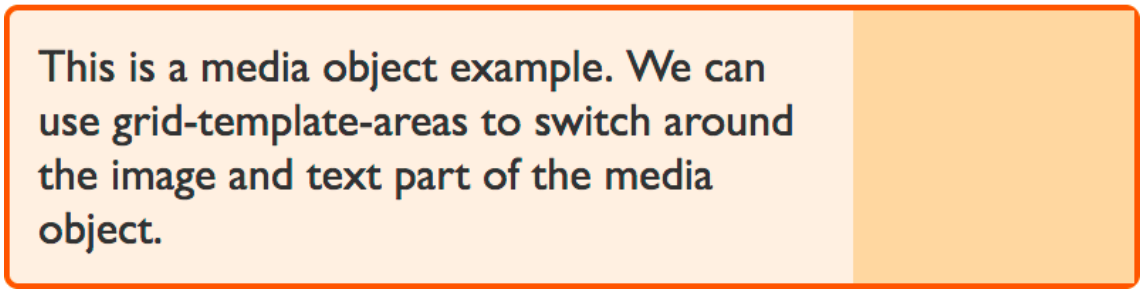
Using <code>grid-template-areas</code> for UI elements

Many of the grid examples you will find online assume you are using grid for whole page layouts; however, grid can be just as useful for laying out small sections. Using `grid-template-areas` can be especially nice as it allows you to visualize what your element looks like in the code.

Media object example

As an example, we can create a "media object". This is a component with space for an image or other media on one side and content on the other. The image might be displayed on the right or left of the box.





Our grid is a two-column track grid, with the column for the image sized at `1fr` and the text `3fr`. If you wanted a fixed width image area, then you could set the image column as a pixel width, and assign the text area `1fr`. A single column track of `1fr` would then take up the rest of the space.

We give the image area a grid area name of `img` and the text area `content`, then we can lay those out using the `grid-template-areas` property.

```
* {
  box-sizing: border-box;
}

.media {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  max-width: 400px;
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-areas: "img content";
  margin-bottom: 1em;
}

.media .image {
  grid-area: img;
  background-color: #ffd8a8;
}

.media .text {
```

```
  grid-area: content;
  padding: 10px;
}
```

```
<div class="media">
  <div class="image"></div>
  <div class="text">
    This is a media object example. We can use grid-template-areas to switch
    around the image and text part of the media object.
  </div>
</div>
```

Displaying the image on the other side of the box

We might want to be able to display our box with the image the other way around. To do this, we redefine the grid to put the `1fr` track last, and flip the values in `grid-template-areas`.

```
* {
  box-sizing: border-box;
}

.media {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  max-width: 400px;
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-areas: "img content";
  margin-bottom: 1em;
}

.media.flipped {
  grid-template-columns: 3fr 1fr;
  grid-template-areas: "content img";
}

.media .image {
  grid-area: img;
  background-color: #ffd8a8;
}

.media .text {
  grid-area: content;
  padding: 10px;
}
```

```
<div class="media flipped">
  <div class="image"></div>
  <div class="text">
    This is a media object example. We can use grid-template-areas to switch
    around the image and text part of the media object.
  </div>
</div>
```

Grid definition shorthands

Having looked at various ways of placing items on our grids and many of the properties used to define grid, this is a good time to take a look at a couple of shorthands that are available for defining the grid and many things about it all in one line of CSS.

These can quickly become difficult to read for other developers, or even your future self. However they are part of the specification and it is likely you will come across them in examples or in use by other developers, even if you choose not to use them.

Before using any shorthand it is worth remembering that shorthands not only enable the setting of many properties in one go, but they also **reset** everything that you do not (or cannot) set in the shorthand to their initial values. Therefore if you use a shorthand, be aware that it may reset things you have applied elsewhere.

The two shorthands for the grid container are the explicit grid shorthand `grid-template` and the grid definition shorthand `grid` .

<code>grid-template</code>

The `grid-template` shorthand property sets the following longhand properties:

- `grid-template-rows`
- `grid-template-columns`
- `grid-template-areas`

The property is referred to as the *explicit grid shorthand* because it sets values that you control when you define an explicit grid, and not those that impact any implicit row or column tracks that might be created.

The following code creates a layout using `grid-template` that is the same as the layout created earlier in this guide.

```
.wrapper {
  display: grid;
  grid-template:
    "hd hd hd hd   hd   hd   hd   hd   hd" minmax(100px, auto)
    "sd sd sd main main main main main main" minmax(100px, auto)
    "ft ft ft ft   ft   ft   ft   ft   ft" minmax(100px, auto)
    / 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr;
}
```

The first value is our `grid-template-areas` value but we also declare the size of the row at the end of each row. This is what the `minmax(100px, auto)` is doing.

Then after `grid-template-areas` we have a forward slash, after that is an explicit track listing of column tracks.

<code>grid</code>

The `grid` shorthand goes a step further and also sets properties used by the implicit grid. So you will be setting:

- `grid-template-rows`
- `grid-template-columns`

- `grid-template-areas`
- `grid-auto-rows`
- `grid-auto-columns`
- `grid-auto-flow`

You can use this syntax in the exact same way as the `grid-template` shorthand. Just be aware that when doing so you will reset the other values set by the property.

```
.wrapper {
  display: grid;
  grid:
    "hd hd hd hd   hd   hd   hd   hd   hd" minmax(100px, auto)
    "sd sd sd main main main main main main" minmax(100px, auto)
    "ft ft ft ft   ft   ft   ft   ft   ft" minmax(100px, auto)
    / 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr;
}
```

We will revisit the other functionality offered by this shorthand when we take a look at auto-placement in grid layout and the `grid-auto-flow` property.

Next steps

If you've been working along the grid guides, you should be in a position to create grid layouts using line-based placement or named template areas. Now let's take a look at creating grid layouts using named grid lines.

Grid layout using line-based placement

In the basic concepts of grid layout guide, we took a brief look at positioning items on a grid using line numbers. In this guide, we will fully explore how this fundamental feature of the specification works.

Starting your exploration of grid with numbered lines is the most logical place to begin because when you use grid layout, you always have numbered lines. The lines are numbered for columns and rows, and are indexed from 1. Note that grid is indexed according to the writing mode of the document. In a left-to-right language, such as English, line 1 is on the left-hand side of the grid. If you are working in a right-to-left language, such as Arabic, then line 1 will be on the right-hand of the grid. We will learn more about the interaction between writing modes and grids in the grids, logical values, and writing modes guide.

A basic example

As a basic example, we create a grid with 3 column tracks and 3 row tracks. This gives us 4 lines in each dimension.

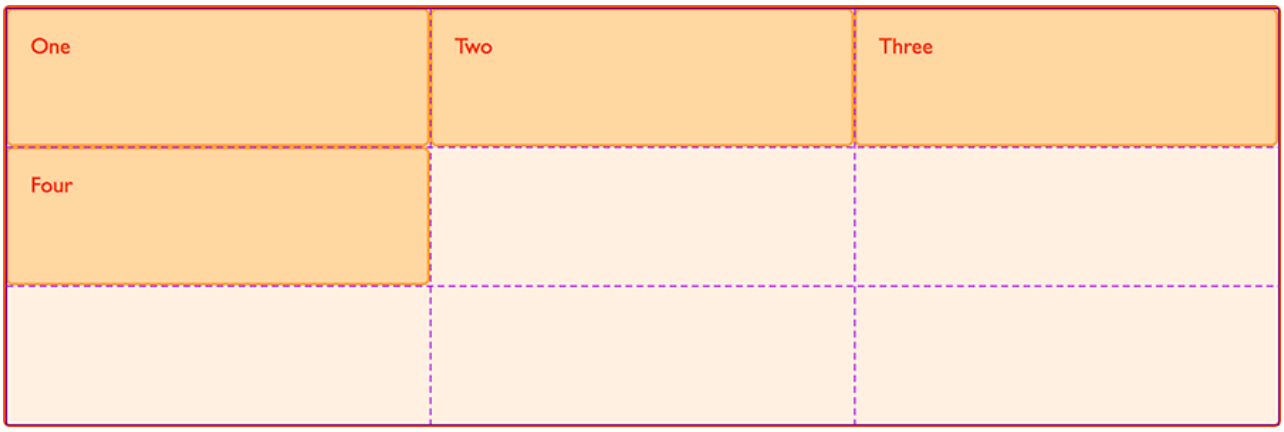
```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}
```

Inside our grid container, we include four child elements.

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

If we do not place these onto the grid in any way, they will lay out according to the auto-placement rules, one item in each of the first four cells. You can inspect the grid with your browser developer tools to see how the grid defines columns and rows.



Positioning items by line number

We can use line-based placement to control where these items sit on the grid. We can use the `grid-column-start` and `grid-column-end` properties to make the first item start on the far left of the grid and span a single column track. With `grid-row-start` and `grid-row-end`, we make the item start on the first row line at the top of the grid, and span to the fourth row line.

```
.box1 {
  grid-column-start: 1;
  grid-column-end: 2;
  grid-row-start: 1;
  grid-row-end: 4;
}
```

As you position some items, other items on the grid will continue to be laid out using the auto-placement rules. This behavior is explained in the Auto-placement in grid layout guide. For now, observe how the grid is laying out un-placed items into empty cells of the grid.

Addressing each item individually using the same properties but with different values, we place all four items, spanning row and column tracks.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column-start: 1;
  grid-column-end: 2;
  grid-row-start: 1;
  grid-row-end: 4;
}
.box2 {
  grid-column-start: 3;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}
.box3 {
  grid-column-start: 2;
  grid-column-end: 3;
  grid-row-start: 1;
  grid-row-end: 2;
}
.box4 {
  grid-column-start: 2;
  grid-column-end: 4;
  grid-row-start: 3;
  grid-row-end: 4;
}
```

Note that we can leave cells empty if we wish. One of the very nice things about grid layout is the ability to have white space in our designs without any hacks.

The <code>grid-column</code> and <code>grid-row</code> shorthands

The previous example had quite a lot of code to position each item. It should come as no surprise to know there is a shorthand. The `grid-column-start` and `grid-column-end` properties can be combined into `grid-column`, `grid-row-start` and `grid-row-end` into `grid-row`. In this example, we replicate the above example using these shorthand properties:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column: 1 / 2;
  grid-row: 1 / 4;
}
.box2 {
  grid-column: 3 / 4;
  grid-row: 1 / 3;
}
.box3 {
  grid-column: 2 / 3;
  grid-row: 1 / 2;
}
.box4 {
  grid-column: 2 / 4;
  grid-row: 3 / 4;
}
```

Default spans

In the above examples, we specified every end row and column line, in order to demonstrate the properties, however in practice if an item only spans one track you can omit the `grid-column-end` or `grid-row-end` value. Grid defaults to spanning one track.

Default spans with longhand placement

This means that our initial, long-hand, example would look like this:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column-start: 1;
  grid-row-start: 1;
  grid-row-end: 4;
}
.box2 {
  grid-column-start: 3;
  grid-row-start: 1;
  grid-row-end: 3;
}
.box3 {
  grid-column-start: 2;
  grid-row-start: 1;
}
.box4 {
  grid-column-start: 2;
  grid-column-end: 4;
  grid-row-start: 3;
}
```

Default spans with shorthand placement

Our shorthand would look like the following code, with no forward slash and second value for the items spanning one track only.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column: 1;
  grid-row: 1 / 4;
}
.box2 {
  grid-column: 3;
  grid-row: 1 / 3;
}
.box3 {
  grid-column: 2;
  grid-row: 1;
}
.box4 {
  grid-column: 2 / 4;
  grid-row: 3;
}
```

The <code>grid-area</code> property

We can take things a step further and define each area with a single property – `grid-area`. The order of the values for `grid-area` is as follows.

– `grid-row-start` – `grid-column-start` – `grid-row-end` – `grid-column-end`

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-area: 1 / 1 / 4 / 2;
}
.box2 {
  grid-area: 1 / 3 / 3 / 4;
}
.box3 {
  grid-area: 1 / 2 / 2 / 3;
}
.box4 {
  grid-area: 3 / 2 / 4 / 4;
}
```

This order of values for `grid-area` can seem a little strange — it is the opposite of the direction in which we specify margins and padding as a shorthand, for example. It may help to realize that this is due to CSS grid layout using the flow-relative directions defined in CSS writing modes. We explore how grids work with writing modes in grids, logical values, and writing modes. For now, consider the concept of four flow-relative directions:

- `block-start`
- `block-end`
- `inline-start`
- `inline-end`

We are working in English, a left-to-right language. Our `block-start` is the top row line of the grid container, `block-end` is the final row line of the container. Our `inline-start` is the left-hand column line as `inline-start` is always the point from which text would be written in the current writing mode, while `inline-end` is the final column line of our grid.

When we specify our grid area using the `grid-area` property we first define both start lines `block-start` and `inline-start`, then both end lines `block-end` and `inline-end`. This seems unusual at first as we are used to the physical properties of `top`, `right`, `bottom`, and `left`, but it makes more sense if you start to think of websites as being multi-directional in different writing modes.

Counting backwards

We can also count backwards from the block and inline ends of the grid, for English that would be the right-hand column line and final row line. The last lines of the explicit grid can be addressed as `-1`, and you can count back from there – so the second to last line is `-2`.

Do note that negative values are relevant only to the explicit grid. The final line is the final line of the grid defined by `grid-template-columns` and `grid-template-rows`, and does not take into account any rows or columns added in the *implicit grid* outside of that.

In this next example, we have flipped the layout we were working with by working from the right and bottom of our grid when placing the items.

```
* {
  box-sizing: border-box;
```

```
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column-start: -1;
  grid-column-end: -2;
  grid-row-start: -1;
  grid-row-end: -4;
}
.box2 {
  grid-column-start: -3;
  grid-column-end: -4;
  grid-row-start: -1;
  grid-row-end: -3;
}
.box3 {
  grid-column-start: -2;
  grid-column-end: -3;
  grid-row-start: -1;
  grid-row-end: -2;
}
.box4 {
  grid-column-start: -2;
  grid-column-end: -4;
  grid-row-start: -3;
  grid-row-end: -4;
}
```

Stretching an item across the grid

Being able to address the start and end lines of the grid is useful as you can then stretch an item right across the grid with:

```
.item {
  grid-column: 1 / -1;
}
```

Gutters or Alleys

CSS grid includes the ability to add gutters between column and row tracks with the `column-gap` and `row-gap` properties, or `gap` shorthand.

Gaps only appear between tracks of the grid, they do not add space to the top and bottom, left or right of the container. We can add gaps to our earlier example by using these properties on the grid container.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column: 1;
  grid-row: 1 / 4;
}
.box2 {
  grid-column: 3;
  grid-row: 1 / 3;
```

```
}
.box3 {
  grid-column: 2;
  grid-row: 1;
}
.box4 {
  grid-column: 2 / 4;
  grid-row: 3;
}
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
  column-gap: 20px;
  row-gap: 1em;
}
```

The gap shorthand

The two properties can also be expressed as a shorthand, `gap`. If you only give one value for `gap` it will apply to both column and row gaps. If you specify two values, the first is used for `row-gap` and the second for `column-gap`.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
  gap: 1em 20px;
}
```

In terms of line-based positioning of items, the gap acts as if the line has gained extra width. Anything starting at that line starts after the gap and you cannot address the gap or place anything into it. If you want gutters that act more like regular tracks you can define a track for that purpose.

Using the <code>span</code> keyword

In addition to specifying the start and end lines by number, you can specify a start line and then the number of tracks you would like the area to span using the `span` keyword.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

```
.box1 {
  grid-column: 1;
  grid-row: 1 / span 3;
}
.box2 {
  grid-column: 3;
  grid-row: 1 / span 2;
}
.box3 {
  grid-column: 2;
  grid-row: 1;
}
.box4 {
  grid-column: 2 / span 2;
  grid-row: 3;
}
```

You can also use the `span` keyword in the value of `grid-row-start`/`grid-row-end` and `grid-column-start`/`grid-column-end`. The following two examples will create the same grid area. In the first we set the start row line, then the end line we specify that we want the area to cover 3 tracks. The area will start at line 1 and end 3 lines from line 1; that is, the area will end at line 4.

```
.box1 {
  grid-column-start: 1;
  grid-row-start: 1;
  grid-row-end: span 3;
}
```

In the second example, we specify the end row line we want the item to finish at and then set the start line as `span 3`. This means the item will need to span upwards from the specified row line. The area will start at line 4 and span 3 lines to line 1.

```
.box1 {
  grid-column-start: 1;
  grid-row-start: span 3;
  grid-row-end: 4;
}
```

To become familiar with line based positioning in grid, try to build a few common layouts by placing items onto grids with varying numbers of columns. Remember that if you do not place all of the items, any leftover items will be placed according to auto-placement rules. This may result in the layout you want, but if something is appearing somewhere unexpected, check that you have set a position for it.

Also, remember that items on the grid can overlap when you place them explicitly like this. Overlapping items can create some nice effects, however, you can also end up with incorrect overlapping if you specify the wrong start or end line. Inspecting grids with your browser developer tools can be very helpful for identifying such problems as you learn, especially if your grid is quite complicated.

Grids, logical values, and writing modes

One of the most important features of CSS grid layout is the support for different writing modes built into the specification. In this guide, we look at this feature of CSS grid layout and other modern layout methods, learning a little about writing modes and logical vs. physical properties as we do so.

Logical and physical properties and values

CSS is full of **physical** positioning properties and keywords – `left` and `right`, `top` and `bottom`. In the code snippet below, we position an item using absolute positioning and use the physical inset properties as offset values to push the item around. The item is placed 20 pixels from the top and 30 pixels from the left of the container:

```
.container {
  position: relative;
}
.item {
  position: absolute;
  top: 20px;
  left: 30px;
}
```

```
<div class="container">
  <div class="item">Item</div>
</div>
```

This example uses the `left` and `right` properties; these are just two of the many **physical properties** in CSS. We can also add margins, padding, and borders using physical properties, for example `margin-left` and `padding-left`. You might also see physical keywords in use, such as when using `text-align: right` to align text to the right.

We call these keywords and properties *physical* because they relate to the screen you are looking at. Left is always left, no matter what direction your text is running.

Issues with physical properties

Physical properties can cause issues when developing a site that has to work in multiple languages, including languages where the text flows from right to left, or top to bottom. Browsers are designed to correctly display content regardless of the language. Some CSS features can override browser defaults and cause content to display less than optimally.

In this example, the `direction` property has been set to rtl, which switches the writing mode from the default for an English language document of `ltr`. We have two paragraphs. These should both flow from right to left because of the `direction` value set on an ancestor element (`<body>`). The first paragraph has `text-align` set to `left`, so it aligns with the left of its container. The second paragraph aligns with the right and flows from right to left.

```
<p class="left">
  I have my text set to <code>text-align: left</code> I will always align left
  even if the direction of the text in this document is rtl.
</p>

<p>I have no alignment set and use the direction set in the document.</p>
```

```
body {
  direction: rtl;
}
.left {
  text-align: left;
}
```

```
p {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  margin: 1em;
  color: #d9480f;
}
```

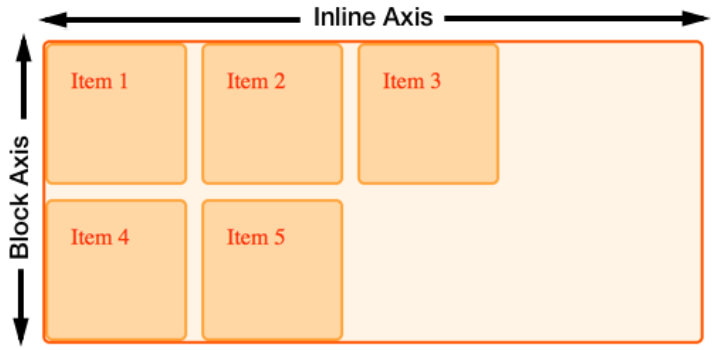
This is a basic demonstration of the problems that can arise when using physical values and properties in CSS. If we write CSS using physical properties and keywords, we tell the browser our assumption as to how the text will flow and prevent it from handling alternative writing modes.

Logical properties and values

Logical properties and values do not assume a text direction. This is why we use the keyword `start` in CSS grid layout to align something with the start of a container. When working with English content, `start` will be on the left, however, it doesn't have to be. The word `start` infers no physical location, which enables websites to start the content on the right when right-to-left languages, such as Arabic, are used.

Block and Inline

When using logical rather than physical properties, we don't see the world as left to right, and top to bottom. We have a different reference point. This is where understanding the *block* and *inline* axes, introduced in the grid alignment guide, becomes very useful. If you think about layout in terms of block and inline, the way things work in CSS grid layout makes a lot of sense.



CSS writing modes

The CSS writing modes module specifies how writing modes work in CSS. These features are not just for supporting languages with a different writing mode to English; they can also be used for creative purposes. The examples in this section make use of the `writing-mode` property to make changes to the writing mode applied to our grid, demonstrating how logical values work in the process.

<code>writing-mode</code>

Writing modes are more than just left to right and right to left text, and the `writing-mode` property helps us display text running in other directions. The `writing-mode` property can have values of:

- `horizontal-tb`
- `vertical-rl`
- `vertical-lr`
- `sideways-rl`
- `sideways-lr`

The value `horizontal-tb`, which stands for "horizontal, top to bottom", is the default for text on the web. It is the direction in which you are reading this guide. The other values change how text flows in our document, matching the different writing modes found worldwide.

As an example, we have two paragraphs below. The first uses the default `horizontal-tb` value, and the second uses `vertical-rl`. In the second writing mode, text still runs left to right, however the direction of the text is vertical — inline text now runs down the page, from top to bottom.

```
.wrapper > p {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  margin: 1em;
  color: #d9480f;
  max-width: 300px;
}
```

```
<div class="wrapper">
  <p class="horizontal-tb">
    I have writing mode set to the default <code>horizontal-tb</code>
  </p>
  <p class="vertical-rl">I have writing mode set to <code>vertical-rl</code></p>
</div>
```

```
.horizontal-tb {
  writing-mode: horizontal-tb;
}
.vertical-rl {
  writing-mode: vertical-rl;
}
```

Writing modes in grid layouts

Applying this to a grid layout example, we can see how changing the writing mode means changing our idea of where the block and inline axes are.

Default writing mode

In this example, the grid has three columns and two row tracks. This means there are three tracks running down the block axis. In default writing mode, grid auto-places items starting at the top left, moving along to the right, filling up the three cells on the inline axis. It then moves to the next line, creating a new row track, and fills in more items:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(2, 100px);
  gap: 10px;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
  <div class="item5">Item 5</div>
</div>
```

Setting writing mode

If we add `writing-mode: vertical-lr` to the grid container in the previous example, we can see that the block and inline axes now run in a different direction. The block or *column* axis now runs across the page from left to right, while the inline axis runs down the page, creating rows from top to bottom.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
```

```
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(2, 100px);
  gap: 10px;
}
```

```
.wrapper {
  writing-mode: vertical-lr;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
  <div class="item4">Item 4</div>
  <div class="item5">Item 5</div>
</div>
```

Logical values for alignment

With the block and inline axis able to change direction, the logical values for the alignment properties start to make more sense.

In this example, we use alignment (the `align-self` and `justify-self` properties) to align items inside a grid that is set to `writing-mode: vertical-lr`. The `start` and `end` properties work in exactly the same way that they do in the default writing mode, and remain logical in a way that using left and right, top and bottom to align items would not do. This occurs once we've flipped the grid onto the side, like this:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  writing-mode: vertical-lr;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 100px);
  gap: 10px;
}

.item1 {
  grid-column: 1 / 4;
  align-self: start;
}

.item2 {
  grid-column: 1 / 3;
  grid-row: 2 / 4;
  align-self: start;
}

.item3 {
  grid-column: 3;
  grid-row: 2 / 4;
  align-self: end;
  justify-self: end;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
</div>
```

If you want to see how these work, with a right to left as well as top to bottom writing mode, switch `vertical-lr` to `vertical-rl`, which is a vertical writing mode running from right to left.

Auto-placement and writing modes

As we've seen in the previous examples, writing mode can change the visual direction in which items place themselves onto the grid. Items will, by default, place themselves along the inline axis, adding new rows in the block direction. We've now seen that the inline axis does not always run from left to right, and the block axis does not always run from top to bottom.

Line-based placement and Writing Modes

The key thing to remember when placing items by line number, is that line 1 is the start line and line -1 is the end line, no matter which writing mode you are in.

Line-based placement with left to right text

In this example, we have a grid laid out in the default `ltr` direction, with three items positioned using line-based placement.

- Item 1 starts at column line 1, spanning one track.
- Item 2 starts at column line -1, spanning to -3.
- Item 3 starts at column line 1, spanning to column line 3.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(2, 100px);
  gap: 10px;
}
.item1 {
  grid-column: 1;
}
.item2 {
  grid-column: -1 / -3;
}
.item3 {
  grid-column: 1 / 3;
  grid-row: 2;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
</div>
```

Line-based placement with right to left text

If we add the `direction` property with a value of `rtl` to the grid container in the previous example, line 1 is placed on the right-hand side of the grid, and line -1 on the left.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(2, 100px);
  gap: 10px;
}

.item1 {
  grid-column: 1;
}

.item2 {
  grid-column: -1 / -3;
}

.item3 {
  grid-column: 1 / 3;
  grid-row: 2;
}
```

```
.wrapper {
  direction: rtl;
}
```

```
<div class="wrapper">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
```

```
  <div class="item3">Item 3</div>
</div>
```

If you are switching the direction of your text, either for entire pages or for parts of pages, and are using lines, you may want to name your lines to avoid the layout completely switching direction. For some things, for example, where a grid contains text content, this switching may be exactly what you want. For other uses it may not.

The strange order of values in the <code>grid-area</code> property

You can use the `grid-area` property to specify all four lines of a grid area as one value. When people first encounter this, they are often surprised that the values do not follow the same order as the shorthand for `margin` – which runs clockwise: top, right, bottom, left.

The order of `grid-area` values is:

- `grid-row-start`
- `grid-column-start`
- `grid-row-end`
- `grid-column-end`

Which for English, in left-to-right means the order is:

- `top`
- `left`
- `bottom`
- `right`

This is counter-clockwise! It's the reverse of what we do for margins and padding. If we remember that `grid-area` sees the world as "block and inline", you'll notice we are setting the two starts, then the two ends, which is much more logical once you know!

Mixed writing modes and grid layout

In addition to displaying documents using the correct writing mode for the language, writing modes can be used creatively within documents that are otherwise `ltr`. In this example, we have a grid layout with a set of links down one side. We use writing modes (`writing-mode: vertical-lr`) to turn these on their side in the column track:

```
.wrapper {
  display: grid;
  grid-gap: 20px;
  grid-template-columns: 1fr auto;
  font:
    1em "Helvetica",
    "Arial",
    sans-serif;
}
nav {
  writing-mode: vertical-lr;
}
nav ul {
  list-style: none;
  margin: 0;
  padding: 1em;
  display: flex;
  justify-content: space-between;
}
nav a {
  text-decoration: none;
}
```

```
<div class="wrapper">
  <div class="content">
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko. Lotus root water spinach fennel kombu maize bamboo
      shoot green bean swiss chard seakale pumpkin onion chickpea gram corn pea.
      Brussels sprout coriander water chestnut gourd swiss chard wakame kohlrabi
      beetroot carrot watercress. Corn amaranth salsify bunya nuts nori azuki
      bean chickweed potato bell pepper artichoke.
    </p>
    <p>
      Nori grape silver beet broccoli kombu beet greens fava bean potato
      quandong celery. Bunya nuts black-eyed pea prairie turnip leek lentil
      turnip greens parsnip. Sea lettuce water chestnut eggplant winter purslane
      fennel azuki bean earthnut pea sierra leone bologi leek soko chicory
      celtuce parsley jícama salsify.
    </p>
  </div>
  <nav>
    <ul>
      <li><a href="">Link 1</a></li>
      <li><a href="">Link 2</a></li>
      <li><a href="">Link 3</a></li>
    </ul>
  </nav>
</div>
```

Physical values and logical properties

If you combine logical grid properties with physical properties, remember that physical properties will not change according to writing mode. In our Aligning items in CSS grid layout guide, we use auto margins to push one item away from the others; this employs physical properties. There are logical property equivalents for most physical properties, which respect writing modes in the same way as grid placement and alignment properties and values.

Similarly, when using absolute positioning within a grid area, you can use logical inset properties to place items inside the grid area. When mixing logical and physical properties or values, be aware of the tension between them. For example, you may need to change your CSS to cope with a switch from `ltr` to `rtl`. Your understanding of block and inline through grid will help you to understand CSS logical properties and values.

Layout using named grid lines

In previous guides, we've looked at placing items on the lines created by defining grid tracks and also how to place items using named template areas. In this guide, we look at how these two things work together when we use named lines.

Line naming is incredibly useful, but some of the more confusing grid syntax comes from this combination of names and track sizes. Once you work through some examples, it should become clearer and easier to work with.

Naming lines when defining a grid

You can assign some or all of the lines in your grid a name when you define your grid with the `grid-template-rows` and `grid-template-columns` properties. To demonstrate, we'll use the basic layout created in the guide on line-based placement. This time, we'll create the grid using named lines.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

When defining the grid, we name our lines inside square brackets (`[]`). Those names can be anything you like. We define a name for the start and end of the container, both for rows and columns. In this case, the grid center block's start rows and columns are both named `content-start`, and its end rows and columns are both named `content-end`.

```
.wrapper {
  display: grid;
  grid-template-columns: [main-start] 1fr [content-start] 1fr [content-end] 1fr [main-end];
  grid-template-rows: [main-start] 100px [content-start] 100px [content-end] 100px [main-end];
}
```

We don't need to name all of the lines in our grids; you may choose to name just the key lines of your layout.

Once the lines have names, we can use the name we defined, rather than the line number, to place the grid items.

```
.box1 {
  grid-column-start: main-start;
  grid-row-start: main-start;
  grid-row-end: main-end;
}

.box2 {
  grid-column-start: content-end;
  grid-row-start: main-start;
  grid-row-end: content-end;
}

.box3 {
  grid-column-start: content-start;
  grid-row-start: main-start;
}

.box4 {
  grid-column-start: content-start;
  grid-column-end: main-end;
  grid-row-start: content-end;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
</div>
```

Everything else about line-based placement still works in the same way. In our grid layout, we provided each numeric line with an alias name. In our grid items, we reference a name rather than a number. Naming lines in this way is useful — when creating a responsive design, we can update the container's grid properties rather than updating the grid items within each media query.

Giving lines multiple names

You may want to give a line more than one name, perhaps it denotes the sidebar-end and the main-start for example. To do this add the names inside the square brackets with whitespace between them `[sidebar-end main-start]`. You can then refer to that line by either of the names.

Implicit grid areas from named lines

When naming the lines, we mentioned that you can name these anything you like. The name is a `<custom-ident>`, an author-defined name. When choosing the name you need to avoid words that might appear in the specification and be confusing - such as `span`. Idents are not quoted.

While you can choose any name, if you append `-start` and `-end` to the lines around an area, as we have in the example above, grid will create a named area of the main name used. Taking the above example, we have `content-start` and `content-end` both for rows and for columns. This means we get a grid area named `content`, and could place something in that area should we wish to.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}
```

```
.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

We use the same grid definitions as above, placing a single item into the named area `content`.

```
.wrapper {
  display: grid;
  grid-template-columns: [main-start] 1fr [content-start] 1fr [content-end] 1fr [main-end];
  grid-template-rows: [main-start] 100px [content-start] 100px [content-end] 100px [main-end];
}
.thing {
  grid-area: content;
}
```

```
<div class="wrapper">
  <div class="thing">I am placed in an area named content.</div>
</div>
```

We don't need to define where our areas are with `grid-template-areas` as our named lines have created an area for us.

Implicit grid lines from named areas

We have seen how named lines create a named area, and this also works in reverse. Named template areas create named lines that you can use to place your items. If we take the layout created in the guide to grid template areas, we can use the lines created by our areas to see how this works.

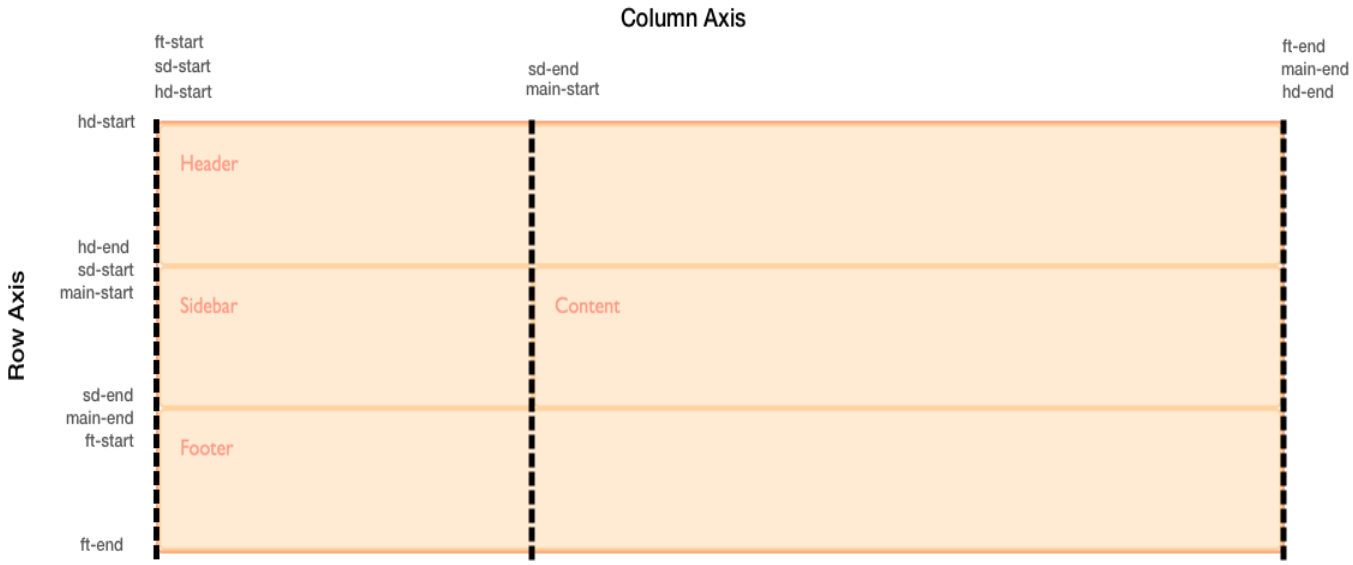
In this example, we have added an extra `<div>` with a class of `overlay`. We have named areas created using the `grid-area` property, then a layout created in `grid-template-areas`. The area names are:

- `hd`
- `ft`
- `main`
- `sd`

This gives us column and row lines:

- `hd-start`
- `hd-end`
- `sd-start`
- `sd-end`
- `main-start`
- `main-end`
- `ft-start`
- `ft-end`

You can see the named lines in the image. Note that some lines have two names - for example, `sd-end` and `main-start` refer to the same column line.



Positioning `overlay` using these implicit named lines is the same as positioning an item using named lines.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
```

```
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-auto-rows: minmax(100px, auto);
  grid-template-areas:
    "hd hd hd hd   hd   hd   hd   hd   hd"
    "sd sd sd main main main main main main"
    "ft ft ft ft   ft   ft   ft   ft   ft";
}

.header {
  grid-area: hd;
}

.footer {
  grid-area: ft;
}

.content {
  grid-area: main;
}

.sidebar {
  grid-area: sd;
}

.wrapper > div.overlay {
  z-index: 10;
  grid-column: main-start / main-end;
  grid-row: hd-start / ft-end;
  border: 4px solid rgb(92 148 13);
  background-color: rgb(92 148 13 / 40%);
  color: rgb(92 148 13);
  font-size: 150%;
}
```

```
<div class="wrapper">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Content</div>
  <div class="footer">Footer</div>
  <div class="overlay">Overlay</div>
</div>
```

Given that we have this ability to position created lines from named areas and areas from named lines, it is worth taking a little bit of time to plan your naming strategy when starting out creating your grid layout. Selecting names that make sense to you and your team will make your layouts more intuitive.

Multiple lines with the same name with repeat()

If you want to give all your grid lines a unique name, you need to define the track definition with long-hand properties rather than using the repeat syntax, as the names need to be added in square brackets when defining tracks. If you do use the repeat syntax, you will get multiple lines that have the same name, which can be useful or confusing, depending on your layout requirements.

Twelve-column grid using repeat()

In this example, we create a grid with 12 equal-width columns. Before defining the `1fr` size of the column track, we define a line named `[col-start]`. This means we will have a grid with 12 column lines all named `col-start` before a `1fr` width column.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(12, [col-start] 1fr);
}
```

Once you have created the grid you can place items onto it. As we have multiple lines named `col-start` if you place an item to start after line `col-start`, the first line named `col-start` will be used. In our case, this is the far left line. To address another line, use the name with the number for that line.

To place an item spanning from the first line named `col-start` to the 5th line with that name, we can use:

```
.item1to5 {
  grid-column: col-start / col-start 5;
}
```

You can also use the `span` keyword. This item will span 3 lines starting from the 7th line named `col-start`:

```
.item7to9 {
  grid-column: col-start 7 / span 3;
}
```

```
<div class="wrapper">
  <div class="item1to5">I am placed from col-start line 1 to col-start 5</div>
  <div class="item7to9">I am placed from col-start line 7 spanning 3 lines</div>
</div>
```

If you look at this layout in your browser's developer tools, you will see how the column lines are shown, and how our items are placed against these lines.

Defining named lines with a track list

The `repeat()` syntax can also take a track list; it is not just single track sizes that can be repeated.

This CSS creates an eight track grid, with a narrower `1fr` width column named `col1-start` followed by a wider `3fr` column named `col2-start`.

```
.wrapper {
  grid-template-columns: repeat(4, [col1-start] 1fr [col2-start] 3fr);
}
```

If your repeating syntax puts two lines next to each other then they will be merged and create the same result as giving a line multiple names in a non-repeating track definition. The following definition creates four `1fr` tracks, each with a start and end line.

```
.wrapper {
  grid-template-columns: repeat(4, [col-start] 1fr [col-end]);
}
```

If we write this declaration without using repeat notation, it looks like this:

```
.wrapper {
  grid-template-columns: [col-start] 1fr [col-end col-start] 1fr [col-end col-start] 1fr [col-end col-start] 1fr [col-end];
}
```

Using a track list, we can use the `span` keyword to span a number of lines, including lines of a certain name:

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(6, [col1-start] 1fr [col2-start] 3fr);
}

.item1 {
  grid-column: col1-start / col2-start 2;
}

.item2 {
  grid-row: 2;
  grid-column: col1-start 2 / span 2 col1-start;
}
```

```
<div class="wrapper">
  <div class="item1">
    I am placed from col1-start line 1 to col2-start line 2
  </div>
  <div class="item2">
    I am placed from col1-start line 2 spanning 2 lines named col1-start
  </div>
</div>
```

Twelve-column grid framework

Having learned about numeric and named line-based placement and grid template areas, we now know there are several ways to place items using CSS grid layout. This may seem overly complex, but you don't need to use all of them. In practice, using named template areas works well for straightforward layouts as this method gives a good visual representation of what your layout looks like, and makes it more intuitive to move things around on the grid. For example, when working with a strict multiple-column layout, the named lines demonstration in the last part of this guide works well.

Legacy grid systems such as Foundation or Bootstrap are based on a 12-column grid. These frameworks import code to do calculations that ensure the columns add up to 100%. Frameworks aren't needed! The only CSS we need for a 12-column grid "framework" is:

```
.wrapper {
  display: grid;
  gap: 10px;
```

```
  grid-template-columns: repeat(12, [col-start] 1fr);
}
```

We can then use this "framework" to lay out our page.

For example, to create a three-column layout with a header and footer, we can use the following markup.

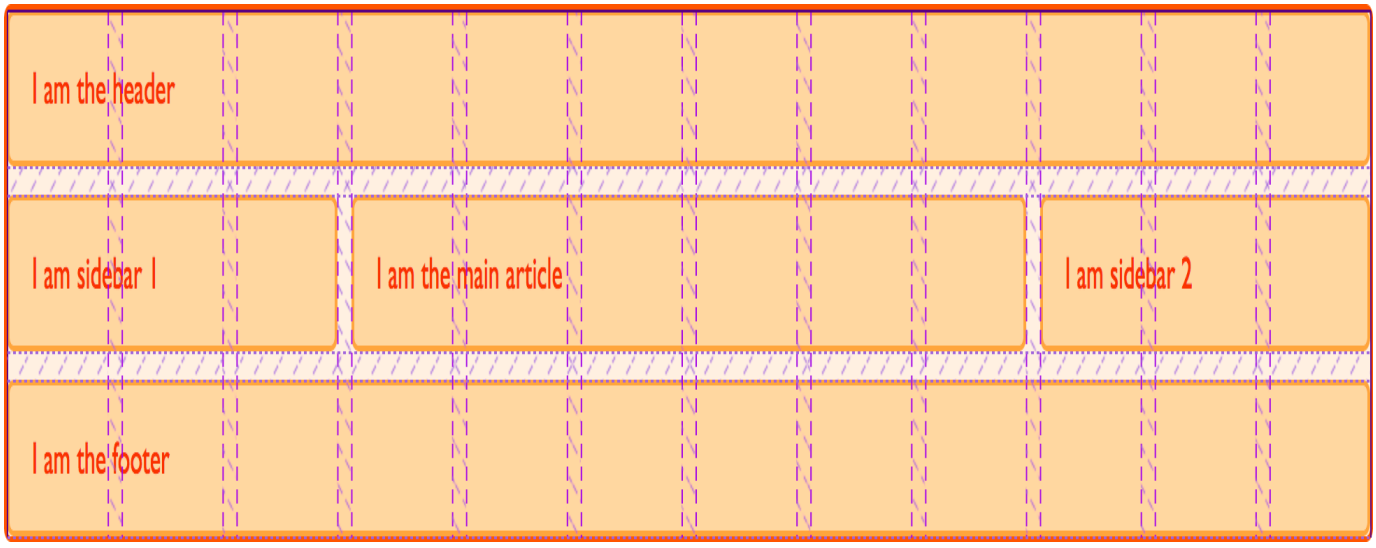
```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > * {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <header class="main-header">I am the header</header>
  <aside class="side1">I am sidebar 1</aside>
  <article class="content">I am the main article</article>
  <aside class="side2">I am sidebar 2</aside>
  <footer class="main-footer">I am the footer</footer>
</div>
```

We can place this on our grid layout framework:

```
.main-header,
.main-footer {
  grid-column: col-start / span 12;
}

.side1 {
  grid-column: col-start / span 3;
  grid-row: 2;
}

.content {
  grid-column: col-start 4 / span 6;
  grid-row: 2;
}

.side2 {
  grid-column: col-start 10 / span 3;
  grid-row: 2;
}
```

Once again, the developer tool's grid highlighter is helpful to show us how the grid we have placed our items on works.



That's all we need. We don't need to do any calculations! CSS grid layout automatically removed our 10-pixel gutter track before assigning the space to the `1fr` column tracks.

Up next, we will look at how CSS grid layout can position items for us without requiring placement properties at all, in the auto-placement in grid layout guide.

Relationship of grid layout to other layout methods

CSS grid layout is designed to work alongside other parts of CSS, as part of a complete system for doing the layout. This guide explains how grid layout fits together with other techniques.

Grid and flexbox

The basic difference between CSS grid layout and CSS flexbox layout is that flexbox was designed for layout in one dimension - either a row *or* a column. Grid was designed for two-dimensional layout - rows, and columns at the same time. The two specifications both use CSS box alignment features. If you have already learned how to use flexbox, the similarities should help you get to grips with grid.

One-dimensional versus two-dimensional layout

A basic example can demonstrate the difference between one- and two-dimensional layouts.

In this first example, we use flexbox to lay out a set of boxes. We have five child items in our container, and we have given the flex properties values so that they can grow and shrink from a flex-basis of 150 pixels.

We also set the `flex-wrap` property to `wrap`, so that if the space in the container becomes too narrow to maintain the flex basis, items will wrap onto a new row.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  width: 500px;
  display: flex;
  flex-wrap: wrap;
}
.wrapper > div {
  flex: 1 1 150px;
}
```

In the image, you can see that two items have wrapped onto a new line. These items are sharing the available space and not lining up underneath the items above. This is because when you wrap flex items, each new row (or column when working by column) is an independent flex line in the flex container. Space distribution happens across the flex line.

A common question then is how to make those items line up. This is where you want a two-dimensional layout method: You want to control the alignment by row and column, and this is where grid comes in.

The same layout with CSS grids

In this next example, we create the same layout using grid. This time we have three `1fr` column tracks. We do not need to set anything on the items themselves; they will lay themselves out one into each cell of the created grid. As you can see, they stay in a strict grid, lining up in rows and columns. With five items, we get a gap on the end of row two.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

An important question to ask yourself when deciding between grid or flexbox is:

- Do we only need to control the layout by row *or* column? If yes, use flexbox.
- Do we need to control the layout by row *and* column? If yes, use grid layout.

Content out or layout in?

In addition to the one-dimensional versus two-dimensional distinction, there is another way to decide if you should use flexbox or grid for a layout. Flexbox works from the content out. An ideal use case for flexbox is when you have a set of items and want to space them out evenly in a container. You let the size of the content decide how much individual space each item takes up. If the items wrap onto a new line, they will work out their spacing based on their size and the available space *on that line*.

Grid works from the layout in. When you use CSS grid layout you create a layout and then you place items into it, or you allow the auto-placement rules to place the items into the grid cells according to that strict grid. It is possible to create tracks that respond to the size of the content, however, they will also change the entire track.

If you are using flexbox and find yourself disabling some of the flexibility, you probably need to use CSS grid layout. For example, if you are setting a width on a flex item to make it line up with other items in a row above, a grid is likely a better choice.

Box alignment

Most grid alignment features were originally defined in the CSS flexible box layout. These features provided proper alignment control for the first time and enabled centering a box on the page. Flex items can stretch to the height of the flex container, meaning that equal height columns were possible. These properties are now defined in the CSS box alignment module, and are used in multiple layout modes, including grid layout.

We will be taking a proper look at Aligning items in CSS grid layout later. For now, here is a comparison between examples of flexbox and grid.

In the first example, which uses flexbox, we have a container with three items inside. The container's `min-height` is set, so it defines the height of the flex container. We have set `align-items` on the flex container to `flex-end` so the items will line up at the end of the flex container. We have also set the `align-self` property on `box1` so it will override the default and stretch to the height of the container and on `box2` so it aligns to the start of the flex container.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
</div>
```

```
.wrapper {
  display: flex;
  align-items: flex-end;
  min-height: 200px;
}
.box1 {
  align-self: stretch;
}
.box2 {
  align-self: flex-start;
}
```

Alignment in CSS grids

This example uses a grid to create the same layout. We use the box alignment properties as they apply to a grid layout. We align to `start` and `end`. (We could have used the `<content-position>` synonyms `flex-start` and `flex-end`.) In the case of a grid layout, we are aligning the items inside their grid area. In this case that is a single grid cell, but it could be an area made up of several grid cells.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  align-items: end;
  grid-auto-rows: 200px;
}
.box1 {
  align-self: stretch;
}
.box2 {
  align-self: start;
}
```

The <code>fr</code> unit and <code>flex-basis</code>

We have already seen how the `fr` unit works to assign a proportion of available space in the grid container to our grid tracks. The `fr` unit, when combined with the `minmax()` function can give us very similar behavior to the `flex` properties in flexbox while still enabling the creation of a layout in two dimensions.

If we look back at the example where we demonstrated the difference between one and two-dimensional layouts, you can see there is a difference between the way that the two layouts work responsively. With the flex layout, if we drag our window wider and smaller, the flexbox does a nice job of adjusting the number of items in each row according to the available space. If we have a lot of space all five items can fit on one row. If we have a very narrow container we may only have space for one.

In comparison, the grid version always has three column tracks. The tracks themselves will grow and shrink, but there are always three since we asked for three when defining our grid.

Auto-filling grid tracks

We can use grid to create a similar effect to flexbox, while still keeping the content arranged in strict rows and columns, by creating our track listing using repeat notation and the `auto-fill` and `auto-fit` properties.

In this next example, we have used the `auto-fill` keyword in place of an integer in the repeat notation and set the track listing to 200 pixels. This means that grid will create as many 200 pixels column tracks as will fit in the container.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(auto-fill, 200px);
}
```

A flexible number of tracks

This isn't quite the same as flexbox. In the flexbox example, the items are larger than the 200 pixel basis before wrapping. We can achieve the same in grid by combining `auto-fit` and the `minmax()` function.

In this example, we create auto filled tracks with `minmax`. We want our tracks to be a minimum of 200 pixels, so we set the maximum to be `1fr`. Once the browser has worked out how many times 200 pixels will fit into the container–also taking account of grid gaps–it will treat the `1fr` maximum as an instruction to share out the remaining space between the items.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
}
```

With grid layout, we can create a grid with a dynamic number of flexible tracks and have the items laid out on the grid aligned by rows and columns.

Grid and absolutely positioned elements

Grid interacts with absolutely positioned elements, which can be useful if you want to position an item inside a grid or grid area. The specification defines the behavior when a grid container is a containing block and a parent of the absolutely positioned item.

A grid container as containing block

To make the grid container a containing block, you need to add the `position` property to the container with a value of `relative`, just as you would make a containing block for any other absolutely positioned items. Once you have done this, if you give a grid item `position: absolute` it will take as its containing block the grid container or, if the item also has a grid position, the area of the grid it is placed into.

In the below example we have a wrapper containing four child items. Item three is absolutely positioned and also placed on the grid using line-based placement. The grid container has `position: relative` and so becomes the positioning context of this item.

```
* {
  box-sizing: border-box;
}
```

```
.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">
    This block is absolutely positioned. In this example the grid container is
    the containing block and so the absolute positioning offset values are
    calculated in from the outer edges of the area it has been placed into.
  </div>
  <div class="box4">Four</div>
</div>
```

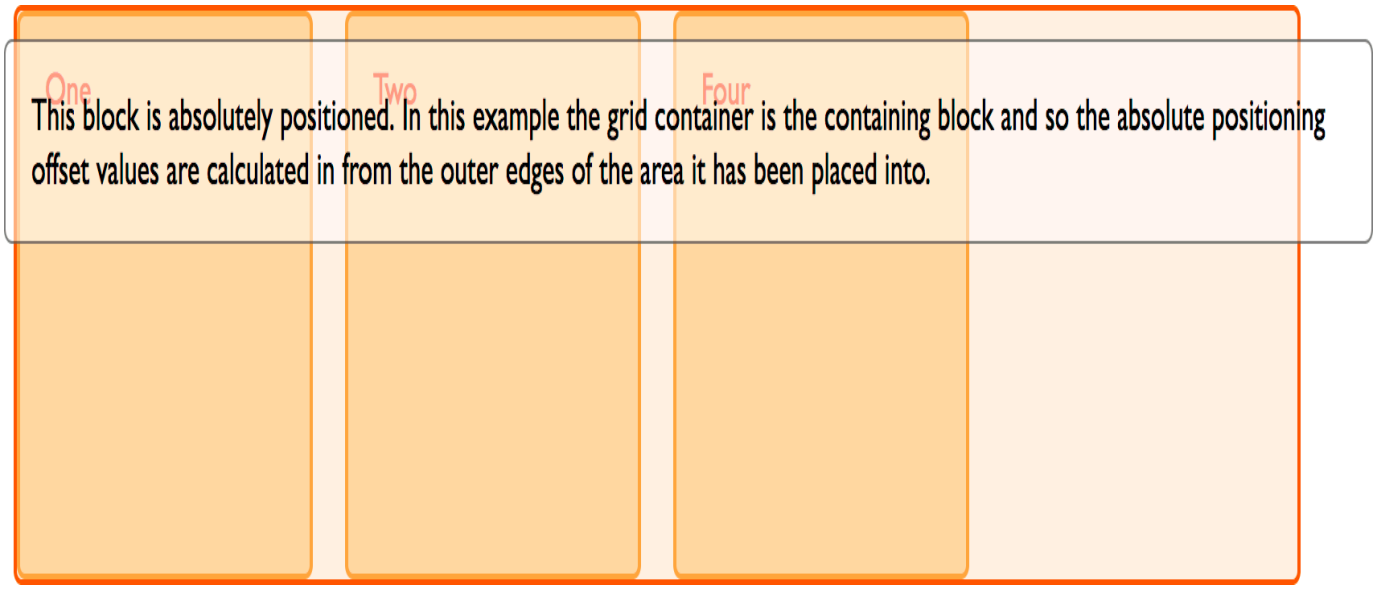
```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 200px;
  gap: 20px;
  position: relative;
}
.box3 {
  grid-column-start: 2;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  position: absolute;
  top: 40px;
  left: 40px;
}
```

You can see that the item is taking the area from grid column line 2 to 4, and starting after line 1. Then it is offset in that area using the top and left properties. However, it has been taken out of flow as is usual for absolutely positioned items and so the auto-placement rules now place items into the same space. The item also doesn't cause the additional row to be created to span to row line 3.

If we remove `position: absolute` from the rules for `.box3`, you can see how it would display without the positioning.

A grid container as parent

If the absolutely positioned child has a grid container as a parent but that container does not create a new positioning context, then it is taken out of flow as in the previous example. The *positioning context* is the element the absolutely positioned element is positioned relative to. The positioning context will be whatever element creates a positioning context as is common to other layout methods. In our case, if we remove `position: relative` from the wrapper above, positioning context is from the viewport, as shown in this image.



Once again, the item no longer participates in the grid layout in terms of sizing or when other items are auto-placed.

With a grid area as the parent

If the absolutely positioned item is nested inside a grid area, then you can create a positioning context on that area. In this example, we have our grid as before, but this time we have nested an item inside `.box3` of the grid.

We have given `.box3` position relative and then positioned the sub-item with the offset properties. In this case, the positioning context is the grid area.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
```

```
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">
    Three
    <div class="abspos">
      This block is absolutely positioned. In this example the grid area is the
      containing block and so the absolute positioning offset values are
      calculated in from the outer edges of the grid area.
    </div>
  </div>
  <div class="box4">Four</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 200px;
  gap: 20px;
}
.box3 {
  grid-column-start: 2;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  position: relative;
}
.abspos {
  position: absolute;
  top: 40px;
  left: 40px;
  background-color: rgb(255 255 255 / 50%);
  border: 1px solid rgb(0 0 0 / 50%);
  color: black;
  padding: 10px;
}
```

Grid and display: contents

A final interaction worth noting is the interaction between CSS grid layout and `display: contents`, defined in the CSS display module. When the `display` property is set to `contents`, the element itself does not generate any boxes, but its children and pseudo-elements still generate boxes as normal. This means that, for the purposes of box generation and layout, the element is treated as if it had been replaced with its children and pseudo-elements in the document tree.

If you set an item to `display: contents`, the box it would normally create disappears and the boxes of the child elements appear as if they have risen up a level. This means that children of a grid item can become grid items. Sound odd? Here is an example.

Grid layout with nested child elements

In this example, first item of our grid is set to span all three column tracks. It contains three nested items. As these items are not direct children, they don't become part of the grid layout and so display using regular block layout.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

```
<div class="wrapper">
  <div class="box box1">
    <div class="nested">a</div>
    <div class="nested">b</div>
    <div class="nested">c</div>
  </div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
```

```
  grid-auto-rows: minmax(100px, auto);
}
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
}
```

Using display: contents

If we now add `display: contents` to the rules for `box1`, the box for that item vanishes and the sub-items now become grid items and lay themselves out using the auto-placement rules.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

```
<div class="wrapper">
  <div class="box box1">
    <div class="nested">a</div>
    <div class="nested">b</div>
    <div class="nested">c</div>
  </div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
}
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  display: contents;
}
```

This can be a way to get items nested into the grid to act as if they are part of the grid. You can also use `display: contents` in a similar way with flexbox to enable nested items to become flex items.

As you can see from this guide, CSS grid layout is just one part of your toolkit. Don't be afraid to mix it with other methods of doing layout to get the different effects you need.

See also

- Flexbox guides
- Multiple-column layout guides

Subgrid

The CSS grid layout module includes a `subgrid` value for `grid-template-columns` and `grid-template-rows`. This guide details what subgrid does and gives some use cases and design patterns that the feature solves.

Introduction to subgrid

When you add `display: grid` to a grid container, only the direct children become grid items, which can then be placed on the grid you created. The children of these items display in normal flow.

You can "nest" grids by making a grid item a grid container. These grids, however, are independent of the parent grid and of each other, meaning that they do not take their track sizing from the parent grid. This makes it difficult to line nested grid items up with the main grid.

If you set the value `subgrid` on `grid-template-columns`, `grid-template-rows` or both, instead of creating a new track listing, the nested grid uses the tracks defined on the parent.

For example, if you use `grid-template-columns: subgrid` and the nested grid spans three column tracks of the parent, the nested grid will have three column tracks of the same size as the parent grid. While gaps are inherited, they can be overridden with a different `gap` value. Line names can be passed from the parent into the subgrid, and the subgrid can also declare its own line names.

Subgrid for columns

In the example below, the grid layout has nine `1fr` column tracks and four rows that are a minimum of `100px` tall.

The `.item` is placed between column lines 2 to 7 and rows 2 to 4. This grid item is itself specified as a grid using `display: grid` and then defined as a subgrid by giving it column tracks that are a subgrid (`grid-template-columns: subgrid`) and normally defined rows. The subgrid has five-column tracks as it spans five column tracks.

Because the `.item` is a subgrid, even though the `.subitem` is not a direct child of the outer `.grid`, it can be placed on that outer grid, with its columns aligned with the outer grid's columns. The rows are not a subgrid, so behave as a nested grid normally does. The grid area on the parent expands to be large enough for this nested grid.

```
<div class="grid">
  <div class="item">
    <div class="subitem"></div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}

.subitem {
  background-color: rgb(40 240 83);
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-template-rows: repeat(4, minmax(100px, auto));
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid;
  grid-template-rows: repeat(3, 80px);
}

.subitem {
  grid-column: 3 / 6;
  grid-row: 1 / 3;
}
```

Note that line numbering restarts inside the subgrid — column line 1, when inside the subgrid, is the first line of the subgrid. The subgridded element doesn't inherit the line numbers of the parent grid. This means that you can safely lay out a component that may be placed in different positions on the main grid, knowing that the line numbers on the component will always be the same.

Subgrid for rows

This example uses the same HTML as above, but here the `subgrid` is applied as the value of `grid-template-rows` instead, with explicitly defined column tracks. In this case, the column tracks behave as a regular nested grid, but the rows are tied to the two tracks that the `.item` spans.

```
<div class="grid">
  <div class="item">
    <div class="subitem"></div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}

.subitem {
  background-color: rgb(40 240 83);
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-template-rows: repeat(4, minmax(100px, auto));
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: subgrid;
}

.subitem {
  grid-column: 2 / 4;
  grid-row: 1 / 3;
}
```

A subgrid in both dimensions

In this example, both rows and columns are defined as a subgrid, tying the subgrid to the parent grid's tracks in both dimensions.

```
<div class="grid">
  <div class="item">
```

```
    <div class="subitem"></div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}

.subitem {
  background-color: rgb(40 240 83);
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-template-rows: repeat(4, minmax(100px, auto));
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid;
  grid-template-rows: subgrid;
}

.subitem {
  grid-column: 3 / 6;
  grid-row: 1 / 3;
}
```

No implicit grid in a subgridded dimension

If you need to autoplace items and do not know how many items you will have, take care when creating a subgrid, as it will prevent additional rows from being created to hold those items.

Take a look at the next example — it uses the same parent and child grid as in the example above. There are twelve items inside the subgrid trying to autoplace themselves into ten grid cells. As the subgrid is on both dimensions, there is nowhere for the extra two items to go, so they go into the last track of the grid. This is the behavior defined in the specification.

```
<div class="grid">
  <div class="item">
    <div class="subitem">1</div>
    <div class="subitem">2</div>
    <div class="subitem">3</div>
    <div class="subitem">4</div>
    <div class="subitem">5</div>
    <div class="subitem">6</div>
    <div class="subitem">7</div>
    <div class="subitem">8</div>
    <div class="subitem">9</div>
    <div class="subitem">10</div>
    <div class="subitem">11</div>
    <div class="subitem">12</div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}
body {
  font: 1.2em sans-serif;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  color: #d9480f;
}

.subitem {
  background-color: #d9480f;
  color: white;
  border-radius: 5px;
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-template-rows: repeat(4, minmax(100px, auto));
}

.item {
  display: grid;
```

```
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid;
  grid-template-rows: subgrid;
}
```

By removing the `grid-template-rows` value, the regular creation of implicit tracks is enabled, creating as many rows as required. These won't line up with the tracks of the parent.

```
<div class="grid">
  <div class="item">
    <div class="subitem">1</div>
    <div class="subitem">2</div>
    <div class="subitem">3</div>
    <div class="subitem">4</div>
    <div class="subitem">5</div>
    <div class="subitem">6</div>
    <div class="subitem">7</div>
    <div class="subitem">8</div>
    <div class="subitem">9</div>
    <div class="subitem">10</div>
    <div class="subitem">11</div>
    <div class="subitem">12</div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}
body {
  font: 1.2em sans-serif;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  color: #d9480f;
}

.subitem {
  background-color: #d9480f;
  color: white;
  border-radius: 5px;
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-template-rows: repeat(4, minmax(100px, auto));
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid;
  grid-auto-rows: minmax(100px, auto);
}
```

The gap properties and subgrid

Any `gap`, `column-gap`, or `row-gap` values specified on the parent are passed into the subgrid, creating the same spacing between tracks as the parent. This default behavior can be overridden by applying `gap-*` properties on the subgrid container.

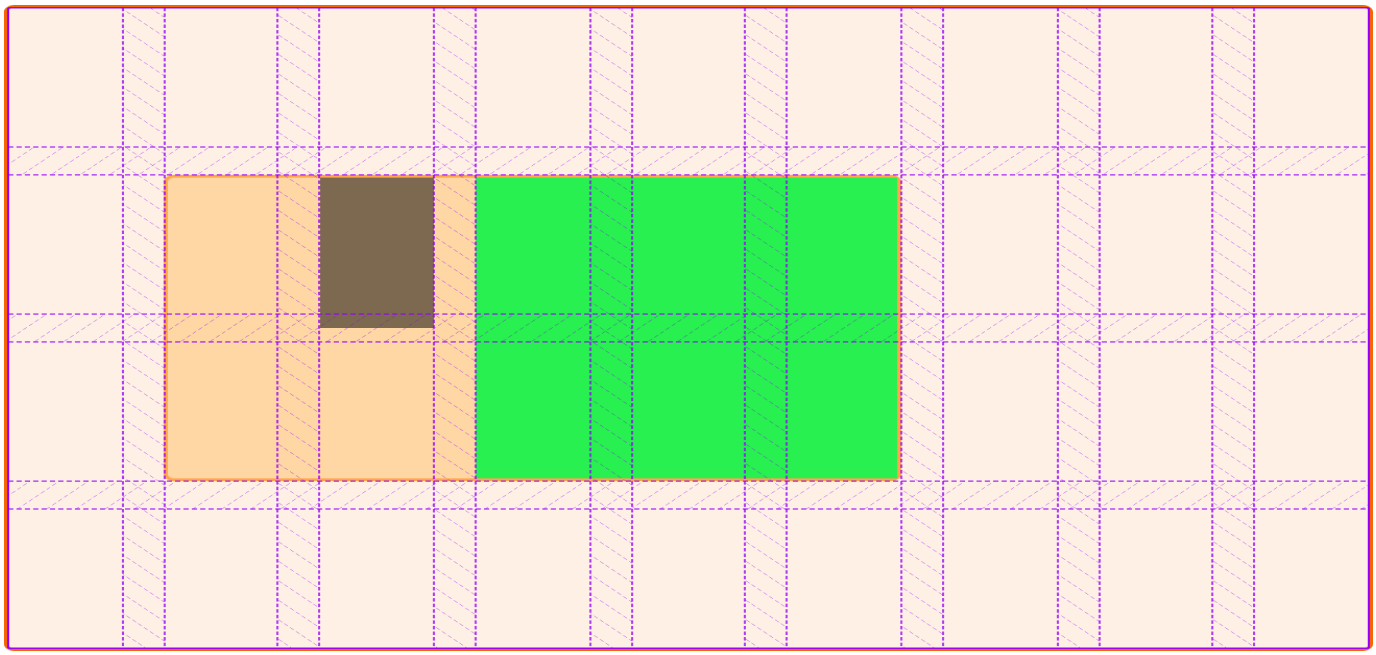
In this example, the parent grid has a gap of `20px` for rows and columns and the subgrid has `row-gap` set to `0`.

```
<div class="grid">
  <div class="item">
    <div class="subitem"></div>
    <div class="subitem2"></div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}

.subitem {
  background-color: rgb(40 240 83);
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  grid-template-rows: repeat(4, minmax(100px, auto));
  gap: 20px;
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid;
  grid-template-rows: subgrid;
  row-gap: 0;
}

.subitem {
  grid-column: 3 / 6;
  grid-row: 1 / 3;
}

.subitem2 {
  background-color: rgb(0 0 0 / 0.5);
  grid-column: 2;
  grid-row: 1;
}
```

If you inspect this in your developer tools grid inspector, you will note that the subgrid line is at the center of the gap. Setting the gap to `0` acts in a similar way to applying a negative margin to an element, giving the space from the gap back to the item.



Named grid lines

When using CSS grid, you can name lines on your grid and then position items based on those names rather than the line number. The line names on the parent grid are passed into the subgrid, and you can place items using them. In the example below, the named lines of the parent `col-start` and `col-end` are used to place the subitem.

```
<div class="grid">
  <div class="item">
    <div class="subitem"></div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}

.subitem {
  background-color: rgb(40 240 83);
}
```

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr [col-start] 1fr 1fr 1fr [col-end] 1fr 1fr 1fr;
  grid-template-rows: repeat(4, minmax(100px, auto));
  gap: 20px;
```

```
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid;
  grid-template-rows: subgrid;
}

.subitem {
  grid-column: col-start / col-end;
  grid-row: 1 / 3;
}
```

You can also specify line names on the subgrid. This is achieved by adding a list of line names enclosed in square brackets after the `subgrid` keyword. For example, if you have four lines in your subgrid, to name them all, you could use the syntax `grid-template-columns: subgrid [line1] [line2] [line3] [line4]`

Lines specified on the subgrid are added to any lines specified on the parent, so you can use either or both. In this example, one item is placed below using the parent lines and one using the subgrid lines.

```
<div class="grid">
  <div class="item">
    <div class="subitem"></div>
    <div class="subitem2"></div>
  </div>
</div>
```

```
* {
  box-sizing: border-box;
}

.grid {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.item {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  color: #d9480f;
}

.subitem {
  background-color: rgb(40 240 83);
}
```

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr [col-start] 1fr 1fr 1fr [col-end] 1fr 1fr 1fr;
  grid-template-rows: repeat(4, minmax(100px, auto));
  gap: 20px;
}

.item {
  display: grid;
  grid-column: 2 / 7;
  grid-row: 2 / 4;
  grid-template-columns: subgrid [sub-a] [sub-b] [sub-c] [sub-d] [sub-e] [sub-f];
  grid-template-rows: subgrid;
}

.subitem {
  grid-column: col-start / col-end;
  grid-row: 1 / 3;
}

.subitem2 {
  background-color: rgb(0 0 0 / 0.5);
  grid-column: sub-b / sub-d;
  grid-row: 1;
}
```

Using subgrids

A subgrid acts very similarly to any nested grid; the only difference is that the track sizing of the subgrid is set on the parent grid. As with any nested grid, however, the size of the content in the subgrid can change the track sizing, assuming a track sizing method is used that allows content to affect the size. In such a case, auto-sized row tracks will grow to fit content in the main grid and content in the subgrid.

As the subgrid value acts in much the same way as a regular nested grid, it is easy to switch between the two. For example, if you realize that you need an implicit grid for rows, you would need to remove the `subgrid` value of `grid-template-rows` and perhaps give a value for `grid-auto-rows` to control the implicit track sizing.

See also

- Video: Laying out forms using subgrid (2019)
- Video: Don't wait to use subgrid for better card layouts (2019)
- Video: Hello subgrid! presentation from CSSConf.eu (2019)

CSS images

The **CSS images** module defines the types of images that can be used (the `<image>` type, containing URLs, gradients and other types of images), how to resize them and how they, and other replaced content, interact with the different layout models.

Reference

Properties

- `image-orientation`
- `image-rendering`

- `object-fit`
- `object-position`
- `object-view-box`

The CSS images module also defines the `image-resolution` property. Currently, no browsers support this feature.

Functions

- `linear-gradient()`
- `radial-gradient()`
- `repeating-linear-gradient()`
- `repeating-radial-gradient()`
- `conic-gradient()`
- `repeating-conic-gradient()`
- `cross-fade()`
- `element()`
- `image-set()`

The CSS images module also defines the `image()` function. Currently, no browsers support this feature.

Data types

- `<gradient>`
- `<image>`

Guides

Using CSS gradients

Presents a specific type of CSS images, *gradients*, and how to create and use these.

Implementing image sprites in CSS

Describes the common technique grouping several images in one single document to save download requests and speed up the availability of a page.

Styling replaced elements

Introduces the properties that only apply to *replaced elements*.

Understanding aspect ratios

Learn about the `aspect-ratio` property, discuss aspect ratios for replaced and non-replaced elements, and examine some common aspect ratio use cases.

Using the CSS `object-view-box` property

Learn the CSS `object-view-box` property, including how to zoom in, zoom out, and pan across images.

Related concepts

- `<url>`
- `url()`
- `<basic-shape-rect>`

See also

- CSS filter effects module
- CSS compositing and blending module
- CSS colors module
- CSS values and units module

Implementing image sprites in CSS

Image sprites are used in numerous web apps where multiple images are used. Rather than include each image as a separate image file, it is much more memory- and bandwidth-friendly to send them as a single image; using background position as a way to distinguish between individual images in the same image file, so the number of HTTP requests is reduced.

> **Note:** When using HTTP/2, it may in fact be more bandwidth-friendly to use multiple small requests.

Implementation

Suppose an image is given to every item with class `tool-btn`:

```
.tool-btn {
  background: url("myfile.png");
  display: inline-block;
  height: 20px;
  width: 20px;
}
```

A background position can be added either as two x, y values after the `<url>` in the background, or as `background-position`. For example:

```
#btn1 {
  background-position: -20px 0px;
}

#btn2 {
  background-position: -40px 0px;
}
```

This would slide the starting point of the background image for the element with the ID `btn1` 20 pixels to the left and the element with the ID `btn2` 40 pixels to the left (assuming they have the class `tool-btn` assigned and are affected by the image rule above).

Similarly, you can also make hover states by targeting `#btn:hover`.

See also

- Full working demo at CSS Tricks

Styling replaced elements

Some CSS properties apply to all elements, some only to grid and flex containers, others only to transformable elements. This guide introduces the properties that only apply to *replaced elements*.

A **replaced element** is an element whose representation is outside the scope of CSS; they're external objects whose representation is independent of the CSS formatting model. Some replaced elements, such as `<iframe>` elements, may have stylesheets of their own, but they don't inherit the styles of the parent document.

Using CSS with replaced elements

CSS handles replaced elements specifically in some cases, like when calculating margins and some `auto` values. Only replaced elements can ever have intrinsic dimensions. Some replaced elements, but not all, have intrinsic dimensions or a defined baseline, which is used by some CSS properties, such as `vertical-align`.

While document styles can set the size and position of replaced elements, document styles don't affect the replaced elements' content, with some exceptions: The CSS images module includes properties which support controlling the positioning of the element's content within its box.

Controlling object position within the content box

The CSS images module defines two properties which can be used to specify how the object contained within the replaced element should be positioned within the element's box area. The `object-fit` property is used to size objects, while the `object-position` property is used to position them.

The <code>object-fit</code> property

The `object-fit` property specifies how the replaced element's content object should be fitted to the containing element's box. The property defines how images, videos and other embeddable media formats respond to the height and width of the replaced element's content box. If the height, width, or aspect-ratio of an element differ from the resource that will be occupying space reserved for the element, the `fill`, `contain`, `cover`, `scale-down`, and `none` values define whether the browser should scale the resource, cover the allocated space, contain the asset within the space, or allow the resource to be distorted.

When contained or scaled down, any areas of the box not covered by the replaced element will show the element's background.

The `object-fit` property has no effect on `<iframe>`, `<embed>`, and `<fencedframe>` elements.

If we place the image, a square with a 1:1 aspect ratio, in a 100px x 300px box (1:3 aspect ratio), by default the image will fill the box, distorting itself. We can use the `object-fit` property to define how the image should be rendered when forced into a box with a different size and aspect ratio:

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<p>
  <label><input type="checkbox" /> Change dimensions</label>
</p>
```

```
body {
  display: flex;
  gap: 20px;
  flex-flow: row wrap;
  grid-auto-flow: column;
  max-width: 98%;
  margin: 10px auto 0;
}
img {
  width: 100px;
  height: 300px;
  outline: 2px solid purple;
}
body:has(:checked) img {
  width: 300px;
  height: 100px;
}
```

```
img:nth-of-type(1) {
  object-fit: fill;
}
img:nth-of-type(2) {
  object-fit: cover;
}
img:nth-of-type(3) {
  object-fit: contain;
}
img:nth-of-type(4) {
  object-fit: scale-down;
```

```
}
img:nth-of-type(5) {
  object-fit: none;
}
img:nth-of-type(6) {
  /* no object-fit property */
  outline: 2px dashed red;
}
```

Check the box to set the height and width values. Note that only the `fill` value (the default) distorts the original image. With all other values, the intrinsic aspect ratio of the image is maintained.

The <code>object-position</code> property

The `object-position` property specifies the alignment of the replaced element's content object within the element's box.

Often used in conjunction with the `object-fit` property, it takes as its value a `<position>` value, the same value type as used for `background-position`.

```
img {
  object-position: bottom right;
}
```

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

It can be used without `object-fit`. In this case, the image is rendered at it's intrinsic size (218px x 218px), with the position of the contents of the image set by the `object-position` value.

```
img {
  margin: 10px 0 0 10px;
}
```

```
img {
  outline: 2px solid;
  object-position: 114px 72px;
}
```

The `object-position` property works equally well with `<iframe>`, `<video>`, and `<embed>` elements as it does for `<img>`.

See also

- Understanding aspect ratios
- CSS images module
- CSS display module
- CSS backgrounds and borders module

Using CSS gradients

CSS gradients are represented by the `<gradient>` data type, a special type of `<image>` made of a progressive transition between two or more colors. You can choose between three types of gradients: *linear* (created with the `linear-gradient()` function), *radial* (created with the `radial-gradient()` function), and *conic* (created with the `conic-gradient()` function). You can also create repeating gradients with the `repeating-linear-gradient()`, `repeating-radial-gradient()`, and `repeating-conic-gradient()` functions.

Gradients can be used anywhere you would use an `<image>`, such as in backgrounds. Because gradients are dynamically generated, they can negate the need for the raster image files that traditionally were used to achieve similar effects. In addition, because gradients are generated by the browser, they look better than raster images when zoomed in, and can be resized on the fly.

We'll start by introducing linear gradients, then introduce features that are supported in all gradient types using linear gradients as the example, then move on to radial, conic and repeating gradients

Using linear gradients

A linear gradient creates a band of colors that progress in a straight line.

A basic linear gradient

To create the most basic type of gradient, all you need is to specify two colors. These are called *color stops*. You must have at least two, but you can have as many as you want.

```
<div class="simple-linear"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.simple-linear {
  background: linear-gradient(blue, pink);
}
```

Changing the direction

By default, linear gradients run from top to bottom. You can change their rotation by specifying a direction.

```
<div class="horizontal-gradient"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.horizontal-gradient {
  background: linear-gradient(to right, blue, pink);
}
```

Diagonal gradients

You can even make the gradient run diagonally, from corner to corner.

```
<div class="diagonal-gradient"></div>
```

```
div {
  width: 200px;
  height: 100px;
}
```

```
.diagonal-gradient {
  background: linear-gradient(to bottom right, blue, pink);
}
```

Using angles

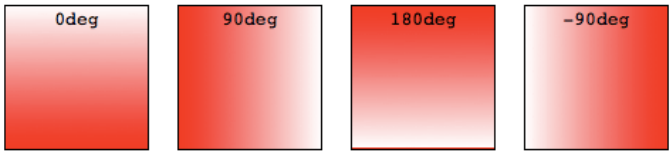
If you want more control over its direction, you can give the gradient a specific angle.

```
<div class="angled-gradient"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.angled-gradient {
  background: linear-gradient(70deg, blue, pink);
}
```

When using an angle, `0deg` creates a vertical gradient running bottom to top, `90deg` creates a horizontal gradient running left to right, and so on in a clockwise direction. Negative angles run in the counterclockwise direction.



Declaring colors & creating effects

All CSS gradient types are a range of position-dependent colors. The colors produced by CSS gradients can vary continuously with position, producing smooth color transitions. It is also possible to create bands of solid colors, and hard transitions between two colors. The following are valid for all gradient functions:

Using more than two colors

You don't have to limit yourself to two colors—you may use as many as you like! By default, colors are evenly spaced along the gradient.

```
<div class="auto-spaced-linear-gradient"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.auto-spaced-linear-gradient {
  background: linear-gradient(red, yellow, blue, orange);
}
```

Positioning color stops

You don't have to leave your color stops at their default positions. To fine-tune their locations, you can give each one zero, one, or two percentage or, for radial and linear gradients, absolute length values. If you specify the location as a percentage, `0%` represents the starting point, while `100%` represents the ending point; however, you can use values outside that range if necessary to get the effect you want. If you leave a location unspecified, the position of that particular color stop will be automatically calculated for you, with the first color stop being at `0%` and the last color stop being at `100%`, and any other color stops being half way between their adjacent color stops.

```
<div class="multicolor-linear"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.multicolor-linear {
  background: linear-gradient(to left, lime 28px, red 77%, cyan);
}
```

Creating hard lines

To create a hard line between two colors, creating a stripe instead of a gradual transition, adjacent color stops can be set to the same location. In this example, the colors share a color stop at the `50%` mark, halfway through the gradient:

```
<div class="striped"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.striped {
  background: linear-gradient(to bottom left, cyan 50%, palegoldenrod 50%);
}
```

Creating color bands & stripes

To include a solid, non-transitioning color area within a gradient, include two positions for the color stop. Color stops can have two positions, which is equivalent to two consecutive color stops with the same color at different positions. The color will reach full saturation at the first color stop, maintain that saturation through to the second color stop, and transition to the adjacent color stop's color through the adjacent color stop's first position.

```
<div class="multiposition-stops"></div>
<div class="multiposition-stop2"></div>
```

```
div {
  width: 120px;
  height: 120px;
  float: left;
  margin-right: 10px;
  box-sizing: border-box;
}
```

```
.multiposition-stops {
  background: linear-gradient(
    to left,
    lime 20%,
    red 30% 45%,
    cyan 55% 70%,
    yellow 80%
  );
}
.multiposition-stop2 {
  background: linear-gradient(
    to left,
    lime 25%,
    red 25% 50%,
    cyan 50% 75%,
    yellow 75%
  );
}
```

In the first example above, the lime goes from the 0% mark, which is implied, to the 20% mark, transitions from lime to red over the next 10% of the width of the gradient, reach solid red at the 30% mark, and staying solid red up until 45% through the gradient, where it fades to cyan, being fully cyan for 15% of the gradient, and so on.

In the second example, the second color stop for each color is at the same location as the first color stop for the adjacent color, creating a striped effect.

Controlling the progression of a gradient using color hints

By default, a gradient evenly progresses between the colors of two adjacent color stops, with the midpoint between those two color stops being the midpoint color value. You can control the interpolation, or progression, between two color stops by including a color hint location. In this example, the color reaches the midpoint between lime and cyan 20% of the way through the gradient rather than 50% of the way through. The second example does not contain the hint to highlight the difference the color hint can make:

```
<div class="color-hint-gradient"></div>
<div class="regular-progression"></div>
```

```
div {
  width: 120px;
  height: 120px;
  float: left;
  margin-right: 10px;
  box-sizing: border-box;
}
```

```
.color-hint-gradient {
  background: linear-gradient(to top, lime, 20%, cyan);
}
.regular-progression {
  background: linear-gradient(to top, lime, cyan);
}
```

Overlaying gradients

Gradients support transparency, so you can stack multiple backgrounds to achieve some pretty fancy effects. The backgrounds are stacked from top to bottom, with the first specified being on top.

```
<div class="layered-image"></div>
```

```
div {
  width: 300px;
  height: 150px;
}
```

```
.layered-image {
  background:
    linear-gradient(to right, transparent, mistyrose), url("critters.png");
}
```

Stacked gradients

You can even stack gradients with other gradients. As long as the top gradients aren't entirely opaque, the gradients below will still be visible.

```
<div class="stacked-linear"></div>
```

```
div {
  width: 200px;
  height: 200px;
}
```

```
.stacked-linear {
  background:
    linear-gradient(217deg, rgb(255 0 0 / 80%), transparent 70.71%),
    linear-gradient(127deg, rgb(0 255 0 / 80%), transparent 70.71%),
```

```
    linear-gradient(336deg, rgb(0 0 255 / 80%), transparent 70.71%);
}
```

Blending gradients

In addition to transparency, stacking multiple semi-transparent gradients and stacking gradients over raster background images, gradients can be used with other CSS effects. In this example, the four `<div>` elements have the same two fully-opaque gradients as background images. We apply different `background-blend-mode` CSS property values to the last three that blend the two background images creating different effects.

```
<div class="original"></div>
<div class="screen"></div>
<div class="overlay"></div>
<div class="difference"></div>
```

```
div {
  width: 120px;
  height: 120px;
  float: left;
  margin-right: 10px;
  box-sizing: border-box;
}
```

```
div {
  background:
    linear-gradient(to top, red, blue),
    linear-gradient(to right, #5500ff, #00ff55);
}

.screen {
  background-blend-mode: screen;
}

.overlay {
  background-blend-mode: overlay;
}

.difference {
  background-blend-mode: difference;
}
```

Using radial gradients

Radial gradients are similar to linear gradients, except that they radiate out from a central point. You can dictate where that central point is. You can also make them circular or elliptical.

A basic radial gradient

As with linear gradients, all you need to create a radial gradient are two colors. By default, the center of the gradient is at the 50% 50% mark, and the gradient is elliptical matching the aspect ratio of its box:

```
<div class="simple-radial"></div>
```

```
div {
  width: 240px;
  height: 120px;
}
```

```
.simple-radial {
  background: radial-gradient(red, blue);
}
```

Positioning radial color stops

Again like linear gradients, you can position each radial color stop with a percentage or absolute length.

```
<div class="radial-gradient"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.radial-gradient {
  background: radial-gradient(red 10px, yellow 30%, dodgerblue 50%);
}
```

Positioning the center of the gradient

You can position the center of the gradient with keyterms, percentage, or absolute lengths, length and percentage values repeating if only one is present, otherwise in the order of position from the left and position from the top.

```
<div class="radial-gradient"></div>
```

```
div {
  width: 120px;
  height: 240px;
}
```

```
.radial-gradient {
  background: radial-gradient(at 0% 30%, red 10px, yellow 30%, dodgerblue 50%);
}
```

Sizing radial gradients

Unlike linear gradients, you can specify the size of radial gradients. Possible values include `closest-corner`, `closest-side`, `farthest-corner`, and `farthest-side`, with `farthest-corner` being the default. Circles can also be sized with a length, and ellipses a length or percentage.

Example: `closest-side` for ellipses

This example uses the `closest-side` size value, which means the size is set by the distance from the starting point (the center) to the closest side of the enclosing box.

```
<div class="radial-ellipse-side"></div>
```

```
div {
  width: 240px;
  height: 100px;
}
```

```
.radial-ellipse-side {
  background: radial-gradient(
    ellipse closest-side,
    red,
    yellow 10%,
    dodgerblue 50%,
    beige
  );
}
```

Example: `farthest-corner` for ellipses

This example is similar to the previous one, except that its size is specified as `farthest-corner`, which sets the size of the gradient by the distance from the starting point to the farthest corner of the enclosing box from the starting point.

```
<div class="radial-ellipse-far"></div>
```

```
div {
  width: 240px;
  height: 100px;
}
```

```
.radial-ellipse-far {
  background: radial-gradient(
    ellipse farthest-corner at 90% 90%,
    red,
    yellow 10%,
    dodgerblue 50%,
    beige
  );
}
```

Example: `closest-side` for circles

This example uses `closest-side`, which makes the circle's radius to be the distance between the center of the gradient and the closest side. In this case the radius is the distance between the center and the bottom edge, because the gradient is placed 25% from the left and 25% from the bottom, and the div element's height is smaller than the width.

```
<div class="radial-circle-close"></div>
```

```
div {
  width: 240px;
  height: 120px;
}
```

```
.radial-circle-close {
  background: radial-gradient(
    circle closest-side at 25% 75%,
    red,
    yellow 10%,
    dodgerblue 50%,
    beige
  );
}
```

Example: length or percentage for ellipses

For ellipses only, you can size the ellipse using a length or percentage. The first value represents the horizontal radius, the second the vertical radius, where you use a percentage this corresponds to the size of the box in that dimension. In the below example I have used a percentage for the horizontal radius.

```
<div class="radial-ellipse-size"></div>
```

```
div {
  width: 240px;
  height: 120px;
}
```

```
.radial-ellipse-size {
  background: radial-gradient(
    ellipse 50% 50px,
    red,
    yellow 10%,
    dodgerblue 50%,
    beige
  );
}
```

Example: length for circles

For circles the size may be given as a `<length>`, which is the size of the circle.

```
<div class="radial-circle-size"></div>
```

```
div {
  width: 240px;
```

```
  height: 120px;
}
```

```
.radial-circle-size {
  background: radial-gradient(
    circle 50px,
    red,
    yellow 10%,
    dodgerblue 50%,
    beige
  );
}
```

Stacked radial gradients

Just like linear gradients, you can also stack radial gradients. The first specified is on top, the last on the bottom.

```
<div class="stacked-radial"></div>
```

```
div {
  width: 200px;
  height: 200px;
}
```

```
.stacked-radial {
  background:
    radial-gradient(circle at 50% 0, rgb(255 0 0 / 50%), transparent 70.71%),
    radial-gradient(circle at 6.7% 75%, rgb(0 0 255 / 50%), transparent 70.71%),
    radial-gradient(circle at 93.3% 75%, rgb(0 255 0 / 50%), transparent 70.71%)
      beige;
  border-radius: 50%;
}
```

Using conic gradients

The **`conic-gradient()`** CSS function creates an image consisting of a gradient with color transitions rotated around a center point (rather than radiating from the center). Example conic gradients include pie charts and color wheels, but they can also be used for creating checker boards and other interesting effects.

The conic-gradient syntax is similar to the radial-gradient syntax, but the color-stops are placed around a gradient arc, the circumference of a circle, rather than on the gradient line emerging from the center of the gradient, and the color-stops are percentages or degrees: absolute lengths are not valid.

In a radial gradient, the colors transition from the center of an ellipse, outward, in all directions. With conic gradients, the colors transition as if spun around the center of a circle, starting at the top and going clockwise. Similar to radial gradients, you can position the center of the gradient. Similar to linear gradients, you can change the gradient angle.

A basic conic gradient

As with linear and radial gradients, all you need to create a conic gradient are two colors. By default, the center of the gradient is at the 50% 50% mark, with the start of the gradient facing up:

```
<div class="simple-conic"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.simple-conic {
  background: conic-gradient(red, blue);
}
```

Positioning the conic center

Like radial gradients, you can position the center of the conic gradient with keyterms, percentage, or absolute lengths, with the keyword "at"

```
<div class="conic-gradient"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.conic-gradient {
  background: conic-gradient(at 0% 30%, red 10%, yellow 30%, dodgerblue 50%);
}
```

Changing the angle

By default, the different color stops you specify are spaced equidistantly around the circle. You can position the starting angle of the conic gradient using the "from" keyword at the beginning followed by an angle or a length, and you can specify different positions for the colors stops by including an angle or length after them.

```
<div class="conic-gradient"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.conic-gradient {
  background: conic-gradient(from 45deg, red, orange 50%, yellow 85%, green);
}
```

Using repeating gradients

The `linear-gradient()`, `radial-gradient()`, and `conic-gradient()` functions don't support automatically repeated color stops. However, the `repeating-linear-gradient()`, `repeating-radial-gradient()`, and `repeating-conic-gradient()` functions are available to offer this functionality.

The size of the gradient line or arc that repeats is the length between the first color stop value and the last color stop length value. If the first color stop just has a color and no color stop length, the value defaults to 0. If the last color stop has just a color and no color stop length, the value defaults to 100%. If neither is declared, the gradient line is 100% meaning the linear and conic gradients will not repeat and the radial gradient will only repeat if the radius of the gradient is smaller than the length between the center of the gradient and the farthest corner. If the first color stop is declared, and the value is greater than 0, the gradient will repeat, as the size of the line or arc is the difference between the first color stop and last color stop is less than 100% or 360 degrees.

Repeating linear gradients

This example uses `repeating-linear-gradient()` to create a gradient that progresses repeatedly in a straight line. The colors get cycled over again as the gradient repeats. In this case the gradient line is 10px long.

```
<div class="repeating-linear"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.repeating-linear {
  background: repeating-linear-gradient(
    -45deg,
    red,
    red 5px,
    blue 5px,
    blue 10px
  );
}
```

Multiple repeating linear gradients

Similar to regular linear and radial gradients, you can include multiple gradients, one on top of the other. This only makes sense if the gradients are partially transparent allowing subsequent gradients to show through the transparent areas, or if you include different background-sizes, optionally with different background-position property values, for each gradient image. We are using transparency.

In this case the gradient lines are 300px, 230px, and 300px long.

```
<div class="multi-repeating-linear"></div>
```

```
div {
  width: 600px;
  height: 400px;
}
```

```
.multi-repeating-linear {
  background:
    repeating-linear-gradient(
      190deg,
      rgb(255 0 0 / 50%) 40px,
      rgb(255 153 0 / 50%) 80px,
      rgb(255 255 0 / 50%) 120px,
      rgb(0 255 0 / 50%) 160px,
      rgb(0 0 255 / 50%) 200px,
      rgb(75 0 130 / 50%) 240px,
      rgb(238 130 238 / 50%) 280px,
      rgb(255 0 0 / 50%) 300px
    ),
    repeating-linear-gradient(
      -190deg,
      rgb(255 0 0 / 50%) 30px,
      rgb(255 153 0 / 50%) 60px,
      rgb(255 255 0 / 50%) 90px,
      rgb(0 255 0 / 50%) 120px,
      rgb(0 0 255 / 50%) 150px,
      rgb(75 0 130 / 50%) 180px,
      rgb(238 130 238 / 50%) 210px,
      rgb(255 0 0 / 50%) 230px
    ),
    repeating-linear-gradient(
      23deg,
      red 50px,
      orange 100px,
      yellow 150px,
      green 200px,
      blue 250px,
      indigo 300px,
      violet 350px,
      red 370px
    );
}
```

Plaid gradient

To create plaid we include several overlapping gradients with transparency. We use the multiple position color stop syntax:

```
<div class="plaid-gradient"></div>
```

```
div {
  width: 200px;
  height: 200px;
}
```

```
.plaid-gradient {
  background:
    repeating-linear-gradient(
      90deg,
      transparent 0 50px,
      rgb(255 127 0 / 25%) 50px 56px,
      transparent 56px 63px,
      rgb(255 127 0 / 25%) 63px 69px,
      transparent 69px 116px,
```

```
      rgb(255 206 0 / 25%) 116px 166px
    ),
    repeating-linear-gradient(
      0deg,
      transparent 0 50px,
      rgb(255 127 0 / 25%) 50px 56px,
      transparent 56px 63px,
      rgb(255 127 0 / 25%) 63px 69px,
      transparent 69px 116px,
      rgb(255 206 0 / 25%) 116px 166px
    ),
    repeating-linear-gradient(
      -45deg,
      transparent 0 5px,
      rgb(143 77 63 / 25%) 5px 10px
    ),
    repeating-linear-gradient(
      45deg,
      transparent 0 5px,
      rgb(143 77 63 / 25%) 5px 10px
    );
}
```

Repeating radial gradients

This example uses `repeating-radial-gradient()` to create a gradient that radiates repeatedly from a central point. The colors get cycled over and over as the gradient repeats.

```
<div class="repeating-radial"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.repeating-radial {
  background: repeating-radial-gradient(
    black,
    black 5px,
    white 5px,
    white 10px
  );
}
```

Multiple repeating radial gradients

```
<div class="multi-target"></div>
```

```
div {
  width: 250px;
  height: 150px;
}
```

```
.multi-target {
  background:
    repeating-radial-gradient(
        ellipse at 80% 50%,
        rgb(0 0 0 / 50%),
        rgb(0 0 0 / 50%) 15px,
        rgb(255 255 255 / 50%) 15px,
        rgb(255 255 255 / 50%) 30px
      )
      top left no-repeat,
    repeating-radial-gradient(
        ellipse at 20% 50%,
        rgb(0 0 0 / 50%),
        rgb(0 0 0 / 50%) 10px,
        rgb(255 255 255 / 50%) 10px,
        rgb(255 255 255 / 50%) 20px
      )
      top left no-repeat yellow;
  background-size:
    200px 200px,
    150px 150px;
}
```

Repeating conic gradients

This example uses `repeating-conic-gradient()` to create a gradient that rotates repeatedly around a center point. In this case, the declared color stops are repeated four times.

```
<div class="repeating-conic"></div>
```

```
div {
  width: 120px;
  height: 120px;
}
```

```
.repeating-conic {
  background: repeating-conic-gradient(
    #66ccff 0% 8.25%,
    #6633ff 8.25% 16.5%,
    #ff3399 16.5% 25%
  );
}
```

Multiple repeating conic gradients

Just like linear and radial repeating gradients, you can stack multiple conic gradients on top of each other, creating interesting effects by using different `at <position>` values so the conic gradients don't overlap at their centers and different `from <angle>` values so the repeating effects don't line up. This example overlaps three semi-transparent repeating radial gradients that each repeat their color schemes four times.

To make overlapping gradients visible, you need to ensure either that the colors of the gradients on the top of the stack are partially transparent or use the `background-blend-mode` CSS property.

```
<div class="multi-repeating-conic"></div>
```

```
div {
  width: 250px;
  height: 250px;
}
```

```
.multi-repeating-conic {
  background:
    repeating-conic-gradient(
      from 0deg at 80% 50%,
      #5691f580 0% 8.25%,
      #b338ff80 8.25% 16.5%,
      #f8305880 16.5% 25%
    ),
    repeating-conic-gradient(
      from 15deg at 50% 50%,
      #e856f580 0% 8.25%,
      #ff384c80 8.25% 16.5%,
      #e7f83080 16.5% 25%
    ),
    repeating-conic-gradient(
      from 0deg at 20% 50%,
      #f58356ff 0% 8.25%,
      #caff38ff 8.25% 16.5%,
      #30f88aff 16.5% 25%
    );
}
```

See also

- Gradient functions: `linear-gradient()`, `radial-gradient()`, `conic-gradient()`, `repeating-linear-gradient()`, `repeating-radial-gradient()`, `repeating-conic-gradient()`
- Gradient-related CSS data types: `<gradient>`, `<image>`
- Gradient-related CSS properties: `background`, `background-image`
- CSS Gradients Patterns Gallery, by Lea Verou
- Gradient CSS Generator
- Advanced CSS Gradient Generator
- HDR gradient generator

Using the CSS object-view-box property

The `object-view-box` property can be used to define a viewbox within replaced elements, enabling the display of just a section of the replaced content. The subsection of the element displayed can be presented as zoomed-in, zoomed-out, or at original size, while maintaining the content's intrinsic aspect ratio. In this guide, we examine this property, comparing it to the similar `object-fit` property, and explore its functionality through zooming in and out, as well as panning across an element.

Intrinsic size, extrinsic size, and <code>object-fit</code>

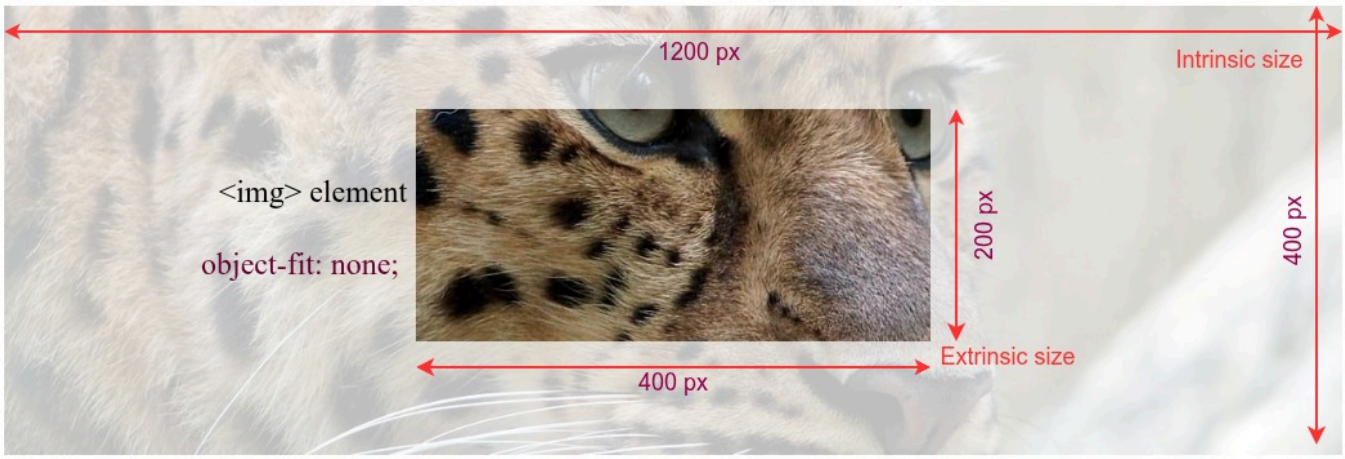
Every replaced elements has two sizes; an extrinsic size and an intrinsic size.

The extrinsic size is the dimension of the HTML element in which the content is rendered based on the box and visual formatting models. The box model and visual formatting model determine the size of rendered elements based on content, HTML attributes, CSS applied to the elements and their ancestors, and the viewport size.

The intrinsic size is the actual size of the content itself; the size the element is when no styles are applied and without any layout constraints. While the intrinsic and extrinsic sizes don't have to be the same, it is generally important to maintain a replaced element's intrinsic aspect ratio.

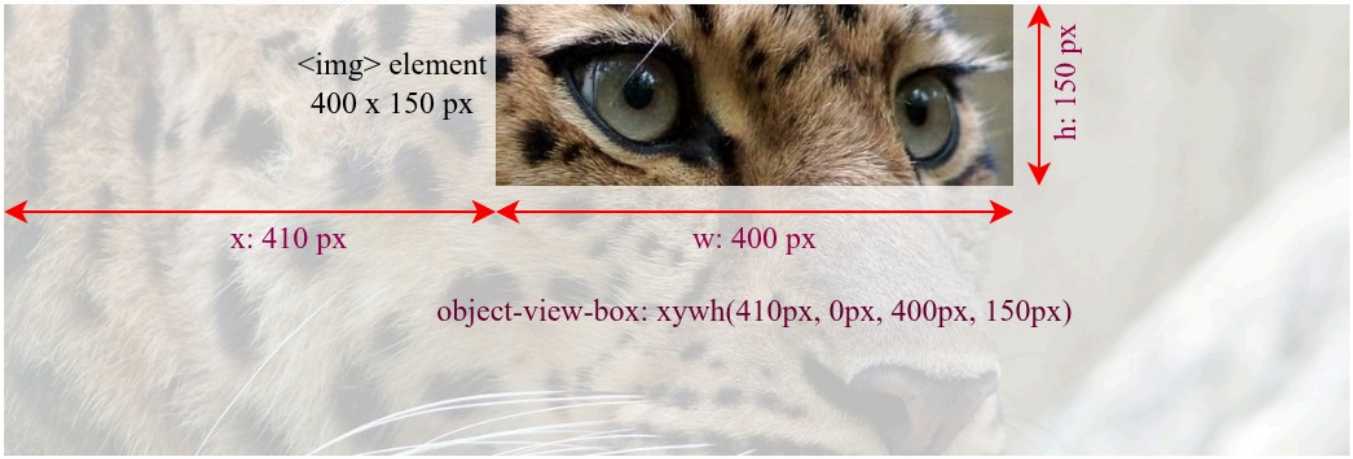
<code>object-view-box</code> versus <code>object-fit</code>

CSS has many sizing properties. When it comes to sizing replaced elements, the `object-fit` property enables us to control, to some extent, how replaced elements are rendered within a defined box. For example, in the following screenshot, a 1200 x 400 image is displayed using an `<img>` element. The `<img>` element is sized to 400 x 200. The image content is positioned using `object-fit: none;` declaration.



The `object-view-box` property is more flexible than the `object-fit` property, and it is capable of doing more things. For example, it can be used to crop, zoom, and pan images. The property sets the viewable area (viewbox), which defines what part of the content to show and how to fit it inside the extrinsic size. The viewbox value contains a rectangle and its position relative to the intrinsic area of the content, but the *physical size of the view box remains equal to the extrinsic size*. The viewbox marks the area in the content to be displayed, and then the content area is transformed to match the extrinsic dimensions fitting into the HTML element.

In the following image, we have the same leopard picture in a 400 x 150 image element. However, this time we have used `object-view-box` property to crop the portion of the picture showing the leopard's eyes.



In this case, as the dimensions of the `<img>` element and the viewbox defined by the `object-view-box` property are the same, i.e., 400 x 150 pixels, the aspect ratios of both are the same, and the replaced element is neither scaled nor distorted.

Maintaining the same aspect ratio prevents image distortion. With `object-view-box`, we can accomplish various image operations while having different extrinsic and view box sizes, without distorting the replaced element as it scales up and down.

Zooming in and out

Reducing the viewbox size, the area of the replaced element that is displayed, increases the zoom-in effect, because smaller content is stretched to fit the HTML element's dimensions. Decreasing it gives it a zoom-out effect.

This example demonstrates using the `object-view-box` property to zoom a section of a replaced element in and out, within a statically sized HTML element. In this case, the eye of the leopard, within a very large image, serves as the focal point of the zoom effect.

HTML

We include an `<img>` element and a `range` `<input>` element, with an associated `<label>`. The natural dimensions, or intrinsic size, of the original leopard image are `1244px` wide by `416px` tall, with an aspect ratio of `3:1`.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/leopard.jpg"
  alt="leopard" />
<p>
  <label for="box-size">Zoom-in: </label>
  <input type="range" id="box-size" min="115" max="380" value="150" />
</p>
<output></output>
```

CSS

We define a `--box-size` custom property, which is used as the height and width in the `xywh()` function, creating a square viewbox with an aspect ratio of `1:1`. The view box's offset point, the focal point in our zoom effect, is set at `500px` for the `x` coordinate and `30px` for the `y` coordinate, which corresponds to the top-left corner of the leopard's right eye.

```
input {
  width: 350px;
}

output {
  text-align: center;
  background-color: #dedede;
  font-family: monospace;
  padding: 5px;
  display: block;
}

@supports not (object-view-box: none) {
  body::before {
    content: "Your browser does not support the 'object-view-box' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
  }
}
```

```
img {
  width: 350px;
  height: 350px;
  border: 2px solid red;

  --box-size: 150px;
  object-view-box: xywh(500px 30px var(--box-size) var(--box-size));
}
```

JavaScript

We add an event listener to the slider that updates the value of the `--box-size` custom property when the user interacts with it. To increase the zoom-in effect when the slider is moved to the right, the slider's value is inverted by subtracting it from `500px`, as reducing the view box size increases the zoom-in effect.

```
const img = document.querySelector("img");
const zoom = document.getElementById("box-size");
const output = document.querySelector("output");
```

```
function update() {
  const size = 500 - zoom.value;
  img.style.setProperty("--box-size", `${size}px`);
  output.innerText = `object-view-box: xywh(500px 30px ${size}px ${size}px);`;
}

zoom.addEventListener("input", update);
update();
```

Result

Move the slider to the right to increase the zoom-in effect and left to reduce it. The slider only affects the dimensions of the view box, while the x and y values, the origin point of the viewbox, remain constant. The size of the `<img>` element also remains constant.

Panning an image

We can create a panning effect by changing the coordinates of the view box window, the `x` and `y` components of the `xywh()` function, while keeping the size of the visible section constant. For example, by keeping the view box dimensions constant and changing only the horizontal position - the `x` parameter - we can create a horizontal panning effect.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/leopard.jpg"
  alt="leopard" />
<p>
  <label for="position">Left offset: </label>
  <input type="range" id="position" min="0" max="900" value="450" />
</p>
<output></output>
```

```
input {
  width: 350px;
}

@supports not (object-view-box: none) {
  body::before {
    content: "Your browser does not support the 'object-view-box' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
  }
}
output {
  text-align: center;
  background-color: #dedede;
  font-family: monospace;
  padding: 5px;
  display: block;
}

img {
  width: 350px;
  height: 350px;

  --x-position: 0;
  object-view-box: xywh(var(--x-position) 30px 350px 350px);
}
```

```
const img = document.querySelector("img");
const position = document.getElementById("position");
const output = document.querySelector("output");

function update() {
  img.style.setProperty("--x-position", `${position.value}px`);
  output.innerText = `xywh(${position.value}px 30px 350px 350px);`;
}

position.addEventListener("input", update);
update();
```

Move the slider. Note how increasing and decreasing the `x` value of the `xywh()` function creates a panning effect.

See also

- `object-view-box`
- `object-fit`
- `object-position`
- `background-size`
- Understanding aspect ratio

CSS inline layout

The **CSS inline layout** module defines the block-axis alignment and sizing of inline-level content and adds a special layout mode for drop-caps. It describes the CSS formatting model for a flow of elements and text inside a container to be wrapped across multiple lines.

Reference

Properties

- `alignment-baseline`
- `baseline-shift`
- `baseline-source`
- `dominant-baseline`
- `initial-letter`
- `line-height`
- `text-box-edge`
- `text-box-trim`

- `text-box` shorthand
- `vertical-align`

The specification also defines the `initial-letter-align`, `initial-letter-wrap`, `inline-sizing`, and `line-fit-edge` properties, which are not yet supported by any browser.

Data types

- `<text-edge>`

Glossary terms

- baseline
- leading

Guides

Inline formatting context

Explains the inline formatting context.

Related concepts

- `font-size` property
- `font-feature-settings` property
- `letter-spacing` property
- `text-anchor` property
- `::first-letter` pseudo-element
- `:first-child` pseudo-class

See also

- CSS text module
- CSS fonts module
- CSS writing modes module

Inline formatting context

This guide explains the inline formatting context.

Core concepts

The inline formatting context is part of the visual rendering of a web page. Inline boxes are laid out one after the other, in the direction sentences run in the writing mode in use:

- In a horizontal writing mode, boxes are laid out horizontally, starting on the left.
- In a vertical writing mode they would be laid out vertically starting at the top.

In the example below, the two `<div>` elements with the black borders are part of a block formatting context, while inside each box, the words participate in an inline formatting context. The words in the horizontal writing mode run horizontally, while words in the vertical writing mode run vertically.

```
<div class="example horizontal">One Two Three</div>
<div class="example vertical">Four Five Six</div>
```

```
body {
  font: 1.2em sans-serif;
}
.example {
  border: 5px solid black;
  margin: 20px;
}

.horizontal {
  writing-mode: horizontal-tb;
}
.vertical {
  writing-mode: vertical-rl;
}
```

Boxes forming a line are contained by a rectangular area called a line box. This box will be large enough to contain all of the inline boxes in that line; when there is no more room in the inline direction another line will be created. Therefore, a paragraph is a set of inline line boxes, stacked in the block direction.

When an inline box is split, margins, borders, and padding have no visual effect where the split occurs. In the next example there is a `<span>` element wrapping a set of words wrapping onto two lines. The border on the `<span>` breaks at the wrapping point.

```
<div class="example">
  Before that night—
  <span
    >a memorable night, as it was to prove— hundreds of millions of people</span
  >
  had watched the rising smoke-wreaths of their fires without drawing any
  special inspiration from the fact.
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.example {
  border: 5px solid black;
  margin: 20px;
}

span {
  border: 5px solid rebeccapurple;
}
```

Margins, borders, and padding in the inline direction are respected. In the example below you can see how the margin, border, and padding on the inline `<span>` element are added.

```
<div class="example horizontal">One <span>Two</span> Three</div>
```

```
<div class="example vertical">Four <span>Five</span> Six</div>
```

```
body {
  font: 1.2em sans-serif;
}

.example {
  border: 5px solid black;
  margin: 20px;
}

span {
  border: 5px solid rebeccapurple;
  padding-inline-start: 20px;
  padding-inline-end: 40px;
  margin-inline-start: 30px;
  margin-inline-end: 10px;
}
.horizontal {
  writing-mode: horizontal-tb;
}

.vertical {
  writing-mode: vertical-rl;
}
```

> **Note:** I am using the logical, flow-relative properties — `padding-inline-start` rather than `padding-left` — so that they work in the inline dimension whether the text is horizontal or vertical. Read more about these properties in Logical Properties and Values.

Alignment in the block direction

Inline boxes may be aligned in the block direction in different ways, using the `vertical-align` property, which will align on the block axis in vertical writing modes (therefore not vertically at all!). In the example below the large text is making the line box of the first sentence larger, therefore the `vertical-align` property can be used to align the inline boxes either side of it. I have used the value `top`, try changing it to `middle`, `bottom`, or `baseline`.

```
<div class="example horizontal">
  Before that night—<span>a memorable night</span>, as it was to prove—hundreds
  of millions of people had watched the rising smoke-wreaths of their fires
  without drawing any special inspiration from the fact.
</div>

<div class="example vertical">
  Before that night—<span>a memorable night</span>, as it was to prove—hundreds
  of millions of people had watched the rising smoke-wreaths of their fires
  without drawing any special inspiration from the fact.
</div>
```

```
body {
  font: 1.2em sans-serif;
}

span {
  font-size: 200%;
  vertical-align: top;
}

.example {
  border: 5px solid black;
  margin: 20px;
  inline-size: 400px;
}

.horizontal {
  writing-mode: horizontal-tb;
}

.vertical {
  writing-mode: vertical-rl;
}
```

Alignment in the inline direction

If there is additional space in the inline direction, the `text-align` property can be used to align the inline boxes within their line box. Try changing the value of `text-align` below to `end`.

```
<div class="example horizontal">One Two Three</div>
<div class="example vertical">Four Five Six</div>
```

```
body {
  font: 1.2em sans-serif;
}

.example {
  border: 5px solid black;
  margin: 20px;
}

.horizontal {
  writing-mode: horizontal-tb;
}

.vertical {
  writing-mode: vertical-rl;
}
```

```
.example {
  text-align: center;
```

```
  inline-size: 250px;
}
```

Effect of floats

Line boxes usually have the same size in the inline direction, therefore the same width if working in a horizontal writing mode, or height if working in a vertical writing mode. If there is a `float` within the same block formatting context however, the float will cause the line boxes that wrap the float to become shorter.

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>I am content inside the container.</p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
}

.float {
  float: left;
  width: 250px;
  height: 150px;
  background-color: white;
  border: 1px solid black;
  padding: 10px;
}
```

See also

- Block formatting context
- Visual formatting model

CSS lists and counters

The **CSS lists and counters** module enables styling and positioning of list item bullets and manipulating their values with a combination of strings, counters, and other features.

A list item's marker, whether a bullet symbol or ordinal counter, is its defining feature. List items are not limited to `<li>` elements nested within `<ol>` or `<ul>` elements. Rather, list items are any element with `display: list-item` set.

This module defines CSS features to set and reset a list's counters, set which counter-styles or symbols to use as its markers, and position those markers. It also provides developers with the ability to create customized markers.

Reference

Properties

- `counter-increment`
- `counter-reset`
- `counter-set`
- `list-style-image`
- `list-style-type`
- `list-style-position`
- `list-style` shorthand

There is also a `marker-side` property, which is yet to be fully defined or implemented.

Pseudo-elements

- `::marker`

Functions

- `counter()`
- `counters()`

Data types

- `<counter>`
- `<counter-name>`
- `<counter-style>`

Guides

Consistent list indentation

Explains how to achieve consistent list indentation across different browsers.

Using CSS Counters

Explains how to use the CSS counter properties to control list counters.

Related concepts

- CSS counter styles
  - `@counter-style` at-rule
  - `<counter-style-name>` data type
  - `<symbol>` data type
  - `symbols()` function
- `<ol>` `start`, `reversed`, and `type` attributes
- `<ul>` `type` attribute

- `<li>` `type` and `value` attributes

See also

- CSS counter styles module
- CSS pseudo-elements module
- CSS generated content module

Consistent list indentation

One of the most common style changes made to lists is a change in the indentation distance—that is, how far the list items are pushed over to the right. This article will help you understand indenting list items so that list item markers are visible.

To understand why this is the case, and more importantly how to avoid the problem altogether, it's necessary to examine the details of list construction.

Creating a list

Stand-alone list item

First, we consider the pure list item, not nested in a list of items. When using the HTML `<li>` element, the browser sets the `display` value to `list-item`. Whether list items not nested in a list are provided a marker (otherwise known as a "bullet") depends on the browser. We can remove that bullet with `list-style-type: none`.

```
li {
  border: 1px dashed red;
}
li:nth-of-type(n + 4) {
  list-style-type: none;
}
```

```
li {
  width: 15em;
}
```

```
<p>Default bullets depend on the browser:</p>
<li>A list item</li>
<li>A list item</li>
<li>A list item</li>
<p>These list items have their bullets removed:</p>
<li>A list item</li>
<li>A list item</li>
<li>A list item</li>
```

That dotted red border represents the outer edges of the content area of each list item. At this point, the list items have no padding or borders.

Nested list items

Now we wrap these in a parent element; in this case, we'll wrap them in an unordered list (i.e., `<ul>`). According to the CSS box model, the list items' boxes must be displayed within the parent element's content area.

```
ul {
  border: 1px dashed blue;
}
li {
  border: 1px dashed red;
  list-style-type: none;
}
```

```
body {
  width: 15em;
}
```

```
<ul>
  <li>A list item</li>
  <li>A list item</li>
  <li>A list item</li>
</ul>
```

The dotted blue border shows us the edges of the `<ul>` element's content area. That parent comes with both margin and padding. Browsers set the following default styles on unordered lists:

```
ul {
  /* user-agent styles */
  display: block;
  list-style-type: disc;
  margin-block-start: 1em;
  margin-block-end: 1em;
  padding-inline-start: 40px;
}
```

Default bullet position

Now we put the list item markers back in. Since this is an unordered list, the list items inherit `list-style-type: disc;` browser styles, which are filled-circle "bullets," from their `<ul>` parent.

```
li {
  border: 1px dashed red;
}
ul {
  border: 1px dotted blue;
}
ul:last-of-type {
  list-style-position: inside;
}
```

```
ul {
  width: 15em;
}
```

```
<p>These default to <code>list-style-position: outside</code>.</p>
<ul>
  <li>A list item</li>
  <li>A list item</li>
  <li>A list item</li>
</ul>
<p>These have <code>list-style-position: inside</code> set.</p>
<ul>
  <li>A list item</li>
  <li>A list item</li>
  <li>A list item</li>
</ul>
```

Visually, the markers are *outside* the content area of the `<ul>`, but that's not the important part here. What's key is that the markers are placed outside the "principal box" of the `<li>` elements, not the `<ul>`. They're sort of like appendages to the list items, hanging outside the content-area of the `<li>` but still attached to the `<li>`.

This is why, in every modern browser, markers are placed outside any border set for an `<li>` element when the value of `list-style-position` defaults to or is explicitly set to `outside`. When we changed it to `inside`, the markers were brought inside the `<li>`'s content, as though they're an inline box placed at the very beginning of the `<li>`.

Default indentation

As noted above, all browsers provide the `<ul>` parent both margin and padding. While user agents CSS differ somewhat, they all include:

```
ul,
ol {
  /* user-agent styles */
  display: block;
  list-style-type: disc;
  margin-block-start: 1em;
  margin-block-end: 1em;
  padding-inline-start: 40px;
}
ol {
  list-style-type: decimal;
}
li {
  display: list-item;
  text-align: match-parent;
}
::marker {
  unicode-bidi: isolate;
  font-variant-numeric: tabular-nums;
  text-transform: none;
}
```

All browsers set `padding-inline-start` to 40 pixels for the `<ul>` element by default. In left-to-right languages, like English, this is the left *padding*. Any padding set in the author style sheets (that's your stylesheet) takes precedence.

If you want to be explicit, set the following in your style sheets to ensure, unless otherwise overridden, the list items in the main content area of your document, contained in the `<main>` section, are properly indented:

```
:where(main ol, main ul) {
  margin-inline-start: 0;
  padding-inline-start: 40px;
}
```

And always nest your `<li>` elements in a `<ul>` or `<ol>`.

CSS logical properties and values

The **CSS logical properties and values** module defines logical properties and values that can control layout through logical rather than physical direction and dimension mappings. Logical properties define direction-relative equivalents to their corresponding physical properties.

The start of a line is not always the left side of a line. Different writing systems operate in various directions. For example:

- English and Portuguese are written from left to right with new lines added below the previous ones.
- Hebrew and Arabic are right-to-left languages with new lines again being added below the previous ones.
- In some writing modes, the text lines are vertical, written from top to bottom. Chinese, Vietnamese, Korean, and Japanese are traditionally written vertically, from top to bottom, with each new vertical line added to the left of the previous one.
- Traditional Mongolian is also a top-to-bottom language, but new lines are to the right of previous ones.

The logical properties defined in this module enable defining properties relative to the content's writing direction, rather than a physical direction. This means content translated into languages with different writing modes will be rendered as intended.

Logical properties and values use the abstract terms *block* and *inline* to describe the direction in which they flow. The physical meaning of these terms depends on the writing mode.

The **block dimension** is perpendicular to the flow of text within a line, i.e., the vertical dimension in horizontal writing modes, and the horizontal dimension in vertical writing modes. For standard English text, it is the vertical dimension.

The **inline dimension** is parallel to the flow of text within a line, i.e., the horizontal dimension in horizontal writing modes, and the vertical dimension in vertical writing modes. For standard English text, it is the horizontal dimension.

CSS was initially designed with only physical coordinates. The logical properties and values module defines flow–relative equivalents for many values and properties. Properties that once only accepted physical values (`top`, `bottom`, `left`, `right`) now also accept flow-relative logical values (`block-start`, `block-end`, `inline-start`, `inline-end`).

Reference

Properties

- `block-size`
- `border-block`
- `border-block-color`
- `border-block-end`
- `border-block-end-color`
- `border-block-end-style`
- `border-block-end-width`
- `border-block-start`

- `border-block-start-color`
- `border-block-start-style`
- `border-block-start-width`
- `border-block-style`
- `border-block-width`
- `border-end-end-radius`
- `border-end-start-radius`
- `border-inline`
- `border-inline-color`
- `border-inline-end`
- `border-inline-end-color`
- `border-inline-end-style`
- `border-inline-end-width`
- `border-inline-start`
- `border-inline-start-color`
- `border-inline-start-style`
- `border-inline-start-width`
- `border-inline-style`
- `border-inline-width`
- `border-start-end-radius`
- `border-start-start-radius`
- `inline-size`
- `inset`
- `inset-block`
- `inset-block-end`
- `inset-block-start`
- `inset-inline`
- `inset-inline-end`
- `inset-inline-start`
- `margin-block`
- `margin-block-end`
- `margin-block-start`
- `margin-inline`
- `margin-inline-end`
- `margin-inline-start`
- `max-block-size`
- `max-inline-size`
- `min-block-size`
- `min-inline-size`
- `padding-block`
- `padding-block-end`
- `padding-block-start`
- `padding-inline`
- `padding-inline-end`
- `padding-inline-start`

Data types and values

Flow relative values:

- `block-start`
- `block-end`
- `inline-start`
- `inline-end`
- `start`
- `end`

Glossary terms

- Flow relative values
- Inset properties
- Logical properties
- Physical properties

Guides

Basic concepts of logical properties and values

Overview of flow relative properties and values.

Logical properties for sizing

Flow-relative mappings between physical properties and logical properties used for sizing elements on the page.

Logical properties for margins, borders, and padding

Flow-relative mappings for the various margin, border, and padding properties and their shorthands.

Logical properties for floating and positioning

Details mappings between the physical and logical values for `float` and `clear`, inset properties, and `resize`.

Related concepts

- `caption-side`
- `clear`
- `float`
- `resize`
- `text-align`

CSS box model

- `margin` shorthand
- `padding` shorthand

CSS box sizing

- `max-height`
- `max-width`
- `min-height`
- `min-width`

CSS backgrounds and borders

- `border-color`
- `border-style`
- `border-width`
- `border` shorthand
- `border-radius`

CSS positioned layout

- `top`
- `right`
- `bottom`
- `left`

CSS writing modes

- `direction`
- `text-orientation`
- `writing-mode`

CSS containment

- `contain-intrinsic-block-size`
- `contain-intrinsic-inline-size`

CSS overflow

- `overflow-block`
- `overflow-inline`

CSS overscroll behavior

- `overscroll-behavior-block`
- `overscroll-behavior-inline`

See also

- Flow layout and writing modes
- CSS flexible box layout module
- CSS grid layout module

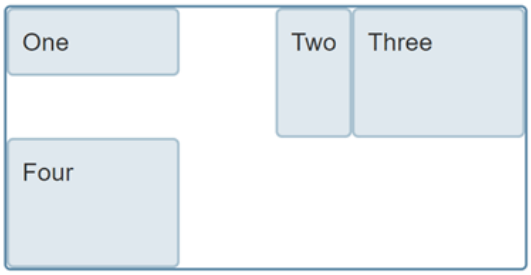
Basic concepts of logical properties and values

The CSS logical properties and values module defines flow-relative mappings for many of the physical properties and values in CSS. This article discusses this module, and explains flow relative values and properties.

Why are logical properties useful

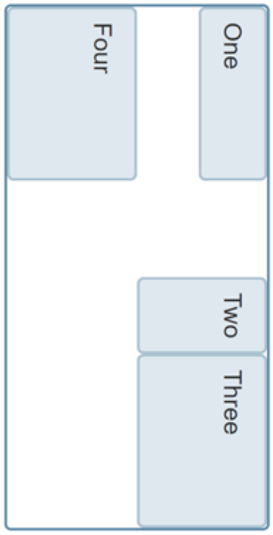
CSS 2.1 and earlier had sized things according to the physical dimensions of the screen. Therefore we describe boxes as having a `width` and `height`, position items from the `top` and `left`, assign borders, margin, and padding to the `top`, `right`, `bottom`, `left`, etc. The Logical properties and values module defines mappings for these physical properties and values to their logical, or flow relative, counterparts — e.g., `start` and `end` as opposed to `left` and `right`/`top` and `bottom`.

These mappings are very useful for sites that get translated into languages with a different writing mode than the original layout. For example, with a CSS grid layout, if the grid container has a width applied with the `align-self` and `justify-self` properties used to align the grid items, as these properties are flow relative, the `justify-self: start` aligns the item to the start on the inline dimension, and `align-self: start` does the same on the block dimension.



If the writing mode of this component is changed to `vertical-rl` using the `writing-mode` property, the alignment continues to work in the same way. The inline dimension will run vertically and the block dimension horizontally. The grid doesn't look the same, however, as the width assigned to the container is a horizontal measure, a measure tied to the physical and not the logical or flow relative running of the text.

If instead of the `width` property, the logical property `inline-size` is used, the component will work the same way no matter which writing mode it is displayed using.



You can try this out in the live example below. Change `writing-mode` from `vertical-rl` to `horizontal-tb` on `.grid` to see how the different properties change the layout.

```
<div class="grid">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
}

.grid > * {
  border-radius: 5px;
  border: 2px solid rgb(96 139 168 / 0.4);
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
}

.grid :nth-child(1) {
  align-self: start;
}

.grid :nth-child(2) {
  justify-self: end;
}
```

```
.grid {
  writing-mode: vertical-rl;
  inline-size: 400px;
}
```

When working with a site in a writing mode other than a horizontal, top-to-bottom one, or when using writing modes for creative reasons, being able to relate to the flow of the content makes a lot of sense.

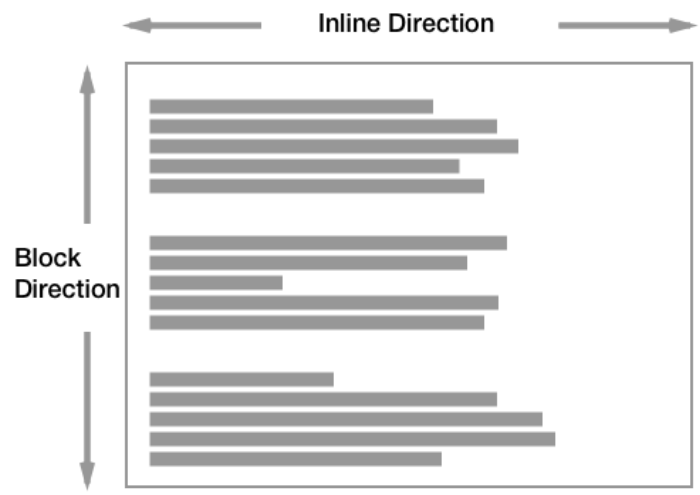
Block and inline dimensions

A key concept of working with flow relative properties and values is the two dimensions of block and inline. CSS layout methods such as flexbox and grid layout use the concepts of `block` and `inline` rather than `right` and `left`/`top` and `bottom` when aligning items.

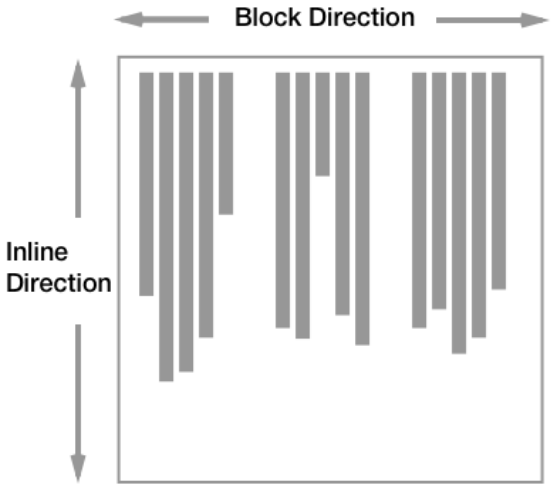
The `inline` dimension is the dimension along which a line of text runs in the writing mode in use. Therefore, in an English document with the text running horizontally left-to-right, or an Arabic document with the text running horizontally right-to-left, the inline dimension is *horizontal*. Switch to a vertical writing mode (e.g., a Japanese document) and the inline dimension is now *vertical*, as lines in that writing mode run vertically.

The block dimension is the other dimension, and the direction in which blocks — such as paragraphs — display one after the other. In English and Arabic, these run vertically, whereas in any vertical writing mode these run horizontally.

The below diagram shows the inline and block directions in a horizontal writing mode:



This diagram shows block and inline in a vertical writing mode:



See also

- Box alignment in grid layout
- Box alignment in flex layout
- Flow layout and writing modes
- Understanding logical properties and values on Smashing Magazine (2018)

Logical properties for floating and positioning

The CSS logical properties and values module contains logical mappings for the physical values of `float` and `clear`, and also for the positioning properties used with positioned layout. This guide takes a look at how to use these.

Mapped properties and values

The table below details the logical properties and values discussed in this guide along with their physical properties and values mappings. They assume a horizontal `writing-mode`, with a left-to-right direction.

Logical property or value	Physical property or value
`float`: inline-start	`float`: left
`float`: inline-end	`float`: right
`clear`: inline-start	`clear`: left
`clear`: inline-end	`clear`: right
`inset-inline-start`	`left`
`inset-inline-end`	`right`
`inset-block-start`	`top`
`inset-block-end`	`bottom`
`text-align`: start	`text-align`: left
`text-align`: end	`text-align`: right

In addition to these mapped properties, there are some additional shorthand properties made possible by being able to address block and inline dimensions. These have no mapping to physical properties, aside from the `inset` property.

Logical property	Purpose
`inset-inline`	Sets both of the above inset values for the inline dimension simultaneously.
`inset-block`	Sets both of the above inset values for the block dimension simultaneously.
`inset`	Sets all four inset values simultaneously with physical mapping of multi-value.

Float and clear example

The physical values used with the `float` and `clear` properties are `left`, `right` and `both`. The CSS logical properties and values module defines the values `inline-start` and `inline-end` as mappings for `left` and `right`.

In the example below, the first box is floated with `float: left`, and the second with `float: inline-start`. If you apply `direction: rtl` to the `.inner` selector, the left-floated box always stays on the left, whereas the `inline-start`-floated item follows the `direction` of the text. You can combine this with `writing-mode: vertical-rl` to see the difference of block layout in combination with `direction` values.

```
<div class="container">
  <div class="inner">
    <div class="physical box"></div>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale.
  </div>
  <div class="inner">
    <div class="logical box"></div>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale.
  </div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.container {
  display: flex;
}

.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  margin: 10px;
  width: 100px;
  height: 100px;
}
```

```
.inner {
  /* direction: rtl; */
  /* writing-mode: vertical-rl; */
}

.physical {
  float: left;
}

.logical {
  float: inline-start;
}
```

Example: Inset properties for positioned layout

CSS positioning generally allows us to position an element in a manner relative to its containing block — we essentially inset the item relative to where it would fall based on normal flow. To position an element relative to the viewport, use the physical properties `top`, `right`, `bottom` and `left`. When you want the positioning to relate to the flow of text in your writing mode, use `inset-block-start`, `inset-block-end`, `inset-inline-start` and `inset-inline-end` instead.

These properties take a length or a percentage as a value, and relate to the user's screen dimensions.

In the below example, the `inset-block-start` and `inset-inline-end` properties are used to position the blue box using absolute positioning inside the area with the gray dotted border, which has `position: relative`. Change the `writing-mode` property to `vertical-rl`, or add `direction: rtl`, and see how the flow relative box stays with the text direction.

```
<div class="container">
  <div class="inner">
    <div class="physical box"></div>
  </div>
  <div class="inner">
    <div class="logical box"></div>
  </div>
</div>
```

```
.container {
  display: flex;
}

.inner {
  width: 200px;
  height: 200px;
  position: relative;
  border: 2px dotted grey;
}

.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  width: 100px;
```

```
  height: 100px;
}
```

```
.inner {
  writing-mode: horizontal-tb;
}

.physical {
  position: absolute;
  top: 20px;
  right: 0;
}

.logical {
  position: absolute;
  inset-block-start: 20px;
  inset-inline-end: 0;
}
```

New two- and four-value shorthands

As with other properties in the module, we have shorthand properties which give the ability to set two or four values at once.

- `inset` — sets all four sides together with physical mapping.
- `inset-inline` — sets both logical inline insets.
- `inset-block` — sets both logical block insets.

Example: Logical values for text-align

The `text-align` property has logical values that relate to text direction — rather than using `left` and `right` you can use `start` and `end`. In the below example, `text-align: right` is set in the first block and `text-align: end` in the second.

If you change the value of `direction` to `rtl`, you will see that the alignment stays to the right for the first block, but goes to the logical end on the left in the second.

```
<div class="container">
  <div class="inner physical">Aligned text</div>
  <div class="inner logical">Aligned text</div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  display: flex;
}

.inner {
  width: 200px;
  border: 2px dotted grey;
  padding: 10px;
}
```

```
.inner {
  direction: ltr;
}

.physical {
  text-align: right;
}

.logical {
  text-align: end;
}
```

This works more consistently when using box alignment that uses start and end rather than physical directions for alignment.

Logical properties for margins, borders, and padding

The CSS logical properties and values module defines flow-relative mappings for the various margin, border, and padding properties and their shorthands. In this guide, we take a look at these.

If you look at the logical properties and values module, you may notice the list of module properties is very long. This is mostly because there are four longhand values each for margin, border, and padding side, plus all the shorthand values.

Mappings for margins, borders, and padding

The module details mappings for each logical value to a physical counterpart. The table below maps these values for when the `writing-mode` is `horizontal-tb` — with a left to right direction. The inline direction therefore runs horizontally — left to right — and `margin-inline-start` would be equivalent to `margin-left`.

If you were using a `horizontal-tb` writing mode with a right-to-left text direction then `margin-inline-start` would be the same as `margin-right`, and in a vertical writing mode it would be the same as using `margin-top`.

Logical property	Physical property
`border-block-end`	`border-bottom`
`border-block-end-color`	`border-bottom-color`
`border-block-end-style`	`border-bottom-style`
`border-block-end-width`	`border-bottom-width`
`border-block-start`	`border-top`
`border-block-start-color`	`border-top-color`
`border-block-start-style`	`border-top-style`
`border-block-start-width`	`border-top-width`
`border-inline-end`	`border-right`

Logical property	Physical property
`border-inline-end-color`	`border-right-color`
`border-inline-end-style`	`border-right-style`
`border-inline-end-width`	`border-right-width`
`border-inline-start`	`border-left`
`border-inline-start-color`	`border-left-color`
`border-inline-start-style`	`border-left-style`
`border-inline-start-width`	`border-left-width`
`border-start-start-radius`	`border-top-left-radius`
`border-end-start-radius`	`border-bottom-left-radius`
`border-start-end-radius`	`border-top-right-radius`
`border-end-end-radius`	`border-bottom-right-radius`
`margin-block-end`	`margin-bottom`
`margin-block-start`	`margin-top`
`margin-inline-end`	`margin-right`
`margin-inline-start`	`margin-left`
`padding-block-end`	`padding-bottom`
`padding-block-start`	`padding-top`
`padding-inline-end`	`padding-right`
`padding-inline-start`	`padding-left`

There are also some additional shorthands, made possible because we can target both block or both inline edges of the box simultaneously. These shorthands have no physical equivalent.

Property	Purpose
`border-block`	Sets `border-color`, `border-style`, and `border-width` for both block borders.
`border-block-color`	Sets `border-color` for both block borders.
`border-block-style`	Sets `border-style` for both block borders.
`border-block-width`	Sets `border-width` for both block borders.
`border-inline`	Sets `border-color`, `-style`, and `-width` for both inline borders.
`border-inline-color`	Sets `border-color` for both inline borders.
`border-inline-style`	Sets `border-style` for both inline borders.
`border-inline-width`	Sets `border-width` for both inline borders.
`margin-block`	Sets all the block `margin`s.
`margin-inline`	Sets all the inline `margin`s.
`padding-block`	Sets the block `padding`.
`padding-inline`	Sets the inline `padding`.

Margin examples

The mapped margin properties of `margin-inline-start`, `margin-inline-end`, `margin-block-start`, and `margin-inline-end` can be used instead of their physical counterparts.

This example has two boxes with different sized margins to each edge. An extra container with a border has been included to make the margin more apparent.

One box uses physical properties and the other logical properties. Try changing the `direction` property to `rtl` to cause the boxes to display in a right-to-left direction; the margins on the first box will stay in the same place, while the margins on the inline dimension of the second box will switch.

Also try changing the `writing-mode` from `horizontal-tb` to `vertical-rl`. Notice how the margins stay in the same place for the first box, but switch around to follow the text direction in the second.

```
<div class="container">
  <div class="inner">
    <div class="physical box">
      margin-top: 5px<br />
      margin-right: 0<br />
      margin-bottom: 2em<br />
      margin-left: 50px
    </div>
  </div>
  <div class="inner">
    <div class="logical box">
      margin-block-start: 5px<br />
      margin-inline-end: 0<br />
      margin-block-end: 2em<br />
      margin-inline-start: 50px
    </div>
  </div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.container {
  display: flex;
}
.inner {
  border: 2px dotted grey;
}
.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
```

```
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  width: 220px;
  height: 220px;
}
```

```
.box {
  writing-mode: horizontal-tb;
  direction: ltr;
}

.physical {
  margin-top: 5px;
  margin-right: 0;
  margin-bottom: 2em;
  margin-left: 50px;
}

.logical {
  margin-block-start: 5px;
  margin-inline-end: 0;
  margin-block-end: 2em;
  margin-inline-start: 50px;
}
```

Margin shorthands

There are shorthands available to target either both the inline sides or both the block sides, `margin-inline` and `margin-block` respectively. Each accepts two values. The first value will apply to the start of that dimension, the second to the end. If only one value is set, it is applied to both.

In a horizontal writing mode this CSS would apply a `5px` margin to the top of the box and a `10px` margin to the bottom.

```
.box {
  margin-block: 5px 10px;
}
```

Padding examples

The mapped padding properties of `padding-inline-start`, `padding-inline-end`, `padding-block-start`, and `padding-inline-end` can be used instead of their physical counterparts.

In this example, there are two boxes. One has physical padding properties set and the other uses logical padding properties. With a `writing-mode` of `horizontal-tb`, both boxes should appear the same.

Try changing the `direction` property to `rtl` to cause the boxes to display in a right-to-left direction. The padding on the first box will stay in the same place, whereas the padding on the inline dimension of the second box will switch.

You can also try changing the `writing-mode` from `horizontal-tb` to `vertical-rl`. Again, notice how the padding stays in the same place for the first box, but switches around to follow the text direction in the second.

```
<div class="container">
  <div class="physical box">
    padding-top: 5px<br />
    padding-right: 0<br />
    padding-bottom: 2em<br />
    padding-left: 50px
  </div>

  <div class="logical box">
    padding-block-start: 5px<br />
    padding-inline-end: 0<br />
    padding-block-end: 2em<br />
    padding-inline-start: 50px
  </div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  display: flex;
}
.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  margin: 10px;
  width: 220px;
  height: 220px;
}
```

```
.box {
  writing-mode: horizontal-tb;
  direction: ltr;
}

.physical {
  padding-top: 5px;
  padding-right: 0;
  padding-bottom: 2em;
  padding-left: 50px;
}

.logical {
  padding-block-start: 5px;
  padding-inline-end: 0;
  padding-block-end: 2em;
  padding-inline-start: 50px;
}
```

Padding shorthands

As with margin, there are two-value shorthands for padding — `padding-inline` and `padding-block` — which allow you to set the padding of the two inline, and two block dimensions, respectively.

In a horizontal `writing-mode`, this CSS would apply `5px` of padding to the top of the box and `10px` of padding to the bottom:

```
.box {
  padding-block: 5px 10px;
}
```

Border examples

The border properties are the main reason that this module seems to have so many properties, as it provides longhand logical properties for the color, width, and style of the border on each side of a box, along with the shorthand to set all three at once for each side. As with margin and padding, there is a mapped version of each physical property.

The demo below uses some longhands and three shorthand values. As with the other demos, try changing the `direction` property to `rtl` to cause the boxes to display in a right-to-left direction, or changing the `writing-mode` from `horizontal-tb` to `vertical-rl`.

```
<div class="container">
  <div class="physical box">Borders using physical properties.</div>
  <div class="logical box">Borders using logical properties.</div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  display: flex;
}
.box {
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  margin: 10px;
  width: 220px;
  height: 220px;
}
```

```
.box {
  writing-mode: horizontal-tb;
  direction: ltr;
}

.physical {
  border-top: 2px solid hotpink;
  border-right-style: dotted;
  border-right-color: goldenrod;
  border-right-width: 5px;
  border-bottom: 4px double black;
  border-left: none;
}

.logical {
  border-block-start: 2px solid hotpink;
  border-inline-end-style: dotted;
  border-inline-end-color: goldenrod;
  border-inline-end-width: 5px;
  border-block-end: 4px double black;
  border-inline-start: none;
}
```

Border shorthands

There are two-value shorthands to set the width, style, and color of the block or inline dimension, and shorthands to set all three values in the block or inline dimension. The below code, in a horizontal writing mode, would give you a `2px green solid` border on the top and bottom of the box, and a `4px dotted purple` border on the left and right.

```
.box {
  border-block: 2px solid green;
  border-inline-width: 4px;
  border-inline-style: dotted;
  border-inline-color: rebeccapurple;
}
```

Flow relative border-radius properties

The module has flow-relative equivalents for the `border-radius` longhands. The below example, in a horizontal `writing-mode`, would set the top-right border radius to `1em`, the bottom-right to `0`, the bottom-left to `20px` and the top-left to `40px`.

```
.box {
  border-end-start-radius: 1em;
  border-end-end-radius: 0;
  border-start-end-radius: 20px;
  border-start-start-radius: 40px;
}
```

Indicating logical values for the 4-value shorthand syntax

The specification makes a suggestion for the four-value shorthands such as the `margin` property, however the final decision on how this should be indicated is as yet unresolved, and is discussed in this issue.

Using any four-value shorthand such as `margin`, `padding`, or `border` will currently use the physical versions, so if following the flow of the document is important, use the longhand properties for the time being.

Logical properties for sizing

In this guide, we will explain the flow-relative mappings between physical dimension properties and logical properties used for sizing elements on our pages.

When specifying the size of an item, the CSS logical properties and values module gives you the ability to indicate sizing as it relates to the flow of text (inline and block) rather than physical sizing which relates to the physical dimensions of horizontal and vertical (e.g., left and right). While these flow relative mappings may well become the default for many of us, in a design you may well use both physical and logical sizing. You might want some features to always relate to the physical dimensions whatever the writing mode.

Mappings for dimensions

The table below provides mappings between logical and physical properties. These mappings assume that you are in a `horizontal-tb` writing mode, such as English or Arabic, in which case `width` would be mapped to `inline-size`.

If you were in a vertical writing mode then `inline-size` would be mapped to `height`.

Logical Property	Physical Property
`inline-size`	`width`
`block-size`	`height`
`min-inline-size`	`min-width`
`min-block-size`	`min-height`
`max-inline-size`	`max-width`
`max-block-size`	`max-height`

Width and height example

The logical mappings for `width` and `height` are `inline-size`, which sets the length in the inline dimension and `block-size`, which sets the length in the block dimension. When working in English, replacing `width` with `inline-size` and `height` with `block-size` will give the same layout.

In the live example below, the `writing-mode` is set to `horizontal-tb`. Change it to `vertical-rl` and you will see that the first example — which uses `width` and `height` — remains the same size in each dimension, despite the text becoming vertical. The second example — which uses `inline-size` and `block-size` — will follow the text direction as if the entire block has rotated.

```
<div class="container">
  <div class="physical box">I have a width of 200px and a height of 100px.</div>

  <div class="logical box">
    I have an inline-size of 200px and a block-size of 100px.
  </div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.container {
  display: flex;
}
.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  margin: 10px;
}
```

```
.box {
  writing-mode: horizontal-tb;
}

.physical {
  width: 200px;
  height: 100px;
}

.logical {
  inline-size: 200px;
  block-size: 100px;
}
```

Min-width and min-height example

There are also mappings for `min-width` and `min-height` — these are `min-inline-size` and `min-block-size`. These work in the same way as the `inline-size` and `block-size` properties, but setting a minimum rather than a fixed size.

Try changing the example below to `vertical-rl`, as with the first example, to see the effect it has. I am using `min-height` in the first example and `min-block-size` in the second.

```
<div class="container">
  <div class="physical box">
    I have a width of 200px and a min-height of 5em.
  </div>

  <div class="logical box">
    I have an inline-size of 200px and a min-block-size of 5em.
  </div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.container {
  display: flex;
}
.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  margin: 10px;
}
```

```
.box {
  writing-mode: horizontal-tb;
}

.physical {
```

```
  width: 200px;
  min-height: 5em;
}

.logical {
  inline-size: 200px;
  min-block-size: 5em;
}
```

Max-width and max-height example

Finally you can use `max-inline-size` and `max-block-size` as logical replacements for `max-width` and `max-height`. Try playing with the below example in the same way as before.

```
<div class="container">
  <div class="physical box">I have a max-width of 200px.</div>

  <div class="logical box">I have a max-inline-size of 200px.</div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.container {
  display: flex;
}
.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  margin: 10px;
}
```

```
.box {
  writing-mode: horizontal-tb;
}

.physical {
  max-width: 200px;
}

.logical {
  max-inline-size: 200px;
}
```

Logical keywords for resize

The `resize` property sets whether or not an item can be resized and has physical values of `horizontal` and `vertical`. The `resize` property also has logical keyword values. Using `resize: inline` allows resizing in the inline dimension and `resize: block` allow resizing in the block dimension.

The keyword value of `both` for the resize property works whether you are thinking physically or logically. It sets both dimensions at once. Try playing with the below example.

```
<div class="container">
  <div class="physical box">
    I have a width of 200px and a height of 100px. I can be resized
    horizontally.
  </div>

  <div class="logical box">
    I have an inline-size of 200px and a block-size of 100px. I can be resized
    in the inline direction.
  </div>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.container {
  display: flex;
}
.box {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
  padding: 10px;
  margin: 10px;
}
```

```
.box {
  writing-mode: horizontal-tb;
  overflow: scroll;
}

.physical {
  width: 200px;
  height: 100px;
  resize: horizontal;
}

.logical {
  inline-size: 200px;
  block-size: 100px;
  resize: inline;
}
```

CSS masking

The **CSS masking** module defines masking and clipping, two different graphical operations that are used to partially or fully hide portions of visual elements.

Clipping involves defining a closed vector path, shape, or polygon as a **clipping path**. Everything inside the clipping path region remains visible while everything outside is hidden, or "clipped out". The `clip-path` property specifies a `<basic-shape>` or references an SVG `<clipPath>` element to be used as a clipping path.

CSS **masking** properties are used to apply a mask to an element or its border. A graphical object is then painted onto the background or border, completely or partially masking out parts of the element or its border, depending on the opacity or luminance of the mask.

The image used as the mask is specified by the `mask-image` or `mask-border-source` properties. The specified mask can be an `<image>`, a `<gradient>`, or an SVG `<mask>` element. The mask can be sized and positioned similarly to background and border images.

Clipping and masking in CSS behaves the same way as it does with SVG: First, the element is styled without filter effects, masking, clipping, and opacity. Then, any effects are applied to the element in the following order: filter effects, clipping, masking, and opacity.

While masking provides more control and options, clipping can perform better if a basic shape is all that's required — they are easier to interpolate.

Reference

Properties

- `clip`
- `clip-path`
- `clip-rule`
- `mask` shorthand
- `mask-clip`
- `mask-composite`
- `mask-image`
- `mask-mode`
- `mask-origin`
- `mask-position`
- `mask-repeat`
- `mask-size`
- `mask-type`
- `mask-border` shorthand
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-slice`
- `mask-border-source`
- `mask-border-width`
- `mask-border-mode`

Data types

- `<geometry-box>`

Functions

- `rect()` function

Interfaces

- `SVGClipPathElement`
- `SVGMaskElement`
  - `SVGMaskElement.maskContentUnits`

Guides

Introduction to CSS clipping

Introduction to clipping in CSS, including the `clip-path` property with examples.

Introduction to CSS masking

Introduction to masking in CSS, the various mask image types, and the effects of luminance and alpha-transparency in masking.

Declaring multiple masks

Introduction to mask layers and how to declare multiple mask images.

CSS mask properties

An exploration of CSS masking and the `mask` shorthand component properties, with explanations and examples.

Related concepts

- `<coord-box>`
- `<image>`
- `<position>`
- `<url>`
- CSS backgrounds and borders module
  - `background` shorthand
  - `background-origin`
  - `background-position`
  - `background-repeat`
  - `background-size`
  - `border-image` shorthand
  - `border-image-repeat`
  - `border-image-slice`
  - `border-image-source`
  - `border-image-width`
  - `<repeat-style>` data type
- CSS shapes module
  - `<basic-shape>` data type

- `polygon()` function
- `<shape-box>` data type

See also

- `background-clip`
- CSS filter effects module
- SVG tutorial: clipping and masking
- `CanvasRenderingContext2D.clip()`
- `WebGLRenderingContext.colorMask()`
- PWA icon masking

Introduction to CSS clipping

CSS clipping enables you to define visible portions of an element while hiding other parts, effectively "clipping" its content within a specific shape or area. With clipping, elements aren't limited to being rendered as rectangles and can be designed in visually engaging ways. This guide explores the `clip-path` property along with some examples.

CSS clipping

Clipping is a CSS technique used to clip (hide) sections of an element, displaying only the area of the element located within a developer-defined path. Clips areas are created by vector paths; anything in the path is visible while areas outside the path are hidden.

The <code>clip-path</code> property

The `clip-path` property applies clipping. The value it accepts is a vector path defining the area of the element that should remain visible. The path can be defined using boxes, a reference to an SVG `<clipPath>`, or CSS shapes and paths. In the following example, we clip a blue square `<div>`, creating a diamond, using the `polygon()` function as the clipping path:

```
<div class="diamond"></div>
```

```
.diamond {
  height: 200px;
  width: 200px;
  background-color: blue;

  clip-path: polygon(0 50%, 50% 100%, 100% 50%, 50% 0);
}
```

With the `clip-path` property, you can make complex shapes by clipping an element to a `<basic-shape>` or to an SVG source. You can animate and transition `clip-path` shapes if the declared states have the same number of vector points.

Values of the <code>clip-path</code> property

To visually clip an element, the `clip-path` property is set to either a `<geometry-box>`, a `url` to a `<clipPath>` clip source, or a `<basic-shape>` created with shape function.

Geometry boxes

The `clip-path` hides everything outside of the clipped region. The most basic clipping is done via a geometry box. You can clip an element based on its margin, border, padding, or content. The effects of these visual box values can be achieved via other CSS properties, such as setting the `border-color` to transparent and the `background-origin` to the desired visual box. We're looking at these values mostly because these values are used in conjunction with the shape functions, which we'll look at later, to define the origin of the shape clip path.

Understanding the reference box used by CSS shapes is important when using `clip-path`, especially with basic shapes, as the reference box defines a shape's coordinate system.

Visual box values

This live example demonstrates the `clip-path` property's different visual box values on an element, while comparing it to the CSS `background-origin` property. We've applied a `border`, a `background-color`, a `background-image`, and `padding` to the `<blockquote>`. Select a radio button to update the `--value` to a different `<geometry-box>` value, which updates the `background-origin` and the `clip-path` resolved values.

```
body {
  display: flex;
  flex-flow: row wrap;
  place-content: center;
}
blockquote {
  float: left;
  font-size: 1.2rem;
}
q {
  color: white;
  font-family: sans-serif;
  display: block;
  margin-bottom: 0.5em;
}
p {
  margin: 0;
  line-height: 1.6;
}

body {
  --value: initial;
}
body:has([value="border-box"]:checked) {
  --value: border-box;
}
body:has([value="padding-box"]:checked) {
  --value: padding-box;
}
body:has([value="content-box"]:checked) {
  --value: content-box;
}
body:has([type="checkbox"]:checked) blockquote {
  border-radius: 70px;
}
```

```
blockquote {
  width: 210px;
  padding: 20px;
  margin: 20px;
  border: 20px dashed #dedede;
```

```
  background-color: #ededed;
  background-image: linear-gradient(rebeccapurple, magenta);
  background-repeat: no-repeat;
}

.clippath {
  background-origin: var(--value);
  clip-path: var(--value);
}

.box-model {
  background-origin: var(--value);
}
```

```
<blockquote class="clippath">
  <q
    >I've learned that people will forget what you said, people will forget what
    you did, but people will never forget how you made them feel.</q
  >
  <cite>&mdash; Maya Angelou</cite>
</blockquote>
<blockquote class="box-model">
  <q
    >I've learned that people will forget what you said, people will forget what
    you did, but people will never forget how you made them feel.</q
  >
  <cite>&mdash; Maya Angelou</cite>
</blockquote>

<fieldset>
  <legend>Select the geometry box value:</legend>
  <p>
    <label
      ><input type="radio" name="gb" value="border-box" /> border-box</label
    >
  </p>
  <p>
    <label
      ><input type="radio" name="gb" value="padding-box" /> padding-box</label
    >
  </p>
  <p>
    <label
      ><input type="radio" name="gb" value="content-box" /> content-box</label
    >
  </p>
  <p>
    <label
      ><input type="radio" name="gb" value="initial" checked /> initial</label
    >
  </p>
</fieldset>
<p>
  <label><input type="checkbox" /> Change the border radius</label>
</p>
```

When a `<geometry>` box is specified in combination with a `<basic-shape>`, the value defines the reference box for the basic shape. If specified by itself, it causes the edges of the specified box, including any corner shaping (such as a `border-radius`), to be the clipping path.

Shape origin

The previous example may make you think that the `<geometry-box>` values are useless, as you can use `background-origin` instead. And you can. But when clipping using basic shapes, the `<geometry-box>`, when included along with a `<basic-shape>` as the `clip-path` value, defines the reference box for, or origin of, that shape. We can combine the two previous examples to demonstrate this.

```
<blockquote class="clippath">
  <q
    >I've learned that people will forget what you said, people will forget what
    you did, but people will never forget how you made them feel.</q
  >
  <cite>&mdash; Maya Angelou</cite>
</blockquote>
<fieldset>
  <legend>Select the origin of the clip path shape:</legend>
  <p>
    <label
      ><input type="radio" name="gb" value="border-box" checked />
      border-box</label
    >
  </p>
  <p>
    <label
      ><input type="radio" name="gb" value="padding-box" /> padding-box</label
    >
  </p>
  <p>
    <label
      ><input type="radio" name="gb" value="content-box" /> content-box</label
    >
  </p>
</fieldset>
```

```
blockquote {
  width: 210px;
  padding: 20px;
  margin: 20px;
  border: 20px dashed #dedede;
  background-color: #ededed;
  background-image: linear-gradient(rebeccapurple, magenta);
  background-repeat: no-repeat;
  background-origin: border-box;
```

```
  clip-path: var(--value) polygon(0 50%, 50% 100%, 100% 50%, 50% 0);
}
```

```
blockquote {
  font-size: 1.2rem;
}
q {
  color: white;
  font-family: sans-serif;
  display: block;
  margin-bottom: 0.5em;
}
p {
  margin: 0;
  line-height: 1.6;
}

body {
  --value: "";
}
body:has([value="border-box"]:checked) {
  --value: border-box;
}
body:has([value="padding-box"]:checked) {
  --value: padding-box;
}
body:has([value="content-box"]:checked) {
  --value: content-box;
}
```

For another example, see `clip-path` shapes and geometry boxes.

Even values like `clip-path: margin-box` can be useful. In addition to creative visuals made by placing the clip-path's edge at the margin-box edge, any computed value for `clip-path`, other than `none`, results in the creation of a new stacking context the same way that CSS `opacity` does for values other than `1`.

Clipping to basic shapes

The `clip-path` property's support of `<basic-shape>` values provides a powerful way to shape elements. The various shape function enable defining precise clipping regions, effectively sculpting elements into unique forms. The basic shape functions, include:

- `circle()`
- `ellipse()`
- `inset()`
- `path()`
- `polygon()`
- `rect()`
- `shape()`
- `xywh()`

The size and position of these shapes are defined by the `<geometry-box>` value, which defaults to border-box being used as the reference box if the `clip-path` value includes a shape without the `<geometry-box>` component value.

Some of these functions appear to only provide basic predefined clipping options. They may appear to replicate effects you can create with `border-radius`, but if you toggled the `border-radius` property in the previous example, you may have noticed the power of CSS clipping. Shapes provide even more control. For example, `inset()` enables clipping a rectangle with precise margins. The real power and control comes with `path()`, `shape()`, and even `polygon()`, which allows for custom multi-point shapes.

Creating polygons

With `polygon()`, by defining pairs of coordinates, each of which represents a vertex of the shape. you can create intricate forms like stars or abstract figures. The coordinates define vector points connected by straight lines.

Here we use the `polygon()` function to create a star:

```
<div class="star"></div>
```

```
.star {
  width: 200px;
  height: 200px;
  background: linear-gradient(rebeccapurple, magenta) blue;
  clip-path: polygon(
    50% 0%,
    61% 35%,
    100% 35%,
    68% 57%,
    79% 91%,
    50% 70%,
    21% 91%,
    32% 57%,
    0% 35%,
    39% 35%,
    50% 0%
  );
}
```

Animation

Clipped shapes can be animated and transitioned by declaring the same number of vector points for the different states.

```
<div class="star"></div>
```

```
.star {
  width: 200px;
  height: 200px;
  background: linear-gradient(rebeccapurple, magenta) blue;
  clip-path: polygon(
    50% 0%,
    61% 35%,
    100% 35%,
```

```
    68% 57%,
    79% 91%,
    50% 70%,
    21% 91%,
    32% 57%,
    0% 35%,
    39% 35%,
    50% 0%
  );
}
```

```
@keyframes morphStar {
  from {
    clip-path: polygon(
      50% 0%,
      61% 35%,
      100% 35%,
      68% 57%,
      79% 91%,
      50% 70%,
      21% 91%,
      32% 57%,
      0% 35%,
      39% 35%,
      50% 0%
    );
  }
  to {
    clip-path: polygon(
      50% 10%,
      65% 30%,
      90% 20%,
      75% 60%,
      85% 95%,
      50% 80%,
      15% 95%,
      25% 60%,
      10% 20%,
      35% 30%,
      50% 10%
    );
  }
}

.star {
  animation: morphStar alternate 3s infinite ease-in-out;
}
```

The <code>path()</code> function

The `path()` function enables drawing shapes using SVG commands. The function accepts the equivalent of the SVG `d` attribute as the function's parameter.

The star from the previous example can be created using `path()`:

```
<div class="star"></div>
```

```
.star {
  width: 200px;
  height: 200px;
  background: linear-gradient(rebeccapurple, magenta) blue;
  clip-path: path(
    "M100,0 L122,70 L200,70 L136,114 L158,182 L100,140 L42,182 L64,114 L0,70 L78,70 L100,0 Z"
  );
}
```

Curved lines

With `path()`, we are not limited to straight lines. In this example, we use the `path()` function to create a heart:

```
<div class="heart"></div>
```

```
.heart {
  width: 200px;
  height: 200px;
  background: linear-gradient(rebeccapurple, magenta) blue;
  clip-path: path(
    "M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
  );
}
```

SVG as source

Instead of passing an SVG `d` attribute string as the `path()` function argument, the value of the `clip-path` property can reference the SVG `<clipPath>` element directly.

```
<div class="heart"></div>
<svg height="0" width="0">
  <clipPath id="heart">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z" />
  </clipPath>
</svg>
```

The `id` of the `<clipPath>` is the parameter of the `url()` function.

```
.heart {
  width: 200px;
  height: 200px;
  background: linear-gradient(rebeccapurple, magenta) blue;
```

```
  clip-path: url("#heart");
}
```

Shape function

The SVG path syntax is not the most intuitive. For this reason, CSS also offers a `shape()` function. The `shape()` function also takes path drawing directive, but with a syntax that is more human readable. We can recreate the heart with more declarative CSS:

```
.heart {
  clip-path: shape(
    from 20px 70px,
    arc to 100px 70px of 1% cw,
    arc to 180px 70px of 1% cw,
    curve to 100px 190px with 180px 130px,
    curve to 20px 70px with 20px 130px
  );
}
```

The `shape()` function is more robust in that it accepts CSS values and units (`path()` is limited to coordinates), including using CSS math functions like `calc()`. By using variables, we can create shapes (and boxes) of many different sizes:

```
:root {
  --m: 10;
}
.heart {
  width: calc(20px * var(--m));
  height: calc(20px * var(--m));
  display: inline-block;
  background: linear-gradient(rebeccapurple, magenta) blue;
  clip-path: border-box
    shape(
      from calc(2px * var(--m)) calc(7px * var(--m)),
      arc to calc(10px * var(--m)) calc(7px * var(--m)) of 1% cw,
      arc to calc(18px * var(--m)) calc(7px * var(--m)) of 1% cw,
      curve to calc(10px * var(--m)) calc(19px * var(--m)) with
        calc(18px * var(--m)) calc(13px * var(--m)),
      curve to calc(2px * var(--m)) calc(7px * var(--m)) with
        calc(2px * var(--m)) calc(13px * var(--m))
    );
}
.small {
  --m: 4;
}

.medium {
  --m: 8;
}

.large {
  --m: 12;
}
```

```
<div class="heart small"></div>
<div class="heart medium"></div>
<div class="heart large"></div>
```

Wrapping text around your clipped shapes

Clipped elements are still rectangular boxed. Clipping means your element doesn't look like a box; but it is still a box. To wrap inline content around the non-rectangular (or rectangular) shapes you define, use the `shape-outside` property. By default, inline content wraps around its margin box; `shape-outside` provides a way to customize this wrapping, making it possible to wrap text around the elements you've clipped, following the clip path you replicated rather than the element's rectangular box.

The content includes two elements to be clipped and the content that will be shaped around them.

```
<div class="leftTriangle"></div>
<div class="rightTriangle"></div>
<blockquote>
  <q>
    I've learned that people will forget what you said, people will forget what
    you did, but people will never forget how you made them feel.</q
  >
  <cite>&mdash; Maya Angelou</cite>
</blockquote>
```

```
:root {
  --m: 10;
  font-size: calc(3px * var(--m));
}
div {
  width: calc(0.75em * var(--m));
  height: calc(0.75em * var(--m));
  display: inline-block;
  background: linear-gradient(rebeccapurple, magenta) blue;
}
cite {
  display: block;
  text-align: right;
}
```

In addition to applying the same shape for both the `clip-shape` and `shape-outside` properties, the clipped element has to be floated so that the clipped element is on the same line as the content.

```
.leftTriangle {
  clip-path: polygon(0 0, 0 100%, 100% 0);
  shape-outside: polygon(0 0, 0 100%, 100% 0);
  float: left;
}
.rightTriangle {
  clip-path: polygon(100% 0, 100% 100%, 0 100%);
  shape-outside: polygon(100% 0, 100% 100%, 0 100%);
```

```
  float: right;
}
```

See also

- `<basic-shape>`
- `shape-image-threshold`
- `shape-margin`
- Overview of shapes
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module
- CSS shapes module

Introduction to CSS masking

CSS masking enables you to reveal or hide parts of an element selectively by applying one or more mask images to it. These mask images can be gradients, images, or SVG sources. Unlike CSS clipping, which either fully shows or hides areas of an element based on the shape of a single path, masking allows for nuanced transparency and blending effects based on the alpha transparency and, optionally, luminance of the mask images.

This guide introduces the concept of masking, the various mask image types, and how the luminance and alpha-transparency of the mask impact the portions of the element that are masked (made visible), versus the portions that are clipped (or hidden).

What is masking in CSS?

In CSS, masks can be used to define areas of an element that are visible and other areas that are hidden. Mask layers, defined by one or more `mask-image` sources, determine the areas of an element that should be visible and at what opacity.

> **Note:** Multiple CSS masking property values can be set using the `mask` shorthand property.

With `alpha` masks, the opacity of the masked element matches the opacity of the mask applied. In CSS, masking is the opposite of a masquerade mask, where the face is hidden wherever the mask is opaque. In CSS, the areas of the element where its mask is fully opaque will be fully opaque and visible. Wherever the mask is fully transparent, the element will be fully hidden. Areas of the element that are masked by partially opaque mask areas will be partially opaque, matching the mask's opacity.

With alpha masks, the color of the mask is irrelevant. Only the opacity of the mask matters. With luminance masks, the brightness of the mask colors is taken into account when determining the opacity of the masked element. The brighter and more opaque the color, the more opaque the element. The darker and more transparent the color, the less opaque the mask will be.

Masks can be defined using CSS gradients, raster images (such as PNGs), and SVG `<mask>` elements. In this guide, we introduce the various mask image types as we discuss opaqueness and transparency, luminance, and masking versus CSS clipping.

Each mask layer consists of a `mask-image`, which is positioned relative to an origin box. The mask images can be sized, repeated, and clipped. In cases where multiple mask images are declared, the way the mask layers are composited, or combined, can be set. These are discussed in the masking properties guide.

> **Note:** All examples will be using the following image as the underlying element upon which masks will be applied:
>
>

Opaqueness versus transparency

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag"
  class="applied-mask" />
<div class="mask-source"></div>
```

```
body {
  display: flex;
  gap: 20px;
  padding: 15px;
  background-image:
    linear-gradient(to right, transparent 50%, rgb(0 0 0 / 0.05) 50%),
    linear-gradient(to bottom, transparent 50%, rgb(0 0 0 / 0.05) 50%);
  background-size: 20px 20px;
}
div,
svg,
img {
  width: 220px;
  aspect-ratio: 1;
}
```

With alpha masks, the visible areas of an element are defined by the alpha-transparency of the mask applied to it. Wherever the mask is fully opaque, the element will be visible. At every pixel where the mask is fully transparent, the element too will be fully hidden. Areas of the element that are masked by a partially opaque section of a mask will be partially opaque, matching the opacity of the mask applied to it.

With gradients

To demonstrate this, let's look at an example using a `conic-gradient()` as the `mask-image`. CSS gradients, including conic gradients, can be used to create smooth transitions between visible and hidden areas.

In this case, the top-right corner of the mask is fully opaque, the top-left quadrant is fully transparent, and the bottom half has a smooth transition between opaque and transparent.

```
.applied-mask {
  mask-image: conic-gradient(black 90deg, transparent 270deg);
}
.mask-source {
  background: conic-gradient(black 90deg, transparent 270deg);
}
```

Note how the element on which the mask is applied has a fully visible top-right corner, the top-left quarter is hidden, and the bottom half transitions smoothly from visible to transparent, reflecting the visibility of the applied mask image.

With alpha masks, the color of the mask doesn't matter, only the transparency. In this example, we have a striped gradient with fully opaque red, semi-opaque red, and fully transparent stripes.

```
.applied-mask {
  mask-image: repeating-linear-gradient(
    to bottom right,
    red 0 20px,
    #ff000055 20px 40px,
    transparent 40px 60px
  );
}
.mask-source {
  background: repeating-linear-gradient(
    to bottom right,
    red 0 20px,
    #ff000055 20px 40px,
    transparent 40px 60px
  );
}
```

Note how the fully opaque mask areas reveal fully opaque element pixels, semi-transparent mask areas create semi-transparent areas, and fully transparent mask areas hide the associated areas completely.

With imported images

The previous two examples used gradients as masks and background images. The mask image doesn't have to be a CSS image. It can be an external image or an SVG.

In this case we use an external PNG. The image contains a colorful heart with a transparent background.

```
.applied-mask {
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/colorful-heart.png");
  mask-size: 220px 220px;
}
.mask-source {
  background: url("https://mdn.github.io/shared-assets/images/examples/colorful-heart.png");
  background-size: 220px 220px;
}
```

Note how the transparent mask areas crop the element; the only parts of the element that are visible are the areas where the mask is opaque. The color of the mask itself doesn't matter.

Alpha transparency versus luminance

The `mask-mode` property's default value — `match-source` — sets the mode to either `alpha` or `luminance`, depending on the value. The `match-source` value resolves to `alpha` for all mask sources other than SVG `<mask>` elements. If the mask source is a `<mask>` element, `match-source` resolves to the `<mask>`'s `mask-type` property value, if set. Otherwise, it resolves to the value of the SVG `mask-type` attribute set on the `<mask>` element. If that is not explicitly set either, `match-source` will resolve to `luminance`.

If `mask-mode` resolves to `luminance`, or we explicitly set it to `luminance`, the colors of the mask will affect the mask opacity. In the previous demo, the `mask-mode` was not set, so the value defaulted to `match-source`. As the colorful heart image is a transparent PNG, `match-source` resolves to `alpha`. By explicitly setting this property, we can control the mode. In this demo, we change the `mask-mode` to `luminance`.

```
.applied-mask {
  mask-mode: luminance;
}
```

When applying `mask-mode: luminance` to the same mask as in the previous example, the areas of the element where the mask is **lightest** are more opaque, while **darker** areas are less opaque.

The opacity of a luminance mask is determined by the `R`, `G`, `B`, and `A` values of an RGB color using the formula:

`((0.2125 * R) + (0.7154 * G) + (0.0721 * B)) * A`

For example, the newest `<named-color>` is `rebeccapurple`, which is `#663399`. While one might assume the lightness might be equivalent to the L of the `hsl()` color function, it's not that simple. The value `#663399` is equivalent to `rgb(40% 20% 60% / 1)` and `hsl(270 50% 40% / 1)`, but the brightness value is `27.134%`, not `40%`.

`((0.2125 * 0.4) + (0.7154 * 0.2) + (0.0721 * 0.6)) * 1 = 0.27134`

White has a brightness value of `100%`.

`((0.2125 * 1) + (0.7154 * 1) + (0.0721 * 1)) * 1 = 1`

Black's brightness is `0%`.

`((0.2125 * 0) + (0.7154 * 0) + (0.0721 * 0)) * 1 = 0`

We'll demonstrate this by adding white (`rgb(100% 100% 100%)`) with a lightness of `100%` at `27.234%` opacity to a `rebeccapurple`, `white`, and `black` linear gradient, which we'll then use to mask our image. This white resolves to the same opacity value:

`((0.2125 * 1) + (0.7154 * 1) + (0.0721 * 1)) * 0.27134 = 0.27134`

```
.applied-mask {
  mask-image: repeating-linear-gradient(
    to bottom left,
    rebeccapurple 0 20px,
    rgb(100% 100% 100% / 0.27134) 20px 40px,
    black 40px 45px,
    white 45px 50px
  );
  mask-mode: luminance;
}
.mask-source {
  background: repeating-linear-gradient(
    to bottom left,
    rebeccapurple 0 20px,
```

```
    rgb(100% 100% 100% / 0.27134) 20px 40px,
    black 40px 45px,
    white 45px 50px
  );
}
```

```
:has(:checked) .applied-mask {
  mask-mode: alpha;
}
```

```
<label><input type="checkbox" /><code>mask-mode: alpha;</code></label>
```

The areas with a `white` mask are fully opaque. The areas with a `black` mask are fully transparent. The areas with a `rebeccapurple` mask and the areas with a `27.1234%` opaque white mask are both `27.1234%` opaque.

If you toggle the `mask-mode` to `alpha`, the color of the gradient no longer matters. The entire element will be opaque except the areas covered by the semi-opaque white.

The `mask-mode` property enables using raster images without alpha transparency, such as JPEGs, as mask images. A JPEG is made up of opaque pixels. Using a JPEG as a mask with its default `alpha` mask mode would hide the entire element. The `luminance` value of `mask-mode`, on the other hand, clips the element where the mask is black (has no lightness), is fully opaque where the mask is opaque white (100% lightness), with other areas being semi-transparent based on the lightness of the mask area masking it.

In this example, we have a white moon against a black night sky.

```
.applied-mask {
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/moon.jpg");
  mask-mode: luminance;
  mask-size: 220px;
}
.mask-source {
  background: url("https://mdn.github.io/shared-assets/images/examples/moon.jpg");
  background-size: 220px;
}
```

The element is clipped and not visible where the sky is black. The image is most visible where the moon is at it's lightest.

In this case, if you toggle the `mask-mode` to `alpha`, the entire element will be visible as the entire mask is opaque.

SVG <code><mask></code> as mask source

A mask can be any type of CSS `<image>` or a `<mask-source>`. A `<mask-source>` is a `<url>` reference to an SVG `<mask>` element. This is similar to clipping with the CSS `clip-path` property, in which case the "mask" is an SVG `<clipPath>` element instead (with `clip-path`, the luminance of the path doesn't matter).

In this example, we define an SVG with a `<mask>` element, an identical `<clipPath>` element, and an identical `<path>` element so you can view the mask and clip path source.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag"
  class="applied-mask" />
<svg class="mask-source">
  <mask id="mask-heart">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="rgb(255 0 0 / 0.5)"
      stroke="rgb(255 255 255 / 1)"
      stroke-width="20" />
  </mask>
  <clippath id="clip-heart">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="rgb(255 0 0 / 0.5)"
      stroke="rgb(255 255 255 / 1)"
      stroke-width="20" />
  </clippath>
  <path
    d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
    fill="rgb(255 0 0 / 0.5)"
    stroke="rgb(255 255 255 / 1)"
    stroke-width="20" />
</svg>
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag"
  class="applied-clip" />
```

```
.applied-mask {
  mask-image: url("#mask-heart");
}
.applied-clip {
  clip-path: url("#clip-heart");
}
```

```
:has(:checked) .applied-mask {
  mask-mode: alpha;
}
body {
  flex-flow: row wrap;
}
```

```
<label><input type="checkbox" /><code>mask-mode: alpha;</code></label>
```

Because the image source is a `<mask>`, and the mask has neither the `mask-type` CSS property nor the `mask-type` SVG attribute set, the `mask-type` defaults to `alpha`, so the default of `mask-mode: match-source` resolves to `luminance`. This is because, for mask sources that are SVG `<mask>` elements, the `mask-type` defaults to `luminance` unless the `mask-type` attribute is explicitly set to `alpha`.

As we haven't set the `mask-type` attribute or CSS property on our mask, the `mask-mode` property default of `match-source` resolves to `luminance`. Toggle the checkbox to set the `mask-mode` value to `alpha` or allow it to default to `match-source`.

This example also demonstrated the difference between masking and clipping in CSS. You'll note that luminance and alpha-transparency are relevant to masking but not to clipping. Masking can be used to control the opacity of an element, while clipping shows everything inside the clipping path and fully hides the parts of the element outside the clip path. Clipped areas are completely invisible, whereas masked areas can be partially or fully visible.

If all you need are shapes, clipping may suffice. But if you need fading, variable opacity, or even control of position and size (which we'll discuss in a separate guide), masking is more suitable.

See also

- Introduction to CSS clipping
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

CSS mask properties

CSS masking is a technique that enables you to define visible portions of an element by applying a mask, which selectively reveals or hides parts of the element based on the alpha channels, and optionally colors, of the applied mask images.

The introductory guide to masking introduces the different types of mask images and their modes. The guide to declaring multiple masks discusses the mask layers and the `mask` shorthand property, providing a brief introduction to the shorthand's component properties. In this guide, we explore these component properties in greater detail and look at how they interact. We also explain how, in cases where multiple mask images are declared, the mask layers are composited, or combined.

CSS masks are composed of one or more mask layers, with a mask layer created for every value in the comma-separated list of `mask` or `mask-image` values, whether the values are images, mask sources, or the keyword `none`. Every `mask-image` is positioned relative to an origin box. The mask images can be sized, repeated, and clipped, then composited together with previous layers to create the final visual mask on the element.

The <code>mask-image</code> property

The minimum requirement to create a mask is a `mask-image` property set to a value other than `none`. The keyword `none` within a list of mask sources creates a mask layer. However, if `none` is the only value of the `mask-image` property, no masking occurs.

The mask image can be a CSS gradient, an imported image (such as a PNG, SVG, etc.), or an SVG `<mask>` element.

In this example, we create five mask layers, including an imported image, two gradients, a layer with no image, and an SVG `<mask>` source as the mask image.

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
}
```

Because one of the mask images is specified as `none`, only four mask images are applied to the `.masked-element` element, while creating five mask layers.

The importance of <code>none</code>

The `none` layer generally has no visual effect (see the `mask-composite` property for how it impacts the applied mask), but as each value in a comma-separated list of `mask-*` values applies to a separate mask layer, the `none` value serves an important purpose even when it doesn't change the composited mask.

This fourth layer in our five-layer structure will match the fourth value of any other comma-separated `mask-*` property values. As previously mentioned, the number of layers is determined by the number of comma-separated values in the `mask-image` property value, even if a value is `none`. Each `mask-*` value is matched up with the `mask-image` values, in order. If the number of values in a `mask-*` property differs from the number of mask layers, any excess values are ignored, or, if the property has fewer values than the number of mask layers, the values are repeated.

If a `mask-*` property has a single value, this value applies to all the layers. If we have five values, the fourth value applies to the `none` layer, with the last value being applied to the `<mask>` source layer. If there are two comma-separated values, the first value will apply to all the odd layers only, including that `<mask>` source layer. For example, each `mask-*` property may have a different number of values:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-repeat: repeat-x, repeat-y;
  mask-position:
    center,
    top left,
    bottom right;
}
```

In this case, every odd layer will be repeated along the x-axis while every even layer is repeated along the y-axis. The first and fourth layer images will be centered, while the second and fifth will be positioned in the top left corner. The `none` means the fifth layer's `#svg-mask` image will be repeated along the x-axis starting at the top left corner.

Learn more about mask layers and the `none` keyword.

The <code>mask-mode</code> property

The `mask-mode` property can be used to set the mode of each mask layer to either `alpha` or `luminance`, or allow it to default to the source's mode by setting the value to `match-source`, which is the default. While most `mask-*` properties have an analogous `background-*` property (`mask-image` is analogous to the `background-image` property, for example), `mask-mode` and `mask-composite` have no analogous `background` property.

Mask types: <code>alpha</code> and <code>luminance</code>

Each mask is either an `alpha` or a `luminance` mask.

With `alpha` masks, the alpha-transparency of each mask pixel is important. Wherever the mask is opaque, the corresponding parts of the element will be visible. Wherever the mask is transparent, the corresponding parts of the element will be hidden. Wherever the mask is semi-opaque, the element will be equally semi-opaque. The color of the mask doesn't matter, just the alpha-transparency of the colors.

With `luminance` masks, both the brightness of mask's colors and the alpha channel determine the opaqueness of the masked areas.

> **Note:** All subsequent examples use the following image as a `background-image` on an element to which masks will be applied:

This example demonstrates the difference between `alpha` and `luminance` masks. The masks are the same, but in the `alpha` mask, only the alpha-transparency of the gradient mask colors matter. In the `luminance` example, the R, G, B, and A all matter.

Two containers contain images, while the last is empty but is included to display the gradient we will be using as our `mask-image`.

```
<div class="alpha">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="luminance">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="gradient"></div>
```

```
body {
  display: flex;
  gap: 20px;
  padding: 15px;
  background-image: conic-gradient(
    transparent 90deg,
    rgb(0 0 0 / 0.05) 90deg 180deg,
    transparent 180deg 270deg,
    rgb(0 0 0 / 0.05) 270deg
  );
  background-size: 30px 30px;
}
div,
svg,
img {
  width: 220px;
  aspect-ratio: 1;
}
div {
  border: 1px solid black;
}
```

We declare a `repeating-linear-gradient` with red, transparent, and semi-transparent red diagonal stripes. This gradient is used as our mask and, for the last container, as the background image:

```
img {
  mask-image: repeating-linear-gradient(
    to bottom right,
    red 0 20px,
    #ff000055 20px 40px,
    transparent 40px 60px
  );
}
.gradient {
  background: repeating-linear-gradient(
    to bottom right,
    red 0 20px,
    #ff000055 20px 40px,
    transparent 40px 60px
  );
}
```

We set different values for the `mask-mode` property for each image:

```
.alpha img {
  mask-mode: alpha;
}

.luminance img {
  mask-mode: luminance;
}
```

In the `alpha` case, only the transparency of the gradient's colors matter. Where the gradient is opaque red, the image is opaque. Where the gradient is transparent, the image is hidden. Where the gradient is 50% opaque, the image is 50% opaque. In the `luminance` case, the color's brightness matters! See Alpha transparency versus luminance to learn about the equation that uses the color's R, G, B, and A channels to determine the opacity of the mask.

The <code>mask-mode</code> default value: <code>match-source</code>

The default value of the `mask-mode` property is `match-source`. This value sets the `mask-mode` to match the mask's mode type. The `match-source` value resolves to `alpha` for every mask except masks where the mask source is an SVG `<mask>` element.

When an SVG `<mask>` element is used as the mask source, the `match-source` value resolves to the value of the `<mask>` element's `mask-type` property. If the `<mask>` element (not the "masked element") doesn't have the CSS `mask-type` property defined, this property defaults to the value of the SVG `mask-type` attribute, if present. If that is omitted as well, the `match-source` value will resolve to `luminance`.

Continuing with the `masked-element` example, if we don't explicitly set the `mask-mode` property, it will default to `match-source` for each layer, as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
}
```

or, using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") match-source,
    linear-gradient(to right, black, transparent) match-source,
    radial-gradient(circle, white 50%, transparent 75%) match-source,
    none match-source,
    url("#svg-mask") match-source;
}
```

The first mask layer, `url("alphaImage.png")`, references an image. As this isn't a `<mask>` element within an `<svg>`, the `mask-mode` resolves to `alpha`, with the opaque parts of this image making the corresponding parts of the element visible, while the transparent or semi-transparent parts are invisible or partially visible.

The `linear-gradient(to right, black, transparent)` is the second mask layer and `radial-gradient(circle, white 50%, transparent 75%)` is the third. Again, these aren't `<mask>` elements, so the `match-source` value resolves to `alpha`. The masking effect of these layers is determined by the opaqueness of the gradient mask by default.

The fourth mask layer has `none` declared, meaning the mask for this layer is transparent black. The `.masked-element` class sets `mask-mode: match-source;`. Had `mask-mode` instead been a comma-separated list of five different values, the fourth value would have applied to this `none` layer, allowing the fifth value to apply to the fifth layer.

The fifth mask layer is comprised of an SVG `<mask>` element that has `svg-mask` as its `id`. While the default mask mode of the other layers is `alpha`, the default mask type of SVG `<mask>` elements is the `mask-type` value, or, if not set, the `mask-type` attribute. If that isn't defined either, the value defaults to `luminance`. In other words, the masking effect of the `<mask>` is determined by both the brightness and transparency of the `<mask>` element's colors.

If we don't declare the `mask-mode` property at all, and allow it to default to `match-source` for each mask layer, the result in this `.masked-element` case would resolve to:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: alpha, alpha, alpha, match-source, luminance;
}
```

or, using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") alpha,
    linear-gradient(to right, black, transparent) alpha,
    radial-gradient(circle, white 50%, transparent 75%) alpha,
    none match-source,
    url("#svg-mask") luminance;
}
```

The <code>mask-position</code> property

Analogous to the `background-position` property, the `mask-position` property sets the initial position of the mask image relative to the mask layer's origin box, defined by the `mask-origin` property. The syntax follows the `background-position`'s `<position>` syntax, with the value being one, two, or four `<position>` values, defining one to two relative or absolute position offsets.

One-value syntax

If only one keyword value is specified, that value specifies the mask origin edge against which the mask will be placed with the other dimension being `center`.

If only one `<length-percentage>` value is specified, this specifies the X coordinate relative to the left edge of the mask origin, with the Y coordinate set to `50%`.

If two keyword values are specified, the order of the value doesn't matter, but the value cannot include two vertical or two horizontal axis values; `left right` and `top bottom` are both invalid.

Two-value syntax

If two values are present, including one keyword and one `<length-percentage>` value, the order only matters if the keyword is `center`:

- If the keyword is `left` or `right`, it defines the X coordinate relative to the left edge, and the value defines the Y coordinate relative to the top edge.
- Similarly, a `top` or `bottom` keyword defines the Y coordinate positioning the element against the top or bottom edge, respectively, with the other value defining the X value relative to the left edge of the mask origin box.
- When one value is the `center` keyword and the other is a `<length-percentage>`, the first value defines the horizontal position and the second value defines the vertical position.

If two values are present and both are `<length-percentage>` values, the order again matters; the first value defines the horizontal positioning as an offset from the left edge of the mask-positioning area while the second value defines the vertical position as an offset from the top edge of the mask-positioning area.

Four-value syntax

Mask positions can also be relative to corners other than the top left. The four-value syntax enables offsetting the mask from any corner. The value includes two `<length-percentage>` offsets, each preceded by the origin side for that offset. Whether you declare the horizontal or vertical pair first doesn't matter, but you must declare the origin-side keyword (`left`, `right`, `top`, `bottom`, `x-start`, `x-end`, `y-start`, `y-end`, `block-start`, `block-end`, `inline-start`, or `inline-end`) before the offset `<length-percentage>` in each pair, and the two origin-sides can't be from the same axis.

In the two `<length-percentage>` syntax, the origin sides are `top` and `left`, in that order. For example, `mask-position: 10px 20px` is the equivalent of `mask-position: left 10px top 20px`. When offsetting from the top and left, the offset sides aren't required, but the order matters. With the four value syntax, you can use `mask-position` to offset the mask image from any edge combinations, such as `left 10px bottom 20px`, and the order of the sides doesn't matter, as the offset edge is defined by the keyword preceding it rather than the declaration order.

Percentage values

When offsetting using percentage values, the mask's dimension is subtracted from the element's dimension, just as is done with percentage offsets with `background-position`.

Positioning repeating mask images

The `mask-position` property defines the initial position of the mask image. By "initial position", if the mask repeats, the browser places the first mask image in the position defined by the `mask-position` property, thus defining the placement of the mask repetitions.

In this example, we set the position of the first image to `bottom right`, meaning the first mask will be placed at the bottom right edge of the mask origin box. Because mask images repeat by default, the repeating masks will be positioned against the first placed mask's top and left sides.

```
<div class="keywords">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="twoValue">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="fourValue">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
```

```
div,
img {
  width: calc(30vw - 20px);
}
```

```
img {
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
}
.keywords img {
  mask-position: bottom right;
}
.twoValue img {
  mask-position: -20px -10px;
}
.fourValue img {
  mask-position: right -20px bottom -10px;
}
```

```
div::before {
  content: 'class="' attr(class) '"';
  display: block;
  text-align: center;
  font-family: monospace;
}
body {
  flex-flow: row wrap;
}
```

The `mask-position` defines the position of the first mask image's placement. This demo shows where the first image is placed:

```
img {
  mask-repeat: no-repeat;
}
```

Because the default value for the `mask-repeat` property is `repeat`, the images are repeated along the X and Y axes, based on the position of that first mask:

The two-value example defines the top and left offsets of the original mask. The four-value example combines the previous two previous examples, position the first mask using the same offsets as the second image, but from the same edges as demonstrated in the first image.

In the first image, the first star to be placed is the one on the bottom right, with the repeated stars above and to the left. Because of this positioning, the initial star is not clipped, but the top-most and left-most stars are.

If we don't explicitly set the `mask-position` property, it will default to `0% 0%` for each layer, with the top-left corner of the mask abutting the top-left corner of the mask origin box. Continuing with the `masked-element` example, it is as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
  mask-position: 0% 0%;
}
```

or, expanding on the example using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") 0% 0% match-source,
    linear-gradient(to right, black, transparent) 0% 0% match-source,
    radial-gradient(circle, white 50%, transparent 75%) 0% 0% match-source,
    none 0% 0% match-source,
    url("#svg-mask") 0% 0% match-source;
}
```

The <code>mask-origin</code> property

When an element has padding, a border, or both, the `mask-origin` property defines which of these box edge values acts as the mask origin box, or the *mask positioning area*, within which a mask image is positioned, for that layer. The `mask-origin` property is analogous to the `background-origin` property, but with a different initial value and SVG-only values.

HTML elements can have masks contained within their content border box, padding box, or content box. For example, if the `mask-position` is `top left`, is that relative to the border's outer edge, the padding's outer edge, or the content's outer edge?

In the `mask-position` masking example, the position defined was relative to the border-box (the default behavior), although it is worth noting that the `<img>` had no border or padding set, therefore the content-box, padding-box, and border-box origins would all be the same in this case.

```
<div class="border-box">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<fieldset>
```

```
  <legend>Set the <code>mask-origin</code> value</legend>
  <label
    ><input type="radio" name="origin" id="origin_border-box" checked />
    border-box</label
  >
  <label
    ><input type="radio" name="origin" id="origin_padding-box" />
    padding-box</label
  >
  <label
    ><input type="radio" name="origin" id="origin_content-box" />
    content-box</label
  >
</fieldset>
```

```
div {
  all: unset;
}
legend {
  align-self: baseline;
}
label {
  display: block;
}
```

In this example, the `mask-position` places the initial mask in the top left corner of the `<img>` element that has a large border and padding, with a green background color to enable seeing the star masking on the padding area.

```
img {
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
  mask-position: top left;
  padding: 15px;
  border: 15px solid;
  background-color: green;
}
:has(#origin_border-box:checked) img {
  mask-origin: border-box;
}
:has(#origin_padding-box:checked) img {
  mask-origin: padding-box;
}
:has(#origin_content-box:checked) img {
  mask-origin: content-box;
}
```

Change the value of the `mask-origin` property by changing the selected radio button, looking at the position of the top left star mask as you do so.

The default value is `border-box`. With this value, the initial mask is placed at the border's top-left edge and is not clipped. When the initial mask is placed at the outer or inner edge of the padding, there is room above it and to the left; these repeating masks are clipped.

Continuing with the `masked-element` example, if we don't explicitly set the `mask-origin` property, it will default to `border-box` for each layer, as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
  mask-position: 0% 0%;
  mask-origin: border-box;
}
```

or, expanding on the example using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") 0% 0% border-box match-source,
    linear-gradient(to right, black, transparent) 0% 0% border-box match-source,
    radial-gradient(circle, white 50%, transparent 75%) 0% 0% border-box
      match-source,
    none 0% 0% border-box match-source,
    url("#svg-mask") 0% 0% border-box match-source;
}
```

For SVG elements, which don't have the associated CSS layout boxes, a mask can be contained inside the SVG element's fill, stroke, or view box.

The <code>mask-clip</code> property

The `mask-clip` property determines the area of the element that will be affected by a mask, effectively clipping the element at the defined box edge. It is analogous to the `background-clip` property, but with some different values.

Because the `mask-clip` property accepts all the `mask-origin` values and both have the same `border-box` default value, the two properties may seem similar, but they serve very different purposes. While `mask-origin` determines where a mask image will be positioned, the `mask-clip` property causes the original element to have its content clipped to the specified box. It is important to understand them both: if the `mask-origin` causes the `mask-position` to place the mask image outside the clipping area, the mask will be clipped.

The `mask-clip` property accepts all the `mask-origin` values, as well as its own `no-clip` value. The `no-clip` value sets the painted content not to be clipped. You can still cause the mask image to be clipped by positioning it outside of the border content area using `mask-position` values that are less than zero or resolve to greater than 100%.

Setting the `mask-clip` and `mask-origin` to different values can cause the mask layer image to be clipped. For example, if an element with a border and padding has `mask-clip` set to `content-box` and `mask-origin` set to `border-box`, and the `mask-position` is set to the `top left` edge, the mask layer image will be clipped at the top-left edge.

The next example adds clipping options to the previous example to demonstrate the different non-SVG `mask-clip` values and to show how they impact the different `mask-origin` values.

```
<div class="border-box">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<fieldset>
  <legend>Set the <code>mask-clip</code> value</legend>
```

```
  <label
    ><input type="radio" name="clip" id="clip_border-box" checked />
    border-box</label
  >
  <label
    ><input type="radio" name="clip" id="clip_padding-box" /> padding-box</label
  >
  <label
    ><input type="radio" name="clip" id="clip_content-box" /> content-box</label
  >

  <label><input type="radio" name="clip" id="clip_no-clip" /> no-clip</label>
</fieldset>
<fieldset>
  <legend>Set the <code>mask-origin</code> value</legend>
  <label
    ><input type="radio" name="origin" id="origin_border-box" checked />
    border-box</label
  >
  <label
    ><input type="radio" name="origin" id="origin_padding-box" />
    padding-box</label
  >
  <label
    ><input type="radio" name="origin" id="origin_content-box" />
    content-box</label
  >
</fieldset>
```

```
:has(#clip_border-box:checked) img {
  mask-clip: border-box;
}
:has(#clip_padding-box:checked) img {
  mask-clip: padding-box;
}
:has(#clip_content-box:checked) img {
  mask-clip: content-box;
}
```

The first mask is placed at the top-left edge of the mask origin container, then repeated. If the origin box is the `border-box` and the clipping region is the `content-box`, the top and left areas of the mask origin container will be clipped. Generally, you will want the `mask-clip` to be the same as the `mask-origin`.

Continuing with the `masked-element` example, if we don't explicitly set the `mask-clip` property, it will default to `border-box` for each layer, as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
  mask-position: 0% 0%;
  mask-origin: border-box;
  mask-clip: border-box;
}
```

or, expanding on the example using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") 0% 0% border-box border-box match-source,
    linear-gradient(to right, black, transparent) 0% 0% border-box border-box
      match-source,
    radial-gradient(circle, white 50%, transparent 75%) 0% 0% border-box
      border-box match-source,
    none 0% 0% border-box border-box match-source,
    url("#svg-mask") 0% 0% border-box border-box match-source;
}
```

In the `mask` shorthand, if only one `<geometry-box>` value is given, it sets both the `mask-origin` and `mask-clip` property values. If two `<geometry-box>` values are present, the first defines the `mask-origin` and the second defines the `mask-clip`.

For mask layer images that do not reference an SVG `<mask>` element, the `mask-clip` property defines whether the mask painting area, or the area affected by the mask, is the border, padding, or content box. The painted content of the element will be restricted to this area.

When the mask layer's `mask-image` source is a `<mask>`, the `mask-clip` property has no effect. Rather, the `<mask>` element's `x`, `y`, `width`, `height`, and `maskUnits` attributes determine the mask painting area.

The <code>mask-size</code> property

The `mask-size` property is used to size mask layers. This property is analogous to the `background-size` property, taking the same values. When sizing your masks, remember that areas of the element not covered by the mask images are hidden.

There are three ways to declare a `mask-size`:

- the `cover` or `contain` keyword,
- one length, percentage, or the keyword `auto`, or
- two values that are a combination of lengths, percentages, and the keyword `auto`.

The mask image can be left to its natural size, stretched, or constrained to fit the available space. The aspect ratio of the mask image is maintained by default, but declaring two `<length-percentage>` values can distort the mask image if the ratio of the two values is not the same as the original image (`mask-repeat: round` is the other property/value pair that may distort the mask image).

If the `mask-size` is set to `contain`, the mask image will be the largest size it can be while being fully contained within the mask positioning area. In this case, the mask image is not clipped, but rather, it is fully contained.

When set to `cover`, the mask image will be the smallest size it can be to fully cover the entire mask-positioning area, with the mask being clipped if the mask's aspect ratio differs from the aspect ratio of the mask-positioning area.

In other words, With `cover` and `contain`, at least one dimension of the mask will be the same size as the mask-positioning area's same dimension; the mask image either grows or shrinks so that either the width is the same width as the mask positioning area or the mask image's height is equal to the mask positioning area's height.

With `cover`, `contain`, and `<percentage>` values, the size is relative to the origin box. In our star-mask and flag image example, the aspect ratio of both the mask image and the `<img>` are `1:1`, meaning in this case `cover`, `contain`, and `100%` will all produce the same sized mask. This example demonstrates how, when `mask-size` is set to `cover`, `contain` or a `<percentage>` value, the actual size of the mask may differ depending on the value of the `mask-origin` property:

```
<div class="border-box">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<fieldset>
  <legend>Set the <code>mask-origin</code> value</legend>
  <label
    ><input type="radio" name="origin" id="border-box" checked />
    border-box</label
  >
  <label
    ><input type="radio" name="origin" id="padding-box" /> padding-box</label
  >
  <label
    ><input type="radio" name="origin" id="content-box" /> content-box</label
  >
</fieldset>
```

```
img {
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
  mask-position: top left;
  padding: 15px;
  border: 15px solid;
  background-color: green;
}
```

```
img {
  mask-size: 100%;
}
:has(#border-box:checked) img {
  mask-origin: border-box;
}
:has(#padding-box:checked) img {
  mask-origin: padding-box;
}
:has(#content-box:checked) img {
  mask-origin: content-box;
}
```

Change the value of the `mask-origin` property to see how the different values impact the mask size:

This example included one `<percentage>` value. When one `<length-percentage>` value is specified, it defines only the mask width, with the height defaulting to `auto`, which maintains the aspect ratio. When two values are specified, the first defines the mask width and the second defines its height.

The default value of `mask-size` is `auto`, rendering the mask at its intrinsic size, the size at which the mask would be displayed if no CSS were applied. The underlying aspect ratio of the mask image is maintained if you set a single `<length-percentage>` value, or two values in the same ratio as the aspect ratio. If you declare two values not in the same ratio as the aspect ratio, the mask image is distorted.

Like with all longhand components of shorthand property, if the `mask` shorthand property is set and the value of the `mask-size` property is not defined within any mask layer, the `mask-size` value is reset to its initial value of `auto` for those mask layers.

If the image has no intrinsic proportion, for example in the case of a CSS gradient, the default `auto` is the entirety of the mask positioning area as set by the `mask-origin` property.

Continuing with the `masked-element` example, if we don't explicitly set the `mask-size` property, it will default to `auto` for each layer, as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
  mask-position: 0% 0%;
  mask-origin: border-box;
  mask-clip: border-box;
  mask-size: auto;
}
```

or, expanding on the example using the `mask` shorthand, with the `mask-size` component going after the `mask-position` value, separated by a forward slash (/):

```
.masked-element {
  mask:
    url("alphaImage.png") 0% 0% / auto border-box border-box match-source,
    linear-gradient(to right, black, transparent) 0% 0% / auto border-box
      border-box match-source,
    radial-gradient(circle, white 50%, transparent 75%) 0% 0% / auto border-box
      border-box match-source,
    none 0% 0% / auto border-box border-box match-source,
    url("#svg-mask") 0% 0% / auto border-box border-box match-source;
}
```

The <code>mask-repeat</code> property

The `mask-repeat` property defines how mask images are repeated, or tiled, after the initial mask image has been sized and positioned. The `mask-repeat` property defines if and how that mask image is repeated along the horizontal and vertical axes. In most of the previous examples, you may have noticed the star mask repeated along the X and Y axes. This is because `repeat` is the default value.

The `mask-repeat` property is analogous to the `background-repeat` property, accepting the same `<repeat-style>` values. As is the case with `background-repeat`, the first (and possibly only) mask-image repetition is positioned by the `*-position` property and sized by the `*-size` property. The positions of the repeated background or mask images are based on this initial image instance.

Continuing with the `masked-element` example, if we don't explicitly set the `mask-repeat` property, it will default to `repeat` for each layer, as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
```

```
  mask-position: 0% 0%;
  mask-origin: border-box;
  mask-clip: border-box;
  mask-size: auto;
  mask-repeat: repeat;
}
```

or, expanding on the example using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") 0% 0% / auto repeat border-box border-box match-source,
    linear-gradient(to right, black, transparent) 0% 0% / auto repeat border-box
      border-box match-source,
    radial-gradient(circle, white 50%, transparent 75%) 0% 0% / auto repeat
      border-box border-box match-source,
    none 0% 0% / auto repeat border-box border-box match-source,
    url("#svg-mask") 0% 0% / auto repeat border-box border-box match-source;
}
```

The <code>mask-composite</code> property

The `mask` shorthand includes the `mask-composite` property, which defines how multiple masks are combined to create the final mask effect. Each value in the comma-separated list of values determines whether the browser should `add`, `subtract`, `intersect`, or `exclude` the associated mask layer from or to the mask layers below it. Similar to `mask-mode`, and the other `mask-*` properties, there is no property in the `background` shorthand that is analogous.

```
<div class="add">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="subtract">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="intersect">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="exclude">
  <img
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
```

In this example, we include two `mask-image` values, including the star and the gradient from the previous examples as mask images:

```
img {
  mask-image:
    repeating-linear-gradient(
      to bottom right,
      red 0 20px,
      #ff000055 20px 40px,
      transparent 40px 60px
    ),
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
}
```

We set a different `mask-composite` value for each image:

```
.add img {
  mask-composite: add;
}
.subtract img {
  mask-composite: subtract;
}
.intersect img {
  mask-composite: intersect;
}
.exclude img {
  mask-composite: exclude;
}
```

```
div::before {
  content: "mask-composite: " attr(class);
  display: block;
  text-align: center;
  font-family: monospace;
}
body {
  flex-flow: row wrap;
}
```

The semi-transparent star mask is added to, subtracted from, intersected with, or excluded from the striped mask, depending on the `mask-composite` value.

The `mask-composite` property is only relevant in cases with two or more mask layers. This reads "mask layers", not "mask images" because, if `none` is included, the transparent black mask is composited. A `none` value can have a profound effect on masking in the case of `subtract` and `intersect`. For example, if the `mask-mode` resolves to `luminance`, subtracting a black mask will remove the entire mask (the element will be hidden). Similarly, if `none` is the last layer with `mask-composite: intersect` set for that layer, the entire element will be hidden. Here, we add a third layer, with `none`, to the previous example:

```
img {
  mask-image:
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg"),
    repeating-linear-gradient(
      to bottom right,
      red 0 20px,
      #ff000055 20px 40px,
```

```
      transparent 40px 60px
    ),
    none;
}
```

Note how the `intersect` example excludes everything because the transparent black mask doesn't intersect anything.

If we reverse the order of the mask layers, we can also get very different results:

```
<div class="subtract">
  <img
    class="gradientFirst"
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
<div class="subtract">
  <img
    class="starFirst"
    src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    alt="Pride flag" />
</div>
```

```
.gradientFirst {
  mask-image:
    repeating-linear-gradient(
      to bottom right,
      red 0 20px,
      #ff000055 20px 40px,
      transparent 40px 60px
    ),
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
}
.starFirst {
  mask-image:
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg"),
    repeating-linear-gradient(
      to bottom right,
      red 0 20px,
      #ff000055 20px 40px,
      transparent 40px 60px
    );
}
```

In the first example, the stars are subtracted from the stripes. In the second, the stripes are subtracted from the stars.

Like all the other `mask` component properties, `mask-composite` takes a comma-separated list of values. Because the property effects how masks are combined, this property is only relevant for multiple mask layers and the number of used values is one less than the number of mask layers.

The last pair of masks is composited first. The previous mask image is then composited with the previous composition.

Continuing with the `masked-element` example, if we don't explicitly set the `mask-composite` property, it will default to `add` for each layer, as if we had set the following:

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
  mask-mode: match-source;
  mask-position: 0% 0%;
  mask-origin: border-box;
  mask-clip: border-box;
  mask-size: auto;
  mask-repeat: repeat;
  mask-composite: add;
}
```

In this case, the `<mask>` element will be composited with the `none` layer. Then the radial gradient will be composited with the result of the previous composition, and so on.

Like we saw with all the other component properties, we could have used the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png") 0% 0% / auto repeat border-box border-box add
      match-source,
    linear-gradient(to right, black, transparent) 0% 0% / auto repeat border-box
      border-box add match-source,
    radial-gradient(circle, white 50%, transparent 75%) 0% 0% / auto repeat
      border-box border-box add match-source,
    none 0% 0% / auto repeat border-box border-box add match-source,
    url("#svg-mask") 0% 0% / auto repeat border-box border-box add match-source;
}
```

See also

- Introduction to CSS masking
- Introduction to CSS clipping
- CSS masking module

Declaring multiple masks

CSS masking is a technique that enables you to use images as masks to define which sections of an element are fully visible or semi-opaque. The CSS mask selectively reveals or hides parts of the element based on the alpha channel and in some cases the brightness of the colors of the applied mask images.

CSS masks are the opposite of masks worn at masquerade (masked) balls. At a masked ball, a wearer's face is hidden wherever the mask is opaque and visible wherever you can see through the mask. In CSS, the areas where the composited mask layers are fully opaque reveal the element, while transparent areas hide it.

CSS masks are made up of one or more mask layers. In this guide, we discuss the concept of mask layers and how to declare multiple mask layers using the `mask` shorthand property.

Understanding mask layers

You can apply CSS masking to all HTML elements and most SVG elements. A mask can consist of one or more composited mask layers. You define multiple layers using comma-separated values in the `mask` shorthand property or the `mask-image` property–even a value set to `none` counts as a layer.

Each mask layer can contain a mask image, which is positioned relative to the mask's origin box. The image can be sized, repeated, and clipped. If you include more than one mask image, you can define the way the mask layers are composited or combined. (These features are briefly introduced in this guide. For more details and examples, see the masking properties guide.)

Syntax for multiple mask layers

The `mask` shorthand property accepts a comma-separated list of mask layers. The syntax for each layer can include the following values:

```
<image> <position> / <size> <repeat> <origin> <clip> <composite> <mode>
```

All the components in a mask layer are optional. However, if you omit the `mask-image` value, it defaults to a transparent black image, which hides the element in that layer completely.

The `mask` shorthand declaration sets values for all the `mask-*` properties. Any component not declared within a layer will default to its initial value. The `mask` property also resets all the `mask-border-*` properties to their initial values. A `mask` declaration that only includes a `mask-image` value implicitly sets the following:

```
mask-mode: match-source;
mask-position: 0% 0%;
mask-size: auto;
mask-repeat: repeat;
mask-origin: border-box;
mask-clip: border-box;
mask-composite: add;

mask-border-source: none;
mask-border-mode: alpha;
mask-border-outset: 0;
mask-border-repeat: stretch;
mask-border-slice: 0;
mask-border-width: auto;
```

Defining mask layers with <code>mask-image</code>

As long as a comma-separated `mask-image` property declaration includes at least one value other than `none`, a mask layer is created for every value in the declaration, even for the `none` values. This behavior applies whether you're using the `mask-image` property or the `mask` shorthand. These mask images can be gradients, images, or SVG sources. You can define them using a CSS gradient, a raster image (such as PNGs), or an SVG `<mask>` element.

```
.gradient-mask {
  mask-image: linear-gradient(to right, black, transparent);
}

.raster-mask {
  mask-image: url("alphaImage.png");
}

.mask-element-mask {
  mask-image: url("#svg-mask");
}
```

The introductory guide to masking introduces the different types of mask images and their modes.

The `mask-image` property is analogous to the `background-image` property. Just as with the `background-image` property, to include multiple mask images, the image values are separated by commas.

```
.multiple-gradient-mask {
  mask-image:
    linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%);
}
```

Each mask image in a multiple-image declaration creates a mask layer. All the examples in this section create one mask layer, except the `multiple-gradient-mask` declaration, which creates two.

Mask layers and the <code>none</code> keyword

If `none` is the only value of the `mask-image` property, no mask layers are created and no masking occurs.

```
.no-masks {
  mask-image: none;
}
```

Similarly, when using the `mask` shorthand, if no `mask-image` value is present other than `none`, no masking occurs. If any of the following are declared, no mask layers are created and nothing is hidden:

```
mask: none;
mask: none 100px 100px no-repeat;
mask: 100px 100px no-repeat;
```

Otherwise, as long as there is a `mask-image` declared that isn't set to `none`, a mask layer is created for every value in the comma-separated list of values, even when the `mask-image` value is omitted from a value in the comma-separated list or is explicitly set to `none`. In other words, a layer is created for each valid comma-separated value, unless the entire property resolves to `none`.

```
.masked-element {
  mask-image:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
}
```

The keyword `none` within a list of mask sources creates a mask layer, albeit a transparent black image layer. Any elements with the class `masked-element` will have five mask layers:

We can also create the layers using the `mask` shorthand:

```
.masked-element {
  mask:
    url("alphaImage.png"), linear-gradient(to right, black, transparent),
    radial-gradient(circle, white 50%, transparent 75%), none, url("#svg-mask");
}
```

If a value in the comma-separated list of values is an empty image, fails to download, references a `<mask>` element that doesn't exist, or otherwise cannot be displayed (or is set to `none`), it still counts as a mask image layer, rendering a transparent black mask image that has no visual effect. If all of the values do this, the element will be fully hidden.

No masking occurs if the entire property resolves to `none`, which makes the element fully visible. On the other hand, if the value includes multiple layers and at least one is not `none`, the `none` layers don't reveal any part of the element (or don't make any part of the element visible). In this example, the value doesn't resolve to `none`; but because all non-`none` images are invalid, masking occurs, and the element will be fully hidden.

A computed value other than `none` creates a CSS stacking context.

How mask layers affect <code>mask-*</code> properties

The number of mask layers matters when you're also using individual `mask-*` properties after or with more specificity than a `mask` declaration.

The `mask-*` properties include:

- `mask-mode`: Sets the mode of each mask layer to either `alpha` or `luminance`, or allows it to default to the source's mode by setting the value to `match-source`. The default is `match-source`.
- `mask-position`: Analogous to the `background-position` property with syntax that follows the `background-position`'s `<position>` syntax, it sets the initial position of the mask image relative to the mask layer's origin box, defined by the `mask-origin` property. You can specify one, two, or four `<position>` values. The default `0% 0%` positions the top-left corner of the mask at the top-left corner of the mask origin box.
- `mask-origin`: Analogous to the `background-origin` property, it specifies the *mask positioning area*, which is the mask origin box area within which a mask image is positioned. For example, if the `mask-position` is `top left`, this property defines whether that is relative to the border's outer edge, the padding's outer edge, or the content's outer edge.
- `mask-clip`: Analogous to the `background-clip` property, it determines the area of the element affected by a mask. It defines whether the mask painting area is the border, padding, or content box, restricting the painted content of the element to this area. If the mask layer's `mask-image` source is an SVG `<mask>` element, the `mask-clip` property has no effect.
- `mask-size`: Analogous to the `background-size` property, this is used to size the mask layer. Values can be a single keyword (`cover`, `contain`, or `auto`), a single length or percentage, or two space-separated values–each of which may be a length, percentage, or `auto`. The default is `auto`.
- `mask-repeat`: Analogous to the `background-repeat` property, this defines how the mask layer image is tiled after it has been sized and positioned.
- `mask-composite`: Defines how a mask is combined with the mask layers below it. Each mask layer is either added to, subtracted from, included with, or excluded from the previously composited mask layers below it. Similar to `mask-mode`, there is no analogous `background-*` property.

Each `mask-*` value in a comma-separated list of `mask` component properties applies to a separate mask layer. As stated earlier, an element can have multiple mask layers applied — the number of layers is determined by the number of comma-separated values in the `mask-image` or `mask` properties. Each `mask-*` value is matched with a mask layer, in order. If the number of values in the `mask-*` property is greater than the number of mask layers, any excess values are ignored. If the mask component property has fewer values than the number of mask layers, the `mask-*` values are repeated.

To learn more about these individual properties, see CSS mask properties.

Order of shorthand component properties

For the most part, the order of the properties is flexible, but there are a few quirks and exceptions.

Ordering rules for <code>mask-origin</code> and <code>mask-clip</code>

The `mask-origin` value, listed in the syntax as `<origin>`, comes before the `mask-clip` values, listed in the syntax as `<clip>`.

```
<image> <position> / <size> <repeat> <origin> <clip> <composite> <mode>
```

Both accept `<geometry-box>` keywords. In addition, `mask-clip` also accepts `no-clip`. Because of this, the order of these two matters when you want to set `mask-clip` to any value other than `no-clip`.

- If one `<geometry-box>` value is present along with the `no-clip` keyword, then the `<geometry-box>` sets the `mask-origin` value, and `mask-clip` is set to `no-clip`. In this case, the order doesn't matter.
- If only one `<geometry-box>` value is present and there is no `no-clip` keyword, both the `mask-origin` and `mask-clip` components are set to that value. As there is only one value, again the ordering doesn't matter.
- If two `<geometry-box>` values are present, the first sets the `mask-origin` component and the second sets the `mask-clip` component. In this case, the order is very important.

Setting the incorrect order for the `mask-origin` and `mask-clip` values may affect the appearance, but will not cause the declaration to fail.

Ordering rules for <code>mask-size</code> and <code>mask-position</code>

You may have noticed a slash between `mask-position` and `mask-size`, listed in the syntax as `<position>` and `<size>`. Both properties accept similar values.

```
<image> <position> / <size> <repeat> <origin> <clip> <composite> <mode>
```

In this case, the order is very important. If only one or a pair of `<length-percentage>` values are present, it will define the position of the image rather than the size. Including both a position and a size in a mask layer without including the slash between the two will invalidate the entire declaration.

```
mask:
  url("star.svg") bottom 2em right 4em / auto 2vw no-repeat padding-box
    content-box luminance,
  url("circle.svg") 100px 100px / 50% repeat-x border-box padding-box alpha;
```

If a single pair of `<length-percentage>` values is present, it sets the `mask-position` property, and the `mask-size` will be `auto`. If a layer includes both a `mask-size` and a `mask-position`, the `mask-size` property value must come after the `mask-position` property value and the values must be separated by a forward slash (`/`). The slash is required even if the `mask-size` is set to a value that is not a valid `mask-position` value.

```
mask: url("star.svg") contain;
mask: url("star.svg") 10px 10px cover;
mask: url("star.svg") top right 100px 100px;
```

```
mask: url("star.svg") 10px 10px / cover;
mask: url("star.svg") top 100px right 100px;
mask: url("star.svg") top right / 100px 100px;
```

To include a `mask-size` in a mask layer using the `mask` shorthand, you must include a `mask-position` value with a slash immediately before it.

> **Warning:** If you include a size in a mask layer but forget the slash after the position, the entire declaration will become invalid.

See also

- Introduction to CSS masking
- CSS mask properties
- Introduction to CSS clipping

- CSS masking module

CSS media queries

The **CSS media queries** module enables testing and querying of viewport values and browser or device features, to conditionally apply CSS styles based on the current user environment. Media queries are used in the CSS `@media` rule and other contexts and languages such as HTML and JavaScript.

Media queries are a key component of responsive design. They enable conditional setting of CSS styles depending on the presence or value of device characteristics. It's common to use a media query based on viewport size to set appropriate layouts on devices with different screen sizes — for example three columns on a wide screen or a single column on a narrow screen.

Other common examples include increasing the font size and hiding navigation menus when printing a page, adjusting the padding between paragraphs when a page is viewed in portrait or landscape mode, or increasing the size of buttons to provide a larger hit area on touchscreens.

In CSS, use the `@media` at-rule to conditionally apply part of a style sheet based on the result of a media query. To conditionally apply an entire style sheet, use `@import`.

When designing reusable HTML components, you may also use container queries, which allow you to apply styles based on the size of a containing element rather than the viewport or other device characteristics.

Reference

At-rules and descriptors

- `@import`
- `@custom-media`
- `@media`
  - `any-hover`
  - `any-pointer`
  - `aspect-ratio`
  - `color`
  - `color-gamut`
  - `color-index`
  - `device-aspect-ratio`
  - `device-height`
  - `device-width`
  - `display-mode`
  - `dynamic-range`
  - `forced-colors`
  - `grid`
  - `height`
  - `horizontal-viewport-segments`
  - `hover`
  - `inverted-colors`
  - `monochrome`
  - `orientation`
  - `overflow-block`
  - `overflow-inline`
  - `pointer`
  - `prefers-color-scheme`
  - `prefers-contrast`
  - `prefers-reduced-data`
  - `prefers-reduced-motion`
  - `prefers-reduced-transparency`
  - `resolution`
  - `scan`
  - `scripting`
  - `update`
  - `vertical-viewport-segments`
  - `video-dynamic-range`
  - `width`

The CSS media queries level 5 module also introduces the `environment-blending`, `nav-controls`, and `video-color-gamut` `@media` descriptors. Currently, no browsers support these features.

> **Note:** CSS media queries level 4 deprecated three `@media` descriptors: `device-aspect-ratio`, `device-height`, and `device-width`.

Data types and operators

- `<media-types>`
- `<media-features>`
- `<resolution>`
- Logical operators

Glossary terms

- media
- media query

Guides

Using media queries

Introduces media queries, their syntax, and the operators and media features used to construct media query expressions.

Learn: Media query fundamentals

Introduction to media queries and approaches for using them to create responsive designs.

Testing media queries

Describes how to use media queries in your JavaScript code to determine the state of a device, and to set up listeners that notify your code when the results of media queries change (such as when the user rotates the screen or resizes the browser).

Using media queries for accessibility

Learn how media queries can help users understand your website better.

Printing

Tips and techniques for helping improve web content printer output.

Responsive images

Learn how to use media queries with `sizes` to implement responsive image solutions on websites.

Related concepts

- CSS containment module
  - `@container` at-rule
  - Using container queries
  - Using size and style container queries
- CSS conditional rules module
  - `@supports` at-rule
  - Using feature queries
- CSS environment variables
  - `env()` function
- CSS paged media module
  - `@page` at-rule
- CSS object model module
  - `MediaQueryList` interface
    - `matches` property
    - `media` property
    - `change` event
  - `MediaList` interface
    - `mediaText` property
  - `MediaQueryListEvent` object
- Device Posture API
  - `device-posture` descriptor
- HTML
  - `sizes` attribute for `<img>`, `<link>`, and `<source>` for `<picture>`
  - `media` attribute for `<link>`, `<source>`, and `<style>` HTML
  - `<meta name="viewport">`
- SVG `media` attribute

See also

- Container queries
- Using the `srcset` and `sizes` attributes
- CSS paged media
- Use `@supports` to apply styles that depend on browser support for various CSS technologies.

Printing

There may be times in which your website or application would like to improve the user's experience when printing content. There are several possible scenarios:

- You wish to adjust layout to take advantage of the size and shape of the paper.
- You wish to use different styles to enhance the appearance of your content on paper.
- You wish to use higher resolution images for a better result.
- You want to adjust the user experience of printing, such as presenting a specially-formatted version of your content before printing begins.

There may be other cases in which you want to manage the printing process, but these are some of the most common scenarios. This article provides tips and techniques for helping your web content print better.

Using a print style sheet

Add the following to your `<head>` tag.

```
<link href="/path/to/print.css" media="print" rel="stylesheet" />
```

Using media queries and @page to control printed content

You can use the CSS `@media` at-rule to set different styles for your webpage when it is printed on paper or as a PDF versus when it is displayed on the screen. The `print` media type sets the styles for printed media; these styles will only be used for printed content.

Add this at the end of your stylesheet. Note that specificity and precedence rules still apply:

```
@media print {
  /* All your print styles go here */
  #header,
  #footer,
  #nav {
    display: none !important;
  }
}
```

You can also use the `@page` at-rule to modify different aspects of printed pages including the page's dimensions, orientation, and margins. The `@page` at-rule can be used to target all pages in a print-out or just a specific subset of pages

Detecting print requests

Browsers send `beforeprint` and `afterprint` events to determine when printing may have occurred. You can use this to adjust the user interface presented during printing (for example displaying or hiding user interface elements during the print process).

Examples

Here are some common examples.

Automatically close the window when finished

The following example will close the window after the user has printed its contents:

```
window.addEventListener("afterprint", () => self.close);
```

Print an external page without opening it

If you want to be able to print an external page without opening it, you can utilize a hidden `<iframe>` (see: HTMLIFrameElement), automatically removing it after the user prints its contents. The following is a possible example which will print a file named `externalPage.html`:

HTML

```
<button id="print_external">Print external page!</button>
```

JavaScript

```
function setPrint() {
  const closePrint = () => {
    document.body.removeChild(this);
  };
  this.contentWindow.onbeforeunload = closePrint;
  this.contentWindow.onafterprint = closePrint;
  this.contentWindow.print();
}

document.getElementById("print_external").addEventListener("click", () => {
  const hideFrame = document.createElement("iframe");
  hideFrame.onload = setPrint;
  hideFrame.style.display = "none"; // hide iframe
  hideFrame.src = "external-page.html";
  document.body.appendChild(hideFrame);
});
```

See also

- `window.print`
- `beforeprint` event
- `afterprint` event
- Media queries
- `@media`
- CSS paged media module

Testing media queries programmatically

The DOM provides features that can test the results of a media query programmatically, via the `MediaQueryList` interface and its methods and properties. Once you've created a `MediaQueryList` object, you can check the result of the query or receive notifications when the result changes.

Creating a media query list

Before you can evaluate the results of a media query, you need to create the `MediaQueryList` object representing the query. To do this, use the `window.matchMedia` method.

For example, to set up a query list that determines if the device is in landscape or portrait orientation:

```
const mediaQueryList = window.matchMedia("(orientation: portrait)");
```

Checking the result of a query

Once you've created your media query list, you can check the result of the query by looking at the value of its `matches` property:

```
if (mediaQueryList.matches) {
  /* The viewport is currently in portrait orientation */
} else {
  /* The viewport is not currently in portrait orientation, therefore landscape */
}
```

Receiving query notifications

If you need to be aware of changes to the evaluated result of the query on an ongoing basis, it's more efficient to register a listener than to poll the query's result. To do this, call the `addEventListener()` method on the `MediaQueryList` object, with a callback function to invoke when the media query status changes (e.g., the media query test goes from `true` to `false`):

```
// Create the query list.
const mediaQueryList = window.matchMedia("(orientation: portrait)");

// Define a callback function for the event listener.
function handleOrientationChange(mql) {
  // …
}

// Run the orientation change handler once.
handleOrientationChange(mediaQueryList);

// Add the callback function as a listener to the query list.
mediaQueryList.addEventListener("change", handleOrientationChange);
```

This code creates the orientation-testing media query list and adds an event listener to it. After defining the listener, we also call the listener directly. This makes our listener perform adjustments based on the current device orientation; otherwise, our code might assume the device is in portrait mode at startup, even if it's actually in landscape mode.

The `handleOrientationChange()` function would look at the result of the query and handle whatever we need to do on an orientation change:

```
function handleOrientationChange(evt) {
  if (evt.matches) {
    /* The viewport is currently in portrait orientation */
  } else {
    /* The viewport is currently in landscape orientation */
  }
}
```

Above, we define the parameter as `evt` — an event object of type `MediaQueryListEvent` that also includes the `media` and `matches` properties, so you can query these features of the `MediaQueryList` by directly accessing it, or accessing the event object.

Ending query notifications

To stop receiving notifications about changes to the value of your media query, call `removeEventListener()` on the `MediaQueryList`, passing it the name of the previously-defined callback function:

```
mediaQueryList.removeEventListener("change", handleOrientationChange);
```

See also

- Media queries
- CSS media queries module
- CSS object model module
- `window.matchMedia()`
- `MediaQueryList`
- `MediaQueryListEvent`

Using media queries

Media queries allow you to apply CSS styles depending on a device's media type (such as print vs. screen) or other features or characteristics such as screen resolution or orientation, aspect ratio, browser viewport width or height, user preferences such as preferring reduced motion, data usage, or transparency.

Media queries are used for the following:

- To conditionally apply styles with the CSS `@media`, `@custom-media` and `@import` at-rules.
- To target specific media for the `<style>`, `<link>`, `<source>`, and other HTML elements with the `media=` or `sizes=` attributes.
- To test and monitor media states using the `Window.matchMedia()` and `EventTarget.addEventListener()` methods.

> **Note:** The examples on this page use CSS's `@media` for illustrative purposes, but the basic syntax remains the same for all types of media queries.

Syntax

A media query is composed of an optional *media type* and any number of *media feature* expressions, which may optionally be combined in various ways using *logical operators*. Media queries are case-insensitive.

- Media types define the broad category of device for which the media query applies: `all`, `print`, `screen`.

  The type is optional (assumed to be `all`) except when using the `only` logical operator.

- Media features describe a specific characteristic of the user agent, output device, or environment:

  - `any-hover`
  - `any-pointer`
  - `aspect-ratio`
  - `color`
  - `color-gamut`
  - `color-index`
  - `device-aspect-ratio`
  - `device-height`
  - `device-posture`
  - `device-width`
  - `display-mode`
  - `dynamic-range`
  - `forced-colors`
  - `grid`
  - `height`
  - `hover`
  - `inverted-colors`
  - `monochrome`
  - `orientation`
  - `overflow-block`
  - `overflow-inline`
  - `pointer`
  - `prefers-color-scheme`
  - `prefers-contrast`
  - `prefers-reduced-motion`
  - `prefers-reduced-transparency`
  - `resolution`
  - `scripting`
  - `update`
  - `video-dynamic-range`
  - `width`

  For example, the `hover` feature allows a query to check whether the device supports hovering over elements. Media feature expressions test for their presence or value, and are entirely optional. Each media feature expression must be surrounded by parentheses.

- Logical operators can be used to compose a complex media query: `not`, `and`, and `only`. You can also combine multiple media queries into a single rule by separating them with commas.

A media query computes to `true` when the media type (if specified) matches the device on which a document is being displayed *and* all media feature expressions compute as true. Queries involving unknown media types are always false.

> **Note:** A style sheet with a media query attached to its `<link>` tag will still download even if the query returns `false`, the download will happen but the priority of downloading will be much lower. Nevertheless, its contents will not apply unless and until the result of the query changes to `true`. You can read why this happens in Tomayac's blog Why Browsers Download Stylesheets With Non-Matching Media Queries.

Targeting media types

Media types describe the general category of a given device. Although websites are commonly designed with screens in mind, you may want to create styles that target special devices such as printers or audio-based screen readers. For example, this CSS targets printers:

```
@media print {
  /* … */
}
```

You can also target multiple devices. For instance, this `@media` rule uses two media queries to target both screen and print devices:

```
@media screen, print {
  /* … */
}
```

See media types for the list of available media types. Because media types describe devices in very broad terms, most of the originally-defined media types were deprecated, with just `screen`, `print`, and `all` remaining. To target more specific attributes, use *media features* instead.

Targeting media features

Media features describe the specific characteristics of a given user agent, output device, or environment. For instance, you can apply specific styles to widescreen monitors, computers that use mice, or devices that are being used in low-light conditions. This example applies styles when the user's *primary* input mechanism (such as a mouse) can hover over elements:

```
@media (hover: hover) {
  /* … */
}
```

Media features are either range or discrete.

Discrete features take their value from an enumerated set of possible keyword values. For example, the discrete `orientation` feature accepts either `landscape` or `portrait`.

```
@media print and (orientation: portrait) {
  /* … */
}
```

Many *range features* can be prefixed with "min-" or "max-" to express "minimum condition" or "maximum condition" constraints. For example, this CSS will apply styles only if your browser's viewport width is equal to or narrower than 1250px:

```
@media (max-width: 1250px) {
  /* … */
}
```

The following media queries are equivalent to the above example:

```
@media (width <= 1250px) {
  /* … */
}

@media (1250px >= width) {
  /* … */
}
```

With media query range features, you can either use the inclusive `min-` and `max-` prefixes or the more concise range syntax operators `<=` and `>=`.

The following media queries are equivalent:

```
@media (min-width: 30em) and (max-width: 50em) {
  /* … */
}

@media (30em <= width <= 50em) {
  /* … */
}

@media (50em >= width >= 30em) {
  /* … */
}
```

The range comparisons above are inclusive. To exclude the comparison value, use `<` and/or `>`.

```
@media (30em < width < 50em) {
  /* … */
}

@media (50em > width > 30em) {
  /* … */
}
```

If you create a media feature query without specifying a value, the nested styles will be used as long as the feature's value is not `0` or `none`. For example, this CSS will apply to any device with a color screen:

```
@media (color) {
  /* … */
}
```

If a feature doesn't apply to the device on which the browser is running, expressions involving that media feature are always false.

For more Media feature examples, please see the reference page for each specific feature.

Creating complex media queries

Sometimes you may want to create a media query that depends on multiple conditions. This is where the *logical operators* come in: `not`, `and`, and `only`. Furthermore, you can combine multiple media queries into a comma-separated list; this allows you to apply the same styles in different situations, with the contained media queries evaluated as a logical `or` composition: interpreted as if each media query were within parentheses with an `or` between them.

In the previous example, we saw the `and` operator used to group a media *type* with a media *feature*. The `and` operator can also combine multiple media features within a single media query. The `not` operator negates a media query, or a media feature when used with brackets, basically reversing their normal meanings. The `or` operator can, under certain conditions, be used to combine multiple media features within a single media query. Lastly, the `only` operator was used to prevent older browsers from applying the styles without evaluating the media feature expressions but it has no effect in modern browsers.

> **Note:** In most cases, the `all` media type is used by default when no other type is specified. However, if you use the `only` operator, you must explicitly specify a media type. You can see `only screen` or `only print` as a whole.

Combining multiple types or features

The `and` keyword combines a media feature with a media type *or* other media features. This example combines two media features to restrict styles to landscape-oriented devices with a width of at least 30 ems:

```
@media (width >= 30em) and (orientation: landscape) {
  /* … */
}
```

To limit the styles to devices with a screen, you can chain the media features to the `screen` media type:

```
@media screen and (width >= 30em) and (orientation: landscape) {
  /* … */
}
```

Testing for multiple queries

You can use a comma-separated list of media queries to apply styles when the user's device matches any one of various media types, features, or states.

The following rule contains two media queries. The block's styles will apply if either the user's device has a height of 680px or more *or* if the browser viewport is in portrait mode (the viewport height is greater than the viewport width):

```
@media (height >= 680px), screen and (orientation: portrait) {
  /* … */
}
```

In this example, if the user is printing to a PDF and the page height is 800px, the media query returns true because the first query component — which tests whether the viewport has a height of `680px` or more — is true. Likewise, if a user is on a smartphone in portrait mode with a viewport height of 480px, the media query returns true because the second query component is true.

In a comma-separated list of media queries, the individual media queries end at the comma or, in the case of the last media query in the list, at the opening bracket ({).

Inverting a query's meaning

The `not` keyword inverts the meaning of a single media query. For example, the CSS styles in this media query will apply to everything *except* printed media:

```
@media not print {
  /* … */
}
```

The `not` negates only the media query it is applied to. The `not`, without parenthesis, negates all the features within the media query in which it is contained. This means, in a comma-separated list of media queries, each `not` applies to the single query it is contained within, applying to *all* the features within that single query. In this example, the `not` applies to the first media query `screen and (color)`, which concludes at the first comma:

```
@media not screen and (color), print and (color) {
  /* … */
}
```

Because the query starts with a media type `screen`, you *cannot* wrap `screen and (color)` with parentheses. On the other hand, if your media query consists of features only, then you *must* parenthesize the query:

```
@media not ((width > 1000px) and (color)), print and (color) {
  /* … */
}
```

Parentheses limit the components of the query that get negated. For example, to negate the `(width > 1000px)` query only:

```
@media (not (width > 1000px)) and (color), print and (color) {
  /* … */
}
```

`not` only negates the query to its right. In this example, we negate the `hover` media feature but not the `screen` media type:

```
@media screen and not (hover) {
  /* … */
}
```

The `not (hover)` matches if the device has no hover capability. In this case, because of its ordering, the `not` applies to `hover` but not to `screen`.

Improving compatibility with older browsers

The `only` keyword prevents older browsers that do not support media queries with media features from applying the given styles. *It has no effect on modern browsers.*

```
@media only screen and (color) {
  /* … */
}
```

Testing for multiple features with <code>or</code>

You can use `or` to test for a match among more than one feature, resolving to `true` if any of the features are true. For example, the following query tests for devices that have a monochrome display or hover capability:

```
@media (not (color)) or (hover) {
  /* … */
}
```

Note that you cannot use the `or` operator on the same level as the `and` and `not` operators. You can either separate the media features with a comma or use parenthesis to group sub-expressions of media features to clarify the order of evaluation.

For example, the following queries are both valid:

```
@media ((color) and (hover)) or (monochrome) {
  /* … */
}

/* or */
@media (color) and (hover), (monochrome) {
  /* … */
}
```

See also

- @media
- Container queries
- Testing media queries programmatically
- CSS Animations Between Media Queries
- Extended Mozilla media features
- Extended WebKit media features

Using media queries for accessibility

CSS media queries can be used to help users with disabilities better experience your website.

Reduced Motion

Blinking and flashing animation can be problematic for people with cognitive concerns such as Attention Deficit Hyperactivity Disorder (ADHD). Additionally, certain kinds of motion can be a trigger for Vestibular disorders, epilepsy, and migraine and Scotopic sensitivity. Reducing animations or switching animation off completely based on the user's preference can also benefit users with low battery or low-end devices.

The `prefers-reduced-motion` media query enables providing an experience with fewer animations and transitions to users who have set their operating system's accessibility preferences to reduce motion. It has two possible values:

`no-preference`

Indicates that the user has made no preference known to the system.

`reduce`

Indicates that user has notified the system that they prefer an interface that minimizes the amount of movement or animation, preferably to the point where all non-essential movement is removed.

Example

This example has an annoying animation unless you turn on Reduce Motion in your accessibility preferences.

HTML

```
<div class="animation">animated box</div>
```

CSS

```
.animation {
  animation: vibrate 0.3s linear infinite both;
}

@media (prefers-reduced-motion: reduce) {
  .animation {
    animation: none;
  }
}
```

The value of `prefers-reduced-motion` is `reduce`, not "none". This preference does not mean all animations must be removed, which could be achieved with `* {animation: none !important;}`. Rather, users expect motion animation, including those triggered by user interaction, to be disabled unless the animation is essential to the functionality or the information being conveyed (see WCAG: Animation from Interactions).

See also

- `prefers-contrast`: to adjust page styles based on user's contrast preference
- `prefers-reduced-transparency`
- `prefers-color-scheme`
- `inverted-colors`
- Designing With Reduced Motion For Motion Sensitivities

CSS motion path

The **CSS motion path** module allows authors to animate any graphical object along a custom path.

The CSS transforms module provides features that enable boxes to be repositioned, rotated, scaled, and skewed relative to its laid out position, without disrupting the layout of any other elements on the page. These transforms can be animated and transitioned, but only in relatively basic ways.

The features of the CSS motion path module provide offset transforms: transforms that align a point on an element to an offset distance along an offset path, optionally rotating the transformed element to follow the path direction. This module enables powerful transform possibilities, for example:

- Positioning using polar coordinates rather than limiting transformation to the standard rectangular `transform` coordinates.
- Animating an element along a defined path.

CSS motion paths allow the definition of complex 2D spatial transitions by leveraging CSS shape functions.

For example, you can define a specific path of any shape you want with the `offset-path` property. You can then animate an element to move along that path by animating the `offset-distance` property, and rotate it at any point with the `offset-rotate` property.

Motion paths in action

```
<div id="heart">
  <div id="motion-demo"></div>
</div>
```

```
#motion-demo {
  offset-path: path(
    "M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
  );
  animation: move 3000ms infinite linear;
```

```
  width: 10px;
  height: 10px;
  background: red;
}

#heart {
  width: 200px;
  height: 200px;
  background-color: lightpink;
  clip-path: path(
    "M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
  );
}

@keyframes move {
  100% {
    offset-distance: 100%;
  }
}
```

In this example, we used CSS masking and CSS shapes to clip a container with a light pink background into a heart shape. We used a `path()` function as the value of the `clip-path` property. Its child is a `10px` by `10px` red box that is made to follow the edge of its parent. We did this by using the same `<basic-shape>` as the path, setting the box's `offset-path` property to the same `path()` function value. We used CSS animations to change the `offset-distance` from `0%` to `100%` over three seconds.

Reference

Properties

- `offset` shorthand
- `offset-anchor`
- `offset-distance`
- `offset-path`
- `offset-position`
- `offset-rotate`

Functions

- `ray()`

Guides

Using CSS animations

Step-by-step tutorial on how to create animations using CSS.

Related concepts

CSS transforms module

- `transform`
- `transform-origin`
- `translate`

CSS masking module

- `clip-path`
- `clip-rule`

CSS shapes module

- `<basic-shape>`
- `circle()`
- `ellipse()`
- `inset()`
- `path()`
- `polygon()`
- `rect()`
- `shape()`
- `xywh()`

CSS animations module

- `animation` shorthand
- `@keyframes`

CSS box model module

- `<coord-box>`

See also

- `<position>`
- `<easing-function>`
- `radial-gradient()` function
- `prefers-reduced-motion` media query
- `will-change` CSS property

CSS multi-column layout

The **CSS multi-column layout** module lets you divide content across multiple columns. By using the properties in this module, you can define the preferred number and width of columns, the gap size between columns, and the visual appearance of the optional column dividing lines (known as column rules). You can also define how content should flow from column to column and how to break content between columns.

Multi-column layout in action

In this example, the 1967 speech from Canada's centennial, *A Lament for Confederation*, by Chief Dan George, is displayed across multiple columns, similar to the way articles are displayed in printed newspapers. If you have JavaScript enabled, controls enable changing the preferred column number and width, the width of the gap between columns, whether the title and a sample blockquote should be contained in a

single column or made to span all columns, and whether breaking within the paragraphs should be avoided.

```
<article>
  <div class="title">
    <h1>A Lament for Confederation</h1>
    <p>&mdash;Chief Dan George, 1967</p>
  </div>
  <p>
    How long have I known you, Oh Canada? A hundred years? Yes, a hundred years.
    And many, many tides more. And today, when you celebrate your hundred years,
    Oh Canada, I am sad for all the Indian people throughout the land.
  </p>
  <p>
    For I have known you when your forests were mine; when they gave me my meat
    and my clothing. I have known you in your fruits and rivers where your fish
    flashed and danced in the sun, where the waters said 'come and eat of my
    abundance.' I have known you in the freedom of your winds. And my spirit,
    like your winds, once roamed this good lands.
  </p>
  <blockquote>
    <p>
      But in long the hundred years since the white man came, I have seen that
      freedom disappear just like the salmon going mysteriously out to sea.
    </p>
  </blockquote>
  <p>
    The white man's strange customs I could not understand, pressed down upon me
    until I could no longer breathe.
  </p>
  <p>
    When I fought to protect my home and my land, I was called a savage. When I
    neither understood nor welcomed this new way of life, I was called lazy.
    When I tried to rule my people, I was stripped of my authority.
  </p>
  <p>
    My nation was ignored in your history textbooks – we were less important in
    the history of Canada than the buffalo that ranged the plains. I was
    ridiculed in your plays and motion pictures, and when I drank your
    fire-water, I got drunk – very, very drunk. And I forgot.
  </p>
  <p>
    Oh Canada, how can I celebrate with you this centenary, this hundred years?
    Shall I thank you for the reserves that are left me of my beautiful forests?
    Shall I thank you for the canned fish of my rivers? Shall I thank you for
    the loss of my pride and authority, even among my own people? For the lack
    of my will to fight back? No! I must forget what is past and gone.
  </p>
  <p>
    Oh God in heaven! Give me the courage of the olden chiefs. Let me wrestle
    with my surroundings. Let me once again, as in the days of old, dominate my
    environment. Let me humbly accept this new culture and through it rise up
    and go on.
  </p>
  <p>
    Oh god, like the thunderbird of old, I shall rise again out of the sea. I
    shall grab the instruments of the white man's success – his education, his
    skills, and with these new tools I shall build my race into the proudest
    segment of your society. And, before I follow the great chiefs who have gone
    before us, I shall see these things come to pass.
  </p>
  <p>
    I shall see our young braves and our chiefs sitting in the house of law and
    government, ruling and being ruled by the knowledge and freedoms of our
    great land. So shall we shatter the barriers of our isolation. So shall the
    next hundred years be the greatest in the proud history of our tribes and
    nations.
  </p>
</article>
<fieldset id="options" class="open">
  <legend>
    <button aria-expanded="true" aria-controls="controls">
      Column options
    </button>
  </legend>
  <div id="controls">
    <p>
      <input type="range" min="0" max="5" value="5" id="colCount" />
      <label for="colCount">Max # of columns:</label>
    </p>
    <p>
      <input type="range" min="2" max="40" value="10" step="2" id="colWidth" />
      <label for="colWidth">Min width of columns:</label>
    </p>
    <p>
      <input type="range" min="0" max="10" value="1" step="0.5" id="gapSize" />
      <label for="gapSize">Gap size:</label>
    </p>
    <p>
      <input type="checkbox" checked id="colSpan" />
      <label for="colSpan">Byline spans all columns</label>
    </p>
    <p>
      <input type="checkbox" id="blockSpan" />
      <label for="blockSpan">Blockquote spans all columns</label>
    </p>
    <p>
      <input type="checkbox" id="colFill" />
      <label for="colFill">Balance columns</label>
    </p>
    <p>
      <input type="checkbox" id="break" />
      <label for="break">Don't break paragraphs</label>
    </p>
```

```
  </div>
</fieldset>
```

```
const page = document.querySelector("article");
const title = document.querySelector(".title");
const options = document.querySelector("#options");
const legend = document.querySelector("#options > legend");
const legendBtn = document.querySelector("#options > legend > button");
const blockquote = document.getElementsByTagName("blockquote")[0];

const colCount = document.getElementById("colCount");
const colWidth = document.getElementById("colWidth");
const gapSize = document.getElementById("gapSize");
const colSpan = document.getElementById("colSpan");
const blockSpan = document.getElementById("blockSpan");
const colFill = document.getElementById("colFill");
const breakP = document.getElementById("break");

// make options visible if js is enabled
options.style.display = "revert";

legendBtn.addEventListener("click", () => {
  showAndHideMenu();
});

colCount.addEventListener("change", () => {
  page.style.columnCount = colCount.value;
});

colWidth.addEventListener("change", () => {
  page.style.columnWidth = `${colWidth.value}em`;
});

gapSize.addEventListener("change", () => {
  page.style.gap = `${gapSize.value}em`;
});

colSpan.addEventListener("change", () => {
  setColSpan(colSpan, title);
});

blockSpan.addEventListener("change", () => {
  setColSpan(blockSpan, blockquote);
});

colFill.addEventListener("change", () => {
  if (colFill.checked) {
    page.style.columnFill = "balance";
  } else {
    page.style.columnFill = "auto";
  }
});

breakP.addEventListener("change", () => {
  if (breakP.checked) {
    page.classList.add("breakInside");
  } else {
    page.classList.remove("breakInside");
  }
});

function showAndHideMenu() {
  if (legendBtn.getAttribute("aria-expanded") === "true") {
    // close it
    legendBtn.setAttribute("aria-expanded", "false");
    legend.classList.add("closed");
    legend.classList.remove("open");
  } else {
    // open it
    legendBtn.setAttribute("aria-expanded", "true");
    legend.classList.remove("closed");
    legend.classList.add("open");
  }
}

function setColSpan(control, element) {
  if (control.checked) {
    element.style.columnSpan = "all";
  } else {
    element.style.columnSpan = "none";
  }
}
```

```
article {
  column-count: 5;
  column-width: 8em;
  widows: 3;
  orphans: 3;
  gap: 1em;
  column-rule: 2px dashed #666666;
  height: 50em;
}
.title {
  column-span: all;
  display: flex;
  align-items: baseline;
  gap: 1em;
  flex-wrap: wrap;
}
p {
  margin: 0 0 1em 0;
  line-height: 1.4;
```

```
}
.breakInside * {
  break-inside: avoid;
}
#options {
  position: fixed;
  top: 1rem;
  right: 1rem;
  background: white;
  display: none;
}
fieldset p {
  margin-bottom: 0.25em;
}
legend {
  position: relative;
  top: 0;
  transition: 200ms;
}
legend.closed {
  top: 0.75em;
}
legend.closed + #controls {
  display: none;
}
legend {
  background-color: #dedede;
  padding: 0.5em;
}
legend > button {
  all: unset;
  cursor: pointer;
}
legend.closed {
  margin: -1em;
  display: inline-block;
}
blockquote {
  border: 2px dashed red;
  margin: 0 0 0.25em 0;
}
blockquote p {
  margin-bottom: 0;
}
blockquote p::before,
blockquote p::after {
  content: '"';
  font-size: 1.2em;
  vertical-align: baseline;
  color: red;
}
```

> **Note:** Multiple-column layout is closely related to paged media. Each column box is a fragment, much like each printed page is a fragment of a document. Using the properties defined in the CSS fragmentation module, you can control how content breaks between columns and pages.

Reference

Properties

- `column-fill`
- `column-span`
- `columns` shorthand
  - `column-count`
  - `column-height`
  - `column-width`
- `column-wrap`

> **Note:** Bear in mind that setting container height and line length can pose challenges for people with visual or cognitive disabilities. WCAG Success Criterion 1.4.8 states that even when the text size is doubled, content should not need to be scrolled.

Selectors and pseudo-elements

- `::column`

Guides

Basic concepts of multi-column layouts

Overview of the multiple-column layout specification.

Using multi-column layouts

Guide to using multi-column properties for laying out text.

Styling columns

Guide to styling columns and managing spacing between columns.

Spanning and balancing

How to make elements span across all columns and control the way columns are filled.

Handling overflow in multi-column layout

What happens when an item overflows the column it is in and what happens when there is too much columned content to fit a container.

Handling content breaks in multi-column layout

Introduction to the fragmentation specification and how to control where column content breaks.

Creating CSS carousels

Create pure-CSS carousel UI features using scroll buttons, scroll markers, and generated columns.

Defining CSS gaps

Understanding and defining gaps in grid, flexbox, and multi-column layouts, including how percentage values are resolved.

Related concepts

CSS gaps module

- `column-rule` shorthand
  - `column-rule-color`
  - `column-rule-style`
  - `column-rule-width`
- `gap` shorthand
  - `column-gap`
  - `row-gap`
- `row-rule`
- `rule` shorthand
- `rule-break`
- `rule-color`
- `rule-inset`
- `rule-overlap`
- `rule-style`
- `rule-width`

CSS fragmentation module

- `break-after`
- `break-before`
- `break-inside`
- `orphans`
- `widows`

CSS box alignment module

- `gap`

CSS box sizing module

- `height`
- `max-height`
- `block-size`
- `width`
- `max-width`

CSS overflow module

- `overflow`
- `::scroll-marker`
- `::scroll-marker-group`
- Scroll container glossary term

CSS display module

- Block formatting context guide

See also

- Learn: multiple-column layout
- CSS fragmentation module
- CSS flexible box layout module
- CSS grid layout module
- CSS paged media module

Basic concepts of multi-column layouts

Multi-column layout, usually referred to as multicol layout, is a specification for laying out content into a set of column boxes much like columns in a newspaper. This guide explains how the specification works with some common use case examples.

Key properties

Multicol layout is unlike any of the other layout methods in CSS; it fragments the content, including all descendant elements, into columns. This happens in the same way that content is fragmented into pages when we work with CSS paged media by creating a print stylesheet.

In this and subsequent guides, we will be discussing the following properties defined in the CSS multi-column layout module:

- `column-width`
- `column-count`
- `columns` shorthand
- `column-rule-color`
- `column-rule-style`
- `column-rule-width`
- `column-rule` shorthand
- `column-span`
- `column-fill`
- `column-gap`
- `break-after`
- `break-before`
- `break-inside`

Defining columns

By adding the `column-count` or the `column-width` property to an element, or using the `columns` shorthand, the element becomes a *multi-column container* or *multicol container* for short. The columns are anonymous boxes; they're described as *column boxes* in the specification.

Specifying the number of columns

The `column-count` property specifies the number of columns that you would like the content to be displayed as. The browser will then assign the correct amount of space to each column box to create the requested number of columns.

In the below example, we use the `column-count` property to create three columns on the `.container` element. The content, including the children of `.container`, is then split between the three columns.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>

  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>

  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-count: 3;
}
```

In the above example, the content is wrapped within the paragraph `<p>` tags with the default styling. Therefore, there is a margin above each paragraph. You can see how this margin causes the first line of text to be pushed down. This is because a multicol container creates a block formatting context (BFC) because of which margins on child elements do not collapse with any margin on the container.

Specifying the width of columns

The `column-width` property is used to set the optimal width for every column box. If you declare a column width, the browser will work out how many columns of that width will fit into the multicol container and distribute any extra space equally between the columns. Therefore, the column width should be seen as minimum width because the column boxes are likely to be wider due to the additional space.

In the case of a single column with less width available than the value of `column-width`, the column box will shrink to be smaller than the declared column width.

In the example below, the `column-width` property is set to `200px`. We get as many 200-pixel columns as will fit the container, with the extra space shared equally.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>

  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>

  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 200px;
}
```

Specifying both number and width of columns

If you specify both the properties on a multicol container, then `column-count` will act as a maximum number of columns. Therefore, the behavior as described for `column-width` will happen, until the number of columns in `column-count` is reached. After this point, no more columns will be drawn, and the extra space is distributed evenly between the existing columns, even if there is enough room for more columns of the specified `column-width` size.

When using both properties together, you may get fewer columns than specified in the value for `column-count`.

In this next example, we use `column-width` of `200px` and `column-count` of `2`. Even if there is space for more than two columns, we get two. If there is not enough space for two columns of at least 200 pixels each, we get one.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>

  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
```

```
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>

  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko. .
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-count: 2;
  column-width: 200px;
}
```

Shorthand for column properties

You can use the `columns` shorthand to set the `column-count` and `column-width` values. If you specify a length unit, the value will be used for `column-width`, and if you specify an integer, the value will be used for `column-count`. You can set both the properties, separating the two values with a space.

This CSS would give the same result as the first example, with `column-count` set to `3`.

```
.container {
  columns: 3;
}
```

This CSS would give the same result as the second example, with `column-width` of `200px`.

```
.container {
  columns: 200px;
}
```

This CSS would give the same result as the third example, with both `column-count` and `column-width` set.

```
.container {
  columns: 2 200px;
}
```

Next steps

In this guide, we've learned the basic use of multi-column layout. In the next guide, we will look at how much we can style the columns themselves.

Handling content breaks in multi-column layout

Content between column boxes in a multicol layout breaks in the same way that it breaks between pages in paged media. In both contexts, you can control where and how content breaks by using properties of the CSS fragmentation module. In this guide, we see how fragmentation works in a *multi-column container* or *multicol container* for short.

Fragmentation basics

The CSS fragmentation module provides details on how content breaks between the fragmentation containers or *fragmentainers*. The multi-column layout module, on the other hand, defines the `break-after`, `break-before`, and `break-inside` properties that provide some control within and between columns. In multicol layout, a column box is a fragment container.

A column box can contain other markup and there are many places where a break would not be ideal. For example, we would generally prefer that the caption of an image not be separated into a new column away from the image it refers to. Also, ending a column with a heading looks strange. The multicol fragmentation properties give us ways to exercise some control over this.

There are various places we might want to control our breaks:

- Breaks inside boxes, for example inside a figure element.
- Breaks before and after boxes, which would include our heading example above.
- Breaks between lines.

Breaks inside boxes

To control breaks inside boxes use the `break-inside` property. This property takes values of:

- `auto`
- `avoid`
- `avoid-page`
- `avoid-column`
- `avoid-region`

In the example below, we have applied break-inside to the figure element to prevent the caption from becoming separated from the image.

```
<div class="container">
  <figure>
    <img
      alt="Multiple hot air balloons in a clear sky, a crowd of spectators gather in the foreground."
      src="https://mdn.github.io/shared-assets/images/examples/balloons.jpg" />
    <figcaption>Balloons</figcaption>
  </figure>
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

img {
  max-width: 100%;
}
figure {
  margin: 0;
  break-inside: avoid;
}
figcaption {
  font-weight: bold;
  border-bottom: 2px solid #999999;
}
.container {
  column-width: 200px;
}
```

Breaks before and after boxes

The `break-before` and `break-after` properties are used to control breaks before and after elements. They take the following values when in a multicol context:

- auto
- avoid
- avoid-column
- column

In this next example, we are forcing a column break before an `h2` element.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon.
  </p>
  <h2>My heading</h2>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce.</p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 250px;
}

h2 {
  break-before: column;
}
```

Breaks between lines

The `orphans` and `widows` properties, part of the CSS fragmentation module, are also useful and worth mentioning. The `orphans` property controls the number of lines left on their own at the end of a fragment. The `widows` property controls the number left on their own at the start of a fragment.

The `orphans` and `widows` properties take an `<integer>` as a value, which represents the number of lines to keep together at the end and start of a fragment, respectively. Note that these properties only work inside a block container, such as a paragraph. If the block has fewer lines in it than the number that you specify as a value, all lines will be kept together.

In the example below, we are using the `orphans` property to control the number of lines left at the bottom of a column. You can change that value to see the effect on the breaking of the content.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce.</p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 250px;
  orphans: 3;
}
```

When things don't work as expected

If you have small amounts of content and are trying to control breaks on several elements, your content needs to break somewhere, so you may not always get the result you intended. To some extent, your use of fragmentation is always a suggestion to the browser, to control breaks in this way if it is possible. If the content doesn't break where you intended, the result may be untidy, but the content is still available for your users.

Handling overflow in multi-column layout

In this guide, we look at how to deal with overflow in a multi-column (*multicol*) layout, both inside the column boxes and in situations where there is more content than will fit into the container.

Overflow inside column boxes

An overflow situation happens when an item's size is larger than the column box. For example, the situation could happen when an image in a column is wider than the `column-width` value or the width of the column based on the number of columns declared with `column-count`.

In this situation, the content should visibly overflow into the next column, rather than be clipped by the column box.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <img
    alt="A close-up of two hot air balloons being inflated."
    src="https://mdn.github.io/shared-assets/images/examples/balloons3.jpg" />
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 250px;
}
```

This example renders like so:

There are two columns of text. In the left column, there is a photo that is wider than the column. The image expands into that second column, appearing behind the text of the right column. The flow of text in the right column isn't affected by the protruding photo, but the appearance is.

If you want an image to fit the column box, setting `max-width: 100%` will prevent the image from growing beyond its container, in this case, the column box.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <img
    alt="A close-up of two hot air balloons being inflated."
    src="https://mdn.github.io/shared-assets/images/examples/balloons3.jpg" />
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 250px;
}
img {
  max-width: 100%;
}
```

The updated rendering looks like this:

More columns than will fit

How overflowing columns are handled depends on whether the media context is fragmented, such as print, or is continuous, such as a web page.

- In fragmented media, after a fragment (for example, a page) is filled with columns, the columns will move to a new page and fill that up with columns.
- In continuous media, if a height is set on a multi-column container, columns overflow in the inline direction by default. On the web, this means that you will get a horizontal scrollbar. This behavior can be overridden by using `column-height` and `column-wrap` to enforce column wrapping.

This example shows the default continuous media overflow behavior. The multicol container has a set `height` and there is more text than space to create columns; therefore, we get columns created outside of the container.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
```

```
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 200px;
  height: 180px;
  border: 2px dashed;
}
```

This example renders like so:

Scroll horizontally to see the overflowing columns.

Using vertical media queries

One issue with multicol on the web is that if the columns are taller than the viewport, the reader will need to scroll the page up and down to read, which is not a good user experience. One way to avoid this is to only apply the column properties if you know there is enough vertical space.

In the example below, we used a `height` @media query to ensure there is enough vertical space before applying the column properties.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

@media (height >= 300px) {
  .container {
    column-width: 200px;
  }
}
```

This example renders like so:

Using column wrapping for multicol

The `column-height` and `column-wrap` properties can be used to set a fixed height for generated columns and force excess columns to overflow into additional rows in the block direction. In horizontal writing mode content, this results in vertically-scrolling rows of columns, rather than a horizontally scrolling single row. Let's look at an example.

The HTML contains basic text content, which we have hidden for brevity.

```
<p>
  This is a bunch of text split into three columns using the CSS
  <code>column-count</code> property with a value of <code>3</code>. It also
  includes a <code>column-height</code> value of <code>95vh</code>. The
  <code>column-wrap</code> value is set to its initial value, <code>auto</code>;
  when a <code>column-height</code> value is included,
  <code>column-wrap: auto</code> resolves to <code>wrap</code>, which allows the
  columns to wrap onto multiple rows. The text is equally distributed over the
  columns, and placed into multiple rows.
</p>

<p>
  The <code>column-height</code> and <code>column-wrap</code> properties were
  introduced in
  <a href="https://drafts.csswg.org/css-multicol-2/"
    >CSS Multi-column Layout Module Level 2</a
  >.
</p>
```

We give our content some styles. Most notably, we set the `<body>` element's `column-count` to `3`, and its `column-height` to `95vh` so that each row of columns fills up the viewport. We don't need to explicitly set `column-wrap` to `wrap`: when `column-height` is set to a `<length>` value, the initial value of `column-wrap` (`auto`) resolves to `wrap`, which is usually the behavior you'll want.

```
body {
  font-size: 1.3em;
  line-height: 1.5;
  column-count: 3;
  column-height: 95vh;
}
```

```
html {
  font-family: sans-serif;
}

body {
  width: 70%;
```

```
  margin: 0 auto;
}

p:first-of-type {
  margin-top: 0;
}
```

```
@supports not (column-height: 5em) {
  body::before {
    content: "Your browser does not support the 'column-height' property.";
    background-color: wheat;
    position: fixed;
    inset: 40% 0;
    height: fit-content;
    text-align: center;
    padding: 1rem 0;
  }
}
```

This example renders as follows:

Next steps

In the final guide in this series, we will see how fragmentation works with multicol layouts to give us control over how content breaks between columns.

Spanning and balancing columns

In this guide, we look at how to make elements span across columns inside the multi-column (*multicol*) container and how to control how the columns are filled.

Spanning the columns

To cause an item to span across columns, use the `column-span` property with a value of `all`. This will cause the element to become a *spanner*, spanning all the columns.

Any descendant element of the multicol container may be turned into a spanner, including both direct and indirect children. For example, a heading nested directly inside the container could become a spanner, as could a heading nested inside a `<section>` nested inside the multicol container.

In the example below, the `<h2>` element is set to `column-span: all` and spans all of the columns.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic. Gumbo beet greens
    corn soko endive gumbo gourd.
  </p>
  <h2>A heading</h2>
  <p>
    Parsley shallot courgette tatsoi pea sprouts fava bean collard greens
    dandelion okra wakame tomato. Dandelion cucumber earthnut pea peanut soko
    zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-width: 250px;
}

h2 {
  column-span: all;
  background-color: #4d4e53;
  color: white;
}
```

In this second example, the heading is inside an `<article>` element, yet still spans the content as expected.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic. Gumbo beet greens
    corn soko endive gumbo gourd.
  </p>
  <article>
    <h2>A heading</h2>
    <p>
      Parsley shallot courgette tatsoi pea sprouts fava bean collard greens
      dandelion okra wakame tomato. Dandelion cucumber earthnut pea peanut soko
      zucchini.
    </p>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </article>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
```

```
.container {
  column-width: 250px;
}

h2 {
  column-span: all;
  background-color: #4d4e53;
  color: white;
}
```

When a spanner is introduced, it breaks the flow of columns; columns restart after the spanner, effectively creating a new set of column boxes. The content does not jump over a spanning element.

Limitations of column-span

The `column-span` can have only two values. The initial value `none` means the item does not span and remains within a column. The value `all` means the item spans all of the columns. There are no values that enable partial spanning, such as having an item span two out of three columns.

Things to watch out for

If the spanning element is inside another element with margins, padding, and a border or background color, the box may appear above the spanner with the rest of the content being displayed below. For this reason, care should be taken when setting an element to span all the columns, ensuring this scenario is accounted for.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic. Gumbo beet greens
    corn soko endive gumbo gourd.
  </p>
  <article>
    <h2>A heading</h2>
    <p>
      Parsley shallot courgette tatsoi pea sprouts fava bean collard greens
      dandelion okra wakame tomato. Dandelion cucumber earthnut pea peanut soko
      zucchini.
    </p>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </article>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

article {
  border: 1px solid red;
  padding: 10px;
}

.container {
  column-width: 250px;
}
h2 {
  background-color: #4d4e53;
  color: white;
  column-span: all;
}
```

Additionally, if a spanning element appears later in the content, it can cause unexpected or unwanted behavior when there is not enough content to create columns after the spanner. Use spanning carefully and test at various breakpoints to make sure you get the effect you intended.

Filling and balancing columns

A balanced set of columns is where all columns have approximately the same amount of content. Filling and balancing are relevant when the amount of content does not match the amount of space provided, such as when a `height` is declared on the container.

The initial value for `column-fill` is `balance`. The value of `balance` means all columns are as balanced as possible. In fragmented contexts, such as paged media, only the last fragment is balanced. This means that on the last page, the final set of column boxes is balanced.

The other balancing value, `balance-all`, balances all columns in fragmented contexts.

The columns in this example contain an image and some text, which are balanced. The image, which cannot break, is in the first column. The other columns are balanced, filling with equal amounts of text.

```
<div class="container">
  <img
    alt="Multiple hot air balloons in a clear sky, a crowd of spectators gather in the foreground."
    src="https://mdn.github.io/shared-assets/images/examples/balloons.jpg" />
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
h2 {
  background-color: #4d4e53;
  color: white;
}
img {
  max-width: 100%;
```

```
}
.container {
  column-width: 200px;
  column-fill: balance;
  height: 250px;
}
```

The `auto` value for `column-fill` fills a column sequentially, filling the first column in the inline-start direction, before placing content in subsequent columns, rather than balancing and filling all the columns equally. In this example, we changed `column-fill` to `auto`. The columns are filled to the height of the container, leaving empty columns at the end.

```
<div class="container">
  <img
    alt="Multiple hot air balloons in a clear sky, a crowd of spectators gather in the foreground."
    src="https://mdn.github.io/shared-assets/images/examples/balloons.jpg" />
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
h2 {
  background-color: #4d4e53;
  color: white;
}
img {
  max-width: 100%;
}

.container {
  column-width: 150px;
  column-fill: auto;
  height: 250px;
}
```

Next steps

In the next guide, you will learn how multicol handles overflow within columns and when there are more columns than can fit in the container.

Styling columns

As column boxes created inside multi-column (*multicol*) containers are anonymous boxes, styling individual columns is not possible, but we can style the gaps between the columns and the container in general. This guide explains how to change the gap and style rules between columns.

Column gaps

The gap between columns is controlled using the `column-gap` or `gap` property. The `column-gap` property is defined in the multi-column layout module. The `gap` property is defined in the box alignment module. This is a unified property to define gaps between boxes in all layouts that support gaps, including CSS grid layout and CSS flexible box layout.

The initial value of `column-gap` is `1em`, which prevents columns from running into each other. In other layout methods, `column-gap` is supported as a synonym for `gap`, but with an initial value of `0`. The keyword value `normal` sets `column-gap` to the initial value.

You can change the gap by using any `<length>` value. In the example below, the `column-gap` is set to `40px`.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-count: 3;
  column-gap: 40px;
}
```

The allowed value for `column-gap` is a `<length-percentage>`, meaning percentages are allowed. Percentage values for `column-gap` are calculated as a percentage of the width of the multicol container.

Column rules

The specification defines `column-rule-width`, `column-rule-style` and `column-rule-color`, providing a shorthand `column-rule`. These properties work in exactly the same way as the `border` properties: any `<line-style>` can be used as a `column-rule-style`, just as for valid `border-style`.

These properties are applied to the element, which is the multicol container, and therefore, all columns will have the same rule. Rules are only drawn between columns and not on the outer edges. Rules are also only drawn between columns that have content.

In this next example, a 5px-dotted rule with a color of `rebeccapurple` has been created using the longhand values.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-count: 3;
  column-rule-width: 5px;
  column-rule-style: dotted;
  column-rule-color: rebeccapurple;
}
```

Note that the rule itself does not take up any space: a wide rule will not push the columns apart to make space for the rule. Instead, the rule overlays the gap.

The example below uses a very wide rule of `40px` and a `10px` gap. The rule displays under the column content. To make space on both sides of the rule, the gap would need to be increased to be larger than `40px`.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.container {
  column-count: 3;
  column-gap: 10px;
  column-rule: 40px solid rebeccapurple;
}
```

Next steps

This article details all the current ways in which column boxes can be styled. In the next guide, we will take a look at making elements inside a container span across all columns.

Using multi-column layouts

The properties defined in the **CSS multi-column layout module** extend the *block layout mode*, enabling the definition of multiple columns of text. People have trouble reading text if lines are too long. If it takes too long for the eyes to move from the end of one line to the beginning of the next, readers can lose track of which line they were on. To provide for a better user experience when reading text making use of a large screen, you should limit the width of text by using columns of text placed side by side, just as newspapers do.

Using columns

Column count and width

Two CSS properties control whether and how many columns will appear: `column-count` and `column-width`.

The `column-count` property sets the number of columns to a particular number. E.g.,

Example 1

HTML

```
<div id="col">
  <p>
    Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua.
  </p>
  <p>
    Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut
    aliquip ex ea commodo consequat.
  </p>
  <p>
    Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore
    eu fugiat nulla pariatur.
  </p>
  <p>
    Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia
    deserunt mollit anim id est laborum.
```

```
  </p>
</div>
```

CSS

```
#col {
  column-count: 2;
}
```

Result

The content will be displayed in two columns:

The `column-width` property sets the minimum desired column width. If `column-count` is not also set, then the browser will automatically make as many columns as fit in the available width.

Example 2

HTML

```
<div id="wid">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
  proident, sunt in culpa qui officia deserunt mollit anim id est laborum
</div>
```

CSS

```
#wid {
  column-width: 100px;
}
```

Result

In a multi-column block, content automatically flows from one column into the next as needed. All HTML, CSS, and DOM functionality is supported within columns, as are editing and printing.

The columns shorthand

You can use either `column-count` or `column-width`. Because values for these properties do not overlap, it is often convenient to use the shorthand `columns`.

Example 3

In this example, the CSS declaration `column-width: 12em` is replaced by `columns: 12em`.

HTML

```
<div id="col_short">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
  proident, sunt in culpa qui officia deserunt mollit anim id est laborum
</div>
```

CSS

```
#col_short {
  columns: 12em;
}
```

Example 4

In this example, the CSS declaration `column-count: 4` is replaced by `columns: 4`.

HTML

```
<div id="columns_4">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
  proident, sunt in culpa qui officia deserunt mollit anim id est laborum
</div>
```

CSS

```
#columns_4 {
  columns: 4;
}
```

Result

Example 5

The two CSS declarations `column-width: 8em` and `column-count: 12` can be replaced by `columns: 12 8em`. The `column-count` portion of the shorthand is the maximum number of columns that will be present. The `column-width` is the minimum width each column should be.

HTML

```
<div id="columns_12">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
```

```
  proident, sunt in culpa qui officia deserunt mollit anim id est laborum
</div>
```

CSS

```
#columns_12 {
  columns: 12 8em;
}
```

Result

Assuming a default `1em` gap between columns, if the container is wider than `103ems` (12 columns * `8em` width each + 7 `1em` gaps), there will be 12 columns, each with a width of `8ems` or more. If the container is less than `103ems` wide, there will be fewer than 12 columns. If the container is less than `17ems` wide (`8em` column + `8em` column + `1em` gap), the content will be displayed as a single column with no column gap.

Height balancing

CSS columns require that the column heights must be balanced: that is, the browser automatically sets the maximum column height so that the heights of the content in each column are approximately equal. Firefox does this.

However, in some situations it is also useful to set the maximum height of the columns explicitly, and then lay out content starting at the first column and creating as many columns as necessary, possibly overflowing to the right. Therefore, if the height is constrained, by setting the CSS `height` or `max-height` properties on a multi-column block, each column is allowed to grow to that height and no further before adding new column. This mode is also much more efficient for layout.

Column Gaps

There is a gap between columns. The recommended default is `1em`. This size can be changed by applying the `column-gap` property to the multi-column block:

Example 6

HTML

```
<div id="column_gap">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
  proident, sunt in culpa qui officia deserunt mollit anim id est laborum
</div>
```

CSS

```
#column_gap {
  column-count: 5;
  column-gap: 2em;
}
```

Result

Conclusion

CSS columns are a layout primitive that can help make large blocks of text easier to read when responsive content is viewed on wide viewports. Imaginative developers may find many uses for them, especially in conjunction with container queries and with the automatic height balancing feature.

CSS namespaces

The **CSS namespaces** module defines the syntax for using namespaces in CSS.

CSS isn't just for styling HTML. A stylesheet may be used to style SVG, MathML, XML, or HTML, each of which has a different namespace or a document containing multiple namespaces.

The `@namespace` at-rule defined in this module enables distinguishing between same-named elements in different namespaces. Element tag names are not unique to a single language. For example, the `<a>` element isn't limited to HTML. You may want to style the `<a>`s within your SVGs differently from the links in your HTML. You also will likely want to ensure that `querySelectorAll("a")` selects the right kind of element. Namespacing can help.

The `@namespace` rule is used for declaring a default namespace and for binding namespaces to namespace prefixes. The namespaces module also defines the syntax for using these prefixes to represent namespace-qualified names. That's it. What a name means or if the name is even valid depends on the context and host language.

Reference

At-rules and descriptors

- `@namespace`

Guides

Namespaces crash course

Deep dive into what a namespace is and how they are used in XML and XML-based markup languages.

Related concepts

- CSS namespace separator (|) combinator
- CSS type selectors
- CSS universal selector
- `CSSNamespaceRule` interface
  - `CSSNamespaceRule.namespaceURI` property
  - `CSSNamespaceRule.prefix` property
- `Document.createAttributeNS()` method
- `Document.createElementNS()` method
- `Document.getElementsByTagNameNS()` method
- `Element.getAttributeNodeNS()` method
- `Element.getAttributeNS()` method
- `Element.getElementsByTagNameNS()` method
- `Element.hasAttributeNS()` method
- `Element.namespaceURI` property
- `Element.removeAttributeNS()` method
- `Element.setAttributeNS()` method

- `Element.setAttributeNodeNS()` method
- `NamedNodeMap.getNamedItemNS()` method
- `NamedNodeMap.removeNamedItemNS()` method
- `NamedNodeMap.setNamedItemNS()` method
- Namespace glossary term

See also

- `<a>` SVG element
- `<url>` data type
- CSS at-rules
- CSS at-rule functions
- CSS selectors

CSS nesting

The **CSS nesting** module defines a syntax for nesting selectors, providing the ability to nest one style rule inside another, with the selector of the child rule relative to the selector of the parent rule.

CSS nesting is different from CSS preprocessors such as Sass in that it is parsed by the browser rather than being pre-compiled by a CSS preprocessor.

CSS nesting helps with the readability, modularity, and maintainability of CSS stylesheets. It also potentially helps reduce the size of CSS files, thereby decreasing the amount of data downloaded by users.

Reference

Selectors

- `&` nesting selector

Guides

Using CSS nesting

> Explains how to use CSS nesting.

CSS nesting at-rules

> Explains how to nest at-rules.

CSS nesting and specificity

> Explains the differences in specificity when nesting CSS.

Related concepts

- Selectors and combinators
- Pseudo-classes
- CSS preprocessor

See also

- Specificity
- CSS cascading and inheritance module
- CSS selectors module

CSS nesting at-rules

Any at-rule whose body contains style rules can be nested inside another style rule using CSS nesting. Style rules nested inside at-rules take their nesting selector definition from the nearest ancestor style rule. Properties can be directly included inside a nested at-rule, acting as if they were nested in a `& {...}` block.

at-rules that can be nested

- `@media`
- `@supports`
- `@layer`
- `@scope`
- `@container`
- `@starting-style`

Examples

Nesting <code>@media</code> at-rule

In this example we see three blocks of CSS. The first one shows how to write typical at-rule nesting, the second is an expanded way of writing the nesting as the browser parses it, and the third one shows the non-nested equivalent.

Nested CSS

```
.foo {
  display: grid;
  @media (orientation: landscape) {
    grid-auto-flow: column;
  }
}
```

Expanded nested CSS

```
.foo {
  display: grid;
  @media (orientation: landscape) {
    & {
      grid-auto-flow: column;
    }
  }
}
```

Non-nested equivalent

```
.foo {
  display: grid;
}

@media (orientation: landscape) {
  .foo {
    grid-auto-flow: column;
  }
}
```

Multiple nested <code>@media</code> at-rules

At-rules can be nested within other at-rules. Below you can see an example of this, and how it would be written without nesting.

Nested at-rules

```
.foo {
  display: grid;
  @media (orientation: landscape) {
    grid-auto-flow: column;
    @media (width >= 1024px) {
      max-inline-size: 1024px;
    }
  }
}
```

Non-nested equivalent

```
.foo {
  display: grid;
}
@media (orientation: landscape) {
  .foo {
    grid-auto-flow: column;
  }
}
@media (orientation: landscape) and (width >= 1024px) {
  .foo {
    max-inline-size: 1024px;
  }
}
```

Nesting Cascade Layers (<code>@layer</code>)

Cascade Layers can be nested to create child-layers. These are joined with a `.` (dot).

Defining the parent & child layers

We start by defining the named cascade layers, prior to using them, without any style assignments.

```
@layer base {
  @layer support;
}
```

Assigning rules to layers with nesting

Here the `.foo` selector assigns its rules to the **base** `@layer`. The nested **support** `@layer` creates the `base.support` sub-layer, and the `&` nesting selector is used to create the rules for the `.foo .bar` selector.

```
.foo {
  @layer base {
    block-size: 100%;
    @layer support {
      & .bar {
        min-block-size: 100%;
      }
    }
  }
}
```

Equivalent without nesting

```
@layer base {
  .foo {
    block-size: 100%;
  }
}
@layer base.support {
  .foo .bar {
    min-block-size: 100%;
  }
}
```

See also

- CSS Nesting module
- `&` nesting selector
- Using CSS nesting
- Nesting and specificity
- Nesting container queries

CSS nesting and specificity

The specificity of the `&` nesting selector is calculated using the largest specificity in the associated selector list. This is identical to how specificity is calculated when using the `:is()` function.

```
<b class="foo">
  <i>Blue text</i>
```

```
</b>
```

<code>&</code> nesting syntax

```
#a, b {
  & i {
    color: blue;
  }
}

.foo i {
  color: red;
}
```

<code>:is()</code> syntax

```
:is(#a, b) {
  & i {
    color: blue;
  }
}

.foo i {
  color: red;
}
```

In this example, the id selector (`#a`) has a specificity of `1-0-0`, while the type selector (`b`) has a specificity of `0-0-1`. The `&` nesting selector and `:is()` pseudo-class both take a specificity of `1-0-0`, even though the `#a` id selector is never used.

The `.foo` class selector has a specificity of `0-1-0`. This makes the total specificity `1-0-1` for `& i` and `0-1-1` for `.foo i`, meaning that `color: blue;` wins out.

See also

- CSS nesting module
- `&` nesting selector
- Using CSS nesting
- Nesting at-rules

Using CSS nesting

The CSS nesting module allows you to write your stylesheets so that they are easier to read, more modular, and more maintainable. As you are not constantly repeating selectors, the file size can also be reduced.

CSS nesting is different from CSS preprocessors such as Sass in that it is parsed by the browser rather than being pre-compiled by a CSS preprocessor. Also, in CSS nesting, the specificity of the `&` nesting selector is similar to the `:is()` function; it is calculated using the highest specificity in the associated selector list.

This guide shows different ways to arrange nesting in CSS.

Child selectors

You can use CSS nesting to create child selectors of a parent, which in turn can be used to target child elements of specific parents. This can be done with or without the `&` nesting selector.

There are certain instances where using the `&` nesting selector can be necessary or helpful:

- When joining selectors together, such as using compound selectors or pseudo-classes.
- For backwards compatibility.
- As a visual indicator to aid with readability, when seeing the `&` nesting selector you know that CSS nesting is being used.

```
/* Without nesting selector */
.parent {
  /* parent styles */
  .child {
    /* child of parent styles */
  }
}

/* With nesting selector */
.parent {
  /* parent styles */
  & .child {
    /* child of parent styles */
  }
}

/* the browser will parse both of these as */
.parent {
  /* parent styles */
}
.parent .child {
  /* child of parent styles */
}
```

Examples

In these examples, one without and one with the `&` nesting selector, the `<input>` inside the `<label>` is being styled differently to the `<input>` that is a sibling of a `<label>`.

Without nesting selector

HTML

```
<form>
  <label for="name">Name:
    <input type="text" id="name" />
  </label>
  <label for="email">email:</label>
  <input type="text" id="email" />
</form>
```

CSS

```
form,
label {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}
```

```
input {
  /* styles for input not in a label  */
  border: tomato 2px solid;
}
label {
  /* styles for label */
  font-family: system-ui;
  font-size: 1.25rem;

  input {
    /* styles for input in a label  */
    border: blue 2px dashed;
  }
}
```

Result

With nesting selector

```
<form>
  <label for="name">Name:
    <input type="text" id="name" />
  </label>
  <label for="email">email:</label>
  <input type="text" id="email" />
</form>
```

CSS

```
form,
label {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}
```

```
input {
  /* styles for input not in a label  */
  border: tomato 2px solid;
}
label {
  /* styles for label */
  font-family: system-ui;
  font-size: 1.25rem;

  & input {
    /* styles for input in a label  */
    border: blue 2px dashed;
  }
}
```

Result

Combinators

CSS Combinators can also be used with or without the `&` nesting selector.

Example

Nesting the sibling combinator

In this example, the first paragraph after each `<h2>` is targeted with the next-sibling combinator (+) using CSS nesting.

HTML

```
<h2>Heading</h2>
<p>This is the first paragraph.</p>
<p>This is the second paragraph.</p>
```

CSS

```
h2 {
  color: tomato;
  + p {
    color: white;
    background-color: black;
  }
}
/* this code can also be written with the & nesting selector */
/*
h2 {
  color: tomato;
  & + p {
    color: white;
    background-color: black;
  }
}
*/
```

Result

Compound selectors

When using compound selectors in nested CSS you **have** to use the `&` nesting selector. This is because the browser will automatically add whitespace between selectors that do not use the `&` nesting selector.

In order to target an element with `class="a b"` the `&` nesting selector is needed otherwise the whitespace will break the compound selector.

```
.a {
  /* styles for element with class="a" */
  .b {
    /* styles for element with class="b" which is a descendant of class="a" */
  }
  &.b {
    /* styles for element with class="a b" */
  }
}

/* the browser parses this as */
.a {
  /* styles for element with class="a" */
}
.a .b {
  /* styles for element with class="b" which is a descendant of class="a" */
}
.a.b {
  /* styles for element with class="a b" */
}
```

Example

Nesting and compound selectors

In this example the `&` nesting selector is used to create compound selectors to style elements with multiple classes.

HTML

```
<div class="notices">
  <div class="notice">
    <h2 class="notice-heading">Notice</h2>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
  </div>
  <div class="notice warning">
    <h2 class="warning-heading">Warning</h2>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
  </div>
  <div class="notice success">
    <h2 class="success-heading">Success</h2>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
  </div>
</div>
```

CSS

Styles for the `.notices` to create a column using flexbox layout.

```
.notices {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  width: 90%;
  margin: auto;
}
```

In the CSS code below, nesting is used to create compound selectors both with and without `&`. The top-level selector defines the basic styles for elements with `class="notice"`. The `&` nesting selector is then used to create compound selectors for elements with either `class="notice warning"` or `class="notice success"`. Additionally, the use of nesting to create compound selectors without explicitly using `&` can be seen in the selector `.notice .notice-heading::before`.

```
.notice {
  width: 90%;
  justify-content: center;
  border-radius: 1rem;
  border: black solid 2px;
  background-color: #ffc107;
  color: black;
  padding: 1rem;
  .notice-heading::before {
    /* equivalent to `.notice .notice-heading::before` */
    content: "i ";
  }
  &.warning {
    /* equivalent to `.notice.warning` */
    background-color: #d81b60;
    border-color: #d81b60;
    color: white;
    .warning-heading::before {
      /* equivalent to `.notice.warning .warning-heading::before` */
      content: "! ";
    }
  }
  &.success {
    /* equivalent to `.notice.success` */
    background-color: #004d40;
    border-color: #004d40;
    color: white;
    .success-heading::before {
      /* equivalent to `.notice.success .success-heading::before` */
      content: "✓ ";
    }
  }
}
```

Result

Appended nesting selector

The `&` nesting selector can also be appended to a nested selector which has the effect of reversing the context.

This can be useful when we have styles for a child element that change when a parent element is given a different class:

```
<div>
  <span class="foo">text</span>
</div>
```

As opposed to:

```
<div class="bar">
  <span class="foo">text</span>
</div>
```

```
.foo {
  /* .foo styles */
  .bar & {
    /* .bar .foo styles */
  }
}
```

Example

Appending nesting selector

In this example there are 3 cards, one of which is featured. The cards are all exactly the same except the featured card will have an alternative color for the heading. By appending the `&` nesting selector the style for the `.featured h2` can be nested in the style for the `h2`.

HTML

```
<div class="wrapper">
  <article class="card">
    <h2>Card 1</h2>
    <p>Lorem ipsum dolor, sit amet consectetur adipisicing elit.</p>
  </article>
  <article class="card featured">
    <h2>Card 2</h2>
    <p>Lorem ipsum dolor, sit amet consectetur adipisicing elit.</p>
  </article>
  <article class="card">
    <h2>Card 3</h2>
    <p>Lorem ipsum dolor, sit amet consectetur adipisicing elit.</p>
  </article>
</div>
```

CSS

```
.wrapper {
  display: flex;
  flex-direction: row;
  gap: 0.25rem;
  font-family: system-ui;
}
```

In the following CSS, we are creating the styles for `.card` and `.card h2`. Then, in the `h2` style block, we nest the `.featured` class with the `&` nesting selector appended which creates a style for `.card :is(.featured h2)`, which is equivalent to `:is(.card h2):is(.featured h2)`.

```
.card {
  padding: 0.5rem;
  border: 1px solid black;
  border-radius: 0.5rem;
  & h2 {
    /* equivalent to `.card h2` */
    color: slateblue;
    .featured & {
      /* equivalent to `:is(.card h2):is(.featured h2)` */
      color: tomato;
    }
  }
}
```

Result

Nested declarations rule

The nested declaration rule is that CSS rules are parsed in the order that they are written in the CSS document.

With the following CSS:

```
.foo {
  background-color: silver;
  @media screen {
    color: tomato;
  }
  color: black;
}
```

The `background-color` is parsed first and set to silver, then the `@media` rule is evaluated, and finally the `color`.

The CSSOM parses the CSS in the following way:

```
↳ CSSStyleRule
  .style
    - background-color: silver
  ↳ CSSMediaRule
```

```
    ↳ CSSNestedDeclarations
      .style (CSSStyleDeclaration, 1) =
      - color: tomato
  ↳ CSSNestedDeclarations
    .style (CSSStyleDeclaration, 1) =
      - color: black
```

Note that in order to preserve the parsing order, all the rules before nesting are handled as top-level `CSSRules`, while any top level rules after nesting are represented as `CSSNestedDeclarations`. That's why the `color-black` is inside a nested declaration even though it is a top level declaration in the original document.

> **Note:** Support for the rule was added with `CSSNestedDeclarations`. Browsers that do not support this interface may parse nested rules in the wrong order.

Concatenation (is not possible)

In CSS preprocessors such as Sass, it is possible to use nesting to join strings to create new classes. This is common in CSS methodologies such as BEM.

```
/* the following Sass syntax... */
.component {
  &__child-element {
  }
}
/* ... would become the following CSS */
.component__child-element {
}
```

> **Warning:** This is not possible in CSS nesting: when a combinator is not used, the nested selector is treated as a type selector. Allowing concatenation would break this.

In compound selectors, the type selector must come first. Writing `&Element` (a type selector) makes the CSS selector, and the entire selector block, invalid. As the type selector must come first, the compound selector must be written as `Element&`.

```
.my-class {
  element& {
  }
}

/* the browser parses this to become a compound selector */
.my-class {
}
element.my-class {
}
```

Invalid nested style rules

If a nested CSS rule is invalid then all of the enclosed styles will be ignored. This does not affect the parent or preceding rules.

In the following example, there is an invalid selector (`%` is not a valid character for selectors). The rule that includes this selector is ignored, but subsequent valid rules are not.

```
.parent {
  /* .parent styles these work fine */
  & %invalid {
    /* %invalid styles all of which are ignored */
  }
  & .valid {
    /* .parent .valid styles these work fine */
  }
}
```

See also

- CSS nesting module
- `&` nesting selector
- Nesting `@` at-rules
- Nesting and specificity
- `CSSNestedDeclarations`
- The Nested Declarations Rule

CSS overflow

The **CSS overflow** module properties enable you to handle scrollable overflow in visual media.

Overflow happens when the content in an element box extends past one or more of the box's edges. **Scrollable overflow** is the content that appears outside the element box for which you might want to add a scrolling mechanism. CSS overflow properties let you control what happens when content overflows an element box, including creating carousels without JavaScript.

Painting effects that overflow the content but do not participate in the CSS box model do not affect layout. This type of overflow is also known as ink overflow. Examples of ink overflows include box shadows, border images, text decoration, overhanging glyphs, and outlines. Ink overflows do not extend the scrollable overflow region.

Overflow in action

Try the following example to see the effects of various `overflow` property values on the content overflow and scrollbars in the adjacent fixed-size box.

The example includes options to change the values for the `overflow-clip-margin` and `width` properties, as well as to programmatically scroll the content if the overflow property creates a scroll container. Select `overflow: clip` and see the effect of different `overflow-clip-margin` values. Select `overflow: hidden` or `overflow: scroll` to check out the various `ScrollLeft` and `ScrollTop` slider settings.

```
<article>
  <fieldset>
    <legend>Select options:</legend>
    <label
      ><code>overflow</code>:
      <select id="overflowValue">
        <option>hidden</option>
        <option>clip</option>
        <option>scroll</option>
        <option>auto</option>
        <option selected>visible</option>
```

```
        <option>overlay</option>
      </select>
    </label>
    <label>
      <code>overflow-clip-margin</code>:
      <input type="number" id="ocm" value="1" min="0" max="10" />
      <code>em</code>
    </label>
    <label
      ><input type="checkbox" id="wide" /> <code>width</code>:
      <code>20em</code> or <code>40em</code></label
    >
    <fieldset>
      <legend>Scroll programmatically:</legend>
      <label
        >ScrollLeft:
        <input type="range" min="0" max="100" value="0" id="scrollL"
      /></label>
      <label
        >ScrollTop:
        <input type="range" min="0" max="100" value="0" id="scrollT"
      /></label>
    </fieldset>
  </fieldset>
  <pre class="visible"> 
    Oh, Rubber Duckie, you're the one
    You make bath time lots of fun
    Rubber Duckie, I'm awfully fond of you

    Rubber Duckie, joy of joys
    When I squeeze you, you make noise
    Rubber Duckie, you're my very best friend, it's true

    Oh, every day when I make my way to the tubby
    I find a little fella who's cute and yellow and chubby
    Rub-a-dub-dubby

    <a href="#">Rubber Duckie</a>, you're so fine
    And I'm lucky that you're mine
    Rubber Duckie, I'm awfully fond of you
      </pre>
</article>
```

```
article {
  display: flex;
  gap: 1em;
}

label {
  display: block;
  white-space: nowrap;
}

pre {
  border: 2px dashed crimson;
  height: 150px;
  width: 20em;
  margin-bottom: 3em;
  overflow-clip-margin: 1em;
  text-align: center;
}

.wide {
  width: 40em;
}

::before {
  font-weight: bold;
  color: white;
  background: crimson;
  display: inline-block;
  min-width: 50%;
  padding: 3px 5px;
  box-sizing: border-box;
}

.hidden {
  overflow: hidden;
}
.hidden::before {
  content: "hidden: ";
}

.clip {
  overflow: clip;
}
.clip::before {
  content: "clip: ";
}

.scroll {
  overflow: scroll;
}
.scroll::before {
  content: "scroll: ";
}

.auto {
  overflow: auto;
}
.auto::before {
  content: "auto: ";
}
```

```
.overlay {
  overflow: clip;
  overflow: overlay;
}
.overlay::before {
  content: "overlay (or clip if not supported): ";
}

.visible {
  overflow: visible;
}
.visible::before {
  content: "visible: ";
}

article:not(:has(pre.clip)) > fieldset > label:nth-of-type(2),
article:not(:has(pre.hidden, pre.scroll, pre.auto, pre.overlay))
  fieldset
  fieldset {
  opacity: 20%;
  pointer-events: none;
}
```

```
const pre = document.querySelector("pre");
const val = document.getElementById("overflowValue");
const check = document.getElementById("wide");
const ocm = document.getElementById("ocm");
const scrollL = document.getElementById("scrollL");
const scrollT = document.getElementById("scrollT");

val.addEventListener("change", () => {
  if (pre.classList.contains("wide")) {
    pre.className = `wide ${val.value}`;
  } else {
    pre.className = `${val.value}`;
  }
  scrollExample();
  clipMargin();
});

check.addEventListener("change", () => {
  pre.classList.toggle("wide");
  scrollExample();
});

ocm.addEventListener("change", () => {
  clipMargin();
});

scrollL.addEventListener("change", () => {
  scrollExample();
});
scrollT.addEventListener("change", () => {
  scrollExample();
});

function scrollExample() {
  pre.scrollTo({
    top: scrollT.value,
    left: scrollL.value * 2,
    behavior: "smooth",
  });
}

function clipMargin() {
  pre.style.overflowClipMargin = `${ocm.value}em`;
}
```

A link is included in the content box above to demonstrate the effects of keyboard focus on overflow and scroll behaviors. Try tabbing to the link or programmatically scrolling the content: the content will scroll only if the enumerated `<overflow>` value creates a scroll container.

Reference

Properties

- `line-clamp`
- `overflow` shorthand
- `overflow-block`
- `overflow-clip-margin`
- `overflow-inline`
- `overflow-x`
- `overflow-y`
- `scroll-behavior`
- `scroll-marker-group`
- `scroll-target-group`
- `scrollbar-gutter`
- `text-overflow`

The CSS overflow level 4 module also introduces the `block-ellipsis`, `continue`, `max-lines`, `overflow-clip-margin-block`, `overflow-clip-margin-block-end`, `overflow-clip-margin-block-start`, `overflow-clip-margin-bottom`, `overflow-clip-margin-inline`, `overflow-clip-margin-inline-end`, `overflow-clip-margin-inline-start`, `overflow-clip-margin-left`, `overflow-clip-margin-right`, and `overflow-clip-margin-top` properties. Currently, no browsers support these features.

Selectors

- `::scroll-button()`
- `::scroll-marker`
- `::scroll-marker-group`
- `:target-after`

- `:target-before`
- `:target-current`

Data types

- `<overflow>` enumerated values

Glossary terms and definitions

- Scroll container
- Scrollport

Guides

Learn: Overflowing content

> Learn what overflow is and how to manage it.

Creating CSS carousels

> Create pure-CSS carousel UI features using scroll buttons, scroll markers, and generated columns.

Creating a named scroll progress timeline animation

> The CSS scroll timeline `scroll-timeline-name` and `scroll-timeline-axis` properties, along with the `scroll-timeline` shorthand, create animations tied to the scroll offset of a scroll container.

Related concepts

- `::column`
- `scrollbar-width` CSS property
- `scrollbar-color` CSS property
- `scrollbar-gutter` CSS property
- `scroll-behavior` CSS property
- `scroll-margin` CSS shorthand property
- `scroll-padding` CSS shorthand property
- `scroll-snap-align` CSS property
- `scroll-snap-stop` CSS property
- `scroll-snap-type` CSS property
- `text-overflow` CSS property
- `::-webkit-scrollbar` pseudo-element
- `scrollbar` ARIA role
- Element `scroll()` method
- Element `scrollBy()` method
- Element `scrollIntoView()` method
- Element `scrollTo()` method
- Element `scrollTop` property
- Element `scrollLeft` property
- Element `scrollWidth` property
- Element `scrollHeight` property
- Document `scroll` event
- Scroll container glossary term
- Ink overflow glossary term

See also

- CSS scrollbars styling module
- CSS scroll snap module
- CSSOM view module
- How to debug scrollable overflow

Creating CSS carousels

The CSS overflow module defines features enabling the creation of flexible pure-CSS carousels with browser-generated and developer-styled scroll buttons and scroll markers. This guide explains how to create a carousel using these features.

Carousel concepts

Carousels are a common feature on the web. They typically take the form of a scrolling content area containing several items, such as presentation slides, ads, headline news stories, or key product features.

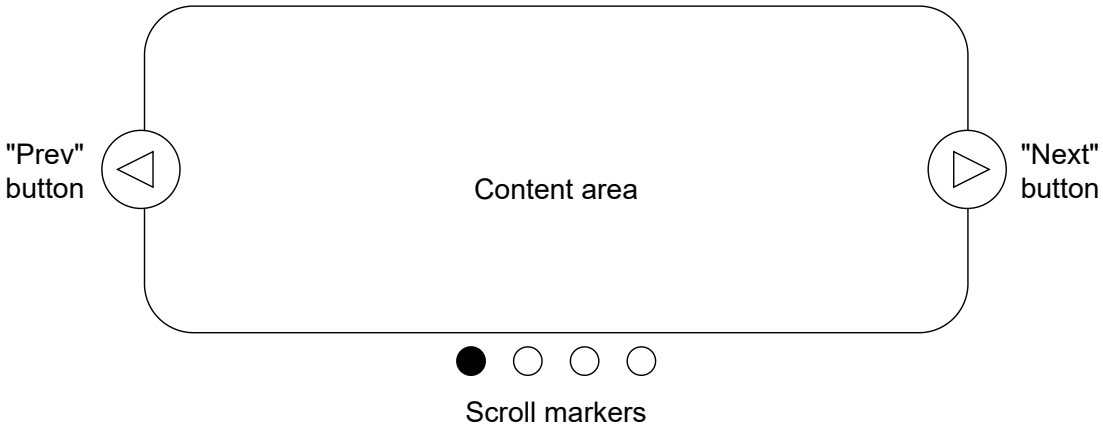
Users can move through the items by clicking or activating navigational buttons or by swiping. The navigation usually includes:

scroll buttons

> Generally "previous" and "next" buttons or links.

scroll markers

> A series of button or link icons, each representing one or more items depending on how many items are shown at each scroll position within the carousel.



A key feature of carousels is **pagination** — the items feel like separate pieces of content that are moved between rather than forming one continuous section of content. You might show one item at a time or several items on each carousel "page". When several items are visible, you might show an entirely new group of items each time the "next" or "previous" button is pressed. Alternatively, you could add a single new item to one end of the list while moving the item at the other end out of view.

Carousels can be quite brittle and challenging to implement with JavaScript. They require scripts to associate scroll markers with the items they represent while continuously updating the scroll buttons to keep them operating correctly.

Fortunately, we can create carousels with associated controls without the use of JavaScript, using CSS carousel features.

CSS carousel features

The CSS carousel features provide pseudo-elements and pseudo-classes that enable the creation of carousels using only CSS and HTML, with the browser handling most of the scrolling and link references in a flexible and consistent manner. These features are as follows:

`::scroll-button()`

Generated inside a scroll container, these pseudo-elements represent scroll buttons, which scroll the container in a specified direction.

`::scroll-marker-group`

Generated inside a scroll container; used to collect together and lay out scroll markers.

`::scroll-marker`

Generated inside the children of a scroll container ancestor or within a scroll container's columns, to represent their scroll markers. These can be selected to scroll the container to their associated child elements or column, and are collected inside the scroll container's `::scroll-marker-group` for layout purposes.

`:target-current`

Used to select the currently active scroll marker and style it. The ability to style the active scroll marker is important for both usability and accessibility.

`:target-before` and `:target-after`

Used to select and style scroll markers before and after the currently active scroll marker, respectively. They are useful for styling navigation items that come before and after the active navigation position, indicating items the user has already viewed and those that are still to come.

`::column`

Represents the individual columns generated when a container is set to display its content in multiple columns via CSS multi-column layout. The `::column` pseudo-element can be used in conjunction with `::scroll-marker` to generate a scroll marker for each column.

Carousel with single pages

Our first demo is a carousel of single pages, with each item taking up the full page. We have scroll markers as bottom navigation and scroll buttons on the sides of the page, enabling the user to move to the next and previous pages.

We'll use flexbox to lay out the carousel, scroll snapping to enforce clear pagination, and anchor positioning to position the scroll buttons and scroll markers relative to the carousel.

The HTML consists of a heading element and an unordered list, with each list item containing some sample content, along with a custom `data-` attribute (which we explain with the styling):

```
<h1>CSS carousel single item per page</h1>
<ul>
  <li data-accName="Item 1">
    <h2>Page 1</h2>
  </li>
  <li data-accName="Item 2">
    <h2>Page 2</h2>
  </li>
  <li data-accName="Item 3">
    <h2>Page 3</h2>
  </li>
  <li data-accName="Item 4">
    <h2>Page 4</h2>
  </li>
</ul>
```

Carousel layout with flexbox

We use flexbox to create a single row of items; the `<ul>` is the flex container, and the `<li>` child list items are displayed horizontally with each item taking up the full width of the carousel.

The unordered list is made to fill the full width of the viewport with a width `width` of `100vw`; it is also given a `height` of `300px`, and some `padding`. We then use flexbox to lay out the list — setting a `display` value of `flex` to cause the child list items to display in a row (due to the default `flex-direction` value of `row`), with a `gap` of `4vw` between each one.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  text-align: center;
  font-size: 1.7rem;
}
```

```
ul {
  width: 100vw;
  height: 300px;
  padding: 20px;
  display: flex;
  gap: 4vw;
}
```

Now it's time to style the list items. The first declarations provide rudimentary styling. The important declaration is the `flex` value of `0 0 100%`, which forces each item to be as wide as the container (the `<ul>`). As a result, the content will overflow its container, and the viewport will scroll horizontally.

```
li {
  list-style-type: none;
  background-color: #eeeeee;
  border: 1px solid #dddddd;
  padding: 20px;

  flex: 0 0 100%;
}

li:nth-child(even) {
  background-color: cyan;
}
```

In addition, every even-numbered list item is given a different background-color via `:nth-child()`, so that it is easier to see the scrolling effect.

Setting up scroll snapping on the list

In this section, we will set an overflow value on the `<ul>` to turn it into a scroll container, then apply CSS scroll snapping to cause the list to snap to the center of each list item as the content is scrolled.

An `overflow-x` value of `scroll` is set on the `<ul>` so that its content will scroll horizontally within the list, rather than the entire viewport scrolling. CSS scroll snap is then used to snap to each "page" — a `scroll-snap-type` value of `x mandatory` is set to make the list into a scroll snap container. The `x` keyword causes the container's snap targets to be snapped to horizontally, while the `mandatory` keyword means that the container will always snap to a snap target at the end of a scrolling action.

```
ul {
  overflow-x: scroll;
  scroll-snap-type: x mandatory;
}
```

Next, a `scroll-snap-align` value of `center` is set on the list items so that, when the list is scrolled, it snaps to the center of each list item.

```
li {
  scroll-snap-align: center;
}
```

The code shown so far renders as follows:

Try scrolling the list by swiping or using the scrollbar to see the scroll snapping effect. No matter where you end your scroll motion, an item will always "snap" into place.

> **Note:** CSS scroll snapping is not mandatory to use the CSS carousel features. However, carousels work a lot better with scroll snapping included. Without scroll snapping, the scroll buttons and markers will be unlikely to navigate cleanly between pages, and the result will be substandard.

Creating scroll buttons

In this section we add "previous" and "next" scroll buttons to the demo to provide a tool to navigate between carousel pages. This is achieved using the `::scroll-button()` pseudo-element.

The `::scroll-button()` pseudo-elements generate buttons inside a scroll container only when their `content` properties are set to a value other than `none`. Each `::scroll-button()` represents a scroll button, the scrolling direction of which is specified by the selector's argument. You can generate up to four scroll buttons per scroll container, each scrolling the container's content towards the start or end of the block or inline axis.

You can also specify an argument of `*` to target all of the `::scroll-button()` pseudo-elements with styles.

First, all scroll buttons are targeted with some rudimentary styles, as well as styling based on different states. It is important to set `:focus` styles for keyboard users. Also, as scroll buttons are automatically set to `disabled` when no more scrolling can occur in that direction, we use the `:disabled` pseudo-class to target this state.

```
ul::scroll-button(*) {
  border: 0;
  font-size: 2rem;
  background: none;
  color: black;
  opacity: 0.7;
  cursor: pointer;
}

ul::scroll-button(*):hover,
ul::scroll-button(*):focus {
  opacity: 1;
}
```

```
ul::scroll-button(*):active {
  translate: 1px 1px;
}

ul::scroll-button(*):disabled {
  opacity: 0.2;
  cursor: unset;
}
```

> **Note:** We also set a `cursor` value of `pointer` on the scroll buttons to make it more obvious that they can be interacted with (an improvement for both general UX and cognitive accessibility), unsetting it when the scroll buttons are `:disabled`.

Next, an appropriate icon is set on the left and right scroll buttons via the `content` property, which is also what causes the scroll buttons to be generated. That icon must also have a corresponding plain text accessible name, so use the alternative text feature of the `content` property (with the caveat that this may still be insufficient to satisfy WCAG):

```
ul::scroll-button(left) {
  content: "◂" / "Previous";
}

ul::scroll-button(right) {
  content: "▸" / "Next";
}
```

> **Note:** User agents should automatically give an appropriate accessible name to generated scroll buttons so that assistive technologies can appropriately announced them and the buttons should have an implicit `role` of `button`. Providing alternative text for generated content ensures the buttons have the accessible names of "scroll left" and "scroll right" in user agents that don't natively include scroll button accessibility features.

Positioning scroll buttons

We've created the scroll buttons. Now we will position them relative to the carousel using CSS anchor positioning.

First of all, a reference `anchor-name` is set on the list. Next, each scroll button has its `position` set to `absolute`, and its `position-anchor` property set to the same reference name defined on the list, to associate the two together.

```
ul {
  anchor-name: --my-carousel;
}

ul::scroll-button(*) {
  position: absolute;
  position-anchor: --my-carousel;
}
```

To actually position each scroll button, we set values on their inset properties. We use `anchor()` functions to position the specified sides of the buttons relative to the sides of the carousel. In each case, the `calc()` function is used to add some space between the button edge and the carousel edge. For example, the right-hand edge of the left scroll button is positioned 70 pixels to the right of the carousel's left-hand edge.

```
ul::scroll-button(left) {
  right: calc(anchor(left) - 70px);
  bottom: calc(anchor(top) + 13px);
}

ul::scroll-button(right) {
  left: calc(anchor(right) - 70px);
  bottom: calc(anchor(top) + 13px);
}
```

Adding in the scroll button code, we get the following result:

Try pressing the "previous" and "next" scroll buttons to see how the pages are scrolled, respecting the scroll-snapping behavior. Also note how the "previous" button is automatically disabled when the list is scrolled to the start of the content, while the "next" button is automatically disabled when the list is scrolled to the end of the content.

Creating scroll markers

Scroll markers are a group of buttons, each one of which scrolls the carousel to a position related to one of the content pages. They provide an additional navigation tool that also indicates your progress through the carousel pages.

In this section, we will add scroll markers to the carousel, which involves three main features:

- The `scroll-marker-group` property is set on the scroll container element. It needs to be set to a non-`none` value for the `::scroll-marker-group` pseudo-element to be generated; its value specifies where the scroll marker group appears in the carousel's tab order and layout box order (but not DOM structure) — `before` puts it at the start, before the scroll buttons, while `after` puts it at the end.
- The `::scroll-marker-group` pseudo-element exists inside a scroll container, and is used to collect together, contain, and lay out scroll markers as a whole group.
- `::scroll-marker` pseudo-elements exist inside the children and `::column` fragments of a scroll container ancestor, and represent their scroll markers. These are collected inside the ancestor's `::scroll-marker-group` for layout purposes.

To begin with, the list's `scroll-marker-group` property is set to `after`, so the `::scroll-marker-group` pseudo-element is placed after the list's DOM content in the focus and layout box order; this means it comes after the scroll buttons:

```
ul {
  scroll-marker-group: after;
}
```

> **Note:** Alternatively, a scroll marker group container can be created from an existing element containing a set of `<a>` elements using `scroll-target-group`.

Next, the list's `::scroll-marker-group` pseudo-element is positioned relative to the carousel using CSS anchor positioning, similar to the scroll button's, except that it is horizontally centered on the carousel using a `justify-self` value of `anchor-center`. The group is laid out using flexbox, with a `justify-content` value of `center` and a `gap` of `20px` so that its children (the `::scroll-marker` pseudo-elements) are centered inside the `::scroll-marker-group` with a gap between each one.

```
ul::scroll-marker-group {
  position: absolute;
  position-anchor: --my-carousel;
  top: calc(anchor(bottom) - 70px);
  justify-self: anchor-center;
```

```
  display: flex;
  justify-content: center;
  gap: 20px;
}
```

Next, we handle the look and feel of the scroll markers themselves; they can be styled just like any other generated content. It is important to note that we need to set a non-`none` value for the `content` property so the scroll markers are actually generated. A value of `""` produces a blank accessible name (a WCAG SC 4.1.2 Name, Role, Value violation), so it's necessary to provide some text value. Because each scroll marker having the same name may not be useful for users, consider using `attr()` to pull the value from a custom `data-` attribute on the `<li>` (this example looks for an attribute `data-accName`). Understand this value will not be picked up by automated translation services.

We also set some rudimentary styles to make the markers appear as outlined circles while visually hiding the text content we've added:

```
li::scroll-marker {
  content: attr(data-accName);
  width: 16px;
  height: 16px;
  background-color: transparent;
  border: 2px solid black;
  border-radius: 50%;
  overflow: hidden;
  text-indent: 16px;
}
```

> **Note:** Generated content is inline by default; we can apply `width` and `height` to our scroll markers because they are being laid out as flex items.

Finally for this section, the `:target-current` pseudo-class is used to select whichever scroll marker corresponds to the currently visible "page", highlighting how far the user has scrolled through the content. In this case, we set the `background-color` to `black` so it is styled as a filled-in circle.

```
li::scroll-marker:target-current {
  background-color: black;
}
```

> **Note:** When a scroll marker group container is created on a scroll container using the `scroll-marker-group` property, the scroll container is rendered with `tablist`/`tab` semantics. You can `Tab` to it with the keyboard, then move between the different "pages" using the left and right (or up and down) cursor keys, which also changes the state of the associated scroll markers and scroll buttons as expected. The scroll markers can also be tabbed between normally, as expected.

Single page carousel final result

All of the above code combines together to create the following result:

Since the previous live example, the scroll markers have been added — try pressing them to jump straight to each page. Note how the current marker is highlighted so you can see where you are in the pagination. Also try tabbing to the scroll marker group, then use the cursor keys to cycle through each page.

You can also navigate between pages by swiping left and right, dragging the scroll bar, or pressing the scroll buttons.

Responsive carousel: multiple items per page

The second demo is a carousel with multiple items per page, which again includes scroll buttons and scroll markers for navigating through the pages. This demo is also responsive — different numbers of items appear on each page depending on the viewport width.

This demo is very similar to the Carousel with single pages demo, except that instead of using flexbox for layout, it uses CSS multi-column layout and the `::column` pseudo-element to create arbitrary columns that span the full width of the carousel and may contain multiple items.

> **Note:** There is currently no way to provide an accessible name for the scroll markers (there is no corresponding HTML element from which to draw a `data-` value as in the prior example). Using this pattern will create a WCAG SC 4.1.2 Name, Role, Value violation.

Using this approach, we can be sure that if the viewport grows or shrinks, while the item size remains constant, we'll never have a partial item displayed off the edge of the scrollport. In this case, the scroll markers are created on scroll container fragments, per-column, rather than on children, per-item.

The HTML is very similar to that of the previous demo, except that there are significantly more list items and, as multiple items will be visible at a time, we are labeling them as items rather than pages:

```
...
  <li>
    <h2>Item 1</h2>
  </li>
...
```

This demo also has very similar CSS, with the exception of the rules explained in the following sections.

```
<h1>CSS carousel multiple items per page</h1>
<ul>
  <li>
    <h2>Item 1</h2>
  </li>
  <li>
    <h2>Item 2</h2>
  </li>
  <li>
    <h2>Item 3</h2>
  </li>
  <li>
    <h2>Item 4</h2>
  </li>
  <li>
    <h2>Item 5</h2>
  </li>
  <li>
    <h2>Item 6</h2>
  </li>
  <li>
    <h2>Item 7</h2>
  </li>
```

```
  <li>
    <h2>Item 8</h2>
  </li>
  <li>
    <h2>Item 9</h2>
  </li>
  <li>
    <h2>Item 10</h2>
  </li>
  <li>
    <h2>Item 11</h2>
  </li>
  <li>
    <h2>Item 12</h2>
  </li>
  <li>
    <h2>Item 13</h2>
  </li>
  <li>
    <h2>Item 14</h2>
  </li>
  <li>
    <h2>Item 15</h2>
  </li>
</ul>
```

Carousel layout using columns

This example uses CSS multi-column layout, rather than flexbox, to layout the carousel items. The `columns` value of `1` forces each column to be the full width of the container, with the contents displaying a single column at a time. A `text-align` value of `center` is also applied, forcing the contents to align with the center of the list.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  text-align: center;
  font-size: 1.7rem;
}
```

```
ul {
  width: 100vw;
  height: 300px;
  padding: 10px;

  overflow-x: scroll;
  scroll-snap-type: x mandatory;

  columns: 1;
  text-align: center;
}
```

We provide rudimentary box styling for the list items, then apply layout styles to allow one or more items to fit into the single content column, depending on the viewport width. The number dynamically changes as the list gets wider or narrower.

```
li {
  list-style-type: none;

  display: inline-block;
  height: 100%;
  width: 200px;

  background-color: #eeeeee;
  border: 1px solid #dddddd;
  padding: 20px;
  margin: 0 10px;

  text-align: left;
}

li:nth-child(even) {
  background-color: cyan;
}
```

The key layout properties are as follows:

- A `display` value of `inline-block` has been set to force the list items to sit alongside one another and make the list scroll horizontally.
- An absolute `width` of `200px` has been set on them to control their sizing, meaning one or more will fit in a column that grows and shrinks along with the width of the viewport.
- A `text-align` value of `left` is set on them to override the `text-align: center` set on the parent container, so the item content will be left-aligned.

The `scroll-snap-align` property is now set on the `::column` pseudo-elements — which represent the content columns generated by the `columns` property — rather than the list items. We want to snap to each complete column rather than every individual list item, showing all new items with each scroll action.

```
ul::column {
  scroll-snap-align: center;
}
```

```
ul::scroll-button(*) {
  border: 0;
  font-size: 2rem;
  background: none;
  color: black;
  opacity: 0.7;
  cursor: pointer;
```

```
}

ul::scroll-button(*):hover,
ul::scroll-button(*):focus {
  opacity: 1;
}

ul::scroll-button(*):active {
  translate: 1px 1px;
}

ul::scroll-button(*):disabled {
  opacity: 0.2;
  cursor: unset;
}

ul::scroll-button(left) {
  content: "◄";
}

ul::scroll-button(right) {
  content: "►";
}

ul {
  anchor-name: --my-carousel;
}

ul::scroll-button(*) {
  position: absolute;
  position-anchor: --my-carousel;
}

ul::scroll-button(left) {
  right: calc(anchor(left) - 70px);
  bottom: calc(anchor(top) + 13px);
}

ul::scroll-button(right) {
  left: calc(anchor(right) - 70px);
  bottom: calc(anchor(top) + 13px);
}

ul {
  scroll-marker-group: after;
}

ul::scroll-marker-group {
  position: absolute;
  position-anchor: --my-carousel;
  top: calc(anchor(bottom) - 70px);
  justify-self: anchor-center;
  display: flex;
  justify-content: center;
  gap: 20px;
}
```

Column scroll markers

The CSS for creating the scroll markers in this demo is nearly identical to the previous demo, except that the selectors are different — the scroll markers are created on the generated `::column` pseudo-elements rather than the list items. Note how we are including two pseudo-elements here to generate scroll markers on the generated columns.

```
ul::column::scroll-marker {
  content: "";
  width: 16px;
  height: 16px;
  background-color: transparent;
  border: 2px solid black;
  border-radius: 10px;
}
```

Finally, we use the `:target-current` pseudo-class to mark the active scroll marker, providing the user with an idea of where they are in the navigation. We also use the `:target-before` and `:target-after` pseudo-classes to apply some custom styling to the scroll markers before and after the active one. We also set a `transition` on the `ul::column::scroll-marker:target-current` rule so that the style changes between the different states animate smoothly.

```
ul::column::scroll-marker:target-before {
  border: 2px solid gray;
}

ul::column::scroll-marker:target-current {
  background-color: black;
  transition: all 0.7s;
}

ul::column::scroll-marker:target-after {
  border: 2px solid red;
  background-color: red;
}
```

Responsive carousel final result

The Responsive carousel is rendered as follows:

Try navigating between the different pages by swiping left and right, using the scroll bar, pressing the scroll buttons, and pressing the scroll markers. The functionality is similar to the single page flexbox example, except that now there are multiple list items in each navigated position; the scroll markers are set on column fragments, potentially containing multiple items, instead of on each item.

Also, try resizing the screen width and you'll see that the number of list items that can fit inside the list changes — and therefore the number of generated columns changes too. As the number of columns changes, the number of scroll markers dynamically updates so that each column is represented in the scroll marker group.

See also

- CSS overflow module
- CSS anchor positioning module
- CSS scroll snap module
- CSS Carousel Gallery via chrome.dev (2025)

CSS overscroll behavior

The **CSS overscroll behavior** module provides properties to control the behavior of a scroll container when its scroll position reaches the scroll boundary. Controlling this aspect is particularly useful in scenarios where embedded scrollable areas should not trigger scrolling of the parent container.

When commenting on a blog, you might notice that if your comment exceeds the length of the provided `<textarea>`, scrolling beyond the end of the text area causes the entire blog to scroll. This is because reaching the end of a scrollable area, known as the scroll boundary, can lead to scrolling other content or the entire page. This continuous scrolling experience is called scroll chaining.

In situations where the contents of an element are larger than its container and `overflow` allows or defaults to scrolling (like in `<textarea>`), continued scrolling past the element's scrollable area will initiate scrolling in the parent element or the underlying page.

Conversely, scrolling through a website's terms and conditions and reaching the end of the content to enable a checkbox, may not force the page to scroll or bounce (as on a phone). This example shows that you can control overscroll behavior and prevent scroll chaining.

This module defines the overscroll behavior, enabling you to specify the actions when a user scrolls beyond the boundaries of a scrollable element.

Reference

CSS properties

- `overscroll-behavior` shorthand
- `overscroll-behavior-block`
- `overscroll-behavior-inline`
- `overscroll-behavior-x`
- `overscroll-behavior-y`

Glossary terms

- Scroll boundary
- Scroll chaining

Guides

Learn: Overflowing content

Learn what overflow is and how to manage it.

Related concepts

- `scrollbar` ARIA role
- Containing block concept
- CSS overflow module:
  - `overflow` shorthand property
    - `overflow-x`
    - `overflow-y`
    - `overflow-block`
    - `overflow-inline`
  - `overflow-clip-margin` property
  - `scroll-behavior` property
  - `text-overflow` property
- Scroll container and scrollport glossary terms
- CSS scroll snap module:
  - `scroll-padding` shorthand property
  - `scroll-snap-type` property
  - `scroll-margin` shorthand property
  - `scroll-snap-stop` property
  - `scroll-snap-align` property
- CSSOM view module:
  - `Element.getBoundingClientRect()` method
  - `Element.scroll()` method
  - `Element.scrollBy()` method
  - `Element.scrollIntoView()` method
  - `Element.scrollTo()` method
  - `scroll` Document event

See also

- CSS scroll anchoring module
- CSS scroll snap module
- CSS box model module
- CSS logical properties and values module

CSS paged media

The **CSS paged media** module defines the properties that control the presentation of content for print or any other media that splits content into discrete pages. It allows you to set page breaks, control printable areas, and style left and right pages differently.

The CSS paged media module specifies how pages are generated and laid out to hold fragmented content in a paged presentation, including content that is printed or represented as a print preview. The module defines functionality for controlling page margins, size, orientation, and headers and footers. It extends generated content to provide functionality for generating page numbers and running headers and footers.

The process of paginating content into generated pages and controlling breaks inside elements is covered in the CSS fragmentation module.

Reference

Properties

- `page`

At-rules and descriptors

- `@page`
  - `page-orientation` descriptor
  - `size` descriptor
  - Margin descriptors
- Margin at-rules

The CSS paged media module also introduces the `bleeds` and `marks` descriptors of the `@page` at-rule. Currently, no browsers support these features.

Selectors

- `:blank`
- `:first`
- `:left`
- `:right`

Guides

Printing

Tips and techniques for helping improve web content printer output.

Related concepts

- CSS fragmentation module
  - `break-after` property
  - `break-before` property
  - `break-inside` property
  - `orphans` property
  - `widows` property

See also

- CSS fragmentation module
- CSS media queries module

CSS positioned layout

The **CSS positioned layout** module defines the coordinate-based positioning and offsetting schemes available in CSS and the properties used to position and stack elements on a web page. The module is best known for defining the basic positioning methods, including relative positioning, sticky positioning, absolute positioning, and fixed positioning. It also defines how positioned elements are painted and layered, clarifying stacking behavior and visual order.

The positioned layout module defines the physical, logical, and shorthand inset properties. The logical properties enable developing with internationalization and responsiveness in mind.

Like all CSS modules, this module impacts and is impacted by other modules. This module describes how positioning interacts with other layout modules such as CSS flexible box layout and CSS Grid layout. Other modules, such as CSS anchor positioning, build upon this module to enable positioning elements relative to other elements and layers.

Reference

Properties

- `bottom`
- `inset`
- `inset-block`
- `inset-block-end`
- `inset-block-start`
- `inset-inline`
- `inset-inline-end`
- `inset-inline-start`
- `left`
- `overlay`
- `position`
- `right`
- `top`

Selectors

- `::backdrop`

Glossary terms and definitions

- Block direction
- Containing block
- Flow relative values
- Inline direction
- Inset properties
- Positioning context
- Relative position
- Stacking context
- Static positioning
- Top layer

Guides

Understanding z-index

Presents the notion of stacking context and explains how z-ordering works, with several examples.

Stacking without the `z-index` property

The stacking rules that apply when `z-index` is not used.

Stacking floating elements

How floating elements are handled with stacking.

Using `z-index`

How to use `z-index` to change default stacking.

Stacking context

CSS stacking context, the CSS features that create new stacking contexts, and nested stacking contexts.

Learn: positioning

The different position values and how to use them.

Related concepts

- `float`
- `clear`
- `transition-property`
- `transition-behavior`
- Popover API
- `<dialog>` element
- CSS anchor positioning module
  - `position-anchor`
  - `position-area`
  - `position-try-fallbacks`
  - `position-try-order`
  - `position-try` shorthand
  - `position-visibility`
- CSS transforms module
  - `transform`
  - `transform-origin`
  - `translate`

See also

- Other CSS `*-position` properties:
  - `background-position`
  - `background-position`
  - `background-position`
  - `font-synthesis-position`
  - `font-variant-position`
  - `list-style-position`
  - `mask-position`
  - `object-position`
  - `offset-position`
  - `ruby-position`
  - `text-emphasis-position`
  - `text-underline-position`
- Position-related data types
  - `<baseline-position>`
  - `<content-position>`
  - `<overflow-position>`
  - `<self-position>`
- CSS Scroll Snap module
  - Snap positions

Stacking context

Stacking context is a three-dimensional conceptualization of HTML elements along an imaginary z-axis relative to the user, who is assumed to be facing the viewport or the webpage. The stacking context determines how elements are layered on top of one another along the z-axis (think of it as the "depth" dimension on your screen). Stacking context determines the visual order of how overlapping content is rendered.

Elements within a stacking context are stacked independently from elements outside of that stacking context, ensuring elements in one stacking context don't interfere with the stacking order of elements in another. Each stacking context is completely independent of its siblings: only descendant elements are considered when stacking is processed.

Each stacking context is self-contained. After an element's contents are stacked, the entire element is considered as a single unit in the stacking order of its parent stacking context.

Within a stacking context, child elements are stacked according to the `z-index` values of all the siblings. The stacking contexts of these nested elements only have meaning in this parent. Stacking contexts are treated atomically as a single unit in the parent stacking context. Stacking contexts can be contained in other stacking contexts, and together create a hierarchy of stacking contexts.

The hierarchy of stacking contexts is a subset of the hierarchy of HTML elements because only certain elements create stacking contexts. Elements that don't create their own stacking contexts are *assimilated* by the parent stacking context.

Features creating stacking contexts

A stacking context is formed, anywhere in the document, by any element in the following scenarios:

- Root element of the document (`<html>`).
- Element with a `position` value `absolute` or `relative` and `z-index` value other than `auto`.
- Element with a `position` value `fixed` or `sticky`.
- Element with a `container-type` value `size` or `inline-size` set (See container queries).

- Element that is a flex item with a `z-index` value other than `auto`.
- Element that is a grid item with `z-index` value other than `auto`.
- Element with an `opacity` value less than `1`.
- Element with a `mix-blend-mode` value other than `normal`.
- Element with any of the following properties with a value other than `none`:
  - `transform`
  - `scale`
  - `rotate`
  - `translate`
  - `filter`
  - `backdrop-filter`
  - `perspective`
  - `clip-path`
  - `mask` / `mask-image` / `mask-border`
- Element with the `isolation` value `isolate`.
- Element with a `will-change` value specifying any property that would create a stacking context on non-initial value.
- Element with a `contain` value of `layout` or `paint`, or a composite value that includes either of these values (i.e., `contain: strict`, `contain: content`).
- Element placed into the top layer and its corresponding `::backdrop`. Examples include fullscreen and popover elements.
- Element that has had stacking context-creating properties (such as `opacity`) animated using `@keyframes`, with `animation-fill-mode` set to `forwards`.

Nested stacking contexts

Stacking contexts can be contained in other stacking contexts, and they can together create a hierarchy of stacking contexts.

The root element of a document is a stacking context which, in most cases, contains nested stacking contexts, many of which will contain additional stacking contexts. Within each stacking context, child elements are stacked according to the same rules explained in Using `z-index`. Importantly, the `z-index` values of its child stacking contexts only have meaning within its parent's stacking context. Stacking contexts are treated atomically as a single unit in the parent stacking context.

To figure out the *rendering order* of stacked elements along the z-axis, think of each index value as a "version number" of sorts, where child elements represent minor version numbers underneath their parent's major version number.

To demonstrate how the stacking order of each element participates in the stacking order of their ancestor stacking contexts, let's look at an example page with six container elements. There are three sibling `<article>` elements. The last `<article>` contains three sibling `<section>` elements, with the `<h1>` and `<code>` of that third article appearing between the first and second sibling `<section>` elements.

```
<article id="container1">
  <h1>Article element #1</h1>
  <code>
    position: relative;<br />
    z-index: 5;
  </code>
</article>

<article id="container2">
  <h1>Article Element #2</h1>
  <code>
    position: relative;<br />
    z-index: 2;
  </code>
</article>

<article id="container3">
  <section id="container4">
    <h1>Section Element #4</h1>
    <code>
      position: relative;<br />
      z-index: 6;
    </code>
  </section>

  <h1>Article Element #3</h1>
  <code>
    position: absolute;<br />
    z-index: 4;
  </code>

  <section id="container5">
    <h1>Section Element #5</h1>
    <code>
      position: relative;<br />
      z-index: 1;
    </code>
  </section>

  <section id="container6">
    <h1>Section Element #6</h1>
    <code>
      position: absolute;<br />
      z-index: 3;
    </code>
  </section>
</article>
```

Every container element has an `opacity` of less than `1` (which creates a stacking context), and a `position` of either `relative` or `absolute` (which creates a stacking context when the element also has a `z-index` value other than `auto`).

```
* {
  margin: 0;
}
html {
  padding: 20px;
```

```
  font:
    12px/20px "Arial",
    sans-serif;
}
h1 {
  font-size: 1.25em;
}
#container1,
#container2 {
  border: 1px dashed #669966;
  padding: 10px;
  background-color: #ccffcc;
}
#container1 {
  margin-bottom: 190px;
}
#container3 {
  border: 1px dashed #990000;
  background-color: #ffdddd;
  padding: 40px 20px 20px;
  width: 330px;
}
#container4 {
  border: 1px dashed #999966;
  background-color: #ffffcc;
  padding: 25px 10px 5px;
  margin-bottom: 15px;
}
#container5 {
  border: 1px dashed #999966;
  background-color: #ffffcc;
  margin-top: 15px;
  padding: 5px 10px;
}
#container6 {
  background-color: #ddddff;
  border: 1px dashed #000099;
  padding-left: 20px;
  padding-top: 125px;
  width: 150px;
  height: 125px;
}
```

```
section,
article {
  opacity: 0.85;
  position: relative;
}
#container1 {
  z-index: 5;
}
#container2 {
  z-index: 2;
}
#container3 {
  z-index: 4;
  position: absolute;
  top: 40px;
  left: 180px;
}
#container4 {
  z-index: 6;
}
#container5 {
  z-index: 1;
}
#container6 {
  z-index: 3;
  position: absolute;
  top: 20px;
  left: 180px;
}
```

The CSS properties for colors, fonts, alignment, and box-model have been hidden for brevity.

The hierarchy of stacking contexts in the above example is as follows:

```
Root
|
├── ARTICLE #1
├── ARTICLE #2
└── ARTICLE #3
  |
  ├── SECTION #4
  ├── SECTION #5
  └── SECTION #6
```

The three `<section>` elements are children of ARTICLE #3. Therefore, the stacking of the section elements is completely resolved within ARTICLE #3. Once stacking and rendering within ARTICLE #3 is completed, the whole ARTICLE #3 element is passed for stacking in the root element with respect to its sibling `<article>` elements.

By comparing the `z-index` as "version numbers", we can see how an element with a `z-index` of `1` (SECTION #5) is stacked above an element with a `z-index` of `2` (ARTICLE #2), and how an element with a `z-index` of `6` (SECTION #4) is stacked below an element with a `z-index` of `5` (ARTICLE #1). SECTION #4 is rendered under ARTICLE #1 because ARTICLE #1's z-index (`5`) is valid within the stacking context of the root element, while SECTION #4's z-index (`6`) is valid within the stacking context of ARTICLE #3 (`z-index: 4`). So SECTION #4 is under ARTICLE #1 because SECTION #4 belongs to ARTICLE #3, which has a lower z-index value (`4-6` is less than `5-0`).

For the same reason, ARTICLE #2 (`z-index: 2`) is rendered under SECTION #5 (`z-index`: 1) because SECTION #5 belongs to ARTICLE #3 (`z-index: 4`), which has a higher z-index value (`2-0` is less than `4-1`).

ARTICLE #3's z-index is `4`, but this value is independent of the `z-index` of three sections nested inside it because they belong to a different stacking context.

In our example (sorted according to the final rendering order):

- Root
  - ARTICLE #2: (`z-index`: 2), which results in a rendering order of `2-0`
  - ARTICLE #3: (`z-index`: 4), which results in a rendering order of `4-0`
    - SECTION #5: (`z-index`: 1), stacked under an element (`z-index`: 4), which results in a rendering order of `4-1`
    - SECTION #6: (`z-index`: 3), stacked under an element (`z-index`: 4), which results in a rendering order of `4-3`
    - SECTION #4: (`z-index`: 6), stacked under an element (`z-index`: 4), which results in a rendering order of `4-6`
  - ARTICLE #1: (`z-index`: 5), which results in a rendering order of `5-0`

Additional examples

Additional examples include a 2-level hierarchy with `z-index` on the last level, a 2-level HTML hierarchy, `z-index` on all levels, and a 3-level HTML hierarchy, `z-index` on the second level.

See also

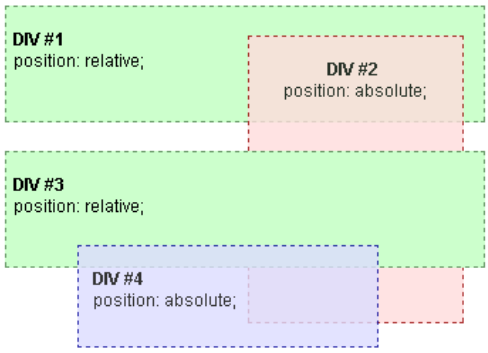
- Understanding z-index
- Stacking without the `z-index` property
- Stacking floating elements
- Using z-index
- Top layer
- CSS positioned layout module

Stacking context example 1

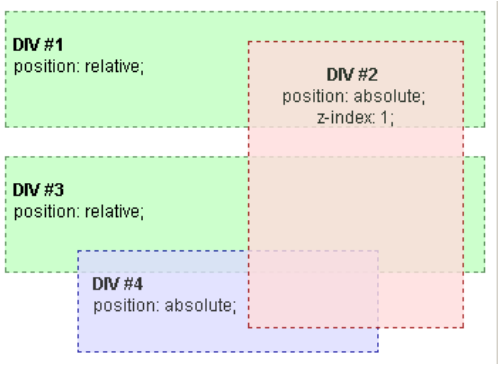
Description

Let's start with a basic example. In the root stacking context, there are two relatively positioned `<div>` elements (DIV #1 and DIV #3) without `z-index` properties. Inside DIV #1, there is an absolutely positioned DIV #2, while in DIV #3, there is an absolutely positioned DIV #4, both without `z-index` properties.

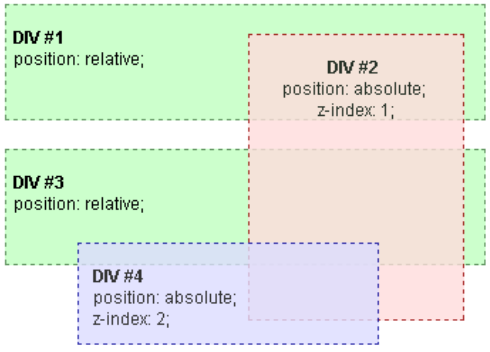
The only stacking context is the root context. Without `z-index` values, elements are stacked in order of occurrence.



If DIV #2 is assigned a positive (non-zero and non-auto) z-index value, it is rendered above all the other DIVs.



Then if DIV #4 is also assigned a positive z-index greater than DIV #2's z-index, it is rendered above all the other DIVs including DIV #2.



In this last example you can see that DIV #2 and DIV #4 are not siblings, because they belong to different parents in the HTML elements' hierarchy. Even so, stacking of DIV #4 with respect of DIV #2 can be controlled through `z-index`. It happens that, since DIV #1 and DIV #3 are not assigned any `z-index` value, or any other features that create a stacking context, they do not create a stacking context. This means that all their content, including DIV #2 and DIV #4, belongs to the same root stacking context.

In terms of stacking contexts, DIV #1 and DIV #3 are assimilated into the root element, and the resulting hierarchy is the following:

- Root stacking context
  - DIV #2 (`z-index`: 1)
  - DIV #4 (`z-index`: 2)

> **Note:** DIV #1 and DIV #3 are not translucent. It is important to remember that assigning an opacity value of less than `1` to an element implicitly creates a stacking context, as does adding a `z-index` value to a positioned element. This example also shows what happens when a parent element doesn't create a stacking context.

Example

HTML

```
<div id="div1">
  <br /><span class="bold">DIV #1</span> <br />position: relative;
  <div id="div2">
    <br /><span class="bold">DIV #2</span> <br />position: absolute;
    <br />z-index: 1;
  </div>
</div>

<br />

<div id="div3">
  <br /><span class="bold">DIV #3</span> <br />position: relative;
  <div id="div4">
    <br /><span class="bold">DIV #4</span> <br />position: absolute;
    <br />z-index: 2;
  </div>
</div>
```

CSS

```
.bold {
  font-family: "Arial";
  font-size: 12px;
  font-weight: bold;
}

#div1,
#div3 {
  height: 80px;
  position: relative;
  border: 1px dashed #669966;
  background-color: #ccffcc;
  padding-left: 5px;
}

#div2 {
  opacity: 0.8;
  z-index: 1;
  position: absolute;
  width: 150px;
  height: 200px;
  top: 20px;
  left: 170px;
  border: 1px dashed #990000;
  background-color: #ffdddd;
  text-align: center;
}

#div4 {
  opacity: 0.8;
  z-index: 2;
  position: absolute;
  width: 200px;
  height: 80px;
  top: 65px;
  left: 50px;
  border: 1px dashed #000099;
  background-color: #ddddff;
  text-align: left;
  padding-left: 10px;
}
```

Result

See also

- Example: 2-level HTML hierarchy, `z-index` on all levels
- Example: 3-level HTML hierarchy, `z-index` on the second level
- Stacking context
- CSS positioned layout module

Stacking context example 2

Description

This is a basic example, but it is the key for understanding the concept of *stacking context*. There are the same four DIVs of the previous example, but now `z-index` properties are assigned on both levels of the hierarchy.

You can see that DIV #2 (`z-index`: 2) is above DIV #3 (`z-index`: 1), because they both belong to the same stacking context (the root one), so z-index values rule how elements are stacked.

What can be considered strange is that DIV #2 (`z-index`: 2) is above DIV #4 (`z-index`: 10), despite their z-index values. The reason is that they do not belong to the same stacking context. DIV #4 belongs to the stacking context created by DIV #3, and as explained previously DIV #3 (and all its content) is under DIV #2.

To better understand the situation, this is the stacking context hierarchy:

- Root stacking context
  - DIV #2 (`z-index`: 2)
  - DIV #3 (`z-index`: 1)
    - DIV #4 (`z-index`: 10)

> **Note:** It is worth remembering that the HTML hierarchy is different from the stacking context hierarchy. In the stacking context hierarchy, elements that do not create a stacking context are collapsed on their parent.

Example

HTML

```
<div id="div1">
  <br />
  <span class="bold">DIV #1</span><br />
  position: relative;
  <div id="div2">
    <br />
    <span class="bold">DIV #2</span><br />
    position: absolute;<br />
    z-index: 2;
  </div>
</div>

<br />

<div id="div3">
  <br />
  <span class="bold">DIV #3</span><br />
  position: relative;<br />
  z-index: 1;
  <div id="div4">
    <br />
    <span class="bold">DIV #4</span><br />
    position: absolute;<br />
    z-index: 10;
  </div>
</div>
```

CSS

```
div {
  font: 12px "Arial";
}

span.bold {
  font-weight: bold;
}

#div2 {
  z-index: 2;
}
#div3 {
  z-index: 1;
}
#div4 {
  z-index: 10;
}

#div1,
#div3 {
  height: 80px;
  position: relative;
  border: 1px dashed #669966;
  background-color: #ccffcc;
  padding-left: 5px;
}

#div2 {
  opacity: 0.8;
  position: absolute;
  width: 150px;
  height: 200px;
  top: 20px;
  left: 170px;
  border: 1px dashed #990000;
  background-color: #ffdddd;
  text-align: center;
}

#div4 {
  opacity: 0.8;
  position: absolute;
  width: 200px;
  height: 70px;
  top: 65px;
  left: 50px;
  border: 1px dashed #000099;
  background-color: #ddddff;
  text-align: left;
  padding-left: 10px;
}
```

Result

See also

- Example: 1-level HTML hierarchy, `z-index` on the last level
- Example: 3-level HTML hierarchy, `z-index` on the second level

- Stacking context
- CSS positioned layout module

Stacking context example 3

Description

This example shows problems that arise when mixing several positioned elements in a multi-level HTML hierarchy and when `z-index` values are assigned using class selectors.

The example has a three-level hierarchical menu made from several positioned `div` elements. Second-level and third-level `div` elements appear when a user hovers or clicks on their parents. Usually this kind of menu is script-generated either client-side or server-side, so style rules are assigned with a class selector instead of the id selector.

If the three menu levels partially overlap, then managing stacking could become a problem.

The first-level menu is relatively positioned, creating a stacking context.

The second-level menu is absolutely positioned inside the parent element. In order to put it above all first-level menus, the `z-index` property is used. The problem is that for each second-level menu, a stacking context is created and each third-level menu belongs to the context of its parent.

So a third-level menu will be stacked under the following second-level menus, because all second-level menus share the same z-index value and the default stacking rules apply.

To better understand the situation, here is the stacking context hierarchy (the three dots "..." represent multiple repetition of the previous line):

- Root stacking context
    - LEVEL #1
        - LEVEL #2 (`z-index`: 1)
            - LEVEL #3
            - ...
            - LEVEL #3
        - LEVEL #2 (`z-index`: 1)
        - ...
        - LEVEL #2 (`z-index`: 1)
    - LEVEL #1
    - ...
    - LEVEL #1

This problem can be avoided by removing overlapping between different level menus, or by using individual (and different) `z`-index values assigned through the id selector instead of class selector, or by flattening the HTML hierarchy.

> **Note:** In the source code you will see that second-level and third level menus are made of several `div` elements contained in an absolutely positioned container. This is useful to group and position all of them at once.

Example

HTML

```
<div class="lev1">
  LEVEL #1

  <div id="container1">
    <div class="lev2">
      LEVEL #2 <br />z-index: 1;

      <div id="container2">
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
        <div class="lev3">LEVEL #3</div>
      </div>
    </div>

    <div class="lev2">LEVEL #2 <br />z-index: 1;</div>
    <div class="lev2">LEVEL #2 <br />z-index: 1;</div>
    <div class="lev2">LEVEL #2 <br />z-index: 1;</div>
  </div>
</div>

<div class="lev1">LEVEL #1</div>
<div class="lev1">LEVEL #1</div>
<div class="lev1">LEVEL #1</div>
```

CSS

```
div {
  font: 12px "Arial";
  font-weight: bold;
  padding-left: 5px;
}
.lev1 {
  border: 2px outset #669966;
  background-color: #ccffcc;
}
.lev2 {
  border: 2px outset #990000;
```

```
  background-color: #ffdddd;
}
.lev3 {
  border: 2px outset #000099;
  background-color: #ddddff;
}
```

```
div {
  opacity: 0.9;
}
.lev1 {
  width: 250px;
  height: 70px;
  position: relative;
}

#container1 {
  z-index: 1;
  position: absolute;
  top: 30px;
  left: 75px;
}

.lev2 {
  width: 200px;
  height: 60px;
  position: relative;
}

#container2 {
  z-index: 1;
  position: absolute;
  top: 20px;
  left: 110px;
}

.lev3 {
  z-index: 10;
  width: 100px;
  position: relative;
}
```

Result

See also

- Stacking without the `z-index` property: The stacking rules that apply when `z-index` is not used.
- Stacking floating elements: How floating elements are handled with stacking.
- Using z-index: How to use `z-index` to change default stacking.
- Stacking context: Notes on the stacking context.
- Stacking context example 1: 2-level HTML hierarchy, `z-index` on the last level
- Stacking context example 2: 2-level HTML hierarchy, `z-index` on all levels

Stacking floating elements

For floating elements, the stacking order is a bit different. Floating elements are placed between non-positioned elements and positioned elements:

1. The background and borders of the root element.
2. Descendant non-positioned elements, in order of appearance in the HTML.
3. *Floating elements*.
4. Descendant positioned elements, in order of appearance in the HTML.

See types of positioning for an explanation of positioned and non-positioned elements.

> **Note:** If an `opacity` value is applied to a non-positioned element (i.e., DIV #4 in the example below), something strange happens: the background and border of that block pop up above the floating blocks and the positioned blocks. This is due to a peculiar part of the specification: applying an `opacity` value creates a new stacking context (see What No One Told You About Z-Index).

Example

You can see in this example that the background and border of the non-positioned element (DIV #4) is completely unaffected by floating elements, but the content is affected. This happens according to standard float behavior which can be shown with a rule added to the above list:

1. The background and borders of the root element.
2. Descendant non-positioned elements, in order of appearance in the HTML.
3. Floating elements.
4. *Descendant non-positioned inline elements*.
5. Descendant positioned elements, in order of appearance in the HTML.

HTML

```
<div id="abs1"><strong>DIV #1</strong><br />position: absolute;</div>

<div id="flo1"><strong>DIV #2</strong><br />float: left;</div>

<div id="flo2"><strong>DIV #3</strong><br />float: right;</div>

<br />

<div id="sta1"><strong>DIV #4</strong><br />no positioning</div>

<div id="abs2"><strong>DIV #5</strong><br />position: absolute;</div>

<div id="rel1"><strong>DIV #6</strong><br />position: relative;</div>
```

CSS

```
div {
  padding: 10px;
  text-align: center;
}

strong {
  font-family: sans-serif;
}

#abs1 {
  position: absolute;
  width: 150px;
  height: 200px;
  top: 10px;
  right: 140px;
  border: 1px dashed #990000;
  background-color: #ffdddd;
}

#sta1 {
  height: 100px;
  border: 1px dashed #999966;
  background-color: #ffffcc;
  margin: 0px 10px;
  text-align: left;
}

#flo1 {
  margin: 0px 10px 0px 20px;
  float: left;
  width: 150px;
  height: 200px;
  border: 1px dashed #009900;
  background-color: #ccffcc;
}

#flo2 {
  margin: 0px 20px 0px 10px;
  float: right;
  width: 150px;
  height: 200px;
  border: 1px dashed #009900;
  background-color: #ccffcc;
}

#abs2 {
  position: absolute;
  width: 150px;
  height: 100px;
  top: 80px;
  left: 100px;
  border: 1px dashed #999900;
  background-color: #ffdddd;
}

#rel1 {
  position: relative;
  border: 1px dashed #999966;
  background-color: #ccffff;
  margin: 0px 10px;
  text-align: left;
}
```

Result

See also

- Understanding z-index
- Stacking without the `z-index` property
- Using z-index
- Stacking context
- CSS positioned layout module

Stacking without the z-index property

When the `z-index` property is not specified on any element, elements are stacked in the following order (from bottom to top):

1. The background and borders of the root element.
2. Descendant non-positioned elements, in order of appearance in the HTML.
3. Descendant positioned elements, in order of appearance in the HTML.

See types of positioning for an explanation of positioned and non-positioned elements.

Keep in mind, when the `order` property alters rendering from the *order of appearance in the HTML* within `flex` containers, it similarly affects the order for stacking context.

Example

In this example, DIV #1 through DIV #4 are positioned elements. DIV #5 is static, and so is drawn below the other four elements, even though it comes later in the HTML markup.

HTML

```
<div id="abs1" class="absolute">
  <strong>DIV #1</strong><br />position: absolute;
</div>
<div id="rel1" class="relative">
  <strong>DIV #2</strong><br />position: relative;
</div>
<div id="rel2" class="relative">
  <strong>DIV #3</strong><br />position: relative;
```

```
</div>
<div id="abs2" class="absolute">
  <strong>DIV #4</strong><br />position: absolute;
</div>
<div id="sta1" class="static">
  <strong>DIV #5</strong><br />position: static;
</div>
```

CSS

```
strong {
  font-family: sans-serif;
}

div {
  padding: 10px;
  border: 1px dashed;
  text-align: center;
}

.static {
  position: static;
  height: 80px;
  background-color: #ffffcc;
  border-color: #999966;
}

.absolute {
  position: absolute;
  width: 150px;
  height: 350px;
  background-color: #ffdddd;
  border-color: #990000;
  opacity: 0.7;
}

.relative {
  position: relative;
  height: 80px;
  background-color: #ccffcc;
  border-color: #669966;
  opacity: 0.7;
}

#abs1 {
  top: 10px;
  left: 10px;
}

#rel1 {
  top: 30px;
  margin: 0px 50px;
}

#rel2 {
  top: 15px;
  left: 20px;
  margin: 0px 50px;
}

#abs2 {
  top: 10px;
  right: 10px;
}

#sta1 {
  background-color: #ffffcc;
  margin: 0px 50px;
}
```

Result

See also

- Understanding z-index
- Using z-index
- Stacking context
- Stacking floating elements
- CSS positioned layout module

Understanding z-index

In the most basic cases, when text, images, and other elements are arranged on the page without overlapping, HTML pages can be considered two-dimensional. In such cases, there is a single rendering flow, and all elements are aware of the space taken by others. CSS isn't that simple — CSS positioning, transformation, containment, and other features can cause elements to overlap. In this guide, we introduce the `z-index` property, which lets you place elements in front or behind other elements in the same stacking context.

Layers on the z-axis

The elements rendered on a page are comprised of a series of boxes. Each box has a position in three dimensions. In addition to their inline and block positions, boxes lie along a third dimension known as the *z-axis*. Controlling an element's z-axis position becomes especially relevant when element boxes overlap visually. Several property values can cause elements to overlap. The `z-index` property provides you a way to control how they overlap!

By default, element boxes are rendered on Layer 0. The `z-index` property allows you to position elements on different layers along the z-axis, in addition to the default rendering layer. Each element's position along the imaginary z-axis (z-index value) is expressed as an integer (positive, negative, or zero) and controls the stacking order during rendering. Greater numbers mean elements are closer to the observer.

If you are not familiar with the term 'z-axis', imagine the page as a stack of layers, each with an assigned number. Layers are rendered in numerical order, with larger numbers appearing on top of smaller numbers (*X* in the table below represents an arbitrary positive integer):

Layer	Description
Bottom layer	Farthest from the observer
Layer -X	Layers with negative `z-index` values
Layer 0	Default rendering layer
Layer X	Layers with positive `z-index` values
Top layer	Closest to the observer

Elements in normal flow

By default, when no `z-index` property is specified, elements are rendered on the default rendering layer (Layer 0).

Consider the following three elements:

```
<div id="div1">#1</div>
<div id="div2">#2</div>
<div id="div3">#3</div>
```

Without any positioning properties applied, these elements flow normally in document order, one after another, without overlapping.

```
div {
  height: 100px;
  width: 100px;
  outline: 1px dotted;
  line-height: 100px;
  font-size: 40px;
  text-align: center;
  font-family: "Helvetica", "Arial", sans-serif;
}

#div1 {
  background-color: lightpink;
}

#div2 {
  background-color: lightyellow;
}

#div3 {
  background-color: lightgreen;
}
```

Default layering behavior

To stack the elements, we can position them. If we use absolute positioning to place them in (almost) the same spot, the default stacking order follows the source order: the first element in the HTML appears at the bottom layer and the last element appears at the top layer.

```
div {
  position: absolute;
}

#div1 {
  top: 0;
  left: 0;
}

#div2 {
  top: 10px;
  left: 10px;
}

#div3 {
  top: 20px;
  left: 20px;
}
```

Rearranging layers

We can use the CSS `z-index` property to position each element along the z-axis, effectively rearranging the stacking order.

By adding `z-index` values, we change the default layer order:

```
#div1 {
  z-index: 5;
}

#div2 {
  z-index: -9;
}

#div3 {
  z-index: 0;
}
```

The element with the lowest `z-index` value appears on the bottom layer. The element with the highest `z-index` value appears on the top layer. In this example, `-9` is the lowest value, so `#div2` is behind all the others. The first element in the source order, `#div1`, has the greatest value, so it appears on top of all the others.

Impact of stacking contexts

Using `z-index` may appear fairly straightforward at first: a single property assigned a single integer number with a seemingly understandable behavior. When `z-index` is applied to complex hierarchies of HTML elements, many find the resulting behavior hard to understand or predict.

If the elements are not siblings, the stacking behavior can become more complicated because each element may belong to a different stacking context. This is shown in the example below.

```
<section>
  <div id="div1">#1</div>
```

```
  <div id="div2">#2</div>
</section>
<div id="div3">#3</div>
```

```
section {
  position: absolute;
  z-index: 2;
}
```

Even though the `z-index` value of `#div3` (`0`) is greater than that of `#div2` (`-9`), `#div2` appears above `#div3` because `#div1` and `#div2` are nested in a separate stacking context created by `<section>`. The `<section>` element and `#div3` are siblings, and since the `<section>` element's z-index is greater than that of `#div3` (`2` versus `0`), `#div3` is placed behind `<section>` and all of `<section>`'s contents. For more in-depth details about the topic, see our Stacking context guide.

Conclusion

As we've seen in this guide, `z-index` provides a way to control how elements stack along z-axis. You learned how the integer values of the `z-index` property can be used to alter the stacking order. However, as demonstrated in the last example, stacking orders can be complicated. Stacking orders follow a series of complex stacking rules to ensure that all browsers stack the same content in the same manner providing consistency and predictability. It's important to understand the features that create stacking contexts and how nested stacking contexts affect layer order.

See also

- Stacking without the `z-index` property
- Stacking floating elements
- Using `z-index`
- Stacking context
- CSS positioned layout module

Using z-index

The first article of this guide, Stacking without the `z-index` property, explains how stacking is arranged by default. If you want to create a custom stacking order, you can use the `z-index` property on a positioned element.

The `z-index` property can be specified with an integer value (positive, zero, or negative), which represents the position of the element along an imaginary z-axis. If you are not familiar with the term 'z-axis', imagine the page as a stack of layers, each one having a number. Layers are rendered in numerical order, with larger numbers above smaller numbers (*X* represents an arbitrary positive integer):

Layer	Description
Bottom layer	Farthest from the observer
Layer -X	Layers with negative `z-index` values
Layer 0	Default rendering layer
Layer X	Layers with positive `z-index` values
Top layer	Closest to the observer

> **Note:**
>
> - When no `z-index` property is specified, elements are rendered on the default rendering layer (Layer 0).
> - If several elements share the same `z-index` value (i.e., they are placed on the same layer), stacking rules explained in the section Stacking without the `z-index` property apply.

Example

In this example, the layers' stacking order is rearranged using `z-index`. The `z-index` of DIV #5 has no effect since it is not a positioned element.

HTML

```
<div id="abs1">
  <strong>DIV #1</strong>
  <br />position: absolute; <br />z-index: 5;
</div>

<div id="rel1">
  <strong>DIV #2</strong>
  <br />position: relative; <br />z-index: 3;
</div>

<div id="rel2">
  <strong>DIV #3</strong>
  <br />position: relative; <br />z-index: 2;
</div>

<div id="abs2">
  <strong>DIV #4</strong>
  <br />position: absolute; <br />z-index: 1;
</div>

<div id="sta1">
  <strong>DIV #5</strong>
  <br />no positioning <br />z-index: 8;
</div>
```

CSS

```
div {
  padding: 10px;
  opacity: 0.7;
  text-align: center;
}

strong {
  font-family: sans-serif;
}

#abs1 {
  z-index: 5;
  position: absolute;
```

```
  width: 150px;
  height: 350px;
  top: 10px;
  left: 10px;
  border: 1px dashed #990000;
  background-color: #ffdddd;
}

#rel1 {
  z-index: 3;
  height: 100px;
  position: relative;
  top: 30px;
  border: 1px dashed #669966;
  background-color: #ccffcc;
  margin: 0px 50px;
}

#rel2 {
  z-index: 2;
  height: 100px;
  position: relative;
  top: 15px;
  left: 20px;
  border: 1px dashed #669966;
  background-color: #ccffcc;
  margin: 0px 50px;
}

#abs2 {
  z-index: 1;
  position: absolute;
  width: 150px;
  height: 350px;
  top: 10px;
  right: 10px;
  border: 1px dashed #990000;
  background-color: #ffdddd;
}

#sta1 {
  z-index: 8;
  height: 70px;
  border: 1px dashed #999966;
  background-color: #ffffcc;
  margin: 0px 50px;
}
```

Result

See also

- Understanding z-index
- Stacking context
- Stacking floating elements
- Stacking without `z-index`
- CSS positioned layout module

CSS properties and values API

The **CSS properties and values API** module defines a method for registering new CSS properties, defining the property's data type, inheritance behavior, and, optionally, an initial value. This API expands on CSS custom properties for cascading variables module, which allows authors to define custom properties in CSS using two dash syntax (`--`). The CSS properties and values API is part of the CSS Houdini umbrella of APIs.

Custom properties let you reuse values across a project to simplify complex or repetitive stylesheets. Basic custom properties are defined in the CSS custom properties for cascading variables module. The CSS properties and values API expands on that module, enabling adding metadata to custom properties using CSS with the `@property` at-rule or, alternatively, using JavaScript's `CSS.registerProperty` method.

Whether registered with CSS or JavaScript, setting metadata on custom properties provides for an expected data type that the browser can use depending on the context, defines an initial value, and lets you control inheritance.

CSS properties and values API custom property registration is more robust than the more basic CSS cascading variable custom property declaration, especially when it comes to transitioning and animating values as browsers can interpolate between custom values of this type, whereas properties that use two dash syntax (`--`) behave more like a string substitution.

Properties and values API in action

To see how custom properties and values can be used via API, hover over the box below.

```
CSS.registerProperty({
  name: "--stop-color",
  syntax: "<color>",
  inherits: false,
  initialValue: "cornflowerblue",
});
```

```
.box {
  padding: 1rem;
  width: 90%;
  height: 4rem;
  font-family: sans-serif;
  font-size: large;
  color: white;
  border-radius: 0.5rem;
}

.box {
  background: linear-gradient(to right, var(--stop-color), lavenderblush);
  transition: --stop-color 2s;
}

.box:hover {
```

```
  --stop-color: aquamarine;
}
```

```
<div class="box"><p>Linear gradient with transition</p></div>
```

The box has a background consisting of a linear gradient from `--stop-color` (the custom property) to `lavenderblush`. The value of `--stop-color` is set to `cornflowerblue` at first, but when you hover over the box, `--stop-color` transitions to `aquamarine` over two seconds (`linear-gradient(to right, aquamarine, lavenderblush)`).

Reference

At-rules and descriptors

- `@property`
  - syntax descriptor
    - `+` and `#` multipliers
    - `|` combinator
  - inherits descriptor
  - initial-value descriptor

Interfaces and APIs

- `CSSPropertyRule`
- `CSS.registerProperty()`

Guides

Registering CSS custom properties

Covers how to register CSS custom properties with the `@property` at-rule and explains the benefits of doing so.

Using the CSS properties and values API

Explains how to register custom properties in CSS and JavaScript, with hints on handling undefined and invalid values, fallbacks, and inheritance.

CSS Houdini

Reference guide to Houdini resources including the CSS modules, API guides, and external resources.

Houdini APIs

Explains what CSS Houdini is and its advantages, along with a list of available APIs and their statuses.

Related concepts

- `var()`
- CSSRule
- CSSStyleValue
- CSS scoping
- Using shadow DOM
- CSS Typed Object Model API
- CSS Painting API
- Worklet

See also

- CSS cascading and inheritance
- CSS scoping module
- Using shadow DOM
- CSS Painting API module
- Worklet interface
- CSS `env()`
- CSS Typed Object Model

CSS Houdini

CSS Houdini is a set of APIs that expose parts of the CSS engine, enabling developers to create CSS extensions. These extensions might polyfill features that are not yet available in browsers, experiment with new layout techniques, or add creative borders and other effects.

While many Houdini examples showcase the creative possibilities of the APIs, there are many practical use cases. For example, you can use Houdini to create advanced custom properties with type checking and default values.

Basic example

A regular CSS custom property consists of a property name and a value. Therefore I might create a custom property called `--background-color` and expect it to be given a color value. The value is then used in the CSS as if it were the color value.

```
:root {
  --background-color: blue;
}

.box {
  background-color: var(--background-color);
}
```

In the above example however, there is nothing to stop someone using some other value for this property, perhaps setting it to a length. Having done so, anywhere that the property is used would have no background color as `background-color: 12px` is not valid. When browsers come across CSS they don't recognize as valid they throw that line away.

Using `@property` however, we can declare the custom property with a `syntax` of `<color>`. This shows that we need this property to have a value which is a valid color.

```
@property --background-color {
  syntax: "<color>";
  inherits: false;
  initial-value: blue;
}
```

Houdini worklets

A feature of Houdini is the `Worklet`. A worklet is a module, written in JavaScript, that extends CSS using one of the Houdini APIs. You can see an example worklet on the `PaintWorkletGlobalScope.registerPaint()` page. Once a worklet has been registered you can use it in CSS just like any other value. This means that even if you are not a JavaScript developer, you can access Houdini APIs by using worklets other people have created.

Reference

CSS at-rule and descriptors

The `@property` at-rule allows you to register an advanced custom property.

- `@property`
- `inherits`
- `initial-value`
- `syntax`

Houdini API references

- CSS Properties and Values API
- CSS Typed Object Model API
- CSS Painting API
- `Worklet` reference

Houdini guides

- Properties and Values API Guide
- Typed OM API Guide
- Using the CSS Painting API

See also

- A Practical Overview of CSS Houdini
- Smarter custom properties with Houdini's new API

Registering custom properties in CSS

The `@property` at-rule, part of the CSS Houdini set of APIs, allows developers to explicitly define CSS custom properties. In this guide, we learn how to use `@property` for type checking and constraining, set default custom property values in CSS, and define whether a custom property can inherit values or not.

Why register custom properties?

CSS custom properties, commonly called CSS variables, can be defined by basic declaration. Generally, these are declared on the `:root`, and can be overridden with a different value in subsequent rulesets — including in rules based on UI state and within container and media queries.

In this example, `--myColor` is assigned the value `#bada55` on `:root`, then set to a different value on `main`, which is inherited by all descendants of `main`. That value is overridden when `main` is hovered or is in a viewport that is less than `750px` wide. Within elements with a class of `circles`, `--myColor` is set to `45deg`, which is a different data type.

```
:root {
  --myColor: #bada55;
}
main {
  --myColor: #cacaca;
}
main:hover {
  --myColor: #aabbbbaa;
}
@media (width < 750px) {
  main {
    --myColor: #aabbbbaa;
  }
}

.circles {
  --myColor: 45deg;
}
```

The CSS properties and values API provides methods of registering custom properties. Registering CSS custom properties sets the variable's type, inheritance, and default value, making them more predictable and performant.

Registered properties are validated when they're computed rather than when they are parsed. This means that:

- Invalid values won't appear as invalid when inspecting the element's properties.
- An invalid property included after a valid one won't fall back to the valid property. An invalid property will, however, fall back to its registered default.

By default, all CSS variables inherit. When you change a non-registered variable value, the browser re-parses the DOM to check which elements in the DOM tree are affected. With registered properties, setting the value `inherits: false;` tells the browser that, if the value changes, the children don't need to be reparsed, narrowing the scope of style recalculation.

Generally, you'll want to register your custom properties if you are animating the variable, implementing type checking, or guaranteeing predictable inheritance behavior. If you're creating a CSS framework, namespace and register your custom properties to ensure that they always have a default value, and prevent framework users from assigning invalid data types to them if they change their values.

In CSS, properties are registered using the `@property` at-rule. In this example, `--myColor` is globally registered as an inheritable color with the value `#bada55`, and used in `main` in the hovered state. The registration improves the animation performance because the value is of the correct type, and the browser doesn't need to parse the value before animating it. The `.circles` assignment is ignored because the property's data type is registered as `<color>`, not `<angle>`.

```
@property --myColor {
  syntax: "<color>";
  inherits: true;
  initial-value: #bada55;
}

main:hover {
  color: var(--myColor);
  animation: color-change 2s linear forwards;
}

@keyframes color-change {
  to {
```

```
    color: red;
  }
}

.circles {
  --myColor: 45deg;
}
```

The @property and descriptors

The `@property` rule enables custom property registration directly inside stylesheets without requiring any JavaScript. Valid `@property` rules produce registered custom properties, producing the same effect as calling `registerProperty()` with equivalent parameters.

The custom property name is a `<dashed-ident>` that starts with `--` and is followed by a valid, user-defined identifier. It is case-sensitive. The declaration includes up to three descriptors: `syntax`, `inherits`, and `initial-value`.

In this example, we create a `--rotation` custom property.

```
@property --rotation {
  syntax: "<angle>";
  inherits: false;
  initial-value: 45deg;
}
```

@property descriptors

For the `@property` rule to be valid, it must include both the `syntax` and `inherits` descriptors. The `initial-value` descriptor is optional.

`syntax`

> The `syntax` descriptor is a string that describes the allowed value types for the registered custom property. The value of the descriptor may be a data type name, such as `<color>`, `<length>`, or `<number>`, with multipliers (`+`, `#`) and combinators (`|`), or a custom ident. In our example, the `--rotation` property is set to `<angle>`, meaning it can only be assigned an `<angle>` value.

`inherits`

> The `inherits` descriptor is a boolean value that controls whether the custom property inherits by default: set it to `true` to have it inherit, `false` to not inherit. See Registering and using custom properties for an example of both values.

`initial-value`

> The `@property` at-rule also accepts the `initial-value` descriptor. The value defines the starting value for the property. This descriptor is optional only if the value of the `syntax` descriptor is the universal syntax definition (that is, `syntax: "*"`). If the syntax has any other values, it is required and has to be a *computationally independent* value, meaning the value can't be of a data-type that is dependent on another value, such as values with `%` or `em` units.

If either the `syntax` or `inherits` descriptor is missing, the entire `@property` rule is invalid and ignored. If the `initial-value` descriptor is required but omitted, the entire `@property` rule is invalid and ignored. Unknown descriptors are invalid and ignored, but do not invalidate the `@property` rule.

Registering and using custom properties

In this example, we will define two custom properties: `--item-size` and `--item-color`. We'll then use these custom properties to define the size (width and height) and background color of the three following items.

```
<div class="container">
  <div class="item one">Item one</div>
  <div class="item two">Item two</div>
  <div class="item three">Item three</div>
</div>
```

Here we use the CSS `@property` at-rule to define the `--item-size` custom property. We set its initial value to `40%` and limit valid values to `<percentage>` values only. This means that when this property is used to size an element, the element's size will always be relative to its parent's size. The property is inheritable.

```
@property --item-size {
  syntax: "<percentage>";
  inherits: true;
  initial-value: 40%;
}
```

You can also use JavaScript to register properties. JavaScript's `CSS.registerProperty()` method is equivalent to the `@property` at-rule. Here we defined our second custom property, `--item-color`, to have an initial value of `aqua`, to accept only `<color>` values, and to not be inherited.

```
window.CSS.registerProperty({
  name: "--item-color",
  syntax: "<color>",
  inherits: false,
  initialValue: "aqua",
});
```

See the JavaScript API guide to registering custom properties for more information.

Using registered custom properties

We can assign new values to registered properties. Here, we set custom property values on the parent container, then set custom property values on the items themselves. The values set on `two` are valid while the values set on `three` are invalid.

```
.container {
  --item-size: 20%;
  --item-color: orange;
}
.two {
  --item-size: initial;
  --item-color: inherit;
}
.three {
  --item-size: 1000px;
  --item-color: xyz;
}
```

We use the two custom properties as values for the items' `width`, `height`, and `background-color` properties:

```
.item {
  width: var(--item-size);
  height: var(--item-size);
  background-color: var(--item-color);
}
```

```
.container {
  display: flex;
  height: 200px;
  border: 1px dashed black;
}
```

Inheritance

The `inherits` descriptor controls whether or not the registered CSS custom property inherits by default.

In our example, the size property was set to be inheritable; the color was not. The two custom properties, `--item-size: 20%` and `--item-color: orange;` were set on the `container` parent, overriding the `40%` and `aqua` default values set when these custom properties were defined.

For item one, none of these custom properties were set. Because the `--item-size` is inheritable, the value `20%` set on its parent `container` was inherited and used. The property `--item-color` was not inheritable, so the value `orange` set on the parent was not considered. Instead, the default initial value `aqua` was used.

The `inherits` descriptor is a required descriptor; if it were missing or invalid, the entire `@property` rule would have been invalid and ignored.

Valid and invalid values

A benefit of registering your custom properties (versus using simple declarations) is that they can be set to be reassigned only to valid values. The `syntax` descriptor defines which values are valid. Any declaration with a value that doesn't meet those constraints will be ignored.

For item two, CSS global keywords were set for both custom properties. The CSS global keyword values — `initial`, `inherit`, `unset`, `revert`, and `revert-layer` — are valid values for all value types and therefore valid no matter the `syntax` descriptor value. The `--item-size` was set to `initial` and therefore used the `initial-value: 40%;` set in the `@property` declaration. The `--item-color` was set to `inherit`, explicitly inheriting the `orange` value from its parent even though the custom property was set to otherwise not be inherited. This is why item two was orange.

For item three, the `--item-size` value was set to `1000px`. While `1000px` is a `<length>` value, the `@property` declaration requires the value be a `<percentage>`, so the declaration was not valid and was ignored, meaning the inheritable `20%` set on the parent was used. The `xyz` value was also invalid. As `registerProperty()` set `--item-color` to not be inherited, the default initial value of `aqua` was used and not the parent's `orange` value.

Animating custom property values

Registered CSS custom properties offer a performance edge because they are treated as typed data rather than strings, providing the browser's rendering engine with a defined syntax and inheritance rules. The browser parses the defined syntax once; when that variable is used elsewhere, the type and internal representation are already known, so it doesn't need to be re-parsed.

Because non-registered custom properties are just strings, the browser cannot interpolate them, so it cannot smoothly animate them. Instead, non-registered custom property values are animated discretely. Registered variables can be interpolated and therefore animated; their interpolation calculations are offloaded to the GPU, which is more performant than animations done on the main thread.

See Animating a custom property value for an animation example.

See also

- `var()`
- Houdini APIs
- Using CSS custom properties (variables) guide
- CSS Properties and Values API module
- CSS Properties and Values API
- CSS custom properties for cascading variables module

CSS pseudo-elements

The **CSS pseudo-element** module defines abstract elements that are not directly present in the document tree. These abstract elements, called pseudo-elements, represent portions of the render tree that can be selected and styled. Pseudo-elements are used to create abstractions about the document tree beyond those provided by the document tree.

> **Note:** This page introduces a CSS module. To find an exhaustive list of all pseudo-elements defined by CSS specifications, see the pseudo-elements reference page.

Pseudo-elements are prefixed with a double colon (`::`). You add pseudo-elements to selectors (as in `p::first-line`) to target and style these faux elements.

Pseudo-elements enable targeting entities not included in HTML and sections of content that cannot be targeted otherwise without adding extra markup. Consider the placeholder of an `<input>` element. This is an abstract element and not a distinct node in the document tree. You can select this placeholder by using the `::placeholder` pseudo-element. As another example, the `::selection` pseudo-element matches the content currently highlighted by a user, allowing you to style what is matched as the user interacts with the content and changes the selection. Similarly, the `::first-line` pseudo-element targets the first line of an element, updating automatically if the character count of the first line changes, without having to query the element's line length.

Reference

Selectors

- `::after`
- `::before`
- `::file-selector-button`
- `::first-letter`
- `::first-line`
- `::grammar-error`
- `::highlight()`
- `::marker`
- `::placeholder`
- `::search-text`
- `::selection`
- `::spelling-error`
- `::target-text`

The specification also defines the `::details-content` pseudo-element and the `::postfix` and `::prefix` sub-pseudo elements. These are not yet supported by any browser. The `::highlight()` pseudo-element is included within this module, but most details are provided in the CSS custom highlight API.

Interfaces

- `CSSPseudoElement` interface
  - `CSSPseudoElement.element` property
  - `CSSPseudoElement.parent` property
  - `CSSPseudoElement.type` property
  - `pseudo()` method

Terms

- Pseudo-element glossary term

Guides

CSS pseudo-elements

Alphabetical list of pseudo-elements defined by all the CSS specifications and WebVTT.

Learn: Pseudo-classes and pseudo-elements

Part of CSS building blocks section on selectors. This article defines what a pseudo-element is and how it can be combined with pseudo-classes and be used for generating content with `::before` and `::after` pseudo-elements.

How to create fancy boxes using pseudo-elements

Example of styling generated content using `::before` and `::after` pseudo-elements for visual effects.

Related concepts

- `::backdrop`
- Web Video Text Tracks Format (WebVTT) cues:
  - `::cue`
  - `::cue()`
- CSS multi-column layout module
  - `::column`
- CSS overflow module
  - `::scroll-button()`
  - `::scroll-marker`
  - `::scroll-marker-group`
  - `:target-current`
- CSS scoping module
  - `:host`
  - `:host()`
  - `:host-context()`
  - `::slotted()`
- CSS shadow parts module
  - `::part`
- CSS view transitions module
  - `::view-transition`
  - `::view-transition-image-pair()`
  - `::view-transition-group()`
  - `::view-transition-new()`
  - `::view-transition-old()`
- CSS selectors
  - Attribute selectors
  - Combinators
  - Class selectors
  - ID selectors
  - Type selectors
  - Pseudo-classes
  - Universal selectors
- `placeholder` attribute of the `<input>` element
- `:placeholder-shown` selector
- CSS generated content
  - `content` property
  - `quotes` property
- Text fragments
- `AnimationEvent.pseudoElement` property
- `KeyframeEffect.pseudoElement` property
- `TransitionEvent.pseudoElement` property

See also

- List of pseudo-elements
- Specificity
- CSS selectors module
- CSS shadow-parts module

- CSS generated content module
- CSS positioned layout module
- CSS custom highlight API

CSS round display

The **CSS round display** module defines CSS extensions to support a round display, such as wrist-watches, to help developers build web pages suitable for those devices.

Reference

Properties

The CSS round display module introduces the `border-boundary` and `shape-inside` properties. Currently, no browsers support these features.

At-rules and descriptors

- `shape` (`@media` feature)

Guides

Using media queries

Introduces media queries, their syntax, and the operators and media features used to construct media query expressions.

Overview of CSS Shapes

Defines basic shapes and how to debug them with developer tools.

Using environment variables

Overview of what environment variables are, browser defined environment variables, and how to use the `env()` function.

Related concepts

- `@media` at-rule
- `offset-distance` property
- `offset-path` property
- `offset-position` property
- `viewport-fit` value for `<meta>` `content` attribute

See also

- `<basic-shape>` data-type
- `shape-outside` property
- CSS shapes module
- CSS media queries module
- CSS environment variables module

CSS ruby layout

The **CSS ruby layout** module provides the rendering model and formatting controls related to the display of ruby annotations. Ruby annotations are a form of interlinear annotation, consisting of short runs of text alongside the base text. They are typically used in East Asian documents to indicate pronunciation or define meaning.

Reference

Properties

- `ruby-align`
- `ruby-position`

The CSS ruby layout module also introduces the `ruby-merge` and `ruby-overhang` properties. Currently, no browsers support these features.

Display values

The CSS ruby layout module adds the following values to the `display` property:

- `ruby`
- `ruby-base`
- `ruby-text`
- `ruby-base-container`
- `ruby-text-container`

Glossary terms

- Ruby

Related concepts

- CSS display module
  - `display`
  - `<display-internal>`
- CSS text decoration module
  - `text-emphasis-color`
  - `text-emphasis-position`
  - `text-emphasis-style`
  - `text-emphasis` shorthand
- HTML elements
  - `<rb>`
  - `<rp>`
  - `<rt>`
  - `<rtc>`
  - `<ruby>`

See also

- `direction`

- `unicode-bidi`
- `font-variant-east-asian`
- `:lang()`
- HTML `lang` attribute

CSS scoping

The **CSS scoping** module defines the CSS scoping and encapsulation mechanisms, focusing on the Shadow DOM scoping mechanism.

CSS styles are either global in scope or scoped to a shadow tree. Globally scoped styles apply to all the elements in the node tree that match the selector, including custom elements in that tree, but not to the shadow trees composing each custom element. Selectors and their associated style definitions don't bleed between scopes.

Within the CSS of a shadow tree, selectors don't select elements outside the tree, either in the global scope or in other shadow trees. Each custom element has its own shadow tree, which contains all the components that make up the custom element (but not the custom element, or "host", itself).

Sometimes it's useful to be able to style a host from inside the shadow tree context. The CSS scoping module makes this possible by defining selectors that:

- Enable a shadow tree to style its host.
- Enable external CSS to style elements within a shadow DOM (but only if the associated custom element is set up to accept external styles).

Reference

Selectors

- `:host`
- `:host()`
- `:host-context()`
- `::slotted`

Guides

Web components

An introduction to the different technologies used to create reusable web components — custom elements whose functionality is encapsulated away from the rest of your code.

Using shadow DOM

Shadow DOM fundamentals, including attaching a shadow DOM to an element, adding to the shadow DOM tree, and styling.

Using templates and slots

Defining reusable HTML structure using `<template>` and `<slot>` elements, and using that structure inside web components.

Using custom elements

Introduction to the Custom Elements API, the JavaScript API used to create custom elements that encapsulate functionality.

Related concepts

- CSS `:defined` pseudo-class
- CSS `::part` pseudo-element
- HTML `<template>` element
- HTML `<slot>` element
- HTML `slot` attribute
- Shadow tree glossary term
- DOM glossary term
- Compound selector term
- Selector list term
- Web components interfaces, properties, and methods
  - `CustomElementRegistry` interface
  - `Element` API
    - `Element.slot` property
    - `Element.assignedSlot` property
    - `Element.attachShadow()` method
  - `HTMLSlotElement` interface
  - `HTMLTemplateElement` interface
  - `ShadowRoot` interface

> **Note:** Despite the name, the `:scope` pseudo-class, which represents elements that are a reference point (or scope) for selectors to match against, is defined in the Selectors module. It is otherwise unrelated to the CSS scoping module, which is focused on scoping as it pertains to the Shadow DOM scoping mechanism.

See also

- CSS selectors module
- CSS pseudo elements module
- CSS namespaces module
- CSS shadow-parts module
- Template, slot, and shadow on web.dev (2023)
- Custom element best practices on web.dev (2019)

CSS scroll anchoring

The **CSS scroll anchoring** module defines a mechanism to prevent page movement due to DOM changes above the visible region of a scrolling box while the user is consuming the visible content.

Scroll anchoring attempts to keep the user's view of the document stable across layout changes. It works by selecting a DOM node (the anchor node) whose movement is used to determine adjustments to the scroll position. The anchor node is always a descendant of the scrolling box.

For scroll containers that are snapped to an element, scroll anchoring is limited to adjustments that would be allowed by re-snapping.

Reference

Properties

- `overflow-anchor`

Glossary terms

- Scroll container
- Scroll snap

Guides

Overview of scroll anchoring

What is scroll anchoring, suppression triggers, and when and how to enable and disable this browser feature.

Related concepts

- `overscroll-behavior`

See also

- CSS overflow module
- CSS scroll snap module
- CSS overscroll behavior module

Overview of scroll anchoring

As a user of the web, you are probably familiar with the problem that scroll anchoring solves. You browse to a long page on a slow connection and begin to scroll to read the content; while you are busy reading, the part of the page you are looking at suddenly jumps. This has happened because large images or some other elements have just loaded further up in the content.

Scroll anchoring is a browser feature that aims to solve this problem of content jumping, which happens if content loads in after the user has already scrolled to a new part of the document.

How does it work?

Scroll anchoring adjusts the scroll position to compensate for the changes outside of the viewport. This means that the point in the document the user is looking at remains in the viewport, which may mean their scroll position actually changes in terms of how *far* they have moved through the document.

How do I turn on scroll anchoring?

You don't! The feature is enabled by default in supporting browsers. In most cases anchored scrolling is exactly what you want — content jumping is a poor experience for anyone.

What if I need to debug it?

If your page is not behaving well with scroll anchoring enabled, it is probably because some `scroll` event listener is not handling the extra scrolling to compensate for the anchor node movement.

You can check whether disabling scroll anchoring fixes the issue in Firefox by changing `layout.css.scroll-anchoring.enabled` to `false` in `about:config`. You can also check what node Firefox is using as the anchor using the `layout.css.scroll-anchoring.highlight` switch. That will show a purple overlay on top of the anchor node.

If a node doesn't seem to be an appropriate anchor, you can exclude it using `overflow-anchor`, as described below.

What if I need to disable it?

The CSS scroll anchoring module provides the `overflow-anchor` property, which can be used to disable scroll anchoring on all or part of the document. It's essentially a way to opt out of the behavior.

The only possible values are `auto` or `none`:

- `auto` is the initial value; as long as the user's browser supports scroll anchoring, the behavior will happen, and they should see fewer content jumps.
- `none` means that you have explicitly opted the document, or part of the document, out of scroll anchoring.

To opt out the entire document, you can set it on the `<body>` element:

```
body {
  overflow-anchor: none;
}
```

To opt out of scroll anchoring for a section of the document, set `overflow-anchor: none` on the section's container element:

```
.container {
  overflow-anchor: none;
}
```

If opting out of scroll anchoring on the document or a section thereof, a descendant of an opted-out area cannot be opted back in. For example, if you opt out the entire document, you can't set `overflow-anchor: auto` on a descendant node to turn scroll anchoring back on for a subsection.

Suppression triggers

There are some *suppression triggers*, which disable scroll anchoring in places where it might be problematic. If any of the triggers happen on the anchor node, or an ancestor of it, anchoring is suppressed.

These suppression triggers are changes to the computed value of any of the following properties:

- `top`, `left`, `right`, or `bottom`
- `margin` or `padding`
- Any `width` or `height`-related properties
- `transform` and the individual transform properties `translate`, `scale`, and `rotate`

Additionally, `position` changes anywhere inside the scroll container also disable scroll anchoring.

To conditionally apply styles based on whether scroll anchoring can be disabled, use `@supports` feature queries to test support for the `overflow-anchor` property.

See also

- Original scroll anchoring explainer via WICG (2016)
- Scroll anchoring for web developers via Chromium (2017)

CSS scroll snap

The **CSS scroll snap** module provides properties that let you control the panning and scrolling behavior by defining snap positions. Content can be snapped into position as the user scrolls overflowing content within a scroll container, providing paging and scroll positioning.

This module includes the scroll container scroll-padding properties to adjust the optimal viewing region of paging during scroll-into-view operations. It also includes scroll-margin and scroll-alignment, set on the scroll container's children, to adjust the children's visual area when that child is scrolled into view, as well as a property to force scrolling to stop on individual children.

Scroll snap in action

To view scroll snapping in the box below, scroll up-and-down and left-and-right through the grid of 45 numbered boxes in the scrollable viewport. Click "Play" in the example below to view or edit the source in the MDN Playground:

```
const positions = ["start", "center", "end"];
const inlineDirection = document.getElementById("inline");
const blockDirection = document.getElementById("block");
const stop = document.getElementById("stop");
const snap = document.getElementById("snap");
const all = document.querySelector("article");
const rules = document.getElementById("css-output").sheet.cssRules;

setSST();
setSSA();

inlineDirection.addEventListener("change", () => {
  setSSA();
});
blockDirection.addEventListener("change", () => {
  setSSA();
});
stop.addEventListener("change", () => {
  setSST();
});
snap.addEventListener("change", () => {
  all.classList.toggle("snapDisabled");
});

function setSSA() {
  rules[0].style.scrollSnapAlign = `${positions[blockDirection.value]} ${
    positions[inlineDirection.value]
  }`;
}

function setSST() {
  if (stop.checked) {
    rules[0].style.scrollSnapStop = "always";
  } else {
    rules[0].style.scrollSnapStop = "normal";
  }
}
```

```
<article>
  <ul>
    <li></li>
    <li></li>
    <li>
```

```
    <fieldset>
      <legend>Change the options</legend>
      <p>
        <label
          ><input
            type="range"
            min="0"
            max="2"
            value="1"
            list="places"
            id="block" />
          block position</label
        >
      </p>
      <p>
        <label>
          <input
            type="range"
            min="0"
            max="2"
            value="1"
            list="places"
            id="inline" />
          inline position
        </label>
      </p>
      <p>
        <label>
          <input type="checkbox" id="stop" />
          Prevent scrolling past boxes
        </label>
      </p>
    </fieldset>

    <p>
      <label><input type="checkbox" id="snap" /> disable snapping</label>
    </p>

    <datalist id="places">
      <option value="0">start</option>
      <option value="1">center</option>
      <option value="2">end</option>
    </datalist>
  </div>
</article>
```

```
li {
  /*
  starts with:
      scroll-snap-align: center center;
      scroll-snap-stop: normal (defaults);

  CSS gets changed with JavaScript when you change the controls.
  the following can be set:
      scroll-snap-stop: always | normal;
      scroll-snap-align: start | center | end {2}
        */
}
ul {
  overflow: auto;
  scroll-snap-type: both mandatory;
  overscroll-behavior-x: contain;
}
article.snapDisabled fieldset {
  opacity: 20%;
  pointer-events: none;
}
article.snapDisabled ul {
  scroll-snap-type: initial;
  overscroll-behavior-x: initial;
}

@layer page-setup {
  article {
    display: flex;
    gap: 2vw;
  }
  div {
    flex: 1;
  }
  ul {
    display: grid;
    gap: 6.25vw;
    padding: 12.5vw;
    box-sizing: border-box;
    border: 1px solid;
    grid-template-columns: repeat(5, 1fr);
    background: conic-gradient(
      at bottom left,
      red 0deg,
      yellow 15deg,
      green 30deg,
      blue 45deg,
      purple 60deg,
      magenta 75deg
    );
    background-attachment: local;
    margin: auto;
    width: 20vw;
    height: 20vw;
  }
  li {
    scroll-snap-align: center;
```

```
    height: 12.5vw;
    width: 12.5vw;
    outline: 3px inset;
    list-style-type: none;
    background: white;
    font-family: monospace;
    font-size: 3rem;
    line-height: 12vw;
    text-align: center;
    counter-increment: items 1;
  }
  li::after {
    content: counter(items);
  }
  input {
    vertical-align: bottom;
  }
  p {
    font-family: monospace;
  }
}
```

With scroll snap, one of the numbered boxes that you scroll to will snap into place. The initial CSS makes the numbered box snap into the center of the viewport. Use the sliders to change the block and inline snap positions.

Using snap properties, you can allow or block the scrolling past an element, a numbered box in this case. Select the "Prevent scrolling past boxes" checkbox to force all scrolling actions to be limited to scrolling to an adjacent box.

To compare scroll snapping to regular scrolling, check the "disable snapping" checkbox and try scrolling again.

Reference

Properties on containers

- `scroll-snap-type`
- `scroll-padding`
  - `scroll-padding-top`
  - `scroll-padding-right`
  - `scroll-padding-bottom`
  - `scroll-padding-left`
  - `scroll-padding-inline`
  - `scroll-padding-inline-start`
  - `scroll-padding-inline-end`
  - `scroll-padding-block`
  - `scroll-padding-block-start`
  - `scroll-padding-block-end`

Properties on children

- `scroll-initial-target`
- `scroll-margin`
  - `scroll-margin-top`
  - `scroll-margin-right`
  - `scroll-margin-bottom`
  - `scroll-margin-left`
  - `scroll-margin-inline`
  - `scroll-margin-inline-start`
  - `scroll-margin-inline-end`
  - `scroll-margin-block`
  - `scroll-margin-block-start`
  - `scroll-margin-block-end`
- `scroll-snap-align`
- `scroll-snap-stop`

Events

- `scrollsnapchange`
- `scrollsnapchanging`

Interfaces

- `SnapEvent`
  - `SnapEvent.snapTargetBlock`
  - `SnapEvent.snapTargetInline`

Guides

Basic concepts of CSS scroll snap

An overview and examples of CSS scroll snap features.

Using scroll snap events

A guide to using the `scrollsnapchanging` and `scrollsnapchange` scroll snap events that are fired when the browser determines a new snap target is pending or selected.

Related concepts

- `:target` pseudo-class
- `overflow` CSS property
- Element `scroll()` method
- Element `scrollBy()` method
- Element `scrollIntoView()` method
- Element `scrollTo()` method
- Element `scroll` event
- Element `scrollend` event

- `scrollbar` ARIA role
- Scroll container glossary term

See also

- CSS overflow module
- CSS scrollbars styling module
- CSS scroll anchoring module
- Keyboard-only scrolling areas on adrianroselli.com (2022)
- Scroll snap examples on CodePen (2022)
- Practical CSS scroll snapping/ on CSS-Tricks (2020)
- CSS scroll snap on 12 Days of Web (2019)
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

Basic concepts of scroll snap

The properties in the CSS scroll snap module enable you to define how scrolling snaps to specific points as a user scrolls through a document.

The scroll snap feature lets you define the snap positions where a scroll container's scrollport may end or "snap to" after a scrolling operation has completed.

Key properties for CSS scroll snap

Before you can define scroll snapping, you need to enable scrolling on a scroll container. You can do this by ensuring that the scroll container has a defined size and that it has `overflow` enabled.

You can then define scroll snapping on the scroll container by using the following two key properties:

- `scroll-snap-type`: Using this property, you can define whether or not the scrollable viewport can be snapped to, whether snapping is required or optional, and the axis on which the snapping should occur.
- `scroll-snap-align`: This property is set on every child of the scroll container and you can use it to define each child's snap position or lack thereof.
- `scroll-snap-stop`: This property ensures that a child is snapped to during scrolling and not passed over.
- `scroll-margin`: This property can be set on child elements that are snapped to during scrolling to create an outset from the defined box.
- `scroll-padding`: This property can be set on the scroll container to create a snapping offset.

The example below demonstrates scroll snapping along the vertical axis, which is defined by `scroll-snap-type`. Additionally, `scroll-snap-align` applies on all the children of the `<section>` element, dictating the point where the scrolling of each child should stop.

```
<article class="scroller">
  <section>
    <h2>Section one</h2>
  </section>
  <section>
    <h2>Section two</h2>
  </section>
  <section>
    <h2>Section three</h2>
  </section>
</article>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.scroller {
  border: 4px solid #333333;
  width: 300px;
}

.scroller section {
  min-height: 100%;
  padding: 10px;
}

.scroller section:nth-child(odd) {
  background-color: #cccccc;
}
```

```
.scroller {
  height: 300px;
  overflow-y: scroll;
  scroll-snap-type: y mandatory;
}

.scroller section {
  scroll-snap-align: start;
}
```

Using scroll-snap-type

The `scroll-snap-type` property needs to know the axis along which scroll snapping happens. This can be `x`, `y`, or the logical mappings `block` or `inline`. You can also use the keyword `both` to have scroll snapping work along both axes.

You can also pass in the keywords `mandatory` or `proximity`. The `mandatory` keyword tells the browser whether the content *has* to snap to a certain point, no matter where the scroll is. The `proximity` keyword means that the content may snap to the point, but does not have to.

Using `mandatory` creates a very consistent scrolling experience — you know the browser will always snap to each defined point. This means that you can be confident that something you expect to be at the top of the screen will be there when scrolling finishes. However, it can cause problems if the content is larger than you expect — users may find themselves in the frustrating position of never being able to scroll and view a certain point in the content. Therefore, the use of `mandatory` should be carefully considered and only used in situations where you know how much content is on the screen or scrollable section at any time.

> **Note:** Never use `mandatory` if the content inside one of your child elements will overflow the parent container because user will not be able to scroll the overflowing content into view.

The `proximity` value only snaps child elements to a position when it is close by, with the browsers determining the exact distance. Click "Play" to edit the example below in the MDN Playground. Alternate the `scroll-snap-type` value between `mandatory` and `proximity` to see the effect this has on the scroll experience.

```
<article class="scroller">
  <section>
```

```
    <h2>Section one</h2>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </section>
  <section>
    <h2>Section two</h2>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </section>
  <section>
    <h2>Section three</h2>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </section>
</article>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.scroller {
  border: 4px solid #333333;
  width: 300px;
}

.scroller section {
  min-height: 100%;
  padding: 10px;
}

.scroller section:nth-child(odd) {
  background-color: #cccccc;
}
```

```
.scroller {
  height: 300px;
  overflow-y: scroll;
  scroll-snap-type: y mandatory;
}

.scroller section {
  scroll-snap-align: start;
}
```

In the above example, both `height: 300px;` and `overflow-y: scroll;` are set on the scroll container. If the content doesn't overflow its container, there is nothing to scroll.

Using scroll-snap-align

The valid values for the `scroll-snap-align` property include `start`, `end`, `center`, and `none`. These values are used to indicate the point in the scroll container to which the content should snap. Click "Play" in the example below and change the value of `scroll-snap-align` to see how this changes the scroll behavior.

```
<article class="scroller">
  <section>
    <h2>Section one</h2>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </section>
  <section>
    <h2>Section two</h2>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </section>
  <section>
    <h2>Section three</h2>
    <p>
      Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
      kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus
      winter purslane kale. Celery potato scallion desert raisin horseradish
      spinach carrot soko.
    </p>
  </section>
</article>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.scroller {
  border: 4px solid #333333;
  width: 300px;
}
```

```
.scroller section {
  min-height: 100%;
  padding: 10px;
}

.scroller section:nth-child(odd) {
  background-color: #cccccc;
}
```

```
.scroller {
  height: 200px;
  overflow-y: scroll;
  scroll-snap-type: y mandatory;
}

.scroller section {
  scroll-snap-align: start;
}
```

If `scroll-snap-type` is `mandatory` and `scroll-snap-align` on a child is either set to `none` or not set (in which case, it defaults to `none`), the user will be unable to scroll that element into view.

Using scroll-padding

When using `start` or `end`, if you do not want the content to snap right to the edge of the scroll container, or if you want the snap position to be slightly offset from center when using `center`, use the `scroll-padding` property or its equivalent longhand values to add some padding.

In the example below, `scroll-padding` is set to `50px`. When the content snaps to the start of the second and third sections, scrolling stops 50 pixels away from the start of the section. Try changing the `scroll-padding` value to see how this changes the distance.

```
<article class="scroller">
  <section>
    <h2>Section one</h2>
  </section>
  <section>
    <h2>Section two</h2>
  </section>
  <section>
    <h2>Section three</h2>
  </section>
</article>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.scroller {
  border: 4px solid #333333;
  width: 300px;
}

.scroller section {
  min-height: 100%;
  padding: 10px;
}

.scroller section:nth-child(odd) {
  background-color: #cccccc;
}
```

```
.scroller {
  height: 300px;
  overflow-y: scroll;
  scroll-snap-type: y mandatory;
  scroll-padding: 50px;
}

.scroller section {
  scroll-snap-align: start;
}
```

This is potentially useful if you have a fixed element such as a navigation bar, which could end up overlapping scrolled content. By using `scroll-padding`, you can reserve space for the fixed element, as shown in the example below, where the `<h1>` element remains on screen as the content scrolls beneath it. Without padding, the heading would overlap some of the content when snapping happens.

```
<article class="scroller">
  <h1>Sticky Heading</h1>
  <section>
    <h2>Section one</h2>
  </section>
  <section>
    <h2>Section two</h2>
  </section>
  <section>
    <h2>Section three</h2>
  </section>
</article>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.scroller {
  border: 4px solid #333333;
  width: 300px;
}

.scroller section {
  min-height: 100%;
  padding: 10px;
```

```
}

.scroller section:nth-child(odd) {
  background-color: #cccccc;
}
```

```
.scroller h1 {
  position: sticky;
  top: 0;
  min-height: 40px;
  background-color: black;
  color: white;
  margin: 0;
  padding: 0;
}

.scroller h2 {
  margin: 0;
  padding: 0;
}

.scroller {
  height: 300px;
  overflow-y: scroll;
  scroll-snap-type: y mandatory;
  scroll-padding: 50px;
}

.scroller section {
  scroll-snap-align: start;
}
```

Using scroll-margin

The `scroll-margin` property or the longhand scroll margin values can be set on child elements, defining an outset from the defined box. This allows for different amounts of space for different child elements and can be used in conjunction with `scroll-padding` on the parent.

```
<article class="scroller">
  <section>
    <h2>Section one</h2>
  </section>
  <section>
    <h2>Section two</h2>
  </section>
  <section>
    <h2>Section three</h2>
  </section>
</article>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.scroller {
  border: 4px solid #333333;
  width: 300px;
}

.scroller section {
  min-height: 100%;
  padding: 10px;
}

.scroller section:nth-child(odd) {
  background-color: #cccccc;
}
```

```
.scroller {
  height: 300px;
  overflow-y: scroll;
  scroll-snap-type: y mandatory;
}

.scroller section {
  scroll-snap-align: start;
  scroll-margin: 40px;
}
```

Using scroll-snap-stop

Using the `scroll-snap-stop` property, you can specify whether the scrolling must snap to the defined snap points. In the above examples, this would mean that the scrolling would stop at the start of each section or be able to skip past sections.

With this property definition, you can ensure that users see each section of the scroller and don't accidentally scroll past them. However, this setting can also negatively affect user experience by preventing the user from quickly scrolling to their desired content.

See also

- CSS scroll snap module
- Practical CSS scroll snapping on CSS-Tricks (2020)
- CSS scroll snap on 12 Days of Web (2019)
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)
- Scroll snap examples on CodePen

Using scroll snap events

The CSS scroll snap module defines two **scroll snap events**: `scrollsnapchanging` and `scrollsnapchange`. These enable running JavaScript in response to the browser determining that new scroll snap targets are pending and selected, respectively.

This guide provides an overview of these events, along with complete examples.

Events overview

Scroll snap events are set on a scrolling container that contains potential scroll snap targets:

- The `scrollsnapchanging` event is fired when the browser determines that a new scroll snap target will be selected when the current scroll gesture ends. This is the *pending* scroll snap target. Specifically, this event fires during a scrolling gesture, each time the user moves over potential new snap targets. While the `scrollsnapchanging` event may fire multiple times for each scrolling gesture, it does not fire on all potential snap targets for a scrolling gesture that moves over multiple snap targets. Rather, it fires just for the last target that the snapping will potentially rest on.
- The `scrollsnapchange` event is fired at the end of a scrolling operation when a new scroll snap target is selected. Specifically, this event fires when a scrolling gesture is completed, but only if a new snap target is selected. This event fires just before the `scrollend` event fires.

Let's look at an example that shows the two events in action (you'll see how this is built later on in the article):

Have a go at scrolling up and down the list of boxes:

- Try slowly scrolling the container up and down without releasing the scrolling gesture. For example, drag your finger(s) over the scrolling area on a touchscreen device or trackpad, or hold down the mouse button on the scroll bar and move the mouse. The boxes you move over should turn a darker gray color as you move over them, and then return to normal as you move away from them again. This is the `scrollsnapchanging` event in action.
- Now try releasing the scrolling gesture; the nearest box to your scrolling position should animate to a purple color, with white text. The animation occurs when the `scrollsnapchange` event fires.
- Finally, try scrolling fast. For example, flick your finger hard on the screen, to scroll past several potential targets before starting to come to rest near a target further down the scroll container. You should only see one `scrollsnapchanging` event fire as the scrolling starts to slow, before the `scrollsnapchange` event fires and the selected snap target turns purple.

The <code>SnapEvent</code> event object

Both of the above events share the `SnapEvent` event object. This has two properties that are key to how scroll snap events work:

- `snapTargetBlock` returns a reference to the element snapped to in the block direction when the event fired, or `null` if scroll snapping only occurs in the inline direction so no element is snapped to in the block direction.
- `snapTargetInline` returns a reference to the element snapped to in the inline direction when the event fired, or `null` if scroll snapping only occurs in the block direction so no element is snapped to in the inline direction.

These properties enable event handler functions to report the element that has been snapped to (in the case of `scrollsnapchange`) or the element that *would be snapped to* if the scrolling gesture were to be finished now (in the case of `scrollsnapchanging`) — in one- and two-dimensions. You can then manipulate these elements in any way you want, for example by directly setting styles on them via their `style` properties, setting classes on them that have styles defined for them in a stylesheet, etc.

Relationship with CSS <code>scroll-snap-type</code>

The property values available on `SnapEvent` correspond directly to the value of the `scroll-snap-type` CSS property set on the scroll container:

- If the snap axis is specified as `block` (or a physical axis value that equates to `block` in the current writing mode), only `snapTargetBlock` returns an element reference.
- If the snap axis is specified as `inline` (or a physical axis value that equates to `inline` in the current writing mode), only `snapTargetInline` returns an element reference.
- If the snap axis is specified as `both`, `snapTargetBlock` and `snapTargetInline` return an element reference.

Handling one-dimensional scrollers

If you are dealing with a horizontal scroller, only the event object's `snapTargetInline` property will change as the snapped element changes if the content has a horizontal `writing-mode`, or the `snapTargetBlock` property if the content has a vertical `writing-mode`.

Conversely, if you are dealing with a vertical scroller, only the `snapTargetBlock` property will change as the snapped element changes if the content has a horizontal `writing-mode`, or the `snapTargetInline` property if the content has a vertical `writing-mode`.

In both cases, the non-changing property of the two returns `null`.

Let's look at an example snippet to show a typical one-dimensional scroll snap event handler function:

```
scrollingElem.addEventListener("scrollsnapchange", (event) => {
  event.snapTargetBlock.className = "select-section";
});
```

In this snippet, a `scrollsnapchange` handler function is set on a block-direction scrolling container element that snap targets appear inside. When the event fires, we set a `select-section` class on the `snapTargetBlock` element, which could be used to style a newly-selected snap target to look like it has been selected (for example, with an animation).

Handling two-dimensional scrollers

If you are dealing with a horizontal *and* vertical scroller, the code gets more complex. This is because the `snapTargetBlock` property *and* the `snapTargetInline` property values both return an element reference (neither returns `null`), and one or the other will change value depending on which direction you scroll in and the `writing-mode` of the content:

- If the scroller is scrolled horizontally, the `snapTargetInline` property will change as the snapped element changes if the content has a horizontal `writing-mode`, or the `snapTargetBlock` property if the content has a vertical `writing-mode`.
- If the scroller is scrolled vertically, the `snapTargetBlock` property will change as the snapped element changes if the content has a horizontal `writing-mode`, or the `snapTargetInline` property if the content has a vertical `writing-mode`.

To handle this, you will likely need to keep track of whether it was the `snapTargetBlock` or the `snapTargetInline` element that changed. Let's look at an example:

```
const prevState = {
  snapTargetInline: "s1",
  snapTargetBlock: "s1",
};

scrollingElem.addEventListener("scrollsnapchange", (event) => {
  if (!(prevState.snapTargetBlock === event.snapTargetBlock.id)) {
    console.log(
      `The container was scrolled in the block direction to element ${event.snapTargetBlock.id}`,
    );
  }

  if (!(prevState.snapTargetInline === event.snapTargetInline.id)) {
    console.log(
      `The container was scrolled in the block direction to element ${event.snapTargetBlock.id}`,
    );
  }

  prevState.snapTargetBlock = event.snapTargetBlock.id;
  prevState.snapTargetInline = event.snapTargetInline.id;
});
```

In this snippet, we first define an object (`prevState`) that stores the ID of the previous `snapTargetBlock` and `snapTargetInline` elements.

In the event handler function, we use `if` statements to test whether:

- The `prevState.snapTargetBlock` ID is equal to the ID of the current `event.snapTargetBlock` element.
- The `prevState.snapTargetInline` ID is equal to the ID of the current `event.snapTargetInline` element.

If the values are different, it means that the scroller has been scrolled in that direction (block or inline), and we log a message to console to indicate this. In a real example, you'd likely style the snapped element in some way to indicate that it has been snapped to.

We then update the values of `prevState.snapTargetBlock` and `prevState.snapTargetInline` ready for when the event handler next runs.

For the remainder of this article, we'll look at a couple of complete scroll snap event examples, which you can play with in the live rendered versions at the end of each section.

One-dimensional scroller example

This example features a vertically-scrolling `<main>` element containing multiple light gray `<section>` elements, which are all scroll snap targets. When a new snap target is pending, it will turn a darker shade of gray. When a new snap target is selected, it will smoothly animate to purple with white text. If a different snap target was previously selected, it will smoothly animate back to gray with black text.

HTML

The HTML for the example is a single `<main>` element. We will add the `<section>` elements dynamically with JavaScript later on, to save on page space.

```
<main></main>
```

CSS

```
* {
  box-sizing: border-box;
}

html {
  height: 100%;
}

body {
  display: flex;
  align-items: center;
  justify-content: center;
  height: inherit;
}

h2 {
  font-size: 1rem;
  letter-spacing: 1px;
}

section {
  font-family: "Helvetica", "Arial", sans-serif;
  border-radius: 5px;
  background: #eeeeee;
  box-shadow:
    inset 1px 1px 4px rgb(255 255 255 / 0.5),
    inset -1px -1px 4px rgb(0 0 0 / 0.5);
  width: 150px;
  height: 150px;

  display: flex;
  align-items: center;
  justify-content: center;
}
```

In the CSS, we start off by giving the `<main>` element a chunky black `border` and a fixed `width` and `height`. We set its `overflow` value to `scroll` so overflowing content will be hidden and can be scrolled to, and set `scroll-snap-type` to `block mandatory` so that snap targets in the block direction only will always be snapped to.

```
main {
  border: 3px solid black;
  width: 250px;
  height: 450px;
  overflow: scroll;
  scroll-snap-type: block mandatory;
}
```

Each `<section>` element is given a `margin` of `50px` to separate out the `<section>` elements and make the scroll snapping behavior more apparent. We then set `scroll-snap-align` to `center`, to specify that we want to snap to the center of each snap target. Finally, we apply a `transition` to smoothly animate to and from the style changes applied when a snap target selection has been made or is pending.

```
section {
  margin: 50px auto;
  scroll-snap-align: center;
  transition: 0.5s ease;
}
```

The style changes mentioned above will be applied through classes applied to the `<section>` elements via JavaScript. The `select-section` class will be applied to signify a selection — this set a purple background and white text color. The `pending` class will be applied to signify a pending snap target selection — this colors the target selection's background a darker gray.

```
.pending {
  background-color: #cccccc;
}

.select-section {
  background: purple;
  color: white;
}
```

JavaScript

In the JavaScript, we start by grabbing a reference to the `<main>` element and defining the number of `<section>` elements to generate (in this case, 21) and a variable to begin counting from. We then use a `while` loop to generate the `<section>` elements, giving each one a child `h2` with text that reads `Section` plus the current value of `n`.

```
const mainElem = document.querySelector("main");
const sectionCount = 21;
let n = 1;

while (n <= sectionCount) {
  mainElem.innerHTML += `
    <section>
      <h2>Section ${n}</h2>
    </section>
  `;
  n++;
}
```

Now on to the `scrollsnapchanging` event handler function. When a child of the `<main>` element (i.e., any `<section>` element) becomes a pending snap target selection, we:

1. Check to see if an element previously had the `pending` class applied and, if so, remove it. This is so that only the current pending target is given the `pending` class and colored darker gray. We don't want previously-pending targets that are no longer pending to keep the styling.
2. Give the element referenced by the `snapTargetBlock` property (which will be one of the `<section>` elements) the `pending` class so it turns a darker gray.

```
mainElem.addEventListener("scrollsnapchanging", (event) => {
  const previousPending = document.querySelector(".pending");
  if (previousPending) {
    previousPending.classList.remove("pending");
  }

  event.snapTargetBlock.classList.add("pending");
});
```

> **Note:** We don't need to worry about the `snapTargetInline` event object property for this demo — we are only scrolling vertically and the demo is using a horizontal writing mode, therefore only the `snapTargetBlock` value will change. In this case, `snapTargetInline` will always return `null`.

When a scrolling gesture ends, and a `<section>` element is actually selected as a snap target, the `scrollsnapchange` event handler function fires. This:

1. Checks to see if a snap target was previously selected — i.e., if a `select-section` class was previously applied to an element. If so, we remove it.
2. Applies the `select-section` class to the `<section>` element referenced in the `snapTargetBlock` property so that the snap target that was just selected will have the selection animation applied to it.

```
mainElem.addEventListener("scrollsnapchange", (event) => {
  const currentlySnapped = document.querySelector(".select-section");
  if (currentlySnapped) {
    currentlySnapped.classList.remove("select-section");
  }

  event.snapTargetBlock.classList.add("select-section");
});
```

Result

Try scrolling up and down the scroll container and observing the behavior described above:

Two-dimensional scroller example

This example is similar to the previous one, except that it features a horizontally- *and* vertically-scrolling `<main>` element containing multiple light gray `<section>` elements, which are all snap targets.

The HTML for the example is the same as for the previous example — a single `<main>` element.

```
<main></main>
```

CSS

```
* {
  box-sizing: border-box;
}

html {
  height: 100%;
}

body {
  display: flex;
  align-items: center;
  justify-content: center;
  height: inherit;
}

section {
  font-family: "Helvetica", "Arial", sans-serif;
  border-radius: 5px;
  background: #eeeeee;
  box-shadow:
    inset 1px 1px 4px rgb(255 255 255 / 0.5),
    inset -1px -1px 4px rgb(0 0 0 / 0.5);
  width: 150px;
  height: 150px;

  display: flex;
  align-items: center;
  justify-content: center;

  scroll-snap-align: center;
}

h2 {
  font-size: 1rem;
  letter-spacing: 1px;
}
```

The CSS for this example is similar to the CSS in the previous example. The most significant differences are as follows.

First let's look at the `<main>` element styling. We want the `<section>` elements to be laid out as a grid, so we use CSS grid layout to specify that we want them displayed in seven columns, using a `grid-template-columns` value of `repeat(7, 1fr)`. We also specify the space around the `<section>` elements by setting `padding` and `gap` on the `<main>` element rather than `margin` on the `<section>` elements.

Finally, since we are scrolling in both directions in this example, we set `scroll-snap-type` to `both mandatory` so that snap targets in the block direction *and* inline direction will always be snapped to.

```
main {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  padding: 100px;
  gap: 50px;
  overflow: scroll;
  border: 3px solid black;
  width: 350px;
  height: 350px;

  scroll-snap-type: both mandatory;
}
```

Next, we are going to use CSS animations in this example instead of transitions. This results in more complex code, but enables more fine-grained control over the animations applied.

We first define the classes that will be applied to signal that a snap target selection has been made or is pending. The `select-section` and `deselect-section` classes will apply keyframe animations to signify a selection or deselection. The `pending` class will be applied to signify a pending snap target selection (it applies a darker gray background to the selection, as in the previous example).

The `@keyframes` animate from a gray background and black (default) text color to a purple background and white text color, and back again, respectively. The latter animation is somewhat different from the first one — it also uses `opacity` to create a fade out/fade in effect.

```
.select-section {
  animation: select 0.8s ease forwards;
}

.deselect-section {
  animation: deselect 0.8s ease forwards;
}

.pending {
  background-color: #cccccc;
}

@keyframes select {
  from {
    background: #eeeeee;
    color: black;
  }

  to {
    background: purple;
    color: white;
  }
}

@keyframes deselect {
  0% {
    background: purple;
    color: white;
    opacity: 1;
  }

  80% {
    background: #eeeeee;
    color: black;
    opacity: 0.1;
  }

  100% {
    background: #eeeeee;
    color: black;
    opacity: 1;
  }
}
```

JavaScript

In the JavaScript, we start off in the same way as with the previous example, except that this time we generate 49 `<section>` elements, and we give each one an ID of `s` plus the current value of `n` to help track them later on. With the CSS grid layout we specified above, we have seven columns of seven `<section>` elements.

```
const mainElem = document.querySelector("main");
const sectionCount = 49;
let n = 1;

while (n <= sectionCount) {
  mainElem.innerHTML += `
    <section id="s${n}">
      <h2>Section ${n}</h2>
    </section>
  `;
  n++;
}
```

Next we specify an object called `prevState`, which allows us to keep track of the previously-selected snap target at any point — its properties store the previous inline and block snap targets' IDs. This is important for figuring out if we need to style the new block target or the new inline target each time an event handler fires.

```
const prevState = {
  snapTargetInline: "s1",
  snapTargetBlock: "s1",
};
```

For example, let's say the scroll container is scrolled so that the ID of the new `SnapEvent.snapTargetBlock` element has changed (it doesn't equal the ID stored in `prevState.snapTargetBlock`), but the ID of the new `SnapEvent.snapTargetInline` element is still the same as the ID stored in `prevState.snapTargetInline`. This means that we've moved to a new snap target in the block direction, so we should style `SnapEvent.snapTargetBlock`, but we've not moved to a new snap target in the inline direction, so we shouldn't style `SnapEvent.snapTargetInline`.

This time around, we'll explain the `scrollsnapchange` event handler function first. In this function, we:

1. Start by making sure that a previously-selected `<section>` element snap target (as signified by the presence of the `select-section` class) has the `deselect-section` class applied so it shows the deselection animation. If no snap target was previously selected, we apply the `select-section` class to the first `<section>` in the DOM so it shows up as selected when the page first loads.
2. Compare the previously-selected snap target ID to the newly-selected snap target ID, for both the block *and* inline selections. If they are different, it indicates that the selection has changed, so we apply the `select-section` class to the appropriate snap target to visually indicate this.
3. Update `prevState.snapTargetBlock` and `prevState.snapTargetInline` to be equal to the IDs of the scroll snap targets that were just selected, so that when the event next fires, they will be the previous selections.

```
mainElem.addEventListener("scrollsnapchange", (event) => {
  if (document.querySelector(".select-section")) {
    document.querySelector(".select-section").className = "deselect-section";
  } else {
    document.querySelector("section").className = "select-section";
  }

  if (!(prevState.snapTargetBlock === event.snapTargetBlock.id)) {
    event.snapTargetBlock.className = "select-section";
  }

  if (!(prevState.snapTargetInline === event.snapTargetInline.id)) {
    event.snapTargetInline.className = "select-section";
  }

  prevState.snapTargetBlock = event.snapTargetBlock.id;
  prevState.snapTargetInline = event.snapTargetInline.id;
});
```

When the `scrollsnapchanging` event handler function fires, we:

1. Remove the `pending` class from the element that previously had it applied so that only the current pending target is given the `pending` class and colored darker gray.
2. Give the current pending element the `pending` class so it turns a darker gray, but only if it has not already got the `select-section` class applied — we want a previously selected target to keep the purple selection styling until a new target is actually selected. We also include an extra check in the `if` statements to make sure we style only the inline or block pending snap target, depending on which one has changed. Again, we compare the previous snap target to the current snap target in each case.

```
mainElem.addEventListener("scrollsnapchanging", (event) => {
  const previousPending = document.querySelector(".pending");
  if (previousPending) {
    previousPending.className = "";
  }

  if (
    !(event.snapTargetBlock.className === "select-section") &&
    !(prevState.snapTargetBlock === event.snapTargetBlock.id)
  ) {
    event.snapTargetBlock.className = "pending";
  }

  if (
    !(event.snapTargetInline.className === "select-section") &&
    !(prevState.snapTargetInline === event.snapTargetInline.id)
  ) {
    event.snapTargetInline.className = "pending";
  }
});
```

Result

Try scrolling horizontally and vertically around the scroll container and observing the behavior described above:

Scroll snap events on <code>Document</code> and <code>Window</code>

In this article, we've covered the scroll snap events that fire on the `Element` interface, but the same events also fire on the `Document` and `Window` objects. See:

- `Document` `scrollsnapchange` and `scrollsnapchanging` event references.
- `Window` `scrollsnapchange` and `scrollsnapchanging` event references.

These work in much the same way as the `Element` versions, except that the overall HTML document has to be set as the scroll snap container (i.e., `scroll-snap-type` is set on the `<html>` element).

For example, if we took a similar example to the ones we've looked at above, where we've got a `<main>` element containing significant content:

```
<main>
  <!-- Significant content -->
</main>
```

The `<main>` element could be turned into a scroll container using a combination of CSS properties, for example:

```
main {
  width: 250px;
  height: 450px;
  overflow: scroll;
}
```

You could then implement scroll snapping behavior on the scrolling content by specifying the `scroll-snap-type` property on the `<html>` element:

```
html {
  scroll-snap-type: block mandatory;
}
```

The following JavaScript snippet would cause the `scrollsnapchange` event to fire on the HTML document when a child of the `<main>` element becomes a newly-selected snap target. In the handler function, we set a `selected` class on the child referenced by the `SnapEvent.snapTargetBlock`, which could be used to style it to look like it has been selected (for example, with an animation) when the event fires.

```
document.addEventListener("scrollsnapchange", (event) => {
  event.snapTargetBlock.classList.add("selected");
});
```

We could fire the event on `Window` instead, to achieve the same functionality:

```
window.addEventListener("scrollsnapchange", (event) => {
  event.snapTargetBlock.classList.add("selected");
});
```

See also

- `scrollsnapchanging` event
- `scrollsnapchange` event
- `SnapEvent`
- CSS scroll snap module
- Scroll Snap Events on developer.chrome.com (2024)

CSS scroll-driven animations

The **CSS scroll-driven animations** module provides functionality that builds on the CSS animations module and Web Animations API. It allows you to animate property values along a scroll-based timeline rather than the default time-based document timeline. This means that you can animate an element by scrolling the element, its scroll container, or its root element, rather than just by the passing of time.

Scroll-driven animations in action

You can define the scroller that controls the animation either by naming the animation or with the `scroll()` or `view()` functions.

```
<main>
  <div></div>
</main>
```

```
main {
  scroll-timeline: --main-timeline;
}

div {
  animation: background-animation linear;
  animation-timeline: scroll(nearest inline);
}

div::after {
  animation: shape-animation linear;
  animation-timeline: --main-timeline;
}
```

```
@layer animations {
  @keyframes background-animation {
    0% {
      background-color: palegoldenrod;
    }
    100% {
      background-color: magenta;
    }
  }
  @keyframes shape-animation {
    0% {
      left: 0;
      top: 0;
      background-color: black;
    }
    50% {
      top: calc(100% - var(--elSize));
      left: calc(50% - var(--elSize));
      background-color: red;
    }
    100% {
      left: calc(100vw - var(--elSize));
      top: 0;
      rotate: 1800deg;
      background-color: white;
    }
  }
}

@layer page-setup {
  :root {
    --elSize: 50px;
  }
  main {
    height: 90vh;
    overflow: scroll;
    border: 1px solid;
    margin: 5vh auto;
  }
  div {
    height: 400vh;
    width: 400vw;
  }
  div::after {
    content: "";
    border: 1px solid red;
    height: var(--elSize);
    width: var(--elSize);
    position: absolute;
    border-radius: 20px;
    corner-shape: superellipse(-4);
  }
}

@layer no-support {
  @supports not (scroll-timeline: --main-timeline) {
    body::before {
      content: "Your browser doesn't support scroll-driven animations.";
      background-color: wheat;
      display: block;
      text-align: center;
```

```
      padding: 1rem 0;
    }
  }
}
```

Scroll the element in the inline direction to see its background color change. Scroll it vertically to see the generated content move, spin, and change colors.

Reference

Properties

- `animation-range` shorthand
  - `animation-range-end`
  - `animation-range-start`
- `scroll-timeline` shorthand
  - `scroll-timeline-axis`
  - `scroll-timeline-name`
- `timeline-scope`
- `view-timeline` shorthand
  - `view-timeline-axis`
  - `view-timeline-inset`
  - `view-timeline-name`

Data types and values

- `<axis>`
- `<timeline-range-name>`

Functions

- `scroll()`
- `view()`

Interfaces

- `ScrollTimeline`
- `ViewTimeline`

Guides

Scroll-driven animation timelines

Scroll-driven animation timelines and creating scroll-driven animations.

Timeline range names

The `<timeline-range-name>` data type: Understanding the various timeline range names.

Using CSS scroll-triggered animations

Creating scroll-triggered animations.

Inserting view progress timelines

Inserting the animation attachment ranges of scroll-driven animations.

Related concepts

- CSS animations module
  - `animation-timeline`
  - `@keyframes` at-rule
  - `<keyframe-selector>`
- CSS animation triggers module
  - `animation-trigger`
  - `timeline-trigger` shorthand
  - `timeline-trigger-activation-range` shorthand
    - `timeline-trigger-activation-range-end`
    - `timeline-trigger-activation-range-start`
  - `timeline-trigger-active-range` shorthand
    - `timeline-trigger-active-range-end`
    - `timeline-trigger-active-range-start`
  - `timeline-trigger-name`
  - `timeline-trigger-source`
  - `trigger-scope`
- CSS overflow module
  - Scroll container
  - Scrollport
- Web Animations API
  - `Element.animate()`
  - `Animation`
  - `AnimationTimeline`
  - `DocumentTimeline`
  - `KeyframeEffect`

See also

- Animate elements on scroll with scroll-driven animations via developer.chrome.com (2023)

Understanding timeline insets

By default, view progress timelines track elements across the entire animation attachment range. The `0%` progress point is at the start of the range, while the `100%` progress point is at the end. The animation attachment range can be changed by setting a timeline range name, and the location of the `0%` and `100%` progress points along the range can be adjusted by setting length or percentage-based inset values.

This guide explains how to limit the animation timeline to a specific portion of the animation timeline range using length or percentage inset values.

Animation timelines: a primer

CSS animations are created by defining named `@keyframes` animations, which specify an animation's behavior, and then attaching the keyframe animation to an element using the animation's name.

The element's animation timeline, defined by the `animation-timeline` property, determines how and when the element progresses through those keyframes. By default, the timeline is time-based, using the document's default time-based `DocumentTimeline`.

The CSS scroll-driven animation module defines scroll progress and view progress timelines, which are methods for animating property values along a scroll-based timeline rather than the default time-based document timeline. In this article, we'll only discuss view progress timelines, as scroll progress timelines are not relevant to timeline insets.

View progress timelines

With view progress timelines, the animation's timeline, or progress, is driven by element visibility instead of the passage of time, with keyframe progression tied to the subject element position and visibility within the scroll container. The animation advances and reverses as the element advances or reverses through the scrollport. The animation only occurs when at least part of the element is visible within its scrollport, pausing when scrolling pauses.

```
.animated_element {
  animation-name: nameOfAnimation;
  animation-timeline: view();
}
```

Setting an `animation-name` applies the animation to the selected element.

> **Note:** The `animation-timeline` property should always come after any `animation` shorthand declarations. While the shorthand property can't be used to set the `animation-timeline` property, it does reset the timeline to the default time-based document timeline.

> **Note:** In all the examples, the scroll container is `250px` tall and we are using the default values for `animation-iteration-count` (`1`), `animation-delay` (`0s`), and `animation-direction` (`normal`). We set the `animation-timing-function` to `step-end` and the `animation-fill-mode` is to `forward` to make the it more apparent when the animation iteration has not yet started, when it is active, and when it is complete. See the Using CSS animations guide to learn more.

As you scroll up, the animation progresses. As you scroll down, the animation reverses.

In this example, whenever any part of the subject element is visible in the scrollport, the animation occurs. By default, view progress animations begin just as the top edge of the subject element aligns with the bottom edge of the scroll container and end, reaching `100%` progress, when the end edge aligns with the start edge of the container, no matter the size of the subject element. By default, the animation is applied when any part of the subject is visible within the scrollport.

Animation attachment ranges

In a view progress timeline, when no animation range properties are defined, the `<timeline-range-name>` is `normal`, which defaults to `cover`. The animation is applied whenever any portion of the subject element is visible, meaning the default **animation attachment range** is the sum of the height of the scroll container and the height of the subject element, with that extra height being at the scroll end edge. In our example, as the scroll container is `250px` tall, and the subject is `50px`, `250px`, or `500px` tall, with the vertical animation attachment range being `300px`, `500px`, or `750px` respectively.

The `0%` progression occurs when the subject element's start edge intersects the scrollport at the end edge, reaching `100%` progress when the subject's end edge exits via the scrollport's start edge. These are the subject and scrollport's top and bottom edges when scrolling vertically, and the left and right or right and left edges when scrolling horizontally, depending on the writing mode.

The following diagram illustrates the position of the subject at the `0%` and `100%` progress points for the three subject sizes:

```
<div>
  <svg viewBox="-1 -1 462 1252" xmlns="http://www.w3.org/2000/svg">
    <title>Default view progress timeline</title>
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small end" width="100" height="50" x="10" y="450" />
    <rect class="medium end" width="100" height="250" x="125" y="250" />
    <rect class="large end" width="100" height="500" x="240" y="0" />
    <rect class="small start" width="100" height="50" x="10" y="750" />
    <rect class="medium start" width="100" height="250" x="125" y="750" />
    <rect class="large start" width="100" height="500" x="240" y="750" />
    <text y="520" x="360">100%</text>
    <line x1="0" x2="350" y1="500" y2="500" />
    <line x1="0" x2="350" y1="750" y2="750" />
    <text y="760" x="360">0%</text>
  </svg>
</div>
```

The yellow subject elements represents the position of the element when the `from` keyframe is applied, which is the animation range's `0%` progress mark. The red represents the location of the animated element relative to the scrollport when the `to` keyframe is applied, which is the end of the animation, or the `100%` progress mark. The gray represents the scrollport.

By default, the element animates while it is "in view", but this default definition of "in view" may not fit your needs. Fortunately, we can control which edges define the edges of the animation attachment range and then offset the start and end of that range with the animation range properties.

Animation range properties

The `animation-range` properties enable specifying a named timeline range, such as `contain` or `exit-crossing`, which changes the range used from the default `cover` range. You can also include a `<length-percentage>` value, which insets the attachment range from the start of the range. Percentages are relative to the named or default timeline range.

Named timeline ranges define the portions of a `ViewTimeline` that define an animation's range, specifying the start and end of the animations's attachment range.

The `animation-range` property is a shorthand property, defining the `animation-range-start` and `animation-range-end` properties. The `animation-range-start` defines the position of the subject element when the animation starts. The `animation-range-end` defines the position of the subject element when the animation ends.

See the timeline range names guide to learn about the different named timeline ranges. This guide focuses on how the `<length-percentage>` inset values work.

Setting insets using lengths

The `animation-range-start` and `animation-range-end` properties each accept a named animation range, a `<length-percentage>` offset value, or both. Any length or percentage offset is measured from the *start* of the animation attachment range.

When a `<length>` is set, the offset is fairly intuitive. Here we use the `animation-range-start` and `animation-range-end` properties to inset the animation timeline. This defines a subsection of the element's full animation attachment range as the active interval, with the `<length>` values specifying distances from the start of the default `normal` animation attachment range.

```
.animated_element {
  animation-range-start: 1em;
  animation-range-end: 125px;
}
```

The start and end of the animation range are `1em` and `125px` from the start of the animation attachment range, respectively. Because the timeline range default is `normal`, which resolves to `cover`, the start of the animation attachment range is the block end edge of the container.

```
:root {
  --start: 1em;
  --end: 125px;
}

article {
  background-image: linear-gradient(
    to top,
    transparent calc(var(--start) - 1px),
    #cccccc calc(var(--start) - 1px) calc(var(--start) + 1px),
    transparent calc(var(--start) + 1px) calc(var(--end) - 1px),
    #cccccc calc(var(--end) - 1px) calc(var(--end) + 1px),
    transparent calc(var(--end) + 1px)
  );
}
```

We've added lines `1em` and `125px` from the block end edge of the scroll container. The animation starts when the block start edge of the subject element reaches the `1em` line and ends when it reaches the `125px` line.

In this case, as the animation attachment range resolves to `cover` for both the start and end inset values, the location of the insets is fairly straightforward.

Effect of named ranges on length offsets

The offset distance is always from the start of the associated animation range. In this example, we set the `animation-range-start` to be `50px` from the start of the default `normal` range and set the `animation-range-end` to be `100px` from the start of the explicitly set `entry` range:

```
.animated_element {
  animation-range-start: 50px;
  animation-range-end: entry 100px;
}
```

```
<main>
  <article>
    <p> </p>
    <p> </p>
    <p>Scroll down ⇩</p>
    <p> </p>
    <p> </p>
    <section class="triple">
      <div>
        <i id="A" class="animated_element">50px</i>
        <i id="B" class="animated_element">250px</i>
        <i id="C" class="animated_element">500px</i>
      </div>
    </section>
    <p> </p>
    <p> </p>
    <p> </p>
    <p> </p>
    <p>Scroll up ⇧</p>
  </article>
</main>
```

As the start edge of both the `normal` and `entry` ranges is the container's end edge, the animation begins when the subject start edge is `50px` from the bottom of the scrollport and ends, reaching `100%` progress, when the subject start edge is `100px` from the bottom of the scrollport, irrespective of the subject size. While the size of the `entry` range is different for the three different subject sizes, in this case, the size of the underlying range didn't matter.

Length offsets with varying ranges

The size of range matters when the range doesn't start at the element's end edge, as is the case with `exit` and `exit-crossing`, or if the offset is a percentage value. This fact, and the fact that you can mix and match animation range names, make view progress timeline offsets a bit more complicated to understand than non-offsetted timeline range names.

For example, when setting `exit` as the timeline range name, the subject size matters as it determines the location of the range's end edge.

```
.animated_element {
  animation-range-start: entry 60px;
  animation-range-end: exit 75px;
}
```

With both `entry` and `exit`, the range is the size of the subject, with the size clamped to the size of the scrollport. This means that the height of the `entry` and `exit` ranges are the height of the box in both the `50px` and `250px` examples, while in the `500px` example, the range is clamped to the height of the scrollport, which is `250px` tall.

We added a few lines to facilitate the explanations that follow: the bottom blue line is `60px` from the end edge of the scrollport, and the top red line is `75px` from that same edge. These are where the animation range start and end respectively.

This example demonstrates several important features, which we'll explain in greater detail, including:

- Offsets are measured from their respective named ranges
- Offsets can occur beyond the edges of the scrollport
- Ranges may be clamped if the subject is larger than the scrollport

Measured from the start edge of the range

As the offset position is always relative to the start of the declaration's animation range, the start of the animation for all three elements occurs when the elements start edge crosses the point that is `60px` from the start of the `entry` range.

The `animation-range-end` value defines the position where the animation. The `exit 75px` value basically means "when `75px` of the subject has exited the start edge of the scrollport." This varies for each subject. For the `50px` subject, this only occurs `25px` after it has left the scrollport; when the element is not visible. The animation range end for both the `250px` and `500px` subjects occurs when their bottom end edge intersects the top blue line; 75 pixels from the end edge of the scrollport. Why are their end offsets the same? Because of clamping! The maximum size of the named animation range is clamped to the size of the scrollport. The `exit` range for both subjects is the same, so the range end offsets are the same.

Beyond the scrollport edges

For our `50px` tall subject, the `exit` range is 50px tall abutting the start edge of the scroll port. Setting `animation-range-end: exit 75px` for any element less than `75px` tall means the end of the range is outside the scrollport, as the point `75px` from the start of the `exit` range is past the start edge of the scrollport. In our example, the end of the animation range for the `50px` subject occurs when the subject's start edge is `75px`

past the scrollport's start edge. The animation ends, reaching the `to` keyframe and the `animationend` event, only when (and if) the element is scrolled `25px` out of view.

The animation ends even if the animation range end is outside of the scrollport, as long as there is room to scroll to that point. Had we set `animation-range-end: exit 250px`, the animation would have ended when the end edge of the medium and tall subjects exited the scrollport at the container's start edge.

With the end set to `exit 250px`, the small subject's animation might not end, as there might not be `450px` of content after the subject for the user to scroll to before the end point is reached.

Effects of clamping

With our `250px` tall container, when the subject is `250px` or `500px` tall, the `exit` range is the size of the container, with the start being the scroll container's end edge. With a `75px` offset, the end of the animation occurs when the end edge of the subject is `75px` from the end edge of the scroll container (denoted by the top red line).

As the offset position is always relative to the start of the named or default animation range, in our example, clamping impacts the large subject's `animation-range-end`. We set the end of the range to `exit 75px` which is `75px` from the start edge of the `exit` range. When the subject is the same size as the scrollport (our `250px` subject) or larger (our `500px` subject), the animation range end is `75px` from the end edge of the scrollport, which is `75px` from the start of the scrollport-clamped `exit` range.

```
article {
  background-image: linear-gradient(
    to top,
    transparent 59.5px,
    blue 59.5px 60.5px,
    transparent 60.5px 74.5px,
    red 74.5px 75.5px,
    transparent 75.5px /* 174.5px,
    green 174.5px 154.5px,
    transparent 175.5px*/
  );
}
.animated_element {
  align-self: flex-end;
}
```

```
@layer setup {
  #A {
    height: 50px;
  }
  #B {
    height: 250px;
  }
  #C {
    height: 500px;
  }
  div {
    display: flex;
    gap: 1em;
  }
  main {
    padding: 20px 0 0 20px;
    margin-bottom: 2em;
  }
  article {
    outline: 3px dashed;
    width: 475px;
    margin: auto;
    overflow: scroll;
    position: relative;
    height: 250px;
    box-sizing: content-box;
    background-image: linear-gradient(
      to top,
      transparent 49.5px,
      #666666 49.5px 50.5px,
      transparent 50.5px 99.5px,
      #666666 99.5px 100.5px,
      transparent 100.5px
    );
    background-origin: content-box;
  }

  p {
    padding: 10px;
    margin: 10px;
  }

  .animated_element {
    --clr: yellow;
    background-color: hsl(from var(--clr) h s calc(l * 1.4));
    display: block;
    animation: showAnim step-end 1 forwards;
    animation-timeline: view();
    flex: 1 0 auto;
  }

  i {
    font-family: sans-serif;
    font-size: 1.5rem;
  }

  @keyframes showAnim {
    from {
      --clr: green;
    }
    to {
      --clr: red;
    }
  }
  @layer no-support {
    @supports not (animation-timeline: view()) {
      body::before {
        content: "Your browser doesn't support view progress scrolling.";
        background-color: wheat;
        display: block;
```

```
        text-align: center;
      }
    }
  }
}
```

Negative lengths

Up to this point, all the offsets have been greater than zero. It's important to note that negative lengths are valid. A negative offset on the `animation-range-start` makes the range longer while a negative offset on the `animation-range-end` makes the range shorter.

Let's compare the negative insets compared to the `0` values:

```
#A {
  animation-range-start: contain -25px;
  animation-range-end: exit -25px;
}
#B {
  animation-range-start: contain 0;
  animation-range-end: exit 0;
}
```

The first animation range is offset by `25px` toward the container end edge.

```
fieldset.double {
  display: none;
}
#A::after {
  content: " (-25px)";
}
#B::after {
  content: " (0)";
}
```

Setting insets using percentages

Like length values, percentage values define offsets from the *start* of the animation attachment range. The percentage offsets are relative to the timeline range dimension, not relative to the scrollport. For this reason, percentage values are not as intuitive as length values for most people (realizing length values weren't that intuitive either).

Here, we use `animation-range-start` and `animation-range-end` to inset the animation timeline. While we are using the same properties, we set `<percentage>` values instead of `<length>` values:

```
.animated_element {
  animation-range-start: 20%;
  animation-range-end: 60%;
}
```

```
i {
  background-image: linear-gradient(
    to bottom,
    transparent calc(20% - 1px),
    #33333333 calc(20% - 1px) calc(20% + 1px),
    transparent calc(20% + 1px) calc(60% - 1px),
    #33333333 calc(60% - 1px) calc(60% + 1px),
    transparent calc(60% + 1px)
  );
}
article {
  --total: calc(var(--animElHeight) + 250px);
  background-image:
    linear-gradient(
      to top,
      transparent 0 calc(var(--total) * 0.2 - 1px),
      green calc(var(--total) * 0.2 - 1px) calc((var(--total) * 0.2) + 1px),
      transparent calc(var(--total) * 0.2 + 1px)
    ),
    linear-gradient(
      to top,
      transparent 0 calc(var(--total) * 0.6 - 1px),
      red calc(var(--total) * 0.6 - 1px) calc((var(--total) * 0.6) + 1px),
      transparent calc(var(--total) * 0.6 + 1px)
    ),
    linear-gradient(
      to top,
      transparent 0 calc(var(--containerHeight) * 0.2 - 0.5px),
      #33333333 calc(var(--containerHeight) * 0.2 - 0.5px)
        calc(var(--containerHeight) * 0.2 + 0.5px),
      transparent calc(var(--containerHeight) * 0.2 + 0.5px)
        calc(var(--containerHeight) * 0.6 - 0.5px),
      #33333333 calc(var(--containerHeight) * 0.6 - 0.5px)
        calc(var(--containerHeight) * 0.6 + 0.5px),
      transparent 0 calc(var(--containerHeight) * 0.6 + 0.5px)
    );
  background-attachment: local, local, fixed;
}
```

This defines the active interval to begin `20%` into the default attachment range and end `60%` through that same range. The default `normal` animation attachment range, which behaves as `cover`, is the height of the scroll container plus the height of the subject element, meaning the range will differ based on which radio button is selected.

For illustrative purposes, there are two dark lines crossing the container at the `20%` and `60%` points of the full animation range. The animation starts when the block-start edge reaches the `20%` point, which is the bottom green line. The animation ends when the start block edge is 60% of the way through the normal range, which is the top red line.

Only when the element is `50px` tall is the top of the subject still in the scrollport when the end of the animation is reached; there are no top red lines when `250px` or `500px` are selected, as the end of the animation range is outside of the scrollport.

Based on the height of our subjects, the `20%` mark is either `60px`, `100px`, or `150px` from the end edge of the scrollport (marked by the green line, which is always in the scrollport), and the `60%` mark is `180px`, `300px`, or `450px` from the same point (marked with a red line, but only visible for the `50px` subject).

For illustrative purposes, there are two light gray lines crossing the container `20%` and `60%` of the way through the scrollport, which are `50px` and `150px` from the bottom of the scrollport, respectively. As the `animation-range-*` percentages are relative to the timeline range, not the scrollport, these lines only show how the percentages **don't** align. We've also included two horizontal light gray lines going across each

subject at their own `20%` and `60%` marks. These lines align with the scrollport's light gray lines when each subjects animation starts and ends.

The following image demonstrates where the subject elements are located when the animation starts (the `0%` keyframe) and ends (the `100%` `keyframe`). This image includes the insets from the animation timeline in the previous demonstration and the timeline without insets for comparison.

```
<div>
  <svg viewBox="-1 -1 482 1252" xmlns="http://www.w3.org/2000/svg">
    <title>Default view progress timeline with insets</title>
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small end" width="100" height="50" x="10" y="571" />
    <rect class="medium end" width="100" height="250" x="120" y="450" />
    <rect class="large end" width="100" height="500" x="230" y="300" />
    <rect class="small start" width="100" height="50" x="10" y="689" />
    <rect class="medium start" width="100" height="250" x="120" y="649" />
    <rect class="large start" width="100" height="500" x="230" y="600" />
    <rect width="96" height="48" x="122" y="602" fill="url(#g)" />
    <rect width="96" height="198" x="232" y="527" fill="url(#g)" />
    <text y="610" x="385">60%</text>
    <line x1="0" x2="385" y1="600" y2="600" />
    <line x1="0" x2="385" y1="700" y2="700" />
    <text y="710" x="385">20%</text>
  </svg>
  <svg viewBox="-1 -1 482 1252" xmlns="http://www.w3.org/2000/svg">
    <title>Default view progress timeline</title>
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small end" width="100" height="50" x="10" y="450" />
    <rect class="medium end" width="100" height="250" x="125" y="250" />
    <rect class="large end" width="100" height="500" x="240" y="0" />
    <rect class="small start" width="100" height="50" x="10" y="750" />
    <rect class="medium start" width="100" height="250" x="125" y="750" />
    <rect class="large start" width="100" height="500" x="240" y="750" />
    <text y="520" x="385">100%</text>
    <line x1="0" x2="385" y1="500" y2="500" />
    <line x1="0" x2="385" y1="750" y2="750" />
    <text y="760" x="390">0%</text>
  </svg>
</div>
```

As before, the yellow represents the position of the element when the `from` keyframe is applied, the red represents the location when the `to` keyframe is applied, and the gray represents the scrollport. The striped areas are where the red and yellow element representations overlap. For illustrative purposes, we've added dashed black horizontal lines `20%` and `60%` way through the scrollport, starting from the bottom.

The animation only begins when the element reaches the `20%` mark along the animation attachment range. This point is `60px`, `100px`, or `150px` from the bottom edge of the scroll port, depending on the size of the element. The location of the subject element at this point, representing the position of the element when the `from` or `0%` keyframe is applied, is shown in yellow.

The red represents the location of the animated element relative to the scrollport when the `to` or `100%` keyframe is applied, which is the end of the animation. This point is either `180px`, `300px`, or `450px` from the bottom edge of the scrollport, depending on the subject size. The animation occurs when the element is between the `to` and the `from` positions.

You may have noticed something interesting about the dashed horizontal lines: when the animation starts, the line that is `20%` from the end edge of the viewport is `20%` from the *top* of the subject element and the line that is `60%` from the end edge of the viewport is `60%` from the *top* of the subject element when the animation ends. This is what was illustrated by the very light gray lines in the live demo for this example.

Subject size matters

As we saw when we set insets with lengths, the size of the subject can make a difference. When setting animation ranges, percentage values are relative to the size of animation attachment range, not the scrollport. For most named ranges, the size of the attachment range depends partially on the subject size. As percentages are based on the size of the range, the named range impacts the resolved size of the insets. Depending on the name, the start position may also change, impacting the location of the range and therefore the location of progress points.

In this example, we define an active range that is `40%` of the size of the subject:

```
.animated_element {
  animation-range-start: exit-crossing -20%;
  animation-range-end: exit-crossing 20%;
}
```

```
article {
  background-image: none;
}
body .animated_element {
  align-self: start;
}
```

The animation lasts `40%` of the animation-attachment range. As you scroll, note how the larger the subject, the longer the range. With exit-crossing, the animation range is not cropped; it is the size of the subject even if the subject is larger than the viewport, with the range abutting the start edge of the scrollport, and extending off the end edge if the subject is larger than the scrollport.

With the `-20%` and `20%` insets, the `50px` subject's will animation over `20px`: the animation starts when the subject's end is `-10px` from range start, or `60px` from exiting the screen, and ends when the subject's end is `40px` from exiting the screen. The middle subject will animate over `100px`: the animation starts when the subject end is `-50px` from range start, which is `50px` off of the scrollport's end edge, and ends when the subject's end is `50px` into the scrollport. The large subject animates over `200px`, starting when the bottom is `600px` from the container's start edge, with only `150px` in view, and ends when the bottom is 400px from that start edge, when `100px` have scrolled off the start edge.

Percentages equal to the scrollport

When it comes to offsetting with percentages, the least complicated named timeline range is `contain`. With `contain`, the animation range is the size of the scrollport, meaning the start and end percentages are relative to the scrollport. For this reason, when using offsets, you may want to use `contain` instead of letting the range default and resolve to `cover`.

The `contain` range fully *contains* the animation within the scrollport. It represents the range during which the principal box is either fully contained by, or fully covers, its view progress visibility range within the scrollport. With `contain`, if the subject is the same size or smaller than the scrollport, it can be fully visible. If the element is the same size as the container, however, the animation occurs over `0px`. This means that it runs, but it is not visible to the user.

In other words, without needing to know the size of the container or the subjects, we are able to limit our animation to the middle of scrollport, though the animation will happen over `0px` if the subject is the same size as the scrollport.

```
.animated_element {
  animation-range-start: contain 25%;
  animation-range-end: contain 75%;
}
```

```
article {
  background-image: linear-gradient(
    transparent 25%,
```

```
    #ededed 25% 75%,
    transparent 75%
  );
}
body .animated_element {
  align-self: center;
}

.animated_element {
  background-image:
    linear-gradient(black, black), linear-gradient(black, black);
  background-size: 1px 1px;
  background-position:
    center 25%,
    center 75%;
  background-repeat: repeat-x;
```

The horizontal lines denote the middle half of the scrollport and the middle half of each subject.

```
<svg class="gradient">
  <title>Striped repeating gradient</title>
  <defs>
    <linearGradient
      id="g"
      x1="0"
      y1="0"
      x2="20"
      y2="20"
      spreadMethod="repeat"
      gradientUnits="userSpaceOnUse">
      <stop offset="50%" stop-color="red" />
      <stop offset="50%" stop-color="yellow" />
    </linearGradient>
  </defs>
</svg>
```

```
body::before {
  display: block;
  text-align: center;
  font-family: sans-serif;
  font-size: 1.5rem;
}
div {
  display: flex;
  gap: 20px;
}
svg {
  width: 260px;
}
rect {
  stroke: black;
  stroke-width: 3;
}
.start {
  fill: yellow;
}
.end {
  fill: red;
}
.container {
  fill: #dedede;
}
text {
  font: 40px monospace;
  fill: black;
}
line {
  stroke: black;
  stroke-width: 2;
  stroke-dasharray: 7;
}
.gradient {
  height: 1px;
  width: 1px;
  position: absolute;
  top: -100px;
}
```

```
<main>
  <article>
    <p> </p>
    <p> </p>
    <p>Scroll down ⇩</p>
    <p> </p>
    <p> </p>
    <section class="one animated_element">
      <div>
        <i>Animated Element</i>
        <span></span>
      </div>
    </section>
    <section class="double">
      <div>
        <i id="A" class="animated_element">A</i>
        <i id="B" class="animated_element">B</i>
      </div>
    </section>
    <p> </p>
    <p> </p>
    <p> </p>
    <p> </p>
    <p>Scroll up ⇧</p>
```

```
  </article>
</main>
```

```
<fieldset>
  <legend>Select the height of the animated element</legend>

  <label><input name="height" value="50" type="radio" checked /> 50px</label>
  <label><input name="height" value="250" type="radio" /> 250px</label>
  <label><input name="height" value="500" type="radio" /> 500px</label>
</fieldset>
<fieldset class="double">
  <legend>Select the animation range</legend>

  <label><input name="range" value="20" type="radio" checked />20% / 60%</label>
  <label><input name="range" value="0" type="radio" /> 0% / 100%</label>
</fieldset>
```

```
@layer {
  :root {
    --animElHeight: 50px;
    --animElHeightWord: "50px";
    --barColor: black;
    padding-top: 20px;
    --containerHeight: 250px;
  }
  body:has(input[value="250"]:checked) {
    --animElHeight: 250px;
    --animElHeightWord: "250px";
  }
  body:has(input[value="500"]:checked) {
    --animElHeight: 500px;
    --animElHeightWord: "500px";
  }
  main {
    padding: 20px 0 0 20px;
    margin-bottom: 2em;
  }
  article {
    outline: 3px dashed;
    width: 475px;
    margin: auto;
    overflow: scroll;
    position: relative;
    height: var(--containerHeight);
    box-sizing: content-box;
  }

  p {
    padding: 10px;
    margin: 10px;
  }

  section {
    --clr: yellow;
    --words: "Animation not started";
    position: relative;
    margin: 20px;
    text-align: center;
  }
  .one,
  .double i {
    animation: showAnim step-end 1 forwards;
    animation-timeline: view();
  }
  i,
  .animated_element {
    background-color: hsl(from var(--clr) h s calc(l * 1.4));
    display: block;
    height: var(--animElHeight);
    line-height: var(--animElHeight);
  }
  span {
    background-color: hsl(from var(--clr) h s 90%);
    border: 5px solid hsl(from var(--clr) h s 20%);
    min-width: 250px;
    height: 30px;
    line-height: 30px;
  }
  span,
  i {
    font-family: sans-serif;
    font-size: 1.5rem;
  }
  span::before {
    content: var(--words);
  }
  span {
    position: fixed;
    top: 10px;
    left: 10px;
    padding: 10px;
  }
  i::after {
    content: " ( " var(--animElHeightWord) " )";
  }
  label {
    padding-right: 2em;
  }
  legend {
    margin-top: 2em;
  }
```

```
  @keyframes showAnim {
    from {
      --clr: green;
      --words: "Currently animating";
    }
    to {
      --clr: red;
      --words: "Animation complete";
    }
  }
  body::before {
    display: block;
    text-align: center;
    font-family: sans-serif;
    font-size: 1.5rem;
  }

  @layer no-support {
    @supports not (animation-timeline: view()) {
      body::before {
        content: "Your browser doesn't support view progress scrolling.";
        background-color: wheat;
        display: block;
        text-align: center;
      }
    }
  }
}
```

```
.double {
  display: none;
}
```

```
.one {
  display: none;
}
.double div {
  display: flex;
  gap: 10px;
}
```

See also

- `<timeline-range-name>` data type
- Keyframe selectors
- Scroll-driven animation timelines
- Scroll-driven animation module
- CSS animations module
- Web Animations API

Understanding timeline range names

By default, view progress timelines track elements across the entire scrollport. The animation timeline begins when the first pixel of the element's start edge crosses the scrollport's end edge, and ends when the element's end edge crosses the scrollport's start edge. You can change this default animation attachment range. For example, you can restrict the view progress timeline to begin only when the subject element has fully entered the scrollport.

This guide explains how to modify timeline range names, specifically examining the various timeline range names, their meanings, and how they are used.

Limiting the animation timeline to a specific portion of a named animation timeline range is discussed in the insetting scroll animations guide.

View progress timeline primer

CSS animations are created by attaching `@keyframes` animations to an element using the `animation-name` property (or `animation` shorthand). The keyframes define the animation's behavior, while the `animation-timeline` determines when and how the element progresses through those keyframes.

By default, the animation's timeline is the document's default time-based `DocumentTimeline`. With CSS scroll-driven animations, the animation's timeline, or progress, is driven either by user scroll (scroll progress timelines) or element visibility (view progress timelines), instead of the passage of time.

With view progress timelines, keyframe progression is tied to how much of the subject element is visible within the scroller, and its position within the scroller. As the element enters the viewport, the timeline advances. If the user reverses the scrolling, the timeline reverses. In other words, as the element comes into or moves out of view, the timeline progresses or reverses, respectively. The animation only occurs when the subject is visible within its scrollport. If scrolling stops while the element is in view, the animation pauses.

Default view progress timeline

By default, the view progress timeline's progression starts when the tracked subject's start edge intersects the scrollport at the end edge and ends when the subject's end edge exits the scrollport at the start edge. These are the subject and scrollport's top and bottom edges when scrolling vertically, and the left and right or right and left edges when scrolling horizontally, depending on the writing mode.

```
<main>
  <article>
    <p>&amp;nbsp;</p>
    <p>&amp;nbsp;</p>
    <p>Scroll down ⇩</p>
    <p>&amp;nbsp;</p>
    <p>&amp;nbsp;</p>
    <section class="one animated_element">
      <div>
        <i>Animated Element</i>
        <span></span>
      </div>
    </section>
    <section class="double">
      <div>
        <i id="A" class="animated_element">A</i>
        <i id="B" class="animated_element">B</i>
      </div>
    </section>
    <p>&amp;nbsp;</p>
    <p>&amp;nbsp;</p>
    <p>&amp;nbsp;</p>
```

```
    <p> </p>
    <p>Scroll up ⇧</p>
  </article>
</main>
```

```
<fieldset>
  <legend>Select the height of the animated element</legend>

  <label><input name="height" value="50" type="radio" checked /> 50px</label>
  <label><input name="height" value="250" type="radio" /> 250px</label>
  <label><input name="height" value="500" type="radio" /> 500px</label>
</fieldset>
```

```
:root {
  --animElHeight: 50px;
  --animElHeightWord: "50px";
}
body:has(input[value="250"]:checked) {
  --animElHeight: 250px;
  --animElHeightWord: "250px";
}
body:has(input[value="500"]:checked) {
  --animElHeight: 500px;
  --animElHeightWord: "500px";
}
main {
  padding: 20px 0 0 20px;
  margin-bottom: 2em;
}
article {
  outline: 3px dashed;
  width: 500px;
  margin: auto;
  overflow: scroll;
  position: relative;
  height: 250px;
  box-sizing: content-box;
}

p {
  padding: 10px;
  margin: 10px;
}

section {
  --clr: yellow;
  --words: "Animation not started";
  position: relative;
  margin: 20px;
  text-align: center;
}
.one,
.double i {
  animation: showAnim step-end 1 forwards;
  animation-timeline: view();
}
i {
  background-color: hsl(from var(--clr) h s calc(l * 1.4));
  display: block;
  height: var(--animElHeight);
  line-height: var(--animElHeight);
}
span {
  background-color: hsl(from var(--clr) h s 90%);
  border: 5px solid hsl(from var(--clr) h s 20%);
  min-width: 250px;
  height: 30px;
  line-height: 30px;
}
span,
i {
  font-family: sans-serif;
  font-size: 1.5rem;
}
span::before {
  content: var(--words);
}
span {
  position: fixed;
  top: 10px;
  left: 10px;
  padding: 10px;
}
i::after {
  content: " ( " var(--animElHeightWord) " )";
}
label {
  padding-right: 2em;
}
legend {
  margin-top: 2em;
}

@keyframes showAnim {
  from {
    --clr: green;
    --words: "Currently animating";
  }
  to {
    --clr: red;
    --words: "Animation complete";
  }
```

```
}
body::before {
  display: block;
  text-align: center;
  font-family: sans-serif;
  font-size: 1.5rem;
}

@layer no-support {
  @supports not (animation-timeline: view()) {
    body::after {
      content: "Your browser doesn't support view progress scrolling.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;

      position: absolute;
      inset: 0;
      bottom: auto;
    }
  }
}
```

```
.double {
  display: none;
}
```

```
.one {
  display: none;
}
.double div {
  display: flex;
  gap: 10px;
}
```

In the following example, try scrolling down. Note how the animation begins just as the top edge of the animated element aligns with the bottom edge of the scroll container and ends, reaching `100%` progress, when the bottom edge aligns with the top edge of the container, no matter how tall the animated element is.

The scroll container's writing mode and scroll direction determine the scroll container's start and end edges.

The animation attachment range

By default, the element is animated the entire time any portion of the subject element is visible. This means the default **animation attachment range** is the sum of the height of the scroll container and the height of the subject element, with that extra height being at the scroll end edge.

In the previous example, the scroll container is `250px` tall, and the animated element is `50px` tall by default, which means the vertical animation attachment range is `300px` tall. When the subject is set to `250px`, the range becomes `500px`; when the element is set to `500px`, the animation attachment range size grows to `750px`.

The CSS scroll driven animations module provides mechanisms for defining different animation attachment ranges. The `animation-range-start` and `animation-range-end` properties, which can both be set using the `animation-range` shorthand, define the attachment range for the animation range's start edge and the end edge, along with any insets from either edge, respectively.

The animation-range properties accept the keyword `normal`, a `<timeline-range-name>`, a `<length-percentage>`, or both a `<timeline-range-name>` and the `<length-percentage>`. In this guide, we are covering the values for the `<timeline-range-name>` component of the value only.

The length-percentage values, such as `20%` or `100px`, inset the animation timeline from the named animation attachment ranges, by the set amount, with the value defaulting to `0`. This is covered in Understanding timeline insets.

Timeline range names

The `<timeline-range-name>` value type accepts six keywords: `cover`, `contain`, `entry`, `exit`, `entry-crossing`, and `exit-crossing`. Each of these represents a predefined *named timeline range*. A named timeline range is a named segment of an animation timeline. These keywords allow the developer to set the animation attachment range base that offsets are relative to. The start of the segment is represented as `0%` progress through the range; the end of the segment is represented as `100%` progress through the range. Where these points are depends on the named range used.

Cover

The animation attachment range in the previous examples all "cover" the entire range. This range represents the full range of the view progress timeline. `0%` progress represents the point at which the start border edge of the subject aligns with the end edge of the scrollport, and `100%` progress represents the point where the subject's end border edge reaches the start edge of the scrollport. As we've seen, the size of the `cover` range is the sum of subject and scrollport dimensions in the scroll direction. In all the examples thus far, the height of the animation attachment range is the height of the container plus the height of the animated element.

The `cover` named timeline range is the default range. We could have explicitly set the `<timeline-range-name>` to achieve the same results:

```
.animated_element {
  animation-range-start: cover;
  animation-range-end: cover;
}
```

```
<div>
  <svg viewBox="-1 -1 462 1252" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="450" />
    <rect class="medium end" width="100" height="250" x="125" y="250" />
    <rect class="large end" width="100" height="500" x="240" y="0" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="750" />
    <rect class="medium start" width="100" height="250" x="125" y="750" />
    <rect class="large start" width="100" height="500" x="240" y="750" />
    <text y="625" x="5">cover</text>
    <text y="520" x="360">100%</text>
    <line x1="0" x2="350" y1="500" y2="500" />
    <line x1="0" x2="350" y1="750" y2="750" />
    <text y="760" x="360">0%</text>
  </svg>
</div>
```

The image demonstrates the animation timeline. The location of the element when it reaches the start of the animation attachment range's `0%` mark is shown as a yellow block. This represents the element position when the `from` keyframe is applied. The red block represents the location of the subject element relative to the scrollport when the `to` keyframe is applied. This is the position of the animated element when it reaches the end of the animation.

Depending on the size of the subject element and the chosen timeline range, the start and end positions may overlap. Areas of overlap (as seen in subsequent diagrams) will be represented by striped red and yellow areas.

Contain

The `contain` named timeline range fully *contains* the animation within the scrollport — the range starts when the animated element is 100% visible and ends when it ceases to be fully visible, if it can be fully visible.

The `contain` value represents the range during which the principal box is either fully contained by or fully covers its view progress visibility range within the scrollport, depending on whether the subject element is smaller (can be fully contained by) or larger than the scrollport.

- If the element is smaller than the scrollport in the scroll direction, `0%` occurs when the animated element's end border edge aligns with the end edge of the scrollport, and `100%` occurs when the animated element's start border edge aligns with the start edge of the scrollport. In other words, the `contain` value ranges from the point where the subject element is first completely contained by the scroll port (`0%`), to the point where it is no longer completely contained by the scroll port (`100%`).
- If the element is larger than the scrollport, the `0%` progress is when the animated element's start border edge reaches the start edge of the scrollport, and `100%` is when the end border edge aligns with the end edge. In other words, if the animation is larger than its container, the animated element is never fully visible as it is not "contained" within the scrollport. The animation starts when the start edge reaches the start edge of the scrollport, and ends when the animated element's end edge reaches the end edge of the container.
- If the animated element is the same size as its container, the animation still happens, but over `0px`, which is not visible to the user.

```
.animated_element {
  animation-range: contain;
}
```

```
body::before {
  content: "Contain";
}
```

In this example, the original height of the animated element is 20% of the height of the scrollport, and therefore can be completely contained within it. As a result, as explained earlier, the animation should start as soon as the element starts to enter the scrollport, and finish as soon as it begins to leave the scrollport.

If you check the `500px` radio button, the subject becomes twice as tall as the scrollport. The animation range begins when the subject element first completely covers the scroll port, with the `0%` occurring when the start edge reaches the start edge of the container. It ends when the element no longer completely covers the scrollport, with the `100%` occurring when the end edge crosses the container's end edge.

When the subject is the same size as the scroll container, as occurs when `250px` is selected, the animation still occurs, but over `0px`. As the `0%` and `100%` occur at the same time, the animation is instantaneous. The difference in styling is only perceivable because the properties defined in the `100%` keyframe state are applied after the animation ends. This is because the `animation-fill-mode` property is set to `forwards`. Otherwise, the middle-sized `250px` subject, being the same height as the scroll container, would appear not to animate at all.

```
<div>
  <svg viewBox="-1 -1 462 1000" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="500" />
    <rect class="medium end" width="100" height="250" x="120" y="500" />
    <rect class="large end" width="100" height="500" x="230" y="500" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="700" />
    <rect class="medium start" width="100" height="250" x="120" y="500" />
    <rect class="large start" width="100" height="500" x="230" y="250" />
    <rect width="100" height="250" x="120" y="500" fill="url(#g)" />
    <rect width="100" height="250" x="230" y="500" fill="url(#g)" />
    <text y="460" x="10">contain</text>
    <text y="520" x="360">100%</text>
    <line x1="0" x2="350" y1="500" y2="500" />
    <line x1="0" x2="350" y1="750" y2="750" />
    <text y="760" x="360">0%</text>
  </svg>
</div>
```

```
body div,
body svg {
  margin: -60px 0 -300px;
}
```

The animation occurs when the element is within the areas represented by the white of the container in the `50px` example and by the red/yellow areas in the `250px` and `500px` examples.

It may be helpful for some to compare and contrast the `cover` and `contain` values. We can use the shorthand `animation-range` property to set the `animation-range-start` and `animation-range-end` properties to the same `<animation-name-range>` value:

```
#A {
  animation-range: contain;
}
#B {
  animation-range: cover;
}
```

```
body::before {
  content: "contain vs cover";
}
#A,
#B {
  width: 140px;
}
#A::after {
  content: " ( contain )";
}
#B::after {
  content: " ( cover ) ";
}
```

Select different radio buttons and scroll the scrollport to compare the effects of the `cover` and `contain` values on animation timelines.

Let's look at the other `<timeline-range-name>` keyword values!

Entry and exit

To make the entire animation happen only when the subject is in the process of entering or exiting the scrollport, use the `entry` or `exit` values, respectively. With these two values, the animation attachment range is based on the size of the animated element, not the size of the scrollport.

Entry

With `entry`, `0%` progress occurs the moment the animated element starts to enter the scrollport, when the subject's start edge crosses the scrollport's end edge.

The entire animation occurs as the subject comes into view, ending when it becomes completely visible or when it reaches the start edge; whichever occurs first. If the animated element is smaller than the scrollport, the animation attachment range is the size of the subject.

The position at `0%` progress is shown in yellow. The position at `100%` progress is displayed in red. When the animated element is larger than the scrollport, these two positions overlap, which is denoted by a striped background.

With `entry`, the animation attachment range is either the size of the animated element or the size of the container, whichever is smaller. If the subject is larger than the scrollport, the animation attachment range is the entire scrollport. With `entry`, the `100%` occurs when the subject's end edge crosses the end edge of the scrollport or, if the animated element is larger than the scrollport in the scroll direction, when the animated element's start edge reaches the start edge of the scrollport.

Setting `animation-range-start: entry` is the same as setting `animation-range-start: cover`. Setting `animation-range-end: entry` is equivalent to setting `animation-range-end: contain`.

```
<div>
  <svg viewBox="-60 400 530 980" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="700" />
    <rect class="medium end" width="100" height="250" x="120" y="500" />
    <rect class="large end" width="100" height="500" x="230" y="500" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="750" />
    <rect class="medium start" width="100" height="250" x="120" y="750" />
    <rect class="large start" width="100" height="500" x="230" y="750" />
    <rect width="100" height="250" x="230" y="750" fill="url(#g)" />
    <text y="460" x="10">entry</text>
    <text y="520" x="360">100%</text>
    <text y="690" x="-50">100%</text>
    <line x1="130" x2="350" y1="500" y2="500" />
    <line x1="-10" x2="120" y1="700" y2="700" />
    <line x1="0" x2="350" y1="750" y2="750" />
    <text y="760" x="360">0%</text>
  </svg>
</div>
```

Exit

The `exit` value is the inverse of `entry`. With `exit`, the attachment range starts when the animated element's start edge crosses the scrollport's start edge.

```
<div>
  <svg viewBox="-60 -1 530 800" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="450" />
    <rect class="medium end" width="100" height="250" x="120" y="250" />
    <rect class="large end" width="100" height="500" x="230" y="0" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="501" />
    <rect class="medium start" width="100" height="250" x="120" y="500" />
    <rect class="large start" width="100" height="500" x="230" y="250" />
    <rect width="100" height="250" x="230" y="250" fill="url(#g)" />
    <text y="600" x="10">exit</text>
    <text y="520" x="370">100%</text>
    <line x1="-5" x2="365" y1="500" y2="500" />
    <line x1="-10" x2="140" y1="550" y2="550" />
    <line x1="110" x2="360" y1="750" y2="750" />
    <text y="760" x="370">0%</text>
    <text y="560" x="-60">0%</text>
  </svg>
</div>
```

```
:root body div {
  margin: 0 0 -140px 0;
}
```

The `0%` progress occurs when the subject's start edge reaches the scrollport's start edge. The `100%` progress occurs when the subject's end edge crosses the start edge. Setting `animation-range-start: exit 0%` is equivalent to setting `animation-range-start: contain 100%`. Setting `animation-range-end: exit 100%` is equivalent to `animation-range-end: cover 100%`.

Comparing entry and exit

It may help to take a look at `entry` and `exit` side-by-side to understand the effect the size of the subject has on the animation attachment range: with these two values, the animation attachment range is never larger than the container.

```
#A {
  animation-range: entry;
}
#B {
  animation-range: exit;
}
```

```
body::before {
  content: "Entry and exit";
}
#A,
#B {
  width: 140px;
}
#A::after {
  content: " ( entry )";
}
#B::after {
  content: " ( exit ) ";
}
```

Scroll the viewbox to see the range of the `entry` and `exit` values, then change the size of the subjects using the radio buttons and scroll again.

When the subjects are small enough to be fully contained within the scrollport, the animation attachment timeline is at the start (`entry`) or end (`exit`) of the scrollport and the size of the attachment range is limited to the size of the animated element in the scroll direction.

With `entry` and `exit`, the animation range is contained to the size of the scrollport, while the element is not.

If the subject element is the size of the scrollport or larger:

- In the case of `entry`, the animation ends when the element fully covers the scrollport in the scroll direction.
- In the case of `exit`, the animation doesn't begin until, the element fully covers the scrollport in the scroll direction.

This may not be the effect you want. If you want an entry animation to keep running until the entire element has finished entering the start edge of the scrollport, or if you want an exit animation to start as soon as the element starts to exit the end edge of the scrollport, you need to use `entry-crossing` and `exit-crossing`.

Entry- and exit-crossing

If the subject element is smaller than the scrollport, and you want the full animation to occur as it enters or exits the scrollport, you can happily use `entry` or `exit`, respectively.

When the subject is larger than the scrollport, the animation doesn't run the full course of the element entering or exiting the scrollport. The `entry` value sets the `100%` progress to be when the element's start edge reaches the start-edge of the scrollport, before the element has fully entered the scrollport. With `exit`, the `0%` progress occurs when the element's end-edge reaches the scroll container's end edge, when part of the subject has already exited the scrollport. With both these values, the animation attachment range is smaller than the subject. If this isn't the effect you want, the `*-crossing` values may be the solution you seek.

Entry-crossing

The `entry-crossing` value represents the range during which the animated element crosses the end edge of the scrollport, with `0%` progress occurring when the element's start edge aligns with the end edge the scrollport and `100%` progress occurring when the element's end edge reaches the end edge of the scrollport, meaning it has finished fully scrolling into the scrollport.

It may be helpful to compare the two values in a live example.

```
#A {
  animation-range: entry;
}
#B {
  animation-range: entry-crossing;
}
```

```
body::before {
  content: "entry vs. entry-crossing";
}
#A::after {
  content: " ( entry )";
}
#B::after {
  content: " ( entry-crossing ) ";
}
```

Select the different height values, then scroll to compare `entry` with `entry-crossing` for each size.

Note how the effects are similar, except for when the `500px` is selected and the animated element is taller than the container. The `entry-crossing` range produces the same results as `entry` when the element is equal to or smaller than the scrollport, but when the element is larger than the scrollport, the `100%` occurs later, occurring only when the end edge has entered the view port. The animation attachment range is the size of the subject, and not clamped to the size of the scrollport.

```
<div>
  <svg viewBox="-60 -1 530 1252" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="700" />
    <rect class="medium end" width="100" height="250" x="120" y="500" />
    <rect class="large end" width="100" height="500" x="230" y="500" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="750" />
    <rect class="medium start" width="100" height="250" x="120" y="750" />
    <rect class="large start" width="100" height="500" x="230" y="750" />
    <rect
      class="orange"
      width="100"
      height="250"
      x="230"
      y="750"
      fill="url(#g)" />
    <text y="460" x="10">entry</text>
    <text y="520" x="360">100%</text>
    <text y="690" x="-50">100%</text>
    <line x1="130" x2="350" y1="500" y2="500" />
    <line x1="-10" x2="120" y1="700" y2="700" />
    <line x1="0" x2="350" y1="750" y2="750" />
    <text y="760" x="360">0%</text>
  </svg>

  <svg viewBox="-60 -1 530 1252" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="700" />
    <rect class="medium end" width="100" height="250" x="120" y="500" />
    <rect class="large end" width="100" height="500" x="230" y="250" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="750" />
    <rect class="medium start" width="100" height="250" x="120" y="750" />
    <rect class="large start" width="100" height="500" x="230" y="750" />
    <text y="200" x="0">entry-crossing</text>
    <text y="260" x="330">100%</text>
    <text y="490" x="80">100%</text>
    <text y="690" x="-50">100%</text>
    <line x1="225" x2="345" y1="250" y2="250" />
    <line x1="0" x2="230" y1="500" y2="500" />
    <line x1="-10" x2="120" y1="700" y2="700" />
    <line x1="0" x2="350" y1="750" y2="750" />
    <text y="760" x="360">0%</text>
  </svg>
</div>
```

```
:root body div {
  margin: -70px 0 -100px;
}
```

Exit-crossing

The `exit-crossing` value represents the range during which the animated element crosses the start edge of the scrollport, with `0%` progress occurring when the element's start edge aligns with the start edge the scrollport and `100%` progress occurring when the element's end edge reaches the start edge of the scrollport.

With `exit-crossing`, the animation starts as soon as the subject covers the scrollport (with `exit`, the animation doesn't start until the end edge of the subject enters into view). In both cases, the animation continues until the subject fully exits the scrollport's start edge.

```
#A {
  animation-range: exit;
}
#B {
  animation-range: exit-crossing;
}
```

```
body::before {
  content: "exit vs. exit-crossing";
}
#A::after {
  content: " ( exit )";
}
#B::after {
  content: " ( exit-crossing ) ";
}
```

The `exit-crossing` produces the same results as `exit` when the element is equal to or smaller than the scrollport, but when the element is larger than the scrollport, the `0%` occurs earlier, occurring as soon as the element's start edge reaches the scrollport's start edge, rather than waiting until the element's end edge enters the scrollport.

Like with `entry-crossing`, the animation attachment range is the size of the subject, and not clamped to the size of the scrollport.

```
<div>
  <svg viewBox="-60 -1 530 1052" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="450" />
    <rect class="medium end" width="100" height="250" x="120" y="250" />
    <rect class="large end" width="100" height="500" x="230" y="0" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="501" />
    <rect class="medium start" width="100" height="250" x="120" y="500" />
    <rect class="large start" width="100" height="500" x="230" y="250" />
    <rect width="100" height="250" x="230" y="250" fill="url(#g)" />
    <text y="800" x="-10">exit</text>
    <text y="520" x="370">100%</text>
    <line x1="-5" x2="365" y1="500" y2="500" />
    <line x1="-10" x2="140" y1="550" y2="550" />
    <line x1="110" x2="360" y1="750" y2="750" />
    <text y="760" x="370">0%</text>
    <text y="560" x="-60">0%</text>
  </svg>
  <svg viewBox="-60 -1 530 1052" xmlns="http://www.w3.org/2000/svg">
    <rect class="small end" width="100" height="50" x="10" y="450" />
    <rect class="medium end" width="100" height="250" x="120" y="250" />
    <rect class="large end" width="100" height="500" x="230" y="0" />
    <rect class="container" width="350" height="250" x="0" y="500" />
    <rect class="small start" width="100" height="50" x="10" y="501" />
    <rect class="medium start" width="100" height="250" x="120" y="500" />
    <rect class="large start" width="100" height="500" x="230" y="500" />
    <text y="800" x="-10">exit-crossing</text>
    <text y="520" x="370">100%</text>
    <line x1="-5" x2="365" y1="500" y2="500" />
    <line x1="-10" x2="140" y1="550" y2="550" />
    <line x1="110" x2="360" y1="750" y2="750" />
    <text y="760" x="370">0%</text>
    <text y="560" x="-60">0%</text>
  </svg>
</div>
```

```
:root body div {
  margin-bottom: -300px;
}
```

```
<svg id="gradient">
  <defs>
    <linearGradient
      id="g"
      x1="0"
      y1="0"
      x2="20"
      y2="20"
      spreadMethod="repeat"
      gradientUnits="userSpaceOnUse">
      <stop offset="50%" stop-color="red" />
      <stop offset="50%" stop-color="yellow" />
    </linearGradient>
  </defs>
</svg>
```

```
body::before {
  display: block;
  text-align: center;
  font-family: sans-serif;
  font-size: 1.5rem;
}
div {
  display: flex;
  gap: 20px;
}
svg {
  width: 260px;
}
#gradient {
```

```
  height: 1px;
  position: absolute;
  width: 1px;
}
rect {
  stroke: black;
  stroke-width: 3;
}
.start {
  fill: yellow;
}
.end {
  fill: red;
}
.container {
  fill: none;
}
text {
  font: 40px monospace;
  fill: black;
}
line {
  stroke: black;
  stroke-width: 8;
  stroke-dasharray: 10;
}
```

Other progress values

In this guide, we have mentioned `0%` and `100%` progress points. The animation-range values aren't limited to these to progress points. You can inset any of the named animation ranges by a set amount or a percentage of the full animation attachment range. This is discussed in the insets guide.

See also

- `<timeline-range-name>` data type
- Keyframe selectors
- Scroll-driven animation timelines
- Understanding timeline insets
- Scroll-driven animation module
- CSS animations module
- Web Animations API

Scroll-driven animation timelines

A common UI pattern involves elements that animate as the user scrolls vertically or horizontally across a page. These *scroll-driven animations* occur in direct response to page scrolling or an overflowing scroll container within a page.

The properties defined in the CSS scroll-driven animations module expand upon CSS animations by enabling animating property values defined in `@keyframes` animations in response to user interaction.

This guide provides an overview of using CSS to create scroll-driven animation timelines and animations.

What is scroll-driven animation?

The CSS scroll-driven animations module defines properties that enable CSS keyframe animations to be linked to scrolling.

Timeline progression

Animations can be set to progress along a *scroll-based timeline* instead of the default time-based document timeline, without needing JavaScript. CSS enables us to define which animation timeline to use, including animating elements by scrolling a scrollable element rather than by the passing of time.

Performance benefits

CSS scroll-driven animations are performant. JavaScript scroll-driven animations require `scroll` event listeners and `IntersectionObserver` objects on the main thread to track elements across the scrollport. Any time you rely on the main thread to render effects with JavaScript, you run the risk of blocking the main thread, which can lead to an unresponsive page and a bad user experience, or jank.

Foundations

Scroll-driven animations build upon CSS animations and the Web Animations API. Before creating scroll-driven animations, you must have an understanding of CSS `@keyframes` animations. See the using CSS animations guide to learn more.

In CSS, animations are created by attaching keyframe animations to an element using the `animation-name` property (or `animation` shorthand). By default, animations run on the default document timeline, moving from the `from` keyframe to the `to` keyframe as time passes by, with the animation lasting as long as the time defined by the `animation-duration` property value. When set to run on the default document timeline, animations play through to completion unless prevented from doing so, for example, by having the `animation-play-state` set to `paused` or by removing the `animation-name` from the element.

Scroll-driven animations are CSS animations that are not run on the default DocumentTimeline. Instead, they run on a scroll-progress or view-progress timeline, which is driven by the scrolling of an element's contents. There's a direct link between the user's scrolling action and the animation's progress along the `@keyframe` keyframes. As the user scrolls up, down, left, or right, the animation moves forward or backward through the keyframe progression. When scrolling is paused, the animation pauses, as if `animation-play-state` were set to `pause`.

Animation timelines

The `animation-timeline` property, defined in the CSS animations module, is used to set the timeline used for the animation.

The CSS scroll-driven animations module defines features for setting the `animation-timeline` as a scroll-progress or view-progress timeline. You can explicitly name an element as a timeline controller using the `scroll-timeline-*` and `view-timeline-*` properties, then set that name as the `animation-timeline` of a descendant element. You can also define *anonymous scroll progress timelines* and *anonymous view progress timelines* using the `scroll()` and `view()` functions.

Alternatively, the `animation-timeline` property can be used to explicitly state that the default document timeline be used or to specify that the animation doesn't have a timeline, and therefore shouldn't occur at all.

Regular CSS animations: default document timeline

Setting `animation-timeline` explicitly to `auto`, or omitting the property and allowing it to default to `auto`, sets the timeline to be the default document timeline. When set to this default value, the animation's progress is determined by the `animation-duration`, the `animation-delay`, and how much time has passed since the animation was associated with the element via the `animation-name` property. The time-based document timeline is the timeline traditionally associated with CSS animations.

```
:checked ~ .container > .item {
  animation-name: action;
  animation-duration: 3s;
  animation-delay: 500ms;
```

```
  animation-timeline: auto;
}
```

We create a rotation keyframe animation called `action`:

```
@keyframes action {
  from {
    rotate: 45deg;
  }
  to {
    rotate: 765deg;
  }
}
```

```
<input type="checkbox" id="i" />
<label for="i">
  Check to apply the animation. Uncheck to remove the animation
</label>
<div class="container">
  <span class="item"></span>
</div>
```

```
div {
  width: 400px;
  height: 100px;
  border: 1px solid;
  background-color: palegoldenrod;
  position: relative;
}
span {
  --size: 50px;
  height: var(--size);
  width: var(--size);
  background-color: magenta;
  border: 1px solid;
  position: absolute;
  left: calc(50% - (var(--size) / 2));
  top: calc(50% - (var(--size) / 2));
}
```

When the checkbox is checked, the `action` animation is applied to the element. When unchecked, the animation is not applied to the `<div>`.

Try checking the checkbox. Nothing will happen during the half-second animation delay. Then, once the animation starts, the box will jump to a 45-degree rotation, and then it will take 3 seconds to rotate an additional 720 degrees, or two additional full rotations. After a total of three and a half seconds, the animation concludes, and the `<div>` will return to its default non-rotated state.

> **Note:** The `animation-timeline` is reset to the default `auto` value by the `animation` shorthand property, but cannot be set using the shorthand. Therefore, when creating scroll-driven animations, always declare the `animation-timeline` after any `animation` shorthand declarations to achieve the desired effect.

Scroll progress timelines

With *scroll progress timeline*, the timeline progresses based on the scrolling of the scrollable element (*scroller*) from top to bottom (or left to right) and back again. By default, the position in the scroll range is converted into a percentage of progress — `0%` at the start and `100%` at the end.

To create a scroll progress timeline, the `animation-timeline` value must reference the scroller, which can be named or anonymous.

Named scroll progress timelines

A *named scroll progress timeline* is one where the scroller is explicitly named using the `scroll-timeline-name` property (or the `scroll-timeline` shorthand). The name is a `<dashed-ident>`. The scroller is linked to the element to be animated by specifying its `scroll-timeline-name` as the value of that element's `animation-timeline` property.

Our HTML includes three elements: the `item`, which we will animate; its `container`, which we will scroll; and the scroller. The `container` needs to be large enough to overflow its `scroller` parent: If there is no scrolling, there will be no scroll timeline.

```
<main class="scroller">
  <div class="container">
    <span class="item"></span>
  </div>
</main>
```

We provide some basic styles. The important ones include setting a height on the container that is taller than the scroller, and then setting the overflow to allow scrolling:

```
.scroller {
  width: 400px;
  height: 100px;
  overflow: scroll;
}
.container {
  height: 200px;
}
```

Setting an `animation-timeline` on the animated element that matches the `scroll-timeline-name` of an ancestor element is what creates the named scroll progress timeline. We also have to include an animation, which we do by setting the value of the `animation-name` component of the `animation` shorthand to the `<custom-ident>` name of our keyframe animation:

```
.scroller {
  scroll-timeline-name: --rotate;
}
.item {
  animation: action 1ms linear;
  animation-timeline: --rotate;
}
```

```
main {
  border: 1px solid;
  background-color: palegoldenrod;
}
div {
```

```
  position: relative;
}
span {
  --size: 50px;
  height: var(--size);
  width: var(--size);
  background-color: magenta;
  border: 1px solid;
  position: absolute;
  left: calc(50% - (var(--size) / 2));
  top: calc(50% - (var(--size) / 2));
}
```

In this case, we don't have a checkbox, as the `action` animation progression is controlled by the scrolling of the overflowing scroller, which, unlike time, does not expire.

Before any scrolling occurs, the container's position is at the top of the scroller, and the animation is at the 0% keyframe. Try scrolling down. As you scroll, the animation progresses through the timeline, rotating an additional 720 degrees. When you can no longer scroll, the animation's progression is at the 100%, or `to`, keyframe. The animated item doesn't return to its default rotation unless the scroller is scrolled back up to the top.

Animation duration

You may have noticed that the `animation-duration` component of the `animation` shorthand was set to `1ms`. When creating CSS scroll-driven animations, specifying an `animation-duration` value does not affect the duration of the animation, and shouldn't be necessary. However, durations can affect non-linear view progress animation timelines, and Firefox requires a non-zero `animation-duration` to apply an animation to an element. For these reasons, it is common practice to set `animation-duration` to `1ms`.

Setting `animation-duration: 1ms` ensures that the animation works in Firefox, the animation effect is consistent across all browsers, and the animation is hidden if a browser doesn't support view progress animation timelines. If the browser supports keyframe animations, the animation will not be visible to the user. However, the animation still happens, and animation events are fired.

Anonymous scroll progress timelines

You don't have to name your scroll progress timeline. Instead, you can associate an *anonymous scroll progress timeline* with the animation. In this case, the `animation-timeline` of the element to animate is set to a `scroll()` function. The function selects the scroller that provides the scroll progress timeline and the scroll axis to use based on the optional arguments you pass to it. One parameter is a `<scroller>` keyword defining the relationship of the scroller element to the current element (`nearest`, `root`, or `self`). The other is the scrollbar `<axis>` value (`block`, `inline`, `y`, or `x`).

This example uses all the same CSS as the previous example, except for the `animation-timeline`, which we set to a `scroll()` function. We are also overriding the size of the container to change the direction of the scroll:

```
.item {
  animation: action 1ms linear;
  animation-timeline: scroll(nearest inline);
}
.container {
  inline-size: 800px;
  block-size: 100%;
}
```

We set an `inline-size` on the container so that it overflows in the inline direction, and set the `block-size` to `100%` so it no longer overflows in the block direction. Try scrolling in the inline direction.

View progress timelines

You can also progress an animation based on the change in visibility of an element inside a scroller — this is done via *view progress timelines*. Instead of tracking the scroll offset of a scroll container, view progress timelines track the relative position of an element, called the *subject*, within a scrollport. The progression of an animation's keyframes is based on the *visibility* of the subject inside the scroller. Unlike scroll progress timelines, with view progress timelines, you can't specify the scroller — the subject's visibility is always tracked within its nearest ancestor scroller.

A view progress timeline animation only occurs when the element is visible within its scrollport. Timeline progress starts at `0%` when the tracked subject starts intersecting the scrollport at the block or inline end edge. The `100%` occurs when the subject exits the scrollport at the block or inline start edge.

Because the `100%` is generally reached when the element leaves the viewport, you likely want to set the final effect of your animation in a keyframe block that occurs well before the end of the animation. You can set your completed effect within the `20%`, `50%`, or `80%` keyframe block rather than using the `to` or `100%` keyframe to ensure the element finishes animating while still in view.

With view progress timelines, you can adjust the view progress visibility range. Use `view-timeline-inset`, part of the `view-timeline` shorthand, to adjust when the subject is considered to be in view. The default value is `auto`. The effect of any non-`auto` inset value is as if you moved the edges of the scroll port: a positive inset value creates an inward adjustment, and a negative value creates an outward adjustment.

Similar to scroll progress timelines, the view progress timeline can be named or anonymous.

Named view progress timeline

A *named view progress timeline* is one where the subject is explicitly named using the `view-timeline-name` property, a component of the `view-timeline` shorthand. The `<dashed-ident>` name is then linked to the element to animate by specifying it as the value of that element's `animation-timeline` property.

With named view progress timelines, the element to animate does not have to be the same as the subject. In other words, the element controlling the timeline doesn't have to be the same as the element being animated. This means you can animate one element based on another element's movement within its scrollable container.

Here we use the `view-timeline-name` property to name an element, identifying the element itself as the source of a view progress timeline. We then set that name as the value of the `animation-timeline` property.

```
.item {
  animation: action 1ms linear;

  view-timeline-name: --a-name;
  animation-timeline: --a-name;
}
```

We applied the animation **before** the animation timeline, as the `animation` resets the `animation-timeline` to `auto`.

The animation is slightly different from the previous examples in that the spinning effect starts at `20%` and ends at `80%` of the way through the animation; this means the element will not be actively spinning when it first comes into view and will stop spinning before it is completely out of view.

```
@keyframes action {
  0%,
  20% {
    rotate: 45deg;
  }
  80%,
  100% {
    rotate: 720deg;
  }
}
```

```
.scroller {
  width: 400px;
  height: 200px;
  line-height: 2;
  overflow: scroll;
  border: 1px solid;
  background-color: palegoldenrod;
}
.item {
  --size: 50px;
  height: var(--size);
  width: var(--size);
  background-color: magenta;
  border: 1px solid;
  left: calc(50% - (var(--size) / 2));
  top: calc(50% - (var(--size) / 2));
}
```

```
<main class="scroller">
  <p>Scroll down to view the animation</p>
  <p> </p>
  <p> </p>
  <p> </p>
  <div class="item"></div>
  <p> </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <p>Scroll up to view the animation</p>
</main>
```

Scroll the element into view. Note that the element animates through the `@keyframes` animation as it moves through the visible area of its ancestor scroller.

Anonymous view progress timeline: the <code>view()</code> function

Alternatively, a `view()` function can be set as the value of the `animation-timeline` property to specify that an element's animation timeline is an *anonymous view progress timeline*. This causes the element to be animated based on its position inside its nearest parent scroller.

The `view()` function creates a view timeline. You attach the timeline to the element you want to animate using the `animation-timeline` property. The function creates a view timeline for each element matched by the selector.

In this example, we again define the `animation` before the `animation-timeline`, so the timeline is not reset. We then include an argument-less `view()` function. We don't specify a scroller, as, by definition, the subject's visibility is tracked by its nearest ancestor scroller.

```
.item {
  animation: action 1ms linear;
  animation-timeline: view();
}
```

Parameters of the <code>view()</code> function

The `view()` function takes up to three optional values as arguments:

- Zero or one `<axis>` parameters. If set, this specifies the scroll axis along which the animation progresses.
- Either the keyword `auto` or zero, one, or two `<length-percentage>` inset values. If set, these values specify offsets for the scrollport start and/or end.

Declaring `view()` is equivalent to `view(block auto)`, which defines `block` as the axis of the parent element that supplies the timeline and the `scroll-padding`, which generally defaults to `0`, as the insets within the visible area at which the animation starts and ends.

The function sets the values of the `view-timeline-axis` and `view-timeline-inset` properties.

The `view-timeline-inset` arguments specify insets (if positive) or outsets (if negative) that adjust the start and end of the scrollport. They are used to determine the scroll positions at which the element is considered "in view", which determines the length of the animation timeline. In other words, instead of starting at the start edge and ending at the end edge of the scrollport, the animation occurs at the start and end of the inset-adjusted view.

Unlike the scroll timeline's `scroll()` function, there is no `<scroller>` argument in the `view()` function, as the view timeline always tracks the subject within its nearest ancestor scroll container.

In this example, as we are using inset values, we can use the `from` and `to` keyframe selectors.

```
@keyframes action {
  from {
    rotate: 45deg;
  }
  to {
    rotate: 720deg;
  }
}

.item {
  animation: action 1ms linear;
  animation-timeline: view(block 20% 20%);
}
```

Accessibility concerns

As with all animations and transitions, always take any user's `prefers-reduced-motion` preference into account.

Removing an animation's timeline

Setting `animation-timeline: none` disassociates the element from all animation timelines, including the default time-based document timeline, meaning the element will not animate. While some animations may be necessary, you can remove animations based on the user's `prefers-reduced-motion` setting with:

```
@media (prefers-reduced-motion: reduce) {
  .optionalAnimations {
    animation-timeline: none;
  }
}
```

Because the `animation` shorthand sets the `animation-timeline` to `auto`, use a selector with enough specificity to ensure your `animation-timeline` isn't overridden by your `animation` shorthand declarations.

See also

- Understanding timeline range names
- Scroll-driven animations module
- CSS animations module
- Web Animations API

CSS scrollbars styling

The **CSS scrollbars styling** module defines properties that you can use for visual styling of scrollbars. You can customize the scrollbar width as needed. You can also customize the scrollbar *track* color (the background) and the scrollbar *thumb* color (the draggable handle).

Scrollbar styling in action

This example defines a thin scrollbar with a teal thumb and lighter teal track. To see the thumb, scroll the text. After the scrollbar is visible, hover over it to see the track.

```
.poem {
  width: 300px;
  height: 100px;
  border: 1px solid;
}
```

```
.poem {
  overflow: scroll;
  scrollbar-color: #0b4f4a #5f9e98;
  scrollbar-width: thin;
}
```

```
<blockquote class="poem">
  <h3>A Small Needful Fact</h3>
  <pre>
Is that Eric Garner worked
for some time for the Parks and Rec.
Horticultural Department, which means,
perhaps, that with his very large hands,
perhaps, in all likelihood,
he put gently into the earth
some plants which, most likely,
some of them, in all likelihood,
continue to grow, continue
to do what such plants do, like house
and feed small and necessary creatures,
like being pleasant to touch and smell,
like converting sunlight
into food, like making it easier
for us to breathe.
</pre>
  <p>
    - <a href="https://onbeing.org/poetry/a-small-needful-fact/">Ross Gay</a>
  </p>
</blockquote>
```

> **Note:** When customizing scrollbars, ensure that the thumb and track have enough contrast with the surrounding background. Also ensure that the scrollbar hit area is large enough for people who use touch input.

Reference

CSS properties

- `scrollbar-width`
- `scrollbar-color`

Related concepts

- `overflow-block` CSS property
- `overflow-inline` CSS property
- `overflow-x` CSS property
- `overflow-y` CSS property
- `overflow` CSS shorthand property
- `overflow-clip-margin` CSS property
- `scrollbar-gutter` CSS property
- `scroll-behavior` CSS property
- `scroll-margin` CSS shorthand property
- `scroll-padding` CSS shorthand property
- `scroll-snap-align` CSS property
- `scroll-snap-stop` CSS property
- `scroll-snap-type` CSS property
- `::-webkit-scrollbar` pseudo-element
- scroll container glossary term
- `scrollbar` ARIA role

See also

- `scroll-timeline`, `scroll-timeline-axis`, `scroll-timeline-name`
- CSS overflow module
- CSS scroll snap module

CSS selectors

The **CSS selectors** module defines the patterns to select elements to which a set of CSS rules are then applied along with their specificity. The CSS selectors module provides us with more than 60 selectors and five combinators. Other modules provide additional pseudo-class selectors and pseudo-elements.

> **Note:** This page introduces a CSS module. To find an exhaustive list of all selectors defined by CSS specifications, see the selectors reference page.

In CSS, selectors are patterns used to match, or select, the elements you want to style. Selectors are also used in JavaScript to enable selecting the DOM nodes to return as a `NodeList`.

Selectors, whether used in CSS or JavaScript, enable targeting HTML elements based on their type, attributes, current states, and even position in the DOM. Combinators allow you to be more precise when selecting elements by enabling selecting elements based on their relationship to other elements.

Reference

Combinators and separators

- `+` (Next-sibling combinator)
- `>` (Child combinator)
- `~` (Subsequent sibling combinator)
- `" "` (Descendant combinator)
- `|` (Namespace separator)
- `,` (Selector list)

The CSS selectors module also introduces the column combinator (`||`). Currently, no browsers support this feature.

Selectors

- `:active`
- `:any-link`
- `:autofill`
- `:buffering`
- `:checked`
- `:default`
- `:defined`
- `:dir()`
- `:disabled`
- `:empty`
- `:enabled`
- `:first-child`
- `:first-of-type`
- `:focus`
- `:focus-visible`
- `:focus-within`
- `:fullscreen`
- `:future`
- `:has()`
- `:hover`
- `:in-range`
- `:indeterminate`
- `:interest-source`
- `:interest-target`
- `:invalid`
- `:is()`
- `:lang()`
- `:last-child`
- `:last-of-type`
- `:link`
- `:matches()` (obsolete legacy selector alias for `:is()`)
- `:modal`
- `:muted`
- `:not()`
- `:nth-child()`
- `:nth-of-type()`
- `:nth-last-child()`
- `:nth-last-of-type()`
- `:only-child`
- `:only-of-type`
- `:open`
- `:optional`
- `:out-of-range`
- `:past`
- `:paused`
- `:picture-in-picture`
- `:placeholder-shown`
- `:playing`
- `:popover-open`
- `:read-only`
- `:read-write`
- `:required`
- `:root`
- `:scope`
- `:seeking`
- `:stalled`
- `:target`
- `:user-invalid`
- `:user-valid`
- `:valid`
- `:visited`
- `:volume-locked`
- `:where()`
- `:-webkit-` pseudo-classes

- Attribute selectors
- Class selector
- ID selectors
- Type selectors
- Universal selectors

The CSS selectors module also introduces the `:blank`, `:current`, and `:local-link` pseudo-classes. Currently, no browsers support these features.

Terms

- Pseudo-class glossary term
- Functional pseudo-classes
- Combinators
- Simple selector
- Compound selector
- Complex selector
- Relative selector
- Specificity

Guides

CSS selectors and combinators

Overview of the different types of simple selectors and various combinators defined in the CSS selectors and the CSS pseudo modules.

CSS selector structure

Explanation of the structure of CSS selectors and the terminologies introduced in the CSS selectors module, ranging from "simple selector" to "forgiving relative selector list".

Pseudo classes

Lists the pseudo-classes, selectors that allow the selection of elements based on state information that is not contained in the document tree, defined in the various CSS modules and HTML.

Using the `:target` pseudo-class in selectors

Learn how to use the `:target` pseudo-class to style the target element a URL's fragment identifier.

Privacy and the `:visited` selector

Explores the style limitations set on the `:visited` class for user privacy.

CSS building blocks: CSS selectors

Introduction to basic CSS selectors, including tutorials on Type, class, and ID selectors, Attribute selectors, Pseudo-classes and pseudo-elements, and Combinators.

Learn: UI pseudo-classes

Learn the different UI pseudo-classes available for styling forms in different states.

Selection and traversal on the DOM tree

The selectors API enables using selectors in JavaScript to retrieve element nodes from the DOM.

Related concepts

- `state()` pseudo-class
- `:xr-overlay` pseudo-class
- CSS nesting module
  - `&` nesting selector
- CSS scoping module
  - `:host` pseudo-class
  - `:host()` pseudo-class
  - `:host-context()` pseudo-class
  - `:has-slotted` pseudo-class
  - `::slotted` pseudo-element
- CSS overflow module
  - `::scroll-button()`
  - `::scroll-marker`
  - `::scroll-marker-group`
  - `:target-current`
- CSS multi-column layout module
  - `::column`
- CSS paged media module
  - `:left` pseudo-class
  - `:right` pseudo-class
  - `:first` pseudo-class
  - `:blank` pseudo-class
- CSS pseudo-element module (representing entities not included in HTML)
  - `::after`
  - `::before`
  - `::file-selector-button`
  - `::first-letter`
  - `::first-line`
  - `::grammar-error`
  - `::marker`
  - `::placeholder`

    - `::selection`
    - `::spelling-error`
    - `::target-text`
- CSS shadow parts module
    - `::part` pseudo-element
- CSS positioned layout module
    - `::backdrop`
- Other pseudo-elements
    - `::cue`
- CSS animations
    - `<keyframe-selector>`
- `@namespace` at-rule
- `!important`
- Specificity
- Cascade
- `Document.querySelector` method
- `Document.querySelectorAll` method
- `NodeList.forEach()` method

See also

- CSS pseudo-element module
- CSS cascading and inheritance module
- CSS nesting module
- Using shadow DOM

Privacy and the :visited selector

Originally, the CSS `:visited` selector was a privacy and security risk. With a little bit of JavaScript, websites could uncover a user's browsing history and figure out what sites the user had visited. This was done using methods like `window.getComputedStyle` and other techniques. This process was quick, enabling websites to not only determine where the user had been on the web, but also to guess a lot of information about the user's identity.

To mitigate this privacy concern, browsers limit the amount of information that can be obtained from visited links.

Little white lies

To preserve users' privacy, browsers lie to web applications under certain circumstances:

- The `window.getComputedStyle` method and similar functions, such as `element.querySelector`, always return values indicating that a user has never visited any of the links on a page.
- When using a sibling selector, such as `:visited + span`, the adjacent element (`span` in this example) is styled as though the link were unvisited.
- In rare scenarios, if you're using nested link elements and the element being matched is different from the link whose presence in history is being tested, the element will be rendered as though the link were unvisited.

Limits to visited link styles

You can style visited links, but there are limits to which styles you can use. Only the following styles can be applied to visited links:

- `color`
- `background-color`
- `border-color` (and its sub-properties)
- `column-rule-color`
- `outline-color`
- `text-decoration-color`
- `text-emphasis-color`
- Color parts of the `fill` and `stroke` attributes

In addition, even for the styles mentioned above, transparency differences between unvisited and visited links are not applied. This restriction prevents the use of the `alpha` parameter in various `<color>` functions or the `transparent` keyword to distinguish between the two states.

Here is an example of how to use styles with the aforementioned restrictions:

```
:link {
  outline: 1px dotted blue;
  background-color: white;
  /* The default value of `background-color` is `transparent`. You need to
     specify a different value, otherwise changes on `:visited` won't apply. */
}

:visited {
  outline-color: orange; /* Visited links have an orange outline */
  background-color: green; /* Visited links have a green background */
  color: yellow; /* Visited links have yellow colored text */
}
```

Impact on web developers

You may want to consider the following when developing sites:

- Changing `background-image` values based on a link's visited state will not work since only colors can be used to style visited links.
- Colors that are otherwise transparent will not apply when styled via a `:visited` selector.

See also

- Preventing attacks on a user's history through CSS `:visited` selectors (2010)

CSS selector structure

The CSS selector represents a particular pattern of element or elements in a tree structure. The term "selector" can refer to a simple selector, a compound selector, or a complex selector. When included in the `:has()` pseudo-class as a parameter, these selectors are referred to as relative selectors, representing elements relative to one or more anchor elements.

These selectors can be combined into a comma-separated selector list. If any selector in a non-forgiving selector list is invalid, the entire selector list is invalidated.

Simple selector

A **simple selector** is a selector with a single component, such as a single type selector, attribute selector, or pseudo-class, that's not used in combination with or contains any other selector component or combinator. A given element is said to match a simple selector when that simple selector accurately describes the element. Any selector that contains a single basic selector, attribute selector, pseudo-class, or pseudo-element selector is a simple selector.

```
#myId {
}

[pattern*="\d"] {
}
```

Compound selector

A **compound selector** is a sequence of simple selectors that are not separated by a combinator. A compound selector represents a set of simultaneous conditions on a single element. A given element is said to match a compound selector when the element matches all the simple selectors in the compound selector.

```
a#selected {
}

[type="checkbox"]:checked:focus {
}
```

In a compound selector, the type selector or universal selector must come first in the sequence of selectors. Only one type selector or universal selector is allowed in the sequence. As whitespace represents the descendant combinator, no whitespace is allowed between the simple selectors that make up a compound selector.

Complex selector

A **complex selector** is a sequence of one or more simple and/or compound selectors that are separated by combinators, including the white space descendant combinator.

A complex selector represents a set of simultaneous conditions on a set of elements.

```
a#selected > .icon {
}

.box h2 + p {
}
```

Selectors can be read from right to left. For example, `a#selected > .icon` matches all elements with a class of `icon` that are the direct children of the `<a>` element with the id `selected`. The selector `.box h2 + p` matches the first `<p>`s to come immediately after any `<h2>` elements that are descendants of any element with the class of `box`.

Selector list

A **selector list** is a comma-separated list of simple, compound, and/or complex selectors. A given element is said to match a selector list when the element matches any (at least one) of the selectors in that selector list.

```
#main,
article.heading {
}
```

If any selector in a non-forgiving selector list is invalid, the entire selector list is invalidated.

```
#main,
:bad-pseudoclass,
.validClass {
  /* `:bad-pseudoclass` is invalid, invalidating this style block */
}
```

The `:is()` and `:where()` pseudo-classes can be used to construct forgiving selector lists.

Relative selector

A **relative selector** is a selector representing an element relative to one or more anchor elements preceded by a combinator. Relative selectors that don't begin with an explicit combinator have an implied descendant combinator.

Relative selectors cannot be used in a selector list. Rather, it is accepted within certain contexts, such as the `:has()` pseudo-class.

```
:has(+ div#topic > #reference) {
}

:has(> .icon) {
}

dt:has(+ img) ~ dd {
}
```

See also

- CSS selectors and combinators
- Forgiving selector list
- `Document.querySelector()`
- `Document.querySelectorAll()`
- CSS selectors module
- CSS pseudo-elements module

CSS selectors and combinators

CSS selectors are used to define a pattern of the elements that you want to select for applying a set of CSS rules on the selected elements. Combinators define the relationship between the selectors. Using various selectors and combinators, you can precisely select and style the desired elements based on their type, attributes, state, or relationship to other elements.

Types of selectors

There are over 80 selectors and combinators. CSS selectors can be grouped into the following categories based on the type of elements they can select.

Basic selectors

The type selector selects all elements that have the given node name. For example, `div` will select all `<div>` elements and `input` will match any `<input>` element. The universal selector, denoted with an asterisk (`*`), is a special type selector that selects all elements.

The class selector selects all elements that have the given `class` attribute denoted by the class name prefixed with a period (`.`). For example, `.index` will match any element that has `class="index"`. The ID selector selects an element based on the value of its `id` attribute. The selector is the `id` prefixed with a "number sign" (U+0023, `#`). For example, `#toc` will match the element that has `id="toc"`. Both `class` and `id` are global attributes. There should be only one element with a given `id` in a document; but if there is more than one, the ID selector will match all the elements with that `id`.

When combining a type or universal selector with a class or id selector to create a compound selector, the type or universal selector must precede the class or id.

CSS

In this example, we declare four simple selectors and one compound selector using the four basic selector types, as described above.

```
* {
  font-style: italic;
}
p {
  color: red;
}
.myClass {
  text-decoration: underline;
}
#myId {
  font-family: monospace;
}
p.myClass#myId {
  font-size: 1.5rem;
}
```

HTML

```
<p class="myClass" id="myId">I match everything.</p>
<p>I match the universal and type selectors only.</p>
```

Result

Combinators

Using CSS combinators, we can combine selectors to select DOM nodes based on their relationship to other elements within the document node tree. This combining of selectors with combinators creates complex selectors.

Descendant combinator

The descendant combinator, denoted with one or more spaces, selects nodes that are descendants of the first element. For example, `div span` will match all `<span>` elements that are inside a `<div>` element.

Child combinator

The child combinator is more specific than the descendant combinator. Denoted with the greater than character (`>`), the child combinator selects nodes that are direct children of the first element. Comparing with our previous example, `div > span` will match only the `<span>` elements that are direct children of a `<div>` element.

Subsequent-sibling combinator

In addition to descendant selectors, CSS also enables selecting elements based on their siblings. The subsequent-sibling combinator, denoted with a tilde (`~`), selects siblings. Given `A ~ B`, all elements matching `B` will be selected if they are preceded by `A`, provided both `A` and `B` share the same parent. For example, `h2 ~ p` will match all `<p>` elements that follow an h2, immediately or not.

Next-sibling combinator

The next-sibling combinator, denoted by the plus symbol (`+`), is similar to the subsequent-sibling. However, given `A + B`, it only matches `B` if `B` is immediately preceded by `A`, with both sharing the same parent. Amending our previous example, `h2 + p` will match only the single `<p>` element that *immediately* follows an `<h2>` element.

Column combinator

There is also a column combinator, denoted by two pipe characters (`||`), which, when supported, selects nodes that belong to a column. For example, `col || td` will match all `<td>` elements that belong to the scope of the `<col>`.

Namespace separator

The namespace separator is another combinator that is generally used in conjunction with the `@namespace` at-rule. This combinator is denoted by a single pipe character (`|`). It enables limiting type selectors and the universal selector to a specific namespace. For example, by defining a namespace such as `@namespace SVG url('http://www.w3.org/2000/svg');`, you can include selectors that target elements nested in an SVG namespace only. Declaring `SVG|a` would match links within SVGs and not those in the rest of the document. Namespacing can be useful to target MathML, SVG, or other XML-based content within your HTML.

CSS

In this example, we declare five relative selectors using simple selectors combined with combinators.

```
h2 + p ~ p {
  font-style: italic;
}
h2 + p + p {
  color: red;
}
.myClass + p {
  text-decoration: underline;
}
#myId > .myClass {
  outline: 3px dashed red;
}
* > p {
  font-size: 1.1rem;
}
```

HTML

```
<h2 class="myClass" id="myId">
  No selectors match. <span class="myClass">This span has an outline</span> as
  it is both myClass and a child of #myId.
</h2>
<p>The first paragraph is underlined. All the paragraphs are 1.1rem.</p>
<p>
  The second paragraph is red. This and the following paragraphs are italic.
</p>
<p>The third paragraph is NOT red. It is italic and 1.1rem.</p>
<p class="myClass">
  Does not have an outline; this is a sibling of H2, not a child. It is italic
  and 1.1rem.
</p>
```

Result

Creating complex selectors with CSS nesting

The above complex selectors can also be defined using simple selectors, combinators, and CSS nesting, with or without the `&` nesting selector.

CSS

In this example, we replicate the same five relative selectors using simple selectors combined with combinators, but this time with CSS nesting.

```
h2 {
  & + p {
    & ~ p {
      font-style: italic;
    }
    & + p {
      color: red;
    }
  }
}
.myClass {
  & + p {
    text-decoration: underline;
  }
}
#myId {
  & > .myClass {
    outline: 3px dashed red;
  }
}
* {
  & > p {
    font-size: 1.1rem;
  }
}
```

HTML

```
<h2 class="myClass" id="myId">
  No selectors match. <span class="myClass">This span has an outline</span> as
  it is both myClass and a child of #myId.
</h2>
<p>The first paragraph is underlined. All the paragraphs are 1.1rem.</p>
<p>
  The second paragraph is red. This and the following paragraphs are italic.
</p>
<p>The third paragraph is NOT red. It is italic and 1.1rem.</p>
<p class="myClass">
  Does not have an outline; this is a sibling of H2, not a child. It is italic
  and 1.1rem.
</p>
```

Result

Attribute selectors

Attribute selectors select all elements that, depending on how the selector is written, either have the given attribute or have the given attribute with a substring value match. For example, `[type]` will match all elements that have the `type` attribute set (to any value), and `[type="submit"]` will match `<input type="submit">` and `<button type="submit">`, or any element with `type="submit"` set, even though this attribute-value pair is only supported on `<input>` and `<button>` elements. The match is case-insensitive.

The case sensitivity of the attribute depends on the language. Generally, in HTML, if an attribute is enumerated, the value in the selector is case-insensitive, even if the value is not one of the enumerated values or if the attribute is not a valid value for the element on which it is set. For non-enumerated attributes, like `class`, `id`, or any `data-*` attribute, or for non-HTML attributes, like `role` or `aria-*` attributes, the value match is case-sensitive; the match can be made case-insensitive with a case-insensitive modifier (`i`).

Pseudo-class selectors

The CSS selectors module defines over 60 pseudo-classes. Pseudo-classes are simple selectors, prefixed with a colon (`:`), that allow the selection of elements based on state information that is not contained in the document tree. `pseudo-classes` can be used to style an element based on its *state*. For example, the `:target` simple selector targets element of a URL containing a fragment identifier, and the `a:visited` compound selector matches all `<a>` elements that have been visited by a user.

The pseudo-classes can be categorized as element display state, input, linguistic, location, resource state, time-dimensional, tree-structural, user action, and functional.

Multiple pseudo-classes can be combined to create compound selectors. When combining a pseudo-class into a compound selector with a type or universal selector, the pseudo-class must follow the type selector or universal selector, if present.

Pseudo-element selectors

Not all CSS selectors are defined in the CSS selectors module. CSS pseudo-element selectors are defined in the CSS pseudo-elements module.

CSS pseudo-elements, prefixed with two colons (`::`), represent entities that are not included in HTML. For example, the simple `::marker` selector selects list item bullets, and the compound selector `p::first-line` matches the first line of all `<p>` elements.

See the pseudo-classes and pseudo-elements specification tables for details on those.

See also

- Selector list
- CSS selector structure
- Specificity
- CSS nesting module

Using the :target pseudo-class in selectors

When a URL points at a specific piece of a document using a URL fragment identifier, it can be difficult for the user to notice. This guide discusses using CSS to draw attention to the target of a URL to improve user experience.

Picking a Target

The pseudo-class `:target` is used to style the target element of the document identified using the URL fragment identifier. For example, the URL `https://developer.mozilla.org/en-US/docs/Web/CSS#reference` contains the fragment identifier `#reference`. In HTML, identifiers are found as the values of either `id` or `name` attributes, since the two share the same namespace. Thus, the example URL would point to the element that has the id `reference` in that document.

To style any `h2` element that is the target of a URL, while not affecting any other kind of element to get a target style, use the `:target` pseudo-class with the type selector:

```
h2:target {
  outline: 2px solid;
}
```

It's also possible to create styles that are specific to a particular fragment of the document. This is done using the same identifying value that is found in the URI. Thus, to add a background color to the `#reference` fragment, we would write:

```
#reference:target {
  background-color: yellow;
}
```

Targeting all elements

If the intent is to create a "blanket" style that will apply to all targeted elements, then the universal selector comes in handy:

```
:target {
  color: red;
}
```

Example

In the following example, there are five links that point to elements in the same document. Selecting the "First" link, for example, will cause `<h1 id="one">` to become the target element. Note that the document may jump to a new scroll position, since target elements are placed on the top of the browser window if possible.

```
<h4 id="one">…</h4>
<p id="two">…</p>
<div id="three">…</div>
<a id="four">…</a> <em id="five">…</em>

<a href="#one">First</a>
<a href="#two">Second</a>
<a href="#three">Third</a>
<a href="#four">Fourth</a>
<a href="#five">Fifth</a>
```

Conclusion

In cases where a fragment identifier points to a portion of the document, readers may become confused about which part of the document they're supposed to be reading. By styling the target of a URI, reader confusion can be reduced or eliminated.

See also

- `:target`

CSS shadow parts

The **CSS shadow parts** module defines the `::part()` pseudo-element that can be set on a shadow host. Using this pseudo-element, you can enable shadow hosts to expose the selected element in the shadow tree to the outside page for styling purposes.

By default, elements in a shadow tree can be styled only within their respective shadow roots. The CSS shadow parts module enables including a `part` attribute on `<template>` descendants that make up the custom element, exposing the shadow tree node to external styling via the `::part()` pseudo-element.

Reference

Selectors

- `::part()`

HTML attributes

- `part`
- `exportparts`

Definitions

- Shadow tree

Guides

CSS pseudo-elements

Alphabetical list of pseudo-elements defined by all the CSS specifications and WebVTT

Web components

Overview of the different APIs that enable creating reusable custom elements or web components.

Related concepts

- HTML `<template>` element
- HTML `<slot>` element
- `Element.part` property
- `Element.shadowRoot` property
- `Element.attachShadow()` method
- `ShadowRoot` interface
- CSS scoping module
  - `:host`
  - `:host()`
  - `:host-context()`
  - `::slotted`

See also

- CSS pseudo elements module
- CSS selectors module
- Using shadow DOM
- Templates: Styling outside of the current scope on web.dev (2023)

CSS shapes

The **CSS shapes** module describes geometric shapes. It also defines CSS properties that can use the shapes to control the geometry of an element's float area; this area can then be applied to exclusions, or specify an element's content area.

The specification defines several ways to create shapes. Content can be wrapped around or within a shape rather than following the default rectangle shape of the element's box.

Shapes define geometries that can be used as CSS values. This module provides functions for creating ellipses, polygons, and arbitrary geometries. Other CSS modules can make use of shapes defined in this specification, including CSS motion path and CSS masking.

CSS shapes in action

The example below shows an image that has been floated left, and the `shape-outside` property applied with a value of `circle(50%)`. This creates a circle shape, and the content wrapping the float now wraps around that shape. This changes the length of the wrapping text's line boxes. You can click "Play" to edit the code in the MDN Playground.

```
<div class="box">
  <img
    alt="A hot air balloon"
    src="https://mdn.github.io/shared-assets/images/examples/round-balloon.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
img {
  float: left;
  shape-outside: circle(50%);
}
```

Reference

Properties

- `shape-image-threshold`
- `shape-margin`
- `shape-outside`

The CSS shapes module also introduces the `shape-inside` and `shape-padding` properties. Currently, no browsers support these features.

Data types

- `<basic-shape>`

Functions

- `circle()`
- `ellipse()`
- `inset()`
- `path()`
- `polygon()`
- `rect()`
- `shape()`
- `xywh()`

Terms

- Reference box

Guides

Overview of shapes

Defining basic shapes with the `shape-margin` and `clip-path` properties, and debugging basic shapes with Developer Tools.

Shapes from box values

Using `border-radius` curvatures and CSS box model values to create shapes.

Basic shapes with `shape-outside`

Creating rectangles, circles, ellipses, and polygons with CSS shapes, the reference box, and the `shape-outside` property.

Shapes from images

Creating shapes from semi-transparent image files and CSS Gradients.

Related concepts

CSS motion path module

- `offset`
- `offset-anchor`
- `offset-distance`
- `offset-path`
- `offset-position`
- `offset-rotate`
- `ray()` function

CSS masking module

- `clip`
- `clip-path`
- `clip-rule`
- `mask`
- `mask-origin`
- `mask-position`

CSS backgrounds and borders module

- `border-radius` shorthand

CSS borders and box decorations module

- `border-shape`

CSS box model module

- `<box-edge>` data type

See also

- Shape generator
- CSS Shapes resources
- CSS Shapes 101 via alistapart.com (2014)
- Creating non-rectangular layouts with CSS Shapes via sarasoueidan.com (2013)
- How to use CSS Shapes in your web design via tutsplus.com (2016)
- Edit CSS shapes with the shape path editor via mozilla.org (2018) (Video)

Shapes from box values

A straightforward way to create a shape is to use a value from the CSS box model module. This article explains how to do this.

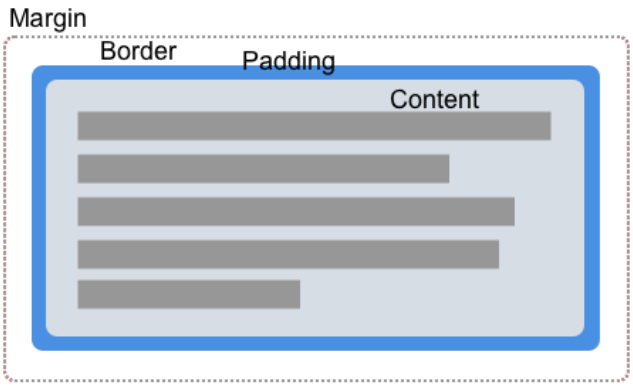
The `<box-edge>` box values allowable as a shape value are:

- `content-box`
- `padding-box`
- `border-box`
- `margin-box`

The `border-radius` values are also supported. This means you can give an element a curved border, and flow your content around the created shape.

CSS box model

The values listed above correspond to the various parts of the CSS Box Model. A box in CSS has content, padding, border, and margin.



By using box values for shapes, we can wrap our content around the edges defined by these values. In each of the examples below, I am using an element that has padding, a border, and a margin defined so that you can see the different ways in which content will flow.

margin-box

The `margin-box` is the shape defined by the outside margin edge and includes the corner radius of the shape, should `border-radius` have been used in defining the element.

In the example below, we have a circular purple item which is a `<div>` with a height, width, and background color. The `border-radius` property has been used to create a circle by setting `border-radius: 50%`. As the element has a margin, you can see that the content is flowing around the circular shape and the margin applied to it.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.shape {
  background-color: rebeccapurple;
  height: 80px;
  width: 80px;
  padding: 20px;
  margin: 20px;
  border: 10px solid black;
  border-radius: 50%;
  float: left;
  shape-outside: margin-box;
}
```

border-box

The `border-box` value is the shape defined by the outside border edge. This shape follows all of the normal border radius shaping rules for the outside of the border. You still have a border, even if you have not used the CSS `border` property. In this case, it will be the same as `padding-box`, the shape defined by the outside padding edge.

In the example below, you can see how the text now follows the line created by the border. Change the border size, and the content will follow it.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.box {
  width: 70%;
}

.shape {
  background-color: rebeccapurple;
  height: 80px;
  width: 80px;
  padding: 20px;
  margin: 20px;
  border: 10px solid black;
  border-radius: 50%;
  float: left;
  shape-outside: border-box;
}
```

padding-box

The `padding-box` value defines the shape enclosed by the outside padding edge. This shape follows all of the normal border radius shaping rules for the inside of the border. If you have no padding then `padding-box` is the same as `content-box`.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.2 sans-serif;
}
.box {
  width: 70%;
}

.shape {
  background-color: rebeccapurple;
  height: 80px;
  width: 80px;
  padding: 20px;
  margin: 20px;
  border: 10px solid black;
  border-radius: 50%;
  float: left;
```

```
  shape-outside: padding-box;
}
```

content-box

The `content-box` value defines the shape enclosed by the outside content edge. Each corner radius of this box is the `border-radius` less the `border-width` and `padding`, or `0`, whichever is larger. This means that it is impossible to have a negative value here.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.2 sans-serif;
}
.box {
  width: 70%;
}

.shape {
  background-color: rebeccapurple;
  height: 80px;
  width: 80px;
  padding: 20px;
  margin: 20px;
  border: 10px solid black;
  border-radius: 50%;
  float: left;
  shape-outside: content-box;
}
```

When to use box values

Using box values is a way to create shapes; however, this naturally works only with very basic shapes that can be defined using the `border-radius` property. The examples shown above show one such use case. You can create a circular shape using `border-radius` and then curve text around it.

With just this basic technique, you can create some interesting effects. In my final example of this section, I have floated two elements left and right, giving each a border radius of 100% in the direction closest to the text.

```
<div class="box">
  <div class="shape-left"></div>
  <div class="shape-right"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.box {
  text-align: justify;
}

.shape-left,
.shape-right {
  height: 100px;
  width: 100px;
}

.shape-left {
  margin: 0 20px 20px 0;
  border-bottom-right-radius: 100%;
  float: left;
  shape-outside: margin-box;
}
.shape-right {
  margin: 0 20px 20px;
  border-bottom-left-radius: 100%;
  float: right;
  shape-outside: margin-box;
}
```

For more complex shapes, you will need to use one of the basic shapes as a value, or define your shape from an image as covered in other guides in this section.

Shapes from images

In this guide, we will take a look at how we can create a shape from an image file with an alpha channel or even from a CSS Gradient. This is a very flexible way to create shapes. Rather than drawing a path with a complex polygon in CSS, you can create the shape in a graphics program and then use the path created by the pixels less opaque than a threshold value.

Creating shapes from images

To use an image for creating a shape, the image needs to have an Alpha Channel, an area that is not fully opaque. The `shape-image-threshold` property is used to set a threshold for this opacity. Pixels that are more opaque than this value will be used to calculate the area of the shape.

In the example below, there is an image of a star with a solid red area and an area that is fully transparent. The path to the image file is used as the value for the `shape-outside` property. The content now wraps around the star shape.

```
<div class="box">
  <img
    alt="A red star"
    src="https://mdn.github.io/shared-assets/images/examples/star-shape.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
img {
  float: left;
  shape-outside: url("https://mdn.github.io/shared-assets/images/examples/star-shape.png");
}
```

You can use `shape-margin` to move the text away from the shape, giving a margin around the created shape and the text.

```
<div class="box">
  <img
    alt="A red star"
    src="https://mdn.github.io/shared-assets/images/examples/star-shape.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

img {
  float: left;
  shape-outside: url("https://mdn.github.io/shared-assets/images/examples/star-shape.png");
  shape-margin: 20px;
}
```

CORS compatibility

Something that you will run into when creating shapes from an image is that the image you use must be CORS compatible. An image hosted on the same domain as your site should work, however, if your images are hosted on a different domain such as on a CDN you should ensure that they are sending the correct headers to enable them to be used for Shapes. Due to this requirement for CORS-compatible images, if you are previewing your file locally without using a local web server, your shape will not work.

Is it a CORS issue?

DevTools can help you to identify CORS errors. In Chrome the Console will alert you to CORS problems. In Firefox if you inspect the property you will be alerted to the fact that the image could not be loaded. This should alert you to the fact that your image cannot be used as the source of a shape due to CORS.

Setting a threshold

The `shape-image-threshold` property enables the creation of shapes from areas which are not fully transparent. If the value of `shape-image-threshold` is `0.0` (which is the initial value) then the area must be fully transparent. If the value is `1.0` then it is fully opaque. Values in between mean that you can set a semi-transparent area as the defining area.

In the example below, the background of the star is not fully transparent, it has a 20% opacity as created in my graphics program. If I set `shape-image-threshold` to `0.2` or greater, then I see the shape, if I set it to something smaller than `0.2` I do not get the shape.

```
<div class="box">
  <img
    alt="A red star"
    src="https://mdn.github.io/shared-assets/images/examples/star-red-20.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

img {
  float: left;
```

```
  shape-outside: url("https://mdn.github.io/shared-assets/images/examples/star-red-20.png");
  shape-image-threshold: 0.2;
}
```

Using images with generated content

In the above example, I have both used the image as the value of `shape-outside` and also added it to the page. Many demos do this as it helps to show the shape we are following, however, the `shape-outside` property is not related to the image that is displayed on the page and so you do not need to display an image to use an image to create a shape.

You do need something to float, but that could be some generated content as in the below example. I am floating generated content and using a larger star image to shape my content without displaying any image on the page.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.box::before {
  content: "";
  float: left;
  width: 400px;
  height: 300px;
  shape-outside: url("https://mdn.github.io/shared-assets/images/examples/star-shape.png");
  shape-image-threshold: 0.3;
}
```

Creating shapes using a gradient

Because a CSS gradient is treated as an image, you can use a gradient to generate a shape by having transparent or semi-transparent areas as part of the gradient.

The next example uses generated content. The content has been floated, giving it a background image of a linear gradient. I am using that same value as the value of `shape-outside`. The linear gradient goes from purple to transparent. By changing the value of `shape-image-threshold`, you can decide how transparent the pixels need to be that create the shape. You can play with that value in the example below to see how the diagonal line will move across the shape depending on that value.

You could also try removing the background image completely, thus using the gradient purely to create the shape and not displaying it on the page at all.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}

.box::before {
  content: "";
  float: left;
  height: 250px;
  width: 400px;
  background-image: linear-gradient(
    to bottom right,
    rebeccapurple,
    transparent
  );
  shape-outside: linear-gradient(to bottom right, rebeccapurple, transparent);
  shape-image-threshold: 0.3;
}
```

The next example uses a radial gradient with an ellipse, once again using a transparent part of the gradient to create the shape.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
```

```
}

.box::before {
  content: "";
  float: left;
  height: 250px;
  width: 400px;
  background-image: radial-gradient(
    ellipse closest-side,
    rebeccapurple,
    blue 50%,
    transparent
  );
  shape-outside: radial-gradient(
    ellipse closest-side,
    rebeccapurple,
    blue 50%,
    transparent
  );
  shape-image-threshold: 0.3;
}
```

You can experiment directly in these live examples to see how changing the gradient will change the path of your shape.

Overview of shapes

The CSS Shapes module describes geometric shapes in CSS. This article provides an overview of how you can use shapes to wrap text around floated elements that are not necessarily rectangular.

When you float an item left, text will wrap around the right and bottom of the item in a rectangular fashion. With CSS shapes you can, for example, apply a circle shape, and the text will wrap around the line of the circle.

There are several ways to create this circle. In this guide, we will look at how CSS Shapes work and how to use them.

What does the specification define?

The specification defines a few properties, including:

- `shape-outside` — Allows definition of basic shapes.
- `shape-image-threshold` — Sets an opacity threshold value. If an image is being used to define a shape, only the parts of the image that are the same opacity or greater than the threshold value are used in the shape. Any other parts are ignored.
- `shape-margin` — Sets a margin around the defined shape.

Defining basic shapes

The `shape-outside` property allows us to define a shape. It takes a variety of values that define different shapes specified in the `<basic-shape>` data type.

In the following example, an image is floated to the left. We apply the `shape-outside` property with a `circle(50%)` value. The result is that the content now curves around the circular shape rather than following the rectangle created by the box of the image.

```
<div class="box">
  <img
    alt="An orange hot air balloon as seen from below"
    src="https://mdn.github.io/shared-assets/images/examples/round-balloon.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.4 sans-serif;
}

img {
  float: left;
  shape-outside: circle(50%);
}
```

Here we used the `circle()` function, which is supported across all modern browsers. If we used a newer shape type that doesn't have full support, users of non-supporting browsers would see the content flowing around the sides of a rectangular, due to the image being floated. Shapes are a visual progressive enhancement.

Basic shapes

The value `circle(50%)` is an example of a basic shape. The specification defines several `<basic-shape>` values, including:

- `circle()`
- `ellipse()`
- `inset()`
- `path()`
- `polygon()`
- `rect()`
- `shape()`
- `xywh()`

Three of these functions only define rectangles. With the `inset()` function, you define four offset values, thus pulling the line boxes of any wrapping content closer to the object than they otherwise would. The `rect()` function defines a rectangle by specifying the distance from the top and left edges of the containing block. The `xywh()` function works by specifying distances from the top and left edges of the reference box and setting the width and height of the rectangle from that starting point.

We have already seen how `circle()` creates a circular shape. An `ellipse()` is essentially a squashed circle. If none of these basic shapes do the trick you can create more complex shapes using the `polygon()` function, which allows the definition of a series of lines. The `path()` and `shape()` functions can be used to create ANY shape via a series of line, curve, and move commands.

In our Guide to Basic Shapes we explore each of the possible Basic Shapes and how to create them.

Shapes from the box value

Shapes can also be created around the box value. Therefore, you could create your shape on:

- `border-box`
- `padding-box`
- `content-box`
- `margin-box`

In the example below, you can change the value `border-box` to any of the other allowed values to see how the shape moves closer or further away from the box.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.4 sans-serif;
}

.shape {
  background-color: rebeccapurple;
  height: 150px;
  width: 150px;
  padding: 20px;
  margin: 20px;
  border-radius: 50%;
  float: left;
  shape-outside: border-box;
}
```

To explore the box values in more detail, see our guide covering Shapes from box values.

Shapes from images

An interesting way to generate your path is to use an image with an alpha channel—the text will then wrap around the non-transparent parts of the image. This allows the overlay of wrapped content around an image or the use of an image that is never displayed on the page purely as a method of creating a complex shape without needing to carefully map a polygon.

Note that images used in this way must be CORS compatible, otherwise `shape-outside` will act as if `none` had been given as the value, and you will get no shape.

In this next example, we have an image with a fully transparent area, and we are using an image as the URL value for `shape-outside`. The shape is created around the opaque area — the image of the balloon.

```
<div class="box">
  <img
    alt="An orange hot air balloon as seen from below"
    src="https://mdn.github.io/shared-assets/images/examples/round-balloon.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.4 sans-serif;
}

img {
  float: left;
  shape-outside: url("https://mdn.github.io/shared-assets/images/examples/round-balloon.png");
}
```

`shape-image-threshold`

The `shape-image-threshold` property is used to set the threshold of image transparency used to define the area of the image used for the shape. If the value of `shape-image-threshold` is `0.0` (which is the initial value) then the area must be fully transparent. If the value is `1.0` then it is fully opaque. Values in between mean that you can set a semi-transparent area as the defining area of the shape.

You can see the threshold in action if we use a gradient as the image on which to define our shape. In the example below, if you change the threshold the path that the shape takes changes based on the level of opacity you have selected.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
```

```
  </p>
</div>
```

```
body {
  font: 1.2em / 1.4 sans-serif;
}

.shape {
  float: left;
  width: 200px;
  height: 200px;
  background-image: linear-gradient(
    45deg,
    rebeccapurple,
    transparent 80%,
    transparent
  );
  shape-outside: linear-gradient(
    45deg,
    rebeccapurple,
    transparent 80%,
    transparent
  );
  shape-image-threshold: 0.4;
}
```

To learn more about creating shapes from images, see the Shapes from images guide.

The <code>shape-margin</code> property

The `shape-margin` property adds a margin to `shape-outside`. This will further shorten the line boxes of any content wrapping the shape, pushing it away from the shape itself.

In the example below we have added a `shape-margin` to a basic shape. Change the margin to push the text further away from the path the shape would take by default.

```
<div class="box">
  <img
    alt="An orange hot air balloon as seen from below"
    src="https://mdn.github.io/shared-assets/images/examples/round-balloon.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.4 sans-serif;
}
img {
  float: left;
  shape-outside: circle(50%);
  shape-margin: 5px;
}
```

Using Generated Content as the floated item

In the examples above, we have used images or a visible element to define the shape, meaning that you can see the shape on the page. Instead, you might want to flow some text along a non-rectangular invisible line. We could, for example, add an empty floated `<div>` or `<span>` element to our DOM and make it invisible. However, we can create a shape with only CSS using generated content and keep all our styling functionality inside the CSS.

In this example, we use generated content to insert an element with a height and width of 150px. We can then use basic shapes, box values, or even the alpha channel of an image to create a shape for the text to wrap around.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box::before {
  content: "";
  display: block;
  height: 150px;
  width: 150px;
  padding: 20px;
  margin: 20px;
  border-radius: 50%;
  float: left;
  shape-outside: border-box;
}
```

Relationship to <code>clip-path</code>

The basic shapes and box values used to create shapes are the same as those used as values for `clip-path`. Therefore, if you want to create a shape using an image, and also clip away part of that image, you can use the same values.

The image below is a square image with a blue background. We have defined a shape using `shape-outside: ellipse(40% 50%);` and also used `clip-path: ellipse(40% 50%);` to clip away the same area that we used to define the shape.

```
<div class="box">
  <img
    alt="An orange hot air balloon as seen from below"
    src="https://mdn.github.io/shared-assets/images/examples/balloon-small.jpg" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.4 sans-serif;
}

img {
  float: left;
  shape-outside: ellipse(40% 50%);
  clip-path: ellipse(40% 50%);
}
```

Developer Tools for Shapes

There is a Shape Path Editor in the Firefox DevTools. This tool can be used to inspect the `circle()`, `inset()`, `ellipse()`, and `polygon()` values. If your polygon isn't quite right you can use the Shapes Editor to tweak it, then copy the new value back into your CSS.

More CSS Shapes Features

In this guide, we discussed wrapping text around floated shapes. See the CSS shapes module for links to all the module features plus additional related features. This includes all the shape functions and relevant guides.

Shape generator

This shape generator tool can be used to define the coordinates and syntax for features that use `<basic-shape>`

- The `coords` attribute of the `<area>` element
- The CSS `inset()`, `xywh()`, `rect()`, `circle()`, `polygon()` functions
- The SVG `<rect>`, `<circle>`, `<polygon>` elements

The shape generator helps you visualize basic shapes, outputting the coordinates of each point, superimposed over your image. Upload an image by either dragging or clicking to select a file. Then select the type of shape you want to create from the dropdown menu. Optionally, you can also adjust the scale of the canvas using the range input. Then click on points on the image to add reference points for the shape you want to create. The tool will then output the markup for the HTML `coords` attribute value, CSS basic shape function with coordinates, and SVG shape.

```
<fieldset>
  <legend>Controls</legend>
  <p>
    <label for="scale">
      Scale:
      <input type="range" id="scale" min="0.1" max="3" value="1" step="any" />
    </label>
  </p>
  <p>
    <label for="shape">
      Shape:
      <select id="shape">
        <option value="rect">Rectangle</option>
        <option value="circle">Circle</option>
        <option value="poly">Polygon</option>
        <option value="points">Discrete points</option>
      </select>
    </label>
  </p>
  <p>
    <button id="reset">Reset</button>
  </p>
</fieldset>
<canvas id="canvas">Drop and image here, or click to upload.</canvas>
<p>
  <output id="coords">Click on the canvas to add points.</output>
</p>
```

```
#canvas {
  display: block;
  border: 1px solid black;
  transform-origin: top left;
  margin: 0.5em 0;
}

#coords {
  display: block;
  position: relative;
  word-break: break-all;
}

dt {
```

```
  font-weight: bold;
  margin-top: 1em;
}
dd {
  white-space: pre-wrap;
  font-family: monospace;
}
```

```
const canvas = document.getElementById("canvas");
const scaleInput = document.getElementById("scale");
const shapeSelect = document.getElementById("shape");
const resetButton = document.getElementById("reset");
const coordsDisplay = document.getElementById("coords");
const ctx = canvas.getContext("2d");
let currentImage = null;
let coords = [];

function init() {
  if (currentImage) {
    URL.revokeObjectURL(currentImage.src);
    currentImage = null;
  }
  resetCoords();
  scaleInput.value = 1;
  canvas.style.transform = "scale(1)";
  canvas.width = 400;
  canvas.height = 300;
  canvas.style.width = "400px";
  canvas.style.height = "300px";
  ctx.font = "20px serif";
  ctx.fillText("Drop an image here, or click to upload.", 10, 20);
}

function initImage(file) {
  const url = URL.createObjectURL(file);
  const img = new Image();
  img.src = url;
  img.addEventListener("load", () => {
    canvas.width = img.width;
    canvas.height = img.height;
    canvas.style.width = `${img.width}px`;
    canvas.style.height = `${img.height}px`;
    ctx.drawImage(img, 0, 0);
    currentImage = img;
  });
}

function displayCoords(htmlCoords, cssFunc, svgElem) {
  const dl = document.createElement("dl");
  const dt1 = document.createElement("dt");
  dt1.innerHTML = "HTML <code>coords</code> attribute";
  const dd1 = document.createElement("dd");
  dd1.innerText = htmlCoords;
  const dt2 = document.createElement("dt");
  dt2.textContent = "CSS shape functions";
  const dd2 = document.createElement("dd");
  dd2.innerText = cssFunc;
  const dt3 = document.createElement("dt");
  dt3.textContent = "SVG element";
  const dd3 = document.createElement("dd");
  dd3.innerText = svgElem;
  dl.append(dt1, dd1, dt2, dd2, dt3, dd3);
  coordsDisplay.textContent = "";
  coordsDisplay.appendChild(dl);
}

function renderShape() {
  ctx.strokeStyle = "magenta";
  ctx.fillStyle = "red";
  ctx.strokeWidth = "3";
  resetDrawnShape();
  if (shapeSelect.value === "rect" && coords.length === 2) {
    const { x: x1, y: y1 } = coords[0];
    const { x: x2, y: y2 } = coords[1];
    const w = x2 - x1;
    const h = y2 - y1;
    ctx.strokeRect(x1, y1, w, h);
    displayCoords(
      `coords="${x1},${y1},${x2},${y2}"`,
      `inset(${y1}px ${x1}px ${canvas.height - y2}px ${canvas.width - x2}px)
xywh(${x1}px ${y1}px ${w}px ${h}px)
rect(${y1}px ${x2}px ${y2}px ${x1}px)`,
      `<rect x="${x1}" y="${y1}" width="${w}" height="${h}" />`,
    );
  } else if (shapeSelect.value === "circle" && coords.length === 2) {
    ctx.beginPath();
    const { x, y } = coords[0];
    const r = Math.sqrt(
      (coords[1].x - x) ** 2 + (coords[1].y - y) ** 2,
    ).toFixed(1);
    ctx.arc(x, y, r, 0, Math.PI * 2);
    ctx.stroke();
    ctx.closePath();
    displayCoords(
      `coords="${x},${y},${r}"`,
      `circle(${r}px at ${x}px ${y}px)`,
      `<circle cx="${x}" cy="${y}" r="${r}" />`,
    );
  } else if (shapeSelect.value === "poly" && coords.length > 2) {
    ctx.beginPath();
    ctx.moveTo(coords[0].x, coords[0].y);
    for (let i = 1; i < coords.length; i++) {
      ctx.lineTo(coords[i].x, coords[i].y);
```

```
    }
    ctx.closePath();
    ctx.stroke();
    displayCoords(
      `coords="${coords.map((coord) => `${coord.x},${coord.y}`).join(",")}"`,
      `polygon(${coords.map((coord) => `${coord.x} ${coord.y}`).join(", ")})`,
      `<polygon points="${coords.map((coord) => `${coord.x},${coord.y}`).join(" ")}" />`,
    );
  } else if (shapeSelect.value === "points") {
    const p = document.createElement("p");
    p.innerText = `Coordinate:\n${coords
      .map((coord) => `${coord.x},${coord.y}`)
      .join("\n")}`;
    coordsDisplay.textContent = "";
    coordsDisplay.appendChild(p);
  }
  for (const coord of coords) {
    ctx.beginPath();
    ctx.arc(coord.x, coord.y, 3, 0, Math.PI * 2);
    ctx.fill();
    ctx.closePath();
  }
}

function resetDrawnShape() {
  if (!currentImage) return;
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.drawImage(currentImage, 0, 0);
}

function resetCoords() {
  coords = [];
  coordsDisplay.textContent = "Click on the canvas to add points.";
}

init();
canvas.addEventListener("dragover", (event) => {
  event.preventDefault();
});
canvas.addEventListener("drop", (event) => {
  event.preventDefault();
  initImage(event.dataTransfer.files[0]);
});
canvas.addEventListener("click", (event) => {
  if (!currentImage) {
    const input = document.createElement("input");
    input.type = "file";
    input.accept = "image/*";
    input.addEventListener("change", (e) => {
      initImage(e.target.files[0]);
    });
    input.click();
    return;
  }
  if (
    (shapeSelect.value === "rect" || shapeSelect.value === "circle") &&
    coords.length === 2
  ) {
    resetCoords();
  }
  coords.push({ x: event.offsetX, y: event.offsetY });
  renderShape();
});
scaleInput.addEventListener("input", () => {
  canvas.style.transform = `scale(${scaleInput.value})`;
  coordsDisplay.style.top = `${canvas.height * (scaleInput.value - 1)}px`;
});
shapeSelect.addEventListener("change", () => {
  resetCoords();
  resetDrawnShape();
});
resetButton.addEventListener("click", init);
```

Basic shapes with shape-outside

CSS shapes can be defined using the `<basic-shape>` type. In this guide, we discuss creating rectangles, circles, ellipses, and polygons with the `shape-outside` property. These are features defined in the CSS shapes module.

Before looking at shapes, it is worth understanding two pieces of information that go together to make these shapes possible:

- The `<basic-shape>` type
- The reference box

The <basic-shape> type

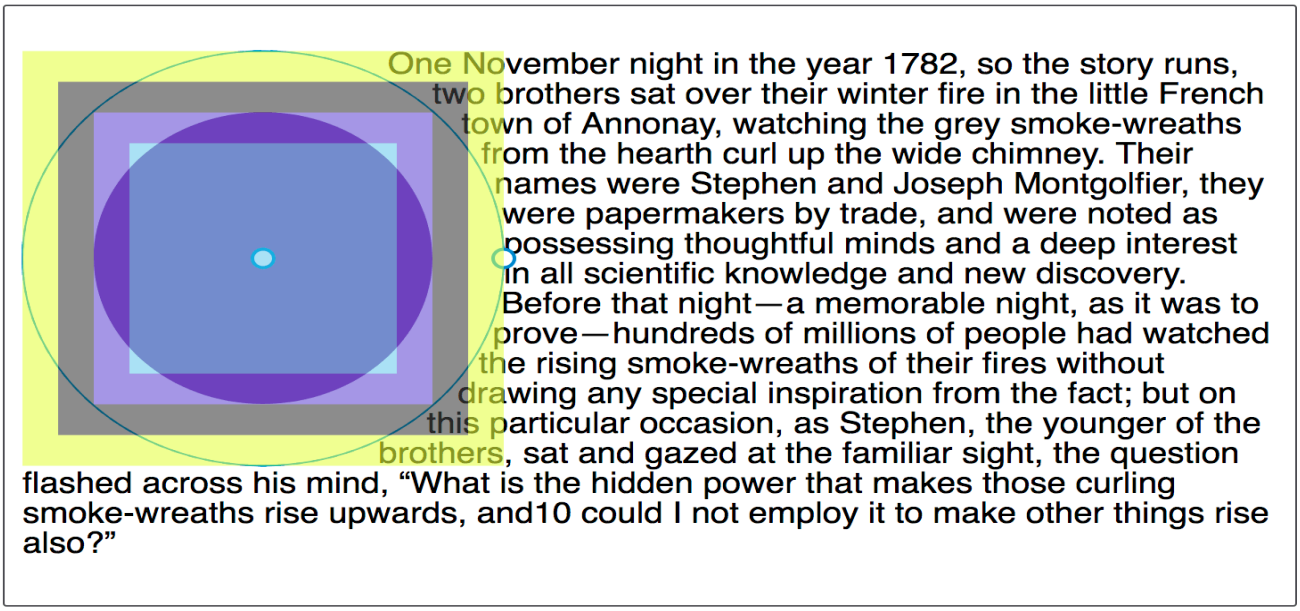
The `<basic-shape>` type is used as the value for all of our basic shapes. This type is a functional notation: the function parentheses contain arguments used to describe the shape.

The accepted arguments vary depending on the shape you are creating. We will cover these in the examples below.

The reference box

Understanding the reference box used by CSS shapes is important when using basic shapes, as it defines each shape's coordinate system. You have already met the reference box in the guide on creating shapes from box values, which directly uses the reference box to create the shape.

The screenshot below shows the Firefox Shapes Inspector displaying the reference box of a circle created using `shape-outside: circle(50%)`. The element has 20 pixels of padding, border, and margin applied. The Shapes Inspector highlights these reference boxes.



The default reference box for a basic shape is the `margin-box`. You can see in the screenshot that the shape is defined relative to that part of the Box Model.

While the default reference box is `margin-box`, this can be modified. To set a different box as the reference box, include the desired box value after your basic shape definition.

These two declarations are the same:

```
shape-outside: circle(50%);
shape-outside: circle(50%) margin-box;
```

For your shape to use a different reference box, include a different `<box-edge>` value, for example, to use the border as a reference box for our circle, set:

```
.shape {
  shape-outside: circle(50%) border-box;
}
```

Shapes created that extend past the margin box will have the shape clipped to the margin box. The following basic shapes demonstrate this.

inset()

The `inset()` function defines a rectangle. This may not seem very useful as floating an item, without shapes, will give you a rectangular shape around it. However, the `inset()` type enables the definition of offsets, thus pulling the wrapping text around the reduced-size rectangle, over parts of the floated element.

The `inset()` function takes up to four side offset values, plus an optional `round` keyword, followed by a `border-radius` value. The below CSS creates a rectangular shape inset from the reference box of the floated element 20 pixels from the top and bottom and 10 pixels from the left and right, with a `border-radius` value of 10 pixels.

```
.shape {
  float: left;
  shape-outside: inset(20px 10px 20px 10px round 10px);
}
```

The offset values use the same rules as the `margin` shorthand. Four space-separated values define the top, right, bottom, and left offsets — in that order. You can also set more than one offset at once:

- If there is only one value, it applies to all sides.
- If there are two values, the top and bottom offsets are set to the first value and the right and left offsets are set to the second.
- If there are three values, the top is set to the first value, the left and right are set to the second, and the bottom is set to the third.

The above rules can therefore also be written as:

```
.shape {
  float: left;
  shape-outside: inset(20px 10px round 10px);
}
```

In the example below we have an `inset()` shape used to pull content over the floated element. Change the offset values to see how the shape changes.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.shape {
```

```
  float: left;
  width: 150px;
  height: 100px;
  shape-outside: inset(20px 50px 10px 0 round 50px);
  background-color: rebeccapurple;
  border: 2px solid black;
  border-radius: 10px;
  margin: 20px;
  padding: 20px;
}
```

You can also add a box value as an alternative reference box. In the example below, try changing the reference box from `margin-box` to `border-box`, `padding-box`, or `content-box` to see how the reference box used as the starting point changes before offsets are calculated.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.shape {
  float: left;
  width: 150px;
  height: 100px;
  shape-outside: inset(20px 50px 10px 0 round 50px) margin-box;
  background-color: rebeccapurple;
  border: 2px solid black;
  border-radius: 10px;
  margin: 20px;
  padding: 20px;
}
```

You can also create rectangles based on distances from the top and left edges of the reference box with the `rect()` function, or by width and height with the `xywh()` function; both of these also support optional rounded corners.

circle()

The `circle()` value for `shape-outside` can accept two possible arguments: a `<shape-radius>` defining the size and the `<position>` defining its location.

The `circle()` and `ellipse()` `shape-outside` values both accept `<shape-radius>` as an argument. This can be a `<length>`, a `<percentage>`, or one of the keywords `closest-side` or `farthest-side`.

The `closest-side` keyword value uses the length from the center of the shape to the closest side of the reference box to create the radius length. The `farthest-side` keyword value uses the length from the center of the shape to the farthest side of the reference box.

The second argument is a `position`, which accepts a one- or two-keyword `<position>` value, to indicate the position of the center of the circle. This is specified the same way as `background-position`; if one or both values are omitted, the values default to `center`.

To create a circle, we include a single radius value, optionally followed by the keyword **at** followed by a position value. This example has a circle applied to an `<img>` with a `width` and `height` of `210px` and a `margin` of `20px`. This gives a total width for the reference box of `250px`. The `50%` value for the `<shape-radius>` means the radius is `125px`. The position value is set to `30%`, which is `30%` from the left and at the default vertical `center`.

```
<div class="box">
  <img
    alt="An orange hot air balloon as seen from below"
    src="https://mdn.github.io/shared-assets/images/examples/round-balloon.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

img {
  float: left;
  shape-outside: circle(50% at 30%);
  margin: 20px;
}
```

Play with increasing or decreasing the size of the circle by changing the size of the radius, moving the circle around with the position value, or setting a reference box as we did for `inset()`.

The below example combines generated content with a `circle()` function that uses the keywords `top left` for position. This creates a quarter circle shape in the top left corner of the page for text to flow around.

```
<div class="box">
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
```

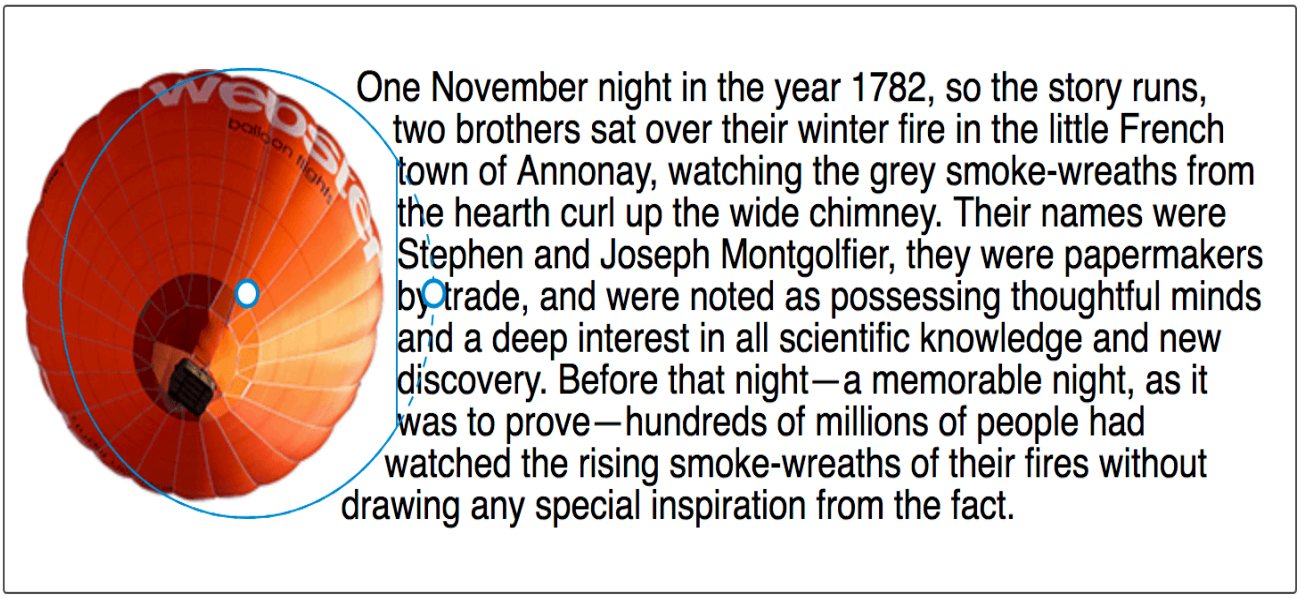
```
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.box::before {
  content: "";
  float: left;
  width: 250px;
  height: 250px;
  shape-outside: circle(50% at top left);
}
```

The shape will be clipped by the margin box

As noted in reference boxes above, the `margin-box` will clip the shape. You can see this by moving the center of our circle towards the content by setting the position to `60%`. The center of the circle will be nearer to the content and the circle will extend past the margin-box. This means that the extension becomes clipped and squared off.

```
img {
  float: left;
  shape-outside: circle(50% at 60%);
}
```



ellipse()

An ellipse is a squashed circle. As such, the `ellipse()` function acts in a very similar way to `circle()` except that we have to specify two radii, `x` and `y`, in that order.

These may then be followed by one or two `<position>` values, as with `circle()`, to define the location of the center of the ellipse. In the example below, we have an ellipse with an `x` radius of `40%`, a `y` radius of `50%` and the `<position>` set to `left`. This means that the center of the ellipse is at the center of the left edge of the reference box. This creates a half ellipse shape around which the text will wrap. You can change these values to see how the ellipse changes.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.shape {
  float: left;
  shape-outside: ellipse(40% 50% at left);
  margin: 20px;
  width: 100px;
  height: 200px;
}
```

The keyword values of `closest-side` and `farthest-side` are useful in creating a quick ellipse based on the size of the floated element reference box.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.shape {
  float: left;
  shape-outside: ellipse(closest-side farthest-side at 30%);
  margin: 20px;
  width: 100px;
  height: 140px;
}
```

polygon()

The `polygon()` function is more complex, enabling the creation of multiple-sided polygon shapes. This shape accepts three or more pairs of values (a polygon must at least draw a triangle). Each space-separated pair of values is separated with a comma and represents the coordinates of a single vertex drawn relative to the reference box. Each pair of coordinates defines an edge of the polygon, with the final edge defined by the first and last set of coordinates.

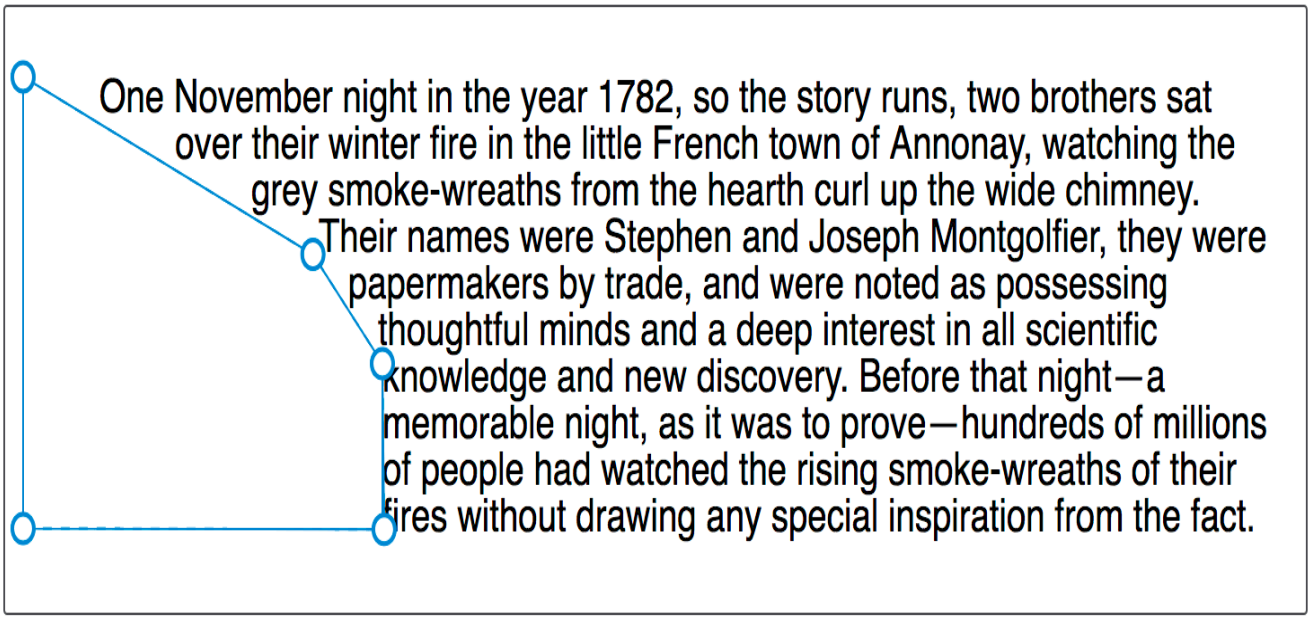
The example below creates a shape for text to follow using the `polygon()` function. Try changing the coordinate values to see how the shape changes.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

.shape {
  float: left;
  shape-outside: polygon(
    0px 0px,
    0px 189px,
    100.48% 94.71%,
    200px 120px,
    80.67% 37.17%
  );
  width: 200px;
  height: 200px;
}
```

To create even more complex shapes, you can define the outline of any shape with the `path()` or `shape()` functions.

The `inset()`, `circle()`, `ellipse()`, and `polygon()` are inspectable and editable using the Firefox Developer Tools Shape Inspector. The screenshot below shows the shape highlighted in the tool.



Another resource is Clippy, a tool for creating shapes with examples using the CSS `clip-path` property, which uses the same basic shape functions and values as the `shape-outside` property.

CSS syntax

The **CSS syntax** module describes, in general terms, the structure and syntax of cascading stylesheets, or CSS. It defines CSS as the language for describing the rendering of structured documents (such as HTML and XML), on the web and elsewhere.

This module doesn't define any properties, data types, functions, or at-rules. Rather, it elaborates on how all of these features should be defined and how user agents should parse CSS.

Reference

At-rules and descriptors

- none

> **Note:** The module explicitly states that `@charset` is not an actual at-rule, but rather an unrecognized legacy rule that should be omitted when a stylesheet is grammar-checked. The only valid `@charset` usage is at the very beginning of a stylesheet, where it is interpreted as a special byte sequence stripped before processing the content.

Key concepts

- At-rules
- character escaping
- CSS comments
- CSS declaration
- CSS declaration block
- CSS function
- Invalid
- Style rule

Glossary terms

- CSS descriptor
- Parse
- Style sheet
- Whitespace

Guides

Introduction to CSS syntax: declarations, rulesets, and statements

Explains the overall CSS syntax and how declarations, declaration blocks, rulesets, and statements form the style rules.

Value definition syntax

Explains the formal grammar for defining valid values for CSS properties and functions, along with semantic constraints. A guide for understanding CSS component value types, combinators, and multipliers.

CSS error handling

Overview of how browsers handle invalid CSS.

Learn CSS first steps: CSS syntax

Introductory guide to CSS, including an introduction to CSS syntax.

Related concepts

CSS selectors module:

- CSS specificity

CSS cascading and inheritance module:

- `@import` at-rule
- `important` flag
- Initial values

- Computed values
- Used values
- Actual values
- CSS inheritance
- CSS property

CSS custom properties for cascading variables module:

- custom property (`--*`)
- `var()` function

CSS conditional rules module:

- `@media` at-rule
- `@supports` at-rule

CSS Object Model (CSSOM) API:

- `cssText` property
- `insertRule(rule)` method
- `replace(text)` method

WHATWG specification:

- `<style>` element
- `<link>` element
- `class` attribute
- `rel` attribute

See also

- CSS at-rule functions
- CSS selectors module
- CSS values and units module

At-rules

At-rules are CSS statements that instruct CSS how to behave. They begin with an at-sign, `@` (U+0040 COMMERCIAL AT), followed by an identifier. They include everything from the at-keyword up to the next semicolon, `;` (U+003B SEMICOLON), or the next CSS block, whichever comes first.

At-rules are used to group and structure style rules and other at-rules, declare style information not directly associated with selected content, and manage syntactic constructs such as imports and namespaces keyword mappings.

Syntax

The at-rule is defined in the CSS syntax module, with different at-rules being defined in their respective modules. They generally take one of two forms depending on the specific rule and its purpose: statement at-rules and block at-rules, which can contain nested qualified rules, at-rules, or declarations.

Statement at-rules

```
/* General structure */
@identifier (RULE);

/* Example: tells browser to use UTF-8 character set */
@charset "utf-8";
```

Statement at-rules end in a semicolon. There are several statement at-rules, designated by their identifiers, each with a different syntax:

`@charset`

: An algorithm (has the syntactic form of an at-rule, but isn't a definition) that determines the fallback character set used by the style sheet (CSS Syntax).

`@import`

: Tells the CSS engine to include an external style sheet (CSS cascading and inheritance).

`@layer`

: Defines the order of precedence in case of multiple cascade layers (CSS cascading and inheritance). Also used as a block at-rule to define a layer's styles.

`@namespace`

: Defines a default namespace for a style sheet or a namespace prefix that a selector only matches if the namespace and other selector components match (CSS namespaces).

Block at-rules

```
@identifier (RULE) {
}
```

Block at-rules end in a `{}`-block that contain nested rules, other at-rules, or property or descriptor declarations.

`@counter-style`

: Define custom counter styles and extend predefined list styles (CSS counter styles).

`@container`

: A conditional group rule that applies its content if the container meets the `<container-condition>`s (CSS containment).

`@font-face`

: Defines font resource locations, both local and external, along with the style characteristics for when those resources are used with a declared `font-family` (CSS fonts).

`@font-feature-values` (plus `@swash`, `@ornaments`, `@annotation`, `@stylistic`, `@styleset` and `@character-variant`)

: Controls font display per font-family by defining font-specific alternates, or custom names, to feature indexes in `font-variant-alternates` in OpenType (CSS fonts).

`@keyframes` (and the `@-webkit-keyframes` alias)

: Define a named animation by describing defining CSS styles for intermediate steps (or keyframes) in the animation sequence (CSS animations).

`@layer`

Creates a named cascade layer with the CSS rules for that layer inside (CSS cascading and inheritance). Also used as a statement at-rule to define the order of precedence in case of multiple cascade layers

`@media`

A conditional group rule that applies its content if the device meets the criteria of the condition defined using a *media query* (CSS conditional rules).

`@page`

Specifies aspects of a page to be printed, such as its dimensions, orientation, and margins (CSS paged media).

`@position-try`

Defines custom position options which can be used to define fallback positioning and alignment options for anchor-positioned elements (CSS anchor positioning).

`@property`

Defines a CSS custom property, allowing for property type checking and constraining, setting default values, and defining whether a custom property can inherit values or not (CSS custom properties for cascading variables).

`@scope`

Defines a scope in which to apply them to selected elements and the styles to apply to the elements in that scope (CSS cascading and inheritance).

`@starting-style`

Define the starting property values for an element to transition from when the element receives its first style update, such as when transitioning from `display: none` (CSS transitions).

`@supports`

A conditional group rule applies its content if the browser supports the CSS features of the given condition (CSS conditional rules).

`@view-transition`

Opts the current document into a view transition, and the destination document as well in the case of cross-document navigation transitions.

See also

- CSS at-rules
- CSS at-rule functions
- Nesting at-rules
- CSS statements
- CSSRule interface
- CSS conditional rules module
- CSS syntax module
- Specificity
- Inheritance

Comments

A CSS **comment** is used to add explanatory notes to the code or to prevent the browser from interpreting specific parts of the style sheet. By design, comments have no effect on the layout of a document.

Syntax

Comments can be placed wherever white space is allowed within a style sheet. They can be used on a single line, or traverse multiple lines.

```
/* Comment */
```

Examples

```
/* A one-line comment */

/*
A comment
which stretches
over several
lines
*/

/* The comment below is used to
   disable specific styling */
/*
span {
  color: blue;
  font-size: 1.5em;
}
*/
```

Notes

The `/* */` comment syntax is used for both single and multiline comments. There is no other way to specify comments in external style sheets. However, when using the `<style>` element, you may use `<!-- -->` to hide CSS from older browsers, although this is not recommended. As with most programming languages that use the `/* */` comment syntax, comments cannot be nested. In other words, the first instance of `*/` that follows an instance of `/*` closes the comment.

See also

- CSS syntax module
- Syntax guide
- CSS error handling

CSS error handling

When an error exists in CSS, such as an invalid value or a missing semicolon, instead of throwing an error like in JavaScript, the browser (or other user agent) will gracefully recover. Browsers don't provide CSS-related alerts or otherwise indicate errors have occurred in styles. They just discard invalid content and parse subsequent valid styles. This is a feature of CSS, not a bug.

This guide discusses how CSS parsers discard invalid CSS.

CSS parser errors

When a CSS error is encountered, the browser's parser ignores the line containing the errors, discarding the minimum amount of CSS code before returning to parsing the CSS as normal. The "error recovery" is just the ignoring or skipping of invalid content.

The fact that browsers ignore invalid code enables the use of new CSS features without worrying about anything being broken in older browsers. A browser may not recognize a new feature, but that is okay. The discarding of invalid content without throwing an error allows both old and new syntaxes to coexist in the same ruleset, although bear in mind that they should be specified in that order. For example:

```
div {
  display: inline-flex;
  display: inline flex;
}
```

The `display` property accepts both legacy single value and multi-keyword syntax. Browsers will render the old syntax until they recognize the new syntax as valid, at which point the new syntax will override the old. If a user has an old browser, the valid fallback won't get overwritten by the new CSS, because the browser perceives it as invalid.

The type and amount of CSS that a browser ignores due to an error depends on the type of error. Some common error situations are listed below:

- For errors in at-rules, whether a single line or the entire at-rule is ignored (fails) depends on the at-rule and the type of error.
- If the error is an invalid selector, the entire declaration block is ignored.
- An error due to a missing semi-colon between property declarations causes an invalid value, in which case, multiple property-value declarations are ignored.
- If the error is a property name or value, such as an unrecognized property name or invalid data type, the property-value declaration is ignored. During the filtering stage, such syntactically invalid declarations are eliminated.
- If the error is due to a missing end-bracket, the extent of what is ignored depends on the browser's ability to parse the error as nested CSS.

After parsing each declaration, style rule, at-rule, and so on, the browser checks the parsed content against its expected grammar for that construct. If the content does not match the expected grammar for that construct, the browser considers it invalid and ignores it.

At-rule errors

The `@` symbol, known in CSS specifications as an `<at-keyword-token>`, indicates the beginning of a CSS at-rule. Once an at-rule begins with the `@` symbol, nothing is considered invalid from the parser's standpoint. Everything up to the first semi-colon (`;`) or the opening curly brace (`{`) is part of the at-rule's prelude. The content of each at-rule is interpreted according to the grammar rules for that particular at-rule.

Statement at-rules, such as `@import` and `@namespace` declarations, contain just a prelude. The semicolon ends the at-rule immediately for statement at-rules. If the contents of the prelude are invalid according to the grammar for that at-rule, the at-rule is ignored, with the browser continuing to parse CSS after it encounters the next semi-colon. For example, if an `@import` at-rule occurs after any CSS declaration other than `@charset`, `@layer` or other `@import` statements, the `@import` declaration is ignored.

```
@import "assets/fonts.css" layer(fonts);
@namespace svg url("http://www.w3.org/2000/svg");
```

If the parser encounters a curly brace (`{`) before a semi-colon is encountered, the at-rule is parsed as a block at-rule. Block at-rules like `@font-face` and `@keyframes`, contain a block of declarations surrounded by curly braces (`{}`). The opening curly brace informs the browser where the at-rule prelude ends and the at-rule's body starts. The parser looks forward, seeking matching blocks (content surrounded by `()`, `{}`, or `[]`) until it finds a closing curly brace (`}`) that isn't matched by any other curly braces: this closes the body of the at-rule.

Different at-rules have different grammar rules, different (or no) descriptors, and different rules for what, if anything, will invalidate the entire at-rule. The expected grammar for each at-rule and how errors are handled are documented on the respective at-rule page. The handling of invalid content depends on the error.

For example, the `@font-face` rule requires both a `font-family` and `src` descriptor. If either of these is omitted or invalid, the entire `@font-face` rule is invalid. Including an unrelated descriptor, any other valid font descriptor with an invalid value, or a property style declaration within the `@font-face` nested block will not invalidate the font declaration. As long as the font name and font source are included and valid, any invalid CSS within the at-rule is ignored, but the `@font-face` block is still parsed.

While the grammar of the `@keyframes` at-rule is very different from the `@font-face` rule grammar, the type of error still impacts what gets ignored. Important declarations (marked with the `important` flag) and properties that can't be animated are ignored in keyframe rules, but they don't impact other styles declared in the same keyframe selector block. Including an invalid keyframe selector (such as a percentage value less than `0%` or greater than `100%`, or a `<number>` omitting the `%`) invalidates the keyframe selector list and therefore the style block is ignored. An invalid keyframe selector only invalidates the invalid selector's style block; it does not invalidate the entire `@keyframes` declaration. Including styles between two keyframe selector blocks, on the other hand, will invalidate the entire `@keyframes` at-rule.

Some at-rules are almost always valid. The `@layer` at-rule comes in both regular and nested forms. The `@layer` statement syntax contains just the prelude, ending with a semi-colon. Alternatively, the nested syntax has layer styles nested between curly braces coming after the prelude. Omitting a closing curly brace may be a logic error but is not a syntax error. In the case of a missing closing brace in `@layer`, any styles coming after where the closing brace should have been are parsed as being in the cascade layer defined in the at-rule's prelude. The CSS is valid as there are no syntax errors; nothing is discarded. A syntax error may cause the named or anonymous layer to be empty, but the layer is still created.

Errors in selector lists

There are many ways you might make a mistake writing a selector, but only invalid selectors cause a selector list to be invalid (See invalid selector list).

If you include a class selector, an id_selectors, or a type selector for a class, id, or element (or custom element) that doesn't exist, it may be a logic error but it's not a syntax error. However, if you have a typo in a pseudo-class or a pseudo-element, it might create an invalid selector, which is an error the parser needs to address.

If a selector list contains any invalid selectors, then the entire style block is ignored. There are exceptions: if the invalid selector is within an `:is` or `:where` pseudo-class (which accept forgiving selector lists) or if the unknown selector is a `-webkit-` prefixed pseudo-element, only the unknown selector is ignored as not matching anything. The selector list is not invalidated.

Outside of these exceptions, a single invalid or unsupported selector in the selector list will invalidate the entire rule and the entire selector block will be ignored. The browser will then look for the closing curly brace and continue parsing from that point onwards.

`-webkit-` exception

Due to legacy issues from the overuse of browser-specific prefixes in selectors and property names (and values), browsers avoid excessive invalidation of selector lists by treating all pseudo-elements that start with a case-insensitive `-webkit-` prefix and don't end with `()` as valid.

This means that a pseudo-element like `::-webkit-works-only-in-samsung` will not invalidate a selector list, regardless of which browser the code is running in. In such cases, the pseudo-element may not be recognized or supported by the browser, but it will not cause the entire selector list and its associated style block to be ignored. On the other hand, an unknown prefixed selector with a function notation of `::-webkit-imaginary-function()` will invalidate the entire selector list, and the browser will ignore the entire selector block.

Errors within CSS declaration blocks

When it comes to CSS properties and values within a declaration block, if either the property or the value is invalid, that property-value pair is ignored and discarded. When a user agent parses or interprets a list of declarations, unknown syntax at any point causes the browser's parser to discard only the current declaration. It then continues parsing CSS after the next semicolon or closing curly bracket is encountered, whichever comes first.

This example contains an error. The parser ignores the error (and the comments), seeks forward until it encounters a semi-colon, then restarts parsing:

```
p {
  /* Invalid syntax due to missing semi-colon */
  border-color: red
  background-color: green;
```

```
  /* Valid syntax but likely a logic error */
  border-width: 100vh;
}
```

The reason the first declaration in this selector block is invalid is because the semi-colon is missing and the declaration is not the last one in the selector block. The property missing the semi-colon is ignored, as is the property-value pair following it because the browser only continues parsing after a semi-colon or closing bracket is encountered. Specifically, the `border-color` value is parsed as `red background-color: green;` which is not a valid `<color>` value.

The `border-width` value of `100vh` is likely a mistake, but it's not an error. As it is syntactically valid, it will be parsed and applied to the elements matching the selector.

Vendor prefixes

Vendor-prefixed property names and property values, when not understood by a browser, are treated as invalid and ignored. Only the individual rules containing an invalid property or value are ignored. The parser looks for the next semi-colon or closing curly brace and then continues parsing from there.

You may come across legacy CSS that looks like the following:

```
/* Prefixed values */
.wrapper {
  display: -webkit-box;
  display: -webkit-flex;
  display: -ms-flexbox;
  display: flex;
  display: block flex;
}
/* Prefixed properties */
.rounded {
  -webkit-border-radius: 50%;
  -moz-border-radius: 50%;
  -ms-border-radius: 50%;
  -o-border-radius: 50%;
  border-radius: 50%;
}
```

In this example, the last declaration in each block is valid in all browsers — `display: flex;` and `border-radius: 50%;`. Because of the cascade order of appearance rule, browsers will apply any prefixed declarations they understand, and then override those values with the standard unprefixed version.

> **Note:** Avoid including prefixed properties or property values where possible. If you must use them, declare the prefixed versions before the non-prefixed version as shown above.

Errors with auto-closed endings

If a stylesheet ends while a rule, declaration, function, string, or comment is still open, the parser will automatically close everything that was left unclosed.

> **Note:** This is true of external style sheets, selector blocks within an HTML `<style>` element, and inline rules within a `style` attribute.

If the content between the last semi-colon and the end of the stylesheet is valid, even if incomplete, the CSS will be parsed normally. For example, if you fail to close out a `@keyframes` declaration before closing your `<style>`, the animation is still valid.

```
<style>
@keyframes move {
  100% {
    transform: translateX(100vw)
</style>
```

Here the `move` animation is valid. Failing to properly close CSS statements doesn't necessarily make the statements invalid. That said, do not take advantage of CSS's forgiving nature. Always close all of your statements and style blocks. It makes your CSS more readable and maintainable and ensures that the browser parses the CSS as you intended.

Unclosed comments

Unclosed comments are logic errors, not syntax errors. If a comment starts with `/*` but is not closed, all CSS code until a closing delimiter (`*/`) in a subsequent comment or the end of the stylesheet, whichever comes first, is part of the comment. While an unclosed comment does not invalidate your CSS, it causes the CSS following the opening delimiter (`/*`) to be ignored.

```
<style>
  /* this comment is not closed
  @keyframes move {
    0% {transform: translateX(0);}
    100% {transform: translateX(100vw);}
  }
</style>
<p style="/* another unclosed comment">Parsed as HTML.</p>
```

In this example, the two CSS comments are not closed, but the closing `</style>` tag closes the first comment and the `style` attribute's close quote closes the second comment.

Grammar check

After parsing each declaration, style rule, at-rule, etc., the user agent checks to make sure the grammar follows the rules for that declaration. For example, if a property value is of the wrong data type or a descriptor is not valid for the at-rule being described, the content that does not match the expected grammar is deemed invalid and gets ignored.

Each CSS property accepts specific data types. For example, the `background-color` property accepts either a valid `<color>` or a CSS global keyword. When the value assigned to a property is of the wrong type, such as `background-color: 45deg`, the declaration is invalid and therefore ignored.

Invalid custom properties

Custom properties are generally considered valid when declared, but may create invalid CSS when accessed, i.e., they may be used as a value (via the `var()` function) for a property that does not accept that value type. The browser parses each custom property when encountered without regard to where the property is consumed.

Generally, when a property value is invalid, the declaration is ignored and the property falls back to the last valid value. Invalid computed custom property values, however, work slightly differently.

When a `var()` substitution is invalid, the declaration is not ignored and the initial or inherited value of the property is used instead. The property is set to a new value, but possibly not the expected one.

Let's look at an example to illustrate this behavior:

```
:root {
  --theme-color: 45deg;
}
```

```
body {
  background-color: var(--theme-color);
}
```

In the above code, the custom property declaration is valid. The `background-color` declaration is also valid at compute time. However, when the browser substitutes the custom property in `var(--theme-color)` with `45deg` as a value of the `background-color` property, the grammar is invalid. An `<angle>` is not a valid `background-color` value. In this case, the declaration is not ignored as invalid. Rather, when a custom property is of the wrong type, if the property is inheritable, the value is inherited from its parent. If the property is not inheritable, the default initial value is used. In the case of `background-color`, the property value is not an inherited value, so the initial value of `transparent` is used.

To better control the way custom properties fall back, use the `@property` at-rule to define the initial value of the property:

```
@property --theme-color {
  syntax: "<color>";
  inherits: false;
  initial-value: rebeccapurple;
}
```

See also

- CSS syntax module
- Syntax guide
- Value definition syntax

Introduction to CSS syntax: declarations, rulesets, and statements

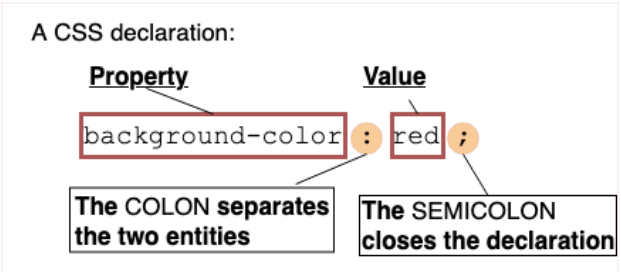
The basic goal of the Cascading Stylesheet (CSS) language is to allow a browser engine to paint elements of the page with specific features, like colors, positioning, or decorations. The *CSS syntax* reflects this goal and its basic building blocks are:

- The **property** which is an identifier, that is a human-readable *name*, that defines which feature is considered.
- The **value** which describe how the feature must be handled by the engine. Each property has a set of valid values, defined by a formal grammar, as well as a semantic meaning, implemented by the browser engine.

CSS declarations

Setting CSS properties to specific values is the core function of the CSS language. A property and value pair is called a **declaration**, and any CSS engine calculates which declarations apply to every single element of a page in order to appropriately lay it out, and to style it.

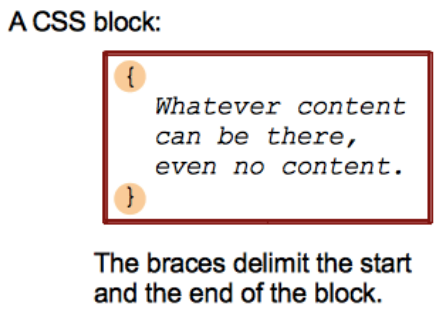
Both properties and values are case-insensitive by default in CSS. The pair is separated by a colon, `:` (U+003A COLON), and white spaces before, between, and after properties and values, but not necessarily inside, are ignored.



There are hundreds of different properties in CSS and a practically endless number of different values. Not all pairs of properties and values are allowed and each property defines what are the valid values. When a value is not valid for a given property, the declaration is deemed *invalid* and is wholly ignored by the CSS engine.

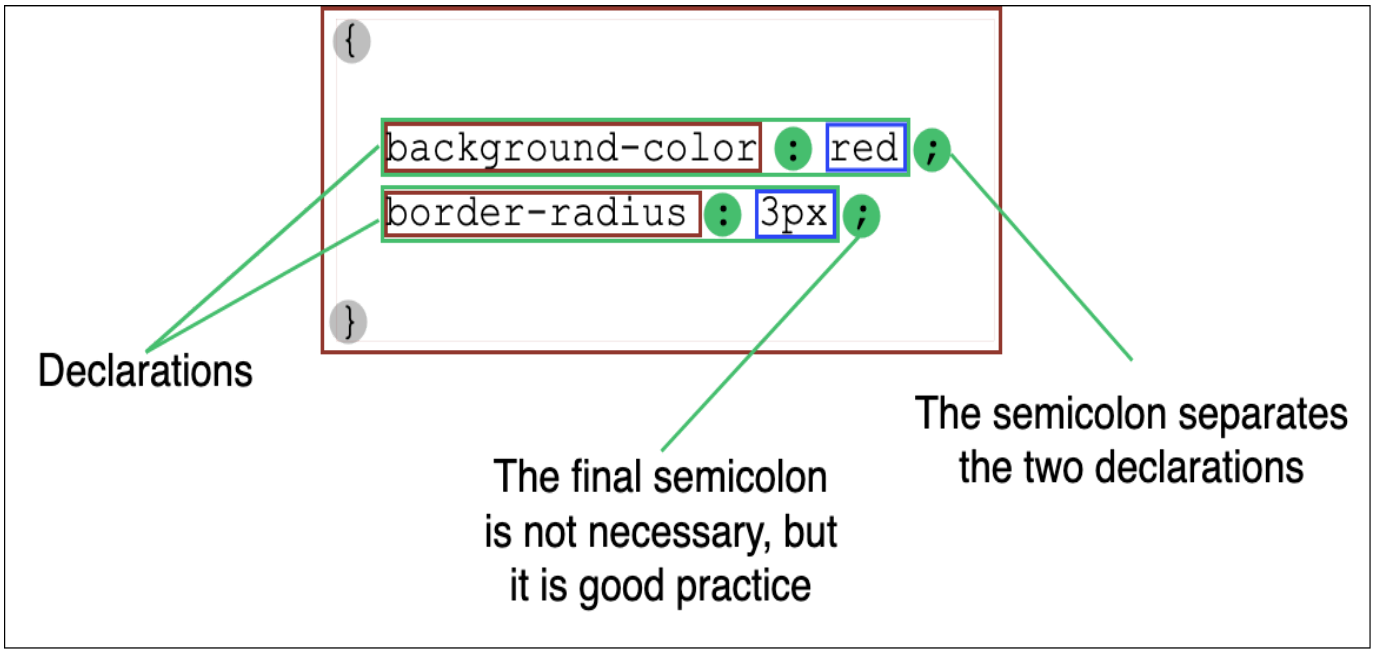
CSS declaration blocks

Declarations are grouped in **blocks**, that is in a structure delimited by an opening brace, `{` (U+007B LEFT CURLY BRACKET), and a closing one, `}` (U+007D RIGHT CURLY BRACKET). Blocks sometimes can be nested, so opening and closing braces must be matched.



Such blocks are naturally called **declaration blocks** and declarations inside them are separated by a semicolon, `;` (U+003B SEMICOLON). A declaration block may be empty, that is containing null declaration. White spaces around declarations are ignored. The last declaration of a block doesn't need to be terminated by a semicolon, though it is often considered *good style* to do it as it prevents forgetting to add it when extending the block with another declaration.

A CSS declaration block is visualized in the diagram below.



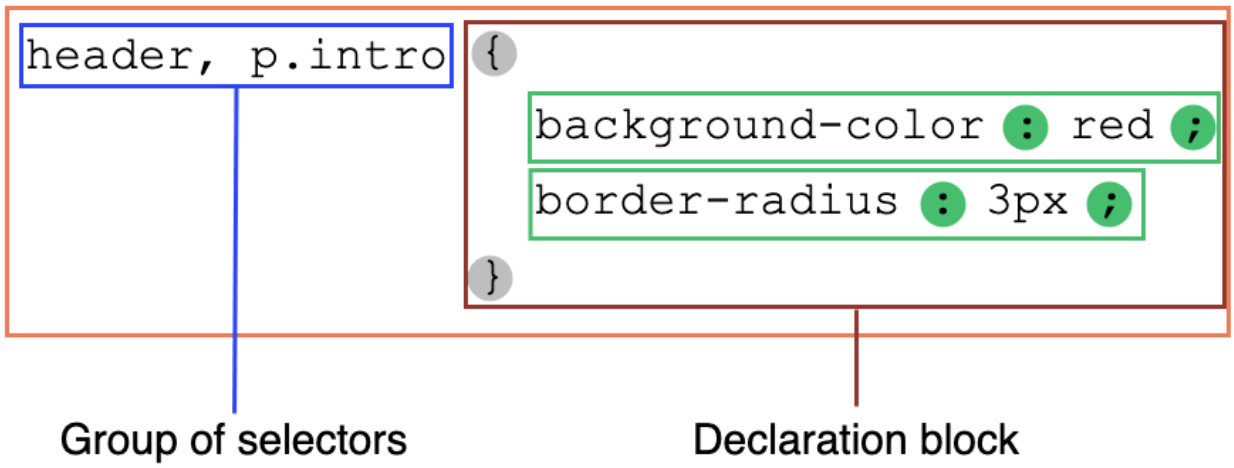
> **Note:** The content of a CSS declaration block, that is a list of semicolon-separated declarations, without the initial and closing braces, can be put inside an HTML `style` attribute.

CSS rulesets

If style sheets could only apply a declaration to each element of a Web page, they would be pretty useless. The real goal is to apply different declarations to different parts of the document.

CSS allows this by associating conditions with declarations blocks. Each (valid) declaration block is preceded by one or more comma-separated **selectors**, which are conditions selecting some elements of the page. A selector list and an associated declarations block, together, are called a **ruleset**, or often a **rule**.

A CSS ruleset (or rule) is visualized in the diagram below.



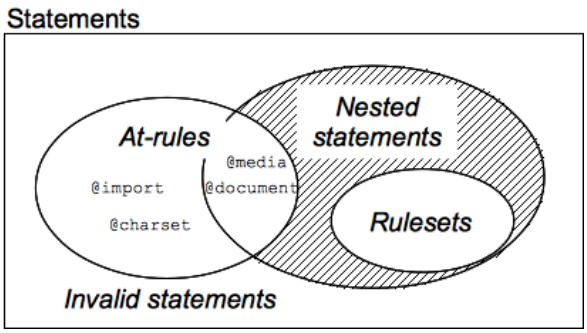
As an element of the page may be matched by several selectors, and therefore by several rules potentially containing a given property several times, with different values, the CSS standard defines which one has precedence over the other and must be applied: this is called the cascade algorithm (see Handling conflicts).

> **Note:** It is important to note that even if a ruleset characterized by a group of selectors is a kind of shorthand replacing rulesets with a single selector each, this doesn't apply to the validity of the ruleset itself.
>
> This leads to an important consequence: if one single basic selector is invalid, like when using an unknown pseudo-element or pseudo-class, the whole *selector* is invalid and therefore the entire rule is ignored (as invalid too).

CSS statements

Rulesets are the main building blocks of a style sheet, which often consists of only a big list of them. But there is other information that a Web author wants to convey in the style sheet, like the character set, other external style sheets to import, font face or list counter descriptions and many more. It will use other and specific kinds of statements to do that.

A **statement** is a building block that begins with any non-space characters and ends at the first closing brace or semicolon (outside a string, non-escaped and not included into another {}, () or [] pair).



There are two kinds of statements:

- **Rulesets** (or *rules*) that, as seen, associate a collection of CSS declarations to a condition described by a selector.
- **At-rules** that start with an at sign, `@` (U+0040 COMMERCIAL AT), followed by an identifier and then continuing up to the end of the statement, that is up to the next semicolon (;) outside of a block, or the end of the next block. Each type of at-rules, defined by the identifier, may have its own internal syntax, and semantics of course. They are used to convey meta-data information (like `@layer` or `@import`), conditional information (like `@media` or `@document`), or descriptive information (like `@font-face`).

Any statement which isn't a ruleset or an at-rule is invalid and ignored.

Nested statements

There is another group of statements – the **nested statements**. These are statements that can be used in a specific subset of at-rules – the *conditional group rules*. These statements only apply if a specific condition is matched: the `@media` at-rule content is applied only if the device on which the browser runs matches the expressed condition; the `@document` at-rule content is applied only if the current page matches some conditions, and so on. In CSS1 and CSS2.1, only *rulesets* could be used inside conditional group rules. That was very restrictive and this restriction was lifted in *CSS Conditionals Level 3*. Now, though still experimental and not supported by every browser, conditional group rules can contain a wider range of content: rulesets but also some, but not all, at-rules.

See also

- CSS syntax module
- Selectors and combinators
- Selector structure
- Error handling
- Specificity
- Inheritance
- Cascade
- Value definition syntax
- Values
  - Initial values
  - Computed values
  - Used values
  - Actual values
- Shorthand properties

CSS table

The **CSS table** module helps you define how to lay out table data.

This CSS module defines styles applicable to the HTML `<table>` element, which is used to render tabular data. By default, tables are rendered as a two-dimensional grid with cells lined up in a series of consecutive rows and columns. This layout is generated from the table structure and sized according to the content of the cells. This module also enables defining the position of the table's `<caption>`, if present.

The properties introduced in this module aren't limited to the `<table>` elements; they can be applied to any element with a table-related CSS `display` value.

Reference

Properties

- `border-collapse`
- `border-spacing`
- `caption-side`
- `empty-cells`
- `table-layout`

Guides

Learn: Styling tables

A guide to improving the appearance of HTML tables, covering table styling techniques.

Learn: HTML table basics

An introduction to HTML tables, including the HTML for creating rows and cells, headings, and making cells span multiple columns and rows.

Learn: HTML table accessibility

A look at advanced HTML table features, including captions and grouping table rows into table head, body and footer sections — as well as looking at the accessibility of tables for visually impaired users.

Related concepts

- `display` property
- `vertical-align` property
- `text-align` property
- CSS box sizing module
  - `box-sizing`
  - `height`
  - `max-width`
  - `min-height`

    - `min-width`
    - `width`
    - `min-content` keyword
- CSS backgrounds and borders module
    - `border` shorthand
    - `border-width`
    - `border-style`
    - `border-color`
- HTML table-related elements:
    - `<table>`
    - `<caption>`
    - `<colgroup>`
    - `<col>`
    - `<thead>`
    - `<tbody>`
    - `<tfoot>`
    - `<tr>`
    - `<th>`
    - `<td>`

> **Note:** The CSS 2.2 specification defines stable standards for web styling, including detailed specifications for table formatting. The CSS Table Module Level 3 specification seeks to expand these capabilities with advanced features for table layout and rendering. However, the table module specification is still being developed and is not yet ready for implementation.

See also

- CSS display module
- CSS grid layout module
- CSS flexible box layout module
- CSS fragmentation module

CSS text

The **CSS text** module defines how to perform text manipulation, like line breaking, justification and alignment, white space handling, and text transformation.

Reference

Properties

- `hanging-punctuation`
- `hyphenate-character`
- `hyphenate-limit-chars`
- `hyphens`
- `letter-spacing`
- `line-break`
- `overflow-wrap` (and the `word-wrap` alias)
- `tab-size`
- `text-align`
- `text-align-last`
- `text-fit`
- `text-indent`
- `text-justify`
- `text-autospace`
- `text-spacing-trim`
- `text-transform`
- `text-wrap`
- `text-wrap-mode`
- `text-wrap-style`
- `white-space`
- `white-space-collapse`
- `word-break`
- `word-spacing`

The specification also defines the `hyphenate-limit-last`, `hyphenate-limit-lines`, `hyphenate-limit-zone`, `line-padding`, `text-align-all`, `text-group-align`, `text-spacing`, `white-space-trim`, `word-space-transform`, `wrap-after`, `wrap-before`, and `wrap-inside` properties, which are not yet supported by any browser.

Guides

Wrapping and breaking text

A guide to the various ways in which overflowing text can be managed in CSS.

Handling whitespace

A guide introducing the various CSS properties that control the rendering of whitespace in the DOM, exploring the pitfalls and their mitigations.

Related concepts

Properties

- `direction`
- `font-feature-settings`
- `initial-letter`
- `orphans`
- `text-combine-upright`
- `text-orientation`
- `text-overflow`
- `text-size-adjust`

- `unicode-bidi`
- `widows`
- `writing-mode`

Values

- `min-content`
- `max-content`

HTML

- `<pre>`
- `<wbr>`

See also

- CSS writing modes module
- CSS overflow module
- CSS fonts module
- CSS ruby layout module
- CSS text decoration module

Handling whitespace

The presence of whitespace in the DOM can cause layout problems and make manipulation of the content tree difficult in unexpected ways, depending on where it is located. This article explores when difficulties can occur, and looks at what can be done to mitigate resulting problems.

What is whitespace?

Whitespace characters consist of different characters in different programming language contexts. *Document white space characters*, as far as the CSS whitespace processing rules are concerned, only include spaces (U+0020), tabs (U+0009), line feeds (LF, U+000A), and carriage returns (CR, U+000D), where CR characters are equivalent to spaces in every regard. These characters allow you to format your code for readability. Much of our source code is full of these whitespace characters, and we tend to remove them only as part of a production build step to reduce file size.

Note that this list does not include non-breaking spaces (U+00A0, ` ` in HTML). So these characters don't trigger any collapsing, which is why they are often used to create longer spaces in HTML.

CSS also defines the concept of *segment breaks*, which in the context of HTML are equivalent to LF characters.

How does HTML process whitespace?

It is a common myth that "HTML ignores whitespace", which is untrue: **HTML preserves all whitespace text content as you wrote them in the source code.** As a markup language, HTML produces a DOM where all whitespace in the text content is preserved, which can be retrieved and manipulated via DOM APIs such as `Node.textContent`. If HTML strips whitespace from the DOM, then CSS, a downstream rendering engine that works on the DOM, would not be able to preserve them using the `white-space` property.

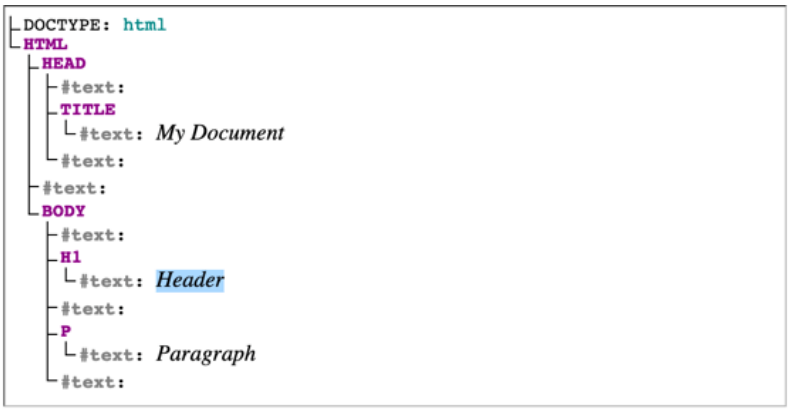
> **Note:** To be clear, we're talking about whitespace *between HTML tags*, which becomes text nodes in the DOM. Any whitespace *inside a tag* (between the angle brackets but not as part of an attribute value) is just part of the HTML syntax and does not appear in the DOM.

> **Note:** Due to the magic that is HTML parsing (quote from DOM spec), there do exist certain places where whitespace characters could be ignored. For example, whitespace between the `<html>` and `<head>` opening tags or between the `</body>` and `</html>` closing tags is ignored and does not appear in the DOM. Also, when parsing the `<pre>` element's text content, a single leading newline character gets stripped. We ignore these edge cases.
>
> Furthermore, the HTML parser does *normalize* certain whitespaces: it replaces CR and CRLF sequences with a single LF. However, CR characters can also be inserted into the DOM either via character references or JavaScript, so the CSS whitespace processing rules still need to define how to handle them.

Take the following document, for example:

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>My Document</title>
  </head>
  <body>
    <h1>Header</h1>
    <p>Paragraph</p>
  </body>
</html>
```

The DOM tree for this looks like so:



Note that:

- Some text nodes will contain only whitespace.
- Other text nodes may have whitespace at the beginning or the end.

> **Note:** Firefox DevTools supports highlighting text nodes, enabling you to see exactly which nodes contain whitespace characters. Pure whitespace nodes are marked with a "whitespace" label.

Conserving whitespace characters in the DOM is useful in many ways, but it can also make certain layouts more difficult to implement and may cause problems for developers who want to iterate over DOM nodes. We'll look at these issues and some solutions later on, in the solving common problems with whitespace nodes section.

How does CSS process whitespace?

When the DOM is passed to CSS for rendering, the whitespace is largely stripped by default. This means that the way your code is formatted is not visible to the end user—creating space around and inside elements is the job of CSS.

```
<!doctype html>

  <h1>      Hello      World!     </h1>
```

This source code contains a couple of line feeds after the `doctype` and a bunch of space characters before, after, and inside the `<h1>` element. But the browser ignores these spaces and just shows the words "Hello World!" as if these characters didn't exist at all:

CSS ignores most, but not all, whitespace characters. In this example, one of the spaces between "Hello" and "World!" still exists when the page is rendered in a browser. CSS uses a specific algorithm to decide which whitespace characters are user-irrelevant and how they are removed or transformed. We'll explain how this processing works in the next few sections.

Collapsing and transformation

Let's look at an example. To make the whitespace characters more salient, we've also added a comment to show all spaces as ◦, all tabs as ↹, and all line breaks as ⏎:

```
<h1>   Hello
                                <span> World!</span>      </h1>

<!--
<h1>◦◦◦Hello◦⏎
↹↹↹↹<span>◦World!</span>↹◦◦</h1>
-->
```

This example is rendered in the browser like so:

The `<h1>` element contains:

- A text node (consisting of some spaces, the word "Hello", a line break, and some tabs).
- An inline element (`<span>`, which contains a space, and the word "World!").
- Another text node (with a tab and spaces after the `<span>`).

Because this `<h1>` element contains only inline elements, it establishes an inline formatting context. This is one of the several layout rendering contexts that browser engines use to arrange content on the page.

Inside this inline formatting context, whitespace characters are processed as follows:

> **Note:** This algorithm can be configured via the `white-space-collapse` property (or its shorthand property `white-space`). We'll start by assuming its default value (`white-space-collapse: collapse`), then look at how different property values affect this algorithm.

1. First, all spaces and tabs immediately before and after a line break are ignored. So, if we take our example markup from before:

   ```
   <h1>◦◦◦Hello◦⏎
   ↹↹↹↹<span>◦World!</span>↹◦◦</h1>
   ```

   ...and apply this first rule, we get:

   ```
   <h1>◦◦◦Hello⏎
   <span>◦World!</span>↹◦◦</h1>
   ```

2. Next, consecutive line breaks are collapsed into a single line break. We don't have any in this example.

3. Next, lines in the source code are joined into single lines by removing any remaining line break characters. They are either transformed into spaces (U+0020) or simply removed, depending on the context before and after the break. The exact choice between the two is browser- and language-dependent. In our example here in English (where spaces separate words), we can expect all line breaks to be "transformed" into spaces. So we end up with:

   ```
   <h1>◦◦◦Hello◦<span>◦World!</span>↹◦◦</h1>
   ```

   Notably, in languages that have no word separators, such as Chinese, lines are joined with no intervening space. So:

   ```
   <div>你好
   世界</div>
   ```

   might be rendered as "你好世界" without any spaces in between, depending on the browser's heuristics.

4. Next, all tab characters are transformed into space characters, so the example becomes:

   ```
   <h1>◦◦◦Hello◦<span>◦World!</span>◦◦◦</h1>
   ```

5. After that, any space immediately following another space (even across two separate inline elements) is ignored, so we end up with:

   ```
   <h1>◦Hello◦<span>World!</span>◦</h1>
   ```

This is why people visiting the web page will see the phrase "Hello World!" nicely written at the top of the page, rather than a weirdly indented "Hello" followed by an even more weirdly indented "World!" on the next line.

After these steps, the browser processes line wrapping and bidirectional text, which we will ignore here. Note that there are still spaces left after the opening `<h1>` tag and before the closing `</h1>` tag, but these are not rendered in the browser. We'll handle that next, as each line is laid out.

Different `white-space-collapse` values skip different steps of this algorithm:

- `preserve` and `break-spaces`: the whole algorithm is skipped, and no whitespace collapsing or transformation happens.
- `preserve-breaks`: steps 2 and 3 are skipped, and line breaks are preserved.
- `preserve-spaces`: the whole algorithm is skipped and replaced with a single step to convert each tab or line break into a space.

In a nutshell, different whitespace characters are collapsed and transformed in the following fashion:

- Tabs are generally converted to spaces.
- If segment breaks are to be collapsed:
  - Sequences of segment breaks are collapsed to a single segment break.
  - They are converted to spaces in languages that separate words with spaces (like English), or removed altogether in languages that do not separate words with spaces (like Chinese).
- If spaces are to be collapsed:
  - Spaces or tabs before or after segment breaks are removed.
  - Sequences of spaces are collapsed to a single space.
- When spaces are preserved, sequences of spaces are treated as non-breaking, except that they will soft wrap at the end of each sequence — that is, the next line will always start with the next non-space character. In the case of the `break-spaces` value, however, a soft wrap could potentially occur after each space, so the next line may start with one or more spaces.

Trimming and positioning

In both inline and block formatting contexts, elements are laid out in *lines*. In an inline formatting context, lines are created by text wrapping. In a block formatting context, on the other hand, each block forms its own line. As each line is laid out, whitespace is processed further. Let's take a look at an example to explain how this works.

In this example, as before, we've marked the whitespace characters in a comment. We have three text nodes that contain only whitespace: one before the first `<div>`, one between the 2 `<div>`s, and one after the second `<div>`.

```
<body>
        <div>   Hello   </div>

   <div>  World!   </div>
</body>

<!--
<body>⏎
→⇥<div>→⇥Hello→⇥</div>⏎
⏎
◦◦◦<div>◦◦World!◦◦◦</div>◦◦⏎
</body>
-->
```

This renders like so:

The whitespace in this example is handled as follows:

> **Note:** This algorithm can be configured via the `white-space-collapse` property (or its shorthand property `white-space`). We'll start by assuming its default value (`white-space-collapse: collapse`), then look at how different property values affect this algorithm.

1. First, the whitespace is collapsed the same way as we saw in the previous section, turning this:

   ```
   <body>⏎
   →⇥<div>→⇥Hello→⇥</div>⏎
   ⏎
   ◦◦◦<div>◦◦World!◦◦◦</div>◦◦⏎
   </body>
   ```

   ...into this:

   ```
   <body>◦<div>◦Hello◦</div>◦<div>◦World!◦</div>◦</body>
   ```

   Lines are then laid out according to the block formatting context established by `<body>`. In this example, each of `<body>`'s five child nodes is laid out as a separate line. (Each line in this code block represents a line in the rendered layout, not a line in our original HTML code):

   ```
   <body>
   ◦
   <div>◦Hello◦</div>
   ◦
   <div>◦World!◦</div>
   ◦
   </body>
   ```

   Note that if the lines become too long, each line may wrap and create more lines. In reality, browsers determine the content of the lines as the lines are laid out. We'll skip the part about how text wrapping works.

2. Sequences of spaces at the beginning of a line are removed, so the example becomes:

   ```
   <body>

   <div>Hello◦</div>

   <div>World!◦</div>

   </body>
   ```

3. Each tab that's preserved at this point is rendered according to `tab-size`. This can only happen with `white-space-collapse` set to `preserve` or `break-spaces` because all other settings turn tabs into spaces.

4. Sequences of spaces at the end of a line are removed, so the above becomes:

   ```
   <body>

   <div>Hello</div>

   <div>World!</div>

   </body>
   ```

The three empty lines we now have are not going to occupy any space in the final layout, because they don't contain any visible content. So we'll end up with only two lines taking up space in the page. People viewing the web page see the words "Hello" and "World!" on two separate lines, just as you'd expect two `<div>`s to be laid out. Browsers essentially ignore all of the whitespace that was included in the HTML code.

Different `white-space-collapse` values skip different steps of this algorithm:

- `preserve` and `break-spaces`: The whole algorithm is skipped except for step 3, so no whitespace collapsing or transformation happens.
- `preserve-spaces`: The whole algorithm is skipped, so whitespace characters at the start and end of lines are preserved.
- `preserve-breaks`: The same algorithm is applied as with the `collapse` value.

How do DOM APIs process whitespace?

As mentioned previously, whitespace is preserved in the DOM. This means that if you retrieve `Node.textContent`, you will get the text content as you wrote it in the HTML source code, and if you retrieve `Node.childNodes`, you will get all the text nodes, including those that contain only whitespace.

Not all DOM APIs preserve whitespace; some APIs deal with the *rendered text* by design. For example, `HTMLElement.innerText` returns the text exactly as it's rendered, with all whitespace collapsed and trimmed. `Selection.toString()` returns the text as it would be pasted, which generally means that whitespace is collapsed. However, in Firefox (which collapses whitespace between Chinese characters, as mentioned in the collapsing and transformation section above), the collapsed whitespace is still preserved both in the string returned by `toString()` and in the pasted text.

```
<div id="test">Hello world!</div>
```

```
const div = document.getElementById("test");
console.log(div.textContent); // "  Hello\n  world!\n"
console.log(div.innerText); // "Hello world!"
const selection = document.getSelection();
selection.selectAllChildren(div);
console.log(selection.toString()); // "Hello world!"
```

Solving common problems with whitespace nodes

Whitespace nodes are invisible to the website visitor due to the CSS processing rules, but they can interfere with certain layouts and DOM manipulation that rely on the exact structure of the DOM. Let's look at some common problems and how to solve them.

Whitespace processing between inline and inline-block elements

Let's look at one layout issue with whitespace nodes: spaces between inline and inline-block elements. As we saw earlier with inline and block elements, most whitespace characters are ignored, but word-separating characters like spaces remain. The extra whitespace that does make it to the layout is helpful to separate the words in the sentence.

With `inline-block` elements, it gets more interesting: these elements behave like inline elements on the outside and blocks on the inside. (They're often used to display more complex pieces of UI, placed side by side on the same line, such as navigation menu items.) Any whitespace between adjacent inline or inline-block elements will result in spaces in the layout, just like the spaces between words in text. (This can surprise developers because they are blocks, and blocks don't normally show extra spaces.)

Consider this example (as before, we've included a comment in the HTML code to show the whitespace characters):

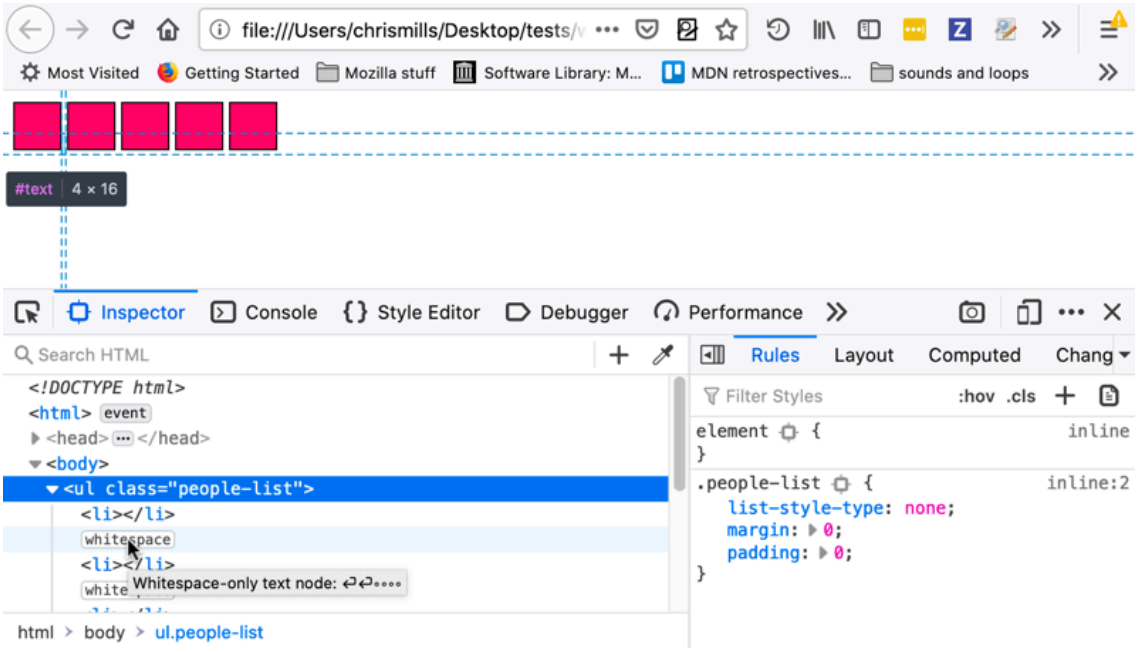
```
.people-list {
  list-style-type: none;
  margin: 0;
  padding: 0;
}

.people-list li {
  display: inline-block;
  width: 2em;
  height: 2em;
  background: #ff0066;
  border: 1px solid;
}
```

```
<ul class="people-list">
  <li></li>

  <li></li>

  <li></li>

  <li></li>

  <li></li>
</ul>

<!--
<ul class="people-list">⏎
◦◦<li></li>⏎
⏎
◦◦<li></li>⏎
⏎
◦◦<li></li>⏎
⏎
◦◦<li></li>⏎
⏎
◦◦<li></li>⏎
</ul>
-->
```

This renders as follows:

You probably don't want the gaps between the blocks. Depending on your use case (such as a list of avatars or a horizontal row of navigation buttons), you probably want the elements to flush against each other, and to be able to control any spacing yourself.

The Firefox DevTools HTML Inspector can highlight text nodes and also show you exactly the area the elements are taking up. This is useful to check out if you suspect there's extra margin or unexpected whitespace causing gaps.



There are a few ways of getting around this problem:

- Use Flexbox to create the horizontal list of items instead of trying an `inline-block` solution. Flexbox handles spacing and alignment for you, and is definitely the preferred solution:

```
ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  display: flex;
}
```

- If you need to rely on `inline-block`, you could set the `font-size` of the list to `0`. This only works if the blocks are not sized with `em` units (since `em` is based on `font-size`, the block size would also end up being sized as `0`). Using `rem` units would be a good choice here:

```
ul {
  font-size: 0;
  /* … */
}

li {
  display: inline-block;
  width: 2rem;
  height: 2rem;
  /* … */
}
```

- Alternatively, you could set negative margin on the list items:

```
li {
  display: inline-block;
  width: 2rem;
  height: 2rem;
  margin-right: -0.25rem;
}
```

- You can also solve this problem by avoiding whitespace nodes between `<li>` items:

```
<li>
  ...
</li><li>
  ...
</li>
```

Working with whitespace in the DOM

As mentioned previously, whitespace is collapsed and trimmed when rendered, but preserved in the DOM. This may present some pitfalls when trying to do DOM manipulation in JavaScript. For example, if you have a reference to a parent node and want to manipulate its first element child using `Node.firstChild`, a rogue whitespace node just after the opening parent tag will give you the wrong result. The text node would be selected instead of the element you want to target.

As another example, if you want to do something to a subset of elements based on whether they are empty (have no child nodes), you could use `Node.hasChildNodes()`. But if any of those elements contain text nodes, you could end up with false results.

The following JavaScript code shows several functions that deal with whitespace in the DOM:

```
/**
 * Throughout, whitespace is defined as one of the characters
 *  "\t" TAB \u0009
 *  "\n" LF  \u000A
 *  "\r" CR  \u000D
 *  " "  SPC \u0020
 *
 * This does not use JavaScript's "\s" because that includes non-breaking
 * spaces (and also some other characters).
 */
```

```
/**
 * Determine whether a node's text content is entirely whitespace.
 *
 * @param nod  A node implementing the `CharacterData` interface (i.e.,
 *             a `Text`, `Comment`, or `CDATASection` node)
 * @return     `true` if all of the text content of `nod` is whitespace,
 *             otherwise `false`.
 */
function isAllWs(nod) {
  return !/[^\t\n\r ]/.test(nod.textContent);
}

/**
 * Determine if a node should be ignored by the iterator functions.
 *
 * @param nod  An object implementing the `Node` interface.
 * @return     `true` if the node is:
 *                1) A `Text` node that is all whitespace
 *                2) A `Comment` node
 *             and otherwise `false`.
 */
function isIgnorable(nod) {
  return (
    nod.nodeType === 8 || // a comment node
    (nod.nodeType === 3 && isAllWs(nod))
  ); // a text node, all ws
}

/**
 * Version of `previousSibling` that skips nodes that are entirely
 * whitespace or comments. (Normally `previousSibling` is a property
 * of all DOM nodes that gives the sibling node, the node that is
 * a child of the same parent, that occurs immediately before the
 * reference node.)
 *
 * @param sib  The reference node.
 * @return     The closest previous sibling to `sib` that is not
 *             ignorable according to `isIgnorable`, or `null` if
 *             no such node exists.
 */
function nodeBefore(sib) {
  while ((sib = sib.previousSibling)) {
    if (!isIgnorable(sib)) {
      return sib;
    }
  }
  return null;
}

/**
 * Version of `nextSibling` that skips nodes that are entirely
 * whitespace or comments.
 *
 * @param sib  The reference node.
 * @return     The closest next sibling to `sib` that is not
 *             ignorable according to `isIgnorable`, or `null`
 *             if no such node exists.
 */
function nodeAfter(sib) {
  while ((sib = sib.nextSibling)) {
    if (!isIgnorable(sib)) {
      return sib;
    }
  }
  return null;
}

/**
 * Version of `lastChild` that skips nodes that are entirely
 * whitespace or comments. (Normally `lastChild` is a property
 * of all DOM nodes that gives the last of the nodes contained
 * directly in the reference node.)
 *
 * @param sib  The reference node.
 * @return     The last child of `sib` that is not ignorable
 *             according to `isIgnorable`, or `null` if no
 *             such node exists.
 */
function lastChild(par) {
  let res = par.lastChild;
  while (res) {
    if (!isIgnorable(res)) {
      return res;
    }
    res = res.previousSibling;
  }
  return null;
}

/**
 * Version of `firstChild` that skips nodes that are entirely
 * whitespace and comments.
 *
 * @param sib  The reference node.
 * @return     The first child of `sib` that is not ignorable
 *             according to `isIgnorable`, or `null` if no
 *             such node exists.
 */
function firstChild(par) {
  let res = par.firstChild;
  while (res) {
    if (!isIgnorable(res)) {
      return res;
```

```
    }
    res = res.nextSibling;
  }
  return null;
}

/**
 * Version of `data` that doesn't include whitespace at the beginning
 * and end and normalizes all whitespace to a single space. (Normally
 * `data` is a property of text nodes that gives the text of the node.)
 *
 * @param txt  The text node whose data should be returned
 * @return     A string giving the contents of the text node with
 *             whitespace collapsed.
 */
function dataOf(txt) {
  let data = txt.textContent;
  data = data.replace(/[\t\n\r ]+/g, " ");
  if (data[0] === " ") {
    data = data.substring(1, data.length);
  }
  if (data[data.length - 1] === " ") {
    data = data.substring(0, data.length - 1);
  }
  return data;
}
```

The following code demonstrates the use of the functions above. It iterates over the children of an element (whose children are all elements) to find the one whose text is `"This is the third paragraph"`, and then changes the class attribute and the contents of that paragraph.

```
let cur = firstChild(document.getElementById("test"));
while (cur) {
  if (dataOf(cur.firstChild) === "This is the third paragraph.") {
    cur.className = "magic";
    cur.firstChild.textContent = "This is the magic paragraph.";
  }
  cur = nodeAfter(cur);
}
```

Wrapping and breaking text

This guide explains the various ways in which overflowing text can be managed in CSS.

What is overflowing text?

In CSS, if you have an unbreakable string such as a very long word, by default it will overflow any container that is too small for it in the inline direction. We can see this happening in the example below: the long word is extending past the boundary of the box it is contained in.

```
<div class="box">
  Llanfairpwllgwyngyllgogerychwyrndrobwllllantysiliogogogoch
</div>
```

```
.box {
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
  inline-size: 150px;
}
```

CSS will display overflow in this way, because doing something else could cause data loss. In CSS data loss means that some of your content vanishes. So the initial value of `overflow` is `visible`, and we can see the overflowing text. It is generally better to be able to see overflow, even if it is messy. If things were to disappear or be cropped as would happen if `overflow` was set to `hidden` you might not spot it when previewing your site. Messy overflow is at least noticeable, and in the worst case, your visitor will be able to see and read the content even if it looks a bit strange.

In this next example, you can see what happens if `overflow` is set to `hidden`.

```
<div class="box">
  Llanfairpwllgwyngyllgogerychwyrndrobwllllantysiliogogogoch
</div>
```

```
.box {
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
  inline-size: 150px;
  overflow: hidden;
}
```

Finding the min-content size

To find the minimum size of the box that will contain its contents with no overflows, set the `width` or `inline-size` property of the box to `min-content`.

```
<div class="box">
  Llanfairpwllgwyngyllgogerychwyrndrobwllllantysiliogogogoch
</div>
```

```
.box {
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
  inline-size: min-content;
}
```

Using `min-content` is therefore one possibility for overflowing boxes. If it is possible to allow the box to grow to be the minimum size required for the content, but no bigger, using this keyword will give you that size.

Breaking long words

If the box needs to be a fixed size, or you are keen to ensure that long words can't overflow, then the `overflow-wrap` property can help. This property will break a word once it is too long to fit on a line by itself.

```
<div class="box">
  Llanfairpwllgwyngyllgogerychwyrndrobwllllantysiliogogogoch
</div>
```

```
.box {
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
  inline-size: 150px;
  overflow-wrap: break-word;
}
```

> **Note:** The `overflow-wrap` property acts in the same way as the non-standard property `word-wrap`. The `word-wrap` property is now treated by browsers as an alias of the standard property.

An alternative property to try is `word-break`. This property will break the word at the point it overflows. It will cause a break-even if placing the word onto a new line would allow it to display without breaking.

In this next example, you can compare the difference between the two properties on the same string of text.

```
<div class="box box1">A Very LongWordThatHasNoBreakingPossibilities</div>

<div class="box box2">A Very LongWordThatHasNoBreakingPossibilities</div>
```

```
.box {
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
  inline-size: 30ch;
  margin-block-end: 1em;
}
.box1 {
  word-break: break-all;
}

.box2 {
  overflow-wrap: break-word;
}
```

This might be useful if you want to prevent a large gap from appearing if there is just enough space for the string. Or, where there is another element that you would not want the break to happen immediately after.

In the example below there is a checkbox and label. Let's say, you want the label to break should it be too long for the box. However, you don't want it to break directly after the checkbox.

```
<div class="field">
  <input id="one" type="checkbox" /><label for="one">
    LongWordThatHasNoBreakingPossibilities
  </label>
</div>

<div class="field field-br">
  <input id="two" type="checkbox" /><label for="two">
    LongWordThatHasNoBreakingPossibilities
  </label>
</div>
```

```
.field {
  inline-size: 150px;
  border: 1px solid #cccccc;
  margin-block-end: 1em;
  padding: 10px;
}

.field-br {
  word-break: break-all;
}
```

Adding hyphens

To add hyphens when words are broken, use the CSS `hyphens` property. Using a value of `auto`, the browser is free to automatically break words at appropriate hyphenation points, following whatever rules it chooses. To have some control over the process, use a value of `manual`, then insert a hard (U+2010) or soft break character (U+00AD) into the string. A hard break character can be added using `-` or `‐`, and a soft break character can be added using the ``, ``, or `` HTML character codes. A hard break will always break, even if it is not necessary to do so. A soft break only breaks if breaking is needed.

```
<div class="box">
  Llanfair&shy;pwllgwyngyll&shy;gogerychwyrndrobwllllantysiliogogogoch
</div>
```

```
.box {
  inline-size: 150px;
  overflow-wrap: break-word;
  hyphens: manual;
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
}
```

You can also use the `hyphenate-character` property to use the string of your choice instead of the default hyphenation character at the end of the line (before the hyphenation line break) for the language. The `auto` value selects the correct value to mark a mid-word line break according to the typographic conventions of the current content language.

CSS provides additional hyphenation control: the `hyphenate-limit-chars` property can be used to set the minimum word length that allows for hyphenation as well as the minimum number of characters before and after the hyphen.

The <code><wbr></code> element

If you know where you want a long string to be allowed to break, you can also use the HTML `<wbr>` element. The `<wbr>` element introduces a line break opportunity—if the word overflows, it will wrap at this position. This allows you to preserve meaningful segments in the string, such as long URLs.

In the example below, the browser may break the text at the location of the `<wbr>`.

```
<div class="box">
  Llanfair<wbr />pwllgwyngyll<wbr />gogerychwyrndrobwllllantysiliogogogoch
</div>
```

```
.box {
  border: 4px solid #f76707;
  border-radius: 5px;
  padding: 10px;
  inline-size: 150px;
}
```

See also

- The HTML `<wbr>` element
- The CSS `word-break` property
- The CSS `overflow-wrap` property
- The CSS `white-space` property
- The CSS `text-wrap` property
- The CSS `hyphens` property
- The CSS `hyphenate-character` property
- The CSS `hyphenate-limit-chars` property
- Overflow and Data Loss in CSS

CSS text decoration

The **CSS text decoration** module defines features relating to text decoration, such as underlines, text shadows, and emphasis marks. Text decoration features can provide visual cues for spelling errors, grammar issues, and links. These features can help improve the usability, accessibility, functionality, and aesthetics of your text.

By varying the color, style, and thickness of text decoration using pseudo-classes and pseudo-elements, you can highlight the emphasis in text in ways that don't rely on color alone.

Several features can help improve text readability:

- `text-decoration-skip-ink` can increase legibility by skipping descenders.
- `text-underline-offset` enables you to fine-tune the underline placement to better match font metrics or design aesthetics, which can be especially useful for unique typefaces.
- `text-shadow` colors that contrast with text `color` can make the text visually pop when placed on a background with insufficient contrast.

These features all help improve legibility, and therefore, accessibility. Reducing visual noise and improving text clarity is especially helpful for users with dyslexia or low vision.

In the writing systems of some languages, overlines and underlines carry semantic meaning. CSS lets you adapt styling to cultural norms. Text decoration features are especially useful for languages that do not use Latin-based scripts and instead follow different underline placements, such as Japanese and Korean.

Text decoration features also enable you to follow editorial and localization standards that come from print media. For example, with a strikethrough (line-through), you can indicate the status of content. Use it to inform users that content has been removed or that prices have been halved. This feature lets you present both the original and the updated content. Overlines or double underlines are commonly used for academic and editorial annotations.

Text decoration in action

```
p {
  margin: 1em 0;
}
.under {
  text-decoration: underline red;
}

.over {
  text-decoration: wavy overline lime;
}

.line {
  text-decoration: line-through;
}

.underover {
  text-decoration: dashed underline overline;
}

.thick {
  text-decoration: solid underline purple 4px;
  text-underline-offset: -5px;
}
.shadow {
  text-shadow: red 4px 4px 0;
}
.emphasis {
  text-emphasis: triangle red;
}
```

```
<p class="under">A red underline</p>
<p class="over">A wavy lime overlines</p>
<p class="line">This has a line-through it</p>
<p class="underover">This has a dashed underline and overline</p>
<p class="thick">An offset solid purple underline</p>
<p class="shadow">This has a red shadow</p>
<p class="emphasis">Emphasized with red triangles</p>
```

Reference

Properties

- `text-decoration`
- `text-decoration-color`

- `text-decoration-inset`
- `text-decoration-line`
- `text-decoration-skip`
- `text-decoration-skip-ink`
- `text-decoration-style`
- `text-decoration-thickness`
- `text-emphasis`
- `text-emphasis-color`
- `text-emphasis-position`
- `text-emphasis-style`
- `text-shadow`
- `text-underline-offset`
- `text-underline-position`

The specification also defines the `text-decoration-skip-box`, `text-decoration-skip-self`, `text-decoration-skip-spaces`, and `text-emphasis-skip` properties, which are not yet supported by any browser.

Guides

Introduction to text shadows

Overview of the components of the `text-shadow` property and creating multiple text shadows

Related concepts

- `::spelling-error`
- `::grammar-error`
- `::first-letter`
- `::first-line`
- `box-shadow`
- `drop-shadow()`
- `box-decoration-break`
- `<line-style>`
- `letter-spacing`
- `word-spacing`
- `font-size`
- `font-variant-position`
- `font-kerning`
- `ruby-overhang`
- `<ruby>`
- `<rt>`
- `<rp>`
- `<sup>`
- `<sub>`
- Inline formatting context
- Introduction to formatting contexts

See also

- CSS fonts module
- CSS ruby layout module
- CSS text module
- CSS writing modes module
- CSS overflow module
- CSS fonts module

Introduction to text shadows

You can apply shadows to your text by using the `text-shadow` property. This property accepts a comma-separated list of shadow values. Each shadow requires at least two `<length>` values but can include up to three `<length>` values and one `color` value.

```
text-shadow: 1px 3px;
text-shadow: 1px -2px 3px white;
text-shadow:
  5px 5px mediumblue,
  10px 10px magenta,
  15px 15px rebeccapurple;
```

To remove any shadow from the text, use the keyword `none`.

```
text-shadow: none;
```

In this guide, we look at the components of text shadows and how you can apply multiple text shadows to an element.

Components of text shadows

Each shadow includes a horizontal offset, a vertical offset, and an optional blur radius, in that order. You can also define the color of the shadow.

Horizontal offset

The first `<length>` in the value of `text-shadow` represents the horizontal offset of the shadow relative to the original text. Positive values move the shadow to the right, while negative values move it to the left. A value of `0` is a common valid value.

In this example, the different `text-shadow` declarations differ only in their horizontal offsets. The first `<length>` value moves the shadow to the left (`-30px`) or to the right (`30px`).

```
.negative {
  text-shadow: -30px 0 1px red;
}

.positive {
  text-shadow: 30px 0 1px red;
}

.zero {
```

```
  text-shadow: 0 0 1px red;
}
```

```
<p class="negative">Negative</p>
<p class="positive">Positive</p>
<p class="zero">Zero</p>
```

```
p {
  border: 1px solid;
  padding: 20px;
  font-family: sans-serif;
  font-size: 1.25rem;
}
```

You may have noticed that the `text-shadow` property has no effect on the CSS box model, similar to the `outline` property. Just like `box-shadow`, text shadows do not affect layout, do not trigger scrolling, and do not impact the size of the scrollable overflow area. While text shadows can make an element's text appear bigger, they have no actual impact on the width (or height) of content.

Vertical offset

The second `<length>` in the value of `text-shadow` represents the vertical offset of the shadow relative to the original text. This required value behaves similarly to the horizontal offset, except that it moves the shadow up or down rather than left or right.

In this example, the `text-shadow` declarations differ only in their vertical offsets. The second `<length>` value moves the shadow up (`-30px`) or down (`30px`).

```
.negative {
  text-shadow: 0 -30px 1px red;
}

.positive {
  text-shadow: 0 30px 1px red;
}

.zero {
  text-shadow: 0 0 1px red;
}
```

Blur radius

The blur radius is defined by the third `<length>` value and is optional. If omitted, the blur radius is `0`, creating a copy of the text positioned by the first two length values. The value must be `0` or greater; the greater the value, the more widely dispersed the shadow effect will be.

In this example, the `text-shadow` declarations differ only in their blur radii. This third `<length>` value is either invalid (`-5px`), blurs the shadow (`5px`), or creates a copy of the text (`0`).

```
.negative {
  /* invalid */
  text-shadow: 30px 30px -5px red;
}

.positive {
  text-shadow: 30px 30px 5px red;
}

.zero {
  text-shadow: 30px 30px 0 red;
}
```

Shadow color

While you can apply multiple shadows to text, each shadow consists of a single base color. That color can be any valid CSS `color` value, defaulting to `currentcolor` if omitted.

The following three shadows are equivalent in terms of their shadow colors:

```
.shadow-color {
  text-shadow:
    5px 5px mediumblue,
    10px 10px magenta,
    15px 15px rebeccapurple;
}

.shadow-color-hex {
  text-shadow:
    5px 5px #0000cd,
    10px 10px #ff00ff,
    15px 15px #663399;
}

.shadow-color-rgb {
  text-shadow:
    5px 5px rgb(0 0 205),
    10px 10px rgb(255 0 255),
    15px 15px rgb(102 51 153);
}
```

Multiple shadows

You can apply multiple shadows to the same text by including multiple shadow values separated by commas.

The shadow effects are applied front-to-back: the first shadow is on top.

In our example of `text-shadow: 5px 5px mediumblue, 10px 10px magenta, 15px 15px rebeccapurple;`, it defines three shadows, with the blue one on top of the pink, which is on top of the purple:

```
p {
  text-shadow:
    5px 5px mediumblue,
    10px 10px magenta,
    15px 15px rebeccapurple;
}
```

```
<p>I have three shadows</p>
```

Multiple shadows with transparent text

Shadows are painted above any background colors or images and below any borders. While shadows overlay each other, they don't overlay the text. There is no equivalent of the `box-shadow` property's `inset` keyword in the text shadow world. Unlike a box shadow, text shadows are not clipped to the shadowed shape and may show through if the text is partially-transparent.

The following examples apply the same shadows to the text but with different `color` property values. The semi-transparent examples are difficult to read but are included to demonstrate how shadows render:

```
p {
  text-shadow:
    5px 5px 0 mediumblue,
    10px 10px 5px magenta,
    15px 15px 10px rebeccapurple;
}

.opaque {
  color: black;
}

.semitransparent {
  color: rgb(0 0 0 / 0.5);
}

.transparent {
  color: transparent;
}

.white {
  color: white;
}

.semi-white {
  color: rgb(255 255 255 / 0.75);
}
```

```
<p class="opaque">The text is opaque black</p>
<p class="semitransparent">The text is semiopaque black</p>
<p class="transparent">The text is transparent black</p>
<p class="white">The text is opaque white</p>
<p class="semi-white">The text is semiopaque white</p>
```

```
p {
  border: 1px solid black;
  padding: 20px;
  font-family: sans-serif;
  font-size: 1.75rem;
  font-weight: bold;
}
```

In the "transparent" example, the text is transparent but is fully legible because the top shadow is not blurred. Note how the shadow appears behind the text and is visible when the text is less than fully opaque. This is especially noticeable in the "semi-opaque white" example. This behavior differs from non-inset box shadows, where shadows are clipped at the border's outer edge.

CSS transforms

The **CSS transforms** module defines how elements styled with CSS can be transformed in two-dimensional or three-dimensional space.

CSS transforms in action

Use the sliders in the example below to modify the translation, rotation, scale, and skew CSS transform properties of the cube in 3D space. As you move the cube through 3D space, notice the way it interacts with the element labeled `z:0px`, which is located at the 3D position `(0, 0, 0)`.

```
<article>
  <fieldset id="allTransformFieldset">
    <legend>
      Transform settings
      <button id="resetAllButton" aria-label="Reset">↻</button>
    </legend>
    <section id="fieldsetSection">
      <fieldset class="transformFieldset">
        <legend>
          <label>
            <span>Translation</span>
          </label>
          <button
            id="translateSectionReset"
            class="resetSectionButton"
            aria-label="Reset">
            ↻
          </button>
        </legend>
        <div class="controlsContainer">
          <div>
            <label>
              <span id="translateXLabel">X</span>
              <input
                type="range"
                min="-100"
                max="100"
                value="0"
                data-default="0"
                id="translateXRange"
                aria-labelledby="translateXLabel" />
            </label>
            <output id="translateXOutput" for="translateXRange"></output>
          </div>
          <div>
            <label>
              <span id="translateYLabel">Y</span>
```

```
          <input
            type="range"
            min="-100"
            max="100"
            value="0"
            data-default="0"
            id="translateYRange"
            aria-labelledby="translateYLabel" />
        </label>
        <output id="translateYOutput" for="translateYRange"></output>
      </div>
      <div>
        <label>
          <span id="translateZLabel">Z</span>
          <input
            type="range"
            min="-100"
            max="100"
            value="0"
            data-default="0"
            id="translateZRange"
            aria-labelledby="translateZLabel" />
        </label>
        <output id="translateZOutput" for="translateZRange"></output>
      </div>
    </div>
  </fieldset>

  <fieldset class="transformFieldset">
    <legend>
      <label>
        <span>Rotation</span>
      </label>
      <button
        id="rotateSectionReset"
        class="resetSectionButton"
        aria-label="Reset">
        ↻
      </button>
    </legend>
    <div class="controlsContainer">
      <div>
        <label>
          <span id="rotateXLabel">X</span>
          <input
            type="range"
            min="-360"
            max="360"
            value="0"
            data-default="0"
            id="rotateXRange"
            aria-labelledby="rotateXLabel" />
        </label>
        <output id="rotateXOutput" for="rotateXRange"></output>
      </div>
      <div>
        <label>
          <span id="rotateYLabel">Y</span>
          <input
            type="range"
            min="-360"
            max="360"
            value="0"
            data-default="0"
            id="rotateYRange"
            aria-labelledby="rotateYLabel" />
        </label>
        <output id="rotateYOutput" for="rotateYRange"></output>
      </div>
      <div>
        <label>
          <span id="rotateZLabel">Z</span>
          <input
            type="range"
            min="-360"
            max="360"
            value="0"
            data-default="0"
            id="rotateZRange"
            aria-labelledby="rotateZLabel" />
        </label>
        <output id="rotateZOutput" for="rotateZRange"></output>
      </div>
    </div>
  </fieldset>

  <fieldset class="transformFieldset">
    <legend>
      <label>
        <span>Scale</span>
      </label>
      <button
        id="scaleSectionReset"
        class="resetSectionButton"
        aria-label="Reset">
        ↻
      </button>
    </legend>
    <div class="controlsContainer">
      <div>
        <label>
          <span id="scaleXLabel">X</span>
          <input
```

```
            type="range"
            min="0.1"
            max="2.5"
            value="1"
            data-default="1"
            step="0.1"
            id="scaleXRange"
            aria-labelledby="scaleXLabel" />
        </label>
        <output id="scaleXOutput" for="scaleXRange"></output>
      </div>
      <div>
        <label id="scaleYLabel">
          <span>Y</span>
          <input
            type="range"
            min="0.1"
            max="2.5"
            value="1"
            data-default="1"
            step="0.1"
            id="scaleYRange"
            aria-labelledby="scaleYLabel" />
        </label>
        <output id="scaleYOutput" for="scaleYRange"></output>
      </div>
      <div>
        <label id="scaleZLabel">
          <span>Z</span>
          <input
            type="range"
            min="0.1"
            max="2.5"
            value="1"
            data-default="1"
            step="0.1"
            id="scaleZRange"
            aria-labelledby="scaleZLabel" />
        </label>
        <output id="scaleZOutput" for="scaleZRange"></output>
      </div>
    </div>
  </fieldset>

  <fieldset class="transformFieldset">
    <legend>
      <label>
        <span>Skew</span>
      </label>
      <button
        id="skewSectionReset"
        class="resetSectionButton"
        aria-label="Reset">
        ↻
      </button>
    </legend>
    <div class="controlsContainer">
      <div>
        <label>
          <span id="skewXLabel">X</span>
          <input
            type="range"
            min="-90"
            max="90"
            value="0"
            data-default="0"
            id="skewXRange"
            aria-labelledby="skewXLabel" />
        </label>
        <output id="skewXOutput" for="skewXRange"></output>
      </div>
      <div>
        <label>
          <span id="skewYLabel">Y</span>
          <input
            type="range"
            min="-90"
            max="90"
            value="0"
            data-default="0"
            id="skewYRange"
            aria-labelledby="skewYLabel" />
        </label>
        <output id="skewYOutput" for="skewYRange"></output>
      </div>
    </div>
  </fieldset>

  <fieldset class="transformFieldset">
    <legend>
      <label>
        <span>Container Perspective</span>
      </label>
      <button
        id="containerPerspectiveSectionReset"
        class="resetSectionButton"
        aria-label="Reset">
        ↻
      </button>
    </legend>
    <div class="controlsContainer">
      <div>
        <label>
```

```
              <span id="perspectiveLabel"><code>perspective</code></span>
              <input
                type="range"
                min="75"
                max="500"
                value="200"
                data-default="200"
                id="perspectiveRange"
                aria-labelledby="perspectiveLabel" />
            </label>
            <output id="perspectiveOutput" for="perspectiveRange"></output>
          </div>
          <div>
            <label>
              <span id="perspectiveOriginXLabel"
                ><code>perspective-origin</code> X (%)</span
              >
              <input
                type="range"
                min="0"
                max="100"
                value="50"
                data-default="50"
                id="perspectiveOriginXRange"
                aria-labelledby="perspectiveOriginXLabel" />
            </label>
            <output
              id="perspectiveOriginXOutput"
              for="perspectiveOriginXRange"></output>
          </div>
          <div>
            <label>
              <span><code>perspective-origin</code> Y (%)</span>
              <input
                type="range"
                min="0"
                max="100"
                value="50"
                data-default="50"
                id="perspectiveOriginYRange"
                aria-labelledby="perspectiveOriginYLabel" />
            </label>
            <output
              id="perspectiveOriginYOutput"
              for="perspectiveOriginYRange"></output>
          </div>
        </div>
      </fieldset>

      <fieldset class="transformFieldset">
        <legend>
          <label>
            <span>Other Properties</span>
          </label>
          <button
            id="otherSectionReset"
            class="resetSectionButton"
            aria-label="Reset">
            ↻
          </button>
        </legend>
        <div class="controlsContainer">
          <div>
            <label>
              <span><code>backface-visibility</code></span>
              <input
                type="checkbox"
                checked
                data-default="checked"
                id="backfaceVisibilityCheckbox" />
            </label>
          </div>
        </div>
      </fieldset>
    </section>
  </fieldset>
  <section id="outputSection">
    <div id="outputContainer">
      <div id="cube">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
      <div id="z0"><code>z:0px</code></div>
    </div>
    <div id="perspectiveDot"></div>
  </section>
</article>
```

```
#allTransformFieldset {
  border: none;
  padding: 0;
  margin-bottom: 4px;
  accent-color: blue; /* or any color */
  font-family:
    "Inter", "Segoe UI", "Roboto", "Oxygen", "Ubuntu", "Cantarell", "Fira Sans",
    "Droid Sans", "Helvetica Neue", sans-serif;
}

#allTransformFieldset > legend {
```

```
  margin-bottom: 4px;
}

fieldset {
  margin: 0;
}

legend {
  font-weight: bold;
  padding: 0;
}

#fieldsetSection {
  display: flex;
  flex-wrap: wrap;
  align-items: start;
  gap: 8px;
}

#outputSection {
  width: 100%;
  min-height: 400px;
  background: linear-gradient(skyblue, khaki);
  display: flex;
  justify-content: center;
  align-items: center;
  position: relative;
  overflow: clip;
}

#outputContainer {
  position: absolute;
  width: 100%;
  height: 100%;
  perspective: 200px;
  display: flex;
  justify-content: center;
  align-items: center;
  transform-style: preserve-3d;
  pointer-events: none;
}

#z0 {
  width: 50px;
  height: 50px;
  background: linear-gradient(
    to right bottom,
    rgb(223 223 223),
    rgb(190 190 190)
  );
  transform: translateZ(0px);
  position: absolute;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 14px;
  color: black;
  border-radius: 50%;
  outline: 1px solid rgb(0 0 0 / 0.35);
  pointer-events: all;
}

#perspectiveDot {
  width: 4px;
  height: 4px;
  border-radius: 50%;
  background-color: rgb(240 0 0 / 0.5);
  transform: translate3d(-2px, -2px, 0px);
  position: absolute;
}

#cube {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  transition: all 0.075s ease-out;
  position: absolute;
  pointer-events: all;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
```

```
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}

.transformFieldset {
  margin: 0;
}

.controlsContainer {
  display: flex;
  flex-direction: column;
  align-items: start;
  gap: 4px;
}

.controlsContainer {
  width: 100%;
}

.controlsContainer > div {
  display: flex;
  flex-direction: row;
  align-items: center;
  gap: 4px;
}

.controlsContainer > div > label {
  display: flex;
  flex-direction: row;
  align-items: center;
  gap: 4px;
}

button {
  font-size: 18px;
  border-radius: 50%;
  border: #cccccc solid 1px;
  padding: 0;
  width: 26px;
  height: 26px;
  margin-left: 4px;
}

input[type="range"] {
  width: 172px;
}

output {
  width: 3em;
}
```

```
allTransformFieldset
  .querySelectorAll("input[type='range']")
  .forEach((rangeInput) => {
    // Event listeners for when the range inputs change
    rangeInput.addEventListener("input", (el) => {
      updateTransform();
    });

    // Reset the relevant transform setting when the range input is double clicked
    rangeInput.addEventListener("dblclick", (el) => {
      resetInput(el.target);
      updateTransform();
    });
  });

// Event listeners for when checkbox inputs change
allTransformFieldset
  .querySelectorAll("input[type='checkbox']")
  .forEach((checkboxInput) => {
    checkboxInput.addEventListener("input", (el) => {
      updateTransform();
    });
  });

// "Reset All" button event listener
resetAllButton.addEventListener("click", () => {
  allTransformFieldset.querySelectorAll("input").forEach((input) => {
    resetInput(input);
  });
  updateTransform();
});

// Section reset button event listeners
allTransformFieldset
  .querySelectorAll(".resetSectionButton")
  .forEach((resetSectionButton) => {
    resetSectionButton.addEventListener("click", (el) => {
```

```
      let allRanges = el.target.parentElement.parentElement.querySelectorAll(
        "input[type='range']",
      );
      allRanges.forEach((range) => {
        resetInput(range);
      });

      let allCheckboxes =
        el.target.parentElement.parentElement.querySelectorAll(
          "input[type='checkbox']",
        );
      allCheckboxes.forEach((check) => {
        resetInput(check);
      });

      updateTransform();
    });
  });

function resetInput(inputEl) {
  if (!inputEl) {
    console.warn(`inputEl \`${inputEl}\` is falsy!`);
    console.trace();
    return;
  }

  const defaultValue = inputEl.getAttribute("data-default");
  if (inputEl.getAttribute("type") === "checkbox") {
    inputEl.checked = defaultValue === "checked";
  } else {
    inputEl.value = defaultValue || "0";
  }
}

function updateOutputs() {
  translateXOutput.value = `${translateXRange.value}px`;
  translateYOutput.value = `${translateYRange.value}px`;
  translateZOutput.value = `${translateZRange.value}px`;

  rotateXOutput.value = `${rotateXRange.value}°`;
  rotateYOutput.value = `${rotateYRange.value}°`;
  rotateZOutput.value = `${rotateZRange.value}°`;

  scaleXOutput.value = `${scaleXRange.value}x`;
  scaleYOutput.value = `${scaleYRange.value}x`;
  scaleZOutput.value = `${scaleZRange.value}x`;

  skewXOutput.value = `${skewXRange.value}°`;
  skewYOutput.value = `${skewYRange.value}°`;

  perspectiveOutput.value = `${perspectiveRange.value}px`;

  perspectiveOriginXOutput.value = `${perspectiveOriginXRange.value}%`;
  perspectiveOriginYOutput.value = `${perspectiveOriginYRange.value}%`;
}

function updateTransform() {
  updateOutputs();

  cube.style.transform = `translate3d(${translateXRange.value}px,
                ${translateYRange.value}px,
                ${translateZRange.value}px)
                rotateX(${rotateXRange.value}deg)
                rotateY(${rotateYRange.value}deg)
                rotateZ(${rotateZRange.value}deg)
                scale3d(${scaleXRange.value},
                ${scaleYRange.value},
                ${scaleZRange.value})
                skewX(${skewXRange.value}deg)
                skewY(${skewYRange.value}deg)`;
  cube.style.backfaceVisibility = `${backfaceVisibilityCheckbox.checked ? "visible" : "hidden"}`;

  outputContainer.style.perspective = `${perspectiveRange.value}px`;
  outputContainer.style.perspectiveOrigin = `${perspectiveOriginXRange.value}% ${perspectiveOriginYRange.value}%`;

  perspectiveDot.style.top = `${perspectiveOriginYRange.value}%`;
  perspectiveDot.style.left = `${perspectiveOriginXRange.value}%`;
}
updateTransform();
```

You can also use the `perspective` slider to modify the `perspective` property of the cube's container, which determines the distance between you and the `z=0` plane.

The `perspective-origin` sliders determine where you, the viewer, are looking into the 3D space for purposes of determining the view's *vanishing point*. This vanishing point is indicated by a small red dot. You can imagine modifying these sliders as physically moving your head up, down, left, and right to see different parts of the cube without moving the cube itself.

The `backface-visibility` checkbox determines whether the cube's back faces are set to `visible` or `hidden`.

The cube in the above example is comprised of six `<div>` elements, all of which are styled with CSS to create the cube's faces. The cube is not drawn using a 2D or 3D canvas context, so **you can inspect the faces of the cube with your browser's developer tools as you would inspect any other DOM element**. Try using your browser's developer tools element picker to inspect different faces of the cube as you transform its position and rotation.

> **Note:** The order in which transformations, including 3D rotations, are applied affects the resultant transformation. In the above example, transforms are translated, scaled, rotated, then skewed. The rotations are applied in the order X → Y → Z.

Reference

Properties

- `backface-visibility`
- `perspective`

- `perspective-origin`
- `rotate`
- `scale`
- `transform`
- `transform-box`
- `transform-origin`
- `transform-style`
- `translate`

Functions

- `matrix()`
- `matrix3d()`
- `perspective()`
- `rotate()`
- `rotate3d()`
- `rotateX()`
- `rotateY()`
- `rotateZ()`
- `scale()`
- `scale3d()`
- `scaleX()`
- `scaleY()`
- `scaleZ()`
- `skew()`
- `skewX()`
- `skewY()`
- `translate()`
- `translate3d()`
- `translateX()`
- `translateY()`
- `translateZ()`

Data types

- `<transform-function>`

Guides

Using CSS transforms

Step-by-step tutorial about how to transform elements styled with CSS.

Coordinate systems

Describes the way pixel locations are defined in the CSS object model.

Performance fundamentals: Use CSS transforms

An overview of web performance fundamentals, including how CSS transforms can improve performance.

Matrix math for the web

Describes how object transformations can be represented by mathematical matrices.

Related concepts

- CSS Properties:
  - `animation`
  - `background-position`
  - `clip`
  - `clip-path`
  - `contain`
  - `content-visibility`
  - `isolation`
  - `mask`
  - `mask-border-source`
  - `mask-image`
  - `mix-blend-mode`
  - `opacity`
  - `overflow`
  - `transition`
  - `visibility`
- Data types:
  - `<angle>`
  - `<length-percentage>`
  - `<length>`
  - `<number>`
  - `<percentage>`
  - `position`
- Glossary terms:
  - Interpolation
  - Stacking context
- SVG concepts:
  - `<animate>` element
  - `<animateTransform>` element
  - `<set>` element
  - `transform` element

See also

- Basic SVG transformations tutorial

- CSS animations module
- CSS transitions module

Using CSS transforms

By modifying the coordinate space, **CSS transforms** change the shape and position of the affected content without disrupting the normal document flow. This guide provides an introduction to using transforms.

CSS transforms are implemented using a set of CSS properties that let you apply affine linear transformations to HTML elements. These transformations include rotation, skewing, scaling, and translation both in the plane and in the 3D space.

> **Warning:** Only transformable elements can be `transformed`; that is, all elements whose layout is governed by the CSS box model except for: non-replaced inline boxes, table-column boxes, and table-column-group boxes.

CSS transforms properties

Two major properties are used to define CSS transforms: `transform` (or the individual `translate`, `rotate`, and `scale` properties) and `transform-origin`.

`transform-origin`

: Specifies the position of the origin. By default, it is at the center of the element and can be moved. It is used by several transforms, like rotations, scaling or skewing, that need a specific point as a parameter.

`transform`

: Specifies the transforms to apply to the element. It is a space-separated list of transforms, which are applied one after the other, as requested by the composition operation. Composite transforms are effectively applied in order from right to left.

Examples

Here is an unaltered image of the MDN logo:

Rotating

Here is the MDN logo rotated 90 degrees from its bottom-left corner.

```
<img src="logo.png" alt="MDN Logo" />
```

```
img {
  rotate: 90deg;
  transform-origin: bottom left;
}
```

Skewing and translating

Here is the MDN logo, skewed by 10 degrees and translated by 150 pixels on the X-axis.

```
<img src="logo.png" alt="MDN logo" />
```

```
img {
  transform: skewX(10deg) translateX(150px);
  transform-origin: bottom left;
}
```

3D specific CSS properties

Performing CSS transformations in 3D space is a bit more complex. You have to start by configuring the 3D space by giving it a perspective, then you have to configure how your 2D elements will behave in that space.

Perspective

The first element to set is the `perspective`. The perspective is what gives us the 3D impression. The farther from the viewer the elements are, the smaller they are.

Setting perspective

This example shows a cube with the perspective set at different positions. How quick the cube shrinks is defined by the `perspective` property. The smaller its value is, the deeper the perspective is.

HTML

The HTML below creates four copies of the same box, with the perspective set at different values.

```
<table>
  <tbody>
    <tr>
      <th><code>perspective: 250px;</code></th>
      <th><code>perspective: 350px;</code></th>
    </tr>
    <tr>
      <td>
        <div class="container">
          <div class="cube perspective-250">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
```

```
      <td>
        <div class="container">
          <div class="cube perspective-350">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
    </tr>
    <tr>
      <th><code>perspective: 500px;</code></th>
      <th><code>perspective: 650px;</code></th>
    </tr>
    <tr>
      <td>
        <div class="container">
          <div class="cube perspective-500">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
      <td>
        <div class="container">
          <div class="cube perspective-650">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
    </tr>
  </tbody>
</table>
```

CSS

The CSS establishes classes that can be used to set the perspective to different distances. It also includes classes for the container box and the cube itself, as well as each of its faces.

```
/* Shorthand classes for different perspective values */
.perspective-250 {
  perspective: 250px;
}

.perspective-350 {
  perspective: 350px;
}

.perspective-500 {
  perspective: 500px;
}

.perspective-650 {
  perspective: 650px;
}

/* Define the container div, the cube div, and a generic face */
.container {
  width: 200px;
  height: 200px;
  margin: 75px 0 0 75px;
  border: none;
}

.cube {
  width: 100%;
  height: 100%;
  perspective-origin: 150% 150%;
  transform-style: preserve-3d;
}

.face {
  display: block;
  position: absolute;
  width: 100px;
  height: 100px;
  border: none;
  line-height: 100px;
  font-family: sans-serif;
  font-size: 60px;
  color: white;
  text-align: center;
  backface-visibility: visible;
}

/* Define each face based on direction */
.front {
  background: rgb(0 0 0 / 30%);
  transform: translateZ(50px);
}

.back {
```

```
  background: lime;
  color: black;
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(196 0 0 / 70%);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 196 / 70%);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(196 196 0 / 70%);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(196 0 196 / 70%);
  transform: rotateX(-90deg) translateZ(50px);
}

/* Make the table a little nicer */
th,
p,
td {
  background-color: #eeeeee;
  padding: 10px;
  font-family: sans-serif;
  text-align: left;
}
```

Result

The second element to configure is the position of the viewer, with the `perspective-origin` property. By default the perspective is centered on the viewer, which is not always adequate.

Changing the perspective origin

This example shows cubes with popular `perspective-origin` values.

HTML

```
<section>
  <figure>
    <figcaption><code>perspective-origin: top left;</code></figcaption>
    <div class="container">
      <div class="cube po-tl">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: top;</code></figcaption>
    <div class="container">
      <div class="cube po-tm">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: top right;</code></figcaption>
    <div class="container">
      <div class="cube po-tr">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: left;</code></figcaption>
    <div class="container">
      <div class="cube po-ml">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
```

```
  <figcaption><code>perspective-origin: 50% 50%;</code></figcaption>
  <div class="container">
    <div class="cube po-mm">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: right;</code></figcaption>
  <div class="container">
    <div class="cube po-mr">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: bottom left;</code></figcaption>
  <div class="container">
    <div class="cube po-bl">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: bottom;</code></figcaption>
  <div class="container">
    <div class="cube po-bm">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: bottom right;</code></figcaption>
  <div class="container">
    <div class="cube po-br">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: -200% -200%;</code></figcaption>
  <div class="container">
    <div class="cube po-200200neg">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: 200% 200%;</code></figcaption>
  <div class="container">
    <div class="cube po-200200pos">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: 200% -200%;</code></figcaption>
  <div class="container">
    <div class="cube po-200200">
      <div class="face front">1</div>
      <div class="face back">2</div>
```

```
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>
</section>
```

CSS

```
/* perspective-origin values (unique per example) */
.po-tl {
  perspective-origin: top left;
}
.po-tm {
  perspective-origin: top;
}
.po-tr {
  perspective-origin: top right;
}
.po-ml {
  perspective-origin: left;
}
.po-mm {
  perspective-origin: 50% 50%;
}
.po-mr {
  perspective-origin: right;
}
.po-bl {
  perspective-origin: bottom left;
}
.po-bm {
  perspective-origin: bottom;
}
.po-br {
  perspective-origin: bottom right;
}
.po-200200neg {
  perspective-origin: -200% -200%;
}
.po-200200pos {
  perspective-origin: 200% 200%;
}
.po-200200 {
  perspective-origin: 200% -200%;
}

/* Define the container div, the cube div, and a generic face */
.container {
  width: 100px;
  height: 100px;
  margin: 24px;
  border: none;
}

.cube {
  width: 100%;
  height: 100%;
  perspective: 300px;
  transform-style: preserve-3d;
}

.face {
  display: block;
  position: absolute;
  width: 100px;
  height: 100px;
  border: none;
  line-height: 100px;
  font-family: sans-serif;
  font-size: 60px;
  color: white;
  text-align: center;
  backface-visibility: visible;
}

/* Define each face based on direction */
.front {
  background: rgb(0 0 0 / 30%);
  transform: translateZ(50px);
}
.back {
  background: lime;
  color: black;
  transform: rotateY(180deg) translateZ(50px);
}
.right {
  background: rgb(196 0 0 / 70%);
  transform: rotateY(90deg) translateZ(50px);
}
.left {
  background: rgb(0 0 196 / 70%);
  transform: rotateY(-90deg) translateZ(50px);
}
.top {
  background: rgb(196 196 0 / 70%);
  transform: rotateX(90deg) translateZ(50px);
}
.bottom {
  background: rgb(196 0 196 / 70%);
  transform: rotateX(-90deg) translateZ(50px);
```

```
}

/* Make the layout a little nicer */
section {
  background-color: #eeeeee;
  padding: 10px;
  font-family: sans-serif;
  text-align: left;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

Result

Once you have done this, you can work on the element in the 3D space.

See also

- The CSS `transform` property and the CSS `<transform-function>` data types
- The individual transforms properties: `translate`, `rotate`, and `scale` (There is no `skew` property)
- Using device orientation with 3D Transforms
- Intro to CSS 3D transforms (Blog post by David DeSandro)
- CSS Transform Playground (Online tool to visualize CSS Transform functions)

CSS transitions

The **CSS transitions** module specifies functionality for creating gradual transitions between different CSS property values. The behavior of these transitions can be controlled by specifying their easing function, duration, and other values.

Normally, when the value of a CSS property changes, the result of the change from the old value to the new value is immediate. The CSS transitions module enables controlling a progression from the old property state to the new state over a specified period of time. It also provides event handlers to allow code to be run in response to different stages of a transition being reached.

In certain cases, there is no originating "from" value for a transition. For example, if an element gets added to the DOM, the styles defined are for the "to" state. This module provides the `@starting-style` at-rule, which enables defining starting styles for such cases. The module also defines how discrete property values should be transitioned, such as transitioning the discretely animated `display` property from the `none` value to a displayed value.

Reference

Properties

- `transition`
- `transition-behavior`
- `transition-delay`
- `transition-duration`
- `transition-property`
- `transition-timing-function`

At rules

- `@starting-style`

Interfaces

- `CSSStartingStyleRule`
- `CSSTransition`
  - `transitionProperty` property
- `TransitionEvent`
  - `TransitionEvent()` constructor
  - `TransitionEvent.propertyName` property
  - `TransitionEvent.elapsedTime` property
  - `TransitionEvent.pseudoElement` property
  - `transitioncancel` event
  - `transitionend` event
  - `transitionrun` event
  - `transitionstart` event

Guides

Using CSS transitions

Step-by-step tutorial explaining how to create transitions using CSS. This article describes each relevant CSS property and explains how they interact with each other.

Animating `display`

Transitioning to and from the `none` value of the discretely animated `display` property.

Transitioning a popover and transitioning a `<dialog>`

Examples of transitioning from `@starting-style` to final `:popover-open` and `:open` pseudo-class styles.

Related concepts

- `interpolate-size` property
- `calc-size()` function
- Intrinsic size glossary term
- CSS easing functions module:
  - `<easing-function>` data-type
- CSS animations module:
  - `animation` shorthand
  - `animation-delay`
  - `animation-direction`

    - `animation-duration`
    - `animation-fill-mode`
    - `animation-iteration-count`
    - `animation-name`
    - `animation-play-state`
    - `animation-timing-function`
- CSS transforms module:
    - `transform`
    - `transform-box`
    - `transform-origin`
    - `transform-style`
- CSS scroll snap module:
    - `scroll-snap-type`
    - `scroll-padding`
    - `scroll-snap-align`
    - `scroll-margin`
    - `scroll-snap-stop`

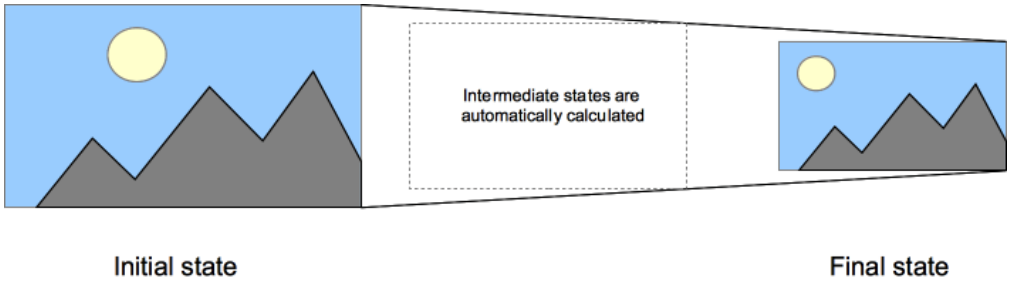
See also

- `opacity`
- `visibility`
- `ViewTransition` interface
- `PageTransitionEvent` interface

Using CSS transitions

CSS transitions provide a way to control animation speed when changing CSS properties. Instead of having property changes take effect immediately, you can cause the changes in a property to take place over a period of time. For example, if you change the color of an element from white to black, usually the change is instantaneous. With CSS transitions enabled, changes occur at time intervals that follow an acceleration curve, all of which can be customized.

Animations that involve transitioning between two states are often called *implicit transitions* as the states in between the start and final states are implicitly defined by the browser.



CSS transitions let you decide which properties to animate (by *listing them explicitly*), when the animation will start (by setting a *delay*), how long the transition will last (by setting a *duration*), and how the transition will run (by defining an *easing function*, e.g., linearly or quick at the beginning, slow at the end).

Which CSS properties can be transitioned?

The Web author can define which property has to be animated and in which way. This allows the creation of complex transitions. However, some properties are not animatable as it doesn't make sense to animate them.

> **Note:** The `auto` value is often a very complex case. The specification recommends not animating from and to `auto`. Some user agents, like those based on Gecko, implement this requirement and others, like those based on WebKit, are less strict. Using animations with `auto` may lead to unpredictable results, depending on the browser and its version, and should be avoided.

Defining transitions

CSS Transitions are controlled using the shorthand `transition` property. This is the best way to configure transitions, as it makes it easier to avoid out of sync parameters, which can be very frustrating to have to spend lots of time debugging in CSS.

You can control the individual components of the transition with the following sub-properties:

`transition-property`

: Specifies the name or names of the CSS properties to which transitions should be applied. Only properties listed here are animated during transitions; changes to all other properties occur instantaneously as usual.

`transition-duration`

: Specifies the duration over which transitions should occur. You can specify a single duration that applies to all properties during the transition, or multiple values to allow each property to transition over a different period of time.

`transition-timing-function`

: Specifies a function to define how intermediate values for properties are computed. *Easing functions* determine how intermediate values of the transition are calculated. Most easing functions can be specified by providing the graph of the corresponding function, as defined by four points defining a cubic bezier. You can also choose easing from Easing functions cheat sheet.

`transition-delay`

: Defines how long to wait between the time a property is changed and the transition actually begins.

The `transition` shorthand CSS syntax is written as follows:

```
transition: <property> <duration> <timing-function> <delay>;
```

Examples

Basic example

This example performs a four-second font size transition with a two-second delay between the time the user mouses over the element and the beginning of the animation effect:

```
#delay {
  font-size: 14px;
  transition-property: font-size;
  transition-duration: 4s;
  transition-delay: 2s;
}

#delay:hover {
  font-size: 36px;
}
```

Multiple animated properties example

```
<body>
  <p>
    The box below combines transitions for: width, height, background-color,
    rotate. Hover over the box to see these properties animated.
  </p>
  <div class="box">Sample</div>
</body>
```

CSS

```
.box {
  border-style: solid;
  border-width: 1px;
  display: block;
  width: 100px;
  height: 100px;
  background-color: blue;
  transition:
    width 2s,
    height 2s,
    background-color 2s,
    rotate 2s;
}

.box:hover {
  background-color: #ffcccc;
  width: 200px;
  height: 200px;
  rotate: 180deg;
}
```

When property value lists are of different lengths

If any property's list of values is shorter than the others, its values are repeated to make them match. For example:

```
div {
  transition-property: opacity, left, top, height;
  transition-duration: 3s, 5s;
}
```

This is treated as if it were:

```
div {
  transition-property: opacity, left, top, height;
  transition-duration: 3s, 5s, 3s, 5s;
}
```

Similarly, if any property's value list is longer than that for `transition-property`, it's truncated, so if you have the following CSS:

```
div {
  transition-property: opacity, left;
  transition-duration: 3s, 5s, 2s, 1s;
}
```

This gets interpreted as:

```
div {
  transition-property: opacity, left;
  transition-duration: 3s, 5s;
}
```

Using transitions when highlighting menus

A common use of CSS is to highlight items in a menu as the user hovers the mouse cursor over them. It's easy to use transitions to make the effect even more attractive.

First, we set up the menu using HTML:

```
<nav>
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact Us</a>
  <a href="#">Links</a>
</nav>
```

Then we build the CSS to implement the look and feel of our menu:

```
nav {
  display: flex;
  gap: 0.5rem;
```

```
}

a {
  flex: 1;
  background-color: #333333;
  color: white;
  border: 1px solid;
  padding: 0.5rem;
  text-align: center;
  text-decoration: none;
  transition: all 0.5s ease-out;
}

a:hover,
a:focus {
  background-color: white;
  color: #333333;
}
```

This CSS establishes the look of the menu, with the background and text colors both changing when the element is in its `:hover` and `:focus` states:

Transitioning display and content-visibility

This example demonstrates how `display` and `content-visibility` can be transitioned. This behavior is useful for creating entry/exit animations where you want to for example remove a container from the DOM with `display: none`, but have it fade out with `opacity` rather than disappearing immediately.

Supporting browsers transition `display` and `content-visibility` with a variation on the discrete animation type. This generally means that properties will flip between two values 50% through animating between the two.

There is an exception, however, which is when animating to/from `display: none` or `content-visibility: hidden`. In this case, the browser will flip between the two values so that the transitioned content is shown for the entire animation duration.

So for example:

- When animating `display` from `none` to `block` (or another visible `display` value), the value will flip to `block` at `0%` of the animation duration so it is visible throughout.
- When animating `display` from `block` (or another visible `display` value) to `none`, the value will flip to `none` at `100%` of the animation duration so it is visible throughout.

When transitioning these properties `transition-behavior: allow-discrete` needs to be set on the transitions. This effectively enables `display`/`content-visibility` transitions.

When transitioning `display`, `@starting-style` is needed to provide a set of starting values for properties set on an element that you want to transition from when the element receives its first style update. This is needed to avoid unexpected behavior. By default, CSS transitions are not triggered on elements' first style updates when they first appear in the DOM, which includes when `display` changes from `none` to another state. `content-visibility` animations do not need starting values specified in a `@starting-style` block. This is because `content-visibility` doesn't hide an element from the DOM like `display` does: it just skips rendering the element's content.

HTML

The HTML contains two `<p>` elements with a `<div>` in between that we will animate from `display none` to `block`.

```
<p>
  Click anywhere on the screen or press any key to toggle the
  <code>&lt;div&gt;</code> between hidden and showing.
</p>

<div>
  This is a <code>&lt;div&gt;</code> element that transitions between
  <code>display: none; opacity: 0</code> and
  <code>display: block; opacity: 1</code>. Neat, huh?
</div>

<p>
  This is another paragraph to show that <code>display: none;</code> is being
  applied and removed on the above <code>&lt;div&gt; </code>. If only its
  <code>opacity</code> was being changed, it would always take up the space in
  the DOM.
</p>
```

CSS

```
html {
  height: 100vh;
}

div {
  font-size: 1.6rem;
  padding: 20px;
  border: 3px solid red;
  border-radius: 20px;
  width: 480px;

  display: none;
  opacity: 0;
  transition:
    opacity 1s,
    display 1s allow-discrete;
  /* Equivalent to
  transition: all 1s allow-discrete; */
}

.showing {
  opacity: 1;
  display: block;
}

@starting-style {
  .showing {
    opacity: 0;
  }
}
```

Note the `@starting-style` block used to specify the starting style for the transition, and the inclusion of the `display` property in the transitions list, with `allow-discrete` set on it.

JavaScript

Finally, we include a bit of JavaScript to set up event listeners to trigger the transition (via the `showing` class).

```
const divElem = document.querySelector("div");
const htmlElem = document.querySelector(":root");

htmlElem.addEventListener("click", showHide);
document.addEventListener("keydown", showHide);

function showHide() {
  divElem.classList.toggle("showing");
}
```

Result

The code renders as follows:

JavaScript examples

> **Note:** Care should be taken when using a transition immediately after:
>
> - adding the element to the DOM using `.appendChild()`
> - removing an element's `display: none;` property.
>
> This is treated as if the initial state had never occurred and the element was always in its final state. One way to overcome this limitation is to apply a `setTimeout()` of a handful of milliseconds before changing the CSS property you intend to transition to.

Using transitions to make JavaScript functionality smooth

Transitions are a great tool to make things look much smoother without having to do anything to your JavaScript functionality. Take the following example.

```
<p>Click anywhere to move the ball</p>
<div id="foo" class="ball"></div>
```

```
// Make the ball move to a certain position:
const f = document.getElementById("foo");
document.addEventListener("click", (ev) => {
  f.style.transform = `translateY(${ev.clientY - 25}px)`;
  f.style.transform += `translateX(${ev.clientX - 25}px)`;
});
```

With CSS, you can smooth the styles applied through JavaScript. Add a transition to the element and any change will happen smoothly:

```
body {
  background-color: white;
  color: #333333;
  font:
    1.2em / 1.5 "Helvetica Neue",
    "Helvetica",
    "Arial",
    sans-serif;
  padding: 0;
  margin: 0;
}

p {
  margin-top: 3em;
}

main {
  box-sizing: border-box;
  display: flex;
  justify-content: center;
  align-items: center;
  max-width: 660px;
  height: 400px;
  border: 1px solid #cccccc;
  padding: 20px;
}
```

```
.ball {
  border-radius: 25px;
  width: 50px;
  height: 50px;
  background: #cc0000;
  position: absolute;
  top: 0;
  left: 0;
  transition: transform 1s;
}
```

Detecting the start and completion of a transition

You can use the `transitionend` event to detect that an animation has finished running. This is a `TransitionEvent` object, which has two added properties beyond a typical `Event` object:

`propertyName`

A string indicating the name of the CSS property whose transition completed.

`elapsedTime`

A float indicating the number of seconds the transition had been running at the time the event fired. This value isn't affected by the value of `transition-delay`.

As usual, you can use the `addEventListener()` method to monitor for this event:

```
el.addEventListener("transitionend", updateTransition);
```

You detect the beginning of a transition using `transitionrun` (fires before any delay) and `transitionstart` (fires after any delay), in the same kind of fashion:

```
el.addEventListener("transitionrun", signalStart);
el.addEventListener("transitionstart", signalStart);
```

> **Note:** The `transitionend` event doesn't fire if the transition is aborted before the transition is completed because either the element is made `display: none` or the animating property's value is changed.

See also

- The `TransitionEvent` interface and the `transitionend` event
- Using CSS animations

CSS values and units

Every CSS declaration consists of a property/value pair. The value can take various forms depending on the property, such as a single integer, keyword, function, or a combination of different items; some values have units, while others do not. Every property also accepts the CSS-wide values. The CSS values and units module defines the data types — values and units — that CSS properties accept. This module also defines the CSS value definition syntax, or formal grammar, used to define the set of valid values for every CSS property and function.

> **Note:** This page introduces a CSS module. To find an exhaustive list of all values, types, and functions defined by CSS specifications, see the values reference page.

Reference

Properties

- `interpolate-size`

Functions

- `abs()`
- `acos()`
- `asin()`
- `atan()`
- `atan2()`
- `attr()`
- `calc()`
- `calc-size()`
- `clamp()`
- `cos()`
- `exp()`
- `hypot()`
- `<ident()>`
- `if()`
- `inherit()`
- `log()`
- `max()`
- `min()`
- `mod()`
- `pow()`
- `progress()`
- `random()`
- `rem()`
- `round()`
- `sibling-count()`
- `sibling-index()`
- `sign()`
- `sin()`
- `sqrt()`
- `tan()`
- `url()`

The CSS values and units module also introduces the `calc-mix()`, `first-valid()`, `integrity()`, `random-item()`, `src()`, `type()`, and `toggle()` functions. Currently, no browsers support these features.

Data types

- `<angle-percentage>`
- `<angle>`
- `<attr-name>`
- `<attr-type>`
- `<calc-keyword>` (`e`, `pi`, `infinity`, NaN)
- `<calc-size-basis>`
- `<calc-sum>`
- `<custom-ident>`
- `<dashed-ident>`
- `<dimension>`
- `<easing-function>`
- `<ident>`
- `<integer>`
- `<length-percentage>`
- `<length>`
- `<number>`
- `<percentage>`
- `<position>`
- `<ratio>`
- `<resolution>`
- `<rounding-strategy>` (`down`, `up`, `to-zero`)
- `<string>`

- `<syntax>`
- `<time-percentage>`
- `<time>`
- `<url>`
- `<url-modifier>`

The CSS values and units module also introduces the `<frequency>` and `<frequency-percentage>` data types. Currently, no browsers support these features.

Units

- `%` (percentage)
- `cap`
- `ch`
- `cm`
- `deg`
- `dpcm`
- `dpi`
- `dppx`
- `dvb`
- `dvh`
- `dvi`
- `dvmax`
- `dvmin`
- `dvw`
- `em`
- `ex`
- `grad`
- `Hz`
- `ic`
- `in`
- `kHz`
- `lh`
- `lvb`
- `lvh`
- `lvi`
- `lvmax`
- `lvmin`
- `lvw`
- `mm`
- `ms`
- `pc`
- `pt`
- `px`
- `Q`
- `rad`
- `rcap`
- `rch`
- `rem`
- `rex`
- `ric`
- `rlh`
- `s`
- `svb`
- `svh`
- `svi`
- `svmax`
- `svmin`
- `svw`
- `turn`
- `vb`
- `vh`
- `vi`
- `vmax`
- `vmin`
- `vw`
- `x`

Flex units (`fr`) and container units (`cqb`, `cqh`, `cqi`, `cqmax`, `cqmin`, `cqw`) are defined in the CSS grid layout and CSS conditional rules modules, respectively.

Unit categorizations

- Absolute length units (`cm`, `in`, `mm`, `pc`, `pt`, `px`, `Q`)
- Angle units (`deg`, `grad`, `rad`, `turn`)
- Default viewport units (`vb` , `vh`, `vi` , `vmax`, `vmin`, `vw`)
- Dynamic viewport units (`dvb`, `dvh`, `dvi`, `dvmax`, `dvmin`, `dvw`)
- Frequency units (`Hz`, `kHz`)
- Large viewport-percentage units (`lvb`, `lvh`, `lvi`, `lvmax`, `lvmin`, `lvw`)
- Local font-relative length units (`cap`, `ch`, `em`, `ex`, `ic`, `lh`)
- Physical units (`cm`, `in`, `mm`, `pc`, `pt`, `Q`)
- Relative length units (`cap`, `ch`, `em`, `ex`, `ic`, `lh`, `rem`, `rlh`, `vb`, `vh`, `vi`, `vmax`, `vmin`, `vw`)
- Resolution units (`dpcm`, `dpi` , `dppx`, `x`)
- Root font-relative length units (`rcap`, `rch`, `rem`, `rex`, `ric`, `rlh`)
- Small viewport-percentage unit (`svb`, `svh`, `svi`, `svmax`, `svmin`, `svw`)
- Time units (`ms`, `s`)
- Viewport units (`dvh`, `dvw`, `lvh`, `lvw`, `svh`, `svw`, `vb` , `vh`, `vi` , `vmax`, `vmin`, `vw`)
- Visual angle unit (`px`)

Key concepts

- Advance measure
- Bracketed range notation
- Component value combinators
- CSS-wide keywords
- Device pixel
- Functional notation
- Identifier
- Interpolation
- Keyword
- Math function
- Numeric data types
- Origin
- Pixel
- Textual data types
- URL
- Value definition syntax

Guides

CSS data types

Introduction to CSS data types that define typical values accepted by CSS properties and functions.

Numeric data types

Overview of the numeric data types, including integers, numbers, percentages, and dimensions, along with relative and absolute dimensions, angles, and time units.

Textual data types

Overview of the textual data types, including pre-defined keyword values, global CSS keyword values, and URLs.

CSS value functions

Overview of the CSS statements that invoke special data processing or calculations to return a CSS value for a CSS property.

Using CSS math functions

The CSS math functions that allow a property value to be written as a mathematical expression.

Value definition syntax

The formal grammar used to define the set of valid values for CSS properties and functions.

Using CSS typed arithmetic

An explanation of CSS typed arithmetic behavior and use cases enabled by it.

Learn: Values and units

A look at some of the most frequently used value types, what they are, and how they work.

CSS value serialization

How CSSOM APIs serialize color and other values into standardized string representations.

Related

- CSS cascading and inheritance module
  - `initial`
  - `inherit`
  - `revert`
  - `revert-layer`
  - `unset`
  - `all`
- CSS grid layout module
  - `<flex>`
  - Flex units (`fr`)
- CSS conditional rules module
  - Container units (`cqb`, `cqh`, `cqi`, `cqmax`, `cqmin`, `cqw`)
- CSS colors module
  - `<color>`
  - `<system-color>`
  - `color-mix()`
- CSS images module
  - `<image>`
  - `<gradient>`

See also

- CSS syntax module
- CSS selectors module

Numeric data types

Every CSS declaration consists of a property/value pair. The value can include various data types depending on the property, such as a single number, keyword, function, or a combination of different types; some values have units, while others do not. Numeric data types include `<integer>`, `<number>`, `<dimension>`, and `<percentage>` values. This guide is an overview of numeric data types. Refer to the page for each value type for more detailed information.

Integers

An integer is one or more decimal digits, `0` through `9`, such as `1024` or `-55`. An integer may be preceded by a `+` or `-` symbol, with no space between the symbol and the integer.

Numbers

A `<number>` represents a real number, which may or may not have a decimal point with a fractional component, for example `0.255`, `128` or `-1.2`. Numbers may also be preceded by a `+` or `-` symbol.

Dimensions

A `<dimension>` is a `<number>` with a unit attached to it, for example `45deg`, `100ms`, or `10px`. The attached unit identifier is case insensitive. There is never a space or any other characters between the number and the unit identifier: i.e., `1 cm` is not valid.

CSS uses dimensions to specify:

- `<length>` (Distance units)
- `<angle>`
- `<time>`
- `<frequency>`
- `<flex>`
- `<resolution>`

These are all covered in subsections below.

Distance units

Where a distance unit, also known as a length, is allowed as a value for a property, this is described as the `<length>` type. There are two types of lengths in CSS: relative and absolute. Relative length units specify a length in relation to something else.

There are two types of relative lengths: font-relative lengths and viewport-percentage lengths. These both come in two types. Font-relative length units are either local font-relative or root font-relative. Viewport percentage lengths are either relative to the viewport height or width size or, as defined in the CSS Containment module, relative to a container.

Local font-relative lengths

Local font-relative lengths are relative to the "local" font size or line height, specifying a length in relation to a computed size of a feature of the element itself, or relative to the element's inherited value in the case of a circular reference, such as the `em` value for a `font-size` property or a `lh` value for a `line-height` property. For example, `em` is relative to the font size on the element and `ex` is relative to the x-height of the element's font.

Unit	Relative to
`cap`	Cap height (the nominal height of capital letters) of the element's font.
`ch`	Average character advance of a narrow glyph in the element's font, as represented by the "0" (ZERO, U+0030) glyph.
`em`	Font size of the element's font.
`ex`	x-height of the element's font.
`ic`	Average character advance of a full-width glyph in the element's font, as represented by the "水" (CJK water ideograph, U+6C34) glyph.
`lh`	Line height of the element.

Root font-relative lengths

Root font-relative lengths specify a length in relation to the element's root element ancestor, such as `<HTML>` or `<SVG>`. For example, `rem` is relative to the font size on the root element and `rex` is the x-height of the root element's font.

Unit	Relative to
`rcap`	Cap height (the nominal height of capital letters) of the root element's font.
`rch`	Average character advance of a narrow glyph in the root element's font, as represented by the "0" (ZERO, U+0030) glyph.
`rem`	Font size of the root element's font.
`rex`	x-height of the root element's font.
`ric`	Average character advance of a full-width glyph in the root element's font, as represented by the "水" (CJK water ideograph, U+6C34) glyph.
`rlh`	Line height of the root element.

Viewport units

Viewport unit lengths specify a length relative to the dimensions of the viewport. For example, `vw` is relative to the width of the viewport and `vh` is relative to the height of the viewport.

Unit	Relative to
`dvh`	1% of the dynamic viewport's height.
`dvw`	1% of the dynamic viewport's width.
`lvh`	1% of the large viewport's height.
`lvw`	1% of the large viewport's width.
`svh`	1% of the small viewport's height.
`svw`	1% of the small viewport's width.
`vb`	1% of viewport's size in the root element's block axis.
`vh`	1% of viewport's height.
`vi`	1% of viewport's size in the root element's inline axis.
`vmax`	1% of viewport's larger dimension.
`vmin`	1% of viewport's smaller dimension.
`vw`	1% of viewport's width.

Container units

Container query length units specify a length relative to the dimensions of a query container. For example, `cqw` is relative to the width of the query container and `cqh` is relative to the height of the query container.

Unit	Relative to
`cqb`	1% of a query container's block size
`cqh`	1% of a query container's height
`cqi`	1% of a query container's inline size
`cqmax`	The larger value of `cqi` or `cqb`
`cqmin`	The smaller value of `cqi` or `cqb`
`cqw`	1% of a query container's width

Absolute length units

Absolute length units are fixed to a physical length: either an inch or a centimeter. Many of these units are therefore more useful when the output is a fixed size media, such as print. For example, `mm` is a physical millimeter, 1/10th of a centimeter.

Unit	Name	Equivalent to
`cm`	Centimeters	1cm = 96px/2.54
`in`	Inches	1in = 2.54cm = 96px
`mm`	Millimeters	1mm = 1/10th of 1cm
`pc`	Picas	1pc = 1/6th of 1in
`pt`	Points	1pt = 1/72th of 1in
`px`	Pixels	1px = 1/96th of 1in
`Q`	Quarter-millimeters	1Q = 1/40th of 1cm

When including a length value, if the length is `0`, the unit identifier is not required. Otherwise, the unit identifier is required, is case insensitive, and must come immediately after the numeric part of the value, with no space in-between.

Angle units

Angle values are represented by the type `<angle>` and accept the following values:

Unit	Name	Description
`deg`	Degrees	There are 360 degrees in a full circle.
`grad`	Gradians	There are 400 gradians in a full circle.
`rad`	Radians	There are 2π radians in a full circle.
`turn`	Turns	There is 1 turn in a full circle.

Time units

Time values are represented by the type `<time>`. When including a time value, the unit identifier — the `s` or `ms` — is required. It accepts the following values.

Unit	Name	Description
`ms`	Milliseconds	There are 1,000 milliseconds in a second.
`s`	Seconds	

Frequency units

Frequency values are represented by the type `<frequency>`. It accepts the following values.

Unit	Name	Description
`Hz`	Hertz	Represents the number of occurrences per second.
`kHz`	KiloHertz	A kiloHertz is 1000 Hertz.

`1Hz`, which can also be written as `1hz` or `1HZ`, is one cycle per second.

Flex units

Flex units are represented by the type `<flex>`. It accepts the following value.

Unit	Name	Description
`fr`	Flex	Represents a flexible length within a grid container

Resolution units

Resolution units are represented by the type `<resolution>`. They represent the size of a single dot in a graphical representation, such as a screen, by indicating how many of these dots fit in a CSS inch, centimeter, or pixel. It accepts the following values:

Unit	Description
`dpcm`	Dots per centimeter.
`dpi`	Dots per inch.
`dppx`, `x`	Dots per px unit.

Percentages

A `<percentage>` is a type that represents a fraction of some other value.

Percentage values are always relative to another quantity, for example a length. Each property that allows percentages also defines the quantity to which the percentage refers. This quantity can be a value of another property of the same element, the value of a property of an ancestor element, a measurement of a containing block, or something else.

As an example, if you specify the `width` of a box as a percentage, it refers to the percentage of the box's parent's computed width:

```
.box {
  width: 50%;
```

```
}
```

Mixing percentages and dimensions

Some properties accept a dimension that could be either one of two types, for example a `<length>` **or** a `<percentage>`. In this case the allowed value is detailed in the specification as a combination unit, e.g., `<length-percentage>`. Other possible combinations are as follows:

- `<frequency-percentage>`
- `<angle-percentage>`
- `<time-percentage>`

Special data types (defined in other specs)

- `<color>`
- `<image>`
- `<position>`

Color

The `<color>` value specifies the color of an element feature (e.g., its background color), and is defined in the CSS Color Module.

Image

The `<image>` value specifies all the different types of image that can be used in CSS, and is defined in the CSS Image Values and Replaced Content Module.

Position

The `<position>` type defines 2D positioning of an object inside a positioning area, for example a background image inside a container. This type is interpreted as a `background-position` and therefore specified in the CSS Backgrounds and Borders specification.

Functional notation

- `calc()`
- `min()`
- `max()`
- `minmax()`
- `clamp()`
- `attr()`

Functional notation is a type of value that can represent more complex types or invoke special processing by CSS. The syntax starts with the name of the function immediately followed by a left parenthesis `(` followed by the argument(s) to the notation followed by a right parenthesis `)`. Functions can take multiple arguments, which are formatted similarly to a CSS property value.

White space is allowed, but optional inside the parentheses. (But see notes regarding whitespace within pages for `min()`, `max()`, `minmax()`, and `clamp()` functions.)

Some legacy functional notations, such as legacy syntax for `rgb()`, `rgba()`, `hsl()`, and `hsla()`, used commas, but commas are generally only used to separate items in a list. If a comma is used to separate arguments, white space is optional before and after the comma.

The spec also defines the `toggle()` function. It has not been implemented anywhere yet.

See also

- Textual data types
- CSS data types
- CSS values and units module
- Learn: Values and units

Textual data types

Every CSS declaration consists of a property/value pair. The value can include various data types depending on the property, such as a single keyword, integer, function, or a combination of different types; some values have units, while others do not. This guide provides an overview of the textual data types. Refer to the page for each value type for more detailed information.

Text data types are either `<string>`, a quoted series of characters, an `<ident>`, a "CSS Identifier" which is an unquoted string, or an optionally quoted `<url>`. A `<string>` is quoted with either single or double quotes. CSS identifiers, listed in the specifications as `<ident>` or `<custom-ident>`, must be unquoted.

In CSS specifications, values that can be defined by the web developer — such as keyframe animation names, font-family names, or grid areas — are listed as a `<custom-ident>`, `<string>`, or both.

When both quoted and unquoted user-defined text values are permitted, the specification will list `<custom-ident> | <string>`, meaning quotes are optional, such as is the case with [keyframe animation names]:

```
@keyframes validIdent {
  /* keyframes go here */
}
@keyframes 'validString' {
  /* keyframes go here */
}
```

Some text values are not valid if encompassed in quotes. For example, the value of `grid-area` can be a `<custom-ident>`, so if we had a grid area named `content` we would use it without quotes:

```
.item {
  grid-area: content;
}
```

In comparison, a data type that is a `<string>`, such as a string value of the `content` property, must be quoted:

```
.item::after {
  content: "This is my content.";
}
```

While you can generally create any name you want, including using emojis, the identifier can't be `none`, `unset`, `initial`, or `inherit`, start with a digit or two dashes, and generally you don't want it to be any other pre-defined CSS keyword. See the `<custom-ident>` and `<string>` reference pages for more details.

Pre-defined keyword values

Pre-defined keywords are text values defined by the specification for that property. These keywords are also CSS identifiers, and are therefore used without quotes.

When viewing CSS property value syntax in a CSS specification or the MDN property page, allowable keywords will be listed in the following form. The following enumerated values are the pre-defined keyword values allowed for `float`.

```
left | right | none | inline-start | inline-end
```

Such values are used without quotes:

```
.box {
  float: left;
}
```

CSS-wide keywords

In addition to the pre-defined keywords that are part of the specification for a property, all CSS properties accept the CSS-wide, or "global", property values `initial`, `inherit`, `unset`, `revert`, and `revert-layer`, which explicitly specify defaulting behaviors.

`initial`

Represents the value specified as the property's initial value.

`inherit`

Represents the computed value of the property on the element's parent, provided it is inherited.

`unset`

Acts as either `inherit` or `initial`, depending on whether the property is inherited or not.

`revert`

Resets the property to its inherited value if it inherits from its parent or to the default value established by the user agent's stylesheet (or by user styles if any exist).

`revert-layer`

Rolls back the value of a property in a cascade layer to the value of the property in a CSS rule matching the element in a previous cascade layer. The value of the property with this keyword is recalculated as if no rules were specified on the target element in the current cascade layer.

URLs

A `<url>` type uses functional notation, which accepts a `<string>` that is a URL. This may be an absolute URL or a relative URL. For example, if you wanted to include a background image, you might use either of the following:

```
.box {
  background-image: url("images/my-background.png");
}

.box {
  background-image: url("https://www.exammple.com/images/my-background.png");
}
```

The parameter for `url()` can be either quoted or unquoted. If unquoted, it is parsed as a `<url-token>`, which has extra requirements including escaping certain characters. See `<url>` for more information.

See also

- Numeric data types
- CSS data types
- CSS values and units module
- Learn: Values and units
- CSS cascade and inheritance module

Using CSS math functions

CSS math functions allow a property value - such as the `height`, `animation-duration`, or `font-size` of an element - to be written as a mathematical expression.

Without using any math, the built-in CSS units like `rem`, `vw`, and `%` are often flexible enough to style HTML elements to achieve a particular user experience.

However, there are cases where we might feel limited by expressing an element's style using a single value and unit. Consider the following examples:

1. We want to set the height of a content area to be "the height of the viewport minus the height of a navbar."
2. We want to add the width of two elements together to define the width of a third element.
3. We want to prevent a variable `font-size` of some text from growing beyond a certain size.

In all of these cases, we need to rely on math to achieve the desired outcomes. One solution could be to rely on mathematical functions defined by JavaScript, and dynamically set element styles based on results calculated by our scripts.

In many instances, including in the examples above, **we can instead utilize math functions built directly into CSS**. This solution is often simpler to implement and faster for the browser to execute than using JavaScript.

In total, developers can use a combination of nearly two dozen CSS math functions in their stylesheets. In this guide, we'll exemplify four of the more commonly-used, and introduce those more advanced.

<code>calc()</code>: Basic math operations

In the first two of our three examples above, we want to set the style of an element according to the result of an addition or subtraction operation. This is exactly one of the use cases for `calc()`.

The **`calc()`** function lets you specify CSS property values using **addition, subtraction, multiplication, and division**. It is often used to combine two CSS values that have different units, such as `%` and `px`.

The `calc()` math function takes a mathematical expression as a parameter and returns the result of that expression, e.g.:

```
property: calc(expression);
```

<code>calc()</code> Example

Click on the play icon below to see the `calc()` example in the code playground and try it for yourself.

```
<div class="calc1">
  <code>width: calc(10px + 100px);</code>
</div>
<div class="calc2">
  <code>width: calc(2em * 5);</code>
</div>
<div class="calc3">
```

```
  <code>width: calc(100% - 32px);</code>
</div>
<div class="calc4">
  <code>width: calc(var(--predefined-width) - calc(16px * 2));</code>
</div>
```

```
div {
  background-color: black;
  margin: 4px 0;
  width: 100%;
}

div > code {
  display: block;
  background-color: red;
  color: white;
  height: 48px;
}

.calc1 > code {
  /* Output width: `110px` */
  width: calc(10px + 100px);
}

.calc2 > code {
  /* Output width: `10em` */
  width: calc(2em * 5);
}

.calc3 > code {
  /* Output width: Depends on the container's width */
  width: calc(100% - 32px);
}

.calc4 > code {
  --predefined-width: 100%;
  /* Output width: Depends on the container's width */
  width: calc(var(--predefined-width) - calc(16px * 2));
}
```

<code>min()</code>: Finding the minimum value in a set

There are cases where we don't want the value of a CSS property to exceed a certain number. Say, for example, we want the width of our content container to be the smaller of "the full width of our screen" and "500 pixels." In those cases, we can use the CSS math function `min()`.

The `min()` math function takes a set of comma-separated values as arguments and returns the smallest of those values, e.g.:

```
width: min(32px, 50%, 2rem);
```

This function is often used to compare two CSS values that have different units, such as `%` and `px`.

<code>min()</code> Example

Click on the play icon below to see the `min()` example in the code playground and try it for yourself.

```
<div class="min1">
  <code>width: min(9999px, 50%);</code>
</div>
<div class="min2">
  <code>width: min(9999px, 100%);</code>
</div>
<div class="min3">
  <code>width: min(120px, 150px, 90%);</code>
</div>
<div class="min4">
  <code>width: min(80px, 90%);</code>
</div>
```

```
div {
  background-color: black;
  margin: 4px 0;
  width: 100%;
}

div > code {
  display: block;
  background-color: darkblue;
  color: white;
  height: 48px;
}

.min1 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `50%` of the container's width */
  width: min(9999px, 50%);
}

.min2 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `100%` of the container's width */
  width: min(9999px, 100%);
}

.min3 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `120px` of the container's width */
  width: min(120px, 150px, 90%);
}

.min4 > code {
  /* Output width: Depends on the container's width; */
```

```
  /* on this page, likely to be `80px` of the container's width */
  width: min(80px, 90%);
}
```

<code>max()</code>: Finding the maximum value in a set

Similar to `min()`, sometimes we don't want the value of a CSS property to go below a certain number. For example, we might want the width of our content container to be the *larger* of "the full width of our screen" and "500 pixels." In those cases, we can use the CSS math function `max()`.

The `max()` math function takes a set of comma-separated values as arguments and returns the largest of those values, e.g.:

```
width: max(32px, 50%, 2rem);
```

This function is often used to compare two CSS values that have different units, such as `%` and `px`.

Notice the similarities and differences between the examples for `min()` and `max()`.

<code>max()</code> Example

Click on the play icon below to see the `max()` example in the code playground and try it for yourself.

```
<div class="max1">
  <code>width: max(50px, 50%);</code>
</div>
<div class="max2">
  <code>width: max(50px, 100%);</code>
</div>
<div class="max3">
  <code>width: max(20px, 50px, 90%);</code>
</div>
<div class="max4">
  <code>width: max(80px, 80%);</code>
</div>
```

```
div {
  background-color: black;
  margin: 4px 0;
  width: 100%;
  height: 48px;
}

div > code {
  display: block;
  background-color: darkmagenta;
  color: white;
  height: 48px;
}

.max1 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `50%` of the container's width */
  width: max(50px, 50%);
}

.max2 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `100%` of the container's width */
  width: max(50px, 100%);
}

.max3 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `90%` of the container's width */
  width: max(20px, 50px, 90%);
}

.max4 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `80%` of the container's width */
  width: max(80px, 80%);
}
```

<code>clamp()</code>: Constraining a value between two values

We can combine the functions of `min()` and `max()` by using `clamp()`. The `clamp()` math function takes a minimum value, the value to be clamped, and the maximum value as arguments, e.g.:

```
/* clamped value: 50%, minimum: 100px, maximum: 300px */
width: clamp(100px, 50%, 300px);
```

- If the value to be clamped is less than the passed minimum value, the function will return the minimum value.
- If the value to be clamped is greater than the passed maximum value, the function will return the maximum value.
- If the value to be clamped is between the passed minimum and maximum values, the function will return the original value to be clamped.

This function is often used to compare two CSS values that have different units, such as `%` and `px`.

<code>clamp()</code> Example

Click on the play icon below to see the `clamp()` example in the code playground and try it for yourself.

```
<div class="clamp1">
  <code>width: clamp(10%, 1px, 90%);</code>
</div>
<div class="clamp2">
  <code>width: clamp(10%, 9999px, 90%);</code>
</div>
<div class="clamp3">
  <code>width: clamp(125px, 1px, 250px);</code>
</div>
<div class="clamp4">
```

```
  <code>width: clamp(25px, 9999px, 150px);</code>
</div>
```

```
div {
  background-color: black;
  margin: 4px 0;
  width: 100%;
  height: 48px;
}

div > code {
  display: block;
  background-color: darkgreen;
  color: white;
  height: 48px;
}

.clamp1 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `20%` of the container's width */
  width: clamp(20%, 1px, 80%);
}

.clamp2 > code {
  /* Output width: Depends on the container's width; */
  /* on this page, likely to be `90%` of the container's width */
  width: clamp(10%, 9999px, 90%);
}

.clamp3 > code {
  /* Output width: `125px` */
  width: clamp(125px, 1px, 250px);
}

.clamp4 > code {
  /* Output width: `150px` */
  width: clamp(25px, 9999px, 150px);
}
```

Advanced CSS Math Functions

When laying out and styling DOM elements, the four basic math functions `calc()`, `min()`, `max()`, and `clamp()` are often sufficient. However, for advanced uses like mathematics learning materials, 3D visualizations, or CSS animations, you may consider using:

- Stepped value functions
  - `round()`: calculates a **value given a rounding strategy**
  - `mod()`: calculates the **remainder** of a division operation with the **same sign as the divisor**
  - `rem()`: calculates the **remainder** of a division operation with the **same sign as the dividend**
- Trigonometric functions
  - `sin()`: calculates the **trigonometric sine** of a number
  - `cos()`: calculates the **trigonometric cosine** of a number
  - `tan()`: calculates the **trigonometric tangent** of a number
  - `asin()`: calculates the **trigonometric inverse** sine of a number
  - `acos()`: calculates the **trigonometric inverse** cosine of a number
  - `atan()`: calculates the **trigonometric inverse** tangent of a number
  - `atan2()`: calculates the **trigonometric inverse** tangent given two numbers
- Exponential functions
  - `pow()`: calculates a number **raised to the power** of another number
  - `sqrt()`: calculates the **square root** of a number
  - `hypot()`: calculates the **square root of the sum of the squares** of the given numbers
  - `log()`: calculates the **logarithm** of a number (with `e` as the default base)
  - `exp()`: calculates `e` **raised to the power** of another number
- Sign functions
  - `abs()`: calculates the **absolute value** of a number
  - `sign()`: calculates the **sign (positive, negative, or zero)** of a number

Final thoughts

- You can use CSS math functions to create responsive user interfaces without writing any JavaScript code.
- CSS math functions can sometimes be used instead of CSS media queries to define layout breakpoints.
- In 2023, members of the Interop Project selected "CSS Math Functions" as a focus area of improvement. This means that browser vendors are working together to ensure that CSS math functions perform the same across browsers and devices.

Using CSS typed arithmetic

CSS typed arithmetic refers to permitted calculations performed with typed CSS values via functions such as `calc()`, and specifically, to the behavior defined in the CSS values and units module. CSS typed arithmetic enables dividing a value with one unit by a value with a different unit of the same data type, resulting in unitless quotients.

These quotients can then be used as a `<number>` in the values of unitless properties, or converted into any numeric data type (such as a `<length>`, `<percentage>`, or `<angle>`) by multiplying them by a numeric typed value.

This typed arithmetic behavior enables relationships to be created between different values on a page. This article explores typed arithmetic and presents several examples that make use of it.

> **Note:** Originally, the CSS values and units module limited the arguments multiplication and division operations could take to make dividing by zero detectable at parse time and avoid issues such as squared units. Typed arithmetic relaxes some of those restrictions in supporting browsers.

CSS typed arithmetic rules

When performing calculations on values in CSS, there are some rules around compatibility of values with different data types.

Addition and subtraction

When adding or subtracting values, all the values must be within the same overall data type. The following examples all produce valid results:

```
/* <length-percentage> units */
calc(250px - 150px)
calc(100% - 50px)
calc(50vw + 2rem)
calc(25cqw + 3in)

/* <angle> units */
calc(40deg + 2rad)
calc(420deg - 1turn)
```

However, the following are invalid. The results of such mixed-type calculations would be meaningless:

```
calc(200px + 100ms)
calc(50% + 90deg)
```

Multiplication

When multiplying values in CSS, only one of the values can have a unit. All other values must be unitless `<number>` values. This is because you want a product that is a greater or lesser amount of the same unit, and do not want to create squared units, which no CSS properties accept.

```
calc(200px * 4) /* 800px */
calc(60deg * 3) /* 180deg */
```

If you try to multiply two values with units — even the same units — normal mathematical rules dictate that the units should be multiplied as well. For example:

```
calc(200px * 4px)
```

In this case, the function is invalid as `800px`^{2} is meaningless in CSS.

Division

When dividing values in CSS, you can divide a value with a unit by a unitless value:

```
calc(1000px / 2) /* 500px */
calc(360deg / 4) /* 90deg */
```

However, you can't divide a unitless value by a value with a unit as that doesn't make sense:

```
calc(1000 / 2px) /* ?!? */
```

When a value of any numeric data type is divided by a value of the same type, the units cancel each other out, and you are left with a unitless value. Behind the scenes, the two values are computed before being divided by one another.

As a result, the same calculation can have very different results depending on the context it is used in and the units of the divisor.

Take the following example:

```
calc(100vw / 1px)
```

The `100vw` is equal to `100%` of the width of the viewport. If the viewport is currently `1000px` wide, the calculation will return the unitless value `1000`. However, if the viewport is `500px` wide, the calculation will return the unitless value `500`.

If we replace the `1px` divisor by `1em`, we get very different results:

```
calc(100vw / 1em)
```

If the viewport is `1000px` wide, and `1em` is equal to the browser default of `16px` at the point of calculation, the previous calculation will return `1000px / 16px = 62.5`.

It used to be the case that you couldn't divide one typed value by another, even if the two values have units of the same type. However, the specification was updated to permit this; check for browser compatibility.

Why is the behavioral update useful?

Being able to divide one typed value by another of the same type may not sound very significant at first, but it enables useful associations between different values that can be used to create responsive UI features.

The key to all of this is being able to represent values in a unitless form via divisions like this one:

```
--viewport-width-in-pixels: calc(100vw / 1px);
```

The result is a `<number>` representing the viewport width in pixels as a unitless value. This can be used anywhere a number is valid, including other `calc()` functions. You can dynamically vary other property values based on this value, regardless of what units they have.

For example, the unitless value can be immediately transferred to `opacity`:

```
opacity: calc(var(--viewport-width-in-pixels) / 1000 - 0.5);
```

You can multiply it by a value such as `1deg` to create an `<angle>` value:

```
rotate: calc(var(--viewport-width-in-pixels) * 1deg);
```

You can multiply it by a value such as `1rem` to create a `<length>` value:

```
font-size: calc(var(--viewport-width-in-pixels) * 1rem / 200);
```

Let's walk through some examples to show how CSS typed arithmetic can be useful.

Basic example: responsive background opacity

The responsive background opacity example (see source code) shows how to vary the opacity of a background image as the viewport width is changed. Having the background image fade out as the screen gets narrower means the image is less likely to affect the readability of the text content as they start to overlap.

HTML

The HTML contains some basic text content wrapped in a `<div>` element.

```
<div class="wrapper">
  <h1>Prose of the century</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</div>
```

CSS

We start by defining a CSS custom property called `--width-percentage` on the `:root` element, which contains the result of the calculation `100vw / 2000px`. This value represents the viewport width as a percentage of `2000px` that we will later use as an alpha channel value. When the viewport is `2000px` wide, the calculation will return `1`, which is equivalent to `100%` alpha. Anything less than a `2000px` viewport width will result in a smaller value.

```
:root {
  --width-percentage: calc((100vw / 2000px));
}
```

> **Note:** Any alpha value greater than `1` is treated as `1`, therefore we don't need to clamp the maximum value.

We then set a fixed `width` and some `padding` on the wrapper `<div>`, and center it horizontally using `margin`.

```
.wrapper {
  width: 480px;
  padding: 20px;
  margin: 0 auto;
}
```

Finally, we set multiple backgrounds on the `<body>` element. We'll go through these in reverse order:

1. The last and therefore bottom-most `background` value is a non-repeating background image located near the top-right corner.
2. The top background is a white semi-transparent overlay created using a `linear-gradient()` function with both color stops set to the same color. The color's alpha channel value is calculated as `1` minus the `--width-percentage` custom property we set earlier: As the viewport width gets narrower, `--width-percentage` will get smaller, therefore the white gradient will become more opaque, reducing the opacity of the background image.

```
body {
  background:
    linear-gradient(
      rgb(255 255 255 / calc(1 - var(--width-percentage))),
      rgb(255 255 255 / calc(1 - var(--width-percentage)))
    ),
    url("https://mdn.github.io/shared-assets/images/examples/colorful-heart.png")
      no-repeat top 50px right 50px;
}
```

Result

To see the result, view our responsive background opacity example live (see source code). Try altering the viewport width to see how the background opacity varies as a result.

Varying values of different types based on a single value

The different type variations example (see source code) works in a very similar way to the previous example, except that in this case, we demonstrate the use of a single variable value in the definition of multiple other values with different data types.

Specifically, we set a paragraph's background color *and* font size based on the viewport width.

HTML

The markup for this example contains a paragraph of text:

```
<p>Hello there!</p>
```

CSS

We start off our CSS in a similar way to the previous demo, creating a unitless value based on the viewport width. This time, we calculate it by dividing `100vw` by `1px` and store the resulting number in a custom property called `--viewport-in-pixels`. The value represents the current viewport width in pixels, without any units.

```
:root {
  --viewport-in-pixels: calc(100vw / 1px);
}
```

Now on to the styling of the paragraph itself. We first give it some basic styles (a `border` and `text-align` of `center`), then set two values based on the `--viewport-in-pixels` property created earlier:

1. We set the `font-size` of the paragraph to a value equal to `--viewport-in-pixels` divided by `200`, multiplied by `1em` to convert the numeric quotient into `em`s.
2. We set the `background-color` of the paragraph to an `lch()` color value. The lightness and chroma components are constant values (`75%` and `50%`, respectively), whereas the hue component is set to `--viewport-in-pixels` divided by `10`, plus `100`. We then multiply the result of that by `1deg` to ensure the value is an `<angle>`.

   > **Note:** This last step is not strictly necessary, as `lch()` also accepts unitless hue values. However, a degree value may be more intuitive, and we wanted to show another example of how the unitless value can be converted into a different data type.

```
p {
  border: 5px solid black;
  text-align: center;
  font-size: calc(1em * (var(--viewport-in-pixels) / 200));
  background-color: lch(
    75% 50% calc((100 + (var(--viewport-in-pixels) / 10)) * 1deg)
```

```
  );
}
```

Result

To see the result, view our different type variations example live (see source code). Try altering the viewport width to see how the paragraph font size and background color both vary as a result.

An animated story circle

The animated story circle example (see source code) demonstrates a more complex effect created by leveraging CSS typed arithmetic rules. In this case, the different paragraphs of a body of text are laid out in a circle emanating from a center point. What's more, the circle of paragraphs closes up into a fan shape when the viewport is made narrower, and opens up again when it is made wider.

HTML

The HTML is fairly basic — a `<div>` containing multiple `<p>` elements. The text of the first paragraph is wrapped in a `<strong>` element to provide an indicator of where to start reading.

```
<div class="story-circle">
  <p><strong>Hello there!</strong></p>
  <p>This is</p>
  <p>quite an</p>
  <p>interesting way</p>
  <p>to tell a</p>
  <p>story</p>
  <p>all around in</p>
  <p>a circle.</p>
  <p>What fun!</p>
</div>
```

CSS

We start the CSS by setting a `height` of `100%` on the `:root` element.

```
:root {
  height: 100%;
}
```

Next, we set several properties on the `<body>` element:

- We start by setting a `height` of `inherit`, meaning the `<body>` will inherit the `:root` element's `100%` height and therefore span the full height of the viewport.
- Next, we horizontally center the `<body>` using `margin`, and give it a `max-width`. As you'll see later on, this upper bound is important for controlling the maximum rotation of the fan/circle shape.
- We center the `<div>` with `class="story-circle"` horizontally and vertically inside the `<body>` using flexbox.
- We use the `container-type` property to declare the `<body>` as an inline size query container. This is important because we want to vary the shape rotation based on the `<body>` width, and not the viewport width as we did in previous examples. Setting it as a size query container allows us to reference its size in calculations.

```
body {
  height: inherit;
  margin: 0 auto;
  max-width: 1600px;
  display: flex;
  align-items: center;
  justify-content: center;
  container-type: inline-size;
}
```

We now come to styling the `story-circle` `<div>`. We set its `width` and `height` to `1px`: it will act as a reference point with its child paragraphs positioned in a circle around it. (We don't even need to set positioning on it, as it is fine for the paragraphs to be positioned relative to the `<body>`).

We then create a custom property called `--width-percentage` that contains the result of `100cqw` (100% of the width of the element's parent query container, which is the `<body>` element) divided by `1200px`, minus `0.33333`. This is the key value that will control the amount the circle rotates by as the viewport width is changed.

```
.story-circle {
  width: 1px;
  height: 1px;
  --width-percentage: calc((100cqw / 1200px) - 0.33333);
}
```

We are using the container query width rather than the viewport width to control the rotation because we want to set a maximum bound for the container, which is controlled by the `<body>` `max-width`. If the rotation value becomes greater than a full circle, the last paragraph will start to overlap the first one, which spoils the effect.

Since the `max-width` is `1600px`, you might have expected the calculation to be `100cqw / 1600px`). This would work, but we've instead gone for `(100cqw / 1200px) - 0.33333` (the `0.33333` comes from `1600px/1200px - 1`). Both would cause the maximum rotation to occur at a `<body>` width of `1600px`, but the fan now has a smaller minimum rotation, which gives a better effect at narrow viewport widths.

The final style rule selects the paragraphs themselves. Most of this styling is rudimentary. It is worth pointing out that we have set `position` to `absolute` to cause all of the paragraphs to sit on top of one another, and we've set a `transform-origin` value of `center left` to cause paragraphs to rotate around the center of their left edge, so they will all fan out from a center point over their containing `<div>`.

Now onto the interesting bit — we define a custom property called `--angle` that contains the unitless rotation angle of the paragraph, before setting the `rotate` property to the resulting value. We multiply the resulting number by `1deg` to convert it to a degree value. The `--angle` custom property is the product of three values:

1. The paragraph's `sibling-index()` minus `1`, which causes the first paragraph to have a rotation angle of `0`, as we want it to be horizontal.
2. `360` divided by the paragraph's `sibling-count()`, which causes all the paragraphs to be equally spaced around the circle, meaning the design will still work if the number of paragraphs changes.
3. our `--width-percentage` custom property, which causes the rotation of the paragraphs around the circle to vary as the viewport width changes. Remember that this property has a maximum value of `1`, which will be achieved when the `<body>` element hits its `max-width` of `1600px`.

```
p {
  padding: 10px;
  width: 150px;
  text-align: right;
  background: linear-gradient(to right, red, orange 50%);
  border-radius: 5px;

  position: absolute;
  transform-origin: center left;

  --angle: calc(
    ((sibling-index() - 1) * (360 / sibling-count())) * var(--width-percentage)
  );
```

```
  rotate: calc(var(--angle) * 1deg);
}
```

Result

To see the result, view our animated story circle example live (see source code). Try increasing and decreasing the viewport width to see how the paragraph fan shape unfolds into a circle and back again.

See also

- `calc()`, `abs()`
- CSS values and units module
- CSS Typed Arithmetic on css-tricks.com (2025)

Value definition syntax

CSS value definition syntax, a formal grammar, is used to define the set of valid values for a CSS property or function. In addition to this syntax, the set of valid values can be further restricted by semantic constraints (for example, for a number to be strictly positive).

The definition syntax describes which values are allowed and the interactions between them. A component can be a *keyword*, some characters considered as a *literal*, or a value of a given CSS data type or another CSS property.

Component value types

Keywords

Generic keywords

A keyword with a predefined meaning appears literally, without quotation marks. For example: `auto`, `smaller`, or `ease-in`.

CSS-wide keywords

All CSS properties accept the keywords `inherit`, `initial`, `revert`, `revert-layer`, and `unset`. They are not shown in the value definition and are implicitly defined.

Literals

In CSS, a few characters can appear on their own, like the slash (`/`) or the comma (`,`), and are used in a property definition to separate its parts. The comma is often used to separate values in enumerations, or parameters in mathematical-like functions; the slash often separates parts of the value that are semantically different, but have a common syntax. Typically, the slash is used in shorthand properties; to separate components of the same type, but belong to different properties.

Both symbols appear literally in a value definition.

Data types

Basic data types

Some data types are used throughout CSS and are defined once for all values in the specification. Called *basic data types*, they are represented with their name surrounded by the symbols < and >: `<angle>`, `<string>`, ...

Non-terminal data types

Less common data types, called *non-terminal data types*, are also surrounded by < and >.

Non-terminal data types are of two kinds:

- data types *sharing the same name of a property*, put between quotes. In this case, the data type shares the same set of values as the property. They are often used in the definition of shorthand properties.
- data types *not sharing the same name of a property*. These data types are very close to the basic data types. They only differ from the basic data types in the physical location of their definition. In this case, the definition is usually physically very close to the definition of the property using them.

Component value combinators

Brackets

Brackets enclose several entities, combinators, and multipliers, then transform them as a single component. They are used to **group components to bypass the precedence rules**.

```
example =
  bold [ thin && <length> ]
```

This example matches the following values:

- `bold thin 2vh`
- `bold 0 thin`
- `bold thin 3.5em`

But not:

- `thin bold 3em`, as `bold` is juxtaposed with the component defined by the brackets, it must appear before it.

Juxtaposition

Placing several keywords, literals, or data types, next to one another, only separated by one or several spaces, is called *juxtaposition*. All juxtaposed components are **mandatory and should appear in the exact order**.

```
example =
  bold <length> , thin
```

This example matches the following values:

- `bold 1em, thin`
- `bold 0, thin`
- `bold 2.5cm, thin`
- `bold 3vh, thin`

But not:

- `thin 1em, bold`, as the entities must be in the expressed order
- `bold 1em thin`, as the entities are mandatory; the comma, a literal, must be present
- `bold 0.5ms, thin`, as the `ms` values are not `<length>`

Double ampersand

Separating two or more components, by a *double ampersand*, `&&`, means that all these entities are **mandatory but may appear in any order**.

```
example =
  bold      &&
  <length>
```

This example matches the following values:

- `bold 1em`
- `bold 0`
- `2.5cm bold`
- `3vh bold`

But not:

- `bold`, as both components must appear in the value.
- `bold 1em bold`, as both components must appear only once.

> **Note:** Juxtaposition has precedence over the double ampersand, meaning that `bold thin && <length>` is equivalent to `[ bold thin ] && <length>`. It describes `bold thin <length>` or `<length> bold thin` but not `bold <length> thin`.

Double bar

Separating two or more components by a *double bar*, `||`, means that all entities are options: **at least one must be present, and they may appear in any order**. Typically this is used to define the different values of a shorthand property.

```
example =
  <number>  ||
  <length>  ||
  <color>
```

This example matches the following values:

- `1em 1 blue`
- `blue 1em`
- `1 1px yellow`

But not:

- `blue yellow`, as a component must appear once at most.
- `bold`, as it isn't a keyword allowed as a value of any of the entities.

> **Note:** The double ampersand has precedence over the double bar, meaning that `bold || thin && <length>` is equivalent to `bold || [ thin && <length> ]`. It describes `bold`, `thin <length>`, `bold thin <length>`, or `thin <length> bold` but not `<length> bold thin` as bold, if not omitted, must be placed before or after the whole `thin && <length>` component.

Single bar

Separating two or more entities by a *single bar*, `|`, means that all entities are exclusive options: **exactly one of these options must be present**. This is typically used to separate a list of possible keywords.

```
example =
  <percentage>  |
  <length>      |
  left          |
  center        |
  right         |
  top           |
  bottom
```

This example matches the following values:

- `3%`
- `0`
- `3.5em`
- `left`
- `center`
- `right`
- `top`
- `bottom`

But not:

- `center 3%`, as only one of the components must be present.
- `3em 4.5em`, as a component must be present at most one time.

> **Note:** The double bar has precedence over the single bar, meaning that `bold | thin || <length>` is equivalent to `bold | [ thin || <length> ]`. It describes `bold`, `thin`, `<length>`, `<length> thin`, or `thin <length>` but not `bold <length>` as only one entity from each side of the `|` combinator can be present.

Component value multipliers

A multiplier is a sign that indicates how many times a preceding entity can be repeated. Without a multiplier, an entity must appear exactly one time.

Multipliers cannot be added and have precedence over all combinators.

Asterisk (<code>*</code>)

The *asterisk multiplier* indicates that the entity may appear **zero, one or several times**.

```
example =
  bold smaller*
```

This example matches the following values:

- `bold`
- `bold smaller`
- `bold smaller smaller`
- `bold smaller smaller smaller`, and so on.

But not:

- `smaller`, as `bold` is juxtaposed, and must appear before any `smaller` keyword.

Plus (<code>+</code>)

The *plus multiplier* indicates that the entity may appear **one or several times**.

```
example =
  bold smaller+
```

This example matches the following values:

- `bold smaller`
- `bold smaller smaller`
- `bold smaller smaller smaller`, and so on.

But not:

- `bold`, as `smaller` must appear at least once.
- `smaller`, as `bold` is juxtaposed and must appear before any `smaller` keyword.

Question mark (<code>?</code>)

The *question mark multiplier* indicates that the entity is optional, and **must appear zero or one time**.

```
example =
  bold smaller?
```

This example matches the following values:

- `bold`
- `bold smaller`

But not:

- `bold smaller smaller`, as `smaller` must appear at most once.
- `smaller`, as `bold` is juxtaposed and must appear before any `smaller` keyword.

Curly braces (<code>{}</code>)

The *curly braces multiplier*, enclosing two integers separated by a comma, A and B, indicates that the entity **must appear at least A times and at most B times**.

```
example =
  bold smaller{1,3}
```

This example matches the following values:

- `bold smaller`
- `bold smaller smaller`
- `bold smaller smaller smaller`

But not:

- `bold`, as `smaller` must appear at least once.
- `bold smaller smaller smaller smaller`, as `smaller` must appear at most three times.
- `smaller`, as `bold` is juxtaposed and must appear before any `smaller` keyword

Hash mark (<code>#</code>)

The *hash mark multiplier* indicates that the entity may be repeated one or more times (for example, the plus multiplier), but each occurrence is separated by a comma (','').

```
example =
  bold smaller#
```

This example matches the following values:

- `bold smaller`
- `bold smaller, smaller`
- `bold smaller, smaller, smaller`, and so on.

But not:

- `bold`, as `smaller` must appear at least once.
- `bold smaller smaller smaller`, as the different occurrences of `smaller` must be separated by commas.
- `smaller`, as `bold` is juxtaposed and must appear before any `smaller` keyword.

The hash mark may optionally be followed by curly braces to indicate how many times the entity repeats.

```
example =
  bold smaller#{1,3}
```

This example matches the following values:

- `bold smaller`
- `bold smaller, smaller`
- `bold smaller, smaller, smaller`

But not:

- `bold smaller, smaller, smaller, smaller`, as `smaller` must appear at most three times.

```
example =
  bold smaller#{2}
```

This example matches the following value:

- `bold smaller, smaller`

But not:

- `bold smaller`, as `smaller` must appear exactly two times.

Exclamation point (<code>!</code>)

The *exclamation point multiplier* after a group indicates that the group is required, and must produce at least one value; even if the grammar of the items within the group would otherwise allow the entire contents to be omitted, at least one component value must not be omitted.

```
example =
  [ bold? smaller? ]!
```

This example matches the following values:

- `bold`
- `smaller`
- `bold smaller`

But not:

- neither `bold` nor `smaller`, as one of them must appear.
- `smaller bold`, as `bold` is juxtaposed and must appear before the `smaller` keyword.
- `bold smaller bold`, as `bold` and `smaller` may only appear once.

Bracketed range notation (<code>[min,max]</code>)

Some types can accept numeric values within a certain range. For example, the `column-count` property can accept an integer value between positive 1 and infinity, inclusive. The corresponding syntax looks like this:

```
example =
  <integer [1,∞]>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Any value outside this specified range causes the whole declaration to be invalid, therefore the browser will ignore it.

The *bracketed range notation* `[min, max]` indicates an inclusive range between a `min` and `max` value. This notation is used in numeric type notations and can include units, e.g., `<angle [0,180deg]>`. Positive and negative Infinity (-∞ and ∞) must not have units specified. Types specified in units can have zero values specified with or without units, for example `<time [0s,10s]>` or `<time [0,10s]>`.

Here are some more examples:

- `<integer [-∞,∞]>`: Any integer from negative infinity to positive infinity.
- `<integer [0,∞]>`: Any integer from 0 to positive infinity is valid. Negative integers are invalid.
- `<time [0s,10s]>` or `<time [0,10s]>`: Any duration from 0 to 10 seconds is valid.
- `<integer [-∞,-1]> | <integer [1,∞]>`: Any integer except zero is valid.

Summary

Sign	Name	Description	Example
Combinators			
	Juxtaposition	Components are mandatory and should appear in that order	`solid <length>`
`&&`	Double ampersand	Components are mandatory but may appear in any order	`<length> && <string>`
`||`	Double bar	At least one of the components must be present, and they may appear in any order.	`<'border-image-outset'> || <'border-image-slice'>`
`|`	Single bar	Exactly one of the components must be present	`smaller | small | normal | big | bigger`
`[ ]`	Brackets	Group components to bypass precedence rules	`bold [ thin && <length> ]`
Multipliers			
	No multiplier	Exactly 1 time	`solid`
`*`	Asterisk	0 or more times	`bold smaller*`
`+`	Plus sign	1 or more times	`bold smaller+`
`?`	Question mark	0 or 1 time (that is *optional*)	`bold smaller?`
`{min,max}`	Curly braces	At least `min` times, at most `max` times	`bold smaller{1,3}`
`#`	Hash mark	1 or more times, with each occurrence separated by a comma (`,`)	`bold smaller#`
`!`	Exclamation point	Group must produce at least 1 value	`[ bold? smaller? ]!`
Ranges			
`[min,max]`	Numeric bracketed range	Specifies a numeric range	`<integer [0,∞]>`

See also

- CSS key concepts:
  - CSS syntax
  - Comments
  - Specificity
  - Inheritance
  - Box model
  - Layout modes

        - Visual formatting model
        - Margin collapsing
        - Values
            - Initial values
            - Computed values
            - Used values
            - Actual values
        - **Value definition syntax**
        - Shorthand properties
        - Replaced elements

CSS view transitions

The **CSS view transitions** module defines the behavior of the View Transition API, which allows developers to create animated transitions between different states within a document and across documents. This module also defines the CSS properties and pseudo-elements for styling these transitions.

Reference

Properties

- `view-transition-class`
- `view-transition-name`
- `view-transition-scope`

At-rules and descriptors

- `@view-transition`
    - `navigation` descriptor

Selectors and pseudo-elements

- `:active-view-transition`
- `:active-view-transition-type()`
- `::view-transition`
- `::view-transition-image-pair()`
- `::view-transition-group()`
- `::view-transition-new()`
- `::view-transition-old()`

Interfaces

- `CSSViewTransitionRule`
- `ViewTransition`
    - `ViewTransition.skipTransition()` method
    - `ViewTransition.updateCallbackDone`
    - `ViewTransition.ready`
    - `ViewTransition.finished`
    - `ViewTransition.transitionRoot`
- `Document.startViewTransition()` method
- `Document.activeViewTransition`
- `Element.startViewTransition()` method
- `Element.activeViewTransition`

Guides

Using the View Transition API

Explains how to create view transitions and customize transition animations, including manipulating active view transitions.

Using view transition types

Shows how to use types along with same-document view transitions and cross-document view transitions.

Using element-scoped view transitions

Covers how element-scoped view transitions work, and how to use them.

Related concepts

- `pagereveal` event
- `pageswap` event
- `Document.visibilityState`
- CSS animations module
    - `animation`
    - `@keyframes`
    - `CSSKeyframesRule`
    - `CSSStyleRule`
    - Web animations API
- CSS transforms module
    - `transform`
    - `<transform-function>`

See also

- Pseudo-elements
- Functional pseudo-classes
- Learn: Pseudo-classes and pseudo-elements

CSS viewport

The **CSS viewport** module enables specifying the size, zoom factor, and orientation of the user-agent's initial containing block, or *viewport*.

Content designed for large viewports may exhibit a variety of bugs when viewed in smaller viewports, including unintended wrapping, clipped content, and incorrectly sized scroll containers. HTML provides a viewport meta tag, `<meta name="viewport">`, to provide hints about the initial size of the viewport. If the site isn't designed to work well on small viewports and this tag is omitted, some mobile browsers render the site using a fixed initial containing block width, typically `980px`. The content is then scaled down, making the CSS pixel size smaller than an actual pixel. The resulting page fits into the available screen space but is illegible, requiring the user to zoom and pan to view the content.

The viewport initial containing block for continuous media has the dimensions of the viewport. Since the viewport is generally no larger than the display, devices with smaller displays, such as phones or tablets, typically present a smaller viewport than larger devices like desktops or laptops.

Reference

Properties

- `zoom`

Interfaces

- `Window.Viewport`
- `Viewport`
  - `Viewport.segments`

Glossary terms and definitions

- Viewport
- Actual viewport
- Initial viewport

Guides

Viewport concepts

> The concept of the viewport — what it is, its impact in terms of CSS, SVG, and mobile devices — and the difference between the visual viewport and the layout viewport.

Using environment variables

> An overview of what environment variables are, browser-defined environment variables, and how to use the `env()` function.

Using the Viewport Segments API

> Create responsive designs optimized for different viewport segment sizes and arrangements with the API and environment variables.

Related concepts

- CSS media queries module
  - `@media`
  - `horizontal-viewport-segments` descriptor
  - `vertical-viewport-segments` descriptor
- CSS environment variables
  - `env()`
  - `<environment-variable-name>`
- Device Posture API

See also

- CSSOM view module

WebXR DOM overlays

The **WebXR DOM overlays** module, an immersive web specification, expands the WebXR Device API to allow HTML content — such as text and controls — to be displayed during an immersive virtual reality (VR) or augmented reality (AR) session.

In a typical WebXR session, content is rendered to a WebGL canvas superimposed on the real-world camera feed (in AR) or displayed as the full immersive view (in VR), with the device tracking the user's position and orientation. The DOM overlay renders a single DOM element and its descendants as a transparent-background 2D rectangle on top of this view, enabling interactive, stylable elements such as menus and dialogs to appear within the immersive experience.

The `:xr-overlay` pseudo-class matches the DOM overlay element when a web page is being viewed in an immersive AR or VR environment, allowing it to be styled appropriately for its underlying content.

Reference

Pseudo-classes

- `:xr-overlay`

Events

- `Element` events
  - `beforexrselect`

Interfaces

- `XRSession`
  - `domOverlayState` property

Guides

Fundamentals of WebXR

> Introduction to WebXR device API fundamentals.

Related concepts

- CSS pseudo-classes
  - `:modal`

    - `:fullscreen`
    - `:picture-in-picture`

- WebXR Device API
    - `XRWebGLLayer` interface

See also

- Stacking context
- CSSOM view module
- CSSOM View API

CSS will-change

The **CSS will change** module provides a hint to the browser to enable optimizations that can improve rendering performance by doing potentially expensive work before it is required via the `will-change` property.

Reference

Properties

- `will-change`

Related concepts

- CSS transforms
    - `transform`
    - `scale`
    - `rotate`
    - `translate`
- CSS filter effects
    - `backdrop-filter`
    - `filter`
    - The CSS filter functions
- CSS animations
    - `animation` shorthand
    - `@keyframes`
- CSS colors
    - `opacity`

See also

- CSS scroll-driven animations module
- Web Animations API
- Animation Interface
- Web performance

CSS writing modes

The **CSS writing modes** module defines support for various international writing modes and their combinations, including left-to-right and right-to-left text ordering as well as horizontal and vertical orientations.

A *writing mode* in CSS is determined by the `writing-mode`, `direction`, and `text-orientation` properties defined in this module. It is defined primarily in terms of its inline base direction and block flow direction.

Some horizontal languages are left-to-right, including Latin and Indic scripts. Other horizontal languages are written right-to-left, including Hebrew and Arabic scripts. Sometimes text needs to be bidirectional, such as when mixing left-to-right and right-to-left scripts. Some languages can be written with a vertical orientation, for example, Chinese, Japanese, and Korean (CJK) scripts.

The CSS writing modes module addresses the orientations of all writing modes. Other modules, such as the CSS ruby layout module, provide rendering models and formatting controls related to the display text annotations.

Reference

Properties

- `direction`
- `glyph-orientation-vertical`
- `text-combine-upright`
- `text-orientation`
- `unicode-bidi`
- `writing-mode`

Glossary and terms

- Baseline
- Internationalization
- Localization
- Leading

Guides

Creating vertical form controls

> The article explains how to use the CSS `writing-mode` and `direction` properties to create and configure vertical form controls.

Introduction to writing mode systems

> A brief overview of writing mode systems and their directionalities.

Related concepts

CSS text module

- `hanging-punctuation`
- `hyphens`
- `letter-spacing`
- `line-break`
- `overflow-wrap`
- `text-align`
- `text-align-last`
- `text-indent`
- `text-justify`
- `word-break`
- `word-spacing`

CSS inline layout module

- `alignment-baseline`
- `dominant-baseline`
- `line-height`
- `text-box-edge`
- `text-box-trim`
- `text-box` shorthand
- `<text-edge>`

CSS logical properties and values module

- Flow relative values
- Inset properties
- Logical properties
- Physical properties

CSS display module

- `display`
- `<display-internal>`
- Visual formatting model

CSS generated content

- `quotes`

SVG

- `glyph-orientation-horizontal`
- `glyph-orientation-vertical`
- `writing-mode`

HTML

- `<bdo>`
- `<blockquote>`
- `<q>`
- `<ruby>`
- `<sub>`
- `<sup>`
- `dir` attribute
- `lang` attribute

JavaScript

- Internationalization guide
- `Intl` object

See also

- CSS fonts module
- CSS ruby layout module
- CSS text decoration module
- CSS counter styles module
- CSS lists module
- CSS containment module: `contain-intrinsic-block-size` and `contain-intrinsic-inline-size`
- CSS overflow module: `overflow-block` and `overflow-inline`
- CSS overscroll behavior module: `overscroll-behavior-block` and `overscroll-behavior-inline`

Creating vertical form controls

The guide explains how to use the CSS `writing-mode` and `direction` properties to create and configure vertical form controls. This includes:

- `<input type="range">` sliders, `<progress>` bars, and `<meter>` elements.
- `<select>` elements.
- `<button>` elements and button input types such as `<input type="button">`, `<input type="reset">`, and `<input type="submit">`.
- `<textarea>` elements and text-based input types such as `<input type="text">`, `<input type="datetime-local">`, and `<input type="url">`.

General technique

In modern browsers, the `writing-mode` property can be set to a vertical value to vertically display form controls with text characters that are normally horizontal (for example in Latin languages), with text displayed at a 90-degree angle from the default. Normally vertical text characters, for example in Chinese or Japanese, are unaffected in this regard.

This is useful when creating vertical language forms.

Specifically:

- `writing-mode: vertical-lr` will create vertical form controls with a left-to-right block flow direction, meaning that in controls with wrapping or multiple lines of text, subsequent lines will appear to the right of previous lines.
- `writing-mode: vertical-rl` will create vertical form controls with a right-to-left block flow direction, meaning that in controls with wrapping or multiple lines of text, subsequent lines will appear to the left of previous lines.

You could use a transform to rotate the controls by 90 degrees, but that would place the controls in their own layer and cause unintended layout side effects such as other content being overlapped. Using `writing-mode` offers a more reliable solution.

> **Note:** While the `writing-mode` property is well supported, creating vertically-oriented form controls with `writing-mode` only gained full browser support in 2024.

> **Note:** The experimental `sideways-lr` and `sideways-rl` values have a similar effect as `vertical-lr` and `vertical-rl` respectively, except that normally vertical text characters (for example in Chinese or Japanese) are rotated 90 degrees to display on their sides, while horizontal text characters (for example in Latin languages) are unaffected by these values.

In addition, the `direction` property can be used to control the direction of the content inside the controls:

- `direction: ltr` will cause the content to start at the top and flow towards the bottom.
- `direction: rtl` will cause the content to start at the bottom and flow towards the top.

The `direction` property can be used to set the **inline base direction** — the primary direction in which content is ordered on a line, which defines on which sides the "start" and "end" of a line are. For text-based form controls the difference is obvious — the flow of text starts at the top or bottom. In non-text-based controls such as range sliders, `direction` sets the direction in which the control is drawn. For example, including `direction: ltr` on a vertical slider sets the lowest value at the top of the slider and the highest value at the bottom of the slider.

The sections below show how to create different types of vertical form control, along with examples of each. Consult the browser compatibility information on each of the linked reference pages to find out the exact support information for each type.

Range sliders, meters, and progress bars

Let's have a look at creating vertical range sliders, meters, and progress bars.

Basic example

A typical set of visual `<input type="range">` slider, `<progress>`, and `<meter>` controls is created like this:

```
<input type="range" min="0" max="11" value="9" step="1" />
<meter id="fuel" min="0" max="100" low="33" high="66" optimum="80" value="20">
  at 50/100
</meter>
<progress id="file" max="100" value="70">70%</progress>
```

> **Note:** Best practice is to include a `<label>` element for each form control, to associate a meaningful text description with each field for accessibility purposes (see Use meaningful text labels for more information). We haven't done that here, as this article focuses purely on aspects of the form controls' visual rendering, but you should make sure you do so in production code.

To display the controls vertically, we can use CSS like this:

```
input[type="range"],
meter,
progress {
  margin-block-end: 20px;
  writing-mode: vertical-lr;
}
```

`writing-mode: vertical-lr` (and `vertical-rl`) causes the controls to be displayed vertically in modern browsers.

The result of this looks like so:

Drawing the control from bottom to top

By default, the controls have a `direction` value of `ltr`. This causes your sliders, meters, and progress bars to be drawn from top to bottom, as seen above.

You can change this by setting `direction: rtl` — this causes them to be drawn from bottom to top instead:

```
<input type="range" min="0" max="11" value="9" step="1" />
<meter id="fuel" min="0" max="100" low="33" high="66" optimum="80" value="20">
  at 50/100
</meter>
<progress id="file" max="100" value="70">70%</progress>
```

```
input[type="range"],
meter,
progress {
  margin-block-end: 20px;
  writing-mode: vertical-lr;
  direction: rtl;
}
```

The result of this looks like so:

Creating vertical range sliders in older browsers

In older browsers that do not support the creation of vertical form controls with `writing-mode` and `direction`, there are limited alternatives available. The following only work on `<input type="range">`, causing the text to flow from bottom to top — they have no effect on `<meter>` and `<progress>` elements:

- The non-standard `appearance: slider-vertical` property can be used in older versions of Safari and Chrome.
- The non-standard `orient="vertical"` attribute can be added to the `<input type="range">` element itself in older versions of Firefox.

The HTML for this example includes an `<input type="range">` slider only, with `orient="vertical"` added to make it display vertically in older Firefox versions:

```
<input type="range" min="0" max="11" value="9" step="1" orient="vertical" />
```

To cause the controls to also display vertically in older versions of Chrome and Safari, we can use `appearance: slider-vertical`:

```
input[type="range"] {
  margin-block-end: 20px;
  appearance: slider-vertical;
}
```

The result looks like so:

Select elements

This section shows how to handle vertical `<select>` elements.

Select basic example

The below HTML creates two `<select>` elements, one where a single selection can be made and one with multiple selections:

```
<select multiple>
  <option>First</option>
  <option>Second</option>
  <option>Third</option>
  <option>Fourth</option>
  <option>Fifth</option>
</select>
<select>
  <option>First</option>
  <option>Second</option>
  <option>Third</option>
  <option>Fourth</option>
  <option>Fifth</option>
</select>
```

To display the controls vertically, we can use CSS like this:

```
select {
  inline-size: 100px;
  margin-block-end: 20px;
  writing-mode: vertical-rl;
}
```

The result of this looks like so:

Adjusting text direction and option order

Again, it is possible to use a `direction` property value of `rtl` to set the text direction to go from bottom to top, instead of the default direction of top to bottom.

It is also worth noting that in the above example, the inline direction for the select options goes from right to left because we used `writing-mode: vertical-rl`. If we use `writing-mode: vertical-lr` instead, the `<option>` text will appear from left to right.

We'll explore these two use cases using three listbox (`multiple`) `<select>` elements, to enable comparing the effects side-by-side.

```
<div class="controls">
  <div>
    <h2>writing-mode: vertical-lr</h2>
    <select multiple>
      <option>First</option>
      <option>Second</option>
      <option>Third</option>
      <option>Fourth</option>
      <option>Fifth</option>
    </select>
  </div>
  <div class="direction">
    <h2>direction: rtl & writing-mode: vertical-lr</h2>
    <select multiple>
      <option>First</option>
      <option>Second</option>
      <option>Third</option>
      <option>Fourth</option>
      <option>Fifth</option>
    </select>
  </div>
  <div class="reverse-option-order">
    <h2>writing-mode: vertical-rl</h2>
    <select multiple>
      <option>First</option>
      <option>Second</option>
      <option>Third</option>
      <option>Fourth</option>
      <option>Fifth</option>
    </select>
  </div>
</div>
```

In the CSS for this example, we set the following properties on the three listboxes:

1. `writing-mode: vertical-rl`, displaying just like in the previous example — text flowing top-to-bottom, and options displaying right-to-left.
2. `writing-mode: vertical-rl` and `direction: rtl`, with the options going from right-to-left but reversing the text flow from bottom-to-top.
3. `writing-mode: vertical-lr`, with the text going from top-to-bottom while reversing the option order from left-to-right.

```
.controls {
  box-sizing: border-box;
  display: flex;
  gap: 20px;
  font-family: sans-serif;
}

h2 {
  margin-top: 0;
  font-size: 1rem;
  text-align: center;
```

```
  flex: 1 0 100%;
}

.controls > div {
  margin-bottom: 20px;
  flex: 1;
  display: flex;
  align-items: flex-start;
  justify-content: space-around;
  flex-flow: row wrap;
}
```

```
select {
  inline-size: 100px;
  margin-block-end: 20px;
  writing-mode: vertical-rl;
}

.direction select {
  direction: rtl;
}

.reverse-option-order select {
  writing-mode: vertical-lr;
}
```

The result looks like so:

Buttons

This section shows how to handle vertical `<button>` elements. Note that while we have only used a `<button>` element in the examples below, the behavior is the same for other elements that create buttons, such as `<input>` types of `button`, `reset`, and `submit`.

Basic button example

The below HTML creates two `<button>` elements, one with a single line of text, and one with three:

```
<button>Press me</button> <button>Press me<br />Please?<br />Thanks</button>
```

To display the buttons vertically, we can use CSS like this:

```
button {
  inline-size: 100px;
  margin-block-end: 20px;
  writing-mode: vertical-rl;
}
```

The result looks like so:

Adjusting button text line order

When you swap the `writing-mode` value of `vertical-rl` to `vertical-lr`, subsequent lines of text will appear to the right of previous lines, rather than the left.

This example uses two copies of the three-text-line button we saw in the previous example, so you can easily see the effects of changing the writing mode:

```
<div>
  <h2>writing-mode: vertical-lr</h2>
  <button>Press me<br />Please?<br />Thanks</button>
</div>
<div class="reverse-line-order">
  <h2>writing-mode: vertical-rl</h2>
  <button>Press me<br />Please?<br />Thanks</button>
</div>
```

In the CSS, we set `writing-mode: vertical-rl` on the first button to lay out the line order from right to left. On the second button, we set `writing-mode: vertical-lr` to reverse the line order — left to right:

```
body {
  box-sizing: border-box;
  display: flex;
  gap: 20px;
  font-family: sans-serif;
}

h2 {
  margin-top: 0;
  font-size: 1rem;
  text-align: center;
  flex: 1 0 100%;
}

div {
  margin-bottom: 20px;
  flex: 1;
  display: flex;
  align-items: flex-start;
  justify-content: space-around;
  flex-flow: row wrap;
}
```

```
button {
  inline-size: 100px;
  margin-block-end: 20px;
  writing-mode: vertical-rl;
}

.reverse-line-order button {
  writing-mode: vertical-lr;
}
```

The result looks like so:

Text-based form controls

Last but not least, we'll look at handling vertical `<textarea>`s and textual `<input>` types. Note that, while the only `<input>` type we are including is an `<input type="text">` element in the examples below, the behavior is the same for other textual `<input>` types: `password`, `number`, `url`, etc.

Basic text input and textarea example

The below HTML creates a `<textarea>` and an `<input type="text">`:

```
<textarea>This is my textarea</textarea>
<input type="text" value="Input text" />
```

To display the input and textarea vertically, we can use CSS like this:

```
textarea,
input[type="text"] {
  inline-size: 100px;
  margin-block-end: 20px;
  writing-mode: vertical-rl;
}
```

The result looks like so:

Adjusting text direction and line layout order

You can use a `direction` property value of `rtl` to change the text direction from the default top-to-bottom to bottom-to-top. You can also set `writing-mode` to `vertical-lr` instead of `vertical-rl`, to cause multiple lines of text in `<textarea>`s to appear from left-to-right rather than the default right-to-left.

This example uses three copies of the same text controls we saw in the previous example, so you can easily see the effects of changing `direction` and `writing-mode` as discussed above:

```
<div class="controls">
  <div>
    <h2>writing-mode: vertical-lr</h2>
    <textarea>This is my textarea</textarea>
    <input type="text" value="Input text" />
  </div>
  <div class="direction">
    <h2>direction: rtl & writing-mode: vertical-lr</h2>
    <textarea>This is my textarea</textarea>
    <input type="text" value="Input text" />
  </div>
  <div class="reverse-line-order">
    <h2>writing-mode: vertical-rl</h2>
    <textarea>This is my textarea</textarea>
    <input type="text" value="Input text" />
  </div>
</div>
```

In the CSS, we set the following properties on the three sets of text controls:

1. `writing-mode: vertical-rl` to make it display just like in the previous example — text flowing top-to-bottom, and lines flowing right-to-left.
2. `writing-mode: vertical-rl` and `direction: rtl` to flow the lines from right-to-left but reverse the text flow from bottom-to-top.
3. `writing-mode: vertical-lr` to flow the text top-to-bottom but reverse the flow of lines — left-to-right. Note that this has no effect on `<input type="text">` elements, as they are always single lines.

```
.controls {
  box-sizing: border-box;
  display: flex;
  gap: 20px;
  font-family: sans-serif;
}

h2 {
  margin-top: 0;
  font-size: 1rem;
  text-align: center;
  flex: 1 0 100%;
}

.controls > div {
  margin-bottom: 20px;
  flex: 1;
  display: flex;
  align-items: flex-start;
  justify-content: space-around;
  flex-flow: row wrap;
}
```

```
textarea,
input[type="text"] {
  inline-size: 100px;
  margin-block-end: 20px;
  writing-mode: vertical-rl;
}

.direction textarea,
.direction input[type="text"] {
  writing-mode: vertical-rl;
  direction: rtl;
}

.reverse-line-order textarea,
.reverse-line-order input[type="text"] {
  writing-mode: vertical-lr;
}
```

The result looks like so:

See also

- The `<input>` element.
- Other relevant elements: `<button>`, `<meter>`, `<progress>`, and `<select>`.
- Learn: Handling different text directions
- Styling web forms

Introduction to writing mode systems

CSS supports different content directionalities, or **writing modes**, including right-to-left, left-to-right, and top-to-bottom. The guide provides a brief overview of writing mode systems and their directionalities.

Block and inline

Before examining the directionality of various writing systems, it's important to understand the terms "block" and "inline". The term **inline** refers to how characters and words flow within a line. The term **block** refers to how lines, or blocks of content, stack up next to one another. The document's writing mode determines a document's block and inline directions. They are not based on physical directions such as left, right, top, and bottom.

Dimensions and directions

Everything on a web page is laid out in either the **inline** or **block** dimension. The *inline dimension* is the dimension along which a line of text runs in the current writing mode, whereas the *block dimension* is the dimension in which blocks — such as paragraphs — display one after the other. The inline dimension is perpendicular to the block dimension.

In an English document, with the text running horizontally left-to-right, or an Arabic document with the text running horizontally right-to-left, the inline dimension is horizontal, while the *inline direction* is left-to-right and right-to-left, respectively. In both cases, the block dimension is vertical, with the *block direction* being top-to-bottom. In a vertical writing mode such as Japanese, the inline dimension is vertical, as lines in that writing mode run vertically, while the block dimension is horizontal.

Inline and block boxes

The *outer display* type of boxes in a normal flowing layout determines how the box behaves alongside other elements on the page. *Inline boxes* wrap each line of text and are laid out along the inline dimension.

Block boxes represent containers on the page that can contain other block and inline elements. They are laid out along the block dimension and will extend in the inline dimension to fill all the space available in their container (provided a specific size is not set in the inline dimension using a property such as `inline-size`). Block boxes are only displayed from the top to the bottom of the page if you are using a writing mode that displays text horizontally, such as English.

The CSS logical properties and values module defines flow-relative mappings for many of the physical properties and values in CSS, which is helpful in understanding the basic concepts of logical properties and values.

Inline base and block flow directions

The *inline base direction* is the primary direction in which content is ordered on a line and defines the "start" and "end" of a line. The `direction` property specifies the inline base direction of a box and, together with the `unicode-bidi` property and the inherent directionality of any text content, determines the ordering of inline-level content within a line.

The *block flow direction* is the direction in which block-level and line boxes stack within a block container. The `writing-mode` property determines the block flow direction.

Writing system modes

Different writing systems have different writing modes. A horizontal writing mode is one with horizontal lines of text, i.e., a downward or upward block flow. A vertical writing mode is one with vertical lines of text, i.e., a leftward or rightward block flow.

Latin- and Slavic-based systems are typically written using a left-to-right inline direction with a top-to-bottom block flow direction. Latin-based languages include English, Spanish, Romanian, and Portuguese. Slavic-based languages include Ukrainian, Polish, and Czech.

```
<p lang="en-US" dir="auto">This is written in English</p>
<p lang="lt-LT" dir="auto">Tai parašyta lietuviu kalba</p>
<p lang="el-GR" dir="auto">Αυτό είναι γραμμένο στα ελληνικά</p>
```

Arabic-based systems are typically written using a right-to-left inline direction with a top-to-bottom block flow direction. There are several horizontal right-to-left languages, including Arabic, Aramaic, Azeri, Divehi, Fula, Hebrew, Kurdish, N'ko, Persian, Rohingya, Syriac, and Urdu.

```
<p lang="ur-PK" dir="auto">یہ اردو میں لکھا ہے۔</p>
<p lang="ku-CRB" dir="auto">ئەمە بە کوردی نووسراوە</p>
```

Han-based systems are commonly written using a left-to-right inline direction with a top-to-bottom block flow direction, or a top-to-bottom inline direction with a right-to-left block flow direction. Traditionally, Chinese, Vietnamese, Korean, and Japanese are written vertically in columns, going from top to bottom, with a right-to-left block direction, but will often be rendered horizontally online, going from left to right.

```
<p lang="ja-JP" dir="auto">これは日本語で書かれています</p>
```

Mongolian-based systems are typically written vertically, top to bottom, in columns that flow left to right; a top-to-bottom inline direction with a left-to-right block flow direction. This differs from Chinese, Japanese, and Korean, whose vertical text columns are read right to left. It derives from the fact that Mongolian script descended from Old Uyghur, which was written left-to-right.

```
<p lang="mn-Mong" dir="auto">ᠮᠣᠩᠭᠣᠯ ᠬᠡᠯᠡᠨ ᠤ ᠪᠢᠴᠢᠭ</p>
```

To render the writing modes correctly, we use the global HTML `dir` attribute. Because browsers can turn off CSS styling, it is recommended to use the `dir` attribute and `<bdo>` element to ensure correct bidirectional layout in the absence of a style sheet, rather than the CSS `direction` property.

For vertical languages, we use the `writing-mode` and `text-orientation` properties:

```
@import "https://fonts.googleapis.com/css2?family=Noto+Sans+Mongolian&display=swap";
```

```
:lang(ja) {
  writing-mode: vertical-rl;
  text-orientation: mixed;
}
:lang(mn-Mong) {
  writing-mode: vertical-lr;
  text-orientation: mixed;
}
```

```
:lang(ja),
:lang(mn-Mong) {
  float: left;
}

:lang(mn-Mong) {
```

```
  font-family: "Noto Sans Mongolian", sans-serif;
}
```

Mixing writing modes

While these different languages have different writing modes, sites that predominantly use one type of writing mode may contain content from a different language or writing mode. For example, articles in an Arabic, right-to-left language news site may include Latin-style numbers, which are written left to right. Many magazines and newspapers mix different writing modes on the same page. This guide, which demonstrates different writing modes, does too.

The typographic mode determines whether to use typographic conventions specific to vertical flow for vertical scripts (vertical typographic mode) or to use the typographic conventions of horizontal writing modes (horizontal typographic mode). This concept distinguishes vertical typesetting from rotated horizontal typesetting.

The `text-orientation` component of the writing mode controls the glyph orientation in vertical typographic modes, dictating whether a particular typographic character unit is typeset upright or typeset sideways.

See also

- CSS writing modes module

How to

This page lists how-to guides for CSS.

CSS layout cookbook

> The CSS layout cookbook aims to bring together recipes for common layout patterns, things you might need to implement in your own sites. In addition to providing code you can use as a starting point in your projects, these recipes highlight the different ways layout specifications can be used, and the choices you can make as a developer.

CSS layout cookbook

The CSS layout cookbook aims to bring together recipes for common layout patterns, things you might need to implement in your own sites. In addition to providing code you can use as a starting point in your projects, these recipes highlight the different ways layout specifications can be used, and the choices you can make as a developer.

> **Note:** If you are new to CSS layout then you might first like to take a look at our CSS layout learning module, as this will give you the basic grounding you need to make use of the recipes here.

The Recipes

Recipe	Description	Layout Methods
Media objects	A two-column box with an image on one side and descriptive text on the other, e.g., a social media post.	CSS grid, `float` fallback, `fit-content` sizing
Columns	When to choose multi-column layout, flexbox or grid for your columns.	CSS grid, Multicol, Flexbox
Center an element	How to center an item horizontally and vertically.	Flexbox, Box Alignment
Sticky footers	Creating a footer which sits at the bottom of the container or viewport when the content is shorter.	CSS grid, Flexbox
Split navigation	A navigation pattern where some links are visually separated from the others.	Flexbox, `margin`
Breadcrumb navigation	Creating a list of links to allow the visitor to navigate back up through the page hierarchy.	Flexbox
List group with badges	A list of items with a badge to display a count.	Flexbox, Box Alignment
Pagination	Links to pages of content (such as search results).	Flexbox, Box Alignment
Card	A card component, which displays in a grid of cards.	Grid Layout
Grid wrapper	For aligning grid content within a central wrapper, while also allowing items to break out.	CSS grid

Contribute a Recipe

As with all of MDN we would love you to contribute a recipe in the same format as the ones shown above. See the guide for adding Layout Cookbook recipes for a template and guidelines for writing your own example.

Breadcrumb navigation

Breadcrumb navigation helps the user to understand their location in the website by providing a breadcrumb trail back to the start page. The items typically display inline, with a separator between each item, indicating the hierarchy between individual pages.

Requirements

Display the hierarchy of the site by displaying inline links, with a separator between the items, indicating the hierarchy between individual pages, with the current page appearing last.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<nav aria-label="Breadcrumb" class="breadcrumb">
  <ol>
    <li><a href="#">Home</a></li>
    <li><a href="#">Category</a></li>
    <li><a href="#">Sub Category</a></li>
    <li><a href="#">Type</a></li>
    <li><span aria-current="page">Product</span></li>
  </ol>
</nav>
```

```
body {
  font: 1.2em sans-serif;
}
```

```
.breadcrumb {
  padding: 0 0.5rem;
}

.breadcrumb ol {
  display: flex;
  flex-wrap: wrap;
  list-style: none;
  margin: 0;
  padding: 0;
  align-items: end;
}

.breadcrumb li:not(:last-child)::after {
  display: inline-block;
  margin: 0 0.25rem;
  content: "→";
}
```

> **Note:** The example above uses a complex selector to insert content before every `li` except the last one. This could also be achieved using a complex selector targeting all `li` elements except the first:
>
> ```
> .breadcrumb li:not(:first-child)::before {
>   content: "→";
> }
> ```
>
> Feel free to choose the solution that you prefer.

Choices made

To display list items inline, we use flexbox layout, thus demonstrating how a line of CSS can give us our navigation. The separators are added using CSS generated content. You could change these to any separator that you like.

Accessibility concerns

We used the `aria-label` and `aria-current` attributes to help assistive technology users understand what this navigation is and where the current page is in the structure. See the related links for more information.

Be aware that the separator arrows → added via the `content` CSS property in the example above are exposed to assistive technologies (AT), including screen readers and braille displays. For a quieter solution, use a decorative `<img>` in your HTML with an empty `alt` attribute. An ARIA `role` set to `none` or `presentation` will also prevent the image from being exposed to AT.

Alternatively, silence the CSS generated content by including an empty string as alternative text, preceded by a slash (/); for example, `content: url("arrow.png") / "";`.

If including generated separators that will be exposed to AT, opt for creating the generated content using the `::after` pseudo-element selector instead of `::before`, so the separator content is announced after the HTML content instead of before it.

See also

- CSS flexible box layout
- Providing a breadcrumb trail
- Using the `aria-current` attribute

Card

This pattern is a list of "card" components with optional footers. A card contains a title, an image, a description or other content, and an attribution or footer. Cards are generally displayed within a group or collection.

Requirements

Create a group of cards, with each card component containing a heading, image, content, and, optionally, a footer.

Each card in the group of cards should be the same height. The optional card footer should stick to the bottom of the card.

The cards in the group should line up in two dimensions — both vertically and horizontally.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="cards">
  <article class="card">
    <header>
      <h2>A short heading</h2>
    </header>

    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons.jpg"
      alt="Hot air balloons" />
    <div class="content">
      <p>
        The idea of reaching the North Pole by means of balloons appears to have
        been entertained many years ago.
      </p>
    </div>
  </article>

  <article class="card">
    <header>
      <h2>A short heading</h2>
    </header>

    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons2.jpg"
      alt="Hot air balloons" />
    <div class="content">
      <p>Short content.</p>
    </div>
    <footer>I have a footer!</footer>
  </article>

  <article class="card">
    <header>
      <h2>A longer heading in this card</h2>
    </header>

    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons.jpg"
      alt="Hot air balloons" />
    <div class="content">
      <p>
        In a curious work, published in Paris in 1863 by Delaville Dedreux,
        there is a suggestion for reaching the North Pole by an aerostat.
      </p>
    </div>
    <footer>I have a footer!</footer>
  </article>
  <article class="card">
    <header>
      <h2>A short heading</h2>
    </header>

    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons2.jpg"
      alt="Hot air balloons" />
    <div class="content">
      <p>
        The idea of reaching the North Pole by means of balloons appears to have
        been entertained many years ago.
      </p>
    </div>
  </article>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

img {
  max-width: 100%;
}

.cards {
  max-width: 700px;
  margin: 1em auto;

  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(230px, 1fr));
  grid-gap: 20px;
}

.card {
  border: 1px solid #999999;
  border-radius: 3px;

  display: grid;
  grid-template-rows: max-content 200px 1fr;
}

.card img {
  object-fit: cover;
  width: 100%;
  height: 100%;
}

.card h2 {
  margin: 0;
  padding: 0.5rem;
```

```
}

.card .content {
  padding: 0.5rem;
}

.card footer {
  background-color: #333333;
  color: white;
  padding: 0.5rem;
}
```

Choices made

Each card is laid out using CSS grid layout despite the layout being one-dimensional. This enables the use of content sizing for the grid tracks. To set up a single-column grid we can use the following:

```
.card {
  display: grid;
  grid-template-rows: max-content 200px 1fr;
}
```

`display: grid` converts the element into a grid container. The three values of the `grid-template-rows` property divide the grid into a minimum of three rows, defining the height of the first three children of the card, in order.

Each `card` contains a `<header>`, `<img>`, and `<div>`, in that order, with some also containing a `<footer>`.

The heading row, or track, is set to `max-content`, which prevents it from stretching. The image track is set to 200 pixels tall. The third track, where the content lives, is set to `1fr`. This means it will fill any additional space.

Any children beyond the three with explicitly defined sizes create rows in the implicit grid, which fits the content added to it. These are auto-sized by default. If a card contains a footer, it is auto-sized. The footer, when present, sticks to the bottom of the grid. The footer is auto-sized to fit its content; the content `<div>` then stretches take up any additional space.

The following ruleset creates the grid of cards:

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(230px, 1fr));
  gap: 20px;
}
```

The `grid-template-columns` property defines the widths of the grid columns. In this case, we set the grid to auto-fill, with repeated columns that are minimally `230px` but allowed to grow to fill the available space. The `gap` property sets a gap of `20px` between adjacent rows and adjacent columns.

> **Note:** The various elements in separate cards do not align with each other, as each card is an independent grid. Lining up the components in each card with the same components in adjacent cards can be done with subgrid.

Alternative methods

Flexbox can also be used to lay out each card. With flexbox, the dimensions of each card's rows are set with the `flex` property on each row, rather than on the card container.

With flexbox, the dimensions of the flex items are defined on the children rather than the parent. Whether you choose to use grid or flexbox depends on your preference, whether you prefer controlling the tracks from the container or prefer adding rules to the items.

We chose grid for the cards as, generally, you want cards to be lined up both vertically and horizontally. Additionally, lining up the components within each card to the components of adjacent cards can be done with subgrid. Flex has no hack-free equivalent to subgrid.

Accessibility concerns

Depending on the content of your card, there may be things you could or should do to enhance accessibility. See Inclusive components: Card by Heydon Pickering, for a very detailed explanation of these issues.

See also

- `grid-template-columns`
- `grid-template-rows`
- `gap`
- Inclusive components: Card
- CSS grid layout module

Center an element

In this recipe, you will see how to center one box inside another by using flexbox and grid, centering content both horizontally and vertically.

Requirements

To place an item into the center of another box horizontally and vertically.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="container">
  <div class="item">I am centered!</div>
</div>
```

```
.item {
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;
  padding: 20px;
  width: 10em;
}

.container {
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  font: 1.2em sans-serif;

  height: 200px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

Using flexbox

To center a box within another box, first turn the containing box into a flex container by setting its `display` property to `flex`. Then set `align-items` to `center` for vertical centering (on the block axis) and `justify-content` to `center` for horizontal centering (on the inline axis). And that's all it takes to center one box inside another!

HTML

```
<div class="container">
  <div class="item">I am centered!</div>
</div>
```

CSS

```
div {
  border: solid 3px;
  padding: 1em;
  max-width: 75%;
}

.item {
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;
  padding: 20px;
  width: 10em;
}

.container {
  height: 8em;
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  font: 1.2em sans-serif;

  display: flex;
  align-items: center;
  justify-content: center;
}
```

We set a height for the container to demonstrate that the inner item is indeed vertically centered within the container.

Result

Instead of applying `align-items: center;` on the container, you can also vertically center the inner item by setting `align-self` to `center` on the inner item itself.

Using grid

Another method you can use for centering one box inside another is to first make the containing box a grid container and then set its `place-items` property to `center` to center align its items on both the block and inline axes.

HTML

```
<div class="container">
  <div class="item">I am centered!</div>
</div>
```

CSS

```
div {
  border: solid 3px;
  padding: 1em;
  max-width: 75%;
}

.item {
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;
  padding: 20px;
  width: 10em;
}

.container {
  height: 8em;
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  font: 1.2em sans-serif;

  display: grid;
  place-items: center;
}
```

Result

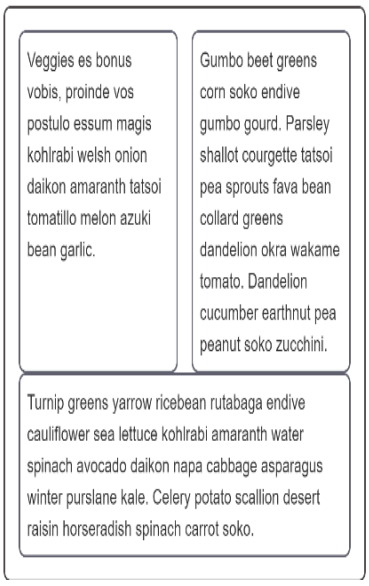
Instead of applying `place-items: center;` on the container, you can achieve the same centering by setting `place-content: center;` on the container or by applying either `place-self: center` or `margin: auto;` on the inner item itself.

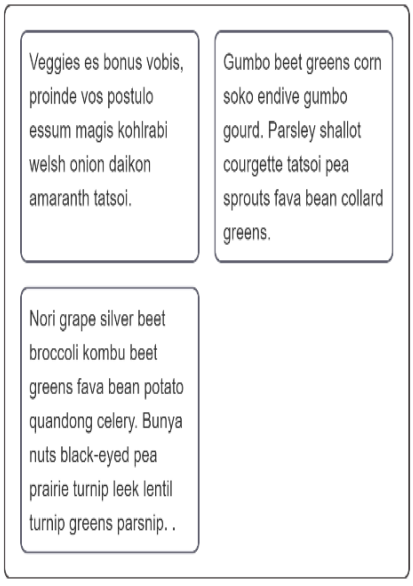
Resources on MDN

- Box alignment in flexbox
- CSS box alignment guide

Column layouts

You will often need to create a layout which has a number of columns, and CSS provides several ways to do this. Whether you use Multi-column, Flexbox, or Grid layout will depend on what you are trying to achieve, and in this recipe we explore these options.





Requirements

There are a number of design patterns you might want to achieve with your columns:

- A continuous thread of content broken up into newspaper-style columns.
- A single row of items arranged as columns, with all heights being equal.
- Multiple rows of columns lined up by row and column.

The recipes

You need to choose different layout methods in order to achieve your requirements.

A continuous thread of content — multi-column layout

If you create columns using multi-column layout your text will remain as a continuous stream filling each column in turn. The columns must all be the same size, and you are unable to target an individual column or the content of an individual column.

You can control the gaps between columns with the `column-gap` or `gap` properties, and add a rule between columns using `column-rule`.

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>
  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
  </p>
</div>
```

```
.container {
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  padding: 20px;
  font: 1.2em sans-serif;

  column-width: 10em;
  column-rule: 1px solid rgb(75 70 74);
}
```

In this example, we used the `column-width` property to set a minimum width that the columns need to be before the browser adds an additional column. The `columns` shorthand property can be used to set the `column-width` and `column-count` properties, either of which can define the maximum number of columns allowed.

Use multicol when:

- You want your text to display in newspaper-like columns.
- You have a set of small items you want to break into columns.
- You do not need to target individual column boxes for styling.

A single row of items with equal heights — flexbox

Flexbox can be used to break content into columns by setting `display: flex;` to make a parent element a flex-container. Just adding this one property turns all the children (child elements, pseudo-elements, and text nodes) into flex items along a single line. Setting the same `flex` shorthand property with a single numeric value distributes all the available space equally, generally making all the flex items the same size as long as none have non-wrapping content forcing the item to be larger.

Margins or the `gap` property can be used to create gaps between items, but there is currently no CSS property that adds rules between flex items.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>

  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>

  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
.container {
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  padding: 20px 10px;
  font: 1.2em sans-serif;

  display: flex;
}

.container > * {
  padding: 10px;
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;

  margin: 0 10px;
  flex: 1;
}
```

To create a layout with flex items that wrap onto new rows, set the `flex-wrap` property on the container to `wrap`. Note that each flex line distributes space for that line only. Items in one line will not necessarily line up with items on other lines, as you'll see in the example below. This is why flexbox is described as one-dimensional. It is designed for controlling layout as a row or a column, but not both at the same time.

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi tomatillo melon azuki bean garlic.
  </p>
```

```
  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
    Dandelion cucumber earthnut pea peanut soko zucchini.
  </p>

  <p>
    Turnip greens yarrow ricebean rutabaga endive cauliflower sea lettuce
    kohlrabi amaranth water spinach avocado daikon napa cabbage asparagus winter
    purslane kale. Celery potato scallion desert raisin horseradish spinach
    carrot soko.
  </p>
</div>
```

```
.container {
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  padding: 20px 10px;
  width: 500px;
  font: 1.2em sans-serif;

  display: flex;
  flex-wrap: wrap;
}

.container > * {
  padding: 10px;
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;

  margin: 0 10px;
  flex: 1 1 200px;
}
```

Use flexbox:

- For single rows or columns of items.
- When you want to do alignment on the cross axis after laying out your items.
- When you are happy for wrapped items to share out space along their line only and not line up with items in other lines.

Lining items up in rows and columns — grid layout

If you want a two-dimensional grid where items line up in rows *and* columns, then you should choose CSS grid layout. Similar to how flexbox works on the direct children of the flex container, grid layout works on the direct children of the grid container. Just set `display: grid;` on the container. Properties set on this container — like `grid-template-columns` and `grid-template-rows` — define how the items are distributed along rows and columns.

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="container">
  <p>
    Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
    daikon amaranth tatsoi.
  </p>

  <p>
    Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
    tatsoi pea sprouts fava bean collard greens.
  </p>

  <p>
    Nori grape silver beet broccoli kombu beet greens fava bean potato quandong
    celery. Bunya nuts black-eyed pea prairie turnip leek lentil turnip greens
    parsnip. .
  </p>
</div>
```

```
.container {
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;
  padding: 20px;
  width: 500px;
  font: 1.2em sans-serif;

  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-gap: 20px;
}

.container > * {
  padding: 10px;
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;
  margin: 0;
}
```

Use grid:

- For multiple rows or columns of items.
- When you want to be able to align the items on the block and inline axes.
- When you want items to line up in rows and columns.

Resources on MDN

- Guide to Multi-column Layout
- Guide to flexbox
- Guide to CSS grid layout

Contribute a recipe

If you would like to contribute an example for the Layout Cookbook, then this page explains the steps to take to have your example published.

What makes a good recipe?

The simplest possible version of any useful web design pattern. Each line of CSS that you include should be there to help demonstrate the pattern, so leave out anything there purely to make the example look pretty. The idea is that someone can take the pattern and use it in a site with their own styles.

A recipe consists of:

- A live example, stored in the CSS Examples GitHub repo.
- A downloadable version of that example, also stored in the CSS Examples repo.
- A page in the CSS layout cookbook section of the site, which should include the following components:
    1. Introduction
    2. Requirements
    3. Recipe
    4. Decisions
    5. Useful fallbacks or alternative methods (when relevant)
    6. Accessibility concerns
    7. Browser compatibility (only for CSS properties not fully supported)
    8. Additional resources

Steps to publish a recipe

To create a recipe and add it to the CSS layout cookbook, follow these steps:

- What makes a good recipe?
- Steps to publish a recipe
    - 1. Build a pattern
    - 2. Create a live example
    - 3. Create a downloadable version
    - 4. Open a pull request with your example
    - 5. Create your page

1. Build a pattern

Before turning your example into a recipe, start by creating your pattern as a basic HTML page. Think about what you are trying to demonstrate and keep it as simple as possible. Use descriptive words like "container" or "item" if adding class names.

Ensure that you've tested the HTML and CSS across multiple browsers, viewport sizes, and devices. Also ensure that any CSS styling follows accessibility guidelines. While it's okay to use CSS properties with limited support, be sure to include browser support information when creating the page, particularly in the browser compatibility section. In some cases, creating a second version of your example with fallback support can be helpful.

2. Create a live example

The live examples on cookbook pages, such as for centering an element, and elsewhere on MDN allow readers to play with the code and change relevant parts without being overwhelmed with all of the code. Your recipe will be demonstrated by way of one or more examples.

Fork the CSS Examples repo and then take a look in the `css-cookbook` folder. There is a cookbook-template.html file. Copy this file and use it as a starting point. Save this in the `css-cookbook` directory with a name that makes sense for your pattern. The template contains comments to guide you in adding various parts at the appropriate places.

The important parts of this template are as follows:

- There is a stylesheet for the editor panels with some basic body styling and a JavaScript file for the editor functionality. Leave these files as they are.
- The `<head>` contains two style blocks. The first style block is for elements that readers do not need to change to play with the example. The second style block contains items that readers might want to play with. We typically add all our CSS to the first block, then choose the rules to move to the second block. The rules in the second block should be those that are fundamental to the pattern, perhaps those where a reader can change a CSS value and see the pattern update.
- You need to add the HTML for your component twice: first inside the section with the class `preview`, and then inside the `<textarea>` with the class `playable-html`.
- The editable CSS from the second `<head>` block must also be copied into the `playable-css` section.

A working example is center.html, which is embedded on the page Center an element. You can use this to see how to add the various bits correctly.

If you've forked the repo and have our CSS and JavaScript files with your example, opening your live example in a browser should display everything working exactly as it would appear on an MDN page.

Useful tips

- Do not indent the CSS and HTML inside the text areas; instead, align the code with the start of the lines. This will look better when it is rendered.
- If any images are required, put them in the directory with the examples. You are welcome to use any images already present.
- You can adjust the height of the text areas by changing the height in the inline styles.

3. Create a downloadable version

As the examples include all of our live example code, we want to provide readers with just the code they need for their example, omitting any unnecessary items. Create a copy of your live example, and add `--download` to its name. For example, the download version of `center.html` is `center--download.html`. This file will be similar to your initial example. It should be a basic version of your pattern as a single HTML page.

It can be handy to include the body CSS rules from our included stylesheet. Take a look at `cookbook-template--download.html` as a guide; these rules are provided as a starting point.

4. Open a pull request with your example

Open a pull request (PR) on the CSS Examples repo. This allows us to help you with any changes to the example that may be needed before you create your page. Plus, the example will need to be live to be included. This is why it makes sense to first create the code examples and then the explanatory guide. In your PR, explain the pattern and what you are demonstrating. See our guidelines for opening a pull request.

5. Create your page

After your example PR is merged, open a pull request to create a new page in the Layout Cookbook directory. Use our template for Cookbook pages. The template explains the requirements of each section, and you can refer to other cookbook examples for more help.

Remember to add a link to your new recipe on the main Layout cookbook page.

If you have any questions or want someone to take a look at your page, get in touch via any of our communication channels.

See also

- Cookbook page template

- CSS Examples repo

Cookbook template

> **Note:** This is a template page for a CSS Cookbook page. Please use this as a raw template when you create a new cookbook page. *Comment in italics are information about how to use part of the template.*

Description of the problem this recipe solves or the pattern you are demonstrating.

Requirements

What does this pattern need to include, or what problems does it need to solve? List that here.

Recipe

Change the example code. The last parameter is the live example height, which you can change as needed. Mention that you can click "Play" in code blocks to edit the example in the MDN Playground.

```
<div class="container">
  <div class="item">I am centered!</div>
</div>
```

```
.container {
  border: 2px solid rgb(75 70 74);
  border-radius: 0.5em;

  height: 200px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.item {
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;
  padding: 20px;

  width: 10em;
}
```

Choices made

Explain your decisions when creating the pattern. Why did you choose a certain method? If you want to add an additional example here — for example a version with fallbacks, please do. This section is deliberately loose as patterns range from the very simple to more complex.

Useful fallbacks or alternative methods

If there are useful alternative methods for building the recipe, or fallback recipes to use if you have to support non-supporting browsers, include them in separate sections down here.

Accessibility concerns

Include this is there are any specific things to watch out for in regard to accessibility. If not relevant for your pattern this can be omitted.

See also

- *Links of link of related properties:* `example-property`
- *Links to article showing how to use the property in context: "Using ... article"*
- *Very good external links. Don't be afraid of external links, but they should be outstanding, and not only mention minor details.*

Grid wrapper

The grid wrapper pattern is useful for aligning grid content within a central wrapper while also allowing items to break out and align to the edge of the containing element or page.

Requirements

Items placed on the grid should be able to align to a horizontally-centered max-width wrapper or the outer edges of the grid, or both.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="grid">
  <div class="wrapper">
    <p>
      This item aligns to a central "wrapper" – columns that have a maximum
      width.
    </p>
  </div>
  <div class="full-width">
    <p>This item aligns to the edge of the grid container.</p>
  </div>
  <div class="left-edge">
    <p>
      This item aligns to the left edge of the grid container and the right edge
      of the wrapper.
    </p>
  </div>
  <div class="right-wrapper">
    <p>This item aligns to the right edge of the "wrapper" columns.</p>
  </div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}
.grid {
  display: grid;
  grid-template-columns: minmax(20px, 1fr) repeat(6, minmax(0, 60px)) minmax(
      20px,
      1fr
```

```
    );
  grid-auto-rows: minmax(100px, auto);
  grid-gap: 10px;
}

.grid > * {
  border: 2px solid rgb(95 97 110);
  border-radius: 0.5em;
  padding: 20px;
}

.full-width {
  grid-column: 1 / -1;
}

.wrapper {
  grid-column: 2 / -2;
}

.left-edge {
  grid-column: 1 / -2;
}

.right-wrapper {
  grid-column: 4 / -2;
}
```

Choices made

This recipe uses the CSS grid `minmax()` function to define the grid track sizes in the `grid-template-columns` property. For the central columns with a maximum width, we can set a minimum value of `0` or greater and a maximum value that specifies the maximum size the column tracks will grow to. Using relative or absolute `<length>` units (pixels, ems, rems) will create a fixed maximum size for the central wrapper, whereas using `<percentage>` values or viewport units will cause the wrapper to grow or shrink in response to its context.

The outer two columns have a maximum size of `1fr`, meaning that they will each expand to fill the remaining available space in the grid container.

Useful fallbacks or alternative methods

To center the grid horizontally on the page you can set a `max-width` along with left and right `auto` `margin`s:

```
.grid {
  max-width: 96vw; /* Limits the width to 96% of the width of the viewport */
  margin: 0 auto; /* horizontally centers the container */
}
```

Accessibility concerns

Although CSS grid enables positioning items anywhere (within reason), ensuring that your underlying markup follows a logical order is important (see CSS grid layout and accessibility for more details).

See also

- `grid-template-columns` property
- CSS grid layout module
- CSS grid: More flexibility with `minmax()` (2018)
- Breaking out with CSS grid (2017)

List group with badges

In this recipe we will create a list group pattern with badges that indicate a count.

Requirements

The list items should be displayed with the badges. The badge should be aligned right and vertically centered. The badge must be centered vertically whether there is a single line of content or multiple lines of text.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<ul class="list-group">
  <li>
    Item One
    <span class="badge">2</span>
  </li>
  <li>
    Item Two
    <span class="badge">10</span>
  </li>
  <li>
    Item Three
    <span class="badge">9</span>
  </li>
  <li>
    Item Four
    <span class="badge">0</span>
  </li>
</ul>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
  padding: 0;
  margin: 1em;
}
.list-group {
  list-style: none;
  margin: 0;
  padding: 0;
  border: 1px solid #cccccc;
  border-radius: 0.5em;
  width: 20em;
}

.list-group li {
  border-top: 1px solid #cccccc;
  padding: 0.5em;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.list-group li:first-child {
  border-top: 0;
}

.list-group .badge {
  background-color: rebeccapurple;
  color: white;
  font-weight: bold;
  font-size: 80%;
  border-radius: 10em;
  min-width: 1.5em;
  padding: 0.25em;
  text-align: center;
}
```

Choices made

Flexbox makes this particular pattern straightforward and also facilitates making changes to the layout.

To ensure the text and badge line up correctly, I use the `justify-content` property with a value of `space-between`. This places any extra space between the items. In the live example, if you remove this property, you will see the badge move to the end of the text on items with text shorter than the one line.

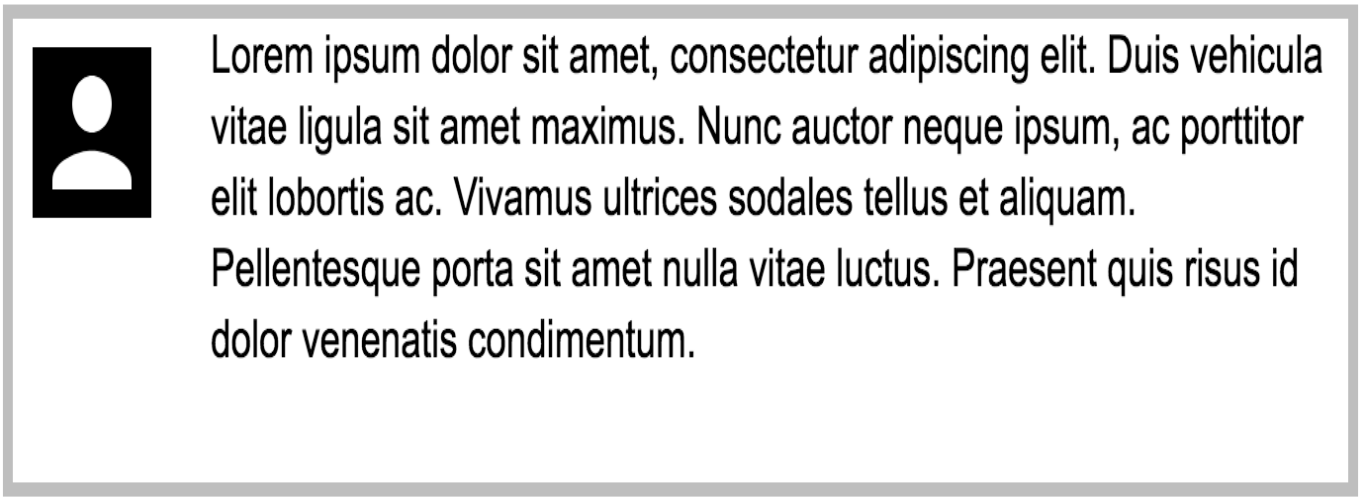
To align the content horizontally, I use the `align-items` property to align the text and badge on the cross axis. If you want the badge to align to the top of the content instead, change this to `align-items: flex-start`.

See also

- Box alignment in flexbox
- Aligning items in a flex container
- CSS flexible box layout module

Media objects

The *Media Object* is a pattern we see all over the web. It refers to a two-column box with an image on one side and descriptive text on the other, e.g., a social media post.



Requirements

Media Object pattern needs some or all of the following characteristics:

- Stacked on Mobile, two columns on Desktop.
- The image can be on the left or right.
- The image might be small or large.
- Media Objects can be nested.
- The Media Object should clear the contents no matter which side is tallest.

The recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="media">
  <div class="img">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons_square.jpg"
      alt="Balloons" />
  </div>

  <div class="content">
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vehicula
      vitae ligula sit amet maximus. Nunc auctor neque ipsum, ac porttitor elit
      lobortis ac. Vivamus ultrices sodales tellus et aliquam. Pellentesque
      porta sit amet nulla vitae luctus. Praesent quis risus id dolor venenatis
      condimentum.
    </p>
  </div>
  <div class="footer">An optional footer goes here.</div>
</div>

<div class="media">
  <div class="img">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/sharp-account_box-24px.svg"
      width="80px"
      alt="Account" />
  </div>
  <div class="content">
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vehicula
      vitae ligula sit amet maximus. Nunc auctor neque ipsum, ac porttitor elit
      lobortis ac. Vivamus ultrices sodales tellus et aliquam. Pellentesque
      porta sit amet nulla vitae luctus. Praesent quis risus id dolor venenatis
      condimentum.
    </p>
  </div>
  <div class="footer"></div>
</div>
```

```
<div class="media media-flip">
  <div class="img">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons_square.jpg"
      alt="Balloons" />
  </div>

  <div class="content">
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vehicula
      vitae ligula sit amet maximus. Nunc auctor neque ipsum, ac porttitor elit
      lobortis ac. Vivamus ultrices sodales tellus et aliquam. Pellentesque
      porta sit amet nulla vitae luctus. Praesent quis risus id dolor venenatis
      condimentum.
    </p>
  </div>
  <div class="footer">An optional footer goes here.</div>
</div>

<div class="media">
  <a class="img">
    <img
      src="https://mdn.github.io/shared-assets/images/examples/balloons_square.jpg"
```

```
      alt="Balloons" />
  </a>

  <div class="content">
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vehicula
      vitae ligula sit amet maximus. Nunc auctor neque ipsum, ac porttitor elit
      lobortis ac. Vivamus ultrices sodales tellus et aliquam. Pellentesque
      porta sit amet nulla vitae luctus. Praesent quis risus id dolor venenatis
      condimentum.
    </p>
  </div>

  <div class="footer"></div>

  <div class="media">
    <a class="img">
      <img
        src="https://mdn.github.io/shared-assets/images/examples/balloons_square.jpg"
        alt="Balloons" />
    </a>

    <div class="content">
      <p>
        Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vehicula
        vitae ligula sit amet maximus.
      </p>
    </div>

    <div class="footer"></div>
  </div>
</div>
```

```
body {
  font: 1.2em sans-serif;
}

img {
  max-width: 100%;
}

p {
  margin: 0 0 1em 0;
}

@media (width >= 500px) {
  .media {
    display: grid;
    grid-template-columns: fit-content(200px) 1fr;
    grid-template-rows: 1fr auto;
    grid-template-areas:
      "image content"
      "image footer";
    grid-gap: 20px;
    margin-bottom: 4em;
  }

  .media-flip {
    grid-template-columns: 1fr fit-content(250px);
    grid-template-areas:
      "content image"
      "footer image";
  }

  .img {
    grid-area: image;
  }

  .content {
    grid-area: content;
  }

  .footer {
    grid-area: footer;
  }
}
```

Choices made

I have chosen to use grid layout for the media object as it allows me to control the layout in two dimensions when I need to. This means that when we have a footer, with short content above, the footer can be pushed down to the bottom of the media object.

Another reason to use grid layout is in order that I can use `fit-content` for the track sizing of the image. By using `fit-content` with a maximum size of 200 pixels, when we have a small image such as the icon, the track only gets as large as the size of that image — the `max-content` size. If the image is larger, the track stops growing at 200 pixels and as the image has a `max-width` of 100% applied, it scales down so that it continues to fit inside the column.

By using `grid-template-areas` to achieve the layout, I can see the pattern in the CSS. I define my grid once the viewport is 500 pixels wide, so on smaller devices the media object stacks.

An option for the pattern is to flip it to switch the image to the other side — this is done by adding the `media-flip` class, which defines a flipped grid template causing the layout to be mirrored.

When we nest one media object inside another we need to place it into the second track in the regular layout, and the first track when flipped.

See also

- `fit-content` property
- Using grid template areas
- CSS grid layout module

Pagination

This cookbook pattern demonstrates the navigation pattern used to display pagination, where the user can move between pages of content such as search results.

Requirements

The pagination pattern typically displays items in a row. To ensure that the pagination is understandable by people using a screen reader, we mark the items up as a list inside a `<nav>` element, and then use CSS to display the layout visually as a row.

Typically, the pagination component will be centered horizontally underneath the content.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<nav aria-label="pagination">
  <ul class="pagination">
    <li>
      <a href="">
        <span aria-hidden="true">&amp;laquo;</span>
        <span class="visuallyhidden">previous set of pages</span>
      </a>
    </li>
    <li>
      <a href=""><span class="visuallyhidden">page </span>1</a>
    </li>
    <li>
      <a href="" aria-current="page">
        <span class="visuallyhidden">page </span>2
      </a>
    </li>
    <li>
      <a href=""> <span class="visuallyhidden">page </span>3 </a>
    </li>
    <li>
      <a href=""> <span class="visuallyhidden">page </span>4 </a>
    </li>
    <li>
      <a href="">
        <span class="visuallyhidden">next set of pages</span
        ><span aria-hidden="true">&amp;raquo;</span>
      </a>
    </li>
  </ul>
</nav>
```

```
.visuallyhidden {
  border: 0;
  clip: rect(0 0 0 0);
  height: auto;
  margin: 0;
  overflow: hidden;
  padding: 0;
  position: absolute;
  width: 1px;
  white-space: nowrap;
}

nav {
  border-top: 1px solid #eeeeee;
  margin-top: 1em;
  padding-top: 0.5em;
  font: 1.2em sans-serif;

  display: flex;
  justify-content: center;
}

.pagination {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
}

.pagination li {
  margin: 0 1px;
}

.pagination a {
  display: block;
  padding: 0.5em 1em;
  border: 1px solid #999999;
  border-radius: 0.2em;
  text-decoration: none;
}

.pagination a[aria-current="page"] {
  background-color: #333333;
  color: white;
}
```

Choices made

This pattern is laid out using flexbox — one flex container nested inside another. The `<nav>` element is designated a flex container in order that we can center the list inside using the `justify-content` property.

The list itself also becomes a flex container to lay the items out as a row. To space the items out we can either use a `margin` on the flex items or add a `gap` on the flex container.

```
.pagination {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  gap: 2px;
}
```

Accessibility concerns

We want to ensure that a person using a screen reader understands what this navigation does, and where they will go when clicking a link. To help with this we have added `aria-label="pagination"` on the `<nav>` element.

We have also added some additional content that would be read by a screen reader but is hidden visually, and set the `aria-hidden` attribute on the paging arrows.

The "See Also" section at the end of this document has links to related accessibility topics.

See also

- `justify-content`, `gap`
- Know your ARIA: 'hidden' vs 'none' (2018)
- Invisible content just for screen reader users via WebAIM.org (2020)
- Writing CSS with accessibility in mind (2017)
- a11y style guide: pagination

Split navigation

The **split navigation** is a navigation pattern where one or more elements are separated from the rest of the navigation items.

Requirements

A common navigation pattern is to have one element pushed away from the others. We can use flexbox to achieve this, without needing to make the two sets of items into two separate flex containers.

Recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<nav>
  <ul class="main-nav">
    <li><a href="">About</a></li>
    <li><a href="">Products</a></li>
    <li><a href="">Our Team</a></li>
    <li class="push"><a href="">Contact</a></li>
  </ul>
</nav>
```

```
.main-nav {
  margin: 0;
  padding: 0;
  list-style: none;
  font: 1.2em sans-serif;

  display: flex;
}

.main-nav a {
  padding: 0.5em 1em;
  display: block;
}

.push {
  margin-left: auto;
}
```

Choices made

This pattern combines auto margins with flexbox to split the items.

An auto margin absorbs all available space in the direction it is applied. This is how centering a block with auto margins works — you have a margin on each side of the block trying to take up all of the space, thus pushing the block into the middle.

In this case the left auto margin takes up any available space and pushes the item over to the right. You could apply the class `push` to any item in the list.

See also

- CSS flexible box layout module
- `display` property
- `margin` property

Sticky footers

A sticky footer pattern is one where the footer of your page "sticks" to the bottom of the viewport in cases where the content is shorter than the viewport height. We'll look at a couple of techniques for creating one in this recipe.

Requirements

The sticky footer pattern needs to meet the following requirements:

- Footer sticks to the bottom of the viewport when content is short.
- If the content of the page extends past the viewport bottom, the footer gets pushed down, always sitting below the content as normal.

The recipe

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<div class="wrapper">
  <header class="page-header">This is the header</header>
  <main class="page-body">
    <p contenteditable>
      The footer sticks to the bottom even though this paragraph is short. Add
      content to this editable area to see the footer push down when needed to
      fit the content.
    </p>
  </main>
  <footer class="page-footer">Sticky footer</footer>
</div>
```

```
* {
  box-sizing: inherit;
}

html {
  height: 100%;
  box-sizing: border-box;
}

body {
  height: 100%;
  font: 1.2em sans-serif;
}

.wrapper {
  min-height: 100%;
  display: grid;
  grid-template-rows: auto 1fr auto;
}

.page-header,
.page-footer {
  background-color: rgb(75 70 74);
  color: white;
  padding: 20px;
}

.page-body {
  padding: 20px;
}

.page-body p {
  border: 1px solid grey;
}
```

> **Note:** In this example and the following one we are using a wrapper set to `min-height: 100%`. You can also achieve this for a full page by setting a `min-height` of `100vh` on the `<body>` and then using it as your grid container.

Choices made

In the above example we achieve the sticky footer using CSS grid layout. The `.wrapper` has a minimum height of `100%` which means it is as tall as the container it is in. We then create a single column grid layout with three rows, one row for each part of our layout.

Grid auto-placement will place our items in source order and so the header goes into the first auto sized track, the main content into the `1fr` track and the footer into the final auto sized track. The `1fr` track will take up all available space and so grows to fill the gap.

Alternate method

You can also use flexbox to create a sticky footer.

```
<div class="wrapper">
  <header class="page-header">This is the header</header>
  <main class="page-body">
    <p contenteditable>
      The footer sticks to the bottom even though this paragraph is short. Add
      content to this editable area to see the footer push down when needed to
      fit the content.
    </p>
  </main>
  <footer class="page-footer">Sticky footer</footer>
</div>
```

```
* {
  box-sizing: border-box;
}

html,
body {
  box-sizing: border-box;
  height: 100%;
  padding: 0;
```

```
  margin: 0;
  font: 1.2em sans-serif;
}

.wrapper {
  box-sizing: border-box;
  min-height: 100%;

  display: flex;
  flex-direction: column;
}

.page-header,
.page-footer {
  background-color: rgb(75 70 74);
  color: white;
  padding: 20px;

  flex-grow: 0;
  flex-shrink: 0;
}

.page-body {
  padding: 20px;

  flex-grow: 1;
}

.page-body p {
  border: 1px solid grey;
}
```

The flexbox example starts out in the same way, but we use `display:flex` rather than `display:grid` on the `.wrapper`; we also set `flex-direction` to `column`. Then we set our main content to `flex-grow: 1` and the other two elements to `flex-shrink: 0` — this prevents them from shrinking smaller when content fills the main area.

Resources on MDN

- CSS properties: `display`, `min-height`, `grid-template-rows`, `flex-direction`, `flex-grow`, `flex-shrink`
- Basic concepts of grid layout
- CSS grid layout module
- Basic concepts of flexbox
- CSS flexible box layout module

CSS reference

Use this **CSS reference** to browse an alphabetical index of all of the standard CSS properties, pseudo-classes, pseudo-elements, data types, functional notations and at-rules. You can also browse key CSS concepts and a list of selectors organized by type. Also included is a brief DOM-CSS / CSSOM reference.

Basic rule syntax

Style rule syntax

```
style-rule ::=
    selectors-list {
      properties-list
    }
```

Where:

```
selectors-list ::=
    selector[:pseudo-class] [::pseudo-element]
    [, selectors-list]

properties-list ::=
    [property : value] [; properties-list]
```

See the index of *selectors*, *pseudo-classes*, and *pseudo-elements* below. The syntax for each specified *value* depends on the data type defined for each specified *property*.

Style rule examples

```
strong {
  color: red;
}

div.menu-bar li:hover > ul {
  display: block;
}
```

For a beginner-level introduction to the syntax of selectors, see our guide on CSS Selectors. Be aware that any syntax error in a rule definition invalidates the entire rule. Invalid rules are ignored by the browser. Note that CSS rule definitions are entirely (Unicode) text-based, whereas DOM-CSS / CSSOM (the rule management system) is object-based.

At-rule syntax

As the structure of at-rules varies widely, please see At-rule to find the syntax of the specific one you want.

Index

> **Note:** This index does not include SVG-exclusive presentation attributes, which can be used as CSS properties on SVG elements.

> **Note:** The property names in this index do **not** include the JavaScript names which do differ from the CSS standard names.

- -
- A
- B
- C
- D

- E
- F
- G
- H
- I
- J
- K
- L
- M
- N
- O
- P
- Q
- R
- S
- T
- U
- V
- W
- X
- Y
- Z

-

- `:-moz-broken` CSS pseudo-class
- `::-moz-color-swatch` CSS pseudo-element
- `:-moz-drag-over` CSS pseudo-class
- `:-moz-first-node` CSS pseudo-class
- `::-moz-focus-inner` CSS pseudo-element
- `:-moz-handler-blocked` CSS pseudo-class
- `:-moz-handler-crashed` CSS pseudo-class
- `:-moz-handler-disabled` CSS pseudo-class
- `:-moz-last-node` CSS pseudo-class
- `::-moz-list-bullet` CSS pseudo-element
- `::-moz-list-number` CSS pseudo-element
- `:-moz-loading` CSS pseudo-class
- `:-moz-locale-dir(ltr)` CSS pseudo-class
- `:-moz-locale-dir(rtl)` CSS pseudo-class
- `::-moz-meter-bar` CSS pseudo-element
- `:-moz-only-whitespace` CSS pseudo-class
- `::-moz-progress-bar` CSS pseudo-element
- `::-moz-range-progress` CSS pseudo-element
- `::-moz-range-thumb` CSS pseudo-element
- `::-moz-range-track` CSS pseudo-element
- `:-moz-submit-invalid` CSS pseudo-class
- `:-moz-suppressed` CSS pseudo-class
- `:-moz-user-disabled` CSS pseudo-class
- `:-moz-window-inactive` CSS pseudo-class
- `::-webkit-inner-spin-button` CSS pseudo-element
- `::-webkit-meter-bar` CSS pseudo-element
- `::-webkit-meter-even-less-good-value` CSS pseudo-element
- `::-webkit-meter-inner-element` CSS pseudo-element
- `::-webkit-meter-optimum-value` CSS pseudo-element
- `::-webkit-meter-suboptimum-value` CSS pseudo-element
- `::-webkit-progress-bar` CSS pseudo-element
- `::-webkit-progress-inner-element` CSS pseudo-element
- `::-webkit-progress-value` CSS pseudo-element
- `::-webkit-scrollbar` CSS pseudo-element
- `::-webkit-search-cancel-button` CSS pseudo-element
- `::-webkit-search-results-button` CSS pseudo-element
- `::-webkit-slider-runnable-track` CSS pseudo-element
- `::-webkit-slider-thumb` CSS pseudo-element

A

- Attribute selectors
- `abs()` CSS function
- `<absolute-size>` CSS type
- `accent-color` CSS property
- `acos()` CSS function
- `:active` CSS pseudo-class
- `:active-view-transition` CSS pseudo-class
- `:active-view-transition-type()` CSS pseudo-class
- `additive-symbols` CSS at-rule descriptor (`@counter-style`)
- `::after` CSS pseudo-element
- `align-content` CSS property
- `align-items` CSS property
- `align-self` CSS property
- `alignment-baseline` CSS property
- `all` CSS property
- `alpha()` CSS function
- `<alpha-value>` CSS type
- `anchor()` CSS function
- `anchor-name` CSS property
- `anchor-scope` CSS property
- `anchor-size()` CSS function

- `<angle-percentage>` CSS type
- `<angle>` CSS type
- `animation` CSS property
- <animation-action>
- `animation-composition` CSS property
- `animation-delay` CSS property
- `animation-direction` CSS property
- `animation-duration` CSS property
- `animation-fill-mode` CSS property
- `animation-iteration-count` CSS property
- `animation-name` CSS property
- `animation-play-state` CSS property
- `animation-range` CSS property
- `animation-range-end` CSS property
- `animation-range-start` CSS property
- `animation-timeline` CSS property
- `animation-timing-function` CSS property
- animation-trigger CSS property
- `:any-link` CSS pseudo-class
- `appearance` CSS property
- `ascent-override` CSS at-rule descriptor (`@font-face`)
- `asin()` CSS function
- `aspect-ratio` CSS property
- `atan()` CSS function
- `atan2()` CSS function
- `attr()` CSS function
- `:autofill` CSS pseudo-class
- `<axis>` CSS type

B

- `::backdrop` CSS pseudo-element
- `backdrop-filter` CSS property
- `backface-visibility` CSS property
- `background` CSS property
- `background-attachment` CSS property
- `background-blend-mode` CSS property
- `background-clip` CSS property
- `background-color` CSS property
- `background-image` CSS property
- `background-origin` CSS property
- `background-position` CSS property
- `background-position-x` CSS property
- `background-position-y` CSS property
- `background-repeat` CSS property
- `background-repeat-x` CSS property
- `background-repeat-y` CSS property
- `background-size` CSS property
- `base-palette` CSS at-rule descriptor (`@font-palette-values`)
- `<baseline-position>` CSS type
- `baseline-shift` CSS property
- `baseline-source` CSS property
- `<basic-shape>` CSS type
- `::before` CSS pseudo-element
- `:blank` CSS pseudo-class
- `<blend-mode>` CSS type
- `block-size` CSS property
- `blur()` CSS function
- `border` CSS property
- `-webkit-border-before` CSS property
- `border-block` CSS property
- `border-block-color` CSS property
- `border-block-end` CSS property
- `border-block-end-color` CSS property
- `border-block-end-style` CSS property
- `border-block-end-width` CSS property
- `border-block-start` CSS property
- `border-block-start-color` CSS property
- `border-block-start-style` CSS property
- `border-block-start-width` CSS property
- `border-block-style` CSS property
- `border-block-width` CSS property
- `border-bottom` CSS property
- `border-bottom-color` CSS property
- `border-bottom-left-radius` CSS property
- `border-bottom-right-radius` CSS property
- `border-bottom-style` CSS property
- `border-bottom-width` CSS property
- `border-collapse` CSS property
- `border-color` CSS property
- `border-end-end-radius` CSS property
- `border-end-start-radius` CSS property
- `border-image` CSS property
- `border-image-outset` CSS property
- `border-image-repeat` CSS property

- `border-image-slice` CSS property
- `border-image-source` CSS property
- `border-image-width` CSS property
- `border-inline` CSS property
- `border-inline-color` CSS property
- `border-inline-end` CSS property
- `border-inline-end-color` CSS property
- `border-inline-end-style` CSS property
- `border-inline-end-width` CSS property
- `border-inline-start` CSS property
- `border-inline-start-color` CSS property
- `border-inline-start-style` CSS property
- `border-inline-start-width` CSS property
- `border-inline-style` CSS property
- `border-inline-width` CSS property
- `border-left` CSS property
- `border-left-color` CSS property
- `border-left-style` CSS property
- `border-left-width` CSS property
- `border-radius` CSS property
- `border-right` CSS property
- `border-right-color` CSS property
- `border-right-style` CSS property
- `border-right-width` CSS property
- `border-shape` CSS property
- `border-spacing` CSS property
- `border-start-end-radius` CSS property
- `border-start-start-radius` CSS property
- `border-style` CSS property
- `border-top` CSS property
- `border-top-color` CSS property
- `border-top-left-radius` CSS property
- `border-top-right-radius` CSS property
- `border-top-style` CSS property
- `border-top-width` CSS property
- `border-width` CSS property
- `bottom` CSS property
- `box-align` CSS property
- `box-decoration-break` CSS property
- `box-direction` CSS property
- `<box-edge>` CSS type
- `box-flex` CSS property
- `box-flex-group` CSS property
- `box-lines` CSS property
- `box-ordinal-group` CSS property
- `box-orient` CSS property
- `box-pack` CSS property
- `-webkit-box-reflect` CSS property
- `box-shadow` CSS property
- `box-sizing` CSS property
- `break-after` CSS property
- `break-before` CSS property
- `break-inside` CSS property
- `brightness()` CSS function
- `:buffering` CSS pseudo-class

C

- Class selectors
- Custom properties (--*): CSS variables
- `calc()` CSS function
- `<calc-keyword>` CSS type
- `calc-size()` CSS function
- `<calc-sum>` CSS type
- `caption-side` CSS property
- `caret` CSS property
- `caret-animation` CSS property
- `caret-color` CSS property
- `caret-shape` CSS property
- `@charset` CSS at-rule
- `:checked` CSS pseudo-class
- `::checkmark` CSS pseudo-element
- `circle()` CSS function
- `clamp()` CSS function
- `clear` CSS property
- `clip` CSS property
- `clip-path` CSS property
- `clip-rule` CSS property
- `color` CSS property
- `color()` CSS function
- `color-interpolation` CSS property
- `color-interpolation-filters` CSS property
- `<color-interpolation-method>` CSS type
- `color-mix()` CSS function
- `@color-profile` CSS at-rule

- color-scheme CSS property
- <color> CSS type
- ::column CSS pseudo-element
- column-count CSS property
- column-fill CSS property
- column-gap CSS property
- column-height CSS property
- column-rule CSS property
- column-rule-break CSS property
- column-rule-color CSS property
- column-rule-inset-cap CSS property
- column-rule-inset-cap-end CSS property
- column-rule-inset-cap-start CSS property
- column-rule-inset-junction CSS property
- column-rule-inset-junction-end CSS property
- column-rule-style CSS property
- column-rule-visibility-items CSS property
- column-rule-width CSS property
- column-span CSS property
- column-width CSS property
- column-wrap CSS property
- columns CSS property
- conic-gradient() CSS function
- contain CSS property
- contain-intrinsic-block-size CSS property
- contain-intrinsic-height CSS property
- contain-intrinsic-inline-size CSS property
- contain-intrinsic-size CSS property
- contain-intrinsic-width CSS property
- @container CSS at-rule
- container CSS property
- container-name CSS property
- container-type CSS property
- content CSS property
- <content-distribution> CSS type
- <content-position> CSS type
- content-visibility CSS property
- contrast() CSS function
- contrast-color() CSS function
- corner-block-end-shape CSS property
- corner-block-start-shape CSS property
- corner-bottom-left-shape CSS property
- corner-bottom-right-shape CSS property
- corner-bottom-shape CSS property
- corner-end-end-shape CSS property
- corner-end-start-shape CSS property
- corner-inline-end-shape CSS property
- corner-inline-start-shape CSS property
- corner-left-shape CSS property
- corner-right-shape CSS property
- corner-shape CSS property
- <corner-shape-value> CSS type
- corner-start-end-shape CSS property
- corner-start-start-shape CSS property
- corner-top-left-shape CSS property
- corner-top-right-shape CSS property
- corner-top-shape CSS property
- cos() CSS function
- counter() CSS function
- counter-increment CSS property
- counter-reset CSS property
- counter-set CSS property
- @counter-style CSS at-rule
- counters() CSS function
- cross-fade() CSS function
- cubic-bezier() CSS function
- ::cue CSS pseudo-element
- :current CSS pseudo-class
- cursor CSS property
- <custom-ident> CSS type
- @custom-media CSS at-rule
- cx CSS property
- cy CSS property

D

- d CSS property
- <dashed-function> CSS type
- <dashed-ident> CSS type
- :default CSS pseudo-class
- :defined CSS pseudo-class
- descent-override CSS at-rule descriptor (@font-face)
- ::details-content CSS pseudo-element
- device-cmyk() CSS function
- <dimension> CSS type

- `:dir()` CSS pseudo-class
- `direction` CSS property
- `:disabled` CSS pseudo-class
- `display` CSS property
- `<display-box>` CSS type
- `<display-inside>` CSS type
- `<display-internal>` CSS type
- `<display-legacy>` CSS type
- `<display-listitem>` CSS type
- `<display-outside>` CSS type
- `@document` CSS at-rule
- `dominant-baseline` CSS property
- `drop-shadow()` CSS function
- `dynamic-range-limit` CSS property
- `dynamic-range-limit-mix()` CSS function

E

- `<easing-function>` CSS type
- `element()` CSS function
- `ellipse()` CSS function
- `:empty` CSS pseudo-class
- `empty-cells` CSS property
- `:enabled` CSS pseudo-class
- `env()` CSS function
- `exp()` CSS function

F

- `fallback` CSS at-rule descriptor (`@counter-style`)
- `field-sizing` CSS property
- `::file-selector-button` CSS pseudo-element
- `fill` CSS property
- `fill-opacity` CSS property
- `fill-rule` CSS property
- `filter` CSS property
- `<filter-function>` CSS type
- `:first` CSS pseudo-class
- `:first-child` CSS pseudo-class
- `::first-letter` CSS pseudo-element
- `::first-line` CSS pseudo-element
- `:first-of-type` CSS pseudo-class
- `fit-content` CSS keyword
- `fit-content()` CSS function
- `flex` CSS property
- `flex-basis` CSS property
- `flex-direction` CSS property
- `flex-flow` CSS property
- `flex-grow` CSS property
- `flex-line-count` CSS property
- `flex-shrink` CSS property
- `flex-wrap` CSS property
- `<flex>` CSS type
- `float` CSS property
- `-moz-float-edge` CSS property
- `flood-color` CSS property
- `flood-opacity` CSS property
- `:focus` CSS pseudo-class
- `:focus-visible` CSS pseudo-class
- `:focus-within` CSS pseudo-class
- `font` CSS property
- `font-display` CSS at-rule descriptor (`@font-face`)
- `font-display` CSS at-rule descriptor (`@font-feature-values`)
- `@font-face` CSS at-rule
- `font-family` CSS at-rule descriptor (`@font-face`)
- `font-family` CSS at-rule descriptor (`@font-palette-values`)
- `font-family` CSS property
- `font-feature-settings` CSS at-rule descriptor (`@font-face`)
- `font-feature-settings` CSS property
- `@font-feature-values` CSS at-rule
- `font-kerning` CSS property
- `font-language-override` CSS property
- `font-optical-sizing` CSS property
- `font-palette` CSS property
- `@font-palette-values` CSS at-rule
- `font-size` CSS property
- `font-size-adjust` CSS property
- `font-smooth` CSS property
- `font-stretch` CSS at-rule descriptor (`@font-face`)
- `font-stretch` CSS property
- `font-style` CSS at-rule descriptor (`@font-face`)
- `font-style` CSS property
- `font-synthesis` CSS property
- `font-synthesis-position` CSS property
- `font-synthesis-small-caps` CSS property
- `font-synthesis-style` CSS property

- font-synthesis-weight CSS property
- font-variant CSS property
- font-variant-alternates CSS property
- font-variant-caps CSS property
- font-variant-east-asian CSS property
- font-variant-emoji CSS property
- font-variant-ligatures CSS property
- font-variant-numeric CSS property
- font-variant-position CSS property
- font-variation-settings CSS at-rule descriptor (@font-face)
- font-variation-settings CSS property
- font-weight CSS at-rule descriptor (@font-face)
- font-weight CSS property
- font-width CSS at-rule descriptor (@font-face)
- font-width CSS property
- -moz-force-broken-image-icon CSS property
- forced-color-adjust CSS property
- frame-sizing CSS property
- <frequency-percentage> CSS type
- <frequency> CSS type
- :fullscreen CSS pseudo-class
- @function CSS at-rule
- :future CSS pseudo-class

G

- gap CSS property
- <generic-family> CSS type
- <gradient> CSS type
- ::grammar-error CSS pseudo-element
- grayscale() CSS function
- grid CSS property
- grid-area CSS property
- grid-auto-columns CSS property
- grid-auto-flow CSS property
- grid-auto-rows CSS property
- grid-column CSS property
- grid-column-end CSS property
- grid-column-start CSS property
- grid-row CSS property
- grid-row-end CSS property
- grid-row-start CSS property
- grid-template CSS property
- grid-template-areas CSS property
- grid-template-columns CSS property
- grid-template-rows CSS property

H

- hanging-punctuation CSS property
- :has() CSS pseudo-class
- :has-slotted CSS pseudo-class
- :heading CSS pseudo-class
- :heading() CSS pseudo-class
- height CSS property
- <hex-color> CSS type
- ::highlight() CSS pseudo-element
- :host CSS pseudo-class
- :host() CSS pseudo-class
- :host-context() CSS pseudo-class
- :hover CSS pseudo-class
- hsl() CSS function
- <hue-interpolation-method> CSS type
- hue-rotate() CSS function
- <hue> CSS type
- hwb() CSS function
- hyphenate-character CSS property
- hyphenate-limit-chars CSS property
- hyphens CSS property
- hypot() CSS function

I

- ID selectors
- <ident> CSS type
- if() CSS function
- image() CSS function
- image-orientation CSS property
- -moz-image-rect() CSS function
- image-rendering CSS property
- image-resolution CSS property
- image-set() CSS function
- <image> CSS type
- @import CSS at-rule
- !important CSS keyword
- :in-range CSS pseudo-class
- :indeterminate CSS pseudo-class

- `inherit` CSS keyword
- `inherits` CSS at-rule descriptor (`@property`)
- `initial` CSS keyword
- `initial-letter` CSS property
- `initial-value` CSS at-rule descriptor (`@property`)
- `inline-size` CSS property
- `inset` CSS property
- `inset()` CSS function
- `inset-block` CSS property
- `inset-block-end` CSS property
- `inset-block-start` CSS property
- `inset-inline` CSS property
- `inset-inline-end` CSS property
- `inset-inline-start` CSS property
- `<integer>` CSS type
- `interactivity` CSS property
- `interest-delay` CSS property
- `interest-delay-end` CSS property
- `interest-delay-start` CSS property
- `:interest-source` CSS pseudo-class
- `:interest-target` CSS pseudo-class
- `interpolate-size` CSS property
- `:invalid` CSS pseudo-class
- `invert()` CSS function
- `:is()` CSS pseudo-class
- `isolation` CSS property

J

- `justify-content` CSS property
- `justify-items` CSS property
- `justify-self` CSS property

K

- Keyframe selectors
- `@keyframes` CSS at-rule

L

- `lab()` CSS function
- `:lang()` CSS pseudo-class
- `:last-child` CSS pseudo-class
- `:last-of-type` CSS pseudo-class
- `@layer` CSS at-rule
- `layer()` CSS function
- `lch()` CSS function
- `left` CSS property
- `:left` CSS pseudo-class
- `<length-percentage>` CSS type
- `<length>` CSS type
- `letter-spacing` CSS property
- `light-dark()` CSS function
- `lighting-color` CSS property
- `line-break` CSS property
- `line-clamp` CSS property
- `line-gap-override` CSS at-rule descriptor (`@font-face`)
- `line-height` CSS property
- `line-height-step` CSS property
- `<line-style>` CSS type
- `<line-width>` CSS type
- `linear()` CSS function
- `linear-gradient()` CSS function
- `:link` CSS pseudo-class
- `link-parameters` CSS property
- `list-style` CSS property
- `list-style-image` CSS property
- `list-style-position` CSS property
- `list-style-type` CSS property
- `:local-link` CSS pseudo-class
- `log()` CSS function

M

- `margin` CSS property
- `margin-block` CSS property
- `margin-block-end` CSS property
- `margin-block-start` CSS property
- `margin-bottom` CSS property
- `margin-inline` CSS property
- `margin-inline-end` CSS property
- `margin-inline-start` CSS property
- `margin-left` CSS property
- `margin-right` CSS property
- `margin-top` CSS property
- `margin-trim` CSS property
- `marker` CSS property
- `::marker` CSS pseudo-element

- `marker-end` CSS property
- `marker-mid` CSS property
- `marker-start` CSS property
- `mask` CSS property
- `mask-border` CSS property
- `mask-border-mode` CSS property
- `mask-border-outset` CSS property
- `mask-border-repeat` CSS property
- `mask-border-slice` CSS property
- `mask-border-source` CSS property
- `mask-border-width` CSS property
- `-webkit-mask-box-image` CSS property
- `mask-clip` CSS property
- `-webkit-mask-composite` CSS property
- `mask-composite` CSS property
- `mask-image` CSS property
- `mask-mode` CSS property
- `mask-origin` CSS property
- `mask-position` CSS property
- `-webkit-mask-position-x` CSS property
- `-webkit-mask-position-y` CSS property
- `mask-repeat` CSS property
- `-webkit-mask-repeat-x` CSS property
- `-webkit-mask-repeat-y` CSS property
- `mask-size` CSS property
- `mask-type` CSS property
- `math-depth` CSS property
- `math-shift` CSS property
- `math-style` CSS property
- `matrix()` CSS function
- `matrix3d()` CSS function
- `max()` CSS function
- `max-block-size` CSS property
- `max-content` CSS keyword
- `max-height` CSS property
- `max-inline-size` CSS property
- `max-width` CSS property
- `@media` CSS at-rule
- `min()` CSS function
- `min-block-size` CSS property
- `min-content` CSS keyword
- `min-height` CSS property
- `min-inline-size` CSS property
- `min-width` CSS property
- `minmax()` CSS function
- `mix-blend-mode` CSS property
- `mod()` CSS function
- `:modal` CSS pseudo-class
- `:muted` CSS pseudo-class

N

- Namespace separator
- `<named-color>` CSS type
- `@namespace` CSS at-rule
- `negative` CSS at-rule descriptor (`@counter-style`)
- & nesting selector
- `:not()` CSS pseudo-class
- `:nth-child()` CSS pseudo-class
- `:nth-last-child()` CSS pseudo-class
- `:nth-last-of-type()` CSS pseudo-class
- `:nth-of-type()` CSS pseudo-class
- `<number>` CSS type

O

- `object-fit` CSS property
- `object-position` CSS property
- `object-view-box` CSS property
- `offset` CSS property
- `offset-anchor` CSS property
- `offset-distance` CSS property
- `offset-path` CSS property
- `offset-position` CSS property
- `offset-rotate` CSS property
- `oklab()` CSS function
- `oklch()` CSS function
- `:only-child` CSS pseudo-class
- `:only-of-type` CSS pseudo-class
- `opacity` CSS property
- `opacity()` CSS function
- `:open` CSS pseudo-class
- `:optional` CSS pseudo-class
- `order` CSS property
- `-moz-orient` CSS property
- `orphans` CSS property

- `:out-of-range` CSS pseudo-class
- `outline` CSS property
- `outline-color` CSS property
- `outline-offset` CSS property
- `outline-style` CSS property
- `outline-width` CSS property
- `overflow` CSS property
- `overflow-anchor` CSS property
- `overflow-block` CSS property
- `overflow-clip-margin` CSS property
- `overflow-inline` CSS property
- `<overflow-position>` CSS type
- `overflow-wrap` CSS property
- `overflow-x` CSS property
- `overflow-y` CSS property
- `<overflow>` CSS type
- `overlay` CSS property
- `override-colors` CSS at-rule descriptor (`@font-palette-values`)
- `overscroll-behavior` CSS property
- `overscroll-behavior-block` CSS property
- `overscroll-behavior-inline` CSS property
- `overscroll-behavior-x` CSS property
- `overscroll-behavior-y` CSS property

P

- `pad` CSS at-rule descriptor (`@counter-style`)
- `padding` CSS property
- `padding-block` CSS property
- `padding-block-end` CSS property
- `padding-block-start` CSS property
- `padding-bottom` CSS property
- `padding-inline` CSS property
- `padding-inline-end` CSS property
- `padding-inline-start` CSS property
- `padding-left` CSS property
- `padding-right` CSS property
- `padding-top` CSS property
- `@page` CSS at-rule
- `page` CSS property
- `page-break-after` CSS property
- `page-break-before` CSS property
- `page-break-inside` CSS property
- `page-orientation` CSS at-rule descriptor (`@page`)
- `paint()` CSS function
- `paint-order` CSS property
- `palette-mix()` CSS function
- `param()` CSS function
- `::part()` CSS pseudo-element
- `:past` CSS pseudo-class
- `path()` CSS function
- `path-length` CSS property
- `:paused` CSS pseudo-class
- `<percentage>` CSS type
- `perspective` CSS property
- `perspective()` CSS function
- `perspective-origin` CSS property
- `::picker()` CSS pseudo-element
- `::picker-icon` CSS pseudo-element
- `:picture-in-picture` CSS pseudo-class
- `place-content` CSS property
- `place-items` CSS property
- `place-self` CSS property
- `::placeholder` CSS pseudo-element
- `:placeholder-shown` CSS pseudo-class
- `:playing` CSS pseudo-class
- `pointer-events` CSS property
- `polygon()` CSS function
- `:popover-open` CSS pseudo-class
- `position` CSS property
- `position-anchor` CSS property
- `position-area` CSS property
- `<position-area>` CSS type
- `@position-try` CSS at-rule
- `position-try` CSS property
- `position-try-fallbacks` CSS property
- `position-try-order` CSS property
- `position-visibility` CSS property
- `<position>` CSS type
- `pow()` CSS function
- `prefix` CSS at-rule descriptor (`@counter-style`)
- `print-color-adjust` CSS property
- `progress()` CSS function
- `@property` CSS at-rule

Q

- `quotes` CSS property

R

- `r` CSS property
- `radial-gradient()` CSS function
- `random()` CSS function
- `range` CSS at-rule descriptor (`@counter-style`)
- `<ratio>` CSS type
- `ray()` CSS function
- `:read-only` CSS pseudo-class
- `:read-write` CSS pseudo-class
- `reading-flow` CSS property
- `reading-order` CSS property
- `rect()` CSS function
- `<relative-size>` CSS type
- `rem()` CSS function
- `repeat()` CSS function
- `repeating-conic-gradient()` CSS function
- `repeating-linear-gradient()` CSS function
- `repeating-radial-gradient()` CSS function
- `:required` CSS pseudo-class
- `resize` CSS property
- `<resolution>` CSS type
- `revert` CSS keyword
- `revert-layer` CSS keyword
- `revert-rule` CSS keyword
- `rgb()` CSS function
- `right` CSS property
- `:right` CSS pseudo-class
- `:root` CSS pseudo-class
- `rotate` CSS property
- `rotate()` CSS function
- `rotate3d()` CSS function
- `rotateX()` CSS function
- `rotateY()` CSS function
- `rotateZ()` CSS function
- `round()` CSS function
- `row-gap` CSS property
- `row-rule` CSS property
- `row-rule-break` CSS property
- `row-rule-color` CSS property
- `row-rule-inset-cap` CSS property
- `row-rule-inset-cap-end` CSS property
- `row-rule-inset-cap-start` CSS property
- `row-rule-inset-junction` CSS property
- `row-rule-style` CSS property
- `row-rule-visibility-items` CSS property
- `row-rule-width` CSS property
- `ruby-align` CSS property
- `ruby-overhang` CSS property
- `ruby-position` CSS property
- `rule` CSS property
- `rule-break` CSS property
- `rule-color` CSS property
- `rule-inset-cap` CSS property
- `rule-inset-junction` CSS property
- `<rule-list>` CSS type
- `rule-overlap` CSS property
- `rule-style` CSS property
- `rule-visibility-items` CSS property
- `rule-width` CSS property
- `rx` CSS property
- `ry` CSS property

S

- Selector list
- `saturate()` CSS function
- `scale` CSS property
- `scale()` CSS function
- `scale3d()` CSS function
- `scaleX()` CSS function
- `scaleY()` CSS function
- `scaleZ()` CSS function
- `@scope` CSS at-rule
- `:scope` CSS pseudo-class
- `scroll()` CSS function
- `scroll-behavior` CSS property
- `::scroll-button()` CSS pseudo-element
- `scroll-initial-target` CSS property
- `scroll-margin` CSS property
- `scroll-margin-block` CSS property
- `scroll-margin-block-end` CSS property

- scroll-margin-block-start CSS property
- scroll-margin-bottom CSS property
- scroll-margin-inline CSS property
- scroll-margin-inline-end CSS property
- scroll-margin-inline-start CSS property
- scroll-margin-left CSS property
- scroll-margin-right CSS property
- scroll-margin-top CSS property
- ::scroll-marker CSS pseudo-element
- scroll-marker-group CSS property
- ::scroll-marker-group CSS pseudo-element
- scroll-padding CSS property
- scroll-padding-block CSS property
- scroll-padding-block-end CSS property
- scroll-padding-block-start CSS property
- scroll-padding-bottom CSS property
- scroll-padding-inline CSS property
- scroll-padding-inline-end CSS property
- scroll-padding-inline-start CSS property
- scroll-padding-left CSS property
- scroll-padding-right CSS property
- scroll-padding-top CSS property
- scroll-snap-align CSS property
- scroll-snap-stop CSS property
- scroll-snap-type CSS property
- scroll-target-group CSS property
- scroll-timeline CSS property
- scroll-timeline-axis CSS property
- scroll-timeline-name CSS property
- scrollbar-color CSS property
- scrollbar-gutter CSS property
- scrollbar-width CSS property
- ::search-text CSS pseudo-element
- :seeking CSS pseudo-class
- ::selection CSS pseudo-element
- <self-position> CSS type
- sepia() CSS function
- shape() CSS function
- shape-image-threshold CSS property
- shape-margin CSS property
- shape-outside CSS property
- shape-rendering CSS property
- <shape> CSS type
- sibling-count() CSS function
- sibling-index() CSS function
- sign() CSS function
- sin() CSS function
- size CSS at-rule descriptor (@page)
- size-adjust CSS at-rule descriptor (@font-face)
- skew() CSS function
- skewX() CSS function
- skewY() CSS function
- ::slotted() CSS pseudo-element
- speak-as CSS at-rule descriptor (@counter-style)
- speak-as CSS property
- ::spelling-error CSS pseudo-element
- sqrt() CSS function
- src CSS at-rule descriptor (@font-face)
- :stalled CSS pseudo-class
- @starting-style CSS at-rule
- :state() CSS pseudo-class
- steps() CSS function
- stop-color CSS property
- stop-opacity CSS property
- <string> CSS type
- stroke CSS property
- stroke-dasharray CSS property
- stroke-dashoffset CSS property
- stroke-linecap CSS property
- stroke-linejoin CSS property
- stroke-miterlimit CSS property
- stroke-opacity CSS property
- stroke-width CSS property
- suffix CSS at-rule descriptor (@counter-style)
- superellipse() CSS function
- @supports CSS at-rule
- symbols CSS at-rule descriptor (@counter-style)
- symbols() CSS function
- syntax CSS at-rule descriptor (@property)
- system CSS at-rule descriptor (@counter-style)
- <system-color> CSS type

T

- Type selectors

- `tab-size` CSS property
- `table-layout` CSS property
- `tan()` CSS function
- `-webkit-tap-highlight-color` CSS property
- `:target` CSS pseudo-class
- `:target-after` CSS pseudo-class
- `:target-before` CSS pseudo-class
- `:target-current` CSS pseudo-class
- `::target-text` CSS pseudo-element
- `text-align` CSS property
- `text-align-last` CSS property
- `text-anchor` CSS property
- `text-autospace` CSS property
- `text-box` CSS property
- `text-box-edge` CSS property
- `text-box-trim` CSS property
- `text-combine-upright` CSS property
- `text-decoration` CSS property
- `text-decoration-color` CSS property
- `text-decoration-inset` CSS property
- `text-decoration-line` CSS property
- `text-decoration-skip` CSS property
- `text-decoration-skip-ink` CSS property
- `text-decoration-style` CSS property
- `text-decoration-thickness` CSS property
- `<text-edge>` CSS type
- `text-emphasis` CSS property
- `text-emphasis-color` CSS property
- `text-emphasis-position` CSS property
- `text-emphasis-style` CSS property
- `-webkit-text-fill-color` CSS property
- `text-fit` CSS property
- `text-indent` CSS property
- `text-justify` CSS property
- `text-orientation` CSS property
- `text-overflow` CSS property
- `text-rendering` CSS property
- `-webkit-text-security` CSS property
- `text-shadow` CSS property
- `text-size-adjust` CSS property
- `text-spacing-trim` CSS property
- `-webkit-text-stroke` CSS property
- `-webkit-text-stroke-color` CSS property
- `-webkit-text-stroke-width` CSS property
- `text-transform` CSS property
- `text-underline-offset` CSS property
- `text-underline-position` CSS property
- `text-wrap` CSS property
- `text-wrap-mode` CSS property
- `text-wrap-style` CSS property
- `<time-percentage>` CSS type
- `<time>` CSS type
- `<timeline-range-name>` CSS type
- `timeline-scope` CSS property
- timeline-trigger CSS property
- timeline-trigger-activation-range CSS property
- timeline-trigger-activation-range-end CSS property
- timeline-trigger-activation-range-start CSS property
- timeline-trigger-active-range CSS property
- timeline-trigger-active-range-end CSS property
- timeline-trigger-active-range-start CSS property
- timeline-trigger-name CSS property
- timeline-trigger-source CSS property
- `top` CSS property
- `touch-action` CSS property
- `-webkit-touch-callout` CSS property
- `transform` CSS property
- `transform-box` CSS property
- `<transform-function>` CSS type
- `transform-origin` CSS property
- `transform-style` CSS property
- `transition` CSS property
- `transition-behavior` CSS property
- `transition-delay` CSS property
- `transition-duration` CSS property
- `transition-property` CSS property
- `transition-timing-function` CSS property
- `translate` CSS property
- `translate()` CSS function
- `translate3d()` CSS function
- `translateX()` CSS function
- `translateY()` CSS function
- `translateZ()` CSS function
- trigger-scope CSS property
- `type()` CSS function

U

- Universal selectors
- `unicode-bidi` CSS property
- `unicode-range` CSS at-rule descriptor (`@font-face`)
- `unset` CSS keyword
- `url()` CSS function
- `<url>` CSS type
- `-moz-user-focus` CSS property
- `-moz-user-input` CSS property
- `:user-invalid` CSS pseudo-class
- `user-modify` CSS property
- `user-select` CSS property
- `:user-valid` CSS pseudo-class

V

- `:valid` CSS pseudo-class
- `var()` CSS function
- `vector-effect` CSS property
- `vertical-align` CSS property
- `view()` CSS function
- `view-timeline` CSS property
- `view-timeline-axis` CSS property
- `view-timeline-inset` CSS property
- `view-timeline-name` CSS property
- `@view-transition` CSS at-rule
- `::view-transition` CSS pseudo-element
- `view-transition-class` CSS property
- `::view-transition-group()` CSS pseudo-element
- `::view-transition-image-pair()` CSS pseudo-element
- `view-transition-name` CSS property
- `::view-transition-new()` CSS pseudo-element
- `::view-transition-old()` CSS pseudo-element
- `view-transition-scope` CSS property
- `visibility` CSS property
- `:visited` CSS pseudo-class
- `:volume-locked` CSS pseudo-class

W

- `:where()` CSS pseudo-class
- `white-space` CSS property
- `white-space-collapse` CSS property
- `widows` CSS property
- `width` CSS property
- `will-change` CSS property
- `word-break` CSS property
- `word-spacing` CSS property
- `writing-mode` CSS property

X

- `x` CSS property
- `:xr-overlay` CSS pseudo-class
- `xywh()` CSS function

Y

- `y` CSS property

Z

- `z-index` CSS property
- `zoom` CSS property

Selectors

The following are the various selectors, which allow styles to be conditional based on various features of elements within the DOM.

Basic selectors

Basic selectors are fundamental selectors; these are the most basic selectors that are frequently combined to create other, more complex selectors.

- Universal selector `*`
- Type selector `elementname`
- Class selector `.classname`
- ID selector `#idname`
- Attribute selector `[attr=value]`

Grouping selectors

Selector list `A, B`

> Specifies that both `A` and `B` elements are selected. This is a grouping method to select several matching elements.

Combinators

Combinators are selectors that establish a relationship between two or more simple selectors, such as "`A` is a child of `B`" or "`A` is adjacent to `B`", creating a complex selector.

Next-sibling combinator `A + B`

> Specifies that the elements selected by both `A` and `B` have the same parent and that the element selected by `B` immediately follows the element selected by `A` horizontally.

Subsequent-sibling combinator `A ~ B`

Specifies that the elements selected by both `A` and `B` share the same parent and that the element selected by `A` comes before—but not necessarily immediately before—the element selected by `B`.

Child combinator `A > B`

Specifies that the element selected by `B` is the direct child of the element selected by `A`.

Descendant combinator `A B`

Specifies that the element selected by `B` is a descendant of the element selected by `A`, but is not necessarily a direct child.

Column combinator `A || B`

Specifies that the element selected by `B` is located within the table column specified by `A`. Elements which span multiple columns are considered to be a member of all of those columns.

Pseudo

Pseudo classes `:`

Specifies a special state of the selected element(s).

Pseudo elements `::`

Represents entities that are not included in HTML.

See also selectors in the Selectors specification and the pseudo-element specification.

Concepts

Syntax and semantics

- CSS syntax
- At-rules
- Cascade
- Comments
- Descriptor
- Inheritance
- Shorthand properties
- Specificity
- Values
- Value definition syntax
- CSS values and units
- CSS functional notations

Values

- Actual value
- Computed value
- Initial value
- Resolved value
- Specified value
- Used value

Layout

- Block formatting context
- Box model
- Containing block
- Layout mode
- Margin collapsing
- Replaced elements
- Stacking context
- Visual formatting model

DOM-CSS / CSSOM

Major object types

- `Document.styleSheets`
- `HTMLElement.style`
- `Element.className`
- `Element.classList`
- `StyleSheetList`
- `CSSRuleList`
- `CSSRule`
- `CSSStyleRule`
- `CSSStyleDeclaration`

Important methods

- `CSSStyleSheet.insertRule()`
- `CSSStyleSheet.deleteRule()`

See also

- Mozilla CSS extensions (prefixed with `-moz-`)
- WebKit CSS extensions (mostly prefixed with `-webkit-`)

External Links

- CSS Indices (w3.org)

CSS at-rules

At-rules are CSS statements that instruct CSS how to behave. They are used to group and structure style rules and other at-rules, declare style information not directly associated with selected content, and manage syntactic constructs such as imports and namespaces keyword mappings. They begin with an at-sign, `@` (U+0040 COMMERCIAL AT), followed by an identifier.

Index of at-rules and at-rule descriptors

- `@charset`
- `@color-profile`
- `@container`
- `@counter-style`
  - `additive-symbols`
  - `fallback`
  - `negative`
  - `pad`
  - `prefix`
  - `range`
  - `speak-as`
  - `suffix`
  - `symbols`
  - `system`
- `@custom-media`
- `@document`
- `@font-face`
  - `ascent-override`
  - `descent-override`
  - `font-display`
  - `font-family`
  - `font-feature-settings`
  - `font-stretch`
  - `font-style`
  - `font-variation-settings`
  - `font-weight`
  - `line-gap-override`
  - `size-adjust`
  - `src`
  - `unicode-range`
- `@font-feature-values`
  - `font-display`
- `@font-palette-values`
  - `base-palette`
  - `font-family`
  - `override-colors`
- `@function`
- `@import`
- `@keyframes`
- `@layer`
- `@media` (see list of media features in the next section)
- `@namespace`
- `@page`
  - `page-orientation`
  - `size`
- `@position-try`
- `@property`
  - `inherits`
  - `initial-value`
  - `syntax`
- `@scope`
- `@starting-style`
- `@supports`
- `@view-transition`

Index of media features

- `-moz-device-pixel-ratio`
- `-webkit-animation`
- `-webkit-device-pixel-ratio`
- `-webkit-transform-2d`
- `-webkit-transform-3d`
- `-webkit-transition`
- `any-hover`
- `any-pointer`
- `aspect-ratio`
- `color`
- `color-gamut`
- `color-index`
- `device-aspect-ratio`
- `device-height`
- `display-mode`
- `dynamic-range`
- `forced-colors`
- `grid`
- `height`
- `horizontal-viewport-segments`
- `hover`
- `inverted-colors`
- `monochrome`
- `orientation`
- `overflow-block`
- `overflow-inline`
- `pointer`

- `prefers-color-scheme`
- `prefers-contrast`
- `prefers-reduced-data`
- `prefers-reduced-motion`
- `prefers-reduced-transparency`
- `resolution`
- `scan`
- `scripting`
- `shape`
- `update`
- `vertical-viewport-segments`
- `video-dynamic-range`
- `width`

See also

- CSS at-rule functions
- Nesting at-rules
- CSS syntax module
- CSS conditional rules module

@charset CSS at-rule

The **`@charset`** CSS rule specifies the character encoding used in the style sheet. This syntax is useful when using non-ASCII characters in some CSS properties, like `content`. Although the first character in `@charset` is the `@` symbol, it is not an at-rule. It is a specific byte sequence that can only be placed at the very beginning of a stylesheet. No other characters, except the Unicode byte-order mark, are allowed before it. It also does not follow normal CSS syntax rules such as use of quotes or whitespace.

If a `@charset` is not recognized as the charset declaration, it is parsed as a normal at-rule. The CSS syntax module deprecates this fallback behavior, defining it as an unrecognized legacy rule to be dropped when a stylesheet is grammar-checked.

As there are several ways to define the character encoding of a style sheet, the browser will try the following methods in the following order (and stop as soon as one yields a result):

1. The value of the Unicode byte-order character placed at the beginning of the file.
2. The value given by the `charset` attribute of the `Content-Type:` HTTP header or the equivalent in the protocol used to serve the style sheet.
3. The `@charset` CSS declaration.
4. Use the character encoding defined by the referring document: the `charset` attribute of the `<link>` element. This method is obsolete and should not be used.
5. Assume that the document is UTF-8.

Syntax

```
@charset "UTF-8";
@charset "iso-8859-15";
```

Parameters

charset

A `<string>` denoting the character encoding to be used. It must be the name of a web-safe character encoding defined in the IANA-registry, and must be double-quoted, following exactly one space character (U+0020), and immediately terminated with a semicolon. If several names are associated with an encoding, only the one marked with *preferred* must be used.

Formal syntax

Note that the `@charset` rule is not parsed via syntax, but via a specific byte sequence of the following form:

```
@charset "<charset>";
```

Examples

Valid and invalid charset declarations

```
@charset "UTF-8"; /* Set the encoding of the style sheet to Unicode UTF-8 */
```

```
@charset 'iso-8859-15'; /* Invalid, wrong quotes used */
@charset  "UTF-8"; /* Invalid, more than one space */
 @charset "UTF-8"; /* Invalid, there is a character (a space) before the declarations */
@charset UTF-8; /* Invalid, the charset is a CSS <string> and requires double-quotes */
```

See also

- Character set glossary entry
- Unicode glossary entry

@color-profile CSS at-rule

The **`@color-profile`** CSS at-rule defines and names a color profile which can later be used in the `color()` function to specify a color.

Syntax

```
@color-profile --swop5c {
  src: url("https://example.org/SWOP2006_Coated5v2.icc");
}
```

Parameters

profile name

Either a `<dashed-ident>` or the identifier `device-cmyk`.

Descriptors

`src`

Specifies the URL to retrieve the color-profile information from.

`rendering-intent`

If the color profile contains more than one rendering intent, this descriptor allows one to be selected as the one to use to define how to map the color to smaller gamuts than this profile is defined over.

If used, it must be one of the following keywords:

`relative-colorimetric`

Media-relative colorimetric is required to leave source colors that fall inside the destination medium gamut unchanged relative to the respective media white points. Source colors that are out of the destination medium gamut are mapped to colors on the gamut boundary using a variety of different methods.

`absolute-colorimetric`

ICC-absolute colorimetric is required to leave source colors that fall inside the destination medium gamut unchanged relative to the adopted white (a perfect reflecting diffuser). Source colors that are out of the destination medium gamut are mapped to colors on the gamut boundary using a variety of different methods.

`perceptual`

This method is often the preferred choice for images, especially when there are substantial differences between the source and destination (such as a screen display image reproduced on a reflection print). It takes the colors of the source image and re-optimizes the appearance for the destination medium using proprietary methods.

`saturation`

This option was created to preserve the relative saturation (chroma) of the original, and to keep solid colors pure. However, it experienced interoperability problems like the perceptual intent.

Formal syntax

```
@color-profile =
  @color-profile [ <dashed-ident> | device-cmyk ] { <declaration-list> }
```

This syntax reflects the latest standard as per CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

This example demonstrates using offset printing to ISO 12647-2:2004 using the CGATS/SWOP TR005 2007 characterization data on grade 5 paper with an ink limit of 300% Total Area Coverage, and medium gray component replacement (GCR). This example is from the specification.

The `src` descriptor specifies the URL to retrieve the color-profile information from.

```
@color-profile --swop5c {
  src: url("https://example.org/SWOP2006_Coated5v2.icc");
}
.header {
  background-color: color(--swop5c 0% 70% 20% 0%);
}
```

Browser compatibility

Currently, no browsers support this feature.

See also

- CSS colors module

@container CSS at-rule

The **`@container`** CSS at-rule is a conditional group rule that applies styles to a containment context. Style declarations are filtered by a condition and applied to the elements within the container if the condition is true. The condition is evaluated when the queried container size, `<style-feature>`, or scroll-state changes.

The condition must specify a `container-name`, a `<container-query>`, or both.

The `container-name` property specifies a list of query container names, which are used to filter which containers are targeted by the `@container` rules. If only a `container-name` value is included (a name-only container query), the contained rules are applied to any containers with one or more of those `container-name` values set on them.

The container features in the `<container-query>` are evaluated against the selected containers. If no `<container-name>` is specified, the `<container-query>` features are evaluated against the nearest ancestor query container that has the matching `container-type`.

Syntax

```
/* With a <size-query> */
@container (width > 400px) {
  h2 {
    font-size: 1.5em;
  }
}

/* With an optional <container-name> */
@container tall (height > 30rem) {
  p {
    line-height: 1.6;
  }
}

/* With a <container-name> only (query is optional) */
@container sidebar {
  h2 {
    background: blue;
  }
}

/* With a <scroll-state> */
@container scroll-state(scrollable: top) {
  .back-to-top-link {
    visibility: visible;
  }
}

/* With an anchored query */
@container anchored(fallback: bottom) {
  .infobox::before {
    content: "▲";
    bottom: 100%;
    top: auto;
```

```
  }
}

/* With a <container-name> and a <scroll-state> */
@container sticky-heading scroll-state(stuck: top) {
  h2 {
    background: purple;
    color: white;
  }
}

/* Multiple queries in a single condition */
@container (width > 400px) and style(--responsive: true) {
  h2 {
    font-size: 1.5em;
  }
}

/* Condition list */
@container card (width > 400px), style(--responsive: true), scroll-state(stuck: top) {
  h2 {
    font-size: 1.5em;
  }
}

/* Boolean style() queries */
@container style(--theme: one) or style(--theme: two) {
  /* matched container styles */
}
@container style((--theme: one) or (--theme: two)) {
  /* matched container styles */
}
@container style(--theme: one) and style(--theme: two) {
  /* matched container styles */
}
@container style((--theme: one) and (--theme: two)) {
  /* matched container styles */
}
@container not style(--theme: one) {
  /* matched container styles */
}

/* range style() queries */
@container style(--number > 4) {
  /* matched container styles */
}
```

Parameters

`<container-condition>`

One or both of `<container-name>` and `<container-query>`. Styles defined in the `<stylesheet>` are applied if the condition is `true`.

`<container-name>` Optional

The name of the container to query; it is specified as an `<ident>`. If the query evaluates to `true`, the declared styles are applied to the container's descendant elements.

`<container-query>` Optional

A set of features that are evaluated against the query container when the size, `<style-feature>`, scroll-state, or applied position-try fallback of the container changes.

Logical keywords in container queries

Logical keywords can be used to define the container condition:

- `and` combines two or more conditions.
- `or` combines two or more conditions.
- `not` negates the condition. Only one `not` condition is allowed per container query and it cannot be used with the `and` or `or` keywords.

```
@container (width > 400px) and (height > 400px) {
  /* <stylesheet> */
}

@container (width > 400px) or (height > 400px) {
  /* <stylesheet> */
}

@container not (width < 400px) {
  /* <stylesheet> */
}
```

Named containment contexts

A containment context can be named using the `container-name` property.

```
.post {
  container-name: sidebar;
  container-type: inline-size;
}
```

The shorthand syntax for this is to use `container` in the form `container: <name> / <type>`, for example:

```
.post {
  container: sidebar / inline-size;
}
```

In container queries, the `container-name` property is used to filter the set of containers to those with a matching query container name:

```
@container sidebar (width > 400px) {
  /* <stylesheet> */
```

```
}
```

Details about usage and naming restrictions are described in the `container-name` page.

Descriptors

The `<container-condition>` queries include size, scroll-state, and anchored container descriptors.

Size container descriptors

The `<container-condition>` can include one or more boolean size queries, each within a set of parentheses. A size query includes a size descriptor, a value, and — depending on the descriptor — a comparison operator. The queries always measure the content box as the comparison. The syntax for including multiple conditions is the same as for `@media` size feature queries.

```
@container (min-width: 400px) {
  /* … */
}
@container (orientation: landscape) and (width > 400px) {
  /* … */
}
@container (15em <= block-size <= 30em) {
  /* … */
}
```

`aspect-ratio`

: The `aspect-ratio` of the container calculated as the width to the height of the container expressed as a `<ratio>` value.

`block-size`

: The `block-size` of the container expressed as a `<length>` value.

`height`

: The height of the container expressed as a `<length>` value.

`inline-size`

: The `inline-size` of the container expressed as a `<length>` value.

`orientation`

: The orientation of the container, either `landscape` or `portrait`.

`width`

: The width of the container expressed as a `<length>` value.

Scroll-state container descriptors

Scroll-state container descriptors are specified inside the `<container-condition>` as an argument for the `scroll-state()` function, for example:

```
@container scroll-state(scrollable: top) {
  /* … */
}
@container scroll-state(scrolled: block-end) {
  /* … */
}
@container scroll-state(stuck: inline-end) {
  /* … */
}
@container scroll-state(snapped: both) {
  /* … */
}
```

Supported keywords for scroll-state container descriptors include physical and flow relative values.

`scrollable`

: Queries whether the container can be scrolled in the given direction via user-initiated scrolling, such as by dragging the scrollbar or using a trackpad gesture. In other words, is there overflowing content in the given direction that can be scrolled to? Valid `scrollable` values include the following keywords:

  `none`

  : The container is not a scroll container or otherwise cannot be scrolled in any direction.

  `top`

  : The container can be scrolled towards its top edge.

  `right`

  : The container can be scrolled towards its right-hand edge.

  `bottom`

  : The container can be scrolled towards its bottom edge.

  `left`

  : The container can be scrolled towards its left-hand edge.

  `x`

  : The container can be scrolled horizontally towards either or both of its left-hand or right-hand edges.

  `y`

  : The container can be scrolled vertically towards either or both of its top or bottom edges.

  `block-start`

  : The container can be scrolled towards its block-start edge.

`block-end`

The container can be scrolled towards its block-end edge.

`inline-start`

The container can be scrolled towards its inline-start edge.

`inline-end`

The container can be scrolled towards its inline-end edge.

`block`

The container can be scrolled in its block direction towards either or both of its block-start or block-end edges.

`inline`

The container can be scrolled in its inline direction towards either or both of its inline-start and inline-end edges.

If the test passes, the rules inside the `@container` block are applied to descendants of the scroll container.

To evaluate whether a container is scrollable, without being concerned about the direction, use the `none` value with the `not` operator:

```
@container not scroll-state(scrollable: none) {
  /* … */
}
```

`scrolled`

Queries whether the container was most recently scrolled in a specified direction. Valid `scrolled` values include the following keywords:

`none`

The container is not a scroll container or otherwise has not previously been scrolled in any direction.

`top`

The container was most recently scrolled towards its top edge.

`right`

The container was most recently scrolled towards its right-hand edge.

`bottom`

The container was most recently scrolled towards its bottom edge.

`left`

The container was most recently scrolled towards its left-hand edge.

`x`

The container was most recently scrolled towards either its left-hand or right-hand edges.

`y`

The container was most recently scrolled towards either its top or bottom edges.

`block-start`

The container was most recently scrolled towards its block-start edge.

`block-end`

The container was most recently scrolled towards its block-end edge.

`inline-start`

The container was most recently scrolled towards its inline-start edge.

`inline-end`

The container was most recently scrolled towards its inline-end edge.

`block`

The container was most recently scrolled towards either its block-start or block-end edges.

`inline`

The container was most recently scrolled towards either its inline-start or inline-end edges.

If the test returns true, the rules nested in the `@container` block are applied to the descendants of the scroll container.

To evaluate whether a container has recently been scrolled, without being concerned about the direction, use the `none` value with the `not` operator:

```
@container not scroll-state(scrolled: none) {
  /* … */
}
```

`snapped`

Queries whether the container is going to be snapped to a scroll snap container ancestor along the given axis. Valid `snapped` values include the following keywords:

`none`

The container is not a scroll snap target for its ancestor scroll container. When implementing a `snapped: none` query, containers that *are* snap targets for the scroll container will *not* have the `@container` styles applied, whereas non-snap targets *will* have the styles applied.

`x`

The container is a horizontal scroll snap target for its ancestor scroll container, that is, it is snapping horizontally to its ancestor.

`y`

The container is a vertical scroll snap target for its ancestor scroll container, that is, it is snapping vertically to its ancestor.

`block`

The container is a block-axis scroll snap target for its ancestor scroll container, that is, it is snapping to its ancestor in the block direction.

`inline`

The container is an inline-axis scroll snap target for its ancestor scroll container, that is, it is snapping to its ancestor in the inline direction.

`both`

The container is both a horizontal and vertical scroll snap target for its ancestor scroll container and is snapping to its ancestor in both directions. The container won't match if it is only snapping to its ancestor along the horizontal *or* vertical axis. It needs to be both.

To evaluate a container with a non-`none` `snapped` scroll-state query, it must be a container with a scroll container ancestor having a `scroll-snap-type` value other than `none`. A `snapped: none` query will match even when there is no scroll container ancestor.

Evaluations occur when `scrollsnapchanging` events fire on the scroll snap container. If the test passes, the rules inside the `@container` block are applied to descendants of the container.

To evaluate whether a container is a snap target, without being concerned about the direction, use the `none` value with the `not` operator:

```
@container not scroll-state(snapped: none) {
  /* … */
}
```

`stuck`

Queries whether a container with a `position` value of `sticky` is stuck to an edge of its scrolling container ancestor. Valid `stuck` values include the following keywords:

`none`

The container is not stuck to any edges of its container. Note that `none` queries will match even if the container does not have `position: sticky` set on it.

`top`

The container is stuck to the top edge of its container.

`right`

The container is stuck to the right-hand edge of its container.

`bottom`

The container is stuck to the bottom edge of its container.

`left`

The container is stuck to the left-hand edge of its container.

`block-start`

The container is stuck to the block-start edge of its container.

`block-end`

The container is stuck to the block-end edge of its container.

`inline-start`

The container is stuck to the inline-start edge of its container.

`inline-end`

The container is stuck to the inline-end edge of its container.

To evaluate a container with a non-`none` `stuck` scroll-state query, it must have `position: sticky` set on it, and be inside a scroll container. If the test passes, the rules inside the `@container` block are applied to descendants of the `position: sticky` container.

It is possible for two values from adjacent axes to match at the same time:

```
@container scroll-state((stuck: top) and (stuck: left)) {
  /* … */
}
```

However, two values from opposite edges will never match at the same time:

```
@container scroll-state((stuck: left) and (stuck: right)) {
  /* … */
}
```

To evaluate whether a container is stuck, without being concerned about the direction, use the `none` value with the `not` operator:

```
@container not scroll-state(stuck: none) {
  /* … */
}
```

Anchored container descriptors

Anchored container descriptors are specified inside the `<container-condition>` as an argument for the `anchored()` function, for example:

```
@container anchored(fallback: top) {
  /* … */
}
@container anchored(fallback: flip-block flip-inline) {
  /* … */
}
@container anchored(fallback: --custom-fallback) {
```

```
  /* … */
}
```

`fallback`

Queries whether a specific position-try fallback is currently active on an anchor-positioned container, as specified via the `position-try-fallbacks` property. Valid `fallback` values include any component value that is valid for inclusion in a `position-try-fallbacks` property value.

If the `fallback` value named in the test is currently active on the anchor-positioned container, the test passes, and the rules inside the `@container` block are applied to descendants of the anchor-positioned container.

Formal syntax

```
@container =
  @container <container-condition># { <rule-list> }

<container-condition> =
  [ <container-name>? <container-query>? ]!

<container-name> =
  <custom-ident>

<container-query> =
  not <query-in-parens>                              |
  <query-in-parens> [ [ and <query-in-parens> ]* | [ or <query-in-parens> ]* ]

<query-in-parens> =
  ( <container-query> )                 |
  ( <size-feature> )                    |
  style( <style-query> )                |
  scroll-state( <scroll-state-query> )  |
  <general-enclosed>

<style-query> =
  not <style-in-parens>                              |
  <style-in-parens> [ [ and <style-in-parens> ]* | [ or <style-in-parens> ]* ]  |
  <style-feature>

<scroll-state-query> =
  not <scroll-state-in-parens>                       |
  <scroll-state-in-parens> [ [ and <scroll-state-in-parens> ]* | [ or <scroll-state-in-parens> ]* ]  |
  <scroll-state-feature>

<general-enclosed> =
  [ <function-token> <any-value>? ) ]  |
  [ ( <any-value>? ) ]

<style-in-parens> =
  ( <style-query> )    |
  ( <style-feature> )  |
  <general-enclosed>

<style-feature> =
  <style-feature-plain>    |
  <style-feature-boolean>  |
  <style-range>

<scroll-state-in-parens> =
  ( <scroll-state-query> )    |
  ( <scroll-state-feature> )  |
  <general-enclosed>

<style-feature-plain> =
  <style-feature-name> : <style-feature-value>

<style-feature-boolean> =
  <style-feature-name>

<style-range> =
  <style-range-value> <mf-comparison> <style-range-value>  |
  <style-range-value> <mf-lt> <style-range-value> <mf-lt> <style-range-value>  |
  <style-range-value> <mf-gt> <style-range-value> <mf-gt> <style-range-value>

<style-range-value> =
  <custom-property-name>  |
  <style-feature-value>

<mf-comparison> =
  <mf-lt>  |
  <mf-gt>  |
  <mf-eq>

<mf-lt> =
  '<' '='?

<mf-gt> =
  '>' '='?

<mf-eq> =
  '='
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 5, Media Queries Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting styles based on a container's size

Consider the following example of a card component with a title and some text:

```
<div class="post">
  <div class="card">
    <h2>Card title</h2>
    <p>Card content</p>
  </div>
</div>
```

A container context can be created using the `container-type` property, in this case using the `inline-size` value on the `.post` class. You can then use the `@container` at-rule to apply styles to the element with the `.card` class in a container that's narrower than `650px`.

```
const post = document.querySelector(".post");
const span = document.createElement("span");
span.textContent = `.post width: ${post.clientWidth}px`;
post.parentNode.insertBefore(span, post.nextSibling);
// update on resize
window.addEventListener("resize", () => {
  span.textContent = `.post width: ${post.clientWidth}px`;
});
```

```
span {
  display: block;
  text-align: center;
}
.card {
  margin: 10px;
  border: 2px dotted;
  font-size: 1.5em;
}
.post {
  border: 2px solid;
}
```

```
/* A container context based on inline size */
.post {
  container-type: inline-size;
}

/* Apply styles if the container is narrower than 650px */
@container (width < 650px) {
  .card {
    width: 50%;
    background-color: lightgray;
    font-size: 1em;
  }
}
```

Creating named container contexts

Given the following HTML example which is a card component with a title and some text:

```
<div class="post">
  <div class="card">
    <h2>Card title</h2>
    <p>Card content</p>
  </div>
</div>
```

First, create a container context using the `container-type` and `container-name` properties. The shorthand syntax for this declaration is described in the `container` page.

```
.post {
  container-type: inline-size;
  container-name: summary;
}
```

Next, target that container by adding the name to the container query:

```
@container summary (width >= 400px) {
  .card {
    font-size: 1.5em;
  }
}
```

Nested container queries

It's not possible to target multiple containers in a single container query. It is possible to nest container queries which has the same effect.

The following query evaluates to true and applies the declared style if the container named `summary` is wider than `400px` and has an ancestor container wider than `800px`:

```
@container summary (width > 400px) {
  @container (width > 800px) {
    /* <stylesheet> */
  }
}
```

Container style queries

Container queries can also evaluate the computed style of the container element. A *container style query* is a `@container` query that uses one or more `style()` functional notations. The boolean syntax and logic combining style features into a style query is the same as for CSS feature queries.

```
@container style(<style-feature>),
    not style(<style-feature>),
    style(<style-feature>) and style(<style-feature>),
    style(<style-feature>) or style(<style-feature>) {
  /* <stylesheet> */
}
```

The parameter of each `style()` is a single `<style-feature>`. A **`<style-feature>`** can be a valid CSS declaration (the **plain** form), a CSS property or `<custom-property-name>` on its own (the **boolean** form), or a range comparison (the **range** form).

```
@container style(--themeBackground),
    not style(background-color: red),
    style(color: green) and style(background-color: transparent),
    style(--themeColor: blue) or style(--themeColor: purple) {
```

```
  /* <stylesheet> */
}
```

A style feature without a value evaluates to true if the computed value is different from the initial value for the given property.

If the `<style-feature>` passed as the `style()` function's argument is a declaration, the style query evaluates to true if the declaration's value is the same as the computed value of that property for the container being queried. Otherwise, it resolves to false.

The following container query checks if the computed value of the container element's `--accent-color` is `blue`:

```
@container style(--accent-color: blue) {
  /* <stylesheet> */
}
```

> **Note:** If a custom property has a value of `blue`, the equivalent hexadecimal code `#0000ff` will not match unless the property has been defined as a color with `@property` so the browser can properly compare computed values.

Style features that query a shorthand property are true if the computed values match for each of its longhand properties, and false otherwise. For example, `@container style(border: 2px solid red)` will resolve to true if all 12 longhand properties (`border-bottom-style`, etc.) that make up that shorthand are true.

Note that `!important` is allowed in style queries but is ignored.

```
/* !important is valid but has no effect */
@container style(--themeColor: purple !important) {
  /* <stylesheet> */
}
```

The global `revert` and `revert-layer` are invalid as values in a `<style-feature>` and cause the container style query to be false.

Range syntax

In addition to the plain `<style-feature-name>: <value>` form described above, a `<style-feature>` can be written as a **range** comparison using `=`, `<`, `<=`, `>`, or `>=`. Range syntax enables **numeric** comparisons that the plain form can't, such as `style(--columns >= 3)` or `style(--gap = 1rem)`. It compares the resolved values of both sides numerically.

To evaluate a range, the browser:

1. Resolves each side (custom property names are looked up as if used with `var()`).
2. Parses each side as a `<number>`, `<percentage>`, `<length>`, `<angle>`, `<time>`, `<frequency>`, or `<resolution>`. If either side can't be parsed as one of those types, or the two sides don't have the same type, the query is false.
3. Computes each side (evaluating any `calc()` expressions) and performs the numeric comparison.

This means range syntax can't be used to compare keyword-like values: `style(--theme = dark)` is always false because `dark` isn't a numeric type. Use the plain syntax for those, for example `style(--theme: dark)`.

Either side of a range can be a custom property name, a `var()` reference, a literal value, or a `calc()` expression, in either order:

```
@container style(3 = --n) {
  /* … */
}
@container style(var(--n) = 3) {
  /* … */
}
@container style(calc(6/2) = var(--n)) {
  /* … */
}
```

A range can also take a three-value form, with both comparators pointing the same way, to test whether a value falls within an interval:

```
@container style(0 < --n < 10) {
  /* true when --n is greater than 0 and less than 10 */
}
@container style(100px > --width > 50px) {
  /* true when --width is less than 100px and greater than 50px */
}
```

In other words, `style(0 < --n < 10)` is equivalent to `style(0 < --n) and style(--n < 10)`. The middle value is tested against both bounds, rather than being chained left-to-right.

> **Note:** Plain and range syntax behave differently even when they look similar. Given `--n: calc(6/2)`, the query `style(--n: 3)` is **false** because the plain form compares the property's computed value (`calc(6/2)`) directly against `3`. The equivalent range query `style(--n = 3)` is **true** because the range form computes both sides numerically before comparing. See Plain versus range syntax in style queries in the container style queries guide for more details.

Scroll-state queries

See Using container scroll-state queries for scroll-state query examples.

Anchored queries

See Using anchored container queries for anchored query examples.

See also

- Using container queries
- Using container size and style queries
- Using container scroll-state queries
- Using anchored container queries
- `container-name`
- `container-type`
- `contain`
- `content-visibility`
- `CSSContainerRule` API
- CSS containment module
- CSS at-rule functions

@counter-style CSS at-rule

The **`@counter-style`** CSS at-rule lets you extend predefined list styles and define your own counter styles that are not part of the predefined set of styles. The `@counter-style` rule contains descriptors defining how the counter value is converted into a string representation.

While CSS provides many useful predefined counter styles, the `@counter-style` at-rule offers an open-ended method for creating counters. This at-rule caters to the needs of worldwide typography by allowing authors to define their own counter styles when the predefined styles don't fit their requirements.

Syntax

```
@counter-style thumbs {
  system: cyclic;
  symbols: "\1F44D";
  suffix: " ";
}
```

The `@counter-style` at-rule is identified by a counter style name, and the style of the named counter can be fine-tuned using a `<declaration-list>` consisting of one or more descriptors and their values.

Counter style name

`<counter-style-name>`

Provides a name for your counter style. It is specified as a case-sensitive `<custom-ident>` without quotes. The value should not be equal to `none`. Like all custom identifiers, the value of your counter style can't be a CSS-wide keyword. Avoid other enumerated CSS property values, including values of list and counter style properties. The name of your counter can't be the case-insensitive `list-style-type` property values of `decimal`, `disc`, `square`, `circle`, `disclosure-open`, and `disclosure-closed`.

> **Note:** The non-overridable counter style names `decimal`, `disc`, `square`, `circle`, `disclosure-open`, and `disclosure-closed` cannot be used as the name of a custom counter. However, they are valid in other contexts where the `<counter-style-name>` data type is expected, such as in `system: extends <counter-style-name>`.

Descriptors

`system`

Specifies the algorithm to be used for converting the integer value of a counter to a string representation. If the value is `cyclic`, `numeric`, `alphabetic`, `symbolic`, or `fixed`, the `symbols` descriptor must also be specified. If the value is `additive`, the `additive-symbols` descriptor must also be specified.

`symbols`

Specifies the symbols that are to be used for the marker representations. Symbols can contain strings, images, or custom identifiers. This descriptor is required if the `system` descriptor is set to `cyclic`, `numeric`, `alphabetic`, `symbolic`, or `fixed`.

`additive-symbols`

Defines the *additive tuples* for additive systems. While the symbols specified in the `symbols` descriptor are used for constructing marker representation by most algorithms, additive counter systems, such as Roman numerals, consist of a series of weighted symbols. The descriptors is a list of counter symbol along with their non-negative integer weights, listed by weight in descending order. This descriptor is required if the `system` descriptor is set to `additive`.

`negative`

Specifies to symbols to be appended or prepended to the counter representation if the value is negative.

`prefix`

Specifies a symbol that should be prepended to the marker representation. Prefixes are added to the representation in the final stage, before any characters added to negative counter values by the `negative` descriptor.

`suffix`

Specifies, similar to the prefix descriptor, a symbol that is appended to the marker representation. Suffixes come after the marker representation, including after any characters added to negative counter values by the `negative` descriptor.

`range`

Defines the range of values over which the counter style is applicable. If a counter style is used to represent a counter value outside of the ranges defined by this descriptor, the counter style will drop back to its `fallback` style.

`pad`

Is used when you need the marker representations to be of a minimum length. For example if you want the counters to start at 01 and go through 02, 03, 04, etc., then the `pad` descriptor is to be used. For representations larger than the specified `pad` value, the marker is constructed as normal.

`speak-as`

Describes how speech synthesizers, such as screen readers, should announce the counter style. For example, the value of the list item marker can be read out as numbers or alphabets for ordered lists or as audio cues for unordered lists, based on the value of this descriptor.

`fallback`

Specifies the counter name of the system to fall back to if either the specified system is unable to construct the representation of a counter value or if the counter value is outside the specified `range`. If the fallback counter also fails to represent the value, then that counter's fallback is used, if one is specified. If there are either no fallback counters described or if the chain of fallback systems are unable to represent a counter value, then it will ultimately fall back to the `decimal` style.

Formal syntax

```
@counter-style =
  @counter-style <counter-style-name> { <declaration-list> }

<counter-style-name> =
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying symbols with counter-style

```
@counter-style circled-alpha {
  system: fixed;
```

```
  symbols: Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ Ⓕ Ⓖ Ⓗ Ⓘ Ⓙ Ⓚ Ⓛ Ⓜ Ⓝ Ⓞ Ⓟ Ⓠ Ⓡ Ⓢ Ⓣ Ⓤ Ⓥ Ⓦ Ⓧ Ⓨ Ⓩ;
  suffix: " ";
}
```

The above counter style rule can be applied to lists like this:

```
.items {
  list-style: circled-alpha;
}
```

The above code produces the following result:

```
<ol class="items">
  <li>one</li>
  <li>two</li>
  <li>three</li>
  <li>four</li>
  <li>five</li>
</ol>
<p>...</p>
<ol class="items" start="25">
  <li>twenty-five</li>
  <li>twenty-six</li>
  <li>twenty-seven</li>
  <li>twenty-eight</li>
</ol>
```

See more examples on the demo page (code).

Ready-made counter styles

Find a collection of over 100 `counter-style` code snippets in the Ready-made Counter Styles document. This document provides counters that meet the needs of languages and cultures around the world.

The Counter styles converter pulls from this list to test and create copy and paste code for counter styles.

See also

- `counter()`
- `counters()`
- `symbols()`
- `list-style`, `list-style-image`, `list-style-position`, `list-style-type`
- CSS counter styles module

additive-symbols CSS at-rule descriptor

The **`additive-symbols`** descriptor of the `@counter-style` at-rule is used to specify counter symbols when the `@counter-style` `system` descriptor value is set as `additive`. The additive system is used to construct sign-value numbering systems such as Roman numerals.

Syntax

```
/* Single tuple */
additive-symbols: 3 "*";

/* Comma-separated list of tuples */
additive-symbols:
  3 "0",
  2 "\2E\20",
  1 url("symbol.png");

/* Binary counter */
additive-symbols:
  2 "1",
  1 "0";

/* Etruscan (a civilization in ancient Italy) counter  */
additive-symbols:
  100 "*",
  50 "↑",
  10 "X",
  5 "Λ",
  1 "|";
```

Values

The descriptor accepts a comma-separated list of *additive tuples* with each tuple consisting of the following two values separated by a space:

`<integer>`

A non-negative integer values specifying the integer weight of the associated symbol value of the tuple.

`<symbol>`

Specifies the counter symbol to be used for the weight value defined by the associated `<integer>` weight value of the tuple.

> **Note:** The additive tuples must be specified in order of descending weight; otherwise, the descriptor declaration isn't valid and is ignored.

Description

The `additive-symbols` descriptor defines a comma-separated list of *additive tuples*. Each *additive tuple* contains a space-separated non-negative integer and counter symbol. To be valid, the list must be in the descending order of integer. The integer and symbol are concatenated together to form the counter symbol.

When the `system` descriptor value is `cyclic`, `numeric`, `alphabetic`, `symbolic`, or `fixed`, use the `symbols()` descriptor instead of `additive-symbols`.

Formal definition

Related at-rule	`@counter-style`

Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
additive-symbols =
  [ <integer [0,∞]> && <symbol> ]#

<integer> =
  <number-token>

<symbol> =
  <string>        |
  <image>         |
  <custom-ident>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Values and Units Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying additive symbols

HTML

In this example, `system: additive` along with the `additive-symbols` descriptor values specify how numbers should be represented as Roman numerals. The value of each `<li>` element in the list is converted to a Roman numeral according to the rules defined in `@counter-style`.

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li value="109">109</li>
  <li>110</li>
</ul>
```

CSS

```
@counter-style uppercase-roman {
  system: additive;
  additive-symbols:
    1000 M,
    900 CM,
    500 D,
    400 CD,
    100 C,
    90 XC,
    50 L,
    40 XL,
    10 X,
    9 IX,
    5 V,
    4 IV,
    1 I;
}

ul {
  list-style: uppercase-roman;
  padding-left: 5em;
}
```

Result

For the list item with the value of `109`, the numeral `C` represents `100`, and `IX` represents `9`. This generates `CIX` counter for the list item `109`. The next list item automatically gets a value of `110`. The roman numeral `CX` is derived from `C` for `100` and `X` for `10`.

See also

- `@counter-style` descriptors: `system`, `symbols`, `negative`, `prefix`, `suffix`, `range`, `pad`, `speak-as`, `fallback`
- List style properties: `list-style`, `list-style-image`, `list-style-position`
- `symbols()` function to create anonymous counter styles
- CSS counter styles module

fallback CSS at-rule descriptor

The **`fallback`** descriptor of the `@counter-style` at-rule can be used to specify a counter style to fall back to if the counter style being defined cannot create a marker representation for a particular counter value.

Syntax

```
/* Keyword values */
fallback: lower-alpha;
fallback: custom-gangnam-style;
```

Value

The descriptor takes a single `<counter-style-name>` as its value:

`<counter-style-name>`

> Provides the name of the counter style to be used as the fallback, which is either the case-sensitive `<custom-ident>` of a custom CSS counter style (without quotes) or a case-insensitive `list-style-type` property value such as `decimal`, `disc`, and so on.

If omitted, the counter fallback defaults to `decimal`.

Description

The counter style provided as the value of the `fallback` descriptor is used when a `range` descriptor is specified for a counter style; the `fallback` style is used to represent all the values that fall outside the range. The `fallback` style is also used when the `fixed` `system` is used and there are not enough symbols to cover all the list items; the `fallback` style is used to represent all the values beyond the scope of the fixed system. In both these cases, and any other time the defined counter cannot create a specific counter value, the `fallback` style is used.

If the specified fallback style is also unable to construct a representation, then the `fallback` value of that fallback counter is used. If that fallback style's fallback is also unable to construct a representation, then that fallback's fallback is used. This falling back continues until a fallback is found that can construct the counter-representation. If no fallback `fallback` value is able to construct a representation—if a fallback style doesn't have a `fallback` value set, or if a `fallback` value is not specified or invalid—the `fallback` defaults to `decimal`.

Formal definition

Related at-rule	`@counter-style`
Initial value	`decimal`
Computed value	as specified

Formal syntax

```
fallback =
  <counter-style-name>

<counter-style-name> =
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying a fallback counter style

HTML

```
<ul class="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
</ul>
```

CSS

```
@counter-style fallback-example {
  system: fixed;
  symbols: "\24B6" "\24B7" "\24B8";
  fallback: upper-alpha;
}

.list {
  list-style: fallback-example;
}
```

Result

See also

- Other `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `negative`, `prefix`, `suffix`, `range`, `pad`, and `speak-as`
- `list-style`, `list-style-image`, `list-style-position`
- `symbols()`: the functional notation for creating anonymous counter styles

negative CSS at-rule descriptor

The **`negative`** descriptor of the `@counter-style` at-rule lets you define how negative counter values are represented when defining custom counter styles. The value of the `negative` descriptor defines the symbols to be added before and after the counter representation when the counter's value is negative.

Syntax

```
/* One <symbol> value */
negative: "--"; /* Adds '--' before if counter value is negative */

/* Two <symbol> values */
negative: "(" ")"; /* Adds '(- before and ')' after if counter value is negative */
```

Values

The `negative` descriptor accepts up to two `<symbol>` values.

`<symbol>`

> If only one value is specified, it is added before the counter representation when the counter is negative. If two values are specified, the first one is added before and the second one is added after the counter representation when the counter is negative.

Description

If the counter value is negative, the specified `<symbol>` for the `negative` descriptor is added before the counter representation, replacing the default `-` for negative values. The second `<symbol>`, if specified, is added after the counter representation.

The `negative` descriptor is relevant in two cases: if counter styles have the `system` value of `symbolic`, `alphabetic`, `numeric`, and `additive` and the count is negative; and if `system` value is `extends` and the extended counter style itself uses a negative sign. For systems that don't support negative counter values, specifying the `negative` descriptor has no effect and is ignored.

Formal definition

Related at-rule	`@counter-style`
Initial value	`"-" hyphen-minus`
Computed value	as specified

Formal syntax

```
negative =
  <symbol> <symbol>?

<symbol> =
  <string>        |
  <image>         |
  <custom-ident>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rendering negative counters

This example extends the `decimal` list style. The `negative` descriptor is used to add `(-` and `)` before and after negative counter values.

HTML

```
<ol start="-3">
  <li>Negative three</li>
  <li>Negative two</li>
  <li>Negative one</li>
  <li>Zero</li>
  <li>One</li>
</ol>
```

CSS

```
@counter-style neg {
  system: extends decimal;
```

```
  negative: "(-" ")";
  suffix: ": ";
}

ol {
  list-style: neg;
}
```

Result

The prefix and suffix listed as the value of the `negative` descriptor are only added to the marker when the counter value is less than zero.

See also

- `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `prefix`, `suffix`, `range`, `pad`, `speak-as`, `fallback`
- List style properties: `list-style`, `list-style-image`, `list-style-position`
- `symbols()` function to create anonymous counter styles
- CSS counter styles module
- CSS lists and counters module

pad CSS at-rule descriptor

The **`pad`** descriptor of the `@counter-style` at-rule is used to set a minimum length for marker representations.

Syntax

```
pad: 3 "0";
pad: "+" 5;
```

Values

The descriptor accepts the following two values, which are separated by a space and can be specified in any order:

`<integer>`

Specifies the minimum length that all marker representations must reach. The value must be non-negative. In the case of the `pad` descriptor, this value is also known as the *pad length*.

`<symbol>`

Specifies the symbol to be used for padding if the minimum length defined by the `<integer>` is not reached. In the case of the `pad` descriptor, this value is also known as the *padding symbol*.

Description

Use the `pad` descriptor when you need the marker representations to be of a minimum length. If a marker representation is shorter than the specified pad length, then the marker representation will be padded with the specified padding symbol. Marker representations longer than the pad length are displayed without any additional padding.

The `pad` descriptor takes an `<integer>` for the minimum marker length and a `<symbol>` for the padding. A common use case of the `pad` descriptor is when you need a list to start numbering from `01` and go through `02`, `03`, `04`, and so on, instead of just `1`, `2`, `3`, and `4`. By specifying the `pad` descriptor as `pad: 2 "0"` in this case, the browser ensures to make the counter at least two characters long and adds a padding with `0` to reach the minimum two-character length where needed. Counters that are already two or more characters long in this example will be displayed normally, without padding.

Formal definition

Related at-rule	`@counter-style`
Initial value	`0 ""`
Computed value	as specified

Formal syntax

```
pad =
  <integer [0,∞]>  &&
  <symbol>

<integer> =
  <number-token>

<symbol> =
  <string>        |
  <image>         |
  <custom-ident>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?
```

```
<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Values and Units Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Padding a counter

This example extends the `decimal` `system` to create a counter that is at least three characters long, padding shorter counters with `0` to reach that minimum length. A `suffix` descriptor has been added to make the output more legible.

HTML

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li value="40">Forty</li>
  <li>Forty-one</li>
  <li value="200">Two hundred</li>
  <li value="3000">Three thousand</li>
  <li>and so on</li>
</ul>
```

CSS

```
@counter-style pad-example {
  system: extends decimal;
  suffix: ": ";
  pad: 3 "0";
}

ul {
  list-style: pad-example;
}
```

Result

See also

- `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `negative`, `prefix`, `suffix`, `range`, `speak-as`, `fallback`
- List style properties: `list-style`, `list-style-image`, `list-style-position`
- `symbols()` function to create anonymous counter styles
- CSS counter styles module
- CSS lists and counters module

prefix CSS at-rule descriptor

The `@counter-style` rule's **`prefix`** descriptor specifies content that will be added to the beginning of the counter's marker representation.

When the counter value is negative, the `prefix` comes before the negative sign and any other `<symbol>`s added by the `negative` descriptor.

Syntax

```
/* <symbol> value: string, image, or identifier */
prefix: "»";
prefix: "Page ";
prefix: url("bullet.png");
```

Values

The **`prefix`** descriptor takes as its value a single `<symbol>`:

`<symbol>`

> Specifies a `<symbol>` — a `<string>`, `<image>`, or `<custom-ident>` — that is prepended to the marker representation.

Formal definition

Related at-rule	`@counter-style`
Initial value	"" (the empty string)
Computed value	as specified

Formal syntax

```
prefix =
  <symbol>

<symbol> =
  <string>        |
  <image>         |
  <custom-ident>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )
```

```
<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adding a prefix to a counter

In this example, each counter number is prefixed by "Book " (with a space) and followed by a colon (`:`). The colon is added with the `suffix` descriptor.

HTML

```
<ol class="books">
  <li>Flamer, by Mike Curato</li>
  <li>Gender Queer: A Memoir, by Maia Kobabe</li>
  <li>Tricks, by Ellen Hopkins</li>
  <li>The Handmaid's Tale: The Graphic Novel, by Margaret Atwood</li>
  <li>Crank, by Ellen Hopkins</li>
</ol>
```

CSS

```
@counter-style books {
  system: numeric;
  symbols: "0" "1" "2" "3" "4" "5" "6" "7" "8" "9";
  prefix: "Book ";
  suffix: ": ";
}

.books {
  list-style: books;
  padding-left: 15ch;
}
```

Result

See also

- Other `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `negative`, `suffix`, `range`, `pad`, `speak-as`, and `fallback`
- `list-style`, `list-style-image`, `list-style-position`
- `symbols()`: the functional notation for creating anonymous counter styles
- CSS counter styles module
- CSS lists and counters module

range CSS at-rule descriptor

The **`range`** descriptor lets the author specify one or more ranges of counter values for which the style is applied when defining custom counter styles with the `@counter-style` at-rule. When the `range` descriptor is included, the defined counter will only be used for values in the set ranges. If the counter value is outside the specified range, the fallback style will be used to construct the representation of that marker.

Syntax

```
/* Keyword value */
range: auto;

/* Range values */
range: 2 5;
range: infinite 10;
range: 6 infinite;
range: infinite infinite;

/* Multiple range values */
range:
  2 5,
  8 10;
range:
  infinite 6,
  10 infinite;
```

Values

The value is a comma-separated list of ranges each including a lower and upper limit or the keyword `auto`.

`auto`

The entire set of numbers representable by the counter `system`. Those range values depends on the counter system:

- For `cyclic`, `numeric`, and `fixed` systems, the range is negative `infinity` to positive `infinity`.
- For `alphabetic` and `symbolic` systems, the range is `1` to positive `infinity`.
- For `additive` systems, the range is `0` to `positive` infinity.
- When using `extend` to extend a system, the range is whatever `auto` would produce for the system being extended, including extensions of complex predefined styles, such as some Japanese, Korean, Chinese, and Ethiopian counter styles.

```
[ [ <integer> | infinite ]{2} ]#
```

Each range within the comma separated list of ranges includes two values, each being either an `<integer>` or the keyword `infinite`. If `infinite` is used as the first value in a range, it represents negative infinity; if it is used as the second value, it represents positive infinity. The first value of each range is the lower bound for the range and the second value is the upper bound, inclusive. If the lower bound of any range in the list is higher than the upper bound, the entire `range` descriptor is invalid and will be ignored.

Description

The value of the `range` descriptor can be either `auto` or a comma separated list of lower and upper bound ranges specified using negative or positive integers or the keyword `infinite`.

Understanding <code>auto</code>

When the value is set to `auto`, the range is the default range for the counter system. If the `system` is `cyclic`, `numeric`, or `fixed`, the range will be from negative infinity to positive infinity. If the `system` is `alphabetic` or `symbolic`, the range will be from `1` to positive `infinity`. For `system: additive`, `auto` results in the range `0` to positive `infinity`.

When extending a counter, if `range` is set to `auto`, the range value will be the range of the `system` of the counter that is being extended, not the `range` value, if any, of that counter. For example, if counter "B" has the `system: extends A` set, with counter being an `alphabetic` counter, setting `range: auto` on "B" sets the range of "B" from `1` to `infinity`. This is the range of the `alphabetic` system, not necessarily the range set in the "A" counter style definition. With `range: auto` set on "B", the `range` is set to the default range of the `alphabetic` system, not the `range` value set in counter A's descriptor list.

<code>infinite</code> explained

When range is specified as integers (versus `auto`), the value `infinite` can be used to denote infinity. If *infinite* is specified as the first value in a range, then it is interpreted as negative infinity. If used as the upper bound, the second value in the range pair, it is taken as positive infinity.

List of ranges

The value of `range` is either `auto`, discussed above, or a comma separated list of one or more ranges. The range of the counter style is the union of all the ranges defined in the list.

Each range in the list of ranges takes two values. Those values are either an `<integer>` or the keyword `infinite`. The first value is the *lower bound*, inclusive. The second value is the *upper bound*, inclusive. For two integer values, the lower value must come first. If the lower bound of any range in the list is higher than the upper bound, the entire `range` descriptor is invalid and will be ignored. The `infinite` keyword will not invalidate the range, because the position of `infinite` determines its value; either negative or positive infinity based on whether it's the lower bound or upper bound, respectively.

Formal definition

Related at-rule	`@counter-style`
Initial value	`auto`
Computed value	as specified

Formal syntax

```
range =
  [ [ <integer> | infinite ]{2} ]#  |
  auto

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting counter style over a range

HTML

```
<ul class="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li>Six</li>
  <li>Seven</li>
  <li>Eight</li>
  <li>Nine</li>
  <li>Ten</li>
</ul>
```

CSS

```
@counter-style range-multi-example {
  system: cyclic;
  symbols: "\25A0" "\25A1";
  range:
    2 4,
    7 9;
}

.list {
  list-style: range-multi-example;
}
```

Result

The first range is the list of ranges includes 2, 3, and 4. The second includes 7, 8, and 9. The range is the union of these two ranges, or 2, 3, 4, 7, 8, and 9.

See also

- Other `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `negative`, `prefix`, `suffix`, `pad`, `speak-as`, and `fallback`
- `list-style`, `list-style-image`, `list-style-position`
- `symbols()`: the functional notation for creating anonymous counter styles.

- CSS counter styles module
- CSS lists and counters module

speak-as CSS at-rule descriptor

The **speak-as** descriptor specifies how a counter symbol constructed with a given `@counter-style` will be represented in the spoken form. For example, an author can specify a counter symbol to be either spoken as its numerical value or just represented with an audio cue.

Syntax

```
/* Keyword values */
speak-as: auto;
speak-as: bullets;
speak-as: numbers;
speak-as: words;
speak-as: spell-out;

/* @counter-style name value */
speak-as: <counter-style-name>;
```

Values

`auto`

If the value of `speak-as` is specified as `auto`, then the effective value of `speak-as` will be determined based on the value of the `system` descriptor:

- If the value of `system` is `alphabetic`, the effective value of `speak-as` will be `spell-out`.
- If `system` is `cyclic`, the effective value of `speak-as` will be `bullets`.
- If `system` is `extends`, the value of `speak-as` will be the same as if `speak-as: auto` is specified on the extended style.
- For all other cases, specifying `auto` has the same effect as specifying `speak-as: numbers`.

`bullets`

A phrase or an audio cue defined by the user agent for representing an unordered list item will be read out.

`numbers`

The numerical value of the counter will be read out in the document language.

`words`

The user agent will generate a counter value as normal and read it out as a word in the document language.

`spell-out`

The user agent will generate a counter representation as normal and would read it out letter by letter. If the user agent doesn't know how to read out a particular counter symbol, the user agent might read it out as if the value of `speak-as` was `numbers`.

`<counter-style-name>`

The name of another counter style, specified as a `<custom-ident>`. If included, the counter will be spoken out in the form specified in that counter style, kind of like specifying the `fallback` descriptor. If the specified style does not exist, `speak-as` defaults to `auto`.

Accessibility

Assistive technology support is very limited for the `speak-as` property. Do not rely on it to convey information critical to understanding the page's purpose.

Let's Talk About Speech CSS | CSS Tricks (2017)

Formal definition

Related at-rule	`@counter-style`
Initial value	`auto`
Computed value	as specified

Formal syntax

```
speak-as =
  auto                |
  bullets             |
  numbers             |
  words               |
  spell-out           |
  <counter-style-name>

<counter-style-name> =
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting the spoken form for a counter

In this example, the counter system is fixed with unintelligible symbols used for the visual markers. However, the `speak-as` descriptor is used to set the list item markers as numbers in the accessibility tree. When supported, numbers rather than visual markers will be read out by screen readers.

To experience the result of the `speak-as` descriptor, use assistive technology such as VoiceOver or another screen reader or view the accessibility panel in the developer tools of a browser that supports `speak-as`.

HTML

```
<ul class="list">
  <li>I had one apple</li>
  <li>I ate two bananas</li>
  <li>I devoured three oranges</li>
  <li>I am not hungry for dinner</li>
  <li>But I'll have five scoops of ice cream for dessert</li>
</ul>
```

CSS

```
@counter-style speak-as-example {
  system: fixed;
  symbols: 🅐 🅑 🅒 🅓 🅔;
  suffix: " ";
  speak-as: numbers;
}

.list {
  list-style: speak-as-example;
}
```

Result

See also

- Other `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `negative`, `prefix`, `suffix`, `range`, `pad`, and `fallback`
- `list-style`, `list-style-image`, `list-style-position`
- `symbols()`: the functional notation for creating anonymous counter styles.
- CSS counter styles module
- CSS lists and counters module

suffix CSS at-rule descriptor

The **`suffix`** descriptor of the `@counter-style` rule specifies content that will be added to the end of the marker representation.

Syntax

```
/* <symbol> value: string, image, or identifier  */
suffix: "";
suffix: ") ";
suffix: url("bullet.png");
```

Values

The **`suffix`** descriptor takes as its value a single `<symbol>`:

`<symbol>`

Specifies a `<symbol>` that is appended to the marker representation. It may be a `<string>`, `<image>`, or `<custom-ident>`.

Formal definition

Related at-rule	`@counter-style`
Initial value	". " (full stop followed by a space)
Computed value	as specified

Formal syntax

```
suffix =
  <symbol>

<symbol> =
  <string>        |
  <image>         |
  <custom-ident>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a suffix for a counter

HTML

```
<ul class="choices">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>None of the above</li>
</ul>
```

CSS

```
@counter-style options {
  system: fixed;
  symbols: A B C D;
  suffix: ") ";
}

.choices {
  list-style: options;
}
```

Result

See also

- Other `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `negative`, `prefix`, `range`, `pad`, `speak-as`, and `fallback`
- `list-style`, `list-style-image`, `list-style-position`
- `symbols()`: the functional notation for creating anonymous counter styles
- CSS counter styles module
- CSS lists and counters module

symbols CSS at-rule descriptor

The **`symbols`** CSS descriptor of the `@counter-style` at-rule is used to specify the symbols for creating counter representations in the specified counter system. Specifying this descriptor is mandatory when the value of the `system` descriptor is `cyclic`, `numeric`, `alphabetic`, `symbolic`, or `fixed`.

Syntax

```
symbols: A B C D E;
symbols: "\24B6" "\24B7" "\24B8" D E;
symbols: "0" "1" "2" "4" "5" "6" "7" "8" "9";
symbols: url("one.svg") url("two.svg") url("three.svg");
symbols: indic-numbers;
```

Values

The `symbols` descriptor is specified as a list of one or more space-separated `<symbol>` values.

`<symbol>`

Specifies the symbol to use within the counter system. Each symbol in the list can be either a `<string>`, an `<image>`, or a `<custom-ident>`. The `<image>` value can, in turn, be specified as a `<url>` or `<gradient>`.

> **Note:** When using an identifier for a symbol, note that ASCII non-letters such as `*`, `"`, and `\` are not considered identifiers. They must be either quoted as a string or escaped.

Description

A symbol can be a string, image, or identifier. It is used within the `@counter-style` at-rule.

When the value of the `system` descriptor is `cyclic`, `numeric`, `alphabetic`, `symbolic`, or `fixed`, the `symbols` descriptor must be specified. For the `additive` system, use the `additive-symbols` descriptor instead to specify the symbols.

While a space between quoted symbols is not required, it makes CSS more readable. To use a quote as a symbol, either escape the quote character or enclose the character within different quotes, such as `"'"`.

When defining symbols with identifiers instead of strings, be sure to use identifier syntax rules. For example, as noted above, ASCII non-letters such as `*` are not identifiers and must be either quoted or escaped. Hex escape characters are followed by a space. This space may look like the space separating two identifiers, but it enables digits to follow hex-escaped characters. This means that two spaces must be included after a hex-escaped identifier to separate it from the next identifier. For example, it is better to use the string `"\2A 1"` instead of `\2A 1` with two spaces, as your code tools might remove double spaces. It is generally safer to quote identifiers that need to be escaped or use strings.

Formal definition

Related at-rule	`@counter-style`
Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
symbols =
  <symbol>+

<symbol> =
  <string>        |
  <image>         |
  <custom-ident>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
```

```
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting counter symbols

In this example, the list of values for the `symbols` descriptor include letters (`A`, `D`, `E`), a number within quotes (`"1"`), and a hex-escape identifier within quotes (`"\24B7"`) for the character Ⓑ.

HTML

```
<ul class="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
</ul>
```

CSS

```
@counter-style symbols-example {
  system: fixed;
  symbols: A "1" "\24B7" D E;
}

.list {
  list-style: symbols-example;
}
```

Result

See also

- `@counter-style` descriptors: `system`, `additive-symbols`, `negative`, `prefix`, `suffix`, `range`, `pad`, `speak-as`, `fallback`
- List style properties: `list-style`, `list-style-image`, `list-style-position`
- `symbols()` function
- `<url>` type
- CSS counter styles module

system CSS at-rule descriptor

The **`system`** descriptor specifies the algorithm to be used for converting the integer value of a counter to a string representation. It is used in a `@counter-style` to define the behavior of the defined style.

If the algorithm specified in the `system` descriptor is unable to construct the representation for a particular counter value, then that value's representation will be constructed using the fallback system provided.

Syntax

```
/* Keyword values */
system: cyclic;
system: numeric;
system: alphabetic;
system: symbolic;
system: additive;
system: fixed;

/* Other values */
system: fixed 3;
system: extends decimal;
system: extends circled-letters;
```

Values

This may take one of three forms:

- One of the keyword values `cyclic`, `numeric`, `alphabetic`, `symbolic`, `additive`, or `fixed`.
- The keyword value `fixed` along with an integer.
- The keyword value `extends` along with a `<counter-style-name>` value.

The values include:

`cyclic`

Cycles through the list of symbols provided in the `symbols` descriptor. Once the end of the list is reached, the cycle will loop back to the beginning and start over. This value is useful both for basic bullet styles with just one symbol and for styles with multiple symbols. At least one symbol must be specified in the `symbols` descriptor, otherwise the counter style is not valid.

`numeric`

Interprets the counter symbols as digits in a place-value numbering system. The numeric system is similar to the `alphabetic` system, described above. The main difference is that in the `alphabetic` system, the first counter symbol given in the `symbols` descriptor is interpreted as `1`, the next as `2`, and so on. However, in the numeric system, the first counter symbol is interpreted as 0, the next as `1`, then `2`, and so on.

At least two counter symbols must be specified in the `symbols` descriptor or the counter style is not valid.

`alphabetic`

Interprets the specified symbols as digits, to an alphabetic numbering system. If the characters `"a"` to `"z"` are specified as symbols in a counter style, with the `alphabetic` system, then the first 26 counter representations will be `"a"`, `"b"` up to `"z"`. Until this point, the behavior is the same as that of the `symbolic` system, described above. However, after `"z"`, it will continue as `"aa"`, `"ab"`, `"ac"`...

The `symbols` descriptor must contain at least two symbols or the counter style is not valid. The first counter symbol provided in the `symbols` descriptor is interpreted as `1`, the next as `2`, and so on. This system is also defined strictly over positive counter values.

`symbolic`

Cycles through the symbols provided in the `symbols` descriptor list repeatedly, doubling, tripling, and so on, the symbols on each successive pass through the list. For example, if two symbols " ▪ " and " ■ " are specified in the `symbols` descriptor, on each successive pass, they will become " ▪ ▪ " and " ■ ■ ", then " ▪ ▪ ▪ " and " ■ ■ ■ ", and so on in subsequent passes. At least one symbol must be specified in the `symbols` descriptor, otherwise the counter style is not valid. This counter system works for positive counter values only.

`additive`

Used to represent "sign-value" numbering systems, such as Roman numerals, which rather than reuse digits in different positions to obtain different values, define additional digits for larger values. The value of a number in such a system can be found out by adding the digits in the number.

An additional descriptor called `additive-symbols` must be specified with at least one *additive tuple*, or else the counter style rule will not be valid. An additive tuple is similar to a composite counter symbol, which is made up of two parts: a normal counter symbol and a non-negative integer weight. The additive tuples must be specified in the descending order of their weights or the system is invalid.

`fixed` Or `fixed <integer>`

Defines a finite set of symbols, iterating once through the list of symbols provided in the `symbols` descriptor. Once the specified symbols have been iterated through, the fallback counter style is used. This keyword value is useful in cases where the counter style values are finite. At least one symbol must be specified in the `symbols` descriptor, otherwise the counter style is not valid. The `fixed` keyword can be followed by an optional `<integer>` value. If specified, the `<integer>` value indicates the item in the list that will get the first symbol from the list of symbols. If omitted, the default value of `integer` is `1`, which gives the first item in the list the first symbol.

`extends`

Extends the algorithm of another browser- or author-defined counter style by allowing the alteration of some aspects of the extended counter style. Any unspecified descriptors and their values are inherited from the extended counter style specified. If the counter style name specified with `extends` is not yet defined, the `decimal` counter style will be extended by default.

It must not contain a `symbols` or `additive-symbols` descriptor, otherwise the counter style rule will be invalid. If one or more counter styles definitions form a cycle with their `extends` values, the browser will treat all the participating counter styles as extending from the `decimal` style.

> **Note:** The `symbols` descriptor is required when the value is `cyclic`, `numeric`, `alphabetic`, `symbolic`, or `fixed`. The `additive-symbols` descriptor is required if the `additive` value is set.

Formal definition

Related at-rule	`@counter-style`
Initial value	`symbolic`
Computed value	as specified

Formal syntax

```
system =
  cyclic                          |
  numeric                         |
  alphabetic                      |
  symbolic                        |
  additive                        |
  [ fixed <integer>? ]            |
  [ extends <counter-style-name> ]

<integer> =
  <number-token>

<counter-style-name> =
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Cyclic counter

The `cyclic` value iterates through the list of symbols, repeating the list as necessary:

CSS

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li>Six</li>
</ul>
```

```
@counter-style fisheye {
  system: cyclic;
  symbols: ◉ ⓘ;
  suffix: ": ";
}
```

```
ul {
  list-style: fisheye;
}
```

Result

Fixed counter

The `fixed` value iterates through the list of symbols only once, starting the single cycle at the list item number indicated by the `integer` value:

CSS

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li>Six</li>
</ul>
```

```
@counter-style circled-digits {
  system: fixed 3;
  symbols: ① ② ③;
  suffix: ": ";
}

ul {
  list-style: circled-digits;
}
```

Result

Symbolic counter

The `symbolic` value loops through the list defined in the `symbols` descriptor, doubling and tripling the number of symbols for the second and third cycles through the list, respectively:

CSS

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li>Six</li>
  <li>Seven</li>
  <li>Eight</li>
</ul>
```

```
@counter-style abc {
  system: symbolic;
  symbols: a b c;
  suffix: ". ";
}

ul {
  list-style: abc;
}
```

Result

Alphabetic counter

CSS

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li>Six</li>
  <li>Seven</li>
  <li>Eight</li>
</ul>
```

```
@counter-style abc {
  system: alphabetic;
  symbols: a b c;
  suffix: ". ";
}

ul {
  list-style: abc;
}
```

Result

Numeric counter

The first symbol provided in the `symbols` descriptor is interpreted as `0` here.

CSS

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
```

```
  <li>Five</li>
  <li>Six</li>
  <li>Seven</li>
  <li>Eight</li>
</ul>
```

```
@counter-style abc {
  system: numeric;
  symbols: a b c;
  suffix: ". ";
}

ul {
  list-style: abc;
}
```

Result

Numeric counter with numeric symbols

As shown in the following example, if digits from `0` to `9` are specified as symbols, this counter style will render symbols same as the decimal counter style.

CSS

```
<ul class="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
  <li>Six</li>
  <li>Seven</li>
  <li>Eight</li>
  <li>Nine</li>
  <li>Ten</li>
</ul>
```

```
@counter-style numbers {
  system: numeric;
  symbols: "0" "1" "2" "3" "4" "5" "6" "7" "8" "9";
  suffix: ".";
}

ul {
  list-style: numbers;
}
```

Result

Additive counter

This example renders a list using Roman numerals. Notice that a `range` is specified. This is because the representation will produce correct Roman numerals only until the counter value of `3999`. Once outside of the range, the rest of the counter representations will be based on the `decimal` style, which is the fallback. If you need to represent counter values as Roman numerals, you could use either one of the predefined counter styles, `upper-roman` or `lower-roman`, rather than recreating the rule yourself.

HTML

We use the `start` attribute on the `<ol>` element to demonstrate that counting doesn't need to start at `1`. Additionally, we use the `value` attribute on the fifth `<li>` element to demonstrate that the counters you define using `@counter-style` behave just like native counters.

```
<ol start="48">
  <li>48</li>
  <li>49</li>
  <li>50</li>
  <li>51</li>
  <li value="109">109</li>
  <li>110</li>
</ol>
```

CSS

```
@counter-style uppercase-roman {
  system: additive;
  range: 1 3999;
  additive-symbols:
    1000 M,
    900 CM,
    500 D,
    400 CD,
    100 C,
    90 XC,
    50 L,
    40 XL,
    10 X,
    9 IX,
    5 V,
    4 IV,
    1 I;
}

ol {
  list-style: uppercase-roman;
  padding-left: 5em;
}
```

Result

Extending a counter

This example uses the algorithm, symbols, and other properties of `lower-alpha`, one of the several native `list-style-type` counter values, but extends it by removing the period (`'.'`) after the counter representation and enclosing the characters in parentheses, as in `(a)` and `(b)`.

HTML

```
<ul class="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
</ul>
```

CSS

```
@counter-style alpha-modified {
  system: extends lower-alpha;
  prefix: "(";
  suffix: ") ";
}

ul {
  list-style: alpha-modified;
}
```

Result

See also

- Other `@counter-style` descriptors, including `symbols`, `additive-symbols`, `negative`, `prefix`, `suffix`, `range`, `pad`, `speak-as`, and `fallback`
- `list-style`, `list-style-image`, `list-style-position`
- `symbols()`, the functional notation for creating anonymous counter styles.

@custom-media CSS at-rule

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`@custom-media`** CSS at-rule defines aliases for long or complex media queries. Instead of repeating the same hardcoded `<media-query-list>` in multiple `@media` at-rules, it can be defined once in a `@custom-media` at-rule and referenced throughout the stylesheet whenever needed.

Syntax

```
@custom-media <extension-name> [<media-query-list> | true | false ];

@custom-media --media-query-name (width < 1200px);
@custom-media --media-query-name (width < 1200px), (orientation: portrait);
```

Values

`<extension-name>`

A `<dashed-ident>`; the name identifying the custom media query.

Represented value

The value aliased by the custom media query. Possible values are:

`<media-query-list>`

A comma-separated list of `<media-query>` values.

`true`

The `@custom-media` value always evaluates to `true`.

`false`

The `@custom-media` value always evaluates to `false`.

Description

When building responsive interfaces, the same media condition often needs to be repeated across multiple `@media` at-rules, sometimes across different files and teams. Duplicating media queries increases the risk of mistakes, makes refactoring harder, and creates unnecessary maintenance overhead. Any time a media query changes, every instance must be found and updated manually — a process that can be both error-prone and difficult to track in large codebases.

The `@custom-media` at-rule solves this problem by letting you define **named aliases** for media queries. Instead of repeating the full media query everywhere, you declare the media condition once as a custom media query and reference its alias throughout your stylesheets. With this in place, updating the underlying media query requires a single change in one location.

Custom media queries can be composed from others by referencing their alias names inside the media query features. This enables building more expressive, layered conditions. However, a custom media query cannot refer to itself, nor can it form part of a circular chain of references. Any circular dependency — direct or indirect — invalidates all custom media queries involved in that loop.

If multiple `@custom-media` rules define the same `<dashed-ident>` name, the rule that is in scope at the time a `@media` rule is evaluated is used. Earlier references are not retroactively updated when a later `@custom-media` rule is declared.

Evaluating media queries with logical operators

Custom media queries accept the full range of CSS logical operators — `not`, `and`, and `or` (comma-separated or using the `or` keyword).

Because a `@custom-media` value is just a normal `<media-query-list>`, you can combine, invert, or group conditions exactly as you would in a regular `@media` rule.

Using the `not` operator

The `not` operator negates an entire media condition. This is useful when you want a rule to apply only when a specific condition is `false`.

```
@custom-media --no-script not (scripting);

@media (--no-script) {
}
```

Using the `and` operator

The `and` operator lets you combine multiple conditions that must all be `true`.

```
@custom-media --medium-screen (min-width: 40em) and (max-width: 60em);

@media (--medium-screen) {
}
```

This alias only matches when the viewport is within the specified width range.

Using the `or` operator

The logical `or` operator (or its comma alias, when combining media types) creates a media query that matches if any of the listed conditions are `true`.

```
@custom-media --screen-or-print screen, print;
@custom-media --narrow-or-tall (width < 600px) or (height > 800px);

@media (--screen-or-print) {
}

@media (--narrow-or-tall) {
}
```

The `--screen-or-print` alias uses a comma to match either the `screen` or `print` media type. The `--narrow-or-tall` alias uses the `or` keyword to combine two media features, matching when the viewport is narrower than 600px, taller than 800px, or both.

Formal syntax

```
@custom-media =
  @custom-media <extension-name> [ <media-query-list> | true | false ] ;
```

This syntax reflects the latest standard as per Media Queries Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Updating multiple media queries

In this example, the `@custom-media` at-rule is used on a responsive website that uses a particular breakpoint in several places:

```
@custom-media --narrow-window (width < 32em);

@media (--narrow-window) {
}

@media (--narrow-window) and (hover) {
}

@media (--narrow-window) and (orientation: portrait) {
}
```

If the breakpoint needs to be changed, it can be updated in one place to adjust all dependent media queries across the whole site.

Grouping multiple responsive breakpoints

Here, the `@custom-media` at-rule is used to set multiple breakpoints in a single place, improving maintainability and simplifying responsive design management across multiple stylesheets:

```
/* general.css */

@custom-media --mobile-screen (width < 480px);
@custom-media --tablet-screen (width < 768px);
@custom-media --laptop-screen (width < 1024px);
@custom-media --desktop-screen (width < 1440px);
@custom-media --widescreen (width > 1441px);
```

```
/* layout.css */

.container {
  padding: 1rem;
}

@media (--mobile-screen) {
  .container {
    padding: 0.5rem;
  }
}

@media (--laptop-screen) {
  .container {
    max-width: 1200px;
  }
}

@media (--widescreen) {
  .container {
    max-width: 1400px;
    padding: 2rem;
  }
}
```

```
/* typography.css */

@media (--tablet-screen) {
  .container {
```

```
    font-size: 0.9rem;
  }
}

@media (--laptop-screen) {
  .container {
    font-size: 1rem;
  }
}

@media (--widescreen) {
  .container {
    font-size: 1.1rem;
  }
}
```

Grouping all the breakpoints in a single location makes it easier to maintain the responsive design. When a breakpoint needs adjustment, it only requires a single update to the associated `@custom-media` definition, ensuring consistency across all stylesheets.

Using <code>true</code> and <code>false</code> keywords

The following example shows how the `true` and `false` keywords can be used with `@custom-media` to create media queries that always or never match, respectively.

```
@custom-media --enabled true;
@custom-media --disabled false;

@media (--enabled) {
  /* These styles always apply */
  body {
    background-color: blue;
  }
}

@media (--disabled) {
  /* These styles never apply */
  body {
    background-color: red;
  }
}
```

This can be useful for feature flags or conditional logic within stylesheets.

Overriding existing <code>@custom-media</code> rules

In this example, one `@custom-media` rule is overridden by another `@custom-media` rule using the same `<dashed-ident>` name.

```
@custom-media --mobile-breakpoint (width < 320px);

@media (--mobile-breakpoint) {
  .container {
    grid-template-columns: 2fr 1fr;
  }
}

@custom-media --mobile-breakpoint (width < 480px);
```

When multiple `@custom-media` rules use the same name, the rule that is in scope at the time a `@media` rule is evaluated is used. Earlier references are not retroactively updated when a later `@custom-media` rule is declared.

For example, in the code above, the `--mobile-breakpoint` reference inside the `@media` rule is evaluated as `(width < 320px)`, so the `.container` rule is only applied when the viewport is less than 320px wide, even though `--mobile-breakpoint` is redefined as `(width < 480px)` later in the stylesheet.

> **Note:** The overriding behavior of `@custom-media` is still under discussion in the CSS specification and may change in the future. See the Browser compatibility section for current support status.

See also

- CSS `@media` at-rule
- CSS `@import` at-rule
- Responsive design
- CSS media queries module

@document CSS at-rule

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`@document`** CSS at-rule restricts the style rules contained within it based on the URL of the document. It is designed primarily for user-defined style sheets (see userchrome.org for more information), though it can be used on author-defined style sheets, too.

Syntax

```
@document url("https://www.example.com/")
{
  h1 {
    color: green;
  }
}
```

An `@document` rule can specify one or more matching functions. If any of the functions apply to a given URL, the rule will take effect on that URL. The functions available are:

`url()`

Matches an exact URL.

`url-prefix()`

Matches if the document URL starts with the value provided.

`domain()`

Matches if the document URL is on the domain provided (or a subdomain of it).

`media-document()`

Matches the media according to the string in parameter, one of `video`, `image`, `plugin` or `all`.

`regexp()`

Matches if the document URL is matched by the regular expression provided. The expression must match the entire URL.

The values provided to the `url()`, `url-prefix()`, `domain()`, and `media-document()` functions can be optionally enclosed by single or double quotes. The values provided to the `regexp()` function *must* be enclosed in quotes.

Escaped values provided to the `regexp()` function must additionally be escaped from the CSS. For example, a `.` (period) matches any character in regular expressions. To match a literal period, you would first need to escape it using regular expression rules (to `\.`), then escape that string using CSS rules (to `\\.`).

`@document` is currently only supported in Firefox; if you wanted to replicate using such functionality in your own non-Firefox browser, you could try using this polyfill by @An-Error94, which uses a combination of a user script, data-* attributes, and attribute selectors.

> **Note:** There is a -moz-prefixed version of this property — `@-moz-document`. This has been limited to use only in user and UA sheets in Firefox 59 in Nightly and Beta — an experiment designed to mitigate potential CSS injection attacks (See Firefox bug 1035091).

Formal syntax

```
@document [ <url>                   |
            url-prefix(<string>)    |
            domain(<string>)        |
            media-document(<string>) |
            regexp(<string>)
          ]# {
  <group-rule-body>
}
```

Examples

Specifying document for CSS rule

```
@document url("http://www.w3.org/"),
          url-prefix("http://www.w3.org/Style/"),
          domain("mozilla.org"),
          media-document("video"),
          regexp("https:.*") {
  /* CSS rules here apply to:
     - The page "http://www.w3.org/"
     - Any page whose URL begins with "http://www.w3.org/Style/"
     - Any page whose URL's host is "mozilla.org"
       or ends with ".mozilla.org"
     - Any standalone video
     - Any page whose URL starts with "https:" */

  /* Make the above-mentioned pages really ugly */
  body {
    color: purple;
    background: yellow;
  }
}
```

Specifications

Initially in Level 3, `@document` was postponed to Level 4, but then subsequently removed.

See also

- Per-site user style sheet rules on the www-style mailing list.

@font-face CSS at-rule

The **`@font-face`** CSS at-rule specifies a custom font with which to display text; the font can be loaded from either a remote server or a locally-installed font on the user's own computer.

Syntax

```
@font-face {
  font-family: "Trickster";
  src:
    local("Trickster"),
    url("trickster-COLRv1.woff2") format("woff2") tech(color-COLRv1),
    url("trickster-outline.woff2") format("woff2");
}
```

Descriptors

`ascent-override`

Defines the ascent metric for the font.

`descent-override`

Defines the descent metric for the font.

`font-display`

Determines how a font face is displayed based on whether and when it is downloaded and ready to use.

`font-family`

Specifies a name that will be used as the font face value for font properties. A `font-family` name is required for the `@font-face` rule to be valid.

`font-stretch`

A `font-stretch` value. Accepts two values to specify a range that is supported by a font face, for example `font-stretch: 50% 200%;`

`font-style`

A `font-style` value. Accepts two values to specify a range that is supported by a font face, for example `font-style: oblique 20deg 50deg;`

`font-weight`

A `font-weight` value. Accepts two values to specify a range that is supported by a font face, for example `font-weight: 100 400;`

`font-feature-settings`

Allows control over advanced typographic features in OpenType fonts.

`font-variation-settings`

Allows low-level control over OpenType or TrueType font variations, by specifying the four-letter axis names of the features to vary, along with their variation values.

`font-width`

Allows authors to specify a normal, condensed, or expanded face for the specified fonts.

`line-gap-override`

Defines the line gap metric for the font.

`size-adjust`

Defines a multiplier for glyph outlines and metrics associated with this font. This makes it easier to harmonize the designs of various fonts when rendered at the same font size.

`src`

Specifies references to font resources including hints about the font format and technology. A `src` is required for the `@font-face` rule to be valid.

`unicode-range`

The range of Unicode code points to be used from the font.

Description

It's common to use both `url()` and `local()` together, so that the user's installed copy of the font is used if available, falling back to downloading a copy of the font if it's not found on the user's device.

If the `local()` function is provided, specifying a font name to look for on the user's device, and if the user agent finds a match, that local font is used. Otherwise, the font resource specified using the `url()` function is downloaded and used.

Browsers attempt to load resources in their list declaration order, so usually `local()` should be written before `url()`. Both functions are optional, so a rule block containing only one or more `local()` without `url()` is possible. If more specific fonts with `format()` or `tech()` values are desired, these should be listed *before* versions that don't have these values, as the less specific variant would otherwise be tried and used first.

For web delivery, it's generally best to serve fonts in WOFF2 format, because it compresses fonts more efficiently than older formats like WOFF or OpenType, reducing file size and improving load times. WOFF2 is also well supported in modern browsers, making it a safe default choice for most websites.

By allowing authors to provide their own fonts, `@font-face` makes it possible to design content without being limited to the so-called "web-safe" fonts (that is, the fonts that are so common that they're considered to be universally available). The ability to specify the name of a locally-installed font to look for and use makes it possible to customize the font beyond the basics while making it possible to do so without relying on an internet connection.

> **Note:** Fallback strategies for loading fonts on older browsers are described in the `src` descriptor page.

The `@font-face` at-rule may be used not only at the top level of a CSS, but also inside any CSS conditional-group at-rule.

Font MIME Types

Format	MIME type
TrueType	`font/ttf`
OpenType	`font/otf`
Web Open Font Format	`font/woff`
Web Open Font Format 2	`font/woff2`

Notes

- Web fonts are subject to the same domain restriction (font files must be on the same domain as the page using them), unless HTTP access controls are used to relax this restriction.
- `@font-face` cannot be declared within a CSS selector. For example, the following will not work:

```
.className {
  @font-face {
    font-family: "MyHelvetica";
    src:
      local("Helvetica Neue Bold"), local("HelveticaNeue-Bold"),
      url("MgOpenModernaBold.woff2");
    font-weight: bold;
  }
}
```

Formal syntax

```
@font-face =
  @font-face { <declaration-list> }
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying a downloadable font

This example specifies a downloadable font to use, applying it to the entire body of the document:

```
<body>
  This is Bitstream Vera Serif Bold.
</body>
```

```
@font-face {
  font-family: "Bitstream Vera Serif Bold";
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-Regular.woff2");
}

body {
  font-family: "Bitstream Vera Serif Bold", serif;
}
```

Specifying local font alternatives

In this example, the user's local copy of "Helvetica Neue Bold" is used; if the user does not have that font installed (both the full font name and the Postscript name are tried), then the downloadable font named "MgOpenModernaBold.woff2" is used instead:

```
@font-face {
  font-family: "MyHelvetica";
  src:
    local("Helvetica Neue Bold"), local("HelveticaNeue-Bold"),
    url("MgOpenModernaBold.woff2");
  font-weight: bold;
}
```

See also

- About WOFF
- Transfonter webfont generator
- Beautiful fonts with @font-face
- Font Library

ascent-override CSS at-rule descriptor

The **`ascent-override`** CSS descriptor for the `@font-face` at-rule defines the ascent metric for the font. The ascent metric is the height above the baseline that CSS uses to lay out line boxes in an inline formatting context.

Syntax

```
ascent-override: normal;
ascent-override: 90%;
```

Values

`normal`

The default value. When used the metric value is obtained from the font file.

`<percentage>`

A `<percentage>` value.

Formal definition

Related at-rule	`@font-face`
Initial value	`normal`
Percentages	as specified
Computed value	as specified

Formal syntax

```
ascent-override =
  [ normal | <percentage [0,∞]> ]{1,2}
```

This syntax reflects the latest standard as per CSS Fonts Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Overriding metrics of a fallback font

The `ascent-override` property can help when overriding the metrics of a fallback font to better match those of a primary web font.

```
@font-face {
  font-family: "web-font";
  src: url("https://example.com/font.woff");
}

@font-face {
  font-family: "local-font";
  src: local("Local Font");
  ascent-override: 125%;
}
```

See also

- `descent-override`
- `font-display`
- `font-family`

- font-weight
- font-style
- font-stretch
- font-feature-settings
- font-variation-settings
- line-gap-override
- src
- size-adjust
- unicode-range descriptor

descent-override CSS at-rule descriptor

The **descent-override** CSS descriptor for the @font-face at-rule defines the descent metric for the font. The descent metric is the height below the baseline that CSS uses to lay out line boxes in an inline formatting context.

Syntax

```
descent-override: normal;
descent-override: 90%;
```

Values

normal

The default value. When used the metric value is obtained from the font file.

<percentage>

A <percentage> value.

Formal definition

Related at-rule	@font-face
Initial value	normal
Percentages	as specified
Computed value	as specified

Formal syntax

```
descent-override =
  [ normal | <percentage [0,∞]> ]{1,2}
```

This syntax reflects the latest standard as per CSS Fonts Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Overriding metrics of a fallback font

The descent-override property can help when overriding the metrics of a fallback font to better match those of a primary web font.

```
@font-face {
  font-family: "web-font";
  src: url("https://example.com/font.woff");
}

@font-face {
  font-family: "local-font";
  src: local("Local Font");
  descent-override: 125%;
}
```

See also

- ascent-override
- font-display
- font-family
- font-weight
- font-style
- font-stretch
- font-feature-settings
- font-variation-settings
- line-gap-override
- src
- size-adjust
- unicode-range descriptor

font-display CSS at-rule descriptor

The **font-display** descriptor for the @font-face at-rule determines how a font face is displayed based on whether and when it is downloaded and ready to use.

Syntax

```
/* Keyword values */
font-display: auto;
font-display: block;
font-display: swap;
font-display: fallback;
font-display: optional;
```

Values

auto

The font display strategy is defined by the user agent.

`block`

Gives the font face a short block period and an infinite swap period.

`swap`

Gives the font face an extremely small block period and an infinite swap period.

`fallback`

Gives the font face an extremely small block period and a short swap period.

`optional`

Gives the font face an extremely small block period and no swap period.

> **Note:** In Firefox, the preferences `gfx.downloadable_fonts.fallback_delay` and `gfx.downloadable_fonts.fallback_delay_short` provide the duration of the "short" and "extremely small" periods, respectively.

Description

The font display timeline is based on a timer that begins the moment the user agent attempts to use a given downloaded font face. The timeline is divided into the three periods below which dictate the rendering behavior of any elements using the font face:

- Font block period: If the font face is not loaded, any element attempting to use it must render an *invisible* fallback font face. If the font face successfully loads during this period, it is used normally.
- Font swap period: If the font face is not loaded, any element attempting to use it must render a fallback font face. If the font face successfully loads during this period, it is used normally.
- Font failure period: If the font face is not loaded, the user agent treats it as a failed load causing normal font fallback.

Formal definition

Related at-rule	`@font-face`
Initial value	`auto`
Computed value	as specified

Formal syntax

```
font-display =
  auto      |
  block     |
  swap      |
  fallback  |
  optional
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying fallback font-display

```
@font-face {
  font-family: "ExampleFont";
  src: url("/path/to/fonts/example-font.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
  font-display: fallback;
}
```

See also

- `font-family`
- `font-stretch`
- `font-style`
- `font-weight`
- `font-feature-settings`
- `font-variation-settings`
- `src`
- `unicode-range`

font-family CSS at-rule descriptor

The **`font-family`** CSS descriptor sets the font family for a font specified in an `@font-face` at-rule.

The value is used for name matching against a particular `@font-face` when styling elements using the `font-family` property. Any name may be used, and this overrides any name specified in the underlying font data.

Syntax

```
/* <string> values */
font-family: "font family";
font-family: "another font family";

/* <custom-ident> value */
font-family: example-font;
```

Values

`<family-name>`

Specifies the name of the font family.

Formal definition

Related at-rule	`@font-face`
Initial value	`n/a (required)`

Computed value	as specified

Formal syntax

```
font-family =
  <font-family-name>

<font-family-name> =
  <string>         |
  <custom-ident>+
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting the font family name

```
@font-face {
  font-family: "Some font family";
  src: url("some_font_name.ttf");
}
```

See also

- font-display
- font-stretch
- font-style
- font-weight
- font-feature-settings
- font-variation-settings
- src
- unicode-range

font-feature-settings CSS at-rule descriptor

The **font-feature-settings** CSS descriptor allows you to define the initial settings to use for the font defined by the @font-face at-rule. You can further use this descriptor to control typographic font features such as ligatures, small caps, and swashes, for the font defined by @font-face. The values for this descriptor are the same as the font-feature-settings property, except for the global keyword values.

Since this descriptor sets feature values on the font object in the @font-face at-rule and not on an entire element, only some glyphs in an element may be rendered using this descriptor.

Syntax

```
/* Use the default settings */
font-feature-settings: normal;

/* Set values for OpenType feature tags */
font-feature-settings: "smcp";
font-feature-settings: "smcp" on;
font-feature-settings: "swsh" 2;
```

Values

This descriptor is specified as either the keyword normal or as a comma-separated list of <feature-tag-value> values. When rendering text, the list of OpenType <feature-tag-value> values are passed to the text layout engine to enable or disable font features.

normal

Indicates that text is laid out using default font settings. This is the default value.

<feature-tag-value>

Represents a space-separated tuple consisting of a tag name and an optional value.

The tag name is always a <string> of four ASCII characters. If the tag name has more or fewer characters or if it contains characters outside the U+20 – U+7E code point range, the descriptor is invalid.

The optional value can be a positive integer or the keyword on or off. The keywords on and off are synonyms for the values 1 and 0, respectively. If no value is set, the default is 1. For non-boolean OpenType features (e.g., stylistic alternates), the value implies a particular glyph to be selected; for boolean features, the value turns the feature on or off.

Formal definition

Related at-rule	@font-face
Initial value	normal
Computed value	as specified

Formal syntax

```
font-feature-settings =
  normal               |
  <feature-tag-value>#

<feature-tag-value> =
  <opentype-tag> [ <integer [0,∞]> | on | off ]?

<opentype-tag> =
  <string>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Enabling swash glyphs using the @font-face at-rule

In this example, the tag name swsh and a boolean value 1 are used as the value for the font-feature-settings descriptor in the @font-face at-rule.

HTML

```
<p class="swash-off">Swash is off here</p>
<p class="swash-on">Swash is on here</p>
```

CSS

```
@font-face {
  font-family: "MonteCarlo";
  src: url("/shared-assets/fonts/monte-carlo/monte-carlo-regular.woff2");
}
@font-face {
  font-family: "MonteCarlo2";
  src: url("/shared-assets/fonts/monte-carlo/monte-carlo-regular.woff2");
  font-feature-settings: "swsh" 1;
}
p {
  font-size: 3rem;
  margin: 0.7rem 3rem;
}
.swash-off {
  font-family: "MonteCarlo", cursive;
}
.swash-on {
  font-family: "MonteCarlo2", cursive;
}
```

Result

Line 1 shows the default ornate design of the MonteCarlo font, and line 2 shows the default glyphs being replaced with swash glyphs.

See also

- Other `@font-face` descriptors: `font-family`, `font-style`, `font-variation-settings`, `font-weight`, `src`
- Related font properties: `font-feature-settings`, `font-variant-alternates`, `font-variation-settings`

font-stretch CSS at-rule descriptor

> **Note:** The `font-stretch` descriptor was renamed to `font-width` in the CSS Fonts specification. To preserve compatibility, the specification retains `font-stretch` as a legacy alias for the `font-width` descriptor.

The **`font-stretch`** CSS descriptor allows authors to specify a normal, condensed, or expanded face for the fonts specified in the `@font-face` at-rule.

For a particular font family, authors can download various font faces corresponding to the different styles of the same font family, and then use the `font-stretch` descriptor to specify the font face's stretch explicitly.

Syntax

```
/* Single values */
font-stretch: ultra-condensed;
font-stretch: extra-condensed;
font-stretch: condensed;
font-stretch: semi-condensed;
font-stretch: normal;
font-stretch: semi-expanded;
font-stretch: expanded;
font-stretch: extra-expanded;
font-stretch: ultra-expanded;
font-stretch: 50%;
font-stretch: 100%;
font-stretch: 200%;

/* Multiple values */
font-stretch: 75% 125%;
font-stretch: condensed ultra-condensed;
```

The `font-stretch` descriptor can take a single value from the list below.

Values

`normal`

Specifies a normally condensed font face.

`semi-condensed, condensed, extra-condensed, ultra-condensed`

Specifies a more condensed font face than normal, with ultra-condensed being the most condensed.

`semi-expanded, expanded, extra-expanded, ultra-expanded`

Specifies a more expanded font face than normal, with ultra-expanded being the most expanded.

`<percentage>`

A `<percentage>` value between 50% and 200% (inclusive). Negative values are not allowed for this descriptor.

In earlier versions of the `font-stretch` specification, the descriptor accepts only the nine keyword values. CSS Fonts Level 4 extends the syntax to accept a `<percentage>` value as well. This enables variable fonts to offer a continuous variation of character widths. For TrueType or OpenType variable fonts, the `wdth` variation is used to implement varying widths.

If the font does not provide a face that exactly matches the given value, then values less than `100%` map to a condensed face, and values greater than or equal to `100%` map to an expanded face.

Keyword to numeric mapping

The table below shows the mapping between keyword values and numeric percentages:

Keyword	Percentage
`ultra-condensed`	50%
`extra-condensed`	62.5%

Keyword	Percentage
`condensed`	75%
`semi-condensed`	87.5%
`normal`	100%
`semi-expanded`	112.5%
`expanded`	125%
`extra-expanded`	150%
`ultra-expanded`	200%

Variable fonts

Most fonts have a particular width that corresponds to one of the keyterm values. However, variable fonts can support a range of widths with fine granularity, giving the designer a greater degree of control over the chosen weight. For this, percentage ranges are useful.

For TrueType or OpenType variable fonts, the `wdth` variation is used to implement varying glyph widths.

Accessibility

People with dyslexia and other cognitive conditions may have difficulty reading fonts that are too condensed, especially if the font has a low color contrast ratio.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | W3C Understanding WCAG 2.0

Formal definition

Related at-rule	`@font-face`
Initial value	`normal`
Computed value	as specified

Formal syntax

```
font-width =
  auto                |
  <'font-width'>{1,2}

<font-width> =
  normal              |
  <percentage [0,∞]>  |
  ultra-condensed     |
  extra-condensed     |
  condensed           |
  semi-condensed      |
  semi-expanded       |
  expanded            |
  extra-expanded      |
  ultra-expanded
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a percentage range for font-stretch

The following example uses the League Mono font. It synthesizes different font families from the same font file using the `font-stretch` descriptor with different keywords and percentages.

```
<p>League Mono</p>
<p>League Mono</p>
<p>League Mono</p>
```

```
@font-face {
  font-family: "League Mono Ultra Condensed";
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-stretch: ultra-condensed; /* same as 50% */
}

@font-face {
  font-family: "League Mono Normal";
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-stretch: 100%; /* same as normal */
}

@font-face {
  font-family: "League Mono Ultra Expanded";
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-stretch: ultra-expanded; /* same as 200% */
}

p:nth-child(1) {
  font-family: "League Mono Ultra Condensed", sans-serif;
}

p:nth-child(2) {
  font-family: "League Mono Normal", sans-serif;
}

p:nth-child(3) {
  font-family: "League Mono Ultra Expanded", sans-serif;
}
```

See also

- Modern `font-width` descriptor, replacing `font-stretch`
- `font-display` descriptor
- `font-family` descriptor

- `font-weight` descriptor
- `font-style` descriptor
- `font-feature-settings` descriptor
- `font-variation-settings` descriptor
- `src` descriptor
- `unicode-range` descriptor

font-style CSS at-rule descriptor

The **`font-style`** CSS descriptor allows authors to specify font styles for the fonts specified in the `@font-face` at-rule.

For a particular font family, authors can download various font faces that correspond to the different styles of the same font family and then use the `font-style` descriptor to explicitly specify the font face's style. The values for this CSS descriptor are the same as that of the corresponding `font-style` property.

Syntax

```
font-style: normal;
font-style: italic;
font-style: oblique;
font-style: oblique 30deg;
font-style: oblique 30deg 50deg;
```

Values

`normal`

Selects the normal version of the font-family.

`italic`

Specifies that font-face is an italicized version of the normal font.

`oblique`

Specifies that the font-face is an artificially sloped version of the normal font.

`oblique` with angle

Selects a font classified as `oblique`, and additionally specifies an angle for the slant of the text.

`oblique` with angle range

Selects a font classified as `oblique`, and additionally specifies a range of allowable angle for the slant of the text. Note that a range is only supported when the `font-style` is `oblique`; for `font-style: normal` or `italic`, no second value is allowed.

Formal definition

Related at-rule	`@font-face`
Initial value	`normal`
Computed value	as specified

Formal syntax

```
font-style =
  auto                                  |
  normal                                |
  italic                                |
  left                                  |
  right                                 |
  oblique [ <angle [-90deg,90deg]>{1,2} ]?
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

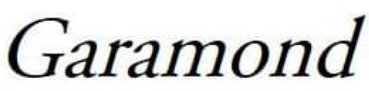
Examples

Specifying an italic font style

As an example, consider the garamond font family, in its normal form, we get the following result:

```
@font-face {
  font-family: "garamond";
  src: url("garamond.woff2");
}
```

The italicized version of this text uses the same glyphs present in the unstyled version, but they are artificially sloped by a few degrees.



On the other hand, if a true italicized version of the font family exists, we can include it in the `src` descriptor and specify the font style as italic, so that it is clear that the font is italicized. True italics use different glyphs and are a bit different from their upright counterparts, having some unique features and generally have a rounded and calligraphic quality. These fonts are specially created by font designers and are **not** artificially sloped.

```
@font-face {
  font-family: "garamond";
  src: url("garamond-italic.woff2");
  font-style: italic;
}
```

See also

- font-display
- font-family
- font-stretch
- font-weight
- font-feature-settings
- font-variation-settings
- src
- unicode-range

font-variation-settings CSS at-rule descriptor

The **font-variation-settings** CSS descriptor allows authors to specify low-level OpenType or TrueType font variations in the @font-face at-rule. The values for this descriptor are the same as the font-variation-settings property, except for the global keyword values.

Since this descriptor sets variation values on the font object in the @font-face at-rule and not on an entire element, only some glyphs in an element may be rendered using this descriptor.

Syntax

```
/* Use the default settings */
font-variation-settings: normal;

/* Set values for OpenType axis names */
font-variation-settings: "xhgt" 0.7;
```

Values

normal

Text is laid out using default settings.

<string> <number>

When rendering text, the list of OpenType axis names is passed to the text layout engine to enable or disable font features. Each setting is always a <string> of 4 ASCII characters, followed by a <number> indicating the axis value. If the <string> has more or fewer characters or contains characters outside the U+20 - U+7E code point range, the whole property is invalid. The <number> can be fractional or negative.

Formal definition

Related at-rule	@font-face
Initial value	normal
Computed value	as specified

Formal syntax

```
font-variation-settings =
  normal                |
  [ <string> <number> ]#
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting font weight and stretch in a @font-face rule

```
@font-face {
  font-family: "OpenTypeFont";
  src: url("open_type_font.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
  font-variation-settings:
    "wght" 400,
    "wdth" 300;
}
```

See also

- Other @font-face descriptors: font-display, font-family, font-feature-settings, font-stretch, font-style, font-weight, src, unicode-range
- Related font properties: font-feature-settings, font-variation-settings

font-weight CSS at-rule descriptor

The **font-weight** CSS @font-face descriptor enables authors to specify a single font weight, or a range of font weights, for the font specified in a @font-face at-rule. This is then used by the browser to select the appropriate font when a CSS rule sets a desired font weight.

Typically, a developer will want to use fonts from a single font family in a range of different weights. With traditional, or *static* fonts, a single font file contains characters from a font family in a specific weight and style: for example, "Helvetica bold italic". To enable displaying light, regular, bold, or extra-bold fonts when the font-weight property calls a specific weight, you can define multiple @font-face at-rules for the same family (all with the same font-family descriptor value), one for each weight or range of weights.

To declare the font to be used for a range of font weights, declare a space-separated pair of font-weight values as the value for the `font-weight` descriptor. When CSS rules set a font weight by setting the `font-weight` property or the `font` shorthand property, the appropriate font will then be used.

For example, if the descriptor is `font-weight: 400 600;`, when the property is `font-weight: 450` or `font-weight: 550`, that font will be used for that font-family. Whether the font is a static or a variable font, the font matching the range will be used. In this case, if the font is a static font, `450` and `550` will appear the same. If the font is a variable font, the latter will be bolder.

The descriptor is the same for all fonts, but the range you'll set for a variable font will generally be greater, possibly even `1 1000` to use the same font for all font weight property values.

Syntax

```
/* Single values */
font-weight: normal;
font-weight: bold;
font-weight: 400;

/* Defining a range */
font-weight: normal bold;
font-weight: 300 500;
```

Values

The `font-weight` descriptor takes one of the following forms:

- The keyword `auto`.
- A single `<font-weight-absolute>` value.
- A pair of `<font-weight-absolute>` values, separated by a space.

Each `<font-weight-absolute>` may be any one of the following:

`normal`

Normal font weight. Same as `400`.

`bold`

Bold font weight. Same as `700`.

`<number>`

A `<number>` value between 1 and 1000, inclusive. Higher numbers represent weights that are bolder than (or as bold as) lower numbers. Certain commonly used values correspond to common weight names, as described in the Common weight name mapping section below.

Common weight name mapping

The numerical values `100` to `900` roughly correspond to the following common weight names:

Value	Common weight name
100	Thin (Hairline)
200	Extra Light (Ultra Light)
300	Light
400	Normal
500	Medium
600	Semi Bold (Demi Bold)
700	Bold
800	Extra Bold (Ultra Bold)
900	Black (Heavy)

Variable fonts

Most fonts have a particular weight which corresponds to one of the numbers in Common weight name mapping. However some fonts, called variable fonts, can support a range of weights with more or less fine granularity, and this can give the designer a much closer degree of control over the chosen weight.

For TrueType or OpenType variable fonts, the "wght" variation is used to implement varying weights.

Accessibility

People experiencing low vision conditions may have difficulty reading text set with a `font-weight` value of `100` (Thin/Hairline) or `200` (Extra Light), especially if the font has a low contrast color ratio.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | W3C Understanding WCAG 2.0

Formal definition

Related at-rule	`@font-face`
Initial value	`normal`
Computed value	as specified

Formal syntax

```
font-weight =
  auto                          |
  <font-weight-absolute>{1,2}

<font-weight-absolute> =
  normal                        |
  bold                          |
  <number [1,1000]>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Selecting normal and bold fonts

In this example we include two fonts, one normal weight, one bold weight, from the "Fira Sans" font family using two `@font-face` at rules. We set `font-weight` descriptors to match the weight of the fonts.

After this, CSS rules can select the normal or the bold font for the "Fira Sans" family just by setting the `font-weight` property. Note that the `<strong>` HTML element also selects the bold font, because by default `<strong>` elements have a CSS `font-weight` property value of `bold`.

HTML

```
<p class="one">Fira Sans, `normal` weight paragraph</p>
<p class="two">Fira Sans, `bold` weight paragraph</p>
<p><strong>Fira Sans, &lt;strong&gt; element (`bold`)</strong></p>
```

CSS

```
@font-face {
  font-family: "Fira Sans";
  font-weight: normal;
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-Regular.woff2");
}

@font-face {
  font-family: "Fira Sans";
  font-weight: bold;
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-Bold.woff2");
}

body {
  font-family: "Fira Sans", serif;
  font-size: 2rem;
}

p.one {
  font-weight: normal;
}

p.two {
  font-weight: bold;
}
```

Result

Setting font-weight ranges

This example demonstrates how authors can include multiple fonts for multiple font-weights (and font-styles), by including multiple `@font-face` declarations with the same `font-family` value. By setting the `font-weight` descriptors using ranges from 1 to 1000, in the rest of your stylesheets, you can declare a `font-weight` (or `font-style`), and know the appropriate font will be used.

HTML

```
<p class="one">This has a font weight of 100</p>
<p class="three">This has a font weight of 300</p>
<p class="four">This has a font weight of 400</p>
<p class="five">This has a font weight of 500</p>
<p class="seven">This has a font weight of 700</p>
<p class="nine">This has a font weight of 900</p>
```

CSS

We include four `@font-face` declarations for four different fonts from the `FireSans` font family, as seen in the previous example. Each declaration is set to a different range of font-weight values, but all use the same font name.

The first declaration uses `FiraSans-Regular` and its associated `font-weight` range includes the entire possible range of font weight values.

The other three declarations use the light, bold, and extra-bold versions of the font, and their associated `font-weight` ranges define subsets of the range as follows:

- the light font is associated with the range 1-300
- the bold font is associated with the range 500-700
- the extra-bold font is associated with the range 700-1000

The CSS cascade ensures that the three latter declarations override parts of the range that was set in the `FiraSans-Regular` declaration.

```
@font-face {
  font-family: "Fira Sans";
  font-weight: 1 1000;
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-Regular.woff2");
}

@font-face {
  font-family: "Fira Sans";
  font-weight: 1 300;
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-Light.woff2");
}

@font-face {
  font-family: "Fira Sans";
  font-weight: 500 700;
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-Bold.woff2");
}

@font-face {
  font-family: "Fira Sans";
  font-weight: 700 1000;
  src: url("https://mdn.github.io/shared-assets/fonts/FiraSans-ExtraBold.woff2");
}

body {
  font-family: "Fira Sans", serif;
  font-size: 2rem;
}
```

```
p.one {
  font-weight: 100;
}

p.three {
  font-weight: 300;
}

p.four {
  font-weight: 400;
}

p.five {
  font-weight: 500;
}

p.seven {
  font-weight: 700;
}

p.nine {
  font-weight: 900;
}
```

Result

The `seven` paragraph uses the extra bold font. While `font-weight: 700` matches both the `FiraSans-Bold` and `FiraSans-ExtraBold` declarations, as the FiraSans-ExtraBold is declared later, it overrides the `FiraSans-Bold` for that value.

Similarly, the `100` and `300` both use the light font; although `FiraSans-Regular` and `FiraSans-Light` both include `300` in their ranges, `FiraSans-Light` is declared later. Alternatively, we could have declared `FiraSans-Regular` after `FiraSans-Light`, but we would need to change the `font-weight` descriptor range if we do so.

Setting a range for a variable font

In this example we're using the `font-weight` descriptor to restrict the range of weights that can be set when using a variable font.

We include a variable font, "League Mono", using a single `@font-face` at-rule. We use a `font-weight: 300 700` value to effectively limit the range of weights that are available. If a CSS rule uses our "League Mono" font, then if it specifies a weight outside this range the weight it gets is clamped to the range.

HTML

We include a paragraph with `<output>` initially set to `400`, as that is the default font-weight for unstyled paragraph text. This paragraph is nestled between two other paragraphs, so you can compare rendered versus declared font weight values.

We include an `<input/range>` of type `range`, nested in a `<label>`, setting the `step` to `50`.

```
<p>LeagueMono, font-weight: 300 (comparison)</p>
<p id="example">LeagueMono, font-weight: <output>400</output> (example)</p>
<p>LeagueMono, font-weight: 700 (comparison)</p>
<label
  >Change the font weight:
  <input type="range" min="50" max="1000" step="50" value="400" />
</label>
```

CSS

We set the `font-weight` descriptor range to `300 700`, clamping the variable font to that range.

```
@font-face {
  font-family: "LeagueMono";
  src: url("https://mdn.github.io/shared-assets/fonts/LeagueMono-VF.ttf");
  font-weight: 300 700;
}

p {
  font-family: "LeagueMono", serif;
  font-size: 1.5rem;
}

p:first-of-type {
  font-weight: 300;
}

p:last-of-type {
  font-weight: 700;
}
```

JavaScript

We include an event handler that updates the paragraph's `font-weight` property value, and updates the text to reflect the change:

```
const text = document.querySelector("#example");
const log = document.querySelector("output");
const range = document.querySelector("input");

range.addEventListener("change", () => {
  text.style.fontWeight = range.value;
  log.innerText = range.value;
});
```

Result

Change the font weight of the paragraph via the range. Note that the example paragraph does not get lighter than the `300` paragraph above it or bolder than the `700` paragraph below it; the font weight is clamped to the range defined by the `font-weight` descriptor.

See also

- `font-display`
- `font-family`

- `font-stretch`
- `font-style`
- `font-feature-settings`
- `font-variation-settings`
- `src`
- `unicode-range` descriptor

font-width CSS at-rule descriptor

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

> **Note:** The `font-width` descriptor is the modern replacement for the `font-stretch` descriptor, which is a legacy alias. While `font-width` is the specification's preferred name, `font-stretch` currently has broader browser support. Check the fallback example and browser compatibility table for details.

The **`font-width`** CSS descriptor allows authors to specify a normal, condensed, or expanded face for the fonts specified in the `@font-face` at-rule.

For a particular font family, authors can download various font faces corresponding to the different styles of the same font family, and then use the `font-width` descriptor to specify the font face's width explicitly. The available `font-width` descriptor values are the same as those of the corresponding `font-width` property.

Syntax

```
/* Single values */
font-width: ultra-condensed;
font-width: extra-condensed;
font-width: condensed;
font-width: semi-condensed;
font-width: normal;
font-width: semi-expanded;
font-width: expanded;
font-width: extra-expanded;
font-width: ultra-expanded;
font-width: 50%;
font-width: 100%;
font-width: 200%;

/* Multiple values */
font-width: 75% 125%;
font-width: condensed ultra-condensed;
```

The `font-width` descriptor can take a single value from the list below.

Values

`normal`

Specifies a normally condensed font face.

`semi-condensed`, `condensed`, `extra-condensed`, `ultra-condensed`

Specifies a more condensed font face than normal, with ultra-condensed being the most condensed.

`semi-expanded`, `expanded`, `extra-expanded`, `ultra-expanded`

Specifies a more expanded font face than normal, with ultra-expanded being the most expanded.

`<percentage>`

A `<percentage>` value between 50% and 200% (inclusive). Negative values are not allowed for this descriptor.

In earlier versions of the `font-width` specification, the descriptor accepts only the nine keyword values. CSS Fonts Level 4 extends the syntax to accept a `<percentage>` value as well. This enables variable fonts to offer a continuous variation of character widths. For TrueType or OpenType variable fonts, the `wdth` variation is used to implement varying widths.

If the font does not provide a face that exactly matches the given value, then values less than `100%` map to a condensed face, and values greater than or equal to `100%` map to an expanded face.

Keyword to numeric mapping

The table below shows the mapping between keyword values and numeric percentages:

Keyword	Percentage
`ultra-condensed`	50%
`extra-condensed`	62.5%
`condensed`	75%
`semi-condensed`	87.5%
`normal`	100%
`semi-expanded`	112.5%
`expanded`	125%
`extra-expanded`	150%
`ultra-expanded`	200%

Variable fonts

Most fonts have a particular width that corresponds to one of the keyterm values. However, variable fonts can support a range of widths with fine granularity, giving the designer a greater degree of control over the chosen weight. For this, percentage ranges are useful.

For TrueType or OpenType variable fonts, the `wdth` variation is used to implement varying glyph widths.

Accessibility

People with dyslexia and other cognitive conditions may have difficulty reading fonts that are too condensed, especially if the font has a low color contrast ratio.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | W3C Understanding WCAG 2.0

Formal definition

Value not found in DB!

Formal syntax

```
font-width =
  auto                |
  <'font-width'>{1,2}

<font-width> =
  normal              |
  <percentage [0,∞]>  |
  ultra-condensed     |
  extra-condensed     |
  condensed           |
  semi-condensed      |
  semi-expanded       |
  expanded            |
  extra-expanded      |
  ultra-expanded
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a percentage range for font-width

The following example uses the League Mono font. It synthesizes different font families from the same font file using the `font-width` descriptor with different keywords and percentages.

```
<p>League Mono</p>
<p>League Mono</p>
<p>League Mono</p>
```

```
@font-face {
  font-family: "League Mono Ultra Condensed";
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-width: ultra-condensed; /* same as 50% */
}

@font-face {
  font-family: "League Mono Normal";
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-width: 100%; /* same as normal */
}

@font-face {
  font-family: "League Mono Ultra Expanded";
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-width: ultra-expanded; /* same as 200% */
}

p:nth-child(1) {
  font-family: "League Mono Ultra Condensed", sans-serif;
}

p:nth-child(2) {
  font-family: "League Mono Normal", sans-serif;
}

p:nth-child(3) {
  font-family: "League Mono Ultra Expanded", sans-serif;
}
```

Providing a font-stretch fallback

Because `font-width` does not yet have broad browser support, you may want to include the legacy `font-stretch` descriptor as a fallback. Place `font-stretch` before `font-width` so that supporting browsers use the modern descriptor:

```
@font-face {
  font-family: "MyFont";
  src: url("my-font.woff2") format("woff2");
  font-stretch: condensed; /* for browsers that don't support font-width */
  font-width: condensed;
}
```

> **Note:** You can automate this fallback pattern using the postcss-preset-env plugin for PostCSS, which includes the postcss-font-width-property transform to convert `font-width` declarations to `font-stretch` automatically.

See also

- Legacy `font-stretch` alias descriptor with better browser support
- `font-display` descriptor
- `font-family` descriptor
- `font-weight` descriptor
- `font-style` descriptor
- `font-feature-settings` property
- `font-variation-settings` descriptor
- `src` descriptor
- `unicode-range` descriptor

line-gap-override CSS at-rule descriptor

The **`line-gap-override`** CSS descriptor for the `@font-face` at-rule defines the line-gap metric for the font. The line-gap metric is the font recommended line-gap or external leading.

Syntax

```
line-gap-override: normal;
line-gap-override: 90%;
```

Values

`normal`

The default value. When used the metric value is obtained from the font file.

`<percentage>`

A `<percentage>` value.

Formal definition

Related at-rule	`@font-face`
Initial value	`normal`
Percentages	as specified
Computed value	as specified

Formal syntax

```
line-gap-override =
  [ normal | <percentage [0,∞]> ]{1,2}
```

This syntax reflects the latest standard as per CSS Fonts Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Overriding metrics of a fallback font

The `line-gap-override` property can help when overriding the metrics of a fallback font to better match those of a primary web font.

```
@font-face {
  font-family: "web-font";
  src: url("https://example.com/font.woff");
}

@font-face {
  font-family: "local-font";
  src: local("Local Font");
  line-gap-override: 125%;
}
```

See also

- `descent-override`
- `font-display`
- `font-family`
- `font-weight`
- `font-style`
- `font-stretch`
- `font-feature-settings`
- `font-variation-settings`
- `src`
- `size-adjust`
- `unicode-range` descriptor

size-adjust CSS at-rule descriptor

The **`size-adjust`** CSS descriptor for the `@font-face` at-rule defines a multiplier for glyph outlines and metrics associated with this font. This makes it easier to harmonize the designs of various fonts when rendered at the same font size.

The `size-adjust` descriptor behaves in a similar fashion to the `font-size-adjust` property. It calculates an adjustment per font by matching ex heights.

Syntax

```
size-adjust: 90%;
```

Values

`<percentage>`

A `<percentage>` value with an initial value of 100%.

All metrics associated with this font are scaled by the given percentage. This includes glyph advances, baseline tables, and overrides provided by `@font-face` descriptors.

Formal definition

Related at-rule	`@font-face`
Initial value	`100%`
Percentages	as specified
Computed value	as specified

Formal syntax

```
size-adjust =
  <percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Overriding metrics of a fallback font

The `size-adjust` property can help when overriding the metrics of a fallback font to better match those of a primary web font.

```
@font-face {
  font-family: "web-font";
  src: url("https://example.com/font.woff");
}

@font-face {
  font-family: "local-font";
  src: local("Local Font");
  size-adjust: 90%;
}
```

See also

- `font-display` descriptor
- `font-family` descriptor
- `font-weight` descriptor
- `font-style` descriptor
- `font-stretch` descriptor
- `font-feature-settings`
- `font-variation-settings` descriptor
- `src` descriptor
- `unicode-range` descriptor
- `font-size-adjust` property

src CSS at-rule descriptor

The **`src`** CSS descriptor for the `@font-face` at-rule specifies the resource containing font data. It is required for the `@font-face` rule to be valid.

Syntax

```
/* <url> values */
src: url("https://example.com/path/to/font.woff"); /* Absolute URL */
src: url("path/to/font.woff"); /* Relative URL */
src: url("path/to/svgFont.svg#example"); /* Fragment identifying font */

/* <font-face-name> values */
src: local(font); /* Unquoted name */
src: local(some font); /* Name containing space */
src: local("font"); /* Quoted name */
src: local("some font"); /* Quoted name containing a space */

/* <tech(<font-tech>)> values */
src: url("path/to/fontCOLRv1.otf") tech(color-COLRv1);
src: url("path/to/fontCOLR-svg.otf") tech(color-SVG);

/* <format(<font-format>)> values */
src: url("path/to/font.woff") format("woff");
src: url("path/to/font.woff2") format("woff2");

/* Multiple resources */
src:
  url("path/to/font.woff") format("woff"),
  url("path/to/font.woff2") format("woff2");

/* Multiple resources with font format and technologies */
src:
  url("trickster-COLRv1.woff2") format("woff2") tech(color-COLRv1),
  url("trickster-outline.woff2") format("woff2");
```

Values

`url()`

Specifies an external reference consisting of a `<url>`, followed by optional hints using the `format()` and `tech()` component values that specify the format and font technology of the resource referenced by the URL. The `format()` and `tech()` components are a comma-separated list of strings of known font formats and technologies. If a user agent doesn't support the font technology or formats, it skips downloading the font resource. If no format or technology hints are supplied, the font resource is always downloaded.

`format()`

An optional declaration that follows the `url()` value that provides a hint for the user agent on the font format. If the value is not supported or invalid, the browser may not download the resource, potentially saving bandwidth. If omitted, the browser will download the resource and then detect the format. If including a font source for backward-compatibility that is not in the list of defined keywords, enclose the format string in quotes. Possible values are described in the Font formats section below.

`tech()`

An optional declaration that follows the `url()` value that provides a hint for the user agent on the font technology. The value for `tech()` may be one of the keywords described in Font technologies.

`local(<font-face-name>)`

Specifies the font name should the font be available on the user's device. Enclosing the font name in quotes is optional.

> **Note:** For OpenType and TrueType fonts, `<font-face-name>` is used to match either the Postscript name or the full font name in the name table of locally available fonts. Which type of name is used varies by platform and font, so you should include both of these names to assure proper matching across platforms. Platform substitutions for a given font name must not be used.

> **Note:** Locally available fonts may have been preinstalled on the user's device, or may have been actively installed by the user.
>
> While the set of preinstalled fonts is likely to be the same for all users of a particular device, the set of user-installed fonts is not. By discovering the set of user-installed fonts, a site can therefore build a fingerprint for the device, helping the site to track users across the web.

> To prevent this, user agents may ignore user-installed fonts when using `local()`.

`<font-face-name>`

Specifies the full name or postscript name of a locally-installed font face using the `local()` component value, which uniquely identifies a single font face within a larger family. The name can optionally be enclosed in quotes. The font face name is not case-sensitive.

> **Note:** The Local Font Access API can be used to access the user's locally installed font data — this includes higher-level details such as names, styles, and families, as well as the raw bytes of the underlying font files.

Description

The value of this descriptor is a prioritized, comma-separated list of external references or locally-installed font face names, where each resource is specified using `url()` or `local()`. When a font is needed, the user agent iterates over the set of references listed using the first one it can successfully activate. Fonts containing invalid data or local font faces that are not found are ignored and the user agent loads the next font in the list.

If multiple `src` descriptors are set, only the last declared rule that is able to load a resource is applied. If the last `src` descriptor can load a resource and doesn't include a `local()` font, the browser may download external font files and ignore the local version even if there is one available on the device.

> **Note:** Values within descriptors that the browser considers invalid are ignored. Some browsers will ignore the whole descriptor if any item is invalid, even if only one item is invalid. This may affect design of fallbacks. See Browser compatibility for more information.

As with other URLs in CSS, the URL may be relative, in which case it is resolved relative to the location of the style sheet containing the `@font-face` rule. In the case of SVG fonts, the URL points to an element within a document containing SVG font definitions. If the element reference is omitted, a reference to the first defined font is implied. Similarly, font container formats that can contain more than one font load only one of the fonts for a given `@font-face` rule. Fragment identifiers are used to indicate which font to load. If a container format lacks a defined fragment identifier scheme, a 1-based indexing scheme (e.g., "font-collection#1" for the first font, "font-collection#2" for the second font, etc.) is used.

If the font file is a container for multiple fonts, a fragment identifier is included to indicate the sub-font that should be used, as shown below:

```
@font-face {
  font-family: "WhichFont";
  /* WhichFont is the PostScript name of a font in the font file */
  src: url("collection.otc#WhichFont");
}
@font-face {
  font-family: "WhichFont-svg";
  /* WhichFont is the element id of a font in the SVG Font file */
  src: url("fonts.svg#WhichFont");
}
```

Font formats

The following table shows the valid font keywords and their corresponding font formats. To check if a font format is supported by a browser within CSS, use the `@supports` rule.

Keyword	Font Format	Common extensions
`collection`	OpenType Collection	.otc, .ttc
`embedded-opentype`	Embedded OpenType	.eot
`opentype`	OpenType	.otf, .ttf
`svg`	SVG Font (deprecated)	.svg, .svgz
`truetype`	TrueType	.ttf
`woff`	WOFF 1.0	.woff
`woff2`	WOFF 2.0	.woff2

> **Note:**
>
> - `format(svg)` stands for SVG fonts, and `tech(color-SVG)` stands for OpenType fonts with SVG table (also called OpenType-SVG color fonts), which are completely different.
> - The `opentype` and `truetype` values are equivalent whether the font file uses cubic bezier curves (within CFF/CFF2 table) or quadratic bezier curves (within glyph table).

Older non-normalized `format()` values have the following equivalent syntax; provided as a string enclosed in quotes for backward-compatibility reasons:

Old syntax	Equivalent syntax
`format("woff2-variations")`	`format(woff2) tech(variations)`
`format("woff-variations")`	`format(woff) tech(variations)`
`format("opentype-variations")`	`format(opentype) tech(variations)`
`format("truetype-variations")`	`format(truetype) tech(variations)`

Font technologies

The following table shows valid values for the `tech()` descriptor and their corresponding font technologies. To check if a font technology is supported by a browser within CSS, use the `@supports` at-rule.

Keyword	Description
`color-cbdt`	Color bitmap data tables
`color-colrv0`	Multi-colored glyphs via COLR version 0 table
`color-colrv1`	Multi-colored glyphs via COLR version 1 table
`color-sbix`	Standard bitmap graphics tables
`color-svg`	SVG multi-colored tables
`features-aat`	TrueType `morx` and `kerx` tables
`features-graphite`	Graphite features, namely `Silf`, `Glat`, `Gloc`, `Feat`, and `Sill` tables
`features-opentype`	OpenType `GSUB` and `GPOS` tables

Keyword	Description
`incremental`	Incremental font loading
`palettes`	Font palettes by means of `font-palette` to select one of many color palettes in the font
`variations`	Font variations in TrueType and OpenType fonts to control the font axis, weight, glyphs, etc.

Formal definition

Related at-rule	`@font-face`
Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
src =
  <font-src-list>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

```
<font-src> =
  <url> [ format( <font-format> ) ]? [ tech( <font-tech># ) ]?  |
  local( <font-family-name> )

<font-format> =
  <string>           |
  collection         |
  embedded-opentype  |
  opentype           |
  svg                |
  truetype           |
  woff               |
  woff2

<font-tech> =
  <font-features-tech>  |
  <color-font-tech>     |
  variations            |
  palettes              |
  incremental

<font-family-name> =
  <string>         |
  <custom-ident>+

<font-features-tech> =
  features-opentype  |
  features-aat       |
  features-graphite

<color-font-tech> =
  color-COLRv0  |
  color-COLRv1  |
  color-SVG     |
  color-sbix    |
  color-CBDT
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Fonts Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying font resources using url() and local()

The example below shows how to define two font faces with the same font family. The `font-family` is named `MainText`. The first font face has a regular font, and the second one is a bold version of the same font family.

```
/* Defining a regular font face */
@font-face {
  font-family: "MainText";
  src:
    local("Futura-Medium"),
    url("FuturaMedium.woff") format("woff"),
    url("FuturaMedium.woff2") format("woff2");
}

/* Defining a different bold font face for the same family */
@font-face {
  font-family: "MainText";
  src:
    local("Gill Sans Bold") /* full font name */,
    local("GillSans-Bold") /* postscript name */,
    url("GillSansBold.woff") format("woff"),
    url("GillSansBold.woff2") format("woff2"),
    url("GillSansBold.svg#MyFontBold"); /* Referencing an SVG font fragment by id */
  font-weight: bold;
}

/* Using the regular font face */
p {
  font-family: "MainText", sans-serif;
}

/* Font-family is inherited, but bold fonts are used */
p.bold {
  font-weight: bold;
}
```

Specifying font resources using tech() and format() values

The following example shows how to use the `tech()` and `format()` values to specify font resources. A font using `color-colrv1` technology and `opentype` format is specified using the `tech()` and `format()` values. A color font will be activated if the user agent supports it, and an `opentype` non-color is provided as a fallback.

```
@font-face {
  font-family: "Trickster";
  src:
    url("trickster-COLRv1.woff2") format("woff2") tech(color-COLRv1),
    url("trickster-outline.woff2") format("woff2");
}

/* Using the font face */
p {
  font-family: "Trickster", fantasy;
}
```

Specifying fallbacks for older browsers

Browsers should use a `@font-face` with a single `src` descriptor listing possible sources for the font. Since the browser will use the first resource that it is able to load, items should be specified in the order of your preference for their usage.

Generally this means that local files should appear before remote files and that resources with `format()` or `tech()` constraints should appear before resources that don't have them (otherwise the less-constrained version would always be selected). For example:

```
@font-face {
  font-family: "MgOpenModernaBold";
  src:
    url("MgOpenModernaBoldIncr.woff2") format("woff2") tech(incremental),
    url("MgOpenModernaBold.woff2") format("woff2");
}
```

A browser that does not support `tech()` above should ignore the first item and attempt to load the second resource.

Some browsers do not yet ignore invalid items, and instead fail the whole `src` descriptor if any value is invalid. If working with these browsers you can specify multiple `src` descriptors as fallbacks. Note that multiple `src` descriptors are attempted in reverse-order, so at the end we have our normal descriptor with all the items.

```
@font-face {
  font-family: "MgOpenModernaBold";
  src: url("MgOpenModernaBold.woff2") format("woff2");
  src: url("MgOpenModernaBoldIncr.woff2") format("woff2") tech(incremental);
  src:
    url("MgOpenModernaBoldIncr.woff2") format("woff2") tech(incremental),
    url("MgOpenModernaBold.woff2") format("woff2");
}
```

Checking if the user agent supports a font

The following example shows how to check if the user agent supports a font technology using the `@supports` rule. The block of CSS inside `@supports` will be applied if the user agent supports `color-COLRv1` technology.

```
@supports font-tech(color-COLRv1) {
  @font-face {
    font-family: "Trickster";
    src: url("trickster-COLRv1.woff2") format("woff2") tech(color-COLRv1);
  }

  .colored_text {
    font-family: "Trickster", fantasy;
  }
}
```

See also

- `@font-face`
- `@supports`
- `font-display`
- `font-family`
- `font-stretch`
- `font-style`
- `font-weight`
- `font-feature-settings`
- `font-variation-settings`
- `unicode-range`

unicode-range CSS at-rule descriptor

The **`unicode-range`** CSS descriptor sets the specific range of characters to be used from a font defined using the `@font-face` at-rule and made available for use on the current page. If the page doesn't use any character in this range, the font is not downloaded; if it uses at least one, the whole font is downloaded.

Syntax

```
/* <unicode-range> values */
unicode-range: U+26; /* single code point */
unicode-range: U+0-7F;
unicode-range: U+0025-00FF; /* code point range */
unicode-range: U+4??; /* wildcard range */
unicode-range: U+0025-00FF, U+4??; /* multiple values */
```

Values

single code point

A single Unicode character code point, for example `U+26`.

code point range

A range of Unicode code points. So for example, `U+0025-00FF` means *include all characters in the range `U+0025` to `U+00FF`*.

wildcard range

A range of Unicode code points containing wildcard characters, that is using the `'?'` character, so for example `U+4??` means *include all characters in the range `U+400` to `U+4FF`*.

Description

The purpose of this descriptor is to allow the font resources to be segmented so that a browser only needs to download the font resource needed for the text content of a particular page. For example, a site with many localizations could provide separate font resources for English, Greek and Japanese. For users viewing the English version of a page, the font resources for Greek and Japanese fonts wouldn't need to be downloaded, saving bandwidth.

Formal definition

Related at-rule	`@font-face`
Initial value	`U+0-10FFFF`
Computed value	as specified

Formal syntax

```
unicode-range =
  <unicode-range-token>#
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using a different font for a single character

In this example, we create a single `<div>` element, with a text string that includes an ampersand that we want to style with a different font. To make it obvious, we will use a sans-serif font, *Helvetica*, for the text, and a serif font, *Times New Roman*, for the ampersand.

In the CSS we are in effect defining a completely separate `@font-face` that only includes a single character in it, meaning that only this character will be styled with this font. We could also have done this by wrapping the ampersand in a `<span>` and applying a different font just to that, but that is an extra element and rule set.

HTML

```
<div>Me & You = Us</div>
```

CSS

```
@font-face {
  font-family: "Ampersand";
  src: local("Times New Roman");
  unicode-range: U+26;
}

div {
  font-size: 4em;
  font-family: "Ampersand", "Helvetica", sans-serif;
}
```

Result

See also

- `font-display`
- `font-family`
- `font-stretch`
- `font-style`
- `font-weight`
- `font-feature-settings`
- `font-variation-settings`
- `src`

@font-feature-values CSS at-rule

The **`@font-feature-values`** CSS at-rule lets you use a common name in the `font-variant-alternates` property for features activated differently in OpenType. This can help simplify your CSS when using multiple fonts.

The `@font-feature-values` at-rule may be used either at the top level of your CSS or inside any CSS conditional-group at-rule.

Syntax

```
@font-feature-values Font Name {
  font-display: swap;
  @styleset {
    nice-style: 12;
  }
  @swash {
    fancy: 2;
  }
}
```

Each `@font-feature-values` block can contain a list of feature value blocks (listed below), as well as the `font-display` descriptor.

Feature value blocks

`@swash`

Specifies a feature name that will work with the `swash()` functional notation of `font-variant-alternates`. A swash feature value definition allows only one value: `ident1: 2` is valid, but `ident2: 2 4` isn't.

`@annotation`

Specifies a feature name that will work with the `annotation()` functional notation of `font-variant-alternates`. An annotation feature value definition allows only one value: `ident1: 2` is valid, but `ident2: 2 4` isn't.

`@ornaments`

Specifies a feature name that will work with the `ornaments()` functional notation of `font-variant-alternates`. An ornaments feature value definition allows only one value: `ident1: 2` is valid, but `ident2: 2 4` isn't.

`@stylistic`

Specifies a feature name that will work with the `stylistic()` functional notation of `font-variant-alternates`. A stylistic feature value definition allows only one value: `ident1: 2` is valid, but `ident2: 2 4` isn't.

`@styleset`

Specifies a feature name that will work with the `styleset()` functional notation of `font-variant-alternates`. A styleset feature value definition allows an unlimited number of values: `ident1: 2 4 12 1` maps to the OpenType values `ss02`, `ss04`, `ss12`, and `ss01`. Note that values higher than `99` are valid, but don't map to any OpenType values and are ignored.

`@character-variant`

Specifies a feature name that will work with the `character-variant()` functional notation of `font-variant-alternates`. A character-variant feature value definition allows either one or two values: `ident1: 3` maps to `cv03=1`, and `ident2: 2 4` maps to `cv02=4`, but `ident2: 2 4 5` is invalid.

Formal syntax

```
@font-feature-values =
  @font-feature-values <font-family-name># { <declaration-rule-list> }

<font-family-name> =
  <string>         |
  <custom-ident>+
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using @styleset in a @font-feature-values rule

```
/* At-rule for "nice-style" in Font One */
@font-feature-values Font One {
  @styleset {
    nice-style: 12;
  }
}

/* At-rule for "nice-style" in Font Two */
@font-feature-values Font Two {
  @styleset {
    nice-style: 4;
  }
}

/* Apply the at-rules with a single declaration */
.nice-look {
  font-variant-alternates: styleset(nice-style);
}
```

See also

- The `font-variant-alternates` property that uses values that this at-rule defines.

font-display CSS at-rule descriptor

The **`font-display`** descriptor for the `@font-feature-values` at-rule sets the default value of how a font face is displayed based on whether and when it is downloaded. Setting a value for the `font-display` descriptor within a `@font-feature-values` block sets the default value of the `font-display` descriptor for the `@font-face` at-rule for all the fonts with the same `font-family` value.

Syntax

```
/* Keyword values */
font-display: auto;
font-display: block;
font-display: swap;
font-display: fallback;
font-display: optional;
```

Values

`auto`

The font display strategy is defined by the user agent.

`block`

Gives the font face a short block period, generally around 3 seconds, and an infinite swap period.

`swap`

Gives the font face an extremely small block period and an infinite swap period.

`fallback`

Gives the font face an extremely small block period and a short swap period.

`optional`

Gives the font face an extremely small block period and no swap period.

Description

The `font-display` descriptor for `@font-feature-values` determines the font display timeline; it does so by setting a default `font-display` value for `@font-face` for the same `font-family` name. When `font-display` is omitted in `@font-face`, the user agent first looks for the `font-display` value that has been set via `@font-feature-values` for the relevant font-family. If no value is found, the user agent uses the `auto` value for `font-display`, in which case, the user agent determines the font display strategy.

The font display timeline is based on a timer that starts when the user agent attempts to use a specific downloaded font face. The timeline is divided into three periods, as listed below. These periods dictate the rendering behavior of any element that uses the font face.

- Font **block** period: If the font face is not loaded, elements attempting to use the font are rendered with an *invisible* fallback font face. The browser blocks visible text rendering, saving a space for the to-be-displayed text based on metrics of the fallback font. During the block period, text is not visible. At the end of the block period, if the font has not loaded, the text is rendered in the fallback font.

- Font **swap** period: The swap period occurs after the block period (if there is one) and if the font face has not yet been loaded successfully. Elements attempting to use the not-yet-loaded font are rendered using the next available fallback font face. The formerly invisible fallback font face is painted onto the screen. If the font loads successfully during the swap period, the text that was rendered in the fallback font is updated — or swapped — with the newly loaded font. This step triggers a repaint.
- Font **failure** period: If the font face has not loaded by the time the swap period expires or by the time the block period expires (if there is no swap period, as is the case with `optional`), the user agent treats the font as a failed load. As a result, the content becomes visible in the fallback font.

The `font-display` descriptor allows you to set a default display policy for all `@font-face` rules, including those not under author control, such as third-party `@font-face` rules. When the `font-display` value is set via `@font-feature-values`, it becomes the default `font-display` value, and is applied to the entire font-family. However, any `font-display` value defined within individual `@font-face` blocks will override this default, but only for those blocks where the descriptor is defined.

Examples

```
@font-feature-values "Leitura" {
  font-display: swap;
  @swash {
    fancy: 1;
  }
}
```

The `font-display` descriptor in this example sets the default `font-display` value for the "Leitura" font to `swap` for all the `@font-face` blocks. Now there might be several `@font-face` blocks importing multiple font files for a single font-family. If one of those `@font-face` blocks includes a `font-display` descriptor, the specified value will be used for that specific font file only. All other blocks not including a `font-display` descriptor will default to `swap` instead of the user-agent's standard default `auto`.

Browser compatibility

Currently, no browsers support this feature.

See also

- `font-display` descriptor for `@font-face` at-rule
- CSS fonts module

@font-palette-values CSS at-rule

The **`@font-palette-values`** CSS at-rule allows you to customize the default values of font-palette created by the font-maker.

Syntax

```
@font-palette-values --identifier {
  font-family: "Bixa";
}
.my-class {
  font-palette: --identifier;
}
```

The <dashed-ident> is a user-defined identifier, that while it looks like a CSS custom property behaves in a different way and is not wrapped in a CSS var() function.

Descriptors

`base-palette`

Specifies the name or index of the base-palette, created by the font-maker, to use.

`font-family`

Specifies the name of the font family that this palette can be applied to. A `font-family` name is required for the `@font-palette-values` rule to be valid.

`override-colors`

Specifies the colors in the base palette to override.

Formal syntax

```
@font-palette-values =
  @font-palette-values <dashed-ident> { <declaration-list> }
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Overriding colors in an existing palette

This example shows how you can change some or all of the colors in a color font.

HTML

```
<p>default colors</p>
<p class="alternate">alternate colors</p>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Bungee+Spice";
p {
  font-family: "Bungee Spice", fantasy;
  font-size: 2rem;
}
@font-palette-values --Alternate {
  font-family: "Bungee Spice";
  override-colors:
    0 #00ffbb,
    1 #007744;
}
.alternate {
  font-palette: --Alternate;
}
```

Result

When overriding colors of the normal or base-palette at index 0 you do not need to declare which base-palette to use. This should only be done when overriding a different base-palette. If you are overriding all the colors then there is also no need to specify the base-palette to use.

See also

- `font-palette` property
- `font-family` descriptor
- `base-palette` descriptor
- `override-colors` descriptor
- `CSSFontPaletteValuesRule`

base-palette CSS at-rule descriptor

The **`base-palette`** CSS descriptor is used to specify the name or index of a pre-defined palette to be used for creating a new palette. If the specified `base-palette` does not exist, then the palette defined at index 0 will be used.

Syntax

```
@font-palette-values --one {
  base-palette: 1;
}
```

The `base-palette` descriptor is specified using a zero-based index of the font-maker created palettes.

Values

`<index>`

Specifies the index of the pre-defined palette to use.

Formal definition

Related at-rule	`@font-palette-values`
Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
base-palette =
  light           |
  dark            |
  <integer [0,∞]>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing the default palette in a font

Using the Rocher Color Font, this example shows two instances of switching the default palette in the font to an alternate palette created by the font-maker.

HTML

```
<h2>default base-palette</h2>
<h2 class="two">base-palette at index 2</h2>
<h2 class="five">base-palette at index 5</h2>
```

CSS

```
@font-face {
  font-family: "Rocher";
  src: url("[path-to-font]/RocherColorGX.woff2") format("woff2");
}

h2 {
  font-family: "Rocher", fantasy;
}

@font-palette-values --two {
  font-family: "Rocher";
  base-palette: 2;
}

@font-palette-values --five {
  font-family: "Rocher";
  base-palette: 5;
}

.two {
  font-palette: --two;
}

.five {
  font-palette: --five;
}
```

Result

See also

- `@font-palette-values`
- `font-family` descriptor
- `override-colors` descriptor
- `font-palette` property
- `CSSFontPaletteValuesRule.basePalette`

font-family CSS at-rule descriptor

The @font-palette-values descriptor **`font-family`** is used to specify which font-family palette values are to be applied to. This need to match exactly the values used when setting the CSS font-family.

Syntax

```
@font-palette-values --Dark-mode {
  font-family: "Bungee Spice";
  /* other palette settings follow */
}
```

Other palette values that follow apply only to the specified font family. You can create @font-palette-values for other font families by using the same <dashed-ident>s. This means that if you have multiple Color Fonts and can use the same identifier for each.

Values

`<family-name>`

Specifies the name of the font-family.

Formal definition

Related at-rule	`@font-palette-values`
Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
font-family =
  <font-family-name>#

<font-family-name> =
  <string>         |
  <custom-ident>+
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using matching family names

In this example, when the `font-family` descriptor is used in the @font-palette-values at-rule, the same value is used for the `font-family`, as when it is declared.

HTML

```
<h2>This is spicy</h2>
<h2 class="extra-spicy">This is extra hot & spicy</h2>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Bungee+Spice";
@font-palette-values --bungee-extra-spicy {
  font-family: "Bungee Spice";
  override-colors:
    0 darkred,
    1 red;
}

h2 {
  font-family: "Bungee Spice", fantasy;
}

h2.extra-spicy {
  font-palette: --bungee-extra-spicy;
}
```

Result

Using the same palette identifier for multiple font-families

In this example, two @font-palette-values at-rules are set for two font families, but both the at-rules use the same dashed-ident identifier, `--Dark-Mode`. This helps to set the font-palette property for multiple elements, `h1` and `h2` in this case, at the same time. This can be useful when you want to update font colors to match your site's branding.

```
@font-palette-values --Dark-Mode {
  font-family: "Bungee Spice";
```

```
  /* palette settings for Bungee Spice */
}

@font-palette-values --Dark-Mode {
  font-family: "Bixa";
  /* palette settings for Bixa */
}

h1,
h2 {
  font-palette: --Dark-Mode;
}

h1 {
  font-family: "Bungee Spice", fantasy;
}

h2 {
  font-family: "Bixa", fantasy;
}
```

See also

- font-family
- @font-palette-values
- override-colors descriptor
- font-palette property
- CSSFontPaletteValuesRule.fontFamily

override-colors CSS at-rule descriptor

The **override-colors** CSS descriptor is used to override colors in the chosen base-palette for a color font.

Syntax

```
/* basic syntax */
override-colors: <index of color> <color>;

/* using color names */
override-colors: 0 red;

/* using hex-color */
override-colors: 0 #ff0000;

/* using rgb */
override-colors: 0 rgb(255 0 0);

/* overriding multiple colors */
override-colors:
  0 red,
  1 green,
  2 blue;
```

The override-colors descriptor takes a comma-separated list of the color index and new color value.

The color index is zero-based and any color value can be used.

For each key-value pair of index and color, the color with the index in the specified base-palette will be overwritten. If the color font does not have a color at the specified index, it will be ignored.

Values

[<integer [0,∞]> <absolute-color-base>]

Specifies the index of a color in a base-palette and the color to overwrite it with.

Formal definition

Related at-rule	@font-palette-values
Initial value	n/a (required)
Computed value	as specified

Formal syntax

```
override-colors =
  [ <integer [0,∞]> <color> ]#

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing colors of emojis

This example shows how to override colors in the Noto Color Emoji color font to match your site's brand.

HTML

```
<section class="hats">
  <div class="hat">
    <h2>Original Hat</h2>
    <div class="emoji">🧢</div>
  </div>
  <div class="hat">
    <h2>Red Hat</h2>
    <div class="emoji red-hat">🧢</div>
```

```
  </div>
</section>
```

CSS

```
.hats {
  display: flex;
  flex-direction: row;
  justify-content: space-around;
}
```

```
@font-face {
  font-family: "Noto Color Emoji";
  font-style: normal;
  font-weight: normal;
  src: url("https://fonts.gstatic.com/l/font?kit=Yq6P-KqIXTD0t4D9z1ESnKM3-HpFabts6diywYkdG3gjD0U&skey=a373f7129eaba270&v=v24")
    format("woff2");
}

.emoji {
  font-family: "Noto Color Emoji", emoji;
  font-size: 3rem;
}
@font-palette-values --red {
  font-family: "Noto Color Emoji";
  override-colors:
    0 rgb(74 11 0),
    1 rgb(149 22 1),
    2 rgb(183 27 1),
    3 rgb(193 28 1),
    4 rgb(230 34 1);
}
.red-hat {
  font-palette: --red;
}
```

Result

Changing a color in an alternate base-palette

Using Rocher Color Font, this example shows how to override one color in the font.

HTML

```
<h2 class="normal-palette">Normal Palette</h2>
<h2 class="override-palette">Override Palette</h2>
```

CSS

```
@font-face {
  font-family: "Rocher";
  src: url("[path-to-font]/RocherColorGX.woff2") format("woff2");
}
h2 {
  font-family: "Rocher", fantasy;
}
@font-palette-values --override-palette {
  font-family: "Rocher";
  base-palette: 3;
}
@font-palette-values --override-palette {
  font-family: "Rocher";
  base-palette: 3;
  override-colors: 0 rebeccapurple;
}
.normal-palette {
  font-palette: --normal-palette;
}
.override-palette {
  font-palette: --override-palette;
}
```

Result

This example shows that in `base-palette 3`, the color at index 0 is overridden with `rebeccapurple`.

See also

- `@font-palette-values`
- `base-palette`
- `font-family`
- `font-palette`
- `CSSFontPaletteValuesRule.overrideColors`

@function CSS at-rule

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`@function`** CSS at-rule enables defining CSS custom functions. Once defined, a custom function can be called using the `<dashed-function>` syntax (for example, `--my-function(30px, 3)`) within any property value.

Syntax

```
@function --function-name(<function-parameter>#?) [returns <css-type>]? {
  <declaration-rule-list>
}

<function-parameter> = --param-name <css-type>? [ : <default-value> ]?
```

The different parts of the `@function` syntax are as follows:

`--function-name`

The identifying name of the function, a `<dashed-ident>` that starts with `--` and is followed by a valid, user-defined identifier. It is case-sensitive.

`<function-parameter>#?` Optional

Zero or more function parameter definitions. Multiple parameter definitions are separated by commas. Each parameter consists of:

`--param-name`

A CSS custom property name to identify the parameter, a `<dashed-ident>` that starts with `--` and is followed by a valid, user-defined identifier. It is case-sensitive. Function parameters can be considered custom properties that are locally scoped to the function body.

`<css-type>` Optional

A CSS data type or a `type()` function that defines the accepted data type(s) for the parameter. If this is not specified, any data type will be valid for the parameter (the same as specifying `type(*)`).

`<default-value>` Optional

A CSS value specifying the default value to assign to the parameter if it is not specified when the function is called. This value must be valid according to the `<css-type>` if specified. The default value is separated from the other parts of the parameter definition with a colon (`:`).

`[returns <css-type>]?` Optional

A CSS data type or a `type()` function, preceded by the keyword `returns`, which defines the accepted return type(s) for the parameter. If this is not specified, any data type will be valid for the parameter (the same as specifying `returns type(*)`), although bear in mind that the function will be invalid if the return type does not match the type produced by the `result` descriptor.

`<declaration-rule-list>`

One or more CSS declarations or at-rules that define the body of the function, which contains its logic. Included declarations can include:

- CSS custom properties, which are locally scoped to the function body.
- The `result` descriptor, either directly inside the `@function` at-rule, or inside a nested at-rule.

Descriptors

`result`

A valid property value that defines the result returned by the CSS custom function. The expression contained in the value is evaluated, and the result is returned.

Description

CSS custom functions allow you to define reusable sections of logic that will return different values depending on the parameters they accept as inputs and the logic defined inside the function body.

A typical CSS function looks like this:

```
@function --transparent(--color, --alpha) {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

The function has a name of `--transparent` and takes two custom properties as parameters, `--color` and `--alpha`, which can be used locally inside the function body. The body contains a single line, which is a `result` descriptor that defines the value returned by the function. The value of the `result` descriptor uses CSS relative color syntax to convert the input `--color` value into an `oklch()` color with the alpha channel value specified in the input `--alpha` value.

You can then call this function anywhere you want to produce a semi-transparent version of an existing color, for example:

```
section {
  --base-color: #faa6ff;
  background-color: --transparent(var(--base-color), 0.8);
}
```

The function is called by using `<dashed-function>` syntax, which is the function name with parentheses on the end. The desired argument values are specified inside the parentheses.

> **Note:** If multiple CSS functions are given the same name, the function in the stronger cascade `@layer` wins. If all of them are in the same layer, the function defined last in the source order wins.

Specifying data types

It is possible to specify data types for the function parameters and return types. For example:

```
@function --transparent(--color <color>, --alpha <number>) returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

Now the function will only produce a valid value if the input arguments are a `<color>` and a `<number>`, respectively, and the `result` is a `<color>`. If not, for example:

```
section {
  --base-color: #faa6ff;
  background-color: --transparent(var(--base-color), 50%);
}
```

then the value will become invalid at computed-value time (because the specified `--alpha` argument is a `<percentage>` and not a `<number>` as expected) and the `background-color` will end up being set to `transparent`.

You can specify multiple accepted data types using a `type()` function with the `|` symbol as a separator, for example:

```
@function --transparent(--color <color>, --alpha type(<number> | <percentage>))
  returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

With this adjustment, the `--transparent(var(--base-color), 50%)` function call is now valid.

Specifying default values

You can also specify default values for parameters, after a colon at the end of their definition. For example:

```
@function --transparent(--color <color>, --alpha <number>: 0.8) returns <color> {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

The `--alpha` parameter's default value is now `0.8`. If you want to use this value, you can omit the second argument when calling the function:

```
section {
  --base-color: #faa6ff;
  background-color: --transparent(var(--base-color));
}
```

> **Note:** If an invalid value is passed in as a function argument and a default value is specified in that parameter definition, the invalid value will be ignored, and the default value will be used instead.

Passing comma-containing values as arguments

In the next example, the `--max-plus-x()` function expects to be passed a comma-separated list of lengths and a single length as arguments. It uses the CSS `max()` function to determine which of the list of lengths is the largest, adds it to the single length, then returns the result.

```
@function --max-plus-x(--list <length>#, --x <length>) {
  result: calc(max(var(--list)) + var(--x));
}
```

The first argument needs to be a comma-separated list, which could be misinterpreted as three separate arguments. To get around this problem, you can wrap the value in curly braces when passing it into the function call:

```
div {
  width: --max-plus-x({1px, 7px, 2px}, 3px); /* 10px */
}
```

Including custom properties inside functions

As we've already seen, function parameters are defined as custom properties, which are then available inside the function body.

You can also specify custom properties inside the function body that will act as locally-scoped constants. In the following example, we define a function called `--anim-1s()`, which returns an `animation` shorthand value where the duration and easing values are always the same, and only the animation name and count are varied.

```
@function --anim-1s(--animation, --count) {
  --duration: 1s;
  --easing: linear;
  result: var(--animation) var(--duration) var(--count) var(--easing);
}
```

This kind of usage allows you to write easier, more expressive syntax for animations, provided you know that you always want the duration and easing function to be the same:

```
animation: --anim-1s(bounce, 2);
```

It is also worth noting that you can call one custom function from inside another. In such cases, a custom function can access local variables and function parameters from functions higher up in the call stack. Here, the outer function's parameter and local custom property will be available inside the scope of the inner function:

```
@function --outer(--outer-arg) {
  --outer-local: 2;
  result: --inner();
}

@function --inner() returns <number> {
  result: calc(var(--outer-arg) + var(--outer-local));
}

div {
  z-index: --outer(1); /* 3 */
}
```

In addition, custom properties defined on the same element where the custom function is being called will be available to it:

```
@function --double-z() returns <number> {
  result: calc(var(--z) * 2);
}

div {
  --z: 3;
  z-index: --double-z(); /* 6 */
}
```

When a custom property of the same name is defined in multiple places, function parameters override custom properties defined on the same element, and local custom properties defined inside the function body override both. In the following example, the `--add-a-b-c()` function uses the `--a` property from the `div` rule's custom property, the `--b` property from the function parameter, and the `--c` local custom property.

```
@function --add-a-b-c(--b, --c) {
  --c: 300;
  result: calc(var(--a) + var(--b) + var(--c));
}

div {
```

```
  --a: 1;
  --b: 2;
  --c: 3;
  z-index: --add-a-b-c(20, 30); /* 321 */
}
```

Including complex logic

You can include more complex logic in functions using constructs such as `@media` at-rules and `if()` functions. For example, the next function takes two arguments, one for a narrow-screen layout and one for a wide-screen layout. It returns the latter by default, but returns the former when the viewport width is less than `700px` wide, as detected using a media query.

```
@function --narrow-wide(--narrow, --wide) {
  result: var(--wide);
  @media (width < 700px) {
    result: var(--narrow);
  }
}
```

You can include multiple `result` descriptors to express different results for different logic outcomes.

> **Note:** CSS functions behave in the same way as the rest of CSS regarding conflict resolution — last in source order wins. Therefore, in the above function, the `result` is `var(--wide)` unless the media query test returns true, in which case it is overridden by `var(--narrow)`.
>
> There are no early returns in CSS functions like there are in JavaScript functions. In the above function, if the media query was written first, before the single `result` line, the `result` would always be `var(--wide)` because it would override `var(--narrow)` in cases where the media query test returns true.

We could rewrite the CSS custom function to use an `if()` function instead:

```
@function --narrow-wide(--narrow, --wide) {
  result: if(media(width < 700px): var(--narrow) ; else: var(--wide));
}
```

Formal syntax

```
@function =
  @function <function-token> <function-parameter>#? ) [ returns <css-type> ]? { <declaration-rule-list> }

<function-parameter> =
  <custom-property-name> <css-type>? [ : <default-value> ]?

<css-type> =
  <syntax-component>  |
  <type()>

<default-value> =
  <declaration-value>

<syntax-component> =
  <syntax-single-component> <syntax-multiplier>?  |
  '<' transform-list '>'

<type()> =
  type( <string> )

<syntax-single-component> =
  '<' <syntax-type-name> '>'  |
  <ident>

<syntax-multiplier> =
  '#'  |
  '+'

<syntax-type-name> =
  angle               |
  color               |
  custom-ident        |
  image               |
  integer             |
  length              |
  length-percentage   |
  number              |
  percentage          |
  resolution          |
  string              |
  time                |
  url                 |
  transform-function
```

This syntax reflects the latest standard as per CSS Functions and Mixins Module, CSS Images Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

For more examples, see our Using CSS custom functions guide.

Basic <code>@function</code> usage

This example shows a basic function that doubles the value passed into it.

HTML

The markup features a `<p>` element containing some text content:

```
<p>Some content</p>
```

CSS

In our styles, we first define the CSS custom function. The function is called `--double`, and accepts a single parameter of any type, which we've called `--value`. Inside the function body, we include a `result` descriptor that uses the `calc()` function to double the passed argument:

```
@function --double(--value) {
  result: calc(var(--value) * 2);
}
```

Next, we define a `--base-spacing` custom property with a value of `10px`. We assign that property to the `border-radius` value, but then double it for the `padding` value using the `--double()` custom function.

```
html,
body {
  height: 100%;
}

body {
  margin: 0;
  display: grid;
  place-items: center;
  font-family: system-ui;
}
```

```
p {
  --base-spacing: 10px;
  border-radius: var(--base-spacing);
  padding: --double(var(--base-spacing));
  width: 50%;
  background-color: wheat;
}
```

Result

See also

- CSS custom properties
- `<dashed-function>` data type
- `type()` function
- Using CSS custom functions
- CSS custom functions and mixins module

@import CSS at-rule

The **`@import`** CSS at-rule is used to import style rules from other valid stylesheets. An `@import` rule *must* be defined at the top of the stylesheet, before any other at-rule (except @charset and @layer) and style declarations, or it will be ignored.

Syntax

```
@import url;
@import url layer;
@import url layer(layer-name);
@import url layer(layer-name) supports(supports-condition);
@import url layer(layer-name) supports(supports-condition) list-of-media-queries;
@import url layer(layer-name) list-of-media-queries;
@import url supports(supports-condition);
@import url supports(supports-condition) list-of-media-queries;
@import url list-of-media-queries;
```

where:

url

> Is a `<string>` or a `<url>` type representing the location of the resource to import. The URL may be absolute or relative.

list-of-media-queries

> Is a comma-separated list of media queries, which specify the media-dependent conditions for applying the CSS rules defined in the linked URL. If the browser does not support any of these queries, it does not load the linked resource.

layer-name

> Is the name of a cascade layer into which the contents of the linked resource are imported. See `layer()` for more information.

supports-condition

> Indicates the feature(s) that the browser must support in order for the stylesheet to be imported. If the browser does not conform to the conditions specified in the *supports-condition*, it may not fetch the linked stylesheet, and even if downloaded through some other path, will not load it. The syntax of `supports()` is almost identical to that described in `@supports`, and that topic can be used as a more complete reference.

Use `@import` together with the `layer` keyword or `layer()` function to import external style sheets (from frameworks, widget stylesheets, libraries, etc.) into layers.

Description

Imported rules must come before all other types of rules, except `@charset` rules and layer creating `@layer` statements.

```
* {
  margin: 0;
  padding: 0;
}
/* more styles */
@import "my-imported-styles.css";
```

As the `@import` at-rule is declared after the styles it is invalid and hence ignored.

```
@import "my-imported-styles.css";
* {
  margin: 0;
  padding: 0;
}
/* more styles */
```

The `@import` rule is not a nested statement. Therefore, it cannot be used inside conditional group at-rules.

So that user agents can avoid retrieving resources for unsupported media types, authors may specify media-dependent import conditions. These conditional imports specify comma-separated media queries after the URL. In the absence of any media query, the import is not conditional on the media used. Specifying `all` for the `list-of-media-queries` has the same effect.

Similarly, user agents can use the `supports()` function in an `@import` at-rule to only fetch resources if a particular feature set is (or is not) supported. This allows authors to take advantage of recently introduced CSS features, while providing graceful fallbacks for older browser versions. Note that the conditions in the `supports()` function of an `@import` at-rule can be obtained in JavaScript using `CSSImportRule.supportsText`.

The `@import` rule can also be used to create a cascade layer by importing rules from a linked resource. Rules can also be imported into an existing cascade layer. The `layer` keyword or the `layer()` function is used with `@import` for this purpose. Declarations in style rules from imported stylesheets interact with the cascade as if they were written literally into the stylesheet at the point of the import.

Formal syntax

```
@import =
  @import [ <url> | <string> ] [ [ layer | layer( <layer-name> ) ] || [ scope | scope( <scope-start> | <scope-boundaries> ) ] || <supports-import-condition> ]? <media-import-condition> ;

<layer-name> =
  <ident> [ '.' <ident> ]*

<scope-start> =
  <selector-list>

<scope-boundaries> =
  [ [ ( <scope-start> ) ]? [ to ( <scope-end> ) ]? ]!

<supports-import-condition> =
  supports( [ <supports-condition> | <declaration> ] )

<media-import-condition> =
  <media-query-list>

<selector-list> =
  <complex-selector-list>

<scope-end> =
  <selector-list>

<supports-condition> =
  not <supports-in-parens>                          |
  <supports-in-parens> [ and <supports-in-parens> ]*  |
  <supports-in-parens> [ or <supports-in-parens> ]*

<complex-selector-list> =
  <complex-selector>#

<supports-in-parens> =
  ( <supports-condition> )  |
  <supports-feature>        |
  <general-enclosed>

<complex-selector> =
  <complex-selector-unit> [ <combinator>? <complex-selector-unit> ]*

<supports-feature> =
  <supports-selector-fn>       |
  <supports-font-tech-fn>      |
  <supports-font-format-fn>    |
  <supports-at-rule-fn>        |
  <supports-named-feature-fn>  |
  <supports-env-fn>            |
  <supports-decl>

<general-enclosed> =
  [ <function-token> <any-value>? ) ]  |
  [ ( <any-value>? ) ]

<complex-selector-unit> =
  [ <compound-selector>? <pseudo-compound-selector>* ]!

<combinator> =
  '>'               |
  '+'               |
  '~'               |
  [ '|' '|' ]       |
  [ / <wq-name> / ]

<supports-selector-fn> =
  selector( <complex-selector> )

<supports-font-tech-fn> =
  font-tech( <font-tech> )

<supports-font-format-fn> =
  font-format( <font-format> )

<supports-at-rule-fn> =
  at-rule( <at-keyword-token> )

<supports-named-feature-fn> =
  named-feature( <ident> )

<supports-env-fn> =
  env( <ident> )

<supports-decl> =
  ( [ <declaration> | <supports-condition-name> ] )

<compound-selector> =
  [ <type-selector>? <subclass-selector>* ]!

<pseudo-compound-selector> =
  <pseudo-element-selector> <pseudo-class-selector>*

<wq-name> =
  <ns-prefix>? <ident-token>

<font-tech> =
  <font-features-tech>  |
  <color-font-tech>     |
```

```
  variations            |
  palettes              |
  incremental

<font-format> =
  <string>              |
  collection            |
  embedded-opentype     |
  opentype              |
  svg                   |
  truetype              |
  woff                  |
  woff2

<type-selector> =
  <wq-name>         |
  <ns-prefix>? '*'

<subclass-selector> =
  <id-selector>             |
  <class-selector>          |
  <attribute-selector>      |
  <pseudo-class-selector>

<pseudo-element-selector> =
  : <pseudo-class-selector>         |
  <legacy-pseudo-element-selector>

<pseudo-class-selector> =
  : <ident-token>                   |
  : <function-token> <any-value> )

<ns-prefix> =
  [ <ident-token> | '*' ]? '|'

<font-features-tech> =
  features-opentype     |
  features-aat          |
  features-graphite

<color-font-tech> =
  color-COLRv0      |
  color-COLRv1      |
  color-SVG         |
  color-sbix        |
  color-CBDT

<id-selector> =
  <hash-token>

<class-selector> =
  '.' <ident-token>

<attribute-selector> =
  '[' <wq-name> ']'                                 |
  '[' <wq-name> <attr-matcher> [ <string-token> | <ident-token> ] <attr-modifier>? ']'

<legacy-pseudo-element-selector> =
  : [ before | after | first-line | first-letter ]

<attr-matcher> =
  [ '~' | '|' | '^' | '$' | '*' ]? '='

<attr-modifier> =
  i   |
  s
```

This syntax reflects the latest standard as per CSS Cascading and Inheritance Level 5, CSS Cascading and Inheritance Level 6, CSS Conditional Rules Module Level 3, CSS Conditional Rules Module Level 4, CSS Conditional Rules Module Level 5, CSS Fonts Module Level 5, Media Queries Level 5, Selectors Level 4, Selectors Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Importing CSS rules

```
@import "custom.css";
@import url("chrome://communicator/skin/communicator.css");
```

The two examples above show how to specify the *url* as a `<string>` and as a `url()` function.

Importing CSS rules conditional on media queries

```
@import "fine-print.css" print;
@import "bluish.css" print, screen;
@import "common.css" screen;
@import "landscape.css" screen and (orientation: landscape);
```

The `@import` rules in the above examples show media-dependent conditions that will need to be true before the linked CSS rules are applied. So for instance, the last `@import` rule will load the `landscape.css` stylesheet only on a screen device in landscape orientation.

Importing CSS rules conditional on feature support

```
@import "grid.css" supports(display: grid) screen and (width <= 400px);
@import "flex.css" supports((not (display: grid)) and (display: flex)) screen
  and (width <= 400px);
```

The `@import` rules above illustrate how you might import a layout that uses a grid if `display: grid` is supported, and otherwise imports CSS that uses `display: flex`. While you can only have one `supports()` statement, you can combine any number of feature checks with `not`, `and`, and `or`. However, you must use parenthesis to define precedence when you mix them, e.g., `supports((..) or (..) and not (..))` is invalid, but `supports((..) or ((..) and (not (..))))` is valid. Note that if you just have a single declaration then you don't need to wrap it in additional parentheses: this is shown in the first example above.

The examples above show support conditions using basic declaration syntax. You can also specify CSS functions in `supports()`, and it will evaluate to `true` if they are supported and can be evaluated on the user-agent. For example, the code below shows an `@import` that is conditional on both child combinators (`selector()`) and the `font-tech()` function:

```
@import "whatever.css"
  supports((selector(h2 > p)) and (font-tech(color-COLRv1)));
```

Importing CSS rules into a cascade layer

```
@import "theme.css" layer(utilities);
```

In the above example, a cascade layer named `utilities` is created and it will include rules from the imported stylesheet `theme`.

```
@import "headings.css" layer(default);
@import "links.css" layer(default);

@layer default {
  audio[controls] {
    display: block;
  }
}
```

In the above example, the rules in `headings.css` and `links.css` stylesheets cascade within the same layer as the `audio[controls]` rule.

```
@import "theme.css" layer();
@import "style.css" layer;
```

This is an example of creating two separate unnamed cascade layers and importing the linked rules into each one separately. A cascade layer declared without a name is an unnamed cascade layer. Unnamed cascade layers are finalized when created: they do not provide any means for re-arranging or adding styles and they cannot be referenced from outside.

See also

- `@media`
- `@supports`
- CSS cascading and inheritance module
- CSS at-rule functions

layer() CSS function

The **`layer()`** CSS function is used along with the `@import` at-rule to put the imported resource in a separate named cascade layer.

Syntax

```
@import url layer(layer-name);
@import "dark.css" layer(framework.themes.dark);
```

The `framework.themes.dark` is the layer into which the CSS file would be imported.

Formal syntax

```
layer() =
  layer( <layer-name> )

<layer-name> =
  <ident> [ '.' <ident> ]*
```

This syntax reflects the latest standard as per CSS Cascading and Inheritance Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- `@import`
- CSS at-rule functions
- CSS cascading and inheritance module

@keyframes CSS at-rule

The **`@keyframes`** CSS at-rule controls the intermediate steps in a CSS animation sequence by defining styles for keyframes (or waypoints) along the animation sequence. This gives more control over the intermediate steps of the animation sequence than transitions.

Syntax

```
@keyframes slide-in {
  from {
    transform: translateX(0%);
  }

  to {
    transform: translateX(100%);
  }
}
```

Values

`<keyframes-name>`

A case-sensitive `<custom-ident>` or string naming the keyframe list.

`from`

Equivalent to the value `0%`.

`to`

Equivalent to the value `100%`.

`<percentage>`

A percentage through the animation sequence at which the specified keyframe should occur.

`<timeline-range-name> <percentage>`

A percentage through the specified `animation-range` at which the specified keyframe should occur.

Description

To use keyframes, create a `@keyframes` rule with a name that is then used by the `animation-name` property to match an animation to its keyframe declaration. Each `@keyframes` rule contains a style list of keyframe selectors, which specify percentages along the animation when the keyframe occurs, and a block containing the styles for that keyframe.

You can list the keyframe percentages in any order; they will be handled in the order they should occur.

JavaScript can access the `@keyframes` at-rule with the CSS object model interface `CSSKeyframesRule`.

Valid keyframe lists

If a keyframe rule doesn't specify the start or end states of the animation (that is, `0%`/`from` and `100%`/`to`), browsers will use the element's existing styles for the start/end states. This can be used to animate an element from its initial state and back.

Properties that can't be animated in keyframe rules are ignored, but supported properties will still be animated.

Resolving duplicates

If multiple keyframe sets exist for a given name, the last one encountered by the parser is used. `@keyframes` rules don't cascade, so animations never derive keyframes from more than one rule set.

If a given animation time offset is duplicated, all keyframes in the `@keyframes` rule for that percentage are used for that frame. There is cascading within a `@keyframes` rule if multiple keyframes specify the same percentage values.

When properties are left out of some keyframes

Properties that aren't specified in every keyframe are interpolated if possible — properties that can't be interpolated are dropped from the animation. For example:

```
@keyframes identifier {
  0% {
    top: 0;
    left: 0;
  }
  30% {
    top: 50px;
  }
  68%,
  72% {
    left: 50px;
  }
  100% {
    top: 100px;
    left: 100%;
  }
}
```

Here, the `top` property animates using the `0%`, `30%`, and `100%` keyframes, and `left` animates using the `0%`, `68%`, `72%` and `100%` keyframes.

When a keyframe is defined multiple times

If a keyframe is defined multiple times but not all affected properties are in each keyframe, all values specified in these keyframes are considered. For example:

```
@keyframes identifier {
  0% {
    top: 0;
  }
  50% {
    top: 30px;
    left: 20px;
  }
  50% {
    top: 10px;
  }
  100% {
    top: 0;
  }
}
```

In this example, at the `50%` keyframe, the values used are `top: 10px` and `left: 20px`.

<code>!important</code> in a keyframe

Declarations in a keyframe qualified with the `important` flag set are ignored.

```
@keyframes important1 {
  0% {
    margin-top: 50px;
  }
  50% {
    margin-top: 150px !important; /* ignored */
  }
  100% {
    margin-top: 100px;
  }
}

@keyframes important2 {
  from {
    margin-top: 50px;
    margin-bottom: 100px;
  }
  to {
    margin-top: 150px !important; /* ignored */
    margin-bottom: 50px;
  }
}
```

Formal syntax

```
@keyframes =
  @keyframes <keyframes-name> { <qualified-rule-list> }
```

```
<keyframes-name> =
  <custom-ident>  |
  <string>
```

This syntax reflects the latest standard as per CSS Animations Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

CSS animation examples

See Using CSS animations and scroll-driven animations for examples.

See also

- `animation-name`
- Using CSS animations
- CSS scroll-driven animation timelines
- Understanding timeline range names
- CSS animations module
- CSS scroll-driven animations module
- `AnimationEvent`
- Animate elements on scroll with Scroll-driven animations

@layer CSS at-rule

The **`@layer`** CSS at-rule is used to declare a cascade layer and can also be used to define the order of precedence in case of multiple cascade layers.

Try it

```
@layer module, state;

@layer state {
  .alert {
    background-color: brown;
  }
  p {
    border: medium solid limegreen;
  }
}

@layer module {
  .alert {
    border: medium solid violet;
    background-color: yellow;
    color: white;
  }
}
```

```
<p class="alert">Beware of the zombies</p>
```

Syntax

```
/* statement at-rules */
@layer layer-name;
@layer layer-name, layer-name, layer-name;

/* block at-rules */
@layer {rules}
@layer layer-name {rules}
```

where:

layer-name

Is the name of the cascade layer.

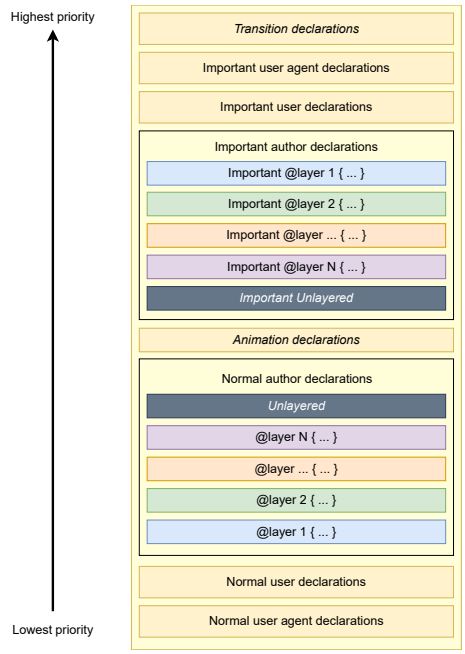
rules

Is the set of CSS rules in the cascade layer.

Description

Rules within a cascade layer cascade together, giving more control over the cascade to web developers. Styles that are not defined in a layer always override styles declared in named and anonymous layers.

The following diagram shows layer priorities where layers are declared in 1, 2, ..., N order.



As noted in the above diagram, *important declarations*, declarations with the `!important` flag, have priority over *normal declarations*, or regular declarations without the `!important` flag. The order of precedence among important rules is the inverse of normal rules. Transitions have the greatest precedence. Next in order of highest to lowest priority are the important user agent declarations, important user declarations, and important author declarations; in that order. Users can specify styles using browser preferences, operating system preferences, or browser extensions. Their important declarations take precedence over *author*, or *web developer* written, important declarations.

Within author styles, all important declarations within CSS layers take precedence over any important declarations declared outside of a layer, while all normal declarations within CSS layers have a lower priority than declarations declared outside of a layer. The declaration order matters. The first declared layer gets the lowest priority and the last declared layer gets the highest priority. However, the priority is reversed when the `!important` flag is used.

The `@layer` at-rule is used to create a cascade layer in one of three ways.

The first way is to use a `@layer` block at-rule to create a named cascade layer with the CSS rules for that layer inside, like so:

```
@layer utilities {
  .padding-sm {
    padding: 0.5rem;
  }

  .padding-lg {
    padding: 0.8rem;
  }
}
```

The second way is to use a `@layer` statement at-rule to create one or more comma-separated named cascade layers without assigning any styles. This can be a single layer, as shown below:

```
@layer utilities;
```

Multiple layers can be defined at once, as shown below:

```
@layer theme, layout, utilities;
```

This is useful because the initial order in which layers are declared indicates which layer has precedence. As with declarations, the last layer to be listed will win if declarations are found in multiple layers. Therefore, with the preceding example, if a competing rule was found in `theme` and `utilities`, the one in `utilities` would win and be applied.

A rule in `utilities` would be applied *even if it has lower specificity* than the rule in `theme`. This is because once the layer order has been established, specificity and order of appearance are ignored. This enables using simpler CSS selectors because you do not have to ensure that a selector will have high enough specificity to override competing rules; all you need to ensure is that it appears in a later layer.

> **Note:** Having declared your layer names, thus setting their order, you can add CSS rules to the layer by re-declaring the name. The styles are then appended to the layer and the layer order will not be changed.

The third way is to create an unnamed layer using a `@layer` block at-rule without including a layer name. For example:

```
@layer {
  p {
    margin-block: 1rem;
  }
}
```

This creates an *anonymous cascade layer*. This layer functions in the same way as named layers; however, rules cannot be assigned to it later. The order of precedence for anonymous layers is the order in which layers are declared, named or not, and lower than the styles declared outside of a layer.

Another way to create a cascade layer is by using `@import`. In this case, the rules would be in the imported stylesheet. Remember that the `@import` at-rule must precede all other types of rules, except `@charset` and `@layer` statements (not `@layer` blocks).

```
@import "theme.css" layer(utilities);
```

Nesting layers

Layers may be nested. For example:

```
@layer framework {
  @layer layout {
```

```
  }
}
```

To append rules to the `layout` layer inside `framework`, join the two names with a `.`.

```
@layer framework.layout {
  p {
    margin-block: 1rem;
  }
}
```

Formal syntax

```
@layer =
  @layer <layer-name>? { <rule-list> }  |
  @layer <layer-name># ;

<layer-name> =
  <ident> [ '.' <ident> ]*
```

This syntax reflects the latest standard as per CSS Cascading and Inheritance Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In the following example, two CSS rules are created. One for the `<p>` element outside of any layer and one inside a layer named `type` for `.box p`.

Without layers, the selector `.box p` would have the highest specificity, and therefore, the text `Hello, world!` will display in green. As the `type` layer comes before the anonymous layer created to hold non-layer content, the text will be purple.

Also notice the order. Even though we declare the non-layered style first, it's still applied *after* the layer styles.

HTML

```
<div class="box">
  <p>Hello, world!</p>
</div>
```

CSS

```
p {
  color: rebeccapurple;
}

@layer type {
  .box p {
    font-weight: bold;
    font-size: 1.3em;
    color: green;
  }
}
```

Result

Assigning rules to existing layers

In the following example, two layers are created with no rules applied, then CSS rules are applied to the two layers. The `base` layer defines a `color`, `border`, `font-size`, and `padding`. The `special` layer defines a different color. As `special` comes last when the layers were defined, the color it provides is used and the text is displayed using `rebeccapurple`. All of the other rules from `base` still apply.

HTML

```
<div class="item">
  I am displayed in <code>color: rebeccapurple</code> because the
  <code>special</code> layer comes after the <code>base</code> layer. My green
  border, font-size, and padding come from the <code>base</code> layer.
</div>
```

CSS

```
@layer base, special;

@layer special {
  .item {
    color: rebeccapurple;
  }
}

@layer base {
  .item {
    color: green;
    border: 5px solid green;
    font-size: 1.3em;
    padding: 0.5em;
  }
}
```

Result

See also

- `@import`
- `CSSLayerBlockRule`
- `CSSLayerStatementRule`
- `!important`
- `revert-layer`
- Introducing the CSS cascade
- Learn: Handling conflicts
- Learn: Cascade layers

- The future of CSS: Cascade layers on bram.us (2021)

@media CSS at-rule

The **`@media`** CSS at-rule can be used to apply part of a style sheet based on the result of one or more media queries. With it, you specify a media query and a block of CSS to apply to the document if and only if the media query matches the device on which the content is being used.

> **Note:** In JavaScript, the rules created using `@media` can be accessed with the `CSSMediaRule` CSS object model interface.

Try it

```
abbr {
  color: #860304;
  font-weight: bold;
  transition: color 0.5s ease;
}

@media (hover: hover) {
  abbr:hover {
    color: #001ca8;
    transition-duration: 0.5s;
  }
}

@media not all and (hover: hover) {
  abbr::after {
    content: " (" attr(title) ")";
  }
}
```

```
<p>
  <abbr title="National Aeronautics and Space Administration">NASA</abbr> is a
  U.S. government agency that is responsible for science and technology related
  to air and space.
</p>
```

Syntax

```
/* At the top level of your code */
@media screen and (width >= 900px) {
  article {
    padding: 1rem 3rem;
  }
}

/* Nested within another conditional at-rule */
@supports (display: flex) {
  @media screen and (width >= 900px) {
    article {
      display: flex;
    }
  }
}
```

The `@media` at-rule may be placed at the top level of your code or nested inside any other conditional group at-rule.

For a discussion of media query syntax, please see Using media queries.

Description

A media query's `<media-query-list>` includes `<media-type>`s, `<media-feature>`s, and logical operators.

Media types

A *`<media-type>`* describes the general category of a device. Except when using the `only` logical operator, the media type is optional and the `all` type is implied.

`all`

Suitable for all devices.

`print`

Intended for paged material and documents viewed on a screen in print preview mode. (Please see paged media for information about formatting issues that are specific to these formats.)

`screen`

Intended primarily for screens.

> **Note:** CSS2.1 and Media Queries 3 defined several additional media types (`tty`, `tv`, `projection`, `handheld`, `braille`, `embossed`, and `aural`, but they were deprecated in Media Queries 4 and shouldn't be used.

Media features

A *`<media feature>`* describes specific characteristics of the user agent, output device, or environment. Media feature expressions test for their presence, value, or range of values, and are entirely optional. Each media feature expression must be surrounded by parentheses.

`any-hover`

Does any available input mechanism allow the user to hover over elements?

`any-pointer`

Is any available input mechanism a pointing device, and if so, how accurate is it?

`aspect-ratio`

Width-to-height aspect ratio of the viewport.

`color`

Number of bits per color component of the output device, or zero if the device isn't color.

`color-gamut`

Approximate range of colors that are supported by the user agent and output device.

`color-index`

Number of entries in the output device's color lookup table, or zero if the device does not use such a table.

`device-aspect-ratio`

Width-to-height aspect ratio of the output device. Deprecated in Media Queries Level 4.

`device-height`

Height of the rendering surface of the output device. Deprecated in Media Queries Level 4.

`device-posture`

Detects the device's current posture, that is, whether the viewport is in a flat or folded state. Defined in the Device Posture API.

`device-width`

Width of the rendering surface of the output device. Deprecated in Media Queries Level 4.

`display-mode`

The mode in which an application is being displayed: for example, fullscreen or picture-in-picture mode. Added in Media Queries Level 5.

`dynamic-range`

Combination of brightness, contrast ratio, and color depth that are supported by the user agent and the output device. Added in Media Queries Level 5.

`forced-colors`

Detect whether user agent restricts color palette. Added in Media Queries Level 5.

`grid`

Does the device use a grid or bitmap screen?

`height`

Height of the viewport.

`horizontal-viewport-segments`

Detects whether the device has a specified number of viewport segments laid out horizontally.

`hover`

Does the primary input mechanism allow the user to hover over elements?

`inverted-colors`

Is the user agent or underlying OS inverting colors? Added in Media Queries Level 5.

`monochrome`

Bits per pixel in the output device's monochrome frame buffer, or zero if the device isn't monochrome.

`orientation`

Orientation of the viewport.

`overflow-block`

How does the output device handle content that overflows the viewport along the block axis?

`overflow-inline`

Can content that overflows the viewport along the inline axis be scrolled?

`pointer`

Is the primary input mechanism a pointing device, and if so, how accurate is it?

`prefers-color-scheme`

Detect if the user prefers a light or dark color scheme. Added in Media Queries Level 5.

`prefers-contrast`

Detects if the user has requested the system increase or decrease the amount of contrast between adjacent colors. Added in Media Queries Level 5.

`prefers-reduced-data`

Detects if the user has requested the web content that consumes less internet traffic.

`prefers-reduced-motion`

The user prefers less motion on the page. Added in Media Queries Level 5.

`prefers-reduced-transparency`

Detects if a user has enabled a setting on their device to reduce the transparent or translucent layer effects used on the device.

`resolution`

Pixel density of the output device.

`scan`

Whether display output is progressive or interlaced.

`scripting`

Detects whether scripting (i.e., JavaScript) is available. Added in Media Queries Level 5.

`shape`

Detects the shape of the device to distinguish rectangular and round displays.

`update`

How frequently the output device can modify the appearance of content.

`vertical-viewport-segments`

Detects whether the device has a specified number of viewport segments laid out vertically. Added in Media Queries Level 5.

`video-dynamic-range`

Combination of brightness, contrast ratio, and color depth that are supported by the video plane of user agent and the output device. Added in Media Queries Level 5.

`width`

Width of the viewport including width of scrollbar.

`-moz-device-pixel-ratio`

The number of device pixels per CSS pixel. Use the `resolution` feature with the `dppx` unit instead.

`-webkit-animation`

The browser supports `-webkit` prefixed CSS `animation`. Use the `@supports (animation)` feature query instead.

`-webkit-device-pixel-ratio`

The number of device pixels per CSS pixel. Use the `resolution` feature with the `dppx` unit instead.

`-webkit-transform-2d`

The browser supports `-webkit` prefixed 2D CSS `transform`. Use the `@supports (transform)` feature query instead.

`-webkit-transform-3d`

The browser supports `-webkit` prefixed 3D CSS `transform`. Use the `@supports (transform)` feature query instead.

`-webkit-transition`

The browser supports `-webkit` prefixed CSS `transition`. Use the `@supports (transition)` feature query instead.

Logical operators

The *logical operators* `not`, `and`, `only`, and `or` can be used to compose a complex media query. You can also combine multiple media queries into a single rule by separating them with commas.

`and`

Used for combining multiple media features together into a single media query, requiring each chained feature to return `true` for the query to be `true`. It is also used for joining media features with media types.

`not`

Used to negate a media query, returning `true` if the query would otherwise return `false`. If present in a comma-separated list of queries, it will only negate the specific query to which it is applied.

> **Note:** In Level 3, the `not` keyword can't be used to negate an individual media feature expression, only an entire media query.

`only`

Applies a style only if an entire query matches. It is useful for preventing older browsers from applying selected styles. When not using `only`, older browsers would interpret the query `screen and (width <= 500px)` as `screen`, ignoring the remainder of the query, and applying its styles on all screens. If you use the `only` operator, you *must also* specify a media type.

`,` (comma)

Commas are used to combine multiple media queries into a single rule. Each query in a comma-separated list is treated separately from the others. Thus, if any of the queries in a list is `true`, the entire media statement returns `true`. In other words, lists behave like a logical `or` operator.

`or`

Equivalent to the `,` operator. Added in Media Queries Level 4.

User agent client hints

Some media queries have corresponding user agent client hints. These are HTTP headers that request content that is pre-optimized for the particular media requirement. They include `Sec-CH-Prefers-Color-Scheme` and `Sec-CH-Prefers-Reduced-Motion`.

Formal syntax

```
@media =
  @media <media-query-list> { <rule-list> }
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

To best accommodate people who adjust a site's text size, use `em`s when you need a `<length>` for your media queries.

Both `em` and `px` are valid units, but `em` works better if the user changes the browser text size.

Also consider media queries or HTTP user agent client hints to improve the user's experience. For example, the media query `prefers-reduced-motion` or the equivalent HTTP header `Sec-CH-Prefers-Reduced-Motion` can be used to minimize the amount of animation or motion used based on user preferences.

Security

Because media queries provide insights into the capabilities—and by extension, the features and design—of the device the user is working with, there is the potential that they could be abused to construct a "fingerprint" which identifies the device, or at least categorizes it to some degree of detail that may be undesirable to users.

Because of this potential, a browser may opt to fudge the returned values in some manner in order to prevent them from being used to precisely identify a computer. A browser might also offer additional measures in this area; for example, if Firefox's "Resist Fingerprinting" setting is enabled, many media queries report default values rather than values representing the actual device state.

Examples

Testing for print and screen media types

```
@media print {
  body {
    font-size: 10pt;
  }
}

@media screen {
  body {
    font-size: 13px;
  }
}

@media screen, print {
  body {
    line-height: 1.2;
  }
}
```

The range syntax allows for less verbose media queries when testing for any feature accepting a range, as shown in the below examples:

```
@media (height > 600px) {
  body {
    line-height: 1.4;
  }
}

@media (400px <= width <= 700px) {
  body {
    line-height: 1.4;
  }
}
```

For more examples, please see Using media queries.

See also

- CSS media queries module
- Using media queries
- `CSSMediaRule` interface
- CSS `@custom-media` at-rule
- Extended Mozilla media features
- Extended WebKit media features

-moz-device-pixel-ratio CSS media feature

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-moz-device-pixel-ratio`** Gecko-only CSS media feature can be used to apply styles based on the number of device pixels per CSS pixel.

> **Warning:** Do not use this feature. Use the `resolution` feature with the `dppx` unit instead.

> **Note:** This media feature is also implemented by WebKit as `-webkit-device-pixel-ratio`. The min and max prefixes as implemented by Gecko are named `min--moz-device-pixel-ratio` and `max--moz-device-pixel-ratio`; but the same prefixes as implemented by WebKit are named `-webkit-min-device-pixel-ratio` and `-webkit-max-device-pixel-ratio`.

Syntax

`<number>`

The number of device pixels per CSS pixel.

Media: `@media` **Accepts min/max prefixes:** yes

Examples

Basic compatibility example

`-moz-device-pixel-ratio` may be used for compatibility with Firefox older than 16, and alongside `-webkit-device-pixel-ratio` for compatibility with WebKit-based browsers that do not support `dppx`.

Example:

```
/* First, set for WebKit-based browsers */
@media (-webkit-min-device-pixel-ratio: 2),
  (min--moz-device-pixel-ratio: 2) /* Older Firefox browsers (prior to firefox 16) */,
  (min-resolution: 2dppx) /* The standard way */,
  (min-resolution: 192dpi); /* dppx fallback */
```

> **Note:** See this CSSWG article for compatibility good practices regarding `resolution` and `dppx`.

Specifications

Not part of any standard.

See also

- Using Media Queries
- @media

-webkit-animation CSS media feature

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** All browsers support the `animation` property without vendor prefixes. Only WebKit (Safari), and not Chromium, based browsers supports the `-webkit-animation` media feature. No browsers support `animation`, without the prefix, as a media query. Use the `@supports (animation)` feature query instead.

The **`-webkit-animation`** Boolean CSS media feature is a WebKit extension whose value is `true` if vendor-prefixed CSS `animation`s are supported.

Apple has a description in Safari CSS Reference.

> **Note:** This media feature is only supported by WebKit. If possible, use an `@supports` feature query instead.

Syntax

The `-webkit-animation` media feature is a Boolean whose value is `true` if the vendor-prefixed CSS animation properties are supported *and* the browser supports prefixed property media queries.

Values

`true`

The browser supports `-webkit` prefixed CSS `animation`.

`false`

The browser doesn't support these prefixed CSS animations.

Examples

Example of -webkit-animation

```
@media (-webkit-animation) {
  /* CSS to use if -webkit- prefixed animations are supported AND the browser supports prefixed properties as media queries */
}
```

Specifications

Not part of any standard.

See also

- `-webkit-transform-3d`
- `-webkit-transform-2d`
- `-webkit-transition`
- `animation` and using CSS animations
- `@media` and Using media queries
- `@supports` and using feature queries

-webkit-device-pixel-ratio CSS media feature

The **`-webkit-device-pixel-ratio`** is a non-standard Boolean CSS media feature which is an alternative to the standard `resolution` media feature.

> **Note:** If possible, use the `resolution` media feature query instead, which is a standard media feature. While this prefixed media feature is a WebKit feature, other browser engines may support it. See browser compatibility below.

Syntax

The `-webkit-device-pixel-ratio` feature is specified as a `<number>` value. It is a range feature, meaning that you can also use the prefixed **`-webkit-min-device-pixel-ratio`** and **`-webkit-max-device-pixel-ratio`** variants to query minimum and maximum values, respectively.

Values

`<number>`

The number of device pixels used to represent each CSS `px`. Although the value is a `<number>`, and thus doesn't syntactically allow units, its implicit unit is `dppx`.

Implementation

```
/* A unit of "dppx" is implied: */
@media (-webkit-min-device-pixel-ratio: 2) {
  /* … */
}
/* It is equivalent to: */
@media (min-resolution: 2dppx) {
  /* … */
}

/* Similarly: */
@media (-webkit-max-device-pixel-ratio: 2) {
  /* … */
}
/* It is equivalent to: */
```

```
@media (max-resolution: 2dppx) {
  /* … */
}
```

Examples

HTML

```
<p>This is a test of your device's pixel density.</p>
```

CSS

```
/* Exact resolution */
@media (-webkit-device-pixel-ratio: 1) {
  p {
    color: red;
  }
}

/* Minimum resolution */
@media (-webkit-min-device-pixel-ratio: 1.1) {
  p {
    font-size: 1.5em;
  }
}

/* Maximum resolution */
@media (-webkit-max-device-pixel-ratio: 3) {
  p {
    background: yellow;
  }
}
```

Result

See also

- Using media queries
- `<resolution>`
- `-webkit-transform-2d`
- `-webkit-transform-3d`
- `-webkit-transition`
- `-webkit-animation`
- W3C Suggested Method to UnPrefix

-webkit-transform-2d CSS media feature

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** All browsers support the `transform` property without vendor prefixes. Only WebKit (Safari), and not Chromium, based browsers supports the `-webkit-transform-2d` media feature. No browsers support `transform`, without the prefix or `2d` extension, as a media query. Use the `@supports (transform)` feature query instead.

The **`-webkit-transform-2d`** Boolean CSS media feature is a WebKit extension whose value is `true` if vendor-prefixed CSS 2D `transform`s and non-standard vendor-prefixed media queries are supported.

Apple has a description in Safari CSS Reference.

Syntax

`-webkit-transform-2d` is a Boolean CSS media feature whose value is `true` if the browser supports `-webkit` prefixed CSS 2D `transform`s.

Values

`true`

The browser supports the 2D CSS transforms with the `-webkit` prefix.

`false`

The 2D CSS transforms prefixed with `-webkit` are not supported by the browser.

Examples

Basic example

```
@media (-webkit-transform-2d) {
  div {
    -webkit-transform: translate(100px, 100px);
  }
}
```

This media feature is only supported by WebKit. The unprefixed `transform` property is supported in all modern browsers. If possible, use an `@supports` feature query instead:

```
@supports (-webkit-transform: translate(100px, 100px)) {
  div {
    -webkit-transform: translate(100px, 100px);
  }
}
```

Specifications

Not part of any standard.

See also

- `-webkit-transform-3d`
- `-webkit-transition`
- `-webkit-animation`
- `transform` and using CSS transforms
- `@media` and Using media queries
- `@supports` and using feature queries

-webkit-transform-3d CSS media feature

The **`-webkit-transform-3d`** Boolean CSS media feature is a WebKit extension whose value is `true` if vendor-prefixed CSS 3D `transform`s are supported.

> **Note:** While this media feature is currently supported by most browsers. If possible, use an `@supports` feature query instead.

Syntax

`-webkit-transform-3d` is a Boolean CSS media feature whose value is `true` if the browser supports `-webkit` prefixed CSS 3D `transform`s.

Values

`true`

The browser supports the 3D CSS transforms with the `-webkit` prefix and supports non-standard, prefixed media queries.

`false`

The 3D CSS transforms prefixed with `-webkit` are not supported by the browser.

Examples

Examples of media queries with -webkit-transform-3d

```
@media (-webkit-transform-3d) {
  .foo {
    transform-style: preserve-3d;
  }
}
```

A better method for checking for browser support is using a feature query:

```
@supports (transform-style) {
  .foo {
    transform-style: preserve-3d;
  }
}
```

See also

- `-webkit-transform-2d`
- `-webkit-transition`
- `-webkit-animation`
- `transform` and using CSS transforms
- `@media` and Using media queries
- `@supports` and using feature queries

-webkit-transition CSS media feature

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** All browsers support the `transition` property without vendor prefixes. Only WebKit (Safari), and not Chromium, based browsers support the `-webkit-transition` media feature. No browsers support `transition` without the prefix as a media query (though some browsers do support - `-webkit-transform-3d`). Use the `@supports (transition)` feature query instead.

The **`-webkit-transition`** Boolean non-standard CSS media feature is a WebKit extension whose value is `true` if the browsing context supports CSS transitions.

Apple has a description in Safari CSS Reference; this is now called `transition` there.

> **Note:** You should not use this media feature; it was never specified, has never been widely implemented, and has been removed from most browsers. Use a `@supports` feature query instead.

Syntax

```
@media (-webkit-transition) {
  /* CSS to use if this media feature and prefixed transitions are supported */
}
```

Examples

Use @supports instead

Do not use the `-webkit-transition` media feature. Instead, test for transition support using the CSS `@supports` at-rule, like this:

```
@supports (transition: initial) {
  /* CSS to use if transitions are supported */
}
```

Obsolete example

Before this became obsolete, you could use `-webkit-transition` in your CSS like this:

```
@media (-webkit-transition) {
  /* CSS to use if transitions are supported */
}
```

Specifications

Not part of any standard.

See also

- `-webkit-transform-3d`
- `-webkit-transform-2d`
- `-webkit-animation`
- `transition` and using CSS transitions
- `@media` and Using media queries
- `@supports` and using feature queries

any-hover CSS media feature

The **`any-hover`** CSS media feature can be used to test whether *any* available input mechanism can hover over elements.

Syntax

The `any-hover` feature is specified as a keyword value chosen from the list below.

`none`

None of the available input mechanism(s) can hover conveniently, or there is no pointing input mechanism.

`hover`

One or more available input mechanisms can conveniently hover over elements.

Examples

Testing whether input methods can hover

HTML

```
<a href="#">Try hovering over me!</a>
```

CSS

```
@media (any-hover: hover) {
  a:hover {
    background: yellow;
  }
}
```

Result

See also

- the `hover` media feature

any-pointer CSS media feature

The **`any-pointer`** CSS media feature tests whether the user has *any* pointing device (such as a mouse), and if so, how accurate it is.

> **Note:** If you want to test the accuracy of the *primary* pointing device, use `pointer` instead.

Syntax

The `any-pointer` feature is specified as a keyword value chosen from the list below.

`none`

No pointing device is available.

`coarse`

At least one input mechanism includes a pointing device of limited accuracy.

`fine`

At least one input mechanism includes an accurate pointing device.

> **Note:** More than one value can match if the available devices have different characteristics, although `none` only matches when none of them are pointing devices.

Examples

This example creates a small checkbox for users with at least one fine pointer and a large checkbox for users with at least one coarse pointer. The big checkbox takes precedence because it is declared after the small one.

HTML

```
<input id="test" type="checkbox" /> <label for="test">Look at me!</label>
```

CSS

```
input[type="checkbox"]:checked {
  background: gray;
}

@media (any-pointer: fine) {
```

```
  input[type="checkbox"] {
    appearance: none;
    width: 15px;
    height: 15px;
    border: 1px solid blue;
  }
}

@media (any-pointer: coarse) {
  input[type="checkbox"] {
    appearance: none;
    width: 30px;
    height: 30px;
    border: 2px solid red;
  }
}
```

Result

See also

- The `pointer` media feature

aspect-ratio CSS media feature

The **`aspect-ratio`** CSS media feature can be used to test the aspect ratio of the viewport.

Syntax

The `aspect-ratio` feature is specified as a `<ratio>` value representing the width-to-height aspect ratio of the viewport. It is a range feature, meaning you can also use the prefixed **`min-aspect-ratio`** and **`max-aspect-ratio`** variants to query minimum and maximum values, respectively.

Examples

In the example below, a `<div>` element is contained in an `<iframe>`. The iframe creates its own viewport. Resize the `<iframe>` to see `aspect-ratio` in action.

Note that, when none of the media query conditions are true, the background will turn white because none of the below rules will be applied to the `<div>` inside the `<iframe>`. See if you can find which width and height values trigger this!

HTML

```
<iframe
  id="outer"
  srcdoc="<div id='inner'>Watch this element as you resize iframe viewport's width and height.</div>">
</iframe>
```

CSS

```
/* Minimum allowed aspect ratio */
/* Select aspect ratios 8/5 = 1.6 and above */
@media (min-aspect-ratio: 8/5) {
  div {
    background: #9999ff; /* blue */
  }
}

/* Maximum allowed aspect ratio */
/* Select aspect ratios 3/2 = 1.5 and below */
@media (max-aspect-ratio: 3/2) {
  div {
    background: #99ff99; /* green */
  }
}

/* Exact aspect ratio, put it at the bottom to avoid override */
@media (aspect-ratio: 1/1) {
  div {
    background: #ff9999; /* red */
  }
}
```

Result

```
<label id="wf" for="w">width:165</label>
<input id="w" name="w" type="range" min="100" max="250" step="5" value="165" />
<label id="hf" for="w">height:165</label>
<input id="h" name="h" type="range" min="100" max="250" step="5" value="165" />
<label id="ratio">aspect-ratio: 165/165 = 1</label>

<iframe
  id="outer"
  srcdoc="<style> @media (min-aspect-ratio: 8/5) { div { background: #9999ff; } } @media (max-aspect-ratio: 3/2) { div { background: #99ff99; } } @media (aspect-ratio: 1/1) { div { background:
#ff9999; } }</style><div id='inner'> Watch this element as you resize iframe viewport's width and height.</div>">
</iframe>
```

```
iframe {
  display: block;
  border: 1px dashed black;
}
```

```
outer.style.width = outer.style.height = "165px";

const updateRatio = () => {
  ratio.textContent = `aspect-ratio: ${w.value}/${h.value} = ${(w.value / h.value).toFixed(2)}`;
};

w.onchange = w.oninput = () => {
  outer.style.width = `${w.value}px`;
  wf.textContent = `width: ${w.value}`;
  updateRatio();
```

```
};

h.onchange = h.oninput = () => {
  outer.style.height = `${h.value}px`;
  hf.textContent = `height: ${h.value}`;
  updateRatio();
};
```

Note the `min-aspect-ratio: 8/5` sets the *lower* bound to 1.6, so this media query selects elements with aspect ratios 1.6 and above. The `max-aspect-ratio: 3/2` sets the *upper* bound, so this media query selects elements with aspect ratios 1.5 and below. The `aspect-ratio: 1/1` overrides the max aspect ratio rule because it has been declared after and selects elements with aspect ratios exactly `1`.

From the initial state, as you reduce height, the aspect ratio starts increasing from one. So the div's background color goes from red(1) < green(1.5) < white < blue(1.6).

See also

- Understanding `aspect-ratio`
- Using Media Queries
- @media

color CSS media feature

The **`color`** CSS media feature can be used to test the number of bits per color component (red, green, blue) of the output device.

Syntax

The `color` feature is specified as an `<integer>` value that represents the number of bits per color component (red, green, blue) of the output device. If the device is not a color device, the value is zero. It is a range feature, meaning that you can also use the prefixed **`min-color`** and **`max-color`** variants to query minimum and maximum values, respectively.

> **Note:** If the various color components are represented by different numbers of bits, the smallest number is used. For example, if a display uses 5 bits for blue and red and 6 bits for green, then the device is considered to use 5 bits per color component. If the device uses indexed colors, the minimum number of bits per color component in the color table is used.

See Applying color to HTML elements using CSS to learn more about using CSS to apply color to HTML.

Examples

HTML

```
<p>
  This text should be black on non-color devices, red on devices with a low
  number of colors, and greenish on devices with a high number of colors.
</p>
```

CSS

```
p {
  color: black;
}

/* Any color device */
@media (color) {
  p {
    color: red;
  }
}

/* Any color device with at least 8 bits per color component */
@media (min-color: 8) {
  p {
    color: #24ba13;
  }
}
```

Result

See also

- The CSS `color` property
- The CSS `<color>` data unit

color-gamut CSS media feature

The **`color-gamut`** CSS media feature is used to apply CSS styles based on the approximate range of color gamut supported by the user agent and the output device.

Syntax

The `color-gamut` feature is specified as one of the following color spaces as keyword values:

`srgb`

> The user agent and the output device can support approximately the sRGB gamut or more. This includes the vast majority of color displays.

`p3`

> The user agent and the output device can support approximately the gamut specified by the Display P3 color space or more. The P3 gamut is larger than and includes the sRGB gamut.

`rec2020`

> The user agent and the output device can support approximately the gamut specified by the ITU-R Recommendation BT.2020 color space or more. The REC. 2020 gamut is larger than and includes the P3 gamut.

Examples

HTML

```
<p>This is a test.</p>
```

CSS

```
p {
  padding: 10px;
  border: solid;
}

@media (color-gamut: srgb) {
  p {
    background: #f4ae8a;
  }
}
```

Result

See also

- `color()` function to specify colors in a defined colorspace.
- CSS colors module
- `@media` at-rule that is used to specify the color-gamut expression.
- Using media queries to understand when and how to use a media query.
- CSS media queries module
- sRGB on Wikipedia

color-index CSS media feature

The **`color-index`** CSS media feature can be used to test the number of entries in the output device's color lookup table.

Syntax

The `color-index` feature is specified as an `<integer>` value representing the number of entries in the output device's color lookup table. (This value is zero if the device does not use such a table.) It is a range feature, meaning that you can also use the prefixed **`min-color-index`** and **`max-color-index`** variants to query minimum and maximum values, respectively.

Examples

Basic example

HTML

```
<p>This is a test.</p>
```

CSS

```
p {
  color: black;
}

@media (color-index) {
  p {
    color: red;
  }
}

@media (min-color-index: 15000) {
  p {
    color: #1475ef;
  }
}
```

Result

Custom stylesheet

This HTML will apply a special stylesheet for devices that have at least 256 colors.

```
<link rel="stylesheet" href="https://cdn.example.com/base.css" />
<link
  rel="stylesheet"
  media="(color-index >= 256)"
  href="https://cdn.example.com/color-stylesheet.css" />
```

See also

- Using Media Queries
- @media

device-aspect-ratio CSS media feature

> **Note:** To query for the aspect ratio of the viewport, developers should use the `aspect-ratio` media feature instead.

The **`device-aspect-ratio`** CSS media feature can be used to test the width-to-height aspect ratio of an output device.

Syntax

The `device-aspect-ratio` feature is specified as a `<ratio>`. It is a range feature, meaning that you can also use the prefixed **`min-device-aspect-ratio`** and **`max-device-aspect-ratio`** variants to query minimum and maximum values, respectively.

Examples

Using min-device-aspect-ratio

```
article {
  padding: 1rem;
}

@media screen and (min-device-aspect-ratio: 16/9) {
  article {
```

```
    padding: 1rem 5vw;
  }
}
```

device-height CSS media feature

> **Note:** To query for the height of the viewport, developers should use the `height` media feature instead.

The **`device-height`** CSS media feature can be used to test the height of an output device's rendering surface.

Syntax

The `device-height` feature is specified as a `<length>` value. It is a range feature, meaning that you can also use the prefixed **`min-device-height`** and **`max-device-height`** variants to query minimum and maximum values, respectively.

Examples

Applying a special stylesheet for devices that are shorter than 800 pixels

```
<link
  rel="stylesheet"
  media="screen and (max-device-height: 799px)"
  href="https://cdn.example.com/short-styles.css" />
```

See also

- Using Media Queries
- @media

device-posture CSS media feature

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`device-posture`** CSS media feature can be used to detect the device's current posture, that is, whether the viewport is in a flat (`continuous`) or folded state (`folded`).

Syntax

The `device-posture` feature is specified as a keyword value chosen from the list below:

`continuous`

> Indicates a flat screen state. Foldable devices are `continuous` while they are flat; either fully opened or fully closed. Non-foldable devices are considered flat and therefore always `continuous` — this includes seamless curved displays and standard desktop, laptop, tablet, and mobile screens.

`folded`

> Indicates a folded screen state. Foldable devices are `folded` while used in a book or laptop posture.

Examples

In this example, the `device-posture` media feature detects when a device is in a folded posture, adding a margin based on its `orientation` to create a larger gutter between the application's two panels for easier reading.

```
@media (device-posture: folded) and (orientation: landscape) {
  .list-view {
    margin-inline-end: 60px;
  }
}

@media (device-posture: folded) and (orientation: portrait) {
  .list-view {
    margin-block-end: 60px;
  }
}
```

To see the above code in action, view the Device Posture API demo on a foldable device if possible. Current browser developer tools enable emulating foldable devices, but don't include emulation of partially folded devices — only fully open or closed devices — so they will always return `continuous`.

See also

- `DevicePosture`
- Device Posture API
- Using Media Queries
- @media

device-width CSS media feature

> **Note:** To query for the width of the viewport, developers should use the `width` media feature instead.

The **`device-width`** CSS media feature can be used to test the width of an output device's rendering surface.

Syntax

The `device-width` feature is specified as a `<length>` value. It is a range feature, meaning that you can also use the prefixed **`min-device-width`** and **`max-device-width`** variants to query minimum and maximum values, respectively.

Examples

Applying a special stylesheet for devices that are narrower than 800 pixels

```
<link
  rel="stylesheet"
```

```
  media="screen and (max-device-width: 799px)"
  href="https://cdn.example.com/narrow-styles.css" />
```

See also

- Using Media Queries
- @media

display-mode CSS media feature

The **`display-mode`** CSS media feature can be used to test whether a web app is being displayed in a normal browser tab or in some alternative way, such as a standalone app or fullscreen mode.

For example:

- A progressive web app can set its display mode by setting the `display` member in its manifest. In this case, the value of `display-mode` identifies the value that was set (but note that this may not be the same as the value requested in the manifest, since a browser may not support the requested mode).
- Any web app can use the Fullscreen API or the Document Picture-in-Picture API to set the display mode, and in this case the value of `display-mode` identifies the mode that was set.

The `display-mode` value applies to the top-level browsing context and any child browsing contexts.

Syntax

The `display-mode` feature is specified as a keyword value chosen from the list below.

`browser`

The application opens in a conventional browser tab or new window, depending on the browser and platform.

`fullscreen`

All of the available display area is used and no user agent chrome is shown. This can be used to apply CSS only when the app has been put in fullscreen mode by the Fullscreen API or by using the `fullscreen` value of the `display` member of the Wep App Manifest.

`minimal-ui`

The application will look and feel like a standalone application, but will have a minimal set of UI elements for controlling navigation. The elements will vary by browser.

`picture-in-picture`

This mode allows users to continue consuming specific content while they interact with other sites or applications on their device. The app is displayed in a floating and always-on-top window. This can be used to apply CSS only when the app has been put in Picture-in-Picture mode by the Document Picture-in-Picture API.

`standalone`

The application will look and feel like a standalone application. This can include the application having a different window, its own icon in the application launcher, etc. In this mode, the user agent will exclude UI elements for controlling navigation, but can include other UI elements such as a status bar.

`window-controls-overlay`

In this mode, the application looks and feels like a standalone desktop application, and the Window Controls Overlay feature is enabled.

Examples

Apply CSS if the application is in fullscreen mode

```
@media (display-mode: fullscreen) {
  body {
    margin: 0;
    border: 5px solid black;
  }
}
```

Provide a light and dark color scheme to Picture-in-Picture content

In this example, we combine the `display-mode: picture-in-picture` value with the `prefers-color-scheme` media feature to create light and dark color schemes that are applied based on the user's color scheme preference, only when the app is being shown in Picture-in-Picture mode.

```
@media (display-mode: picture-in-picture) and (prefers-color-scheme: light) {
  body {
    background: antiquewhite;
  }
}

@media (display-mode: picture-in-picture) and (prefers-color-scheme: dark) {
  body {
    background: #333333;
  }

  a {
    color: antiquewhite;
  }
}
```

See Using the Document Picture-in-Picture API for more information and a full example.

See also

- Using Media Queries
- @media

dynamic-range CSS media feature

The **`dynamic-range`** CSS media feature can be used to test the combination of brightness, contrast ratio, and color depth that are supported by the user agent and the output device.

> **Note:** Some devices have high dynamic range capabilities that are not always 'on' and need to be activated (sometimes programmatically, sometimes by the user, sometimes based on the content). This media feature does not test whether the dynamic range capability is active; it only tests whether the device is capable of high dynamic range visuals.

Syntax

The `dynamic-range` feature is specified as a keyword value chosen from the list below.

`standard`

> This value matches any visual device and excludes devices lacking visual capabilities. A user agent or an output device that matches `high` will also match the `standard` value.

`high`

> This value matches user agents and output devices that support high peak brightness, high contrast ratio, and color depth greater than 24 bit or 8 bit per color component of RGB. **Peak brightness** refers to how bright the brightest point a light-emitting device, such as an LCD screen, can produce. In the case of a light-reflective device, such as paper or e-ink, peak brightness refers to the point that at least absorbs light. **Contrast ratio** refers to the ratio of the luminance of the brightest color to that of the darkest color that the system is capable of producing. Currently, there is no precise way to measure peak brightness and contrast ratio, and the determination of what counts as high peak brightness and high contrast ratio depends on the user agent.

Examples

```
@media (dynamic-range: standard) {
  p {
    color: red;
  }
}

@media (dynamic-range: high) {
  p {
    color: green;
  }
}
```

See also

- Using Media Queries
- @media

forced-colors CSS media feature

The **`forced-colors`** CSS media feature is used to detect if the user agent has enabled a forced colors mode where it enforces a user-chosen limited color palette on the page. An example of a forced colors mode is Windows High Contrast mode.

Syntax

The `forced-colors` media feature indicates whether or not the browser is currently in forced-colors mode.

Values

`none`

> Forced colors mode is not active; the page's colors are not being forced into a limited palette.

`active`

> Indicates that forced colors mode is active. The browser provides the color palette to authors through the CSS system color keywords and, if appropriate, triggers the appropriate value of `prefers-color-scheme` so that authors can adapt the page. The browser selects the value for `prefers-color-scheme` based on the lightness of the `Canvas` system color (see the color adjust spec for more details).

Usage notes

Properties affected by forced-color mode

In forced colors mode, the values of the following properties are treated as if they have no author-level values specified. That is, browser-specified values are used instead. The browser-specified values do not affect the style cascade; the values are instead forced by the browser at paint time.

These browser-specified values are selected from the set of system colors — this ensures a consistent contrast for common UI elements for users that have forced colors enabled.

- `color`
- `background-color`
- `text-decoration-color`
- `text-emphasis-color`
- `border-color`
- `outline-color`
- `column-rule-color`
- `-webkit-tap-highlight-color`
- SVG fill attribute
- SVG stroke attribute

Additionally the following properties have special behavior in forced colors mode:

- `box-shadow` is forced to 'none'
- `text-shadow` is forced to 'none'
- `background-image` is forced to 'none' for values that are not url-based
- `color-scheme` is forced to 'light dark'
- `scrollbar-color` is forced to 'auto'

The system colors that are forced for the above properties depend on the context of the element. For example the `color` property on button element will be forced to `ButtonText`. On normal text it will be forced to `CanvasText`. See the list of system colors for additional details of when each might be appropriate in various UI contexts.

> **Note:** User agents choose system colors based on native element semantics, *not* on added ARIA roles. As an example, adding `role="button"` to a `div` will **not** cause an element's color to be forced to `ButtonText`

In addition to these adjustments, browsers will help ensure text legibility by drawing "backplates" behind text. This is particularly important for preserving contrast when text is placed on top of images.

There are some cases where the user agent does not force the values for the above properties:

When `forced-color-adjust` is set to `none` on an element, none of the forced color values will apply, and author styles will be applied as normal. Additionally, the backplate for text will be disabled.

When `forced-color-adjust` is set to `preserve-parent-color` on an element, and the `color` value on the element does not inherit from its parent, then the element will behave the same as setting `preserve-parent-color` to `none`.

When a system color is specified, it will be used instead of the value that would otherwise have been forced.

You can also use system colors with any property *other* than those listed above, to ensure that the rest of the page integrates with the restricted color palette available in forced colors mode.

Accessibility concerns

In general, web authors should **not** be using the `forced-colors` media feature to create a separate design for users with this feature enabled. Instead, its intended usage is to make small tweaks to improve usability or legibility when the default application of forced colors does not work well for a given portion of a page.

The high contrast provided by forced colors mode's reduced palette and text backplates is often essential for some users to be able to read or use a given website, so adjustments that affect content should be chosen carefully and targeted to content that is otherwise not legible.

User preferences

This media feature is active only if the user has enabled color scheme preferences in their operating system. One example of such a feature is High Contrast mode on Windows.

Examples

> **Note:** The below example will only work when using a browser that supports this media feature, and with a preference such as High Contrast mode enabled in your OS.

This example is a button that normally gets its contrast via `box-shadow`. Under forced colors mode, box-shadow is forced to none, so the example uses the forced-colors media feature to ensure there is a border of the appropriate color (ButtonText in this case)

HTML

```
<button class="button">Press me!</button>
```

CSS

```
.button {
  border: 0;
  padding: 10px;
  box-shadow:
    -2px -2px 5px gray,
    2px 2px 5px gray;
}

@media (forced-colors: active) {
  .button {
    /* Use a border instead, since box-shadow is forced to 'none' in forced-colors mode */
    border: 2px ButtonText solid;
  }
}
```

Result

See also

- @media
- Styling for Windows high contrast with standards for forced colors.
- `forced-color-adjust`

grid CSS media feature

The **`grid`** CSS media feature can be used to test whether the output device uses a grid-based screen.

Most modern computers and smartphones have bitmap-based screens. Examples of grid-based devices include text-only terminals and basic phones with only one fixed font.

Syntax

The `grid` feature is specified as a value representing whether the output device is grid-based — `1` if it is, and `0` if not.

Examples

HTML

```
<p class="unknown">I don't know if you're using a grid device. :-(</p>
<p class="bitmap">You are using a bitmap device.</p>
<p class="grid">You are using a grid device! Neato!</p>
```

CSS

```
:not(.unknown) {
  color: lightgray;
}

@media (grid: 0) {
  .unknown {
    color: lightgray;
  }

  .bitmap {
    color: red;
    text-transform: uppercase;
  }
}

@media (grid: 1) {
  .unknown {
    color: lightgray;
  }

  .grid {
    color: black;
    text-transform: uppercase;
  }
}
```

Result

See also

- Using Media Queries
- @media

height CSS media feature

The **`height`** CSS media feature can be used to apply styles based on the height of the viewport (or the page box, for paged media).

Syntax

The `height` feature is specified as a `<length>` value representing the viewport height. It is a range feature, meaning that you can also use the prefixed **`min-height`** and **`max-height`** variants to query minimum and maximum values, respectively.

Examples

HTML

```
<div>Watch this element as you resize your viewport's height.</div>
```

CSS

```
/* Exact height */
@media (height: 360px) {
  div {
    color: red;
  }
}

/* Minimum height */
@media (min-height: 25rem) {
  div {
    background: yellow;
  }
}

/* Maximum height */
@media (max-height: 40rem) {
  div {
    border: 2px solid blue;
  }
}
```

Result

See also

- Using media queries
- @media

horizontal-viewport-segments CSS media feature

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`horizontal-viewport-segments`** CSS media feature detects whether the device has a specified number of viewport segments laid out horizontally (side by side).

Related to the Viewport Segments API, the `vertical-viewport-segments` feature can be used to create responsive designs that work well on multi-viewport devices — devices with a display that is divided into logically separate viewport segments, such as foldable or hinged devices.

Syntax

The `horizontal-viewport-segments` feature is specified as an `<integer>` value of `1` or greater, representing the number of horizontal viewport segments the device has.

- The value will be `1` for:
  - A non-foldable device (for example, a standard single-screen smartphone or tablet)
  - A foldable device that is either unfolded or closed (in the `continuous` device posture).
  - A two-screen hinged device or a foldable device that is currently folded and is oriented vertically so the segments are on top of each other.
- The value will be `2` for a two-screen hinged device or a foldable device that is currently folded (in the `folded` device posture) and is oriented horizontally so the segments are side by side.
- The value may be greater than `2` for foldable devices with more than one fold.

Examples

Basic <code>horizontal-viewport-segments</code> usage

In this snippet we use a `horizontal-viewport-segments` media query to handle the case of foldable devices where the segments are side-by-side.

We set the left container to have a width equal to the left segment width (`env(viewport-segment-width 0 0)`), and the right container to have a width equal to the right segment width (`env(viewport-segment-width 1 0)`).

To calculate how much width the fold takes up in between the two, we subtract the left edge offset of the right container from the right edge offset of the left container (`calc(env(viewport-segment-left 1 0) - env(viewport-segment-right 0 0));`).

```
.wrapper {
  height: 100%;
  display: flex;
}

@media (horizontal-viewport-segments: 2) {
  .wrapper {
    flex-direction: row;
  }

  .list-view {
    width: env(viewport-segment-width 0 0);
```

```
  }

  .fold {
    width: calc(
      env(viewport-segment-left 1 0) - env(viewport-segment-right 0 0)
    );
    background-color: black;
    height: 100%;
  }

  .detail-view {
    width: env(viewport-segment-width 1 0);
  }
}
```

See our Viewport segment API demo for a full working demo (source code). Also check out Using the Viewport Segments API for a full demo explanation.

See also

- `vertical-viewport-segments` `@media` feature
- Viewport Segments API
- Origin trial for Foldable APIs via developer.chrome.com (2024)

hover CSS media feature

The **`hover`** CSS media feature can be used to test whether the user's *primary* input mechanism can hover over elements.

Syntax

The `hover` feature is specified as a keyword value chosen from the list below.

`none`

> The primary input mechanism cannot hover at all or cannot conveniently hover (e.g., many mobile devices emulate hovering when the user performs an inconvenient long tap), or there is no primary pointing input mechanism.

`hover`

> The primary input mechanism can conveniently hover over elements.

Examples

HTML

```
<a href="#">Try hovering over me!</a>
```

CSS

```
/* default hover effect */
a:hover {
  color: black;
  background: yellow;
}

@media (hover: hover) {
  /* when hover is supported */
  a:hover {
    color: white;
    background: black;
  }
}
```

Result

See also

- Using Media Queries
- @media

inverted-colors CSS media feature

The **`inverted-colors`** CSS media feature is used to test if the user agent or the underlying operating system has inverted all colors.

Inversion of colors can have unpleasant side effects, such as shadows turning into highlights, which can reduce the readability of the content. Using this media feature, you can detect if inversion is happening and style the content accordingly while respecting user preference.

Syntax

```
/* Keyword value */
@media (inverted-colors: inverted) {
  /* styles to apply if inversion of colors is detected */
}
```

The `inverted-colors` feature is specified as one of the following keyword values:

`none`

> Indicates that the colors are displayed normally and no inversion of colors has happened. This keyword value evaluates as false.

`inverted`

> Indicates that all pixels within the displayed area have been inverted. This keyword value evaluates as true.

Examples

Applying styles if color inversion is detected

This example demonstrates the effects of both `inverted-colors` media feature keyword values and when the `inverted-colors` media feature is not supported.

HTML

```
<p>
  If color inversion is detected, this text will appear blue on white (the
  inverse of yellow on black) along with a line over the text. If no color
  inversion is happening, the text will appear red on light gray without the
  line over the text.
</p>
<p>
  If the text is gray and no overline is present, it means your browser doesn't
  support the
  <code>inverted-colors</code> media feature.
</p>
```

CSS

```
p {
  color: gray;
}

@media (inverted-colors: inverted) {
  p {
    background: black;
    color: yellow;
    text-decoration: overline;
  }
}

@media (inverted-colors: none) {
  p {
    background: #eeeeee;
    color: red;
  }
}
```

Result

See also

- @media
- CSS media queries module
- Using media queries

monochrome CSS media feature

The **`monochrome`** CSS media feature can be used to test the number of bits per pixel in the monochrome frame buffer of the output device.

Syntax

The `monochrome` feature is specified as an `<integer>` representing the number of bits per pixel in the monochrome frame buffer. If the device is not a monochrome device, the value is zero. It is a range feature, meaning that you can also use the prefixed **`min-monochrome`** and **`max-monochrome`** variants to query minimum and maximum values, respectively.

Examples

HTML

```
<p class="mono">Your device supports monochrome pixels!</p>
<p class="no-mono">Your device doesn't support monochrome pixels.</p>
```

CSS

```
p {
  display: none;
}

/* Any monochrome device */
@media (monochrome) {
  p.mono {
    display: block;
    color: #333333;
  }
}

/* Any non-monochrome device */
@media (monochrome: 0) {
  p.no-mono {
    display: block;
    color: #ee3636;
  }
}
```

Result

orientation CSS media feature

The **`orientation`** CSS media feature can be used to test the orientation of the viewport (or the page box, for paged media).

> **Note:** This feature does not correspond to *device* orientation. Opening the soft keyboard on many devices in portrait orientation will cause the viewport to become wider than it is tall, thereby causing the browser to use landscape styles instead of portrait.

Syntax

The `orientation` feature is specified as a keyword value chosen from the list below.

Keyword values

`portrait`

The viewport is in a portrait orientation, i.e., the height is greater than or equal to the width.

`landscape`

The viewport is in a landscape orientation, i.e., the width is greater than the height.

Examples

Portrait orientation

In this example we have three boxes in the HTML, and use the `orientation` media feature to switch between a row layout (in landscape) and a column layout (in portrait).

The example output is embedded in an `<iframe>` whose height is greater than its width, so the boxes get a column layout.

HTML

```
<div>Box 1</div>
<div>Box 2</div>
<div>Box 3</div>
```

CSS

```
body {
  display: flex;
}

div {
  background: yellow;
  width: 200px;
  height: 200px;
  margin: 0.5rem;
  padding: 0.5rem;
}

@media (orientation: landscape) {
  body {
    flex-direction: row;
  }
}

@media (orientation: portrait) {
  body {
    flex-direction: column;
  }
}
```

Result

Landscape orientation

This example has exactly the same code as the previous example: it has three boxes in the HTML, and uses the `orientation` media feature to switch between a row layout (in landscape) and a column layout (in portrait).

However, in this example, the example output is embedded in an `<iframe>` whose height is less than its width, so the boxes get a row layout.

HTML

```
<div>Box 1</div>
<div>Box 2</div>
<div>Box 3</div>
```

CSS

```
body {
  display: flex;
}

div {
  background: yellow;
  width: 200px;
  height: 200px;
  margin: 0.5rem;
  padding: 0.5rem;
}

@media (orientation: landscape) {
  body {
    flex-direction: row;
  }
}

@media (orientation: portrait) {
  body {
    flex-direction: column;
  }
}
```

overflow-block CSS media feature

The **`overflow-block`** CSS media feature can be used to test how the output device handles content that overflows the initial containing block along the block axis.

> **Note:** The `overflow-block` property does not determine whether overflow occurs; rather, it reveals the device's handling of such overflow. Typically, on screens in most browsers, the behavior will be "scroll": when content exceeds the available vertical space, the device allows you to scroll to access the overflowed content.

Syntax

The `overflow-block` feature is specified as a keyword value chosen from the list below.

`none`

Content that overflows the block axis is not displayed.

`scroll`

Content that overflows the block axis can be seen by scrolling to it.

`optional-paged`

Content that overflows the block axis can be seen by scrolling to it, but page breaks can be manually triggered (such as via `break-inside`, etc.) to cause the following content to display on the following page.

`paged`

Content is broken up into discrete pages; content that overflows one page in the block axis is displayed on the following page.

Examples

HTML

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam ac turpis
  eleifend, fringilla velit ac, aliquam tellus. Vestibulum ante ipsum primis in
  faucibus orci luctus et ultrices posuere cubilia Curae; Nunc velit erat,
  tempus id rutrum sed, dapibus ut urna. Integer vehicula nibh a justo imperdiet
  rutrum. Nam faucibus pretium orci imperdiet sollicitudin. Nunc id facilisis
  dui. Proin elementum et massa et feugiat. Integer rutrum ullamcorper eleifend.
  Proin sit amet tincidunt risus. Sed nec augue congue eros accumsan tincidunt
  sed eget ex.
</p>
```

CSS

```
@media (overflow-block: scroll) {
  p {
    color: red;
  }
}
```

Result

See also

- Using Media Queries
- @media

overflow-inline CSS media feature

The **`overflow-inline`** CSS media feature can be used to test how the output device handles content that overflows the initial containing block along the inline axis.

> **Note:** The `overflow-inline` property does not determine whether overflow occurs; rather, it reveals the device's handling of such overflow. Typically, on screens in most browsers, the behavior will be "scroll": when content exceeds the available horizontal space, the device allows you to scroll to access the overflowed content.

Syntax

The `overflow-inline` feature is specified as a keyword value chosen from the list below.

`none`

Content that overflows the inline axis is not displayed.

`scroll`

Content that overflows the inline axis can be seen by scrolling to it.

Examples

HTML

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam ac turpis
  eleifend, fringilla velit ac, aliquam tellus. Vestibulum ante ipsum primis in
  faucibus orci luctus et ultrices posuere cubilia Curae; Nunc velit erat,
  tempus id rutrum sed, dapibus ut urna. Integer vehicula nibh a justo imperdiet
  rutrum. Nam faucibus pretium orci imperdiet sollicitudin. Nunc id facilisis
  dui. Proin elementum et massa et feugiat. Integer rutrum ullamcorper eleifend.
  Proin sit amet tincidunt risus. Sed nec augue congue eros accumsan tincidunt
  sed eget ex.
</p>
```

CSS

```
p {
  white-space: nowrap;
}

@media (overflow-inline: scroll) {
  p {
    color: red;
  }
}
```

Result

pointer CSS media feature

The **`pointer`** CSS media feature tests whether the user has a pointing device (such as a mouse), and if so, how accurate the *primary* pointing device is.

> **Note:** If you want to test the accuracy of *any* pointing device, use `any-pointer` instead.

Syntax

The `pointer` feature is specified as a keyword value chosen from the list below.

`none`

The primary input mechanism does not include a pointing device.

`coarse`

The primary input mechanism includes a pointing device of limited accuracy, such as a finger on a touchscreen.

`fine`

The primary input mechanism includes an accurate pointing device, such as a mouse.

Examples

This example creates a small checkbox for users with fine primary pointers and a large checkbox for users with coarse primary pointers.

HTML

```
<input id="test" type="checkbox" /> <label for="test">Look at me!</label>
```

CSS

```
input[type="checkbox"] {
  appearance: none;
  border: solid;
  margin: 0;
}

input[type="checkbox"]:checked {
  background: gray;
}

@media (pointer: fine) {
  input[type="checkbox"] {
    width: 15px;
    height: 15px;
    border-width: 1px;
    border-color: blue;
  }
}

@media (pointer: coarse) {
  input[type="checkbox"] {
    width: 30px;
    height: 30px;
    border-width: 2px;
    border-color: red;
  }
}
```

Result

See also

- The `any-pointer` media feature

prefers-color-scheme CSS media feature

The **`prefers-color-scheme`** CSS media feature is used to detect if a user has requested light or dark color themes. A user indicates their preference through an operating system setting (e.g., light or dark mode) or a user agent setting.

Embedded elements

For SVG and iframes, `prefers-color-scheme` lets you set a CSS style for the SVG or iframe based on the `color-scheme` of the parent element in the web page. SVGs must be used embedded (i.e., `<img src="circle.svg" alt="circle" />`) as opposed to inlined in HTML. An example of using `prefers-color-scheme` in SVGs can be found in the "Inherited color scheme in embedded elements" section.

Using `prefers-color-scheme` is allowed in cross-origin `<svg>` and `<iframe>` elements. Cross-origin elements are elements retrieved from a different host than the page that is referencing them. To learn more about SVGs, see the SVG documentation and for more information about iframes, see the iframe documentation.

Syntax

`light`

Indicates that user has notified that they prefer an interface that has a light theme, or has not expressed an active preference.

`dark`

Indicates that user has notified that they prefer an interface that has a dark theme.

Examples

Detecting a dark or light theme

A common usage is to use a light color scheme by default, and then use `prefers-color-scheme: dark` to override the colors to a darker variant. It is also possible to do it the other way around.

This example shows both options: Theme A uses light colors, but can be overridden to dark colors. Theme B uses dark colors, but can be overridden to light colors. In the end, if the browser supports `prefers-color-scheme`, both themes will be light or dark.

HTML

```
<div class="box theme-a">Theme A (initial)</div>
<div class="box theme-a adaptive">Theme A (changed if dark preferred)</div>
<br />
```

```
<div class="box theme-b">Theme B (initial)</div>
<div class="box theme-b adaptive">Theme B (changed if light preferred)</div>
```

CSS

```
div.box {
  display: inline-block;
  padding: 1em;
  margin: 6px;
  outline: 2px solid black;
  width: 12em;
  height: 2em;
  vertical-align: middle;
}
```

Theme A (brown) uses a light color scheme by default, but will switch to a dark scheme based on the media query:

```
.theme-a {
  background: #ddccaa;
  color: #773311;
}
@media (prefers-color-scheme: dark) {
  .theme-a.adaptive {
    background: #775533;
    color: #ddccbb;
    outline: 5px dashed black;
  }
}
```

Theme B (blue) uses a dark color scheme by default, but will switch to a light scheme based on the media query:

```
.theme-b {
  background: #444477;
  color: #bbbbdd;
}
@media (prefers-color-scheme: light) {
  .theme-b.adaptive {
    background: #bbccdd;
    color: #333344;
    outline: 5px dotted black;
  }
}
```

Result

The left boxes shows Theme A and Theme B as they would appear without the `prefers-color-scheme` media query. The right boxes show the same themes, but one of them will be changed to a darker or lighter variant based on the user's active color scheme. The outline of one box will be dashed or dotted if it was changed based on your browser or operating systems settings.

Inherited color scheme in embedded elements

The following example shows how to use the `prefers-color-scheme` media feature in an embedded element to inherit a color scheme from a parent element. A script is used to set the source of the `<img>` elements and their `alt` attributes. This would normally be done in HTML as `<img src="circle.svg" alt="circle" />`.

You should see three circles, with one drawn in a different color. The first circle inherits the `color-scheme` from the OS and can be toggled using the system OS's theme switcher.

The second and third circles inherit the `color-scheme` from the embedding element; the `@media` query allows setting styles of the SVG content based on the parent element's `color-scheme`. In this case, the parent element with a `color-scheme` CSS property is a `<div>`.

```
<div>
  <img alt="circle" src="" />
</div>
<div class="light">
  <img alt="circle" src="" />
</div>
<div class="dark">
  <img alt="circle" src="" />
</div>
```

```
.light {
  color-scheme: light;
}

.dark {
  color-scheme: dark;
}
```

```
// Embed an SVG for all <img> elements
for (const img of document.querySelectorAll("img")) {
  img.src = `${window.btoa(`
    <svg width="32" height="32" viewBox="0 0 32 32" xmlns="http://www.w3.org/2000/svg">
      <style>
        :root { color: blue }
        @media (prefers-color-scheme: dark) {
          :root { color: purple }
        }
      </style>
      <circle fill="currentColor" cx="16" cy="16" r="16"/>
    </svg>
  `)}`;
}
```

See also

- `color-scheme` property
- `<meta name="color-scheme">`
- `Sec-CH-Prefers-Color-Scheme` HTTP Header User Agent Client Hint
- Simulate prefers-color-scheme in Firefox

- Video: Coding a Dark Mode for your Website
- Redesigning your product and website for dark mode
- Changing color schemes in Windows, macOS, Android, or other platforms.

prefers-contrast CSS media feature

The **`prefers-contrast`** CSS media feature is used to detect whether the user has requested the web content to be presented with a lower or higher contrast.

Syntax

`no-preference`

Indicates that the user has made no preference known to the system. This keyword value evaluates as false in the Boolean context.

`more`

Indicates that user has notified the system that they prefer an interface that has a higher level of contrast.

`less`

Indicates that user has notified the system that they prefer an interface that has a lower level of contrast.

`custom`

Indicates that user has notified the system for using a specific set of colors, and the contrast implied by these colors matches neither `more` nor `less`. This value will match the color palette specified by users of `forced-colors: active`.

User preferences

Various operating systems do support such preferences and user agents are likely to rely on the settings provided by the operating system.

Examples

This example includes a box with a dashed `outline` set on it by default. When the `prefers-contrast: more` media query matches, the applied outline is given a higher contrast `solid` style instead.

HTML

```
<div class="contrast">low contrast box</div>
```

CSS

```
.contrast {
  margin: 5px;
  width: 100px;
  height: 100px;
  outline: 2px dashed black;
}

@media (prefers-contrast: more) {
  .contrast {
    outline: 2px solid black;
  }
}
```

Result

See also

- CSS forced-colors media query

prefers-reduced-data CSS media feature

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

> **Warning:** This feature is not supported by any user agent and its specifics are subject to change.

The **`prefers-reduced-data`** CSS media feature is used to detect if the user has requested web content that consumes less internet traffic.

The browser typically achieves this by querying the underlying operating system's preferences for reduced data.

Syntax

`no-preference`

Indicates that the user has made no preference known to the system. This keyword evaluates to false in the Boolean context.

`reduce`

Indicates that the user has expressed a preference for lightweight alternate content.

Examples

Basic usage

In this example, the `montserrat-regular.woff2` font file will neither be preloaded nor downloaded if the user prefers reduced data; in this case, the "system font stack" will serve as the fallback font:

HTML

```
<head>
  <link
    rel="preload"
    href="fonts/montserrat-regular.woff2"
    as="font"
    media="(prefers-reduced-data: no-preference)"
    crossorigin />
```

```
  <link rel="stylesheet" href="style.css" />
</head>
```

CSS

```
@media (prefers-reduced-data: no-preference) {
  @font-face {
    font-family: "Montserrat";
    font-style: normal;
    font-weight: normal;
    font-display: swap;
    /* latin */
    src:
      local("Montserrat Regular"),
      local("Montserrat-Regular"),
      url("fonts/montserrat-regular.woff2") format("woff2");
    unicode-range:
      U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC,
      U+2000-206F, U+2074, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215,
      U+FEFF, U+FFFD;
  }
}

body {
  font-family:
    "Montserrat",
    -apple-system,
    BlinkMacSystemFont,
    "Segoe UI",
    "Roboto",
    "Helvetica",
    "Arial",
    sans-serif;
}
```

See also

- `Save-Data` HTTP header

prefers-reduced-motion CSS media feature

> **Warning:** An embedded example at the bottom of this page has a scaling movement that may be problematic for some readers. Readers with vestibular motion disorders may wish to enable the reduce motion feature on their device before viewing the animation.

The **`prefers-reduced-motion`** CSS media feature is used to detect if a user has enabled a setting on their device to minimize the amount of non-essential motion. The setting is used to convey to the browser on the device that the user prefers an interface that removes, reduces, or replaces motion-based animations.

Such animations can trigger discomfort for those with vestibular motion disorders. Animations such as scaling or panning large objects can be vestibular motion triggers.

Syntax

`no-preference`

Indicates that a user has made no preference known on the device. This keyword value evaluates as false.

`reduce`

Indicates that a user has enabled the setting on their device for reduced motion. The `reduce` keyword value evaluates as true; therefore, `@media (prefers-reduced-motion)` is equivalent to `@media (prefers-reduced-motion: reduce)`.

User preferences

Use the following settings to honor the `reduce` request:

- In GTK/GNOME: Settings > Accessibility > Seeing > turn on "Reduced animation".
  - In older versions of GNOME, GNOME Tweaks > General tab (or Appearance, depending on version) > Animations is turned off.
  - Alternatively, add `gtk-enable-animations = false` to the `[Settings]` block of the GTK 3 configuration file.
  - Additionally, try running `gsettings set org.gnome.desktop.interface enable-animations false` to make Firefox (and other programs relying on GTK version 4) respect the `reduce` setting.
- In Plasma/KDE: System Settings > Workspace Behavior > General Behavior > set "Animation speed" to "Instant".
  - Alternatively, add `AnimationDurationFactor=0` to the `[KDE]` block of `~/.config/kdeglobals`.
  - Or just run `kwriteconfig6 --key AnimationDurationFactor 0` in your terminal.
- In Windows 10: Settings > Ease of Access > Display > Show animations in Windows.
- In Windows 11: Settings > Accessibility > Visual Effects > Animation Effects.
- In macOS up to 15 (Sequoia): System Settings > Accessibility > Display > Reduce motion.
- In macOS 25 (Tahoe) onwards: System Settings > Accessibility > Motion > Reduce motion.
- In iOS: Settings > Accessibility > Motion.
- In Android 9+: Settings > Accessibility > Remove animations.
- In Firefox `about:config`: Add a number preference called `ui.prefersReducedMotion` and set its value to either `0` for full animation or to `1` to indicate a preference for reduced motion. Changes to this preference take effect immediately.

Examples

This example uses a scaling animation to demonstrate `prefers-reduced-motion`. If you enable the setting for reducing motion in the accessibility preferences on your device, the `prefers-reduced-motion` media query will detect your preference and the CSS within the reduced motion rules, with the same specificity but coming later in the CSS source order, will take precedence. As a result, the animation on the box will tone down to the `dissolve` animation, which is a more muted animation that is not a vestibular motion trigger.

Toning down the animation scaling

HTML

```
<div class="animation">animated box</div>
```

CSS

```
.animation {
  animation: pulse 1s linear infinite both;
  background-color: purple;
}

/* Tone down the animation to avoid vestibular motion triggers. */
@media (prefers-reduced-motion: reduce) {
  .animation {
    animation: dissolve 4s linear infinite both;
    background-color: green;
    text-decoration: overline;
  }
}
```

```
.animation {
  color: white;
  font: 1.2em sans-serif;
  width: 10em;
  padding: 1em;
  border-radius: 1em;
  text-align: center;
}

@keyframes pulse {
  0% {
    transform: scale(1);
  }
  25% {
    transform: scale(0.9);
  }
  50% {
    transform: scale(1);
  }
  75% {
    transform: scale(1.1);
  }
  100% {
    transform: scale(1);
  }
}

@keyframes dissolve {
  0% {
    opacity: 1;
  }
  50% {
    opacity: 0.3;
  }
  100% {
    opacity: 1;
  }
}
```

Result

You can enable the setting for reducing motion on your device to view the change in animation scaling. This example uses the background color and the line over the text to visually highlight when the keyframe animation switches in response to the setting being enabled or disabled.

See also

- `Sec-CH-Prefers-Reduced-Motion` HTTP Header User Agent Client Hint
- An introduction to the reduced motion media query on CSS-Tricks (2019)
- Responsive design for motion on WebKit Blog (2017)

prefers-reduced-transparency CSS media feature

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`prefers-reduced-transparency`** CSS media feature is used to detect if a user has enabled a setting on their device to reduce the transparent or translucent layer effects used on the device. Switching on such a setting can help improve contrast and readability for some users.

Syntax

`no-preference`

Indicates that a user has made no preference known on the device. This keyword value evaluates as false in the boolean context.

`reduce`

Indicates that a user has enabled the setting on their device to minimize the amount of transparent or translucent layer effects.

User preferences

Various operating systems provide a preference for reducing transparency, and user agents are likely to rely on these system settings. They may also rely on less explicit signals on platforms which don't offer a specific setting.

- In Windows 10/11: Settings > Personalization > Colors > Transparency effects.
- In macOS: System Settings > Accessibility > Display > Reduce transparency.
- In iOS: Settings > Accessibility > Display & Text Size > Reduce Transparency.

Examples

This example has a translucent box by default. If the setting to reduce transparency is enabled in the accessibility preferences on your device, the translucent box becomes more opaque.

HTML

```
<div class="translucent">translucent box</div>
```

CSS

```
.translucent {
  opacity: 0.4;
}

@media (prefers-reduced-transparency) {
  .translucent {
    opacity: 0.8;
  }
}
```

See also

- CSS prefers-reduced-motion media query
- Using media queries

resolution CSS media feature

The **`resolution`** CSS media feature can be used to test the pixel density of the output device.

Syntax

The `resolution` feature is specified as a `<resolution>` value representing the pixel density of the output device. It is a range feature, meaning that you can also use the prefixed **`min-resolution`** and **`max-resolution`** variants to query minimum and maximum values, respectively.

Examples

HTML

```
<p>This is a test of your device's pixel density.</p>
```

CSS

```
/* Exact resolution with unit `dpi` */
@media (resolution: 150dpi) {
  p {
    color: red;
  }
}

/* Minimum resolution synonym units: `dppx` and `x` */
@media (min-resolution: 2dppx) {
  p {
    text-decoration: underline;
  }
}

@media (min-resolution: 2x) {
  p {
    text-decoration: underline;
  }
}

/* Maximum resolution with unit `dpcm` */
@media (max-resolution: 2dpcm) {
  p {
    background: yellow;
  }
}
```

Result

See also

- `window.devicePixelRatio`
- The `image-resolution` property

scan CSS media feature

The **`scan`** CSS media feature is used to apply CSS styles based on the scanning process of the output device.

Syntax

The `scan` feature is specified as one of the following keyword values:

`interlace`

> The output device uses "interlaced" rendering, where video frames alternate between specifying only the "even" lines on the screen and only the "odd" lines.

`progressive`

> The output device renders content to the screen with no special treatment.

Description

Most modern screens (and all computer screens) use progressive rendering, displaying each screen fully with no special treatment.

Interlacing was used by CRT monitors and some plasma TVs to enable the appearance of faster frames per second (FPS) while reducing bandwidth. With interlacing, video frames alternate between rendering the even lines and the odd lines on the screen, downloading and rendering only half the screen for each frame, exploiting the human image-smoothing ability so the brain simulates a higher FPS broadcast at half the bandwidth cost.

When targeting interlaced screens, avoid very fast movement across the screen and ensure animated details are wider than 1px to reduce flickering.

Examples

HTML

```
<p>This is a test.</p>
```

CSS

```
p {
  padding: 10px;
  border: solid;
}

@media screen and (scan: interlace) {
  p {
    background: #f4ae8a;
  }
}
@media screen and (scan: progressive) {
  p {
    text-decoration: underline;
  }
}
@media not screen and (scan: progressive) {
  p {
    border-style: dashed;
  }
}
@media not screen and (scan: interlace) {
  p {
    color: purple;
  }
}
```

Result

See also

- The @media at-rule, which is used to specify the scan expression.
- Using media queries to understand when and how to use a media query.

scripting CSS media feature

The **`scripting`** CSS media feature can be used to test whether scripting (such as JavaScript) is available.

> **Note:** The detection is done by the browsers based on the user settings. Some browser extensions can implement script blocking using different techniques. In such cases the `scripting` media feature may not work as expected.

Syntax

The `scripting` feature is specified as a keyword value chosen from the list below.

`none`

Scripting is completely unavailable on the current document.

`initial-only`

Scripting is enabled during the initial page load, but not afterwards.

`enabled`

Scripting is supported and active on the current document.

Examples

HTML

```
<p class="script-none">You do not have scripting available. :-(</p>
<p class="script-initial-only">
  Your scripting is only enabled during the initial page load. Weird.
</p>
<p class="script-enabled">You have scripting enabled! :-)</p>
```

CSS

```
p {
  color: lightgray;
}

@media (scripting: none) {
  .script-none {
    color: red;
  }
}

@media (scripting: initial-only) {
  .script-initial-only {
    color: red;
  }
}

@media (scripting: enabled) {
  .script-enabled {
    color: red;
  }
}
```

Result

See also

- @media
- Using media queries

shape CSS media feature

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The `shape` CSS media feature can be used to test the shape of the device to distinguish rectangular and round displays.

Syntax

The `shape` discrete feature is specified as one of two acceptable strings, either `rect` representing a rectangular screen or `round` representing a circular, oval or elliptical screen.

`rect`

The shape is an axis aligned rectangle or square, or a similar shape such as rounded rectangle for which the traditional designs are appropriate.

`round`

The shape is rounded or a similar shape to the circle such as an oval, an ellipse for which distinctively rounded designs are appropriate.

Examples

Basic example

HTML

```
<h1>Hello World!</h1>
```

CSS

```
h1 {
  text-align: left;
}

@media (shape: rect) {
  h1 {
    text-align: left;
  }
}

@media (shape: round) {
  h1 {
    text-align: center;
  }
}
```

Custom stylesheet

This HTML will apply a special stylesheet for devices that have round screens.

```
<head>
  <link rel="stylesheet" href="default.css" />
  <link
    media="screen and (shape: rect)"
    rel="stylesheet"
    href="rectangle.css" />
  <link media="screen and (shape: round)" rel="stylesheet" href="round.css" />
</head>
```

Browser compatibility

Currently, no browsers support this feature.

See also

- Using Media Queries
- @media
- CSS round display module
- Using environment variables
- CSS environment variables module

update CSS media feature

The **`update`** CSS media feature can be used to test how frequently (if at all) the output device is able to modify the appearance of content once rendered.

Syntax

`none`

Once it has been rendered, the layout can no longer be updated. Example: documents printed on paper.

`slow`

The layout may change dynamically according to the usual rules of CSS, but the output device is not able to render or display changes quickly enough for them to be perceived as a smooth animation. Examples: e-book readers or severely underpowered devices.

`fast`

The layout may change dynamically according to the usual rules of CSS, and the output device is not unusually constrained in speed, so regularly-updating things like CSS Animations can be used. Example: computer screens.

Examples

HTML

```
<p>
  If this text animates for you, your browser supports `update` and you are
  using a fast-updating device.
</p>
```

CSS

```
@keyframes jiggle {
  from {
    transform: translateY(0);
  }

  to {
    transform: translateY(25px);
  }
}

@media (update: fast) {
  p {
    animation: 1s jiggle linear alternate infinite;
  }
}
```

Result

See also

- Using Media Queries
- @media

vertical-viewport-segments CSS media feature

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`vertical-viewport-segments`** CSS media feature detects whether the device has a specified number of viewport segments laid out vertically (top to bottom).

Related to the Viewport Segments API, the `vertical-viewport-segments` feature can be used to create responsive designs that work well on multi-viewport devices — devices with a display that is divided into logically separate viewport segments, such as foldable or hinged devices.

Syntax

The `vertical-viewport-segments` feature is specified as an `<integer>` value of `1` or greater, representing the number of vertical viewport segments the device has.

- The value will be `1` for:
  - A non-foldable device (for example, a standard single-screen smartphone or tablet).
  - A foldable device that is either unfolded or closed (in the `continuous` device posture).
  - A two-screen hinged device or a foldable device that is currently folded and is oriented horizontally so the segments are side by side.
- The value will be `2` for a two-screen hinged device or a foldable device that is currently folded (in the `folded` device posture) and is oriented vertically so the segments are on top of one another.
- The value may be greater than `2` for foldable devices with more than one fold.

Examples

Basic <code>vertical-viewport-segments</code> usage

In this snippet we use a `vertical-viewport-segments` media query to handle the case of foldable devices where the segments are top-to-bottom.

We set the top container to have a height equal to the top segment height (`env(viewport-segment-height 0 0)`), and the bottom container to have a height equal to the bottom segment height (`env(viewport-segment-height 0 1)`).

To calculate how much height the fold takes up in between the two, we subtract the bottom edge offset of the top container from the top edge offset of the bottom container (`calc(env(viewport-segment-top 0 1) - env(viewport-segment-bottom 0 0));`).

```
.wrapper {
  height: 100%;
  display: flex;
}

@media (vertical-viewport-segments: 2) {
  .wrapper {
    flex-direction: column;
  }

  .list-view {
    height: env(viewport-segment-height 0 0);
  }

  .fold {
    width: 100%;
    height: calc(
      env(viewport-segment-top 0 1) - env(viewport-segment-bottom 0 0)
    );
    background-color: black;
  }

  .detail-view {
    height: env(viewport-segment-height 0 1);
  }
}
```

See our Viewport segment API demo for a full working demo (source code). Also check out Using the Viewport Segments API for a full demo explanation.

See also

- `horizontal-viewport-segments` `@media` feature
- Viewport Segments API
- Origin trial for Foldable APIs via developer.chrome.com (2024)

video-dynamic-range CSS media feature

The **`video-dynamic-range`** CSS media feature can be used to test the combination of brightness, contrast ratio, and color depth that are supported by the video plane of the user agent and the output device.

Some user agents, including many TVs, render video and graphics in two separate planes (bi-plane) with distinct screen characteristics. The `video-dynamic-range` feature is used to test the characteristics in the video plane.

Syntax

The `video-dynamic-range` feature is specified as a keyword value chosen from the list below.

`standard`

This value matches any visual device and excludes devices lacking visual capabilities. A user agent or an output device that matches `high` will also match the `standard` value.

`high`

This value matches user agents and output devices that support high peak brightness, high contrast ratio, and color depth greater than 24 bit or 8 bit per color component of RGB. **Peak brightness** refers to how bright the brightest point a light-emitting device, such as an LCD screen, can produce. In the case of a light-reflective device, such as paper or e-ink, peak brightness refers to the point that at least absorbs light. **Contrast ratio** refers to the ratio of the luminance of the brightest color to that of the darkest color that the system is capable of producing. Currently, there is no precise way to measure peak brightness and contrast ratio, and the determination of what counts as high peak brightness and high contrast ratio depends on the user agent.

See also

- Using Media Queries
- @media

width CSS media feature

The **`width`** CSS media feature can be used to test the width of the viewport (or the page box, for paged media).

Syntax

The `width` feature is specified as a `<length>` value representing the viewport width. It is a range feature, meaning that you can also use the prefixed **`min-width`** and **`max-width`** variants to query minimum and maximum values, respectively.

Examples

HTML

```
<div>Watch this element as you resize your viewport's width.</div>
```

CSS

```
/* Exact width */
@media (width: 360px) {
  div {
    color: red;
  }
}

/* Minimum width */
@media (min-width: 35rem) {
  div {
    background: yellow;
  }
}

/* Maximum width */
@media (max-width: 50rem) {
  div {
    border: 2px solid blue;
  }
}
```

Result

See also

- Using media queries
- @media

@namespace CSS at-rule

`@namespace` is an at-rule that defines XML namespaces to be used in a CSS style sheet.

Try it

```
@namespace svg url("http://www.w3.org/2000/svg");

a {
  color: orangered;
  text-decoration: underline dashed;
  font-weight: bold;
}

svg|a {
  fill: blueviolet;
  text-decoration: underline solid;
  text-transform: uppercase;
}
```

```
<p>
  <a href="#">This is an ordinary HTML link</a>
</p>

<svg width="250px" viewBox="0 0 250 20" xmlns="http://www.w3.org/2000/svg">
  <a href="#">
    <text x="0" y="15">This is a link created in SVG</text>
  </a>
</svg>
```

Syntax

```
/* Default namespace */
@namespace url("XML-namespace-URL");
@namespace "XML-namespace-URL";

/* Prefixed namespace */
@namespace prefix url("XML-namespace-URL");
@namespace prefix "XML-namespace-URL";
```

Description

The defined namespaces can be used to restrict the universal, type, and attribute selectors to only select elements within that namespace. The `@namespace` rule is generally only useful when dealing with documents containing multiple namespaces—such as HTML with inline SVG or MathML, or XML that mixes multiple vocabularies.

Any `@namespace` rules must follow all `@charset` and `@import` rules, and precede all other at-rules and style declarations in a style sheet.

`@namespace` can be used to define the **default namespace** for the style sheet. When a default namespace is defined, all universal and type selectors (but not attribute selectors, see note below) apply only to elements in that namespace.

The `@namespace` rule can also be used to define a **namespace prefix**. When a universal, type, or attribute selector is prefixed with a namespace prefix, then that selector only matches if the namespace *and* name of the element or attribute matches.

In HTML, known foreign elements will automatically be assigned namespaces. This means that HTML elements will act as though they are in the XHTML namespace (`http://www.w3.org/1999/xhtml`), even if there is no `xmlns` attribute anywhere in the document, and the `<svg>` and `<math>` elements will be assigned their proper namespaces (`http://www.w3.org/2000/svg` and `http://www.w3.org/1998/Math/MathML`, respectively).

> **Note:** In XML, unless a prefix is defined directly on an attribute (*e.g.*, `xlink:href`), that attribute has no namespace. In other words, attributes do not inherit the namespace of the element they're on. To match this behavior, the default namespace in CSS does not apply to attribute selectors.

Formal syntax

```
@namespace =
  @namespace <namespace-prefix>? [ <string> | <url> ] ;

<namespace-prefix> =
  <ident>
```

This syntax reflects the latest standard as per CSS Namespaces Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying default and prefixed namespaces

```
@namespace url("http://www.w3.org/1999/xhtml");
@namespace svg url("http://www.w3.org/2000/svg");

/* This matches all XHTML <a> elements, as XHTML is the default unprefixed namespace */
a {
}

/* This matches all SVG <a> elements */
svg|a {
}

/* This matches both XHTML and SVG <a> elements */
*|a {
}
```

See also

- Namespaces crash course
- CSS at-rule functions

@page CSS at-rule

The **`@page`** at-rule is a CSS at-rule used to modify different aspects of printed pages. It targets and modifies the page's dimensions, orientation, and margins. The `@page` at-rule can be used to target all pages in a print-out or a subset using its various pseudo-classes.

Syntax

```
/* Targets all the pages */
@page {
  size: 8.5in 9in;
  margin-top: 4in;
}

/* Targets all even-numbered pages */
@page :left {
  margin-top: 4in;
}

/* Targets all odd-numbered pages */
@page :right {
  size: 11in;
  margin-top: 4in;
}

/* Targets all selectors with `page: wide;` set */
```

```
@page wide {
  size: a4 landscape;
}

@page {
  /* margin box at top right showing page number */
  @top-right {
    content: "Page " counter(pageNumber);
  }
}
```

Page properties

The `@page` at-rule can contain only page descriptors and margin at-rules. The following descriptors have been implemented by at least one browser:

`margin`

Specifies the page margins. Individual margin properties `margin-top`, `margin-right`, `margin-bottom`, and `margin-left` can also be used.

`page-orientation`

Specifies the orientation of the page. This does not affect the layout of the page; the rotation is applied after the layout in the output medium.

`size`

Specifies the target size and orientation of the page box's containing block. In the general case, where one page box is rendered onto one page sheet, it also indicates the size of the destination page sheet.

The specification mentions following CSS properties to be applicable to page boxes via @page at-rule. But these have *not been supported* by any user agent yet.

Description

The @page rule defines properties of the page box. The `@page` at-rule can be accessed via the CSS object model interface `CSSPageRule`.

> **Note:** The W3C is discussing how to handle viewport-related `<length>` units, `vh`, `vw`, `vmin`, and `vmax`. Meanwhile do not use them within a `@page` at-rule.

Related properties

The `@page` at-rule, allows the user to assign a name to the rule, which is then called in a declaration using the `page` property.

`page`

Allows a selector to use a user-defined **named page**

Formal syntax

```
@page =
  @page <page-selector-list>? { <declaration-rule-list> }

<page-selector-list> =
  <page-selector>#

<page-selector> =
  [ <ident-token>? <pseudo-page>* ]!

<pseudo-page> =
  : [ left | right | first | blank ]
```

This syntax reflects the latest standard as per CSS Paged Media Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Where the `<page-body>` includes:

- page-properties
- page-margin properties

and `<pseudo-page>` represents these pseudo-classes:

- `:blank`
- `:first`
- `:left`
- `:right`

Margin at-rules

The margin at-rules are used inside of the `@page` at-rule. They each target a different section of the document printed page, styling the area of the printed page based on the property values set in the style block:

```
@page {
  @top-left {
    /* page-margin-properties */
  }
}
```

`@top-left` targets the top-left of the document and applies the changes based on the page-margin-properties set.

Other margin-at rules include:

```
@top-left-corner
@top-left
@top-center
@top-right
@top-right-corner
@bottom-left-corner
@bottom-left
@bottom-center
@bottom-right
@bottom-right-corner
@left-top
@left-middle
@left-bottom
@right-top
```

```
@right-middle
@right-bottom
```

Page-margin properties

The page-margin properties are the set of CSS properties can be set in any individual margin at-rule. They include:

Named pages

Named pages enable performing per-page layout and adding page-breaks in a declarative manner when printing.

Named pages can be applied using the `page` property. This allows the user to create different page configurations for use in print layouts.

An example of this can be found on the `page` examples.

Examples

Using the size property to change the page orientation

This example shows how to split the `<section>`s into individual pages in `landscape` format with each page having a 20% margin when printed. Clicking the print button will launch a print dialog with the HTML sections split into individual pages.

```
<button>Print page</button>
<article>
  <section>
    <h2>Header one</h2>
    <p>Paragraph one.</p>
  </section>
  <section>
    <h2>Header two</h2>
    <p>Paragraph two.</p>
  </section>
  <section>
    <h2>Header three</h2>
    <p>Paragraph three.</p>
  </section>
</article>
```

```
const button = document.querySelector("button");

button.addEventListener("click", () => {
  window.print();
});
```

```
@page {
  size: landscape;
  margin: 2cm;
}

section {
  page-break-after: always;
  break-after: page;
}

@media print {
  button {
    display: none;
  }
}
```

```
body {
  font-family: "Helvetica", sans-serif;
  background-color: silver;
}

article {
  width: 100%;
}

section {
  display: grid;
  background-color: white;
  border-radius: 0.6rem;
  justify-items: center;
  padding: 1rem;
  width: 50%;
  print-color-adjust: exact;
  -webkit-print-color-adjust: exact;
  margin: 0 auto;
  margin-block-end: 1rem;
  border: 1px dashed;
}
```

@page pseudo-class examples

See the various pseudo-classes of `@page` for examples.

- `:blank`
- `:first`
- `:left`
- `:right`

See also

- The `page` property
- The `@page` `size` descriptor
- CSS paged media module
- [META] CSS Paged Media Module Level 3 Bugzilla for tracking progress on the subject (page-based counters, etc.)

page-orientation CSS at-rule descriptor

The **`page-orientation`** CSS descriptor for the `@page` at-rule controls the rotation of a printed page. It handles the flow of content across pages when the orientation of a page is changed. This behavior differs from the `size` descriptor in that a user can define the direction in which to rotate the page.

This descriptor helps with the layout and orientation of printed documents, especially when documents are printed double-sided. A user can specify how the pages will be rotated when printed. This is particularly useful to lay out content such as tables, which may be wider than the rest of the content, in a different orientation.

> **Note:** Margin boxes and other positional elements have no special interaction with this descriptor. Margins are laid out as normal in the unrotated page, then rotated along with everything else.

Syntax

```
/* Displays the print content in an upright position */
@page {
  page-orientation: upright;
}

/* Displays the print content rotated counter-clockwise */
@page {
  page-orientation: rotate-left;
}

/* Displays the print content rotated clockwise */
@page {
  page-orientation: rotate-right;
}
```

Values

`upright`

No orientation is applied and the page is laid out and formatted as normal.

`rotate-left`

After a page is laid out, the page must be displayed rotated a quarter turn to the left (counter-clockwise).

`rotate-right`

After the page is laid out, the page must be displayed rotated a quarter turn to the right (clockwise).

Formal definition

Related at-rule	`@page`
Initial value	`upright`
Computed value	as specified

Formal syntax

```
page-orientation =
  upright       |
  rotate-left   |
  rotate-right
```

This syntax reflects the latest standard as per CSS Paged Media Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotating printed pages

This example shows how the contents of a print document can be rotated to suit the page content and the page position. In this first part of the CSS code, named pages are set up to define the direction in which to rotate the content.

```
@page upright {
  size: portrait;
  page-orientation: upright;
}

@page left {
  size: landscape;
  page-orientation: rotate-left;
}

@page right {
  size: landscape;
  page-orientation: rotate-right;
}
```

```
fieldset {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  gap: 1rem;
  width: fit-content;
}
p {
  max-width: 60ch;
}
@media print {
  fieldset {
    display: none;
  }
  section {
    font-family: "Roboto", sans-serif;
    font-size: 1.5rem;
  }
}
```

The second part of the CSS code declares a named page rule defined above for the selectors, such as `<section class="left">…</section>`.

```
@media print {
  .upright {
    page: upright;
  }
  .left {
    page: left;
  }
  .right {
    page: right;
  }
}
```

```
<fieldset id="printStyle">
  <legend>
    Click Print. The page will be laid out in the defined orientation.
  </legend>
  <button id="print">Print</button>
</fieldset>
<div id="print-area">
  <section class="upright">
    <h2>Section in Portrait/Upright</h2>
    <p>This section will be printed in portrait and upright with:</p>
    <pre>
.upright {
  size: portrait;
  page-orientation: upright;
}
    </pre>
    <p>Paragraph one.</p>
    <p>Paragraph two.</p>
  </section>
  <section class="left">
    <h2>Section in Landscape/Left</h2>
    <p>This section will be printed in landscape and rotated left with:</p>
    <pre>
.left {
  size: landscape;
  page-orientation: rotate-left;
}
    </pre>
    <table>
      <thead>
        <tr>
          <th></th>
          <th>100m</th>
          <th>1500m</th>
          <th>Hurdles</th>
          <th>Long Jump</th>
          <th>High Jump</th>
          <th>Javelin</th>
          <th>Discus</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th>Dave</th>
          <td>9.65</td>
          <td>3:49</td>
          <td>12.64</td>
          <td>8.54m</td>
          <td>1.95m</td>
          <td>56.65m</td>
          <td>47.63m</td>
        </tr>
        <tr>
          <th>Simon</th>
          <td>9.87</td>
          <td>3:52</td>
          <td>11.89</td>
          <td>8.16m</td>
          <td>1.96m</td>
          <td>59.03m</td>
          <td>45.72m</td>
        </tr>
        <tr>
          <th>Fred</th>
          <td>9.67</td>
          <td>3:52</td>
          <td>12.03</td>
          <td>8.04m</td>
          <td>2.01m</td>
          <td>62.58m</td>
          <td>46.83m</td>
        </tr>
        <tr>
          <th>Gary</th>
          <td>9.77</td>
          <td>3:56</td>
          <td>13.04</td>
          <td>7.96m</td>
          <td>2.02m</td>
          <td>63.87m</td>
          <td>47.73m</td>
        </tr>
        <tr>
          <th>Dick</th>
          <td>9.95</td>
          <td>3:44</td>
          <td>12.99</td>
          <td>5.66m</td>
```

```
          <td>1.97m</td>
          <td>56.43m</td>
          <td>43.87m</td>
        </tr>
        <tr>
          <th>Tom</th>
          <td>9.84</td>
          <td>3:48</td>
          <td>12.86</td>
          <td>6.87m</td>
          <td>1.95m</td>
          <td>67.03m</td>
          <td>42.73m</td>
        </tr>
        <tr>
          <th>Harry</th>
          <td>9.91</td>
          <td>3:53</td>
          <td>13.77</td>
          <td>7.34m</td>
          <td>1.94m</td>
          <td>62.84m</td>
          <td>46.72m</td>
        </tr>
      </tbody>
    </table>
  </section>
  <section class="right">
    <h2>Section in Landscape/Right</h2>
    <p>This section will be printed in landscape and rotated right with:</p>
    <pre>
.right {
  size: landscape;
  page-orientation: rotate-right;
}
    </pre>
    <table>
      <thead>
        <tr>
          <th></th>
          <th>100m</th>
          <th>1500m</th>
          <th>Hurdles</th>
          <th>Long Jump</th>
          <th>High Jump</th>
          <th>Javelin</th>
          <th>Discus</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th>Dave</th>
          <td>9.65</td>
          <td>3:49</td>
          <td>12.64</td>
          <td>8.54m</td>
          <td>1.95m</td>
          <td>56.65m</td>
          <td>47.63m</td>
        </tr>
        <tr>
          <th>Simon</th>
          <td>9.87</td>
          <td>3:52</td>
          <td>11.89</td>
          <td>8.16m</td>
          <td>1.96m</td>
          <td>59.03m</td>
          <td>45.72m</td>
        </tr>
        <tr>
          <th>Fred</th>
          <td>9.67</td>
          <td>3:52</td>
          <td>12.03</td>
          <td>8.04m</td>
          <td>2.01m</td>
          <td>62.58m</td>
          <td>46.83m</td>
        </tr>
        <tr>
          <th>Gary</th>
          <td>9.77</td>
          <td>3:56</td>
          <td>13.04</td>
          <td>7.96m</td>
          <td>2.02m</td>
          <td>63.87m</td>
          <td>47.73m</td>
        </tr>
        <tr>
          <th>Dick</th>
          <td>9.95</td>
          <td>3:44</td>
          <td>12.99</td>
          <td>5.66m</td>
          <td>1.97m</td>
          <td>56.43m</td>
          <td>43.87m</td>
        </tr>
        <tr>
          <th>Tom</th>
          <td>9.84</td>
```

```
          <td>3:48</td>
          <td>12.86</td>
          <td>6.87m</td>
          <td>1.95m</td>
          <td>67.03m</td>
          <td>42.73m</td>
        </tr>
        <tr>
          <th>Harry</th>
          <td>9.91</td>
          <td>3:53</td>
          <td>13.77</td>
          <td>7.34m</td>
          <td>1.94m</td>
          <td>62.84m</td>
          <td>46.72m</td>
        </tr>
      </tbody>
    </table>
  </section>
</div>
```

```
const printButton = document.querySelector("#print");
printButton.addEventListener("click", () => {
  window.print();
});
```

Click the print button to see the page orientation on print.

size CSS at-rule descriptor

The **`size`** CSS at-rule descriptor, used with the `@page` at-rule, defines the size and orientation of the box which is used to represent a page. Most of the time, this size corresponds to the target size of the printed page if applicable.

Size may either be defined with a "scalable" keyword (in this case the page will fill the available dimensions) or with absolute dimensions.

Syntax

```
/* Keyword values for scalable size */
size: auto;
size: portrait;
size: landscape;

/* <length> values */
/* 1 value: height = width */
size: 6in;

/* 2 values: width then height */
size: 4in 6in;

/* Keyword values for absolute size */
size: A4;
size: B5;
size: JIS-B4;
size: letter;

/* Mixing size and orientation */
size: A4 portrait;
```

Values

`auto`

The user agent decides the size of the page. In most cases, the dimensions and orientation of the target sheet are used.

`landscape`

The content of the page is displayed in landscape mode (i.e., the longest side of the box is horizontal).

`portrait`

The content of the page is displayed in portrait mode (i.e., the longest side of the box is vertical). This is the default orientation.

`<length>`

Any length value (see `<length>`). The first value corresponds to the width of the page box and the second one corresponds to its height. If only one value is provided, it is used for both width and height.

`<page-size>`

A keyword which may be any of the following values:

A5

This matches the standard, ISO dimensions: 148mm x 210mm.

A4

This matches the standard, ISO dimensions: 210mm x 297mm. (most frequently used dimensions for personal printing.)

A3

This matches the standard, ISO dimensions: 297mm x 420mm.

B5

This matches the standard, ISO dimensions: 176mm x 250mm.

B4

This matches the standard, ISO dimensions: 250mm x 353mm.

JIS-B5

This correspond to the JIS standard dimensions: 182mm x 257mm.

JIS-B4

This correspond to the JIS standard dimensions: 257mm x 364mm.

letter

This keyword is an equivalent to the dimensions of letter paper in North America i.e., 8.5in x 11in.

legal

This keyword is an equivalent to the dimensions of legal papers in North America i.e., 8.5in x 14in.

ledger

This keyword is an equivalent to the dimensions of ledger pages in North America i.e., 11in x 17in.

Formal definition

Related at-rule	`@page`
Initial value	`auto`
Computed value	as specified, but with relative lengths converted into absolute lengths

Formal syntax

```
size =
  <length [0,∞]>{1,2}                        |
  auto                                       |
  [ <page-size> || [ portrait | landscape ] ]

<page-size> =
  A5      |
  A4      |
  A3      |
  B5      |
  B4      |
  JIS-B5  |
  JIS-B4  |
  letter  |
  legal   |
  ledger
```

This syntax reflects the latest standard as per CSS Paged Media Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying size and orientation

```
@page {
  size: A4 landscape;
}
```

Specifying a custom size

```
@page {
  size: 4in 6in;
}
```

Nesting inside a @media rule

```
@media print {
  @page {
    size: 50mm 150mm;
  }
}
```

@position-try CSS at-rule

The **`@position-try`** CSS at-rule is used to define a custom position try fallback option, which can be used to define positioning and alignment for anchor-positioned elements. One or more sets of position try fallback options can be applied to the anchored element via the `position-try-fallbacks` property or `position-try` shorthand. When the positioned element is moved to a position where it starts to overflow its containing block or the viewport, the browser will select the first position try fallback option it finds that places the positioned element fully back on-screen.

Each position option is named with a `<dashed-ident>` and contains a descriptor list specifying declarations that define information such as inset position, margin, sizing, and self-alignment. The `<dashed-ident>` is used to reference the custom position option in the `position-try-fallbacks` property and `position-try` shorthand.

For detailed information on anchor features and position try fallback usage, see the CSS anchor positioning module and the Fallback options and conditional hiding for overflow guide.

Syntax

```
@position-try --try-option-name {
  descriptor-list
}
```

> **Note:** The `--try-option-name` is a `<dashed-ident>` specifying an identifying name for the custom position option, which can then be used to add that fallback option to the `position-try-fallbacks` list.

Descriptors

The descriptors specify property values that define the behavior of the custom position option, i.e., where it will result in the positioned element being placed.

`position-anchor`

Specifies a `position-anchor` property value that defines the anchor element that the positioned element is tethered to, by specifying a `<dashed-ident>` value equal to the anchor element's `anchor-name` property value.

`position-area`

Specifies a `position-area` property value that defines the position of the anchor-positioned element relative to the anchor.

Inset property descriptors

Specify `anchor()` function values that define the position of the anchor-positioned element's edges relative to the anchor element's edge. Inset property descriptors can be set that represent the following properties:

- `top`
- `left`
- `bottom`
- `right`.
- `inset-block-start`
- `inset-block-end`
- `inset-inline-start`
- `inset-inline-end`
- `inset-block`
- `inset-inline`
- `inset`

Margin property descriptors

Specify the margin set on the anchor-positioned element. Margin property descriptors can be set that represent the following properties:

- `margin-top`
- `margin-left`
- `margin-bottom`
- `margin-right`
- `margin-block-start`
- `margin-block-end`
- `margin-inline-start`
- `margin-inline-end`
- `margin`
- `margin-block`
- `margin-inline`

Sizing property descriptors

Specify `anchor-size()` function values that define the size of the anchor-positioned element relative to the anchor element size. Sizing property descriptors can be set that represent the following properties:

- `width`
- `height`
- `min-width`
- `min-height`
- `max-width`
- `max-height`
- `block-size`
- `inline-size`
- `min-block-size`
- `min-inline-size`
- `max-block-size`
- `max-inline-size`

Self-alignment property descriptors

Specify the `anchor-center` value to align the anchor-positioned element relative to the anchor element's center, in the block or inline direction. `align-self` and `justify-self` property descriptors can take the `anchor-center` value.

> **Note:** When a custom position option is applied to an element, the property values defined in the `@position-try` at-rule descriptor takes precedence over the values set on the element via standard CSS properties.

Formal syntax

```
@position-try =
  @position-try <dashed-ident> { <declaration-list> }
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Custom position option usage

In this example, we define an anchor element and an anchor-positioned element, then create four named custom position try fallback options. These options are applied to the positioned element to ensure its contents are always visible no matter where the anchor element is within the viewport.

HTML

We include two `<div>` elements that will become an anchor and an anchor-positioned element:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

CSS

We first style the `<body>` element to be very large, so that we can scroll the anchor and the positioned element around in the viewport, both horizontally and vertically:

```
body {
  width: 1500px;
  height: 500px;
}
```

```
.anchor {
  font-size: 1.8rem;
  color: white;
```

```
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

The anchor is given an `anchor-name` and has a `position` value of `absolute` set on it. We then position it somewhere near the center of the initial `<body>` rendering using `top` and `left` values:

```
.anchor {
  anchor-name: --my-anchor;
  position: absolute;
  top: 100px;
  left: 350px;
}
```

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

Next, we use the `@position-try` at-rule to define four custom position options, with descriptive `<dashed-ident>` names to identify them and describe their purpose. Each one places the positioned element in a specific position around the anchor element and gives it an appropriate `10px` margin between the positioned element and its anchor. The positioning is handled in a variety of ways, to demonstrate the different techniques available:

- The first and last position options use a `position-area`.
- The second position option uses `top` with an `anchor()` value and `justify-self: anchor-center` to center the positioned element on the anchor in the inline direction.
- The third position option uses `left` with an `anchor()` value, wrapped inside a `calc()` function that adds `10px` of spacing (rather than creating the spacing with `margin` like the other options do). It then uses `align-self: anchor-center` to center the positioned element on the anchor in the block direction.

Finally, the left and right position options are given a narrower `width`

```
@position-try --custom-left {
  position-area: left;
  width: 100px;
  margin-right: 10px;
}

@position-try --custom-bottom {
  top: anchor(bottom);
  justify-self: anchor-center;
  margin-top: 10px;
  position-area: none;
}

@position-try --custom-right {
  left: calc(anchor(right) + 10px);
  align-self: anchor-center;
  width: 100px;
  position-area: none;
}

@position-try --custom-bottom-right {
  position-area: bottom right;
  margin: 10px 0 0 10px;
}
```

The infobox is given fixed positioning, a `position-anchor` property that references the anchor's `anchor-name` to associate the two together, and it is tethered to the anchor's top edge using a `position-area`. We also give it a fixed `width` and some bottom `margin`. The custom position options are then referenced in the `position-try-fallbacks` property to prevent the positioned element from overflowing, or being scrolled out of view, when the anchor gets near the edge of the viewport.

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top;
  width: 200px;
  margin-bottom: 10px;
  position-try-fallbacks:
    --custom-left, --custom-bottom, --custom-right, --custom-bottom-right;
}
```

Result

Scroll the page and notice the change in the positioned element's placement as the anchor nears the different edges of the viewport. This is due to different fallback position options being applied.

Let's talk through how these position options work:

- First of all, note that our default position is defined by `position-area: top`. When the infobox isn't overflowing the page in any direction, the infobox sits above the anchor, and the position try fallback options set in the `position-try-fallbacks` property are ignored. Also note that the infobox has a fixed width and bottom margin set. These values will change as different position try fallback options are applied.
- If the infobox starts to overflow, the browser first tries the `--custom-left` position. This moves the infobox to the left of the anchor using `position-area: left`, adjusts the margin to suit, and also gives the infobox a different width.
- Next, the browser tries the `--custom-bottom` position. This moves the infobox to the bottom of the anchor using `top` and `justify-self` instead of a `position-area`, and sets an appropriate margin. It doesn't include a `width` descriptor, so the infobox returns to its default width of `200px` set by the `width` property.
- The browser next tries the `--custom-right` position. This works much the same as the `--custom-left` position, with the same `width` descriptor value applied. However, we are using `left` and `align-self` to place the positioned element instead of a `position-area`. And we are wrapping the `left` value in a `calc()` function inside which we are adding `10px` to create spacing, instead of using `margin`.
- If none of the other try fallback options succeed in stopping the positioned element from overflowing, the browser tries the `--custom-bottom-right` position as a last resort. This places the positioned element to the bottom-right of the anchor using `position-area: bottom right`.

When a position option is applied, its values override the initial values set on the positioned element. For example, the `width` initially set on the positioned element is `200px`, but when the `--custom-right` position option is applied, its width is set to `100px`.

In some cases, we need to set values inside the custom position options to turn off the initial values. The `--custom-bottom` and `--custom-right` options use inset property and `*-self: anchor-center` values to place the positioned element, therefore we remove the previously-set `position-area` value in each case by setting `position-area: none`. If we didn't do this, the initially set `position-area: top` value would still take effect and interfere with the other positioning information.

See also

- `position-area`
- `position-anchor`
- `position-try-fallbacks`
- `position-try`
- The `anchor()` function
- The `anchor-size()` function
- CSS anchor positioning module
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide
- `CSSPositionTryRule`

@property CSS at-rule

The **`@property`** CSS at-rule is used to explicitly define CSS custom properties, allowing for property type checking and constraining, setting default values, and defining whether a custom property can inherit values or not.

> **Note:** The JavaScript `registerProperty()` method is equivalent to the `@property` at-rule.

Syntax

```
@property --canBeAnything {
  syntax: "*";
  inherits: true;
}

@property --rotation {
  syntax: "<angle>";
  inherits: false;
  initial-value: 45deg;
}

@property --defaultSize {
  syntax: "<length> | <percentage>";
  inherits: true;
  initial-value: 200px;
}
```

The custom property name is a `<dashed-ident>` that starts with `--` and is followed by a valid, user-defined identifier. It is case-sensitive.

Descriptors

`syntax`

A string that describes the allowed value types for the registered custom property.

`inherits`

A boolean value that controls whether the custom property registration specified by `@property` inherits by default.

`initial-value`

A value that sets the starting value for the property.

Description

The `@property` at-rule is part of the CSS Houdini set of APIs. It allows developers to explicitly define CSS custom properties, allowing for property type checking and constraining, setting default values, and defining whether a custom property can inherit values or not.

The `@property` rule enables custom property registration directly inside stylesheets without requiring any JavaScript. Valid `@property` rules produce registered custom properties, producing the same effect as calling `registerProperty()` with equivalent parameters.

The following conditions must be met for the `@property` rule to be valid:

- The `@property` rule must include both the `syntax` and `inherits` descriptors. If either is missing, the entire `@property` rule is invalid and ignored.
- The `syntax` may be a data type name (such as `<color>`, `<length>`, or `<number>`, etc.), with multipliers (`+` to accept a space-separated list, or `#` to accept a comma-separated list) and combinators (`|` to accept one data type or another), a custom ident, or the universal syntax definition (`*`), meaning the syntax can be any valid token stream. The value is a `<string>`. As such, it must be in quotes.
- The `initial-value` descriptor is optional if the value of the `syntax` descriptor is the universal syntax definition (`syntax: "*"`). If the `initial-value` descriptor is required but omitted, the entire `@property` rule is invalid and ignored.
- If the value of the `syntax` descriptor is not the universal syntax definition, the `initial-value` descriptor has to be a computationally independent value. This means the value can be converted into a computed value without depending on other values, except for "global" definitions independent of CSS. For example, `10px` is computationally independent—it doesn't change when converted to a computed value. `2in` is also valid, because `1in` is always equivalent to `96px`. However, `3em` is not valid, because the value of an `em` is dependent on the parent's `font-size`.
- Unknown descriptors are invalid and ignored, but do not invalidate the `@property` rule.

If multiple valid `@property` rules are defined using the same name, the last one in stylesheet order "wins". If custom properties are registered with the same name using `@property` in CSS and `CSS.registerProperty()` in JavaScript, the JavaScript registration wins.

Formal syntax

```
@property =
  @property <custom-property-name># { <declaration-list> }
```

This syntax reflects the latest standard as per CSS Properties and Values API Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we use the `@property` at-rule to declare two custom properties, and then use those properties in our style declarations.

HTML

```
<p>Hello world!</p>
```

CSS

```
@property --myColor {
  syntax: "<color>";
  inherits: true;
  initial-value: rebeccapurple;
}

@property --myWidth {
  syntax: "<length> | <percentage>";
  inherits: true;
  initial-value: 200px;
}

p {
  background-color: var(--myColor);
  width: var(--myWidth);
  color: white;
}
```

Results

The paragraph should be `200px` wide, with a purple background and white text.

Animating a custom property value

In this example, we define a custom property called `--progress` using `@property`: this accepts `<percentage>` values and has an initial value of `25%`. We use `--progress` to define the position value of the color stops in a `linear-gradient()`, specifying where a green color stops, and black starts. We then animate the value of `--progress` to `100%` over 2.5 seconds, giving the effect of animating a progress bar.

HTML

```
<div class="bar"></div>
```

CSS

```
@property --progress {
  syntax: "<percentage>";
  inherits: false;
  initial-value: 25%;
}

.bar {
  display: inline-block;
  --progress: 25%;
  width: 100%;
  height: 5px;
  background: linear-gradient(
    to right,
    #00d230 var(--progress),
    black var(--progress)
  );
  animation: progressAnimation 2.5s ease infinite;
}

@keyframes progressAnimation {
  to {
    --progress: 100%;
  }
}
```

Results

See also

- `var()`
- Custom properties (`--*`)
- Registering CSS custom properties
- CSS Properties and Values API module
- CSS Properties and Values API documentation
- Using CSS custom properties (variables) guide
- CSS custom properties for cascading variables module

inherits CSS at-rule descriptor

The **`inherits`** CSS descriptor of the `@property` at-rule controls whether or not the registered CSS custom property inherits by default. It is a required descriptor; if missing or invalid, the entire `@property` rule is invalid and ignored.

Syntax

```
/* Custom property does not inherit values */
inherits: false;

/* Custom property inherits values */
inherits: true;
```

Values

`true`

The property inherits by default.

`false`

The property does not inherit by default.

Formal definition

Related at-rule	`@property`
Initial value	`auto`
Computed value	as specified

Formal syntax

```
inherits =
  true  |
  false
```

This syntax reflects the latest standard as per CSS Properties and Values API Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inheritance behavior of a custom property

This example shows how to define a custom property `--my-color` that does not inherit its value from its parent elements:

```
@property --my-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #c0ffee;
}
```

Using JavaScript `CSS.registerProperty()`:

```
window.CSS.registerProperty({
  name: "--my-color",
  syntax: "<color>",
  inherits: false,
  initialValue: "#c0ffee",
});
```

See also

- Other `@property` descriptors: `initial-value` and `syntax`
- Registering CSS custom properties
- CSS Properties and Values API
- CSS Painting API
- CSS Typed Object Model
- Houdini APIs

initial-value CSS at-rule descriptor

The **`initial-value`** descriptor of the `@property` at-rule specifies the initial value for the registered CSS custom property. It is a required descriptor unless the `syntax` descriptor value is the universal syntax (`*`). If required but missing or invalid, the entire `@property` rule is invalid and ignored.

Syntax

```
/* Set initial color value */
initial-value: rebeccapurple;

/* Set initial length value */
initial-value: 16px;
```

Values

A value that matches the type specified in the `syntax` descriptor. For example, if `syntax` is `<color>`, then the `initial-value` must be a valid `color` value.

If the value of the `syntax` descriptor is not the universal syntax definition, the `initial-value` descriptor has to be a computationally independent value. This means the value can be converted into a computed value without depending on other values, except for "global" definitions independent of CSS. For example, `10px` is computationally independent—it doesn't change when converted to a computed value. `2in` is also valid, because `1in` is always equivalent to `96px`. However, `3em` is not valid, because the value of an `em` is dependent on the parent's `font-size`.

Formal definition

Related at-rule	`@property`
Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
initial-value =
  <declaration-value>?
```

This syntax reflects the latest standard as per CSS Properties and Values API Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting an initial value for a custom property

This example shows how to define a custom property `--my-color` with an initial color value of `#c0ffee`. This initial value will be used when the property is not inherited (`inherits: false`) and no other value is set on the element.

```
@property --my-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #c0ffee;
}
```

Using JavaScript `CSS.registerProperty()`:

```
window.CSS.registerProperty({
  name: "--my-color",
  syntax: "<color>",
  inherits: false,
  initialValue: "#c0ffee",
});
```

See also

- Other `@property` descriptors: `inherits` and `syntax`
- Registering CSS custom properties
- CSS Properties and Values API
- CSS Painting API
- CSS Typed Object Model
- Houdini APIs

syntax CSS at-rule descriptor

The **`syntax`** descriptor of the `@property` at-rule defines the allowed value types for the registered CSS custom property. It controls how the property's specified value is processed to derive the computed value. It is a required descriptor; if missing or invalid, the entire `@property` rule is invalid and ignored.

Syntax

```
/* A data type name */
syntax: "<color>";

/* A '|' combinator for multiple data types */
syntax: "<length> | <percentage>";

/* Space-separated list of values */
syntax: "<color>+";

/* Comma-separated list of values */
syntax: "<length>#";

/* Keywords */
syntax: "small | medium | large";

/* Combination of data type and keyword */
syntax: "<length> | auto";

/* Universal syntax value */
syntax: "*";
```

Values

A string (known as the syntax string) that defines the allowed values. It can be one of the following:

- One or more syntax component names, which can be:
  - Data type names (written with angle brackets, such as `<color>` or `<length>`)
  - Keywords (written without angle brackets, such as `auto` or `none`)
- The universal syntax `*`, which accepts any valid CSS value. It cannot be multiplied or combined with other syntax components.

The syntax component names can be used alone or multiplied and combined in different ways:

- The `+` (space-separated) and `#` (comma-separated) multipliers indicate that a list of values is expected. For example, `<color>#` means a comma-separated list of `<color>` values is the expected syntax.
- The vertical line (`|`) combinator can create "or" conditions for the expected syntax. For example, `<length> | auto` accepts `<length>` or `auto`, and `<color># | <integer>#` expects a comma-separated list of `<color>` values or a comma-separated list of `<integer>` values.

The following syntax component names are supported:

`"<angle>"`

Accepts any valid `<angle>` value.

`"<color>"`

Accepts any valid `<color>` value.

`"<custom-ident>"`

Accepts any valid `<custom-ident>` value.

`"<image>"`

Accepts any valid `<image>` value.

`"<integer>"`

Accepts any valid `<integer>` value.

`"<length>"`

Accepts any valid `<length>` value.

`"<length-percentage>"`

Accepts any valid `<length>` or `<percentage>` value and any valid `calc()` expression combining `<length>` and `<percentage>` values.

`"<number>"`

Accepts any valid `<number>` value.

`"<percentage>"`

Accepts any valid `<percentage>` value.

`"<resolution>"`

Accepts any valid `<resolution>` value.

`"<string>"`

Accepts any valid `<string>` value.

`"<time>"`

Accepts any valid `<time>` value.

`"<transform-function>"`

Accepts any valid `<transform-function>` value.

`"<transform-list>"`

Accepts a list of valid `<transform-function>` values. It is equivalent to `"<transform-function>+"`.

`"<url>"`

Accepts any valid `<url>` value.

Formal definition

Related at-rule	`@property`
Initial value	`n/a (required)`
Computed value	as specified

Formal syntax

```
syntax =
  <string>
```

This syntax reflects the latest standard as per CSS Properties and Values API Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Registering a custom property with type checking

This example shows how to define a custom property `--my-color` that allows only `<color>` values:

```
@property --my-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #c0ffee;
}
```

Using JavaScript `CSS.registerProperty()`:

```
window.CSS.registerProperty({
  name: "--my-color",
  syntax: "<color>",
  inherits: false,
  initialValue: "#c0ffee",
});
```

See also

- Other `@property` descriptors: `inherits` and `initial-value`
- Registering CSS custom properties
- CSS Properties and Values API
- CSS Painting API
- CSS Typed Object Model
- Houdini APIs

@scope CSS at-rule

The **`@scope`** CSS at-rule enables you to select elements in specific DOM subtrees, targeting elements precisely without writing overly-specific selectors that are hard to override, and without coupling your selectors too tightly to the DOM structure.

In JavaScript, `@scope` can be accessed via the CSS object model interface `CSSScopeRule`.

Syntax

The `@scope` at-rule contains one or more rulesets (termed **scoped style rules**) and defines a scope in which to apply them to selected elements. `@scope` can be used in two ways:

1. As a standalone block inside your CSS, in which case it includes a prelude section that includes **scope root** and optional **scope limit** selectors — these define the upper and lower bounds of the scope.

   ```
   @scope (scope root) to (scope limit) {
     /* … */
   }
   ```

2. As inline styles included inside a `<style>` element in your HTML, in which case the prelude is omitted, and the enclosed ruleset is automatically scoped to the `<style>` element's enclosing parent element.

   ```
   <parent-element>
     <style>
       @scope {
         /* rulesets */
       }
     </style>
   </parent-element>
   ```

   It is also possible to combine an inline `@scope` with a scope limit selector, as in `@scope to (scope limit) { ... }`.

Description

A complex web document might include components such as headers, footers, news articles, maps, media players, ads, and others. As complexity increases, effectively managing the styling for these components becomes more of a concern, and effective scoping of styles helps us manage this complexity. Let's consider the following DOM tree:

```
body
└ article.feature
   ├ section.article-hero
   │  ├ h2
   │  └ img
   │
   ├ section.article-body
   │  ├ h3
   │  ├ p
   │  ├ img
   │  ├ p
   │  └ figure
   │     ├ img
   │     └ figcaption
   │
   └ footer
      ├ p
      └ img
```

If you wanted to select the `<img>` element inside the `<section>` with a class of `article-body`, you could do the following:

- Write a selector like `.feature > .article-body > img`. However, that has high specificity so is hard to override, and is also tightly coupled to the DOM structure. If your markup structure changes in the future, you might need to rewrite your CSS.
- Write something less specific like `.article-body img`. However, that will select all images inside the `section`.

This is where `@scope` is useful. It allows you to define a precise scope inside which your selectors are allowed to target elements. For example, you could solve the above problem using a standalone `@scope` block like the following:

```
@scope (.article-body) to (figure) {
  img {
    border: 5px solid black;
    background-color: goldenrod;
  }
}
```

The `.article-body` scope root selector defines the upper bound of the DOM tree scope in which the ruleset will be applied, and the `figure` scope limit selector defines the lower bound. As a result, only `<img>` elements inside a `<section>` with a class of `article-body`, but not inside `<figure>` elements, will be selected.

> **Note:** This kind of scoping — with an upper and lower bound — is commonly referred to as a **donut scope**.

The scope's upper bound is inclusive and its lower bound is exclusive. To change this behavior, you can combine either selector with a universal child selector. For example, `@scope (scope root) to (scope limit > *)` would make both bounds inclusive, `@scope (scope root > *) to (scope limit)` would make both bounds exclusive, while `@scope (scope root > *) to (scope limit > *)` would give an exclusive upper bound and an inclusive lower bound.

If you want to select all images inside a `<section>` with a class of `article-body`, you can omit the scope limit:

```
@scope (.article-body) {
  img {
    border: 5px solid black;
    background-color: goldenrod;
  }
}
```

Or you could include your `@scope` block inline inside a `<style>` element, which in turn is inside the `<section>` with a class of `article-body`:

```
<section class="article-body">
  <style>
    @scope {
      img {
        border: 5px solid black;
        background-color: goldenrod;
      }
    }
  </style>

  <!-- ... -->
</section>
```

> **Note:** It is important to understand that, while `@scope` allows you to isolate the application of selectors to specific DOM subtrees, it does not completely isolate the applied styles to within those subtrees. This is most noticeable with inheritance — properties that are inherited by children (for example `color` or `font-family`) will still be inherited, beyond any set scope limit.

<code>:scope</code> pseudo-class within <code>@scope</code> blocks

In the context of an `@scope` block, the `:scope` pseudo-class provides a convenient way to directly apply styles to the scope root, like so:

```
@scope (.feature) {
  :scope {
    background: rebeccapurple;
    color: antiquewhite;
    font-family: sans-serif;
  }
}
```

Here's some considerations for `:scope` within `@scope` blocks:

- `:scope` adds class-level specificity (see Specificity in @scope for details).
- A scope limit can use `:scope` to specify a specific relationship requirement between the scope limit and root. For example:

```
/* figure is only a limit when it is a direct child of the :scope */
@scope (.article-body) to (:scope > figure) {
  /* … */
}
```

- A scope limit can reference elements outside the scope root using `:scope`. For example:

```
/* figure is only a limit when the :scope is inside .feature */
@scope (.article-body) to (.feature :scope figure) {
  /* … */
}
```

- Scoped style rules can't escape the subtree. Selections like `:scope + p` are invalid because that selection would be outside the subtree.
- It is perfectly valid to define the scope root and limit as a selector list, in which case multiple scopes will be defined. In the following example the styles are applied to any `<img>` inside a `<section>` with a class of `article-hero` or `article-body`, but not if it is nested inside a `<figure>`:

```
@scope (.article-hero, .article-body) to (figure) {
  img {
    border: 5px solid black;
    background-color: goldenrod;
  }
}
```

Specificity in <code>@scope</code>

Inside an `@scope` rule, both bare selectors and the `&` nesting selector behave as if `:where(:scope)` were prepended to the selector. Because `:where()` has zero specificity, bare selectors and `&` add zero weight. The specificity weight is determined by the rest of the selector. For example, the specificity of the `& img` selector is equivalent to the specificity of `:where(:scope) img` (0-0-1).

> **Warning:** The specificity of `&` inside `@scope` blocks is handled differently according to the browser engine and release version. Check Browser compatibility for details.

In both cases in the following code block, the only specificity comes from `img`:

```
@scope (.article-body) {
  /* img has a specificity of 0-0-1, as expected */
  img {
    /* … */
  }

  /* & img also has a specificity of 0-0-1 */
  & img {
    /* … */
  }
}
```

By contrast, using `:scope` explicitly selects the scope root and adds class-level specificity (0-1-0), since `:scope` is a pseudo-class. In the following code block, `:scope img` has a specificity of 0-1-1:

```
@scope (.article-body) {
  /* :scope img has a specificity of 0-1-0 + 0-0-1 = 0-1-1 */
  :scope img {
    /* … */
  }
}
```

How <code>@scope</code> conflicts are resolved

`@scope` adds a new criterion to the CSS cascade: **scoping proximity**. This states that when two scopes have conflicting styles, the style that has the smallest number of hops up the DOM tree hierarchy to the scope root is applied. Let's look at an example to see what this means.

Take the following HTML snippet, where different-themed cards are nested inside one another:

```
<div class="light-theme">
  <p>Light theme text</p>
  <div class="dark-theme">
    <p>Dark theme text</p>
    <div class="light-theme">
      <p>Light theme text</p>
    </div>
  </div>
</div>
```

If you wrote the theme CSS like so, you'd run into trouble:

```
.light-theme {
  background: #cccccc;
}

.dark-theme {
  background: #333333;
}

.light-theme p {
  color: black;
}

.dark-theme p {
  color: white;
}
```

The innermost paragraph is supposed to be colored black because it is inside a light theme card. However, it's targeted by both `.light-theme p` and `.dark-theme p`. Because the `.dark-theme p` rule appears later in the source order, it is applied, and the paragraph ends up being wrongly colored white.

To fix this, you can use `@scope` as follows:

```
@scope (.light-theme) {
  :scope {
    background: #cccccc;
  }
  p {
    color: black;
  }
}

@scope (.dark-theme) {
  :scope {
    background: #333333;
  }
  p {
    color: white;
  }
}
```

Now the innermost paragraph is correctly colored black. This is because it is only one DOM tree hierarchy level away from the `.light-theme` scope root, but two levels away from the `.dark-theme` scope root. Therefore, the light style wins.

> **Note:** Scoping proximity overrules source order but is itself overridden by other, higher-priority criteria such as importance, layers, and specificity.

Formal syntax

```
@scope =
  @scope <scope-boundaries>? { <block-contents> }

<scope-boundaries> =
  [ [ ( <scope-start> ) ]? [ to ( <scope-end> ) ]? ]!

<scope-start> =
  <selector-list>

<scope-end> =
  <selector-list>

<selector-list> =
  <complex-selector-list>

<complex-selector-list> =
  <complex-selector>#

<complex-selector> =
  <complex-selector-unit> [ <combinator>? <complex-selector-unit> ]*

<complex-selector-unit> =
  [ <compound-selector>? <pseudo-compound-selector>* ]!

<combinator> =
  '>'               |
  '+'               |
  '~'               |
  [ '|' '|' ]       |
  [ / <wq-name> / ]

<compound-selector> =
  [ <type-selector>? <subclass-selector>* ]!

<pseudo-compound-selector> =
  <pseudo-element-selector> <pseudo-class-selector>*

<wq-name> =
  <ns-prefix>? <ident-token>

<type-selector> =
  <wq-name>         |
  <ns-prefix>? '*'

<subclass-selector> =
  <id-selector>           |
  <class-selector>        |
  <attribute-selector>    |
  <pseudo-class-selector>

<pseudo-element-selector> =
  : <pseudo-class-selector>        |
  <legacy-pseudo-element-selector>

<pseudo-class-selector> =
  : <ident-token>                  |
  : <function-token> <any-value> )

<ns-prefix> =
  [ <ident-token> | '*' ]? '|'

<id-selector> =
  <hash-token>

<class-selector> =
  '.' <ident-token>

<attribute-selector> =
  '[' <wq-name> ']'                                |
  '[' <wq-name> <attr-matcher> [ <string-token> | <ident-token> ] <attr-modifier>? ']'

<legacy-pseudo-element-selector> =
  : [ before | after | first-line | first-letter ]

<attr-matcher> =
  [ '~' | '|' | '^' | '$' | '*' ]? '='

<attr-modifier> =
```

```
  i  |
  s
```

This syntax reflects the latest standard as per CSS Cascading and Inheritance Level 6, Selectors Level 4, Selectors Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic style inside scope roots

In this example, we use two separate `@scope` blocks to match links inside elements with a `.light-scheme` and `.dark-scheme` class respectively. Note how `:scope` is used to select and provide styling to the scope roots themselves. In this example, the scope roots are the `<div>` elements that have the classes applied to them.

HTML

```
<div class="light-scheme">
  <p>
    MDN contains lots of information about
    <a href="/en-US/docs/Web/HTML">HTML</a>,
    <a href="/en-US/docs/Web/CSS">CSS</a>, and
    <a href="/en-US/docs/Web/JavaScript">JavaScript</a>.
  </p>
</div>

<div class="dark-scheme">
  <p>
    MDN contains lots of information about
    <a href="/en-US/docs/Web/HTML">HTML</a>,
    <a href="/en-US/docs/Web/CSS">CSS</a>, and
    <a href="/en-US/docs/Web/JavaScript">JavaScript</a>.
  </p>
</div>
```

CSS

```
div {
  padding: 10px;
}
```

```
@scope (.light-scheme) {
  :scope {
    background-color: plum;
  }

  a {
    color: darkmagenta;
  }
}

@scope (.dark-scheme) {
  :scope {
    background-color: darkmagenta;
    color: antiquewhite;
  }

  a {
    color: plum;
  }
}
```

Result

The above code renders like this:

Scope roots and scope limits

In this example, we have an HTML snippet that matches the DOM structure we talked about earlier in the Description section. This structure represents a typical article summary. The key features to note are the `<img>` elements, which are nested at various levels in the structure.

The aim of this example is to show how to use a scope root and limit to style `<img>` elements starting at the top of the hierarchy, but only down as far as (and not including) the `<img>` inside the `<figure>` element — in effect creating a donut scope.

HTML

```
<article class="feature">
  <section class="article-hero">
    <h2>Article heading</h2>
    <img alt="image" src="" />
  </section>

  <section class="article-body">
    <h3>Article subheading</h3>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam euismod
      consectetur leo, nec eleifend quam volutpat vitae. Duis quis felis at
      augue imperdiet aliquam. Morbi at felis et massa mattis lacinia. Cras
      pharetra velit nisi, ac efficitur magna luctus nec.
    </p>

    <img alt="image" src="" />

    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>

    <figure>
      <img alt="image" src="" />
      <figcaption>My infographic</figcaption>
    </figure>
  </section>

  <footer>
    <p>Written by Chris Mills.</p>
    <img alt="image" src="" />
```

```
  </footer>
</article>
```

CSS

```
* {
  box-sizing: border-box;
}

article {
  margin-bottom: 20px;
  padding: 10px;
  border: 2px solid gray;
}

.article-hero,
.article-body,
article footer {
  padding: 20px;
  margin: 10px;
  border: 2px solid lightgray;
}

img {
  height: 100px;
  width: 100%;
  display: block;
  background-color: lightgray;
  color: black;
  padding: 10px;
}
```

In our CSS, we have two `@scope` blocks:

- The first `@scope` block defines its scope root as elements with a class of `.feature` (in this case, the outer `<article>` only), demonstrating how `@scope` can be used to theme a specific HTML subset.
- The second `@scope` block also defines its scope root as elements with a class of `.feature`, but also defines a scope limit of `figure`. This ensures that contained rulesets will only be applied to matching elements within the scope root (`<article class="feature"> ... </article>` in this case) that **are not** nested inside descendant `<figure>` elements. This `@scope` block contains a single ruleset that styles `<img>` elements with a thick black border and a golden background color.

```
/* Scoped CSS */

@scope (.feature) {
  :scope {
    background: rebeccapurple;
    color: antiquewhite;
    font-family: sans-serif;
  }

  figure {
    background-color: white;
    border: 2px solid black;
    color: black;
    padding: 10px;
  }
}

/* Donut scope */

@scope (.feature) to (figure) {
  img {
    border: 5px solid black;
    background-color: goldenrod;
  }
}
```

Result

In the rendered code, note how all of the `<img>` elements are styled with the thick border and golden background, except for the one inside the `<figure>` element (labeled "My infographic").

Browser compatibility

See also

- `:scope`
- `CSSScopeRule`
- Specificity
- Defining the `&` selector in a `@scope` rule on css.oddbird.net (2025)
- Limit the reach of your selectors with the CSS `@scope` at-rule on developer.chrome.com (2023)

@starting-style CSS at-rule

The **`@starting-style`** CSS at-rule is used to define starting values for properties set on an element that you want to transition from when the element receives its first style update, i.e., when an element is first displayed on a previously loaded page.

Syntax

The `@starting-style` at rule can be used in two ways:

1. As a standalone block, in which case it contains one or more rulesets defining starting style declarations and selecting the elements they apply to:

   ```
   @starting-style {
     /* rulesets */
   }
   ```

2. Nested within an existing ruleset, in which case it contains one or more declarations defining starting property values for the elements already selected by that ruleset:

   ```
   selector {
     /* existing ruleset */
     /* ... */
   ```

```
  @starting-style {
    /* declarations */
  }
}
```

Description

To avoid unexpected behavior, CSS transitions are by default not triggered on an element's initial style update, or when its `display` type changes from `none` to another value. To enable first-style transitions, `@starting-style` rules are needed. They provide starting styles for elements that do not have a previous state, defining the property values to transition from.

`@starting-style` is especially useful when creating entry and exit transitions for elements displayed in the top layer (such as popovers and modal `<dialog>`s), elements that are changing to and from `display: none`, and elements when first added to or removed from the DOM.

> **Note:** `@starting-style` is only relevant to CSS transitions. When using CSS animations to implement such effects, `@starting-style` is not needed. See Using CSS animations for an example.

There are two ways to use `@starting-style`: as a standalone rule or nested within a ruleset.

Let's consider a scenario where we want to animate a popover when shown (that is, when added to the top layer). The "original rule" specifying the styles for the open popover could look something like this (see the popover example below):

```
[popover]:popover-open {
  opacity: 1;
  transform: scaleX(1);
}
```

To specify the starting values of the popover's properties that will be animated using the first method, you include a standalone `@starting-style` block in your CSS:

```
@starting-style {
  [popover]:popover-open {
    opacity: 0;
    transform: scaleX(0);
  }
}
```

> **Note:** The `@starting-style` at-rule and the "original rule" have the same specificity. To ensure that starting styles get applied, include the `@starting-style` at-rule *after* the "original rule". If you specify the `@starting-style` at-rule before the "original rule", the original styles will override the starting styles.

To specify the starting style for the popover using the nested method, you can nest the `@starting-style` block inside the "original rule":

```
[popover]:popover-open {
  opacity: 1;
  transform: scaleX(1);

  @starting-style {
    opacity: 0;
    transform: scaleX(0);
  }
}
```

When exactly are starting styles used?

It is important to understand that an element will transition from its `@starting-style` styles when it is first rendered in the DOM, or when it transitions from `display: none` to a visible value. When it transitions back from its initial visible state, it will no longer use the `@starting-style` styles as it is now visible in the DOM. Instead, it will transition back to whatever styles exist for that element's default state.

In effect, there are three style states to manage in these situations — starting-style state, transitioned state, and default state. It is possible for the "to" and "from" transitions to be different in such cases. You can see a proof of this in our Demonstration of when starting styles are used example, below.

Formal syntax

```
@starting-style =
  @starting-style { <rule-list> }
```

Examples

Basic @starting-style usage

Transition an element's `background-color` from transparent to green when it is initially rendered:

```
#target {
  transition: background-color 1.5s;
  background-color: green;
}

@starting-style {
  #target {
    background-color: transparent;
  }
}
```

Transition the `opacity` of an element when it changes its `display` value to or from `none`:

```
#target {
  transition-property: opacity, display;
  transition-duration: 0.5s;
  display: block;
  opacity: 1;
  @starting-style {
    opacity: 0;
  }
}

#target.hidden {
  display: none;
```

```
  opacity: 0;
}
```

Demonstration of when starting styles are used

In this example, a button is pressed to create a `<div>` element, give it a `class` of `showing`, and add it to the DOM.

`showing` is given a `@starting-style` of `background-color: red` and a style of `background-color: blue` to transition to. The default `div` ruleset contains `background-color: yellow`, and is also where the `transition` is set.

When the `<div>` is first added to the DOM, you'll see the background transition from red to blue. After a timeout, we remove the `showing` class from the `<div>` via JavaScript. At that point it transitions from blue back to yellow, not red. This proves that the starting styles are only used when the element is first rendered in the DOM. Once it has appeared, the element transitions back to the default style set on it.

After another timeout, we then remove the `<div>` from the DOM altogether, resetting the initial state of the example so it can be run again.

HTML

```
<button>Display <code>&lt;div&gt;</code></button>
```

CSS

```
div {
  width: 200px;
  height: 100px;
  border: 1px solid black;
  margin-top: 10px;
}

div::after {
  content: "class: " attr(class);
  position: relative;
  top: 3px;
  left: 3px;
}
```

```
div {
  background-color: yellow;
  transition: background-color 3s;
}

div.showing {
  background-color: skyblue;
}

@starting-style {
  div.showing {
    background-color: red;
  }
}
```

JavaScript

```
const btn = document.querySelector("button");

btn.addEventListener("click", () => {
  btn.disabled = true;
  const divElem = document.createElement("div");
  divElem.classList.add("showing");
  document.body.append(divElem);

  setTimeout(() => {
    divElem.classList.remove("showing");

    setTimeout(() => {
      divElem.remove();
      btn.disabled = false;
    }, 3000);
  }, 3000);
});
```

Result

The code renders as follows:

Animating a popover

In this example, a popover is animated using CSS transitions. Basic entry and exit animations are provided using the `transition` property.

HTML

The HTML contains a `<div>` element declared as a popover using the popover attribute and a `<button>` element designated as the popover's display control using its popovertarget attribute.

```
<button popovertarget="mypopover">Show the popover</button>
<div popover="auto" id="mypopover">I'm a Popover! I should animate.</div>
```

CSS

In this example, we want to animate two properties, `opacity` and `transform` (specifically, a horizontally scaling transform), to make the popover fade in and out as well as grow and shrink horizontally.

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

[popover]:popover-open {
  opacity: 1;
  transform: scaleX(1);
}

[popover] {
  font-size: 1.2rem;
```

```
  padding: 10px;

  /* Final state of the exit animation */
  opacity: 0;
  transform: scaleX(0);

  transition:
    opacity 0.7s,
    transform 0.7s,
    overlay 0.7s allow-discrete,
    display 0.7s allow-discrete;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

/* Include after the [popover]:popover-open rule */
@starting-style {
  [popover]:popover-open {
    opacity: 0;
    transform: scaleX(0);
  }
}

/* Transition for the popover's backdrop */
[popover]::backdrop {
  background-color: transparent;
  transition:
    display 0.7s allow-discrete,
    overlay 0.7s allow-discrete,
    background-color 0.7s;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

[popover]:popover-open::backdrop {
  background-color: rgb(0 0 0 / 25%);
}

/* Nesting (&) is not supported for pseudo-elements
so specify a standalone starting-style block. */
@starting-style {
  [popover]:popover-open::backdrop {
    background-color: transparent;
  }
}
```

To achieve this, we have set a starting state for these properties on the default hidden state of the popover element (selected via `[popover]`), and an ending state on the open state of the popover (selected via the `:popover-open` pseudo-class).

We then set a `transition` property to animate between the two states. A starting state for the animation is included inside a `@starting-style` at-rule to enable the entry animation.

Because the animated element is being promoted to the top layer when shown and removed from the top layer when hidden (with `display: none`), some extra steps are required to ensure the animation works in both directions:

- `display` is added to the list of transitioned elements to ensure the animated element is visible (set to `display: block` or another visible `display` value) throughout both the entry and exit animations. Without this, the exit animation would not be visible; in effect, the popover would just disappear. Note that the `transition-behavior: allow-discrete` value is also set in the shorthand to activate the animation.
- `overlay` is added to the list of transitioned elements to ensure that the removal of the element from the top layer is deferred until the animation ends. This doesn't make a huge difference for animations such as this one, but in more complex cases, not doing this can result in the element being removed from the overlay too quickly, meaning the animation is not smooth or effective. Again, `transition-behavior: allow-discrete` is required in this case for the animation to occur.

> **Note:** We've also included a transition on the `::backdrop` that appears behind the popover when it opens, to provide a nice darkening animation. `[popover]:popover-open::backdrop` is used to select the backdrop when the popover is open.

Result

The code renders as follows:

> **Note:** Because popovers change from `display: none` to `display: block` each time they are shown, the popover transitions from its `@starting-style` styles to its `[popover]:popover-open` styles every time the entry transition occurs. When the popover closes, it transitions from its `[popover]:popover-open` state to the default `[popover]` state.

> **Note:** You can find an example that demonstrates transitioning a `<dialog>` element and its backdrop as it is shown and hidden on the `<dialog>` reference page — see Transitioning dialog elements.

Transitioning elements on DOM addition and removal

This example contains a button which, when pressed, appends new elements to a `<section>` container. Each element, in turn, contains a nested button, which when pressed, removes the element. This example demonstrates how to use transitions to animate elements when they are added to or removed from the DOM.

HTML

```
<button>Create new column</button>
<section></section>
```

JavaScript

JavaScript enables the addition and removal of elements:

```
const btn = document.querySelector("button");
const sectionElem = document.querySelector("section");

btn.addEventListener("click", createColumn);

function randomBackground() {
  function randomNum() {
    return Math.floor(Math.random() * 255);
```

```
  }
  const baseColor = `${randomNum()} ${randomNum()} ${randomNum()}`;

  return `linear-gradient(to right, rgb(${baseColor} / 0), rgb(${baseColor} / 0.5))`;
}

function createColumn() {
  const divElem = document.createElement("div");
  divElem.style.background = randomBackground();

  const closeBtn = document.createElement("button");
  closeBtn.textContent = "✕";
  closeBtn.setAttribute("aria-label", "close");
  divElem.append(closeBtn);
  sectionElem.append(divElem);

  closeBtn.addEventListener("click", () => {
    divElem.classList.add("fade-out");

    setTimeout(() => {
      divElem.remove();
    }, 1000);
  });
}
```

When the "Create new column" button is clicked, the `createColumn()` function is called. This creates a `<div>` element with a randomly generated background color and a `<button>` element to close the `<div>`. It then appends the `<button>` to the `<div>` and the `<div>` to the `<section>` container.

We then add an event listener to the close button via `addEventListener()`. Clicking the close button does two things:

- Adds the `fade-out` class to the `<div>`. Adding the class triggers the exit animation set on that class.
- Removes the `<div>` after a 1000ms delay. The `setTimeout()` delays removal of the `<div>` from the DOM (via `Element.remove()`) until after the animation ends.

CSS

We include a `transition` that animates the `opacity` and `scale` of each column as they are added and removed:

```
html * {
  box-sizing: border-box;
  font-family: sans-serif;
}

body {
  margin: 0;
  display: flex;
  flex-direction: column;
  height: 100vh;
  gap: 10px;
}

body > button {
  margin: 10px 10px 0 10px;
}

section {
  display: flex;
  flex: 1;
  gap: 10px;
  margin: 10px;
}
```

```
div {
  flex: 1;
  border: 1px solid gray;
  position: relative;
  opacity: 1;
  scale: 1 1;

  transition:
    opacity 0.7s,
    scale 0.7s,
    display 0.7s allow-discrete,
    all 0.7s allow-discrete;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

/* Include after the `div` rule */
@starting-style {
  div {
    opacity: 0;
    scale: 1 0;
  }
}

.fade-out {
  opacity: 0;
  display: none;
  scale: 1 0;
}

div > button {
  font-size: 1.6rem;
  background: none;
  border: 0;
  text-shadow: 2px 1px 1px white;
  border-radius: 15px;
  position: absolute;
  top: 1px;
  right: 1px;
```

```
  cursor: pointer;
}
```

To animate the `opacity` and `scale` of each `<div>` as it is added to the DOM and then reverse the animation as it is removed from the DOM, we:

- Specify the ending state of the properties we want to transition on the `div { ... }` rule.
- Specify the starting state from which to transition the properties inside a `@starting-style` block.
- Specify the exit animation inside the `.fade-out` rule — this is the class that the JavaScript assigns to the `<div>` elements when their close buttons are pressed. Besides setting the `opacity` and `scale` ending states, we also set `display: none` on the `<div>`s — we want them to become immediately unavailable when removed from the UI.
- Specify the `transition` list inside the `div { ... }` rule to animate `opacity`, `scale`, and `display`. Note that for `display`, the `transition-behavior: allow-discrete` value is also set in the shorthand so that it will animate.

Result

The final result looks like this:

See also

- CSS transitions module
- `overlay`
- `transition-behavior`
- `CSSStartingStyleRule`
- Four new CSS features for smooth entry and exit animations on developer.chrome.com (2023)

@supports CSS at-rule

The **`@supports`** CSS at-rule lets you specify CSS declarations that depend on a browser's support for CSS features. Using this at-rule is commonly called a *feature query*. The rule must be placed at the top level of your code or nested inside any other conditional group at-rule.

In JavaScript, `@supports` can be accessed via the CSS object model interface `CSSSupportsRule`.

Try it

```
.flex-container > * {
  padding: 0.3em;
  list-style-type: none;
  text-shadow: 0 0 2px red;
  float: left;
}

@supports (display: flex) {
  .flex-container > * {
    text-shadow: 0 0 2px blue;
    float: none;
  }

  .flex-container {
    display: flex;
  }
}
```

```
<ul class="flex-container">
  <li><a href="#">Index</a></li>
  <li><a href="#">About me</a></li>
  <li><a href="#">Contact</a></li>
</ul>
```

Syntax

```
@supports (<supports-condition>) {
  /* If the condition is true, use the CSS in this block. */
}

@supports (<supports-condition>) and (<supports-condition>) {
  /* If both conditions are true, use the CSS in this block. */
}
```

The `@supports` at-rule consists of a block of statements with a *supports condition*. The conditions can be combined by conjunctions (`and`), disjunctions (`or`), and/or negations (`not`). The precedence of operators can be defined with parentheses.

Supports conditions can use either a `<property>: <value>` declaration syntax or a `<function()>` syntax. The following sections describe the use of each type of supports condition.

Declaration syntax

The declaration syntax checks if a browser supports the specified `<property>: <value>` declaration. The declaration must be surrounded by parentheses. The following example returns true if the browser supports the expression `transform-origin: 5% 5%`:

```
@supports (transform-origin: 5% 5%) {
}
```

Function syntax

The function syntax checks if a browser supports values or expressions within the function. The functions supported in the function syntax are described in the following sections.

`at-rule()`

This function checks if a browser supports the specified at-rule. The following example returns true and applies the contained CSS styles if the browser supports the `@keyframes` at-rule:

```
@supports at-rule(@keyframes) {
}
```

`selector()`

This function evaluates if a browser supports the specified selector syntax. The following example returns true and applies the contained CSS styles if the browser supports the child combinator:

```
@supports selector(h2 > p) {
}
```

`font-tech()`

This function checks if a browser supports the specified font technology for layout and rendering. The following example returns true and applies the contained CSS styles if the browser supports the `COLRv1` font technology:

```
@supports font-tech(color-COLRv1) {
}
```

The table below describes the font technologies (`<font-tech>`), including color font technologies (`<color-font-tech>`), font feature technologies (`<font-features-tech>`), and other available font technologies that can be queried using the `font-tech()` function:

Technology	Supports
`<color-font-tech>`	
`color-colrv0`	Multi-colored glyphs via COLR version 0 table
`color-colrv1`	Multi-colored glyphs via COLR version 1 table
`color-svg`	SVG multi-colored tables
`color-sbix`	Standard bitmap graphics tables
`color-cbdt`	Color bitmap data tables
`<font-features-tech>`	
`features-opentype`	OpenType `GSUB` and `GPOS` tables
`features-aat`	TrueType `morx` and `kerx` tables
`features-graphite`	Graphite features, namely `Silf`, `Glat`, `Gloc`, `Feat`, and `Sill` tables
Other `<font-tech>` values	
`incremental-patch`	Incremental font loading using the patch subset method
`incremental-range`	Incremental font loading using the range request method
`incremental-auto`	Incremental font loading using method negotiation
`variations`	Font variations in TrueType and OpenType fonts to control the font axis, weight, glyphs, etc.
`palettes`	Font palettes by means of `font-palette` to select one of many color palettes in the font

`font-format()`

This function checks if a browser supports the specified font format for layout and rendering. The following example returns true and applies the contained CSS styles if the browser supports the `opentype` font format:

```
@supports font-format(opentype) {
}
```

The following table describes the available formats (`<font-format>` values) that can be queried with this function:

Format	Description	File extensions
`collection`	OpenType Collection	`.otc`, `.ttc`
`embedded-opentype`	Embedded OpenType	`.eot`
`opentype`	OpenType	`.ttf`, `.otf`
`svg`	SVG Font (deprecated)	`.svg`, `.svgz`
`truetype`	TrueType	`.ttf`
`woff`	WOFF 1.0 (Web Open Font Format)	`.woff`
`woff2`	WOFF 2.0 (Web Open Font Format)	`.woff2`

`named-feature()`

This function takes one predefined keyword and evaluates if a browser supports the specified named feature. This is to allow for feature support that cannot be tested for using the other `@supports` functions.

```
@supports named-feature(anchor-position-follows-transform) {
}
```

> **Note:** More features will be added rarely and only when there is real demand for feature testing something specific that would not be covered by more general testing methods.

`anchor-position-follows-transforms`

This `named-feature()` checks if the browser supports transform-aware anchor positioning.

`single-axis-scroll-container`

This `named-feature()` checks if the browser supports the ability to have single-axis scroll containers, where one axis is either `scroll`, or `auto`, or `hidden` and the other is `clip`.

The not operator

The `not` operator precedes an expression resulting in the negation of the expression. The following returns true if the browser's `transform-origin` property considers `10em 10em 10em` **to be invalid:**

```
@supports not (transform-origin: 10em 10em 10em) {
}
```

As with any operator, the `not` operator can be applied to a declaration of any complexity. The following examples are both valid:

```
@supports not (not (transform-origin: 2px)) {
}
@supports (display: grid) and (not (display: inline-grid)) {
}
```

> **Note:** There is no need to enclose the `not` operator between two parentheses at the top level. To combine it with other operators, like `and` and `or`, the parentheses are required.

The and operator

The `and` operator creates a new expression from the conjunction of two shorter expressions. It returns true only if **both** of the shorter expressions are also true. The following example returns true if and only if the two shorter expressions are simultaneously true:

```
@supports (display: table-cell) and (display: list-item) {
}
```

Multiple conjunctions can be juxtaposed without the need of more parentheses. The following are both equivalent:

```
@supports (display: table-cell) and (display: list-item) and (display: contents) {
}
@supports (display: table-cell) and
  ((display: list-item) and (display: contents)) {
}
```

The or operator

The `or` operator creates a new expression from the disjunction of two shorter expressions. It returns true if **one or both** of the shorter expressions is also true. The following example returns true if at least one of the two shorter expressions is true:

```
@supports (transform-style: preserve) or (-moz-transform-style: preserve) {
}
```

Multiple disjunctions can be juxtaposed without the need of more parentheses. The following are both equivalent:

```
@supports (transform-style: preserve) or (-moz-transform-style: preserve) or
  (-webkit-transform-style: preserve) {
}

@supports (transform-style: preserve-3d) or
  (
    (-moz-transform-style: preserve-3d) or
      (-webkit-transform-style: preserve-3d)
  ) {
}
```

> **Note:** When using both `and` and `or` operators, the parentheses must be used to define the order in which they apply. Otherwise, the condition is invalid and the whole rule is ignored.

Formal syntax

```
@supports =
  @supports <supports-condition> { <rule-list> }

<supports-condition> =
  not <supports-in-parens>                          |
  <supports-in-parens> [ and <supports-in-parens> ]*  |
  <supports-in-parens> [ or <supports-in-parens> ]*

<supports-in-parens> =
  ( <supports-condition> )  |
  <supports-feature>        |
  <general-enclosed>

<supports-feature> =
  <supports-selector-fn>       |
  <supports-font-tech-fn>      |
  <supports-font-format-fn>    |
  <supports-at-rule-fn>        |
  <supports-named-feature-fn>  |
  <supports-env-fn>            |
  <supports-decl>

<general-enclosed> =
  [ <function-token> <any-value>? ) ]  |
  [ ( <any-value>? ) ]

<supports-selector-fn> =
  selector( <complex-selector> )

<supports-font-tech-fn> =
  font-tech( <font-tech> )

<supports-font-format-fn> =
  font-format( <font-format> )

<supports-at-rule-fn> =
  at-rule( <at-keyword-token> )

<supports-named-feature-fn> =
  named-feature( <ident> )

<supports-env-fn> =
  env( <ident> )

<supports-decl> =
  ( [ <declaration> | <supports-condition-name> ] )

<complex-selector> =
  <complex-selector-unit> [ <combinator>? <complex-selector-unit> ]*

<font-tech> =
  <font-features-tech>  |
  <color-font-tech>     |
  variations            |
  palettes              |
  incremental
```

```
<font-format> =
  <string>           |
  collection         |
  embedded-opentype  |
  opentype           |
  svg                |
  truetype           |
  woff               |
  woff2

<complex-selector-unit> =
  [ <compound-selector>? <pseudo-compound-selector>* ]!

<combinator> =
  '>'                |
  '+'                |
  '~'                |
  [ '|' '|' ]        |
  [ / <wq-name> / ]

<font-features-tech> =
  features-opentype  |
  features-aat       |
  features-graphite

<color-font-tech> =
  color-COLRv0  |
  color-COLRv1  |
  color-SVG     |
  color-sbix    |
  color-CBDT

<compound-selector> =
  [ <type-selector>? <subclass-selector>* ]!

<pseudo-compound-selector> =
  <pseudo-element-selector> <pseudo-class-selector>*

<wq-name> =
  <ns-prefix>? <ident-token>

<type-selector> =
  <wq-name>         |
  <ns-prefix>? '*'

<subclass-selector> =
  <id-selector>            |
  <class-selector>         |
  <attribute-selector>     |
  <pseudo-class-selector>

<pseudo-element-selector> =
  : <pseudo-class-selector>         |
  <legacy-pseudo-element-selector>

<pseudo-class-selector> =
  : <ident-token>                   |
  : <function-token> <any-value> )

<ns-prefix> =
  [ <ident-token> | '*' ]? '|'

<id-selector> =
  <hash-token>

<class-selector> =
  '.' <ident-token>

<attribute-selector> =
  '[' <wq-name> ']'                                  |
  '[' <wq-name> <attr-matcher> [ <string-token> | <ident-token> ] <attr-modifier>? ']'

<legacy-pseudo-element-selector> =
  : [ before | after | first-line | first-letter ]

<attr-matcher> =
  [ '~' | '|' | '^' | '$' | '*' ]? '='

<attr-modifier> =
  i  |
  s
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 3, CSS Conditional Rules Module Level 4, CSS Conditional Rules Module Level 5, CSS Fonts Module Level 5, Media Queries Level 5, Selectors Level 4, Selectors Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Testing for the support of a CSS property

```
@supports (animation-name: test) {
  /* CSS applied when animations are supported without a prefix */
  @keyframes {
    /* Other at-rules can be nested inside */
  }
}
```

Testing for the support of a given CSS property or a prefixed version

```
@supports (text-stroke: 10px) or (-webkit-text-stroke: 10px) {
  /* CSS applied when text-stroke, prefixed or not, is supported */
}
```

Testing for the non-support of a specific CSS property

```
@supports not ((text-align-last: justify) or (-moz-text-align-last: justify)) {
  /* CSS to provide fallback alternative for text-align-last: justify */
```

```
}
```

Testing for the support of a selector

CSS conditional rules provide the ability to test for the support of a selector such as `:has()`.

```
/* This rule won't be applied in browsers that don't support :has() */
ul:has(> li li) {
  /* CSS is applied when the :has(…) pseudo-class is supported */
}

@supports not selector(:has(a, b)) {
  /* Fallback for when :has() is unsupported */
  ul > li,
  ol > li {
    /* The above expanded for browsers that don't support :has(…) */
  }
}

/* Note: So far, there's no browser that supports the `of` argument of :nth-child(…) */
@supports selector(:nth-child(1n of a, b)) {
  /* This rule needs to be inside the @supports block, otherwise
     it will be partially applied in browsers which don't support
     the `of` argument of :nth-child(…) */
  :is(:nth-child(1n of ul, ol) a, details > summary) {
    /* CSS applied when the :is(…) selector and
       the `of` argument of :nth-child(…) are both supported */
  }
}
```

Testing for the support of a font technology

The following example applies the Bungee Spice color font if the browser supports the `COLRv1` font technology:

```
@supports font-tech(color-COLRv1) {
  body {
    font-family: "Bungee Spice", fantasy;
  }
}
```

It's also possible to test for the support of a font technology by using the `tech` function inside the `@font-face` at-rule. In the following example, if a browser doesn't support the color font technology in the `bungee-spice.woff2` font, a regular `bungee.woff2` font will be used instead.

```
@font-face {
  font-family: "Bungee Spice";
  src:
    url("bungee-spice.woff2") tech(color-COLRv1) format("woff2"),
    url("bungee.woff2") format("woff2");
}
```

Testing for the support of a font format

The following example uses the WOFF2 version of the font if the browser supports this font format, otherwise it falls back to the previously specified WOFF version:

```
@font-face {
  font-family: "Open Sans WOFF";
  src: url("open-sans.woff") format("woff");
}

@font-face {
  font-family: "Open Sans WOFF2";
  src: url("open-sans.woff2") format("woff2");
}

body {
  font-family: "Open Sans WOFF", sans-serif;
}

@supports font-format(woff2) {
  body {
    font-family: "Open Sans WOFF2", sans-serif;
  }
}
```

However, a more efficient way to specify multiple font formats is to list them in the `src` descriptor of a single `@font-face` at-rule in the order from the most preferred format to the least preferred:

```
@font-face {
  font-family: "Open Sans";
  src:
    url("open-sans.woff2") format("woff2"),
    url("open-sans.woff") format("woff");
}

body {
  font-family: "Open Sans", sans-serif;
}
```

Testing for the support of an at-rule

The following example applies a set of scoped color scheme styles if the browser supports the `@scope` at-rule:

```
@supports at-rule(@scope) {
  @scope (.light-scheme) {
    :scope {
      background-color: plum;
    }

    a {
      color: darkmagenta;
```

```
    }
  }

  @scope (.dark-scheme) {
    :scope {
      background-color: darkmagenta;
      color: antiquewhite;
    }

    a {
      color: plum;
    }
  }
}
```

See also

- Using feature queries
- CSS at-rule functions
- `CSSSupportsRule`
- `CSS.supports()` method

@view-transition CSS at-rule

The **`@view-transition`** CSS at-rule is used to opt in the current and destination documents to undergo a view transition, in the case of a cross-document navigation.

For a cross-document view transition to work, the current and destination documents of the navigation need to be on the same origin.

Syntax

```
@view-transition {
  navigation: auto | none;
  types: none | <custom-ident>#;
}
```

Descriptors

`navigation`

A keyword specifying the effect this at-rule will have on the document's view transition behavior. Possible values are:

`auto`

The document will undergo a view transition when taking part in a navigation, provided the navigation is same-origin, without cross-origin redirects, and its `navigationType` is `traverse`, `push`, or `replace`. In the case of `push` or `replace`, the navigation must be initiated by a user interacting with the page content, not by a browser UI feature.

`none`

The document will not undergo a view transition.

`types`

Specifies the view transition types to set on the active view transition for the current and destination documents. Possible values are:

`<custom-ident>+`

One or more space-separated `<custom-ident>` values representing the types to set.

`none`

No types are set.

Formal syntax

```
@view-transition =
  @view-transition { <declaration-list> }
```

This syntax reflects the latest standard as per CSS View Transitions Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Transitioning page view

The following code snippets show key concepts used in a page transition demo. The demo uses cross-document view-transitions; a half second transition that occurs when navigating between two pages of a site. For the full demo, see the View transitions multi-page app demo.

The `@view-transition` at-rule is specified in the CSS for both your current and destination documents of a navigation to opt them both in to the view transition:

```
@view-transition {
  navigation: auto;
}
```

In addition to the `@view-transition` at-rule, we use the `@keyframes` at-rule to define two keyframe animations and use the `animation` shorthand property to apply those keyframe animations to the elements in the outbound (`::view-transition-old()`) and inbound (`::view-transition-new()`) pages that we want to animate.

```
/* Create a custom animation */
@keyframes move-out {
  from {
    transform: translateY(0%);
  }

  to {
    transform: translateY(-100%);
  }
}

@keyframes move-in {
  from {
    transform: translateY(100%);
  }
```

```
  to {
    transform: translateY(0%);
  }
}

/* Apply the custom animation to the old and new page states */
::view-transition-old(root) {
  animation: 0.4s ease-in both move-out;
}

::view-transition-new(root) {
  animation: 0.4s ease-in both move-in;
}
```

See this transitions multi-page app demo live.

Using view transition types

Our MPA transition types example (source code) demonstrates how to use `types` via `@view-transition`:

```
@view-transition {
  navigation: auto;
  types: slide;
}
```

See Using types with cross-document view transitions via `@view-transition` for a walkthrough of the referenced example.

See also

- `::view-transition`
- `::view-transition-new()`
- `::view-transition-old()`
- `::view-transition-group()`
- `::view-transition-image-pair()`
- View Transition API
- Using view transition types
- CSS at-rules
- CSS at-rule functions

CSS at-rule functions

CSS at-rule functions are at-rule statements that represent complex rules or can invoke special data processing or calculations.

Syntax

```
@identifier function([argument]? [, argument]!) {
}
```

The syntax begins with the at symbol `@` and an at-rule identifier, such as `import`. This is followed by the **name of the at-rule function**, such as `url`, followed by a pair of opening and closing parentheses. One or more arguments are specified inside the parentheses.

Some at-rule functions can take multiple arguments, which are formatted similar to CSS property values. White space is allowed, but it is optional inside the parentheses. Multiple arguments can be separated using a comma or a space.

@import functions

The `@import` at-rule is used to import styles from other stylesheets.

`@import url()`

: Imports a stylesheet file from the specified URL.

`@import supports()`

: Imports a stylesheet file based on browser support.

`@import layer()`

: Imports a stylesheet file into the specified cascade layer.

@supports functions

The `@supports` at-rule checks for a browser's support for the specified CSS feature and then applies the CSS styling.

`@supports selector()`

: Applies CSS rules after checking browser's support for the specified selector syntax.

`@supports font-tech()`

: Applies CSS rules after checking browser's support for the specified font technology.

`@supports font-format()`

: Applies CSS rules after checking browser's support for the specified font format.

@namespace functions

The `@namespace` at-rule is used to specify XML namespaces to be used in a CSS stylesheet.

`@namespace url()`

: Defines XML namespace from the specified URL.

@container functions

The `@container` at-rule is used to specify styles for a containment context.

`@container style()`

Defines the containment context style.

See also

- CSS at-rules
- CSS syntax module

Firefox (-moz-) vendor-prefixed CSS extensions

A vendor prefix is used to indicate that a feature is specific to a certain browser. Firefox supports several extensions to CSS, which are prefixed with `-moz-`.

-moz-prefixed properties without standard equivalents

> **Note:** These extensions are mostly experimental or deprecated but kept for backward compatibility. You should avoid using them on production websites.

- `-moz-float-edge`
- `-moz-force-broken-image-icon` : Use `alt` text instead.
- `-moz-orient`
- `-moz-osx-font-smoothing`: A similar `font-smooth` equivalent exists.
- `-moz-user-focus`
- `-moz-user-input`
- `-moz-user-modify`: A non-prefixed `user-modify` equivalent exists, but the HTML `contenteditable` attribute is recommended instead.

-moz-prefixed properties with standard equivalents

The following properties were implemented first using the `-moz-` vendor prefix, but are now supported in Firefox using the standard (non-prefixed) syntax. Support for the prefixed version is typically dropped eventually, so use the standard property instead.

A

- `-moz-animation` : Prefixed version still supported.
- `-moz-animation-delay` : Prefixed version still supported.
- `-moz-animation-direction` : Prefixed version still supported.
- `-moz-animation-duration` : Prefixed version still supported.
- `-moz-animation-fill-mode` : Prefixed version still supported.
- `-moz-animation-iteration-count` : Prefixed version still supported.
- `-moz-animation-name` : Prefixed version still supported.
- `-moz-animation-play-state` : Prefixed version still supported.
- `-moz-animation-timing-function` : Prefixed version still supported.
- `-moz-appearance`: The prefixed version of `appearance` is still supported.

B

- `-moz-backface-visibility` : Prefixed version still supported.
- `-moz-background-clip` : Not supported; use `background-clip`.
- `-moz-background-origin` : Not supported; use `background-origin`.
- `-moz-box-align` : Use CSS flexbox with `align-items`.
- `-moz-background-inline-policy` : Not supported; use `box-decoration-break`.
- `-moz-box-direction` : Use CSS flexbox with `flex-direction`.
- `-moz-box-flex` : Use CSS flexbox with `flex-grow`.
- `-moz-box-ordinal-group` : Use CSS flexbox with `order`.
- `-moz-box-orient` : Use CSS flexbox with `flex-direction`.
- `-moz-box-pack` : Use CSS flexbox with `justify-content`.
- `-moz-background-size` : Not supported; use `background-size`.
- `-moz-border-end` : Not supported; use `border-inline-end`.
- `-moz-border-end-color` : Not supported; use `border-inline-end-color`.
- `-moz-border-end-style` : Not supported; use `border-inline-end-style`.
- `-moz-border-end-width` : Not supported; use `border-inline-end-width`.
- `-moz-border-image` : Not supported; use `border-inline-end-width`.
- `-moz-border-start` : Not supported; use `border-inline-start`.
- `-moz-border-start-color` : Not supported; use `border-inline-start-color`.
- `-moz-border-start-style` : Not supported; use `border-inline-start-style`.
- `-moz-border-start-width` : Not supported; use `border-inline-start-width`.
- `-moz-box-sizing` : Prefixed version still supported.

C

- `-moz-column-count` : Not supported; use `column-count`.
- `-moz-column-fill` : Not supported; use `column-fill`.
- `-moz-column-gap` : Not supported; use `column-gap`.
- `-moz-column-width` : Not supported; use `column-width`.
- `-moz-column-rule` : Not supported; use `column-rule`.
- `-moz-column-rule-width` : Not supported; use `column-rule-width`.
- `-moz-column-rule-style` : Not supported; use `column-rule-style`.
- `-moz-column-rule-color` : Not supported; use `column-rule-color`.

F–M

- `-moz-font-feature-settings` : Prefixed version still supported.
- `-moz-font-language-override` : Prefixed version still supported.
- `-moz-hyphens` : Prefixed version still supported.
- `-moz-margin-end` : Not supported; use `margin-inline-end`.
- `-moz-margin-start` : Not supported; use `margin-inline-start`.

O-P

- `-moz-opacity` : Not supported; use `opacity`.
- `-moz-outline` : Not supported; use `outline`.
- `-moz-outline-color` : Not supported; use `outline-color`.
- `-moz-outline-offset` : Not supported; use `outline-offset`.

- `-moz-outline-style` : Not supported; use `outline-style`.
- `-moz-outline-width` : Not supported; use `outline-width`.
- `-moz-padding-end` : Not supported; use `padding-inline-end`.
- `-moz-padding-start` : Not supported; use `padding-inline-start`.
- `-moz-perspective` : Prefixed version still supported.
- `-moz-perspective-origin` : Prefixed version still supported.

T–Z

- `-moz-tab-size`: Use `tab-size`
- `-moz-text-align-last` : Not supported; use `text-align-last`.
- `-moz-text-decoration-color` : Not supported; use `text-decoration-color`.
- `-moz-text-decoration-line` : Not supported; use `text-decoration-line`.
- `-moz-text-decoration-style` : Not supported; use `text-decoration-style`.
- `-moz-text-size-adjust`: See `text-size-adjust` .
- `-moz-transform` : Prefixed version still supported.
- `-moz-transform-origin` : Prefixed version still supported.
- `-moz-transform-style` : Prefixed version still supported.
- `-moz-transition` : Not supported; use `transition`.
- `-moz-transition-delay` : Not supported; use `transition-delay`.
- `-moz-transition-duration` : Not supported; use `transition-duration`.
- `-moz-transition-property` : Not supported; use `transition-property`.
- `-moz-transition-timing-function` : Not supported; use `transition-timing-function`.
- `-moz-user-select`: Use `user-select`.

-moz-prefixed property values

Global values

- `-moz-initial`: See `initial`.

background-image

Property: `background-image`

- `-moz-linear-gradient` : Use `linear-gradient()`.
- `-moz-radial-gradient` : Use `radial-gradient()`.
- `-moz-element` : Use `element()`.
- `-moz-image-rect()`

border-style and outline-style

Properties: `border-style` and `outline-style`.

- `-moz-bg-inset`
- `-moz-bg-outset`
- `-moz-bg-solid`

<color> keywords

Type: `<color>`

- `-moz-activehyperlinktext`
- `-moz-hyperlinktext`
- `-moz-visitedhyperlinktext`
- `-moz-buttonhoverface`
- `-moz-buttonhovertext`
- `-moz-default-background-color`
- `-moz-default-color`
- `-moz-cellhighlight`
- `-moz-cellhighlighttext`
- `-moz-field`
- `-moz-fieldtext`
- `-moz-dialog`
- `-moz-dialogtext`
- `-moz-menuhover`
- `-moz-menuhovertext`

empty-cells

Property: `empty-cells`

- `-moz-show-background` (default value in quirks mode)

font-family

Property: `font-family`

- `-moz-fixed`

image-rendering

Property: `image-rendering`

- `-moz-crisp-edges` : Use `crisp-edges`.

<length>

Type: `<length>`

- `-moz-calc`: Use `calc()`.

list-style-type

Property: `list-style-type`

Several vendor-prefixed values for `list-style-type` are now supported as standard values without a `-moz-` prefix, except for those in the following list. See Browser Compatibility for details.

- `-moz-ethiopic-halehame`
- `-moz-ethiopic-halehame-am`
- `-moz-ethiopic-halehame-ti-er`
- `-moz-ethiopic-halehame-ti-et`
- `-moz-hangul`
- `-moz-hangul-consonant`
- `-moz-urdu`

text-align

Property: `text-align`

- `-moz-center` : Use `text-align: center`.
- `-moz-left` : Use `text-align: left`.
- `-moz-right` : Use `text-align: right`.

width, min-width, and max-width

Properties: `width`, `min-width`, and `max-width`

- `-moz-min-content`: See `min-content`.
- `-moz-fit-content`: See `fit-content`.
- `-moz-max-content`: See `max-content`.
- `-moz-available`: See `stretch`.

Pseudo-classes

- `:-moz-any` : Use `:is`.
- `:-moz-any-link` : Use `:any-link`.
- `:-moz-broken`
- `:-moz-drag-over`
- `:-moz-first-node`
- `:-moz-full-screen` : Not supported; use `:fullscreen`.
- `:-moz-full-screen-ancestor` : Not supported; use `:fullscreen`.
- `:-moz-handler-blocked`
- `:-moz-handler-crashed`
- `:-moz-handler-disabled`
- `:-moz-last-node`
- `:-moz-loading`
- `:-moz-locale-dir(ltr)`
- `:-moz-locale-dir(rtl)`
- `:-moz-native-anonymous`
- `:-moz-placeholder` : Use `:placeholder-shown`.
- `:-moz-only-whitespace`
- `:-moz-read-only` : Use `:read-only`.
- `:-moz-read-write` : Use `:read-write`.
- `:-moz-submit-invalid`: See `:invalid`.
- `:-moz-suppressed`
- `:-moz-ui-invalid` : Use `:user-invalid`.
- `:-moz-ui-valid` : Use `:user-valid`.
- `:-moz-user-disabled`
- `:-moz-window-inactive`

Pseudo-elements

- `::-moz-canvas`
- `::-moz-cell-content`
- `::-moz-color-swatch`
- `::-moz-focus-inner`
- `::-moz-focus-outer`
- `::-moz-inline-table`
- `::-moz-list-bullet`
- `::-moz-list-number`
- `::-moz-meter-bar`
- `::-moz-pagebreak`
- `::-moz-pagecontent`
- `::-moz-placeholder` : Use `::placeholder`.
- `::-moz-progress-bar`
- `::-moz-range-progress`
- `::-moz-range-thumb`
- `::-moz-range-track`
- `::-moz-scrolled-canvas`
- `::-moz-scrolled-content`
- `::-moz-selection` : Use `::selection`.
- `::-moz-svg-foreign-content`
- `::-moz-table`
- `::-moz-table-cell`
- `::-moz-table-column`
- `::-moz-table-column-group`
- `::-moz-table-outer`
- `::-moz-table-row`
- `::-moz-table-row-group`
- `::-moz-viewport`
- `::-moz-viewport-scroll`

At-rules

- `@-moz-document`

Media features

- `-moz-device-pixel-ratio`
- `-moz-platform`
- `-moz-windows-glass`

Other

- `-moz-alt-content`: See Firefox bug 11011

See also

- WebKit (-webkit-) vendor-prefixed CSS extensions
- Vendor Prefix glossary entry

CSS properties

A CSS ***property*** is a parameter used in a CSS declaration that lets you style certain aspects of selected elements. For example, the `opacity` property is used to set the opacity of a selected element, allowing you to control if content behind that element is visible:

```
/* Set 0.8 opacity on <img> elements */
img {
  opacity: 0.8;
}
```

Each property has a name (e.g., `opacity`), a value (e.g., `0.8`), and a defined behavior on the rendering of the document. CSS also defines shorthand properties, so you can specify multiple related properties in a single declaration. For example, the `margin` property is a shorthand for `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`, setting the margin of all four sides of an element:

```
/* Give <img> elements 1rem of margin */
img {
  margin: 1rem;
}
```

Syntax

```
selector {
  property: value;
}
```

Alphabetical index of properties

Standard properties defined by CSS specifications include the following:

Misc

- Custom properties (`--*`): CSS variables

A

- `accent-color`
- `align-content`
- `align-items`
- `align-self`
- `alignment-baseline`
- `all` (shorthand)
- `anchor-name`
- `anchor-scope`
- `animation-composition`
- `animation-delay`
- `animation-direction`
- `animation-duration`
- `animation-fill-mode`
- `animation-iteration-count`
- `animation-name`
- `animation-play-state`
- `animation-range-end`
- `animation-range-start`
- `animation-range` (shorthand)
- `animation-timeline`
- `animation-timing-function`
- `animation` (shorthand)
- `appearance`
- `aspect-ratio`

B

- `backdrop-filter`
- `backface-visibility`
- `background-attachment`
- `background-blend-mode`
- `background-clip`
- `background-color`
- `background-image`
- `background-origin`
- `background-position-x`
- `background-position-y`
- `background-position`
- `background-repeat` (shorthand)

- background-repeat-x
- background-repeat-y
- background-size
- background (shorthand)
- baseline-shift
- baseline-source
- block-size

Border-*

- border-block-color
- border-block-end-color
- border-block-end-style
- border-block-end-width
- border-block-end (shorthand)
- border-block-start-color
- border-block-start-style
- border-block-start-width
- border-block-start (shorthand)
- border-block-style
- border-block-width
- border-block (shorthand)
- border-bottom-color
- border-bottom-left-radius
- border-bottom-right-radius
- border-bottom-style
- border-bottom-width
- border-bottom (shorthand)
- border-collapse
- border-color (shorthand)
- border-end-end-radius
- border-end-start-radius
- border-image-outset
- border-image-repeat
- border-image-slice
- border-image-source
- border-image-width
- border-image (shorthand)
- border-inline-color
- border-inline-end-color
- border-inline-end-style
- border-inline-end-width
- border-inline-end (shorthand)
- border-inline-start-color
- border-inline-start-style
- border-inline-start-width
- border-inline-start (shorthand)
- border-inline-style
- border-inline-width
- border-inline (shorthand)
- border-left-color
- border-left-style
- border-left-width
- border-left (shorthand)
- border-radius (shorthand)
- border-right-color
- border-right-style
- border-right-width
- border-right (shorthand)
- border-spacing
- border-start-end-radius
- border-start-start-radius
- border-style (shorthand)
- border-top-color
- border-top-left-radius
- border-top-right-radius
- border-top-style
- border-top-width
- border-top (shorthand)
- border-width (shorthand)
- border (shorthand)

B - C

- bottom
- box-align
- box-decoration-break
- box-direction
- box-flex-group
- box-flex
- box-lines
- box-ordinal-group
- box-orient
- box-pack
- box-shadow
- box-sizing

- break-after
- break-before
- break-inside
- caption-side
- caret-animation
- caret-color
- caret-shape
- caret (shorthand)
- clear
- clip-path
- clip-rule
- clip
- color-interpolation-filters
- color-interpolation
- color-scheme
- color
- column-count
- column-fill
- column-gap
- column-height
- column-rule-color
- column-rule-style
- column-rule-width
- column-rule (shorthand)
- column-span
- column-width
- column-wrap
- columns (shorthand)
- contain-intrinsic-block-size
- contain-intrinsic-height
- contain-intrinsic-inline-size
- contain-intrinsic-size (shorthand)
- contain-intrinsic-width
- contain
- container-name
- container-type
- container (shorthand)
- content-visibility
- content
- corner-block-end-shape
- corner-block-start-shape
- corner-bottom-left-shape
- corner-bottom-right-shape
- corner-bottom-shape (shorthand)
- corner-end-end-shape
- corner-end-start-shape
- corner-inline-end-shape
- corner-inline-start-shape
- corner-left-shape (shorthand)
- corner-right-shape (shorthand)
- corner-shape (shorthand)
- corner-start-end-shape
- corner-start-start-shape
- corner-top-left-shape
- corner-top-right-shape
- corner-top-shape (shorthand)
- counter-increment
- counter-reset
- counter-set
- cursor
- cx
- cy

D - F

- d
- direction
- display
- dominant-baseline
- dynamic-range-limit
- empty-cells
- field-sizing
- fill-opacity
- fill-rule
- fill
- filter
- flex-basis
- flex-direction
- flex-flow (shorthand)
- flex-grow
- flex-shrink
- flex-wrap
- flex (shorthand)
- float
- flood-color

- flood-opacity
- font-family
- font-feature-settings
- font-kerning
- font-language-override
- font-optical-sizing
- font-palette
- font-size-adjust
- font-size
- font-smooth
- font-stretch
- font-style
- font-synthesis-position
- font-synthesis-small-caps
- font-synthesis-style
- font-synthesis-weight
- font-synthesis (shorthand)
- font-variant-alternates
- font-variant-caps
- font-variant-east-asian
- font-variant-emoji
- font-variant-ligatures
- font-variant-numeric
- font-variant-position
- font-variant (shorthand)
- font-variation-settings
- font-weight
- font-width
- font (shorthand)
- forced-color-adjust

G - I

- gap (shorthand)
- grid-area (shorthand)
- grid-auto-columns
- grid-auto-flow
- grid-auto-rows
- grid-column-end
- grid-column-start
- grid-column (shorthand)
- grid-row-end
- grid-row-start
- grid-row (shorthand)
- grid-template-areas
- grid-template-columns
- grid-template-rows
- grid-template (shorthand)
- grid (shorthand)
- hanging-punctuation
- height
- hyphenate-character
- hyphenate-limit-chars
- hyphens
- image-orientation
- image-rendering
- image-resolution
- initial-letter
- inline-size
- inset-block-end
- inset-block-start
- inset-block (shorthand)
- inset-block (shorthand)
- inset-inline-end
- inset-inline-start
- inset-inline (shorthand)
- inset-inline (shorthand)
- inset (shorthand)
- inset (shorthand)
- interactivity
- interest-delay
- interest-delay-end
- interest-delay-start
- interpolate-size
- isolation

J - M

- justify-content
- justify-items
- justify-self
- left
- letter-spacing
- lighting-color
- line-break
- line-clamp

- line-height-step
- line-height
- list-style-image
- list-style-position
- list-style-type
- list-style (shorthand)
- list-style (shorthand)
- margin-block-end
- margin-block-start
- margin-block (shorthand)
- margin-bottom
- margin-inline-end
- margin-inline-start
- margin-inline (shorthand)
- margin-left
- margin-right
- margin-top
- margin-trim
- margin (shorthand)
- marker-end
- marker-mid
- marker-start
- marker
- mask-border-mode
- mask-border-outset
- mask-border-repeat
- mask-border-slice
- mask-border-source
- mask-border-width
- mask-border (shorthand)
- mask-clip
- mask-composite
- mask-image
- mask-mode
- mask-origin
- mask-position
- mask-repeat
- mask-size
- mask-type
- mask (shorthand)
- math-depth
- math-shift
- math-style
- max-block-size
- max-height
- max-inline-size
- max-width
- min-block-size
- min-height
- min-inline-size
- min-width
- mix-blend-mode

O - P

- object-fit
- object-position
- object-view-box
- offset-anchor
- offset-distance
- offset-path
- offset-position
- offset-rotate
- offset (shorthand)
- opacity
- order
- orphans
- outline-color
- outline-offset
- outline-style
- outline-width
- outline (shorthand)
- overflow-anchor
- overflow-block
- overflow-clip-margin
- overflow-inline
- overflow-wrap
- overflow-x
- overflow-y
- overflow (shorthand)
- overlay
- overscroll-behavior-block
- overscroll-behavior-inline
- overscroll-behavior-x
- overscroll-behavior-y

- overscroll-behavior (shorthand)
- padding-block-end
- padding-block-start
- padding-block (shorthand)
- padding-bottom
- padding-inline-end
- padding-inline-start
- padding-inline (shorthand)
- padding-left
- padding-right
- padding-top
- padding (shorthand)
- page-break-after
- page-break-before
- page-break-inside
- page
- paint-order
- path-length
- perspective-origin
- perspective
- place-content (shorthand)
- place-items (shorthand)
- place-self (shorthand)
- pointer-events
- position-anchor
- position-area
- position-try-fallbacks
- position-try-order
- position-try (shorthand)
- position-visibility
- position
- print-color-adjust

Q - S

- quotes
- r
- reading-flow
- reading-order
- resize
- right
- rotate
- row-gap
- ruby-align
- ruby-overhang
- ruby-position
- rx
- ry
- scale
- scroll-behavior
- scroll-initial-target
- scroll-margin-block-end
- scroll-margin-block-start
- scroll-margin-block (shorthand)
- scroll-margin-bottom
- scroll-margin-inline-end
- scroll-margin-inline-start
- scroll-margin-inline (shorthand)
- scroll-margin-left
- scroll-margin-right
- scroll-margin-top
- scroll-margin (shorthand)
- scroll-marker-group
- scroll-padding-block-end
- scroll-padding-block-start
- scroll-padding-block (shorthand)
- scroll-padding-bottom
- scroll-padding-inline-end
- scroll-padding-inline-start
- scroll-padding-inline (shorthand)
- scroll-padding-left
- scroll-padding-right
- scroll-padding-top
- scroll-padding (shorthand)
- scroll-snap-align
- scroll-snap-stop
- scroll-snap-type
- scroll-target-group
- scroll-timeline-axis
- scroll-timeline-name
- scroll-timeline (shorthand)
- scrollbar-color
- scrollbar-gutter
- scrollbar-width
- shape-image-threshold

- shape-margin
- shape-outside
- shape-rendering
- speak-as
- stop-color
- stop-opacity
- stroke-dasharray
- stroke-dashoffset
- stroke-linecap
- stroke-linejoin
- stroke-miterlimit
- stroke-opacity
- stroke-width
- stroke

T - Z

- tab-size
- table-layout
- text-align-last
- text-align
- text-anchor
- text-autospace
- text-box-edge
- text-box-trim
- text-box (shorthand)
- text-combine-upright
- text-decoration-color
- text-decoration-inset
- text-decoration-line
- text-decoration-skip-ink
- text-decoration-skip
- text-decoration-style
- text-decoration-thickness
- text-decoration (shorthand)
- text-emphasis-color
- text-emphasis-position
- text-emphasis-style
- text-emphasis (shorthand)
- text-indent
- text-justify
- text-orientation
- text-overflow
- text-rendering
- text-shadow
- text-size-adjust
- text-spacing-trim
- text-transform
- text-underline-offset
- text-underline-position
- text-wrap-mode
- text-wrap-style
- text-wrap (shorthand)
- timeline-scope
- top
- touch-action
- transform-box
- transform-origin
- transform-style
- transform
- transition-behavior
- transition-delay
- transition-duration
- transition-property
- transition-timing-function
- transition (shorthand)
- translate
- unicode-bidi
- user-modify
- user-select
- vector-effect
- vertical-align (shorthand)
- view-timeline-axis
- view-timeline-inset
- view-timeline-name
- view-timeline (shorthand)
- view-transition-class
- view-transition-name
- visibility
- white-space-collapse
- white-space
- widows
- width
- will-change
- word-break

- `word-spacing`
- `writing-mode`
- `x`
- `y`
- `z-index`
- `zoom`

Non-standard properties

Non-standard vendor-prefixed properties include:

`-moz-` prefix

- `-moz-float-edge`
- `-moz-force-broken-image-icon`
- `-moz-orient`
- `-moz-user-focus`
- `-moz-user-input`

`-webkit-` prefix

- `-webkit-box-reflect`
- `-webkit-border-before` (shorthand)
- `-webkit-mask-box-image` (shorthand)
- `-webkit-mask-composite`
- `-webkit-mask-position-x`
- `-webkit-mask-position-y`
- `-webkit-mask-repeat-x`
- `-webkit-mask-repeat-y`
- `-webkit-tap-highlight-color`
- `-webkit-text-fill-color`
- `-webkit-text-security`
- `-webkit-text-stroke` (shorthand)
- `-webkit-text-stroke-color`
- `-webkit-text-stroke-width`
- `-webkit-touch-callout`

See also

- CSS syntax
- Pseudo-elements
- Firefox (-moz-) vendor-prefixed CSS extensions
- WebKit (-webkit-) vendor-prefixed CSS extensions
- Vendor prefix

Custom properties (--*): CSS variables

Property names that are prefixed with `--`, like `--example-name`, represent *custom properties* that contain a value that can be used in other declarations using the `var()` function.

Custom properties are scoped to the element(s) they are declared on, and participate in the cascade: the value of such a custom property is that from the declaration decided by the cascading algorithm.

Initial value	see prose
Applies to	all elements
Inherited	yes
Computed value	as specified with variables substituted
Animation type	discrete

Syntax

```
--some-keyword: left;
--some-color: #123456;
--some-complex-value: 3px 6px rgb(20 32 54);
```

`<declaration-value>`

This value matches any sequence of one or more tokens, so long as the sequence does not contain any disallowed token. It represents the entirety of what a valid declaration can have as its value.

> **Note:** Custom property names are case sensitive — `--my-color` will be treated as a separate custom property to `--My-color`.

Example

Basic example

HTML

```
<p id="firstParagraph">
  This paragraph should have a blue background and yellow text.
</p>
<p id="secondParagraph">
  This paragraph should have a yellow background and blue text.
</p>
<div id="container">
  <p id="thirdParagraph">
    This paragraph should have a green background and yellow text.
  </p>
</div>
```

CSS

```
:root {
  --first-color: #1166ff;
```

```
  --second-color: #ffff77;
}

#firstParagraph {
  background-color: var(--first-color);
  color: var(--second-color);
}

#secondParagraph {
  background-color: var(--second-color);
  color: var(--first-color);
}

#container {
  --first-color: #229900;
}

#thirdParagraph {
  background-color: var(--first-color);
  color: var(--second-color);
}
```

Result

Registering custom properties with @property

In this example, we use the `@property` at-rule to register a custom property.

HTML

Our HTML includes an ordered list (`<ol>`) containing three list items (`<li>`).

```
<ol>
  <li class="one">Item one</li>
  <li class="two">Item two</li>
  <li class="three">Item three</li>
</ol>
```

CSS

We use the `@property` at-rule to register two custom properties.

```
@property --itemSize {
  syntax: "<length> | <percentage>";
  inherits: true;
  initial-value: 200px;
}

@property --borderWidth {
  syntax: "<length>";
  inherits: false;
  initial-value: 10px;
}
```

We try to override the custom property values. The values set on `.two` are valid while the values set on `.three` are invalid.

```
ol {
  --itemSize: 100px;
  --borderWidth: 1px;
}
.two {
  --itemSize: initial;
  --borderWidth: inherit;
}
.three {
  --itemSize: large;
  --borderWidth: 3%;
}
```

We use the two custom properties to style the items, setting the border and width for all the items at once:

```
li {
  width: var(--itemSize);
  border: var(--borderWidth) solid red;
  background-color: yellow;
  margin-bottom: 10px;
}
```

Results

The `--itemSize` property is inheritable; the `--borderWidth` is not. The properties are set on the `ol` parent, overriding the default values defined in their registration. Item one inherits the size but not the border width from the OL. The global keywords, declared for `.two`, are valid for `<length>`, so are used. The values in `.three` are invalid ("large" is not a `<length-percentage>` and `3%` is not a `<length>`). See

See also

- The `var()` function
- `@property` at-rule
- Using CSS custom properties (variables) guide
- Registering CSS custom properties guide
- CSS custom properties for cascading variables module

-moz-float-edge CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The non-standard **`-moz-float-edge`** CSS property specifies whether the height and width properties of the element include the margin, border, or padding thickness.

Syntax

```
/* Keyword values */
-moz-float-edge: content-box;
-moz-float-edge: margin-box;

/* Global values */
-moz-float-edge: inherit;
-moz-float-edge: initial;
-moz-float-edge: revert;
-moz-float-edge: revert-layer;
-moz-float-edge: unset;
```

Values

This property is specified as one of the following keyword values:

`content-box`

The height and width properties include the content, but not the padding, border or margin.

`margin-box`

The height and width properties include the content, padding, border and margin.

Formal definition

Initial value	`content-box`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-moz-float-edge =
  content-box  |
  margin-box
```

Examples

HTML

```
<div class="box">
  <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
</div>
```

CSS

```
.box {
  display: block;
  height: 5px;
  margin: 0.5em auto;
  color: gray;
  -moz-float-edge: margin-box;
  box-sizing: border-box;
}
```

Result

Specifications

Not part of any standard.

See also

- Firefox bug 432891

-moz-force-broken-image-icon CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-moz-force-broken-image-icon`** extended CSS property can be used to force the broken image icon to be shown even when a broken image has an `alt` attribute.

Syntax

```
-moz-force-broken-image-icon: 1;
-moz-force-broken-image-icon: 0;

/* Global values */
-moz-force-broken-image-icon: inherit;
-moz-force-broken-image-icon: initial;
-moz-force-broken-image-icon: revert;
-moz-force-broken-image-icon: revert-layer;
-moz-force-broken-image-icon: unset;
```

Values

This property is specified as the following value:

`<integer>`

A value of `1` means that the broken image icon is shown even if the image has an `alt` attribute. When the value `0` is used, the image will act as usual and only display the `alt` attribute.

> **Note:** Even if the value is set to `1` the `alt` attribute will still be displayed, alongside the broken image icon.

Formal definition

Initial value	`0`
Applies to	images
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-moz-force-broken-image-icon =
  <integer>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<img src="/broken/image/link.png" alt="Broken image link" />
```

CSS

```
img {
  -moz-force-broken-image-icon: 1;
  height: 100px;
  width: 100px;
}
```

Result

> **Note:** The alt text may not be visible if `-moz-force-broken-image-icon` is set to `1` and the image has no (or too small) `height` or `width` set.

Notes

- This property will only work in Gecko-based browsers.
- The use of this property is not recommended. A proper `alt` attribute should be used instead.

Specifications

Not part of any standard.

See also

- Firefox bug 58646

-moz-orient CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-moz-orient`** CSS property specifies the orientation of the element to which it's applied.

Syntax

```
-moz-orient: inline;

/* Global values */
-moz-orient: inherit;
-moz-orient: initial;
-moz-orient: revert;
-moz-orient: revert-layer;
-moz-orient: unset;
```

Values

This property is specified as one of the following keyword values:

`inline`

The element is rendered in the same direction as the axis of the text: horizontally for horizontal writing modes, vertically for vertical writing modes.

`block`

The element is rendered orthogonally to the axis of the text: vertically for horizontal writing modes, horizontal for vertical writing modes.

`horizontal`

The element is rendered horizontally.

`vertical`

The element is rendered vertically.

Formal definition

Initial value	`inline`

Applies to	any element; it has an effect on `<progress>` and `<meter>`, but not on <input type="range"> or other elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-moz-orient =
  inline     |
  block      |
  horizontal |
  vertical
```

Examples

HTML

```
<p>The following progress meter is horizontal (the default):</p>
<progress max="100" value="75"></progress>

<p>The following progress meter is vertical:</p>
<progress class="vert" max="100" value="75"></progress>
```

CSS

```
.vert {
  -moz-orient: vertical;
  width: 16px;
  height: 150px;
}
```

Result

Specifications

Not part of any standard. Though submitted to the W3C, with positive initial feedback, this property is not yet part of any specification; currently, this is a Mozilla-specific extension (that is, `-moz-orient`).

See also

- `box-orient`

-moz-user-focus CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-moz-user-focus`** CSS property is used to indicate whether an element can have the focus.

By setting its value to `ignore`, you can disable focusing the element, which means that the user will not be able to activate the element, and the element will be skipped in the tab sequence. The default is `none`, which disables focussing on the element and removes focus on other elements if there is an attempt to select the element.

Syntax

```
/* Keyword values */
-moz-user-focus: none;
-moz-user-focus: normal;
-moz-user-focus: ignore;

/* Global values */
-moz-user-focus: inherit;
-moz-user-focus: initial;
-moz-user-focus: unset;
```

Values

This property is specified as one of the following keyword values:

`ignore`

The element does not accept keyboard focus and will be skipped in the tab order.

`normal`

The element can accept keyboard focus.

`none`

The element does not accept keyboard focus. Attempting to select the element removes focus from any other element.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-moz-user-focus =
  ignore  |
  normal  |
  none
```

Examples

HTML

```
<input class="ignored" value="The user cannot focus on this element." />
```

CSS

```
.ignored {
  -moz-user-focus: ignore;
}
```

Specifications

Not part of any standard.

See also

- `-moz-user-input`
- `user-modify`
- `-moz-user-select`

-moz-user-input CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

In Mozilla applications, **`-moz-user-input`** determines if an element will accept user input.

As of Firefox 60, this property can no longer grant an element the ability to accept user input if it normally does not. It may only be used to disable user input.

The `user-input` property is currently not on a standards track.

Syntax

```
/* Keyword values */
-moz-user-input: auto;
-moz-user-input: none;

/* Global values */
-moz-user-input: inherit;
-moz-user-input: initial;
-moz-user-input: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The element will respond to user input if it normally takes user input, such as a `<textarea>`.

`none`

The element does not respond to user input, and it does not become `:active`.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
-moz-user-input =
  auto  |
  none
```

Examples

Disabling user input for an element

```
input.example {
  /* The user will be able to select the text, but not change it. */
  -moz-user-input: none;
}
```

Specifications

Not part of any standard.

See also

- `-moz-user-focus`
- `-moz-user-modify`
- `-moz-user-select`

-webkit-border-before CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they

> can be a suitable alternative in specific cases where no standard option exists.

The **`-webkit-border-before`** CSS shorthand property sets the individual logical block start border property values in a single place in the style sheet.

Constituent properties

This property is a shorthand for the following CSS properties:

- `-webkit-border-before-color`
- `-webkit-border-before-style`
- `-webkit-border-before-width`

Syntax

```
/* Border values */
-webkit-border-before: 1px;
-webkit-border-before: 2px dotted;
-webkit-border-before: medium dashed blue;

/* Global values */
-webkit-border-before: inherit;
-webkit-border-before: initial;
-webkit-border-before: revert;
-webkit-border-before: revert-layer;
-webkit-border-before: unset;
```

Values

This property is specified as a space-separated list of one to three of the following values:

`<'border-width'>`

See `border-width`

`<'border-style'>`

See `border-style`

`<'color'>`

See `color`

Description

The `-webkit-border-before` property maps to a physical border depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top`, `border-right`, `border-bottom`, or `border-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

It relates to `-webkit-border-after`, `-webkit-border-start`, and `-webkit-border-end`, which define the other borders of the element.

The standard-track equivalent of this property is `border-block-start`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-block-start-width`: `medium` • `border-block-start-style`: `none` • `border-block-start-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-block-start-width`: the absolute `<length>`, snapped as a line width • `border-block-start-style`: as specified • `border-block-start-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-block-start-width`: by computed value type • `border-block-start-style`: discrete • `border-block-start-color`: by computed value type

Formal syntax

```
-webkit-border-before =
  <'border-width'>  ||
  <'border-style'>  ||
  <color>

<border-width> =
  <'border-top-width'>{1,4}

<border-style> =
  <'border-top-style'>{1,4}

<border-top-width> =
  <line-width>

<border-top-style> =
  <line-style>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

```
<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying a border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  -webkit-border-before: 5px dashed blue;
}
```

Result

Specifications

Not part of any standard, but it relates to the standards-track `border-block-start` property.

See also

- `border-block-start`
- The mapped physical properties: `border-top`, `border-right`, `border-bottom`, and `border-left`
- `writing-mode`, `direction`, `text-orientation`

-webkit-box-reflect CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-webkit-box-reflect`** CSS property lets you reflect the content of an element in one specific direction.

Syntax

```
/* Direction values */
-webkit-box-reflect: above;
-webkit-box-reflect: below;
-webkit-box-reflect: left;
-webkit-box-reflect: right;

/* Offset value */
-webkit-box-reflect: below 10px;

/* Mask value */
-webkit-box-reflect: below 0 linear-gradient(transparent, white);

/* Global values */
-webkit-box-reflect: inherit;
-webkit-box-reflect: initial;
-webkit-box-reflect: revert;
-webkit-box-reflect: revert-layer;
-webkit-box-reflect: unset;
```

Values

This property is specified as a space-separated list of the following values:

`above`, `below`, `right`, `left`

Are keywords indicating in which direction the reflection is to happen.

`<length>`

Indicates the size of the reflection.

`<image>`

Describes the mask to be applied to the reflection.

Formal definition

Initial value	none
Applies to	all elements

Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-webkit-box-reflect =
  [ above | below | right | left ]? <length>? <image>?

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Specifications

Not part of any standard. The standard way to do reflection in CSS is to use the CSS `element()` function.

See also

- The Apple documentation.
- The WebKit specification.

-webkit-mask-box-image CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The non-standard prefixed **`-webkit-mask-box-image`** CSS shorthand property sets the mask image for an element's border box.

> **Note:** This property is not standard and is not on any standards track. Consider using the `mask-border` property instead.

Constituent properties

This property is a shorthand for the following CSS properties:

- `-webkit-mask-border-source`
- `-webkit-mask-border-outset`
- `-webkit-mask-border-repeat`

The values includes the `<image>` to be used as the mask border, and optionally four border outset values and up to two border repeat styles.

Syntax

```
/* default */
-webkit-mask-box-image: none;

/* image */
-webkit-mask-box-image: url("image.png");

/* image edge-offset */
-webkit-mask-box-image: url("image.png") 10 20 20 10;
-webkit-mask-box-image: url("image.png") 10px 20px 20px 10px;

/* image repeat-style */
-webkit-mask-box-image: url("image.png") space repeat;

/* image edge-offset repeat-style */
-webkit-mask-box-image: url("image.png") 10px 20px 20px 10px space repeat;

/* Global values */
-webkit-mask-box-image: inherit;
```

```
-webkit-mask-box-image: initial;
-webkit-mask-box-image: revert;
-webkit-mask-box-image: revert-layer;
-webkit-mask-box-image: unset;
```

Values

`<image>`

The location of the image resource to be used as a mask image, `<gradient>`, or other `<image>` value.

`none`

Used to specify that a border box is to have no mask image.

`<length>`

The size of the mask image's offset. See `<length>` for possible units.

`<percentage>`

The mask image's offset has a percentage value relative to the border box's corresponding dimension (width or height).

`<number>`

The size of the mask image's offset in pixels.

`repeat`

The mask image is repeated as many times as is necessary to span the border box. May include a partial image if the mask image does not divide evenly into the border box.

`stretch`

The mask image is stretched to contain the border box exactly.

`round`

The mask image is stretched somewhat and repeated such that there is no partial mask image at the end of the border box.

`space`

The mask image is repeated as many times as possible without stretching. There is no partial mask image at the end of the border box.

The outset values, or edge offsets, define the distances from the top, right, bottom, and left edges of the image, in that order. The values can be set as `<length>`, `<number>`, or `<percentage>`, with numbers interpreted as pixel lengths.

Border repeat styles, when included, are interpreted in the order of `<repeat-x> <repeat-y>`. If only one value is declared, the value is the same for both axes. While similar to `background-repeat`, the `cover` and `contain` values are not supported.

Formal definition

- Initial value: `none`
- Applies to: all elements
- Inherited: no
- Computed value: as specified

Formal syntax

```
-webkit-mask-box-image =
  <mask-image-source> [ <mask-image-offset>{4} <mask-border-repeat>{1,2} ]
```

Examples

Setting an image

```
.example-one {
  -webkit-mask-box-image: url("mask.png");
}
```

Offsetting and filling an image

```
.example-two {
  -webkit-mask-box-image: url("logo.png") 100px 100px 0px 0px round round;
}
```

Specifications

Not part of any standard.

See also

- CSS `mask-border` property
- CSS `border-image` property
- Safari CSS reference: `-webkit-mask-box-image`

-webkit-mask-composite CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-webkit-mask-composite`** property specifies the manner in which multiple mask images applied to the same element are composited with one another. Mask images are composited in the opposite order that they are declared with the `-webkit-mask-image` property.

> **Note:** There is a standardized `mask-composite` property covering parts of this non-standard property using different keywords.

Syntax

```
/* Keyword values */
-webkit-mask-composite: clear;
-webkit-mask-composite: copy;
-webkit-mask-composite: source-over;
-webkit-mask-composite: source-in;
-webkit-mask-composite: source-out;
-webkit-mask-composite: source-atop;
-webkit-mask-composite: destination-over;
-webkit-mask-composite: destination-in;
-webkit-mask-composite: destination-out;
-webkit-mask-composite: destination-atop;
-webkit-mask-composite: xor;

/* Global values */
-webkit-mask-composite: inherit;
-webkit-mask-composite: initial;
-webkit-mask-composite: revert;
-webkit-mask-composite: revert-layer;
-webkit-mask-composite: unset;
```

Values

This property is specified as one of the following keyword values:

`clear`

Overlapping pixels in the source mask image and the destination mask image are cleared.

`copy`

The source mask image replaces the destination mask image.

`source-over`

The source mask image is rendered over the destination mask image.

`source-in`

Overlapping pixels in the source mask image and the destination mask image are replaced by the pixels of the source mask image; all other pixels are cleared.

`source-out`

Overlapping pixels in the source mask image and the destination mask image are cleared; all remaining pixels of the source mask image are rendered.

`source-atop`

The pixels of the destination mask image are rendered. The pixels of the source mask image are rendered only if they overlap a nontransparent portion of the destination mask image. This causes the source mask image to have no effect.

`destination-over`

The destination mask image is rendered over the source mask image.

`destination-in`

Overlapping pixels in the source mask image and the destination mask image remain the pixels of the destination mask image; all other pixels are cleared.

`destination-out`

Overlapping pixels in the source mask image and the destination mask image are cleared; all remaining pixels of the source mask image are rendered.

`destination-atop`

The pixels of the source mask image are rendered. The pixels of the destination mask image are rendered only if they overlap a nontransparent portion of the destination mask image. This causes the destination mask image to have no effect.

`xor`

Overlapping pixels in the source mask image and the destination mask image become fully transparent if they are both fully opaque.

Formal definition

Initial value	`source-over`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-webkit-mask-composite =
  <composite-style>#
```

Examples

Compositing with XOR

```
.example {
  -webkit-mask-image: url("mask1.png"), url("mask2.png");
  -webkit-mask-composite: xor, source-over;
}
```

Specifications

Not part of any standard. This property is specified as `mask-composite` using different values.

See also

- `mask-composite`

-webkit-mask-position-x CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The `-webkit-mask-position-x` CSS property sets the initial horizontal position of a mask image.

Syntax

```
/* Keyword values */
-webkit-mask-position-x: left;
-webkit-mask-position-x: center;
-webkit-mask-position-x: right;

/* <percentage> values */
-webkit-mask-position-x: 100%;
-webkit-mask-position-x: -50%;

/* <length> values */
-webkit-mask-position-x: 50px;
-webkit-mask-position-x: -1cm;

/* Multiple values */
-webkit-mask-position-x:
  50px,
  25%,
  -3em;

/* Global values */
-webkit-mask-position-x: inherit;
-webkit-mask-position-x: initial;
-webkit-mask-position-x: revert;
-webkit-mask-position-x: revert-layer;
-webkit-mask-position-x: unset;
```

Values

This property is specified as one or more comma-separated values:

`<length-percentage>`

A length indicating the position of the left edge of the image relative to the box's left padding edge. Percentages are calculated against the horizontal dimension of the box padding area. That means, a value of `0%` means the left edge of the image is aligned with the box's left padding edge and a value of `100%` means the right edge of the image is aligned with the box's right padding edge.

`left`

Equivalent to `0%`.

`center`

Equivalent to `50%`.

`right`

Equivalent to `100%`.

Formal definition

Initial value	`0%`
Applies to	all elements
Inherited	no
Percentages	refer to the size of the box itself
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	discrete

Formal syntax

```
-webkit-mask-position-x =
  [ <length-percentage> | left | center | right ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Horizontally positioning a mask image

```
.exampleOne {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-position-x: right;
}

.exampleTwo {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-position-x: 25%;
}
```

Specifications

Not part of any standard.

See also

- `-webkit-mask-position`
- `-webkit-mask-position-y`
- `-webkit-mask-origin`

-webkit-mask-position-y CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The `-webkit-mask-position-y` CSS property sets the initial vertical position of a mask image.

Syntax

```
/* Keyword values */
-webkit-mask-position-y: top;
-webkit-mask-position-y: center;
-webkit-mask-position-y: bottom;

/* <percentage> values */
-webkit-mask-position-y: 100%;
-webkit-mask-position-y: -50%;

/* <length> values */
-webkit-mask-position-y: 50px;
-webkit-mask-position-y: -1cm;

/* Multiple values */
-webkit-mask-position-y:
  50px,
  25%,
  -3em;

/* Global values */
-webkit-mask-position-y: inherit;
-webkit-mask-position-y: initial;
-webkit-mask-position-y: revert;
-webkit-mask-position-y: revert-layer;
-webkit-mask-position-y: unset;
```

Values

This property is specified as one or more comma-separated values:

`<length-percentage>`

A length indicating the position of the top side of the image relative to the box's top padding edge. Percentages are calculated against the vertical dimension of the box padding area. A value of `0%` means the top edge of the image is aligned with the box's top padding edge and a value of `100%` means the bottom edge of the image is aligned with the box's bottom padding edge.

`top`

Equivalent to `0%`.

`bottom`

Equivalent to `100%`.

`center`

Equivalent to `50%`.

Formal definition

Initial value	`0%`
Applies to	all elements
Inherited	no
Percentages	refer to the size of the box itself
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	discrete

Formal syntax

```
-webkit-mask-position-y =
  [ <length-percentage> | top | center | bottom ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Vertically positioning a mask image

```
.exampleOne {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-position-y: bottom;
}

.exampleTwo {
```

```
  -webkit-mask-image: url("mask.png");
  -webkit-mask-position-y: 25%;
}
```

Specifications

Not part of any standard.

See also

- `-webkit-mask-position`
- `-webkit-mask-position-x`
- `-webkit-mask-origin`

-webkit-mask-repeat-x CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The `-webkit-mask-repeat-x` property specifies whether and how a mask image is repeated (tiled) horizontally.

Syntax

```
/* Keyword values */
-webkit-mask-repeat-x: repeat;
-webkit-mask-repeat-x: no-repeat;
-webkit-mask-repeat-x: space;
-webkit-mask-repeat-x: round;

/* Multiple values */
-webkit-mask-repeat-x: repeat, no-repeat, space;

/* Global values */
-webkit-mask-repeat-x: inherit;
-webkit-mask-repeat-x: initial;
-webkit-mask-repeat-x: revert;
-webkit-mask-repeat-x: revert-layer;
-webkit-mask-repeat-x: unset;
```

Values

This property is specified as one or more comma-separated values:

`repeat`

The mask image is repeated both horizontally and vertically.

`no-repeat`

The mask image is not repeated horizontally; only one copy of the mask image is drawn in that direction. The remainder of the masked element's content is not displayed.

`space`

The image is repeated as much as possible without clipping. The first and last images are pinned to either side of the element, and whitespace is distributed evenly between the images. The `mask-position` property is ignored unless only one image can be displayed without clipping. The only case where clipping happens using space is when there isn't enough room to display one image.

`round`

As the allowed space increases in size, the repeated images will stretch (leaving no gaps) until there is room for another one to be added. When the next image is added, all of the current ones compress to allow room. Example: An image with an original width of 260px, repeated three times, might stretch until each repetition is 300px wide, and then another image will be added. They will then compress to 225px.

Formal definition

Initial value	`repeat`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
-webkit-mask-repeat-x =
  [ repeat | no-repeat | space | round ]#
```

Examples

Using a repeating or non-repeating mask image

```
.example-one {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-repeat-x: repeat;
}

.example-two {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-repeat-x: no-repeat;
}
```

Using multiple mask images

You can specify a different `<repeat-style>` for each mask image, separated by commas:

```
.example-three {
  -webkit-mask-image: url("mask1.png"), url("mask2.png");
```

```
  -webkit-mask-repeat-x: repeat, space;
}
```

Each image is matched with the corresponding repeat style, from first specified to last.

Specifications

Not part of any standard.

See also

- `-webkit-mask-repeat`
- `-webkit-mask-repeat-y`

-webkit-mask-repeat-y CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The `-webkit-mask-repeat-y` property sets whether and how a mask image is repeated (tiled) vertically.

Syntax

```
/* Keyword values */
-webkit-mask-repeat-y: repeat;
-webkit-mask-repeat-y: no-repeat;
-webkit-mask-repeat-y: space;
-webkit-mask-repeat-y: round;

/* Multiple values */
-webkit-mask-repeat-y: repeat, no-repeat, space;

/* Global values */
-webkit-mask-repeat-y: inherit;
-webkit-mask-repeat-y: initial;
-webkit-mask-repeat-y: revert;
-webkit-mask-repeat-y: revert-layer;
-webkit-mask-repeat-y: unset;
```

Values

This property is specified as one or more comma-separated values:

`repeat`

The mask image is repeated vertically.

`no-repeat`

The mask image is not repeated vertically; only one copy of the mask image is drawn in that direction. The vertical remainder of the masked element's content is not displayed.

`space`

The image is repeated as much as possible without clipping. The first and last images are pinned to the top and bottom edge of the element, and whitespace is distributed evenly between the images. The `mask-position` property is ignored unless only one image can be displayed without clipping. The only case where clipping happens using space is when there isn't enough room to display one image.

`round`

As the allowed vertical space increases in size, the repeated images will stretch (leaving no gaps) until there is room for another one to be added. When the next image is added, all of the current ones compress to allow room. Example: An image with an original height of 260px, repeated three times, might stretch until each repetition is 300px high, and then another image will be added. They will then compress to a height of 225px.

Formal definition

Initial value	`repeat`
Applies to	all elements
Inherited	no
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	discrete

Formal syntax

```
-webkit-mask-repeat-y =
  [ repeat | no-repeat | space | round ]#
```

Examples

Using a repeating or non-repeating mask image

```
.example-one {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-repeat-y: repeat;
}

.example-two {
  -webkit-mask-image: url("mask.png");
  -webkit-mask-repeat-y: no-repeat;
}
```

Using multiple mask images

You can specify a different `<repeat-style>` for each mask image, separated by commas:

```
.example-three {
  -webkit-mask-image: url("mask1.png"), url("mask2.png");
```

```
  -webkit-mask-repeat-y: repeat, space;
}
```

Each image is matched with the corresponding repeat style, from first specified to last.

Specifications

Not part of any standard.

See also

- `-webkit-mask-repeat`
- `-webkit-mask-repeat-x`

-webkit-tap-highlight-color CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

`-webkit-tap-highlight-color` is a non-standard CSS property that sets the color of the highlight that appears over a link while it's being tapped. The highlighting indicates to the user that their tap is being successfully recognized, and indicates which element they're tapping on.

Syntax

```
-webkit-tap-highlight-color: red;
-webkit-tap-highlight-color: transparent; /* for removing the highlight */

/* Global values */
-webkit-tap-highlight-color: inherit;
-webkit-tap-highlight-color: initial;
-webkit-tap-highlight-color: revert;
-webkit-tap-highlight-color: revert-layer;
-webkit-tap-highlight-color: unset;
```

Values

This property is specified as the following value:

`<color>`

Defines the color of the highlight.

Formal definition

Initial value	`black`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
-webkit-tap-highlight-color =
  <color>
```

Specifications

Not part of any standard. Apple has a description in the Safari Web Content Guide.

See also

- WebKit CSS extensions
- Related CSS pseudo-classes:
  - `:hover`
  - `:active`
  - `:visited`

-webkit-text-fill-color CSS property

The **`-webkit-text-fill-color`** CSS property specifies the fill color of characters of text. If this property is not set, the value of the `color` property is used.

Syntax

```
/* <color> values */
-webkit-text-fill-color: red;
-webkit-text-fill-color: #123456;
-webkit-text-fill-color: rgb(100 200 0);

/* Global values */
-webkit-text-fill-color: inherit;
-webkit-text-fill-color: initial;
-webkit-text-fill-color: revert;
-webkit-text-fill-color: revert-layer;
-webkit-text-fill-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The foreground fill color of the element's text content.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	yes
Computed value	computed color
Animation type	a color

Formal syntax

```
-webkit-text-fill-color =
  <color>
```

This syntax reflects the latest standard as per Compatibility. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing the fill color

CSS

```
p {
  margin: 0;
  font-size: 3em;
  -webkit-text-fill-color: green;
}
```

HTML

```
<p>This text is green.</p>
```

Results

See also

- Introducing Text-Stroke on webkit.org (2006)
- CSS-Tricks article explaining this feature
- `-webkit-text-stroke-color`
- `-webkit-text-stroke-width`
- `-webkit-text-stroke`

-webkit-text-security CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

`-webkit-text-security` is a non-standard CSS property that obfuscates characters in a `<form>` field (such as `<input>` or `<textarea>`) by replacing them with a shape. It only affects fields that are *not* of `type=password`.

Syntax

```
-webkit-text-security: circle;
-webkit-text-security: disc;
-webkit-text-security: square;
-webkit-text-security: none;

/* Global values */
-webkit-text-security: inherit;
-webkit-text-security: initial;
-webkit-text-security: revert;
-webkit-text-security: revert-layer;
-webkit-text-security: unset;
```

Formal syntax

```
-webkit-text-security =
  circle  |
  disc    |
  square  |
  none
```

Examples

Obscuring a text input

Try typing in the field below. If your browser supports this property, the characters should be visually replaced with squares.

HTML

```
<label for="name">Name:</label> <input type="text" name="name" id="name" />
```

CSS

```
input {
  -webkit-text-security: square;
}
```

Result

Specifications

Not part of any standard.

See also

- WebKit CSS extensions

-webkit-text-stroke CSS property

The **`-webkit-text-stroke`** CSS shorthand property specifies the width and color of strokes for text characters.

Constituent properties

This property is a shorthand for the following CSS properties:

- `-webkit-text-stroke-color`
- `-webkit-text-stroke-width`

Syntax

```
/* Width and color values */
-webkit-text-stroke: 4px navy;

/* Global values */
-webkit-text-stroke: inherit;
-webkit-text-stroke: initial;
-webkit-text-stroke: revert;
-webkit-text-stroke: revert-layer;
-webkit-text-stroke: unset;
```

Values

This property is specified as two space-separated values:

`<length>`

The width of the stroke.

`<color>`

The color of the stroke.

Formal definition

Initial value	as each of the properties of the shorthand: • `-webkit-text-stroke-width`: `0` • `-webkit-text-stroke-color`: `currentcolor`
Applies to	all elements
Inherited	yes
Computed value	as each of the properties of the shorthand: • `-webkit-text-stroke-width`: absolute `<length>` • `-webkit-text-stroke-color`: computed color
Animation type	as each of the properties of the shorthand: • `-webkit-text-stroke-width`: discrete • `-webkit-text-stroke-color`: a color

Formal syntax

```
-webkit-text-stroke =
  <line-width>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, Compatibility. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adding a red text stroke

HTML

```
<p id="example">The stroke of this text is red.</p>
```

CSS

```
#example {
  font-size: 3em;
  margin: 0;
  -webkit-text-stroke: 2px red;
}
```

Result

See also

- Introducing Text-Stroke on webkit.org (2006)
- CSS-Tricks article explaining this feature
- `-webkit-text-stroke-width`

- `-webkit-text-stroke-color`
- `-webkit-text-fill-color`

-webkit-text-stroke-color CSS property

The **`-webkit-text-stroke-color`** CSS property specifies the stroke color of characters of text. If this property is not set, the value of the `color` property is used.

Syntax

```
/* <color> values */
-webkit-text-stroke-color: red;
-webkit-text-stroke-color: #e08ab4;
-webkit-text-stroke-color: rgb(200 100 0);

/* Global values */
-webkit-text-stroke-color: inherit;
-webkit-text-stroke-color: initial;
-webkit-text-stroke-color: revert;
-webkit-text-stroke-color: revert-layer;
-webkit-text-stroke-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The color of the stroke.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	yes
Computed value	computed color
Animation type	a color

Formal syntax

```
-webkit-text-stroke-color =
  <color>
```

This syntax reflects the latest standard as per Compatibility. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Varying the stroke color

HTML

```
<p>Text with stroke</p>
<input type="color" value="#ff0000" />
```

CSS

```
p {
  margin: 0;
  font-size: 4em;
  -webkit-text-stroke-width: 3px;
  -webkit-text-stroke-color: red; /* Can be changed in the live sample */
}
```

```
const colorPicker = document.querySelector("input");
colorPicker.addEventListener("change", (evt) => {
  document.querySelector("p").style.webkitTextStrokeColor = evt.target.value;
});
```

Results

See also

- Introducing Text-Stroke on webkit.org (2006)
- CSS-Tricks article explaining this feature
- `-webkit-text-fill-color`
- `-webkit-text-stroke-width`
- `-webkit-text-stroke`

-webkit-text-stroke-width CSS property

The **`-webkit-text-stroke-width`** CSS property specifies the width of the stroke for text.

Syntax

```
/* Keyword values */
-webkit-text-stroke-width: thin;
-webkit-text-stroke-width: medium;
-webkit-text-stroke-width: thick;

/* <length> values */
-webkit-text-stroke-width: 2px;
-webkit-text-stroke-width: 0.1em;
-webkit-text-stroke-width: 1mm;
-webkit-text-stroke-width: 5pt;

/* Global values */
-webkit-text-stroke-width: inherit;
```

```
-webkit-text-stroke-width: initial;
-webkit-text-stroke-width: revert;
-webkit-text-stroke-width: revert-layer;
-webkit-text-stroke-width: unset;
```

Values

This property is specified as the following value:

`<line-width>`

The width of the stroke.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	yes
Computed value	absolute `<length>`
Animation type	discrete

Formal syntax

```
-webkit-text-stroke-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, Compatibility. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Varying stroke widths

CSS

```
p {
  margin: 0;
  font-size: 4em;
  -webkit-text-stroke-color: red;
}

#thin {
  -webkit-text-stroke-width: thin;
}

#medium {
  -webkit-text-stroke-width: 3px;
}

#thick {
  -webkit-text-stroke-width: 1.5mm;
}
```

HTML

```
<p id="thin">Thin stroke</p>
<p id="medium">Medium stroke</p>
<p id="thick">Thick stroke</p>
```

Results

See also

- Introducing Text-Stroke on webkit.org (2006)
- CSS-Tricks article explaining this feature
- `-webkit-text-stroke-color`
- `-webkit-text-stroke`
- `-webkit-text-fill-color`

-webkit-touch-callout CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The `-webkit-touch-callout` CSS property controls the display of the default callout shown when you touch and hold a touch target.

When a target is touched and held on iOS, Safari displays a callout information about the link. This property allows disabling that behavior.

Syntax

```
/* Keyword values */
-webkit-touch-callout: default;
-webkit-touch-callout: none;

/* Global values */
-webkit-touch-callout: initial;
-webkit-touch-callout: inherit;
-webkit-touch-callout: revert;
```

```
-webkit-touch-callout: revert-layer;
-webkit-touch-callout: unset;
```

Values

This property is specified as one of the following keyword values:

`default`

The default callout is displayed.

`none`

The callout is disabled.

Formal definition

Initial value	`default`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
-webkit-touch-callout =
  default  |
  none
```

Examples

Turn off touch callout

```
.example {
  -webkit-touch-callout: none;
}
```

Specifications

Not part of any standard. Apple has a description in the Safari CSS Reference.

See also

- WebKit CSS extensions
- -webkit-touch-callout in the Apple developer documentation

accent-color CSS property

The **`accent-color`** CSS property sets the accent color for user-interface controls generated by some elements.

Try it

```
accent-color: red;
```

```
accent-color: #74992e;
```

```
accent-color: rgb(255 255 128);
```

```
accent-color: hsl(250 100% 34%);
```

```
<section class="default-example container" id="default-example">
  <div>
    <input checked id="example-element" type="checkbox" />
    <label for="example-element" id="example-label">Example Label</label>
  </div>
</section>
```

```
.container > div {
  display: flex;
  align-items: center;
}

#example-element {
  width: 40px;
  height: 40px;
}

#example-label {
  margin-left: 10px;
  font-size: x-large;
}
```

Syntax

```
/* Keyword value */
accent-color: auto;

/* <color> values */
accent-color: darkred;
accent-color: #5729e9;
accent-color: rgb(0 200 0);
accent-color: hsl(228 4% 24%);

/* Global values */
accent-color: inherit;
accent-color: initial;
```

```
accent-color: revert;
accent-color: revert-layer;
accent-color: unset;
```

Values

This property is specified as the keyword `auto` or one `<color>` value:

`auto`

Represents a UA-chosen color, which should match the accent color of the platform, if any.

`<color>`

Specifies the color to be used as the accent color.

Description

Browsers that support `accent-color` currently apply it to the following HTML elements:

- `<input type="checkbox">`
- `<input type="radio">`
- `<input type="range">`
- `<progress>`

Each user agent has an accent color, with variations to ensure legibility and contrast. That accent color is not used by every user-interface control nor in every state of the control. The `accent-color` is only applied to user-interface controls that use an accent color in the states where it is applicable.

> **Note:** To reduce the risk of fingerprinting, some browsers return a fixed value for `accent-color: auto` unless it is used in certain restricted circumstances. See browser compatibility for details.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	`auto` is computed as specified and `<color>` values are computed as defined for the `color` property.
Animation type	by computed value type

Formal syntax

```
accent-color =
  auto     |
  <color>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a custom accent color

HTML

```
<input type="checkbox" checked />
<input type="checkbox" class="custom" checked />
```

CSS

```
input {
  accent-color: auto;
  display: block;
  width: 30px;
  height: 30px;
}

input.custom {
  accent-color: rebeccapurple;
}
```

Result

See also

- `background-color`, `border-color`, `caret-color`, `color`, `column-rule-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`: Other color-related properties
- `<color>`: Related data type
- `<input>`: Related HTML element

align-content CSS property

The CSS **`align-content`** property sets the distribution of space between and around content items along a flexbox's cross axis, or a grid or block-level element's block axis.

This property has no effect on single line flex containers (i.e., ones with `flex-wrap: nowrap`).

The interactive example below uses grid layout to demonstrate some of the values of this property.

Try it

```
align-content: start;
```

```
align-content: center;
```

```
align-content: space-between;
```

```
align-content: space-around;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 60px 60px;
  grid-auto-rows: 40px;
  column-gap: 10px;
  height: 180px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Normal alignment */
align-content: normal;

/* Basic positional alignment */
/* align-content does not take left and right values */
align-content: start;
align-content: center;
align-content: end;
align-content: flex-start;
align-content: flex-end;

/* Baseline alignment */
align-content: baseline;
align-content: first baseline;
align-content: last baseline;

/* Distributed alignment */
align-content: space-between;
align-content: space-around;
align-content: space-evenly;
align-content: stretch;

/* Overflow alignment */
align-content: safe center;
align-content: unsafe center;

/* Global values */
align-content: inherit;
align-content: initial;
align-content: revert;
align-content: revert-layer;
align-content: unset;
```

Values

This property is specified as one or two of the following keyword values:

`normal`

> The items are packed in their default position as if no `align-content` value was set.

`start`

> The items are packed flush to each other against the start edge of the alignment container in the cross axis.

`center`

> The items are packed flush to each other in the center of the alignment container along the cross axis.

`end`

> The items are packed flush to each other against the end edge of the alignment container in the cross axis.

`flex-start`

> The items are packed flush to each other against the edge of the alignment container depending on the flex container's cross-start side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `start`.

`flex-end`

> The items are packed flush to each other against the edge of the alignment container depending on the flex container's cross-end side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `end`.

`baseline`, `first baseline`, `last baseline`

> Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group.

The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`space-between`

The items are evenly distributed within the alignment container along the cross axis. The spacing between each pair of adjacent items is the same. The first item is flush with the start edge of the alignment container in the cross axis, and the last item is flush with the end edge of the alignment container in the cross axis.

`space-around`

The items are evenly distributed within the alignment container along the cross axis. The spacing between each pair of adjacent items is the same. The empty space before the first and after the last item equals half of the space between each pair of adjacent items.

`space-evenly`

The items are evenly distributed within the alignment container along the cross axis. The spacing between each pair of adjacent items, the start edge and the first item, and the end edge and the last item, are all exactly the same.

`stretch`

If the combined size of the items along the cross axis is less than the size of the alignment container, any `auto`-sized items have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size exactly fills the alignment container along the cross axis.

`safe`

Used alongside an alignment keyword. If the chosen keyword means that the item overflows the alignment container causing data loss, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Used alongside an alignment keyword. Regardless of the relative sizes of the item and alignment container and whether overflow which causes data loss might happen, the given alignment value is honored.

> **Note:** The `<content-distribution>` values (`space-between`, `space-around`, `space-evenly`, and `stretch`) have no effect in block layout as all the content in that block is treated as a single alignment-subject.

Formal definition

Initial value	`normal`
Applies to	Block-containers, multi-column containers, flex containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
align-content =
  normal                                |
  <baseline-position>                   |
  <content-distribution>                |
  <overflow-position>? <content-position>

<baseline-position> =
  [ first | last ]?  &&
  baseline

<content-distribution> =
  space-between  |
  space-around   |
  space-evenly   |
  stretch

<overflow-position> =
  unsafe  |
  safe

<content-position> =
  center      |
  start       |
  end         |
  flex-start  |
  flex-end
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Effects of different align-content values

In this example, you can switch between three different `display` property values, including `flex`, `grid`, and `block`. You can also switch between the different values for `align-content`.

HTML

```
<section>
  <div class="olive">Olive</div>
  <div class="coral">Coral</div>
  <div class="deepskyblue">Deep<br />sky<br />blue</div>
  <div class="orchid">Orchid</div>
  <div class="slateblue">Slateblue</div>
  <div class="maroon">Maroon</div>
</section>
```

```
<fieldset class="controls">
  <legend>Controls</legend>
  <div class="row">
    <label for="display">display: </label>
    <select id="display">
      <option value="block" selected>block</option>
      <option value="flex">flex</option>
      <option value="grid">grid</option>
    </select>
  </div>
  <div class="row">
    <label for="alignContent">align-content: </label>
    <select id="alignContent">
      <option value="normal" selected>normal</option>
      <option value="start">start</option>
      <option value="center">center</option>
      <option value="end">end</option>
      <option value="flex-start">flex-start</option>
      <option value="flex-end">flex-end</option>
      <option value="space-between">space-between</option>
      <option value="space-around">space-around</option>
      <option value="space-evenly">space-evenly</option>
    </select>
  </div>
  <p>CSS applied</p>
  <pre>
section {
  display: <span id="displayStyle">block</span>;
  align-content: <span id="align">normal</span>
}
  </pre>
</fieldset>
```

CSS

```
body {
  font-size: 1.25rem;
  display: flex;
  gap: 1rem;
}
section div {
  font-family: monospace;
  padding: 3px;
}
```

```
section {
  border: solid 1.5px tomato;
  height: 300px;
  width: 300px;
  flex-wrap: wrap; /* used by flex only */
  gap: 0.2rem; /* not used by block */
}
.olive {
  background-color: olive;
}
.coral {
  background-color: coral;
}
.deepskyblue {
  background-color: deepskyblue;
}
.orchid {
  background-color: orchid;
}
.slateblue {
  background-color: slateblue;
  color: white;
}
.maroon {
  background-color: maroon;
  color: white;
}
```

```
const alignContent = document.querySelector("#alignContent");
const display = document.querySelector("#display");
const container = document.querySelector("section");
const displayStyle = document.querySelector("#displayStyle");
const alignStyle = document.querySelector("#align");

alignContent.addEventListener("change", () => {
  updatePage();
});
display.addEventListener("change", () => {
  updatePage();
});
function updatePage() {
  const alVal = alignContent.value;
  const dVal = display.value;
  container.style.alignContent = alVal;
  container.style.display = dVal;
  alignStyle.innerText = alVal;
  displayStyle.innerText = dVal;
}
updatePage();
```

Result

Try changing the `display` value and the `align-content` value.

In block layout, child elements are treated as a single element, meaning `space-between`, `space-around`, and `space-evenly` behave differently.

See also

- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment
- Block and inline layout in normal flow
- Block-level content
- `display`

align-items CSS property

The CSS **`align-items`** property sets the `align-self` value on all direct children as a group. In flexbox, it controls the alignment of items on the cross axis. In grid layout, it controls the alignment of items on the block axis within their grid areas.

The interactive example below uses grid layout to demonstrate some of the values of this property.

Try it

```
align-items: stretch;
```

```
align-items: center;
```

```
align-items: start;
```

```
align-items: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  width: 200px;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Basic keywords */
align-items: normal;
align-items: stretch;

/* Positional alignment */
/* align-items does not take left and right values */
align-items: center;
align-items: start;
align-items: end;
align-items: flex-start;
align-items: flex-end;
align-items: self-start;
align-items: self-end;
align-items: anchor-center;

/* Baseline alignment */
align-items: baseline;
align-items: first baseline;
align-items: last baseline;

/* Overflow alignment (for positional alignment only) */
align-items: safe center;
align-items: unsafe center;

/* Global values */
align-items: inherit;
align-items: initial;
align-items: revert;
align-items: revert-layer;
align-items: unset;
```

Values

This property is specified as one or two of the following keyword values:

`normal`

The effect of this keyword is dependent of the layout mode we are in:

- In absolutely-positioned layouts, the keyword behaves like `start` on *replaced* absolutely-positioned boxes, and as `stretch` on *all other* absolutely-positioned boxes.
- In static position of absolutely-positioned layouts, the keyword behaves as `stretch`.
- For flex items, the keyword behaves as `stretch`.
- For grid items, this keyword leads to a behavior similar to the one of `stretch`, except for boxes with an aspect ratio or an intrinsic size where it behaves like `start`.
- The property doesn't apply to block-level boxes, and to table cells.

`center`

The flex items' margin boxes are centered within the line on the cross-axis. If the cross-size of an item is larger than the flex container, it will overflow equally in both directions.

`start`

The items are packed flush to each other toward the start edge of the alignment container in the appropriate axis.

`end`

The items are packed flush to each other toward the end edge of the alignment container in the appropriate axis.

`self-start`

The items are packed flush to the edge of the alignment container's start side of the item, in the appropriate axis.

`self-end`

The items are packed flush to the edge of the alignment container's end side of the item, in the appropriate axis.

`baseline`, `first baseline`, `last baseline`

All flex items are aligned such that their flex container baselines align. The item with the largest distance between its cross-start margin edge and its baseline is flushed with the cross-start edge of the line.

`stretch`

If the item's cross-size is `auto`, the used size is set to the length necessary to be as close to filling the container as possible, respecting the item's width and height limits. If the item is not auto-sized, this value falls back to `flex-start`, and to `self-start` or `self-end` if the container's `align-content` is `first baseline` (or `baseline`) or `last baseline`.

`anchor-center`

In the case of anchor-positioned elements, aligns the items to the center of the associated anchor element in the block direction. See Centering on the anchor using `anchor-center`.

`safe`

Used alongside an alignment keyword. If the chosen keyword means that the item overflows the alignment container causing data loss, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Used alongside an alignment keyword. Regardless of the relative sizes of the item and alignment container and whether overflow which causes data loss might happen, the given alignment value is honored.

There are also two values that were defined for flexbox, as they are base on flex model axes concepts, that work in grid layouts as well:

`flex-start`

Used in flex layout only, aligns the flex items flush against the flex container's main-start or cross-start side. When used outside of a flex formatting context, this value behaves as `start`.

`flex-end`

Used in flex layout only, aligns the flex items flush against the flex container's main-end or cross-end side. When used outside of a flex formatting context, this value behaves as `end`.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
align-items =
  normal                                |
  stretch                               |
  <baseline-position>                   |
  <overflow-position>? <self-position>

<baseline-position> =
  [ first | last ]?  &&
  baseline

<overflow-position> =
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In this example we have a container with six children. A `<select>` dropdown menu enables toggling the `display` of the container between `grid` and `flex`. A second menu enables changing the value of the container's `align-items` property.

CSS

We style the container and items in a manner that ensures we have two lines or rows of items. We defined `.flex` and `.grid` classes, which will be applied to the container with JavaScript. They set the `display` value of the container, and change its background and border colors providing an additional indicator that the layout has changed. The six flex items each have a different background color, with the 4th item being two lines long and the 6th item having an enlarged font.

```
.flex,
.grid {
  height: 200px;
```

```
  width: 500px;
  align-items: initial; /* Change the value in the live sample */
  border: solid 5px transparent;
  gap: 3px;
}

.flex {
  display: flex;
  flex-wrap: wrap;
  background-color: #8c8c9f;
  border-color: magenta;
}

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, 100px);
  background-color: #9f8c8c;
  border-color: slateblue;
}

#item1 {
  background-color: #8cffa0;
  min-height: 30px;
}

#item2 {
  background-color: #a0c8ff;
  min-height: 50px;
}

#item3 {
  background-color: #ffa08c;
  min-height: 40px;
}

#item4 {
  background-color: #ffff8c;
  min-height: 60px;
}

#item5 {
  background-color: #ff8cff;
  min-height: 70px;
}

#item6 {
  background-color: #8cffff;
  min-height: 50px;
  font-size: 30px;
}
```

```
select {
  font-size: 16px;
}

.row {
  margin-top: 10px;
}

div > div {
  box-sizing: border-box;
  border: 2px solid white;
  width: 100px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

HTML

We include a container `<div>` with six nested `<div>` children. The HTML for the form and the JavaScript that changes the container's class have been hidden for the sake of brevity.

```
<div id="container" class="flex">
  <div id="item1">1</div>
  <div id="item2">2</div>
  <div id="item3">3</div>
  <div id="item4">4<br />line 2</div>
  <div id="item5">5</div>
  <div id="item6">6</div>
</div>
```

```
<div class="row">
  <label for="display">display: </label>
  <select id="display">
    <option value="flex">flex</option>
    <option value="grid">grid</option>
  </select>
</div>

<div class="row">
  <label for="values">align-items: </label>
  <select id="values">
    <option value="normal">normal</option>
    <option value="flex-start">flex-start</option>
    <option value="flex-end">flex-end</option>
    <option value="center" selected>center</option>
    <option value="baseline">baseline</option>
    <option value="stretch">stretch</option>

    <option value="start">start</option>
    <option value="end">end</option>
    <option value="self-start">self-start</option>
```

```
    <option value="self-end">self-end</option>

    <option value="first baseline">first baseline</option>
    <option value="last baseline">last baseline</option>

    <option value="safe center">safe center</option>
    <option value="unsafe center">unsafe center</option>
    <option value="safe right">safe right</option>
    <option value="unsafe right">unsafe right</option>
    <option value="safe end">safe end</option>
    <option value="unsafe end">unsafe end</option>
    <option value="safe self-end">safe self-end</option>
    <option value="unsafe self-end">unsafe self-end</option>
    <option value="safe flex-end">safe flex-end</option>
    <option value="unsafe flex-end">unsafe flex-end</option>
  </select>
</div>
```

```
const values = document.getElementById("values");
const display = document.getElementById("display");
const container = document.getElementById("container");

values.addEventListener("change", (evt) => {
  container.style.alignItems = evt.target.value;
});

display.addEventListener("change", (evt) => {
  container.className = evt.target.value;
});
```

Result

See also

- `align-self`
- `align-content`
- `justify-items`
- `place-items` shorthand
- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment module
- CSS flexible box layout module
- CSS grid layout module

align-self CSS property

The **`align-self`** CSS property overrides a grid or flex item's `align-items` value. In grid, it aligns the item inside the grid area. In flexbox, it aligns the item on the cross axis.

The property doesn't apply to block-level boxes, or to table cells. If a flexbox item's cross-axis margin is `auto`, then `align-self` is ignored.

Try it

```
align-self: stretch;
```

```
align-self: center;
```

```
align-self: start;
```

```
align-self: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  width: 200px;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Keyword values */
align-self: auto;
align-self: normal;

/* Positional alignment */
/* align-self does not take left and right values */
align-self: center; /* Put the item around the center */
align-self: start; /* Put the item at the start */
align-self: end; /* Put the item at the end */
align-self: self-start; /* Align the item flush at the start */
align-self: self-end; /* Align the item flush at the end */
align-self: flex-start; /* Put the flex item at the start */
```

```
align-self: flex-end; /* Put the flex item at the end */
align-self: anchor-center;

/* Baseline alignment */
align-self: baseline;
align-self: first baseline;
align-self: last baseline;
align-self: stretch; /* Stretch 'auto'-sized items to fit the container */

/* Overflow alignment */
align-self: safe center;
align-self: unsafe center;

/* Global values */
align-self: inherit;
align-self: initial;
align-self: revert;
align-self: revert-layer;
align-self: unset;
```

Values

This property is specified as one or two of the following keyword values:

`auto`

Computes to the parent's `align-items` value.

`normal`

The effect of this keyword is dependent of the layout mode we are in:

- In absolutely-positioned layouts, the keyword behaves like `start` on *replaced* absolutely-positioned boxes, and as `stretch` on *all other* absolutely-positioned boxes.
- In static position of absolutely-positioned layouts, the keyword behaves as `stretch`.
- For flex items, the keyword behaves as `stretch`.
- For grid items, this keyword leads to a behavior similar to the one of `stretch`, except for boxes with an aspect ratio or an intrinsic size where it behaves like `start`.
- The property doesn't apply to block-level boxes, and to table cells.

`self-start`

Aligns the items to be flush with the edge of the alignment container corresponding to the item's start side in the cross axis.

`self-end`

Aligns the items to be flush with the edge of the alignment container corresponding to the item's end side in the cross axis.

`flex-start`

The cross-start margin edge of the flex item is flushed with the cross-start edge of the line.

`flex-end`

The cross-end margin edge of the flex item is flushed with the cross-end edge of the line.

`center`

The flex item's margin box is centered within the line on the cross-axis. If the cross-size of the item is larger than the flex container, it will overflow equally in both directions.

`baseline`, `first baseline`, `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group. The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`stretch`

If the item's cross-size is `auto`, the used size is set to the length necessary to be as close to filling the container as possible, respecting the item's width and height limits. If the item is not auto-sized, this value falls back to `flex-start`, and to `self-start` or `self-end` if the container's `align-content` is `first baseline` (or `baseline`) or `last baseline`.

`anchor-center`

In the case of anchor-positioned elements, aligns the item to the center of the associated anchor element in the block direction. See Centering on the anchor using `anchor-center`.

`safe`

If the size of the item overflows the alignment container, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Regardless of the relative sizes of the item and alignment container, the given alignment value is honored.

Formal definition

Initial value	`auto`
Applies to	flex items, grid items, and absolutely-positioned boxes
Inherited	no
Computed value	`auto` computes to itself on absolutely-positioned elements, and to the computed value of `align-items` on the parent (minus any legacy keywords) on all other boxes, or `start` if the box has no parent. Its behavior depends on the layout model, as described for `justify-self`. Otherwise the specified value.
Animation type	discrete

Formal syntax

```
align-self =
  auto                                          |
  <overflow-position>? [ normal | <self-position> ]  |
  stretch                                       |
  <baseline-position>                           |
  anchor-center

<overflow-position> =
```

```
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end

<baseline-position> =
  [ first | last ]?  &&
  baseline
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3, CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<section>
  <div>Item #1</div>
  <div>Item #2</div>
  <div>Item #3</div>
</section>
```

CSS

```
section {
  display: flex;
  align-items: center;
  height: 120px;
  background: beige;
}

div {
  height: 60px;
  background: cyan;
  margin: 5px;
}

div:nth-child(3) {
  align-self: flex-end;
  background: pink;
}
```

Result

See also

- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment
- `align-items`
- `justify-self`
- `place-self`

alignment-baseline CSS property

The **`alignment-baseline`** CSS property specifies the specific baseline used to align the box's text and inline-level contents. **Baseline alignment** is the relationship among the baselines of multiple alignment subjects within an alignment context. When performing baseline alignment, the `alignment-baseline` property value specifies which baseline of the box is aligned to the corresponding baseline of its alignment context.

> **Note:** The `alignment-baseline` property only has an effect on inline-level boxes, flex items, grid items, table cells, and the `<text>`, `<textPath>`, and `<tspan>` SVG elements. If present, it overrides the shape's `alignment-baseline` attribute.

In an inline formatting context, inline-level box fragments and glyphs share an alignment context established by their parent inline box fragment along its inline axis. In SVG text layout, these values instead specify the baseline that is aligned to the SVG current text position.

Syntax

```
/* Initial value */
alignment-baseline: baseline;

/* Keyword values */
alignment-baseline: alphabetic;
alignment-baseline: central;
alignment-baseline: ideographic;
alignment-baseline: mathematical;
alignment-baseline: middle;
alignment-baseline: text-bottom;
alignment-baseline: text-top;

/* Mapped values */
alignment-baseline: text-before-edge; /* text-top */
alignment-baseline: text-after-edge; /* text-bottom */

/* Deprecated values  */
alignment-baseline: auto;
alignment-baseline: before-edge;
alignment-baseline: after-edge;
alignment-baseline: hanging;

/* Global values */
alignment-baseline: inherit;
alignment-baseline: initial;
```

```
alignment-baseline: revert;
alignment-baseline: revert-layer;
alignment-baseline: unset;
```

Values

This property is specified as one of the following keyword values:

`baseline`

Use the `dominant-baseline` value of the parent.

`alphabetic`

Used in writing Latin, Cyrillic, Greek, and many other scripts; matches the box's alphabetic baseline to that of its parent, corresponding to the bottom of most, but not all characters.

`central`

Matches the box's central baseline to the central baseline of its parent, corresponding to the ideographic central baseline, halfway between the ideographic-under and ideographic-over baselines.

`ideographic`

Matches the box's ideographic character face under-side baseline to that of its parent, with the derived baseline-table constructed using the ideographic baseline-table in the font.

`mathematical`

Matches the box's mathematical baseline to that of its parent, corresponding to the center baseline around which mathematical characters are designed.

`middle`

Aligns the vertical midpoint of the box with the baseline of the parent box plus half the x-height of the parent. Uses the x-middle baselines; except under `text-orientation: upright;` (where the alphabetic and x-height baselines are essentially meaningless), in which case it uses the `central` baseline instead.

`text-bottom`

Matches the bottom of the box to the top of the parent's content area, using the line-under edge of an inline's content box.

`text-top`

Matches the top of the box to the top of the parent's content area; the line-over edge of an inline's content box.

> **Note:** In SVG2, the `auto`, `before-edge`, and `after-edge` were deprecated and `text-before-edge` is an alias for `text-top`, and `text-after-edge` is an alias for `text-bottom`. These keywords should not be used as part of the `vertical-align` shorthand property. Browsers support `auto` as a synonym for `baseline` and `hanging`, wherein the alignment-point of the object being aligned is aligned with the "hanging" baseline of the parent text content element, but neither is part of the specification.

Formal definition

Initial value	`baseline`
Applies to	inline-level boxes, flex items, grid items, table cells, and SVG text content elements
Inherited	no
Computed value	the specified keyword
Animation type	discrete

Formal syntax

```
alignment-baseline =
  baseline           |
  <baseline-metric>

<baseline-metric> =
  text-bottom   |
  alphabetic    |
  ideographic   |
  middle        |
  central       |
  mathematical  |
  hanging       |
  text-top
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

```
<svg viewBox="0 0 450 160" width="700" height="200">
  <text x="50" y="20">alphabetic</text>
  <text x="50" y="60">central</text>
  <text x="50" y="100">hanging</text>
  <text x="50" y="140">ideographic</text>
  <text x="250" y="20">mathematical</text>
  <text x="250" y="60">middle</text>
  <text x="250" y="100">text-bottom</text>
  <text x="250" y="140">text-top</text>
  <path
    d="M   0,20 l 400,0
       m -400,40 l 400,0
       m -400,40 l 400,0
       m -400,40 l 400,0"
    stroke="grey" />
  <text x="0" y="20" fill="red">baseline</text>
  <text x="0" y="60" fill="red">baseline</text>
  <text x="0" y="100" fill="red">baseline</text>
  <text x="0" y="140" fill="red">baseline</text>
</svg>
```

```
text {
  font-size: 20px;
  alignment-baseline: baseline;
}
```

```
text:nth-of-type(1) {
  alignment-baseline: alphabetic;
}
text:nth-of-type(2) {
  alignment-baseline: central;
}
text:nth-of-type(3) {
  alignment-baseline: hanging;
}
text:nth-of-type(4) {
  alignment-baseline: ideographic;
}
text:nth-of-type(5) {
  alignment-baseline: mathematical;
}
text:nth-of-type(6) {
  alignment-baseline: middle;
}
text:nth-of-type(7) {
  alignment-baseline: text-bottom;
}
text:nth-of-type(8) {
  alignment-baseline: text-top;
}
```

See also

- `dominant-baseline`
- SVG `alignment-baseline` attribute
- CSS inline layout module
- CSS box alignment module

all CSS property

The **`all`** CSS shorthand property resets all of an element's properties except `unicode-bidi`, `direction`, and CSS Custom Properties. It can set properties to their initial or inherited values, or to the values specified in another cascade layer or stylesheet origin.

Try it

```
/* no all property */
```

```
all: initial;
```

```
all: inherit;
```

```
all: unset;
```

```
all: revert;
```

```
<section id="default-example">
  <div class="example-container-bg">
    <div class="example-container">
      <p id="example-element">
        This paragraph has a font size of 1.5rem and a color of gold. It also
        has 1rem of vertical margin set by the user-agent. The parent of the
        paragraph is a &lt;div&gt; with a dashed blue border.
      </p>
    </div>
  </div>
</section>
```

```
#example-element {
  color: gold;
  padding: 10px;
  font-size: 1.5rem;
  text-align: left;
  width: 100%;
}

.example-container {
  border: 2px dashed #2d5ae1;
}

.example-container-bg {
  background-color: #77767b;
  padding: 20px;
}
```

Constituent properties

This property is a shorthand for all CSS properties except for `unicode-bidi`, `direction`, and custom properties.

Syntax

```
/* Global values */
all: initial;
all: inherit;
all: unset;
all: revert;
all: revert-layer;
```

Values

The `all` property is specified as one of the CSS global keyword values. Note that none of these values affect the `unicode-bidi` and `direction` properties.

`initial`

Specifies that all the element's properties should be changed to their initial values.

`inherit`

Specifies that all the element's properties should be changed to their inherited values.

`unset`

Specifies that all the element's properties should be changed to their inherited values if they inherit by default, or to their initial values if not.

`revert`

Specifies behavior that depends on the stylesheet origin to which the declaration belongs:

- If the rule belongs to the author origin, the `revert` value rolls back the cascade to the user level, so that the specified values are calculated as if no author-level rules were specified for the element. For purposes of `revert`, the author origin includes the Override and Animation origins.
- If the rule belongs to the user origin, the `revert` value rolls back the cascade to the user-agent level, so that the specified values are calculated as if no author-level or user-level rules were specified for the element.
- If the rule belongs to the user-agent origin, the `revert` value acts like `unset`.

`revert-layer`

Specifies that all the element's properties should roll back the cascade to a previous cascade layer, if one exists. If no other cascade layer exists, the element's properties will roll back to the matching rule, if one exists, in the current layer or to a previous style origin.

Formal definition

Initial value	There is no practical initial value for it.
Applies to	all elements
Inherited	no
Computed value	as the specified value applies to each property this is a shorthand for.
Animation type	as each of the properties of the shorthand (all properties but `unicode-bidi` and `direction`)

Formal syntax

```
all =
  initial      |
  inherit      |
  unset        |
  revert       |
  revert-layer |
  revert-rule
```

This syntax reflects the latest standard as per CSS Cascading and Inheritance Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In this example, the CSS file contains styling for the `<blockquote>` element in addition to some styling for the parent `<body>` element. Various outputs in the Results subsection demonstrate how the styling of the `<blockquote>` element is affected when different values are applied to the `all` property inside the `blockquote` rule.

HTML

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
</blockquote>
Phasellus eget velit sagittis.
```

CSS

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
  margin: 0;
  padding: 0;
}

blockquote {
  background-color: skyblue;
  color: red;
}
```

Results

A. No `all` property

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
</blockquote>
Phasellus eget velit sagittis.
```

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
}
blockquote {
  background-color: skyblue;
  color: red;
}
```

This is the scenario in which no `all` property is set inside the `blockquote` rule. The `<blockquote>` element uses the browser's default styling which gives it a margin, together with a specific background and text color as specified in the stylesheet. It also behaves as a *block* element: the text that follows it is beneath it.

B. `all: initial`

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
```

```
</blockquote>
Phasellus eget velit sagittis.
```

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
}
blockquote {
  background-color: skyblue;
  color: red;
  all: initial;
}
```

With the `all` property set to `initial` in the `blockquote` rule, the `<blockquote>` element doesn't use the browser default styling anymore: it is an *inline* element now (initial value), its `background-color` is `transparent` (initial value), its `font-size` is `medium`, and its `color` is `black` (initial value).

C. `all: inherit`

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
</blockquote>
Phasellus eget velit sagittis.
```

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
}
blockquote {
  background-color: skyblue;
  color: red;
  all: inherit;
}
```

In this case, the `<blockquote>` element doesn't use the browser default styling. Instead, it inherits style values from its parent `<body>` element: it is a *block* element now (inherited value), its `background-color` is `#F0F0F0` (inherited value), its `font-size` is `small` (inherited value), and its `color` is `blue` (inherited value).

D. `all: unset`

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
</blockquote>
Phasellus eget velit sagittis.
```

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
}
blockquote {
  background-color: skyblue;
  color: red;
  all: unset;
}
```

When the `unset` value is applied to the `all` property in the `blockquote` rule, the `<blockquote>` element doesn't use the browser default styling. Because `background-color` is a non-inherited property and `font-size` and `color` are inherited properties, the `<blockquote>` element is an *inline* element now (initial value), its `background-color` is `transparent` (initial value), but its `font-size` is still `small` (inherited value), and its `color` is `blue` (inherited value).

E. `all: revert`

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
</blockquote>
Phasellus eget velit sagittis.
```

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
}
blockquote {
  background-color: skyblue;
  color: red;
  all: revert;
}
```

When the `all` property is set to `revert` in the `blockquote` rule, the `blockquote` rule is considered to be non-existent and the styling property values are inherited from the ones applied to the parent element `<body>`. So the `<blockquote>` element gets styled as a *block* element, with `background-color` `#F0F0F0`, `font-size` `small`, and `color` `blue` - all values inherited from the `body` rule.

F. `all: revert-layer`

```
<blockquote id="quote">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.
</blockquote>
Phasellus eget velit sagittis.
```

```
body {
  font-size: small;
  background-color: #f0f0f0;
  color: blue;
}
blockquote {
  background-color: skyblue;
  color: red;
  all: revert-layer;
}
```

There are no cascade layers defined in the CSS file, so the `<blockquote>` element inherits its style from the matching `body` rule. The `<blockquote>` element here is styled as a *block* element, with `background-color` `#F0F0F0`, `font-size` `small`, and `color` `blue` - all values inherited from the `body` rule. This scenario is an example of the case when `all` set to `revert-layer` behaves the same as when `all` is set to `revert`.

See also

CSS global keyword values: `initial`, `inherit`, `unset`, `revert`, `revert-layer`

anchor-name CSS property

The **`anchor-name`** CSS property enables defining an element as an **anchor element** by giving it one or more identifying **anchor names**. Each name can then be set as the value of a positioned element's `position-anchor` property to associate it with the anchor.

Syntax

```
/* Single values */
anchor-name: none;
anchor-name: --name;

/* Multiple values */
anchor-name: --name, --another-name;

/* Global values */
anchor-name: inherit;
anchor-name: initial;
anchor-name: revert;
anchor-name: revert-layer;
anchor-name: unset;
```

Values

This property is specified as a single value from the following list:

`none`

> The default value. Setting `anchor-name: none` on an element means that it is not defined as an anchor element. If the element was previously defined as an anchor and associated with a positioned element, setting `anchor-name: none` disassociates the two.

`<dashed-ident>`

> One or more comma-separated arbitrary custom identifiers defining the name or names of the anchor, which can then be referenced in a `position-anchor` property.

Description

To position an element relative to an anchor element, the positioned element requires three features: an association, a position, and a location. The `anchor-name` and `position-anchor` properties provide an explicit association.

The anchor element accepts one or more `<dashed-ident>` anchor names set on it via the `anchor-name` property. When one of those names is then set as the value of the `position-anchor` property of an element that has its `position` set to `absolute` or `fixed`, the two elements are associated. The two elements become tethered by setting a location on the associated element relative to the anchor, making it an "anchor-positioned" element.

If multiple anchor elements have the same anchor name set on them, and that name is referenced by the `position-anchor` property value of a positioned element, that positioned element will be associated with the last anchor element with that anchor name in the source order.

Anchor positioning changes the containing block of anchor-positioned elements, making its `position` relative to its anchor rather than to the nearest positioned ancestor element.

To tether and place a positioned element in a specific location relative to an anchor element, an anchor positioning feature is needed, such as the `anchor()` function (set within an inset property's value) or the `position-area` property.

You cannot associate a positioned element with an anchor element if the anchor is hidden, such as with `display: none` or `visibility: hidden`, or if the anchor is part of the skipped contents of another element due to it having `content-visibility: hidden` set on it.

The `anchor-name` property is supported on all elements that generate a principal box. This means that pseudo-elements, including generated content created using `::before` and `::after`, and UI features like the `range` input thumb (`::-webkit-slider-thumb`) can be anchor elements. Pseudo elements are implicitly anchored to the same element as the pseudo-element's originating element, unless otherwise specified.

For more information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

Formal definition

Initial value	none
Applies to	All elements that generate a principal box
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
anchor-name =
  none            |
  <anchor-name>#

<anchor-name> =
  <dashed-ident>
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example tethers a positioned element to an anchor, positioning the element to the right of the anchor.

HTML

We specify two `<div>` elements; an anchor element with a class of `anchor` and a positioned element with a class of `infobox`.

We also include some filler text around the two `<div>`s to make the `<body>` taller so that it will scroll.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

CSS

We first declare the `anchor` `<div>` as an anchor element by setting an anchor name on it via the `anchor-name` property:

```
body {
  width: 50%;
  margin: 0 auto;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}
```

We associate the second `<div>` with the anchor element by setting its anchor name as the value of the positioned element's `position-anchor` property. We then set the positioned element's:

- `position` property to `fixed`, converting it to an *anchor-positioned element* so it can be positioned relative to the anchor's position on the page.
- `left` and `top` properties to `anchor()` functions with values of `right` and `top` respectively. This positions the infobox's left edge flush to the right edge of its anchor, and its top edge relative to the top edge of its anchor.
- `margin-left` to `10px`, creating space between the anchor positioned element and its anchor.

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position-anchor: --my-anchor;
  position: fixed;
  left: anchor(right);
  top: anchor(top);
  margin-left: 10px;
}
```

Result

Scroll the page to see how the infobox is positioned relative to the anchor. As the anchor scrolls upwards, the positioned element moves along with it.

Multiple positioned elements

This example demonstrates how you can associate multiple positioned elements with one anchor.

HTML

The HTML is the same as for the previous example, except this time we have multiple positioned element `<div>`s with different `id`s to identify them.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox" id="infobox1">
  <p>This is an information box.</p>
</div>
```

```
<div class="infobox" id="infobox2">
  <p>This is another information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

CSS

We declare the `anchor` `<div>` as an anchor element using the `anchor-name` property, giving it an anchor name as before.

```
body {
  width: 50%;
  margin: 0 auto;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}
```

Each of the two positioned elements are associated with the anchor element by setting its anchor name as the positioned element's `position-anchor` property value. Both are also given `fixed` positioning, making them **anchor positioned elements**. The positioned elements are then positioned in different places relative to the anchor using a combination of inset properties as seen above and `align-self` / `justify-self` properties with a value of `anchor-center`, centrally aligning the infobox to the center of the anchor in the inline/block directions respectively.

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position-anchor: --my-anchor;
  position: fixed;
}

#infobox1 {
  left: anchor(right);
  align-self: anchor-center;
  margin-left: 10px;
}

#infobox2 {
  bottom: anchor(top);
  justify-self: anchor-center;
  margin-bottom: 15px;
}
```

Result

Scroll the page to see how both of the infoboxes are tethered to the anchor.

Multiple anchor names

This example demonstrates how an anchor element can have more than one anchor name.

HTML

The HTML is the same as for the previous example.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor">⚓</div>

<div class="infobox" id="infobox1">
  <p>This is an information box.</p>
</div>

<div class="infobox" id="infobox2">
  <p>This is another information box.</p>
</div>
```

```
<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

CSS

The CSS is the same as the previous example too, except we include two comma-separated names in the target's `anchor-name` property value and each positioned element has a different value for `position-anchor`.

```
body {
  width: 50%;
  margin: 0 auto;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.anchor {
  anchor-name: --anchor1, --anchor2;
}

.infobox {
  position: fixed;
}

#infobox1 {
  position-anchor: --anchor1;
  left: anchor(right);
  align-self: anchor-center;
  margin-left: 10px;
}

#infobox2 {
  position-anchor: --anchor2;
  bottom: anchor(top);
  justify-self: anchor-center;
  margin-bottom: 15px;
}
```

Result

Scroll the page to see how both of the infoboxes are tethered to the anchor.

See also

- `position-anchor`
- `anchor-scope`
- HTML `anchor` attribute
- CSS anchor positioning module
- Using CSS anchor positioning guide

anchor-scope CSS property

The **`anchor-scope`** CSS property can be used to limit the scope in which a positioned element can be associated with anchor elements to a particular subtree.

Syntax

```
/* Single values */
anchor-scope: none;
anchor-scope: all;
anchor-scope: --anchor-name;

/* Multiple <dashed-ident> values */
anchor-scope: --anchor-name, --another-name;

/* Global values */
anchor-scope: inherit;
anchor-scope: initial;
anchor-scope: revert;
anchor-scope: revert-layer;
anchor-scope: unset;
```

Values

This property is specified as the keyword `none` or `all` or a comma-separated list of `<dashed-ident>` values:

`none`

No anchor scope limiting happens on an element. This is the default value.

`all`

Sets the scope so that *any* `anchor-name` values set in the subtree can only be bound to by positioned elements in the same subtree.

`<dashed-ident>#`

One or more comma-separated `<dashed-ident>`s representing anchor names. Sets the scope so that the specified `anchor-name` values, when set in the subtree, can only be bound to by positioned elements in the same subtree.

Description

When multiple anchor elements on a page are given the same `anchor-name` value and a positioned element is associated with that anchor name (by specifying the name as its `position-anchor` property value), the positioned element will be associated with the *last* anchor element in the source order with that anchor name.

This can be a problem in certain situations. For example, if a document contains multiple repeated components, each with a positioned element tethered to an anchor, all the positioned elements will be anchored to the last anchor on the page unless each component uses a different anchor name. This is likely not the desired behavior.

The `anchor-scope` property can fix this problem by limiting the visibility, or "scope", of an `anchor-name` value to a specific subtree. The result is that each positioned element can only be anchored to an element within the same subtree of the element that has the scope set on it.

- `anchor-scope: all` sets the scope so that *any* `anchor-name` values set in the subtree can only be bound to by positioned elements in the same subtree. Let's say we include multiple anchors in a document, all with `anchor-name: --my-anchor` set on them, and place them in separate containers. We will then set `anchor-scope: all` on each container. If we then include a positioned element inside one of the containers and give it `--my-anchor` as the value of its `position-anchor` property, it will be positioned relative to the anchor inside the same container.

  Furthermore, if we create another positioned element outside the containers and give it the same anchor name, or a different anchor name, it will not be anchor-positioned relative to any of the anchors, regardless of whether the anchors have those anchor names included inside their `anchor-name` values. `anchor-scope: all` limits the anchor scope for containers it is set on for *any* anchors, regardless of `anchor-name`, to only positioned elements inside the same containers.

- `anchor-scope: <dashed-ident>#` sets the scope so that the specified `anchor-name` values, when set in the subtree, can only be bound to by positioned elements in the same subtree. If we return to the same example described in the previous bullet, but change the `anchor-scope` value set on the containers to `--my-anchor`:

  - Positioned elements with `position-anchor: --my-anchor` set on them will be limited to the scope imposed by the `anchor-scope` setting. They will only be positioned relative to the anchors if they are placed inside the containers.
  - However, positioned elements with different `position-anchor` names — for example, `--another-anchor` — *can* be positioned relative to one of the anchors, whether it is placed inside or outside the containers, provided you add the `--another-anchor` anchor name to the anchor's `anchor-name` property. The `anchor-scope` property only limits the scope for the `--my-anchor` anchor name, so it will have no effect on other anchor names.

    If multiple anchors are given the `--another-anchor` anchor name, the positioned elements with that `position-anchor` value will be positioned relative to the last anchor in the source order with that name.

- `anchor-scope: none` is the default value, which specifies that no anchor scope is set. If multiple anchors exist in a document with the same `anchor-name`, and a positioned element is given this name as the value of its `position-anchor` property, it will be positioned relative to the last anchor element in the source order, regardless of where it is placed in the DOM hierarchy.

If you have, for example, three `anchor-name` values set inside a subtree (say, `--anchor1`, `--anchor2`, and `--anchor3`), setting `anchor-scope: --anchor1, --anchor2, --anchor3` on the top-level element of the subtree would be equivalent to setting `anchor-scope: all`.

Anchor scopes only affect explicit anchor associations, that is, those made between an anchor element with an `anchor-name` set on it, and a positioned element referencing the anchor element's anchor name in its `position-anchor` value. Anchor scopes do not affect implicit anchor associations.

For more information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
anchor-scope =
  none            |
  all             |
  <anchor-name>#

<anchor-name> =
  <dashed-ident>
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates how anchor scope works at a basic level. It shows how an anchor element inside a scoped container can be limited to only having positioned elements inside the same scoped container tethered to it.

HTML

We specify a `<section>` element that we will set an anchor scope on. This contains two `<div>` elements, one to be turned into an anchor, and one to be an anchor-positioned element.

We also include a third `<div>` outside the `<section>`, which we will also turn into an anchor-positioned element.

```
<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 1</div>
</section>
```

```
<div class="positioned">Positioned 2</div>
```

CSS

We first set `anchor-scope: --my-anchor` on the `<section>`. This limits its scope so that anchor element descendants of the `<section>` with a name of `--my-anchor` can only be bound to by positioned elements that are also descendants of the `<section>`.

To test this out, we declare the `anchor` `<div>` as an anchor element by setting an `anchor-name` of `--my-anchor` on it. We then absolutely position the `.positioned` elements, tether them to the anchor by setting their `position-anchor` values to `--my-anchor`, and position them to the right of the anchor by setting their `position-area` values to `right`:

```
.scoped {
  padding: 20px;
  background: #eeeeee;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: blue;
  width: fit-content;
  padding: 3px;
}

.positioned {
  background: orange;
  width: fit-content;
  padding: 3px;
}
```

```
.scoped {
  anchor-scope: --my-anchor;
}

.anchor {
  anchor-name: --my-anchor;
}

.positioned {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: right;
}
```

Result

The example renders like so:

Note how the first positioned element is positioned to the right of the anchor. It is in scope for positioning relative to the `--my-anchor` anchor, as it is inside the `<section>` element where the `anchor-scope: --my-anchor` is set.

However, the second positioned element is not positioned relative to the anchor. It is not a descendant of the `<section>` element, so it is outside the anchor scope.

Comparing different <code>anchor-scope</code> values

This example demonstrates the effects of the different `anchor-scope` values by allowing you to apply different `anchor-scope` values to multiple containers, all containing anchors with the same `anchor-name` value.

HTML

We specify three `<section>` elements that we will set an anchor scope on. Each `<section>` contains two `<div>` elements, one to be turned into an anchor, and one to be an anchor-positioned element.

We also include an additional `<div>` outside the `<section>` elements, which we will also turn into an anchor-positioned element. This will have different anchor positioning settings from the others.

Finally, we include a `<form>` containing three different `<input type="radio">` elements to allow different `anchor-scope` values to be set on the `<section>` elements.

```
<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 1</div>
</section>

<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 2</div>
</section>

<section class="scoped">
  <div class="anchor">⚓</div>
  <div class="positioned">Positioned 3</div>
</section>

<div class="positioned2">Positioned 4</div>

<form>
  <fieldset>
    <legend>Select an <code>anchor-scope</code> value</legend>

    <input type="radio" id="all" name="scope" value="all" checked />
    <label for="all"><code>all</code></label>
    <input type="radio" id="my-anchor" name="scope" value="--my-anchor" />
    <label for="my-anchor"><code>--my-anchor</code></label>
    <input type="radio" id="none" name="scope" value="none" />
    <label for="none"><code>none</code></label>
  </fieldset>
</form>
```

CSS

We start by specifying our anchor elements as anchors by giving them two `anchor-name` values: `--my-anchor` and `--another-anchor`.

```
body {
  display: flex;
  justify-content: center;
  gap: 50px;
  margin-top: 5px;
}

.scoped {
  padding: 20px;
  background: #eeeeee;
  border: 2px solid #dddddd;
  border-radius: 10px;
  width: 100px;
  height: 100px;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: blue;
  width: fit-content;
  padding: 3px;
}

.positioned,
.positioned2 {
  border: 1px solid black;
  border-radius: 3px;
  width: fit-content;
  padding: 3px 6px;
  box-shadow: 3px 3px 3px rgb(0 0 0 / 0.2);
}

form {
  position: absolute;
  bottom: 5px;
  right: 5px;
}
```

```
.anchor {
  anchor-name: --my-anchor, --another-anchor;
}
```

Next, we position our `.positioned` elements relative to an anchor element. We absolutely position them, give them a `position-anchor` value of `--my-anchor` to associate them with an anchor, and position them relative to the anchor with a `position-area` value of `right`.

The `.positioned2` element is positioned in a similar way, except that it is given the other available anchor name as its `position-anchor` value — `--another-anchor` — and it is positioned to the `bottom` of the anchor instead. We also give it a `bottom` value of `5px`, so that if the anchor positioning is not working, it will be positioned to the bottom of the `<body>`. This element is not contained inside any scoped element, therefore it will only be anchor positioned when certain `anchor-scope` values are set on the scoped elements, as explained later on.

```
.positioned {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: right;
}

.positioned2 {
  position: absolute;
  bottom: 5px;
  position-anchor: --another-anchor;
  position-area: bottom;
}
```

We handle setting `anchor-scope` on the `<section>` elements when different radio buttons are pressed using JavaScript, which has been hidden for brevity.

```
const sections = document.querySelectorAll("section");
const form = document.querySelector("form");

function updateScope(val) {
  sections.forEach((section) => (section.style.anchorScope = val));
}

form.addEventListener("input", (e) => {
  updateScope(e.target.value);
});

updateScope("all");
```

Result

The example renders as follows:

Check out the initial positioning effect applied to the positioned elements with `anchor-scope: all` set on the `<section>` elements, and then try selecting the other available `anchor-scope` values to see what their effect is. You should observe the following:

- `all`: The scope for positioning elements relative to anchor elements that are descendants of the `<section>` elements is limited to positioned elements that are themselves descendants of the `<section>` elements, regardless of the `anchor-name` value used to associate them. As a result, the positioned elements inside the `<section>` elements ("Positioned 1–3") are anchor-positioned as expected, but the positioned element outside the `<section>` elements ("Positioned 4") is not anchor-positioned. It is out of scope.
- `--my-anchor`: The scope for positioning elements relative to anchor elements that are descendants of the `<section>` elements is limited to positioned elements that are themselves descendants of the `<section>` elements, only if the `--my-anchor` `anchor-name` is used to associate them. As a result, the positioned elements inside the `<section>` elements ("Positioned 1–3") are anchor-positioned as expected, and the positioned element outside the `<section>` elements ("Positioned 4") is also anchor-positioned as expected. In the former case, the positioned elements are inside the set scope, and in the latter case, the positioned element is not affected by the set scope, as it is using an anchor name outside the scope (`--another-anchor`). The "Positioned 4" element is positioned relative to the last anchor element in the source that has the matching anchor name.
- `none`: As no anchor scope is set on the `<section>` elements, all of the positioned elements are positioned relative to the last anchor element in the source order.

See also

- anchor-name
- position-anchor
- CSS anchor positioning module
- Using CSS anchor positioning guide

animation CSS property

The **animation** CSS shorthand property applies an animation between styles. It is a shorthand for animation-name, animation-duration, animation-timing-function, animation-delay, animation-iteration-count, animation-direction, animation-fill-mode, animation-play-state, and animation-timeline.

Try it

```
animation: 3s ease-in 1s infinite reverse both running slide-in;
```

```
animation: 3s linear 1s infinite running slide-in;
```

```
animation: 3s linear 1s infinite alternate slide-in;
```

```
animation: 0.5s linear 1s infinite alternate slide-in;
```

```
<section class="flex-column" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  background-color: #1766aa;
  margin: 20px;
  border: 5px solid #333333;
  width: 150px;
  height: 150px;
  border-radius: 50%;
}

@keyframes slide-in {
  from {
    margin-left: -20%;
  }
  to {
    margin-left: 100%;
  }
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- animation-name
- animation-duration
- animation-timing-function
- animation-delay
- animation-direction
- animation-iteration-count
- animation-fill-mode
- animation-play-state
- animation-timeline

Reset-only sub-properties

This property resets the following CSS properties to their initial values:

- animation-range-end
- animation-range-start
- animation-delay-end
- animation-composition
- animation-trigger

Syntax

```
/* Duration | easing-function | delay |
iteration-count | direction | fill-mode | play-state | name */
animation: 3s ease-in 1s 2 reverse both paused slide-in;

/* Duration | easing-function | delay | name */
animation: 3s linear 1s slide-in;

/* Duration | name */
animation: 3s slide-in;

/* Multiple animations */
animation:
  3s linear slide-in,
  3s ease-out 5s slide-out;
```

Values

This property is specified as a comma-separated list of `<animation>` values, with each value being a space-separated list of the following values:

`<keyframes-name>` or `none`

The name of a `@keyframes` at-rules that specifies the animation to apply to an element. The initial value for `animation-name` is `none`.

`<animation-duration>`

Determines the length of time that an animation takes to complete one cycle. The value must be one of those available in `animation-duration`. The initial value is `0s`.

`<easing-function>`

Determines the type of transition. The value must be one of those available in `animation-timing-function`. The initial value is `ease`.

`<animation-delay>`

Determines the amount of time to wait from applying the animation to an element before beginning to perform the animation. The value must be one of those available in `animation-delay`. The initial value is `0s`.

`<single-animation-direction>`

The direction in which the animation is played. The value must be one of those available in `animation-direction`. The initial value for `animation-direction` is `normal`.

`<single-animation-iteration-count>`

The number of times the animation is played. The value must be one of those available in `animation-iteration-count`. The initial value for `animation-iteration-count` is `1`.

`<single-animation-fill-mode>`

Determines how styles should be applied to the animation's target before and after its execution. The value must be one of those available in `animation-fill-mode`. The initial value for `animation-fill-mode` is `none`.

`<single-animation-play-state>`

Determines whether the animation is playing or not. The value must be one of those available in `animation-play-state`. The initial value for `animation-play-state` is `running`.

`<single-animation-timeline>`

Determines the timeline used to control the progress of the animation. The value must be one of those available in `animation-timeline`. The initial value is `auto`.

Description

The `animation` property is specified as one or more single animations, separated by commas. Each `animation` within the comma-separated list of animations sets the `animation-name`, `animation-duration`, `animation-timing-function`, `animation-delay`, `animation-iteration-count`, `animation-direction`, `animation-fill-mode`, `animation-play-state`, and `animation-timeline`. If any of the components are not included in an `animation` declaration, the component value is set to the component's initial value.

> **Note:** The `animation` shorthand property resets the `animation-range-start`, `animation-range-end` and `animation-trigger` properties to their initial values; `normal`, `normal` and `none`, respectively. These reset-only sub-properties cannot be set in the `animation` shorthand, and including them invalidates the entire declaration. Because the `animation` declaration sets each of these properties to their initial values, they must either be declared after any `animation` shorthand declarations or with greater specificity.

animation-name

The `<animation-name>` component of each animation is the name for the animation, which may be `none`, a `<custom-ident>`, or a `<string>`. The initial value of `animation-name` is `none`, meaning if no `animation-name` value is declared in the `animation` shorthand property, no animation is applied to any of the properties.

The order of other values within an animation definition is important for distinguishing an `animation-name` value from other values. If a value in the `animation` shorthand can be parsed as a value for an animation property other than `animation-name`, then the value will be applied to that property first and not to `animation-name`. For this reason, the recommended practice is to specify a value for `animation-name` as the last value in a list of values when using the `animation` shorthand; this holds true even when you specify multiple, comma-separated animations using the `animation` shorthand.

Time values

Each animation can include zero, one, or two occurrences of the `<time>` value. The order of time values within each animation definition is important: the first value that can be parsed as a `<time>` is assigned to the `animation-duration`, and the second one is assigned to `animation-delay`.

When no `animation-duration` value is specified in the `animation` shorthand property, the duration defaults to `0s`. In this case, the animation will still occur (the `animationStart` and `animationEnd` events will be fired), but no animation will be visible to the user.

animation-timeline

If no `<animation-timeline>` is included in the `animation` shorthand, the shorthand declaration will reset any previously-declared `animation-timeline` values to `auto`, which sets the timeline to the default `documentTimeline`.

If an `<animation-timeline>` value is included, but the user-agent doesn't support `<animation-timeline>` values within the shorthand, the entire `animation` declaration is invalid and ignored. For this reason, when creating CSS scroll-driven animations, you need to declare the `animation-timeline` property after declaring any `animation` shorthand for it to take effect.

Alternatively, the `<animation-timeline>` can be set within the `animation` shorthand within a CSS `@supports` block, such as:

```
@supports (animation: view()) {
  /* CSS for browsers supporting <animation-timeline> within `animation` shorthand */
}
```

animation-fill-mode and new stacking contexts

In the case of the `animation-fill-mode` forwards value, animated properties behave as if included in a set `will-change` property value. If a new stacking context is created during the animation, the target element retains the stacking context after the animation has finished.

Accessibility

Blinking and flashing animation can be problematic for people with cognitive concerns such as Attention Deficit Hyperactivity Disorder (ADHD). Additionally, certain kinds of motion can be a trigger for vestibular disorders, epilepsy, and migraine and scotopic sensitivity.

Consider providing a mechanism for pausing or disabling animation as well as using the reduced motion `@media` query to create a complementary experience for users who have expressed a preference for reduced animated experiences.

- Designing Safer Web Animation For Motion Sensitivity via A List Apart (2015)
- An Introduction to the Reduced Motion Media Query via CSS-Tricks (2017)
- Responsive Design for Motion via WebKit (2017)
- Understanding WCAG, Guideline 2.2 — Enough Time: Provide users enough time to read and use content
- Understanding WCAG Success Criterion 2.2.2: Pause, Stop, Hide via W3C (2026)

Formal definition

Initial value	as each of the properties of the shorthand: • `animation-name`: `none` • `animation-duration`: `0s`

	• animation-timing-function: ease • animation-delay: 0s • animation-iteration-count: 1 • animation-direction: normal • animation-fill-mode: none • animation-play-state: running • animation-timeline: auto
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • animation-name: as specified • animation-duration: as specified • animation-timing-function: as specified • animation-delay: as specified • animation-direction: as specified • animation-iteration-count: as specified • animation-fill-mode: as specified • animation-play-state: as specified • animation-timeline: a list, each item either a case-sensitive CSS identifier or the keywords none, auto
Animation type	Not animatable

Formal syntax

```
animation =
  <single-animation>#

<single-animation> =
  <'animation-duration'>              ||
  <easing-function>                   ||
  <'animation-delay-start'>           ||
  <single-animation-iteration-count>  ||
  <single-animation-direction>        ||
  <single-animation-fill-mode>        ||
  <single-animation-play-state>       ||
  [ none | <keyframes-name> ]         ||
  <single-animation-timeline>

<animation-duration> =
  [ auto | <time [0s,∞]> ]#

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<animation-delay-start> =
  <time>#

<single-animation-iteration-count> =
  infinite        |
  <number [0,∞]>

<single-animation-direction> =
  normal             |
  reverse            |
  alternate          |
  alternate-reverse

<single-animation-fill-mode> =
  none       |
  forwards   |
  backwards  |
  both

<single-animation-play-state> =
  running  |
  paused

<keyframes-name> =
  <custom-ident>  |
  <string>

<single-animation-timeline> =
  auto            |
  none            |
  <dashed-ident>  |
  <scroll()>      |
  <view()>

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<scroll()> =
  scroll( [ <scroller> || <axis> ]? )

<view()> =
```

```
  view( [ <axis> || <'view-timeline-inset'> ]? )

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<scroller> =
  root     |
  nearest  |
  self

<axis> =
  block   |
  inline  |
  x       |
  y

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Animations Level 1, CSS Animations Level 2, CSS Easing Functions Level 2, CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

> **Note:** Animating CSS box model properties is discouraged as it leads to layout reflow and repaints. Animating any box model property is inherently CPU-intensive; consider animating the transform property instead.

Basic usage: a sunrise

In this example, we demonstrate basic usage of the `animation` shorthand by animating a yellow sun across a light blue sky. The sun rises to the center of the viewport and then falls out of sight.

HTML

We include a single `<div>` element to represent our sun.

```
<div class="sun"></div>
```

CSS

We start by creating the sun and the sky. The sky is the `:root` of the HTML document. We hide any content that is outside the viewport, which in our case will be any part of the sun below the horizon, by setting the `overflow` to hidden. We also use the `justify-content` property to center the sun in the background. We make the sun yellow, declare its `height` to be the height of the viewport (`100vh`), and set its width to equal its height by setting the `aspect-ratio` to `1`. We turn the square `<div>` into a circle using the `border-radius` property.

```
:root {
  overflow: hidden;
  background-color: lightblue;
  display: flex;
  justify-content: center;
}

.sun {
  background-color: yellow;
  border-radius: 50%;
  height: 100vh;
  aspect-ratio: 1;
  animation: 4s linear 0s infinite alternate sunrise;
}
```

Next, we define some animation `@keyframes` that will push the element on which they are applied down past the viewport and then return the element to its default position using CSS transforms:

```
@keyframes sunrise {
  from {
    transform: translateY(110vh);
  }
  to {
    transform: translateY(0);
  }
}
```

The last step is to apply the animation! We use the `animation` shorthand property to apply the `sunrise` keyframe animation to the `.sun` `<div>`. The animation is set to play for infinite iterations, with each one lasting 4 seconds; the animation direction alternates with each iteration:

```
.sun {
  animation: 4s linear 0s infinite alternate sunrise;
}
```

Results

Applying multiple animations

This example demonstrates applying multiple animations to a single element. Expanding on the previous example, with a sun that rises and falls on a light blue background, here we will gradually rotate the sun through a rainbow of colors. The timing of the sun's position and color are independent.

```
<div class="sun"></div>
```

```
:root {
  overflow: hidden;
  background-color: lightblue;
  display: flex;
  justify-content: center;
}

.sun {
  background-color: yellow;
  border-radius: 50%;
  height: 100vh;
  aspect-ratio: 1 / 1;
}

@keyframes sunrise {
  from {
    transform: translateY(110vh);
  }
  to {
    transform: translateY(0);
  }
}
```

We include the same HTML and CSS as in the previous example, and add a second set of animation `@keyframes` to apply a `filter` that rotates the hue through all possible values using the `hue-rotate()` filter function:

```
@keyframes psychedelic {
  from {
    filter: hue-rotate(0deg);
  }
  to {
    filter: hue-rotate(360deg);
  }
}
```

We then apply the two animations to our sun. Multiple animations are separated by commas, and each animation's parameters are set independently:

```
.sun {
  animation:
    4s linear 0s infinite alternate sunrise,
    24s linear 0s infinite psychedelic;
}
```

Results

Cascading multiple animations

This example demonstrates what happens when multiple animations define values for the same property. This example expands upon the basic usage example, with two animations applied that both set a `transform` value.

```
<div class="sun"></div>
```

```
:root {
  overflow: hidden;
  background-color: lightblue;
  display: flex;
  justify-content: center;
}

.sun {
  background-color: yellow;
  border-radius: 50%;
  height: 100vh;
  aspect-ratio: 1 / 1;
}
```

We use the same HTML and CSS as in the first example, including the original `sunrise` animation, and a second animation named `bounce`. The two animations declare values for the same property:

```
@keyframes sunrise {
  from {
    transform: translateY(110vh);
  }
  to {
    transform: translateY(0);
  }
}

@keyframes bounce {
  from {
    transform: translateX(-50vw);
  }
  to {
    transform: translateX(50vw);
  }
}
```

We apply both animations to the sun. When two animations apply different values to the same property, animations declared later in the cascade override previously-declared animations. In this case, the `transform` value on the `bounce` animation "wins" the cascade, and overrides the transform set by `sunrise`, so the sun will only move horizontally.

```
.sun {
  animation:
```

```
    4s linear 0s infinite alternate sunrise,
    4s linear 0s infinite alternate bounce;
}
```

Results

The sun bounces between the left- and right-hand sides of the viewport. The sun remains in the viewport even though the `sunrise` animation is defined. The `sunrise` animation's transform property is overridden by the bounce animation.

See also

- Using CSS animations
- CSS animations module
- CSS scroll-driven animations module
- JavaScript `AnimationEvent` API

animation-composition CSS property

The **`animation-composition`** CSS property specifies the composite operation to use when multiple animations affect the same property simultaneously.

Syntax

```
/* Single animation */
animation-composition: replace;
animation-composition: add;
animation-composition: accumulate;

/* Multiple animations */
animation-composition: replace, add;
animation-composition: add, accumulate;
animation-composition: replace, add, accumulate;

/* Global values */
animation-composition: inherit;
animation-composition: initial;
animation-composition: revert;
animation-composition: revert-layer;
animation-composition: unset;
```

Values

This property is specified as one or more comma-separated keyword values:

`replace`

> The effect value overrides the underlying value of the property. This is the default value.

`add`

> The effect value builds on the underlying value of the property. This operation produces an additive effect. For animation types where the addition operation is not commutative, the order of the operands is the underlying value followed by the effect value.

`accumulate`

> The effect and underlying values are combined. For animation types where the addition operation is not commutative, the order of the operands is the underlying value followed by the effect value.

Description

Each property that is targeted by the @keyframes at-rule is associated with an effect stack. The value of the effect stack is calculated by combining the *underlying value* of a property in a CSS style rule with the *effect value* of that property in the keyframe. The `animation-composition` property helps to specify how to combine the underlying value with the effect value.

For example, in the CSS below, `blur(5px)` is the underlying value, and `blur(10px)` is the effect value. The `animation-composition` property specifies the operation to perform to produce the final effect value after compositing the effect of the underlying value and the effect value.

```
.icon:hover {
  filter: blur(5px);
  animation: 3s infinite pulse;
  animation-composition: add;
}

@keyframes pulse {
  0% {
    filter: blur(10px);
  }
  100% {
    filter: blur(20px);
  }
}
```

Consider different values for the `animation-composition` property in the above example. The final effect value in each of those cases will be calculated as explained below:

- With `replace`, `blur(10px)` will replace `blur(5px)` in the `0%` keyframe. This is the default behavior of the property.
- With `add`, the composite effect value in the `0%` keyframe will be `blur(5px) blur(10px)`.
- With `accumulate`, the composite effect value in `0%` keyframe will be `blur(15px)`.

> **Note:** A composite operation can also be specified in a keyframe. In that case, the specified composite operation is used for each property first within that keyframe and then on each property in the next keyframe.

Multiple values

When you specify multiple comma-separated values on an `animation-*` property, they will be applied to the animations in the order in which the `animation-name`s appear. If the number of animations and compositions differ, the values listed in the `animation-composition` property will cycle from the first to the last `animation-name`, looping until all the animations have an assigned `animation-composition` value. For more information, see Setting multiple animation property values.

Formal definition

Initial value	`replace`

Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-composition =
  <single-animation-composition>#

<single-animation-composition> =
  replace     |
  add         |
  accumulate
```

This syntax reflects the latest standard as per CSS Animations Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Understanding the animation-composition values

The example below shows the effect of different `animation-composition` values side-by-side.

HTML

```
<div class="container">
  replace
  <div id="replace" class="target"></div>
</div>
<div class="container">
  add
  <div id="add" class="target"></div>
</div>
<div class="container">
  accumulate
  <div id="accumulate" class="target"></div>
</div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
  gap: 3px;
}

.container {
  flex: 1;
  min-width: 200px;
  height: 200px;
  background: lightblue;
  display: inline-block;
  text-align: center;
}

.target {
  width: 20px;
  height: 50px;
  background: green;
  border-radius: 10px;
  margin: 20px 0;
}
```

```
@keyframes slide {
  50% {
    transform: translateY(30px);
  }
  100% {
    transform: translateX(150px);
  }
}

.target {
  transform: translateX(30px) rotate(45deg);
  animation: slide 5s linear infinite;
}
#replace {
  animation-composition: replace;
}
#add {
  animation-composition: add;
}
#accumulate {
  animation-composition: accumulate;
}
```

Result

The underlying value for the `transform` property in all cases is `translateX(30px) rotate(45deg)`. The different `animation-composition` values' effects are as follows:

- With `replace`, the `transform` property in each keyframe entirely replaces the underlying `transform` property set on the animated element. The final effect value for the `transform` property at the `50%` keyframe is `translateY(30px)` (no `rotate` or `translateX`); at the `100%` keyframe, it is `translateX(150px)` (no `rotate` or `translateY`).

  The target starts at `transform: translateX(30px) rotate(45deg)` and effectively animates to `transform: translateY(30px)`, then to `transform: translateX(150px)`.

- With `add`, the final effect value at each keyframe is the underlying `transform` value with the effect value placed immediately after it.

  Therefore, the target starts at `transform: translateX(30px) rotate(45deg)` and effectively animates first to `transform: translateX(30px) rotate(45deg) translateY(30px)` (which is `30px` "downwards" on the rotated Y-axis), and then to `transform: translateX(30px) rotate(45deg) translateX(150px)`. Since the additive operation is relative to the underlying `transform` and not the previous keyframe, there

is no `translateY(30px)` at `100%`, placing the element `150px` along the rotated X-axis from the original position.

- With `accumulate`, the final effect value is the keyframe's effect `transform` combined with the underlying original. At `50%`, `translateY(30px)` combines with the original `translateX(30px)` into a single translation (`translate(30px, 30px)`). At `100%`, the `translateX(150px)` combines with the original `translateX(30px)` to create `translateX(180px)`.

  Therefore, the target starts at `transform: translateX(30px) rotate(45deg)` and effectively animates first to `transform: translate(30px, 30px) rotate(45deg)`, and then to `transform: translateX(180px) rotate(45deg)`.

See also

- Using CSS animations
- Composite property of KeyFrameEffect
- Other related animation properties: `animation`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-delay CSS property

The **`animation-delay`** CSS property specifies the amount of time to wait from applying the animation to an element before beginning to perform the animation. The animation can start later, immediately from its beginning, or immediately and partway through the animation.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-delay: 250ms;
```

```
animation-delay: 2s;
```

```
animation-delay: -2s;
```

```
<section class="flex-column" id="default-example">
  <div>Animation <span id="play-status"></span></div>
  <div id="example-element">Select a delay to start!</div>
</section>
```

```
#example-element {
  background-color: #1766aa;
  color: white;
  margin: auto;
  margin-left: 0;
  border: 5px solid #333333;
  width: 150px;
  height: 150px;
  border-radius: 50%;
  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
}

#play-status {
  font-weight: bold;
}

.animating {
  animation-name: slide;
  animation-duration: 3s;
  animation-timing-function: ease-in;
  animation-iteration-count: 2;
  animation-direction: alternate;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

```
const el = document.getElementById("example-element");
const status = document.getElementById("play-status");

function update() {
  status.textContent = "delaying";
  el.className = "";
  window.requestAnimationFrame(() => {
    window.requestAnimationFrame(() => {
      el.className = "animating";
    });
  });
}

el.addEventListener("animationstart", () => {
  status.textContent = "playing";
});

el.addEventListener("animationend", () => {
  status.textContent = "finished";
});

const observer = new MutationObserver(() => {
  update();
});
```

```
observer.observe(el, {
  attributes: true,
  attributeFilter: ["style"],
});

update();
```

Syntax

```
/* Single animation */
animation-delay: 3s;
animation-delay: 0s;
animation-delay: -1500ms;

/* Multiple animations */
animation-delay: 2.1s, 480ms;

/* Global values */
animation-delay: inherit;
animation-delay: initial;
animation-delay: revert;
animation-delay: revert-layer;
animation-delay: unset;
```

Values

This property is specified as a comma-separated list of `<time>` values:

`<time>`

> The time offset, from the moment at which the animation is applied to the element, at which the animation should begin. This may be specified in either seconds (`s`) or milliseconds (`ms`). The unit is required.
>
> A positive value indicates that the animation should begin after the specified amount of time has elapsed. A value of `0s`, which is the default, indicates that the animation should begin as soon as it's applied.
>
> A negative value causes the animation to begin immediately, but partway through its cycle. For example, if you specify `-1s` as the animation delay time, the animation will begin immediately but will start 1 second into the animation sequence. If you specify a negative value for the animation delay, but the starting value is implicit, the starting value is taken from the moment the animation is applied to the element.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

> **Note:** `animation-delay` has no effect on CSS scroll-driven animations.

Formal definition

Initial value	`0s`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-delay =
  [ <'animation-delay-start'> <'animation-delay-end'>? ]#

<animation-delay-start> =
  <time>#

<animation-delay-end> =
  <time [0s,∞]>#
```

This syntax reflects the latest standard as per CSS Animations Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting an animation delay

This animation has a delay of 2 seconds.

HTML

```
<div class="box"></div>
```

CSS

```
.box {
  background-color: rebeccapurple;
  border-radius: 10px;
  width: 100px;
  height: 100px;
}

.box:hover {
  animation-name: rotate;
  animation-duration: 0.7s;
  animation-delay: 2s;
}

@keyframes rotate {
  0% {
    transform: rotate(0);
  }
  100% {
```

```
    transform: rotate(360deg);
  }
}
```

Result

Hover over the rectangle to start the animation.

See CSS animations for examples.

See also

- Using CSS animations
- JavaScript `AnimationEvent` API
- Other related animation properties: `animation`, `animation-composition`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-direction CSS property

The **`animation-direction`** CSS property sets whether an animation should play forward, backward, or alternate back and forth between playing the sequence forward and backward.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-direction: normal;
```

```
animation-direction: reverse;
```

```
animation-direction: alternate;
```

```
animation-direction: alternate-reverse;
```

```
<section class="flex-column" id="default-example">
  <div id="example-element"></div>
  <button id="play-pause">Play</button>
</section>
```

```
#example-element {
  animation-duration: 3s;
  animation-iteration-count: infinite;
  animation-name: slide;
  animation-play-state: paused;
  animation-timing-function: ease-in;
  background-color: #1766aa;
  border-radius: 50%;
  border: 5px solid #333333;
  color: white;
  height: 150px;
  margin: auto;
  margin-left: 0;
  width: 150px;
}

#example-element.running {
  animation-play-state: running;
}

#play-pause {
  font-size: 2rem;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

```
const el = document.getElementById("example-element");
const button = document.getElementById("play-pause");

button.addEventListener("click", () => {
  if (el.classList.contains("running")) {
    el.classList.remove("running");
    button.textContent = "Play";
  } else {
    el.classList.add("running");
    button.textContent = "Pause";
  }
});
```

Syntax

```
/* Single animation */
animation-direction: normal;
animation-direction: reverse;
animation-direction: alternate;
animation-direction: alternate-reverse;

/* Multiple animations */
animation-direction: normal, reverse;
animation-direction: alternate, reverse, normal;
```

```
/* Global values */
animation-direction: inherit;
animation-direction: initial;
animation-direction: revert;
animation-direction: revert-layer;
animation-direction: unset;
```

Values

This property is specified as a comma-separated list of the following keyword values:

`normal`

The animation plays *forwards* each cycle. In other words, each time the animation cycles, the animation will reset to the beginning state and start over again. This is the default value.

`reverse`

The animation plays *backwards* each cycle. In other words, each time the animation cycles, the animation will reset to the end state and start over again. Animation steps are performed backwards, and easing functions are also reversed. For example, an `ease-in` easing function becomes `ease-out`.

`alternate`

The animation reverses direction each cycle, with the first iteration being played *forwards*. The count to determine if a cycle is even or odd starts at one.

`alternate-reverse`

The animation reverses direction each cycle, with the first iteration being played *backwards*. The count to determine if a cycle is even or odd starts at one.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

> **Note:** When creating CSS scroll-driven animations, specifying an `animation-direction` works as expected, for example `reverse` causes the animation to run in reverse over the course of the timeline's progression. A value of `alternate` (combined with an `animation-iteration-count`) causes the animation to run forwards and backwards as the timeline is progressed.

Formal definition

Initial value	`normal`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-direction =
  <single-animation-direction>#

<single-animation-direction> =
  normal             |
  reverse            |
  alternate          |
  alternate-reverse
```

This syntax reflects the latest standard as per CSS Animations Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Reversing the animation direction

HTML

```
<div class="box"></div>
```

CSS

```
.box {
  background-color: rebeccapurple;
  border-radius: 10px;
  width: 100px;
  height: 100px;
}

.box:hover {
  animation-name: rotate;
  animation-duration: 0.7s;
  animation-direction: reverse;
}

@keyframes rotate {
  0% {
    transform: rotate(0);
  }
  100% {
    transform: rotate(360deg);
  }
}
```

Result

See CSS animations for examples.

See also

- Using CSS animations

- JavaScript `AnimationEvent` API
- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-duration CSS property

The **`animation-duration`** CSS property sets the length of time that an animation takes to complete one cycle.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-duration: 750ms;
```

```
animation-duration: 3s;
```

```
animation-duration: 0s;
```

```
<section class="flex-column" id="default-example">
  <div class="animating" id="example-element"></div>
  <button id="play-pause">Play</button>
</section>
```

```
#example-element {
  animation-direction: alternate;
  animation-iteration-count: infinite;
  animation-name: slide;
  animation-play-state: paused;
  animation-timing-function: ease-in;
  background-color: #1766aa;
  border-radius: 50%;
  border: 5px solid #333333;
  color: white;
  height: 150px;
  margin: auto;
  margin-left: 0;
  width: 150px;
}

#example-element.running {
  animation-play-state: running;
}

#play-pause {
  font-size: 2rem;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

```
const el = document.getElementById("example-element");
const button = document.getElementById("play-pause");

button.addEventListener("click", () => {
  if (el.classList.contains("running")) {
    el.classList.remove("running");
    button.textContent = "Play";
  } else {
    el.classList.add("running");
    button.textContent = "Pause";
  }
});
```

Syntax

```
/* Single animation */
animation-duration: auto; /* Default */
animation-duration: 6s;
animation-duration: 120ms;

/* Multiple animations */
animation-duration: 1.64s, 15.22s;
animation-duration: 10s, 35s, 230ms;

/* Global values */
animation-duration: inherit;
animation-duration: initial;
animation-duration: revert;
animation-duration: revert-layer;
animation-duration: unset;
```

Values

This property is specified as a comma-separated list of the following values:

`auto`

For time-based animations, `auto` is equivalent to a value of `0s` (see below). For CSS scroll-driven animations, `auto` fills the entire timeline with the animation.

`<time>`

The time that an animation takes to complete one cycle. This may be specified in either seconds (`s`) or milliseconds (`ms`). The value must be positive or zero and the unit is required.

If no value is provided, the default value of `0s` is used, in which case the animation still executes (the `animationStart` and `animationEnd` events are fired). Whether or not the animation will be visible when the duration is `0s` will depend on the value of `animation-fill-mode`, as explained below:

- If `animation-fill-mode` is set to `backwards` or `both`, the first frame of the animation as defined by `animation-direction` will be displayed during `animation-delay` countdown.
- If `animation-fill-mode` is set to `forwards` or `both`, the last frame of the animation will be displayed, as defined by `animation-direction`, after the `animation-delay` expires.
- If `animation-fill-mode` is set to `none`, the animation will have no visible effect.

> **Note:** Negative values are invalid, causing the declaration to be ignored. Some early, prefixed, implementations may consider them as identical to `0s`.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

> **Note:** When creating CSS scroll-driven animations, specifying an `animation-duration` value in seconds or milliseconds doesn't really make sense. In tests, it seemed to have no effect on scroll progress timeline animations, while on view progress timeline animations it seemed to push the animation to happen nearer the end of the timeline. However, Firefox requires an `animation-duration` to be set for it to successfully apply the animation. You are therefore advised to set `animation-duration` to `1ms` so that animations will work in Firefox, but the effect is not altered too much by it.

Formal definition

Initial value	`0s`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-duration =
  [ auto | <time [0s,∞]> ]#
```

This syntax reflects the latest standard as per CSS Animations Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting animation duration

This animation has an animation-duration of 0.7 seconds.

HTML

```
<div class="box"></div>
```

CSS

```
.box {
  background-color: rebeccapurple;
  border-radius: 10px;
  width: 100px;
  height: 100px;
}

.box:hover {
  animation-name: rotate;
  animation-duration: 0.7s;
}

@keyframes rotate {
  0% {
    transform: rotate(0);
  }
  100% {
    transform: rotate(360deg);
  }
}
```

Result

Hover over the rectangle to start the animation.

See CSS animations for more examples.

See also

- Using CSS animations
- JavaScript `AnimationEvent` API
- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-fill-mode CSS property

The **`animation-fill-mode`** CSS property sets how a CSS animation applies styles to its target before and after its execution.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-fill-mode: none;
animation-delay: 1s;
```

```
animation-fill-mode: forwards;
animation-delay: 1s;
```

```
animation-fill-mode: backwards;
animation-delay: 1s;
```

```
animation-fill-mode: both;
animation-delay: 1s;
```

```
<section class="flex-column" id="default-example">
  <div>Animation <span id="play-status"></span></div>
  <div id="example-element">Select a mode to start!</div>
</section>
```

```
#example-element {
  background-color: #1766aa;
  color: white;
  margin: auto;
  margin-left: 0;
  border: 5px solid #333333;
  width: 150px;
  height: 150px;
  border-radius: 50%;

  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
}

#play-status {
  font-weight: bold;
}

.animating {
  animation: slide 1s ease-in 1;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

```
const el = document.getElementById("example-element");
const status = document.getElementById("play-status");

function update() {
  status.textContent = "delaying";
  el.className = "";
  window.requestAnimationFrame(() => {
    window.requestAnimationFrame(() => {
      el.className = "animating";
    });
  });
}

el.addEventListener("animationstart", () => {
  status.textContent = "playing";
});

el.addEventListener("animationend", () => {
  status.textContent = "finished";
});

const observer = new MutationObserver(() => {
  update();
});

observer.observe(el, {
  attributes: true,
  attributeFilter: ["style"],
});

update();
```

Syntax

```
/* Single animation */
animation-fill-mode: none;
animation-fill-mode: forwards;
animation-fill-mode: backwards;
animation-fill-mode: both;

/* Multiple animations */
animation-fill-mode: none, backwards;
animation-fill-mode: both, forwards, none;

/* Global values */
animation-fill-mode: inherit;
animation-fill-mode: initial;
animation-fill-mode: revert;
```

```
animation-fill-mode: revert-layer;
animation-fill-mode: unset;
```

Values

This property is specified as a comma-separated list of the following keyword values:

`none`

The animation will not apply any styles to the target when it's not executing. The element will instead be displayed using any other CSS rules applied to it. This is the default value.

`forwards`

The target will retain the computed values set by the last keyframe encountered during execution. The last keyframe depends on the value of `animation-direction` and `animation-iteration-count`:

`animation-direction`	`animation-iteration-count`	last keyframe encountered
`normal`	even or odd	`100%` or `to`
`reverse`	even or odd	`0%` or `from`
`alternate`	even	`0%` or `from`
`alternate`	odd	`100%` or `to`
`alternate-reverse`	even	`100%` or `to`
`alternate-reverse`	odd	`0%` or `from`

Animated properties behave as if included in a set `will-change` property value. If a new stacking context was created during the animation, the target element retains the stacking context after the animation has finished.

`backwards`

The animation will apply the values defined in the first relevant keyframe as soon as it is applied to the target, and retain this during the `animation-delay` period. The first relevant keyframe depends on the value of `animation-direction`:

`animation-direction`	first relevant keyframe
`normal` or `alternate`	`0%` or `from`
`reverse` or `alternate-reverse`	`100%` or `to`

`both`

The animation will follow the rules for both forwards and backwards, thus extending the animation properties in both directions.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

> **Note:** `animation-fill-mode` has the same effect when creating CSS scroll-driven animations as it does for regular time-based animations.

Formal definition

Initial value	none
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-fill-mode =
  <single-animation-fill-mode>#

<single-animation-fill-mode> =
  none      |
  forwards  |
  backwards |
  both
```

This syntax reflects the latest standard as per CSS Animations Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting fill mode

You can see the effect of `animation-fill-mode` in the following example. It demonstrates how you can make the animation remain in its final state rather than reverting to the original state (which is the default).

HTML

```
<p>Move your mouse over the gray box!</p>
<div class="demo">
  <div class="grows-and-stays">This grows and stays big.</div>
  <div class="grows">This just grows.</div>
</div>
```

CSS

```
.demo {
  border-top: 100px solid #cccccc;
  height: 300px;
}

@keyframes grow {
  0% {
    font-size: 0;
```

```
  }
  100% {
    font-size: 40px;
  }
}

.demo:hover .grows {
  animation-name: grow;
  animation-duration: 3s;
}

.demo:hover .grows-and-stays {
  animation-name: grow;
  animation-duration: 3s;
  animation-fill-mode: forwards;
}
```

Result

See CSS animations for more examples.

See also

- Using CSS animations
- JavaScript `AnimationEvent` API
- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-iteration-count CSS property

The **`animation-iteration-count`** CSS property sets the number of times an animation sequence should be played before stopping.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-iteration-count: 0;
```

```
animation-iteration-count: 2;
```

```
animation-iteration-count: 1.5;
```

```
<section class="flex-column" id="default-example">
  <div>Animation <span id="play-status"></span></div>
  <div id="example-element">Select a count to start!</div>
</section>
```

```
#example-element {
  align-items: center;
  background-color: #1766aa;
  border-radius: 50%;
  border: 5px solid #333333;
  color: white;
  display: flex;
  flex-direction: column;
  height: 150px;
  justify-content: center;
  margin: auto;
  margin-left: 0;
  width: 150px;
}

#play-status {
  font-weight: bold;
}

.animating {
  animation-name: slide;
  animation-duration: 3s;
  animation-timing-function: ease-in;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

```
const el = document.getElementById("example-element");
const status = document.getElementById("play-status");

function update() {
  status.textContent = "delaying";
  el.className = "";
  window.requestAnimationFrame(() => {
    window.requestAnimationFrame(() => {
      el.className = "animating";
    });
  });
}

el.addEventListener("animationstart", () => {
```

```
  status.textContent = "playing";
});

el.addEventListener("animationend", () => {
  status.textContent = "finished";
});

const observer = new MutationObserver(() => {
  update();
});

observer.observe(el, {
  attributes: true,
  attributeFilter: ["style"],
});

update();
```

Syntax

```
/* Keyword value */
animation-iteration-count: infinite;

/* <number> values */
animation-iteration-count: 3;
animation-iteration-count: 2.4;

/* Multiple values */
animation-iteration-count: 2, 0, infinite;

/* Global values */
animation-iteration-count: inherit;
animation-iteration-count: initial;
animation-iteration-count: revert;
animation-iteration-count: revert-layer;
animation-iteration-count: unset;
```

Values

This property is specified as one or more comma-separated values:

`infinite`

The animation will repeat forever.

`<number>`

The number of times the animation will repeat; this is `1` by default. You may specify non-integer values to play part of an animation cycle: for example, `0.5` will play half of the animation cycle. Negative values are invalid.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

> **Note:** When creating CSS scroll-driven animations, specifying an `animation-iteration-count` causes the animation to repeat that number of times over the course of the timeline's progression. If an `animation-iteration-count` is not provided, the animation will only occur once. `infinite` is a valid value for scroll-driven animations, but it results in an animation that doesn't work.

Formal definition

Initial value	`1`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-iteration-count =
  <single-animation-iteration-count>#

<single-animation-iteration-count> =
  infinite       |
  <number [0,∞]>
```

This syntax reflects the latest standard as per CSS Animations Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting iteration count

This animation will run 10 times.

HTML

```
<div class="box"></div>
```

CSS

```
.box {
  background-color: rebeccapurple;
  border-radius: 10px;
  width: 100px;
  height: 100px;
}

.box:hover {
```

```
  animation-name: rotate;
  animation-duration: 0.7s;
  animation-iteration-count: 10;
}

@keyframes rotate {
  0% {
    transform: rotate(0);
  }
  100% {
    transform: rotate(360deg);
  }
}
```

Result

Hover over the rectangle to start the animation.

See CSS animations for examples.

See also

- Using CSS animations
- JavaScript `AnimationEvent` API
- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-name`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-name CSS property

The **`animation-name`** CSS property specifies the names of one or more `@keyframes` at-rules that describe the animation to apply to an element. Multiple `@keyframes` at-rules are specified as a comma-separated list of names. If the specified name does not match any `@keyframes` at-rule, no properties are animated.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-name: none;
```

```
animation-name: slide;
```

```
animation-name: bounce;
```

```
<section class="flex-column" id="default-example">
  <div class="animating" id="example-element"></div>
</section>
```

```
#example-element {
  animation-direction: alternate;
  animation-duration: 1s;
  animation-iteration-count: infinite;
  animation-timing-function: ease-in;
  background-color: #1766aa;
  border-radius: 50%;
  border: 5px solid #333333;
  color: white;
  height: 150px;
  margin: auto;
  margin-left: 0;
  width: 150px;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}

@keyframes bounce {
  from {
    background-color: orange;
    color: black;
    margin-top: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-top: 40%;
  }
}
```

Syntax

```
/* No animation */
animation-name: none;

/* Single animation */
animation-name: test_05;
animation-name: -specific;
animation-name: "sliding-vertically";

/* Multiple animations */
animation-name: test1, animation4;
```

```
animation-name:
  none,
  -moz-specific,
  sliding;

/* Global values */
animation-name: inherit;
animation-name: initial;
animation-name: revert;
animation-name: revert-layer;
animation-name: unset;
```

Values

This property is specified as one or more comma-separated values:

`none`

A special keyword denoting no keyframes. It can be used to deactivate an animation without changing the ordering of the other identifiers, or to deactivate animations coming from the cascade.

`<custom-ident>`

An unquoted name identifying the animation. This identifier is composed of a combination of case-sensitive letters `a` to `z`, numbers `0` to `9`, underscores (_), and/or dashes (-). The first non-dash character must be a letter. Also, two dashes are forbidden at the beginning of the identifier. Furthermore, the identifier can't be `none`, `unset`, `initial`, or `inherit`.

`<string>`

A series of characters following the same rules as custom identifiers, as described above, except that they are surrounded by either double (") or single (') quotes. If using a quoted string for both the `animation-name` and the corresponding `@keyframes` at-rule name, `none`, global keywords, and names starting with an underscore or double dashes are valid, though not recommended.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-names` appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

Formal definition

Initial value	none
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-name =
  [ none | <keyframes-name> ]#

<keyframes-name> =
  <custom-ident>  |
  <string>
```

This syntax reflects the latest standard as per CSS Animations Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Naming an animation

This animation has an `animation-name` of `rotate`.

HTML

```
<div class="box"></div>
```

CSS

```
.box {
  background-color: rebeccapurple;
  border-radius: 10px;
  width: 100px;
  height: 100px;
}

.box:hover {
  animation-name: rotate;
  animation-duration: 0.7s;
}

@keyframes rotate {
  0% {
    transform: rotate(0);
  }
  100% {
    transform: rotate(360deg);
  }
}
```

Result

Hover over the rectangle to start the animation.

See CSS animations for examples.

See also

- Using CSS animations
- JavaScript `AnimationEvent` API

- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-play-state`, `animation-timeline`, `animation-timing-function`

animation-play-state CSS property

The **`animation-play-state`** CSS property sets whether an animation is running or paused.

Resuming a paused animation will start the animation from where it left off at the time it was paused, rather than starting over from the beginning of the animation sequence.

Try it

```
animation-play-state: paused;
```

```
animation-play-state: running;
```

```
<section class="flex-column" id="default-example">
  <div class="animating" id="example-element"></div>
</section>
```

```
#example-element {
  background-color: #1766aa;
  color: white;
  margin: auto;
  margin-left: 0;
  border: 5px solid #333333;
  width: 150px;
  height: 150px;
  border-radius: 50%;
}

.animating {
  animation-name: slide;
  animation-duration: 3s;
  animation-timing-function: ease-in;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

Syntax

```
/* Single animation */
animation-play-state: running;
animation-play-state: paused;

/* Multiple animations */
animation-play-state: paused, running, running;

/* Global values */
animation-play-state: inherit;
animation-play-state: initial;
animation-play-state: revert;
animation-play-state: revert-layer;
animation-play-state: unset;
```

Values

This property is specified as a comma-separated list of the following keyword values:

`running`

The **animation** is currently **playing**.

`paused`

The **animation** is currently **paused**.

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

Formal definition

Initial value	`running`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-play-state =
  <single-animation-play-state>#
```

```
<single-animation-play-state> =
  running  |
  paused
```

This syntax reflects the latest standard as per CSS Animations Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Pausing an animation

This animation is paused, but runs when you hover over it.

HTML

```
<div class="box"></div>
```

CSS

```
.box {
  background-color: rebeccapurple;
  border-radius: 10px;
  width: 100px;
  height: 100px;
  animation-name: rotate;
  animation-duration: 0.7s;
  animation-iteration-count: infinite;
  animation-play-state: paused;
}

.box:hover {
  animation-play-state: running;
}

@keyframes rotate {
  0% {
    transform: rotate(0);
  }
  100% {
    transform: rotate(360deg);
  }
}
```

Result

Hover over the rectangle to play the animation.

See CSS animations for examples.

See also

- Using CSS animations
- JavaScript `AnimationEvent` API
- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-timeline`, `animation-timing-function`

animation-range CSS property

The **`animation-range`** CSS shorthand property is used to set the start and end of an animation's attachment range along its timeline, i.e., where along the timeline an animation will start and end.

Constituent properties

This property is a shorthand for the following CSS properties:

- `animation-range-start`
- `animation-range-end`

Syntax

```
/* Range start value only*/
/* Single value syntax */
animation-range: normal;
animation-range: 20%;
animation-range: 100px;
animation-range: cover;
animation-range: contain;
/* Two value syntax */
animation-range: cover 20%;
animation-range: contain 100px;

/* Range start and end values */
/* Two value syntax */
animation-range: normal 25%;
animation-range: 25% normal;
animation-range: 25% 50%;
animation-range: entry exit;
/* Three value syntax */
animation-range: cover cover 200px;
animation-range: 10% exit 90%;
animation-range: entry 10% 90%;
/* Four value syntax */
animation-range: cover 0% cover 200px;
animation-range: entry 10% exit 100%;

/* Global values */
animation-range: inherit;
animation-range: initial;
animation-range: revert;
animation-range: revert-layer;
animation-range: unset;
```

Values

The `animation-range` shorthand property is specified as one or more single animation ranges, separated by commas. Each animation range is specified as one to four space separated values composed of `<timeline-range-name>` values, `<length-percentage>` values, and/or the keyword `normal`.

`<animation-range-start>`

> The keyword `normal`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name> <length-percentage>` pair, representing the `animation-range-start`. If a `<timeline-range-name>` is set without a `<length-percentage>`, the `<length-percentage>` defaults to `0%`.

`<animation-range-end>`

> The keyword `normal`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name> <length-percentage>` pair, representing the `animation-range-end`. If a `<timeline-range-name>` is set without a `<length-percentage>`, the `<length-percentage>` defaults to `100%`.

Description

The `animation-range` shorthand property sets the `animation-range-start` and `animation-range-end` values, defining where along the animation timeline the animation will start and end. By default, the styles defined in a keyframe animation are only applied to an element while that element is being animated. When a keyframe animation is applied to an element depends on the animation timeline of that animation. By default, animations are only applied between the timeline's range start and range end. To apply the animation outside of this range, set the `animation-fill-mode` to `backwards`, `forwards`, or `both`. These three `animation-fill-mode` values apply the first keyframe styles until range start, the last keyframe styles after the animation ends, or both before and after, respectively.

The scroll port area known as the view progress visibility range is the area inside which the subject element of a named view progress timeline animation is deemed to be visible. By default, this is the full range of the scrollport, but it can be adjusted using the `view-timeline-inset` property.

If two values are specified as components of the `<animation-range>` property, they will be interpreted in the order `<animation-range-start>` then `<animation-range-end>`. The value of each component is either the keyword `normal`, a `<length-percentage>`, or a `<timeline-range-name>` optionally followed by a `<length-percentage>`. These values are space-separated. Normal is equal to `0%` for start and `100%` for end. Setting `normal` with a `<length-percentage>` for either the start or end range is invalid.

Defining range start and defaulting range end

When you define only the `<animation-range-start>`, either by setting a single `<length-percentage>`, a single `<timeline-range-name>`, or the keyword `normal`, or by specifying a single `<timeline-range-name>` followed by a single `<length-percentage>`, the computed value of the `<animation-range-end>` follows specific rules:

If the value is a single `<length-percentage>` or the keyword `normal`, this value defines the `<animation-range-start>` and the `<animation-range-end>` is implicitly set to `normal`. For example:

- `animation-range: 20%;` is the equivalent to `animation-range-start: 20%; animation-range-end: normal;`
- `animation-range: normal;` is the equivalent to `animation-range-start: normal; animation-range-end: normal;`

If the value is a single `<timeline-range-name>` (without a `<length-percentage>` following it), that timeline range name is applied to both the `<animation-range-start>` and `<animation-range-end>` components, and the ranges of `0%` and `100%`, respectively, are implied. For example:

- `animation-range: contain;` is equivalent to `animation-range-start: contain 0%; animation-range-end: contain 100%;`
- `animation-range: cover;` is equivalent to `animation-range-start: cover 0%; animation-range-end: cover 100%;`

If the value is a single `<timeline-range-name>` with a single `<length-percentage>` following it, in that order, the pair defines the `<animation-range-start>`, and the defined `<timeline-range-name>` is applied to the `<animation-range-end>` at `100%`. For example:

- `animation-range: cover 20%;` is equivalent to `animation-range-start: cover 20%; animation-range-end: cover 100%;`
- `animation-range: contain 100px;` is equivalent to `animation-range-start: contain 100px; animation-range-end: contain 100%;`

Explicitly defining both range start and range end with two values

If two or more values are included in your `animation-range` declaration and the values are anything other than a single `<timeline-range-name>` followed by a `<length-percentage>`, both `<animation-range-start>` and `<animation-range-end>` are explicitly set.

If you include two values and the first value is the keyword `normal` or a `<length-percentage>`, that value defines the `<animation-range-start>`, and the second value defines the `<animation-range-end>`. For example:

- `animation-range: normal 25%;` is equivalent to `animation-range-start: normal; animation-range-end: 25%;`
- `animation-range: 25% 50%;` is equivalent to `animation-range-start: 25%; animation-range-end: 50%;`
- `animation-range: 25% contain;` is equivalent to `animation-range-start: 25%; animation-range-end: contain 100%;`
- `animation-range: 25% normal;` is equivalent to `animation-range-start: 25%; animation-range-end: normal;`

Multiple animations

When specifying ranges for multiple animations, the `animation-range` shorthand property is specified as one or more single animation ranges, separated by commas. Each animation-range is applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and the `animation-range` property values do not match, if there are more `animation-range` values than animations, the extra ranges are ignored. If there are more animations than ranges, the list of `animation-range` values are repeated until there is a corresponding range for each animation. For example, if we set `animation-range: 25% 75%, normal;`, the animation range of all the odd numbered animations `25% 75%` and all the even animations will be `0% 100%`.

Formal definition

Initial value	as each of the properties of the shorthand: • `animation-range-start`: `normal` • `animation-range-end`: `normal`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	as each of the properties of the shorthand: • `animation-range-start`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage • `animation-range-end`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	as each of the properties of the shorthand: • `animation-range-start`: Not animatable • `animation-range-end`: Not animatable

Formal syntax

```
animation-range =
  [ <'animation-range-start'> <'animation-range-end'>? ]#
```

```
<animation-range-start> =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<animation-range-end> =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage of the <code>animation-range</code> property

In this example, we reduce the duration of the view progress scroll animation by using the `animation-range` property to offset the start and end of the animation, and demonstrate the effect of the `animation-fill-mode` property on shortened animation timelines.

HTML

In the middle of a wall of text, we include an element that we will animate. We include a lot of text to ensure that our content overflows its container, but this is hidden for the sake of brevity. We also include a checkbox to toggle the `animation-fill-mode` property on and off, to demonstrate its effect on shortened animation timelines. This is also hidden.

```
<div class="content">
  <h1>Content</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>
```

```
<div class="animatedElement"></div>
```

```
  <p>
    Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
    cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
    dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
    tellus orci ac auctor augue mauris. Risus quis varius quam quisque id diam
    vel quam elementum. Nibh praesent tristique magna sit amet purus gravida
    quis. Duis ultricies lacus sed turpis tincidunt id aliquet. In egestas erat
    imperdiet sed euismod nisi. Eget egestas purus viverra accumsan in nisl nisi
    scelerisque. Netus et malesuada fames ac.
  </p>
  <label>
    <input type="checkbox" /> Add <code>animation-fill-mode: both;</code>
  </label>
</div>
```

CSS

We define an animation that animates an element's opacity, scale, and background color, causing it to fade in, scale up, and change color as the animation progresses. We apply this animation to the `animatedElement` with the `animation` shorthand.

A view progress timeline is created by setting the `view()` function as the value of the `animation-timeline` property on our `animatedElement`. The result is that the element animates as it moves upwards through the document as it is scrolled. We declare `animation-timeline` property after the shorthand, as the shorthand resets this property.

Lastly, an `animation-range` declaration is set to make the animation begin later than expected and finish earlier.

```
.animatedElement {
  background-color: deeppink;
  animation: appear 1ms linear;
  animation-timeline: view();
  animation-range: entry 10% exit -25%;
}

@keyframes appear {
  from {
    background-color: rebeccapurple;
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    background-color: darkturquoise;
    opacity: 0.75;
    transform: scaleX(0.75);
  }
}
```

We also include conditional styling: when the checkbox is checked, the `animation-fill-mode` property gets applied to the animated element:

```
:has(:checked) .animatedElement {
  animation-fill-mode: both;
}
```

The other styles are hidden for the sake of brevity.

```
.animatedElement {
  width: 300px;
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

:has(:checked) .animatedElement {
  animation-fill-mode: both;
}

.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p,
h1 {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}
@supports not (animation-range: normal) {
  body::before {
    content: "Your browser does not support the 'animation-range' property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Scroll to see the element being animated.

Note how the `from`, or `0%`, keyframe property values are not applied to the animated element until the top block border edge is `10%` past the container's bottom edge; it is full size, fully opaque, and magenta. At that point, the animation is applied and it is styled with the values defined by the `0%` keyframe selector. When the `animation-range-end` is reached, 25% from the top of the scrollport, it jumps back to its original styling.

Generally, you will want to set `animation-fill-mode: both` when creating scroll-driven animations. The jump to the default state occurs because we did not set the `animation-fill-mode` property on the element, which can be used to apply an animation's styles to an element before and after the animation's execution. We initially omitted the property in this example to better enable visualizing the effects of `animation-range`.

Check the checkbox to apply the `animation-fill-mode` property to the animated element in then re-scroll: the animation styles should now be continuously applied.

See also

- `animation-timeline`
- `animation-range-end`
- `animation-range-start`
- `scroll-timeline`
- `timeline-scope`
- `view-timeline-inset`
- `animation-fill-mode`
- Understanding timeline range names
- CSS scroll-driven animations module
- View timeline range visualizer

animation-range-end CSS property

The **`animation-range-end`** CSS property sets the point on the timeline where an animation should end.

Syntax

```
/* Keyword or length percentage value */
animation-range-end: normal;
animation-range-end: 80%;
animation-range-end: 700px;

/* Named timeline range value */
animation-range-end: cover;
animation-range-end: contain;
animation-range-end: cover 80%;
animation-range-end: contain 700px;

/* Multiple values */
animation-range-end:
  cover 80%,
  contain 700px;

/* Global values */
animation-range-end: inherit;
animation-range-end: initial;
animation-range-end: revert;
animation-range-end: revert-layer;
animation-range-end: unset;
```

Values

This property is specified as a comma-separated list of values. Each value can be one of the following:

`normal`

Represents the end of the timeline. This is the default value.

`<length-percentage>`

Specifies a length or percentage value measured from the beginning of the timeline.

`<timeline-range-name>`

Specifies a named timeline range within the overall timeline. The range ends at `100%`.

`<timeline-range-name> <length-percentage>`

Specifies a length or percentage value measured from the beginning of the specified named timeline range.

Description

The `animation-range-end` property specifies the end of the animation's attachment range. Changing the end of the attachment range can potentially shift the end of the animation, that is, the point where keyframes mapped to `100%` progress land when the iteration count is `1`, and can also reduce the effective duration of the animation.

The property value can be `normal`, a `<length-percentage>`, or a `<timeline-range-name>` with an optional `<length-percentage>`. If the `<timeline-range-name>` value does not include a `<length-percentage>`, the percentage defaults to `100%`.

When specifying multiple comma-separated values, they apply to the animations in the order in which the `animation-name` values appear. See Setting multiple animation property values.

The `animation-range-end` property is included in the `animation` shorthand as a reset-only value. This means that using the `animation` shorthand resets any previously declared `animation-range-end` value to `normal`; the shorthand cannot be used to set a new `animation-range-end` value. When creating CSS scroll-driven animations, you should declare `animation-range-end` *after* declaring any `animation` shorthand to avoid resetting the value to `normal`.

The `animation-range-end` property, along with the `animation-range-start` property, can also be set by using the `animation-range` shorthand.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
animation-range-end =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a view progress timeline with a range end

In this example, the `animation-range-end` is applied to an element animated via a view progress timeline. This makes the animation reach its last keyframe well before the element reaches the end of it's containing viewport.

HTML

```
<div class="content">
  <h1>Content</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>
</div>
```

In the middle of a long block of text, we've included an element that we'll animate. We've added a lot of text to ensure that the content overflows its container; the extra text is hidden here for brevity.

```
<div class="animatedElement">
```

```
<p>
  Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
  cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
  dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non tellus
  orci ac auctor augue mauris. Risus quis varius quam quisque id diam vel quam
  elementum. Nibh praesent tristique magna sit amet purus gravida quis. Duis
  ultricies lacus sed turpis tincidunt id aliquet. In egestas erat imperdiet sed
  euismod nisi. Eget egestas purus viverra accumsan in nisl nisi scelerisque.
  Netus et malesuada fames ac.
</p>
<p>
```

We've also included a checkbox that will toggle the `animation-fill-mode` property, so you can see how this property affects animations with shortened timelines.

```
<label>
  <input type="checkbox" /> Add <code>animation-fill-mode: forwards;</code>
</label>
```

```
  </p>
</div>
```

CSS

We've defined a view progress timeline by setting a `view()` function as the value of the `animation-timeline` property. This is declared **after** the `animation` shorthand to avoid resetting the longhand property value.

We've also set `animation-range-end` to make the animation end earlier than expected.

```
.animatedElement {
  background-color: deeppink;
  animation: appear 1ms linear;
  animation-timeline: view();
  animation-range-end: exit 25%;
}

@keyframes appear {
  from {
    background-color: rebeccapurple;
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    background-color: darkturquoise;
    opacity: 0.75;
    transform: scaleX(0.75);
  }
}
```

When the checkbox is checked, the `animation-fill-mode` property gets applied to the animated element:

```
:has(:checked) .animatedElement {
  animation-fill-mode: forwards;
}
```

The other styles applied in this example has been hidden here for brevity.

```
.animatedElement {
  width: 300px;
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

:has(:checked) .animatedElement {
  animation-fill-mode: both;
}

.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p,
h1 {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}
@supports not (animation-range-end: normal) {
  body::before {
    content: "Your browser does not support the 'animation-range-end' property.";
    color: black;
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Scroll to see the element animate. Then toggle the checkbox at the end of the block of text and scroll again. Notice how the element finishes animating when it is 75% of the way through the viewport, and how it returns to its default state at that point when the `animation-fill-mode` property is not applied.

See also

- `animation-timeline`
- `animation-range`
- `animation-range-start`
- `view-timeline-inset`
- `Element.animate()` `rangeStart` property

- Scroll-driven animation timelines
- Understanding timeline range names
- CSS scroll-driven animations module
- View progress timeline: Ranges and animation progress visualizer

animation-range-start CSS property

The **`animation-range-start`** CSS property sets the point on the timeline where an animation should start.

Syntax

```
/* Keyword or length percentage value */
animation-range-start: normal;
animation-range-start: 20%;
animation-range-start: 100px;

/* Named timeline range value */
animation-range-start: cover;
animation-range-start: contain;
animation-range-start: cover 20%;
animation-range-start: contain 100px;

/* Multiple values */
animation-range-start:
  cover 20%,
  contain 100px;

/* Global values */
animation-range-start: inherit;
animation-range-start: initial;
animation-range-start: revert;
animation-range-start: revert-layer;
animation-range-start: unset;
```

Values

This property is specified as a comma-separated list of values. Each value can be one of the following:

`normal`

Represents the start of the timeline. This is the default value.

`<length-percentage>`

Specifies a length or percentage value measured from the beginning of the timeline.

`<timeline-range-name>`

Specifies a named timeline range within the overall timeline. The range starts at `0%`.

`<timeline-range-name> <length-percentage>`

Specifies a length or percentage value measured from the beginning of the specified named timeline range.

Description

Allowed values for the `animation-range-start` property are `normal`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name>` followed by a `<length-percentage>`. If the `<timeline-range-name>` value does not include a `<length-percentage>`, the percentage defaults to `0%`.

See `animation-range` for a detailed description of the available values.

When specifying multiple comma-separated values, they apply to the animations in the order in which the `animation-name` values appear. See Setting multiple animation property values.

The `animation-range-start` is included in the `animation` shorthand as a reset-only value. This means that using the `animation` shorthand resets any previously declared `animation-range-start` value of equal or lower specificity to `normal`; the shorthand cannot be used to set a new `animation-range-start` value. When creating CSS scroll-driven animations, you should declare `animation-range-start` *after* declaring any `animation` shorthand to avoid resetting the value to `normal`.

The `animation-range-start`, along with the `animation-range-end` property, can also be set using the `animation-range` shorthand.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
animation-range-start =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a scroll progress timeline with range start

In this example, the `animation-range-start` is applied to an element animated via a scroll progress timeline. This makes the animation start well before the element enters the scrollport.

HTML

```
<div class="content">
  <h1>Content</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>
```

In the middle of a long block of text, we've included an element that we'll animate. We've added a lot of text to ensure that the content overflows its container; the extra text is hidden here for brevity.

```
<div class="animatedElement"></div>
```

```
<p>
  Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
  cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
  dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
  tellus orci ac auctor augue mauris. Risus quis varius quam quisque id diam
  vel quam elementum. Nibh praesent tristique magna sit amet purus gravida
  quis. Duis ultricies lacus sed turpis tincidunt id aliquet. In egestas erat
  imperdiet sed euismod nisi. Eget egestas purus viverra accumsan in nisl nisi
  scelerisque. Netus et malesuada fames ac.
</p>
</div>
```

CSS

A scroll progress timeline is defined using setting a `scroll()` function as the value of the `animation-timeline` property. This is declared **after** the `animation` shorthand to avoid resetting the longhand property value.

We've also set `animation-range-start` to make the animation start earlier than expected.

```
.animatedElement {
  background-color: deeppink;

  animation: appear 1ms linear;

  animation-timeline: scroll();
  animation-range-start: -25%;
}

@keyframes appear {
  from {
    background-color: rebeccapurple;
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    background-color: darkturquoise;
    opacity: 0.75;
    transform: scaleX(0.75);
  }
}
```

Other styles applied in this example have been hidden here for brevity.

```
.animatedElement {
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p,
h1 {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}

@supports not (animation-range-start: normal) {
  body::before {
    content: "Your browser does not support the 'animation-range-start' property.";
    color: black;
    background-color: wheat;
```

```
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Scroll to see the element animate. Notice how the element is already scaled and semi-opaque as it enters the viewport. This is because the element started animating well before it entered the viewport.

See also

- `animation-timeline`
- `animation-range`
- `animation-range-end`
- `view-timeline-inset`
- `Element.animate()` `rangeStart` property
- Scroll-driven animation timelines
- Understanding timeline range names
- CSS scroll-driven animations module
- View progress timeline: Ranges and animation progress visualizer

animation-timeline CSS property

The **`animation-timeline`** CSS property specifies the timeline used to control the progress of a CSS animation.

Syntax

```
/* Keyword values */
animation-timeline: none;
animation-timeline: auto;

/* Named timeline */
animation-timeline: --timeline_name;

/* Anonymous scroll progress timeline */
animation-timeline: scroll();
animation-timeline: scroll(x root);

/* Anonymous view progress timeline */
animation-timeline: view();
animation-timeline: view(inline);
animation-timeline: view(x 200px auto);

/* Multiple values */
animation-timeline: --progress-bar-timeline, --carousel-timeline;
animation-timeline: auto, view(20% 80%), none, scroll(inline nearest);

/* Global values */
animation-timeline: inherit;
animation-timeline: initial;
animation-timeline: revert;
animation-timeline: revert-layer;
animation-timeline: unset;
```

Values

The `animation-timeline` property is specified as one or more comma-separated values, each of which can be one of the following:

`none`

The animation is not associated with a timeline, and no animation occurs.

`auto`

The animation's timeline is the document's default `DocumentTimeline`. This is the default value.

`scroll()`

Defines the root element, nearest scroller, or self as an anonymous scroll progress timeline, and optionally the scroll direction of the scroller.

`view()`

Defines the nearest ancestor scroll container as an anonymous view progress timeline, optionally overriding the default `baseline` axis direction and the `auto` start and ending insets.

`<dashed-ident>`

The name of a scroll-driven or view-progress timeline, as defined by the scroll container's `scroll-timeline-name` or `view-timeline-name` property (or the `scroll-timeline` or `view-timeline` shorthand property).

Description

The default timeline for a CSS keyframe animation is the time-based `DocumentTimeline`. The `animation-timeline` property can be used to set a named or anonymous scroll progress or view progress timeline. Alternatively, it can be used to explicitly set the default time-based document timeline to control the progress of an element's animation or to have no timeline at all, in which case the element does not animate.

The following types of timelines can be set via `animation-timeline`:

`DocumentTimeline`

The default document timeline, which is progressed through by the passing of time since the document was first loaded in the browser. This is the timeline traditionally associated with CSS animations and is selected with a value of `auto`, or by not specifying an `animation-timeline` value at all, as this is the default value.

Scroll progress timeline

The animation is progressed through by scrolling a scrollable element, or *scroller*, horizontally or vertically. The element that provides the scroll progress timeline can be specified in two ways:

Named scroll progress timeline

The scroller is explicitly named by setting the `scroll-timeline-name` property (or the `scroll-timeline` shorthand property) to a `<dashed-ident>`; that `<dashed-ident>` name is then set as the value of the `animation-timeline` property.

Anonymous scroll progress timeline

The `animation-timeline` property of the element to animate is set to the `scroll()` function. The function's two optional parameters define the scroller providing the scroll progress timeline and the scroll axis to be used.

View progress timeline

A keyframe animation is progressed through based on the change in visibility of an element inside a scroller; this element is known as the *subject*. By default, the timeline is at `0%` when the element first becomes visible at one edge of the scroller, and is at `100%` when its end edge exits the scroller's opposite edge. A view progress timeline can be specified in two ways:

Named view progress timeline

The subject is explicitly named by setting the `view-timeline-name` property (or the `view-timeline` shorthand property) to a `<dashed-ident>`. When you set the `animation-timeline` property of the element to animate to that `<dashed-ident>`, the subject's visibility controls the progression of the element's animation. Note that the element to animate does not have to be the same as the subject.

Anonymous view progress timeline

The `animation-timeline` property of the element to animate is set to a `view()` function, causing it to be animated based on its visibility within the scrollport of its nearest parent scroller.

No timeline

All animation timelines can be removed by selecting a value of `none`. When `animation-timeline: none` is set, no animation will occur as there is no timeline to follow.

The `animation-timeline` property is included in the `animation` shorthand as a reset-only value. This means that including `animation` resets a previously-declared `animation-timeline` value to `auto`. As this component of the shorthand is reset-only, a specific value cannot be set via `animation`. When creating CSS scroll-driven animations, you need to declare `animation-timeline` after declaring any `animation` shorthand for it to take effect.

If you specify multiple comma-separated values, each `animation-timeline` value is applied to a single animation in the order in which the `animation-name` values appear. If the number of values in the `animation-timeline` declaration is greater than the number of `animation-name` values, the excess timeline values are ignored. If there are fewer `animation-timeline` values than `animation-name` values, the `animation-timeline` values are repeated, in order, until every `animation-name` has an associated timeline.

If two or more timelines share the same `<dashed-ident>` name and the same specificity, the one last declared within the cascade will be used. If no timeline is found that matches a name included within the `animation-timeline`, the `animation-name` associated with that value is not associated with a timeline.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Computed value	a list, each item either a case-sensitive CSS identifier or the keywords `none`, `auto`
Animation type	Not animatable

Formal syntax

```
animation-timeline =
  <single-animation-timeline>#

<single-animation-timeline> =
  auto            |
  none            |
  <dashed-ident>  |
  <scroll()>      |
  <view()>

<scroll()> =
  scroll( [ <scroller> || <axis> ]? )

<view()> =
  view( [ <axis> || <'view-timeline-inset'> ]? )

<scroller> =
  root     |
  nearest  |
  self

<axis> =
  block   |
  inline  |
  x       |
  y

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Animations Level 2, CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates basic usage of the `animation-timeline` property, along with the `none`, `auto`, and default (`auto`) values.

HTML

We have an `<article>` with three `<section>` children. Each `<section>` has a unique `id` and a `<div>` child.

```
<article>
  <section id="none">
    <div></div>
  </section>
  <section id="auto">
    <div></div>
  </section>
  <section id="default">
    <div></div>
```

```
  </section>
</article>
```

CSS

We use flexible box layout to set the three sections side-by-side. We use generated content to display the `id`. We style all elements the same, applying the `rotate` `@keyframes` animation that rotates the element by 1 full turn. Using the `animation` shorthand, we declare infinite, 2-second, linearly progressing iterations of the `rotate` animation, alternating the direction of each animation.

```
article {
  display: flex;
  gap: 10px;
  text-align: center;
}
section {
  background-color: beige;
  padding: 20px;
}
section::after {
  content: attr(id);
  display: block;
}
div {
  height: 100px;
  width: 100px;
  background-color: magenta;
  animation: rotate 2s infinite alternate linear;
}
@keyframes rotate {
  to {
    rotate: 1turn;
  }
}
```

The only difference is the `animation-timeline` declaration (or lack thereof in the case of `default`) for each `<div>`.

```
#none div {
  animation-timeline: none;
}
#auto div {
  animation-timeline: auto;
}
```

```
@layer no-support {
  @supports not (animation-timeline: none) {
    body::before {
      content: "Your browser doesn't support the `animation-timeline` property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Because the `animation-timeline` property is included in the `animation` shorthand as a reset-only value, the `animation-timeline` must come after the `animation` shorthand, or be applied with greater specificity than the `animation` shorthand, to be applied.

Results

Note that declaring a value of `auto` has the same effect as allowing the `animation-timeline` to default to that value, and that `none` removes all timelines from the element, so no animation in the `none` case.

Setting a named scroll progress timeline

In this example, the animation timeline is set to a horizontal scroll progress timeline.

HTML

Our container includes three stretcher elements which will be wide enough to ensure our container is a scroll container. The middle one contains a shape that we will animate.

```
<div id="container">
  <div class="stretcher"></div>
  <div class="stretcher">
    <div id="shape"></div>
  </div>
  <div class="stretcher"></div>
</div>
```

CSS

We define the container as a flex container, setting a `width` on the container that is half the width of it's combined flex children. Adding an `overflow-x` value of `scroll` sets it to have a horizontal scrollbar.

Our scroll progress timeline, defined using the `scroll-timeline-name` and `scroll-timeline-axis` properties, is named `--square-timeline`. This timeline is applied to our `#shape` element using `animation-timeline: --square-timeline`.

```
#container {
  display: flex;
  width: 300px;
  border: 1px solid;

  overflow-x: scroll;
  scroll-timeline-axis: inline;
  scroll-timeline-name: --square-timeline;
}

.stretcher {
  flex: 0 0 200px;
}
```

The CSS below defines a square that rotates in alternate directions according to the timeline provided by the `animation-timeline` property, which is set to the `--square-timeline` timeline named above. The animation is set two occur twice, in alternating directions, as it passes through the viewport. We added notched corners to make the animation effect more apparent.

```
#shape {
  background-color: deeppink;
  width: 100px;
  height: 100px;
  border-radius: 25px;
  corner-shape: notch;

  animation: rotateAnimation 1ms linear 2 alternate;
  animation-timeline: --square-timeline;
}

@keyframes rotateAnimation {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
```

```
@layer no-support {
  @supports not (animation-timeline: --square-timeline) {
    body::before {
      content: "Your browser doesn't support named animation timelines.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Scroll to see the element being animated.

Setting an anonymous scroll progress timeline

This example expands on the previous one, applying an anonymous scroll progress timeline using the `scroll()` function.

CSS

We include all the CSS from the previous example, only setting the `animation-timeline` property to override the previous example's value. The timeline is provided by the `scroll(inline nearest)` value, which selects the scrollbar in the inline direction of the nearest ancestor element that has scrollbars. This is the vertical scrollbar of the `#container` element, as the `.stretcher` elements don't have overflowing content and therefore aren't scroll containers.

```
#shape {
  animation-timeline: scroll(inline nearest);
}
```

```
@layer no-support {
  @supports not (animation-timeline: scroll()) {
    body::before {
      content: "Your browser doesn't support the scroll() function.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Scroll to see the square element being animated.

Setting a named view progress timeline

In this example, we demonstrate how to create and apply a named view progress timeline. Two elements will be animated, with different elements serving as the scroller.

HTML

```
<main class="scroller">
  <div class="container">
    <h1>Directions</h1>
    <h2>Lincoln Memorial to Martin Luther King, Jr. Memorial</h2>
    <ol>
      <li>Head south on Lincoln Memorial Circle</li>
      <li>Turn right toward Independence Ave</li>
      <li>Turn left onto Independence Ave</li>
      <li>Turn right onto West Basin Dr</li>
      <li>Look up when you reach 64 Independence Ave!</li>
    </ol>
    <section>
```

Our HTML includes a lot of text in a container within a scroller, which we've hidden for brevity. In the middle of the wall of text, we include two `<div>` elements that we will animate based on the visibility of the element itself in the first case and based on the visibility of its parent in the second case:

```
<div class="animated-element self">SELF</div>
<div class="animated-element parent">PARENT</div>
```

```
</section>
<h2>Martin Luther King, Jr. Memorial to Lincoln Memorial</h2>
<ol>
  <li>Head north toward Independence Ave SW</li
  ><li>Turn left onto Independence Ave</li
```

```
  ><li>Turn right toward the Korean War Veterans Memorial</li
  ><li>Take a photo of the memorial as you walk past it.</li
  ><li>Head north on Lincoln Memorial Circle</li
  ><li>Look up. You can't miss it!</li>
</ol>
  </div>
</main>
```

CSS

We create a keyframe animation that changes the opacity and scale of the element, and apply it to both animated elements:

```
@keyframes animation-effect {
  0% {
    opacity: 0;
    scale: 0;
  }
  100% {
    opacity: 1;
    scale: 1;
  }
}

.animated-element {
  animation: animation-effect 1ms linear;
}
```

The `self` element is explicitly named as the scroller for itself by setting the `view-timeline-name` property to a `<dashed-ident>` and also setting that `<dashed-ident>` name as the value of the `animation-timeline` property. In the `parent` case, we set the `container` as the scroller for the animated element:

```
.self {
  view-timeline-name: --self-scroller-element;
  animation-timeline: --self-scroller-element;
}
.container {
  view-timeline-name: --parent-scroller-element;
}
.parent {
  animation-timeline: --parent-scroller-element;
}
```

Additional CSS declarations were hidden for brevity.

```
@layer setup {
  section {
    display: flex;
    gap: 10px;
  }
  main {
    width: 400px;
    padding: 1em;
    height: 300px;
    overflow: scroll;
    border: 1px solid;
    background-color: beige;
    font-family: sans-serif;
  }

  li {
    margin-bottom: 0.75lh;
  }

  .animated-element {
    height: 200px;
    width: calc(100% - 2em);
    margin: auto;
    background-color: forestgreen;
    background-image:
      repeating-linear-gradient(37deg, transparent 0 52px, beige 52px 57px),
      repeating-linear-gradient(94deg, transparent 0 52px, beige 52px 57px);
    border: 1px solid;
    font-size: 3em;
    place-content: center;
  }
}
```

Result

Scroll the container to see both elements animate.

Note how the visibility of the `self` element controls its own animation. In this case, the element is at the `0%` keyframe when the top edge enters the viewport, or visible part of the scrollport, and only reaches the `100%` keyframe when the bottom edge exits the viewport.

The `parent` element only becomes visible when that parent is visible, meaning that when it comes into view, it is already about `25%` of the way through the animation. It is only about `75%` through its animation when it exits the top of the viewport.

See also

- `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timing-function`
- `scroll-timeline-name`, `scroll-timeline-axis`, `scroll-timeline`
- `view-timeline-name`, `view-timeline-axis`, `view-timeline`, `view-timeline-inset`
- `AnimationTimeline`
- Guide: Using CSS animations
- CSS animations module
- CSS scroll-driven animations module

scroll() CSS function

The **`scroll()`** CSS function can be used with the `animation-timeline` property to create an anonymous scroll progress timeline, defining the scroller and axis of the timeline.

Syntax

```
/* No arguments */
animation-timeline: scroll();

/* <scroller> argument only */
animation-timeline: scroll(nearest);
animation-timeline: scroll(root);
animation-timeline: scroll(self);

/*`<axis>` argument only */
animation-timeline: scroll(block);
animation-timeline: scroll(inline);
animation-timeline: scroll(y);
animation-timeline: scroll(x);

/* <scroller> and <axis> */
animation-timeline: scroll(block nearest);
animation-timeline: scroll(inline root);
animation-timeline: scroll(x self);
```

Parameters

`<scroller>`

The scroller element that will provide the scroll progress timeline. Valid values include:

`nearest`

The nearest ancestor of the current element that has scrollbars on either axis. This is the default value.

`root`

The root element of the document.

`self`

The current element itself.

`<axis>`

An `<axis>` keyword value describing the direction, or axis, of the scrollport that controls the scroll-driven animation. The default value is `block`.

Description

The `scroll()` CSS function can be used as a single value within the comma-separated `animation-timeline` property to specify a scroll progress timeline for a `@keyframes` animation. It defines the scrollable element (*scroller*) and scrollbar axis that will provide the animation timeline.

By default, `scroll()` references the `block` axis of the `nearest` ancestor scroll container. The scroller and axis values can be specified in any order.

The following five declarations are equivalent:

```
animation-timeline: scroll();
animation-timeline: scroll(block);
animation-timeline: scroll(nearest);
animation-timeline: scroll(block nearest);
animation-timeline: scroll(nearest block);
```

The scroll progress timeline is progressed through by scrolling the scroller horizontally or vertically, depending on the `<axis>` and writing mode. If the indicated axis does not contain a scrollbar, then the animation timeline will be inactive.

Formal syntax

```
<scroll()> =
  scroll( [ <scroller> || <axis> ]? )

<scroller> =
  root     |
  nearest  |
  self

<axis> =
  block   |
  inline  |
  x       |
  y
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting an anonymous scroll progress timeline

In this example, the `#square` element is animated using an anonymous scroll progress timeline, which is applied to the element to be animated using the `scroll()` function. The timeline in this particular example is provided by the nearest parent element that has (any) scrollbar, from the scrollbar in the block direction.

HTML

The HTML for the example is shown below.

```
<div id="container">
  <div id="square"></div>
  <div id="stretcher"></div>
</div>
```

CSS

The CSS below defines a square that rotates in alternate directions according to the timeline provided by the `animation-timeline` property. In this case, the timeline is provided by `scroll(block nearest)`, which means that it will select the scrollbar in the block direction of the nearest ancestor element that has scrollbars; in this case the vertical scrollbar of the "container" element.

> **Note:** `block` and `nearest` are actually the default parameter values, so we could have used just `scroll()`.

```
#square {
  background-color: deeppink;
  width: 100px;
  height: 100px;
  margin-top: 100px;
  position: absolute;
  bottom: 0;

  animation-name: rotateAnimation;
  animation-duration: 1ms; /* Firefox requires this to apply the animation */
  animation-direction: alternate;
  animation-timeline: scroll(block nearest);
}

@keyframes rotateAnimation {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
```

The CSS for the container sets its height to 300px and we also set the container to create a vertical scrollbar if it overflows. The "stretcher" CSS sets the block height to 600px, which forces the container element to overflow. These two together ensure that the container has a vertical scrollbar, which allows it to be used as the source of the anonymous scroll progress timeline.

```
#container {
  height: 300px;
  overflow-y: scroll;
  position: relative;
}

#stretcher {
  height: 600px;
}
```

```
@layer no-support {
  @supports not (animation-timeline: scroll()) {
    body::before {
      content: "Your browser doesn't support the CSS `scroll()` function.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Scroll to see the square element being animated.

See also

- Using CSS animations
- CSS animations module
- CSS scroll-driven animations module
- `ScrollTimeline`
- Web Animations API

view() CSS function

The **`view()`** CSS function is used with the `animation-timeline` property to create an anonymous view progress timeline based on when an element comes into view inside its nearest scroll container. You can adjust the tracking axis and the optional insets to control when the element is considered "in view".

Syntax

```
/* No parameters */
animation-timeline: view();

/* Axis parameter */
animation-timeline: view(block);
animation-timeline: view(x);

/* Inset parameter */
animation-timeline: view(auto);
animation-timeline: view(20%);
animation-timeline: view(200px);
animation-timeline: view(20% 40%);
animation-timeline: view(20% 200px);
animation-timeline: view(100px 200px);
animation-timeline: view(auto 200px);

/* Axis and inset parameters */
animation-timeline: view(block auto);
animation-timeline: view(inline 20%);
animation-timeline: view(x 200px auto);
```

Parameters

`<axis>`

Specifies the scroll direction used by the view progress timeline. The value can be one of the `<axis>` keywords: `block`, `inline`, `x`, or `y`. The default value is `block`.

`<view-timeline-inset>`

Specifies the inset area that defines when an element is considered "in view". The value can be the keyword `auto` or up to two `<length-percentage>` values.

Description

A view progress timeline progresses based on changes in the visibility of a subject element inside its nearest scroll container. The `view()` function is used with the `animation-timeline` property to create such a view progress timeline.

The function's parameters can specify the scrollbar axis along which timeline progress is tracked and insets that adjust the position of the box in which the subject is considered visible.

- **Axis**: By default, `view()` uses the block axis. You can change this by providing an explicit `<axis>` value. If the chosen axis does not contain a scrollbar, then the animation timeline will be inactive (zero progress).
- **Inset**: By default, the timeline is at `0%` (the `from` keyframe in the `@keyframes` animation) when the subject is first visible at one edge of the scroller, and at `100%` (the `to` keyframe) when the subject's outer border edge reaches the opposite edge of the scroller. You can control these points with the `<view-timeline-inset>` parameters. The animation lasts as long as the element is in the inset-adjusted view. The inset is used to determine whether the element is in view, which in turn determines the length of the animation timeline. The inset consists of up to two values, each of which can be either `auto` or a `<length-percentage>`.
  - The first value defines the start, an inward offset from the scrollport's beginning.
  - The second value, if present, specifies the end, an inward offset from the scrollport's end. If the value is greater than `0`, it specifies an inset (positive). A negative value defines an outset adjustment to the scrollport.

The axis and inset components can be specified in any order. Within the inset component, the first value defines the start inset, and the second value defines the end inset.

Formal syntax

```
<view()> =
  view( [ <axis> || <'view-timeline-inset'> ]? )

<axis> =
  block   |
  inline  |
  x       |
  y

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating an anonymous view progress timeline using <code>view()</code>

In this example, we create an anonymous view progress timeline for the element with the `subject` and `animation` classes using `animation-timeline: view()`. The result is that as you scroll the document, this element animates as it moves upward through the document.

HTML

In the middle of the text, we include the following (note that the HTML includes a lot of content that we've hidden for brevity):

```
<div class="subject-container">
  <div class="subject animation"></div>
</div>
```

```
<div class="content">
  <h1>Content</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>

  <div class="subject-container">
    <div class="subject animation"></div>
  </div>

  <p>
    Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
    cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
    dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
    tellus orci ac auctor augue mauris. Risus quis varius quam quisque id diam
    vel quam elementum. Nibh praesent tristique magna sit amet purus gravida
    quis. Duis ultricies lacus sed turpis tincidunt id aliquet. In egestas erat
    imperdiet sed euismod nisi. Eget egestas purus viverra accumsan in nisl nisi
    scelerisque. Netus et malesuada fames ac.
  </p>
</div>
```

We also include two overlays to enable visualizing the animation range:

```
<div class="overlay top">inset start 50%</div>
<div class="overlay bottom">inset end 10%</div>
```

CSS

The styles for the `subject` and `subject-container` include:

```
.subject {
  width: 300px;
  height: 200px;
  background-color: deeppink;
}
.subject-container {
  border: 2px dashed black;
  width: 300px;
  margin: 0 auto;
}
```

The `subject-container` class shows the bounds of the animation. We define `top` and `bottom` overlays to mark the inset-adjusted scrollport.

```
.overlay {
  position: fixed;
}

.top {
  top: 0;
  height: 50%;
}

.bottom {
  bottom: 0;
  height: 10%;
}
```

The `<div>` element with the `subject` class is also given a class of `animation`. The `grow` animation causes the `subject` element to grow or shrink. The `animation-timeline: view(block 50% 10%)` rule sets the element to be animated as it progresses through the view progress timeline created by its nearest scroll container (in this case, the document's root element).

While scrolling down, note how the inset values `50% 10%` cause the animation to start when the element is 10% from the bottom of the scrollport and to finish when it is 50% from the top. As the animation progresses along the timeline, the `subject` grows. When scrolling up the animation proceeds in the reverse direction, starting at 50% from the top, moving backward through the keyframes, and ending at 10% from the bottom. So, as the animation runs backward, the `subject` shrinks.

An important point to remember is that the animation lasts only as long as the `subject` element is within view, which is defined here by `50% 10%` inset values.

```
.animation {
  animation-timeline: view(block 50% 10%);
  animation-name: grow;
  animation-timing-function: linear;
}

@keyframes grow {
  from {
    transform: scaleX(0);
  }

  to {
    transform: scaleX(1);
  }
}
```

```
.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p {
  font-size: 1.5rem;
  line-height: 1.8;
}

.overlay {
  width: 100%;
  background-color: #f5deb3aa;
  display: flex;
  font-size: 1.2rem;
  font-weight: bold;
  color: red;
  justify-content: flex-end;
}
.top {
  align-items: end;
}

@layer no-support {
  @supports not (animation-timeline: view()) {
    body::after {
      content: "Your browser doesn't support the CSS `view()` function.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;

      position: absolute;
      inset: 0;
      bottom: auto;
    }
  }
}
```

The rest of the CSS is hidden for brevity.

Result

Scroll to see the element with the `subject` class animate as it enters and leaves the adjusted inset view.

See also

- `animation-timeline`
- Scroll-driven animation timelines
- Using CSS animations
- CSS scroll-driven animations module
- CSS animations module
- `ViewTimeline`
- Web Animations API

animation-timing-function CSS property

The **`animation-timing-function`** CSS property sets how an animation progresses through the duration of each cycle.

It is often convenient to use the shorthand property `animation` to set all animation properties at once.

Try it

```
animation-timing-function: linear;
```

```
animation-timing-function: ease-in-out;
```

```
animation-timing-function: steps(5, end);
```

```
animation-timing-function: cubic-bezier(0.1, -0.6, 0.2, 0);
```

```
<section class="flex-column" id="default-example">
  <div class="animating" id="example-element"></div>
  <button id="play-pause">Play</button>
</section>
```

```
#example-element {
  animation-duration: 3s;
  animation-iteration-count: infinite;
  animation-name: slide;
  animation-play-state: paused;
  background-color: #1766aa;
  border-radius: 50%;
  border: 5px solid #333333;
  color: white;
  height: 150px;
  margin: auto;
  margin-left: 0;
  width: 150px;
}

#example-element.running {
  animation-play-state: running;
}

#play-pause {
  font-size: 2rem;
}

@keyframes slide {
  from {
    background-color: orange;
    color: black;
    margin-left: 0;
  }
  to {
    background-color: orange;
    color: black;
    margin-left: 80%;
  }
}
```

```
const el = document.getElementById("example-element");
const button = document.getElementById("play-pause");

button.addEventListener("click", () => {
  if (el.classList.contains("running")) {
    el.classList.remove("running");
    button.textContent = "Play";
  } else {
    el.classList.add("running");
    button.textContent = "Pause";
  }
});
```

Syntax

```
/* Keyword values */
animation-timing-function: ease;
animation-timing-function: ease-in;
animation-timing-function: ease-out;
animation-timing-function: ease-in-out;
animation-timing-function: linear;
animation-timing-function: step-start;
animation-timing-function: step-end;

/* cubic-bezier() function values */
```

```
animation-timing-function: cubic-bezier(0.42, 0, 1, 1); /* ease-in */
animation-timing-function: cubic-bezier(0, 0, 0.58, 1); /* ease-out */
animation-timing-function: cubic-bezier(0.42, 0, 0.58, 1); /* ease-in-out */

/* linear() function values */
animation-timing-function: linear(0, 0.25, 1);
animation-timing-function: linear(0 0%, 0.25 50%, 1 100%);
animation-timing-function: linear(0, 0.25 50% 75%, 1);
animation-timing-function: linear(0, 0.25 50%, 0.25 75%, 1);

/* steps() function values */
animation-timing-function: steps(4, jump-start);
animation-timing-function: steps(10, jump-end);
animation-timing-function: steps(20, jump-none);
animation-timing-function: steps(5, jump-both);
animation-timing-function: steps(6, start);
animation-timing-function: steps(8, end);

/* Multiple animations */
animation-timing-function: ease, step-start, cubic-bezier(0.1, 0.7, 1, 0.1);

/* Global values */
animation-timing-function: inherit;
animation-timing-function: initial;
animation-timing-function: revert;
animation-timing-function: revert-layer;
animation-timing-function: unset;
```

Values

`<easing-function>`

The easing function that corresponds to a given animation, as determined by `animation-name`.

The non-step keyword values (`ease`, `linear`, `ease-in-out`, etc.) each represent cubic Bézier curves with fixed four-point values, while the `cubic-bezier()` function value allows non-predefined values to be specified. The `steps()` easing function divides the input time into a specified number of equal-length intervals. Its parameters include a number of steps and a step position.

`linear`

Equal to `cubic-bezier(0.0, 0.0, 1.0, 1.0)`, animates at an even speed.

`ease`

Equal to `cubic-bezier(0.25, 0.1, 0.25, 1.0)`, the default value, increases in velocity towards the middle of the animation, slowing back down at the end.

`ease-in`

Equal to `cubic-bezier(0.42, 0, 1.0, 1.0)`, starts off slowly, with the speed of the transition of the animating property increasing until complete.

`ease-out`

Equal to `cubic-bezier(0, 0, 0.58, 1.0)`, starts quickly, slowing down the animation continues.

`ease-in-out`

Equal to `cubic-bezier(0.42, 0, 0.58, 1.0)`, with the animating properties slowly transitioning, speeding up, and then slowing down again.

`cubic-bezier(<number [0,1]> , <number> , <number [0,1]> , <number>)`

An author defined cubic-bezier curve, where the first and third values must be in the range of 0 to 1.

`linear(<number> <percentage>{1,2}, …)`

The function interpolates linearly between the provided easing stop points. A stop point is a pair of an output progress and an input percentage. The input percentage is optional and is inferred if not specified. If an input percentage is not provided then the first and last stop points are set to `0%` and `100%` respectively, and the stop points in the middle receive percentage values derived by linearly interpolating between the closest previous and next points that have a percentage value.

`steps(<integer>, <step-position>)`

Displays an animation iteration along *n* stops along the transition, displaying each stop for equal lengths of time. For example, if *n* is 5, there are 5 steps. Whether the animation holds temporarily at 0%, 20%, 40%, 60% and 80%, on the 20%, 40%, 60%, 80% and 100%, or makes 5 stops between the 0% and 100% along the animation, or makes 5 stops including the 0% and 100% marks (on the 0%, 25%, 50%, 75%, and 100%) depends on which of the following step position is used:

`jump-start`

Denotes a left-continuous function, so that the first jump happens when the animation begins.

`jump-end`

Denotes a right-continuous function, so that the last jump happens when the animation ends. This is the default.

`jump-none`

There is no jump on either end, effectively removing a step during the interpolation iteration. Instead, it holds at both the 0% mark and the 100% mark, each for 1/n of the duration.

`jump-both`

Includes pauses at both the 0% and 100% marks, effectively adding a step during the animation iteration.

`start`

Same as `jump-start`.

`end`

Same as `jump-end`.

`step-start`

Equal to `steps(1, jump-start)`

`step-end`

Equal to `steps(1, jump-end)`

> **Note:** When you specify multiple comma-separated values on an `animation-*` property, they are applied to the animations in the order in which the `animation-name`s appear. For situations where the number of animations and `animation-*` property values do not match, see Setting multiple animation property values.

> **Note:** `animation-timing-function` has the same effect when creating CSS scroll-driven animations as it does for regular time-based animations.

Description

Easing functions may be specified on individual keyframes in a `@keyframes` rule. If no **`animation-timing-function`** is specified on a keyframe, the corresponding value of **`animation-timing-function`** from the element to which the animation is applied is used for that keyframe.

Within a keyframe, `animation-timing-function` is an at-rule-specific descriptor, not the property of the same name. The timing is not being animated. Rather, a keyframe's easing function is applied on a property-by-property basis from the keyframe on which it is specified until the next keyframe specifying that property, or until the end of the animation if there is no subsequent keyframe specifying that property. As a result, an **`animation-timing-function`** specified on the **`100%`** or **`to`** keyframe will never be used.

Formal definition

Initial value	ease
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-timing-function =
  <easing-function>#

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

This syntax reflects the latest standard as per CSS Animations Level 1, CSS Easing Functions Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

All the examples in this section animate the `width` and `background-color` properties of several `<div>` elements with different `animation-timing-function` values. The width is being animated from `0` to `100%`, and the background color is being animated from lime to magenta.

Linear function examples

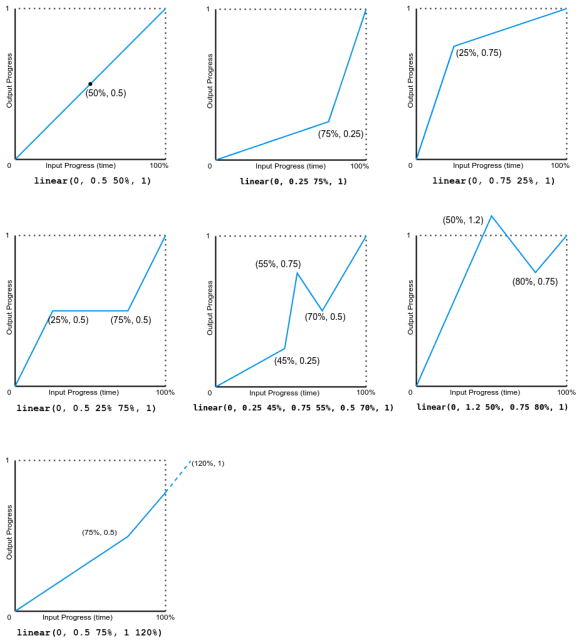
The example demonstrates the effects of various `linear()` easing function values.

```
<div class="parent">
  <div class="linear">'linear' value</div>
  <div class="linear-fn1">linear(0, 0.5 50%, 1)</div>
  <div class="linear-fn2">linear(0, 0.25 75%, 1)</div>
  <div class="linear-fn3">linear(0, 0.75 25%, 1)</div>
  <div class="linear-fn4">linear(0, 0.5 25% 75%, 1)</div>
  <div class="linear-fn5">linear(0, 0.25 45%, 0.75 55%, 0.5 70%, 1)</div>
  <div class="linear-fn6">linear(0, 1.2 50%, 0.75 80%, 1)</div>
  <div class="linear-fn7">linear(0, 0.5 75%, 1 120%)</div>
</div>
<div class="x-axis"><span>25%</span><span>50%</span><span>75%</span></div>
<button>Play animation</button>
```

```
const btn = document.querySelector("button");
const divs = document.querySelectorAll(".parent > div[class]");

btn.addEventListener("click", () => {
  btn.setAttribute("disabled", "true");
  for (const div of divs) {
    div.classList.remove("animate");
    void div.offsetWidth;
    div.classList.add("animate");
  }
  setTimeout(() => {
    btn.removeAttribute("disabled");
  }, 11000);
});
```

```
.x-axis {
  display: flex;
  justify-content: space-evenly;
  width: 80vw;
  margin-left: 4px;
}

.parent {
  background: linear-gradient(
    to right,
    white 24.8%,
    grey 24.8%,
    grey 25.2%,
    white 25.2%,
    white 49.8%,
    grey 49.8%,
    grey 50.2%,
    white 50.2%,
    white 74.8%,
    grey 74.8%,
    grey 75.2%,
    white 75.2%
  );
  width: 80vw;
  font-family: monospace;
  font-weight: bold;
  border: 2px solid grey;
}

.animate {
  animation-name: changeme;
}

.parent > div[class] {
  animation-fill-mode: forwards;
  animation-duration: 10s;

  width: 0;
  margin-bottom: 4px;
  padding: 5px 0;
  box-sizing: border-box;
  text-wrap: nowrap;
  background-color: lime;
}

@keyframes changeme {
  0% {
    width: 0em;
  }
  100% {
    width: 100%;
    background-color: orange;
  }
}

.linear {
  animation-timing-function: linear;
}
.linear-fn1 {
  animation-timing-function: linear(0, 0.5 50%, 1);
}
.linear-fn2 {
  animation-timing-function: linear(0, 0.25 75%, 1);
}
.linear-fn3 {
  animation-timing-function: linear(0, 0.75 25%, 1);
}
.linear-fn4 {
  animation-timing-function: linear(0, 0.5 25% 75%, 1);
}
.linear-fn5 {
  animation-timing-function: linear(0, 0.25 45%, 0.75 55%, 0.5 70%, 1);
}
.linear-fn6 {
  animation-timing-function: linear(0, 1.2 50%, 0.75 80%, 1);
}
.linear-fn7 {
  animation-timing-function: linear(0, 0.5 75%, 1 120%);
}
```

The following image shows graphs of all the `linear()` function values used in this example. Input progress (time) is plotted on the x-axis and output progress is plotted on the y-axis. As per the syntax, the input progress ranges from 0 to 100%, and the output ranges from 0 to 1.



Note that the output can go forward or backward.

Cubic-Bézier examples

The example demonstrates the effects of various bézier curve easing functions.

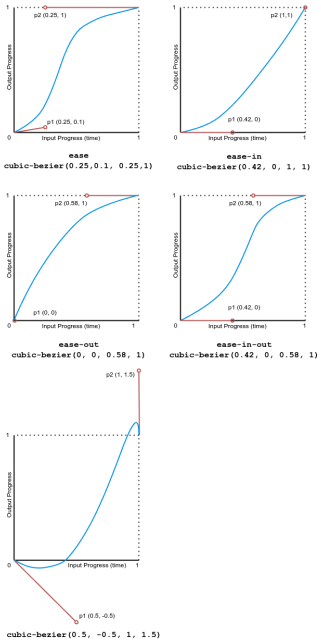
```
<div class="parent">
  <div class="linear">linear</div>
  <div class="ease">ease</div>
  <div class="ease-in">ease-in</div>
  <div class="ease-out">ease-out</div>
  <div class="ease-in-out">ease-in-out</div>
  <div class="cb">cubic-bezier(.5, -0.5, 1, 1.5)</div>
</div>
<div class="x-axis"><span>50%</span></div>
<button>Play animation</button>
```

```
const btn = document.querySelector("button");
const divs = document.querySelectorAll(".parent > div[class]");

btn.addEventListener("click", () => {
  btn.setAttribute("disabled", "true");
  for (const div of divs) {
    div.classList.remove("animate");
    void div.offsetWidth;
    div.classList.add("animate");
  }
  setTimeout(() => {
    btn.removeAttribute("disabled");
  }, 11000);
});
```

```
.x-axis {
  display: flex;
  justify-content: space-evenly;
  width: 80vw;
  margin-left: 4px;
}

.parent {
  background: linear-gradient(
    to right,
    white 49.8%,
    grey 49.8%,
    grey 50.2%,
    white 50.2%
  );
  width: 80vw;
  font-family: monospace;
  font-weight: bold;
  border: 2px solid grey;
}

.animate {
  animation-name: changeme;
}

.parent > div[class] {
  animation-fill-mode: forwards;
  animation-duration: 10s;

  width: 0;
  margin-bottom: 4px;
  padding: 5px 0;
  box-sizing: border-box;
  text-wrap: nowrap;
  background-color: lime;
}
```

```
@keyframes changeme {
  0% {
    width: 0em;
  }
  100% {
    width: 100%;
    background-color: orange;
  }
}

.linear {
  animation-timing-function: linear;
}
.ease {
  animation-timing-function: ease;
}
.ease-in {
  animation-timing-function: ease-in;
}
.ease-out {
  animation-timing-function: ease-out;
}
.ease-in-out {
  animation-timing-function: ease-in-out;
}
.cb {
  animation-timing-function: cubic-bezier(0.5, -0.5, 1, 1.5);
}
```

The following image shows graphs of all the cubic bézier function values used in this example. The input progress (time) ranges from 0 to 1 and the output progress ranges from 0 to 1.



Step examples

This example demonstrates the effects of several step easing function values.

```
<div class="parent">
  <div class="linear">linear</div>
  <div class="start">steps(4, start)</div>
  <div class="jump-start">steps(4, jump-start)</div>
  <div class="end">steps(4, end)</div>
  <div class="jump-end">steps(4, jump-end)</div>
  <div class="jump-both">steps(4, jump-both)</div>
  <div class="jump-none">steps(4, jump-none)</div>
  <div class="step-start">step-start</div>
  <div class="step-end">step-end</div>
</div>
<div class="x-axis"><span>25%</span><span>50%</span><span>75%</span></div>
<button>Play animation</button>
```

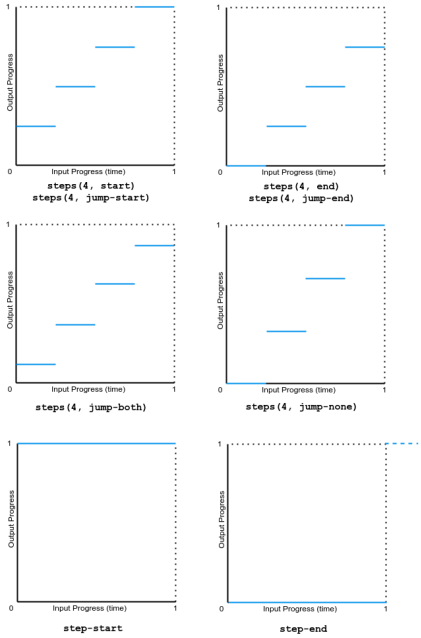
```
const btn = document.querySelector("button");
const divs = document.querySelectorAll(".parent > div[class]");

btn.addEventListener("click", () => {
  btn.setAttribute("disabled", "true");
  for (const div of divs) {
    div.classList.remove("animate");
    void div.offsetWidth;
    div.classList.add("animate");
  }
  setTimeout(() => {
    btn.removeAttribute("disabled");
  }, 11000);
});
```

```
.x-axis {
  display: flex;
  justify-content: space-evenly;
```

```
  width: 80vw;
  margin-left: 4px;
}

.parent {
  background: linear-gradient(
    to right,
    white 24.8%,
    grey 24.8%,
    grey 25.2%,
    white 25.2%,
    white 49.8%,
    grey 49.8%,
    grey 50.2%,
    white 50.2%,
    white 74.8%,
    grey 74.8%,
    grey 75.2%,
    white 75.2%
  );
  width: 80vw;
  font-family: monospace;
  font-weight: bold;
  border: 2px solid grey;
}

.animate {
  animation-name: changeme;
}

.parent > div[class] {
  animation-fill-mode: forwards;
  animation-duration: 10s;

  width: 0;
  margin-bottom: 4px;
  padding: 5px 0;
  box-sizing: border-box;
  text-wrap: nowrap;
  background-color: lime;
}

@keyframes changeme {
  0% {
    width: 0em;
  }
  100% {
    width: 100%;
    background-color: orange;
  }
}
```

```
.linear {
  animation-timing-function: linear;
}

.start {
  animation-timing-function: steps(4, start);
}
.jump-start {
  animation-timing-function: steps(4, jump-start);
}
.end {
  animation-timing-function: steps(4, end);
}
.jump-end {
  animation-timing-function: steps(4, jump-end);
}
.jump-both {
  animation-timing-function: steps(4, jump-both);
}
.jump-none {
  animation-timing-function: steps(4, jump-none);
}
.step-start {
  animation-timing-function: step-start;
}
.step-end {
  animation-timing-function: step-end;
}
```

The following image shows graphs of all the `step()` function values used in this example. The input progress (time) and output progress range from 0 to 1.



See also

- Using CSS animations
- `<easing-function>`
- CSS easing functions module
- JavaScript `AnimationEvent` API
- Cubic bézier generation tool
- Other related animation properties: `animation`, `animation-composition`, `animation-delay`, `animation-direction`, `animation-duration`, `animation-fill-mode`, `animation-iteration-count`, `animation-name`, `animation-play-state`, `animation-timeline`

animation-trigger CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`animation-trigger`** CSS property specifies whether CSS animations declared on an element are triggered animations (or not) and, if so, what their triggers are and how they should behave when the trigger becomes active or inactive. This is used to create scroll-triggered animations.

Syntax

```
/* Keywords */
animation-trigger: none;

/* One trigger */
animation-trigger: --my-trigger play;
animation-trigger: --my-other-trigger play-once;
animation-trigger: --my-trigger play-forwards play-backwards;
animation-trigger: --my-other-trigger play reset;

/* Multiple values */
animation-trigger:
  none,
  -forwards play-backwards,
  --my-other-trigger play reset;

/* Global values */
animation-trigger: inherit;
animation-trigger: initial;
animation-trigger: revert;
animation-trigger: revert-layer;
animation-trigger: unset;
```

Values

Specified as a comma-separated list of values. Each value is either the keyword `none` or a `<dashed-ident>` followed by one or two `<animation-action>` values.

`none`

The associated animation is not a triggered animation.

`<dashed-ident>`

A custom identifier defining the name of the trigger that will trigger the animation.

`<animation-action>`

An `<animation-action>`: The keywords `none`, `play`, `play-forwards`, `play-backwards`, `play-once`, `pause`, `replay`, or `reset`.

Description

The `animation-trigger` property specifies which trigger will control an animated element's animations. A value other than `none` turns the animation into a scroll-triggered animation.

Defining a trigger

The trigger is identified via a `<dashed-ident>` value, which is defined in the tracked element's `timeline-trigger-name` property.

For example:

```
.animated {
  animation: rotate 3s infinite linear both;
  animation-trigger: --my-trigger play;
}
```

In this case, the animation will play when an element with a `timeline-trigger-name` of `--my-trigger` enters the activation range defined on the trigger.

Here we create a trigger by setting the `timeline-trigger-name` using the `timeline-trigger` shorthand property. The `.trigger` can be any element, including the `.animated` element.

```
.trigger {
  timeline-trigger: --my-trigger view();
}
```

If an element has an animation *and* an `animation-trigger` set on it, but no scrolling element exists with the same `<dashed-ident>` set as its `timeline-trigger-name` value, the animation will not have a trigger and therefore will never play.

Defining the triggered animation actions

The `animation-trigger` value must include one or two `<animation-action>` keywords after the `<dashed-ident>` to specify the animation behavior when the trigger is activated and deactivated. If two `<animation-action>`s are specified, the first is the activation action and the second is the deactivation action. If only one `<animation-action>` is set, this is the activation action, and nothing happens on deactivation.

For example:

```
.animated {
  animation: rotate 3s infinite linear both;
  animation-trigger: --my-trigger play-forwards play-backwards;
}
```

When the trigger is activated, the animation will `play-forwards`. When the trigger is deactivated, the animation will `play-backwards`.

There are eight `<animation-action>` values, each providing different animation behaviors.

Setting `play-forwards play-backwards` is a common pattern, often used to "animate in" an element when its trigger becomes active, for example by scrolling it into view, and then "animate out" the element again when the trigger becomes inactive, for example by scrolling it out of view.

The `play-once` action is generally used on its own or as part of `play-once pause`; setting `play-once` as the activation action causes the animation to play only once when it scrolls into view. Adding `pause` on deactivation pauses the animation when the trigger scrolls out of its active range; if the trigger is re-activated, the animation will restart from where it was paused.

See the `<animation-action>` data type for examples and more about each keyword value.

Triggering one animation via multiple triggers

If you want to define triggers on multiple elements that all trigger the same animation on an element, you need to specify the animation multiple times on that animated element, giving each `animation` instance a different trigger.

For example:

```
.animated {
  animation:
    spinOnce 2s 1 ease-out,
    spinOnce 2s 1 ease-out;
  animation-trigger:
    --t1 play-forwards play-backwards,
    --t2 play-forwards play-backwards;
}

.trigger1 {
  timeline-trigger: --t1 view();
}

.trigger2 {
  timeline-trigger: --t2 view();
}
```

See Multiple triggers for the same animation for a working example.

Reset by the <code>animation</code> shorthand

The `animation-trigger` property is a reset-only sub-property of the `animation` shorthand property. This means trigger names and animation actions can not be included in the `animation` shorthand, but setting the `animation` shorthand resets `animation-trigger` to its initial value of `none`. For this reason, you should always set `animation-trigger` after a corresponding `animation` property in a declaration list or declare the `animation-trigger` in a declaration block with selectors with stronger `specificity`.

Multiple <code>animation-trigger</code> values

Setting multiple `animation-trigger` values works in the same way as setting multiple values for the `animation` shorthand property and the other animation longhand properties:

- If multiple `animation-name` values are set, but only a single `animation-trigger` value is set, the `animation-trigger` will apply to all the animations.
- If two or more comma-separated `animation-trigger` values are set, they will cycle between the animations until all of them have an `animation-trigger` value set. See an example of declaring multiple scroll-triggered animations.

Given the following CSS:

```
.animated {
  animation:
    fade-in linear 1s forwards,
    rotate infinite 5s both,
    shrink ease-in 3s forwards,
    color-change steps(5) 5s forwards;

  animation-trigger:
    --t1 play pause,
    --t2 forwards backwards;
}
.trigger1 {
  timeline-trigger: --t1 view();
}
.trigger2 {
```

```
  timeline-trigger: --t2 view();
}
```

With `animation-trigger: --t1 play pause, --t2 forwards backwards` set on the animated element, `--t1` triggers the `fade-in` and `shrink` animations while `--t2` triggers the `rotate` and `color-change` animations.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
animation-trigger =
  [ none | [ <dashed-ident> <animation-action>+ ]+ ]#
```

This syntax reflects the latest standard as per Animation Triggers. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates how to create a scroll-triggered animation that plays when activated and pauses when deactivated.

HTML

The markup contains two `<div>` elements, one to animate and one to create a trigger, plus some basic text content to make the page scrollable. The text is hidden for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport to allow us to see the animation play and pause.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` for the `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` element. Without an associated trigger, the animation would start when the page loads. The `animation-trigger` property makes it a triggered animation. The value references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause` — which specify that the animation will play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear both;
  animation-trigger: --t play pause;
}
```

We set a `timeline-trigger-name` of `--t` on the `.trigger` element. As this value is the identifier referenced in the `.animated` declaration block's `animation-trigger` property value, this associates the two together, creating the animated element's trigger. We also include a `timeline-trigger-source` value of `view()`, which sets the timeline trigger as a view progress timeline and both the active and activation ranges as the full `cover` timeline. We could have declared both together as `timeline-trigger: view() --t`.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
```

Result

Try scrolling the content up and down. When any part of the `.trigger` element appears in the scrollport, the animation plays; when it has completely left the scrollport at either edge, the animation pauses.

Making the animated element create the trigger

In this example, we demonstrate how an animated element can also create its own trigger.

HTML

This time, the markup contains only a single `<div>` element, plus basic text content to cause the page to scroll. We have hidden the content markup for brevity.

```
<div>I create my own trigger</div>
```

```
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
```

```
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div>I create my own trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

We create an `@keyframes` block that inverts the background and foreground colors:

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  background: orange;
  border: 5px solid black;
}
```

```
@keyframes invert-colors {
  from {
    background: orange;
    color: black;
  }

  to {
    background: black;
    color: orange;
  }
}
```

We set the `animation` on the `<div>` element, inverting the colors smoothly over 600ms. We also set an `animation-trigger` value that references a `timeline-trigger-name` of `--t` and include two `<animation-action>` values — `play-forwards` and `play-backwards` — which specify that the animation will play forwards on activation and play in reverse on deactivation.

We also specify a `timeline-trigger` value of `--t view() contain` on the `<div>`, so it creates the trigger for its own animation. The `timeline-trigger` shorthand includes three longhand property values:

- A `timeline-trigger-name` value: A `<dashed-ident>` identifier referenced in the `animation-trigger` property.
- A `timeline-trigger-source` value: The `view()` value sets the timeline trigger to a view progress timeline tracking the element inside its nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` value: The `<timeline-range-name>` value of `contain` sets the trigger to activate when the `<div>` is fully inside the scrollport. Because we didn't set a value for the `timeline-trigger-active-range` component, the active range is the same as the activation range, so it deactivates when it starts to exit the scrollport.

```
div {
  animation: invert-colors 0.6s ease-in both;
  animation-trigger: --t play-forwards play-backwards;
  timeline-trigger: --t view() contain;
}
```

Result

Try scrolling the content up. When the `<div>` is fully in the scrollport, its animation plays. When any part leaves the scrollport at either edge, the animation plays backward.

Multiple triggers for the same animation

This example shows how to assign multiple triggers to control the same animation. This example is similar to the basic usage example, but with multiple triggers triggering the same keyframe animation.

HTML

We include three `<div>` elements as triggers. The text content is hidden for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger1">I create a trigger</div>

...
```

```
<div class="trigger2">I create another trigger</div>

...

<div class="trigger3">I create yet another trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger1">I create a trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger2">I create another trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger3">I create yet another trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger1,
.trigger2,
.trigger3 {
  background: palegoldenrod;
}
```

The `animation` shorthand property value is a comma-separated list of animations, applying the same `animation-name` — `rotate` — three times. The `animation-trigger` value is a comma-separated list with three animation triggers, one for each animation instance.

```
.animated {
  animation:
    rotate 3s infinite linear both,
    rotate 3s infinite linear forwards,
    rotate 3s infinite linear forwards;
  animation-trigger:
    --t1 play-forwards play-backwards,
    --t2 play-forwards play-backwards,
    --t3 play-forwards play-backwards;
}
```

We define a timeline trigger with a different name on each trigger `<div>` element. These names correspond to the names referenced in the `.animated` element's `animation-trigger` property.

```
.trigger1 {
  timeline-trigger: --t1 view();
}

.trigger2 {
  timeline-trigger: --t2 view();
}

.trigger3 {
  timeline-trigger: --t3 view();
}
```

```
@supports not (animation-trigger: --t play-forwards play-backwards) {
  body::before {
    content: "Your browser does not support the animation-trigger property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up and down, and note how the animation activates and then deactivates when each trigger element scrolls into and out of view.

See also

- `<animation-action>` type
- `timeline-trigger-name`, `timeline-trigger-source`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand
- `trigger-scope`
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module
- CSS scroll-driven animations module

appearance CSS property

The **`appearance`** CSS property specifies the rendered appearance of replaced UI widget elements such as form controls. Most commonly, such elements are given native, platform-specific styling based on the operating system's theme, or a primitive appearance with styles that can be overridden using CSS.

Try it

```
appearance: auto;
```

```
appearance: none;
```

```
appearance: textfield;
```

```
<section id="default-example">
  <div class="background" id="example-element">
    <input type="search" value="search" aria-label="unlabeled search" />
    <input type="checkbox" aria-label="unlabeled checkbox" />
    <input type="radio" aria-label="unlabeled radio button" />
    <button>Button</button>
  </div>
</section>
```

```
input,
button {
  appearance: inherit;
}
```

Syntax

```
/* CSS Basic User Interface Module Level 4 values */
appearance: none;
appearance: auto;
appearance: menulist-button;
appearance: textfield;
appearance: base-select;

/* Global values */
appearance: inherit;
appearance: initial;
appearance: revert;
appearance: revert-layer;
appearance: unset;

/* <compat-auto> values have the same effect as 'auto' */
```

```
appearance: button;
appearance: checkbox;
```

Values

The `appearance` property can be applied to all elements and pseudo-elements, but the effect of the specified value, if any, depends on the element to which it is applied.

`none`

Gives the widget a *primitive* appearance, making it stylable via CSS, while maintaining the widget's native functionality. This value does not affect non-widgets.

`auto`

Sets interactive widgets to render with their *OS-native* appearance. Behaves as `none` on elements with no OS-native styling.

`base-select`

Relevant only to the `<select>` element and `::picker(select)` pseudo-element, allowing them to be fully styled.

`<compat-special>`

Has an effect similar to `auto` on certain elements.

`textfield`

Causes the appearance of certain `<input>` types to match the appearance of the `text` type.

`menulist-button`

When set on the `<select>` element, the drop-down picker's style matches that of its default state.

`<compat-auto>`

Included for backwards compatibility; possible values include `button`, `checkbox`, `listbox`, `menulist`, `meter`, `progress-bar`, `push-button`, `radio`, `searchfield`, `slider-horizontal`, `square-button`, and `textarea`. The values all behave as `auto`: use `auto` instead.

> **Note:** The specification also defines a `base` value. This is not yet supported by any browser.

Non-standard values

Some non-standard values are also supported in some browsers:

`slider-vertical`

Makes the slider vertical when applied to `<input type="range">` elements. To create a vertical slider you should instead set the `writing-mode` to `vertical-lr` and the `direction` to `rtl`.

`-apple-pay-button`

Displays the Apple Pay logo when set on a `<button>`, `<a>`, or `<input>` element of type `button` or `reset`.

Description

The `appearance` property enables displaying elements in their OS-native style based on the operating system's theme, as well as the removal of any platform-native styling with the `none` value. Setting `appearance: none`, or otherwise changing the appearance of UI widgets, does not change the element's functionality.

While most elements in a document can be fully styled by CSS, UI controls (*widgets*) are typically rendered by the browser using the operating system's native UI styles. This *native* appearance differs between operating systems and browsers. In this default state, widgets have limited, if any, features that can be styled with CSS. Which elements have this native UI appearance is defined in HTML.

The `appearance` property provides some control over the appearance of HTML widgets that, by default, look like native operating system controls. Most notably, the `none` value suppresses some of a widget's native appearance. This results in a *primitive* look that can be styled via CSS while still maintaining functionality and supporting native user interactions.

Some widgets disappear completely when set to `appearance: none`. The hidden controls are still interactive, however. For example, clicking on a `<label>` associated with an `appearance: none` checkbox will toggle the checkbox's checked state.

Because `none` can cause a widget to be hidden, the `base` value is being added to provide widgets with a base appearance. When supported, the `base` value will ensure widgets maintain their native appearance while enabling CSS to be used to change a widget's styles that are not changeable by default. Unlike `none`, which can make radio buttons and checkboxes disappear, `base` gives the widget a primitive appearance with default native styles that are usable and interoperable, as well as enabling a good degree of customization via CSS. While this `base` value is not yet supported, the many `<compat-auto>` values provide similar functionality but are type-specific and not global.

Customizable select elements

The `base-select` value, which is relevant only to the `<select>` element and `::picker(select)` pseudo-element, enables styling `<select>` elements and the select picker (which contains the `<option>` elements). The picker is rendered in the top layer, similar to a popover. When `base-select` is set, the picker can be positioned relative to the select (or other elements) using CSS anchor positioning features. In addition, the `base-select` value causes the `<select>` not to render outside the browser pane or to trigger built-in mobile operating system components. It is also no longer sized based on the width of the widest `<option>`.

See Customizable select elements for more information.

Prefixed non-standard values

Before standardization, the prefixed **`-moz-appearance`** and **`-webkit-appearance`** properties allowed elements to be shown as widgets such as buttons or checkboxes. The following non-standard values may be encountered in legacy style sheets, most commonly as values of shadow DOM component prefixed pseudo-elements.

Authors are encouraged to use only standard keywords.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
appearance =
  none              |
  auto              |
```

```
  base              |
  base-select       |
  <compat-auto>     |
  <compat-special>

<compat-auto> =
  searchfield   |
  textarea      |
  checkbox      |
  radio         |
  menulist      |
  listbox       |
  meter         |
  progress-bar  |
  button

<compat-special> =
  textfield        |
  menulist-button
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

This example demonstrates basic usage of the `appearance` property, altering the appearance of an `<input>` element in some browsers.

HTML

We include two `number` form controls along with their labels.

```
<p>
  <label>Enter a number: <input type="number" min="0" max="10" /></label>
</p>
<p>
  <label
    >Enter a number: <input type="number" min="0" max="10" class="text"
  /></label>
</p>
```

CSS

We set the element with the class of `text` to look like a text field.

```
.text {
  appearance: textfield;
}
```

Results

Depending on the browser, the spinner may be visually removed when the control is set to look like a text field. The `appearance` property has no effect on the functionality: for example, while there may no longer be a spinner to click on, the up and down cursor keys will still increment and decrement the value.

Appearance set to <code>none</code>

The following example shows how to remove the default styling from a checkbox, a radio button, and a `<select>` element, and apply custom styling.

HTML

We include pairs of checkboxes, radio buttons, and `<select>` elements, along with their associated labels:

```
<label><input type="checkbox" /> Default unchecked </label>
<label><input type="checkbox" checked /> Default checked </label>

<hr />

<label><input type="radio" name="radio" /> Default unchecked </label>
<label><input type="radio" name="radio" checked /> Default checked </label>

<hr />

<label
  >Unstyled select
  <select>
    <option>Option 1</option>
    <option>Option 2</option>
  </select>
</label>

<label
  >Styled select
  <select class="none">
    <option>Option 1</option>
    <option>Option 2</option>
  </select>
</label>
```

CSS

```
label {
  display: block;
  margin: 0.5em 0;
}
```

We apply styles to both `<input>` elements of type `checkbox`; these styles create a red square if the element is stylable. We set `appearance: none` on the `:checked` UI state for all inputs (`checkbox` and `radio`), as well as to elements with the `.none` class. This removes all the style of the radio button and checkbox, other than the margins, and allows any set styles to be applied. There are no alternative styles provided for the radio buttons or `<select>` elements for when `none` is set.

```
[type="checkbox"] {
  width: 1em;
```

```
  height: 1em;
  display: inline-block;
  background: red;
}
input:checked,
.none {
  appearance: none;
}
```

Result

Setting `appearance: none` enables UI elements to be styled, but it also runs the risk of hiding the widget. The unchecked checkbox, with its `appearance` defaulting to `auto`, looks like a checkbox. Setting `appearance: none` in the `:checked` state enables it to be styled.

Like the unchecked checkbox, the unchecked radio button looks like the native UI widget, because it is. When in the checked state, with `appearance: none` applied, the radio button disappears; its functionality is retained, and only its margins affect page rendering.

Setting the appearance of a select

We can use the `appearance` property to opt-in to custom select functionality, enabling styling the `<select>` element and its picker, which represents the part of the form control that pops out of the page.

HTML

We include three `<select>` elements, with the same multiple `<option>` children. As with every `<select>`, we also include associated `<label>` elements. The third option has more text to demonstrate the effect of `base-select` on the width of the `<select>`:

```
<label for="ice-cream1"
  >Default flavor:
  <select id="ice-cream1">
    <option>Asparagus</option>
    <option>Dulce de leche</option>
    <option>Pistachio, rum raisin, and coffee</option>
  </select>
</label>
<label for="ice-cream2"
  >Base select flavor:
  <select id="ice-cream2" class="baseSelect">
    <option>Asparagus</option>
    <option>Dulce de leche</option>
    <option>Pistachio, rum raisin, and coffee</option>
  </select>
</label>
<label for="ice-cream3"
  >Menulist button flavor:
  <select id="ice-cream3" class="menulistButton">
    <option>Asparagus</option>
    <option>Dulce de leche</option>
    <option>Pistachio, rum raisin, and coffee</option>
  </select>
</label>
```

CSS

We select the pickers of all the `<select>` elements using the `::picker()` pseudo-element with the `select` parameter. We set the `appearance` value of all the pickers and one `<select>` element to `base-select`. We set the last `<select>` to `menulist-button`. The first `<select>` defaults to the `auto` state:

```
.baseSelect,
::picker(select) {
  appearance: base-select;
}
.menulistButton {
  appearance: menulist-button;
}
```

```
label {
  display: block;
}
```

We set values for the `<select>` and pickers' `background-color` and `border` properties to demonstrate the effects of the `appearance` values:

```
select {
  border: 1px solid red;
  background-color: orange;
}

::picker(select) {
  background-color: yellow;
  border: none;
}
```

Results

While the `background-color` and `border` styles are defined on all the `<select>` elements and their pickers, the `::picker(select)` styles only affect the picker where both the select and the picker have the `appearance` property set to `base-select`. The first and third selects look the same because `menulist-button` is a compatibility keyword.

Notice that, by default, the inline-size of the `<select>` is generally the inline-size of the `<option>` with the most text, and that the drop-down picker appears over the top of the rendered page when opened, making it not constrained by the surrounding page and therefore fully visible. These statements are no longer true when `base-select` is set.

See also

- `prefers-color-scheme`

aspect-ratio CSS property

The **`aspect-ratio`** CSS property allows you to define the desired width-to-height ratio of an element's box.

Try it

```
aspect-ratio: auto;
```

```
aspect-ratio: 1 / 1;
```

```
aspect-ratio: 16 / 9;
```

```
aspect-ratio: 0.5;
```

```
<section id="default-example">
  <img
    class="transition-all"
    height="640"
    id="example-element"
    src="/shared-assets/images/examples/plumeria.jpg"
    width="466" />
</section>
```

```
#example-element {
  height: 100%;
  width: auto;
}
```

Syntax

```
aspect-ratio: 1 / 1;
aspect-ratio: 1;

/* fallback to 'auto' for replaced elements */
aspect-ratio: auto 3/4;
aspect-ratio: 9/6 auto;

/* Global values */
aspect-ratio: inherit;
aspect-ratio: initial;
aspect-ratio: revert;
aspect-ratio: revert-layer;
aspect-ratio: unset;
```

Values

This property is specified as one or both of the keyword `auto` or a `<ratio>`.

`auto`

> Replaced elements with an intrinsic aspect ratio use *that* aspect ratio, otherwise the box has no preferred aspect ratio. Size calculations involving intrinsic aspect ratio always work with the content box dimensions.

`<ratio>`

> The box's preferred aspect ratio is the specified ratio of `width` / `height`. If `height` and the preceding slash character are omitted, `height` defaults to `1`. Size calculations involving preferred aspect ratio work with the dimensions of the box specified by `box-sizing`.

`auto && <ratio>`

> When both `auto` and a `<ratio>` are specified together, `auto` is used if the element is a replaced element with a natural aspect ratio, like an `<img>` element. Otherwise, the specified ratio of `width` / `height` is used as the preferred aspect ratio.

Description

The `aspect-ratio` property defines a desired width-to-height ratio of an element's box. This means that even if the parent container or viewport size changes, the browser will adjust the element's dimensions to maintain the specified width-to-height ratio. The specified aspect ratio is used in the calculation of auto sizes and some other layout functions.

At least one of the box's sizes needs to be automatic in order for `aspect-ratio` to have any effect. If neither the width nor height is an automatic size, then the provided aspect ratio has no effect on the box's preferred sizes.

This property is specified as one or both of the keyword auto or a `<ratio>`. If both are given, and the element is a replaced element, such as `<img>`, then the given ratio is used until the content is loaded. After the content is loaded, the `auto` value is applied, so the intrinsic aspect ratio of the loaded content is used.

If the element is not a replaced element, then the given `ratio` is used.

Formal definition

Initial value	`auto`
Applies to	all elements except inline boxes and internal ruby or table boxes
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
aspect-ratio =
  auto     ||
  <ratio>

<ratio> =
  <number [0,∞]> [ / <number [0,∞]> ]?
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Exploring aspect-ratio effects with fixed width

In this example, the width of the `<div>` elements has been set to `100px` and height to `auto`. Since the width value is fixed here, the `aspect-ratio` property affects only the height of the `<div>` elements to maintain the specified width-to-height ratio.

```
<div>1/1</div>
<div>0.5</div>
<div>1</div>
<div>1/0.5</div>
<div>16/9</div>
```

```
div {
  display: inline-flex;
  background-color: lime;
  justify-content: center;
}
```

```
div {
  width: 100px;
  height: auto;
}
div:nth-child(1) {
  aspect-ratio: 1/1;
}
div:nth-child(2) {
  aspect-ratio: 0.5;
}
div:nth-child(3) {
  aspect-ratio: 1;
}
div:nth-child(4) {
  aspect-ratio: 1/0.5;
}
div:nth-child(5) {
  aspect-ratio: 16/9;
}
```

Fallback to natural aspect ratio

In this example we are using two `<img>` elements. The first element does not have its `src` attribute set to an image file.

```
<img src="" /> <img src="plumeria.jpg" />
```

The following code sets `3/2` as the preferred aspect ratio and `auto` as a fallback.

```
img {
  display: inline;
  width: 200px;
  border: 2px dashed red;
  background-color: lime;
  vertical-align: top;

  aspect-ratio: 3/2 auto;
}
```

Note how the first image without replaced content keeps the `3/2` aspect ratio, while the second image after the content is loaded uses the image's natural aspect ratio.

See also

- Understanding aspect ratios
- Image aspect-ratio: preventing jank
- Designing an aspect ratio unit for CSS
- Setting Height And Width On Images Is Important Again

backdrop-filter CSS property

The **`backdrop-filter`** CSS property lets you apply graphical effects such as blurring or color shifting to the area behind an element. Because it applies to everything *behind* the element, to see the effect the element or its background needs to be transparent or partially transparent.

Try it

```
backdrop-filter: blur(10px);
```

```
backdrop-filter: invert(80%);
```

```
backdrop-filter: sepia(90%);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div id="example-element">Example</div>
  </div>
</section>
```

```
.example-container {
  background-image: url("/shared-assets/images/examples/balloon.jpg");
  background-size: cover;
  width: 200px;
  height: 200px;
  display: flex;
  align-items: center;
  justify-content: center;
  color: black;
}

#example-element {
```

```
  font-weight: bold;
  flex: 1;
  text-align: center;
  padding: 20px 10px;
  background-color: rgb(255 255 255 / 0.2);
}
```

Syntax

```
/* Keyword value */
backdrop-filter: none;

/* URL to SVG filter */
backdrop-filter: url("common-filters.svg#filter");

/* <filter-function> values */
backdrop-filter: blur(2px);
backdrop-filter: brightness(60%);
backdrop-filter: contrast(40%);
backdrop-filter: drop-shadow(4px 4px 10px blue);
backdrop-filter: grayscale(30%);
backdrop-filter: hue-rotate(120deg);
backdrop-filter: invert(70%);
backdrop-filter: opacity(20%);
backdrop-filter: sepia(90%);
backdrop-filter: saturate(80%);

/* Multiple filters */
backdrop-filter: url("filters.svg#filter") blur(4px) saturate(150%);

/* Global values */
backdrop-filter: inherit;
backdrop-filter: initial;
backdrop-filter: revert;
backdrop-filter: revert-layer;
backdrop-filter: unset;
```

Values

`none`

No filter is applied to the backdrop.

`<filter-value-list>`

A space-separated list of `<filter-function>`s or an SVG filter that will be applied to the backdrop. CSS `<filter-function>`s include `blur()`, `brightness()`, `contrast()`, `drop-shadow()`, `grayscale()`, `hue-rotate()`, `invert()`, `opacity()`, `saturate()`, and `sepia()`.

Description

The `backdrop-filter` property applies filter effects to the pixels painted *behind* an element, up to the nearest ancestor that is a **backdrop root**. Content above the backdrop root is not affected.

Backdrop root

A backdrop root is an element that establishes a boundary for `backdrop-filter` effects. The following elements are backdrop roots:

- The root element (`<html>`)
- An element with a `filter` value other than `none`
- An element with an `opacity` value less than `1`
- An element with a `mask`, `mask-image`, `mask-border`, or `clip-path` value other than `none`
- An element with a `backdrop-filter` value other than `none`
- An element with a `mix-blend-mode` value other than `normal`
- An element with `will-change` set to any of the above properties

This means that if a parent element has `opacity: 0.9`, it becomes a backdrop root and any child's `backdrop-filter` will only blur the content between that parent and the child - not the content behind the parent. This is a common source of confusion when `backdrop-filter` appears to have no visible effect despite being correctly applied.

The following example demonstrates how backdrop roots affect `backdrop-filter`. The first container has `will-change: opacity`, making it a backdrop root - notice that the blur circle only affects the text and square inside the container, not the checkered background behind it. The second container is not a backdrop root, so its blur circle affects everything behind it, including the page background.

```
<div class="parent backdrop-root">
  <div class="text">Text</div>
  <div class="square"></div>
  <div class="overlay"></div>
</div>
<div class="parent">
  <div class="text">Text</div>
  <div class="square"></div>
  <div class="overlay"></div>
</div>
```

```
body {
  display: flex;
  column-gap: 16px;
  padding: 16px;
  background-image: conic-gradient(
    gray 90deg,
    silver 90deg 180deg,
    gray 180deg 270deg,
    silver 270deg
  );
  background-size: 32px 32px;
}

.parent {
  position: relative;
  width: 256px;
  height: 256px;
}
```

```
.backdrop-root {
  outline: 2px solid crimson;
  will-change: opacity;
}

.square {
  position: absolute;
  top: 35px;
  left: 40%;
  width: 25%;
  height: 25%;
  border: 10px solid white;
}

.text {
  position: absolute;
  left: 40%;
  color: white;
  font-size: 32px;
  font-weight: 500;
  text-align: center;
  line-height: 256px;
  filter: blur(1px);
}

.overlay {
  position: absolute;
  top: 25%;
  left: 50%;
  width: 50%;
  height: 50%;
  outline: 3px solid gainsboro;
  border-radius: 9999px;
  backdrop-filter: blur(10px);
}
```

Formal definition

Initial value	none
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	a filter function list

Formal syntax

```
backdrop-filter =
  none                |
  <filter-value-list>

<filter-value-list> =
  [ <filter-function> | <url> ]+

<filter-function> =
  <blur()>         |
  <brightness()>   |
  <contrast()>     |
  <drop-shadow()>  |
  <grayscale()>    |
  <hue-rotate()>   |
  <invert()>       |
  <opacity()>      |
  <sepia()>        |
  <saturate()>

<blur()> =
  blur( <length>? )

<brightness()> =
  brightness( [ <number> | <percentage> ]? )

<contrast()> =
  contrast( [ <number> | <percentage> ]? )

<drop-shadow()> =
  drop-shadow( [ <color>? && <length>{2,3} ] )

<grayscale()> =
  grayscale( [ <number> | <percentage> ]? )

<hue-rotate()> =
  hue-rotate( [ <angle> | <zero> ]? )

<invert()> =
  invert( [ <number> | <percentage> ]? )

<opacity()> =
  opacity( [ <number> | <percentage> ]? )

<sepia()> =
  sepia( [ <number> | <percentage> ]? )

<saturate()> =
  saturate( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1, Filter Effects Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

CSS

```
.box {
  background-color: rgb(255 255 255 / 30%);
  backdrop-filter: blur(10px);
}

body {
  background-image: url("anemones.jpg");
}
```

```
html,
body {
  height: 100%;
  width: 100%;
}

.container {
  background-size: cover;
  align-items: center;
  display: flex;
  justify-content: center;
  height: 100%;
  width: 100%;
}
.box {
  border-radius: 5px;
  font-family: sans-serif;
  text-align: center;
  max-width: 50%;
  max-height: 50%;
  padding: 20px 40px;
}
```

HTML

```
<div class="container">
  <div class="box">
    <p>backdrop-filter: blur(10px)</p>
  </div>
</div>
```

Result

See also

- `filter`
- `<filter-function>`
- `background-blend-mode`, `mix-blend-mode`
- CSS filter effects
- CSS compositing and blending

backface-visibility CSS property

The **`backface-visibility`** CSS property sets whether the back face of an element is visible when turned towards the user.

An element's back face is a mirror image of its front face. Though invisible in 2D, the back face can become visible when a transformation causes the element to be rotated in 3D space. (This property has no effect on 2D transforms, which have no perspective.)

Try it

```
backface-visibility: visible;
```

```
backface-visibility: hidden;
```

```
<section class="default-example" id="default-example">
  <div id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  perspective-origin: 220% 220%;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  background: rgb(0 0 0 / 0.4);
  font-size: 60px;
  color: white;
```

```
}

.front {
  transform: translateZ(50px);
}

.back {
  background: rgb(230 0 0);
  color: white;
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(0 0 0 / 0.6);
  transform: rotateY(90deg) translateZ(50px);
}

.bottom {
  background: rgb(0 0 0 / 0.6);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Syntax

```
/* Keyword values */
backface-visibility: visible;
backface-visibility: hidden;

/* Global values */
backface-visibility: inherit;
backface-visibility: initial;
backface-visibility: revert;
backface-visibility: revert-layer;
backface-visibility: unset;
```

Values

This property is specified as one of the following keywords:

`visible`

The back face is visible when turned towards the user.

`hidden`

The back face is hidden, effectively making the element invisible when turned away from the user.

Formal definition

Initial value	`visible`
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
backface-visibility =
  visible  |
  hidden
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Cube with transparent and opaque faces

This example shows a cube with transparent faces, and one with opaque faces.

HTML

```
<table>
  <thead>
    <tr>
      <th><code>backface-visibility: visible;</code></th>
      <th><code>backface-visibility: hidden;</code></th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <div class="container">
          <div class="cube show-bf">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
        <p>
          Since all faces are partially transparent, the back faces (2, 4, 5)
          are visible through the front faces (1, 3, 6).
        </p>
      </td>
      <td>
        <div class="container">
          <div class="cube hide-bf">
            <div class="face front">1</div>
```

```
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
        <p>The three back faces (2, 4, 5) are hidden.</p>
      </td>
    </tr>
  </tbody>
</table>
```

CSS

```
/* Classes that will show or hide the
   three back faces of the "cube" */
.show-bf div {
  backface-visibility: visible;
}

.hide-bf div {
  backface-visibility: hidden;
}

/* Define the container div, the cube div, and a generic face */
.container {
  width: 150px;
  height: 150px;
  margin: 75px 0 0 75px;
  border: none;
}

.cube {
  width: 100%;
  height: 100%;
  perspective: 550px;
  perspective-origin: 150% 150%;
  transform-style: preserve-3d;
}

.face {
  display: block;
  position: absolute;
  width: 100px;
  height: 100px;
  border: none;
  line-height: 100px;
  font-family: sans-serif;
  font-size: 60px;
  color: white;
  text-align: center;
}

/* Define each face based on direction */
.front {
  background: rgb(0 0 0 / 30%);
  transform: translateZ(50px);
}

.back {
  background: lime;
  color: black;
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(196 0 0 / 70%);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 196 / 70%);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(196 196 0 / 70%);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(196 0 196 / 70%);
  transform: rotateX(-90deg) translateZ(50px);
}

/* Make the table a little nicer */
th,
p,
td {
  background-color: #eeeeee;
  margin: 0px;
  padding: 6px;
  font-family: sans-serif;
  text-align: left;
}
```

Result

See also

- Using CSS transforms

background CSS property

The **background** CSS shorthand property sets all background style properties at once, such as color, image, origin, size, and repeat method.

Try it

```
background: green;
```

```
background: content-box radial-gradient(crimson, skyblue);
```

```
background: no-repeat url("/shared-assets/images/examples/lizard.png");
```

```
background: left 5% / 15% 60% repeat-x
  url("/shared-assets/images/examples/star.png");
```

```
background:
  center / contain no-repeat
    url("/shared-assets/images/examples/firefox-logo.svg"),
  #eeeeee 35% url("/shared-assets/images/examples/lizard.png");
```

```
<section id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- background-attachment
- background-clip
- background-color
- background-image
- background-origin
- background-position
- background-repeat
- background-size

Reset-only sub-properties

This property resets the following CSS properties to their initial values:

- background-blend-mode

Syntax

```
/* Using a <background-color> */
background: green;

/* Using a <bg-image> and <repeat-style> */
background: url("test.jpg") repeat-y;

/* Using a <visual-box> and <'background-color'> */
background: border-box red;

/* A single image, centered and scaled */
background: no-repeat center/80% url("../img/image.png");

/* Global values */
background: inherit;
background: initial;
background: revert;
background: revert-layer;
background: unset;
```

Values

`<attachment>`

See background-attachment. Default: scroll.

`<visual-box>`

See background-clip and background-origin. Default: border-box and padding-box respectively.

`<'background-color'>`

See background-color. Default: transparent.

`<bg-image>`

See background-image. Default: none.

`<bg-position>`

See background-position. Default: 0% 0%.

`<repeat-style>`

See background-repeat. Default: repeat.

`<bg-size>`

See background-size. Default: auto.

Description

The background shorthand property enables you to declare all CSS background properties in a single declaration. The background lies underneath the content of an element. When you have multiple, comma-separated background values, each is a background layer that is painted on top of the previous layers.

The background property is specified as one or more background layers, separated by commas. Each layer may include zero, one, or two <visual-box> components and zero or one <attachment>, <bg-image>, <bg-position>, <bg-size>, and <repeat-style> components. If two <bg-position>, <bg-size>, or <repeat-style> components are specified, the first value is the horizontal value and the second value is the vertical value. If only a single value is set, that value is applied to both dimensions.

The <'background-color'> component may only be included in the last background layer specified.

Component properties not set in the background shorthand property value declaration are set to their default values.

Component property order

Because some of the component properties share value types, the order of those component properties within the shorthand is important.

The <bg-size> value may only be included immediately after <bg-position>, separated with the / character. For example: 10px 10px / 80% 80% means the background image is 80% as tall and as wide as the element, and will be positioned 10px from the top and 10px from the left of the element's top-left corner. Within the <bg-position>, if both values are lengths, or one is a length and the other is center, the first value refers to the horizontal position and the second value refers to the vertical position.

Each background layer can include zero, one, or two <visual-box> values. If only one value is included, it sets both background-origin and background-clip. If two values are present, the first occurrence specifies the background-origin and the second specifies the background-clip value. If no <visual-box> values are present, the background-origin defaults to padding-box and the background-clip defaults to border-box.

While there is no order requirement for the other background properties, the following order is recommended for consistency and legibility; remember that none of the values are required:

<bg-image> <bg-position> / <bg-size> <repeat-style> <attachment> <bg-clip> <bg-origin> <'background-color'>

The following background explicitly sets all the default values in this order:

```
background: none 0% 0% / auto auto repeat scroll border-box padding-box
  transparent;
```

The following three lines of CSS are equivalent to the above, even if the order differs:

```
background: none;
background: transparent;
background: repeat scroll 0% 0% / auto padding-box border-box none transparent;
```

Image painting order

If multiple comma-separated backgrounds are included, they create multiple background layers on top of one another. The first background in the list creates the top layer. If the top layer contains no transparent areas, this is the only layer that will be visible.

The last layer is the bottom layer. The background color is always included in this layer.

Body background applied to the entire document

If the document <html> :root element's computed background-image value is none and its background-color is transparent, the browser will transfer the background styles set on the <body> element onto the :root and treat the <body> as if background: initial were set. In other words, the <html> element gets all the background styles set on the <body> element, and the <body> element's background properties are set to their initial values.

Because of this behavior, the specification authors recommend setting your document's background styles in your body style block rather than your html style block. However, it's important to note that using containment disables this behavior. When the contain property is set to anything other than none on either the <html> or <body> element, the background property and any longhand components do not propagate from the <body> element to the root <html> element.

Formal definition

Initial value	as each of the properties of the shorthand: • background-image: none • background-position: 0% 0% • background-size: auto auto • background-repeat: repeat • background-origin: padding-box • background-clip: border-box • background-attachment: scroll • background-color: transparent
Applies to	all elements. It also applies to ::first-letter and ::first-line.
Inherited	no
Percentages	as each of the properties of the shorthand: • background-position: refer to the size of the background positioning area minus size of background image; size refers to the width for horizontal offsets and to the height for vertical offsets • background-size: relative to the background positioning area

Computed value	as each of the properties of the shorthand:
	• background-image: as specified, but with <url> values made absolute
	• background-position: as each of the properties of the shorthand:
	  ◦ background-position-x: A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword
	  ◦ background-position-y: A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword
	• background-size: as specified, but with relative lengths converted into absolute lengths
	• background-repeat: a list, each item consisting of two keywords, one per dimension
	• background-origin: as specified
	• background-clip: as specified
	• background-attachment: as specified
	• background-color: computed color

Animation type	as each of the properties of the shorthand:

	• background-color: a color • background-image: discrete • background-clip: a repeatable list • background-position: a repeatable list • background-size: a repeatable list • background-repeat: discrete • background-attachment: discrete

Formal syntax

```
background =
  <bg-layer>#? , <final-bg-layer>

<bg-layer> =
  <bg-image>                    ||
  <bg-position> [ / <bg-size> ]?  ||
  <repeat-style>                ||
  <attachment>                  ||
  <visual-box>                  ||
  <bg-clip>

<final-bg-layer> =
  <bg-image>                    ||
  <bg-position> [ / <bg-size> ]?  ||
  <repeat-style>                ||
  <attachment>                  ||
  <visual-box>                  ||
  <bg-clip>                     ||
  <'background-color'>

<bg-image> =
  <image>  |
  none

<bg-position> =
  <position>        |
  <position-three>

<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<repeat-style> =
  repeat-x          |
  repeat-y          |
  repeat-block      |
  repeat-inline     |
  <repetition>{1,2}

<attachment> =
  scroll  |
  fixed   |
  local

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<bg-clip> =
  <visual-box>            |
  [ border-area || text ]

<background-color> =
  <color>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-three> =
  [ left | center | right ] && [ [ top | bottom ] <length-percentage> ]  |
  [ [ left | right ] <length-percentage> ] && [ top | center | bottom ]

<length-percentage> =
  <length>      |
  <percentage>

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )
```

```
<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Box Model Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Examples

Setting backgrounds with color keywords and images

HTML

```
<p class="top-banner">
  Starry sky<br />
  Twinkle twinkle<br />
  Starry sky
</p>
<p class="warning">Here is a paragraph</p>
```

CSS

```
.warning {
  background: pink;
}

.top-banner {
  background: url("star-solid.gif") #9999ff repeat-y fixed;
}
```

Result

See also

- box-decoration-break
- Using gradients
- Using multiple backgrounds

background-attachment CSS property

The **background-attachment** CSS property sets whether a background image's position is fixed within the viewport, or scrolls with its containing block.

Try it

```
background-attachment: scroll;
```

```
background-attachment: fixed;
```

```
background-attachment: local;
```

```
background-attachment: fixed, scroll;
```

```
background-attachment: scroll, fixed;
```

```
<section id="default-example">
  <div id="example-element">
    <p>
      From there to here<br />
      from here to there,<br />
      Funny things<br />
      Are everywhere.
    </p>
    <p>--Dr. Seuss</p>
  </div>
</section>
```

```
body {
  overflow: scroll;
}

#default-example {
  height: 600px;
}

#example-element {
  max-width: 20rem;
  height: 100%;
  background:
    url("/shared-assets/images/examples/lizard.png") right 3rem top 1rem / 15rem
      no-repeat,
    url("/shared-assets/images/examples/moon.jpg") center / 10rem;
  font-size: 1.2rem;
  font-weight: bolder;
  overflow: auto;
  padding: 20px;
  color: red;
  text-shadow:
    0 0 0.5rem black,
    0 0 0.5rem black;
}
```

Syntax

```
/* Keyword values */
background-attachment: scroll;
background-attachment: fixed;
background-attachment: local;

/* Global values */
background-attachment: inherit;
background-attachment: initial;
background-attachment: revert;
background-attachment: revert-layer;
background-attachment: unset;
```

Values

This property is specified as one or more comma-separated keyword values:

`fixed`

The background is fixed relative to the viewport. Even if an element has a scrolling mechanism, the background doesn't move with the element. If set, the `background-origin` property is ignored.

`local`

The background is fixed relative to the element's contents. If the element has a scrolling mechanism, the background scrolls with the element's contents, and the background painting area and background positioning area are relative to the scrollable area of the element rather than to the border framing them.

`scroll`

The background is fixed relative to the element itself and does not scroll with its contents. (It is effectively attached to the element's border.)

Formal definition

Initial value	`scroll`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
background-attachment =
  <attachment>#

<attachment> =
  scroll  |
  fixed   |
  local
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

HTML

We include an unordered list (`<ul>`) with some list items (`<li>`).

```
<ul>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  <li>Black fish</li>
  <li>Blue fish</li>
  <li>Old fish</li>
  <li>New fish.</li>
  <li>This one has a little star.</li>
  <li>This one has a little car.</li>
  <li>Say! What a lot</li>
  <li>Of fish there are.</li>
</ul>
```

CSS

We define a `background-image` and set the `background-attachment` to `fixed`. We also include a `height`, `width`, and `overflow` to ensure the element scrolls.

```
ul {
  background-image: url("star-solid.gif");
  background-attachment: fixed;

  width: 300px;
  height: 70px;
  overflow: scroll;
}
```

Result

Note how the background remains fixed relative to the list's viewport when you scroll the overflowing text into view.

Multiple background images

This property supports multiple background images. You can specify a different `<attachment>` for each background, separated by commas. Each image is matched with the corresponding `<attachment>` type, from first specified to last.

HTML

We include all of Dr. Seuss's poem.

```
<div>
  <ul>
    <li>One fish</li>
    <li>Two fish</li>
    <li>Red fish</li>
    <li>Blue fish</li>
    <li>Black fish</li>
    <li>Blue fish</li>
    <li>Old fish</li>
    <li>New fish.</li>
    <li>This one has a little star.</li>
    <li>This one has a little car.</li>
    <li>Say! What a lot</li>
    <li>Of fish there are.</li>
    <li>Yes. Some are red. And some are blue.</li>
    <li>Some are old. And some are new.</li>
    <li>Some are sad.</li>
    <li>And some are glad.</li>
    <li>And some are very, very bad.</li>
    <li>Why are they</li>
    <li>Sad and glad and bad?</li>
    <li>I do not know.</li>
    <li>Go ask your dad.</li>
    <li>Some are thin.</li>
    <li>And some are fat.</li>
    <li>The fat one has</li>
    <li>A yellow hat.</li>
    <li>From there to here, from here to there,</li>
    <li>Funny things</li>
    <li>Are everywhere.</li>
  </ul>
  <p>--Dr. Seuss</p>
</div>
```

CSS

We include a `height`, `width`, and `overflow` on the parent `<div>` to ensure the contents scroll.

We define two comma-separated background images on the list, and set the `background-attachment` to `fixed, scroll`, meaning the first background image will be `fixed` and the second will `scroll`. We set the `background-repeat` to make both background images repeat vertically, separating them with the `background-position` property.

```
div {
  width: 300px;
  height: 200px;
  overflow: scroll;
}
ul {
  background-image: url("star-solid.gif"), url("star-transparent.gif");
  background-attachment: fixed, scroll;
  background-repeat: repeat-y;
  background-position:
    0 0,
    100px 0;
}
```

Result

Note how the first background image is fixed to the viewport while the second background image is fixed relative to the list.

See also

- The other `background` CSS properties:
  - `background-clip`
  - `background-color`
  - `background-image`
  - `background-origin`
  - `background-position`
  - `background-repeat`
  - `background-size`
- Using multiple backgrounds
- CSS backgrounds and borders module

background-blend-mode CSS property

The **`background-blend-mode`** CSS property sets how an element's background images should blend with each other and with the element's background color.

Blending modes should be defined in the same order as the `background-image` property. If the blending modes' and background images' list lengths are not equal, it will be repeated and/or truncated until lengths match.

Try it

```
background-blend-mode: normal;
```

```
background-blend-mode: multiply;
```

```
background-blend-mode: hard-light;
```

```
background-blend-mode: difference;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element"></div>
  </div>
</section>
```

```
#example-element {
  background-color: green;
  background-image: url("/shared-assets/images/examples/balloon.jpg");
  width: 250px;
  height: 305px;
}
```

Syntax

```
/* One value */
background-blend-mode: normal;

/* Two values, one per background */
background-blend-mode: darken, luminosity;

/* Global values */
background-blend-mode: inherit;
background-blend-mode: initial;
background-blend-mode: revert;
background-blend-mode: revert-layer;
background-blend-mode: unset;
```

Values

`<blend-mode>`

The blending mode to be applied. There can be several values, separated by commas.

Formal definition

Initial value	`normal`
Applies to	All elements. In SVG, it applies to container elements, graphics elements, and graphics referencing elements.. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
background-blend-mode =
  <'mix-blend-mode'>#

<mix-blend-mode> =
  <blend-mode>  |
  plus-lighter

<blend-mode> =
  normal       |
  darken       |
  multiply     |
  color-burn   |
  lighten      |
  screen       |
  color-dodge  |
  overlay      |
  soft-light   |
  hard-light   |
  difference   |
  exclusion    |
  hue          |
```

```
  saturation   |
  color        |
  luminosity
```

This syntax reflects the latest standard as per Compositing and Blending Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
.item {
  width: 300px;
  height: 300px;
  background: url("image1.png"), url("image2.png");
  background-blend-mode: screen;
}
```

Try out different blend modes

```
<div id="div"></div>
<select id="select">
  <option>normal</option>
  <option>multiply</option>
  <option selected>screen</option>
  <option>overlay</option>
  <option>darken</option>
  <option>lighten</option>
  <option>color-dodge</option>
  <option>color-burn</option>
  <option>hard-light</option>
  <option>soft-light</option>
  <option>difference</option>
  <option>exclusion</option>
  <option>hue</option>
  <option>saturation</option>
  <option>color</option>
  <option>luminosity</option>
</select>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: screen;
}
```

```
document.getElementById("select").onchange = (event) => {
  document.getElementById("div").style.backgroundBlendMode =
    document.getElementById("select").selectedOptions[0].innerHTML;
};
console.log(document.getElementById("div"));
```

See also

- `<blend-mode>`
- `mix-blend-mode`

background-clip CSS property

The **`background-clip`** CSS property sets whether an element's background extends underneath its border box, padding box, or content box.

Try it

```
background-clip: border-box;
```

```
background-clip: padding-box;
```

```
background-clip: content-box;
```

```
background-clip: text;
color: transparent;
text-shadow: none;
```

```
<section id="default-example">
  <div id="example-element">This is the content of the element.</div>
</section>
```

```
#example-element {
  background-image: url("/shared-assets/images/examples/leopard.jpg");
  color: white;
  text-shadow: 2px 2px black;
  padding: 20px;
  border: 10px dashed #333333;
  font-size: 2em;
  font-weight: bold;
}
```

Syntax

```
/* Keyword values */
background-clip: border-box;
background-clip: padding-box;
background-clip: content-box;
background-clip: text;
background-clip: border-area;

/* Global values */
background-clip: inherit;
```

```
background-clip: initial;
background-clip: revert;
background-clip: revert-layer;
background-clip: unset;
```

Values

`border-box`

The background extends to the outside edge of the border (but underneath the border in z-ordering).

`padding-box`

The background extends to the outside edge of the padding. No background is drawn beneath the border.

`content-box`

The background is painted within (clipped to) the content box.

`text`

The background is painted within (clipped to) the foreground text.

`border-area`

The background is painted within (clipped to) the area painted by the border, taking `border-width` and `border-style` into account but ignoring any transparency introduced by `border-color`.

Description

The background is always drawn behind the border, so `background-clip: border-box` has a visual effect only when the border is partially opaque or has transparent or partially opaque regions. Also, the `background-clip: text` property has little to no visual effect if the text is fully or partially opaque.

> **Note:** Because the root element has a different background painting area, the `background-clip` property has no effect when specified on it. See "The backgrounds of special elements."

> **Note:** For documents whose root element is an HTML element: if the computed value of `background-image` on the root element is `none` and its `background-color` is `transparent`, user agents must instead propagate the computed values of the `background` properties from that element's first HTML `<body>` child element. The used values of that `<body>` element's `background` properties are their initial values, and the propagated values are treated as if they were specified on the root element. It is recommended that authors of HTML documents specify the canvas background for the `<body>` element rather than the HTML element.

Accessibility

When using `background-clip: text`, check that the contrast ratio between the background color and the color of the text placed over it is high enough that people experiencing low vision conditions will be able to read the content of the page.

If the background image does not load, this could also lead to the text becoming unreadable. Add a fallback `background-color` to prevent this from happening, and test without the image.

Consider using feature queries with `@supports` to test for support of `background-clip: text` and provide an accessible alternative where it is not supported.

Formal definition

Initial value	`border-box`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified
Animation type	a repeatable list

Formal syntax

```
background-clip =
  <bg-clip>#

<bg-clip> =
  <visual-box>             |
  [ border-area || text ]

<visual-box> =
  content-box  |
  padding-box  |
  border-box
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Box Model Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p class="border-box">The background extends behind the border.</p>
<p class="padding-box">
  The background extends to the inside edge of the border.
</p>
<p class="content-box">
  The background extends only to the edge of the content box.
</p>
<p class="text">The background is clipped to the foreground text.</p>
<p class="border-area">
  The background is clipped to the area painted by the border.
</p>
```

CSS

```
p {
  border: 0.8em darkviolet;
  border-style: dotted double;
  margin: 1em 0;
```

```
  padding: 1.4em;
  background: linear-gradient(60deg, red, yellow, red, yellow, red);
  font: 900 1.2em sans-serif;
  text-decoration: underline;
}

.border-box {
  background-clip: border-box;
}
.padding-box {
  background-clip: padding-box;
}
.content-box {
  background-clip: content-box;
}

.text {
  background-clip: text;
  color: rgb(0 0 0 / 20%);
}

.border-area {
  background-clip: border-area;
  border-color: transparent;
}
```

Result

See also

- The `clip-path` property creates a clipping region that defines what part of an *entire element* should be displayed.
- Background properties: `background`, `background-color`, `background-image`, `background-origin`
- Introduction to the CSS box model

background-color CSS property

The **`background-color`** CSS property sets the background color of an element.

Try it

```
background-color: brown;
```

```
background-color: #74992e;
```

```
background-color: rgb(255 255 128);
```

```
background-color: rgb(255 255 128 / 0.5);
```

```
background-color: hsl(50 33% 25%);
```

```
background-color: hsl(50 33% 25% / 0.75);
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Syntax

```
/* Keyword values */
background-color: red;
background-color: indigo;

/* Hexadecimal value */
background-color: #bbff00; /* Fully opaque */
background-color: #bf0; /* Fully opaque shorthand */
background-color: #11ffee00; /* Fully transparent */
background-color: #1fe0; /* Fully transparent shorthand */
background-color: #11ffeeff; /* Fully opaque */
background-color: #1fef; /* Fully opaque shorthand */

/* RGB value */
background-color: rgb(255 255 128); /* Fully opaque */
background-color: rgb(117 190 218 / 50%); /* 50% transparent */

/* HSL value */
background-color: hsl(50 33% 25%); /* Fully opaque */
background-color: hsl(50 33% 25% / 75%); /* 75% opaque, i.e. 25% transparent */

/* Special keyword values */
background-color: currentColor;
background-color: transparent;

/* Global values */
background-color: inherit;
background-color: initial;
background-color: revert;
background-color: revert-layer;
background-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The uniform color of the background. It is rendered behind any `background-image` that is specified, although the color will still be visible through any transparency in the image.

Description

The `background-color` property sets the background color of an element box. The color is drawn behind any background images. By default, the background color is painted within the border-box, meaning it's painted behind the border, ending at the outer edge of the border-box.

Clipping of the `background-color` paint area is controlled via the `background-clip` property. If multiple background images are set, the background color's clipping is determined by the value of the `background-clip` associated with the bottom-most background image.

Accessibility

It is important to ensure that the contrast ratio between the background color and the color of the text placed over it is high enough that people experiencing low vision conditions will be able to read the content of the page. A high contrast ratio also improves content accessibility for glossy-screened mobile device users when in a bright environment, such as sunlight.

Color contrast ratio is determined by comparing the luminance of the text and background color values. In order to meet current Web Content Accessibility Guidelines (WCAG), a ratio of 4.5:1 is required for text content and 3:1 for larger text such as headings. Large text is defined as 18.66px and bold or larger, or 24px or larger.

- Understanding WCAG: Perceivable Guideline 1.4.3
- Understanding color contrast
- Understanding color contrast
- WebAIM: Color Contrast Checker
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`transparent`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
background-color =
  <color>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

This example demonstrates applying `background-color` to HTML `<p>` elements using different CSS `<color>` values.

HTML

```
<p class="example-one">Lorem ipsum dolor sit amet, consectetuer</p>

<p class="example-two">Lorem ipsum dolor sit amet, consectetuer</p>

<p class="example-three">Lorem ipsum dolor sit amet, consectetuer</p>
```

CSS

Each paragraph is set to a different background color, including explicitly setting the default `transparent`, an `rgb()` color function, and a `<hex-color>`. We also set the `color` property to ensure enough contrast between the text and its background.

```
.example-one {
  background-color: transparent;
}

.example-two {
  background-color: rgb(153 102 153);
  color: rgb(255 255 204);
}

.example-three {
  background-color: #777799;
  color: white;
}
```

Result

Colorized tables

This example demonstrates the use of `background-color` on HTML `<table>` elements, including `<tr>` rows and `<td>` cells. It also demonstrates how background colors are painted behind any borders.

HTML

```
<table>
  <tbody>
    <tr id="r1">
      <td id="c11">11</td>
      <td id="c12">12</td>
      <td id="c13">13</td>
    </tr>
    <tr id="r2">
      <td id="c21">21</td>
      <td id="c22">22</td>
      <td id="c23">23</td>
    </tr>
    <tr id="r3">
      <td id="c31">31</td>
      <td id="c32">32</td>
      <td id="c33">33</td>
```

```
    </tr>
  </tbody>
</table>
```

CSS

We use CSS to set different `<named-color>` values. We also set a large dashed border on the table and every cell to demonstrate how the `background-color` is painted to the outer edge of the border-box.

```
table {
  border-collapse: collapse;
  border: dashed black 5px;
  width: 250px;
  height: 150px;
}
td {
  border: dashed 5px black;
}
#r1 {
  background-color: lightblue;
}
#c12 {
  background-color: cyan;
}
#r2 {
  background-color: grey;
}
#r3 {
  background-color: olive;
}
```

Result

See also

- `background-clip`
- Multiple backgrounds
- The `<color>` data type
- Other color-related properties: `color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`

background-image CSS property

The **`background-image`** CSS property sets one or more background images on an element.

Try it

```
background-image: url("/shared-assets/images/examples/lizard.png");
```

```
background-image:
  url("/shared-assets/images/examples/lizard.png"),
  url("/shared-assets/images/examples/star.png");
```

```
background-image:
  url("/shared-assets/images/examples/star.png"),
  url("/shared-assets/images/examples/lizard.png");
```

```
background-image:
  linear-gradient(rgb(0 0 255 / 0.5), rgb(255 255 0 / 0.5)),
  url("/shared-assets/images/examples/lizard.png");
```

```
<section id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Syntax

```
/* single image */
background-image: linear-gradient(black, white);
background-image: url("cat-front.png");

/* multiple images */
background-image:
  radial-gradient(circle, transparent 45%, black 48%),
  radial-gradient(ellipse farthest-corner, #fc1c14 20%, #cf15cf 80%);

/* Global values */
background-image: inherit;
background-image: initial;
background-image: revert;
background-image: revert-layer;
background-image: unset;
```

Values

This property is specified as the keyword `none` or as a comma-separated list of `<image>` values.

`none`

Is a keyword denoting the absence of images.

`<image>`

Is an `<image>` denoting the image to display.

Description

The `background-image` property sets background images on an element.

You can define multiple backgrounds on an element by specifying a list of comma-separated image. The background images are drawn on stacking context layers on top of each other. The first layer specified is drawn as if it is closest to the user.

The borders of the element are then drawn on top of them, and the `background-color` is drawn beneath them. How the images are drawn relative to the box and its borders is defined by the `background-clip` and `background-origin` CSS properties.

If a specified image cannot be drawn (for example, when the file denoted by the specified URI cannot be loaded), browsers handle it as they would a `none` value.

> **Note:** Even if the images are opaque and the color won't be displayed in normal circumstances, web developers should always specify a `background-color`. If the images cannot be loaded—for instance, when the network is down—the background color will be used as a fallback.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Additionally, it is important to ensure that the contrast ratio between the background image and the foreground text is high enough that people with low vision can read the page content.

Color contrast ratio is determined by comparing the luminance of the text and background color values. To meet Web Content Accessibility Guidelines (WCAG), a ratio of 4.5:1 is required for body text content and 3:1 for larger text such as headings. Large text is defined as 24px or larger, or bolded 18.66px or larger.

- WebAIM: Color Contrast Checker
- Understanding WCAG, Guideline 1.4 explanation
- Understanding Success Criterion 1.4.3 | Understanding WCAG 2.0, W3C (2023)

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
background-image =
  <bg-image>#

<bg-image> =
  <image>  |
  none

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Layering background images

Note that the star image is partially transparent and is layered over the cat image.

HTML

```
<div>
  <p class="cats-and-stars">This paragraph is full of cats<br />and stars.</p>
  <p>This paragraph is not.</p>
  <p class="cats-and-stars">Here are more cats for you.<br />Look at them!</p>
  <p>And no more.</p>
</div>
```

CSS

```
p {
  font-weight: bold;
  font-size: 1.5em;
  color: white;
  text-shadow: 0.07em 0.07em 0.05em black;
  background-image: none;
  background-color: transparent;
}

div {
  background-image: url("mdn_logo_only_color.png");
}

.cats-and-stars {
  background-image: url("star-transparent.gif"), url("cat-front.png");
  background-color: transparent;
}
```

Result

See also

- `<img>`
- Image-related functions:
  - `linear-gradient()`
  - `radial-gradient()`
  - `conic-gradient()`
  - `repeating-linear-gradient()`
  - `repeating-radial-gradient()`
  - `repeating-conic-gradient()`
  - `<url>`
- Using CSS gradients
- Implementing image sprites in CSS
- CSS images module
- Background-related properties
  - `background-attachment`
  - `background-clip`
  - `background-color`
  - `background-origin`
  - `background-position`
  - `background-repeat`
  - `background-size`
  - `background` shorthand
- Learn: Backgrounds and borders
- Using multiple backgrounds
- Resizing background images
- CSS backgrounds and borders module

background-origin CSS property

The **`background-origin`** CSS property sets the background's origin: from the border start, inside the border, or inside the padding.

Note that `background-origin` is ignored when `background-attachment` is `fixed`.

Try it

```
background-origin: border-box;
background-repeat: no-repeat;
```

```
background-origin: padding-box;
background-repeat: no-repeat;
```

```
background-origin: content-box;
background-repeat: no-repeat;
```

```
<section id="default-example">
  <div id="example-element">This is the content of the element.</div>
</section>
```

```
#example-element {
  background-image: url("/shared-assets/images/examples/leopard.jpg");
  color: #d73611;
  text-shadow: 2px 2px black;
  padding: 20px;
  border: 10px dashed #333333;
  font-size: 2em;
```

```
  font-weight: bold;
}
```

Syntax

```
/* Keyword values */
background-origin: border-box;
background-origin: padding-box;
background-origin: content-box;

/* Global values */
background-origin: inherit;
background-origin: initial;
background-origin: revert;
background-origin: revert-layer;
background-origin: unset;
```

Values

This property is specified as one of the following keyword values:

`border-box`

The background is positioned relative to the border box.

`padding-box`

The background is positioned relative to the padding box. Default value.

`content-box`

The background is positioned relative to the content box.

Formal definition

Initial value	`padding-box`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified
Animation type	a repeatable list

Formal syntax

```
background-origin =
  <visual-box>#

<visual-box> =
  content-box  |
  padding-box  |
  border-box
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Box Model Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting background origins

```
.example {
  border: 10px double;
  padding: 10px;
  background: url("image.jpg");
  background-position: center left;
  background-origin: content-box;
}
```

```
#example2 {
  border: 4px solid black;
  padding: 10px;
  background: url("image.gif");
  background-repeat: no-repeat;
  background-origin: border-box;
}
```

```
div {
  background-image:
    url("logo.jpg"), url("main-back.png"); /* Applies two images to the background */
  background-position:
    top right,
    0px 0px;
  background-origin: content-box, padding-box;
}
```

Using two gradients

In this example the box has a thick dotted border. The first gradient uses the `padding-box` as the `background-origin` and therefore the background sits inside the border. The second uses the `content-box` and so only displays behind the content.

```
.box {
  margin: 10px 0;
  color: white;
  background:
    linear-gradient(
      90deg,
      rgb(131 58 180 / 100%) 0%,
      rgb(253 29 29 / 60%) 60%,
      rgb(252 176 69 / 100%) 100%
    ),
```

```
    radial-gradient(circle, white 0%, black 28%);
  border: 20px dashed black;
  padding: 20px;
  width: 400px;
  background-origin: padding-box, content-box;
  background-repeat: no-repeat;
}
```

```
<div class="box">Hello!</div>
```

See also

- background-clip

background-position CSS property

The **background-position** CSS property sets the initial position for each background image.

Try it

```
background-position: top;
```

```
background-position: left;
```

```
background-position: center;
```

```
background-position: 25% 75%;
```

```
background-position: bottom 50px right 100px;
```

```
background-position: right 35% bottom 45%;
```

```
<section class="display-block" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  background-color: navajowhite;
  background-image: url("/shared-assets/images/examples/star.png");
  background-repeat: no-repeat;
  height: 100%;
}
```

Syntax

```
/* Keyword values */
background-position: top;
background-position: bottom;
background-position: left;
background-position: right;
background-position: center;

/* <percentage> values */
background-position: 25% 75%;

/* <length> values */
background-position: 0 0;
background-position: 1cm 2cm;
background-position: 10ch 8em;

/* Multiple images */
background-position:
  0 0,
  center;

/* Edge offsets values */
background-position: bottom 10px right 20px;
background-position: right 3em bottom 10px;
background-position: bottom 10px right;
background-position: top right 10px;

/* Global values */
background-position: inherit;
background-position: initial;
background-position: revert;
background-position: revert-layer;
background-position: unset;
```

Values

This property is specified as one or more comma-separated `<position>` values:

`<position>`

A `<position>`. A two-dimensional coordinate, made up of one to two offsets, and optionally one to two offset sides, used to set a location relative to the edges of an element's box.

Description

The `background-position` property sets the initial position for each background image. The position is relative to the position layer set by `background-origin`.

The value is specified as a list of comma-separated image positions, with each image position defined using one to four values. If two non-keyword values are used, the first value represents the horizontal position and the second represents the vertical position. If only one value is specified, the second value is assumed to be `center`. If three or four values are used, the length-percentage values are offsets for the preceding keyword value(s).

1-value syntax

The value may be:

- The keyword value `center`, which centers the image.
- One of the keyword values `top`, `left`, `bottom`, or `right`. This specifies an edge against which to place the item. The other dimension is then set to 50%, so the item is placed in the middle of the edge specified.
- A `<length>` or `<percentage>`. This specifies the X coordinate relative to the left edge, with the Y coordinate set to 50%.

2-value syntax

One value defines X and the other defines Y. Each value may be:

- One of the keyword values `top`, `left`, `bottom`, or `right`. If `left` or `right` is given, then this defines X and the other given value defines Y. If `top` or `bottom` is given, then this defines Y and the other value defines X.
- A `<length>` or `<percentage>`. If the other value is `left` or `right`, then this value defines Y, relative to the top edge. If the other value is `top` or `bottom`, then this value defines X, relative to the left edge. If both values are `<length>` or `<percentage>` values, then the first defines X and the second Y.
- Note that: If one value is `top` or `bottom`, then the other value may not be `top` or `bottom`. If one value is `left` or `right`, then the other value may not be `left` or `right`. This means, e.g., that `top top` and `left right` are not valid.
- Order: When pairing keywords, placement is not important as the browser can reorder it; the values `top left` and `left top` will yield the same result. When pairing `<length>` or `<percentage>` with a keyword, the placement is important: the value defining X should come first followed by Y, so for example the value `right 20px` is valid while `20px right` is invalid. The values `left 20%` and `20% bottom` are valid as X and Y values are clearly defined and the placement is correct.
- The default value is `left top` or `0% 0%`.

3-value syntax

Two values are keyword values, and the third is the offset for the preceding value:

- The first value is one of the keyword values `top`, `left`, `bottom`, `right`, or `center`. If `left` or `right` are given here, then this defines X. If `top` or `bottom` are given, then this defines Y and the other keyword value defines X.
- The `<length>` or `<percentage>` value, if it is the second value, is the offset for the first value. If it is the third value, it is the offset for the second value.
- The single length or percentage value is an offset for the keyword value that precedes it. The combination of one keyword with two `<length>` or `<percentage>` values is not valid.

4-value syntax

The first and third values are keyword values defining X and Y. The second and fourth values are offsets for the preceding X and Y keyword values:

- The first and third values are equal to one of the keyword values `top`, `left`, `bottom`, or `right`. If `left` or `right` is given for the first value, then this defines X and the other value defines Y. If `top` or `bottom` is given for the first value, then this defines Y and the other keyword value defines X.
- The second and fourth values are `<length>` or `<percentage>` values. The second value is the offset for the first keyword. The fourth value is the offset for the second keyword.

Regarding Percentages

The percentage offset of the given background image's position is relative to the container. A value of 0% means that the left (or top) edge of the background image is aligned with the corresponding left (or top) edge of the container, or the 0% mark of the image will be on the 0% mark of the container. A value of 100% means that the *right* (or *bottom*) edge of the background image is aligned with the *right* (or *bottom*) edge of the container, or the 100% mark of the image will be on the 100% mark of the container. Thus a value of 50% horizontally or vertically centers the background image as the 50% of the image will be at the 50% mark of the container. Similarly, `background-position: 25% 75%` means the spot on the image that is 25% from the left and 75% from the top will be placed at the spot of the container that is 25% from the container's left and 75% from the container's top.

Essentially what happens is the background image dimension is *subtracted* from the corresponding container dimension, and then a percentage of the resulting value is used as the direct offset from the left (or top) edge.

```
(container width - image width) * (position x%) = (x offset value)
(container height - image height) * (position y%) = (y offset value)
```

Using the X axis for an example, let's say we have an image that is 300px wide and we are using it in a container that is 100px wide, with `background-size` set to auto:

```
100px - 300px = -200px (container & image difference)
```

So that with position percentages of -25%, 0%, 50%, 100%, 125%, we get these image-to-container edge offset values:

```
-200px * -25% = 50px
-200px * 0% = 0px
-200px * 50% = -100px
-200px * 100% = -200px
-200px * 125% = -250px
```

So with these resultant values for our example, the **left edge** of the **image** is offset from the **left edge** of the **container** by:

- + 50px (putting the left image edge in the center of the 100-pixel-wide container)
- 0px (left image edge coincident with the left container edge)
- -100px (left image edge 100px to the left of the container, in this example that means the middle 100px image area is centered in the container)
- -200px (left image edge 200px to the left of the container, in this example that means the right image edge is coincident with the right container edge)
- -250px (left image edge 250px to the left of the container, in this example that puts the right edge of the 300px-wide image in the center of the container)

It's worth mentioning that if your `background-size` is equal to the container size for a given axis, then a *percentage* position for that axis will have no effect because the "container-image difference" will be zero. You will need to offset using absolute values.

Formal definition

Initial value	`0% 0%`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the size of the background positioning area minus size of background image; size refers to the width for horizontal offsets and to the height for vertical offsets
Computed value	as each of the properties of the shorthand: • `background-position-x`: A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword • `background-position-y`: A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword
Animation type	a repeatable list

Formal syntax

```
background-position =
  <bg-position>#

<bg-position> =
  <position>        |
```

```
  <position-three>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-three> =
  [ left | center | right ] && [ [ top | bottom ] <length-percentage> ]  |
  [ [ left | right ] <length-percentage> ] && [ top | center | bottom ]

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Positioning background images

Each of these three examples uses the `background` property to create a yellow, rectangular element containing a star image. In each example, the star is in a different position. The third example illustrates how to specify positions for two different background images within one element.

HTML

```
<div class="example-one">Example One</div>
<div class="example-two">Example Two</div>
<div class="example-three">Example Three</div>
```

CSS

```
/* Shared among all <div>s */
div {
  background-color: #ffee99;
  background-repeat: no-repeat;
  width: 300px;
  height: 80px;
  margin-bottom: 12px;
}

/* These examples use the `background` shorthand property */
.example-one {
  background: url("star-transparent.gif") #ffee99 2.5cm bottom no-repeat;
}
.example-two {
  background: url("star-transparent.gif") #ffee99 left 4em bottom 1em no-repeat;
}

/* Multiple background images: Each image is matched with the
   corresponding position, from first specified to last. */
.example-three {
  background-image: url("star-transparent.gif"), url("cat-front.png");
  background-position:
    0px 0px,
    right 3em bottom 2em;
}
```

Result

See also

- `background-position-x`
- `background-position-y`
- Using multiple backgrounds
- `transform-origin`

background-position-x CSS property

The **`background-position-x`** CSS property sets the initial horizontal position for each background image. The position is relative to the position layer set by `background-origin`.

The value of this property is overridden by any declaration of the `background` or `background-position` shorthand properties applied to the element after it.

Try it

```
background-position-x: left;
```

```
background-position-x: center;
```

```
background-position-x: 25%;
```

```
background-position-x: 2rem;
```

```
background-position-x: right 32px;
```

```
<section class="display-block" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  background-color: navajowhite;
  background-image: url("/shared-assets/images/examples/star.png");
  background-repeat: no-repeat;
  height: 100%;
}
```

Syntax

```
/* Keyword values */
background-position-x: left;
background-position-x: center;
background-position-x: right;

/* <percentage> values */
background-position-x: 25%;

/* <length> values */
background-position-x: 0px;
background-position-x: 1cm;
background-position-x: 8em;

/* Side-relative values */
background-position-x: right 3px;
background-position-x: left 25%;

/* Multiple values */
background-position-x: 0px, center;

/* Global values */
background-position-x: inherit;
background-position-x: initial;
background-position-x: revert;
background-position-x: revert-layer;
background-position-x: unset;
```

Values

This property is specified as one or more comma-separated values:

`left`

Aligns the left edge of the background image with the left edge of the background position layer.

`center`

Aligns the center of the background image with the center of the background position layer.

`right`

Aligns the right edge of the background image with the right edge of the background position layer.

`<length>`

The offset of the given background image's left vertical edge from the background position layer's left vertical edge. (Some browsers allow assigning the right edge for offset).

`<percentage>`

The offset of the given background image's horizontal position relative to the container. A value of 0% means that the left edge of the background image is aligned with the left edge of the container, and a value of 100% means that the *right* edge of the background image is aligned with the *right* edge of the container, thus a value of 50% horizontally centers the background image.

Formal definition

Initial value	`0%`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to width of background positioning area minus width of background image
Computed value	A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword
Animation type	a repeatable list

Formal syntax

```
background-position-x =
  [ center | [ [ left | right | x-start | x-end ]? <length-percentage>? ]! ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

The following example shows a background image implementation, with background-position-x and background-position-y used to define the image's horizontal and vertical positions separately.

HTML

```
<div></div>
```

CSS

```
div {
  width: 300px;
  height: 300px;
  background-color: skyblue;
  background-image: url("https://mdn.dev/archives/media/attachments/2020/07/29/17350/3b4892b7e820122ac6dd7678891d4507/firefox.png");
  background-repeat: no-repeat;
  background-position-x: center;
  background-position-y: bottom;
}
```

Result

Side-relative values

The following example shows support for side-relative offset syntax, which allows the developer to offset the background from any edge.

HTML

```
<div></div>
```

CSS

```
div {
  width: 300px;
  height: 300px;
  background-color: seagreen;
  background-image: url("https://mdn.dev/archives/media/attachments/2020/07/29/17350/3b4892b7e820122ac6dd7678891d4507/firefox.png");
  background-repeat: no-repeat;
  background-position-x: right 20px;
  background-position-y: bottom 10px;
}
```

Result

See also

- `background-position`
- `background-position-y`
- Using multiple backgrounds

background-position-y CSS property

The **`background-position-y`** CSS property sets the initial vertical position for each background image. The position is relative to the position layer set by `background-origin`.

The value of this property is overridden by any declaration of the `background` or `background-position` shorthand properties applied to the element after it.

Try it

```
background-position-y: top;
```

```
background-position-y: center;
```

```
background-position-y: 25%;
```

```
background-position-y: 2rem;
```

```
background-position-y: bottom 32px;
```

```
<section class="display-block" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  background-color: navajowhite;
  background-image: url("/shared-assets/images/examples/star.png");
  background-repeat: no-repeat;
  height: 100%;
}
```

Syntax

```
/* Keyword values */
background-position-y: top;
background-position-y: center;
background-position-y: bottom;

/* <percentage> values */
background-position-y: 25%;

/* <length> values */
background-position-y: 0px;
background-position-y: 1cm;
background-position-y: 8em;
```

```
/* Side-relative values */
background-position-y: bottom 3px;
background-position-y: bottom 10%;

/* Multiple values */
background-position-y: 0px, center;

/* Global values */
background-position-y: inherit;
background-position-y: initial;
background-position-y: revert;
background-position-y: revert-layer;
background-position-y: unset;
```

Values

This property is specified as one or more comma-separated values:

`top`

Aligns the top edge of the background image with the top edge of the background position layer.

`center`

Aligns the vertical center of the background image with the vertical center of the background position layer.

`bottom`

Aligns the bottom edge of the background image with the bottom edge of the background position layer.

`<length>`

The offset of the given background image's horizontal edge from the corresponding background position layer's top horizontal edge. (Some browsers allow assigning the bottom edge for offset).

`<percentage>`

The offset of the given background image's vertical position relative to the container. A value of 0% means that the top edge of the background image is aligned with the top edge of the container, and a value of 100% means that the *bottom* edge of the background image is aligned with the *bottom* edge of the container, thus a value of 50% vertically centers the background image.

Formal definition

Initial value	`0%`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to height of background positioning area minus height of background image
Computed value	A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword
Animation type	a repeatable list

Formal syntax

```
background-position-y =
  [ center | [ [ top | bottom | y-start | y-end ]? <length-percentage>? ]! ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

The following example shows a background image implementation, with background-position-x and background-position-y used to define the image's horizontal and vertical positions separately.

HTML

```
<div></div>
```

CSS

```
div {
  width: 300px;
  height: 300px;
  background-color: skyblue;
  background-image: url("https://mdn.dev/archives/media/attachments/2020/07/29/17350/3b4892b7e820122ac6dd7678891d4507/firefox.png");
  background-repeat: no-repeat;
  background-position-x: center;
  background-position-y: bottom;
}
```

Result

Side-relative values

The following example shows support for side-relative offset syntax, which allows the developer to offset the background from any edge.

HTML

```
<div></div>
```

CSS

```
div {
  width: 300px;
  height: 300px;
  background-color: seagreen;
```

```
  background-image: url("https://mdn.dev/archives/media/attachments/2020/07/29/17350/3b4892b7e820122ac6dd7678891d4507/firefox.png");
  background-repeat: no-repeat;
  background-position-x: right 20px;
  background-position-y: bottom 10px;
}
```

Result

See also

- background-position
- background-position-x
- Using multiple backgrounds

background-repeat CSS property

The **background-repeat** CSS shorthand property sets how background images are repeated. A background image can be repeated along the horizontal and vertical axes, or not repeated at all.

Try it

```
background-repeat: repeat-x;
```

```
background-repeat: repeat;
```

```
background-repeat: space;
```

```
background-repeat: round;
```

```
background-repeat: no-repeat;
```

```
background-repeat: space repeat;
```

```
<section id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  background: #cccccc url("/shared-assets/images/examples/moon.jpg") center /
    120px;
  min-width: 100%;
  min-height: 100%;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- background-repeat-x
- background-repeat-y

Syntax

```
/* Keyword values */
background-repeat: repeat;
background-repeat: repeat-x;
background-repeat: repeat-y;
background-repeat: space;
background-repeat: round;
background-repeat: no-repeat;

/* Two-value syntax: horizontal | vertical */
background-repeat: repeat space;
background-repeat: repeat repeat;
background-repeat: round space;
background-repeat: no-repeat round;

/* Global values */
background-repeat: inherit;
background-repeat: initial;
background-repeat: revert;
background-repeat: revert-layer;
background-repeat: unset;
```

Description

The property accepts a comma-separated list of two <repeat-style> keyterms, or one keyterm as a shorthand for the two values. When two values are provided, the first value defines the horizontal repetition behavior and the second value defines the vertical behavior. Property values can be used to repeat only horizontally, vertically, or not at all.

The default value is repeat repeat. With this value, the background image maintains its intrinsic aspect ratio, repeating both horizontally and vertically to cover the entire background paint area, with edge images being clipped to the size of the element. Which edges clipped depends on the value of the corresponding background-position value. How many times they are repeated and how much the images on the edges are clipped depends on the size of the background painting area and the corresponding background-size value.

The repeating images can be evenly spaced apart, ensuring the repeated image maintains its aspect ratio without being clipped. With the space value, if the background paint area has a different aspect ratio than the image or does not otherwise have a size that is a multiple of the background size in either direction, there will be areas not covered by the background image.

Alternatively, the repeated background image can be stretched to cover the entire area without clipping. With round, the repeated image is stretched to fill all the available space until there is room to add an additional repeated image if the aspect ratio of the background image is not the same as the paint area's aspect ratio. For example, given a background image that is 100px x 100px and a background paint area of 1099px x 750px, the image will be repeated 10 times in the horizontal direction and 7 times vertically, for a total of 70 repetitions, with each image stretched in both directions to be 109.9px x 105px, altering the image's aspect ratio and potentially distorting it. If the width of the paint area increases by 1px, becoming 1100px wide, an 11th image will fit horizontally for a total of 77 image repetitions, with each image being painted at 100px wide and 105px tall, stretched only in the vertical direction.

Values

The property accepts a comma-separated list of two <repeat-style> keyterms or one keyterm as a shorthand for the two values. The first value is the horizontal repetition. The second value is the vertical behavior. If only a single value is set to a value other than repeat-x or repeat-y, that value is applied the both vertices. The values include:

`repeat`

The default value. The image is repeated as many times as needed to cover the entire background image painting area, with the edge image being clipped if the dimension of the painting area is not a multiple of the dimension of your background image.

`no-repeat`

The image is not repeated (and hence the background image painting area will not necessarily be entirely covered). The position of the non-repeated background image is defined by the `background-position` CSS property.

`space`

The image is repeated as much as possible without clipping. The first and last images are pinned to either side of the element, and whitespace is distributed evenly between the images. The `background-position` property is ignored unless only one image can be displayed without clipping. The only case where clipping happens using `space` is when there isn't enough room to display one image.

`round`

As the allowed space increases in size, the repeated images will stretch (leaving no gaps) until there is room for another one to be added. This is the only `<repeat-style>` value that can lead to the distortion of the background image's aspect ratio, which will occur if the aspect ratio of the background image differs from the aspect ratio of the background paint area.

`repeat-x`

Shorthand for `repeat no-repeat`, the background image repeats horizontally only, with the edge image being clipped if the width of the paint area is not a multiple of the background image's width.

`repeat-y`

Shorthand for `no-repeat repeat`, the background image repeats vertically only, with the edge image being clipped if the height of the paint area is not a multiple of the background image's height.

When one `<repeat-style>` keyterm is provided, the value is shorthand for the following two-value syntax:

Single value	Two-value equivalent
`repeat-x`	`repeat no-repeat`
`repeat-y`	`no-repeat repeat`
`repeat`	`repeat repeat`
`space`	`space space`
`round`	`round round`
`no-repeat`	`no-repeat no-repeat`

Formal definition

Initial value	`repeat`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	a list, each item consisting of two keywords, one per dimension
Animation type	discrete

Formal syntax

```
background-repeat =
  <repeat-style>#

<repeat-style> =
  repeat-x           |
  repeat-y           |
  repeat-block       |
  repeat-inline      |
  <repetition>{1,2}

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting background-repeat

HTML

```
<ol>
  <li>
    no-repeat
    <div class="one"></div>
  </li>
  <li>
    repeat
    <div class="two"></div>
  </li>
  <li>
    repeat-x
    <div class="three"></div>
  </li>
  <li>
    repeat-y
    <div class="four"></div>
  </li>
  <li>
    space
    <div class="five"></div>
  </li>
  <li>
    round
```

```
    <div class="six"></div>
  </li>
  <li>
    repeat-x, repeat-y (multiple images)
    <div class="seven"></div>
  </li>
</ol>
```

CSS

```
/* Shared for all DIVS in example */
ol,
li {
  margin: 0;
  padding: 0;
}
li {
  margin-bottom: 12px;
}
div {
  background-image: url("star-solid.gif");
  width: 160px;
  height: 70px;
}

/* Background repeats */
.one {
  background-repeat: no-repeat;
}
.two {
  background-repeat: repeat;
}
.three {
  background-repeat: repeat-x;
}
.four {
  background-repeat: repeat-y;
}
.five {
  background-repeat: space;
}
.six {
  background-repeat: round;
}

/* Multiple images */
.seven {
  background-image:
    url("star-solid.gif"), url("/shared-assets/images/examples/favicon32.png");
  background-repeat: repeat-x, repeat-y;
  height: 144px;
}
```

Result

In this example, each list item is matched with a different value of `background-repeat`.

See also

- The other `background` shorthand components: `background-attachment`, `background-clip`, `background-color`, `background-image`, `background-origin`, `background-position` (`background-position-x` and `background-position-y`), and `background-size`
- Using multiple backgrounds
- CSS backgrounds and borders module
- Understanding aspect ratios

background-repeat-x CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`background-repeat-x`** CSS property sets how background images are repeated, or not, along the horizontal axis.

The `background-repeat-x` and `background-repeat-y` properties can also be set using the `background-repeat` or `background` shorthand properties.

Try it

```
background-repeat-x: repeat;
```

```
background-repeat-x: space;
```

```
background-repeat-x: round;
```

```
background-repeat-x: no-repeat;
```

```
<section id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  background: #cccccc url("/shared-assets/images/examples/moon.jpg") no-repeat
    center / 120px;
  min-width: 100%;
  min-height: 100%;
}
```

Syntax

```
/* Keyword values */
background-repeat-x: repeat;
background-repeat-x: space;
background-repeat-x: round;
background-repeat-x: no-repeat;

/* Global values */
background-repeat-x: inherit;
background-repeat-x: initial;
background-repeat-x: revert;
background-repeat-x: revert-layer;
background-repeat-x: unset;
```

The `background-repeat-x` property is specified as one or more values, separated by commas.

Values

`repeat`

> The default value. The image is repeated as many times as needed to cover the entire width of the background image painting area. The edge image is clipped if multiple images do not fit exactly into the background width.

`no-repeat`

> The image is not repeated. The position of the non-repeated background image is defined by the `background-position` CSS property.

`space`

> The image is repeated as much as possible without clipping. The first and last images are pinned to the left and right of the element, and the whitespace is evenly distributed between them. The `background-position-x` property is ignored unless one or more images can be displayed without clipping. If the image is wider than the element, it is clipped, as there isn't enough room to display it.

`round`

> The image is repeated horizontally. As the available space increases in size, the repeated images will stretch (leaving no gaps) until there is room for another one to be added. If multiple images don't fit the background exactly, they are rescaled to fit.

Description

The `background-repeat-x` property accepts a comma-separated list of `<repetition>` keyterms, which define how the background image(s) should repeat horizontally, or not repeat at all.

The default value is `repeat`. With this value, the background image repeats horizontally, covering the entire width of the background paint area, with edge images being clipped to the size of the element. Whether the left, right, or both edges are clipped depends on the value of the corresponding `background-position` value. How many times images are repeated and how much the images on the edges are clipped depends on the size of the background painting area and width value of the corresponding `background-size` property.

The repeating images can be evenly spaced apart, ensuring the repeated images aren't clipped in the horizontal direction. With the `space` value, if the width of the background paint area is not a multiple of the width of the image or does not otherwise have a size that is a multiple of the background size in the width direction, there will be areas not covered by the background image.

Alternatively, the repeated background image can be stretched to cover the entire width of the background area without clipping. With `round`, the repeated image is stretched to fill all the available space until there is room to add an additional repeated image.

For example, given a background image that is `100px` x `100px` and a background paint area that is `1099px` wide, the image will be resized to `109.9px wide` and repeated 10 times in the horizontal direction. This will alter the image's aspect ratio and distort it. If the width of the paint area increases by `1px` to `1100px`, an 11th image will fit horizontally, with each image being painted at `100px` wide, and no longer distorted horizontally.

Formal definition

Value not found in DB!

Formal syntax

```
background-repeat-x =
  <repetition>#

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting background-repeat-x

HTML

```
<ol>
  <li>
    no-repeat
    <div class="one"></div>
  </li>
  <li>
    repeat
    <div class="two"></div>
  </li>
  <li>
    space
    <div class="three"></div>
  </li>
  <li>
    round
    <div class="four"></div>
  </li>
</ol>
```

CSS

```
ol,
li {
```

```
  margin: 0;
  padding: 0;
}
li {
  margin-bottom: 12px;
}
div {
  background-image: url("star-solid.gif");
  width: 160px;
  height: 70px;
}

/* Background repeats */
.one {
  background-repeat-x: no-repeat;
}
.two {
  background-repeat-x: repeat;
}
.three {
  background-repeat-x: space;
}
.four {
  background-repeat-x: round;
}
```

```
@layer no-support {
  @supports not (background-repeat-x: repeat) {
    body::before {
      content: "Your browser doesn't support the `background-repeat-x` property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

In this example, each list item is matched with a different value of `background-repeat-x`.

See also

- `background-repeat-y` and the `background-repeat` shorthand.
- `background` shorthand components: `background-attachment`, `background-clip`, `background-color`, `background-image`, `background-origin`, `background-position` (`background-position-x` and `background-position-y`), and `background-size`
- Using multiple backgrounds
- CSS backgrounds and borders module
- Understanding aspect ratios

background-repeat-y CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`background-repeat-y`** CSS property sets how background images are repeated, or not, along the vertical axis.

The `background-repeat-x` and `background-repeat-y` properties can also be set using the `background-repeat` or `background` shorthand properties.

Try it

```
background-repeat-y: repeat;
```

```
background-repeat-y: space;
```

```
background-repeat-y: round;
```

```
background-repeat-y: no-repeat;
```

```
<section id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  background: #cccccc url("/shared-assets/images/examples/moon.jpg") center /
    120px;
  min-width: 100%;
  min-height: 100%;
}
```

Syntax

```
/* Keyword values */
background-repeat-y: repeat;
background-repeat-y: space;
background-repeat-y: round;
background-repeat-y: no-repeat;

/* Global values */
background-repeat-y: inherit;
background-repeat-y: initial;
background-repeat-y: revert;
background-repeat-y: revert-layer;
background-repeat-y: unset;
```

The `background-repeat-y` property is specified as one or more values, separated by commas.

Values

`repeat`

> The default value. The image is repeated as many times as needed to cover the entire height of the background image painting area. The edge image is clipped if multiple images do not fit exactly into the background height.

`no-repeat`

> The image is not repeated. The position of the non-repeated background image is defined by the `background-position` CSS property.

`space`

> The image is repeated as much as possible without clipping. The first and last images are pinned to the top and bottom of the element, and the whitespace is distributed evenly between the images. The `background-position-y` property is ignored unless one or more images can be displayed without clipping. If the image is taller than the element, it is clipped, as there isn't enough room to display it.

`round`

> The image is repeated vertically. As the available space increases in size, the repeated images will stretch (leaving no gaps) until there is room for another one to be added. If multiple images don't fit the background exactly, they are rescaled to fit.

Description

The `background-repeat-y` property accepts a comma-separated list of `<repetition>` keyterms, which define how background image(s) should repeat vertically, or not repeat at all.

The default value is `repeat`. With this value, the background image repeats vertically, covering the entire height of the background paint area, with edge images being clipped to the size of the element. Whether the top, bottom, or both edges are clipped depends on the value of the corresponding `background-position` value. How many times images are repeated and how much the images on the edges are clipped depends on the size of the background painting area and height value of the corresponding `background-size` property.

The repeating images can be evenly spaced apart, ensuring the repeated images aren't clipped in the vertical direction. With the `space` value, if the height of the background paint area is not a multiple of the image's height or does not otherwise have a size that is a multiple of the background size in along the y-axis, there will be areas not covered by the background image.

Alternatively, the repeated background image can be stretched to cover the entire height of the background area without clipping. With `round`, if the height of the background paint area is not a multiple of the height of the background image, the repeated image is stretched to fill all the available space until there is room to add an additional repeated image.

For example, given a background image that is 100px x 100px and a background paint area that is 1099px tall, the image will be repeated 10 times in the vertical direction with each image stretched to be 109.9px tall, possibly altering the image's aspect ratio and distorting it. If the height of the paint area increases by 1px, becoming 1100px tall, an 11th image will fit vertically, with each image will be painted at 100px tall, no longer being stretched in the vertical direction.

Formal definition

Value not found in DB!

Formal syntax

```
background-repeat-y =
  <repetition>#

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting background-repeat-y

HTML

```
<ol>
  <li>
    no-repeat
    <div class="one"></div>
  </li>
  <li>
    repeat
    <div class="two"></div>
  </li>
  <li>
    space
    <div class="three"></div>
  </li>
  <li>
    round
    <div class="four"></div>
  </li>
</ol>
```

CSS

```
ol,
li {
  margin: 0;
  padding: 0;
}
li {
  margin-bottom: 12px;
}
div {
  background-image: url("star-solid.gif");
  width: 160px;
  height: 70px;
}

/* Background repeats */
.one {
  background-repeat-y: no-repeat;
}
```

```
.two {
  background-repeat-y: repeat;
}
.three {
  background-repeat-y: space;
}
.four {
  background-repeat-y: round;
}
```

```
@layer no-support {
  @supports not (background-repeat-y: repeat) {
    body::before {
      content: "Your browser doesn't support the `background-repeat-y` property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

In this example, each list item is matched with a different value of `background-repeat-y`.

See also

- `background-repeat-x` and the `background-repeat` shorthand.
- `background` shorthand components: `background-attachment`, `background-clip`, `background-color`, `background-image`, `background-origin`, `background-position` (`background-position-x` and `background-position-y`), and `background-size`
- Using multiple backgrounds
- CSS backgrounds and borders module
- Understanding aspect ratios

background-size CSS property

The **`background-size`** CSS property sets the size of the element's background image. The image can be left to its natural size, stretched, or constrained to fit the available space.

Spaces not covered by a background image are filled with the `background-color` property, and the background color will be visible behind background images that have transparency/translucency.

Try it

```
background-size: contain;
```

```
background-size: contain;
background-repeat: no-repeat;
```

```
background-size: cover;
```

```
background-size: 30%;
```

```
background-size: 200px 100px;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  background-image: url("/shared-assets/images/examples/hand.jpg");
  min-width: 100%;
  min-height: 100%;
}
```

Syntax

```
/* Keyword values */
background-size: cover;
background-size: contain;

/* One-value syntax */
/* the width of the image (height becomes 'auto') */
background-size: 50%;
background-size: 3.2em;
background-size: 12px;
background-size: auto;

/* Two-value syntax */
/* first value: width of the image, second value: height */
background-size: 50% auto;
background-size: 3em 25%;
background-size: auto 6px;
background-size: auto auto;

/* Multiple backgrounds */
background-size: auto, auto; /* Not to be confused with `auto auto` */
background-size: 50%, 25%, 25%;
background-size: 6px, auto, contain;

/* Global values */
background-size: inherit;
background-size: initial;
background-size: revert;
background-size: revert-layer;
background-size: unset;
```

The `background-size` property is specified in one of the following ways:

- Using the keyword values `contain` or `cover`.
- Using a width value only, in which case the height defaults to `auto`.
- Using both a width and a height value, in which case the first sets the width and the second sets the height. Each value can be a `<length>`, a `<percentage>`, or `auto`.

To specify the size of multiple background images, separate the value for each one with a comma.

Values

`contain`

Scales the image as large as possible within its container without cropping or stretching the image. If the container is larger than the image, this will result in image tiling, unless the `background-repeat` property is set to `no-repeat`.

`cover`

Scales the image (while preserving its ratio) to the smallest possible size to fill the container (that is: both its height and width completely *cover* the container), leaving no empty space. If the proportions of the background differ from the element, the image is cropped either vertically or horizontally.

`auto`

Scales the background image in the corresponding direction such that its intrinsic proportions are maintained.

`<length>`

Stretches the image in the corresponding dimension to the specified length. Negative values are not allowed.

`<percentage>`

Stretches the image in the corresponding dimension to the specified percentage of the *background positioning area*. The background positioning area is determined by the value of `background-origin` (by default, the padding box). However, if the background's `background-attachment` value is `fixed`, the positioning area is instead the entire viewport. Negative values are not allowed.

Intrinsic dimensions and proportions

The computation of values depends on the image's intrinsic dimensions (width and height) and intrinsic proportions (width-to-height ratio). These attributes are as follows:

- A bitmap image (such as JPG) always has intrinsic dimensions and proportions.
- A vector image (such as SVG) does not necessarily have intrinsic dimensions. If it has both horizontal and vertical intrinsic dimensions, it also has intrinsic proportions. If it has no dimensions or only one dimension, it may or may not have proportions.
- CSS `<gradient>`s have no intrinsic dimensions or intrinsic proportions.
- Background images created with the `element()` function use the intrinsic dimensions and proportions of the generating element.

> **Note:** In Gecko, background images created using the `element()` function are currently treated as images with the dimensions of the element, or of the background positioning area if the element is SVG, with the corresponding intrinsic proportion. This is non-standard behavior.

Based on the intrinsic dimensions and proportions, the rendered size of the background image is computed as follows:

- **If both components of `background-size` are specified and are not `auto`:** The background image is rendered at the specified size.
- **If the `background-size` is `contain` or `cover`:** While preserving its intrinsic proportions, the image is rendered at the largest size contained within, or covering, the background positioning area. If the image has no intrinsic proportions, then it's rendered at the size of the background positioning area.
- **If the `background-size` is `auto` or `auto auto`:**
  - If the image has both horizontal and vertical intrinsic dimensions, it's rendered at that size.
  - If the image has no intrinsic dimensions and has no intrinsic proportions, it's rendered at the size of the background positioning area.
  - If the image has no intrinsic dimensions but has intrinsic proportions, it's rendered as if `contain` had been specified instead.
  - If the image has only one intrinsic dimension and has intrinsic proportions, it's rendered at the size corresponding to that one dimension. The other dimension is computed using the specified dimension and the intrinsic proportions.
  - If the image has only one intrinsic dimension but has no intrinsic proportions, it's rendered using the specified dimension and the other dimension of the background positioning area.

  > **Note:** SVG images have a `preserveAspectRatio` attribute that defaults to the equivalent of `contain`; an explicit `background-size` causes `preserveAspectRatio` to be ignored.

- **If the `background-size` has one `auto` component and one non-`auto` component:**
  - If the image has intrinsic proportions, it's stretched to the specified dimension. The unspecified dimension is computed using the specified dimension and the intrinsic proportions.
  - If the image has no intrinsic proportions, it's stretched to the specified dimension. The unspecified dimension is computed using the image's corresponding intrinsic dimension, if there is one. If there is no such intrinsic dimension, it becomes the corresponding dimension of the background positioning area.

> **Note:** Background sizing for vector images that lack intrinsic dimensions or proportions is not yet fully implemented in all browsers. Be careful about relying on the behavior described above, and test in multiple browsers to be sure the results are acceptable.

Formal definition

Initial value	`auto auto`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	relative to the background positioning area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a repeatable list

Formal syntax

```
background-size =
  <bg-size>#

<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Tiling a large image

Let's consider a large image, a 2982x2808 Firefox logo image. We want to tile four copies of this image into a 300x300-pixel element. To do this, we can use a fixed `background-size` value of 150 pixels.

HTML

```
<div class="tiledBackground"></div>
```

CSS

```
.tiledBackground {
  background-image: url("https://www.mozilla.org/media/img/logos/firefox/logo-quantum.9c5e96634f92.png");
  background-size: 150px;
  width: 300px;
  height: 300px;
  border: 2px solid;
  color: pink;
}
```

Result

See Resizing background images for more examples.

See also

- Resizing background images
- Scaling SVG backgrounds
- `object-fit`

baseline-shift CSS property

The **`baseline-shift`** CSS property repositions the dominant-baseline of a text element relative to the dominant-baseline of its parent text content element. The shifted element might be a sub- or superscript. If the property is present, the value overrides the element's `baseline-shift` attribute.

> **Note:** The `baseline-shift` property only applies to `<textPath>` and `<tspan>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* <length-percentage> values */
baseline-shift: -0.5px;
baseline-shift: 4%;

/* keyword values */
baseline-shift: sub;
baseline-shift: super;

/* Global values */
baseline-shift: inherit;
baseline-shift: initial;
baseline-shift: revert;
baseline-shift: revert-layer;
baseline-shift: unset;
```

Values

`sub`

: The dominant-baseline is shifted to the default position for subscripts.

`super`

: The dominant-baseline is shifted to the default position for superscripts.

`<length-percentage>`

: Raises (if positive) or lowers (if negative) the dominant-baseline of the text content element by the specified length or percentage, with the percentage being relative to the dominant-baseline of the parent text content element's `line-height`.

Formal definition

Initial value	`0`
Applies to	inline-level boxes and SVG text content elements
Inherited	no
Percentages	refer to the used value of `line-height`
Computed value	the specified keyword or a computed value
Animation type	by computed value type

Formal syntax

```
baseline-shift =
  <length-percentage>  |
  sub                  |
  super                |
  top                  |
  center               |
  bottom
```

```
<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using keyword values

This example demonstrates the basic usage of the `sub` and `super` keyword values of the `baseline-shift` property, as well as how the `baseline-shift` CSS property takes precedence over the `baseline-shift` SVG attribute.

HTML

We define an SVG with two SVG `<text>` elements, each containing a `<tspan>` element with the SVG `baseline-shift` attribute set on it.

```
<p>Original SVG</p>
```

```
<svg viewBox="0 0 500 120" xmlns="http://www.w3.org/2000/svg">
  <text x="10" y="50">
    Ceci
    <tspan baseline-shift="super">n'est pas</tspan>
    une pipe super!
  </text>
  <text x="10" y="90">
    Ceci
    <tspan baseline-shift="sub">n'est pas</tspan>
    une pipe sub!
  </text>
</svg>
```

```
<p><code>baseline-shift: sub;</code></p>
<svg viewBox="0 0 500 120" xmlns="http://www.w3.org/2000/svg">
  <text x="10" y="50">
    Ceci
    <tspan baseline-shift="super">n'est pas</tspan>
    une pipe super!
  </text>
  <text x="10" y="90">
    Ceci
    <tspan baseline-shift="sub">n'est pas</tspan>
    une pipe sub!
  </text>
</svg>
<p><code>baseline-shift: super;</code></p>
<svg viewBox="0 0 500 120" xmlns="http://www.w3.org/2000/svg">
  <text x="10" y="50">
    Ceci
    <tspan baseline-shift="super">n'est pas</tspan>
    une pipe super!
  </text>
  <text x="10" y="90">
    Ceci
    <tspan baseline-shift="sub">n'est pas</tspan>
    une pipe sub!
  </text>
</svg>
```

The SVG is repeated three times; we've only shown one copy for the sake of brevity.

CSS

We make the text in all three SVG images large and cursive, adding some color to differentiate the content within the `<tspan>` elements.

We set the `baseline-shift` property value to `sub` on the second SVG's `<tspan>` element and `super` on the third SVG's `<tspan>` element. The first SVG has no additional CSS applied to it.

```
text {
  font-family: cursive;
  font-size: 2rem;
}
[baseline-shift="sub"] {
  fill: deeppink;
}
[baseline-shift="super"] {
  fill: rebeccapurple;
}

svg:nth-of-type(2) tspan {
  baseline-shift: sub;
}
svg:nth-of-type(3) tspan {
  baseline-shift: super;
}
```

```
svg {
  border: 1px solid;
  width: 15em;
  margin-bottom: 10px;
}
@supports not (baseline-shift: sub) {
  body::before {
    content: "Your browser doesn't support the `baseline-shift` property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Results

The SVG `baseline-shift` attribute values are used in the first SVG. In the second and third SVGs, the CSS `baseline-shift` values override the attribute values.

See also

- SVG `baseline-shift` attribute
- Presentation properties: `baseline-shift`, `clip-rule`, `color-interpolation-filters`, `fill-opacity`, `fill-rule`, `fill`, `marker-end`, `marker-mid`, `marker-start`, `shape-rendering`, `stop-color`, `stop-opacity`, `stroke`, `stroke-dasharray`, `stroke-dashoffset`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `stroke-opacity`, `stroke-width`, `text-anchor`, and `vector-effect`

baseline-source CSS property

The **`baseline-source`** CSS property defines which baseline to use when inline-level boxes have multiple possible baselines, such as multi-line inline blocks or inline flex containers. The values allow for choosing between aligning to the box's first baseline, last baseline, or letting the browser decide automatically based on the box type.

Syntax

```
/* Keyword values */
baseline-source: auto;
baseline-source: first;
baseline-source: last;

/* Global values */
baseline-source: inherit;
baseline-source: initial;
baseline-source: revert;
baseline-source: revert-layer;
baseline-source: unset;
```

Values

`auto`

Specifies `last baseline` alignment for inline-block, `first baseline` alignment for everything else.

`first`

Specifies `first baseline` alignment.

`last`

Specifies `last baseline` alignment.

Formal definition

Initial value	`auto`
Applies to	inline-level boxes
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
baseline-source =
  auto   |
  first  |
  last
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Baseline selection in inline flex containers

This example demonstrates using the `baseline-source` property to control the baseline alignment of inline flex containers.

HTML

Our HTML includes several `<span>` elements, which are generic inline containers for phrasing content. Three of the `<span>` elements contain nested `<span>` children.

```
<span>Baseline ___</span>

<span class="box first">
  <span>First</span>
  <span>A</span>
  <span>B</span>
  <span>C</span>
</span>

<span class="box auto">
  <span>Auto</span>
  <span>A</span>
  <span>B</span>
  <span>C</span>
</span>

<span class="box last">
  <span>A</span>
  <span>B</span>
  <span>C</span>
  <span>Last</span>
</span>
```

CSS

```
body {
  font-family: sans-serif;
}
```

```
.box {
  border: 2px solid #888888;
  width: 50px;
}

span {
  padding: 0.4rem;
}
```

We define all the boxes to be inline flex containers. We set the `.first` box to use the first baseline, the `.auto` box uses the default baseline (which is `first` for inline flex containers), and the `.last` box uses the last baseline.

```
.box {
  display: inline-flex;
  flex-direction: column;
}

.first {
  baseline-source: first;
}

.auto {
  baseline-source: auto;
}

.last {
  baseline-source: last;
}
```

Result

See also

- `vertical-align` property
- CSS box alignment overview

block-size CSS property

The **`block-size`** CSS property defines the size of an element's block along the block axis. If the `writing-mode` is horizontal, it corresponds to the `height`; if the writing mode is vertical, it corresponds to the `width`. A related property is `inline-size`, which defines the other dimension of the element.

Try it

```
block-size: 150px;
writing-mode: horizontal-tb;
```

```
block-size: 150px;
writing-mode: vertical-rl;
```

```
block-size: auto;
writing-mode: horizontal-tb;
```

```
block-size: auto;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the block-size.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> values */
block-size: 300px;
block-size: 25em;
block-size: anchor-size(height);
block-size: calc(anchor-size(--my-anchor block) * 0.75);

/* <percentage> values */
block-size: 75%;

/* Keyword values */
block-size: max-content;
block-size: min-content;
block-size: fit-content;
block-size: fit-content(20em);
block-size: auto;

/* Global values */
block-size: inherit;
block-size: initial;
block-size: revert;
block-size: revert-layer;
block-size: unset;
```

Values

The `block-size` property takes the same values as the `width` and `height` properties.

Formal definition

Initial value	`auto`
Applies to	same as `width` and `height`
Inherited	no
Percentages	block-size of containing block
Computed value	same as `width` and `height`
Animation type	a length, percentage or calc();

Formal syntax

```
block-size =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Block size with vertical text

HTML

```
<p class="exampleText">Example text</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  block-size: 200px;
}
```

Result

See also

- The mapped physical properties: `width` and `height`
- `writing-mode`

border CSS property

The **`border`** CSS shorthand property sets an element's border. It sets the values of `border-width`, `border-style`, and `border-color`.

Try it

```
border: solid;
```

```
border: dashed red;
```

```
border: 1rem solid;
```

```
border: thick double #32a1ce;
```

```
border: 4mm ridge rgb(211 220 50 / 0.6);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-width`
- `border-style`
- `border-color`

Reset-only sub-properties

This property resets the following CSS properties to their initial values:

- `border-image-outset`
- `border-image-repeat`
- `border-image-slice`
- `border-image-source`
- `border-image-width`

Syntax

```
/* style */
border: solid;

/* width | style */
border: 2px dotted;

/* style | color */
border: outset #ff3333;

/* width | style | color */
border: medium dashed green;

/* Global values */
border: inherit;
border: initial;
border: revert;
border: revert-layer;
border: unset;
```

Values

This property is specified as a space-separated list of one to three of the following values:

`<line-width>`

Sets the thickness of the border. Defaults to `medium` if absent. See `border-width`.

`<line-style>`

Sets the style of the border. Defaults to `none` if absent. See `border-style`.

`<color>`

Sets the color of the border. Defaults to `currentColor` if absent. See `border-color`.

Description

The `border` shorthand property sets the width, style, and color of all four sides of an element's border. It is specified using one, two, or all three of the component property values, in any order.

Any omitted sub-values are set to their initial value. For a border to be displayed, the `<line-style>` component must be set because the style defaults to `none`.

Importantly, `border` cannot be used to specify a custom value for `border-image`, but instead sets it to its initial value, i.e., `none`.

The `border` shorthand is especially useful when you want all four borders to be the same. To make them different from each other, however, you can use the longhand `border-width`, `border-style`, and `border-color` properties, which accept different values for each side. Alternatively, you can target one border at a time with the physical (e.g., `border-top`) and logical (e.g., `border-block-start`) border properties.

Borders vs. outlines

Borders and outlines are very similar. However, outlines differ from borders in the following ways:

- Outlines never take up space, as they are drawn outside of an element's content.
- According to the spec, outlines don't have to be rectangular, although they usually are.

Formal definition

Initial value	as each of the properties of the shorthand:
	• `border-width`: as each of the properties of the shorthand:
	  ◦ `border-top-width`: `medium`
	  ◦ `border-right-width`: `medium`
	  ◦ `border-bottom-width`: `medium`
	  ◦ `border-left-width`: `medium`
	• `border-style`: as each of the properties of the shorthand:
	  ◦ `border-top-style`: `none`
	  ◦ `border-right-style`: `none`
	  ◦ `border-bottom-style`: `none`
	  ◦ `border-left-style`: `none`
	• `border-color`: as each of the properties of the shorthand:
	  ◦ `border-top-color`: `currentcolor`
	  ◦ `border-right-color`: `currentcolor`
	  ◦ `border-bottom-color`: `currentcolor`
	  ◦ `border-left-color`: `currentcolor`

Applies to	all elements. It also applies to `::first-letter`.
Inherited	no

Computed value	as each of the properties of the shorthand:
	• `border-width`: as each of the properties of the shorthand:
	  ◦ `border-top-width`: the absolute `<length>`, snapped as a line width
	  ◦ `border-right-width`: the absolute `<length>`, snapped as a line width
	  ◦ `border-bottom-width`: the absolute `<length>`, snapped as a line width
	  ◦ `border-left-width`: the absolute `<length>`, snapped as a line width
	• `border-style`: as each of the properties of the shorthand:
	  ◦ `border-top-style`: as specified
	  ◦ `border-right-style`: as specified
	  ◦ `border-bottom-style`: as specified
	  ◦ `border-left-style`: as specified
	• `border-color`: as each of the properties of the shorthand:
	  ◦ `border-top-color`: computed color
	  ◦ `border-right-color`: computed color
	  ◦ `border-bottom-color`: computed color
	  ◦ `border-left-color`: computed color

Animation type	as each of the properties of the shorthand:
	• `border-width`: as each of the properties of the shorthand:
	  ◦ `border-top-width`: a length
	  ◦ `border-right-width`: a length
	  ◦ `border-bottom-width`: a length
	  ◦ `border-left-width`: a length
	• `border-style`: discrete
	• `border-color`: as each of the properties of the shorthand:
	  ◦ `border-top-color`: a color
	  ◦ `border-right-color`: a color
	  ◦ `border-bottom-color`: a color
	  ◦ `border-left-color`: a color

Formal syntax

```
border =
  <line-width> ||
  <line-style> ||
  <color>

<line-width> =
  <length [0,∞]> |
  hairline       |
  thin           |
  medium         |
  thick

<line-style> =
  none   |
  hidden |
  dotted |
  dashed |
  solid  |
  double |
  groove |
  ridge  |
  inset  |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a pink outset border

HTML

```
<div>I have a border, an outline, and a box shadow! Amazing, isn't it?</div>
```

CSS

```
div {
  border: 0.5rem outset pink;
  outline: 0.5rem solid khaki;
  box-shadow: 0 0 0 2rem skyblue;
  border-radius: 12px;
  font: bold 1rem sans-serif;
  margin: 2rem;
  padding: 1rem;
  outline-offset: 0.5rem;
}
```

Result

See also

- `border-width`
- `border-style`
- `border-color`
- `outline`
- Backgrounds and borders
- Learn CSS: Backgrounds and borders

border-block CSS property

The **`border-block`** CSS shorthand property sets the individual logical block border property values in a single place in the style sheet.

Try it

```
border-block: solid;
writing-mode: horizontal-tb;
```

```
border-block: dashed red;
writing-mode: vertical-rl;
```

```
border-block: 1rem solid;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-block-color`
- `border-block-style`
- `border-block-width`

Syntax

```
border-block: 1px;
border-block: 2px dotted;
border-block: medium dashed blue;

/* Global values */
border-block: inherit;
border-block: initial;
border-block: revert;
border-block: revert-layer;
border-block: unset;
```

Values

The `border-block` is specified with one or more of the following, in any order:

`<line-width>`

The width of the border. See `border-width`.

`<line-style>`

The line style of the border. See `border-style`.

`<color>`

The color of the border.

Description

`border-block` can be used to set the values for one or more of `border-block-width`, `border-block-style`, and `border-block-color` setting both the start and end in the block dimension at once. The physical borders to which it maps depends on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top` and `border-bottom` or `border-right`, and `border-left` properties depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The borders in the other dimension can be set with `border-inline`, which sets `border-inline-start`, and `border-inline-end`.

Formal definition

Initial value	as each of the properties of the shorthand:
	• `border-block-width`: as each of the properties of the shorthand:
	  ◦ `border-block-start-width`: `medium`
	  ◦ `border-block-end-width`: `medium`
	• `border-block-style`: as each of the properties of the shorthand:
	  ◦ `border-block-start-style`: `none`
	  ◦ `border-block-end-style`: `none`
	• `border-block-color`: as each of the properties of the shorthand:
	  ◦ `border-block-start-color`: `currentcolor`
	  ◦ `border-block-end-color`: `currentcolor`

Applies to	all elements
Inherited	no

Computed value	as each of the properties of the shorthand:
	• `border-block-width`: as each of the properties of the shorthand:
	  ◦ `border-block-start-width`: the absolute `<length>`, snapped as a line width
	  ◦ `border-block-end-width`: the absolute `<length>`, snapped as a line width
	• `border-block-style`: as each of the properties of the shorthand:
	  ◦ `border-block-start-style`: as specified
	  ◦ `border-block-end-style`: as specified
	• `border-block-color`: as each of the properties of the shorthand:
	  ◦ `border-block-start-color`: computed color
	  ◦ `border-block-end-color`: computed color

Animation type	as each of the properties of the shorthand:
	• `border-block-width`: as each of the properties of the shorthand:
	  ◦ `border-block-start-width`: by computed value type
	  ◦ `border-block-end-width`: by computed value type
	• `border-block-style`: discrete
	• `border-block-color`: as each of the properties of the shorthand:
	  ◦ `border-block-start-color`: by computed value type
	  ◦ `border-block-end-color`: by computed value type

Formal syntax

```
border-block =
  <'border-block-start'>

<border-block-start> =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}
```

```
.exampleText {
  writing-mode: vertical-rl;
  border-block: 5px dashed blue;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top`, `border-right`, `border-bottom`, or `border-left`.
- `writing-mode`, `direction`, `text-orientation`

border-block-color CSS property

The **`border-block-color`** CSS property defines the color of the logical block borders of an element, which maps to a physical border color depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-color` and `border-bottom-color`, or `border-right-color` and `border-left-color` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The border color in the other dimension can be set with `border-inline-color` which sets `border-inline-start-color`, and `border-inline-end-color`.

Try it

```
border-block-color: red;
writing-mode: horizontal-tb;
```

```
border-block-color: #32a1ce;
writing-mode: vertical-rl;
```

```
border-block-color: rgb(170 50 220 / 0.6);
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
border-block-color: yellow;
border-block-color: #f5f6f7;

/* Global values */
border-block-color: inherit;
border-block-color: initial;
border-block-color: revert;
border-block-color: revert-layer;
border-block-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

> The color of the border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-block-start-color`: `currentcolor` • `border-block-end-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-block-start-color`: computed color • `border-block-end-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-block-start-color`: by computed value type • `border-block-end-color`: by computed value type

Formal syntax

```
border-block-color =
  <'border-top-color'>{1,2}

<border-top-color> =
```

```
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 10px solid blue;
  border-block-color: red;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to the physical border properties: `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color`.
- `writing-mode`, `direction`, `text-orientation`

border-block-end CSS property

The **`border-block-end`** CSS shorthand property sets the individual logical block-end border property values in a single place in the style sheet.

Try it

```
border-block-end: solid;
writing-mode: horizontal-tb;
```

```
border-block-end: dashed red;
writing-mode: vertical-rl;
```

```
border-block-end: 1rem solid;
writing-mode: horizontal-tb;
```

```
border-block-end: thick double #32a1ce;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-block-end-color`
- `border-block-end-style`
- `border-block-end-width`

Syntax

```
border-block-end: 1px;
border-block-end: 2px dotted;
border-block-end: medium dashed blue;
```

```
/* Global values */
border-block-end: inherit;
border-block-end: initial;
border-block-end: revert;
border-block-end: revert-layer;
border-block-end: unset;
```

`border-block-end` can be used to set the values for one or more of `border-block-end-width`, `border-block-end-style`, and `border-block-end-color`. The physical border to which it maps depends on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top`, `border-right`, `border-bottom`, or `border-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Related properties are `border-block-start`, `border-inline-start`, and `border-inline-end`, which define the other borders of the element.

Values

The `border-block-end` is specified with one or more of the following, in any order:

`<line-width>`

The width of the border. See `border-width`.

`<line-style>`

The line style of the border. See `border-style`.

`<color>`

The color of the border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-block-end-width`: `medium` • `border-block-end-style`: `none` • `border-block-end-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-block-end-width`: the absolute `<length>`, snapped as a line width • `border-block-end-style`: as specified • `border-block-end-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-block-end-width`: by computed value type • `border-block-end-style`: discrete • `border-block-end-color`: by computed value type

Formal syntax

```
border-block-end =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
```

```
  border-block-end: 5px dashed blue;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top`, `border-right`, `border-bottom`, or `border-left`.
- `writing-mode`, `direction`, `text-orientation`

border-block-end-color CSS property

The **`border-block-end-color`** CSS property defines the color of the logical block-end border of an element, which maps to a physical border color depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-block-end-color: red;
writing-mode: horizontal-tb;
```

```
border-block-end-color: #32a1ce;
writing-mode: vertical-rl;
```

```
border-block-end-color: rgb(170 50 220 / 0.6);
writing-mode: horizontal-tb;
```

```
border-block-end-color: hsl(60 90% 50% / 0.8);
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
border-block-end-color: yellow;
border-block-end-color: #f5f6f7;

/* Global values */
border-block-end-color: inherit;
border-block-end-color: initial;
border-block-end-color: revert;
border-block-end-color: revert-layer;
border-block-end-color: unset;
```

Related properties are `border-block-start-color`, `border-inline-start-color`, and `border-inline-end-color`, which define the other border colors of the element.

Values

`<color>`

The color of the border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	no
Computed value	computed color
Animation type	by computed value type

Formal syntax

```
border-block-end-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border color with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 10px solid blue;
  border-block-end-color: red;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color`.
- `writing-mode`, `direction`, `text-orientation`

border-block-end-style CSS property

The **`border-block-end-style`** CSS property defines the style of the logical block-end border of an element, which maps to a physical border style depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-block-end-style: dotted;
writing-mode: horizontal-tb;
```

```
border-block-end-style: dotted;
writing-mode: vertical-rl;
```

```
border-block-end-style: groove;
writing-mode: horizontal-tb;
```

```
border-block-end-style: dashed;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* Keyword values */
border-block-end-style: dashed;
border-block-end-style: dotted;
border-block-end-style: groove;

/* Global values */
border-block-end-style: inherit;
border-block-end-style: initial;
border-block-end-style: revert;
border-block-end-style: revert-layer;
border-block-end-style: unset;
```

Related properties are `border-block-start-style`, `border-inline-start-style`, and `border-inline-end-style`, which define the other border styles of the element.

Values

`<line-style>`

The line style of the border. See `border-style`.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified

Animation type	discrete

Formal syntax

```
border-block-end-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Dashed border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 5px solid blue;
  border-block-end-style: dashed;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-style`, `border-right-style`, `border-bottom-style`, and `border-left-style`.
- `writing-mode`, `direction`, `text-orientation`

border-block-end-width CSS property

The **`border-block-end-width`** CSS property defines the width of the logical block-end border of an element, which maps to a physical border width depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-width`, `border-right-width`, `border-bottom-width`, or `border-left-width` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-block-end-width: thick;
writing-mode: horizontal-tb;
```

```
border-block-end-width: thick;
writing-mode: vertical-rl;
```

```
border-block-end-width: 4px;
writing-mode: horizontal-tb;
```

```
border-block-end-width: 4px;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <'line-width'> values */
border-block-end-width: 5px;
border-block-end-width: thick;

/* Global values */
border-block-end-width: inherit;
border-block-end-width: initial;
border-block-end-width: revert;
```

```
border-block-end-width: revert-layer;
border-block-end-width: unset;
```

Related properties are `border-block-start-width`, `border-inline-start-width`, and `border-inline-end-width`, which define the other border widths of the element.

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	by computed value type

Formal syntax

```
border-block-end-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border width with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 1px solid blue;
  border-block-end-width: 5px;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-left-width`
- `writing-mode`, `direction`, `text-orientation`

border-block-start CSS property

The **`border-block-start`** CSS shorthand property sets the individual logical block-start border property values in a single place in the style sheet.

Try it

```
border-block-start: solid;
writing-mode: horizontal-tb;
```

```
border-block-start: dashed red;
writing-mode: vertical-rl;
```

```
border-block-start: 1rem solid;
writing-mode: horizontal-tb;
```

```
border-block-start: thick double #32a1ce;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
```

```
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-block-start-color`
- `border-block-start-style`
- `border-block-start-width`

Syntax

```
border-block-start: 1px;
border-block-start: 2px dotted;
border-block-start: medium dashed blue;

/* Global values */
border-block-start: inherit;
border-block-start: initial;
border-block-start: revert;
border-block-start: revert-layer;
border-block-start: unset;
```

`border-block-start` can be used to set the values for one or more of `border-block-start-width`, `border-block-start-style`, and `border-block-start-color`. The physical border to which it maps depends on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top`, `border-right`, `border-bottom`, or `border-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Related properties are `border-block-end`, `border-inline-start`, and `border-inline-end`, which define the other borders of the element.

Values

The `border-block-start` is specified with one or more of the following, in any order:

`<line-width>`

The width of the border. See `border-width`.

`<line-style>`

The line style of the border. See `border-style`.

`<color>`

The color of the border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-block-start-width`: `medium` • `border-block-start-style`: `none` • `border-block-start-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-block-start-width`: the absolute `<length>`, snapped as a line width • `border-block-start-style`: as specified • `border-block-start-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-block-start-width`: by computed value type • `border-block-start-style`: discrete • `border-block-start-color`: by computed value type

Formal syntax

```
border-block-start =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  border-block-start: 5px dashed blue;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top`, `border-right`, `border-bottom`, or `border-left`.
- `writing-mode`, `direction`, `text-orientation`

border-block-start-color CSS property

The **`border-block-start-color`** CSS property defines the color of the logical block-start border of an element, which maps to a physical border color depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-block-start-color: red;
writing-mode: horizontal-tb;
```

```
border-block-start-color: #32a1ce;
writing-mode: vertical-rl;
```

```
border-block-start-color: rgb(170 50 220 / 0.6);
writing-mode: horizontal-tb;
```

```
border-block-start-color: hsl(60 90% 50% / 0.8);
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
border-block-start-color: blue;
border-block-start-color: #4c5d21;

/* Global values */
border-block-start-color: inherit;
border-block-start-color: initial;
border-block-start-color: revert;
border-block-start-color: revert-layer;
border-block-start-color: unset;
```

Related properties are `border-block-end-color`, `border-inline-start-color`, and `border-inline-end-color`, which define the other border colors of the element.

Values

`<color>`

The color of the border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	no
Computed value	computed color
Animation type	by computed value type

Formal syntax

```
border-block-start-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border color with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 10px solid blue;
  border-block-start-color: red;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color`.
- `writing-mode`, `direction`, `text-orientation`

border-block-start-style CSS property

The **`border-block-start-style`** CSS property defines the style of the logical block start border of an element, which maps to a physical border style depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-block-start-style: dotted;
writing-mode: horizontal-tb;
```

```
border-block-start-style: dotted;
writing-mode: vertical-rl;
```

```
border-block-start-style: groove;
writing-mode: horizontal-tb;
```

```
border-block-start-style: dashed;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* Keyword values */
border-block-start-style: dashed;
border-block-start-style: dotted;
border-block-start-style: groove;

/* Global values */
```

```
border-block-start-style: inherit;
border-block-start-style: initial;
border-block-start-style: revert;
border-block-start-style: revert-layer;
border-block-start-style: unset;
```

Related properties are `border-block-end-style`, `border-inline-start-style`, and `border-inline-end-style`, which define the other border styles of the element.

Values

`<line-style>`

The line style of the border. See `border-style`.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-block-start-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Dashed border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 5px solid blue;
  border-block-start-style: dashed;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style`.
- `writing-mode`, `direction`, `text-orientation`

border-block-start-width CSS property

The **`border-block-start-width`** CSS property defines the width of the logical block-start border of an element, which maps to a physical border width depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-width`, `border-right-width`, `border-bottom-width`, or `border-left-width` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-block-start-width: thick;
writing-mode: horizontal-tb;
```

```
border-block-start-width: thick;
writing-mode: vertical-rl;
```

```
border-block-start-width: 4px;
writing-mode: horizontal-tb;
```

```
border-block-start-width: 4px;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
```

```
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <'line-width'> values */
border-block-start-width: 5px;
border-block-start-width: thick;

/* Global values */
border-block-start-width: inherit;
border-block-start-width: initial;
border-block-start-width: revert;
border-block-start-width: revert-layer;
border-block-start-width: unset;
```

Related properties are `border-block-end-width`, `border-inline-start-width`, and `border-inline-end-width`, which define the other border widths of the element.

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	by computed value type

Formal syntax

```
border-block-start-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border width with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 1px solid blue;
  border-block-start-width: 5px;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-left-width`
- `writing-mode`, `direction`, `text-orientation`

border-block-style CSS property

The **`border-block-style`** CSS property defines the style of the logical block borders of an element, which maps to a physical border style depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-style` and `border-bottom-style`, or `border-left-style` and `border-right-style` properties depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The border style in the other dimension can be set with `border-inline-style`, which sets `border-inline-start-style`, and `border-inline-end-style`.

Try it

```
border-block-style: dotted;
writing-mode: horizontal-tb;
```

```
border-block-style: dotted;
writing-mode: vertical-rl;
```

```
border-block-style: groove;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* Keyword values */
border-block-style: dashed;
border-block-style: dotted;
border-block-style: groove;

/* Global values */
border-block-style: inherit;
border-block-style: initial;
border-block-style: revert;
border-block-style: revert-layer;
border-block-style: unset;
```

Values

`<line-style>`

The line style of the border. See `border-style`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-block-start-style`: none • `border-block-end-style`: none
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-block-start-style`: as specified • `border-block-end-style`: as specified
Animation type	discrete

Formal syntax

```
border-block-style =
  <'border-top-style'>{1,2}

<border-top-style> =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Dashed border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 5px solid blue;
  border-block-style: dashed;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style`.
- `writing-mode`, `direction`, `text-orientation`

border-block-width CSS property

The **`border-block-width`** CSS property defines the width of the logical block borders of an element, which maps to a physical border width depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-width` and `border-bottom-width`, or `border-left-width`, and `border-right-width` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The border width in the other dimension can be set with `border-inline-width`, which sets `border-inline-start-width`, and `border-inline-end-width`.

Try it

```
border-block-width: thick;
writing-mode: horizontal-tb;
```

```
border-block-width: thick;
writing-mode: vertical-rl;
```

```
border-block-width: 4px;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <'line-width'> values */
border-block-width: 5px;
border-block-width: thick;

/* Global values */
border-block-width: inherit;
border-block-width: initial;
border-block-width: revert;
border-block-width: revert-layer;
border-block-width: unset;
```

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-block-start-width`: `medium` • `border-block-end-width`: `medium`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-block-start-width`: the absolute `<length>`, snapped as a line width • `border-block-end-width`: the absolute `<length>`, snapped as a line width
Animation type	as each of the properties of the shorthand: • `border-block-start-width`: by computed value type • `border-block-end-width`: by computed value type

Formal syntax

```
border-block-width =
  <'border-top-width'>{1,2}

<border-top-width> =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border width with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 1px solid blue;
  border-block-width: 5px;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-left-width`
- `writing-mode`, `direction`, `text-orientation`

border-bottom CSS property

The **`border-bottom`** CSS shorthand property sets an element's bottom border. It sets the values of `border-bottom-width`, `border-bottom-style` and `border-bottom-color`.

Try it

```
border-bottom: solid;
```

```
border-bottom: dashed red;
```

```
border-bottom: 1rem solid;
```

```
border-bottom: thick double #32a1ce;
```

```
border-bottom: 4mm ridge rgb(211 220 50 / 0.6);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-bottom-color`
- `border-bottom-style`
- `border-bottom-width`

Syntax

```
border-bottom: 1px;
border-bottom: 2px dotted;
border-bottom: medium dashed blue;

/* Global values */
border-bottom: inherit;
border-bottom: initial;
border-bottom: revert;
```

```
border-bottom: revert-layer;
border-bottom: unset;
```

The three values of the shorthand property can be specified in any order, and one or two of them may be omitted.

Values

`<br-width>`

See `border-bottom-width`.

`<br-style>`

See `border-bottom-style`.

`<color>`

See `border-bottom-color`.

Description

As with all shorthand properties, `border-bottom` always sets the values of all of the properties that it can set, even if they are not specified. It sets those that are not specified to their default values. Consider the following code:

```
border-bottom-style: dotted;
border-bottom: thick green;
```

It is actually the same as this one:

```
border-bottom-style: dotted;
border-bottom: none thick green;
```

The value of `border-bottom-style` given before `border-bottom` is ignored. Since the default value of `border-bottom-style` is `none`, not specifying the `border-style` part results in no border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-bottom-width`: `medium` • `border-bottom-style`: `none` • `border-bottom-color`: `currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-bottom-width`: the absolute `<length>`, snapped as a line width • `border-bottom-style`: as specified • `border-bottom-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-bottom-width`: a length • `border-bottom-style`: discrete • `border-bottom-color`: a color

Formal syntax

```
border-bottom =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying a bottom border

HTML

```
<div>This box has a border on the bottom side.</div>
```

CSS

```
div {
  border-bottom: 4px dashed blue;
  background-color: gold;
  height: 100px;
```

```
  width: 100px;
  font-weight: bold;
  text-align: center;
}
```

Results

See also

- `border`
- `border-block`
- `outline`

border-bottom-color CSS property

The **`border-bottom-color`** CSS property sets the color of an element's bottom border. It can also be set with the shorthand CSS properties `border-color` or `border-bottom`.

Try it

```
border-bottom-color: red;
```

```
border-bottom-color: #32a1ce;
```

```
border-bottom-color: rgb(170 50 220 / 0.6);
```

```
border-bottom-color: hsl(60 90% 50% / 0.8);
```

```
border-bottom-color: transparent;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* <color> values */
border-bottom-color: red;
border-bottom-color: #ffbb00;
border-bottom-color: rgb(255 0 0);
border-bottom-color: hsl(100deg 50% 25% / 75%);
border-bottom-color: currentColor;
border-bottom-color: transparent;

/* Global values */
border-bottom-color: inherit;
border-bottom-color: initial;
border-bottom-color: revert;
border-bottom-color: revert-layer;
border-bottom-color: unset;
```

Values

This property is specified as one value:

`<color>`

The color of the bottom border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
border-bottom-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A div with a border

HTML

```
<div class="my-box">
  <p>
    This is a box with a border around it. Note which side of the box is
    <span class="red-text">red</span>.
  </p>
</div>
```

CSS

```
.my-box {
  border: solid 0.3em gold;
  border-bottom-color: red;
  width: auto;
}

.red-text {
  color: red;
}
```

Result

See also

- The border-related CSS shorthand properties: `border`, `border-bottom`, and `border-color`.
- The color-related CSS properties for the other borders: `border-right-color`, `border-top-color`, and `border-left-color`.
- The other border-related CSS properties applying to the same border: `border-bottom-style` and `border-bottom-width`.
- The default `currentColor` color value.

border-bottom-left-radius CSS property

The **`border-bottom-left-radius`** CSS property rounds the bottom-left corner of an element by specifying the radius (or the radius of the semi-major and semi-minor axes) of the ellipse defining the curvature of the corner.

Try it

```
border-bottom-left-radius: 80px 80px;
```

```
border-bottom-left-radius: 250px 100px;
```

```
border-bottom-left-radius: 50%;
```

```
border-bottom-left-radius: 50%;
border: black 10px double;
background-clip: content-box;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a bottom left rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* the corner is a circle */
/* border-bottom-left-radius: radius */
border-bottom-left-radius: 3px;

/* Percentage values */

/* circle if box is a square or ellipse if box is a rectangle */
border-bottom-left-radius: 20%;

/* same as above: 20% of horizontal(width) and vertical(height) */
border-bottom-left-radius: 20% 20%;

/* 20% of horizontal(width) and 10% of vertical(height) */
border-bottom-left-radius: 20% 10%;

/* the corner is an ellipse */
/* border-bottom-left-radius: horizontal vertical */
border-bottom-left-radius: 0.5em 1em;

/* Global values */
border-bottom-left-radius: inherit;
border-bottom-left-radius: initial;
```

```
border-bottom-left-radius: revert;
border-bottom-left-radius: revert-layer;
border-bottom-left-radius: unset;
```

With one value:

- the value is a `<length>` or a `<percentage>` denoting the radius of the circle to use for the border in that corner.

With two values:

- the first value is a `<length>` or a `<percentage>` denoting the horizontal semi-major axis of the ellipse to use for the border in that corner.
- the second value is a `<length>` or a `<percentage>` denoting the vertical semi-major axis of the ellipse to use for the border in that corner.

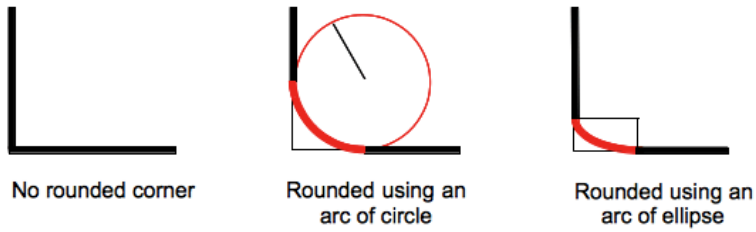
Values

`<length-percentage>`

Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Description

The rounding can be a circle or an ellipse, or if one of the values is `0` no rounding is done and the corner is square.



A background, being an image or a color, is clipped at the border, even a rounded one; the exact location of the clipping is defined by the value of the `background-clip` property.

> **Note:** If the value of this property is not set in a `border-radius` shorthand property that is applied to the element after the `border-bottom-left-radius` CSS property, the value of this property is then reset to its initial value by the shorthand property.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-bottom-left-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Arc of a circle

A single `<length>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-bottom-left-radius: 40px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Arc of an ellipse

Two different `<length>` values produce an arc of an ellipse.

```
<div></div>
```

```
div {
  border-bottom-left-radius: 40px 20px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Square element with percentage radius

A square element with a single `<percentage>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-bottom-left-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Non-square element with percentage radius

A non-square element with a single `<percentage>` value produces an arc of an ellipse.

```
<div></div>
```

```
div {
  border-bottom-left-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 200px;
  height: 100px;
}
```

See also

- `border-radius` shorthand property
- `border-top-right-radius`, `border-bottom-right-radius`, and `border-top-left-radius`

border-bottom-right-radius CSS property

The **`border-bottom-right-radius`** CSS property rounds the bottom-right corner of an element by specifying the radius (or the radius of the semi-major and semi-minor axes) of the ellipse defining the curvature of the corner.

Try it

```
border-bottom-right-radius: 80px 80px;
```

```
border-bottom-right-radius: 250px 100px;
```

```
border-bottom-right-radius: 50%;
```

```
border-bottom-right-radius: 50%;
border: black 10px double;
background-clip: content-box;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a bottom right rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* The corner is a circle */
/* border-bottom-right-radius: radius */
border-bottom-right-radius: 3px;

/* Percentage values */
border-bottom-right-radius: 20%; /* corner of a circle if box is a square or else corner of a rectangle */
border-bottom-right-radius: 20% 20%; /* same as above */ /* 20% of horizontal(width) and vertical(height) */
border-bottom-right-radius: 20% 10%; /* 20% of horizontal(width) and 10% of vertical(height) */

/*The corner is an ellipse */
/* border-bottom-right-radius: horizontal vertical */
border-bottom-right-radius: 0.5em 1em;

/* Global values */
border-bottom-right-radius: inherit;
border-bottom-right-radius: initial;
border-bottom-right-radius: revert;
```

```
border-bottom-right-radius: revert-layer;
border-bottom-right-radius: unset;
```

With one value:

- the value is a `<length>` or a `<percentage>` denoting the radius of the circle to use for the border in that corner.

With two values:

- the first value is a `<length>` or a `<percentage>` denoting the horizontal semi-major axis of the ellipse to use for the border in that corner.
- the second value is a `<length>` or a `<percentage>` denoting the vertical semi-major axis of the ellipse to use for the border in that corner.

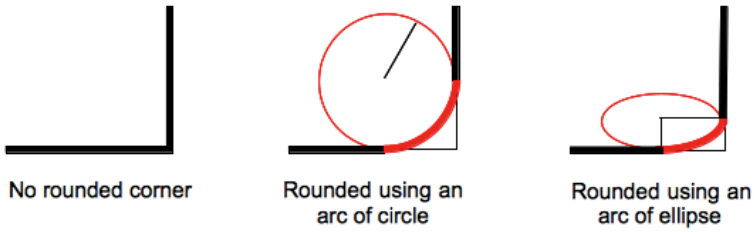
Values

`<length-percentage>`

> Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Description

The rounding can be a circle or an ellipse, or if one of the values is `0` no rounding is done and the corner is square.



A background, being an image or a color, is clipped at the border, even a rounded one; the exact location of the clipping is defined by the value of the `background-clip` property.

> **Note:** If the value of this property is not set in a `border-radius` shorthand property that is applied to the element after the `border-bottom-right-radius` CSS property, the value of this property is then reset to its initial value by the shorthand property.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-bottom-right-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Arc of a circle

A single `<length>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-bottom-right-radius: 40px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Arc of an ellipse

Two different `<length>` values produce an arc of an ellipse.

```
<div></div>
```

```
div {
  border-bottom-right-radius: 40px 20px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Square element with percentage radius

A square element with a single `<percentage>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-bottom-right-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Non-square element with percentage radius

A non-square element with a single `<percentage>` value produces an arc of an ellipse.

```
<div></div>
```

```
div {
  border-bottom-right-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 200px;
  height: 100px;
}
```

See also

- `border-radius` shorthand property
- `border-top-right-radius`, `border-bottom-left-radius`, and `border-top-left-radius`

border-bottom-style CSS property

The **`border-bottom-style`** CSS property sets the line style of an element's bottom `border`.

> **Note:** The specification doesn't define how borders of different styles connect in the corners.

Try it

```
border-bottom-style: none;
```

```
border-bottom-style: dotted;
```

```
border-bottom-style: dashed;
```

```
border-bottom-style: solid;
```

```
border-bottom-style: groove;
```

```
border-bottom-style: inset;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}

body {
  background-color: white;
}
```

Syntax

```
/* Keyword values */
border-bottom-style: none;
border-bottom-style: hidden;
border-bottom-style: dotted;
border-bottom-style: dashed;
border-bottom-style: solid;
border-bottom-style: double;
border-bottom-style: groove;
border-bottom-style: ridge;
```

```
border-bottom-style: inset;
border-bottom-style: outset;

/* Global values */
border-bottom-style: inherit;
border-bottom-style: initial;
border-bottom-style: revert;
border-bottom-style: revert-layer;
border-bottom-style: unset;
```

The `border-bottom-style` property is specified as a single `<line-style>` keyword value.

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-bottom-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Demonstrating all border styles

HTML

```
<table>
  <tbody>
    <tr>
      <td class="b1">none</td>
      <td class="b2">hidden</td>
      <td class="b3">dotted</td>
      <td class="b4">dashed</td>
    </tr>
    <tr>
      <td class="b5">solid</td>
      <td class="b6">double</td>
      <td class="b7">groove</td>
      <td class="b8">ridge</td>
    </tr>
    <tr>
      <td class="b9">inset</td>
      <td class="b10">outset</td>
    </tr>
  </tbody>
</table>
```

CSS

```
/* Define look of the table */
table {
  border-width: 3px;
  background-color: #52e385;
}
tr,
td {
  padding: 3px;
}

/* border-bottom-style example classes */
.b1 {
  border-bottom-style: none;
}
.b2 {
  border-bottom-style: hidden;
}
.b3 {
  border-bottom-style: dotted;
}
.b4 {
  border-bottom-style: dashed;
}
.b5 {
  border-bottom-style: solid;
}
.b6 {
  border-bottom-style: double;
}
.b7 {
  border-bottom-style: groove;
}
```

```
.b8 {
  border-bottom-style: ridge;
}
.b9 {
  border-bottom-style: inset;
}
.b10 {
  border-bottom-style: outset;
}
```

Result

See also

- The other style-related border properties: `border-left-style`, `border-right-style`, `border-top-style`, and `border-style`.
- The other bottom-border-related properties: `border-bottom`, `border-bottom-color`, and `border-bottom-width`.

border-bottom-width CSS property

The **`border-bottom-width`** CSS property sets the width of the bottom border of an element.

Try it

```
border-bottom-width: thick;
```

```
border-bottom-width: 2em;
```

```
border-bottom-width: 4px;
```

```
border-bottom-width: 2ex;
```

```
border-bottom-width: 0;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* Keyword values */
border-bottom-width: thin;
border-bottom-width: medium;
border-bottom-width: thick;

/* <length> values */
border-bottom-width: 10em;
border-bottom-width: 3vmax;
border-bottom-width: 6px;

/* Global keywords */
border-bottom-width: inherit;
border-bottom-width: initial;
border-bottom-width: revert;
border-bottom-width: revert-layer;
border-bottom-width: unset;
```

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
border-bottom-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing bottom border widths

HTML

```
<div>Element 1</div>
<div>Element 2</div>
```

CSS

```
div {
  border: 1px solid red;
  margin: 1em 0;
}

div:nth-child(1) {
  border-bottom-width: thick;
}
div:nth-child(2) {
  border-bottom-width: 2em;
}
```

Result

See also

- The other border-width-related CSS properties: `border-left-width`, `border-right-width`, `border-top-width`, and `border-width`.
- The other border-bottom-related CSS properties: `border`, `border-bottom`, `border-bottom-style`, and `border-bottom-color`.

border-collapse CSS property

The **`border-collapse`** CSS property sets whether cells inside a `<table>` have shared or separate borders.

When cells are collapsed, the `border-style` value of `inset` behaves like `ridge`, and `outset` behaves like `groove`.

When cells are separated, the distance between cells is defined by the `border-spacing` property.

Try it

```
border-collapse: collapse;
```

```
border-collapse: separate;
```

```
<section class="default-example" id="default-example">
  <table class="transition-all" id="example-element">
    <tbody>
      <tr>
        <td>Cell 1.1</td>
        <td>Cell 1.2</td>
      </tr>
      <tr>
        <td>Cell 2.1</td>
        <td>Cell 2.2</td>
      </tr>
      <tr>
        <td>Cell 3.1</td>
        <td>Cell 3.2</td>
      </tr>
    </tbody>
  </table>
</section>
```

```
table {
  width: 15rem;
  table-layout: fixed;
}

td {
  border: 5px solid;
  border-color: crimson dodgerblue orange limegreen;
  padding: 0.75rem;
}
```

Syntax

```
/* Keyword values */
border-collapse: collapse;
border-collapse: separate;

/* Global values */
border-collapse: inherit;
border-collapse: initial;
border-collapse: revert;
border-collapse: revert-layer;
border-collapse: unset;
```

Values

This property is specified as one of the following keyword values:

`collapse`

Adjacent cells have shared borders (the collapsed-border table rendering model).

`separate`

Adjacent cells have distinct borders (the separated-border table rendering model).

Formal definition

Initial value	`separate`
Applies to	`table` and `inline-table` elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-collapse =
  separate  |
  collapse
```

This syntax reflects the latest standard as per CSS Table Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A colorful table of browser engines

HTML

```
<table class="separate">
  <caption>
    <code>border-collapse: separate</code>
  </caption>
  <tbody>
    <tr>
      <th>Browser</th>
      <th>Layout Engine</th>
    </tr>
    <tr>
      <td class="fx">Firefox</td>
      <td class="gk">Gecko</td>
    </tr>
    <tr>
      <td class="ed">Edge</td>
      <td class="tr">EdgeHTML</td>
    </tr>
    <tr>
      <td class="sa">Safari</td>
      <td class="wk">WebKit</td>
    </tr>
    <tr>
      <td class="ch">Chrome</td>
      <td class="bk">Blink</td>
    </tr>
    <tr>
      <td class="op">Opera</td>
      <td class="bk">Blink</td>
    </tr>
  </tbody>
</table>
<table class="collapse">
  <caption>
    <code>border-collapse: collapse</code>
  </caption>
  <tbody>
    <tr>
      <th>Browser</th>
      <th>Layout Engine</th>
    </tr>
    <tr>
      <td class="fx">Firefox</td>
      <td class="gk">Gecko</td>
    </tr>
    <tr>
      <td class="ed">Edge</td>
      <td class="tr">EdgeHTML</td>
    </tr>
    <tr>
      <td class="sa">Safari</td>
      <td class="wk">WebKit</td>
    </tr>
    <tr>
      <td class="ch">Chrome</td>
      <td class="bk">Blink</td>
    </tr>
    <tr>
      <td class="op">Opera</td>
      <td class="bk">Blink</td>
    </tr>
  </tbody>
</table>
```

CSS

```
.collapse {
  border-collapse: collapse;
}

.separate {
  border-collapse: separate;
}

table {
  display: inline-table;
  margin: 1em;
  border: dashed 5px;
}
```

```
table th,
table td {
  border: solid 3px;
}

.fx {
  border-color: orange blue;
}
.gk {
  border-color: black red;
}
.ed {
  border-color: blue gold;
}
.tr {
  border-color: aqua;
}
.sa {
  border-color: silver blue;
}
.wk {
  border-color: gold blue;
}
.ch {
  border-color: red yellow green blue;
}
.bk {
  border-color: navy blue teal aqua;
}
.op {
  border-color: red;
}
```

Result

See also

- `border-spacing`, `border-style`
- The `border-collapse` property alters the appearance of the `<table>` HTML element.
- CSS table module

border-color CSS property

The **`border-color`** CSS shorthand property sets the color of an element's border.

Try it

```
border-color: red;
```

```
border-color: red #32a1ce;
```

```
border-color: red rgb(170 50 220 / 0.6) green;
```

```
border-color: red yellow green hsl(60 90% 50% / 0.8);
```

```
border-color: red yellow green transparent;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-bottom-color`
- `border-left-color`
- `border-right-color`
- `border-top-color`

Syntax

```
/* <color> values */
border-color: red;

/* top and bottom | left and right */
border-color: red #f015ca;

/* top | left and right | bottom */
border-color: red rgb(240 30 50 / 70%) green;

/* top | right | bottom | left */
border-color: red yellow green blue;

/* Global values */
border-color: inherit;
```

```
border-color: initial;
border-color: revert;
border-color: revert-layer;
border-color: unset;
```

The `border-color` property may be specified using one, two, three, or four values.

- When **one** value is specified, it applies the same color to **all four sides**.
- When **two** values are specified, the first color applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first color applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the colors apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<color>`

Defines the color of the border.

Description

Each side can be set individually using `border-top-color`, `border-right-color`, `border-bottom-color`, and `border-left-color`; or using the writing mode-aware `border-block-start-color`, `border-block-end-color`, `border-inline-start-color`, and `border-inline-end-color`.

You can find out more information about border colors in Applying colors to HTML elements.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-top-color`: `currentcolor` • `border-right-color`: `currentcolor` • `border-bottom-color`: `currentcolor` • `border-left-color`: `currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-top-color`: computed color • `border-right-color`: computed color • `border-bottom-color`: computed color • `border-left-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-top-color`: a color • `border-right-color`: a color • `border-bottom-color`: a color • `border-left-color`: a color

Formal syntax

```
border-color =
  [ <color> | <image-1D> ]{1,4}

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Complete border-color usage

HTML

```
<div id="justone">
  <p><code>border-color: red;</code> is equivalent to</p>
  <ul>
    <li><code>border-top-color: red;</code></li>
    <li><code>border-right-color: red;</code></li>
    <li><code>border-bottom-color: red;</code></li>
    <li><code>border-left-color: red;</code></li>
  </ul>
</div>
<div id="horzvert">
  <p><code>border-color: gold red;</code> is equivalent to</p>
  <ul>
    <li><code>border-top-color: gold;</code></li>
    <li><code>border-right-color: red;</code></li>
    <li><code>border-bottom-color: gold;</code></li>
    <li><code>border-left-color: red;</code></li>
  </ul>
</div>
<div id="topvertbott">
  <p><code>border-color: red cyan gold;</code> is equivalent to</p>
```

```
  <ul>
    <li><code>border-top-color: red;</code></li>
    <li><code>border-right-color: cyan;</code></li>
    <li><code>border-bottom-color: gold;</code></li>
    <li><code>border-left-color: cyan;</code></li>
  </ul>
</div>
<div id="trbl">
  <p><code>border-color: red cyan black gold;</code> is equivalent to</p>
  <ul>
    <li><code>border-top-color: red;</code></li>
    <li><code>border-right-color: cyan;</code></li>
    <li><code>border-bottom-color: black;</code></li>
    <li><code>border-left-color: gold;</code></li>
  </ul>
</div>
```

CSS

```
#justone {
  border-color: red;
}

#horzvert {
  border-color: gold red;
}

#topvertbott {
  border-color: red cyan gold;
}

#trbl {
  border-color: red cyan black gold;
}

/* Set width and style for all divs */
div {
  border: solid 0.3em;
  width: auto;
  margin: 0.5em;
  padding: 0.5em;
}

ul {
  margin: 0;
  list-style: none;
}
```

Result

See also

- Border-color related CSS properties: `border`, `border-top-color`, `border-right-color`, `border-bottom-color`, `border-left-color`,
- Other border-related CSS properties: `border-width`, `border-style`
- The `<color>` data type
- Other color-related properties: `color`, `background-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`
- Applying color to HTML elements using CSS

border-end-end-radius CSS property

The **`border-end-end-radius`** CSS property defines a logical border radius on an element, which maps to a physical border radius that depends on the element's `writing-mode`, `direction`, and `text-orientation`. This is useful when building styles to work regardless of the text orientation and writing mode.

This property affects the corner between the block-end and the inline-end sides of the element. For instance, in a `horizontal-tb` writing mode with `ltr` direction, it corresponds to the `border-bottom-right-radius` property.

Try it

```
border-end-end-radius: 80px 80px;
```

```
border-end-end-radius: 250px 100px;
direction: rtl;
```

```
border-end-end-radius: 50%;
writing-mode: vertical-lr;
```

```
border-end-end-radius: 50%;
writing-mode: vertical-rl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a bottom right rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* <length> values */
/* With one value the corner will be a circle */
border-end-end-radius: 10px;
border-end-end-radius: 1em;

/* With two values the corner will be an ellipse */
border-end-end-radius: 1em 2em;

/* Global values */
border-end-end-radius: inherit;
border-end-end-radius: initial;
border-end-end-radius: revert;
border-end-end-radius: revert-layer;
border-end-end-radius: unset;
```

Values

`<length-percentage>`

Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-end-end-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border radius with vertical text

HTML

```
<div>
  <p class="exampleText">Example</p>
</div>
```

CSS

```
div {
  background-color: rebeccapurple;
  width: 120px;
  height: 120px;
  border-end-end-radius: 10px;
}

.exampleText {
  writing-mode: vertical-rl;
  padding: 10px;
  background-color: white;
  border-end-end-radius: 10px;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical property: `border-bottom-right-radius`
- `writing-mode`, `direction`, `text-orientation`

border-end-start-radius CSS property

The **`border-end-start-radius`** CSS property defines a logical border radius on an element, which maps to a physical border radius depending on the element's `writing-mode`, `direction`, and `text-orientation`. This is useful when building styles to work regardless of the text orientation and writing mode.

This property affects the corner between the block-end and the inline-start sides of the element. For instance, in a `horizontal-tb` writing mode with `ltr` direction, it corresponds to the `border-bottom-left-radius` property.

Try it

```
border-end-start-radius: 80px 80px;
```

```
border-end-start-radius: 250px 100px;
direction: rtl;
```

```
border-end-start-radius: 50%;
writing-mode: vertical-lr;
```

```
border-end-start-radius: 50%;
writing-mode: vertical-rl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a bottom left rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* <length> values */
/* With one value the corner will be a circle */
border-end-start-radius: 10px;
border-end-start-radius: 1em;

/* With two values the corner will be an ellipse */
border-end-start-radius: 1em 2em;

/* Global values */
border-end-start-radius: inherit;
border-end-start-radius: initial;
border-end-start-radius: revert;
border-end-start-radius: revert-layer;
border-end-start-radius: unset;
```

Values

`<length-percentage>`

> Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-end-start-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border radius with vertical text

HTML

```
<div>
  <p class="exampleText">Example</p>
</div>
```

CSS

```
div {
  background-color: rebeccapurple;
  width: 120px;
  height: 120px;
  border-end-start-radius: 10px;
}

.exampleText {
  writing-mode: vertical-rl;
  padding: 10px;
  background-color: white;
  border-end-start-radius: 10px;
}
```

Results

See also

- CSS Logical Properties and Values
- The mapped physical property: `border-top-right-radius`
- `writing-mode`, `direction`, `text-orientation`

border-image CSS property

The **`border-image`** CSS shorthand property draws an image around a given element. It replaces the element's regular border.

> **Note:** You should specify a separate `border-style` in case the border image fails to load. Although the specification doesn't strictly require it, some browsers don't render the border image if `border-style` is `none` or `border-width` is `0`.

Try it

```
border-image: url("/shared-assets/images/examples/border-diamonds.png") 30;
```

```
border-image: url("/shared-assets/images/examples/border-diamonds.png") 30 /
  19px round;
```

```
border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
  fill / 30px / 30px space;
```

```
border-image: linear-gradient(#f6b73c, #4d9f0c) 30;
```

```
border-image: repeating-linear-gradient(30deg, #4d9f0c, #9198e5, #4d9f0c 20px)
  60;
```

```
<section id="default-example">
  <div id="example-element">This is a box with a border around it.</div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px;
  background: #fff3d4;
  color: black;
  border: 30px solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
    round;
  font-size: 1.2em;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-image-outset`
- `border-image-repeat`
- `border-image-slice`
- `border-image-source`
- `border-image-width`

This shorthand property is reset, with all the longhand properties set to their initial values, by the CSS `border` shorthand property.

Syntax

```
/* source | slice */
border-image: linear-gradient(red, blue) 27;

/* source | slice | repeat */
border-image: url("/images/border.png") 27 space;
```

```
/* source | slice | width */
border-image: linear-gradient(red, blue) 27 / 35px;

/* source | slice | width | outset | repeat */
border-image: url("/images/border.png") 27 23 / 50px 30px / 1rem round space;

/* Global values */
border-image: inherit;
border-image: initial;
border-image: revert;
border-image: revert-layer;
border-image: unset;
```

The `border-image` property may be specified with anywhere from one to five of the values listed below.

> **Note:** If the computed value of `border-image-source` is `none`, or if the image cannot be displayed, the `border-style` will be displayed instead.

Values

`<'border-image-source'>`

The source image. See `border-image-source`.

`<'border-image-slice'>`

The dimensions for slicing the source image into regions. Up to four values may be specified. See `border-image-slice`.

`<'border-image-width'>`

The width of the border image. Up to four values may be specified. See `border-image-width`.

`<'border-image-outset'>`

The distance of the border image from the element's outside edge. Up to four values may be specified. See `border-image-outset`.

`<'border-image-repeat'>`

Defines how the edge regions of the source image are adjusted to fit the dimensions of the border image. Up to two values may be specified. See `border-image-repeat`.

Accessibility

Assistive technology cannot parse border images. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | Understanding WCAG 2.0

Formal definition

Initial value	as each of the properties of the shorthand: • `border-image-source`: `none` • `border-image-slice`: `100%` • `border-image-width`: `1` • `border-image-outset`: `0` • `border-image-repeat`: `stretch`
Applies to	all elements, except internal table elements when `border-collapse` is `collapse`. It also applies to `::first-letter`.
Inherited	no
Percentages	as each of the properties of the shorthand: • `border-image-slice`: refer to the size of the border image • `border-image-width`: refer to the width or height of the border image area
Computed value	as each of the properties of the shorthand: • `border-image-source`: `none` or the image with its URI made absolute • `border-image-slice`: one to four percentage(s) (as specified) or absolute length(s), plus the keyword `fill` if specified • `border-image-width`: as specified, but with relative lengths converted into absolute lengths • `border-image-outset`: as specified, but with relative lengths converted into absolute lengths • `border-image-repeat`: as specified
Animation type	as each of the properties of the shorthand: • `border-image-source`: discrete • `border-image-slice`: by computed value type • `border-image-width`: by computed value type • `border-image-outset`: by computed value type • `border-image-repeat`: discrete

Formal syntax

```
border-image =
  <'border-image-source'>                           ||
  <'border-image-slice'> [ / <'border-image-width'> | / <'border-image-width'>? / <'border-image-outset'> ]?  ||
  <'border-image-repeat'>

<border-image-source> =
  none     |
  <image>

<border-image-slice> =
  [ <number [0,∞]> | <percentage [0,∞]> ]{1,4}  &&
  fill?

<border-image-width> =
```

```
  [ <length-percentage [0,∞]> | <number [0,∞]> | auto ]{1,4}

<border-image-outset> =
  [ <length [0,∞]> | <number [0,∞]> ]{1,4}

<border-image-repeat> =
  [ stretch | repeat | round | space ]{1,2}

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<length-percentage> =
  <length>      |
  <percentage>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Values and Units Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Bitmap

In this example, we will apply a diamond pattern to an element's borders. The source for the border image is a ".png" file of 81 by 81 pixels, with three diamonds going vertically and horizontally:

HTML

```
<div id="bitmap">
  This element is surrounded by a bitmap-based border image!
</div>
```

CSS

To match the size of a single diamond, we will use a value of 81 divided by 3, or 27, for slicing the image into corner and edge regions. To center the border image on the edge of the element's background, we will make the outset values equal to half of the width values. Finally, a repeat value of `round` will make the border slices fit evenly, i.e., without clipping or gaps.

```
#bitmap {
  width: 200px;
  background-color: #ffffaa;
  border: 36px solid orange;
  margin: 30px;
  padding: 10px;

  border-image: url("border.png") 27 / 36px 28px 18px 8px / 18px 14px 9px 4px
    round;
}
```

Result

Gradient

HTML

```
<div id="gradient">
  This element is surrounded by a gradient-based border image!
</div>
```

CSS

```
#gradient {
  width: 200px;
  border: 30px solid;
  border-image: repeating-linear-gradient(45deg, #ff3333, #33bbff, #ff3333 30px)
    60;
```

```
  padding: 20px;
}
```

Result

Rounded borders

`border-radius` has no effect on the border image. This is because `border-image-outset` is able to place the image outside the border box, so it doesn't make sense for the border image to be clipped by the border area. To create rounded borders when using a border image, you should create the image itself with rounded corners, or, in the case of a gradient, draw it as the background instead. Below, we show one approach to do this, which is to use two `background-image`s: one that extends the border box, and another for the padding box.

HTML

```
<div id="rounded">
  This element is surrounded by a border image with rounded corners!
</div>
```

CSS

```
#rounded {
  width: 200px;
  /* Use transparent so the background image is visible */
  border: 10px solid transparent;
  padding: 20px;
  border-radius: 20px;
  background-image:
    linear-gradient(white, white), linear-gradient(to right, cyan, lime);
  background-origin: border-box;
  background-clip: padding-box, border-box;
}
```

Result

> **Note:** There is a new `background-clip: border-area` value being proposed to address this use case.

See also

- `border`
- `outline`
- `box-shadow`
- `background-image`
- `<url>` type
- Gradient functions: `conic-gradient()`, `repeating-conic-gradient()`, `linear-gradient()`, `repeating-linear-gradient()`, `radial-gradient()`, `repeating-radial-gradient()`
- Border images in CSS: A key focus area for Interop 2023 on MDN blog (2023)

border-image-outset CSS property

The **`border-image-outset`** CSS property sets the distance by which an element's border image is set out from its border box.

The parts of the border image that are rendered outside the element's border box with `border-image-outset` do not trigger overflow scrollbars and don't capture mouse events.

Try it

```
border-image-outset: 0;
```

```
border-image-outset: 15px;
```

```
border-image-outset: 30px;
```

```
border-image-outset: 40px;
```

```
<section id="default-example">
  <div id="example-element">This is a box with a border around it.</div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px;
  background: #fff3d4;
  color: black;
  border: 30px solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
    round;
  font-size: 1.2em;
}
```

Syntax

```
/* <length> value */
border-image-outset: 1rem;

/* <number> value */
border-image-outset: 1.5;

/* top and bottom | left and right */
border-image-outset: 1 1.2;

/* top | left and right | bottom */
border-image-outset: 30px 2 45px;

/* top | right | bottom | left */
border-image-outset: 7px 12px 14px 5px;
```

```
/* Global values */
border-image-outset: inherit;
border-image-outset: initial;
border-image-outset: revert;
border-image-outset: revert-layer;
border-image-outset: unset;
```

The `border-image-outset` property may be specified as one, two, three, or four values. Each value is a `<length>` or `<number>`. Negative values are invalid and will cause the `border-image-outset` declaration to be ignored.

1. If **one** value is specified, it applies to **all four sides**.
2. If **two** values are specified, the first applies to the **top and bottom** and the second to the **left and right**.
3. If **three** values are specified, the first applies to the **top**, the second to the **left and right**, and the third to the **bottom**.
4. If **four** values are specified, they apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<length>`

> The size of the `border-image` outset as a dimension — a number with a unit.

`<number>`

> The size of the `border-image` outset as a multiple of the element's corresponding `border-width`s. For example, if an element has `border-width: 1em 2px 0 1.5rem`, and `border-image-outset: 2`, the final `border-image-outset` would be calculated as `2em 4px 0 3rem`.

Formal definition

Initial value	`0`
Applies to	all elements, except internal table elements when `border-collapse` is `collapse`. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	by computed value type

Formal syntax

```
border-image-outset =
  [ <length [0,∞]> | <number [0,∞]> ]{1,4}
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Outsetting a border image

HTML

```
<div id="outset">This element has an outset border image!</div>
```

CSS

```
#outset {
  width: 10rem;
  background: #cceeff;
  border: 1.4rem solid;
  border-image: radial-gradient(#ffff22, #5555ff) 40;
  border-image-outset: 1.5; /* 1.5 × 1.4rem = 2.1rem */
  margin: 2.1rem;
}
```

Result

See also

- Backgrounds and borders
- Learn CSS: Backgrounds and borders
- Border images in CSS: A key focus area for Interop 2023 on MDN blog (2023)

border-image-repeat CSS property

The **`border-image-repeat`** CSS property defines how the images for the sides and the middle part of the border image are scaled and tiled. The middle region can be displayed by using the keyword "fill" in the `border-image-slice` property.

Try it

```
border-image-repeat: stretch;
```

```
border-image-repeat: repeat;
```

```
border-image-repeat: round;
```

```
border-image-repeat: space;
```

```
border-image-repeat: round stretch;
```

```
<section id="default-example">
  <div id="example-element">This is a box with a border around it.</div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  padding: 50px;
  background: #fff3d4;
  color: black;
  border: 30px solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
    round;
  font-size: 1.2em;
}
```

Syntax

```
/* Keyword values */
border-image-repeat: stretch;
border-image-repeat: repeat;
border-image-repeat: round;
border-image-repeat: space;

/* top and bottom | left and right */
border-image-repeat: round stretch;

/* Global values */
border-image-repeat: inherit;
border-image-repeat: initial;
border-image-repeat: revert;
border-image-repeat: revert-layer;
border-image-repeat: unset;
```

The `border-image-repeat` property may be specified using one or two values chosen from the list of values below.

- When **one** value is specified, it applies the same behavior on **all four sides**.
- When **two** values are specified, the first applies to the **top, middle, and bottom**, the second to the **left and right**.

Values

`stretch`

The source image's edge regions are stretched to fill the gap between each border.

`repeat`

The source image's edge regions are tiled (repeated) to fill the gap between each border. Tiles may be clipped to achieve the proper fit.

`round`

The source image's edge regions are tiled (repeated) to fill the gap between each border. Tiles may be stretched to achieve the proper fit.

`space`

The source image's edge regions are tiled (repeated) to fill the gap between each border. Extra space will be distributed in between tiles to achieve the proper fit.

Formal definition

Initial value	`stretch`
Applies to	all elements, except internal table elements when `border-collapse` is `collapse`. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-image-repeat =
  [ stretch | repeat | round | space ]{1,2}
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Repeating border images

CSS

```
#bordered {
  width: 12rem;
  margin-bottom: 1rem;
  padding: 1rem;
  border: 40px solid;
  border-image: url("border.png") 27;
  border-image-repeat: stretch; /* Can be changed in the live sample */
}
```

```
<div id="bordered">You can try out various border repetition rules on me!</div>

<select id="repetition">
  <option value="stretch">stretch</option>
  <option value="repeat">repeat</option>
  <option value="round">round</option>
  <option value="space">space</option>
  <option value="stretch repeat">stretch repeat</option>
  <option value="space round">space round</option>
</select>
```

```
const repetition = document.getElementById("repetition");
repetition.addEventListener("change", (evt) => {
  document.getElementById("bordered").style.borderImageRepeat =
    evt.target.value;
});
```

Results

See also

- Backgrounds and borders
- Learn CSS: Backgrounds and borders
- Border images in CSS: A key focus area for Interop 2023 on MDN blog (2023)

border-image-slice CSS property

The **`border-image-slice`** CSS property divides the image specified by `border-image-source` into regions. These regions form the components of an element's border image.

Try it

```
border-image-slice: 30;
```

```
border-image-slice: 30 fill;
```

```
border-image-slice: 44;
```

```
border-image: url("/shared-assets/images/examples/border-florid.svg") round;
border-image-slice: calc(50 / 184 * 100%) calc(80 / 284 * 100%) fill;
border-image-width: 30px 48px;
```

```
<section id="default-example">
  <div id="example-element">This is a box with a border around it.</div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px;
  background: #fff3d4;
  color: black;
  border: 30px solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
    round;
  font-size: 1.2em;
}
```

Syntax

```
/* All sides */
border-image-slice: 30%;

/* top and bottom | left and right */
border-image-slice: 10% 30%;

/* top | left and right | bottom */
border-image-slice: 30 30% 45;

/* top | right | bottom | left */
border-image-slice: 7 12 14 5;

/* Using the `fill` keyword */
border-image-slice: 10% fill;
border-image-slice: fill 10%;

/* Global values */
border-image-slice: inherit;
border-image-slice: initial;
border-image-slice: revert;
border-image-slice: revert-layer;
border-image-slice: unset;
```

The `border-image-slice` property may be specified using one to four `<number-percentage>` values to represent the position of each image slice. Negative values are invalid; values greater than their corresponding dimension are clamped to `100%`.

- When **one** position is specified, it creates all four slices at the same distance from their respective sides.
- When **two** positions are specified, the first value creates slices measured from the **top and bottom**, the second creates slices measured from the **left and right**.
- When **three** positions are specified, the first value creates a slice measured from the **top**, the second creates slices measured from the **left and right**, the third creates a slice measured from the **bottom**.
- When **four** positions are specified, they create slices measured from the **top**, **right**, **bottom**, and **left** in that order (clockwise).

The optional `fill` value, if used, can be placed anywhere in the declaration.

Values

`<number>`

Represents an edge offset in *pixels* for raster images and *coordinates* for vector images. For vector images, the number is relative to the element's size, not the size of the source image, so percentages are generally preferable in these cases.

`<percentage>`

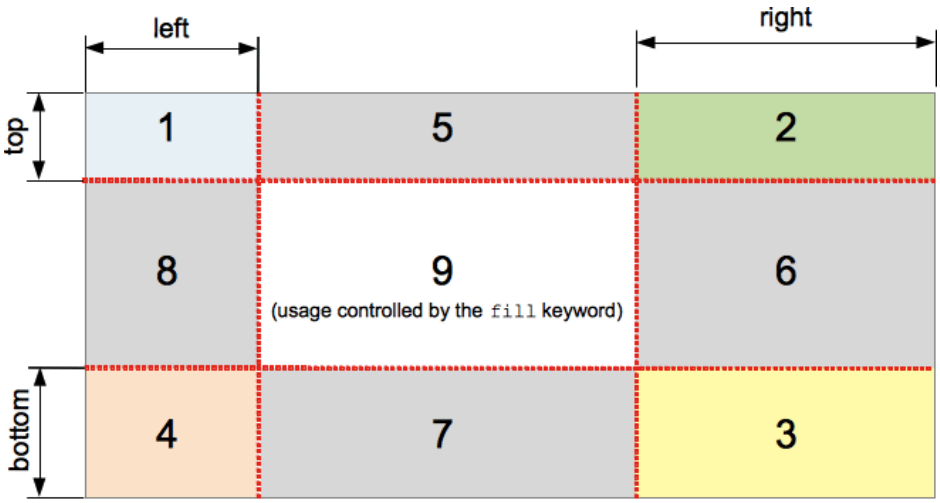
Represents an edge offset as a percentage of the source image's size: the width of the image for horizontal offsets, the height for vertical offsets.

`fill`

Preserves the middle image region and displays it like a background image, but stacked above the actual `background`. Its width and height are sized to match the top and left image regions, respectively.

Description

The slicing process creates nine regions in total: four corners, four edges, and a middle region. Four slice lines, set a given distance from their respective sides, control the size of the regions.



The above diagram illustrates the location of each region.

- Zones 1-4 are corner regions. Each one is used a single time to form the corners of the final border image.
- Zones 5-8 are edge regions. These are repeated, scaled, or otherwise modified in the final border image to match the dimensions of the element.
- Zone 9 is the middle region. It is discarded by default, but is used like a background image if the keyword `fill` is set.

The `border-image-repeat`, `border-image-width`, and `border-image-outset` properties determine how these regions are used to form the final border image.

Formal definition

Initial value	`100%`
Applies to	all elements, except internal table elements when `border-collapse` is `collapse`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the size of the border image
Computed value	one to four percentage(s) (as specified) or absolute length(s), plus the keyword `fill` if specified
Animation type	by computed value type

Formal syntax

```
border-image-slice =
  [ <number [0,∞]> | <percentage [0,∞]> ]{1,4}  &&
  fill?
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adjustable border width and slice

The following example shows a `<div>` with a border image set on it. The source image for the borders is as follows:

The diamonds in the source image are 30px across, so setting 30 pixels as the value for both `border-width` and `border-image-slice` will get you complete and fairly crisp diamonds in your border:

```
border-width: 30px;
border-image-slice: 30;
```

These are the default values we have used in this example. However, we have also provided two sliders to allow you to dynamically change the values of the above two properties, allowing you to appreciate the effect they have:

`border-image-slice` Changes the size of the image slice sampled for use in each border and border corner (and the content area, if the `fill` keyword is used) — varying this away from 30 causes the border to look somewhat irregular, but can have some interesting effects.

`border-width`: Changes the width of the border. The sampled image size is scaled to fit inside the border, which means that if the width is bigger than the slice, the image can start to look somewhat pixelated (unless of course you use an SVG image).

HTML

```
<div class="wrapper">
  <div></div>
</div>

<ul>
  <li>
    <label for="width">slide to adjust <code>border-width</code></label>
    <input type="range" min="10" max="45" id="width" />
    <output id="width-output">30px</output>
  </li>
  <li>
    <label for="slice">slide to adjust <code>border-image-slice</code></label>
    <input type="range" min="10" max="45" id="slice" />
    <output id="slice-output">30</output>
  </li>
</ul>
```

CSS

```
.wrapper {
  width: 400px;
  height: 300px;
}

div > div {
  width: 300px;
  height: 200px;
  border-width: 30px;
  border-style: solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png");
  border-image-slice: 30;
  border-image-repeat: round;
}

li {
  display: flex;
  place-content: center;
}
```

JavaScript

```
const widthSlider = document.getElementById("width");
const sliceSlider = document.getElementById("slice");
const widthOutput = document.getElementById("width-output");
const sliceOutput = document.getElementById("slice-output");
const divElem = document.querySelector("div > div");

widthSlider.addEventListener("input", () => {
  const newValue = `${widthSlider.value}px`;
  divElem.style.borderWidth = newValue;
  widthOutput.textContent = newValue;
});

sliceSlider.addEventListener("input", () => {
  const newValue = sliceSlider.value;
  divElem.style.borderImageSlice = newValue;
  sliceOutput.textContent = newValue;
});
```

Result

See also

- Illustrated description of the 1-to-4-value syntax
- Border images in CSS: A key focus area for Interop 2023 on MDN blog (2023)

border-image-source CSS property

The **`border-image-source`** CSS property sets the source image used to create an element's border image.

The `border-image-slice` property is used to divide the source image into regions, which are then dynamically applied to the final border image.

Try it

```
border-image-source: url("/shared-assets/images/examples/border-diamonds.png");
```

```
border-image-source: url("/shared-assets/images/examples/border-stars.png");
```

```
border-image-source: repeating-linear-gradient(
  45deg,
  transparent,
  #4d9f0c 20px
);
```

```
border-image-source: none;
```

```
<section id="default-example">
  <div id="example-element">This is a box with a border around it.</div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px;
  background: #fff3d4;
  color: black;
  border: 30px solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
    round;
  font-size: 1.2em;
}
```

Syntax

```
/* Keyword value */
border-image-source: none;

/* <image> values */
border-image-source: url("image.jpg");
border-image-source: linear-gradient(to top, red, yellow);

/* Global values */
border-image-source: inherit;
```

```
border-image-source: initial;
border-image-source: revert;
border-image-source: revert-layer;
border-image-source: unset;
```

Values

`none`

No border image is used. The appearance defined by `border-style` is displayed instead.

`<image>`

Image reference to use for the border.

Formal definition

Initial value	none
Applies to	all elements, except internal table elements when `border-collapse` is `collapse`. It also applies to `::first-letter`.
Inherited	no
Computed value	none or the image with its URI made absolute
Animation type	discrete

Formal syntax

```
border-image-source =
  none     |
  <image>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
.box {
  border-image-source: url("image.png");
}
```

See also

- `border`
- `outline`
- `box-shadow`
- `background-image`
- `<url>` type
- Border images in CSS: A key focus area for Interop 2023 on MDN blog (2023)

border-image-width CSS property

The **`border-image-width`** CSS property sets the width of an element's border image.

If this property's value is greater than the element's `border-width`, the border image will extend beyond the padding (and/or content) edge.

Try it

```
border-image-width: 30px;
```

```
border-image-width: 15px 40px;
```

```
border-image-width: 2.6rem;
```

```
border-image-width: 20% 8%;
```

```
<section id="default-example">
  <div id="example-element">This is a box with a border around it.</div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px;
  background: #fff3d4;
  color: black;
  border: 30px solid;
  border-image: url("/shared-assets/images/examples/border-diamonds.png") 30
    round;
  font-size: 1.2em;
}
```

Syntax

```
/* Keyword value */
border-image-width: auto;

/* <length> value */
border-image-width: 1rem;

/* <percentage> value */
border-image-width: 25%;

/* <number> value */
border-image-width: 3;

/* top and bottom | left and right */
border-image-width: 2em 3em;

/* top | left and right | bottom */
border-image-width: 5% 15% 10%;

/* top | right | bottom | left */
border-image-width: 5% 2em 10% auto;

/* Global values */
border-image-width: inherit;
border-image-width: initial;
border-image-width: revert;
border-image-width: revert-layer;
border-image-width: unset;
```

The `border-image-width` property may be specified using one, two, three, or four values chosen from the list of values below.

- When **one** value is specified, it applies the same width to **all four sides**.
- When **two** values are specified, the first width applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first width applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the widths apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<length-percentage>`

The width of the border, specified as a `<length>` or a `<percentage>`. Percentages are relative to the *width* of the border image area for horizontal offsets and the *height* of the border image area for vertical offsets. Must not be negative.

`<number>`

The width of the border, specified as a multiple of the corresponding `border-width`. Must not be negative.

`auto`

The width of the border is made equal to the intrinsic width or height (whichever is applicable) of the corresponding `border-image-slice`. If the image does not have the required intrinsic dimension, the corresponding `border-width` is used instead.

Formal definition

Initial value	`1`
Applies to	all elements, except internal table elements when `border-collapse` is `collapse`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the width or height of the border image area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	by computed value type

Formal syntax

```
border-image-width =
  [ <length-percentage [0,∞]> | <number [0,∞]> | auto ]{1,4}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Tiling a border image

This example creates a border image using the following ".png" file, which is 90 by 90 pixels:

Thus, each circle in the source image is 30 by 30 pixels.

HTML

```
<p>
  Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
  eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
  voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita
  kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.
</p>
```

CSS

```
p {
  border: 20px solid;
  border-image: url("border.png") 30 round;
  border-image-width: 16px;
  padding: 40px;
}
```

Result

See also

- Backgrounds and borders
- Learn CSS: Backgrounds and borders
- Border images in CSS: A key focus area for Interop 2023 on MDN blog (2023)

border-inline CSS property

The **`border-inline`** CSS shorthand property sets the individual logical inline border property values in a single place in the style sheet.

Try it

```
border-inline: solid;
writing-mode: horizontal-tb;
```

```
border-inline: dashed red;
writing-mode: vertical-rl;
```

```
border-inline: 1rem solid;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-inline-color`
- `border-inline-style`
- `border-inline-width`

Syntax

```
border-inline: 1px;
border-inline: 2px dotted;
border-inline: medium dashed blue;

/* Global values */
border-inline: inherit;
border-inline: initial;
border-inline: revert;
border-inline: revert-layer;
border-inline: unset;
```

Values

The `border-inline` is specified with one or more of the following, in any order:

`<line-width>`

The width of the border. See `border-width`.

`<line-style>`

The line style of the border. See `border-style`.

`<color>`

The color of the border.

Description

The physical borders to which `border-inline` maps depends on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top` and `border-bottom` or `border-right`, and `border-left` properties, depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The borders in the other dimension can be set with `border-block`, which sets `border-block-start`, and `border-block-end`.

Formal definition

Initial value	as each of the properties of the shorthand:
	• `border-inline-width`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-width`: `medium`
	  ◦ `border-inline-end-width`: `medium`
	• `border-inline-style`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-style`: `none`
	  ◦ `border-inline-end-style`: `none`
	• `border-inline-color`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-color`: `currentcolor`
	  ◦ `border-inline-end-color`: `currentcolor`

Applies to	all elements
Inherited	no

Computed value	as each of the properties of the shorthand:
	• `border-inline-width`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-width`: the absolute `<length>`, snapped as a line width
	  ◦ `border-inline-end-width`: the absolute `<length>`, snapped as a line width
	• `border-inline-style`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-style`: as specified
	  ◦ `border-inline-end-style`: as specified
	• `border-inline-color`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-color`: computed color
	  ◦ `border-inline-end-color`: computed color

Animation type	as each of the properties of the shorthand:
	• `border-inline-width`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-width`: by computed value type
	  ◦ `border-inline-end-width`: by computed value type
	• `border-inline-style`: discrete
	• `border-inline-color`: as each of the properties of the shorthand:
	  ◦ `border-inline-start-color`: by computed value type
	  ◦ `border-inline-end-color`: by computed value type

Formal syntax

```
border-inline =
  <'border-block-start'>

<border-block-start> =
  <line-width> ||
  <line-style> ||
  <color>

<line-width> =
  <length [0,∞]> |
  hairline       |
  thin           |
  medium         |
  thick

<line-style> =
  none   |
  hidden |
  dotted |
  dashed |
  solid  |
  double |
  groove |
  ridge  |
  inset  |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  border-inline: 5px dashed blue;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top`, `border-right`, `border-bottom`, or `border-left`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-color CSS property

The **`border-inline-color`** CSS property defines the color of the logical inline borders of an element, which maps to a physical border color depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-color` and `border-bottom-color`, or `border-right-color` and `border-left-color` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The border color in the other dimension can be set with `border-block-color` which sets `border-block-start-color`, and `border-block-end-color`.

Try it

```
border-inline-color: red;
writing-mode: horizontal-tb;
```

```
border-inline-color: #32a1ce;
writing-mode: vertical-rl;
```

```
border-inline-color: rgb(170 50 220 / 0.6);
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
border-inline-color: yellow;
border-inline-color: #f5f6f7;

/* Global values */
border-inline-color: inherit;
border-inline-color: initial;
border-inline-color: revert;
border-inline-color: revert-layer;
border-inline-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The color of the border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-inline-start-color`: `currentcolor` • `border-inline-end-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-inline-start-color`: computed color • `border-inline-end-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-inline-start-color`: by computed value type • `border-inline-end-color`: by computed value type

Formal syntax

```
border-inline-color =
  <'border-top-color'>{1,2}

<border-top-color> =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border color with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 10px solid blue;
  border-inline-color: red;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to the physical border properties: `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-end CSS property

The **`border-inline-end`** CSS shorthand property sets the individual logical inline-end border property values in a single place in the style sheet.

Try it

```
border-inline-end: solid;
writing-mode: horizontal-tb;
```

```
border-inline-end: dashed red;
writing-mode: vertical-rl;
```

```
border-inline-end: 1rem solid;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-inline-end-color`
- `border-inline-end-style`
- `border-inline-end-width`

Syntax

```
border-inline-end: 1px;
border-inline-end: 2px dashed;
border-inline-end: medium dashed blue;

/* Global values */
border-inline-end: inherit;
border-inline-end: initial;
border-inline-end: revert;
border-inline-end: revert-layer;
border-inline-end: unset;
```

The physical border to which `border-inline-end` maps depends on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top`, `border-right`, `border-bottom`, or `border-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Related properties are `border-block-start`, `border-block-end`, and `border-inline-start`, which define the other borders of the element.

Values

The `border-inline-end` is specified with one or more of the following, in any order:

`<line-width>`

The width of the border. See `border-width`.

`<line-style>`

The line style of the border. See `border-style`.

`<color>`

The color of the border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-inline-end-width`: `medium` • `border-inline-end-style`: `none` • `border-inline-end-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-inline-end-width`: the absolute `<length>`, snapped as a line width • `border-inline-end-style`: as specified • `border-inline-end-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-inline-end-width`: by computed value type • `border-inline-end-style`: discrete • `border-inline-end-color`: by computed value type

Formal syntax

```
border-inline-end =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}
```

```
.exampleText {
  writing-mode: vertical-rl;
  border-inline-end: 5px dashed blue;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top`, `border-right`, `border-bottom`, or `border-left`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-end-color CSS property

The **`border-inline-end-color`** CSS property defines the color of the logical inline-end border of an element, which maps to a physical border color depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-inline-end-color: red;
writing-mode: horizontal-tb;
```

```
border-inline-end-color: #32a1ce;
writing-mode: vertical-rl;
```

```
border-inline-end-color: rgb(170 50 220 / 0.6);
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
border-inline-end-color: rebeccapurple;
border-inline-end-color: #663399;

/* Global values */
border-inline-end-color: inherit;
border-inline-end-color: initial;
border-inline-end-color: revert;
border-inline-end-color: revert-layer;
border-inline-end-color: unset;
```

Related properties are `border-block-start-color`, `border-block-end-color`, and `border-inline-start-color`, which define the other border colors of the element.

Values

`<color>`

The color of the border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	no
Computed value	computed color
Animation type	by computed value type

Formal syntax

```
border-inline-end-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 10px solid blue;
  border-inline-end-color: red;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-end-style CSS property

The **`border-inline-end-style`** CSS property defines the style of the logical inline end border of an element, which maps to a physical border style depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-inline-end-style: dotted;
writing-mode: horizontal-tb;
```

```
border-inline-end-style: dotted;
writing-mode: vertical-rl;
```

```
border-inline-end-style: groove;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* Keyword values */
border-inline-end-style: dashed;
border-inline-end-style: dotted;
border-inline-end-style: groove;

/* Global values */
border-inline-end-style: inherit;
border-inline-end-style: initial;
border-inline-end-style: revert;
border-inline-end-style: revert-layer;
border-inline-end-style: unset;
```

Related properties are `border-block-start-style`, `border-block-end-style`, and `border-inline-start-style`, which define the other border styles of the element.

Values

`<line-style>`

The line style of the border. See `border-style`.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-inline-end-style =
  <line-style>
```

```
<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline-end-style

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 5px solid blue;
  border-inline-end-style: dashed;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-end-width CSS property

The **`border-inline-end-width`** CSS property defines the width of the logical inline-end border of an element, which maps to a physical border width depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-width`, `border-right-width`, `border-bottom-width`, or `border-left-width` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-inline-end-width: thick;
writing-mode: horizontal-tb;
```

```
border-inline-end-width: thick;
writing-mode: vertical-rl;
```

```
border-inline-end-width: 4px;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <'line-width'> values */
border-inline-end-width: 2px;
border-inline-end-width: thick;

/* Global values */
border-inline-end-width: inherit;
border-inline-end-width: initial;
border-inline-end-width: revert;
border-inline-end-width: revert-layer;
border-inline-end-width: unset;
```

Related properties are `border-block-start-width`, `border-block-end-width`, and `border-inline-start-width`, which define the other border widths of the element.

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	by computed value type

Formal syntax

```
border-inline-end-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying a border with vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 1px solid blue;
  border-inline-end-width: 5px;
}
```

Results

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-left-width`
- `writing-mode`, `direction`, `text-orientation`

border-inline-start CSS property

The **`border-inline-start`** CSS shorthand property sets the individual logical inline-start border property values in a single place in the style sheet.

Try it

```
border-inline-start: solid;
writing-mode: horizontal-tb;
```

```
border-inline-start: dashed red;
writing-mode: vertical-rl;
```

```
border-inline-start: 1rem solid;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-inline-start-color`
- `border-inline-start-style`
- `border-inline-start-width`

Syntax

```
border-inline-start: 1px;
border-inline-start: 2px dotted;
border-inline-start: medium dashed green;

/* Global values */
border-inline-start: inherit;
border-inline-start: initial;
border-inline-start: revert;
border-inline-start: revert-layer;
border-inline-start: unset;
```

The physical border to which `border-inline-start` maps depends on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top`, `border-right`, `border-bottom`, or `border-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Related properties are `border-block-start`, `border-block-end`, and `border-inline-end`, which define the other borders of the element.

Values

The `border-inline-start` is specified with one or more of the following, in any order:

`<line-width>`

The width of the border. See `border-width`.

`<line-style>`

The line style of the border. See `border-style`.

`<color>`

The color of the border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-inline-start-width`: `medium` • `border-inline-start-style`: `none` • `border-inline-start-color`: `currentcolor`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-inline-start-width`: the absolute `<length>`, snapped as a line width • `border-inline-start-style`: as specified • `border-inline-start-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-inline-start-width`: by computed value type • `border-inline-start-style`: discrete • `border-inline-start-color`: by computed value type

Formal syntax

```
border-inline-start =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}
```

```
.exampleText {
  writing-mode: vertical-rl;
  border-inline-start: 5px dashed blue;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top`, `border-right`, `border-bottom`, or `border-left`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-start-color CSS property

The **`border-inline-start-color`** CSS property defines the color of the logical inline start border of an element, which maps to a physical border color depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-color`, `border-right-color`, `border-bottom-color`, or `border-left-color` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-inline-start-color: red;
writing-mode: horizontal-tb;
```

```
border-inline-start-color: #32a1ce;
writing-mode: vertical-rl;
```

```
border-inline-start-color: rgb(170 50 220 / 0.6);
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
border-inline-start-color: red;
border-inline-start-color: #ee4141;

/* Global values */
border-inline-start-color: inherit;
border-inline-start-color: initial;
border-inline-start-color: revert;
border-inline-start-color: revert-layer;
border-inline-start-color: unset;
```

Related properties are `border-block-start-color`, `border-block-end-color`, and `border-inline-end-color`, which define the other border colors of the element.

Values

`<color>`

The color of the border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	no
Computed value	computed color
Animation type	by computed value type

Formal syntax

```
border-inline-start-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 10px solid blue;
  border-inline-start-color: red;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-color`, `border-right-color`, `border-bottom-color`, and `border-left-color`
- `writing-mode`, `direction`, `text-orientation`

border-inline-start-style CSS property

The **`border-inline-start-style`** CSS property defines the style of the logical inline start border of an element, which maps to a physical border style depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-inline-start-style: dotted;
writing-mode: horizontal-tb;
```

```
border-inline-start-style: dotted;
writing-mode: vertical-rl;
```

```
border-inline-start-style: groove;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* Keyword values */
border-inline-start-style: dashed;
border-inline-start-style: dotted;
border-inline-start-style: groove;

/* Global values */
border-inline-start-style: inherit;
border-inline-start-style: initial;
border-inline-start-style: revert;
border-inline-start-style: revert-layer;
border-inline-start-style: unset;
```

Related properties are `border-block-start-style`, `border-block-end-style`, and `border-inline-end-style`, which define the other border styles of the element.

Values

`<line-style>`

The line style of the border. See `border-style`.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-inline-start-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 5px solid blue;
  border-inline-start-style: dashed;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-start-width CSS property

The **`border-inline-start-width`** CSS property defines the width of the logical inline-start border of an element, which maps to a physical border width depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-width`, `border-right-width`, `border-bottom-width`, or `border-left-width` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
border-inline-start-width: thick;
writing-mode: horizontal-tb;
```

```
border-inline-start-width: thick;
writing-mode: vertical-rl;
```

```
border-inline-start-width: 4px;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <'line-width'> values */
border-inline-start-width: 5px;
border-inline-start-width: thick;

/* Global values */
border-inline-start-width: inherit;
border-inline-start-width: initial;
border-inline-start-width: revert;
border-inline-start-width: revert-layer;
border-inline-start-width: unset;
```

Related properties are `border-block-start-width`, `border-block-end-width`, and `border-inline-end-width`, which define the other border widths of the element.

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	by computed value type

Formal syntax

```
border-inline-start-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 1px solid blue;
  border-inline-start-width: 5px;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-left-width`
- `writing-mode`, `direction`, `text-orientation`

border-inline-style CSS property

The **`border-inline-style`** CSS property defines the style of the logical inline borders of an element, which maps to a physical border style depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-style` and `border-bottom-style`, or `border-left-style` and `border-right-style` properties depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The border style in the other dimension can be set with `border-block-style`, which sets `border-block-start-style`, and `border-block-end-style`.

Try it

```
border-inline-style: dotted;
writing-mode: horizontal-tb;
```

```
border-inline-style: dotted;
writing-mode: vertical-rl;
```

```
border-inline-style: groove;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* Keyword values */
border-inline-style: dashed;
border-inline-style: dotted;
border-inline-style: groove;

/* Global values */
```

```
border-inline-style: inherit;
border-inline-style: initial;
border-inline-style: revert;
border-inline-style: revert-layer;
border-inline-style: unset;
```

Values

`<line-style>`

The line style of the border. See `border-style`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-inline-start-style`: none • `border-inline-end-style`: none
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-inline-start-style`: as specified • `border-inline-end-style`: as specified
Animation type	discrete

Formal syntax

```
border-inline-style =
  <'border-top-style'>{1,2}

<border-top-style> =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting border-inline-style

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 5px solid blue;
  border-inline-style: dashed;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-style`, `border-right-style`, `border-bottom-style`, or `border-left-style`.
- `writing-mode`, `direction`, `text-orientation`

border-inline-width CSS property

The **`border-inline-width`** CSS property defines the width of the logical inline borders of an element, which maps to a physical border width depending on the element's writing mode, directionality, and text orientation. It corresponds to the `border-top-width` and `border-bottom-width`, or `border-left-width`, and `border-right-width` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The border width in the other dimension can be set with `border-block-width`, which sets `border-block-start-width`, and `border-block-end-width`.

Try it

```
border-inline-width: thick;
writing-mode: horizontal-tb;
```

```
border-inline-width: thick;
writing-mode: vertical-rl;
```

```
border-inline-width: 4px;
writing-mode: horizontal-tb;
```

```
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <'line-width'> values */
border-inline-width: 5px 10px;
border-inline-width: 5px;
border-inline-width: thick;

/* Global values */
border-inline-width: inherit;
border-inline-width: initial;
border-inline-width: revert;
border-inline-width: revert-layer;
border-inline-width: unset;
```

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-inline-start-width`: `medium` • `border-inline-end-width`: `medium`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-inline-start-width`: the absolute `<length>`, snapped as a line width • `border-inline-end-width`: the absolute `<length>`, snapped as a line width
Animation type	as each of the properties of the shorthand: • `border-inline-start-width`: by computed value type • `border-inline-end-width`: by computed value type

Formal syntax

```
border-inline-width =
  <'border-top-width'>{1,2}

<border-top-width> =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  border: 1px solid blue;
  border-inline-width: 5px 10px;
}
```

See also

- CSS Logical Properties and Values
- This property maps to one of the physical border properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-left-width`
- `writing-mode`, `direction`, `text-orientation`

border-left CSS property

The **`border-left`** CSS shorthand property sets all the properties of an element's left border.

Try it

```
border-left: solid;
```

```
border-left: dashed red;
```

```
border-left: 1rem solid;
```

```
border-left: thick double #32a1ce;
```

```
border-left: 4mm ridge rgb(211 220 50 / 0.6);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-left-color`
- `border-left-style`
- `border-left-width`

Syntax

```
border-left: 1px;
border-left: 2px dotted;
border-left: medium dashed blue;

/* Global values */
border-left: inherit;
border-left: initial;
border-left: revert;
border-left: revert-layer;
border-left: unset;
```

The three values of the shorthand property can be specified in any order, and one or two of them may be omitted.

Values

`<br-width>`

See `border-left-width`.

`<br-style>`

See `border-left-style`.

`<color>`

See `border-left-color`.

Description

As with all shorthand properties, `border-left` always sets the values of all of the properties that it can set, even if they are not specified. It sets those that are not specified to their default values. Consider the following code:

```
border-left-style: dotted;
border-left: thick green;
```

It is actually the same as this one:

```
border-left-style: dotted;
border-left: none thick green;
```

The value of `border-left-style` given before `border-left` is ignored. Since the default value of `border-left-style` is `none`, not specifying the `border-style` part results in no border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-left-width`: `medium` • `border-left-style`: `none` • `border-left-color`: `currentcolor`

Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-left-width`: the absolute `<length>`, snapped as a line width • `border-left-style`: as specified • `border-left-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-left-width`: a length • `border-left-style`: discrete • `border-left-color`: a color

Formal syntax

```
border-left =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying a left border

HTML

```
<div>This box has a border on the left side.</div>
```

CSS

```
div {
  border-left: 4px dashed blue;
  background-color: gold;
  height: 100px;
  width: 100px;
  font-weight: bold;
  text-align: center;
}
```

Results

See also

- `border`
- `border-block`
- `outline`
- Backgrounds and borders
- Learn CSS: Backgrounds and borders

border-left-color CSS property

The **`border-left-color`** CSS property sets the color of an element's left border. It can also be set with the shorthand CSS properties `border-color` or `border-left`.

Try it

```
border-left-color: red;
```

```
border-left-color: #32a1ce;
```

```
border-left-color: rgb(170 50 220 / 0.6);
```

```
border-left-color: hsl(60 90% 50% / 0.8);
```

```
border-left-color: transparent;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
```

```
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* <color> values */
border-left-color: red;
border-left-color: #ffbb00;
border-left-color: rgb(255 0 0);
border-left-color: hsl(100deg 50% 25% / 75%);
border-left-color: currentColor;
border-left-color: transparent;

/* Global values */
border-left-color: inherit;
border-left-color: initial;
border-left-color: revert;
border-left-color: revert-layer;
border-left-color: unset;
```

Values

This property is specified as one value:

`<color>`

The color of the left border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
border-left-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A div with a border

HTML

```
<div class="my-box">
  <p>
    This is a box with a border around it. Note which side of the box is
    <span class="red-text">red</span>.
  </p>
</div>
```

CSS

```
.my-box {
  border: solid 0.3em gold;
  border-left-color: red;
  width: auto;
}

.red-text {
  color: red;
}
```

Result

See also

- The border-related CSS shorthand properties: `border`, `border-left`, and `border-color`.
- The color-related CSS properties for the other borders: `border-right-color`, `border-bottom-color`, and `border-top-color`.
- The other border-related CSS properties applying to the same border: `border-left-style` and `border-left-width`.
- The default `currentColor` color value.

border-left-style CSS property

The **`border-left-style`** CSS property sets the line style of an element's left `border`.

> **Note:** The specification doesn't define how borders of different styles connect in the corners.

Try it

```
border-left-style: none;
```

```
border-left-style: dotted;
```

```
border-left-style: dashed;
```

```
border-left-style: solid;
```

```
border-left-style: groove;
```

```
border-left-style: inset;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}

body {
  background-color: white;
}
```

Syntax

```
/* Keyword values */
border-left-style: none;
border-left-style: hidden;
border-left-style: dotted;
border-left-style: dashed;
border-left-style: solid;
border-left-style: double;
border-left-style: groove;
border-left-style: ridge;
border-left-style: inset;
border-left-style: outset;

/* Global values */
border-left-style: inherit;
border-left-style: initial;
border-left-style: revert;
border-left-style: revert-layer;
border-left-style: unset;
```

The `border-left-style` property is specified as a single `<line-style>` keyword value.

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-left-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting border-left-style

HTML

```
<table>
  <tbody>
    <tr>
      <td class="b1">none</td>
      <td class="b2">hidden</td>
      <td class="b3">dotted</td>
      <td class="b4">dashed</td>
    </tr>
    <tr>
      <td class="b5">solid</td>
      <td class="b6">double</td>
      <td class="b7">groove</td>
      <td class="b8">ridge</td>
    </tr>
    <tr>
      <td class="b9">inset</td>
      <td class="b10">outset</td>
    </tr>
  </tbody>
</table>
```

CSS

```
/* Define look of the table */
table {
  border-width: 2px;
  background-color: #52e385;
}
tr,
td {
  padding: 3px;
}

/* border-left-style example classes */
.b1 {
  border-left-style: none;
}
.b2 {
  border-left-style: hidden;
}
.b3 {
  border-left-style: dotted;
}
.b4 {
  border-left-style: dashed;
}
.b5 {
  border-left-style: solid;
}
.b6 {
  border-left-style: double;
}
.b7 {
  border-left-style: groove;
}
.b8 {
  border-left-style: ridge;
}
.b9 {
  border-left-style: inset;
}
.b10 {
  border-left-style: outset;
}
```

Result

See also

- The other style-related border properties: `border-bottom-style`, `border-right-style`, `border-top-style`, and `border-style`.
- The other left-border-related properties: `border-left`, `border-left-color`, and `border-left-width`.

border-left-width CSS property

The **`border-left-width`** CSS property sets the width of the left border of an element.

Try it

```
border-left-width: thick;
```

```
border-left-width: 2em;
```

```
border-left-width: 4px;
```

```
border-left-width: 2ex;
```

```
border-left-width: 0;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* Keyword values */
border-left-width: thin;
border-left-width: medium;
border-left-width: thick;

/* <length> values */
border-left-width: 10em;
border-left-width: 3vmax;
border-left-width: 6px;

/* Global keywords */
border-left-width: inherit;
border-left-width: initial;
border-left-width: revert;
border-left-width: revert-layer;
border-left-width: unset;
```

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
border-left-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing border widths

HTML

```
<div>Element 1</div>
<div>Element 2</div>
```

CSS

```
div {
  border: 1px solid red;
  margin: 1em 0;
}

div:nth-child(1) {
  border-left-width: thick;
}
div:nth-child(2) {
  border-left-width: 2em;
}
```

Result

See also

- The other border-width-related CSS properties: `border-top-width`, `border-right-width`, `border-bottom-width`, and `border-width`.
- The other border-left-related CSS properties: `border`, `border-left`, `border-left-style`, and `border-left-color`.

border-radius CSS property

The **`border-radius`** CSS shorthand property rounds the corners of an element's outer border edge. You can set a single radius to make circular corners, or two radii to make elliptical corners.

Try it

```
border-radius: 30px;
```

```
border-radius: 25% 10%;
```

```
border-radius: 10% 30% 50% 70%;
```

```
border-radius: 10% / 50%;
```

```
border-radius: 10px 100px / 120px;
```

```
border-radius: 50% 20% / 10% 40%;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with rounded corners.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-top-left-radius`
- `border-top-right-radius`
- `border-bottom-right-radius`
- `border-bottom-left-radius`

Syntax

```
/* The syntax of the first radius allows one to four values */
/* Radius is set for all 4 sides */
border-radius: 10px;

/* top-left-and-bottom-right | top-right-and-bottom-left */
border-radius: 10px 5%;

/* top-left | top-right-and-bottom-left | bottom-right */
border-radius: 2px 4px 2px;

/* top-left | top-right | bottom-right | bottom-left */
border-radius: 1px 0 3px 4px;

/* The syntax of the second radius allows one to four values */
/* (first radius values) / radius */
border-radius: 10px / 20px;

/* (first radius values) / top-left-and-bottom-right | top-right-and-bottom-left */
border-radius: 10px 5% / 20px 30px;

/* (first radius values) / top-left | top-right-and-bottom-left | bottom-right */
border-radius: 10px 5px 2em / 20px 25px 30%;

/* (first radius values) / top-left | top-right | bottom-right | bottom-left */
border-radius: 10px 5% / 20px 25em 30px 35em;

/* Global values */
border-radius: inherit;
border-radius: initial;
border-radius: revert;
border-radius: revert-layer;
border-radius: unset;
```

The `border-radius` property is specified as:

- one, two, three, or four `<length>` or `<percentage>` values. This is used to set a single radius for the corners.
- followed optionally by "/" and one, two, three, or four `<length>` or `<percentage>` values. This is used to set an additional radius, so you can have elliptical corners.

Values

radius		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in each corner of the border. It is used only in the one-value syntax.
top-left-and-bottom-right		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in the top-left and bottom-right corners of the element's box. It is used only in the two-value syntax.
top-right-and-bottom-left		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in the top-right and bottom-left corners of the element's box. It is used only in the two- and three-value syntaxes.
top-left		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in the top-left corner of the element's box. It is used only in the three- and four-value syntaxes.

top-right		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in the top-right corner of the element's box. It is used only in the four-value syntax.
bottom-right		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in the bottom-right corner of the element's box. It is used only in the three- and four-value syntaxes.
bottom-left		Is a `<length>` or a `<percentage>` denoting a radius to use for the border in the bottom-left corner of the element's box. It is used only in the four-value syntax.

`<length>`

Denotes the size of the circle radius, or the semi-major and semi-minor axes of the ellipse, using length values. Negative values are invalid.

`<percentage>`

Denotes the size of the circle radius, or the semi-major and semi-minor axes of the ellipse, using percentage values. Percentages for the horizontal axis refer to the width of the box; percentages for the vertical axis refer to the height of the box. Negative values are invalid.

For example:

```
border-radius: 1em / 5em;

/* It is equivalent to: */
border-top-left-radius: 1em 5em;
border-top-right-radius: 1em 5em;
border-bottom-right-radius: 1em 5em;
border-bottom-left-radius: 1em 5em;
```

```
border-radius: 4px 3px 6px / 2px 4px;

/* It is equivalent to: */
border-top-left-radius: 4px 2px;
border-top-right-radius: 3px 4px;
border-bottom-right-radius: 6px 2px;
border-bottom-left-radius: 3px 4px;
```

Description

The radius applies to the whole `background`, even if the element has no border; the exact position of the clipping is defined by the `background-clip` property.

The `border-radius` property does not apply to table elements when `border-collapse` is `collapse`.

> **Note:** As with any shorthand property, individual sub-properties cannot inherit, such as in `border-radius:0 0 inherit inherit`, which would partially override existing definitions. Instead, the individual longhand properties have to be used.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-top-left-radius`: `0` • `border-top-right-radius`: `0` • `border-bottom-right-radius`: `0` • `border-bottom-left-radius`: `0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	as each of the properties of the shorthand: • `border-top-left-radius`: two absolute `<length>`s or `<percentage>`s • `border-top-right-radius`: two absolute `<length>`s or `<percentage>`s • `border-bottom-right-radius`: two absolute `<length>`s or `<percentage>`s • `border-bottom-left-radius`: two absolute `<length>`s or `<percentage>`s
Animation type	as each of the properties of the shorthand: • `border-top-left-radius`: a length, percentage or calc(); • `border-top-right-radius`: a length, percentage or calc(); • `border-bottom-right-radius`: a length, percentage or calc(); • `border-bottom-left-radius`: a length, percentage or calc();

Formal syntax

```
border-radius =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing border styles

The following example has seven `<pre>` elements, each of which demonstrates combinations of `border` and `border-radius` styles. The styles applied to each `<pre>` element are included as the element's contents, so you can see the CSS declarations necessary to create the associated border style:

```
<pre id="example-1">
  border: solid 10px;
  border-radius: 10px 40px 40px 10px;
</pre>

<pre id="example-2">
  border: groove 1em red;
  border-radius: 2em;
</pre>

<pre id="example-3">
  background: gold;
  border: ridge gold;
  border-radius: 13em / 3em;
</pre>

<pre id="example-4">
  border: none;
  border-radius: 40px 10px;
  background: gold;
</pre>

<pre id="example-5">
  border: none;
  border-radius: 50%;
  background: burlywood;
</pre>

<pre id="example-6">
  border: dotted;
  border-width: 10px 4px;
  border-radius: 10px 40px;
</pre>

<pre id="example-7">
  border: dashed;
  border-width: 2px 4px;
  border-radius: 40px;
</pre>
```

```
pre {
  margin: 20px;
  padding: 20px;
  width: 80%;
  height: 50px;
}

pre#example-1 {
  border: solid 10px;
  border-radius: 10px 40px 40px 10px;
}

pre#example-2 {
  border: groove 1em red;
  border-radius: 2em;
}

pre#example-3 {
  background: gold;
  border: ridge gold;
  border-radius: 13em / 3em;
}

pre#example-4 {
  border: none;
  border-radius: 40px 10px;
  background: gold;
}

pre#example-5 {
  border: none;
  border-radius: 50%;
  background: burlywood;
}

pre#example-6 {
  border: dotted;
  border-width: 10px 4px;
  border-radius: 10px 40px;
}

pre#example-7 {
  border: dashed;
  border-width: 2px 4px;
  border-radius: 40px;
}
```

Using <code>corner-shape</code> with <code>border-radius</code>

When a non-`0` `border-radius` value is applied to a box corner, you can use the `corner-shape` property (or one of its longhands and shorthands) to apply custom shapes to that corner, such as a bevel, notch, or squircle. This example demonstrates `corner-shape` usage.

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `0 20% 50px 30%`, and a `corner-shape` of `superellipse(0.5) bevel notch squircle`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: green;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 0 20% 50px 30%;
  corner-shape: superellipse(0.5) bevel notch squircle;
}
```

Result

The rendered result looks like this:

Note how no corner shape is applied to the top-left corner, because it has a `border-radius` of `0` set.

See also

- Border-radius-related CSS properties: `border-top-left-radius`, `border-top-right-radius`, `border-bottom-right-radius`, `border-bottom-left-radius`, `border-start-start-radius`, `border-start-end-radius`, `border-end-start-radius`, `border-end-end-radius`

border-right CSS property

The **`border-right`** CSS shorthand property sets all the properties of an element's right border.

Try it

```
border-right: solid;
```

```
border-right: dashed red;
```

```
border-right: 1rem solid;
```

```
border-right: thick double #32a1ce;
```

```
border-right: 4mm ridge rgb(211 220 50 / 0.6);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-right-color`
- `border-right-style`
- `border-right-width`

Syntax

```
border-right: 1px;
border-right: 2px dotted;
border-right: medium dashed green;

/* Global values */
border-right: inherit;
border-right: initial;
border-right: revert;
border-right: revert-layer;
border-right: unset;
```

The three values of the shorthand property can be specified in any order, and one or two of them may be omitted.

Values

`<br-width>`

See `border-right-width`.

`<br-style>`

See `border-right-style`.

`<color>`

See `border-right-color`.

Description

As with all shorthand properties, `border-right` always sets the values of all of the properties that it can set, even if they are not specified. It sets those that are not specified to their default values. Consider the following code:

```
border-right-style: dotted;
border-right: thick green;
```

It is actually the same as this one:

```
border-right-style: dotted;
border-right: none thick green;
```

The value of `border-right-style` given before `border-right` is ignored. Since the default value of `border-right-style` is `none`, not specifying the `border-style` part results in no border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-right-width`: `medium` • `border-right-style`: `none` • `border-right-color`: `currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-right-width`: the absolute `<length>`, snapped as a line width • `border-right-style`: as specified • `border-right-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-right-width`: a length • `border-right-style`: discrete • `border-right-color`: a color

Formal syntax

```
border-right =
  <line-width> ||
  <line-style> ||
  <color>

<line-width> =
  <length [0,∞]> |
  hairline       |
  thin           |
  medium         |
  thick

<line-style> =
  none   |
  hidden |
  dotted |
  dashed |
  solid  |
  double |
  groove |
  ridge  |
  inset  |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying a right border

HTML

```
<div>This box has a border on the right side.</div>
```

CSS

```
div {
  border-right: 4px dashed blue;
  background-color: gold;
  height: 100px;
  width: 100px;
  font-weight: bold;
```

```
  text-align: center;
}
```

Results

See also

- `border`
- `border-block`
- `outline`

border-right-color CSS property

The **`border-right-color`** CSS property sets the color of an element's right border. It can also be set with the shorthand CSS properties `border-color` or `border-right`.

Try it

```
border-right-color: red;
```

```
border-right-color: #32a1ce;
```

```
border-right-color: rgb(170 50 220 / 0.6);
```

```
border-right-color: hsl(60 90% 50% / 0.8);
```

```
border-right-color: transparent;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* <color> values */
border-right-color: red;
border-right-color: #ffbb00;
border-right-color: rgb(255 0 0);
border-right-color: hsl(100deg 50% 25% / 75%);
border-right-color: currentColor;
border-right-color: transparent;

/* Global values */
border-right-color: inherit;
border-right-color: initial;
border-right-color: revert;
border-right-color: revert-layer;
border-right-color: unset;
```

Values

This property is specified as one value:

`<color>`

The color of the right border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
border-right-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A div with a border

HTML

```
<div class="my-box">
  <p>
    This is a box with a border around it. Note which side of the box is
    <span class="red-text">red</span>.
  </p>
</div>
```

CSS

```
.my-box {
  border: solid 0.3em gold;
  border-right-color: red;
  width: auto;
}

.red-text {
  color: red;
}
```

Result

See also

- The border-related CSS shorthand properties: `border`, `border-right`, and `border-color`.
- The color-related CSS properties for the other borders: `border-left-color`, `border-bottom-color`, and `border-top-color`.
- The other border-related CSS properties applying to the same border: `border-right-style` and `border-right-width`.
- The default `currentColor` color value.

border-right-style CSS property

The **`border-right-style`** CSS property sets the line style of an element's right `border`.

> **Note:** The specification doesn't define how borders of different styles connect in the corners.

Try it

```
border-right-style: none;
```

```
border-right-style: dotted;
```

```
border-right-style: dashed;
```

```
border-right-style: solid;
```

```
border-right-style: groove;
```

```
border-right-style: inset;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}

body {
  background-color: white;
}
```

Syntax

```
/* Keyword values */
border-right-style: none;
border-right-style: hidden;
border-right-style: dotted;
border-right-style: dashed;
border-right-style: solid;
border-right-style: double;
border-right-style: groove;
border-right-style: ridge;
border-right-style: inset;
border-right-style: outset;

/* Global values */
border-right-style: inherit;
border-right-style: initial;
border-right-style: revert;
```

```
border-right-style: revert-layer;
border-right-style: unset;
```

The `border-right-style` property is specified as a single `<line-style>` keyword value.

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-right-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border styles

HTML

```
<table>
  <tbody>
    <tr>
      <td class="b1">none</td>
      <td class="b2">hidden</td>
      <td class="b3">dotted</td>
      <td class="b4">dashed</td>
    </tr>
    <tr>
      <td class="b5">solid</td>
      <td class="b6">double</td>
      <td class="b7">groove</td>
      <td class="b8">ridge</td>
    </tr>
    <tr>
      <td class="b9">inset</td>
      <td class="b10">outset</td>
    </tr>
  </tbody>
</table>
```

CSS

```
/* Define look of the table */
table {
  border-width: 2px;
  background-color: #52e385;
}
tr,
td {
  padding: 3px;
}

/* border-right-style example classes */
.b1 {
  border-right-style: none;
}
.b2 {
  border-right-style: hidden;
}
.b3 {
  border-right-style: dotted;
}
.b4 {
  border-right-style: dashed;
}
.b5 {
  border-right-style: solid;
}
.b6 {
  border-right-style: double;
}
.b7 {
  border-right-style: groove;
}
.b8 {
  border-right-style: ridge;
}
.b9 {
  border-right-style: inset;
}
.b10 {
```

```
  border-right-style: outset;
}
```

Result

See also

- The other style-related border properties: `border-bottom-style`, `border-left-style`, `border-top-style`, and `border-style`.
- The other right-border-related properties: `border-right`, `border-right-color`, and `border-right-width`.

border-right-width CSS property

The **`border-right-width`** CSS property sets the width of the right border of an element.

Try it

```
border-right-width: thick;
```

```
border-right-width: 2em;
```

```
border-right-width: 4px;
```

```
border-right-width: 2ex;
```

```
border-right-width: 0;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* Keyword values */
border-right-width: thin;
border-right-width: medium;
border-right-width: thick;

/* <length> values */
border-right-width: 10em;
border-right-width: 3vmax;
border-right-width: 6px;

/* Global keywords */
border-right-width: inherit;
border-right-width: initial;
border-right-width: revert;
border-right-width: revert-layer;
border-right-width: unset;
```

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
border-right-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing border widths

HTML

```
<div>Element 1</div>
```

```
<div>Element 2</div>
```

CSS

```
div {
  border: 1px solid red;
  margin: 1em 0;
}

div:nth-child(1) {
  border-right-width: thick;
}
div:nth-child(2) {
  border-right-width: 2em;
}
```

Result

See also

- The other border-width-related CSS properties: `border-bottom-width`, `border-left-width`, `border-top-width`, and `border-width`.
- The other border-right-related CSS properties: `border`, `border-right`, `border-right-style`, and `border-right-color`.

border-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`border-shape`** CSS property defines an element's border shape using `<basic-shape>` values.

Syntax

```
/* Keyword value */
border-shape: none;

/* Single <basic-shape> value */
border-shape: circle(50%);
border-shape: rect(10px 460px 130px 20px round 20px);
border-shape: shape(
  from 5% 0.5rem,
  arc to 80px 1pt of 10% ccw large rotate 25deg
);

/* Two <basic-shape> values */
border-shape: circle(50%) ellipse(40% 30%);
border-shape: polygon(0% 0%, 0% 100%, 100% 0%)
  polygon(10% 10%, 10% 70%, 70% 10%);

/* <basic-shape> and <geometry-box> values */
border-shape: circle(50%) border-box ellipse(40% 30%) view-box;
border-shape: rect(5px 198px 189px 0px round 20px) view-box circle(50%);
border-shape: path(
    "M 35,95 C 35,50 60,15 100,20 C 120,5 160,5 180,22 C 200,5 250,5 270,22 C 295,5 340,5 360,22 C 395,10 440,35 440,75 C 455,90 450,120 430,128 C 400,145 360,145 330,130 C 300,145 260,145
230,130 C 200,145 160,145 130,130 C 80,142 35,120 35,95 Z"
  )
  view-box;

/* Global values */
border-shape: inherit;
border-shape: initial;
border-shape: revert;
border-shape: revert-layer;
border-shape: unset;
```

The `border-shape` property may be specified using the keyword `none`, or one or two space-separated shape definitions, each consisting of a `<basic-shape>` value or a `<basic-shape>` value and a `<geometry-box>` value.

Values

`none`

Specifies that no border shape is defined. This is the initial value.

`<basic-shape>`

Defines the shape of the border using one of the `<basic-shape>` functions.

`<geometry-box>` Optional

Defines the reference box relative to which the border shape is drawn. If not included, the shape's reference geometry box defaults to:

- `half-border-box` if a single basic shape is specified, which means that any defined border is drawn on top of the shape path, with the path going down its center.
- `border-box` for the first (outer) shape and `padding-box` for the second (inner) shape if two basic shapes are specified. The border then occupies the area between the two shapes.

Description

The `border-shape` property can be applied to elements to create precisely-shaped containers such as speech bubbles, as shown here, or abstract tooltip designs, without having to use workarounds.

```
<img src="https://mdn.github.io/shared-assets/images/examples/leopard.jpg" />
<p>I am a leopard</p>
```

```
html {
  height: 100%;
}

body {
  margin: 0;
  height: inherit;
  display: flex;
```

```
  justify-content: center;
  align-items: center;
}

img {
  display: block;
  width: 300px;
  anchor-name: --leopard;
  position: relative;
  top: 45px;
}

p {
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
  padding: 15px 0;
  margin: 0 0 -30px -110px;
  background-color: chartreuse;
  border: 7px solid rgb(50 50 50);
  box-shadow: 5px 5px 10px rgb(0 0 0 / 0.75);

  width: 250px;
  height: 100px;
  border-shape: shape(
      from 50.08% 0%,
      curve to 97.71% 25.55% with 72.95% 0%/93.9% 8.76%,
      curve to 97.71% 62.04% with 100.76% 37.96%/100.76% 49.64%,
      curve to 50.08% 83.94% with 93.9% 75.18%/72.95% 83.94%,
      curve to 23.41% 82.48% with 36.75% 83.94%/27.22% 83.21%,
      curve to 17.7% 100% with 21.51% 87.59%/19.22% 94.16%,
      curve to 11.98% 82.48% with 16.17% 94.16%/13.89% 87.59%,
      curve to 1.31% 62.04% with 13.5% 81.75%/3.98% 76.64%,
      curve to 2.45% 25.55% with -0.59% 49.64%/-0.59% 37.96%,
      curve to 50.08% 0% with 6.26% 8.76%/27.22% 0%,
      close
    )
    content-box;

  text-align: center;
  line-height: 4;

  position: absolute;
  position-anchor: --leopard;
  bottom: anchor(top);
  left: anchor(right);
}
```

Note how the applied `border-shape` plays well with the `border` and `box-shadow` defined on the element — they follow the shape of the border.

Supported shape functions

You can define an element's `border-shape` (including that of an inline or pseudo-element) using one of the `<basic-shape>` functions. These functions include:

- `inset()`, `rect()`, and `xywh()`: Provide different ways to define basic rectangle shapes.
- `circle()`: Defines circle shapes.
- `ellipse()`: Defines ellipse shapes.
- `path()`: Defines any kind of shape using SVG path string syntax. SVG path syntax has limitations — it can use only pixel values and the path has to be defined as a single string, so custom properties can't be included via `var()`. You are advised to use `shape()` instead.
- `polygon()`: Defines any kind of polygon via pairs of vertex coordinates. If your desired shape includes smooth curves, you are advised to use `shape()`.
- `shape()`: Defines any kind of shape. The syntax of `shape()` is more CSS-compatible than that of `path()` and solves its shortcomings.

You can also include an optional `<geometry-box>` keyword after each `<basic-shape>` value to specify the reference box relative to which the shapes should be drawn.

Stroke and fill modes

The `border-shape` property has the following two modes:

- If a single `<basic-shape>` is provided in the value, that shape defines the shape of the element's border, with the defined border styles drawn as a stroke around the shape, as shown in the previous example. This is known as **stroke mode**.
- If two `<basic-shape>`s are provided in the value, the first shape defines the outer boundary of the border, the second shape defines the inner boundary of the border, and any defined border color fills the area between the two boundaries. This is known as **fill mode**.

> **Note:** Avoid defining a shape for the inner boundary that's larger than the outer boundary. If you do this, the border area does not render properly; you might end up with no border fill rendered or one shape rendered behind the other.

Effect on layout and rendering

The `border-shape` property creates a purely visual effect — the element's layout is still computed using the underlying rectangular box definition, and the content flow is not affected.

The element's content and background are clipped by the `border-shape` property (the inner shape in fill mode). When the specified `border-shape` is the same size or smaller than the content/background, you won't need to adjust the reference box, unless you want to create some kind of offset effect. However, when the specified `border-shape` is larger than the content/background, you will see gaps between the edge of the background and the shape(s). In such cases, you may need to use a different reference box to fix the display (see Handling border-shapes larger than the element's background for more information).

Limitations on border styles applied to border shapes

Not all border styles are applied to elements with a `border-shape` property set. The following list explains how each property is affected:

- `border-color`: This property is applied. However, when multiple element borders have different colors applied, the browser chooses the first edge with a border color, in the following order:
  - Block start edge
  - Inline start edge
  - Block end edge
  - Inline end edge

  The browser then applies that edge's border color to the entire rendered `border-shape`.
- `border-image`: Not applied.

- `border-style`: Not applied. All borders are rendered with a `solid` style.
- `border-width`: In stroke mode, `border-width` is applied directly to the rendered `border-shape`. When multiple edges have different border widths, the browser selects a width to apply to the entire border using the same process as described for `border-color`.

  In fill mode, the border area is defined by the difference between the areas of the outer and inner shapes; therefore, `border-width` doesn't have any direct effect on the width of the rendered border. However, it does have an indirect effect — it still affects the size of the reference boxes the shapes are drawn relative to (unless you set their `<geometry-box>` to `content-box` or `padding-box`). You therefore still need to be mindful of the `border-width` set on the underlying element while using fill mode.

As an example, if an element has the following declarations applied:

```
border-shape: rect(5px 198px 189px 0px round 20px);
border-bottom: 30px dashed blue;
border-left: 40px dotted hotpink;
border-right: 50px double yellow;
```

The rendered box will have a rectangular border with rounded corners. The border style will be `solid` because other styles are ignored. The border width and color will be `40px` and `hotpink`, respectively – this is because the `border-left` property applies styles to the inline start edge (assuming that the page has a horizontal `writing-mode`), which is the first edge with border styles; this gets chosen by the browser per the priority list described earlier.

Interaction with <code>border-radius</code> and <code>corner-shape</code>

The `border-radius` and `corner-shape` properties are incompatible with `border-shape`. When a `border-shape` is set on an element, any set `border-radius` is ignored, therefore `corner-shape` will also have no effect. The `border-shape` and `border-radius`/`corner-shape` properties have different effects, and are used separately.

If you want to use shaped corners in a `border-shape`, you will have to draw them directly as part of the shape.

<code>border-shape</code> versus <code>clip-path</code>

The `clip-path` property takes values similar to `border-shape` and produces similar effects. Both properties can be used to change the shape of an element and therefore the element's hit area, altering the boundary inside which `:hover` effects and pointer-related events activate.

However, there is a fundamental difference in how the two properties work:

- `clip-path` hides the area of the element that sits outside the region defined by the provided shape.
- `border-shape` changes the visual rendering of the element so that it sits inside the region defined by the provided shape.

This means `border-shape` clips the element's content while still letting the `overflow` property control its display, whereas `clip-path` hides the element's content altogether, so overflow control is not possible.

More significantly, properties such as `box-shadow` and `outline` do not follow the shape created by `clip-path` — it chops off the outside of the element, meaning that such effects are truncated in an ugly fashion or removed altogether. The `border-shape` property, on the other hand, creates a differently-shaped border that is neatly followed by such effects.

Handling border-shapes larger than the element's background

As mentioned earlier, one issue with `border-shape` is that when you define a shape that is larger than the element's content/background, you can end up with a gap between the background and the border.

The recommended approach to fixing this is to set the reference `<geometry-box>` to `content-box`. and then use `padding` to fill in the gaps between the content and the border. For example:

```
border-shape: shape(
    from 0% 0%,
    hline to 33%,
    arc by 33% 0% of 16% 20% small cw,
    hline to 100%,
    line to 100% 33%,
    arc by 0% 33% of 20% 16% small cw,
    line to 100% 100%,
    hline to 66%,
    arc by -33% 0% of 16% 20% small ccw,
    hline to 0%,
    line to 0% 66%,
    arc by 0% -33% of 20% 16% small ccw,
    close
  )
  content-box;
padding: 24px;
```

This way, the `padding` will be set outside the shape, causing it to get smaller, and forcing the background to fill up the parts of the shape extending outside the content area. You can see this technique in action in our irregular jigsaw piece navigation menu example.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
border-shape =
  none                                |
  [ <basic-shape> <geometry-box>? ]{1,2}

<basic-shape> =
  <basic-shape-rect>  |
  <circle()>          |
  <ellipse()>         |
  <polygon()>         |
  <path()>            |
  <shape()>

<geometry-box> =
  <shape-box>  |
  fill-box     |
  stroke-box   |
  view-box

<basic-shape-rect> =
```

```
  <inset()>  |
  <rect()>   |
  <xywh()>

<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<path()> =
  path( <'fill-rule'>? , <string> )

<shape()> =
  shape( <'fill-rule'>? from <position> , <shape-command># )

<shape-box> =
  <visual-box>     |
  margin-box       |
  half-border-box

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<fill-rule> =
  nonzero  |
  evenodd

<length-percentage> =
  <length>      |
  <percentage>

<shape-command> =
  <move-command>             |
  <line-command>             |
  close                      |
  <horizontal-line-command>  |
  <vertical-line-command>    |
  <curve-command>            |
  <smooth-command>           |
  <arc-command>

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<move-command> =
  move <command-end-point>

<line-command> =
  line <command-end-point>

<horizontal-line-command> =
```

```
  hline [ to [ <length-percentage> | left | center | right | x-start | x-end ] | by <length-percentage> ]

<vertical-line-command> =
  vline [ to [ <length-percentage> | top | center | bottom | y-start | y-end ] | by <length-percentage> ]

<curve-command> =
  curve [ [ to <position> with <control-point> [ / <control-point> ]? ] | [ by <coordinate-pair> with <relative-control-point> [ / <relative-control-point> ]? ] ]

<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Box Model Module Level 4, CSS Fill and Stroke Module Level 3, CSS Images Module Level 3, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>border-shape</code> stroke usage

This example shows how to use `border-shape` in stroke mode.

HTML

The markup for this example contains a single `<p>` element.

```
<p>Circle</p>
```

CSS

```
html {
  height: 100%;
}

body {
  margin: 0;
  height: inherit;
  display: flex;
  justify-content: center;
  align-items: center;
}

p {
  box-sizing: border-box;
  font-family: "Helvetica", "Arial";
  font-size: 1.6rem;
  padding: 40px 50px;
  background-color: chartreuse;
  display: flex;
  align-items: center;
}
```

We give the box a `width` of `fit-content` and an `aspect-ratio` of `1/1` to neatly fit the content inside a square. We also set a thick black `border` and a `box-shadow` before setting a `border-shape` of `circle(50%)` to create a circular border that neatly fits the content and background.

```
p {
  width: fit-content;
  aspect-ratio: 1/1;
  border: 15px solid black;
  box-shadow: 5px 5px 10px rgb(0 0 0 / 0.5);
  border-shape: circle(50%);
}
```

Result

Note how the `border` and `box-shadow` neatly follow the defined shape.

Basic <code>border-shape</code> fill usage

This example builds on the previous one, showing how to use `border-shape` in fill mode to create an irregular filled border.

The HTML is the same as in the previous example.

CSS

The CSS is the same as in the previous example, except this time we color the border `hotpink`, and we include two `<basic-shape>` definitions inside the `border-shape` value. There's an outer rectangle that covers the whole area of the content and an inner circle that's the same as in the previous example.

```
p {
  width: fit-content;
  aspect-ratio: 1/1;
  border: 15px solid hotpink;
  box-shadow: 5px 5px 10px rgb(0 0 0 / 0.5);
  border-shape: rect(0% 100% 100% 0% round 20px) circle(50%);
}
```

Result

Note how this time, the border covers the area between the rectangle and the circle, and it adopts the color set in the `border` declaration.

Comparing <code>border-shape</code> values

In this example, we let you select from different `border-shape` values to apply to an element, so you can compare and contrast how they render.

HTML

The HTML is similar to that in the previous example, except this time we include a bit more text in the `<p>` element. We've also included a `<select>` element so you can choose different classes to apply to the `<p>` via JavaScript (we've hidden both the `<select>` and the JavaScript for brevity).

```
<p>Blessed are the cheesemakers.</p>
```

```
<form>
  <label for="shape-select">Select border-shape type:</label>
  <br />
  <select id="shape-select">
    <option>circle</option>
    <option selected>ellipse</option>
    <option>inset</option>
    <option>path</option>
    <option>polygon</option>
    <option>rect</option>
    <option>shape</option>
    <option>two-polygons</option>
    <option>xywh</option>
  </select>
</form>
```

The classes set different `border-shape` values on the `<p>` element. To begin with, a `class` of `ellipse` is set on the `<p>` element, so initially it has an `ellipse()` `border-shape`.

```
const box = document.querySelector("p");
const select = document.querySelector("select");

select.addEventListener("change", selectClass);

function selectClass() {
  box.className = select.value;
}

selectClass();
```

CSS

In the CSS, we give the box a `width` of `550px`, a thick black `border`, and a `box-shadow`.

```
p {
  width: 550px;
  border: 15px solid black;
  box-shadow: 5px 5px 10px rgb(0 0 0 / 0.5);
}
```

Next, we define the rules for each of the classes that will be applied when you select the different options in the `<select>` element:

```
.circle {
  border-shape: circle(60%);
}

.ellipse {
  border-shape: ellipse(50% 40%);
}

.inset {
  border-shape: inset(10px 20px 10px 20px round 20px);
}

.path {
  border-shape: path(
      "M 35,95 C 35,50 60,15 100,20 C 120,5 160,5 180,22 C 200,5 250,5 270,22 C 295,5 340,5 360,22 C 460,10 477,35 496,75 C 515,157 450,120 430,128 C 400,145 360,145 330,130 C 300,145 260,145
230,130 C 200,145 160,145 130,130 C 80,142 35,120 35,95 Z"
    )
    view-box;
}

.polygon {
  border-shape: polygon(0% 60%, 0% 85%, 8% 100%, 18% 88%, 30% 100%, 42% 88%, 55% 100%, 68% 88%, 80% 100%, 86% 88%, 90% 75%, 100% 60%, 90% 30%, 85% 5%, 75% 18%, 65% 3%, 52% 16%, 40% 3%, 27% 16%,
15% 3%, 5% 18%) view-box;
}

.rect {
  border-shape: rect(10px 500px 130px 20px round 20px);
}

.shape {
  border-shape: shape(
    from 0% 64.5%,
    curve to 15.71% 8.26% with 0% 30.76%/6.04% 4.51%,
    curve to 35.05% 9.76% with 20.55% -2.99%/30.21% -2.99%,
    curve to 56.8% 9.76% with 39.88% -2.99%/51.97% -2.99%,
```

```
    curve to 78.56% 9.76% with 62.84% -2.99%/73.72% -2.99%,
    curve to 97.89% 49.5% with 87.02% 0.76%/97.89% 19.51%,
    curve to 95.47% 89.25% with 101.52% 60.75%/100.31% 83.25%,
    curve to 71.3% 90.75% with 88.22% 102%/78.56% 102%,
    curve to 47.13% 90.75% with 64.05% 102%/54.38% 102%,
    curve to 22.96% 90.75% with 39.88% 102%/30.21% 102%,
    curve to 0% 64.5% with 10.88% 99.75%/0% 83.25%,
    close
  );
}

.two-polygons {
  border-shape: polygon(0% 60%, 0% 85%, 8% 100%, 18% 88%, 30% 100%, 42% 88%, 55% 100%, 68% 88%, 80% 105%, 86% 88%, 91% 75%, 101% 60%, 93% 30%, 86% 5%, 75% 18%, 65% 3%, 52% 16%, 40% 3%, 27% 16%,
15% 3%, 5% 18%)
    polygon(0% 55%, 0% 90%, 6% 104%, 17% 93%, 30% 100%, 43% 93%, 56% 102%, 69% 93%, 81% 102%, 88% 93%, 94% 78%, 100% 58%, 94% 24%, 88% -2%, 76% 13%, 64% -4%, 51% 11%, 39% -4%, 26% 11%, 13% -4%,
3% 13%);
}

.xywh {
  border-shape: xywh(5% 5% 90% 90% round 20px);
}
```

```
form {
  position: absolute;
  border: 2px solid black;
  background: white;
  padding: 5px;
  bottom: 0;
  right: 0;
}

form select {
  width: 100%;
  padding: 2px 5px;
  margin-top: 10px;
}
```

Result

Select different options from the drop-down menu to see how the different border shapes render. At any point, feel free to use your browser's developer tools to inspect the `border-shape` values applied to the `<p>` element and edit them to get an idea of how the values work.

Animating a <code>border-shape</code>

This example demonstrates how you can animate the `border-shape` property.

HTML

The same `<p>` element is used as in the previous example, except this time we've included a `tabindex` attribute so it can be focused via the keyboard.

```
<p tabindex="0">Blessed are the cheesemakers.</p>
```

CSS

For this example, we apply a `polygon()` `border-shape` to the `<p>`.

```
p {
  width: 550px;
  border: 15px solid black;
  box-shadow: 5px 5px 10px rgb(0 0 0 / 0.5);
  border-shape: polygon(0% 60%, 0% 85%, 8% 100%, 18% 88%, 30% 100%, 42% 88%, 55% 100%, 68% 88%, 80% 100%, 86% 88%, 90% 75%, 100% 60%, 90% 30%, 85% 5%, 75% 18%, 65% 3%, 52% 16%, 40% 3%, 27% 16%,
15% 3%, 5% 18%) view-box;
}
```

We also set an `animation` on the `<p>` element's `:hover` and `:focus` states, so that when the element is hovered or focused, it smoothly animates back and forth between two polygon border shapes for infinite iterations.

```
p:hover,
p:focus {
  animation: morph 1s ease-in-out infinite alternate;
}

@keyframes morph {
  from {
    border-shape: polygon(0% 60%, 0% 85%, 8% 100%, 18% 88%, 30% 100%, 42% 88%, 55% 100%, 68% 88%, 80% 100%, 86% 88%, 90% 75%, 100% 60%, 90% 30%, 85% 5%, 75% 18%, 65% 3%, 52% 16%, 40% 3%, 27%
16%, 15% 3%, 5% 18%) view-box;
  }
  to {
    border-shape: polygon(0% 55%, 0% 90%, 6% 104%, 17% 93%, 30% 100%, 43% 93%, 56% 102%, 69% 93%, 81% 102%, 88% 93%, 94% 78%, 100% 58%, 94% 24%, 88% -2%, 76% 13%, 64% -4%, 51% 11%, 39% -4%, 26%
11%, 13% -4%, 3% 13%) view-box;
  }
}
```

Result

Hover over or focus the paragraph to see the animation.

```
@supports not (border-shape: circle(50%)) {
  body::before {
    content: "Your browser does not support the 'border-shape' property.";
    font-family: sans-serif;
    background-color: wheat;
    padding: 1rem 0;
    text-align: center;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
```

```
  }
}
```

See also

- `border`
- `corner-shape`
- Creating an irregular nav menu with border-shape
- CSS borders and box decorations module
- CSS backgrounds and borders module
- border-shape: the future of the non-rectangular web by Una Kravets (2026)
- Get Ready For the Powerful CSS border-shape Property! on CSS Tricks (2026)

border-spacing CSS property

The **`border-spacing`** CSS property sets the distance between the borders of adjacent cells in a `<table>`. This property applies only when `border-collapse` is `separate`.

Try it

```
border-spacing: 0;
```

```
border-spacing: 5px;
```

```
border-spacing: 5px 1rem;
```

```
<section class="default-example" id="default-example">
  <table class="transition-all" id="example-element">
    <tbody>
      <tr>
        <td>Cell 1.1</td>
        <td>Cell 1.2</td>
      </tr>
      <tr>
        <td>Cell 2.1</td>
        <td>Cell 2.2</td>
      </tr>
      <tr>
        <td>Cell 3.1</td>
        <td>Cell 3.2</td>
      </tr>
    </tbody>
  </table>
</section>
```

```
table {
  width: 15rem;
  table-layout: fixed;
}

td {
  border: 5px solid;
  border-color: crimson dodgerblue;
  padding: 0.75rem;
}
```

Syntax

```
/* <length> */
border-spacing: 2px;

/* horizontal <length> | vertical <length> */
border-spacing: 1cm 2em;

/* Global values */
border-spacing: inherit;
border-spacing: initial;
border-spacing: revert;
border-spacing: revert-layer;
border-spacing: unset;
```

The `border-spacing` property may be specified as either one or two values.

- When **one** `<length>` value is specified, it defines both the horizontal and vertical spacings between cells.
- When **two** `<length>` values are specified, the first value defines the horizontal spacing between cells (i.e., the space between cells in adjacent *columns*), and the second value defines the vertical spacing between cells (i.e., the space between cells in adjacent *rows*).

Values

`<length>`

The size of the spacing as a fixed value.

Description

The `border-spacing` value is also used along the outside edge of the table, where the distance between the table's border and the cells in the first/last column or row is the sum of the relevant (horizontal or vertical) `border-spacing` and the relevant (top, right, bottom, or left) `padding` on the table.

> **Note:** The `border-spacing` property is equivalent to the deprecated `cellspacing` attribute of the `<table>` element, except that `border-spacing` has an optional second value that can be used to set different horizontal and vertical spacing.

Formal definition

Initial value	`0`
Applies to	`table` and `inline-table` elements

Inherited	yes
Computed value	two absolute lengths
Animation type	by computed value type

Formal syntax

```
border-spacing =
  <length [0,∞]>{1,2}
```

This syntax reflects the latest standard as per CSS Table Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Spacing and padding table cells

This example applies a spacing of `.5em` vertically and `1em` horizontally between a table's cells. Note how, along its outside edges, the table's `padding` values are added to its `border-spacing` values.

HTML

```
<table>
  <tbody>
    <tr>
      <td>1</td>
      <td>2</td>
      <td>3</td>
    </tr>
    <tr>
      <td>4</td>
      <td>5</td>
      <td>6</td>
    </tr>
    <tr>
      <td>7</td>
      <td>8</td>
      <td>9</td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  border-spacing: 1em 0.5em;
  padding: 0 2em 1em 0;
  border: 1px solid orange;
}

td {
  width: 1.5em;
  height: 1.5em;
  background: #d2d2d2;
  text-align: center;
  vertical-align: middle;
}
```

Result

See also

- `border-collapse`, `border-style`
- The `border-spacing` property alters the appearance of the `<table>` HTML element.
- CSS table module

border-start-end-radius CSS property

The **`border-start-end-radius`** CSS property defines a logical border radius on an element, which maps to a physical border radius depending on the element's `writing-mode`, `direction`, and `text-orientation`. This is useful when building styles to work regardless of the text orientation and writing mode.

This property affects the corner between the block-start and the inline-end sides of the element. For instance, in a `horizontal-tb` writing mode with `ltr` direction, it corresponds to the `border-top-right-radius` property.

Try it

```
border-start-end-radius: 80px 80px;
```

```
border-start-end-radius: 250px 100px;
direction: rtl;
```

```
border-start-end-radius: 50%;
writing-mode: vertical-lr;
```

```
border-start-end-radius: 50%;
writing-mode: vertical-rl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a top right rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
```

```
  color: white;
  padding: 10px;
}
```

Syntax

```
/* <length> values */
/* With one value the corner will be a circle */
border-start-end-radius: 10px;
border-start-end-radius: 1em;

/* With two values the corner will be an ellipse */
border-start-end-radius: 1em 2em;

/* Global values */
border-start-end-radius: inherit;
border-start-end-radius: initial;
border-start-end-radius: revert;
border-start-end-radius: revert-layer;
border-start-end-radius: unset;
```

Values

`<length-percentage>`

Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-start-end-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border radius with vertical text

HTML

```
<div>
  <p class="exampleText">Example</p>
</div>
```

CSS

```
div {
  background-color: rebeccapurple;
  width: 120px;
  height: 120px;
  border-start-end-radius: 10px;
}

.exampleText {
  writing-mode: vertical-rl;
  padding: 10px;
  background-color: white;
  border-start-end-radius: 10px;
}
```

Results

See also

- CSS Logical Properties and Values
- The mapped physical property: `border-bottom-left-radius`
- `writing-mode`, `direction`, `text-orientation`

border-start-start-radius CSS property

The **`border-start-start-radius`** CSS property defines a logical border radius on an element, which maps to a physical border radius that depends on the element's `writing-mode`, `direction`, and `text-orientation`. This is useful when building styles to work regardless of the text orientation and writing mode.

This property affects the corner between the block-start and the inline-start sides of the element. For instance, in a `horizontal-tb` writing mode with `ltr` direction, it corresponds to the `border-top-left-radius` property.

Try it

```
border-start-start-radius: 80px 80px;
```

```
border-start-start-radius: 250px 100px;
direction: rtl;
```

```
border-start-start-radius: 50%;
writing-mode: vertical-lr;
```

```
border-start-start-radius: 50%;
writing-mode: vertical-rl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a top left rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* <length> values */
/* With one value the corner will be a circle */
border-start-start-radius: 10px;
border-start-start-radius: 1em;

/* With two values the corner will be an ellipse */
border-start-start-radius: 1em 2em;

/* Global values */
border-start-start-radius: inherit;
border-start-start-radius: initial;
border-start-start-radius: revert;
border-start-start-radius: revert-layer;
border-start-start-radius: unset;
```

Values

`<length-percentage>`

> Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-start-start-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Border radius with vertical text

HTML

```
<div>
  <p class="exampleText">Example</p>
</div>
```

CSS

```
div {
  background-color: rebeccapurple;
  width: 120px;
  height: 120px;
  border-start-start-radius: 10px;
}

.exampleText {
  writing-mode: vertical-rl;
  padding: 10px;
  background-color: white;
  border-start-start-radius: 10px;
}
```

Results

See also

- CSS Logical Properties and Values
- The mapped physical property: `border-top-left-radius`
- `writing-mode`, `direction`, `text-orientation`

border-style CSS property

The **`border-style`** CSS shorthand property sets the line style for all four sides of an element's border.

Try it

```
border-style: none;
```

```
border-style: dotted;
```

```
border-style: inset;
```

```
border-style: dashed solid;
```

```
border-style: dashed double none;
```

```
border-style: dashed groove none dotted;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}

body {
  background-color: white;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-bottom-style`
- `border-left-style`
- `border-right-style`
- `border-top-style`

Syntax

```
/* Keyword values */
border-style: none;
border-style: hidden;
border-style: dotted;
border-style: dashed;
border-style: solid;
border-style: double;
border-style: groove;
border-style: ridge;
border-style: inset;
border-style: outset;

/* top and bottom | left and right */
border-style: dotted solid;
```

```
/* top | left and right | bottom */
border-style: hidden double dashed;

/* top | right | bottom | left */
border-style: none solid dotted dashed;

/* Global values */
border-style: inherit;
border-style: initial;
border-style: revert;
border-style: revert-layer;
border-style: unset;
```

The `border-style` property may be specified using one, two, three, or four values.

- When **one** value is specified, it applies the same style to **all four sides**.
- When **two** values are specified, the first style applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first style applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the styles apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Each value is a keyword chosen from the list below.

Values

`<line-style>`

Describes the style of the border. It can have the following values:

`none`

Like the `hidden` keyword, displays no border. Unless a `background-image` is set, the used value of the same side's `border-width` will be `0`, even if the specified value is something else. In the case of table cell and border collapsing, the `none` value has the *lowest* priority: if any other conflicting border is set, it will be displayed.

`hidden`

Like the `none` keyword, displays no border. Unless a `background-image` is set, the used value of the same side's `border-width` will be `0`, even if the specified value is something else. In the case of table cell and border collapsing, the `hidden` value has the *highest* priority: if any other conflicting border is set, it won't be displayed.

`dotted`

Displays a series of rounded dots. The spacing of the dots is not defined by the specification and is implementation-specific. The radius of the dots is half the computed value of the same side's `border-width`.

`dashed`

Displays a series of short square-ended dashes or line segments. The exact size and length of the segments are not defined by the specification and are implementation-specific.

`solid`

Displays a single, straight, solid line.

`double`

Displays two straight lines that add up to the pixel size defined by `border-width`.

`groove`

Displays a border with a carved appearance. It is the opposite of `ridge`.

`ridge`

Displays a border with an extruded appearance. It is the opposite of `groove`.

`inset`

Displays a border that makes the element appear embedded. It is the opposite of `outset`. When applied to a table cell with `border-collapse` set to `collapsed`, this value behaves like `ridge`.

`outset`

Displays a border that makes the element appear embossed. It is the opposite of `inset`. When applied to a table cell with `border-collapse` set to `collapsed`, this value behaves like `groove`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-top-style`: none • `border-right-style`: none • `border-bottom-style`: none • `border-left-style`: none
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-top-style`: as specified • `border-right-style`: as specified • `border-bottom-style`: as specified • `border-left-style`: as specified
Animation type	discrete

Formal syntax

```
border-style =
  <'border-top-style'>{1,4}

<border-top-style> =
  <line-style>
```

```
<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

All property values

Here is an example of all the property values.

HTML

```
<pre class="b1">none</pre>
<pre class="b2">hidden</pre>
<pre class="b3">dotted</pre>
<pre class="b4">dashed</pre>
<pre class="b5">solid</pre>
<pre class="b6">double</pre>
<pre class="b7">groove</pre>
<pre class="b8">ridge</pre>
<pre class="b9">inset</pre>
<pre class="b10">outset</pre>
```

CSS

```
pre {
  height: 80px;
  width: 120px;
  margin: 20px;
  padding: 20px;
  display: inline-block;
  background-color: palegreen;
  border-width: 5px;
  box-sizing: border-box;
}

/* border-style example classes */
.b1 {
  border-style: none;
}

.b2 {
  border-style: hidden;
}

.b3 {
  border-style: dotted;
}

.b4 {
  border-style: dashed;
}

.b5 {
  border-style: solid;
}

.b6 {
  border-style: double;
}

.b7 {
  border-style: groove;
}

.b8 {
  border-style: ridge;
}

.b9 {
  border-style: inset;
}

.b10 {
  border-style: outset;
}
```

Result

See also

- The border-related shorthand CSS properties: `border`, `border-width`, `border-color`, `border-radius`

border-top CSS property

The **`border-top`** CSS shorthand property sets all the properties of an element's top border.

Try it

```
border-top: solid;
```

```
border-top: dashed red;
```

```
border-top: 1rem solid;
```

```
border-top: thick double #32a1ce;
```

```
border-top: 4mm ridge rgb(211 220 50 / 0.6);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: darkmagenta;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-top-color`
- `border-top-style`
- `border-top-width`

Syntax

```
border-top: 1px;
border-top: 2px dotted;
border-top: medium dashed green;

/* Global values */
border-top: inherit;
border-top: initial;
border-top: revert;
border-top: revert-layer;
border-top: unset;
```

The three values of the shorthand property can be specified in any order, and one or two of them may be omitted.

Values

`<br-width>`

See `border-top-width`.

`<br-style>`

See `border-top-style`.

`<color>`

See `border-top-color`.

Description

As with all shorthand properties, `border-top` always sets the values of all of the properties that it can set, even if they are not specified. It sets those that are not specified to their default values. Consider the following code:

```
border-top-style: dotted;
border-top: thick green;
```

It is actually the same as this one:

```
border-top-style: dotted;
border-top: none thick green;
```

The value of `border-top-style` given before `border-top` is ignored. Since the default value of `border-top-style` is `none`, not specifying the `border-style` part results in no border.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-top-width`: `medium` • `border-top-style`: `none` • `border-top-color`: `currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-top-width`: the absolute `<length>`, snapped as a line width • `border-top-style`: as specified • `border-top-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-top-width`: a length • `border-top-style`: discrete • `border-top-color`: a color

Formal syntax

```
border-top =
  <line-width> ||
  <line-style> ||
  <color>

<line-width> =
  <length [0,∞]> |
  hairline       |
  thin           |
  medium         |
  thick

<line-style> =
  none   |
  hidden |
  dotted |
  dashed |
  solid  |
  double |
  groove |
  ridge  |
  inset  |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying a top border

HTML

```
<div>This box has a border on the top side.</div>
```

CSS

```
div {
  border-top: 4px dashed blue;
  background-color: gold;
  height: 100px;
  width: 100px;
  font-weight: bold;
  text-align: center;
}
```

Results

See also

- `border`
- `border-block`
- `outline`

border-top-color CSS property

The **`border-top-color`** CSS property sets the color of an element's top border. It can also be set with the shorthand CSS properties `border-color` or `border-top`.

Try it

```
border-top-color: red;
```

```
border-top-color: #32a1ce;
```

```
border-top-color: rgb(170 50 220 / 0.6);
```

```
border-top-color: hsl(60 90% 50% / 0.8);
```

```
border-top-color: transparent;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* <color> values */
border-top-color: red;
border-top-color: #ffbb00;
border-top-color: rgb(255 0 0);
border-top-color: hsl(100deg 50% 25% / 75%);
border-top-color: currentColor;
border-top-color: transparent;

/* Global values */
border-top-color: inherit;
border-top-color: initial;
```

```
border-top-color: revert;
border-top-color: revert-layer;
border-top-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The color of the top border.

Formal definition

Initial value	`currentcolor`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
border-top-color =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>     |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A div with a border

HTML

```
<div class="my-box">
  <p>
    This is a box with a border around it. Note which side of the box is
    <span class="red-text">red</span>.
  </p>
</div>
```

CSS

```
.my-box {
  border: solid 0.3em gold;
  border-top-color: red;
  width: auto;
}

.red-text {
  color: red;
}
```

Result

See also

- The border-related CSS shorthand properties: `border`, `border-top`, and `border-color`.
- The color-related CSS properties for the other borders: `border-right-color`, `border-bottom-color`, and `border-left-color`.
- The other border-related CSS properties applying to the same border: `border-top-style` and `border-top-width`.
- The default `currentColor` color value.

border-top-left-radius CSS property

The **`border-top-left-radius`** CSS property rounds the top-left corner of an element by specifying the radius (or the radius of the semi-major and semi-minor axes) of the ellipse defining the curvature of the corner.

Try it

```
border-top-left-radius: 80px 80px;
```

```
border-top-left-radius: 250px 100px;
```

```
border-top-left-radius: 50%;
```

```
border-top-left-radius: 50%;
border: black 10px double;
background-clip: content-box;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
```

```
    This is a box with a top left rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* the corner is a circle */
/* border-top-left-radius: radius */
border-top-left-radius: 3px;

/* the corner is an ellipse */
/* border-top-left-radius: horizontal vertical */
border-top-left-radius: 0.5em 1em;

border-top-left-radius: inherit;

/* Global values */
border-top-left-radius: inherit;
border-top-left-radius: initial;
border-top-left-radius: revert;
border-top-left-radius: revert-layer;
border-top-left-radius: unset;
```

With one value:

- the value is a `<length>` or a `<percentage>` denoting the radius of the circle to use for the border in that corner.

With two values:

- the first value is a `<length>` or a `<percentage>` denoting the horizontal semi-major axis of the ellipse to use for the border in that corner.
- the second value is a `<length>` or a `<percentage>` denoting the vertical semi-major axis of the ellipse to use for the border in that corner.

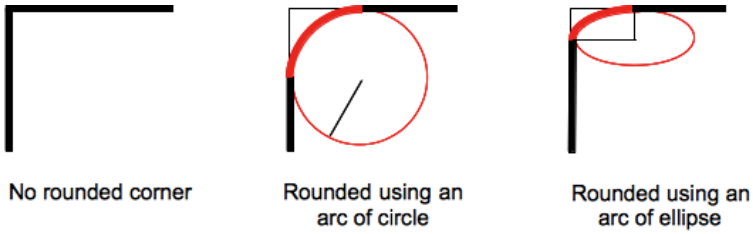
Values

`<length-percentage>`

> Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Description

The rounding can be a circle or an ellipse, or if one of the values is `0`, no rounding is done and the corner is square.



A background, being an image or a color, is clipped at the border, even a rounded one; the exact location of the clipping is defined by the value of the `background-clip` property.

> **Note:** If the value of this property is not set in a `border-radius` shorthand property that is applied to the element after the `border-top-left-radius` CSS property, the value of this property is then reset to its initial value by the shorthand property.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-top-left-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?
```

```
<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Arc of a circle

A single `<length>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-top-left-radius: 40px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Arc of an ellipse

Two different `<length>` values produce an arc of an ellipse.

```
<div></div>
```

```
div {
  border-top-left-radius: 40px 20px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Square element with percentage radius

A square element with a single `<percentage>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-top-left-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Non-square element with percentage radius

A non-square element with a single `<percentage>` value produces an arc of an ellipse.

```
<div></div>
```

```
div {
  border-top-left-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 200px;
  height: 100px;
}
```

See also

- `border-radius` shorthand property
- `border-top-right-radius`, `border-bottom-right-radius`, and `border-bottom-left-radius`

border-top-right-radius CSS property

The **`border-top-right-radius`** CSS property rounds the top-right corner of an element by specifying the radius (or the radius of the semi-major and semi-minor axes) of the ellipse defining the curvature of the corner.

Try it

```
border-top-right-radius: 80px 80px;
```

```
border-top-right-radius: 250px 100px;
```

```
border-top-right-radius: 50%;
```

```
border-top-right-radius: 50%;
border: black 10px double;
background-clip: content-box;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a top right rounded corner.
  </div>
</section>
```

```
#example-element {
  width: 80%;
  height: 80%;
  display: flex;
  justify-content: center;
  flex-direction: column;
  background-color: #5b6dcd;
  color: white;
  padding: 10px;
}
```

Syntax

```
/* the corner is a circle */
/* border-top-right-radius: radius */
border-top-right-radius: 3px;

/* the corner is an ellipse */
/* border-top-right-radius: horizontal vertical */
border-top-right-radius: 0.5em 1em;

border-top-right-radius: inherit;

/* Global values */
border-top-right-radius: inherit;
border-top-right-radius: initial;
border-top-right-radius: revert;
border-top-right-radius: revert-layer;
border-top-right-radius: unset;
```

With one value:

- the value is a `<length>` or a `<percentage>` denoting the radius of the circle to use for the border in that corner.

With two values:

- the first value is a `<length>` or a `<percentage>` denoting the horizontal semi-major axis of the ellipse to use for the border in that corner.
- the second value is a `<length>` or a `<percentage>` denoting the vertical semi-major axis of the ellipse to use for the border in that corner.

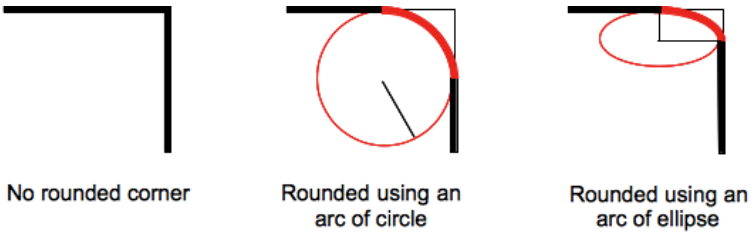
Values

`<length-percentage>`

> Denotes the size of the circle radius or the semi-major and semi-minor axes of the ellipse. As absolute length it can be expressed in any unit allowed by the CSS `<length>` data type. Percentages for the horizontal axis refer to the width of the box, percentages for the vertical axis refer to the height of the box. Negative values are invalid.

Description

The rounding can be a circle or an ellipse, or if one of the values is `0` no rounding is done and the corner is square.



A background, being an image or a color, is clipped at the border, even a rounded one; the exact location of the clipping is defined by the value of the `background-clip` property.

> **Note:** If the value of this property is not set in a `border-radius` shorthand property that is applied to the element after the `border-top-right-radius` CSS property, the value of this property is then reset to its initial value by the shorthand property.

Formal definition

Initial value	`0`
Applies to	all elements; but User Agents are not required to apply to `table` and `inline-table` elements when `border-collapse` is `collapse`. The behavior on internal table elements is undefined for the moment.. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the corresponding dimension of the border box
Computed value	two absolute `<length>`s or `<percentage>`s
Animation type	a length, percentage or calc();

Formal syntax

```
border-top-right-radius =
  <border-radius>

<border-radius> =
  <slash-separated-border-radius-syntax>  |
  <legacy-border-radius-syntax>

<slash-separated-border-radius-syntax> =
  <length-percentage [0,∞]> [ / <length-percentage [0,∞]> ]?

<legacy-border-radius-syntax> =
  <length-percentage [0,∞]>{1,2}

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Arc of a circle

A single `<length>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-top-right-radius: 40px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Arc of an ellipse

Two different `<length>` values produce an arc of an ellipse.

```
<div></div>
```

```
div {
  border-top-right-radius: 40px 20px;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Square element with percentage radius

A square element with a single `<percentage>` value produces an arc of a circle.

```
<div></div>
```

```
div {
  border-top-right-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 100px;
  height: 100px;
}
```

Non-square element with percentage radius

A non-square element with a single `<percentage>` value produces an arc of an ellipse.

```
<div></div>
```

```
div {
  border-top-right-radius: 40%;
  background-color: lightgreen;
  border: solid 1px black;
  width: 200px;
  height: 100px;
}
```

See also

- `border-radius` shorthand property
- `border-bottom-right-radius`, `border-bottom-left-radius`, and `border-top-left-radius`

border-top-style CSS property

The **`border-top-style`** CSS property sets the line style of an element's top `border`.

> **Note:** The specification doesn't define how borders of different styles connect in the corners.

Try it

```
border-top-style: none;
```

```
border-top-style: dotted;
```

```
border-top-style: dashed;
```

```
border-top-style: solid;
```

```
border-top-style: groove;
```

```
border-top-style: inset;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
```

```
  </div>
</section>
```

```
#example-element {
  background-color: #eeeeee;
  color: black;
  border: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}

body {
  background-color: white;
}
```

Syntax

```
/* Keyword values */
border-top-style: none;
border-top-style: hidden;
border-top-style: dotted;
border-top-style: dashed;
border-top-style: solid;
border-top-style: double;
border-top-style: groove;
border-top-style: ridge;
border-top-style: inset;
border-top-style: outset;

/* Global values */
border-top-style: inherit;
border-top-style: initial;
border-top-style: revert;
border-top-style: revert-layer;
border-top-style: unset;
```

The `border-top-style` property is specified as a single `<line-style>` keyword value.

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
border-top-style =
  <line-style>

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting border-top-style

HTML

```
<table>
  <tbody>
    <tr>
      <td class="b1">none</td>
      <td class="b2">hidden</td>
      <td class="b3">dotted</td>
      <td class="b4">dashed</td>
    </tr>
    <tr>
      <td class="b5">solid</td>
      <td class="b6">double</td>
      <td class="b7">groove</td>
      <td class="b8">ridge</td>
    </tr>
    <tr>
      <td class="b9">inset</td>
      <td class="b10">outset</td>
    </tr>
  </tbody>
</table>
```

CSS

```
/* Define look of the table */
table {
  border-width: 2px;
  background-color: #52e385;
```

```
}
tr,
td {
  padding: 3px;
}

/* border-top-style example classes */
.b1 {
  border-top-style: none;
}
.b2 {
  border-top-style: hidden;
}
.b3 {
  border-top-style: dotted;
}
.b4 {
  border-top-style: dashed;
}
.b5 {
  border-top-style: solid;
}
.b6 {
  border-top-style: double;
}
.b7 {
  border-top-style: groove;
}
.b8 {
  border-top-style: ridge;
}
.b9 {
  border-top-style: inset;
}
.b10 {
  border-top-style: outset;
}
```

Result

See also

- The other style-related border properties: `border-left-style`, `border-right-style`, `border-bottom-style`, and `border-style`.
- The other top-border-related properties: `border-top`, `border-top-color`, and `border-top-width`.

border-top-width CSS property

The **`border-top-width`** CSS property sets the width of the top border of an element.

Try it

```
border-top-width: thick;
```

```
border-top-width: 2em;
```

```
border-top-width: 4px;
```

```
border-top-width: 2ex;
```

```
border-top-width: 0;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* Keyword values */
border-top-width: thin;
border-top-width: medium;
border-top-width: thick;

/* <length> values */
border-top-width: 10em;
border-top-width: 3vmax;
border-top-width: 6px;

/* Global keywords */
border-top-width: inherit;
border-top-width: initial;
border-top-width: revert;
border-top-width: revert-layer;
border-top-width: unset;
```

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
border-top-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Element 1</div>
<div>Element 2</div>
```

CSS

```
div {
  border: 1px solid red;
  margin: 1em 0;
}

div:nth-child(1) {
  border-top-width: thick;
}
div:nth-child(2) {
  border-top-width: 2em;
}
```

Result

See also

- The other border-width-related CSS properties: `border-left-width`, `border-right-width`, `border-bottom-width`, and `border-width`.
- The other border-top-related CSS properties: `border`, `border-top`, `border-top-style`, and `border-top-color`.

border-width CSS property

The **`border-width`** CSS shorthand property sets the width of an element's border.

Try it

```
border-width: thick;
```

```
border-width: 1em;
```

```
border-width: 4px 1.25em;
```

```
border-width: 2ex 1.25ex 0.5ex;
```

```
border-width: 0 4px 8px 12px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: palegreen;
  color: black;
  border: 0 solid crimson;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `border-bottom-width`
- `border-left-width`
- `border-right-width`
- `border-top-width`

Syntax

```
/* Keyword values */
border-width: thin;
border-width: medium;
border-width: thick;

/* <length> values */
border-width: 4px;
border-width: 1.2rem;

/* top and bottom | left and right */
border-width: 2px 1.5em;

/* top | left and right | bottom */
border-width: 1px 2em 1.5cm;

/* top | right | bottom | left */
border-width: 1px 2em 0 4rem;

/* Global values */
border-width: inherit;
border-width: initial;
border-width: revert;
border-width: revert-layer;
border-width: unset;
```

The `border-width` property may be specified using one, two, three, or four values.

- When **one** value is specified, it applies the same width to **all four sides**.
- When **two** values are specified, the first width applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first width applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the widths apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<line-width>`

Defines the width of the border, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	as each of the properties of the shorthand: • `border-top-width`: `medium` • `border-right-width`: `medium` • `border-bottom-width`: `medium` • `border-left-width`: `medium`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `border-top-width`: the absolute `<length>`, snapped as a line width • `border-right-width`: the absolute `<length>`, snapped as a line width • `border-bottom-width`: the absolute `<length>`, snapped as a line width • `border-left-width`: the absolute `<length>`, snapped as a line width
Animation type	as each of the properties of the shorthand: • `border-top-width`: a length • `border-right-width`: a length • `border-bottom-width`: a length • `border-left-width`: a length

Formal syntax

```
border-width =
  <'border-top-width'>{1,4}

<border-top-width> =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A mix of values and lengths

HTML

```
<p id="one-value">one value: 6px wide border on all 4 sides</p>
<p id="two-values">
  two different values: 2px wide top and bottom border, 10px wide right and left
  border
</p>
<p id="three-values">
  three different values: 0.3em top, 9px bottom, and zero width right and left
</p>
<p id="four-values">
```

```
  four different values: "thin" top, "medium" right, "thick" bottom, and 1em
  left
</p>
```

CSS

```
#one-value {
  border: ridge #cccccc;
  border-width: 6px;
}
#two-values {
  border: solid red;
  border-width: 2px 10px;
}
#three-values {
  border: dotted orange;
  border-width: 0.3em 0 9px;
}
#four-values {
  border: solid lightgreen;
  border-width: thin medium thick 1em;
}
p {
  width: auto;
  margin: 0.25em;
  padding: 0.25em;
}
```

Result

See also

- The border-related shorthand properties: `border`, `border-style`, `border-color`
- The border-width-related properties: `border-bottom-width`, `border-left-width`, `border-right-width`, `border-top-width`

bottom CSS property

The **`bottom`** CSS property participates in setting the vertical position of a positioned element. This inset property has no effect on non-positioned elements.

Try it

```
bottom: 0;
```

```
bottom: 4em;
```

```
bottom: 10%;
```

```
bottom: 20px;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #264653;
  border: 4px solid #ffb500;
  color: white;
  position: absolute;
  width: 140px;
  height: 60px;
}
```

Syntax

```
/* <length> values */
bottom: 3px;
bottom: 2.4em;
bottom: calc(anchor(--my-anchor 50%) + 5px);
bottom: anchor-size(width);

/* <percentage>s of the height of the containing block */
bottom: 10%;

/* Keyword value */
bottom: auto;

/* Global values */
bottom: inherit;
bottom: initial;
bottom: revert;
```

```
bottom: revert-layer;
bottom: unset;
```

Values

`<length>`

A negative, null, or positive `<length>`:

- for *absolutely positioned elements*, it represents the distance to the bottom edge of the containing block.
- for *relatively positioned elements*, it represents the distance that the element is moved above its normal position.
- for *anchor-positioned elements*, the `anchor()` function resolves to a `<length>` value relative to the position of the associated *anchor element*'s top or bottom edge (see Using inset properties with `anchor()` function values), and the `anchor-size()` function resolves to a `<length>` value relative to the associated anchor element's width or height (see Setting element position based on anchor size).

`<percentage>`

A `<percentage>` of the containing block's height.

`auto`

Specifies that:

- for *absolutely positioned elements*, the position of the element is based on the `top` property, while `height: auto` is treated as a height based on the content; or if `top` is also `auto`, the element is positioned where it should vertically be positioned if it were a static element.
- for *relatively positioned elements*, the distance of the element from its normal position is based on the `top` property; or if `top` is also `auto`, the element is not moved vertically at all.

Description

The effect of `bottom` depends on how the element is positioned (i.e., the value of the `position` property):

- When `position` is set to `absolute` or `fixed`, the `bottom` property specifies the distance between the outer edge of the element's bottom margin and the outer edge of the containing block's bottom padding, or, in the case of anchor positioned elements when the `anchor()` function is used within the value, relative to the position of the specified `<anchor-side>` edge. The `bottom` property is compatible with the `top`, `bottom`, `start`, `end`, `self-start`, `self-end`, `center`, and `<percentage>` values.
- When `position` is set to `relative`, the `bottom` property specifies the distance the element's bottom edge is moved above its normal position.
- When `position` is set to `sticky`, the `bottom` property is used to compute the sticky-constraint rectangle.
- When `position` is set to `static`, the `bottom` property has *no effect*.

When both `top` and `bottom` are specified, `position` is set to `absolute` or `fixed`, *and* `height` is unspecified (either `auto` or `100%`) both the `top` and `bottom` distances are respected. In all other situations, if `height` is constrained in any way or `position` is set to `relative`, the `top` property takes precedence and the `bottom` property is ignored.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	refer to the height of the containing block
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length, percentage or calc();

Formal syntax

```
bottom =
  auto                |
  <length-percentage>  |
  <anchor()>           |
  <anchor-size()>

<length-percentage> =
  <length>     |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                 &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside       |
  outside      |
  top          |
  left         |
  right        |
  bottom       |
  start        |
  end          |
  self-start   |
  self-end     |
  <percentage>  |
  center

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Absolute and fixed positioning

This example demonstrates the difference in behavior of the `bottom` property, when `position` is `absolute` versus `fixed`.

HTML

```
<p>
  This<br />is<br />some<br />tall,<br />tall,<br />tall,<br />tall,<br />tall<br />content.
</p>
<div class="fixed"><p>Fixed</p></div>
<div class="absolute"><p>Absolute</p></div>
```

CSS

```
p {
  font-size: 30px;
  line-height: 2em;
}

div {
  width: 48%;
  text-align: center;
  background: rgb(55 55 55 / 20%);
  border: 1px solid blue;
}

.absolute {
  position: absolute;
  bottom: 0;
  left: 0;
}

.fixed {
  position: fixed;
  bottom: 0;
  right: 0;
}
```

Result

See also

- `top`, `left`, and `right`
- `inset` shorthand
- `inset-block-start`, `inset-block-end`, `inset-inline-start`, and `inset-inline-end`
- `inset-block` and `inset-inline` shorthands
- `position`
- CSS positioned layout module

box-align CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft, and has been replaced by a newer standard.

The **`box-align`** CSS property specifies how an element aligns its contents across its layout in a perpendicular direction. The effect of the property is only visible if there is extra space in the box.

See flexbox for information about the current standard.

The direction of layout depends on the element's orientation: horizontal or vertical.

Syntax

```
/* Keyword values */
box-align: start;
box-align: center;
box-align: end;
box-align: baseline;
box-align: stretch;

/* Global values */
box-lines: inherit;
box-lines: initial;
box-lines: unset;
```

Values

This property is specified as one of the following keyword values:

`start`

The box aligns contents at the start, leaving any extra space at the end.

`center`

The box aligns contents in the center, dividing any extra space equally between the start and the end.

`end`

The box aligns contents at the end, leaving any extra space at the start.

`baseline`

The box aligns the baselines of the contents (lining up the text). This only applies if the box's orientation is horizontal.

`stretch`

The box stretches the contents so that there is no extra space in the box.

Notes

The edge of the box designated the *start* for alignment purposes depends on the box's orientation:

- For horizontal elements, the *start* is the top edge.
- For vertical elements, the *start* is the left edge.

The edge opposite to the start is designated the *end*.

If the alignment is set using the element's `align` attribute, then the style is ignored.

Formal definition

Initial value	`stretch`
Applies to	elements with a CSS `display` value of `box` or `inline-box`
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-align =
  start    |
  center   |
  end      |
  baseline |
  stretch
```

Examples

Setting box alignment

```
<div class="example">
  <p>I will be second from the bottom of div.example, centered horizontally.</p>
  <p>I will be on the bottom of div.example, centered horizontally.</p>
</div>
```

```
div.example {
  display: box; /* As specified */
  display: -moz-box; /* Mozilla */
  display: -webkit-box; /* WebKit */

  /* Make this box taller than the children,
     so there is room for the box-pack */
  height: 400px;

  /* Make this box wider than the children
     so there is room for the box-align */
  width: 300px;

  /* Children should be oriented vertically */
  box-orient: vertical; /* As specified */
  -moz-box-orient: vertical; /* Mozilla */
  -webkit-box-orient: vertical; /* WebKit */

  /* Align children to the horizontal center of this box */
  box-align: center; /* As specified */
  -moz-box-align: center; /* Mozilla */
  -webkit-box-align: center; /* WebKit */

  /* Pack children to the bottom of this box */
  box-pack: end; /* As specified */
  -moz-box-pack: end; /* Mozilla */
  -webkit-box-pack: end; /* WebKit */
}

div.example > p {
  /* Make children narrower than their parent,
     so there is room for the box-align */
  width: 200px;
}
```

Specifications

Not part of any standard.

See also

- `align-items`, `box-orient`, `box-direction`, `box-pack`

box-decoration-break CSS property

The **`box-decoration-break`** CSS property specifies how an element's fragments should be rendered when broken across multiple lines, columns, or pages.

Try it

```
-webkit-box-decoration-break: slice;
box-decoration-break: slice;
```

```
-webkit-box-decoration-break: clone;
box-decoration-break: clone;
```

```
<section id="default-example">
  <div id="example-container">
    <span id="example-element">This text breaks across multiple lines.</span>
  </div>
</section>
```

```
#example-container {
  width: 14rem;
}

#example-element {
  background: linear-gradient(to bottom right, #6f6f6f, black);
  color: white;
  box-shadow:
    8px 8px 10px 0 #ff1492,
    -5px -5px 5px 0 blue,
    5px 5px 15px 0 yellow;
  padding: 0 1em;
  border-radius: 16px;
  border-style: solid;
  margin-left: 10px;
  font: 24px sans-serif;
  line-height: 2;
}
```

Syntax

```
/* Keyword values */
box-decoration-break: slice;
box-decoration-break: clone;

/* Global values */
box-decoration-break: inherit;
box-decoration-break: initial;
box-decoration-break: revert;
box-decoration-break: revert-layer;
box-decoration-break: unset;
```

Values

This property is specified as one of the following keyword values:

`slice`

> The element is initially rendered as if its box were not fragmented, after which the rendering for this hypothetical box is sliced into pieces for each line/column/page. Note that the hypothetical box can be different for each fragment since it uses its own height if the break occurs in the inline direction, and its own width if the break occurs in the block direction. See the CSS specification for details.

`clone`

> Each box fragment is rendered independently with the specified border, padding, and margin wrapping each fragment. The `border-radius`, `border-image`, and `box-shadow` are applied to each fragment independently. The background is also drawn independently for each fragment, which means that a background image with `background-repeat: no-repeat` may nevertheless repeat multiple times.

Description

The specified value will impact the appearance of the following properties:

- `background`
- `border`
- `border-image`
- `box-shadow`
- `clip-path`
- `margin`
- `padding`

Formal definition

Initial value	`slice`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-decoration-break =
  slice  |
  clone
```

This syntax reflects the latest standard as per CSS Fragmentation Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Inline box fragments

An inline element with a box decoration may have unexpected appearance when it contains line breaks due to the initial `slice` value. The following example shows the effect of adding `box-decoration-break: clone` to a `<span>` that contains `<br>` tags:

```
body {
  display: flex;
  background-color: grey;
  justify-content: space-around;
}

span {
  padding: 0em 1em;
  border-radius: 1rem;
```

```
  border-style: solid;
  margin: 1rem;
  font: 22px sans-serif;
  line-height: 2;
}
```

```
span {
  background: linear-gradient(to bottom right, yellow, green);
  box-shadow:
    8px 8px 10px 0px deeppink,
    -5px -5px 5px 0px blue,
    5px 5px 15px 0px yellow;
}

#clone {
  -webkit-box-decoration-break: clone;
  box-decoration-break: clone;
}
```

```
<p>
  <span>The<br />quick<br />orange fox</span>
</p>
<p>
  <span id="clone">The<br />quick<br />orange fox</span>
</p>
```

Block box fragments

The following example shows how block elements with box decoration look when they contain line breaks in a multi-column layout. Notice how the result of `box-decoration-break: slice` would be the equivalent of the first `<div>` if stacked vertically.

```
body {
  background-color: grey;
}
span {
  padding: 0em 2em;
  border-radius: 250px;
  border-style: solid;
  margin-left: 1em;
  font: 20px sans-serif;
  line-height: 1.5;
}
```

```
span {
  display: block;
  background: linear-gradient(to bottom right, yellow, green);
  box-shadow:
    inset 8px 8px 10px 0px deeppink,
    inset -5px -5px 5px 0px blue,
    inset 5px 5px 15px 0px yellow;
}
#base {
  width: 33%;
}
.columns {
  columns: 3;
}

.clone {
  -webkit-box-decoration-break: clone;
  box-decoration-break: clone;
}
```

```
<div id="base">
  <span>The<br />quick<br />orange fox</span>
</div>
<br />

<h2>'box-decoration-break: slice'</h2>
<div class="columns">
  <span>The<br />quick<br />orange fox</span>
</div>

<h2>'box-decoration-break: clone'</h2>
<div class="columns">
  <span class="clone">The<br />quick<br />orange fox</span>
</div>
```

See also

- `break-after`, `break-before`, `break-inside`

box-direction CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft, and has been replaced by a newer standard. The `-moz-box-direction` will only be used for XUL while the previous standard `box-direction` has been replaced by `flex-direction`. See flexbox for information about the current standard.

The **`box-direction`** CSS property specifies whether a box lays out its contents normally (from the top or left edge), or in reverse (from the bottom or right edge).

Syntax

```
/* Keyword values */
box-direction: normal;
```

```
box-direction: reverse;

/* Global values */
box-direction: inherit;
box-direction: initial;
box-direction: revert;
box-direction: revert-layer;
box-direction: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

The box lays out its contents from the start (the left or top edge).

`reverse`

The box lays out its contents from the end (the right or bottom edge).

Notes

The edge of the box designated the *start* for layout purposes depends on the box's orientation:

- For horizontal elements, the *start* is the top edge.
- For vertical elements, the *start* is the left edge.

The edge opposite to the start is designated the *end*.

If the direction is set using the element's `dir` attribute, then the style is ignored.

Formal definition

Initial value	`normal`
Applies to	elements with a CSS `display` value of `box` or `inline-box`
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-direction =
  normal   |
  reverse
```

Examples

Setting box direction

```
.example {
  /* bottom-to-top layout */
  -moz-box-direction: reverse; /* Mozilla */
  -webkit-box-direction: reverse; /* WebKit */
  box-direction: reverse; /* As specified */
}
```

Specifications

Not part of any standard.

See also

- `box-orient`
- `box-pack`
- `box-align`
- `flex-direction`

box-flex CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property for controlling parts of the XUL box model. It does not match either the old CSS flexible box layout Module drafts for `box-flex` (which were based on this property) or the behavior of `-webkit-box-flex` (which is based on those drafts). See flexbox for information about the current standard.

The **`-moz-box-flex`** and **`-webkit-box-flex`** CSS properties specify how a `-moz-box` or `-webkit-box` grows to fill the box that contains it, in the direction of the containing box's layout.

Syntax

```
/* <number> values */
-moz-box-flex: 0;
-moz-box-flex: 2;
-moz-box-flex: 3.5;
-webkit-box-flex: 0;
-webkit-box-flex: 2;
-webkit-box-flex: 3.5;

/* Global values */
-moz-box-flex: inherit;
-moz-box-flex: initial;
-moz-box-flex: revert;
-moz-box-flex: revert-layer;
-moz-box-flex: unset;
```

```
-webkit-box-flex: inherit;
-webkit-box-flex: initial;
-webkit-box-flex: revert;
-webkit-box-flex: revert-layer;
-webkit-box-flex: unset;
```

The `box-flex` property is specified as a `<number>`. If the value is 0, the box does not grow. If it is greater than 0, the box grows to fill a proportion of the available space.

Notes

The containing box allocates the available extra space in proportion to the flex value of each of the content elements.

Content elements that have zero flex do not grow.

If only one content element has nonzero flex, then it grows to fill the available space.

Content elements that have the same flex grow by the same absolute amounts.

If the flex value is set using the element's `flex` attribute, then the style is ignored.

To make XUL elements in a containing box the same size, set the containing box's `equalsize` attribute to the value `always`. This attribute does not have a corresponding CSS property.

A trick to make all content elements in a containing box the same size, is to give them all a fixed size (e.g., `height: 0`), and the same `box-flex` value greater than zero (e.g., `-moz-box-flex: 1`).

Formal definition

Initial value	`0`
Applies to	elements that are direct children of an element with a CSS `display` value of `-moz-box` or `-moz-inline-box` or `-webkit-box` or `-webkit-inline-box`
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-flex =
  <number>
```

Examples

Setting box-flex

```
<div class="example">
  <p>I will expand to fill extra space</p>
  <p>I will not expand</p>
</div>
```

```
div.example {
  display: -moz-box;
  display: -webkit-box;
  border: 1px solid black;
  width: 100%;
}
div.example > p:nth-child(1) {
  -moz-box-flex: 1; /* Mozilla */
  -webkit-box-flex: 1; /* WebKit */
  border: 1px solid black;
}
div.example > p:nth-child(2) {
  -moz-box-flex: 0; /* Mozilla */
  -webkit-box-flex: 0; /* WebKit */
  border: 1px solid black;
}
```

Specifications

Not part of any standard.

See also

- `box-orient`
- `box-pack`
- `box-direction`
- `flex`

box-flex-group CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft. It has been replaced in the specification. See flexbox for information about the current standard.

The **`box-flex-group`** CSS property assigns the flexbox's child elements to a flex group.

For flexible elements assigned to flex groups, the first flex group is 1 and higher values specify subsequent flex groups. The initial value is 1. When dividing up the box's extra space, the browser first considers all elements within the first flex group. Each element within that group is given extra space based on the ratio of that element's flexibility compared to the flexibility of other elements within the same flex group. If the space of all flexible children within the group has been increased to the maximum, the process repeats for the children within the next flex group, using any space left over from the previous flex group. Once there are no more flex groups, and there is still space remaining, the extra space is divided within the containing box according to the `box-pack` property.

If the box would overflow after the preferred space of the children has been computed, then space is removed from flexible elements in a manner similar to that used when adding extra space. Each flex group is examined in turn and space is removed according to the ratio of the flexibility of each element. Elements do not shrink below their minimum widths.

Syntax

```
/* <integer> values */
box-flex-group: 1;
box-flex-group: 5;

/* Global values */
box-flex-group: inherit;
box-flex-group: initial;
box-flex-group: unset;
```

The `box-flex-group` property is specified as any positive `<integer>`.

Formal definition

Initial value	`1`
Applies to	in-flow children of box elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-flex-group =
  <integer>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage example

In the original flexbox spec, `box-flex-group` could be used to assign flex children to different groups to distribute flexible space between:

```
article:nth-child(1) {
  -webkit-box-flex-group: 1;
}

article:nth-child(2) {
  -webkit-box-flex-group: 2;
}
```

This was only ever supported in WebKit-based browsers, with a prefix, and in subsequent versions of the spec this functionality does not have an equivalent. Instead, distribution of space inside the flex container is now handled using `flex-basis`, `flex-grow`, and `flex-shrink`.

Specifications

Not part of any standard.

See also

- `flex`
- `flex-basis`
- `flex-grow`
- `flex-shrink`

box-lines CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft. It has been replaced in the specification. See flexbox for information about the current standard.

The **`box-lines`** CSS property determines whether the box may have a single or multiple lines (rows for horizontally oriented boxes, columns for vertically oriented boxes).

By default a horizontal box will lay out its children in a single row, and a vertical box will lay out its children in a single column. This behavior can be changed using the `box-lines` property. The default value is `single`, which means that all elements will be placed in a single row or column, and any elements that don't fit will be considered overflow.

If a value of `multiple` is specified, however, then the box is allowed to expand to multiple lines (that is, multiple rows or columns) in order to accommodate all of its children. The box must attempt to fit its children on as few lines as possible by shrinking all elements down to their minimum widths or heights if necessary.

If the children in a horizontal box still do not fit on a line after being reduced to their minimum widths, then children are moved one by one onto a new line, until the elements remaining on the previous line fit. This process can repeat to an arbitrary number of lines. If a line contains only a single element that doesn't fit, then the element should stay on that line and overflow out of the box. The later lines are placed below the earlier lines in normal direction boxes and above in reverse direction boxes. The height of a line is the height of the largest child in that line. No additional space appears between the lines apart from the margins on the largest elements in each line. For calculating the height of a line, margins with a computed value of auto should be treated as having a value of 0.

A similar process occurs for children in a vertical box. Later lines in normal direction boxes are placed to the right of earlier lines and to the left in reverse direction boxes.

Once the number of lines has been determined, elements with a computed value for `box-flex` other than `0` stretch as necessary in an attempt to fill the remaining space on the lines. Each line computes flexes independently, so only elements on that line are considered when evaluating `box-flex` and `box-flex-group`. The packing of elements in a line, as specified by the `box-pack` property, is also computed independently for each line.

Syntax

```
/* Keyword values */
box-lines: single;
box-lines: multiple;

/* Global values */
box-lines: inherit;
```

```
box-lines: initial;
box-lines: unset;
```

Values

This property is specified as one of the following keyword values:

`single`

Box elements lay out in a single row or column.

`multiple`

Box elements layout in multiple rows or columns.

Formal definition

Initial value	`single`
Applies to	box elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-lines =
  single    |
  multiple
```

Examples

Basic usage example

In the original version of the spec, `box-lines` allowed you to specify that you wanted your flex container's children to wrap onto multiple lines. This was only supported in WebKit-based browsers, with a prefix.

```
div {
  display: box;
  box-orient: horizontal;
  box-lines: multiple;
}
```

The modern flexbox equivalent is `flex-wrap`.

Specifications

Not part of any standard.

See also

- `flex`
- `flex-basis`
- `flex-grow`
- `flex-shrink`

box-ordinal-group CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft. It has been replaced in the specification. See flexbox for information about the current standard.

The **`box-ordinal-group`** CSS property assigns the flexbox's child elements to an ordinal group.

Ordinal groups may be used in conjunction with the `box-direction` property to control the order in which the direct children of a box appear. When the computed `box-direction` is normal, a box will display its elements starting from the lowest numbered ordinal group and ensure that those elements appear to the left (for horizontal boxes) or at the top (for vertical boxes) of the container. Elements with the same ordinal group are flowed in the order they appear in the source document tree. In the reverse direction, the ordinal groups are examined in the same order, except the elements appear reversed.

Syntax

```
/* <integer> values */
box-ordinal-group: 1;
box-ordinal-group: 5;

/* Global values */
box-ordinal-group: inherit;
box-ordinal-group: initial;
box-ordinal-group: unset;
```

The `box-ordinal-group` property is specified as any positive `<integer>`.

Formal definition

Initial value	`1`
Applies to	children of box elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-ordinal-group =
  <integer>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage example

In an older version of the spec, `box-ordinal-group` was included to allow you to change the display order of flex children inside a flex container:

```
article:nth-child(1) {
  -webkit-box-ordinal-group: 2;
  -moz-box-ordinal-group: 2;
  box-ordinal-group: 2;
}

article:nth-child(2) {
  -webkit-box-ordinal-group: 1;
  -moz-box-ordinal-group: 1;
  box-ordinal-group: 1;
}
```

The modern flexbox equivalent is `order`.

Specifications

Not part of any standard.

See also

- `flex`
- `flex-basis`
- `flex-grow`
- `flex-shrink`

box-orient CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft, and has been replaced by a newer standard. See flexbox for information about the current standard.

The **`box-orient`** CSS property sets whether an element lays out its contents horizontally or vertically.

Syntax

```
/* Keyword values */
box-orient: horizontal;
box-orient: vertical;
box-orient: inline-axis;
box-orient: block-axis;

/* Global values */
box-orient: inherit;
box-orient: initial;
box-orient: unset;
```

Values

This property is specified as one of the following keyword values:

`horizontal`

: The box lays out its contents horizontally.

`vertical`

: The box lays out its contents vertically.

`inline-axis` (HTML)

: The box displays its children along the inline axis.

`block-axis` (HTML)

: The box displays its children along the block axis.

The inline and block axes are the writing-mode dependent keywords which, in English, map to `horizontal` and `vertical` respectively.

Description

HTML DOM elements lay out their contents along the inline-axis by default. This CSS property will only apply to HTML elements with a CSS `display` value of `box` or `inline-box`.

Formal definition

Initial value	`inline-axis`
Applies to	elements with a CSS `display` value of `box` or `inline-box`
Inherited	no
Computed value	as specified

Animation type	discrete

Formal syntax

```
box-orient =
  horizontal   |
  vertical     |
  inline-axis  |
  block-axis
```

Examples

Setting horizontal box orientation

Here, the `box-orient` property will cause the two `<p>` sections in the example to display in the same line.

HTML

```
<div class="example">
  <p>I will be to the left of my sibling.</p>
  <p>I will be to the right of my sibling.</p>
</div>
```

CSS

```
div.example {
  display: -moz-box; /* Mozilla */
  display: -webkit-box; /* WebKit */
  display: box; /* As specified */

  /* Children should be oriented vertically */
  -moz-box-orient: horizontal; /* Mozilla */
  -webkit-box-orient: horizontal; /* WebKit */
  box-orient: horizontal; /* As specified */
}
```

Result

Specifications

Not part of any standard.

See also

- `box-direction`
- `box-pack`
- `box-align`
- `flex-direction`

box-pack CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Warning:** This is a property of the original CSS flexible box layout Module draft, and has been replaced by a newer standard. See flexbox for information about the current standard.

The **`-moz-box-pack`** and **`-webkit-box-pack`** CSS properties specify how a `-moz-box` or `-webkit-box` packs its contents in the direction of its layout. The effect of this is only visible if there is extra space in the box.

The direction of layout depends on the element's orientation: horizontal or vertical.

Syntax

```
/* Keyword values */
box-pack: start;
box-pack: center;
box-pack: end;
box-pack: justify;

/* Global values */
box-pack: inherit;
box-pack: initial;
box-pack: unset;
```

Values

This property is specified as one of the following keyword values:

`start`

The box packs contents at the start, leaving any extra space at the end.

`center`

The box packs contents in the center, dividing any extra space equally between the start and the end.

`end`

The box packs contents at the end, leaving any extra space at the start.

`justify`

The space is divided evenly in-between each child, with none of the extra space placed before the first child or after the last child. If there is only one child, treat the value as if it were `start`.

Notes

The edge of the box designated the *start* for packing purposes depends on the box's orientation and direction:

- For horizontal elements, the *start* is the top edge.
- For vertical elements, the *start* is the left edge.

	Normal	**Reverse**
Horizontal	left	right
Vertical	top	bottom

The edge opposite to the start is designated the *end*.

If the packing is set using the element's `pack` attribute, then the style is ignored.

Formal definition

Initial value	`start`
Applies to	elements with a CSS `display` value of `-moz-box`, `-moz-inline-box`, `-webkit-box` or `-webkit-inline-box`
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-pack =
  start    |
  center   |
  end      |
  justify
```

Examples

Examples of box-pack

```
div.example {
  border-style: solid;

  display: -moz-box; /* Mozilla */
  display: -webkit-box; /* WebKit */

  /* Make this box taller than the children,
     so there is room for the box-pack */
  height: 300px;
  /* Make this box wide enough to show the contents
     are centered horizontally */
  width: 300px;

  /* Children should be oriented vertically */
  -moz-box-orient: vertical; /* Mozilla */
  -webkit-box-orient: vertical; /* WebKit */

  /* Align children to the horizontal center of this box */
  -moz-box-align: center; /* Mozilla */
  -webkit-box-align: center; /* WebKit */

  /* Pack children to the bottom of this box */
  -moz-box-pack: end; /* Mozilla */
  -webkit-box-pack: end; /* WebKit */
}

div.example p {
  /* Make children narrower than their parent,
     so there is room for the box-align */
  width: 200px;
}
```

```
<div class="example">
  <p>I will be second from the bottom of div.example, centered horizontally.</p>
  <p>I will be on the bottom of div.example, centered horizontally.</p>
</div>
```

Specifications

Not part of any standard.

See also

- `box-orient`
- `box-direction`
- `box-align`

box-shadow CSS property

The **`box-shadow`** CSS property adds shadow effects around an element's frame. You can set multiple effects separated by commas. A box shadow is described by X and Y offsets relative to the element, blur and spread radius, and color.

Try it

```
box-shadow: 10px 5px 5px red;
```

```
box-shadow: 60px -16px teal;
```

```
box-shadow: 12px 12px 2px 1px rgb(0 0 255 / 0.2);
```

```
box-shadow: inset 5em 1em gold;
```

```
box-shadow:
  3px 3px red,
  -1em 0 0.4em olive;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <p>This is a box with a box-shadow around it.</p>
  </div>
</section>
```

```
#example-element {
  margin: 20px auto;
  padding: 0;
  border: 2px solid #333333;
  width: 80%;
  text-align: center;
}
```

Syntax

```
/* Keyword value */
box-shadow: none;

/* A color and two length values */
box-shadow: red 60px -16px;

/* Three length values and a color */
box-shadow: 10px 5px 5px black;

/* Four length values and a color */
box-shadow: 2px 2px 2px 1px rgb(0 0 0 / 20%);

/* inset, length values, and a color */
box-shadow: inset 5em 1em gold;

/* Multiple shadows, separated by commas */
box-shadow:
  3px 3px red inset,
  -1em 0 0.4em olive;

/* Global values */
box-shadow: inherit;
box-shadow: initial;
box-shadow: revert;
box-shadow: revert-layer;
box-shadow: unset;
```

Specify a single box-shadow using:

- Two, three, or four `<length>` values.
    - If only two values are given, they are interpreted as `<offset-x>` and `<offset-y>` values.
    - If a third value is given, it is interpreted as a `<blur-radius>`.
    - If a fourth value is given, it is interpreted as a `<spread-radius>`.
- Optionally, the `inset` keyword.
- Optionally, a `<color>` value.

To specify multiple shadows, provide a comma-separated list of shadows.

Values

`<color>` Optional

Specifies color for the shadow. See `<color>` values for possible keywords and notations. If not specified, the value of the `color` property defined in the parent element is used.

`<length>`

Specifies the offset length of the shadow. This parameter accepts two, three, or four values. Third and fourth values are optional. They are interpreted as follows:

- If two values are specified, they are interpreted as `<offset-x>` (horizontal offset) and `<offset-y>` (vertical offset) values. Negative `<offset-x>` value places the shadow to the left of the element. Negative `<offset-y>` value places the shadow above the element.
  If not specified, the value of `0` is used for the missing length. If both `<offset-x>` and `<offset-y>` are set to `0`, the shadow is placed behind the element (and may generate a blur effect if `<blur-radius>` and/or `<spread-radius>` is set).
- If three values are specified, the third value is interpreted as `<blur-radius>`. The larger this value, the bigger the blur, so the shadow becomes bigger and lighter. Negative values are not allowed. If not specified, it will be set to `0` (meaning that the shadow's edge will be sharp). The specification does not include an exact algorithm for how the blur radius should be calculated; however, it does elaborate as follows:

  > ...for a long, straight shadow edge, this should create a color transition the length of the blur distance that is perpendicular to and centered on the shadow's edge, and that ranges from the full shadow color at the radius endpoint inside the shadow to fully transparent at the endpoint outside it.
- If four values are specified, the fourth value is interpreted as `<spread-radius>`. Positive values will cause the shadow to expand and grow bigger, negative values will cause the shadow to shrink. If not specified, it will be set to `0` (that is, the shadow will be the same size as the element).

`inset` Optional

Changes the shadow from an outer box-shadow to an inner box-shadow (as if the content is pressed into the box). Inset shadows are clipped to the element's padding box and appear above the background but below the content. By default, the shadow behaves like a drop shadow, giving the appearance that the box is elevated above its content. This is the default behavior when `inset` is not specified.

Interpolation

When animating shadows, such as when multiple shadow values on a box transition to new values on hover, the values are interpolated. Interpolation determines intermediate values of properties, such as the blur radius, spread radius, and color, as shadows transition. For each shadow in a list of shadows, the color, x, y, blur, and spread transition; the color as `<color>`, and the other values as `<length>`s.

In interpolating multiple shadows between two comma-separated lists of multiple box shadows, the shadows are paired, in order, with interpolation occurring between paired shadows. If the lists of shadows have different lengths, then the shorter list is padded at the end with shadows whose color is `transparent`, and X, Y, and blur are `0`, with the inset, or lack of inset, being set to match. If in any pair of shadows, one

has `inset` set and the other does not, the entire shadow list is uninterpolated; the shadows will change to the new values without an animating effect.

Description

The `box-shadow` property enables you to cast a drop shadow from the frame of almost any element. If a `border-radius` is specified on the element with a box shadow, the box shadow takes on the same rounded corners. The z-ordering of multiple box shadows is the same as multiple text shadows (the first specified shadow is on top).

Box-shadow generator is an interactive tool allowing you to generate a `box-shadow`.

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	any length made absolute; any specified color computed; otherwise as specified
Animation type	a shadow list

Formal syntax

```
box-shadow =
  <spread-shadow>#

<spread-shadow> =
  <'box-shadow-color'>?                              &&
  [ [ none | <length>{2} ] [ <'box-shadow-blur'> <'box-shadow-spread'>? ]? ]  &&
  <'box-shadow-position'>?

<box-shadow-color> =
  <color>#

<box-shadow-blur> =
  <length [0,∞]>#

<box-shadow-spread> =
  <length>#

<box-shadow-position> =
  [ outset | inset ]#
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting three shadows

In this example, we include three shadows: an inset shadow, a regular drop shadow, and a 2px shadow that creates a border effect (we could have used an `outline` instead for that third shadow).

HTML

```
<blockquote>
  <q>
    You may shoot me with your words,<br />
    You may cut me with your eyes,<br />
    You may kill me with your hatefulness,<br />
    But still, like air, I'll rise.
  </q>
  <p>&mdash; Maya Angelou</p>
</blockquote>
```

CSS

```
blockquote {
  padding: 20px;
  box-shadow:
    inset 0 -3em 3em rgb(0 200 0 / 30%),
    0 0 0 2px white,
    0.3em 0.3em 1em rgb(200 0 0 / 60%);
}
```

Result

Setting zero for offset and blur

When the `x-offset`, `y-offset`, and `blur` are all zero, the box shadow will be a solid-colored outline of equal-size on all sides. The shadows are drawn back to front, so the first shadow sits on top of subsequent shadows. When the `border-radius` is set to 0, as is the default, the corners of the shadow will be, well, corners. Had we put in a `border-radius` of any other value, the corners would have been rounded.

We added a margin the size of the widest box-shadow to ensure the shadow doesn't overlap adjacent elements or go beyond the border of the containing box. A box-shadow does not impact box model dimensions.

HTML

```
<div><p>Hello World</p></div>
```

CSS

```
p {
  box-shadow:
    0 0 0 2em #f4aab9,
    0 0 0 4em #66ccff;
  margin: 4em;
  padding: 1em;
}
```

Result

See also

- `<color>`
- `text-shadow`

- `drop-shadow()`
- Introduction to text shadows
- Applying color to HTML elements using CSS
- CSS backgrounds and borders module

box-sizing CSS property

The **`box-sizing`** CSS property sets how the total width and height of an element is calculated.

Try it

```
box-sizing: content-box;
width: 100%;
```

```
box-sizing: content-box;
width: 100%;
border: solid #5b6dcd 10px;
padding: 5px;
```

```
box-sizing: border-box;
width: 100%;
border: solid #5b6dcd 10px;
padding: 5px;
```

```
<section id="default-example">
  <div id="example-element-parent">
    <p>Parent container</p>
    <div class="transition-all" id="example-element">
      <p>Child container</p>
    </div>
  </div>
</section>
```

```
#example-element-parent {
  width: 220px;
  height: 200px;
  border: solid 10px #ffc129;
  margin: 0.8em;
}

#example-element {
  height: 60px;
  margin: 2em auto;
  background-color: rgb(81 81 81 / 0.6);
}

#example-element > p {
  margin: 0;
}
```

Syntax

```
box-sizing: border-box;
box-sizing: content-box;

/* Global values */
box-sizing: inherit;
box-sizing: initial;
box-sizing: revert;
box-sizing: revert-layer;
box-sizing: unset;
```

Values

This property is specified as one of the following keyword values:

`content-box`

> This is the initial and default value as specified by the CSS standard. The `width` and `height` properties include the content, but does not include the padding, border, or margin. For example, `.box {width: 350px; border: 10px solid black;}` renders a box that is 370px wide.
>
> Here, the dimensions of the element are calculated as: *width = width of the content*, and *height = height of the content*. (Borders and padding are not included in the calculation.)

`border-box`

> The `width` and `height` properties include the content, padding, and border, but do not include the margin. Note that padding and border will be inside of the box. For example, `.box {width: 350px; border: 10px solid black;}` renders a box that is 350px wide, with the area for content being 330px wide. The content box can't be negative and is floored to 0, making it impossible to use `border-box` to make the element disappear.
>
> Here the dimensions of the element are calculated as: *width = border + padding + width of the content*, and *height = border + padding + height of the content*.

Description

By default in the CSS box model, the `width` and `height` you assign to an element is applied only to the element's content box. If the element has any border or padding, this is then added to the `width` and `height` to arrive at the size of the box that's rendered on the screen. This means that when you set `width` and `height`, you have to adjust the value you give to allow for any border or padding that may be added. For example, if you have four boxes with `width: 25%;`, if any has left or right padding or a left or right border, they will not by default fit on one line within the constraints of the parent container.

The `box-sizing` property can be used to adjust this behavior:

- `content-box` gives you the default CSS box-sizing behavior. If you set an element's width to 100 pixels, then the element's content box will be 100 pixels wide, and the width of any border or padding will be added to the final rendered width, making the element wider than 100px.
- `border-box` tells the browser to account for any border and padding in the values you specify for an element's width and height. If you set an element's width to 100 pixels, that 100 pixels will include any border or padding you added, and the content box will shrink to absorb that extra width. This typically makes it much easier to size elements.

  `box-sizing: border-box` is the default styling that browsers use for the `<table>`, `<select>`, and `<button>` elements, and for `<input>` elements whose type is `radio`, `checkbox`, `reset`, `button`, `submit`, `color`, or `search`.

> **Note:** It is often useful to set `box-sizing` to `border-box` to lay out elements. This makes dealing with the sizes of elements much easier, and generally eliminates a number of pitfalls you can stumble on while laying out your content. On the other hand, when using `position: relative` or `position: absolute`, use of `box-sizing: content-box` allows the positioning values to be relative to the content, and independent of changes to border and padding sizes, which is sometimes desirable.

Formal definition

Initial value	`content-box`
Applies to	all elements that accept width or height
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
box-sizing =
  content-box  |
  border-box
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Box sizes with content-box and border-box

This example shows how different `box-sizing` values alter the rendered size of two otherwise identical elements.

HTML

```
<div class="content-box">Content box</div>
<br />
<div class="border-box">Border box</div>
```

CSS

```
div {
  width: 160px;
  height: 80px;
  padding: 20px;
  border: 8px solid red;
  background: yellow;
}

.content-box {
  box-sizing: content-box;
  /* Total width: 160px + (2 * 20px) + (2 * 8px) = 216px
     Total height: 80px + (2 * 20px) + (2 * 8px) = 136px
     Content box width: 160px
     Content box height: 80px */
}

.border-box {
  box-sizing: border-box;
  /* Total width: 160px
     Total height: 80px
     Content box width: 160px - (2 * 20px) - (2 * 8px) = 104px
     Content box height: 80px - (2 * 20px) - (2 * 8px) = 24px */
}
```

Result

See also

- CSS box model

break-after CSS property

The **`break-after`** CSS property sets how page, column, or region breaks should behave after a generated box. If there is no generated box, the property is ignored.

Try it

```
break-after: auto;
```

```
break-after: page;
```

```
<div>
  <p>
    The effect of this property can be noticed when the document is being
    printed or a preview of a print is displayed.
  </p>
  <button id="print-btn">Show Print Preview</button>
  <div class="box-container">
    <div class="box">Content before the property</div>
    <div class="box" id="example-element">Content with 'break-after'</div>
    <div class="box">Content after the property</div>
  </div>
</div>
```

```
.box {
  border: solid #5b6dcd 5px;
  background-color: #5b6dcd;
  margin: 10px 0;
  padding: 5px;
}

#example-element {
```

```
  border: solid 5px #ffc129;
  background-color: #ffc129;
  color: black;
}
```

```
const btn = document.getElementById("print-btn");

btn.addEventListener("click", () => {
  window.print();
});
```

Syntax

```
/* Generic break values */
break-after: auto;
break-after: avoid;
break-after: always;
break-after: all;

/* Page break values */
break-after: avoid-page;
break-after: page;
break-after: left;
break-after: right;
break-after: recto;
break-after: verso;

/* Column break values */
break-after: avoid-column;
break-after: column;

/* Region break values */
break-after: avoid-region;
break-after: region;

/* Global values */
break-after: inherit;
break-after: initial;
break-after: revert;
break-after: revert-layer;
break-after: unset;
```

Each possible break point (in other words, each element boundary) is affected by three properties: the `break-after` value of the previous element, the `break-before` value of the next element, and the `break-inside` value of the containing element.

To determine if a break must be done, the following rules are applied:

1. If any of the three concerned values is a *forced break value* (`always`, `left`, `right`, `page`, `column`, or `region`), it has precedence. If more than one of them are such a break, the one of the element that appears the latest in the flow is taken (i.e., the `break-before` value has precedence over the `break-after` value, which itself has precedence over the `break-inside` value).
2. If any of the three concerned values is an *avoid break value* (`avoid`, `avoid-page`, `avoid-region`, or `avoid-column`), no such break will be applied at that point.

Once forced breaks have been applied, soft breaks may be added if needed, but not on element boundaries that resolve in a corresponding `avoid` value.

Values

Generic break values

`auto`

Allows, but does not force, any break (page, column, or region) to be inserted right after the principal box.

`avoid`

Avoids any break (page, column, or region) from being inserted right after the principal box.

`always`

Forces a page break right after the principal box. The type of this break is that of the immediately-containing fragmentation context. If we are inside a multicol container then it would force a column break, inside paged media (but not inside a multicol container) a page break.

`all`

Forces a page break right after the principal box. Breaking through all possible fragmentation contexts. So a break inside a multicol container, which was inside a page container would force a column and page break.

Page break values

`avoid-page`

Avoids any page break right after the principal box.

`page`

Forces a page break right after the principal box.

`left`

Forces one or two page breaks right after the principal box, whichever will make the next page into a left page. It's the page placed on the left side of the spine of the book or the back side of the page in duplex printing.

`right`

Forces one or two page breaks right after the principal box, whichever will make the next page into a right page. It's the page placed on the right side of the spine of the book or the front side of the page in duplex printing.

`recto`

Forces one or two page breaks right after the principal box, whichever will make the next page into a recto page. (A recto page is a right page in a left-to-right spread or a left page in a right-to-left spread.)

`verso`

Forces one or two page breaks right after the principal box, whichever will make the next page into a verso page. (A verso page is a left page in a left-to-right spread or a right page in a right-to-left spread.)

Column break values

`avoid-column`

Avoids any column break right after the principal box.

`column`

Forces a column break right after the principal box.

Region break values

`avoid-region`

Avoids any region break right after the principal box.

`region`

Forces a region break right after the principal box.

Page break aliases

For compatibility reasons, the legacy `page-break-after` property should be treated by browsers as an alias of `break-after`. This ensures that sites using `page-break-after` continue to work as designed. A subset of values should be aliased as follows:

page-break-after	break-after
`auto`	`auto`
`left`	`left`
`right`	`right`
`avoid`	`avoid`
`always`	`page`

> **Note:** The `always` value of `page-break-*` was implemented by browsers as a page break, and not as a column break. Therefore the aliasing is to `page`, rather than the `always` value in the Level 4 spec.

Formal definition

Initial value	`auto`
Applies to	block-level elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
break-after =
  auto          |
  avoid         |
  always        |
  all           |
  avoid-page    |
  page          |
  left          |
  right         |
  recto         |
  verso         |
  avoid-column  |
  column        |
  avoid-region  |
  region
```

This syntax reflects the latest standard as per CSS Fragmentation Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Breaking into neat columns

In the following example we have a container that contains an `<h1>` spanning all columns (achieved using `column-span: all`) and a series of `<h2>`s and paragraphs laid out in multiple columns using `column-width: 200px`.

By default, the subheadings and paragraphs were laid out rather messily because the headings were not in a uniform place. However, we used `break-after: column` on the `<p>` elements to force a column break after each one, meaning that you end up with an `<h2>` neatly at the top of each column.

HTML

```
<article>
  <h1>Main heading</h1>

  <h2>Subheading</h2>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla vitae
    fringilla mauris. Quisque commodo eget nisi sed pretium. Mauris luctus nec
    lacus in ultricies. Mauris vitae hendrerit arcu, ac scelerisque lacus.
    Aliquam lobortis in lacus sit amet posuere. Fusce iaculis urna id neque
    dapibus, eu lacinia lectus dictum.
  </p>

  <h2>Subheading</h2>

  <p>
    Praesent condimentum dui dui, sit amet rutrum diam tincidunt eu. Cras
    suscipit porta leo sit amet rutrum. Sed vehicula ornare tincidunt. Curabitur
    a ipsum ac diam mattis volutpat ac ut elit. Nullam luctus justo non
    vestibulum gravida. Morbi metus libero, pharetra non porttitor a, molestie
```

```
    nec nisi.
  </p>

  <h2>Subheading</h2>

  <p>
    Vivamus eleifend metus vitae neque placerat, eget interdum elit mattis.
    Donec eu vulputate nibh. Ut turpis leo, malesuada quis nisl nec, volutpat
    egestas tellus.
  </p>

  <h2>Subheading</h2>

  <p>
    In finibus viverra enim vel suscipit. Quisque consequat velit eu orci
    malesuada, ut interdum tortor molestie. Proin sed pellentesque augue. Nam
    risus justo, faucibus non porta a, congue vel massa. Cras luctus lacus nisl,
    sed tincidunt velit pharetra ac. Duis suscipit faucibus dui sed ultricies.
  </p>
</article>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
  letter-spacing: 2px;
  column-span: all;
}

h2 {
  font-size: 1.2rem;
  color: red;
  letter-spacing: 1px;
}

p {
  line-height: 1.5;
  break-after: column;
}

article {
  column-width: 200px;
  gap: 20px;
}
```

Result

See also

- Learn: Multiple-column Layout
- Breaking Boxes With CSS Fragmentation

break-before CSS property

The **`break-before`** CSS property sets how page, column, or region breaks should behave before a generated box. If there is no generated box, the property is ignored.

Try it

```
break-before: auto;
```

```
break-before: page;
```

```
<div>
  <p>
    The effect of this property can be noticed when the document is being
    printed or a preview of a print is displayed.
  </p>
  <button id="print-btn">Show Print Preview</button>
  <div class="box-container">
    <div class="box">Content before the property</div>
    <div class="box" id="example-element">Content with 'break-before'</div>
    <div class="box">Content after the property</div>
  </div>
</div>
```

```
.box {
  border: solid #5b6dcd 5px;
  background-color: #5b6dcd;
  margin: 10px 0;
  padding: 5px;
}

#example-element {
  border: solid 5px #ffc129;
  background-color: #ffc129;
  color: black;
}
```

```
const btn = document.getElementById("print-btn");

btn.addEventListener("click", () => {
  window.print();
});
```

Syntax

```
/* Generic break values */
break-before: auto;
break-before: avoid;
break-before: always;
break-before: all;

/* Page break values */
break-before: avoid-page;
break-before: page;
break-before: left;
break-before: right;
break-before: recto;
break-before: verso;

/* Column break values */
break-before: avoid-column;
break-before: column;

/* Region break values */
break-before: avoid-region;
break-before: region;

/* Global values */
break-before: inherit;
break-before: initial;
break-before: revert;
break-before: revert-layer;
break-before: unset;
```

Each possible break point (in other words, each element boundary) is affected by three properties: the `break-after` value of the previous element, the `break-before` value of the next element, and the `break-inside` value of the containing element.

To determine if a break must be done, the following rules are applied:

1. If any of the three concerned values is a *forced break value* (`always`, `left`, `right`, `page`, `column`, or `region`), it has precedence. If more than one of them are such a break, the one of the element that appears the latest in the flow is taken (i.e., the `break-before` value has precedence over the `break-after` value, which itself has precedence over the `break-inside` value).
2. If any of the three concerned values is an *avoid break value* (`avoid`, `avoid-page`, `avoid-region`, or `avoid-column`), no such break will be applied at that point.

Once forced breaks have been applied, soft breaks may be added if needed, but not on element boundaries that resolve in a corresponding `avoid` value.

Values

Generic break values

`auto`

Allows, but does not force, any break (page, column, or region) to be inserted right before the principal box.

`avoid`

Avoids any break (page, column, or region) from being inserted right before the principal box.

`always`

Forces a page break right before the principal box. The type of this break is that of the immediately-containing fragmentation context. If we are inside a multicol container then it would force a column break, inside paged media (but not inside a multicol container) a page break.

`all`

Forces a page break right before the principal box. Breaking through all possible fragmentation contexts. So a break inside a multicol container, which was inside a page container would force a column and page break.

Page break values

`avoid-page`

Avoids any page break right before the principal box.

`page`

Forces a page break right before the principal box.

`left`

Forces one or two page breaks right before the principal box, whichever will make the next page into a left page. It's the page placed on the left side of the spine of the book or the back side of the page in duplex printing.

`right`

Forces one or two page breaks right before the principal box, whichever will make the next page into a right page. It's the page placed on the right side of the spine of the book or the front side of the page in duplex printing.

`recto`

Forces one or two page breaks right before the principal box, whichever will make the next page into a recto page. (A recto page is a right page in a left-to-right spread or a left page in a right-to-left spread.)

`verso`

Forces one or two page breaks right before the principal box, whichever will make the next page into a verso page. (A verso page is a left page in a left-to-right spread or a right page in a right-to-left spread.)

Column break values

`avoid-column`

Avoids any column break right before the principal box.

`column`

Forces a column break right before the principal box.

Region break values

`avoid-region`

Avoids any region break right before the principal box.

`region`

Forces a region break right before the principal box.

Page break aliases

For compatibility reasons, the legacy `page-break-before` property should be treated by browsers as an alias of `break-before`. This ensures that sites using `page-break-before` continue to work as designed. A subset of values should be aliased as follows:

page-break-before	break-before
`auto`	`auto`
`left`	`left`
`right`	`right`
`avoid`	`avoid`
`always`	`page`

> **Note:** The `always` value of `page-break-*` was implemented by browsers as a page break, and not as a column break. Therefore the aliasing is to `page`, rather than the `always` value in the Level 4 spec.

Formal definition

Initial value	`auto`
Applies to	block-level elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
break-before =
  auto          |
  avoid         |
  always        |
  all           |
  avoid-page    |
  page          |
  left          |
  right         |
  recto         |
  verso         |
  avoid-column  |
  column        |
  avoid-region  |
  region
```

This syntax reflects the latest standard as per CSS Fragmentation Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Breaking into neat columns

In the following example we have a container that contains an `<h1>` spanning all columns (achieved using `column-span: all`) and a series of `<h2>`s and paragraphs laid out in multiple columns using `column-width: 200px`.

By default, the subheadings and paragraphs were laid out rather messily because the headings were not in a uniform place. However, we used `break-before: column` on the `<h2>` elements to force a column break before each one, meaning that you end up with an `<h2>` neatly at the top of each column.

HTML

```
<article>
  <h1>Main heading</h1>

  <h2>Subheading</h2>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla vitae
    fringilla mauris. Quisque commodo eget nisi sed pretium. Mauris luctus nec
    lacus in ultricies. Mauris vitae hendrerit arcu, ac scelerisque lacus.
    Aliquam lobortis in lacus sit amet posuere. Fusce iaculis urna id neque
    dapibus, eu lacinia lectus dictum.
  </p>

  <h2>Subheading</h2>

  <p>
    Praesent condimentum dui dui, sit amet rutrum diam tincidunt eu. Cras
    suscipit porta leo sit amet rutrum. Sed vehicula ornare tincidunt. Curabitur
    a ipsum ac diam mattis volutpat ac ut elit. Nullam luctus justo non
    vestibulum gravida. Morbi metus libero, pharetra non porttitor a, molestie
    nec nisi.
  </p>

  <h2>Subheading</h2>

  <p>
    Vivamus eleifend metus vitae neque placerat, eget interdum elit mattis.
    Donec eu vulputate nibh. Ut turpis leo, malesuada quis nisl nec, volutpat
```

```
    egestas tellus.
  </p>

  <h2>Subheading</h2>

  <p>
    In finibus viverra enim vel suscipit. Quisque consequat velit eu orci
    malesuada, ut interdum tortor molestie. Proin sed pellentesque augue. Nam
    risus justo, faucibus non porta a, congue vel massa. Cras luctus lacus nisl,
    sed tincidunt velit pharetra ac. Duis suscipit faucibus dui sed ultricies.
  </p>
</article>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
  letter-spacing: 2px;
  column-span: all;
}

h2 {
  font-size: 1.2rem;
  color: red;
  letter-spacing: 1px;
  break-before: column;
}

p {
  line-height: 1.5;
}

article {
  column-width: 200px;
  gap: 20px;
}
```

Result

See also

- Learn: Multiple-column Layout
- Breaking Boxes With CSS Fragmentation

break-inside CSS property

The **`break-inside`** CSS property sets how page, column, or region breaks should behave inside a generated box. If there is no generated box, the property is ignored.

Try it

```
break-inside: auto;
```

```
break-inside: avoid;
```

```
<div>
  <p>
    The effect of this property can be noticed when the document is being
    printed or a preview of a print is displayed.
  </p>
  <button id="print-btn">Show Print Preview</button>
  <div class="box-container">
    <div class="box">Content before the property</div>
    <div class="box" id="example-element">Content with 'break-inside'</div>
    <div class="box">Content after the property</div>
  </div>
</div>
```

```
.box {
  border: solid #5b6dcd 5px;
  background-color: #5b6dcd;
  margin: 10px 0;
  padding: 5px;
}

#example-element {
  border: solid 5px #ffc129;
  background-color: #ffc129;
  color: black;
}

@media print {
  #example-element {
    height: 25cm;
  }
}
```

```
const btn = document.getElementById("print-btn");

btn.addEventListener("click", () => {
  window.print();
});
```

Syntax

```
/* Keyword values */
break-inside: auto;
break-inside: avoid;
break-inside: avoid-page;
break-inside: avoid-column;
break-inside: avoid-region;

/* Global values */
break-inside: inherit;
break-inside: initial;
break-inside: revert;
break-inside: revert-layer;
break-inside: unset;
```

Each possible break point (in other words, each element boundary) is affected by three properties: the `break-after` value of the previous element, the `break-before` value of the next element, and the `break-inside` value of the containing element.

To determine if a break must be done, the following rules are applied:

1. If any of the three concerned values is a *forced break value* (`always`, `left`, `right`, `page`, `column`, or `region`), it has precedence. If more than one of them are such a break, the value of the element that appears the latest in the flow is used. Thus, the `break-before` value has precedence over the `break-after` value, which in turn has precedence over the `break-inside` value.
2. If any of the three concerned values is an *avoid break value* (`avoid`, `avoid-page`, `avoid-region`, or `avoid-column`), no such break will be applied at that point.

Once forced breaks have been applied, soft breaks may be added if needed, but not on element boundaries that resolve in a corresponding `avoid` value.

Values

`auto`

Allows, but does not force, any break (page, column, or region) to be inserted within the principal box.

`avoid`

Avoids any break (page, column, or region) from being inserted within the principal box.

`avoid-page`

Avoids any page break within the principal box.

`avoid-column`

Avoids any column break within the principal box.

`avoid-region`

Avoids any region break within the principal box.

Page break aliases

For compatibility reasons, the legacy `page-break-inside` property should be treated by browsers as an alias of `break-inside`. This ensures that sites using `page-break-inside` continue to work as designed. A subset of values should be aliased as follows:

page-break-inside	break-inside
`auto`	`auto`
`avoid`	`avoid`

Formal definition

Initial value	`auto`
Applies to	block-level elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
break-inside =
  auto          |
  avoid         |
  avoid-page    |
  avoid-column  |
  avoid-region
```

This syntax reflects the latest standard as per CSS Fragmentation Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Avoiding breaking inside a figure

In the following example we have a container that contains an `<h1>` spanning all columns (achieved using `column-span: all`) and a series of paragraphs laid out in multiple columns using `column-width: 200px`. We also have a `<figure>` containing an image and a caption.

By default, it is possible for you to get a break between the image and its caption, which is not what we want. To avoid this, we have set `break-inside: avoid` on the `<figure>`, which causes them to always stay together.

HTML

```
<article>
  <h1>Main heading</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla vitae
    fringilla mauris. Quisque commodo eget nisi sed pretium. Mauris luctus nec
    lacus in ultricies. Mauris vitae hendrerit arcu, ac scelerisque lacus.
    Aliquam lobortis in lacus sit amet posuere. Fusce iaculis urna id neque
```

```
    dapibus, eu lacinia lectus dictum.
  </p>

  <figure>
    <img
      src="https://mdn.dev/archives/media/attachments/2020/07/29/17350/3b4892b7e820122ac6dd7678891d4507/firefox.png" />
    <figcaption>The Firefox logo — fox wrapped around the world</figcaption>
  </figure>

  <p>
    Praesent condimentum dui dui, sit amet rutrum diam tincidunt eu. Cras
    suscipit porta leo sit amet rutrum. Sed vehicula ornare tincidunt. Curabitur
    a ipsum ac diam mattis volutpat ac ut elit. Nullam luctus justo non
    vestibulum gravida. Morbi metus libero, pharetra non porttitor a, molestie
    nec nisi.
  </p>

  <p>
    In finibus viverra enim vel suscipit. Quisque consequat velit eu orci
    malesuada, ut interdum tortor molestie. Proin sed pellentesque augue. Nam
    risus justo, faucibus non porta a, congue vel massa. Cras luctus lacus nisl,
    sed tincidunt velit pharetra ac. Duis suscipit faucibus dui sed ultricies.
  </p>
</article>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

body {
  width: 80%;
  margin: 0 auto;
}

h1 {
  font-size: 3rem;
  letter-spacing: 2px;
  column-span: all;
}

h1 + p {
  margin-top: 0;
}

p {
  line-height: 1.5;
  break-after: column;
}

figure {
  break-inside: avoid;
}

img {
  max-width: 70%;
  display: block;
  margin: 0 auto;
}

figcaption {
  font-style: italic;
  font-size: 0.8rem;
  width: 70%;
}

article {
  column-width: 200px;
  gap: 20px;
}
```

Result

See also

- Learn: Multiple-column Layout
- Breaking Boxes With CSS Fragmentation

caption-side CSS property

The **`caption-side`** CSS property puts the content of a table's `<caption>` on the specified side. The values are relative to the `writing-mode` of the table.

Try it

```
caption-side: top;
```

```
caption-side: bottom;
```

```
<section class="default-example" id="default-example">
  <table class="transition-all" id="example-element">
    <caption>
      Famous animals
    </caption>
    <tbody>
      <tr>
        <th>Name</th>
        <th>Location</th>
      </tr>
      <tr>
        <td>Giraffe</td>
```

```
        <td>Africa</td>
      </tr>
      <tr>
        <td>Penguin</td>
        <td>Antarctica</td>
      </tr>
      <tr>
        <td>Sloth</td>
        <td>South America</td>
      </tr>
      <tr>
        <td>Tiger</td>
        <td>Asia</td>
      </tr>
    </tbody>
  </table>
</section>
```

```
table {
  font-size: 1.2rem;
  text-align: left;
  color: black;
}

th,
td {
  padding: 0.2rem 1rem;
}

caption {
  background: #ffcc33;
  padding: 0.5rem 1rem;
}

tr {
  background: #eeeeee;
}

tr:nth-child(even) {
  background: #cccccc;
}
```

Syntax

```
/* Directional values */
caption-side: top;
caption-side: bottom;

/* Global values */
caption-side: inherit;
caption-side: initial;
caption-side: revert;
caption-side: revert-layer;
caption-side: unset;
```

Values

This property is specified as one of the following keyword values:

`top`

The caption box should be positioned at the block start side of the table.

`bottom`

The caption box should be positioned at the block end side of the table.

> **Note:** The CSS logical properties and values module defines two logical values, `inline-start` and `inline-end`, to position the caption box at the inline start edge and inline end edge of the table, respectively. These values are not supported in any browsers.

Formal definition

Initial value	`top`
Applies to	table-caption elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
caption-side =
  top    |
  bottom
```

This syntax reflects the latest standard as per CSS Table Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting captions above and below

HTML

```
<table class="top">
  <caption>
    Caption ABOVE the table
  </caption>
  <tbody>
```

```
    <tr>
      <td>Some data</td>
      <td>Some more data</td>
    </tr>
  </tbody>
</table>

<br />

<table class="bottom">
  <caption>
    Caption BELOW the table
  </caption>
  <tbody>
    <tr>
      <td>Some data</td>
      <td>Some more data</td>
    </tr>
  </tbody>
</table>
```

CSS

```
.top caption {
  caption-side: top;
}

.bottom caption {
  caption-side: bottom;
}

table {
  border: 1px solid red;
}

td {
  border: 1px solid blue;
}
```

Result

See also

- `<caption>`
- CSS table module
- CSS logical properties and values module

caret CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`caret`** CSS shorthand property sets the appearance and behavior of the **insertion caret** in a single declaration.

Try it

```
caret: red;
```

```
caret: block manual;
```

```
caret: underscore auto green;
```

```
caret: bar manual orange;
```

```
<section class="default-example container" id="default-example">
  <div>
    <label for="example-element">Edit text field:</label>
    <input id="example-element" type="text" value="Sample text" />
  </div>
</section>
```

```
div {
  text-align: left;
}

#example-element {
  font-size: 1.2rem;
  padding: 8px;
  width: 300px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `caret-color`
- `caret-animation`
- `caret-shape`

Syntax

```
/* Individual values */
caret: red; /* caret-color only */
caret: block; /* caret-shape only */
caret: manual; /* caret-animation only */

/* Two values */
```

```
caret: red manual; /* caret-color + caret-animation */
caret: block auto; /* caret-shape + caret-animation */
caret: underscore orange; /* caret-shape + caret-color */

/* Three values */
caret: bar manual red; /* caret-shape + caret-animation + caret-color */
caret: block auto #00ff00; /* caret-shape + caret-animation + caret-color */

/* Global values */
caret: inherit;
caret: initial;
caret: revert;
caret: revert-layer;
caret: unset;
```

The `caret` property is specified as one, two, or three values from the constituent properties. Values can be specified in any order, and omitted values are set to their initial values.

Values

`caret-color`

Sets the color of the caret.

`caret-animation`

Controls whether the caret blinks.

`caret-shape`

Sets the visual shape of the caret.

Description

The `caret` shorthand allows you to set multiple caret properties in a single declaration, making it convenient to customize the complete appearance and behavior of the insertion caret.

Value resolution

When values are omitted from the shorthand, they reset to their initial values:

- `caret-color`: `auto` (resolves to `currentColor`).
- `caret-animation`: `auto` (caret blinks).
- `caret-shape`: `auto` (browser-determined shape).

Order independence

Unlike some CSS shorthands, the `caret` property accepts values in any order. The browser determines which value applies to which property based on the value type:

- `<color>` values apply to `caret-color`.
- `auto`/`manual` keywords apply to `caret-animation`.
- Shape keywords (`bar`, `block`, `underscore`) apply to `caret-shape`.

Formal definition

Initial value	as each of the properties of the shorthand: • `caret-color`: `auto` • `caret-animation`: `auto` • `caret-shape`: `auto`
Applies to	Text or elements that accept text input
Inherited	yes
Computed value	as each of the properties of the shorthand: • `caret-color`: `auto` is computed as specified and `<color>` values are computed as defined for the `color` property. • `caret-animation`: as specified • `caret-shape`: as specified
Animation type	as each of the properties of the shorthand: • `caret-color`: a color • `caret-animation`: discrete • `caret-shape`: by computed value type

Formal syntax

```
caret =
  <'caret-color'>      ||
  <'caret-animation'>  ||
  <'caret-shape'>

<caret-color> =
  auto                        |
  <color> [ auto | <color> ]?

<caret-animation> =
  auto    |
  manual

<caret-shape> =
  auto        |
  bar         |
  block       |
  underscore
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Retro terminal with animated caret

This example creates a vintage terminal interface using the `caret` shorthand to combine multiple caret properties, demonstrating how it replaces older border-based techniques.

The main advantage of the `caret` shorthand is combining multiple properties in one declaration. Here we set the shape to `block`, disable default blinking, and set the color to `green`, all in a single line.

HTML

```
<label for="terminal">Enter a command</label>
<div class="old-screen">
  <span>></span>
  <textarea id="terminal" class="terminal-input"></textarea>
</div>
```

CSS

```
label {
  background: #092104;
  display: block;
  padding: 10px 20px;
  color: #00ad00;
  font-weight: bold;
  font-family: monospace;
}

.old-screen {
  background: repeating-linear-gradient(
    #092104,
    #092104 2px,
    #123208 2px,
    #123208 4px
  );
  height: 140px;
  display: flex;
  align-items: flex-start;
  padding: 20px;
  font-family: monospace;
}

span {
  display: inline-block;
  padding: 2px 5px;
  color: #00ad00;
  font-weight: bold;
  margin-right: 8px;
}

.terminal-input {
  background: transparent;
  height: 100%;
  border: none;
  color: #00ad00;
  font-family: inherit;
  font-size: 1rem;
  outline: none;
  flex: 1;
  resize: none;
}
```

```
.terminal-input {
  caret: block manual green;
  animation: vintage-caret 2s infinite;
}

@keyframes vintage-caret {
  0%,
  50% {
    caret-color: #00ad00;
  }
  75%,
  100% {
    caret-color: transparent;
  }
}
```

Result

See also

- `caret-color`, `caret-animation`, `caret-shape`
- CSS basic user interface module

caret-animation CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`caret-animation`** CSS property is used to enable or disable the blinking behavior of the **insertion caret**, the visible marker that appears in editable elements to indicate where the next character will be inserted or deleted.

The `caret-animation` property can also be set as part of the `caret` shorthand property.

> **Note:** When applying a custom animation to the caret, you should stop the default blinking so that it doesn't interfere with the animation.

Syntax

```
/* Keyword values */
caret-animation: auto;
caret-animation: manual;

/* Global values */
caret-animation: inherit;
caret-animation: initial;
caret-animation: revert;
caret-animation: revert-layer;
caret-animation: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The caret blinks on and off. This is the default (initial) value.

`manual`

The caret does not blink on and off.

Formal definition

Initial value	`auto`
Applies to	Text or elements that accept text input
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
caret-animation =
  auto    |
  manual
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>caret-animation</code> usage

This example shows the difference between having `caret-animation` set to `auto` versus `manual` on an editable element.

HTML

The markup features two `<p>` elements with `contenteditable` set to make them editable.

```
<p contenteditable="true">
  My caret animates because <code>caret-animation</code> is set to
  <code>auto</code>.
</p>
<p contenteditable="true">
  My caret doesn't animate because <code>caret-animation</code> is set to
  <code>manual</code>.
</p>
```

CSS

The CSS sets the `caret-color` value to `red`. It then gives the first paragraph a `caret-animation` value of `auto` and the second paragraph a `caret-animation` value of `manual`.

```
p {
  caret-color: red;
}

p:first-of-type {
  caret-animation: auto;
}

p:last-of-type {
  caret-animation: manual;
}
```

Result

The rendered result looks like so:

Try focusing the two paragraphs to see the difference in caret behavior.

Creating a custom caret animation

In this example, a custom caret animation is applied to an editable paragraph and a text input.

HTML

The markup features a `<p>` element and two text `<input>` elements. The `<p>` element has the `contenteditable` attribute set on it to make it editable. The paragraph and first text input have a `class` of `custom-caret` set on them.

```
<p contenteditable="true" class="custom-caret">
  This paragraph has a custom animation applied to it, plus
  <code>caret-animation: manual</code> to stop the default caret blinking and
  allow the smooth animation to be seen.
</p>

<input
```

```
  type="text"
  class="custom-caret"
  value="I've got a custom caret animation" />

<input type="text" value="I've got the default blinking caret" />
```

CSS

We first define a set of `@keyframes` that change the `caret-color` from `transparent` to `darkblue`.

```
@keyframes custom-caret-animation {
  from {
    caret-color: transparent;
  }

  to {
    caret-color: darkblue;
  }
}
```

We then style the `<p>` and the first `<input>` with the custom `@keyframes` animation, a `caret-color`, and a `caret-animation` value of `manual` to turn the default caret blinking behavior off.

```
body {
  display: flex;
  flex-direction: column;
}

input {
  margin-bottom: 20px;
}
```

```
.custom-caret {
  animation: custom-caret-animation infinite linear alternate 0.75s;
  caret-color: darkblue;
  caret-animation: manual;
}

p,
input {
  font-size: 1.6rem;
}
```

Result

The rendered result looks like so:

Try focusing the first two elements to see what the custom caret animation looks like. To compare it with the default blinking caret, you can focus the third element.

See also

- `caret-color`, `caret-shape`
- `caret` shorthand
- CSS basic user interface module

caret-color CSS property

The **`caret-color`** CSS property sets the color of the **insertion caret**, sometimes referred to as the **text input cursor**. This is the visible marker appearing at the insertion point where the next character typed will be added or where the next character deleted will be removed.

The `caret-color` property can also be set as part of the `caret` shorthand property.

Try it

```
caret-color: red;
```

```
caret-color: auto;
```

```
caret-color: transparent;
```

```
<section class="default-example container" id="default-example">
  <div>
    <p>Enter text in the field to see the caret:</p>
    <p><input id="example-element" type="text" /></p>
  </div>
</section>
```

```
#example-element {
  font-size: 1.2rem;
}
```

Syntax

```
/* Keyword values */
caret-color: auto;
caret-color: transparent;
caret-color: currentColor;

/* <color> values */
caret-color: red;
caret-color: #5729e9;
caret-color: rgb(0 200 0);
caret-color: hsl(228deg 4% 24% / 80%);

/* Global values */
caret-color: inherit;
caret-color: initial;
caret-color: revert;
```

```
caret-color: revert-layer;
caret-color: unset;
```

Values

This property is specified as a single value from the following list:

`auto`

Generally resolves to `currentColor`, the `color` of the text that will be modified.

`<color>`

The color of the caret.

Description

An insertion caret is a visible indicator of the spot within editable text — or within an element that accepts text input — where content will be inserted (or deleted) by the user. The caret is typically a thin vertical line indicating where the addition or deletion of characters will occur. It generally blinks (flashes on and off), so it is more noticeable. The caret only appears when the editable element has focus. By default, this caret is the color of the text. The `caret-color` property can be used to set the color of this caret to something other than the `currentColor`, or to reset a colored caret back to its default.

The `auto` value sets the insertion caret to `currentColor`, which is the `color` of the text that is being added or deleted. User agents may choose a different color to ensure good visibility and contrast with the surrounding content, taking into account the `color`, `background-color`, shadows, and other factors. In practice, however, all browsers use the current color by default, and when `caret-color` is set to `auto`. You can set any valid `<color>` as the value.

Understanding insertion carets

The insertion caret, and therefore this property, only applies to text or elements that can accept text input. The caret appears in focused user interface elements where users can update content, such as `<input>` elements that accept freeform text, the `<textarea>` element, and elements with the `contenteditable` attribute set.

The caret may appear in `<input>` elements of type `password`, `text`, `search`, `tel`, and `email`. No caret exists with date, `color`, `hidden`, `radio`, or `checkbox` input types. Some browsers display a caret with the `number` input type. It is possible in some browsers to make a caret appear in elements that never have text content — for example, by setting `appearance: none` and adding the `contenteditable` attribute. However, this is not recommended.

A caret may be displayed in an editable element or its descendants, provided the editability is not disabled, for example by setting a descendant element's `contentEditable` attribute to `false`. If an element is not editable or selectable, for example if `user-select` is set to `none`, the caret should not appear.

Caret versus cursor

There are multiple types of carets. The insertion caret is the only type affected by the `caret-color` property.

Many browsers have a **navigation caret**, which acts similarly to an insertion caret but can be moved around in non-editable text.

The mouse cursor image shown for certain `cursor` property values (for example, `auto` or `text`) may resemble a caret, but it's not. It's a cursor.

Animating from <code>auto</code>

Generally, when the `caret-color` is set to or defaults to `auto`, user agents use `currentColor`, which is animatable. However, `auto` is not an animatable value by default: when animating or transitioning the `caret-color` from `auto` to any color value, no interpolation happens. The animation is discrete; the color switches from or to the `currentColor` color at the midpoint of the `animation-duration` or `transition-duration`.

Formal definition

Initial value	`auto`
Applies to	Text or elements that accept text input
Inherited	yes
Computed value	`auto` is computed as specified and `<color>` values are computed as defined for the `color` property.
Animation type	a color

Formal syntax

```
caret-color =
  auto                        |
  <color> [ auto | <color> ]?
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a custom caret color

HTML

```
<input value="This field uses a default caret." size="64" />
<input class="custom" value="I have a custom caret color!" size="64" />
<p contenteditable class="custom">
  This paragraph can be edited, and its caret has a custom color as well!
</p>
```

CSS

```
input {
  caret-color: auto;
  display: block;
  margin-bottom: 0.5em;
}

input.custom {
  caret-color: orange;
}

p.custom {
  caret-color: green;
}
```

Result

See also

- `caret-animation`, `caret-shape`
- `caret` shorthand
- `color`
- `text-emphasis`
- `cursor`
- `text-emphasis`
- CSS `<color>` data type
- `<input>` element
- HTML `contenteditable` attribute

caret-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`caret-shape`** CSS property sets the shape of the **insertion caret**, the visible marker that appears in editable elements to indicate where the next character will be inserted or deleted.

The `caret-shape` property can also be set as part of the `caret` shorthand property.

Try it

```
caret-shape: auto;
```

```
caret-shape: bar;
```

```
caret-shape: block;
```

```
caret-shape: underscore;
```

```
<section class="default-example container" id="default-example">
  <div>
    <label for="example-element">Edit text field:</label>
    <input id="example-element" type="text" value="Sample text" />
  </div>
</section>
```

```
div {
  text-align: left;
}

#example-element {
  font-size: 1.2rem;
  padding: 8px;
  width: 300px;
}
```

Syntax

```
/* Keyword values */
caret-shape: auto;
caret-shape: bar;
caret-shape: block;
caret-shape: underscore;

/* Global values */
caret-shape: inherit;
caret-shape: initial;
caret-shape: revert;
caret-shape: revert-layer;
caret-shape: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The default value. The browser determines the caret shape. This typically follows platform conventions and may change based on context.

`bar`

The caret appears as a thin vertical line at the insertion point, positioned between characters rather than over them.

`block`

The caret appears as a rectangle that overlaps the next character after the insertion point. If no character follows, it appears after the last character.

`underscore`

The caret appears as a thin horizontal line underneath the next character after the insertion point. If no character follows, it appears after the last character.

Description

The insertion caret is the blinking cursor that indicates where text will be inserted when typing. Different caret shapes can provide visual feedback about the current editing mode or offer visual customization.

Editing modes and caret shapes

Text editors typically operate in one of two modes:

- **Insert mode**: New characters are inserted at the caret position, pushing existing text to the end of the line. This is the default behavior in most modern applications.
- **Overtype mode** (also called "overwrite mode"): New characters replace existing characters at the caret position instead of being inserted between them. This mode is often toggled with the `Insert` key.

Different caret shapes have traditional uses, for example:

- **Bar carets** are positioned between characters and are most common in modern interfaces.
- **Block carets** overlay the next character and are often used in terminal applications or to indicate overtype mode.
- **Underscore carets** appear below characters and can be useful for certain design aesthetics, such as mimicking typewriter or underline text input styles.

Caret positioning and behavior

The `caret-shape` property affects how the caret is visually rendered but doesn't change its logical position in the text. The caret always represents the insertion point between characters, regardless of its visual shape.

Interaction with writing modes

The caret shape adapts to the `writing-mode` of the text. In vertical writing modes, bar carets become horizontal, and underscore carets position themselves appropriately relative to the text direction.

Formal definition

Initial value	`auto`
Applies to	Text or elements that accept text input
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
caret-shape =
  auto        |
  bar         |
  block       |
  underscore
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Retro terminal with animated caret

This example shows how to create a vintage terminal interface using `caret-shape: block` with animated caret color, replacing the old technique of using borders.

The key part is using the modern caret properties instead of the old border-based technique. We set the caret to block shape, disable the default blinking, and create our own custom animation.

HTML

```
<label for="terminal">Enter a command</label>
<div class="old-screen">
  <span>></span>
  <textarea id="terminal" class="terminal-input"></textarea>
</div>
```

CSS

```
label {
  background: #092104;
  display: block;
  padding: 10px 20px;
  color: #00ad00;
  font-weight: bold;
  font-family: monospace;
}

.old-screen {
  background: repeating-linear-gradient(
    #092104,
    #092104 2px,
    #123208 2px,
    #123208 4px
  );
  height: 140px;
  display: flex;
  align-items: flex-start;
  padding: 20px;
  font-family: monospace;
}

span {
  display: inline-block;
  padding: 2px 5px;
  color: #00ad00;
  font-weight: bold;
  margin-right: 8px;
}

.terminal-input {
  background: transparent;
  height: 100%;
  border: none;
  color: #00ad00;
  font-family: inherit;
  font-size: 1rem;
  outline: none;
  flex: 1;
  resize: none;
}
```

```
.terminal-input {
  caret-shape: block;
  caret-animation: manual;
  animation: old-caret 2s infinite;
}

@keyframes old-caret {
```

```
  0%,
  50% {
    caret-color: #00ad00;
  }
  75%,
  100% {
    caret-color: transparent;
  }
}
```

Result

Console interface with underscore caret

This example demonstrates using `caret-shape: underscore` to create a console-style interface where the underscore caret complements the terminal aesthetic.

HTML

```
<label for="console">Enter a command</label>
<div class="console-demo">
  <div class="console-output">
    <p>user@host:css-ui-4 $ ls -a</p>
    <p>. .. Overview.bs Overview.html</p>
  </div>
  <div class="console-input">
    <span class="prompt">user@host:css-ui-4 $ </span>
    <input type="text" id="console" class="console" value="cd" />
  </div>
</div>
```

CSS

```
label {
  background: #2a2a2c;
  color: white;
  display: block;
  padding: 10px 20px;
  font-weight: bold;
  font-family: monospace;
}

.console-demo {
  background: black;
  color: white;
  font-family: monospace;
  padding: 10px 20px;
  height: 60px;
}

.console-output {
  margin-bottom: 0.5rem;
}

.console-output p {
  margin: 0 0.25rem;
}

.console-input {
  display: flex;
  align-items: center;
}

.prompt {
  margin-right: 0;
}

.console {
  background: transparent;
  border: none;
  color: white;
  padding-left: 1ch;
  font-family: inherit;
  outline: none;
  flex: 1;
}
```

```
.console {
  caret-shape: underscore;
}
```

Result

See also

- `caret-color`, `caret-animation`
- `caret` shorthand
- CSS basic user interface module

clear CSS property

The **`clear`** CSS property sets whether an element must be moved below (cleared) floating elements that precede it. The `clear` property applies to floating and non-floating elements.

Try it

```
clear: none;
```

```
clear: left;
```

```
clear: right;
```

```
clear: both;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="floated-left">Left</div>
    <div class="floated-right">Right</div>
    <div class="transition-all" id="example-element">
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  padding: 0.75em;
  text-align: left;
  line-height: normal;
}

.floated-left {
  border: solid 10px #ffc129;
  background-color: rgb(81 81 81 / 0.6);
  padding: 1em;
  float: left;
}

.floated-right {
  border: solid 10px #ffc129;
  background-color: rgb(81 81 81 / 0.6);
  padding: 1em;
  float: right;
  height: 150px;
}
```

Syntax

```
/* Keyword values */
clear: none;
clear: left;
clear: right;
clear: both;
clear: inline-start;
clear: inline-end;

/* Global values */
clear: inherit;
clear: initial;
clear: revert;
clear: revert-layer;
clear: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

Is a keyword indicating that the element is *not* moved down to clear past floating elements.

`left`

Is a keyword indicating that the element is moved down to clear past *left* floats.

`right`

Is a keyword indicating that the element is moved down to clear past *right* floats.

`both`

Is a keyword indicating that the element is moved down to clear past *both* left and right floats.

`inline-start`

Is a keyword indicating that the element is moved down to clear floats on *start side of its containing block*, that is the *left* floats on ltr scripts and the *right* floats on rtl scripts.

`inline-end`

Is a keyword indicating that the element is moved down to clear floats on *end side of its containing block*, that is the *right* floats on ltr scripts and the *left* floats on rtl scripts.

Description

When applied to non-floating blocks, it moves the border edge of the element down until it is below the margin edge of all relevant floats. The non-floated block's top margin collapses.

Vertical margins between two floated elements on the other hand will not collapse. When applied to floating elements, the margin edge of the bottom element is moved below the margin edge of all relevant floats. This affects the position of later floats, since later floats cannot be positioned higher than earlier ones.

The floats that are relevant to be cleared are the earlier floats within the same block formatting context.

> **Note:** If an element contains only floated elements, its height collapses to nothing. If you want it to always be able to resize, so that it contains floating elements inside it, set the value of the element's `display` property to `flow-root`.
>
> ```
> #container {
>   display: flow-root;
> }
> ```

Formal definition

Initial value	none
Applies to	block-level elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
clear =
  inline-start  |
  inline-end    |
  block-start   |
  block-end     |
  left          |
  right         |
  top           |
  bottom        |
  both-inline   |
  both-block    |
  both          |
  none
```

This syntax reflects the latest standard as per CSS Page Floats. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

clear: left

HTML

```
<div class="wrapper">
  <p class="black">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor.
  </p>
  <p class="red">Lorem ipsum dolor sit amet, consectetuer adipiscing elit.</p>
  <p class="left">This paragraph clears left.</p>
</div>
```

CSS

```
.wrapper {
  border: 1px solid black;
  padding: 10px;
}
.left {
  border: 1px solid black;
  clear: left;
}
.black {
  float: left;
  margin: 0;
  background-color: black;
  color: white;
  width: 20%;
}
.red {
  float: left;
  margin: 0;
  background-color: pink;
  width: 20%;
}
p {
  width: 50%;
}
```

clear: right

HTML

```
<div class="wrapper">
  <p class="black">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor.
  </p>
  <p class="red">Lorem ipsum dolor sit amet, consectetuer adipiscing elit.</p>
  <p class="right">This paragraph clears right.</p>
</div>
```

CSS

```
.wrapper {
  border: 1px solid black;
  padding: 10px;
}
.right {
  border: 1px solid black;
  clear: right;
}
.black {
  float: right;
  margin: 0;
  background-color: black;
  color: white;
  width: 20%;
}
.red {
```

```
  float: right;
  margin: 0;
  background-color: pink;
  width: 20%;
}
p {
  width: 50%;
}
```

clear: both

HTML

```
<div class="wrapper">
  <p class="black">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor. Fusce pulvinar lacus
    ac dui.
  </p>
  <p class="red">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor.
  </p>
  <p class="both">This paragraph clears both.</p>
</div>
```

CSS

```
.wrapper {
  border: 1px solid black;
  padding: 10px;
}
.both {
  border: 1px solid black;
  clear: both;
}
.black {
  float: left;
  margin: 0;
  background-color: black;
  color: white;
  width: 20%;
}
.red {
  float: right;
  margin: 0;
  background-color: pink;
  width: 20%;
}
p {
  width: 45%;
}
```

See also

- CSS basic box model

clip CSS property

> **Warning:** Authors are encouraged to use the `clip-path` property instead.

The **`clip`** CSS property defines a visible portion of an element. The `clip` property applies only to absolutely positioned elements — that is, elements with `position:absolute` or `position:fixed`.

Syntax

```
/* Keyword value */
clip: auto;

/* <shape> values */
clip: rect(1px, 10em, 3rem, 2ch);

/* Global values */
clip: inherit;
clip: initial;
clip: revert;
clip: revert-layer;
clip: unset;
```

Values

This property is specified as a single value from the following list:

`rect()`

A rectangle defined using a `rect()` function of the form `rect(<top>, <right>, <bottom>, <left>)`. The `<top>` and `<bottom>` values are offsets from the *inside top border edge* of the box, while `<right>` and `<left>` are offsets from the *inside left border edge* of the box — that is, the extent of the padding box.

The `<top>`, `<right>`, `<bottom>`, and `<left>` values may be either a `<length>` or `auto`. If any side's value is `auto`, the element is clipped to that side's *inside border edge*.

> **Note:** The `rect()` `<shape>` function used in the deprecated `clip` property is different from the CSS `rect()` function used to define a CSS `<basic-shape>`.

`auto`

The element does not clip (default). This is different from `rect(auto, auto, auto, auto)`, which clips to the element's inside border edges.

Formal definition

Initial value	`auto`
Applies to	absolutely positioned elements
Inherited	no
Computed value	`auto` if specified as `auto`, otherwise a rectangle with four values, each of which is `auto` if specified as `auto` or the computed length otherwise
Animation type	a rectangle

Formal syntax

```
clip =
  <rect()>  |
  auto

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )
```

This syntax reflects the latest standard as per CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Clipping an image

```
<p class="dotted-border">
  <img src="macarons.png" alt="Original graphic" />
  <img id="top-left" src="macarons.png" alt="Graphic clipped to upper left" />
  <img id="middle" src="macarons.png" alt="Graphic clipped towards middle" />
  <img
    id="bottom-right"
    src="macarons.png"
    alt="Graphic clipped to bottom right" />
</p>
```

```
.dotted-border {
  border: dotted;
  position: relative;
  width: 390px;
  height: 400px;
}

#top-left,
#middle,
#bottom-right {
  position: absolute;
  top: 0;
}

#top-left {
  left: 400px;
  clip: rect(0, 130px, 90px, 0);
}

#middle {
  left: 270px;
  clip: rect(100px, 260px, 190px, 130px);
}

#bottom-right {
  left: 140px;
  clip: rect(200px, 390px, 290px, 260px);
}
```

See also

- `clip-path`
- `position`
- `mask`
- `shape-image-threshold`
- `shape-outside`

clip-path CSS property

The **`clip-path`** CSS property creates a clipping region that sets what part of an element should be shown. Parts that are inside the region are shown, while those outside are hidden.

Try it

```
clip-path: circle(40%);
```

```
clip-path: ellipse(130px 140px at 10% 20%);
```

```
clip-path: polygon(50% 0, 100% 50%, 50% 100%, 0 50%);
```

```
clip-path: path("M 0 200 L 0,75 A 5,5 0,0,1 150,75 L 200 200 z");
```

```
clip-path: rect(5px 145px 160px 5px round 20%);
```

```
clip-path: xywh(0 5px 100% 75% round 15% 0);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <img
      class="transition-all"
      id="example-element"
      src="/shared-assets/images/examples/balloon-small.jpg"
      width="150" />
```

```
    We had agreed, my companion and I, that I should call for him at his house,
    after dinner, not later than eleven o'clock. This athletic young Frenchman
    belongs to a small set of Parisian sportsmen, who have taken up "ballooning"
    as a pastime. After having exhausted all the sensations that are to be found
    in ordinary sports, even those of "automobiling" at a breakneck speed, the
    members of the "Aéro Club" now seek in the air, where they indulge in all
    kinds of daring feats, the nerve-racking excitement that they have ceased to
    find on earth.
  </div>
</section>
```

```
section {
  align-items: flex-start;
}

.example-container {
  text-align: left;
  padding: 20px;
}

#example-element {
  float: left;
  width: 150px;
  margin: 20px;
}
```

Syntax

```
/* Keyword value */
clip-path: none;

/* <clip-source> values */
clip-path: url("resources.svg#c1");

/* <geometry-box> values */
clip-path: margin-box;
clip-path: border-box;
clip-path: padding-box;
clip-path: content-box;
clip-path: fill-box;
clip-path: stroke-box;
clip-path: view-box;

/* <basic-shape> values */
clip-path: inset(100px 50px);
clip-path: circle(50px at 0 100px);
clip-path: ellipse(50px 60px at 10% 20%);
clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
clip-path: path(
  "M0.5,1 C0.5,1,0,0.7,0,0.3 A0.25,0.25,1,1,1,0.5,0.3 A0.25,0.25,1,1,1,1,0.3 C1,0.7,0.5,1,0.5,1 Z"
);
clip-path: rect(5px 5px 160px 145px round 20%);
clip-path: shape(from 0% 0%, line to 100% 0%, line to 50% 100%, close);
clip-path: xywh(0 5px 100% 75% round 15% 0);

/* Box and shape values combined */
clip-path: padding-box circle(50px at 0 100px);

/* Global values */
clip-path: inherit;
clip-path: initial;
clip-path: revert;
clip-path: revert-layer;
clip-path: unset;
```

The `clip-path` property is specified as one or a combination of the values listed below.

Values

`<clip-source>`

A `<url>` referencing an SVG `<clipPath>` element.

`<basic-shape>`

A shape whose size and position is defined by the `<geometry-box>` value. If no geometry box is specified, the `border-box` will be used as the reference box. One of:

`inset()`

Defines an inset rectangle.

`circle()`

Defines a circle using a radius and a position.

`ellipse()`

Defines an ellipse using two radii and a position.

`polygon()`

Defines a polygon using an SVG filling rule and a set of vertices.

`path()`

Defines a shape using an optional SVG filling rule and an SVG path definition.

`rect()`

Defines a rectangle using the specified distances from the edges of the reference box.

`shape()`

Defines a shape using an optional SVG filling rule and shape commands for lines, curves, and arcs.

`xywh()`

Defines a rectangle using the specified distances from the top and left edges of the reference box and the specified width and height of the rectangle.

`<geometry-box>`

If specified in combination with a `<basic-shape>`, this value defines the reference box for the basic shape. If specified by itself, it causes the edges of the specified box, including any corner shaping (such as a `border-radius`), to be the clipping path. The geometry box can be one of the following values:

`margin-box`

Uses the margin box as the reference box.

`border-box`

Uses the border box as the reference box.

`padding-box`

Uses the padding box as the reference box.

`content-box`

Uses the content box as the reference box.

`fill-box`

Uses the object bounding box as the reference box.

`stroke-box`

Uses the stroke bounding box as the reference box.

`view-box`

Uses the nearest SVG viewport as the reference box. If a `viewBox` attribute is specified for the element creating the SVG viewport, the reference box is positioned at the origin of the coordinate system established by the `viewBox` attribute and the dimension of the size of the reference box is set to the width and height values of the `viewBox` attribute.

`none`

No clipping path is created.

> **Note:** A computed value other than `none` results in the creation of a new stacking context the same way that CSS `opacity` does for values other than `1`.

Formal definition

Initial value	none
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Percentages	refer to reference box when specified, otherwise border-box
Computed value	as specified, but with `<url>` values made absolute
Animation type	yes, as specified for `<basic-shape>`, otherwise no

Formal syntax

```
clip-path =
  <clip-source>                      |
  [ <basic-shape> || <geometry-box> ]  |
  none

<clip-source> =
  <url>

<basic-shape> =
  <basic-shape-rect>  |
  <circle()>          |
  <ellipse()>         |
  <polygon()>         |
  <path()>            |
  <shape()>

<geometry-box> =
  <shape-box>  |
  fill-box     |
  stroke-box   |
  view-box

<basic-shape-rect> =
  <inset()>  |
  <rect()>   |
  <xywh()>

<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<path()> =
  path( <'fill-rule'>? , <string> )

<shape()> =
  shape( <'fill-rule'>? from <position> , <shape-command># )

<shape-box> =
```

```
  <visual-box>     |
  margin-box       |
  half-border-box

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<fill-rule> =
  nonzero  |
  evenodd

<length-percentage> =
  <length>      |
  <percentage>

<shape-command> =
  <move-command>             |
  <line-command>             |
  close                      |
  <horizontal-line-command>  |
  <vertical-line-command>    |
  <curve-command>            |
  <smooth-command>           |
  <arc-command>

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<position-one> =
  left                 |
  center               |
  right                |
  top                  |
  bottom               |
  x-start              |
  x-end                |
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<move-command> =
  move <command-end-point>

<line-command> =
  line <command-end-point>

<horizontal-line-command> =
  hline [ to [ <length-percentage> | left | center | right | x-start | x-end ] | by <length-percentage> ]

<vertical-line-command> =
  vline [ to [ <length-percentage> | top | center | bottom | y-start | y-end ] | by <length-percentage> ]

<curve-command> =
  curve [ [ to <position> with <control-point> [ / <control-point> ]? ] | [ by <coordinate-pair> with <relative-control-point> [ / <relative-control-point> ]? ] ]

<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
```

```
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Box Model Module Level 4, CSS Fill and Stroke Module Level 3, CSS Images Module Level 3, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Shapes and geometry boxes

In this example, two triangles are created by defining a `polygon()` as the clip path on `<div>` elements. Each one has a solid colored background and a thick `border`. The second `<div>` element has its reference box set to `content-box`:

HTML

```
<div></div>
<div></div>
```

CSS

```
body {
  display: flex;
  gap: 20px;
  flex-flow: row wrap;
}
```

```
div {
  height: 200px;
  width: 200px;
  box-sizing: border-box;
  background-color: rebeccapurple;
  border: 20px solid magenta;

  clip-path: polygon(50% 0, 100% 100%, 0 100%);
}

div:last-of-type {
  clip-path: content-box polygon(50% 0, 100% 100%, 0 100%);
}
```

Results

For the first triangle, we didn't specify a reference box; it therefore defaults to `border-box`, with the 0% and 100% positions located on the outside edge of the border. In the second example, we set the `<geometry-box>` to `content-box`, meaning the reference box for the basic shape is the outer edge of the content area, which is inside the padding box. Because our example has no `padding`, this is the inner edge of the border.

<code>shape()</code> versus <code>path()</code> functions

Expanding on the previous example, we create the same triangle with different `<basic-shape>` values, demonstrating how the `shape()` and `path()` functions can also be used to create clipping paths, with `shape()` being a more flexible solution.

We use `path()` to define the first element's clipping path, and `shape()` for the second, both using the default `border-box` as their reference box:

```
div {
  clip-path: path("M100 0 L200 200 L0 200 Z");
}

div:last-of-type {
  clip-path: shape(from 50% 0, line to 100% 100%, line to 0 100%, close);
}
```

As a result, the path defined with the shape() function grows with the element, whereas the path() version does not:

Because the `shape()` function allows using `<percentage>` values (and custom properties too), it is more robust.

We'll demonstrate this by increasing the size of the underlying element:

```
div {
  width: 250px;
  height: 250px;
}
```

The visibility, or at least partial visibility, of the four border sides in the clip path example defined by the `shape()` function is due to the percentage values allowing the path to grow with the element. In the `path()` version, the element grew, but not the shape. As a result, the top and left borders are partially visible while the right and bottom borders are clipped out.

SVG as clip source

In this example, we define SVG `<clipPath>` elements to use as a `clip-path` source.

HTML

We include two `<div>` elements and an `<svg>` element containing two `<clipPath>` elements. One `<clipPath>` contains four `<rect>` elements that together define window panes, leaving a cross of blank space in the middle, and the other contains two crossing `<rect>` elements.

```
<svg height="0" width="0">
  <defs>
```

```
    <clipPath id="window">
      <rect y="0" x="0" width="80" height="80" />
      <rect y="0" x="120" width="80" height="80" />
      <rect y="120" x="0" width="80" height="80" />
      <rect y="120" x="120" width="80" height="80" />
    </clipPath>
    <clipPath id="cross">
      <rect y="0" x="80" width="40" height="200" />
      <rect y="80" x="0" width="200" height="40" />
    </clipPath>
  </defs>
</svg>

<div class="window">Window</div>
<div class="cross">Cross</div>
```

CSS

We use flexbox to allow our elements to sit side-by-side with a gap between them, if there is space available. We define a `conic-gradient()` background image on both `<div>` elements, providing an interesting visual to clip, along with a `border`.

```
body {
  display: flex;
  gap: 20px;
  flex-flow: row wrap;
  font: 2em sans-serif;
}

div {
  width: 200px;
  height: 200px;
  background-image: conic-gradient(
    at center,
    rebeccapurple,
    green,
    lightblue,
    rebeccapurple
  );

  border: 5px solid;
  box-sizing: border-box;
}
```

We then set the `id` of the `<clipPath>` as the `<clip-source>`. We center the text in the `cross` example vertically using `align-content`, as otherwise the text would be clipped, as is happening in the `window` example.

```
.window {
  clip-path: url("#window");
}

.cross {
  clip-path: url("#cross");
  align-content: center;
}
```

Results

The elements, including their border and text, are clipped, with only the parts overlapping the `<clipPath>` elements being drawn to the page.

The various value types

This example demonstrates the various values of the `clip-path` property clipping an HTML `<img>`.

HTML

The HTML includes an `<img>` that will be clipped, a star-shaped `<clipPath>`, and a `<select>` element to choose a `clip-path` property value from.

```
<img
  id="clipped"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<svg height="0" width="0">
  <defs>
    <clipPath id="star">
      <path d="M100,0 42,180 196,70 4,70 158,180z" />
    </clipPath>
  </defs>
</svg>

<select id="clipPath">
  <option value="none">none</option>
  <option value="circle(100px at 110px 100px)">circle</option>
  <option value="url(#star)" selected>star</option>
  <option value="inset(20px round 20px)">inset</option>
  <option value="rect(20px 150px 200px 20px round 10%)">rect</option>
  <option value="xywh(0 20% 90% 67% round 0 0 5% 5px)">xywh</option>
  <option value="path('M 0 200 L 0,110 A 110,90 0,0,1 240,100 L 200 340 z')">
    path
  </option>
</select>
```

```
<pre id="log"></pre>
```

CSS

The initial rendering includes the star as the `clip-path` source.

```
#clipped {
  margin-bottom: 20px;
```

```
  clip-path: url("#star");
}
```

JavaScript

When you select a new option from the `<select>` menu, an event handler updates the value of the `clip-path` set on the `<img>`.

```
const clipPathSelect = document.getElementById("clipPath");
clipPathSelect.addEventListener("change", (evt) => {
  const path = evt.target.value;
  document.getElementById("clipped").style.clipPath = path;
  log(`clip-path: ${path};`);
});
```

```
function log(text) {
  const logElement = document.querySelector("#log");
  logElement.innerText = `${text}`;
  logElement.scrollTop = logElement.scrollHeight;
}
```

Result

Select different options to change the `clip-path` value.

> **Note:** While it is possible to define a path of text, if you want to clip a background image to text rather than a shape, see the `background-clip` property.

See also

- `clip-rule`
- `mask`
- `filter`
- `background-clip`
- Introduction to CSS clipping
- CSS masking module
- SVG `clip-path` attribute
- Applying SVG effects to HTML content

clip-rule CSS property

The **`clip-rule`** CSS property determines, when parts of the path overlap other parts, which pixels in a mask's box are inside the clipping shape defined by a clip path and which are outside.

The `clip-rule` property only applies to SVG elements that are contained within a `<clipPath>` element, overriding the element's `clip-rule` attribute value if present. The `clip-rule` property basically works as the `fill-rule` property, except that it applies to `<clipPath>` definitions. It does not have any effect on CSS `<basic-shape>`s.

Syntax

```
/* Keyword values */
clip-rule: nonzero;
clip-rule: evenodd;

/* Global values */
clip-rule: inherit;
clip-rule: initial;
clip-rule: revert;
clip-rule: revert-layer;
clip-rule: unset;
```

Values

This property is specified as one of the following keyword values:

`nonzero`

> For every point in the clipping mask's box, a ray is drawn in a random direction. Every time the ray intersects with any part of the clipping path, a tally is increased by one if the clipping path's part is moving from left to right across the ray, whereas it is decreased by one if the path part is moving right to left across the ray. If the final total of the tally is zero, the point is outside the path's shape. Otherwise, it's inside the path's shape.

`even-odd`

> For every point in the clipping mask's box, a ray is drawn in a random direction. Every time the ray intersects with any part of the clipping path, a tally is increased by one. If the final total of the tally is even, the point is outside the path's shape; otherwise, it's inside the path's shape. Zero is taken to be even.

Formal syntax

```
clip-rule =
  nonzero  |
  evenodd
```

This syntax reflects the latest standard as per CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Value comparison

In this example, we will apply different CSS `clip-rule` values to similar SVG `<path>` elements to demonstrate the difference between `evenodd` and `non-zero`.

HTML

The markup has multiple `<svg>` containers, each containing a `<clipPath>` element that defines a star shape, and a `<rect>` element to draw the star inside. The stars are created with overlapping lines. Other than the `id`, the markup of the first two SVG elements is identical. The third SVG includes just the `<path>` element, showing how the lines of the path that created the star overlap.

```
<svg>
  <clipPath id="star1">
    <path d="M50,0 21,90 98,35 2,35 79,90z" />
  </clipPath>
  <rect clip-path="url(#star1)" width="95" height="95" />
</svg>
```

```
<svg>
  <clipPath id="star2">
    <path d="M50,0 21,90 98,35 2,35 79,90z" />
  </clipPath>
  <rect clip-path="url(#star2)" width="95" height="95" />
</svg>

<svg id="star3">
  <path d="M50,0 21,90 98,35 2,35 79,90z" />
</svg>
```

CSS

The `clip-rule` for the `<path>` in the first SVG is set to `evenodd`; `nonzero` in the second SVG. For the path-only SVG, we removed the default `fill` and defined both a `stroke` color and `stroke-width` to make the overlapping path lines visible:

```
body {
  display: flex;
  gap: 20px;
}
svg {
  width: 110px;
  height: 110px;
}
```

```
#star1 path {
  clip-rule: evenodd;
}

#star2 path {
  clip-rule: nonzero;
}

#star3 path {
  fill: none;
  stroke: black;
  stroke-width: 1;
}
```

Results

Within basic shape definitions

This example demonstrates that, while the `clip-rule` does not have any effect on CSS `<basic-shape>`s, it can affect a `<clipPath>` used as the source of a shape.

HTML

We include an SVG with two `<clipPath>` elements that define star shapes, which are identical except for their `id` attribute values. We also include two `<div>` elements that will contain our star shapes.

```
<svg height="0" width="0">
  <defs>
    <clipPath id="star1">
      <path d="M100,0 42,180 196,70 4,70 158,180z" />
    </clipPath>
    <clipPath id="star2">
      <path d="M100,0 42,180 196,70 4,70 158,180z" />
    </clipPath>
  </defs>
</svg>

<div></div>
<div></div>
```

CSS

We provide the `<div>` elements with a set `width` and `height`, adding a `conic-gradient()` for their `background-image` value:

```
body {
  display: flex;
  gap: 20px;
}
```

```
div {
  height: 200px;
  width: 200px;
  background-image: conic-gradient(
    at center,
    rebeccapurple,
    green,
    lightblue,
    rebeccapurple
  );
}
```

We use the `clip-path` property to set the different `<clipPath>` elements as the clipping path for each `<div>`:

```
div:first-of-type {
  clip-path: url("#star1");
}
div:last-of-type {
  clip-path: url("#star2");
}
```

Finally, we set the different `clip-rule` values for each of the `<clipPath>` element's `<path>`s:

```
#star1 path {
  clip-rule: evenodd;
}
```

```
#star2 path {
  clip-rule: nonzero;
}
```

Results

Choosing between rules for a path with all clockwise paths

In this SVG image, we have two rectangles that are clipped, once with each clipping rule. There are two `<clipPath>` elements, so that one can be set to use the non-zero clipping rule and the other uses the even-odd rule. Both paths are drawn in a clockwise direction for both its inner and outer parts.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 50">
  <g stroke="#112233" fill="#bbccdd">
    <!-- basic rectangle and clipping path visualization follow -->
    <rect x="10" y="10" width="30" height="30" />
    <path
      d="M 65,5 l 20,20 -20,20 -20,-20 20,-20 m 0,10 l 10,10 -10,10 -10,-10 10,-10 z"
      fill="none"
      stroke-width="0.5" />
    <!-- rectangles to be clipped follow -->
    <rect x="110" y="10" width="30" height="30" clip-path="url(#clipper1)" />
    <rect x="160" y="10" width="30" height="30" clip-path="url(#clipper2)" />
  </g>
  <!-- clipping paths follow -->
  <clipPath id="clipper1" clipPathUnits="objectBoundingBox">
    <path
      d="M 0.5 -0.15 l 0.65 0.65 -0.65,0.65 -0.65,-0.65 0.65,-0.65 m 0,0.33 l 0.33,0.33 -0.33,0.33 -0.33,-0.33 0.33,-0.33 z"
      clip-rule="evenodd" />
  </clipPath>
  <clipPath id="clipper2" clipPathUnits="objectBoundingBox">
    <path
      d="M 0.5 -0.15 l 0.65 0.65 -0.65,0.65 -0.65,-0.65 0.65,-0.65 m 0,0.33 l 0.33,0.33 -0.33,0.33 -0.33,-0.33 0.33,-0.33 z"
      clip-rule="nonzero" />
  </clipPath>
</svg>
```

To the clipping paths that are applied to the clipped rectangles, the CSS `clip-rule` property is used to set one path to use the `nonzero` rules, and the other to use the `evenodd` rule. These override the values of the `clip-path` attributes in the SVG, which have been intentionally set to the opposite values as the CSS imposes.

```
#clipper1 {
  clip-rule: nonzero;
}
#clipper2 {
  clip-rule: evenodd;
}
```

Because both the inner and outer parts of the path move in a clockwise (left-to-right) direction, the resulting clip shape will be different between the two clipping rules. For `nonzero`, any ray inside the outer part of the shape will tally to a value above zero, because it will encounter one or more left-to-right path fragments. For `even-odd`, points between the two parts of the path will have an odd-numbered tally, whereas any point either inside the inner path or outside the outer part will have an even-numbered tally.

Choosing between rules for a path with different winding paths

This example uses the same SVG as the previous example, with the change that the interior part of the clipping path winds in a counterclockwise direction.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 50">
  <g stroke="#112233" fill="#bbccdd">
    <!-- basic rectangle and clipping path visualization follow -->
    <rect x="10" y="10" width="30" height="30" />
    <path
      d="M 65,5 l 20,20 -20,20 -20,-20 20,-20 m 0,10 l 10,10 -10,10 -10,-10 10,-10 z"
      fill="none"
      stroke-width="0.5" />
    <!-- rectangles to be clipped follow -->
    <rect x="110" y="10" width="30" height="30" clip-path="url(#clipper1)" />
    <rect x="160" y="10" width="30" height="30" clip-path="url(#clipper2)" />
  </g>
  <!-- clipping paths follow -->
  <clipPath id="clipper1" clipPathUnits="objectBoundingBox">
    <path
      d="M 0.5 -0.15 l 0.65 0.65 -0.65,0.65 -0.65,-0.65 0.65,-0.65 m 0,0.33 l -0.33,0.33 0.33,0.33 0.33,-0.33 -0.33,-0.33 z" />
  </clipPath>
  <clipPath id="clipper2" clipPathUnits="objectBoundingBox">
    <path
      d="M 0.5 -0.15 l 0.65 0.65 -0.65,0.65 -0.65,-0.65 0.65,-0.65 m 0,0.33 l 0.33,0.33 -0.33,0.33 -0.33,-0.33 0.33,-0.33 z" />
  </clipPath>
</svg>
```

We apply the same CSS as before.

```
#clipper1 {
  clip-rule: nonzero;
}
#clipper2 {
  clip-rule: evenodd;
}
```

In this case, because the outer part of the path moves in a clockwise (left-to-right) direction and the interior part of the path moves in a counterclockwise (right-to-left) direction, the resulting clip shape will be the same regardless of which clipping rule is used.

See also

- `fill-rule`
- `clip-path`
- Introduction to CSS clipping
- CSS masking module
- SVG `clip-rule` attribute
- SVG `<clipPath>` element
- SVG `fill-rule` attribute

color CSS property

The **`color`** CSS property sets the foreground color value of an element's text and text decorations, and sets the `currentColor` value. `currentColor` may be used as an indirect value on *other* properties and is the default for other color properties, such as `border-color`.

For an overview of using color in HTML, see Applying color to HTML elements using CSS.

Try it

```
color: rebeccapurple;
```

```
color: #00a400;
```

```
color: rgb(214 122 127);
```

```
color: hsl(30deg 82% 43%);
```

```
color: hsl(237deg 74% 33% / 61%);
```

```
color: hwb(152deg 0% 58% / 70%);
```

```
<section id="default-example">
  <div class="example-container">
    <p id="example-element">
      London. Michaelmas term lately over, and the Lord Chancellor sitting in
      Lincoln's Inn Hall. Implacable November weather.
    </p>
  </div>
</section>
```

```
#example-element {
  font-size: 1.5em;
}

.example-container {
  background-color: white;
  padding: 10px;
}
```

Syntax

```
/* Keyword value */
color: currentColor;

/* <named-color> values */
color: red;
color: orange;
color: tan;
color: rebeccapurple;

/* <hex-color> values */
color: #090;
color: #009900;
color: #090a;
color: #009900aa;

/* <rgb()> values and legacy <rgba()> values*/
color: rgb(34, 12, 64);
color: rgb(34, 12, 64, 0.6);
color: rgba(34, 12, 64, 0.6);
color: rgb(34 12 64 / 0.6);
color: rgba(34 12 64 / 0.6);
color: rgb(34.6 12 64 / 60%);
color: rgba(34.6 12 64 / 60%);

/* <hsl()> values and legacy <hsla()> values */
color: hsl(30, 100%, 50%);
color: hsl(30, 100%, 50%, 0.6);
color: hsla(30, 100%, 50%, 0.6);
color: hsl(30 100% 50% / 0.6);
color: hsla(30 100% 50% / 0.6);
color: hsl(30.2 100% 50% / 60%);
color: hsla(30.2 100% 50% / 60%);

/* <hwb()> values */
color: hwb(90 10% 10%);
color: hwb(90 10% 10% / 0.5);
color: hwb(90deg 10% 10%);
color: hwb(1.5708rad 60% 0%);
color: hwb(0.25turn 0% 40% / 50%);

/* Global values */
color: inherit;
color: initial;
color: revert;
color: revert-layer;
color: unset;
```

Values

This property is specified as a single value from the following list:

`<color>`

Sets the color of the textual and decorative parts of the element.

`currentColor`

Sets the color to the element's `color` property value. However, if set as the value of `color`, `currentColor` is treated as `inherit`.

Accessibility

It is important to ensure that the contrast ratio between the color of the text and the background the text is placed over is high enough that people experiencing low vision conditions will be able to read the content of the page.

Color contrast ratio is determined by comparing the luminosity of the text and background color values. In order to meet current Web Content Accessibility Guidelines (WCAG), a ratio of 4.5:1 is required for text content and 3:1 for larger text such as headings. Large text is defined as 18.66px and bold or larger, or 24px or larger.

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`canvastext`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	computed color
Animation type	by computed value type

Formal syntax

```
color =
  <color>
```

This syntax reflects the latest standard as per CSS Color Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Making text red

The following are all ways to make a paragraph's text red:

```
p {
  color: red;
}
p {
  color: #f00;
}
p {
  color: #ff0000;
}
p {
  color: rgb(255 0 0);
}
p {
  color: rgb(100% 0% 0%);
}
p {
  color: hsl(0 100% 50%);
}

/* 50% translucent */
p {
  color: #ff000080;
}
p {
  color: rgb(255 0 0 / 50%);
}
p {
  color: hsl(0 100% 50% / 50%);
}
```

See also

- The `<color>` data type
- Other color-related properties: `background-color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, `column-rule-color`, and `print-color-adjust`
- SVG `color` attribute
- `color()` function
- Applying color to HTML elements using CSS
- WCAG: color contrast

color-interpolation CSS property

The `color-interpolation` CSS property is used in SVG to specify which color space to use for `<linearGradient>` and `<radialGradient>` SVG elements.

Syntax

```
/* Keyword values */
color-interpolation: auto;
color-interpolation: sRGB;
color-interpolation: linearRGB;
```

Values

This property is specified as one of the following keyword values:

`auto`

> Indicates that the user agent can choose either the `sRGB` or `linearRGB` spaces for color interpolation. This option indicates that the author doesn't require that color interpolation occur in a particular color space.

`sRGB`

> Indicates that color interpolation should occur in the sRGB color space. Defaults to this initial value if no `color-interpolation` property is set.

`linearRGB`

Indicates that color interpolation should occur in the linearized RGB color space as described in the sRGB specification.

Formal definition

Value	`auto` | `sRGB` | `linearRGB`
Applies to	`<linearGradient>` and `<radialGradient>`
Default value	`auto`
Animatable	discrete

Formal syntax

```
color-interpolation =
  auto       |
  sRGB       |
  linearRGB
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

In the first SVG, the `color-interpolation` property is not included on the `<linearGradient>` element and color interpolation defaults to `sRGB`. The second example shows color interpolation using the `linearRGB` value.

```
<svg width="450" height="70">
  <title>Example of using the color-interpolation CSS Property</title>
  <defs>
    <linearGradient id="sRGB">
      <stop offset="0%" stop-color="white" />
      <stop offset="25%" stop-color="blue" />
      <stop offset="50%" stop-color="white" />
      <stop offset="75%" stop-color="red" />
      <stop offset="100%" stop-color="white" />
    </linearGradient>
  </defs>
  <rect x="0" y="0" width="400" height="40" fill="url(#sRGB)" stroke="black" />
  <text x="0" y="60" font-family="courier" font-size="16">
    no color-interpolation (CSS property)
  </text>
</svg>
```

```
<svg width="450" height="70">
  <title>Example of using the color-interpolation CSS Property</title>
  <defs>
    <linearGradient id="linearRGB">
      <stop offset="0%" stop-color="white" />
      <stop offset="25%" stop-color="blue" />
      <stop offset="50%" stop-color="white" />
      <stop offset="75%" stop-color="red" />
      <stop offset="100%" stop-color="white" />
    </linearGradient>
  </defs>
  <rect
    x="0"
    y="0"
    width="400"
    height="40"
    fill="url(#linearRGB)"
    stroke="black" />
  <text x="0" y="60" font-family="courier" font-size="16">
    color-interpolation: linearRGB; (CSS property)
  </text>
</svg>
```

```
svg {
  display: block;
}

#linearRGB {
  color-interpolation: linearRGB;
}
```

See also

- `<linearGradient>`
- `<radialGradient>`
- SVG `color-interpolation` attribute

color-interpolation-filters CSS property

The **`color-interpolation-filters`** CSS property specifies the color space for imaging operations performed via SVG filter effects. If explicitly declared, the value of the CSS property overrides any value given in the element's `color-interpolation-filters` attribute.

> **Note:** The `color-interpolation-filters` property is only relevant to SVG filter operations. It has *no* effect on filter primitives like `<feOffset>`, `<feImage>`, `<feTile>`, and `<feFlood>`, but instead applies to the various filter effect elements (e.g., `<feBlend>`); see the SVG `color-interpolation-filters` page for a full list.

> **Note:** It is important to remember that the SVG `color-interpolation` attribute has an initial value of `sRGB`, while `color-interpolation-filters` has an initial value of `linearRGB`. This means, in the default case, filter effect interpolations occur in a different color space than all other color interpolations.

Syntax

```
color-interpolation-filters: auto;
color-interpolation-filters: linearRGB;
color-interpolation-filters: sRGB;
```

```
/* Global values */
color-interpolation-filters: inherit;
color-interpolation-filters: initial;
color-interpolation-filters: revert;
color-interpolation-filters: revert-layer;
color-interpolation-filters: unset;
```

Values

This property is specified as one of the following keyword values:

`linearRGB`

Indicates that color interpolation should occur in the linearized RGB color space as described in the sRGB specification. This is the default property value.

`sRGB`

Indicates that color interpolation should occur in the gamma-encoded sRGB color space.

`auto`

Indicates that the user agent can choose either the `sRGB` or `linearRGB` spaces for color interpolation. This option indicates that the author doesn't require that color interpolation occur in a particular color space.

Formal definition

Initial value	`linearRGB`
Applies to	The set of elements that control the output of a `<filter>` element in `<svg>`
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
color-interpolation-filters =
  auto       |
  sRGB       |
  linearRGB
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- `color-interpolation`
- SVG `color-interpolation-filters` attribute
- sRGB specification

color-scheme CSS property

The **`color-scheme`** CSS property allows an element to indicate which color schemes it can comfortably be rendered in. User agents change the following aspects of the UI chrome to match the used color scheme:

- The color of the canvas surface.
- The default colors of scrollbars and other interaction UI.
- The default colors of form controls.
- The default colors of other browser-provided UI, such as "spellcheck" underlines.

Component authors must use the `prefers-color-scheme` media feature to support the color schemes on the rest of the elements.

Common choices for operating system color schemes are "light" and "dark", or "day mode" and "night mode". When a user selects one of these color schemes, the operating system makes adjustments to the user interface. This includes form controls, scrollbars, and the used values of CSS system colors.

Try it

```
color-scheme: normal;
```

```
color-scheme: dark;
```

```
color-scheme: light;
```

```
<section class="default-example container" id="default-example">
  <textarea id="example-element">Text Area</textarea>
</section>
```

```
#example-element {
  width: 80%;
  height: 50%;
}
```

Syntax

```
color-scheme: normal;
color-scheme: light;
color-scheme: dark;
color-scheme: light dark;
color-scheme: only light;

/* Global values */
color-scheme: inherit;
color-scheme: initial;
color-scheme: revert;
color-scheme: revert-layer;
color-scheme: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

Indicates that the element can be rendered using the page's color scheme settings. If the page does not have a color scheme set, the element is rendered using the page's default color settings.

`light`

Indicates that the element can be rendered using the operating system *light* color scheme.

`dark`

Indicates that the element can be rendered using the operating system *dark* color scheme.

`only`

Forbids the user agent from overriding the color scheme for the element.

Can be used to turn off color overrides caused by Chrome's Auto Dark Theme, by applying `color-scheme: only light;` on a specific element or `:root`.

Formal definition

Initial value	`normal`
Applies to	all elements and text
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
color-scheme =
  normal                                  |
  [ light | dark | <custom-ident> ]+ && only?
```

This syntax reflects the latest standard as per CSS Color Adjustment Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Declaring color scheme preferences

To opt the entire page into the user's color scheme preferences, declare `color-scheme` on the `:root` element.

```
:root {
  color-scheme: light dark;
}
```

To opt in specific elements to the user's color scheme preferences, declare `color-scheme` on those elements.

```
header {
  color-scheme: only light;
}
main {
  color-scheme: light dark;
}
footer {
  color-scheme: only dark;
}
```

Along with the above CSS, also include the `<meta name="color-scheme">` HTML `<meta>` tag in the `<head>`, before any CSS style information, to inform user agents about the preferred color scheme, helping prevent unwanted screen flashes during the page load.

Styling based on color schemes

To style elements based on color scheme preferences, use the `prefers-color-scheme` media query. The example below opts in the entire page to using both light and dark operating system color schemes via the `color-scheme` property, and then uses `prefers-color-scheme` to specify the desired foreground and background colors for individual elements in those color schemes.

```
:root {
  color-scheme: light dark;
}

@media (prefers-color-scheme: light) {
  .element {
    color: black;
    background-color: white;
  }
}

@media (prefers-color-scheme: dark) {
  .element {
    color: white;
    background-color: black;
  }
}
```

Alternatively, use the `light-dark()` `<color>` function to set the foreground and background colors for the different color schemes using a more compact code structure:

```
:root {
  color-scheme: light dark;
}

.element {
  color: light-dark(black, white);
  background-color: light-dark(white, black);
}
```

See also

- `prefers-color-scheme` media query to detect user preferences for color schemes.
- `light-dark()` color function to set colors for both light and dark color schemes.
- Other color-related properties: `color`, `accent-color`, `background-color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`
- `background-image`
- `print-color-adjust`
- Using relative colors

column-count CSS property

The **`column-count`** CSS property breaks an element's content into the specified number of columns.

Try it

```
column-count: 2;
```

```
column-count: 3;
```

```
column-count: 4;
```

```
column-count: auto;
column-width: 8rem;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
#example-element {
  width: 100%;
  text-align: left;
}
```

Syntax

```
/* Keyword value */
column-count: auto;

/* <integer> value */
column-count: 3;

/* Global values */
column-count: inherit;
column-count: initial;
column-count: revert;
column-count: revert-layer;
column-count: unset;
```

Values

This property is specified as a single value from the following list:

`auto`

> The number of columns is determined by other CSS properties, such as `column-width`.

`<integer>`

> Is a strictly positive `<integer>` describing the ideal number of columns into which the content of the element will be flowed. If the `column-width` is also set to a non-`auto` value, it merely indicates the maximum allowed number of columns.

Description

Setting the `column-count` to a value greater than `1` creates a multi-column layout. The element's content will be evenly distributed between the specified number of columns.

The `column-count` property, along with the `column-height` and `column-width` properties, can also be set by using the `columns` shorthand.

Formal definition

Initial value	`auto`
Applies to	Block containers except table wrapper boxes
Inherited	no
Computed value	as specified
Animation type	an integer

Formal syntax

```
column-count =
  auto             |
  <integer [1,∞]>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Multi-column Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Splitting a paragraph across three columns

HTML

```
<p class="content-box">
  This is a bunch of text split into three columns using the CSS
  <code>column-count</code>
  property. The text is equally distributed over the columns.
</p>
```

CSS

```
.content-box {
  column-count: 3;
}
```

Result

See also

- `column-height`
- `column-width`
- `columns` shorthand
- `column-rule-color`, `column-rule-style`, `column-rule-width`, `column-rule` shorthand
- Learn: Multiple-column Layout (Learn Layout)
- Basic Concepts of Multicol

column-fill CSS property

The **`column-fill`** CSS property controls how an element's contents are balanced when broken into columns.

Try it

```
column-fill: auto;
```

```
column-fill: balance;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather.
  </p>
</section>
```

```
#example-element {
  width: 100%;
  height: 90%;
  columns: 3;
  text-align: left;
}
```

Syntax

```
/* Keyword values */
column-fill: auto;
column-fill: balance;

/* Global values */
column-fill: inherit;
column-fill: initial;
column-fill: revert;
column-fill: revert-layer;
column-fill: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

> Columns are filled sequentially. Content takes up only the room it needs, possibly resulting in some columns remaining empty.

`balance`

> Content is equally divided between columns. In fragmented contexts, such as paged media, only the last fragment is balanced. Therefore in paged media, only the last page would be balanced. This is the default value.

The specification defines a `balance-all` value, in which content is equally divided between columns in fragmented contexts, such as paged media. This value is not yet supported in any browser.

Formal definition

Initial value	`balance`
Applies to	multicol elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
column-fill =
  auto         |
  balance      |
  balance-all
```

This syntax reflects the latest standard as per CSS Multi-column Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Balancing column content

HTML

```
<p class="fill-auto">
  This paragraph fills columns one at a time. Since all of the text can fit in
  the first column, the others are empty.
</p>

<p class="fill-balance">
  This paragraph attempts to balance the amount of content in each column.
</p>
```

CSS

```
p {
  height: 7em;
  background: #ffff99;
  columns: 3;
  column-rule: 1px solid;
}

p.fill-auto {
  column-fill: auto;
}

p.fill-balance {
  column-fill: balance;
}
```

Result

> **Warning:** There are some interoperability issues and bugs with `column-fill` across browsers, due to unresolved issues in the specification.
>
> In particular, when using `column-fill: auto` to fill columns sequentially, Chrome will only consult this property if the multicol container has a size in the block dimension (e.g., height in a horizontal writing mode). Firefox will always consult this property, therefore filling the first column with all of the content in cases where there is no size.

See also

- Learn: Multiple-column Layout
- `column-count`
- `column-width`

column-gap CSS property

The **`column-gap`** CSS property sets the size of the gap (gutter) between an element's columns in multi-column, flexible box, and grid layouts.

Try it

```
column-gap: 0;
```

```
column-gap: 10%;
```

```
column-gap: 1em;
```

```
column-gap: 20px;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  width: 200px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Keyword value */
column-gap: normal;

/* <length-percentage> value */
column-gap: 3px;
column-gap: 2.5em;
column-gap: 3%;
column-gap: calc(3% - 6px);

/* <line-width> keyword value */
```

```
column-gap: thin;
column-gap: medium;
column-gap: thick;

/* Global values */
column-gap: inherit;
column-gap: initial;
column-gap: revert;
column-gap: revert-layer;
column-gap: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

For multi-column layout, sets the gap to `1em`; otherwise `0`. This is the default value.

`<line-width>`

Sets the size of the gap using the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value.

`<length-percentage>`

Sets a non-negative `<length>` or `<percentage>` value. Percentages are relative to the inline-size of the content box.

Description

The `column-gap` property sets the size of the gap between an element's columns. The property specifies a fixed-length gutter between items in a container, separating boxes in the container's inline axis. Negative values are invalid. Column gaps in multi-col containers are `1em` wide by default, while no gap is added between grid columns or flex items or rows. See Defining CSS gaps for more information about gaps by layout type.

Percentages are calculated against the content box size of the container element's inline axis when this size is definite, against `0` otherwise, except in grid layout, for which cyclic percentage sizes resolve against zero for determining intrinsic size contributions but resolve against the element's content box when laying out the contents. See Specifying gap values as percentages for more information.

The column gap may contain a visible separator as a gap decoration. To draw a rule between the columns, use the `column-rule` or the `rule` shorthand. The rule appears in the middle of the gap but has no effect on the size of the gap between the columns.

The `column-gap` property, along with the `row-gap` property, can also be set by using the `gap` shorthand, which sets both `row-gap` and `column-gap` in one declaration, in that order.

The `column-gap` property replaced the `grid-column-gap` property, which was initially defined in CSS grid layout for creating gaps between grid columns. `grid-column-gap` is now an alias for `column-gap`.

Formal definition

Initial value	`normal`
Applies to	multi-column elements, flex containers, grid containers
Inherited	no
Percentages	refer to corresponding dimension of the content area
Computed value	as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements
Animation type	a length, percentage or calc();

Formal syntax

```
column-gap =
  normal                    |
  <length-percentage [0,∞]> |
  <line-width>

<length-percentage> =
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Flex layout

This example demonstrates using the `column-gap` property to create horizontal space between adjacent flex items.

HTML

We include six items in a container element:

```
<div class="flexbox">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

We set the `display` property to `flex` and the `flex-flow` property to `row wrap` to create a flex container with rows of items. This allows the items to flow onto new lines if needed. The flex items are each given a flex basis of either `200px` or `300px`.

We define a column rule with the `column-rule` property. The `column-gap` value is set to `20px` on the flex container, creating a `20px` gap between the adjacent flex items in each row. The column rule is drawn in the middle of the gap.

```
.flexbox {
  display: flex;
  flex-flow: row wrap;
  column-rule: 1px solid magenta;

  column-gap: 20px;
}

.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  height: 35px;
  flex: 200px;
}
div:nth-of-type(3n) {
  flex: 300px;
}
```

Result

To set vertical space between flex rows, specify a non-zero value for the `row-gap` property, optionally setting both `row-gap` and `column-gap` by using the `gap` shorthand.

Grid layout

This example demonstrates using the `column-gap` property with a `<percentage>` value in a grid layout. It also demonstrates how the `column-gap` size is not affected by the size of the column rule.

HTML

We include seven items in a container element:

```
<div id="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

We set the `display` to `grid`, the `width` to `400px` and the `grid-template-columns` to `repeat(3, 1fr)` to create a 400px-wide grid container with three columns and as many rows as needed. Each row is `100px` tall, as defined by the `grid-auto-rows` property. Every odd grid item has a `lime` background color and the even grid items are semi-opaque.

We've also set a very wide, semi-opaque `column-rule` to demonstrate how the rule is painted behind the content, with the width of the rule having no impact on the size of the gap. The `column-gap` is set to `5%`, which creates a gap that is `20px` wide.

```
#grid {
  display: grid;
  width: 400px;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
  column-rule: 30px solid #ff00ff33;

  column-gap: 5%;
}

#grid > div {
  outline: 1px solid green;
  background-color: lime;
}
#grid > div:nth-of-type(even) {
  background-color: #00ff0033;
}
```

```
@layer no-support {
  @supports not (column-gap: 5%) {
    body::before {
      content: "Your browser doesn't support percent values";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

The column rule is wider than the column gap, and is visible only where the items drawn on top are semi-transparent.

Multi-column layout

This example demonstrates using the `column-gap` property with a `<line-width>` keyword value in a multi-col layout.

HTML

```
<p class="content-box">
  This is some multi-column text with a thin column gap created with the CSS
  `column-gap` property. The `normal` default value for the `column-gap`
  property in multi-col layout is 1em.
</p>
```

CSS

```
.content-box {
  column-count: 3;
  column-gap: thin;
}
```

```
@layer no-support {
  @supports not (column-gap: thick) {
    body::before {
      content: "Your browser doesn't support the <line-width> keyword values";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

See also

- `row-gap`
- `gap`
- `column-rule`
- `rule`
- Defining CSS gaps
- Basic concepts of grid layout: gutters
- Styling Columns
- CSS gaps module

column-height CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-height`** CSS property specifies the height of the columns in a CSS multi-column layout.

Syntax

```
/* Keyword value */
column-height: auto;

/* <length> value */
column-height: 300px;
column-height: 25em;
column-height: 70vh;

/* Global values */
column-height: inherit;
column-height: initial;
column-height: revert;
column-height: revert-layer;
column-height: unset;
```

Values

This property is specified as a single value from the following list:

`auto`

> The initial value. If the content container has a set height, the content columns will grow to that height, overflowing to the side if the content doesn't fit inside the container. If the content container does not have a set height, the content will be equally distributed between the columns generated inside the container.

`<length>`

> The height of the columns. Must be non-negative.

Description

The `column-height` property sets the height of the columns in a multi-column layout. This is useful for constraining the column height for readability when setting multiple columns using the `column-count` or `column-width` property.

Without `column-height`, if the height of the multi-col content exceeds the viewport height, readers will need to scroll down to the end of a column and then back up to the top of the next column. One possible fix would be to set a fixed height on the content container, however, excess columns will then overflow to the side, and readers will have to scroll in the inline direction to read all the content.

The `column-height` property, along with `column-wrap`, allows you to set a specific height for the columns and wrap them onto a new row of columns when the container edge is reached.

The default value of `column-wrap` is `auto`, which resolves to `wrap` when `column-height` is set to a `<length>` value; `wrap` allows the fixed-height columns to wrap onto multiple rows. When `column-height` is equal to `auto`, `column-wrap: auto` resolves to `nowrap`, allowing the columns to overflow horizontally if a fixed container height is set. As a result of this default behavior, generally you don't need to explicitly set the `column-wrap` property.

The `column-height` property, along with the `column-count` and `column-width` properties, can also be set by using the `columns` shorthand.

Formal definition

Initial value	`auto`
Applies to	Block containers except table wrapper boxes
Inherited	no
Computed value	`auto` if specified as `auto`, otherwise for `<length>` the absolute value specified
Animation type	by computed value type

Formal syntax

```
column-height =
  auto            |
  <length [0,∞]>
```

This syntax reflects the latest standard as per CSS Multi-column Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates basic usage of the `column-height` property to create a wrapped multi-col layout.

HTML

We include a poem by Dr. Seuss using an `<ol>` containing 28 `<li>`s, followed by the author's name in a `<p>`.

```
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  ...
</ol>
<p>-- Dr. Seuss</p>
```

```
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  <li>Black fish</li>
  <li>Blue fish</li>
  <li>Old fish</li>
  <li>New fish.</li>
  <li>This one has a little star.</li>
  <li>This one has a little car.</li>
  <li>Say! What a lot</li>
  <li>Of fish there are.</li>
  <li>Yes. Some are red. And some are blue.</li>
  <li>Some are old. And some are new.</li>
  <li>Some are sad.</li>
  <li>And some are glad.</li>
  <li>And some are very, very bad.</li>
  <li>Why are they</li>
  <li>Sad and glad and bad?</li>
  <li>I do not know.</li>
  <li>Go ask your dad.</li>
  <li>Some are thin.</li>
  <li>And some are fat.</li>
  <li>The fat one has</li>
  <li>A yellow hat.</li>
  <li>From there to here, from here to there,</li>
  <li>Funny things</li>
  <li>Are everywhere.</li>
</ol>
<p>--Dr. Seuss</p>
```

CSS

We define the `<ol>` to be a multi-column container by setting the `column-width` property to `150px`, meaning the container will contain as many columns as possible with each being at least `150px` wide. The `gap` property of `2em` sets a horizontal gap between columns and a vertical gap between rows of columns. We then set the `column-height` to `2em`, causing the `column-wrap` property's default `auto` value to resolve to `wrap` to create wrapped rows of columns.

```
ol {
  column-width: 150px;
  gap: 2em;
  column-height: 3em;
}
```

Result

Scroll-snapped columns

This example combines a wrapped multi-column layout with CSS scroll snapping, creating a usable experience where each scrolling action snaps a new row of columns neatly inside the full height of the viewport for comfortable reading.

HTML

The HTML, which contains multiple paragraphs of content from the MDN HTML, CSS, and JavaScript home pages, has been hidden for brevity.

```
<h1>HTML, CSS, and JavaScript summaries</h1>
<p>
  <strong>HTML</strong> (HyperText Markup Language) is the most basic building
  block of the Web. It defines the meaning and structure of web content. Other
  technologies besides HTML are generally used to describe a web page's
  appearance (CSS) or behavior (JavaScript).
</p>
<p>
  "Hypertext" refers to text containing links that connect web pages to one
  another, either within a single website or between websites. Links are a
  fundamental aspect of the Web. By uploading content to the Internet and
  linking it to pages created by other people, you become an active participant
  in the World Wide Web.
</p>
<p>
  HTML uses "markup" to annotate text, images, and other content for display in
  a Web browser. HTML markup includes special "elements" such as
  <a href="/en-US/docs/Web/HTML/Reference/Elements/head"
    ><code>&lt;head&gt;</code></a
```

```
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/title"
    ><code>&lt;title&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/body"
    ><code>&lt;body&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/header"
    ><code>&lt;header&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/footer"
    ><code>&lt;footer&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/article"
    ><code>&lt;article&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/section"
    ><code>&lt;section&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/p"><code>&lt;p&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/div"
    ><code>&lt;div&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/span"
    ><code>&lt;span&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/img"
    ><code>&lt;img&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/aside"
    ><code>&lt;aside&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/audio"
    ><code>&lt;audio&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/canvas"
    ><code>&lt;canvas&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/datalist"
    ><code>&lt;datalist&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/details"
    ><code>&lt;details&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/embed"
    ><code>&lt;embed&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/nav"
    ><code>&lt;nav&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/search"
    ><code>&lt;search&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/output"
    ><code>&lt;output&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/progress"
    ><code>&lt;progress&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/video"
    ><code>&lt;video&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/ul"
    ><code>&lt;ul&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/ol"
    ><code>&lt;ol&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/li"
    ><code>&lt;li&gt;</code></a
  >
  and many others.
</p>
<p>
  An HTML element is set off from other text in a document by "tags", which
  consist of the element name surrounded by <code>&lt;</code> and
  <code>&gt;</code>. The name of an element inside a tag is case-insensitive.
  That is, it can be written in uppercase, lowercase, or a mixture. For example,
  the <code>&lt;title&gt;</code> tag can be written as
  <code>&lt;Title&gt;</code>, <code>&lt;TITLE&gt;</code>, or in any other way.
  However, the convention and recommended practice is to write tags in
  lowercase.
</p>

<hr />

<p>
  <strong>Cascading Style Sheets</strong> (<strong>CSS</strong>) is a
  <a href="/en-US/docs/Web/API/StyleSheet">stylesheet</a> language used to
  describe the presentation of a document written in
  <a href="/en-US/docs/Web/HTML">HTML</a> or
  <a href="/en-US/docs/Web/XML/Guides/XML_introduction">XML</a> (including XML
  dialects such as <a href="/en-US/docs/Web/SVG">SVG</a>,
  <a href="/en-US/docs/Web/MathML">MathML</a> or
  <a href="/en-US/docs/Glossary/XHTML">XHTML</a>). CSS describes how elements
  should be rendered on screen, on paper, in speech, or on other media.
</p>
<p>
  CSS is among the core languages of the <strong>open web</strong> and is
  standardized across web browsers according to
  <a href="https://www.w3.org/Style/CSS/#specs" class="external" target="_blank"
    >W3C specifications</a
```

```
  >. Previously, the development of various parts of CSS specification was done
  synchronously, which allowed the versioning of the latest recommendations. You
  might have heard about CSS1, CSS2.1, or even CSS3. There will never be a CSS3
  or a CSS4; rather, everything is now just "CSS" with individual CSS modules
  having version numbers.
</p>
<p>
  After CSS 2.1, the scope of the specification increased significantly and the
  progress on different CSS modules started to differ so much, that it became
  more effective to
  <a
    href="https://www.w3.org/Style/CSS/current-work"
    class="external"
    target="_blank"
    >develop and release recommendations separately per module</a
  >. Instead of versioning the CSS specification, W3C now periodically takes a
  snapshot of
  <a href="https://www.w3.org/TR/css/" class="external" target="_blank"
    >the latest stable state of the CSS specification</a
  >
  and individual modules progress. CSS modules now have version numbers, or
  levels, such as
  <a
    href="https://drafts.csswg.org/css-color-5/"
    class="external"
    target="_blank"
    >CSS Color Module Level 5</a
  >.
</p>

<hr />

<p>
  <strong>JavaScript</strong> (<strong>JS</strong>) is a lightweight interpreted
  (or
  <a href="/en-US/docs/Glossary/Just_In_Time_Compilation"
    >just-in-time compiled</a
  >) programming language with
  <a href="/en-US/docs/Glossary/First-class_Function">first-class functions</a>.
  While it is most well-known as the scripting language for Web pages,
  <a
    href="https://en.wikipedia.org/wiki/JavaScript#Other_usage"
    class="external"
    target="_blank"
    >many non-browser environments</a
  >
  also use it, such as <a href="/en-US/docs/Glossary/Node.js">Node.js</a>,
  <a href="https://couchdb.apache.org/" class="external" target="_blank"
    >Apache CouchDB</a
  >
  and
  <a
    href="https://opensource.adobe.com/dc-acrobat-sdk-docs/acrobatsdk/"
    class="external"
    target="_blank"
    >Adobe Acrobat</a
  >. JavaScript is a
  <a href="/en-US/docs/Glossary/Prototype-based_programming">prototype-based</a
  >, <a href="/en-US/docs/Glossary/Garbage_collection">garbage-collected</a>,
  <a href="/en-US/docs/Glossary/Dynamic_typing">dynamic</a> language, supporting
  multiple paradigms such as imperative, functional, and object-oriented.
</p>
<p>
  JavaScript's dynamic capabilities include runtime object construction,
  variable parameter lists, function variables, dynamic script creation (via
  <a href="/en-US/docs/Web/JavaScript/Reference/Global_Objects/eval"
    ><code>eval</code></a
  >), object introspection (via
  <a href="/en-US/docs/Web/JavaScript/Reference/Statements/for...in"
    ><code>for...in</code></a
  >
  and
  <a
    href="/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object#static_methods"
    ><code>Object</code> utilities</a
  >), and source-code recovery (JavaScript functions store their source text and
  can be retrieved through
  <a
    href="/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/toString"
    ><code>toString()</code></a
  >).
</p>
<p>
  This section is dedicated to the JavaScript language itself, and not the parts
  that are specific to Web pages or other host environments. For information
  about <a href="/en-US/docs/Glossary/API">APIs</a> that are specific to Web
  pages, please see <a href="/en-US/docs/Web/API">Web APIs</a> and
  <a href="/en-US/docs/Glossary/DOM">DOM</a>.
</p>
<p>
  The standards for JavaScript are the
  <a href="https://tc39.es/ecma262/" class="external" target="_blank"
    >ECMAScript Language Specification</a
  >
  (ECMA-262) and the
  <a href="https://tc39.es/ecma402/" class="external" target="_blank"
    >ECMAScript Internationalization API specification</a
  >
  (ECMA-402). As soon as one browser implements a feature, we try to document
  it. This means that cases where some
  <a href="https://github.com/tc39/proposals" class="external" target="_blank"
    >proposals for new ECMAScript features</a
  >
```

```
  have already been implemented in browsers, documentation and examples in MDN
  articles may use some of those new features. Most of the time, this happens
  between the
  <a href="https://tc39.es/process-document/" class="external" target="_blank"
    >stages</a
  >
  3 and 4, and is usually before the spec is officially published.
</p>
<p>
  Do not confuse JavaScript with the
  <a
    href="https://en.wikipedia.org/wiki/Java_(programming_language)"
    class="external"
    target="_blank"
    >Java programming language</a
  >
  — <strong>JavaScript is <em>not</em> "Interpreted Java"</strong>. Both "Java"
  and "JavaScript" are trademarks or registered trademarks of Oracle in the U.S.
  and other countries. However, the two programming languages have very
  different syntax, semantics, and use.
</p>
<p>
  JavaScript documentation of core language features (pure
  <a
    href="/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview"
    >ECMAScript</a
  >, for the most part) includes the following:
</p>
```

CSS

We start by setting `column-width` on the `<body>` element to define the preferred width for the columns. A `gap` of `3em 2em` results in a `3em` gap between rows and a `2em` gap between columns. The `column-rule` adds a line in the center of the gap between the columns. The `column-height` of `95vh` makes the columns nearly as tall as the viewport.

We explicitly set `column-wrap` to `wrap` as a reminder of the applied wrapping behavior. We could have set the value to `auto` or omitted the property altogether, as, by default, `column-wrap` resolves to `wrap` when the `column-height` is set to a `<length>` value.

```
body {
  column-width: 150px;
  column-rule: 2px solid red;
  gap: 3em 2em;
  padding: 0 2em;
  column-height: 95vh;
  column-wrap: wrap;
}
```

Next, we set the `<h1>` element's `column-span` property to `all`, to make the heading span across all the columns, and set the `margin-top` property of the first `<p>` to `0` so that it lines up with the top of the columns.

```
h1 {
  column-span: all;
}

p:first-of-type {
  margin-top: 0;
}
```

Finally, we add scroll snapping by setting `scroll-snap-type` to `y mandatory` on the `<html>` element, and `scroll-snap-align` to `start` on the `::column` pseudo-elements that represent each generated column. This causes the content to snap to the top of a new column each time it is scrolled.

```
html {
  scroll-snap-type: y mandatory;
}

::column {
  scroll-snap-align: start;
}
```

```
* {
  box-sizing: border-box;
}

html {
  font-family: "Arial", sans-serif;
}

p {
  line-height: 1.5;
}
```

```
@supports not (column-height: 15em) {
  body::before {
    content: "Your browser does not support the 'column-height' property.";
    background-color: wheat;
    position: fixed;
    inset: 40% 0;
    height: fit-content;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Try scrolling the content. Note how each new row of columns fills the viewport, and how the content snaps neatly to the top of a new row with each scroll.

<code>column-height</code> and <code>column-count</code> playground

This example builds upon the previous one by including two range sliders that allow you to adjust the multiple-column layout's column count and column height.

HTML and JavaScript

The HTML is the same as the previous example, with the addition of a form containing two `<input="range">` elements that update the `column-count` and `column-height` values via JavaScript. The HTML and JavaScript are hidden for brevity.

```
<form>
  <div>
    <label for="columns">
      <code>column-count <output id="columns-output">3</output></code>
    </label>
    <input type="range" min="3" max="6" value="3" id="columns" />
  </div>
  <div>
    <label for="column-height">
      <code>column-height <output id="column-height-output">20em</output></code>
    </label>
    <input type="range" min="10" max="30" value="20" id="column-height" />
  </div>
</form>
```

```
const columnsRange = document.getElementById("columns");
const columnsOutput = document.getElementById("columns-output");
const columnHeightRange = document.getElementById("column-height");
const columnHeightOutput = document.getElementById("column-height-output");

columnsRange.addEventListener("input", () => {
  document.body.style.columnCount = columnsRange.value;
  columnsOutput.textContent = columnsRange.value;
});

columnHeightRange.addEventListener("input", () => {
  document.body.style.columnHeight = `${columnHeightRange.value}em`;
  columnHeightOutput.textContent = `${columnHeightRange.value}em`;
});
```

CSS

We specify the `column-rule` and `gap` with the same values as the previous example. We don't specify a `column-width`; instead, we create a multi-column layout using the `column-count` property, interactively setting the number of columns and the height of the column rows using JavaScript. Scroll snapping is not included in this example.

```
body {
  column-count: 3;
  column-height: 20em;
  column-rule: 2px solid red;
  gap: 3em 2em;
  padding: 0 2em;
}
```

```
form {
  border: 1px solid black;
  background-color: white;
  padding: 10px;
  position: fixed;
  bottom: 1px;
  right: 1px;
}

form div {
  display: flex;
  gap: 1em;
  align-items: center;
  justify-content: space-between;
}
```

Result

Adjust the number of columns and the column height to see the effect of these properties.

See also

- `column-count`
- `column-width`
- `columns` shorthand
- `column-wrap`
- CSS multi-column layout module

column-rule CSS property

The **`column-rule`** CSS shorthand property sets the width, style, and color of the lines drawn between columns in multi-column grid, flex, and multi-col layouts.

Try it

```
column-rule: solid;
```

```
column-rule: groove 0.8em teal;
```

```
column-rule:
  dotted thick teal,
  repeat(3, dashed pink 1px, outset olive 5px);
```

```
column-rule:
  dotted thick teal,
  repeat(auto, dashed pink 1px, dashed pink 5px),
  dotted thick teal;
```

```
column-rule:
  dashed medium olive,
```

```
  repeat(auto, dotted pink 1px),
  inset orange 5px;
```

```
<section id="default-example">
  <p id="example-element">
    London. Lady Catnip sitting in Lincoln's Inn Hall. Nice May weather. As much
    mud in the streets as if the waters had but newly retired from the face of
    the earth, and it would not be great to meet a Fred, two feet long or so,
    waddling like an iguana up Holborn Hill.
  </p>
</section>
```

```
#example-element {
  columns: 7;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-color`
- `column-rule-style`
- `column-rule-width`

Syntax

```
/* One value */
column-rule: dashed;
column-rule: inset 8px;
column-rule: solid teal;
column-rule: thick outset rgb(18 122 67);

/* Multiple values */
column-rule: groove, dashed, solid;
column-rule:
  dotted medium teal,
  dashed pink 0.5em,
  outset olive 1px;
column-rule:
  solid #0ff,
  repeat(3, dashed pink 1px, outset olive 5px);
column-rule:
  inset 3px yellow,
  repeat(auto, dashed pink 1px, groove olive 5px);

/* Global values */
column-rule: inherit;
column-rule: initial;
column-rule: revert;
column-rule: revert-layer;
column-rule: unset;
```

Values

This property is specified as a comma-separated list of values. Each value can be one of the following value types:

`<gap-rule>`

Specified as one, two, or three of the values listed below, in any order.

`<'line-width'>`

A `<line-width>`: This can be one of the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value, representing the width of the line. The default value is `medium`.

`<'line-style'>`

A `<line-style>`: one of `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`. The default value is `none`. See `column-rule-style`.

`<'color'>`

A `<color>` value representing the color of the line. The default value is `currentcolor`. See `column-rule-color`.

`<gap-repeat-rule>`

A `repeat()` function, with an `<integer>` of `1` or more as the first argument and one or more `<gap-rule>` values as subsequent arguments. The `<integer>` specifies how many times the list of `<gap-rule>` values should be repeated.

`<gap-auto-repeat-rule>`

A `repeat()` function, with `auto` as the first argument and one or more `<gap-rule>` values as subsequent arguments. The provided list of `<gap-rule>` values is repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

Description

The `column-rule` property defines the line style of any rule lines drawn in the gaps between columns in multi-column, flex, and grid containers with more than one column.

The `column-rule` is shorthand for `column-rule-color`, `column-rule-style`, and `column-rule-width`. The `column-rule`, along with the `row-rule` shorthand property, can also be set using the `rule` shorthand.

The property value is a comma-separated list of components, which can include `<gap-rule>`, `<gap-repeat-rule>`, and `<gap-auto-repeat-rule>` types. Each `<gap-rule>` defines the width, color, and style of one or more rule lines.

If the property value consists of only one `<gap-rule>`, all the column rules will be that style. If we declare the following, all column rules will be `dashed maroon 3px`:

```
column-rule: dashed maroon 3px;
```

When more than one `<gap-rule>` is declared, they will be applied to column-rules in the order specified. If there are more gutters between columns than `<gap-rule>` values, the list of values is repeated until every column rule has a gap line. If we declare the following, for example, every odd rule will be `dashed maroon 3px`, and every even rule will be `dotted navy 5px`.

```
column-rule:
  dashed maroon 3px,
```

```
  dotted navy 5px;
```

Repeated line styles

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<gap-rule>` values passed as subsequent arguments the specified number of times. This allows the same `<gap-rule>` to be repeated a set number of times without repeating the same CSS multiple times. The following declarations are equivalent:

```
column-rule:
  solid maroon 5px,
  outset navy 10px,
  inset olive 1px,
  outset navy 10px,
  inset olive 1px,
  outset navy 10px,
  inset olive 1px;
column-rule:
  solid maroon 5px,
  repeat(3, outset navy 10px, inset olive 1px);
```

This creates a list of seven rules. If the number of styles in the `column-rule` value's style list exceeds the number of gaps between columns, the excess style values are ignored. If the container on which this is applied has three columns, the rule in the first gutter will be `solid maroon 5px` and the second `outset navy 10px`.

If there are more gutters than styles, the list of styles is repeated. If the container has 8, 15, 22, or 29 columns, this sequence of styles will be repeated one, two, three, or four times, respectively, with the last rule being `inset olive 1px`.

Auto-repeating line styles

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<gap-rule>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

```
column-rule:
  solid maroon 5px,
  repeat(auto, dotted olive 1px, dashed navy 1px),
  solid maroon 5px;
```

In this case, the first and last column rules will be `solid maroon 5px`, and all others will alternate between `dotted olive 1px` and `dashed navy 1px`. It doesn't matter if the container has 3, 6, 11, 16, or 21 columns; the first and last gutters will always have a thick solid maroon line painted between them (unless `column-rule-visibility-items` leads to no line being drawn), and all the other column rules will be thin, dotted olive or dashed navy lines. If there are only 2 or 3 columns, there will be no dotted or dashed lines.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for column rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. At most, only one `repeat(auto, <gap-rule>)` can be present in a `column-rule` value.

Formal definition

Initial value	as each of the properties of the shorthand: • `column-rule-width`: `medium` • `column-rule-style`: `none` • `column-rule-color`: `currentcolor`
Applies to	multicol elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `column-rule-width`: the absolute `<length>`, snapped as a line width • `column-rule-style`: as specified • `column-rule-color`: computed color
Animation type	as each of the properties of the shorthand: • `column-rule-width`: a length • `column-rule-style`: discrete • `column-rule-color`: a color

Formal syntax

```
column-rule =
  <gap-rule-list>      |
  <gap-auto-rule-list>

<gap-rule-list> =
  <gap-rule-or-repeat>#

<gap-auto-rule-list> =
  <gap-rule-or-repeat>#? , <gap-auto-repeat-rule> , <gap-rule-or-repeat>#?

<gap-rule-or-repeat> =
  <gap-rule>         |
  <gap-repeat-rule>

<gap-auto-repeat-rule> =
  repeat( auto , <gap-rule># )

<gap-rule> =
  <line-width>  ||
  <line-style>  ||
  <color>

<gap-repeat-rule> =
  repeat( <integer [1,∞]> , <gap-rule># )

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

```
<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single rule for the lines drawn between flex items.

HTML

We include a list of authors:

```
<ul>
  <li>Kimberlé Crenshaw</li>
  <li>Angela Y. Davis</li>
  <li>Bernardine Evaristo</li>
  <li>Audre Lorde</li>
  <li>Cathy Park Hong</li>
  <li>Zoya Patel</li>
  <li>Juno Mac</li>
  <li>Molly Smith</li>
  <li>Tara Westover</li>
</ul>
```

CSS

We define the list to be a flex container, creating columns by setting the `flex-direction` to `row` using the `flex-flow` shorthand. We include a `gap` of `12px` to provide enough room between the columns to fit our `10px groove maroon` rule:

```
ul {
  display: flex;
  flex-flow: row;
  gap: 12px;
  list-style-type: none;

  column-rule: 10px groove maroon;
}
```

Result

Repeating values

This example demonstrates how the values are repeated when there are fewer values in the list of styles than column rules. It also demonstrates the default values for the width, color, and style of `medium`, `currentcolor` and `none`, respectively.

Using the same HTML and CSS as in the previous example, we include four comma-separated `<gap-rule>` values as the `column-rule` value, omitting the width in the first `<gap-rule>`, the color in the second, and the style from the third, with the fourth including all three components:

```
ul {
  column-rule:
    maroon dashed,
    1px dotted,
    5px teal,
    10px orange solid;
}
```

The maroon line is `3px` wide. The dotted line is the same color as the text. There are no teal lines, as the `<line-style>` of the third `<gap-rule>` defaults to `none`, so no line is painted. There are more gutters than `<gap-rule>` values, so the list of values gets repeated.

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `column-rule` property value. We use the same HTML and CSS as in the previous examples. We include a `repeat()` function, setting the list of two `<gap-rule>` values to repeat four times.

```
ul {
  column-rule:
    10px maroon dashed,
    repeat(4, dotted olive 3px, dashed teal 3px),
    10px maroon dashed;
}
```

The flex container has nine columns, so eight gutters. The `repeat()` function repeats two style values four times, creating a list of ten `<gap-rule>` values. As there are fewer column gutters than total `<gap-rule>` values, the last two values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using the `auto` argument instead of an integer in the `repeat()` function.

Using `repeat(auto, <gap-rule>)`, we set all column rules to `1px dotted` and default to the current color, except the first and last, which we set to `10px groove maroon`.

```
ul {
  column-rule:
    10px groove maroon,
```

```
    repeat(auto, 3px dotted maroon),
    10px groove maroon;
}
```

```
@layer no-support {
  @supports not (column-rule: thin, thick) {
    body::before {
      content: "Your browser doesn't support multiple values for the column-rule property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `column-rule-color`
- `column-rule-width`
- `column-rule-style`
- `row-rule` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

column-rule-break CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-break`** CSS property sets the behavior for breaking column rules into segments wherever column rules intersect row gaps.

Try it

```
column-rule-break: none;
```

```
column-rule-break: normal;
```

```
column-rule-break: intersection;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  column-rule: solid thick orange;
  row-rule: solid thick lavender;
  gap: 15px;
  rule-overlap: column-over-row;
}
#example-element i {
  padding: 5px;
}
```

Syntax

```
/* Keyword values */
column-rule-break: none;
column-rule-break: normal;
column-rule-break: intersection;

/* Global values */
column-rule-break: inherit;
column-rule-break: initial;
column-rule-break: revert;
```

```
column-rule-break: revert-layer;
column-rule-break: unset;
```

Values

This property is specified as a single keyword from the following list:

`none`

There are no breaks in column rules when they intersect row gaps; rather, a continuous column rule is painted the whole height of the container, from edge to edge.

`normal`

In grid and flex containers, behaves as `none`. In multi-col, behaves as `intersection`. This is the default value.

`intersection`

Column rules always break when they intersect row gaps, with column rule segments starting and ending at container and gap edges.

Description

The `column-rule-break` property specifies whether or not to break column rules into segments when they cross row gaps.

Column rules are painted within a column gap as one or more segments, with segments occurring between adjacent grid items in separate columns, between flex items or flex lines in flex layouts depending on the `flex-direction`, or between columns in multi-col layouts.

The `column-rule-break` property only determines if the break occurs. By default, the break between column rule segments is the height of the row gap, as each segment starts and ends at the edge of the gap (or edge of the container). If the row gap is `0`, this break may not be visible. The end positions can be controlled with the `column-rule-inset` properties.

If `column-rule-break` is set to `none`, there are no breaks. In this case, the column rule line is continuous, and any `column-rule-inset` values only affect the left and right edges of the column rule at the container's edge. When there are breaks, the `column-rule-inset` properties affect the start and end of every column rule segment.

The `column-rule-break` property, along with the `row-rule-break` property, can be set using the `rule-break` shorthand.

Whether a column rule is by default a single continuous segment or segments that break at row gaps depends on the container type.

Grid containers

In grid containers, by default, column rule segments continue through row gap intersections, equivalent to `column-rule-break: none`. Setting `column-rule-break: intersection` forces the segments to break at every row gap they would otherwise cross.

```
<h1>Default rule breaks in grid</h1>
<div class="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
<p>
  <label
    ><input type="checkbox" /> Set
    <code>column-rule-break: intersection</code></label
  >
</p>
```

```
h1,
div {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}

:has(:checked) .grid {
  column-rule-break: intersection;
}
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  rule: 5px solid blue;
  row-rule-color: lightblue;
  rule-overlap: column-over-row;
  width: 100%;
}

.grid > div {
  border: 1px solid green;
  background-color: lime;
  height: 30px;
}

@layer no-support {
  @supports not (column-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the column-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

By default, there are no column rule breaks. Check the checkbox to set the `column-rule-break` to `intersection`, which makes the otherwise continuous rules break at every "cross" intersection. By default, the break between segments is the height of the `row-gap`, which was set to `20px` in this case.

Flex containers

In flexbox, whether the column rules break at every row gap by default depends on the `flex-direction`. In horizontal writing modes, when set to `row` or `row-reverse`, the column rule breaks at every row gap, equivalent to `column-rule-break: intersection`. When the `flex-direction` is `column` or `column-reverse`, the column rule is continuous by default, equivalent to `column-rule-break: none`.

```
<h1>Default rule breaks in flexbox</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
<p>
  <label
    ><input type="checkbox" /> Set
    <code>column-rule-break: intersection</code></label
  >
</p>
```

```
h1,
article {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}

:has(:checked) .flexbox {
  column-rule-break: intersection;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox {
  display: flex;
  flex-flow: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid blue;
  row-rule-color: lightblue;
  width: 100%;
}
.column {
  flex-flow: column balance;
  gap: 20px;
}

.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}

@layer no-support {
  @supports not (column-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the column-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

In horizontal writing modes, setting the `column-rule-break` to `intersection` only impacts the column rules in the `column` and `column-reverse` scenarios.

Multi-col containers

In multi-col containers, the default `normal` value behaves the same as `intersection`. While the row decorations are continuous by default, column rules break at every intersection. Column rules break into segments at every row gap, with each segment starting and ending at the edge of the gap. These start and end positions can be changed with the `column-rule-inset` properties.

```
<h1>Default rule breaks in multi-col</h1>
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  <li>Black fish</li>
  <li>Blue fish</li>
  <li>Old fish</li>
  <li>New fish.</li>
  <li>This one has a little star.</li>
  <li>This one has a little car.</li>
  <li>Say! What a lot</li>
  <li>Of fish there are.</li>
  <li>Yes. Some are blue.</li>
  <li>And some are blue.</li>
  <li>Some are old.</li>
  <li>And some are new.</li>
  <li>Some are sad.</li>
  <li>And some are glad.</li>
  <li>And some are very, very bad.</li>
  <li>Why are they</li>
  <li>Sad and glad and bad?</li>
  <li>I do not know.</li>
  <li>Go ask your dad.</li>
</ol>
<fieldset>
  <legend>Set <code>column-rule-break:</code></legend>
  <label
    ><input type="radio" name="break" value="none" /> <code>none</code></label
  >
  <label
    ><input type="radio" name="break" value="normal" checked />
    <code>normal</code></label
  >
  <label
    ><input type="radio" name="break" value="intersection" />
    <code>intersection</code></label
  >
</fieldset>
```

```
h1,
ol,
fieldset {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
ol {
  columns: 3 / 4em;
  gap: 20px;
  rule: 5px solid blue;
  row-rule-color: lightblue;
  rule-overlap: column-over-row;
}
li {
  border: 1px solid green;
  background-color: lime;
  list-style-type: none;
  margin-bottom: 5px;
}
:has([value="normal"]:checked) ol {
  column-rule-break: normal;
}
:has([value="intersection"]:checked) ol {
  column-rule-break: intersection;
}
:has([value="none"]:checked) ol {
  column-rule-break: none;
}
label {
  margin-right: 20px;
}
@layer no-support {
  @supports not (column-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the column-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

If you select `none`, the column rule will no longer break into segments; rather, it will continue from the top of the container, until it reaches the bottom edge. The `column-rule-inset` properties can be used to offset the ends of the column gap decorations.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-break =
  none          |
  normal        |
  intersection
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we use the `column-rule-break` property to break each rule segment in a grid container, creating column rule segments between row gaps. Changing the `row-gap` property changes the segment size.

HTML

We create a list of 50 items and a slider to select the row gap width. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>49</li>
  <li>50</li>
</ul>
```

```
<p>
  <label
    >Change the width of the row gap.
    <input type="range" min="0" max="32" value="16" id="gap"
  /></label>
  <output id="o"></output>
</p>
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
</ul>
```

CSS

We define the unordered list as an 8-column container, creating rows and columns with the `grid-template-columns` property and setting `list-style-type` to `none` to remove the bullets. We include a `gap` of `20px` to provide enough room between the rows and columns to fit our `20px` solid row and column rules. We include the `rule-overlap` property to paint the column decoration on top of any row decorations. Last, we set the column rules to break at every intersection.

```
ul {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  list-style-type: none;
  gap: 20px;

  column-rule: 10px solid olive;
  row-rule: 10px solid palegoldenrod;
```

```
  rule-overlap: column-over-row;

  column-rule-break: intersection;
}
```

The rest of the CSS is hidden for brevity.

```
ol {
  place-items: center;
  width: 95vw;
}
li {
  text-align: center;
  font-family: sans-serif;
  line-height: 50px;
}
@layer no-support {
  @supports not (column-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the column-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const gap = document.getElementById("gap");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

gap.addEventListener("input", () => {
  output.innerText = ul.style.rowGap = `${gap.value}px`;
});
```

Result

Make the row gaps wider and note how the breaks between column segments grow. Bring the row gap width down to `0px`, and notice how the column decoration appears continuous. It isn't! The `0px` gap between segments may not be visible, but the segments still start and end at the gap, so any offsets set with `column-rule-inset-*` properties still apply.

See also

- `row-rule-break`
- `rule-break` shorthand
- `rule-inset` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

column-rule-color CSS property

The **`column-rule-color`** CSS property defines the colors of the lines drawn between columns in multi-column grid, flex, and multi-col layouts.

Try it

```
column-rule-color: purple;
```

```
column-rule-color: rgb(48 125 222), rgb(222 48 125);
```

```
column-rule-color: rgb(48 125 222), repeat(3, rgb(222 48 125));
```

```
column-rule-color: purple, repeat(auto, orange, yellow);
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
#example-element {
  columns: 7;
  column-rule: solid thick;
  gap: 7px;
}
```

Syntax

```
/* Single <color> value */
column-rule-color: purple;
column-rule-color: rgb(192 56 78);
column-rule-color: transparent;
column-rule-color: hsl(0 100% 50% / 60%);

/* Multiple values */
column-rule-color: purple, magenta;
column-rule-color: repeat(3, purple), repeat(3, transparent);
column-rule-color: repeat(3, purple), repeat(3, yellow, blue);
column-rule-color: purple, repeat(auto, transparent), purple;
column-rule-color: purple, repeat(auto, blue, yellow), purple;
column-rule-color:
  repeat(3, purple), repeat(auto, transparent), repeat(3, purple);
```

```
/* Global values */
column-rule-color: inherit;
column-rule-color: initial;
column-rule-color: revert;
column-rule-color: revert-layer;
column-rule-color: unset;
```

Values

The `column-rule-color` property accepts a comma-separated list of values, including:

`<line-color>`

A `<color>` representing the color of the line.

`<repeat-line-color>`

A `repeat()` function, with an `<integer>` of `1` or more as the first argument and one or more `<color>` values as subsequent arguments. The `<integer>` specifies how many times the `<color>` values should be repeated.

`<auto-repeat-line-color>`

A `repeat()` function, with `auto` as the first argument and one or more `<color>` values as subsequent arguments. The provided `<color>` values are repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

Description

The `column-rule-color` property defines the colors of any lines drawn in the gaps between columns in multi-column, flex, and grid containers with more than one column.

The value is a comma-separated list of components, which can include `<line-color>`, `<repeat-line-color>`, and `<auto-repeat-line-color>` types.

The `column-rule-color`, along with the `column-rule-width` and `column-rule-style` properties, can be set using the `column-rule` shorthand. The `column-rule-color`, along with the `row-rule-color` property, can also be set using the `rule-color` shorthand.

A `<line-color>` can be declared as any valid CSS `<color>` value. If the property value consists of only one `<color>`, all the rule lines will be that color. For example, if we declare the following, the lines in the gutters between columns will all be blue:

```
column-rule-color: blue;
```

When more than one `<line-color>` is declared, they will be applied to lines painted in the column gutters in the order specified. If there are more rules than `<line-color>` values, the list of colors is repeated until every column-rule has a color. If we declare the following, for example, every odd rule will be red, and every even rule will be yellow.

```
column-rule-color: red, yellow;
```

Repeated line colors

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<color>` values passed as subsequent arguments the specified number of times. This allows the color values to be repeated as many times as you need without having to list them individually. The following declarations are equivalent:

```
column-rule-color: blue, yellow, red, yellow, red;
column-rule-color: blue, repeat(2, yellow, red);
```

This creates a list of five colors. If the number of colors in the `column-rule-color` value's color list exceeds the number of gaps between columns, the excess color values are ignored. If the container has three columns, the rule in the first gutter will be blue and the second yellow.

Auto-repeating line colors

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<color>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

```
column-rule-color: blue, repeat(auto, yellow), red;
```

In this case, the first column-rule will be blue, the last will be red, and all others will be yellow. As long as there are at least two column-rules, the first will always be blue and the last will always be red. All the other rules will be yellow, which means if there are only 2 or 3 columns, there will be no yellow lines.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for the column-rule line colors that would not otherwise receive values from other parts of the list, preventing the list from being cycled. A `column-rule-color` value can include, at most, one `repeat(auto, <color>)`.

Formal definition

Initial value	`currentcolor`
Applies to	multicol elements
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
column-rule-color =
  <line-color-list>      |
  <auto-line-color-list>

<line-color-list> =
  <line-color-or-repeat>#

<auto-line-color-list> =
  <line-color-or-repeat>#? , <auto-repeat-line-color> , <line-color-or-repeat>#?

<line-color-or-repeat> =
  <color>               |
  <repeat-line-color>

<auto-repeat-line-color> =
  repeat( auto , [ <color> ]# )
```

```
<repeat-line-color> =
  repeat( [ <integer [1,∞]> ] , [ <color> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single color for the lines drawn between columns in a multi-column layout.

HTML

We include a paragraph of text.

```
<p>
  This is a bunch of text split into three columns. The `column-rule-color`
  property is used to change the color of the line that is drawn between
  columns. Don't you think that's wonderful?
</p>
```

CSS

We define the `<p>` element as a multi-column container. We include a `gap` of `7px` to provide padding for the `3px` dashed rule drawn between the columns:

```
p {
  column-count: 5;
  gap: 7px;
  column-rule-style: dashed;
  column-rule-width: 3px;

  column-rule-color: blue;
}
```

Result

Multiple color values

This example demonstrates declaring more than one color, and how the values are repeated when there are fewer values in the list of colors than gutters between columns.

Using the same HTML and CSS as in the previous example, we include three comma-separated colors as the `column-rule-color` value:

```
<p>
  This is a bunch of text split into three columns. The `column-rule-color`
  property is used to change the color of the line that is drawn between
  columns. Don't you think that's wonderful?
</p>
```

```
p {
  column-count: 5;
  gap: 7px;
  column-rule-style: dashed;
  column-rule-width: 3px;
}

@layer no-support {
  @supports not (column-rule-color: red, blue) {
    body::before {
      content: "Your browser doesn't support multiple values for the column-rule-color property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
p {
  column-rule-color: blue, yellow, red;
}
```

Result

There are four gutters but only three colors, so the list gets repeated, with the first and fourth lines both being blue.

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `column-rule-color` property value and how this function can help prevent complex values from becoming unwieldy.

HTML

We include a list of authors:

```
<ul>
  <li>Kimberlé Crenshaw</li>
  <li>Angela Y. Davis</li>
  <li>Bernardine Evaristo</li>
  <li>Audre Lorde</li>
  <li>Cathy Park Hong</li>
  <li>Zoya Patel</li>
  <li>Juno Mac</li>
  <li>Molly Smith</li>
  <li>Tara Westover</li>
</ul>
```

CSS

We start by defining the list as a grid container, creating columns with the `grid-template-columns` property. We include a `gap` of `7px` to provide enough room between the columns to fit our `3px` dashed rule, and remove the bullets with `list-style-type` set to `none`.

Then, to demonstrate how values can become complicated and the utility of the `repeat()` function, we declare two custom properties, which we use in three `color-mix()` color function declarations to create blue, red, and yellow colors. The yellow `color-mix()` color is within a `repeat()` function, set to repeat 3 times.

We also added a border around each grid item so you can see how the line is rule in the middle of the gutter between the columns.

```
ul {
  display: grid;
  grid-template-columns: repeat(9, 1fr);
  gap: 7px;
  list-style-type: none;
  column-rule-style: dashed;
  column-rule-width: 3px;

  --base: yellow;
  --mixin: blue;
  column-rule-color:
    color-mix(in lch decreasing hue, var(--base) 0%, var(--mixin)),
    repeat(3, color-mix(in lch decreasing hue, var(--base) 100%, var(--mixin))),
    color-mix(in lch decreasing hue, var(--base) 58%, var(--mixin));
}
li {
  border: 1px solid #dddddd;
}
```

Result

The grid has nine cells across, so eight gutters. The `repeat()` function repeats our two mixed color three times, creating a color list with seven colors. Since there are as more column gutters than list colors, the last color in the list is not used.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto`, instead of an integer, within the `repeat()` function.

We use the same HTML and CSS as in the previous examples, but override the `column-rule-color` value. Here, we use `repeat(auto, <color>)` to set all the lines to be almost transparent black (`#00000033`), except the first and last, which we set to a solid `black`.

```
ul {
  column-rule-color: black, repeat(auto, #00000033), black;
}
```

Result

```
@layer no-support {
  @supports not (column-rule-color: repeat(3, red)) {
    body::before {
      content: "Your browser doesn't support `repeat()` functions within a column-rule-color property value";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- The `<color>` data type
- `column-rule-width`
- `column-rule-style`
- `row-rule-color`
- `column-rule` shorthand
- `rule-color` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

column-rule-inset-cap CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-inset-cap`** shorthand CSS property can be used to offset the column rule segment cap endpoints at the container's start and end edges, and endpoints where the segments don't intersect other row or column segments.

Try it

```
column-rule-inset-cap: -20px;
```

```
column-rule-inset-cap: 0;
```

```
column-rule-inset-cap: 1em;
```

```
column-rule-inset-cap: 100%;
```

```
column-rule-inset-cap: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
```

```
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>

    <i id="y">Y</i>
    <i id="z">Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick rebeccapurple;
  column-rule-color: magenta;
  gap: 1em;
  rule-overlap: column-over-row;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
}
#y {
  grid-column: 4 / 5;
  grid-row: 4 / 5;
}
#z {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
#bang {
  grid-column: 6 / 7;
  grid-row: 4 / 5;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-inset-cap-end`
- `column-rule-inset-cap-start`

Syntax

```
/* Keywords */
column-rule-inset-cap: overlap-join;

/* <length-percentage> values */
column-rule-inset-cap: 0;
column-rule-inset-cap: 1em;
column-rule-inset-cap: -5px;
column-rule-inset-cap: -25%;

/* Two values */
column-rule-inset-cap: overlap-join 1em;
column-rule-inset-cap: -5px -25%;

/* Global values */
column-rule-inset-cap: inherit;
column-rule-inset-cap: initial;
column-rule-inset-cap: revert;
column-rule-inset-cap: revert-layer;
column-rule-inset-cap: unset;
```

Values

This property is specified as one or two values from the following list:

`overlap-join`

Resolves to `0`.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the crossing gap width, which is the width of the `row-gap` for segment endpoints at gap junctions and `0` for endpoints at the container's edge.

Description

The `column-rule-inset-cap` shorthand property can be used to set the `column-rule-inset-cap-end` and `column-rule-inset-cap-start` properties, insetting or outsetting both the start and end edges of cap segment endpoints in a single declaration.

If one value is specified, both properties are set to that value. If two values are specified, `-start` is set to the first value and `-end` is set to the second. The default value is `0`, which, in the case of cap endpoints, is the same as `overlap-join`. Positive values reduce (or inset) the segment size, while negative values increase (or outset) it.

Column rules are painted within a column gap as one or more segments, with segments occurring between:

- Adjacent columns in CSS grid layouts.
- Flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Columns in multi-col layouts.

Whether a column rule spans multiple rows or is broken into multiple segments is defined by the `column-rule-break` property, with interior breaks between column rule segments being the size of the `row-gap`.

Length `column-rule-inset-cap` values inset segments by the specified value; with negative length values creating an outset, causing end edge cap segments to extend beyond the container's top and bottom edges. Percentage values for end edge cap segments are relative to `0`, so have no effect.

For interior segments, percentage values are relative to the size of the `row-gap`. Setting `-50%` will extend the segment to the middle of the gap, while `-100%` will extend the segment across the entire gap.

The `column-rule-inset-cap` property is a constituent property of a few shorthand properties:

- To inset the ends of all column segments, the `column-rule-inset-cap` property, along with the `column-rule-inset-junction` property, can be set using the `column-rule-inset` shorthand.
- To inset all row and column cap endpoints, the `column-rule-inset-cap` property, along with the `row-rule-inset-cap` property, can be set using the `rule-inset-cap` shorthand.

All segment endpoints, including this property's `-junction` and `row-` equivalents, can be set using the `rule-inset` shorthand.

Understanding cap endpoints

A *cap segment endpoint* is any segment endpoint that is not a junction segment endpoint. This includes endpoints at the container's content edges, as well as endpoints at a gap junction where no other row or column segments are present.

The `column-rule-inset-cap` property can reduce or extend the top, bottom, or both ends of column segments at the top and bottom edge of the container and the segment end at any interior junction where no other segments are present.

Column cap segments are impacted by the `rule-visibility-items` properties, which define whether row- and column-rule segments are painted in gaps adjacent to empty areas. Changing the value from `auto` to `between` or `around` can create additional interior cap segments.

In the following demonstration, the top of the column segments in the top row and the bottom of the column segments in the bottom row start and end in cap endpoints, respectively. With `column-rule-inset-cap: 16px` set, all the column segment cap endpoints are inset by `16px`. Change the inset `<length>` value to better visualize which segments start or end in cap segment endpoints.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>15</li>
  <li>16</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset" data-unit="px"
  /></label>
  <output id="o">16px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-450"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: none;
  gap: 20px;
  column-rule: 10px solid olive;
  row-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: normal;
  rule-break: intersection;
  column-rule-inset-cap: 16px;

  border: 1px solid;
}
ul {
```

```
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
@layer no-support {
  @supports not (column-rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-cap property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  column-rule-inset-cap: 100%;
  column-rule-style: inset;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  o.innerText =
    ul.style.columnRuleInsetCap = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "repeat(2, solid), double";
  } else if (visibility.value === "around") {
    ul.style.columnRuleStyle = "repeat(3, solid), repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "repeat(2, inset), double, repeat(2, solid)";
  } else if (visibility.value === "around") {
    ul.style.columnRuleStyle = "repeat(3, inset), repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

Change the inset. When `0px` is set, the ends of the column rules align with the top and bottom edges of the container. This is the default. Setting `-32px` outsets the segments by `32px`, with the lines being drawn `32px` past the edge of the container. As column rules don't impact the box model, these lines have no impact on the layout of the container or the rest of the content.

Select `around` as the `rule-visibility-items` value. This value paints rules in a gap segment if at least one of the two adjacent areas is occupied by an item. The start end of the top-most segments are still cap segment endpoints. The bottom-most segments of the double line-style column rules, which appear when the `rule-visibility-items` property is set to `around` do not end in cap endpoints. Rather, the bottom segment of the last two column rules end at *junctions*: interior gaps where row rule segments are present. Therefore, the `column-rule-inset-cap` property does not affect those segment endpoints.

Select `between` as the `rule-visibility-items` value. This value only paints rules in gap segments if both adjacent areas are occupied by an item. In this example, the bottom row rule ends at the third column gap, with the last row segment being between `9` and `16`. The bottom of the third column rule, shown as a double line, ends at an interior gap. As a row rule segment is present, the column segment does not end in a cap segment endpoint, so it is not affected by the `column-rule-inset-cap` property. The last two column rules, however, end at interior gaps where no other rule segments are present, so these column segments are cap segment endpoints and therefore are affected by the `column-rule-inset-cap` property.

Understanding percentage values

What a percentage value is relative to depends on the endpoint's location. Interior endpoint percentage values are relative to the width of the gap the cap endpoint touches, so generally relative to the `row-gap`, plus any additional spacing added due to `align-content` values. Percentages at container-edge endpoints are relative to `0`. For example, `column-rule-inset-cap: 50%` resolves to half the gap junction size at an interior cap and `0` at the container edges.

This example is not broken. When `rule-visibility-items` is set to `normal`, every column cap endpoint abuts the top or bottom edge of the container, so any percentage value set will be relative to `0`.

Select `around` as the `rule-visibility-items` value. The first three columns end at the container edge, so any percentage value will resolve to `0`. The last two column rules, with the double lines, end at interior gaps where row rule segments are present, so these column segments are not cap segment endpoints.

Select `between` as the `rule-visibility-items` value. The first two column rules, with the light and dark lines, end at the container edge, so they have a `0` inset. The third column rule, with the double line, ends at an interior gap where a row rule segment is present, so this column segment is not a cap segment endpoint. The last two column rules, with the solid lines, end at interior gaps where no other rule segments are present, so the percentage offset is relative to the size of the `row-gap` width, which in this case is `20px`.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-inset-cap =
  <inset-value> <inset-value>?

<inset-value> =
  <length-percentage>  |
  overlap-join
```

```
<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `column-rule-inset-cap` to inset the column rule cap segment endpoints on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap column rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a `lightblue` `rule` to paint both row and column gaps, then overwrite the `column-rule-color`, setting darker `blue` column gap decorations. Finally, we set the `column-rule-inset-cap` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  column-rule-color: blue;

  column-rule-inset-cap: 16px;
}
```

We also set the `flex-direction` on the `.column` container, to change the main axis of the flex container, and make the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
```

```
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (column-rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-cap property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.columnRuleInsetCap = val;
  containers[1].style.columnRuleInsetCap = val;
  output.innerText = val;
});
```

Result

Change the size of the inset. Note how the column segments grow and shrink only at their segment cap ends — the ends that don't intersect other column or row segments.

See also

- `column-rule-inset-cap-end`
- `column-rule-inset-cap-start`
- `column-rule-inset` shorthand
- `row-rule-inset-cap` shorthand
- `rule-inset` shorthand
- `column-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

column-rule-inset-cap-end CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-inset-cap-end`** CSS property can be used to offset the bottom of column rule segment cap endpoints at the container's end edge, and cap endpoints where no rule segments intersect.

Try it

```
column-rule-inset-cap-end: -20px;
```

```
column-rule-inset-cap-end: 0;
```

```
column-rule-inset-cap-end: 1em;
```

```
column-rule-inset-cap-end: 100%;
```

```
column-rule-inset-cap-end: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>

    <i id="y">Y</i>
    <i id="z">Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick magenta;
  column-rule-color: rebeccapurple;
  gap: 1em;
  rule-overlap: column-over-row;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
}
#y {
  grid-column: 4 / 5;
  grid-row: 4 / 5;
}
#z {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
#bang {
  grid-column: 6 / 7;
  grid-row: 4 / 5;
}
```

Syntax

```
/* Keyword value */
column-rule-inset-cap-end: overlap-join;

/* <length-percentage> values */
column-rule-inset-cap-end: 0;
column-rule-inset-cap-end: 1em;
column-rule-inset-cap-end: -5px;
column-rule-inset-cap-end: -25%;

/* Global values */
column-rule-inset-cap-end: inherit;
column-rule-inset-cap-end: initial;
column-rule-inset-cap-end: revert;
column-rule-inset-cap-end: revert-layer;
column-rule-inset-cap-end: unset;
```

Values

This property is specified as a single value from the following list:

`overlap-join`

Resolves to `0`.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the cap endpoint, which is either the width of the `row-gap` or `0`.

Description

The `column-rule-inset-cap-end` property can be used to inset the end edge of cap segment endpoints. The default value is `0`, which is the same as `overlap-join`. Positive values reduce the segment size, while negative values increase it.

Column rules are painted within a column gap as one or more segments, with segments occurring between:

- Adjacent columns in CSS grid layouts.
- Flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Columns in multi-col layouts.

Whether a column rule spans multiple rows or is broken into multiple segments is defined by the `column-rule-break` property, with interior breaks between column rule segments being the size of the `row-gap`.

Length `column-rule-inset-cap-end` values inset segments by the specified value — for both interior and end edge cap segments. Negative length values create an outset, with end edge cap segments extending beyond the container's end edge.

Percentage values are relative to the size of the `row-gap` for interior segments. For end edge cap segments, percentage values are relative to `0`, so percentage values never cause cap segment endpoints at the container's end edge to extend beyond the container.

The `column-rule-inset-cap-end` property is a constituent property of several shorthand properties:

- To inset the start and end caps, the `column-rule-inset-cap-end` property, along with the `column-rule-inset-cap-start` property, can be set using the `column-rule-inset-cap` shorthand.
- To inset the ends of all column segments, the `column-rule-inset-cap-end` property, along with the `column-rule-inset-junction-end` property, can be set using the `column-rule-inset-end` shorthand.
- To inset all row and column cap and junction endpoints, the `column-rule-inset-end` property, along with the `row-rule-inset-end` property, can be set using the `rule-inset-end` shorthand.

All segment endpoints, including this property's `-start`, `-junction`, and `row-` equivalents, can be set using the `rule-inset` shorthand.

Understanding cap end

A *cap segment endpoint* is any segment endpoint that is not a junction segment endpoint. This includes endpoints at the container's content edges, as well as endpoints at a gap junction where no other rule or column segments are present.

The `column-rule-inset-cap-end` property controls the inset of the bottom edge of column cap segment endpoints, allowing the segments to be shrunk or extended.

Column cap segment endpoints are not affected by `column-rule-break` property value settings, which only control junction segment breaks. They are, however, affected by the `rule-visibility-items` properties, which define whether column- and row-rule segments are painted in gaps adjacent to empty areas.

Column cap segment endpoints only exist at the end edge of the container and at interior gaps where no other column or row rule segments are present. Therefore, whether segments are painted (or would otherwise be painted if the `rule` were set to a visible value) impacts which column segments are end cap segments.

In the following demonstration, the bottom of column rule segments end in cap endpoints. With `column-rule-inset-cap-end: 16px` set, all the column segments are inset by `16px`. Change the inset `<length>` value to better visualize which segments end in cap segment endpoints.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>15</li>
  <li>16</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset" data-unit="px"
  /></label>
  <output id="o">16px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-450"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: none;
  gap: 20px;
  column-rule: 10px solid olive;
  row-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: normal;
  rule-break: intersection;
  column-rule-inset-cap-end: 16px;

  border: 1px solid;
}
ul {
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}

@layer no-support {
  @supports not (column-rule-inset-cap-end: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-cap-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  column-rule-inset-cap-end: 100%;
  column-rule-style: inset;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  o.innerText =
    ul.style.columnRuleInsetCapEnd = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "repeat(2, solid), double";
  } else if (visibility.value === "around") {
    ul.style.columnRuleStyle = "repeat(3, solid), repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "repeat(2, inset), double, repeat(2, solid)";
  } else if (visibility.value === "around") {
    ul.style.columnRuleStyle = "repeat(3, inset), repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

Change the inset. If `0px` is set, the end of the column rules will align with the end of the container. This is the default. Setting `-32px` outsets the segments by `32px`, with the lines being drawn `32px` past the end edge of the container. As column rules don't impact the box model, these lines have no impact on the layout of the container or the rest of the content.

Select `around` as the `rule-visibility-items` value. This value paints rules in a gap segment if at least one of the two adjacent areas is occupied by an item. The double line-style column rules, which appear when the `rule-visibility-items` property is set to `around` do not end in a cap endpoint. The last two column rules end at interior gaps where row rule segments are present, so these column segments are not cap segment endpoints. Therefore, the `column-rule-inset-cap-end` property does not affect those segment endpoints.

Select `between` as the `rule-visibility-items` value, which paints rules in gap segments only if both adjacent areas are occupied by an item. The last row rule in the second row gutter ends at the third column gap. The third column rule ends at an interior gap where a row rule segment is present, so this column segment is not a cap segment endpoint and is not affected by the `column-rule-inset-cap-end` property. The last two column rules, however, end at interior gaps where no other rule segments are present, so these column segments are cap segment endpoints and therefore are affected by the `column-rule-inset-cap-end` property.

Understanding percentage values

The length a percentage value is relative to depends on the endpoint location. Interior endpoint percentage values are relative to the width of the gap at the cap endpoint, so relative to the `row-gap` if abutting a rule gap, and `0` at the bottom edge of the container.

In this demonstration, these endpoints are denoted by the inset, dark and light, line style. If the cap segment endpoint is at the container's edge, the percentage is relative to `0`, so it always computes to `0` (which is why only the `between` value has an effect).

Select `around` as the `rule-visibility-items` value. The first three columns end at the container edge, so any percentage value will resolve to `0`. The last two column rules end at interior gaps where row rule segments are present, so these column segments are not cap segment endpoints.

Select `between` as the `rule-visibility-items` value. The first two columns end at the container edge, so they have a `0` inset. The third column rule ends at an interior gap where a row rule segment is present, so this column segment is not a cap segment endpoint. The last two column rules end at interior gaps where no other rule segments are present, so the percentage offset is relative to the size of the `row-gap` width, which in this case is `20px`.

Setting `100%` insets the end of the last two column rule segments by `20px`. Setting `-200%` will outset these segments by `40px`, with the lines being drawn through the `20px` gap and `20px` protruding into the last row of items. Negative percentage values that compute to a length larger than the combined height of the last row and row gap will cause the last two column rules to protrude below the container's end edge.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-inset-cap-end =
  <inset-value>

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `column-rule-inset-cap-end` to inset the end edge of cap segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap column rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
```

```
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a light blue `rule` to paint both row and column gaps, then overwrite the `column-rule-color`, setting darker `blue` column gap decorations. Finally, we set the `column-rule-inset-cap-end` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  column-rule-color: blue;

  column-rule-inset-cap-end: 16px;
}
```

We also set the `flex-direction` on the `.column` container, to change the main axis of the flex container, and make the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (column-rule-inset-cap-end: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-cap-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");
```

```
inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.columnRuleInsetCapEnd = val;
  containers[1].style.columnRuleInsetCapEnd = val;
  output.innerText = val;
});
```

Result

Change the size of the inset.

See also

- `column-rule-inset` shorthand
- `rule-inset` shorthand
- `column-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

column-rule-inset-cap-start CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-inset-cap-start`** CSS property can be used to offset the top of column rule segment cap endpoints at the container's start edge, and cap endpoints where no rule segments intersect.

Try it

```
column-rule-inset-cap-start: -20px;
```

```
column-rule-inset-cap-start: 0;
```

```
column-rule-inset-cap-start: 1em;
```

```
column-rule-inset-cap-start: 100%;
```

```
column-rule-inset-cap-start: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>

    <i id="y">Y</i>
    <i id="z">Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick magenta;
  column-rule-color: rebeccapurple;
  gap: 1em;
  rule-overlap: column-over-row;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
}
#y {
  grid-column: 4 / 5;
  grid-row: 4 / 5;
}
#z {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
#bang {
  grid-column: 6 / 7;
  grid-row: 4 / 5;
}
```

Syntax

```
/* Keywords */
column-rule-inset-cap-start: overlap-join;

/* <length-percentage> values */
column-rule-inset-cap-start: 0;
column-rule-inset-cap-start: 1em;
column-rule-inset-cap-start: -5px;
column-rule-inset-cap-start: -25%;

/* Global values */
column-rule-inset-cap-start: inherit;
column-rule-inset-cap-start: initial;
column-rule-inset-cap-start: revert;
column-rule-inset-cap-start: revert-layer;
column-rule-inset-cap-start: unset;
```

Values

This property is specified as a single value from the following list:

`overlap-join`

Resolves to `0`.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the cap endpoint, which is either the width of the `row-gap` or `0`.

Description

The `column-rule-inset-cap-start` property can be used to inset the start edge of cap segment endpoints. The default value is `0`, which is the same as `overlap-join`. Positive values reduce the segment size, while negative values increase it.

Column rules are painted within a column gap as one or more segments, with segments occurring between:

- Adjacent columns in CSS grid layouts.
- Flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Columns in multi-col layouts.

Length `column-rule-inset-cap-start` values inset segments by the specified value — for both interior and end edge cap segments. Negative length values create an outset, with end edge cap segments extending beyond the container's start edge.

Percentage values for interior segments are relative to the size of the `row-gap`. For start edge cap segments, percentage values are relative to `0`, so percentage values never cause cap segment endpoints at the container's start edge to extend above the container.

The `column-rule-inset-cap-start` property is a constituent property of several shorthand properties:

- To inset both start and end caps, the `column-rule-inset-cap-start` property, along with the `column-rule-inset-cap-end` property, can be set using the `column-rule-inset-cap` shorthand.
- To inset the start of all column segment endpoints, the `column-rule-inset-cap-start` property, along with the `column-rule-inset-junction-end` property, can be set using the `column-rule-inset-end` shorthand.

All segment endpoints, including this property's `-end`, `-junction`, and `row-` equivalents, can be set using the `rule-inset` shorthand.

Understanding cap start

A *cap segment endpoint* is any segment endpoint that is not a junction segment endpoint. This includes endpoints at the container's content edges, as well as endpoints at a gap junction where no other rule segments are present.

The `column-rule-inset-cap-start` property controls the inset of the top edge of column cap segment endpoints, allowing the segments to be shrunk or extended. In other words, the property can be used to shrink or extend the top edge of column rule segments that abut the top edge of the container and the segments whose top ends at an interior gaps where no other column or row rule segments are present.

Column cap segment endpoints are not affected by the value of the `column-rule-break` property value settings, which only control junction segment breaks. They are, however, affected by the `rule-visibility-items` properties, which define whether column- and row-rule segments are painted in gaps adjacent to empty areas.

Column cap segment endpoints only exist at the edge of the container and at interior gaps where no other column or row rule segments are present, whether segments are painted (or would otherwise be painted if the `rule` were set to a visible value), impacts which column segments are cap start segments.

In the following demonstration, the top of the column rule segments with a solid line style start in a cap endpoint. With `column-rule-inset-cap-start: 16px` set, all the column cap segments at the top edge of the container are all inset by `16px`. Change the inset `<length>` value to better visualize which segments start with cap segment endpoints.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
```

```
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset" data-unit="px"
  /></label>
  <output id="o">16px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-200"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: none;
  gap: 20px;
  column-rule: 10px solid olive;
  row-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: normal;
  rule-break: intersection;
  column-rule-inset-cap-start: 16px;

  border: 1px solid;
}
ul {
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
li:nth-of-type(n + 9) {
  grid-row: 3 / 4;
}
li:nth-of-type(n + 12) {
  grid-row: 4 / 5;
}
li:nth-of-type(10) {
  grid-column: 5/6;
}
li:nth-of-type(11) {
  grid-column: 6/7;
}

@layer no-support {
  @supports not (column-rule-inset-cap-start: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-cap-start property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  column-rule-inset-cap-start: 100%;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  output.innerText =
    ul.style.columnRuleInsetCapStart = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "solid, repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

Change the inset. If `0px` is set, the start of the column rules will align with the start of the container. This is the default. Setting `-32px` outsets the segments by `32px`, with the lines being drawn `32px` past the start edge of the container. As column rules don't impact the box model, these lines have no impact on the layout of the container or the rest of the content.

Select `between` as the `rule-visibility-items` value. This value paints rules in a gap segment only if the two adjacent areas are occupied by items. Change the value of the inset while observing the three column rules with a double line style: in addition to having a cap endpoint at the start edge of the container, these rules have an additional cap endpoint. They each have column rule segments that start at interior gaps where no row rule segments are present, so these column segments are also cap segment endpoints, and are affected by the `column-rule-inset-cap-start` property.

Selecting `around` as the `rule-visibility-items` value paints rules in a gap segment as long as one adjacent area is occupied by an item. In these cases, there is a row rule segment at the top end of the endpoint segments of the interior gaps. The starts of these segments are junction (not cap) segment endpoints and are not affected by the `column-rule-inset-cap-start` property. The inset of these column segments that start at an interior gap junction can be controlled by the `column-rule-inset-junction-start` property.

Understanding percentage values

The length a percentage value is relative to depends on the endpoint location. Interior endpoint percentage values are relative to the width of the gap at the cap endpoint, so relative to the `row-gap` if abutting a rule gap, and `0` at the top edge of the container.

This example isn't broken; all the cap segments start at the container's edge, so all the insets are `0` by default.

If you select `around` as the `rule-visibility-items` value, again, none of the segments will be inset. Percentage inset values for the column segments that start at container edge all resolve to `0`. The start edge of column segments that start at interior gaps don't and have a segment drawn above start at intersections where row rule segments are present; they don't start with a cap segment endpoint, so their insets are defined by the `column-rule-inset-junction-start` property instead.

Select `between` as the `rule-visibility-items` value. The column rules that have two cap start segments have their line-style set to `double`. As before, each has a column rule segment that starts at interior gaps where no other rule segments are present. For these, the percentage offset is relative to the size of the `row-gap` width, which in this case is `20px`.

Setting `100%` insets the start of the cap segments by `20px`. Setting `-200%` will outset these segments by `40px`, with the lines being drawn through the `20px` gap, with `20px` protruding into the row above the items. If the negative value was larger than the combined heights of the first row and the row gap, the rule would protrude above the container's start edge.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-inset-cap-start =
  <inset-value>

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>     |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `column-rule-inset-cap-start` to inset the start edge of cap segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap column rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a light blue `rule` to be painted in both row and column gaps, then overwrite the `column-rule-color`, setting darker `blue` column gap decorations. Finally, we set the `column-rule-inset-cap-start` to `16px`.

```
.flexbox {
  display: flex;
```

```
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  column-rule-color: blue;

  column-rule-inset-cap-start: 16px;
}
```

We then set the `flex-direction` on the `.column` container to make its items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (column-rule-inset-cap-start: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-cap-start property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.columnRuleInsetCapStart = val;
  containers[1].style.columnRuleInsetCapStart = val;
  output.innerText = val;
});
```

Result

Change the size of the inset.

See also

- `row-rule-inset-cap-start`
- `column-rule-inset-cap-end`
- `column-rule-inset-junction-start`
- `column-rule-inset-start` shorthand
- `column-rule-inset-cap` shorthand
- `column-rule-inset` shorthand
- `rule-inset-start` shorthand
- `rule-inset-cap`
- `rule-inset` shorthand
- `column-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

column-rule-inset-junction CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-inset-junction`** CSS shorthand property can be used to offset both the top and bottom endpoints of column rule segments that are junction endpoints; that is, endpoints at gap junctions where rule segments intersect.

Try it

```
column-rule-inset-junction: 0;
```

```
column-rule-inset-junction: 10px;
```

```
column-rule-inset-junction: 0.5em -0.5em;
```

```
column-rule-inset-junction: overlap-join;
```

```
column-rule-inset-junction: overlap-join 10px;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  rule: solid thick lightpink;
  column-rule-color: magenta;
  column-rule-break: intersection;
  gap: 1.5em;
  rule-overlap: column-over-row;
  border: 1px solid rebeccapurple;
  margin: auto;
  rule-visibility-items: around;
}
#example-element i {
  background-color: #efefef;
  padding: 1em;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-inset-junction-end`
- `column-rule-inset-junction-start`

Syntax

```
/* Keywords */
column-rule-inset-junction: overlap-join;

/* <length-percentage> values */
column-rule-inset-junction: 0;
column-rule-inset-junction: 1em;
column-rule-inset-junction: -5px;
column-rule-inset-junction: -25%;

/* Two values */
column-rule-inset-junction: 0 1em;
column-rule-inset-junction: -5px -25%;
column-rule-inset-junction: overlap-join 10px;

/* Global values */
column-rule-inset-junction: inherit;
column-rule-inset-junction: initial;
column-rule-inset-junction: revert;
column-rule-inset-junction: revert-layer;
column-rule-inset-junction: unset;
```

Values

This property is specified as one or two values from the following list:

`overlap-join`

Specifies that the junction segment should extend across the row-rule, resolving to half the `row-gap` value plus half the used `row-rule-width` value.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the junction endpoint, which is the `row-gap` value.

Description

The `column-rule-inset-junction` shorthand property can be used to set the `column-rule-inset-junction-start` and `column-rule-inset-junction-end` properties, insetting or outsetting both the top and bottom endpoints of column rule segments that are junction segments.

If one value is specified, both properties are set to that value. If two values are specified, `-start` is set to the first value and `-end` is set to the second. Positive values reduce (or inset) the segment size, while negative values and the `overlap-join` keyword increase (or outset) it. The default value is `0`.

The `column-rule-inset-junction` property is a constituent property of a few shorthand properties:

- To inset both column and row segment junction endpoints, the `column-rule-inset-junction` shorthand property, along with the `row-rule-inset-junction` shorthand property, can be set using the `rule-inset-junction` shorthand.
- To inset both column cap and junction segment endpoints, the `column-rule-inset-junction` shorthand property, along with the `column-rule-inset-cap` shorthand property, can be set using the `column-rule-inset` shorthand.

All segment endpoints, including this property's `-cap` and `-row` equivalents, can be set using the `rule-inset` shorthand.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-inset-junction =
  <inset-value> <inset-value>?

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `column-rule-inset-junction` to inset the edges of junction segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting junction column rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a light blue `rule` to paint both row and column gaps, then overwrite the `column-rule-color`, setting darker `blue` column gap decorations. We also set the `rule-overlap` property to `column-over-row` to ensure the column segments are drawn on top of the row segments when the segments overlap. Finally, we set the `column-rule-inset-junction` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  column-rule-color: blue;
  rule-overlap: column-over-row;

  column-rule-inset-junction: 16px;
}
```

We also set the `flex-direction` on the `.column` container to `column`, changing the main axis of the flex container to run up and down the page and making the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
h1,
article {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (column-rule-inset-junction: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-junction property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.columnRuleInsetJunction = val;
  containers[1].style.columnRuleInsetJunction = val;
  output.innerText = val;
});
```

Result

Change the size of the inset. Note that in the right-hand example, where the column rule is a single segment that goes from top to bottom, the segment has two cap endpoints and no junction endpoints. Therefore, changing the `column-rule-inset-junction` value does not affect this example.

With grid layout

This example demonstrates using the `column-rule-inset-junction` property to inset the top and bottom junction segment endpoints to two different values on a grid container.

HTML

We include the `<ul>` element as a container with several `<li>` children which will each be converted into a grid item.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>15</li>
  <li>16</li>
</ul>
```

```
<p>
  <label
    >Change the <code>-start</code> value.
    <input type="range" min="-40" max="40" value="16" id="start" data-unit="px"
  /></label>
```

```
  <output id="og">16px</output>
</p>
<p>
  <label
    >Change the <code>-end</code> value.
    <input type="range" min="-40" max="40" value="0" id="end" data-unit="px"
  /></label>
  <output id="ow">0px</output>
</p>
```

CSS

We turn the `<ul>` into a grid container by setting the `display` property to `grid`. The `grid-template-columns` property specifies that the grid has six columns. We remove the bullets with `list-style-type` and set the row and column gaps to `20px` with the `gap` shorthand. We set the color, size, and line style of all the rules using the `rule` shorthand, then change just the row rule color with the `row-rule-color` property.

We break the column rules at every intersection using the `column-rule-break` property. If the column rules didn't break, there would be no column junction segments to style!

Finally, we set the start of each column junction to be inset by `16px` and the end to not be inset using the `column-rule-inset-junction` property.

We also set the 6th grid item to span three columns.

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: "";
  gap: 20px;
  rule: 10px solid olive;
  row-rule-color: palegoldenrod;
  column-rule-break: intersection;

  column-rule-inset-junction: 16px 0;
}

li:nth-of-type(8) {
  grid-column-end: span 3;
}
```

The rest of the CSS is hidden for brevity.

```
ul {
  border: 1px solid;
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 2em 1em;
  width: 100%;
  box-sizing: border-box;
}
li:nth-of-type(8) {
  background-color: transparent;
}

li code {
  display: block;
  margin: 0 auto;
  text-align: left;
  width: 30vw;
}
output {
  font-family: monospace;
}
input {
  accent-color: olive;
}
@layer no-support {
  @supports not (column-rule-inset-junction: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-junction property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const ul = document.querySelector("ul");
const startSize = document.getElementById("start");
const og = document.getElementById("og");
const endSize = document.getElementById("end");
const ow = document.getElementById("ow");
const cell = document.querySelector("li:nth-of-type(8)");
let text = "";
function update() {
  ul.style.columnRuleInsetJunction =
    text = `${startSize.value}px ${endSize.value}px`;
  cell.innerHTML = `<code>rule-inset-cap: ${text};</code>`;
}

update();

startSize.addEventListener("input", () => {
  og.innerText = `${startSize.value}px`;
  update();
});
```

```
endSize.addEventListener("input", () => {
  ow.innerText = `${endSize.value}px`;
  update();
});
```

Result

See also

- `column-rule-inset-junction-start`
- `column-rule-inset` shorthand
- `rule-inset` shorthand
- `column-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

column-rule-inset-junction-end CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-inset-junction-end`** CSS property can be used to offset the bottom endpoints of column rule segments that are junction endpoints; that is, endpoints at gap junctions where rule segments intersect.

Try it

```
column-rule-inset-junction-end: 0;
```

```
column-rule-inset-junction-end: 0.5em;
```

```
column-rule-inset-junction-end: 10px;
```

```
column-rule-inset-junction-end: -100%;
```

```
column-rule-inset-junction-end: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  rule: solid thick lightpink;
  column-rule-color: magenta;
  column-rule-break: intersection;
  gap: 1.5em;
  rule-overlap: column-over-row;
  border: 1px solid rebeccapurple;
  margin: auto;
  rule-visibility-items: around;
}
#example-element i {
  background-color: #efefef;
  padding: 1em;
}
```

Syntax

```
/* Keywords */
column-rule-inset-junction-end: overlap-join;

/* <length-percentage> values */
column-rule-inset-junction-end: 0;
column-rule-inset-junction-end: 1em;
column-rule-inset-junction-end: -5px;
column-rule-inset-junction-end: -25%;

/* Global values */
column-rule-inset-junction-end: inherit;
column-rule-inset-junction-end: initial;
column-rule-inset-junction-end: revert;
column-rule-inset-junction-end: revert-layer;
column-rule-inset-junction-end: unset;
```

Values

This property is specified as a single value from the following list:

`overlap-join`

Specifies that the junction segment should extend across the row-rule, resolving to half the `row-gap` value plus half the used `row-rule-width` value.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the junction endpoint, which is the `row-gap` value.

Description

The `column-rule-inset-junction-end` property can be used to inset or outset junction segment endpoints occurring at the bottom of column rule segments. The default value is `0`. Positive values reduce the segment size, while negative values and the `overlap-join` keyword increase it.

Column rules are painted within a column gap as one or more segments, with segments occurring between:

- Adjacent columns in CSS grid layouts.
- Adjacent flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Adjacent columns in multi-col layouts.

Whether a column rule spans multiple rows or is broken into multiple segments is defined by the `column-rule-break` property; interior breaks between column rule segments are the size of the `row-gap`. A junction end occurs at the bottom of every column segment where the segment's bottom ends at a gap junction with other column or rule segments present.

The `column-rule-inset-junction-end` property is a constituent property of several shorthand properties:

- To inset top and bottom column segment junction endpoints, the `column-rule-inset-junction-end` property, along with the `column-rule-inset-junction-start` property, can be set using the `column-rule-inset-junction` shorthand.
- To inset all bottom column segment endpoints, the `column-rule-inset-junction-end` property, along with the `column-rule-inset-cap-end` property, can be set using the `column-rule-inset-end` shorthand.
- To inset bottom column segment junction endpoints and right row segment junction endpoints, the `column-rule-inset-junction-end` property, along with the `row-rule-inset-junction-end` property, can be set using the `rule-inset-junction-end` shorthand.

All segment endpoints, including this property's `-start`, `-cap`, and `row-` equivalents, can be set using the `rule-inset` shorthand.

Understanding junction endpoints

A *junction segment endpoint* is any segment endpoint at an interior gap that ends at a gap intersection where other rule or column segments are present. The `column-rule-inset-junction-end` property controls the inset of the bottom edge of column junction segments, allowing the segments to be shrunk or extended.

Length `column-rule-inset-junction-end` values inset segments by the specified value. Negative length values create an outset, extending the bottom end of the junction segment. Percentage values are relative to the size of the `row-gap`. Setting `-50%` outsets the bottom of the junction segment half way through the row gap below the segment, no matter how wide the row gap is.

In the following demonstration, the column rule segments in the top two rows end in junction endpoints. With `column-rule-inset-junction-end: 16px` set, the bottom, or end, of these segments are inset by `16px`. Change the inset `<length>` value to better visualize which segments end in junction segment endpoints.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>15</li>
  <li>16</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset" data-unit="px"
  /></label>
  <output id="o">16px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-450"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
```

```
  list-style-type: none;
  gap: 20px;
  column-rule: 10px solid olive;
  row-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: normal;
  rule-break: intersection;
  column-rule-inset-junction-end: 16px;

  border: 1px solid;
}
ul {
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
@layer no-support {
  @supports not (column-rule-inset-junction-end: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-junction-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  column-rule-inset-junction-end: 100%;
  column-rule-style: inset;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  o.innerText =
    ul.style.columnRuleInsetJunctionEnd = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "repeat(3, solid), repeat(2, double)";
  } else if (visibility.value === "around") {
    ul.style.columnRuleStyle = "repeat(3, solid), repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.columnRuleStyle = "repeat(2, inset), double, repeat(2, solid)";
  } else if (visibility.value === "around") {
    ul.style.columnRuleStyle = "repeat(3, inset), repeat(2, double)";
  } else {
    ul.style.columnRuleStyle = "solid";
  }
});
```

If you set `0px` as the value, the end of the column rules will align with the end of the row, abutting the row gap. This is the default. Note how only the bottom ends of the segments in the middle of the grid change when the property value changes. The segments in the bottom row do not change: these are *cap endpoints*, and are not affected by the `column-rule-inset-junction-end` property.

Select `around` as the `rule-visibility-items` value. This value paints rules in a gap segment if an item occupies at least one of the two adjacent areas. The double line-style column rule segments that appear when the `rule-visibility-items` property is set to `around` end at an interior intersection where one or more row-rule segments are present; these column segments are junction segment endpoints.

Select `between` as the `rule-visibility-items` value, which paints rules in gap segments only if items occupy both adjacent areas. The double line-style column rule segments now end at an interior intersection where no other rule segments are present: these column segments are *cap segment endpoints* and therefore aren't affected by the `column-rule-inset-junction-end` property.

The <code>overlap-join</code> value

The `overlap-join` value outsets the bottom end of interior segments so they align with the bottom of the intersected row-rule. The value resolves to half the `row-gap` size (which would extend it to the middle of the gap) plus half the row rule width.

When `column-rule-inset-junction-end` is set to the `overlap-join` keyword value, the bottom ends of the junction end segments extend down into the row gap to meet, or "join", the bottom edge of the row rule painted in that gap.

In the following live example, `column-rule-inset-junction-end` is set to `overlap-join`:

```
<p>
  <label
    >Change the <code>row-gap</code>.
    <input type="range" min="10" max="40" value="30" id="gap" data-unit="px"
  /></label>
  <output id="og">30px</output>
</p>
<p>
```

```
  <label
    >Change the <code>row-rule-width</code>.
    <input type="range" min="0" max="40" value="16" id="rrw" data-unit="px"
  /></label>
  <output id="ow">16px</output>
</p>

<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
</ul>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(4, auto);
  list-style-type: none;
  gap: 30px;
  column-rule: 16px solid olive;
  row-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: around;
  column-rule-break: intersection;
  column-rule-inset-junction-end: overlap-join;
  border: 1px solid;
}
ul {
  place-items: center;
  width: 70vw;
  margin: auto;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 2em;
  width: 100%;
  box-sizing: border-box;
}
@layer no-support {
  @supports not (column-rule-inset-junction-end: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-junction-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const ul = document.querySelector("ul");
const gapSize = document.getElementById("gap");
const og = document.getElementById("og");
const ruleWidth = document.getElementById("rrw");
const ow = document.getElementById("ow");

gapSize.addEventListener("input", () => {
  og.innerText = ul.style.rowGap = `${gapSize.value}px`;
});

ruleWidth.addEventListener("input", () => {
  ow.innerText = ul.style.rowRuleWidth = `${ruleWidth.value}px`;
});
```

Change the size of the `row-rule-width` and the `row-gap`. Notice how, no matter the size of the gap or row rule, the bottom end of the column segment is always stretched to align with the bottom of the row rule painted in the gap.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-inset-junction-end =
  <inset-value>

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `column-rule-inset-junction-end` to inset the end edge of junction segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting junction column rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a light blue `rule` to paint both row and column gaps, then overwrite the `column-rule-color`, setting darker `blue` column gap decorations. We also set the `rule-overlap` property to `column-over-row` to ensure the column segments are drawn on top of the row segments when the segments overlap. Finally, we set the `column-rule-inset-junction-end` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  column-rule-color: blue;
  rule-overlap: column-over-row;

  column-rule-inset-junction-end: 16px;
}
```

We also set the `flex-direction` on the `.column` container to `column`, changing the main axis of the flex container to run up and down the page and making the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
h1,
article {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
```

```
  @supports not (column-rule-inset-junction-end: 16px) {
    body::before {
      content: "Your browser doesn't support the column-rule-inset-junction-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.columnRuleInsetJunctionEnd = val;
  containers[1].style.columnRuleInsetJunctionEnd = val;
  output.innerText = val;
});
```

Result

Change the size of the inset. Note that in the right-hand example, where the column rule is a single segment that goes from top to bottom, the segment has two cap endpoints and no junction endpoints. Therefore, changing the `column-rule-inset-junction-end` value does not affect this example.

See also

- `column-rule-inset-junction-start`
- `column-rule-inset-junction` shorthand
- `rule-inset-junction-start` shorthand
- `column-rule-inset` shorthand
- `rule-inset` shorthand
- `column-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

column-rule-style CSS property

The **`column-rule-style`** CSS property defines the line style of the lines drawn between columns in multi-column grid, flex, and multi-col layouts.

Try it

```
column-rule-style: dotted;
```

```
column-rule-style: dashed, dotted;
```

```
column-rule-style: repeat(2, inset, outset), double;
```

```
column-rule-style: double, repeat(auto, dashed, solid), double;
```

```
column-rule-style: hidden;
```

```
<section id="default-example">
  <p id="example-element">
    London. Noel term lately over, and the Lord George sitting in Lincoln's Inn
    Hall. Great May weather. As much mud in the streets as if the waters had but
    newly retired from the face of the earth, and it would not be weird to meet
    an platypus, two feet long or so, waddling like an lizard up Morgan Hill.
  </p>
</section>
```

```
#example-element {
  columns: 6;
  column-rule-width: thick;
  column-rule-color: teal;
  gap: 7px;
}
```

Syntax

```
/* Keyword values */
column-rule-style: none;
column-rule-style: hidden;
column-rule-style: dotted;
column-rule-style: dashed;
column-rule-style: solid;
column-rule-style: double;
column-rule-style: groove;
column-rule-style: ridge;
column-rule-style: inset;
column-rule-style: outset;

/* Multiple values */
column-rule-style: groove, double, dashed;
column-rule-style: solid, repeat(5, ridge), solid;
column-rule-style: dotted, repeat(auto, inset, outset), dotted;

/* Global values */
column-rule-style: inherit;
column-rule-style: initial;
column-rule-style: revert;
```

```
column-rule-style: revert-layer;
column-rule-style: unset;
```

Values

The `column-rule-style` property accepts a comma-separated list of values, including:

`<line-style>`

A `<line-style>`: one of `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`. The default value is `none`.

`<repeat-line-style>`

A `repeat()` function, with the first argument being an `<integer>` of `1` or more, and subsequent arguments being `<line-style>` values. The integer specifies how many times the `<line-style>` values should be repeated.

`<auto-repeat-line-style>`

A `repeat()` function, with `auto` as the first argument and one or more `<line-style>` values as subsequent arguments. The provided `<line-style>` values are repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

Description

The `column-rule-style` property defines the line style of any column rule lines drawn in the gaps between columns in multi-column, flex, and grid containers with more than one column.

The value is a comma-separated list of components, which can include `<line-style>`, `<repeat-line-style>`, and `<auto-repeat-line-style>` types.

The `column-rule-style`, along with the `column-rule-color` and `column-rule-width` properties, can be set using the `column-rule` shorthand. The `column-rule-style`, along with the `row-rule-style` property, can also be set using the `rule-style` shorthand.

If the property value has only one `<line-style>`, all the column rules will be that style. If we declare the following, all column rules will be `double`:

```
column-rule-style: double;
```

When more than one `<line-style>` is declared, they will be applied to column-rules in the order specified. If there are more column-rules than `<line-style>` values, the list of line styles is repeated until every column rule has a style. If we declare the following, for example, every odd rule will be `double`, and every even rule will be `groove`.

```
column-rule-style: double, groove;
```

Repeated line styles

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<line-style>` values passed as subsequent arguments the specified number of times. This allows the same style to be repeated a set number of times without repeating the same value. You can include `<line-style>` keyword values or custom properties that resolve to a valid `<line-style>`. Using `repeat()` can make values easier to write, enabling recurring patterns to be written using a single function, regardless of the number of columns. The following declarations are equivalent:

```
column-rule-style: solid, outset, inset, outset, inset;
column-rule-style: solid, repeat(2, outset, inset);
```

This creates a list of five styles. If the number of styles in the `column-rule-style` value's style list exceeds the number of gaps between columns, the excess style values are ignored. If the container has three columns, the rule in the first gutter will be `solid` and the second `outset`.

If there are more gutters than styles, the list of styles is repeated. If the container has 6, 11, 16, or 21 columns, this sequence of styles will be repeated one, two, three, or four times, respectively, with the last rule being `inset`.

Auto-repeating line styles

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<line-style>` values passed as subsequent parameters will be repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

```
column-rule-style: solid, repeat(auto, dotted), solid;
```

In this case, it doesn't matter if the container has 3, 6, 11, 16, or 21 columns; the first and last column rules will always be `solid`, and all the other column rules will be `dotted`. If there are only 2 or 3 columns, there will be no dotted column rules.

The `auto` keyword within the `repeat()` function creates an auto repeater that fills in values for column rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. Only one `repeat(auto, <line-style>)` is allowed within a `column-rule-style` value.

Formal definition

Initial value	none
Applies to	multicol elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
column-rule-style =
  <line-style-list>      |
  <auto-line-style-list>

<line-style-list> =
  <line-style-or-repeat>#

<auto-line-style-list> =
  <line-style-or-repeat>#? , <auto-repeat-line-style> , <line-style-or-repeat>#?

<line-style-or-repeat> =
  <line-style>         |
  <repeat-line-style>

<auto-repeat-line-style> =
  repeat( auto , [ <line-style> ]# )

<line-style> =
  none    |
```

```
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<repeat-line-style> =
  repeat( [ <integer [1,∞]> ] , [ <line-style> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

HTML

```
<p>
  This is a bunch of text split into three columns. The `column-rule-style`
  property is used to change the style of the line that is drawn between
  columns. Don't you think that's wonderful?
</p>
```

CSS

```
p {
  column-count: 3;
  column-rule-style: dashed;
}
```

Result

Multiple values

HTML

We include a list of authors:

```
<ul>
  <li>Kimberlé Crenshaw</li>
  <li>Angela Y. Davis</li>
  <li>Bernardine Evaristo</li>
  <li>Audre Lorde</li>
  <li>Cathy Park Hong</li>
  <li>Zoya Patel</li>
  <li>Juno Mac</li>
  <li>Molly Smith</li>
  <li>Tara Westover</li>
</ul>
```

CSS

We define the list to be a flex container, creating columns by setting the `flex-direction` to `row` using the `flex-flow` shorthand. We include a `gap` of `5px` to provide enough room between the columns to fit our `3px` double teal rule:

```
ul {
  display: flex;
  flex-flow: row;
  gap: 5px;
  column-rule-width: 3px;
  column-rule-color: teal;

  column-rule-style:
    dotted, dashed, solid, double, groove, ridge, inset, outset, none, hidden;
}
```

Result

As there are more values (10) than gutters (8), the `none` and `hidden` values are not used.

Repeating values

This example demonstrates how, when there are fewer values in the list of styles than column rules, the values are repeated.

CSS

Using the same HTML and CSS as in the previous example, we include three comma-separated styles as the `column-rule-style` value:

```
ul {
  column-rule-style: solid, groove, double;
}
```

Result

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `column-rule-style` property value.

CSS

We use the same HTML and CSS as in the previous examples. We include a `repeat()` function, setting the list of two `<line-style>` values to be repeated 3 times.

```
ul {
  column-rule-style: solid, repeat(3, inset, outset), solid;
}
```

Result

The flex container has six columns, so five gutters. The `repeat()` function repeats two style values three times, creating a list of eight style values, so the last three values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto` instead of an integer within the `repeat()` function.

CSS

Using `repeat(auto, <line-style>)` we set all column rules to `groove`, except the first and last, which we set to `solid`.

```
ul {
  column-rule-style: solid, repeat(auto, groove), solid;
}
```

Result

```
@layer no-support {
  @supports not (column-rule-style: solid, groove) {
    body::before {
      content: "Your browser doesn't support multiple values for the column-rule-style property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `column-rule-color`
- `column-rule-width`
- `row-rule-style`
- `column-rule` shorthand
- `rule-style` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

column-rule-visibility-items CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-rule-visibility-items`** CSS property defines whether a column-rule segment is painted in gaps adjacent to empty areas.

Try it

```
column-rule-visibility-items: all;
```

```
column-rule-visibility-items: around;
```

```
column-rule-visibility-items: between;
```

```
column-rule-visibility-items: normal;
```

```
<section id="default-example">
  <section id="example-element">
    <p>One fish</p>
    <p>Two fish</p>
    <p>Red fish</p>
    <p>Blue fish</p>
    <cite>-- Dr. Seuss</cite>
  </section>
</section>
```

```
#example-element {
  display: grid;
  column-rule: solid 5px red;
  gap: 10px;
  grid-template-rows: repeat(3, 1fr);
  grid-template-columns: repeat(3, 1fr);
}
cite {
  grid-row: 3;
  grid-column: 3;
}
```

Syntax

```
/* Keyword values */
column-rule-visibility-items: all;
column-rule-visibility-items: around;
column-rule-visibility-items: between;
column-rule-visibility-items: normal;

/* Global values */
column-rule-visibility-items: inherit;
column-rule-visibility-items: initial;
```

```
column-rule-visibility-items: revert;
column-rule-visibility-items: revert-layer;
column-rule-visibility-items: unset;
```

Values

This property is specified as one of the following keyword values:

`all`

The column rule should be painted in all gap segments, regardless of whether adjacent areas contain an item.

`around`

The column rule should be painted in a gap segment if at least one of the two adjacent areas is occupied by an item.

`between`

The column rule should be painted in a gap segment if both adjacent areas are occupied by items.

`normal`

With grid containers, behaves the same as `all`. In multicol layout, behaves the same as `between`. This is the default.

Description

The `column-rule-visibility-items` property defines whether a column-rule segment is painted in column gaps adjacent to empty areas in multi-row and grid containers with more than one column.

The `column-rule-visibility-items` and `row-rule-visibility-items` properties can both be set using the `rule-visibility-items` shorthand.

Formal definition

Value not found in DB!

Formal syntax

```
column-rule-visibility-items =
  all     |
  around  |
  between |
  normal
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a column rule to be drawn between two grid areas if at least one adjacent grid area contains a grid item.

HTML

We include a list of dynamic sports duos:

```
<ol>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
  <li>Kerri Walsh + Misty May Treanor</li>
</ol>
```

CSS

We define the ordered list (`<ol>`) to be a grid container, creating 4 columns and 4 rows by setting both the `grid-template-columns` and `grid-template-rows` properties to `repeat(4, 1fr)`, and move the last item to the bottom-right grid area using `grid-column` and `grid-row` properties. We include a `gap` of `20px` to provide enough room between the columns to fit our `5px` dashed rule.

Finally, we set `column-rule-visibility-items` to `between`, so that a column rule is painted in a column gap only if both adjacent grid areas contain a grid item.

```
ol {
  display: grid;
  grid-template-rows: repeat(4, 1fr);
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;

  column-rule: dashed 5px blue;
  column-rule-visibility-items: between;
}
li:last-child {
  grid-row: 4;
  grid-column: 4;
}
```

```
li {
  margin-left: 1em;
}
@layer no-support {
  @supports not (column-rule-visibility-items: around) {
    body::before {
      content: "Your browser doesn't support the column-rule-visibility-items property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

See also

- `rule-visibility-items` shorthand
- `row-rule-visibility-items`
- `column-rule` shorthand
- `rule` shorthand
- CSS gaps module

column-rule-width CSS property

The **`column-rule-width`** CSS property defines the widths of the lines drawn between columns in multi-column grid, flex, and multi-col layouts.

Try it

```
column-rule-width: thin;
```

```
column-rule-width: 4px;
```

```
column-rule-width: thin, medium, thick;
```

```
column-rule-width: repeat(2, 1px, thick), 10px;
```

```
column-rule-width: 10px, repeat(auto, 1px, 2px), 10px;
```

```
<section id="default-example">
  <p id="example-element">
    London. Noel term lately over, and the Lord George sitting in Lincoln's Inn
    Hall. Great May weather. As much mud in the streets as if the waters had but
    newly retired from the face of the earth, and it would not be weird to meet
    an platypus, two feet long or so, waddling like an lizard up Morgan Hill.
  </p>
</section>
```

```
#example-element {
  columns: 6;
  column-rule-style: solid;
  column-rule-color: teal;
  gap: 7px;
}
```

Syntax

```
/* Keyword values */
column-rule-width: thin;
column-rule-width: medium;
column-rule-width: thick;
column-rule-width: thin, medium, thick;
column-rule-width: thick, repeat(5, thin), thick;
column-rule-width: thick, repeat(auto, thin, medium), thick;

/* Length values */
column-rule-width: 0.1em;
column-rule-width: 5px;
column-rule-width: 1px, 3px, 5px;
column-rule-width: 0.1rem, repeat(auto, 1px), 10px, 0.5rem;
column-rule-width: 5px, repeat(5, 1px, 3px), 5px;

/* Global values */
column-rule-width: inherit;
column-rule-width: initial;
column-rule-width: revert;
column-rule-width: revert-layer;
column-rule-width: unset;
```

Values

The `column-rule-width` property accepts a comma-separated list of values, including:

`<line-width>`

Defines the width of the line, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

`<repeat-line-width>`

A `repeat()` function, with the first argument being an `<integer>` of `1` or more, and one or more `<line-width>` values as subsequent arguments. The integer defines the number of times the `<line-width>` values should be repeated.

`<auto-repeat-line-width>`

A `repeat()` function, with `auto` as the first argument and one or more `<line-width>` values as subsequent arguments. The provided `<line-width>` values are repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

Description

The `column-rule-width` property defines the widths of any column rule lines drawn in the gutters between adjacent columns in multi-column, flex, and grid containers with more than one column.

> **Note:** The `column-rule-width` only defines the width of the lines painted in the gaps. These lines have no effect on the box model or layout. The size of the gutter is defined by the `gap` property; with the default value being `1em` on multi-column containers and `0` in all other contexts. If the width of a rule is wider than the `gap`, the line will be painted behind the column content.

The value is a comma-separated list of components, which can include `<line-width>`, `<repeat-line-width>`, and `<auto-repeat-line-width>` types.

The `column-rule-width`, along with the `column-rule-color` and `column-rule-style` properties, can also be set using the `column-rule` shorthand, while `rule-width` is a shorthand that sets both the `column-rule-width` and the `row-rule-width` properties.

A `<line-width>` can be declared as any valid CSS `<line-width>` value: the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value. Percentage values are invalid.

If the property value consists of only one `<line-width>`, all the column rules will be that width. If we declare the following, all column rules will be `2px`:

```
column-rule-width: 2px;
```

When more than one `<line-width>` is declared, they will be applied to column-rules in the order specified. If there are more column-rules than `<line-width>` values, the list of line widths is repeated until every rule has a width. If we declare the following, for example, every odd rule will be `thick`, and every even rule will be `0.25rem`.

```
column-rule-width: thick, 0.25rem;
```

Repeated line widths

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<line-width>` values passed as subsequent arguments the specified number of times. This allows the same width to be repeated a set number of times without repeating the same `<line-width>` multiple times. The following declarations are equivalent:

```
column-rule-width: 1rem, thick, thin, thick, thin, thick, thin;
column-rule-width: 1rem, repeat(3, thick, thin);
```

You can use any `<line-width>` values, including custom properties that resolve to a `<line-width>`. Using `repeat()` can make values easier to write, especially when using complex length calculations. It enables a recurring pattern to be written using a single function, regardless of the number of columns. The following declarations are equivalent:

```
column-rule-width:
  1rem, min(calc(var(--base) - 3px), 10px), abs(calc(var(--secondary) - 30px)),
  min(calc(var(--base) - 3px), 10px), abs(calc(var(--secondary) - 30px)),
  min(calc(var(--base) - 3px), 10px), abs(calc(var(--secondary) - 30px)),
  min(calc(var(--base) - 3px), 10px), abs(calc(var(--secondary) - 30px)),
  min(calc(var(--base) - 3px), 10px), abs(calc(var(--secondary) - 30px)), thin;
column-rule-width:
  1rem,
  repeat(
    5,
    min(calc(var(--base) - 3px), 10px),
    abs(calc(var(--secondary) - 30px))
  ),
  thin;
```

This creates a list of 12 widths. If the number of widths in the `column-rule-width` value's width list exceeds the number of gaps between columns, the excess width values are ignored. If the container has three columns, the rule in the first gutter will be `1rem` wide, and the second is determined by the `min()` function.

If there are more gutters than widths, the list of widths is repeated. If the container has 13 or 25 columns, this sequence of widths will be repeated one or two, respectively, with the last rule being `thin`. For any other number of columns, up to 25, the last rule will not be `thin`.

Auto-repeating line widths

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the list of `<line-width>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any column-rules that are not explicitly specified by other components of the property value.

```
column-rule-width: 10px, repeat(auto, thin), 10px;
```

In this case, the first column rule will be `10px`, the last will be `10px`, and all others will be `thin`. It doesn't matter if the container has 3, 6, 11, 16, or 21 columns, the first and last columns will always be `10px`. Which means, if there are only 2 or 3 columns, there will be no thin-sized column rules.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for the row-rule line widths that would not otherwise receive values from other parts of the list, preventing the list from being cycled. A `column-rule-width` value can include, at most, one `repeat(auto, <line-width>)`.

Formal definition

Initial value	`medium`
Applies to	multicol elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
column-rule-width =
  <line-width-list>      |
  <auto-line-width-list>

<line-width-list> =
  <line-width-or-repeat>#

<auto-line-width-list> =
  <line-width-or-repeat>#? , <auto-repeat-line-width> , <line-width-or-repeat>#?

<line-width-or-repeat> =
  <line-width>         |
  <repeat-line-width>

<auto-repeat-line-width> =
  repeat( auto , [ <line-width> ]# )

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<repeat-line-width> =
  repeat( [ <integer [1,∞]> ] , [ <line-width> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates using a single keyword value, setting all the column rules to the same size.

HTML

We include a paragraph of text:

```
<p>
  This is a bunch of text split into three columns. The `column-rule-width`
  property is used to change the width of the line that is drawn between
  columns. Don't you think that's wonderful?
</p>
```

CSS

We create a multi-column container with the `column-count` property. As the `column-rule-style` property defaults to `none`, we have to set it to a visible value for the column rules to be drawn. We then set the `column-rule-width` to `thick`, letting the `column-rule-color` default to `currentcolor`.

```
p {
  column-count: 3;
  column-rule-style: solid;

  column-rule-width: thick;
}
```

Result

In multi-column layouts, the `gap` property defaults to `1em`, which is wider than our `column-rule-width`, so the lines are not drawn over our content.

Multiple values

This example demonstrates using multiple values for the `column-rule-width` property. It also demonstrates rules that overflow gaps are painted behind the content.

HTML

We include a list of authors:

```
<ul>
  <li>Kimberlé Crenshaw</li>
  <li>Angela Y. Davis</li>
  <li>Bernardine Evaristo</li>
  <li>Audre Lorde</li>
  <li>Cathy Park Hong</li>
  <li>Zoya Patel</li>
  <li>Juno Mac</li>
  <li>Molly Smith</li>
  <li>Tara Westover</li>
</ul>
```

CSS

We define the list to be a flex container, creating columns by setting the `flex-direction` to `row` using the `flex-flow` shorthand. We include a `column-rule-width` with ten `<line-width>` values, each larger than the previous.

```
ul {
  display: flex;
  flex-flow: row;
  list-style-type: none;
  column-rule-style: solid;
  column-rule-color: teal;

  column-rule-width: 1px, 2px, 3px, 4px, 5px, 6px, 7px, 8px, 9px, 10px;
}
```

Result

As there are more values (10) than gutters (8), the `9px` and `10px` values are not used.

The `gap` defaults to `normal` which resolves to `0` in flexbox. The `column-rule-width` only defines the width of a painted line and does not impact the layout. The lines are drawn behind the content.

Repeating values

This example demonstrates how, when there are fewer values in the list of widths than column rules, the values are repeated.

Using the same HTML and CSS as in the previous example, we include three comma-separated widths as the `column-rule-width` value:

```
ul {
  column-rule-width: 1px, 5px, 10px;
}
```

Result

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `column-rule-width` property value and how this function can help reduce the verbosity of value declarations.

We use the same HTML and CSS as in the previous examples. To demonstrate how values can become verbose and the utility of the `repeat()` function, we declare two custom properties, which we use in `repeat()` function declarations. The `repeat()` function sets the list of two `<line-width>` values to repeat 3 times.

```
ul {
  --base: 0.5vw;
  --secondary: 1vw;
  column-rule-width:
    15px,
    repeat(
      4,
```

```
      min(calc(var(--secondary) + 3px), 10px),
      abs(calc(var(--base) - 2px))
    ),
    15px;
}
```

Result

The flex container has nine columns, so eight gutters. The `repeat()` function repeats two width values four times, creating a list of ten width values. As there are fewer column gutters than total widths, the last two values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto` instead of an integer within the `repeat()` function.

Using `repeat(auto, <line-width>)` we set all column rules to `1px`, except the first and last, which we set to `5px`.

```
ul {
  column-rule-width: 5px, repeat(auto, 1px), 5px;
}
```

Result

```
@layer no-support {
  @supports not (column-rule-width: thin, thick) {
    body::before {
      content: "Your browser doesn't support multiple values for the column-rule-width property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `column-rule-color`
- `column-rule-style`
- `column-rule` shorthand
- `row-rule-width`
- `rule-width` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

column-span CSS property

The **`column-span`** CSS property makes it possible for an element to span across all columns when its value is set to `all`.

An element that spans more than one column is called a **spanning element**.

Try it

```
column-span: none;
```

```
column-span: all;
```

```
<section id="default-example">
  <div class="multicol-element">
    <p>
      London. Michaelmas term lately over, and the Lord Chancellor sitting in
      Lincoln's Inn Hall.
    </p>
    <div id="example-element">Spanner?</div>
    <p>
      Implacable November weather. As much mud in the streets as if the waters
      had but newly retired from the face of the earth, and it would not be
      wonderful to meet a Megalosaurus, forty feet long or so, waddling like an
      elephantine lizard up Holborn Hill.
    </p>
  </div>
</section>
```

```
.multicol-element {
  width: 100%;
  text-align: left;
  column-count: 3;
}

.multicol-element p {
  margin: 0;
}

#example-element {
  background-color: rebeccapurple;
  padding: 10px;
  color: white;
}
```

Syntax

```
/* Keyword values */
column-span: none;
column-span: all;

/* Global values */
column-span: inherit;
```

```
column-span: initial;
column-span: revert;
column-span: revert-layer;
column-span: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

The element does not span multiple columns.

`all`

The element spans across all columns. Content in the normal flow that appears before the element is automatically balanced across all columns before the element appears. The element establishes a new block formatting context.

Formal definition

Initial value	none
Applies to	in-flow block-level elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
column-span =
  none             |
  <integer [1,∞]>  |
  all              |
  auto

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Multi-column Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Making a heading span columns

In this example, the heading is made to span across all the columns of the article.

HTML

```
<article>
  <h2>Header spanning all of the columns</h2>
  <p>
    The h2 should span all the columns. The rest of the text should be
    distributed among the columns.
  </p>
  <p>
    This is a bunch of text split into three columns using the CSS `columns`
    property. The text is equally distributed over the columns.
  </p>
  <p>
    This is a bunch of text split into three columns using the CSS `columns`
    property. The text is equally distributed over the columns.
  </p>
  <p>
    This is a bunch of text split into three columns using the CSS `columns`
    property. The text is equally distributed over the columns.
  </p>
  <p>
    This is a bunch of text split into three columns using the CSS `columns`
    property. The text is equally distributed over the columns.
  </p>
</article>
```

CSS

```
article {
  columns: 3;
}

h2 {
  column-span: all;
}
```

Result

See also

- Spanning and balancing columns
- Inline-level elements
- `HTMLSpanElement`

column-width CSS property

The **`column-width`** CSS property sets the ideal column width in a multi-column layout.

Try it

```
column-width: auto;
```

```
column-width: 6rem;
```

```
column-width: 120px;
```

```
column-width: 18ch;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
#example-element {
  width: 100%;
  columns: auto;
  text-align: left;
}
```

Syntax

```
/* Keyword value */
column-width: auto;

/* <length> values */
column-width: 60px;
column-width: 15.5em;
column-width: 3.3vw;

/* Global values */
column-width: inherit;
column-width: initial;
column-width: revert;
column-width: revert-layer;
column-width: unset;
```

Values

This property is specified as one of the following values:

`<length>`

> Indicates the optimal column width. The actual column width may differ from the specified value: it may be wider when necessary to fill available space, and narrower when the available space is too small. The value must be strictly positive or the declaration is invalid. Percentage values are also invalid.

`auto`

> The width of the column is determined by other CSS properties, such as `column-count`.

Description

The `column-width` property specifies the preferred column width in a multi-column layout. The container will have as many columns as can fit without any of them having a width less than the `column-width` value. If the width of the container is narrower than the specified value, the single column's width will be smaller than the declared column width.

This property can help you create responsive designs that fit different screen sizes. Especially in the presence of the `column-count` property (which has precedence), you must specify all related length values to achieve an exact column width. In horizontal text, these are `width`, `column-width`, `column-gap`, and `column-rule-width`.

The `column-width` property, along with the `column-height` and `column-count` properties, can also be set by using the `columns` shorthand.

Formal definition

Initial value	`auto`
Applies to	Block containers except table wrapper boxes
Inherited	no
Computed value	`auto` if specified as `auto`, otherwise for `<length>` the absolute value specified
Animation type	by computed value type

Formal syntax

```
column-width =
  auto        |
  <box-size>  |
  auto        |
  <box-size>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<length-percentage> =
  <length>     |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<calc-size-basis> =
  <size-keyword>  |
```

```
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting column width in pixels

HTML

```
<p class="content-box">
  Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy
  nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi
  enim ad minim veniam, quis nostrud exercitation ullamcorper suscipit lobortis
  nisl ut aliquip ex ea commodo consequat.
</p>
```

CSS

```
.content-box {
  column-width: 100px;
}
```

Result

See also

- `column-count`
- `column-height`
- `columns` shorthand
- Learn: Multiple-column Layout (Learn Layout)
- Basic Concepts of Multicol

column-wrap CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`column-wrap`** CSS property specifies the wrapping behavior of overflow columns in a CSS multi-column layout.

Syntax

```
/* Keyword values */
column-wrap: auto;
column-wrap: nowrap;
column-wrap: wrap;

/* Global values */
column-wrap: inherit;
column-wrap: initial;
column-wrap: revert;
column-wrap: revert-layer;
column-wrap: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The initial value. If the content container's `column-height` is set to a `<length>`, `auto` resolves to `wrap`, otherwise it resolves to `nowrap`.

`nowrap`

Columns overflow in the inline direction.

`wrap`

Overflow columns are placed in a new row in the block direction.

Description

The `column-wrap` property can be used to set the columns of a CSS multi-column layout to wrap onto a new row when they start to overflow the column width. This is useful for creating more readable layouts when using the `column-count` or `column-width` property to set multiple columns.

Without `column-wrap`, excess columns will overflow to the side, and readers will have to scroll in the inline direction to read all the content. The `column-height` property, along with `column-wrap`, allows you to set a specific height for the columns and wrap them onto a new row of columns when the container edge is reached.

The default value of `column-wrap` is `auto`, which resolves to `wrap` when `column-height` is set to a `<length>` value; `wrap` allows the fixed-height columns to wrap onto multiple rows. When `column-height` is equal to `auto`, `column-wrap: auto` resolves to `nowrap`, allowing the columns to overflow horizontally if a fixed container height is set.

As a result of this default behavior, generally you don't need to explicitly set the `column-wrap` property.

Formal definition

Initial value	`auto`
Applies to	multicol elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
column-wrap =
  auto    |
  nowrap  |
  wrap
```

This syntax reflects the latest standard as per CSS Multi-column Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates basic usage of the `column-wrap` property to create a wrapped multi-col layout via setting a `column-height` property.

HTML

We include a poem by Dr. Seuss using an `<ol>` containing 28 `<li>`s, followed by the author's name in a `<p>`.

```
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  ...
</ol>
<p>-- Dr. Seuss</p>
```

```
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  <li>Black fish</li>
  <li>Blue fish</li>
  <li>Old fish</li>
  <li>New fish.</li>
  <li>This one has a little star.</li>
  <li>This one has a little car.</li>
  <li>Say! What a lot</li>
  <li>Of fish there are.</li>
  <li>Yes. Some are red. And some are blue.</li>
  <li>Some are old. And some are new.</li>
  <li>Some are sad.</li>
  <li>And some are glad.</li>
  <li>And some are very, very bad.</li>
  <li>Why are they</li>
  <li>Sad and glad and bad?</li>
  <li>I do not know.</li>
  <li>Go ask your dad.</li>
  <li>Some are thin.</li>
  <li>And some are fat.</li>
  <li>The fat one has</li>
  <li>A yellow hat.</li>
  <li>From there to here, from here to there,</li>
  <li>Funny things</li>
  <li>Are everywhere.</li>
</ol>
<p>--Dr. Seuss</p>
```

CSS

We define the `<ol>` to be a multi-column container by setting the `column-width` property to `150px`, meaning the container will contain as many columns as possible with each being at least `150px` wide. The `gap` property sets a horizontal gap between columns and a vertical gap between rows of columns. We then set the `column-height` to `3em`, causing the `column-wrap` property's default `auto` value to resolve to `wrap` to create wrapped rows of columns.

```
ol {
  column-width: 150px;
  gap: 2em;
  column-height: 3em;
}
```

Result

Comparing <code>wrap</code> and <code>nowrap</code>

This example features a multi-column layout that demonstrates the difference between the `wrap` and `nowrap` values by allowing you to toggle the column container's `column-wrap` value between the two. The result is a layout that dynamically changes between horizontal and vertical scrolling.

HTML and JavaScript

The markup for this example contains multiple paragraphs of content, taken from MDN's HTML, CSS, and JavaScript home pages, and a JavaScript-powered `<input type="checkbox">` element to toggle the container's `column-wrap` property value between `nowrap` and `wrap`. The HTML and JavaScript have been hidden for brevity.

```
<form>
  <label for="set-wrap"
    >Set <code>column-wrap</code> to <code>wrap</code></label
  >
  <input type="checkbox" id="set-wrap" />
</form>
<h1>HTML, CSS, and JavaScript summaries</h1>
<p>
  <strong>HTML</strong> (HyperText Markup Language) is the most basic building
  block of the Web. It defines the meaning and structure of web content. Other
  technologies besides HTML are generally used to describe a web page's
  appearance (CSS) or behavior (JavaScript).
</p>
<p>
  "Hypertext" refers to text containing links that connect web pages to one
  another, either within a single website or between websites. Links are a
  fundamental aspect of the Web. By uploading content to the Internet and
  linking it to pages created by other people, you become an active participant
  in the World Wide Web.
</p>
<p>
  HTML uses "markup" to annotate text, images, and other content for display in
  a Web browser. HTML markup includes special "elements" such as
  <a href="/en-US/docs/Web/HTML/Reference/Elements/head"
    ><code>&lt;head&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/title"
    ><code>&lt;title&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/body"
    ><code>&lt;body&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/header"
    ><code>&lt;header&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/footer"
    ><code>&lt;footer&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/article"
    ><code>&lt;article&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/section"
    ><code>&lt;section&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/p"><code>&lt;p&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/div"
    ><code>&lt;div&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/span"
    ><code>&lt;span&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/img"
    ><code>&lt;img&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/aside"
    ><code>&lt;aside&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/audio"
    ><code>&lt;audio&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/canvas"
    ><code>&lt;canvas&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/datalist"
    ><code>&lt;datalist&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/details"
    ><code>&lt;details&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/embed"
    ><code>&lt;embed&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/nav"
    ><code>&lt;nav&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/search"
    ><code>&lt;search&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/output"
    ><code>&lt;output&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/progress"
    ><code>&lt;progress&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/video"
    ><code>&lt;video&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/ul"
    ><code>&lt;ul&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/ol"
    ><code>&lt;ol&gt;</code></a
  >,
  <a href="/en-US/docs/Web/HTML/Reference/Elements/li"
    ><code>&lt;li&gt;</code></a
  >
  and many others.
</p>
<p>
  An HTML element is set off from other text in a document by "tags", which
  consist of the element name surrounded by <code>&lt;</code> and
  <code>&gt;</code>. The name of an element inside a tag is case-insensitive.
```

```
  That is, it can be written in uppercase, lowercase, or a mixture. For example,
  the <code>&lt;title&gt;</code> tag can be written as
  <code>&lt;Title&gt;</code>, <code>&lt;TITLE&gt;</code>, or in any other way.
  However, the convention and recommended practice is to write tags in
  lowercase.
</p>

<hr />

<p>
  <strong>Cascading Style Sheets</strong> (<strong>CSS</strong>) is a
  <a href="/en-US/docs/Web/API/StyleSheet">stylesheet</a> language used to
  describe the presentation of a document written in
  <a href="/en-US/docs/Web/HTML">HTML</a> or
  <a href="/en-US/docs/Web/XML/Guides/XML_introduction">XML</a> (including XML
  dialects such as <a href="/en-US/docs/Web/SVG">SVG</a>,
  <a href="/en-US/docs/Web/MathML">MathML</a> or
  <a href="/en-US/docs/Glossary/XHTML">XHTML</a>). CSS describes how elements
  should be rendered on screen, on paper, in speech, or on other media.
</p>
<p>
  CSS is among the core languages of the <strong>open web</strong> and is
  standardized across web browsers according to
  <a href="https://www.w3.org/Style/CSS/#specs" class="external" target="_blank"
    >W3C specifications</a
  >. Previously, the development of various parts of CSS specification was done
  synchronously, which allowed the versioning of the latest recommendations. You
  might have heard about CSS1, CSS2.1, or even CSS3. There will never be a CSS3
  or a CSS4; rather, everything is now just "CSS" with individual CSS modules
  having version numbers.
</p>
<p>
  After CSS 2.1, the scope of the specification increased significantly and the
  progress on different CSS modules started to differ so much, that it became
  more effective to
  <a
    href="https://www.w3.org/Style/CSS/current-work"
    class="external"
    target="_blank"
    >develop and release recommendations separately per module</a
  >. Instead of versioning the CSS specification, W3C now periodically takes a
  snapshot of
  <a href="https://www.w3.org/TR/css/" class="external" target="_blank"
    >the latest stable state of the CSS specification</a
  >
  and individual modules progress. CSS modules now have version numbers, or
  levels, such as
  <a
    href="https://drafts.csswg.org/css-color-5/"
    class="external"
    target="_blank"
    >CSS Color Module Level 5</a
  >.
</p>

<hr />

<p>
  <strong>JavaScript</strong> (<strong>JS</strong>) is a lightweight interpreted
  (or
  <a href="/en-US/docs/Glossary/Just_In_Time_Compilation"
    >just-in-time compiled</a
  >) programming language with
  <a href="/en-US/docs/Glossary/First-class_Function">first-class functions</a>.
  While it is most well-known as the scripting language for Web pages,
  <a
    href="https://en.wikipedia.org/wiki/JavaScript#Other_usage"
    class="external"
    target="_blank"
    >many non-browser environments</a
  >
  also use it, such as <a href="/en-US/docs/Glossary/Node.js">Node.js</a>,
  <a href="https://couchdb.apache.org/" class="external" target="_blank"
    >Apache CouchDB</a
  >
  and
  <a
    href="https://opensource.adobe.com/dc-acrobat-sdk-docs/acrobatsdk/"
    class="external"
    target="_blank"
    >Adobe Acrobat</a
  >. JavaScript is a
  <a href="/en-US/docs/Glossary/Prototype-based_programming">prototype-based</a
  >, <a href="/en-US/docs/Glossary/Garbage_collection">garbage-collected</a>,
  <a href="/en-US/docs/Glossary/Dynamic_typing">dynamic</a> language, supporting
  multiple paradigms such as imperative, functional, and object-oriented.
</p>
<p>
  JavaScript's dynamic capabilities include runtime object construction,
  variable parameter lists, function variables, dynamic script creation (via
  <a href="/en-US/docs/Web/JavaScript/Reference/Global_Objects/eval"
    ><code>eval</code></a
  >), object introspection (via
  <a href="/en-US/docs/Web/JavaScript/Reference/Statements/for...in"
    ><code>for...in</code></a
  >
  and
  <a
    href="/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object#static_methods"
    ><code>Object</code> utilities</a
  >), and source-code recovery (JavaScript functions store their source text and
  can be retrieved through
  <a
```

```
    href="/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/toString"
    ><code>toString()</code></a
  >).
</p>
<p>
  This section is dedicated to the JavaScript language itself, and not the parts
  that are specific to Web pages or other host environments. For information
  about <a href="/en-US/docs/Glossary/API">APIs</a> that are specific to Web
  pages, please see <a href="/en-US/docs/Web/API">Web APIs</a> and
  <a href="/en-US/docs/Glossary/DOM">DOM</a>.
</p>
<p>
  The standards for JavaScript are the
  <a href="https://tc39.es/ecma262/" class="external" target="_blank"
    >ECMAScript Language Specification</a
  >
  (ECMA-262) and the
  <a href="https://tc39.es/ecma402/" class="external" target="_blank"
    >ECMAScript Internationalization API specification</a
  >
  (ECMA-402). As soon as one browser implements a feature, we try to document
  it. This means that cases where some
  <a href="https://github.com/tc39/proposals" class="external" target="_blank"
    >proposals for new ECMAScript features</a
  >
  have already been implemented in browsers, documentation and examples in MDN
  articles may use some of those new features. Most of the time, this happens
  between the
  <a href="https://tc39.es/process-document/" class="external" target="_blank"
    >stages</a
  >
  3 and 4, and is usually before the spec is officially published.
</p>
<p>
  Do not confuse JavaScript with the
  <a
    href="https://en.wikipedia.org/wiki/Java_(programming_language)"
    class="external"
    target="_blank"
    >Java programming language</a
  >
  — <strong>JavaScript is <em>not</em> "Interpreted Java"</strong>. Both "Java"
  and "JavaScript" are trademarks or registered trademarks of Oracle in the U.S.
  and other countries. However, the two programming languages have very
  different syntax, semantics, and use.
</p>
<p>
  JavaScript documentation of core language features (pure
  <a
    href="/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview"
    >ECMAScript</a
  >, for the most part) includes the following:
</p>
```

```
const checkbox = document.getElementById("set-wrap");
checkbox.addEventListener("change", () => {
  if (checkbox.checked) {
    document.body.style.columnWrap = "wrap";
  } else {
    document.body.style.columnWrap = "nowrap";
  }
});
```

CSS

We make the `<body>` element a multi-col container by setting the `column-count` to `3`. We then set a `gap` of `3em 2em`, resulting in a `3em` gap between rows and a `2em` gap between columns.

We then set a `column-height` of `90vh`, making the columns nearly as tall as the viewport. We also set `column-wrap` to `nowrap`, causing excess content columns to overflow horizontally. This is required because the initial `column-wrap` value is `auto`, which resolves to `wrap` when `column-height` is set to a `<length>` value.

The checkbox toggles the `column-wrap` property between `nowrap` and `wrap`. When set to `wrap`, the excess content columns overflow vertically into new rows of columns, creating the vertical layout. The `column-height` value causes each row of columns to fill the viewport.

```
body {
  column-count: 3;
  gap: 3em 2em;
  padding: 0 2em;
  column-height: 90vh;
  column-wrap: nowrap;
}
```

Next, we set the `<h1>` element's `column-span` property to `all`, to make the heading span across all the columns, and set the `margin-top` property of the first `<p>` to `0` so that it lines up with the top of the columns.

```
h1 {
  column-span: all;
}

p:first-of-type {
  margin-top: 0;
}
```

```
* {
  box-sizing: border-box;
}

html {
  font-family: "Arial", sans-serif;
}

p {
  line-height: 1.5;
```

```
}

@supports not (column-wrap: wrap) {
  body::before {
    content: "Your browser does not support the 'column-wrap' property.";
    background-color: wheat;
    position: fixed;
    inset: 40% 0;
    height: fit-content;
    text-align: center;
    padding: 1rem 0;
  }
}

form {
  position: fixed;
  top: 0;
  right: 0;
  background-color: white;
  padding: 5px;
  border: 1px solid black;
}
```

Result

Toggle the checkbox to change the value of the `column-wrap` property and switch the layout between horizontal and vertical scrolling. When `column-wrap` is set to `nowrap`, the columns overflow horizontally; when `column-wrap` is set to `wrap`, new rows of columns are added vertically.

See also

- `column-count`
- `column-width`
- `columns` shorthand
- `column-height`
- CSS multi-column layout module

columns CSS property

The **`columns`** CSS shorthand property sets the maximum number of columns to use when drawing an element's contents, along with the minimum width and maximum height of the element's columns.

Try it

```
columns: 2;
```

```
columns: 6rem auto;
```

```
columns: 12em;
```

```
columns: 2 / 5em;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
#example-element {
  min-width: 21rem;
  text-align: left;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-count`
- `column-width`
- `column-height`

Syntax

```
/* Column width */
columns: 18em;

/* Column count */
columns: auto;
columns: 2;

/* Column width and count */
columns: 2 auto;
columns: auto 12em;
columns: auto auto;

/* Column width and/or count, and column height */
columns: 18em / 10em;
columns: 2 / 90vh;
columns: 2 auto / 300px;

/* Global values */
columns: inherit;
columns: initial;
columns: revert;
columns: revert-layer;
columns: unset;
```

The `columns` property value may be specified as a `<column-count>` and/or a `<column-width>` value, in any order, optionally followed by a `<column-height>` value preceded by a forward slash (/).

Values

`<'column-width'>`

The ideal column width, defined as a `<length>` or the keyword `auto`. The actual width may be wider or narrower to fit the available space. See `column-width`.

`<'column-count'>`

The ideal number of columns into which the element's content should be flowed, defined as an `<integer>` or the keyword `auto`. If specified as an `<integer>`, it defines the maximum allowable number of columns. See `column-count`.

`<'column-height'>`

The height of the columns defined as a `<length>` or the keyword `auto`. See `column-height`.

> **Note:** Setting a `<column-height>` value resets the `column-wrap` property to its initial value, `auto`. When `<column-height>` is set to a `<length>`, the `column-wrap` `auto` value resolves to `wrap`. When `<column-height>` is set to `auto`, it resolves to `nowrap`.

Formal definition

Initial value	as each of the properties of the shorthand: • `column-width`: `auto` • `column-count`: `auto` • `column-height`: `auto`
Applies to	Block containers except table wrapper boxes
Inherited	no
Computed value	as each of the properties of the shorthand: • `column-width`: `auto` if specified as `auto`, otherwise for `<length>` the absolute value specified • `column-count`: as specified • `column-height`: `auto` if specified as `auto`, otherwise for `<length>` the absolute value specified
Animation type	as each of the properties of the shorthand: • `column-width`: by computed value type • `column-count`: an integer • `column-height`: by computed value type

Formal syntax

```
columns =
  [ <'column-width'> || <'column-count'> ] [ / <'column-height'> ]?

<column-width> =
  auto        |
  <box-size>  |
  auto        |
  <box-size>

<column-count> =
  auto            |
  <integer [1,∞]>

<column-height> =
  auto           |
  <length [0,∞]>

<box-size> =
  <length-percentage [0,∞]>               |
  stretch                                 |
  contain                                 |
  min-content                             |
  max-content                             |
  fit-content                             |
  stretch( <length-percentage [0,∞]> )    |
  fit-content( <length-percentage [0,∞]> )  |
  <calc-size()>

<integer> =
  <number-token>

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
```

```
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Multi-column Layout Module Level 2, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting three equal columns

This example shows how to split a text container into three equal columns.

HTML

We include a basic `<p>` element containing text content.

```
<p class="content-box">
  This is a bunch of text split into three columns using the CSS
  <code>columns</code>
  property. The text is equally distributed over the columns.
</p>
```

CSS

We set a `columns` property value on the paragraph that includes a `<column-count>` value of `3` and a `<column-width>` value of `auto`.

```
body {
  width: 60%;
  margin: 0 auto;
}
```

```
.content-box {
  columns: 3 auto;
}
```

Result

Note how the text is split up into three columns.

Creating wrapping, fixed height columns

This example demonstrates including a `<column-height>` value with the `columns` shorthand property to split a text container into fixed-height columns that wrap onto new lines when the container inline edge is reached.

HTML

We include a basic `<p>` element with some text.

```
<p class="content-box">
  This is a bunch of text split into three columns using the CSS
  <code>columns</code> property. This includes a <code>column-count</code> value
  of <code>3</code>, a <code>column-width</code> value of <code>auto</code>, and
  a <code>column-height</code> value of <code>5em</code>. The
  <code>column-wrap</code> value is set to its initial value, <code>auto</code>;
  when a <code>column-height</code> value is included,
  <code>column-wrap: auto</code> resolves to <code>wrap</code>, which allows the
  columns to wrap onto multiple rows. The text is equally distributed over the
  columns, and placed into multiple rows.
</p>
```

CSS

We set a `columns` property value on the paragraph that includes a `<column-count>` value of `3`, a `<column-width>` value of `auto`, and a `<column-height>` value of `5em`. Because the `<column-height>` is set to a `<length>`, the paragraph's `column-wrap` value computes to `wrap`, which allows the columns to wrap onto multiple rows.

```
.content-box {
  columns: 3 auto / 5em;
}
```

```
body {
  width: 60%;
  margin: 0 auto;
}
```

```
@supports not (columns: 3 auto / 5em) {
  body::before {
    content: "Your browser does not support the 'column-height' property.";
    background-color: wheat;
    position: fixed;
    inset: 40% 0;
    height: fit-content;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Note how the text is split up into three columns. Each column is `5em` high. After every third column, the columns wrap onto a new line in the block direction.

See also

- `widows`

- `orphans`
- Paged media
- Learn: Multiple-column Layout

contain CSS property

The **`contain`** CSS property indicates that an element and its contents are, as much as possible, independent from the rest of the document tree. Containment enables isolating a subsection of the DOM, providing performance benefits by limiting calculations of layout, style, paint, size, or any combination to a DOM subtree rather than the entire page. Containment can also be used to scope CSS counters and quotes.

Try it

```
contain: none;
```

```
contain: size;
```

```
contain: layout;
```

```
contain: paint;
```

```
contain: strict;
```

```
<section class="default-example" id="default-example">
  <div class="card" id="example-element">
    <h2>Element with '<code>contain</code>'</h2>
    <p>
      The Goldfish is a species of domestic fish best known for its bright
      colors and patterns.
    </p>
    <div class="fixed"><p>Fixed right 4px</p></div>
  </div>
</section>
```

```
h2 {
  margin-top: 0;
}

#default-example {
  text-align: left;
  padding: 4px;
  font-size: 16px;
}

.card {
  text-align: left;
  border: 3px dotted;
  padding: 20px;
  margin: 10px;
  width: 85%;
  min-height: 150px;
}

.fixed {
  position: fixed;
  border: 3px dotted;
  right: 4px;
  padding: 4px;
  margin: 4px;
}
```

Syntax

```
/* Keyword values */
contain: none;
contain: strict;
contain: content;
contain: size;
contain: inline-size;
contain: layout;
contain: style;
contain: paint;

/* Multiple keywords */
contain: size paint;
contain: size layout paint;
contain: inline-size layout;

/* Global values */
contain: inherit;
contain: initial;
contain: revert;
contain: revert-layer;
contain: unset;
```

Values

This property is specified as a single value (`none`, `strict`, or `content`) or as a space-separated list of one or more of `size` (or `inline-size`), `layout`, `style`, and `paint`, in any order:

`none`

The element renders as normal, with no containment applied.

`strict`

All containment rules are applied to the element. This is equivalent to `contain: size layout paint style`.

`content`

All containment rules except `size` are applied to the element. This is equivalent to `contain: layout paint style`.

`size`

Size containment is applied to the element in both the inline and block directions. The size of the element can be computed in isolation, ignoring the child elements. This value cannot be combined with `inline-size`.

`inline-size`

Inline size containment is applied to the element. The inline size of the element can be computed in isolation, ignoring the child elements. This value cannot be combined with `size`.

`layout`

The internal layout of the element is isolated from the rest of the page. This means nothing outside the element affects its internal layout, and vice versa.

`style`

For properties that can affect more than just an element and its descendants, the effects don't escape the containing element. Counters and quotes are scoped to the element and its contents.

`paint`

Descendants of the element don't display outside its bounds. If the containing box is offscreen, the browser does not need to paint its contained elements — these must also be offscreen as they are contained completely by that box. If a descendant overflows the containing element's bounds, then that descendant will be clipped to the containing element's overflow clip edge. By default, this edge corresponds to the padding-box for non-replaced elements.

Description

There are four types of CSS containment: size, layout, style, and paint, which are set on the container. The property is a space-separated list of a subset of the five standard values or one of the two shorthand values. Changes to the contained properties within the container are not propagated outside of the contained element to the rest of the page. The main benefit of containment is that the browser does not have to re-render the DOM or page layout as often, leading to small performance benefits during the rendering of static pages and greater performance benefits in more dynamic applications.

Using the `contain` property is useful on pages with groups of elements that are supposed to be independent, as it can prevent element internals from having side effects outside of its bounding-box.

> **Note:** Using `layout`, `paint`, `strict` or `content` values for this property creates:
>
> 1. A new containing block (for the descendants whose `position` property is `absolute` or `fixed`).
> 2. A new stacking context.
> 3. A new block formatting context.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
contain =
  none                                          |
  strict                                        |
  content                                       |
  [ [ size | inline-size ] || layout || style || paint ]
```

This syntax reflects the latest standard as per CSS Containment Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Paint containment

The following example shows how to use `contain: paint` to prevent an element's descendants from painting outside of its bounds.

```
div {
  width: 100px;
  height: 100px;
  background: red;
  margin: 10px;
  font-size: 20px;
}

.contain-paint {
  contain: paint;
}
```

```
<div class="contain-paint">
  <p>This text will be clipped to the bounds of the box.</p>
</div>
<div>
  <p>This text will not be clipped to the bounds of the box.</p>
</div>
```

Layout containment

Consider the example below which shows how elements behave with and without layout containment applied:

```
<div class="card contain-layout">
  <h2>Card 1</h2>
  <div class="fixed"><p>Fixed box 1</p></div>
  <div class="float"><p>Float box 1</p></div>
</div>
<div class="card">
  <h2>Card 2</h2>
  <div class="fixed"><p>Fixed box 2</p></div>
  <div class="float"><p>Float box 2</p></div>
</div>
<div class="card">
  <h2>Card 3</h2>
```

```
  <!-- ... -->
</div>
```

```
p {
  margin: 4px;
  padding: 4px;
}

h2 {
  margin-bottom: 4px;
  padding: 10px;
}

div {
  border-radius: 4px;
  box-shadow: 0 2px 4px 0 gray;
  padding: 6px;
  margin: 6px;
}
```

```
.card {
  width: 70%;
  height: 90px;
}

.fixed {
  position: fixed;
  right: 10px;
  top: 10px;
  background: coral;
}

.float {
  float: left;
  height: 40px;
  margin: 10px;
  background: aquamarine;
}

.contain-layout {
  contain: layout;
}
```

The first card has layout containment applied, and its layout is isolated from the rest of the page. We can reuse this card in other places on the page without worrying about layout recalculation of the other elements. If floats overlap the card bounds, elements on the rest of the page are not affected. When the browser recalculates the containing element's subtree, only that element is recalculated. Nothing outside of the contained element needs to be recalculated. Additionally, the fixed box uses the card as a layout container to position itself.

The second and third cards have no containment. The layout context for the fixed box in the second card is the root element so the fixed box is positioned in the top right corner of the page. A float overlaps the second card's bounds causing the third card to have unexpected layout shift that's visible in the positioning of the `<h2>` element. When recalculation occurs, it is not limited to a container. This impacts performance and interferes with the rest of the page layout.

Style containment

Style containment scopes counters and quotes to the contained element. For CSS counters, the `counter-increment` and `counter-set` properties are scoped to the element as if the element is at the root of the document.

Containment and counters

The example below takes a look at how counters work when style containment is applied:

```
<ul>
  <li>Item A</li>
  <li>Item B</li>
  <li class="container">Item C</li>
  <li>Item D</li>
  <li>Item E</li>
</ul>
```

```
body {
  counter-reset: list-items;
}

li::before {
  counter-increment: list-items;
  content: counter(list-items) ": ";
}

.container {
  contain: style;
}
```

Without containment, the counter would increment from 1 to 5 for each list item. Style containment causes the `counter-increment` property to be scoped to the element's subtree and the counter begins again at 1:

Containment and quotes

CSS quotes are similarly affected in that the `content` values relating to quotes are scoped to the element:

```
<!-- With style containment -->
<span class="open-quote">
  outer
  <span class="contain-style">
    <span class="open-quote">inner</span>
  </span>
</span>
<span class="close-quote">close</span>
<br />
<!-- Without containment -->
<span class="open-quote">
  outer
```

```
  <span>
    <span class="open-quote">inner</span>
  </span>
</span>
<span class="close-quote">close</span>
```

```
body {
  quotes: "[" "]" "‹" "›";
}
.open-quote::before {
  content: open-quote;
}

.close-quote::after {
  content: close-quote;
}

.contain-style {
  contain: style;
}
```

Because of containment, the first closing quote ignores the inner span and uses the outer span's closing quote instead:

See also

- CSS containment
- CSS container queries
- CSS `content-visibility` property
- CSS `position` property

contain-intrinsic-block-size CSS property

The **`contain-intrinsic-block-size`** CSS logical property defines the block size of an element that a browser can use for layout when the element is subject to size containment.

Block size is the size of an element in the dimension perpendicular to the flow of text within a line. In a horizontal writing mode like standard English, block size is the vertical dimension (height); in a vertical writing mode, block size is the horizontal dimension.

Syntax

```
/* Keyword value */
contain-intrinsic-block-size: none;

/* <length> values */
contain-intrinsic-block-size: 1000px;
contain-intrinsic-block-size: 10rem;

/* auto <length> */
contain-intrinsic-block-size: auto 300px;

/* Global values */
contain-intrinsic-block-size: inherit;
contain-intrinsic-block-size: initial;
contain-intrinsic-block-size: revert;
contain-intrinsic-block-size: revert-layer;
contain-intrinsic-block-size: unset;
```

Values

The following values can be specified for the intrinsic block size of an element:

`none`

The element has no intrinsic block size.

`<length>`

The element has the specified block size, expressed using the (`<length>`) data type.

`auto <length>`

When the element is in size containment and skipping its contents (for example, when it is offscreen and `content-visibility: auto` is set) the block size is remembered from the actual size of the element when it was last able to render its child elements. If the element has never rendered its child elements and hence has no remembered value for the normally rendered element size, or if it is not skipping its contents, the block size is the specified `<length>`.

Description

The property is commonly applied alongside elements that can trigger size containment, such as `contain: size` and `content-visibility`.

Size containment allows a user agent to lay out an element as though it had a fixed size. This prevents unnecessary reflows by avoiding the re-rendering of child elements to determine the actual size (thereby improving user experience). By default, size containment treats elements as though they had no contents and may collapse the layout in the same way as if the contents had no width or height. The `contain-intrinsic-block-size` property allows authors to specify an appropriate value to be used as the block-size for layout.

The `auto <length>` value allows the block-size of an element to be stored if the element is ever "normally rendered" (with its child elements) and then used instead of the specified value when the element does not have any content. This allows offscreen elements with `content-visibility: auto` to benefit from size containment without developers having to be precise in their estimates of element size. The remembered value is not used if the child elements are being rendered; if size containment is enabled, the `<length>` value will be used.

Formal definition

Initial value	none
Applies to	elements for which size containment can apply
Inherited	no
Computed value	as specified, with <length>s values computed
Animation type	by computed value type

Formal syntax

```
contain-intrinsic-block-size =
  auto? [ none | <length [0,∞]> ]
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting the intrinsic block size

The HTML below defines an element "contained_element" that will be subject to size constraint, and which contains a child element.

```
<div id="contained_element">
  <div class="child_element"></div>
</div>
```

The CSS below sets the `content-visibility` of `contained_element` to `auto`, so if the element is hidden it will be size constrained. The intrinsic block size and inline size that are used when it is size constrained are set at the same time using `contain-intrinsic-block-size` and `contain-intrinsic-inline-size`, respectively.

```
#contained_element {
  border: 2px solid green;
  inline-size: 151px;
  content-visibility: auto;
  contain-intrinsic-inline-size: 152px;
  contain-intrinsic-block-size: 52px;
}
.child_element {
  border: 1px solid red;
  background: blue;
  block-size: 50px;
  inline-size: 150px;
}
```

See also

- content-visibility: the new CSS property that boosts your rendering performance (web.dev)
- `contain-intrinsic-inline-size`
- `contain-intrinsic-size`
- `contain-intrinsic-width`
- `contain-intrinsic-height`

contain-intrinsic-height CSS property

The **`contain-intrinsic-height`** CSS property sets the height of an element that a browser can use for layout when the element is subject to size containment.

Syntax

```
/* Keyword value */
contain-intrinsic-height: none;

/* <length> values */
contain-intrinsic-height: 1000px;
contain-intrinsic-height: 10rem;

/* auto <length> */
contain-intrinsic-height: auto 300px;

/* Global values */
contain-intrinsic-height: inherit;
contain-intrinsic-height: initial;
contain-intrinsic-height: revert;
contain-intrinsic-height: revert-layer;
contain-intrinsic-height: unset;
```

Values

The following values may be specified for an element.

`none`

The element has no intrinsic height.

`<length>`

The element has the specified height (`<length>`).

`auto <length>`

A remembered value of the "normally rendered" element height if one exists and the element is skipping its contents (for example, when it is offscreen); otherwise the specified `<length>`.

Description

The property is commonly applied alongside elements that can trigger size containment, such as `contain: size` and `content-visibility`, and may also be set using the `contain-intrinsic-size` shorthand property.

Size containment allows a user agent to lay out an element as though it had a fixed size, preventing unnecessary reflows by avoiding the re-rendering of child elements to determine the actual size (thereby improving user experience). By default, size containment treats elements as though they had no contents, and may collapse the layout in the same way as if the contents had no height (or width). The `contain-intrinsic-height` property allows authors to specify an appropriate value to be used as the height for layout.

The `auto <length>` value allows the height of the element to be stored if the element is ever "normally rendered" (with its child elements), and then used instead of the specified height when the element is skipping its contents. This allows offscreen elements with `content-visibility: auto` to benefit from size containment without developers having to be as precise in their estimates of element size. The remembered value is not used if the child elements are being rendered (if size containment is enabled, the `<length>` will be used).

Formal definition

Initial value	none
Applies to	elements for which size containment can apply

Inherited	no
Computed value	as specified, with <length>s values computed
Animation type	by computed value type

Formal syntax

```
contain-intrinsic-height =
  auto? [ none | <length [0,∞]> ]
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In addition to the example below, the `contain-intrinsic-size` page contains a live example that demonstrates the effect of modifying the intrinsic width and height.

Setting the intrinsic height

The HTML below defines an element "contained_element" that will be subject to size constraint, and which contains a child element.

```
<div id="contained_element">
  <div class="child_element"></div>
</div>
```

The CSS below sets the `content-visibility` of `contained_element` to `auto`, so if the element is hidden it will be size constrained. The width and height that are used when it is size constrained are set at the same time using `contain-intrinsic-width` and `contain-intrinsic-height`, respectively.

```
#contained_element {
  border: 2px solid green;
  width: 151px;
  content-visibility: auto;
  contain-intrinsic-width: 152px;
  contain-intrinsic-height: 52px;
}
.child_element {
  border: 1px solid red;
  background: blue;
  height: 50px;
  width: 150px;
}
```

See also

- content-visibility: the new CSS property that boosts your rendering performance (web.dev)
- `contain-intrinsic-size`
- `contain-intrinsic-width`
- `contain-intrinsic-block-size`
- `contain-intrinsic-inline-size`

contain-intrinsic-inline-size CSS property

The **`contain-intrinsic-inline-size`** CSS logical property defines the inline-size of an element that a browser can use for layout when the element is subject to size containment.

Inline-size is the size of the element in the dimension parallel to the flow of text within a line. In a horizontal writing mode like standard English, inline size is the horizontal dimension (width); for a vertical writing mode, inline size is the vertical dimension.

Syntax

```
/* Keyword value */
contain-intrinsic-inline-size: none;

/* <length> values */
contain-intrinsic-inline-size: 1000px;
contain-intrinsic-inline-size: 10rem;

/* auto <length> */
contain-intrinsic-inline-size: auto 300px;

/* Global values */
contain-intrinsic-inline-size: inherit;
contain-intrinsic-inline-size: initial;
contain-intrinsic-inline-size: revert;
contain-intrinsic-inline-size: revert-layer;
contain-intrinsic-inline-size: unset;
```

Values

The following values can be specified for the intrinsic inline size of an element:

`none`

> The element has no intrinsic inline-size.

`<length>`

> The element has the specified inline-size (`<length>`).

`auto <length>`

> When the element is in size containment and skipping its contents (for example, when it is offscreen and `content-visibility: auto` is set) the inline size is remembered from the actual size of the element when it was last able to render its child elements. If the element has never rendered its child elements and hence has no remembered value for the normally rendered element size, or if it is not skipping its contents, the inline size is the specified `<length>`.

Description

The property is commonly applied alongside elements that can trigger size containment, such as `contain: size` and `content-visibility`.

Size containment allows a user agent to lay out an element as though it had a fixed size, preventing unnecessary reflows by avoiding the re-rendering of child elements to determine the actual size (thereby improving user experience). By default, size containment treats elements as though they had no contents, and may collapse the layout in the same way as if the contents had no width or height. The `contain-intrinsic-inline-size` property allows authors to specify an appropriate value to be used as the inline-size for layout.

The `auto <length>` value allows the inline-size of the element to be stored if the element is ever "normally rendered" (with its child elements), and then used instead of the specified value when the element is skipping its contents. This allows offscreen elements with `content-visibility: auto` to benefit from size containment without developers having to be as precise in their estimates of element size. The remembered value is not used if the child elements are being rendered (if size containment is enabled, the `<length>` will be used).

Formal definition

Initial value	none
Applies to	elements for which size containment can apply
Inherited	no
Computed value	as specified, with <length>s values computed
Animation type	by computed value type

Formal syntax

```
contain-intrinsic-inline-size =
  auto? [ none | <length [0,∞]> ]
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting the intrinsic inline size

The HTML below defines an element "contained_element" that will be subject to size constraint, and which contains a child element.

```
<div id="contained_element">
  <div class="child_element"></div>
</div>
```

The CSS below sets the `content-visibility` of `contained_element` to `auto`, so if the element is hidden it will be size constrained. The intrinsic block size and inline size that are used when it is size constrained are set at the same time using `contain-intrinsic-block-size` and `contain-intrinsic-inline-size`, respectively.

```
#contained_element {
  border: 2px solid green;
  inline-size: 151px;
  content-visibility: auto;
  contain-intrinsic-inline-size: 152px;
  contain-intrinsic-block-size: 52px;
}
.child_element {
  border: 1px solid red;
  background: blue;
  block-size: 50px;
  inline-size: 150px;
}
```

See also

- content-visibility: the new CSS property that boosts your rendering performance (web.dev)
- `contain-intrinsic-block-size`
- `contain-intrinsic-size`
- `contain-intrinsic-width`
- `contain-intrinsic-height`

contain-intrinsic-size CSS property

The **`contain-intrinsic-size`** CSS shorthand property sets the size of an element that a browser will use for layout when the element is subject to size containment.

Constituent properties

This property is a shorthand for the following CSS properties:

- `contain-intrinsic-width`
- `contain-intrinsic-height`

Syntax

```
/* Keyword value */
contain-intrinsic-size: none;

/* <length> values */
contain-intrinsic-size: 1000px;
contain-intrinsic-size: 10rem;

/* width | height */
contain-intrinsic-size: 1000px 1.5em;

/* auto <length> */
contain-intrinsic-size: auto 300px;
contain-intrinsic-size: auto none;

/* auto width | auto height */
contain-intrinsic-size: auto 300px auto 4rem;

/* Global values */
contain-intrinsic-size: inherit;
contain-intrinsic-size: initial;
contain-intrinsic-size: revert;
contain-intrinsic-size: revert-layer;
contain-intrinsic-size: unset;
```

Values

The following values may be specified for the `contain-intrinsic-size` property:

`none`

The element has no intrinsic size in the given dimension(s).

`<length>`

The element has the specified `<length>` in the given dimension(s).

`auto [<length> | none]`

A remembered value of the "normally rendered" element size if one exists and the element is skipping its contents (for example, when it is offscreen); otherwise the specified `<length>`. The `none` keyword may be used in place of `<length>` where `0px` fixed lengths behave differently than `none` (such as in multi column, or grid layouts).

If one value is provided as a keyword, a length or an `auto [<length> | none]` pair, it applies to both width and height.

Two length values may be specified, which apply to the width and height in that order. If two `auto [<length> | none]` pairs are specified, the first pair applies to the width, and the second to the height.

Description

The property is commonly applied alongside elements that can trigger size containment, such as `contain: size` and `content-visibility`.

Size containment allows a user agent to lay out an element as though it had a fixed size, preventing unnecessary reflows by avoiding the re-rendering of child elements to determine the actual size (thereby improving user experience). By default, size containment treats elements as though they had no contents, and may collapse the layout in the same way as if the contents had no width or height. The `contain-intrinsic-size` property allows authors to specify an appropriate value to be used as the size for layout.

The `auto <length>` value allows the size of the element to be stored if the element is ever "normally rendered" (with its child elements), and then used instead of the specified length when the element is skipping its contents. This allows offscreen elements with `content-visibility: auto` to benefit from size containment without developers having to be as precise in their estimates of element size. The remembered value is not used if the child elements are being rendered (if size containment is enabled, the `<length>` will be used).

In grid and multi column layouts, an explicit size is treated differently than implicit content-based height. Elements might lay out substantially differently than it would have were it simply filled with content up to that height. The `auto none` value allows the element to fall back to `contain-intrinsic-size: none` if no remembered value exists, which will allow the element to be laid out as though it had no contents. This is almost always preferred to setting 0px as the intrinsic size in grid and multi column layouts, where contained elements may overflow their parents and can result in unexpected page layout.

Formal definition

Initial value	as each of the properties of the shorthand: • `contain-intrinsic-width`: none • `contain-intrinsic-height`: none
Applies to	elements for which size containment can apply
Inherited	no
Percentages	as each of the properties of the shorthand: • `contain-intrinsic-width`: no • `contain-intrinsic-height`: no
Computed value	as each of the properties of the shorthand: • `contain-intrinsic-width`: as specified, with <length>s values computed • `contain-intrinsic-height`: as specified, with <length>s values computed
Animation type	as each of the properties of the shorthand: • `contain-intrinsic-width`: by computed value type • `contain-intrinsic-height`: by computed value type

Formal syntax

```
contain-intrinsic-size =
  [ auto? [ none | <length [0,∞]> ] ]{1,2}
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using auto value pairs for intrinsic size

This example demonstrates `contain-intrinsic-size: auto <length>` and `contain-intrinsic-size: auto none`, using a layout where there are many elements displayed vertically that have both accurate and incorrect intrinsic size estimations. Using `content-visibility: auto` skips rendering elements when they are offscreen, so this property is a good candidate to combine with `contain-intrinsic-size` to improve rendering performance and minimize reflows.

The `contain-intrinsic-size: auto 500px` value pair tells the browser to use 500px as a kind of 'placeholder' size (width and height) for the element when it is offscreen and the page is being laid out. When the user scrolls to the element and it needs to be displayed, the browser will calculate the actual size of the element and its contents. If there is a difference between the placeholder and calculated size this might force a new layout, with accompanying changes to the sidebar position.

Once the browser has actual size information for the element, it will remember this size when the element scrolls offscreen again, and use the remembered size for layout calculations instead of the placeholder value. The benefit is that the browser does not need to repeatedly render the element contents to calculate its size and is especially useful when the contents are complex or depend on network resources or JavaScript.

HTML

```
<div id="container">
  <div id="auto-length-note">
    <p>
      Your browser does not support
      <code>contain-intrinsic-size: auto &amp;lt;length&amp;gt;</code>.
    </p>
  </div>
  <div class="auto-length">
    <p>Item one</p>
  </div>
  <div class="auto-length">
```

```
    <p>Item two</p>
  </div>
  <div class="auto-length large-intrinsic-size">
    <p class="small">Item three</p>
  </div>
  <div class="auto-length large-intrinsic-size">
    <p class="small">Item four</p>
  </div>
  <div id="auto-none-note">
    <p>
      Your browser does not support
      <code>contain-intrinsic-size: auto none</code>.
    </p>
  </div>
  <div class="auto-length none">
    <p>Item five</p>
  </div>
  <div class="auto-length none">
    <p>Item six</p>
  </div>
</div>
```

CSS

```
div,
p {
  padding: 1rem;
  margin-bottom: 1rem;
  font-size: 2rem;
  font-family: sans-serif;
}

code {
  background-color: lightgray;
  padding: 0.25rem;
  border-radius: 0.25rem;
}

#container {
  width: 90%;
  height: 80%;
}

.auto-length,
.auto-length.none {
  display: none;
}

#auto-length-note,
#auto-none-note {
  display: block;
  padding: 0;
}

#auto-length-note p,
#auto-none-note p {
  padding: 0.5rem;
  width: 100%;
  height: max-content;
  font-size: 1rem;
  line-height: 1.5rem;
  background-color: tomato;
}

@supports (contain-intrinsic-size: auto none) {
  .auto-length.none {
    display: block;
  }
  #auto-none-note {
    display: none;
  }
}
@supports (contain-intrinsic-size: auto 500px) {
  .auto-length {
    display: block;
  }
  #auto-length-note {
    display: none;
  }
}
```

```
p {
  height: 500px;
  width: 500px;
  border: 4px dotted;
  background: lightblue;
}

.auto-length {
  content-visibility: auto;
  contain-intrinsic-size: auto 500px;
  background-color: linen;
  outline: 4px dotted blue;
}

.large-intrinsic-size {
  /* Setting an inaccurate intrinsic size for the element */
  contain-intrinsic-size: auto 5000px;
  background-color: lightgray;
  outline: 4px dotted red;
}
```

```
.small {
  /* This element is a lot smaller than expected */
  height: 100px;
  width: 100px;
}

.none {
  background-color: papayawhip;
  contain-intrinsic-size: auto none;
  outline: 4px dotted red;
}
```

Result

- The first two boxes have an intrinsic size that matches their actual size, so as they flow into view, the layout is recalculated but we see no change in the scrollbar or the scroll position.
- The third and fourth boxes have a huge intrinsic size, so the initial layout that the browser calculated is far too big, and we've made these boxes smaller so that it's obvious when you've reached a point that forces a drastic layout change.

  When the third and fourth boxes scroll into view, the size is recalculated, making the box and its parent less tall. The effect is that the scroller jumps down the page (we've effectively scrolled further through the box than we'd estimated) and the scroller is longer, because the entire page is less tall than we'd estimated.
- The last boxes have `auto none`, so they have zero estimated size. When they scroll into view the size of the element and its parent are recalculated to be much larger, so the scroller decreases in size and moves up the bar.

After scrolling all the way to the bottom you can subsequently scroll up and down smoothly, because using `content-visibility: auto` saves the actual rendered size of the element for next time it is displayed.

Setting the intrinsic size

This example provides selection lists that can be used to modify `contain-intrinsic-size`, `content-visibility` and `contain` on an element in order to observe the effect of the different settings.

CSS

```
#contained_element {
  border: 2px solid green;
  width: 120px;
}
.child_element {
  border: 1px solid red;
  background: blue;
  height: 50px;
  width: 150px;
}
```

JavaScript

The code below adds styles to, and removes styles from, the containing element based on the selected options.

```
const containedElement = document.querySelector("#contained_element");
const intrinsicSizeSelector = document.querySelector(
  "#contain_intrinsic_size_selector",
);
const containSelector = document.querySelector("#contain_selector");
const contentVisibilitySelector = document.querySelector(
  "#content_visibility_selector",
);

containedElement.style["contain-intrinsic-size"] =
  intrinsicSizeSelector.options[intrinsicSizeSelector.selectedIndex].text;
containedElement.style["contain"] =
  containSelector.options[containSelector.selectedIndex].text;
containedElement.style["content-visibility"] =
  contentVisibilitySelector.options[
    contentVisibilitySelector.selectedIndex
  ].text;

intrinsicSizeSelector.addEventListener("change", () => {
  containedElement.style["contain-intrinsic-size"] =
    intrinsicSizeSelector.options[intrinsicSizeSelector.selectedIndex].text;
});

containSelector.addEventListener("change", () => {
  containedElement.style["contain"] =
    containSelector.options[containSelector.selectedIndex].text;
});

contentVisibilitySelector.addEventListener("change", () => {
  containedElement.style["content-visibility"] =
    contentVisibilitySelector.options[
      contentVisibilitySelector.selectedIndex
    ].text;
});
```

HTML

The HTML defines two buttons, a container element that is subject to containment via the `content-visibility` property.

```
<p>
  <label for="contain_intrinsic_size_selector">contain-intrinsic-size:</label>
  <select id="contain_intrinsic_size_selector">
    <option>none</option>
    <option>40px 130px</option>
    <option>auto 40px auto 130px</option></select
  >;<br />

  <label for="contain_selector">contain:</label>
  <select id="contain_selector">
    <option>none</option>
    <option>size</option>
    <option>strict</option></select
  >;<br />
```

```
  <label for="content_visibility_selector">content-visibility:</label>
  <select id="content_visibility_selector">
    <option>visible</option>
    <option>auto</option>
    <option>hidden</option></select
  >;
</p>

<div id="contained_element">
  <div class="child_element"></div>
</div>
```

Result

Use the selectors to apply the given styles to the containing `div` element. Note that when `content-visibility` is `visible` or `auto`, changing `contain-intrinsic-size` makes no difference. However if the content is hidden, having a `contain-intrinsic-size` of `none` collapses the parent element as though its child element had no size.

See also

- `contain-intrinsic-block-size`
- `contain-intrinsic-inline-size`
- Using CSS containment
- CSS containment module
- `content-visibility`: the new CSS property that boosts your rendering performance via web.dev (2020)

contain-intrinsic-width CSS property

The **`contain-intrinsic-width`** CSS property sets the width of an element that a browser will use for layout when the element is subject to size containment.

Syntax

```
/* Keyword value */
contain-intrinsic-width: none;

/* <length> values */
contain-intrinsic-width: 1000px;
contain-intrinsic-width: 10rem;

/* auto <length> */
contain-intrinsic-width: auto 300px;

/* Global values */
contain-intrinsic-width: inherit;
contain-intrinsic-width: initial;
contain-intrinsic-width: revert;
contain-intrinsic-width: revert-layer;
contain-intrinsic-width: unset;
```

Values

The following values may be specified for an element.

`none`

The element has no intrinsic width.

`<length>`

The element has the specified width (`<length>`).

`auto <length>`

A remembered value of the "normally rendered" element width if one exists and the element is skipping its contents (for example, when it is offscreen); otherwise the specified `<length>`.

Description

The property is commonly applied alongside elements that can trigger size containment, such as `contain: size` and `content-visibility`, and may also be set using the `contain-intrinsic-size` shorthand property.

Size containment allows a user agent to lay out an element as though it had a fixed size, preventing unnecessary reflows by avoiding the re-rendering of child elements to determine the actual size (thereby improving user experience). By default, size containment treats elements as though they had no contents, and may collapse the layout in the same way as if the contents had no width or height. The `contain-intrinsic-width` property allows authors to specify an appropriate value to be used as the width for layout.

The `auto <length>` value allows the width of the element to be stored if the element is ever "normally rendered" (with its child elements), and then used instead of the specified width when the element is skipping its contents. This allows offscreen elements with `content-visibility: auto` to benefit from size containment without developers having to be as precise in their estimates of element size. The remembered value is not used if the child elements are being rendered (if size containment is enabled, the `<length>` will be used).

Formal definition

Initial value	none
Applies to	elements for which size containment can apply
Inherited	no
Computed value	as specified, with <length>s values computed
Animation type	by computed value type

Formal syntax

```
contain-intrinsic-width =
  auto? [ none | <length [0,∞]> ]
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In addition to the example below, the `contain-intrinsic-size` page contains a live example that demonstrates the effect of modifying the intrinsic width and height.

Setting the intrinsic width

The HTML below defines an element "contained_element" that will be subject to size constraint, and which contains a child element.

```
<div id="contained_element">
  <div class="child_element"></div>
</div>
```

The CSS below sets the `content-visibility` of `contained_element` to `auto`, so if the element is hidden it will be size constrained. The width and height that are used when it is size constrained are set at the same time using `contain-intrinsic-width` and `contain-intrinsic-height`, respectively.

```
#contained_element {
  border: 2px solid green;
  width: 151px;
  content-visibility: auto;
  contain-intrinsic-width: 152px;
  contain-intrinsic-height: 52px;
}
.child_element {
  border: 1px solid red;
  background: blue;
  height: 50px;
  width: 150px;
}
```

See also

- content-visibility: the new CSS property that boosts your rendering performance (web.dev)
- `contain-intrinsic-size`
- `contain-intrinsic-height`
- `contain-intrinsic-block-size`
- `contain-intrinsic-inline-size`

container CSS property

The **container** shorthand CSS property establishes the element as a query container and specifies the name and type of the containment context used in a container query.

Constituent properties

This property is a shorthand for the following CSS properties:

- `container-name`
- `container-type`

Syntax

```
/* <container-name> */
container: my-layout;

/* <container-name> / <container-type> */
container: my-layout / size;

/* Global Values */
container: inherit;
container: initial;
container: revert;
container: revert-layer;
container: unset;
```

Values

`<container-name>`

A case-sensitive name for the containment context. More details on the syntax are covered in the `container-name` property page.

`<container-type>`

The type of containment context. More details on the syntax are covered in the `container-type` property page.

Formal definition

Initial value	as each of the properties of the shorthand: • `container-name`: `none` • `container-type`: `normal`
Applies to	all elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `container-name`: no • `container-type`: no
Computed value	as each of the properties of the shorthand: • `container-name`: `none` or an ordered list of identifiers • `container-type`: as specified
Animation type	as each of the properties of the shorthand: • `container-name`: Not animatable • `container-type`: a color

Formal syntax

```
container =
  <'container-name'> [ / <'container-type'> ]?

<container-name> =
  none            |
  <custom-ident>+

<container-type> =
  normal                                  |
  [ [ size | inline-size ] || scroll-state ]
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Establishing inline size containment

Given the following HTML example which is a card component with an image, a title, and some text:

```
<div class="post">
  <div class="card">
    <h2>Card title</h2>
    <p>Card content</p>
  </div>
</div>
```

The explicit way to create a container context is to declare a `container-type` with an optional `container-name`:

```
.post {
  container-type: inline-size;
  container-name: sidebar;
}
```

The `container` shorthand is intended to make this definable in a single declaration:

```
.post {
  container: sidebar / inline-size;
}
```

You can then target that container by name using the `@container` at-rule:

```
@container sidebar (width >= 400px) {
  /* <stylesheet> */
}
```

See also

- CSS container queries
- Using container size and style queries
- `@container` at-rule
- CSS `contain` property
- CSS `container-type` property
- CSS `container-name` property
- CSS `content-visibility` property

container-name CSS property

The **container-name** CSS property specifies a list of query container names used by the @container at-rule in a container query.

Syntax

```
container-name: none;

/* A single name */
container-name: my-layout;

/* Multiple names */
container-name: my-page-layout my-component-library;

/* Global Values */
container-name: inherit;
container-name: initial;
container-name: revert;
container-name: revert-layer;
container-name: unset;
```

Values

This property is specified as a space-separated list of `<custom-ident>` values or the keyword `none`:

`none`

The default value. The query container has no name.

`<custom-ident>`

A case-sensitive string that is used to identify the container. The following conditions apply:

- The name must not equal `or`, `and`, `not`, or `default`.
- The name value must not be in quotes.
- The dashed ident intended to denote author-defined identifiers (e.g., `--container-name`) is permitted.
- A list of multiple names separated by a space is allowed.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	none or an ordered list of identifiers
Animation type	Not animatable

Formal syntax

```
container-name =
  none             |
  <custom-ident>+
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Description

With no name specified, a container query will apply styles to elements based on attributes such as the size or scroll-state of the nearest ancestor with a containment context.

> **Note:** Size containers' descendants can be sized using container query length units.

When a containment context is given a name, it can be specifically targeted by setting that name on a `@container` at-rule.

It is possible to create a query container by assigning a `container-name` to an element, and then query only the existence of that name in the associated `@container` at-rule, with no query expression specified. These so-called **name-only container queries** enable selectively applying styles to elements based only on whether they have an ancestor with a specific `container-name` set.

Examples

Using a container name

Given the following HTML example which is a card component with a title and some text:

```
<div class="card">
  <div class="post-meta">
    <h2>Card title</h2>
    <p>My post details.</p>
  </div>
  <div class="post-excerpt">
    <p>
      A preview of my <a href="https://example.com">blog post</a> about cats.
    </p>
  </div>
</div>
```

To create a containment context, add the `container-type` property to an element in CSS. The following example creates two containment contexts, one for the card meta information and one for the post excerpt:

> **Note:** A shorthand syntax for these declarations are described in the `container` page.

```
.post-meta {
  container-type: inline-size;
}

.post-excerpt {
  container-type: inline-size;
  container-name: excerpt;
}
```

Writing a container query via the `@container` at-rule will apply styles to the elements of the container when the query evaluates to true. The following example has two container queries, one that will apply only to the contents of the `.post-excerpt` element and one that will apply to both the `.post-meta` and `.post-excerpt` contents:

```
@container excerpt (width >= 400px) {
  p {
    visibility: hidden;
  }
}

@container (width >= 400px) {
  p {
    font-size: 2rem;
  }
}
```

For more information on writing container queries, see the CSS Container Queries page.

Using multiple container names

You can also provide multiple names to a container context separated by a space:

```
.post-meta {
  container-type: inline-size;
  container-name: meta card;
}
```

This will allow you to target the container using either name in the `@container` at-rule. This is useful if you want to target the same container with multiple container queries where either condition could be true:

```
@container meta (width <= 500px) {
  p {
    visibility: hidden;
  }
}

@container card (width <= 200px) {
```

```
  h2 {
    font-size: 1.5em;
  }
}
```

Using a name-only container query

This example demonstrates how to use a name-only container query.

HTML

We include a `<div>` with an `id` of `container`, and three `<p>` elements, two inside the container, and one outside the container:

```
<div id="container">
  <p>I'm in the container.</p>
  <p>I'm also in the container.</p>
</div>
<p>I'm not in the container.</p>
```

CSS

We assign a name to the container:

```
#container {
  container-name: my-container;
}
```

We can then selectively apply styles only to elements inside containers with that name set, as shown in the next snippet.

```
@container my-container {
  p {
    background-color: lime;
    font-size: 1.3rem;
    width: 50vw;
    padding: 0.5rem;
    font-family: sans-serif;
  }
}
```

Result

The specified styles should be applied only to the first and second `<p>` elements, but not to the third.

See also

- CSS container queries
- Using container size and style queries
- Using container scroll-state queries
- `@container` at-rule
- CSS `container` shorthand property
- CSS `container-type` property
- CSS `content-visibility` property

container-type CSS property

The **container-type** CSS property specifies the type of container context used in a container query.

Syntax

```
/* Keyword values */
container-type: normal;
container-type: size;
container-type: inline-size;
container-type: scroll-state;
container-type: anchored;

/* Two values */
container-type: size scroll-state;

/* Global Values */
container-type: inherit;
container-type: initial;
container-type: revert;
container-type: revert-layer;
container-type: unset;
```

Values

This property is specified as one or two keyword values from the following list. In the two-value case, one must be `scroll-state` and the other must be `inline-size` or `size`:

`anchored`

> Establishes a query container for anchored container queries on the container. In this case, the size of the element is not computed in isolation; no containment is applied.

`inline-size`

> Establishes a query container for dimensional queries on the inline axis of the container. Applies style and inline-size containment to the element. The inline size of the element can be computed in isolation, ignoring the child elements (see Using CSS containment).

`normal`

> Default value. The element is not a query container for any container size, scroll-state, or anchored queries, but can be used as a query container for container style queries and name-only container queries.

`scroll-state`

> Establishes a query container for scroll-state queries on the container. In this case, the size of the element is not computed in isolation; no containment is applied.

`size`

Establishes a query container for container size queries in both the inline and block dimensions. Applies style and size containment to the element. Size containment is applied to the element in both the inline and block directions. The size of the element can be computed in isolation, ignoring the child elements.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	a color

Formal syntax

```
container-type =
  normal                                |
  [ [ size | inline-size ] || scroll-state ]
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Description

Container queries allow you to selectively apply styles inside a container based on conditional queries performed on the container. The `@container` at-rule is used to specify the tests performed on a container, and the rules that will apply to the container's contents if the query returns `true`.

Certain types of container query can only be performed on elements with specific `container-type` property values set, which establish specific container contexts on those containers:

- Size: Enable selectively applying CSS rules to a container's children based on a general size or inline size condition such as a maximum or minimum dimension, aspect ratio, or orientation.
- Scroll-state: Enable selectively applying CSS rules to a container's children based on a scroll-state condition such as whether the container is a scroll container that is partially scrolled or whether the container is a snap target that is going to be snapped to its scroll snap container.
- Anchored: Enable selectively applying CSS rules to a container's children based on whether the container is anchor-positioned and has a position-try fallback option applied to it.

If a `container-type` is not set on a container, the element is not a query container for container size, scroll-state, or anchored queries, but can still be used as a query container for container style queries and name-only container queries.

Container size queries

Container size queries allow you to selectively apply CSS rules to a container's descendants based on a size condition such as a maximum or minimum dimension, aspect ratio, or orientation.

Size containers additionally have size containment applied to them — this turns off the ability of an element to get size information from its contents, which is important for container queries to avoid infinite loops. If this were not the case, a CSS rule inside a container query could change the content size, which in turn could make the query evaluate to false and change the parent element's size, which in turn could change the content size and flip the query back to true, and so on. This sequence would then repeat itself in an endless loop.

The container size has to be set by context, such as block-level elements that stretch to the full width of their parent, or explicitly defined. If a contextual or explicit size is not available, elements with size containment will collapse.

> **Note:** Size containers' descendants can be sized using container query length units.

Container scroll-state queries

Container scroll-state queries allow you to selectively apply CSS rules to a container's children based on a scroll-state condition such as:

- Whether the container's contents are partially scrolled.
- Whether the container is a snap target that is going to be snapped to a scroll snap container.
- Whether the container is positioned via `position: sticky` and stuck to a boundary of a scrolling container.

In the first case, the queried container is the scroll container itself. In the other two cases the queried container is an element that is affected by the scroll position of an ancestor scroll container.

Anchored container queries

Anchored container queries allow you to selectively apply CSS rules to the descendants of an anchor-positioned container when it has a position-try fallback active on it, as specified via the `position-try-fallbacks` property.

For example, you might have an anchor-positioned tooltip element that is positioned above its anchor by default via a `position-area` value of `top`, but has a `position-try-fallbacks` value of `flip-block` specified. This will cause the tooltip to flip in the block direction to the bottom of its anchor when it starts to overflow the top of the viewport. If we set `container-type: anchored` on it, we can detect when the position-try fallback is applied via a `@container` at-rule and apply CSS as a result.

```
.tooltip {
  position: absolute;
  position-anchor: --my-anchor;
  position-area: top;
  position-try-fallbacks: flip-block;
  container-type: anchored;
}
```

Examples

Establishing inline size containment

Given the following HTML example which is a card component with an image, a title, and some text:

```
<div class="container">
  <div class="card">
    <h3>Normal card</h3>
    <div class="content">
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </div>
  </div>
</div>

<div class="container wide">
  <div class="card">
    <h3>Wider card</h3>
    <div class="content">
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
```

```
      tempor incididunt ut labore et dolore magna aliqua.
    </div>
  </div>
</div>
```

To create an inline size container context, add the `container-type` property to an element with a value of `inline-size`:

```
.container {
  container-type: inline-size;
  width: 300px;
  height: 120px;
}

.wide {
  width: 500px;
}
```

```
h3 {
  height: 2rem;
  margin: 0.5rem;
}

.card {
  height: 100%;
}

.content {
  background-color: wheat;
  height: 100%;
}

.container {
  margin: 1rem;
  border: 2px dashed red;
  overflow: hidden;
}
```

Writing a container query via the `@container` at-rule will apply styles to the elements of the container when it is wider than `400px`:

```
@container (width > 400px) {
  .card {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
}
```

See also

- CSS container queries
- Using container size and style queries
- Using container scroll-state queries
- Using anchored container queries
- `@container` at-rule
- CSS `container` shorthand property
- CSS `container-name` property
- CSS `content-visibility` property

content CSS property

The **`content`** CSS property replaces content with a generated value. It can be used to define what is rendered inside an element or pseudo-element. For elements, the `content` property specifies whether the element renders normally (`normal` or `none`) or is replaced with an image (and associated "alt" text). For pseudo-elements and margin boxes, `content` defines the content as images, text, both, or none, which determines whether the element renders at all.

Objects inserted using the `content` property are **anonymous replaced elements**.

Try it

```
.topic-games::before {
  content: "🎮 " / "games";
}

.topic-weather::before {
  content: "🌥️ " / "cloudy";
}

.topic-hot::before {
  content: url("/shared-assets/images/examples/fire.png") / "On fire";
  margin-right: 6px;
}
```

```
<p class="topic-games">Game News: A new inFamous is not planned</p>

<p class="topic-weather">
  Weather for Today: Heat, violent storms and twisters
</p>

<p class="topic-hot">Trending Article: Must-watch videos of the week</p>
```

Syntax

```
/* Keywords that cannot be combined with other values */
content: normal;
content: none;

/* <content-replacement>: <image> values */
content: url("http://www.example.com/test.png");
content: linear-gradient(#e66465, #9198e5);
content: image-set("image1x.png" 1x, "image2x.png" 2x);
```

```
/* speech output: alternative text after a "/"  */
content: url("../img/test.png") / "This is the alt text";
content: counter(chapter) / "Chapter " counter(chapter);

/* <string> value */
content: "unparsed text";

/* <counter> values, optionally with <list-style-type> */
content: counter(chapter_counter);
content: counter(chapter_counter, upper-roman);
content: counters(section_counter, ".");
content: counters(section_counter, ".", decimal-leading-zero);

/* attr() value linked to the HTML attribute value */
content: attr(href);

/* <quote> values */
content: open-quote;
content: close-quote;
content: no-open-quote;
content: no-close-quote;

/* <content-list>: a list of content values.
Several values can be used simultaneously */
content: "prefix" url("http://www.example.com/test.png");
content: "prefix" url("/img/test.png") "suffix" / "Alt text";
content: open-quote counter(chapter_counter);

/* Global values */
content: inherit;
content: initial;
content: revert;
content: revert-layer;
content: unset;
```

Values

The value can be:

- One of two keywords: `none` or `normal`. `normal` is the default property value.
- `<content-replacement>` when replacing a DOM node. `<content-replacement>` is always an `<image>`.
- A `<content-list>` when replacing pseudo-elements and margin boxes. A `<content-list>` is a list of one or more anonymous inline boxes appearing in the order specified. Each `<content-list>` item is of type `<string>`, `<image>`, `<counter>`, `<quote>`, `<target>`, or `<leader()>`.
- An optional alternative text value that can include `<string>`, `<counter>`, or `attr()` function values, preceded by a slash (`/`).

The keywords and data types mentioned above are described in more detail below:

`none`

When applied to a pseudo-element, the pseudo-element is not generated. When applied to an element, the value has no effect.

`normal`

For the `::before` and `::after` pseudo-elements, this value computes to `none` . For other pseudo-elements such as `::marker`, `::placeholder`, or `::file-selector-button`, it produces the element's initial (or normal) content. For regular elements or page margin boxes, it computes to the element's descendants. This is the default value.

`<string>`

A sequence of characters enclosed in matching single or double quotes. Multiple string values will be concatenated (there is no concatenation operator in CSS).

`<image>`

An `<image>`, representing an image to display. This can be equal to a `<url>`, `image-set()`, or `<gradient>` data type, or a part of the webpage itself, defined by the `element()` function.

`<counter>`

The `<counter>` value is a CSS counter, generally a number produced by computations defined by `counter-reset` and `counter-increment` properties. It can be displayed using either the `counter()` or `counters()` function.

`counter()`

The `counter()` function has two forms: 'counter(*name*)' or 'counter(*name*, style)'. The generated text is the value of the innermost counter of the given name in scope at the given pseudo-element. It is formatted in the specified `list-style-type` (`decimal` by default).

`counters()`

The `counters()` function also has two forms: 'counters(*name*, *string*)' or 'counters(*name*, *string*, *style*)'. The generated text is the value of all counters with the given name in scope at the given pseudo-element, from outermost to innermost, separated by the specified string. The counters are rendered in the indicated `list-style-type` (`decimal` by default).

`<quote>`

The `<quote>` data type includes language- and position-dependent keywords:

`open-quote` and `close-quote`

These values are replaced by the appropriate string from the `quotes` property.

`no-open-quote` and `no-close-quote`

Introduces no content, but increments (decrements) the level of nesting for quotes.

`<target>`

The `<target>` data type includes three target functions, `<target-counter()>`, `<target-counters()>`, and `<target-text()>` that create cross-references obtained from the target end of a link. See Formal syntax.

`<leader()>`

The `<leader()>` data type includes a leader function: `leader( <leader-type> )`. This function accepts the keyword values `dotted`, `solid`, or `space` (equal to `leader(".")`, `leader("_")`, and `leader(" ")`, respectively), or a `<string>` as a parameter. When supported and used as a value for `content`, the leader-type provided will be inserted as a repeating pattern, visually connecting content across a horizontal line.

`attr(x)`

The `attr(x)` CSS function retrieves the value of an attribute of the selected element, or the pseudo-element's originating element. The value of the element's attribute `x` is an unparsed string representing the attribute name. If there is no attribute `x`, an empty string is returned. The case sensitivity of the attribute name parameter depends on the document language.

alternative text: `/ <string> | <counter> | attr()`

Alternative text may be specified for an image or any `<content-list>` items, by appending a forward slash and then a combination of strings, counters, and `attr()` functions. The alternative text is intended for speech output by screen-readers, but may also be displayed in some browsers.

Formal definition

Initial value	`normal`
Applies to	All elements, tree-abiding pseudo-elements, and page margin boxes
Inherited	no
Computed value	On elements, always computes to `normal`. On `::before` and `::after`, if `normal` is specified, computes to `none`. Otherwise, for URI values, the absolute URI; for `attr()` values, the resulting string; for other keywords, as specified.
Animation type	discrete

Formal syntax

```
content =
  normal                                          |
  none                                            |
  [ <content-replacement> | <content-list> ] [ / <alt-text> ]?  |
  <element()>

<content-replacement> =
  <image>

<content-list> =
  [ <string> | <image> | <attr()> | contents | <quote> | <leader()> | <target> | <string()> | <content()> | <counter> | <symbolic-counter-style> ]+

<alt-text> =
  [ <string> | <counter> | <attr()> ]+

<element()> =
  element( <id-selector> )

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<attr()> =
  attr( <attr-name> <attr-type>? , <declaration-value>? )

<quote> =
  open-quote      |
  close-quote     |
  no-open-quote   |
  no-close-quote

<leader()> =
  leader( <leader-type> )

<target> =
  <target-counter()>   |
  <target-counters()>  |
  <target-text()>

<string()> =
  string( <custom-ident> , [ first | start | last | first-except ]? )

<content()> =
  content( [ text | before | after | first-letter | marker ]? )

<counter> =
  <counter()>   |
  <counters()>

<id-selector> =
  <hash-token>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<attr-name> =
  [ <ident-token>? '|' ]? <ident-token>

<attr-type> =
  type( <syntax> )  |
  raw-string        |
  number            |
  <attr-unit>

<leader-type> =
  dotted    |
  solid     |
  space     |
  <string>

<target-counter()> =
  target-counter( [ <string> | <url> ] , <custom-ident> , <counter-style>? )
```

```
<target-counters()> =
  target-counters( [ <string> | <url> ] , <custom-ident> , <string> , <counter-style>? )

<target-text()> =
  target-text( [ <string> | <url> ] , [ content | before | after | first-letter ]? )

<counter()> =
  counter( <counter-name> , <counter-style>? )

<counters()> =
  counters( <counter-name> , <string> , <counter-style>? )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<syntax> =
  '*'                                                 |
  <syntax-component> [ <syntax-combinator> <syntax-component> ]*  |
  <syntax-string>

<attr-unit> =
  <custom-ident>

<counter-style> =
  <counter-style-name>  |
  <symbols()>

<counter-name> =
  <custom-ident>

<syntax-component> =
  <syntax-single-component> <syntax-multiplier>?  |
  '<' transform-list '>'

<syntax-combinator> =
  '|'

<syntax-string> =
  <string>

<counter-style-name> =
  <custom-ident>

<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<syntax-single-component> =
  '<' <syntax-type-name> '>'  |
  <ident>

<syntax-multiplier> =
  '#'  |
  '+'

<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed

<syntax-type-name> =
  angle               |
  color               |
  custom-ident        |
  image               |
  integer             |
  length              |
  length-percentage   |
  number              |
  percentage          |
  resolution          |
  string              |
  time                |
  url                 |
  transform-function
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Generated Content Module Level 3, CSS Generated Content for Paged Media Module, CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, CSS Values and Units Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

CSS-generated content is not included in the DOM. Because of this, it will not be represented in the accessibility tree and certain assistive technology/browser combinations will not announce it. If the content conveys information that is critical to understanding the page's purpose, it is better to include it in the main document.

If inserted content is not decorative, check that the information is provided to assistive technologies and is also available when CSS is turned off.

- Accessibility support for CSS generated content – Tink (2015)
- WCAG, Guideline 1.3: Create content that can be presented in different ways
- Understanding Success Criterion 1.3.1 | W3C Understanding WCAG 2.0
- Failure of Success Criterion 1.3.1: inserting non-decorative generated content Techniques for WCAG 2.0

Examples

The first five examples create generated content on pseudo-elements. The last three are examples of element replacement.

Appending strings based on an element's class

This example inserts generated text after the text of elements that have a particular class name. The text is colored red.

HTML

```
<h2>Paperback Best Sellers</h2>
<ol>
  <li>Political Thriller</li>
  <li class="new-entry">Halloween Stories</li>
  <li>My Biography</li>
  <li class="new-entry">Vampire Romance</li>
</ol>
```

CSS

```
.new-entry::after {
  content: " New!"; /* The leading space creates separation
                       between the DOM node's content and the generated content
                       being added. */
  color: red;
}
```

Result

Quotes

This example inserts differently colored quotation marks around quotes.

HTML

```
<p>
  According to Sir Tim Berners-Lee,
  <q cite="http://www.w3.org/People/Berners-Lee/FAQ.html#Internet">
    I was lucky enough to invent the Web at the time when the Internet already
    existed - and had for a decade and a half.
  </q>
  We must understand that there is nothing fundamentally wrong with building on
  the contributions of others.
</p>
<p lang="fr-fr">
  Mais c'est Magritte qui a dit,
  <q lang="fr-fr"> Ceci n'est pas une pipe. </q>.
</p>
```

CSS

```
q {
  color: blue;
}

q::before,
q::after {
  font-size: larger;
  color: red;
  background: #cccccc;
}

q::before {
  content: open-quote;
}

q::after {
  content: close-quote;
}
```

Result

Note the type of quotes generated is based on the language. Browsers add open- and close-quotes before and after `<q>` elements by default, so the quotes in this example would appear without them being explicitly set. They could have been turned off by setting the respective `content` property values to `no-open-quote` and `no-close-quote`, or by setting them both to `none`. They can also be turned off by setting the `quotes` property to `none` instead.

Adding text to list item counters

This example combines a counter sandwiched between two `<string>`s prepended to all list items, creating a more detailed marker for list items (`<li>`) within unordered lists (`<ol>`).

HTML

```
<ol>
  <li>Dogs</li>
  <li>Cats</li>
  <li>
    Birds
    <ol>
      <li>Owls</li>
      <li>Ducks</li>
      <li>Flightless</li>
    </ol>
  </li>
  <li>Marsupials</li>
</ol>
```

CSS

```
ol {
  counter-reset: items;
  margin-left: 2em;
```

```
}
li {
  counter-increment: items;
}
li::marker {
  content: "item " counters(items, ".", numeric) ": ";
}
```

Result

The generated content on each list item's marker adds the text "item " as a prefix, including a space to separate the prefix from the counter, which is followed by ": ", a colon and an additional space. The `counters()` function defines a numeric `items` counter, in which the numbers of nested ordered lists have their numbers separated with a period (`.`) in most browsers.

Strings with attribute values

This example is useful for print stylesheets. It uses an attribute selector to select every fully qualified secure link, adding the value of the `href` attribute after the link text as the content of the `::after` pseudo-element.

HTML

```
<ul>
  <li><a href="https://mozilla.com">Mozilla</a></li>
  <li><a href="/">MDN</a></li>
  <li><a href="https://openwebdocs.org">OpenWebDocs</a></li>
</ul>
```

CSS

```
a[href^="https://"]::after {
  content: " (URL: " attr(href) ")";
  color: darkgreen;
}
```

Result

The generated content is the value of the `href` attribute, prepended by "URL: ", with a space, all in parentheses.

Adding an image with alternative text

This example inserts an image before all links. Two `content` values are provided. The later `content` value includes an image with alternative text that a screen reader can output as speech.

HTML

```
<a href="https://www.mozilla.org/en-US/">Mozilla Home Page</a>
```

CSS

The CSS to show the image and set the alternative text is shown below. This also sets the font and color for the content.

```
a::before {
  content: url("https://mozorg.cdn.mozilla.net/media/img/favicon.ico") /
    " MOZILLA: ";
}
```

Result

> **Note:** The alternative text value is exposed in the browser's accessibility tree. Refer to the See also section for browser-specific accessibility panels.

If using a screen reader, it should speak the word "MOZILLA" when it reaches the image. You can select the `::before` pseudo-element with your developer tools selection tool, and view the accessible name in the accessibility panel.

Including counters in alternative text

This example features a list of links to a set of book chapters, and shows how to use generated content to include a book icon and a counter before each one, with alternative text that includes the literal word "Chapter" in place of the icon. This results in the word "chapter" and the chapter number preceding the text in each link's accessible name, which will be announced to screen reader users when the link receives focus.

HTML

We include a heading followed by an ordered list of chapter title links using `<ol>`, `<li>`, and `<a>` elements.

```
<h2>Chapter list</h2>
<ol>
  <li><a href="#">A stranger calls</a></li>
  <li><a href="#">Two owls</a></li>
  <li><a href="#">Dinner was bland</a></li>
  <li><a href="#">Three owls</a></li>
  <li><a href="#">No-one answered the door</a></li>
  <li><a href="#">The stranger leaves</a></li>
  <li><a href="#">Bedtime</a></li>
</ol>
```

CSS

The CSS includes a `counter-reset` for the `chapter` counter on the `<ol>` element. We also increment the `chapter` counter on each `<li>` element using `counter-increment`, and remove the list markers by setting a `list-style-type` value of `none`.

```
ol {
  counter-reset: chapter;
}

li {
  counter-increment: chapter;
  list-style-type: none;
}
```

Next, we set the `<a>` elements' `::before` pseudo-elements to have generated `content` equal to a book emoji to represent a chapter, plus the current `chapter` counter value, and a space character so that the generated content is separated from the link text. Finally, we set the generated content's alt text to the current `chapter` counter value preceded by the word "Chapter".

```
a::before {
  content: "📖 " counter(chapter) " " / "Chapter " counter(chapter);
}
```

Result

When a screen reader navigates to a link within the list, supporting browsers will announce "Chapter" followed by the current counter number, followed by the link text, for example, "Chapter 1 A stranger calls" and "Chapter 2 Two owls".

Element replacement with URL

This example replaces a regular element! The element's contents are replaced with an SVG using the `<url>` type.

Pseudo-elements aren't rendered on replaced elements. As this element is replaced, any matching `::after` or `::before` are not generated or applied. To demonstrate this, we include an `::after` declaration block, attempting to add the `id` as generated content. This pseudo-element will not be generated as the element is replaced.

HTML

```
<div id="replaced">This content is replaced!</div>
```

CSS

```
#replaced {
  content: url("mdn.svg");
}

/* will not show if element replacement is supported */
div::after {
  content: " (" attr(id) ")";
}
```

Result

When generating content on regular elements (rather than just on pseudo-elements), the entire element is replaced. This means that `::before` and `::after` pseudo-elements are not generated.

Element replacement with <code><gradient></code>

This example demonstrates how an element's contents can be replaced by any type of `<image>`, in this case, a CSS gradient. The element's contents are replaced with a `linear-gradient()`. We provide alt text because all images should be described for accessibility.

HTML

```
<div id="replaced">I disappear</div>
```

CSS

```
div {
  border: 1px solid;
  background-color: #cccccc;
  min-height: 100px;
  min-width: 100px;
}

#replaced {
  content: repeating-linear-gradient(blue 0, orange 10%) /
    "Gradients and alt text are supported";
}
```

Result

Check the browser compatibility chart. All browsers support gradients and all browsers support replacing elements with images, but not all browsers support gradients as a `content` value.

Element replacement with <code>image-set()</code>

This example replaces an element's content with an `image-set()`. If the users display has normal resolution the `1x.png` will be displayed. Screens with a higher resolution will display the `2x.png` image.

HTML

```
<div id="replaced">I disappear!</div>
```

CSS

```
div {
  width: 100px;
  border: 1px solid lightgrey;
}
```

```
#replaced {
  content: image-set(
    "1x.png" 1x,
    "2x.png" 2x
  ) / "DPI";
}
```

Result

See also

- `::after`
- `::before`
- `::marker`
- `::scroll-button()`
- `::scroll-marker`
- `:target-current`
- `contain`

- `quotes`
- `<gradient>`
- `image-set()`
- `<url>`
- Replaced elements
- CSS generated content module
- CSS lists and counters module
- Browser accessibility panels: Firefox Accessibility inspector, Chrome Accessibility pane, and Safari Accessibility tree

content-visibility CSS property

The **`content-visibility`** CSS property controls whether or not an element renders its contents at all, along with forcing a strong set of containments, allowing user agents to potentially omit large swathes of layout and rendering work until it becomes needed. It enables the user agent to skip an element's rendering work (including layout and painting) until it is needed — which makes the initial page load much faster.

> **Note:** The `contentvisibilityautostatechange` event fires on any element with `content-visibility: auto` set on it when its rendering work starts or stops being skipped. This provides a convenient way for an app's code to start or stop rendering processes (e.g., drawing on a `<canvas>`) when they are not needed, thereby conserving processing power.

Try it

```
content-visibility: visible;
```

```
content-visibility: hidden;
```

```
<section class="default-example" id="default-example">
  <div class="container" id="example-element">
    <div class="child">
      <span>This is an inner div</span>
    </div>
  </div>
</section>
```

```
.container {
  width: 140px;
  height: 140px;
  border: 3px solid rgb(64 28 163);
  background-color: rgb(135 136 184);
  display: flex;
  align-items: center;
  justify-content: center;
}

.child {
  border: 3px solid rgb(64 28 163);
  background-color: wheat;
  color: black;
  width: 80%;
  height: 80%;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

Syntax

```
/* Keyword values */
content-visibility: visible;
content-visibility: hidden;
content-visibility: auto;

/* Global values */
content-visibility: inherit;
content-visibility: initial;
content-visibility: revert;
content-visibility: revert-layer;
content-visibility: unset;
```

Values

This property is specified as one of the following keyword values:

`visible`

No effect. The element's contents are laid out and rendered as normal. This is the default value.

`hidden`

The element skips its contents. The skipped contents must not be accessible to user-agent features, such as find-in-page, tab-order navigation, etc., nor be selectable or focusable. This is similar to giving the contents `display: none`.

`auto`

The element turns on layout containment, style containment, and paint containment. If the element is not relevant to the user, it also skips its contents. Unlike hidden, the skipped contents must still be available as normal to user-agent features such as find-in-page, tab order navigation, etc., and must be focusable and selectable as normal.

Description

Animating and transitioning content-visibility

Supporting browsers animate/transition `content-visibility` with a variation on the discrete animation type.

Discrete animation generally means that the property will flip between two values 50% of the way through the animation. In the case of `content-visibility`, however, the browser will flip between the two values to show the animated content for the entire animation duration. So, for example:

- When animating `content-visibility` from `hidden` to `visible`, the value will flip to `visible` at `0%` of the animation duration so it is visible throughout.
- When animating `content-visibility` from `visible` to `hidden`, the value will flip to `hidden` at `100%` of the animation duration so it is visible throughout.

This behavior is useful for creating entry/exit animations where you want to, for example, remove some content from the DOM with `content-visibility: hidden`, but you want a smooth transition (such as a fade-out) rather than it disappearing immediately.

When animating `content-visibility` with CSS transitions, `transition-behavior: allow-discrete` needs to be set on `content-visibility`. This effectively enables `content-visibility` transitions.

> **Note:** When transitioning an element's `content-visibility` value, you don't need to provide a set of starting values for transitioned properties using a `@starting-style` block, like you do when transitioning `display`. This is because `content-visibility` doesn't hide an element from the DOM like `display` does: it just skips rendering the element's content.

Formal definition

Initial value	`visible`
Applies to	elements for which size containment can apply
Inherited	no
Computed value	as specified
Animation type	Discrete behavior except when animating to or from `hidden` is visible for the entire duration

Formal syntax

```
content-visibility =
  visible  |
  auto     |
  hidden
```

This syntax reflects the latest standard as per CSS Containment Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Off-screen content within a `content-visibility: auto` property remains in the document object model and the accessibility tree. This allows improving page performance with `content-visibility: auto` without negatively impacting accessibility.

Since styles for off-screen content are not rendered, elements intentionally hidden with `display: none` or `visibility: hidden` *will still appear in the accessibility tree*. If you don't want an element to appear in the accessibility tree, use `aria-hidden="true"`.

Examples

Using auto to reduce rendering cost of long pages

The following example shows the use of `content-visibility: auto` to skip painting and rendering of off-screen sections. When a `section` is out of the viewport then the painting of the content is skipped until the section comes close to the viewport, this helps with both load and interactions on the page.

HTML

```
<section>
  <!-- Content for each section… -->
</section>
<section>
  <!-- Content for each section… -->
</section>
<section>
  <!-- Content for each section… -->
</section>
<!-- … -->
```

CSS

The `contain-intrinsic-size` property adds a default size of 500px to the height and width of each `section` element. After a section is rendered, it will retain its rendered intrinsic size, even when it is scrolled out of the viewport.

```
section {
  content-visibility: auto;
  contain-intrinsic-size: auto 500px;
}
```

Using hidden to manage visibility

The following example shows how to manage content visibility with JavaScript. Using `content-visibility: hidden;` instead of `display: none;` preserves the rendering state of content when hidden and rendering is faster.

HTML

```
<div class="hidden">
  <button class="toggle">Show</button>
  <p>
    This content is initially hidden and can be shown by clicking the button.
  </p>
</div>
<div class="visible">
  <button class="toggle">Hide</button>
  <p>
    This content is initially visible and can be hidden by clicking the button.
  </p>
</div>
```

CSS

The `content-visibility` property is set on paragraphs that are direct children of elements with the `visible` and `hidden` classes. In our example, we can show and hide content in paragraphs depending on the CSS class of parent div elements.

The `contain-intrinsic-size` property is included to represent the content size. This helps to reduce layout shift when content is hidden.

```
p {
  contain-intrinsic-size: 0 1.1em;
  border: dotted 2px;
}
```

```
.hidden > p {
  content-visibility: hidden;
}

.visible > p {
  content-visibility: visible;
}
```

JavaScript

```
const handleClick = (event) => {
  const button = event.target;
  const div = button.parentElement;
  button.textContent = div.classList.contains("visible") ? "Show" : "Hide";
  div.classList.toggle("hidden");
  div.classList.toggle("visible");
};

document.querySelectorAll("button.toggle").forEach((button) => {
  button.addEventListener("click", handleClick);
});
```

Result

Animating content-visibility

In this example, we have a `<div>` element, the content of which can be toggled between shown and hidden by clicking or pressing any key.

HTML

```
<p>
  Click anywhere on the screen or press any key to toggle the
  <code>&lt;div&gt;</code> content between hidden and showing.
</p>

<div>
  This is a <code>&lt;div&gt;</code> element that animates between
  <code>content-visibility: hidden;</code>and
  <code>content-visibility: visible;</code>. We've also animated the text color
  to create a smooth animation effect.
</div>
```

CSS

In the CSS we initially set `content-visibility: hidden;` on the `<div>` to hide its content. We then set up `@keyframes` animations and attach them to classes to show and hide the `<div>`, animating `content-visibility` and `color` so that you get a smooth animation effect as the content is shown/hidden.

```
div {
  font-size: 1.6rem;
  padding: 20px;
  border: 3px solid red;
  border-radius: 20px;
  width: 480px;

  content-visibility: hidden;
}

/* Animation classes */

.show {
  animation: show 0.7s ease-in forwards;
}

.hide {
  animation: hide 0.7s ease-out forwards;
}

/* Animation keyframes */

@keyframes show {
  0% {
    content-visibility: hidden;
    color: transparent;
  }

  100% {
    content-visibility: visible;
    color: black;
  }
}

@keyframes hide {
  0% {
    content-visibility: visible;
    color: black;
  }

  100% {
    content-visibility: hidden;
    color: transparent;
  }
}
```

JavaScript

Finally, we use JavaScript to apply the `.show` and `.hide` classes to the `<div>` as appropriate to apply the animations as it is toggled between shown and hidden states.

```
const divElem = document.querySelector("div");
const htmlElem = document.querySelector(":root");

htmlElem.addEventListener("click", showHide);
```

```
document.addEventListener("keydown", showHide);

function showHide() {
  if (divElem.classList.contains("show")) {
    divElem.classList.remove("show");
    divElem.classList.add("hide");
  } else {
    divElem.classList.remove("hide");
    divElem.classList.add("show");
  }
}
```

Result

The rendered result looks like this:

See also

- CSS Containment
- `contain-intrinsic-size`
- `contentvisibilityautostatechange`
- content-visibility: the new CSS property that boosts your rendering performance (web.dev)

corner-block-end-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-block-end-shape`** CSS property specifies the shape of both the corners on a box's block-end edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following logical properties:

- `corner-end-start-shape`
- `corner-end-end-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-block-end-shape: square;
corner-block-end-shape: scoop;

/* Single superellipse() value set for both corners */
corner-block-end-shape: superellipse(3.5);
corner-block-end-shape: superellipse(-1.9);

/* Block-end/inline-start corner, block-end/inline-end corner */
corner-block-end-shape: square scoop;
corner-block-end-shape: square superellipse(-1.9);

/* Global values */
corner-block-end-shape: inherit;
corner-block-end-shape: initial;
corner-block-end-shape: revert;
corner-block-end-shape: revert-layer;
corner-block-end-shape: unset;
```

Values

The `corner-block-end-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both block-end** corners.
- If **two values** are used, the first one specifies the shape of the **block-end/inline-start** corner, and the second one specifies the shape of the **block-end/inline-end** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-end-start-shape`: round • `corner-end-end-shape`: round
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-end-start-shape`: The corresponding superellipse() value. • `corner-end-end-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-end-start-shape`: Animates as per superellipse interpolation. • `corner-end-end-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-block-end-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
```

```
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-block-end-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `80px / 40px`, and a `corner-block-end-shape` of `notch square`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: cyan;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 80px / 40px;
  corner-block-end-shape: notch square;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-block-start-shape`, `corner-inline-start-shape`, and `corner-inline-end-shape`
- `corner-top-shape`, `corner-bottom-shape`, `corner-left-shape`, and `corner-right-shape`
- `border-radius` shorthand property
- `border-end-start-radius` and `border-end-end-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-block-start-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-block-start-shape`** CSS property specifies the shape of both the corners on a box's block-start edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following logical properties:

- `corner-start-start-shape`
- `corner-start-end-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-block-start-shape: notch;
corner-block-start-shape: squircle;

/* Single superellipse() value set for both corners */
corner-block-start-shape: superellipse(2.7);
corner-block-start-shape: superellipse(-2.5);

/* Block-start/inline-start corner, block-start/inline-end corner */
corner-block-start-shape: notch squircle;
```

```
corner-block-start-shape: notch superellipse(-2.5);

/* Global values */
corner-block-start-shape: inherit;
corner-block-start-shape: initial;
corner-block-start-shape: revert;
corner-block-start-shape: revert-layer;
corner-block-start-shape: unset;
```

Values

The `corner-block-start-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of both **block-start** corners.
- If **two values** are used, the first one specifies the shape of the **block-start/inline-start** corner, and the second one specifies the shape of the **block-start/inline-end** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-start-start-shape`: `round` • `corner-start-end-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-start-start-shape`: The corresponding superellipse() value. • `corner-start-end-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-start-start-shape`: Animates as per superellipse interpolation. • `corner-start-end-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-block-start-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-block-start-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `40px 60px`, and a `corner-block-start-shape` of `scoop notch`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: goldenrod;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 40px 60px;
```

```
  corner-block-start-shape: scoop notch;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-block-end-shape`, `corner-inline-start-shape`, and `corner-inline-end-shape`
- `corner-top-shape`, `corner-bottom-shape`, `corner-left-shape`, and `corner-right-shape`
- `border-radius` shorthand property
- `border-start-start-radius` and `border-start-end-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-bottom-left-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-bottom-left-shape`** CSS property specifies the shape of a box's bottom-left corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-bottom-left-shape: bevel;
corner-bottom-left-shape: round;

/* superellipse() function values */
corner-bottom-left-shape: superellipse(0.7);
corner-bottom-left-shape: superellipse(-1.9);

/* Global values */
corner-bottom-left-shape: inherit;
corner-bottom-left-shape: initial;
corner-bottom-left-shape: revert;
corner-bottom-left-shape: revert-layer;
corner-bottom-left-shape: unset;
```

Values

The `corner-bottom-left-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-bottom-left-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-bottom-left-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `80px 40% 60px 20%`, and a `corner-bottom-left-shape` of `squircle`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
```

```
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: goldenrod;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 80px 40% 60px 20%;
  corner-bottom-left-shape: squircle;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-left-shape`, `corner-top-right-shape`, and `corner-bottom-right-shape`
- `corner-start-start-shape`, `corner-start-end-shape`, `corner-end-start-shape`, and `corner-end-end-shape`
- `border-radius` shorthand property
- `border-bottom-left-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-bottom-right-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-bottom-right-shape`** CSS property specifies the shape of a box's bottom-right corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-bottom-right-shape: notch;
corner-bottom-right-shape: scoop;

/* superellipse() function values */
corner-bottom-right-shape: superellipse(1.7);
corner-bottom-right-shape: superellipse(-3);

/* Global values */
corner-bottom-right-shape: inherit;
corner-bottom-right-shape: initial;
corner-bottom-right-shape: revert;
corner-bottom-right-shape: revert-layer;
corner-bottom-right-shape: unset;
```

Values

The `corner-bottom-right-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-bottom-right-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity               |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-bottom-right-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `30% / 20%`, and a `corner-bottom-right-shape` of `notch`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: cyan;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 30% / 20%;
  corner-bottom-right-shape: notch;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-left-shape`, `corner-top-right-shape`, and `corner-bottom-left-shape`
- `corner-start-start-shape`, `corner-start-end-shape`, `corner-end-start-shape`, and `corner-end-end-shape`
- `border-radius` shorthand property
- `border-bottom-right-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-bottom-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-bottom-shape`** CSS property specifies the shape of both the corners on a box's bottom edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following physical properties:

- `corner-bottom-left-shape`
- `corner-bottom-right-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-bottom-shape: scoop;
corner-bottom-shape: square;

/* Single superellipse() value set for both corners */
corner-bottom-shape: superellipse(0.7);
corner-bottom-shape: superellipse(-2.8);

/* Left-hand corner, right-hand corner */
corner-bottom-shape: scoop square;
corner-bottom-shape: scoop superellipse(0.7);

/* Global values */
corner-bottom-shape: inherit;
corner-bottom-shape: initial;
corner-bottom-shape: revert;
corner-bottom-shape: revert-layer;
corner-bottom-shape: unset;
```

Values

The `corner-bottom-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both bottom corners**.
- If **two values** are used, the first one specifies the shape of the **bottom-left** corner, and the second one specifies the shape of the **bottom-right** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-bottom-left-shape`: `round` • `corner-bottom-right-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-bottom-left-shape`: The corresponding superellipse() value. • `corner-bottom-right-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-bottom-left-shape`: Animates as per superellipse interpolation. • `corner-bottom-right-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-bottom-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-bottom-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `60px 30px 20% 40%`, and a `corner-bottom-shape` of `square squircle`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: orange;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 60px 30px 20% 40%;
  corner-bottom-shape: square squircle;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-shape`, `corner-left-shape`, and `corner-right-shape`
- `corner-block-start-shape`, `corner-block-end-shape`, `corner-inline-start-shape`, and `corner-inline-end-shape`
- `border-radius` shorthand property

- `border-bottom-left-radius` and `border-bottom-right-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-end-end-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-end-end-shape`** CSS property specifies the shape of a box's block-end and inline-end corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-end-end-shape: scoop;
corner-end-end-shape: notch;

/* superellipse() function values */
corner-end-end-shape: superellipse(4);
corner-end-end-shape: superellipse(-0.5);

/* Global values */
corner-end-end-shape: inherit;
corner-end-end-shape: initial;
corner-end-end-shape: revert;
corner-end-end-shape: revert-layer;
corner-end-end-shape: unset;
```

Values

The `corner-end-end-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-end-end-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-end-end-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of 60 pixels, and a `corner-end-end-shape` of `squircle`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: green;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
```

```
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 60px;
  corner-end-end-shape: squircle;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-start-start-shape`, `corner-start-end-shape`, and `corner-end-start-shape`
- `corner-top-left-shape`, `corner-top-right-shape`, `corner-bottom-left-shape`, and `corner-bottom-right-shape`
- `border-radius` shorthand property
- `border-end-end-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-end-start-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-end-start-shape`** CSS property specifies the shape of a box's block-end and inline-start corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-end-start-shape: round;
corner-end-start-shape: scoop;

/* superellipse() function values */
corner-end-start-shape: superellipse(0.2);
corner-end-start-shape: superellipse(-5);

/* Global values */
corner-end-start-shape: inherit;
corner-end-start-shape: initial;
corner-end-start-shape: revert;
corner-end-start-shape: revert-layer;
corner-end-start-shape: unset;
```

Values

The `corner-end-start-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-end-start-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-end-start-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `60px / 20%`, and a `corner-end-start-shape` of `bevel`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: orange;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 60px / 20%;
  corner-end-start-shape: bevel;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-start-start-shape`, `corner-start-end-shape`, and `corner-end-end-shape`
- `corner-top-left-shape`, `corner-top-right-shape`, `corner-bottom-left-shape`, and `corner-bottom-right-shape`
- `border-radius` shorthand property
- `border-end-start-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-inline-end-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-inline-end-shape`** CSS property specifies the shape of both the corners on a box's inline-end edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following logical properties:

- `corner-start-end-shape`
- `corner-end-end-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-inline-end-shape: squircle;
corner-inline-end-shape: scoop;

/* Single superellipse() value set for both corners */
corner-inline-end-shape: superellipse(1.5);
corner-inline-end-shape: superellipse(-0.8);

/* Block-start/inline-end corner, block-end/inline-end corner */
corner-inline-end-shape: squircle scoop;
corner-inline-end-shape: squircle superellipse(-0.8);

/* Global values */
corner-inline-end-shape: inherit;
corner-inline-end-shape: initial;
corner-inline-end-shape: revert;
corner-inline-end-shape: revert-layer;
corner-inline-end-shape: unset;
```

Values

The `corner-inline-end-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both inline-end** corners.
- If **two values** are used, the first one specifies the shape of the **block-start/inline-end** corner, and the second one specifies the shape of the **block-end/inline-end** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-start-end-shape`: `round` • `corner-end-end-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.

Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-start-end-shape`: The corresponding superellipse() value. • `corner-end-end-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-start-end-shape`: Animates as per superellipse interpolation. • `corner-end-end-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-inline-end-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-inline-end-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `15% 30%`, and a `corner-inline-end-shape` of `bevel squircle`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: green;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 15% 30%;
  corner-inline-end-shape: bevel squircle;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-block-start-shape`, `corner-block-end-shape`, and `corner-inline-start-shape`
- `corner-top-shape`, `corner-bottom-shape`, `corner-left-shape`, and `corner-right-shape`
- `border-radius` shorthand property
- `border-start-end-radius` and `border-end-end-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-inline-start-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-inline-start-shape`** CSS property specifies the shape of both the corners on a box's inline-start edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following logical properties:

- `corner-start-start-shape`
- `corner-end-start-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-inline-start-shape: round;
corner-inline-start-shape: bevel;

/* Single superellipse() value set for both corners */
corner-inline-start-shape: superellipse(0.5);
corner-inline-start-shape: superellipse(-3);

/* Block-start/inline-start corner, block-end/inline-start corner */
corner-inline-start-shape: round bevel;
corner-inline-start-shape: round superellipse(0.5);

/* Global values */
corner-inline-start-shape: inherit;
corner-inline-start-shape: initial;
corner-inline-start-shape: revert;
corner-inline-start-shape: revert-layer;
corner-inline-start-shape: unset;
```

Values

The `corner-inline-start-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both inline-start** corners.
- If **two values** are used, the first one specifies the shape of the **block-start/inline-start** corner, and the second one specifies the shape of the **block-end/inline-start** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-start-start-shape`: `round` • `corner-start-end-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-start-start-shape`: The corresponding superellipse() value. • `corner-start-end-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-start-start-shape`: Animates as per superellipse interpolation. • `corner-start-end-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-inline-start-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-inline-start-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of 60 pixels, and a `corner-inline-start-shape` of `square bevel`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: orange;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 60px;
  corner-inline-start-shape: square bevel;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-block-start-shape`, `corner-block-end-shape`, and `corner-inline-end-shape`
- `corner-top-shape`, `corner-bottom-shape`, `corner-left-shape`, and `corner-right-shape`
- `border-radius` shorthand property
- `border-start-start-radius` and `border-end-start-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-left-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-left-shape`** CSS property specifies the shape of both the corners on a box's left-hand edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following physical properties:

- `corner-top-left-shape`
- `corner-bottom-left-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-left-shape: round;
corner-left-shape: bevel;

/* Single superellipse() value set for both corners */
corner-left-shape: superellipse(1.2);
corner-left-shape: superellipse(-2.5);

/* Top corner, bottom corner */
corner-left-shape: round bevel;
corner-left-shape: bevel superellipse(1.2);

/* Global values */
corner-left-shape: inherit;
corner-left-shape: initial;
corner-left-shape: revert;
corner-left-shape: revert-layer;
corner-left-shape: unset;
```

Values

The `corner-left-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both left-hand corners**.
- If **two values** are used, the first one specifies the shape of the **top-left** corner, and the second one specifies the shape of the **bottom-left** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-top-left-shape`: round • `corner-bottom-left-shape`: round
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand:

	• `corner-top-left-shape`: The corresponding superellipse() value. • `corner-bottom-left-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-top-left-shape`: Animates as per superellipse interpolation. • `corner-bottom-left-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-left-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-left-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `20px 40px / 60px 80px`, and a `corner-left-shape` of `notch bevel`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: green;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 20px 40px / 60px 80px;
  corner-left-shape: notch bevel;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-shape`, `corner-bottom-shape`, and `corner-right-shape`
- `corner-block-start-shape`, `corner-block-end-shape`, `corner-inline-start-shape`, and `corner-inline-end-shape`
- `border-radius` shorthand property
- `border-top-left-radius` and `border-bottom-left-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-right-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-right-shape`** CSS property specifies the shape of both the corners on a box's right-hand edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following physical properties:

- `corner-top-right-shape`
- `corner-bottom-right-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-right-shape: bevel;
corner-right-shape: notch;

/* Single superellipse() value set for both corners */
corner-right-shape: superellipse(4);
corner-right-shape: superellipse(-0.9);

/* Top corner, bottom corner */
corner-right-shape: bevel notch;
corner-right-shape: notch superellipse(-0.9);

/* Global values */
corner-right-shape: inherit;
corner-right-shape: initial;
corner-right-shape: revert;
corner-right-shape: revert-layer;
corner-right-shape: unset;
```

Values

The `corner-right-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both right-hand corners**.
- If **two values** are used, the first one specifies the shape of the **top-right** corner, and the second one specifies the shape of the **bottom-right** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-top-right-shape`: `round` • `corner-bottom-right-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-top-right-shape`: The corresponding superellipse() value. • `corner-bottom-right-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-top-right-shape`: Animates as per superellipse interpolation. • `corner-bottom-right-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-right-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-right-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `20% 30% / 50% 40%`, and a `corner-right-shape` of `square scoop`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
```

```
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: cyan;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 20% 30% / 50% 40%;
  corner-right-shape: square scoop;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-shape`, `corner-bottom-shape`, and `corner-left-shape`
- `corner-block-start-shape`, `corner-block-end-shape`, `corner-inline-start-shape`, and `corner-inline-end-shape`
- `border-radius` shorthand property
- `border-top-right-radius` and `border-bottom-right-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-shape`** CSS shorthand property specifies the shape of a box's corners, within the area specified by its `border-radius` property value.

Constituent properties

This property is a shorthand for the following physical properties:

- `corner-top-left-shape`
- `corner-top-right-shape`
- `corner-bottom-left-shape`
- `corner-bottom-right-shape`

Syntax

```
/* Single value set for all four corners */
corner-shape: bevel;

/* top-left and bottom-right, top-right and bottom-left */
corner-shape: notch superellipse(0.6);

/* top-left, top-right and bottom-left, bottom-right */
corner-shape: superellipse(-1.2) square squircle;

/* top-left, top-right, bottom-right, bottom-left */
corner-shape: scoop superellipse(-1.6) superellipse(-2.2) round;

/* Global values */
corner-shape: inherit;
corner-shape: initial;
corner-shape: revert;
corner-shape: revert-layer;
corner-shape: unset;
```

The `corner-shape` property may be specified using one, two, three, or four `<corner-shape-value>` values:

- If **one** value is used, it specifies the shape of **all four corners**.
- If **two** values are used, the first shape applies to the **top-left and bottom-right** corners, and the second to the **top-right and bottom-left corners**.
- If **three** values are used, the first shape specifies the shape of the **top-left corner**, the second to the **top-right and bottom-left corners**, and the third to the **bottom-right corner**.
- If four values are used, they specify the shape of the **top-left**, **top-right**, **bottom-right**, and **bottom-left** corners, in that order (clockwise).

Values

`<corner-shape-value>`

A `superellipse()` or keyword equivalent describing the shape of the corner.

Description

The `corner-shape` property is used to modify the shape of rounded corners created by the `border-radius` property and its associated longhands. Already-rounded corners can be further customized in terms of the degree of rounding applied to them, allowing the creation of, for example, beveled, notched, and squircle corners. Borders, outlines, shadows, and background effects applied to the container will follow the defined corner shape.

If a `border-radius` is not applied to a container, or the `border-radius` resolves to `0`, `corner-shape` will have no effect.

The `corner-shape` shorthand property and its associated `corner-*-shape` shorthands and longhands accept one to four `<corner-shape-value>` values. Each is specified directly as a `superellipse()` function or a keyword describing a common shape. Each keyword is equivalent to a specific `superellipse()` value.

The default (initial) value of `corner-shape` is `round`, which gives the same effect as using `border-radius` on its own, without `corner-shape`. There is also a keyword value `square`, which gives the same effect as default square corners, effectively removing any `border-radius` applied. The `bevel` value has the effect of drawing a straight line between the two ends of a `border-radius`.

Different `corner-shape` values can be smoothly animated between, as the `superellipse()` equivalents of the keyword values are used as interpolation values.

The `corner-shape` shorthand is especially useful when you want all four borders to be the same, or you want to set different values using a single declaration. To set only one or two corner shapes at a time, use the `corner-*-shape` shorthands and longhands.

<code>corner-*-shape</code> shorthands and longhands

The `corner-shape` shorthand defines the shapes of all four corners in one declaration.

To set only one corner shape at a time, use the corner shape longhands:

- Physical longhand corner shape properties:
  - `corner-bottom-left-shape`
  - `corner-bottom-right-shape`
  - `corner-top-left-shape`
  - `corner-top-right-shape`
- Logical longhand corner shape properties:
  - `corner-start-start-shape`
  - `corner-start-end-shape`
  - `corner-end-start-shape`
  - `corner-end-end-shape`

To set two corner shapes at a time, use the side shorthands:

- Physical side shorthand properties:
  - `corner-top-shape`
  - `corner-right-shape`
  - `corner-bottom-shape`
  - `corner-left-shape`
- Logical side shorthand properties:
  - `corner-block-start-shape`
  - `corner-block-end-shape`
  - `corner-inline-start-shape`
  - `corner-inline-end-shape`

Constraining opposite corner shape radii

When opposite corners have `border-radius` and `corner-shape` values set that would cause the shapes to overlap, the browser constrains the values to prevent the overlap.

For example, the following values would cause the top-left and bottom-right corners to overlap, therefore the browser adjusts the first `border-radius` component to a value that avoids this.

```
div {
  width: 480px;
  height: 200px;
  background-color: goldenrod;
  border-radius: 80% 20px;
  corner-shape: scoop;
}
```

Properties that follow <code>corner-shape</code>

The following properties all follow the shape of the corner when set on the container:

- `background-color`
- `background-image`
- `border`
- `outline`
- `box-shadow`
- `overflow`
- `backdrop-filter`

See Demonstration of properties that follow the `corner-shape` for some examples.

Interaction with <code>border-shape</code>

The `corner-shape` property has no effect when the `border-shape` property is set on an element. This is because `border-shape` is incompatible with `border-radius`: when `border-shape` is set, any specified `border-radius` is ignored; `corner-shape` depends on `border-radius` and therefore has no effect either.

If you want to use shaped corners in a `border-shape`, you will have to draw them directly as part of the shape.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-top-left-shape`: `round` • `corner-top-right-shape`: `round` • `corner-bottom-left-shape`: `round` • `corner-bottom-right-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-top-left-shape`: The corresponding superellipse() value. • `corner-top-right-shape`: The corresponding superellipse() value. • `corner-bottom-left-shape`: The corresponding superellipse() value. • `corner-bottom-right-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand:

	• `corner-top-left-shape`: Animates as per superellipse interpolation. • `corner-top-right-shape`: Animates as per superellipse interpolation. • `corner-bottom-left-shape`: Animates as per superellipse interpolation. • `corner-bottom-right-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-shape =
  <'corner-top-left-shape'>{1,4}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>corner-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div>Nice scooped corners</div>
```

CSS

We give the box a fixed `height`, a `box-shadow`, a `border-radius` of 30 pixels, and a `corner-shape` of `scoop`, along with some additional styles that we've hidden for brevity.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: cyan;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}

@supports not (corner-shape: scoop) {
  body {
    all: unset !important;
  }

  body::before {
    content: "Your browser does not support the 'corner-shape' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
div {
  height: 180px;
  box-shadow: 1px 1px 3px gray;
  border-radius: 30px;
  corner-shape: scoop;
}
```

Result

The rendered result looks like this:

Note how the `corner-shape` value of `scoop` gives the container concave corners — the curve is an inversion of the default `border-radius` curve. Note also how the background, border, and box shadow follow the shape of the curve.

Demonstration of properties that follow the <code>corner-shape</code>

HTML

The markup for this example contains a single `<div>` element with some text content inside it.

```
<div>
  Some styles follow the corner shape, such as border, outline, box-shadow,
  overflow, and backdrop-filter. This is useful for helping various aspects of
```

```
  your design to not clash. As shown, it can result in some interesting visual
  effects, so you should test your design carefully.
</div>
```

CSS

To demonstrate how some styles follow the shape of a container's corners, we apply a `background-image` to the document `<body>`, then apply a `border-radius` of `40px` and a `corner-shape` of `scoop notch` to the `<div>`.

We then apply the following to the `<div>`:

- A semi-transparent `background-color`.
- A different color and style of `border` on each edge.
- A `backdrop-filter` that inverts the `background-image` set on the `<body>`.
- A `:hover` style so you can see that the clickable content area falls outside the corner shape.

Additional set up styles have been hidden for brevity.

```
html {
  height: 100%;
}

body {
  font-family: "Helvetica", "Arial", sans-serif;
  height: inherit;
  margin: 0;
  display: flex;
  justify-content: center;
  align-items: center;
}

div {
  width: 240px;
  height: 180px;
}

@supports not (corner-shape: scoop notch) {
  :root::before {
    content: "Your browser does not support the 'corner-shape' property.";
    color: black;
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
body {
  background: url("https://mdn.github.io/shared-assets/images/examples/leopard.jpg")
    no-repeat;
  background-size: cover;
}

div {
  border-radius: 40px;
  corner-shape: scoop notch;
  background-color: rgb(255 255 255 / 0.2);
  border-top: 3px solid blue;
  border-left: 6px dashed red;
  border-bottom: 9px solid yellow;
  border-right: 12px double green;
  backdrop-filter: invert(100%);
}

div:hover {
  background-color: white;
}
```

Result

The rendered result looks like this:

Note how most of the set styles follow the shape of the `<div>` resulting from its `corner-shape` styles, but not all. The content is displayed relative to the original box, and the hover effect is still applied if you hover over the text sticking out past the top- and bottom-left corners.

Comparing <code>corner-shape</code> values

In this demonstration, you can select different `corner-shape` values and set different `border-radius` values on a container and compare the effects.

HTML

The markup for this example contains a `<select>` picker from which different `corner-shape` values can be selected, an `<input type="range">` slider to select different `border-radius` values, and a `<section>` element to create a container to apply those values to. The select `<option>` elements provide multiple keyword and `superellipse()` value choices, broken into two groups using `<optgroup>` elements. In the case of the keyword values, we've also included the `superellipse()` value equivalent for each one, separated by a pipe character.

```
<form>
  <div>
    <label for="corner-shape-choice">Choose a corner-shape value:</label>
    <select id="corner-shape-choice">
      <optgroup label="Keywords">
        <option value="square">square | superellipse(infinity)</option>
        <option selected value="squircle">squircle | superellipse(2)</option>
        <option value="round">round | superellipse(1)</option>
        <option value="bevel">bevel | superellipse(0)</option>
        <option value="scoop">scoop | superellipse(-1)</option>
        <option value="notch">notch | superellipse(-infinity)</option>
      </optgroup>
      <optgroup label="Functions">
        <option>superellipse(3)</option>
        <option>superellipse(1.5)</option>
```

```
        <option>superellipse(0.5)</option>
        <option>superellipse(-0.5)</option>
        <option>superellipse(-1.5)</option>
        <option>superellipse(-3)</option>
      </optgroup>
    </select>
  </div>
  <div>
    <label for="radius-slider">Choose a border-radius value:</label>
    <input
      type="range"
      id="radius-slider"
      min="0"
      value="45"
      max="90"
      step="1" />
  </div>
</form>
<section></section>
```

CSS

We apply a `box-shadow` to the `<section>`. We also give the `<section>` and form elements some basic styles, which we've hidden for brevity.

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

body {
  width: fit-content;
  margin: 20px auto;
}

section {
  display: flex;
  justify-content: center;
  align-items: center;
  margin-top: 20px;
}

select {
  padding: 3px 5px;
}

form div:nth-of-type(2) {
  margin-top: 5px;
  display: flex;
}

section {
  width: 100%;
  height: 180px;
  background-color: gold;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}

@supports not (corner-shape: scoop) {
  :root::before {
    content: "Your browser does not support the 'corner-shape' property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
section {
  box-shadow: 1px 1px 3px gray;
}
```

```
const rectangle = document.querySelector("section");
const select = document.querySelector("select");
const range = document.getElementById("radius-slider");

function setCorners() {
  rectangle.style.cornerShape = select.value;
  const brValue = `${range.value}px`;
  rectangle.style.borderRadius = brValue;
  rectangle.innerHTML = `<div><code>corner-shape: ${select.value};</code><br><code>border-radius: ${brValue};</code></div>`;
}

select.addEventListener("change", setCorners);
range.addEventListener("input", setCorners);
setCorners();
```

The JavaScript that applies the user-selected values to the `<section>` has been hidden for brevity.

Result

The rendered result looks like this:

Try selecting different values to see how this affects the shape of the corners.

<code>superellipse()</code> value comparison

In this example, we provide two `<input type="range">` sliders allowing you to cycle through many different `corner-shape` `superellipse()` values and `border-radius` values to compare the effects of each on a container.

HTML

The markup for this example contains two `<input type="range">` elements from which different `corner-shape` `superellipse()` and `border-radius` values can be selected, and a `<section>` element to apply those values to.

```
<form>
  <div>
    <label for="superellipse-slider">Choose a superellipse() value:</label>
    <input
      type="range"
      id="superellipse-slider"
      min="-5"
      value="0"
      max="5"
      step="0.1" />
  </div>
  <div>
    <label for="radius-slider">Choose a border-radius value:</label>
    <input
      type="range"
      id="radius-slider"
      min="0"
      value="45"
      max="90"
      step="1" />
  </div>
</form>
<section></section>
```

CSS

We apply a `box-shadow` to the `<section>` element. Additional basic styles have been hidden for brevity.

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

body {
  width: fit-content;
  margin: 20px auto;
}

section {
  display: flex;
  justify-content: center;
  align-items: center;
  margin-top: 20px;
}

form div {
  margin-top: 5px;
  display: flex;
}

section {
  width: 100%;
  height: 180px;
  background-color: orange;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}

@supports not (corner-shape: superellipse(0)) {
  :root::before {
    content: "Your browser does not support the 'corner-shape' property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
section {
  box-shadow: 1px 1px 3px gray;
}
```

```
const rectangle = document.querySelector("section");
const superEllipseRange = document.getElementById("superellipse-slider");
const borderRadiusRange = document.getElementById("radius-slider");

function setCorners() {
  const seValue = `superellipse(${superEllipseRange.value})`;
  rectangle.style.cornerShape = seValue;
  const brValue = `${borderRadiusRange.value}px`;
  rectangle.style.borderRadius = brValue;
  rectangle.innerHTML = `<div><code>corner-shape: ${seValue};</code><br><code>border-radius: ${brValue};</code></div>`;
}

superEllipseRange.addEventListener("input", setCorners);
borderRadiusRange.addEventListener("input", setCorners);
setCorners();
```

The JavaScript that applies the user-selected values to the `<section>` has been hidden for brevity.

Result

The rendered result looks like this:

Try selecting different values to see how this affects the shape of the corners.

Animating <code>corner-shape</code>

In this example, we demonstrate how the `corner-shape` property can be animated.

HTML

```
<div></div>
```

CSS

We create a set of `@keyframes` that animate smoothly between the `corner-shape` values of `square` and `notch`. We then apply an `animation` based on those `@keyframes` to the `<div>` when its containing `<html>` element is hovered over or focused. Additional basic `<div>` styles have been hidden for brevity.

```
body {
  width: 200px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 200px;
  background-color: green;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
  box-shadow: 1px 1px 3px gray;
  border-radius: 50%;
  corner-shape: square;
  outline: none;
}

@supports not (corner-shape: square) {
  :root::before {
    content: "Your browser does not support the 'corner-shape' property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
@keyframes corner-pulse {
  0% {
    corner-shape: square;
  }

  /* To make the starting point apparent, let us keep
  the shape the same for a small duration. */
  20% {
    corner-shape: square;
  }

  100% {
    corner-shape: notch;
  }
}

div {
  animation: corner-pulse infinite alternate 4s linear;
}
```

Result

The rendered result looks like this:

See also

- `border-radius`
- `border-shape`
- CSS borders and box decorations module
- CSS backgrounds and borders module
- CSS animations module

corner-start-end-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-start-end-shape`** CSS property specifies the shape of a box's block-start and inline-end corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-start-end-shape: square;
corner-start-end-shape: bevel;

/* superellipse() function values */
corner-start-end-shape: superellipse(1.5);
corner-start-end-shape: superellipse(-2.2);

/* Global values */
corner-start-end-shape: inherit;
```

```
corner-start-end-shape: initial;
corner-start-end-shape: revert;
corner-start-end-shape: revert-layer;
corner-start-end-shape: unset;
```

Values

The `corner-start-end-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-start-end-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-start-end-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `20px 40px 60px 80px`, and a `corner-start-end-shape` of `notch`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: cyan;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 20px 40px 60px 80px;
  corner-start-end-shape: notch;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-start-start-shape`, `corner-end-start-shape`, and `corner-end-end-shape`
- `corner-top-left-shape`, `corner-top-right-shape`, `corner-bottom-left-shape`, and `corner-bottom-right-shape`
- `border-radius` shorthand property
- `border-start-end-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-start-start-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-start-start-shape`** CSS property specifies the shape of a box's block-start and inline-start corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-start-start-shape: notch;
corner-start-start-shape: squircle;

/* superellipse() function values */
corner-start-start-shape: superellipse(3);
corner-start-start-shape: superellipse(-1.5);

/* Global values */
corner-start-start-shape: inherit;
corner-start-start-shape: initial;
corner-start-start-shape: revert;
corner-start-start-shape: revert-layer;
corner-start-start-shape: unset;
```

Values

The `corner-start-start-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-start-start-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-start-start-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `40% 10px 10px`, and a `corner-start-start-shape` of `scoop`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: goldenrod;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
```

```
  border-radius: 40% 10px 10px;
  corner-start-start-shape: scoop;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-start-end-shape`, `corner-end-start-shape`, and `corner-end-end-shape`
- `corner-top-left-shape`, `corner-top-right-shape`, `corner-bottom-left-shape`, and `corner-bottom-right-shape`
- `border-radius` shorthand property
- `border-start-start-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-top-left-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-top-left-shape`** CSS property specifies the shape of a box's top-left corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-top-left-shape: notch;
corner-top-left-shape: squircle;

/* superellipse() function values */
corner-top-left-shape: superellipse(3);
corner-top-left-shape: superellipse(-1.5);

/* Global values */
corner-top-left-shape: inherit;
corner-top-left-shape: initial;
corner-top-left-shape: revert;
corner-top-left-shape: revert-layer;
corner-top-left-shape: unset;
```

Values

The `corner-top-left-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-top-left-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-top-left-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of `25% 100px`, and a `corner-top-left-shape` of `scoop`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
```

```
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: orange;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 25% 100px;
  corner-top-left-shape: scoop;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-right-shape`, `corner-bottom-left-shape`, and `corner-bottom-right-shape`
- `corner-start-start-shape`, `corner-start-end-shape`, `corner-end-start-shape`, and `corner-end-end-shape`
- `border-radius` shorthand property
- `border-top-left-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-top-right-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-top-right-shape`** CSS property specifies the shape of a box's top-right corner, within its `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Syntax

```
/* Keyword values */
corner-top-right-shape: scoop;
corner-top-right-shape: square;

/* superellipse() function values */
corner-top-right-shape: superellipse(2.2);
corner-top-right-shape: superellipse(-2.5);

/* Global values */
corner-top-right-shape: inherit;
corner-top-right-shape: initial;
corner-top-right-shape: revert;
corner-top-right-shape: revert-layer;
corner-top-right-shape: unset;
```

Values

The `corner-top-right-shape` property is specified using a `<corner-shape-value>` value.

Formal definition

Initial value	`round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	The corresponding superellipse() value.
Animation type	Animates as per superellipse interpolation.

Formal syntax

```
corner-top-right-shape =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-top-right-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of 60 pixels, and a `corner-top-right-shape` of `bevel`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: green;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 60px;
  corner-top-right-shape: bevel;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-top-left-shape`, `corner-bottom-left-shape`, and `corner-bottom-right-shape`
- `corner-start-start-shape`, `corner-start-end-shape`, `corner-end-start-shape`, and `corner-end-end-shape`
- `border-radius` shorthand property
- `border-top-right-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

corner-top-shape CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`corner-top-shape`** CSS property specifies the shape of both the corners on a box's top edge, within their `border-radius` area.

For a full description of corner shape behavior and multiple examples, see the `corner-shape` shorthand property page.

Constituent properties

This property is a shorthand for the following physical properties:

- `corner-top-left-shape`
- `corner-top-right-shape`

Syntax

```
/* Single keyword value set for both corners */
corner-top-shape: notch;
corner-top-shape: squircle;

/* Single superellipse() value set for both corners */
corner-top-shape: superellipse(3);
corner-top-shape: superellipse(-1.5);

/* Left-hand corner, right-hand corner */
corner-top-shape: notch squircle;
corner-top-shape: notch superellipse(-1.5);

/* Global values */
corner-top-shape: inherit;
corner-top-shape: initial;
corner-top-shape: revert;
corner-top-shape: revert-layer;
corner-top-shape: unset;
```

Values

The `corner-top-shape` property is specified using one or two `<corner-shape-value>` values:

- If **one value** is used, it specifies the shape of **both top corners**.
- If **two values** are used, the first one specifies the shape of the **top-left** corner, and the second one specifies the shape of the **top-right** corner.

Formal definition

Initial value	as each of the properties of the shorthand: • `corner-top-left-shape`: `round` • `corner-top-right-shape`: `round`
Applies to	all elements. It also applies to `::first-letter`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `corner-top-left-shape`: The corresponding superellipse() value. • `corner-top-right-shape`: The corresponding superellipse() value.
Animation type	as each of the properties of the shorthand: • `corner-top-left-shape`: Animates as per superellipse interpolation. • `corner-top-right-shape`: Animates as per superellipse interpolation.

Formal syntax

```
corner-top-shape =
  <'corner-top-left-shape'>{1,2}

<corner-top-left-shape> =
  <corner-shape-value>

<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find other related examples on the `corner-shape` reference page.

Basic <code>corner-top-shape</code> usage

HTML

The markup for this example contains a single `<div>` element.

```
<div></div>
```

CSS

We give the box some basic styles, which we've hidden for brevity. We also apply a `box-shadow`, a `border-radius` of 60 pixels, and a `corner-top-shape` of `scoop notch`.

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 240px;
  margin: 20px auto;
}

div {
  width: 100%;
  height: 180px;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: goldenrod;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}
```

```
div {
  box-shadow: 1px 1px 3px gray;
  border-radius: 60px;
  corner-top-shape: scoop notch;
}
```

Result

The rendered result looks like this:

See also

- `corner-shape` shorthand property
- `corner-bottom-shape`, `corner-left-shape`, and `corner-right-shape`
- `corner-block-start-shape`, `corner-block-end-shape`, `corner-inline-start-shape`, and `corner-inline-end-shape`
- `border-radius` shorthand property

- `border-top-left-radius` and `border-top-right-radius`
- CSS borders and box decorations module
- CSS backgrounds and borders module

counter-increment CSS property

The **`counter-increment`** CSS property can be used to increase or decrease the value of the named CSS counters by the specified values, or to prevent all counters or an individual counter's value from being changed.

If a named counter in the list of space-separated counters and values doesn't exist, it will be created. If no value is provided for a counter in the list of counters, the counter will be increased by `1`.

The counter's value can be reset to any integer value with the `counter-reset` CSS property.

Try it

```
counter-increment: chapter-count;
```

```
counter-increment: chapter-count 0;
```

```
counter-increment: chapter-count 5;
```

```
counter-increment: chapter-count -5;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="chapters">
    <h1>Alice's Adventures in Wonderland</h1>
    <h2>Down the Rabbit-Hole</h2>
    <h2 id="example-element">The Pool of Tears</h2>
    <h2>A Caucus-Race and a Long Tale</h2>
    <h2>The Rabbit Sends in a Little Bill</h2>
  </div>
</section>
```

```
#default-example {
  text-align: left;
  counter-reset: chapter-count;
}

#example-element {
  background-color: lightblue;
  color: black;
}

h2 {
  counter-increment: chapter-count;
  font-size: 1em;
}

h2::before {
  content: "Chapter " counter(chapter-count) ": ";
}
```

Syntax

```
/* Increases "my-counter" by 1 */
counter-increment: my-counter;

/* Decreases "my-counter" by 1 */
counter-increment: my-counter -1;

/* Increases "counter1" by 1 and decreases "counter2" by 4 */
counter-increment: counter1 counter2 -4;

/* Increases "page" by 1, "section" by 2, while "chapter" doesn't change */
counter-increment: chapter 0 section 2 page;

/* Do not increment/decrement anything: used to override less specific rules */
counter-increment: none;

/* Global values */
counter-increment: inherit;
counter-increment: initial;
counter-increment: revert;
counter-increment: revert-layer;
counter-increment: unset;
```

Values

This property is specified as a space-separated list of `<custom-ident>` values, each optionally followed by an `<integer>`, or the keyword `none`:

`<custom-ident>`

Specifies the name of the counter to increase or decrease.

`<integer>`

Specifies the value to add to the counter. If the integer is preceded by a `-` sign, the value will be subtracted from the counter. Defaults to `1` if no value is specified.

`none`

Indicates that no counter must be increased or decreased. This value can also be used to cancel all counters from being increased or decreased in more specific rules. This is the default value of the property.

> **Note:** Using the `none` value prevents all counters from being increased or decreased for the selected elements where this rule applies. To prevent only specific counters from being increased or decreased, set the `integer` value as `0` on the relevant counter(s).

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
counter-increment =
  [ <counter-name> <integer>? ]+  |
  none

<counter-name> =
  <custom-ident>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Lists and Counters Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Decreasing the counter value

In this example, we display a sequence of numbers counting backward. To do this, we use a counter to display numbers starting from 100 and decreasing by 7 each time.

HTML

```
<div>
  <i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i> <i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i> <i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i>
```

```
counter-reset: chapter-count -5;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="chapters">
    <h1>Alice's Adventures in Wonderland</h1>
    <h2>Down the Rabbit-Hole</h2>
    <h2 id="example-element">The Pool of Tears</h2>
    <h2>A Caucus-Race and a Long Tale</h2>
    <h2>The Rabbit Sends in a Little Bill</h2>
  </div>
</section>
```

```
#default-example {
  text-align: left;
  counter-reset: chapter-count;
}

#example-element {
  background-color: lightblue;
  color: black;
}

h2 {
  counter-increment: chapter-count;
  font-size: 1em;
}

h2::before {
  content: "Chapter " counters(chapter-count, ".") ": ";
}
```

Syntax

```
/* Keyword value */
counter-reset: none;

/* Regular counters with default initial values */
counter-reset: my-counter;
counter-reset: another-counter;

/* Regular counters with an initial value */
counter-reset: my-counter -3;
counter-reset: another-counter 15;

/* Reversed counters */
counter-reset: reversed(my-counter);
counter-reset: reversed(my-counter) 3;
counter-reset: reversed(another-counter) 15;

/* Multiple counters */
counter-reset: my-counter -3 another-counter 15;
counter-reset: reversed(pages) items 1 reversed(sections) 4;

/* Global values */
counter-reset: inherit;
counter-reset: initial;
counter-reset: revert;
counter-reset: revert-layer;
counter-reset: unset;
```

Values

This property is specified as a space-separated list of counter or reversed counter names, each optionally followed by an `<integer>`, or the keyword `none`:

`<custom-ident>`

Specifies the counter name to create and initialize. The `reversed()` functional notation can be used to create a reversed counter.

`<integer>`

Specifies the initial value to set on the newly created counter. If not specified, defaults to `0` for regular counters. The initial value for reversed counters is calculated automatically.

`none`

Specifies that no counters are created.

Description

The `counter-reset` property can be used to create and initialize named counters or reversed counters to number elements in ascending or descending order.

The `none` value can be used to override a `counter-reset` declaration in a rule with lower specificity.

> **Warning:** There is a difference between `counter-reset` and `counter-set` properties. After creating a counter using `counter-reset`, you can adjust its value by using the `counter-set` property. This is counterintuitive because, despite its name, the `counter-reset` property creates and initializes counters, while the `counter-set` property resets the value of existing counters.

Separate multiple counter names or name-value pairs with spaces. For counter names, regular counters use the format `<counter-name>` and reversed counters use the format `reversed(<counter-name>)`, where `<counter-name>` is a `<custom-ident>` or `list-item` for the built-in `<ol>` counter.

Default initial values

Declaring regular and reversed counters without including an integer enables you to implement the two most common numbering patterns: counting up from `1` to the number of elements and counting down from the number of elements to `1`, with increments or decrements of `1`. By specifying an initial value for a named counter, you can change its starting value. The increment or decrement can be adjusted by using the `counter-increment` property.

The following example creates three counters. The `chapter` and `page` counters are set to the initial default value of `0`, while `section` is set to `4`:

```
h1 {
  counter-reset: chapter section 4 page;
```

```
}
```

Reversed counters created without an `<integer>` have an automatically calculated initial value. The browser counts the number of elements in the set, with the calculated initial value ensuring that if the `counter-increment` is set to `-1`, thereby decrementing the named counter by `1` for each element, the counter will be at `1` on the last element in the set.

The following example creates two reversed counters, `chapter` and `section`, and a regular counter, `pages`. The `section` counter starts at `10`, and `pages` uses the default initial value of `0`. The initial value of the `chapter` counter is calculated automatically. The counter is decremented by `1` each time an `<h2>` is encountered, so its value is `1` on the last `<h2>`.

```
h1 {
  counter-reset: reversed(chapter) reversed(section) 10 pages;
}
h2 {
  counter-increment: chapter -1;
}
```

Built-in <code>list-item</code> counter

Ordered lists (`<ol>`) come with built-in `list-item` counters that control their numbering. These counters automatically increase or decrease by one with each list item. The `counter-reset` property can be used to reset the `list-item` counters. Like with other counters, you can override the default increment value for `list-item` counters by using the `counter-increment` property.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
counter-reset =
  [ <counter-name> <integer>? | <reversed-counter-name> <integer>? ]+  |
  none

<counter-name> =
  <custom-ident>

<integer> =
  <number-token>

<reversed-counter-name> =
  reversed( <counter-name> )
```

This syntax reflects the latest standard as per CSS Lists and Counters Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Overriding the <code>list-item</code> counter

In this example, the `counter-reset` property is used to set a starting value for an implicit `list-item` counter.

HTML

We include an ordered list (`<ol>`) containing five list items (`<li>`).

```
<ol>
  <li>First</li>
  <li>Second</li>
  <li>Third</li>
  <li>Fourth</li>
  <li>Fifth</li>
</ol>
```

CSS

Using `counter-reset`, we set the implicit `list-item` counter to start at a value other than the default `1`:

```
ol {
  counter-reset: list-item 3;
}
```

Result

The first item is numbered `4`. This is similar to the effect of writing `<ol start="4">` in HTML.

Using a reverse counter

In this example, the `reversed()` function is used to create a reversed counter named `priority` that numbers five tasks.

HTML

We include an unordered list (`<ul>`) containing five list items (`<li>`).

```
<ul class="stack">
  <li>Task A</li>
  <li>Task B</li>
  <li>Task C</li>
  <li>Task D</li>
  <li>Task E</li>
</ul>
```

CSS

We create a reversed counter named `priority` on the `<ul>` and remove the default bullets. We decrement the counter by `1` on each `<li>`. We then use generated content to display the counter value before each list item's content.

```
.stack {
  counter-reset: reversed(priority);
  list-style: none;
}
li {
  counter-increment: priority -1;
}
li::before {
  content: counter(priority) ". ";
}
```

```
@supports not (counter-reset: reversed(priority)) {
  body::before {
    content: "Your browser doesn't support the reversed() function.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

The items are numbered in reverse order from `5` to `1`. Notice that we haven't specified the counter's initial value. The browser automatically calculates the initial value at layout time.

See also

- Using CSS Counters guide
- `counter-increment` property
- `counter-set` property
- `@counter-style` at-rule
- `counter()` and `counters()` functions
- `content` property
- `::marker` pseudo-class
- CSS lists and counters module
- CSS counter styles module

counter-set CSS property

The **`counter-set`** CSS property sets CSS counters on the element to the given values.

If the counters don't exist the `counter-set` property creates a new counter for each named counter in the list of space-separated counter and value pairs. However, to create a new counter it is recommended to use the `counter-reset` CSS property.

If a named counter in the list is missing a value, the value of the counter will be set to `0`.

> **Note:** The counter's value can be incremented or decremented using the `counter-increment` CSS property.

Try it

```
counter-set: none;
```

```
counter-set: chapter-count 0;
```

```
counter-set: chapter-count;
```

```
counter-set: chapter-count 5;
```

```
counter-set: chapter-count -5;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="chapters">
    <h1>Alice's Adventures in Wonderland</h1>
    <h2>Down the Rabbit-Hole</h2>
    <h2 id="example-element">The Pool of Tears</h2>
    <h2>A Caucus-Race and a Long Tale</h2>
    <h2>The Rabbit Sends in a Little Bill</h2>
  </div>
</section>
```

```
#default-example {
  text-align: left;
  counter-set: chapter-count;
}

#example-element {
  background-color: #37077c;
  color: white;
}

h2 {
  counter-increment: chapter-count;
  font-size: 1em;
}

h2::before {
  content: "Chapter " counter(chapter-count) ": ";
}
```

Syntax

```
/* Set "my-counter" to 0 */
counter-set: my-counter;
```

```
/* Set "my-counter" to -1 */
counter-set: my-counter -1;

/* Set "counter1" to 1, and "counter2" to 4 */
counter-set: counter1 1 counter2 4;

/* Cancel any counter that could have been set in less specific rules */
counter-set: none;

/* Global values */
counter-set: inherit;
counter-set: initial;
counter-set: revert;
counter-set: revert-layer;
counter-set: unset;
```

Values

This property is specified as either a space-separated list of `<custom-ident>` values, each optionally followed by an `<integer>`, or the keyword `none`:

`<custom-ident>`

> The name of the counter to set.

`<integer>`

> The value to set the counter to on each occurrence of the element. Defaults to `0` if not specified. If there isn't currently a counter of the given name on the element, the element will create a new counter of the given name with a starting value of `0` (though it may then immediately set or increment that value to something different).

`none`

> No counter set is to be performed. This can be used to override a `counter-set` defined in a less specific rule.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
counter-set =
  [ <counter-name> <integer>? ]+  |
  none

<counter-name> =
  <custom-ident>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Lists and Counters Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting named counters

```
h1 {
  counter-set: chapter section 1 page;
  /* Sets the chapter and page counters to 0,
     and the section counter to 1 */
}
```

See also

- Using CSS Counters
- `counter-increment`
- `counter-reset`
- `@counter-style`
- `counter()` and `counters()` functions
- `content` property
- CSS lists and counters module
- CSS counter styles module

cursor CSS property

The **`cursor`** CSS property sets the mouse cursor, if any, to show when the mouse pointer is over an element.

The cursor setting should inform users of the mouse operations that can be performed at the current location, including: text selection, activating help or context menus, copying content, resizing tables, and so on. You can specify either the *type* of cursor using a keyword, or load a specific icon to use (with optional fallback images and mandatory keyword as a final fallback).

Try it

```
cursor: pointer;
```

```
cursor: help;
```

```
cursor: wait;
```

```
cursor: crosshair;
```

```
cursor: not-allowed;
```

```
cursor: zoom-in;
```

```
<section class="default-example container" id="default-example">
  <div id="example-element">
    Move over this element to see the cursor style.
  </div>
</section>
```

```
#example-element {
  display: flex;
  background-color: #1766aa;
  color: white;
  height: 180px;
  width: 360px;
  justify-content: center;
  align-items: center;
  font-size: 14pt;
  padding: 5px;
}
```

Syntax

```
/* Keyword values */
cursor: auto;
cursor: pointer;
/* … */
cursor: zoom-out;

/* URL with mandatory keyword fallback */
cursor: url("hand.cur"), pointer;

/* URL and coordinates, with mandatory keyword fallback */
cursor:
  url("cursor_1.png") 4 12,
  auto;
cursor:
  url("cursor_2.png") 2 2,
  pointer;

/* URLs and fallback URLs (some with coordinates), with mandatory keyword fallback */
cursor:
  url("cursor_1.svg") 4 5,
  url("cursor_2.svg"),
  /* …, */ url("cursor_n.cur") 5 5,
  progress;

/* Global values */
cursor: inherit;
cursor: initial;
cursor: revert;
cursor: revert-layer;
cursor: unset;
```

The `cursor` property is specified as zero or more `<url>` values, separated by commas, followed by a single mandatory keyword value. Each `<url>` should point to an image file. The browser will try to load the first image specified, falling back to the next if it can't, and falling back to the keyword value if no images could be loaded (or if none were specified).

Each `<url>` may be optionally followed by a pair of space-separated numbers, which set the `<x>` and `<y>` coordinates of the cursor's hotspot relative to the top-left corner of the image.

Values

`<url>` Optional

A `url()` or a comma separated list `url(), url(), …`, pointing to an image file. More than one `<url>` may be provided as fallbacks, in case some cursor image types are not supported. A non-URL fallback (one or more of the keyword values) *must* be at the end of the fallback list.

`<x>`, `<y>` Optional

Optional x- and y-coordinates indicating the cursor hotspot; the precise position within the cursor that is being pointed to.

The numbers are in units of image pixels. They are relative to the top left corner of the image, which corresponds to `0 0`, and are clamped within the boundaries of the cursor image. If these values are not specified, they may be read from the file itself, and will otherwise default to the top-left corner of the image.

`keyword`

A keyword value *must* be specified, indicating either the type of cursor to use, or the fallback cursor to use if all specified icons fail to load.

The available keywords are listed in the table below. Other than `none`, which means no cursor, there is an image showing how the cursors used to be rendered. You can hover your mouse over the table rows to see the effect of the different cursor keyword values on your browser today.

Category	Keyword	Example	Description
General	`auto`		The UA will determine the cursor to display based on the current context. E.g., equivalent to `text` when hovering text.
	`default`		The platform-dependent default cursor. Typically an arrow.
	`none`		No cursor is rendered.
Links & status	`context-menu`		A context menu is available.
	`help`		Help information is available.
	`pointer`		The cursor is a pointer that indicates a link. Typically an image of a pointing hand.
	`progress`		The program is busy in the background, but the user can still interact with the interface (in contrast to `wait`).
	`wait`		The program is busy, and the user can't interact with the interface (in contrast to `progress`). Sometimes an image of an hourglass or a watch.

Category	Keyword	Example	Description
Selection	`cell`		The table cell or set of cells can be selected.
	`crosshair`		Cross cursor, often used to indicate selection in a bitmap.
	`text`		The text can be selected. Typically the shape of an I-beam.
	`vertical-text`		The vertical text can be selected. Typically the shape of a sideways I-beam.
Drag & drop	`alias`		An alias or shortcut is to be created.
	`copy`		Something is to be copied.
	`move`		Something is to be moved.
	`no-drop`		An item may not be dropped at the current location. Firefox bug 275173: On Windows and macOS, `no-drop` is the same as `not-allowed`.
	`not-allowed`		The requested action will not be carried out.
	`grab`		Something can be grabbed (dragged to be moved).
	`grabbing`		Something is being grabbed (dragged to be moved).
Resizing & scrolling	`all-scroll`		Something can be scrolled in any direction (panned). Firefox bug 275174: On Windows, `all-scroll` is the same as `move`.
	`col-resize`		The item/column can be resized horizontally. Often rendered as arrows pointing left and right with a vertical bar separating them.
	`row-resize`		The item/row can be resized vertically. Often rendered as arrows pointing up and down with a horizontal bar separating them.
	`n-resize`		Some edge is to be moved. For example, the `se-resize` cursor is used when the movement starts from the *south-east* corner of the box. In some environments, an equivalent bidirectional resize cursor is shown. For example, `n-resize` and `s-resize` are the same as `ns-resize`.
	`e-resize`		
	`s-resize`		
	`w-resize`		
	`ne-resize`		
	`nw-resize`		
	`se-resize`		
	`sw-resize`		
	`ew-resize`		Bidirectional resize cursor.
	`ns-resize`		
	`nesw-resize`		
	`nwse-resize`		
Zooming	`zoom-in`		Something can be zoomed (magnified) in or out.
	`zoom-out`		

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
cursor =
  [ <cursor-image> , ]* <cursor-predefined>

<cursor-image> =
  [ <url> | <url-set> ] <number>{2}?

<cursor-predefined> =
  auto            |
  default         |
  none            |
  context-menu    |
  help            |
```

```
  pointer       |
  progress      |
  wait          |
  cell          |
  crosshair     |
  text          |
  vertical-text |
  alias         |
  copy          |
  move          |
  no-drop       |
  not-allowed   |
  grab          |
  grabbing      |
  e-resize      |
  n-resize      |
  ne-resize     |
  nw-resize     |
  s-resize      |
  se-resize     |
  sw-resize     |
  w-resize      |
  ew-resize     |
  ns-resize     |
  nesw-resize   |
  nwse-resize   |
  col-resize    |
  row-resize    |
  all-scroll    |
  zoom-in       |
  zoom-out
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Usage notes

Icon size limits

While the specification does not limit the `cursor` image size, user agents commonly restrict them to avoid potential misuse. For example, on Firefox and Chromium cursor images are restricted to 128x128 pixels by default, but it is recommended to limit the cursor image size to 32x32 pixels. Cursor changes using images that are larger than the user-agent maximum supported size will generally just be ignored.

Supported image file formats

User agents are required by the specification to support PNG files, SVG v1.1 files in secure static mode that contain a natural size, and any other non-animated image file formats that they support for images in other properties. Desktop browsers also broadly support the `.cur` file format.

The specification further indicates that user agents *should* also support SVG v1.1 files in secure animated mode that contain a natural size, along with any other animated images file formats they support for images in other properties. User agents *may* support both static and animated SVG images that do not contain a natural size.

iPadOS

iPadOS supports pointer devices like trackpads and mouses. By default, the iPad cursor is displayed as a circle, and the only supported value that will change an appearance of the pointer is `text`.

Other notes

Cursor changes that intersect toolbar areas are commonly blocked to avoid spoofing.

Examples

Setting cursor types

```
.foo {
  cursor: crosshair;
}

.bar {
  cursor: zoom-in;
}

/* A fallback keyword value is required when using a URL */
.baz {
  cursor: url("hyper.cur"), auto;
}
```

See also

- `pointer-events`
- `<url>` type
- SVG `cursor` attribute

cx CSS property

The **cx** CSS property defines the x-axis center point of an SVG `<circle>` or `<ellipse>` element. If present, it overrides the element's `cx` attribute.

> **Note:** While SVG the `cx` attribute is relevant to the SVG `<radialGradient>` element, the `cx` property only applies to `<circle>` and `<ellipse>` elements nested in an `<svg>`. It doesn't apply to `<radialGradient>` or other SVG elements nor to HTML elements or pseudo-elements.

Syntax

```
/* length and percentage values */
cx: 20px;
cx: 20%;

/* Global values */
cx: inherit;
cx: initial;
cx: revert;
cx: revert-layer;
cx: unset;
```

Values

The `<length>` and `<percentage>` values denote the horizontal center of the circle or ellipse.

`<length>`

As an absolute or relative length, it can be expressed in any unit allowed by the CSS `<length>` data type. Negative values are invalid.

`<percentage>`

Percentages refer to the width of the current SVG viewport.

Formal definition

Initial value	`0`
Applies to	`<ellipse>` and `<circle>` elements in `<svg>`
Inherited	no
Percentages	refer to the width of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
cx =
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the x-axis coordinate of a circle and ellipse

This example demonstrates the basic use case of `cx`, and how the CSS `cx` property takes precedence over the `cx` attribute.

HTML

We include two identical `<circle>` and two identical `<ellipse>` elements in an SVG; their `cx` attribute values are `50` and `150`, respectively.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="30" />
  <circle cx="50" cy="50" r="30" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
</svg>
```

CSS

With CSS, we style only the first circle and first ellipse, allowing their twin shapes to use default styles (with `fill` defaulting to black). We use the `cx` property to override the value of the SVG `cx` attribute and also give it a `fill` and `stroke` to differentiate the first shapes in each pair from their twin. The browser renders SVG images as `300px` wide and `150px` tall by default.

```
svg {
  border: 1px solid;
}

circle:first-of-type {
  cx: 30px;
  fill: lightgreen;
  stroke: black;
}
ellipse:first-of-type {
  cx: 180px;
  fill: pink;
  stroke: black;
}
```

Results

The style circle's center is `30px` from the left edge of the SVG viewport and the styled ellipse is `180px` from that edge, as defined in the CSS `cx` property values. The unstyled shapes centers are `50px` and `150px` from the left edge of the SVG viewport, as defined in their SVG `cx` attribute values.

x-axis coordinates as percentage values

This example demonstrates using percentage values for `cx`.

HTML

We use the same markup as the previous example.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="30" />
  <circle cx="50" cy="50" r="30" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
</svg>
```

CSS

We use CSS which is similar to the previous example. The only difference is the CSS `cx` property value; in this case, we use percentage values of `30%` for the `<circle>` and `80%` for the `<ellipse>`.

```
svg {
  border: 1px solid;
}
```

```
circle:first-of-type {
  cx: 30%;
  fill: lightgreen;
  stroke: black;
}
ellipse:first-of-type {
  cx: 80%;
  fill: pink;
  stroke: black;
}
```

Results

When using percentage values for `cx`, the values are relative to the width of the SVG viewport. Here, the x-axis coordinates of the center of the style circle and ellipse are `30%` and `80%`, respectively, of the width of the current SVG viewport. As the width defaulted to `300px`, the `cx` values are `90px` and `240px` from the SVG viewport's left edge.

See also

- SVG `cx` attribute
- Geometry properties: `cx`, `cy`, `r`, `rx`, `ry`, `x`, `y`, `width`, `height`
- `fill`
- `stroke`
- `paint-order`
- `border-radius` shorthand property
- `radial-gradient`
- `<basic-shape>` data type

cy CSS property

The **`cy`** CSS property defines the y-axis center point of an SVG `<circle>` or `<ellipse>` elements. If present, it overrides the element's `cy` attribute.

> **Note:** While the SVG `<radialGradient>` element supports the `cy` attribute, the `cy` property only applies to `<circle>` and `<ellipse>` elements nested in an `<svg>`. This attribute does not apply to `<radialGradient>` or other SVG elements nor to HTML elements or pseudo-elements.

Syntax

```
/* length and percentage values */
cy: 3px;
cy: 20%;

/* Global values */
cy: inherit;
cy: initial;
cy: revert;
cy: revert-layer;
cy: unset;
```

Values

The `<length>` and `<percentage>` values denote the vertical center of the circle or ellipse.

`<length>`

As an absolute or relative length, it can be expressed in any unit allowed by the CSS `<length>` data type. Negative values are invalid.

`<percentage>`

Percentages refer to the height of the current SVG viewport.

Formal definition

Initial value	`0`
Applies to	`<ellipse>` and `<circle>` elements in `<svg>`
Inherited	no
Percentages	refer to the height of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
cy =
  <length-percentage>

<length-percentage> =
  <length>     |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the y-axis coordinate of a circle and ellipse

In this example, we have two identical `<circle>` and two identical `<ellipse>` elements in an SVG; their `cy` attribute values ar `50` and `150`, respectively.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="30" />
  <circle cx="50" cy="50" r="30" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
</svg>
```

With CSS, we style only the first circle and first ellipse, allowing their twin shapes to use default styles (with `fill` defaulting to black). We use the `cy` property to override the value of the SVG `cy` attribute and also give it a `fill` and `stroke` to differentiate the first shapes in each pair from their twin. The browser renders SVG images as `300px` wide and `150px` tall by default.

```
svg {
  border: 1px solid;
}

circle:first-of-type {
  cy: 30px;
  fill: lightgreen;
  stroke: black;
}
ellipse:first-of-type {
  cy: 100px;
  fill: pink;
  stroke: black;
}
```

The styled circle's center is `30px` from the top edge of the SVG viewport and the styled ellipse is `100px` from that edge, as defined in the CSS `cy` property values. The unstyled shapes centers are both `50px` from the top edge of the SVG viewport, as defined in their SVG `cy` attribute values.

y-axis coordinates as percentage values

In this example, we use the same markup as the previous example. The only difference is the CSS `cy` property value; in this case, we use percentage values of `30%` and `50%`.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="30" />
  <circle cx="50" cy="50" r="30" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
  <ellipse cx="150" cy="50" rx="20" ry="40" />
</svg>
```

```
svg {
  border: 1px solid;
}

circle:first-of-type {
  cy: 30%;
  fill: lightgreen;
  stroke: black;
}
ellipse:first-of-type {
  cy: 50%;
  fill: pink;
  stroke: black;
}
```

In this case, the y-axis coordinates of the center of the circle and ellipse are `30%` and `50%` of the height of the current SVG viewport, respectively. As the image's height defaulted to `150px`, meaning the `cy` value is the equivalent of `45px` and `120px`.

See also

- SVG `cy` attribute
- Geometry properties: `cy`, `cx`, `r`, `rx`, `ry`, `x`, `y`, `width`, `height`
- `fill`
- `stroke`
- `paint-order`
- `border-radius` shorthand property
- `radial-gradient`
- `<basic-shape>` data type

d CSS property

The **`d`** CSS property defines a path to be drawn by the SVG `<path>` element. If present, it overrides the element's `d` attribute.

> **Note:** The `d` property only applies to `<path>` elements nested in an `<svg>`. It doesn't apply to other SVG elements nor to HTML elements or pseudo-elements.

Syntax

```
/* Default */
d: none;

/* Basic usage */
d: path("m 5,5 h 35 L 20,30 z");
d: path("M 0,25 l 50,150 l 200,50 z");
d: path("M 10,5 l 90,0 -80,80 0,-60 80,80 -90,0 z");

/* Global values */
d: inherit;
d: initial;
d: revert;
d: revert-layer;
d: unset;
```

Values

The value is either a `path()` function with a single `<string>` parameter or the keyword `none`:

`none`

No path is drawn.

`path(<string>)`

A `path()` function with a quoted data string parameter. The data string defines an SVG path. The SVG path data string contains path commands that implicitly use pixel units. An empty path is considered invalid.

Formal definition

Initial value	none
Applies to	`<path>` element in `<svg>`
Inherited	no
Computed value	as specified
Animation type	yes, as specified for `<basic-shape>`, otherwise no

Formal syntax

```
d =
  none      |
  <string>
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying path data

This example demonstrates the basic use case of `d`, and how the CSS `d` property takes precedence over the `d` attribute.

HTML

We include two identical `<path>` elements in an SVG; their `d` attribute values are `"m 5,5 h 90 v 90 h -90 v -90 z"`, which creates a `90px` square.

```
<svg>
  <path d="m 5,5 h 90 v 90 h -90 v -90 z" />
  <path d="m 5,5 h 90 v 90 h -90 v -90 z" />
</svg>
```

CSS

With CSS, we style both paths, providing a black `stroke` and semi-opaque red `fill`. We then use the `d` property to override the value of the SVG `d` attribute for the last path only. The browser renders SVG images as `300px` wide and `150px` tall by default.

```
svg {
  border: 1px solid;
}

path {
  fill: #ff333388;
  stroke: black;
}

path:last-of-type {
  d: path(
    "M10,30 A20,20 0,0,1 50,30 A20,20 0,0,1 90,30 Q90,60 50,90 Q10,60 10,30 z"
  );
}
```

Results

The second `<path>` is a heart, as defined in the CSS `d` property's `path()` function value. The unstyled `<path>`'s path remained a square, as defined in its SVG `d` attribute value.

Animating data paths

This example demonstrates animating the `d` attribute value.

HTML

We create a `<svg>` containing a single `<path>` element.

```
<svg>
  <path />
</svg>
```

CSS

We use the `d` attribute to define a heart with a line through it. We use CSS to define the `fill`, `stroke`, and `stroke-width` of that path, and add a two-second `transition`. We add a `:hover` style that contains a slightly different `path()` function; the path has the same number of data points as the default state, making the path animatable.

```
svg {
  border: 1px solid;
}

path {
  d: path(
    "M10,30 A20,20 0,0,1 50,30 A20,20 0,0,1 90,30 Q90,60 50,90 Q10,60 10,30 z M90,5 L5,90"
  );
  transition: all 2s;
  fill: none;
  stroke: red;
  stroke-width: 3px;
}

svg:hover path {
  d: path(
    "M10,30 A20,20 0,0,1 50,30 A20,20 0,0,1 90,30 Q90,60 50,90 Q10,60 10,30 z M5,5 L90,90"
  );
  stroke: black;
}
```

Results

To view the animation, hover over the SVG.

See also

- SVG `d` attribute
- `fill`
- `stroke`
- `path()` function
- `<basic-shape>` data type
- Overview of CSS shapes
- CSS shapes module

direction CSS property

> **Warning:** Where possible, authors are encouraged to avoid using the `direction` CSS property and use the HTML `dir` global attribute instead.

The **`direction`** CSS property sets the direction of text, table and grid columns, and horizontal overflow. Use `rtl` for languages written from right to left (like Hebrew or Arabic), and `ltr` for those written from left to right (like English and most other languages).

Note that text direction is usually defined within a document (e.g., with HTML's `dir` attribute) rather than through direct use of the `direction` property.

Try it

```
direction: ltr;
```

```
direction: rtl;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  padding: 0.75em;
  width: 80%;
  max-height: 300px;
  display: flex;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex: 1;
}
```

Syntax

```
/* Keyword values */
direction: ltr;
direction: rtl;

/* Global values */
direction: inherit;
direction: initial;
direction: revert;
direction: revert-layer;
direction: unset;
```

Values

This property is specified as one of the following keyword values:

`ltr`

Text and other elements go from left to right. This is the default value.

`rtl`

Text and other elements go from right to left.

For the `direction` property to have any effect on inline-level elements, the `unicode-bidi` property's value must be `embed` or `override`.

Description

The property sets the base text direction of block-level elements and the direction of embeddings created by the `unicode-bidi` property. It also sets the default alignment of text, block-level elements, and the direction that cells flow within a table or grid row.

Unlike the `dir` attribute in HTML, the `direction` property is not inherited from table columns into table cells, since CSS inheritance follows the document tree, and table cells are inside of rows but not inside of columns.

The `direction` and `unicode-bidi` properties are the only two properties which are not affected by the `all` shorthand property.

Formal definition

Initial value	`ltr`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
direction =
  ltr  |
  rtl
```

This syntax reflects the latest standard as per CSS Writing Modes Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting right-to-left direction

In the example below are two strings of text, both which are displaying using `direction: rtl`. While the Arabic text is displayed correctly with this setting, the English text now has a full stop in an unusual location.

```
blockquote {
  direction: rtl;
  width: 300px;
}
```

```
<blockquote>
  <p>This paragraph is in English but incorrectly goes right to left.</p>
</blockquote>

<blockquote>
  <p>هذه الفقرة باللغة العربية ، لذا يجب الانتقال من اليمين إلى اليسار.</p>
</blockquote>
```

See also

- `unicode-bidi`
- `writing-mode`
- SVG `direction` attribute
- The HTML `dir` global attribute
- Creating vertical form controls
- Handling different text directions

display CSS property

The **`display`** CSS property sets whether an element is treated as a block or inline box and the layout used for its children, such as flow layout, grid or flex.

Formally, the **`display`** property sets an element's inner and outer *display types*. The outer type sets an element's participation in flow layout; the inner type sets the layout of children. Some values of `display` are fully defined in their own individual specifications; for example the detail of what happens when `display: flex` is declared is defined in the CSS Flexible Box Model specification.

Try it

```
display: block;
```

```
display: inline flow-root;
```

```
display: none;
```

```
display: flex;
```

```
display: grid;
```

```
display: grid-lanes;
```

```
<p>
  Apply different <code>display</code> values on the dashed orange-bordered
  <code>div</code>, which contains three child elements.
</p>
<section class="default-example" id="default-example">
  <div class="example-container">
    Some text A.
    <div id="example-element">
      <div class="child">Child 1</div>
      <div class="child">Child 2</div>
      <div class="child">Child 3</div>
    </div>
    Some text B.
  </div>
</section>
```

```
.example-container {
  width: 100%;
  height: 100%;
}

code {
  background: #88888888;
}

#example-element {
  border: 3px dashed orange;
}

.child {
  display: inline-block;
  padding: 0.5em 1em;
  background-color: #ccccff;
  border: 1px solid #ababab;
  color: black;
}
```

Syntax

```
/* short display */
display: none;
display: contents;
display: block;
display: flow-root;
display: inline;
display: inline-block;
display: list-item;
display: inline list-item;
display: flex;
display: inline-flex;
display: grid;
display: inline-grid;
display: grid-lanes;
display: inline-grid-lanes;
display: table;
display: inline-table;

/* full display */
display: block flow;
display: block flow-root;
display: inline flow;
display: inline flow-root;
display: block flow list-item;
display: inline flow list-item;
display: block flex;
display: inline flex;
display: block grid;
display: inline grid;
display: block grid-lanes;
display: inline grid-lanes;
display: block table;
display: inline table;

/* global values */
display: inherit;
display: initial;
display: revert;
display: revert-layer;
display: unset;
```

The CSS `display` property is specified using keyword values.

Grouped values

The keyword values can be grouped into six value categories.

Outside

`<display-outside>`

These keywords specify the element's outer display type, which is essentially its role in flow layout:

`block`

The element generates a block box, generating line breaks both before and after the element when in the normal flow.

`inline`

The element generates one or more inline boxes that do not generate line breaks before or after themselves. In normal flow, the next element will be on the same line if there is space.

> **Note:** When a display property is specified with only an **outer** value (e.g., `display: block` or `display: inline`), the inner value defaults to `flow` (e.g., `display: block flow` and `display: inline flow`).

> **Note:** You may use the single-value syntax as a fallback for multi-keyword syntax, for example `display: inline flex` could have the following fallback
>
> ```
> .container {
>   display: inline-flex;
>   display: inline flex;
> }
> ```
>
> See Using the multi-keyword syntax with CSS display for more information.

Inside

`<display-inside>`

These keywords specify the element's inner display type, which defines the type of formatting context that its contents are laid out in (assuming it is a non-replaced element). When one of these keywords is used by itself as a single value, the element's outer display type defaults to `block` (with the exception of `ruby`, which defaults to `inline`).

`flow`

The element lays out its contents using flow layout (block-and-inline layout).

If its outer display type is `inline`, and it is participating in a block or inline formatting context, then it generates an inline box. Otherwise it generates a block box.

Depending on the value of other properties (such as `position`, `float`, or `overflow`) and whether it is itself participating in a block or inline formatting context, it either establishes a new block formatting context (BFC) for its contents or integrates its contents into its parent formatting context.

`flow-root`

The element generates a block box that establishes a new block formatting context, defining where the formatting root lies.

`table`

These elements behave like HTML `<table>` elements. It defines a block-level box.

`flex`

The element behaves like a block-level element and lays out its content according to the flexbox model.

`grid`

The element behaves like a block-level element and lays out its content according to the grid model.

`grid-lanes`

The element behaves like a block-level element and lays out its content using grid lanes layout. Columns are defined by `grid-template-columns` and behave like a strict grid, while items are packed in the block direction to fill gaps between items of different sizes. See Grid lanes layout for details.

`ruby`

The element behaves like an inline-level element and lays out its content according to the ruby formatting model. It behaves like the corresponding HTML `<ruby>` elements.

> **Note:** When a display property is specified with only an **inner** value (e.g., `display: flex` or `display: grid`), the outer value defaults to `block` (e.g., `display: block flex` and `display: block grid`).

List Item

`<display-listitem>`

The element generates a block box for the content and a separate list-item inline box.

A single value of `list-item` will cause the element to behave like a list item. This can be used together with `list-style-type` and `list-style-position`.

`list-item` can also be combined with any `<display-outside>` keyword and the `flow` or `flow-root` `<display-inside>` keyword.

> **Note:** If no inner value is specified, it will default to `flow`. If no outer value is specified, the principal box will have an outer display type of `block`.

Internal

`<display-internal>`

Some layout models such as `table` and `ruby` have a complex internal structure, with several different roles that their children and descendants can fill. This section defines those "internal" display values, which only have meaning within that particular layout mode.

`table-row-group`

These elements behave like `<tbody>` HTML elements.

`table-header-group`

These elements behave like `<thead>` HTML elements.

`table-footer-group`

These elements behave like `<tfoot>` HTML elements.

`table-row`

These elements behave like `<tr>` HTML elements.

`table-cell`

These elements behave like `<td>` HTML elements.

`table-column-group`

These elements behave like `<colgroup>` HTML elements.

`table-column`

These elements behave like `<col>` HTML elements.

`table-caption`

These elements behave like `<caption>` HTML elements.

`ruby-base`

These elements behave like `<rb>` HTML elements.

`ruby-text`

These elements behave like `<rt>` HTML elements.

`ruby-base-container`

These elements are generated as anonymous boxes.

`ruby-text-container`

These elements behave like `<rtc>` HTML elements.

Box

`<display-box>`

These values define whether an element generates display boxes at all.

`contents`

These elements don't produce a specific box by themselves. They are replaced by their pseudo-box and their child boxes. Please note that the CSS Display Level 3 spec defines how the `contents` value should affect "unusual elements" — elements that aren't rendered purely by CSS box concepts such as replaced elements. See Appendix B: Effects of display: contents on Unusual Elements for more details.

`none`

Turns off the display of an element so that it has no effect on layout (the document is rendered as though the element did not exist). All descendant elements also have their display turned off. To have an element take up the space that it would normally take, but without actually rendering anything, use the `visibility` property instead.

Precomposed

`<display-legacy>`

CSS 2 used a single-keyword, precomposed syntax for the `display` property, requiring separate keywords for block-level and inline-level variants of the same layout mode.

`inline-block`

The element generates a block box that will be flowed with surrounding content as if it were a single inline box (behaving much like a replaced element would).

It is equivalent to `inline flow-root`.

`inline-table`

The `inline-table` value does not have a direct mapping in HTML. It behaves like an HTML `<table>` element, but as an inline box, rather than a block-level box. Inside the table box is a block-level context.

It is equivalent to `inline table`.

`inline-flex`

The element behaves like an inline-level element and lays out its content according to the flexbox model.

It is equivalent to `inline flex`.

`inline-grid`

The element behaves like an inline-level element and lays out its content according to the grid model.

It is equivalent to `inline grid`.

`inline-grid-lanes`

The element behaves like an inline-level element and lays out its content using grid lanes layout. Rows are defined by `grid-template-rows` and behave like a strict grid, while items are packed in the inline direction to fill gaps between items of different sizes. See Grid lanes layout for details.

It is equivalent to `inline grid-lanes`.

Which syntax should you use?

The CSS display module describes a multi-keyword syntax for values you can use with the `display` property to explicitly define **outer** and **inner** display. The single keyword values (precomposed `<display-legacy>` values) are supported for backward-compatibility.

For example, using two values you can specify an inline flex container as follows:

```
.container {
  display: inline flex;
}
```

This can also be specified using the legacy single value:

```
.container {
  display: inline-flex;
}
```

For more information on these changes, see the Using the multi-keyword syntax with CSS display guide.

Description

The individual pages for the different types of value that `display` can have set on it feature multiple examples of those values in action — see the Syntax section. In addition, see the following material, which covers the various values of display in depth.

Multi-keyword values

- Using the multi-keyword syntax with CSS display

CSS Flow Layout (display: block, display: inline)

- Block and inline layout in normal flow
- Flow layout and overflow
- Flow layout and writing modes
- Introduction to formatting contexts
- In flow and out of flow

display: flex

- Basic concepts of flexbox
- Aligning items in a flex container
- Controlling ratios of flex items along the main axis
- Mastering wrapping of flex items
- Ordering flex items
- Relationship of flexbox to other layout methods
- Typical use cases of flexbox

display: grid

- Basic concepts of grid layout
- Relationship to other layout methods
- Line-based placement
- Grid template areas
- Layout using named grid lines
- Auto-placement in grid layout
- Aligning items in CSS grid layout
- Grids, logical values and writing modes
- CSS grid layout and accessibility

- Realizing common layouts using grids
- Grid lanes layout

Animating display

Supporting browsers animate `display` with a discrete animation type. This generally means that the property will flip between two values 50% through animating between the two.

There is one exception, which is when animating to or from `display: none`. In this case, the browser will flip between the two values so that the animated content is shown for the entire animation duration. So for example:

- When animating `display` from `none` to `block` (or another visible `display` value), the value will flip to `block` at `0%` of the animation duration so it is visible throughout.
- When animating `display` from `block` (or another visible `display` value) to `none`, the value will flip to `none` at `100%` of the animation duration so it is visible throughout.

This behavior is useful for creating entry/exit animations where you want to for example remove a container from the DOM with `display: none`, but have it fade out with `opacity` rather than disappearing immediately.

When animating `display` with CSS animations, you need to provide the starting `display` value in an explicit keyframe (for example using `0%` or `from`). See Using CSS animations for an example.

When animating `display` with CSS transitions, two additional features are needed:

- `@starting-style` provides starting values for properties you want to transition from when the animated element is first shown. This is needed to avoid unexpected behavior. By default, CSS transitions are not triggered on an element's first style update or when the `display` type changes from `none` to another type.
- `transition-behavior: allow-discrete` needs to be set on the `transition-property` declaration (or the `transition` shorthand) to enable `display` transitions.

For examples of transitioning the `display` property, see the `@starting-style` and `transition-behavior` pages.

Accessibility

display: none

Using a `display` value of `none` on an element will remove it from the accessibility tree. This will cause the element and all its descendant elements to no longer be announced by screen reading technology.

If you want to visually hide the element, a more accessible alternative is to use a combination of properties to remove it visually from the screen but still make it available to assistive technology such as screen readers.

While `display: none` hides content from the accessibility tree, elements that are hidden but are referenced from visible elements' `aria-describedby` or `aria-labelledby` attributes are exposed to assistive technologies.

display: contents

Current implementations in some browsers will remove from the accessibility tree any element with a `display` value of `contents` (but descendants will remain). This will cause the element itself to no longer be announced by screen reading technology. This is incorrect behavior according to the CSS specification.

- More accessible markup with display: contents | Hidde de Vries
- Display: Contents Is Not a CSS Reset | Adrian Roselli

Tables

In some browsers, changing the `display` value of a `<table>` element to `block`, `grid`, or `flex` will alter its representation in the accessibility tree. This will cause the table to no longer be announced properly by screen reading technology.

- Hidden content for better a11y | Go Make Things
- MDN Understanding WCAG, Guideline 1.3 explanations
- Understanding Success Criterion 1.3.1 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`inline`
Applies to	all elements
Inherited	no
Computed value	as the specified value, except for positioned and floating elements and the root element. In both cases the computed value may be a keyword other than the one specified.
Animation type	Discrete behavior except when animating to or from `none` is visible for the entire duration

Formal syntax

```
display =
  [ <display-outside> || <display-inside> ]        |
  <display-listitem>                               |
  <display-internal>                               |
  <display-box>                                    |
  <display-legacy>                                 |
  grid-lanes                                       |
  inline-grid-lanes                                |
  <display-outside> || [ <display-inside> | math ]

<display-outside> =
  block   |
  inline  |
  run-in

<display-inside> =
  flow       |
  flow-root  |
  table      |
  flex       |
  grid       |
  ruby

<display-listitem> =
  <display-outside>?    &&
  [ flow | flow-root ]?  &&
  list-item

<display-internal> =
  table-row-group      |
  table-header-group   |
  table-footer-group   |
  table-row            |
  table-cell           |
```

```
  table-column-group   |
  table-column         |
  table-caption        |
  ruby-base            |
  ruby-text            |
  ruby-base-container  |
  ruby-text-container

<display-box> =
  contents  |
  none

<display-legacy> =
  inline-block  |
  inline-table  |
  inline-flex   |
  inline-grid
```

This syntax reflects the latest standard as per CSS Display Module Level 4, CSS Grid Layout Module Level 3, MathML Core. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

display value comparison

In this example we have two block-level container elements, each one with three inline children. Below that, we have a select menu that allows you to apply different `display` values to the containers, allowing you to compare and contrast how the different values affect the element's layout, and that of their children.

We have included `padding` and `background-color` on the containers and their children, so that it is easier to see the effect the display values are having.

HTML

```
<article class="container">
  <span>First</span>
  <span>Second</span>
  <span>Third</span>
</article>

<article class="container">
  <span>First</span>
  <span>Second</span>
  <span>Third</span>
</article>

<div>
  <label for="display">Choose a display value:</label>
  <select id="display">
    <option selected>block</option>
    <option>block flow</option>
    <option>inline</option>
    <option>inline flow</option>
    <option>flow</option>
    <option>flow-root</option>
    <option>block flow-root</option>
    <option>table</option>
    <option>block table</option>
    <option>flex</option>
    <option>block flex</option>
    <option>grid</option>
    <option>block grid</option>
    <option>grid-lanes</option>
    <option>block grid-lanes</option>
    <option>list-item</option>
    <option>block flow list-item</option>
    <option>inline flow list-item</option>
    <option>block flow-root list-item</option>
    <option>inline flow-root list-item</option>
    <option>contents</option>
    <option>none</option>
    <option>inline-block</option>
    <option>inline flow-root</option>
    <option>inline-table</option>
    <option>inline table</option>
    <option>inline-flex</option>
    <option>inline flex</option>
    <option>inline-grid</option>
    <option>inline grid</option>
    <option>inline-grid-lanes</option>
    <option>inline grid-lanes</option>
  </select>
</div>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
  letter-spacing: 1px;
  padding-top: 10px;
}

article {
  background-color: red;
}

article span {
  background-color: black;
  color: white;
  margin: 1px;
}

article,
span {
  padding: 10px;
  border-radius: 7px;
```

```
}

article,
div {
  margin: 20px;
}
```

JavaScript

```
const articles = document.querySelectorAll(".container");
const select = document.querySelector("select");

function updateDisplay() {
  articles.forEach((article) => {
    article.style.display = select.value;
  });
}

select.addEventListener("change", updateDisplay);

updateDisplay();
```

Result

Note that some multi-keyword values are added for illustration which have the following equivalents:

- `block` = `block flow`
- `inline` = `inline flow`
- `flow` = `block flow`
- `flow-root` = `block flow-root`
- `table` = `block table`
- `flex` = `block flex`
- `grid` = `block grid`
- `list-item` = `block flow list-item`
- `inline-block` = `inline flow-root`
- `inline-table` = `inline table`
- `inline-flex` = `inline flex`
- `inline-grid` = `inline grid`

You can find more examples in the pages for each separate display type under Grouped values.

See also

- `visibility`, `float`, `position`
- `grid`, `flex`
- CSS ruby layout module
- SVG `display` attribute
- Block and inline layout in normal flow
- Introduction to formatting contexts
- Grid lanes layout

dominant-baseline CSS property

The **`dominant-baseline`** CSS property specifies the specific baseline used to align the box's text and inline-level contents. It also indicates the default alignment baseline of any boxes participating in baseline alignment in the box's alignment context. If present, it overrides the shape's `dominant-baseline` attribute.

Baselines are selected from the font baseline table. If there is no baseline table in the nominal font, or if the baseline table lacks an entry for the desired baseline, then the browser may use heuristics to determine the position of the desired baseline.

The `dominant-baseline` property is used to determine or re-determine a *scaled-baseline-table*. A scaled-baseline-table is a compound value with three components:

1. a baseline-identifier for the dominant-baseline,
2. a baseline-table, and
3. a baseline-table font-size.

Some values of `dominant-baseline` re-determine all three values. Others only re-establish the baseline-table font-size. When the initial value, `auto`, would give an undesired result, this property can be used to explicitly set the desired scaled-baseline-table.

> **Note:** The `dominant-baseline` property only has an effect on the `<text>`, `<textPath>`, and `<tspan>` SVG elements.

Syntax

```
/* Initial value */
dominant-baseline: auto;

/* Keyword values */
dominant-baseline: alphabetic;
dominant-baseline: central;
dominant-baseline: hanging;
dominant-baseline: ideographic;
dominant-baseline: mathematical;
dominant-baseline: middle;
dominant-baseline: text-bottom;
dominant-baseline: text-top;

/* Global values */
dominant-baseline: inherit;
dominant-baseline: initial;
dominant-baseline: revert;
dominant-baseline: revert-layer;
dominant-baseline: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

If this property is applied to a `<text>` element, then the computed value depends on the value of the `writing-mode` attribute.

If the `writing-mode` is horizontal, then the value of the dominant-baseline component is `alphabetic`. Otherwise, if the `writing-mode` is vertical, then the value of the dominant-baseline component is `central`.

If this property is applied to a `<tspan>`, or `<textPath>` element, then the dominant-baseline and the baseline-table components remain the same as those of the parent text content element.

If the computed `baseline-shift` value actually shifts the baseline, then the baseline-table font-size component is set to the value of the `font-size` attribute on the element on which the `dominant-baseline` attribute occurs, otherwise the baseline-table font-size remains the same as that of the element.

If there is no parent text content element, the scaled-baseline-table value is constructed as for `<text>` elements.

`alphabetic`

The baseline-identifier for the dominant-baseline is set to be `alphabetic`, the derived baseline-table is constructed using the `alphabetic` baseline-table in the font, and the baseline-table font-size is changed to the value of the element's `font-size` SVG attribute or the CSS `font-size`, if set.

`central`

The baseline-identifier for the dominant-baseline is set to be `central`. The derived baseline-table is constructed from the defined baselines in the font's baseline-table. That font baseline-table is chosen using the following priority order of baseline-table names: `ideographic`, `alphabetic`, `hanging`, `mathematical`. The baseline-table font-size is changed to the value of the element's `font-size` SVG attribute or the CSS `font-size`, if set.

`hanging`

The baseline-identifier for the dominant-baseline is set to be `hanging`, the derived baseline-table is constructed using the `hanging` baseline-table in the font, and the baseline-table font-size is changed to the value of the `font-size` SVG attribute of `font-size` CSS property on this element.

`ideographic`

The baseline-identifier for the dominant-baseline is set to be `ideographic`, the derived baseline-table is constructed using the `ideographic` baseline-table in the font, and the baseline-table font-size is changed to the value of the value of the element's `font-size` SVG attribute or the CSS `font-size`, if set.

`mathematical`

The baseline-identifier for the dominant-baseline is set to be `mathematical`, the derived baseline-table is constructed using the `mathematical` baseline-table in the font, and the baseline-table font-size is changed to the value of the value of the element's `font-size` SVG attribute or the CSS `font-size`, if set.

`middle`

The baseline-identifier for the dominant-baseline is set to be `middle`. The derived baseline-table is constructed from the defined baselines in a baseline-table in the font. That font baseline-table is chosen using the following priority order of baseline-table names: `ideographic`, `alphabetic`, `hanging`, `mathematical`. The baseline-table font-size is changed to the value of the element's `font-size` SVG attribute or the CSS `font-size`, if set.

`text-bottom`

The *line-under* edge is used as the baseline, which is usually the bottom edge of the font's em box.

`text-top`

The *line-over* edge is used as the baseline, which is usually the top edge of the font's em box.

Formal definition

Initial value	`auto`
Applies to	Block-containers, flex containers, grid containers, inline boxes, table rows, and SVG text content elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
dominant-baseline =
  auto               |
  <baseline-metric>

<baseline-metric> =
  text-bottom   |
  alphabetic    |
  ideographic   |
  middle        |
  central       |
  mathematical  |
  hanging       |
  text-top
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

```
<svg viewBox="0 0 450 160" width="700" height="200">
  <text x="50" y="20">alphabetic</text>
  <text x="50" y="60">central</text>
  <text x="50" y="100">hanging</text>
  <text x="50" y="140">ideographic</text>
  <text x="250" y="20">mathematical</text>
  <text x="250" y="60">middle</text>
  <text x="250" y="100">text-bottom</text>
  <text x="250" y="140">text-top</text>
  <path
    d="M   0,20 l 400,0
       m -400,40 l 400,0
       m -400,40 l 400,0
       m -400,40 l 400,0"
    stroke="grey" />
  <text x="0" y="20" fill="red">auto</text>
  <text x="0" y="60" fill="red">auto</text>
  <text x="0" y="100" fill="red">auto</text>
```

```
  <text x="0" y="140" fill="red">auto</text>
</svg>
```

```
text {
  font-size: 20px;
}
text:nth-of-type(1) {
  dominant-baseline: alphabetic;
}
text:nth-of-type(2) {
  dominant-baseline: central;
}
text:nth-of-type(3) {
  dominant-baseline: hanging;
}
text:nth-of-type(4) {
  dominant-baseline: ideographic;
}
text:nth-of-type(5) {
  dominant-baseline: mathematical;
}
text:nth-of-type(6) {
  dominant-baseline: middle;
}
text:nth-of-type(7) {
  dominant-baseline: text-bottom;
}
text:nth-of-type(8) {
  dominant-baseline: text-top;
}
```

See also

- `alignment-baseline`
- `text-anchor`
- `vertical-align`
- SVG `dominant-baseline` attribute

dynamic-range-limit CSS property

The **`dynamic-range-limit`** CSS property specifies the maximum luminance allowed for High Dynamic Range (HDR) content.

Syntax

```
/* Keyword values */
dynamic-range-limit: standard;
dynamic-range-limit: no-limit;
dynamic-range-limit: constrained;

/* dynamic-range-limit-mix() function */
dynamic-range-limit: dynamic-range-limit-mix(standard 70%, no-limit 30%);

/* Global values */
dynamic-range-limit: inherit;
dynamic-range-limit: initial;
dynamic-range-limit: revert;
dynamic-range-limit: revert-layer;
dynamic-range-limit: unset;
```

Values

This property is specified as a single value from the following list:

`standard`

Specifies the maximum luminance as High Dynamic Range (HDR) reference white, which is the CSS color `white`.

`no-limit`

Specifies the maximum luminance as much greater than that of HDR reference white. The precise level is not specified. This is the initial value.

`constrained`

Specifies the maximum luminance as somewhat greater than that of HDR reference white, such that a mix of Standard Dynamic Range (SDR) and HDR content can be comfortably viewed together. The precise level is not specified.

`dynamic-range-limit-mix()`

Specifies the maximum luminance as a custom value that is a combination of the different keyword values proportionate to the provided percentages. It takes two or more pairs, each consisting of a `dynamic-range-limit` keyword or a nested `dynamic-range-limit-mix()` function and a percentage.

Description

The `dynamic-range-limit` property specifies the maximum luminance allowed in displays capable of displaying high dynamic range colors. A **dynamic range** is the difference in luminance (brightness) between the lightest and darkest parts of content. Dynamic range is measured in photographic stops, where an increase of one stop represents a doubling of the luminance.

SDR, HDR, and headroom

Traditional web content uses **Standard Dynamic Range (SDR)**, where the brightest color is equivalent to the CSS color `white` (`#ffffff` in hexadecimal). The brightness in **High Dynamic Range (HDR)** content, on the other hand, can go beyond the standard white. In HDR terminology, the standard CSS `white` is also called the HDR reference white.

The peak luminance at which content can be displayed depends on the content, the available display hardware, and user preferences. The amount by which the peak luminance of white can exceed HDR reference white is termed **HDR headroom** and is typically expressed in photographic stops.

SDR content always has an HDR headroom of `0` because its brightest white *is* the HDR reference white. Older monitors may also have an HDR headroom of `0` because they are not able to display brighter colors. Newer monitors may have an HDR headroom greater than `0`, allowing them to display the brighter colors available in HDR content.

The use-case for <code>dynamic-range-limit</code>

The brightness of HDR content can be jarring to the viewer. This is especially apparent in apps where a mixture of HDR and SDR content is displayed, leading to inconsistency in the brightness.

The `dynamic-range-limit` property allows you to control the brightness of HDR content. For example, you might want to constrain the maximum brightness of all thumbnails in a photo or video gallery to HDR reference white (this is what the `standard` keyword value does) or to a brightness only slightly more than the HDR reference white (using the `constrained` keyword value or a custom limit created using `dynamic-range-limit-mix()`). When a user views a single HDR image, or if the user selects a preference to enable it, you could then set the image's `dynamic-range-limit` to `no-limit`.

Formal definition

Initial value	`no-limit`
Applies to	all elements
Inherited	yes
Computed value	See Computed value for dynamic-range-limit
Animation type	By `dynamic-range-limit-mix()`

Formal syntax

```
dynamic-range-limit =
  standard                    |
  no-limit                    |
  constrained                 |
  <dynamic-range-limit-mix()>

<dynamic-range-limit-mix()> =
  dynamic-range-limit-mix( [ <'dynamic-range-limit'> && <percentage [0,100]> ]# )

<dynamic-range-limit> =
  standard                    |
  no-limit                    |
  constrained                 |
  <dynamic-range-limit-mix()>
```

This syntax reflects the latest standard as per CSS Color HDR Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>dynamic-range-limit</code> usage

This example demonstrates the basic use of the `dynamic-range-limit` property and the difference between HDR and SDR images.

HTML

In the markup, we embed an HDR image using an `<img>` element. We include a `tabindex` value of `0` to make the image focusable via the keyboard.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/ultra-hdr.jpg"
  alt="A subway station platform with bright white overhead strip lights"
  tabindex="0" />
```

CSS

We constrain the image to SDR brightness by setting the `dynamic-range-limit` property to `standard`, which makes the image no brighter than HDR reference white. We also set a `transition` property so that the `<img>` element's `dynamic-range-limit` value transitions over `0.6` seconds when its state changes.

```
img {
  dynamic-range-limit: standard;
  transition: dynamic-range-limit 0.6s;
}
```

On hover or focus, we change the value of the `<img>` element's `dynamic-range-limit` to `no-limit` so that it displays as brightly as the browser and display setup allow.

```
img:hover,
img:focus {
  dynamic-range-limit: no-limit;
}
```

```
img {
  max-height: 100vh;
}
@media not (dynamic-range: high) {
  body::before {
    content: "Your device may not display the image at full brightness.";
    background-color: wheat;
    display: block;
    text-align: center;
  }
}
@supports not (dynamic-range-limit: standard) {
  body::before {
    content: "Your browser doesn't support the dynamic-range-limit property.";
    background-color: wheat;
    display: block;
    text-align: center;
  }
}
```

Results

The image is ultra HDR but is constrained to SDR brightness by default. Hover over or focus the image. Note how it transitions to vivid HDR colors on capable displays.

See also

- `dynamic-range` and `video-dynamic-range` media features

empty-cells CSS property

The **`empty-cells`** CSS property sets whether borders and backgrounds appear around `<table>` cells that have no visible content.

This property has an effect only when the `border-collapse` property is `separate`.

Try it

```
empty-cells: show;
```

```
empty-cells: hide;
```

```
<section class="default-example" id="default-example">
  <table class="transition-all" id="example-element">
    <tbody>
      <tr>
        <th>Client Name</th>
        <th>Age</th>
      </tr>
      <tr>
        <td></td>
        <td>25</td>
      </tr>
      <tr>
        <td>Louise Q.</td>
        <td></td>
      </tr>
      <tr>
        <td>Owen B.</td>
        <td></td>
      </tr>
      <tr>
        <td>Stan L.</td>
        <td>71</td>
      </tr>
    </tbody>
  </table>
</section>
```

```
th,
td {
  border: 2px solid #aa1199;
  padding: 0.25rem 0.5rem;
}
```

Syntax

```
/* Keyword values */
empty-cells: show;
empty-cells: hide;

/* Global values */
empty-cells: inherit;
empty-cells: initial;
empty-cells: revert;
empty-cells: revert-layer;
empty-cells: unset;
```

Values

This property is specified as one of the following keyword values:

`show`

Borders and backgrounds are drawn like in normal cells.

`hide`

No borders or backgrounds are drawn.

Formal definition

Initial value	`show`
Applies to	table-cell elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
empty-cells =
  show  |
  hide
```

This syntax reflects the latest standard as per CSS Table Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Showing and hiding empty table cells

HTML

```
<table class="table_1">
  <tbody>
    <tr>
      <td>Moe</td>
      <td>Larry</td>
    </tr>
    <tr>
      <td>Curly</td>
      <td></td>
    </tr>
  </tbody>
</table>
```

```
<br />
<table class="table_2">
  <tbody>
    <tr>
      <td>Moe</td>
      <td>Larry</td>
    </tr>
    <tr>
      <td>Curly</td>
      <td></td>
    </tr>
  </tbody>
</table>
```

CSS

```
.table_1 {
  empty-cells: show;
}

.table_2 {
  empty-cells: hide;
}

td,
th {
  border: 1px solid gray;
  padding: 0.5rem;
}
```

Result

See also

- `border-collapse`
- Learn: Styling tables
- CSS table module

field-sizing CSS property

The **`field-sizing`** CSS property enables you to control the sizing behavior of elements that are given a default preferred size, such as form control elements. This property enables you to override the default sizing behavior, allowing form controls to adjust in size to fit their contents.

This property is typically used to style text `<input>` and `<textarea>` elements to allow them to shrinkwrap their content as well as grow when more text is entered into the form control.

Syntax

```
/* Keyword values */
field-sizing: content;
field-sizing: fixed;

/* Global values */
field-sizing: inherit;
field-sizing: initial;
field-sizing: revert;
field-sizing: revert-layer;
field-sizing: unset;
```

Values

This property is specified as one of the following keyword values:

`content`

Allows the element to adjust its size to fit its contents.

`fixed`

Sets a fixed size for the element. This is the default value.

Description

`field-sizing: content` overrides the default preferred sizing of form elements. This setting provides a way to configure text inputs to shrinkwrap their content and grow as more text is entered. They stop expanding when they reach maximum size limits (defined by the size of their containing element or set via CSS), at which point scrolling is required to view all the content.

Elements affected by <code>field-sizing: content</code>

Specifically, `field-sizing` to `content` affects the following elements:

- Form input types that accept direct text input from users. This includes `email`, `number`, `password`, `search`, `tel`, `text`, and `url` types.
  - If no minimum width is set on the control, it will only be as wide as the text cursor.
  - Controls with `placeholder` attributes will be rendered large enough to display the placeholder text.
  - The `size` attribute modifies the default preferred size of such `<input>` elements. As a result, `size` has no effect on `<input>` elements with `field-sizing: content` set.
- `file` inputs. Direct text input is not possible; however, the displayed filename changes as the user selects a new file to upload. When `field-sizing: content` is set, the control will change size to shrinkwrap the filename.
- `<textarea>` controls. It is worth noting that `<textarea>` elements with `field-sizing: content` set behave much like single-line text controls, with the following additions:
  - If `<textarea>` elements are unable to grow due to a width constraint, they will start to grow in height to display additional rows of content. When a height constraint is then reached, they will then start to show a scrollbar to allow all the content to be viewed.
  - `rows` and `cols` attributes modify the default preferred size of a `<textarea>`. As a result, `rows`/`cols` have no effect on `<textarea>` elements with `field-sizing: content` set.
- `<select>` controls. These behave a bit differently to what you might expect with `field-sizing: content` set. The effect depends on the type of `<select>` control you are creating:
  - Regular drop-down boxes will change their width to always fit the displayed option value as new values are selected. (By default, the drop-down's size is set to be large enough to display the longest option value.)
  - List boxes (`<select>` elements with the `multiple` or `size` attribute) will be large enough to display all the options without needing to scroll. (By default, the drop-down box will require scrolling to view all the option values.)
  - The `size` attribute has very little effect on `<select>` elements that have `field-sizing: content` set. In such cases, the browser checks if the `size` is equal to `1` to determine whether the `<select>` control should appear as a drop-down or a listbox. However, it will always display all the options of a listbox, even if `size` is smaller than the number of options.

<code>field-sizing</code> interaction with other size settings

The sizing flexibility provided to form controls by `field-sizing: content` can be overridden if you use other CSS sizing properties. Avoid setting a fixed `width` and `height` when using `field-sizing: content` because they will reimpose a fixed size on the control. However, using properties like `min-width` and `max-width` alongside `field-sizing: content` is quite effective because they allow the control to grow and shrink with the entered text and also prevent the control from becoming too large or too small.

The `maxlength` attribute causes the control to stop growing in size when the maximum character limit is reached.

Formal definition

Initial value	`fixed`
Applies to	Elements with default preferred size
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
field-sizing =
  fixed    |
  content
```

This syntax reflects the latest standard as per CSS Form Control Styling Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Growing and shrinking text fields

This example illustrates the effect of `field-sizing: content` on single- and multi-line text fields. The fields adjust their size as text is added or removed, effectively shrinkwrapping the contents, until a lower or upper size limit is reached.

HTML

The HTML in this example contains four form fields, each with an associated `<label>`: three `<input>` elements of types `text`, `email`, and `tel`, and a `<textarea>` element.

```
<div>
  <label for="name">Enter name:</label>
  <input type="text" id="name" />
</div>
<div>
  <label for="email">Enter email:</label>
  <input type="email" id="email" placeholder="e.g. a@b.com" />
</div>
<div>
  <label for="tel">Enter telephone:</label>
  <input type="tel" id="tel" maxlength="15" />
</div>
<div>
  <label for="comment">Enter comment:</label>
  <textarea id="comment">This is a comment.</textarea>
</div>
```

Note the following points about the HTML:

- The first two fields have no length restrictions set.
- The third field (of type `tel`) has a `maxlength` attribute set, which stops the size of the field from increasing when the character limit is reached.
- The `<textarea>` will grow in the inline direction until the edge of the `min-width` constraint (set in the CSS code below) is reached, then start to add new lines in the block direction to contain subsequent characters.
- The `email` input has a placeholder set. This causes the field to render big enough to show the entire placeholder. Once the field is focused and the user starts typing, the field changes size to the `min-width` value. The `text` field, which doesn't have a placeholder, renders initially at `min-width`.

CSS

In the CSS, we set `field-sizing: content` on the three form fields, along with a `min-width` and `max-width` to constrain the input size. It is worth reiterating that, if no minimum width was set on the fields, they would be rendered only as wide as the text cursor.

We also give the `<label>`s some rudimentary styling so that they sit neatly next to the fields.

```
body {
  box-sizing: border-box;
  padding: 20px;
}

div {
  margin-bottom: 20px;
  display: flex;
}
```

```
input,
textarea {
  field-sizing: content;
  min-width: 50px;
  max-width: 350px;
}

label {
  width: 150px;
  margin-right: 20px;
  text-align: right;
}
```

Result

Try entering and removing text in the fields below to explore the effects of `field-sizing: content` alongside other sizing properties.

Controlling <code><select></code> element display

This example illustrates the effect of `field-sizing: content` on `<select>` elements, both drop-down menu types and multiline listbox types.

HTML

The HTML contains two sets of `<select>` elements: one with `field-sizing: content` applied, and one without, allowing you to see the difference (though the effect may be less obvious than on text fields). Each set includes one drop-down menu type and one multiline listbox type (with the `multiple` attribute set).

```
<div class="field-sizing">
  <h2>With <code>field-sizing: content</code></h2>
  <select>
    <option>Bananas</option>
    <option>Strawberries</option>
    <option selected>Apples</option>
    <option>Raspberries</option>
    <option>Pomegranate</option>
  </select>
  <select multiple>
    <option>Bananas</option>
    <option>Strawberries</option>
    <option>Apples</option>
    <option>Raspberries</option>
    <option>Pomegranate</option>
  </select>
</div>
<div>
  <h2>Without <code>field-sizing: content</code></h2>
  <select>
    <option>Bananas</option>
    <option>Strawberries</option>
    <option selected>Apples</option>
    <option>Raspberries</option>
    <option>Pomegranate</option>
  </select>
  <select multiple>
    <option>Bananas</option>
    <option>Strawberries</option>
    <option>Apples</option>
    <option>Raspberries</option>
    <option>Pomegranate</option>
  </select>
</div>
```

> **Note:** Best practice is to include a `<label>` element for each form control, to associate a meaningful text description with each field for accessibility purposes (see Use meaningful text labels for more information). We haven't done so in this example, as it focuses purely on aspects of the form controls' visual rendering, but you should make sure you include form labels in production code.

CSS

In the CSS, `field-sizing: content` is set only on the first set of `<select>` elements.

```
body {
  box-sizing: border-box;
  display: flex;
  gap: 20px;
  font-family: sans-serif;
}

h2 {
  margin-top: 0;
  font-size: 1rem;
  text-align: center;
  flex: 1 0 100%;
}

div {
  margin-bottom: 20px;
  flex: 1;
  display: flex;
  align-items: flex-start;
  justify-content: space-around;
  flex-flow: row wrap;
}
```

```
.field-sizing select {
  field-sizing: content;
}
```

Result

Note the following effects of `field-sizing: content`:

- The drop-down menu always fits the size of the displayed option, changing size as different options are selected. Without `field-sizing: content`, the size is fixed as wide as the longest option.
- The multi-select list box displays all of the options at once. Without `field-sizing: content`, the user has to scroll the box to view all the options.

See also

- `<input>`
- `<select>`
- `<textarea>`

fill CSS property

The **`fill`** CSS property defines how SVG text content and the interior canvas of SVG shapes are filled or painted. If present, it overrides the element's `fill` attribute.

The areas inside the outline of the SVG shape or text are painted. What is "inside" a shape may not always be clear. The paths defining a shape may overlap. The areas considered "inside" these complex shapes are clarified by the `fill-rule` property or attribute.

If subpaths are open, `fill` closes the path before painting, as if a "closepath" command were included connecting the last point of the subpath with the first point of the subpath. In other words, `fill` applies to open subpaths within `path` elements (i.e., subpaths without a closepath command) and `polyline` elements.

> **Note:** The `fill` property only applies to `<circle>`, `<ellipse>`, `<path>`, `<polygon>`, `<polyline>`, `<rect>`, `<text>`, `<textPath>`, and `<tspan>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* Keyword values */
fill: none;
fill: context-fill;
fill: context-stroke;

/* <color> values */
fill: red;
fill: hsl(120deg 75% 25% / 60%);

/* <url> values */
fill: url("#gradientElementID");
fill: url("star.png");

/* <url> with fallback */
fill: url("#gradientElementID") blue;
fill: url("star.png") none;

/* Global values */
fill: inherit;
fill: initial;
fill: revert;
fill: revert-layer;
fill: unset;
```

Values

This property is specified as a single value from the following list or as two values when using `<url>`:

`none`

No `fill` is painted; the areas inside the stroke, if any, are transparent.

`context-fill`

Uses the paint value of `fill` from a context element.

`context-stroke`

Uses the paint value of `stroke` from a context element.

`<color>`

The color of the fill as any valid CSS `<color>` value.

`<url>`

A URL reference to an SVG paint server element, such as a `<linearGradient>`, `<radialGradient>`, or `<pattern>`. The resource reference can optionally be followed by a `<color>` or `none`, which will be used as a fallback if the referenced paint server doesn't resolve.

Formal definition

Initial value	`black`
Applies to	SVG shapes and text content elements
Inherited	yes
Computed value	as specified, but with `<color>` values computed and `<url>` values made absolute
Animation type	by computed value type

Formal syntax

```
fill =
  <paint>

<paint> =
  none         |
  <image>      |
  <svg-paint>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<svg-paint> =
  child               |
  child( <integer> )

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )
```

```
<integer> =
  <number-token>

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Specification, CSS Values and Units Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining fill values for SVG elements

This example demonstrates how a `fill` is declared, the effect of the property, and how the CSS `fill` property takes precedence over the `fill` attribute.

HTML

We have an SVG with two complex shapes defined using the SVG `<polygon>` and `<path>` elements. Both have the `fill` attribute set to the default `black`. We add a dark gray stroke of `#666666` using the SVG `stroke` attribute but could have used the `stroke` property.

```
<svg viewBox="0 0 220 120" xmlns="http://www.w3.org/2000/svg">
  <path
    d="M 10,5 l 90,0 -80,80 0,-60 80,80 -90,0 z"
    stroke="#666666"
    fill="black" />
  <polygon
    points="180,10 150,100 220,40 140,40 210,100"
    stroke="#666666"
    fill="black" />
</svg>
```

CSS

We set `fill` values on the shapes in the SVG.

```
svg {
  border: 1px solid;
  height: calc(100vh - 20px);
  margin-bottom: 10px;
}
```

```
path {
  fill: red;
}
polygon {
  fill: hsl(0deg 100% 50% / 60%);
}
```

Results

The CSS `fill` property value overrides the SVG `fill` attribute value, resulting in both shapes being filled with a red color; the polygon's red is translucent.

Using fill keyword values

This example demonstrates using keyword values for `fill`.

HTML

We include three `<path>` elements and a `<marker>` element that adds a `<circle>` to every path point. We set the circle marker to be black with a gray fill with the SVG `stroke` and `fill` attributes.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 300 90">
  <path d="M 10 44.64 L 30 10 L 70 10 L 90 44.64 L 70 79.28 L 30 79.28 Z" />
  <path d="M 100 44.64 L 80 10 L 120 10 L 140 44.64 L 120 79.28 L 80 79.28 Z" />
  <path
    d="M 150 44.64 L 130 10 L 170 10 L 190 44.64 L 170 79.28 L 130 79.28 Z" />
  <marker
    id="circle"
    markerWidth="12"
    markerHeight="12"
    refX="6"
    refY="6"
    markerUnits="userSpaceOnUse">
    <circle cx="6" cy="6" r="3" stroke-width="2" stroke="black" fill="grey" />
  </marker>
</svg>
```

CSS

We set different `stroke` and `fill` colors on each path. The first path, the one with a red border, has its `fill` set to `none`. We set the circle marker's stroke and fill to the same color as the stroke of the element they're marking, using the `context-stroke` value.

```
svg {
  border: 1px solid;
  height: calc(100vh - 20px);
  margin-bottom: 10px;
}
```

```
path {
  stroke-width: 2px;
  marker: url("#circle");
}
path:nth-of-type(1) {
  stroke: red;
  fill: none;
}
path:nth-of-type(2) {
  stroke: green;
  fill: lightgreen;
}
path:nth-of-type(3) {
  stroke: blue;
  fill: lightblue;
}
circle {
  stroke: context-stroke;
  fill: context-stroke;
}
```

Results

Note how the first path has a transparent background because the `fill` is `none`, overriding the default `fill` of `black`. The circles are filled with the color of the stroke. If you change the value to `context-fill`, the circles will be transparent, `lightgreen` and `lightblue` instead of `red`, `green`, and `blue`.

Fills and fallbacks

This example demonstrates how to include a `url()` value with a fallback as a `fill` value.

HTML

We have an SVG containing two `<polygon>` stars and a `<linearGradient>` that goes from green to gold to red.

```
<svg viewBox="0 0 220 120" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="myGradient">
      <stop offset="5%" stop-color="green" />
      <stop offset="50%" stop-color="gold" />
      <stop offset="95%" stop-color="red" />
    </linearGradient>
  </defs>
  <polygon points="80,10 50,100 120,40 40,40 110,100" />
  <polygon points="180,10 150,100 220,40 140,40 210,100" />
</svg>
```

CSS

We set `fill` values on the polygons in the SVG, providing a `url()` value and a fallback.

```
svg {
  border: 1px solid;
  height: calc(100vh - 20px);
  margin-bottom: 10px;
}
```

```
polygon:first-of-type {
  fill: url("#myGradient") magenta;
}
polygon:last-of-type {
  fill: url("#MISSINGIMAGE") magenta;
}
```

Results

The first star has a gradient as a background. The second star uses the fallback value, as the element referenced in the `url()` does not exist.

See also

- SVG `fill` attribute
- Presentation properties: `fill`, `clip-rule`, `color-interpolation-filters`, `fill-opacity`, `fill-rule`, `marker-end`, `marker-mid`, `marker-start`, `shape-rendering`, `stop-color`, `stop-opacity`, `stroke`, `stroke-dasharray`, `stroke-dashoffset`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `stroke-opacity`, `stroke-width`, `text-anchor`, and `vector-effect`
- `opacity`
- `background-color`
- `<color>`
- `<basic-shape>` data type

fill-opacity CSS property

The **`fill-opacity`** CSS property defines the opacity of the painting operation (color, gradient, pattern, etc.) applied to SVG shapes or text content elements to fill the element. The property defines the opacity of the element's `fill` only; it does not affect the stroke. If present, it overrides the element's `fill-opacity` attribute.

> **Note:** The `fill-opacity` property only applies to `<circle>`, `<ellipse>`, `<path>`, `<polygon>`, `<polyline>`, `<rect>`, `<text>`, `<textPath>`, and `<tspan>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* numeric and percentage values */
fill-opacity: 0.2;
fill-opacity: 20%;

/* Global values */
fill-opacity: inherit;
fill-opacity: initial;
fill-opacity: revert;
```

```
fill-opacity: revert-layer;
fill-opacity: unset;
```

Values

The `<number>` and `<percentage>` values denote the opacity of the `fill` of the element.

`<number>`

A numeric value between `0` and `1`, inclusive.

`<percentage>`

A percentage value between `0%` and `100%`, inclusive.

With `0` or `0%`, the element is fully transparent. With `1` or `100%`, the element is fully opaque. With values in between, the element is semi-transparent, with content behind the element being visible.

Formal definition

Initial value	`1`
Applies to	SVG shapes and text content elements
Inherited	yes
Percentages	map to the range `[0,1]`
Computed value	The same as the specified value after clipping the `<number>` to the range [0.0, 1.0].
Animation type	by computed value type

Formal syntax

```
fill-opacity =
  <'opacity'>

<opacity> =
  <opacity-value>

<opacity-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Fill and Stroke Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the fill opacity of SVG elements

This example demonstrates the basic use case of `fill-opacity`, and how the CSS `fill-opacity` property takes precedence over the `fill-opacity` attribute and has no effect on any `stroke` applied to a shape.

HTML

We include several different SVG graphic elements and set the `fill-opacity` attribute of each one to `1` (except `line`), meaning the fill of each element is opaque. The `fill-opacity` SVG attribute does not apply to `<line>`.

```
<svg viewBox="0 0 100 150" xmlns="http://www.w3.org/2000/svg">
  <rect x="10" y="10" width="30" height="30" fill-opacity="1" />
  <rect x="60" y="10" rx="10" ry="10" width="30" height="30" fill-opacity="1" />
  <circle cx="25" cy="75" r="20" fill-opacity="1" />
  <ellipse cx="75" cy="75" rx="20" ry="10" fill-opacity="1" />
  <line x1="50" x2="90" y1="40" y2="60" stroke="black" stroke-width="5" />
  <polyline
    points="60 90 65 100 70 95 75 110 80 105 85 120 90 115 95 130 100 125"
    fill-opacity="1" />
  <path d="M20,130 Q40,105 50,130 T90,130" fill-opacity="1" />
</svg>
```

CSS

With CSS, we use the `fill-opacity` property to override the value of the SVG `fill-opacity` attribute, giving each SVG element a different value.

We add a `stroke` to the circle and ellipse, to demonstrate that the opacity of the stroke is not impacted by the `fill-opacity` property.

Other SVG styles are set, including a background image to allow the opacity of each element to be more easily seen. These are not shown for the sake of brevity.

```
svg {
  border: 1px solid;
  height: calc(100vh - 20px);
  background: repeating-linear-gradient(
    to bottom right,
    transparent 0 10px,
    #cccccc 10px 11px
  );
}
```

```
svg > * {
  fill: black;
}
rect:last-of-type {
  fill-opacity: 0.1;
}
circle {
  fill-opacity: 0.6;
}
ellipse {
  fill-opacity: 40%;
}
line {
  fill-opacity: 10%;
}
polyline {
  fill-opacity: 50%;
```

```
}
path {
  fill-opacity: 0.5;
}

circle,
ellipse {
  stroke: black;
  stroke-width: 3px;
}
```

Results

Only two elements are fully opaque: the first rectangle and the line. The first rectangle is not matched by any of the selectors, therefore no CSS is applied and the `fill` is fully opaque. The `line` is matched, with `fill-opacity: 10%` set. However, the line has no `fill` paint operation — only the `stroke` is visible — therefore the declaration has no effect.

See also

- SVG `fill-opacity` attribute
- Presentation properties: `fill-opacity`, `clip-rule`, `color-interpolation-filters`, `fill-rule`, `fill`, `marker-end`, `marker-mid`, `marker-start`, `shape-rendering`, `stop-color`, `stop-opacity`, `stroke`, `stroke-dasharray`, `stroke-dashoffset`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `stroke-opacity`, `stroke-width`, `text-anchor`, and `vector-effect`
- `opacity`
- `background-color`
- `<color>`
- `<basic-shape>` data type

fill-rule CSS property

The **`fill-rule`** CSS property defines the rule used to determine which parts of the SVG shape's canvas are included inside a shape to be filled. If present, it overrides the element's `fill-rule` attribute.

The `fill-rule` clarifies which areas of a shape should be considered "inside" the shape. It provides two values you can set to tell the browser how the inside of a shape should be determined. For shapes that don't have intersecting paths, like a circle, the bounds of what is inside a shape to be filled are intuitively clear. With complex shapes that include intersecting paths (such as a Venn diagram) or paths enclosing other paths (such as a donut), the interpretation of which sections of the shape are "inside" the shape and should be filled by the `fill` property, may not be obvious.

> **Note:** The `fill-rule` property only applies to `<path>`, `<polygon>`, `<polyline>`, `<text>`, `<textPath>`, and `<tspan>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* Keyword values */
fill-rule: evenodd;
fill-rule: nonzero;

/* Global values */
fill-rule: inherit;
fill-rule: initial;
fill-rule: revert;
fill-rule: revert-layer;
fill-rule: unset;
```

Values

This property is specified as one of the following keyword values:

`nonzero`

> For every point in the shape, a ray is drawn in a random direction to beyond the shape's outer edges. Each ray is examined to determine the places where the ray crosses the shape. Starting with a count of zero, add one each time a path segment crosses the ray from left to right and subtract one each time a path segment crosses the ray from right to left. After counting the crossings, if the result is zero then the point is outside the path. Otherwise, it is inside.

`evenodd`

> For every point in the fill rule's box, a ray is drawn in a random direction. The number of path segments from the given shape that the ray crosses are counted. If this number is odd, the point is inside; if even, the point is outside. Zero is taken to be even.

Formal definition

Initial value	`nonzero`
Applies to	SVG shapes and text content elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
fill-rule =
  nonzero  |
  evenodd
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the fill rules for SVG elements

This example demonstrates how a `fill-rule` is declared, the effect of the property, and how the CSS `fill-rule` property takes precedence over the `fill-rule` attribute.

HTML

We define an SVG with two complex shapes defined using the SVG `<polygon>` and `<path>` elements. The polygon has the SVG `fill-rule` attribute set to `evenodd` and the star-shaped path is set to `nonzero`, which is the default. To make the lines visible, we set the outline to `red` using the SVG `stroke` attribute (we could have alternatively used the `stroke` property).

```
<p>Original SVG</p>
```

```
<svg viewBox="0 0 220 120" xmlns="http://www.w3.org/2000/svg">
  <polygon
    points="180,10 150,100 220,40 140,40 210,100"
    stroke="red"
    fill-rule="evenodd" />
  <path
    d="M 10,5 l 90,0 -80,80 0,-60 80,80 -90,0 z"
    stroke="red"
    fill-rule="nonzero" />
</svg>
```

```
<p><code>fill-rule: nonzero;</code></p>
<svg viewBox="0 0 220 120" xmlns="http://www.w3.org/2000/svg">
  <polygon
    points="180,10 150,100 220,40 140,40 210,100"
    stroke="red"
    fill-rule="evenodd" />
  <path
    d="M 10,5 l 90,0 -80,80 0,-60 80,80 -90,0 z"
    stroke="red"
    fill-rule="nonzero" />
</svg>
<p><code>fill-rule: evenodd;</code></p>
<svg viewBox="0 0 220 120" xmlns="http://www.w3.org/2000/svg">
  <polygon
    points="180,10 150,100 220,40 140,40 210,100"
    stroke="red"
    fill-rule="evenodd" />
  <path
    d="M 10,5 l 90,0 -80,80 0,-60 80,80 -90,0 z"
    stroke="red"
    fill-rule="nonzero" />
</svg>
```

The above SVG is repeated three times; we've only shown one copy for the sake of brevity.

CSS

The shapes nested in the first SVG have no CSS applied. We set the shapes inside the second SVG to use the `nonzero` value. The third SVG has all its nested shapes set to `evenodd`.

```
svg {
  border: 1px solid;
  height: calc(33vh - 2.5em);
  margin-bottom: 10px;
}
p {
  margin: 0;
}
```

```
svg:nth-of-type(2) > * {
  fill-rule: nonzero;
}
svg:nth-of-type(3) > * {
  fill-rule: evenodd;
}
```

Results

With the `nonzero` value for `fill-rule`, the "inside" of the shape is the entire shape. The `evenodd` value defines some space as empty. The first image renders the `fill-rule` included as an attribute. Declaring the `fill-rule` in the CSS overrides the attribute values in the second and third images.

See also

- SVG `fill-rule` attribute
- Presentation properties: `fill-rule`, `clip-rule`, `color-interpolation-filters`, `fill-opacity`, `fill`, `marker-end`, `marker-mid`, `marker-start`, `shape-rendering`, `stop-color`, `stop-opacity`, `stroke`, `stroke-dasharray`, `stroke-dashoffset`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `stroke-opacity`, `stroke-width`, `text-anchor`, and `vector-effect`
- `opacity`
- `background-color`
- `<color>`
- `<basic-shape>` data type

filter CSS property

The **`filter`** CSS property applies graphical effects like blur or color shift to an element. Filters are commonly used to adjust the rendering of images, backgrounds, and borders.

Several functions, such as `blur()` and `contrast()`, are available to help you achieve predefined effects.

Try it

```
filter: url("/shared-assets/images/examples/shadow.svg#element-id");
```

```
filter: blur(5px);
```

```
filter: contrast(200%);
```

```
filter: grayscale(80%);
```

```
filter: hue-rotate(90deg);
```

```
filter: drop-shadow(16px 16px 20px red) invert(75%);
```

```
<section id="default-example">
  <div class="example-container">
    <img
      id="example-element"
      src="/shared-assets/images/examples/firefox-logo.svg"
```

```
      width="200" />
  </div>
</section>
```

```
.example-container {
  background-color: white;
  width: 260px;
  height: 260px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#example-element {
  flex: 1;
  padding: 30px;
}
```

Syntax

```
/* <filter-function> values */
filter: blur(5px);
filter: brightness(0.4);
filter: contrast(200%);
filter: drop-shadow(16px 16px 20px blue);
filter: grayscale(50%);
filter: hue-rotate(90deg);
filter: invert(75%);
filter: opacity(25%);
filter: saturate(30%);
filter: sepia(60%);

/* URL */
filter: url("filters.svg#filter-id");

/* Multiple filters */
filter: contrast(175%) brightness(3%);
filter: drop-shadow(3px 3px red) sepia(100%) drop-shadow(-3px -3px blue);

/* Use no filter */
filter: none;

/* Global values */
filter: inherit;
filter: initial;
filter: revert;
filter: revert-layer;
filter: unset;
```

Referencing an SVG filter

You can use `url()` to reference an SVG filter element. For a reference to an SVG `<filter>` element, use the following syntax:

```
filter: url("file.svg#filter-element-id");
```

Functions

The `filter` property is specified as `none` or one or more of the functions listed below. If the parameter for any function is invalid, the function returns `none`. Except where noted, the functions that take a value expressed with a percent sign (as in `34%`) also accept the value expressed as decimal (as in `0.34`).

When the `filter` property values contains multiple functions, the filters are applied in order.

`blur()`

Applies a Gaussian blur to the input image.

```
filter: blur(5px);
```

`brightness()`

Applies a linear multiplier to the input image, making it appear more or less bright. Values are linear multipliers on the effect, with `0%` creating a completely black image, `100%` having no effect, and values over `100%` brightening the image.

```
filter: brightness(2);
```

`contrast()`

Adjusts the contrast of the input image. A value of `0%` makes the image gray, `100%` has no effect, and values over `100%` create a contrast.

```
filter: contrast(200%);
```

`drop-shadow()`

Applies the parameter `<shadow>` as a drop shadow, following the contours of the image. The shadow syntax is similar to `<box-shadow>` (defined in the CSS backgrounds and borders module), with the exception that the `inset` keyword and `spread` parameter are not allowed. As with all `filter` property values, any filters after the `drop-shadow()` are applied to the shadow.

```
filter: drop-shadow(16px 16px 10px black);
```

`grayscale()`

Converts the image to grayscale. A value of `100%` is completely grayscale. The initial value of `0%` leaves the input unchanged. Values between `0%` and `100%` produce linear multipliers on the effect.

```
filter: grayscale(100%);
```

`hue-rotate()`

Applies a hue rotation. The `<angle>` value defines the number of degrees around the hue color circle at which the input samples will be adjusted. A value of `0deg` leaves the input unchanged.

```
filter: hue-rotate(90deg);
```

`invert()`

Inverts the samples in the input image. A value of `100%` completely inverts the image. A value of `0%` leaves the input unchanged. Values between `0%` and `100%` have linear multipliers on the effect.

```
filter: invert(100%);
```

`opacity()`

Applies transparency. `0%` makes the image completely transparent and `100%` leaves the image unchanged.

```
filter: opacity(50%);
```

`saturate()`

Saturates the image, with `0%` being completely unsaturated, `100%` leaving the image unchanged, and values of over `100%` increasing saturation.

```
filter: saturate(200%);
```

`sepia()`

Converts the image to sepia, with a value of `100%` making the image completely sepia and `0%` making no change.

```
filter: sepia(100%);
```

Combining functions

You may combine any number of functions to manipulate the rendering. The filters are applied in the order declared. The following example enhances the contrast and brightness of the image:

```
filter: contrast(175%) brightness(103%);
```

Interpolation

When animated, if both the beginning and end filters have a function list of the same length without `<url>` in the same order, each of their filter functions is interpolated according to the filter function's specific rules.

If the filter lists are of different lengths, the missing equivalent filter functions from the longer list are added to the end of the shorter list. The added functions use their initial, no filter modification values. All the filters listed are then interpolated according to the filter function's specific rules. Otherwise, discrete interpolation is used.

Formal definition

Initial value	none
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	a filter function list

Formal syntax

```
filter =
  none                |
  <filter-value-list>

<filter-value-list> =
  [ <filter-function> | <url> ]+

<filter-function> =
  <blur()>          |
  <brightness()>    |
  <contrast()>      |
  <drop-shadow()>   |
  <grayscale()>     |
  <hue-rotate()>    |
  <invert()>        |
  <opacity()>       |
  <sepia()>         |
  <saturate()>

<blur()> =
  blur( <length>? )

<brightness()> =
  brightness( [ <number> | <percentage> ]? )

<contrast()> =
  contrast( [ <number> | <percentage> ]? )

<drop-shadow()> =
  drop-shadow( [ <color>? && <length>{2,3} ] )

<grayscale()> =
  grayscale( [ <number> | <percentage> ]? )

<hue-rotate()> =
  hue-rotate( [ <angle> | <zero> ]? )

<invert()> =
  invert( [ <number> | <percentage> ]? )

<opacity()> =
  opacity( [ <number> | <percentage> ]? )

<sepia()> =
  sepia( [ <number> | <percentage> ]? )

<saturate()> =
  saturate( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying filter functions

The `filter` property is applied to the second image, graying and blurring both the image and its border.

```
img {
  border: 5px solid yellow;
}
/* Gray the second image by 40% and blur by 5px */
img:nth-of-type(2) {
  filter: grayscale(0.4) blur(5px);
}
```

```
<img src="pencil.jpg" alt="Original image is sharp" />
<img src="pencil.jpg" alt="The image and border are blurred and muted" />
```

Repeating filter functions

Filter functions are applied in order of appearance. The same filter function can be repeated.

```
#MDN-logo {
  border: 1px solid blue;
  filter: drop-shadow(5px 5px 0 red) hue-rotate(180deg)
    drop-shadow(5px 5px 0 red);
}
```

```
<svg
  id="MDN-logo"
  xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 361 104.2"
  xml:space="preserve"
  role="img">
  <title>MDN Web Docs</title>
  <path
    d="M197.6 73.2h-17.1v-5.5h3.8V51.9c0-3.7-.7-6.3-2.1-7.9-1.4-1.6-3.3-2.3-5.7-2.3-3.2 0-5.6 1.1-7.2 3.4s-2.4 4.6-2.5 6.9v15.6h6v5.5h-17.1v-5.5h3.8V51.9c0-3.8-.7-6.4-2.1-7.9-1.4-1.5-3.3-2.3-5.6-2.3-3.2 0-5.5 1.1-7.2 3.3-1.6 2.2-2.4 4.5-2.5 6.9v15.8h6.9v5.5h-20.2v-5.5h6V42.4h-6.1v-5.6h13.4v6.4c1.2-2.1 2.7-3.8 4.7-5.2 2-1.3 4.4-2 7.3-2s5.3.7 7.5 2.1c2.2 1.4 3.7 3.5 4.5 6.4 1.1-2.5 2.7-4.5 4.9-6.1s4.8-2.4 7.9-2.4c3.5 0 6.5 1.1 8.9 3.3s3.7 5.6 3.7 10.2v18.2h6.1v5.5zm42.5 0h-13.2V66c-1.2 2.2-2.8 4.1-4.9 5.6-2.1 1.6-4.8 2.4-8.3 2.4-4.8 0-8.7-1.6-11.6-4.9-2.9-3.2-4.3-7.7-4.3-13.3 0-5 1.3-9.6 4-13.7 2.6-4.1 6.9-6.2 12.8-6.2s9.8 2.2 12.3 6.5V22.7h-8.6v-5.6h15.8v50.6h6v5.5zm-13.3-16.8V52c-.1-3-1.2-5.5-3.2-7.3s-4.4-2.8-7.2-2.8c-3.6 0-6.3 1.3-8.2 3.9-1.9 2.6-2.8 5.8-2.8 9.6 0 4.1 1 7.3 3 9.5s4.5 3.3 7.4 3.3c3.2 0 5.8-1.3 7.8-3.8 2.1-2.6 3.1-5.3 3.2-8zm61.5 16.8H269v-5.5h6V51.9c0-3.7-.7-6.3-2.2-7.9-1.4-1.6-3.4-2.3-5.7-2.3-3.1 0-5.6 1-7.4 3s-2.8 4.4-2.9 7v15.9h6v5.5h-19.3v-5.5h6V42.4h-6.2v-5.6h13.6V43c2.6-4.6 6.8-6.9 12.7-6.9 3.6 0 6.7 1.1 9.2 3.3s3.7 5.6 3.7 10.2v18.2h6v5.4h-.2z"></path>
  <g fill="blue">
    <path
      d="M42 .2 13.4 92.3H1.7L30.2.2H42zM52.4.2v92.1H42V.2h10.4zm40.3 0L64.2 92.3H52.5L81 .2h11.7zM103.1.2v92.1H92.7V.2h10.4zM294 95h67v8.8h-67V95z"></path>
  </g>
</svg>
```

The filters are applied in order. This is why the drop shadows are not the same color: the first drop shadow's hue is altered by the `hue-rotate()` function but the second one is not.

See also

- `backdrop-filter`
- `mask`
- SVG `filter` attribute
- CSS compositing and blending module, including the CSS `background-blend-mode` and `mix-blend-mode` properties.
- SVG, including the SVG `<filter>` element and SVG `filter` attribute.
- Applying SVG effects to HTML content

flex CSS property

The **`flex`** CSS shorthand property sets how a flex item will grow or shrink to fit the space available in its flex container.

Try it

```
flex: 1;
```

```
flex: 2;
```

```
flex: 1 30px;
```

```
flex: 1 1 100px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">Change me</div>
  <div>flex: 1</div>
  <div>flex: 1</div>
</section>
```

```
.default-example {
  border: 1px solid #c5c5c5;
  width: auto;
  max-height: 300px;
  display: flex;
}

.default-example > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 0;
}

#example-element {
```

```
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `flex-grow`
- `flex-shrink`
- `flex-basis`

Syntax

```
/* Keyword values */
flex: none; /* 0 0 auto */

/* One value, unitless number: flex-grow
flex-basis is then equal to 0%. */
flex: 2; /* 2 1 0% */

/* One value, width/height: flex-basis */
flex: auto; /* 1 1 auto */
flex: 10em; /* 1 1 10em */
flex: 30%;
flex: min-content;

/* Two values: flex-grow | flex-basis */
flex: 1 30px; /* 1 1 30px */

/* Two values: flex-grow | flex-shrink */
flex: 2 2; /* 2 2 0% */

/* Three values: flex-grow | flex-shrink | flex-basis */
flex: 2 2 10%;

/* Global values */
flex: inherit;
flex: initial; /* 0 1 auto */
flex: revert;
flex: revert-layer;
flex: unset;
```

The `flex` property may be specified using one, two, or three values.

- **One-value syntax:** the value must be one of:
  - a valid value for `flex-grow`: then the shorthand expands to `flex: <flex-grow> 1 0%`.
  - a valid value for `flex-basis`: then the shorthand expands to `flex: 1 1 <flex-basis>`.
  - the keyword `none` or one of the global keywords.
- **Two-value syntax:**
  - The first value must be a valid value for `flex-grow`.
  - The second value must be one of:
    - a valid value for `flex-shrink`: then the shorthand expands to `flex: <flex-grow> <flex-shrink> 0%`.
    - a valid value for `flex-basis`: then the shorthand expands to `flex: <flex-grow> 1 <flex-basis>`.
- **Three-value syntax:** the values must be in the following order:
  1. a valid value for `flex-grow`.
  2. a valid value for `flex-shrink`.
  3. a valid value for `flex-basis`.

Values

`<'flex-grow'>`

Defines the `flex-grow` of the flex item. Negative values are considered invalid. Defaults to `1` when omitted. (initial is `0`)

`<'flex-shrink'>`

Defines the `flex-shrink` of the flex item. Negative values are considered invalid. Defaults to `1` when omitted. (initial is `1`)

`<'flex-basis'>`

Defines the `flex-basis` of the flex item. Defaults to `0%` when omitted. The initial value is `auto`.

`none`

The item is sized according to its `width` and `height` properties. It is fully inflexible: it neither shrinks nor grows in relation to the flex container. This is equivalent to setting `flex: 0 0 auto`.

Commonly desired flexbox effects can be achieved using the following `flex` values:

- `initial`: Flex item doesn't grow but can shrink. This default value expands to `flex: 0 1 auto`. The item is sized according to its `width` or `height` properties, depending on the `flex-direction`. If there is negative available space, the item will shrink to its minimum size to fit within the container but will not grow to absorb any positive space available in the flex container.
- `auto`: Flex item can grow and shrink. This value expands to `flex: 1 1 auto`. The item is sized according to its `width` or `height` properties, depending on the `flex-direction`, but grows to absorb available positive space in the flex container or shrink down to its minimum size to fit the container in the case of negative space. The flex item is fully flexible.
- `none`: The flex item neither grows nor shrinks. This value expands to `flex: 0 0 auto`. The item is sized according to its `width` or `height` properties, depending on the direction of the flex container. The flex item is fully inflexible.
- `flex: <number [1,∞]>`: The flex item's main size will be proportional to the number set. This value expands to `flex: <number> 1 0%`. This sets the `flex-basis` to `0%` and makes the flex item flexible. The item will be at least as wide or tall as its minimum size, with the container's positive available space being proportionally distributed based on the growth factors of this item and its sibling flex items. If all the flex items use this pattern, all will be sized in proportion to their numeric values.

Description

For most purposes, authors should set `flex` to one of the following values: `auto`, `initial`, `none`, or a positive unitless number. To see the effect of these values, try resizing the flex containers below:

```
<div class="flex-container">
  <div class="item auto">auto</div>
  <div class="item auto">auto</div>
  <div class="item auto">auto</div>
</div>

<div class="flex-container">
  <div class="item auto">auto</div>
  <div class="item initial">initial</div>
  <div class="item initial">initial</div>
</div>

<div class="flex-container">
  <div class="item auto">auto</div>
  <div class="item auto">auto</div>
  <div class="item none">none</div>
</div>

<div class="flex-container">
  <div class="item initial">initial</div>
  <div class="item none">none</div>
  <div class="item none">none</div>
</div>

<div class="flex-container">
  <div class="item four">4</div>
  <div class="item two">2</div>
  <div class="item one">1</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.flex-container {
  background-color: #f4f7f8;
  resize: horizontal;
  overflow: hidden;
  display: flex;
  margin: 1em;
}

.item {
  margin: 1em;
  padding: 0.5em;
  width: 110px;
  min-width: 0;
  background-color: #1b5385;
  color: white;
  font-family: monospace;
  font-size: 13px;
}

.initial {
  flex: initial;
}

.auto {
  flex: auto;
}

.none {
  flex: none;
}

.four {
  flex: 4;
}

.two {
  flex: 2;
}

.one {
  flex: 1;
}
```

By default flex items don't shrink below their `min-content` size. To change this, set the item's `min-width` or `min-height`.

Formal definition

Initial value	as each of the properties of the shorthand: • `flex-grow`: `0` • `flex-shrink`: `1` • `flex-basis`: `auto`
Applies to	flex items, including in-flow pseudo-elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `flex-grow`: as specified • `flex-shrink`: as specified • `flex-basis`: as specified, but with relative lengths converted into absolute lengths
Animation type	as each of the properties of the shorthand: • `flex-grow`: a number

	• `flex-shrink`: a number • `flex-basis`: a length, percentage or calc();

Formal syntax

```
flex =
  none                                          |
  [ <'flex-grow'> <'flex-shrink'>? || <'flex-basis'> ]

<flex-grow> =
  <number [0,∞]>

<flex-shrink> =
  <number [0,∞]>

<flex-basis> =
  content    |
  <'width'>

<width> =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Flexible Box Layout Module Level 2, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting flex: auto

This example shows how a flex item with `flex: auto` grows to absorb any free space in the container.

HTML

```
<div id="flex-container">
  <div id="flex-auto">
    flex: auto (click to remove/add the `flex: initial` box)
  </div>
  <div id="default">flex: initial</div>
</div>
```

CSS

```
body * {
  padding: 1rem;
  user-select: none;
  box-sizing: border-box;
```

```
  font-family: "Consolas", "Arial", sans-serif;
}
```

```
#flex-container {
  border: 2px dashed gray;
  display: flex;
}

#flex-auto {
  cursor: pointer;
  background-color: wheat;

  flex: auto;
}

#default {
  background-color: lightblue;
}
```

JavaScript

```
const flexAutoItem = document.getElementById("flex-auto");
const defaultItem = document.getElementById("default");
flexAutoItem.addEventListener("click", () => {
  defaultItem.style.display =
    defaultItem.style.display === "none" ? "block" : "none";
});
```

Result

The flex container contains two flex items:

- The `#flex-auto` item has a `flex` value of `auto`. The `auto` value expands to `1 1 auto`, i.e., the item is allowed to expand.
- The `#default` item has no `flex` value set so it defaults to the `initial` value. The `initial` value expands to `0 1 auto`, i.e., the item is not allowed to expand.

The `#default` item takes up as much space as its width requires, but does not expand to take up any more space. All the remaining space is taken up by the `#flex-auto` item.

When you click the `#flex-auto` item, we set the `#default` item's `display` property to `none`, removing it from the layout. The `#flex-auto` item then expands to occupy all the available space in the container. Clicking the `#flex-auto` item again adds the `#default` item back to the container.

See also

- Basic concepts of flexbox
- Controlling ratios of flex items along the main axis
- CSS flexible box layout module

flex-basis CSS property

The **`flex-basis`** CSS property sets the initial main size of a flex item. It sets the size of the content box unless otherwise set with `box-sizing`.

> **Note:** It is recommended to use the `flex` shorthand with a keyword value like `auto` or `initial` instead of setting `flex-basis` on its own. The keyword values expand to reliable combinations of `flex-grow`, `flex-shrink`, and `flex-basis`, which help to achieve the commonly desired flex behaviors.

Try it

```
flex-basis: auto;
```

```
flex-basis: 0;
```

```
flex-basis: 200px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">Item One</div>
  <div>Item Two</div>
  <div>Item Three</div>
</section>
```

```
.default-example {
  border: 1px solid #c5c5c5;
  width: auto;
  max-height: 300px;
  display: flex;
}

.default-example > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: auto;
}
```

In this example, the `flex-grow` and `flex-shrink` properties are both set to `1` on all three items, indicating that the flex item can grow and shrink from the initial `flex-basis`.

The demo changes the `flex-basis` value set on the first flex item, causing it to grow or shrink to fill the available space. The other flex items will also change size; they will be at least `min-content`-sized. For example, when the `flex-basis` of the first item is set to `200px`, it will start at `200px` but then shrink to fit the space available.

If `flex-basis` is set to a value other than `auto` and there is a `width` (or `height` in case of `flex-direction: column`) set for that same flex item, the `flex-basis` value takes precedence.

Syntax

```
/* Specify <'width'> */
flex-basis: 10em;
flex-basis: 3px;
flex-basis: 50%;
```

```
flex-basis: auto;

/* Intrinsic sizing keywords */
flex-basis: max-content;
flex-basis: min-content;
flex-basis: fit-content;

/* Automatically size based on the flex item's content */
flex-basis: content;

/* Global values */
flex-basis: inherit;
flex-basis: initial;
flex-basis: revert;
flex-basis: revert-layer;
flex-basis: unset;
```

The `flex-basis` property is specified as either the keyword `content` or a `<'width'>`.

Values

`content`

Indicates automatic sizing, based on the flex item's content.

`<'width'>`

Any of the following units:

- `<length>` sets an absolute value.
- `<percentage>` sets a percentage of the width or height of the containing block's content area. Percentage values of `flex-basis` are resolved against the flex container. If the flex container's size is indefinite, the used value for `flex-basis` is `content`.
- `auto` uses the value of the `width` in horizontal writing mode, and the value of the `height` in vertical writing mode; when the corresponding value is also `auto`, the `content` value is used instead.
- `max-content` sets the intrinsic preferred width.
- `min-content` sets the intrinsic minimum width.
- `fit-content` sets the maximum possible size of a containing block's content area, bounded by the `min-content` and `max-content` values, and calculated based on the content of the current element.

Formal definition

Initial value	`auto`
Applies to	flex items, including in-flow pseudo-elements
Inherited	no
Percentages	refer to the flex container's inner main size
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a length, percentage or calc();

Formal syntax

```
flex-basis =
  content    |
  <'width'>

<width> =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*
```

```
<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Flexible Box Layout Module Level 2, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting flex item initial sizes

HTML

```
<ul class="container">
  <li class="flex flex1">1: flex-basis test</li>
  <li class="flex flex2">2: flex-basis test</li>
  <li class="flex flex3">3: flex-basis test</li>
  <li class="flex flex4">4: flex-basis test</li>
  <li class="flex flex5">5: flex-basis test</li>
</ul>

<ul class="container">
  <li class="flex flex6">6: flex-basis test</li>
</ul>
```

CSS

```
.container {
  font-family: "Arial", sans-serif;
  margin: 0;
  padding: 0;
  list-style-type: none;
  display: flex;
  flex-wrap: wrap;
}

.flex {
  background: #6ab6d8;
  padding: 10px;
  margin-bottom: 50px;
  border: 3px solid #2e86bb;
  color: white;
  font-size: 14px;
  text-align: center;
  position: relative;
}

.flex::after {
  position: absolute;
  z-index: 1;
  left: 0;
  top: 100%;
  margin-top: 10px;
  width: 100%;
  color: #333333;
  font-size: 12px;
}

.flex1 {
  flex-basis: auto;
}

.flex1::after {
  content: "auto";
}

.flex2 {
  flex-basis: max-content;
}

.flex2::after {
  content: "max-content";
}

.flex3 {
  flex-basis: min-content;
}

.flex3::after {
  content: "min-content";
}

.flex4 {
  flex-basis: fit-content;
}

.flex4::after {
  content: "fit-content";
}

.flex5 {
  flex-basis: content;
}
```

```
.flex5::after {
  content: "content";
}
```

Results

Flex basis <code>0</code> vs <code>0%</code>

This example demonstrates the difference between a `flex-basis` of `0` versus a `flex-basis` of `0%` when `flex-direction` is set to `column` and the flex containers and flex items don't have a set height; while `0` is an absolute length, percentage flex-basis values resolve to `content` values.

HTML

We include two same-structure flex containers, which will be styled similarly except for their `flex-basis` values. The containers each have two children: a heading `<div>` and a `<section>`. The `<section>` element has a content `<div>` child, which will not be set as a flex item but will be given a height.

```
<div class="container basis-0">
  <div>heading</div>
  <section>
    <div class="content">flex-basis: 0;</div>
  </section>
</div>
<div class="container basis-0-percent">
  <div>heading</div>
  <section>
    <div class="content">flex-basis: 0%;</div>
  </section>
</div>
```

CSS

We style the containers as inline flex containers that will appear side by side to better enable comparing them. We set the `flex-direction` to `column`. The first container's flex items have a `flex-basis` value of `0`, while the second container's flex items have a `flex-basis` value of `0%`. Neither the flex containers nor their flex items have a height explicitly set; however, the heights of `section` elements must be at least `200px`, and their children have a height of `300px`.

```
.container {
  width: 40vw;
  padding: 1rem;
  border: 1px dashed blue;

  display: inline-flex;
  flex-direction: column;
}

section {
  outline: 1px solid red;

  overflow: auto;
  min-height: 200px;
}

.content {
  background: wheat;
  height: 300px;
}

.container.basis-0 > * {
  flex-basis: 0;
}
.container.basis-0-percent > * {
  flex-basis: 0%;
}
```

Results

In the first container, with `flex-basis: 0`, the `<section>` element has an initial main size of zero, and it grows to the `200px` minimum height. In the second container, with `flex-basis: 0%`, the `<section>` element has an initial main size of `300px` because, as the flex container doesn't have a set height, the percentage flex-basis values resolve to the `content` value.

See also

- `flex` shorthand
- `inline-size`
- Basic concepts of flexbox
- Controlling ratios of flex items along the main axis
- CSS flexible box layout module

flex-direction CSS property

The **`flex-direction`** CSS property sets how flex items are placed in the flex container defining the main axis and the direction (normal or reversed).

Try it

```
flex-direction: row;
```

```
flex-direction: row-reverse;
```

```
flex-direction: column;
```

```
flex-direction: column-reverse;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
```

```
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  display: flex;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 10px;
}
```

Syntax

```
/* Keyword values */
flex-direction: row;
flex-direction: row-reverse;
flex-direction: column;
flex-direction: column-reverse;

/* Global values */
flex-direction: inherit;
flex-direction: initial;
flex-direction: revert;
flex-direction: revert-layer;
flex-direction: unset;
```

Values

This property is specified as one of the following keyword values:

`row`

Specifies the flex container's main-axis to be the same as the text direction. This is the default value.

`row-reverse`

Behaves the same as `row` but specifies the content direction to be inverted, with the first item placed at the inline-end edge.

`column`

Specifies the flex container's main-axis to be the same as the block-axis.

`column-reverse`

Behaves the same as `column` but specifies the content direction to be inverted, with the first item placed at the block-end edge.

Description

The `flex-direction` property enables you to define how flex items are placed in a flex container by setting the container's main axis and the direction of the flex items. This property is relevant only on container elements that have their `display` property set to `flex` or `inline-flex`. The `flex-direction` property, along with the `flex-wrap` property, should be set using the `flex-flow` shorthand.

When set on a flex container, the `flex-direction` property defines whether the flex items are laid out in the same direction or perpendicular to the text direction, and whether the items are laid out normally or reversed.

The default value is `row`. By default, or when `row` is explicitly set, the flex container's main-axis is defined to be the same as the text direction. The first flex item in the DOM order is placed at the inline-start and block start edges. Additional items are placed at the previous item's inline-end edge. Additional rows, if the container is set to wrap with `flex-wrap: wrap`, are added on the block-end edge. The **main-start** and **main-end** points are the same as the content direction, with main-start being the inline-start edge and the main-end being the inline-end edge, and cross-start and cross-end being the block-start and block-end edges, respectively.

The `row-reverse` value is used to change the inline-direction, behaving like `row`, but reversed. The first item is placed at the inline-end and block-start edges, with subsequent flex items being placed at the previous item's inline-start edge, and additional rows added on the block-end edge. The main-start and main-end points are the same as the inline-end and inline-start ends of the writing-mode, respectively, and cross-start and cross-end are the block-start and block-end edges, respectively.

When `flex-direction` is set to `column`, the main axis is the block axis. As with `row`, the first item is placed at the inline-start and block-start edges, but additional items are placed at the previous item's block-end edge rather than the inline edge. If set to be able to wrap, additional columns are added on the inline-end edge. The **main-start** and **main-end** points are based on the block direction of the writing mode, with main-start being the block-start edge and the main-end being the block-end edge, and cross-start and cross-end being the inline-start and inline-end edges, respectively.

With `column-reverse`, the main axis is the block axis, but starting at the block-end edge. The first item is placed at the inline-start and block-end edges, with subsequent items placed at the previous item's block-start edge. If set to be able to wrap, additional columns are added on the inline-end edge. The **main-start** and **main-end** points are based on the block direction of the writing mode, with main-start being the block-end edge and the main-end being the block-start edge, and cross-start and cross-end being the inline-start and inline-end edges, respectively.

The values `row` and `row-reverse` are affected by the directionality of the flex container. If its `dir` attribute is `ltr`, `row` represents the horizontal axis oriented from the left to the right, and `row-reverse` from the right to the left; if the `dir` attribute is `rtl`, `row` represents the axis oriented from the right to the left, and `row-reverse` from the left to the right.

Accessibility

Using the `flex-direction` property with values of `row-reverse` or `column-reverse` will create a disconnect between the visual presentation of content and DOM order. This will adversely affect the experience of users with low vision navigating the page with the aid of assistive technology such as a screen reader. If the visual (CSS) order is important, then screen reader users will not have access to the correct reading order.

- Source Order Matters by Adrian Roselli (2015)
- Flexbox & the keyboard navigation disconnect by Léonie Watson (2016)
- Understanding SC 1.3.2: Meaningful Sequence via WCAG 2.2 (2023)

Formal definition

Initial value	`row`
Applies to	flex containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
flex-direction =
  row             |
  row-reverse     |
  column          |
  column-reverse
```

This syntax reflects the latest standard as per CSS Flexible Box Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Reversing flex container columns and rows

HTML

```
<h4>This is a Column-Reverse</h4>
<div id="col-rev" class="content">
  <div class="box red">A</div>
  <div class="box lightblue">B</div>
  <div class="box yellow">C</div>
</div>
<h4>This is a Row-Reverse</h4>
<div id="row-rev" class="content">
  <div class="box red">A</div>
  <div class="box lightblue">B</div>
  <div class="box yellow">C</div>
</div>
```

CSS

```
.content {
  width: 200px;
  height: 200px;
  border: 1px solid #c3c3c3;
  display: flex;
}

.box {
  width: 50px;
  height: 50px;
}

#col-rev {
  flex-direction: column-reverse;
}

#row-rev {
  flex-direction: row-reverse;
}

.red {
  background-color: red;
}

.lightblue {
  background-color: lightblue;
}

.yellow {
  background-color: yellow;
}
```

Result

See also

- `flex-flow` shorthand
- `flex-wrap`
- `gap`
- Basic concepts of flexbox
- Ordering flex items
- CSS flexible box layout module

flex-flow CSS property

The **`flex-flow`** CSS shorthand property specifies the direction of a flex container, as well as its wrapping behavior.

Try it

```
flex-flow: row wrap;
```

```
flex-flow: row-reverse nowrap;
```

```
flex-flow: row wrap balance;
```

```
flex-flow: column wrap-reverse;
```

```
flex-flow: column wrap;
```

```
flex-flow: column balance wrap;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
    <div>Item Four</div>
```

```
    <div>Item Five</div>
    <div>Item Six</div>
    <div>Item Seven</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  max-height: 300px;
  display: flex;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 5px 10px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `flex-direction`
- `flex-wrap`

Syntax

```
/* flex-flow: <'flex-direction'> */
flex-flow: row;
flex-flow: row-reverse;
flex-flow: column;
flex-flow: column-reverse;

/* flex-flow: <'flex-wrap'> */
flex-flow: nowrap;
flex-flow: wrap;
flex-flow: wrap-reverse;
flex-flow: wrap balance;
flex-flow: balance wrap-reverse;

/* flex-flow: <'flex-direction'> and <'flex-wrap'> */
flex-flow: row nowrap;
flex-flow: column wrap;
flex-flow: column-reverse wrap-reverse;
flex-flow: row-reverse balance wrap

/* Global values */
flex-flow: inherit;
flex-flow: initial;
flex-flow: revert;
flex-flow: revert-layer;
flex-flow: unset;
```

Values

This property is specified as a space-separated list of keywords of the following types:

`flex-direction`

A keyword specifying the main axis and direction of how flex items are placed in the flex container.

`flex-wrap`

One or two keywords specifying whether flex items can wrap onto multiple lines, and, if wrapping is allowed, sets the direction that lines are stacked and whether they are balanced.

Description

The `flex-flow` shorthand property specifies the `flex-direction` and `flex-wrap` properties, defining the direction of a flex container and its wrapping behavior. It can also define flex items to be balanced when wrapping is allowed.

For example, `column-reverse wrap` will set the main-axis to the block direction with a reversed main-start and main-end, with flex items being allowed to wrap, creating new lines if needed.

```
.container {
  flex-flow: column-reverse wrap;
}
```

To distribute the flex items evenly across each flex line, you can include the `flex-wrap` keyword `balance` in addition to `wrap`:

```
.container {
  flex-flow: column-reverse wrap balance;
}
```

Formal definition

Initial value	as each of the properties of the shorthand: • `flex-direction`: `row` • `flex-wrap`: `nowrap`
Applies to	flex containers
Inherited	no
Computed value	as each of the properties of the shorthand: • `flex-direction`: as specified • `flex-wrap`: as specified

Animation type	as each of the properties of the shorthand: • `flex-direction`: discrete • `flex-wrap`: discrete

Formal syntax

```
flex-flow =
  <'flex-direction'>  ||
  <'flex-wrap'>

<flex-direction> =
  row             |
  row-reverse     |
  column          |
  column-reverse

<flex-wrap> =
  nowrap                              |
  [ wrap | wrap-reverse ] || balance
```

This syntax reflects the latest standard as per CSS Flexible Box Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates using the `flex-flow` shorthand on a flex container so the items are laid out backwards across multiple rows.

HTML

We include a list of words in alphabetical order:

```
<ul>
  <li>Alphabet</li>
  <li>Banana</li>
  <li>Crayons</li>
  <li>Dinosaurs</li>
  <li>Eggplant</li>
  <li>Foundation</li>
  <li>Ghosts</li>
  <li>Happy</li>
  <li>Igloo</li>
  <li>Janitors</li>
  <li>Kittens</li>
  <li>Lasso</li>
  <li>Magic 8-ball</li>
  <li>Nincompoop</li>
  <li>Orange</li>
  <li>Petunia</li>
  <li>Quality</li>
  <li>Rancid</li>
  <li>Shoelace</li>
  <li>Terydactyl</li>
  <li>Umbrella</li>
  <li>Valentine</li>
  <li>Westward</li>
  <li>Xylophone</li>
</ul>
```

CSS

We set the `<ul>` to be a flex container with the `display` property, define a `width`, add a `gap` so there is some room between flex items and flex lines, and then set the `flex-flow` to wrap the items in reverse order. Additional CSS has been hidden for brevity.

```
ul {
  display: flex;
  width: 31em;
  gap: 1em;

  flex-flow: row-reverse wrap-reverse;
}
```

```
ul {
  list-style: none;
  border: 1px solid;
  font-family: sans-serif;
}
li {
  font-size: 1.25rem;
  padding: 5px;
  border: 1px solid;
  background-color: lightpink;
}
li:nth-of-type(even) {
  background-color: lightgreen;
}
```

Result

See also

- Basic concepts of flexbox
- Ordering flex items

flex-grow CSS property

The **`flex-grow`** CSS property sets the flex grow factor, which specifies how much of the flex container's **positive free space**, if any, should be assigned to the flex item's main size.

When the flex-container's main size is larger than the combined main sizes of its flex items, this positive free space can be distributed among the flex items, with each item's growth being their growth factor value as a proportion of the sum total of all the flex items' flex grow factors.

> **Note:** It is recommended to use the `flex` shorthand with a keyword value like `auto` or `initial` instead of setting `flex-grow` on its own. The keyword values expand to reliable combinations of `flex-grow`, `flex-shrink`, and `flex-basis`, which help to achieve the commonly desired flex behaviors.

Try it

```
flex-grow: 1;
```

```
flex-grow: 2;
```

```
flex-grow: 3;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">I grow</div>
  <div>Item Two</div>
  <div>Item Three</div>
</section>
```

```
.default-example {
  border: 1px solid #c5c5c5;
  width: auto;
  max-height: 300px;
  display: flex;
}

.default-example > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 0;
}
```

Syntax

```
/* <number> values */
flex-grow: 3;
flex-grow: 0.6;

/* Global values */
flex-grow: inherit;
flex-grow: initial;
flex-grow: revert;
flex-grow: revert-layer;
flex-grow: unset;
```

Values

This property is specified as the following value:

`<number>`

See `<number>`. Negative values are invalid. Defaults to 0, which prevents the flex item from growing.

Description

This property specifies how much of the remaining space in the flex container should be assigned to the item (the flex grow factor).

The main size is either the width or height of the item, depending on the `flex-direction` value.

The remaining space, or positive free space, is the size of the flex container minus the size of all flex items' sizes together. If all sibling items have the same flex grow factor, then all items will receive the same share of remaining space. The common practice is to set `flex-grow: 1`, but setting the flex grow factor for all the flex items to `88`, `100`, `1.2`, or any other value greater than `0` will produce the same result: the value is a ratio.

If the `flex-grow` values differ, the positive free space is distributed according to the ratio defined by the different flex grow factors. The `flex-grow` factor values of all the sibling flex items are added together. The flex container's positive free space, if any, is then divided by that total. The main size of each flex item with a `flex-grow` value greater than `0` will grow by this quotient multiplied by its own growth factor.

For example, if four `100px` flex items are in a `700px` container and the flex items have `flex-grow` factors of `0`, `1`, `2`, and `3`, respectively, the total main size of the four items is `400px`, meaning there is `300px` of positive free space to be distributed. The sum of the four grow factors (`0 + 1 + 2 + 3 = 6`) is equal to six. Therefore, each grow factor is equal to `50px` (`300px / 6`). Each flex item is given 50px of free space multiplied by its `flex-grow` factor — so `0`, `50px`, `100px`, and `150px` respectively. The total flex item sizes become `100px`, `150px`, `200px`, and `250px`, respectively.

`flex-grow` is generally used alongside the other `flex` shorthand properties, `flex-shrink` and `flex-basis`. Using the `flex` shorthand property is recommended to ensure all values are set.

Formal definition

Initial value	`0`
Applies to	flex items, including in-flow pseudo-elements
Inherited	no
Computed value	as specified
Animation type	a number

Formal syntax

```
flex-grow =
  <number [0,∞]>
```

This syntax reflects the latest standard as per CSS Flexible Box Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting flex item grow factor

In this example, the sum of six flex-grow factors is equal to eight, meaning each growth-factor value is `12.5%` of the remaining space.

HTML

```
<h1>This is a <code>flex-grow</code> example</h1>
<p>
  A, B, C, and F have <code>flex-grow: 1</code> set. D and E have
  <code>flex-grow: 2</code> set.
</p>
<div id="content">
  <div class="box1">A</div>
  <div class="box2">B</div>
  <div class="box3">C</div>
  <div class="box4">D</div>
  <div class="box5">E</div>
  <div class="box6">F</div>
</div>
```

CSS

```
#content {
  display: flex;
}

div > div {
  border: 3px solid rgb(0 0 0 / 20%);
}

.box1,
.box2,
.box3,
.box6 {
  flex-grow: 1;
}

.box4,
.box5 {
  flex-grow: 2;
  border: 3px solid rgb(0 0 0 / 20%);
}

.box1 {
  background-color: red;
}
.box2 {
  background-color: lightblue;
}
.box3 {
  background-color: yellow;
}
.box4 {
  background-color: brown;
}
.box5 {
  background-color: lightgreen;
}
.box6 {
  background-color: brown;
}
```

Result

When the six flex items are distributed along the container's main axis, if the sum of the main content of those flex items is less than the size of the container's main axis, the extra space is distributed among the size flex items, with `A`, `B`, `C`, and `F`, each getting `12.5%` of the remaining space and `D` and `E` each getting `25%` of the extra space.

See also

- `flex` shorthand
- Basic Concepts of Flexbox
- Controlling Ratios of flex items along the main axis
- CSS flexible box layout module
- `flex-grow` is weird. Or is it? via CSS-Tricks (2017)

flex-line-count CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`flex-line-count`** CSS property sets the minimum number of flex lines that flex items will be balanced over in cases where a flex container's `flex-wrap` or `flex-flow` property includes the `balance` keyword.

Try it

```
flex-line-count: 1;
```

```
flex-line-count: 3;
```

```
flex-line-count: 4;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
    <div>Item Four</div>
```

```
    <div>Item Five</div>
    <div>Item Six</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  display: flex;
  flex-wrap: wrap balance;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 10px;
}
```

Syntax

```
/* Integer values */
flex-line-count: 1;
flex-line-count: 3;
flex-line-count: 12;

/* Global values */
flex-line-count: inherit;
flex-line-count: initial;
flex-line-count: revert;
flex-line-count: revert-layer;
flex-line-count: unset;
```

Values

This property is specified as the following value:

`<integer>`

A positive integer setting the minimum number of flex lines that balanced, wrapped flex items will be distributed over. The default value is `1`.

Description

The `flex-line-count` property sets the minimum number of flex lines that flex items will be distributed over in wrapping, balanced flex containers — in other words, flex containers that include a `flex-wrap` or `flex-flow` property with the `balance` keyword set in addition to the `wrap` or `wrap-reverse` keyword.

A key use case for `flex-line-count` is creating a balanced set of two (or more) columns, regardless of the number of items in a list. In such cases, setting an explicit `height` or `max-height` won't work as you don't know how much content you will have, and may end up with fewer or more columns than desired. See Creating balanced columns for an example implementation.

If `balance` is not set, or if flex items are not set to wrap onto multiple flex lines, the `flex-line-count` property has no effect.

If the `flex-line-count` value is equal to or greater than the number of flex items, there will be one flex item per flex line.

Formal definition

Initial value	`1`
Applies to	multi-line flex containers
Inherited	no
Computed value	as specified
Animation type	a number

Formal syntax

```
flex-line-count =
  <integer [1,∞]>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Flexible Box Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Effect of different <code>flex-line-count</code> values

This example demonstrates the effects of different values of `flex-line-count` on four boxes.

HTML

We include four container `<div>`s, each with a `class` of `box` and ten child `<div>`s; each container `<div>` has a different `id` value.

```
<div class="box" id="box-no-balance">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>

<div class="box" id="box1">...</div>
<div class="box" id="box2">...</div>
<div class="box" id="box3">...</div>
```

```
<p>No <code>balance</code></p>

<div class="box" id="box-no-balance">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>

<p><code>flex-line-count: 3</code></p>

<div class="box" id="box1">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>

<p><code>flex-line-count: 4</code></p>

<div class="box" id="box2">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>

<p><code>flex-line-count: 5</code></p>

<div class="box" id="box3">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

CSS

```
* {
  box-sizing: border-box;
}

.box {
  width: 100%;
  border: 2px dotted gray;
  margin-bottom: 20px;
  gap: 10px;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: lightgray;
}
```

We apply `display: flex` to all the boxes to make them flex containers, then give them a `flex-wrap` value of `wrap balance` to make all their flex children wrap onto multiple, balanced lines.

```
.box {
  display: flex;
  flex-wrap: wrap balance;
}
```

We also set a `flex` value of `1 1 150px` on the flex children, so they have a base width of `150px` and will distribute any excess space evenly across the items in each flex line.

```
.box > * {
  flex: 1 1 150px;
}
```

For the `#box-no-balance` flex container, we remove the balancing, thereby nullifying the line count, by overriding the original `flex-wrap: wrap balance` value with `wrap`. We apply different `flex-line-count` values to each flex container, incrementing them so their children are laid out over a progressively greater number of flex lines.

```
#box-no-balance {
  flex-line-count: 6;
  flex-wrap: wrap;
}
```

```
#box1 {
  flex-line-count: 3;
}

#box2 {
  flex-line-count: 4;
}

#box3 {
  flex-line-count: 5;
}
```

We've hidden the rest of the CSS for brevity.

Results

Note the following:

- As the first flex container doesn't have the `balance` keyword set in its `flex-wrap` value, its children are not given a balanced distribution and its `flex-line-count` value is ignored.
- The second flex container's `flex-line-count: 3` declaration doesn't affect the layout of the flex children; as the flex items are by default distributed over four flex lines, any value of `4` or less has no effect.

Creating balanced columns

This example demonstrates how `flex-line-count` can be used to create a balanced set of two columns.

HTML

We include an `<ol>` element containing ten `<li>` elements.

```
<ol>
  <li>
    <a href="#">The Silent Cartographer</a>, published by Meridian House,
    released March 12, 2014.
  </li>
  <li>
    <a href="#">Echoes of the Fallow Field</a>, published by Northbridge Press,
    released July 4, 2009.
  </li>

  ...
</ol>
```

```
<ol>
  <li>
    <a href="#">The Silent Cartographer</a>, published by Meridian House,
    released March 12, 2014.
  </li>
  <li>
    <a href="#">Echoes of the Fallow Field</a>, published by Northbridge Press,
    released July 4, 2009.
  </li>
  <li>
    <a href="#">A Ledger of Small Regrets</a>, published by Ashwood & Kline,
    released November 21, 2017.
  </li>
  <li>
    <a href="#">The Clockmaker's Daughter's Shadow</a>, published by Hollow Pine
    Publishing, released February 8, 2011.
  </li>
  <li>
    <a href="#">Salt and Signal</a>, published by Redcliffe Editions, released
    September 30, 2019.
  </li>
  <li>
    <a href="#">Under a Borrowed Sky</a>, published by Fenwick & Marsh, released
    May 16, 2006.
  </li>
  <li>
    <a href="#">The Last Cartel of Winter</a>, published by Graywolf Bindery,
    released January 2, 2021.
  </li>
  <li>
    <a href="#">Notes from an Unfinished Atlas</a>, published by Coastline
    Books, released June 27, 2013.
  </li>
  <li>
    <a href="#">The Weight of Empty Rooms</a>, published by Draymoor House,
    released October 15, 2008.
  </li>
  <li>
    <a href="#">A Brief History of Almost Everyone</a>, published by Ferngate
    Press, released April 9, 2022.
  </li>
</ol>
```

CSS

We set the list's `display` to `flex`. We set a `flex-direction` value of `column` and a `flex-wrap` value of `balance` using the `flex-flow` shorthand so that the flex lines are arranged in columns and will balance when wrapped. The `gap` value `10px 40px` specifies a gap of `10px` between flex items within each column and `40px` between flex lines.

Finally, we set a `flex-line-count` value of `2`, meaning that, even though no fixed height is set on the list, its content will always be wrapped over two balanced columns, regardless of how much content is included.

```
ol {
  display: flex;
  gap: 10px 40px;
  flex-flow: column balance;
  flex-line-count: 2;
}
```

```
* {
  box-sizing: border-box;
}

body {
  padding: 10px 30px;
}

@supports not (flex-line-count: 3) {
  body::before {
    content: "Your browser does not support the flex-line-count property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

We've hidden the rest of the CSS for brevity.

Results

See also

- `flex-wrap`
- `flex-flow` shorthand
- Basic concepts of flexbox
- Mastering wrapping of flex items > Balanced wrapping
- CSS flexible box layout module

flex-shrink CSS property

The **`flex-shrink`** CSS property sets the flex shrink factor of a flex item. If the size of all flex items is larger than the flex container, the flex items can shrink to fit according to their `flex-shrink` value. Each flex line's negative free space is distributed between the line's flex items that have a `flex-shrink` value greater than `0`.

> **Note:** It is recommended to use the `flex` shorthand with a keyword value like `auto` or `initial` instead of setting `flex-shrink` on its own. The keyword values expand to reliable combinations of `flex-grow`, `flex-shrink`, and `flex-basis`, which help to achieve the commonly desired flex behaviors.

Try it

```
flex-shrink: 0;
```

```
flex-shrink: 1;
```

```
flex-shrink: 2;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">I shrink</div>
  <div>Item Two</div>
  <div>Item Three</div>
</section>
```

```
.default-example {
  border: 1px solid #c5c5c5;
  width: auto;
  max-height: 300px;
  display: flex;
}

.default-example > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 300px;
}
```

Syntax

```
/* <number> values */
flex-shrink: 2;
flex-shrink: 0.6;

/* Global values */
flex-shrink: inherit;
flex-shrink: initial;
flex-shrink: revert;
flex-shrink: revert-layer;
flex-shrink: unset;
```

Values

This property is specified as the following value:

`<number>`

See `<number>`. Negative values are invalid. Defaults to 1.

Description

The `flex-shrink` property specifies the flex shrink factor, which determines how much the flex item will shrink relative to the rest of the flex items in the flex container when negative free space is distributed.

This property deals with situations where the browser calculates the flex-basis values of the flex items, and finds that they are too large to fit into the flex container. As long as flex-shrink has a positive value the items will shrink in order that they do not overflow the container.

The `flex-grow` property deals with distributing available positive free space proportional to each item's flex grow factor, with the value of the `flex-grow` property as the only consideration. The `flex-shrink` property manages removing negative free space to make boxes fit into their container without overflowing. Removing space is a bit more complicated than adding space. The flex shrink factor is multiplied by the flex base size; this distributes negative space in proportion to how much the item can shrink. This prevents smaller items from shrinking to `0px` before a larger item is noticeably reduced.

Generally, `flex-shrink` is used alongside the `flex-grow` and `flex-basis` properties. Within the `flex` shorthand, the shrink factor is always the second `<number>`. If the shorthand only includes one number value, that value is assumed to be the `flex-grow` value.

Formal definition

Initial value	`1`
Applies to	flex items, including in-flow pseudo-elements
Inherited	no
Computed value	as specified
Animation type	a number

Formal syntax

```
flex-shrink =
  <number [0,∞]>
```

This syntax reflects the latest standard as per CSS Flexible Box Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting flex item shrink factor

This example demonstrates how negative free space is distributed based on the item's shrink factor. It includes five flex items with a `flex-shrink` value greater than 0, which have a combined width greater than their parent flex container's width.

HTML

```
<div id="content">
  <div class="box1">A</div>
  <div class="box2">B</div>
  <div class="box3">C</div>
  <div class="box4">D</div>
  <div class="box5">E</div>
</div>
```

CSS

We give each flex item a `width` of `200px`. As the `flex-basis` property defaults to `auto`, each item's flex-basis is `200px`. This gives the flex items a total width of `1000px`, twice the size of the container. We set all flex items to be shrinkable, with `flex-shrink` values greater than `0`. The last two items have greater `flex-shrink` values set so they will shrink more.

```
#content {
  display: flex;
  width: 500px;
}

#content div {
  width: 200px;
}

.box1,
.box2,
.box3 {
  flex-shrink: 1;
}

.box4 {
  flex-shrink: 1.5;
}

.box5 {
  flex-shrink: 2;
}
```

```
#content {
  margin: 5px;
}
div {
  font-family: monospace;
  outline: 1px solid;
  line-height: 4em;
  text-align: center;
}
.box1 {
  background-color: red;
}
.box2 {
  background-color: lightblue;
}
.box3 {
  background-color: yellow;
}
.box4 {
  background-color: lightsalmon;
}
.box5 {
  background-color: lightgreen;
}
```

Result

The flex items don't overflow their container because they can shrink: the `500px` of negative free space is distributed among the five items based on their `flex-shrink` values. The total of all the shrink values for the five items is `1 + 1 + 1 + 1.5 + 2 = 6.5`. As a result, the width of items with `flex-shrink: 1` is reduced by `1/6.5 * 500px = 76.92px`, the width of items with `flex-shrink: 1.5` is reduced by `1.5/6.5 * 500px = 115.38px`, and the width of items with `flex-shrink: 2` is reduced by `2/6.5 * 500px = 153.85px`.

See also

- Basic concepts of flexbox
- Controlling ratios of flex items along the main axis
- CSS flexible box layout module

flex-wrap CSS property

The **`flex-wrap`** CSS property sets whether flex items are forced onto one line or can wrap onto multiple lines. If wrapping is allowed, it sets the direction that lines are stacked.

The `flex-flow` property shorthand can be used to set both the `flex-direction` and `flex-wrap` properties, which define the flex container's main and cross axes, respectively.

Try it

```
flex-wrap: nowrap;
```

```
flex-wrap: wrap;
```

```
flex-wrap: wrap-reverse;
```

```
flex-wrap: wrap balance;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
    <div>Item Four</div>
    <div>Item Five</div>
    <div>Item Six</div>
    <div>Item Seven</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  display: flex;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 10px;
}
```

Syntax

```
/* Keyword values */
flex-wrap: nowrap;
flex-wrap: wrap;
flex-wrap: wrap-reverse;
flex-wrap: balance;
flex-wrap: wrap balance;
flex-wrap: balance wrap-reverse;

/* Global values */
flex-wrap: inherit;
flex-wrap: initial;
flex-wrap: revert;
flex-wrap: revert-layer;
flex-wrap: unset;
```

Values

This property is specified as a single value from the following list or as two space-separated values when using `balance`:

`nowrap`

> The flex items are laid out in a single line which may cause the flex container to overflow. This is the initial value.

`wrap`

> The flex items can break into multiple lines.

`wrap-reverse`

> Behaves the same as `wrap`, but with the lines laid out in reverse order.

`balance`

> Specifies that flex items broken across multiple lines are distributed so that the line lengths are as similar as possible. If `balance` is specified as the only keyword, the other keyword defaults to `wrap`.

Description

The `flex-wrap` property is used to specify whether the flex container's child flex items may wrap onto multiple flex lines or not and, if set to wrap, whether the children should be evenly distributed across flex lines.

The initial value of `nowrap` specifies that all items should be laid out on a single flex line, which means they may overflow the container. The `wrap` and `wrap-reverse` keywords prevent overflow by specifying that flex items can flow, or wrap, across multiple lines.

With `wrap`, cross-start is the equivalent of inline-start or block-start, depending on the `flex-direction` value and the current writing mode. With `wrap-reverse`, cross-start is the equivalent of inline-end or block-end.

Balancing flex item distribution

By default, when `wrap` or `wrap-reverse` is specified, each flex line is filled with flex items before items are wrapped onto the next line. This can result in an uneven distribution of items, with the last flex line composed of fewer items. If those items' `flex-grow` values are non-zero, the extra available space is distributed across these fewer items, making them much larger than the items on filled flex lines.

The `balance` keyword can be used to distribute the items across lines in a more even manner. Optionally, the `flex-line-count` property can be used to define the minimum number of lines across which the balanced items should be distributed.

If `balance` is the only keyword specified, the other keyword defaults to `wrap`.

A balanced wrapping flex container can have a `flex-line-count` value applied to specify the minimum number of lines the flex items should be distributed over. The `flex-line-count` property has no effect if `balance` is not specified inside `flex-wrap`.

If `balance` is specified alongside `nowrap`, the property is invalid.

Formal definition

Initial value	`nowrap`
Applies to	flex containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
flex-wrap =
  nowrap                              |
  [ wrap | wrap-reverse ] || balance
```

This syntax reflects the latest standard as per CSS Flexible Box Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting flex container wrap values

HTML

```
<h4>This is an example for flex-wrap:wrap</h4>
<div class="content">
  <div class="red">1</div>
  <div class="green">2</div>
  <div class="blue">3</div>
</div>
<h4>This is an example for flex-wrap:nowrap</h4>
<div class="content1">
  <div class="red">1</div>
  <div class="green">2</div>
  <div class="blue">3</div>
</div>
<h4>This is an example for flex-wrap:wrap-reverse</h4>
<div class="content2">
  <div class="red">1</div>
  <div class="green">2</div>
  <div class="blue">3</div>
</div>
```

CSS

```
/* Common Styles */
.content,
.content1,
.content2 {
  color: white;
  font: 100 24px/100px sans-serif;
  height: 150px;
  width: 897px;
  text-align: center;
}

.content div,
.content1 div,
.content2 div {
  height: 50%;
  width: 300px;
}
.red {
  background: orangered;
}
.green {
  background: yellowgreen;
}
.blue {
  background: steelblue;
}

/* Flexbox Styles */
.content {
  display: flex;
  flex-wrap: wrap;
}
.content1 {
  display: flex;
  flex-wrap: nowrap;
}
.content2 {
  display: flex;
  flex-wrap: wrap-reverse;
}
```

Results

Demonstrating the <code>balance</code> keyword

This example demonstrates the effects of the `balance` keyword within the `flex-wrap` property and different `flex-line-count` values.

HTML

```
<form>
  <div>
    <input type="checkbox" id="reverse" name="reverse" />
    <label for="reverse">Set <code>wrap-reverse</code>?</label>
  </div>
  <div>
    <input type="checkbox" id="balance" name="balance" checked />
    <label for="balance">Set <code>balance</code>?</label>
  </div>
  <div>
    <label for="line-count"><code>flex-line-count</code></label>
    <input
      type="range"
      id="line-count"
      name="line-count"
      min="1"
      max="12"
      value="3" />
    <output>3</output>
  </div>
</form>

<hr />

<p>Currently set: <code>flex-wrap: wrap; flex-line-count: 3;</code></p>

<hr />
```

We include a container `<div>` with a `class` of `box`, which has ten child `<div>`s.

```
<div class="box">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
  <div>Six</div>
  <div>Seven</div>
  <div>Eight</div>
  <div>Nine</div>
  <div>Ten</div>
</div>
```

We also include controls to enable changing features of the flex container, including:

- Toggling `flex-wrap` between the `wrap` and `wrap-reverse` values.
- Toggling the `balance` keyword on and off.
- Changing the `flex-line-count` value.

We've hidden the HTML and JavaScript for brevity.

CSS

We apply `display: flex` to the `.box` to turn it into a flex container, then set some `flex-wrap` and `flex-line-count` values to make the flex children wrap evenly over a minimum of three flex lines. You can change the values in the live demo. We also set a `flex` value of `1 1 150px` on the flex children so they will have a base width of `150px`, with any remaining space distributed evenly between items on the same flex line.

```
.box {
  display: flex;
  flex-wrap: wrap balance;
  flex-line-count: 3;
}

.box > * {
  flex: 1 1 150px;
}
```

```
.box {
  width: 100%;
  border: 2px dotted rgb(96 139 168);
  gap: 10px;
}

.box > * {
  border: 2px solid rgb(96 139 168);
  border-radius: 5px;
  background-color: rgb(96 139 168 / 0.2);
}

* {
  box-sizing: border-box;
}

body {
  padding: 10px 30px;
}

@supports not (flex-wrap: balance) {
  body::before {
    content: "Your browser does not support flex-wrap: balance.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;
```

```
    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

We've hidden the rest of the CSS for brevity.

```
const boxElem = document.querySelector(".box");
const outputElem = document.querySelector("output");
const pCodeElem = document.querySelector("p code");
const reverseInput = document.getElementById("reverse");
const balanceInput = document.getElementById("balance");
const lineCountInput = document.getElementById("line-count");

let wrapValue = "wrap";
let lineCountValue = "3";

function updateCurrentlySet() {
  pCodeElem.textContent = `flex-wrap: ${wrapValue}; flex-line-count: ${lineCountValue};`;
}

function setFlexWrap() {
  wrapValue = "";
  if (reverseInput.checked) {
    wrapValue += "wrap-reverse";
  } else {
    wrapValue += "wrap";
  }

  if (balanceInput.checked) {
    wrapValue += " balance";
  }

  boxElem.style.flexWrap = wrapValue;
  updateCurrentlySet();
}

function setFlexLineCount() {
  lineCountValue = lineCountInput.value;
  boxElem.style.flexLineCount = lineCountValue;
  outputElem.textContent = lineCountValue;
  updateCurrentlySet();
}

reverseInput.addEventListener("change", setFlexWrap);
balanceInput.addEventListener("change", setFlexWrap);
lineCountInput.addEventListener("input", setFlexLineCount);
```

Results

Change the form control settings to see how they affect the output. Note how:

- Toggling between `wrap` and `wrap-reverse` changes the direction in which the flex lines are laid out along the cross-axis.
- Setting `balance` causes the flex items to be evenly distributed across the available flex lines.
- Changing the `flex-line-count` value changes the minimum number of lines the items will be distributed across. The `flex-line-count` property, and therefore the slider, has no effect unless `balance` is checked.
- As the flex items are distributed over four flex lines by default, changing the `flex-line-count` to a value of less than `5` has no effect. Values from `5` to `10` add an extra flex line.
- As there are 10 flex items, `flex-line-count` values of `10` or greater will all have the same effect — one item will be placed on each of 10 flex lines.

See also

- `flex-direction`
- `flex-flow` shorthand
- Basic concepts of flexbox
- Mastering wrapping of flex items
- CSS flexible box layout module

float CSS property

The **`float`** CSS property places an element on the left or right side of its container, allowing text and inline elements to wrap around it. The element is removed from the normal flow of the page, though still remaining a part of the flow (in contrast to absolute positioning).

Try it

```
float: none;
```

```
float: left;
```

```
float: right;
```

```
float: inline-start;
```

```
float: inline-end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">Float me</div>
    As much mud in the streets as if the waters had but newly retired from the
    face of the earth, and it would not be wonderful to meet a Megalosaurus,
    forty feet long or so, waddling like an elephantine lizard up Holborn Hill.
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  padding: 0.75em;
```

```
  text-align: left;
  width: 80%;
  line-height: normal;
}

#example-element {
  border: solid 10px #efac09;
  background-color: #040d46;
  color: white;
  padding: 1em;
  width: 40%;
}
```

A *floating element* is one where the computed value of `float` is not `none`.

As `float` implies the use of the block layout, it modifies the computed value of the `display` values, in some cases:

Specified value	Computed value
`inline`	`block`
`inline-block`	`block`
`inline-table`	`table`
`table-row`	`block`
`table-row-group`	`block`
`table-column`	`block`
`table-column-group`	`block`
`table-cell`	`block`
`table-caption`	`block`
`table-header-group`	`block`
`table-footer-group`	`block`
`inline-flex`	`flex`
`inline-grid`	`grid`
other	*unchanged*

> **Note:** When accessing a CSS property in JavaScript through the `HTMLElement.style` object, single-word property names are used as is. Although `float` is a reserved keyword in JavaScript, the CSS `float` property is accessed as `float` in modern browsers. In older browsers, you must use `cssFloat` to access the `float` property. (This is similar to how the "class" attribute is accessed as "className" and the "for" attribute of a `<label>` element is accessed as "htmlFor".)

Syntax

```
/* Keyword values */
float: left;
float: right;
float: none;
float: inline-start;
float: inline-end;

/* Global values */
float: inherit;
float: initial;
float: revert;
float: revert-layer;
float: unset;
```

Values

This property is specified as one of the following keyword values:

`left`

The element must float on the left side of its containing block.

`right`

The element must float on the right side of its containing block.

`none`

The element must not float.

`inline-start`

The element must float on the start side of its containing block. That is the left side with `ltr` scripts, and the right side with `rtl` scripts.

`inline-end`

The element must float on the end side of its containing block. That is the right side with `ltr` scripts, and the left side with `rtl` scripts.

Formal definition

Initial value	none
Applies to	all elements, but has no effect if the value of `display` is `none`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
float =
  block-start      |
  block-end        |
  inline-start     |
  inline-end       |
  snap-block       |
  <snap-block()>   |
  snap-inline      |
  <snap-inline()>  |
  left             |
  right            |
  top              |
  bottom           |
  none             |
  footnote

<snap-block()> =
  snap-block( <length> , [ start | end | near ]? )

<snap-inline()> =
  snap-inline( <length> , [ left | right | near ]? )
```

This syntax reflects the latest standard as per CSS Generated Content for Paged Media Module, CSS Page Floats. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

How floated elements are positioned

As mentioned above, when an element is floated, it is taken out of the normal flow of the document (though still remaining part of it). It is shifted to the left, or right, until it touches the edge of its containing box, *or another floated element.*

In this example, there are three colored squares. Two are floated left, and one is floated right. Note that the second "left" square is placed to the right of the first. Additional squares would continue to stack to the right, until they filled the containing box, after which they would wrap to the next line.

A floated element is at least as tall as its tallest nested floated children. We gave the parent `width: 100%` and floated it to ensure it is tall enough to encompass its floated children, and to make sure it takes up the width of the parent so we don't have to clear its adjacent sibling.

HTML

```
<section>
  <div class="left">1</div>
  <div class="left">2</div>
  <div class="right">3</div>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi tristique
    sapien ac erat tincidunt, sit amet dignissim lectus vulputate. Donec id
    iaculis velit. Aliquam vel malesuada erat. Praesent non magna ac massa
    aliquet tincidunt vel in massa. Phasellus feugiat est vel leo finibus
    congue.
  </p>
</section>
```

CSS

```
section {
  box-sizing: border-box;
  border: 1px solid blue;
  width: 100%;
  float: left;
}

div {
  margin: 5px;
  width: 50px;
  height: 150px;
}

.left {
  float: left;
  background: pink;
}

.right {
  float: right;
  background: cyan;
}
```

Result

Clearing floats

Sometimes you may want to force an item to move below any floated elements. For instance, you may want paragraphs to remain adjacent to floats, but force headings to be on their own line. See `clear` for examples.

See also

- Block formatting context
- Use `clear` to force an item to move below a floated element.

flood-color CSS property

The **`flood-color`** CSS property defines the color of the current filter primitive subregion in `<feFlood>` and `<feDropShadow>` elements within a `<filter>`. If present, it overrides the element's `flood-color` attribute.

> **Note:** The `flood-color` property only applies to `<feFlood>` and `<feDropShadow>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* <color> values */
flood-color: red;
flood-color: hsl(120deg 75% 25% / 60%);
flood-color: currentColor;

/* Global values */
flood-color: inherit;
flood-color: initial;
flood-color: revert;
flood-color: revert-layer;
flood-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The flood's color. This can be any valid CSS `<color>` value.

Formal definition

Initial value	`black`
Applies to	`<feFlood>` and `<feDropShadow>` elements in `<svg>`
Inherited	no
Computed value	as specified
Animation type	by computed value

Formal syntax

```
flood-color =
  <color>
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the color of a filters flood

This example demonstrates the basic use case of `flood-color`, and how the CSS `flood-color` property takes precedence over the `flood-color` attribute.

HTML

We have an SVG with two `<filter>` elements, each with a `<feFlood>` child. Each `<feFlood>` element includes the SVG `flood-color` attribute defining the flood color as `seagreen`. We included two `<rect>` elements with a filter attribute; this is where the filters will be displayed.

```
<svg viewBox="0 0 420 120" xmlns="http://www.w3.org/2000/svg">
  <filter id="flood1">
    <feFlood flood-color="seagreen" />
  </filter>
  <filter id="flood2">
    <feFlood flood-color="seagreen" />
  </filter>

  <rect id="r1" filter="url(#flood1)" />
  <rect id="r2" filter="url(#flood2)" />
</svg>
```

CSS

We define the size and position of our `<rect>` using the CSS `height`, `width`, `x`, and `y` properties:

```
rect {
  width: 100px;
  height: 100px;
  x: 10px;
  y: 10px;
}
#r2 {
  x: 150px;
}
```

We then apply different flood color values to the `<feFlood>` elements using the CSS `flood-color` property. We use a named color and a 3-digit hexadecimal color, but we can use any valid CSS color syntax:

```
#flood1 feFlood {
  flood-color: rebeccapurple;
}
#flood2 feFlood {
  flood-color: #ff3366;
}
```

Results

The attributes defined the squares as seagreen, but these values were overridden by the CSS `flood-color` values.

See also

- `flood-opacity`
- `color`
- `fill`
- `lighting-color`
- `stop-color`
- `stroke`
- `box-shadow`

- `text-shadow`
- `background-color`
- `<color>`
- `<filter-function>`
- SVG `flood-color` attribute

flood-opacity CSS property

The **`flood-opacity`** CSS property defines the opacity of the current filter primitive subregion in `<feFlood>` and `<feDropShadow>` elements within a `<filter>`. If present, it overrides the element's `flood-opacity` attribute.

The property value impacts the `flood-color`'s alpha channel; it can increase the transparency of a `flood-color` but cannot make the color defined by the `flood-color` property more opaque.

> **Note:** The `flood-opacity` property only applies to `<feFlood>` and `<feDropShadow>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* numeric and percentage values */
flood-opacity: 0.2;
flood-opacity: 20%;

/* Global values */
flood-opacity: inherit;
flood-opacity: initial;
flood-opacity: revert;
flood-opacity: revert-layer;
flood-opacity: unset;
```

Values

The `<opacity-value>` is a `<number>` or `<percentage>` denoting the opacity of the SVG gradient `<flood>` element.

`<number>`

A numeric value between `0` and `1`, inclusive.

`<percentage>`

A percentage value between `0%` and `100%`, inclusive.

With `0` or `0%` set, the flood is fully transparent. With `1` or `100%` set, the element is the full opacity of the `flood-color` value, which may or may not be partially opaque.

Formal definition

Initial value	`black`
Applies to	`<feFlood>` and `<feDropShadow>` elements in `<svg>`
Inherited	no
Computed value	the specified value, clipped in the range `[0,1]`
Animation type	by computed value

Formal syntax

```
flood-opacity =
  <'opacity'>

<opacity> =
  <opacity-value>

<opacity-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the flood opacity of a filter

This example demonstrates the basic use case of `flood-opacity`, and how the CSS `flood-opacity` property takes precedence over the `flood-opacity` attribute.

HTML

We have an SVG with a few `<filter>` elements, each with a `<feFlood>` child. The `<feFlood>` define the filters as `seagreen`, with the first being declared by its `flood-opacity` attribute as fully opaque and the second being fully transparent. We included two `<rect>` elements, each with a filter attribute.

```
<svg viewBox="0 0 420 120" xmlns="http://www.w3.org/2000/svg">
  <filter id="flood1">
    <feFlood flood-color="seagreen" flood-opacity="1" />
  </filter>
  <filter id="flood2">
    <feFlood flood-color="seagreen" flood-opacity="0" />
  </filter>

  <rect id="r1" filter="url(#flood1)" />
  <rect id="r2" filter="url(#flood2)" />
</svg>
```

CSS

We define the `height`, `width`, `x`, and `y`,positioning of our rectangles with CSS, and include a repeating linear gradient as a `background-image` on the SVG so the opacity of the flood-color is more apparent:

```
svg {
  background-image: repeating-linear-gradient(
    45deg,
    transparent 0 9px,
```

```
    #cccccc 0px 10px
  );
}
rect {
  width: 100px;
  height: 100px;
  x: 10px;
  y: 10px;
}
#r2 {
  x: 150px;
}
```

We then apply different flood opacity values to the `<feFlood>` elements using the CSS `flood-opacity`: property:

```
#flood1 feFlood {
  flood-opacity: 0.5;
}
#flood2 feFlood {
  flood-opacity: 90%;
}
```

Results

The attributes defined the first square as fully opaque and the second as fully transparent, but these values were overridden by the CSS `flood-opacity` values. The seagreen filters are 50% and 90% opaque, respectively.

See also

- `flood-color`
- `fill`
- `stop-opacity`
- `stroke-opacity`
- `opacity`
- `box-shadow`
- `text-shadow`
- `<filter-function>`, including `opacity()`
- SVG `flood-opacity` attribute

font CSS property

The **`font`** CSS shorthand property sets all the different properties of an element's font. Alternatively, it sets an element's font to a system font.

Try it

```
font:
  1.2rem "Fira Sans",
  sans-serif;
```

```
font:
  italic 1.2rem "Fira Sans",
  serif;
```

```
font: italic small-caps bold 16px/2 cursive;
```

```
font: small-caps bold 24px/1 sans-serif;
```

```
font: caption;
```

```
<section id="default-example">
  <q id="example-element">
    Prejudices, it is well known, are most difficult to eradicate from the heart
    whose soil has never been loosened or fertilized by education: they grow
    there, firm as weeds among stones.
  </q>
</section>
```

```
@font-face {
  font-family: "Fira Sans";
  src:
    local("FiraSans-Regular"),
    url("/shared-assets/fonts/FiraSans-Regular.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
}

@font-face {
  font-family: "Fira Sans";
  src:
    local("FiraSans-Italic"),
    url("/shared-assets/fonts/FiraSans-Italic.woff2") format("woff2");
  font-weight: normal;
  font-style: italic;
}

section {
  margin-top: 10px;
  font-size: 1.1em;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `font-family`
- `font-size`

- font-width
- font-style
- font-variant
- font-weight
- line-height

Reset-only sub-properties

This property resets the following CSS properties to their initial values:

- font-feature-settings
- font-kerning
- font-language-override
- font-optical-sizing
- font-size-adjust
- font-variant-alternates
- font-variant-caps
- font-variant-east-asian
- font-variant-emoji
- font-variant-ligatures
- font-variant-numeric
- font-variant-position
- font-variation-settings

Syntax

```
/* font-size font-family */
font: 1.2em sans-serif;

/* font-size/line-height font-family */
font: 1.2em/2 "Fira Sans", sans-serif;

/* font-style font-weight font-size font-family */
font: italic bold 1.2em monospace;

/* font-width font-variant font-size font-family */
font: ultra-condensed small-caps 1.2em Montserrat, Helvetica, sans-serif;

/* system font */
font: caption;
```

Values

The value is either a shorthand specifying the various font-related properties or a single `<system-font-family-name>` keyword:

`<'font-style'>` Optional

See the `font-style` CSS property. Defaults to `normal`.

`<font-variant-css2>` Optional

Either the `normal` or `small-caps` value of the `font-variant` property. Defaults to `normal`.

`<'font-weight'>` Optional

See the `font-weight` CSS property. Defaults to `normal`.

`<font-width-css3>` Optional

The keywords supported by the `font-width` CSS property. Defaults to `normal`.

`<'font-size'>`

See the `font-size` CSS property.

`<'line-height'>` Optional

See the `line-height` CSS property. Defaults to `normal`.

`<'font-family'>`

See the `font-family` CSS property. Must be the last value.

`<system-font-family-name>`

A single keyword representing a system font, including:

`caption`

The system font used for captioned controls (buttons, drop-downs, etc.).

`icon`

The system font used to label icons.

`menu`

The system font used in menus (e.g., dropdown menus and menu lists).

`message-box`

The system font used in dialog boxes.

`small-caption`

The system font used for labeling small controls.

`status-bar`

The system font used in window status bars.

There are several non-standard values implemented with prefixes.

Description

The `font` property value is either a single keyword representing a system-font-family-name or multiple longhand property values used to set all the different properties of an element's font.

System font declarations

If `font` is specified as a `<system-font-family-name>` keyword, the full property value must be set to that single, case-insensitive keyword. Valid values include `caption`, `icon`, `menu`, `message-box`, `small-caption`, or `status-bar`.

Browsers also support non-standard prefixed values:

- Chromium implements `-webkit-control`, `-webkit-small-control`, and `-webkit-mini-control`.
- Webkit includes the Chromium values, and adds `-webkit-body`, `-webkit-pictograph`, and `-webkit-ruby-text`, along with several `-apple-system-*` prefixed system font names.
- Gecko implements `-moz-window`, `-moz-document`, `-moz-desktop`, `-moz-info`, `-moz-dialog`, `-moz-button`, `-moz-pull-down-menu`, `-moz-list`, and `-moz-field`.

The system font, or `<system-font-family-name>`, can only be set with the `font` property. Defining a single keyword value, such as `font: icon`, sets the font's family, size, weight, style, etc., to the values the browser defines for the named system font. These values can all be changed with longhand declarations placed *after* the `font` declaration.

Including any `font` longhand components after the `<system-font-family-name>` keyword within a `font` property value invalidates the declaration. For example, `font: icon small` is invalid.

If a `<system-font-family-name>` keyword appears anywhere in the value other than as the first component, the keyword is treated as an `<ident>` representing a standard `font-family` name. For example, the declaration `font: small icon` sets the `font-family` to a font named `icon`, a non-system font which may or may not exist. This declaration also sets the `font-size` to `small` and resets all the other shorthand component properties to their initial values.

Shorthand font declarations

If `font` is specified as a shorthand for several font-related properties, then:

- it must include values for:
  - `font-size`
  - `font-family`
- it may optionally include values for:
  - `font-style`
  - `font-variant`
  - `font-weight`
  - `font-width`
  - `line-height`

As with any shorthand property, any of the longhand component properties not specified are set to their initial values, possibly overriding values previously set using non-shorthand properties. In addition, the shorthand resets the following properties to their initial values. The shorthand cannot explicitly set them:

- `font-feature-settings`
- `font-kerning`
- `font-language-override`
- `font-optical-sizing`
- `font-size-adjust`
- `font-variant-alternates`
- `font-variant-caps`
- `font-variant-east-asian`
- `font-variant-emoji`
- `font-variant-ligatures`
- `font-variant-numeric`
- `font-variant-position`
- `font-variation-settings`

Shorthand property order

The order of some of the longhand values within the shorthand `font` declaration must follow a few rules:

- Both the `font-size` and `font-family` components are required (except for system font declarations).
- The `font-style`, `font-variant` and `font-weight` components must precede the `font-size` value.
- A `line-height` can only be included if `font-size` is included. If present, the `line-height` must immediately follow the `font-size`, with the two values separated by a forward slash (`/`), for example: `16px / 3`.
- The `font-family` must be the last value specified.

Components with limited values

For backward compatibility, the valid values of the `font-variant` and `font-width` components do not include all the valid values or the longhand equivalents.

The valid values for the `font-variant` component are limited to `normal` or `small-caps`. While no other values are supported, the shorthand `font` declaration resets all the `font-variant-*` longhand properties to `normal`, including `font-variation-settings`, `font-variant-position`, `font-variant-emoji`, `font-variant-caps`, `font-variant-ligatures`, `font-variant-numeric`, `font-variant-east-asian`, and `font-variant-alternates`.

The valid values for the `font-width` component are limited to keyword values: `normal`, `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded`, `ultra-expanded`. The `font-width` longhand property also supports `<percentage>` values, but they are not valid within the shorthand.

Formal definition

Initial value	as each of the properties of the shorthand: • `font-style`: `normal` • `font-variant`: `normal` • `font-weight`: `normal` • `font-stretch`: `normal` • `font-size`: `medium` • `line-height`: `normal` • `font-family`: depends on user agent
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Percentages	as each of the properties of the shorthand:

	• `font-size`: refer to the parent element's font size • `line-height`: refer to the font size of the element itself
Computed value	as each of the properties of the shorthand: • `font-style`: as specified • `font-variant`: as specified • `font-weight`: the keyword or the numerical value as specified, with `bolder` and `lighter` transformed to the real value • `font-stretch`: as specified • `font-size`: absolute `<length>` • `line-height`: for percentage and length values, the absolute length, otherwise as specified • `font-family`: as specified
Animation type	as each of the properties of the shorthand: • `font-style`: by computed value type; `normal` animates as `oblique 0deg` • `font-variant`: discrete • `font-weight`: by computed value type • `font-stretch`: by computed value type • `font-size`: by computed value type • `line-height`: either number or length • `font-family`: discrete

Formal syntax

```
font =
  [ [ <'font-style'> || <font-variant-css2> || <'font-weight'> || <font-width-css3> ]? <'font-size'> [ / <'line-height'> ]? <'font-family'># ]  |
  <system-font-family-name>

<font-style> =
  normal                          |
  italic                          |
  left                            |
  right                           |
  oblique <angle [-90deg,90deg]>?

<font-variant-css2> =
  normal      |
  small-caps

<font-weight> =
  <font-weight-absolute>  |
  bolder                  |
  lighter

<font-width-css3> =
  normal           |
  ultra-condensed  |
  extra-condensed  |
  condensed        |
  semi-condensed   |
  semi-expanded    |
  expanded         |
  extra-expanded   |
  ultra-expanded

<font-size> =
  <absolute-size>            |
  <relative-size>            |
  <length-percentage [0,∞]>  |
  math

<line-height> =
  normal                     |
  <number [0,∞]>             |
  <length-percentage [0,∞]>

<font-family> =
  [ <font-family-name> | <generic-font-family> ]#

<system-font-family-name> =
  caption        |
  icon           |
  menu           |
  message-box    |
  small-caption  |
  status-bar

<font-weight-absolute> =
  normal             |
  bold               |
  <number [1,1000]>

<absolute-size> =
  xx-small   |
  x-small    |
  small      |
  medium     |
  large      |
  x-large    |
  xx-large   |
  xxx-large

<relative-size> =
  larger   |
  smaller

<length-percentage> =
  <length>      |
  <percentage>

<font-family-name> =
  <string>         |
```

```
  <custom-ident>+

<generic-font-family> =
  <generic-font-script-specific>  |
  <generic-font-complete>         |
  <generic-font-incomplete>

<generic-font-script-specific> =
  generic( fangsong )   |
  generic( kai )        |
  generic( khmer-mul )  |
  generic( nastaliq )

<generic-font-complete> =
  serif       |
  sans-serif  |
  system-ui   |
  cursive     |
  fantasy     |
  math        |
  monospace

<generic-font-incomplete> =
  ui-serif       |
  ui-sans-serif  |
  ui-monospace   |
  ui-rounded
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Inline Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example defines the `font` for all `<p>` elements. We set the `font-size` to `12px` and the `line-height` to `14px`, separating them with a forward slash (`/`). The declaration also sets the `font-family` to `sans-serif`.

```
p {
  font: 12px / 14px sans-serif;
}
```

```
<p>
  This is a paragraph of sans-serif text. The font-size is small, at just 12px.
  If this text wraps, the line height is pretty tight at 14px, so this may be
  difficult to read.
</p>
```

Multiple properties

In this example, we set the `font-weight` to `bold`, the `font-style` to `italic`, the `font-size` to `large`, the `line-height` to `1.6`, and the `font-family` to `serif`.

```
p {
  font: bold italic large / 1.6 serif;
}
```

```
<p>
  In this example, we set the font weight to bold, the font style to italic, the
  font size to large, the line height to 1.6, and the font family to serif.
</p>
```

System font

This example demonstrates using the `font` property to set a system font.

CSS

We set the paragraph's font to have the same `font-family`, `line-height`, `font-size`, etc., as the status bar of the window, then we set the `line-height` to `1.6`.

```
p {
  font: status-bar;
  line-height: 1.6;
}
```

HTML

Our HTML includes a paragraph (`<p>`) containing a link (`<a>`) with a convoluted `href` attribute value. When you hover or focus the rendered link, your browser's status bar should display the value of the `href` attribute.

```
<p>
  <a
    href="/%20The%20font%20should%20be%20the%20same%20
family%20and%20size%20and%20the%20text%20in%20the%20example."
    >Hover or focus this text. The font should be the same family and size and
    the text in your status bar.</a
  >
</p>
```

JavaScript

As the URL in our HTML link is not good practice, we include a script that prevents the document from redirecting to a non-existent page when the link is clicked.

```
const aElem = document.querySelector("a");
aElem.addEventListener("click", (e) => {
  e.preventDefault();
  return false;
});
```

Result

Hover or focus the link. The font should be the same family and size as the text in your status bar at the bottom of your browser window.

Shorthand declaration creator

In this live demonstration, you can select different radio buttons to generate different shorthand values, while visualizing the effects of the shorthand declarations you create.

```
<p>
  Change the radio buttons below to see the generated shorthand and its effect.
</p>
<form action="createShortHand()">
  <div class="cf">
    <div class="setPropCont">
      font-style<br />
      <input
        type="radio"
        id="font-style-none"
        name="font_style"
        checked
        value="" />
      <label for="font-style-none">omit value</label><br />
      <input
        type="radio"
        id="font-style-normal"
        name="font_style"
        value="normal" />
      <label for="font-style-normal">normal</label><br />
      <input
        type="radio"
        id="font-style-italic"
        name="font_style"
        value="italic" />
      <label for="font-style-italic">italic</label><br />
      <input
        type="radio"
        id="font-style-oblique"
        name="font_style"
        value="oblique" />
      <label for="font-style-oblique">oblique</label>
    </div>

    <div class="setPropCont">
      font-variant<br />
      <input
        type="radio"
        id="font-variant-none"
        name="font_variant"
        checked
        value=" " />
      <label for="font-variant-none">omit value</label><br />
      <input
        type="radio"
        id="font-variant-normal"
        name="font_variant"
        value="normal" />
      <label for="font-variant-normal">normal</label><br />
      <input
        type="radio"
        id="font-variant-small-caps"
        name="font_variant"
        value="small-caps" />
      <label for="font-variant-small-caps">small-caps</label>
    </div>

    <div class="setPropCont">
      font-weight<br />
      <input type="radio" id="font-weight-none" name="font_weight" value="" />
      <label for="font-weight-none">omit value</label><br />
      <input
        type="radio"
        id="font-weight-normal"
        checked
        name="font_weight"
        value="400" />
      <label for="font-weight-normal">normal</label><br />
      <input
        type="radio"
        id="font-weight-bold"
        name="font_weight"
        value="700" />
      <label for="font-weight-bold">bold</label>
    </div>

    <div class="setPropCont">
      font-size<br />
      <input type="radio" id="font-size-12px" name="font_size" value="12px" />
      <label for="font-size-12px">12px</label><br />
      <input
        type="radio"
        id="font-size-16px"
        name="font_size"
        value="16px"
        checked />
      <label for="font-size-16px">16px</label><br />
      <input type="radio" id="font-size-24px" name="font_size" value="24px" />
      <label for="font-size-24px">24px</label>
    </div>

    <div class="setPropCont">
      line-height<br />
      <input
        type="radio"
        id="line-height-none"
        name="line_height"
        checked
        value="" />
```

```
      <label for="line-height-none">omit value</label><br />
      <input
        type="radio"
        id="line-height-1-2"
        name="line_height"
        value="/1.2" />
      <label for="line-height-1-2">1.2</label><br />
      <input type="radio" id="line-height-3" name="line_height" value="/3" />
      <label for="line-height-3">3</label>
    </div>
    <br />

    <div class="setPropCont fontfamily">
      font-family<br />
      <input
        type="radio"
        id="font-family-courier"
        name="font_family"
        checked
        value="courier" />
      <label for="font-family-courier">courier</label><br />
      <input
        type="radio"
        id="font-family-serif"
        name="font_family"
        value="serif" />
      <label for="font-family-serif">serif</label><br />
      <input
        type="radio"
        id="font-family-sans-serif"
        name="font_family"
        value="sans-serif" />
      <label for="font-family-sans-serif">sans-serif</label><br />
      <input
        type="radio"
        id="font-family-arial"
        name="font_family"
        value="arial" />
      <label for="font-family-arial">Arial</label><br />
      <input
        type="radio"
        id="font-family-monospace"
        name="font_family"
        value="monospace" />
      <label for="font-family-monospace">monospace</label><br />
      <input
        type="radio"
        id="font-family-cursive"
        name="font_family"
        value="cursive" />
      <label for="font-family-cursive">cursive</label><br />
      <input
        type="radio"
        id="font-family-fantasy"
        name="font_family"
        value="fantasy" />
      <label for="font-family-fantasy">fantasy</label><br />
      <input
        type="radio"
        id="font-family-system-ui"
        name="font_family"
        value="system-ui" />
      <label for="font-family-system-ui">system-ui</label><br />
    </div>
  </div>

  <div class="cf propInputs">
    <div class="propInputCont tar">font :</div>
    <div class="propInputCont">
      <input type="text" class="curCss" id="input_font_style" /><br />
      font-style <br />
      optional
    </div>
    <div class="propInputCont">
      <input type="text" class="curCss" id="input_font_variant" /> <br />
      font-variant <br />
      optional
    </div>
    <div class="propInputCont">
      <input type="text" class="curCss" id="input_font_weight" /> <br />
      font-weight <br />
      optional
    </div>
    <div class="propInputCont">
      <input type="text" class="curCss mandatory" id="input_font_size" /> <br />
      font-size <br />
      mandatory
    </div>
    <div class="propInputCont">
      <input type="text" class="curCss" id="input_line_height" /> <br />
      line-height <br />
      optional
    </div>
    <div class="propInputCont">
      <input type="text" class="curCss mandatory" id="input_font_family" />
      <br />
      font-family <br />
      mandatory
    </div>
  </div>
</form>

<div class="fontShortHand">
```

```
  This is some sample text.<br />
  This is some more sample text.
</div>
```

```
body,
input {
  font: 14px "Arial";
  overflow: hidden;
}

.propInputCont {
  float: left;
  text-align: center;
  margin-right: 5px;
  width: 80px;
}

.setPropCont {
  float: left;
  margin-right: 5px;
  width: 120px;
}

.propInputs,
.setPropCont {
  margin-bottom: 1em;
}

.curCss {
  border: none;
  border-bottom: 1px solid black;
  text-align: center;
  width: 80px;
}

.mandatory {
  border-bottom-color: red;
}

.cf::before,
.cf::after {
  content: " ";
  display: table;
}

.cf::after {
  clear: both;
}

.tar {
  width: 40px;
  text-align: right;
}
.fontfamily {
  display: inline-block;
}
```

```
const textAreas = document.getElementsByClassName("curCss");

function getProperties() {
  return (
    `${getCheckedValue("font_style")} ` +
    `${getCheckedValue("font_variant")} ` +
    `${getCheckedValue("font_weight")} ` +
    `${getCheckedValue("font_size")}` +
    `${getCheckedValue("line_height")} ` +
    `${getCheckedValue("font_family")}`
  );
}

function getCheckedValue(radioName) {
  const radios = document.forms[0].elements[radioName];
  for (const radio of radios) {
    if (radio.checked) {
      const curElemName = `input_${radioName}`;
      const curElem = document.getElementById(curElemName);
      curElem.value = radio.value;

      return radio.value;
    }
  }
}

function setCss() {
  injectCss(getProperties());
}

function injectCss(cssFragment) {
  const old = document.body.getElementsByTagName("style");
  if (old.length > 1) {
    old[1].parentElement.removeChild(old[1]);
  }
  css = document.createElement("style");
  css.textContent = `.fontShortHand{font: ${cssFragment}}`;
  document.body.appendChild(css);
}

setCss();

document.querySelectorAll("input[type='radio']").forEach((el) => {
  el.addEventListener("change", setCss);
});
```

See also

- `font-style`
- `font-weight`
- System font stack on CSS-Tricks (2017)

font-family CSS property

The **`font-family`** CSS property specifies a prioritized list of one or more font family names and/or generic family names for the selected element.

Try it

```
font-family: "Georgia", serif;
```

```
font-family: "Gill Sans", sans-serif;
```

```
font-family: sans-serif;
```

```
font-family: serif;
```

```
font-family: cursive;
```

```
font-family: system-ui;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
section {
  font-size: 1.2em;
}
```

Values are separated by commas to indicate that they are alternatives. The browser will select the first font in the list that is installed or that can be downloaded using a `@font-face` at-rule.

It is often convenient to use the shorthand property `font` to set `font-size` and other font related properties all at once.

You should always include at least one generic family name in a `font-family` list, since there's no guarantee that any given font is available. This lets the browser select an acceptable fallback font when necessary.

The `font-family` property specifies a list of fonts, from highest priority to lowest. Font selection *does not* stop at the first font in the list that is on the user's system. Rather, font selection is done *one character at a time*, so that if an available font does not have a glyph for a needed character, the latter fonts are tried. When a font is only available in some styles, variants, or sizes, those properties may also influence which font family is chosen.

Syntax

```
/* A font family name and a generic family name */
font-family: "Gill Sans Extrabold", sans-serif;
font-family: "Goudy Bookletter 1911", sans-serif;

/* A generic family name only */
font-family: serif;
font-family: sans-serif;
font-family: monospace;
font-family: cursive;
font-family: fantasy;
font-family: system-ui;
font-family: ui-serif;
font-family: ui-sans-serif;
font-family: ui-monospace;
font-family: ui-rounded;
font-family: math;
font-family: fangsong;

/* Global values */
font-family: inherit;
font-family: initial;
font-family: revert;
font-family: revert-layer;
font-family: unset;
```

The `font-family` property lists one or more font families, separated by commas. Each font family is specified as either a `<family-name>` or a `<generic-name>` value.

The example below lists two font families, the first with a `<family-name>` and the second with a `<generic-name>`:

```
font-family: "Gill Sans Extrabold", sans-serif;
```

Values

`<family-name>`

> The name of a font family. This must be either a single `<string>` value or a space-separated sequence of `<custom-ident>` values. String values must be quoted but may contain any Unicode character. Custom identifiers are not quoted, but certain characters must be escaped.
>
> It is good practice to quote font family names that contain white space, digits, or punctuation characters other than hyphens.
>
> See also Valid family names.

`<generic-name>`

> Generic font families are a fallback mechanism, a means of preserving some of the style sheet author's intent when none of the specified fonts are available. Generic family names are keywords and must not be quoted. A generic font family should be the last item in the list of font family names. The following keywords are defined:

`serif`

Glyphs have finishing strokes, flared or tapering ends, or have actual serifed endings.

For example: Lucida Bright, Lucida Fax, Palatino, Palatino Linotype, Palladio, URW Palladio, serif.

`sans-serif`

Glyphs have stroke endings that are plain.

For example: Open Sans, Fira Sans, Lucida Sans, Lucida Sans Unicode, Trebuchet MS, Liberation Sans, Nimbus Sans L, sans-serif.

`monospace`

All glyphs have the same fixed width.

For example: Fira Mono, DejaVu Sans Mono, Menlo, Consolas, Liberation Mono, Monaco, Lucida Console, monospace.

> **Note:** When `font-family` is set to the single keyword `monospace` and no `font-size` is set, browsers use the user's monospace font size preference, which is often smaller than the size used for proportional fonts. See Monospace font size for details and how to control this.

`cursive`

Glyphs in cursive fonts generally have either joining strokes or other cursive characteristics beyond those of italic typefaces. The glyphs are partially or completely connected, and the result looks more like handwritten pen or brush writing than printed letter work.

For example: Brush Script MT, Brush Script Std, Lucida Calligraphy, Lucida Handwriting, Apple Chancery, cursive.

`fantasy`

Fantasy fonts are primarily decorative fonts that contain playful representations of characters.

For example: Papyrus, Herculanum, Party LET, Curlz MT, Harrington, fantasy.

`system-ui`

Glyphs are taken from the default user interface font on a given platform. Because typographic traditions vary widely across the world, this generic is provided for typefaces that don't map cleanly into the other generics.

> **Note:** As the name implies, `system-ui` is intended to make UI elements look like native apps, and not for typesetting large paragraphs of text. It may cause the displayed typeface to be undesirable for some users—for example, the default Windows CJK font may render Latin scripts poorly, and the `lang` attribute may not affect the displayed font. Some operating systems do not allow customizing `system-ui`, while browsers generally allow customizing the `sans-serif` font family. For large paragraphs, use `sans-serif` or some other non-UI font family instead.

`ui-serif`

The default user interface serif font.

`ui-sans-serif`

The default user interface sans-serif font.

`ui-monospace`

The default user interface monospace font.

`ui-rounded`

The default user interface font that has rounded features.

`math`

Font that addresses the particular stylistic concerns of representing mathematics: superscript and subscript, brackets that cross several lines, nesting expressions, and double struck glyphs with distinct meanings. UA stylesheets may set `math { font-family: math }` so that the `<math>` element uses appropriate fonts by default.

`fangsong`

A particular style of Chinese characters that are between serif-style Song and cursive-style Kai forms. This style is often used for government documents.

Formal definition

Initial value	depends on user agent
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-family =
  [ <font-family-name> | <generic-font-family> ]#

<font-family-name> =
  <string>         |
  <custom-ident>+

<generic-font-family> =
  <generic-font-script-specific>  |
  <generic-font-complete>         |
  <generic-font-incomplete>

<generic-font-script-specific> =
  generic( fangsong )   |
  generic( kai )        |
  generic( khmer-mul )  |
  generic( nastaliq )

<generic-font-complete> =
  serif       |
```

```
  sans-serif  |
  system-ui   |
  cursive     |
  fantasy     |
  math        |
  monospace

<generic-font-incomplete> =
  ui-serif       |
  ui-sans-serif  |
  ui-monospace   |
  ui-rounded
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Some common font families

```
.serif {
  font-family: "Times", "Times New Roman", "Georgia", serif;
}

.sansserif {
  font-family: "Verdana", "Helvetica", "Arial", sans-serif;
}

.monospace {
  font-family: "Lucida Console", "Courier New", monospace;
}

.cursive {
  font-family: cursive;
}

.fantasy {
  font-family: fantasy;
}

.math {
  font-family: math;
}

.fangsong {
  font-family: fangsong;
}
```

```
div {
  margin: 0.5rem;
}
```

```
<div class="serif">This is an example of a serif font.</div>

<div class="sansserif">This is an example of a sans-serif font.</div>

<div class="monospace">This is an example of a monospace font.</div>

<div class="cursive">This is an example of a cursive font.</div>

<div class="fantasy">This is an example of a fantasy font.</div>

<div class="fangsong">This is an example of a fangsong font.</div>

<div class="math">This is an example of a math font: ℝ, ∫, ∑…</div>
```

Monospace font size

Browsers maintain separate default font size preferences for proportional fonts (such as `serif` and `sans-serif`) and for monospace fonts. Typical defaults are `16px` for proportional fonts and `13px` for monospace fonts.

When `font-family` is set to the single keyword `monospace`, browsers apply the monospace size preference. If users have not changed their browser preferences and `font-size` is not explicitly set, monospace text may be rendered smaller than surrounding content. The same reduction applies to elements whose user agent stylesheet sets `font-family: monospace`, such as `<code>` and `<pre>`.

If the `font-family` value includes more than one family, even if the value is `monospace, monospace`, browsers do not apply the monospace font size preference, and will render the text at the inherited `font-size`. Declaring `font-family: monospace, monospace` (or a named monospace font followed by `monospace`) therefore renders at the same size as surrounding proportional text. This behavior is not specified in CSS, but it is interoperable across the major browser engines.

```
<p class="proportional">
  This proportional text is at the default size; generally 16px.
</p>
<p class="mono-only">
  This monospace text is at the preferred size for monospace fonts; generally
  13px.
</p>
<p class="mono-list">
  This is in "monospace, monospace" font, rendered at the same size as
  proportional text; generally 16px.
</p>
```

```
.mono-only {
  font-family: monospace;
}

.mono-list {
  font-family: monospace, monospace;
}
```

If you haven't changed your font size in your browser settings, the first and last paragraphs will be the same font-size, rendered at the browser preferred font size. The middle paragraph will be rendered at the preferred mono-space font size, which is generally set to be smaller than the preferred font-size for other text.

Valid family names

The following declarations are valid:

```
font-family: "Goudy Bookletter 1911", sans-serif;
```

The following declarations are invalid:

```
font-family: Goudy Bookletter 1911, sans-serif;
font-family: Red/Black, sans-serif;
font-family: "Lucida" Grande, sans-serif;
font-family: Ahem!, sans-serif;
font-family: test@foo, sans-serif;
font-family: #POUND, sans-serif;
font-family: Hawaii 5-0, sans-serif;
```

The following example is technically valid but is not recommended:

```
font-family:
  Gill Sans Extrabold,
  sans-serif;
```

See also

- `font-style`
- `font-weight`
- `font-variant-emoji`
- SVG `font-family` attribute
- Learn: Fundamental text and font styling

font-feature-settings CSS property

The **`font-feature-settings`** CSS property controls advanced typographic features in OpenType fonts.

Try it

```
font-feature-settings: normal;
```

```
font-feature-settings: "liga" 0;
```

```
font-feature-settings: "tnum";
```

```
font-feature-settings: "smcp", "zero";
```

```
<section id="default-example">
  <div id="example-element">
    <p>Difficult waffles</p>
    <table>
      <tbody>
        <tr>
          <td><span class="tabular">0O</span></td>
        </tr>
        <tr>
          <td><span class="tabular">3.14</span></td>
        </tr>
        <tr>
          <td><span class="tabular">2.71</span></td>
        </tr>
      </tbody>
    </table>
  </div>
</section>
```

```
@font-face {
  font-family: "Fira Sans";
  src:
    local("FiraSans-Regular"),
    url("/shared-assets/fonts/FiraSans-Regular.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
}

section {
  font-family: "Fira Sans", sans-serif;
  margin-top: 10px;
  font-size: 1.5em;
}

#example-element table {
  margin-left: auto;
  margin-right: auto;
}

.tabular {
  border: 1px solid;
}
```

Syntax

```
/* Use the default settings */
font-feature-settings: normal;

/* Set values for OpenType feature tags */
font-feature-settings: "smcp";
font-feature-settings: "smcp" on;
font-feature-settings: "swsh" 2;
font-feature-settings:
```

```
  "smcp",
  "swsh" 2;

/* Global values */
font-feature-settings: inherit;
font-feature-settings: initial;
font-feature-settings: revert;
font-feature-settings: revert-layer;
font-feature-settings: unset;
```

Whenever possible, Web authors should instead use the `font-variant` shorthand property or an associated longhand property such as `font-variant-ligatures`, `font-variant-caps`, `font-variant-east-asian`, `font-variant-alternates`, `font-variant-numeric` or `font-variant-position`.

These lead to more effective, predictable, understandable results than `font-feature-settings`, which is a low-level feature designed to handle special cases where no other way exists to enable or access an OpenType font feature. In particular, `font-feature-settings` shouldn't be used to enable small caps.

Values

This property is specified as either the keyword `normal` or as a comma-separated list of `<feature-tag-value>` values. When rendering text, the list of OpenType `<feature-tag-value>` values are passed to the text layout engine to enable or disable font features.

`normal`

Indicates that text is laid out using default font settings. This is the default value.

`<feature-tag-value>`

Represents a space-separated tuple consisting of a tag name and an optional value.

Tag name

The tag name is always a `<string>` of four ASCII characters. If the tag name has more or fewer characters or if it contains characters outside the `U+20` – `U+7E` code point range, the descriptor is invalid.

Optional value

The optional value can be a positive integer or the keyword `on` or `off`. The keywords `on` and `off` are synonyms for the values `1` and `0`, respectively. If no value is set, the default is `1`. For non-boolean OpenType features (e.g., stylistic alternates), the value implies a particular glyph to be selected; for boolean features, the value turns the feature on or off.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-feature-settings =
  normal               |
  <feature-tag-value>#

<feature-tag-value> =
  <opentype-tag> [ <integer [0,∞]> | on | off ]?

<opentype-tag> =
  <string>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Enabling various font features

```
/* use small-cap alternate glyphs */
.small-caps {
  font-feature-settings: "smcp" on;
}

/* convert both upper and lowercase to small caps (affects punctuation also) */
.all-small-caps {
  font-feature-settings: "c2sc", "smcp";
}

/* use zeros with a slash through them to differentiate from "O" */
.nice-zero {
  font-feature-settings: "zero";
}

/* enable historical forms */
.historical {
  font-feature-settings: "hist";
}

/* disable common ligatures, usually on by default */
.no-ligatures {
  font-feature-settings: "liga" 0;
}

/* enable tabular (monospaced) figures */
td.tabular {
  font-feature-settings: "tnum";
}

/* enable automatic fractions */
.fractions {
```

```
  font-feature-settings: "frac";
}

/* use the second available swash character */
.swash {
  font-feature-settings: "swsh" 2;
}

/* enable stylistic set 7 */
.fancy-style {
  font-family: "Gabriola", cursive;
  font-feature-settings: "ss07";
}
```

See also

- `font-display`
- `font-family`
- `font-stretch`
- `font-style`
- `font-weight`
- `font-variation-settings`
- `src`
- `unicode-range`
- OpenType feature tags list
- OpenType features in CSS

font-kerning CSS property

The **`font-kerning`** CSS property sets the use of the kerning information stored in a font.

Try it

```
font-kerning: auto;
```

```
font-kerning: normal;
```

```
font-kerning: none;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    "We took Tracy to see 'THE WATERFALL' in W. Virginia."
  </div>
</section>
```

```
section {
  font-family: serif;
}
```

Kerning affects how letters are spaced. In *well-kerned* fonts, this feature makes character spacing more uniform and pleasant to read by reducing white space between certain character combinations.

In the image below, for instance, the examples on the left do not use kerning, while the ones on the right do:

Syntax

```
font-kerning: auto;
font-kerning: normal;
font-kerning: none;

/* Global values */
font-kerning: inherit;
font-kerning: initial;
font-kerning: revert;
font-kerning: revert-layer;
font-kerning: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The browser determines whether font kerning should be used or not. For example, some browsers will disable kerning on small fonts, since applying it could harm the readability of text.

`normal`

Font kerning information stored in the font must be applied.

`none`

Font kerning information stored in the font is disabled.

Formal definition

Initial value	`auto`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-kerning =
  auto    |
  normal  |
  none
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Enabling and disabling kerning

HTML

```
<div id="kern"></div>
<div id="no-kern"></div>
<textarea id="input">AV T. ij</textarea>
```

CSS

```
div {
  font-size: 2rem;
  font-family: serif;
}

#no-kern {
  font-kerning: none;
}

#kern {
  font-kerning: normal;
}
```

JavaScript

```
const input = document.getElementById("input");
const kern = document.getElementById("kern");
const noKern = document.getElementById("no-kern");

input.addEventListener("keyup", () => {
  kern.textContent = input.value; /* Update content */
  noKern.textContent = input.value;
});

kern.textContent = input.value; /* Initialize content */
noKern.textContent = input.value;
```

See also

- `font-variant`, `font-variant-position`, `font-variant-east-asian`, `font-variant-caps`, `font-variant-ligatures`, `font-variant-numeric`, `font-variant-alternates`, `font-synthesis`, `letter-spacing`

font-language-override CSS property

The **`font-language-override`** CSS property controls the use of language-specific glyphs in a typeface.

By default, HTML's `lang` attribute tells browsers to display glyphs designed specifically for that language. For example, a lot of fonts have a special character for the digraph `fi` that merge the dot on the "i" with the "f." However, if the language is set to Turkish the typeface will likely know not to use the merged glyph; Turkish has two versions of the "i," one with a dot (`i`) and one without (`ı`), and using the ligature would incorrectly transform a dotted "i" into a dotless "i."

The `font-language-override` property lets you override the typeface behavior for a specific language. This is useful, for example, when the typeface you're using lacks proper support for the language. For instance, if a typeface doesn't have proper rules for the Azeri language, you can force the font to use Turkish glyphs, which follow similar rules.

Syntax

```
/* Keyword value */
font-language-override: normal;

/* <string> values */
font-language-override: "ENG"; /* Use English glyphs */
font-language-override: "TRK"; /* Use Turkish glyphs */

/* Global values */
font-language-override: inherit;
font-language-override: initial;
font-language-override: revert;
font-language-override: revert-layer;
font-language-override: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

Tells the browser to use font glyphs that are appropriate for the language specified by the `lang` attribute. This is the default value.

`<string>`

Tells the browser to use font glyphs that are appropriate for the language specified by the string. The string must match a language tag found in the OpenType language system. For example, "ENG" is English, and "KOR" is Korean.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-language-override =
  normal    |
  <string>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using Danish glyphs

HTML

```
<p class="para1">Default language setting.</p>
<p class="para2">
  This is a string with the <code>font-language-override</code> set to Danish.
</p>
```

CSS

```
p.para1 {
  font-language-override: normal;
}

p.para2 {
  font-language-override: "DAN";
}
```

Result

See also

- `font-variant`, `font-variant-position`, `font-variant-east-asian`, `font-variant-caps`, `font-variant-ligatures`, `font-variant-numeric`, `font-variant-alternates`, `font-synthesis`, `font-kerning`.

font-optical-sizing CSS property

The **`font-optical-sizing`** CSS property sets whether text rendering is optimized for viewing at different sizes.

Try it

```
font-optical-sizing: auto;
```

```
font-optical-sizing: none;
```

```
<section id="default-example">
  <div id="example-element">
    <h2>Chapter 3</h2>
    <p>
      On this particular Thursday, something was moving quietly through the
      ionosphere many miles above the surface of the planet; several somethings
      in fact, several dozen huge yellow chunky slablike somethings, huge as
      office blocks, silent as birds.
    </p>
  </div>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2");
  font-family: "Amstelvar";
  font-style: normal;
}

#example-element {
  font-family: "Amstelvar", serif;
  text-align: left;
}

#example-element h2 {
  font-size: 36px;
}

#example-element p {
  font-size: 12px;
}
```

Syntax

```
/* keyword values */
font-optical-sizing: none;
font-optical-sizing: auto; /* default */

/* Global values */
font-optical-sizing: inherit;
font-optical-sizing: initial;
font-optical-sizing: revert;
```

```
font-optical-sizing: revert-layer;
font-optical-sizing: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

The browser will not modify the shape of glyphs for optimal viewing.

`auto`

The browser will modify the shape of glyphs for optimal viewing.

Description

Optical sizing is enabled by default for fonts that have an optical size variation axis. The optical size variation axis is represented by `opsz` in `font-variation-settings`.

When optical sizing is used, small text sizes are often rendered with thicker strokes and larger serifs, whereas larger text is often rendered more delicately with more contrast between thicker and thinner strokes.

Formal definition

Initial value	`auto`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-optical-sizing =
  auto  |
  none
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling optical sizing

```
<p class="optical-sizing">
  This paragraph is optically sized. This is the default across browsers.
</p>

<p class="no-optical-sizing">
  This paragraph is not optically sized. You should see a difference in
  supporting browsers.
</p>
```

```
@font-face {
  src: url("AmstelvarAlpha-VF.woff2");
  font-family: "Amstelvar";
  font-style: normal;
}

p {
  font-size: 36px;
  font-family: "Amstelvar", serif;
}

.no-optical-sizing {
  font-optical-sizing: none;
}
```

> **Note:** The font referenced above — which includes optical sizing and is freely-licensed — is good for testing. You can download it on GitHub.

See also

- `font-size`
- `font-size-adjust`
- Learn: Fundamental text and font styling

font-palette CSS property

The **`font-palette`** CSS property allows specifying one of the many palettes contained in a color font that a user agent may use for the font. Users can also override the values in a palette or create a new palette by using the `@font-palette-values` at-rule.

> **Note:** A `font-palette` palette takes precedence when coloring a font. The `color` property will not override a font palette, even if specified with `!important`.

Syntax

```
/* Using a font-defined palette */
font-palette: normal;

/* Using a user-defined palette */
font-palette: --one;

/* Creating a new palette by blending two others */
font-palette: palette-mix(in lch, --blue, --yellow);
```

Values

This property is specified as a single value from the following list:

`normal`

Specifies the default color palette or the default glyph colorization (set by the font maker) to be used for the font. With this setting, the palette in the font at index 0 is rendered.

`light`

Specifies the first palette in the font that matches 'light' to be used for the font. Some fonts contain metadata that identify a palette as applicable for a light (close to white) background. If a font does not have this metadata, the `light` value behaves as `normal`.

`dark`

Specifies the first palette in the font that matches 'dark' to be used for the font. Some fonts contain metadata that identify a palette as applicable for a dark (close to black) background. If a font does not have this metadata, the value behaves as `normal`.

`<dashed-ident>`

Allows you to specify your own values for the font palette by using the `<palette-identifier>` of a @font-palette-values at-rule.

`palette-mix()`

Creates a new `font-palette` value by blending together two `font-palette` values by specified percentages and color interpolation methods.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	by computed value

Formal syntax

```
font-palette =
  normal                |
  light                 |
  dark                  |
  <palette-identifier>  |
  <palette-mix()>

<palette-identifier> =
  <dashed-ident>

<palette-mix()> =
  palette-mix( <color-interpolation-method>? , [ <'font-palette'> && <percentage [0,100]>? ]# )

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<font-palette> =
  normal                |
  light                 |
  dark                  |
  <palette-identifier>  |
  <palette-mix()>

<rectangular-color-space> =
  srgb              |
  srgb-linear       |
  display-p3        |
  display-p3-linear |
  a98-rgb           |
  prophoto-rgb      |
  rec2020           |
  lab               |
  oklab             |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<xyz-space> =
  xyz     |
  xyz-d50 |
  xyz-d65
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying a dark palette

This example allows you to use the first palette marked as *dark* (works best on a near black background) by the font-maker.

```
@media (prefers-color-scheme: dark) {
  .banner {
    font-palette: dark;
  }
}
```

Animating between two palettes

This example illustrates how to animate `font-palette` value changes to create a smooth font animation.

HTML

The HTML contains a single paragraph of text to animate:

```
<p>color-palette<br />animation</p>
```

CSS

In the CSS, we import a color font called Nabla from Google Fonts, and define two custom `font-palette` values using the `@font-palette-values` at-rule. We then create `@keyframes` that animate between these two palettes, and apply this animation to our paragraph.

```
@import "https://fonts.googleapis.com/css2?family=Nabla&display=swap";

@font-palette-values --blue-nabla {
  font-family: "Nabla";
  base-palette: 2; /* this is Nabla's blue palette */
}

@font-palette-values --grey-nabla {
  font-family: "Nabla";
  base-palette: 3; /* this is Nabla's grey palette */
}

@keyframes animate-palette {
  from {
    font-palette: --grey-nabla;
  }

  to {
    font-palette: --blue-nabla;
  }
}

p {
  font-family: "Nabla", fantasy;
  font-size: 5rem;
  margin: 0;
  text-align: center;
  animation: animate-palette 4s infinite alternate linear;
}
```

Result

The output looks like this:

> **Note:** Browsers that still implement `discrete` `font-palette` animation will flip between the two palettes rather than smoothly animating.

See also

- `palette-mix()`
- `@font-palette-values`
- `base-palette` descriptor
- `font-family` descriptor
- `override-colors` descriptor

palette-mix() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`palette-mix()`** CSS function can be used to create a new `font-palette` value by blending together two `font-palette` values by specified percentages and color interpolation methods.

Syntax

```
/* Blending font-defined palettes */
font-palette: palette-mix(in lch, normal, dark)

/* Blending author-defined palettes */
font-palette: palette-mix(in lch, --blues, --yellows)

/* Varying percentage of each palette mixed */
font-palette: palette-mix(in lch, --blues 50%, --yellows 50%)
font-palette: palette-mix(in lch, --blues 70%, --yellows 30%)

/* Varying color interpolation method */
font-palette: palette-mix(in srgb, --blues, --yellows)
font-palette: palette-mix(in hsl, --blues, --yellows)
font-palette: palette-mix(in hsl shorter hue, --blues, --yellows)
```

Values

Functional notation:

```
palette-mix(method, palette1 [p1], palette2 [p2])
```

`method`

A `<color-interpolation-method>` specifying the interpolation color space.

`palette1`, `palette2`

The `font-palette` values to blend together. These can be *any* `font-palette` values, including `palette-mix()` functions, `normal`, `dark`, and `light`.

`p1`, `p2` Optional

`<percentage>` values between `0%` and `100%` specifying the amount of each palette to mix. They are normalized as follows:

- If both `p1` and `p2` are omitted, then `p1 = p2 = 50%`.
- If `p1` is omitted, then `p1 = 100% - p2`.
- If `p2` is omitted, then `p2 = 100% - p1`.
- If `p1 = p2 = 0%`, the function is invalid.
- If `p1 + p2 ≠ 100%`, then `p1' = p1 / (p1 + p2)` and `p2' = p2 / (p1 + p2)`, where `p1'` and `p2'` are the normalization results.

Formal syntax

```
<palette-mix()> =
  palette-mix( <color-interpolation-method>? , [ <'font-palette'> && <percentage [0,100]>? ]# )

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<font-palette> =
  normal                |
  light                 |
  dark                  |
  <palette-identifier>  |
  <palette-mix()>

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<palette-identifier> =
  <dashed-ident>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using <code>palette-mix()</code> to blend two palettes

This example shows how to use the `palette-mix()` function to create a new palette by blending two others.

HTML

The HTML contains three paragraphs to apply our font information to:

```
<p class="yellowPalette">Yellow palette</p>
<p class="bluePalette">Blue palette</p>
<p class="mixedPalette">Mixed palette</p>
```

CSS

In the CSS, we import a color font from Google Fonts, and define two custom `font-palette` values using the `@font-palette-values` at-rule. We then apply three different `font-palette` values to the paragraphs — `--yellow`, `--blue`, and a new green palette, created using `palette-mix()` to blend the blue and yellow palettes together.

```
@import "https://fonts.googleapis.com/css2?family=Nabla&display=swap";

@font-palette-values --blue-nabla {
  font-family: "Nabla";
  base-palette: 2; /* this is Nabla's blue palette */
}

@font-palette-values --yellow-nabla {
  font-family: "Nabla";
  base-palette: 7; /* this is Nabla's yellow palette */
}

p {
  font-family: "Nabla", fantasy;
  font-size: 4rem;
  text-align: center;
  margin: 0;
}

.yellowPalette {
  font-palette: --yellow-nabla;
}

.bluePalette {
  font-palette: --blue-nabla;
}
```

```
.mixedPalette {
  font-palette: palette-mix(in lch, --blue-nabla 55%, --yellow-nabla 45%);
}
```

Result

The output looks like this:

See also

- `font-palette`
- `@font-palette-values`
- `color-mix()`
- CSS color values guide
- Color space

font-size CSS property

The **`font-size`** CSS property sets the size of the font. Changing the font size also updates the sizes of the font size-relative `<length>` units, such as `em`, `ex`, and so forth.

Try it

```
font-size: 1.2rem;
```

```
font-size: x-small;
```

```
font-size: smaller;
```

```
font-size: 12px;
```

```
font-size: 80%;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

Syntax

```
/* <absolute-size> values */
font-size: xx-small;
font-size: x-small;
font-size: small;
font-size: medium;
font-size: large;
font-size: x-large;
font-size: xx-large;
font-size: xxx-large;

/* <relative-size> values */
font-size: smaller;
font-size: larger;

/* <length> values */
font-size: 12px;
font-size: 0.8em;

/* <percentage> values */
font-size: 80%;

/* math value */
font-size: math;

/* Global values */
font-size: inherit;
font-size: initial;
font-size: revert;
font-size: revert-layer;
font-size: unset;
```

Values

This property is specified as a single value from the following list:

`xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, `xx-large`, `xxx-large`

Absolute-size keywords, based on the user's default font size (which is `medium`).

`larger`, `smaller`

Relative-size keywords. The font will be larger or smaller relative to the parent element's font size, roughly by the ratio used to separate the absolute-size keywords above.

`<length>`

A positive `<length>` value. For most font-relative units (such as `em` and `ex`), the font size is relative to the parent element's font size.

For font-relative units that are root-based (such as `rem`), the font size is relative to the size of the font used by the `<html>` (root) element.

`<percentage>`

A positive `<percentage>` value, relative to the parent element's font size.

> **Note:** To maximize accessibility, it is generally best to use values that are relative to the user's default font size.

`math`

> Scaling rules are applied when determining the computed value of the `font-size` property for math elements relative to the `font-size` of the containing parent. See the math-depth property for more information.

Description

There are several ways to specify the font size, including keywords or numerical values for pixels or ems. Choose the appropriate method based on the needs of the particular web page.

Keywords

Keywords are a good way to set the size of fonts on the web. By setting a keyword font size on the `<body>` element, you can set relative font-sizing everywhere else on the page, giving you the ability to easily scale the font up or down on the entire page accordingly.

Pixels

Setting the font size in pixel values (`px`) is a good choice when you need pixel accuracy. A px value is static. This is an OS-independent and cross-browser way of literally telling the browsers to render the letters at exactly the number of pixels in height that you specified. The results may vary slightly across browsers, as they may use different algorithms to achieve a similar effect.

Font sizing settings can also be used in combination. For example, if a parent element is set to `16px` and its child element is set to `larger`, the child element displays larger than the parent element on the page.

> **Note:** Defining font sizes in `px` is *not accessible*, because the user cannot change the font size in some browsers. For example, users with limited vision may wish to set the font size much larger than the size chosen by a web designer. Avoid using them for font sizes if you wish to create an inclusive design.

Ems

Using an `em` value creates a dynamic or computed font size (historically the `em` unit was derived from the width of a capital "M" in a given typeface.). The numeric value acts as a multiplier of the `font-size` property of the element on which it is used. Consider this example:

```
p {
  font-size: 2em;
}
```

In this case, the font size of `<p>` elements will be double the computed `font-size` inherited by `<p>` elements. By extension, a `font-size` of `1em` equals the computed `font-size` of the element on which it is used.

If a `font-size` has not been set on any of the `<p>`'s ancestors, then `1em` will equal the default browser `font-size`, which is usually `16px`. So, by default `1em` is equivalent to `16px`, and `2em` is equivalent to `32px`. If you were to set a `font-size` of 20px on the `<body>` element say, then `1em` on the `<p>` elements would instead be equivalent to `20px`, and `2em` would be equivalent to `40px`.

In order to calculate the `em` equivalent for any pixel value required, you can use this formula:

```
em = desired element pixel value / parent element font-size in pixels
```

For example, suppose the `font-size` of the `<body>` of the page is set to `16px`. If the font-size you want is `12px`, then you should specify `0.75em` (because 12/16 = 0.75). Similarly, if you want a font size of `10px`, then specify `0.625em` (10/16 = 0.625); for `22px`, specify `1.375em` (22/16).

The `em` is a very useful unit in CSS since it automatically adapts its length relative to the font that the reader chooses to use.

One important fact to keep in mind: em values compound. Take the following HTML and CSS:

```
html {
  font-size: 100%;
}
span {
  font-size: 1.6em;
}
```

```
<div>
  <span>Outer <span>inner</span> outer</span>
</div>
```

The result is:

Assuming that the browser's default `font-size` is 16px, the words "outer" would be rendered at 25.6px, but the word "inner" would be rendered at 40.96px. This is because the inner `<span>`'s `font-size` is 1.6em which is relative to its parent's `font-size`, which is in turn relative to its parent's `font-size`. This is often called **compounding**.

Rems

`rem` values were invented in order to sidestep the compounding problem. `rem` values are relative to the root `html` element, not the parent element. In other words, it lets you specify a font size in a relative fashion without being affected by the size of the parent, thereby eliminating compounding.

The CSS below is nearly identical to the previous example. The only exception is that the unit has been changed to `rem`.

```
html {
  font-size: 100%;
}
span {
  font-size: 1.6rem;
}
```

Then we apply this CSS to the same HTML, which looks like this:

```
<span>Outer <span>inner</span> outer</span>
```

In this example, the words "outer inner outer" are all displayed at 25.6px (assuming that the browser's `font-size` has been left at the default value of 16px).

Ex

Like the `em` unit, an element's `font-size` set using the `ex` unit is computed or dynamic. It behaves in exactly the same way, except that when setting the `font-size` property using `ex` units, the `font-size` equals the x-height of the first available font used on the page. The number value multiplies the element's inherited `font-size` and the `font-size` compounds relatively.

See the W3C Editor's Draft for a more detailed description of font-relative length units such as `ex`.

Formal definition

Initial value	`medium`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Percentages	refer to the parent element's font size
Computed value	absolute `<length>`
Animation type	by computed value type

Formal syntax

```
font-size =
  <absolute-size>           |
  <relative-size>           |
  <length-percentage [0,∞]> |
  math

<absolute-size> =
  xx-small   |
  x-small    |
  small      |
  medium     |
  large      |
  x-large    |
  xx-large   |
  xxx-large

<relative-size> =
  larger   |
  smaller

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting font sizes

CSS

```
.small {
  font-size: xx-small;
}
.larger {
  font-size: larger;
}
.point {
  font-size: 24pt;
}
.percent {
  font-size: 200%;
}
```

HTML

```
<h1 class="small">Small H1</h1>
<h1 class="larger">Larger H1</h1>
<h1 class="point">24 point H1</h1>
<h1 class="percent">200% H1</h1>
```

Result

See also

- `font-size-adjust`
- `font-style`
- `font-weight`
- `math-depth`
- `math-style`
- SVG `font-size` attribute
- Learn: Fundamental text and font styling

font-size-adjust CSS property

The **`font-size-adjust`** CSS property adjusts the size of a font to match a chosen font metric, such as the height of lowercase letters, to a specified proportion of the `font-size`. This property is intended for adjusting fallback fonts to have similar metrics as the first-choice font.

Legibility can become an issue when the first-choice `font-family` is unavailable and its replacement fallback font has a significantly different aspect value (height of lowercase letters divided by font size). Legibility of fonts, especially at small font sizes, is determined more by the size of lowercase letters than by the size of uppercase letters. The `font-size-adjust` property is useful for adjusting the font size of fallback fonts to keep the aspect value across fonts consistent, ensuring that the text appears similar regardless of the font used.

Syntax

```
/* Keyword value */
font-size-adjust: none;

/* One value: <number> or from-font */
font-size-adjust: 0.5;
font-size-adjust: from-font;

/* Two values */
font-size-adjust: ex-height 0.5;
font-size-adjust: ch-width from-font;
```

```
/* Global values */
font-size-adjust: inherit;
font-size-adjust: initial;
font-size-adjust: revert;
font-size-adjust: revert-layer;
font-size-adjust: unset;
```

Values

This property is specified as the keyword `none`, or one (`<number>` or `from-font`) or two (`<font-metric>` and either `<number>` or `from-font`) values.

`none`

No font size adjustment is applied.

`<font-metric>` Optional

Defines the font metric to use for adjusting the font size. Defaulting to `ex-height`, this parameter is specified as one of the following keywords:

`ex-height`

Uses the ratio of x-height (height of lowercase "x" in a font) to font size (aspect value) to adjust the font size. This keyword value is used to normalize lowercase letters across fonts.

`cap-height`

Uses the ratio of cap-height (height of uppercase letters) to font size to adjust the font size. This keyword value is used to normalize uppercase letters across fonts.

`ch-width`

Uses the ratio of the advance width (horizontal space taken up by a character in a font) of the character "0" (ZERO, U+0030) to font size. This keyword value is used to normalize horizontal narrow pitch of fonts.

`ic-width`

Uses the ratio of the advance width of the character "水" (CJK water ideograph, U+6C34) to font size. This keyword value is used to normalize horizontal wide pitch of fonts, particularly those that include CJK (Chinese, Japanese, Korean) characters.

`ic-height`

Uses the ratio of the advance height (vertical space taken up by a character in a font) of the character "水" (CJK water ideograph, U+6C34) to font size. This keyword value is used to normalize vertical wide pitch of fonts, particularly those that include CJK characters.

`<number>`

The proportion to adjust the font size used, depending on the specified `<font-metric>`.

When a `<font-metric>` value is specified, the `<number>` value adjusts the font size as per the chosen `<font-metric>` to maintain a consistent appearance for the specified font metric across different fonts.

The `<number>` value accepts any number from `0` to infinity. `0` yields text of zero height (that is, the text is hidden). Negative values are invalid.

`from-font`

Uses the `<number>` value for the specified `<font-metric>` from the first available font.

Description

The `font-size-adjust` property adjusts the size of a font to match a chosen font metric, such as the height of lowercase letters, to a specified proportion of the `font-size`. The adjustment is applied to each font used to render text, not only to fallback fonts.

Setting a `<number>` adjusts the font size based on the default or stated `<font-metric>`. For example, when set to `ex-height`, the `<number>` value adjusts the selected font size so the height of its `x` glyph is the specified multiple of the font size. This value should generally match the aspect value (ratio of x-height to font size) of the first-choice font. This means that the first-choice font, when available, will display consistently across browsers. Selecting a different value also scales the first-choice font, but in this case, you should prefer to change the `font-size` instead.

> **Note:** If the specified `<font-metric>` has been overridden in `@font-face`, e.g., by using the `size-adjust` descriptor, then the overridden metric will be used in the `font-size-adjust` calculation. This means that when `font-size-adjust` and `size-adjust` are applied together, `size-adjust` does not have any effect.

> **Note:** Font metrics like x-height and cap height can vary between font faces (such as bold or italic variants) within the same `font-family`. When `font-size-adjust` uses `from-font` or a fixed value, the metrics of each font face are adjusted independently, regardless of the relative differences between font faces in the same font family.

The adjusted font size is calculated using the formula `u = ( m / m' ) s`, where:

- `m` is the desired ratio specified by `<number>` or obtained using `from-font`.
- `m'` is the ratio of the chosen metric to the font size in the font being adjusted.
- `s` is the value of the `font-size` property.
- `u` is the new, adjusted font size.

Consider this example to see how the adjusted font size is calculated. A first-choice font has a `font-size` of `12px` (`s`), and the ratio of `cap-height` to font size is `0.20` (`m`). The `cap-height` to font size ratio in the fallback font is `0.15` (`m'`). The `font-size-adjust` value has been specified as `cap-height 0.20`. If the primary font is unavailable, the adjusted font size of the fallback font will be calculated to be `16px` (`(0.20 / 0.15) * 12`). This will ensure that the `cap-height` of the fallback font is similar to that of the first-choice font when displayed.

Formal definition

Initial value	none
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	a number

Formal syntax

```
font-size-adjust =
  none                                          |
  [ ex-height | cap-height | ch-width | ic-width | ic-height ]? [ from-font | <number [0,∞]> ]
```

This syntax reflects the latest standard as per CSS Fonts Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Normalizing font size by lowercase and uppercase letters

This example demonstrates how the `font-size-adjust` property can be used to retain the same aspect value across fonts. The Verdana font has a relatively high aspect value of `0.545`, which means that the lowercase letters are relatively tall compared to uppercase letters. This makes the text in small font sizes appear legible. However, the Times font has a lower aspect value of `0.447`, so the text is less legible at small sizes. If Verdana is the first-choice font and Times is the fallback font, specifying the `font-size-adjust` property can help to retain the same aspect value in Times. So if the font falls back to Times, the text will maintain a similar level of legibility as it would have with Verdana.

Similarly, the cap-height to font size ratio in Verdana is `0.73` and that in Times is `0.66`. When `font-size-adjust` property is applied to Times to adjust its uppercase letters to match the ratio in Verdana, the Times font displays in adjusted font size ((0.73 / 0.66) * 14) `15.48px`.

```
<p class="verdana">
  A: This text uses the Verdana font (14px), which has relatively large
  lowercase letters.
</p>
<p class="times">
  B: This text uses the Times font (14px), which is hard to read in small sizes.
</p>
<p class="times adj-times-ex-height">
  C: This text in 14px Times font is adjusted to the same aspect value as the
  Verdana font, so lowercase letters are normalized across the two fonts.
</p>
<p class="times adj-times-cap-height">
  D: This text in 14px Times font is adjusted to the same cap-height to font
  size ratio as the Verdana font, so uppercase letters are normalized across the
  two fonts.
</p>
```

```
.times {
  font-family: "Times", serif;
  font-size: 14px;
}

.verdana {
  font-family: "Verdana", sans-serif;
  font-size: 14px;
}

.adj-times-ex-height {
  font-size-adjust: 0.545;
}

.adj-times-cap-height {
  font-size-adjust: cap-height 0.73;
}
```

Without `font-size-adjust` in `B`, the switch from Verdana font to Times font could result in a noticeable decrease in legibility due to its lower aspect value. In `C`, notice that only one value is specified for the `font-size-adjust` property, so the default `<font-metric>` value `ex-height` is used. `D` shows how the font would look compared to `A` if its uppercase letter height is adjusted.

Determining the aspect value of a font

For a given font, the same content in two side-by-side `<span>` elements can be used to determine the font's aspect value. If the same font size is used for content in both spans, the spans will match when the `font-size-adjust` value in one span is accurate for the given font.

In the example below, there are three pairs of side-by-side `<span>` elements, each containing the letter "b". The goal is to adjust the `font-size-adjust` property for the right `<span>` in each pair until the borders around the two letters align. The resulting `font-size-adjust` value can be considered the aspect value for the font.

Starting at `0.6` in the first pair and adjusting to `0.5` in the second, we continue adjusting the `font-size-adjust` property value until the borders around the "b" letters align perfectly in the third pair. In this example, the aspect value is determined to be `0.482`.

```
<div>
  <p><span>b</span><span class="adjust1">b</span></p>
  0.6
</div>

<div>
  <p><span>b</span><span class="adjust2">b</span></p>
  0.5
</div>

<div>
  <p><span>b</span><span class="adjust3">b</span></p>
  0.482
</div>
```

```
body {
  display: flex;
}

div {
  text-align: center;
}

p {
  margin: 0 30px 10px 30px;
}
```

```
body {
  display: flex;
}

div {
  text-align: center;
}

p {
```

```
  font-family: "Futura", sans-serif;
  font-size: 50px;
}

span {
  border: solid 1px red;
}

.adjust1 {
  font-size-adjust: 0.6;
}

.adjust2 {
  font-size-adjust: 0.5;
}

.adjust3 {
  font-size-adjust: 0.482;
}
```

See also

- `font-size`
- `font-weight`
- `size-adjust` `@font-face` descriptor
- SVG `font-size-adjust` attribute
- Learn: Fundamental text and font styling

font-smooth CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`font-smooth`** CSS property controls the application of anti-aliasing when fonts are rendered.

Syntax

```
/* Keyword values */
font-smooth: auto;
font-smooth: never;
font-smooth: always;

/* <length> value */
font-smooth: 2em;

/* Global values */
font-smooth: inherit;
font-smooth: initial;
font-smooth: revert;
font-smooth: revert-layer;
font-smooth: unset;
```

> **Note:** WebKit implements a similar property, but with different values: **`-webkit-font-smoothing`**. It only works on macOS.
>
> - `auto` - Let the browser decide (Uses subpixel anti-aliasing when available; this is the default)
> - `none` - Turn font smoothing off; display text with jagged sharp edges.
> - `antialiased` - Smooth the font on the level of the pixel, as opposed to the subpixel. Switching from subpixel rendering to anti-aliasing for light text on dark backgrounds makes it look lighter.
> - `subpixel-antialiased` - On most non-retina displays, this will give the sharpest text.

> **Note:** Firefox implements a similar property, but with different values: **`-moz-osx-font-smoothing`**. It only works on macOS.
>
> - `auto` - Allow the browser to select an optimization for font smoothing, typically `grayscale`.
> - `grayscale` - Render text with grayscale anti-aliasing, as opposed to the subpixel. Switching from subpixel rendering to anti-aliasing for light text on dark backgrounds makes it look lighter.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-smooth =
  auto            |
  never           |
  always          |
  <absolute-size> |
  <length>

<absolute-size> =
  xx-small   |
  x-small    |
  small      |
  medium     |
  large      |
  x-large    |
  xx-large   |
  xxx-large
```

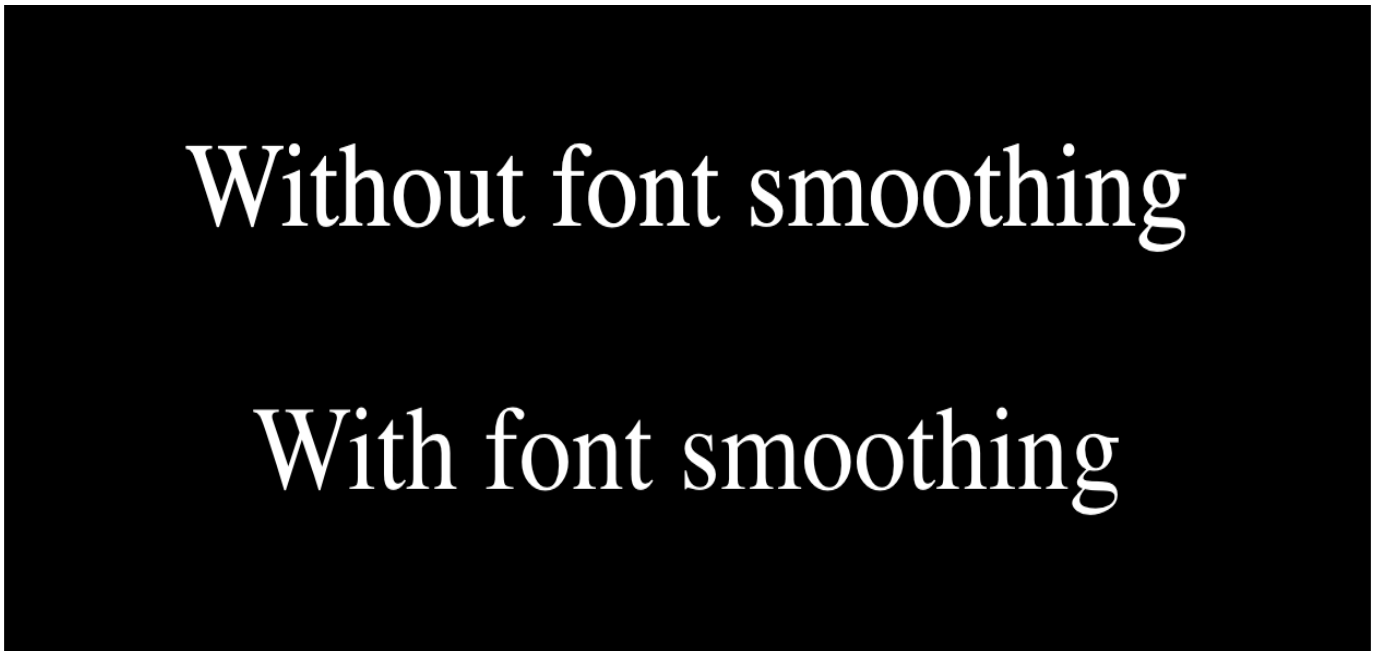
This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage example

The following example shows the Safari/Chromium and Firefox equivalents that turn on font-smoothing on macOS. In both cases the smoothed font should look slightly lighter in weight.

For those of you not on a macOS system, here is a screenshot (the live version appears later on):



HTML

```
<p>Without font smoothing</p>

<p class="smoothed">With font smoothing</p>
```

CSS

```
html {
  background-color: black;
  color: white;
  font-size: 3rem;
}

p {
  text-align: center;
}

.smoothed {
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
```

Result

Specifications

Not part of any standard.

See also

- Please Stop "Fixing" Font Smoothing by Dmitry Fadeyev (2012)
- Laissez-faire font smoothing and anti-aliasing by Zach Leatherman (2017)
- Improve text rendering (A Modern CSS Reset) by Josh W. Comeau (2021)
- What's the deal with WebKit Font Smoothing? by David Bushell (2024)

font-stretch CSS property

> **Note:** The `font-stretch` property was renamed to `font-width` in the CSS Fonts specification. To preserve compatibility, the specification retains `font-stretch` as a legacy alias for the `font-width` property.

The **`font-stretch`** CSS property selects a normal, condensed, or expanded face from a font.

Try it

```
font-stretch: condensed;
```

```
font-stretch: expanded;
```

```
font-stretch: ultra-expanded;
```

```
font-stretch: 50%;
```

```
font-stretch: 100%;
```

```
font-stretch: 150%;
```

```
<section class="default-example" id="default-example">
  <p class="transition-all" id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-family: "League";
  font-style: normal;
  font-weight: normal;
}

section {
  font-size: 1.2em;
  font-family: "League", sans-serif;
}
```

Syntax

```
/* Keyword values */
font-stretch: normal;
font-stretch: ultra-condensed;
font-stretch: extra-condensed;
font-stretch: condensed;
font-stretch: semi-condensed;
font-stretch: semi-expanded;
font-stretch: expanded;
font-stretch: extra-expanded;
font-stretch: ultra-expanded;

/* Percentage values */
font-stretch: 50%;
font-stretch: 100%;
font-stretch: 200%;

/* Global values */
font-stretch: inherit;
font-stretch: initial;
font-stretch: revert;
font-stretch: revert-layer;
font-stretch: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

Specifies a normally condensed font face.

`semi-condensed`, `condensed`, `extra-condensed`, `ultra-condensed`

Specifies a more condensed font face than normal, with `ultra-condensed` being the most condensed.

`semi-expanded`, `expanded`, `extra-expanded`, `ultra-expanded`

Specifies a more expanded font face than normal, with `ultra-expanded` being the most expanded.

`<percentage>`

A `<percentage>` value between 50% and 200% (inclusive). Negative values are not allowed for this property.

Keyword to numeric mapping

The table below shows the mapping between the keyword values and numeric percentages:

Keyword	Percentage
`ultra-condensed`	50%
`extra-condensed`	62.5%
`condensed`	75%
`semi-condensed`	87.5%
`normal`	100%
`semi-expanded`	112.5%
`expanded`	125%
`extra-expanded`	150%
`ultra-expanded`	200%

Description

Some font families offer additional faces in which the characters are narrower than the normal face (*condensed* faces) or wider than the normal face (*expanded* faces).

You can use `font-stretch` to select a condensed or expanded face from such fonts. If the font you are using does not offer condensed or expanded faces, this property has no effect.

Font face selection

The face selected for a given value of `font-stretch` depends on the faces supported by the font in question. If the font does not provide a face that exactly matches the given value, then values less than `100%` map to a condensed face, and values greater than or equal to `100%` map to an expanded face.

The table below demonstrates the effect of setting different percentage values of `font-stretch` on two different fonts:

```css
@font-face {
  font-family: "Inconsolata";
  src: url("https://fonts.gstatic.com/s/inconsolata/v31/QldduNThLqRwH-OJ1UHjlKENVzlm-WkL3GZQmAwPyya15.woff2")
    format("woff2");
}

@font-face {
  font-family: "Anek Malayalam";
  src: url("https://fonts.gstatic.com/s/anekmalayalam/v4/6qLUKZActRTs_mZAJUZWWkhke0nYa-f6__Azq3-gP1W7db9_.woff2")
    format("woff2");
}

body {
  font-family: system-ui;
}

table {
  border-collapse: collapse;
  width: 100%;
}

tbody th {
  text-align: right;
}

th,
td {
  padding: 0.25rem;
  font-weight: normal;
  text-align: center;
}

td {
  border: solid;
  border-width: 1px;
  font-size: 5rem;
}

.inconsolata {
  font-family: "Inconsolata", sans-serif;
}

.anek-malayalam {
  font-family: "Anek Malayalam", sans-serif;
}

td:nth-child(2) {
  font-stretch: 50%;
}
td:nth-child(3) {
  font-stretch: 62.5%;
}
td:nth-child(4) {
  font-stretch: 75%;
}
td:nth-child(5) {
  font-stretch: 87.5%;
}
td:nth-child(6) {
  font-stretch: 100%;
}
td:nth-child(7) {
  font-stretch: 112.5%;
}
td:nth-child(8) {
  font-stretch: 125%;
}
td:nth-child(9) {
  font-stretch: 150%;
}
td:nth-child(10) {
  font-stretch: 200%;
}
```

```html
<table>
  <thead>
    <tr>
      <th scope="row"></th>
      <th scope="col">50%</th>
      <th scope="col">62.5%</th>
      <th scope="col">75%</th>
      <th scope="col">87.5%</th>
      <th scope="col">100%</th>
      <th scope="col">112.5%</th>
      <th scope="col">125%</th>
      <th scope="col">150%</th>
      <th scope="col">200%</th>
    </tr>
  </thead>
  <tbody>
    <tr class="inconsolata">
      <th scope="row">Inconsolata</th>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
    </tr>
```

```
    <tr class="anek-malayalam">
      <th scope="row">Anek Malayalam</th>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
    </tr>
  </tbody>
</table>
```

- Inconsolata is a variable font that offers a continuous range of widths from 50% to 200%.
- Anek Malayalam is a variable google font that supports widths from 75% to 125%. Values below and above this range select the closest matching font.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
font-stretch =
  normal              |
  <percentage [0,∞]>  |
  ultra-condensed     |
  extra-condensed     |
  condensed           |
  semi-condensed      |
  semi-expanded       |
  expanded            |
  extra-expanded      |
  ultra-expanded
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting font stretch percentages

```
<p class="condensed">an elephantine lizard</p>
<p class="normal">an elephantine lizard</p>
<p class="expanded">an elephantine lizard</p>
```

```
@font-face {
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-family: "LeagueMonoVariable";
  font-style: normal;
}

p {
  font:
    1.5rem "LeagueMonoVariable",
    sans-serif;
}

.condensed {
  font-stretch: 50%;
}

.normal {
  font-stretch: 100%;
}

.expanded {
  font-stretch: 200%;
}
```

See also

- `font-stretch` descriptor for `@font-face`
- Modern `font-width` property, replacing `font-stretch`
- `font-style` property
- `font-weight` property
- SVG `font-stretch` attribute
- Learn: Fundamental text and font styling
- CSS fonts module

font-style CSS property

The **`font-style`** CSS property sets whether a font should be styled with a normal, italic, or oblique face from its `font-family`.

Italic font faces are generally cursive in nature, usually using less horizontal space than their unstyled counterparts, while **oblique** faces are usually just sloped versions of the regular face. When the specified style is not available, both italic and oblique faces are simulated by artificially sloping the glyphs of the regular face (use `font-synthesis` to control this behavior).

Try it

```
font-style: normal;
```

```
font-style: italic;
```

```
font-style: oblique;
```

```
font-style: oblique 40deg;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2");
  font-family: "Amstelvar";
  font-style: normal;
}

section {
  font-size: 1.2em;
  font-family: "Amstelvar", serif;
}
```

Syntax

```
font-style: normal;
font-style: italic;
font-style: oblique;
font-style: oblique 10deg;

/* Global values */
font-style: inherit;
font-style: initial;
font-style: revert;
font-style: revert-layer;
font-style: unset;
```

Values

This property is specified as one of the following keyword values. The `oblique` keyword can optionally be followed by an `<angle>`:

`normal`

Selects a font that is classified as `normal` within a `font-family`.

`italic`

Selects a font that is classified as `italic`. If no italic version of the face is available, one classified as `oblique` is used instead. If neither is available, the style is artificially simulated.

`oblique`

Selects a font that is classified as `oblique`. If no oblique version of the face is available, one classified as `italic` is used instead. If neither is available, the style is artificially simulated.

`oblique <angle>`

Selects a font classified as `oblique`, and additionally specifies an angle for the slant of the text. If one or more oblique faces are available in the chosen font family, the one that most closely matches the specified angle is chosen. If no oblique faces are available, the browser will synthesize an oblique version of the font by slanting a normal face by the specified amount. Valid values are degree values of `-90deg` to `90deg` inclusive. If an angle is not specified, an angle of 14 degrees is used. Positive values are slanted to the end of the line, while negative values are slanted towards the beginning.

In general, for a requested angle of 14 degrees or greater, larger angles are preferred; otherwise, smaller angles are preferred (see the spec's font matching section for the precise algorithm).

The CSS fonts module also defines `left` and `right` values to select an italic or oblique face with a specific slant direction; however, these values are not supported in any browsers.

Variable fonts

Variable fonts can offer a fine control over the degree to which an oblique face is slanted. You can select this using the `<angle>` modifier for the `oblique` keyword.

For TrueType or OpenType variable fonts, the `"slnt"` variation is used to implement varying slant angles for oblique, and the `"ital"` variation with a value of 1 is used to implement italic values. See `font-variation-settings`.

Click "Play" in the code blocks below to edit the example in the MDN Playground. Change the angle value to see the slant of the text change.

```
<p class="sample">
  ...it would not be wonderful to meet a Megalosaurus, forty feet long or so,
  waddling like an elephantine lizard up Holborn Hill.
</p>
```

```
@font-face {
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2");
  font-family: "AmstelvarAlpha";
  font-style: normal;
}

.sample {
  font:
    2rem "AmstelvarAlpha",
    sans-serif;
  /* font-variation-settings: "slnt" 12; */
  font-style: oblique 23deg;
}
```

Accessibility

Large sections of text set with a `font-style` value of `italic` may be difficult for people with cognitive concerns such as Dyslexia to read.

- MDN Understanding WCAG, Guideline 1.4 explanations
- W3C Understanding WCAG 2.2

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	by computed value type; `normal` animates as `oblique 0deg`

Formal syntax

```
font-style =
  normal                          |
  italic                          |
  left                            |
  right                           |
  oblique <angle [-90deg,90deg]>?
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Font styles

```
<p class="normal">This paragraph is normal.</p>
<p class="italic">This paragraph is italic.</p>
<p class="oblique">This paragraph is oblique.</p>
```

```
.normal {
  font-style: normal;
}

.italic {
  font-style: italic;
}

.oblique {
  font-style: oblique;
}
```

See also

- `font-family`
- `font-weight`
- SVG `font-style` attribute
- Learn: Fundamental text and font styling

font-synthesis CSS property

The **`font-synthesis`** CSS shorthand property lets you specify whether or not the browser may synthesize the bold, italic, small-caps, and/or subscript and superscript typefaces when they are missing in the specified font-family.

Try it

```
font-synthesis: weight style small-caps;
```

```
font-synthesis: none;
```

```
font-synthesis: weight;
```

```
font-synthesis: style;
```

```
font-synthesis: small-caps;
```

```
font-synthesis: position;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <p class="english">
      This font does not include <span class="bold">bold</span>,
      <span class="italic">italic</span>,
      <span class="small-caps">small-caps</span>, and
      <span class="sub">subscript</span> or
      <span class="sup">superscript</span> variants.
    </p>
    <p class="chinese">
      中文排版通常不用<span class="bold">粗体</span>或<span class="italic"
        >斜体</span
      >，也不会使用<span class="small-caps">小型大写字母</span>和<span
        class="sub"
        >下标</span
      ><span class="sup">上标</span>变体。
    </p>
  </div>
</section>
```

```
@import "https://fonts.googleapis.com/css2?family=Oxygen&display=swap";
@import "https://fonts.googleapis.com/css2?family=Ma+Shan+Zheng&display=swap";

.english {
  font-size: 1.2em;
  font-family: "Oxygen", sans-serif;
}

.chinese {
```

```
  font-size: 1.2em;
  font-family: "Ma Shan Zheng", cursive;
}

.bold {
  font-weight: bold;
}

.italic {
  font-style: italic;
}

.small-caps {
  font-variant: small-caps;
}

.sub {
  font-variant: sub;
}

.sup {
  font-variant: super;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- font-synthesis-weight
- font-synthesis-style
- font-synthesis-small-caps
- font-synthesis-position

Syntax

```
/* none or one or more of the other keyword values */
font-synthesis: none;
font-synthesis: weight;
font-synthesis: style;
font-synthesis: position;
font-synthesis: small-caps style; /* property values can be in any order */
font-synthesis: style small-caps weight position; /* property values can be in any order */

/* Global values */
font-synthesis: inherit;
font-synthesis: initial;
font-synthesis: revert;
font-synthesis: revert-layer;
font-synthesis: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

Indicates that no bold, italic, or small-caps typeface may be synthesized by the browser.

`weight`

Indicates that the missing bold typeface may be synthesized by the browser if needed.

`style`

Indicates that the italic typeface may be synthesized by the browser if needed.

`small-caps`

Indicates that the small-caps typeface may be synthesized by the browser if needed.

`position`

Indicates that the subscript and superscript typeface may be synthesized by the browser, if needed, when using `font-variant-position`.

Description

Most standard Western fonts include italic and bold variants, and some fonts include a small-caps and subscript/superscript variants. However, many fonts do not. Fonts used for Chinese, Japanese, Korean and other logographic scripts tend not to include these variants and synthesizing them might impede the legibility or change the meaning of the text. In these cases, it may be desirable to switch off the browser's default font-synthesis.

For example, using the :lang() pseudo-class, you can disable the browser from synthesizing bold and oblique characters for a language, in this case Arabic:

```
*:lang(ar) {
  font-synthesis: none;
}
```

The table below shows how a value of the shorthand `font-synthesis` property maps to the constituent longhand properties.

font-synthesis value	font-synthesis-weight value	font-synthesis-style value	font-synthesis-small-caps value	font-synthesis-position value
none	none	none	none	none
weight	auto	none	none	none
style	none	auto	none	none
small-caps	none	none	auto	none
position	none	none	none	auto
weight style	auto	auto	none	none
weight small-caps	auto	none	auto	none

font-synthesis value	font-synthesis-weight value	font-synthesis-style value	font-synthesis-small-caps value	font-synthesis-position value
weight position	auto	none	none	auto
style small-caps	none	auto	auto	none
style position	none	auto	none	auto
weight style small-caps	auto	auto	auto	none
weight style position	auto	auto	none	auto
weight small-caps position	auto	none	auto	auto
style small-caps position	none	auto	auto	auto
weight style small-caps position	auto	auto	auto	auto

Formal definition

Initial value	`weight style small-caps position`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-synthesis =
  none                                    |
  [ weight || style || small-caps || position ]
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling font synthesis

This example shows the browser's default font-synthesis behavior and compares it with when the synthesis behavior is turned off. Notice that the example uses two imported fonts to demonstrate this behavior. You might not be able to replicate turning off of font-synthesis on fonts available on your operating system by default.

HTML

```
<pre> DEFAULT </pre>
<p class="english">
  This font supports <strong>bold</strong> and <em>italic</em>.
</p>
<p class="chinese">这个字体支持<strong>加粗</strong>和<em>斜体</em></p>
<br />

<pre> SYNTHESIS IS DISABLED </pre>
<p class="english no-syn">
  This font supports <strong>bold</strong> and <em>italic.</em>
</p>
<p class="chinese no-syn">这个字体支持<strong>加粗</strong>和<em>斜体</em></p>
<br />

<pre> SYNTHESIS IS ENABLED </pre>
<p class="english">
  This font supports <strong>bold</strong> and <em>italic</em>.
</p>
<p class="chinese syn">这个字体支持<strong>加粗</strong>和<em>斜体</em></p>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Montserrat&display=swap";
@import "https://fonts.googleapis.com/css2?family=Ma+Shan+Zheng&display=swap";

.english {
  font-family: "Montserrat", sans-serif;
}
.chinese {
  font-family: "Ma Shan Zheng", cursive;
}
.no-syn {
  font-synthesis: none;
}
.syn {
  font-synthesis: style weight;
}
```

Result

See also

- `font-style`
- `font-weight`
- `font-variant-caps`
- `font-variant-position`

font-synthesis-position CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`font-synthesis-position`** CSS property lets you specify whether or not a browser may synthesize the subscript and superscript "position" typefaces when they are missing in a font family, while using `font-variant-position` to set the positions.

The **`font-synthesis-position`** property has no effect when using the `<sup>` and `<sub>` elements.

It is often convenient to use the shorthand property `font-synthesis` to control all typeface synthesis values.

Syntax

```
/* Keyword values */
font-synthesis-position: auto;
font-synthesis-position: none;

/* Global values */
font-synthesis-position: inherit;
font-synthesis-position: initial;
font-synthesis-position: revert;
font-synthesis-position: revert-layer;
font-synthesis-position: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Indicates that a missing position typeface may be synthesized by the browser if needed.

`none`

Indicates that the synthesis of a missing position typeface by the browser is not allowed.

Formal definition

Initial value	none
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-synthesis-position =
  auto  |
  none
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling synthesis of position typeface

This example shows turning off synthesis of the superscript and subscript typefaces by the browser in the `Montserrat` font.

HTML

```
<p>
  These are the default position <span class="super">superscript</span>,
  position <span class="sub">subscript</span>, <strong>bold</strong> and
  <em>oblique</em> typefaces.
</p>

<p class="no-syn">
  The positions <span class="super">superscript</span> and
  <span class="sub">subscript</span> typeface is turned off here but not the
  <strong>bold</strong> and <em>oblique</em> typefaces (on browsers that support
  <code>font-synthesis-position</code>).
</p>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Montserrat&display=swap";

* {
  font-family: "Montserrat", sans-serif;
}
.super {
  font-variant-position: super;
}
.sub {
  font-variant-position: sub;
}
.no-syn {
  font-synthesis-position: none;
}
```

Result

See also

- `font-synthesis` shorthand, `font-synthesis-style`, `font-synthesis-weight`
- `font-style`, `font-variant`, `font-variant-position`, `font-weight`

font-synthesis-small-caps CSS property

The **`font-synthesis-small-caps`** CSS property lets you specify whether or not the browser may synthesize small-caps typeface when it is missing in a font family. Small-caps glyphs typically use the form of uppercase letters but are reduced to the size of lowercase letters.

It is often convenient to use the shorthand property `font-synthesis` to control all typeface synthesis values.

Syntax

```
/* Keyword values */
font-synthesis-small-caps: auto;
font-synthesis-small-caps: none;

/* Global values */
font-synthesis-small-caps: inherit;
font-synthesis-small-caps: initial;
font-synthesis-small-caps: revert;
font-synthesis-small-caps: revert-layer;
font-synthesis-small-caps: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Indicates that the missing small-caps typeface may be synthesized by the browser if needed.

`none`

Indicates that the synthesis of the missing small-caps typeface by the browser is not allowed.

Formal definition

Initial value	`auto`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-synthesis-small-caps =
  auto  |
  none
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling synthesis of small-caps typeface

This example shows turning off synthesis of the small-caps typeface by the browser in the `Montserrat` font.

HTML

```
<p class="english">
  These are the default <span class="small-caps">small-caps</span>,
  <strong>bold</strong>, and <em>oblique</em> typefaces.
</p>

<p class="english no-syn">
  The <span class="small-caps">small-caps</span> typeface is turned off here but
  not the <strong>bold</strong> and <em>oblique</em> typefaces.
</p>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Montserrat&display=swap";

.english {
  font-family: "Montserrat", sans-serif;
}
.small-caps {
  font-variant: small-caps;
}
.no-syn {
  font-synthesis-small-caps: none;
}
```

Result

See also

- font-synthesis shorthand, font-synthesis-style, font-synthesis-weight
- `font-style`, `font-variant`, `font-variant-caps`, `font-weight`
- CanvasRenderingContext2D: fontVariantCaps property

font-synthesis-style CSS property

The **`font-synthesis-style`** CSS property lets you specify whether or not the browser may synthesize the oblique typeface when it is missing in a font family.

It is often convenient to use the shorthand property `font-synthesis` to control all typeface synthesis values.

Syntax

```
/* Keyword values */
font-synthesis-style: auto;
font-synthesis-style: none;
font-synthesis-style: oblique-only;

/* Global values */
font-synthesis-style: inherit;
font-synthesis-style: initial;
font-synthesis-style: revert;
```

```
font-synthesis-style: revert-layer;
font-synthesis-style: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Indicates that the missing oblique typeface may be synthesized by the browser if needed.

`none`

Indicates that the synthesis of the missing oblique typeface by the browser is *not* allowed.

`oblique-only`

Same as `auto`, but no font synthesis occurs if `font-style: italic` is set.

Formal definition

Initial value	`auto`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-synthesis-style =
  auto          |
  none          |
  oblique-only
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling synthesis of oblique typeface

This example shows turning off synthesis of the oblique typeface by the browser in the `Montserrat` font.

HTML

```
<p class="english">
  This is the default <em>oblique typeface</em> and
  <strong>bold typeface</strong>.
</p>

<p class="english no-syn">
  The <em>oblique typeface</em> is turned off here but not the
  <strong>bold typeface</strong>.
</p>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Montserrat&display=swap";

.english {
  font-family: "Montserrat", sans-serif;
}

.no-syn {
  font-synthesis-style: none;
}
```

Result

Comparison of font-synthesis-style values

This example compares all the `font-synthesis-style` values using italic and oblique styled texts.

HTML

```
<div class="fss-none">
  <h2>font-synthesis-style: none</h2>
  <p class="oblique">This text is set to <code>oblique</code></p>
  <p class="italic">This text is set to <code>italic</code></p>
</div>

<div class="fss-auto">
  <h2>font-synthesis-style: auto</h2>
  <p class="oblique">This text is set to <code>oblique</code></p>
  <p class="italic">This text is set to <code>italic</code></p>
</div>

<div class="fss-oblique-only">
  <h2>font-synthesis-style: oblique-only</h2>
  <p class="oblique">This text is set to <code>oblique</code></p>
  <p class="italic">This text is set to <code>italic</code></p>
</div>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Montserrat&display=swap";

p {
  font-family: "Montserrat", sans-serif;
  font-size: 1.2rem;
```

```
}

@supports not (font-synthesis-style: oblique-only) {
  body::before {
    content: "Your browser doesn't support the 'oblique-only' value.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
/* Specify style of the font synthesis */
.fss-none {
  font-synthesis-style: none;
}

.fss-auto {
  font-synthesis-style: auto;
}

.fss-oblique-only {
  font-synthesis-style: oblique-only;
}

/* Set font styles */
.oblique {
  font-style: oblique;
}

.italic {
  font-style: italic;
}

/* Styles for the demonstration */
.oblique::after {
  content: " (font-style: oblique)";
  font-size: 0.8rem;
  vertical-align: sub;
}

.italic::after {
  content: " (font-style: italic)";
  font-size: 0.8rem;
  vertical-align: sub;
}
```

Result

See also

- font-synthesis shorthand, font-synthesis-small-caps, font-synthesis-weight
- `font-style`, `font-variant`, `font-weight`

font-synthesis-weight CSS property

The **`font-synthesis-weight`** CSS property lets you specify whether or not the browser may synthesize the bold typeface when it is missing in a font family.

It is often convenient to use the shorthand property `font-synthesis` to control all typeface synthesis values.

Syntax

```
/* Keyword values */
font-synthesis-weight: auto;
font-synthesis-weight: none;

/* Global values */
font-synthesis-weight: inherit;
font-synthesis-weight: initial;
font-synthesis-weight: revert;
font-synthesis-weight: revert-layer;
font-synthesis-weight: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Indicates that the missing bold typeface may be synthesized by the browser if needed.

`none`

Indicates that the synthesis of the missing bold typeface by the browser is not allowed.

Formal definition

Initial value	`auto`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-synthesis-weight =
  auto  |
  none
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling synthesis of bold typeface

This example shows turning off synthesis of the bold typeface by the browser in the `Montserrat` font.

HTML

```
<p class="english">
  This is the default <strong>bold typeface</strong> and
  <em>oblique typeface</em>.
</p>

<p class="english no-syn">
  The <strong>bold typeface</strong> is turned off here but not the
  <em>oblique typeface</em>.
</p>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Montserrat&display=swap";

.english {
  font-family: "Montserrat", sans-serif;
}
.no-syn {
  font-synthesis-weight: none;
}
```

Result

See also

- font-synthesis shorthand, font-synthesis-small-caps, font-synthesis-style
- `font-style`, `font-variant`, `font-weight`

font-variant CSS property

The **`font-variant`** CSS shorthand property allows you to set all the font variants for a font.

You can also set the `<font-variant-css2>` values of `font-variant` defined in CSS Level 2.1, (that is, `normal` or `small-caps`), by using the `font` shorthand.

Try it

```
font-variant: normal;
```

```
font-variant: no-common-ligatures proportional-nums;
```

```
font-variant: common-ligatures tabular-nums;
```

```
font-variant: small-caps slashed-zero;
```

```
<section id="default-example">
  <div id="example-element">
    <p>Difficult waffles</p>
    <table>
      <tbody>
        <tr>
          <td><span class="tabular">0O</span></td>
        </tr>
        <tr>
          <td><span class="tabular">3.14</span></td>
        </tr>
        <tr>
          <td><span class="tabular">2.71</span></td>
        </tr>
      </tbody>
    </table>
  </div>
</section>
```

```
@font-face {
  font-family: "Fira Sans";
  src:
    local("FiraSans-Regular"),
    url("/shared-assets/fonts/FiraSans-Regular.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
}

section {
  font-family: "Fira Sans", sans-serif;
  margin-top: 10px;
  font-size: 1.5em;
}

#example-element table {
  margin-left: auto;
  margin-right: auto;
}

.tabular {
  border: 1px solid;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `font-variant-alternates`
- `font-variant-caps`
- `font-variant-east-asian`
- `font-variant-emoji`
- `font-variant-ligatures`
- `font-variant-numeric`
- `font-variant-position`

Syntax

```
font-variant: small-caps;
font-variant: common-ligatures small-caps;

/* Global values */
font-variant: inherit;
font-variant: initial;
font-variant: revert;
font-variant: revert-layer;
font-variant: unset;
```

Values

This property is specified as a space-separated list of the following values:

`normal`

Specifies a normal font face. Each longhand property has an initial value of `normal`.

`none`

Sets the value of the `font-variant-ligatures` as `none` and the values of the other longhand properties as `normal`, their initial value.

`<common-lig-values>`, `<discretionary-lig-values>`, `<historical-lig-values>`, `<contextual-alt-values>`

Specifies the keywords related to the `font-variant-ligatures` longhand property. The possible values are `common-ligatures`, `no-common-ligatures`, `discretionary-ligatures`, `no-discretionary-ligatures`, `historical-ligatures`, `no-historical-ligatures`, `contextual`, and `no-contextual`.

`stylistic()`, `historical-forms`, `styleset()`, `character-variant()`, `swash()`, `ornaments()`, `annotation()`

Specifies the keywords and functions related to the `font-variant-ligatures` longhand property.

`small-caps`, `all-small-caps`, `petite-caps`, `all-petite-caps`, `unicase`, `titling-caps`

Specifies the keywords and functions related to the `font-variant-caps` longhand property. The `small-caps` value is the only non-`normal` font variant valid within the `font` shorthand property.

`<numeric-figure-values>`, `<numeric-spacing-values>`, `<numeric-fraction-values>`, `ordinal`, `slashed-zero`

Specifies the keywords related to the `font-variant-numeric` longhand property. The possible values are `lining-nums`, `oldstyle-nums`, `proportional-nums`, `tabular-nums`, `diagonal-fractions`, `stacked-fractions`, `ordinal`, and `slashed-zero`.

`<east-asian-variant-values>`, `<east-asian-width-values>`, `ruby`

Specifies the keywords related to the `font-variant-east-asian` longhand property. The possible values are `jis78`, `jis83`, `jis90`, `jis04`, `simplified`, `traditional`, `full-width`, `proportional-width`, and `ruby`.

`sub`, `super`

Specifies the keywords and functions related to the `font-variant-position` longhand property.

`text`, `emoji`, `unicode`

Specifies the keywords and functions related to the `font-variant-emoji` longhand property.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant =
  normal                                                |
  none                                                  |
  [ [ <common-lig-values> || <discretionary-lig-values> || <historical-lig-values> || <contextual-alt-values> ] || [ small-caps | all-small-caps | petite-caps | all-petite-caps | unicase | titling-caps ]
|| [ stylistic( <font-feature-value-name> ) || historical-forms || styleset( <font-feature-value-name># ) || character-variant( <font-feature-value-name># ) || swash( <font-feature-value-name> ) ||
ornaments( <font-feature-value-name> ) || annotation( <font-feature-value-name> ) ] || [ <numeric-figure-values> || <numeric-spacing-values> || <numeric-fraction-values> || ordinal || slashed-zero ] || [
<east-asian-variant-values> || <east-asian-width-values> || ruby ] || [ sub | super ] || [ text | emoji | unicode ] ]

<common-lig-values> =
  common-ligatures     |
  no-common-ligatures

<discretionary-lig-values> =
  discretionary-ligatures     |
  no-discretionary-ligatures

<historical-lig-values> =
  historical-ligatures     |
  no-historical-ligatures

<contextual-alt-values> =
  contextual     |
  no-contextual
```

```
<font-feature-value-name> =
  <ident>

<numeric-figure-values> =
  lining-nums    |
  oldstyle-nums

<numeric-spacing-values> =
  proportional-nums  |
  tabular-nums

<numeric-fraction-values> =
  diagonal-fractions  |
  stacked-fractions

<east-asian-variant-values> =
  jis78       |
  jis83       |
  jis90       |
  jis04       |
  simplified  |
  traditional

<east-asian-width-values> =
  full-width          |
  proportional-width
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting the small-caps font variant

HTML

```
<p class="normal">Firefox rocks!</p>
<p class="small">Firefox rocks!</p>
```

CSS

```
p.normal {
  font-variant: normal;
}
p.small {
  font-variant: small-caps;
}
```

Result

See also

- `text-transform`
- `text-combine-upright`
- `text-orientation`
- SVG `font-variant` attribute

font-variant-alternates CSS property

The **`font-variant-alternates`** CSS property controls the usage of alternate glyphs. These alternate glyphs may be referenced by alternative names defined in `@font-feature-values`.

The `@font-feature-values` at-rule can be used to associate, for a given font face, a human-readable name with a numeric index that controls a particular OpenType font feature. For features that select alternative glyphs (`stylistic`, `styleset`, `character-variant`, `swash`, `ornament` or `annotation`), the `font-variant-alternates` property can then reference the human-readable name in order to apply the associated feature.

This allows CSS rules to enable alternative glyphs without needing to know the specific index values that a particular font uses to control them.

Syntax

```
/* Keyword values */
font-variant-alternates: normal;
font-variant-alternates: historical-forms;

/* Functional notation values */
font-variant-alternates: stylistic(user-defined-ident);
font-variant-alternates: styleset(user-defined-ident);
font-variant-alternates: character-variant(user-defined-ident);
font-variant-alternates: swash(user-defined-ident);
font-variant-alternates: ornaments(user-defined-ident);
font-variant-alternates: annotation(user-defined-ident);
font-variant-alternates: swash(ident1) annotation(ident2);

/* Global values */
font-variant-alternates: inherit;
font-variant-alternates: initial;
font-variant-alternates: revert;
font-variant-alternates: revert-layer;
font-variant-alternates: unset;
```

Values

This property is specified as `normal` or as a space-separated list of the following values:

`normal`

> This keyword deactivates alternate glyphs.

`historical-forms`

> This keyword enables historical forms — glyphs that were common in the past but not today. It corresponds to the OpenType value `hist`.

`stylistic()`

This function enables stylistic alternates for individual characters. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `salt`, like `salt 2`.

`styleset()`

This function enables stylistic alternatives for sets of characters. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `ssXY`, like `ss02`.

`character-variant()`

This function enables specific stylistic alternatives for characters. It is similar to `styleset()`, but doesn't create coherent glyphs for a set of characters; individual characters will have independent and not necessarily coherent styles. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `cvXY`, like `cv02`.

`swash()`

This function enables swash glyphs. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType values `swsh` and `cswh`, like `swsh 2` and `cswh 2`.

`ornaments()`

This function enables ornaments, like fleurons and other dingbat glyphs. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `ornm`, like `ornm 2`.

> **Note:** In order to preserve text semantics, font designers should include ornaments that don't match Unicode dingbat characters as ornamental variants of the bullet character (U+2022). Be aware that some existing fonts don't follow this advice.

`annotation()`

This function enables annotations, like circled digits or inverted characters. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `nalt`, like `nalt 2`.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-alternates =
  normal                                                |
  [ stylistic( <font-feature-value-name> ) || historical-forms || styleset( <font-feature-value-name># ) || character-variant( <font-feature-value-name># ) || swash( <font-feature-value-name> ) ||
ornaments( <font-feature-value-name> ) || annotation( <font-feature-value-name> ) ]

<font-feature-value-name> =
  <ident>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Enabling swash glyphs

In this example, we use the `@font-feature-values` at-rule to define a name for the `swash` feature of the MonteCarlo font. The rule maps the name `"fancy"` to the index value `1`.

We can then use that name inside `font-variant-alternates` to switch on swashes for that font. This is the equivalent of a line like `font-feature-settings: "swsh" 1`, except that the CSS applying the feature does not need to include, or even know, the index value needed for this particular font.

HTML

```
<p>A Fancy Swash</p>
<p class="variant">A Fancy Swash</p>
```

CSS

```
@font-face {
  font-family: "MonteCarlo";
  src: url("/shared-assets/fonts/monte-carlo/monte-carlo-regular.woff2");
}

@font-feature-values "MonteCarlo" {
  @swash {
    fancy: 1;
  }
}

p {
  font-family: "MonteCarlo", cursive;
  font-size: 3rem;
  margin: 0.7rem 3rem;
}

.variant {
  font-variant-alternates: swash(fancy);
}
```

Result

See also

- `font-variant`
- `font-variant-caps`
- `font-variant-east-asian`
- `font-variant-emoji`
- `font-variant-ligatures`
- `font-variant-numeric`
- `font-variant-position`
- `@font-feature-values`
- `font-feature-settings`

font-variant-caps CSS property

The **`font-variant-caps`** CSS property controls the use of alternate glyphs used for small or petite capitals or for titling.

Try it

```
font-variant-caps: normal;
```

```
font-variant-caps: small-caps;
```

```
font-variant-caps: all-small-caps;
```

```
<section id="default-example">
  <div id="example-element">
    <p>Difficult waffles</p>
  </div>
</section>
```

```
@font-face {
  font-family: "Fira Sans";
  src:
    local("FiraSans-Regular"),
    url("/shared-assets/fonts/FiraSans-Regular.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
}

section {
  font-family: "Fira Sans", sans-serif;
  margin-top: 10px;
  font-size: 1.5em;
}
```

Syntax

```
/* Keyword values */
font-variant-caps: normal;
font-variant-caps: small-caps;
font-variant-caps: all-small-caps;
font-variant-caps: petite-caps;
font-variant-caps: all-petite-caps;
font-variant-caps: unicase;
font-variant-caps: titling-caps;

/* Global values */
font-variant-caps: inherit;
font-variant-caps: initial;
font-variant-caps: revert;
font-variant-caps: revert-layer;
font-variant-caps: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

Deactivates of the use of alternate glyphs.

`small-caps`

Enables display of small capitals (OpenType feature: `smcp`). Small-caps glyphs typically use the form of uppercase letters but are displayed using the same size as lowercase letters.

`all-small-caps`

Enables display of small capitals for both upper and lowercase letters (OpenType features: `c2sc`, `smcp`).

`petite-caps`

Enables display of petite capitals (OpenType feature: `pcap`).

`all-petite-caps`

Enables display of petite capitals for both upper and lowercase letters (OpenType features: `c2pc`, `pcap`).

`unicase`

Enables display of mixture of small capitals for uppercase letters with normal lowercase letters (OpenType feature: `unic`).

`titling-caps`

Enables display of titling capitals (OpenType feature: `titl`). Uppercase letter glyphs are often designed for use with lowercase letters. When used in all uppercase titling sequences they can appear too strong. Titling capitals are designed specifically for this situation.

Description

The `font-variant-caps` property controls the use of alternate glyphs used for small or petite capitals or for titling.

When a given font includes capital letter glyphs of multiple different sizes, this property selects the most appropriate ones. If petite capital glyphs are not available, they are rendered using small capital glyphs. If these are not present, the browser synthesizes them from the uppercase glyphs.

The value is a single keyword. For each value, if the font doesn't support the OpenType value, then it synthesizes the glyphs. Fonts sometimes include special glyphs for various caseless characters (such as punctuation marks) to better match the capitalized characters around them. However, small capital glyphs are never synthesized for caseless characters.

Language-specific rules

This property accounts for language-specific case mapping rules. For example:

- In Turkic languages, such as Turkish (tr), Azerbaijani (az), Crimean Tatar (crh), Volga Tatar (tt), and Bashkir (ba), there are two kinds of `i` (one with the dot, one without) and two case pairings: `i`/`İ` and `ı`/`I`.
- In German (de), the `ß` may become `ẞ` (U+1E9E) in uppercase.

- In Greek (el), vowels lose their accent when the whole word is in uppercase (ά/Α), except for the disjunctive eta (ή/Η). Also, diphthongs with an accent on the first vowel lose the accent and gain a diacritic on the second vowel (άι/Αΐ).

Accessibility

Large sections of text set with a `font-variant` value of `all-small-caps` or `all-petite-caps` may be difficult for people with cognitive concerns such as Dyslexia to read.

- MDN Understanding WCAG, Guideline 1.4 explanations
- W3C Understanding WCAG 2.2

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-caps =
  normal           |
  small-caps       |
  all-small-caps   |
  petite-caps      |
  all-petite-caps  |
  unicase          |
  titling-caps
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting the small-caps font variant

HTML

```
<p class="small-caps">Firefox rocks, small caps!</p>
<p class="normal">Firefox rocks, normal caps!</p>
```

CSS

```
.small-caps {
  font-variant-caps: small-caps;
  font-style: italic;
}
.normal {
  font-variant-caps: normal;
  font-style: italic;
}
```

Result

See also

- `font-variant`
- `font-variant-alternates`
- `font-variant-east-asian`
- `font-variant-emoji`
- `font-variant-ligatures`
- `font-variant-numeric`
- `font-variant-position`

font-variant-east-asian CSS property

The **`font-variant-east-asian`** CSS property controls the use of alternate glyphs for East Asian scripts, like Japanese and Chinese.

Try it

```
font-variant-east-asian: normal;
```

```
font-variant-east-asian: ruby;
```

```
font-variant-east-asian: jis78;
```

```
font-variant-east-asian: proportional-width;
```

```
<section id="default-example">
  <div id="example-element">
    <p>
      JIS78とJIS83以降では、檜と桧、籠と篭など、一部の文字の入れ替えが行われている。また、「唖然」や「躯体」などの書体が変更されている。
    </p>
  </div>
</section>
```

```
section {
  font-family:
    "YuGothic Medium", "YuGothic", "Yu Gothic Medium", "Yu Gothic", sans-serif;
  margin-top: 10px;
  font-size: 1.5em;
}
```

Syntax

```
font-variant-east-asian: normal;
font-variant-east-asian: ruby;
font-variant-east-asian: jis78; /* <east-asian-variant-values> */
font-variant-east-asian: jis83; /* <east-asian-variant-values> */
font-variant-east-asian: jis90; /* <east-asian-variant-values> */
font-variant-east-asian: jis04; /* <east-asian-variant-values> */
font-variant-east-asian: simplified; /* <east-asian-variant-values> */
font-variant-east-asian: traditional; /* <east-asian-variant-values> */
font-variant-east-asian: full-width; /* <east-asian-width-values> */
font-variant-east-asian: proportional-width; /* <east-asian-width-values> */
font-variant-east-asian: ruby full-width jis83;

/* Global values */
font-variant-east-asian: inherit;
font-variant-east-asian: initial;
font-variant-east-asian: revert;
font-variant-east-asian: revert-layer;
font-variant-east-asian: unset;
```

Values

This property is specified either as `normal` or `ruby` or as a space-separated list of the following values:

`normal`

> This keyword leads to the deactivation of the use of such alternate glyphs.

`ruby`

> This keyword forces the use of special glyphs for ruby characters. As these are usually smaller, font creators often designs specific forms, usually slightly bolder to improve the contrast. This keyword corresponds to the OpenType values `ruby`.

`<east-asian-variant-values>`

> These values specify a set of logographic glyph variants which should be used for display. Possible values are:

Keyword	Standard defining the glyphs	OpenType equivalent
`jis78`	JIS X 0208:1978	`jp78`
`jis83`	JIS X 0208:1983	`jp83`
`jis90`	JIS X 0208:1990	`jp90`
`jis04`	JIS X 0213:2004	`jp04`
`simplified`	None, use the simplified Chinese glyphs	`smpl`
`traditional`	None, use the traditional Chinese glyphs	`trad`

`<east-asian-width-values>`

> These values control the sizing of figures used for East Asian characters. Two values are possible:
>
> - `proportional-width` activating the set of East Asian characters which vary in width. It corresponds to the OpenType values `pwid`.
> - `full-width` activating the set of East Asian characters which are all of the same, roughly square, width metric. It corresponds to the OpenType values `fwid`.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-east-asian =
  normal                                          |
  [ <east-asian-variant-values> || <east-asian-width-values> || ruby ]

<east-asian-variant-values> =
  jis78       |
  jis83       |
  jis90       |
  jis04       |
  simplified  |
  traditional

<east-asian-width-values> =
  full-width          |
  proportional-width
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting East Asian glyph variants

This example require font "Yu Gothic" installed in your OS, other fonts may not support OpenType features.

HTML

```
<table>
  <thead></thead>
  <tbody>
    <tr>
      <th>normal/jis78:</th>
      <td>麹町</td>
```

```
      <td class="jis78">麹町</td>
    </tr>
    <tr>
      <th>normal/ruby:</th>
      <td>しんかんせん</td>
      <td class="ruby">しんかんせん</td>
    </tr>
    <tr>
      <th>normal/traditional:</th>
      <td>大学</td>
      <td class="traditional">大学</td>
    </tr>
  </tbody>
</table>
```

CSS

```
tbody {
  border: 0;
}

td {
  font-family: "Yu Gothic", fantasy;
  font-size: 20px;
}
th {
  color: grey;
  padding-right: 10px;
}

.ruby {
  font-variant-east-asian: ruby;
}

.jis78 {
  font-variant-east-asian: jis78;
}

.traditional {
  font-variant-east-asian: traditional;
}
```

Result

See also

- font-variant
- font-variant-alternates
- font-variant-caps
- font-variant-emoji
- font-variant-ligatures
- font-variant-numeric
- font-variant-position

font-variant-emoji CSS property

The **font-variant-emoji** CSS property specifies the default presentation style for displaying emojis.

Traditionally, this was done by appending a *Variation Selector*, U+FE0E for text and U+FE0F for emoji, to the emoji code point. Only emojis listed as contributing to a Unicode emoji presentation sequence are affected by this property.

Syntax

```
/* Keyword values */
font-variant-emoji: normal;
font-variant-emoji: text;
font-variant-emoji: emoji;
font-variant-emoji: unicode;

/* Global values */
font-variant-emoji: inherit;
font-variant-emoji: initial;
font-variant-emoji: revert;
font-variant-emoji: revert-layer;
font-variant-emoji: unset;
```

Values

This property is specified as one of the following keyword values:

normal

Allows a browser to choose how to display the emoji. This often follows the operating system setting.

text

Renders the emoji as if it were using the unicode text variation selector (U+FE0E).

emoji

Renders the emoji as if it were using the unicode emoji variation selector (U+FE0F).

unicode

Renders the emoji in accordance with the Emoji presentation properties. If the U+FE0E or U+FE0F variation selector is present, then it will override this value setting.

Formal definition

Initial value	normal

Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-emoji =
  normal   |
  text     |
  emoji    |
  unicode
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

While the use of emojis may seem fun, you should consider their impact on accessibility, specifically for users with visual and cognitive impairments. Consider the following factors while using emojis:

- Display on screen-readers: Screen-readers will read out the alt text of an emoji. Keep this in mind to consider the position of an emoji in the content. Repeated and overuse of emojis will have a detrimental effect on screen-reader users. It is better to use emojis than emoticons; emoticons will be read out as punctuation characters.
- Contrast with background: When using emojis, consider their colors and how that will work with the background color, especially if you have background colors that can change, such as light/dark modes.
- Intent of use: Do not use emojis to replace words because your understanding of the emoji meaning may differ from that of the users'. Also consider that emojis might have different meanings in different cultures and geographies. Our recommendation is to preferably limit usage to commonly known emojis.

Examples

Changing the way an emoji is displayed

This example shows how you can render an emoji in its `text` or `emoji` presentation.

HTML

```
<p class="no-support">
  Your Browser does not support <code>font-variant-emoji</code>. This image
  shows how it is rendered with support.
</p>
<img
  class="no-support"
  src="./font-variant-emoji-example.jpg"
  alt="a telephone emoji show as text, black and white next to a telephone emoji shown as emoji full color and graphical representation" />
```

```
<section class="emojis">
  <div class="emoji">
    <h2>text presentation</h2>
    <div class="text-presentation">☎</div>
  </div>
  <div class="emoji">
    <h2>emoji presentation</h2>
    <div class="emoji-presentation">☎</div>
  </div>
</section>
```

CSS

```
@supports (font-variant-emoji: emoji) {
  .no-support {
    display: none;
  }
  .emojis {
    display: flex;
    flex-direction: row;
    justify-content: space-around;
    font-family:
      "Noto Color Emoji", "Segoe UI Emoji", "Apple Color Emoji", sans-serif;
  }
  .emoji > div {
    font-size: 2rem;
  }
}

@supports not (font-variant-emoji: emoji) {
  .emojis {
    display: none;
  }
}
```

```
.text-presentation {
  font-variant-emoji: text;
}

.emoji-presentation {
  font-variant-emoji: emoji;
}
```

Result

See also

- font-variant
- font-variant-alternates
- font-variant-caps
- font-variant-east-asian
- font-variant-ligatures
- font-variant-numeric

- Emojis and accessibility: How to use them properly

font-variant-ligatures CSS property

The **`font-variant-ligatures`** CSS property controls which ligatures and contextual forms are used in the textual content of the elements it applies to. This leads to more harmonized forms in the resulting text.

Try it

```
font-variant-ligatures: normal;
```

```
font-variant-ligatures: no-common-ligatures;
```

```
font-variant-ligatures: common-ligatures;
```

```
<section id="default-example">
  <div id="example-element">
    <p>Difficult waffles</p>
  </div>
</section>
```

```
@font-face {
  font-family: "Fira Sans";
  src:
    local("FiraSans-Regular"),
    url("/shared-assets/fonts/FiraSans-Regular.woff2") format("woff2");
  font-weight: normal;
  font-style: normal;
}

section {
  font-family: "Fira Sans", sans-serif;
  margin-top: 10px;
  font-size: 1.5em;
}
```

Syntax

```
/* Keyword values */
font-variant-ligatures: normal;
font-variant-ligatures: none;
font-variant-ligatures: common-ligatures; /* <common-lig-values> */
font-variant-ligatures: no-common-ligatures; /* <common-lig-values> */
font-variant-ligatures: discretionary-ligatures; /* <discretionary-lig-values> */
font-variant-ligatures: no-discretionary-ligatures; /* <discretionary-lig-values> */
font-variant-ligatures: historical-ligatures; /* <historical-lig-values> */
font-variant-ligatures: no-historical-ligatures; /* <historical-lig-values> */
font-variant-ligatures: contextual; /* <contextual-alt-values> */
font-variant-ligatures: no-contextual; /* <contextual-alt-values> */

/* Two keyword values */
font-variant-ligatures: no-contextual common-ligatures;

/* Four keyword values */
font-variant-ligatures: common-ligatures no-discretionary-ligatures
  historical-ligatures contextual;

/* Global values */
font-variant-ligatures: inherit;
font-variant-ligatures: initial;
font-variant-ligatures: revert;
font-variant-ligatures: revert-layer;
font-variant-ligatures: unset;
```

Values

This property is specified as a single keyword or as a space-separated list of the following values:

`normal`

This keyword activates the usual ligatures and contextual forms needed for correct rendering. The ligatures and forms activated depend on the font, language, and kind of script. This is the default value.

`none`

This keyword specifies that all ligatures and contextual forms are disabled, even common ones.

`<common-lig-values>`

These values control the most common ligatures, like for `fi`, `ffi`, `th`, or similar. They correspond to the OpenType values `liga` and `clig`. Two values are possible:

- `common-ligatures` activating these ligatures. Note that the keyword `normal` activates these ligatures.
- `no-common-ligatures` deactivating these ligatures.

`<discretionary-lig-values>`

These values control specific ligatures, specific to the font and defined by the type designer. They correspond to the OpenType values `dlig`. Two values are possible:

- `discretionary-ligatures` activating these ligatures.
- `no-discretionary-ligatures` deactivating the ligatures. Note that the keyword `normal` usually deactivates these ligatures.

`<historical-lig-values>`

These values control the ligatures used historically, in old books, like the German tz digraph being displayed as ʧ. They correspond to the OpenType values `hlig`. Two values are possible:

- `historical-ligatures` activating these ligatures.
- `no-historical-ligatures` deactivating the ligatures. Note that the keyword `normal` usually deactivates these ligatures.

`<contextual-alt-values>`

These values control whether letters adapt to their context—that is, whether they adapt to the surrounding letters. These values correspond to the OpenType values `calt`. Two values are possible:

- `contextual` specifies that the contextual alternates are to be used. Note that the keyword `normal` usually activates these ligatures too.

- `no-contextual` prevents their use.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-ligatures =
  normal                                          |
  none                                            |
  [ <common-lig-values> || <discretionary-lig-values> || <historical-lig-values> || <contextual-alt-values> ]

<common-lig-values> =
  common-ligatures     |
  no-common-ligatures

<discretionary-lig-values> =
  discretionary-ligatures     |
  no-discretionary-ligatures

<historical-lig-values> =
  historical-ligatures     |
  no-historical-ligatures

<contextual-alt-values> =
  contextual     |
  no-contextual
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting font ligatures and contextual forms

HTML

```
<link href="//fonts.googleapis.com/css?family=Lora" rel="stylesheet" />
<p class="normal">
  normal<br />
  if fi ff tf ft jf fj
</p>
<p class="none">
  none<br />
  if fi ff tf ft jf fj
</p>
<p class="common-ligatures">
  common-ligatures<br />
  if fi ff tf ft jf fj
</p>
<p class="no-common-ligatures">
  no-common-ligatures<br />
  if fi ff tf ft jf fj
</p>
<p class="discretionary-ligatures">
  discretionary-ligatures<br />
  if fi ff tf ft jf fj
</p>
<p class="no-discretionary-ligatures">
  no-discretionary-ligatures<br />
  if fi ff tf ft jf fj
</p>
<p class="historical-ligatures">
  historical-ligatures<br />
  if fi ff tf ft jf fj
</p>
<p class="no-historical-ligatures">
  no-historical-ligatures<br />
  if fi ff tf ft jf fj
</p>
<p class="contextual">
  contextual<br />
  if fi ff tf ft jf fj
</p>
<p class="no-contextual">
  no-contextual<br />
  if fi ff tf ft jf fj
</p>
```

CSS

```
p {
  font-family: "Lora", serif;
}
.normal {
  font-variant-ligatures: normal;
}

.none {
  font-variant-ligatures: none;
}

.common-ligatures {
  font-variant-ligatures: common-ligatures;
}
```

```
.no-common-ligatures {
  font-variant-ligatures: no-common-ligatures;
}

.discretionary-ligatures {
  font-variant-ligatures: discretionary-ligatures;
}

.no-discretionary-ligatures {
  font-variant-ligatures: no-discretionary-ligatures;
}

.historical-ligatures {
  font-variant-ligatures: historical-ligatures;
}

.no-historical-ligatures {
  font-variant-ligatures: no-historical-ligatures;
}

.contextual {
  font-variant-ligatures: contextual;
}

.no-contextual {
  font-variant-ligatures: no-contextual;
}
```

Result

See also

- font-variant
- font-variant-caps
- font-variant-emoji
- font-variant-east-asian
- font-variant-numeric
- font-variant-position
- CSS fonts module

font-variant-numeric CSS property

The **font-variant-numeric** CSS property controls the usage of alternate glyphs for numbers, fractions, and ordinal markers.

Try it

```
font-variant-numeric: normal;
```

```
font-variant-numeric: ordinal;
```

```
font-variant-numeric: slashed-zero;
```

```
font-variant-numeric: tabular-nums;
```

```
font-variant-numeric: oldstyle-nums;
```

```
font-variant-numeric: lining-nums;
```

```
font-variant-numeric: proportional-nums;
```

```
font-variant-numeric: diagonal-fractions;
```

```
<section id="default-example">
  <div id="example-element">
    <table>
      <tbody>
        <tr>
          <td><span class="tabular">0</span></td>
        </tr>
        <tr>
          <td><span class="tabular">3.54</span></td>
        </tr>
        <tr>
          <td><span class="tabular">1.71</span></td>
        </tr>
        <tr>
          <td><span class="tabular">1st</span></td>
        </tr>
        <tr>
          <td><span class="tabular">3/4</span></td>
        </tr>
      </tbody>
    </table>
  </div>
</section>
```

```
@font-face {
  font-family: "Source Sans Pro";
  src:
    local("SourceSansPro-Regular"),
    url("/shared-assets/fonts/SourceSansPro-Regular.otf") format("opentype");
  font-weight: normal;
  font-style: normal;
}

section {
  font-family: "Source Sans Pro", sans-serif;
```

```
  margin-top: 10px;
  font-size: 1.5em;
}

#example-element table {
  margin-left: auto;
  margin-right: auto;
}

.tabular {
  border: 1px solid;
}
```

Syntax

```
font-variant-numeric: normal;
font-variant-numeric: ordinal;
font-variant-numeric: slashed-zero;
font-variant-numeric: lining-nums; /* <numeric-figure-values> */
font-variant-numeric: oldstyle-nums; /* <numeric-figure-values> */
font-variant-numeric: proportional-nums; /* <numeric-spacing-values> */
font-variant-numeric: tabular-nums; /* <numeric-spacing-values> */
font-variant-numeric: diagonal-fractions; /* <numeric-fraction-values> */
font-variant-numeric: stacked-fractions; /* <numeric-fraction-values> */
font-variant-numeric: oldstyle-nums stacked-fractions;

/* Global values */
font-variant-numeric: inherit;
font-variant-numeric: initial;
font-variant-numeric: revert;
font-variant-numeric: revert-layer;
font-variant-numeric: unset;
```

Values

This property is specified either as `normal` or a space-separated list of the following values:

`normal`

This keyword leads to the deactivation of the use of such alternate glyphs.

`ordinal`

This keyword forces the use of special glyphs for the ordinal markers, like 1st, 2nd, 3rd, 4th in English or a 1a in Italian. It corresponds to the OpenType values `ordn`.

`slashed-zero`

This keyword forces the use of a 0 with a slash; this is useful when a clear distinction between O and 0 is needed. It corresponds to the OpenType values `zero`.

`<numeric-figure-values>`

These values control the figures used for numbers. Two values are possible:

- `lining-nums` activating the set of figures where numbers are all lying on the baseline. It corresponds to the OpenType values `lnum`.
- `oldstyle-nums` activating the set of figures where some numbers, like 3, 4, 7, 9 have descenders. It corresponds to the OpenType values `onum`.

`<numeric-spacing-values>`

These values controls the sizing of figures used for numbers. Two values are possible:

- `proportional-nums` activating the set of figures where numbers are not all of the same size. It corresponds to the OpenType values `pnum`.
- `tabular-nums` activating the set of figures where numbers are all of the same size, allowing them to be easily aligned like in tables. It corresponds to the OpenType values `tnum`.

`<numeric-fraction-values>`

These values controls the glyphs used to display fractions. Two values are possible:

- `diagonal-fractions` activating the set of figures where the numerator and denominator are made smaller and separated by a slash. It corresponds to the OpenType values `frac`.
- `stacked-fractions` activating the set of figures where the numerator and denominator are made smaller, stacked and separated by a horizontal line. It corresponds to the OpenType values `afrc`.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-numeric =
  normal                                          |
  [ <numeric-figure-values> || <numeric-spacing-values> || <numeric-fraction-values> || ordinal || slashed-zero ]

<numeric-figure-values> =
  lining-nums    |
  oldstyle-nums

<numeric-spacing-values> =
  proportional-nums  |
  tabular-nums

<numeric-fraction-values> =
  diagonal-fractions  |
  stacked-fractions
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting ordinal numeric forms

Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<p class="ordinal">1st, 2nd, 3rd, 4th, 5th</p>
```

```
@font-face {
  font-family: "Source Sans Pro";
  src: url("https://mdn.github.io/shared-assets/fonts/SourceSansPro-Regular.otf")
    format("opentype");
  font-weight: normal;
  font-style: normal;
}

.ordinal {
  font-family: "Source Sans Pro", sans-serif;
  font-size: 2rem;
  font-variant-numeric: ordinal;
}
```

See also

- font-variant
- font-variant-alternates
- font-variant-caps
- font-variant-east-asian
- font-variant-emoji
- font-variant-ligatures
- font-variant-position

font-variant-position CSS property

The **font-variant-position** CSS property controls the use of alternate, smaller glyphs that are positioned as superscript or subscript.

The glyphs are positioned relative to the baseline of the font, which remains unchanged. These glyphs are typically used in <sub> and <sup> elements.

When the usage of these alternate glyphs is activated, if one character in the run doesn't have such a typographically-enhanced glyph, the whole set of characters of the run is rendered using a fallback method, synthesizing these glyphs.

These alternate glyphs share the same em-box and the same baseline as the rest of the font. They are merely graphically enhanced, and have no effect on the line-height and other box characteristics.

Syntax

```
/* Keyword values */
font-variant-position: normal;
font-variant-position: sub;
font-variant-position: super;

/* Global values */
font-variant-position: inherit;
font-variant-position: initial;
font-variant-position: revert;
font-variant-position: revert-layer;
font-variant-position: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

Deactivates alternate superscript and subscript glyphs.

`sub`

Activates subscript alternate glyphs. If, in a given run, one such glyph is not available for a character, all the characters in the run are rendered using synthesized glyphs.

`super`

Activates superscript alternate glyphs. If, in a given run, one such glyph is not available for a character, all the characters in the run are rendered using synthesized glyphs.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
font-variant-position =
  normal  |
  sub     |
  super
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting superscript and subscript forms

HTML

```
<p class="normal">Normal!</p>
<p class="super">Super!</p>
<p class="sub">Sub!</p>
```

CSS

```
p {
  display: inline;
}

.normal {
  font-variant-position: normal;
}

.super {
  font-variant-position: super;
}

.sub {
  font-variant-position: sub;
}
```

Result

See also

- font-variant
- font-variant-alternates
- font-variant-caps
- font-variant-east-asian
- font-variant-emoji
- font-variant-ligatures
- font-variant-numeric

font-variation-settings CSS property

The **font-variation-settings** CSS property provides low-level control over variable font characteristics by letting you specify the four letter axis names of the characteristics you want to vary along with their values.

Try it

```
font-variation-settings: "wght" 50;
```

```
font-variation-settings: "wght" 850;
```

```
font-variation-settings: "wdth" 25;
```

```
font-variation-settings: "wdth" 75;
```

```
<section id="default-example">
  <p id="example-element">
    ...it would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2");
  font-family: "Amstelvar";
  font-style: normal;
}

p {
  font-size: 1.5rem;
  font-family: "Amstelvar", serif;
}
```

Syntax

```
/* Use the default settings */
font-variation-settings: normal;

/* Set values for variable font axis names */
font-variation-settings: "xhgt" 0.7;

/* Global values */
font-variation-settings: inherit;
font-variation-settings: initial;
font-variation-settings: revert;
font-variation-settings: revert-layer;
font-variation-settings: unset;
```

Values

This property is specified as `normal` or a comma-separated list of string-number value pairs:

`normal`

Text is laid out using default settings.

`<string> <number>`

When rendering text, the list of variable font axis names is passed to the text layout engine to enable or disable font features. Each setting is always one or more pairs consisting of a `<string>` of 4 ASCII characters followed by a `<number>` indicating the axis value to set. If the `<string>` has more or fewer characters or contains characters outside the U+20 - U+7E code point range, the whole property is invalid. The `<number>` can be fractional or negative, depending on the value range available in your font, as defined by the font designer.

Description

This property is a low-level mechanism designed to set variable font features where no other way to enable or access those features exist. You should only use it when no basic properties exist to set those features (e.g., `font-weight`, `font-style`).

Font characteristics set using `font-variation-settings` will always override those set using the corresponding basic font properties, e.g., `font-weight`, no matter where they appear in the cascade. In some browsers, this is currently only true when the `@font-face` declaration includes a `font-weight` range.

Registered and custom axes

Variable font axes come in two types: **registered** and **custom**.

Registered axes are the most commonly encountered — common enough that the authors of the specification felt they were worth standardizing. Note that this doesn't mean that the author has to include all of these in their font.

Here are the registered axes along with their corresponding CSS properties:

Axis Tag	CSS Property
"wght"	`font-weight`
"wdth"	`font-stretch`
"slnt" (slant)	`font-style`: `oblique + angle`
"ital"	`font-style`: `italic`
"opsz"	`font-optical-sizing`

Custom axes can be anything the font designer wants to vary in their font, for example ascender or descender heights, the size of serifs, or anything else they can imagine. Any axis can be used as long as it is given a unique 4-character axis. Some will end up becoming more common, and may even become registered over time.

> **Note:** Registered axis tags are identified using lower-case tags, whereas custom axes should be given upper-case tags. Note that font designers aren't forced to follow this practice in any way, and some won't. The important takeaway here is that axis tags are case-sensitive.

To use variable fonts on your operating system, you need to make sure that it is up to date. For example Linux OSes need the latest Linux FreeType version, and macOS before 10.13 does not support variable fonts. If your operating system is not up to date, you will not be able to use variable fonts in web pages or the Firefox Developer Tools.

Formal definition

Initial value	`normal`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	a transform

Formal syntax

```
font-variation-settings =
  normal                        |
  [ <opentype-tag> <number> ]#

<opentype-tag> =
  <string>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find a number of other variable font examples in our Variable fonts guide.

Controlling variable font weight (wght)

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS to play with different font weight values. See what happens when you specify a value outside the weight range.

```
<div>
  <p class="p1">Weight</p>
  <span>(font-weight: 625)</span>
</div>
<div>
  <p class="p2">Weight</p>
  <span>(font-variation-settings: "wght" 625)</span>
</div>
<div class="adjustable">
  <p class="p3">Weight</p>
  (font-variation-settings: "wght" <span id="angle-text">625</span>)<br />
  <label for="text-axis">Adjust Weight: </label>
  <input
    type="range"
    id="text-axis"
    name="text-axis"
    min="300"
    max="900"
    value="625" />
</div>
```

```
@font-face {
  font-family: "Amstelvar VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2")
    format("woff2-variations");
  font-weight: 300 900;
  font-stretch: 35% 100%;
  font-style: normal;
```

```
  font-display: swap;
}

p {
  font:
    1.2em "Amstelvar VF",
    "Georgia",
    serif;
  font-size: 4rem;
  margin: 1rem;
  display: inline-block;
}

.adjustable {
  border: 1px dashed;
  --text-axis: 625;
}
```

```
/* weight range is 300 to 900 */
.p1 {
  font-weight: 625;
}

/* weight range is 300 to 900 */
.p2 {
  font-variation-settings: "wght" 625;
}

/* Adjust with slider & custom property */
.p3 {
  font-variation-settings: "wght" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

Controlling variable font slant (slnt)

Click "Play" in the code blocks below to edit the example in the MDN Playground. Edit the CSS to play with different font slant/oblique values.

```
<div>
  <p class="p1">Slant</p>
  <span>(font-style: oblique 14deg)</span>
</div>
<div>
  <p class="p2">Slant</p>
  <span>(font-variation-settings: 'slnt' 12)</span>
</div>
<div class="adjustable">
  <p class="p3">Slant</p>
  (font-variation-settings: 'slnt' <span id="angle-text">5</span>)<br />
  <label for="text-axis">Adjust Weight: </label>
  <input
    type="range"
    id="text-axis"
    name="text-axis"
    min="0"
    max="12"
    value="5" />
</div>
```

```
@font-face {
  font-family: "Roboto VF";
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/Roboto-VF.woff2")
    format("woff2-variations");
  font-weight: 100 900;
  font-stretch: 75% 100%;
  font-style: oblique 0deg 12deg;
  font-display: swap;
}

p {
  font:
    1.2em "Roboto VF",
    "Helvetica",
    sans-serif;
  font-size: 4rem;
  margin: 1rem;
  display: inline-block;
}

.adjustable {
  border: 1px dashed;
  --text-axis: 5;
}
```

```
/* slant range is from 0deg to 12deg */
.p1 {
  font-style: oblique 14deg;
}

/* slant range is from 0 to 12 */
.p2 {
  font-variation-settings: "slnt" 12;
```

```
}

/* Adjust with slider & custom property */
.p3 {
  font-variation-settings: "slnt" var(--text-axis);
}
```

```
const angle = document.querySelector("#text-axis");
const text = document.querySelector("#angle-text");
const adjustable = document.querySelector(".adjustable");

angle.addEventListener("input", (e) => {
  const angle = e.target.value;
  text.innerText = angle;
  adjustable.style.setProperty("--text-axis", angle);
});
```

See also

- Variable fonts guide
- OpenType font variations overview on microsoft.com
- OpenType design-variation axis tag registry on microsoft.com
- OpenType variable fonts on axis-praxis.org
- Variable fonts on v-fonts.com

font-weight CSS property

The **`font-weight`** CSS property sets the weight (or boldness) of the font. The weights available depend on the `font-family` that is currently set.

Try it

```
font-weight: normal;
```

```
font-weight: bold;
```

```
font-weight: lighter;
```

```
font-weight: bolder;
```

```
font-weight: 100;
```

```
font-weight: 900;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
section {
  font-size: 1.2em;
}
```

Syntax

```
/* <font-weight-absolute> keyword values */
font-weight: normal;
font-weight: bold;

/* <font-weight-absolute> numeric values [1,1000] */
font-weight: 100;
font-weight: 200;
font-weight: 300;
font-weight: 400; /* normal */
font-weight: 500;
font-weight: 600;
font-weight: 700; /* bold */
font-weight: 800;
font-weight: 900;

/* Keyword values relative to the parent */
font-weight: lighter;
font-weight: bolder;

/* Global values */
font-weight: inherit;
font-weight: initial;
font-weight: revert;
font-weight: revert-layer;
font-weight: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

Normal font weight. Same as `400`.

`bold`

Bold font weight. Same as `700`.

`<number>`

A `<number>` value between 1 and 1000, both values included. Higher numbers represent weights that are bolder than (or as bold as) lower numbers. This allows fine-grain control for variable fonts. For non-variable fonts, if the exact specified weight is unavailable, a fallback weight algorithm is used — numeric values that are divisible by 100 correspond to common weight names, as described in the Common weight name mapping section below.

`lighter`

One relative font weight lighter than the parent element. Note that only four font weights are considered for relative weight calculation; see the Meaning of relative weights section below.

`bolder`

One relative font weight heavier than the parent element. Note that only four font weights are considered for relative weight calculation; see the Meaning of relative weights section below.

Fallback weights

If the exact weight given is unavailable, then the following rule is used to determine the weight actually rendered:

- If the target weight given is between `400` and `500` inclusive:
  - Look for available weights between the target and `500`, in ascending order.
  - If no match is found, look for available weights less than the target, in descending order.
  - If no match is found, look for available weights greater than `500`, in ascending order.
- If a weight less than `400` is given, look for available weights less than the target, in descending order. If no match is found, look for available weights greater than the target, in ascending order.
- If a weight greater than `500` is given, look for available weights greater than the target, in ascending order. If no match is found, look for available weights less than the target, in descending order.

> **Note:** The fallback weight algorithm is only used for rendering. The computed value of the property is still the specified value.

Meaning of relative weights

When `lighter` or `bolder` is specified, the below chart shows how the absolute font weight of the element is determined.

Note that when using relative weights, only four font weights are considered — thin (100), normal (400), bold (700), and heavy (900). If a font family has more weights available, they are ignored for the purposes of relative weight calculation.

Inherited value	`bolder`	`lighter`
100	400	100
200	400	100
300	400	100
400	700	100
500	700	100
600	900	400
700	900	400
800	900	700
900	900	700

Common weight name mapping

The numerical values `100` to `900` roughly correspond to the following common weight names (see the OpenType specification):

Value	Common weight name
100	Thin (Hairline)
200	Extra Light (Ultra Light)
300	Light
400	Normal (Regular)
500	Medium
600	Semi Bold (Demi Bold)
700	Bold
800	Extra Bold (Ultra Bold)
900	Black (Heavy)
950	Extra Black (Ultra Black)

Variable fonts

While many fonts have a particular weight corresponding to one of the numbers in Common weight name mapping, most variable fonts support a range of weights providing much finer granularity, giving designers and developers more control over the chosen weight.

For TrueType or OpenType variable fonts, the "wght" variation is used to implement varying widths.

This demo loads with `font-weight: 500;` set. Change the value of the `font-weight` property in the `.sample` selector to see the weight of the text change (e.g., 200, 700). Click "Play" in the code blocks below to edit the example in the MDN Playground:

```
<p class="sample">
  ...it would not be wonderful to meet a Megalosaurus, forty feet long or so,
  waddling like an elephantine lizard up Holborn Hill.
</p>
```

```
@font-face {
  src: url("https://mdn.github.io/shared-assets/fonts/variable-fonts/MutatorSans.ttf");
  font-family: "MutatorSans";
  font-style: normal;
  font-weight: 1 1000;
}
```

```
.sample {
  text-transform: uppercase;
  font-weight: 500;
  font-size: 1.5rem;
  font-family: "MutatorSans", sans-serif;
}
```

Accessibility

People experiencing low vision conditions may have difficulty reading text set with a `font-weight` value of `100` (Thin/Hairline) or `200` (Extra Light), especially if the font has a low contrast color ratio.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	the keyword or the numerical value as specified, with `bolder` and `lighter` transformed to the real value
Animation type	by computed value type

Formal syntax

```
font-weight =
  <font-weight-absolute>  |
  bolder                  |
  lighter

<font-weight-absolute> =
  normal             |
  bold               |
  <number [1,1000]>
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting font weights

HTML

```
<p>
  Alice was beginning to get very tired of sitting by her sister on the bank,
  and of having nothing to do: once or twice she had peeped into the book her
  sister was reading, but it had no pictures or conversations in it, "and what
  is the use of a book," thought Alice "without pictures or conversations?"
</p>

<div>
  I'm heavy<br />
  <span>I'm lighter</span>
</div>
```

CSS

```
/* Set paragraph text to be bold. */
p {
  font-weight: bold;
}

/* Set div text to two steps heavier than
   normal but less than a standard bold. */
div {
  font-weight: 600;
}

/* Set span text to be one step lighter
   than its parent. */
span {
  font-weight: lighter;
}
```

Result

See also

- `font-family`
- `font-style`
- SVG `font-weight` attribute
- Learn: Fundamental text and font styling
- CSS fonts module

font-width CSS property

> **Note:** The `font-width` property is the modern replacement for `font-stretch`, which is a legacy alias. While `font-width` is the specification's preferred name, `font-stretch` currently has broader browser support. Check the fallback example and browser compatibility table for details.

The **`font-width`** CSS property selects a normal, condensed, or expanded face from a font.

Try it

```
font-width: condensed;
```

```
font-width: expanded;
```

```
font-width: ultra-expanded;
```

```
font-width: 50%;
```

```
font-width: 100%;
```

```
font-width: 150%;
```

```
<section class="default-example" id="default-example">
  <p class="transition-all" id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-family: "League";
  font-style: normal;
  font-weight: normal;
}

section {
  font-size: 1.2em;
  font-family: "League", sans-serif;
}
```

Syntax

```
/* Keyword values */
font-width: normal;
font-width: ultra-condensed;
font-width: extra-condensed;
font-width: condensed;
font-width: semi-condensed;
font-width: semi-expanded;
font-width: expanded;
font-width: extra-expanded;
font-width: ultra-expanded;

/* Percentage values */
font-width: 50%;
font-width: 100%;
font-width: 200%;

/* Global values */
font-width: inherit;
font-width: initial;
font-width: revert;
font-width: revert-layer;
font-width: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

Specifies a normally condensed font face.

`semi-condensed`, `condensed`, `extra-condensed`, `ultra-condensed`

Specifies a more condensed font face than normal, with `ultra-condensed` being the most condensed.

`semi-expanded`, `expanded`, `extra-expanded`, `ultra-expanded`

Specifies a more expanded font face than normal, with `ultra-expanded` being the most expanded.

`<percentage>`

A `<percentage>` value between 50% and 200% (inclusive). Negative values are not allowed for this property.

Keyword to numeric mapping

The table below shows the mapping between the keyword values and numeric percentages:

Keyword	Percentage
`ultra-condensed`	50%
`extra-condensed`	62.5%
`condensed`	75%
`semi-condensed`	87.5%
`normal`	100%
`semi-expanded`	112.5%
`expanded`	125%
`extra-expanded`	150%
`ultra-expanded`	200%

Description

Some font families offer additional faces in which the characters are narrower than the normal face (*condensed* faces) or wider than the normal face (*expanded* faces).

You can use `font-width` to select a condensed or expanded face from such fonts. If the font you are using does not offer condensed or expanded faces, this property has no effect.

Font face selection

The face selected for a given value of `font-width` depends on the faces supported by the font in question. If the font does not provide a face that exactly matches the given value, then values less than 100% map to a condensed face, and values greater than or equal to 100% map to an expanded face.

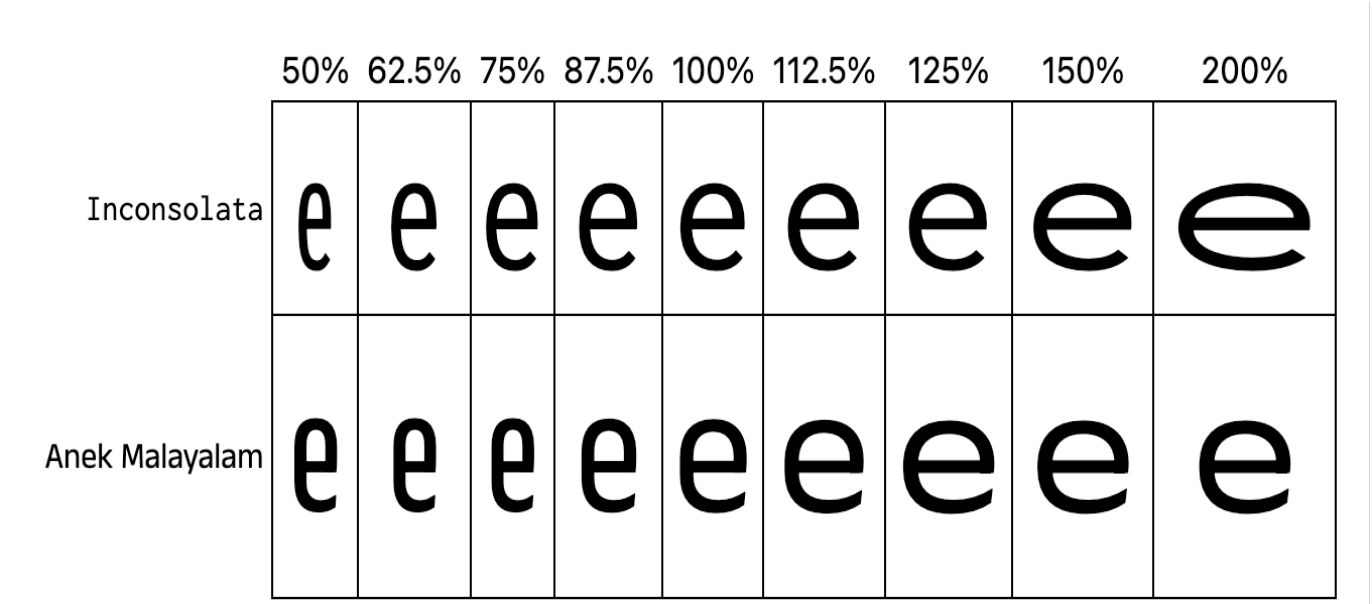
The table below demonstrates the effect of setting different percentage values of `font-width` on two different fonts:

```
@font-face {
  font-family: "Inconsolata";
  src: url("https://fonts.gstatic.com/s/inconsolata/v31/QldduNThLqRwH-OJ1UHjlKENVzlm-WkL3GZQmAwPyya15.woff2")
    format("woff2");
}

@font-face {
  font-family: "Anek Malayalam";
  src: url("https://fonts.gstatic.com/s/anekmalayalam/v4/6qLUKZActRTs_mZAJUZWWkhke0nYa-f6__Azq3-gP1W7db9_.woff2")
    format("woff2");
}

body {
  font-family: system-ui;
}

table {
  border-collapse: collapse;
  width: 100%;
}

tbody th {
  text-align: right;
}

th,
td {
  padding: 0.25rem;
  font-weight: normal;
  text-align: center;
}

td {
  border: solid;
  border-width: 1px;
  font-size: 5rem;
}

.inconsolata {
  font-family: "Inconsolata", sans-serif;
}

.anek-malayalam {
  font-family: "Anek Malayalam", sans-serif;
}

td:nth-child(2) {
  font-width: 50%;
}
td:nth-child(3) {
  font-width: 62.5%;
}
td:nth-child(4) {
  font-width: 75%;
}
td:nth-child(5) {
  font-width: 87.5%;
}
td:nth-child(6) {
  font-width: 100%;
}
td:nth-child(7) {
  font-width: 112.5%;
}
td:nth-child(8) {
  font-width: 125%;
}
td:nth-child(9) {
  font-width: 150%;
}
td:nth-child(10) {
  font-width: 200%;
}
```

```
<table>
  <thead>
    <tr>
      <th scope="row"></th>
      <th scope="col">50%</th>
      <th scope="col">62.5%</th>
      <th scope="col">75%</th>
      <th scope="col">87.5%</th>
      <th scope="col">100%</th>
      <th scope="col">112.5%</th>
      <th scope="col">125%</th>
      <th scope="col">150%</th>
      <th scope="col">200%</th>
    </tr>
  </thead>
  <tbody>
    <tr class="inconsolata">
      <th scope="row">Inconsolata</th>
      <td>e</td>
```

```
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
    </tr>
    <tr class="anek-malayalam">
      <th scope="row">Anek Malayalam</th>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
      <td>e</td>
    </tr>
  </tbody>
</table>
```

The following screenshot shows how the above table is rendered, in case your browser doesn't support the `font-width` property:



- Inconsolata is a variable font that offers a continuous range of widths from 50% to 200%.
- Anek Malayalam is a variable google font that supports widths from 75% to 125%. Values below and above this range select the closest matching font.

Formal definition

Initial value	`normal`
Applies to	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	percentage
Animation type	by computed value type

Formal syntax

```
font-width =
  normal             |
  <percentage [0,∞]> |
  ultra-condensed    |
  extra-condensed    |
  condensed          |
  semi-condensed     |
  semi-expanded      |
  expanded           |
  extra-expanded     |
  ultra-expanded
```

This syntax reflects the latest standard as per CSS Fonts Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting font width percentages

```
<p class="condensed">an elephantine lizard</p>
<p class="normal">an elephantine lizard</p>
<p class="expanded">an elephantine lizard</p>
```

```
@font-face {
  src: url("/shared-assets/fonts/LeagueMono-VF.ttf") format("truetype");
  font-family: "LeagueMonoVariable";
  font-style: normal;
```

```
}

p {
  font:
    1.5rem "LeagueMonoVariable",
    sans-serif;
}

.condensed {
  font-width: 50%;
}

.normal {
  font-width: 100%;
}

.expanded {
  font-width: 200%;
}
```

Providing a font-stretch fallback

Because `font-width` does not yet have broad browser support, you may want to include the legacy `font-stretch` property as a fallback. Place `font-stretch` before `font-width` so that supporting browsers use the modern property:

```
p {
  font-stretch: condensed; /* for browsers that don't support font-width */
  font-width: condensed;
}
```

> **Note:** You can automate this fallback pattern using the postcss-preset-env plugin for PostCSS, which includes the postcss-font-width-property transform to convert `font-width` properties to `font-stretch` automatically.

See also

- `font-width` descriptor for `@font-face`
- Legacy `font-stretch` alias property with better browser support
- `font-style` property
- `font-weight` property
- SVG `font-width` attribute
- SVG `font-stretch` attribute
- Learn: Fundamental text and font styling
- CSS fonts module

forced-color-adjust CSS property

The **`forced-color-adjust`** CSS property allows authors to opt certain elements out of forced colors mode. This then restores the control of those values to CSS.

Syntax

```
forced-color-adjust: auto;
forced-color-adjust: none;
forced-color-adjust: preserve-parent-color;

/* Global values */
forced-color-adjust: inherit;
forced-color-adjust: initial;
forced-color-adjust: revert;
forced-color-adjust: revert-layer;
forced-color-adjust: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

> The element's colors are adjusted by the user agent in forced colors mode. This is the default value.

`none`

> The element's colors are not automatically adjusted by the user agent in forced colors mode.

`preserve-parent-color`

> In forced colors mode, if the `color` property inherits from its parent (i.e., there is no cascaded value or the cascaded value is `currentColor`, `inherit`, or another keyword that inherits from the parent), then it computes to the used color of its parent's `color` property. In all other cases, it behaves the same as `none`.

Usage notes

This property should only be used to makes changes that will support a user's color and contrast requirements. For example, if you become aware that the color optimizations made by the user agent result in a poor experience when in a high contrast or dark mode. Using this property would enable tweaking of the result in that mode to provide a better experience. **It should not be used to prevent user choices being respected**.

Formal definition

Initial value	`auto`
Applies to	all elements and text
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
forced-color-adjust =
  auto                  |
  none                  |
  preserve-parent-color
```

This syntax reflects the latest standard as per CSS Color Adjustment Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preserving colors

In the example below the first box will use the color scheme that the user has set. For example in Windows High Contrast mode black scheme it will have a black background and white text. The second box will preserve the site colors set on the `.box` class.

By using the `forced-colors` media feature, you could add any other optimizations for forced color mode alongside the `forced-color-adjust` property.

CSS

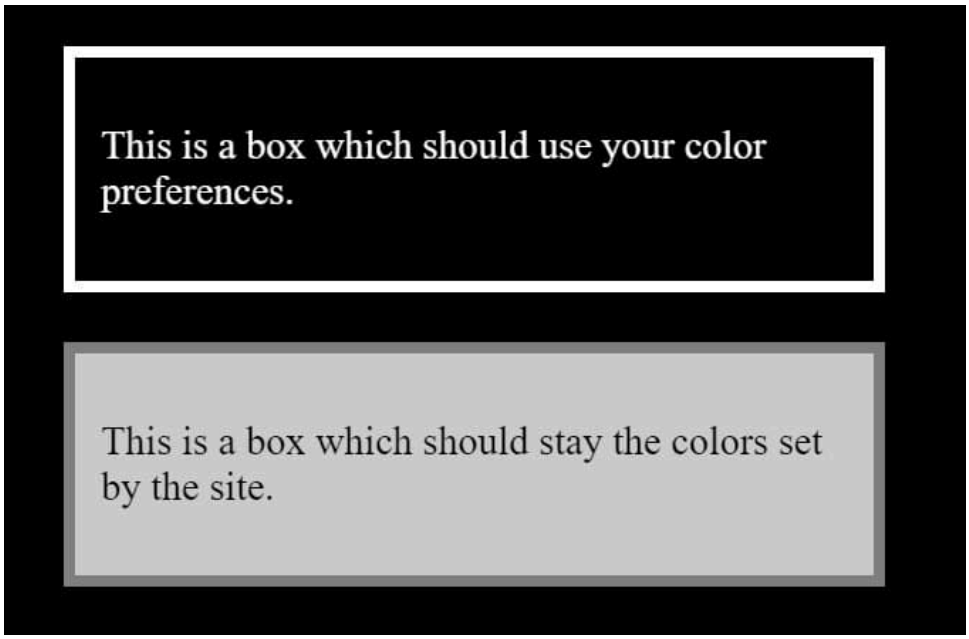
```
.box {
  border: 5px solid grey;
  background-color: #cccccc;
  width: 300px;
  margin: 20px;
  padding: 10px;
}

@media (forced-colors: active) {
  .forced {
    forced-color-adjust: none;
  }
}
```

HTML

```
<div class="box">
  <p>This is a box which should use your color preferences.</p>
</div>

<div class="box forced">
  <p>This is a box which should stay the colors set by the site.</p>
</div>
```

Result

The following screenshot shows the image above in Windows High Contrast Mode:



See also

- Styling for Windows high contrast with standards for forced colors.
- `print-color-adjust`

frame-sizing CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`frame-sizing`** CSS property can be used to set an `<iframe>` element's horizontal or vertical size to equal the layout size of its embedded document in the same dimension, but only if the embedded document has opted in to sharing its size information.

Syntax

```
/* Keyword values */
frame-sizing: auto;
frame-sizing: content-width;
frame-sizing: content-height;
```

```
frame-sizing: content-inline-size;
frame-sizing: content-block-size;

/* Global values */
frame-sizing: inherit;
frame-sizing: initial;
frame-sizing: revert;
frame-sizing: revert-layer;
frame-sizing: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The initial value. The `<iframe>` element's size is not affected by the layout size of its embedded document.

`content-width`

The `<iframe>` element's `width` is set to the layout width of its embedded document.

`content-height`

The `<iframe>` element's `height` is set to the layout height of its embedded document.

`content-inline-size`

The `<iframe>` element's `inline-size` is set to the layout size of its embedded document in the inline direction.

`content-block-size`

The `<iframe>` element's `block-size` is set to the layout size of its embedded document in the block direction.

Description

For security and privacy reasons, `<iframe>` elements do not by default expose any information to the parent document about the size of the content in the document they are embedding.

To enable responsive sizing of `<iframe>` elements based on their content, the `<meta name="responsive-embedded-sizing">` tag can be included in an embedded document to opt it in to sharing its size information with the parent document. The `frame-sizing` property can then be set on the `<iframe>` to cause it to adopt the same horizontal or vertical size as the embedded document's actual content size (termed the **internal layout intrinsic size** in the spec, but abbreviated to "layout size" in our documentation). The document content will then fit seamlessly into the embedding `<iframe>`, avoiding unnecessary scrollbars.

The `frame-sizing` property can take values of `content-width` or `content-height` to cause the `<iframe>` element's `width` or `height` to adopt the embedded document's layout width or layout height, respectively.

There are also logical equivalents available — `frame-sizing` can take values of `content-inline-size` or `content-block-size` to cause the `<iframe>` element's `inline-size` or `block-size` to adopt the embedded document's inline size or block size, respectively. The block or inline direction is determined by the `<iframe>` element's `writing-mode`, not that of the embedded document's content.

To resize the `<iframe>` dynamically as the embedded document changes layout size, you can call the `Window.requestResize()` method from the embedded document to make it report an updated size.

Formal definition

Initial value	`auto`
Applies to	replaced elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
frame-sizing =
  auto                |
  content-width       |
  content-height      |
  content-block-size  |
  content-inline-size
```

This syntax reflects the latest standard as per CSS Box Sizing Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates usage of the `frame-sizing` property.

We have two documents, the main `index.html` document, and the embedded `frame.html` document.

The main `index.html`

The `index.html` document's HTML contains a heading and an `<iframe>`, into which is embedded the `frame.html` document:

```
<h1>Responsive iframes — basic example</h1>

<iframe src="frame.html"></iframe>
```

In the `index.html` CSS, we give the `<iframe>` a `frame-sizing` value of `content-block-size`. Because the `<iframe>` has a horizontal `writing-mode`, its `height` will be set to the embedded document's layout height.

```
iframe {
  frame-sizing: content-block-size;
  border: 2px solid gray;
}
```

The embedded `frame.html`

The `frame.html` document contains a heading and some paragraphs. More significantly, however, it includes the `<meta name="responsive-embedded-sizing" />` tag, which opts it in to sharing its content layout size with the parent document.

```
<head>
  ...
```

```
  <meta name="responsive-embedded-sizing" />

  ...
</head>
<body>
  <h1>This is my frame</h1>
  <p>This is the content of my discontent.</p>
  <p>This is some more content.</p>
</body>
```

Result

Open our basic responsive `<iframe>` sizing demo in a separate tab to see it in action (see the source code).

Even though no explicit `height` has been set on the `<iframe>`, it is sized to the right height to exactly contain its embedded document, with no scroll bars.

See also

- CSS box sizing module
- `<meta name="responsive-embedded-sizing">`
- `Window.requestResize()`

gap CSS property

The **gap** CSS shorthand property sets the gaps (also called gutters) between rows and columns on multi-column, flex, and grid containers.

Try it

```
gap: 0;
```

```
gap: 10%;
```

```
gap: 1em;
```

```
gap: 10px 20px;
```

```
gap: calc(20px + 10%);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  width: 200px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `row-gap`
- `column-gap`

Syntax

```
/* One value */
gap: normal;
gap: 20px;
gap: 1em;
gap: 3vmin;
gap: 0.5cm;
gap: 16%;
gap: 100%;
gap: thick;
gap: calc(10% + 20px);

/* Two values */
gap: 20px 10px;
gap: 1em 0.5em;
gap: 3vmin 2vmax;
gap: 0.5cm 2mm;
gap: 16% 100%;
gap: 21px 82%;
gap: normal thin;
gap: thin thick;
gap: calc(20px + 10%) calc(10% - 5px);
gap: calc(20px + 10%) medium;

/* Global values */
gap: inherit;
gap: initial;
gap: revert;
```

```
gap: revert-layer;
gap: unset;
```

Values

This property is specified as one or two values from the following list:

`normal`

Sets the gap to `1em` in multi-column layouts and to `0` in all other contexts. This is the default value.

`<line-width>`

Sets the size of the gap using the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value.

`<length-percentage>`

Sets the gap to a non-negative `<length>` or `<percentage>` value.

Description

The `gap` property defines gaps between columns and rows, with the effect of the definition depending on whether the container is a grid container, a flexbox container, or a multi-column layout container. See Defining CSS gaps for more information about gaps by layout type.

The shorthand accepts one or two values. A single value sets both the `row-gap` and the `column-gap`. Two values set `row-gap` first, then `column-gap`. The default value is `normal` for both sub-properties; but if you declare only one value, it applies to both.

Percentage gap values are always calculated against the content box size of the container element. The behavior is well-defined and consistent across layout modes when the container size is definite.

The generated gaps create empty spaces that have the width or height of the gap's specified size, much like an empty item or track. The visible space between elements may differ from the provided `gap` value because margins, padding, and distributed alignment may increase the separation between elements beyond what is determined by `gap`.

Gaps may contain visible separators as gap decorations. If there are decorative rules between the columns, rows, or both, they will appear in the middle of their gap, but have no effect on the gaps' size. These decorative lines can be added to the otherwise "empty space" by using the `rule` shorthand.

In grid layouts

In CSS grid layout, the `gap` property defines the space between rows and columns. If two values are included, the first value defines the gutter between rows, and the second defines the gutter between columns.

Percentage values are calculated against the content box size of the container element. Cyclic percentage sizes resolve against zero for determining intrinsic size contributions but resolve against the grid container's content box when laying out the contents. Two examples below demonstrate percentage gap values with explicit container size and implicit container size in the examples section.

The effect of positive `gap` values is as though the grid lines acquired thickness: the grid track between two grid lines is the space between the gutters that represent them. If a grid item spans multiple rows or columns, for the purpose of track sizing, the gutter is treated as an extra, empty, fixed-size track of the specified size, added to the dimension in the spanning direction. For example, if `gap: 10px` is set on a 3x3 grid of 100px by 100px boxes, if a grid item spans two vertical columns, its width would be `210px`. If it spans all three, it has a width of `320px`.

The space between grid rows and columns may be larger than the value of the `gap` property due to space added between tracks by the `justify-content` and `align-content` properties.

Gutters only appear between tracks of the implicit grid. If a grid is fragmented between tracks, no gutter spacing is added between those tracks. There is no gutter before the first track or after the last track, and if a track is collapsed, it will have no gutter.

Early versions of the CSS grid specification called this property `grid-gap`. To maintain compatibility with legacy websites, browsers accept `grid-gap` as an alias for `gap`.

In flexbox

With flex containers, the `gap` property defines the space between both flex items and flex lines. Whether the first value is the gap between flex items or between flex lines depends on the direction. Flex items are laid out in either rows or columns depending on the value of the `flex-direction` property. For rows (`row` (the default) or `row-reverse`), the first value defines the gap between flex lines, and the second value defines the gap between items within each line. If only one value is included, that value is used for both dimensions.

For columns (`column` or `column-reverse`), the first value defines the gap between flex items within a flex line, and the second value defines the gaps between each flex line. Again, if only one value is included, that value is used for both dimensions.

In multi column layouts

In CSS multi-column layout, the property defines the gutter between columns and rows of columns. The first value defines the gap between adjacent column boxes, while the second value defines the size of the gutter between rows of column boxes, if multiple rows were established by the `column-height` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `row-gap`: `normal` • `column-gap`: `normal`
Applies to	multi-column elements, flex containers, grid containers
Inherited	no
Computed value	as each of the properties of the shorthand: • `row-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements • `column-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements
Animation type	as each of the properties of the shorthand: • `row-gap`: a length, percentage or calc(); • `column-gap`: a length, percentage or calc();

Formal syntax

```
gap =
  <'row-gap'> <'column-gap'>?

<row-gap> =
  normal                      |
  <length-percentage [0,∞]>   |
  <line-width>

<column-gap> =
  normal                      |
```

```
  <length-percentage [0,∞]>  |
  <line-width>

<length-percentage> =
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Flex layout

HTML

```
<div id="flexbox">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

```
#flexbox {
  display: flex;
  flex-wrap: wrap;
  width: 300px;
  gap: 20px 5px;
}

#flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  width: 100px;
  height: 50px;
}
```

Result

Grid layout

HTML

```
<div id="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

```
#grid {
  display: grid;
  height: 200px;
  grid-template: repeat(3, 1fr) / repeat(3, 1fr);
  gap: 20px 5px;
}

#grid > div {
  border: 1px solid green;
  background-color: lime;
}
```

Result

Multi-column layout

HTML

```
<p class="content-box">
  This is some multi-column text with a 40px column gap created with the CSS
  <code>gap</code> property. Don't you think that's fun and exciting? I sure do!
</p>
```

CSS

```
.content-box {
  column-count: 3;
  gap: 40px;
}
```

Result

Percentage gap value and explicit container size

If the container has a fixed size set, then gap percentage value calculations are based on the size of the container. Thus, gap behavior is consistent across all layouts. In the following example, there are two containers, one with a grid layout and the other with a flex layout. The containers have five red 20x20px children. Both containers are explicitly set to 200px high using `height: 200px` and the gap is set with `gap: 12.5% 0`. See Specifying gap values as percentages for more information.

```
<span>Grid</span>
<div id="grid">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
<span>Flex</span>
<div id="flex">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
```

```
body > div {
  background-color: #cccccc;
  width: 200px;
  flex-flow: column;
}
```

```
#grid {
  display: inline-grid;
  height: 200px;
  gap: 12.5% 0;
}

#flex {
  display: inline-flex;
  height: 200px;
  gap: 12.5% 0;
}

#grid > div,
#flex > div {
  background-color: coral;
  width: 20px;
  height: 20px;
}
```

Now inspect the grid and flex elements using Inspector tab in Web Developer Tools. In order to see the actual gaps hover mouse over `<div id="grid">` and `<div id="flex">` tags in the inspector. You will notice that the gap is the same in both cases which is 25px.

Percentage gap value and implicit container size

If size is not explicitly set on the container, then the percentage gap behaves differently in case of grid and flex layouts. In the following example the containers don't have height explicitly set.

```
<span>Grid</span>
<div id="grid">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
<span>Flex</span>
<div id="flex">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
```

```
body > div {
  background-color: #cccccc;
  width: 200px;
}

#grid {
  display: inline-grid;
  gap: 12.5% 0;
}

#flex {
  display: inline-flex;
  gap: 12.5% 0;
  flex-flow: column;
}

#grid > div,
#flex > div {
  background-color: coral;
  width: 20px;
  height: 20px;
}
```

In case of the grid layout, percentage gap doesn't contribute to the actual height of the grid. The container's height is calculated using `0px` gap, so the actual height turns out to be 100px (20px x 5). Then the actual percentage gap is calculated using the content box's height, the gap turns out to be `12.5px` (100px x 12.5%). The gap is applied just before rendering. Thus the grid remains 100px high but it overflows due to the percentage gap added later just before rendering.

In case of the flex layout, the percentage gap always results in zero value.

See also

- `row-gap`
- `column-gap`
- `rule`
- Defining CSS gaps
- Basic concepts of grid layout: gutters
- CSS gaps module
- CSS box alignment module
- CSS flexible box layout module
- CSS grid layout module
- CSS multi-column layout module

grid CSS property

The **`grid`** CSS shorthand property sets all of the explicit and implicit grid properties in a single declaration.

Using `grid` you specify one axis using `grid-template-rows` or `grid-template-columns`, you then specify how content should auto-repeat in the other axis using the implicit grid properties: `grid-auto-rows`, `grid-auto-columns`, and `grid-auto-flow`.

Try it

```
grid: auto-flow / 1fr 1fr 1fr;
```

```
grid: auto-flow dense / 40px 40px 1fr;
```

```
grid: repeat(3, 80px) / auto-flow;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-gap: 10px;
  width: 200px;
}

#example-element :nth-child(1) {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element :nth-child(2) {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
  grid-column: auto / span 3;
  grid-row: auto / span 2;
}

#example-element :nth-child(3) {
  background-color: rgb(94 255 0 / 0.2);
  border: 3px solid green;
  grid-column: auto / span 2;
}
```

> **Note:** The sub-properties you don't specify are set to their initial value, as normal for shorthands. Also, the gutter properties are NOT reset by this shorthand.

Constituent properties

This property is a shorthand for the following CSS properties:

- `grid-auto-columns`
- `grid-auto-flow`
- `grid-auto-rows`
- `grid-template-areas`
- `grid-template-columns`
- `grid-template-rows`

Syntax

```
/* <'grid-template'> values */
grid: none;
grid: "a" 100px "b" 1fr;
grid: [line-name1] "a" 100px [line-name2];
grid: "a" 200px "b" min-content;
grid: "a" minmax(100px, max-content) "b" 20%;
grid: 100px / 200px;
grid: minmax(400px, min-content) / repeat(auto-fill, 50px);

/* <'grid-template-rows'> /
   [ auto-flow && dense? ] <'grid-auto-columns'>? values */
grid: 200px / auto-flow;
grid: 30% / auto-flow dense;
grid: repeat(3, 200px) / auto-flow 300px;
grid: [line1] minmax(20em, max-content) / auto-flow dense 40%;
```

```
/* [ auto-flow && dense? ] <'grid-auto-rows'>? /
   <'grid-template-columns'> values */
grid: auto-flow / 200px;
grid: auto-flow dense / 30%;
grid: auto-flow 300px / repeat(3, 200px);
grid: auto-flow dense 40% / [line1] minmax(20em, max-content);

/* Global values */
grid: inherit;
grid: initial;
grid: revert;
grid: revert-layer;
grid: unset;
```

Values

`<'grid-template'>`

Defines the `grid-template` including `grid-template-columns`, `grid-template-rows` and `grid-template-areas`.

`<'grid-template-rows'> / [ auto-flow && dense? ] <'grid-auto-columns'>?`

Sets up an auto-flow by setting the row tracks explicitly via the `grid-template-rows` property (and the `grid-template-columns` property to `none`) and specifying how to auto-repeat the column tracks via `grid-auto-columns` (and setting `grid-auto-rows` to `auto`). `grid-auto-flow` is also set to `column` accordingly, with `dense` if it's specified.

All other `grid` sub-properties are reset to their initial values.

`[ auto-flow && dense? ] <'grid-auto-rows'>? / <'grid-template-columns'>`

Sets up an auto-flow by setting the column tracks explicitly via the `grid-template-columns` property (and the `grid-template-rows` property to `none`) and specifying how to auto-repeat the row tracks via `grid-auto-rows` (and setting `grid-auto-columns` to `auto`). `grid-auto-flow` is also set to `row` accordingly, with `dense` if it's specified.

All other `grid` sub-properties are reset to their initial values.

Formal definition

Initial value	as each of the properties of the shorthand: • `grid-template-rows`: `none` • `grid-template-columns`: `none` • `grid-template-areas`: `none` • `grid-auto-rows`: `auto` • `grid-auto-columns`: `auto` • `grid-auto-flow`: `row` • `grid-column-gap`: `0` • `grid-row-gap`: `0` • `column-gap`: `normal` • `row-gap`: `normal`
Applies to	grid containers
Inherited	no
Percentages	as each of the properties of the shorthand: • `grid-template-rows`: refer to corresponding dimension of the content area • `grid-template-columns`: refer to corresponding dimension of the content area • `grid-auto-rows`: refer to corresponding dimension of the content area • `grid-auto-columns`: refer to corresponding dimension of the content area
Computed value	as each of the properties of the shorthand: • `grid-template-rows`: as specified, but with relative lengths converted into absolute lengths • `grid-template-columns`: as specified, but with relative lengths converted into absolute lengths • `grid-template-areas`: as specified • `grid-auto-rows`: the percentage as specified or the absolute length • `grid-auto-columns`: the percentage as specified or the absolute length • `grid-auto-flow`: as specified • `grid-column-gap`: the percentage as specified or the absolute length • `grid-row-gap`: the percentage as specified or the absolute length • `column-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements • `row-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements
Animation type	as each of the properties of the shorthand: • `grid-template-rows`: simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list • `grid-template-columns`: simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list • `grid-template-areas`: discrete • `grid-auto-rows`: by computed value type • `grid-auto-columns`: by computed value type • `grid-auto-flow`: discrete • `grid-column-gap`: a length • `grid-row-gap`: a length • `column-gap`: a length, percentage or calc(); • `row-gap`: a length, percentage or calc();

Formal syntax

```
grid =
  <'grid-template'>                                |
  <'grid-template-rows'> / [ auto-flow && dense? ] <'grid-auto-columns'>?  |
  [ auto-flow && dense? ] <'grid-auto-rows'>? / <'grid-template-columns'>
```

```
<grid-template> =
  none                                          |
  [ <'grid-template-rows'> / <'grid-template-columns'> ]  |
  [ <line-names>? <string> <track-size>? <line-names>? ]+ [ / <explicit-track-list> ]?

<grid-template-rows> =
  none                      |
  <track-list>              |
  <auto-track-list>         |
  subgrid <line-name-list>?

<grid-auto-columns> =
  <track-size>+

<grid-auto-rows> =
  <track-size>+

<grid-template-columns> =
  none                      |
  <track-list>              |
  <auto-track-list>         |
  subgrid <line-name-list>?

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                               |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<explicit-track-list> =
  [ <line-names>? <track-size> ]+ <line-names>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?

<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                               |
  minmax( <fixed-breadth> , <track-breadth> )       |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a grid layout

HTML

```
<div id="container">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

```
#container {
  display: grid;
  grid: repeat(2, 60px) / auto-flow 80px;
}

#container > div {
  background-color: #8ca0ff;
  width: 50px;
  height: 50px;
}
```

Result

See also

- `grid-template`
- `grid-template-rows`
- `grid-template-columns`
- `grid-template-areas`
- `grid-auto-columns`
- `grid-auto-rows`
- `grid-auto-flow`
- Line-based placement with CSS grid
- Grid template areas: grid definition shorthands

grid-area CSS property

The **`grid-area`** CSS shorthand property specifies a grid item's size and location within a grid by contributing a line, a span, or nothing (automatic) to its grid placement, thereby specifying the edges of its grid area.

Try it

```
grid-area: a;
```

```
grid-area: b;
```

```
grid-area: c;
```

```
grid-area: 2 / 1 / 2 / 4;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">Example</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-template-areas:
    "a a a"
    "b c c"
    "b c c";
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

If four `<grid-line>` values are specified, `grid-row-start` is set to the first value, `grid-column-start` is set to the second value, `grid-row-end` is set to the third value, and `grid-column-end` is set to the fourth value.

When `grid-column-end` is omitted, if `grid-column-start` is a `<custom-ident>`, `grid-column-end` is set to that `<custom-ident>`; otherwise, it is set to `auto`.

When `grid-row-end` is omitted, if `grid-row-start` is a `<custom-ident>`, `grid-row-end` is set to that `<custom-ident>`; otherwise, it is set to `auto`.

When `grid-column-start` is omitted, if `grid-row-start` is a `<custom-ident>`, all four longhands are set to that value. Otherwise, it is set to `auto`.

The `grid-area` property can also be set to a `<custom-ident>` which acts as a name for the area, which can then be placed using `grid-template-areas`.

Constituent properties

This property is a shorthand for the following CSS properties:

- `grid-row-start`
- `grid-column-start`
- `grid-row-end`
- `grid-column-end`

Syntax

```
/* Keyword values */
grid-area: auto;
grid-area: auto / auto;
grid-area: auto / auto / auto;
grid-area: auto / auto / auto / auto;
```

```
/* <custom-ident> values */
grid-area: some-grid-area;
grid-area: some-grid-area / another-grid-area;

/* <integer> && <custom-ident>? values */
grid-area: 4 some-grid-area;
grid-area: 4 some-grid-area / 2 another-grid-area;

/* span && [ <integer> || <custom-ident> ] values */
grid-area: span 3;
grid-area: span 3 / span some-grid-area;
grid-area: 2 span / another-grid-area span;

/* Global values */
grid-area: inherit;
grid-area: initial;
grid-area: revert;
grid-area: revert-layer;
grid-area: unset;
```

Values

`auto`

Is a keyword indicating that the property contributes nothing to the grid item's placement, indicating auto-placement or a default span of `1`.

`<custom-ident>`

If there is a named line with the name `<custom-ident>-start` or `<custom-ident>-end`, it contributes the first such line to the grid item's placement.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-area: foo;` will choose the start/end edge of that named grid area (unless another line named `foo-start`/`foo-end` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

`<integer> && <custom-ident>?`

Contributes the n-th grid line to the grid item's placement. If a negative integer is given, it instead counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement such that the corresponding edge of the grid item's grid area is *n* lines from the opposite edge.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid corresponding to the search direction are assumed to have that name for the purpose of counting this span.

If the `<integer>` is omitted, it defaults to `1`. Negative integers or 0 are invalid.

Formal definition

Initial value	as each of the properties of the shorthand: • `grid-row-start`: `auto` • `grid-column-start`: `auto` • `grid-row-end`: `auto` • `grid-column-end`: `auto`
Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as each of the properties of the shorthand: • `grid-row-start`: as specified • `grid-column-start`: as specified • `grid-row-end`: as specified • `grid-column-end`: as specified
Animation type	discrete

Formal syntax

```
grid-area =
  <grid-line> [ / <grid-line> ]{0,3}

<grid-line> =
  auto                                          |
  <custom-ident>                                |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting grid areas

HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
</div>
```

CSS

```
#grid {
  display: grid;
  height: 100px;
  grid-template: repeat(4, 1fr) / 50px 100px;
}

#item1 {
  background-color: lime;
  grid-area: 2 / 2 / auto / span 3;
}

#item2 {
  background-color: yellow;
}

#item3 {
  background-color: blue;
}
```

Result

See also

- `grid-row`
- `grid-row-start`
- `grid-row-end`
- `grid-column`
- `grid-column-start`
- `grid-column-end`
- `grid-template-areas`
- Grid template areas
- Video: Grid template areas

grid-auto-columns CSS property

The **`grid-auto-columns`** CSS property specifies the size of an implicitly-created grid column track or pattern of tracks.

If a grid item is positioned into a column that is not explicitly sized by `grid-template-columns`, implicit grid tracks are created to hold it. This can happen either by explicitly positioning into a column that is out of range, or by the auto-placement algorithm creating additional columns.

Try it

```
grid-auto-columns: auto;
```

```
grid-auto-columns: 1fr;
```

```
grid-auto-columns: min-content;
```

```
grid-auto-columns: minmax(10px, auto);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div></div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  width: 220px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element > div:nth-child(1) {
  grid-column: 1 / 3;
}

#example-element > div:nth-child(2) {
  grid-column: 2;
}
```

Syntax

```
/* Keyword values */
grid-auto-columns: min-content;
grid-auto-columns: max-content;
grid-auto-columns: auto;

/* <length> values */
grid-auto-columns: 100px;
grid-auto-columns: 20cm;
grid-auto-columns: 50vmax;

/* <percentage> values */
grid-auto-columns: 10%;
grid-auto-columns: 33.3%;

/* <flex> values */
grid-auto-columns: 0.5fr;
grid-auto-columns: 3fr;

/* minmax() values */
grid-auto-columns: minmax(100px, auto);
grid-auto-columns: minmax(max-content, 2fr);
grid-auto-columns: minmax(20%, 80vmax);

/* fit-content() values */
grid-auto-columns: fit-content(400px);
grid-auto-columns: fit-content(5cm);
grid-auto-columns: fit-content(20%);

/* multiple track-size values */
grid-auto-columns: min-content max-content auto;
grid-auto-columns: 100px 150px 390px;
grid-auto-columns: 10% 33.3%;
grid-auto-columns: 0.5fr 3fr 1fr;
grid-auto-columns: minmax(100px, auto) minmax(max-content, 2fr)
  minmax(20%, 80vmax);
grid-auto-columns: 100px minmax(100px, auto) 10% 0.5fr fit-content(400px);

/* Global values */
grid-auto-columns: inherit;
grid-auto-columns: initial;
grid-auto-columns: revert;
grid-auto-columns: revert-layer;
grid-auto-columns: unset;
```

Values

`<length>`

Is a non-negative length.

`<percentage>`

Is a non-negative `<percentage>` value relative to the block size of the grid container. If the block size of the grid container is indefinite, the percentage value is treated like `auto`.

`<flex>`

Is a non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor.

When appearing outside a `minmax()` notation, it implies an automatic minimum (i.e., `minmax(auto, <flex>)`).

`max-content`

Is a keyword representing the largest maximal content contribution of the grid items occupying the grid track.

`min-content`

Is a keyword representing the largest minimal content contribution of the grid items occupying the grid track.

`minmax(min, max)`

Is a functional notation that defines a size range greater than or equal to *min* and less than or equal to *max*. If *max* is smaller than *min*, then *max* is ignored and the function is treated as *min*. As a maximum, a `<flex>` value sets the track's flex factor. As a minimum, it is treated as zero (or minimal content, if the grid container is sized under a minimal content constraint).

`fit-content( [ <length> | <percentage> ] )`

Represents the formula `min(max-content, max(auto, argument))`, which is calculated similar to `auto` (i.e., `minmax(auto, max-content)`), except that the track size is clamped at *argument* if it is greater than the `auto` minimum.

`auto`

As a maximum represents the largest `max-content` size of the items in that track.

As a minimum represents the largest minimum size of items in that track (specified by the `min-width`/`min-height` of the items). This is often, though not always, the `min-content` size.

If used outside of `minmax()` notation, `auto` represents the range between the minimum and maximum described above. This behaves similarly to `minmax(min-content,max-content)` in most cases.

> **Note:** `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties. Therefore by default, an `auto` sized track will take up any remaining space in the grid container.

Formal definition

Initial value	`auto`
Applies to	grid containers
Inherited	no
Percentages	refer to corresponding dimension of the content area

Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
grid-auto-columns =
  <track-size>+

<track-size> =
  <track-breadth>                                  |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting grid column size

HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
</div>
```

CSS

```
#grid {
  height: 100px;
  display: grid;
  grid-template-areas: "a a";
  gap: 10px;
  grid-auto-columns: 200px;
}

#grid > div {
  background-color: lime;
}
```

Result

See also

- grid-auto-rows
- grid-auto-flow
- grid
- Auto-placement in grid layout: sizing rows in the implicit grid
- Video: Introducing grid auto-placement and order

grid-auto-flow CSS property

The **`grid-auto-flow`** CSS property controls how the auto-placement algorithm works, specifying exactly how auto-placed items get flowed into the grid.

Try it

```
grid-auto-flow: row;
```

```
grid-auto-flow: column;
```

```
grid-auto-flow: row dense;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
```

```
  width: 220px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element > div:nth-child(1) {
  grid-column: auto / span 2;
}

#example-element > div:nth-child(2) {
  grid-column: auto / span 2;
}
```

Syntax

```
/* Keyword values */
grid-auto-flow: row;
grid-auto-flow: column;
grid-auto-flow: dense;
grid-auto-flow: row dense;
grid-auto-flow: column dense;

/* Global values */
grid-auto-flow: inherit;
grid-auto-flow: initial;
grid-auto-flow: revert;
grid-auto-flow: revert-layer;
grid-auto-flow: unset;
```

This property may take one of two forms:

- a single keyword: one of `row`, `column`, or `dense`.
- two keywords: `row dense` or `column dense`.

Values

`row`

Items are placed by filling each row in turn, adding new rows as necessary. If neither `row` nor `column` is provided, `row` is assumed.

`column`

Items are placed by filling each column in turn, adding new columns as necessary.

`dense`

"dense" packing algorithm attempts to fill in holes earlier in the grid, if smaller items come up later. This may cause items to appear out-of-order, when doing so would fill in holes left by larger items.

If it is omitted, a "sparse" algorithm is used, where the placement algorithm only ever moves "forward" in the grid when placing items, never backtracking to fill holes. This ensures that all of the auto-placed items appear "in order", even if this leaves holes that could have been filled by later items.

Formal definition

Initial value	`row`
Applies to	grid containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
grid-auto-flow =
  [ row | column ]  ||
  dense
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting grid auto-placement

HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
  <div id="item4"></div>
  <div id="item5"></div>
</div>
<select id="direction">
  <option value="column">column</option>
  <option value="row">row</option>
</select>
<input id="dense" type="checkbox" />
<label for="dense">dense</label>
```

CSS

```
#grid {
  height: 200px;
  width: 200px;
  display: grid;
  gap: 10px;
  grid-template: repeat(4, 1fr) / repeat(2, 1fr);
  grid-auto-flow: column; /* or 'row', 'row dense', 'column dense' */
```

```
}

#item1 {
  background-color: lime;
  grid-row-start: 3;
}

#item2 {
  background-color: yellow;
}

#item3 {
  background-color: blue;
}

#item4 {
  grid-column-start: 2;
  background-color: red;
}

#item5 {
  background-color: aqua;
}
```

```
function changeGridAutoFlow() {
  const grid = document.getElementById("grid");
  const direction = document.getElementById("direction");
  const dense = document.getElementById("dense");
  let gridAutoFlow = direction.value === "row" ? "row" : "column";

  if (dense.checked) {
    gridAutoFlow += " dense";
  }

  grid.style.gridAutoFlow = gridAutoFlow;
}

const selectElem = document.querySelector("select");
const inputElem = document.querySelector("input");
selectElem.addEventListener("change", changeGridAutoFlow);
inputElem.addEventListener("change", changeGridAutoFlow);
```

Result

See also

- `grid-auto-rows`
- `grid-auto-columns`
- `grid`
- Auto-placement in grid layout
- Video: Introducing grid auto-placement and order

grid-auto-rows CSS property

The **`grid-auto-rows`** CSS property specifies the size of an implicitly-created grid row track or pattern of tracks.

If a grid item is positioned into a row that is not explicitly sized by `grid-template-rows`, implicit grid tracks are created to hold it. This can happen either by explicitly positioning into a row that is out of range, or by the auto-placement algorithm creating additional rows.

Try it

```
grid-auto-rows: auto;
```

```
grid-auto-rows: 50px;
```

```
grid-auto-rows: min-content;
```

```
grid-auto-rows: minmax(30px, auto);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  width: 220px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  font-size: 22px;
}

#example-element div:last-child {
```

```
  font-size: 13px;
}
```

Syntax

```
/* Keyword values */
grid-auto-rows: min-content;
grid-auto-rows: max-content;
grid-auto-rows: auto;

/* <length> values */
grid-auto-rows: 100px;
grid-auto-rows: 20cm;
grid-auto-rows: 50vmax;

/* <percentage> values */
grid-auto-rows: 10%;
grid-auto-rows: 33.3%;

/* <flex> values */
grid-auto-rows: 0.5fr;
grid-auto-rows: 3fr;

/* minmax() values */
grid-auto-rows: minmax(100px, auto);
grid-auto-rows: minmax(max-content, 2fr);
grid-auto-rows: minmax(20%, 80vmax);

/* fit-content() values */
grid-auto-rows: fit-content(400px);
grid-auto-rows: fit-content(5cm);
grid-auto-rows: fit-content(20%);

/* multiple track-size values */
grid-auto-rows: min-content max-content auto;
grid-auto-rows: 100px 150px 390px;
grid-auto-rows: 10% 33.3%;
grid-auto-rows: 0.5fr 3fr 1fr;
grid-auto-rows: minmax(100px, auto) minmax(max-content, 2fr) minmax(20%, 80vmax);
grid-auto-rows: 100px minmax(100px, auto) 10% 0.5fr fit-content(400px);

/* Global values */
grid-auto-rows: inherit;
grid-auto-rows: initial;
grid-auto-rows: revert;
grid-auto-rows: revert-layer;
grid-auto-rows: unset;
```

Values

`<length>`

Is a non-negative length.

`<percentage>`

Is a non-negative `<percentage>` value relative to the block size of the grid container. If the block size of the grid container is indefinite, the percentage value is treated like `auto`.

`<flex>`

Is a non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor.

When appearing outside a `minmax()` notation, it implies an automatic minimum (i.e., `minmax(auto, <flex>)`).

`max-content`

Is a keyword representing the largest maximal content contribution of the grid items occupying the grid track.

`min-content`

Is a keyword representing the largest minimal content contribution of the grid items occupying the grid track.

`minmax(min, max)`

Is a functional notation that defines a size range greater than or equal to *min* and less than or equal to *max*. If *max* is smaller than *min*, then *max* is ignored and the function is treated as *min*. As a maximum, a `<flex>` value sets the track's flex factor. As a minimum, it is treated as zero (or minimal content, if the grid container is sized under a minimal content constraint).

`fit-content( [ <length> | <percentage> ] )`

Represents the formula `min(max-content, max(auto, argument))`, which is calculated similar to `auto` (i.e., `minmax(auto, max-content)`), except that the track size is clamped at *argument* if it is greater than the `auto` minimum.

`auto`

As a maximum represents the largest `max-content` size of the items in that track.

As a minimum represents the largest minimum size of items in that track (specified by the `min-width`/`min-height` of the items). This is often, though not always, the `min-content` size.

If used outside of `minmax()` notation, `auto` represents the range between the minimum and maximum described above. This behaves similarly to `minmax(min-content,max-content)` in most cases.

> **Note:** `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties. Therefore by default, an `auto` sized track will take up any remaining space in the grid container.

Formal definition

Initial value	`auto`
Applies to	grid containers
Inherited	no

Percentages	refer to corresponding dimension of the content area
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
grid-auto-rows =
  <track-size>+

<track-size> =
  <track-breadth>                                  |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting grid row size

HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
</div>
```

CSS

```
#grid {
  width: 200px;
  display: grid;
  grid-template-areas: "a a";
  gap: 10px;
  grid-auto-rows: 100px;
}

#grid > div {
  background-color: lime;
}
```

Result

See also

- grid-auto-columns
- grid-auto-flow
- grid
- Auto-placement in grid layout - sizing rows in the implicit grid
- Video: Introducing grid auto-placement and order

grid-column CSS property

The **`grid-column`** CSS shorthand property specifies a grid item's size and location within a grid column by contributing a line, a span, or nothing (automatic) to its grid placement, thereby specifying the inline-start and inline-end edge of its grid area.

Try it

```
grid-column: 1;
```

```
grid-column: 1 / 3;
```

```
grid-column: 2 / -1;
```

```
grid-column: 1 / span 2;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
```

```
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `grid-column-end`
- `grid-column-start`

Syntax

```
/* Keyword values */
grid-column: auto;
grid-column: auto / auto;

/* <custom-ident> values */
grid-column: some-grid-area;
grid-column: some-grid-area / some-other-grid-area;

/* <integer> + <custom-ident> values */
grid-column: some-grid-area 4;
grid-column: 4 some-grid-area / 6;

/* span + <integer> + <custom-ident> values */
grid-column: span 3;
grid-column: span some-grid-area;
grid-column: 5 some-grid-area span;
grid-column: span 3 / 6;
grid-column: span some-grid-area / span some-other-grid-area;
grid-column: 5 some-grid-area span / 2 span;

/* Global values */
grid-column: inherit;
grid-column: initial;
grid-column: revert;
grid-column: revert-layer;
grid-column: unset;
```

This property is specified as one or two `<grid-line>` values.

If two `<grid-line>` values are given, they are separated by `/`. The `grid-column-start` longhand is set to the value before the slash, and the `grid-column-end` longhand is set to the value after the slash.

Each `<grid-line>` value can be specified as:

- either the `auto` keyword
- or a `<custom-ident>` value
- or an `<integer>` value
- or both `<custom-ident>` and `<integer>`, separated by a space
- or the keyword `span` together with either a `<custom-ident>` or an `<integer>` or both.

Values

`auto`

Is a keyword indicating that the property contributes nothing to the grid item's placement, indicating auto-placement, an automatic span, or a default span of `1`.

`<custom-ident>`

If there is a named line with the name `<custom-ident>-start`/`<custom-ident>-end`, it contributes the first such line to the grid item's placement.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-column: foo;` will choose the start/end edge of that named grid area (unless another line named `foo-start`/`foo-end` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

`<integer> && <custom-ident>?`

Contributes the nth grid line to the grid item's placement. If a negative integer is given, it instead counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement such that the corresponding edge of the grid item's grid area is n lines from the opposite edge.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid corresponding to the search direction are assumed to have that name for the purpose of counting this span.

If the `<integer>` is omitted, it defaults to `1`. Negative integers or 0 are invalid.

Formal definition

Initial value	as each of the properties of the shorthand:

	• grid-column-start: auto • grid-column-end: auto
Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as each of the properties of the shorthand: • grid-column-start: as specified • grid-column-end: as specified
Animation type	discrete

Formal syntax

```
grid-column =
  <grid-line> [ / <grid-line> ]?

<grid-line> =
  auto                                          |
  <custom-ident>                                |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting grid column size and location

HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
</div>
```

CSS

```
#grid {
  display: grid;
  height: 100px;
  grid-template-columns: repeat(6, 1fr);
  grid-template-rows: 100px;
}

#item1 {
  background-color: lime;
}

#item2 {
  background-color: yellow;
  grid-column: 2 / 4;
}

#item3 {
  background-color: blue;
  grid-column: span 2 / 7;
}
```

Result

See also

- grid-row
- grid-row-start
- grid-row-end
- grid-column-start
- grid-column-end
- Line-based placement with CSS grid
- Video: Line-based placement

grid-column-end CSS property

The **grid-column-end** CSS property specifies a grid item's end position within the grid column by contributing a line, a span, or nothing (automatic) to its grid placement, thereby specifying the block-end edge of its grid area.

Try it

```
grid-column-end: auto;
```

```
grid-column-end: 3;
```

```
grid-column-end: -1;
```

```
grid-column-end: span 3;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
```

```
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Syntax

```
/* Keyword value */
grid-column-end: auto;

/* <custom-ident> values */
grid-column-end: some-grid-area;

/* <integer> + <custom-ident> values */
grid-column-end: 2;
grid-column-end: some-grid-area 4;

/* span + <integer> + <custom-ident> values */
grid-column-end: span 3;
grid-column-end: span some-grid-area;
grid-column-end: 5 some-grid-area span;

/* Global values */
grid-column-end: inherit;
grid-column-end: initial;
grid-column-end: revert;
grid-column-end: revert-layer;
grid-column-end: unset;
```

Values

`auto`

Contributes nothing to the grid item's placement, indicating auto-placement, an automatic span, or a default span of `1`. This is the default value.

`<custom-ident>`

Contributes the first line to the grid item's placement if there is a named line with the name `<custom-ident>-end`.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-column-end: foo;` will choose the end edge of that named grid area (unless another line named `foo-end` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

`<integer> && <custom-ident>?`

Contributes the nth grid line to the grid item's placement. If a negative integer is given, it instead counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a <custom-ident>, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement such that the column end edge of the grid item's grid area is n lines from the start edge.

If a name is given as a <custom-ident>, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid corresponding to the search direction are assumed to have that name for the purpose of counting this span.

If the <integer> is omitted, it defaults to `1`. Negative integers or 0 are invalid.

The `<custom-ident>` cannot take the `span` and `auto` values.

Formal definition

Initial value	`auto`
Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
grid-column-end =
  <grid-line>

<grid-line> =
  auto                                    |
```

```
  <custom-ident>                                  |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting column end for a grid item

HTML

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
  <div class="box5">Five</div>
</div>
```

CSS

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 3;
  grid-row-end: 5;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

Result

See also

- grid-column-start
- grid-column
- grid-row-start
- grid-row-end
- grid-row
- Line-based placement with CSS grid
- Video: Line-based placement

grid-column-start CSS property

The **grid-column-start** CSS property specifies a grid item's start position within the grid column by contributing a line, a span, or nothing (automatic) to its grid placement. This start position defines the block-start edge of the grid area.

Try it

```
grid-column-start: auto;
```

```
grid-column-start: 2;
```

```
grid-column-start: -1;
```

```
grid-column-start: span 2;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Syntax

```
/* Keyword value */
grid-column-start: auto;

/* <custom-ident> value */
grid-column-start: some-grid-area;

/* <integer> + <custom-ident> values */
grid-column-start: 2;
grid-column-start: some-grid-area 4;

/* span + <integer> + <custom-ident> values */
grid-column-start: span 3;
grid-column-start: span some-grid-area;
grid-column-start: span some-grid-area 5;

/* Global values */
grid-column-start: inherit;
grid-column-start: initial;
grid-column-start: revert;
grid-column-start: revert-layer;
grid-column-start: unset;
```

This property is specified as a single `<grid-line>` value. A `<grid-line>` value can be specified as:

- either the `auto` keyword
- or a `<custom-ident>` value
- or an `<integer>` value
- or both `<custom-ident>` and `<integer>`, separated by a space
- or the keyword `span` together with either a `<custom-ident>` or an `<integer>` or both.

Values

`auto`

A keyword indicating that the property contributes nothing to the grid item's placement, indicating auto-placement, an automatic span, or a default span of `1`.

`<custom-ident>`

If there is a named line with the name `<custom-ident>-start`, it contributes the first such line to the grid item's placement.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-column-start: foo;` will choose the start edge of that named grid area (unless another line named `foo-start` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

`<integer> && <custom-ident>?`

Contributes the nth grid line to the grid item's placement. If a negative integer is given, it counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement, such that the column start edge of the grid item's grid area is n lines from the end edge.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid corresponding to the search direction are assumed to have that name for the purpose of counting this span.

If the <integer> is omitted, it defaults to `1`. Negative integers and `0` are invalid.

The `<custom-ident>` cannot take the `span` and `auto` values.

Formal definition

Initial value	`auto`

Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
grid-column-start =
  <grid-line>

<grid-line> =
  auto                                          |
  <custom-ident>                                |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting column start for a grid item

HTML

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
  <div class="box5">Five</div>
</div>
```

CSS

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 3;
  grid-row-end: 5;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

Result

See also

- `grid-column-end`
- `grid-column`
- `grid-row-start`
- `grid-row-end`
- `grid-row`
- Line-based placement with CSS grid
- Video: Line-based placement

grid-row CSS property

The **`grid-row`** CSS shorthand property specifies a grid item's size and location within a grid row by contributing a line, a span, or nothing (automatic) to its grid placement, thereby specifying the inline-start and inline-end edge of its grid area.

Try it

```
grid-row: 1;
```

```
grid-row: 1 / 3;
```

```
grid-row: 2 / -1;
```

```
grid-row: 1 / span 2;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `grid-row-end`
- `grid-row-start`

Syntax

```
/* Keyword values */
grid-row: auto;
grid-row: auto / auto;

/* <custom-ident> values */
grid-row: some-grid-area;
grid-row: some-grid-area / some-other-grid-area;

/* <integer> + <custom-ident> values */
grid-row: some-grid-area 4;
grid-row: 4 some-grid-area / 6;

/* span + <integer> + <custom-ident> values */
grid-row: span 3;
grid-row: span some-grid-area;
grid-row: 5 some-grid-area span;
grid-row: span 3 / 6;
grid-row: span some-grid-area / span some-other-grid-area;
grid-row: 5 some-grid-area span / 2 span;

/* Global values */
grid-row: inherit;
grid-row: initial;
grid-row: revert;
grid-row: revert-layer;
grid-row: unset;
```

This property is specified as one or two `<grid-line>` values.

If two `<grid-line>` values are given, they are separated by `/`. The `grid-row-start` longhand is set to the value before the slash, and the `grid-row-end` longhand is set to the value after the slash.

Each `<grid-line>` value can be specified as:

- either the `auto` keyword
- or a `<custom-ident>` value
- or an `<integer>` value
- or both `<custom-ident>` and `<integer>`, separated by a space
- or the keyword `span` together with either a `<custom-ident>` or an `<integer>` or both.

Values

`auto`

Is a keyword indicating that the property contributes nothing to the grid item's placement, indicating auto-placement, an automatic span, or a default span of `1`.

`<custom-ident>`

If there is a named line with the name `<custom-ident>-start/<custom-ident>-end`, it contributes the first such line to the grid item's placement.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-row: foo;` will choose the start/end edge of that named grid area (unless another line named `foo-start`/`foo-end` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

`<integer> && <custom-ident>?`

Contributes the nth grid line to the grid item's placement. If a negative integer is given, it instead counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement such that the corresponding edge of the grid item's grid area is n lines from the opposite edge.

If a name is given as a `<custom-ident>`, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid corresponding to the search direction are assumed to have that name for the purpose of counting this span.

If the `<integer>` is omitted, it defaults to `1`. Negative integers or 0 are invalid.

Formal definition

Initial value	as each of the properties of the shorthand: • `grid-row-start`: `auto` • `grid-row-end`: `auto`
Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as each of the properties of the shorthand: • `grid-row-start`: as specified • `grid-row-end`: as specified
Animation type	discrete

Formal syntax

```
grid-row =
  <grid-line> [ / <grid-line> ]?

<grid-line> =
  auto                                          |
  <custom-ident>                                |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting grid row size and location

HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
</div>
```

CSS

```
#grid {
  display: grid;
  height: 200px;
  grid-template-columns: 200px;
  grid-template-rows: repeat(6, 1fr);
}

#item1 {
  background-color: lime;
}

#item2 {
  background-color: yellow;
  grid-row: 2 / 4;
}

#item3 {
  background-color: blue;
  grid-row: span 2 / 7;
}
```

Result

See also

- `grid-row-start`
- `grid-row-end`
- `grid-column`
- `grid-column-start`
- `grid-column-end`
- Line-based placement with CSS grid

- Video: Line-based placement

grid-row-end CSS property

The **`grid-row-end`** CSS property specifies a grid item's end position within the grid row by contributing a line, a span, or nothing (automatic) to its grid placement, thereby specifying the inline-end edge of its grid area.

Try it

```
grid-row-end: auto;
```

```
grid-row-end: 3;
```

```
grid-row-end: -1;
```

```
grid-row-end: span 3;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Syntax

```
/* Keyword value */
grid-row-end: auto;

/* <custom-ident> values */
grid-row-end: some-grid-area;

/* <integer> + <custom-ident> values */
grid-row-end: 2;
grid-row-end: some-grid-area 4;

/* span + <integer> + <custom-ident> values */
grid-row-end: span 3;
grid-row-end: span some-grid-area;
grid-row-end: 5 some-grid-area span;

/* Global values */
grid-row-end: inherit;
grid-row-end: initial;
grid-row-end: revert;
grid-row-end: revert-layer;
grid-row-end: unset;
```

Values

`auto`

Is a keyword indicating that the property contributes nothing to the grid item's placement, indicating auto-placement, an automatic span, or a default span of `1`.

`<custom-ident>`

If there is a named line with the name '<custom-ident>-end', it contributes the first such line to the grid item's placement.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-row-end: foo;` will choose the end edge of that named grid area (unless another line named `foo-end` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

The `<custom-ident>` cannot take the `span` and `auto` values.

`<integer> && <custom-ident>?`

Contributes the nth grid line to the grid item's placement. If a negative integer is given, it instead counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a <custom-ident>, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement such that the row end edge of the grid item's grid area is n lines from the start edge.

If a name is given as a <custom-ident>, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid corresponding to the search direction are assumed to have that name for the purpose of counting this span.

If the <integer> is omitted, it defaults to 1. Negative integers or 0 are invalid.

Formal definition

Initial value	auto
Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
grid-row-end =
  <grid-line>

<grid-line> =
  auto                                          |
  <custom-ident>                                |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting row end for a grid item

HTML

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
  <div class="box5">Five</div>
</div>
```

CSS

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 3;
  grid-row-end: 5;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

Result

See also

- grid-row-start
- grid-row
- grid-column-start
- grid-column-end

- `grid-column`
- Line-based placement with CSS grid
- Video: Line-based placement

grid-row-start CSS property

The **`grid-row-start`** CSS property specifies a grid item's start position within the grid row by contributing a line, a span, or nothing (automatic) to its grid placement, thereby specifying the inline-start edge of its grid area.

Try it

```
grid-row-start: auto;
```

```
grid-row-start: 3;
```

```
grid-row-start: -1;
```

```
grid-row-start: span 2;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
  width: 200px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Syntax

```
/* Keyword value */
grid-row-start: auto;

/* <custom-ident> values */
grid-row-start: some-grid-area;

/* <integer> + <custom-ident> values */
grid-row-start: 2;
grid-row-start: some-grid-area 4;

/* span + <integer> + <custom-ident> values */
grid-row-start: span 3;
grid-row-start: span some-grid-area;
grid-row-start: 5 some-grid-area span;

/* Global values */
grid-row-start: inherit;
grid-row-start: initial;
grid-row-start: revert;
grid-row-start: revert-layer;
grid-row-start: unset;
```

This property is specified as a single `<grid-line>` value. A `<grid-line>` value can be specified as:

- either the `auto` keyword
- or a `<custom-ident>` value
- or an `<integer>` value
- or both `<custom-ident>` and `<integer>`, separated by a space
- or the keyword `span` together with either a `<custom-ident>` or an `<integer>` or both.

Values

`auto`

Is a keyword indicating that the property contributes nothing to the grid item's placement, indicating auto-placement, an automatic span, or a default span of `1`.

`<custom-ident>`

If there is a named line with the name '<custom-ident>-start', it contributes the first such line to the grid item's placement.

> **Note:** Named grid areas automatically generate implicit named lines of this form, so specifying `grid-row-start: foo;` will choose the start edge of that named grid area (unless another line named `foo-start` was explicitly specified before it).

Otherwise, this is treated as if the integer `1` had been specified along with the `<custom-ident>`.

`<integer> && <custom-ident>?`

Contributes the nth grid line to the grid item's placement. If a negative integer is given, it instead counts in reverse, starting from the end edge of the explicit grid.

If a name is given as a <custom-ident>, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines are assumed to have that name for the purpose of finding this position.

An `<integer>` value of `0` is invalid.

`span && [ <integer> || <custom-ident> ]`

Contributes a grid span to the grid item's placement; such that the row start edge of the grid item's grid area is n lines from the end edge.

If a name is given as a <custom-ident>, only lines with that name are counted. If not enough lines with that name exist, all implicit grid lines on the side of the explicit grid, corresponding to the search direction, are assumed to have that name for the purpose of counting this span.

If the <integer> is omitted, it defaults to `1`. Negative integers or 0 are invalid.

The `<custom-ident>` cannot take the `span` and `auto` values.

Formal definition

Initial value	`auto`
Applies to	grid items and absolutely-positioned boxes whose containing block is a grid container
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
grid-row-start =
  <grid-line>

<grid-line> =
  auto                                          |
  <custom-ident>                                |
  [ [ <integer [-∞,-1]> | <integer [1,∞]> ] && <custom-ident>? ]  |
  [ span && [ <integer [1,∞]> || <custom-ident> ] ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting row start for a grid item

HTML

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
  <div class="box5">Five</div>
</div>
```

CSS

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 3;
  grid-row-end: 5;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
```

```
  padding: 1em;
}
```

Result

See also

- grid-row-end
- grid-row
- grid-column-start
- grid-column-end
- grid-column
- Line-based placement with CSS grid
- Video: Line-based placement

grid-template CSS property

The **grid-template** CSS shorthand property specifies the grid columns, grid rows, and grid areas.

Try it

```
grid-template:
  "a a a" 40px
  "b c c" 40px
  "b c c" 40px / 1fr 1fr 1fr;
```

```
grid-template:
  "b b a" auto
  "b b c" 2ch
  "b b c" 1em / 20% 20px 1fr;
```

```
grid-template:
  "a a ." minmax(50px, auto)
  "a a ." 80px
  "b b c" auto / 2em 3em auto;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-gap: 10px;
  width: 200px;
}

#example-element :nth-child(1) {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  grid-area: a;
}

#example-element :nth-child(2) {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
  grid-area: b;
}

#example-element :nth-child(3) {
  background-color: rgb(94 255 0 / 0.2);
  border: 3px solid green;
  grid-area: c;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- grid-template-areas
- grid-template-columns
- grid-template-rows

Syntax

```
/* Keyword value */
grid-template: none;

/* grid-template-rows / grid-template-columns values */
grid-template: 100px 1fr / 50px 1fr;
grid-template: auto 1fr / auto 1fr auto;
grid-template: [line-name] 100px / [column-name1] 30% [column-name2] 70%;
grid-template: fit-content(100px) / fit-content(40%);

/* grid-template-areas grid-template-rows / grid-template-column values */
grid-template:
  "a a a"
  "b b b";
grid-template:
  "a a a" 20%
  "b b b" auto;
grid-template:
```

```
  [header-top] "a a a" [header-bottom]
  [main-top] "b b b" 1fr [main-bottom]
  / auto 1fr auto;

/* Global values */
grid-template: inherit;
grid-template: initial;
grid-template: revert;
grid-template: revert-layer;
grid-template: unset;
```

Values

`none`

Sets all three longhand properties to `none`, meaning there is no explicit grid. There are no named grid areas. Rows and columns will be implicitly generated; their size will be determined by the `grid-auto-rows` and `grid-auto-columns` properties. This is the default value.

`<'grid-template-rows'> / <'grid-template-columns'>`

Sets `grid-template-rows` and `grid-template-columns` to the specified values, and sets `grid-template-areas` to `none`.

`[ <line-names>? <string> <track-size>? <line-names>? ]+ [ / <explicit-track-list> ]?`

Sets `grid-template-areas` to the strings listed, `grid-template-rows` to the track sizes following each string (filling in `auto` for any missing sizes), and splicing in the named lines defined before/after each size, and `grid-template-columns` to the track listing specified after the slash (or `none`, if not specified).

> **Note:** The `repeat()` function isn't allowed in these track listings, as the tracks are intended to visually line up one-to-one with the rows/columns in the "ASCII art".

> **Note:** The `grid` shorthand accepts the same syntax, but also resets the implicit grid properties to their initial values. Use `grid` (as opposed to `grid-template`) to prevent these values from cascading in separately.

Formal definition

Initial value	as each of the properties of the shorthand: • `grid-template-columns`: none • `grid-template-rows`: none • `grid-template-areas`: none
Applies to	grid containers
Inherited	no
Percentages	as each of the properties of the shorthand: • `grid-template-columns`: refer to corresponding dimension of the content area • `grid-template-rows`: refer to corresponding dimension of the content area
Computed value	as each of the properties of the shorthand: • `grid-template-columns`: as specified, but with relative lengths converted into absolute lengths • `grid-template-rows`: as specified, but with relative lengths converted into absolute lengths • `grid-template-areas`: as specified
Animation type	as each of the properties of the shorthand: • `grid-template-columns`: simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list • `grid-template-rows`: simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list • `grid-template-areas`: discrete

Formal syntax

```
grid-template =
  none                                          |
  [ <'grid-template-rows'> / <'grid-template-columns'> ]  |
  [ <line-names>? <string> <track-size>? <line-names>? ]+ [ / <explicit-track-list> ]?

<grid-template-rows> =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<grid-template-columns> =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                               |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<explicit-track-list> =
  [ <line-names>? <track-size> ]+ <line-names>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?
```

```
<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                              |
  minmax( <fixed-breadth> , <track-breadth> )  |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining a grid template

CSS

```
#page {
  display: grid;
  width: 100%;
  height: 200px;
  grid-template:
    [header-left] "head head" 30px [header-right]
    [main-left] "nav  main" 1fr [main-right]
    [footer-left] "nav  foot" 30px [footer-right]
    / 120px 1fr;
}

header {
  background-color: lime;
  grid-area: head;
}

nav {
  background-color: lightblue;
  grid-area: nav;
}

main {
  background-color: yellow;
  grid-area: main;
}

footer {
  background-color: red;
  grid-area: foot;
}
```

HTML

```
<div id="page">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main area</main>
  <footer>Footer</footer>
</div>
```

Result

See also

- grid-template-rows
- grid-template-columns
- grid-template-areas
- Line-based placement with CSS grid
- Grid template areas: grid definition shorthands

- Video: Grid template shorthand

grid-template-areas CSS property

The **`grid-template-areas`** CSS property specifies named grid areas, establishing the cells in the grid and assigning them names.

Those areas are not associated with any particular grid item, but can be referenced from the grid-placement properties `grid-row-start`, `grid-row-end`, `grid-column-start`, `grid-column-end`, and their shorthands `grid-row`, `grid-column`, and `grid-area`.

Try it

```
grid-template-areas:
  "a a a"
  "b c c"
  "b c c";
```

```
grid-template-areas:
  "b b a"
  "b b c"
  "b b c";
```

```
grid-template-areas:
  "a a ."
  "a a ."
  ". b c";
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One (a)</div>
      <div>Two (b)</div>
      <div>Three (c)</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: repeat(3, minmax(40px, auto));
  grid-gap: 10px;
  width: 200px;
}

#example-element :nth-child(1) {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  grid-area: a;
}

#example-element :nth-child(2) {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
  grid-area: b;
}

#example-element :nth-child(3) {
  background-color: rgb(94 255 0 / 0.2);
  border: 3px solid green;
  grid-area: c;
}
```

Syntax

```
/* Keyword value */
grid-template-areas: none;

/* <string> values */
grid-template-areas: "a b";
grid-template-areas:
  "a b ."
  "a c d";

/* Global values */
grid-template-areas: inherit;
grid-template-areas: initial;
grid-template-areas: revert;
grid-template-areas: revert-layer;
grid-template-areas: unset;
```

Values

`none`

The grid container doesn't define any named grid areas.

`<string>`

A row is created for every separate string listed, and a column is created for each cell in the string. Multiple cell tokens with the same name within and between rows create a single named grid area that spans the corresponding grid cells. Unless those cells form a rectangle, the declaration is invalid.

All the remaining unnamed areas in a grid can be referred using *null cell tokens*. A null cell token is a sequence of one or more `.` (U+002E FULL STOP) characters, e.g., `.`, `...`, or `.....` etc. A null cell token can be used to create empty spaces in the grid.

Formal definition

Initial value	none

Applies to	grid containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
grid-template-areas =
  none       |
  <string>+
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying named grid areas

HTML

```
<div id="page">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main area</main>
  <footer>Footer</footer>
</div>
```

CSS

```
#page {
  display: grid;
  width: 100%;
  height: 250px;
  grid-template-areas:
    "head head"
    "nav  main"
    ".  foot";
  grid-template-rows: 50px 1fr 30px;
  grid-template-columns: 150px 1fr;
}

#page > header {
  grid-area: head;
  background-color: #8ca0ff;
}

#page > nav {
  grid-area: nav;
  background-color: #ffa08c;
}

#page > main {
  grid-area: main;
  background-color: #ffff64;
}

#page > footer {
  grid-area: foot;
  background-color: #8cffa0;
}
```

In the above code, a null token (`.`) was used to create an unnamed area in the grid container, which we used to create an empty space at the bottom left corner of the grid.

Result

See also

- `grid-template-rows`
- `grid-template-columns`
- `grid-template`
- Grid template areas
- Video: Grid template areas

grid-template-columns CSS property

The **`grid-template-columns`** CSS property defines the line names and track sizing functions of the grid columns.

Try it

```
grid-template-columns: 60px 60px;
```

```
grid-template-columns: 1fr 60px;
```

```
grid-template-columns: 1fr 2fr;
```

```
grid-template-columns: 8ch auto;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
```

```
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  width: 200px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Keyword value */
grid-template-columns: none;

/* <track-list> values */
grid-template-columns: 100px 1fr;
grid-template-columns: [line-name] 100px;
grid-template-columns: [line-name1] 100px [line-name2 line-name3];
grid-template-columns: minmax(100px, 1fr);
grid-template-columns: fit-content(40%);
grid-template-columns: repeat(3, 200px);
grid-template-columns: subgrid;

/* <auto-track-list> values */
grid-template-columns: 200px repeat(auto-fill, 100px) 300px;
grid-template-columns:
  minmax(100px, max-content)
  repeat(auto-fill, 200px) 20%;
grid-template-columns:
  [line-name1] 100px [line-name2]
  repeat(auto-fit, [line-name3 line-name4] 300px)
  100px;
grid-template-columns:
  [line-name1 line-name2] 100px
  repeat(auto-fit, [line-name1] 300px) [line-name3];

/* Global values */
grid-template-columns: inherit;
grid-template-columns: initial;
grid-template-columns: revert;
grid-template-columns: revert-layer;
grid-template-columns: unset;
```

Values

`none`

Indicates that there is no explicit grid. Any columns will be implicitly generated and their size will be determined by the `grid-auto-columns` property.

`[line-name]`

A `<custom-ident>` specifying a name for the line in that location. The ident may be any valid string other than the reserved words `span` and `auto`. Lines may have multiple names separated by a space inside the square brackets, for example `[line-name-a line-name-b]`.

`<length>`

A non-negative length, giving the width of the column.

`<percentage>`

A non-negative `<percentage>` value relative to the inline size of the grid container. If the size of the grid container depends on the size of its tracks, the browser treats the percentage as `auto`. The browser may adjust the intrinsic size contributions of the track to the size of the grid container and may increase the final size of the track by the minimum amount that would result in honoring the percentage.

`<flex>`

Is a non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor.

When appearing outside a `minmax()` notation, it implies an automatic minimum (i.e., `minmax(auto, <flex>)`).

`max-content`

Is a keyword representing the largest maximal content contribution of the grid items occupying the grid track. For example, if the first element of the grid track contains the sentence "*Repetitio est mater studiorum*" and the second element contains the sentence "*Dum spiro, spero*", maximal content contribution will be defined by the size of the largest sentence among all of the grid elements - "*Repetitio est mater studiorum*".

`min-content`

Is a keyword representing the largest minimal content contribution of the grid items occupying the grid track. For example, if the first element of the grid track contains the sentence "*Repetitio est mater studiorum*" and the second element contains the sentence "*Dum spiro, spero*", minimal content contribution will be defined by the size of the largest word among all of the sentences in the grid elements - "*studiorum*".

`minmax(min, max)`

Is a functional notation that defines a size range greater than or equal to *min* and less than or equal to *max*. If *max* is smaller than *min*, then *max* is ignored and the function is treated as *min*. As a maximum, a `<flex>` value sets the track's flex factor. It is invalid as a minimum.

`auto`

As a maximum value, it represents the largest `max-content` size of the items in that track.

As a minimum value, it represents the largest minimum size of items in that track (specified by the `min-width`/`min-height` properties of the items). This often corresponds to the `min-content` size, but not always.

If used outside of `minmax()` notation, `auto` represents the range between the minimum and maximum values described above. In most cases, this behaves similarly to `minmax(min-content,max-content)`.

> **Note:** `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties. Therefore, by default, an `auto`-sized track will take up any remaining space in the grid container.

`fit-content( [ <length> | <percentage> ] )`

Represents the formula `max(minimum, min(limit, max-content))`, where *minimum* represents an `auto` minimum (which is often, but not always, equal to a `min-content` minimum), and *limit* is the track sizing function passed as an argument to fit-content(). This is essentially calculated as the smaller of `minmax(auto, max-content)` and `minmax(auto, limit)`.

`repeat( [ <positive-integer> | auto-fill | auto-fit ] , <track-list> )`

Represents a repeated fragment of the track list, allowing a large number of columns that exhibit a recurring pattern to be written in a more compact form.

`subgrid`

The `subgrid` value indicates that the grid will adopt the spanned portion of its parent grid in that axis. Rather than being specified explicitly, the sizes of the grid rows/columns will be taken from the parent grid's definition.

Formal definition

Initial value	none
Applies to	grid containers
Inherited	no
Percentages	refer to corresponding dimension of the content area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list

Formal syntax

```
grid-template-columns =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?

<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                             |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                             |
  minmax( <fixed-breadth> , <track-breadth> )       |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying grid column sizes

HTML

```
<div id="grid">
  <div id="areaA">A</div>
  <div id="areaB">B</div>
</div>
```

CSS

```
#grid {
  display: grid;
  width: 100%;
  grid-template-columns: 50px 1fr;
}

#areaA {
  background-color: lime;
}

#areaB {
  background-color: yellow;
}
```

Result

See also

- grid-template-rows
- grid-template-areas
- grid-template
- Basic concepts of grid layout: grid tracks
- Video: Defining a grid
- Subgrid

grid-template-rows CSS property

The **grid-template-rows** CSS property defines the line names and track sizing functions of the grid rows.

Try it

```
grid-template-rows: auto;
```

```
grid-template-rows: 40px 4em 40px;
```

```
grid-template-rows: 1fr 2fr 1fr;
```

```
grid-template-rows: 3ch auto minmax(10px, 60px);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-gap: 10px;
  width: 200px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Keyword value */
grid-template-rows: none;

/* <track-list> values */
grid-template-rows: 100px 1fr;
grid-template-rows: [line-name] 100px;
grid-template-rows: [line-name1] 100px [line-name2 line-name3];
grid-template-rows: minmax(100px, 1fr);
grid-template-rows: fit-content(40%);
grid-template-rows: repeat(3, 200px);
grid-template-rows: subgrid;

/* <auto-track-list> values */
grid-template-rows: 200px repeat(auto-fill, 100px) 300px;
grid-template-rows:
  minmax(100px, max-content)
  repeat(auto-fill, 200px) 20%;
grid-template-rows:
  [line-name1] 100px [line-name2]
```

```
  repeat(auto-fit, [line-name3 line-name4] 300px)
  100px;
grid-template-rows:
  [line-name1 line-name2] 100px
  repeat(auto-fit, [line-name1] 300px) [line-name3];

/* Global values */
grid-template-rows: inherit;
grid-template-rows: initial;
grid-template-rows: revert;
grid-template-rows: revert-layer;
grid-template-rows: unset;
```

This property may be specified as:

- either the keyword value `none`
- or a `<track-list>` value
- or an `<auto-track-list>` value.

Values

`none`

A keyword meaning that there is no explicit grid. Any rows will be implicitly generated and their size will be determined by the `grid-auto-rows` property.

`[line-name]`

A `<custom-ident>` specifying a name for the line in that location. The ident may be any valid string other than the reserved words `span` and `auto`. Lines may have multiple names separated by a space inside the square brackets, for example `[line-name-a line-name-b]`.

`<length>`

A non-negative length.

`<percentage>`

A non-negative `<percentage>` value, relative to the block size of the grid container. If the size of the grid container depends on the size of its tracks, the browser treats the percentage as `auto` for the purpose of calculating the intrinsic size of the grid container. The percentage is then resolved against the resulting grid container size for laying out the grid and its items. The browser may adjust the intrinsic size contributions of the track to the size of the grid container and may increase the final size of the track by the minimum amount that would result in honoring the percentage.

`<flex>`

A non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor. When appearing outside a `minmax()` notation, it implies an automatic minimum (i.e., `minmax(auto, <flex>)`).

`max-content`

A keyword representing the largest maximal content contribution of the grid items occupying the grid track.

`min-content`

A keyword representing the largest minimal content contribution of the grid items occupying the grid track.

`minmax(min, max)`

A functional notation that defines a size range, greater than or equal to *min*, and less than or equal to *max*. If *max* is smaller than *min*, then *max* is ignored and the function is treated as *min*. As a maximum, a `<flex>` value sets the track's flex factor. It is invalid as a minimum.

`auto`

As a maximum value, it represents the largest `max-content` size of the items in that track.

As a minimum value, it represents the largest minimum size of items in that track (specified by the `min-width`/`min-height` properties of the items). This often corresponds to the `min-content` size, but not always.

If used outside of `minmax()` notation, `auto` represents the range between the minimum and maximum values described above. In most cases, this behaves similarly to `minmax(min-content,max-content)`.

> **Note:** `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties. Therefore, by default, an `auto`-sized track will take up any remaining space in the grid container.

`fit-content( [ <length> | <percentage> ] )`

Represents the formula `min(max-content, max(auto, argument))`, which is calculated similar to `auto` (i.e., `minmax(auto, max-content)`), except that the track size is clamped at *argument* if it is greater than the `auto` minimum.

`repeat( [ <positive-integer> | auto-fill | auto-fit ] , <track-list> )`

Represents a repeated fragment of the track list, allowing a large number of rows that exhibit a recurring pattern to be written in a more compact form.

`subgrid`

Indicates that the grid will adopt the spanned portion of its parent grid in that axis. Rather than being specified explicitly, the sizes of the grid rows/columns will be taken from the parent grid's definition.

Formal definition

Initial value	none
Applies to	grid containers
Inherited	no
Percentages	refer to corresponding dimension of the content area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list

Formal syntax

```
grid-template-rows =
  none                    |
```

```
  <track-list>              |
  <auto-track-list>         |
  subgrid <line-name-list>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?

<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                           |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                           |
  minmax( <fixed-breadth> , <track-breadth> )       |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>              |
  min-content               |
  max-content               |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content               |
  max-content               |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying grid row sizes

HTML

```
<div id="grid">
  <div id="areaA">A</div>
  <div id="areaB">B</div>
</div>
```

CSS

```
#grid {
  display: grid;
  height: 100px;
  grid-template-rows: 30px 1fr;
}

#areaA {
  background-color: lime;
}

#areaB {
  background-color: yellow;
}
```

Result

See also

- `grid-template-columns`
- `grid-template-areas`
- `grid-template`
- Basic concepts of grid layout: grid tracks
- Video: Defining a grid
- Subgrid

hanging-punctuation CSS property

The **`hanging-punctuation`** CSS property specifies whether a punctuation mark should hang at the start or end of a line of text. Hanging punctuation may be placed outside the line box.

Syntax

```
/* Keyword values */
hanging-punctuation: none;
hanging-punctuation: first;
hanging-punctuation: last;
hanging-punctuation: allow-end;

/* Two keywords */
hanging-punctuation: first allow-end;
hanging-punctuation: first last;
hanging-punctuation: last allow-end;

/* Three keywords */
hanging-punctuation: first allow-end last;

/* Global values */
hanging-punctuation: inherit;
hanging-punctuation: initial;
hanging-punctuation: revert;
hanging-punctuation: revert-layer;
hanging-punctuation: unset;
```

The `hanging-punctuation` property may be specified with one, two, or three space-separated values.

Values

`none`

No character hangs.

`first`

An opening bracket or quote at the start of the first formatted line of an element hangs. This applies to:

- all characters in the Unicode categories Ps, Pf, Pi
- the quote marks `U+0027` APOSTROPHE (') and `U+0022` QUOTATION MARK (").

`last`

A closing bracket or quote at the end of the last formatted line of an element hangs. This applies to:

- all characters in the Unicode categories Pe, Pf, Pi
- the quote marks `U+0027` APOSTROPHE (') and `U+0022` QUOTATION MARK (").

`allow-end`

A stop or comma at the end of a line hangs if it does not otherwise fit prior to justification.

Stops and commas that are allowed to hang include:

- `U+002C`, COMMA
- `U+002E`, FULL STOP
- `U+060C`, ARABIC COMMA
- `U+06D4`, ARABIC FULL STOP
- `U+3001`, IDEOGRAPHIC COMMA
- `U+3002`, IDEOGRAPHIC FULL STOP
- `U+FF0C`, FULLWIDTH COMMA
- `U+FF0E`, FULLWIDTH FULL STOP
- `U+FE50`, SMALL COMMA
- `U+FE51`, SMALL IDEOGRAPHIC COMMA
- `U+FE52`, SMALL FULL STOP
- `U+FF61`, HALFWIDTH IDEOGRAPHIC FULL STOP
- `U+FF64`, HALFWIDTH IDEOGRAPHIC COMMA

User agents may include additional characters.

Formal definition

Initial value	none
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
hanging-punctuation =
  none                                  |
  [ first || [ force-end | allow-end ] || last ]
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting opening and closing quotes to hang

HTML

```
<p>
  «For a moment, nothing happened. Then, after a second or so, nothing continued
  to happen.»
</p>
```

```
<p class="hanging">
  «For a moment, nothing happened. Then, after a second or so, nothing continued
  to happen.»
</p>

<p class="hanging right">
  «For a moment, nothing happened. Then, after a second or so, nothing continued
  to happen.»
</p>
```

CSS

```
p {
  width: 15em;
  border: 1px solid #cccccc;
  font-size: 2rem;
  font-style: italic;
  margin: 1em;
}

p.hanging {
  hanging-punctuation: first last;
}

p.right {
  text-align: right;
}
```

Result

See also

- `text-indent`
- CSS Tricks: Hanging punctuation

height CSS property

The **`height`** CSS property specifies the height of an element. By default, the property defines the height of the content area. If `box-sizing` is set to `border-box`, however, it instead determines the height of the border area.

Try it

```
height: 150px;
```

```
height: 6em;
```

```
height: 75%;
```

```
height: auto;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the height.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: white;
}
```

The `min-height` and `max-height` properties override `height`.

> **Note:** As a geometric property, `height` also applies to the `<svg>`, `<rect>`, `<image>`, and `<foreignObject>` SVG elements, with `auto` resolving to `0` and percent values being relative to the SVG viewport height for `<rect>`. The CSS `height` property value overrides any SVG `height` attribute value set on the SVG element.

Syntax

```
/* <length> values */
height: 120px;
height: 10em;
height: 100vh;
height: anchor-size(height);
height: anchor-size(--my-anchor self-block, 250px);
height: clamp(200px, anchor-size(width));

/* <percentage> value */
height: 75%;

/* Keyword values */
height: max-content;
height: min-content;
height: fit-content;
height: fit-content(20em);
height: auto;
height: stretch;

/* Global values */
height: inherit;
height: initial;
height: revert;
height: revert-layer;
height: unset;
```

Values

`<length>`

Defines the height as a distance value.

`<percentage>`

Defines the height as a percentage of the containing block's height.

`auto`

The browser will calculate and select a height for the specified element.

`max-content`

The intrinsic preferred height.

`min-content`

The intrinsic minimum height.

`fit-content`

Use the available space, but not more than max-content, i.e., `min(max-content, max(min-content, stretch))`.

`fit-content(<length-percentage>)`

Uses the fit-content formula with the available space replaced by the specified argument, i.e., `min(max-content, max(min-content, <length-percentage>))`.

`stretch`

Sets the height of the element's margin box to the height of its containing block. It attempts to make the margin box fill the available space in the containing block, so in a way behaving similar to `100%` but applying the resulting size to the margin box rather than the box determined by box-sizing.

Accessibility

Ensure that elements set with a `height` aren't truncated and/or don't obscure other content when the page is zoomed to increase text size.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`auto`
Applies to	all elements but non-replaced inline elements, table columns, and column groups
Inherited	no
Percentages	The percentage is calculated with respect to the height of the generated box's containing block. If the height of the containing block is not specified explicitly (i.e., it depends on content height), and this element is not absolutely positioned, the value computes to `auto`. A percentage height on the root element is relative to the initial containing block.
Computed value	a percentage or `auto` or the absolute length
Animation type	a length, percentage or calc();

Formal syntax

```
height =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
```

```
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting height using pixels and percentages

HTML

```
<div id="taller">I'm 50 pixels tall.</div>
<div id="shorter">I'm 25 pixels tall.</div>
<div id="parent">
  <div id="child">I'm half the height of my parent.</div>
</div>
```

CSS

```
div {
  width: 250px;
  margin-bottom: 5px;
  border: 2px solid blue;
}

#taller {
  height: 50px;
}

#shorter {
  height: 25px;
}

#parent {
  height: 100px;
}

#child {
  height: 50%;
  width: 75%;
}
```

Result

Stretch height to fill the containing block

HTML

```
<div class="parent">
  <div class="child">text</div>
</div>

<div class="parent">
  <div class="child stretch">stretch</div>
</div>
```

CSS

```
@supports not (height: stretch) {
  body::before {
    content: "Your browser doesn't support the `stretch` value yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
.parent {
  height: 150px;
  margin: 1rem;
  border: solid;
}

.child {
  margin: 1rem;
  background: #00999999;
}

.stretch {
  height: stretch;
}
```

Result

See also

- `width`
- `box-sizing`
- `min-height`, `max-height`
- `block-size`, `inline-size`
- `anchor-size()`
- `clamp()`
- `minmax()`
- SVG `height` attribute
- Introduction to the CSS box model
- CSS box model module

hyphenate-character CSS property

The **`hyphenate-character`** CSS property sets the character (or string) used at the end of a line before a hyphenation break.

Both automatic and soft hyphens are displayed according to the specified hyphenate-character value.

Try it

```
hyphenate-character: auto;
```

```
hyphenate-character: "=";
```

```
hyphenate-character: "—";
```

```
<section id="default-example">
  <p id="example-element">An extraordinarily long English word!</p>
</section>
```

```
#example-element {
  border: 2px dashed #999999;
  font-size: 1.5rem;
  text-align: left;
  width: 7rem;
  hyphens: auto;
}
```

Syntax

```
hyphenate-character: <string>;
hyphenate-character: auto;
```

The value either sets the string to use instead of a hyphen, or indicates that the user agent should select an appropriate string based on the current typographic conventions (default).

Values

`<string>`

> The `<string>` to use at the end of the line before a hyphenation break. The user agent may truncate this value if too many characters are used.

`auto`

> The user-agent selects an appropriate string based on the content language's typographic conventions. This is the default property value, and only needs to be explicitly set in order to override a different inherited value.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
hyphenate-character =
  auto      |
  <string>
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

This example shows two identical blocks of text that have `hyphens` set to ensure that they break wherever needed, and on soft hyphen breaks (created using ``). The first block has the value of the hyphen changed to the equals symbol (=). The second block has no hyphenate-character set, which is equivalent to `hyphenate-character: auto` for user agents that support this property.

HTML

```
<dl>
  <dt><code>hyphenate-character: "="</code></dt>
  <dd id="string" lang="en">Superc&shy;alifragilisticexpialidocious</dd>
  <dt><code>hyphenate-character is not set</code></dt>
  <dd lang="en">Superc&shy;alifragilisticexpialidocious</dd>
</dl>
```

CSS

```
dd {
  width: 90px;
  border: 1px solid black;
  hyphens: auto;
}
```

```
dd#string {
  -webkit-hyphenate-character: "=";
  hyphenate-character: "=";
}
```

Result

See also

- Related CSS properties: `hyphens`, `overflow-wrap`.

hyphenate-limit-chars CSS property

The **`hyphenate-limit-chars`** CSS property specifies the minimum word length to allow hyphenation of words as well as the minimum number of characters before and after the hyphen.

Syntax

```
/* Numeric values */
hyphenate-limit-chars: 10 4 4;
hyphenate-limit-chars: 10 4;
hyphenate-limit-chars: 10;

/* Keyword values */
hyphenate-limit-chars: auto auto auto;
hyphenate-limit-chars: auto auto;
hyphenate-limit-chars: auto;

/* Mix of numeric and keyword values */
hyphenate-limit-chars: 10 auto 4;
hyphenate-limit-chars: 10 auto;
hyphenate-limit-chars: auto 3;

/* Global values */
hyphenate-limit-chars: inherit;
hyphenate-limit-chars: initial;
hyphenate-limit-chars: revert;
hyphenate-limit-chars: revert-layer;
hyphenate-limit-chars: unset;
```

Values

This property is specified as one to three values from the following list:

`<integer>`

Specifies either the number of characters for the minimum word length for hyphenation, the minimum number of characters before the hyphen, or the minimum number of characters after the hyphen.

`auto`

Specifies the user agent to choose appropriate values for the current layout. This is the default.

Description

The `hyphenate-limit-chars` property provides fine-grained control over hyphenation in text. This control enables you to avoid awkward hyphenations and set appropriate hyphenation for different languages, which, in turn, allows for better typography.

The property accepts one to three values, each an `<integer>` or the keyword `auto`, specifying the number of characters for the minimum word length for hyphenation, the minimum number of characters before the hyphen, and the minimum number of characters after the hyphen, in that order:

- If one `<integer>` is specified, it sets the word length for the minimum number of characters a word must have in order for it to be able to be hyphenated, while the minimum number of characters before and after the hyphen are set to `auto`.
- If two values are specified, the first value sets the minimum word length and the second sets the minimum number of characters both before and after the hyphen; the omitted third value defaults to the second value.
- If three values are specified, they set the minimum word length, the minimum number of characters before the hyphen, and the minimum number of characters after the hyphen, respectively.

For `auto`, the user agent will choose an appropriate value for the current layout. Unless the user agent can calculate a better value, the following default values will be used:

- Minimum word length to allow hyphenation: 5
- Minimum number of characters before the hyphen: 2
- Minimum number of characters after the hyphen: 2

Note that if a word is too short to meet the given constraints, it will not be hyphenated. For example, given a value like `hyphenate-limit-chars: auto 3 4`, words shorter than 7 characters will never be hyphenated, since it is impossible to have 3 characters before the hyphen and 4 characters after it.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
hyphenate-limit-chars =
  [ auto | <integer [0,∞]> ]{1,3}

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Text Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting hyphenation limits

In this example, we have four boxes each containing the same text. For the purpose of comparison, the first box shows the default hyphenation applied by the browser. The next three boxes demonstrate the result of constraining the browser's default behavior using different `hyphenate-limit-chars` values.

HTML

```
<div class="container">
  <p id="ex1">juxtaposition and acknowledgement</p>
  <p id="ex2">juxtaposition and acknowledgement</p>
  <p id="ex3">juxtaposition and acknowledgement</p>
  <p id="ex4">juxtaposition and acknowledgement</p>
</div>
```

CSS

```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
}

p {
  margin: 1rem;
  width: 120px;
  border: 2px dashed #999999;
  font-size: 1.5rem;
  hyphens: auto;
}

#ex2 {
  hyphenate-limit-chars: 14;
}

#ex3 {
  hyphenate-limit-chars: 5 9 2;
}

#ex4 {
  hyphenate-limit-chars: 5 2 7;
}
```

Result

In the first box, we don't set `hyphenate-limit-chars`, allowing the browser to apply its default algorithm. By default, the browser uses the values `5 2 2` unless it can find better values.

In the second box, we prevent the browser from hyphenating words unless they are at least 14 characters long by setting `hyphenate-limit-chars: 14`. As a result, "juxtaposition" is not hyphenated in the second box because it is only 13 characters long.

In the third box, we constrain the browser to include at least 9 characters before the hyphen by setting `hyphenate-limit-chars: 5 9 2`. The effect is that "acknowledgement" is now hyphenated as "acknowledge-ment" rather than the default version "acknowl-edgement", as shown in the first box.

Note that the browser does not have to include exactly 9 characters before the hyphen: as long as the constraints given in `hyphenate-limit-chars` are satisfied, the browser can break the word in the place it considers best. So in this case, for example, it chooses "acknowledge-ment" rather than the less readable "acknowled-gement".

In the fourth box, we make the browser include at least 7 characters after the hyphen by setting `hyphenate-limit-chars: 5 2 7`. The effect is that "juxtaposition" is hyphenated as "juxta-position" rather than the default "juxtaposi-tion".

See also

- hyphens
- CSS Text module

hyphens CSS property

The **`hyphens`** CSS property specifies how words should be hyphenated when text wraps across multiple lines.

Try it

```
hyphens: none;
```

```
hyphens: manual;
```

```
hyphens: auto;
```

```
<section id="default-example">
  <p id="example-element">An extraordinarily long English word!</p>
</section>
```

```
#example-element {
  border: 2px dashed #999999;
  font-size: 1.5rem;
  text-align: left;
  width: 7rem;
}
```

The example text "An extraordinarily long English word!" contains the hidden `` (soft hyphen) character: `An extraordinarily long English word!`. This character is used to indicate a potential place to insert a hyphen when `hyphens: manual;` is specified.

Syntax

```
/* Keyword values */
hyphens: none;
hyphens: manual;
hyphens: auto;

/* Global values */
```

```
hyphens: inherit;
hyphens: initial;
hyphens: revert;
hyphens: revert-layer;
hyphens: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

Words are not broken at line breaks, even if characters inside the words suggest line break points. Lines will only wrap at whitespace.

`manual`

Default value. Words are broken for line-wrapping only where characters inside the word suggest line break opportunities. See Suggesting line break opportunities below for details.

`auto`

The browser is free to automatically break words at appropriate hyphenation points, following whatever rules it chooses. However, suggested line break opportunities (see Suggesting line break opportunities below) will override automatic break point selection when present.

Description

The `hyphens` property can be used to define how words should be hyphenated when text wraps across multiple lines. It can prevent hyphenation entirely, hyphenate at manually-specified points within the text, or let the browser automatically insert hyphens where appropriate.

Hyphenation rules are language-specific. In HTML, the language is determined by the `lang` attribute. Browsers will only hyphenate if this attribute is present and the appropriate hyphenation dictionary is available. The `auto` setting's behavior depends on the language being properly tagged to select the appropriate hyphenation rules.

In XML, the `xml:lang` attribute must be used. The specification does not define rules for hyphenation in XML content, so the exact hyphenation may vary from browser to browser.

The `hyphenate-character` property can be used to specify an alternative hyphenation character to use at the end of the line, instead of the default language-specific hyphenation character.

If you apply `word-break: break-all` then no hyphens are shown, even if the word breaks at a hyphenation point. If you apply `text-wrap-mode: nowrap`, no wrapping occurs, so no hyphens will appear.

Suggesting line break opportunities

There are two Unicode characters used to manually specify potential line break points within text:

U+2010 (HYPHEN)

The "hard" hyphen character indicates a visible line break opportunity. Even if the line is not actually broken at that point, the hyphen is still rendered.

U+00AD (SHY)

An invisible, "**s**oft" **hy**phen. This character is not rendered visibly; instead, it marks a place where the browser should break the word if hyphenation is necessary. In HTML, use `` to insert a soft hyphen.

> **Note:** When the HTML `<wbr>` element leads to a line break, no hyphen is added.

Formal definition

Initial value	`manual`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
hyphens =
  none    |
  manual  |
  auto
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

This example demonstrates the three values of the `hyphens` property.

HTML

We include three `<dd>` elements containing the same text but with three different classes.

```
<dl>
  <dt><code>none</code>: no hyphen; overflow if needed</dt>
  <dd lang="en" class="none">An extreme&shy;ly long English word</dd>
  <dt>
    <code>manual</code>: hyphen only at &amp;hyphen; or &amp;shy; (if needed)
  </dt>
  <dd lang="en" class="manual">An extreme&shy;ly long English word</dd>
  <dt><code>auto</code>: hyphens where the algorithm decides (if needed)</dt>
  <dd lang="en" class="auto">An extreme&shy;ly long English word</dd>
</dl>
```

CSS

Each of the three classes is set to a different `hyphens` value.

```
dd {
  width: 55px;
  border: 1px solid black;
}
```

```
dd.none {
  hyphens: none;
}
dd.manual {
  hyphens: manual;
}
dd.auto {
  hyphens: auto;
}
```

Result

See also

- `content`
- `overflow-wrap` (formerly `word-wrap`)
- `word-break`
- Guide to wrapping and breaking text
- CSS text module

image-orientation CSS property

The **`image-orientation`** CSS property specifies a layout-independent correction to the orientation of an image.

Try it

```
image-orientation: none;
```

```
image-orientation: from-image;
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/hummingbird.jpg" />
</section>
```

```
#example-element {
  height: inherit;
}
```

Syntax

```
/* keyword values */
image-orientation: none;
image-orientation: from-image; /* Use EXIF data from the image */

/* Global values */
image-orientation: inherit;
image-orientation: initial;
image-orientation: revert;
image-orientation: revert-layer;
image-orientation: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

Does not apply any additional image rotation; the image is oriented as encoded or as other CSS property values dictate.

`from-image`

Default initial value. The EXIF information contained in the image is used to rotate the image appropriately.

> **Warning:** `image-orientation: none;` **does not** override the orientation of non-secure-origin images as encoded by their EXIF information, due to security concerns. Find out more from the CSS working group draft issue.

Description

This property is intended *only* to be used for the purpose of correcting the orientation of images which were shot with the camera rotated. It should *not* be used for arbitrary rotations. For any purpose other than correcting an image's orientation due to how it was shot or scanned, use a `transform` property with the `rotate` keyword to specify rotation. This includes any user-directed changes to the orientation of the image, or changes required for printing in portrait versus landscape orientation.

If used in conjunction with other CSS properties, such as a `<transform-function>`, any `image-orientation` rotation is applied before any other transformations.

Formal definition

Initial value	`from-image`
Applies to	all elements
Inherited	yes
Computed value	an `<angle>`, rounded to the next quarter turn from `0deg` and normalized, that is moduloing the value by `1turn`
Animation type	discrete

Formal syntax

```
image-orientation =
  from-image           |
  none                 |
  [ <angle> || flip ]
```

This syntax reflects the latest standard as per CSS Images Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Orienting image from image data

The following image has been rotated through 180 degrees, and the `image-orientation` property is used to correct its orientation based on the EXIF data in the image. By changing the `image-orientation` to `none` you can see the effect of the property.

CSS

```
#image {
  image-orientation: from-image; /* Can be changed in the live sample */
}
```

```
img {
  margin: 0.5rem 0;
}

label {
  font-family: monospace;
}
```

```
<img id="image" src="oriole.jpg" alt="Orientation taken from the image" />

<div>
  <input
    type="radio"
    id="from-image"
    name="orientation"
    value="from-image"
    checked />
  <label for="from-image">from-image</label>
</div>

<div>
  <input type="radio" id="none" name="orientation" value="none" />
  <label for="none">none</label>
</div>
```

```
document.addEventListener("change", (evt) => {
  document.getElementById("image").style.imageOrientation = evt.target.value;
});
```

Result

See also

- Other image-related CSS properties: `object-fit`, `object-position`, `image-rendering`, `image-resolution`.
- `transform` and `rotate`

image-rendering CSS property

The **`image-rendering`** CSS property sets an image scaling algorithm. The property applies to an element itself, to any images set in its other properties, and to its descendants.

The user agent will scale an image when the page author specifies dimensions other than its natural size. Scaling may also occur due to user interaction (zooming). For example, if the natural size of an image is `100×100px`, but its actual dimensions are `200×200px` (or `50×50px`), then the image will be upscaled (or downscaled) using the algorithm specified by `image-rendering`. This property has no effect on non-scaled images.

Try it

```
image-rendering: auto;
```

```
image-rendering: smooth;
```

```
image-rendering: crisp-edges;
```

```
image-rendering: pixelated;
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/lizard.png" />
</section>
```

```
#example-element {
  height: 480px;
  object-fit: cover;
}
```

Syntax

```
/* Keyword values */
image-rendering: auto;
image-rendering: smooth;
image-rendering: crisp-edges;
image-rendering: pixelated;

/* Global values */
image-rendering: inherit;
image-rendering: initial;
image-rendering: revert;
image-rendering: revert-layer;
image-rendering: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The scaling algorithm is UA dependent. Since version 1.9 (Firefox 3.0), Gecko uses *bilinear* resampling (high quality).

`smooth`

The image should be scaled with an algorithm that maximizes the appearance of the image. In particular, scaling algorithms that "smooth" colors are acceptable, such as bilinear interpolation. This is intended for images such as photos.

`crisp-edges`

The image is scaled with an algorithm such as "nearest neighbor" that preserves contrast and edges in the image. Generally intended for images such as pixel art or line drawings, no blurring or color smoothing occurs.

`pixelated`

The image is scaled with the "nearest neighbor" or similar algorithm to the nearest integer multiple of the original image size, then uses smooth interpolation to bring the image to the final desired size. This is intended to preserve a "pixelated" look without introducing scaling artifacts when the upscaled resolution isn't an integer multiple of the original.

> **Note:** The values `optimizeQuality` and `optimizeSpeed` present in an early draft (and coming from its SVG counterpart `image-rendering`) are defined as synonyms for the `smooth` and `pixelated` values respectively.

> **Note:** The CSS images module defines a `high-quality` value for the `image-rendering` property to provide a preference for higher-quality scaling, however, this is not supported in any browsers.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
image-rendering =
  auto          |
  smooth        |
  high-quality  |
  pixelated     |
  crisp-edges
```

This syntax reflects the latest standard as per CSS Images Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting image scaling algorithms

In this example, an image is repeated four times, with each having a different `image-rendering` value applied.

```
<div>
  <img
    class="auto"
    alt="A small photo of some white and yellow flower against a leafy green background. The image is about 33% smaller than the size it is being displayed at. This upscaling causes the image to appear blurry, with notable soft edges between objects."
    src="blumen.jpg" />
  <img
    class="smooth"
    alt="The same photo as the previous image, which is also being upscaled the same amount. Browsers that support the smooth value for the image-rendering property display the image with the maximize appearance."
    src="blumen.jpg" />
  <img
    class="pixelated"
    alt="The same photo as the previous image, which is also being upscaled the same amount. Browsers that support the pixelated value for the image-rendering property display the image as very pixelated. Individual pixels are clearly visible and edges appear much sharper."
    src="blumen.jpg" />
  <img
    class="crisp-edges"
    alt="The same photo as the previous images, which is also being upscaled the same amount. Browsers that support the crisp-edges value for the image-rendering property display the image as very pixelated. In these examples, there is virtually no perceivable difference between the pixelated and crisp-edges versions."
    src="blumen.jpg" />
</div>
```

```
img {
  height: 200px;
}
```

CSS

```
.auto {
  image-rendering: auto;
}

.smooth {
  image-rendering: smooth;
}

.pixelated {
  image-rendering: pixelated;
}

.crisp-edges {
  image-rendering: crisp-edges;
}
```

Result

See also

- `object-fit`
- `object-position`
- `image-orientation`
- `image-resolution`
- CSS images module
- SVG `image-rendering` attribute

image-resolution CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`image-resolution`** CSS property specifies the intrinsic resolution of all raster images used in or on the element. It affects content images such as replaced elements and generated content, and decorative images such as `background-image` images.

The image resolution is defined as the number of image pixels per unit length, e.g., pixels per inch. By default, CSS assumes a resolution of one image pixel per CSS px unit; however, the `image-resolution` property allows a different resolution to be specified.

Syntax

```
image-resolution: from-image;
image-resolution: 300dpi;
image-resolution: from-image 300dpi;
image-resolution: 300dpi snap;

/* Global values */
image-resolution: inherit;
image-resolution: initial;
image-resolution: revert;
image-resolution: revert-layer;
image-resolution: unset;
```

Values

This property is specified as one or more space-separated values from the following list:

`<resolution>`

Specifies the intrinsic resolution explicitly.

`from-image`

Uses the intrinsic resolution as specified by the image format. If the image does not specify its own resolution, the explicitly specified resolution is used (if given), else it defaults to `1dppx` (1 image pixel per CSS px unit).

`snap`

If the `snap` keyword is provided, the computed resolution is the specified resolution rounded to the nearest value that would map one image pixel to an integer number of device pixels. If the resolution is taken from the image, then the used intrinsic resolution is the image's native resolution similarly adjusted.

> **Note:** As vector formats such as SVG do not have an intrinsic resolution, this property has no effect on vector images.

Formal definition

Initial value	`1dppx`
Applies to	all elements
Inherited	yes
Computed value	as specified, except with <resolution> possibly altered by computed for 'snap' value
Animation type	discrete

Formal syntax

```
image-resolution =
  [ from-image || <resolution> ]  &&
  snap?
```

This syntax reflects the latest standard as per CSS Images Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a high dpi for print

When printing the document, use a higher resolution.

```
@media print {
  .my-image {
    image-resolution: 300dpi;
  }
}
```

Use image resolution with fallback

Uses the resolution from the image. If the image does not have a resolution, use 300dpi rather than the default 1dppx.

```
.my-image {
  image-resolution: from-image 300dpi;
}
```

Browser compatibility

Currently, no browsers support this feature.

See also

- Other image-related CSS properties: `object-fit`, `object-position`, `image-orientation`, `image-rendering`.
- Chromium bug: 1086473.

initial-letter CSS property

The **`initial-letter`** CSS property sets the size and sink for dropped, raised, and sunken initial letters. This property applies to `::first-letter` pseudo-elements and inline-level first children of block containers.

Syntax

```
/* Keyword value */
initial-letter: normal;

/* One value */
initial-letter: 3; /* 3 lines tall, baseline at line 3 */
initial-letter: 1.5; /* 1.5 lines tall, baseline at line 2 */

/* Two values */
initial-letter: 3 2; /* 3 lines tall, baseline at line 2 (raised 1 line) */
initial-letter: 3 1; /* 3 lines tall, baseline unchanged (raised 2 lines) */

/* Global values */
initial-letter: inherit;
initial-letter: initial;
initial-letter: revert;
initial-letter: revert-layer;
initial-letter: unset;
```

Values

This property is specified as `normal` or a `<number>` optionally followed by an `<integer>`:

`normal`

No special initial-letter effect. Text behaves as normal.

`<number>`

Defines the size of the initial letter, in terms of how many lines it occupies. Negative values are not allowed.

`<integer>`

Defines the number of lines the initial letter should sink when the size of it is given. Values must be greater than zero. If omitted, it duplicates the size value, floored to the nearest positive whole number.

Formal definition

Initial value	`normal`
Applies to	`::first-letter` pseudo-elements and inline-level first child of a block container
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
initial-letter =
  normal                          |
  <number [1,∞]> <integer [1,∞]>  |
  <number [1,∞]> && [ drop | raise ]?

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting initial letter size

HTML

```
<p class="normal">Initial letter is normal</p>
<p class="onefive">Initial letter occupies 1.5 lines</p>
<p class="three">Initial letter occupies 3 lines</p>
```

CSS

```
.normal::first-letter {
  -webkit-initial-letter: normal;
  initial-letter: normal;
}

.onefive::first-letter {
  -webkit-initial-letter: 1.5;
  initial-letter: 1.5;
}

.three::first-letter {
  -webkit-initial-letter: 3;
  initial-letter: 3;
}

p {
```

```
  outline: 1px dashed red;
}
```

Result

Setting the sink value

In this example, all the initial letters are the same size, but with different sink values.

HTML

```
<p class="four">Initial letter: Sink value = 4</p>
<p class="same">Initial letter: Sink value not declared (same as size)</p>
<p class="two">Initial letter: Sink value = 2</p>
<p class="one">Initial letter: Sink value = 1</p>
```

CSS

```
.four::first-letter {
  -webkit-initial-letter: 3 4;
  initial-letter: 3 4;
}

.same::first-letter {
  -webkit-initial-letter: 3;
  initial-letter: 3;
}

.two::first-letter {
  -webkit-initial-letter: 3 2;
  initial-letter: 3 2;
}

.one::first-letter {
  -webkit-initial-letter: 3 1;
  initial-letter: 3 1;
}

p {
  outline: 1px dashed red;
}
```

Result

See also

- `::first-letter`
- `:first-child`
- Drop caps in CSS via Oddbird (2017)

inline-size CSS property

The **`inline-size`** CSS property defines the size of an element's block along the inline axis. If the `writing-mode` is horizontal, it corresponds to the `width`; if the writing mode is vertical, it corresponds to the `height`. A related property is `block-size`, which defines the other dimension of the element.

Try it

```
inline-size: 150px;
writing-mode: horizontal-tb;
```

```
inline-size: 150px;
writing-mode: vertical-rl;
```

```
inline-size: auto;
writing-mode: horizontal-tb;
```

```
inline-size: auto;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the inline-size.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> values */
inline-size: 300px;
inline-size: 25em;
inline-size: anchor-size(width);
inline-size: anchor-size(--my-anchor inline);

/* <percentage> values */
inline-size: 75%;

/* Keyword values */
inline-size: max-content;
inline-size: min-content;
```

```
inline-size: fit-content;
inline-size: fit-content(20em);
inline-size: auto;

/* Global values */
inline-size: inherit;
inline-size: initial;
inline-size: revert;
inline-size: revert-layer;
inline-size: unset;
```

Values

The `inline-size` property takes the same values as the `width` and `height` properties.

Formal definition

Initial value	`auto`
Applies to	same as `width` and `height`
Inherited	no
Percentages	inline-size of containing block
Computed value	same as `width` and `height`
Animation type	a length, percentage or calc();

Formal syntax

```
inline-size =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline size in pixels

HTML

```
<p class="exampleText">Example text</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  inline-size: 110px;
}
```

Result

See also

- The mapped physical properties: `width` and `height`
- `writing-mode`

inset CSS property

The **`inset`** CSS shorthand property corresponds to the `top`, `right`, `bottom`, and/or `left` properties. It has the same multi-value syntax of the `margin` shorthand.

This inset properties, including `inset`, have no effect on non-positioned elements.

While part of the CSS logical properties and values module, it does not define *logical* offsets. It defines *physical* offsets, regardless of the element's writing mode, directionality, and text orientation.

Try it

```
inset: 1em;
```

```
inset: 5% 0;
```

```
inset: 2em 50px 20px;
```

```
inset: 10px 30% 20px 0;
```

```
inset: 0;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid #ad1457;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #07136c;
  border: 6px solid #ffa000;
  color: white;
  position: absolute;
  inset: 0;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `top`
- `right`
- `bottom`
- `left`

Syntax

```
/* <length> values */
inset: 10px; /* value applied to all edges */
inset: 4px 8px; /* top/bottom left/right */
inset: 5px 15px 10px; /* top left/right bottom */
inset: 2.4em 3em 3em 3em; /* top right bottom left */
inset: calc(anchor(50%) + 10px) anchor(self-start) auto auto;
inset: anchor-size(block) calc(anchor(50%) + 10px) auto
  calc(anchor-size(width) / 4);

/* <percentage>s of the width (left/right) or height (top/bottom) of the containing block */
inset: 10% 5% 5% 5%;

/* Keyword value */
inset: auto;

/* Global values */
inset: inherit;
inset: initial;
inset: revert;
inset: revert-layer;
inset: unset;
```

Values

The `inset` property takes the same values as the `top` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `top`: `auto` • `bottom`: `auto` • `left`: `auto` • `right`: `auto`
Applies to	positioned elements
Inherited	no
Percentages	relative to the containing block's size in the corresponding axis (e.g. width for left or right, height for top or bottom)
Computed value	as each of the properties of the shorthand: • `top`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto` • `bottom`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto` • `left`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto` • `right`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length, percentage or calc();

Formal syntax

```
inset =
  <'top'>{1,4}

<top> =
  auto                  |
  <length-percentage>   |
  <anchor()>            |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                  &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside        |
  outside       |
  top           |
  left          |
  right         |
  bottom        |
  start         |
  end           |
  self-start    |
  self-end      |
  <percentage>  |
  center

<anchor-size> =
  width         |
  height        |
  block         |
  inline        |
  self-block    |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting offsets for an element

HTML

```
<div>
  <span class="exampleText">Example text</span>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 150px;
  height: 120px;
  position: relative;
}

.exampleText {
  writing-mode: sideways-rl;
  position: absolute;
  inset: 20px 40px 30px 10px;
  background-color: #c8c800;
}
```

Result

See also

- `top`, `right`, `bottom`, and `left`
- `inset-block` and `inset-inline`
- `position`
- CSS positioned layout module
- CSS logical properties and values

inset-block CSS property

The **`inset-block`** CSS shorthand property defines the logical block start and end offsets of an element, which maps to physical offsets depending on the element's writing mode, directionality, and text orientation. It corresponds to the `top` and `bottom`, or `right` and `left` properties depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

This inset property has no effect on non-positioned elements.

Try it

```
inset-block: 10px 20px;
writing-mode: horizontal-tb;
```

```
inset-block: 20px 40px;
writing-mode: vertical-rl;
```

```
inset-block: 5% 20%;
writing-mode: horizontal-tb;
```

```
inset-block: 1rem auto;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid #ad1457;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #07136c;
  border: 6px solid #ffa000;
  color: white;
  position: absolute;
  inset: 0;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `inset-block-end`
- `inset-block-start`

Syntax

```
/* <length> values */
inset-block: 3px 10px;
inset-block: 2.4em 3em;
inset-block: 10px; /* value applied to start and end */
inset-block: auto anchor(start);
inset-block: 10em anchor-size(--my-anchor height, 10%);

/* <percentage>s of the width or height of the containing block */
inset-block: 10% 5%;

/* Keyword value */
inset-block: auto;

/* Global values */
inset-block: inherit;
inset-block: initial;
inset-block: revert;
inset-block: revert-layer;
inset-block: unset;
```

Values

The `inset-block` property takes the same values as the `top` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `inset-block-start`: `auto`

	• `inset-block-end`: `auto`
Applies to	positioned elements
Inherited	no
Percentages	logical-height of containing block
Computed value	as each of the properties of the shorthand: • `inset-block-start`: same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical • `inset-block-end`: same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical
Animation type	a length, percentage or calc();

Formal syntax

```
inset-block =
  <'top'>{1,2}

<top> =
  auto                  |
  <length-percentage>   |
  <anchor()>            |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                  &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside        |
  outside       |
  top           |
  left          |
  right         |
  bottom        |
  start         |
  end           |
  self-start    |
  self-end      |
  <percentage>  |
  center

<anchor-size> =
  width         |
  height        |
  block         |
  inline        |
  self-block    |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block start and end offsets

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  position: relative;
  inset-block: 20px 50px;
  background-color: #c8c800;
}
```

Result

See also

- The mapped physical properties: `top`, `right`, `bottom`, and `left`
- The mapped physical shortcut: `inset`
- The mapped inline shortcut: `inset-inline`
- `writing-mode`, `direction`, `text-orientation`

inset-block-end CSS property

The **`inset-block-end`** CSS property defines the logical block end offset of an element, which maps to a physical inset depending on the element's writing mode, directionality, and text orientation. It corresponds to the `top`, `right`, `bottom`, or `left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

This inset property has no effect on non-positioned elements.

Try it

```
writing-mode: horizontal-tb;
```

```
writing-mode: vertical-rl;
```

```
writing-mode: horizontal-tb;
direction: rtl;
```

```
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="example-container" id="example-element">
    <div id="abspos">I am absolutely positioned with inset-block-end: 20px</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
#example-element {
  border: 0.75em solid;
  padding: 0.75em;
  position: relative;
  width: 100%;
  min-height: 200px;
  unicode-bidi: bidi-override;
}

#abspos {
  background-color: yellow;
  color: black;
  border: 3px solid red;
  position: absolute;
  inset-block-end: 20px;
  inline-size: 140px;
  min-block-size: 200px;
}
```

Syntax

```
/* <length> values */
inset-block-end: 3px;
inset-block-end: 2.4em;
inset-block-end: calc(anchor(start) + 20px);
inset-block-end: anchor-size(--my-anchor width, 10%);

/* <percentage>s of the width or height of the containing block */
inset-block-end: 10%;

/* Keyword value */
inset-block-end: auto;

/* Global values */
inset-block-end: inherit;
inset-block-end: initial;
inset-block-end: revert;
inset-block-end: revert-layer;
inset-block-end: unset;
```

Values

The `inset-block-end` property takes the same values as the `top` property.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	logical-height of containing block
Computed value	same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical
Animation type	a length, percentage or calc();

Formal syntax

```
inset-block-end =
  auto                |
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block end offset

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  position: relative;
  inset-block-end: 20px;
  background-color: #c8c800;
}
```

Result

See also

- The properties which defines other insets: `inset-block-start`, `inset-inline-start`, and `inset-inline-end`
- The mapped physical properties: `top`, `right`, `bottom`, and `left`
- `writing-mode`, `direction`, `text-orientation`

inset-block-start CSS property

The **`inset-block-start`** CSS property defines the logical block start offset of an element, which maps to a physical inset depending on the element's writing mode, directionality, and text orientation. It corresponds to the `top`, `right`, `bottom`, or `left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

This inset property has no effect on non-positioned elements.

Try it

```
writing-mode: horizontal-tb;
```

```
writing-mode: vertical-rl;
```

```
writing-mode: horizontal-tb;
direction: rtl;
```

```
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="example-container" id="example-element">
    <div id="abspos">
      I am absolutely positioned with inset-block-start: 50px
    </div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
#example-element {
  border: 0.75em solid;
  padding: 0.75em;
  position: relative;
  width: 100%;
  min-height: 200px;
  unicode-bidi: bidi-override;
}

#abspos {
  background-color: yellow;
  color: black;
  border: 3px solid red;
  position: absolute;
  inset-block-start: 50px;
  inline-size: 140px;
}
```

Syntax

```
/* <length> values */
inset-block-start: 3px;
inset-block-start: 2.4em;
inset-block-start: anchor(end);
inset-block-start: calc(anchor-size(--my-anchor height, 70px) * 2);

/* <percentage>s of the width or height of the containing block */
inset-block-start: 10%;

/* Keyword value */
inset-block-start: auto;

/* Global values */
inset-block-start: inherit;
```

```
inset-block-start: initial;
inset-block-start: revert;
inset-block-start: revert-layer;
inset-block-start: unset;
```

Values

The `inset-block-start` property takes the same values as the `top` property.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	logical-height of containing block
Computed value	same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical
Animation type	a length, percentage or calc();

Formal syntax

```
inset-block-start =
  auto                |
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block start offset

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  position: relative;
  inset-block-start: 20px;
  background-color: #c8c800;
}
```

Result

See also

- The properties that define other insets: `inset-block-end`, `inset-inline-start`, and `inset-inline-end`
- The mapped physical properties: `top`, `right`, `bottom`, and `left`
- `writing-mode`, `direction`, `text-orientation`

inset-inline CSS property

The **`inset-inline`** CSS shorthand property defines the logical start and end offsets of an element in the inline direction, which maps to physical offsets depending on the element's writing mode, directionality, and text orientation. It corresponds to the `top` and `bottom`, or `right` and `left` properties depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

This inset property has no effect on non-positioned elements.

Try it

```
inset-inline: 5% 10%;
writing-mode: horizontal-tb;
```

```
inset-inline: 10px 40px;
writing-mode: vertical-rl;
```

```
inset-inline: 5% 10%;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid #ad1457;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #07136c;
  border: 6px solid #ffa000;
  color: white;
  position: absolute;
  inset: 0;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `inset-inline-end`
- `inset-inline-start`

Syntax

```
/* <length> values */
inset-inline: 3px 10px;
inset-inline: 2.4em 3em;
inset-inline: 10px; /* value applied to start and end */
inset-inline: auto calc(anchor(self-start) + 20px);
inset-inline: 400px anchor-size(--my-anchor height, 100px);

/* <percentage>s of the width or height of the containing block */
inset-inline: 10% 5%;

/* Keyword value */
inset-inline: auto;

/* Global values */
inset-inline: inherit;
inset-inline: initial;
inset-inline: revert;
inset-inline: revert-layer;
inset-inline: unset;
```

Values

The `inset-inline` property takes the same values as the `top` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `inset-inline-start`: `auto` • `inset-inline-end`: `auto`
Applies to	positioned elements
Inherited	no
Percentages	logical-width of containing block
Computed value	as each of the properties of the shorthand: • `inset-inline-start`: same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical • `inset-inline-end`: same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical
Animation type	a length, percentage or calc();

Formal syntax

```
inset-inline =
  <'top'>{1,2}

<top> =
  auto                |
  <length-percentage> |
  <anchor()>          |
  <anchor-size()>

<length-percentage> =
  <length>     |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside       |
  outside      |
  top          |
  left         |
  right        |
  bottom       |
  start        |
```

```
  end           |
  self-start    |
  self-end      |
  <percentage>  |
  center

<anchor-size> =
  width         |
  height        |
  block         |
  inline        |
  self-block    |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline start and end offsets

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  position: relative;
  inset-inline: 20px 50px;
  background-color: #c8c800;
}
```

Result

See also

- The mapped physical properties: `top`, `right`, `bottom`, and `left`
- The mapped physical shortcut: `inset`
- The mapped block shortcut: `inset-block`
- `writing-mode`, `direction`, `text-orientation`

inset-inline-end CSS property

The **`inset-inline-end`** CSS property defines the logical inline end inset of an element, which maps to a physical offset depending on the element's writing mode, directionality, and text orientation. It corresponds to the `top`, `right`, `bottom`, or `left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

This inset property has no effect on non-positioned elements.

Try it

```
writing-mode: horizontal-tb;
```

```
writing-mode: vertical-rl;
```

```
writing-mode: horizontal-tb;
direction: rtl;
```

```
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="example-container" id="example-element">
    <div id="abspos">
      I am absolutely positioned with inset-inline-end: 50px
    </div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
#example-element {
  border: 0.75em solid;
  padding: 0.75em;
  position: relative;
  width: 100%;
  min-height: 200px;
  unicode-bidi: bidi-override;
}

#abspos {
  background-color: yellow;
  color: black;
  border: 3px solid red;
  position: absolute;
  inset-inline-end: 50px;
  inline-size: 140px;
```

```
  min-block-size: 80px;
}
```

Syntax

```
/* <length> values */
inset-inline-end: 3px;
inset-inline-end: 2.4em;
inset-inline-end: calc(anchor(self-start) + 5px);
inset-inline-end: anchor-size(height);

/* <percentage>s of the width or height of the containing block */
inset-inline-end: 10%;

/* Keyword value */
inset-inline-end: auto;

/* Global values */
inset-inline-end: inherit;
inset-inline-end: initial;
inset-inline-end: revert;
inset-inline-end: revert-layer;
inset-inline-end: unset;
```

The shorthand for `inset-inline-start` and `inset-inline-end` is `inset-inline`.

Values

The `inset-inline-end` property takes the same values as the `top` property.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	logical-width of containing block
Computed value	same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical
Animation type	a length, percentage or calc();

Formal syntax

```
inset-inline-end =
  auto                |
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline end offset

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  position: relative;
  inset-inline-end: 20px;
  background-color: #c8c800;
}
```

Result

See also

- The properties which defines other insets: `inset-block-start`, `inset-block-end`, and `inset-inline-start`
- The mapped physical properties: `top`, `right`, `bottom`, and `left`
- `writing-mode`, `direction`, `text-orientation`

inset-inline-start CSS property

The **`inset-inline-start`** CSS property defines the logical inline start inset of an element, which maps to a physical offset depending on the element's writing mode, directionality, and text orientation. It corresponds to the `top`, `right`, `bottom`, or `left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

This inset property has no effect on non-positioned elements.

Try it

```
writing-mode: horizontal-tb;
```

```
writing-mode: vertical-rl;
```

```
writing-mode: horizontal-tb;
direction: rtl;
```

```
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="example-container" id="example-element">
    <div id="abspos">
      I am absolutely positioned with inset-inline-start: 50px
    </div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
#example-element {
  border: 0.75em solid;
  padding: 0.75em;
  position: relative;
  width: 100%;
  min-height: 200px;
  unicode-bidi: bidi-override;
}

#abspos {
  background-color: yellow;
  color: black;
  border: 3px solid red;
  position: absolute;
  inset-inline-start: 50px;
  inline-size: 140px;
  min-block-size: 80px;
}
```

Syntax

```
/* <length> values */
inset-inline-start: 3px;
inset-inline-start: 2.4em;
inset-inline-start: calc(anchor(--my-anchor 50%) + 10px);
inset-inline-start: anchor-size(width);

/* <percentage>s of the width or height of the containing block */
inset-inline-start: 10%;

/* Keyword value */
inset-inline-start: auto;

/* Global values */
inset-inline-start: inherit;
inset-inline-start: initial;
inset-inline-start: revert;
inset-inline-start: revert-layer;
inset-inline-start: unset;
```

The shorthand for `inset-inline-start` and `inset-inline-end` is `inset-inline`.

Values

The `inset-inline-start` property takes the same values as the `top` property.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	logical-width of containing block
Computed value	same as box offsets: `top`, `right`, `bottom`, `left` properties except that directions are logical
Animation type	a length, percentage or calc();

Formal syntax

```
inset-inline-start =
  auto                |
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline start offset

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  position: relative;
  inset-inline-start: 20px;
  background-color: #c8c800;
}
```

Result

See also

- The properties which defines other insets: `inset-block-start`, `inset-block-end`, and `inset-inline-end`
- The mapped physical properties: `top`, `right`, `bottom`, and `left`
- `writing-mode`, `direction`, `text-orientation`

interactivity CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`interactivity`** CSS property specifies whether an element and its descendant nodes are set to be inert.

Syntax

```
/* Keyword values */
interactivity: auto;
interactivity: inert;

/* Global values */
interactivity: inherit;
interactivity: initial;
interactivity: revert;
interactivity: revert-layer;
interactivity: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Selected elements are in their default state in terms of inertness. This usually means that they are interactive, but this is not always the case. This is the default value.

`inert`

Selected elements and their descendants are inert.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
interactivity =
  auto   |
  inert
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Description

The `interactivity` property can be used to set whether an element and its descendants are inert. See the HTML `inert` attribute reference page for a detailed description of the inert state.

A typical use case for `interactivity: inert` is in paginated content, like carousels, when you only want the currently-visible page's content and controls to be interacted with. In such cases, unexpectedly focusing on an off-screen link or button could spoil the experience.

If an element's inert state is specified both by HTML (the `inert` attribute, or an automatic browser setting) and CSS (the `interactive` property) at the same time, the CSS will have no effect — it cannot override the HTML's inertness.

For example, the following HTML element will be inert:

```
<button inert>You can't press me</button>
```

Setting `interactive: auto` on it will have no effect.

Default inertness

Most elements are interactive by default, but this is not always the case:

- An element's ancestor may be set to an inert state, via the `interactive` property or the `inert` attribute.
- While a modal `<dialog>` is displayed, the rest of the page is set to an inert state automatically.

Examples

Basic <code>interactivity</code> usage

In this example we have two `<input>` elements. The second one has `interactivity: inert` set on it via a class, and therefore is not focusable or editable in supporting browsers.

```
<p>
  <label>
    This input is interactive:
    <input type="text" name="one" value="editable" />
  </label>
</p>
<p>
  <label>
    This input is not interactive:
    <input type="text" name="two" value="Not editable" class="inert" />
  </label>
</p>
```

```
.inert {
  interactivity: inert;
  background-color: lightpink;
}
```

Result

The output looks like this:

Exploring the effects of inertness

In this example we explore the effects of the `interactivity` property.

HTML

The markup features two `<p>` elements, each of which contain a link. The second paragraph also has a class of `inert` set on it, and a child `<span>` element with `contenteditable` set on it to make it editable.

```
<p>
  This paragraph is not
  <a
    href="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/inert"
    >inert</a
  >. You should be able to select the text content, search for it using
  in-browser search features, and focus and click the link. There is a
  <code>click</code> event handler set on the paragraph that changes the border
  color for a second when it is clicked anywhere.
  <span contenteditable>This sentence has <code>contenteditable</code> set on
  it, so it is editable</span>.
</p>

<p class="inert">
  This paragraph is
  <a
    href="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/inert"
    >inert</a
  >. You won't be able to select the text content, search for it using
  in-browser search features, focus and click the link, or issue
  <code>click</code> events on it (the border color won't change when it is
  clicked).
  <span contenteditable
    >This sentence has <code>contenteditable</code> set on it, but it is not
    editable because it is inert</span
  >.
</p>
```

CSS

We set the `interactivity` property on the second paragraph to a value of `inert`, making it inert. This means that you should be able to edit the `contenteditable` text in the first paragraph, but not in the second, and you shouldn't be able to search for text, select text, or interact with the link within the second paragraph.

```
.inert {
  interactivity: inert;
}

[contenteditable] {
  outline: 1px dashed lightblue;
}

.borderChanged {
  border-color: orange;
}
```

```
body {
  font: 1.2em / 1.5 system-ui;
}

p {
  border: 5px solid black;
  padding: 10px;
  width: 90%;
  margin: 20px auto;
}
```

JavaScript

We set an event handler on each paragraph that toggles a class name when clicked, adding the classname then removing the class after two seconds.

```
const paras = document.querySelectorAll("p");

function tempBorderChange(e) {
  const targetPara = e.currentTarget;
  targetPara.classList.add("borderChanged");
  setTimeout(() => {
    targetPara.classList.remove("borderChanged");
  }, 2000);
}

for (para of paras) {
  para.addEventListener("click", tempBorderChange);
}
```

Result

Note how the second paragraph is inert; it therefore does not behave like the first paragraph. For example, the link cannot be clicked or focused, the text cannot be selected or searched, the `contenteditable` `<span>` is not editable, and `click` events do not register on it.

Setting off-screen elements to inert using a view timeline

This example shows horizontally-scrolling paginated content, with each page snapped to using CSS Scroll Snap, and the inertness controlled via a scroll-driven animation that uses a view progress timeline. Content displayed in the scroll container is interactive; it becomes inert when it moves out into the overflowing content.

HTML

The HTML consists of a top-level heading and an unordered list with four list items, each containing the content for a separate page.

```
<h1>Pagination interactivity demo</h1>
<ul>
  <li>
    <h2>Page 1</h2>
    <p>This is the first page of content.</p>
    <p><a href="#">A demo link</a>.</p>
    <p><button>Press me</button></p>
  </li>
  <li>
    <h2>Page 2</h2>
    <p>This is the second page of content.</p>
    <p><a href="#">A demo link</a>.</p>
    <p><button>Press me</button></p>
  </li>
  <li>
    <h2>Page 3</h2>
    <p>This is the third page of content.</p>
    <p><a href="#">A demo link</a>.</p>
    <p><button>Press me</button></p>
  </li>
  <li>
    <h2>Page 4</h2>
    <p>This is the fourth page of content.</p>
    <p><a href="#">A demo link</a>.</p>
    <p><button>Press me</button></p>
  </li>
</ul>
```

CSS

A `width` of `100vw` is set on the unordered list to make it as wide as the viewport. We add a fixed `height`, some `padding`, and an `overflow-x` value of `scroll` so overflowing content will scroll. Its child list items are laid out horizontally with `display: flex`. This flex container is given a `scroll-snap-type` value of `x mandatory` to make it into a scroll snap container. The `x` keyword causes the container's snap targets to be snapped horizontally. The `mandatory` keyword means that the container will always snap to a snap target at the end of a scrolling action.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  text-align: center;
  margin: 0;
}

button {
  background-color: white;
}
```

```
ul {
  width: 100vw;
  height: 250px;
  padding: 1vw;
  overflow-x: scroll;
  display: flex;
  gap: 1vw;
  scroll-snap-type: x mandatory;
}
```

Each list item has the following styles applied:

- A `flex` value of `0 0 98vw`, forcing each item to be as big as the scroll container minus the `gap` set on the list (see the `gap` declaration in the `ul` rule shown earlier). This also has the effect of centering each page inside the scroll container.
- A `scroll-snap-align` value of `center`, to cause the scroll container to snap to the center of each snap target.

- A `view-timeline` value of `--inner-change inline`, to declare the element as the subject of the `--inner-change` view progress timeline, and set that timeline to progress in the inline direction as it moves through its ancestor scroll container.
- An `animation-timeline` value with the same name as the `view-timeline-name`, as defined in the `view-timeline` shorthand, which means that the named view progress timeline will be used to control the progress of animations applied to the element.
- An `animation-name` and `animation-fill-mode` defining the animation applied to this element and its fill mode. The `both` value is required because you want the starting animation state to apply to the element before the animation starts, and the end animation state to apply to the element after the animation finishes. If the animation isn't persisted, the `interactivity: inert` declaration applied via the animation won't apply to list items when they are outside the scroll container.

```
li {
  list-style-type: none;
  background-color: #eeeeee;
  border: 1px solid #dddddd;
  padding: 20px;

  flex: 0 0 98vw;

  scroll-snap-align: center;

  view-timeline: --inner-change inline;
  animation-timeline: --inner-change;
  animation-name: inert-change;
  animation-fill-mode: both;
}
```

Finally, the animation `@keyframes` are defined. `interactivity: inert` is set at positions `entry 0%` and `exit 100%` of the view timeline. Combined with the `animation-fill-mode: both` value, this means that the list items will be inert before the start and after the end of the view timeline, that is, when they are outside the scroll container. Between positions `entry 1%` and `exit 99%`, `interactivity: auto` is set on the list items, meaning they can be interacted with normally when they are inside the scroll container.

```
@keyframes inert-change {
  entry 0%,
  exit 100% {
    interactivity: inert;
  }

  entry 1%,
  exit 99% {
    interactivity: auto;
  }
}
```

See the `animation-range` reference page for an explanation of the position values.

Result

Scroll the unordered list horizontally to see the pagination effect — each page snaps into view. Try tabbing between the links and the buttons; you'll find that only the ones on-screen are interactive and can be tabbed to.

Accessibility concerns

Consider accessibility carefully when making elements inert. By default, there is no visual way to tell whether or not an element or its subtree is inert. As a web developer, it is your responsibility to clearly indicate the content parts that are active and those that are inert.

While providing visual and non-visual cues about content inertness, also remember that the visual viewport may contain only sections of content. Users may be zoomed in to a small section of content, or users may not be able to view the content at all. Inert sections not being obviously inert can lead to frustration and bad user experience.

See also

- HTML `inert` attribute
- `HTMLElement.inert`

interest-delay CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`interest-delay`** CSS property specifies the delay between the user showing interest in an interest invoker element and the `interest` event firing, and the delay between the user losing interest and the `loseinterest` event firing.

Constituent properties

This property is a shorthand for the following properties:

- `interest-delay-start`
- `interest-delay-end`

Syntax

```
/* Single value */
interest-delay: normal;
interest-delay: 2s;
interest-delay: 250ms;

/* Two values */
interest-delay: 1s normal;
interest-delay: 0s 500ms;

/* Global values */
interest-delay: inherit;
interest-delay: initial;
interest-delay: revert;
interest-delay: revert-layer;
interest-delay: unset;
```

Values

The `interest-delay` property accepts one or two values. The first value sets the delay before interest is shown (`interest-delay-start`); the second value, if provided, sets the delay before interest is lost (`interest-delay-end`). Each value can be either the keyword `normal` or a `<time>` value:

`normal`

Sets the delay to the browser's default delay. This is the initial value.

`<time>`

Sets the delay to a specific duration. The value must be positive, otherwise the property becomes invalid.

Description

A control element such as an `<a>` or `<button>` can be set as an interest invoker by giving it the `interestfor` attribute. Its value should be the `id` of a target element. When this relationship is established, the target element is affected when the user "shows interest" in the invoker (for example, by hovering or focusing it). A common use case is to display a popover on hover or focus. When the user "loses interest", the effect stops.

When the user shows or loses interest, the associated effect doesn't start or stop immediately — the browser adds a short delay (which may vary by browser). This prevents preview popovers, for example, from appearing immediately when the user hovers a link, which could be annoying and distracting on a link-heavy page.

The `interest-delay` property allows you to customize these delays. You can use `interest-delay` to specify the delay before the interest effect starts (specified by the `interest-delay-start` property) and the delay before the interest effect ends (specified by the `interest-delay-end` property) in a single declaration.

The `interest-delay` property can take one or two values. These values can be the keyword `normal`, which sets the default browser delay, or a `<time>` value, which sets a custom delay. If a single value is specified, it applies to both `interest-delay-start` and `interest-delay-end`. If two values are specified, the first value sets `interest-delay-start`, and the second value sets `interest-delay-end`.

Formal definition

Initial value	as each of the properties of the shorthand: • `interest-delay-start`: `normal` • `interest-delay-end`: `normal`
Applies to	all elements
Inherited	yes
Computed value	as each of the properties of the shorthand: • `interest-delay-start`: `normal` or a computed time • `interest-delay-end`: `normal` or a computed time
Animation type	as each of the properties of the shorthand: • `interest-delay-start`: by computed value type • `interest-delay-end`: by computed value type

Formal syntax

```
interest-delay =
  <'interest-delay-start'>{1,2}

<interest-delay-start> =
  normal  |
  <time>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a basic <code>interest-delay</code> effect

In this example, we demonstrate how `interest-delay` affects interest invoker behavior.

HTML

The markup includes a `<button>`, a `<p>`, and an `<input>` of type `checkbox`. We specify the `<button>` as an interest invoker by giving it the `interestfor` attribute whose value matches the `id` of the `<p>` element. This makes the paragraph the target element. The paragraph is turned into a popover by giving it the `popover` attribute, which hides it initially.

```
<button interestfor="mypopover">Button</button>
<p id="mypopover" popover>Hover tooltip</p>
<form>
  <input type="checkbox" id="apply-delay" />
  <label for="apply-delay">
    Apply an <code>interest-delay</code> of <code>1s 2s</code>
  </label>
</form>
```

CSS

In the CSS, we define a `.delay` rule that applies an `interest-delay` value of `1s 2s` to any interest invoker the `delay` class is set on. We will set this class on the `<button>` when the checkbox is checked using JavaScript.

```
.delay {
  interest-delay: 1s 2s;
}
```

JavaScript

In our script, we get references to the `<button>` and the checkbox, then create an event listener that toggles the `delay` class on the `<button>` whenever the checkbox value changes (when it is checked or unchecked).

```
const btn = document.querySelector("button");
const checkbox = document.querySelector("input");
checkbox.addEventListener("change", () => {
```

```
  btn.classList.toggle("delay");
});
```

Result

This renders as follows:

Try showing interest in the button (for example, by hovering or focusing it) and then losing interest to observe the popover showing and hiding. By default, the popover shows and hides after a very short delay.

Now check the checkbox and try the same actions again. This time, the popover should appear after a delay of `1s` when interest is shown, and hide after a delay of `2s` when interest is lost.

See also

- `interest-delay-start`, `interest-delay-end`
- Popover API
- Using interest invokers
- CSS basic user interface module

interest-delay-end CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`interest-delay-end`** CSS property specifies the delay between the user losing interest in an interest invoker element and the `loseinterest` event firing.

The `interest-delay-end` and `interest-delay-start` properties can both be set using the `interest-delay` shorthand.

Syntax

```
/* Keyword or custom delay */
interest-delay-end: normal;
interest-delay-end: 2s;
interest-delay-end: 250ms;

/* Global values */
interest-delay-end: inherit;
interest-delay-end: initial;
interest-delay-end: revert;
interest-delay-end: revert-layer;
interest-delay-end: unset;
```

Values

This property is specified as one `<time>` value or the keyword `normal`:

`normal`

Sets the delay to the browser's default delay. This is the initial value.

`<time>`

Sets the delay to a specific duration. The value must be positive, otherwise the property becomes invalid.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	yes
Computed value	`normal` or a computed time
Animation type	by computed value type

Formal syntax

```
interest-delay-end =
  normal  |
  <time>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a basic <code>interest-delay-end</code> effect

In this example, we demonstrate how `interest-delay-end` affects interest invoker behavior.

HTML

The markup includes a `<button>`, a `<p>`, and an `<input>` of type `checkbox`. We specify the `<button>` as an interest invoker by giving it the `interestfor` attribute, whose value matches the `id` of the `<p>` element. This makes the paragraph the target element. The paragraph is turned into a popover by giving it the `popover` attribute, which hides it initially.

```
<button interestfor="mypopover">Button</button>
<p id="mypopover" popover>Hover tooltip</p>
<form>
  <input type="checkbox" id="apply-delay" />
  <label for="apply-delay">
    Apply an <code>interest-delay-end</code> of <code>2s</code>
  </label>
</form>
```

CSS

In the CSS, we specify a rule with a `.delay` selector that applies an `interest-delay-end` value of `2s` to any interest invoker the `delay` class is set on. We will set this on the `<button>` when the checkbox is checked using JavaScript.

```
.delay {
  interest-delay-end: 2s;
}
```

JavaScript

In our script, we get references to the `<button>` and the checkbox, then create an event listener that toggles the `delay` class on the `<button>` whenever the checkbox value changes (when it is checked or unchecked).

```
const btn = document.querySelector("button");
const checkbox = document.querySelector("input");
checkbox.addEventListener("change", () => {
  btn.classList.toggle("delay");
});
```

Result

This renders as follows:

Try showing interest in the button (for example, by hovering or focusing it) and then losing interest to observe the popover showing and hiding. By default, the popover shows and hides after a very short delay.

Now check the checkbox and try the same actions again. This time, the delay between showing interest and the popover appearing is unaffected, but the delay between losing interest and the popover disappearing should be increased to `2s`.

See also

- `interest-delay-start`, `interest-delay`
- Popover API
- Using interest invokers
- CSS basic user interface module

interest-delay-start CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`interest-delay-start`** CSS property specifies the delay between the user showing interest in an interest invoker element and the `interest` event firing.

The `interest-delay-start` and `interest-delay-end` properties can both be set using the `interest-delay` shorthand.

Syntax

```
/* Keyword or custom delay */
interest-delay-start: normal;
interest-delay-start: 2s;
interest-delay-start: 250ms;

/* Global values */
interest-delay-start: inherit;
interest-delay-start: initial;
interest-delay-start: revert;
interest-delay-start: revert-layer;
interest-delay-start: unset;
```

Values

This property is specified as one `<time>` value or the keyword `normal`:

`normal`

Sets the delay to the browser's default delay. This is the initial value.

`<time>`

Sets the delay to a specific duration. The value must be positive, otherwise the property becomes invalid.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	yes
Computed value	`normal` or a computed time
Animation type	by computed value type

Formal syntax

```
interest-delay-start =
  normal  |
  <time>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a basic <code>interest-delay-start</code> effect

In this example, we demonstrate how `interest-delay-start` affects interest invoker behavior.

HTML

The markup includes a `<button>`, a `<p>`, and an `<input>` of type `checkbox`. We specify the `<button>` as an interest invoker by giving it the `interestfor` attribute, whose value matches the `id` of the `<p>` element. This makes the paragraph the target element. The paragraph is turned into a popover by giving it the `popover` attribute, which hides it initially.

```
<button interestfor="mypopover">Button</button>
<p id="mypopover" popover>Hover tooltip</p>
```

```
<form>
  <input type="checkbox" id="apply-delay" />
  <label for="apply-delay">
    Apply an <code>interest-delay-start</code> of <code>2s</code>
  </label>
</form>
```

CSS

In the CSS, we specify a rule with a `.delay` selector that applies an `interest-delay-start` value of `2s` to any interest invoker the `delay` class is set on. We will set this on the `<button>` when the checkbox is checked using JavaScript.

```
.delay {
  interest-delay-start: 2s;
}
```

JavaScript

In our script, we get references to the `<button>` and the checkbox, then create an event listener that toggles the `delay` class on the `<button>` whenever the checkbox value changes (when it is checked or unchecked).

```
const btn = document.querySelector("button");
const checkbox = document.querySelector("input");
checkbox.addEventListener("change", () => {
  btn.classList.toggle("delay");
});
```

Result

This renders as follows:

Try showing interest in the button (for example, by hovering or focusing it) and then losing interest to observe the popover showing and hiding. By default, the popover shows and hides after a very short delay.

Now check the checkbox and try the same actions again. This time, the popover should appear after a delay of `2s` when interest is shown. The delay after interest is lost should be unaffected.

Removing <code>interest-delay-start</code> after interest has been shown

In this example, we show how to remove the `interest-delay-start` from multiple interest invoker elements after interest has been shown on one of them.

This is a useful technique. Having a popover appear as soon as interest is shown on any invoker would create a distracting and annoying user experience, which is why browsers add a small delay by default (see the `interest-delay` description for more details). However, once users have shown interest in an invoker, it is convenient to allow them to move between other invokers quickly without delay.

HTML

The markup includes three `<button>` elements wrapped in a paragraph with a `container` class, and another paragraph that has been turned into a popover using the `popover` attribute. All three buttons are set up as interest invokers and reference the popover as their target using the `interestfor` attribute.

```
<p class="container">
  <button interestfor="mypopover">Button 1</button>
  <button interestfor="mypopover">Button 2</button>
  <button interestfor="mypopover">Button 3</button>
</p>
<p id="mypopover" popover>Hover tooltip</p>
```

CSS

In the CSS, we apply an `interest-delay-start` value of `1s` to the buttons, then position the popover below whatever button is having interest shown on it by giving it a `position-area` value of `bottom` (see Popover anchoring positioning for more information).

```
button {
  interest-delay-start: 1s;
}

#mypopover {
  position-area: bottom;
}
```

Finally, we combine the `:interest-source` pseudo-class with the `:has()` pseudo-class to apply `interest-delay-start: 0s` to all buttons inside the paragraph, but only when the paragraph contains a button on which interest has been shown (that is, matched by `button:interest-source`).

```
.container:has(button:interest-source) button {
  interest-delay-start: 0s;
}
```

Result

This renders as follows:

Try showing interest in any button and notice how, when you then immediately show interest in another button, the popover appears without delay. If you stop showing interest in the buttons and then start again, the initial delay will return.

See also

- `interest-delay-end`, `interest-delay`
- Popover API
- Using interest invokers
- CSS basic user interface module

interpolate-size CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`interpolate-size`** CSS property allows you to enable animations and transitions between a `<length-percentage>` value and an intrinsic size, `fit-content`, or `max-content`.

This property is typically used to animate the `width` and/or `height` of a container between a `<length-percentage>` and the full size of its content (i.e., between "closed" and "open" or "hide" and "reveal" states) when animating a non-box-model CSS property, such as `transform`, is not a viable solution.

> **Note:** The behavior opted-into by `interpolate-size` cannot be enabled by default across the web because many sites in the wild use stylesheets that assume intrinsic size values cannot be animated. Enabling it by default would cause several backwards-compatibility issues (see relevant CSS WG discussion).

Syntax

```
/* Keyword values */
interpolate-size: allow-keywords;
interpolate-size: numeric-only;

/* Global values */
interpolate-size: inherit;
interpolate-size: initial;
interpolate-size: revert;
interpolate-size: revert-layer;
interpolate-size: unset;
```

Values

This property is specified as one of the following keyword values:

`allow-keywords`

Enables interpolation between a `<length-percentage>` value and an intrinsic size value, to allow animation between the two.

`numeric-only`

The default behavior — intrinsic size values cannot be interpolated.

Description

Setting `interpolate-size: allow-keywords` enables interpolation between a `<length-percentage>` value and an intrinsic size value. Note that it does not enable animating between two intrinsic size values. One end of the animation must be a `<length-percentage>`.

The `interpolate-size` value is inherited, so animating to (or from) an intrinsic size value can be enabled for an entire document by setting it on the document root:

```
:root {
  interpolate-size: allow-keywords;
}
```

If you want to limit the scope, you can set it on the relevant container element. The following enables interpolating intrinsic sizes only for `<main>` and its descendants:

```
main {
  interpolate-size: allow-keywords;
}
```

> **Note:** The `calc-size()` function's return values can also be interpolated. In effect, including `calc-size()` in a property value automatically applies `interpolate-size: allow-keywords` to the selection. However, because `interpolate-size` is inherited as explained above, it is the preferred solution for enabling intrinsic size animations in most cases. You should only use `calc-size()` to enable intrinsic size animations if they also require calculations.

Values that can be interpolated

The following intrinsic values can currently be opted-in to animations:

- `auto`
- `min-content`
- `max-content`
- `fit-content`
- `content` (for containers sized using `flex-basis`).

Formal definition

Initial value	`numeric-only`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
interpolate-size =
  numeric-only    |
  allow-keywords
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>interpolate-size</code> usage

This example demonstrates how to set `interpolate-size: allow-keywords` on a document to enable animations involving an intrinsic size. The demo features a character badge/"name tag", which can be hovered or focused to reveal information about the character. The reveal is handled by a `height` transition between a set length and `max-content`.

HTML

The HTML contains a single `<section>` element with `tabindex="0"` set on it so it can receive keyboard focus. The `<section>` contains `<header>` and `<main>` elements, each with their own child content.

```
<section tabindex="0">
  <header>
    <h2>Tanuki</h2>
```

```
  </header>
  <main>
    <p>Tanuki is the silent phantom of MDN.</p>
    <ul>
      <li><strong>Height</strong>: 3.03m</li>
      <li><strong>Weight</strong>: 160kg</li>
      <li><strong>Tech Fu</strong>: 7</li>
      <li><strong>Bad Jokes</strong>: 9</li>
    </ul>
  </main>
</section>
```

CSS

```
* {
  box-sizing: border-box;
}

section {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 175px;
  border-radius: 5px;
  background: #eeeeee;
  box-shadow:
    inset 1px 1px 4px rgb(255 255 255 / 0.5),
    inset -1px -1px 4px rgb(0 0 0 / 0.5);
}

header {
  padding: 0.7rem;
  border-bottom: 2px solid #cccccc;
}

main {
  padding: 10px;
}

h2 {
  margin: 0;
  font-weight: normal;
  font-size: 1.1rem;
  text-align: center;
  letter-spacing: 1px;
}

p,
li {
  font-size: 0.8rem;
  line-height: 1.5;
}

p {
  margin-top: 0;
}
```

In the CSS, we first set `interpolate-size: allow-keywords` on the `:root`, to enable it for the whole document.

```
:root {
  interpolate-size: allow-keywords;
}
```

We then set the `<section>`'s `height` to `2.5rem` and `overflow` to `hidden` so only the `<header>` is shown by default, then specify a `transition` that animates the `<section>` `height` over 1 second during state change. Finally, we set the `<section>` `height` on `:hover` and `:focus` to `max-content`.

```
section {
  height: 2.5rem;
  overflow: hidden;
  transition: height ease 1s;
}

section:hover,
section:focus {
  height: max-content;
}
```

The rest of the CSS has been hidden for brevity.

Result

Try hovering over the `<section>` or focusing it via the keyboard — it will animate to its full height, revealing all the content.

See also

- `calc-size()`
- Animate to height: auto; (and other intrinsic sizing keywords) in CSS on developer.chrome.com (2024)

isolation CSS property

The **`isolation`** CSS property determines whether an element must create a new stacking context.

This property is especially helpful when used in conjunction with `mix-blend-mode` and `z-index`.

Try it

```
isolation: auto;
```

```
isolation: isolate;
```

```
<section class="default-example" id="default-example">
  <div class="background-container">
    <div id="example-element">
      <img src="/shared-assets/images/examples/firefox-logo.svg" />
      <p><code>mix-blend-mode: multiply;</code></p>
    </div>
  </div>
</section>
```

```
.background-container {
  background-color: #f4f460;
  width: 250px;
}

#example-element {
  border: 1px solid black;
  margin: 2em;
}

#example-element * {
  mix-blend-mode: multiply;
  color: #8245a3;
}
```

Syntax

```
/* Keyword values */
isolation: auto;
isolation: isolate;

/* Global values */
isolation: inherit;
isolation: initial;
isolation: revert;
isolation: revert-layer;
isolation: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

A new stacking context is created only if one of the properties applied to the element requires it.

`isolate`

A new stacking context must be created.

Formal definition

Initial value	auto
Applies to	All elements. In SVG, it applies to container elements, graphics elements, and graphics referencing elements.
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
isolation =
  <isolation-mode>

<isolation-mode> =
  auto     |
  isolate
```

This syntax reflects the latest standard as per Compositing and Blending Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Forcing a new stacking context for an element

HTML

```
<div class="big-square">
  <div class="isolation-auto">
    <div class="small-square">auto</div>
  </div>
  <div class="isolation-isolate">
    <div class="small-square">isolate</div>
  </div>
</div>
```

CSS

```
.isolation-auto {
  isolation: auto;
}

.isolation-isolate {
  isolation: isolate;
}

.big-square {
  background-color: lime;
  width: 200px;
  height: 210px;
}
```

```
.small-square {
  background-color: lime;
  width: 100px;
  height: 100px;
  border: 1px solid black;
  padding: 2px;
  mix-blend-mode: difference;
}
```

Result

See also

- `<blend-mode>`
- `mix-blend-mode`, `background-blend-mode`

justify-content CSS property

The CSS **`justify-content`** property defines how the browser distributes space between and around content items along the main axis of a flex container and the inline axis of grid and multicol containers.

The interactive example below demonstrates some `justify-content` values using grid layout.

Try it

```
justify-content: start;
```

```
justify-content: center;
```

```
justify-content: space-between;
```

```
justify-content: space-around;
```

```
justify-content: space-evenly;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 220px;
  display: grid;
  grid-template-columns: 60px 60px;
  grid-auto-rows: 40px;
  row-gap: 10px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Positional alignment */
justify-content: center;
justify-content: start;
justify-content: end;
justify-content: flex-start;
justify-content: flex-end;
justify-content: left;
justify-content: right;

/* Normal alignment */
justify-content: normal;

/* Distributed alignment */
justify-content: space-between;
justify-content: space-around;
justify-content: space-evenly;
justify-content: stretch;

/* Overflow alignment (for positional alignment only)*/
justify-content: safe center;
justify-content: unsafe center;

/* Global values */
justify-content: inherit;
justify-content: initial;
justify-content: revert;
justify-content: revert-layer;
justify-content: unset;
```

Values

This property is specified as one of the following keyword values:

`start`

> The items are packed flush to each other toward the start edge of the alignment container in the main axis.

`end`

The items are packed flush to each other toward the end edge of the alignment container in the main axis.

`flex-start`

The items are packed flush to each other toward the start edge of the alignment container on the flex container's main-start side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `start`.

`flex-end`

The items are packed flush to each other at the end edge of the alignment container on the flex container's main-end side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `end`.

`center`

The items are packed flush to each other toward the center of the alignment container along the main axis.

`left`

The items are packed flush to each other toward the left edge of the alignment container. When the property's horizontal axis is not parallel with the inline axis, such as when `flex-direction: column;` is set, this value behaves like `start`.

`right`

The items are packed flush to each other toward the right edge of the alignment container in the appropriate axis. If the property's axis is not parallel with the inline axis (in a grid container) or the main-axis (in a flexbox container), this value behaves like `start`.

`normal`

Behaves as `stretch`, except in the case of multi-column containers with a non-`auto` `column-width`, in which case the columns take their specified `column-width` rather than stretching to fill the container. As `stretch` behaves as `start` in flex containers, `normal` also behaves as `start`.

`space-between`

The items are evenly distributed within the alignment container along the main axis. The spacing between each pair of adjacent items is the same. The first item is flush with the main-start edge, and the last item is flush with the main-end edge.

`space-around`

The items are evenly distributed within the alignment container along the main axis. The spacing between each pair of adjacent items is the same. The empty space before the first and after the last item equals half of the space between each pair of adjacent items. If there is only one item, it will be centered.

`space-evenly`

The items are evenly distributed within the alignment container along the main axis. The spacing between each pair of adjacent items, the main-start edge and the first item, and the main-end edge and the last item, are all exactly the same.

`stretch`

If the combined size of the items along the main axis is less than the size of the alignment container, any `auto`-sized items have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size exactly fills the alignment container along the main axis.

> **Note:** For flexboxes, the `stretch` value behaves as `flex-start` or `start`. This is because, in flexboxes, stretching is controlled using the `flex-grow` property.

`safe`

If the item overflows the alignment container, then the item is aligned as if the alignment mode is `start`. The desired alignment will not be implemented.

`unsafe`

Even if the item overflows the alignment container, the desired alignment will be implemented. Unlike `safe`, which will ignore the desired alignment in favor of preventing overflow.

Description

Defined in the CSS box alignment module, `justify-content` applies to multicol containers, flex containers, and grid containers. The property does not apply to and has no effect on block containers.

This property shares many keyword values with the `align-content` property, but not all! `justify-content` isn't involved in baseline alignment, and therefore does not take baseline values.

In flex layouts, the property defines how positive free space is distributed between or around flex items along the main axis. This property impacts the space between elements in a line, not the space between lines. The alignment is done after the lengths and auto margins are applied, which means that when one or more flex items in a line have a `flex-grow` factor greater than `0`, the property has no effect as there is no space to distribute along that line. Also, as stretching along the main axis is controlled by `flex`, the `stretch` value behaves as `flex-start`.

With grid layout, this property distributes available space between or around grid columns, not grid items. If the grid container is larger than the grid itself, the `justify-content` property can be used to justify the grid along the inline axis, aligning the grid columns.

Grid `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties. Therefore by default, an `auto` sized track will take up any remaining space in the grid container. As the grid's inline size has to be less than the grid container's inline size for there to be space to distribute, the property has no effect in this case.

Formal definition

Initial value	`normal`
Applies to	flex containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
justify-content =
  normal                                    |
  <content-distribution>                    |
  <overflow-position>? [ <content-position> | left | right ]

<content-distribution> =
  space-between  |
  space-around   |
  space-evenly   |
  stretch
```

```
<overflow-position> =
  unsafe  |
  safe

<content-position> =
  center      |
  start       |
  end         |
  flex-start  |
  flex-end
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic grid example

In this example, we have a grid that is narrower than its grid container, and we use `justify-content` to distribute the available space evenly around and between the grid columns.

HTML

The `<section>` container, our grid container to-be, has 16 nested `<div>`s that will become grid items.

```
<section class="container">
  <div>A</div>
  <div>B</div>
  <div>C</div>
  <div>D</div>
  <div>E</div>
  <div>F</div>
  <div>G</div>
  <div>H</div>
  <div>I</div>
  <div>J</div>
  <div>K</div>
  <div>L</div>
  <div>M</div>
  <div>N</div>
  <div>O</div>
  <div>P</div>
</section>
```

CSS

```
.container {
  margin: 5px;
  border: 1px solid;
  box-sizing: border-box;
}

div {
  line-height: 2em;
  border: 1px solid;
  box-sizing: border-box;
  text-align: center;
}
```

We set the container width to `500px` and specify three columns, each `80px` wide, meaning there is `260px` of available space to distribute between or around them. We then set `justify-content: space-evenly`, which means there will be `65px` of space before, between, and after each column.

We set different widths (and background colors) to demonstrate how the justification is applied to the columns.

```
.container {
  display: grid;
  grid: auto-flow / repeat(3, 80px);
  width: 500px;
  justify-content: space-evenly;
}

div {
  background-color: pink;
  width: 80px;
}

div:nth-of-type(n + 9) {
  width: 35px;
  background-color: lightgreen;
}

div:nth-last-of-type(3) {
  width: 250px;
  background-color: lightblue;
}
```

Result

Note that `justify-contents` aligns the columns and has no effect on the items or alignment in grid areas. Grid items, even those that overflow their grid cell, do not impact column justification.

The safe keyterm

This example demonstrates the `safe` keyterm. We specify four centered flex items that are not allowed to wrap, and as a result, overflow their single flex-line container. By adding `safe` to `center` in the `justify-content` property, overflowing content behaves as if the alignment mode is `start`

```
<p><code>justify-content: center;</code></p>
<section class="container safe">
  <div>A</div>
  <div>B</div>
  <div>C</div>
  <div>D</div>
</section>
```

```
<p><code>justify-content: safe center;</code></p>
<section class="container safe-center">
  <div>A</div>
  <div>B</div>
  <div>C</div>
  <div>D</div>
</section>
<p><code>justify-content: safe center;</code> with 3 items</p>
<section class="container safe-center">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</section>
```

```
.container {
  margin: 5px auto;
  border: 1px dashed;
  box-sizing: border-box;
  background-color: lightblue;
}

div {
  line-height: 1em;
  border: 1px solid;
  box-sizing: border-box;
  text-align: center;
  background-color: pink;
}
```

The container is set to `center`, with every container other than the first having the `safe` keyword added:

```
.container {
  align-items: baseline;
  display: flex;
  width: 350px;
  height: 2em;
}

.safe {
  justify-content: center;
}

.safe-center {
  justify-content: safe center;
}

div {
  flex: 0 0 100px;
}
```

Result

As an item overflows the alignment container, with `safe` included the alignment mode behaves as `start` and the `center` alignment is not implemented. The `safe` keyterm has no effect if the items do not overflow the container.

Visualizing flex item distribution

This example includes a multi-line wrapping flex layout. The second flex item has a positive flex growth factor, consuming all the available free space in the first flex line.

CSS

```
#container {
  margin: 5px auto;
  border: 1px dashed black;
  box-sizing: border-box;
}

div {
  line-height: 2em;
  border: 1px solid;
  box-sizing: border-box;
  text-align: center;
}
```

```
#container {
  display: flex;
  flex-flow: row wrap;
  justify-content: space-between; /* Can be changed in the live sample */
  width: 510px;
}

div {
  line-height: 2em;
  flex: 0 0 120px;
  background-color: pink;
}

div:nth-of-type(2) {
  flex-grow: 1;
  background-color: yellow;
}

div:nth-of-type(n + 9) {
  flex: 0 0 35px;
  background-color: lightgreen;
}
div:last-of-type {
  flex: 0 0 300px;
  background-color: lightblue;
}
```

```
<section id="container">
  <div>A</div>
  <div>GROW</div>
  <div>C</div>
  <div>D</div>
  <div>E</div>
  <div>F</div>
  <div>G</div>
  <div>H</div>
  <div>I</div>
  <div>J</div>
  <div>K</div>
  <div>L</div>
  <div>M</div>
  <div>N</div>
  <div>O</div>
  <div>P</div>
</section>
<select id="justifyContent">
  <option value="start">start</option>
  <option value="end">end</option>
  <option value="flex-start">flex-start</option>
  <option value="flex-end">flex-end</option>
  <option value="center">center</option>
  <option value="left">left</option>
  <option value="right">right</option>
  <option value="space-between" selected>space-between</option>
  <option value="space-around">space-around</option>
  <option value="space-evenly">space-evenly</option>
  <option value="stretch">stretch</option>
</select>
```

```
const justifyContent = document.getElementById("justifyContent");
justifyContent.addEventListener("change", (evt) => {
  document.getElementById("container").style.justifyContent = evt.target.value;
});
```

Result

Select different keywords from the drop-down menu to visualize the different `justify-content` keyword values. Because an item on the first line can grow, there is no available space on that line for the `justify-content` property to distribute. With the `space-between` value, the first item on each line is flush with the main-start edge, and the last item is flush with the main-end edge. As a result, if a line has only one item, it will be aligned with the main-start edge (as seen in the last line). This is not the case for other `space-*` values, such as `space-evenly` and `space-around`, which center one-item flex-lines.

See also

- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment module

justify-items CSS property

The CSS **`justify-items`** property defines the default `justify-self` for all items of the box, giving them all a default way of justifying each box along the appropriate axis.

Try it

```
justify-items: stretch;
```

```
justify-items: center;
```

```
justify-items: start;
```

```
justify-items: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  width: 220px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

The effect of this property is dependent of the layout mode we are in:

- In block-level layouts, it aligns the items inside their containing block on the inline axis.
- For absolutely-positioned elements, it aligns the items inside their containing block on the inline axis, accounting for the offset values of top, left, bottom, and right.
- In table cell layouts, this property is *ignored* (see Box alignment for block, absolutely positioned, and table layouts)
- In flexbox layouts, this property is *ignored* (see Box alignment in flexbox)
- In grid layouts, it aligns the items inside their grid areas on the inline axis (see Box alignment in grid layout)

Syntax

```
/* Basic keywords */
justify-items: normal;
justify-items: stretch;

/* Positional alignment */
justify-items: center; /* Pack items around the center */
justify-items: start; /* Pack items from the start */
justify-items: end; /* Pack items from the end */
justify-items: flex-start; /* Equivalent to 'start'. Note that justify-items is ignored in flexbox layouts. */
justify-items: flex-end; /* Equivalent to 'end'. Note that justify-items is ignored in flexbox layouts. */
justify-items: self-start;
justify-items: self-end;
justify-items: left; /* Pack items from the left */
justify-items: right; /* Pack items from the right */
justify-items: anchor-center;

/* Baseline alignment */
justify-items: baseline;
justify-items: first baseline;
justify-items: last baseline;

/* Overflow alignment (for positional alignment only) */
justify-items: safe center;
justify-items: unsafe center;

/* Legacy alignment */
justify-items: legacy left;
justify-items: legacy right;
justify-items: legacy center;

/* Global values */
justify-items: inherit;
justify-items: initial;
justify-items: revert;
justify-items: revert-layer;
justify-items: unset;
```

This property can take one of four different forms:

- Basic keywords: one of the keyword values `normal` or `stretch`.
- Baseline alignment: the `baseline` keyword, plus optionally one of `first` or `last`.
- Positional alignment: one of: `center`, `start`, `end`, `flex-start`, `flex-end`, `self-start`, `self-end`, `left`, or `right`. Plus optionally `safe` or `unsafe`.
- Legacy alignment: the `legacy` keyword, followed by one of `left`, `right`, or `center`.

Values

`normal`

The effect of this keyword is dependent of the layout mode we are in:

- In block-level layouts, the keyword is a synonym of `start`.
- In absolutely-positioned layouts, the keyword behaves like `start` on *replaced* absolutely-positioned boxes, and as `stretch` on *all other* absolutely-positioned boxes.
- In table cell layouts, this keyword has no meaning as this property is *ignored*.
- In flexbox layouts, this keyword has no meaning as this property is *ignored*.
- In grid layouts, this keyword leads to a behavior similar to the one of `stretch`, except for boxes with an aspect ratio or an intrinsic size where it behaves like `start`.

`start`

The item is packed flush to each other toward the start edge of the alignment container in the appropriate axis.

`end`

The item is packed flush to each other toward the end edge of the alignment container in the appropriate axis.

`flex-start`

For items that are not children of a flex container, this value is treated like `start`.

`flex-end`

For items that are not children of a flex container, this value is treated like `end`.

`self-start`

The item is packed flush to the edge of the alignment container of the start side of the item, in the appropriate axis.

`self-end`

The item is packed flush to the edge of the alignment container of the end side of the item, in the appropriate axis.

`center`

The items are packed flush to each other toward the center of the alignment container.

`left`

The items are packed flush to each other toward the left edge of the alignment container. If the property's axis is not parallel with the inline axis, this value behaves like `start`.

`right`

The items are packed flush to each other toward the right edge of the alignment container in the appropriate axis. If the property's axis is not parallel with the inline axis, this value behaves like `start`.

`baseline`, `first baseline`, `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group. The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`stretch`

If the combined size of the items is less than the size of the alignment container, any `auto`-sized items have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size exactly fills the alignment container.

`anchor-center`

In the case of anchor-positioned elements, aligns the items to the center of the associated anchor element in the inline direction. See Centering on the anchor using `anchor-center`.

`safe`

If the size of the item overflows the alignment container, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Regardless of the relative sizes of the item and alignment container, the given alignment value is honored.

`legacy`

Makes the value inherited by the box descendants. Note that if a descendant has a `justify-self: auto` value, the `legacy` keyword is not considered by the descend, only the `left`, `right`, or `center` value associated to it.

Formal definition

Initial value	`legacy`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
justify-items =
  normal                                          |
  stretch                                         |
  <baseline-position>                             |
  <overflow-position>? [ <self-position> | left | right ]  |
  legacy                                          |
  legacy && [ left | right | center ]

<baseline-position> =
  [ first | last ]?  &&
  baseline

<overflow-position> =
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

In this example, we have a 2 x 2 grid layout. Initially the grid container is given a `justify-items` value of `stretch` (the default), which causes the grid items to stretch across the entire width of their cells.

If you hover or tab onto the grid container however, it is given a `justify-items` value of `center`, which causes the grid items to span only as wide as their content width, and align in the center of their cells.

HTML

```
<article class="container" tabindex="0">
  <span>First child</span>
  <span>Second child</span>
  <span>Third child</span>
  <span>Fourth child</span>
</article>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
  letter-spacing: 1px;
}

article {
  background-color: red;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  margin: 20px;
  width: 300px;
  justify-items: stretch;
}

article:hover,
article:focus {
  justify-items: center;
}

article span {
  background-color: black;
  color: white;
  margin: 1px;
```

```
  text-align: center;
}

article,
span {
  padding: 10px;
  border-radius: 7px;
}
```

Result

See also

- `justify-self`
- `align-items`
- `place-items` shorthand
- Box alignment in grid layout
- CSS box alignment module

justify-self CSS property

The CSS **`justify-self`** property sets the way a box is justified inside its alignment container along the appropriate axis.

Try it

```
justify-self: stretch;
```

```
justify-self: center;
```

```
justify-self: start;
```

```
justify-self: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  width: 220px;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 40px;
  grid-gap: 10px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

The effect of this property is dependent of the layout mode we are in:

- In block-level layouts, it aligns an item inside its containing block on the inline axis.
- For absolutely-positioned elements, it aligns an item inside its containing block on the inline axis, accounting for the offset values of top, left, bottom, and right.
- In table cell layouts, this property is *ignored*. Read more about alignment in block, absolute positioned and table layout.
- In flexbox layouts, this property is *ignored*. Read more about alignment in flexbox.
- In grid layouts, it aligns an item inside its grid area on the inline axis. Read more about alignment in grid layouts.

Syntax

```
/* Basic keywords */
justify-self: auto;
justify-self: normal;
justify-self: stretch;

/* Positional alignment */
justify-self: center; /* Pack item around the center */
justify-self: start; /* Pack item from the start */
justify-self: end; /* Pack item from the end */
justify-self: flex-start; /* Equivalent to 'start'. Note that justify-self is ignored in flexbox layouts. */
justify-self: flex-end; /* Equivalent to 'end'. Note that justify-self is ignored in flexbox layouts. */
justify-self: self-start;
justify-self: self-end;
justify-self: left; /* Pack item from the left */
justify-self: right; /* Pack item from the right */
justify-self: anchor-center;

/* Baseline alignment */
justify-self: baseline;
justify-self: first baseline;
justify-self: last baseline;

/* Overflow alignment (for positional alignment only) */
justify-self: safe center;
justify-self: unsafe center;

/* Global values */
justify-self: inherit;
justify-self: initial;
justify-self: revert;
justify-self: revert-layer;
justify-self: unset;
```

This property can take one of three different forms:

- Basic keywords: one of the keyword values `normal`, `auto`, or `stretch`.
- Baseline alignment: the `baseline` keyword, plus optionally one of `first` or `last`.
- Positional alignment:
  - one of: `center`, `start`, `end`, `flex-start`, `flex-end`, `self-start`, `self-end`, `left`, or `right`.
  - Plus optionally `safe` or `unsafe`.

Values

`auto`

The value used is the value of the `justify-items` property of the parents box, unless the box has no parent, or is absolutely positioned, in these cases, `auto` represents `normal`.

`normal`

The effect of this keyword is dependent of the layout mode we are in:

- In block-level layouts, the keyword is a synonym of `start`.
- In absolutely-positioned layouts, the keyword behaves like `start` on *replaced* absolutely-positioned boxes, and as `stretch` on *all other* absolutely-positioned boxes.
- In table cell layouts, this keyword has no meaning as this property is *ignored*.
- In flexbox layouts, this keyword has no meaning as this property is *ignored*.
- In grid layouts, this keyword leads to a behavior similar to the one of `stretch`, except for boxes with an aspect ratio or an intrinsic size where it behaves like `start`.

`start`

The item is packed flush to each other toward the start edge of the alignment container in the appropriate axis.

`end`

The item is packed flush to each other toward the end edge of the alignment container in the appropriate axis.

`flex-start`

For items that are not children of a flex container, this value is treated like `start`.

`flex-end`

For items that are not children of a flex container, this value is treated like `end`.

`self-start`

The item is packed flush to the edge of the alignment container of the start side of the item, in the appropriate axis.

`self-end`

The item is packed flush to the edge of the alignment container of the end side of the item, in the appropriate axis.

`center`

The items are packed flush to each other toward the center of the alignment container.

`left`

The items are packed flush to each other toward the left edge of the alignment container. If the property's axis is not parallel with the inline axis, this value behaves like `start`.

`right`

The items are packed flush to each other toward the right edge of the alignment container in the appropriate axis. If the property's axis is not parallel with the inline axis, this value behaves like `start`.

`baseline`, `first baseline`, `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group. The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`stretch`

If the combined size of the items is less than the size of the alignment container, any `auto`-sized items have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size exactly fills the alignment container.

`anchor-center`

In the case of anchor-positioned elements, aligns the item to the center of the associated anchor element in the inline direction. See Centering on the anchor using `anchor-center`.

`safe`

If the size of the item overflows the alignment container, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Regardless of the relative sizes of the item and alignment container, the given alignment value is honored.

Formal definition

Initial value	`auto`
Applies to	block-level boxes, absolutely-positioned boxes, and grid items
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
justify-self =
  auto                                          |
  <overflow-position>? [ normal | <self-position> | left | right ]  |
  stretch                                       |
  <baseline-position>                           |
  anchor-center

<overflow-position> =
```

```
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end

<baseline-position> =
  [ first | last ]?  &&
  baseline
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3, CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

In the following example we have a 2 x 2 grid layout. Initially the grid container is given a `justify-items` value of `stretch` — the default — which causes the grid items to stretch across the entire width of their cells.

The second, third, and fourth grid items are then given different values of `justify-self`, to show how these override the `justify-items` value. These values cause the grid items to span only as wide as their content width, and align in different positions across their cells.

HTML

```
<article class="container">
  <span>First child</span>
  <span>Second child</span>
  <span>Third child</span>
  <span>Fourth child</span>
</article>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
  letter-spacing: 1px;
}

article {
  background-color: red;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  margin: 20px;
  width: 300px;
  justify-items: stretch;
}

span:nth-child(2) {
  justify-self: start;
}

span:nth-child(3) {
  justify-self: center;
}

span:nth-child(4) {
  justify-self: end;
}

article span {
  background-color: black;
  color: white;
  margin: 1px;
  text-align: center;
}

article,
span {
  padding: 10px;
  border-radius: 7px;
}
```

Result

See also

- Box alignment in grid layout
- CSS box alignment module
- `justify-items`
- `align-self`
- `place-self`

left CSS property

The **`left`** CSS property participates in specifying the horizontal position of a positioned element. This inset property has no effect on non-positioned elements.

Try it

```
left: 0;
```

```
left: 4em;
```

```
left: 10%;
```

```
left: 20px;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #264653;
  border: 4px solid #ffb500;
  color: white;
  position: absolute;
  width: 140px;
  height: 60px;
}
```

Syntax

```
/* <length> values */
left: 3px;
left: 2.4em;
left: anchor(--my-anchor 50%);
left: calc(anchor-size(--my-anchor inline, 100px) * 2);

/* <percentage>s of the width of the containing block */
left: 10%;

/* Keyword value */
left: auto;

/* Global values */
left: inherit;
left: initial;
left: revert;
left: revert-layer;
left: unset;
```

Values

`<length>`

A negative, null, or positive `<length>`:

- for *absolutely positioned elements*, it represents the distance to the left edge of the containing block.
- for *anchor-positioned elements*, the `anchor()` function resolves to a `<length>` value relative to the position of the associated *anchor element*'s left or right edge (see Using inset properties with `anchor()` function values), and the `anchor-size()` function resolves to a `<length>` value relative to the associated anchor element's width or height (see Setting element position based on anchor size).
- for *relatively positioned elements*, it represents the distance that the element is moved to the right of its normal position.

`<percentage>`

A `<percentage>` of the containing block's width.

`auto`

Specifies that:

- for *absolutely positioned elements*, the position of the element is based on the `right` property, while `width: auto` is treated as a width based on the content; or if `right` is also `auto`, the element is positioned where it should horizontally be positioned if it were a static element.
- for *relatively positioned elements*, the distance of the element from its normal position is based on the `right` property; or if `right` is also `auto`, the element is not moved horizontally at all.

Description

The effect of `left` depends on how the element is positioned (i.e., the value of the `position` property):

- When `position` is set to `absolute` or `fixed`, the `left` property specifies the distance between the element's outer margin of left edge and the inner border of left edge of its containing block. (The containing block is the ancestor to which the element is relatively positioned.) If the positioned element has an associated *anchor element*, and the property value includes an `anchor()` function, `left` positions the left edge of the positioned element relative to the position of the specified `<anchor-side>` edge. The `left` property is compatible with the `left`, `right`, `start`, `end`, `self-start`, `self-end`, `center`, and `<percentage>` values.
- When `position` is set to `relative`, the `left` property specifies the distance the element's left edge is moved to the right from its normal position.
- When `position` is set to `sticky`, the `left` property is used to compute the sticky-constraint rectangle.
- When `position` is set to `static`, the `left` property has *no effect*.

When both `left` and `right` are defined, and width constraints don't prevent it, the element will stretch to satisfy both. If the element cannot stretch to satisfy both, the position of the element is *overspecified*. When this is the case, the `left` value has precedence when the container is left-to-right; the `right` value has precedence when the container is right-to-left.

Formal definition

Initial value	`auto`

Applies to	positioned elements
Inherited	no
Percentages	refer to the width of the containing block
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length, percentage or calc();

Formal syntax

```
left =
  auto                |
  <length-percentage>  |
  <anchor()>           |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside        |
  outside       |
  top           |
  left          |
  right         |
  bottom        |
  start         |
  end           |
  self-start    |
  self-end      |
  <percentage>  |
  center

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Positioning elements

HTML

```
<div id="wrap">
  <div id="example_1">
    <pre>
      position: absolute;
      left: 20px;
      top: 20px;
    </pre>
    <p>
      The only containing element for this div is the main window, so it
      positions itself in relation to it.
    </p>
  </div>

  <div id="example_2">
    <pre>
      position: relative;
      top: 0;
      right: 0;
    </pre>
    <p>Relative position in relation to its siblings.</p>
  </div>

  <div id="example_3">
    <pre>
      float: right;
      position: relative;
      top: 20px;
      left: 20px;
    </pre>
    <p>Relative to its sibling div above, but removed from flow of content.</p>

    <div id="example_4">
      <pre>
        position: absolute;
        bottom: 10px;
        right: 20px;
      </pre>
      <p>Absolute position inside of a parent with relative position</p>
    </div>

    <div id="example_5">
      <pre>
        position: absolute;
```

```
        right: 0;
        left: 0;
        top: 200px;
      </pre>
      <p>Absolute position with both left and right declared</p>
    </div>
  </div>
</div>
```

CSS

```
#wrap {
  width: 700px;
  margin: 0 auto;
  background: #5c5c5c;
}

pre {
  white-space: pre-line;
  word-wrap: break-word;
}

#example_1 {
  width: 200px;
  height: 200px;
  position: absolute;
  left: 20px;
  top: 20px;
  background-color: #d8f5ff;
}

#example_2 {
  width: 200px;
  height: 200px;
  position: relative;
  top: 0;
  right: 0;
  background-color: #c1ffdb;
}
#example_3 {
  width: 600px;
  height: 400px;
  position: relative;
  top: 20px;
  left: 20px;
  background-color: #ffd7c2;
}

#example_4 {
  width: 200px;
  height: 200px;
  position: absolute;
  bottom: 10px;
  right: 20px;
  background-color: #ffc7e4;
}
#example_5 {
  position: absolute;
  right: 0;
  left: 0;
  top: 100px;
  background-color: #d7ffc2;
}
```

Result

See also

- `top`, `bottom`, and `right`
- `inset` shorthand
- `inset-block-start`, `inset-block-end`, `inset-inline-start`, and `inset-inline-end`
- `inset-block` and `inset-inline` shorthands
- `position`
- CSS positioned layout module

letter-spacing CSS property

The **`letter-spacing`** CSS property sets the spacing between text characters. This value is added to the natural spacing between characters while rendering the text. Positive values of `letter-spacing` spread characters further apart, while negative values of `letter-spacing` bring characters closer together.

Try it

```
letter-spacing: normal;
```

```
letter-spacing: 0.2rem;
```

```
letter-spacing: 1px;
```

```
letter-spacing: 30%;
```

```
letter-spacing: -1px;
```

```
<section id="default-example">
  <p id="example-element">
    As much mud in the streets as if the waters had but newly retired from the
    face of the earth, and it would not be wonderful to meet a Megalosaurus,
    forty feet long or so, waddling like an elephantine lizard up Holborn Hill.
```

```
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2");
  font-family: "Amstelvar";
  font-style: normal;
}

section {
  font-size: 1.2em;
  font-family: "Amstelvar", serif;
}
```

Syntax

```
/* Keyword value */
letter-spacing: normal;

/* <length-percentage> values */
letter-spacing: 0.3em;
letter-spacing: 3px;
letter-spacing: -0.5px;
letter-spacing: 50%;

/* Global values */
letter-spacing: inherit;
letter-spacing: initial;
letter-spacing: revert;
letter-spacing: revert-layer;
letter-spacing: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

The normal letter spacing for the current font. Unlike a value of `0`, this keyword allows the user agent to alter the space between characters in order to justify text.

`<length-percentage>`

Specifies extra inter-character space *in addition to* the default space between characters. While values can be negative, these may be constrained to implementation-specific limits. User agents may not further increase or decrease the inter-character space in order to justify text.

Percentage values are calculated relative to the width of the space character of the font applied to the text.

> **Note:** When `letter-spacing` is non-zero, user agents do not apply optional ligatures, such as the `liga` (standard ligatures) and `clig` (contextual ligatures) OpenType features normally controlled by `font-variant-ligatures`. These features can be explicitly re-enabled with `font-feature-settings`.

Accessibility

A large positive or negative `letter-spacing` value will make the word(s) the styling is applied to unreadable. For text styled with a very large positive value, the letters will be so far apart that the word(s) will appear like a series of individual, unconnected letters. For text styled with a very large negative value, the letters can overlap each other to the point where the word(s) may be unrecognizable.

Legible letter-spacing must be determined on a case-by-case basis, as different font families have different character widths. There is no one value that can ensure all font families automatically maintain their legibility.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | W3C Understanding WCAG 2.0

Internationalization concerns

Some written languages should not have any letter spacing applied. For instance, languages that use the Arabic script expect connected letters to remain visually connected, as in the following example. Applying letter spacing may lead to the text looking broken.

```
<p lang="ar" dir="rtl">شسيبتنمك</p>
```

```
p {
  font-size: 3em;
  margin-inline-start: 5px;
}
```

Formal definition

Initial value	`normal`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	an optimum value consisting of either an absolute length or the keyword `normal`
Animation type	a length

Formal syntax

```
letter-spacing =
  normal               |
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Text Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting letter-spacing with length values

This example shows several paragraphs with different length `letter-spacing` values set, allowing you to compare them.

HTML

The HTML contains several `<p>` elements containing text content.

```
<p class="normal">letter spacing</p>
<p class="em-wide">letter spacing</p>
<p class="em-wider">letter spacing</p>
<p class="em-tight">letter spacing</p>
<p class="px-wide">letter spacing</p>
```

CSS

Our CSS applies a different `letting-spacing` value to each paragraph.

```
.normal {
  letter-spacing: normal;
}
.em-wide {
  letter-spacing: 0.4em;
}
.em-wider {
  letter-spacing: 1em;
}
.em-tight {
  letter-spacing: -0.05em;
}
.px-wide {
  letter-spacing: 6px;
}
```

Result

The rendered result looks like this:

Comparing letter-spacing set with length and percentage

This example demonstrates that percentage `letter-spacing` values are useful for responsive text sizing.

The code displays several paragraphs that have the same `letter-spacing` set on text with increasing font size. We provide functionality to switch between a length and a percentage `letter-spacing` value, so that you can observe the responsive qualities of using a percentage value.

HTML

The HTML contains several `<p>` elements containing text content, and an `<input type="checkbox">` that we'll use to toggle between a length `letter-spacing` and a percentage `letter-spacing` value.

```
<p class="x-small">X-small font-size (0.8em)</p>
<p class="small">Small font-size (1.3em)</p>
<p class="medium">Medium font-size (2em)</p>
<p class="large">Large font-size (3em)</p>
<p class="x-large">X-Large (3.5em)</p>

<form>
  <label for="ls-toggle">
    Toggle <code>letter-spacing</code> (off: <code>8px</code>, on:
    <code>12%</code>)
  </label>
  <input type="checkbox" id="ls-toggle" />
</form>
```

CSS

Our CSS starts by applying increasing `font-size` values to each successive paragraph:

```
html {
  font-family: "Arial", sans-serif;
}
```

```
.x-small {
  font-size: 0.8em;
}

.small {
  font-size: 1.3em;
}

.medium {
  font-size: 2em;
}

.large {
  font-size: 3em;
}

.x-large {
  font-size: 3.5em;
}
```

We apply a `letter-spacing` value of `8px` to all paragraphs by default. When the checkbox is checked, however, we change the `letter-spacing` value to `12%`:

```
p {
  letter-spacing: 8px;
}

p:has(~ form > input:checked) {
```

```
  letter-spacing: 12%;
}
```

Result

The rendered result looks like this:

First, note how the initial length letter spacing value looks OK when applied to the larger font sizes, but it doesn't look good on the smaller font sizes. Now toggle the checkbox, and note how the percentage letter spacing looks appropriate on all lines, as it scales with the font size.

See also

- `font-kerning`
- `word-spacing`
- SVG `letter-spacing` attribute

lighting-color CSS property

The **`lighting-color`** CSS property defines the color of the light source for the `<feDiffuseLighting>` and `<feSpecularLighting>` SVG lighting filter primitives within an SVG `<filter>`. If present, it overrides the element's `lighting-color` attribute.

> **Note:** The `lighting-color` property only applies to `<feDiffuseLighting>` and `<feSpecularLighting>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* <color> values */
lighting-color: red;
lighting-color: hsl(120deg 75% 25% / 60%);
lighting-color: currentColor;

/* Global values */
lighting-color: inherit;
lighting-color: initial;
lighting-color: revert;
lighting-color: revert-layer;
lighting-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The lighting's color. This can be any valid CSS `<color>` value.

Formal definition

Initial value	`white`
Applies to	`<feDiffuseLighting>` and `<feSpecularLighting>` elements in `<svg>`
Inherited	no
Computed value	as specified
Animation type	by computed value

Formal syntax

```
lighting-color =
  <color>
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the color of filter lighting

This example demonstrates the basic use case of `lighting-color`, and how the CSS `lighting-color` property takes precedence over the `lighting-color` attribute.

HTML

We have an SVG with two `<filter>` elements, one with a `<feDiffuseLighting>` and one with a `<feSpecularLighting>` child. Each includes the SVG `lighting-color` attribute defining the lighting color as `red`. Both of these children have a `<fePointLight>`, the required child that sets the light source. We included two `<rect>` elements with a filter attribute; this is where the filters will be displayed.

```
<svg viewBox="0 0 420 120" xmlns="http://www.w3.org/2000/svg">
  <filter id="flood1">
    <feDiffuseLighting lighting-color="red">
      <fePointLight x="75" y="30" z="10" />
    </feDiffuseLighting>
  </filter>
  <filter id="flood2">
    <feSpecularLighting specularExponent="5" lighting-color="red">
      <fePointLight x="225" y="75" z="5" />
    </feSpecularLighting>
  </filter>

  <rect id="r1" filter="url(#flood1)" />
  <rect id="r2" filter="url(#flood2)" />
</svg>
```

CSS

We define the size and position of our `<rect>` using the CSS `height`, `width`, `x`, and `y` properties. We also add a background image to the SVG to make any color alpha transparency more apparent:

```
svg {
  background-image: repeating-linear-gradient(
    45deg,
    transparent 0 9px,
```

```
    #cccccc 0px 10px
  );
}

rect {
  width: 100px;
  height: 100px;
  x: 10px;
  y: 10px;
}

#r2 {
  x: 150px;
}
```

We then apply different lighting color values to the filter's child elements using the CSS `lighting-color` property. We use a named color and a 3-digit hexadecimal color, but we can use any valid CSS color syntax:

```
feDiffuseLighting {
  lighting-color: blue;
}

feSpecularLighting {
  lighting-color: #ff0099;
}
```

Results

The attributes defined the color of both light filters as `red`, but these values were overridden by the CSS `lighting-color` values.

See also

- `color`
- `fill`
- `flood-color`
- `stop-color`
- `stroke`
- `flood-opacity`
- `background-color`
- `<color>`
- `<filter-function>`
- SVG `lighting-color` attribute

line-break CSS property

The **`line-break`** CSS property sets how to break lines of Chinese, Japanese, or Korean (CJK) text when working with punctuation and symbols.

Try it

```
line-break: auto;
```

```
line-break: anywhere;
```

```
line-break: normal;
```

```
line-break: loose;
```

```
<section id="default-example">
  <p id="example-element">
    この喫茶店は、いつでもコーヒーの香りを漂わせています。<br />彼女はこの喫茶店で働いて、着々と実力をつけていきました。<br />今では知る人ぞ知る、名人です。
  </p>
</section>
```

```
#example-element {
  font-family: "Yu Gothic", "YuGothic", "Meiryo", "ＭＳ ゴシック", sans-serif;
  border: 2px dashed #999999;
  text-align: left;
  width: 240px;
  font-size: 16px;
}
```

Syntax

```
/* Keyword values */
line-break: auto;
line-break: loose;
line-break: normal;
line-break: strict;
line-break: anywhere;

/* Global values */
line-break: inherit;
line-break: initial;
line-break: revert;
line-break: revert-layer;
line-break: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Break text using the default line break rule.

`loose`

Break text using the least restrictive line break rule. Typically used for short lines, such as in newspapers.

`normal`

Break text using the most common line break rule.

`strict`

Break text using the most stringent line break rule.

`anywhere`

There is a soft wrap opportunity around every typographic character unit, including around any punctuation character or preserved white spaces, or in the middle of words, disregarding any prohibition against line breaks, even those introduced by characters with the GL, WJ, or ZWJ character class or mandated by the `word-break` property. The different wrapping opportunities must not be prioritized. Hyphenation is not applied.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
line-break =
  auto      |
  loose     |
  normal    |
  strict    |
  anywhere
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting text wrapping

See whether the text is wrapped before "々", "ぁ" and "。".

HTML

```
<div lang="ja">
  <p class="wrap-box auto">
    auto:<br />そこは湖のほとりで木々が輝いていた。<br />その景色に、美しいなぁと思わずつぶやいた。
  </p>
  <p class="wrap-box loose">
    loose:<br />そこは湖のほとりで木々が輝いていた。<br />その景色に、美しいなぁと思わずつぶやいた。
  </p>
  <p class="wrap-box normal">
    normal:<br />そこは湖のほとりで木々が輝いていた。<br />その景色に、美しいなぁと思わずつぶやいた。
  </p>
  <p class="wrap-box strict">
    strict:<br />そこは湖のほとりで木々が輝いていた。<br />その景色に、美しいなぁと思わずつぶやいた。
  </p>
  <p class="wrap-box anywhere">
    anywhere:<br />そこは湖のほとりで木々が輝いていた。<br />その景色に、美しいなぁと思わずつぶやいた。
  </p>
</div>
```

CSS

```
.wrap-box {
  width: 10em;
  margin: 0.5em;
  white-space: normal;
  vertical-align: top;
  display: inline-block;
}
.auto {
  line-break: auto;
}
.loose {
  line-break: loose;
}
.normal {
  line-break: normal;
}
.strict {
  line-break: strict;
}
.anywhere {
  line-break: anywhere;
}
```

Result

See also

- CSS and international text on W3C

line-clamp CSS property

The **`line-clamp`** CSS property allows limiting of the contents of a block to the specified number of lines. Optionally, it also allows inserting content into the last line to indicate that content was truncated.

Syntax

```
/* Keyword value */
line-clamp: none;

/* <integer> value only */
line-clamp: 3;
line-clamp: 10;

/* <integer> and <'block-ellipsis'> values */
line-clamp: 3 no-ellipsis;
line-clamp: 10 "… (there is extra content)";

/* <'block-ellipsis'> value only */
line-clamp: no-ellipsis;
line-clamp: "… (there is extra content)";

/* Global values */
line-clamp: inherit;
line-clamp: initial;
line-clamp: revert;
line-clamp: revert-layer;
line-clamp: unset;
```

Values

This property is specified as one or two space-separated values from the following list:

`none`

Specifies that content is not clamped. This keyword cannot be combined with the other values. This is the default.

`<integer>` Optional

Specifies the number of lines after which content is clamped. It must be greater than 0.

`<'block-ellipsis'>` Optional

Specifies the content inserted into the last line. It can take one of the following values:

- `no-ellipsis`: No ellipsis (character U+2026) is added if the text is truncated due to the number of line specified.
- `auto`: Renders an ellipsis character (U+2026) if the text is truncated.
- `<string>`: Renders the specified string at the end of the affected line. Browsers may truncate this string if it is very long. An empty string behaves like the `no-ellipsis` value.

> **Note:** If `<'block-ellipsis'>` is specified without an `<integer>`, it applies only when content is clamped by the height of the container.

Description

In most cases you will also want to set `overflow` to `hidden`, otherwise the contents won't be clipped but an ellipsis will still be shown after the specified number of lines.

When applied to anchor elements, the truncating can happen in the middle of the text, not necessarily at the end.

> **Note:** For legacy support, the vendor-prefixed `-webkit-line-clamp` property only works in combination with the `display` property set to `-webkit-box` or `-webkit-inline-box` and the `-webkit-box-orient` property set to `vertical`. Despite these prefixed properties being deprecated, the co-dependency of these three properties is a fully specified behavior and will continue to be supported.

Formal definition

Initial value	none
Applies to	Block containers except multi-column containers
Inherited	no
Computed value	as specified
Animation type	an integer

Formal syntax

```
line-clamp =
  none                                          |
  [ <'max-lines'> || <'block-ellipsis'> ] -webkit-legacy?

<max-lines> =
  auto            ||
  <integer [1,∞]>

<block-ellipsis> =
  no-ellipsis  |
  ellipsis     |
  <string>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Overflow Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Truncating content with an ellipsis or a custom string

In this example, there are three cards, each with a different `line-clamp` value:

- The first has only an `<integer>` to restrict the number of lines.
- The second has an `<integer>` and the `no-ellipsis` value.
- The third has an `<integer>` and a custom `<string>`.

HTML

```
<section>
  <div class="card">
    <h2>number of lines</h2>
    <p class="integer">
      In this card, the <em>number of lines</em> is specified by an
      <code>&lt;integer&gt;</code>. Any content that does not fit is truncated
      and an ellipsis is shown.
    </p>
  </div>
  <div class="card">
    <h2>no ellipsis</h2>
    <p class="no-ellipsis">
      In this card, an <code>&lt;integer&gt;</code> and a
      <em>no-ellipsis</em> value are specified. Any content that does not fit is
      truncated and no ellipsis is shown.
    </p>
  </div>
  <div class="card">
    <h2>custom string</h2>
    <p class="string">
      In this card, an <code>&lt;integer&gt;</code> and a <em>string</em> value
      are specified. Any content that does not fit is truncated and the custom
      string is shown instead of an ellipsis.
    </p>
  </div>
</section>
```

CSS

```
/* layout for the cards */
section {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 0.5rem;
  justify-content: space-between;
}
em {
  color: tomato;
}
.card {
  padding: 0.2rem;
  border: 1px solid tomato;
  border-radius: 0.2rem;
  max-width: 250px;
}
```

```
.integer {
  line-clamp: 2;
  overflow: hidden;
}
.no-ellipsis {
  line-clamp: 2 no-ellipsis;
  overflow: hidden;
}
.string {
  line-clamp: 2 "… (my custom text)";
  overflow: hidden;
}
```

Result

Truncating a paragraph with the legacy property

This example uses the legacy `-webkit-line-clamp` property with `display` set to `-webkit-box`.

> **Note:** When using the legacy `-webkit-line-clamp` property, you will also want to set `overflow` to `hidden`, otherwise the content is not clipped, although an ellipsis is still shown after the specified number of lines.

HTML

```
<p>
  In this example the <code>-webkit-line-clamp</code> property is set to
  <code>3</code>, which means the text is clamped after three lines. An ellipsis
  will be shown at the point where the text is clamped.
</p>
```

CSS

```
p {
  width: 300px;
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 3;
  overflow: hidden;
}
```

Result

See also

- Line Clampin' (Truncating Multiple Line Text)

line-height CSS property

The **`line-height`** CSS property sets the height of a line box in horizontal writing modes. In vertical writing modes, it sets the width of a line box. It's commonly used to set the distance between lines of text. On block-level elements in horizontal writing modes, it specifies the preferred height of line boxes within the element, and on non-replaced inline elements, it specifies the height that is used to calculate line box

height.

Try it

```
line-height: normal;
```

```
line-height: 2.5;
```

```
line-height: 3em;
```

```
line-height: 150%;
```

```
line-height: 32px;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    Far out in the uncharted backwaters of the unfashionable end of the western
    spiral arm of the Galaxy lies a small unregarded yellow sun.
  </div>
</section>
```

```
#example-element {
  font-family: "Georgia", serif;
  max-width: 200px;
}
```

Syntax

```
/* Keyword value */
line-height: normal;

/* Unitless values: use this number multiplied
by the element's font size */
line-height: 3.5;

/* <length> values */
line-height: 3em;

/* <percentage> values */
line-height: 34%;

/* Global values */
line-height: inherit;
line-height: initial;
line-height: revert;
line-height: revert-layer;
line-height: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

Depends on the user agent. Desktop browsers (including Firefox) use a default value of roughly **`1.2`**, depending on the element's `font-family`.

`<number>` (unitless)

The used value is this unitless `<number>` multiplied by the element's own font size. The computed value is the same as the specified `<number>`. In most cases, **this is the preferred way** to set `line-height` and avoid unexpected results due to inheritance.

`<length>`

The specified `<length>` is used in the calculation of the line box height. Values given in **em** units may produce unexpected results (see example below).

`<percentage>`

Relative to the font size of the element itself. The computed value is this `<percentage>` multiplied by the element's computed font size. **Percentage** values may produce unexpected results (see the second example below).

Accessibility

Use a minimum value of `1.5` for `line-height` for main paragraph content. This will help people experiencing low vision conditions, as well as people with cognitive concerns such as Dyslexia. If the page is zoomed to increase the text size, using a unitless value ensures that the line height will scale proportionately.

W3C Understanding WCAG 2.2

Formal definition

Initial value	`normal`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Percentages	refer to the font size of the element itself
Computed value	for percentage and length values, the absolute length, otherwise as specified
Animation type	either number or length

Formal syntax

```
line-height =
  normal                  |
  <number [0,∞]>          |
  <length-percentage [0,∞]>

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
/* All rules below have the same resultant line height */

/* number/unitless */
div {
  line-height: 1.2;
  font-size: 10pt;
}

/* length */
div {
  line-height: 1.2em;
  font-size: 10pt;
}

/* percentage */
div {
  line-height: 120%;
  font-size: 10pt;
}

/* font shorthand */
div {
  font:
    10pt/1.2 "Bitstream Charter",
    "Georgia",
    serif;
}
```

It is often more convenient to set `line-height` by using the `font` shorthand as shown above, but this requires the `font-family` property to be specified as well.

Prefer unitless numbers for line-height values

This example shows why it is better to use `<number>` values instead of `<length>` values. We will use two `<div>` elements. The first div, with the green border, uses a unitless `line-height` value. The second div, with the red border, uses a `line-height` value defined in `em`s.

HTML

```
<div class="box green">
  <h1>Avoid unexpected results by using unitless line-height.</h1>
  Length and percentage line-heights have poor inheritance behavior.
</div>

<div class="box red">
  <h1>Avoid unexpected results by using unitless line-height.</h1>
  Length and percentage line-heights have poor inheritance behavior
</div>

<!-- The first <h1> line-height is calculated from its own font-size   (30px × 1.1) = 33px -->
<!-- The second <h1> line-height results from the red div's font-size  (15px × 1.1) = 16.5px, probably not what you want -->
```

CSS

```
.green {
  line-height: 1.1;
  border: solid limegreen;
}

.red {
  line-height: 1.1em;
  border: solid red;
}

h1 {
  font-size: 30px;
}

.box {
  width: 18em;
  display: inline-block;
  vertical-align: top;
  font-size: 15px;
}
```

Result

Space between lines in vertical writing modes

The `line-height` property can be used to adjust space between vertical lines in vertical writing modes.

```
<div class="haiku">
  古池や蛙飛び込む水の音<br />
  ふるいけやかわずとびこむみずのおと<br />
  富士の風や扇にのせて江戸土産<br />
  ふじのかぜやおうぎにのせてえどみやげ<br />
</div>

<div class="haiku wide">
  古池や蛙飛び込む水の音<br />
  ふるいけやかわずとびこむみずのおと<br />
  富士の風や扇にのせて江戸土産<br />
```

```
  ふじのかぜやおうぎにのせてえどみやげ<br />
</div>
```

```
.haiku {
  border: 1px solid;
  margin-bottom: 1rem;
  padding: 0.5rem;
  background-color: wheat;

  writing-mode: vertical-rl;
}

.wide {
  line-height: 2;
}
```

Result

See also

- `font`, `font-size`
- Leading

line-height-step CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`line-height-step`** CSS property sets the step unit for line box heights. When the property is set, line box heights are rounded up to the closest multiple of the unit.

Syntax

```
/* Point values */
line-height-step: 18pt;

/* Global values */
line-height-step: inherit;
line-height-step: initial;
line-height-step: revert;
line-height-step: revert-layer;
line-height-step: unset;
```

Values

This property is specified as the following value:

`<length>`

The specified `<length>` is used in the calculation of the line box height step.

Formal definition

Initial value	`0`
Applies to	block containers
Inherited	yes
Computed value	absolute `<length>`
Animation type	by computed value type

Formal syntax

```
line-height-step =
  <length [0,∞]>
```

This syntax reflects the latest standard as per CSS Rhythmic Sizing Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting step unit for line box height

In the following example, the height of line box in each paragraph is rounded up to the step unit. The line box in `<h1>` does not fit into one step unit and thus occupies two, but it is still centered within the two step unit.

```
:root {
  font-size: 12pt;
  --my-grid: 18pt;
  line-height-step: var(--my-grid);
}
h1 {
  font-size: 20pt;
  margin-top: calc(2 * var(--my-grid));
}
```

The result of these rules is shown below in the following screenshot:

Specifications

Currently, no browsers support this feature.

See also

- `font`
- `font-size`
- `line-height`
- CSS Rhythmic Sizing specification

link-parameters CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`link-parameters`** CSS property sets values of external resources, such as SVGs, whose attributes have been set with the `env()` CSS function

Syntax

```
/* single value */
link-parameters: param(--color, red);

/* multiple values */
link-parameters:
  param(--color1, red),
  param(--color2, blue),
  param(--color3, green);
```

Values

`none`

No link parameters are specified.

`param()`

A list of one or more link parameters.

Formal definition

Initial value	none
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	discrete

Example

Updating the colors of an external SVG file

In this example the original SVG, on the left, is a square with `stroke` attribute set with `env(--color1, chartreuse)` and `fill` attribute set with `env(--color2, darkgreen)`. The `link-parameters` property is used to update both of these attributes on the updated square, on the right, with multiple `param()` CSS functions.

```
<div class="squares">
  <img
    class="original"
    src="square.svg"
    alt="A square with a chartreuse border and a dark green fill." />
  <img
    class="updated"
    src="square.svg"
    alt="A square with a red border and a tomato fill." />
</div>
```

```
.squares {
  height: 200px;
  display: flex;
  flex-direction: row;
  justify-content: space-between;
```

```
}
img {
  height: 100%;
}
```

```
.updated {
  link-parameters:
    param(--color1, red),
    param(--color2, tomato);
}
```

See also

- `param()`
- `env()`

list-style CSS property

The **`list-style`** CSS shorthand property allows you to set all the list style properties at once.

Try it

```
list-style: square;
```

```
list-style: inside;
```

```
list-style: url("/shared-assets/images/examples/rocket.svg");
```

```
list-style: none;
```

```
list-style: georgian inside url("/shared-assets/images/examples/rocket.svg");
```

```
list-style: georgian outside url("/non-existent.svg");
```

```
<section class="default-example" id="default-example">
  <div>
    <p>NASA Notable Missions</p>
    <ul class="transition-all" id="example-element">
      <li>Apollo</li>
      <li>Hubble</li>
      <li>Chandra</li>
      <li>Cassini-Huygens</li>
      <li>Spitzer</li>
    </ul>
  </div>
</section>
```

```
.default-example {
  font-size: 1.2rem;
}

#example-element {
  width: 100%;
  background: #be094b;
  color: white;
}

section {
  text-align: left;
  flex-direction: column;
}

hr {
  width: 50%;
  color: lightgray;
  margin: 0.5em;
}

.note {
  font-size: 0.8rem;
}

.note a {
  color: #009e5f;
}

@counter-style space-counter {
  symbols: "\1F680" "\1F6F8" "\1F6F0" "\1F52D";
  suffix: " ";
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `list-style-image`
- `list-style-position`
- `list-style-type`

Syntax

```
/* type */
list-style: square;

/* image */
list-style: url("../img/shape.png");
```

```
/* position */
list-style: inside;

/* two values */
list-style: georgian outside;
list-style: url("img/pip.svg") inside;

/* three values */
list-style: lower-roman url("img/shape.png") outside;

/* Keyword value */
list-style: none;

/* Global values */
list-style: inherit;
list-style: initial;
list-style: revert;
list-style: revert-layer;
list-style: unset;
```

The `list-style` property is specified as one, two, or three values in any order. If `list-style-type` and `list-style-image` are both set, the `list-style-type` is used as a fallback if the image is unavailable.

Values

`list-style-type`

A `<counter-style>`, `<string>`, or `none`. If omitted in the shorthand, it defaults to `disc`. See `list-style-type`.

`list-style-image`

An `<image>` or `none`. If omitted, it defaults to `none`. See `list-style-image`.

`list-style-position`

Either `inside` or `outside`. If omitted, it defaults to `outside`. See `list-style-position`.

`none`

No list style is used.

Description

The `list-style` property lets you customize the appearance of list items. The values of this property are applied to list items, including `<li>` elements and elements with `display: list-item;`.

List style inheritance

Because the `list-style-type` longhand property is inherited, `list-style` can be set on a parent element (normally `<ol>` or `<ul>`) to make the same list styling apply to its list items. However, user-agent stylesheets set `list-style-type` on `<ul>`, `<ol>`, and `<menu>` elements, so nested lists use these default values instead of inheriting the ancestor list's `list-style`. The `list-style-position` and `list-style-image` values normally inherit into nested lists. To make nested lists inherit the marker type as well, set `list-style` or `list-style-type` to `inherit` on the nested list elements.

For example, in cases like this:

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>
    Three
    <ul>
      <li>Four</li>
      <li>Five</li>
      <li>Six</li>
    </ul>
  </li>
</ul>
```

Where a custom style is set on the outer list:

```
body > ul {
  list-style: square;
}
```

The inner list won't inherit the custom style unless you set it to inherit:

```
ul ul {
  list-style: inherit;
}
```

Accessibility

Safari does not recognize ordered or unordered lists as lists in the accessibility tree if they have a `list-style` value of `none`, unless the list is nested within the `<nav>` navigation element. This behavior is intentional and is not considered a bug.

To ensure lists are announced as lists, include `role="list"` to `<ol>` and `<ul>` elements, especially if the list is not nested in a `<nav>`. This restores list semantics without affecting the design:

```
<ul role="list">
  <li>An item</li>
  <li>Another item</li>
</ul>
```

If an ARIA `role` is not an option for your code, CSS can be used instead. Adding non-empty pseudo-content such as text or images before each list item can restore list semantics, but impacts the visual appearance. Safari determines if the added pseudo-content suffices as accessible content, restoring list semantics if so. Generally, Safari considers text and images as sufficient, which is why the `content: "+ ";` shown below works (but requires additional styling to not affect the design).

```
ul {
  list-style: none;
}

ul li::before {
```

```
  content: "+ ";
}
```

A declaration of `content: "";` (an empty string) is ignored, as are `content` values that contain only spaces, such as `content: " ";`.

These CSS workarounds should only be used when an HTML solution is unavailable, and only after testing to ensure that they don't result in unexpected behaviors that may negatively impact user experience.

- 'Fixing' Lists (2023)
- VoiceOver and list-style-type: none (2017)
- Understanding WCAG: Create content that can be presented in different ways
- Understanding success criterion 1.3.1: Info and relationships | WCAG 2.1

Formal definition

Initial value	as each of the properties of the shorthand: • `list-style-type`: `disc` • `list-style-position`: `outside` • `list-style-image`: `none`
Applies to	list items
Inherited	yes
Computed value	as each of the properties of the shorthand: • `list-style-image`: The keyword `none` or the computed <image> • `list-style-position`: as specified • `list-style-type`: as specified
Animation type	as each of the properties of the shorthand: • `list-style-image`: discrete • `list-style-position`: discrete • `list-style-type`: discrete

Formal syntax

```
list-style =
  <'list-style-position'>  ||
  <'list-style-image'>     ||
  <'list-style-type'>

<list-style-position> =
  inside   |
  outside

<list-style-image> =
  <image>  |
  none

<list-style-type> =
  <counter-style>  |
  <string>         |
  none

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<counter-style> =
  <counter-style-name>  |
  <symbols()>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<counter-style-name> =
  <custom-ident>

<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

```
<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting list style type and position

HTML

```
List 1
<ul class="one">
  <li>List Item1</li>
  <li>List Item2</li>
  <li>List Item3</li>
</ul>
List 2
<ul class="two">
  <li>List Item A</li>
  <li>List Item B</li>
  <li>List Item C</li>
</ul>
```

CSS

```
.one {
  list-style: circle;
}

.two {
  list-style: square inside;
}
```

Result

See also

- Component properties: `list-style-type`, `list-style-image`, and `list-style-position`
- `::marker` pseudo-element
- CSS lists and counters module
- CSS counter styles module

list-style-image CSS property

The **`list-style-image`** CSS property sets an image to be used as the list item marker.

It is often more convenient to use the shorthand `list-style`.

Try it

```
list-style-image: url("/shared-assets/images/examples/rocket.svg");
```

```
list-style-image: none;
```

```
<section class="default-example" id="default-example">
  <div>
    <p>NASA Notable Missions</p>
    <ul class="transition-all unhighlighted" id="example-element">
      <li>Apollo</li>
      <li>Hubble</li>
      <li>Chandra</li>
      <li>Cassini-Huygens</li>
      <li>Spitzer</li>
    </ul>
  </div>
</section>
```

```
.default-example {
  font-size: 1.2rem;
}

#example-element {
  width: 100%;
  background: #be094b;
  color: white;
}

section {
  text-align: left;
  flex-direction: column;
}

hr {
  width: 50%;
  color: lightgray;
  margin: 0.5em;
}

.note {
  font-size: 0.8rem;
}

.note a {
```

```
  color: #009e5f;
}

@counter-style space-counter {
  symbols: "\1F680" "\1F6F8" "\1F6F0" "\1F52D";
  suffix: " ";
}
```

> **Note:** This property is applied to list items, i.e., elements with `display: list-item;` by default this includes `<li>` elements. Because this property is inherited, it can be set on the parent element (normally `<ol>` or `<ul>`) to let it apply to all list items.

Syntax

```
/* Keyword value */
list-style-image: none;

/* <url> values */
list-style-image: url("star-solid.gif");

/* valid image values */
list-style-image: linear-gradient(to left bottom, red, blue);

/* Global values */
list-style-image: inherit;
list-style-image: initial;
list-style-image: revert;
list-style-image: revert-layer;
list-style-image: unset;
```

Values

This property is specified as a single value from the following list:

`<image>`

A valid image to use as the marker.

`none`

Specifies that no image is used as the marker. If this value is set, the marker defined in `list-style-type` will be used instead. This is the default value for `list-style`.

Formal definition

Initial value	none
Applies to	list items
Inherited	yes
Computed value	The keyword none or the computed <image>
Animation type	discrete

Formal syntax

```
list-style-image =
  <image>  |
  none

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using a url value

This example has a star as a marker, which we include using the `<url>` image function.

HTML

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

CSS

```
ul {
  list-style-image: url("star-solid.gif");
}
```

Result

Using a gradient

This example has a CSS gradient as a marker, which we create uses the `linear-gradient()` image function.

HTML

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

CSS

```
ul {
  font-size: 200%;
  list-style-image: linear-gradient(to left bottom, red, blue);
}
```

Result

See also

- `list-style` shorthand
- `list-style-type` property
- `list-style-position` property
- `::marker` pseudo-element
- CSS lists and counters module
- CSS counter styles module

list-style-position CSS property

The **`list-style-position`** CSS property sets the position of the `::marker` relative to a list item.

Try it

```
list-style-position: inside;
```

```
list-style-position: outside;
```

```
<section class="default-example" id="default-example">
  <div>
    <p>NASA Notable Missions</p>
    <ul class="transition-all" id="example-element">
      <li>Apollo 11: First Human Landing</li>
      <li>City in Space: The International Space Station</li>
      <li>Great Observatory: The Hubble Space Telescope</li>
      <li>Everlasting Mars Rovers</li>
    </ul>
  </div>
</section>
```

```
.default-example {
  font-size: 1.2rem;
}

#example-element {
  width: 100%;
  background: #be094b;
  color: white;
}

section {
  text-align: left;
  flex-direction: column;
}

hr {
  width: 50%;
  color: lightgray;
  margin: 0.5em;
}

.note {
  font-size: 0.8rem;
}

.note a {
  color: #009e5f;
```

```
}

@counter-style space-counter {
  symbols: "\1F680" "\1F6F8" "\1F6F0" "\1F52D";
  suffix: " ";
}
```

Syntax

```
/* Keyword values */
list-style-position: inside;
list-style-position: outside;

/* Global values */
list-style-position: inherit;
list-style-position: initial;
list-style-position: revert;
list-style-position: revert-layer;
list-style-position: unset;
```

Values

This property is specified as one of the following keyword values:

`inside`

> The `::marker` is the first element among the list item's contents.

`outside`

> The `::marker` is outside the principal block box. This is the default value for `list-style`.

Description

This property is applied to list items, i.e., elements with `display: list-item;`. By default this includes `<li>` elements. Because this property is inherited, it can be set on the parent element (normally `<ol>` or `<ul>`) to let it apply to all list items.

If a block element is the first child of a list element declared as `list-style-position: inside`, then the block element is placed on the line after the marker-box.

It is often more convenient to use the shorthand `list-style`.

Formal definition

Initial value	`outside`
Applies to	list items
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
list-style-position =
  inside   |
  outside
```

This syntax reflects the latest standard as per CSS Lists and Counters Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting list item position

HTML

```
<p>List 1</p>
<ul class="inside">
  <li>List Item 1-1</li>
  <li>List Item 1-2</li>
  <li>List Item 1-3</li>
  <li>List Item 1-4</li>
</ul>
<p>List 2</p>
<ul class="outside">
  <li>List Item 2-1</li>
  <li>List Item 2-2</li>
  <li>List Item 2-3</li>
  <li>List Item 2-4</li>
</ul>
<p>List 3</p>
<ul class="inside-img">
  <li>List Item 3-1</li>
  <li>List Item 3-2</li>
  <li>List Item 3-3</li>
  <li>List Item 3-4</li>
</ul>
```

CSS

```
.inside {
  list-style-position: inside;
  list-style-type: square;
}

.outside {
  list-style-position: outside;
  list-style-type: circle;
}

.inside-img {
  list-style-position: inside;
```

```
  list-style-image: url("star-solid.gif");
}
```

Result

See also

- `list-style` shorthand
- `list-style-type` property
- `list-style-image` property
- `::marker` pseudo-element
- CSS lists and counters module
- CSS counter styles module

list-style-type CSS property

The **`list-style-type`** CSS property sets the marker (such as a disc, character, or custom counter style) of a list item element.

Try it

```
list-style-type: space-counter;
```

```
list-style-type: disc;
```

```
list-style-type: circle;
```

```
list-style-type: "\1F44D";
```

```
<section class="default-example" id="default-example">
  <div>
    <p>NASA Notable Missions</p>
    <ul class="transition-all unhighlighted" id="example-element">
      <li>Apollo</li>
      <li>Hubble</li>
      <li>Chandra</li>
      <li>Cassini-Huygens</li>
    </ul>
  </div>
  <hr />
  <div class="note">
    <p>
      <code>space-counter</code> is defined with
      <a
        href="//developer.mozilla.org/docs/Web/CSS/Reference/At-rules/@counter-style"
        target="_parent"
        ><code>@counter-style</code></a
      >
    </p>
  </div>
</section>
```

```
.default-example {
  font-size: 1.2rem;
}

#example-element {
  width: 100%;
  background: #be094b;
  color: white;
}

section {
  text-align: left;
  flex-direction: column;
}

hr {
  width: 50%;
  color: lightgray;
  margin: 0.5em;
}

.note {
  font-size: 0.8rem;
}

.note a {
  color: #009e5f;
}

@counter-style space-counter {
  symbols: "\1F680" "\1F6F8" "\1F6F0" "\1F52D";
  suffix: " ";
}
```

Syntax

```
/* Partial list of types */
list-style-type: disc;
list-style-type: circle;
list-style-type: square;
list-style-type: decimal;
list-style-type: georgian;
list-style-type: trad-chinese-informal;
list-style-type: kannada;

/* <string> value */
list-style-type: "-";
```

```
/* Identifier matching an @counter-style rule */
list-style-type: custom-counter-style;

/* Keyword value */
list-style-type: none;

/* Global values */
list-style-type: inherit;
list-style-type: initial;
list-style-type: revert;
list-style-type: revert-layer;
list-style-type: unset;
```

The `list-style-type` property may be defined as any one of:

- a `<custom-ident>` value,
- a `symbols()` value,
- a `<string>` value, or
- the keyword `none`.

Note that:

- Some types require a suitable font installed to display as expected.
- The `cjk-ideographic` is identical to `trad-chinese-informal`; it exists for legacy reasons.

Values

`<custom-ident>`

An identifier matching the value of a `@counter-style` or one of the predefined styles:

`symbols()`

Defines an anonymous style of the list.

`<string>`

The specified string will be used as the item's marker.

`none`

No item marker is shown.

`disc`

A filled circle (default value).

`circle`

A hollow circle.

`square`

A filled square.

`decimal`

Decimal numbers, beginning with 1.

`cjk-decimal`

Han decimal numbers.

`decimal-leading-zero`

Decimal numbers, padded by initial zeros.

`lower-roman`

Lowercase roman numerals.

`upper-roman`

Uppercase roman numerals.

`lower-greek`

Lowercase classical Greek.

`lower-alpha`, `lower-latin`

Lowercase ASCII letters.

`upper-alpha`, `upper-latin`

Uppercase ASCII letters.

`arabic-indic`, `-moz-arabic-indic`

Arabic-Indic numbers.

`armenian`

Traditional Armenian numbering.

`bengali`, `-moz-bengali`

Bengali numbering.

`cambodian/khmer`

Cambodian/Khmer numbering.

`cjk-earthly-branch`, `-moz-cjk-earthly-branch`

Han "Earthly Branch" ordinals.

`cjk-heavenly-stem`, `-moz-cjk-heavenly-stem`

Han "Heavenly Stem" ordinals.

`cjk-ideographic`

Identical to `trad-chinese-informal`.

`devanagari`, `-moz-devanagari`

Devanagari numbering.

`ethiopic-numeric`

Ethiopic numbering.

`georgian`

Traditional Georgian numbering.

`gujarati`, `-moz-gujarati`

Gujarati numbering.

`gurmukhi`, `-moz-gurmukhi`

Gurmukhi numbering.

`hebrew`

Traditional Hebrew numbering.

`hiragana`

Dictionary-order hiragana lettering.

`hiragana-iroha`

Iroha-order hiragana lettering.

`japanese-formal`

Japanese formal numbering to be used in legal or financial documents. The kanjis are designed so that they can't be modified to look like another correct one.

`japanese-informal`

Japanese informal numbering.

`kannada`, `-moz-kannada`

Kannada numbering.

`katakana`

Dictionary-order katakana lettering.

`katakana-iroha`

Iroha-order katakana lettering.

`korean-hangul-formal`

Korean hangul numbering.

`korean-hanja-formal`

Formal Korean Han numbering.

`korean-hanja-informal`

Korean hanja numbering.

`lao`, `-moz-lao`

Laotian numbering.

`lower-armenian`

Lowercase Armenian numbering.

`malayalam`, `-moz-malayalam`

Malayalam numbering.

`mongolian`

Mongolian numbering.

`myanmar`, `-moz-myanmar`

Myanmar (Burmese) numbering.

`oriya`, `-moz-oriya`

Oriya numbering.

`persian`, `-moz-persian`

Persian numbering.

`simp-chinese-formal`

Simplified Chinese formal numbering.

`simp-chinese-informal`

Simplified Chinese informal numbering.

`tamil`, `-moz-tamil`

Tamil numbering.

`telugu`, `-moz-telugu`

Telugu numbering.

`thai`, `-moz-thai`

Thai numbering.

`tibetan`

Tibetan numbering.

`trad-chinese-formal`

Traditional Chinese formal numbering.

`trad-chinese-informal`

Traditional Chinese informal numbering.

`upper-armenian`

Traditional uppercase Armenian numbering.

`disclosure-open`

Symbol indicating that a disclosure widget such as `<details>` is opened.

`disclosure-closed`

Symbol indicating that a disclosure widget, like `<details>` is closed.

Refer to the All list style types example to see the above values in action. Details of all the available counter styles used by various cultures around the world can be found in Ready-made Counter Styles.

Non-standard extensions

Mozilla (Firefox) supports a few predefined types with a `-moz-` prefix.

- `ethiopic-halehame`: `-moz-ethiopic-halehame`
- `ethiopic-halehame-am`: `-moz-ethiopic-halehame-am`
- `ethiopic-halehame-ti-er`: `-moz-ethiopic-halehame-ti-er`
- `ethiopic-halehame-ti-et`: `-moz-ethiopic-halehame-ti-et`
- `ethiopic-numeric`: `-moz-ethiopic-numeric`
- `hangul`: `-moz-hangul`
- `hangul-consonant`: `-moz-hangul-consonant`
- `urdu`: `-moz-urdu`

See the compatibility table to check which browsers support which extension.

Description

The `list-style-type` property sets the marker (such as a disc, character, or custom counter style) of a list item element. The marker will be `currentColor`, the same as the computed color of the element it applies to.

Only a few elements (`<li>` and `<summary>`) have a default value of `display: list-item`. However, the `list-style-type` property may be applied to any element whose `display` value is set to `list-item`.

List style inheritance

Because `list-style-type` is inherited, it can be set on a parent element (commonly `<ol>` or `<ul>`) to make it apply to its list items. However, user-agent stylesheets set `list-style-type` on `<ul>`, `<ol>`, and `<menu>` elements, so nested lists use these default values instead of inheriting the ancestor list's value. To make nested lists inherit this value, set `list-style-type: inherit` on the nested list elements.

For example, in cases like this:

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>
    Three
    <ul>
      <li>Four</li>
      <li>Five</li>
      <li>Six</li>
    </ul>
  </li>
</ul>
```

Where a custom style is set on the outer list:

```
body > ul {
  list-style-type: square;
}
```

The inner list won't inherit the custom style unless you set it to inherit:

```
ul ul {
  list-style-type: inherit;
}
```

Accessibility

Safari will not recognize an ordered or unordered list as a list in the accessibility tree if it has a `list-style-type` value of `none`. This can be resolved by adding `role="list"` to the list's opening tag. To learn more about this and potential workarounds, see `list-style`.

Formal definition

Initial value	`disc`
Applies to	list items
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
list-style-type =
  <counter-style>  |
  <string>         |
  none

<counter-style> =
  <counter-style-name>  |
  <symbols()>

<counter-style-name> =
  <custom-ident>

<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting list item markers

HTML

```
List 1
<ol class="normal">
  <li>Hello</li>
  <li>World</li>
  <li>What's up?</li>
</ol>

List 2
<ol class="shortcut">
  <li>Looks</li>
  <li>Like</li>
  <li>The</li>
  <li>Same</li>
</ol>
```

CSS

```
ol.normal {
  list-style-type: upper-alpha;
}

/* or use the shortcut "list-style": */
ol.shortcut {
```

```
  list-style: upper-alpha;
}
```

Result

All list style types

HTML

```
<ol>
  <li>Apollo</li>
  <li>Hubble</li>
  <li>Chandra</li>
  <li>Cassini-Huygens</li>
  <li>Spitzer</li>
</ol>

<h2>Choose a list style type:</h2>

<div class="container">
  <label for="disc">
    <input type="radio" id="disc" name="type" value="disc" />disc
  </label>

  <label for="circle">
    <input type="radio" id="circle" name="type" value="circle" />circle
  </label>

  <label for="square">
    <input type="radio" id="square" name="type" value="square" />square
  </label>

  <label for="decimal">
    <input type="radio" id="decimal" name="type" value="decimal" />decimal
  </label>

  <label for="cjk-decimal">
    <input
      type="radio"
      id="cjk-decimal"
      name="type"
      value="cjk-decimal" />cjk-decimal
  </label>

  <label for="decimal-leading-zero">
    <input
      type="radio"
      id="decimal-leading-zero"
      name="type"
      value="decimal-leading-zero" />decimal-leading-zero
  </label>

  <label for="lower-roman">
    <input
      type="radio"
      id="lower-roman"
      name="type"
      value="lower-roman" />lower-roman
  </label>

  <label for="upper-roman">
    <input
      type="radio"
      id="upper-roman"
      name="type"
      value="upper-roman" />upper-roman
  </label>

  <label for="lower-greek">
    <input
      type="radio"
      id="lower-greek"
      name="type"
      value="lower-greek" />lower-greek
  </label>

  <label for="lower-alpha">
    <input
      type="radio"
      id="lower-alpha"
      name="type"
      value="lower-alpha" />lower-alpha, lower-latin
  </label>

  <label for="upper-alpha">
    <input
      type="radio"
      id="upper-alpha"
      name="type"
      value="upper-alpha" />upper-alpha, upper-latin
  </label>

  <label for="arabic-indic">
    <input
      type="radio"
      id="arabic-indic"
      name="type"
      value="arabic-indic" />arabic-indic
  </label>

  <label for="armenian">
    <input type="radio" id="armenian" name="type" value="armenian" />armenian
  </label>
```

```
<label for="bengali">
  <input type="radio" id="bengali" name="type" value="bengali" />bengali
</label>

<label for="cambodian">
  <input type="radio" id="cambodian" name="type" value="cambodian" />cambodian
</label>

<label for="cjk-earthly-branch">
  <input
    type="radio"
    id="cjk-earthly-branch"
    name="type"
    value="cjk-earthly-branch" />cjk-earthly-branch
</label>

<label for="cjk-heavenly-stem">
  <input
    type="radio"
    id="cjk-heavenly-stem"
    name="type"
    value="cjk-heavenly-stem" />cjk-heavenly-stem
</label>

<label for="cjk-ideographic">
  <input
    type="radio"
    id="cjk-ideographic"
    name="type"
    value="cjk-ideographic" />cjk-ideographic
</label>

<label for="devanagari">
  <input
    type="radio"
    id="devanagari"
    name="type"
    value="devanagari" />devanagari
</label>

<label for="ethiopic-numeric">
  <input
    type="radio"
    id="ethiopic-numeric"
    name="type"
    value="ethiopic-numeric" />ethiopic-numeric
</label>

<label for="georgian">
  <input type="radio" id="georgian" name="type" value="georgian" />georgian
</label>

<label for="gujarati">
  <input type="radio" id="gujarati" name="type" value="gujarati" />gujarati
</label>

<label for="gurmukhi">
  <input type="radio" id="gurmukhi" name="type" value="gurmukhi" />gurmukhi
</label>

<label for="hebrew">
  <input type="radio" id="hebrew" name="type" value="hebrew" />hebrew
</label>

<label for="hiragana">
  <input type="radio" id="hiragana" name="type" value="hiragana" />hiragana
</label>

<label for="hiragana-iroha">
  <input
    type="radio"
    id="hiragana-iroha"
    name="type"
    value="hiragana-iroha" />hiragana-iroha
</label>

<label for="japanese-formal">
  <input
    type="radio"
    id="japanese-formal"
    name="type"
    value="japanese-formal" />japanese-formal
</label>

<label for="japanese-informal">
  <input
    type="radio"
    id="japanese-informal"
    name="type"
    value="japanese-informal" />japanese-informal
</label>

<label for="kannada">
  <input type="radio" id="kannada" name="type" value="kannada" />kannada
</label>

<label for="katakana">
  <input type="radio" id="katakana" name="type" value="katakana" />katakana
</label>

<label for="katakana-iroha">
  <input
```

```
    type="radio"
    id="katakana-iroha"
    name="type"
    value="katakana-iroha" />katakana-iroha
</label>

<label for="khmer">
  <input type="radio" id="khmer" name="type" value="khmer" />khmer
</label>

<label for="korean-hangul-formal">
  <input
    type="radio"
    id="korean-hangul-formal"
    name="type"
    value="korean-hangul-formal" />korean-hangul-formal
</label>

<label for="korean-hanja-formal">
  <input
    type="radio"
    id="korean-hanja-formal"
    name="type"
    value="korean-hanja-formal" />korean-hanja-formal
</label>

<label for="korean-hanja-informal">
  <input
    type="radio"
    id="korean-hanja-informal"
    name="type"
    value="korean-hanja-informal" />korean-hanja-informal
</label>

<label for="lao">
  <input type="radio" id="lao" name="type" value="lao" />lao
</label>

<label for="lower-armenian">
  <input
    type="radio"
    id="lower-armenian"
    name="type"
    value="lower-armenian" />lower-armenian
</label>

<label for="malayalam">
  <input type="radio" id="malayalam" name="type" value="malayalam" />malayalam
</label>

<label for="mongolian">
  <input type="radio" id="mongolian" name="type" value="mongolian" />mongolian
</label>

<label for="myanmar">
  <input type="radio" id="myanmar" name="type" value="myanmar" />myanmar
</label>

<label for="oriya">
  <input type="radio" id="oriya" name="type" value="oriya" />oriya
</label>

<label for="persian">
  <input type="radio" id="persian" name="type" value="persian" />persian
</label>

<label for="simp-chinese-formal">
  <input
    type="radio"
    id="simp-chinese-formal"
    name="type"
    value="simp-chinese-formal" />simp-chinese-formal
</label>

<label for="simp-chinese-informal">
  <input
    type="radio"
    id="simp-chinese-informal"
    name="type"
    value="simp-chinese-informal" />simp-chinese-informal
</label>

<label for="tamil">
  <input type="radio" id="tamil" name="type" value="tamil" />tamil
</label>

<label for="telugu">
  <input type="radio" id="telugu" name="type" value="telugu" />telugu
</label>

<label for="thai">
  <input type="radio" id="thai" name="type" value="thai" />thai
</label>

<label for="tibetan">
  <input type="radio" id="tibetan" name="type" value="tibetan" />tibetan
</label>

<label for="trad-chinese-formal">
  <input
    type="radio"
    id="trad-chinese-formal"
    name="type"
```

```
    value="trad-chinese-formal" />trad-chinese-formal
</label>

<label for="trad-chinese-informal">
  <input
    type="radio"
    id="trad-chinese-informal"
    name="type"
    value="trad-chinese-informal" />trad-chinese-informal
</label>

<label for="upper-armenian">
  <input
    type="radio"
    id="upper-armenian"
    name="type"
    value="upper-armenian" />upper-armenian
</label>

<label for="disclosure-open">
  <input
    type="radio"
    id="disclosure-open"
    name="type"
    value="disclosure-open" />disclosure-open
</label>

<label for="disclosure-closed">
  <input
    type="radio"
    id="disclosure-closed"
    name="type"
    value="disclosure-closed" />disclosure-closed
</label>

<label for="moz-ethiopic-halehame">
  <input
    type="radio"
    id="moz-ethiopic-halehame"
    name="type"
    value="-moz-ethiopic-halehame" />-moz-ethiopic-halehame
</label>

<label for="moz-ethiopic-halehame-am">
  <input
    type="radio"
    id="moz-ethiopic-halehame-am"
    name="type"
    value="-moz-ethiopic-halehame-am" />-moz-ethiopic-halehame-am
</label>

<label for="ethiopic-halehame-ti-er">
  <input
    type="radio"
    id="ethiopic-halehame-ti-er"
    name="type"
    value="ethiopic-halehame-ti-er" />ethiopic-halehame-ti-er
</label>

<label for="ethiopic-halehame-ti-et">
  <input
    type="radio"
    id="ethiopic-halehame-ti-et"
    name="type"
    value="ethiopic-halehame-ti-et" />ethiopic-halehame-ti-et
</label>

<label for="hangul">
  <input type="radio" id="hangul" name="type" value="hangul" />hangul
</label>

<label for="hangul-consonant">
  <input
    type="radio"
    id="hangul-consonant"
    name="type"
    value="hangul-consonant" />hangul-consonant
</label>

<label for="urdu">
  <input type="radio" id="urdu" name="type" value="urdu" />urdu
</label>

<label for="moz-ethiopic-halehame-ti-er">
  <input
    type="radio"
    id="moz-ethiopic-halehame-ti-er"
    name="type"
    value="-moz-ethiopic-halehame-ti-er" />-moz-ethiopic-halehame-ti-er
</label>

<label for="moz-ethiopic-halehame-ti-et">
  <input
    type="radio"
    id="moz-ethiopic-halehame-ti-et"
    name="type"
    value="-moz-ethiopic-halehame-ti-et" />-moz-ethiopic-halehame-ti-et
</label>

<label for="moz-hangul">
  <input
    type="radio"
    id="moz-hangul"
```

```
      name="type"
      value="-moz-hangul" />-moz-hangul
  </label>

  <label for="moz-hangul-consonant">
    <input
      type="radio"
      id="moz-hangul-consonant"
      name="type"
      value="-moz-hangul-consonant" />-moz-hangul-consonant
  </label>

  <label for="moz-urdu">
    <input type="radio" id="moz-urdu" name="type" value="-moz-urdu" />-moz-urdu
  </label>
</div>
```

CSS

```
ol {
  font-size: 1.2rem;
}

.container {
  column-count: 3;
}

label {
  display: block;
}

input {
  margin: 0.4rem;
}
```

JavaScript

```
const container = document.querySelector(".container");
const list = document.querySelector("ol");

container.addEventListener("change", (event) => {
  list.setAttribute("style", `list-style-type: ${event.target.value}`);
});
```

Result

We're not limited to the list style types defined on this page or the specification. The `@counter-style` at-rule enables creating counters using any alphabet.

See also

- `list-style` shorthand property
- `list-style-image` property
- `list-style-position` property
- `::marker` pseudo-element
- CSS lists and counters module
- CSS counter styles module

margin CSS property

The **`margin`** CSS shorthand property sets the margin area on all four sides of an element.

Try it

```
margin: 1em;
```

```
margin: 5% 0;
```

```
margin: 10px 50px 20px;
```

```
margin: 10px 50px 20px 0;
```

```
margin: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="row"></div>
    <div class="row transition-all" id="example-element"></div>
    <div class="row"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  flex-shrink: 0;
}
```

```
#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `margin-top`
- `margin-right`
- `margin-bottom`
- `margin-left`

Syntax

```
/* apply to all four sides */
margin: 1em;
margin: -3px;

/* top and bottom | left and right */
margin: 5% auto;

/* top | left and right | bottom */
margin: 1em auto 2em;

/* top | right | bottom | left */
margin: 2px 1em 0 auto;

/* anchor-size() values */
margin: 5% anchor-size(width);
margin: calc(anchor-size(width) / 4) 1em 0
  anchor-size(--my-anchor self-inline, 50px);

/* Keyword value */
margin: auto;

/* Global values */
margin: inherit;
margin: initial;
margin: revert;
margin: revert-layer;
margin: unset;
```

The `margin` property may be specified using one, two, three, or four values. Each value is a `<length>`, a `<percentage>`, or the keyword `auto`. Negative values draw the element closer to its neighbors than it would be by default.

- When **one** value is specified, it applies the same margin to **all four sides**.
- When **two** values are specified, the first margin applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first margin applies to the **top**, the second to the **right and left**, the third to the **bottom**.
- When **four** values are specified, the margins apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<length>`

The size of the margin as a fixed value.

- For *anchor-positioned elements*, the `anchor-size()` function resolves to a `<length>` value relative to the associated *anchor element*'s width or height (see Setting element margin based on anchor size).

`<percentage>`

The size of the margin as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block.

`auto`

The browser selects a suitable margin to use. For example, in certain cases this value can be used to center an element.

Description

This property can be used to set a margin on all four sides of an element. Margins create extra space *around* an element, unlike `padding`, which creates extra space *within* an element.

The top and bottom margins have no effect on *non-replaced* inline elements, such as `<span>` or `<code>`.

Horizontal centering

You can horizontally center an element within its parent by setting `margin: 0 auto;`.

A more common method to center an element horizontally is by setting `display: flex;` and `justify-content: center;` on a container, which centers its flex item children.

Margin collapsing

Elements' top and bottom margins are sometimes collapsed into a single margin that is equal to the larger of the two margins. See Mastering margin collapsing for more information.

Formal definition

Initial value	as each of the properties of the shorthand: • `margin-bottom`: `0` • `margin-left`: `0` • `margin-right`: `0` • `margin-top`: `0`
Applies to	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	as each of the properties of the shorthand:

	• `margin-bottom`: the percentage as specified or the absolute length • `margin-left`: the percentage as specified or the absolute length • `margin-right`: the percentage as specified or the absolute length • `margin-top`: the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
margin =
  <'margin-top'>{1,4}

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

HTML

```
<div class="center">This element is centered.</div>

<div class="outside">This element is positioned outside of its container.</div>
```

CSS

```
.center {
  margin: auto;
  background: lime;
  width: 66%;
}

.outside {
  margin: 3rem 0 0 -3rem;
  background: cyan;
  width: 66%;
}
```

More examples

```
margin: 5%; /* All sides: 5% margin */

margin: 10px; /* All sides: 10px margin */

margin: 1.6em 20px; /* top and bottom: 1.6em margin */
/* left and right: 20px margin */

margin: 10px 3% -1em; /* top:           10px margin */
/* left and right: 3% margin   */
/* bottom:         -1em margin */

margin: 10px 3px 30px 5px; /* top:    10px margin */
/* right:  3px margin  */
/* bottom: 30px margin */
/* left:   5px margin  */

margin: 2em auto; /* top and bottom: 2em margin   */
/* Box is horizontally centered */

margin: auto; /* top and bottom: 0 margin     */
/* Box is horizontally centered */
```

See also

- `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `margin-block-start`, `margin-block-end`, `margin-inline-start`, and `margin-inline-end`
- `margin-block` and `margin-inline` shorthands
- Mastering margin collapsing
- Introduction to the CSS box model guide
- CSS box model module

margin-block CSS property

The **`margin-block`** CSS shorthand property defines the logical block start and end margins of an element, which maps to physical margins depending on the element's writing mode, directionality, and text orientation.

Try it

```
margin-block: 10px 20px;
writing-mode: horizontal-tb;
```

```
margin-block: 20px 40px;
writing-mode: vertical-rl;
```

```
margin-block: 5% 20%;
writing-mode: horizontal-tb;
```

```
margin-block: 1rem auto;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div id="container">
    <div class="row">One</div>
    <div class="row transition-all" id="example-element">Two</div>
    <div class="row">Three</div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  color: white;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `margin-block-start`
- `margin-block-end`

Syntax

```
/* <length> values */
margin-block: 10px 20px; /* An absolute length */
margin-block: 1em 2em; /* relative to the text size */
margin-block: 5% 2%; /* relative to the nearest block container's width */
margin-block: 10px; /* sets both start and end values */
margin-block: anchor-size(inline);
margin-block: calc(anchor-size(width) / 4) 1em;

/* Keyword value */
margin-block: auto;

/* Global values */
margin-block: inherit;
margin-block: initial;
margin-block: revert;
margin-block: revert-layer;
margin-block: unset;
```

This property corresponds to the `margin-top` and `margin-bottom`, or the `margin-right` and `margin-left` properties, depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The `margin-block` property may be specified using one or two values.

- When **one** value is specified, it applies the same margin to **both start and end**.
- When **two** values are specified, the first margin applies to the **start**, the second to the **end**.

Values

The `margin-block` property takes the same values as the `margin-top` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `margin-block-start`: `0` • `margin-block-end`: `0`
Applies to	same as `margin`
Inherited	no
Percentages	depends on layout model
Computed value	as each of the properties of the shorthand: • `margin-block-start`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`

	• `margin-block-end`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length

Formal syntax

```
margin-block =
  <'margin-top'>{1,2}

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block start and end margins

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: auto;
  border: 1px solid green;
}

p {
  margin: 0;
  margin-block: 20px 40px;
  background-color: tan;
}

.verticalExample {
  writing-mode: vertical-rl;
}
```

HTML

```
<div>
  <p>Example text</p>
</div>
<div class="verticalExample">
  <p>Example text</p>
</div>
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `writing-mode`, `direction`, `text-orientation`

margin-block-end CSS property

The **`margin-block-end`** CSS property defines the logical block end margin of an element, which maps to a physical margin depending on the element's writing mode, directionality, and text orientation.

Try it

```
margin-block-end: 20px;
writing-mode: horizontal-tb;
```

```
margin-block-end: 20px;
writing-mode: vertical-rl;
```

```
margin-block-end: 20%;
writing-mode: horizontal-tb;
```

```
margin-block-end: auto;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div id="container">
    <div class="row">One</div>
    <div class="row transition-all" id="example-element">Two</div>
    <div class="row">Three</div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  color: white;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

Syntax

```
/* <length> values */
margin-block-end: 10px; /* An absolute length */
margin-block-end: 1em; /* relative to the text size */
margin-block-end: 5%; /* relative to the nearest block container's width */
margin-block-end: anchor-size(inline);
margin-block-end: calc(anchor-size(--my-anchor block, 20px) / 4);

/* Keyword value */
margin-block-end: auto;

/* Global values */
margin-block-end: inherit;
margin-block-end: initial;
margin-block-end: revert;
margin-block-end: revert-layer;
margin-block-end: unset;
```

It corresponds to the `margin-top`, `margin-right`, `margin-bottom`, or `margin-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

It relates to `margin-block-start`, `margin-inline-start`, and `margin-inline-end`, which define the other margins of the element.

Values

The `margin-block-end` property takes the same values as the `margin-top` property.

Formal definition

Initial value	`0`
Applies to	same as `margin`
Inherited	no
Percentages	depends on layout model
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length

Formal syntax

```
margin-block-end =
  <'margin-top'>

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block end margin

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  margin-block-end: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `writing-mode`, `direction`, `text-orientation`

margin-block-start CSS property

The **`margin-block-start`** CSS property defines the logical block start margin of an element, which maps to a physical margin depending on the element's writing mode, directionality, and text orientation.

Try it

```
margin-block-start: 20px;
writing-mode: horizontal-tb;
```

```
margin-block-start: 20px;
writing-mode: vertical-rl;
```

```
margin-block-start: 20%;
writing-mode: horizontal-tb;
```

```
margin-block-start: auto;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div id="container">
    <div class="row">One</div>
    <div class="row transition-all" id="example-element">Two</div>
    <div class="row">Three</div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  color: white;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

Syntax

```
/* <length> values */
margin-block-start: 10px; /* An absolute length */
margin-block-start: 1em; /* relative to the text size */
margin-block-start: 5%; /* relative to the nearest block container's width */
margin-block-start: anchor-size(width);
margin-block-start: calc(anchor-size(--my-anchor block, 20px) / 3);

/* Keyword value */
margin-block-start: auto;

/* Global values */
margin-block-start: inherit;
margin-block-start: initial;
margin-block-start: revert;
margin-block-start: revert-layer;
margin-block-start: unset;
```

It corresponds to the `margin-top`, `margin-right`, `margin-bottom`, or `margin-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

It relates to `margin-block-end`, `margin-inline-start`, and `margin-inline-end`, which define the other margins of the element.

Values

The `margin-block-start` property takes the same values as the `margin-top` property.

Formal definition

Initial value	`0`
Applies to	same as `margin`
Inherited	no
Percentages	depends on layout model
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length

Formal syntax

```
margin-block-start =
  <'margin-top'>

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block start margin

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  margin-block-start: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `writing-mode`, `direction`, `text-orientation`

margin-bottom CSS property

The **`margin-bottom`** CSS property sets the margin area on the bottom of an element. A positive value places it farther from its neighbors, while a negative value places it closer.

Try it

```
margin-bottom: 1em;
```

```
margin-bottom: 10%;
```

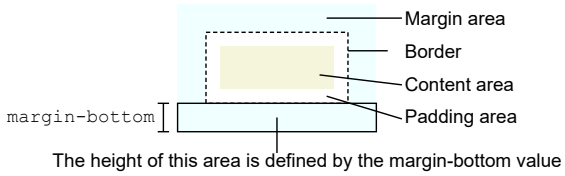
```
margin-bottom: 10px;
```

```
margin-bottom: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="row"></div>
    <div class="row transition-all" id="example-element"></div>
    <div class="row"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
```

```
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```



This property has no effect on *non-replaced* inline elements, such as `<span>` or `<code>`.

Syntax

```
/* <length> values */
margin-bottom: 10px; /* An absolute length */
margin-bottom: 1em; /* relative to the text size */
margin-bottom: 5%; /* relative to the nearest block container's width */
margin-bottom: anchor-size(width);
margin-bottom: calc(anchor-size(--my-anchor self-block, 20px) / 3);

/* Keyword value */
margin-bottom: auto;

/* Global values */
margin-bottom: inherit;
margin-bottom: initial;
margin-bottom: revert;
margin-bottom: revert-layer;
margin-bottom: unset;
```

The `margin-bottom` property is specified as the keyword `auto`, or a `<length>`, or a `<percentage>`. Its value can be positive, zero, or negative.

Values

`<length>`

The size of the margin as a fixed value.

- For *anchor-positioned elements*, the `anchor-size()` function resolves to a `<length>` value relative to the associated *anchor element*'s width or height (see Setting element margin based on anchor size).

`<percentage>`

The size of the margin as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block.

`auto`

The browser selects a suitable value to use. See `margin`.

Formal definition

Initial value	`0`
Applies to	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
margin-bottom =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
```

```
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting positive and negative bottom margins

HTML

```
<div class="container">
  <div class="box0">Box 0</div>
  <div class="box1">Box 1</div>
  <div class="box2">Box one's negative margin pulls me up</div>
</div>
```

CSS

CSS for divs to set margin-bottom and height

```
.box0 {
  margin-bottom: 1em;
  height: 3em;
}
.box1 {
  margin-bottom: -1.5em;
  height: 4em;
}
.box2 {
  border: 1px dashed black;
  border-width: 1px 0;
  margin-bottom: 2em;
}
```

Some definitions for container and divs so margins' effects can be seen more clearly

```
.container {
  background-color: orange;
  width: 320px;
  border: 1px solid black;
}
div {
  width: 320px;
  background-color: gold;
}
```

Result

See also

- `margin-top`, `margin-right`, and `margin-left`
- `margin` shorthand
- `margin-block-start`, `margin-block-end`, `margin-inline-start`, and `margin-inline-end`
- `margin-block` and `margin-inline` shorthands
- CSS box model module

margin-inline CSS property

The **`margin-inline`** CSS shorthand property defines both the logical inline start and end margins of an element, which maps to physical margins depending on the element's writing mode, directionality, and text orientation.

Try it

```
margin-inline: 5% 10%;
writing-mode: horizontal-tb;
```

```
margin-inline: 10px 40px;
writing-mode: vertical-rl;
```

```
margin-inline: 5% 10%;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div id="container">
    <div class="col">One</div>
    <div class="col transition-all" id="example-element">Two</div>
    <div class="col">Three</div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  justify-content: flex-start;
}

.col {
  width: 33.33%;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  color: white;
```

```
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `margin-inline-start`
- `margin-inline-end`

Syntax

```
/* <length> values */
margin-inline: 10px 20px; /* An absolute length */
margin-inline: 1em 2em; /* relative to the text size */
margin-inline: 5% 2%; /* relative to the nearest block container's width */
margin-inline: 10px; /* sets both start and end values */
margin-inline: anchor-size(width);
margin-inline: calc(anchor-size(self-block) / 5) auto;

/* Keyword value */
margin-inline: auto;

/* Global values */
margin-inline: inherit;
margin-inline: initial;
margin-inline: revert;
margin-inline: revert-layer;
margin-inline: unset;
```

This property corresponds to the `margin-top` and `margin-bottom`, or the `margin-right` and `margin-left` properties, depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

The `margin-inline` property may be specified using one or two values.

- When **one** value is specified, it applies the same margin to **both start and end**.
- When **two** values are specified, the first margin applies to the **start**, the second to the **end**.

Values

The `margin-inline` property takes the same values as the `margin-top` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `margin-inline-start`: `0` • `margin-inline-end`: `0`
Applies to	same as `margin`
Inherited	no
Percentages	depends on layout model
Computed value	as each of the properties of the shorthand: • `margin-inline-start`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto` • `margin-inline-end`: if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length

Formal syntax

```
margin-inline =
  <'margin-top'>{1,2}

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline start and end margins

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: auto;
  border: 1px solid green;
}

p {
  margin: 0;
  margin-inline: 20px 40px;
  background-color: tan;
}

.verticalExample {
  writing-mode: vertical-rl;
}
```

HTML

```
<div>
  <p>Example text</p>
</div>
<div class="verticalExample">
  <p>Example text</p>
</div>
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `writing-mode`, `direction`, `text-orientation`

margin-inline-end CSS property

The **`margin-inline-end`** CSS property defines the logical inline end margin of an element, which maps to a physical margin depending on the element's writing mode, directionality, and text orientation. In other words, it corresponds to the `margin-top`, `margin-right`, `margin-bottom` or `margin-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
margin-inline-end: 20px;
writing-mode: horizontal-tb;
```

```
margin-inline-end: 20px;
writing-mode: vertical-rl;
```

```
margin-inline-end: 20%;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div id="container">
    <div class="col">One</div>
    <div class="col transition-all" id="example-element">Two</div>
    <div class="col">Three</div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  justify-content: flex-start;
}

.col {
  width: 33.33%;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  color: white;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <length> values */
margin-inline-end: 10px; /* An absolute length */
margin-inline-end: 1em; /* relative to the text size */
margin-inline-end: 5%; /* relative to the nearest block container's width */
margin-inline-end: anchor-size(height);
margin-inline-end: calc(anchor-size(--my-anchor self-inline, 25px) / 5);

/* Keyword value */
margin-inline-end: auto;
```

```
/* Global values */
margin-inline-end: inherit;
margin-inline-end: initial;
margin-inline-end: revert;
margin-inline-end: revert-layer;
margin-inline-end: unset;
```

It relates to `margin-block-start`, `margin-block-end`, and `margin-inline-start`, which define the other margins of the element.

Values

The `margin-inline-end` property takes the same values as the `margin-top` property.

Formal definition

Initial value	`0`
Applies to	same as `margin`
Inherited	no
Percentages	depends on layout model
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length

Formal syntax

```
margin-inline-end =
  <'margin-top'>

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>     |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline end margin

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  margin-inline-end: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- `margin-inline-start`
- The mapped physical properties: `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `writing-mode`, `direction`, `text-orientation`

margin-inline-start CSS property

The **`margin-inline-start`** CSS property defines the logical inline start margin of an element, which maps to a physical margin depending on the element's writing mode, directionality, and text orientation. It corresponds to the `margin-top`, `margin-right`, `margin-bottom`, or `margin-left` property depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Try it

```
margin-inline-start: 20px;
writing-mode: horizontal-tb;
```

```
margin-inline-start: 20px;
writing-mode: vertical-rl;
```

```
margin-inline-start: 20%;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div id="container">
    <div class="col">One</div>
    <div class="col transition-all" id="example-element">Two</div>
    <div class="col">Three</div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  justify-content: flex-start;
}

.col {
  width: 33.33%;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  color: white;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <length> values */
margin-inline-start: 10px; /* An absolute length */
margin-inline-start: 1em; /* relative to the text size */
margin-inline-start: 5%; /* relative to the nearest block container's width */
margin-inline-start: anchor-size(block);
margin-inline-start: calc(anchor-size(--my-anchor width, 30px) / 4);

/* Keyword value */
margin-inline-start: auto;

/* Global values */
margin-inline-start: inherit;
margin-inline-start: initial;
margin-inline-start: revert;
margin-inline-start: revert-layer;
margin-inline-start: unset;
```

It relates to `margin-block-start`, `margin-block-end`, and `margin-inline-end`, which define the other margins of the element.

Values

The `margin-inline-start` property takes the same values as the `margin-top` property.

Formal definition

Initial value	`0`
Applies to	same as `margin`
Inherited	no
Percentages	depends on layout model
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length

Formal syntax

```
margin-inline-start =
  <'margin-top'>

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
```

```
  self-block   |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline start margin

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  margin-inline-start: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- `margin-inline-end`
- The mapped physical properties: `margin-top`, `margin-right`, `margin-bottom`, and `margin-left`
- `writing-mode`, `direction`, `text-orientation`

margin-left CSS property

The **`margin-left`** CSS property sets the margin area on the left side of an element. A positive value places it farther from its neighbors, while a negative value places it closer.

Try it

```
margin-left: 1em;
```

```
margin-left: 10%;
```

```
margin-left: 10px;
```

```
margin-left: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="col"></div>
    <div class="col transition-all" id="example-element"></div>
    <div class="col"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  justify-content: flex-start;
}

.col {
  width: 33.33%;
  border: solid #5b6dcd 10px;
  background-color: rgb(229 232 252 / 0.6);
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffc129;
  background-color: rgb(255 244 219 / 0.6);
}
```

The vertical margins of two adjacent boxes may fuse. This is called *margin collapsing*.

In the rare cases where width is overconstrained (i.e., when all of `width`, `margin-left`, `border`, `padding`, the content area, and `margin-right` are defined), `margin-left` is ignored, and will have the same calculated value as if the `auto` value was specified.

Syntax

```
/* <length> values */
margin-left: 10px; /* An absolute length */
margin-left: 1em; /* relative to the text size */
margin-left: 5%; /* relative to the nearest block container's width */
margin-left: anchor-size(self-inline);
margin-left: calc(anchor-size(--my-anchor width, 20px) / 4);

/* Keyword value */
margin-left: auto;
```

```
/* Global values */
margin-left: inherit;
margin-left: initial;
margin-left: revert;
margin-left: revert-layer;
margin-left: unset;
```

The `margin-left` property is specified as the keyword `auto`, or a `<length>`, or a `<percentage>`. Its value can be positive, zero, or negative.

Values

`<length>`

The size of the margin as a fixed value.

- For *anchor-positioned elements*, the `anchor-size()` function resolves to a `<length>` value relative to the associated *anchor element*'s width or height (see Setting element margin based on anchor size).

`<percentage>`

The size of the margin as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block.

`auto`

The left margin receives a share of the unused horizontal space, as determined mainly by the layout mode that is used. If the values of `margin-left` and `margin-right` are both `auto`, the calculated space is evenly distributed. This table summarizes the different cases:

Value of `display`	Value of `float`	Value of `position`	Computed value of `auto`	Comment
`inline`, `inline-block`, `inline-table`	***any***	`static` or `relative`	`0`	Inline layout mode
`block`, `inline`, `inline-block`, `block`, `table`, `inline-table`, `list-item`, `table-caption`	***any***	`static` or `relative`	`0`, except if both `margin-left` and `margin-right` are set to `auto`. In this case, it is set to the value centering the element inside its parent.	Block layout mode
`block`, `inline`, `inline-block`, `block`, `table`, `inline-table`, `list-item`, `table-caption`	`left` or `right`	`static` or `relative`	`0`	Block layout mode (floating element)
any `table-*`***, except*** `table-caption`	***any***	***any***	`0`	Internal `table-*` elements don't have margins, use `border-spacing` instead
any, except `flex`***,*** `inline-flex`***, or*** `table-*`	***any***	***fixed*** or `absolute`	`0`, except if both `margin-left` and `margin-right` are set to `auto`. In this case, it is set to the value centering the border area inside the available `width`, if fixed.	Absolutely positioned layout mode
`flex`, `inline-flex`	***any***	***any***	`0`, except if there is any positive horizontal free space. In this case, it is evenly distributed to all horizontal `auto` margins.	Flexbox layout mode

Formal definition

Initial value	`0`
Applies to	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
margin-left =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>     |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting margin-left as a percentage

Percentage values for `margin-left` are relative to the container's inline size.

HTML

```
<p>
  A large rose-tree stood near the entrance of the garden: the roses growing on
  it were white, but there were three gardeners at it, busily painting them red.
```

```
</p>
<p class="example">
  Alice thought this a very curious thing, and she went nearer to watch them,
  and just as she came up to them she heard one of them say, "Look out now,
  Five! Don't go splashing paint over me like that!"
</p>
<p>
  "I couldn't help it," said Five, in a sulky tone; "Seven jogged my elbow."
</p>
```

CSS

```
.example {
  margin-left: 50%;
}
```

Result

See also

- `margin-top`, `margin-right`, and `margin-bottom`
- `margin` shorthand
- `margin-block-start`, `margin-block-end`, `margin-inline-start`, and `margin-inline-end`
- `margin-block` and `margin-inline` shorthands
- CSS box model module

margin-right CSS property

The **`margin-right`** CSS property sets the margin area on the right side of an element. A positive value places it farther from its neighbors, while a negative value places it closer.

The vertical margins of two adjacent boxes may fuse. This is called *margin collapsing*.

Try it

```
margin-right: 1em;
```

```
margin-right: 10%;
```

```
margin-right: 10px;
```

```
margin-right: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="col"></div>
    <div class="col transition-all" id="example-element"></div>
    <div class="col"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  justify-content: flex-start;
}

.col {
  width: 33.33%;
  border: solid #5b6dcd 10px;
  background-color: rgb(229 232 252 / 0.6);
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffc129;
  background-color: rgb(255 244 219 / 0.6);
}
```

Syntax

```
/* <length> values */
margin-right: 20px; /* An absolute length */
margin-right: 1em; /* relative to the text size */
margin-right: 5%; /* relative to the nearest block container's width */
margin-right: anchor-size(self-block);
margin-right: calc(anchor-size(--my-anchor height, 20px) / 4);

/* Keyword value */
margin-right: auto;

/* Global values */
margin-right: inherit;
margin-right: initial;
margin-right: revert;
margin-right: revert-layer;
margin-right: unset;
```

The `margin-right` property is specified as the keyword `auto`, or a `<length>`, or a `<percentage>`. Its value can be positive, zero, or negative.

Values

`<length>`

The size of the margin as a fixed value.

- For *anchor-positioned elements*, the `anchor-size()` function resolves to a `<length>` value relative to the associated *anchor element*'s width or height (see Setting element margin based on anchor size).

`<percentage>`

The size of the margin as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block.

`auto`

The right margin receives a share of the unused horizontal space, as determined mainly by the layout mode that is used. If the values of `margin-left` and `margin-right` are both `auto`, the calculated space is evenly distributed. This table summarizes the different cases:

Value of `display`	Value of `float`	Value of `position`	Computed value of `auto`	Comment
`inline`, `inline-block`, `inline-table`	***any***	`static` or `relative`	`0`	Inline layout mode
`block`, `inline`, `inline-block`, `block`, `table`, `inline-table`, `list-item`, `table-caption`	***any***	`static` or `relative`	`0`, except if both `margin-left` and `margin-right` are set to `auto`. In this case, it is set to the value centering the element inside its parent.	Block layout mode
`block`, `inline`, `inline-block`, `block`, `table`, `inline-table`, `list-item`, `table-caption`	`left` or `right`	`static` or `relative`	`0`	Block layout mode (floating element)
any `table-*`***, except*** `table-caption`	***any***	***any***	`0`	Internal `table-*` elements don't have margins, use `border-spacing` instead
any, except `flex`***,*** `inline-flex`***, or*** `table-*`	***any***	***fixed*** or `absolute`	`0`, except if both `margin-left` and `margin-right` are set to `auto`. In this case, it is set to the value centering the border area inside the available `width`, if fixed.	Absolutely positioned layout mode
`flex`, `inline-flex`	***any***	***any***	`0`, except if there is any positive horizontal free space. In this case, it is evenly distributed to all horizontal `auto` margins.	Flexbox layout mode

Formal definition

Initial value	`0`
Applies to	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
margin-right =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting right margin using pixels and percentages

```
.content {
  margin-right: 5%;
}
.side-box {
  margin-right: 10px;
}
.logo {
  margin-right: -5px;
}
```

See also

- `margin-top`, `margin-bottom`, and `margin-left`
- `margin` shorthand
- `margin-block-start`, `margin-block-end`, `margin-inline-start`, and `margin-inline-end`
- `margin-block` and `margin-inline` shorthands
- CSS box model module

margin-top CSS property

The **`margin-top`** CSS property sets the margin area on the top of an element. A positive value places it farther from its neighbors, while a negative value places it closer.

This property has no effect on *non-replaced* inline elements, such as `<span>` or `<code>`.

Try it

```
margin-top: 1em;
```

```
margin-top: 10%;
```

```
margin-top: 10px;
```

```
margin-top: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="row"></div>
    <div class="row transition-all" id="example-element"></div>
    <div class="row"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

Syntax

```
/* <length> values */
margin-top: 10px; /* An absolute length */
margin-top: 1em; /* relative to the text size */
margin-top: 5%; /* relative to the nearest block container's width */
margin-top: anchor-size(height);
margin-top: calc(anchor-size(--my-anchor self-inline, 25px) / 4);

/* Keyword value */
margin-top: auto;

/* Global values */
margin-top: inherit;
margin-top: initial;
margin-top: revert;
margin-top: revert-layer;
margin-top: unset;
```

The `margin-top` property is specified as the keyword `auto`, or a `<length>`, or a `<percentage>`. Its value can be positive, zero, or negative.

Values

`<length>`

The size of the margin as a fixed value.

- For *anchor-positioned elements*, the `anchor-size()` function resolves to a `<length>` value relative to the associated *anchor element*'s width or height (see Setting element margin based on anchor size).

`<percentage>`

The size of the margin as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block.

`auto`

The browser selects a suitable value to use. See `margin`.

Formal definition

Initial value	`0`
Applies to	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
margin-top =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting positive and negative top margins

```
.content {
  margin-top: 5%;
}
.side-box {
  margin-top: 10px;
}
.logo {
  margin-top: -5px;
}
#footer {
  margin-top: 1em;
}
```

See also

- `margin-right`, `margin-bottom`, and `margin-left`
- `margin` shorthand
- `margin-block-start`, `margin-block-end`, `margin-inline-start`, and `margin-inline-end`
- `margin-block` and `margin-inline` shorthands
- CSS box model module

margin-trim CSS property

The `margin-trim` property allows the container to trim the margins of its children where they adjoin the container's edges.

Syntax

```
margin-trim: none;
margin-trim: block;
margin-trim: block-start;
margin-trim: block-end;
margin-trim: inline;
margin-trim: inline-start;
margin-trim: inline-end;

/* Global values */
margin-trim: inherit;
margin-trim: initial;
margin-trim: revert;
margin-trim: revert-layer;
margin-trim: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

Margins are not trimmed by the container.

`block`

Margins provided to the block children where they adjoin the container's edges are trimmed to zero without affecting the margins provided to the container.

`block-start`

Margin of the first block child with the container's edge is trimmed to zero.

`block-end`

Margin of last block child with the container's edge is trimmed to zero.

`inline`

Margins provided to the inline children where they adjoin the container's edges are trimmed to zero, without affecting the spacing at the beginning and end of the row.

`inline-start`

Margin between the container's edge and the first inline child is trimmed to zero.

`inline-end`

Margin between the container's edge and the last inline child is trimmed to zero.

Formal definition

Initial value	none
Applies to	Block containers and multi-column containers. It also applies to `::first-letter`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
margin-trim =
  none                          |
  block                         |
  [ block-start || block-end ]
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

Once support is implemented for this property, it will probably work like so:

When you've got a container with some inline children and you want to put a margin between each child but not have it interfere with the spacing at the end of the row, you might do something like this:

```
article {
  background-color: red;
  margin: 20px;
  padding: 20px;
  display: inline-block;
}

article > span {
  background-color: black;
  color: white;
  text-align: center;
  padding: 10px;
  margin-right: 20px;
  margin-left: 30px;
}
```

The problem here is that you'd end up with 20px too much spacing at the right of the row, so you'd maybe do this to fix it:

```
span:last-child {
  margin-right: 0;
  margin-left: 0;
}
```

It is a pain having to write another rule to achieve this, and it is also not very flexible. Instead, `margin-trim` could fix it:

```
article {
  margin-trim: inline-end;
  /* … */
}
```

Similarly, to remove left margin with the container's edge:

```
article {
  margin-trim: inline-start;
  /* … */
}
```

See also

- `margin`

marker CSS property

The **`marker`** CSS property points to a marker that will be drawn on the first, middle, and last vertices of the element's path; that is, at all of its vertices. The marker must have been defined using an SVG `<marker>` element, and can only be referenced with a `<url>` value. The value of the CSS property overrides any values of the `marker-start`, `marker`, and `marker-end` attributes in the SVG.

For many marker-supporting shapes, the first and last vertices are in the same place: for example, the top left corner of a `<rect>`. In such shapes, if both the first and last markers are defined, two markers will be drawn at that point, though they may not point in the same direction.

For the middle vertices, the direction each marker points is defined as the direction halfway between the direction at the end of the preceding path segment and the direction of the start of the following path segment. This can be thought of as the cross product of the vectors defined by the two path directions.

> **Note:** The `marker` property will only have an effect for elements that can use SVG markers. See `marker-start` for a list.

Syntax

```
marker: none;
marker: url("markers.svg#arrow");

/* Global values */
marker: inherit;
marker: initial;
marker: revert;
marker: revert-layer;
marker: unset;
```

Values

`none`

This means no marker will be drawn at each vertex of the element's path.

`<marker-ref>`

A `<url>` that refers to a marker defined by an SVG `<marker>` element, to be drawn at each vertex of the element's path. If the URL reference is invalid, no marker will be drawn at the path's vertices.

Formal definition

Initial value	as each of the properties of the shorthand: • `marker-start`: none • `marker-mid`: none • `marker-end`: none
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
marker =
  none          |
  <marker-ref>

<marker-ref> =
  <url>
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

```
html,
body,
svg {
  height: 100%;
}
```

```
<svg viewBox="0 0 240 120" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <marker
      id="triangle"
      viewBox="0 0 10 10"
      markerWidth="10"
      markerHeight="10"
      refX="1"
      refY="5"
      markerUnits="strokeWidth"
      orient="auto">
      <path d="M 0 0 L 10 5 L 0 10 z" fill="red" />
    </marker>
  </defs>
  <polyline
    id="test"
    fill="none"
    stroke="black"
    points="20,100 40,60 70,80 100,20 130,10 150,10 170,20 170,100 120,100" />
</svg>
```

```
polyline#test {
  marker: url("#triangle");
}
```

See also

- `marker-start`
- `marker-end`
- SVG `<marker>` element

marker-end CSS property

The **`marker-end`** CSS property points to a marker that will be drawn on the last vertex of the element's path; that is, at its ending vertex. The marker must have been defined using an SVG `<marker>` element, and can only be referenced with a `<url>` value. The value of the CSS property overrides any values of the `marker-end` attribute in the SVG.

For many marker-supporting shapes, the first and last vertices are the same point: for example, the top left corner of a `<rect>`. In such shapes, if both the first and last markers are defined, two markers will be drawn at that point, though they may not face the same direction.

> **Note:** The `marker-end` property will only have an effect for elements that can use SVG markers. See `marker-end` for a list.

Syntax

```
marker-end: none;
marker-end: url("markers.svg#arrow");

/* Global values */
marker-end: inherit;
marker-end: initial;
marker-end: revert;
marker-end: revert-layer;
marker-end: unset;
```

Values

`none`

This means no marker will be drawn at the last vertex of the element's path.

`<marker-ref>`

A `<url>` that refers to a marker defined by an SVG `<marker>` element, to be drawn at the last vertex of the element's path. If the URL reference is invalid, no marker will be drawn at the path's last vertex.

Formal definition

Initial value	none
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
marker-end =
  none          |
  <marker-ref>

<marker-ref> =
  <url>
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

```
html,
body,
svg {
  height: 100%;
}
```

```
<svg viewBox="0 0 120 120" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <marker
      id="triangle"
      viewBox="0 0 10 10"
      markerWidth="10"
      markerHeight="10"
      refX="1"
      refY="5"
      markerUnits="strokeWidth"
      orient="auto">
      <path d="M 0 0 L 10 5 L 0 10 z" fill="red" />
    </marker>
  </defs>
  <polyline
    id="test"
    fill="none"
    stroke="black"
    points="20,100 40,60 70,80 100,20" />
</svg>
```

```
polyline#test {
  marker-end: url("#triangle");
}
```

See also

- `marker-start`
- `marker-mid`
- `marker`
- SVG `marker-end` attribute

marker-mid CSS property

The **`marker-mid`** CSS property points to a marker that will be drawn on the middle vertices of the element's path; that is, at each of its vertices between the start and end vertices. The marker must have been defined using an SVG `<marker>` element, and can only be referenced with a `<url>` value. The value of the CSS property overrides any values of the `marker-mid` attribute in the SVG.

The direction each marker points is defined as the direction halfway between the direction at the end of the preceding path segment and the direction of the start of the following path segment. This can be thought of as the cross product of the vectors defined by the two path directions.

> **Note:** The `marker-mid` property will only have an effect for elements that can use SVG markers. See `marker-mid` for a list.

Syntax

```
marker-mid: none;
marker-mid: url("markers.svg#arrow");

/* Global values */
marker-mid: inherit;
marker-mid: initial;
marker-mid: revert;
marker-mid: revert-layer;
marker-mid: unset;
```

Values

`none`

This means no marker will be drawn at each middle vertex of the element's path.

`<marker-ref>`

A `<url>` that refers to a marker defined by an SVG `<marker>` element, to be drawn at each middle vertex of the element's path. If the URL reference is invalid, no marker will be drawn at the path's middle vertices.

Formal definition

Initial value	none
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
marker-mid =
  none          |
  <marker-ref>

<marker-ref> =
  <url>
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

```
html,
body,
svg {
  height: 100%;
}
```

```
<svg viewBox="0 0 240 120" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <marker
      id="triangle"
      viewBox="0 0 10 10"
      markerWidth="10"
      markerHeight="10"
      refX="1"
      refY="5"
      markerUnits="strokeWidth"
      orient="auto">
      <path d="M 0 0 L 10 5 L 0 10 z" fill="red" />
    </marker>
  </defs>
  <polyline
    id="test"
    fill="none"
    stroke="black"
    points="20,100 40,60 70,80 100,20 130,10 150,10 170,20 170,100 120,100" />
</svg>
```

```
polyline#test {
  marker-mid: url("#triangle");
}
```

See also

- `marker-start`
- `marker-end`
- `marker`
- SVG `marker-mid` attribute

marker-start CSS property

The **`marker-start`** CSS property points to a marker that will be drawn on the first vertex of the element's path; that is, at its starting vertex. The marker must have been defined using an SVG `<marker>` element, and can only be referenced with a `<url>` value. The value of the CSS property overrides any values of the `marker-start` attribute in the SVG.

For many marker-supporting shapes, the first and last vertices are in the same place: for example, the top left corner of a `<rect>`. In such shapes, if both the first and last markers are defined, two markers will be drawn at that point, though they may not point in the same direction.

> **Note:** The `marker-start` property will only have an effect for elements that can use SVG markers. See `marker-start` for a list.

Syntax

```
marker-start: none;
marker-start: url("markers.svg#arrow");

/* Global values */
marker-start: inherit;
marker-start: initial;
marker-start: revert;
marker-start: revert-layer;
marker-start: unset;
```

Values

`none`

This means no marker will be drawn at the first vertex of the element's path.

`<marker-ref>`

A `<url>` that refers to a marker defined by an SVG `<marker>` element, to be drawn at the first vertex of the element's path. If the URL reference is invalid, no marker will be drawn at the path's first vertex.

Formal definition

Initial value	none
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
marker-start =
  none          |
  <marker-ref>

<marker-ref> =
  <url>
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

```
html,
body,
svg {
  height: 100%;
}
```

```
<svg viewBox="0 0 120 120" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <marker
      id="triangle"
      viewBox="0 0 10 10"
      markerWidth="10"
      markerHeight="10"
      refX="1"
      refY="5"
      markerUnits="strokeWidth"
      orient="auto">
      <path d="M 0 0 L 10 5 L 0 10 z" fill="red" />
    </marker>
  </defs>
  <polyline
    id="test"
    fill="none"
    stroke="black"
    points="20,100 40,60 70,80 100,20" />
</svg>
```

```
polyline#test {
  marker-start: url("#triangle");
}
```

See also

- `marker-mid`
- `marker-end`
- `marker`
- SVG `marker-start` attribute

mask CSS property

The **`mask`** CSS shorthand property hides an element (partially or fully) by masking or clipping a specified area of the image. It is a shorthand for all the `mask-*` properties. The property accepts one or more comma-separated values, where each value corresponds to a `<mask-layer>`.

Constituent properties

This property is a shorthand for the following CSS properties:

- `mask-clip`
- `mask-composite`
- `mask-image`
- `mask-mode`
- `mask-origin`
- `mask-position`
- `mask-repeat`
- `mask-size`

Reset-only sub-properties

This property resets the following CSS properties to their initial values:

- `mask-border-mode`
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-slice`
- `mask-border-source`
- `mask-border-width`

Syntax

```
/* Keyword value */
mask: none;

/* Image values */
mask: url("mask.png"); /* Raster image used as mask */
mask: url("masks.svg#star"); /* SVG used as mask */

/* Combined values */
mask: url("masks.svg#star") luminance; /* Luminance mask */
mask: url("masks.svg#star") 40px 20px; /* Mask positioned 40px from the top and 20px from the left */
mask: url("masks.svg#star") 0 0/50px 50px; /* Mask with a width and height of 50px */
mask: url("masks.svg#star") repeat-x; /* Horizontally-repeated mask */
mask: url("masks.svg#star") stroke-box; /* Mask extends to the inside edge of the stroke box */
mask: url("masks.svg#star") exclude; /* Mask combined with background using non-overlapping parts */

/* Multiple masks */
mask:
  url("masks.svg#star") left / 16px repeat-y,
  /* 16px-wide mask on the left side */ url("masks.svg#circle") right / 16px
    repeat-y; /* 16px-wide mask against right side */

/* Global values */
mask: inherit;
mask: initial;
mask: revert;
mask: revert-layer;
mask: unset;
```

Values

`<mask-layer>`

One or more comma-separated mask layers, consisting of the following components:

`<mask-reference>`

Sets the mask image source. See `mask-image`.

`<masking-mode>`

Sets the masking mode of the mask image. See `mask-mode`.

`<position>`

Sets the position of the mask image. See `mask-position`.

`<bg-size>`

Sets the size of the mask image. See `mask-size`.

`<repeat-style>`

Sets the repetition of the mask image. See `mask-repeat`.

`<geometry-box>`

If only one `<geometry-box>` value is given, it sets both the `mask-origin` and `mask-clip` property values. If two `<geometry-box>` values are present, the first defines the `mask-origin` and the second defines the `mask-clip`.

`<geometry-box> | no-clip`

Sets the area affected by the mask image. See `mask-clip`.

`<compositing-operator>`

Sets the compositing operation used on the current mask layer. See `mask-composite`.

Description

The `mask` shorthand property hides part or all of the element it is applied to. The parts of the element that are hidden, visible, or partially shown depend on either the opacity (alpha channel of the mask) or the brightness (luminance) of the mask. In alpha masking, opaque areas of the mask reveal the element, and transparent areas hide it. In luminance masking, light opaque areas of the mask reveal the element, and dark or transparent areas hide it.

While not all constituent mask properties need to be declared, any values that are omitted default to their initial values, which are:

```
mask-image: none;
mask-mode: match-source;
mask-position: 0% 0%;
mask-size: auto;
mask-repeat: repeat;
mask-origin: border-box;
mask-clip: border-box;
mask-composite: add;
```

Within each `<mask-layer>`, the `mask-size` component must go after the `mask-position` value, with a forward slash (`/`) separating the two.

If there are two `<geometry-box>` values present, the first is the `mask-origin` value, while the second is the `mask-clip` value. If one `<geometry-box>` value and the `no-clip` keyword are present, the `<geometry-box>` is the value of the `mask-origin` property, as the `no-clip` is only valid for the `mask-clip` property. In this case, the order of the two values doesn't matter. If only one `<geometry-box>` value is present (with no `no-clip` keyterm specified), this value is used for both the `mask-origin` and `mask-clip` properties.

As the `mask` shorthand resets all the `mask-border-*` properties to their `initial` value, you should declare these properties — or the `mask-border` shorthand — after any `mask` declarations. When setting `mask` in your declaration block, you also implicitly set the following:

```
mask-border-source: none;
mask-border-mode: alpha;
mask-border-outset: 0;
mask-border-repeat: stretch;
```

```
mask-border-slice: 0;
mask-border-width: auto;
```

For this reason, the specification recommends using the `mask` shorthand rather than the individual component properties to override any masks set earlier in the cascade. This ensures that `mask-border` has also been reset.

Formal definition

Initial value	as each of the properties of the shorthand: • `mask-image`: `none` • `mask-mode`: `match-source` • `mask-repeat`: `repeat` • `mask-position`: `0% 0%` • `mask-clip`: `border-box` • `mask-origin`: `border-box` • `mask-size`: `auto` • `mask-composite`: `add`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `mask-position`: refer to size of mask painting area minus size of mask layer image (see the text for `background-position`)
Computed value	as each of the properties of the shorthand: • `mask-image`: as specified, but with `<url>` values made absolute • `mask-mode`: as specified • `mask-repeat`: Consists of two keywords, one per dimension • `mask-position`: Consists of two keywords representing the origin and two offsets from that origin, each given as an absolute length (if given a <length>), otherwise as a percentage. • `mask-clip`: as specified • `mask-origin`: as specified • `mask-size`: as specified, but with relative lengths converted into absolute lengths • `mask-composite`: as specified
Animation type	as each of the properties of the shorthand: • `mask-image`: discrete • `mask-mode`: discrete • `mask-repeat`: discrete • `mask-position`: a repeatable list • `mask-clip`: discrete • `mask-origin`: discrete • `mask-size`: a repeatable list • `mask-composite`: discrete
Creates stacking context	yes

Formal syntax

```
mask =
  <mask-layer>#

<mask-layer> =
  <mask-reference>             ||
  <position> [ / <bg-size> ]?  ||
  <repeat-style>               ||
  <geometry-box>               ||
  [ <geometry-box> | no-clip ] ||
  <compositing-operator>       ||
  <masking-mode>

<mask-reference> =
  none          |
  <image>       |
  <mask-source>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<repeat-style> =
  repeat-x          |
  repeat-y          |
  repeat-block      |
  repeat-inline     |
  <repetition>{1,2}

<geometry-box> =
  <shape-box>  |
  fill-box     |
  stroke-box   |
  view-box

<compositing-operator> =
  add        |
  subtract   |
  intersect  |
```

```
  exclude

<masking-mode> =
  alpha         |
  luminance     |
  match-source

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<mask-source> =
  <url>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat

<shape-box> =
  <visual-box>     |
  margin-box       |
  half-border-box

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Box Model Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Masking an image

In this example, an image is masked using a CSS-generated repeating conic gradient as a mask source. We'll also show the gradient as a background image for comparison.

HTML

We include an `<img>` and an empty `<div>` element.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<div></div>
```

CSS

We set the same `border`, `padding`, and sizing on both the `<img>` and `<div>`.

```
img,
div {
  border: 20px dashed rebeccapurple;
  box-sizing: content-box;
  padding: 20px;
  height: 220px;
  width: 220px;
}
```

We then apply a mask to the `<img>`. The `mask-image` is generated using a `repeating-conic-gradient()` function. We define it to be a `100px` by `100px` gradient, which repeats starting at the top and left corner of the image's `content-box`. We include two `<geometry-box>` values; the first sets the `mask-origin` and the second defines the `mask-clip` property value. The gradient goes from transparent to solid `lightgreen`. We used `lightgreen` to demonstrate that it isn't the color of the mask that is important, but rather its transparency.

```
img {
  mask: repeating-radial-gradient(
      circle,
      transparent 0 5px,
      lightgreen 15px 20px
    )
    content-box border-box 0% 0% / 100px 100px repeat;
}
```

Finally, we use the same value for the `<div>`'s `background` shorthand property as we used for the `mask`.

```
div {
  background: repeating-radial-gradient(
      circle,
      transparent 0 5px,
      lightgreen 15px 20px
    )
    content-box border-box 0% 0% / 100px 100px repeat;
}
```

Results

See also

- `clip-path`
- `filter`
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module
- SVG `mask` attribute
- Applying SVG effects to HTML content

mask-border CSS property

The **`mask-border`** CSS shorthand property lets you create a mask along the edge of an element's border.

Constituent properties

This property is a shorthand for the following CSS properties:

- `mask-border-mode`
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-slice`
- `mask-border-source`
- `mask-border-width`

Syntax

```
/* source | slice */
mask-border: url("border-mask.png") 25;

/* source | slice | repeat */
mask-border: url("border-mask.png") 25 space;

/* source | slice | width */
mask-border: url("border-mask.png") 25 / 35px;

/* source | slice | width | outset | repeat | mode */
mask-border: url("border-mask.png") 25 / 35px / 12px space alpha;

/* Global values */
mask-border: inherit;
mask-border: initial;
mask-border: revert;
mask-border: revert-layer;
mask-border: unset;
```

Values

`<'mask-border-source'>`

The source image. See `mask-border-source`.

`<'mask-border-slice'>`

The dimensions for slicing the source image into regions. Up to four values may be specified. See `mask-border-slice`.

`<'mask-border-width'>`

The width of the border mask. Up to four values may be specified. See `mask-border-width`.

`<'mask-border-outset'>`

The distance of the border mask from the element's outside edge. Up to four values may be specified. See `mask-border-outset`.

`<'mask-border-repeat'>`

Defines how the edge regions of the source image are adjusted to fit the dimensions of the border mask. Up to two values may be specified. See `mask-border-repeat`.

`<'mask-border-mode'>`

Defines whether the source image is treated as a luminance mask or alpha mask. See `mask-border-mode`.

Formal definition

Initial value	as each of the properties of the shorthand: • `mask-border-mode`: `alpha` • `mask-border-outset`: `0` • `mask-border-repeat`: `stretch` • `mask-border-slice`: `0` • `mask-border-source`: `none` • `mask-border-width`: `auto`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `mask-border-slice`: refer to size of the mask border image • `mask-border-width`: relative to width/height of the mask border image area
Computed value	as each of the properties of the shorthand: • `mask-border-mode`: as specified • `mask-border-outset`: as specified, but with relative lengths converted into absolute lengths • `mask-border-repeat`: as specified • `mask-border-slice`: as specified • `mask-border-source`: as specified, but with `<url>` values made absolute • `mask-border-width`: as specified, but with relative lengths converted into absolute lengths
Animation type	as each of the properties of the shorthand: • `mask-border-mode`: discrete • `mask-border-outset`: discrete • `mask-border-repeat`: discrete • `mask-border-slice`: discrete • `mask-border-source`: discrete • `mask-border-width`: discrete
Creates stacking context	yes

Formal syntax

```
mask-border =
  <'mask-border-source'>                            ||
  <'mask-border-slice'> [ / <'mask-border-width'>? [ / <'mask-border-outset'> ]? ]?  ||
  <'mask-border-repeat'>                            ||
  <'mask-border-mode'>

<mask-border-source> =
  <'border-image-source'>

<mask-border-slice> =
  <'border-image-slice'>

<mask-border-width> =
  <'border-image-width'>

<mask-border-outset> =
  <'border-image-outset'>

<mask-border-repeat> =
  <'border-image-repeat'>

<mask-border-mode> =
  luminance  |
  alpha

<border-image-source> =
  none     |
  <image>

<border-image-slice> =
  [ <number [0,∞]> | <percentage [0,∞]> ]{1,4}  &&
  fill?

<border-image-width> =
  [ <length-percentage [0,∞]> | <number [0,∞]> | auto ]{1,4}

<border-image-outset> =
  [ <length [0,∞]> | <number [0,∞]> ]{1,4}
```

```
<border-image-repeat> =
  [ stretch | repeat | round | space ]{1,2}

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<length-percentage> =
  <length>      |
  <percentage>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, CSS Values and Units Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a bitmap-based mask border

In this example, we will mask an element's border with a diamond pattern. The source for the mask is a ".png" file of 90 by 90 pixels, with three diamonds going vertically and horizontally:

To match the size of a single diamond, we will use a value of 90 divided by 3, or `30`, for slicing the image into corner and edge regions. A repeat value of `round` will make the mask slices fit evenly, i.e., without clipping or gaps.

```
<div class="masked">
  This element is surrounded by a bitmap-based mask border! Pretty neat, isn't
  it?
</div>
```

```
.masked {
  width: 200px;
  background-color: lavender;
  border: 18px solid salmon;
  padding: 10px;

  -webkit-mask-box-image: url("https://mdn.github.io/shared-assets/images/examples/mask-border-diamonds.png")
    30 fill /          /* slice */
    20px /             /* width */
    1px                /* outset */
    round;             /* repeat */

  mask-border:
    url("https://mdn.github.io/shared-assets/images/examples/mask-border-diamonds.png")
    30 fill /        /* slice */
    20px /           /* width */
    1px              /* outset */
    round;           /* repeat */
}
```

See also

- `mask-border-mode`
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-source`
- `mask-border-width`

mask-border-mode CSS property

The **`mask-border-mode`** CSS property specifies the blending mode used in a mask border.

Syntax

```
/* Keyword values */
mask-border-mode: luminance;
mask-border-mode: alpha;

/* Global values */
mask-border-mode: inherit;
mask-border-mode: initial;
mask-border-mode: revert;
mask-border-mode: revert-layer;
mask-border-mode: unset;
```

Values

This property is specified as one of the following keyword values:

`luminance`

The luminance values of the mask border image are used as the mask values.

`alpha`

The alpha values of the mask border image are used as the mask values.

Formal definition

Initial value	`alpha`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-border-mode =
  luminance  |
  alpha
```

This syntax reflects the latest standard as per CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Browser compatibility

Currently, no browsers support this feature.

See also

- `mask-mode`
- `mask-type`
- Alpha transparency versus luminance
- `mask-border` shorthand
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-source`
- `mask-border-width`
- `mask-mode`
- CSS masking module

mask-border-outset CSS property

The **`mask-border-outset`** CSS property specifies the distance by which an element's mask border is set out from its border box.

Syntax

```
/* <length> value */
mask-border-outset: 1rem;

/* <number> value */
mask-border-outset: 1.5;

/* top and bottom | left and right */
mask-border-outset: 1 1.2;

/* top | left and right | bottom */
mask-border-outset: 30px 2 45px;

/* top | right | bottom | left */
mask-border-outset: 7px 12px 14px 5px;

/* Global values */
mask-border-outset: inherit;
mask-border-outset: initial;
mask-border-outset: revert;
mask-border-outset: revert-layer;
mask-border-outset: unset;
```

The `mask-border-outset` property may be specified as one, two, three, or four values. Each value is a `<length>` or `<number>`. Negative values are invalid.

- When **one** value is specified, it applies the same outset to **all four sides**.
- When **two** values are specified, the first outset applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first outset applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the outsets apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<length>`

The size of the mask border outset as a dimension.

`<number>`

The size of the mask border outset as a multiple of the corresponding `border-width`.

Formal definition

Initial value	`0`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	discrete

Formal syntax

```
mask-border-outset =
  <'border-image-outset'>

<border-image-outset> =
  [ <length [0,∞]> | <number [0,∞]> ]{1,4}
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This property doesn't appear to be supported anywhere yet. When it eventually starts to be supported, it will serve to move the mask away from the inner edge of the element's border box — you can use it to make the mask start from part way across the border, rather than the inside of it.

```
mask-border-outset: 1rem;
```

Chromium-based browsers support an outdated version of this property — `mask-box-image-outset` — with a prefix:

```
-webkit-mask-box-image-outset: 1rem;
```

> **Note:** The `mask-border` page features a working example (using the out-of-date prefixed border mask properties supported in Chromium), so you can get an idea of the effect.

See also

- `mask-border`
- `mask-border-mode`
- `mask-border-repeat`
- `mask-border-source`
- `mask-border-width`

mask-border-repeat CSS property

The **`mask-border-repeat`** CSS property specifies how the images for the sides and the middle part of the mask border image are scaled and tiled.

Syntax

```
/* Keyword values */
mask-border-repeat: stretch;
mask-border-repeat: repeat;
mask-border-repeat: round;
mask-border-repeat: space;

/* top and bottom | left and right */
mask-border-repeat: round stretch;

/* Global values */
mask-border-repeat: inherit;
mask-border-repeat: initial;
mask-border-repeat: revert;
mask-border-repeat: revert-layer;
mask-border-repeat: unset;
```

The `mask-border-repeat` property may be specified using one or two values chosen from the list of values below.

- When **one** value is specified, it applies the same behavior on **all four sides**.
- When **two** values are specified, the first applies to the **top and bottom**, the second to the **left and right**.

Values

`stretch`

The source image's edge regions are stretched to fill the gap between each border.

`repeat`

The source image's edge regions are tiled (repeated) to fill the gap between each border. Tiles may be clipped to achieve the proper fit.

`round`

The source image's edge regions are tiled (repeated) to fill the gap between each border. Tiles may be stretched to achieve the proper fit.

`space`

The source image's edge regions are tiled (repeated) to fill the gap between each border. Extra space will be distributed in between tiles to achieve the proper fit.

Formal definition

Initial value	`stretch`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-border-repeat =
  <'border-image-repeat'>

<border-image-repeat> =
  [ stretch | repeat | round | space ]{1,2}
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This property doesn't appear to be supported anywhere yet. When it eventually starts to be supported, it will serve to define how the border mask slice will repeat around the border — i.e., will it just repeat, or be scaled slightly so a whole number of slices fits, or be stretched so one slice fits?

```
mask-border-repeat: round;
```

Chromium-based browsers support an outdated version of this property — `mask-box-image-repeat` — with a prefix:

```
-webkit-mask-box-image-repeat: round;
```

> **Note:** The `mask-border` page features a working example (using the out-of-date prefixed border mask properties supported in Chromium), so you can get an idea of the effect.

See also

- `mask-border`
- `mask-border-mode`
- `mask-border-outset`
- `mask-border-source`
- `mask-border-width`

mask-border-slice CSS property

The **`mask-border-slice`** CSS property divides the image set by `mask-border-source` into regions. These regions are used to form the components of an element's mask border.

Syntax

```
/* All sides */
mask-border-slice: 30%;

/* top and bottom | left and right */
mask-border-slice: 10% 30%;

/* top | left and right | bottom */
mask-border-slice: 30 30% 45;

/* top | right | bottom | left */
mask-border-slice: 7 12 14 5;

/* Using the `fill` keyword */
mask-border-slice: 10% fill 7 12;

/* Global values */
mask-border-slice: inherit;
mask-border-slice: initial;
mask-border-slice: revert;
mask-border-slice: revert-layer;
mask-border-slice: unset;
```

The `mask-border-slice` property may be specified using one to four `<number-percentage>` values to represent the position of each image slice. Negative values are invalid; values greater than their corresponding dimension are clamped to `100%`.

- When **one** position is specified, it creates all four slices at the same distance from their respective sides.
- When **two** positions are specified, the first value creates slices measured from the **top and bottom**, the second creates slices measured from the **left and right**.
- When **three** positions are specified, the first value creates a slice measured from the **top**, the second creates slices measured from the **left and right**, the third creates a slice measured from the **bottom**.
- When **four** positions are specified, they create slices measured from the **top**, **right**, **bottom**, and **left** in that order (clockwise).

The optional `fill` value, if used, can be placed anywhere in the declaration.

Values

`<number>`

Represents an edge offset in *pixels* for raster images and *coordinates* for vector images. For vector images, the number is relative to the element's size, not the size of the source image, so percentages are generally preferable in these cases.

`<percentage>`

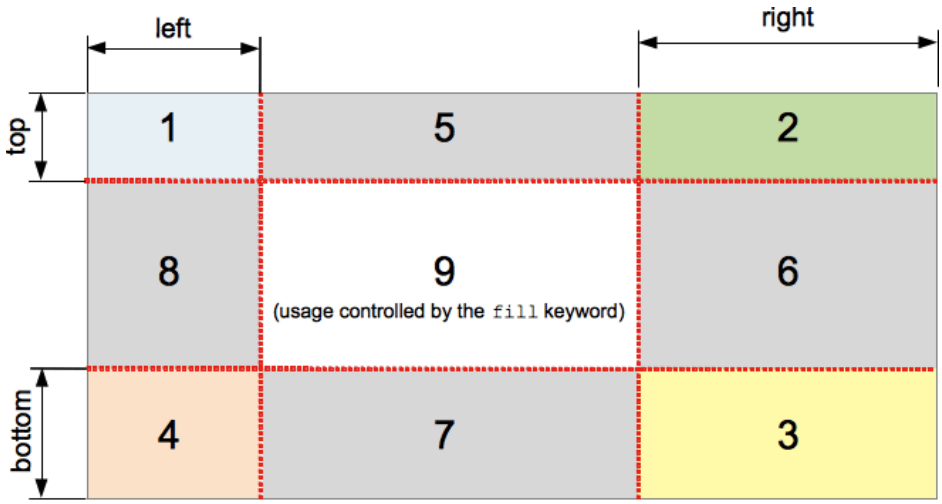
Represents an edge offset as a percentage of the source image's size: the width of the image for horizontal offsets, the height for vertical offsets.

`fill`

Preserves the middle image region. Its width and height are sized to match the top and left image regions, respectively.

Description

The slicing process creates nine regions in total: four corners, four edges, and a middle region. Four slice lines, set a given distance from their respective sides, control the size of the regions.



The above diagram illustrates the location of each region.

- Zones 1-4 are corner regions. Each one is used a single time to form the corners of the final border image.
- Zones 5-8 are edge regions. These are repeated, scaled, or otherwise modified in the final border image to match the dimensions of the element.
- Zone 9 is the middle region. It is discarded by default, but is used like a background image if the keyword `fill` is set.

The `mask-border-repeat`, `mask-border-width`, and `mask-border-outset` properties determine how these regions are used to form the final mask border.

Formal definition

Initial value	`0`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Percentages	refer to size of the mask border image
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-border-slice =
  <'border-image-slice'>

<border-image-slice> =
  [ <number [0,∞]> | <percentage [0,∞]> ]{1,4}  &&
  fill?
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

`mask-border-slice` serves to define the size of the slices taken from the source image, and is used to create the border mask.

```
mask-border-slice: 30 fill;
```

Chromium-based browsers support an outdated version of this property — `mask-box-image-slice` — with a prefix:

```
-webkit-mask-box-image-slice: 30 fill;
```

> **Note:** The `mask-border` page features a working example (using the out-of-date prefixed border mask properties supported in Chromium), so you can get an idea of the effect.

> **Note:** The fill keyword needs to be included if you want the element's content to be visible.

See also

- `mask-border`
- `mask-border-mode`
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-source`
- `mask-border-width`
- Illustrated description of the 1-to-4-value syntax

mask-border-source CSS property

The **`mask-border-source`** CSS property sets the source image used to create an element's mask border.

The `mask-border-slice` property is used to divide the source image into regions, which are then dynamically applied to the final mask border.

Syntax

```
/* Keyword value */
mask-border-source: none;

/* <image> values */
mask-border-source: url("image.jpg");
mask-border-source: linear-gradient(to top, red, yellow);

/* Global values */
mask-border-source: inherit;
mask-border-source: initial;
mask-border-source: revert;
mask-border-source: revert-layer;
mask-border-source: unset;
```

Values

`none`

No mask border is used.

`<image>`

Image reference to use for the mask border.

Formal definition

Initial value	none
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
mask-border-source =
  <'border-image-source'>

<border-image-source> =
  none     |
  <image>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This property doesn't appear to be supported anywhere yet. When it eventually starts to be supported, it will serve to define the source of the border mask.

```
mask-border-source: url("image.jpg");
```

Chromium-based browsers support an outdated version of this property — `mask-box-image-source` — with a prefix:

```
-webkit-mask-box-image-source: url("image.jpg");
```

> **Note:** The `mask-border` page features a working example (using the out-of-date prefixed border mask properties supported in Chromium), so you can get an idea of the effect.

See also

- `mask-border`
- `mask-border-mode`
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-width`

mask-border-width CSS property

The **`mask-border-width`** CSS property sets the width of an element's mask border.

Syntax

```
/* Keyword value */
mask-border-width: auto;

/* <length> value */
mask-border-width: 1rem;

/* <percentage> value */
mask-border-width: 25%;

/* <number> value */
mask-border-width: 3;

/* top and bottom | left and right */
mask-border-width: 2em 3em;

/* top | left and right | bottom */
mask-border-width: 5% 15% 10%;

/* top | right | bottom | left */
mask-border-width: 5% 2em 10% auto;

/* Global values */
mask-border-width: inherit;
mask-border-width: initial;
mask-border-width: revert;
mask-border-width: revert-layer;
mask-border-width: unset;
```

The `mask-border-width` property may be specified using one, two, three, or four values chosen from the list of values below.

- When **one** value is specified, it applies the same width to **all four sides**.
- When **two** values are specified, the first width applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first width applies to the **top**, the second to the **left and right**, the third to the **bottom**.
- When **four** values are specified, the widths apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<length-percentage>`

> The width of the mask border, specified as a `<length>` or a `<percentage>`. Percentages are relative to the *width* of the border area for horizontal offsets and the *height* of the border area for vertical offsets. Must not be negative.

`<number>`

> The width of the mask border, specified as a multiple of the corresponding `border-width`. Must not be negative.

`auto`

> The width of the mask border is made equal to the intrinsic width or height (whichever is applicable) of the corresponding `mask-border-slice`. If the image does not have the required intrinsic dimension, the corresponding `border-width` is used instead.

Formal definition

Initial value	`auto`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Percentages	relative to width/height of the mask border image area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	discrete

Formal syntax

```
mask-border-width =
  <'border-image-width'>

<border-image-width> =
  [ <length-percentage [0,∞]> | <number [0,∞]> | auto ]{1,4}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Masking Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

`mask-border-width` defines the width of the border mask — setting this to a different value to `mask-border-slice` will cause the slices to be scaled to fit the border mask.

```
/* Final mask scaled down to 20px wide */
mask-border-slice: 30 fill;
mask-border-width: 20px;
```

Chromium-based browsers support an outdated version of this property — `mask-box-image-width` — with a prefix:

```
-webkit-mask-box-image-width: 20px;
```

> **Note:** The `mask-border` page features a working example (using the out-of-date prefixed border mask properties supported in Chromium), so you can get an idea of the effect.

See also

- `mask-border`
- `mask-border-mode`
- `mask-border-outset`
- `mask-border-repeat`
- `mask-border-source`

mask-clip CSS property

The **`mask-clip`** CSS property determines the area which is affected by a mask. The painted content of an element must be restricted to this area.

Syntax

```
/* <coord-box> values */
mask-clip: content-box;
mask-clip: padding-box;
mask-clip: border-box;
mask-clip: fill-box;
mask-clip: stroke-box;
mask-clip: view-box;

/* Keyword value */
mask-clip: no-clip;

/* Multiple values */
mask-clip: padding-box, no-clip;
mask-clip: view-box, fill-box, border-box;

/* Global values */
mask-clip: inherit;
mask-clip: initial;
mask-clip: revert;
mask-clip: revert-layer;
mask-clip: unset;
```

Values

The property accepts a comma-separated list of keyword values. Each value is a `<coord-box>` or `no-clip`:

`content-box`

The painted content is clipped to the content box.

`padding-box`

The painted content is clipped to the padding box.

`border-box`

The painted content is clipped to the border box.

`fill-box`

The painted content is clipped to the object bounding box.

`stroke-box`

The painted content is clipped to the stroke bounding box.

`view-box`

Uses the nearest SVG viewport as reference box. If a `viewBox` attribute is specified for the element creating the SVG viewport, the reference box is positioned at the origin of the coordinate system established by the `viewBox` attribute and the dimension of the reference box is set to the width and height values of the `viewBox` attribute.

`no-clip`

The painted content is not clipped.

`border`

This keyword behaves the same as `border-box`.

`padding`

This keyword behaves the same as `padding-box`.

`content`

This keyword behaves the same as `content-box`.

`text`

This keyword clips the mask image to the text of the element.

Description

The `mask-clip` property defines the area of the element that is affected by the applied mask.

For mask layer images that do not reference an SVG `<mask>` element, the `mask-clip` property defines the mask painting area, or the area affected by the mask. The painted content of the element will be restricted to this area.

The `mask-clip` property has no effect on a mask layer image that references a `<mask>` element. The `<mask>` element's `x`, `y`, `width`, `height`, and `maskUnits` attributes determine the mask painting area when the source of the `mask-image` is a `<mask>`.

An element can have multiple mask layers applied. The number of layers is determined by the number of comma-separated values in the `mask-image` property value (even if a value is `none`). Each `mask-clip` value in the comma-separated list of values is matched up with the `mask-image` values, in order. If the number of values in the two properties differs, any excess values of `mask-clip` are not used, or, if `mask-clip` has fewer values than `mask-image`, the `mask-clip` values are repeated.

Formal definition

Initial value	`border-box`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-clip =
  [ <coord-box> | no-clip ]#

<coord-box> =
  <paint-box>  |
  view-box

<paint-box> =
  <visual-box>  |
  fill-box      |
  stroke-box

<visual-box> =
  content-box  |
  padding-box  |
  border-box
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Clipping a mask to the border box

This examples demonstrates three `mask-clip` values.

HTML

We include three elements, each with a different `<coord-box>` value as a class name.

```
<div class="border-box"></div>
<div class="padding-box"></div>
<div class="content-box"></div>
```

CSS

The CSS defines the element to have a background, border, padding, and margin, along with a mask image, with each `<div>` having a different `<coord-box>`. We generated content with the name of the class, moving that text up 10px to prevent it from being masked out of view.

```
div {
  width: 100px;
  height: 100px;
  background-color: #8cffa0;
  margin: 10px;
  border: 20px solid #8ca0ff;
  padding: 20px;
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mdn.svg");
  mask-size: 100% 100%;
}
.content-box {
  mask-clip: content-box;
}
.border-box {
  mask-clip: border-box;
}
.padding-box {
  mask-clip: padding-box;
}
div::before {
  content: attr(class);
  position: relative;
  top: -10px;
}
```

```
body {
  display: flex;
  flex-flow: row wrap;
}
```

Results

See also

- `mask` shorthand
- `mask-image`
- `mask-origin`
- `mask-position`
- `mask-repeat`

- `mask-size`
- `mask-border`
- `clip-path`
- `background-clip`
- Introduction to CSS clipping
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-composite CSS property

The **`mask-composite`** CSS property represents a compositing operation used on the current mask layer with the mask layers below it.

Syntax

```
/* Keyword values */
mask-composite: add;
mask-composite: subtract;
mask-composite: intersect;
mask-composite: exclude;

/* Global values */
mask-composite: inherit;
mask-composite: initial;
mask-composite: revert;
mask-composite: revert-layer;
mask-composite: unset;
```

Values

The property accepts a comma-separated list of `<compositing-operator>` keyword values, each representing a Porter-Duff compositing operator which defines the compositing operation used on the current mask layer with the mask layers below it, including:

`add`

The associated mask image is placed over all mask layers below it (with the corresponding compositing operators applied). This is the default value.

`subtract`

The associated mask image is placed, where it falls outside of all mask layers below it (with the corresponding compositing operators applied).

`intersect`

The parts of the associated mask image that overlap all composited mask layers below it replace those previously composited layers.

`exclude`

The non-overlapping regions of the associated mask image and the mask layers below it, with their corresponding compositing operators applied, are combined.

Description

When an element has multiple mask layers applied, the `mask-composite` property can be used to define how the multiple masks interact with each other, or are combined, in creating the final mask effect.

The number of layers is determined by the number of comma-separated values in the `mask-image` property value (even if a value is `none`). Each `mask-composite` value in the comma-separated list of values is matched up with a `mask-image` value, in order. If the number of values in `mask-composite` are equal to or greater than the number of `mask-image` values, the extra values are ignored. If the `mask-composite` property doesn't have enough comma-separated values to match the number of layers, the list of values are repeated until there are enough.

For processing, the *source layer*, which is current or associated mask layer image, is either added to (the default), subtracted from, intersected with, or is excluded from, the destination layers. The *destination layers* are the mask layers below the source with their corresponding compositing operators applied; this includes all the previous layers, composed in order of appearance within the comma-separated list of masks. All mask layers below the current mask layer must be composited before applying the compositing operation for the current mask layer. Mask layer images are transformed to alpha masks for processing before being combined by the defined compositing value.

The multiple mask layers applied to any element or pseudo-element act as if they are rendered into an isolated group. In other words, the mask layers are composited with other mask layers, not with the element's content or the content behind the element.

Formal definition

Initial value	`add`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-composite =
  <compositing-operator>#

<compositing-operator> =
  add       |
  subtract  |
  intersect |
  exclude
```

This syntax reflects the latest standard as per CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the basic usage of the `mask-composite` property.

HTML

We include an HTML `<div>` element, that we will then style.

```
<div></div>
```

CSS

We provide size and color our `<div>`, then add two `mask-image`s, and make their size match that of the element they're masking with the `mask-size` property. Lastly, we subtract the second mask image from the first mask image with the `mask-composite` property set to `subtract`.

```
div {
  width: 100px;
  height: 100px;
  background-color: red;

  mask-image:
    url("https://mdn.github.io/shared-assets/images/examples/mdn.svg"),
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
  mask-size: 100% 100%;

  mask-composite: subtract;
}
```

Results

Value comparison

This example demonstrates the `mask-composite` property's four `<compositing-operator>` keyword values, along with comparing the effects of `alpha` and `luminance` mask modes.

HTML

We have a `<table>` that contains eight images. The `<table>` has been hidden for brevity.

```
<table>
  <tbody>
    <tr>
      <th colspan="4">mask-type: alpha</th>
    </tr>
    <tr>
      <th>mask-composite: add</th>
      <th>mask-composite: subtract</th>
      <th>mask-composite: intersect</th>
      <th>mask-composite: exclude</th>
    </tr>
    <tr class="alphaMaskType">
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>

    <tr>
      <th colspan="4">
        <code>mask-type: luminance</code>
      </th>
    </tr>
    <tr>
      <th>mask-composite: add</th>
      <th>mask-composite: subtract</th>
      <th>mask-composite: intersect</th>
      <th>mask-composite: exclude</th>
    </tr>
    <tr class="luminanceMaskType">
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

And an SVG with 4 masks; an alpha heart and circle and a luminance heart and circle. The heart masks are defined using solid colors. The circle masks are created using semi-transparent white and black `stroke` and `fill` colors.

```
<svg height="0" width="0">
  <mask id="heartAlpha" class="alpha">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="green"
      stroke="white"
      stroke-width="20" />
  </mask>
  <mask id="circleAlpha" class="alpha">
    <circle
      cx="130"
      cy="130"
      r="50"
      fill="rgb(0 0 0 / 0.5)"
      stroke="rgb(255 255 255 / 0.5)"
      stroke-width="20" />
  </mask>
  <mask id="heartLuminance" class="luminance">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="green"
      stroke="white"
      stroke-width="20" />
  </mask>
  <mask id="circleLuminance" class="luminance">
    <circle
      cx="130"
      cy="130"
      r="50"
      fill="rgb(0 0 0 / 0.5)"
      stroke="rgb(255 255 255 / 0.5)"
      stroke-width="20" />
  </mask>
</svg>
```

CSS

First we style the `<mask>` elements, providing each mask with a `mask-type` property value of either `alpha` or `luminance`.

```
mask.luminance {
  mask-type: luminance;
}

mask.alpha {
  mask-type: alpha;
}
```

We then apply the heart and circle masks as the comma-separated `mask-image` property values. These are applied to each `<img>` element, with all the images in a row getting the same masks.

```
/* apply the mask images */
tr.alphaMaskType img {
  mask-image: url("#heartAlpha"), url("#circleAlpha");
}

tr.luminanceMaskType img {
  mask-image: url("#heartLuminance"), url("#circleLuminance");
}
```

Finally, we compose the masks using the `mask-composite` property, applying the four different enumerated `mask-composite` values by table column.

```
/* property we're testing */
td:nth-of-type(1) img {
  mask-composite: add;
}
td:nth-of-type(2) img {
  mask-composite: subtract;
}
td:nth-of-type(3) img {
  mask-composite: intersect;
}
td:nth-of-type(4) img {
  mask-composite: exclude;
}
```

The table styles have been hidden for the sake of brevity.

```
mask {
  height: 50%;
}

table,
td,
th {
  border: 1px solid;
}
th {
  font-family: monospace;
}
th {
  color: green;
}
body > img {
```

```
  display: none;
}
```

Results

See also

- `mask` shorthand
- `mask-type`
- `mask-mode`
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-image CSS property

The **`mask-image`** CSS property sets the image that is used as the mask layer for an element, hiding sections of the element on which the masking image is set based on the alpha channel of the mask image and, depending on the `mask-mode` property value, the luminance of the mask image's colors.

Syntax

```
/* Keyword value */
mask-image: none;

/* <mask-source> value */
mask-image: url("masks.svg#mask1");

/* <image> values */
mask-image: linear-gradient(black, transparent);
mask-image: image(url("mask.png"), skyblue);

/* Multiple values */
mask-image: url("mask.png"), linear-gradient(black 25%, transparent 35%);

/* Global values */
mask-image: inherit;
mask-image: initial;
mask-image: revert;
mask-image: revert-layer;
mask-image: unset;
```

Values

`none`

This keyword is interpreted as a transparent black image layer.

`<mask-source>`

A `<url>` reference to a `<mask>` or to a CSS image.

`<image>`

An image value used as a mask image layer.

Description

The `mask-image` property provides a mask that hides part of the element to which it is applied. The value is a comma-separated list of mask references. Each mask reference is an `<image>`, a `<mask-source>`, or the keyword `none`.

An `<image>` can be any type of image, including generated images such as CSS gradients.

If only one value is specified in the `mask-image` property value, and that value is `none`, no masking effect will be apparent. If multiple values are specified, a `none` value included in the list may have no direct effect, however, other `mask-*` values in the same list position will apply to a transparent black mask layer and have no visual effect.

Only image sources served over HTTP and HTTPS protocols are accepted as `<image>` values due to the CORS policy. Images served locally, including relative or absolute `file://` protocols, are not accepted, and will render as transparent black. To test URL image sources locally, set up a local server.

A mask will be counted as a transparent black image layer, not revealing anything, in the following cases:

- The mask image is empty (zero width or zero height).
- The mask image fails to download.
- The browser does not support the mask image format.
- The mask image doesn't exist.
- The mask value doesn't point to a mask image.

The default value of the `mask-mode` property is `match-source`, which means the mode is defined by the mode of the mask image itself. The mask image's mode is generally `alpha` except when the mask source is an SVG `<mask>` element, in which case the mode is `luminance` unless the mode is changed to `alpha` via the CSS `mask-type` property or SVG `mask-type` attribute.

The `mask-mode` value is significant, because it determines whether the masking effect depends on the image source's alpha channel values alone or a combination of those and the mask's luminance (the lightness/darkness of the colors that make up the `mask-image`):

- In all cases, the alpha transparency of the mask matters; element areas masked by opaque sections of the `mask-image` will be rendered, while areas masked by transparent image sections are hidden.
- When the `mask-mode` value is set or resolves to `alpha`, only the alpha-channel of the colors matter; the hue, lightness, etc., don't matter.
- If the `mask-mode` property is set or defaults to `luminance`, the masking value is the luminance value of each color multiplied by its alpha value. The `mask-mode` will resolve to `luminance` if explicitly set to that value, or if the property is set to `match-source` and the `mask-image` source is an SVG `<mask>` that does not have its `mask-type` property or `mask-type` attribute explicitly set to `alpha`.

Formal definition

Initial value	none
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Formal syntax

```
mask-image =
  <mask-reference>#

<mask-reference> =
  none          |
  <image>       |
  <mask-source>

<image> =
  <url>          |
  <image()>      |
  <image-set()>  |
  <cross-fade()> |
  <element()>    |
  <gradient>

<mask-source> =
  <url>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Gradient as a mask image

In this example, we use an `<image>` value as a mask, defining a CSS radial gradient as our mask image to create a round image with a soft edge.

HTML

We include an HTML `<img>` element, which will also be used in all other examples.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

CSS

We use the CSS `radial-gradient()` function to create a mask that has a black circle with a radius that is half the width of the mask, before transitioning to being transparent over 10%.

```
img {
  mask-image: radial-gradient(black 50%, transparent 60%);
}
```

Results

The part of the original element that is masked by the black circle is fully opaque, fading to transparent as the mask fades to transparent.

Image resource as a mask image

In this example, the `<mask-source>` used as our mask image is an external SVG.

HTML

We include the same image as the previous example. We've also included the image we will be using as the mask; a star whose `fill-opacity` is `0.5`, or 50% opaque.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />

<img
  src="https://mdn.github.io/shared-assets/images/examples/mask-star.svg"
  alt="A star" />
```

CSS

We use `mask-star.svg` as a mask image on our first image:

```
img:first-of-type {
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
}
```

Results

The mask is semi-opaque, which is why the colors are not as vibrant as the previous example. The part of the image that is visible is 50% opaque; the opacity of the mask applied. The mask is smaller than the image, so repeats by default. We could have used `mask-repeat` to control the repeating or `mask-size` to change the size of the mask, which we do in the next example.

Multiple masks

This example demonstrates applying multiple masks.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
```

CSS

We apply two masks — the same semi-transparent SVG as in the previous example, and a `repeating-radial-gradient()`. We control the size of the masks using the `mask-size` property. Because our first mask is not sized at 100%, we make sure our masks are centered and don't repeat with the `mask-position` and `mask-repeat` properties, respectively.

```
img {
  mask-size: 95%, 100%;
  mask-position: center;
  mask-repeat: no-repeat;
  mask-image:
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg"),
    repeating-radial-gradient(transparent 0 5px, black 5px 10px);
}
```

Results

Masking with SVG <code><mask></code>

This example demonstrates using SVG `<mask>` elements as masks. In this case, the color of the mask matters as the `mask-type` value for SVG masks defaults to `luminance`, which means white opaque areas (100% luminance) will be masked and visible, transparent and black areas (0% luminance) will be clipped, and anything in between will be partially masked.

HTML

We've included an `id` for each of our four images, and an SVG that contains an equal number of `<mask>` elements.

```
<img
  id="green"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  id="stroke"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  id="both"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  id="alphaMode"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />

<svg height="0" width="0">
  <mask id="greenMask">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="green" />
  </mask>
  <mask id="strokeMask">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="none"
      stroke="white"
      stroke-width="20" />
  </mask>
  <mask id="bothMask">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="green"
      stroke="white"
      stroke-width="20" />
  </mask>
  <mask id="black">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="black" />
  </mask>
</svg>
```

```
<p>
  <label>
    <input type="checkbox" />
    Set the <code>mask-mode</code> to <code>alpha</code>.
  </label>
</p>
```

CSS

We apply a different `<mask>` to each `<img>`. No part of the last image, with the `black` fill, will be visible by default. In this case, while all colors used in this example are fully opaque, the `mask-mode` defaults to `match-type`, which resolves to `luminance` in this case.

```
#green {
  mask-image: url("#greenMask");
}
#stroke {
  mask-image: url("#strokeMask");
}
```

```
#both {
  mask-image: url("#bothMask");
}
#alphaMode {
  mask-image: url("#black");
}

body:has(:checked) img {
  mask-mode: alpha;
}
```

The luminance values of `black`, `white`, and `green` are `0`, `100`, and `46.228`, respectively. This means the areas where the mask is white will be visible, whereas areas where the mask is black or fully transparent will be clipped (not visible). Areas where the mask is green will be visible but lighter, equivalent to having a white mask that is 46.228% opaque.

Results

Toggle the checkbox to toggle the value of the `mask-mode` between `alpha` (checked) and the initial value, which resolves to `luminance` (unchecked). When `alpha` is used, the color of the mask doesn't matter; all that matters is the alpha-transparency. When the value resolves to `luminance`, `white` areas are visible, `black` areas are not, and `green` areas are visible but at an opacity that matches the luminance of the color `green`. When `mask-mode` is set to `alpha`, the colors are equivalent as they are all fully opaque.

See also

- `mask` shorthand
- `mask-origin`
- `mask-position`
- `mask-repeat`
- `mask-size`
- `mask-border`
- `clip-path`
- `background-image`
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-mode CSS property

The **`mask-mode`** CSS property is set on the element being masked. It sets whether the mask reference defined by the `mask-image` is treated as a luminance or alpha mask.

Syntax

```
/* Keyword values */
mask-mode: alpha;
mask-mode: luminance;
mask-mode: match-source;

/* Multiple values */
mask-mode: alpha, match-source;

/* Global values */
mask-mode: inherit;
mask-mode: initial;
mask-mode: revert;
mask-mode: revert-layer;
mask-mode: unset;
```

Values

The `mask-mode` property can take multiple comma-separated `<masking-mode>` keyword values, including:

`alpha`

> Indicates that the alpha (transparency) values of the mask image should be used.

`luminance`

> Indicates that the luminance (brightness) values of the mask image should be used.

`match-source`

> Indicates that the type of mask is determined by the source. This is the default property value.
>
> - If the `mask-image` references an SVG `<mask>`, its `mask-type` property value is used, or its `mask-type` attribute, if present. If neither is explicitly set, this value defaults to `luminance`.
> - If the mask image source is an `<image>` or a `<gradient>`, the `alpha` values of the mask image are used.

Description

A mask transfers its transparency, and depending on the mask type, its luminance, to the element it is masking. If the mask is of type `<image>`, by default the alpha values of the mask image determine the visibility of each part of the masked element: where the mask is opaque, the corresponding part of the masked element is visible; where the mask is translucent, the element is as well, with those areas of the element being hidden. This is the default behavior for `<image>` masks when the `mask-mode` property is set to or defaults to `match-source`, and it is always the case when the `mask-mode` is explicitly set to `alpha`.

Understanding luminance

In the case of `luminance` masks, the visibility of the masked element depends on both the opacity of the mask and the lightness of the color of the opaque areas. White (100% luminance) opaque areas (alpha = 1) will be masked and visible, and black areas (0% luminance) transparent (alpha = 0) will be clipped. Areas with colors in between white and black and with partial opacity will be partially masked, reflecting the luminance and alpha transparency of each color making up the mask.

The opacity of a `luminance` mask is determined by the `R`, `G`, `B`, and `A` values of an `rgb()` color using the formula:

```
((0.2125 * R) + (0.7154 * G) + (0.0721 * B)) * A
```

For example, the color `green` is `#008000` or `rgb(0% 50% 0% / 1)`. In a `luminance` mask, any area masked by a solid `green` mask will be `35.77%` opaque. If the `mask-mode` for this mask is set to `alpha`, since `green` is a fully opaque color, the masked area will be `100%` opaque.

Multiple values

Each `mask-mode` value is a comma-separated list of values. When multiple values are present, each value corresponds to a mask layer in the same position in the `mask-image` property. The values define whether the associated mask images are treated as a `luminance` or an `alpha` mask.

Understanding <code>match-source</code>

In the case of `match-source`, whether `luminance` or `alpha` is used depends on the mask mode of the mask source. If the `mask-image` property values is a reference to an SVG `<mask>` element, the `<mask>` element's `mask-type` property value is used. If there is no CSS `mask-type` property set on the `<mask>` element, the value of the `<mask>` element's `mask-type` attribute is used, if present and supported. If neither is explicitly set, this value defaults to `luminance`; but only in the case of the `<mask>` element as the mask source. Otherwise, as noted before, if the mask image source is an `<image>`, rather than an SVG `<mask>`, the `alpha` values of the mask layer image is used.

Formal definition

Initial value	`match-source`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-mode =
  <masking-mode>#

<masking-mode> =
  alpha         |
  luminance     |
  match-source
```

This syntax reflects the latest standard as per CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Usage and values

This example demonstrates the basic usage and different values of the `mask-mode` property.

HTML

We include three `<div>` elements, so we can demonstrate the three enumerated `mask-mode` keyword values.

```
<div class="alpha">ALPHA</div>
<div class="luminance">LUMINANCE</div>
<div class="matchSource">MATCH-SOURCE</div>
```

CSS

Each `<div>` is provided with the same background and masking image. The only difference between each `<div>` is the value of the `mask-mode` property:

```
div {
  background: blue linear-gradient(red, blue);
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mdn.svg");
}

.alpha {
  mask-mode: alpha;
}

.luminance {
  mask-mode: luminance;
}

.matchSource {
  mask-mode: match-source;
}
```

```
div {
  width: 227px;
  height: 200px;
  float: left;
  text-align: center;
  line-height: 65px;
  color: white;
  text-shadow: 1px 1px 2px black;
  font-family: sans-serif;
}
```

Results

Because the mask source is an `<image>` and not an SVG `<mask>`, the `match-source` value resolves to `alpha`.

See also

- `mask-type`
- `mask-image`
- `mask` shorthand
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-origin CSS property

The **`mask-origin`** CSS property sets the origin of a mask. This property determines the mask positioning area: the area within which a mask image is positioned. HTML elements can have masks contained within their content border box, padding box, or content box, whereas SVG elements (which don't have the associated CSS layout boxes) can have masks contained inside their fill, stroke, or view box. For elements rendered as multiple boxes, such as a `<span>` of text that spans more than one line, the `mask-origin` property specifies which boxes the `box-decoration-break` property operates on to determine the mask positioning area.

Syntax

```
/* Keyword values */
mask-origin: content-box;
mask-origin: padding-box;
mask-origin: border-box;
mask-origin: fill-box;
mask-origin: stroke-box;
mask-origin: view-box;

/* Multiple values */
mask-origin: padding-box, content-box;
mask-origin: view-box, fill-box, border-box;

/* Global values */
mask-origin: inherit;
mask-origin: initial;
mask-origin: revert;
mask-origin: revert-layer;
mask-origin: unset;
```

Values

The `mask-origin` property is a comma-separated list of `<coord-box>` keyword values, including:

`content-box`

: The position is relative to the content box.

`padding-box`

: The position is relative to the padding box.

`border-box`

: The position is relative to the border box.

`fill-box`

: The position is relative to the object bounding box.

`stroke-box`

: The position is relative to the stroke bounding box.

`view-box`

: Uses the nearest SVG viewport as reference box. If a `viewBox` attribute is specified for the element creating the SVG viewport, the reference box is positioned at the origin of the coordinate system established by the `viewBox` attribute and the dimension of the reference box is set to the width and height values of the `viewBox` attribute.

There are three non-standard values that are shortcuts for standard `<coord-box>` values: `content` is an alias for `content-box`, `padding` is an alias for `padding-box`, and `border` is an alias for `border-box`.

Description

The `mask-origin` property is very similar to the `background-origin` property, but it has a different set of values and a different initial value. The initial value depends on if there is an associated CSS layout box; if yes, the default value is `border-box`. In comparison, the default for `background-origin` is `padding-box`.

For SVG elements without an associated CSS layout box, the values `content-box`, `padding-box`, and `border-box` (the default value) compute to `fill-box`, meaning the position is relative to the object bounding box. For HTML elements, if an SVG-related value of `fill-box`, `stroke-box`, or `view-box` is set, the value is computed to `border-box`.

An element can have multiple mask layers applied. The number of layers is determined by the number of comma-separated values in the `mask-image` property value (even if one or more of those values is `none`). Each `mask-origin` value in the comma-separated list of values is matched with a comma-separated `mask-image` value, in the same order.

If the number of values in the two properties differs, any excess values of `mask-origin` are not used in cases where `mask-origin` has more values than `mask-image`. If `mask-origin` has fewer values than `mask-image`, the `mask-origin` values are repeated.

For elements rendered as a single box, this property specifies the mask positioning area — or the origin position — of the image referenced by the `mask-image` property.

For elements rendered as multiple boxes, such as inline boxes that span more than one line, the `mask-origin` property specifies which boxes the `box-decoration-break` property operates upon to determine the mask positioning area.

The `mask-origin` can cause the mask layer image to be clipped. For example, if the `mask-clip` property is set to `padding-box`, the `mask-origin` is set to `border-box`, the `mask-position` is set to the `top left` edge, and the element has a border, then the mask layer image will be clipped at the top-left edge.

Formal definition

Initial value	`border-box`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-origin =
  <coord-box>#

<coord-box> =
  <paint-box>  |
  view-box

<paint-box> =
  <visual-box>  |
  fill-box      |
  stroke-box

<visual-box> =
  content-box  |
```

```
  padding-box  |
  border-box
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing content, padding, and border

This example demonstrates basic usage while comparing three values of the `mask-origin` property.

HTML

We include four `<section>` elements, each containing one `<div>` element.

```
<section class="content">
  <div></div>
</section>
<section class="padding">
  <div></div>
</section>
<section class="border">
  <div></div>
</section>
<section class="comparison">
  <div></div>
</section>
```

CSS

We apply `border`, `padding`, and `margin` to every `<div>`. These create the reference points for the mask image origin. The `border` shorthand includes a `border-color`. We also include a `background-color`. These provide a green background and a blue border to mask. Finally, all our `<div>` elements are provided with a `mask-image`.

```
div {
  width: 100px;
  height: 100px;
  margin: 10px;
  border: 10px solid blue;
  background-color: #8cffa0;
  padding: 10px;
  mask-image: url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
}
section {
  border: 1px solid black;
}
```

We give each `<div>` a different `mask-origin` value.

```
.content div {
  mask-origin: content-box;
}

.padding div {
  mask-origin: padding-box;
}

.border div {
  mask-origin: border-box;
}

.comparison div {
  mask-image: none;
}
```

We also generate some text inside each `<section>` to indicate the mask origin for each `<div>` container.

```
section::before {
  content: attr(class);
  display: block;
  text-align: center;
}
```

```
body {
  display: flex;
  flex-flow: row wrap;
  gap: 10px;
}
```

Result

Notice the difference between the three values. In the first three boxes, respectively, the mask originates from:

- The outside edge of the border.
- The inside border edge, which is the outer edge of the padding box.
- The inside padding edge, which is the outer edge of the content box.

The fourth box has no `mask-image` specified: it is a reference image, included to allow you to easily visualize the extent of the content and padding areas.

Multiple values

This example demonstrates using different `mask-origin` values for different `mask-image`s applied to a single element.

HTML

We include a single `<div>`.

```
<div></div>
```

CSS

We apply three mask images instead of one, each with a different `mask-position`. We also set the mask images not to repeat.

```
div {
  width: 120px;
  height: 120px;
  margin: 10px;
  border: 10px solid blue;
  background-color: #8cffa0;
  padding: 10px;
  mask-image:
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg"),
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg"),
    url("https://mdn.github.io/shared-assets/images/examples/mask-star.svg");
  mask-position:
    top left,
    top right,
    bottom center;
  mask-repeat: no-repeat;
  mask-origin: content-box, border-box;
}
```

Results

We have three `mask-image` values, but only two `mask-origin` values. This means the `mask-origin` values are repeated, as if we had set `mask-origin: content-box, padding-box, content-box;`. The `border-box` star, the only mask overlapping the border, is the top-right star.

See also

- `background-origin`
- `mask-image`
- `mask-position`
- `mask-repeat`
- `mask-size`
- `mask` shorthand
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-position CSS property

The **`mask-position`** CSS property sets the initial position, relative to the mask position layer set by `mask-origin`, for each defined mask image.

Syntax

```
/* Single <position> keyword value */
/* Sets second value to 'center' */
mask-position: left;
mask-position: center;
mask-position: right;
mask-position: top;
mask-position: bottom;

/* Two <position> keyword values */
mask-position: left center;
mask-position: right top;

/* One length or percentage <position> value */
/* Horizontal position. Vertical position set to 'center' */
mask-position: 25%;
mask-position: 0px;
mask-position: 8em;

/* Two length or percentage <position> values */
/* First value: horizontal position. Second value: vertical position */
mask-position: 25% 75%;
mask-position: 0px 0px;
mask-position: 10% 8em;

/* Edge offsets: Four <position> values */
mask-position: bottom 10px right 20px;
mask-position: right 3em bottom 10px;
mask-position: bottom 10px right 0;

/* Multiple <position> values */
mask-position:
  top left,
  bottom 10px right 10px;
mask-position:
  1rem 1rem,
  center;

/* Global values */
mask-position: inherit;
mask-position: initial;
mask-position: revert;
mask-position: revert-layer;
mask-position: unset;
```

Values

This property is specified as a comma-separated list of values. Each value has the following type:

`<position>`

> One, two, or four values representing a 2D position specifying the edges of the element's box. Relative or absolute offsets can be given.

Description

The `mask-position` property defines the position of each mask layer. An element can have multiple mask layers applied. The number of layers is determined by the number of comma-separated values in the `mask-image` property value (even `none` values create a layer).

Each `mask-position` value in the comma-separated list of values is matched up with an associated mask layer as defined by the list of `mask-image` values, in order. If the number of values in the two properties differs:

- If `mask-position` has more values than mask-image, the excess values of `mask-position` are not used.
- If `mask-position` has fewer values than mask-image, the `mask-position` values are repeated.

Each `mask-position` defines the associated mask layer's position relative to the associated `mask-origin` value. The `mask-origin` property values are similarly matched up with the `mask-image` values, in order, with excess `mask-position` values being unused or `mask-position` values being repeated if they are fewer in number than the `mask-origin` values. Each mask layer, therefore, has an associated `mask-origin` and `mask-position` value.

If no `mask-origin` is set, the value defaults to `padding-box`, meaning the origin of each `mask-position` is the element's padding-box.

One-value syntax

If only one `mask-position` value is specified, the second value is assumed to be `center`. If the value is a `<length>` or `<percentage>`, it defines the position of the mask along the horizontal axis, with the mask being vertically centered within the origin box. For example, `mask-position: 0%;` is equal to `mask-position: 0% center`.

If you use a single keyword for positioning, the other value will resolve to `center`. The default of `mask-position` is `0% 0%`, which equates to `mask-position: top left`. However:

- `mask-position: top;` is equivalent to `mask-position: top center;`.
- `mask-position: left;` is equivalent to `mask-position: center left`.
- `mask-position: center;` is equal to `mask-position: center center`.

If the value is a `<length>` value, it represents the horizontal position as an offset from the left edge of the mask positioning. A positive value represents an offset inward from the left edge of the box container. The position can be set outside of the element's box using a negative value — this creates an outward offset that places the item outside the container's left edge.

Percentage values

A `<percentage>` value represents the mask's horizontal position value relative to the width of the container, positioned relative to the left edge. However, the offset is not from the mask edge to the box edge. Instead, the mask image dimension is subtracted from the container's dimension, and then a percentage of the resulting value is used as the direct offset from the box's left edge, which is the same as percentage values for `background-position`.

The equation is:

```
(container dimension - mask dimension) * position percentage = dimension offset value
```

Given a `100px`-wide mask and a `1000px`-wide origin box, setting `mask-position: 10%;` (the equivalent of `10% 50%`) results in the mask being vertically centered at `90px` from the left edge. The equation is `(1000 - 100) * 10% = 90`. If the left offset had been `0%`, the mask's left edge would be flush to the left of the container (`(1000 - 100) * 0% = 0`).

If the left offset had been `100%`, the mask's right edge would be flush to the right of the container as the left edge of the `100px` wide mask would be `900px` (`(1000 - 100) * 100% = 900`) from the left edge of the container (the `100px` mask width plus `900px` distance from the left edge means the right edge would be `1000px` from the left edge, which is the right edge of the container).

Two-value syntax

A two-value `<position>` specifies the position of the mask image inside its mask positioning area, with length and percentage values specifying offsets from the `left` and `top` of the area.

If the two values are `<length>` values, `<percentage>` values, or the keyword `center`, the first value represents the horizontal position as an offset from the left edge of the mask positioning area, and the second value represents the vertical position as an offset from it's top edge, with percentages being offset by the mask's size in that dimension.

In addition, if `<percentage>` values are specified, the first value is also the horizontal position value relative to the left edge, and the second value is also the vertical position value relative to the top edge.

A pair of axis-specific keywords can be reordered, as can an axis-specific keyword and a length or percentage, but two length or percentage values are not interchangeable. If one of the two values is `top`, `right`, `bottom`, or `left`, the order of the two values doesn't matter. Any `center` or `<length-percentage>` value in the pair of values will be applied to the other dimension.

Four-value syntax

The four-value syntax consists of two pairs of values, each pair containing a keyword specifying the edge to offset from, and a `<length>` and `<percentage>` value specifying the offset distance. For example, `mask-position: left 1em top 2em` specifies a `1em` horizontal offset from the left box edge and a `2em` vertical offset from the top edge. The two-value equivalent would be `mask-position: 1em 2em`.

Because we're defining the offset edges when using the four-value syntax, the order isn't important: `mask-position: top 2em left 1em` and `mask-position: left 1em top 2em` both produce the same result.

The real power of the four-value syntax is that it allows us to specify offset edges other than `left` and `top`. For example, `mask-position: bottom 10px right 20px` creates a `10px` vertical offset up from the bottom edge and a `20px` horizontal offset leftward from the right edge. Usually, the four-value syntax is used to offset from the bottom and/or right. But this syntax is also helpful if you can't remember the offset edge order for the two-value syntax.

One thing to note is that, unlike the `<bg-position>` data type values for `background-position`, the `<position>` values for `mask-position` do not allow for a 3-value syntax and do not allow offsetting from `center`. When offsetting the mask from the `bottom` or `right`, the `mask-position` requires all four values to be declared.

For the four-value syntax to be valid, it needs to specify either `top` or `bottom` as the vertical offset edge, along with the vertical length or percentage offset value, and either `left` or `right` as the horizontal offset edge, along with the horizontal length or percentage offset value.

Formal definition

Initial value	`0% 0%`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Percentages	refer to size of mask painting area minus size of mask layer image (see the text for `background-position`)
Computed value	Consists of two keywords representing the origin and two offsets from that origin, each given as an absolute length (if given a <length>), otherwise as a percentage.
Animation type	a repeatable list

Formal syntax

```
mask-position =
  <position>#

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-one> =
```

```
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Masking Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

```
<section>
  <div></div>
</section>
```

```
section {
  border: 1px solid black;
  width: 250px;
  height: 250px;
}

div {
  width: 250px;
  height: 250px;
  margin-bottom: 10px;
  background: blue linear-gradient(red, blue);

  mask-image: url("/shared-assets/images/examples/mask-star.svg");
  mask-repeat: no-repeat;
  mask-position: top right;
}
```

See also

- `background-position`
- `mask-image`
- `mask-origin`
- `mask-repeat`
- `mask-size`
- `mask` shorthand
- `mask-border`
- `mask-border-outset`
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module
- `background-position`
- `<position>`

mask-repeat CSS property

The **`mask-repeat`** CSS property sets how mask images are repeated. A mask image can be repeated along the horizontal axis, the vertical axis, both axes, or not repeated at all.

Syntax

```
/* One-value syntax */
mask-repeat: repeat-x;
mask-repeat: repeat-y;
mask-repeat: repeat;
mask-repeat: space;
mask-repeat: round;
mask-repeat: no-repeat;

/* Two-value syntax: horizontal | vertical */
mask-repeat: repeat space;
mask-repeat: repeat repeat;
mask-repeat: round space;
mask-repeat: no-repeat round;

/* Multiple values */
mask-repeat:
  space round,
  no-repeat;
mask-repeat:
```

```
  round repeat,
  space,
  repeat-x;

/* Global values */
mask-repeat: inherit;
mask-repeat: initial;
mask-repeat: revert;
mask-repeat: revert-layer;
mask-repeat: unset;
```

Values

The `mask-repeat` property is a comma-separated list of two `<repeat-style>` values, with the first `<repeat-style>` value being the horizontal repetition value and the second value the vertical repetition value, or one keyword value that is a shorthand for two `<repeat-style>` values.

`<repeat-style>` values

The `<repeat-style>` values include:

`repeat`

> The image is repeated as much as needed to cover the whole mask painting area. Mask images along the edges are clipped when the size of the mask origin box is not an exact multiple of the mask image's size.

`space`

> The mask image is repeated as many times as possible without clipping. If the element's origin size is at least twice the size as the mask image's size in the associated dimension, the `mask-position` property is ignored and the first and last images are positioned at the edges of the mask origin container. If the mask origin box is not an exact multiple of the mask image's size, whitespace is distributed evenly between the repeated mask images.
>
> If the origin box size is less than twice the mask image's size in the given dimension, only one mask image can be displayed. In this case, the image is positioned as defined by the `mask-position` property, which defaults to `0% 0%`. The mask image will only be clipped if the mask image is larger than the mask origin box.

`round`

> The mask image is repeated as many times as possible in its original dimensions. If the size of the mask origin box is not an exact multiple of the mask image's size, all mask images will be rescaled, shrinking or stretching to ensure no repetitions are clipped.

`no-repeat`

> The mask image is not repeated (and hence the mask painting area will not necessarily be entirely covered). The position of the non-repeated mask image is defined by the `mask-position` CSS property.

Shorthand values

The one-value syntax is a shorthand for the full two-value syntax:

Single value	Two-value equivalent
`repeat-x`	`repeat no-repeat`
`repeat-y`	`no-repeat repeat`
`repeat`	`repeat repeat`
`space`	`space space`
`round`	`round round`
`no-repeat`	`no-repeat no-repeat`

`repeat-x`

> The equivalent of `repeat no-repeat`. The image is repeated in the horizontal direction as many times as needed to cover the width of the mask painting area. Mask images along the right or left edges, or both depending on the `mask-position` value, will be clipped if the width of the mask origin box is not an exact multiple of the mask image's width.

`repeat-y`

> The equivalent of `no-repeat repeat`. The image is repeated in the vertical direction as many times as needed to cover the height of the mask painting area. Mask images along the top or bottom edges, or both depending on the `mask-position` value, will be clipped if the height of the mask origin box is not an exact multiple of the mask image's height.

Description

The `mask-repeat` property accepts a comma-separated pair of values or one shorthand value. In the two-value syntax, the first value represents the horizontal repetition behavior and the second value represents the vertical behavior.

Multiple values

Each `mask-repeat` value in this comma-separated list applies to a separate mask layer. An element can have multiple mask layers applied. The number of layers is determined by the number of comma-separated values in the `mask-image` property value (even if a value is `none`). Each `mask-repeat` value is matched up with the `mask-image` values, in order. If the number of values in the two properties differs, any excess values of `mask-repeat` values are ignored, or, if `mask-repeat` has fewer values than `mask-image`, the `mask-repeat` values are repeated.

Sizing and positioning

The `mask-repeat` property value defines how mask images are tiled after they have been sized and positioned. The first (and possibly only) mask-image repetition is positioned by the `mask-position` property, which defaults to `0% 0%`, the top-left corner of the origin box. The size is defined by the `mask-size` property, which defaults to `auto`. The positions of the repeated masks are based on this initial mask instance.

Clipping

Mask images will not repeat but will be clipped if the mask image's size is larger than the origin box, except in the case of `round`, where a single mask will be scaled down to fit the origin box.

With `repeat` values, mask images may be clipped at the edge of the origin box if the dimension (width or height) of the box is not an exact multiple of the mask's.

Aspect ratio

By default, mask images maintain the aspect ratio set by the `mask-size` property or their default aspect ratio if the `mask-size` defaults to or is explicitly set to `auto`. Only in the case of `round` value in both directions might the mask aspect ratio be distorted.

If `round` is set in both directions, the resulting repeated mask images will match the aspect ratio of the origin box. As the mask images are scaled to fit, they may be distorted, ensuring the mask images are not clipped. If `round` is set in only one dimension, the aspect ratio of the mask size is respected.

Rounded repetitions

In the case of `round`, mask images are scaled up or down to fit the mask image in the positioning area a whole number of times. The mask size increases or decreases to fit the nearest natural number or masks, with a minimum of one mask.

The rendered dimensions of the mask is the size of the origin box divided by the number of iterations of masks in that dimension, where the iterations being an integer greater than zero. The number of iterations is: `X' = D / round(D / X)` where `D` is the width or height, and `round()` is a function that returns the nearest integer greater than zero.

For example, if `mask-repeat` is set to `round` and the `mask-size` sets the mask to be `40px` wide, when the origin box is present (greater than `0px` wide) but less than `60px` wide, there will be a single iteration that is 100% of the width of that box. If the box is at least `60px` wide but less than `100px` wide, there will be two iterations that are each `50%` of the box. From 100px to 140px, three masks will fit along the horizontal axis. These "`40px`-wide" masks will only be `40px` wide if the origin box is an exact multiple of `40px`.

Formal definition

Initial value	`repeat`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	Consists of two keywords, one per dimension
Animation type	discrete

Formal syntax

```
mask-repeat =
  <repeat-style>#

<repeat-style> =
  repeat-x          |
  repeat-y          |
  repeat-block      |
  repeat-inline     |
  <repetition>{1,2}

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting the `mask-repeat` property for a single mask.

HTML

Our HTML includes a basic `<div>` element:

```
<div></div>
```

CSS

We define a `250px` square with a red to blue gradient with a `100px` by `100px` star as a mask image. We use the `mask-repeat` property, setting `round` for the horizontal direction and `space` for the vertical value.

```
div {
  width: 250px;
  height: 250px;
  background-image: linear-gradient(red, blue);

  mask-image: url("/shared-assets/images/examples/mask-star.svg");
  mask-size: 100px 100px;

  mask-repeat: round space;
}
```

Results

With `space` and `round` on a mask image that is smaller than the origin box, the mask is not clipped. Rather, the `round` value distorts the star to prevent clipping and preventing white space, while `space` maintains the star's aspect ratio, but adds space as needed between masks.

Round iterations

Using the same HTML and CSS, this demonstration includes a slider that changes the width of the container to show how, with `round`, masks will grow as space allows until another iteration of the mask fits, or shrink until the number of iterations no longer fits.

```
<div></div>
<label
  >width: <span></span><br />
  <input type="range" min="0" max="400" value="250" id="width" />
</label>
```

```
div {
  width: 250px;
  height: 250px;
  background-image: linear-gradient(red, blue);

  mask-image: url("/shared-assets/images/examples/mask-star.svg");
  mask-size: 100px 100px;

  mask-repeat: round space;
}
input {
  clear: both;
```

```
  width: 90%;
}
```

```
const div = document.querySelector("div");
const range = document.getElementById("width");
const output = document.querySelector("span");

range.addEventListener("change", () => {
  const value = `${range.value}px`;
  output.innerText = value;
  div.style.width = value;
});
```

The mask is defined as `100px` wide. There is a single star when the origin box is from `1px` to `149px` wide, two stars from `150px` to `249px`, three stars from `250px` to `349px`, and so on.

The shorthand values

This examples demonstrates all `mask-repeat` shorthand (single-keyword) values.

HTML

We include several `<section>` elements each containing a `<div>`, each with a different class name.

```
<section class="repeat">
  <div></div>
</section>
<section class="space">
  <div></div>
</section>
<section class="round">
  <div></div>
</section>
<section class="no-repeat">
  <div></div>
</section>
<section class="repeat-x">
  <div></div>
</section>
<section class="repeat-y">
  <div></div>
</section>
```

CSS

We give every `<div>` the same CSS, except for the `mask-repeat` value, which we match to their parent's class name. We define a mask size, setting the initial `mask-image` at the bottom right, meaning any clipping will occur on the top-most and left-most masks, on their top and left sides.

```
div {
  width: 180px;
  height: 180px;
  background-image: linear-gradient(red, blue);

  mask-image: url("/shared-assets/images/examples/mask-star.svg");

  mask-size: 50px 50px;
  mask-position: bottom right;
}

.repeat div {
  mask-repeat: repeat;
}
.space div {
  mask-repeat: space;
}
.round div {
  mask-repeat: round;
}
.no-repeat div {
  mask-repeat: no-repeat;
}
.repeat-x div {
  mask-repeat: repeat-x;
}
.repeat-y div {
  mask-repeat: repeat-y;
}
```

We display the class name using generated content.

```
section::before {
  content: attr(class);
  display: block;
  text-align: center;
  border-bottom: 1px solid;
}
```

```
body {
  display: flex;
  flex-flow: row wrap;
  gap: 10px;
}
section {
  border: 1px solid;
}
```

Results

The first (and, in the case of `no-repeat`, only) mask star is sized to be 50px by 50px, and positioned at the bottom-right of the painting area, with repeated stars placed above and/or to the left of it with any clipping occurring on the top and left of the top-most and left-most stars. Note that all the stars are the same size and shape, except for `round`, where all the masks shrank to 45px x 45px to fit four complete

masks in each direction. Had the container been 174px, there would have been three stars in each direction, instead of four, and each star would have been stretched.

Multiple mask images and repeats

You can specify a different `<repeat-style>` for each mask image, separated by commas:

```
.extra-repeats {
  mask-image: url("mask1.png"), url("mask2.png");
  mask-repeat: repeat-x, repeat-y, space;
}
```

Each image is matched with the corresponding repeat style. As there are more `mask-repeat` values than `mask-image` values, the excess `mask-repeat` values are ignored.

```
.missing-repeats {
  mask-image:
    url("mask1.png"), url("mask2.png"), url("mask3.png"), url("mask4.png");
  mask-repeat: repeat-x, repeat-y;
}
```

Each image is matched with a corresponding repeat style. As there are more `mask-image` values than `mask-repeat` values, the `mask-repeat` is repeated until every `mask-image` has an associated `mask-repeat` value. Here, odd numbered masks repeat along the x-axis, while the even numbered masks repeat along the y-axis.

See also

- `background-repeat`
- `mask-border-repeat`
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-size CSS property

The **`mask-size`** CSS property specifies the sizes of specified mask images. Mask image sizes can be fully or partially constrained to preserve their intrinsic aspect ratios.

Syntax

```
/* Keyword syntax */
mask-size: cover;
mask-size: contain;
mask-size: auto;

/* One-value syntax */
/* Mask width. Sets height to 'auto'. */
mask-size: 50%;
mask-size: 3em;
mask-size: 12px;

/* Two-value syntax */
/* First value: mask width. Second value: mask height */
mask-size: 3em 25%;
mask-size: auto 6px;
mask-size: auto 50%;

/* Multiple values */
mask-size: auto, contain;
mask-size:
  50%,
  50% 25%,
  auto 25%;
mask-size: 6px, auto, contain;

/* Global values */
mask-size: inherit;
mask-size: initial;
mask-size: revert;
mask-size: revert-layer;
mask-size: unset;
```

Values

The `mask-size` property accepts a comma-separated list of `<bg-size>` values. A `<bg-size>` value is either `cover`, `contain`, a pair of values specifying the width and height (in that order), or a single value specifying the width (in which case, the height is set to `auto`). Values include:

`contain`

> Scales the mask image up or down, while preserving its aspect-ratio, making the mask as large as possible within its container without cropping or stretching it. If the mask image is smaller than the container, the mask will tile, or repeat, unless the `mask-repeat` property is set to `no-repeat`.

`cover`

> Scales the mask image to the smallest possible size to fill the container while preserving its aspect ratio. If the aspect ratio of the mask image differs from the element, it will be cropped vertically or horizontally.

`auto`

> Maintains the original aspect ratio of the mask source. When set for both the width and height, the origin size of the mask resource is used. Otherwise, `auto` scales the mask image in the corresponding direction such that its original aspect ratio is maintained.

`<length>`

> Renders the mask image at the specified length in the corresponding dimension (width if set as the first or only value, height if set as the second value). Negative values are not allowed.

`<percentage>`

> Renders the mask image in the corresponding dimension to the specified percentage of the box origin area as defined by the `mask-origin` property, which defaults to `padding-box`. Negative values are not allowed.

Description

The `mask-size` property is used to size mask layers.

An element can have multiple mask layers applied. The number of mask layers is determined by the number of comma-separated values in the `mask-image` property value (a value creates a mask layer, even if it is `none`).

Each `mask-size` value in the comma-separated list of values is matched up with an associated mask layer as defined by the list of `mask-image` values, in order. If the number of values in the two properties differs:

- If `mask-size` has more values than `mask-image`, the excess values of `mask-size` are not used.
- If `mask-size` has fewer values than `mask-image`, the `mask-size` values are repeated.

Each `mask-size` value is a `<bg-size>` value. There are three ways to declare each `<bg-size>`: one keyword, two lengths, percentages or the keyword `auto`, or one length, percentage, or `auto`:

- The available keywords are `cover` and `contain`.
- When two values are specified, the first defines the mask width and the second defines its height.
- When one value is specified, it defines only the mask width, with the height set to `auto`.

The width and height values are a `<length>`, a `<percentage>`, or the `auto` keyword, which is the default. Setting one or both values to `auto` maintains the mask image's original aspect ratio.

The rendered size of the mask image is computed as follows:

- If both components of `mask-size` are specified and are not `auto`, the mask image renders at the specified size.
- If the `mask-size` is `contain` or `cover`, the image is rendered by preserving its aspect ratio at the largest size contained within or covering the mask positioning area. If the image has no intrinsic proportion, such as with gradients, then it is rendered at the size of the mask positioning area.
- If the `mask-size` is `auto` (which resolves to `auto auto`), it is rendered at the size at which the mask would be displayed if no CSS were applied to change the rendering; this is its intrinsic size. If it has no intrinsic dimensions and no intrinsic proportion, as is the case with CSS gradients, it is rendered at the size of the mask positioning area, defined by the `mask-origin` (which defaults to `border-box`). If the mask source has no dimensions but has a proportion (aspect-ratio), a value of `auto` will render it as if `contain` had been specified instead. If the image has one intrinsic dimension and a proportion, it is rendered at the size determined by that one dimension and the proportion. If the image has one intrinsic dimension but no proportion, it's rendered using the intrinsic dimension and the corresponding dimension of the mask positioning area.
- If `mask-size` has one `auto` component and one non-`auto` component, which applies to all single-value values, the aspect ratio is maintained if the mask source has an intrinsic proportion. If there are no intrinsic proportions, the `auto` value is assumed to be the dimension of the mask positioning area.

Like with all longhand components of shorthand property, if the `mask` shorthand property is set and the value of the `mask-size` property is not defined within any mask layer, the `mask-size` value is reset to its initial value of `auto` for those mask layers.

Formal definition

Initial value	`auto`
Applies to	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	no
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a repeatable list

Formal syntax

```
mask-size =
  <bg-size>#

<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Backgrounds Module Level 4, CSS Masking Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting mask size as a percentage

This example demonstrates basic usage, while also demonstrating percentage values for `mask-size`.

HTML

We include two `<div>` elements:

```
<div class="width"></div>
<div class="height"></div>
```

CSS

The `<div>` elements are defined to be twice as tall as they are wide, with a gradient background and mask:

```
div {
  width: 200px;
  height: 400px;
  background: blue linear-gradient(red, blue);
  mask-image: url("/shared-assets/images/examples/mdn.svg");
}
```

The width of one `<div>` element's mask is set to `50%`, with the height defaulting to `auto`. The mask's height for the second `<div>` element is set to `50%` with the width set to `auto`:

```
.width {
  mask-size: 50%;
}

.height {
  mask-size: auto 50%;
}
```

In the `width` case, the mask is rendered `100px` wide (`50%` of the `200px`-wide element). The height defaults to `auto`, maintaining the mask's aspect ratio. In the `height` case, the mask is rendered `200px` tall (`50%` of the `400px`-high container). The width is explicitly set to `auto`, maintaining the mask's aspect ratio.

```
body {
  display: flex;
  gap: 20px;
}
```

Results

Cover and contain

This example demonstrates the keyword values for `mask-size`.

HTML

Three `<section>` elements are defined, each with a different class name, and each containing a `<div>`.

```
<section class="auto">
  <div></div>
</section>
<section class="cover">
  <div></div>
</section>
<section class="contain">
  <div></div>
</section>
```

CSS

The `<div>` elements are defined to be twice as tall as they are wide, with a gradient background and mask:

```
div {
  width: 200px;
  height: 400px;
  background: blue linear-gradient(red, blue);
  mask-image: url("/shared-assets/images/examples/mask-star.svg");
}
```

The `mask-size` of two of the `<div>` elements is set to one of the property's keyword values. The third `<div>` has a `mask-size` of `auto` set demonstrating the original intrinsic dimensions of the mask:

```
.auto div {
  mask-size: auto;
}

.cover div {
  mask-size: cover;
}

.contain div {
  mask-size: contain;
}
```

The property values is displayed using generated content.

```
section::before {
  content: "mask-size: " attr(class) ";";
  display: block;
  text-align: center;
  border-bottom: 1px solid;
}
```

```
body {
  display: flex;
  flex-flow: row wrap;
  gap: 10px;
}
section {
  border: 1px solid;
}
```

Results

With `auto`, the star is displayed at its intrinsic `100px` by `100px` size. With `cover`, the star grows to be `400px` tall, covering the entire origin box. With `contain`, the star grows until one dimension equals the same dimension of the origin box, meaning that the star is as large as it can be (`200px` wide) but still contained by it.

When the mask is larger than the container

Using the same HTML and CSS as above, with just a different origin box size, this example explores what happens when the origin box is smaller than the intrinsic dimensions of the mask.

```
<section class="auto">
  <div></div>
</section>
<section class="cover">
  <div></div>
</section>
<section class="contain">
  <div></div>
</section>
```

```
div {
  background: blue linear-gradient(red, blue);
  mask-image: url("/shared-assets/images/examples/mask-star.svg");
}

.auto div {
  mask-size: auto;
}
```

```
.cover div {
  mask-size: cover;
}

.contain div {
  mask-size: contain;
}

section::before {
  content: attr(class);
  display: block;
  text-align: center;
  border-bottom: 1px solid;
}

body {
  display: flex;
  flex-flow: row wrap;
  gap: 10px;
}
section {
  border: 1px solid;
}
```

The only difference is the size of the containing box (and the generated content):

```
div {
  width: 70px;
  height: 70px;
}
```

The `contain` value contains the mask within the origin box. The `cover` value covers it. In both cases, the mask shrinks while maintaining the original aspect ratio. With `auto`, the mask is clipped because the intrinsic dimensions are larger than the box dimensions.

See also

- `background-size`
- `mask` shorthand
- `mask-image`
- `mask-origin`
- `mask-position`
- `mask-repeat`
- `mask-image`
- `mask-border`
- `background-size`
- `mask-border-width`
- Introduction to CSS masking
- CSS `mask` properties
- Declaring multiple masks
- CSS masking module

mask-type CSS property

The **`mask-type`** CSS property applies to the SVG `<mask>` element. It defines whether to use the *luminance* (brightness) or *alpha* (transparency) content of the mask. This property may be overridden by the `mask-mode` property. The `mask-type` property has no effect on image or gradient masks.

Syntax

```
/* Keyword values */
mask-type: luminance;
mask-type: alpha;

/* Global values */
mask-type: inherit;
mask-type: initial;
mask-type: revert;
mask-type: revert-layer;
mask-type: unset;
```

Values

`alpha`

Indicates that the alpha (transparency) values of the `<mask>` should be used.

`luminance`

Indicates that the luminance (brightness) values of the `<mask>` should be used.

Description

The `mask-type` property is only relevant for the SVG `<mask>` element. If you set `mask-type: luminance`, the mask will use how bright each part of the mask is; if you set `mask-type: alpha`, it will use how transparent or opaque each part of the mask is.

Default behavior

By default, the SVG `<mask>` element uses `mask-type: luminance`. This means both the color and the transparency of the mask content affect masking. Whether the mask is opaque partially depends on the lightness of the color of the opaque areas:

- Fully opaque white areas (100% luminance) will be masked and visible.
- Black (0% luminance) or fully transparent areas will be clipped and invisible.
- Areas with intermediate luminance values will be partially masked, reflecting both the luminance, or lightness of the mask color, and the alpha transparency of each area of the mask.

Understanding luminance

The opacity of a `luminance` mask is determined by the `R`, `G`, `B`, and `A` values of an `rgb()` color using the following formula:

`((0.2125 * R) + (0.7154 * G) + (0.0721 * B)) * A`

For example, the color `green` (`#008000` or `rgb(0% 50% 0% / 1)`) has a luminance value of `35.77%`. Any area masked by a solid `green` luminance mask will be `35.77%` visible. If the `mask-type` is set to `alpha`, the same fully opaque `green` color will make the masked area `100%` visible.

If the SVG `<mask>` element has `fill="rgb(0 0 0 / 0.5)"`, which is a 50% transparent black, the corresponding shape on the masked element will display at 50% opacity when `mask-type` is set to `alpha` because the alpha values is `0.5` (50% opacity). But if the `mask-type` defaults to or is set as `luminance` the masked area will be fully clipped and invisible because its luminance is `0`.

Effect of <code>mask-mode</code> on <code>mask-type</code>

While the `mask-type` property is set on the SVG `<mask>` element, the `mask-mode` property is set on the element being masked (the element you're applying the mask to). If the mask image source is not an SVG `<mask>`, this property has no effect.

The default value of `mask-mode` is `match-source`, which means the browser uses the `mask-type` value from the `<mask>` element to determine how to interpret it. If `mask-mode` is set to a value other than `match-source`, that value takes precedence and overrides the `mask-type` value of the applied mask.

If the `<mask>` is defined as the mask image's source, and the `mask-mode` is set as or defaults to `match-source`, the `mask-mode` will resolve to the `<mask>` element's `mask-type` value: `luminance` or `alpha`. If not explicitly set, the value defaults to `luminance`.

Formal definition

Initial value	`luminance`
Applies to	`<mask>` elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
mask-type =
  luminance  |
  alpha
```

This syntax reflects the latest standard as per CSS Masking Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using the <code>mask-type</code> property

This example demonstrates how to use the `mask-type` property and the effect of its different values.

HTML

We've included two images that will be masked. Other than their class names, the two images are identical. We've also included an SVG with two `<mask>` elements. Other than their `id` values, the two masks are also identical: each has a green and white heart shape and a semi-transparent white-and-black-filled circle.

```
<img
  class="alphaMaskType"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />
<img
  class="luminanceMaskType"
  src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
  alt="Pride flag" />

<svg height="0" width="0">
  <mask id="alphaMask">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="green"
      stroke="white"
      stroke-width="20" />
    <circle
      cx="170"
      cy="170"
      r="40"
      fill="rgb(0 0 0 / 0.5)"
      stroke="rgb(255 255 255 / 0.5)"
      stroke-width="20" />
  </mask>
  <mask id="luminanceMask">
    <path
      d="M20,70 A40,40,0,0,1,100,70 A40,40,0,0,1,180,70 Q180,130,100,190 Q20,130,20,70 Z"
      fill="green"
      stroke="white"
      stroke-width="20" />
    <circle
      cx="170"
      cy="170"
      r="40"
      fill="rgb(0 0 0 / 0.5)"
      stroke="rgb(255 255 255 / 0.5)"
      stroke-width="20" />
  </mask>
</svg>
```

CSS

We apply the `mask-type` property to the `<mask>` elements, and then apply the `<mask>` elements and the mask source to the images.

```
mask#alphaMask {
  mask-type: alpha;
}

mask#luminanceMask {
  mask-type: luminance;
}
```

```
img.alphaMaskType {
  mask-image: url("#alphaMask");
}

img.luminanceMaskType {
  mask-image: url("#luminanceMask");
}
```

Result

As the default value for the `mask-mode` property is `match-source`, the first mask uses the alpha channels only to define the mask: the white and green are fully opaque, and the 50% white and black colors are 50% opaque because only the alpha value of the colors matters. The second example uses the luminance of the colors to determine the opacity of the mask, with white being brighter than green, and semi-transparent white being brighter than semi-transparent black.

See also

- `mask`
- `mask-mode`
- Introduction to CSS masking
- CSS masking module
- SVG `mask-type` attribute

math-depth CSS property

The **`math-depth`** property describes a notion of *depth* for each element of a mathematical formula, with respect to the top-level container of that formula. This is used to scale the computed value of the font-size of elements when `font-size: math` is applied.

> **Note:** `font-size: math` is the default for `<math>` elements in the MathML Core User Agent stylesheet, so it's not necessary to specify it explicitly.

Syntax

```
/* Keyword value */
math-depth: auto-add;

/* Relative values */
math-depth: add(2);
math-depth: add(-2);

/* Absolute value */
math-depth: 4;

/* Global values */
math-depth: inherit;
math-depth: initial;
math-depth: revert;
math-depth: revert-layer;
math-depth: unset;
```

Values

`auto-add`

Set to the inherited `math-depth` plus 1 when inherited math-style is `compact`.

`add(<integer>)`

Set to the inherited `math-depth` plus the specified integer.

`<integer>`

Set to the specified integer.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
math-depth =
  auto-add          |
  add( <integer> )  |
  <integer>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, MathML Core. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying a math depth

The following example shows the effect of changing the `math-depth` property on the font size of subformulas. The numbers in each subformula indicate the `math-depth` and scale factor applied.

The first `<mtext>` element is used as a reference to other subformulas and has no specific styles applied. The second and third subformulas have `math-depth` set to `auto-add` and show the effect of scaling depending on the `math-style`.

The last two subformulas show the effect of setting `math-depth` to a specific value.

HTML

```
<p>
  <math>
    <mtext>0</mtext>

    <!-- auto-add value has no effect when math-style is normal -->
    <mrow style="math-style: normal">
      <mrow style="math-depth: auto-add">
        <mtext>0</mtext>
      </mrow>
    </mrow>

    <!-- the inherited math-style is compact, so math-depth is set to 1 -->
    <mrow style="math-depth: auto-add">
      <mtext>1</mtext>
    </mrow>

    <mrow style="math-depth: add(2)">
      <mtext>2</mtext>
      <mrow style="math-depth: add(-1)">
        <mtext>1</mtext>
      </mrow>
      <mrow style="math-depth: 0">
        <mtext>0</mtext>
      </mrow>
    </mrow>
  </math>
</p>
```

```
p {
  font-size: 3rem;
  margin: 1rem 0;
}
```

Result

See also

- font-size
- math-style

math-shift CSS property

The `math-shift` property indicates whether superscripts inside MathML formulas should be raised by a normal or compact shift.

Syntax

```
/* Keyword values */
math-shift: normal;
math-shift: compact;

/* Global values */
math-shift: inherit;
math-shift: initial;
math-shift: revert;
math-shift: revert-layer;
math-shift: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

The initial value, indicates normal rendering. Superscripts in MathML formulas use the superscriptShiftUp parameter from the OpenType MATH table.

`compact`

Indicates compact rendering. Superscripts in MathML formulas use the superscriptShiftUpCramped parameter from the OpenType MATH table, which is generally smaller.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
math-shift =
  normal   |
  compact
```

This syntax reflects the latest standard as per MathML Core. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Normal and compact shift

The following MathML displays two versions of "x squared" using a font with an OpenType MATH table. The `math-shift` property is used to raise the superscripts using the `normal` and `compact` shift.

CSS

```
math {
  font-size: 64pt;
  math-shift: normal;
```

```
}

.compact-shift {
  math-shift: compact;
}
```

MathML

```
<math>
  <msup>
    <mi>x</mi>
    <mn>2</mn>
  </msup>
  <msup class="compact-shift">
    <mi>x</mi>
    <mn>2</mn>
  </msup>
</math>
```

Results

Note that the second "2" has a more compact (lower) shift.

See also

- `math-depth`
- `font-size`

math-style CSS property

The `math-style` property indicates whether MathML equations should render with normal or compact height.

Syntax

```
/* Keyword values */
math-style: normal;
math-style: compact;

/* Global values */
math-style: inherit;
math-style: initial;
math-style: revert;
math-style: revert-layer;
math-style: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

The initial value, indicates normal rendering.

`compact`

The math layout on descendants tries to minimize the logical height.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
math-style =
  normal   |
  compact
```

This syntax reflects the latest standard as per MathML Core. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing the style of a formula to compact

CSS

```
math {
  math-style: normal;
}
.compact {
  math-style: compact;
}
```

HTML

```
<p>
  Normal height
  <math>
    <mrow>
      <munderover>
        <mo>∑</mo>
        <mrow>
          <mi>n</mi>
          <mo>=</mo>
          <mn>1</mn>
```

```
        </mrow>
        <mrow>
          <mo>+</mo>
          <mn>∞</mn>
        </mrow>
      </munderover>
    </mrow>
  </math>
  and compact height
  <math class="compact">
    <mrow>
      <munderover>
        <mo>∑</mo>
        <mrow>
          <mi>n</mi>
          <mo>=</mo>
          <mn>1</mn>
        </mrow>
        <mrow>
          <mo>+</mo>
          <mn>∞</mn>
        </mrow>
      </munderover>
    </mrow>
  </math>
  equations.
</p>
```

Result

See also

- `math-depth`
- `font-size`

max-block-size CSS property

The **`max-block-size`** CSS property specifies the maximum size of an element in the direction opposite that of the writing direction as specified by `writing-mode`. That is, if the writing direction is horizontal, then `max-block-size` is equivalent to `max-height`; if the writing direction is vertical, `max-block-size` is the same as `max-width`.

The other dimension's maximum length is specified using the `max-inline-size` property.

This is useful because the `max-width` is always used for horizontal sizes and `max-height` is always used for vertical sizes, and if you need to set lengths based on the size of your text content, you need to be able to do so with the writing direction in mind.

Any time you would normally use `max-height` or `max-width`, you should instead use `max-block-size` to set the maximum "height" of the content (even though this may not be a vertical value) and `max-inline-size` to set the maximum "width" of the content (although this may instead be vertical rather than horizontal). See `writing-mode` examples, which show the different writing modes in action.

Try it

```
max-block-size: 150px;
writing-mode: horizontal-tb;
```

```
max-block-size: 150px;
writing-mode: vertical-rl;
```

```
max-block-size: 20px;
writing-mode: horizontal-tb;
```

```
max-block-size: 75%;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the maximum block size. <br />This will
    limit the size in the block dimension, potentially causing an overflow.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> values */
max-block-size: 300px;
max-block-size: 25em;
max-block-size: anchor-size(--my-anchor self-inline, 250px);
max-block-size: calc(anchor-size(width) / 2);

/* <percentage> values */
max-block-size: 75%;

/* Keyword values */
max-block-size: none;
max-block-size: max-content;
max-block-size: min-content;
max-block-size: fit-content;
max-block-size: fit-content(20em);

/* Global values */
max-block-size: inherit;
max-block-size: initial;
max-block-size: revert;
```

```
max-block-size: revert-layer;
max-block-size: unset;
```

Values

The `max-block-size` property's value can be any value that's legal for the `max-width` and `max-height` properties:

`<length>`

Defines the `max-block-size` as an absolute value.

`<percentage>`

Defines the `max-block-size` as a percentage of the containing block's size in block axis.

`none`

No limit on the size of the box.

`max-content`

The intrinsic preferred `max-block-size`.

`min-content`

The intrinsic minimum `max-block-size`.

`fit-content`

Use the available space, but not more than max-content, i.e., `min(max-content, max(min-content, stretch))`.

`fit-content(<length-percentage>)`

Uses the `fit-content` formula with the available space replaced by the specified argument, i.e., `min(max-content, max(min-content, argument))`.

How writing-mode affects directionality

The values of `writing-mode` affect the mapping of `max-block-size` to `max-width` or `max-height` as follows:

Values of `writing-mode`	**`max-block-size` is equivalent to**
`horizontal-tb`, `lr`, `lr-tb`, `rl`, `rb`, `rb-rl`	`max-height`
`vertical-rl`, `vertical-lr`, `sideways-rl`, `sideways-lr`, `tb`, `tb-rl`	`max-width`

Note: The `writing-mode` values `sideways-lr` and `sideways-rl` were removed from the CSS Writing Modes Level 3 specification late in its design process. They may be restored in Level 4.

Note: The writing modes `lr`, `lr-tb`, `rl`, `rb`, and `rb-tl` are no longer allowed in HTML contexts; they may only be used in SVG 1.x contexts.

Formal definition

Initial value	none
Applies to	same as `width` and `height`
Inherited	no
Percentages	block-size of containing block
Computed value	same as `max-width` and `max-height`
Animation type	a length, percentage or calc();

Formal syntax

```
max-block-size =
  none             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>               |
  stretch                                 |
  contain                                 |
  min-content                             |
  max-content                             |
  fit-content                             |
  stretch( <length-percentage [0,∞]> )    |
  fit-content( <length-percentage [0,∞]> )  |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
```

```
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting max-block-size with horizontal and vertical text

In this example, the same text (the opening sentences from Herman Melville's novel *Moby-Dick*) is presented in both the `horizontal-tb` and `vertical-rl` writing modes.

Everything else about the two boxes is identical, including the values used for `max-block-size`.

HTML

The HTML establishes the two `<div>` blocks that will be presented with their `writing-mode` set using the classes `horizontal` or `vertical`. Both boxes share the `standard-box` class, which establishes coloring, padding, and their respective values of `max-block-size`.

```
<p>Writing mode <code>horizontal-tb</code> (the default):</p>
<div class="standard-box horizontal">
  Call me Ishmael. Some years ago—never mind how long precisely—having little or
  no money in my purse, and nothing particular to interest me on shore, I
  thought I would sail about a little and see the watery part of the world. It
  is a way I have of driving off the spleen and regulating the circulation.
</div>

<p>Writing mode <code>vertical-rl</code>:</p>
<div class="standard-box vertical">
  Call me Ishmael. Some years ago—never mind how long precisely—having little or
  no money in my purse, and nothing particular to interest me on shore, I
  thought I would sail about a little and see the watery part of the world. It
  is a way I have of driving off the spleen and regulating the circulation.
</div>
```

CSS

The CSS defines three classes. The first, `standard-box`, is applied to both boxes, as seen above. It provides standard styling including the minimum and maximum block sizes, font size, and so forth.

After that come the classes `horizontal` and `vertical`, which add the `writing-mode` property to the box, with the value set to `horizontal-tb` or `vertical-rl` depending on which class is used.

```
.standard-box {
  padding: 4px;
  background-color: #abcdef;
  color: black;
  font:
    16px "Open Sans",
    "Helvetica",
    "Arial",
    sans-serif;
  max-block-size: 160px;
  min-block-size: 100px;
}

.horizontal {
  writing-mode: horizontal-tb;
}

.vertical {
  writing-mode: vertical-rl;
}
```

Result

See also

- The mapped physical properties: `max-width` and `max-height`
- Setting the other direction's maximum size: `max-inline-size`
- `writing-mode`

max-height CSS property

The **`max-height`** CSS property sets the maximum height of an element. It prevents the used value of the `height` property from becoming larger than the value specified for `max-height`.

`max-height` overrides `height`, but `min-height` overrides `max-height`.

Try it

```
max-height: 150px;
```

```
max-height: 7em;
```

```
max-height: 75%;
```

```
max-height: 10px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the maximum height. <br />This will limit
    how tall the box can be, potentially causing an overflow.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> value */
max-height: 3.5em;
max-height: anchor-size(height);
max-height: calc(anchor-size(--my-anchor self-block, 250px) + 2em);

/* <percentage> value */
max-height: 75%;

/* Keyword values */
max-height: none;
max-height: max-content;
max-height: min-content;
max-height: fit-content;
max-height: fit-content(20em);
max-height: stretch;

/* Global values */
max-height: inherit;
max-height: initial;
max-height: revert;
max-height: revert-layer;
max-height: unset;
```

Values

`<length>`

Defines the `max-height` as an absolute value.

`<percentage>`

Defines the `max-height` as a percentage of the containing block's height.

`none`

No limit on the size of the box.

`max-content`

The intrinsic preferred `max-height`.

`min-content`

The intrinsic minimum `max-height`.

`fit-content`

Use the available space, but not more than max-content, i.e., `min(max-content, max(min-content, stretch))`.

`fit-content(<length-percentage>)`

Uses the `fit-content` formula with the available space replaced by the specified argument, i.e., `min(max-content, max(min-content, argument))`.

`stretch`

Limits the maximum height of the element's margin box to the height of its containing block. It attempts to make the margin box fill the available space in the containing block, so in a way behaving similar to `100%` but applying the resulting size to the margin box rather than the box determined by box-sizing.

> **Note:** To check aliases used by browsers for the `stretch` value and its implementation status, see the Browser compatibility section.

Accessibility

Ensure that elements set with a `max-height` are not truncated and/or do not obscure other content when the page is zoomed to increase text size.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4 | W3C Understanding WCAG 2.0

Formal definition

Initial value	none
Applies to	all elements but non-replaced inline elements, table columns, and column groups
Inherited	no

Percentages	The percentage is calculated with respect to the height of the generated box's containing block. If the height of the containing block is not specified explicitly (i.e., it depends on content height), and this element is not absolutely positioned, the percentage value is treated as none.
Computed value	the percentage as specified or the absolute length or none
Animation type	a length, percentage or calc();

Formal syntax

```
max-height =
  none              |
  <box-size>        |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                 |
  stretch                                   |
  contain                                   |
  min-content                               |
  max-content                               |
  fit-content                               |
  stretch( <length-percentage [0,∞]> )      |
  fit-content( <length-percentage [0,∞]> )  |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting max-height using percentage and keyword values

```
table {
  max-height: 75%;
}

form {
  max-height: none;
}
```

See also

- min-height
- height
- max-inline-size
- max-block-size
- box-sizing
- Introduction to the CSS box model guide
- CSS box model module

max-inline-size CSS property

The **max-inline-size** CSS property defines the horizontal or vertical maximum size of an element's block, depending on its writing mode. It corresponds to either the max-width or the max-height property, depending on the value of writing-mode.

If the writing mode is vertically oriented, the value of `max-inline-size` relates to the maximal height of the element; otherwise, it relates to the maximal width of the element. A related property is `max-block-size`, which defines the other dimension of the element.

Try it

```
max-inline-size: 150px;
writing-mode: horizontal-tb;
```

```
max-inline-size: 150px;
writing-mode: vertical-rl;
```

```
max-inline-size: 20px;
writing-mode: horizontal-tb;
```

```
max-inline-size: 75%;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the max-inline-size.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> values */
max-inline-size: 300px;
max-inline-size: 25em;
max-inline-size: anchor-size(width);
max-inline-size: anchor-size(--my-anchor self-block, 200px);

/* <percentage> values */
max-inline-size: 75%;

/* Keyword values */
max-inline-size: none;
max-inline-size: max-content;
max-inline-size: min-content;
max-inline-size: fit-content;
max-inline-size: fit-content(20em);

/* Global values */
max-inline-size: inherit;
max-inline-size: initial;
max-inline-size: revert;
max-inline-size: revert-layer;
max-inline-size: unset;
```

Values

The `max-inline-size` property takes the same values as the `max-width` and `max-height` properties.

Formal definition

Initial value	none
Applies to	same as `width` and `height`
Inherited	no
Percentages	inline-size of containing block
Computed value	same as `max-width` and `max-height`
Animation type	a length, percentage or calc();

Formal syntax

```
max-inline-size =
  none             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>               |
  stretch                                 |
  contain                                 |
  min-content                             |
  max-content                             |
  fit-content                             |
  stretch( <length-percentage [0,∞]> )    |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )
```

```
<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting maximum inline size in pixels

HTML

```
<p class="exampleText">Example text</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  block-size: 100%;
  max-inline-size: 200px;
}
```

Result

See also

- The mapped physical properties: `max-width` and `max-height`
- `writing-mode`

max-width CSS property

The **`max-width`** CSS property sets the maximum width of an element. It prevents the used value of the `width` property from becoming larger than the value specified by `max-width`.

`max-width` overrides `width`, but `min-width` overrides `max-width`.

Try it

```
max-width: 150px;
```

```
max-width: 20em;
```

```
max-width: 75%;
```

```
max-width: 20ch;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    Change the maximum width.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> value */
max-width: 3.5em;
max-width: anchor-size(--my-anchor inline, 245px);
```

```
max-width: calc(anchor-size(width) + 4em);

/* <percentage> value */
max-width: 75%;

/* Keyword values */
max-width: none;
max-width: max-content;
max-width: min-content;
max-width: fit-content;
max-width: fit-content(20em);
max-width: stretch;

/* Global values */
max-width: inherit;
max-width: initial;
max-width: revert;
max-width: revert-layer;
max-width: unset;
```

Values

`<length>`

Defines the `max-width` as an absolute value.

`<percentage>`

Defines the `max-width` as a percentage of the containing block's width.

`none`

No limit on the size of the box.

`max-content`

The intrinsic preferred `max-width`.

`min-content`

The intrinsic minimum `max-width`.

`fit-content`

Use the available space, but not more than max-content, i.e., `min(max-content, max(min-content, stretch))`.

`fit-content(<length-percentage>)`

Uses the `fit-content` formula with the available space replaced by the specified argument, i.e., `min(max-content, max(min-content, argument))`.

`stretch`

Limits the maximum width of the element's margin box to the width of its containing block. It attempts to make the margin box fill the available space in the containing block, so in a way behaving similar to `100%` but applying the resulting size to the margin box rather than the box determined by box-sizing.

> **Note:** To check aliases used by browsers for the `stretch` value and its implementation status, see the Browser compatibility section.

Accessibility

Ensure that elements set with a `max-width` are not truncated and/or do not obscure other content when the page is zoomed to increase text size.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4 | W3C Understanding WCAG 2.0

Formal definition

Initial value	none
Applies to	all elements but non-replaced inline elements, table rows, and row groups
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length or `none`
Animation type	a length, percentage or calc();

Formal syntax

```
max-width =
  none             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>
```

```
<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting max width in pixels

In this example, the "child" will be either 150 pixels wide or the width of the "parent," whichever is smaller.

HTML

```
<div id="parent">
  <div id="child">
    Fusce pulvinar vestibulum eros, sed luctus ex lobortis quis.
  </div>
</div>
```

CSS

```
#parent {
  background: lightblue;
  width: 300px;
}

#child {
  background: gold;
  width: 100%;
  max-width: 150px;
}
```

Result

See also

- `min-width`
- `width`
- `max-inline-size`
- `max-block-size`
- `box-sizing`
- Introduction to the CSS box model guide
- CSS box model module

min-block-size CSS property

The **`min-block-size`** CSS property defines the minimum horizontal or vertical size of an element's block, depending on its writing mode. It corresponds to either the `min-width` or the `min-height` property, depending on the value of `writing-mode`.

If the writing mode is vertically oriented, the value of `min-block-size` relates to the minimum width of the element; otherwise, it relates to the minimum height of the element. A related property is `min-inline-size`, which defines the other dimension of the element.

Try it

```
min-block-size: 150px;
writing-mode: horizontal-tb;
```

```
min-block-size: 150px;
writing-mode: vertical-rl;
```

```
min-block-size: 20px;
writing-mode: horizontal-tb;
```

```
min-block-size: 15em;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the minimum block size. <br />If there is
    more content than the minimum the box will grow in the block dimension as
    needed by the content.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> values */
min-block-size: 100px;
min-block-size: 5em;
min-block-size: anchor-size(self-inline);

/* <percentage> values */
min-block-size: 10%;

/* Keyword values */
min-block-size: max-content;
min-block-size: min-content;
min-block-size: fit-content;
min-block-size: fit-content(20em);

/* Global values */
min-block-size: inherit;
min-block-size: initial;
min-block-size: revert;
min-block-size: revert-layer;
min-block-size: unset;
```

Values

The `min-block-size` property takes the same values as the `min-width` and `min-height` properties.

Formal definition

Initial value	`0`
Applies to	same as `width` and `height`
Inherited	no
Percentages	block-size of containing block
Computed value	same as `min-width` and `min-height`
Animation type	a length, percentage or calc();

Formal syntax

```
min-block-size =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any
```

```
<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting minimum block size for vertical text

HTML

```
<p class="exampleText">Example text</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  min-block-size: 200px;
}
```

Result

See also

- The mapped physical properties: `min-width` and `min-height`
- `writing-mode`

min-height CSS property

The **`min-height`** CSS property sets the minimum height of an element. It prevents the used value of the `height` property from becoming smaller than the value specified for `min-height`.

The element's height is set to the value of `min-height` whenever `min-height` is larger than `max-height` or `height`.

Try it

```
min-height: 150px;
```

```
min-height: 7em;
```

```
min-height: 75%;
```

```
min-height: 10px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the minimum height. <br />If there is
    more content than the minimum the box will grow to the height needed by the
    content.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> value */
min-height: 3.5em;
min-height: anchor-size(height);
min-height: anchor-size(--my-anchor block, 200px);

/* <percentage> value */
min-height: 10%;

/* Keyword values */
min-height: max-content;
min-height: min-content;
min-height: fit-content;
min-height: fit-content(20em);
min-height: stretch;

/* Global values */
min-height: inherit;
min-height: initial;
min-height: revert;
```

```
min-height: revert-layer;
min-height: unset;
```

Values

`<length>`

Defines the `min-height` as an absolute value.

`<percentage>`

Defines the `min-height` as a percentage of the containing block's height.

`auto`

The browser will calculate and select a `min-height` for the specified element.

`max-content`

The intrinsic preferred `min-height`.

`min-content`

The intrinsic minimum `min-height`.

`fit-content`

Use the available space, but not more than max-content, i.e., `min(max-content, max(min-content, stretch))`.

`fit-content(<length-percentage>)`

Uses the `fit-content` formula with the available space replaced by the specified argument, i.e., `min(max-content, max(min-content, argument))`.

`stretch`

Limits the minimum height of the element's margin box to the height of its containing block. It attempts to make the margin box fill the available space in the containing block, so in a way behaving similar to `100%` but applying the resulting size to the margin box rather than the box determined by box-sizing.

> **Note:** To check aliases used by browsers for the `stretch` value and its implementation status, see the Browser compatibility section.

Formal definition

Initial value	`auto`
Applies to	all elements but non-replaced inline elements, table columns, and column groups
Inherited	no
Percentages	The percentage is calculated with respect to the height of the generated box's containing block. If the height of the containing block is not specified explicitly (i.e., it depends on content height), and this element is not absolutely positioned, the percentage value is treated as `0`.
Computed value	the percentage as specified or the absolute length
Animation type	a length, percentage or calc();

Formal syntax

```
min-height =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*
```

```
<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>       |
  <dimension>    |
  <percentage>   |
  <calc-keyword> |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting min-height

```
table {
  min-height: 75%;
}

form {
  min-height: 0;
}
```

See also

- `max-height`
- `height`
- `min-inline-size`
- `min-block-size`
- `box-sizing`
- Introduction to the CSS box model guide
- CSS box model module

min-inline-size CSS property

The **`min-inline-size`** CSS property defines the horizontal or vertical minimal size of an element's block, depending on its writing mode. It corresponds to either the `min-width` or the `min-height` property, depending on the value of `writing-mode`.

Try it

```
min-inline-size: 200px;
writing-mode: horizontal-tb;
```

```
min-inline-size: 200px;
writing-mode: vertical-rl;
```

```
min-inline-size: 20px;
writing-mode: horizontal-tb;
```

```
min-inline-size: 75%;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">Change min-inline-size</div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> values */
min-inline-size: 100px;
min-inline-size: 5em;
min-inline-size: anchor-size(width);

/* <percentage> values */
min-inline-size: 10%;

/* Keyword values */
min-inline-size: max-content;
min-inline-size: min-content;
min-inline-size: fit-content;
min-inline-size: fit-content(20em);

/* Global values */
min-inline-size: inherit;
min-inline-size: initial;
min-inline-size: revert;
min-inline-size: revert-layer;
min-inline-size: unset;
```

If the writing mode is vertically oriented, the value of `min-inline-size` relates to the minimum height of the element; otherwise, it relates to the minimum width of the element. A related property is `min-block-size`, which defines the other dimension of the element.

Values

The `min-inline-size` property takes the same values as the `min-width` and `min-height` properties.

Formal definition

Initial value	`0`
Applies to	same as `width` and `height`
Inherited	no
Percentages	inline-size of containing block
Computed value	same as `min-width` and `min-height`
Animation type	a length, percentage or calc();

Formal syntax

```
min-inline-size =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>     |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting minimum inline size for vertical text

HTML

```
<p class="exampleText">Example text</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  block-size: 5%;
  min-inline-size: 200px;
}
```

Result

See also

- The mapped physical properties: `min-width` and `min-height`
- `writing-mode`

min-width CSS property

The **`min-width`** CSS property sets the minimum width of an element. It prevents the used value of the `width` property from becoming smaller than the value specified for `min-width`.

The element's width is set to the value of `min-width` whenever `min-width` is larger than `max-width` or `width`.

Try it

```
min-width: 150px;
```

```
min-width: 20em;
```

```
min-width: 75%;
```

```
min-width: 40ch;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    Change the minimum width.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: white;
}
```

Syntax

```
/* <length> value */
min-width: 3.5em;
min-width: anchor-size(width);
min-width: anchor-size(--my-anchor self-inline, 200%);

/* <percentage> value */
min-width: 10%;

/* Keyword values */
min-width: max-content;
min-width: min-content;
min-width: fit-content;
min-width: fit-content(20em);
min-width: stretch;

/* Global values */
min-width: inherit;
min-width: initial;
min-width: revert;
min-width: revert-layer;
min-width: unset;
```

Values

`<length>`

Defines the `min-width` as an absolute value.

`<percentage>`

Defines the `min-width` as a percentage of the containing block's width.

`auto`

The default value. The source of the automatic value for the specified element depends on its display value. For block boxes, inline boxes, inline blocks, and all table layout boxes `auto` resolves to `0`.

For flex items and grid items, the minimum width value is either the specified suggested size, such as the value of the `width` property, the transferred size, calculated if the element has an `aspect-ratio` set and the height is a definite size, otherwise, the `min-content` size is used. If the flex or grid item is a scroll container, or if a grid item spans more than one flexible column track, the automatic minimum size is `0`.

`max-content`

The intrinsic preferred `min-width`.

`min-content`

The intrinsic minimum `min-width`.

`fit-content`

Use the available space, but not more than `max-content`, i.e., `min(max-content, max(min-content, stretch))`.

`fit-content(<length-percentage>)`

Uses the `fit-content` formula with the available space replaced by the specified argument, i.e., `min(max-content, max(min-content, argument))`.

`stretch`

Limits the minimum width of the element's margin box to the width of its containing block. It attempts to make the margin box fill the available space in the containing block, so in a way behaving similar to `100%` but applying the resulting size to the margin box rather than the box determined by box-sizing.

> **Note:** To check aliases used by browsers for the `stretch` value and its implementation status, see the Browser compatibility section.

Formal definition

Initial value	`auto`
Applies to	all elements but non-replaced inline elements, table rows, and row groups
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length, percentage or calc();

Formal syntax

```
min-width =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>     |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting minimum element width

```
table {
  min-width: 75%;
}

form {
  min-width: 0;
}
```

See also

- `max-width`
- `width`
- `min-inline-size`

- `min-block-size`
- `box-sizing`
- Introduction to the CSS box model guide
- CSS box model module

mix-blend-mode CSS property

The **`mix-blend-mode`** CSS property sets how an element's content should blend with its *backdrop* — the content rendered behind the element within the same stacking context.

> **Note:** This property blends the element with content *behind* it in the stacking context. To blend an element's own background layers with each other, use `background-blend-mode` instead.

Try it

```
mix-blend-mode: normal;
```

```
mix-blend-mode: multiply;
```

```
mix-blend-mode: hard-light;
```

```
mix-blend-mode: difference;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <img
      id="example-element"
      src="/shared-assets/images/examples/firefox-logo.svg"
      width="200" />
  </div>
</section>
```

```
.example-container {
  background-color: sandybrown;
}
```

Syntax

```
/* Keyword values */
mix-blend-mode: normal;
mix-blend-mode: multiply;
mix-blend-mode: screen;
mix-blend-mode: overlay;
mix-blend-mode: darken;
mix-blend-mode: lighten;
mix-blend-mode: color-dodge;
mix-blend-mode: color-burn;
mix-blend-mode: hard-light;
mix-blend-mode: soft-light;
mix-blend-mode: difference;
mix-blend-mode: exclusion;
mix-blend-mode: hue;
mix-blend-mode: saturation;
mix-blend-mode: color;
mix-blend-mode: luminosity;
mix-blend-mode: plus-darker;
mix-blend-mode: plus-lighter;

/* Global values */
mix-blend-mode: inherit;
mix-blend-mode: initial;
mix-blend-mode: revert;
mix-blend-mode: revert-layer;
mix-blend-mode: unset;
```

Values

`<blend-mode>`

: The blending mode that should be applied.

`plus-darker`

: Blending using the *plus-darker* compositing operator.

`plus-lighter`

: Blending using the *plus-lighter* compositing operator. Useful for cross-fade effects (prevents unwanted blinking when two overlaying elements animate their opacity in opposite directions).

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable
Creates stacking context	yes

Formal syntax

```
mix-blend-mode =
  <blend-mode>  |
  plus-lighter

<blend-mode> =
```

```
  normal      |
  darken      |
  multiply    |
  color-burn  |
  lighten     |
  screen      |
  color-dodge |
  overlay     |
  soft-light  |
  hard-light  |
  difference  |
  exclusion   |
  hue         |
  saturation  |
  color       |
  luminosity
```

This syntax reflects the latest standard as per Compositing and Blending Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Effect of different mix-blend-mode values

```
<div class="grid">
  <div class="col">
    <div class="note">
      Blending in isolation (no blending with the background)
    </div>
    <div class="row isolate">
      <div class="cell">
        normal
        <div class="container normal">
          <div class="group">
            <div class="item firefox"></div>
            <svg class="item" viewBox="0 0 150 150">
              <defs>
                <linearGradient id="red">
                  <stop offset="0" stop-color="hsl(0 100% 50%)" />
                  <stop offset="100%" stop-color="hsl(0 0% 100%)" />
                </linearGradient>
                <linearGradient id="green">
                  <stop offset="0" stop-color="hsl(120 100% 50%)" />
                  <stop offset="100%" stop-color="hsl(120 0% 100%)" />
                </linearGradient>
                <linearGradient id="blue">
                  <stop offset="0" stop-color="hsl(240 100% 50%)" />
                  <stop offset="100%" stop-color="hsl(240 0% 100%)" />
                </linearGradient>
              </defs>
              <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
            </svg>
          </div>
        </div>
      </div>
      <div class="cell">
        multiply
        <div class="container multiply">
          <div class="group">
            <div class="item firefox"></div>
            <svg class="item" viewBox="0 0 150 150">
              <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
            </svg>
          </div>
        </div>
      </div>
      <div class="cell">
        darken
        <div class="container darken">
          <div class="group">
            <div class="item firefox"></div>
            <svg class="item" viewBox="0 0 150 150">
              <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
            </svg>
          </div>
        </div>
      </div>
      <div class="cell">
        screen
        <div class="container screen">
          <div class="group">
            <div class="item firefox"></div>
            <svg class="item" viewBox="0 0 150 150">
              <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
            </svg>
          </div>
        </div>
      </div>
      <div class="cell">
        lighten
        <div class="container lighten">
          <div class="group">
            <div class="item firefox"></div>
            <svg class="item" viewBox="0 0 150 150">
              <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
              <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
```

```
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    overlay
    <div class="container overlay">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    color-dodge
    <div class="container color-dodge">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    color-burn
    <div class="container color-burn">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    hard-light
    <div class="container hard-light">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    soft-light
    <div class="container soft-light">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    difference
    <div class="container difference">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    exclusion
    <div class="container exclusion">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    hue
    <div class="container hue">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
```

```
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      saturation
      <div class="container saturation">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      color
      <div class="container color">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      luminosity
      <div class="container luminosity">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      plus-darker
      <div class="container plus-darker">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      plus-lighter
      <div class="container plus-lighter">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
  </div>

  <div class="note">Blending globally (blend with the background)</div>
  <div class="row">
    <div class="cell">
      normal
      <div class="container normal">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      multiply
      <div class="container multiply">
        <div class="group">
          <div class="item firefox"></div>
          <svg class="item" viewBox="0 0 150 150">
            <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
            <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
          </svg>
        </div>
      </div>
    </div>
    <div class="cell">
      darken
      <div class="container darken">
```

```
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    screen
    <div class="container screen">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    lighten
    <div class="container lighten">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    overlay
    <div class="container overlay">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    color-dodge
    <div class="container color-dodge">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    color-burn
    <div class="container color-burn">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    hard-light
    <div class="container hard-light">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    soft-light
    <div class="container soft-light">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    difference
```

```
    <div class="container difference">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    exclusion
    <div class="container exclusion">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    hue
    <div class="container hue">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    saturation
    <div class="container saturation">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    color
    <div class="container color">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    luminosity
    <div class="container luminosity">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    plus-darker
    <div class="container plus-darker">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
  <div class="cell">
    plus-lighter
    <div class="container plus-lighter">
      <div class="group">
        <div class="item firefox"></div>
        <svg class="item" viewBox="0 0 150 150">
          <ellipse class="item R" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item G" cx="75" cy="75" rx="25" ry="70"></ellipse>
          <ellipse class="item B" cx="75" cy="75" rx="25" ry="70"></ellipse>
        </svg>
      </div>
    </div>
  </div>
</div>
```

```
  </div>
</div>
```

```
html,
body {
  height: 100%;
  box-sizing: border-box;
  background: #eeeeee;
}

.grid {
  width: 100%;
  display: flex;
  font: 1em monospace;
}

.row {
  display: flex;
  flex: 1 auto;
  flex-direction: row;
  flex-wrap: wrap;
  height: auto;
}

.col {
  display: flex;
  flex: 1 auto;
  flex-direction: column;
  height: auto;
}

.cell {
  margin: 0.5em;
  padding: 0.5em;
  background-color: white;
  overflow: hidden;
  text-align: center;
}

.note {
  background: #fff3d4;
  padding: 1em;
  margin: 0.5em 0.5em 0;
  font: 0.8em sans-serif;
  text-align: left;
  white-space: nowrap;
}

.note + .row .cell {
  margin-top: 0;
}

.container {
  position: relative;
  background:
    linear-gradient(to right, black 0%, transparent 50%, white 100%),
    linear-gradient(to bottom, yellow 0%, magenta 50%, cyan 100%);
  width: 150px;
  height: 150px;
  margin: 0 auto;
}

.R {
  transform-origin: center;
  transform: rotate(-30deg);
  fill: url("#red");
}

.G {
  transform-origin: center;
  transform: rotate(90deg);
  fill: url("#green");
}

.B {
  transform-origin: center;
  transform: rotate(210deg);
  fill: url("#blue");
}

.isolate .group {
  isolation: isolate;
}

.normal .item {
  mix-blend-mode: normal;
}
.multiply .item {
  mix-blend-mode: multiply;
}
.screen .item {
  mix-blend-mode: screen;
}
.overlay .item {
  mix-blend-mode: overlay;
}
.darken .item {
  mix-blend-mode: darken;
}
.lighten .item {
  mix-blend-mode: lighten;
}
.color-dodge .item {
```

```
  mix-blend-mode: color-dodge;
}
.color-burn .item {
  mix-blend-mode: color-burn;
}
.hard-light .item {
  mix-blend-mode: hard-light;
}
.soft-light .item {
  mix-blend-mode: soft-light;
}
.difference .item {
  mix-blend-mode: difference;
}
.exclusion .item {
  mix-blend-mode: exclusion;
}
.hue .item {
  mix-blend-mode: hue;
}
.saturation .item {
  mix-blend-mode: saturation;
}
.color .item {
  mix-blend-mode: color;
}
.luminosity .item {
  mix-blend-mode: luminosity;
}
.plus-darker .item {
  mix-blend-mode: plus-darker;
}
.plus-lighter .item {
  mix-blend-mode: plus-lighter;
}
```

Using mix-blend-mode with HTML

HTML

```
<div class="isolate">
  <div class="circle circle-1"></div>
  <div class="circle circle-2"></div>
  <div class="circle circle-3"></div>
</div>
```

CSS

```
.circle {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  mix-blend-mode: screen;
  position: absolute;
}

.circle-1 {
  background: red;
}

.circle-2 {
  background: lightgreen;
  left: 40px;
}

.circle-3 {
  background: blue;
  left: 20px;
  top: 40px;
}

.isolate {
  isolation: isolate; /* Without isolation, the background color will be taken into account */
  position: relative;
}
```

Result

Using mix-blend-mode with SVG

SVG

```
<svg>
  <g class="isolate">
    <circle cx="40" cy="40" r="40" fill="red" />
    <circle cx="80" cy="40" r="40" fill="lightgreen" />
    <circle cx="60" cy="80" r="40" fill="blue" />
  </g>
</svg>
```

CSS

```
circle {
  mix-blend-mode: screen;
}
.isolate {
  isolation: isolate;
} /* Without isolation, the background color will be taken into account */
```

Result

Using mix-blend-mode with text

This example uses `mix-blend-mode` to blend text color with the background color of its parent element.

HTML

```
<div class="container">
  <p>Mostly Harmless</p>
  <p class="multiply">Mostly Harmless</p>
  <p class="screen">Mostly Harmless</p>
  <p class="hard-light">Mostly Harmless</p>
</div>
```

CSS

```
@import "https://fonts.googleapis.com/css2?family=Rubik+Moonrocks&display=swap";

.container {
  background-color: blue;
}

p {
  font:
    4rem "Rubik Moonrocks",
    cursive;
  font-weight: bold;
  color: orange;
  padding: 0.5rem;
  margin: 0;
}

.multiply {
  mix-blend-mode: multiply;
}

.screen {
  mix-blend-mode: screen;
}

.hard-light {
  mix-blend-mode: hard-light;
}
```

Result

See also

- `<blend-mode>`
- `background-blend-mode`

object-fit CSS property

The **`object-fit`** CSS property sets how the content of a replaced element, such as an `<img>` or `<video>`, should be resized to fit its container.

> **Note:** The `object-fit` property has no effect on `<iframe>`, `<embed>`, and `<fencedframe>` elements.

You can alter the alignment of the replaced element's content object within the element's box using the `object-position` property.

Try it

```
object-fit: fill;
```

```
object-fit: contain;
```

```
object-fit: cover;
```

```
object-fit: none;
```

```
object-fit: scale-down;
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/plumeria-146x200.jpg" />
</section>
```

```
#example-element {
  height: 100%;
  width: 100%;
  border: 2px dotted #888888;
}
```

Syntax

```
object-fit: contain;
object-fit: cover;
object-fit: fill;
object-fit: none;
object-fit: scale-down;

/* Global values */
object-fit: inherit;
object-fit: initial;
object-fit: revert;
```

```
object-fit: revert-layer;
object-fit: unset;
```

Values

This property is specified as one of the following keyword values:

`contain`

> The replaced content is scaled to maintain its aspect ratio while fitting within the element's content box. The entire object is made to fill the box, while preserving its aspect ratio, so the object will be "letterboxed" or "pillarboxed" if its aspect ratio does not match the aspect ratio of the box.

`cover`

> The replaced content is sized to maintain its aspect ratio while filling the element's entire content box. If the object's aspect ratio does not match the aspect ratio of its box, then the object will be clipped to fit.

`fill`

> The replaced content is sized to fill the element's content box. This is the `initial` value. The entire object will completely fill the box. If the object's aspect ratio does not match the aspect ratio of its box, then the object will be stretched to fit.

`none`

> The replaced content is not resized.

`scale-down`

> The content is sized as if `none` or `contain` were specified, whichever would result in a smaller concrete object size.

Formal definition

Initial value	`fill`
Applies to	replaced elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
object-fit =
  fill                          |
  none                          |
  [ contain | cover ] || scale-down
```

This syntax reflects the latest standard as per CSS Images Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting object-fit for an image

HTML

```
<section>
  <h2>object-fit: fill</h2>
  <img class="fill" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <img class="fill narrow" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <h2>object-fit: contain</h2>
  <img class="contain" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <img class="contain narrow" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <h2>object-fit: cover</h2>
  <img class="cover" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <img class="cover narrow" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <h2>object-fit: none</h2>
  <img class="none" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <img class="none narrow" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <h2>object-fit: scale-down</h2>
  <img class="scale-down" src="mdn_logo_only_color.png" alt="MDN Logo" />

  <img class="scale-down narrow" src="mdn_logo_only_color.png" alt="MDN Logo" />
</section>
```

CSS

```
h2 {
  font-family: "Courier New", monospace;
  font-size: 1em;
  margin: 1em 0 0.3em;
}

img {
  width: 150px;
  height: 100px;
  border: 1px solid black;
  margin: 10px 0;
}

.narrow {
  width: 100px;
  height: 150px;
}
```

```
.fill {
  object-fit: fill;
}

.contain {
  object-fit: contain;
}

.cover {
  object-fit: cover;
}

.none {
  object-fit: none;
}

.scale-down {
  object-fit: scale-down;
}
```

Result

See also

- Other image-related CSS properties: `object-position`, `image-orientation`, `image-rendering`, `image-resolution`.
- `background-size`
- Understanding aspect ratios

object-position CSS property

The **`object-position`** CSS property specifies the alignment of the selected replaced element's contents within the element's box. Areas of the box which aren't covered by the replaced element's object will show the element's background.

You can adjust how the replaced element's object's intrinsic size (that is, its natural size) is adjusted to fit within the element's box using the `object-fit` property.

Try it

```
object-position: 50% 50%;
```

```
object-position: right top;
```

```
object-position: left bottom;
```

```
object-position: 250px 125px;
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/moon.jpg" />
</section>
```

```
#example-element {
  height: 250px;
  width: 250px;
  object-fit: none;
  border: 1px solid red;
}
```

Syntax

```
/* Keyword values */
object-position: top;
object-position: bottom;
object-position: left;
object-position: right;
object-position: center;

/* <percentage> values */
object-position: 25% 75%;

/* <length> values */
object-position: 0 0;
object-position: 1cm 2cm;
object-position: 10ch 8em;

/* Edge offsets values */
object-position: bottom 10px right 20px;
object-position: right 3em bottom 10px;
object-position: top 0 right 10px;

/* Global values */
object-position: inherit;
object-position: initial;
object-position: revert;
object-position: revert-layer;
object-position: unset;
```

Values

`<position>`

From one to four values that define the 2D position of the element. Relative or absolute offsets can be used.

> **Note:** The position can be set so that the replaced element is drawn outside its box.

Formal definition

Initial value	`50% 50%`
Applies to	replaced elements
Inherited	no
Percentages	refer to width and height of element itself
Computed value	as specified
Animation type	a repeatable list

Formal syntax

```
object-position =
  <position>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Images Module Level 3, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Positioning image content

HTML

Here we see HTML that includes two `<img>` elements, each displaying the MDN logo.

```
<img id="object-position-1" src="mdn.svg" alt="MDN Logo" />
<img id="object-position-2" src="mdn.svg" alt="MDN Logo" />
```

CSS

The CSS includes default styling for the `<img>` element itself, as well as separate styles for each of the two images.

```
img {
  width: 300px;
  height: 250px;
  border: 1px solid black;
  background-color: silver;
  margin-right: 1em;
  object-fit: none;
}

#object-position-1 {
  object-position: 10px;
}

#object-position-2 {
  object-position: 100% 10%;
}
```

The first image is positioned with its left edge inset 10 pixels from the left edge of the element's box. The second image is positioned with its right edge flush against the right edge of the element's box and is located 10% of the way down the height of the element's box.

Result

See also

- Other image-related CSS properties: `object-fit`, `image-orientation`, `image-rendering`, `image-resolution`.

object-view-box CSS property

> **Experimental: This is an experimental technology**

> Check the Browser compatibility table carefully before using this in production.

The **`object-view-box`** CSS property defines a rectangle as a viewable area (viewbox) within a replaced element, enabling the content of the replaced element to be zoomed or panned. It works similarly to the SVG `viewBox` attribute.

Try it

```
object-view-box: inset(0 0);
```

```
object-view-box: inset(20%);
```

```
object-view-box: xywh(95px 20px 60px 60px);
```

```
object-view-box: rect(110px 120px 200px 45px);
```

```
object-view-box: none;
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/plumeria-146x200.jpg"
    alt="A close-up of red flowers and buds on a branch." />
</section>
```

```
#example-element {
  height: 100%;
  width: 100%;
  border: 2px dotted #888888;
}

@supports not (object-view-box: none) {
  body::before {
    content: "Your browser does not support the 'object-view-box' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
  }
}
```

Syntax

```
/* Keyword value */
object-view-box: none;

/* <basic-shape-rect> functions */
object-view-box: inset(20%);
object-view-box: inset(20% 30%);
object-view-box: inset(10px 0 25px 33px);
object-view-box: xywh(95px 20px 60px 60px);
object-view-box: rect(10px 30px 30px 10px);

/* Global values */
object-view-box: inherit;
object-view-box: initial;
object-view-box: revert;
object-view-box: revert-layer;
object-view-box: unset;
```

Values

`none`

The element does not have a view box. This is the default.

`<basic-shape-rect>`

An `inset()`, `xywh()`, or `rect()` function specifying a view box for an element with natural dimensions (replaced elements). Resolves to `none` otherwise.

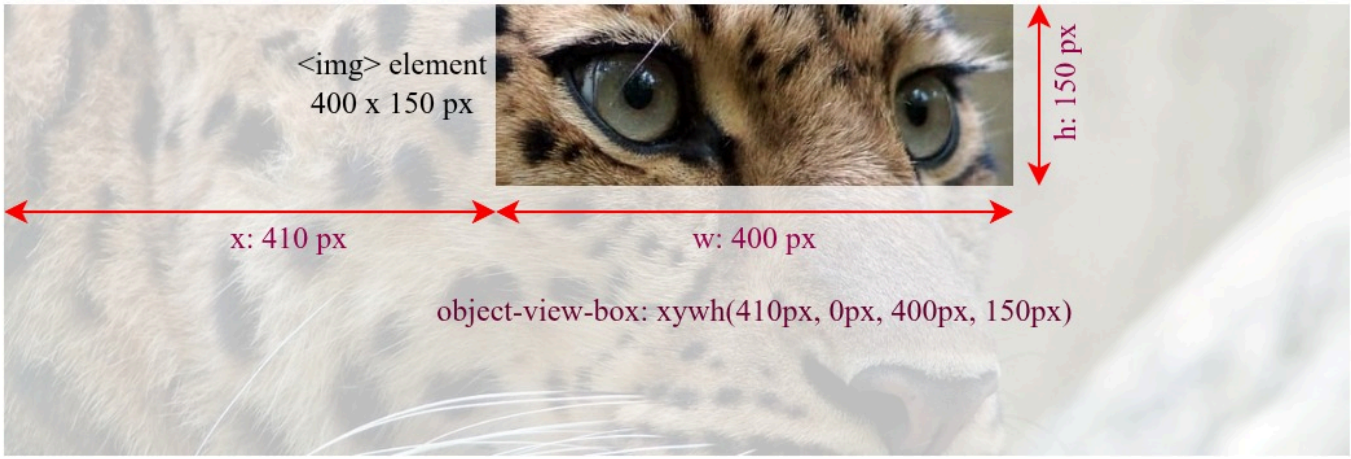
Description

The `object-view-box` property can be used to crop or resize replaced elements, including images and videos. It works by showing a section of the content of the replaced element in the available space reserved for that element. The section of the replaced element that is displayed is defined by the value of the property. The available reserved space is determined by the element's default extrinsic size. The subsection of the content displayed can be presented zoomed-in, panned-out, or at its original size, while maintaining the content's intrinsic aspect ratio.

The property value is a `<basic-shape-rect>`, one of the `<basic-shape>` functions limited to defining a rectangular shape. For example, the value can be a `xywh()` function:

```
img {
  object-view-box: xywh(410px 0 400px 150px);
}
```

In this case, the top left corner of the section of the image that will be displayed is `410px` from the left edge and `0` from the top, as defined by x and y coordinate parameters. The size of the section of the original image that will be displayed is `400px` wide by `150px` tall; the `w` and `h` components of the function. This 400x150 section of the image will be displayed in the space that was reserved for the image itself when the page was laid out. Whether the content section displayed is its original intrinsic size, zoomed in, or zoomed out depends on whether the viewbox is 400px x 150px, smaller, or larger than the image element's size, respectively.



In this case, as the rectangular viewbox defined by the `object-view-box` property and the rectangular area of the `<img>` element are the same size, i.e., 400 x 150 pixels, the replaced element is not scaled.

Decrease the `w` and `h` values to create a zoomed-in effect; as the smaller image section is stretched, it has a zoomed-in effect.

Increase the `w` and `h` argument values to create a zoomed-out effect; as the larger image section is shrunk, it has a zoomed-out effect.

Animating the `x` and `y` coordinates creates a panning effect by moving the element's viewbox while the element itself remains in its original location.

Formal definition

Initial value	none
Applies to	replaced elements
Inherited	no
Computed value	specified keyword, or computed function
Animation type	as if possible, otherwise discrete

Formal syntax

```
object-view-box =
  none                |
  <basic-shape-rect>

<basic-shape-rect> =
  <inset()>  |
  <rect()>   |
  <xywh()>

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<length-percentage> =
  <length>      |
  <percentage>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the basic usage and the effects of the `object-view-box` property, with examples using the three different `<basic-shape-rect>` functions.

HTML

We have three almost identical `<img>` elements wrapped in `<figure>` elements; the only difference is their class names.

```
<figure>
  <img
    class="intrinsic"
    src="https://mdn.github.io/shared-assets/images/examples/painted-hand.jpg"
    alt="Original scale section of a painted hand" />
  <figcaption>intrinsic</figcaption>
</figure>
<figure>
  <img
    class="zoom-in"
    src="https://mdn.github.io/shared-assets/images/examples/painted-hand.jpg"
    alt="Zoomed in section of a painted hand" />
  <figcaption>zoomed in</figcaption>
</figure>
<figure>
  <img
    class="zoom-out"
    src="https://mdn.github.io/shared-assets/images/examples/painted-hand.jpg"
    alt="Zoomed out section of a painted hand" />
```

```
  <figcaption>zoomed out</figcaption>
</figure>
```

CSS

We style all the images to have the same width and height, then set each class, and therefore each image, to have a different `object-view-box` value. The intrinsic size of the painted hand image is `298px` by `332px`. We set the `height` and `width`, setting the extrinsic size to `200px` by `200px`.

```
img {
  width: 200px;
  height: 200px;
  border: 1px solid red;
}
```

We set three different `object-view-box` property values using three different shape functions. The `intrinsic` element's `xywh()` function value displays a `200px` square section of the image content, starting from the `70px` from the left and `90px` from the top. The `zoom-in` element's `rect()` function displays a `160px` square section of the original element, going from `110px` to `270px` from the top edge and `90px` to `250px` from the left edge. The `zoom-out` replaced element's `inset()` function displays a `298px` square section of the original element; showing the entire width of the image while cutting of a 17px from the top and bottom.

```
.intrinsic {
  object-view-box: xywh(70px 90px 200px 200px);
}

.zoom-in {
  object-view-box: rect(110px 250px 270px 90px);
}

.zoom-out {
  object-view-box: inset(17px 0 17px 0);
}
```

```
@supports not (object-view-box: none) {
  body::before {
    content: "Your browser does not support the 'object-view-box' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
  }
}

figure {
  display: inline-block;
  margin: 0;
}

figcaption {
  text-align: center;
}
```

Results

The `intrinsic` version is showing an unscaled section of the image. The `zoom-in` version is showing a smaller section (`160px` square) of the image, scaled up to fit `200px` square viewbox. As the section is stretched, it looks zoomed in. The `zoom-out` version is showing a larger (`298px` square) section of the image, scaled down to fit `200px` square viewbox. As the section is shrunk, it looks zoomed out.

Live zoom-in using object-view-box property

This example demonstrates using the `object-view-box` property to zoom a section of a replaced element in and out, within a statically sized HTML element. In this case, the eye of the leopard, within a very large image, serves as the focal point of the zoom effect.

HTML

We include an `<img>` element and a `range` `<input>` element, with an associated `<label>`. The natural dimensions, or intrinsic size, of the original leopard image are `1244px` wide by `416px` tall, with an aspect ratio of `3:1`.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/leopard.jpg"
  alt="leopard" />
<p>
  <label for="box-size">Zoom-in: </label>
  <input type="range" id="box-size" min="115" max="380" value="150" />
</p>
<output></output>
```

CSS

We define a `--box-size` custom property, which is used as the height and width in the `xywh()` function, creating a square viewbox with an aspect ratio of `1:1`. The view box's offset point, the focal point in our zoom effect, is set at `500px` for the `x` coordinate and `30px` for the `y` coordinate, which corresponds to the top-left corner of the leopard's right eye.

```
input {
  width: 350px;
}

output {
  text-align: center;
  background-color: #dedede;
  font-family: monospace;
  padding: 5px;
  display: block;
}

@supports not (object-view-box: none) {
  body::before {
    content: "Your browser does not support the 'object-view-box' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
```

```
  }
}
```

```
img {
  width: 350px;
  height: 350px;
  border: 2px solid red;

  --box-size: 150px;
  object-view-box: xywh(500px 30px var(--box-size) var(--box-size));
}
```

JavaScript

We add an event listener to the slider that updates the value of the `--boxSize` custom property when the user interacts with it. To increase the zoom-in effect when the slider is moved to the right, the slider's value is inverted by subtracting it from `500px`, as reducing the view box size increases the zoom-in effect.

```
const img = document.querySelector("img");
const zoom = document.getElementById("box-size");
const output = document.querySelector("output");

function update() {
  const size = 500 - zoom.value;
  img.style.setProperty("--box-size", `${size}px`);
  output.innerText = `object-view-box: xywh(500px 30px ${size}px ${size}px);`;
}

zoom.addEventListener("input", update);
update();
```

Result

Move the slider to the right to increase the zoom-in effect and left to reduce it. The slider only affects the dimensions of the view box, while the x and y values, the origin point of the viewbox, remain constant. The size of the `<img>` element doesn't change either.

Panning using the object-view-box property

This example demonstrates panning an image by animating the `object-view-box` property value.

HTML

The HTML includes a single image.

```
<img
  src="https://mdn.github.io/shared-assets/images/examples/leopard.jpg"
  alt="leopard" />
```

CSS

We define an image size and keep the viewbox dimensions, the `w` and `h` components of the `xywh()` function, constant, as we animate the top and left positions, changing the position of the `x` and `y` parameters over five seconds.

```
img {
  width: 350px;
  height: 350px;

  object-view-box: xywh(0 30px 400px 400px);

  animation: panning 5s linear infinite alternate;
}

@keyframes panning {
  from {
    object-view-box: xywh(0 -50px 400px 400px);
  }
  to {
    object-view-box: xywh(800px 68px 400px 400px);
  }
}
```

```
@supports not (object-view-box: none) {
  body::before {
    content: "Your browser does not support the 'object-view-box' property.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
  }
}
```

Result

See also

- Using the CSS `object-view-box` property
- `object-fit`
- `object-position`
- `background-size`
- Understanding aspect ratio
- CSS images module

offset CSS property

The **`offset`** CSS shorthand property sets all the properties required for animating an element along a defined path. The offset properties together help to define an *offset transform*, a transform that aligns a point in an element (offset-anchor) to an *offset position* (offset-position) on a path (offset-path) at various points along the path (offset-distance) and optionally rotates the element (offset-rotate) to follow the direction of the path.

> **Note:** Early versions of the spec called this property `motion`.

Try it

```
offset: path("M 20 60 L 120 60 L 70 10 L 20 60") 0% auto 90deg;
```

```
offset: path(
    "M 20 210 L 74 210 L 118 140 \
 L 62 140 L 20 210"
  )
  20% auto;
```

```
<section class="default-example" id="default-example">
  <div class="wrapper">
    <div id="example-element"></div>
  </div>
  <button id="playback" type="button">Play</button>
</section>
```

```
#example-element {
  width: 24px;
  height: 24px;
  background: #2bc4a2;
  clip-path: polygon(0% 0%, 70% 0%, 100% 50%, 70% 100%, 0% 100%, 30% 50%);
  animation: distance 3000ms infinite normal ease-in-out;
  animation-play-state: paused;
}

#example-element.running {
  animation-play-state: running;
}

.wrapper {
  height: 220px;
  width: 200px;
  background:
    url('data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 150 140" width="200" height="140"><path d="M 0 60 L 100 60 L 50 10 L 0 60" fill="none" stroke="lightgrey"
stroke-width="2" stroke-dasharray="4.5"/></svg>')
      no-repeat,
    url('data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 -140 150 230" width="200" height="230"><path d="M 0 70 L 56 70 L 98 0 L 42 0 L 0 70" fill="none"
stroke="lightgrey" stroke-width="2" stroke-dasharray="4.5"/></svg>');
}

@keyframes distance {
  to {
    offset-distance: 100%;
  }
}

#playback {
  position: absolute;
  top: 0;
  left: 0;
  font-size: 1em;
}
```

```
const example = document.getElementById("example-element");
const button = document.getElementById("playback");

button.addEventListener("click", () => {
  if (example.classList.contains("running")) {
    example.classList.remove("running");
    button.textContent = "Play";
  } else {
    example.classList.add("running");
    button.textContent = "Pause";
  }
});
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `offset-anchor`
- `offset-distance`
- `offset-path`
- `offset-position`
- `offset-rotate`

Syntax

```
/* Offset position */
offset: auto;
offset: 10px 30px;
offset: none;

/* Offset path */
offset: ray(45deg closest-side);
offset: path("M 100 100 L 300 100 L 200 300 z");
offset: url("arc.svg");

/* Offset path with distance and/or rotation */
offset: url("circle.svg") 100px;
offset: url("circle.svg") 40%;
offset: url("circle.svg") 30deg;
offset: url("circle.svg") 50px 20deg;

/* Including offset anchor */
```

```
offset: ray(45deg closest-side) / 40px 20px;
offset: url("arc.svg") 2cm / 0.5cm 3cm;
offset: url("arc.svg") 30deg / 50px 100px;

/* Global values */
offset: inherit;
offset: initial;
offset: revert;
offset: revert-layer;
offset: unset;
```

Formal definition

Initial value	as each of the properties of the shorthand: • `offset-position`: `normal` • `offset-path`: `none` • `offset-distance`: `0` • `offset-anchor`: `auto` • `offset-rotate`: `auto`
Applies to	transformable elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `offset-position`: refer to the size of containing block • `offset-distance`: refer to the total path length • `offset-anchor`: relative to the width and the height of the element's reference box
Computed value	as each of the properties of the shorthand: • `offset-position`: for `<length>` the absolute value, otherwise a percentage • `offset-path`: as specified • `offset-distance`: for `<length>` the absolute value, otherwise a percentage • `offset-anchor`: for `<length>` the absolute value, otherwise a percentage • `offset-rotate`: as specified
Animation type	as each of the properties of the shorthand: • `offset-position`: a position • `offset-path`: by computed value type • `offset-distance`: a length, percentage or calc(); • `offset-anchor`: a position • `offset-rotate`: as <angle>, <basic-shape> or <path()>
Creates stacking context	yes

Formal syntax

```
offset =
  [ <'offset-position'>? [ <'offset-path'> [ <'offset-distance'> || <'offset-rotate'> ]? ]! [ / <'offset-anchor'> ]?

<offset-position> =
  normal      |
  auto        |
  <position>

<offset-path> =
  none                          |
  <offset-path> || <coord-box>

<offset-distance> =
  <length-percentage>

<offset-rotate> =
  [ auto | reverse ]  ||
  <angle>

<offset-anchor> =
  auto        |
  <position>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<offset-path> =
  <ray()>        |
  <url>          |
  <basic-shape>

<coord-box> =
  <paint-box>  |
  view-box

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
```

```
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<ray()> =
  ray( <angle>          &&
  <ray-size>?           &&
  contain?              &&
  [ at <position> ]? )

<basic-shape> =
  <basic-shape-rect>  |
  <circle()>          |
  <ellipse()>         |
  <polygon()>         |
  <path()>            |
  <shape()>

<paint-box> =
  <visual-box>  |
  fill-box      |
  stroke-box

<ray-size> =
  <radial-extent>  |
  sides

<basic-shape-rect> =
  <inset()>  |
  <rect()>   |
  <xywh()>

<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<path()> =
  path( <'fill-rule'>? , <string> )

<shape()> =
  shape( <'fill-rule'>? from <position> , <shape-command># )

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<fill-rule> =
  nonzero  |
  evenodd

<shape-command> =
  <move-command>             |
  <line-command>             |
  close                      |
  <horizontal-line-command>  |
  <vertical-line-command>    |
  <curve-command>            |
  <smooth-command>           |
  <arc-command>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<move-command> =
  move <command-end-point>

<line-command> =
  line <command-end-point>
```

```
<horizontal-line-command> =
  hline [ to [ <length-percentage> | left | center | right | x-start | x-end ] | by <length-percentage> ]

<vertical-line-command> =
  vline [ to [ <length-percentage> | top | center | bottom | y-start | y-end ] | by <length-percentage> ]

<curve-command> =
  curve [ [ to <position> with <control-point> [ / <control-point> ]? ] | [ by <coordinate-pair> with <relative-control-point> [ / <relative-control-point> ]? ] ]

<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Box Model Module Level 4, CSS Fill and Stroke Module Level 3, CSS Images Module Level 3, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Animating an element along a path

HTML

```
<div id="offsetElement"></div>
```

CSS

```
@keyframes move {
  from {
    offset-distance: 0%;
  }

  to {
    offset-distance: 100%;
  }
}

#offsetElement {
  width: 50px;
  height: 50px;
  background-color: blue;
  offset: path("M 100 100 L 300 100 L 200 300 z") auto;
  animation: move 3s linear infinite;
}
```

Result

See also

- `offset-anchor`
- `offset-distance`
- `offset-path`
- `offset-position`
- `offset-rotate`

offset-anchor CSS property

The **`offset-anchor`** CSS property specifies the point inside the box of an element traveling along an `offset-path` that is actually moving along the path.

Try it

```
offset-anchor: auto;
```

```
offset-anchor: right top;
```

```
offset-anchor: left bottom;
```

```
offset-anchor: 20% 80%;
```

```
<section class="default-example" id="default-example">
  <div class="wrapper">
    <div id="example-element"></div>
  </div>
  <button id="playback" type="button">Play</button>
</section>
```

```
#example-element {
  offset-path: path("M 0,20 L 200,20");
  animation: distance 3000ms infinite alternate ease-in-out;
  width: 40px;
  height: 40px;
  background: cyan;
  animation-play-state: paused;
}

#example-element.running {
  animation-play-state: running;
}

.wrapper {
  background-image: linear-gradient(
    to bottom,
    transparent,
    transparent 49%,
    black 50%,
    black 51%,
    transparent 52%
  );
  border: 1px solid #cccccc;
  width: 90%;
}

@keyframes distance {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}

#playback {
  position: absolute;
  top: 0;
  left: 0;
  font-size: 1em;
}
```

```
const example = document.getElementById("example-element");
const button = document.getElementById("playback");

button.addEventListener("click", () => {
  if (example.classList.contains("running")) {
    example.classList.remove("running");
    button.textContent = "Play";
  } else {
    example.classList.add("running");
    button.textContent = "Pause";
  }
});
```

Syntax

```
/* Keyword values */
offset-anchor: top;
offset-anchor: bottom;
offset-anchor: left;
offset-anchor: right;
offset-anchor: center;
offset-anchor: auto;

/* <percentage> values */
offset-anchor: 25% 75%;

/* <length> values */
offset-anchor: 0 0;
offset-anchor: 1cm 2cm;
offset-anchor: 10ch 8em;

/* Edge offsets values */
offset-anchor: bottom 10px right 20px;
offset-anchor: right 3em bottom 10px;

/* Global values */
offset-anchor: inherit;
offset-anchor: initial;
offset-anchor: revert;
offset-anchor: revert-layer;
offset-anchor: unset;
```

Values

`auto`

`offset-anchor` is given the same value as the element's `transform-origin`, unless `offset-path` is `none`, in which case it takes its value from `offset-position`.

`<position>`

A `<position>` defines an x/y coordinate, to place an item relative to the edges of an element's box. It can be defined using one to four values. For more specifics, see the `<position>` and `background-position` reference pages. Note that the 3-value position syntax does not work for any usage of `<position>`, except for in `background(-position)`.

Formal definition

Initial value	`auto`
Applies to	transformable elements

Inherited	no
Percentages	relative to the width and the height of the element's reference box
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	a position

Formal syntax

```
offset-anchor =
  auto        |
  <position>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting various offset-anchor values

In the following example, we have three `<div>` elements nested in `<section>` elements. Each `<div>` is given the same `offset-path` (a horizontal line 200 pixels long) and animated to move along it. The three are then given different `background-color` and `offset-anchor` values.

Each `<section>` has been styled with a linear gradient to give it a horizontal line running through its center, to give you a visual display of where the `<div>`'s offset paths are running.

This allows you to see what effect the different `offset-anchor` values have — the first one, `auto`, causes the `<div>`'s center point to move along the path. The other two cause the `<div>`'s top-right and bottom-left points to move along the path, respectively.

HTML

```
<section>
  <div class="offset-anchor1"></div>
</section>
<section>
  <div class="offset-anchor2"></div>
</section>
<section>
  <div class="offset-anchor3"></div>
</section>
```

CSS

```
div {
  offset-path: path("M 0,20 L 200,20");
  animation: move 3000ms infinite alternate ease-in-out;
  width: 40px;
  height: 40px;
}

section {
  background-image: linear-gradient(
    to bottom,
    transparent,
    transparent 49%,
    black 50%,
    black 51%,
    transparent 52%
  );
  border: 1px solid #cccccc;
  margin-bottom: 10px;
}

.offset-anchor1 {
  offset-anchor: auto;
  background: cyan;
}

.offset-anchor2 {
```

```
  offset-anchor: right top;
  background: purple;
}

.offset-anchor3 {
  offset-anchor: left bottom;
  background: magenta;
}

@keyframes move {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}
```

Result

See also

- `offset`
- `offset-distance`
- `offset-rotate`
- SVG `<path>`

offset-distance CSS property

The **`offset-distance`** CSS property specifies a position along an `offset-path` for an element to be placed.

Try it

```
offset-distance: 0%;
```

```
offset-distance: 80%;
```

```
offset-distance: 50px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  width: 24px;
  height: 24px;
  background: #2bc4a2;
  offset-path: path("M-70,-40 C-70,70 70,70 70,-40");
  clip-path: polygon(0% 0%, 70% 0%, 100% 50%, 70% 100%, 0% 100%, 30% 50%);
}

/* Provides a reference image of what path the element is following */
#default-example {
  background-position: calc(50% - 12px) calc(50% + 14px);
  background-repeat: no-repeat;
  background-image: url('data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg" viewBox="-75 -45 150 140" width="150" height="140"><path d="M-70,-40 C-70,70 70,70 70,-40" fill="none"
stroke="lightgrey" stroke-width="2" stroke-dasharray="4.5"/></svg>');
}
```

Syntax

```
/* Default value */
offset-distance: 0;

/* the middle of the offset-path */
offset-distance: 50%;

/* a fixed length positioned along the path */
offset-distance: 40px;

/* Global values */
offset-distance: inherit;
offset-distance: initial;
offset-distance: revert;
offset-distance: revert-layer;
offset-distance: unset;
```

`<length-percentage>`

A length that specifies how far the element is along the path (defined with `offset-path`).

100% represents the total length of the path (when the `offset-path` is defined as a basic shape or `path()`).

Formal definition

Initial value	`0`
Applies to	transformable elements
Inherited	no
Percentages	refer to the total path length
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	a length, percentage or calc();

Formal syntax

```
offset-distance =
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using offset-distance in an animation

The motion aspect in CSS Motion Path typically comes from animating the `offset-distance` property. If you want to animate an element along its full path, you would define its `offset-path` and then set up an animation that takes the `offset-distance` from `0%` to `100%`.

HTML

```
<div id="motion-demo"></div>
```

CSS

```
#motion-demo {
  offset-path: path("M20,20 C20,100 200,0 200,100");
  animation: move 3000ms infinite alternate ease-in-out;
  width: 40px;
  height: 40px;
  background: cyan;
}

@keyframes move {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}
```

Result

See also

- `offset`
- `offset-anchor`
- `offset-path`
- `offset-position`
- `offset-rotate`

offset-path CSS property

The **`offset-path`** CSS property specifies a path for an element to follow and determines the element's positioning within the path's parent container or the SVG coordinate system. The path is a line, a curve, or a geometrical shape along which the element gets positioned or moves.

The `offset-path` property is used in combination with the `offset-distance`, `offset-rotate`, and `offset-anchor` properties to control the position and orientation of the element along a path.

Try it

```
offset-path: path("M-70,-40 C-70,70 70,70 70,-40");
```

```
offset-path: path("M0,0 L60,70 L-60,30z");
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
  <button id="playback" type="button">Play</button>
</section>
```

```
#example-element {
  width: 24px;
  height: 24px;
  background: #2bc4a2;
  animation: distance 8000ms infinite linear;
  animation-play-state: paused;
  clip-path: polygon(0% 0%, 70% 0%, 100% 50%, 70% 100%, 0% 100%, 30% 50%);
}

#example-element.running {
  animation-play-state: running;
}

#playback {
  position: absolute;
  top: 0;
  left: 0;
  font-size: 1em;
}

@keyframes distance {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}
```

```
#default-example {
  position: relative;
}
```

```
const example = document.getElementById("example-element");
const button = document.getElementById("playback");

button.addEventListener("click", () => {
  if (example.classList.contains("running")) {
    example.classList.remove("running");
    button.textContent = "Play";
  } else {
    example.classList.add("running");
    button.textContent = "Pause";
  }
});
```

Syntax

```
/* Default */
offset-path: none;

/* Line segment */
offset-path: ray(45deg closest-side contain);
offset-path: ray(contain 150deg at center center);
offset-path: ray(45deg);

/* URL */
offset-path: url("#my-circle");

/* Basic shape */
offset-path: circle(50% at 25% 25%);
offset-path: ellipse(50% 50% at 25% 25%);
offset-path: inset(50% 50% 50% 50%);
offset-path: polygon(30% 0%, 70% 0%, 100% 50%, 30% 100%, 0% 70%, 0% 30%);
offset-path: path("M 0,200 Q 200,200 260,80 Q 290,20 400,0 Q 300,100 400,200");
offset-path: rect(5px 5px 160px 145px round 20%);
offset-path: xywh(0 5px 100% 75% round 15% 0);

/* Coordinate box */
offset-path: content-box;
offset-path: padding-box;
offset-path: border-box;
offset-path: fill-box;
offset-path: stroke-box;
offset-path: view-box;

/* Global values */
offset-path: inherit;
offset-path: initial;
offset-path: revert;
offset-path: revert-layer;
offset-path: unset;
```

Values

The `offset-path` property takes as its value an `<offset-path>` value, a `<coord-box>` value, or both, or the `none` keyword. The `<offset-path>` value is a `ray()` function, a `<url>` value, or a `<basic-shape>` value.

`none`

Specifies that the element does not follow any offset path. The `none` value is equivalent to the element not having any offset transform. The element's movement in this case is determined by its default position properties, such as `top` and `left`, instead of an offset path. This is the default value.

`<offset-path>`

A `ray()` function, a `<url>` value, or a `<basic-shape>` value that specifies the geometrical offset path. If omitted, the path shape for the `<coord-box>` value is `inset(0 round X)`, where `X` is the value of `border-radius` of the element that establishes the containing block.

`ray()`

Defines a line starting at a set position, of a set length, and extending at the specified angle. The `ray()` function accepts up to four parameters – an `<angle>`, an optional size value, the optional keyword `contain`, and an optional `at <position>`.

`<url>`

Specifies the ID of an SVG shape element. The path is the shape of the SVG `<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, or `<rect>` element referenced by its `id` in the `url()` function. If the URL does not reference a shape element or is otherwise invalid, the resolved value for the offset path is `path("M0,0")` (which is a valid `<basic-shape>` value).

`<basic-shape>`

Specifies the offset path as the equivalent path of a CSS basic shape function, such as `circle()`, `ellipse()`, `inset()`, `path()`, `polygon()`, `rect()`, or `xywh()`. For example, if the `<basic_shape>` is an `ellipse()` function, then the path is the outline of the ellipse, starting at the rightmost point of the ellipse, proceeding clockwise through a full rotation. For `ellipse()` and `circle()`, which accept the `at <position>` parameter, if the `<position>` is omitted, the position defaults to `center` unless the element has an `offset-position` specified. In this case, the `offset-position` value is used for the `at <position>` parameter. More complex shapes can be defined using the `shape()` function.

`<coord-box>`

Specifies the size information of the reference box containing the path. The reference box is derived from the element that establishes the containing block for this element. This parameter is optional. If not specified, the default value is `border-box` in CSS contexts. In SVG contexts, the value is treated as `view-box`. If `ray()` or `<basic-shape>` is used to define the offset path, the `<coord-box>` value provides the reference box for the ray or the `<basic-shape>`, respectively. If `<url>` is used to define the offset path, the `<coord-box>` value provides the viewport and user coordinate system for the shape element, with the origin (`0 0`) at the top left corner and size being `1px`.

Description

The `offset-path` property defines a path an animated element can follow. An offset path is either a specified path with one or multiple sub-paths or the geometry of a not-styled basic shape. The element's exact position on the offset path is determined by the `offset-distance` property. Each shape or path must define an initial position for the computed value of `0` for `offset-distance` and an initial direction which specifies the rotation of the object to the initial position.

Early versions of the spec called this property `motion-path`. It was changed to `offset-path` because the property describes static positions, not motion.

Formal definition

Initial value	none
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	by computed value type
Creates stacking context	yes

Formal syntax

```
offset-path =
  none                          |
  <offset-path> || <coord-box>

<offset-path> =
  <ray()>         |
  <url>           |
  <basic-shape>

<coord-box> =
  <paint-box>  |
  view-box

<ray()> =
  ray( <angle>          &&
  <ray-size>?           &&
  contain?              &&
  [ at <position> ]? )

<basic-shape> =
  <basic-shape-rect>  |
  <circle()>          |
  <ellipse()>         |
  <polygon()>         |
  <path()>            |
  <shape()>

<paint-box> =
  <visual-box>  |
  fill-box      |
  stroke-box

<ray-size> =
  <radial-extent>  |
  sides

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<basic-shape-rect> =
  <inset()>  |
  <rect()>   |
  <xywh()>

<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<path()> =
  path( <'fill-rule'>? , <string> )

<shape()> =
  shape( <'fill-rule'>? from <position> , <shape-command># )

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}
```

```
<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<fill-rule> =
  nonzero  |
  evenodd

<length-percentage> =
  <length>      |
  <percentage>

<shape-command> =
  <move-command>             |
  <line-command>             |
  close                      |
  <horizontal-line-command>  |
  <vertical-line-command>    |
  <curve-command>            |
  <smooth-command>           |
  <arc-command>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<move-command> =
  move <command-end-point>

<line-command> =
  line <command-end-point>

<horizontal-line-command> =
  hline [ to [ <length-percentage> | left | center | right | x-start | x-end ] | by <length-percentage> ]

<vertical-line-command> =
  vline [ to [ <length-percentage> | top | center | bottom | y-start | y-end ] | by <length-percentage> ]

<curve-command> =
  curve [ [ to <position> with <control-point> [ / <control-point> ]? ] | [ by <coordinate-pair> with <relative-control-point> [ / <relative-control-point> ]? ] ]

<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Box Model Module Level 4, CSS Fill and Stroke Module Level 3, CSS Images Module Level 3, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating an offset-path using box-edge positioning

This example demonstrates using various `<coord-box>` values in the `offset-path` property.

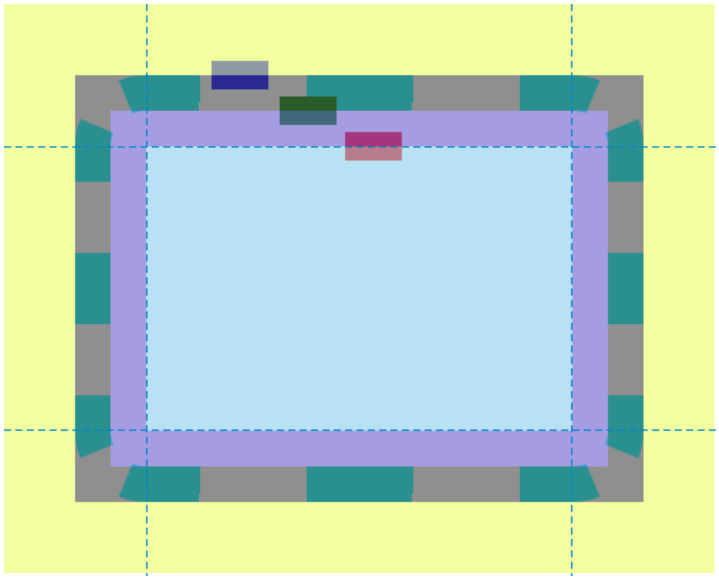
```
<div class="box blueBox"></div>
<div class="box redBox"></div>
<div class="box greenBox"></div>
```

```
body {
  width: 300px;
  height: 200px;
  border-radius: 50px;
  border: dashed aqua;
  border-width: 25px;
  padding: 25px;
```

```
  margin: 50px;
}
```

```
.box {
  width: 40px;
  height: 20px;
  animation: move 8000ms infinite ease-in-out;
}

.blueBox {
  background-color: blue;
  offset-path: border-box;
  offset-distance: 5%;
}

.greenBox {
  background-color: green;
  offset-path: padding-box;
  offset-distance: 8%;
}

.redBox {
  background-color: red;
  offset-path: content-box;
  offset-distance: 12%;
}

@keyframes move {
  0%,
  20% {
    offset-distance: 0%;
  }
  80%,
  100% {
    offset-distance: 100%;
  }
}
```

In this example, the margin, border, and padding have been purposely given large values to demonstrate the placement of the blue, green, and red rectangles on their respective `<coord-box>` edges: border-box, padding-box, and content-box.



Result

Creating an offset-path using path()

In this example, the `<svg>` element creates a house with a chimney and also defines two halves of a scissor. The house and chimney are composed of rectangles and polygons, and the scissor halves are represented by two distinct path elements. In the CSS code, the `offset-path` property is used to specify a path to follow for the two scissor halves. This CSS-defined path is identical to the one represented by the `<path>` element in the SVG, which is the outline of the house including the chimney.

```
<svg
  xmlns="http://www.w3.org/2000/svg"
  width="700"
  height="450"
  viewBox="350 0 1400 900">
  <title>House and Scissors</title>
  <rect x="595" y="423" width="610" height="377" fill="blue" />
  <polygon points="506,423 900,190 1294,423" fill="yellow" />
  <polygon points="993,245 993,190 1086,190 1086,300" fill="red" />
  <path
    id="house"
    d="M900,190 L993,245 V201 A11,11 0 0,1 1004,190 H1075 A11,11 0 0,1 1086,201 V300 L1294,423 H1216 A11,11 0 0,0 1205,434 V789 A11,11 0 0,1 1194,800 H606 A11,11 0 0,1 595,789 V434 A11,11 0 0,0
584,423 H506 L900,190"
    fill="none"
    stroke="black"
    stroke-width="13"
    stroke-linejoin="round"
    stroke-linecap="round" />
  <path
    id="first-scissor-half"
    class="scissor-half"
    d="M30,0 H-10 A10,10 0 0,0 -20,10 A20,20 0 1,1 -40,-10 H20 A10,10 0 0,1 30,0 M-40,20 A10,10 1 0,0 -40,0 A10,10 1 0,0 -40,20 M0,0" />
```

```
  <path
    id="second-scissor-half"
    class="scissor-half"
    d="M30,0 H-10 A10,10 0 0,1 -20,-10 A20,20 0 1,0 -40,10 H20 A10,10 0 0,0 30,0 M-40,-20 A10,10 1 0,0 -40,0 A10,10 1 0,0 -40,-20 M0,0" />
</svg>
```

```
.scissor-half {
  offset-path: path(
    "M900,190 L993,245 V201 A11,11 0 0,1 1004,190 H1075 A11,11 0 0,1 1086,201 V300 L1294,423 H1216 A11,11 0 0,0 1205,434 V789 A11,11 0 0,1 1194,800 H606 A11,11 0 0,1 595,789 V434 A11,11 0 0,0
584,423 H506 L900,190"
  );
  transform: translate(0px, 0px);
  fill: green;
  stroke: black;
  stroke-width: 5px;
  stroke-linejoin: round;
  stroke-linecap: round;
  fill-rule: evenodd;
  offset-anchor: 0 0;
}

#first-scissor-half {
  animation:
    move 12s linear infinite,
    rotate-left 1s infinite;
}
#second-scissor-half {
  animation:
    move 12s linear infinite,
    rotate-right 1s infinite;
}

@keyframes move {
  from {
    offset-distance: 0%;
  }
  to {
    offset-distance: 100%;
  }
}

@keyframes rotate-left {
  0% {
    offset-rotate: auto 0deg;
  }
  50% {
    offset-rotate: auto -45deg;
  }
  100% {
    offset-rotate: auto 0deg;
  }
}

@keyframes rotate-right {
  0% {
    offset-rotate: auto 0deg;
  }
  50% {
    offset-rotate: auto 45deg;
  }
  100% {
    offset-rotate: auto 0deg;
  }
}
```

Result

Without the `offset-path` property, the two halves of the scissors would default to the top-left corner of the canvas. However, by using `offset-path`, the two scissor halves are aligned with the starting point of the SVG path, allowing them to move along it.

Creating an offset-path using url()

This example illustrates how to refer to an SVG shape to define the shape of the path that an element can follow. The green circle (defined by `.target`) follows the path of a rectangle, which is defined by passing the SVG shape's ID (`svgRect`) to the `offset-path` property by using `url()`.

The SVG rectangle that defines the path shape is shown here only to visually demonstrate that the green circle is indeed following the path defined by this rectangle.

```
<div class="outer">
  <div class="target"></div>
</div>
<svg width="400" height="200" xmlns="http://www.w3.org/2000/svg">
  <rect id="svgRect" x="50" y="50" width="200" height="100" />
</svg>
```

```
.outer {
  position: absolute;
}
```

```
.target {
  width: 50px;
  height: 50px;
  border-radius: 50%;
  background-color: green;
  offset-path: url("#svgRect");
  offset-anchor: auto;
  animation: move 5s linear infinite;
}

#svgRect {
  fill: antiquewhite;
  stroke: black;
```

```
  stroke-width: 2;
}

@keyframes move {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}
```

Different shapes

This example involves different `<basic-shape>` values: `circle()`, `ellipse()`, `inset()`, `polygon()`.

```
<div class="container">
  <div class="mover mover-path">path()</div>
  <div class="mover mover-circle">circle()</div>
  <div class="mover mover-ellipse">ellipse()</div>
  <div class="mover mover-inset">inset()</div>
  <div class="mover mover-polygon">polygon()</div>
</div>
```

```
.container {
  border: 1px solid black;
  width: 80vw;
  height: 80vh;
  position: relative;
  left: 10vw;
  top: 10vh;
}

.mover {
  width: 100px;
  height: 80px;
  border-radius: 50%;
  line-height: 80px;
  text-indent: 10px;
  background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' id='e644da42-a34e-4ceb-a89a-89a4eb6dcc51' data-name='Layer 1' viewBox='0 0 71.08 54.62'%3E%3Ctitle%3Epointer-hand%3C/title%3E%3Cpath d='M43.56,49.35a5.24,5.24,0,0,0-1.27-3.43,5.26,5.26,0,0,0,1.86-9,5.26,5.26,0,0,0-.5-9.53L66.12,27c2.28-.07,5-1.57,5-4.58a5.06,5.06,0,0,0-4.58-4.83L34.08,17c3.48-2.89,6.26-6.55,6.73-11.08C41.45-.14,36.07-1.15,35,1.09,32,7.11,23,12.75,17.42,15.52,8.64,19.08,0,19.77,0,34.56,0,42.7,2.7,47.94,9.42,51c5.51,2.52,13.71,3.59,25.36,3.59H38.3A5.27,5.27,0,0,0,43.56,49.35Z'/%3E%3C/svg%3E")
    no-repeat;
  background-size: cover;
  color: white;
  animation: move 10s linear infinite;
  font-family: monospace;
  position: absolute;
  left: 50%;
  transform: translateX(-50%);
  transform-origin: center center;
}
.mover-path {
  top: 50px;
  motion-path: path(
    "M18.45,58.46s52.87-70.07,101.25-.75,101.75-6.23,101.75-6.23S246.38,5.59,165.33,9.08s-15,71.57-94.51,74.56S18.45,58.46,18.45,58.46Z"
  );
  offset-path: path(
    "M18.45,58.46s52.87-70.07,101.25-.75,101.75-6.23,101.75-6.23S246.38,5.59,165.33,9.08s-15,71.57-94.51,74.56S18.45,58.46,18.45,58.46Z"
  );
}
.mover-circle {
  top: 150px;
  offset-path: circle(100px at 50px 50px);
  motion-path: circle(100px at 50px 50px);
}
.mover-ellipse {
  top: 250px;
  offset-path: ellipse(25% 40% at 50% 50%);
  motion-path: ellipse(25% 40% at 50% 50%);
}
.mover-inset {
  top: 350px;
  offset-path: inset(5% 20% 15% 10%);
  motion-path: inset(5% 20% 15% 10%);
}
.mover-polygon {
  top: 450px;
  offset-path: polygon(
    30% 0%,
    70% 0%,
    100% 30%,
    100% 70%,
    70% 100%,
    30% 100%,
    0% 70%,
    0% 30%
  );
  motion-path: polygon(
    30% 0%,
    70% 0%,
    100% 30%,
    100% 70%,
    70% 100%,
    30% 100%,
    0% 70%,
    0% 30%
  );
}

@keyframes move {
```

```
  100% {
    motion-offset: 100%;
    offset-distance: 100%;
  }
}
```

See also

- `offset`
- `offset-distance`
- `offset-rotate`
- SVG paths tutorial
- `path()`

offset-position CSS property

The **`offset-position`** CSS property defines the initial position of an element along a path. This property is typically used in combination with the `offset-path` property to create a motion effect. The value of `offset-position` determines where the element gets placed initially for moving along an offset path if an `offset-path` function such as `path()` does not specify its own starting position.

The `offset-position` property is part of a motion system based on `offset` constituent properties, including `offset-anchor`, `offset-distance`, and `offset-path`. These properties work together to create various motion effects along a path.

Syntax

```
/* Keyword values */
offset-position: normal;
offset-position: auto;
offset-position: top;
offset-position: bottom;
offset-position: left;
offset-position: right;
offset-position: center;

/* <percentage> values */
offset-position: 25% 75%;

/* <length> values */
offset-position: 0 0;
offset-position: 1cm 2cm;
offset-position: 10ch 8em;

/* Edge offsets values */
offset-position: bottom 10px right 20px;
offset-position: right 3em bottom 10px;
offset-position: bottom 10px right;
offset-position: top right 10px;

/* Global values */
offset-position: inherit;
offset-position: initial;
offset-position: revert;
offset-position: revert-layer;
offset-position: unset;
```

Values

`normal`

Indicates that the element does not have an offset starting position and places the element at `50% 50%` of the containing block. This is the default value.

`auto`

Indicates that the offset starting position is the top-left corner of the element's box.

`<position>`

Specifies the position as an x/y coordinate to place an element relative to its box edges. The position can be defined using one to four values. If two non-keyword values are used, the first value represents the horizontal position and the second represents the vertical position. If only one value is specified, the second value is assumed to be `center`. If three or four values are used, the `<length-percentage>` values are offsets for the preceding keyword value(s). For more explanation of these value types, see `background-position`.

Formal definition

Initial value	`normal`
Applies to	transformable elements
Inherited	no
Percentages	refer to the size of containing block
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	a position

Formal syntax

```
offset-position =
  normal      |
  auto        |
  <position>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-one> =
  left               |
  center             |
  right              |
  top                |
  bottom             |
```

```
  x-start              |
  x-end                |
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting initial offset-position for an offset-path

In this example, the `offset-path` property is used to define the path along which the `cyan` element should move. The value of the `path()` CSS function is an SVG data path that describes a curved path. The element moves along this curved path during the `move` animation.

HTML

```
<div id="wrap">
  <div id="motion-demo"></div>
</div>
```

CSS

```
#wrap {
  width: 260px;
  height: 160px;
  border: 1px dashed black;
}
```

```
#motion-demo {
  offset-path: path("M20,20 C20,100 200,0 200,100");
  offset-position: left top;
  animation: move 3000ms infinite alternate ease-in-out;
  width: 40px;
  height: 40px;
  background: cyan;
}

@keyframes move {
  0%,
  20% {
    offset-distance: 0%;
  }
  80%,
  100% {
    offset-distance: 100%;
  }
}
```

Result

Comparing various offset starting positions

This example visually compares various initial offset starting position of an element when `ray()` is used to specify a value for the `offset-path` property. The number inside the element box indicates the element to which CSS is applied as well as the element's anchor point.

```
<div class="wrap">
  <div class="box">0</div>
  <div class="box box0">0</div>
  <pre>
    offset-position: normal;
    /* No offset-path specified */
  </pre>
</div>

<div class="wrap">
  <div class="box">0</div>
  <div class="box box1">1</div>
  <pre>
    offset-position: normal;
    offset-path: ray(0deg);
  </pre>
</div>

<div class="wrap">
  <div class="box">0</div>
  <div class="box box2">2</div>
  <pre>
    offset-position: auto;
    offset-path: ray(0deg);
  </pre>
</div>
```

```
<div class="wrap">
  <div class="box">0</div>
  <div class="box box3">3</div>
  <pre>
    offset-position: left top;
    offset-path: ray(0deg);
  </pre>
</div>

<div class="wrap">
  <div class="box">0</div>
  <div class="box box4">4</div>
  <pre>
    offset-position: 30% 70%;
    offset-path: ray(120deg);
  </pre>
</div>
```

```
.wrap {
  position: relative;
  width: 80vw;
  height: 120px;
  border: 1px solid black;
  margin: 0 2em 4em 5em;
  text-align: center;
}

pre {
  font-size: 1em;
  text-align: right;
  padding-right: 10px;
  line-height: 1em;
}

.box {
  display: flex;
  justify-content: center;
  align-items: center;
  width: 50px;
  height: 50px;
}

.box + .box {
  opacity: 1;
}
```

```
.box {
  background-color: green;
  border-top: 6px dashed white;
  background-clip: border-box;
  position: absolute;
  top: 20px;
  left: 20px;
  opacity: 20%;
  color: white;
}

.box0 {
  offset-position: normal;
}

.box1 {
  offset-position: normal;
  offset-path: ray(0deg);
}

.box2 {
  offset-position: auto;
  offset-path: ray(0deg);
}

.box3 {
  offset-position: left top;
  offset-path: ray(0deg);
}

.box4 {
  offset-position: 30% 70%;
  offset-path: ray(120deg);
}
```

Result

In `box0`, the absence of the `offset-path` property means that an `offset-position` of either `normal` or `auto` has no effect. When `offset-position` is `normal`, the ray starts at the center of the containing block (i.e., `50% 50%`). This is the default starting position of an offset path and is used when no `offset-position` is specified. Notice the difference between offset starting positions `auto` and `left top`. The value `auto` aligns the element's anchor point to its own top-left corner, whereas the value `left top` aligns the element's anchor point to the top-left corner of the containing block.

See also

- `offset`
- `offset-anchor`
- `offset-distance`
- `offset-path`
- `offset-rotate`

offset-rotate CSS property

The **`offset-rotate`** CSS property defines the orientation/direction of the element as it is positioned along the `offset-path`.

Try it

```
offset-rotate: auto;
```

```
offset-rotate: 90deg;
```

```
offset-rotate: auto 90deg;
```

```
offset-rotate: reverse;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
  <button id="playback" type="button">Play</button>
</section>
```

```
#example-element {
  width: 24px;
  height: 24px;
  background: #2bc4a2;
  offset-path: path("M-70,-40 C-70,70 70,70 70,-40");
  animation: distance 8000ms infinite linear;
  animation-play-state: paused;
  clip-path: polygon(0% 0%, 70% 0%, 100% 50%, 70% 100%, 0% 100%, 30% 50%);
}

#example-element.running {
  animation-play-state: running;
}

#playback {
  position: absolute;
  top: 0;
  left: 0;
  font-size: 1em;
}

@keyframes distance {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}

/* Provides a reference image of what path the element is following */
#default-example {
  position: relative;
  background-position: calc(50% - 12px) calc(50% + 14px);
  background-repeat: no-repeat;
  background-image: url('data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg" viewBox="-75 -45 150 140" width="150" height="140"><path d="M-70,-40 C-70,70 70,70 70,-40" fill="none"
stroke="lightgrey" stroke-width="2" stroke-dasharray="4.5"/></svg>');
}
```

```
const example = document.getElementById("example-element");
const button = document.getElementById("playback");

button.addEventListener("click", () => {
  if (example.classList.contains("running")) {
    example.classList.remove("running");
    button.textContent = "Play";
  } else {
    example.classList.add("running");
    button.textContent = "Pause";
  }
});
```

> **Note:** Early versions of the spec called this property `motion-rotation`.

Syntax

```
/* Follow the path direction, with optional additional angle */
offset-rotate: auto;
offset-rotate: auto 45deg;

/* Follow the path direction but facing the opposite direction of `auto` */
offset-rotate: reverse;

/* Keep a constant rotation regardless the position on the path */
offset-rotate: 90deg;
offset-rotate: 0.5turn;

/* Global values */
offset-rotate: inherit;
offset-rotate: initial;
offset-rotate: revert;
offset-rotate: revert-layer;
offset-rotate: unset;
```

`auto`

The element is rotated by the angle of the direction of the `offset-path`, relative to the positive x-axis. This is the default value.

`<angle>`

The element has a constant clockwise rotation transformation applied to it by the specified rotation angle.

`auto <angle>`

If `auto` is followed by an `<angle>`, the computed value of the angle is added to the computed value of `auto`.

`reverse`

The element is rotated similar to `auto`, except it faces the opposite direction. It is the same as specifying a value of `auto 180deg`.

Formal definition

Initial value	`auto`
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	as <angle>, <basic-shape> or <path()>

Formal syntax

```
offset-rotate =
  [ auto | reverse ]  ||
  <angle>
```

This syntax reflects the latest standard as per Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting element orientation along its offset path

HTML

```
<div></div>
<div></div>
<div></div>
```

CSS

```
div {
  width: 40px;
  height: 40px;
  background: #2bc4a2;
  margin: 20px;
  clip-path: polygon(0% 0%, 70% 0%, 100% 50%, 70% 100%, 0% 100%, 30% 50%);
  animation: move 5000ms infinite alternate ease-in-out;

  offset-path: path("M20,20 C20,50 180,-10 180,20");
}
div:nth-child(1) {
  offset-rotate: auto;
}
div:nth-child(2) {
  offset-rotate: auto 90deg;
}
div:nth-child(3) {
  offset-rotate: 30deg;
}

@keyframes move {
  100% {
    offset-distance: 100%;
  }
}
```

Result

See also

- `offset`
- `offset-anchor`
- `offset-distance`
- `offset-path`
- `offset-position`

opacity CSS property

The **`opacity`** CSS property sets the opacity of an element. Opacity is the degree to which content behind an element is hidden, and is the opposite of transparency.

Try it

```
opacity: 0;
```

```
opacity: 0.33;
```

```
opacity: 1;
```

```
<section class="default-example" id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
#example-element {
  background-color: #963770;
  color: white;
```

```
  padding: 1em;
}
```

Syntax

```
opacity: 0.9;
opacity: 90%;

/* Global values */
opacity: inherit;
opacity: initial;
opacity: revert;
opacity: revert-layer;
opacity: unset;
```

Values

`<alpha-value>`

A `<number>` in the range `0.0` to `1.0`, inclusive, or a `<percentage>` in the range `0%` to `100%`, inclusive, representing the opacity of the channel (that is, the value of its alpha channel). Any value outside the interval, though valid, is clamped to the nearest limit in the range.

Value	Meaning
`0`	The element is fully transparent (that is, invisible).
Any `<number>` strictly between `0` and `1`	The element is translucent (that is, content behind the element can be seen).
`1` (default value)	The element is fully opaque (visually solid).

Description

`opacity` applies to the element as a whole, including its contents, even though the value is not inherited by child elements. Thus, the element and its children all have the same opacity relative to the element's background, even if they have different opacities relative to one another.

To change the opacity of a background only, use the `background` property with a color value that allows for an alpha channel. For example:

```
background: rgb(0 0 0 / 40%);
```

When `opacity` value is set to `0`, the element and all of its children appear invisible, but they are still part of the DOM. That means they still register pointer events and, if the elements are in a tabbing order, they do get focus. For good usability, make sure to make such elements visible when they receive user interactions or use the CSS `pointer-events` property to disable pointer events and take the element out of the tab order by disabling with the `disabled` attribute or setting `tab-index="-1"` for non-form-related interactive elements.

Using `opacity` with a value other than `1` places the element in a new stacking context.

Opacity alone should not be used to provide information to screen readers. Use the HTML `hidden` attribute, CSS `visibility`, or CSS `display` style properties. It's best to avoid using `aria-hidden` attribute, but if the element is hidden with opacity, then hide it from screen readers as well.

Transitioning opacity

When transitioning the opacity of elements as you add them to the page when content was formerly hidden with `visibility: hidden`, `display: none`, or `content-visibility: hidden`, you need to include both a `@starting-style` and `transition-behavior: allow-discrete`:

```
.card {
  transition:
    opacity 5s,
    display 5s;
  background-color: orange;

  transition-behavior: allow-discrete;
  @starting-style {
    opacity: 0;
  }
}

.card.hidden {
  display: none;
  opacity: 0;
}
```

To enable first-style transitions, `@starting-style` rules are needed. In the above code, setting `opacity: 0` in `@starting-style` provides a starting point for the transition when the element receives its initial style update. For more details, see `@starting-style`.

Setting `transition-behavior: allow-discrete` is required to transition to `display: none`. See the `transition-behavior` property for more details.

Accessibility

If text opacity is adjusted, it is important to ensure that the contrast ratio between the color of the text and the background the text is placed over is high enough that people experiencing low vision conditions will be able to read the content of the page.

Color contrast ratio is determined by comparing the luminosity of the opacity-adjusted text and background color values. In order to meet current Web Content Accessibility Guidelines (WCAG), a ratio of 4.5:1 is required for text content and 3:1 for larger text such as headings. Large text is defined as 18.66px and bold or larger, or 24px or larger.

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Various operating systems provide a preference for reducing transparency. To set the `opacity` based on the user's operating systems transparency preferences, use the `prefers-reduced-transparency` media query.

Formal definition

Initial value	`1`
Applies to	all elements
Inherited	no
Percentages	map to the range `[0,1]`
Computed value	The same as the specified value after clipping the `<number>` to the range [0.0, 1.0].

Animation type	by computed value type

Formal syntax

```
opacity =
  <opacity-value>

<opacity-value> =
  <number>     |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting opacity

The following example demonstrates how the `opacity` property changes the opacity of the entire element and content, thus making the text very hard to read.

HTML

```
<div class="light">You can barely see this.</div>
<div class="medium">This is easier to see.</div>
<div class="heavy">This is very easy to see.</div>
```

CSS

```
div {
  background-color: yellow;
  font-weight: bold;
  font-size: 130%;
}
.light {
  opacity: 0.2; /* Barely see the text over the background */
}
.medium {
  opacity: 0.5; /* See the text more clearly over the background */
}
.heavy {
  opacity: 0.9; /* See the text very clearly over the background */
}
```

Result

Setting opacity on hover

In the following example opacity is changed on hover, so the striped background image on the parent element shows through the image.

HTML

```
<div class="wrapper">
  <img
    src="/shared-assets/images/examples/dino.svg"
    alt="MDN Dino"
    width="128"
    height="146"
    class="opacity" />
</div>
```

CSS

```
img.opacity {
  opacity: 1;
}

img.opacity:hover {
  opacity: 0.5;
}

.wrapper {
  width: 200px;
  height: 160px;
  background-color: #f03cc3;
  background-image: linear-gradient(
    90deg,
    transparent 50%,
    rgb(255 255 255 / 50%) 50%
  );
  background-size: 20px 20px;
}
```

Result

Styling based on user preferences

To style elements based on user's operating systems transparency preferences, use the `prefers-reduced-transparency` media query. The following example demonstrates how to use the `prefers-color-scheme` media query to specify the desired `opacity` based on the user's preferences.

```
.element {
  opacity: 0.5;
}

@media (prefers-reduced-transparency) {
  .element {
    opacity: 1;
  }
}
```

See also

- `prefers-reduced-transparency` media query

- CSS color module
- SVG `opacity` attribute

order CSS property

The **`order`** CSS property sets the order to lay out an item in a flex or grid container. Items in a container are sorted by ascending `order` value and then by their source code order. Items not given an explicit `order` value are assigned the default value of `0`.

Try it

```
order: 0;
```

```
order: 3;
```

```
order: -1;
```

```
order: 2;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">Box 1:</div>
  <div style="order: 1">Box 2: <code>order: 1;</code></div>
  <div style="order: 2">Box 3: <code>order: 2;</code></div>
  <div style="order: 2">Box 4: <code>order: 2;</code></div>
  <div style="order: 3">Box 5: <code>order: 3;</code></div>
</section>
```

```
.default-example {
  max-height: 300px;
  display: flex;
  flex-flow: column;
}

.default-example > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 0.5rem;
  padding: 0.5rem;
  flex: 1;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}

#example-element::after {
  content: attr(style);
  outline: 2px dashed;
  font-family: monospace;
}
```

In the above demo, select the options on the left-hand side to change the value of the pink box's `order` property. The light blue boxes have been given fixed `order` values.

Bear in mind the effect of source order. For example, when `order: 2;` is selected, the pink box is placed before the two blue boxes with `order: 2;`. This is because the pink box appears before the blue boxes in the source code.

Syntax

```
/* <integer> values */
order: 5;
order: -5;

/* Global values */
order: inherit;
order: initial;
order: revert;
order: revert-layer;
order: unset;
```

Since `order` is only meant to affect the *visual order* of elements and not their logical or tab order, it must not be used on non-visual media such as speech.

Defined in the CSS display module, this property impacts only grid and flex items. When `order` is set on an element whose parent's `display` property is not creating a flex or grid container, it has no effect.

Values

`<integer>`

Represents the ordinal group to be used by the item.

Accessibility

Using the `order` property will create a disconnect between the visual presentation of content and DOM order. This will adversely affect low vision users navigating with the aid of assistive technology such as a screen reader. If the visual order differs from the DOM order, your users will have different experiences depending on how they access your content.

- Flexbox & the keyboard navigation disconnect via Tink (2016)
- Source Order Matters via Adrian Roselli (2015)
- Understanding WCAG, Guideline 1.3 explanations
- Understanding Success Criterion 1.3.2 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`0`
Applies to	Flex items, grid items, and absolutely-positioned flex and grid container children
Inherited	no
Computed value	as specified

Animation type	an integer

Formal syntax

```
order =
  <integer>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Display Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Ordering items in a flex container

In this example, we create a classic two-sidebar layout.

HTML

We include a header, a footer, and a main content area. The main content includes an article and two side bars. Note their order! We'll use the CSS `order` property to change their visual order.

```
<header>Header</header>
<main>
  <article>Article</article>
  <nav>Nav</nav>
  <aside>Aside</aside>
</main>
<footer>Footer</footer>
```

CSS

We style the main area using flexible box layout module features; by setting `display` to `flex`, the `<main>` element becomes a flex container. By default, this creates flex items of equal vertical size. The sidebars are both given an absolute `width`, while the `<article>` will consume all the positive free space with a `flex-grow` factor set via the `flex` shorthand.

We then set different `order` property values on each of the flex container's three children; this means the CSS is defining that component's visual order rather than it appearing in the order declared in the HTML.

```
main {
  display: flex;
  text-align: center;
}
main > article {
  flex: 1;
  order: 2;
}
main > nav {
  width: 200px;
  order: 1;
}
main > aside {
  width: 200px;
  order: 3;
}
```

Result

The `<article>` appears first in the source order but visually rendered in the center.

See also

- Basic concepts of flexbox
- Ordering flex items
- CSS grid layout and accessibility
- CSS display module
- Reading order

orphans CSS property

The **`orphans`** CSS property sets the minimum number of lines in a block container that must be shown at the *bottom* of a page, region, or column.

In typography, an *orphan* is the first line of a paragraph that appears alone at the bottom of a page. (The paragraph continues on a following page.)

Syntax

```
/* <integer> values */
orphans: 2;
orphans: 3;

/* Global values */
orphans: inherit;
orphans: initial;
orphans: revert;
orphans: revert-layer;
orphans: unset;
```

Values

`<integer>`

The minimum number of lines that can stay by themselves at the bottom of a fragment before a fragmentation break. The value must be positive.

Formal definition

Initial value	2
Applies to	block container elements
Inherited	yes

Computed value	as specified
Animation type	by computed value type

Formal syntax

```
orphans =
  <integer [1,∞]>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Fragmentation Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Controlling column orphans

This example uses a checkbox to switch `orphans` between `2` and `1`. With `orphans: 2`, at least two lines of a paragraph must appear at the bottom of a column. With `orphans: 1`, the paragraph can be broken anywhere.

HTML

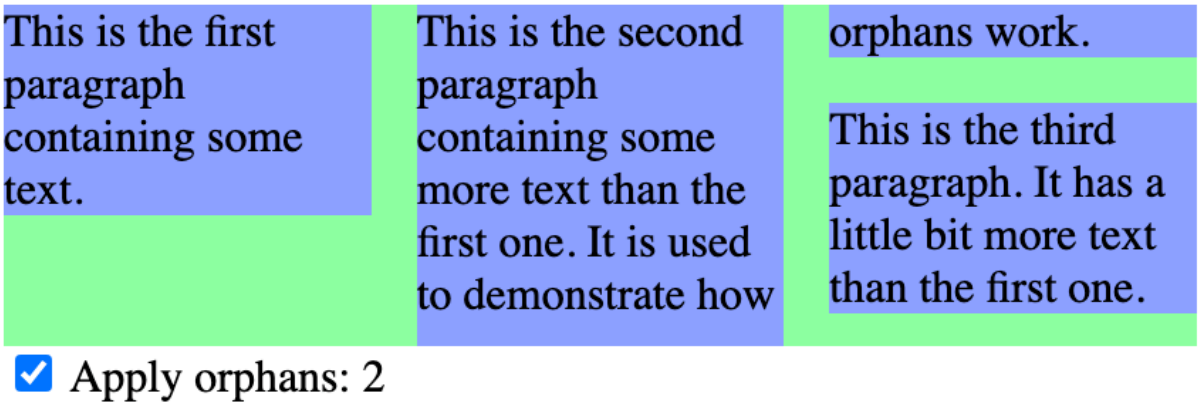
```
<div>
  <p>This is the first paragraph containing some text.</p>
  <p>
    This is the second paragraph containing some more text than the first one.
    It is used to demonstrate how orphans work.
  </p>
  <p>
    This is the third paragraph. It has a little bit more text than the first
    one.
  </p>
</div>
<form>
  <input type="checkbox" id="orphans" checked />
  <label for="orphans">Apply orphans: 2</label>
</form>
```

CSS

```
div {
  background-color: #8cffa0;
  width: 420px;
  height: 120px;
  columns: 3;
  column-fill: auto;
  orphans: 1;
  widows: 1;
}

div:has(~ form input:checked) {
  orphans: 2;
}

p {
  background-color: #8ca0ff;
}

p:first-child {
  margin-top: 0;
}
```

Result

In Firefox, which does not support `orphans`, the behavior in this example is essentially equivalent to `orphans: 1`, regardless of the checkbox state. The following screenshot shows the result with `orphans: 2` in a supporting browser:



See also

- `widows`
- Paged media

outline CSS property

The **`outline`** CSS shorthand property sets the style, color, and width of an element's outline.

Try it

```
outline: solid;
```

```
outline: dashed red;
```

```
outline: 1rem solid;
```

```
outline: thick double #32a1ce;
```

```
outline: 8px ridge rgb(170 50 220 / 0.6);
border-radius: 2rem;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with an outline around it.
  </div>
</section>
```

```
#example-element {
  padding: 0.75rem;
  width: 80%;
  height: 100px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `outline-width`
- `outline-style`
- `outline-color`

Syntax

```
/* style */
outline: solid;

/* style | color */
outline: dashed #ff6666;

/* width | style */
outline: thick inset;

/* width | style | color*/
outline: 3px solid green;

/* Global values */
outline: inherit;
outline: initial;
outline: revert;
outline: revert-layer;
outline: unset;
```

Values

This property is specified as one, two, or three values from the following list:

`<'outline-width'>`

A `<line-width>` value setting the thickness of the outline. Defaults to `medium` if absent. See `outline-width`.

`<'outline-style'>`

Sets the style of the outline. Defaults to `none` if absent. See `outline-style`.

`<'outline-color'>`

Sets the color of the outline. Defaults to `invert` for browsers supporting it, `currentColor` for the others. See `outline-color`.

Description

The `outline` shorthand property sets the style, color, and width of an element's outline. It can be specified using one, two, or three values. The order of the values does not matter. Any omitted sub-values are set to their initial value.

The outline will be invisible for many elements if its style is not defined. This is because the style defaults to `none`. A notable exception is `input` elements, which are given default styling by browsers.

Outline is a line outside of the element's border. Unlike other areas of the box, outlines don't take up space, so they don't affect the layout of the document in any way.

There are a few properties that affect an outline's appearance. It is possible to change the style, color, and width using the `outline` property, the distance from the border using the `outline-offset` property, and corner angles using the `border-radius` property.

An outline is not required to be rectangular: While dealing with multiline text, some browsers will draw an outline for each line box separately, while others will wrap the whole text with a single outline.

Accessibility

Assigning `outline` a value of `0` or `none` will remove the browser's default focus style. If an element can be interacted with it must have a visible focus indicator. Provide obvious focus styling if the default focus style is removed.

- How to Design Useful and Usable Focus Indicators
- WCAG 2.1: Understanding Success Criterion 2.4.7: Focus Visible

Formal definition

Initial value	as each of the properties of the shorthand: • `outline-width`: `medium` • `outline-style`: none • `outline-color`: `auto`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `outline-width`: the absolute `<length>`, snapped as a line width • `outline-style`: as specified • `outline-color`: For the keyword `auto`, the computed value is `currentcolor`. For the color value, if the value is translucent, the computed value will be the `rgba()` corresponding one. If it isn't, it will be the `rgb()` corresponding one. The `transparent` keyword maps to `rgba(0,0,0,0)`.
Animation type	as each of the properties of the shorthand: • `outline-width`: a length • `outline-style`: by computed value type • `outline-color`: a color

Formal syntax

```
outline =
  <'outline-width'>  ||
  <'outline-style'>  ||
  <'outline-color'>

<outline-width> =
  <line-width>

<outline-style> =
  auto                  |
  <outline-line-style>

<outline-color> =
  auto                  |
  <'border-top-color'>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<outline-line-style> =
  none    |
  auto    |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<border-top-color> =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                           &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4, CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using outline to set a focus style

HTML

```
<a href="#">This link has a special focus style.</a>
```

CSS

```
a {
  border: 1px solid;
  border-radius: 3px;
  display: inline-block;
  margin: 10px;
  padding: 5px;
}

a:focus {
  outline: 4px dotted #ee7733;
```

```
  outline-offset: 4px;
  background: #ffffaa;
}
```

Result

See also

- outline-width
- outline-style
- outline-color
- border

outline-color CSS property

The **outline-color** CSS property sets the color of an element's outline.

Try it

```
outline-color: red;
```

```
outline-color: #32a1ce;
```

```
outline-color: rgb(170 50 220 / 0.6);
```

```
outline-color: hsl(60 90% 50% / 0.8);
```

```
outline-color: currentColor;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with an outline around it.
  </div>
</section>
```

```
#example-element {
  outline: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* <color> values */
outline-color: #f92525;
outline-color: rgb(30 222 121);
outline-color: blue;

/* Global values */
outline-color: inherit;
outline-color: initial;
outline-color: revert;
outline-color: revert-layer;
outline-color: unset;
```

The `outline-color` property is specified as any one of the values listed below.

Values

`<color>`

The color of the outline, specified as a `<color>`.

The specification also lists an additional value, `auto`, which is not currently supported in any browsers. When implemented, `auto` will compute to `currentColor` unless `outline-style` is set to `auto` then it will compute to the accent color.

Description

An outline is a line that is drawn around an element, outside the `border`. Unlike the element's border, the outline is drawn outside the element's frame, and may overlap other content. The border, on the other hand, will actually alter the page's layout to ensure that it fits without overlapping anything else (unless you explicitly set it to overlap).

It is often more convenient to use the shorthand property `outline` when defining the appearance of an outline.

Accessibility

Custom focus styles commonly involve making adjustments to the `outline` property. If the color of the outline is adjusted, it is important to ensure that the contrast ratio between it and the background the outline is placed over is high enough that people experiencing low vision conditions will be able to perceive it.

An outline is a visual indicator. In order to meet current Web Content Accessibility Guidelines (WCAG), Success Criterion 1.4.11 Non-text Contrast requires a contrast ratio of at least 3:1 between the outline and the adjacent colors it sits against. Note that when the outline is used as a focus indicator, the adjacent colors include both the background outside the element and the element's own background when the outline borders both.

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.11: Non-text Contrast | W3C Understanding WCAG 2.2

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Computed value	For the keyword `auto`, the computed value is `currentcolor`. For the color value, if the value is translucent, the computed value will be the `rgba()` corresponding one. If it isn't, it will be the `rgb()` corresponding one. The `transparent` keyword maps to `rgba(0,0,0,0)`.

Animation type	a color

Formal syntax

```
outline-color =
  auto                |
  <'border-top-color'>

<border-top-color> =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4, CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a solid blue outline

HTML

```
<p>My outline is blue, as you can see.</p>
```

CSS

```
p {
  outline: 2px solid; /* Set the outline width and style */
  outline-color: blue; /* Set the outline color */
  margin: 5px;
}
```

Result

See also

- `outline`
- `outline-width`
- `outline-style`
- The `<color>` data type
- Other color-related properties: `color`, `background-color`, `border-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`

outline-offset CSS property

The **`outline-offset`** CSS property sets the amount of space between an outline and the edge or border of an element.

Try it

```
outline-offset: 4px;
```

```
outline-offset: 0.6rem;
```

```
outline-offset: 12px;
outline: 5px dashed blue;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with an outline around it.
  </div>
</section>
```

```
#example-element {
  border: 2px solid crimson;
  outline: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* <length> values */
outline-offset: 3px;
outline-offset: 0.2em;

/* Global values */
outline-offset: inherit;
outline-offset: initial;
outline-offset: revert;
outline-offset: revert-layer;
outline-offset: unset;
```

Values

`<length>`

The width of the space between the element and its outline. A negative value places the outline inside the element. A value of `0` places the outline so that there is no space between it and the element.

Description

An outline is a line that is drawn around an element, outside the border edge. The space between an element and its outline is transparent. In other words, it is the same as the parent element's background.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
outline-offset =
  <length>
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting outline offset in pixels

HTML

```
<p>Gallia est omnis divisa in partes tres.</p>
```

CSS

```
p {
  outline: 1px dashed red;
  outline-offset: 10px;
  background: yellow;
  border: 1px solid blue;
  margin: 15px;
}
```

Result

See also

- `outline`
- `outline-width`
- `outline-style`
- `outline-color`

outline-style CSS property

The **`outline-style`** CSS property sets the style of an element's outline. An outline is a line that is drawn around an element, outside the `border`.

It is often more convenient to use the shorthand property `outline` when defining the appearance of an outline.

Try it

```
outline-style: none;
```

```
outline-style: dotted;
```

```
outline-style: solid;
```

```
outline-style: groove;
```

```
outline-style: inset;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with an outline around it.
  </div>
</section>
```

```
#example-element {
  outline: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* Keyword values */
outline-style: auto;
outline-style: none;
outline-style: dotted;
outline-style: dashed;
outline-style: solid;
outline-style: double;
outline-style: groove;
outline-style: ridge;
outline-style: inset;
outline-style: outset;

/* Global values */
outline-style: inherit;
```

```
outline-style: initial;
outline-style: revert;
outline-style: revert-layer;
outline-style: unset;
```

Values

This property is specified as one of the following values:

`auto`

Permits the user agent to render a custom outline style.

`none`

No outline is used.

`dotted`

The outline is a series of dots.

`dashed`

The outline is a series of short line segments.

`solid`

The outline is a single line.

`double`

The outline is two single lines. The `outline-width` is the sum of the two lines and the space between them.

`groove`

The outline looks as though it were carved into the page.

`ridge`

The opposite of `groove`: the outline looks as though it were extruded from the page.

`inset`

The outline makes the box look as though it were embedded in the page.

`outset`

The opposite of `inset`: the outline makes the box look as though it were coming out of the page.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
outline-style =
  auto                 |
  <outline-line-style>

<outline-line-style> =
  none    |
  auto    |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting outline style to auto

The `auto` value indicates a custom outline style, described in the specification as "typically a style [that] is either a user interface default for the platform, or perhaps a style that is richer than can be described in detail in CSS, e.g., a rounded edge outline with semi-translucent outer pixels that appears to glow".

HTML

```
<div>
  <p class="auto">Outline Demo</p>
</div>
```

CSS

```
.auto {
  outline-style: auto; /* same result as "outline: auto" */
}

/* To make the Demo clearer */
* {
  outline-width: 10px;
```

```
  padding: 15px;
}
```

Result

Setting outline style to dashed and dotted

HTML

```
<div>
  <div class="dotted">
    <p class="dashed">Outline Demo</p>
  </div>
</div>
```

CSS

```
.dotted {
  outline-style: dotted; /* same result as "outline: dotted" */
}
.dashed {
  outline-style: dashed;
}

/* To make the Demo clearer */
* {
  outline-width: 10px;
  padding: 15px;
}
```

Result

Setting outline style to solid and double

HTML

```
<div>
  <div class="solid">
    <p class="double">Outline Demo</p>
  </div>
</div>
```

CSS

```
.solid {
  outline-style: solid;
}
.double {
  outline-style: double;
}

/* To make the Demo clearer */
* {
  outline-width: 10px;
  padding: 15px;
}
```

Result

Setting outline style to groove and ridge

HTML

```
<div>
  <div class="groove">
    <p class="ridge">Outline Demo</p>
  </div>
</div>
```

CSS

```
.groove {
  outline-style: groove;
}
.ridge {
  outline-style: ridge;
}

/* To make the Demo clearer */
* {
  outline-width: 10px;
  padding: 15px;
}
```

Result

Setting outline style to inset and outset

HTML

```
<div>
  <div class="inset">
    <p class="outset">Outline Demo</p>
  </div>
</div>
```

CSS

```
.inset {
  outline-style: inset;
}
```

```
.outset {
  outline-style: outset;
}

/* To make the Demo clearer */
* {
  outline-width: 10px;
  padding: 15px;
}
```

Result

See also

- `outline`
- `outline-width`
- `outline-color`

outline-width CSS property

The CSS **`outline-width`** property sets the thickness of an element's outline. An outline is a line that is drawn around an element, outside the `border`.

It is often more convenient to use the shorthand property `outline` when defining the appearance of an outline.

Try it

```
outline-width: 12px;
```

```
outline-width: thin;
```

```
outline-width: medium;
```

```
outline-width: thick;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with an outline around it.
  </div>
</section>
```

```
#example-element {
  outline: 0.75em solid;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Syntax

```
/* Keyword values */
outline-width: thin;
outline-width: medium;
outline-width: thick;

/* <length> values */
outline-width: 1px;
outline-width: 0.1em;

/* Global values */
outline-width: inherit;
outline-width: initial;
outline-width: revert;
outline-width: revert-layer;
outline-width: unset;
```

Values

This property is specified as one `<line-width>` value.

`<line-width>`

Defines the width of the outline, either as an explicit non-negative `<length>` or the keywords: `thin`, `medium`, or `thick`. The default is `medium`.

Formal definition

Initial value	`medium`
Applies to	all elements
Inherited	no
Computed value	the absolute `<length>`, snapped as a line width
Animation type	a length

Formal syntax

```
outline-width =
  <line-width>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4, CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting an element's outline width

HTML

```
<span id="thin">thin</span>
<span id="medium">medium</span>
<span id="thick">thick</span>
<span id="twopixels">2px</span>
<span id="oneex">1ex</span>
<span id="em">1.2em</span>
```

CSS

```
span {
  outline-style: solid;
  display: inline-block;
  margin: 20px;
}

#thin {
  outline-width: thin;
}

#medium {
  outline-width: medium;
}

#thick {
  outline-width: thick;
}

#twopixels {
  outline-width: 2px;
}

#oneex {
  outline-width: 1ex;
}

#em {
  outline-width: 1.2em;
}
```

Result

See also

- outline
- outline-style
- outline-color

overflow CSS property

The **overflow** CSS shorthand property sets the desired behavior when content does not fit in the element's padding box (overflows) in the horizontal and/or vertical direction.

Try it

```
overflow: visible;
```

```
overflow: hidden;
```

```
overflow: clip;
```

```
overflow: scroll;
```

```
overflow: auto;
```

```
<section class="default-example" id="default-example">
  <p id="example-element">
    Michaelmas term lately over, and the Lord Chancellor sitting in Lincoln's
    Inn Hall. Implacable November weather. As much mud in the streets as if the
    waters had but newly retired from the face of the earth.
  </p>
</section>
```

```
#example-element {
  width: 15em;
  height: 9em;
  border: medium dotted;
  padding: 0.75em;
  text-align: left;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- overflow-x
- overflow-y

Syntax

```
/* Keyword values */
overflow: visible;
overflow: hidden;
overflow: clip;
```

```
overflow: scroll;
overflow: auto;

/* Two-value syntax: horizontal | vertical */
overflow: hidden visible;

/* Global values */
overflow: inherit;
overflow: initial;
overflow: revert;
overflow: revert-layer;
overflow: unset;
```

Values

This property is specified as one or two space-separated `<overflow>` keyword values:

`visible`

> Overflow content is not clipped and may be visible outside the element's padding box. The element box is not a scroll container. This is the default value.

`hidden`

> Overflow content is clipped at the element's padding box, hiding the clipped content. When overflowing, the element box is a scroll container with no scroll bars; scrolling is still possible via other methods including tabbing to hidden focusable elements, properties such as `scrollLeft` property, and methods like `scrollTo()`.

`clip`

> Overflow content is clipped at the element's *overflow clip edge* defined by the `overflow-clip-margin` property. The element box is not a scroll container, clipped content is not visible, and programmatic scrolling is not supported.

`scroll`

> Overflow content is clipped at the element's padding box. Whether overflowing or not, the element box is always a scroll container displaying scroll bars.

`auto`

> Overflow content is clipped at the element's padding box. When overflowing, the element box is a scroll container displaying scroll bars.

Non-standard values

Some non-standard values are also supported in some browsers:

`overlay`

> A legacy alias for `auto`, defined in the specification for web compatibility. Originally implemented as a non-standard value to render scrollbars on top of content rather than taking up space. Its use in new code is discouraged.

Description

By default, block-level elements grow to fit their content. If the size of a container is constrained, the content will overflow. The `overflow` property determines how a container handles content that overflows its edges.

The `overflow` property is shorthand for the horizontal `overflow-x` and vertical `overflow-y` properties. If only one keyword is specified, both `overflow-x` and `overflow-y` are set to the same value. If two keywords are specified, the first value applies to `overflow-x` in the horizontal direction and the second one applies to `overflow-y` in the vertical direction.

Overflow options include hiding overflow content, enabling scroll bars to view overflow content or displaying the content flowing out of an element box into the surrounding area, and combinations there of.

All values, except `visible` and `clip`, create a new block formatting context. The new block formatting context is necessary for scroll containers because if a float were to intersect with a scrolling element, it would forcibly rewrap the content after each scroll step, leading to a slow scrolling experience.

What creates overflow

Overflow occurs when a block-level element has content that overflows its constrained space. The allotted space may be constrained by a height (`height` or `max-height`) for vertical overflow, a width (`width` or `max-width`) for horizontal overflow, a block-size (`block-size` or `max-block-size`) for block direction overflow, or an inline-size (`inline-size`, `max-inline-size`, or `white-space` set to `nowrap`) for inline direction overflow.

The following CSS limits the size of the container, defining box model property values that limit the container's size. A background color, clipped to the `content-box`, is also added to help demonstrate how overflowing content overflows the content box in the explanations that follow.

```
div {
  height: 10em;
  width: 15em;
  border: 3px solid;
  padding: 10px;

  background-color: #ededed;
  background-clip: content-box;
}
```

Understanding overflow values

The different `overflow` values define whether an element has scrollbars, whether it is user and programmatically scrollable, and whether it is a scroll container (which creates a new block formatting context) when that element's contents overflow and, in the case of `scroll`, even when they don't.

The `visible` value

The default value is `visible`. By default, if content overflows a container's constraints, the contents are not contained to the container. An element that defaults or is explicitly set to `visible` doesn't have scrollbars, is not user or programmatically scrollable, and is not a scroll container. This value does not create a new block formatting context.

```
div {
  overflow: visible;
}
```

```
<div>
  <h2>overflow: visible;</h2>
</div>
<div>
  <p>
    The <code>overflow</code> property in this example is set to
    <code>visible</code>. No scroll container is created; the content just
```

```
    overflows the container.
  </p>
</div>
```

With `visible`, no clipping occurs, so the overflowing content is visible outside the element's padding box, potentially overlapping adjacent content.

The `scroll` value

With `scroll`, content is clipped at the element's padding box and can be scrolled into view. User agents display scroll bars in both directions, whether or not any content is overflowing, which prevents scroll bars from appearing and disappearing as content changes. Printers may still print overflow content.

The element is always a scroll container — even when there is no overflowing content — and is both user-scrollable and programmatically scrollable.

```
div {
  overflow: scroll;
}
```

```
<div>
  <h2>overflow: scroll;</h2>
</div>
<div>
  <p>
    The <code>overflow</code> property in this example is set to
    <code>scroll</code>. A scroll container is created; a scrollbar is always
    visible, even if not needed.
  </p>
</div>
```

In the first example, even though the content is not overflowing, a scroll container is still created. In the second example, the content is clipped to the padding box, with a scroll bar enabling scrolling to the overflowing content.

The `auto` value

With `auto`, whether the element has scrollbars and is a scroll container depends on whether the element has overflowing content. When the element contains overflowing content, it behaves like `scroll`: the overflowing content is clipped at the element's padding box, and overflow content can be scrolled into view using scroll bars. Unlike `scroll`, user agents display scroll bars *only if* the content is overflowing. If content fits inside the element's padding box, it looks the same as with `visible`, but still establishes a new formatting context. The element box is a scroll container only when there is overflowing content.

```
div {
  overflow: auto;
}
```

```
<div>
  <h2>overflow: auto;</h2>
</div>
<div>
  <p>
    The <code>overflow</code> property in this example is set to
    <code>auto</code>. Because there is overflowing content, a scroll container
    is created. The content is clipped to the padding box and a scroll bar
    enables scrolling to the overflowing content.
  </p>
</div>
```

When using the two value syntax, if one value is set to `visible`, that `visible` overflow direction will behave as `auto` if the other value is not `visible` or `clip`.

The `hidden` value

With `hidden`, overflowing content is clipped at the element's padding box. There are no scroll bars and the clipped content is not visible (i.e., it's "hidden").

If there is overflowing content, the element is a scroll container. While there are no scroll bars and the user cannot view the content outside the clipped region by actions such as dragging on a touch screen or using the scroll wheel on a mouse, the hidden overflow content can be scrolled into view.

If the content contains focusable elements, tabbing brings the currently focused element into view. Content can also be scrolled to programmatically. Setting a value for the `scrollLeft` or `scrollTop` property scrolls that distance from the left or top edge, respectively. You can also scroll using the `scrollTo()` method.

```
div {
  overflow: hidden;
}
```

```
<div>
  <h2>overflow: hidden;</h2>
</div>
<div>
  <p>
    The <code>overflow</code> property in this example is set to
    <code>hidden</code>. While hidden,
    <a
      href="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Content_categories#interactive_content"
      >interactive content</a
    >
    can be scrolled into view when focused. Because content overflows, the
    element is a scroll container, even though no scroll bars are present.
    Tabbing will give this
    <input aria-label="input" placeholder="input" /> focus and scroll it into
    view. This content is also programmatically scrollable.
  </p>
</div>
```

The first example does not have overflowing content and is not a scroll container. The second example has overflowing content that is clipped to the padding box. Even though there is no scroll bar enabling scrolling to the overflowing content, the content can be scrolled into view, such as by tabbing to the `<input>` in the hidden content. The second example is a scroll container.

The `clip` value

With `clip`, content that overflows is by default hidden, there are no scroll bars, and programmatic scrolling is not possible. The element is not a scroll container and no new formatting context is created. If the clipped content includes interactive content, hidden focusable content still receives keyboard focus, but that content will not scroll into view, making it inaccessible to keyboard users.

With `clip`, overflow content is clipped at the element's *overflow clip edge* that is defined using the `overflow-clip-margin` property. The clipped content overflows the element's padding box by the `<length>` value of `overflow-clip-margin`, which defaults to `0px`.

```
div {
  overflow: clip;
}
```

```
<div>
  <h2>overflow: clip;</h2>
</div>
<div>
  <p>
    The <code>overflow</code> property in this example is set to
    <code>clip</code>. When hidden,
    <a
      href="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Content_categories#interactive_content"
      >interactive content</a
    >
    can NOT be scrolled into view when focused. Overflowing content is clipped
    to the container. No scroll container is created. Tabbing will give this
    <input aria-label="input" placeholder="input" /> focus but will not scroll
    it into view. Keyboard users will not be able to reach this content. This
    content is also not programmatically scrollable.
  </p>
</div>
<fieldset>
  <legend>Select a <code>overflow-clip-margin</code> value</legend>
  <ul>
    <li>
      <label
        ><input type="radio" name="overflow" value="0" /> overflow-clip-margin:
        0;</label
      >
    </li>
    <li>
      <label
        ><input type="radio" name="overflow" value="3em" />
        overflow-clip-margin: 3em;</label
      >
    </li>
  </ul>
</fieldset>
```

The second example's overflowing content is clipped. Tabbing to the `<input>` in the hidden content gives the element focus, but does not scroll it into view, making that content inaccessible to keyboard users.

When using the two value syntax, if one value is set to `clip`, that `clip` overflow direction will behave as `hidden` if the other value is not set to `visible` or `clip`.

```
ul {
  list-style-type: none;
}
label {
  font-family: monospace;
}
:has([value="0"]:checked) div {
  overflow-clip-margin: 0;
}
:has([value="3em"]:checked) div {
  overflow-clip-margin: 3em;
}

@supports not (overflow-clip-margin: 0) {
  body::before {
    content: "Your browser doesn't support overflow-clip-margin yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;

    width: 100%;
  }
}
```

With scroll-driven animations

When creating scroll-driven animations using the `scroll()` function, consider using `clip` instead of `hidden` if there is no interactive content within the clipped area, unless you explicitly want to create a scroll container.

With both `hidden` and `clip`, the overflow gets clipped, but `overflow: clip` doesn't create a scroll container, so clipped elements are skipped when the user agent goes up the DOM tree looking for the nearest ancestor scroll container.

Because `overflow: hidden` creates a scroll container when there is overflowing content, you might accidentally create a scrolling-ancestor that doesn't scroll. But only use `clip` if you are certain you won't clip any interactive content.

```
div {
  height: 10em;
  width: 15em;
  border: 3px solid;
  padding: 10px;
  background-color: #ededed;
  background-clip: content-box;
}
p {
  font-size: 1.5rem;
  line-height: 1.6;
  font-family: sans-serif;
}
h2 {
  font-family: monospace;
}
body {
  height: 98vh;
  overflow: clip;
  display: flex;
```

```
  flex-flow: row wrap;
  gap: 3em;
}
```

Overflowing replaced elements

Setting `overflow` on images and other replaced elements works as expected in browsers that support CSS Overflow Module Level 4; in previous versions of the specification, replaced elements were always clipped to the bounding container.

See the browser compatibility chart for current browser support information.

Formal definition

Initial value	`visible`
Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	as each of the properties of the shorthand: • `overflow-x`: as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip • `overflow-y`: as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip
Animation type	discrete

Formal syntax

```
overflow =
  <'overflow-block'>{1,2}

<overflow-block> =
  visible  |
  hidden   |
  clip     |
  scroll   |
  auto
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

In some browsers, scrolling content areas are not keyboard-focusable, so they cannot be scrolled by a keyboard-only user. To ensure all keyboard-only users can scroll the container, enable the element to receive focus by setting `tabindex="0"` on the container. To give screen reader users context when the container receives focus, set an appropriate WAI-ARIA role on the container, such as `role="region"`, and an accessible name using the `aria-label` or `aria-labelledby` attribute.

Examples

Demonstrating results of various overflow keywords

HTML

```
<div>
  <code>visible</code>
  <p class="visible">
    Maya Angelou: "I've learned that people will forget what you said, people
    will forget what you did, but people will never forget how you made them
    feel."
  </p>
</div>

<div>
  <code>hidden</code>
  <p class="hidden">
    Maya Angelou: "I've learned that people will forget what you said, people
    will forget what you did, but people will never forget how you made them
    feel."
  </p>
</div>

<div>
  <code>clip</code>
  <p class="clip">
    Maya Angelou: "I've learned that people will forget what you said, people
    will forget what you did, but people will never forget how you made them
    feel."
  </p>
</div>

<div>
  <code>scroll</code>
  <p class="scroll">
    Maya Angelou: "I've learned that people will forget what you said, people
    will forget what you did, but people will never forget how you made them
    feel."
  </p>
</div>

<div>
  <code>auto</code>
  <p class="auto">
    Maya Angelou: "I've learned that people will forget what you said, people
    will forget what you did, but people will never forget how you made them
    feel."
  </p>
</div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
```

```
  justify-content: start;
}

div {
  margin: 2em;
  font-size: 1.2em;
}

p {
  width: 5em;
  height: 5em;
  border: dotted;
  margin-top: 0.5em;
}

div:nth-of-type(5),
div:nth-of-type(6) {
  margin-top: 200px;
}
```

```
p.visible {
  overflow: visible;
}

p.hidden {
  overflow: hidden;
}

p.clip {
  overflow: clip;
  overflow-clip-margin: 1em;
}

p.scroll {
  overflow: scroll;
}

p.auto {
  overflow: auto;
}
```

Result

See also

- `overflow-x`, `overflow-y`
- `overflow-block`, `overflow-clip-margin`, `overflow-inline`
- `clip`, `display`, `text-overflow`, `white-space`
- SVG `overflow` attribute
- Scroll progress timelines
- CSS overflow module
- CSS scroll-driven animations module
- Keyboard-only scrolling areas on adrianroselli.com (2022)

overflow-anchor CSS property

The **`overflow-anchor`** CSS property provides a way to opt out of the browser's scroll anchoring behavior, which adjusts scroll position to minimize content shifts.

Scroll anchoring behavior is enabled by default in any browser that supports it. Therefore, changing the value of this property is typically only required if you are experiencing problems with scroll anchoring in a document or part of a document and need to turn the behavior off.

Try it

```
overflow-anchor: auto;
```

```
overflow-anchor: none;
```

```
<section class="default-example" id="default-example">
  <div class="whole-content-wrapper">
    <button id="playback" type="button">Start lottery</button>
    <p>Magic numbers for today are:</p>
    <div id="example-element"></div>
  </div>
</section>
```

```
.whole-content-wrapper {
  display: flex;
  flex-direction: column;
  height: 100%;
  width: 100%;
}

#example-element {
  height: 100%;
  border: 2px dashed dodgerblue;
  padding: 0.75em;
  text-align: left;
  overflow: scroll;
}

#playback {
  font-size: 1em;
  width: 10em;
  height: 4em;
  font-weight: bold;
  margin: 1em auto;
  background-color: aliceblue;
  border: solid 2px dodgerblue;
  border-radius: 5px;
```

```
}

#playback:hover {
  border-color: lightseagreen;
}

#playback:active {
  filter: brightness(0.9);
}
```

```
const example = document.getElementById("example-element");
const button = document.getElementById("playback");
let intervalId;

function setInitialState() {
  example.innerHTML = "";
  Array.from({ length: 10 }, (_, i) => i).forEach(addContent);
  example.scrollTop = example.scrollHeight;
}

function addContent() {
  console.log("adding content");
  const magicNumber = Math.floor(Math.random() * 10000);
  example.insertAdjacentHTML(
    "afterbegin",
    `<div class="new-content-container">New Magic Number: ${magicNumber}</div>`,
  );
}

button.addEventListener("click", () => {
  if (example.classList.contains("running")) {
    example.classList.remove("running");
    button.textContent = "Start lottery";
    clearInterval(intervalId);
  } else {
    example.classList.add("running");
    button.textContent = "Stop lottery";
    setInitialState();
    intervalId = setInterval(addContent, 1000);
  }
});
```

Syntax

```
/* Keyword values */
overflow-anchor: auto;
overflow-anchor: none;

/* Global values */
overflow-anchor: inherit;
overflow-anchor: initial;
overflow-anchor: revert;
overflow-anchor: revert-layer;
overflow-anchor: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

: The element becomes a potential anchor when adjusting scroll position.

`none`

: The element won't be selected as a potential anchor.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
overflow-anchor =
  auto  |
  none
```

This syntax reflects the latest standard as per CSS Scroll Anchoring Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Prevent scroll anchoring

To prevent scroll anchoring in a document, use the `overflow-anchor` property.

```
* {
  overflow-anchor: none;
}
```

See also

- Overview of scroll anchoring
- CSS scroll anchoring module

overflow-block CSS property

The **`overflow-block`** CSS property sets what shows when content overflows the block start and block end edges of a box. This may be nothing, a scroll bar, or the overflow content.

> **Note:** The `overflow-block` property maps to `overflow-y` or `overflow-x` depending on the writing mode of the document.

Syntax

```
/* Keyword values */
overflow-block: visible;
overflow-block: hidden;
overflow-block: clip;
overflow-block: scroll;
overflow-block: auto;

/* Global values */
overflow-block: inherit;
overflow-block: initial;
overflow-block: revert;
overflow-block: revert-layer;
overflow-block: unset;
```

The `overflow-block` property is specified as a single `<overflow>` keyword value:

Values

`visible`

Content is not clipped and may be rendered outside the padding box's block start and block end edges.

`hidden`

Content is clipped if necessary to fit the block dimension in the padding box. No scrollbars are provided.

`clip`

Overflow content is clipped at the element's overflow clip edge that is defined using the `overflow-clip-margin` property.

`scroll`

Content is clipped if necessary to fit in the block dimension in the padding box. Browsers display scrollbars whether or not any content is actually clipped. (This prevents scrollbars from appearing or disappearing when the content changes.) Printers may still print overflowing content.

`auto`

Depends on the user agent. If content fits inside the padding box, it looks the same as `visible`, but still establishes a new block-formatting context.

Formal definition

Initial value	`auto`
Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip
Animation type	discrete

Formal syntax

```
overflow-block =
  visible  |
  hidden   |
  clip     |
  scroll   |
  auto
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<ul>
  <li>
    <code>overflow-block: hidden</code> (hides the text outside the box)
    <div id="hidden">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-block: scroll</code> (always adds a scrollbar)
    <div id="scroll">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-block: clip</code> (hides the text outside the box beyond the
    overflow clip edge)
    <div id="clip">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
```

```
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-block: visible</code> (displays the text outside the box if
    needed)
    <div id="visible">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-block: auto</code> (on most browsers, equivalent to
    <code>scroll</code>)
    <div id="auto">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>
</ul>
```

CSS

```
div {
  border: 1px solid black;
  width: 250px;
  height: 100px;
  margin-bottom: 120px;
}

#hidden {
  overflow-block: hidden;
}
#scroll {
  overflow-block: scroll;
}
#clip {
  overflow-block: clip;
}
#visible {
  overflow-block: visible;
}
#auto {
  overflow-block: auto;
}
```

Result

See also

- `text-overflow`, `white-space`, `overflow`, `overflow-inline`, `overflow-x`, `overflow-y`, `clip`, `display`
- CSS logical properties
- CSS overflow module
- CSS scrollbars style module
- CSS writing modes
- Learn: Overflowing content

overflow-clip-margin CSS property

The **`overflow-clip-margin`** CSS property determines how far outside its bounds an element with `overflow: clip` may be painted before being clipped. The bound defined by this property is called the *overflow clip edge* of the box.

Syntax

```
/* <length> values */
overflow-clip-margin: 20px;
overflow-clip-margin: 1em;

/* <visual-box> | <length> */
overflow-clip-margin: content-box 5px;

/* Global values */
overflow-clip-margin: inherit;
overflow-clip-margin: initial;
overflow-clip-margin: revert;
overflow-clip-margin: revert-layer;
overflow-clip-margin: unset;
```

The `<visual-box>` value, which defaults to `padding-box`, specifies the box edge to use as the overflow clip edge origin. The `<length>` value specified in `overflow-clip-margin` must be non-negative.

> **Note:** If the element does not have `overflow: clip` then this property will be ignored.

Formal definition

Initial value	`0px`
Applies to	all elements

Inherited	no
Computed value	the computed <length> and a <visual-box> keyword
Animation type	discrete

Formal syntax

```
overflow-clip-margin =
  <visual-box>  ||
  <length>

<visual-box> =
  content-box  |
  padding-box  |
  border-box
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Overflow Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div class="box">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur.
</div>
```

CSS

```
.box {
  border: 3px solid black;
  width: 250px;
  height: 100px;
  overflow: clip;
  overflow-clip-margin: 20px;
}
```

Result

See also

- Related CSS properties: `text-overflow`, `white-space`, `overflow`, `overflow-inline`, `overflow-x`, `overflow-y`, `clip`, `display`

overflow-inline CSS property

The **`overflow-inline`** CSS property sets what shows when content overflows the inline start and end edges of a box. This may be nothing, a scroll bar, or the overflow content.

> **Note:** The `overflow-inline` property maps to `overflow-y` or `overflow-x` depending on the writing mode of the document.

Syntax

```
/* Keyword values */
overflow-inline: visible;
overflow-inline: hidden;
overflow-inline: clip;
overflow-inline: scroll;
overflow-inline: auto;

/* Global values */
overflow-inline: inherit;
overflow-inline: initial;
overflow-inline: revert;
overflow-inline: revert-layer;
overflow-inline: unset;
```

Values

This property is specified as one of the following `<overflow>` keyword values:

`visible`

Content is not clipped and may be rendered outside the padding box's inline start and end edges.

`hidden`

Content is clipped if necessary to fit the inline dimension in the padding box. No scrollbars are provided.

`clip`

Overflow content is clipped at the element's overflow clip edge that is defined using the `overflow-clip-margin` property.

`scroll`

Content is clipped if necessary to fit in the padding box in the inline dimension. Browsers display scrollbars whether or not any content is actually clipped. (This prevents scrollbars from appearing or disappearing when the content changes.) Printers may still print overflowing content.

`auto`

Depends on the user agent. If content fits inside the padding box, it looks the same as `visible`, but still establishes a new block-formatting context. Desktop browsers provide scrollbars if content overflows.

Formal definition

Initial value	`auto`

Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip
Animation type	discrete

Formal syntax

```
overflow-inline =
  visible  |
  hidden   |
  clip     |
  scroll   |
  auto
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline overflow behavior

HTML

```
<ul>
  <li>
    <code>overflow-inline: hidden</code> (hides the text outside the box)
    <div id="div1">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-inline: scroll</code> (always adds a scrollbar)
    <div id="div2">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-inline: visible</code> (displays the text outside the box if
    needed)
    <div id="div3">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-inline: auto</code> (equivalent to <code>scroll</code>
    in most browsers)
    <div id="div4">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-inline: clip</code> (hides the text outside the box beyond
    the overflow clip edge)
    <code>clip</code>
    <div id="div5">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>
</ul>
```

CSS

```
div {
  border: 1px solid black;
  width: 250px;
  margin-bottom: 12px;
}

#div1 {
  overflow-inline: hidden;
}
#div2 {
  overflow-inline: scroll;
}
#div3 {
  overflow-inline: visible;
}
#div4 {
  overflow-inline: auto;
}
#div5 {
  overflow-inline: clip;
  overflow-clip-margin: 2em;
}
```

Result

See also

- `clip`, `display`, `overflow`, `overflow-block`, `overflow-clip-margin`, `overflow-x`, `overflow-y`, `text-overflow`, `white-space`
- CSS overflow module
- CSS logical properties
- CSS writing modes
- Learn: Overflowing content

overflow-wrap CSS property

The **`overflow-wrap`** CSS property applies to text, setting whether the browser should insert line breaks within an otherwise unbreakable string to prevent text from overflowing its line box.

> **Note:** The property was originally a nonstandard and unprefixed Microsoft extension called `word-wrap`, and was implemented by most browsers with the same name. It has since been renamed to `overflow-wrap`, with `word-wrap` being an alias.

Try it

```
overflow-wrap: normal;
```

```
overflow-wrap: anywhere;
```

```
overflow-wrap: break-word;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    Most words are short &amp; don't need to break. But
    <strong class="transition-all" id="example-element"
      >Antidisestablishmentarianism</strong
    >
    is long. The width is set to min-content, with a max-width of 11em.
  </div>
</section>
```

```
.example-container {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
  padding: 0.75em;
  width: min-content;
  max-width: 11em;
  height: 200px;
}
```

> **Note:** In contrast to `word-break`, `overflow-wrap` will only create a break if an entire word cannot be placed on its own line without overflowing.

Syntax

```
/* Keyword values */
overflow-wrap: normal;
overflow-wrap: break-word;
overflow-wrap: anywhere;

/* Global values */
overflow-wrap: inherit;
overflow-wrap: initial;
overflow-wrap: revert;
overflow-wrap: revert-layer;
overflow-wrap: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

Lines may only break at normal word break points (such as a space between two words).

`anywhere`

To prevent overflow, an otherwise unbreakable string of characters — like a long word or URL — may be broken at any point if there are no otherwise-acceptable break points in the line. No hyphenation character is inserted at the break point. Soft wrap opportunities introduced by the word break are considered when calculating min-content intrinsic sizes.

`break-word`

The same as the `anywhere` value, with normally unbreakable words allowed to be broken at arbitrary points if there are no otherwise acceptable break points in the line, but soft wrap opportunities introduced by the word break are NOT considered when calculating min-content intrinsic sizes.

Formal definition

Initial value	`normal`
Applies to	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
overflow-wrap =
  normal      |
  break-word  |
  anywhere
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing overflow-wrap, word-break, and hyphens

This example compares the results of `overflow-wrap`, `word-break`, and `hyphens` when breaking up a long word.

HTML

```
<p>
  They say the fishing is excellent at Lake
  <em class="normal">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>, though
  I've never been there myself. (<code>normal</code>)
</p>
<p>
  They say the fishing is excellent at Lake
  <em class="ow-anywhere">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>,
  though I've never been there myself. (<code>overflow-wrap: anywhere</code>)
</p>
```

```
<p>
  They say the fishing is excellent at Lake
  <em class="ow-break-word">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>,
  though I've never been there myself. (<code>overflow-wrap: break-word</code>)
</p>
<p>
  They say the fishing is excellent at Lake
  <em class="word-break">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>,
  though I've never been there myself. (<code>word-break</code>)
</p>
<p>
  They say the fishing is excellent at Lake
  <em class="hyphens">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>, though
  I've never been there myself. (<code>hyphens</code>, without
  <code>lang</code> attribute)
</p>
<p lang="en">
  They say the fishing is excellent at Lake
  <em class="hyphens">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>, though
  I've never been there myself. (<code>hyphens</code>, English rules)
</p>
<p class="hyphens" lang="de">
  They say the fishing is excellent at Lake
  <em class="hyphens">Chargoggagoggmanchauggagoggchaubunagungamaugg</em>, though
  I've never been there myself. (<code>hyphens</code>, German rules)
</p>
```

CSS

```
p {
  width: 13em;
  margin: 2px;
  background: gold;
}

.ow-anywhere {
  overflow-wrap: anywhere;
}

.ow-break-word {
  overflow-wrap: break-word;
}

.word-break {
  word-break: break-all;
}

.hyphens {
  hyphens: auto;
}
```

Result

See also

- `word-break`
- `white-space`
- `hyphens`
- `text-overflow`
- Guide to wrapping and breaking text

overflow-x CSS property

The **`overflow-x`** CSS property sets what shows when content overflows a block-level element's left and right edges. This may be nothing, a scroll bar, or the overflow content. This property may also be set by using the `overflow` shorthand property.

Try it

```
overflow-x: visible;
```

```
overflow-x: hidden;
```

```
overflow-x: clip;
```

```
overflow-x: scroll;
```

```
overflow-x: auto;
```

```
<section class="default-example" id="default-example">
  <div id="example-element">
    The value of Pi is 3.14159265358979323846264338327950 29. The value of e is
    2.71828182845904523536028747135266 25.
  </div>
</section>
```

```
#example-element {
  width: 15em;
  height: 9em;
  border: medium dotted;
  padding: 0.75em;
  text-align: left;
}
```

Syntax

```
/* Keyword values */
overflow-x: visible;
overflow-x: hidden;
```

```
overflow-x: clip;
overflow-x: scroll;
overflow-x: auto;

/* Global values */
overflow-x: inherit;
overflow-x: initial;
overflow-x: revert;
overflow-x: revert-layer;
overflow-x: unset;
```

The `overflow-x` property is specified as a single `<overflow>` keyword value.

If `overflow-y` is `hidden`, `scroll`, or `auto`, and the `overflow-x` property is `visible` (default), the value will be implicitly computed as `auto`.

Values

`visible`

Overflow content is not clipped and may be visible outside the element's padding box on left and right edges. The element box is not a scroll container.

`hidden`

Overflow content is clipped if necessary to fit horizontally in the elements' padding box. No scroll bars are provided.

`clip`

Overflow content is clipped at the element's *overflow clip edge* that is defined using the `overflow-clip-margin` property. As a result, content overflows the element's padding box by the `<length>` value of `overflow-clip-margin` or by `0px` if not set. The difference between `clip` and `hidden` is that the `clip` keyword also forbids all scrolling, including programmatic scrolling. No new formatting context is created. To establish a formatting context, use `overflow: clip` along with `display: flow-root`. The element box is not a scroll container.

`scroll`

Overflow content is clipped if necessary to fit horizontally inside the element's padding box. Browsers display scroll bars in the horizontal direction whether or not any content is actually clipped. (This prevents scroll bars from appearing or disappearing when the content changes.) Printers may still print overflowing content.

`auto`

Overflow content is clipped at the element's padding box, and overflow content can be scrolled into view. Unlike `scroll`, user agents display scroll bars *only if* the content is overflowing and hide scroll bars by default. If content fits inside the element's padding box, it looks the same as with `visible`, but still establishes a new block-formatting context. Desktop browsers provide scroll bars if content overflows.

> **Note:** The keyword value `overlay` is a legacy value alias for `auto`. With `overlay`, the scroll bars are drawn on top of the content instead of taking up space.

Formal definition

Initial value	`visible`
Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip
Animation type	discrete

Formal syntax

```
overflow-x =
  visible  |
  hidden   |
  clip     |
  scroll   |
  auto
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<ul>
  <li>
    <code>overflow-x:hidden</code> — hides the text outside the box
    <div id="div1">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-x:scroll</code> — always adds a scrollbar
    <div id="div2">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-x:visible</code> — displays the text outside the box if
    needed
    <div id="div3">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>

  <li>
    <code>overflow-x:auto</code> — on most browsers, equivalent to
    <code>scroll</code>
    <div id="div4">ABCDEFGHIJKLMOPQRSTUVWXYZABCDEFGHIJKLMOPQRSTUVWXYZ</div>
  </li>
</ul>
```

CSS

```
#div1,
#div2,
#div3,
#div4 {
  border: 1px solid black;
```

```
  width: 250px;
  margin-bottom: 12px;
}

#div1 {
  overflow-x: hidden;
}
#div2 {
  overflow-x: scroll;
}
#div3 {
  overflow-x: visible;
}
#div4 {
  overflow-x: auto;
}
```

Result

See also

- `clip`, `display`, `text-overflow`, `white-space`
- CSS overflow module
- Learn: Overflowing content

overflow-y CSS property

The **`overflow-y`** CSS property sets what shows when content overflows a block-level element's top and bottom edges. This may be nothing, a scroll bar, or the overflow content. This property may also be set by using the `overflow` shorthand property.

Try it

```
overflow-y: visible;
```

```
overflow-y: hidden;
```

```
overflow-y: clip;
```

```
overflow-y: scroll;
```

```
overflow-y: auto;
```

```
<section class="default-example" id="default-example">
  <p id="example-element">
    Michaelmas term lately over, and the Lord Chancellor sitting in Lincoln's
    Inn Hall. Implacable November weather. As much mud in the streets as if the
    waters had but newly retired from the face of the earth.
  </p>
</section>
```

```
#example-element {
  width: 15em;
  height: 9em;
  border: medium dotted;
  padding: 0.75em;
  text-align: left;
}
```

Syntax

```
/* Keyword values */
overflow-y: visible;
overflow-y: hidden;
overflow-y: clip;
overflow-y: scroll;
overflow-y: auto;

/* Global values */
overflow-y: inherit;
overflow-y: initial;
overflow-y: revert;
overflow-y: revert-layer;
overflow-y: unset;
```

The `overflow-y` property is specified as a single `<overflow>` keyword value.

If `overflow-x` is `hidden`, `scroll`, or `auto` and the `overflow-y` property is `visible` (default), the value will be implicitly computed as `auto`.

Values

`visible`

Overflow content is not clipped and may be visible outside the element's padding box at the top and bottom edges. The element box is not a scroll container.

`hidden`

Overflow content is clipped if necessary to fit vertically in the elements' padding box. No scroll bars are provided.

`clip`

Overflow content is clipped at the element's *overflow clip edge* that is defined using the `overflow-clip-margin` property. As a result, content overflows the element's padding box by the `<length>` value of `overflow-clip-margin` or by `0px` if not set. The difference between `clip` and `hidden` is that the `clip` keyword also forbids all scrolling, including programmatic scrolling. No new formatting context is created. To establish a formatting context, use `overflow: clip` along with `display: flow-root`. The element box is not a scroll container.

`scroll`

Overflow content is clipped if necessary to fit vertically inside the element's padding box. Browsers display scroll bars in the vertical direction whether or not any content is actually clipped. (This prevents scroll bars from appearing or disappearing when the content changes.) Printers may still print overflowing content.

`auto`

Overflow content is clipped at the element's padding box, and overflow content can be scrolled into view. Unlike `scroll`, user agents display scroll bars *only if* the content is overflowing, hiding scroll bars by default. If content fits inside the element's padding box, it looks the same as with `visible`, but still establishes a new block-formatting context.

> **Note:** The keyword value `overlay` is a legacy value alias for `auto`. With `overlay`, the scroll bars are drawn on top of the content instead of taking up space.

Formal definition

Initial value	`visible`
Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip
Animation type	discrete

Formal syntax

```
overflow-y =
  visible  |
  hidden   |
  clip     |
  scroll   |
  auto
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting overflow-y behavior

HTML

```
<ul>
  <li>
    <code>overflow-y:hidden</code> – hides the text outside the box
    <div id="div1">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-y:scroll</code> – always adds a scrollbar
    <div id="div2">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-y:visible</code> – displays the text outside the box if
    needed
    <div id="div3">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>

  <li>
    <code>overflow-y:auto</code> – equivalent to <code>scroll</code>
    on most browsers
    <div id="div4">
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur.
    </div>
  </li>
</ul>
```

CSS

```
div {
  border: 1px solid black;
  width: 250px;
  height: 100px;
}

#div1 {
  overflow-y: hidden;
  margin-bottom: 12px;
}
#div2 {
  overflow-y: scroll;
  margin-bottom: 12px;
}
#div3 {
```

```
  overflow-y: visible;
  margin-bottom: 120px;
}
#div4 {
  overflow-y: auto;
  margin-bottom: 120px;
}
```

Result

See also

- `clip`, `display`, `text-overflow`, `white-space`
- CSS overflow module
- Learn: Overflowing content

overlay CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`overlay`** CSS property specifies whether an element appearing in the top layer (for example, a shown popover or modal `<dialog>` element) is actually rendered in the top layer. This property is only relevant within a list of `transition-property` values, and only if `allow-discrete` is set as the `transition-behavior`.

It is important to note that `overlay` can *only* be set by the browser — author styles cannot change the `overlay` value of any element. You can, however, add `overlay` to the list of transition properties set on an element. This causes its removal from the top layer to be deferred so it can be animated instead of disappearing immediately.

> **Note:** When transitioning `overlay`, you need to set `transition-behavior: allow-discrete` on the transition so that it will animate. `overlay` animations differ from normal discrete animations in that the visible (i.e., `auto`) state will always be shown for the full duration of the transition, regardless of whether it is the start or end state.

Syntax

```
/* Keyword values */
overlay: auto;
overlay: none;

/* Global values */
overlay: inherit;
overlay: initial;
overlay: revert;
overlay: revert-layer;
overlay: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The element is rendered in the top layer if it is promoted to the top layer.

`none`

The element is not rendered in the top layer.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Discrete behavior except when animating to or from `none` is visible for the entire duration

Formal syntax

```
overlay =
  none  |
  auto
```

This syntax reflects the latest standard as per CSS Positioned Layout Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Transitioning a popover

In this example, a popover is animated as it transitions from hidden to shown and back again.

HTML

The HTML contains a `<div>` element declared as a popover using the popover attribute, and a `<button>` element designated as the popover's display control using its popovertarget attribute.

```
<button popovertarget="mypopover">Show the popover</button>
<div popover="auto" id="mypopover">I'm a Popover! I should animate.</div>
```

CSS

The `overlay` property is only present in the list of transitioned properties. As `overlay` is a user-agent controlled property, it is not declared in the pre-transition or post-transition states.

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

[popover]:popover-open {
```

```
  opacity: 1;
  transform: scaleX(1);
}

[popover] {
  font-size: 1.2rem;
  padding: 10px;

  /* Final state of the exit animation */
  opacity: 0;
  transform: scaleX(0);

  transition:
    opacity 0.7s,
    transform 0.7s,
    overlay 0.7s allow-discrete,
    display 0.7s allow-discrete;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

/* Needs to be included after the previous [popover]:popover-open
   rule to take effect, as the specificity is the same */
@starting-style {
  [popover]:popover-open {
    opacity: 0;
    transform: scaleX(0);
  }
}

/* Transition for the popover's backdrop */

[popover]::backdrop {
  background-color: transparent;
  transition:
    display 0.7s allow-discrete,
    overlay 0.7s allow-discrete,
    background-color 0.7s;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

[popover]:popover-open::backdrop {
  background-color: rgb(0 0 0 / 25%);
}

/* Nesting selectors (&) cannot represent pseudo-elements, so this
   starting-style rule cannot be nested. */

@starting-style {
  [popover]:popover-open::backdrop {
    background-color: transparent;
  }
}
```

The two properties we want to animate are `opacity` and `transform`: we want the popover to fade in and out while growing and shrinking in the horizontal direction. We set a starting state for these properties on the default hidden state of the popover element (selected via `[popover]`), and an end state on the open state of the popover (selected via the `:popover-open` pseudo-class). We then set a `transition` property to animate between the two.

Because the animated element is being promoted to the top layer when shown and removed from the top layer when hidden, `overlay` is added to the list of transitioned elements. This ensures that the removal of the element from the top layer is deferred until the animation has ended. This doesn't make a huge difference for basic animations such as this one, but in more complex cases not doing this can result in the element being removed from the overlay too quickly, meaning the animation is not smooth or effective. Note that the `transition-behavior: allow-discrete` value is also set in the shorthand to enable discrete transitions.

The following steps are also required to get the animation working in both directions:

- A starting state for the animation is set inside the `@starting-style` at-rule. This is needed to avoid unexpected behavior. By default transitions are not triggered on elements' first style updates, or when the `display` type changes from `none` to another type. `@starting-style` allows you to override that default in a specific controlled fashion. Without this, the entry animation would not occur and the popover would just appear.
- `display` is also added to the list of transitioned elements so that the animated element is visible (set to `display: block`) throughout both the entry and exit animation. Without this, the exit animation would not be visible; in effect, the popover would just disappear. Again, `transition-behavior: allow-discrete` is required in this case for the animation to occur.

You'll note that we've also included a transition on the `::backdrop` that appears behind the popover when it opens, to provide a nice darkening animation. `[popover]:popover-open::backdrop` is needed to select the backdrop when the popover is open.

Result

The code renders as follows:

> **Note:** Because popovers change from `display: none` to `display: block` each time they are shown, the popover transitions from its `@starting-style` styles to its `[popover]:popover-open` styles every time the entry transition occurs. When the popover closes, it transitions from its `[popover]:popover-open` state to the default `[popover]` state.
>
> It is possible for the style transition on entry and exit to be different in such cases. See our Demonstration of when starting styles are used example for a proof of this.

See also

- CSS transitions module
- `@starting-style`
- `transition-behavior`
- Four new CSS features for smooth entry and exit animations on developer.chrome.com (2023)

overscroll-behavior CSS property

The **`overscroll-behavior`** CSS property controls what happens when scrolling reaches the edge of a scroll container.

Try it

```
overscroll-behavior: auto;
```

```
overscroll-behavior: contain;
```

```
overscroll-behavior: chain;
```

```
overscroll-behavior: none;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="box">
      This is a scrollable container. Michaelmas term lately over, and the Lord
      Chancellor sitting in Lincoln's Inn Hall. Implacable November weather. As
      much mud in the streets as if the waters had but newly retired from the
      face of the earth.
      <br /><br />
      Lorem Ipsum has been the industry's standard dummy text ever since the
      1500s, when an unknown printer took a galley of type and scrambled it to
      make a type specimen book. It has survived not only five centuries, but
      also the leap into electronic typesetting, remaining essentially
      unchanged.
    </div>
    <div id="example-element">
      This is the inner container. Focus on this container, scroll to the bottom
      and when you reach the bottom keep scrolling.
      <p>
        If you have
        <code class="language-css">overscroll-behavior: auto;</code> selected
        the outer container will start to scroll.
      </p>
      If you have
      <code class="language-css">overscroll-behavior: contain;</code> selected,
      the outer container will not scroll unless you move your cursor out of the
      inner container and try to perform scroll on the outer container.
    </div>
  </div>
</section>
```

```
.example-container {
  width: 35em;
  height: 18em;
  border: medium dotted;
  padding: 0.75em;
  text-align: left;
  overflow: auto;
  display: flex;
}

.box {
  width: 50%;
}

#example-element {
  width: 50%;
  height: 12em;
  border: medium dotted #1b76c4;
  padding: 0.3em;
  margin: 0 0.3em;
  text-align: left;
  overflow: auto;
  overscroll-behavior: contain;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `overscroll-behavior-x`
- `overscroll-behavior-y`

Syntax

```
/* Keyword values */
overscroll-behavior: auto; /* default */
overscroll-behavior: chain;
overscroll-behavior: contain;
overscroll-behavior: none;

/* Two values */
overscroll-behavior: auto contain;

/* Global values */
overscroll-behavior: inherit;
overscroll-behavior: initial;
overscroll-behavior: revert;
overscroll-behavior: revert-layer;
overscroll-behavior: unset;
```

Values

The `overscroll-behavior` property is specified as one or two keywords chosen from the list of values below.

Two keywords specifies the `overscroll-behavior` value on the `x` and `y` axes respectively. If only one value is specified, both x and y are assumed to have the same value.

`auto`

Allows the default behavior at a scroll boundary. Scrolling may continue in another scroll container.

`chain`

Allows scrolling to continue outside the scroll container, but prevents overscroll "bounce" effects.

`contain`

Prevents scrolling from continuing outside the scroll container. "Bounce" effects may still occur.

`none`

Prevents scrolling from continuing outside the scroll container and also prevents overscroll "bounce" effects.

Description

By default, mobile browsers tend to provide a "bounce" effect or even a page refresh when the top or bottom of a page (or other scroll area) is reached. You may also have noticed that when you have a dialog box with scrolling content at the top of a page that also has scrolling content, once the dialog box's scroll boundary is reached, the underlying page will then start to scroll — this is called scroll chaining.

In some cases, these behaviors are not desirable. You can use `overscroll-behavior` to get rid of unwanted scroll chaining and the browser's "pull to refresh"-type behavior.

Note that this property applies only to scroll containers. In particular, since an `<iframe>` is not a scroll container, setting this property on an iframe has no effect. To control scroll chaining from an iframe, set `overscroll-behavior` on both the `<html>` and the `<body>` elements of the iframe's document.

A scroll container that has no scrollable overflow, such as an element with `overflow: hidden`, is always considered to be at its scroll boundary. So setting a non-default `overscroll-behavior` such as `contain` or `none` on it will prevent scroll chaining to ancestor scroll containers. This can be used to prevent background scrolling while a dialog or overlay is open.

Formal definition

Initial value	`auto`
Applies to	non-replaced block-level elements and non-replaced inline-block elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `overscroll-behavior-x`: as specified • `overscroll-behavior-y`: as specified
Animation type	discrete

Formal syntax

```
overscroll-behavior =
  [ contain | none | auto | chain ]{1,2}
```

This syntax reflects the latest standard as per CSS Overscroll Behavior Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

<code>overscroll-behavior</code> on <code><dialog></code> elements

In this example there are 4 buttons that launch a dialog, each dialog has a different `overscroll-behavior` value. There is also a `<div>` that forces the content to be larger than the container so that the scroll effect of the page can be seen.

HTML

```
<div class="buttons">
  <button popovertarget="dialog-auto"><code>auto</code></button>
  <button popovertarget="dialog-contain"><code>contain</code></button>
  <button popovertarget="dialog-chain"><code>chain</code></button>
  <button popovertarget="dialog-none"><code>none</code></button>
</div>
<div class="force-scroll"></div>
<dialog id="dialog-auto" popover>
  <code>overscroll-behavior: auto;</code>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Ipsa odio
    voluptatum minus dolorem assumenda nemo quos autem culpa, pariatur illo
    deserunt ea nam commodi fugiat ipsum quam alias suscipit facilis?
  </p>
</dialog>
<dialog id="dialog-contain" popover>
  <code>overscroll-behavior: contain;</code>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Ipsa odio
    voluptatum minus dolorem assumenda nemo quos autem culpa, pariatur illo
    deserunt ea nam commodi fugiat ipsum quam alias suscipit facilis?
  </p>
</dialog>
<dialog id="dialog-chain" popover>
  <code>overscroll-behavior: chain;</code>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Ipsa odio
    voluptatum minus dolorem assumenda nemo quos autem culpa, pariatur illo
    deserunt ea nam commodi fugiat ipsum quam alias suscipit facilis?
  </p>
</dialog>
<dialog id="dialog-none" popover>
  <code>overscroll-behavior: none;</code>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Ipsa odio
    voluptatum minus dolorem assumenda nemo quos autem culpa, pariatur illo
    deserunt ea nam commodi fugiat ipsum quam alias suscipit facilis?
  </p>
</dialog>
```

CSS

```
::backdrop {
  backdrop-filter: blur(0.5rem);
}
.buttons {
```

```
  display: flex;
  flex-direction: row;
  justify-content: space-around;
  margin: 2vb;
}
.force-scroll {
  width: 20vi;
  height: 200vb;
  margin-inline: auto;
  border: solid 1px black;
  background: linear-gradient(45deg, tomato, olivedrab);
}
```

```
[popover] {
  width: 30vi;
  height: 10vi;
}
#dialog-auto {
  overscroll-behavior: auto;
}
#dialog-contain {
  overscroll-behavior: contain;
}
#dialog-chain {
  overscroll-behavior: chain;
}
#dialog-none {
  overscroll-behavior: none;
}
```

Result

Click one of the buttons to see that behavior applied to the dialog that appears, when you scroll within the dialog:

- `auto`: if you overscroll the dialog you will see the content "bounce" and if you then continue to scroll you will see the background scroll too.
- `contain`: if you overscroll the dialog you will see the content "bounce" and if you then continue to scroll you will see the background does not scroll.
- `chain`: if you overscroll the dialog you will see that the content does not "bounce" and if you then continue to scroll you will see the background scroll too.
- `none`: if you overscroll the dialog you will see that the content does not "bounce" and if you then continue to scroll you will see the background does not scroll.

An alternate example of controlling scroll of "chat" window can be seen on the `overscroll-behavior-y` property.

See also

- CSS overscroll behavior module
- CSS scroll anchoring module
- Take control of your scroll: customizing pull-to-refresh and overflow effects on developer.chrome.com (2017)

overscroll-behavior-block CSS property

The **`overscroll-behavior-block`** CSS property sets the browser's behavior when the block direction boundary of a scrolling area is reached.

See `overscroll-behavior` for a full explanation.

Syntax

```
/* Keyword values */
overscroll-behavior-block: auto; /* default */
overscroll-behavior-block: chain;
overscroll-behavior-block: contain;
overscroll-behavior-block: none;

/* Global values */
overscroll-behavior-block: inherit;
overscroll-behavior-block: initial;
overscroll-behavior-block: revert;
overscroll-behavior-block: revert-layer;
overscroll-behavior-block: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Allows the default behavior at a scroll boundary. Scrolling may continue in another scroll container.

`chain`

Allows scrolling to continue outside the scroll container, but prevents overscroll "bounce" effects.

`contain`

Prevents scrolling from continuing outside the scroll container. "Bounce" effects may still occur.

`none`

Prevents scrolling from continuing outside the scroll container and also prevents overscroll "bounce" effects.

Formal definition

Initial value	`auto`
Applies to	non-replaced block-level elements and non-replaced inline-block elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
overscroll-behavior-block =
  contain |
  none    |
  auto    |
  chain
```

This syntax reflects the latest standard as per CSS Overscroll Behavior Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preventing block overscrolling

In this demo we have two block-level boxes, one inside the other. The outer box has a large `height` set on it so the page will scroll vertically. The inner box has a small `width` (and `height`) set on it so it sits comfortably inside the viewport, but its content is given a large `height` so it will also scroll vertically.

By default, when the inner box is scrolled and a scroll boundary is reached, the whole page will begin to scroll, which is probably not what we want. To avoid this happening in the block direction, we've set `overscroll-behavior-block: contain` on the inner box.

HTML

```
<main>
  <div>
    <div>
      <p>
        <code>overscroll-behavior-block</code> has been used to make it so that
        when the scroll boundaries of the yellow inner box are reached, the
        whole page does not begin to scroll.
      </p>
    </div>
  </div>
</main>
```

CSS

```
main {
  height: 3000px;
  width: 500px;
  background-color: white;
  background-image: repeating-linear-gradient(
    to bottom,
    transparent 0px,
    transparent 19px,
    rgb(0 0 0 / 50%) 20px
  );
}

main > div {
  height: 300px;
  width: 400px;
  overflow: auto;
  position: relative;
  top: 50px;
  left: 50px;
  overscroll-behavior-block: contain;
}

div > div {
  height: 1500px;
  width: 100%;
  background-color: yellow;
  background-image: repeating-linear-gradient(
    to bottom,
    transparent 0px,
    transparent 19px,
    rgb(0 0 0 / 50%) 20px
  );
}

p {
  padding: 10px;
  background-color: rgb(255 0 0 / 50%);
  margin: 0;
  width: 340px;
  position: relative;
  top: 10px;
  left: 10px;
}
```

Result

See also

- `overscroll-behavior`
- `overscroll-behavior-x`
- `overscroll-behavior-y`
- `overscroll-behavior-inline`
- CSS overscroll behavior module

overscroll-behavior-inline CSS property

The **`overscroll-behavior-inline`** CSS property sets the browser's behavior when the inline direction boundary of a scrolling area is reached.

See `overscroll-behavior` for a full explanation.

Syntax

```
/* Keyword values */
overscroll-behavior-inline: auto; /* default */
overscroll-behavior-inline: chain;
```

```
overscroll-behavior-inline: contain;
overscroll-behavior-inline: none;

/* Global values */
overscroll-behavior-inline: inherit;
overscroll-behavior-inline: initial;
overscroll-behavior-inline: revert;
overscroll-behavior-inline: revert-layer;
overscroll-behavior-inline: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Allows the default behavior at a scroll boundary. Scrolling may continue in another scroll container.

`chain`

Allows scrolling to continue outside the scroll container, but prevents overscroll "bounce" effects.

`contain`

Prevents scrolling from continuing outside the scroll container. "Bounce" effects may still occur.

`none`

Prevents scrolling from continuing outside the scroll container and also prevents overscroll "bounce" effects.

Formal definition

Initial value	`auto`
Applies to	non-replaced block-level elements and non-replaced inline-block elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
overscroll-behavior-inline =
  contain  |
  none     |
  auto     |
  chain
```

This syntax reflects the latest standard as per CSS Overscroll Behavior Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preventing inline overscrolling

In this demo we have two block-level boxes, one inside the other. The outer box has a large `width` set on it so the page will scroll horizontally. The inner box has a small width (and `height`) set on it so it sits comfortably inside the viewport, but its content is given a large width so it will also scroll horizontally.

By default, when the inner box is scrolled and a scroll boundary is reached, the whole page will begin to scroll, which is probably not what we want. To avoid this happening in the inline direction, we've set `overscroll-behavior-inline: contain` on the inner box.

HTML

```
<main>
  <div>
    <div>
      <p>
        <code>overscroll-behavior-inline</code> has been used to make it so that
        when the scroll boundaries of the yellow inner box are reached, the
        whole page does not begin to scroll.
      </p>
    </div>
  </div>
</main>
```

CSS

```
main {
  height: 400px;
  width: 3000px;
  background-color: white;
  background-image: repeating-linear-gradient(
    to right,
    transparent 0px,
    transparent 19px,
    rgb(0 0 0 / 50%) 20px
  );
}

main > div {
  height: 300px;
  width: 400px;
  overflow: auto;
  position: relative;
  top: 50px;
  left: 50px;
  overscroll-behavior-inline: contain;
}

div > div {
  height: 100%;
  width: 1500px;
  background-color: yellow;
```

```
  background-image: repeating-linear-gradient(
    to right,
    transparent 0px,
    transparent 19px,
    rgb(0 0 0 / 50%) 20px
  );
}

p {
  padding: 10px;
  background-color: rgb(255 0 0 / 50%);
  margin: 0;
  width: 360px;
  position: relative;
  top: 10px;
  left: 10px;
}
```

Result

See also

- `overscroll-behavior`
- `overscroll-behavior-x`
- `overscroll-behavior-y`
- `overscroll-behavior-block`
- CSS overscroll behavior module

overscroll-behavior-x CSS property

The **`overscroll-behavior-x`** CSS property sets the browser's behavior when the horizontal boundary of a scrolling area is reached.

See `overscroll-behavior` for a full explanation.

Syntax

```
/* Keyword values */
overscroll-behavior-x: auto; /* default */
overscroll-behavior-x: chain;
overscroll-behavior-x: contain;
overscroll-behavior-x: none;

/* Global values */
overscroll-behavior-x: inherit;
overscroll-behavior-x: initial;
overscroll-behavior-x: revert;
overscroll-behavior-x: revert-layer;
overscroll-behavior-x: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Allows the default behavior at a scroll boundary. Scrolling may continue in another scroll container.

`chain`

Allows scrolling to continue outside the scroll container, but prevents overscroll "bounce" effects.

`contain`

Prevents scrolling from continuing outside the scroll container. "Bounce" effects may still occur.

`none`

Prevents scrolling from continuing outside the scroll container and also prevents overscroll "bounce" effects.

Formal definition

Initial value	`auto`
Applies to	non-replaced block-level elements and non-replaced inline-block elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
overscroll-behavior-x =
  contain  |
  none     |
  auto     |
  chain
```

This syntax reflects the latest standard as per CSS Overscroll Behavior Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preventing an underlying element from scrolling horizontally

In our overscroll-behavior-x example (see source code also), we have two block-level boxes, one inside the other. The outer box has a large `width` set on it so the page will scroll horizontally. The inner box has a small width (and `height`) set on it so it sits comfortably inside the viewport, but its content is given a large `width` so it will scroll horizontally.

By default, when the inner box is scrolled and a scroll boundary is reached, the whole page will begin to scroll, which is probably not what we want. To avoid this, you can set `overscroll-behavior-x: contain` on the inner box:

```
main > div {
  height: 300px;
  width: 500px;
  overflow: auto;
  position: relative;
  top: 100px;
  left: 100px;
  overscroll-behavior-x: contain;
}
```

See also

- `overscroll-behavior`
- `overscroll-behavior-y`
- `overscroll-behavior-inline`
- `overscroll-behavior-block`
- CSS overscroll behavior module

overscroll-behavior-y CSS property

The **`overscroll-behavior-y`** CSS property sets the browser's behavior when the vertical boundary of a scrolling area is reached.

See `overscroll-behavior` for a full explanation.

Syntax

```
/* Keyword values */
overscroll-behavior-y: auto; /* default */
overscroll-behavior-y: chain;
overscroll-behavior-y: contain;
overscroll-behavior-y: none;

/* Global values */
overscroll-behavior-y: inherit;
overscroll-behavior-y: initial;
overscroll-behavior-y: revert;
overscroll-behavior-y: revert-layer;
overscroll-behavior-y: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Allows the default behavior at a scroll boundary. Scrolling may continue in another scroll container.

`chain`

Allows scrolling to continue outside the scroll container, but prevents overscroll "bounce" effects.

`contain`

Prevents scrolling from continuing outside the scroll container. "Bounce" effects may still occur.

`none`

Prevents scrolling from continuing outside the scroll container and also prevents overscroll "bounce" effects.

Formal definition

Initial value	`auto`
Applies to	non-replaced block-level elements and non-replaced inline-block elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

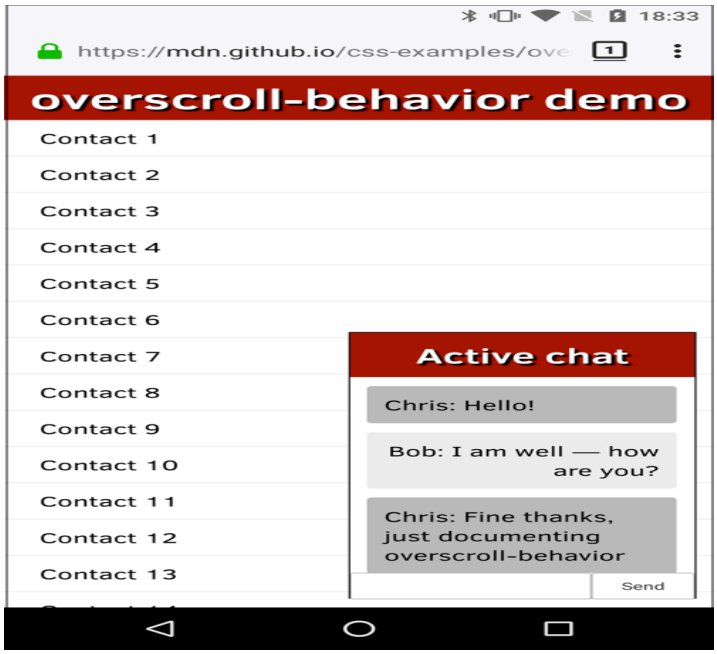
```
overscroll-behavior-y =
  contain  |
  none     |
  auto     |
  chain
```

This syntax reflects the latest standard as per CSS Overscroll Behavior Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preventing an underlying element from scrolling

In our overscroll-behavior example (see the source code also), we present a full-page list of fake contacts, and a dialog box containing a chat window.



Both of these areas scroll; normally if you scrolled the chat window until you hit a scroll boundary, the underlying contacts window would start to scroll too, which is not desirable. This can be stopped using `overscroll-behavior-y` (`overscroll-behavior` would also work) on the chat window, like this:

```
.messages {
  height: 220px;
  overflow: auto;
  overscroll-behavior-y: contain;
}
```

We also wanted to get rid of the standard overscroll effects when the contacts are scrolled to the top or bottom (e.g., Chrome on Android refreshes the page when you scroll past the top boundary). This can be prevented by setting `overscroll-behavior-y: none` on the `<html>` element:

```
html {
  margin: 0;
  overscroll-behavior-y: none;
}
```

See also

- `overscroll-behavior`
- `overscroll-behavior-x`
- `overscroll-behavior-inline`
- `overscroll-behavior-block`
- CSS overscroll behavior module

padding CSS property

The **`padding`** CSS shorthand property sets the padding area on all four sides of an element at once.

Try it

```
padding: 1em;
```

```
padding: 10% 0;
```

```
padding: 10px 50px 20px;
```

```
padding: 10px 50px 30px 0;
```

```
padding: 0;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
}
```

An element's padding area is the space between its content and its border.

> **Note:** Padding creates extra space within an element. In contrast, `margin` creates extra space *around* an element.

Constituent properties

This property is a shorthand for the following CSS properties:

- `padding-top`
- `padding-right`
- `padding-bottom`
- `padding-left`

Syntax

```
/* Apply to all four sides */
padding: 1em;

/* top and bottom | left and right */
padding: 5% 10%;

/* top | left and right | bottom */
padding: 1em 2em 2em;

/* top | right | bottom | left */
padding: 5px 1em 0 2em;

/* Global values */
padding: inherit;
padding: initial;
padding: revert;
padding: revert-layer;
padding: unset;
```

The `padding` property may be specified using one, two, three, or four values. Each value is a `<length>` or a `<percentage>`. Negative values are invalid.

- When **one** value is specified, it applies the same padding to **all four sides**.
- When **two** values are specified, the first padding applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first padding applies to the **top**, the second to the **right and left**, the third to the **bottom**.
- When **four** values are specified, the paddings apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

Values

`<length>`

The size of the padding as a fixed value.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block.

Formal definition

Initial value	as each of the properties of the shorthand: • `padding-bottom`: `0` • `padding-left`: `0` • `padding-right`: `0` • `padding-top`: `0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	as each of the properties of the shorthand: • `padding-bottom`: the percentage as specified or the absolute length • `padding-left`: the percentage as specified or the absolute length • `padding-right`: the percentage as specified or the absolute length • `padding-top`: the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
padding =
  <'padding-top'>{1,4}

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting padding with pixels

HTML

```
<h4>This element has moderate padding.</h4>
<h3>The padding is huge in this element!</h3>
```

CSS

```
h4 {
  background-color: lime;
  padding: 20px 50px;
}

h3 {
  background-color: cyan;
  padding: 110px 50px 50px 110px;
}
```

Result

Setting padding with pixels and percentages

```
padding: 5%; /* All sides: 5% padding */

padding: 10px; /* All sides: 10px padding */

padding: 10px 20px; /* top and bottom: 10px padding */
/* left and right: 20px padding */

padding: 10px 3% 20px; /* top:           10px padding */
/* left and right: 3% padding   */
/* bottom:         20px padding */

padding: 1em 3px 30px 5px; /* top:    1em padding  */
/* right:  3px padding  */
/* bottom: 30px padding */
/* left:   5px padding  */
```

See also

- `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `padding-block-start`, `padding-block-end`, `padding-inline-start`, and `padding-inline-end`
- `padding-block` and `padding-inline` shorthands
- Introduction to the CSS box model guide
- CSS box model module

padding-block CSS property

The **`padding-block`** CSS shorthand property defines the logical block start and end padding of an element, which maps to physical padding properties depending on the element's writing mode, directionality, and text orientation.

Try it

```
padding-block: 10px 20px;
writing-mode: horizontal-tb;
```

```
padding-block: 20px 40px;
writing-mode: vertical-rl;
```

```
padding-block: 5% 10%;
writing-mode: horizontal-tb;
```

```
padding-block: 2em 4em;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `padding-block-start`
- `padding-block-end`

Syntax

```
/* <length> values */
padding-block: 10px 20px; /* An absolute length */
padding-block: 1em 2em; /* relative to the text size */
padding-block: 10px; /* sets both start and end values */

/* <percentage> values */
padding-block: 5% 2%; /* relative to the nearest block container's width */

/* Global values */
padding-block: inherit;
```

```
padding-block: initial;
padding-block: revert;
padding-block: revert-layer;
padding-block: unset;
```

The `padding-block` property may be specified with one or two values. If one value is given, it is used as the value for both `padding-block-start` and `padding-block-end`. If two values are given, the first is used for `padding-block-start` and the second for `padding-block-end`.

Values

The `padding-block` property takes the same values as the `padding-top` property.

Description

The padding values specified by `padding-block` can be equivalent to the `padding-top` and `padding-bottom` properties or the `padding-right` and `padding-left` properties, depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Formal definition

Initial value	as each of the properties of the shorthand: • `padding-block-start`: `0` • `padding-block-end`: `0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`
Inherited	no
Percentages	logical-width of containing block
Computed value	as each of the properties of the shorthand: • `padding-block-start`: as `<length>` • `padding-block-end`: as `<length>`
Animation type	a length

Formal syntax

```
padding-block =
  <'padding-top'>{1,2}

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block padding for vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  padding-block: 20px 40px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `writing-mode`, `direction`, `text-orientation`

padding-block-end CSS property

The **`padding-block-end`** CSS property defines the logical block end padding of an element, which maps to a physical padding depending on the element's writing mode, directionality, and text orientation.

Try it

```
padding-block-end: 20px;
writing-mode: horizontal-tb;
```

```
padding-block-end: 20px;
writing-mode: vertical-rl;
```

```
padding-block-end: 5em;
writing-mode: horizontal-tb;
```

```
padding-block-end: 5em;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <length> values */
padding-block-end: 10px; /* An absolute length */
padding-block-end: 1em; /* A length relative to the text size */

/* <percentage> value */
padding-block-end: 5%; /* A padding relative to the block container's width */

/* Global values */
padding-block-end: inherit;
padding-block-end: initial;
padding-block-end: revert;
padding-block-end: revert-layer;
padding-block-end: unset;
```

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Description

The `padding-block-end` property takes the same values as physical padding properties such as `padding-top`. However, it can be equivalent to `padding-bottom`, `padding-top`, `padding-left`, or `padding-right` depending on the values set for `writing-mode`, `direction`, and `text-orientation`.

It relates to `padding-block-start`, `padding-inline-start`, and `padding-inline-end`, which define the other padding values of the element.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`
Inherited	no
Percentages	logical-width of containing block
Computed value	as `<length>`
Animation type	a length

Formal syntax

```
padding-block-end =
  <'padding-top'>

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block end padding for vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}
```

```
.exampleText {
  writing-mode: vertical-lr;
  padding-block-end: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `writing-mode`, `direction`, `text-orientation`

padding-block-start CSS property

The **`padding-block-start`** CSS property defines the logical block start padding of an element, which maps to a physical padding depending on the element's writing mode, directionality, and text orientation.

Try it

```
padding-block-start: 20px;
writing-mode: horizontal-tb;
```

```
padding-block-start: 20px;
writing-mode: vertical-rl;
```

```
padding-block-start: 5em;
writing-mode: horizontal-tb;
```

```
padding-block-start: 5em;
writing-mode: vertical-lr;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <length> values */
padding-block-start: 10px; /* An absolute length */
padding-block-start: 1em; /* A length relative to the text size */

/* <percentage> value */
padding-block-start: 5%; /* A padding relative to the block container's width */

/* Global values */
padding-block-start: inherit;
padding-block-start: initial;
padding-block-start: revert;
padding-block-start: revert-layer;
padding-block-start: unset;
```

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline-size (*width* in a horizontal language) of the containing block. Must be non-negative.

Description

The `padding-block-start` property takes the same values as physical padding properties such as `padding-top`. However, it can be equivalent to `padding-top`, `padding-bottom`, `padding-left`, or `padding-right` depending on the values set for `writing-mode`, `direction`, and `text-orientation`.

It relates to `padding-block-end`, `padding-inline-start`, and `padding-inline-end`, which define the other padding values of the element.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`
Inherited	no
Percentages	logical-width of containing block
Computed value	as `<length>`
Animation type	a length

Formal syntax

```
padding-block-start =
  <'padding-top'>

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting block start padding for vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  padding-block-start: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `writing-mode`, `direction`, `text-orientation`

padding-bottom CSS property

The **`padding-bottom`** CSS property sets the height of the padding area on the bottom of an element.

Try it

```
padding-bottom: 1em;
```

```
padding-bottom: 10%;
```

```
padding-bottom: 20px;
```

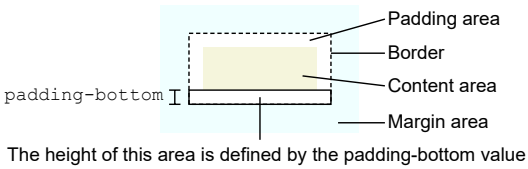
```
padding-bottom: 1ch;
```

```
padding-bottom: 0;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
}
```

An element's padding area is the space between its content and its border.



> **Note:** The `padding` property can be used to set paddings on all four sides of an element with a single declaration.

Syntax

```
/* <length> values */
padding-bottom: 0.5em;
padding-bottom: 0;
padding-bottom: 2cm;

/* <percentage> value */
padding-bottom: 10%;

/* Global values */
padding-bottom: inherit;
padding-bottom: initial;
padding-bottom: revert;
padding-bottom: revert-layer;
padding-bottom: unset;
```

The `padding-bottom` property is specified as a single value chosen from the list below. Unlike margins, negative values are not allowed for padding.

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
padding-bottom =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>     |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting padding bottom with pixels and percentages

```
.content {
  padding-bottom: 5%;
}
.side-box {
  padding-bottom: 10px;
}
```

See also

- `padding-top`, `padding-right`, and `padding-left`
- `padding` shorthand
- `padding-block-start`, `padding-block-end`, `padding-inline-start`, and `padding-inline-end`
- `padding-block` and `padding-inline` shorthands
- Introduction to the CSS box model guide
- CSS box model module

padding-inline CSS property

The **`padding-inline`** CSS shorthand property defines the logical inline start and end padding of an element, which maps to physical padding properties depending on the element's writing mode, directionality, and text orientation.

Try it

```
padding-inline: 5% 10%;
writing-mode: horizontal-tb;
```

```
padding-inline: 15px 40px;
writing-mode: vertical-rl;
```

```
padding-inline: 5% 20%;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
```

```
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
  unicode-bidi: bidi-override;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `padding-inline-end`
- `padding-inline-start`

Syntax

```
/* <length> values */
padding-inline: 10px 20px; /* An absolute length */
padding-inline: 1em 2em; /* relative to the text size */
padding-inline: 10px; /* sets both start and end values */

/* <percentage> values */
padding-inline: 5% 2%; /* relative to the nearest block container's width */

/* Global values */
padding-inline: inherit;
padding-inline: initial;
padding-inline: revert;
padding-inline: revert-layer;
padding-inline: unset;
```

The `padding-inline` property may be specified with one or two values. If one value is given, it is used as the value for both `padding-inline-start` and `padding-inline-end`. If two values are given, the first is used for `padding-inline-start` and the second for `padding-inline-end`.

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Description

The padding values specified by `padding-inline` can be equivalent to the `padding-top` and `padding-bottom` properties or the `padding-right` and `padding-left` properties, depending on the values defined for `writing-mode`, `direction`, and `text-orientation`.

Formal definition

Initial value	as each of the properties of the shorthand: • `padding-inline-start`: `0` • `padding-inline-end`: `0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`
Inherited	no
Percentages	logical-width of containing block
Computed value	as each of the properties of the shorthand: • `padding-inline-start`: as `<length>` • `padding-inline-end`: as `<length>`
Animation type	a length

Formal syntax

```
padding-inline =
  <'padding-top'>{1,2}

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline padding for vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-rl;
  padding-inline: 20px 40px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `writing-mode`, `direction`, `text-orientation`

padding-inline-end CSS property

The **`padding-inline-end`** CSS property defines the logical inline end padding of an element, which maps to a physical padding depending on the element's writing mode, directionality, and text orientation.

Try it

```
padding-inline-end: 20px;
writing-mode: horizontal-tb;
```

```
padding-inline-end: 20px;
writing-mode: vertical-rl;
```

```
padding-inline-end: 5em;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <length> values */
padding-inline-end: 10px; /* An absolute length */
padding-inline-end: 1em; /* A length relative to the text size */

/* <percentage> value */
padding-inline-end: 5%; /* A padding relative to the block container's width */

/* Global values */
padding-inline-end: inherit;
padding-inline-end: initial;
padding-inline-end: revert;
padding-inline-end: revert-layer;
padding-inline-end: unset;
```

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline-size (*width* in a horizontal language) of the containing block. Must be non-negative.

Description

The `padding-inline-end` property takes the same values as physical padding properties such as `padding-top`. However, it can be equivalent to `padding-right`, `padding-left`, `padding-top`, or `padding-bottom` depending on the values set for `writing-mode`, `direction`, and `text-orientation`.

It relates to `padding-block-start`, `padding-block-end`, and `padding-inline-start`, which define the other padding values of the element.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`
Inherited	no

Percentages	logical-width of containing block
Computed value	as `<length>`
Animation type	a length

Formal syntax

```
padding-inline-end =
  <'padding-top'>

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline end padding for vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  padding-inline-end: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `writing-mode`, `direction`, `text-orientation`

padding-inline-start CSS property

The **`padding-inline-start`** CSS property defines the logical inline start padding of an element, which maps to a physical padding depending on the element's writing mode, directionality, and text orientation.

Try it

```
padding-inline-start: 20px;
writing-mode: horizontal-tb;
```

```
padding-inline-start: 20px;
writing-mode: vertical-rl;
```

```
padding-inline-start: 5em;
writing-mode: horizontal-tb;
direction: rtl;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
  unicode-bidi: bidi-override;
}
```

Syntax

```
/* <length> values */
padding-inline-start: 10px; /* An absolute length */
padding-inline-start: 1em; /* A length relative to the text size */

/* <percentage> value */
padding-inline-start: 5%; /* A padding relative to the block container's width */

/* Global values */
padding-inline-start: inherit;
```

```
padding-inline-start: initial;
padding-inline-start: revert;
padding-inline-start: revert-layer;
padding-inline-start: unset;
```

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Description

The `padding-inline-start` property takes the same values as physical properties such as `padding-top`. However, it can be equivalent to `padding-left`, `padding-right`, `padding-top`, or `padding-bottom` depending on the values set for `writing-mode`, `direction`, and `text-orientation`.

It relates to `padding-block-start`, `padding-block-end`, and `padding-inline-end`, which define the other padding values of the element.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`
Inherited	no
Percentages	logical-width of containing block
Computed value	as `<length>`
Animation type	a length

Formal syntax

```
padding-inline-start =
  <'padding-top'>

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Logical Properties and Values Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting inline start padding for vertical text

HTML

```
<div>
  <p class="exampleText">Example text</p>
</div>
```

CSS

```
div {
  background-color: yellow;
  width: 120px;
  height: 120px;
}

.exampleText {
  writing-mode: vertical-lr;
  padding-inline-start: 20px;
  background-color: #c8c800;
}
```

Result

See also

- CSS Logical Properties and Values
- The mapped physical properties: `padding-top`, `padding-right`, `padding-bottom`, and `padding-left`
- `writing-mode`, `direction`, `text-orientation`

padding-left CSS property

The **`padding-left`** CSS property sets the width of the padding area to the left of an element.

Try it

```
padding-left: 1.5em;
```

```
padding-left: 10%;
```

```
padding-left: 20px;
```

```
padding-left: 1ch;
```

```
padding-left: 0;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
}
```

An element's padding area is the space between its content and its border.

> **Note:** The `padding` property can be used to set paddings on all four sides of an element with a single declaration.

Syntax

```
/* <length> values */
padding-left: 0.5em;
padding-left: 0;
padding-left: 2cm;

/* <percentage> value */
padding-left: 10%;

/* Global values */
padding-left: inherit;
padding-left: initial;
padding-left: revert;
padding-left: revert-layer;
padding-left: unset;
```

The `padding-left` property is specified as a single value chosen from the list below. Unlike margins, negative values are not allowed for padding.

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
padding-left =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting left padding using pixels and percentages

```
.content {
  padding-left: 5%;
}
.side-box {
  padding-left: 10px;
}
```

See also

- `padding-top`, `padding-right`, and `padding-bottom`
- `padding` shorthand
- `padding-block-start`, `padding-block-end`, `padding-inline-start`, and `padding-inline-end`
- `padding-block` and `padding-inline` shorthands
- Introduction to the CSS basic box model
- CSS box model module

padding-right CSS property

The **`padding-right`** CSS property sets the width of the padding area on the right of an element.

Try it

```
padding-right: 1.5em;
```

```
padding-right: 10%;
```

```
padding-right: 20px;
```

```
padding-right: 1ch;
```

```
padding-right: 0;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
}
```

An element's padding area is the space between its content and its border.

> **Note:** The `padding` property can be used to set paddings on all four sides of an element with a single declaration.

Syntax

```
/* <length> values */
padding-right: 0.5em;
padding-right: 0;
padding-right: 2cm;

/* <percentage> value */
padding-right: 10%;

/* Global values */
padding-right: inherit;
padding-right: initial;
padding-right: revert;
padding-right: revert-layer;
padding-right: unset;
```

The `padding-right` property is specified as a single value chosen from the list below. Unlike margins, negative values are not allowed for padding.

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
padding-right =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting right padding using pixels and percentages

```
.content {
  padding-right: 5%;
}
.side-box {
  padding-right: 10px;
}
```

See also

- `padding-top`, `padding-bottom`, and `padding-left`
- `padding` shorthand
- `padding-block-start`, `padding-block-end`, `padding-inline-start`, and `padding-inline-end`
- `padding-block` and `padding-inline` shorthands
- Introduction to the CSS box model guide
- CSS box model module

padding-top CSS property

The **`padding-top`** CSS property sets the height of the padding area on the top of an element.

Try it

```
padding-top: 1em;
```

```
padding-top: 10%;
```

```
padding-top: 20px;
```

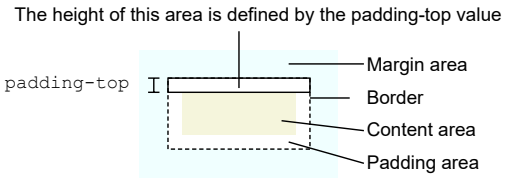
```
padding-top: 1ch;
```

```
padding-top: 0;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <div class="box">
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
    </div>
  </div>
</section>
```

```
#example-element {
  border: 10px solid #ffc129;
  overflow: hidden;
  text-align: left;
}

.box {
  border: dashed 1px;
}
```

An element's padding area is the space between its content and its border.



> **Note:** The `padding` property can be used to set paddings on all four sides of an element with a single declaration.

Syntax

```
/* <length> values */
padding-top: 0.5em;
padding-top: 0;
padding-top: 2cm;

/* <percentage> value */
padding-top: 10%;

/* Global values */
padding-top: inherit;
padding-top: initial;
padding-top: revert;
padding-top: revert-layer;
padding-top: unset;
```

The `padding-top` property is specified as a single value chosen from the list below. Unlike margins, negative values are not allowed for padding.

Values

`<length>`

The size of the padding as a fixed value. Must be non-negative.

`<percentage>`

The size of the padding as a percentage, relative to the inline size (*width* in a horizontal language, defined by `writing-mode`) of the containing block. Must be non-negative.

Formal definition

Initial value	`0`
Applies to	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Formal syntax

```
padding-top =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting top padding using pixels and percentages

```
.content {
  padding-top: 5%;
}
.side-box {
  padding-top: 10px;
}
```

See also

- `padding-right`, `padding-bottom`, `padding-left`
- `padding` shorthand
- `padding-block-start`, `padding-block-end`, `padding-inline-start`, and `padding-inline-end`
- `padding-block` and `padding-inline` shorthands
- Introduction to the CSS box model guide
- CSS box model module

page CSS property

The **page** CSS property is used to specify the named page, a specific type of page defined by the `@page` at-rule.

If there are multiple selectors that are using a named page consecutively then a forced page break using `break-after` may be needed.

Syntax

```
/* set a named page */
page: exampleName;
page: chapterIntro;

/* Use ancestors named page */
page: auto; /* default value */

/* Global values */
page: inherit;
page: initial;
page: revert;
page: revert-layer;
page: unset;
```

Values

`auto`

Default value. Use the value of the nearest ancestor with a non-`auto` value. If no ancestor has a named page value set, the used value for auto is the empty string.

`<custom-ident>`

Case-sensitive name defined in a `@page` at-rule.

Formal definition

Initial value	`auto`
Applies to	block-level elements in the normal flow of the root element. User agents may also apply it to other elements like `table-row` elements.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
page =
  auto           |
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Paged Media Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Named page example

In this example, there are two parts to this HTML; print controls and the content to be printed. The print controls allow the user to select how the `sections` in the `article` will be printed.

```
<!-- print options in a fieldset -->
<fieldset id="printStyle">
  <legend>How would you like to print</legend>
  <label for="single">
    <input type="radio" id="single" name="type" value="single" checked />
    No Pages
  </label>
  <label for="grouped">
    <input type="radio" id="grouped" name="type" value="grouped" />Pages with
    Grouped Chapters
  </label>
  <label for="paged">
    <input type="radio" id="paged" name="type" value="paged" />
    Chapters Paged
  </label>
  <button id="print">Print</button>
</fieldset>

<!-- Content to be printed -->
<article id="print-area" data-print="single">
  <section id="toc">
    <h2>Table of contents</h2>
    <ul>
      <li>Foreword</li>
      <li>Introduction</li>
      <li>Chapter One - named pages</li>
      <li>Chapter Two - page orientation</li>
      <li>Chapter Three - page margins</li>
      <li>Conclusion</li>
    </ul>
  </section>
  <section id="foreword">
    <h2>Foreword</h2>
    <p>
      This book is all about how the CSS <code>@page</code> at-rule can help
      with printing HTML books.
    </p>
  </section>
  <section id="introduction">
    <h2>Introduction</h2>
    <p>
      This book is a concept to show how an <em>HTML</em> document can easily be
      printed out in pages.
    </p>
  </section>
  <section id="chapter1" class="chapter">
    <h2>Named pages</h2>
    <p>Lorem ipsum</p>
  </section>
  <section id="chapter2" class="chapter">
    <h2>Page Orientation</h2>
    <p>Lorem ipsum</p>
  </section>
  <section id="chapter3" class="chapter">
    <h2>Page Margins</h2>
    <p>There are 16 page margins that can be set:</p>
    <ul>
      <li>@top-left-corner</li>
      <li>@top-left</li>
      <li>@top-center</li>
      <li>@top-right</li>
      <li>@top-right-corner</li>
      <li>@left-top</li>
      <li>@left-middle</li>
      <li>@left-bottom</li>
      <li>@right-top</li>
      <li>@right-middle</li>
      <li>@right-bottom</li>
      <li>@bottom-left-corner</li>
      <li>@bottom-left</li>
      <li>@bottom-center</li>
      <li>@bottom-right</li>
      <li>@bottom-right-corner</li>
    </ul>
    <p>They can be used to show what appears in these parts of the margin</p>
  </section>
  <section id="conclusion">
    <h2>Conclusion</h2>
    <p>Now go ahead and write books.</p>
  </section>
</article>
```

The first part of the CSS sets up the **named** pages, these include the size and orientation and also some content to go in the `@top-center` margin of the printed pages.

```
@page toc {
  size: a4 portrait;
  @top-center {
    content: "Table of contents";
  }
}

@page foreword {
  size: a4 portrait;
  @top-center {
    content: "Foreword";
  }
```

```
}

@page introduction {
  size: a4 portrait;
  @top-center {
    content: "Introduction";
  }
}

@page conclusion {
  size: a4 portrait;
  @top-center {
    content: "Conclusion";
  }
}

@page chapter {
  size: a4 landscape;
  @top-center {
    content: "Chapter";
  }
}
```

```
fieldset {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  gap: 1rem;
  width: fit-content;
}
body {
  font: 1.1em sans-serif;
}
```

The next part of the CSS uses attribute selectors to apply the print dimensions, orientation, and margins defined in the named `@page` rules defined in the previous CSS section to elements using the `page` property. The sections with `class="chapter"` are concurrent and appear as one page. The `break-after: page;` is used to split them up, which splits each chapter into a separately printed page.

```
@media print {
  fieldset {
    display: none;
  }
  section {
    font-size: 2rem;
    font-family: "Roboto", sans-serif;
  }
  .chapter {
    border: tomato 2px solid;
  }
  [data-print="grouped"] > #toc,
  [data-print="paged"] > #toc {
    page: toc;
    font-family: "Courier New";
  }
  [data-print="grouped"] > #foreword,
  [data-print="paged"] > #foreword {
    page: foreword;
    font-family: "Courier New";
  }
  [data-print="grouped"] > #introduction,
  [data-print="paged"] > #introduction {
    page: introduction;
    font-family: "Courier New";
  }
  [data-print="grouped"] > #conclusion,
  [data-print="paged"] > #conclusion {
    page: conclusion;
    font-family: "Courier New";
  }
  [data-print="grouped"] > .chapter,
  [data-print="paged"] > .chapter {
    page: chapter;
  }
  [data-print="paged"] > .chapter {
    border: none;
    break-after: page;
  }
  .chapter > ul {
    columns: 2;
  }
}
```

The JavaScript updates the value of the `data-print` attribute, which is the attribute on which the named page is applied, when you select a different printing option:

```
const printArea = document.querySelector("#print-area");
const printButton = document.querySelector("#print");
const printOption = document.querySelector("#printStyle");

printOption.addEventListener("change", (event) => {
  if (event.target.value === "single") {
    printArea.dataset.print = "single";
  } else if (event.target.value === "grouped") {
    printArea.dataset.print = "grouped";
  } else {
    printArea.dataset.print = "paged";
  }
});

printButton.addEventListener("click", () => {
  window.print();
});
```

What is printed, and what is shown in the print preview dialog, will change depending on which print style radio button is selected:

page-break-after CSS property

> **Warning:** This property has been replaced by the `break-after` property.

The **`page-break-after`** CSS property adjusts page breaks *after* the current element.

Try it

```
page-break-after: auto;
```

```
page-break-after: always;
```

```
<div>
  <p>
    The effect of this property can be noticed when the document is being
    printed or a preview of a print is displayed.
  </p>
  <button id="print-btn">Show Print Preview</button>
  <div class="box-container">
    <div class="box">Content before the property</div>
    <div class="box" id="example-element">Content with 'page-break-after'</div>
    <div class="box">Content after the property</div>
  </div>
</div>
```

```
.box {
  border: solid #5b6dcd 5px;
  background-color: #5b6dcd;
  margin: 10px 0;
  padding: 5px;
}

#example-element {
  border: solid 5px #ffc129;
  background-color: #ffc129;
  color: black;
}
```

```
const btn = document.getElementById("print-btn");

btn.addEventListener("click", () => {
  window.print();
});
```

Syntax

```
/* Keyword values */
page-break-after: auto;
page-break-after: always;
page-break-after: avoid;
page-break-after: left;
page-break-after: right;
page-break-after: recto;
page-break-after: verso;

/* Global values */
page-break-after: inherit;
page-break-after: initial;
page-break-after: revert;
page-break-after: revert-layer;
page-break-after: unset;
```

This property applies to block elements that generate a box. It won't apply on an empty `<div>` that won't generate a box.

Values

`auto`

Initial value. Automatic page breaks (neither forced nor forbidden).

`always`

Always force page breaks after the element.

`avoid`

Avoid page breaks after the element.

`left`

Force page breaks after the element so that the next page is formatted as a left page. It's the page placed on the left side of the spine of the book or the back side of the page in duplex printing.

`right`

Force page breaks after the element so that the next page is formatted as a right page. It's the page placed on the right side of the spine of the book or the front side of the page in duplex printing.

`recto`

If pages progress left-to-right, then this acts like `right`. If pages progress right-to-left, then this acts like `left`.

`verso`

If pages progress left-to-right, then this acts like `left`. If pages progress right-to-left, then this acts like `right`.

Page break aliases

The `page-break-after` property is now a legacy property, replaced by `break-after`.

For compatibility reasons, `page-break-after` should be treated by browsers as an alias of `break-after`. This ensures that sites using `page-break-after` continue to work as designed. A subset of values should be aliased as follows:

page-break-after	break-after
auto	auto
left	left
right	right
avoid	avoid
always	page

Formal definition

Initial value	auto
Applies to	block-level elements in the normal flow of the root element. User agents may also apply it to other elements like `table-row` elements.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
page-break-after =
  auto     |
  always   |
  avoid    |
  left     |
  right    |
  inherit
```

This syntax reflects the latest standard as per Cascading Style Sheets Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a page break after footnotes

```
/* move to a new page after footnotes */
div.footnotes {
  page-break-after: always;
}
```

See also

- `break-before`, `break-after`, `break-inside`
- `page-break-before`, `page-break-inside`
- `orphans`, `widows`

page-break-before CSS property

> **Warning:** This property has been replaced by the `break-before` property.

The **`page-break-before`** CSS property adjusts page breaks *before* the current element.

This property applies to block elements that generate a box. It won't apply on an empty `<div>` that won't generate a box.

Try it

```
page-break-before: auto;
```

```
page-break-before: always;
```

```
<div>
  <p>
    The effect of this property can be noticed when the document is being
    printed or a preview of a print is displayed.
  </p>
  <button id="print-btn">Show Print Preview</button>
  <div class="box-container">
    <div class="box">Content before the property</div>
    <div class="box" id="example-element">Content with 'page-break-before'</div>
    <div class="box">Content after the property</div>
  </div>
</div>
```

```
.box {
  border: solid #5b6dcd 5px;
  background-color: #5b6dcd;
  margin: 10px 0;
  padding: 5px;
}

#example-element {
  border: solid 5px #ffc129;
  background-color: #ffc129;
  color: black;
}
```

```
const btn = document.getElementById("print-btn");

btn.addEventListener("click", () => {
  window.print();
});
```

Syntax

```
/* Keyword values */
page-break-before: auto;
page-break-before: always;
page-break-before: avoid;
page-break-before: left;
page-break-before: right;
page-break-before: recto;
page-break-before: verso;

/* Global values */
page-break-before: inherit;
page-break-before: initial;
page-break-before: revert;
page-break-before: revert-layer;
page-break-before: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Initial value. Automatic page breaks (neither forced nor forbidden).

`always`

Always force page breaks before the element.

`avoid`

Avoid page breaks before the element.

`left`

Force page breaks before the element so that the next page is formatted as a left page. It's the page placed on the left side of the spine of the book or the back side of the page in duplex printing.

`right`

Force page breaks before the element so that the next page is formatted as a right page. It's the page placed on the right side of the spine of the book or the front side of the page in duplex printing.

`recto`

If pages progress left-to-right, then this acts like `right`. If pages progress right-to-left, then this acts like `left`.

`verso`

If pages progress left-to-right, then this acts like `left`. If pages progress right-to-left, then this acts like `right`.

Page break aliases

The `page-break-before` property is now a legacy property, replaced by `break-before`.

For compatibility reasons, `page-break-before` should be treated by browsers as an alias of `break-before`. This ensures that sites using `page-break-before` continue to work as designed. A subset of values should be aliased as follows:

page-break-before	break-before
`auto`	`auto`
`left`	`left`
`right`	`right`
`avoid`	`avoid`
`always`	`page`

Formal definition

Initial value	`auto`
Applies to	block-level elements in the normal flow of the root element. User agents may also apply it to other elements like `table-row` elements.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
page-break-before =
  auto    |
  always  |
  avoid   |
  left    |
  right   |
  inherit
```

This syntax reflects the latest standard as per Cascading Style Sheets Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Avoid a page break before an element

```
/* Avoid page break before div elements of class note */
div.note {
  page-break-before: avoid;
}
```

See also

- `break-before`, `break-after`, `break-inside`

- page-break-after, page-break-inside
- orphans, widows

page-break-inside CSS property

> **Warning:** This property has been replaced by the break-inside property.

The **page-break-inside** CSS property adjusts page breaks *inside* the current element.

Try it

```
page-break-inside: auto;
```

```
page-break-inside: avoid;
```

```
<div>
  <p>
    The effect of this property can be noticed when the document is being
    printed or a preview of a print is displayed.
  </p>
  <button id="print-btn">Show Print Preview</button>
  <div class="box-container">
    <div class="box">Content before the property</div>
    <div class="box" id="example-element">Content with 'page-break-inside'</div>
    <div class="box">Content after the property</div>
  </div>
</div>
```

```
.box {
  border: solid #5b6dcd 5px;
  background-color: #5b6dcd;
  margin: 10px 0;
  padding: 5px;
}

#example-element {
  border: solid 5px #ffc129;
  background-color: #ffc129;
  color: black;
}

@media print {
  #example-element {
    height: 25cm;
  }
}
```

```
const btn = document.getElementById("print-btn");

btn.addEventListener("click", () => {
  window.print();
});
```

Syntax

```
/* Keyword values */
page-break-inside: auto;
page-break-inside: avoid;

/* Global values */
page-break-inside: inherit;
page-break-inside: initial;
page-break-inside: revert;
page-break-inside: revert-layer;
page-break-inside: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

Initial value. Automatic page breaks (neither forced nor forbidden).

`avoid`

Avoid page breaks inside the element.

Page break aliases

The `page-break-inside` property is now a legacy property, replaced by `break-inside`.

For compatibility reasons, `page-break-inside` should be treated by browsers as an alias of `break-inside`. This ensures that sites using `page-break-inside` continue to work as designed. A subset of values should be aliased as follows:

page-break-inside	break-inside
auto	auto
avoid	avoid

Formal definition

Initial value	auto
Applies to	block-level elements in the normal flow of the root element. User agents may also apply it to other elements like `table-row` elements.
Inherited	no

Computed value	as specified
Animation type	discrete

Formal syntax

```
page-break-inside =
  avoid    |
  auto     |
  inherit
```

This syntax reflects the latest standard as per Cascading Style Sheets Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Avoiding page breaks inside elements

HTML

```
<div class="page">
  <p>This is the first paragraph.</p>
  <section class="list">
    <span>A list</span>
    <ol>
      <li>one</li>
      <!-- <li>two</li> -->
    </ol>
  </section>
  <ul>
    <li>one</li>
    <!-- <li>two</li> -->
  </ul>
  <p>This is the second paragraph.</p>
  <p>This is the third paragraph, it contains more text.</p>
  <p>
    This is the fourth paragraph. It has a little bit more text than the third
    one.
  </p>
</div>
```

CSS

```
.page {
  background-color: #8cffa0;
  height: 90px;
  width: 200px;
  columns: 1;
  column-width: 100px;
}

.list,
ol,
ul,
p {
  break-inside: avoid;
}

p {
  background-color: #8ca0ff;
}

ol,
ul,
.list {
  margin: 0.5em 0;
  display: block;
  background-color: orange;
}

p:first-child {
  margin-top: 0;
}
```

Result

See also

- break-before, break-after, break-inside
- page-break-after, page-break-before
- orphans, widows

paint-order CSS property

The **`paint-order`** CSS property lets you control the order in which the fill and stroke (and painting markers) of text content and shapes are drawn.

Syntax

```
/* Normal */
paint-order: normal;

/* Single values */
paint-order: stroke; /* draw the stroke first, then fill and markers */
paint-order: markers; /* draw the markers first, then fill and stroke */

/* Multiple values */
paint-order: stroke fill; /* draw the stroke first, then the fill, then the markers */
paint-order: markers stroke fill; /* draw markers, then stroke, then fill */

/* Global values */
paint-order: inherit;
paint-order: initial;
```

```
paint-order: revert;
paint-order: revert-layer;
paint-order: unset;
```

If no value is specified, the default paint order is `fill`, `stroke`, `markers`.

When one value is specified, that one is painted first, followed by the other two in their default order relative to one another. When two values are specified, they will be painted in the order they are specified in, followed by the unspecified one.

> **Note:** In the case of this property, markers are only appropriate when drawing SVG shapes involving the use of the `marker-*` properties (e.g., `marker-start`) and `<marker>` element. They do not apply to HTML text, so in that case, you can only determine the order of `stroke` and `fill`.

Values

`normal`

Paint the different items in normal paint order.

`stroke`, `fill`, `markers`

Specify some or all of these values in the order you want them to be painted in.

Formal definition

Initial value	`normal`
Applies to	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
paint-order =
  normal                        |
  [ fill || stroke || markers ]
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Reversing the paint order of stroke and fill

SVG

```
<svg xmlns="http://www.w3.org/2000/svg" width="400" height="200">
  <text x="10" y="75">stroke in front</text>
  <text x="10" y="150" class="stroke-behind">stroke behind</text>
</svg>
```

CSS

```
text {
  font-family: sans-serif;
  font-size: 50px;
  font-weight: bold;
  fill: black;
  stroke: red;
  stroke-width: 4px;
}

.stroke-behind {
  paint-order: stroke fill;
}
```

Result

Reversing the paint order of stroke and fill using HTML

To control the fill and stroke order in HTML, you can use the `-webkit-text-stroke-color` and `-webkit-text-stroke-width` CSS properties.

HTML

```
<div>stroke in front</div>
<div class="stroke-behind">stroke behind</div>
```

CSS

```
div {
  font-family: sans-serif;
  font-size: 50px;
  font-weight: bold;
  fill: black;
  padding-top: 10px;
  padding-bottom: 10px;
  -webkit-text-stroke-color: red;
  -webkit-text-stroke-width: 4px;
}

.stroke-behind {
  paint-order: stroke fill;
}
```

Result

See also

- CSS Tricks: paint-order

path-length CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`path-length`** CSS property specifies a total path length, in user units. All path computations are then scaled using the ratio `path-length` / *(computed value of path length)* — this includes text paths, animation paths, and various stroke operations.

The `path-length` property only applies to `<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements nested in an `<svg>`.

> **Note:** If present, the `path-length` CSS property overrides an SVG element's `pathLength` attribute. This property doesn't apply to SVG, HTML, or pseudo-elements other than those listed above.

Syntax

```
/* Keyword value */
path-length: none;

/* <length> values */
path-length: 0;
path-length: 70px;
path-length: 500px;

/* Global values */
path-length: inherit;
path-length: initial;
path-length: revert;
path-length: revert-layer;
path-length: unset;
```

Values

`none`

No author path length is specified and the user agent's own computed path length is used for all path-related calculations.

`<length>`

A non-negative `<length>` representing an author-defined total path length.

Formal definition

Initial value	none
Applies to	The SVG `<path>`, `<circle>`, `<ellipse>`, `<line>`, `<polygon>`, `<polyline>`, and `<rect>` elements.
Inherited	no
Computed value	The keyword `none` or an absolute length.
Animation type	by computed value

Formal syntax

```
path-length =
  none                |
  <length> [ 0 , ∞ ]
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example defines a path and shows how to apply a path length to it using the `path-length` CSS property.

SVG

Our SVG defines a single curved `<path>` element with a colored `stroke`. It includes a `stroke-dasharray` attribute that defines a regular dashed pattern for the stroke.

```
<svg viewBox="0 0 600 200">
  <path
    d="M 30 100 C 150 20, 250 180, 380 100 S 520 20, 570 100"
    fill="none"
    stroke="#D85A30"
    stroke-width="4"
    stroke-dasharray="24 24"></path>
</svg>
```

CSS

We set a `path-length` value on the `<path>`:

```
path {
  path-length: 500px;
}
```

Results

Setting a large `path-length` value results in the dashes becoming smaller and more frequent.

Animating <code>path-length</code>

One major advantage of making `path-length` available as a CSS property is that you can apply standard CSS functionality such as animations and transitions to it. This example builds on the previous one, showing how to animate a `path-length` with a CSS animation.

HTML and SVG

This example includes the same SVG `<path>` as the previous one. In addition, it includes an `<input type="range">` element that can be used to change the value of `path-length` applied to the `<path>` at runtime. We also include an `<output>` element to display the current slider value.

```
<div>
  <label for="path-slider">Adjust path-length</label>
  <input type="range" id="path-slider" min="0" max="800" value="200" />
  <output>200</output>
</div>
```

CSS

On the `:root` element, we define a CSS custom property called `--path-length` and give it an initial value of `200px`. We then set the `<path>` element's `path-length` value to the `--path-length` property, and set an `animation` on it that runs an infinite number of times and alternates between forwards and backwards.

```
:root {
  --path-length: 200px;
}

path {
  path-length: var(--path-length);
  animation: path-length-anim 2s alternate infinite ease-in-out;
}
```

```
div {
  position: fixed;
  bottom: 0;
  left: 0;
  display: flex;
  align-items: center;
}
```

Next, we define the `@keyframes` block for the animation — it animates the `path-length` property between the `--path-length` value and the `--path-length` value multiplied by `1.5`.

```
@keyframes path-length-anim {
  from {
    path-length: var(--path-length);
  }

  to {
    path-length: calc(var(--path-length) * 1.5);
  }
}
```

JavaScript

We start our script by grabbing references to the `<input type="range">`, `<output>`, and `:root` elements.

```
const slider = document.querySelector("input");
const output = document.querySelector("output");
const rootElem = document.querySelector(":root");
```

Next, we add an `input` event handler to the range slider so that, when its value is changed, the `<output>` element's `textContent` and the value of the `--path-length` custom property are set to equal the slider's new value.

```
slider.addEventListener("input", () => {
  output.textContent = `${slider.value}px`;
  rootElem.style.setProperty("--path-length", `${slider.value}px`);
});
```

Results

Adjust the slider, and note how larger values result in a smaller dash size.

See also

- SVG `pathLength` attribute

perspective CSS property

The **`perspective`** CSS property determines the distance between the z=0 plane and the user in order to give a 3D-positioned element some perspective.

Try it

```
perspective: none;
```

```
perspective: 800px;
```

```
perspective: 23rem;
```

```
perspective: 5.5cm;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
```

```
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Syntax

```
/* Keyword value */
perspective: none;

/* <length> values */
perspective: 20px;
perspective: 3.5em;

/* Global values */
perspective: inherit;
perspective: initial;
perspective: revert;
perspective: revert-layer;
perspective: unset;
```

Values

`none`

Indicates that no perspective transform is to be applied.

`<length>`

A `<length>` giving the distance from the user to the z=0 plane. It is used to apply a perspective transform to the children of the element. Negative values are syntax errors. If the value is smaller than `1px`, it is clamped to `1px`.

Description

Each 3D element with z>0 becomes larger; each 3D-element with z<0 becomes smaller. The strength of the effect is determined by the value of this property. Large values of `perspective` cause a small transformation; small values of `perspective` cause a large transformation.

The parts of the 3D elements that are behind the user — i.e., their z-axis coordinates are greater than the value of the `perspective` CSS property — are not drawn.

The *vanishing point* is by default placed at the center of the element, but its position can be changed using the `perspective-origin` property.

Using this property with a value other than `none` creates a new stacking context. Also, in that case, the object will act as a containing block for `position: fixed` elements that it contains.

Formal definition

Initial value	none
Applies to	transformable elements

Inherited	no
Computed value	the absolute length or none
Animation type	a length
Creates stacking context	yes

Formal syntax

```
perspective =
  none            |
  <length [0,∞]>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting perspective

An example showing how a cube varies if the `perspective` is set at different positions is given in Using CSS transforms > Setting perspective.

See also

- Using CSS Transforms

perspective-origin CSS property

The **`perspective-origin`** CSS property determines the position at which the viewer is looking. It is used as the *vanishing point* by the `perspective` property.

The **`perspective-origin`** and `perspective` properties are attached to the parent of a child transformed in 3-dimensional space, unlike the `perspective()` transform function which is placed on the element being transformed.

Try it

```
perspective-origin: center;
```

```
perspective-origin: top;
```

```
perspective-origin: bottom right;
```

```
perspective-origin: -170%;
```

```
perspective-origin: 500% 200%;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 550px;
}

#example-element {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  perspective: 250px;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}
```

```
.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Syntax

```
/* One-value syntax */
perspective-origin: x-position;

/* Two-value syntax */
perspective-origin: x-position y-position;

/* When both x-position and y-position are keywords,
   the following is also valid */
perspective-origin: y-position x-position;

/* Global values */
perspective-origin: inherit;
perspective-origin: initial;
perspective-origin: revert;
perspective-origin: revert-layer;
perspective-origin: unset;
```

Values

x-position

Indicates the position of the abscissa of the *vanishing point*. It can have one of the following values:

- `<length-percentage>` indicating the position as an absolute length value or relative to the width of the element. The value may be negative.
- `left`, a keyword being a shortcut for the `0` length value.
- `center`, a keyword being a shortcut for the `50%` percentage value.
- `right`, a keyword being a shortcut for the `100%` percentage value.

y-position

Indicates the position of the ordinate of the *vanishing point*. It can have one of the following values:

- `<length-percentage>` indicating the position as an absolute length value or relative to the height of the element. The value may be negative.
- `top`, a keyword being a shortcut for the `0` length value.
- `center`, a keyword being a shortcut for the `50%` percentage value.
- `bottom`, a keyword being a shortcut for the `100%` percentage value.

Formal definition

Initial value	`50% 50%`
Applies to	transformable elements
Inherited	no
Percentages	refer to the size of bounding box
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	simple list of length, percentage, or calc

Formal syntax

```
perspective-origin =
  <position>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing the perspective origin

An example showing how to change `perspective-origin` is given in Using CSS transforms > Changing the perspective origin.

See also

- Using CSS Transforms
- `transform-style`
- `<transform-function>`
- `perspective`
- `transform: perspective()` function

place-content CSS property

The **`place-content`** CSS shorthand property allows you to align content along both the block and inline directions at once (i.e., the `align-content` and `justify-content` properties) in a relevant layout system such as Grid or Flexbox.

Try it

```
place-content: end space-between;
```

```
place-content: space-around start;
```

```
place-content: start space-evenly;
```

```
place-content: end center;
```

```
place-content: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 60px 60px;
  grid-auto-rows: 40px;
  height: 180px;
  width: 220px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `align-content`
- `justify-content`

Syntax

```
/* Positional alignment */
/* align-content does not take left and right values */
place-content: center start;
place-content: start center;
place-content: end left;
place-content: flex-start center;
place-content: flex-end center;

/* Baseline alignment */
/* justify-content does not take baseline values */
place-content: baseline center;
place-content: first baseline space-evenly;
place-content: last baseline right;

/* Distributed alignment */
place-content: space-between space-evenly;
place-content: space-around space-evenly;
place-content: space-evenly stretch;
place-content: stretch space-evenly;

/* Global values */
place-content: inherit;
place-content: initial;
place-content: revert;
place-content: revert-layer;
place-content: unset;
```

The first value is the `align-content` property value, the second the `justify-content` one.

> **Note:** If the second value is not present, the first value is used for both, provided it is a valid value for both. If it is invalid for one or the other, the whole value will be invalid.

Values

`start`

The items are packed flush to each other toward the start edge of the alignment container in the appropriate axis.

`end`

The items are packed flush to each other toward the end edge of the alignment container in the appropriate axis.

`flex-start`

The items are packed flush to each other toward the edge of the alignment container depending on the flex container's main-start or cross-start side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `start`.

`flex-end`

The items are packed flush to each other toward the edge of the alignment container depending on the flex container's main-end or cross-end side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `end`.

`center`

The items are packed flush to each other toward the center of the alignment container.

`left`

The items are packed flush to each other toward the left edge of the alignment container. If the property's axis is not parallel with the inline axis, this value behaves like `start`.

`right`

The items are packed flush to each other toward the right edge of the alignment container in the appropriate axis. If the property's axis is not parallel with the inline axis, this value behaves like `start`.

`space-between`

The items are evenly distributed within the alignment container. The spacing between each pair of adjacent items is the same. The first item is flush with the main-start edge, and the last item is flush with the main-end edge.

`baseline`, `first baseline`, `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group. The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`space-around`

The items are evenly distributed within the alignment container. The spacing between each pair of adjacent items is the same. The empty space before the first and after the last item equals half of the space between each pair of adjacent items.

`space-evenly`

The items are evenly distributed within the alignment container. The spacing between each pair of adjacent items, the main-start edge and the first item, and the main-end edge and the last item, are all exactly the same.

`stretch`

If the combined size of the items is less than the size of the alignment container, any `auto`-sized items have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size exactly fills the alignment container

`safe`

Used alongside an alignment keyword. If the chosen keyword means that the item overflows the alignment container causing data loss, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Used alongside an alignment keyword. Regardless of the relative sizes of the item and alignment container, and regardless of whether overflow which causes data loss might happen, the given alignment value is honored.

Formal definition

Initial value	as each of the properties of the shorthand: • `align-content`: `normal` • `justify-content`: `normal`
Applies to	multi-line flex containers
Inherited	no
Computed value	as each of the properties of the shorthand: • `align-content`: as specified • `justify-content`: as specified
Animation type	discrete

Formal syntax

```
place-content =
  <'align-content'> <'justify-content'>?

<align-content> =
  normal                                |
  <baseline-position>                   |
  <content-distribution>                |
  <overflow-position>? <content-position>

<justify-content> =
  normal                                          |
  <content-distribution>                          |
```

```
  <overflow-position>? [ <content-position> | left | right ]

<baseline-position> =
  [ first | last ]?  &&
  baseline

<content-distribution> =
  space-between  |
  space-around   |
  space-evenly   |
  stretch

<overflow-position> =
  unsafe  |
  safe

<content-position> =
  center      |
  start       |
  end         |
  flex-start  |
  flex-end
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Placing content in a flex container

HTML

```
<div id="container">
  <div class="small">Lorem</div>
  <div class="small">Lorem<br />ipsum</div>
  <div class="large">Lorem</div>
  <div class="large">Lorem<br />ipsum</div>
  <div class="large"></div>
  <div class="large"></div>
</div>
```

```
<code>writing-mode:</code
><select id="writingMode">
  <option value="horizontal-tb" selected>horizontal-tb</option>
  <option value="vertical-rl">vertical-rl</option>
  <option value="vertical-lr">vertical-lr</option>
  <option value="sideways-rl">sideways-rl</option>
  <option value="sideways-lr">sideways-lr</option></select
><code>;</code><br />

<code>direction:</code
><select id="direction">
  <option value="ltr" selected>ltr</option>
  <option value="rtl">rtl</option></select
><code>;</code><br />

<code>place-content:</code
><select id="alignContentAlignment">
  <option value="normal">normal</option>
  <option value="first baseline">first baseline</option>
  <option value="last baseline">last baseline</option>
  <option value="baseline">baseline</option>
  <option value="space-between">space-between</option>
  <option value="space-around">space-around</option>
  <option value="space-evenly" selected>space-evenly</option>
  <option value="stretch">stretch</option>
  <option value="center">center</option>
  <option value="start">start</option>
  <option value="end">end</option>
  <option value="flex-start">flex-start</option>
  <option value="flex-end">flex-end</option>
  <option value="safe">safe</option>
  <option value="unsafe">unsafe</option>
</select>
<select id="justifyContentAlignment">
  <option value="normal">normal</option>
  <option value="space-between">space-between</option>
  <option value="space-around">space-around</option>
  <option value="space-evenly">space-evenly</option>
  <option value="stretch">stretch</option>
  <option value="center" selected>center</option>
  <option value="start">start</option>
  <option value="end">end</option>
  <option value="flex-start">flex-start</option>
  <option value="flex-end">flex-end</option>
  <option value="left">left</option>
  <option value="right">right</option>
  <option value="safe">safe</option>
  <option value="unsafe">unsafe</option></select
><code>;</code>
```

```
function update() {
  document.getElementById("container").style.placeContent =
    `${document.getElementById("alignContentAlignment").value} ` +
    `${document.getElementById("justifyContentAlignment").value}`;
}

const alignContentAlignment = document.getElementById("alignContentAlignment");
alignContentAlignment.addEventListener("change", update);

const justifyContentAlignment = document.getElementById(
  "justifyContentAlignment",
);
justifyContentAlignment.addEventListener("change", update);
```

```
const writingM = document.getElementById("writingMode");
writingM.addEventListener("change", (evt) => {
  document.getElementById("container").style.writingMode = evt.target.value;
});

const direction = document.getElementById("direction");
direction.addEventListener("change", (evt) => {
  document.getElementById("container").style.direction = evt.target.value;
});
```

CSS

```
#container {
  display: flex;
  height: 240px;
  width: 240px;
  flex-wrap: wrap;
  background-color: #8c8c8c;
  writing-mode: horizontal-tb; /* Can be changed in the live sample */
  direction: ltr; /* Can be changed in the live sample */
  place-content: flex-end center; /* Can be changed in the live sample */
}

div > div {
  border: 2px solid #8c8c8c;
  width: 50px;
  background-color: #a0c8ff;
}

.small {
  font-size: 12px;
  height: 40px;
}

.large {
  font-size: 14px;
  height: 50px;
}
```

Result

See also

- `align-content`
- `justify-content`
- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment module

place-items CSS property

The **`place-items`** CSS shorthand property aligns items along both block and inline directions at the same time.

Try it

```
place-items: center stretch;
```

```
place-items: center start;
```

```
place-items: start end;
```

```
place-items: end center;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
  width: 220px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `align-items`
- `justify-items`

Syntax

```
/* Positional alignment */
place-items: center;
place-items: normal start;
place-items: center normal;
place-items: start legacy;
place-items: end normal;
place-items: self-start legacy;
place-items: self-end normal;
place-items: flex-start legacy;
place-items: flex-end normal;
place-items: anchor-center;

/* Baseline alignment */
place-items: baseline normal;
place-items: first baseline legacy;
place-items: last baseline normal;
place-items: stretch legacy;

/* Global values */
place-items: inherit;
place-items: initial;
place-items: revert;
place-items: revert-layer;
place-items: unset;
```

Values

This property is specified as one or two space-separated values of the following types, in the order listed:

`align-items`

Specifies the alignment in the block direction.

`justify-items` Optional

Specifies the alignment in the inline direction. If omitted, the `align-items` value is used.

Description

The `place-items` shorthand property sets `align-items` and `justify-items` in a single declaration, aligning items along both the block and inline directions.

If only one value is provided, the value is used to set alignment in both block and inline directions. If two values are provided, the first value is the `align-items` value, setting the alignment in the block direction, and the second value is the `justify-items` value, setting alignment in the inline direction.

Formal definition

Initial value	as each of the properties of the shorthand: • `align-items`: `normal` • `justify-items`: `legacy`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `align-items`: as specified • `justify-items`: as specified
Animation type	discrete

Formal syntax

```
place-items =
  <'align-items'> <'justify-items'>?

<align-items> =
  normal                                |
  stretch                               |
  <baseline-position>                   |
  <overflow-position>? <self-position>

<justify-items> =
  normal                                            |
  stretch                                           |
  <baseline-position>                               |
  <overflow-position>? [ <self-position> | left | right ]  |
  legacy                                            |
  legacy && [ left | right | center ]

<baseline-position> =
  [ first | last ]?  &&
  baseline

<overflow-position> =
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Placing items in a flex container

In flexbox `justify-self` or `justify-items` do not apply, as on the main axis items are treated as a group. Therefore, the second value will be ignored.

```
div > div {
  box-sizing: border-box;
  border: 2px solid #8c8c8c;
  width: 50px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#item1 {
  background-color: #8cffa0;
  min-height: 30px;
  font-size: 2em;
}

#item2 {
  background-color: #a0c8ff;
  min-height: 50px;
}

#item3 {
  background-color: #ffa08c;
  min-height: 40px;
}

#item4 {
  background-color: #ffff8c;
  min-height: 60px;
}

#item5 {
  background-color: #ff8cff;
  min-height: 70px;
}

#item6 {
  background-color: #8cffff;
  min-height: 50px;
}

select {
  font-size: 16px;
}

.row {
  margin-top: 10px;
}
```

```
<div id="container">
  <div id="item1">1</div>
  <div id="item2">2</div>
  <div id="item3">3</div>
</div>

<div class="row">
  <label for="values">place-items: </label>
  <select id="values">
    <option value="stretch">stretch</option>
    <option value="start">start</option>
    <option value="center">center</option>
    <option value="end">end</option>
    <option value="left">left</option>
    <option value="right">right</option>
    <option value="auto center">auto center</option>
    <option value="normal start">normal start</option>
    <option value="center normal">center normal</option>
    <option value="start auto">start auto</option>
    <option value="end normal">end normal</option>
    <option value="self-start auto">self-start auto</option>
    <option value="self-end normal">self-end normal</option>
    <option value="flex-start auto">flex-start auto</option>
    <option value="flex-end normal">flex-end normal</option>
    <option value="left auto">left auto</option>
    <option value="right normal">right normal</option>
    <option value="baseline normal">baseline normal</option>
    <option value="first baseline auto">first baseline auto</option>
    <option value="last baseline normal">last baseline normal</option>
    <option value="stretch auto">stretch auto</option>
  </select>
</div>
```

```
const values = document.getElementById("values");
const container = document.getElementById("container");

values.addEventListener("change", (evt) => {
  container.style.placeItems = evt.target.value;
});
```

CSS

```
#container {
  height: 200px;
  width: 240px;
  place-items: stretch; /* You can change this value by selecting another option in the list */
  background-color: #8c8c8c;
```

```
  display: flex;
}
```

Result

Placing items in a grid container

The following grid container has items which are smaller than the grid areas they are placed in, therefore `place-items` will move them in the block and inline dimensions.

```
div > div {
  box-sizing: border-box;
  border: 2px solid #8c8c8c;
}

#item1 {
  background-color: #8cffa0;
  min-height: 30px;
  font-size: 2em;
}

#item2 {
  background-color: #a0c8ff;
  min-height: 50px;
}

#item3 {
  background-color: #ffa08c;
  min-height: 40px;
}

#item4 {
  background-color: #ffff8c;
  min-height: 60px;
}

#item5 {
  background-color: #ff8cff;
  min-height: 70px;
}

#item6 {
  background-color: #8cffff;
  min-height: 50px;
}

select {
  font-size: 16px;
}

.row {
  margin-top: 10px;
}
```

```
<div id="grid-container">
  <div id="item1">1</div>
  <div id="item2">2</div>
  <div id="item3">3</div>
  <div id="item4">4</div>
  <div id="item5">5</div>
</div>

<div class="row">
  <label for="grid-values">place-items: </label>
  <select id="grid-values">
    <option value="stretch">stretch</option>
    <option value="start">start</option>
    <option value="center">center</option>
    <option value="end">end</option>
    <option value="left">left</option>
    <option value="right">right</option>
    <option value="auto center">auto center</option>
    <option value="normal start">normal start</option>
    <option value="center normal">center normal</option>
    <option value="start auto">start auto</option>
    <option value="end normal">end normal</option>
    <option value="self-start auto">self-start auto</option>
    <option value="self-end normal">self-end normal</option>
    <option value="flex-start auto">flex-start auto</option>
    <option value="flex-end normal">flex-end normal</option>
    <option value="left auto">left auto</option>
    <option value="right normal">right normal</option>
    <option value="baseline normal">baseline normal</option>
    <option value="first baseline auto">first baseline auto</option>
    <option value="last baseline normal">last baseline normal</option>
    <option value="stretch auto">stretch auto</option>
  </select>
</div>
```

```
const values = document.getElementById("grid-values");
const container = document.getElementById("grid-container");

values.addEventListener("change", (evt) => {
  container.style.placeItems = evt.target.value;
});
```

CSS

```
#grid-container {
  height: 200px;
  width: 240px;
  place-items: stretch; /* You can change this value by selecting another option in the list */
```

```
  background-color: #8c8c8c;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

#grid-container > div {
  width: 50px;
}
```

Result

See also

- `align-items`
- `align-self`
- `justify-items`
- `justify-self`
- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment module

place-self CSS property

The **`place-self`** CSS shorthand property allows you to align an individual item in both the block and inline directions at once (i.e., the `align-self` and `justify-self` properties). This property applies to block-level boxes, absolutely-positioned boxes, and grid items. If the second value is not present, the first value is also used for it.

Try it

```
place-self: stretch center;
```

```
place-self: center start;
```

```
place-self: start end;
```

```
place-self: end center;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  width: 220px;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `align-self`
- `justify-self`

Syntax

```
/* Positional alignment */
place-self: auto center;
place-self: normal start;
place-self: center normal;
place-self: start auto;
place-self: end normal;
place-self: self-start auto;
place-self: self-end normal;
place-self: flex-start auto;
place-self: flex-end normal;
place-self: anchor-center;

/* Baseline alignment */
place-self: baseline normal;
place-self: first baseline auto;
place-self: last baseline normal;
place-self: stretch auto;

/* Global values */
place-self: inherit;
place-self: initial;
place-self: revert;
place-self: revert-layer;
place-self: unset;
```

Values

This property is specified as one or two of the following keyword values:

`auto`

Computes to the parent's `align-items` value.

`normal`

The effect of this keyword is dependent of the layout mode we are in:

- In absolutely-positioned layouts, the keyword behaves like `start` on *replaced* absolutely-positioned boxes, and as `stretch` on *all other* absolutely-positioned boxes.
- In static position of absolutely-positioned layouts, the keyword behaves as `stretch`.
- For flex items, the keyword behaves as `stretch`.
- For grid items, this keyword leads to a behavior similar to the one of `stretch`, except for boxes with an aspect ratio or an intrinsic size where it behaves like `start`.
- The property doesn't apply to block-level boxes, and to table cells.

`self-start`

Aligns the items to be flush with the edge of the alignment container corresponding to the item's start side in the cross axis.

`self-end`

Aligns the items to be flush with the edge of the alignment container corresponding to the item's end side in the cross axis.

`flex-start`

The cross-start margin edge of the flex item is flushed with the cross-start edge of the line.

`flex-end`

The cross-end margin edge of the flex item is flushed with the cross-end edge of the line.

`center`

The flex item's margin box is centered within the line on the cross-axis. If the cross-size of the item is larger than the flex container, it will overflow equally in both directions.

`baseline`, `first baseline`. `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group. The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`stretch`

If the combined size of the items along the cross axis is less than the size of the alignment container and the item is `auto`-sized, its size is increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size of all `auto`-sized items exactly fills the alignment container along the cross axis.

`anchor-center`

In the case of anchor-positioned elements, aligns the item to the center of the associated anchor element in the block and inline direction. See Centering on the anchor using `anchor-center`.

Formal definition

Initial value	as each of the properties of the shorthand: • `align-self`: `auto` • `justify-self`: `auto`
Applies to	block-level boxes, absolutely-positioned boxes, and grid items
Inherited	no
Computed value	as each of the properties of the shorthand: • `align-self`: `auto` computes to itself on absolutely-positioned elements, and to the computed value of `align-items` on the parent (minus any legacy keywords) on all other boxes, or `start` if the box has no parent. Its behavior depends on the layout model, as described for `justify-self`. Otherwise the specified value. • `justify-self`: as specified
Animation type	discrete

Formal syntax

```
place-self =
  <'align-self'> <'justify-self'>?

<align-self> =
  auto                                          |
  <overflow-position>? [ normal | <self-position> ]  |
  stretch                                       |
  <baseline-position>                           |
  anchor-center

<justify-self> =
  auto                                           |
  <overflow-position>? [ normal | <self-position> | left | right ]  |
  stretch                                        |
  <baseline-position>                            |
  anchor-center

<overflow-position> =
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end

<baseline-position> =
  [ first | last ]?  &&
  baseline
```

This syntax reflects the latest standard as per CSS Box Alignment Module Level 3, CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

In the following example we have a 2 x 2 grid layout. Initially the grid container has `justify-items` and `align-items` values of `stretch` — the defaults — which causes the grid items to stretch across the entire width of their cells.

The second, third, and fourth grid items are then given different values of `place-self`, to show how these override the default placements. These values cause the grid items to span only as wide/tall as their content width/height, and align in different positions across their cells, in the block and inline directions.

HTML

```
<article class="container">
  <span>First</span>
  <span>Second</span>
  <span>Third</span>
  <span>Fourth</span>
</article>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
  letter-spacing: 1px;
}

article {
  background-color: red;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
  margin: 20px;
  width: 300px;
}

span:nth-child(2) {
  place-self: start center;
}

span:nth-child(3) {
  place-self: center start;
}

span:nth-child(4) {
  place-self: end;
}

article span {
  background-color: black;
  color: white;
  margin: 1px;
  text-align: center;
}

article,
span {
  padding: 10px;
  border-radius: 7px;
}
```

Result

See also

- `align-self`
- `justify-self`
- Basic concepts of flexbox
- Aligning items in a flex container
- Box alignment in grid layout
- CSS box alignment module

pointer-events CSS property

The **`pointer-events`** CSS property sets under what circumstances (if any) a particular graphic element can become the target of pointer events.

Try it

```
pointer-events: auto;
```

```
pointer-events: none;
```

```
pointer-events: stroke; /* SVG-only */
```

```
pointer-events: fill; /* SVG-only */
```

```
<section class="flex-column" id="default-example">
  <div id="example-element">
    <p>
      <a href="#">example link</a>
    </p>
    <p>
      <svg viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
        <a xlink:href="#">
          <circle
            cx="50"
            cy="50"
```

```
            fill="#3E6E84"
            r="40"
            stroke="#ffb500"
            stroke-width="5"></circle>
          <text fill="white" text-anchor="middle" x="50" y="55">SVG</text>
        </a>
      </svg>
    </p>
  </div>
</section>
```

```
#example-element {
  font-weight: bold;
}

#example-element a {
  color: #009e5f;
}

#example-element svg {
  width: 10em;
  height: 10em;
}
```

Syntax

```
/* Keyword values */
pointer-events: auto;
pointer-events: none;

/* Values used in SVGs */
pointer-events: visiblePainted;
pointer-events: visibleFill;
pointer-events: visibleStroke;
pointer-events: visible;
pointer-events: painted;
pointer-events: fill;
pointer-events: stroke;
pointer-events: bounding-box;
pointer-events: all;

/* Global values */
pointer-events: inherit;
pointer-events: initial;
pointer-events: revert;
pointer-events: revert-layer;
pointer-events: unset;
```

The `pointer-events` property is specified as a single keyword chosen from the list of values below.

Values

`auto`

The element behaves as it would if the `pointer-events` property were not specified. In SVG content, this value and the value `visiblePainted` have the same effect.

`none`

The element is generally not the target of pointer events, with the exception of `pointerenter` and `pointerleave` events fired at the element or its descendants. Note that pointer events targeted at descendants (that don't set `pointer-events` to `none`) will still trigger event listeners on this parent element during the event capture and bubble phases.

> **Note:** The `pointerenter` and `pointerleave` events are fired when a pointing device is moved into an element or one of its descendants. So, even if `pointer-events: none` is set on the parent and not set on children, the events are triggered on the parent after the pointer is moved in or out of a descendant.

SVG only (experimental for HTML)

`visiblePainted`

SVG only (experimental for HTML). The element can only be the target of a pointer event when the `visibility` property is set to `visible` and e.g., when a mouse cursor is over the interior (i.e., 'fill') of the element and the `fill` property is set to a value other than `none`, or when a mouse cursor is over the perimeter (i.e., 'stroke') of the element and the `stroke` property is set to a value other than `none`.

`visibleFill`

SVG only. The element can only be the target of a pointer event when the `visibility` property is set to `visible` and when e.g., a mouse cursor is over the interior (i.e., fill) of the element. The value of the `fill` property does not affect event processing.

`visibleStroke`

SVG only. The element can only be the target of a pointer event when the `visibility` property is set to `visible` and e.g., when the mouse cursor is over the perimeter (i.e., stroke) of the element. The value of the `stroke` property does not affect event processing.

`visible`

SVG only (experimental for HTML). The element can be the target of a pointer event when the `visibility` property is set to `visible` and e.g., the mouse cursor is over either the interior (i.e., fill) or the perimeter (i.e., stroke) of the element. The values of the `fill` and `stroke` do not affect event processing.

`painted`

SVG only (experimental for HTML). The element can only be the target of a pointer event when e.g., the mouse cursor is over the interior (i.e., 'fill') of the element and the `fill` property is set to a value other than `none`, or when the mouse cursor is over the perimeter (i.e., 'stroke') of the element and the `stroke` property is set to a value other than `none`. The value of the `visibility` property does not affect event processing.

`fill`

SVG only. The element can only be the target of a pointer event when the pointer is over the interior (i.e., fill) of the element. The values of the `fill` and `visibility` properties do not affect event processing.

`stroke`

SVG only. The element can only be the target of a pointer event when the pointer is over the perimeter (i.e., stroke) of the element. The values of the `stroke` and `visibility` properties do not affect event processing.

`bounding-box`

SVG only. The element can only be the target of a pointer event when the pointer is over the bounding box of the element.

`all`

SVG only (experimental for HTML). The element can only be the target of a pointer event when the pointer is over the interior (i.e., fill) or the perimeter (i.e., stroke) of the element. The values of the `fill`, `stroke`, and `visibility` properties do not affect event processing.

Description

When this property is unspecified, the same characteristics of the `visiblePainted` value apply to SVG content.

In addition to indicating that the element is not the target of pointer events, the value `none` instructs the pointer event to go "through" the element and target whatever is "underneath" that element instead.

Note that preventing an element from being the target of pointer events by using `pointer-events` does *not* necessarily mean that pointer event listeners on that element *cannot* or *will not* be triggered. If one of the element's children has `pointer-events` explicitly set to *allow* that child to be the target of pointer events, then any events targeting that child will pass through the parent as the event travels along the parent chain, and trigger event listeners on the parent as appropriate. Of course any pointer activity at a point on the screen that is covered by the parent but not by the child will not be caught by either the child or the parent (it will go "through" the parent and target whatever is underneath).

Elements with `pointer-events: none` will still receive focus through sequential keyboard navigation using the `Tab` key.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
pointer-events =
  auto            |
  bounding-box    |
  visiblePainted  |
  visibleFill     |
  visibleStroke   |
  visible         |
  painted         |
  fill            |
  stroke          |
  all             |
  none
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling pointer events on all images

This example disables pointer events (clicking, dragging, hovering, etc.) on all images.

```
img {
  pointer-events: none;
}
```

Disabling pointer events on a single link

This example disables pointer events on the link to `http://example.com`.

HTML

```
<ul>
  <li><a href="https://developer.mozilla.org">MDN</a></li>
  <li><a href="http://example.com">example.com</a></li>
</ul>
```

CSS

```
a[href="http://example.com"] {
  pointer-events: none;
}
```

Result

See also

- `user-select`
- SVG `pointer-events` attribute
- SVG `visibility` attribute
- `PointerEvent`

position CSS property

The **`position`** CSS property sets how an element is positioned in a document. The `top`, `right`, `bottom`, and `left` physical properties and the `inset-block-start`, `inset-block-end`, `inset-inline-start`, and `inset-inline-end` flow-relative logical properties can be used to determine the final location of positioned elements.

Try it

```
position: static;
```

```
position: relative;
top: 40px;
left: 40px;
```

```
position: absolute;
inset-inline-start: 40px;
inset-block-start: 40px;
```

```
position: sticky;
top: 20px;
```

```
<section class="default-example" id="default-example">
  <div id="example-element-container">
    <p>
      In this demo you can control the <code>position</code> property for the
      yellow box.
    </p>
    <div class="box"></div>
    <div class="box" id="example-element"></div>
    <div class="box"></div>
    <p class="clear">
      To see the effect of <code>sticky</code> positioning, select the
      <code>position: sticky</code> option and scroll this container.
    </p>
    <p>
      The element will scroll along with its container, until it is at the top
      of the container (or reaches the offset specified in <code>top</code>),
      and will then stop scrolling, so it stays visible.
    </p>
    <p>
      The rest of this text is only supplied to make sure the container
      overflows, so as to enable you to scroll it and see the effect.
    </p>
    <hr />
    <p>
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
      Orbiting this at a distance of roughly ninety-two million miles is an
      utterly insignificant little blue green planet whose ape-descended life
      forms are so amazingly primitive that they still think digital watches are
      a pretty neat idea.
    </p>
  </div>
</section>
```

```
section {
  align-items: flex-start;
  overflow: auto;
}

.box {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  float: left;
  width: 65px;
  height: 65px;
}

.box + .box {
  margin-left: 10px;
}

.clear {
  clear: both;
  padding-top: 1em;
}

#example-element-container {
  position: relative;
  text-align: left;
}

#example-element {
  background-color: yellow;
  border: 3px solid red;
  z-index: 1;
}
```

Syntax

```
position: static;
position: relative;
position: absolute;
position: fixed;
position: sticky;

/* Global values */
position: inherit;
position: initial;
position: revert;
position: revert-layer;
position: unset;
```

Values

This property is specified as one of the following keyword values:

`static`

The element is positioned according to the Normal Flow of the document. The `top`, `right`, `bottom`, `left`, and `z-index` properties have *no effect*. This is the default value.

`relative`

The element is positioned according to the normal flow of the document, and then offset *relative to itself* based on the values of `top`, `right`, `bottom`, and `left`. The offset does not affect the position of any other elements; thus, the space given for the element in the page layout is the same as if position were `static`.

This value creates a new stacking context when the value of `z-index` is not `auto`. Its effect on `table-*-group`, `table-row`, `table-column`, `table-cell`, and `table-caption` elements is undefined.

`absolute`

The element is removed from the normal document flow, and no space is created for the element in the page layout. The element is positioned relative to its closest positioned ancestor (if any) or to the initial containing block. Its final position is determined by the values of `top`, `right`, `bottom`, and `left`.

This value creates a new stacking context when the value of `z-index` is not `auto`. The margins of absolutely positioned boxes do not collapse with other margins.

`fixed`

The element is removed from the normal document flow, and no space is created for the element in the page layout. The element is positioned relative to its initial containing block, which is the viewport in the case of visual media. Its final position is determined by the values of `top`, `right`, `bottom`, and `left`.

This value always creates a new stacking context. In printed documents, the element is placed in the same position on *every page*.

`sticky`

The element is positioned according to the normal flow of the document, and then offset relative to its *nearest scrolling ancestor* and containing block (nearest block-level ancestor), including table-related elements, based on the values of `top`, `right`, `bottom`, and `left`. The offset does not affect the position of any other elements.

This value always creates a new stacking context. Note that a sticky element "sticks" to its nearest ancestor that has a "scrolling mechanism" (created when `overflow` is `hidden`, `scroll`, `auto`, or `overlay`), even if that ancestor isn't the nearest actually scrolling ancestor.

> **Note:** At least one inset property (`top`, `inset-block-start`, `right`, `inset-inline-end`, etc.) needs to be set to a non-`auto` value for the axis on which the element needs to be made sticky. If both `inset` properties for an axis are set to `auto`, on that axis the `sticky` value will behave as `relative`.

Description

Types of positioning

- A **positioned element** is an element whose computed `position` value is either `relative`, `absolute`, `fixed`, or `sticky`. (In other words, it's anything except `static`.)
- A **relatively positioned element** is an element whose computed `position` value is `relative`. The `top` and `bottom` properties specify the vertical offset from its normal position; the `left` and `right` properties specify the horizontal offset.
- An **absolutely positioned element** is an element whose computed `position` value is `absolute` or `fixed`. The `top`, `right`, `bottom`, and `left` properties specify offsets from the edges of the element's containing block. (The containing block is the ancestor relative to which the element is positioned.) If the element has margins, they are added to the offset. The element establishes a new block formatting context (BFC) for its contents.
- A **stickily positioned element** is an element whose computed `position` value is `sticky`. It's treated as relatively positioned until its containing block crosses a specified threshold (such as setting `top` to value other than auto) within its flow root (or the container it scrolls within), at which point it is treated as "stuck" until meeting the opposite edge of its containing block.

Most of the time, absolutely positioned elements that have `height` and `width` set to `auto` are sized so as to fit their contents. However, non-replaced, absolutely positioned elements can be made to fill the available vertical space by specifying both `top` and `bottom` and leaving `height` unspecified (that is, `auto`). They can likewise be made to fill the available horizontal space by specifying both `left` and `right` and leaving `width` as `auto`.

Except for the case just described (of absolutely positioned elements filling the available space):

- If both `top` and `bottom` are specified (technically, not `auto`), `top` wins.
- If both `left` and `right` are specified, `left` wins when `direction` is `ltr` (English, horizontal Japanese, etc.) and `right` wins when `direction` is `rtl` (Persian, Arabic, Hebrew, etc.).

Accessibility

Ensure that elements positioned with an `absolute` or `fixed` value do not obscure other content when the page is zoomed to increase text size.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Visual Presentation: Understanding SC 1.4.8 | Understanding WCAG 2.0

Performance & Accessibility

Scrolling elements containing `fixed` or `sticky` content can cause performance and accessibility issues. As a user scrolls, the browser must repaint the sticky or fixed content in a new location. Depending on the content needing to be repainted, the browser performance, and the device's processing speed, the browser may not be able to manage repaints at 60 fps. Such a scenario can lead to jank and, more importantly, accessibility concerns for people with sensitivities. One solution is to add `will-change: transform` to the positioned elements to render the element in its own layer, improving repaint speed and therefore improving performance and accessibility.

Formal definition

Initial value	`static`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete
Creates stacking context	yes

Formal syntax

```
position =
  static       |
  relative     |
  absolute     |
  sticky       |
  fixed        |
  <running()>

<running()> =
  running( <custom-ident> )
```

This syntax reflects the latest standard as per CSS Generated Content for Paged Media Module, CSS Positioned Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Relative positioning

Relatively positioned elements are offset a given amount from their normal position within the document, but without the offset affecting other elements. In the example below, note how the other elements are placed as if "Two" were taking up the space of its normal location.

HTML

```
<div class="box" id="one">One</div>
<div class="box" id="two">Two</div>
<div class="box" id="three">Three</div>
<div class="box" id="four">Four</div>
```

CSS

```
* {
  box-sizing: border-box;
}

.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#two {
  position: relative;
  top: 20px;
  left: 20px;
  background: blue;
}
```

Absolute positioning

Elements that are relatively positioned remain in the normal flow of the document. In contrast, an element that is absolutely positioned is taken out of the flow; thus, other elements are positioned as if it did not exist. The absolutely positioned element is positioned relative to its *nearest positioned ancestor* (i.e., the nearest ancestor that is not `static`). If a positioned ancestor doesn't exist, it is positioned relative to the ICB (initial containing block), which is the containing block of the document's root element.

HTML

```
<h1>Absolute positioning</h1>

<p>
  I am a basic block level element. My adjacent block level elements sit on new
  lines below me.
</p>

<p class="positioned">
  By default we span 100% of the width of our parent element, and we are as tall
  as our child content. Our total width and height is our content + padding +
  border width/height.
</p>

<p>
  We are separated by our margins. Because of margin collapsing, we are
  separated by the width of one of our margins, not both.
</p>

<p>
  inline elements <span>like this one</span> and <span>this one</span> sit on
  the same line as one another, and adjacent text nodes, if there is space on
  the same line. Overflowing inline elements
  <span>wrap onto a new line if possible — like this one containing text</span>,
  or just go on to a new line if not, much like this image will do:
  <img src="https://mdn.github.io/shared-assets/images/examples/long.jpg" />
</p>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  width: 500px;
  margin: 0 auto;
}

p {
  background: aqua;
  border: 3px solid blue;
  padding: 10px;
  margin: 10px;
}

span {
  background: red;
  border: 1px solid black;
}

.positioned {
  position: absolute;
  background: yellow;
  inset-block-start: 30px;
  inset-inline-start: 30px;
}
```

Result

Fixed positioning

Fixed positioning is similar to absolute positioning, with the exception that the element's containing block is the initial containing block established by the *viewport*, unless any ancestor has `transform`, `perspective`, or `filter` property set to something other than `none` (see fixed positioning containing block), which then causes that ancestor to take the place of the elements containing block. This can be used to create a "floating" element that stays in the same position regardless of scrolling. In the example below, box "One" is fixed at 80 pixels from the top of the page and 10 pixels from the left. Even after scrolling, it remains in the same place relative to the viewport. Also, when the `will-change` property is set to `transform`, a new containing block is established.

HTML

```
<div class="outer">
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam congue tortor
    eget pulvinar lobortis. Vestibulum ante ipsum primis in faucibus orci luctus
    et ultrices posuere cubilia Curae; Nam ac dolor augue. Pellentesque mi mi,
    laoreet et dolor sit amet, ultrices varius risus. Nam vitae iaculis elit.
    Aliquam mollis interdum libero. Sed sodales placerat egestas. Vestibulum ut
    arcu aliquam purus viverra dictum vel sit amet mi. Duis nisl mauris, aliquam
    sit amet luctus eget, dapibus in enim. Sed velit augue, pretium a sem
    aliquam, congue porttitor tortor. Sed tempor nisl a lorem consequat, id
    maximus erat aliquet. Sed sagittis porta libero sed condimentum. Aliquam
    finibus lectus nec ante congue rutrum. Curabitur quam quam, accumsan id
    ultrices ultrices, tempor et tellus.
  </p>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam congue tortor
    eget pulvinar lobortis. Vestibulum ante ipsum primis in faucibus orci luctus
    et ultrices posuere cubilia Curae; Nam ac dolor augue. Pellentesque mi mi,
    laoreet et dolor sit amet, ultrices varius risus. Nam vitae iaculis elit.
    Aliquam mollis interdum libero. Sed sodales placerat egestas. Vestibulum ut
    arcu aliquam purus viverra dictum vel sit amet mi. Duis nisl mauris, aliquam
    sit amet luctus eget, dapibus in enim. Sed velit augue, pretium a sem
    aliquam, congue porttitor tortor. Sed tempor nisl a lorem consequat, id
    maximus erat aliquet. Sed sagittis porta libero sed condimentum. Aliquam
    finibus lectus nec ante congue rutrum. Curabitur quam quam, accumsan id
    ultrices ultrices, tempor et tellus.
  </p>
  <div class="box" id="one">One</div>
</div>
```

CSS

```
* {
  box-sizing: border-box;
}

.box {
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#one {
  position: fixed;
  top: 80px;
  left: 10px;
  background: blue;
}

.outer {
  width: 500px;
  height: 300px;
  overflow: scroll;
  padding-left: 150px;
}
```

Result

Sticky positioning

The following CSS rule positions the element with id `one` relatively until the viewport is scrolled such that the element is 10 pixels from the top. Beyond that threshold, the element is fixed to 10 pixels from the top.

```
#one {
  position: sticky;
  top: 10px;
}
```

List with sticky headings

A common use for sticky positioning is for the headings in an alphabetized list. The "B" heading will appear just below the items that begin with "A" until they are scrolled offscreen. Rather than sliding offscreen with the rest of the content, the "B" heading will then remain fixed to the top of the viewport until all the "B" items have scrolled offscreen, at which point it will be covered up by the "C" heading, and so on.

You must specify a threshold with at least one of `top`, `right`, `bottom`, or `left` for sticky positioning to behave as expected. Otherwise, it will be indistinguishable from relative positioning.

HTML

```
<dl>
  <div>
    <dt>A</dt>
    <dd>Andrew W.K.</dd>
    <dd>Apparat</dd>
    <dd>Arcade Fire</dd>
    <dd>At The Drive-In</dd>
    <dd>Aziz Ansari</dd>
  </div>
  <div>
    <dt>C</dt>
    <dd>Chromeo</dd>
    <dd>Common</dd>
    <dd>Converge</dd>
```

```
    <dd>Crystal Castles</dd>
    <dd>Cursive</dd>
  </div>
  <div>
    <dt>E</dt>
    <dd>Explosions In The Sky</dd>
  </div>
  <div>
    <dt>T</dt>
    <dd>Ted Leo &amp;amp; The Pharmacists</dd>
    <dd>T-Pain</dd>
    <dd>Thrice</dd>
    <dd>TV On The Radio</dd>
    <dd>Two Gallants</dd>
  </div>
</dl>
```

CSS

```
* {
  box-sizing: border-box;
}

dl > div {
  background: white;
  padding-top: 24px;
}

dt {
  background: #b8c1c8;
  border-bottom: 1px solid #989ea4;
  border-top: 1px solid #717d85;
  color: white;
  font:
    bold 18px/21px "Helvetica",
    "Arial",
    sans-serif;
  margin: 0;
  padding: 2px 0 0 12px;
  position: -webkit-sticky;
  position: sticky;
  top: -1px;
}

dd {
  font:
    bold 20px/45px "Helvetica",
    "Arial",
    sans-serif;
  margin: 0;
  padding-left: 12px;
  white-space: nowrap;
}

dd + dd {
  border-top: 1px solid #cccccc;
}
```

Result

Sticky position with all the inset boundaries set

The following example demonstrates an element's behavior when all inset boundaries are set. Here, we have two light bulb emojis in a paragraph. The light bulbs use sticky positioning, and the inset boundaries are specified as 50px from the top and bottom, and 100px from the left and right. A gray background on the parent div element marks the inset area.

HTML

```
Use scrollbars to put the light bulbs(💡) in the right place in the following
text:
<div>
  <p>
    The representation of an idea by a light bulb(<span class="bulb">💡</span>)
    is a commonly used metaphor that symbolizes the moment of inspiration or the
    birth of a new idea. The association between a light bulb and an idea can be
    traced back to the invention of the incandescent light bulb(<span
      class="bulb"
      >💡</span
    >) by Thomas Edison in the late 19th century. The light bulb is a powerful
    symbol because it represents illumination, clarity, and the sudden
    brightening of one's thoughts or understanding. When someone has an idea, it
    is often described as a light bulb turning on in their mind, signifying a
    moment of insight or creativity. The image of a light bulb also suggests the
    idea of energy, power, and the potential for growth and development.
  </p>
</div>
```

CSS

```
div {
  width: 400px;
  height: 200px;
  overflow: scroll;
  scrollbar-width: thin;
  font-size: 16px;
  font-family: "Verdana";
  border: 1px solid;
}

p {
  width: 600px;
  user-select: none;
  margin: 0;
```

```
  border: 110px solid transparent;
}
```

```
.bulb {
  position: sticky;
  inset: 50px 100px;
}

div {
  /* mark area defined by the inset boundaries using gray color */
  background: linear-gradient(#99999999, #99999999) 100px 50px / 192px 100px
    no-repeat;
}
```

Result

When you put both bulbs in their proper place, you'll notice that they are relatively positioned inside the inset area. When you move them out of the inset area, they are fixed (sticky) to the inset boundary in that direction.

See also

- Inset properties
- Learn CSS: Positioning
- Inset properties for positioned layout
- CSS positioned layout modules

position-anchor CSS property

The **`position-anchor`** CSS property specifies the default **anchor element** for a positioned element. This default is used by `position-area` and `position-try`, and by anchor functions (`anchor()` and `anchor-size()`) when no `<anchor-name>` argument is provided to those functions.

Syntax

```
/* Single values */
position-anchor: normal;
position-anchor: auto;
position-anchor: none;
position-anchor: --anchor-name;
position-anchor: match-parent;

/* Global values */
position-anchor: inherit;
position-anchor: initial;
position-anchor: revert;
position-anchor: revert-layer;
position-anchor: unset;
```

Values

`normal`

If `position-area` is `none`, it behaves as `none`. Otherwise it behaves as `auto`.

`auto`

Associates a positioned element with its implicit anchor element, if it has one — for example as set by the non-standard HTML `anchor` attribute.

`none`

The positioned element is not associated with an anchor element.

`<dashed-ident>`

The name of the anchor element to associate the positioned element with, as listed in the anchor element's `anchor-name` property. This is known as the **default anchor specifier**.

`match-parent`

Uses the same default anchor element as the parent element, or the originating element if this is a pseudo-element, provided that would be an acceptable anchor element (that is part of the tree). Otherwise, it has no default anchor element.

Description

This property is only relevant to "positioned" elements — elements and pseudo-elements that have a `position` of `absolute` or `fixed` set.

To position an element relative to an anchor element, the positioned element requires three features: an association, a position, and a location. The `position-anchor` and `anchor-name` properties provide an explicit association.

The anchor element accepts one or more `<dashed-ident>` anchor names set on it via the `anchor-name` property. When one of those names is then set as the value of the positioned element's `position-anchor` property, the two elements are associated.

If there are multiple anchor elements with the anchor name listed in the `position-anchor` property, the positioned element will be associated with the last anchor element in the source order with that anchor name.

To cancel a previously-made association between an anchor-positioned element and an anchor, you can set the anchor-positioned element's `position-anchor` value to `none`.

To tether a positioned element to its anchor, it must be placed relative to an anchor element using an anchor positioning feature, such as the `anchor()` function (set as a value on inset properties) or the `position-area` property.

If the associated anchor is hidden, for example with `display: none` or `visibility: hidden`, or if it is part of the skipped contents of another element due to it having `content-visibility: hidden` set on it, the anchor positioned element will not be displayed.

The `position-anchor` property is supported on all elements that are positioned, including pseudo-elements like `::before` and `::after`. Pseudo elements are implicitly anchored to the same element as the pseudo-element's originating element, unless otherwise specified.

For more information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

Formal definition

Initial value	`normal`

Applies to	absolutely positioned elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
position-anchor =
  normal         |
  none           |
  auto           |
  <anchor-name>  |
  match-parent

<anchor-name> =
  <dashed-ident>
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

See the `anchor-name` documentation for basic usage and additional `position-anchor` examples.

Multiple positioned elements and anchors

In this example, you can move multiple positioned elements around, associating them with different anchors. This example demonstrates how an anchor can be associated with multiple positioned elements, but an anchor-positioned element can only be associated with a single anchor at a time, the anchor defined by the `anchor-position` property.

HTML

We have four anchors and two positioned elements, distinguished with different `id` values. The positioned elements contain `<select>` boxes that allow you to choose which anchor you want to associate them with.

```
<div id="anchor-container">
  <div class="anchor" id="anchor1">⚓</div>
  <div class="anchor" id="anchor2">⚓</div>
  <div class="anchor" id="anchor3">⚓</div>
  <div class="anchor" id="anchor4">⚓</div>
</div>

<div class="infobox" id="infobox1">
  <form>
    <label for="anchor1-anchor-select">Place infobox on:</label>
    <select id="anchor1-anchor-select">
      <option value="1">Anchor 1</option>
      <option value="2">Anchor 2</option>
      <option value="3">Anchor 3</option>
      <option value="4">Anchor 4</option>
    </select>
  </form>
</div>

<div class="infobox" id="infobox2">
  <form>
    <label for="anchor2-anchor-select">Place infobox on:</label>
    <select id="anchor2-anchor-select">
      <option value="1">Anchor 1</option>
      <option value="2">Anchor 2</option>
      <option value="3">Anchor 3</option>
      <option value="4">Anchor 4</option>
    </select>
  </form>
</div>
```

CSS

We declare the first `anchor <div>` as an anchor using the `anchor-name` property, which is given two comma-separated anchor names, one for each positioned element. This is the initial state of the demo — both positioned elements will be tethered to the first anchor.

```
body {
  height: 100vh;
}

#anchor-container {
  display: grid;
  height: 100%;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 1fr 1fr;
  justify-items: center;
  align-items: center;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
#anchor1 {
  anchor-name: --my-anchor1, --my-anchor2;
}
```

Each of the positioned elements is given a `position-anchor` property with a value matching one of the two anchor names. The positioned elements are then given anchor-relative positioning information using a combination of inset, alignment, and margin properties.

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
  width: 120px;
}
```

```
#infobox1 {
  position-anchor: --my-anchor1;
  position: fixed;
  left: anchor(right);
  align-self: anchor-center;
  margin-left: 10px;
}

#infobox2 {
  position-anchor: --my-anchor2;
  position: fixed;
  bottom: anchor(top);
  justify-self: anchor-center;
  margin-bottom: 15px;
}
```

JavaScript

We dynamically change which anchor elements the `anchor-name` values are set on in response to different anchors being selected in the positioned elements' `<select>` menus. The key functionality here is the `change` event handler, `updateAnchorNames()`. It sets both anchor names on one anchor, if the anchors chosen in the two `<select>` menus are the same. Otherwise, it sets a single anchor name on two separate anchors as appropriate.

```
// Get references to the two select menus
const select1 = document.querySelector("#anchor1-anchor-select");
const select2 = document.querySelector("#anchor2-anchor-select");
// Store references to all the anchors in a NodeList (array-like)
const anchors = document.querySelectorAll("#anchor-container > div");

// Set the same change event handler on both select menus
select1.addEventListener("change", updateAnchorNames);
select2.addEventListener("change", updateAnchorNames);

function updateAnchorNames() {
  // Remove all anchor names from all anchors
  for (const anchor of anchors) {
    anchor.style.anchorName = "none";
  }

  // convert the select menu values to numbers, and remove one to
  // make them match the selected anchors' index positions in the NodeList
  const value1 = Number(select1.value) - 1;
  const value2 = Number(select2.value) - 1;

  if (value1 === value2) {
    // If the chosen anchors are both the same, set both anchor
    // names on the same anchor
    anchors[value1].style.anchorName = "--my-anchor1, --my-anchor2";
  } else {
    // If they are not the same, set the anchor names separately
    // on each selected anchor
    anchors[value1].style.anchorName = "--my-anchor1";
    anchors[value2].style.anchorName = "--my-anchor2";
  }
}
```

Result

Select different values from the drop-down menus to change the anchors that the elements are positioned relative to.

Using a slider thumb as an anchor

In this example, an `<output>` is positioned relative to an anchor that is the thumb of a range slider.

HTML

We include an `<input type="range">` element and an `<output>` element to display the value of the range. The value displayed in the `<output>` element is updated via JavaScript as the slider value changes.

```
<label for="slider">Change the value:</label>
<input type="range" min="0" max="100" value="25" id="slider" />
<output>25</output>
```

CSS

We give the thumb, represented by the `::-webkit-slider-thumb` and `::-moz-range-thumb` pseudo-elements, an anchor name of `--thumb`. We then set that name as the value of the `<output>` element's `position-anchor` property, and give it a `position` value of `fixed`. These steps associated the `<output>` with the thumb.

Finally, we use `left` and `top` properties with `anchor()` values to position the `<output>` relative to the thumb.

```
body {
  display: grid;
  place-items: center;
  height: 100vh;
  font-size: 1.4rem;
  font-family: sans-serif;
}
input {
  width: 33vw;
```

```
}
output {
  background: cyan;
  border: 2px solid darkblue;
  padding: 3px 6px;
  border-radius: 10px;
  font-size: 1.2rem;
}
```

```
input::-webkit-slider-thumb {
  anchor-name: --thumb;
}

input::-moz-range-thumb {
  anchor-name: --thumb;
}

output {
  position-anchor: --thumb;
  position: absolute;
  left: anchor(right);
  bottom: anchor(top);
}
```

JavaScript

We include an event listener that updates the content of the `<output>` element when the value of the `<input>` changes:

```
const input = document.querySelector("input");
const output = document.querySelector("output");

input.addEventListener("input", (event) => {
  output.innerText = `${input.value}`;
});
```

Results

The output is anchored to the thumb. Change the value and the output will remain above and to the right of the thumb, no matter where it is along the slider.

> **Note:** The anchor positioning CSS for this demo currently doesn't work in Firefox. The slider's thumb can be targeted via the `::-moz-range-thumb` pseudo-element in Firefox, but it is currently not available as an anchor (see Firefox bug 1993699).

See also

- `anchor-name`
- `anchor-scope`
- HTML `anchor` attribute
- CSS anchor positioning module
- Using CSS anchor positioning guide

position-area CSS property

The **`position-area`** CSS property enables an anchor-positioned element to be positioned relative to the edges of its associated anchor element by placing the positioned element on one or more tiles of an implicit 3x3 grid, where the anchoring element is the center cell.

`position-area` provides a convenient alternative to tethering and positioning an element relative to its anchor via inset properties and the `anchor()` function. The grid-based concept solves the common use-case of positioning the edges of the positioned element's containing block relative to the edges of its default anchor element.

If an element does not have a default anchor element, or is not an absolutely-positioned element, this property has no effect.

> **Note:** This property was originally named and supported in Chromium browsers as `inset-area`, with the same property values. Both property names will be supported for a short while, for backwards compatibility purposes.

Syntax

```
/* Default value */
position-area: none;

/* Two <position-area> keywords defining a single specific tile */
position-area: top left;
position-area: start end;
position-area: block-start center;
position-area: inline-start block-end;
position-area: x-start y-end;
position-area: center self-y-end;

/* Two <position-area> keywords spanning two tiles */
position-area: top span-left;
position-area: center span-start;
position-area: inline-start span-block-end;
position-area: y-start span-x-end;

/* Two <position-area> keywords spanning three tiles */
position-area: top span-all;
position-area: block-end span-all;
position-area: self-x-start span-all;

/* One <position-area> keyword with an implicit second <position-area> keyword  */
position-area: top; /* equiv: top span-all */
position-area: inline-start; /* equiv: inline-start span-all */
position-area: center; /* equiv: center center */
position-area: span-all; /* equiv: span-all span-all */
position-area: end; /* equiv: end end */

/* Global values */
position-area: inherit;
```

```
position-area: initial;
position-area: revert;
position-area: revert-layer;
position-area: unset;
```

Values

The property value is two `<position-area>` keyterms, or the keyword `none`. If only one `<position-area>` keyterm is provided, the second keyterm is implied.

`<position-area>`

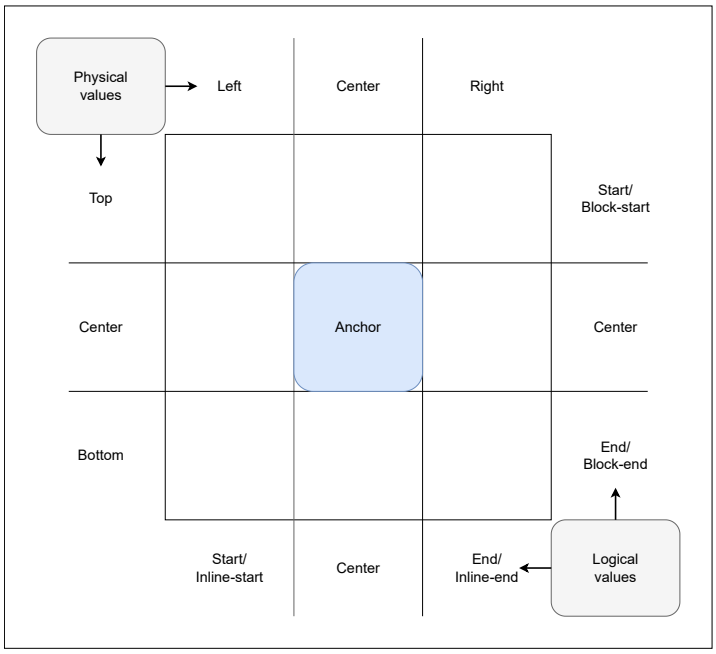
Specifies the area of the position area grid on which to place selected positioned elements.

`none`

No position area is set.

Description

The `position-area` property provides an alternative to the `anchor()` function for positioning elements relative to anchors. `position-area` works on the concept of a 3x3 grid of tiles, called the **position-area grid**, with the anchor element being the center tile:



The grid tiles are broken up into rows and columns:

- The three rows are represented by the physical values `top`, `center`, and `bottom`. They also have logical equivalents such as `block-start`, `center`, and `block-end`, and coordinate equivalents — `y-start`, `center`, and `y-end`.
- The three columns are represented by the physical values `left`, `center`, and `right`. They also have logical equivalents such as `inline-start`, `center`, and `inline-end`, and coordinate equivalents — `x-start`, `center`, and `x-end`.

The dimensions of the center tile are defined by the containing block of the anchor element, while the dimensions of the grid's outer edge are defined by the positioned element's containing block.

The `<position-area>` value is composed of one or two keywords, which define the region of the grid the positioned element should be placed inside. To be exact, the containing block of the positioned element is set to the grid area.

For example:

- You can specify a row value and a column value to place the positioned element in a single, specific grid square — for example, `top left` (logical equivalent `start start`) or `bottom center` (logical equivalent `end center`) will place the positioned element in the top-right or bottom-center square.
- You can specify a row or column value plus a `span-*` value to span two or three cells. The first value specifies the row or column to place the positioned element in, placing it initially in the center, and the other one specifies the other tiles of that row or column to span. For example:
  - `top span-left` causes the positioned element to be placed in the center of the top row, and span across the center and left tiles of that row.
  - `block-end span-inline-end` causes the positioned element to be placed in the center of the block end row, and span across the center and inline end tiles of that row.
  - `bottom span-all` and `y-end span-all` cause the positioned element to be placed in the center of the bottom row, and span across three cells, in this case the left, center, and right tiles of the bottom row.

For detailed information on anchor features, usage, and the `position-area` property, see the CSS anchor positioning module and the Using CSS anchor positioning guide, specifically the section on setting a `position-area`.

Adjusted default behavior

When a `<position-area>` value is set on a positioned element, some of its properties will have their default behavior adjusted to provide a good default alignment.

Self-alignment property `normal` value

The `normal` value of the self-alignment properties, including `align-items`, `align-self`, `justify-items`, and `justify-self`, behaves as either `start`, `end`, or `anchor-center`. Which value a self-alignment property defaults to depends on the positioning of the element:

- If the `position-area` value specifies the center region in an axis, the default alignment in that axis is `anchor-center`.
- Otherwise, the behavior is the opposite of the region specified by the `position-area` property. For example, if the `position-area` value specifies the start region of its axis, the default alignment in that axis is `end`.

For example, if the `writing-mode` is set to `horizontal-tb`, `position-area: top span-x-start` causes the positioned element to be placed in the center of the top row, and span across the center and start tiles of that row. In this case, the self-alignment properties will default to `align-self: end` and `justify-self: anchor-center`.

inset properties and values

When an anchor-positioned element is positioned using the `position-area` property, any inset properties set, such as `top` or `inset-inline-end`, specify offsets from the position-area. Some other property values, like `max-block-size: 100%`, will also be relative to the position-area. Any inset properties set or defaulting to `auto` will behave as if their value was set to `0`.

An aside on positioned element width

If the positioned element does not have a specific size set on it, its size will default to its intrinsic size, but it will also be affected by the size of the position-area grid.

If the positioned element is placed in a single top-center, bottom-center, or center-center cell, its block size will be the same as the anchor's containing block size, growing up, down, or in both directions respectively. The positioned element will align with the specified grid square but adopt the same width as the anchor element. However, it won't allow its content to overflow — its minimum `width` will be its `min-content` (as defined by the width of its longest word).

If the positioned element is placed in any other single grid square (say with `position-area: top left`) or is set to span two or more grid squares (for example using `position-area: bottom span-all`), it will align with the specified grid area but behave as if it has a `width` of `max-content` set on it. It is being sized according to its containing block size, which is the size imposed on it when it was set to `position: fixed`. It will stretch as wide as the text content, although it may also be constrained by the edge of the `<body>`.

Using <code>position-area</code> to position popovers

When using `position-area` to position popovers, be aware that the default styles for popovers may conflict with the position you're trying to achieve. The usual culprits are the default styles for `margin` and `inset`, so it's advisable to reset those:

```
.my-popover {
  margin: 0;
  inset: auto;
}
```

The CSS working group is looking at ways to avoid requiring this workaround.

Formal definition

Initial value	none
Applies to	Positioned elements with a default anchor element
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
position-area =
  none              |
  <position-area>

<position-area> =
  [ left | center | right | span-left | span-right | x-start | x-end | span-x-start | span-x-end | self-x-start | self-x-end | span-self-x-start | span-self-x-end | span-all ] || [ top | center | bottom |
span-top | span-bottom | y-start | y-end | span-y-start | span-y-end | self-y-start | self-y-end | span-self-y-start | span-self-y-end | span-all ]  |
  [ block-start | center | block-end | span-block-start | span-block-end | span-all ] || [ inline-start | center | inline-end | span-inline-start | span-inline-end | span-all ]  |
  [ self-block-start | center | self-block-end | span-self-block-start | span-self-block-end | span-all ] || [ self-inline-start | center | self-inline-end | span-self-inline-start | span-self-inline-end
| span-all ]  |
  [ start | center | end | span-start | span-end | span-all ]{1,2}  |
  [ self-start | center | self-end | span-self-start | span-self-end | span-all ]{1,2}
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, a positioned element is tethered and positioned relative to its associated anchor using the `position-area` property.

HTML

The HTML includes a `<div>` and a `<p>`. The `<p>` will be positioned relative to the `<div>` with CSS. We also include a style block that will be made visible. All elements are set to be directly editable via the `contenteditable` attribute.

```
<div class="anchor" contenteditable="true">⚓</div>

<p class="positionedElement" contenteditable="true">This can be edited.</p>

<style contenteditable="true">.positionedElement {
    position-area: top center;
  }
</style>
```

CSS

We convert the `<div>` to an anchor element with the `anchor-name` property. We then associate the absolutely-positioned `<p>` with it by setting its `position-anchor` value to the same anchor name.

We set the initial `position-area` value to `top center`. This value is set on a `p` selector, so the value has less specificity than any value added to the `<style>` block's `.positionedElement` class selector. As a result, you can override the initial `position-area` value by setting a `position-area` value inside the style block.

```
.anchor {
  anchor-name: --infobox;
  background: palegoldenrod;
  font-size: 3em;
  width: fit-content;
  border: 1px solid goldenrod;
  margin: 100px auto;
}

p {
  position: absolute;
  position-anchor: --infobox;
  position-area: top center;
  margin: 0;
  background-color: darkkhaki;
  border: 1px solid darkolivegreen;
```

```
}

style {
  display: block;
  white-space: pre;
  font-family: monospace;
  background-color: #ededed;
  -webkit-user-modify: read-write-plaintext-only;
  line-height: 1.5;
  padding: 10px;
}
```

Results

Try changing the amount of text in the anchor-positioned element to see how it grows. Also, try changing the value of the `position-area` property to something else, like `center`.

<code>position-area</code> value comparison

This demo creates an anchor and tethers a positioned element to it. It also provides a drop-down menu allowing you to choose various `position-area` values to apply to the positioned element, to see their effect. One of the options causes a text field to appear that enables you to enter a custom value. Finally, a checkbox is provided to turn `writing-mode: vertical-lr` on and off, allowing you to observe how `position-area` value effects differ across different writing modes.

HTML

In the HTML, we specify two `<div>` elements, one with a class of `anchor` and one with a class of `infobox`. These are intended to be the anchor element and the positioned element we will associate with it, respectively. We've included the `contenteditable` attribute on both, making them directly editable.

We've also included two forms that contain the `<select>` and `<input type="text">` elements for setting different `position-area` values, and the `<input type="checkbox">` element for toggling the vertical `writing-mode` on and off. The code for these, along with the JavaScript, has been hidden for the sake of brevity.

```
<div class="anchor" contenteditable>⚓</div>

<div class="infobox">
  <p contenteditable>You can edit this text.</p>
</div>
```

```
<form id="position-area-form">
  <div>
    <label for="position-area-select">Choose a position-area:</label>
    <select id="position-area-select" name="position-area-select">
      <optgroup label="Enter custom value">
        <option>Custom</option>
      </optgroup>
      <optgroup label="Physical, single tile">
        <option selected>top left</option>
        <option>bottom right</option>
      </optgroup>
      <optgroup label="Physical, span two">
        <option>bottom span-right</option>
        <option>left span-top</option>
      </optgroup>
      <optgroup label="Physical, span all">
        <option>top</option>
        <option>left</option>
      </optgroup>
      <optgroup label="Logical, single tile">
        <option>start center</option>
        <option>inline-start block-end</option>
      </optgroup>
      <optgroup label="Logical, span two">
        <option>start span-end</option>
        <option>center span-start</option>
        <option>inline-start span-block-end</option>
        <option>span-block-start center</option>
      </optgroup>
      <optgroup label="Logical, span all">
        <option>start span-all</option>
        <option>block-end</option>
      </optgroup>
      <optgroup label="Coordinates, single tile">
        <option>x-start center</option>
        <option>x-end y-end</option>
      </optgroup>
      <optgroup label="Coordinates, span two">
        <option>center span-y-start</option>
        <option>y-start span-x-end</option>
      </optgroup>
      <optgroup label="Coordinates, span all">
        <option>x-start span-all</option>
        <option>y-end</option>
      </optgroup>
    </select>
  </div>
  <div id="position-area-custom-container">
    <label for="position-area-custom">Enter a custom position-area:</label
    ><br />
    <input type="text" id="position-area-custom" name="position-area-custom" />
  </div>
</form>

<form id="writing-mode-form">
  <label for="writing-mode-checkbox">writing-mode: vertical-lr</label><br />
  <input
    type="checkbox"
    id="writing-mode-checkbox"
    name="writing-mode-checkbox" />
</form>
```

CSS

In the CSS, we first declare the `anchor` `<div>` as an anchor element by setting an anchor name on it via the `anchor-name` property.

The positioned element is associated with the anchor element by setting its anchor name as the value of the positioned element's `position-anchor` property. We also give it an initial position with `position-area: top left`; this will be overridden when new values are selected from the `<select>` menu. Finally, we set its `opacity` to `0.8`, so that when the positioned element is given a `position-area` value that places it over the top of the anchor, you can still see the elements' position relative to one another.

```
.anchor {
  font-size: 1.8rem;
  text-align: center;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  min-width: 50px;
  min-height: 50px;
  line-height: 50px;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

html {
  height: 100%;
}

body {
  margin: 0;
  width: 100vw;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

form {
  background: white;
  border: 1px solid black;
  padding: 5px;
  position: fixed;
}

#position-area-form {
  top: 0;
  right: 2px;
}

#position-area-form div:last-child {
  margin-top: 10px;
}

#position-area-form div :last-child {
  margin-top: 5px;
}

#writing-mode-form {
  bottom: 0;
  left: 2px;
  writing-mode: horizontal-tb;
}

#position-area-custom-container {
  display: none;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 5px 2px;
  border-radius: 5px;
  font-size: 1rem;
  box-sizing: border-box;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}

.infobox {
  position-anchor: --my-anchor;
  position: fixed;
  opacity: 0.8;
  position-area: top left;
}
```

```
const anchorContainer = document.querySelector("body");
const infobox = document.querySelector(".infobox");

const positionAreaForm = document.querySelector("#position-area-form");
const selectElem = document.querySelector("select");
const inputElemContainer = document.querySelector(
  "#position-area-custom-container",
);
const inputElem = document.querySelector("#position-area-custom");
const checkboxElem = document.querySelector("#writing-mode-checkbox");

// Stop form from submitting when enter is pressed on it
positionAreaForm.addEventListener("submit", (e) => {
  e.preventDefault();
});

// Update positioned element position-area to value selected in drop-down
selectElem.addEventListener("change", () => {
  const area = selectElem.value;
```

```
  if (area === "Custom") {
    // Show the custom value input field
    inputElemContainer.style.display = "block";
  } else {
    // Hide the custom value input field
    inputElemContainer.style.display = "none";
    // Set the position-area to the value chosen in the select box
    infobox.style.positionArea = area;
  }
});

// Update positioned element position-area to entered custom value
inputElem.addEventListener("change", () => {
  const customArea = inputElem.value;

  // Set the position-area to the value in the input box
  infobox.style.positionArea = customArea;
});

// Change writing-mode to match checkbox setting
checkboxElem.addEventListener("change", () => {
  if (checkboxElem.checked) {
    anchorContainer.style.writingMode = "vertical-lr";
  } else {
    anchorContainer.style.writingMode = "horizontal-tb";
  }
});
```

Result

The result is as follows:

Try selecting new `position-area` values from the `<select>` menu to see the effect they have on the position of the infobox. Select the "Custom" value and try entering some custom `position-area` values into the text input to see their effect. Add text to the anchor and the anchor positioned elements to see how the anchor positioned element grows based on the `position-area` value. Finally, check the checkbox and then experiment with different `position-area` values to see which ones give the same result across different writing modes, and which ones give different results.

See also

- `anchor-name`
- `position-anchor`
- `position-try-fallbacks`
- The `anchor()` function
- The `<position-area>` value
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide
- CSS anchor positioning module

position-try CSS property

The **`position-try`** CSS shorthand property corresponds to the `position-try-order` and `position-try-fallbacks` properties.

Constituent properties

This property is a shorthand for the following CSS properties:

- `position-try-order`
- `position-try-fallbacks`

Syntax

```
/* position-try-fallbacks only */
position-try: normal flip-block;
position-try: top;
position-try: --custom-try-option;
position-try: flip-block flip-inline;
position-try: top, right, bottom;
position-try: --custom-try-option1, --custom-try-option2;
position-try:
  normal flip-block,
  right,
  --custom-try-option;

/* position-try-order and position-try-fallbacks */
position-try: normal none;
position-try:
  most-width --custom-try-option1,
  --custom-try-option2;
position-try:
  most-height flip-block,
  right,
  --custom-try-option;

/* Global values */
position-try: inherit;
position-try: initial;
position-try: revert;
position-try: revert-layer;
position-try: unset;
```

Values

This property is specified as one or both of the following values:

`position-try-order` Optional

A keyword specifying the priority order of the position-try fallback options. If omitted, defaults to `normal`.

`position-try-fallbacks`

The keyword `none`, or a comma-separated list of `<position-area>` values, `<try-tactic>` options, and custom positions.

Description

The `position-try` shorthand property can be used to specify the `position-try-order` and `position-try-fallbacks` properties in a single declaration. It specifies the `position-try-order` and `position-try-fallbacks`, in that order. If the `position-try-order` component is omitted, it defaults to `normal`.

The `position-try-fallbacks` is a comma-separated list of one or more alternative position try fallback options for anchor-positioned elements to be placed relative to their associated anchor elements. When the element would otherwise overflow its inset-modified containing block, the browser will try placing the positioned element in these different fallback positions, in the order provided, until it finds a value that stops it from overflowing its container or the viewport.

The `position-try-order` prioritizes the `position-try-fallbacks` option applied to an anchor-positioned element when it first renders, based on which option creates the most space around the element in the given direction. If omitted, it is set to the property's initial value, which is `normal`, meaning the position-try fallback options are tried in the order they appear in the property.

Formal definition

Initial value	as each of the properties of the shorthand: • `position-try-fallbacks`: `none` • `position-try-order`: `normal`
Applies to	absolutely positioned elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `position-try-fallbacks`: no • `position-try-order`: no
Computed value	as each of the properties of the shorthand: • `position-try-fallbacks`: as specified • `position-try-order`: as specified
Animation type	as each of the properties of the shorthand: • `position-try-fallbacks`: discrete • `position-try-order`: discrete

Formal syntax

```
position-try =
  <'position-try-order'>? <'position-try-fallbacks'>

<position-try-order> =
  normal      |
  <try-size>

<position-try-fallbacks> =
  none                                          |
  [ [ <dashed-ident> || <try-tactic> ] | <position-area> ]#

<try-size> =
  most-width       |
  most-height      |
  most-block-size  |
  most-inline-size

<try-tactic> =
  flip-block   ||
  flip-inline  ||
  flip-start   ||
  flip-x       ||
  flip-y

<position-area> =
  [ left | center | right | span-left | span-right | x-start | x-end | span-x-start | span-x-end | self-x-start | self-x-end | span-self-x-start | span-self-x-end | span-all ] || [ top | center | bottom |
span-top | span-bottom | y-start | y-end | span-y-start | span-y-end | self-y-start | self-y-end | span-self-y-start | span-self-y-end | span-all ]  |
  [ block-start | center | block-end | span-block-start | span-block-end | span-all ] || [ inline-start | center | inline-end | span-inline-start | span-inline-end | span-all ]  |
  [ self-block-start | center | self-block-end | span-self-block-start | span-self-block-end | span-all ] || [ self-inline-start | center | self-inline-end | span-self-inline-start | span-self-inline-end
| span-all ]  |
  [ start | center | end | span-start | span-end | span-all ]{1,2}  |
  [ self-start | center | self-end | span-self-start | span-self-end | span-all ]{1,2}
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>position-try</code> usage

This demo shows the effect of `position-try`.

HTML

The HTML includes two `<div>` elements that will become an anchor and an anchor-positioned element.

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

CSS

In the CSS, the anchor is given an `anchor-name` and has a `position` value of `absolute` set on it. We position it in the top-half of the viewport using `top` and `left` values:

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
```

```
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
.anchor {
  anchor-name: --my-anchor;
  position: absolute;
  top: 100px;
  left: 45%;
}
```

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
  text-align: center;
}

form {
  position: fixed;
  bottom: 2px;
  right: 2px;
}
```

We then include a custom position option — `--custom-bottom` — which positions the element below the anchor and gives it an appropriate margin:

```
@position-try --custom-bottom {
  top: anchor(bottom);
  bottom: unset;
  margin-top: 10px;
}
```

We initially position the element above its anchor, and then set a `position-try` value on it that gives it a `position-try-order` of `most-height`, and a `position-try-fallbacks` list that just includes our custom fallback option:

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;

  bottom: anchor(top);
  margin-bottom: 10px;
  justify-self: anchor-center;

  position-try: most-height --custom-bottom;
}
```

Result

The element appears below its anchor, even though it is initially positioned above it. This occurs because there is more vertical space below the anchor than there is above it. The `most-height` try order causes the `--custom-bottom` try fallback option to be applied, placing the positioned element in the position that gives its containing block the most height.

See also

- `position-area`
- `position-try-fallbacks`
- `position-try-order`
- The `@position-try` at-rule
- The `<position-area>` value
- CSS anchor positioning module
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide

position-try-fallbacks CSS property

The **`position-try-fallbacks`** CSS property enables you to specify a list of one or more alternative **position try fallback options** for anchor-positioned elements to be placed relative to their associated anchor elements. When the element would otherwise overflow its inset-modified containing block, the browser will try placing the positioned element in these different fallback positions, in the order provided, until it finds a value that stops it from overflowing its container or the viewport.

Syntax

```
/* Default value: no try fallback options */
position-try-fallbacks: none;

/* Single try option */
position-try-fallbacks: flip-block;
position-try-fallbacks: top;
position-try-fallbacks: --custom-try-option;

/* Multiple value combination option */
position-try-fallbacks: flip-block flip-inline;

/* Multiple values */
position-try-fallbacks: flip-block, flip-inline;
position-try-fallbacks: top, right, bottom;
position-try-fallbacks: --custom-try-option1, --custom-try-option2;
position-try-fallbacks:
  flip-block,
  flip-inline,
  flip-block flip-inline;
position-try-fallbacks:
  flip-block,
  --custom-try-option,
```

```
  --custom-try-option flip-inline,
  right;

/* Global values */
position-try-fallbacks: inherit;
position-try-fallbacks: initial;
position-try-fallbacks: revert;
position-try-fallbacks: revert-layer;
position-try-fallbacks: unset;
```

The `position-try-fallbacks` property may be specified as either the keyword value `none` or as a comma-separated list of one or more space-separated custom position option names or `<try-tactic>`s or a `position-area` value.

Values

This property is specified as the keyword `none` or a comma-separated list of the following values:

`none`

The default value. There are no position try fallback options set.

`<try-tactic>`

Predefined fallback options move the positioned element by taking its computed position and transforming it across a particular axis of the anchor, mirroring any margin offsets. Possible values are:

`flip-block`

Flips the element's position along the block axis.

`flip-inline`

Flips the element's position along the inline axis.

`flip-start`

Flips both the inline and block axis values, swapping the `start` properties with each other, and the `end` properties with each other.

`position-area` value

Positions the element relative to the edges of its associated anchor element by placing the positioned element on one or more tiles of an implicit 3x3 position area grid based on the specified `<position-area>` value; the effect is the same as a custom `@position-try` fallback option containing only a `position-area` descriptor.

`<dashed-ident>`

Adds a custom `@position-try` option to the fallback options list, the identifying name of which matches the specified `dashed-ident`. If no custom position option exists with that name, the option is ignored.

Description

Anchor-positioned elements should always appear in a convenient place for the user to interact with, if at all possible, regardless of where their anchor is positioned. To stop the positioned element from overflowing the viewport, it is often necessary to change its location when its anchor gets close to the edge of its containing element or the viewport.

This is achieved by providing one or more position-try fallback options in the `position-try-fallbacks` property. If the positioned element's initial position would overflow, the browser will try each fallback position option; the first fallback option that doesn't cause the element to overflow its containing block is applied. By default, the browser will try them in the order they appear in the list, applying the first one it finds that will stop the positioned element from overflowing.

If no option can be found that will place the positioned element completely on-screen, the browser will revert to displaying the positioned element at its default position before any try fallback options were applied.

> **Note:** In some situations you might want to just hide overflowing positioned elements, which can be achieved using the `position-visibility` property. In most cases however it is better to keep them on-screen and usable.

The `position-try-fallbacks` property, along with the `position-try-order` property, can also be set by using the `position-try` shorthand.

For detailed information on anchor features and position try fallback usage, see the CSS anchor positioning module and the Fallback options and conditional hiding for overflow guide.

Predefined <try-tactic> values

Referred to as a `<try-tactic>` in the specification, the predefined values move the positioned element by taking its computed position and transforming it across a particular axis of the anchor. The predefined values are:

`flip-block`

Flips the element's position along the block axis so that it appears the same distance away from the anchor but on the opposite side of it. To put it another way, it mirrors the element's position across an inline axis drawn through the center of the anchor. As an example, if the positioned element started to overflow at the top of the anchor, this value would flip the position to the bottom.

`flip-inline`

Flips the element's position along the inline axis so that it appears the same distance away from the anchor but on the opposite side of it. To put it another way, it mirrors the element's position across a block axis drawn through the center of the anchor. As an example, if the positioned element started to overflow at the left of the anchor, this value would flip the position to the right.

`flip-start`

Mirrors the element's position across an axis drawn diagonally through the center of the anchor, passing through the point at the intersection of the block axis start and the inline axis start, and the point at the intersection of the block axis end and the inline axis end. As an example, if the positioned element started to overflow at the left of the anchor, this value would flip the positioned element to the top.

Combination options

A single position-try fallback option can include more than one `<try-tactic>` or `dashed-ident` options, or a combination of both by declaring them as a single space-separated option:

- In the case of multiple predefined `<try-tactic>` options, their transformations are composed together.
- In the case of declaring a predefined `<try-tactic>` and a `<dashed-ident>` named `@position-try` option, the custom position option is applied first, then the `<try-tactic>` transformation is applied.

`position-area` values cannot be combined like this.

Formal definition

Initial value	none
Applies to	absolutely positioned elements

Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
position-try-fallbacks =
  none                                          |
  [ [ <dashed-ident> || <try-tactic> ] | <position-area> ]#

<try-tactic> =
  flip-block   ||
  flip-inline  ||
  flip-start   ||
  flip-x       ||
  flip-y

<position-area> =
  [ left | center | right | span-left | span-right | x-start | x-end | span-x-start | span-x-end | self-x-start | self-x-end | span-self-x-start | span-self-x-end | span-all ] || [ top | center | bottom |
span-top | span-bottom | y-start | y-end | span-y-start | span-y-end | self-y-start | self-y-end | span-self-y-start | span-self-y-end | span-all ]  |
  [ block-start | center | block-end | span-block-start | span-block-end | span-all ] || [ inline-start | center | inline-end | span-inline-start | span-inline-end | span-all ]  |
  [ self-block-start | center | self-block-end | span-self-block-start | span-self-block-end | span-all ] || [ self-inline-start | center | self-inline-end | span-self-inline-start | span-self-inline-end
| span-all ]  |
  [ start | center | end | span-start | span-end | span-all ]{1,2}  |
  [ self-start | center | self-end | span-self-start | span-self-end | span-all ]{1,2}
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example shows the basic usage of a couple of predefined `<try-tactic>` fallback options.

HTML

The HTML includes two `<div>` elements that will become an anchor and an anchor-positioned element:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

CSS

We style the `<body>` element to be very large, to enable both horizontal and vertical scrolling.

The anchor is given an `anchor-name` and large margins to place it somewhere near the center of the visible section of the `<body>`:

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
body {
  width: 1500px;
  height: 500px;
}

.anchor {
  anchor-name: --my-anchor;
  margin: 100px 350px;
}
```

The infobox is given fixed positioning, a `position-anchor` property that references the anchor's `anchor-name`, to associate the two together, and it is tethered to the anchor's top-left corner using a `position-area`.

We include a `position-try-fallbacks` list (and re-declare it with the `position-try` shorthand in case the longhand property name is not yet supported), providing two predefined position-try fallback options to prevent it from overflowing when the anchor gets near the edge of the viewport, by flipping it along the inline or block axis of the anchor.

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top left;

  position-try-fallbacks: flip-block, flip-inline;
  position-try: flip-block, flip-inline;
}
```

Result

This gives us the following result:

Try scrolling so the anchor nears the edges:

- If you move the anchor near the top of the viewport, you will see the positioned element flip to the bottom-left of the anchor to avoid overflowing.
- If you move the anchor near the left of the viewport, you will see the positioned element flip to the top-right of the anchor to avoid overflowing.

Depending on the browser, once the positioned element moves to the fallback position, it may remain in the fallback position even if the fallback positioning is no longer necessary, such as when the space allows it to return to the position defined by the `position-area`.

However, if you move the anchor towards the top-left corner of the viewport, you'll notice a problem — as the positioned element starts to overflow in the block and inline direction, it flips back to its default top-left position and overflows in both directions, which is not what we want.

This is because we only gave the browser position options of `flip-block` *or* `flip-inline`. We didn't give it the option of trying both at the same time. The next example will show you how to fix this issue.

Combining multiple values into one option

Let's use a combined try fallback option to fix the problem we found with the previous demo.

HTML and CSS

All of the HTML and CSS in this demo is the same, except for the positioned element code. In this case, it is given a third position try fallback option: `flip-block flip-inline`:

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

```
body {
  width: 1500px;
  height: 500px;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
  margin: 100px 350px;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top left;

  position-try:
    flip-block,
    flip-inline,
    flip-block flip-inline;
  position-try-fallbacks:
    flip-block,
    flip-inline,
    flip-block flip-inline;
}
```

Result

The third position-try fallback option means that the browser will try `flip-block` then `flip-inline` to avoid overflow, and if those fallbacks fail, it will combine the two, flipping the element's position in the block and inline directions at the same time. Now when you scroll the anchor towards the top *and* left edges of the viewport, the positioned element will flip over to the bottom-right.

<code>position-area</code> try fallback options

This example shows some `position-area` position-try fallback options in action.

HTML and CSS

All of the HTML and CSS in this demo is the same, except for the positioned element code. In this case, our position try fallback options are all `position-area` values — `top`, `top right`, `right`, `bottom right`, `bottom`, `bottom left`, and `left`.

This means that the positioned element will find a reasonable position to display in, whatever viewport edges the anchor is near. This approach is a bit more longwinded than the predefined values approach, but it is also more granular and flexible.

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

```
body {
  width: 1500px;
  height: 500px;
}

.anchor {
  font-size: 1.8rem;
```

```
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

.anchor {
  anchor-name: --my-anchor;
  margin: 100px 350px;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  position-area: top left;

  position-try:
    top, top right, right,
    bottom right, bottom,
    bottom left, left;

  position-try-fallbacks:
    top, top right, right,
    bottom right, bottom,
    bottom left, left;
}
```

Result

Scroll the page and check out the effect of these position-try fallback options as the anchor nears the edge of the viewport.

Custom try option examples

See the `@position-try` reference page.

See also

- `position-try`
- `position-try-order`
- `@position-try` at-rule
- `position-area`
- `<position-area>` value
- Fallback options and conditional hiding for overflow guide
- Using CSS anchor positioning guide
- CSS anchor positioning module

position-try-order CSS property

The **`position-try-order`** CSS property allows you to prioritize the `position-try-fallbacks` option applied to an anchor-positioned element when it first renders, based on which option creates the most space around the element in the given direction.

Syntax

```
/* Keyword values */
position-try-order: normal;
position-try-order: most-height;
position-try-order: most-width;
position-try-order: most-block-size;
position-try-order: most-inline-size;

/* Global values */
position-try-order: inherit;
position-try-order: initial;
position-try-order: revert;
position-try-order: revert-layer;
position-try-order: unset;
```

Values

The `position-try-order` property may be specified as either the keyword value `normal` or a `<try-size>`.

`normal`

The default. No position-try fallback options will be tried when the element is first rendered.

`<try-size>`

Defines which criteria will be used to determine what try fallback should be applied to the anchor-positioned element when it initially renders. Available values are:

`most-height`

Applies the position try fallback option that gives the element the most vertical space.

`most-width`

Applies the position try fallback option that gives the element the most horizontal space.

`most-block-size`

Applies the position try fallback option that gives the element the most space in the block direction.

`most-inline-size`

Applies the position try fallback option that gives the element the most space in the inline direction.

Description

The `position-try-order` property has a slightly different focus from the rest of the position-try features, in that it influences which position-try fallback option is applied when the positioned element is first displayed, rather than when it is being scrolled. For example, you might want to initially display the element in a space that has more available height or width than the default initial position.

The browser tests the available `position-try-fallbacks` to find which one gives the anchor-positioned element the most space in the specified direction. It will then apply that option, overriding the element's initial styling when the page first renders.

If no position try fallback option is available that provides more width/height than the initial positioning assigned to the element, no position try option will be applied, the same as if `position-try-order` were set to `normal`.

For detailed information on anchor features and position try option usage, see the CSS anchor positioning module and the Fallback options and conditional hiding for overflow guide.

The `position-try-order` property, along with the `position-try-fallbacks` property, can also be set by using the `position-try` shorthand.

Formal definition

Initial value	`normal`
Applies to	absolutely positioned elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
position-try-order =
  normal      |
  <try-size>

<try-size> =
  most-width       |
  most-height      |
  most-block-size  |
  most-inline-size
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example shows the effect of `position-try-order`.

HTML

The HTML includes two `<div>` elements that will become an anchor and an anchor-positioned element.

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

CSS

In the CSS, we start by setting a `position-try-order` value of `normal` on the anchor-positioned element, so it is easier to find when we ask you to edit it later on:

```
.infobox {
  position-try-order: normal;
}
```

The anchor is given an `anchor-name` and a large `margin` to position it near center of the viewport:

```
.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}
```

```
.anchor {
  anchor-name: --my-anchor;
  margin: 90px auto;
}
```

```
.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
  text-align: center;
}
```

We include a custom position option named `--custom-bottom`, which will positions the anchor-positioned element below the anchor and give it an appropriate margin:

```
@position-try --custom-bottom {
  top: anchor(bottom);
  bottom: unset;
  margin-top: 10px;
}
```

We initially position the anchor-positioned element above its anchor, and then give it our custom position option using the `position-try-fallbacks` property.

```
.infobox {
  position: fixed;
  position-anchor: --my-anchor;

  bottom: anchor(top);
  margin-bottom: 10px;
  justify-self: anchor-center;

  position-try-fallbacks: --custom-bottom;
}
```

Result

Initially, the anchor-positioned element is positioned above its anchor, which is the default position we've given it.

Now open the example in the MDN Playground by pressing the **Play** button, run the example, then change the `position-try-order` to `most-height` or `most-block-size`. When the example re-renders, the anchor-positioned element is positioned below its anchor: `--custom-bottom` fallback is applied because it gives the positioned element more surrounding height than the default position.

See also

- `position-try`
- `position-try-fallbacks`
- The `@position-try` at-rule
- CSS anchor positioning module
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide

position-visibility CSS property

The **`position-visibility`** CSS property enables conditionally hiding an anchor-positioned element depending on, for example, whether it is overflowing its containing element or the viewport.

Syntax

```
/* Single values */
position-visibility: always;
position-visibility: anchors-valid;
position-visibility: anchors-visible;
position-visibility: no-overflow;

/* Global values */
position-visibility: inherit;
position-visibility: initial;
position-visibility: revert;
position-visibility: revert-layer;
position-visibility: unset;
```

Values

This property is specified as one of the following keyword values:

`always`

The positioned element is always displayed.

`anchors-valid`

If the positioned element's `position-anchor` value doesn't point to a valid anchor element, the positioned element will be strongly hidden.

`anchors-visible`

If the anchor is completely hidden, either by overflowing its containing element (or the viewport) or being covered by other elements, the positioned element will be strongly hidden.

`no-overflow`

If the positioned element starts to overflow its containing element or the viewport, it will be strongly hidden.

Description

In some situations, you might not want to display an anchor-positioned element. For example, if its associated anchor has been scrolled offscreen but the anchor-positioned element would otherwise still be partially or fully visible, it might be unclear what it refers to and take up space unnecessarily, so you may want to hide it altogether.

The `position-visibility` property can be used to `always` show the anchor-positioned element, or conditionally hide it in certain situations:

- `anchors-visible`: The associated anchor element is completely hidden.
- `anchors-valid`: The anchor-positioned element's `position-anchor` property doesn't reference a valid `anchor-name` set on an anchor element in the same document.
- `no-overflow`: The anchor-positioned element is partially or completely hidden.

When an element is hidden due to `position-visibility`, it is referred to as **strongly hidden**. This means that it will act as though it and its descendant elements have a `visibility` value of `hidden` set, regardless of what their actual visibility value is.

`position-visibility` should only be used in situations in which hiding the positioned element altogether is preferred. In most cases, it makes more sense to attempt to change the placement of positioned elements when they start to overflow, to keep them on-screen and usable. This can be done with the `position-try-fallbacks` property and `@position-try` at-rule. See the Fallback options and conditional hiding for overflow guide for more information.

Formal definition

Initial value	`anchors-visible`
Applies to	absolutely positioned elements
Inherited	no

Computed value	as specified
Animation type	discrete

Formal syntax

```
position-visibility =
  always                                          |
  [ anchor-valid || anchor-visible || no-overflow ]
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example enables changing the value of an anchor positioned element's `position-visibility` property to demonstrate the effects of each value.

HTML

We specify two `<div>` elements: an anchor element with a class of `anchor` and a positioned element with a class of `infobox`.

```
<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique.
</p>

<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque.
</p>
```

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

```
<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>

<form>
  <fieldset>
    <legend>Choose position visibility</legend>
    <div>
      <label for="radio-always">always</label>
      <input
        type="radio"
        id="radio-always"
        name="position-visibility"
        value="always"
        checked />
    </div>
    <div>
      <label for="radio-anchors-visible">anchors-visible</label>
      <input
        type="radio"
        id="radio-anchors-visible"
        name="position-visibility"
        value="anchors-visible" />
    </div>
    <div>
      <label for="radio-no-overflow">no-overflow</label>
      <input
        type="radio"
        id="radio-no-overflow"
        name="position-visibility"
        value="no-overflow" />
    </div>
  </fieldset>
</form>
```

The HTML also includes filler text to make the content taller than the viewport so scrolling is required. We've also included a `<fieldset>` with a group of radio inputs with different `position-visibility` values. The markup for these is not shown for the sake of brevity.

CSS

We style an `anchor` `<div>` as an anchor element and tether the `infobox` `<div>` to it. The relevant CSS is as follows:

```
body {
  width: 50%;
  margin: 0 auto;
}

.anchor {
  font-size: 1.8rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

form {
  position: fixed;
  background: white;
  bottom: 2px;
  right: 2px;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}

.infobox {
  position-anchor: --my-anchor;
  position: fixed;
  position-area: top span-all;
  margin-bottom: 5px;
  position-visibility: always;
}
```

JavaScript

We include a change event handler on the radio buttons so that, when a new value is selected, we update the infobox's `position-visibility` property value.

```
const infobox = document.querySelector(".infobox");
const radios = document.querySelectorAll('[name="position-visibility"]');

for (const radio of radios) {
  radio.addEventListener("change", setPositionVisibility);
}

function setPositionVisibility(e) {
  infobox.style.positionVisibility = e.target.value;
}
```

Result

Select different `position-visibility` values and then scroll the page up and down to see their effects. With `position-visibility: always` set, the positioned element will not be hidden. With `position-visibility: anchors-visible` set, the positioned element will only be visible when the anchor is partially or fully on-screen. With `position-visibility: no-overflow` set, the positioned element will be hidden as soon as it starts to overflow the viewport.

See also

- `anchor-name`
- `position-anchor`
- `position`
- `position-area`
- CSS anchor positioning module
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide

print-color-adjust CSS property

The **`print-color-adjust`** CSS property sets what, if anything, the user agent may do to optimize the appearance of the element on the output device. By default, the browser is allowed to make any adjustments to the element's appearance it determines to be necessary and prudent given the type and capabilities of the output device.

Syntax

```
print-color-adjust: economy;
print-color-adjust: exact;

/* Global values */
print-color-adjust: inherit;
print-color-adjust: initial;
print-color-adjust: revert;
print-color-adjust: revert-layer;
print-color-adjust: unset;
```

Values

This property is specified as one of the following keyword values:

`economy`

The user agent is allowed to make adjustments to the element as it deems appropriate and prudent in order to optimize the output for the device it's being rendered for. For example, when printing, a browser might opt to leave out all background images and to adjust text colors to be sure the contrast is optimized for reading on white paper. This is the default.

`exact`

The element's content has been specifically and carefully crafted to use colors, images, and styles in a thoughtful and/or important way, such that being altered by the browser might actually make things worse rather than better. The appearance of the content should not be changed except by the user's request. For example, a page might include a list of information with rows whose background colors alternate between white and a light gray. Removing the background color would decrease the legibility of the content.

Usage notes

There are a number of reasons a browser might wish to deviate from the specified appearance, such as:

- The content uses text and background colors that will be too similar on the output device for legibility purposes.
- If the output device is a printer, and to save ink, dark or extremely dense background images might be removed.
- When printing a page, the browser might want to replace light-colored text on a dark background with dark text on a white background.

Any options the user agent offers the user to allow them to control the use of color and images will take priority over the value of `print-color-adjust`. In other words, there isn't any guarantee that `print-color-adjust` will do anything. Not only can the user override the behavior, but each user agent is allowed to decide for itself how to handle `print-color-adjust` in any given situation.

Formal definition

Initial value	`economy`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
print-color-adjust =
  economy  |
  exact
```

This syntax reflects the latest standard as per CSS Color Adjustment Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preserving low contrast

In this example, a box is shown which uses a `background-image` and a translucent `linear-gradient()` function atop a black background color to have a dark blue gradient behind medium red text. For whatever reason, this is the desired appearance in any rendering environment, including on paper, so we also use `print-color-adjust: exact` to tell the browser not to make color or style adjustments to the box when rendering it.

CSS

```
.my-box {
  background-color: black;
  background-image: linear-gradient(rgb(0 0 180 / 50%), rgb(70 140 220 / 50%));
  color: #990000;
  width: 15rem;
  height: 6rem;
  text-align: center;
  font:
    24px "Helvetica",
    sans-serif;
  display: flex;
  align-items: center;
  justify-content: center;
  print-color-adjust: exact;
}
```

HTML

```
<div class="my-box">
  <p>Need more contrast!</p>
</div>
```

Result

See also

- Other color-related properties: `color`, `background-color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`
- `background-image`

quotes CSS property

The CSS **`quotes`** property sets how the browser should render quotation marks that are automatically added to the HTML `<q>` element or added using the `open-quotes` or `close-quotes` (or omitted using the `no-open-quote` and `no-close-quote`) values of the of the CSS `content` property.

Browsers insert quotation marks at the opening and closing of `<q>` elements and for the `open-quote` and `close-quote` values of the `content` property. Each opening or closing quote is replaced by one of the strings from the value of `quotes`, based on the depth of nesting, or, if `quotes` is explicitly set to or otherwise resolves to `auto`, the quotation marks used are language dependent.

Try it

```
quotes: none;
```

```
quotes: initial;
```

```
quotes: "'" "'";
```

```
quotes: "„" "“" "‚" "‘";
```

```
quotes: "«" "»" "‹" "›";
```

```
<section id="default-example">
  <q id="example-element"
    >Show us the wonder-working <q>Brothers,</q> let them come out publicly—and
    we will believe in them!</q
  >
</section>
```

```
q {
  font-size: 1.2rem;
}
```

Syntax

```
/* Keyword values */
quotes: none;
quotes: auto;

/* <string> values */
quotes: "«" "»"; /* Set open-quote and close-quote to use French quotation marks */
quotes: "«" "»" "‹" "›"; /* Set two levels of quotation marks */

/* Global values */
quotes: inherit;
quotes: initial;
quotes: revert;
quotes: revert-layer;
quotes: unset;
```

Values

`none`

The `open-quote` and `close-quote` values of the `content` property produce no quotation marks, as if `no-open-quote` and `no-close-quote` were set, respectively.

`auto`

Quotation marks that are typographically appropriate for the inherited language (i.e., via the `lang` attribute set on the parent or other ancestor).

`<string>`

Defines one or more pairs of quotation mark values for opening and closing quotes. In each pair, the first of each pair of quotes are used as the values for the `open-quote` and the second of each pair is the `close-quote`.

The first pair represents the quotation's outer level. The second pair, if present, represents the first nested level. The next pair is used for doubly nested levels, and so on. If the depth of quote nesting is greater than the number of pairs, the last pair in the `quotes` value is repeated.

Which pair of quotes is used depends on the depth, or nesting level, of quotes: the number of occurrences of `<q>` quotes or `open-quote` (or `no-open-quote`) in all generated text before the current occurrence, minus the number of occurrences of closing quotes, either as `</q>`, `close-quote`, or `no-close-quote`. If the depth is 0, the first pair is used, if the depth is 1, the second pair is used, etc.

> **Note:** The CSS `content` property value `no-open-quote` increments and `no-close-quote` decrements the quoting level, but they do not insert quotation marks.

Formal definition

Initial value	depends on user agent
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
quotes =
  auto                  |
  none                  |
  bikeshed-this         |
  [ <string> <string> ]+
```

This syntax reflects the latest standard as per CSS Generated Content Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Default quotes and overrides

This example compares the default quotes provided by the semantic HTML `<q>` element to those we define using the CSS `quotes` property.

The default value of `quotes` is `auto`. In this example, the first list item has `quotes: auto` set, so gets the default quotes for the language specified; the same as if no `quotes` property was set. The second list item defines which quotation marks to use for quotes and nested quotes; these quotation marks will be used for descendants of an element with `specialQuotes` class regardless of the language (like any `lang` attribute values set).

HTML

```
<ul>
  <li>
    Default quotes:
    <p lang="ru">
      <q
        >Митч Макконнелл - это человек, который знает о России и ее влиянии
        меньше, чем даже Дональд Трамп, и <q>я ничего не знаю</q>, сказал
        Трамп</q
      >, - писал Раджу.
    </p>
  </li>
  <li class="specialQuotes">
    Defined by <code>quotes</code> property :
    <p lang="ru">
      <q
```

```
        >Митч Макконнелл - это человек, который знает о России и ее влиянии
        меньше, чем даже Дональд Трамп, и <q>я ничего не знаю</q>, сказал
        Трамп</q
      >, - писал Раджу.
    </p>
  </li>
</ul>
```

CSS

```
li {
  quotes: auto;
}

.specialQuotes {
  quotes: "“" "”" "‘" "’";
}
```

Result

By default, browser provide language specific quotation marks when the `<q>` element is used. If the `quotes` property is defined, the values provided override the browser defaults. Note the `quotes` property is inherited. The `quotes` property is set on the `<li>` with the `specialQuotes` class, but the quotes are applied the `<q>` elements.

Note that each open-quote and close-quote is replaced by one of the strings from the value of quotes, based on the depth of nesting.

Auto quotes

The default value of `quotes` is `auto`. This example works without it being explicitly being set.

HTML

```
<ul>
  <li lang="fr">
    <q>Ceci est une citation française.</q>
  </li>
  <li lang="ru">
    <q>Это русская цитата</q>
  </li>
  <li lang="de">
    <q>Dies ist ein deutsches Zitat</q>
  </li>
  <li lang="en">
    <q>This is an English quote.</q>
  </li>
</ul>
```

CSS

```
q {
  quotes: auto;
}
li:not(:last-of-type) {
  border-bottom: 1px solid;
}
```

```
li {
  padding: 0.5em 0;
}
```

Result

Note that the `lang` attribute was placed on an ancestor of the `<q>`, not the `<q>` itself. If a quotation is in a different language than the surrounding text, it is customary to quote the text with the quote marks of the language of the surrounding text, not the language of the quotation itself.

With generated content

In this example, instead of using the `<q>` element, we are adding quotation marks to the `::before` and `::after` pseudo-elements before and after the content of each element with a specific class name.

HTML

```
<p>
  <span class="quote">I should be using quotes</span>, I thought,
  <span class="quote"
    >But why use semantic HTML elements when I can add classes to
    <span class="quote">ALL THE THINGS!</span>?
  </span>
</p>
```

CSS

```
.quote {
  quotes: '"' '"' "'" "'";
}
.quote::before {
  content: open-quote;
}
.quote::after {
  content: close-quote;
}
```

Result

Text as quotes and empty quotes

This example demonstrates using something other than quotation marks as the `<string>` values. The open-quote indicates the speaker and, as there is no opening quotation mark, the close-quote is empty. (Mixing a `<string>` with an enumerated keyword to create a pair is not supported). We set `auto` for the nested quotes. These nested quotes will be book-ended by whatever the language dictates is normal for nested quotes.

HTML

```
<ul>
  <li><q data-speaker="karen">Hello</q></li>
  <li><q data-speaker="chad">Hi</q></li>
  <li><q data-speaker="karen">this conversation is not interesting</q></li>
  <li>
    <q data-speaker="pat"
      >OMG! <q>Hi</q>? Seriously? at least <q>hello</q> is five letters long.</q
    >
  </li>
</ul>
```

CSS

```
[data-speaker="karen" i] {
  quotes: "She said: " "";
}
[data-speaker="chad" i] {
  quotes: "He said: " "";
}
[data-speaker="pat" i] {
  quotes: "They said: " "";
}
[data-speaker] q {
  quotes: auto;
}
```

```
li {
  padding: 0.5em 0;
}
```

Result

See also

- CSS generated content module
- `contain`
- `content`

r CSS property

The **`r`** CSS property defines the radius of a circle. It can only be used with the SVG `<circle>` element. If present, it overrides the circle's `r` attribute.

> **Note:** The `r` property only applies to `<circle>` elements nested in an `<svg>`. It doesn't apply to other SVG elements or HTML elements or pseudo-elements.

Syntax

```
/* Length and percentage values */
r: 3px;
r: 20%;

/* Global values */
r: inherit;
r: initial;
r: revert;
r: revert-layer;
r: unset;
```

Values

The `<length>` and `<percentage>` values define the radius of the circle.

`<length>`

Absolute or relative lengths can be expressed in any unit allowed by the CSS `<length>` data type. Negative values are invalid.

`<percentage>`

Percentages refer to the normalized diagonal of the current SVG viewport, which is calculated as $\frac{\sqrt{<\text{width}>^2 + <\text{height}>^2}}{\sqrt{2}}$.

Formal definition

Initial value	`0`
Applies to	`<circle>` element in `<svg>`
Inherited	no
Percentages	refer to the normalized diagonal of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
r =
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the radius of a circle

In this example, we have two identical `<circle>` elements in an SVG, each with a `10` radius and the same `x`- and `y`-axis coordinates for their center points.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="10" />
  <circle cx="50" cy="50" r="10" />
</svg>
```

With CSS, we style only the first circle, allowing the second circle to use default styles (with `fill` defaulting to black). We use the `r` property to override the value of the SVG `r` attribute, giving it a `fill` and `stroke`. The default size of an SVG is `300px` wide and `150px` tall.

```
svg {
  border: 1px solid black;
}

circle:first-of-type {
  r: 30px;
  fill: lightgreen;
  stroke: black;
}
```

ViewBox versus viewport pixels

This example contains two SVGs, each with two `<circle>` elements. The second SVG includes a `viewBox` attribute to demonstrate the difference between SVG viewBox and SVG viewports.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="10" />
  <circle cx="50" cy="50" r="10" />
</svg>
<svg viewBox="0 0 200 100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="10" />
  <circle cx="50" cy="50" r="10" />
</svg>
```

The CSS is similar to the previous example, with `r: 30px` set, but we set a `width` to ensure the images are both `300px` wide:

```
svg {
  border: 1px solid black;
  width: 300px;
}

circle:first-of-type {
  r: 30px;
  fill: lightgreen;
  stroke: black;
}
```

Because the `viewBox` attribute defines the SVG as 200 SVG coordinate system pixels wide, and the image is scaled up to `300px`, the `30` SVG coordinate pixels are scaled to be rendered as `45` CSS pixels.

Defining the radius of a circle using percentages

In this example, we use the same markup as the previous example. The only difference is the `r` value; in this case, we use a percentage value.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="10" />
  <circle cx="50" cy="50" r="10" />
</svg>
<svg viewBox="0 0 200 100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="10" />
  <circle cx="50" cy="50" r="10" />
</svg>
```

```
svg {
  border: 1px solid black;
  width: 300px;
}

circle:first-of-type {
  r: 30%;
  fill: lightgreen;
  stroke: black;
}
```

In both cases, the circle radius is `30%` of the normalized diagonal of the SVG viewport. The radius `r` is equal to $0.3 \times \frac{\sqrt{\text{<width>}^2 + \text{<height>}^2}}{\sqrt{2}}$. While the first image is using `300` and `150` CSS pixels and the second is using `200` and `100` SVG view box units, 30% is a proportional value. As a result, the `r` value is the same: `47.43` viewBox units, which resolves to `71.15` CSS pixels.

While the `r` is the same, the center points differ because the second SVG is scaled up by 50%, pushing its center down and to the right by 50%.

See also

- Geometry properties: `r`, `cx`, `cy`, `rx`, `ry`, `x`, `y`, `width`, `height`
- `fill`
- `stroke`
- `paint-order`
- `border-radius` shorthand property
- `radial-gradient`
- `<basic-shape>` data type
- SVG `r` attribute

reading-flow CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`reading-flow`** CSS property enables modifying the reading order of child elements of a block, flex, or grid layout. This affects the order in which they are rendered to speech and navigated to when using sequential navigation such as tabbing to links or buttons.

Syntax

```
/* Keyword values */
reading-flow: normal;
reading-flow: flex-visual;
reading-flow: flex-flow;
reading-flow: grid-columns;
reading-flow: grid-rows;
reading-flow: grid-order;
reading-flow: source-order;

/* Global values */
reading-flow: inherit;
reading-flow: initial;
reading-flow: revert;
reading-flow: revert-layer;
reading-flow: unset;
```

Value

The `reading-flow` property takes one of the following keywords as its value:

`normal`

The default value. The reading order follows the order of elements in the DOM.

`flex-visual`

Only affects flex containers. The reading order follows the visual order of the `flex` items, taking the `writing-mode` into account. Therefore, a document in English with `flex-direction: row-reverse` and `reading-flow: flex-visual` set would have a reading order of left-to-right.

`flex-flow`

Only affects flex containers. The reading order follows the `flex-flow` direction.

`grid-columns`

Only affects grid containers. The reading order follows the visual order of grid items, column by column, taking the writing mode into account.

`grid-rows`

Only affects grid containers. The reading order follows the visual order of grid items, row by row, taking the writing mode into account.

`grid-order`

Only affects grid containers. if the `order` property is applied to any of the container's children, the reading order follows the modified item order. If the `order` property is not applied to the grid items, `grid-order` behaves as `normal`.

`source-order`

Affects grid, flex, and block containers. Has no effect on its own — the container's reading order continues to follow the order of elements in the DOM — but allows the reading order to be modified by setting the `reading-order` property on the container's children.

Description

The `reading-flow` property modifies the reading order of elements within a block, flex, or grid container when set to a value other than `normal`. Such a container is referred to as a reading flow container.

By default, web content is read out in DOM source order. Generally, the source order should express a sensible reading order for the content, and this should also be reflected by the visual order in the content layout. However, sometimes the visual order or tab order differs from the source order. For example, when applying multiple flexbox or grid layouts to a document via media queries to suit different device or user requirements, the content order may differ based on the viewport width. In such a case, the `reading-flow` can be used to ensure the reading and tabbing order reflect the visual order.

In some cases you may wish to further fine-tune the reading order within a reading flow container. You can use `reading-order` property values on the container's children, putting them into ordinal groups which are then read out in numerical order.

Interaction with <code>tabindex</code>

If a set of reading flow container child elements that are not normally focusable are made focusable with `tabindex="0"` attributes, their reading order will be modified as expected by the `reading-flow` and `reading-order` properties, in just the same way as interactive elements such as `<a>` or `<button>`.

However, any attempt to modify the tabbing order of a reading flow container's content using positive `tabindex` values will be ignored — overridden by the effects of `reading-flow` and `reading-order`. You generally shouldn't be using these anyway; see Don't Use Tabindex Greater than 0. The `reading-flow` and `reading-order` properties provide a much better way to modify tabbing order if required.

Formal definition

Initial value	`normal`
Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	as specified
Animation type	Not animatable

Examples

Flex value comparison

In this example, we demonstrate the effects of different `reading-flow` values on a flex container with reversed flex items.

HTML

The markup includes a `<select>` element for selecting different `reading-flow` values, and a wrapper `<div>` containing three `<a>` elements.

```
<form>
  <label for="reading-flow">Choose reading flow:</label>
  <select id="reading-flow">
    <option>normal</option>
    <option>flex-visual</option>
    <option>flex-flow</option>
  </select>
</form>
<div class="wrapper">
  <a href="#">Item 1</a>
  <a href="#">Item 2</a>
  <a href="#">Item 3</a>
</div>
```

CSS

We use a `display` value of `flex` to turn the `<div>` into a flex container, and display the flex items in a row in reverse DOM order with a `flex-direction` value of `row-reverse`. Initially, we set a `reading-flow` of `normal` so the items are read or tabbed through in DOM source order.

We also set an `order` value of `1` on the first `<a>` element to cause it to display after the second and third item in the flex flow. The resulting visual order of the flex items from left to right is "Item 1", "Item 3", then "Item 2", but the DOM order remains unchanged.

```
.wrapper {
  display: flex;
  flex-direction: row-reverse;
  reading-flow: normal;
  gap: 1em;
}

a:first-child {
  order: 1;
}
```

JavaScript

In our script, we grab references to the `<select>` element and wrapper `<div>`, then add a `change` event listener to the `<select>` element. When a new value is selected, it is set as a `reading-flow` property value on the wrapper.

```
const selectElem = document.getElementById("reading-flow");
const wrapperElem = document.querySelector(".wrapper");

selectElem.addEventListener("change", () => {
  wrapperElem.style.readingFlow = selectElem.value;
});
```

Result

The demo renders like this:

First, try tabbing through the links with `reading-flow: normal` set. The tabbing order is "Item 1", "Item 2", then "Item 3", as that is the order of the elements in the DOM.

Now try changing the `reading-flow` value and tabbing through the links again:

- A value of `flex-visual` causes the items to be tabbed through in the order "Item 1", "Item 3", then "Item 2", which is the visual display order resulting from the applied flexbox properties.
- A value of `flex-flow` causes the items to be tabbed through in the order "Item 2", "Item 3", then "Item 1", which matches the direction of the `flex-flow` — in this case, `row-reverse`. Here, the tabbing order is the reverse of the display order.

Grid value comparison

In this example, we demonstrate the effects of different `reading-flow` values on a grid container.

HTML

The markup includes a `<select>` element for selecting different `reading-flow` values, and a wrapper `<div>` containing four `<a>` elements.

```
<form>
  <label for="reading-flow">Choose reading flow:</label>
  <select id="reading-flow">
    <option>normal</option>
    <option>grid-rows</option>
    <option>grid-columns</option>
    <option>grid-order</option>
  </select>
</form>
<div class="wrapper">
  <a class="a" href="#">Item A</a>
  <a class="b" href="#">Item B</a>
  <a class="c" href="#">Item C</a>
  <a class="d" href="#">Item D</a>
</div>
```

CSS

We use a `display` value of `grid` to turn the `<div>` into a grid container, and display the grid items in three columns using `grid-template-columns`. We also set `grid-template-areas` to describe different placement areas in those columns, and place the `<a>` elements in those areas using `grid-area`. Initially, we set a `reading-flow` of `normal` so the items are read or tabbed through in the default DOM source order.

Finally, we set an `order` value of `1` on the first `<a>` element; this has no effect on the layout because it does not override the grid area placement, but it does have an effect when a certain `reading-flow` value is set, as you'll se later on.

Reading from left to right, the resulting display order of the grid items is "Item D", "Item B", "Item C", then "Item A".

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 150px);
  grid-template-areas:
```

```
    "d b b"
    "c c a";
  reading-flow: normal;
}

.a {
  grid-area: a;
}
.b {
  grid-area: b;
}
.c {
  grid-area: c;
}
.d {
  grid-area: d;
}

a:first-child {
  order: 1;
}
```

JavaScript

In our script, we grab references to the `<select>` element and wrapper `<div>`, then add a `change` event listener to the `<select>` element. When a new value is selected, it is set as a `reading-flow` property value on the wrapper.

```
const selectElem = document.getElementById("reading-flow");
const wrapperElem = document.querySelector(".wrapper");

selectElem.addEventListener("change", () => {
  wrapperElem.style.readingFlow = selectElem.value;
});
```

Result

The demo renders like this:

First, try tabbing through the links with `reading-flow: normal` set. The tabbing order is "Item A", "Item B", "Item C", and "Item D", as that is the order of the elements in the DOM.

Now change the `reading-flow` value and then try tabbing through the links again:

- A value of `grid-rows` causes the items to be tabbed through in the visual display order by row. This is "Item D", "Item B", "Item C", then "Item A".
- A value of `grid-columns` causes the items to be tabbed through in the visual display order by column. This is "Item D", "Item C", "Item B", then "Item A".
- A value of `grid-order` causes the items to be tabbed through in DOM order, but takes into account any `order` value changes. Since we have `order: 1;` set on the first `<a>` element, the tabbing order is "Item B", "Item C", "Item D", then "Item A".

Reading order adjustment on block containers

In this example, we demonstrate the effects of the `reading-flow: source-order` value on a block container.

HTML

The markup includes a wrapper `<div>` containing four `<a>` elements.

```
<div class="wrapper">
  <a class="a" href="#">Item A</a>
  <a class="b" href="#">Item B</a>
  <a class="c" href="#">Item C</a>
  <a class="d" href="#">Item D</a>
</div>
```

CSS

We set a `reading-flow` of `source-order` so the items are read or tabbed through in DOM source order, but modifications are allowed to the reading order via `reading-order`. We set a `reading-order` value of `1` on the first `<a>` element.

```
.wrapper {
  reading-flow: source-order;
}

a:first-child {
  reading-order: 1;
}
```

Result

The demo renders like this:

Try tabbing through the links: the tabbing order is "Item B", "Item C", "Item D", then "Item A" — the order of the elements in the DOM is followed, but Item A has been put in a higher reading order ordinal group than the others (the default `reading-order` value is `0`), therefore it is tabbed to last.

See also

- `reading-order`
- CSS `reading-flow` examples via chrome.dev

reading-order CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`reading-order`** CSS property enables changing the order in which a child of a reading flow container is read relative to its element siblings.

Syntax

```
/* <integer> values */
reading-order: 1;
reading-order: -1;

/* Global values */
reading-order: inherit;
reading-order: initial;
reading-order: revert;
reading-order: revert-layer;
reading-order: unset;
```

Value

`<integer>`

Represents the ordinal group the element belongs to.

Description

When an element's block, flex, or grid parent container has a `reading-flow` property set to a value other than `normal`, the `reading-order` property can be used to set the element's reading flow position relative to that of its siblings.

For reading and navigation, elements inside a block, flex, or grid container are sorted by ascending `reading-order` value. If multiple siblings have the same `reading-order` value, those items are sorted according to the container's `reading-flow`. Siblings not given an explicit `reading-order` value are assigned the default value of `0`, which puts all the children of a reading flow container in the same ordinal group by default.

Sibling elements are ordered starting from the lowest numbered ordinal group to the highest. Therefore, to cause an element to be read out after its siblings, you could set a `reading-order` value or `1` or more on it. To cause an element to be read out before its siblings, you could set a `reading-order` value or `-1` or less on it.

The `reading-order` defines the reading and tabbing order, but has no effect on visual order.

Interaction with <code>tabindex</code>

If a set of reading flow container child elements that are not normally focusable are made focusable with `tabindex="0"` attributes, their reading order will be modified as expected by the `reading-flow` and `reading-order` properties, in just the same way as interactive elements such as `<a>` or `<button>`.

However, any attempt to modify the tabbing order of a reading flow container's content using positive `tabindex` values will be ignored — overridden by the effects of `reading-flow` and `reading-order`. You generally shouldn't be using these anyway; see Don't Use Tabindex Greater than 0. The `reading-flow` and `reading-order` properties provide a much better way to modify tabbing order if required.

Formal definition

Initial value	`0`
Applies to	Block-containers, flex containers, and grid containers
Inherited	no
Computed value	The specified integer
Animation type	by computed value type

Examples

Grid row order

This example demonstrates using `reading-order` to control the positions of individual grid items within a grid container's reading order. One grid item has a `reading-order` value that is lower than the default `0`, so will be read out before its sibling elements. Another has a higher `reading-order` value set, so will be read out after the others, regardless of source or display order.

HTML

In our markup we have six `<a>` elements contained inside a wrapper `<div>`.

```
<div class="wrapper">
  <a href="#">Item 1</a>
  <a class="bottom" href="#">Item 2</a>
  <a href="#">Item 3</a>
  <a class="top" href="#">Item 4</a>
  <a href="#">Item 5</a>
  <a href="#">Item 6</a>
</div>
```

CSS

On the `<div>`, we set the grid-auto-flow property to `dense`, therefore items may display out of source order. The `reading-order` property on the `<a>` element with a class of `top` is set to `-1`, therefore "Item 4" will be the first item in reading flow. The `reading-order` property on the `<a>` element with a class of `bottom` is set to `21`, therefore "Item 4" will be the last item in the reading order. The remaining items will be visited in between, in grid row order, as the `<div>` element's `reading-flow` property is set to grid-rows.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 150px);
  grid-auto-flow: dense;
  reading-flow: grid-rows;
}

.top {
  reading-order: -1;
}

.bottom {
  reading-order: 21;
}
```

Result

The above demo renders as follows:

Try tabbing through the links. Note how "Item 4" is tabbed to first and "Item 2" is tabbed to last because of their modified `reading-order` values. In between, the items are tabbed to in grid row order.

Source order override

In this example, the odd-numbered items have a lower `reading-order` value set, so will be read out before the others in a group, regardless of source or display order.

HTML

In our markup, we have five `<a>` elements contained inside a wrapper `<div>`.

```
<div class="wrapper">
  <a href="#">Item 1</a>
  <a href="#">Item 2</a>
  <a href="#">Item 3</a>
  <a href="#">Item 4</a>
  <a href="#">Item 5</a>
</div>
```

CSS

The `<div>` element's `reading-flow` property is set to `source-order`, which allows `reading-order` to be used to override the default source reading order. The odd-numbered `<a>` elements have a `reading-order` value of `-1`, therefore they are read out before the even-numbered items.

```
.wrapper {
  reading-flow: source-order;
}

.wrapper > a {
  display: block;
}

.wrapper a:nth-child(odd) {
  reading-order: -1;
}
```

Result

The above demo renders as follows:

Try tabbing through the links, and note how "Item 1", "Item 3", and "Item 5" are tabbed to first because of their modified `reading-order`. After that, the items are tabbed to in source order.

See also

- `reading-flow`
- CSS `reading-flow` examples via chrome.dev

resize CSS property

The **`resize`** CSS property sets whether an element is resizable, and if so, in which directions.

Try it

```
resize: both;
```

```
resize: horizontal;
```

```
resize: vertical;
```

```
resize: none;
```

```
<section class="default-example" id="default-example">
  <div id="example-element">Try resizing this element.</div>
</section>
```

```
#example-element {
  background: linear-gradient(135deg, cyan 0%, cyan 94%, white 95%);
  border: 3px solid #c5c5c5;
  overflow: auto;
  width: 250px;
  height: 250px;
  font-weight: bold;
  color: black;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 10px;
}
```

`resize` does not apply to the following:

- Inline elements
- Block elements for which the `overflow` property is set to `visible` or `clip`

Syntax

```
/* Keyword values */
resize: none;
resize: both;
resize: horizontal;
resize: vertical;
resize: block;
resize: inline;

/* Global values */
resize: inherit;
resize: initial;
resize: revert;
resize: revert-layer;
resize: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

The element offers no user-controllable method for resizing it.

`both`

The element displays a mechanism for allowing the user to resize it, which may be resized both horizontally and vertically.

`horizontal`

The element displays a mechanism for allowing the user to resize it in the *horizontal* direction.

`vertical`

The element displays a mechanism for allowing the user to resize it in the *vertical* direction.

`block`

The element displays a mechanism for allowing the user to resize it in the *block* direction (either horizontally or vertically, depending on the `writing-mode` and `direction` value).

`inline`

The element displays a mechanism for allowing the user to resize it in the *inline* direction (either horizontally or vertically, depending on the `writing-mode` and `direction` value).

Formal definition

Initial value	none
Applies to	elements with `overflow` other than `visible`, and optionally replaced elements representing images or videos, and iframes
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
resize =
  none        |
  both        |
  horizontal  |
  vertical    |
  block       |
  inline
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling resizability of text areas

In many browsers, `<textarea>` elements are resizable by default. You may override this behavior with the `resize` property.

HTML

```
<textarea>Type some text here.</textarea>
```

CSS

```
textarea {
  resize: none; /* Disables resizability */
}
```

Result

Using resize with arbitrary elements

You can use the `resize` property to make any element resizable. In the example below, a resizable `<div>` contains a resizable paragraph (`<p>` element).

HTML

```
<div class="resizable">
  <p class="resizable">
    This paragraph is resizable in all directions, because the CSS `resize`
    property is set to `both` on this element.
  </p>
</div>
```

CSS

```
.resizable {
  resize: both;
  overflow: scroll;
  border: 1px solid black;
}

div {
  height: 300px;
  width: 300px;
}

p {
  height: 200px;
  width: 200px;
}
```

Result

See also

- `<textarea>`

right CSS property

The **`right`** CSS property participates in specifying the horizontal position of a positioned element. This inset property has no effect on non-positioned elements.

Try it

```
right: 0;
```

```
right: 4em;
```

```
right: 10%;
```

```
right: 20px;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #264653;
  border: 4px solid #ffb500;
  color: white;
  position: absolute;
  width: 140px;
  height: 60px;
}
```

Syntax

```
/* <length> values */
right: 3px;
right: 2.4em;
right: anchor(--my-anchor 50%);
right: anchor-size(--my-anchor height, 65px);

/* <percentage>s of the width of the containing block */
right: 10%;

/* Keyword value */
right: auto;

/* Global values */
right: inherit;
right: initial;
right: revert;
right: revert-layer;
right: unset;
```

Values

`<length>`

A negative, null, or positive `<length>`:

- for *absolutely positioned elements*, it represents the distance to the right edge of the containing block.
- for *anchor-positioned elements*, the `anchor()` function resolves to a `<length>` value relative to the position of the associated *anchor element*'s left or right edge (see Using inset properties with `anchor()` function values), and the `anchor-size()` function resolves to a `<length>` value relative to the associated anchor element's width or height (see Setting element position based on anchor size).
- for *relatively positioned elements*, it represents the distance that the element is moved to the left of its normal position.

`<percentage>`

A `<percentage>` of the containing block's width.

`auto`

Specifies that:

- for *absolutely positioned elements*, the position of the element is based on the `left` property, while `width: auto` is treated as a width based on the content; or if `left` is also `auto`, the element is positioned where it should horizontally be positioned if it were a static element.
- for *relatively positioned elements*, the distance of the element from its normal position is based on the `left` property; or if `left` is also `auto`, the element is not moved horizontally at all.

Description

The effect of `right` depends on how the element is positioned (i.e., the value of the `position` property):

- When `position` is set to `absolute` or `fixed`, the `right` property specifies the distance between the element's outer margin of right edge and the inner border of the right edge of its containing block. If the positioned element has an associated *anchor element*, and the property value includes an `anchor()` function, `right` positions the right edge of the positioned element relative to the specified `<anchor-side>` edge. The `right` property is compatible with the `left`, `right`, `start`, `end`, `self-start`, `self-end`, `center`, and `<percentage>` values.
- When `position` is set to `relative`, the `right` property specifies the distance the element's right edge is moved to the left from its normal position.
- When `position` is set to `sticky`, the `right` property is used to compute the sticky-constraint rectangle.
- When `position` is set to `static`, the `right` property has *no effect*.

When both `left` and `right` are defined, if not prevented from doing so by other properties, the element will stretch to satisfy both. If the element cannot stretch to satisfy both — for example, if a `width` is declared — the position of the element is *over-constrained*. When this is the case, the `left` value has precedence when the container is left-to-right; the `right` value has precedence when the container is right-to-left.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	refer to the width of the containing block
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length, percentage or calc();

Formal syntax

```
right =
  auto                |
  <length-percentage>  |
  <anchor()>           |
  <anchor-size()>

<length-percentage> =
  <length>     |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                 &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside       |
  outside      |
  top          |
  left         |
  right        |
  bottom       |
  start        |
  end          |
  self-start   |
  self-end     |
  <percentage>  |
  center

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Absolute and relative positioning using right

HTML

```
<div id="relative">Relatively positioned</div>
<div id="absolute">Absolutely positioned</div>
```

CSS

```
#relative {
  width: 100px;
  height: 100px;
  background-color: #ffc7e4;
  position: relative;
  top: 20px;
  left: 20px;
}

#absolute {
  width: 100px;
  height: 100px;
  background-color: #ffd7c2;
  position: absolute;
  bottom: 10px;
  right: 20px;
}
```

Result

Declaring both left and right

When both `left` and `right` are declared, the element will stretch to meet both, unless other constraints prevent it from doing so. If the element will not stretch or shrink to meet both. When the position of the element is *overspecified*, the precedence is based on the container's direction: The `left` will take precedence if the container's direction is left-to-right. The `right` will take precedence if the container's direction is right-to-left.

HTML

```
<div id="parent">
  Parent
  <div id="noWidth">No width</div>
  <div id="width">width: 100px</div>
</div>
```

CSS

```
div {
  outline: 1px solid #cccccc;
}
#parent {
  width: 200px;
  height: 200px;
  background-color: #ffc7e4;
  position: relative;
}
/* declare both a left and a right */
#width,
#noWidth {
  background-color: #c2ffd7;
  position: absolute;
  left: 0;
  right: 0;
}
/* declare a width */
#width {
  width: 100px;
  top: 60px;
}
```

Result

See also

- `top`, `bottom`, and `left`
- `inset` shorthand
- `inset-block-start`, `inset-block-end`, `inset-inline-start`, and `inset-inline-end`
- `inset-block` and `inset-inline` shorthands
- `position`
- CSS positioned layout module

rotate CSS property

The **`rotate`** CSS property allows you to specify rotation transforms individually and independently of the `transform` property. This maps better to typical user interface usage, and saves having to remember the exact order of transform functions to specify in the `transform` property.

Try it

```
rotate: none;
```

```
rotate: -45deg;
```

```
rotate: z 45deg;
```

```
rotate: x 45deg;
```

```
rotate: y 45deg;
```

```
rotate: 3 0.5 2 45deg;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 550px;
}

#example-element {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
```

```
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Syntax

```
/* Keyword value */
rotate: none;

/* Angle value */
rotate: 90deg;
rotate: 0.25turn;
rotate: 1.57rad;

/* x, y, or z axis name plus angle */
rotate: x 90deg;
rotate: y 0.25turn;
rotate: z 1.57rad;

/* Vector plus angle value */
rotate: 1 1 1 90deg;

/* Global values */
rotate: inherit;
rotate: initial;
rotate: revert;
rotate: revert-layer;
rotate: unset;
```

Values

angle value

An `<angle>` specifying the angle to rotate the affected element through, around the Z axis. Equivalent to a `rotate()` (2D rotation) function.

x, y, or z axis name plus angle value

The name of the axis you want to rotate the affected element around (`x`, `y`, or `z`), plus an `<angle>` specifying the angle to rotate the element through. Equivalent to a `rotateX()`/`rotateY()`/`rotateZ()` (3D rotation) function.

vector plus angle value

Three `<number>`s representing an origin-centered vector that defines a line around which you want to rotate the element, plus an `<angle>` specifying the angle to rotate the element through. Equivalent to a `rotate3d()` (3D rotation) function.

none

Specifies that no rotation should be applied.

Formal definition

Initial value	none
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	a transform
Creates stacking context	yes

Formal syntax

```
rotate =
  none                                  |
  <angle>                               |
  [ x | y | z | <number>{3} ] && <angle>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotating an element on hover

The following example shows how to use the `rotate` property to rotate an element along various axes on hover. The first box rotates 90 degrees on the Z axis hover, the second rotates 180 degrees on the Y axis on hover, and the third rotates 360 degrees on hover around a vector defined by coordinates.

HTML

```
<div class="box" id="box1">rotate Z</div>
<div class="box" id="box2">rotate Y</div>
<div class="box" id="box3">vector & angle</div>
```

CSS

```
.box {
  display: inline-block;
  margin: 1em;
  min-width: 6.5em;
  line-height: 6.5em;
  text-align: center;
  transition: 1s ease-in-out;
  border: 0.25em dotted;
}

#box1:hover {
  rotate: 90deg;
}

#box2:hover {
  rotate: y 180deg;
}

#box3:hover {
  rotate: 1 2 1 360deg;
}
```

Result

See also

- `translate`
- `scale`
- `transform`

Note: `skew` is not an independent `transform` value

row-gap CSS property

The **`row-gap`** CSS property sets the size of the gap (gutter) between an element's rows in multi-column, flexible box, and grid layouts.

Try it

```
row-gap: 0;
```

```
row-gap: 1ch;
```

```
row-gap: 1em;
```

```
row-gap: 20px;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  width: 200px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Keyword value */
row-gap: normal;

/* <length-percentage> value */
row-gap: 20px;
row-gap: 1em;
row-gap: 3vmin;
row-gap: 0.5cm;
row-gap: 10%;
row-gap: calc(10% - 6px);

/* <line-width> values */
```

```
row-gap: thin;
row-gap: medium;
row-gap: thick;

/* Global values */
row-gap: inherit;
row-gap: initial;
row-gap: revert;
row-gap: revert-layer;
row-gap: unset;
```

Values

This property is specified as a single value from the following list:

`normal`

For multi-column layout, resolves to `1em`; otherwise `0`. This is the default value.

`<line-width>`

Sets the size of the gap using the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value.

`<length-percentage>`

Sets a non-negative `<length>` or `<percentage>` value. Percentages are relative to the block-size of the content box or `0`.

Description

The `row-gap` property sets the size of the gap between an element's rows. This gap may contain a visible separator as a gap decoration. If there is a rule between rows, it will appear in the middle of the gap, but has no impact on the gap size. These decorative lines can be added to the otherwise "empty space" by using the `row-rule` property or `rule` shorthand.

Defined in CSS gaps, the property can be used in multi-column, flexible box, and grid layouts. The `row-gap`, along with the `column-gap` property, can also be set using the `gap` shorthand property, in that order. The `row-gap` property replaced the `grid-row-gap` property, which was limited to CSS grid layouts. Now `grid-row-gap` is an alias for `row-gap`.

The property specifies a fixed-length gutter between items in a container, separating boxes in the container's block axis. Negative values are invalid. The default value `normal` resolves to `1em` on multi-column containers, and `0` everywhere else. See Defining CSS gaps for more information about gaps by layout type.

Percentages resolve against the content box size of the container element's block axis when this size is definite, against `0` otherwise, except in grid layout, for which cyclic percentage sizes resolve against zero for determining intrinsic size contributions but resolve against the element's content box when laying out the contents. See Specifying gap values as percentages for more information.

In grid layouts, the effect of the gap is as though the grid lines between grid rows acquired the thickness of the property's value: the grid track between two rows is the space between the gutters that represent them. When it comes to track sizing, each gutter is treated as an extra, empty, fixed-size track of the specified size, which is spanned by any grid items that spans across more than one row. While treated as empty for sizing, the gap created may contain a `row-rule`.

Formal definition

Initial value	`normal`
Applies to	multi-column elements, flex containers, grid containers
Inherited	no
Percentages	refer to corresponding dimension of the content area
Computed value	as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements
Animation type	a length, percentage or calc();

Formal syntax

```
row-gap =
  normal                    |
  <length-percentage [0,∞]>  |
  <line-width>

<length-percentage> =
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Flex layout

This example demonstrates using the `row-gap` property to create horizontal space between adjacent rows of flex items. It also demonstrates how the `row-gap` size is not affected by the size of the row rule.

HTML

We include six items in a container element:

```
<div id="flexbox">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

We set the `display` to `flex`, set the `flex-flow` to `row wrap` to create a flex container with rows of flex items that flow onto new lines as needed, and limit the `width` to `300px`. We also add a `row-rule`, which will draw a 30px-wide, dashed, magenta line in the middle of the gap.

The `row-gap` value is set as `20px` on the flex container to create a `20px` gap between the adjacent flex rows.

We also set a background color on the flex items, with most being semi-opaque, to demonstrate how the rule is visible under the flex items when it is wider than the gap.

```
#flexbox {
  display: flex;
  flex-flow: row wrap;
  width: 300px;
  row-rule: 30px dashed magenta;

  row-gap: 20px;
}

#flexbox > div {
  border: 1px solid green;
  background-color: #00ff0033;
  flex: 1 1 100px;
  height: 50px;
}
#flexbox > div:nth-of-type(3n-1) {
  background-color: lime;
}
```

Result

To set vertical space between flex items, specify a non-zero value for the `column-gap` property, optionally setting both `row-gap` and `column-gap` by using the `gap` shorthand.

Grid layout

This example demonstrates using the `row-gap` property with a `<percentage>` value in a grid layout.

HTML

We include five items in a container element:

```
<div id="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

We set the `display` property to `grid`, the `height` to `240px`, the `width` to `350px` and `grid-template-rows` to `repeat(3, 1fr)` to create a 350px-wide grid container with three columns and as many rows as needed. Each row is `100px` tall, as defined by the `grid-template-rows` property.

The `row-gap` is set to `5%`. The container height is `240px`. The `5%` value creates a row gap that is `12px` tall, leaving `216px` for three rows of grid items, meaning each row is `72px` tall.

```
#grid {
  display: grid;
  height: 240px;
  width: 350px;
  grid-template-rows: repeat(3, 1fr);
  grid-template-columns: 150px 1fr;

  row-gap: 5%;
}
```

```
body {
  padding: 1em;
}

#grid > div {
  outline: 1px solid green;
  background-color: lime;
}
@layer no-support {
  @supports not (row-gap: 5%) {
    body::before {
      content: "Your browser doesn't support percent values";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

See also

- `column-gap`
- `row-rule`
- `rule`
- `gap`
- Defining CSS gaps
- Basic concepts of grid layout: gutters
- CSS gaps module

row-rule CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule`** CSS shorthand property sets the width, style, and color of the line drawn between rows in multi-row grid, flex, and multi-col layouts.

Try it

```
row-rule: solid;
```

```
row-rule: dotted medium blue;
```

```
row-rule:
  dotted medium blue,
  repeat(3, dashed magenta 1px, outset green 5px);
```

```
row-rule:
  dotted medium blue,
  repeat(auto, dashed magenta 1px, dashed magenta 5px),
  dotted medium blue;
```

```
row-rule:
  dotted medium blue,
  repeat(auto, dashed magenta 1px),
  outset green 5px;
```

```
<section id="default-example">
  <ul id="example-element">
    <li>One fish</li>
    <li>Two fish</li>
    <li>Red fish</li>
    <li>Blue fish</li>
  </ul>
</section>
```

```
#example-element {
  display: flex;
  flex-flow: column;
  gap: 7px;
  text-align: left;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `row-rule-color`
- `row-rule-style`
- `row-rule-width`

Syntax

```
/* One value */
row-rule: dotted;
row-rule: solid 8px;
row-rule: solid blue;
row-rule: thick inset blue;

/* Multiple values */
row-rule: groove, dashed, solid;
row-rule:
  dotted medium blue,
  dashed magenta 1px,
  outset green 5px;
row-rule:
  solid #0ff,
  repeat(3, dashed magenta 1px, outset green 5px);
row-rule:
  inset 3px yellow,
  repeat(auto, dashed magenta 1px, groove green 5px);

/* Global values */
row-rule: inherit;
row-rule: initial;
row-rule: revert;
row-rule: revert-layer;
row-rule: unset;
```

Values

This property is specified as a comma-separated list of values. Each value can be one of the following value types:

`<gap-rule>`

Specified as one, two, or three of the values listed below, in any order.

`<'line-width'>`

A `<line-width>`: a positive `<length>` or one of the three keywords, `thin`, `medium`, or `thick`. The default value is `medium`. See `row-rule-width`.

`<'line-style'>`

A `<line-style>`: one of `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`. The default value is `none`. See `row-rule-style`.

`<'color'>`

A `<color>` value representing the color of the line. The default value is `currentcolor`. See `row-rule-color`.

`<gap-repeat-rule>`

A `repeat()` function, with an `<integer>` of `1` or more as the first argument and one or more `<gap-rule>` values as subsequent arguments. The `<integer>` specifies how many times the list of `<gap-rule>` values should be repeated.

`<gap-auto-repeat-rule>`

A `repeat()` function, with `auto` as the first argument and one or more `<gap-rule>` values as subsequent arguments. The provided list of `<gap-rule>` values is repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value.

Description

The `row-rule` property defines the line style of any rule lines drawn in the gaps between rows in multi-column, flex, and grid containers with more than one row.

The `row-rule` is shorthand for `row-rule-color`, `row-rule-style`, and `row-rule-width`. The `row-rule`, along with the `column-rule` shorthand property, can also be set using the `rule` shorthand.

The property value is a comma-separated list of components, which can include `<gap-rule>`, `<gap-repeat-rule>`, and `<gap-auto-repeat-rule>` types. Each `<gap-rule>` defines the width, color, and style of one or more rule lines.

If the property value consists of only one `<gap-rule>`, all the row rules will be that style. If we declare the following, all row rules will be `dashed red 3px`:

```
row-rule: dashed red 3px;
```

When more than one `<gap-rule>` is declared, they will be applied to row-rules in the order specified. If there are more gutters between rows than `<gap-rule>` values, the list of values is repeated until every row rule has a gap line. If we declare the following, for example, every odd rule will be `dashed red 3px`, and every even rule will be `dotted blue 5px`.

```
row-rule:
  dashed red 3px,
  dotted blue 5px;
```

Repeated line styles

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<gap-rule>` values passed as subsequent arguments the specified number of times. This allows the same `<gap-rule>` to be repeated a set number of times without repeating the same CSS multiple times. The following declarations are equivalent:

```
row-rule:
  solid red 5px,
  outset blue 10px,
  inset green 1px,
  outset blue 10px,
  inset green 1px,
  outset blue 10px,
  inset green 1px;
row-rule:
  solid red 5px,
  repeat(3, outset blue 10px, inset green 1px);
```

This creates a list of seven rules. If the number of styles in the `row-rule` value's style list exceeds the number of gaps between rows, the excess style values are ignored. If the container on which this is applied has three rows, the rule in the first gutter will be `solid red 5px` and the second `outset blue 10px`.

If there are more gutters than styles, the list of styles is repeated. If the container has 8, 15, 22, or 29 rows, this sequence of styles will be repeated one, two, three, or four times, respectively, with the last rule being `inset green 1px`.

Auto-repeating line styles

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<gap-rule>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

```
row-rule:
  solid red 5px,
  repeat(auto, dotted green 1px, dashed blue 1px),
  solid red 5px;
```

In this case, the first and last row rules will be `solid red 5px`, and all others will alternate between `dotted green 1px` and `dashed blue 1px`. It doesn't matter if the container has 3, 6, 11, 16, or 21 rows; the first and last gutters will always have a thick solid red line painted between them (unless `row-rule-visibility-items` leads to no line being drawn), and all the other row rules will be thin, dotted green or dashed blue lines. If there are only 2 or 3 rows, there will be no dotted or dashed lines.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for row rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. At most, only one `repeat(auto, <gap-rule>)` can be present in a `row-rule` value.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule =
  <gap-rule-list>       |
  <gap-auto-rule-list>

<gap-rule-list> =
  <gap-rule-or-repeat>#

<gap-auto-rule-list> =
  <gap-rule-or-repeat>#? , <gap-auto-repeat-rule> , <gap-rule-or-repeat>#?

<gap-rule-or-repeat> =
  <gap-rule>         |
  <gap-repeat-rule>

<gap-auto-repeat-rule> =
  repeat( auto , <gap-rule># )

<gap-rule> =
  <line-width>  ||
  <line-style>  ||
  <color>

<gap-repeat-rule> =
  repeat( <integer [1,∞]> , <gap-rule># )

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
```

```
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single rule for the lines drawn between flex items.

HTML

We include a list of dynamic sports duos:

```
<ul>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
  <li>Kerri Walsh + Misty May Treanor</li>
</ul>
```

CSS

We define the list to be a flex container, creating rows by setting the `flex-direction` to `column` using the `flex-flow` shorthand. We include a `gap` of `5px` to provide enough room between the rows to fit our `3px dashed red` rule:

```
ul {
  display: flex;
  flex-flow: column;
  gap: 5px;

  row-rule: 3px red dashed;
}
```

Result

Repeating values

This example demonstrates how the values are repeated when there are fewer values in the list of styles than row rules. It also demonstrates the default values for the width, color, and style of `medium`, `currentcolor` and `none`, respectively.

CSS

Using the same HTML and CSS as in the previous example, we include four comma-separated `<gap-rule>` values as the `row-rule` value, omitting the width in the first `<gap-rule>`, the color in the second, and the style from the third, with the fourth including all three components:

```
ul {
  row-rule:
    red dashed,
    1px dotted,
    5px blue,
    10px magenta solid;
}
```

Result

The red line is `3px` wide, the dotted line is the same color as the text, and there is no `5px`-wide blue line, as the style of the third `<gap-rule>` defaults to `none`, so no line is painted.

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `row-rule` property value.

CSS

We use the same HTML and CSS as in the previous examples. We include a `repeat()` function, setting the list of two `<gap-rule>` values to repeat 3 times.

```
ul {
  row-rule:
    3px red dashed,
    repeat(3, dotted green 1px, dashed blue 1px),
    3px red dashed;
}
```

Result

The flex container has six rows, so five gutters. The `repeat()` function repeats two style values three times, creating a list of eight style values. As there are fewer row gutters than total gap-rules, the last three values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using the `auto` argument instead of an integer in the `repeat()` function.

CSS

Using `repeat(auto, <gap-rule>)`, we set all row rules to `1px dotted` and default to the current color, except the first and last, which we set to `3px solid red`.

```
ul {
  row-rule:
    3px red solid,
    repeat(auto, 1px dotted),
    3px red solid;
}
```

Result

```
@layer no-support {
  @supports not (row-rule: thin, thick) {
    body::before {
      content: "Your browser doesn't support the row-rule property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `row-rule-color`
- `row-rule-width`
- `row-rule-style`
- `column-rule` shorthand
- `rule` shorthand
- CSS gaps module

row-rule-break CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-break`** CSS property sets the behavior for breaking row rules into segments where row rules intersect column gaps.

Try it

```
row-rule-break: none;
```

```
row-rule-break: normal;
```

```
row-rule-break: intersection;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  row-rule: solid thick orange;
  column-rule: solid thick lavender;
  gap: 15px;
}
#example-element i {
  padding: 5px;
}
```

Syntax

```
/* Keyword values */
row-rule-break: none;
row-rule-break: normal;
```

```
row-rule-break: intersection;

/* Global values */
row-rule-break: inherit;
row-rule-break: initial;
row-rule-break: revert;
row-rule-break: revert-layer;
row-rule-break: unset;
```

Values

This property is specified as a single keyword from the following list:

`none`

There are no breaks in row rules when they intersect column gaps; rather, a continuous row rule is painted the whole width of the container, from edge to edge.

`normal`

In grid, flex containers, and multi-col layout, behaves as `none`. This is the default value.

`intersection`

Row rules always break when they intersect column gaps, with row rule segments starting and ending at container and gap edges.

Description

The `row-rule-break` property specifies whether or not to break row rules into segments when they cross column gaps.

Row rules are painted within a row gap as one or more segments, with segments occurring between adjacent grid items in separate rows, between flex items or flex lines, depending on the `flex-direction` in flex layouts, or in gaps between adjacent rows of columns in multi-col layouts when `column-height` creates multiple rows of columns.

The `row-rule-break` property only determines if the break occurs. By default, the break, or space, between row rule segments is the width of the column gap, as each segments starts and ends at the edge of the gap (or edge of the container). If the gap is `0`, this break may not be visible. The end positions can be controlled with the `row-rule-inset` properties.

If `row-rule-break` is set to `none`, there are no breaks, the row rule line is continuous, and any `row-rule-inset` values only affect the row rule on the left and right edge of the container. When there are breaks, the `row-rule-inset` properties affect the start and end of every row rule segment.

The `row-rule-break` property, along with the `column-rule-break` property, can be set using the `rule-break` shorthand.

Whether a row rule is by default composed of a single continuous segment or segments that break when intersecting column gaps depends on the container type.

Grid containers

In grid containers, by default, row rule segments continue through visible "cross" intersections. Setting `row-rule-break: intersection` will force the segments to break at every column gap, where they would otherwise cross.

```
<h1>Default rule breaks in grid</h1>
<div class="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
<p>
  <label
    ><input type="checkbox" /> Set
    <code>row-rule-break: intersection</code></label
  >
</p>
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}

:has(:checked) .grid {
  row-rule-break: intersection;
}
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  rule: 5px solid blue;
  column-rule-color: lightblue;
  width: 100%;
}

.grid > div {
  border: 1px solid green;
  background-color: lime;
  height: 30px;
}

@layer no-support {
  @supports not (row-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the row-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
```

```
  }
}
```

By default, there are no row rule breaks. Check the checkbox to set `ow-rule-break` to `intersection`, which makes the continuous rules break at every "cross" intersection. By default, the break between segments is the width of the `column-gap`.

Flex containers

In flexbox, when the `flex-direction` is `row` or `row-reverse` in horizontal writing modes, the row rule is continuous, with the column segments starting and ending at the at the edge of the row gaps. When the `flex-direction` is `column` or `column-reverse`, the column rule is continuous, with the row segments starting and ending at the at the edge of the column gaps.

```
<h1>Default rule breaks in flexbox</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
<p>
  <label
    ><input type="checkbox" /> Set
    <code>row-rule-break: intersection</code></label
  >
</p>
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}

:has(:checked) .flexbox {
  row-rule-break: intersection;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox {
  display: flex;
  flex-flow: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid blue;
  column-rule-color: lightblue;
  width: 100%;
}
.column {
  flex-flow: column balance;
  gap: 20px;
}

.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}

@layer no-support {
  @supports not (row-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the row-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

In horizontal writing modes, when set to `row` or `row-reverse`, the row rule is continuous by default, where as with `column` or `column-reverse`, the row rule breaks at every column gap. Setting the `row-rule-break` to `intersection` only impacts the row rules in the `row` and `row-reverse` scenarios.

Multi-col containers

In multi-col containers, the default `normal` behavior behaves the same as `none`.

```
<h1>Default rule breaks in multi-col</h1>
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  <li>Black fish</li>
  <li>Blue fish</li>
  <li>Old fish</li>
  <li>New fish.</li>
  <li>This one has a little star.</li>
  <li>This one has a little car.</li>
  <li>Say! What a lot</li>
  <li>Of fish there are.</li>
  <li>Yes. Some are blue.</li>
  <li>And some are blue.</li>
  <li>Some are old.</li>
  <li>And some are new.</li>
  <li>Some are sad.</li>
  <li>And some are glad.</li>
  <li>And some are very, very bad.</li>
  <li>Why are they</li>
  <li>Sad and glad and bad?</li>
  <li>I do not know.</li>
  <li>Go ask your dad.</li>
</ol>
<fieldset>
  <legend>Set <code>row-rule-break:</code></legend>
  <label
    ><input type="radio" name="break" value="none" /> <code>none</code></label
  >
  <label
    ><input type="radio" name="break" value="normal" checked />
    <code>normal</code></label
  >
  <label
    ><input type="radio" name="break" value="intersection" />
    <code>intersection</code></label
  >
</fieldset>
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
ol {
  columns: 3 / 4em;
  gap: 20px;
  rule: 5px solid blue;
  column-rule-color: lightblue;
}
li {
  border: 1px solid green;
  background-color: lime;
  list-style-type: none;
  margin-bottom: 5px;
}
:has([value="normal"]:checked) ol {
  row-rule-break: normal;
}
:has([value="intersection"]:checked) ol {
  row-rule-break: intersection;
}
:has([value="none"]:checked) ol {
  row-rule-break: none;
}
label {
  margin-right: 20px;
}
@layer no-support {
  @supports not (row-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the row-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

If you select `intersection`, the row rule will break into segments whenever it reaches a column gap, with each segment starting and ending at the edge of the gap. The start and end positions can be changed with the `row-rule-inset` properties.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-break =
  none          |
  normal        |
  intersection
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we use the `row-rule-break` property to break each rule segment in a grid container so row rule segments are created between column gaps. Changing the `column-gap` property will change the size of the segments.

HTML

We create a list of 50 items and a slider to change the column gap width. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>49</li>
  <li>50</li>
</ul>
```

```
<p>
  <label
    >Change the width of the column gap.
    <input type="range" min="0" max="32" value="16" id="gap"
  /></label>
  <output id="o"></output>
</p>
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
</ul>
```

CSS

We define the unordered list as a 8-column container, creating columns and rows with the `grid-template-columns` property and setting `list-style-type` to `none` to remove the bullets. We include a `gap` of `20px` to provide enough room between the columns and rows to fit our `20px` solid column and row rules. Last, we set the row rules to break into segments at every intersection.

```
ul {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  list-style-type: none;
  gap: 20px;

  row-rule: 10px solid olive;
  column-rule: 10px solid palegoldenrod;
```

```
  row-rule-break: intersection;
}
```

The rest of the CSS is hidden for brevity.

```
ol {
  place-items: center;
  width: 95vw;
}
li {
  text-align: center;
  font-family: sans-serif;
  line-height: 50px;
}
@layer no-support {
  @supports not (row-rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the row-rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const gap = document.getElementById("gap");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

gap.addEventListener("input", () => {
  output.innerText = ul.style.columnGap = `${gap.value}px`;
});
```

Result

Make the column gaps wider and note how the breaks between row segments grow. Bring the column gap width down to `0px`, and notice how the row decoration appears continuous. It isn't! The `0px` gap between segments may not be visible, but the segments still start and end at the gap, so any offsets set with `row-rule-inset` properties will still be applied.

See also

- `column-rule-break`
- `rule-break` shorthand
- `rule-inset` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

row-rule-color CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-color`** CSS property defines the colors of the lines drawn between rows in multi-row grid, flex, and multi-col layouts.

Try it

```
row-rule-color: magenta;
```

```
row-rule-color: magenta, goldenrod;
```

```
row-rule-color: repeat(2, magenta), goldenrod;
```

```
row-rule-color: goldenrod, repeat(auto, magenta), goldenrod;
```

```
row-rule-color: currentColor;
```

```
<section id="default-example">
  <ul id="example-element">
    <li>One fish</li>
    <li>Two fish</li>
    <li>Red fish</li>
    <li>Blue fish</li>
  </ul>
</section>
```

```
#example-element {
  display: flex;
  flex-flow: column;
  row-rule-style: solid;
  row-rule-width: 5px;
  gap: 5px;
  text-align: left;
}
```

Syntax

```
/* Single value */
row-rule-color: red;
row-rule-color: rgb(192 56 78);
row-rule-color: transparent;
row-rule-color: hsl(0 100% 50% / 60%);
row-rule-color: var(--primaryColor);
```

```
/* Multiple values */
row-rule-color: red, transparent;
row-rule-color: repeat(3, red), repeat(3, transparent);
row-rule-color: repeat(3, red), repeat(3, yellow, blue);
row-rule-color: red, repeat(auto, transparent), red;
row-rule-color: red, repeat(auto, blue, yellow), red;
row-rule-color: repeat(3, red), repeat(auto, transparent), repeat(3, red);

/* Global values */
row-rule-color: inherit;
row-rule-color: initial;
row-rule-color: revert;
row-rule-color: revert-layer;
row-rule-color: unset;
```

Values

The `row-rule-color` property accepts a comma-separated list of values, including:

`<line-color>`

A `<color>` representing the color of the line.

`<repeat-line-color>`

A `repeat()` function, with an `<integer>` of `1` or more as the first argument and one or more `<color>` values as subsequent arguments. The `<integer>` specifies how many times the `<color>` values should be repeated.

`<auto-repeat-line-color>`

A `repeat()` function, with `auto` as the first argument and one or more `<color>` values as subsequent arguments. The provided `<color>` values are repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value.

Description

The `row-rule-color` property defines the colors of any lines drawn in the gaps between rows in multi-column, flex, and grid containers with more than one row.

The value is a comma-separated list of components, which can include `<line-color>`, `<repeat-line-color>`, and `<auto-repeat-line-color>` types.

The `row-rule-color`, along with the `row-rule-width` and `row-rule-style` properties, can be set using the `row-rule` shorthand. The `row-rule-color`, along with the `column-rule-color` property, can also be set using the `rule-color` shorthand.

A `<line-color>` can be declared as any valid CSS `<color>` value. If the property value consists of only one `<color>`, all the rule lines will be that color. If we declare the following, the lines will all be blue:

```
row-rule-color: blue;
```

When more than one `<line-color>` is declared, they will be applied to row-rules in the order specified. If there are more row-rules than `<line-color>` values, the list of line colors is repeated until every row-rule has a color. If we declare the following, for example, every odd rule will be blue, and every even rule will be yellow.

```
row-rule-color: blue, yellow;
```

Repeated line colors

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<color>` values passed as subsequent arguments the specified number of times. This allows the same color to be repeated a set number of times without repeating the same `<line-color>` multiple times. The following declarations are equivalent:

```
row-rule-color: blue, yellow, red, yellow, red;
row-rule-color: blue, repeat(2, yellow, red);
```

You can use any valid color value from any color space, including CSS color functions, custom properties, etc. Using `repeat()` can make values easier to write, especially as your color values become more complex. It enables a recurring pattern to be written using a single function, regardless of the number of rows.

If we set `--base: yellow` and `--mixin: blue`, the following will provide similar results to the previous declaration:

```
row-rule-color:
  color-mix(in lch decreasing hue, var(--base) 0%, var(--mixin)),
  repeat(
    2,
    color-mix(in lch decreasing hue, var(--base) 100%, var(--mixin)),
    color-mix(in lch decreasing hue, var(--base) 58%, var(--mixin))
  );
```

This creates a list of five colors. If the number of colors in the `row-rule-color` value's color list exceeds the number of gaps between rows, the excess color values are ignored. If the container has three rows, the rule in the first gutter will be blue and the second yellow.

If there are more gutters than colors, the list of colors is repeated until all row-rules receive a color. If the container has 6, 11, 16, or 21 rows, this sequence of colors will be repeated one, two, three, or four times, respectively, with the last one being red.

Auto-repeating line colors

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<color>` values passed in subsequent arguments will be repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value, if any.

```
row-rule-color: blue, repeat(auto, yellow), red;
```

In this case, the first row-rule will be blue, the last will be red, and all others will be yellow. It doesn't matter if the container has 3, 6, 11, 16, or 21 rows; the first will always be blue and, as long as there are at least two row-rules, the last will always be red. All the other rules will be yellow, which means if there are only 2 or 3 rows, there will be no yellow lines.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for the row-rule line colors that would not otherwise receive values from other parts of the list, preventing the list from being cycled. A `row-rule-color` value can include, at most, one `repeat(auto, <color>)`.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-color =
  <line-color-list>       |
  <auto-line-color-list>

<line-color-list> =
  <line-color-or-repeat>#

<auto-line-color-list> =
  <line-color-or-repeat>#? , <auto-repeat-line-color> , <line-color-or-repeat>#?

<line-color-or-repeat> =
  <color>               |
  <repeat-line-color>

<auto-repeat-line-color> =
  repeat( auto , [ <color> ]# )

<repeat-line-color> =
  repeat( [ <integer [1,∞]> ] , [ <color> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single color for the lines drawn between flex items.

HTML

We include a list of dynamic sports duos:

```
<ul>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
  <li>Kerri Walsh + Misty May Treanor</li>
</ul>
```

CSS

We define the list as a flex container, creating rows by setting `flex-direction` to `column` with the `flex-flow` shorthand. We include a `gap` of `5px` to provide enough room between the rows to fit our `3px` dashed rule:

```
ul {
  display: flex;
  flex-flow: column;
  gap: 5px;
  row-rule-style: dashed;
  row-rule-width: 3px;
  row-rule-color: blue;
}
```

Result

Repeating values

This example demonstrates how, when there are fewer values in the list of colors than gutters between rows, the values are repeated.

CSS

Using the same HTML and CSS as in the previous example, we include three comma-separated colors as the `row-rule-color` value:

```
ul {
  row-rule-color: blue, yellow, red;
}
```

Result

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `row-rule-color` property value and how this function can help prevent complex values from becoming unwieldy.

CSS

We use the same HTML and CSS as in the previous examples. To demonstrate how values can become complicated and the utility of the `repeat()` function, we declare two custom properties, which we use in three `color-mix()` color function declarations to create the same blue, red, and yellow colors as in the previous example. The second declaration is within a `repeat()` function, set to repeat 3 times.

```
ul {
  --base: yellow;
  --mixin: blue;
  row-rule-color:
    color-mix(in lch decreasing hue, var(--base) 0%, var(--mixin)),
    repeat(3, color-mix(in lch decreasing hue, var(--base) 100%, var(--mixin))),
    color-mix(in lch decreasing hue, var(--base) 58%, var(--mixin));
}
```

Result

The flex container has six rows, so five gutters. The `repeat()` function repeats our second color three times, creating a color list with five colors. Since there are as many row gutters as there are total colors, the colors are not repeated.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto`, instead of an integer, within the `repeat()` function.

CSS

Using `repeat(auto, <color>)`, we set all the lines to be almost transparent black (`#00000033`), except the first and last, which we set to a solid `black`.

```
ul {
  row-rule-color: black, repeat(auto, #00000033), black;
}
```

Result

```
@layer no-support {
  @supports not (row-rule-color: red, blue) {
    body::before {
      content: "Your browser doesn't support the row-rule-color property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `row-rule-width`
- `row-rule-style`
- `column-rule-color`
- `row-rule` shorthand
- `rule-color` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

row-rule-inset-cap CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-inset-cap`** shorthand CSS property can be used to offset the row rule segment cap endpoints at the container's left and right edges, and endpoints where the segments don't intersect other column or row segments.

Try it

```
row-rule-inset-cap: -20px;
```

```
row-rule-inset-cap: 0;
```

```
row-rule-inset-cap: 1em;
```

```
row-rule-inset-cap: 100%;
```

```
row-rule-inset-cap: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>

    <i id="r">R</i>
    <i id="z">Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick rebeccapurple;
  row-rule-color: magenta;
  gap: 0.5em;
  rule-overlap: row-over-column;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
}
#r {
```

```
  grid-column: 5 / 6;
  grid-row: 3 / 4;
}
#z {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
#bang {
  grid-column: 5 / 6;
  grid-row: 5 / 6;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `row-rule-inset-cap-end`
- `row-rule-inset-cap-start`

Syntax

```
/* Keywords */
row-rule-inset-cap: overlap-join;

/* <length-percentage> values */
row-rule-inset-cap: 0;
row-rule-inset-cap: 1em;
row-rule-inset-cap: -5px;
row-rule-inset-cap: -25%;

/* Two values */
row-rule-inset-cap: overlap-join 1em;
row-rule-inset-cap: -5px -25%;

/* Global values */
row-rule-inset-cap: inherit;
row-rule-inset-cap: initial;
row-rule-inset-cap: revert;
row-rule-inset-cap: revert-layer;
row-rule-inset-cap: unset;
```

Values

This property is specified as a one or two values from the following list:

`overlap-join`

> Resolves to `0`.

`<length-percentage>`

> Specifies the size of the inset. Percentage values are relative to the crossing gap width, which is the width of the `column-gap` for segment endpoints at gap junctions and `0` for endpoints at the container's edge.

Description

The `row-rule-inset-cap` shorthand property can be used to set the `row-rule-inset-cap-start` and `row-rule-inset-cap-end` properties, insetting or outsetting both the left and right edges of cap segment endpoints in a single declaration.

If one value is specified, both properties are set to that value. If two values are specified, `-start` is set to the first value and `-end` is set to the second. The default value is `0`, which in the case of cap endpoints, is the same as `overlap-join`. Positive values reduce, or inset, the segment size, while negative values increase, or outset, it.

Row rules are painted within a row gap as one or more segments, with segments occurring between:

- Adjacent rows in CSS grid layouts.
- Adjacent flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Adjacent rows in multi-col layouts, which may exist when `column-height` is set to a `<length>`.

Whether a row rule spans multiple columns or is broken into multiple segments is defined by the `row-rule-break` property, with interior breaks between row rule segments generally being the size of the `column-gap`.

Length `row-rule-inset-cap` values inset segments by the specified value. Negative length values create an outset, making the segment wider and causing cap segments on the left and right edges of the container to extend beyond the container's edge.

Percentage values are relative to the size of the `column-gap` for interior segments. Setting `-50%` will extend the segment to the middle of the gap, while `-100%` will extend the segment across the entire gap. For cap segments at the container's edge, percentage values are relative to `0`, so percentage values have no effect on cap segment endpoints at the container's edge.

The `row-rule-inset-cap` property is a constituent property of a few shorthand properties:

- To inset the left and right sides of all row segments, the `row-rule-inset-cap` property, along with the `row-rule-inset-junction` property, can be set using the `row-rule-inset` shorthand.
- To inset both row and column cap segment endpoints, the `row-rule-inset-cap` property, along with the `column-rule-inset-cap` property, can be set using the `rule-inset-cap` shorthand.

All segment endpoints, including this property's `-junction` and `column-` equivalents, can be set using the `rule-inset` shorthand.

Understanding cap endpoints

A *cap segment endpoint* is any segment endpoint that is not a junction segment endpoint. This includes endpoints at the container's content edges, as well as endpoints at a gap junction where no other column or row segments are present.

The `row-rule-inset-cap` property can outset or inset the start and end of row rules at the container edge, and the start and end of interior row segments where no other segments are present.

Cap segments are impacted by the `rule-visibility-items` properties, which define whether row- and column-rule segments are painted in gaps adjacent to empty areas. Changing the value from `auto` to `between` or `around` can create additional interior cap segments.

In the following demonstration, the rows end in cap endpoints at the container's left and right edges. With `row-rule-inset-cap: -32px` set, these endpoints are outset by `32px`. Change the inset `<length>` value to better visualize which segments start or end with cap segment endpoints.

```
<ul id="ul">
  <li>1</li>
```

```
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-40"
      max="16"
      value="-32"
      id="inset"
      data-unit="px"
  /></label>
  <output id="o">-32px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-200"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: none;
  gap: 20px;
  row-rule: 10px solid olive;
  column-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: normal;
  rule-break: intersection;
  row-rule-inset-cap: -32px;

  border: 1px solid;
  margin: auto 40px;
}
ul {
  place-items: center;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
li:nth-of-type(n + 9) {
  grid-row: 3 / 4;
}
li:nth-of-type(n + 13) {
  grid-row: 4 / 5;
}
li:nth-of-type(12),
li:nth-of-type(16) {
  grid-column: 5/6;
}
li:nth-of-type(17) {
  grid-column: 6/7;
}
@layer no-support {
  @supports not (row-rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-cap property";
```

```
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  row-rule-inset-cap: 100%;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  o.innerText =
    ul.style.rowRuleInsetCap = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.rowRuleStyle = "repeat(2, solid), double";
  } else if (visibility.value === "around") {
    ul.style.rowRuleStyle = "repeat(3, solid), repeat(2, double)";
  } else {
    ul.style.rowRuleStyle = "solid";
  }
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.rowRuleStyle = "repeat(2, inset), double, repeat(2, solid)";
  } else if (visibility.value === "around") {
    ul.style.rowRuleStyle = "repeat(3, inset), repeat(2, double)";
  } else {
    ul.style.rowRuleStyle = "solid";
  }
});
```

Setting `0px` aligns the row rule ends with the left and right container edges. This is the default.

Select `between` as the `rule-visibility-items` value. This value paints rules in gap segments only if the two adjacent areas are occupied by items. Again, we have row rule cap endpoints at the start edge of the container, but now we have none at the end edge. The third row rule, denoted with a double-line style, has two additional cap endpoints: the end-side of the segment between `11` and `15` and the start-side of the segment between `12` and `16`. These do not meet any other rule segments; therefore, they are affected by the `row-rule-inset-cap-start` property.

In this case, the `around` value of the `rule-visibility-items` property, which paints rules in a gap segment as long as one adjacent area is occupied by an item, did not create additional cap segment endpoints. All the interior segment endpoints end in junctions where there are column segments, thus creating junction, not cap, segment endpoints. Junction endpoints can be inset using the `row-rule-inset-junction` shorthand property.

Understanding percentage values

The length a percentage value is relative to depends on the endpoint location. Interior endpoint percentage values are relative to the width of the gap at the cap endpoint, so relative to the `column-gap` if abutting a rule gap, plus any additional spacing added due to `justify-content` settings, and `0` at the edge of the container. For example, `row-rule-inset-cap: 50%` resolves to half the gap junction size at an interior cap (half the size of the `column-gap` value), and `0` at the container edges.

This example is not broken. When `rule-visibility-items` is set to `normal`, every row cap endpoint abuts the left or right edge of the container, so any percentage value set will be relative to `0`.

Select `around` as the `rule-visibility-items` value. The first three rows start and the first and last row end at the container edge. Any percentage values for these row cap segment endpoints will resolve to `0`. All the other segments end at interior gaps where column rule segments are present, so these row segments are not cap segment endpoints.

Select `between` as the `rule-visibility-items` value. As in the previous demonstration, this value creates two interior cap segment endpoints: the right side of the segment between items `11` and `15` and the left side of segment between items `12` and `16` again do not meet any other rule segments. For these two cap endpoints, the percentage offset is relative to the size of the `column-gap` width, which in this case is `20px`.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-inset-cap =
  <inset-value> <inset-value>?

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `row-rule-inset-cap` to inset the row rule cap segment endpoints on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap row rule endpoints</h1>
<article>
```

```
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a `lightblue` `rule` to paint both column and row gaps, then overwrite the `row-rule-color`, setting darker `blue` row gap decorations. Finally, we set the `row-rule-inset-cap` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  row-rule-color: blue;

  row-rule-inset-cap: 16px;
}
```

We also set the `flex-direction` on the `.column` container, to change the main axis of the flex container, and make the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (row-rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-cap property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
```

```
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.rowRuleInsetCap = val;
  containers[1].style.rowRuleInsetCap = val;
  output.innerText = val;
});
```

Result

Change the size of the inset. Note how the row segments grow and shrink only at segment cap ends, either their left side, their right side, or both directions — the ends that don't intersect other row or column segments.

See also

- `row-rule-inset-cap-end`
- `row-rule-inset-cap-start`
- `row-rule-inset` shorthand
- `column-rule-inset-cap` shorthand
- `rule-inset` shorthand
- `row-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

row-rule-inset-cap-end CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-inset-cap-end`** CSS property can be used to offset the end of row rule segment cap endpoints.

Try it

```
row-rule-inset-cap-end: -20px;
```

```
row-rule-inset-cap-end: 1.25em;
```

```
row-rule-inset-cap-end: 0;
```

```
row-rule-inset-cap-end: 100%;
```

```
row-rule-inset-cap-end: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>

    <i id="u">U</i>
    <i id="x">X</i>
    <i id="y">Y</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick magenta;
  row-rule-color: rebeccapurple;
  gap: 1em;
  rule-overlap: row-over-column;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
```

```
}
#u {
  grid-row: 3 / 4;
  grid-column: 7 / 8;
}
#y {
  grid-row: 4 / 5;
  grid-column: 4 / 5;
}
#x {
  grid-column: 3 / 4;
  grid-row: 4 / 5;
}
#bang {
  grid-column: 6 / 7;
  grid-row: 4 / 5;
}
```

Syntax

```
/* Keywords */
row-rule-inset-cap-end: overlap-join;

/* <length-percentage> values */
row-rule-inset-cap-end: 0;
row-rule-inset-cap-end: 1rem;
row-rule-inset-cap-end: -15px;
row-rule-inset-cap-end: -50%;

/* Global values */
row-rule-inset-cap-end: inherit;
row-rule-inset-cap-end: initial;
row-rule-inset-cap-end: revert;
row-rule-inset-cap-end: revert-layer;
row-rule-inset-cap-end: unset;
```

Values

This property is specified as a single value from the following list:

`overlap-join`

Resolves to `0`.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the cap endpoint, which is either the height of the `column-gap` width or `0`.

Description

The `row-rule-inset-cap-end` property can be used to inset the end edge of row rule cap segment endpoints at the container's end edge and cap endpoints where no rule segments intersect. The default value is `0`, which is the same as `overlap-join`. Positive values reduce the row rule segment size, while negative values increase it.

Row rules are painted within a row gap as one or more segments, with segments occurring between:

- Adjacent rows in CSS grid layouts.
- Adjacent flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Adjacent rows in multi-col layouts if the `column-height` is set to a `<length>`.

Whether a row rule spans multiple columns or is broken into multiple segments is defined by the `row-rule-break` property, with interior breaks between row rule segments being the size of the `column-gap`.

Length `row-rule-inset-cap-end` values are inset by the value specified for both interior and end edge cap segments. Negative length values create an outset, with end edge cap segments extending beyond the container's end edge.

Percentage values for interior cap segment endpoint insets are relative to the size of the `column-gap`. For end edge cap segments, percentage values are relative to `0`, so percentage values never inset or outset cap segment endpoints at the container's end edge.

The `row-rule-inset-cap-end` property is a constituent property of several shorthand properties:

- To inset both the start and ends of row rule caps, the `row-rule-inset-cap-end` property, along with the `row-rule-inset-cap-start` property, can be set using the `row-rule-inset-cap` shorthand.
- To inset the ends of all row segments, the `row-rule-inset-cap-end` property, along with the `row-rule-inset-junction-end` property, can be set using the `row-rule-inset-end` shorthand.

All segment endpoints, including this property's `-start`, `-junction`, and `column-` equivalents, can be set using the `rule-inset` shorthand.

Understanding cap end

A *cap segment endpoint* is any segment endpoint that is not a junction segment endpoint. This includes endpoints at the container's content edges, as well as endpoints at a gap junction where no other column or row segments are present.

The `row-rule-inset-cap-end` controls the inset of cap segment endpoints occurring at the end of a row rule segment. Depending on the writing mode, the property can inset the right or left edge of row rule segments at interior gaps with no other row or column rule segments, or at the container's end edge.

This property is affected by the `rule-visibility-items` properties, which define whether row- and column-rule segments are painted in gaps adjacent to empty areas. As row cap segment endpoints only exist at the end edge of the container and at interior gaps where no other row or column rule segments are present, whether segments are painted (or would otherwise be painted if the `rule` were set to a visible value) impacts which row segments are end cap segments.

In the following demonstration, the end segments of row rules with a solid line style end in a cap endpoint. With `row-rule-inset-cap-end: 16px` set, these endpoints are all inset by `16px`. Change the inset `<length>` value to better visualize which segments end in cap segment endpoints.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
```

```
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li class="a">18</li>
  <li class="b">22</li>
  <li class="c">24</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-40"
      max="16"
      value="-32"
      id="inset"
      data-unit="px"
  /></label>
  <output id="o">-32px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-200"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: none;
  gap: 20px;
  row-rule: 10px solid olive;
  column-rule: 10px solid palegoldenrod;
  rule-overlap: row-over-column;
  rule-visibility-items: normal;
  rule-break: intersection;
  row-rule-inset-cap-end: -32px;

  border: 1px solid;
}
ul {
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
.a {
  grid-column: 5 / 6;
  grid-row: 3 / 4;
}
.b {
  grid-column: 3 / 4;
  grid-row: 4 / 5;
}
.c {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
@layer no-support {
  @supports not (row-rule-inset-cap-end: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-cap-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  row-rule-inset-cap-end: 100%;
  row-rule-style: inset;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  output.innerText =
    ul.style.rowRuleInsetCapEnd = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.rowRuleStyle = "repeat(2, solid), double";
  } else if (visibility.value === "around") {
    ul.style.rowRuleStyle = "repeat(3, solid), repeat(2, double)";
  } else {
    ul.style.rowRuleStyle = "solid";
  }
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.rowRuleStyle = "repeat(2, inset), double, repeat(2, solid)";
  } else if (visibility.value === "around") {
    ul.style.rowRuleStyle = "repeat(3, inset), repeat(2, double)";
  } else {
    ul.style.rowRuleStyle = "solid";
  }
});
```

Setting `-32px` outsets the end of all the row rules by `32px`: the lines are drawn `32px` past the right edge of the container. As row rules don't impact the box model, these lines have no impact on the layout of the container or the rest of the content. If `0px` is set, the end of the row rules will align with the edge of the container. This is the default.

The double line-style row rule segments, which appear when the `rule-visibility-items` property is set to `between`, add a cap endpoint. The `between` value only paints rules in gap segments between two adjacent areas occupied by an item. Because the row rule segment above item `24` ends at an intersection with no other row or column rule segments present, this is a cap endpoint, and therefore is affected by the `row-rule-inset-cap-end` property. The segment above item `22` ends at a gap junction where a rule segment is present, so it is not a cap endpoint. There are fewer segments abutting the right edge of the container, but the remaining segment, between `6` and `12`, remains a cap endpoint.

Rule segments abutting the end edge of the container are always row-rule cap end segments affected by the value of the `row-rule-inset-cap-end` property.

Understanding percentage values

The length a percentage value is relative to depends on the endpoint location. Interior endpoint percentage values are relative to the width of the gap at the cap endpoint, so relative to the `column-gap` if abutting a rule gap, and `0` at the top edge of the container.

This example isn't broken; all the cap segments end at the container's edge, so all the insets are `0` by default.

If you select `around` as the `rule-visibility-items` value, the only cap segment endpoints are at the edge of the container, so percentage value insets continue to resolve to `0`.

Select `between` as the `rule-visibility-items` value. We now have a row rule end cap affected by the inset — see the segment between grid items `18` and `24`! This segment ends at an interior gap where no other rule segments are present, so the percentage inset is relative to the size of the `column-gap` width, which in this case is `20px`. Setting `100%` insets the end by `20px`. Setting `-200%` will outset the segment by `40px`, with the lines drawn through the `20px` gap into the next column.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-inset-cap-end =
  <inset-value>

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `row-rule-inset-cap-end` to inset the end edge of cap segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap row rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
```

```
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using with `flex-wrap` and `flex-line-count`. We define a light blue `rule` to be painted in both column and row gaps, then overwrite the `row-rule-color`, setting darker `blue` vertical gap decorations. Finally, we set the `row-rule-inset-cap-end` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  row-rule-color: blue;

  row-rule-inset-cap-end: 16px;
}
```

We also set the `flex-direction` on the `.column` container, to change the main axis of the flex container, and make the items flow in columns rather than rows. The rest of the CSS is hidden for brevity.

```
.column {
  flex-direction: column;
}
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (row-rule-inset-cap-end: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-cap-end property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.rowRuleInsetCapEnd = val;
  containers[1].style.rowRuleInsetCapEnd = val;
```

```
  output.innerText = val;
});
```

Result

Change the size of the inset.

See also

- `row-rule-inset` shorthand
- `rule-inset` shorthand
- `row-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

row-rule-inset-cap-start CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-inset-cap-start`** CSS property can be used to offset the start of row rule segment cap endpoints.

Try it

```
row-rule-inset-cap-start: -20px;
```

```
row-rule-inset-cap-start: 0;
```

```
row-rule-inset-cap-start: 1em;
```

```
row-rule-inset-cap-start: 100%;
```

```
row-rule-inset-cap-start: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>

    <i id="r">R</i>
    <i id="z">Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick magenta;
  row-rule-color: rebeccapurple;
  gap: 1em;
  rule-overlap: row-over-column;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
}
#r {
  grid-column: 5 / 6;
  grid-row: 3 / 4;
}
#z {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
#bang {
  grid-column: 5 / 6;
  grid-row: 5 / 6;
}
```

Syntax

```
/* Keywords */
row-rule-inset-cap-start: overlap-join;
```

```
/* <length-percentage> values */
row-rule-inset-cap-start: 0;
row-rule-inset-cap-start: 1em;
row-rule-inset-cap-start: -5px;
row-rule-inset-cap-start: -25%;

/* Global values */
row-rule-inset-cap-start: inherit;
row-rule-inset-cap-start: initial;
row-rule-inset-cap-start: revert;
row-rule-inset-cap-start: revert-layer;
row-rule-inset-cap-start: unset;
```

Values

This property is specified as a single value from the following list:

`overlap-join`

Resolves to `0`.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the cap endpoint, which is either the `column-gap` or `0`.

Description

The `row-rule-inset-cap-start` property insets the start of cap segment endpoints at the container's start edge and at cap endpoints where no rule segments intersect. The default value is `0`, which is the same as `overlap-join`. Positive values reduce the segment size, while negative values increase it.

Row rules are painted within a row gap as one or more segments, with segments occurring between:

- Adjacent rows in CSS grid layouts.
- Adjacent flex items or flex lines in flex layouts, depending on the `flex-direction`.
- Adjacent rows in multi-col layouts, which may exist when `column-height` is set to a `<length>`.

Length `row-rule-inset-cap-start` values inset the start of both interior and start-edge cap segment endpoints by the specified value. Negative length values create an outset, with container-edge cap segments extending beyond the container's start edge.

Percentage value insets for interior cap segments are relative to the size of the `column-gap`. For cap segments at the container start edge, percentage values are relative to `0`, so they are always `0px`.

The `row-rule-inset-cap-start` property is a constituent property of several shorthand properties:

- To inset both the left and right ends of row cap segments, the `row-rule-inset-cap-start` property, along with the `row-rule-inset-cap-end` property, can be set using the `row-rule-inset-cap` shorthand.
- To inset the start edge of all row segments, the `row-rule-inset-cap-start` property, along with the `row-rule-inset-junction-start` property, can be set using the `row-rule-inset-start` shorthand.

All segment endpoints, including this property's `-end`, `-junction`, and `column-` equivalents, can be set using the `rule-inset` shorthand.

Understanding cap start

A *cap segment endpoint* is any segment endpoint that is not a junction segment endpoint. This includes endpoints at the container's content edges, as well as endpoints at a gap junction where no other rule segments are present.

The `row-rule-inset-cap-start` controls the start inset of row rules with cap endpoints. The property can shrink or extend the start of:

- Row rule segments abutting the container start edge.
- Row rule segments whose left side abuts an interior gap where no other row or column rule segments are present.

Row cap segments are impacted by the `rule-visibility-items` properties, which define whether row- and column-rule segments are painted in gaps adjacent to empty areas. Changing the value from `auto` to `between` or `around` can create additional interior cap segments.

In the following demonstration, the leftmost segments of the row rules abutting the container edge start in a cap endpoint. With `row-rule-inset-cap-start: -32px` set, these endpoints are all outset by `32px`. As row rules don't impact the box model, these protruding lines don't affect the content's layout. Change the inset `<length>` value to better visualize which segments start with cap segment endpoints.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
```

```
      type="range"
      min="-40"
      max="16"
      value="-32"
      id="inset"
      data-unit="px"
  /></label>
  <output id="o">-32px</output>
</p>
```

```
<p>
  <label
    >Change the size of the inset.
    <input
      type="range"
      min="-200"
      max="100"
      value="100"
      id="inset"
      data-unit="%"
  /></label>
  <output id="o">100%</output>
</p>
```

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: none;
  gap: 20px;
  row-rule: 10px solid olive;
  column-rule: 10px solid palegoldenrod;
  rule-overlap: column-over-row;
  rule-visibility-items: normal;
  rule-break: intersection;
  row-rule-inset-cap-start: -32px;

  border: 1px solid;
  margin: auto 40px;
}
ul {
  place-items: center;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
li:nth-of-type(n + 9) {
  grid-row: 3 / 4;
}
li:nth-of-type(n + 13) {
  grid-row: 4 / 5;
}
li:nth-of-type(12),
li:nth-of-type(16) {
  grid-column: 5/6;
}
li:nth-of-type(17) {
  grid-column: 6/7;
}

@layer no-support {
  @supports not (row-rule-inset-cap-start: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-cap-start property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
ul {
  row-rule-inset-cap-start: 100%;
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  output.innerText =
    ul.style.rowRuleInsetCapStart = `${inset.value}${inset.dataset["unit"]}`;
});
```

```
visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
  if (visibility.value === "between") {
    ul.style.rowRuleStyle = "repeat(2, solid), double";
  } else {
    ul.style.rowRuleStyle = "solid";
  }
});
```

Change the inset. Setting `0px` aligns the start of the row rules with the start of the container. This is the default.

Select `between` as the `rule-visibility-items` value. This value paints rules in a gap segment only if the two adjacent areas are occupied by items. Again, we have row rule cap endpoints at the start edge of the container. The third row rule, denoted with a double line style, has an additional cap endpoint segment: the left side of the segment between items `12` and `16`. This does not meet any other rule segments and is therefore affected by the `row-rule-inset-cap-start` property.

In this case, the `around` value of the `rule-visibility-items` property, which paints rules in a gap segment as long as one adjacent area is occupied by an item, did not create additional cap segment endpoints. The rule start of the segment between `12` and `16` intersects with the column rule segments in the column-gap to the left of those items, creating junction, not cap, segment endpoints. Junction endpoints are inset using the `row-rule-inset-junction-start` property instead.

Understanding percentage values

The length a percentage value is relative to depends on the endpoint location. Interior endpoint percentage values are relative to the width of the gap at the cap endpoint, so relative to the `column-gap` if abutting a rule gap, and `0` at the edge of the container.

This example isn't broken; all the cap segments start at the container's edge, so all the insets are `0` by default.

The slider only has an effect when the `rule-visibility-items` value is set to `between`, and then only on the single inner cap endpoint segment that this value creates — the segment between `12` and `16`. For this segment only, the percentage offset is relative to the size of the `column-gap` width, which in this case is `20px`. Setting `100%` insets the start of the cap segment by `20px`. Setting `-200%` outsets the segment by `40px`, with the rule segment drawn through the `20px` gap and into the previous column.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-inset-cap-start =
  <inset-value>

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `row-rule-inset-cap-start` to inset the start edge of cap segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap row rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using with `flex-wrap` and `flex-line-count`. We define a light blue `rule` to be painted in both column and row gaps, then overwrite the `row-rule-color`, setting darker `blue` row gap decorations. Finally, we set the `row-rule-inset-cap-start` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  row-rule-color: blue;
```

```
  row-rule-inset-cap-start: 16px;
}
```

We also set the `flex-direction` on the `.column` container, to change the main axis of the flex container, and make the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 42px;
  rule: 1px solid black;
  width: 100vw;
  padding: 0 50px;
  box-sizing: border-box;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (row-rule-inset-cap-start: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-cap-start property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.rowRuleInsetCapStart = val;
  containers[1].style.rowRuleInsetCapStart = val;
  output.innerText = val;
});
```

Result

Change the size of the inset.

See also

- `column-rule-inset-cap-start`
- `row-rule-inset-cap-end`
- `row-rule-inset-junction-start`
- `row-rule-inset-start` shorthand
- `row-rule-inset-cap` shorthand
- `row-rule-inset` shorthand
- `rule-inset-start` shorthand
- `rule-inset-cap`
- `rule-inset` shorthand
- `row-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

row-rule-inset-junction CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-inset-junction`** CSS shorthand property can be used to offset both the left and right endpoints of row rule segments that are junction endpoints; that is, endpoints at gap junctions where rule segments intersect.

Try it

```
row-rule-inset-junction: 0;
```

```
row-rule-inset-junction: 10px;
```

```
row-rule-inset-junction: 0.5em -0.5em;
```

```
row-rule-inset-junction: overlap-join;
```

```
row-rule-inset-junction: overlap-join 10px;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  rule: solid thick lightpink;
  row-rule-color: magenta;
  row-rule-break: intersection;
  gap: 1.5em;
  rule-overlap: row-over-column;
  border: 1px solid rebeccapurple;
  margin: auto;
  rule-visibility-items: around;
}
#example-element i {
  background-color: #efefef;
  padding: 1em;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `row-rule-inset-junction-end`
- `row-rule-inset-junction-start`

Syntax

```
/* Keywords */
row-rule-inset-junction: overlap-join;

/* <length-percentage> values */
row-rule-inset-junction: 0;
row-rule-inset-junction: 1em;
row-rule-inset-junction: -5px;
row-rule-inset-junction: -25%;

/* Two values */
row-rule-inset-junction: 0 1em;
row-rule-inset-junction: -5px -25%;
row-rule-inset-junction: overlap-join 10px;

/* Global values */
row-rule-inset-junction: inherit;
row-rule-inset-junction: initial;
row-rule-inset-junction: revert;
row-rule-inset-junction: revert-layer;
row-rule-inset-junction: unset;
```

Values

This property is specified as one or two values from the following list:

`overlap-join`

Specifies that the junction segment should extend across the column-rule, resolving to half the `column-gap` value plus half the used `column-rule-width` value.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the junction endpoint, which is the `column-gap` value.

Description

The `row-rule-inset-junction` shorthand property can be used to set the `row-rule-inset-junction-start` and `row-rule-inset-junction-end` properties, insetting or outsetting both the start and end edges of junction segment endpoints occurring at the left and right (or right and left is right to left languages) of row rule segments.

If one value is specified, both properties are set to that value. If two values are specified, `-start` is set to the first value and `-end` is set to the second. Positive values reduce (or inset) the segment size, while negative values and the `overlap-join` keyword increase (or outset) it. The default value is `0`.

The `row-rule-inset-junction` property is a constituent property of a few shorthand properties:

- To inset both row and column segment junction endpoints, the `row-rule-inset-junction` shorthand property, along with the `column-rule-inset-junction` shorthand property, can be set using the `rule-inset-junction` shorthand.
- To inset both row cap and junction segment endpoints, the `row-rule-inset-junction` shorthand property, along with the `row-rule-inset-cap` shorthand property, can be set using the `row-rule-inset` shorthand.

All segment endpoints, including this property's `-cap` and `-column` equivalents, can be set using the `rule-inset` shorthand.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-inset-junction =
  <inset-value> <inset-value>?

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `row-rule-inset-junction` to inset the edges of junction segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting junction row rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a light blue `rule` to paint both column and row gaps, then overwrite the `row-rule-color`, setting darker `blue` row gap decorations. We also set the `rule-overlap` property to `row-over-column` to ensure the row segments are drawn on top of the column segments when the segments overlap. Finally, we set the `row-rule-inset-junction` to `16px`, setting the start and end to the same value.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  row-rule-color: blue;
  rule-overlap: row-over-column;

  row-rule-inset-junction: 16px;
}
```

We also set the `flex-direction` on the `.column` container to `column`, changing the main axis of the flex container to run up and down the page and making the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
h1,
article {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (row-rule-inset-junction: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-junction property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.rowRuleInsetJunction = val;
  containers[1].style.rowRuleInsetJunction = val;
  output.innerText = val;
});
```

Result

Change the size of the inset. Note that this property affects only row rules split into multiple segments. When a row rule is a single segment that spans the container, the segment has two cap endpoints and no junction endpoints, and remains unaffected by changes to the `row-rule-inset-junction` value.

With grid layout

This example demonstrates using the `row-rule-inset-junction` to inset the start and end row junction segment endpoints to two different values on a grid container.

HTML

We include the `<ul>` element as a container with several `<li>` children which will each be converted into a grid item.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
</ul>
```

```
<p>
  <label
    >Change the <code>-start</code> value.
    <input type="range" min="-40" max="40" value="16" id="start" data-unit="px"
  /></label>
  <output id="og">16px</output>
</p>
```

```
<p>
  <label
    >Change the <code>-end</code> value.
    <input type="range" min="-40" max="40" value="0" id="end" data-unit="px"
  /></label>
  <output id="ow">0px</output>
</p>
```

CSS

We turn the `<ul>` into a grid container by setting the `display` property to `grid`. The `grid-template-columns` property specifies that the grid has six columns. We remove the bullets with `list-style-type` and set the row and column gaps to `20px` with the `gap` shorthand. We set the color, size, and line style of all the rules using the `rule` shorthand, then change just the column rule color with the `column-rule-color` property.

We break the row rules at every intersection using the `row-rule-break` property. If the row rules didn't break, there would be no row junction segments to style!

Finally, we set the start of each row junction to be inset by `16px` and the end to not be inset using the `row-rule-inset-junction` property.

We also set the 6th grid item to span three columns.

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  list-style-type: "";
  gap: 20px;
  rule: 10px solid olive;
  column-rule-color: palegoldenrod;
  row-rule-break: intersection;

  row-rule-inset-junction: 16px 0;
}

li:nth-of-type(8) {
  grid-column-end: span 3;
}
```

The rest of the CSS is hidden for brevity.

```
ul {
  border: 1px solid;
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 2em 1em;
  width: 100%;
  box-sizing: border-box;
}

li code {
  display: block;
  margin: 0 auto;
  text-align: left;
  width: 30vw;
}
output {
  font-family: monospace;
}
input {
  accent-color: olive;
}
@layer no-support {
  @supports not (row-rule-inset-junction: 16px) {
    body::before {
      content: "Your browser doesn't support the row-rule-inset-junction property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const ul = document.querySelector("ul");
const startSize = document.getElementById("start");
const og = document.getElementById("og");
const endSize = document.getElementById("end");
const ow = document.getElementById("ow");
const cell = document.querySelector("li:nth-of-type(8)");
let text = "";
function update() {
  ul.style.rowRuleInsetJunction =
    text = `${startSize.value}px ${endSize.value}px`;
  cell.innerHTML = `<code>rule-inset-cap: ${text};</code>`;
}

update();

startSize.addEventListener("input", () => {
  og.innerText = `${startSize.value}px`;
  update();
});

endSize.addEventListener("input", () => {
  ow.innerText = `${endSize.value}px`;
  update();
});
```

Result

See also

- `column-rule-inset-junction` shorthand
- `row-rule-inset-cap` shorthand
- `row-rule-inset` shorthand
- `rule-inset` shorthand
- `row-rule-break`
- `rule-break` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

row-rule-style CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-style`** CSS property defines the line style of the lines drawn between rows in multi-row grid, flex, and multi-col layouts.

Try it

```
row-rule-style: solid;
```

```
row-rule-style: inset, outset;
```

```
row-rule-style: repeat(2, dashed, dotted), solid;
```

```
row-rule-style: solid, repeat(auto, dashed, dotted), solid;
```

```
row-rule-style: hidden;
```

```
<section id="default-example">
  <ul id="example-element">
    <li>One fish</li>
    <li>Two fish</li>
    <li>Red fish</li>
    <li>Blue fish</li>
  </ul>
</section>
```

```
#example-element {
  display: flex;
  flex-flow: column;
  row-rule-width: thick;
  row-rule-color: magenta;
  gap: 7px;
  text-align: left;
}
```

Syntax

```
/* One value */
row-rule-style: none;
row-rule-style: hidden;
row-rule-style: dotted;

/* Multiple values */
row-rule-style: groove, dashed, solid;
row-rule-style: double, repeat(5, ridge), double;
row-rule-style: solid, repeat(auto, inset, outset), solid;

/* Global values */
row-rule-style: inherit;
row-rule-style: initial;
row-rule-style: revert;
row-rule-style: revert-layer;
row-rule-style: unset;
```

Values

The `row-rule-style` property accepts a comma-separated list of values, including:

`<line-style>`

> A `<line-style>`: one of `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`. The default value is `none`.

`<repeat-line-style>`

> A `repeat()` function, with the first argument being an `<integer>` of `1` or more, and subsequent arguments being `<line-style>` values. The integer specifies how many times the `<line-style>` values should be repeated.

`<auto-repeat-line-style>`

> A `repeat()` function, with `auto` as the first argument and one or more `<line-style>` values as subsequent arguments. The provided `<line-style>` values are repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value.

Description

The `row-rule-style` property defines the line style of any row rule lines drawn in the gaps between rows in multi-column, flex, and grid containers with more than one row.

The value is a comma-separated list of components, which can include `<line-style>`, `<repeat-line-style>`, and `<auto-repeat-line-style>` types.

The `row-rule-style`, along with the `row-rule-color` and `row-rule-width` properties, can be set using the `row-rule` shorthand. The `row-rule-style`, along with the `column-rule-style` property, can also be set using the `rule-style` shorthand.

If the property value has only one `<line-style>`, all the row rules will be that style. If we declare the following, all row rules will be `dashed`:

```
row-rule-style: dashed;
```

When more than one `<line-style>` is declared, they will be applied to row-rules in the order specified. If there are more row-rules than `<line-style>` values, the list of line styles is repeated until every row rule has a style. If we declare the following, for example, every odd rule will be `dashed`, and every even rule will be `dotted`.

```
row-rule-style: dashed, dotted;
```

Repeated line styles

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<line-style>` values passed as subsequent arguments the specified number of times. This allows the same style to be repeated a set number of times without repeating the same value. You can include `<line-style>` keyword values or custom properties that resolve to a valid `<line-style>`. Using `repeat()` can make values easier to write, enabling recurring patterns to be written using a single function, regardless of the number of rows. The following declarations are equivalent:

```
row-rule-style: solid, outset, inset, outset, inset;
row-rule-style: solid, repeat(2, outset, inset);
```

This creates a list of five styles. If the number of styles in the `row-rule-style` value's style list exceeds the number of gaps between rows, the excess style values are ignored. If the container has three rows, the rule in the first gutter will be `solid` and the second `outset`.

If there are more gutters than styles, the list of styles is repeated. If the container has 6, 11, 16, or 21 rows, this sequence of styles will be repeated one, two, three, or four times, respectively, with the last rule being `inset`.

Auto-repeating line styles

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<line-style>` values passed as subsequent parameters will be repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value.

```
row-rule-style: solid, repeat(auto, dotted), solid;
```

In this case, it doesn't matter if the container has 3, 6, 11, 16, or 21 rows; the first and last row rules will always be `solid`, and all the other row rules will be `dotted`. If there are only 2 or 3 rows, there will be no dotted row rules.

The `auto` keyword within the `repeat()` function creates an auto repeater that fills in values for row rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. Only one `repeat(auto, <line-style>)` is allowed within a `row-rule-style` value.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-style =
  <line-style-list>       |
  <auto-line-style-list>

<line-style-list> =
  <line-style-or-repeat>#

<auto-line-style-list> =
  <line-style-or-repeat>#? , <auto-repeat-line-style> , <line-style-or-repeat>#?

<line-style-or-repeat> =
  <line-style>          |
  <repeat-line-style>

<auto-repeat-line-style> =
  repeat( auto , [ <line-style> ]# )

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<repeat-line-style> =
  repeat( [ <integer [1,∞]> ] , [ <line-style> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single style for the lines drawn between flex items.

HTML

We include a list of dynamic sports duos:

```
<ul>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
```

```
  <li>Kerri Walsh + Misty May Treanor</li>
</ul>
```

CSS

We define the list to be a flex container, creating rows by setting the `flex-direction` to `column` using the `flex-flow` shorthand. We include a `gap` of `5px` to provide enough room between the rows to fit our `3px` dashed red rule:

```
ul {
  display: flex;
  flex-flow: column;
  gap: 5px;
  row-rule-width: 3px;
  row-rule-color: red;

  row-rule-style: dashed;
}
```

Result

Repeating values

This example demonstrates how, when there are fewer values in the list of styles than row rules, the values are repeated.

CSS

Using the same HTML and CSS as in the previous example, we include three comma-separated styles as the `row-rule-style` value:

```
ul {
  row-rule-style: solid, dotted, dashed;
}
```

Result

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `row-rule-style` property value.

CSS

We use the same HTML and CSS as in the previous examples. We include a `repeat()` function, setting the list of two `<line-style>` values to be repeated 3 times.

```
ul {
  row-rule-style: double, repeat(3, inset, dashed), double;
}
```

Result

The flex container has six rows, so five gutters. The `repeat()` function repeats two style values three times, creating a list of eight style values, so the last three values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto` instead of an integer within the `repeat()` function.

CSS

Using `repeat(auto, <line-style>)` we set all row rules to `dotted`, except the first and last, which we set to `solid`.

```
ul {
  row-rule-style: solid, repeat(auto, dotted), solid;
}
```

Result

```
@layer no-support {
  @supports not (row-rule-style: solid, dotted) {
    body::before {
      content: "Your browser doesn't support the row-rule-style property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `row-rule-color`
- `row-rule-width`
- `column-rule-style`
- `row-rule` shorthand
- `rule-style` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

row-rule-visibility-items CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-visibility-items`** CSS property defines whether a row-rule segment is painted in gaps adjacent to empty areas.

Try it

```
row-rule-visibility-items: all;
```

```
row-rule-visibility-items: around;
```

```
row-rule-visibility-items: between;
```

```
row-rule-visibility-items: normal;
```

```
<section id="default-example">
  <section id="example-element">
    <p>One fish</p>
    <p>Two fish</p>
    <p>Red fish</p>
    <p>Blue fish</p>
    <cite>-- Dr. Seuss</cite>
  </section>
</section>
```

```
#example-element {
  display: grid;
  row-rule: solid 5px red;
  gap: 10px;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(3, 1fr);
}
cite {
  grid-column: 3;
  grid-row: 3;
}
```

Syntax

```
/* Keyword values */
row-rule-visibility-items: all;
row-rule-visibility-items: around;
row-rule-visibility-items: between;
row-rule-visibility-items: normal;

/* Global values */
row-rule-visibility-items: inherit;
row-rule-visibility-items: initial;
row-rule-visibility-items: revert;
row-rule-visibility-items: revert-layer;
row-rule-visibility-items: unset;
```

Values

This property is specified as one of the following keyword values:

`all`

The row rule should be painted in all gap segments, regardless of whether adjacent areas contain an item.

`around`

The row rule should be painted in a gap segment if at least one of the two adjacent areas is occupied by an item.

`between`

The row rule should be painted in a gap segment if both adjacent areas are occupied by items.

`normal`

Behaves the same as `all`. This is the default.

Description

The `row-rule-visibility-items` property defines whether, in multi-column and grid containers with more than one row, row rule segments are painted in the gaps between two adjacent areas if one or both of the areas are empty.

The `row-rule-visibility-items` and `column-rule-visibility-items` properties can both be set to the same values using the `rule-visibility-items` shorthand.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-visibility-items =
  all     |
  around  |
  between |
  normal
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a row rules to be drawn between two grid areas if at least one adjacent grid area contains grid items.

HTML

We include a list of dynamic sports duos:

```
<ol>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
```

```
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
  <li>Kerri Walsh + Misty May Treanor</li>
</ol>
```

CSS

We define the ordered list (`<ol>`) to be a grid container, creating 4 rows and 4 columns by setting both the `grid-template-rows` and `grid-template-columns` properties to `repeat(4, 1fr)`, and move the last item to the bottom-right grid area using `grid-row` and `grid-column`. We include a `gap` of `20px` to provide enough room between the rows to fit our `5px` dashed rule.

Finally, we set `row-rule-visibility-items` to `around`, so that a row rule segment is only painted in a row gap if one or both adjacent grid areas contain a grid item.

```
ol {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: repeat(4, 1fr);
  gap: 10px;

  row-rule: dashed 5px blue;
  row-rule-visibility-items: around;
}
li:last-child {
  grid-column: 4;
  grid-row: 4;
}
```

```
li {
  margin-left: 1em;
}
@layer no-support {
  @supports not (row-rule-visibility-items: around) {
    body::before {
      content: "Your browser doesn't support the row-rule-visibility-items property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

See also

- `rule-visibility-items` shorthand
- `column-rule-visibility-items`
- `row-rule` shorthand
- `rule` shorthand
- CSS gaps module

row-rule-width CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`row-rule-width`** CSS property defines the widths of the lines drawn between rows in multi-row grid, flex, and multi-col layouts.

Try it

```
row-rule-width: thin;
```

```
row-rule-width: thin, thick;
```

```
row-rule-width: repeat(2, thin, thick), 10px;
```

```
row-rule-width: thick, repeat(auto, 1px, 2px), thick;
```

```
row-rule-width: medium;
```

```
<section id="default-example">
  <ul id="example-element">
    <li>One fish</li>
    <li>Two fish</li>
    <li>Red fish</li>
    <li>Blue fish</li>
  </ul>
</section>
```

```
#example-element {
  display: flex;
  flex-flow: column;
  row-rule-style: solid;
  row-rule-color: magenta;
  gap: 5px;
  text-align: left;
}
```

Syntax

```
/* Keyword values */
row-rule-width: thin;
row-rule-width: medium;
row-rule-width: thick;
```

```
row-rule-width: thin, medium, thick;
row-rule-width: thick, repeat(5, thin), thick;
row-rule-width: thick, repeat(auto, thin, medium), thick;

/* Length values */
row-rule-width: 1px;
row-rule-width: 5px;
row-rule-width: 1px, 3px, 5px;
row-rule-width: 5px, repeat(auto, 1px), 10px, 15px;
row-rule-width: 5px, repeat(5, 1px, 3px), 5px;

/* Global values */
row-rule-width: inherit;
row-rule-width: initial;
row-rule-width: revert;
row-rule-width: revert-layer;
row-rule-width: unset;
```

Values

The `row-rule-width` property accepts a comma-separated list of values, including:

`<line-width>`

A `<line-width>`: This can be one of the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value, representing the width of the line. The default value is `medium`.

`<repeat-line-width>`

A `repeat()` function, with the first argument being an `<integer>` of `1` or more, and one or more `<line-width>` values as subsequent arguments. The integer defines the number of times the `<line-width>` values should be repeated.

`<auto-repeat-line-width>`

A `repeat()` function, with `auto` as the first argument and one or more `<line-width>` values as subsequent arguments. The provided `<line-width>` values are repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value.

Description

The `row-rule-width` property defines the widths of any row rule lines drawn in the gaps between rows in multi-column, flex, and grid containers with more than one row.

The value is a comma-separated list of components, which can include `<line-width>`, `<repeat-line-width>`, and `<auto-repeat-line-width>` types.

The `row-rule-width`, along with the `row-rule-color` and `row-rule-style` properties, can be set using the `row-rule` shorthand. The `row-rule-width`, along with the `column-rule-width` property, can also be set using the `rule-width` shorthand.

If the property value consists of only one `<line-width>`, all the row rules will be that width. If we declare the following, all row rules will be `3px`:

```
row-rule-width: 3px;
```

When more than one `<line-width>` is declared, they will be applied to row-rules in the order specified. If there are more row-rules than `<line-width>` values, the list of line widths is repeated until every rule has a width. If we declare the following, for example, every odd rule will be `thin`, and every even rule will be `1em`.

```
row-rule-width: thin, 1em;
```

Repeated line widths

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<line-width>` values passed as subsequent arguments the specified number of times. This allows the same widths to be repeated a set number of times without repeating the same values multiple times. The following declarations are equivalent:

```
row-rule-width: 1rem, thick, thin, thick, thin;
row-rule-width: 1rem, repeat(2, thick, thin);
```

You can use any `<line-width>` values, including custom properties that resolve to a `<line-width>`. Using `repeat()` can make values easier to write, especially when using complex length calculations. It enables a recurring pattern to be written using a single function, regardless of the number of rows.

If we set `--base: 1vh` and `--secondary: 1vw`, the following will provide similar results to the previous declaration:

```
row-rule-width:
  1rem,
  repeat(
    2,
    min(calc(var(--base) - 3px), 10px),
    abs(calc(var(--secondary) - 30px))
  ),
  thin;
```

This creates a list of six widths. If the number of widths in the `row-rule-width` value's width list exceeds the number of gaps between rows, the excess width values are ignored. If the container has three rows, the rule in the first gutter will be `1rem` wide, and the second is determined by the `min()` function.

If there are more gutters than widths, the list of widths is repeated. If the container has 7, 13, 19, or 25 rows, this sequence of widths will be repeated one, two, three, or four times, respectively, with the last rule being `thin`.

Auto-repeating line widths

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the list of `<line-width>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any row-rules that are not explicitly specified by other components of the property value.

```
row-rule-width: thin, repeat(auto, medium), thin;
```

In this case, it doesn't matter if the container has 3, 6, 11, 16, or 21 rows; the first and last rows will always be `thin`, and all other row rules will be `medium`. If there are only 2 or 3 rows, there will be no medium-sized row rules.

The `auto` keyword within the `repeat()` function creates an auto repeater that fills in values for row rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. At most, only one `repeat(auto, <width>)` can be present in a `row-rule-width` value.

Formal definition

Value not found in DB!

Formal syntax

```
row-rule-width =
  <line-width-list>       |
  <auto-line-width-list>

<line-width-list> =
  <line-width-or-repeat>#

<auto-line-width-list> =
  <line-width-or-repeat>#? , <auto-repeat-line-width> , <line-width-or-repeat>#?

<line-width-or-repeat> =
  <line-width>          |
  <repeat-line-width>

<auto-repeat-line-width> =
  repeat( auto , [ <line-width> ]# )

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<repeat-line-width> =
  repeat( [ <integer [1,∞]> ] , [ <line-width> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single width for the lines drawn between flex items.

HTML

We include a list of dynamic sports duos:

```
<ul>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
  <li>Kerri Walsh + Misty May Treanor</li>
</ul>
```

CSS

We define the list to be a flex container, creating rows by setting the `flex-direction` to `column` using the `flex-flow` shorthand. We include a `gap` of `5px` to provide enough room between the rows to fit our `3px` dashed red rule:

```
ul {
  display: flex;
  flex-flow: column;
  gap: 5px;
  row-rule-style: dashed;
  row-rule-color: red;
  row-rule-width: 3px;
}
```

Result

Repeating values

This example demonstrates how, when there are fewer values in the list of widths than row rules, the values are repeated.

CSS

Using the same HTML and CSS as in the previous example, we include three comma-separated widths as the `row-rule-width` value:

```
ul {
  row-rule-width: 1px, 3px, 5px;
}
```

Result

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `row-rule-width` property value and how this function can help reduce the verbosity of value declarations.

CSS

We use the same HTML and CSS as in the previous examples. To demonstrate how values can become verbose and the utility of the `repeat()` function, we declare two custom properties, which we use in `repeat()` function declarations. The `repeat()` function sets a list of two `<line-width>` values to repeat 3 times.

```
ul {
  --base: 0.5vw;
  --secondary: 1vw;
  row-rule-width:
    15px,
    repeat(
      3,
      min(calc(var(--base) + 3px), 10px),
      abs(calc(var(--secondary) - 2px))
```

```
    ),
    15px;
}
```

Result

The flex container has six rows, so five gutters. The `repeat()` function repeats two width values three times, creating a list of eight width values. As there are fewer row gutters than total widths, the last three values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto` instead of an integer within the `repeat()` function.

CSS

Using `repeat(auto, <line-width>)` we set all row rules to `1px`, except the first and last, which we set to `5px`.

```
ul {
  row-rule-width: 5px, repeat(auto, 1px), 5px;
}
```

Result

```
@layer no-support {
  @supports not (row-rule-width: thin, thick) {
    body::before {
      content: "Your browser doesn't support the row-rule-width property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `row-rule-color`
- `row-rule-style`
- `column-rule-width`
- `row-rule` shorthand
- `rule-width` shorthand
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

ruby-align CSS property

The **`ruby-align`** CSS property defines the distribution of the different ruby elements over the base.

Syntax

```
/* Keyword values */
ruby-align: start;
ruby-align: center;
ruby-align: space-between;
ruby-align: space-around;

/* Global values */
ruby-align: inherit;
ruby-align: initial;
ruby-align: revert;
ruby-align: revert-layer;
ruby-align: unset;
```

Values

This property is specified as one of the following keyword values:

`start`

Is a keyword indicating that the ruby will be aligned with the start of the base text.

`center`

Is a keyword indicating that the ruby will be aligned at the middle of the base text.

`space-between`

Is a keyword indicating that the extra space will be distributed between the elements of the ruby.

`space-around`

Is a keyword indicating that the extra space will be distributed between the elements of the ruby, and around it.

Formal definition

Initial value	`space-around`
Applies to	ruby bases, ruby annotations, ruby base containers, ruby annotation containers
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
ruby-align =
  start          |
```

```
  center         |
  space-between  |
  space-around
```

This syntax reflects the latest standard as per CSS Ruby Annotation Layout Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Ruby aligned at the start of the base text

HTML

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

CSS

```
ruby {
  ruby-align: start;
}
```

Result

Ruby aligned at the center of the base text

HTML

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

CSS

```
ruby {
  ruby-align: center;
}
```

Result

Extra space distributed between ruby elements

HTML

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

CSS

```
ruby {
  ruby-align: space-between;
}
```

Result

Extra space distributed between and around ruby elements

HTML

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

CSS

```
ruby {
  ruby-align: space-around;
}
```

Result

See also

- `ruby-position`
- CSS ruby layout module
- HTML Ruby elements: `<ruby>`, `<rt>`, `<rp>`, and `<rtc>`

ruby-overhang CSS property

The **`ruby-overhang`** CSS property specifies whether or not a `<ruby>` annotation overhangs any surrounding text.

Try it

```
ruby-overhang: auto;
```

```
ruby-overhang: none;
```

```
<section id="default-example">
  <p id="example-element">
    あの<ruby>表<rp>(</rp><rt>ひょう</rt><rp>)</rp></ruby
    ><ruby>現<rp>(</rp><rt>げん</rt><rp>)</rp></ruby>は面白い。
  </p>
</section>
```

```
#default-example {
  font-size: 2em;
}
```

Syntax

```
/* Keyword values */
ruby-overhang: auto;
ruby-overhang: none;

/* Global values */
ruby-overhang: inherit;
ruby-overhang: initial;
ruby-overhang: revert;
ruby-overhang: revert-layer;
ruby-overhang: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

> When a ruby annotation container is longer than its corresponding base container, the annotation may partially overlap adjacent text. Whether, and how much to overhang are determined by the user agent.

`none`

> A keyword indicating that the ruby is never allowed to extend past adjacent containers.

Description

The `ruby-overhang` property controls whether the ruby annotation text box (`<rt>`) may overlap adjacent text outside the `<ruby>` container box.

When ruby annotation text is not allowed to overhang — when `ruby-overhang: none` is set on the `<ruby>` element — that element behaves like an inline box, as if its `display` property were set to `inline` with only its own contents rendered within its boundaries and adjacent elements not crossing the boundary box.

By default, the content of an `<rt>` element is allowed to overhang, so content may overlap the `<ruby>` container box, partially rendering over or under surrounding inline-level content. With `auto`, the default, content may overhang, but it will not overhang if doing so would overlap adjacent `<rt>` elements or elements with a `display` value resolving to `ruby-base` or `ruby-text`.

Formal definition

Initial value	`auto`
Applies to	ruby annotation containers
Inherited	yes
Computed value	the specified keyword
Animation type	by computed value type

Formal syntax

```
ruby-overhang =
  auto    |
  spaces
```

This syntax reflects the latest standard as per CSS Ruby Annotation Layout Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Ruby overhanging base text

This examples demonstrates both values of the `ruby-overhang` property.

HTML

We include two paragraphs with identical `<ruby>` content and structures, other than their class names.

```
<p class="auto">
  あの<ruby>表<rp>(</rp><rt>ひょう</rt><rp>)</rp></ruby
  ><ruby>現<rp>(</rp><rt>げん</rt><rp>)</rp></ruby>は面白い。
</p>

<p class="none">
  あの<ruby>表<rp>(</rp><rt>ひょう</rt><rp>)</rp></ruby
  ><ruby>現<rp>(</rp><rt>げん</rt><rp>)</rp></ruby>は面白い。
</p>
```

CSS

A red `outline` of `1px` helps highlight the text annotation of `<rt>` elements. The first paragraph has `ruby-overhang: auto` and the second has `ruby-overhang: none`.

```
p {
  font-size: 40px;
  display: block;
  margin: 0.5rem;
}
rt {
  font-size: 28px;
  outline: 1px solid red;
}
.auto {
  ruby-overhang: auto;
}
.none {
  ruby-overhang: none;
}
```

Results

When `ruby-overhang` is set to `none`, the annotation text is not allowed to overlap the adjacent boxes of base ruby text. If you look closely, you may notice that in the first paragraph, the red box encasing the ruby text slightly overlaps parts of non-associated `<ruby>` content, whereas in the `none` example in supporting browsers, there is no overlap between ruby content and not-associated ruby text.

See also

- `ruby-align`
- `text-transform`: full-size-kana
- `<ruby>`
- `<rt>`
- `<rp>`

ruby-position CSS property

The **`ruby-position`** CSS property defines the position of a ruby element relative to its base element. It can be positioned over the element (`over`), under it (`under`), or to the right (`inter-character`).

Try it

```
ruby-position: over;
```

```
ruby-position: under;
```

```
<section id="default-example">
  <ruby id="example-element"> 明日 <rp>(</rp><rt>Ashita</rt><rp>)</rp> </ruby>
</section>
```

```
#example-element {
  font-size: 2em;
}
```

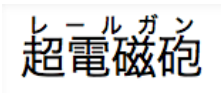
Syntax

```
/* Keyword values */
ruby-position: over;
ruby-position: under;
ruby-position: alternate;
ruby-position: alternate over;
ruby-position: alternate under;
ruby-position: inter-character;

/* Global values */
ruby-position: inherit;
ruby-position: initial;
ruby-position: revert;
ruby-position: revert-layer;
ruby-position: unset;
```

Values

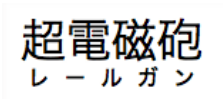
This property is specified as one of the following keyword values:

`over`



Is a keyword indicating that the ruby has to be placed over the main text for horizontal scripts and right to it for vertical scripts.

`under`



Is a keyword indicating that the ruby has to be placed under the main text for horizontal scripts and left to it for vertical scripts.

`alternate`

Is a keyword indicating that the ruby alternates between over and under, when there are multiple levels of annotation.

`inter-character`

Is a keyword indicating that the ruby annotation is placed to the right of the base text characters. This value is intended for bopomofo (zhuyin fuhao) ruby.

Formal definition

Initial value	`alternate`
Applies to	ruby annotation containers
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
ruby-position =
  [ alternate || [ over | under ] ]  |
  inter-character
```

This syntax reflects the latest standard as per CSS Ruby Annotation Layout Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Ruby positioned over the text

HTML

```
<ruby>
  <rb>超電磁砲</rb>
  <rp> (</rp><rt>レールガン</rt><rp>) </rp>
</ruby>
```

CSS

```
ruby {
  ruby-position: over;
}
```

Result

Ruby positioned under the text

HTML

```
<ruby>
  <rb>超電磁砲</rb>
  <rp> (</rp><rt>レールガン</rt><rp>) </rp>
</ruby>
```

CSS

```
ruby {
  ruby-position: under;
}
```

Result

Ruby alternate

HTML

```
<ruby>
  <rb>A</rb><rb>B</rb><rb>C</rb>
  <rtc>Above</rtc>
  <rtc>Below</rtc>
</ruby>
```

CSS

```
ruby {
  ruby-position: alternate; /* this is also the initial value */
}
```

Result

See also

- `<ruby>`, `<rt>`, `<rp>`, and `<rtc>` HTML elements
- `ruby-align`
- CSS ruby layout module

rule CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule`** CSS shorthand property sets the width, style, and color of the line drawn between rows and columns in multi-row grid, flex, and multi-col layouts, setting both the column and row rules to the same values.

Try it

```
rule: solid;
```

```
rule: dotted medium blue;
```

```
rule:
  dotted medium blue,
  repeat(3, dotted red 2px, double orange 5px);
```

```
rule:
  dashed medium magenta,
  repeat(auto, dotted blue 2px, dotted blue 5px),
  dashed medium magenta;
```

```
rule:
  dashed medium magenta,
  repeat(auto, dotted blue 2px),
  outset goldenrod 5px;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
```

```
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick;
}
#example-element i {
  padding: 5px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `rule-color`
- `rule-style`
- `rule-width`

Syntax

```
/* One value */
rule: dotted;
rule: solid 8px;
rule: solid blue;
rule: thick inset blue;

/* Multiple values */
rule: groove, dashed, solid;
rule:
  dotted medium blue,
  dashed magenta 1px,
  outset green 5px;
rule:
  solid #0ff,
  repeat(3, dashed magenta 1px, outset green 5px);
rule:
  inset 3px yellow,
  repeat(auto, dashed magenta 1px, groove green 5px),
  inset 3px yellow;

/* Global values */
rule: inherit;
rule: initial;
rule: revert;
rule: revert-layer;
rule: unset;
```

Values

The `rule` property accepts a comma-separated list of values, including:

`<gap-rule>`

Specified as one, two, or three of the values listed below, in any order.

`<'line-width'>`

A `<line-width>`: a positive `<length>` or one of the three keywords, `thin`, `medium`, or `thick`. The default value is `medium`. See `rule-width`.

`<'line-style'>`

A `<line-style>`: one of `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`. The default value is `none`. See `rule-style`.

`<'color'>`

A `<color>` value representing the color of the line. The default value is `currentcolor`. See `rule-color`.

`<gap-repeat-rule>`

A `repeat()` function, with an `<integer>` of `1` or more as the first argument and one or more `<gap-rule>` values as subsequent arguments. The `<integer>` specifies how many times the list of `<gap-rule>` values should be repeated.

`<gap-auto-repeat-rule>`

A `repeat()` function, with `auto` as the first argument and one or more `<gap-rule>` values as subsequent arguments. The provided list of `<gap-rule>` values is repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

Description

The `rule` property defines the line style of any rule lines drawn in the gaps between rows and columns in multi-column, flex, and grid containers with more than one row or column.

The `rule` is shorthand for `rule-color`, `rule-style`, and `rule-width`. It sets the `row-rule` and `column-rule` shorthand properties to the same value.

The property value is a comma-separated list of components, which can include `<gap-rule>`, `<gap-repeat-rule>`, and `<gap-auto-repeat-rule>` types. Each `<gap-rule>` defines the width, color, and style of one or more rule lines.

If the property value consists of only one `<gap-rule>`, all the row and column rules will be that style, color, and size. If we declare the following, all row and column rules will be `dashed red 3px`:

```
rule: dashed red 3px;
```

When more than one `<gap-rule>` is declared, they will be applied to rules in the order specified. If there are more gutters between rows and columns than `<gap-rule>` values, the list of values is repeated until every row and column rule has a gap line. If we declare the following, for example, every odd rule will be `dashed red 3px`, and every even rule will be `dotted blue 5px`, in both directions.

```
rule:
  dashed red 3px,
  dotted blue 5px;
```

Repeated line styles

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<gap-rule>` values passed as subsequent arguments the specified number of times. This allows the same `<gap-rule>` to be repeated a set number of times without repeating the same CSS multiple times. The following declarations are equivalent:

```
rule:
  solid red 5px,
  outset blue 10px,
  inset green 1px,
  outset blue 10px,
  inset green 1px,
  outset blue 10px,
  inset green 1px;
rule:
  solid red 5px,
  repeat(3, outset blue 10px, inset green 1px);
```

This creates a list of seven rules. If the number of styles in the `rule` value's style list exceeds the number of gaps between rows and columns, the excess style values are ignored. If the container on which this is applied has three rows and columns, the rule in the first gutter will be `solid red 5px` and the second `outset blue 10px`, in both directions.

If there are more gutters than styles, the list of styles is repeated. If the container has 8, 15, 22, or 29 rows or columns, this sequence of styles will be repeated in that direction one, two, three, or four times, respectively, with the last rule being `inset green 1px`.

Auto-repeating line styles

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<gap-rule>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any row and column rules that are not explicitly specified by other components of the property value.

```
rule:
  solid red 5px,
  repeat(auto, dotted green 1px, dashed blue 1px),
  solid red 5px;
```

In this case, the first and last row and column rules will be `solid red 5px`, and all others will alternate between `dotted green 1px` and `dashed blue 1px`. It doesn't matter if the container has 3, 6, 11, 16, or 21 rows and columns; the first and last gutters will always have a thick solid red line painted between them (unless `rule-visibility-items` leads to no line being drawn), and all the other row and column rules will be thin, dotted green or dashed blue lines. If there are only 2 or 3 rows and columns, there will be no dotted or dashed lines.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for row and column rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. At most, only one `repeat(auto, <gap-rule>)` can be present in a `rule` value.

Formal definition

Value not found in DB!

Formal syntax

```
rule =
  <'column-rule'>

<column-rule> =
  <gap-rule-list>      |
  <gap-auto-rule-list>

<gap-rule-list> =
  <gap-rule-or-repeat>#

<gap-auto-rule-list> =
  <gap-rule-or-repeat>#? , <gap-auto-repeat-rule> , <gap-rule-or-repeat>#?

<gap-rule-or-repeat> =
  <gap-rule>         |
  <gap-repeat-rule>

<gap-auto-repeat-rule> =
  repeat( auto , <gap-rule># )

<gap-rule> =
  <line-width>  ||
  <line-style>  ||
  <color>

<gap-repeat-rule> =
  repeat( <integer [1,∞]> , <gap-rule># )

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
```

```
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single rule for the lines drawn in the gutters between grid items.

HTML

We create a list of 75 items. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>74</li>
  <li>75</li>
</ul>
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
  <li>51</li>
  <li>52</li>
  <li>53</li>
  <li>54</li>
  <li>55</li>
  <li>56</li>
  <li>57</li>
  <li>58</li>
  <li>59</li>
  <li>60</li>
  <li>61</li>
  <li>62</li>
  <li>63</li>
  <li>64</li>
  <li>65</li>
  <li>66</li>
  <li>67</li>
  <li>68</li>
  <li>69</li>
  <li>70</li>
  <li>71</li>
```

```
  <li>72</li>
  <li>73</li>
  <li>74</li>
  <li>75</li>
</ul>
```

CSS

We define the unordered list as a 10-column container, creating columns and rows with the `grid-template-columns` property and setting `list-style-type` to `none` to remove the bullets. We include a `gap` of `5px` to provide enough room between the columns and rows to fit our `dashed 3px magenta` rule.

```
ul {
  display: grid;
  grid-template-columns: repeat(10, 1fr);
  list-style-type: none;
  gap: 5px;

  rule: dashed 3px magenta;
}
li {
  text-align: center;
  aspect-ratio: 1;
}
```

Result

Multiple gap-rules and default values

This example demonstrates using multiple, comma-separated values. It also demonstrates the default values for the width, color, and style of `medium`, `currentcolor` and `none`, respectively.

CSS

Using the same HTML and CSS as in the previous example, we include four comma-separated `<gap-rule>` values as the `rule` value, omitting the `<line-width>` in the first `<gap-rule>`, the `<color>` in the second, and the `<line-style>` from the third, with the fourth including all three components:

```
ul {
  rule:
    red dashed,
    1px dotted,
    5px blue,
    10px magenta solid;
}
```

Result

The red line is `3px` wide, the dotted line is the same color as the text, and there is no `5px`-wide blue line, as the style of the third `<gap-rule>` defaults to `none`, so no line is painted. As there are fewer rule styles than gutters, the list of rules is repeated until all rule lines are styled.

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `rule` property value.

CSS

We use the same HTML and CSS as in the previous examples, overriding the `rule` value with a comma-separated list of three components: two `<gap-rule>` values and a `<gap-repeat-rule>` that sets a list of two `<gap-rule>` values to repeat 3 times.

```
ul {
  rule:
    3px red dashed,
    repeat(3, dotted green 1px, dashed blue 1px),
    3px red dashed;
}
```

Result

The grid has ten columns and eight rows, so nine column gutters and seven row gutters. The `repeat()` function repeats two style values three times, creating a list of eight style values. As there are fewer row gutters than values, the last value is not used in the row direction. As there are more column gutters than values, the list repeats in the column direction.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using the `auto` argument instead of an integer in the `repeat()` function.

CSS

Using `repeat(auto, <gap-rule>)`, we set all row and column rules to `1px dotted` (with the color defaulting to the current color), except the first and last rules, which we set to `3px solid red`.

```
ul {
  rule:
    3px red solid,
    repeat(auto, 1px dotted),
    3px red solid;
}
```

Result

```
@layer no-support {
  @supports not (rule: thin, thick) {
    body::before {
      content: "Your browser doesn't support the rule property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `rule-color`
- `rule-width`
- `rule-style`
- `column-rule` shorthand
- `row-rule` shorthand
- CSS gaps module

rule-break CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-break`** CSS shorthand property sets the behavior for breaking column and row rules into segments where rows and column gaps intersect, setting `column-rule-break` and `row-rule-break` to the same value.

Try it

```
rule-break: none;
```

```
rule-break: normal;
```

```
rule-break: intersection;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  row-rule: solid thick orange;
  column-rule: solid thick purple;
  gap: 10px;
}
#example-element i {
  padding: 5px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-break`
- `row-rule-break`

Syntax

```
/* Keyword values */
rule-break: none;
rule-break: normal;
rule-break: intersection;

/* Global values */
rule-break: inherit;
rule-break: initial;
rule-break: revert;
rule-break: revert-layer;
rule-break: unset;
```

Values

This property is specified as a single keyword from the following list:

`none`

There are no breaks where row and column rules intersect; rather, a single continuous decoration is painted from one end of each gap to the other.

`normal`

In flex and grid containers, behaves as `none`. In multi-col, `column-rule-break` behaves as `intersection` and `row-rule-break` behaves as `none`. This is the default value.

`intersection`

Rules always break when they intersect a gap, with rule segments starting and ending at container and gap edges.

Description

The `rule-break` property is used to set the behavior for breaking rules into segments when they cross gaps.

Gap decorations are painted within a gap as one or more gap decoration segments, with segments occurring between any two adjacent items. Depending on the container type, by default, these segments may either end at the edge of a gap, or the column and row rule may extend the entire height and width of the container.

When there is a break in a rule, by default, the segments start and end at the edge of the gaps. If the gap size is `0`, the break may not be visible. The `rule-break` property determines if the break occurs. You can control the size of the break at the end of each segment with the `rule-inset` properties. When there are no breaks, and the line is continuous, `rule-inset` properties only affect the rule at the start and end edges of the container. When there are breaks, the `rule-inset` properties affect the start and end of every rule segment.

Whether a rule is by default composed of a single continuous segment or segments that break when intersecting gaps depends on the container type.

Grid containers

In grid containers, by default, rule segments continue through visible "cross" intersections. Setting `rule-break: intersection` will force the segments to break where they would otherwise cross.

```
<h1>Default rule breaks in grid</h1>
<div class="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
<p>
  <label
    ><input type="checkbox" /> Set <code>rule-break: intersection</code></label
  >
</p>
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}

:has(:checked) .grid {
  rule-break: intersection;
}
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  rule: 5px solid red;
  width: 100%;
}

.grid > div {
  border: 1px solid green;
  background-color: lime;
  height: 30px;
}

@layer no-support {
  @supports not (rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Check the checkbox to set the `rule-break` to `intersection`, which makes the continuous rules break at every "cross" intersection.

Flex containers

In flexbox, when the `flex-direction` is `row` or `row-reverse`, the row rule is continuous, with the column segments starting and ending at the edge of the row gaps. When the `flex-direction` is `column` or `column-reverse`, the column rule is continuous, with the row segments starting and ending at the edge of the column gaps.

```
<h1>Default rule breaks in flexbox</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
```

```
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
<p>
  <label
    ><input type="checkbox" /> Set <code>rule-break: intersection</code></label
  >
</p>
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}

:has(:checked) .flexbox {
  rule-break: intersection;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox {
  display: flex;
  flex-flow: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid red;
  width: 100%;
}
.column {
  flex-flow: column balance;
  gap: 20px;
}

.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
@layer no-support {
  @supports not (rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Note how, depending on the `flex-direction`, the column or row rules are continuous by default. If you set the `rule-break` to `intersection`, those continuous rules break at every intersection.

Multi-col containers

In multi-col containers, the default `normal` behavior differs between row rules and column rules. Column rule segments start and end when they intersect a row gap, behaving as `column-rule-break: intersection`, while row rules don't break when they intersect a column gap, behaving as `row-rule-break: none`.

```
<h1>Default rule breaks in multi-col</h1>
<ol>
  <li>One fish</li>
  <li>Two fish</li>
  <li>Red fish</li>
  <li>Blue fish</li>
  <li>Black fish</li>
  <li>Blue fish</li>
  <li>Old fish</li>
  <li>New fish.</li>
  <li>This one has a little star.</li>
  <li>This one has a little car.</li>
  <li>Say! What a lot</li>
  <li>Of fish there are.</li>
  <li>Yes. Some are red.</li>
  <li>And some are blue.</li>
  <li>Some are old.</li>
  <li>And some are new.</li>
  <li>Some are sad.</li>
  <li>And some are glad.</li>
  <li>And some are very, very bad.</li>
  <li>Why are they</li>
  <li>Sad and glad and bad?</li>
```

```
  <li>I do not know.</li>
  <li>Go ask your dad.</li>
</ol>
<fieldset>
  <legend>Set <code>rule-break:</code></legend>
  <label
    ><input type="radio" name="break" value="none" /> <code>none</code></label
  >
  <label
    ><input type="radio" name="break" value="normal" checked />
    <code>normal</code></label
  >
  <label
    ><input type="radio" name="break" value="intersection" />
    <code>intersection</code></label
  >
</fieldset>
```

```
body {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
ol {
  columns: 3 / 4em;
  gap: 20px;
  rule: 5px solid red;
}
li {
  border: 1px solid green;
  background-color: lime;
  list-style-type: none;
  margin-bottom: 5px;
}
:has([value="normal"]:checked) ol {
  rule-break: normal;
}
:has([value="intersection"]:checked) ol {
  rule-break: intersection;
}
:has([value="none"]:checked) ol {
  rule-break: none;
}
label {
  margin-right: 20px;
}
@layer no-support {
  @supports not (rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Check the radio buttons for each value to see the value's effect.

Formal definition

Value not found in DB!

Formal syntax

```
rule-break =
  <'column-rule-break'>

<column-rule-break> =
  none          |
  normal        |
  intersection
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we use the `rule-break` property to break each rule segment in a grid container so no rules intersect.

HTML

We create a list of 50 items. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>49</li>
  <li>50</li>
</ul>
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
```

```
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
</ul>
```

CSS

We define the unordered list as a 8-column container, creating columns and rows with the `grid-template-columns` property and setting `list-style-type` to `none` to remove the bullets. We include a `gap` of `20px` to provide enough room between the columns and rows to fit our `20px` solid column and row rules. Last, we set the rule segments to break instead of intersect.

```
ul {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  list-style-type: none;
  gap: 20px;

  row-rule: 20px solid palegoldenrod;
  column-rule: 20px solid olive;

  rule-break: intersection;
}
```

The rest of the CSS is hidden for brevity.

```
ol {
  place-items: center;
}
li {
  text-align: center;
  aspect-ratio: 1;
  font-family: sans-serif;
  line-height: 50px;
}
@layer no-support {
  @supports not (rule-break: intersection) {
    body::before {
      content: "Your browser doesn't support the rule-break property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

See also

- `rule-inset` shorthand
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

rule-color CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-color`** CSS shorthand property defines the colors of the lines drawn between columns and rows in multi-column grid, flex, and multi-col layouts, setting the colors of the column and row rules to the same value.

Try it

```
rule-color: purple;
```

```
rule-color: rgb(48 125 222), rgb(222 48 125);
```

```
rule-color: rgb(48 125 222), repeat(3, rgb(222 48 125));
```

```
rule-color: purple, repeat(auto, red, yellow);
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick;
}
#example-element i {
  padding: 5px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-color`
- `row-rule-color`

Syntax

```
/* Single <color> value */
rule-color: purple;
rule-color: rgb(192 56 78);
rule-color: transparent;
rule-color: hsl(0 100% 50% / 60%);

/* Multiple values */
rule-color: purple, magenta;
rule-color: repeat(3, purple), repeat(3, transparent);
rule-color: repeat(3, purple), repeat(3, yellow, blue);
rule-color: purple, repeat(auto, transparent), purple;
rule-color: purple, repeat(auto, blue, yellow), purple;
rule-color: repeat(3, purple), repeat(auto, transparent), repeat(3, purple);

/* Global values */
rule-color: inherit;
rule-color: initial;
rule-color: revert;
rule-color: revert-layer;
rule-color: unset;
```

Values

The `rule-color` property accepts a comma-separated list of values, including:

`<line-color>`

A `<color>` representing the color of the line.

`<repeat-line-color>`

A `repeat()` function, with an `<integer>` of `1` or more as the first argument and one or more `<color>` values as subsequent arguments. The `<integer>` specifies how many times the `<color>` values should be repeated.

`<auto-repeat-line-color>`

A `repeat()` function, with `auto` as the first argument and one or more `<color>` values as subsequent arguments. The provided `<color>` values are repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

Description

The `rule-color` property defines the colors of any lines drawn in the gaps between columns and rows in multi-column, flex, and grid containers with more than one column or row. It is a shorthand property that sets both the `row-rule-color` and `column-rule-color` properties to the same value.

The value is a comma-separated list of components, which can include `<line-color>`, `<repeat-line-color>`, and `<auto-repeat-line-color>` types. The `rule-color` property, along with the `rule-width` and `rule-style` properties, can be set using the `rule` shorthand.

Line colors

A `<line-color>` can be declared as any valid CSS `<color>` value. If the property value consists of only one `<color>`, all the rule lines will be that color. For example, if we declare the following, the lines in the gutters between columns and rows will all be blue:

```
rule-color: blue;
```

When multiple `<line-color>` values are declared, they will be applied to lines painted in the column and row gutters in the order specified. If there are more rules than `<line-color>` values, the list of colors is repeated until every column-rule has a color. If we declare the following, for example, every odd rule will be red, and every even rule will be yellow.

```
rule-color: red, yellow;
```

Repeated line colors

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<color>` values passed as subsequent arguments the specified number of times. This allows the color values to be repeated as many times as you need without having to list them individually. The following declarations are equivalent:

```
rule-color: blue, yellow, red, yellow, red, yellow, red;
rule-color: blue, repeat(3, yellow, red);
```

This creates a list of seven colors. If the number of colors in the `rule-color` value's color list exceeds the number of gaps between columns and rows, the excess color values are ignored. If there are fewer colors than gutters, the list of values is repeated until every rule has an associated color. For example, if the container has three columns and 18 rows, the rule in the first column gutter will be blue and the second yellow. For the row rules, the sequence repeats, with the first, eighth, and fifteenth row rules being blue.

Auto-repeating line colors

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<color>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any column and row rules that are not explicitly specified by other components of the property value.

```
rule-color: blue, repeat(auto, yellow), red;
```

In this case, the first column and row rules will be blue, the last will be red, and all others will be yellow. As long as there are at least two rules in either direction, the first rule will always be blue and the last will always be red. All the other rules will be yellow, which means if there are only 2 or 3 columns and rows, there will be no yellow lines.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for the rule line colors that would not otherwise receive values from other parts of the list, preventing the list from being cycled. A `rule-color` value can include, at most, one `repeat(auto, <color>)`.

Formal definition

Value not found in DB!

Formal syntax

```
rule-color =
  <'column-rule-color'>

<column-rule-color> =
  <line-color-list>      |
  <auto-line-color-list>

<line-color-list> =
  <line-color-or-repeat>#

<auto-line-color-list> =
  <line-color-or-repeat>#? , <auto-repeat-line-color> , <line-color-or-repeat>#?

<line-color-or-repeat> =
  <color>              |
  <repeat-line-color>

<auto-repeat-line-color> =
  repeat( auto , [ <color> ]# )

<repeat-line-color> =
  repeat( [ <integer [1,∞]> ] , [ <color> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single `<color>` for the lines drawn between the columns and rows of items in a grid container.

HTML

We create a list of 75 items. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
```

```
  <li>2</li>
  ...
  <li>74</li>
  <li>75</li>
</ul>
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
  <li>51</li>
  <li>52</li>
  <li>53</li>
  <li>54</li>
  <li>55</li>
  <li>56</li>
  <li>57</li>
  <li>58</li>
  <li>59</li>
  <li>60</li>
  <li>61</li>
  <li>62</li>
  <li>63</li>
  <li>64</li>
  <li>65</li>
  <li>66</li>
  <li>67</li>
  <li>68</li>
  <li>69</li>
  <li>70</li>
  <li>71</li>
  <li>72</li>
  <li>73</li>
  <li>74</li>
  <li>75</li>
</ul>
```

CSS

We define the unordered list as a 10-column container, creating columns and rows with the `grid-template-columns` property. We include a `gap` of `5px` to provide enough room between the columns and rows to fit our `3px` dashed rule, and set `list-style-type` to `none` to remove the bullets.

We include a `gap` of `5px` to provide enough room between the items to fit our medium dashed rule. We set the `rule-color` to `#22BB22`, a green `<hex-color>` value:

```
ul {
  display: grid;
  grid-template-columns: repeat(10, 1fr);
  list-style-type: none;
  gap: 5px;
  rule-style: dashed;
  rule-width: medium;

  rule-color: #22bb22;
}
li {
```

```
  text-align: center;
  aspect-ratio: 1;
}
```

```
@layer no-support {
  @supports not (rule-color: red) {
    body::before {
      content: "Your browser doesn't support the rule-color property";
      background-color: wheat;
      text-align: center;
      padding: 1rem 0;

      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
    }
  }
}
```

Result

Multiple color values

This example demonstrates declaring more than one color, and how the values are repeated when there are fewer values in the list of colors than gutters between columns and rows.

Using the same HTML and CSS as in the previous example, we include three comma-separated colors as the `rule-color` value:

```
@layer no-support {
  @supports not (rule-color: red, blue) {
    body::before {
      content: "Your browser doesn't support multiple values for the rule-color property";
      background-color: wheat;
      text-align: center;
      padding: 1rem 0;

      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
    }
  }
}
```

```
ul {
  rule-color: blue, yellow, red;
}
```

Result

There are nine column gutters and six row gutters, but only three colors in our color list, so the list gets repeated, with the first, fourth, and seventh lines being blue.

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `rule-color` property value and how this function can help prevent complex values from becoming unwieldy.

CSS

To demonstrate how values can become complicated and the utility of the `repeat()` function, we declare two custom properties, which we use in four `color-mix()` color function declarations to create blue, redish, tealish, and yellow colors. The redish and tealish `color-mix()` colors are within a `repeat()` function, set to repeat 3 times.

We also added a border around each grid item so you can see how the line is rule in the middle of the gutter between the columns and rows.

```
ul {
  --base: yellow;
  --mixin: blue;

  rule-color:
    color-mix(in lch decreasing hue, var(--base) 0%, var(--mixin)),
    repeat(
      3,
      color-mix(in lch decreasing hue, var(--base) 58%, var(--mixin)),
      color-mix(in lch increasing hue, var(--base) 58%, var(--mixin))
    ),
    color-mix(in lch decreasing hue, var(--base) 100%, var(--mixin));
}
```

Result

The grid has 10 columns and 7 rows, creating 9 column and 6 row gutters. The `repeat()` function repeats the two contained mixed colors three times, creating a color list with eight colors in all. While there is a lot of CSS to create the four colors, at least we didn't have to write out all eight `color-mix()` functions. Since there are more column gutters than list colors, the colors are repeated for the column gutters. As there are fewer row gutters than colors, the last two colors in the list are not used for the row gutters.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto`, instead of an integer, within the `repeat()` function.

We use the same HTML and CSS as in the previous examples, but override the `rule-color` value. Here, we use `repeat(auto, <color>)` to set all the lines to be almost transparent black (`#00000033`), except the first and last, which we set to a solid `black`.

```
ul {
  rule-color: black, repeat(auto, #00000033), black;
}
```

Result

Even though there are more column rule lines than row rule lines, the `<auto-repeat-line-color>` value enables the creation of this symmetric effect.

```
@layer no-support {
  @supports not (rule-color: repeat(3, red)) {
    body::before {
      content: "Your browser doesn't support `repeat()` functions within a rule-color property value";
      background-color: wheat;
      text-align: center;
      padding: 1rem 0;

      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
    }
  }
}
```

See also

- The `<color>` data type
- `rule-width`
- `rule-style`
- `row-rule-color`
- `column-rule-color`
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

rule-inset-cap CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-inset-cap`** CSS shorthand property can be used to offset the column and row rule segment cap endpoints to the same value.

Try it

```
rule-inset-cap: -20px;
```

```
rule-inset-cap: 0;
```

```
rule-inset-cap: 1em;
```

```
rule-inset-cap: 100%;
```

```
rule-inset-cap: overlap-join;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>

    <i id="y">Y</i>
    <i id="z">Z</i>
    <i id="bang">!</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid thick magenta;
  column-rule-color: rebeccapurple;
  gap: 1em;
  rule-overlap: column-over-row;
  rule-visibility-items: between;
  border: 1px solid rebeccapurple;
  overflow: visible;
  margin: 1em;
}
#example-element i {
  padding: 8px;
  border: 1px dashed;
}
#y {
  grid-column: 4 / 5;
  grid-row: 4 / 5;
}
#z {
  grid-column: 5 / 6;
  grid-row: 4 / 5;
}
```

```
#bang {
  grid-column: 6 / 7;
  grid-row: 4 / 5;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-inset-cap`
- `row-rule-inset-cap`

Syntax

```
/* Keywords */
rule-inset-cap: overlap-join;

/* A <length-percentage> value */
rule-inset-cap: 0;
rule-inset-cap: 1em;
rule-inset-cap: -5px;
rule-inset-cap: -25%;

/* Two values */
rule-inset-cap: 0 20px;
rule-inset-cap: 1em -5px;
rule-inset-cap: overlap-join -25%;

/* Global values */
rule-inset-cap: inherit;
rule-inset-cap: initial;
rule-inset-cap: revert;
rule-inset-cap: revert-layer;
rule-inset-cap: unset;
```

Values

This property is specified as one or two values from the following list:

`overlap-join`

> Resolves to `0`.

`<length-percentage>`

> Specifies the size of the inset. Percentage values resolve to `0` for caps at the container's edge. For interior gaps, percentages are relative to the height of the adjacent `row-gap` for column segment caps, or the width of the adjacent `column-gap` for row segment caps.

Description

The `rule-inset-cap` shorthand property can be used to set the `row-rule-inset-cap` and `column-rule-inset-cap` properties to the same value in a single declaration, insetting row and column cap segment endpoints by the specified values.

If you specify one value, both the start and end cap segment endpoints are set to that value. If you specify two values, the row and column cap start segment endpoints are set to the first value, and the row and column cap end segment endpoints are set to the second value.

The default value is `0`, which, in the case of cap endpoints, is the same as `overlap-join`. Positive values reduce the segment size, while negative values increase it.

The `rule-inset-cap` property, along with the `rule-inset-junction` property, can be set using the `rule-inset` shorthand.

Formal definition

Value not found in DB!

Formal syntax

```
rule-inset-cap =
  <'column-rule-inset-cap'>

<column-rule-inset-cap> =
  <inset-value> <inset-value>?

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example shows how to set `rule-inset-cap` to inset the endpoints of cap segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting cap endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
```

```
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a light blue `rule` to paint in both row and column gaps, then override the `column-rule-color`, setting darker `blue` vertical gap decorations. Finally, we set the `rule-inset-cap` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid lightblue;
  column-rule-color: blue;

  rule-inset-cap: 16px;
}
```

We also set the `flex-direction` on the `.column` container to make its items flow in columns rather than rows. We've hidden the other CSS styles, and the code that make the form interactive, for brevity.

```
.column {
  flex-direction: column;
}
```

```
h1,
article {
  font-family: sans-serif;
  text-align: center;
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 90vw;
  margin: auto;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the rule-inset-cap property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
```

```
  containers[0].style.ruleInsetCap = val;
  containers[1].style.ruleInsetCap = val;
  output.innerText = val;
});
```

Result

Change the size of the inset.

Setting two values

HTML

```
<p>
  <label
    >Change the <code>row-rule-inset-cap</code> value.
    <input type="range" min="-40" max="40" value="40" id="row" data-unit="px"
  /></label>
  <output id="og">40px</output>
</p>
<p>
  <label
    >Change the <code>column-rule-inset-cap</code> value.
    <input type="range" min="-40" max="40" value="-40" id="col" data-unit="px"
  /></label>
  <output id="ow">-40px</output>
</p>
```

Our HTML includes an unordered list (`<ul>`) with nine list items (`<li>`):

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
</ul>
```

The user interface for selecting different values and the JavaScript that enables the interactivity have been hidden for brevity.

CSS

We create a grid container by setting the `display` to grid, creating 4 columns with `grid-template-columns`, and adding a `30px` `gap`. We use the `row-rule` and `column-rule` properties to define our rules. We then use the `rule-inset-cap` property to inset the start cap segment endpoints by `40px` and outset the end cap segment endpoints by `-40px`.

We also set the 6th grid item to span two columns.

```
ul {
  display: grid;
  grid-template-columns: repeat(4, auto);
  gap: 30px;
  row-rule: 16px solid olive;
  column-rule: 10px solid palegoldenrod;

  rule-inset-cap: 40px -40px;
}
li:nth-of-type(6) {
  grid-column-end: span 2;
}
```

```
ul {
  width: calc(95vw - 80px);
  border: 1px solid;
  list-style-type: none;
  margin: auto;
  padding: 0;
  place-items: center;
}
li {
  place-content: center;
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 5vw;
  width: 100%;
  box-sizing: border-box;
}
li:nth-of-type(6) {
  padding: 2em 0;
}
li code {
  display: block;
  margin: 0 auto;
  text-align: left;
  background-color: #fcfcfc;
  width: calc(47.5vw - 40px);
}
output {
  font-family: monospace;
}
input {
  accent-color: olive;
}
@layer no-support {
  @supports not (rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the rule-inset-cap property";
```

```
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const ul = document.querySelector("ul");
const rowSize = document.getElementById("row");
const og = document.getElementById("og");
const colSize = document.getElementById("col");
const ow = document.getElementById("ow");
const cell = document.querySelector("li:nth-of-type(6)");
let text = "";
function update() {
  ul.style.ruleInsetCap = text = `${rowSize.value}px ${colSize.value}px`;
  cell.innerHTML = `<code>rule-inset-cap: ${text};</code>`;
}

update();

rowSize.addEventListener("input", () => {
  og.innerText = `${rowSize.value}px`;
  update();
});

colSize.addEventListener("input", () => {
  ow.innerText = `${colSize.value}px`;
  update();
});
```

Result

Change the inset values for the starts and ends of the cap segment endpoints.

Inner cap segments

This example shows how to set `rule-inset-cap` to inset the endpoints of cap segments on a grid container, and how the `rule-visibility-items` property can turn inner endpoints into cap segment endpoints.

HTML

We include an unordered list (`<ul>`) as our container for several list items (`<li>`).

We also include a `<select>` element with an `<option>` for each `rule-visibility-items` keyword, and an `<input>` of type range.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
</ul>

<p>
  <label>
    <code>rule-visibility-items</code> value
    <select id="visibility">
      <option>all</option>
      <option>between</option>
      <option>around</option>
      <option selected>normal</option>
    </select>
  </label>
</p>

<p>
  <label
    >Change the inset size.
    <input type="range" min="-40" max="16" value="0" id="inset" data-unit="px"
  /></label>
  <output id="o"></output>
</p>
```

CSS

We create a grid container by setting the `display` to grid, creating 6 columns with `grid-template-columns`, and adding a `20px` `gap`. We use the `rule` property to define our rules, overriding the row rule color with the `row-rule-color` property. We set the `rule-break` property to break the rules at each intersection, making each rule segment distinct. We explicitly set `rule-visibility-items` to the default value of `normal`. We then use the `rule-inset-cap` property to inset all the cap segment endpoints by `16px`.

We also set the seventh list item to span two columns using the `grid-column-end` property.

```
ul {
  display: grid;
  grid-template-columns: repeat(6, auto);
  gap: 20px;
  rule: 10px solid olive;
  row-rule-color: palegoldenrod;
  rule-break: intersection;
```

```
  rule-visibility-items: normal;

  rule-inset-cap: 16px;
}

li:nth-of-type(7) {
  grid-column-end: span 2;
}
```

We've hidden the other CSS styles, and the code that make the form interactive, for brevity.

```
ul {
  place-items: center;
  padding: 0;
  list-style-type: none;
  border: 1px solid;
  margin: 40px;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 1em;
  width: 100%;
  box-sizing: border-box;
}
li:nth-of-type(n + 9) {
  grid-row: 3 / 4;
}
li:nth-of-type(n + 12) {
  grid-row: 4 / 5;
}
li:nth-of-type(10) {
  grid-column: 5/6;
}
li:nth-of-type(11) {
  grid-column: 6/7;
}

li:nth-of-type(15) {
  grid-area: 4 / 5 / 5 / 6;
}
li:nth-of-type(16) {
  grid-area: 4 / 6 / 5 / 7;
}
li:nth-of-type(17) {
  display: none;
  grid-area: 5 / 6 / 6 / 7;
}
@layer no-support {
  @supports not (rule-inset-cap: 16px) {
    body::before {
      content: "Your browser doesn't support the rule-inset-cap property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const visibility = document.getElementById("visibility");
const ul = document.getElementById("ul");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  output.innerText =
    ul.style.ruleInsetCap = `${inset.value}${inset.dataset["unit"]}`;
});

visibility.addEventListener("change", () => {
  ul.style.ruleVisibilityItems = `${visibility.value}`;
});
```

Result

Select `between` from the dropdown to paint only rule segments when both adjacent grid areas contain a grid item, thereby creating interior cap segments. Now change the inset value to visualize which segment endpoints are cap segment endpoints.

See also

- `column-rule-inset-cap` shorthand
- `row-rule-inset-cap` shorthand
- `column-rule-inset` shorthand
- `row-rule-inset` shorthand
- `rule-inset-start` shorthand
- `rule-inset` shorthand
- `rule-break` shorthand\
- `rule-overlap`
- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

rule-inset-junction CSS property

> **Experimental: This is an experimental technology**

> Check the Browser compatibility table carefully before using this in production.

The **`rule-inset-junction`** CSS shorthand property can be used to offset the column and row rule segment junction endpoints to the same value.

Try it

```
rule-inset-junction: 0;
```

```
rule-inset-junction: 10px;
```

```
rule-inset-junction: 0.5em -0.5em;
```

```
rule-inset-junction: overlap-join;
```

```
rule-inset-junction: overlap-join 10px;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  rule: solid thick lightpink;
  column-rule-color: magenta;
  rule-break: intersection;
  gap: 1.5em;
  rule-overlap: column-over-row;
  border: 1px solid rebeccapurple;
  margin: auto;
  rule-visibility-items: around;
}
#example-element i {
  background-color: #efefef;
  padding: 1em;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-inset-junction`
- `row-rule-inset-junction`

Syntax

```
/* Keywords */
rule-inset-junction: overlap-join;

/* <length-percentage> values */
rule-inset-junction: 0;
rule-inset-junction: 1em;
rule-inset-junction: -5px;
rule-inset-junction: -25%;

/* Two values */
rule-inset-junction: 0 1em;
rule-inset-junction: -5px -25%;
rule-inset-junction: overlap-join 10px;

/* Global values */
rule-inset-junction: inherit;
rule-inset-junction: initial;
rule-inset-junction: revert;
rule-inset-junction: revert-layer;
rule-inset-junction: unset;
```

Values

This property is specified as one or two values from the following list:

`overlap-join`

Specifies that the junction segment should extend across the row-rule, resolving to half the `row-gap` value plus half the used `row-rule-width` value.

`<length-percentage>`

Specifies the size of the inset. Percentage values are relative to the junction endpoint, which is the `row-gap` value for column segments and the `column-gap` value for row segments.

Description

The `rule-inset-junction` shorthand property can be used to set the `row-rule-inset-junction` and `column-rule-inset-junction` properties to the same value in a single declaration, insetting row and column junction segment endpoints by the specified values.

If one value is specified, both properties are set to that value. If two values are specified, `-start` is set to the first value and `-end` is set to the second. Positive values reduce (or inset) the segment size, while negative values and the `overlap-join` keyword increase (or outset) it. The default value is `0`.

To inset both cap and junction segment endpoints, the `rule-inset-junction` shorthand property, along with the `rule-inset-cap` shorthand property, can be set using the `rule-inset` shorthand.

Formal definition

Value not found in DB!

Formal syntax

```
rule-inset-junction =
  <'column-rule-inset-junction'>

<column-rule-inset-junction> =
  <inset-value> <inset-value>?

<inset-value> =
  <length-percentage>  |
  overlap-join

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates setting `rule-inset-junction` to inset the edges of junction segments on flex containers.

HTML

The markup includes two `<div>` elements, each containing seven children. The only difference between the two containers is that the second one has an added `column` class.

```
<h1>Insetting junction column rule endpoints</h1>
<article>
  <section>
    <h2>flex-direction: row</h2>
    <div class="flexbox">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
  <section>
    <h2>flex-direction: column</h2>
    <div class="flexbox column">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </section>
</article>
```

```
<p>
  <label
    >Change the size of the inset.
    <input type="range" min="-40" max="16" value="16" id="inset"
  /></label>
  <output id="o">16px</output>
</p>
```

CSS

We use the `display` property to turn the `.flexbox` elements into flex containers. We balance the items into three flex lines using `flex-wrap` and `flex-line-count`. We define a `rule` and set the `rule-inset-junction` to `16px`.

```
.flexbox {
  display: flex;
  flex-wrap: balance;
  flex-line-count: 3;
  gap: 20px;
  rule: 5px solid blue;

  rule-inset-junction: 16px;
}
```

We also set the `flex-direction` on the `.column` container to `column`, changing the main axis of the flex container to run up and down the page and making the items flow in columns rather than rows.

```
.column {
  flex-direction: column;
}
```

The rest of the CSS is hidden for brevity.

```
h1,
article {
  font-family: sans-serif;
  text-align: center;
```

```
}
h1 {
  font-size: 1.25em;
}
h2 {
  font-size: 1em;
}
article {
  display: flex;
  gap: 5vw;
  rule: 1px solid black;
  width: 100vw;
}
section {
  flex-basis: 45vw;
}
.flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  height: 30px;
}
output {
  display: inline-block;
  width: 2em;
}
p {
  margin-top: 2.5em;
}
@layer no-support {
  @supports not (rule-inset-junction: 16px) {
    body::before {
      content: "Your browser doesn't support the rule-inset-junction property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const inset = document.getElementById("inset");
const containers = document.querySelectorAll(".flexbox");
const output = document.getElementById("o");

inset.addEventListener("input", () => {
  const val = `${inset.value}px`;
  containers[0].style.ruleInsetJunction = val;
  containers[1].style.ruleInsetJunction = val;
  output.innerText = val;
});
```

Result

Change the size of the inset. Note that where the rule is a single segment that goes from one edge of the container to the opposite edge, the segment has two cap endpoints and no junction endpoints. Therefore, changing the `rule-inset-junction` value does not affect these segments.

With grid layout

This example demonstrates using the `rule-inset-junction` property to inset junction segment endpoints to two different values on a grid container.

HTML

We include the `<ul>` element as a container with several `<li>` children which will each be converted into a grid item.

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
</ul>
```

```
<p>
  <label
    >Change the <code>-start</code> value.
    <input type="range" min="-40" max="40" value="16" id="start" data-unit="px"
  /></label>
  <output id="og">16px</output>
</p>
<p>
  <label
    >Change the <code>-end</code> value.
    <input type="range" min="-40" max="40" value="0" id="end" data-unit="px"
  /></label>
  <output id="ow">0px</output>
</p>
```

CSS

We turn the `<ul>` into a grid container by setting the `display` property to `grid`. The `grid-template-columns` property specifies that the grid has five columns. We remove the bullets with the `list-style-type` property and set the row and column gaps to `20px` with the `gap` shorthand. We set the color, size, and line style of all the rules using the `rule` shorthand, then change just the row rule color with the `row-rule-color` property.

We break the rules at every intersection using the `rule-break` property. If the rules didn't break, there would be no junction segments to style!

Finally, we set the start of each column junction to be inset by `16px` and the end to not be inset using the `rule-inset-junction` property.

We also set the 6th grid item to span three columns.

```
ul {
  display: grid;
  grid-template-columns: repeat(5, auto);
  list-style-type: "";
  gap: 20px;
  rule: 10px solid olive;
  row-rule-color: palegoldenrod;
  rule-break: intersection;

  rule-inset-junction: 16px 0;
}

li:nth-of-type(7) {
  grid-column-end: span 3;
}
```

The rest of the CSS is hidden for brevity.

```
ul {
  border: 1px solid;
  place-items: center;
  width: 95vw;
  padding: 0;
}
li {
  text-align: center;
  font-family: sans-serif;
  background-color: #ededed;
  padding: 2em 1em;
  width: 100%;
  box-sizing: border-box;
}
li code {
  display: block;
  margin: 0 auto;
  text-align: left;
  width: 30vw;
}
output {
  font-family: monospace;
}
input {
  accent-color: olive;
}
@layer no-support {
  @supports not (rule-inset-junction: 16px) {
    body::before {
      content: "Your browser doesn't support the rule-inset-junction property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

```
const ul = document.querySelector("ul");
const startSize = document.getElementById("start");
const og = document.getElementById("og");
const endSize = document.getElementById("end");
const ow = document.getElementById("ow");
const cell = document.querySelector("li:nth-of-type(7)");
let text = "";
function update() {
  ul.style.ruleInsetJunction = text = `${startSize.value}px ${endSize.value}px`;
  cell.innerHTML = `<code>rule-inset-cap: ${text};</code>`;
}

update();

startSize.addEventListener("input", () => {
  og.innerText = `${startSize.value}px`;
  update();
});

endSize.addEventListener("input", () => {
  ow.innerText = `${endSize.value}px`;
  update();
});
```

Result

See also

- `rule-inset-cap` shorthand
- `rule-inset` shorthand
- `rule-break` shorthand
- `rule-overlap`

- `rule-visibility-items`
- `rule` shorthand
- CSS gaps module

rule-overlap CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-overlap`** CSS property specifies which rule is painted on top when column rules and row rules intersect.

Try it

```
rule-overlap: row-over-column;
```

```
rule-overlap: column-over-row;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  row-rule: solid thick orange;
  column-rule: solid thick purple;
}
#example-element i {
  padding: 5px;
}
```

Syntax

```
/* Keywords */
rule-overlap: row-over-column;
rule-overlap: column-over-row;

/* Global values */
rule-overlap: inherit;
rule-overlap: initial;
rule-overlap: revert;
rule-overlap: revert-layer;
rule-overlap: unset;
```

Values

This property is specified as a single keyword from the following list:

`row-over-column`

Specifies that row rules are painted over column rules. This is the default value.

`column-over-row`

Specifies that column rules are painted over row rules.

Formal definition

Value not found in DB!

Formal syntax

```
rule-overlap =
  row-over-column  |
  column-over-row
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we use the `rule-overlap` property to paint column rules on top of row rules.

HTML

We create a list of 75 items. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>74</li>
  <li>75</li>
</ul>
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
  <li>51</li>
  <li>52</li>
  <li>53</li>
  <li>54</li>
  <li>55</li>
  <li>56</li>
  <li>57</li>
  <li>58</li>
  <li>59</li>
  <li>60</li>
  <li>61</li>
  <li>62</li>
  <li>63</li>
  <li>64</li>
  <li>65</li>
  <li>66</li>
  <li>67</li>
  <li>68</li>
  <li>69</li>
  <li>70</li>
  <li>71</li>
  <li>72</li>
  <li>73</li>
  <li>74</li>
  <li>75</li>
</ul>
<p>
  <label
    ><input type="checkbox" /> set to
    <code>rule-overlap: row-over-column;</code></label
  >
</p>
```

CSS

We define the unordered list as a 10-column grid container using the `grid-template-columns` property. We set `list-style-type` to an empty string to remove the bullets. We include a `gap` of `20px` to provide enough room between the columns and rows to fit our `20px` solid column and row rules. Finally, we use the `rule-overlap` property to draw the column rules on top of the row rules.

```
ul {
  display: grid;
  grid-template-columns: repeat(10, 1fr);
  list-style-type: "";
  gap: 20px;

  row-rule: 20px solid palegoldenrod;
  column-rule: 20px solid olive;

  rule-overlap: column-over-row;
}
```

The rest of the CSS has been hidden for brevity.

```
:has(:checked) ul {
  rule-overlap: row-over-column;
}
li {
  text-align: center;
  aspect-ratio: 1;
  line-height: 1.5em;
}
@layer no-support {
  @supports not (rule-overlap: row-over-column) {
    body::before {
      content: "Your browser doesn't support the rule-overlap property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Select the checkbox to toggle the value of the `rule-overlap` property.

See also

- `rule-color`
- `rule-width`
- `rule-style`
- `column-rule` shorthand
- `row-rule` shorthand
- `rule-visibility-items`
- CSS gaps module

rule-style CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-style`** CSS shorthand property defines the line style of the lines drawn between columns and rows in multi-column grid, flex, and multi-col layouts, setting the styles of the column and row rules to the same value.

Try it

```
rule-style: solid;
```

```
rule-style: dashed, dotted;
```

```
rule-style: repeat(2, inset, dashed, double);
```

```
rule-style: solid, repeat(auto, double), solid;
```

```
rule-style: hidden;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
```

```
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid rebeccapurple 7px;
  gap: 7px;
}
#example-element i {
  padding: 5px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-style`
- `row-rule-style`

Syntax

```
/* One value */
rule-style: none;
rule-style: hidden;
rule-style: dotted;
rule-style: dashed;
rule-style: solid;
rule-style: double;
rule-style: groove;
rule-style: ridge;
rule-style: inset;
rule-style: outset;

/* Multiple values */
rule-style: groove, double, dashed;
rule-style: solid, repeat(5, ridge), solid;
rule-style: dotted, repeat(auto, inset, outset), dotted;

/* Global values */
rule-style: inherit;
rule-style: initial;
rule-style: revert;
rule-style: revert-layer;
rule-style: unset;
```

Values

The `rule-style` property accepts a comma-separated list of values, including:

`<line-style>`

> A `<line-style>`: one of `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`. The default value is `none`.

`<repeat-line-style>`

> A `repeat()` function, with the first argument being an `<integer>` of `1` or more, and subsequent arguments being `<line-style>` values. The integer specifies how many times the `<line-style>` values should be repeated.

`<auto-repeat-line-style>`

> A `repeat()` function, with `auto` as the first argument and one or more `<line-style>` values as subsequent arguments. The provided `<line-style>` values are repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

Description

The `rule-style` property defines the line style of any column and row rule lines drawn in the gaps between columns and rows in multi-column, flex, and grid containers with more than one column or row.

The `rule-style` sets both the `column-rule-style` and `row-rule-style` properties to the same value. The `rule-style` property, along with the `rule-color` and `rule-width` properties, can also be set using the `rule` shorthand.

The value is a comma-separated list of components, which can include `<line-style>`, `<repeat-line-style>`, and `<auto-repeat-line-style>` types.

If the property value has only one `<line-style>`, all the column and row rules will be that style. If we declare the following, all column and row rules will be `double`:

```
rule-style: double;
```

When multiple `<line-style>` values are declared, they will be applied to rules in the order specified. If there are more rules than `<line-style>` values, the list of line styles is repeated until every column and row rule has a style. For example, if we declare the following, every odd rule will be `double`, and every even rule will be `inset`.

```
rule-style: double, inset;
```

Repeated line styles

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<line-style>` values passed as subsequent arguments the specified number of times. This allows the same style to be repeated a set number of times without repeating the same value. You can include `<line-style>` keyword values or custom properties that resolve to a valid `<line-style>`. Using `repeat()` can make values easier to write, enabling recurring patterns to be written using a single function, regardless of the number of columns or rows. The following declarations are equivalent:

```
rule-style: solid, outset, inset, outset, inset, outset, inset;
rule-style: solid, repeat(3, outset, inset);
```

This creates a list of seven styles. If the number of styles in the `rule-style` value's style list exceeds the number of gaps between columns or rows, the excess style values are ignored. If the container has three columns or rows, the rule in the first gutter will be `solid` and the second `outset`.

If there are more gutters than styles, the list of styles is repeated. If the container has 8, 15, 22, or 29 columns or rows, this sequence of styles will be repeated one, two, three, or four times, respectively, with the last rule being `inset`.

Auto-repeating line styles

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the `<line-style>` values passed as subsequent parameters will be repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

The `auto` keyword within the `repeat()` function creates an auto repeater that fills in values for column and row rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. Only one `repeat(auto, <line-style>)` is allowed within a `rule-style` value.

```
rule-style: solid, repeat(auto, dotted), solid;
```

In this case, it doesn't matter if the container has 8, 15, 22, or 29 columns or rows; the first and last rules will always be `solid`, and all the other rules will be `dotted`. If there are only 2 or 3 columns and rows, there will be no dotted rules.

Formal definition

Value not found in DB!

Formal syntax

```
rule-style =
  <'column-rule-style'>

<column-rule-style> =
  <line-style-list>       |
  <auto-line-style-list>

<line-style-list> =
  <line-style-or-repeat>#

<auto-line-style-list> =
  <line-style-or-repeat>#? , <auto-repeat-line-style> , <line-style-or-repeat>#?

<line-style-or-repeat> =
  <line-style>         |
  <repeat-line-style>

<auto-repeat-line-style> =
  repeat( auto , [ <line-style> ]# )

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<repeat-line-style> =
  repeat( [ <integer [1,∞]> ] , [ <line-style> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we define a single `<line-style>` for the lines drawn between the columns and rows of items in a grid container.

HTML

We create a list of 75 items. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>74</li>
  <li>75</li>
</ul>
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
```

```
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
  <li>51</li>
  <li>52</li>
  <li>53</li>
  <li>54</li>
  <li>55</li>
  <li>56</li>
  <li>57</li>
  <li>58</li>
  <li>59</li>
  <li>60</li>
  <li>61</li>
  <li>62</li>
  <li>63</li>
  <li>64</li>
  <li>65</li>
  <li>66</li>
  <li>67</li>
  <li>68</li>
  <li>69</li>
  <li>70</li>
  <li>71</li>
  <li>72</li>
  <li>73</li>
  <li>74</li>
  <li>75</li>
</ul>
```

CSS

We define the unordered list as a 10-column container, creating columns and rows with the `grid-template-columns` property; we then set `list-style-type` to `none` to remove the bullets. We include a `gap` of `5px` to provide enough room between the columns and rows to fit our `thick dashed orange` rule.

```
ul {
  display: grid;
  grid-template-columns: repeat(10, 1fr);
  list-style-type: none;
  gap: 5px;
  rule-width: thick;
  rule-color: orange;

  rule-style: dashed;
}
li {
  text-align: center;
  aspect-ratio: 1;
}
```

```
@layer no-support {
  @supports not (rule-style: solid) {
    body::before {
      content: "Your browser doesn't support the rule-style property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Multiple values

This example demonstrates using multiple `<line-style>` values as a property value, and what happens when more `<line-style>` values are specified than there are gutters to style.

We set the `rule-style` property to be a comma-separated list of all the possible `<line-style>` values.

```
ul {
  rule-style:
```

```
    dotted, dashed, solid, double, groove, ridge, inset, outset, none, hidden;
}
```

Result

There are more values than gutters for both the rows and columns; the last values are not used in each case.

Repeating values

This example demonstrates how, when there are fewer values in the list of styles than column and row rules, the values are repeated.

CSS

Using the same HTML and CSS as in the previous example, we include three comma-separated styles as the `rule-style` value:

```
ul {
  rule-style: solid, groove, double;
}
```

Result

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `rule-style` property value.

CSS

We use the same HTML and CSS as in the previous examples. We include a `repeat()` function that sets a list of two `<line-style>` values to be repeated 3 times.

```
ul {
  rule-style: solid, repeat(3, inset, outset), solid;
}
```

Result

The `repeat()` function repeats two style values three times, creating a list of eight style values. The styles are repeated for the columns; however, the last values in the list are discarded for the rows.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto` instead of an integer within the `repeat()` function.

CSS

Using `repeat(auto, <line-style>)`, we set all column and row rules to `groove`, except the first and last, which we set to `solid`.

```
ul {
  rule-style: solid, repeat(auto, groove), solid;
}
```

Result

Even though there are more column rule lines than row rule lines, the `<auto-repeat-line-color>` enables the creation of this symmetric effect.

```
@layer no-support {
  @supports not (rule-style: solid, groove) {
    body::before {
      content: "Your browser doesn't support multiple values for the rule-style property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `rule-color`
- `rule-width`
- `column-rule-style`
- `row-rule-style`
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

rule-visibility-items CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-visibility-items`** CSS shorthand property defines whether rule segments are painted in both row and column gaps adjacent to empty areas.

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-visibility-items`
- `row-rule-visibility-items`

Try it

```
rule-visibility-items: all;
```

```
rule-visibility-items: around;
```

```
rule-visibility-items: between;
```

```
rule-visibility-items: normal;
```

```
<section id="default-example">
  <section id="example-element">
    <p>One fish</p>
    <p>Two fish</p>
    <p>Red fish</p>
    <p>Blue fish</p>
    <cite>-- Dr. Seuss</cite>
  </section>
</section>
```

```
#example-element {
  display: grid;
  rule: solid 5px red;
  gap: 10px;
  grid-template-rows: repeat(3, 1fr);
  grid-template-columns: repeat(3, 1fr);
}
cite {
  grid-row: 3;
  grid-column: 3;
}
```

Syntax

```
/* Keyword values */
rule-visibility-items: all;
rule-visibility-items: around;
rule-visibility-items: between;
rule-visibility-items: normal;

/* Global values */
rule-visibility-items: inherit;
rule-visibility-items: initial;
rule-visibility-items: revert;
rule-visibility-items: revert-layer;
rule-visibility-items: unset;
```

Values

This property is specified as one of the following keyword values:

`all`

Rules should be painted in all gap segments, regardless of whether adjacent areas contain an item.

`around`

A rule should be painted in a gap segment if at least one of the two adjacent areas is occupied by an item.

`between`

A rule should be painted in a gap segment only if both adjacent areas are occupied by items.

`normal`

With grid containers, behaves the same as `all`. In multicol layout, behaves the same as `between`. This is the default.

Description

The `rule-visibility-items` property defines whether rule segments are painted in gaps adjacent to empty areas in the gaps between rows and columns in multi-row and grid containers with more than one row or column.

The value is a single keyword that sets the same value for both the `column-rule-visibility-items` and `row-rule-visibility-items` properties.

Formal definition

Value not found in DB!

Formal syntax

```
rule-visibility-items =
  <'column-rule-visibility-items'>

<column-rule-visibility-items> =
  all     |
  around  |
  between |
  normal
```

This syntax reflects the latest standard as per CSS Gap Decorations Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a rule to be drawn between two grid areas if at least one adjacent grid area contains a grid item.

HTML

We include a list of dynamic sports duos:

```
<ol>
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
```

```
  <li>Kerri Walsh + Misty May Treanor</li>
</ol>
```

CSS

We define the ordered list (`<ol>`) to be a grid container, creating 4 columns and 4 rows by setting both the `grid-template-columns` and `grid-template-rows` properties to `repeat(4, 1fr)`, and move the last item to the bottom-right grid area using the `grid-column` and `grid-row` properties. We include a `gap` of `20px` to provide enough room between the columns to fit our `5px` rules. We set the column rules to `dashed` and the row rules to `solid`.

Finally, we set `rule-visibility-items` to `between`, so that row- and column-rules are painted only if both grid areas adjacent to them contain a grid item.

```
ol {
  display: grid;
  grid-template-rows: repeat(4, 1fr);
  grid-template-columns: repeat(4, 1fr);
  gap: 20px;

  column-rule: dashed 5px blue;
  row-rule: solid 5px red;

  rule-visibility-items: around;
}
li:last-child {
  grid-row: 4;
  grid-column: 4;
}
```

```
li {
  margin-left: 1em;
}
@layer no-support {
  @supports not (rule-visibility-items: around) {
    body::before {
      content: "Your browser doesn't support the rule-visibility-items shorthand";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

See also

- `column-rule-visibility-items` shorthand
- `row-rule-visibility-items`
- `rule` shorthand
- CSS gaps module

rule-width CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`rule-width`** CSS shorthand property defines the widths of any lines drawn in the gutters of multi-row grid, flex, and multi-col layouts, setting the widths of the column and row rules to the same value.

Try it

```
rule-width: thin;
```

```
rule-width: thin, thick;
```

```
rule-width: 1px, 10px;
```

```
rule-width: repeat(2, thin, thick), 10px;
```

```
rule-width: thick, repeat(auto, 1px, 2px), thick;
```

```
<section id="default-example">
  <div id="example-element">
    <i>A</i>
    <i>B</i>
    <i>C</i>
    <i>D</i>
    <i>E</i>
    <i>F</i>
    <i>G</i>
    <i>H</i>
    <i>I</i>
    <i>J</i>
    <i>K</i>
    <i>L</i>
    <i>M</i>
    <i>N</i>
    <i>O</i>
    <i>P</i>
    <i>Q</i>
    <i>R</i>
    <i>S</i>
    <i>T</i>
    <i>U</i>
    <i>V</i>
    <i>W</i>
    <i>X</i>
```

```
    <i>Y</i>
    <i>Z</i>
  </div>
</section>
```

```
#example-element {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  rule: solid magenta;
}
#example-element i {
  padding: 5px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `column-rule-width`
- `row-rule-width`

Syntax

```
/* Keyword values */
rule-width: thin;
rule-width: medium;
rule-width: thick;
rule-width: thin, medium, thick;
rule-width: thick, repeat(5, thin), thick;
rule-width: thick, repeat(auto, thin, medium), thick;

/* Length values */
rule-width: 1px;
rule-width: 5px;
rule-width: 1px, 3px, 5px;
rule-width: 5px, repeat(auto, 1px), 10px, 15px;
rule-width: 5px, repeat(5, 1px, 3px), 5px;

/* Global values */
rule-width: inherit;
rule-width: initial;
rule-width: revert;
rule-width: revert-layer;
rule-width: unset;
```

Values

The `rule-width` property accepts a comma-separated list of values, including:

`<line-width>`

A `<line-width>`: This can be one of the keywords `thin`, `medium`, or `thick`, or a positive `<length>` value, representing the width of the line. The default value is `medium`.

`<repeat-line-width>`

A `repeat()` function, with the first argument being an `<integer>` of `1` or more, and one or more `<line-width>` values as subsequent arguments. The integer defines the number of times the `<line-width>` values should be repeated.

`<auto-repeat-line-width>`

A `repeat()` function, with `auto` as the first argument and one or more `<line-width>` values as subsequent arguments. The provided `<line-width>` values are repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

Description

The `rule-width` shorthand property defines the widths of rule lines drawn in the gaps between columns and rows in multi-column, flex, and grid containers with more than one row or column.

The value is a comma-separated list of components, which can include `<line-width>`, `<repeat-line-width>`, and `<auto-repeat-line-width>` types.

The `rule-width` property, along with the `rule-color` and `rule-style` properties, can be set using the `rule` shorthand.

If the property value consists of only one `<line-width>`, all the row and column rules will be that width. If we declare the following, all rules will be `3px`:

```
rule-width: 3px;
```

When multiple `<line-width>` values are declared, they will be applied to rules in the order specified. If there are more rules than `<line-width>` values, the list of line widths is repeated until every rule has a width. If we declare the following, for example, every odd horizontal and vertical rule will be `thin`, and every even rule will be `1em`.

```
rule-width: thin, 1em;
```

Repeated line widths

The `repeat()` function, with an integer of `1` or greater as the first argument, can be used to repeat a valid list of CSS `<line-width>` values passed as subsequent arguments the specified number of times. This allows the same widths to be repeated a set number of times without repeating the values. The following declarations are equivalent:

```
rule-width: 1rem, thick, thin, thick, thin, thick, thin;
rule-width: 1rem, repeat(3, thick, thin);
```

You can use any `<line-width>` values, including custom properties that resolve to a `<line-width>`. Using `repeat()` can make values easier to write, especially when using complex length calculations. It enables a recurring pattern to be written using a single function, regardless of the number of columns or rows.

Auto-repeating line widths

The `repeat()` function also accepts `auto` as the first argument instead of a positive integer. With `auto` as the first argument, the list of `<line-width>` values passed as subsequent arguments will be repeated as many times as needed to fill in values for any rules that are not explicitly specified by other components of the property value.

```
rule-width: thin, repeat(auto, medium), thin;
```

In this case, the first and last column and row rules will always be `thin`, and all other rules will be `medium`. If there are only 2 or 3 columns and rows, there will be no medium-sized rules.

The `auto` keyword within the `repeat()` function creates an auto-repeater that fills in values for column and row rules that would not otherwise receive values from other parts of the list, preventing the list from being cycled. At most, only one `repeat(auto, <width>)` can be present in a `rule-width` value.

Formal definition

Value not found in DB!

Formal syntax

```
rule-width =
  <'column-rule-width'>

<column-rule-width> =
  <line-width-list>       |
  <auto-line-width-list>

<line-width-list> =
  <line-width-or-repeat>#

<auto-line-width-list> =
  <line-width-or-repeat>#? , <auto-repeat-line-width> , <line-width-or-repeat>#?

<line-width-or-repeat> =
  <line-width>          |
  <repeat-line-width>

<auto-repeat-line-width> =
  repeat( auto , [ <line-width> ]# )

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<repeat-line-width> =
  repeat( [ <integer [1,∞]> ] , [ <line-width> ]# )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, we define a single width for the lines drawn between the columns and rows of items in a grid container.

HTML

We create a list of 75 items. Most of the HTML is hidden for brevity.

```
<ul>
  <li>1</li>
  <li>2</li>
  ...
  <li>74</li>
  <li>75</li>
</ul>
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
  <li>11</li>
  <li>12</li>
  <li>13</li>
  <li>14</li>
  <li>15</li>
  <li>16</li>
  <li>17</li>
  <li>18</li>
  <li>19</li>
  <li>20</li>
  <li>21</li>
  <li>22</li>
  <li>23</li>
  <li>24</li>
  <li>25</li>
  <li>26</li>
  <li>27</li>
  <li>28</li>
  <li>29</li>
  <li>30</li>
  <li>31</li>
  <li>32</li>
  <li>33</li>
  <li>34</li>
  <li>35</li>
  <li>36</li>
  <li>37</li>
  <li>38</li>
```

```
  <li>39</li>
  <li>40</li>
  <li>41</li>
  <li>42</li>
  <li>43</li>
  <li>44</li>
  <li>45</li>
  <li>46</li>
  <li>47</li>
  <li>48</li>
  <li>49</li>
  <li>50</li>
  <li>51</li>
  <li>52</li>
  <li>53</li>
  <li>54</li>
  <li>55</li>
  <li>56</li>
  <li>57</li>
  <li>58</li>
  <li>59</li>
  <li>60</li>
  <li>61</li>
  <li>62</li>
  <li>63</li>
  <li>64</li>
  <li>65</li>
  <li>66</li>
  <li>67</li>
  <li>68</li>
  <li>69</li>
  <li>70</li>
  <li>71</li>
  <li>72</li>
  <li>73</li>
  <li>74</li>
  <li>75</li>
</ul>
```

CSS

We define the unordered list to be a 10-column grid container. We include a gap of `5px` to provide enough room between the items to fit our `3px` dashed red rule:

```
ul {
  display: grid;
  grid-template-columns: repeat(10, 1fr);
  list-style-type: none;
  gap: 5px;
  rule-style: dashed;
  rule-color: red;
  rule-width: 3px;
}
li {
  text-align: center;
  aspect-ratio: 1;
}
```

```
@layer no-support {
  @supports not (rule-width: medium) {
    body::before {
      content: "Your browser doesn't support the rule-width property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Repeating values

This example demonstrates how, when there are fewer values in the list of widths than column or row rules, the values are repeated.

CSS

Using the same HTML and CSS as in the previous example, we include three comma-separated widths as the `rule-width`.

```
ul {
  rule-width: thin, 6px, 12px;
}
```

Result

As the grid container has 8 rows and 10 columns, there are seven and nine gutters in each direction, respectively, so the sequence of three `<line-width>` values is repeated in both directions.

Using the <code>repeat()</code> function

This example demonstrates using the `repeat()` function within the `rule-width` property value and how this function can help reduce the verbosity of value declarations.

CSS

We use the same HTML and CSS as in the previous examples. In addition, we declare two custom properties, which we use in a `repeat()` function inside our `rule-width` value. The `repeat()` function sets a list of two `<line-width>` values to repeat 3 times.

```
ul {
  --base: 0.5vw;
  --secondary: 1vw;
  rule-width:
    15px,
```

```
    repeat(
      4,
      min(calc(var(--base) + 3px), 10px),
      abs(calc(var(--secondary) - 2px))
    ),
    15px;
}
```

Result

The `repeat()` function repeats two width values four times, creating a list of ten width values. As there are fewer column and row gutters than total widths, the last values in the list are discarded.

Using <code>auto</code> within <code>repeat()</code>

This example demonstrates using `auto` instead of an integer within the `repeat()` function.

CSS

Using `repeat(auto, <line-width>)`, we set all column and row rules to `1px`, except the first and last, which we set to `5px`.

```
ul {
  rule-width: 5px, repeat(auto, 1px), 5px;
}
```

Result

```
@layer no-support {
  @supports not (rule-width: thin, thick) {
    body::before {
      content: "Your browser doesn't support the rule-width property";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

See also

- `rule-color`
- `rule-style`
- `column-rule-width`
- `row-rule-width`
- `rule` shorthand
- Defining CSS gaps
- CSS gaps module

rx CSS property

The **`rx`** CSS property defines the x-axis, or horizontal, radius of an SVG `<ellipse>` and the horizontal curve of the corners of an SVG `<rect>` rectangle. If present, it overrides the shape's `rx` attribute.

> **Note:** The `rx` property only applies to `<ellipse>` and `<rect>` elements nested in an `<svg>`. It doesn't apply to other SVG elements or HTML elements or pseudo-elements.

Syntax

```
/* Initial value */
rx: auto;

/* Length and percentage values */
rx: 20px;
rx: 20%;

/* Global values */
rx: inherit;
rx: initial;
rx: revert;
rx: revert-layer;
rx: unset;
```

Values

The `<length>`, `<percentage>`, or `auto` keyword value denotes the horizontal radius of ellipses and the horizontal border-radius of rectangles.

`<length>`

Absolute or relative lengths can be expressed in any unit allowed by the CSS `<length>` data type. Negative values are invalid.

`<percentage>`

Percentages refer to the width of the current SVG viewport. The used value for a `<rect>` is never more than 50% of the width of the rectangle.

`auto`

When set or defaulting to `auto`, the `rx` value equals the absolute length value used for `ry`. If both `rx` and `ry` have a computed value of `auto`, the used value is `0`.

Formal definition

Initial value	`auto`
Applies to	`<ellipse>` and `<rect>` elements in `<svg>`
Inherited	no
Percentages	refer to the width of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
rx =
  <length-percentage>  |
  auto

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating rounded corners

This example demonstrates creating rectangles with rounded corners by applying the `rx` property to SVG `<rect>` elements.

HTML

We include an SVG image with four `<rect>` elements; all the rectangles are the same except for their `x` attribute value, which positions them along the x-axis.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <rect width="50" height="120" y="5" x="5" />
  <rect width="50" height="120" y="5" x="60" />
  <rect width="50" height="120" y="5" x="115" />
  <rect width="50" height="120" y="5" x="170" />
  <rect width="50" height="120" y="5" x="225" />
</svg>
```

CSS

With CSS, we set an `rx` value on four of the rectangles:

```
svg {
  border: 1px solid;
}

rect:nth-of-type(2) {
  rx: 10px;
  fill: red;
}

rect:nth-of-type(3) {
  rx: 2em;
  fill: blue;
}

rect:nth-of-type(4) {
  rx: 5%;
  fill: orange;
}

rect:nth-of-type(5) {
  rx: 80%;
  fill: green;
}
```

Results

The first rectangle defaults to `auto`; as all the `ry` values in this example also default to `auto`, the used value of `rx` is `0`. The red and blue rectangles have `10px` and `2em` rounded corners, respectively. As the SVG viewport defaults to 300px by 150px, the orange rectangle's `5%` value creates a `15px` radius. The green rectangle has `rx: 80%` set. However, as the value of `rx` is never more than `50%` of the width of the rectangle, the effect is as if `rx: 50%; ry: 50%` was set.

Defining the horizontal radius of an ellipse

This basic `<ellipse>` example demonstrates the CSS `rx` property taking precedence over an SVG `rx` attribute to define the shape's horizontal radius.

HTML

We include two identical `<ellipse>` elements in an SVG; each with the `rx` attribute set to `20`.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <ellipse cx="80" cy="50" rx="20" ry="40" />
  <ellipse cx="80" cy="50" rx="20" ry="40" />
</svg>
```

CSS

We only style the first ellipse, letting its twin use default user agent styles (with `fill` defaulting to black). The geometric `rx` property overrides the value of the SVG `rx` attribute. We also set the `fill` and `stroke` properties to differentiate the styled shape from its twin.

```
svg {
  border: 1px solid;
}

ellipse:first-of-type {
  rx: 80px;
  fill: magenta;
  stroke: rebeccapurple;
}
```

Results

The styled ellipse's horizontal radius is `80px`, as defined in the CSS `rx` property value. The unstyled ellipse's horizontal radius is `20px`, which was defined by the `rx` attribute.

Ellipse horizontal radius percentage values

This example demonstrates using percentage values for `rx`.

HTML

We use the same markup as the previous example.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <ellipse cx="80" cy="50" rx="20" ry="40" />
  <ellipse cx="80" cy="50" rx="20" ry="40" />
</svg>
```

CSS

The CSS is similar to the previous example, with the only difference being the `rx` property value; in this case, we use a percentage value.

```
svg {
  border: 1px solid;
}

ellipse:first-of-type {
  rx: 30%;
  fill: magenta;
  stroke: rebeccapurple;
}
```

Results

When using percentage values for `rx`, the values are relative to the width of the SVG viewport. Here, the size of the styled ellipse horizontal radius is `30%` of the width of the current SVG viewport. As the width defaulted to `300px`, the `rx` value is `90px`.

See also

- Geometry properties: `rx`, `cx`, `cy`, `r`, `ry`, `x`, `y`, `width`, `height`
- `fill`
- `stroke`
- `paint-order`
- `border-radius` shorthand property
- `radial-gradient`
- `<basic-shape>` data type
- SVG `rx` attribute

ry CSS property

The **`ry`** CSS property defines the y-axis, or vertical, radius of an SVG `<ellipse>` and the vertical curve of the corners of an SVG `<rect>` rectangle. If present, it overrides the shape's `ry` attribute.

> **Note:** The `ry` property only applies to `<ellipse>` and `<rect>` elements nested in an `<svg>`. It doesn't apply to other SVG elements or HTML elements or pseudo-elements.

Syntax

```
/* Initial value */
ry: auto;

/* Length and percentage values */
ry: 30px;
ry: 30%;

/* Global values */
ry: inherit;
ry: initial;
ry: revert;
ry: revert-layer;
ry: unset;
```

Values

The `<length>`, `<percentage>`, or `auto` keyword value denotes the vertical radius of ellipses and the vertical border-radius of rectangles.

`<length>`

Absolute or relative lengths can be expressed in any unit allowed by the CSS `<length>` data type. Negative values are invalid.

`<percentage>`

Percentages refer to the height of the current SVG viewport. The used value for a `<rect>` is never more than 50% of the height of the rectangle.

`auto`

When set or defaulting to `auto`, the `ry` value equals the absolute length value used for `rx`. If both `ry` and `rx` have a computed value of `auto`, the used value is `0`.

Formal definition

Initial value	`auto`
Applies to	`<ellipse>` and `<rect>` elements in `<svg>`
Inherited	no
Percentages	refer to the height of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
ry =
  <length-percentage>  |
  auto

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating rounded corners

This example demonstrates creating rectangles with rounded corners by applying the `ry` property to SVG `<rect>` elements.

HTML

We include an SVG image with four `<rect>` elements; all the rectangles are the same except for their `x` attribute value, which positions them along the x-axis.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <rect width="50" height="120" y="5" x="5" />
  <rect width="50" height="120" y="5" x="60" />
  <rect width="50" height="120" y="5" x="115" />
  <rect width="50" height="120" y="5" x="170" />
  <rect width="50" height="120" y="5" x="225" />
</svg>
```

CSS

With CSS, we set an `ry` value on four of the rectangles:

```
svg {
  border: 1px solid;
}

rect:nth-of-type(2) {
  ry: 10px;
  fill: red;
}

rect:nth-of-type(3) {
  ry: 2em;
  fill: blue;
}

rect:nth-of-type(4) {
  ry: 5%;
  fill: orange;
}

rect:nth-of-type(5) {
  ry: 80%;
  fill: green;
}
```

Results

The first rectangle defaults to `auto`; as all the `rx` values in this example also default to `auto`, the used value of `ry` is `0`. The red and blue rectangles have `10px` and `2em` rounded corners, respectively. As the SVG viewport defaults to 300px by 150px, the orange rectangle's `5%` value creates a `7.5px` radius. The green rectangle has `ry: 80%` set. However, as the value of `ry` is never more than `50%` of the height of the rectangle, the effect is as if `ry: 50%; rx: 50%` was set.

Defining the vertical radius of an ellipse

This basic `<ellipse>` example demonstrates the CSS `ry` property taking precedence over an SVG `ry` attribute to define the shape's vertical radius.

HTML

We include two identical `<ellipse>` elements in an SVG; each with the `ry` attribute set to `20`.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <ellipse cx="80" cy="50" rx="40" ry="20" />
  <ellipse cx="80" cy="50" rx="40" ry="20" />
</svg>
```

CSS

We only style the first ellipse, letting its twin use default user agent styles (with `fill` defaulting to black). The geometric `ry` property overrides the value of the SVG `ry` attribute. We also set the `fill` and `stroke` properties to differentiate the styled shape from its twin.

```
svg {
  border: 1px solid;
}

ellipse:first-of-type {
  ry: 80px;
  fill: magenta;
  stroke: rebeccapurple;
}
```

Results

The styled ellipse's vertical radius is `80px`, as defined in the CSS `ry` property value. The unstyled ellipse's vertical radius is `20px`, which was defined by the `ry` attribute.

Ellipse vertical radius percentage values

This example demonstrates using percentage values for `ry`.

HTML

We use the same markup as the previous example.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <ellipse cx="80" cy="50" rx="40" ry="20" />
```

```
  <ellipse cx="80" cy="50" rx="40" ry="20" />
</svg>
```

CSS

The CSS is similar to the previous example, with the only difference being the `ry` property value; in this case, we use a percentage value.

```
svg {
  border: 1px solid;
}

ellipse:first-of-type {
  ry: 30%;
  fill: magenta;
  stroke: rebeccapurple;
}
```

Results

When using percentage values for `ry`, the values are relative to the height of the SVG viewport. Here, the size of the styled ellipse vertical radius is `30%` of the height of the current SVG viewport. As the height defaulted to `150px`, the `ry` value is `45px`, making the ellipse `90px` tall.

See also

- Geometry properties: `ry`, `cx`, `cy`, `r`, `rx`, `x`, `y`, `height`, `height`
- `fill`
- `stroke`
- `paint-order`
- `border-radius` shorthand property
- `radial-gradient`
- `<basic-shape>` data type
- SVG `ry` attribute

scale CSS property

The **`scale`** CSS property allows you to specify scale transforms individually and independently of the `transform` property. This maps better to typical user interface usage, and saves having to remember the exact order of transform functions to specify in the `transform` value.

Try it

```
scale: none;
```

```
scale: 1.5;
```

```
scale: 1.7 50%;
```

```
scale: 1 -1;
```

```
scale: 1.2 1.2 2;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
```

```
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Syntax

```
/* Keyword value */
scale: none;

/* Single values */
/* values of more than 1 or 100% make the element grow */
scale: 2;
/* values of less than 1 or 100% make the element shrink */
scale: 50%;

/* Two values */
scale: 2 0.5;

/* Three values */
scale: 200% 50% 200%;

/* Global values */
scale: inherit;
scale: initial;
scale: revert;
scale: revert-layer;
scale: unset;
```

Values

Single value

A `<number>` or `<percentage>` specifying a scale factor to make the affected element scale by the same factor along both the X and Y axes. Equivalent to a `scale()` (2D scaling) function with a single value specified.

Two values

Two `<number>` or `<percentage>` values that specify the X and Y axis scaling values (respectively) of a 2D scale. Equivalent to a `scale()` (2D scaling) function with two values specified.

Three values

Three `<number>` or `<percentage>` values that specify the X, Y, and Z axis scaling values (respectively) of a 3D scale. Equivalent to a `scale3d()` (3D scaling) function.

`none`

Specifies that no scaling should be applied.

Formal definition

Initial value	none
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	a transform
Creates stacking context	yes

Formal syntax

```
scale =
  none                              |
  [ <number> | <percentage> ]{1,3}
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Scaling an element on hover

The following example shows how to scale an element on hover. Two boxes are shown, one with a single `scale` value which scales the element along both axes. The second box has two `scale` values which scales the element along the X and Y axes independently.

HTML

```
<div class="box" id="box1">single value</div>
<div class="box" id="box2">two values</div>
```

CSS

```
.box {
  float: left;
  margin: 1em;
  width: 7em;
  line-height: 7em;
```

```
  text-align: center;
  transition: 0.5s ease-in-out;
  border: 3px dotted;
}

#box1:hover {
  scale: 1.25;
}

#box2:hover {
  scale: 1.25 0.75;
}
```

Result

See also

- translate
- rotate
- transform

Note: skew is not an independent transform value

scroll-behavior CSS property

The **`scroll-behavior`** CSS property sets the behavior for a scrolling box when scrolling is triggered by the navigation or CSSOM scrolling APIs.

Try it

```
scroll-behavior: auto;
```

```
scroll-behavior: smooth;
```

```
<section id="default-example">
  <div class="container">
    <p class="nav">
      Scroll to:
      <a href="#pageA">A</a>
      <a href="#pageB">B</a>
      <a href="#pageC">C</a>
    </p>
    <scroll-container id="example-element">
      <scroll-page id="pageA">A</scroll-page>
      <scroll-page id="pageB">B</scroll-page>
      <scroll-page id="pageC">C</scroll-page>
    </scroll-container>
  </div>
</section>
```

```
.container {
  flex-direction: column;
}

.nav a {
  color: #009e5f;
}

scroll-container {
  border: 1px solid black;
  display: block;
  height: 200px;
  overflow-y: scroll;
  width: 200px;
}

scroll-page {
  align-items: center;
  display: flex;
  font-size: 5em;
  height: 100%;
  justify-content: center;
}
```

Note that any other scrolls, such as those performed by the user, are not affected by this property. When this property is specified on the root element, it applies to the viewport instead. This property specified on the `body` element will *not* propagate to the viewport.

User agents are allowed to ignore this property.

Syntax

```
/* Keyword values */
scroll-behavior: auto;
scroll-behavior: smooth;

/* Global values */
scroll-behavior: inherit;
scroll-behavior: initial;
scroll-behavior: revert;
scroll-behavior: revert-layer;
scroll-behavior: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The scrolling box scrolls instantly.

`smooth`

The scrolling box scrolls in a smooth fashion using a user-agent-defined easing function over a user-agent-defined period of time. User agents should follow platform conventions, if any.

Formal definition

Initial value	`auto`
Applies to	scrolling boxes
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
scroll-behavior =
  auto    |
  smooth
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting smooth scroll behavior

HTML

```
<nav>
  <a href="#page-1">1</a>
  <a href="#page-2">2</a>
  <a href="#page-3">3</a>
</nav>
<div class="scroll-container">
  <div class="scroll-page" id="page-1">1</div>
  <div class="scroll-page" id="page-2">2</div>
  <div class="scroll-page" id="page-3">3</div>
</div>
```

CSS

```
a {
  display: inline-block;
  width: 50px;
  text-decoration: none;
}
nav,
.scroll-container {
  display: block;
  margin: 0 auto;
  text-align: center;
}
nav {
  width: 339px;
  padding: 5px;
  border: 1px solid black;
}
.scroll-container {
  width: 350px;
  height: 200px;
  overflow-y: scroll;
  scroll-behavior: smooth;
}
.scroll-page {
  display: flex;
  align-items: center;
  justify-content: center;
  height: 100%;
  font-size: 5em;
}
```

Result

scroll-initial-target CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`scroll-initial-target`** CSS property enables the definition of elements that are potential snap targets when their ancestor scroll container is first rendered.

Syntax

```
/* Keyword values */
scroll-initial-target: none;
scroll-initial-target: nearest;

/* Global values */
scroll-initial-target: inherit;
scroll-initial-target: initial;
scroll-initial-target: revert;
scroll-initial-target: revert-layer;
scroll-initial-target: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

The element is not an initial scroll target.

`nearest`

The element is potentially an initial scroll target for its nearest scroll container ancestor.

Description

The `scroll-initial-target` property enables defining elements that should be snapped when their parent scroll snap containers are first rendered. Setting the value to `nearest` defines the element as a potential target that should be snapped to when the nearest ancestor scroll container first shows up on the page.

If multiple elements or pseudo-elements in the scroll container are set to `nearest`, the first element in the tree order is the initial scroll-snapping target.

The initial value is `none`, meaning a scroll-snappable element is not by default an initial scroll target. The `none` value can also be set on an element to explicitly make it not be an initial scroll target.

If a scroll container's initial scroll position is potentially set by both the `place-content` content-distribution property and by `scroll-initial-target` on any descendants — the first descendant with `scroll-initial-target: nearest` wins.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	the specified keyword
Animation type	Not animatable

Formal syntax

```
scroll-initial-target =
  none     |
  nearest
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using <code>scroll-initial-target</code>

The example below demonstrates the two values of `scroll-initial-target`, and how the first element with `scroll-initial-target` gets snapped.

HTML

We include 5 containers, preceded by a paragraph explaining the expected effect.

```
<p><code>none</code> on #4 only</p>
<div class="none">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div class="set">4</div>
  <div>5</div>
</div>

<p><code>nearest</code> on #4 only</p>
<div class="nearest">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div class="set">4</div>
  <div>5</div>
</div>

<p><code>nearest</code> on even elements</p>
<div class="nearest">
  <div>1</div>
  <div class="set">2</div>
  <div>3</div>
  <div class="set">4</div>
  <div>5</div>
</div>

<p><code>nearest</code> on odd elements</p>
<div class="nearest">
  <div class="set">1</div>
  <div>2</div>
  <div class="set">3</div>
  <div>4</div>
  <div class="set">5</div>
</div>

<p><code>nearest</code> on odd elements, with <code>none</code> on #1</p>
<div class="nearest">
  <div class="set unset">1</div>
  <div>2</div>
  <div class="set">3</div>
  <div>4</div>
  <div class="set">5</div>
</div>
```

CSS

We set up the nearest and none elements as scroll-snap containers, centering the snapped elements.

```
/* mandatory scroll-snap on parent */
div.nearest,
div.none {
  scroll-snap-type: x mandatory;
}

/* scroll-snap alignment for children */
div > div {
```

```
  scroll-snap-align: center;
  scroll-snap-stop: always;
}
```

We then set `scroll-initial-target` of either `none` or `nearest` on all the elements with the `.set` class.

```
.none .set,
.nearest .set.unset {
  scroll-initial-target: none;
}
.nearest .set {
  scroll-initial-target: nearest;
}
```

```
/* setup */
body {
  height: 100%;
  display: flex;
  align-items: center;
  flex-flow: column nowrap;
  font-family: sans-serif;
  text-align: center;
}

div.nearest,
div.none {
  display: flex;
  overflow: auto;
  font-size: 3rem;
}

div div {
  width: 90%;
  min-width: 15rem;
  flex: none;
  outline: 1px solid #333333;
}

/* coloration */
div > div:nth-child(even) {
  background-color: #87ea87;
}

div > div:nth-child(odd) {
  background-color: #87ccea;
}

p {
  margin: 1em 0 0;
}

@supports not (scroll-initial-target: nearest) {
  :root::before {
    content: "Your browser doesn't support the scroll-initial-target property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

The property's effect is demonstrated when the scroll-snap container is drawn to the page.

Each row snaps to the first element with `nearest` set, if any, in tree order. In the last example, we've overridden the `nearest` value with `none` on the first element, so the first element with `nearest` applied is #3.

See also

- CSS scroll snap

scroll-margin CSS property

The **`scroll-margin`** CSS shorthand property sets all of the scroll margins of an element at once, assigning values much like the `margin` property does for margins of an element.

Try it

```
scroll-margin: 0;
```

```
scroll-margin: 20px;
```

```
scroll-margin: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}
```

```
.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `scroll-margin-bottom`
- `scroll-margin-left`
- `scroll-margin-right`
- `scroll-margin-top`

Syntax

```
/* <length> values */
scroll-margin: 10px;
scroll-margin: 1em 0.5em 1em 1em;

/* Global values */
scroll-margin: inherit;
scroll-margin: initial;
scroll-margin: revert;
scroll-margin: revert-layer;
scroll-margin: unset;
```

Values

`<length>`

An outset from the corresponding edge of the scroll container.

Description

You can see the effect of `scroll-margin` by scrolling to a point partway between two of the "pages" of the example's content. The value specified for `scroll-margin` determines how much of the page that's primarily outside the snapport should remain visible.

Thus, the `scroll-margin` values represent outsets defining the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-margin-bottom`: `0` • `scroll-margin-left`: `0` • `scroll-margin-right`: `0` • `scroll-margin-top`: `0`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `scroll-margin-bottom`: as specified • `scroll-margin-left`: as specified • `scroll-margin-right`: as specified • `scroll-margin-top`: as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin =
  <length>{1,4}
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

This example implements something very similar to the interactive example above, except that here we'll explain to you how it's implemented.

The aim here is to create four horizontally-scrolling blocks, the second and third of which snap into place, near but not quite at the left of each block.

HTML

The HTML includes a scroller with four children:

```
<div class="scroller">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

CSS

Let's walk through the CSS. The outer container is styled like this:

```
.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}
```

The main parts relevant to the scroll snapping are `overflow-x: scroll`, which makes sure the contents will scroll and not be hidden, and `scroll-snap-type: x mandatory`, which dictates that scroll snapping must occur along the horizontal axis, and the scrolling will always come to rest on a snap point.

The child elements are styled as follows:

```
.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(2n) {
  background-color: white;
  color: rebeccapurple;
}
```

The most relevant part here is `scroll-snap-align: start`, which specifies that the left-hand edges (the "starts" along the x axis, in our case) are the designated snap points.

Last of all we specify the scroll margin-values, a different one for the second and third child elements:

```
.scroller > div:nth-child(2) {
  scroll-margin: 1rem;
}

.scroller > div:nth-child(3) {
  scroll-margin: 2rem;
}
```

This means that when scrolling past the middle child elements, the scrolling will snap to `1rem` outside the left edge of the second `<div>`, and `2rems` outside the left edge of the third `<div>`.

> **Note:** Here we are setting `scroll-margin` on all sides at once, but only the start edge is really relevant. It would work just as well here to only set a scroll margin on that one edge, for example with `scroll-margin-inline-start: 1rem`, or `scroll-margin: 0 0 0 1rem`.

Result

Try it for yourself:

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-block CSS property

The **`scroll-margin-block`** CSS shorthand property sets the scroll margins of an element in the block dimension.

Try it

```
scroll-margin-block: 0;
```

```
scroll-margin-block: 20px;
```

```
scroll-margin-block: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `scroll-margin-block-end`
- `scroll-margin-block-start`

Syntax

```
/* <length> values */
scroll-margin-block: 10px;
scroll-margin-block: 1em 0.5em;

/* Global values */
scroll-margin-block: inherit;
scroll-margin-block: initial;
scroll-margin-block: revert;
scroll-margin-block: revert-layer;
scroll-margin-block: unset;
```

Values

`<length>`

An outset from the corresponding edge of the scroll container.

Description

The scroll-margin values represent outsets defining the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-margin-block-start`: `0` • `scroll-margin-block-end`: `0`
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `scroll-margin-block-start`: as specified • `scroll-margin-block-end`: as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-block =
  <length>{1,2}
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-block-end CSS property

The `scroll-margin-block-end` property defines the margin of the scroll snap area at the end of the block dimension that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-block-end: 0;
```

```
scroll-margin-block-end: 20px;
```

```
scroll-margin-block-end: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-block-end: 10px;
scroll-margin-block-end: 1em;

/* Global values */
scroll-margin-block-end: inherit;
scroll-margin-block-end: initial;
scroll-margin-block-end: revert;
scroll-margin-block-end: revert-layer;
scroll-margin-block-end: unset;
```

Values

`<length>`

An outset from the block end edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-block-end =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-block-start CSS property

The `scroll-margin-block-start` property defines the margin of the scroll snap area at the start of the block dimension that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-block-start: 0;
```

```
scroll-margin-block-start: 20px;
```

```
scroll-margin-block-start: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-block-start: 10px;
scroll-margin-block-start: 1em;

/* Global values */
scroll-margin-block-start: inherit;
scroll-margin-block-start: initial;
scroll-margin-block-start: revert;
scroll-margin-block-start: revert-layer;
scroll-margin-block-start: unset;
```

Values

`<length>`

An outset from the block start edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-block-start =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-bottom CSS property

The `scroll-margin-bottom` property defines the bottom margin of the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-bottom: 0;
```

```
scroll-margin-bottom: 20px;
```

```
scroll-margin-bottom: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-bottom: 10px;
scroll-margin-bottom: 1em;

/* Global values */
scroll-margin-bottom: inherit;
scroll-margin-bottom: initial;
scroll-margin-bottom: revert;
scroll-margin-bottom: revert-layer;
scroll-margin-bottom: unset;
```

Values

`<length>`

An outset from the bottom edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-bottom =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-inline CSS property

The **`scroll-margin-inline`** CSS shorthand property sets the scroll margins of an element in the inline dimension.

Try it

```
scroll-margin-inline: 0;
```

```
scroll-margin-inline: 40px 20px;
```

```
scroll-margin-inline: 4em 0;
```

```
scroll-margin-inline: 0px 3em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `scroll-margin-inline-end`
- `scroll-margin-inline-start`

Syntax

```
/* <length> values */
scroll-margin-inline: 10px;
scroll-margin-inline: 1em 0.5em;

/* Global values */
scroll-margin-inline: inherit;
scroll-margin-inline: initial;
scroll-margin-inline: revert;
scroll-margin-inline: revert-layer;
scroll-margin-inline: unset;
```

Values

`<length>`

An outset from the corresponding edge of the scroll container.

Description

The scroll-margin values represent outsets defining the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-margin-inline-start`: `0` • `scroll-margin-inline-end`: `0`
Applies to	all elements
Inherited	no

Computed value	as each of the properties of the shorthand: • `scroll-margin-inline-start`: as specified • `scroll-margin-inline-end`: as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-inline =
  <length>{1,2}
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

This example implements something very similar to the interactive example above, except that here we'll explain to you how it's implemented.

The aim here is to create four horizontally-scrolling blocks, the second and third of which snap into place, near but not quite at the right of each block.

HTML

The HTML includes a scroller with four children:

```
<div class="scroller">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

CSS

Let's walk through the CSS. The outer container is styled like this:

```
.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}
```

The main parts relevant to the scroll snapping are `overflow-x: scroll`, which makes sure the contents will scroll and not be hidden, and `scroll-snap-type: x mandatory`, which dictates that scroll snapping must occur along the horizontal axis, and the scrolling will always come to rest on a snap point.

The child elements are styled as follows:

```
.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(2n) {
  background-color: white;
  color: rebeccapurple;
}
```

The most relevant part here is `scroll-snap-align: end`, which specifies that the right-hand edges (the "ends" along the x axis, in our case) are the designated snap points.

Last of all we specify the scroll margin values, a different one for the second and third child elements:

```
.scroller > div:nth-child(2) {
  scroll-margin-inline: 1rem;
}

.scroller > div:nth-child(3) {
  scroll-margin-inline: 2rem;
}
```

This means that when scrolling past the middle child elements, the scrolling will snap to `1rem` outside the inline end edge of the second `<div>`, and `2rems` outside the inline end edge of the third `<div>`.

> **Note:** Here we are setting `scroll-margin` on the start *and* end of the inline axis (x in our case), but only the end edge is really relevant. It would work just as well here to only set a scroll margin on that one edge, for example with `scroll-margin-inline: 0 1rem`, or `scroll-margin-inline-end: 1rem`.

Result

Try it for yourself:

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-inline-end CSS property

The `scroll-margin-inline-end` property defines the margin of the scroll snap area at the end of the inline dimension that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-inline-end: 0;
```

```
scroll-margin-inline-end: 20px;
```

```
scroll-margin-inline-end: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-inline-end: 10px;
scroll-margin-inline-end: 1em;

/* Global values */
scroll-margin-inline-end: inherit;
scroll-margin-inline-end: initial;
scroll-margin-inline-end: revert;
scroll-margin-inline-end: revert-layer;
scroll-margin-inline-end: unset;
```

Values

`<length>`

An outset from the inline end edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-inline-end =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

This example implements something very similar to the interactive example above, except that here we'll explain to you how it's implemented.

The aim here is to create four horizontally-scrolling blocks, the second and third of which snap into place, near but not quite at the right of each block.

HTML

The HTML includes a scroller with four children:

```
<div class="scroller">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

CSS

Let's walk through the CSS. The outer container is styled like this:

```
.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}
```

The main parts relevant to the scroll snapping are `overflow-x: scroll`, which makes sure the contents will scroll and not be hidden, and `scroll-snap-type: x mandatory`, which dictates that scroll snapping must occur along the horizontal axis, and the scrolling will always come to rest on a snap point.

The child elements are styled as follows:

```
.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(2n) {
  background-color: white;
  color: rebeccapurple;
}
```

The most relevant part here is `scroll-snap-align: end`, which specifies that the right-hand edges (the "ends" along the x axis, in our case) are the designated snap points.

Last of all we specify the scroll margin values, a different one for the second and third child elements:

```
.scroller > div:nth-child(2) {
  scroll-margin-inline-end: 1rem;
}

.scroller > div:nth-child(3) {
  scroll-margin-inline-end: 2rem;
}
```

This means that when scrolling past the middle child elements, the scrolling will snap to `1rem` outside the inline end edge of the second `<div>`, and `2rem`s outside the inline end edge of the third `<div>`.

Result

Try it for yourself:

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-inline-start CSS property

The `scroll-margin-inline-start` property defines the margin of the scroll snap area at the start of the inline dimension that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-inline-start: 0;
```

```
scroll-margin-inline-start: 20px;
```

```
scroll-margin-inline-start: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
```

```
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-inline-start: 10px;
scroll-margin-inline-start: 1em;

/* Global values */
scroll-margin-inline-start: inherit;
scroll-margin-inline-start: initial;
scroll-margin-inline-start: revert;
scroll-margin-inline-start: revert-layer;
scroll-margin-inline-start: unset;
```

Values

`<length>`

An outset from the inline start edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-inline-start =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic demonstration

This example implements something very similar to the interactive example above, except that here we'll explain to you how it's implemented.

The aim here is to create four horizontally-scrolling blocks, the second and third of which snap into place, near but not quite at the left of each block.

HTML

The HTML includes a scroller with four children:

```
<div class="scroller">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

CSS

Let's walk through the CSS. The outer container is styled like this:

```
.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}
```

The main parts relevant to the scroll snapping are `overflow-x: scroll`, which makes sure the contents will scroll and not be hidden, and `scroll-snap-type: x mandatory`, which dictates that scroll snapping must occur along the horizontal axis, and the scrolling will always come to rest on a snap point.

The child elements are styled as follows:

```
.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(2n) {
  background-color: white;
  color: rebeccapurple;
}
```

The most relevant part here is `scroll-snap-align: start`, which specifies that the left-hand edges (the "starts" along the x axis, in our case) are the designated snap points.

Last of all we specify the scroll margin-values, a different one for the second and third child elements:

```
.scroller > div:nth-child(2) {
  scroll-margin-inline-start: 1rem;
}

.scroller > div:nth-child(3) {
  scroll-margin-inline-start: 2rem;
}
```

This means that when scrolling past the middle child elements, the scrolling will snap to `1rem` outside the inline start edge of the second `<div>`, and `2rems` outside the inline start edge of the third `<div>`.

Result

Try it for yourself:

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-left CSS property

The `scroll-margin-left` property defines the left margin of the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-left: 0;
```

```
scroll-margin-left: 20px;
```

```
scroll-margin-left: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}
```

```
.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-left: 10px;
scroll-margin-left: 1em;

/* Global values */
scroll-margin-left: inherit;
scroll-margin-left: initial;
scroll-margin-left: revert;
scroll-margin-left: revert-layer;
scroll-margin-left: unset;
```

Values

`<length>`

An outset from the left edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-left =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-right CSS property

The `scroll-margin-right` property defines the right margin of the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-right: 0;
```

```
scroll-margin-right: 20px;
```

```
scroll-margin-right: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
```

```
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-right: 10px;
scroll-margin-right: 1em;

/* Global values */
scroll-margin-right: inherit;
scroll-margin-right: initial;
scroll-margin-right: revert;
scroll-margin-right: revert-layer;
scroll-margin-right: unset;
```

Values

`<length>`

An outset from the right edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-right =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-margin-top CSS property

The `scroll-margin-top` property defines the top margin of the scroll snap area that is used for snapping this box to the snapport. The scroll snap area is determined by taking the transformed border box, finding its rectangular bounding box (axis-aligned in the scroll container's coordinate space), then adding the specified outsets.

Try it

```
scroll-margin-top: 0;
```

```
scroll-margin-top: 20px;
```

```
scroll-margin-top: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
```

```
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* <length> values */
scroll-margin-top: 10px;
scroll-margin-top: 1em;

/* Global values */
scroll-margin-top: inherit;
scroll-margin-top: initial;
scroll-margin-top: revert;
scroll-margin-top: revert-layer;
scroll-margin-top: unset;
```

Values

`<length>`

An outset from the top edge of the scroll container.

Formal definition

Initial value	`0`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-margin-top =
  <length>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-marker-group CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`scroll-marker-group`** CSS property controls whether a scroll container element has a `::scroll-marker-group` pseudo-element generated. If present, the property also specifies whether the scroll marker group should be placed `before` *or* `after` the contents of the scroll group container in the default visual and tab order.

> **Note:** To create a scroll marker group container from an existing element that contains a set of `<a>` elements, use the `scroll-target-group` property. Read about the differences in behavior between the two.

Syntax

```
/* Single values */
scroll-marker-group: before;
scroll-marker-group: after;
scroll-marker-group: none;

/* Global values */
scroll-marker-group: inherit;
scroll-marker-group: initial;
scroll-marker-group: revert;
scroll-marker-group: revert-layer;
scroll-marker-group: unset;
```

Values

This property is specified as one of the following keyword values:

`after`

A `::scroll-marker-group` pseudo-element is generated as a sibling of the scroll container's child DOM elements, immediately preceding them, and any generated `::scroll-button()` pseudo-elements. It appears at the end of the container's tab order and layout box order (but not DOM structure).

`before`

A `::scroll-marker-group` pseudo-element is generated as a sibling of the scroll container's child DOM elements, immediately preceding them, and any generated `::scroll-button()` pseudo-elements. The scroll marker group appears at the start of the container's tab order and layout box order.

`none`

No `::scroll-marker-group` pseudo-element will be generated on the element. This is the default value.

> **Note:** As an accessibility best practice, match the visual rendering position of the scroll marker group container with the tab order. When positioning the marker group at the start of the content with styles applied to `::scroll-marker-group`, put it at the beginning of the tab order using `before`. When positioning the group at the end of the content, put it at the end of the tab order using `after`.

Formal definition

Initial value	none
Applies to	scroll containers
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scroll-marker-group =
  none                                  |
  [ [ before | after ] || [ links | tabs ] ]
```

This syntax reflects the latest standard as per CSS Overflow Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

See Creating CSS carousels for full examples that use the `scroll-marker-group` property.

Placement of the scroll markers

In this example, we demonstrate the three values of the `scroll-marker-group` property.

HTML

We have a basic HTML `<ul>` list with several `<li>` list items.

```
<fieldset>
  <legend>Select <code>scroll-marker-group</code> value:</legend>
  <label><input type="radio" name="p" value="before" />before</label>
  <label><input type="radio" name="p" value="after" checked />after</label>
  <label><input type="radio" name="p" value="none" />none</label>
</fieldset>
```

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>
  <li>Item 5</li>
  <li>Item 6</li>
  <li>Item 7</li>
  <li>Item 8</li>
</ul>
```

CSS

We convert our `<ul>` into a carousel by setting the `display` to `flex`, creating a single, non-wrapping row of `<li>` elements. The `overflow-x` property is set to `auto`, meaning if the items overflow their container on the x-axis, the content will scroll horizontally. We then convert the `<ul>` into a scroll-snap container, ensuring that items always snap into place when the container is scrolled with a `scroll-snap-type` value of `mandatory`.

We create a scroll marker group container with the `scroll-marker-group` property, placing the group after all the content.

```
ul {
  display: flex;
  gap: 4vw;
  padding-left: 0;
  margin: 32px 0;
  overflow-x: auto;
  overscroll-behavior-x: contain;
  scroll-snap-type: x mandatory;

  scroll-marker-group: after;
}
```

Next, we style the `<li>` elements, using the `flex` property to make them `33%` of the width of the container. The `scroll-snap-align` value of `start` makes the left-hand side of the left-most visible item snap to the left edge of the container when the content is scrolled.

```
li {
  list-style-type: none;
  background-color: #eeeeee;
  flex: 0 0 33%;
  scroll-snap-align: start;
  text-align: center;
  line-height: 5;
}
```

We then use the `::scroll-marker` pseudo-element to create a square marker for each list item with a red border, and apply styles to the `::scroll-marker-group` pseudo-element to lay out the scroll markers in a row with a `0.2em` gap between each one.

```
li::scroll-marker {
  content: " ";
  border: 1px solid red;
```

```
  height: 1em;
  width: 1em;
}

::scroll-marker-group {
  display: flex;
  gap: 0.2em;
}
```

Finally, to ensure good user experience, we style the marker of the currently-scrolled element differently from the others, targeting the marker with the `:target-current` pseudoclass.

```
::scroll-marker:target-current {
  background: red;
}
```

```
fieldset {
  width: 20em;
}

label {
  font-family: monospace;
  display: block;
}

:has([value="before"]:checked) ul {
  scroll-marker-group: before;
}
:has([value="after"]:checked) ul {
  scroll-marker-group: after;
}

:has([value="none"]:checked) ul {
  scroll-marker-group: none;
}
```

Result

Note the placement of the scroll marker group container. Check out how the keyboard tabbing order is different for `before` versus `after`, and note how the group disappears when the value is set to `none`.

See also

- `scroll-target-group`
- `::scroll-button()`
- `::scroll-marker-group`
- `::scroll-marker`
- `:target-current`
- `:target-before`
- `:target-after`
- Creating CSS carousels
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

scroll-padding CSS property

The **`scroll-padding`** CSS shorthand property sets scroll padding on all sides of an element at once. It specifies offsets that define the optimal viewing region of a scrollport within a scroll container.

Try it

```
scroll-padding: 0;
```

```
scroll-padding: 20px;
```

```
scroll-padding: 20%;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
```

```
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `scroll-padding-bottom`
- `scroll-padding-left`
- `scroll-padding-right`
- `scroll-padding-top`

Syntax

```
/* Keyword value */
scroll-padding: auto;

/* <length> values */
scroll-padding: 10px;
scroll-padding: 1em 0.5em 1em 1em;
scroll-padding: 10%;

/* Global values */
scroll-padding: inherit;
scroll-padding: initial;
scroll-padding: revert;
scroll-padding: revert-layer;
scroll-padding: unset;
```

Values

`<length-percentage>`

An inwards offset from the corresponding edge of the scrollport, as a valid `<length>` or a `<percentage>`.

`auto`

The offset is determined by the user agent. This will generally be `0px`, but the user agent is free to detect and do something else if a non-zero value is more appropriate.

Description

The `scroll-padding` property is a shorthand that sets `scroll-padding-top`, `scroll-padding-right`, `scroll-padding-bottom`, and `scroll-padding-left`, in that order, setting the top, right, bottom, and left scroll padding of a scroll container, respectively.

Useful when creating scroll-snap containers, the `scroll-padding` property enables defining offsets for the *optimal viewing region* of the scrollport: the region used as the target region for placing elements in view of the user. This allows you to create insets in the scrollport to make room for objects that might obscure the content, such as fixed-positioned toolbars or sidebars, or to put more breathing room between a targeted element and the edges of the scrollport.

While defined in the CSS scroll snap module, this property applies to all scroll containers, no matter the value of the `scroll-snap-type` property.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-padding-bottom`: `auto` • `scroll-padding-left`: `auto` • `scroll-padding-right`: `auto` • `scroll-padding-top`: `auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as each of the properties of the shorthand: • `scroll-padding-bottom`: as specified • `scroll-padding-left`: as specified • `scroll-padding-right`: as specified • `scroll-padding-top`: as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding =
  [ auto | <length-percentage [0,∞]> ]{1,4}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preventing content from being hidden by a fixed header

A common use case for `scroll-padding` is to prevent scrolling content from being hidden by a fixed-position header. This example demonstrates how to use `scroll-padding` for this purpose.

HTML

The header contains links to three sections on the page.

```
<header>
  <nav>
    <a href="#section-1">Section 1</a>
    <a href="#section-2">Section 2</a>
    <a href="#section-3">Section 3</a>
  </nav>
</header>
<main>
  <section id="section-1">
    <h2>Section 1</h2>
    <p>Use the links in the header to jump to another section.</p>
  </section>
  <section id="section-2">
    <h2>Section 2</h2>
    <p>This heading remains visible below the fixed header.</p>
  </section>
  <section id="section-3">
    <h2>Section 3</h2>
    <p>This heading also remains visible below the fixed header.</p>
  </section>
</main>
```

CSS

```
header {
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #eeeeee;
}

nav {
  display: flex;
  gap: 1rem;
}

section {
  box-sizing: border-box;
  padding: 1rem;
  min-height: 60vh;
}

h2 {
  margin-top: 0;
}
```

The header is fixed to the top of the viewport and is `60px` tall. We set `scroll-padding` on the root element to reserve `60px` at the top of the viewport when scrolling to a section, with no offset on the other sides.

```
html {
  --navbar-height: 60px;

  scroll-padding: var(--navbar-height) 0 0;
  scroll-behavior: smooth;
}

header {
  position: fixed;
  top: 0;
  height: var(--navbar-height);
  width: 100%;
}
```

We add padding to the main content so the first section starts below the header on initial load.

```
main {
  padding-top: var(--navbar-height);
}
```

Result

Click the links in the header to jump between sections. Each section appears below the header, keeping its heading visible. Open this example in the playground, remove the `scroll-padding` line, and click the links again to see the header obscure the headings.

See also

- `scroll-snap-type`
- Basic concepts of scroll-snap
- CSS scroll snap module
- CSS overflow module
- CSS scroll-driven animations module

scroll-padding-block CSS property

The **`scroll-padding-block`** CSS shorthand property sets the scroll padding of an element in the block dimension.

The scroll-padding properties define offsets for the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-block: 0;
```

```
scroll-padding-block: 20px;
```

```
scroll-padding-block: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `scroll-padding-block-end`
- `scroll-padding-block-start`

Syntax

```
/* Keyword value */
scroll-padding-block: auto;

/* <length> values */
scroll-padding-block: 10px;
scroll-padding-block: 1em 0.5em;
scroll-padding-block: 10%;

/* Global values */
scroll-padding-block: inherit;
scroll-padding-block: initial;
scroll-padding-block: revert;
scroll-padding-block: revert-layer;
scroll-padding-block: unset;
```

Values

`<length-percentage>`

An inwards offset from the corresponding edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-padding-block-start`: `auto` • `scroll-padding-block-end`: `auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as each of the properties of the shorthand: • `scroll-padding-block-start`: as specified • `scroll-padding-block-end`: as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-block =
  [ auto | <length-percentage [0,∞]> ]{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-block-end CSS property

The `scroll-padding-block-end` property defines offsets for the end edge in the block dimension of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-block-end: 0;
```

```
scroll-padding-block-end: 20px;
```

```
scroll-padding-block-end: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-block-end: auto;

/* <length> values */
scroll-padding-block-end: 10px;
scroll-padding-block-end: 1em;
scroll-padding-block-end: 10%;

/* Global values */
scroll-padding-block-end: inherit;
scroll-padding-block-end: initial;
scroll-padding-block-end: revert;
scroll-padding-block-end: revert-layer;
scroll-padding-block-end: unset;
```

Values

`<length-percentage>`

An inwards offset from the block end edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-block-end =
  auto                      |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-block-start CSS property

The `scroll-padding-block-start` property defines offsets for the start edge in the block dimension of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-block-start: 0;
```

```
scroll-padding-block-start: 20px;
```

```
scroll-padding-block-start: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-block-start: auto;

/* <length> values */
scroll-padding-block-start: 10px;
scroll-padding-block-start: 1em;
scroll-padding-block-start: 10%;
```

```
/* Global values */
scroll-padding-block-start: inherit;
scroll-padding-block-start: initial;
scroll-padding-block-start: revert;
scroll-padding-block-start: revert-layer;
scroll-padding-block-start: unset;
```

Values

`<length-percentage>`

An inwards offset from the block start edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-block-start =
  auto                     |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-bottom CSS property

The `scroll-padding-bottom` property defines offsets for the bottom of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-bottom: 0;
```

```
scroll-padding-bottom: 20px;
```

```
scroll-padding-bottom: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}
```

```
.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-bottom: auto;

/* <length> values */
scroll-padding-bottom: 10px;
scroll-padding-bottom: 1em;
scroll-padding-bottom: 10%;

/* Global values */
scroll-padding-bottom: inherit;
scroll-padding-bottom: initial;
scroll-padding-bottom: revert;
scroll-padding-bottom: revert-layer;
scroll-padding-bottom: unset;
```

Values

`<length-percentage>`

An inwards offset from the bottom edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-bottom =
  auto                     |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-inline CSS property

The **`scroll-padding-inline`** CSS shorthand property sets the scroll padding of an element in the inline dimension.

The scroll-padding properties define offsets for the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-inline: 0;
```

```
scroll-padding-inline: 20px;
```

```
scroll-padding-inline: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
```

```
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `scroll-padding-inline-end`
- `scroll-padding-inline-start`

Syntax

```
/* Keyword value */
scroll-padding-inline: auto;

/* <length> values */
scroll-padding-inline: 10px;
scroll-padding-inline: 1em 0.5em;
scroll-padding-inline: 10%;

/* Global values */
scroll-padding-inline: inherit;
scroll-padding-inline: initial;
scroll-padding-inline: revert;
scroll-padding-inline: revert-layer;
scroll-padding-inline: unset;
```

Values

`<length-percentage>`

An inwards offset from the corresponding edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-padding-inline-start`: `auto` • `scroll-padding-inline-end`: `auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as each of the properties of the shorthand: • `scroll-padding-inline-start`: as specified • `scroll-padding-inline-end`: as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-inline =
  [ auto | <length-percentage [0,∞]> ]{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-inline-end CSS property

The `scroll-padding-inline-end` property defines offsets for the end edge in the inline dimension of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element

and the edges of the scrollport.

Try it

```
scroll-padding-inline-end: 0;
```

```
scroll-padding-inline-end: 20px;
```

```
scroll-padding-inline-end: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-inline-end: auto;

/* <length> values */
scroll-padding-inline-end: 10px;
scroll-padding-inline-end: 1em;
scroll-padding-inline-end: 10%;

/* Global values */
scroll-padding-inline-end: inherit;
scroll-padding-inline-end: initial;
scroll-padding-inline-end: revert;
scroll-padding-inline-end: revert-layer;
scroll-padding-inline-end: unset;
```

Values

`<length-percentage>`

An inwards offset from the inline end edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-inline-end =
  auto                      |
```

```
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-inline-start CSS property

The `scroll-padding-inline-start` property defines offsets for the start edge in the inline dimension of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-inline-start: 0;
```

```
scroll-padding-inline-start: 20px;
```

```
scroll-padding-inline-start: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-inline-start: auto;

/* <length> values */
scroll-padding-inline-start: 10px;
scroll-padding-inline-start: 1em;
scroll-padding-inline-start: 10%;

/* Global values */
scroll-padding-inline-start: inherit;
scroll-padding-inline-start: initial;
scroll-padding-inline-start: revert;
scroll-padding-inline-start: revert-layer;
scroll-padding-inline-start: unset;
```

Values

`<length-percentage>`

An inwards offset from the inline start edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-inline-start =
  auto                      |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-left CSS property

The `scroll-padding-left` property defines offsets for the left of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-left: 0;
```

```
scroll-padding-left: 20px;
```

```
scroll-padding-left: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-left: auto;

/* <length> values */
scroll-padding-left: 10px;
```

```
scroll-padding-left: 1em;
scroll-padding-left: 10%;

/* Global values */
scroll-padding-left: inherit;
scroll-padding-left: initial;
scroll-padding-left: revert;
scroll-padding-left: revert-layer;
scroll-padding-left: unset;
```

Values

`<length-percentage>`

An inwards offset from the left edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-left =
  auto                          |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-right CSS property

The `scroll-padding-right` property defines offsets for the right of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-right: 0;
```

```
scroll-padding-right: 20px;
```

```
scroll-padding-right: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.scroller {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
```

```
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: end;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-right: auto;

/* <length> values */
scroll-padding-right: 10px;
scroll-padding-right: 1em;
scroll-padding-right: 10%;

/* Global values */
scroll-padding-right: inherit;
scroll-padding-right: initial;
scroll-padding-right: revert;
scroll-padding-right: revert-layer;
scroll-padding-right: unset;
```

Values

`<length-percentage>`

An inwards offset from the right edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-right =
  auto                          |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-padding-top CSS property

The **`scroll-padding-top`** property defines offsets for the top of the *optimal viewing region* of the scrollport: the region used as the target region for placing things in view of the user. This allows the author to exclude regions of the scrollport that are obscured by other content (such as fixed-positioned toolbars or sidebars) or to put more breathing room between a targeted element and the edges of the scrollport.

Try it

```
scroll-padding-top: 0;
```

```
scroll-padding-top: 20px;
```

```
scroll-padding-top: 2em;
```

```
<section class="default-example" id="default-example">
  <div class="scroller" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example .info {
  inline-size: 100%;
  padding: 0.5em 0;
  font-size: 90%;
  writing-mode: vertical-rl;
}

.scroller {
  text-align: left;
```

```
  height: 250px;
  width: 270px;
  overflow-y: scroll;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: y mandatory;
}

.scroller > div {
  flex: 0 0 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.scroller > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword value */
scroll-padding-top: auto;

/* <length> values */
scroll-padding-top: 10px;
scroll-padding-top: 1em;
scroll-padding-top: 10%;

/* Global values */
scroll-padding-top: inherit;
scroll-padding-top: initial;
scroll-padding-top: revert;
scroll-padding-top: revert-layer;
scroll-padding-top: unset;
```

Values

`<length-percentage>`

An inwards offset from the top edge of the scrollport, as a valid length or a percentage.

`auto`

The offset is determined by the user agent. This will generally be 0px, but a user agent is able to detect and do something else if a non-zero value is more appropriate.

Formal definition

Initial value	`auto`
Applies to	scroll containers
Inherited	no
Percentages	relative to the scroll container's scrollport
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scroll-padding-top =
  auto                  |
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-snap-align CSS property

The **`scroll-snap-align`** CSS property specifies the box's snap position as an alignment of its snap area.

Try it

```
scroll-snap-align: start;
```

```
scroll-snap-align: end;
```

```
scroll-snap-align: center;
```

```
<section class="default-example" id="default-example">
  <div id="example-parent">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
```

```
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

#example-parent {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

#example-parent > div {
  flex: 0 0 66%;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#example-parent > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Single keyword value */
scroll-snap-align: none;
scroll-snap-align: center;
scroll-snap-align: start;
scroll-snap-align: end;

/* Two keyword values */
scroll-snap-align: start end;
scroll-snap-align: end center;
scroll-snap-align: center start;

/* Global values */
scroll-snap-align: inherit;
scroll-snap-align: initial;
scroll-snap-align: revert;
scroll-snap-align: revert-layer;
scroll-snap-align: unset;
```

Values

This property is specified as one or two of the following keyword values, space-separated:

`none`

The box does not define a snap position in that axis.

`start`

The start alignment of this box's scroll snap area, within the scroll container's snapport is a snap position in this axis.

`end`

The end alignment of this box's scroll snap area, within the scroll container's snapport is a snap position in this axis.

`center`

The center alignment of this box's scroll snap area, within the scroll container's snapport is a snap position in this axis.

Description

The `scroll-snap-align` property specifies the box's snap position as an alignment of its snap area (as the alignment subject) within its snap container's snap port (as the alignment container).

One or two values can be specified for the `scroll-snap-align` property. If one value is set, it is applied to both the block and inline axes. If two values are set, the first value controls the block axis and the second value controls the inline axis.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scroll-snap-align =
  [ none | start | end | center ]{1,2}
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- `scroll-snap-stop`
- `scroll-initial-target`
- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-snap-stop CSS property

The **`scroll-snap-stop`** CSS property defines whether or not the scroll container is allowed to "pass over" possible snap positions.

Try it

```
scroll-snap-stop: normal;
```

```
scroll-snap-stop: always;
```

```
<section class="default-example" id="default-example">
  <p class="explanation">
    The effect of this property can be noticed on devices with a touchpad. Try
    to scroll through all items with a single swing. Value
    <b class="keyword">'normal'</b> should pass through all pages, while
    <b class="keyword">'always'</b> will stop at the second page.
  </p>
  <div class="snap-container">
    <div>1</div>
    <div id="example-element">2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-direction: column;
}

.explanation {
  margin-top: 0;
}

.keyword {
  color: darkorange;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

.snap-container {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
  scroll-snap-type: x mandatory;
}

.snap-container > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

.snap-container > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

Syntax

```
/* Keyword values */
scroll-snap-stop: normal;
scroll-snap-stop: always;

/* Global values */
scroll-snap-stop: inherit;
scroll-snap-stop: initial;
scroll-snap-stop: revert;
scroll-snap-stop: revert-layer;
scroll-snap-stop: unset;
```

Values

This property is specified as one of the following keyword values:

`normal`

When the visual viewport of this element's scroll container is scrolled, it may "pass over" possible snap positions.

`always`

The scroll container must not "pass over" a possible snap position; and must snap to the first of this elements' snap positions.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scroll-snap-stop =
  normal  |
  always
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting different snap stops

The example below demonstrates the contrast between the `always` and `normal` values of `scroll-snap-stop`. The difference in the two `scroll-snap-stop` values is more noticeable when the `scroll-snap-type` property is set to `mandatory`, which is what is used in this example.

HTML

```
<p>scroll-snap-stop: always (X Mandatory)</p>
<div class="x mandatory-scroll-snapping always-stop">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>

<p>scroll-snap-stop: always (X Mandatory) on odd child elements</p>
<div class="x mandatory-scroll-snapping always-stop-odd">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>

<p>scroll-snap-stop: always (X Mandatory) on even child elements</p>
<div class="x mandatory-scroll-snapping always-stop-even">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>

<p>scroll-snap-stop: normal (X Mandatory)</p>
<div class="x mandatory-scroll-snapping normal-stop">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>

<p>scroll-snap-stop: always (Y Mandatory)</p>
<div class="y mandatory-scroll-snapping always-stop">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>

<p>scroll-snap-stop: normal (Y Mandatory)</p>
<div class="y mandatory-scroll-snapping normal-stop">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
```

CSS

```
/* setup */
body {
  height: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-flow: column nowrap;
```

```
  font-family: monospace;
  text-align: center;
}

div[class] {
  font-size: 3rem;
  display: flex;
  overflow: auto;
  flex: none;
}

.x {
  width: 50%;
  min-width: 15rem;
  height: 6rem;
  flex-flow: row nowrap;
}

.y {
  width: 30%;
  min-width: 12rem;
  height: 12rem;
  flex-flow: column nowrap;
}

div > div {
  flex: none;
  outline: 1px solid #333333;
}

.x > div {
  width: 90%;
  height: 100%;
}

.y > div {
  width: 100%;
  height: 80%;
}

/* coloration */
div > div:nth-child(even) {
  background-color: #87ea87;
}

div > div:nth-child(odd) {
  background-color: #87ccea;
}
```

```
/* setting up mandatory scroll-snap on parent */
.x.mandatory-scroll-snapping {
  scroll-snap-type: x mandatory;
}

.y.mandatory-scroll-snapping {
  scroll-snap-type: y mandatory;
}

/* defining scroll-snap alignment on children */
div > div {
  scroll-snap-align: center;
}

/* defining scroll-snap stop on children */
.always-stop > div {
  scroll-snap-stop: always;
}

.always-stop-odd > div:nth-of-type(odd) {
  scroll-snap-stop: always;
}

.always-stop-even > div:nth-of-type(even) {
  scroll-snap-stop: always;
}

.normal-stop > div {
  scroll-snap-stop: normal;
}
```

Result

Scroll from left to right and from top to bottom in the X and Y boxes below, respectively. In the X and Y boxes where the `scroll-snap-stop` property is set to `always`, the scrolling is forced to stop at the snap point even when you scroll fast. However, in the boxes where the `scroll-snap-stop` property is set to `normal`, the snap points are skipped when you scroll fast.

If required, you can be selective about the items that are `always` stopped at inside the scroll container. This is demonstrated in the example below by targeting odd and even items; you can choose a different strategy based on your requirement. In the example below, scrolling does not "pass over" odd and even items in the second and third boxes, respectively.

See also

- `scroll-snap-align`
- `scroll-initial-target`
- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-snap-type CSS property

The **`scroll-snap-type`** CSS property is set on a scroll container, opting it into scroll snapping by setting the direction and strictness of snap point enforcement within the snap port.

Try it

```
scroll-snap-type: none;
```

```
scroll-snap-type: x mandatory;
```

```
scroll-snap-type: x proximity;
```

```
<section class="default-example" id="default-example">
  <div id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
  </div>
  <div class="info">Scroll »</div>
</section>
```

```
.default-example {
  flex-wrap: wrap;
}

.default-example .info {
  width: 100%;
  padding: 0.5em 0;
  font-size: 90%;
}

#example-element {
  text-align: left;
  width: 250px;
  height: 250px;
  overflow-x: scroll;
  display: flex;
  box-sizing: border-box;
  border: 1px solid black;
}

#example-element > div {
  flex: 0 0 250px;
  width: 250px;
  background-color: rebeccapurple;
  color: white;
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  scroll-snap-align: start;
}

#example-element > div:nth-child(even) {
  background-color: white;
  color: rebeccapurple;
}
```

If the content in the scroll port changes — for example, if content is added, moved, deleted, or resized — the scroll container will re-snap to the previously snapped content if that content is still present.

If the value of a scroll snap-related property, such as `scroll-snap-type` or `scroll-margin`, is changed, the scroll container will re-snap based on the current value of `scroll-snap-type`.

Specifying any precise animations or physics used to enforce those snap points is not covered by this property but instead left up to the user agent.

Syntax

```
/* No snapping */
scroll-snap-type: none;

/* Keyword values for snap axes */
scroll-snap-type: x;
scroll-snap-type: y;
scroll-snap-type: block;
scroll-snap-type: inline;
scroll-snap-type: both;

/* Optional keyword values for snap strictness */
/* mandatory | proximity */
scroll-snap-type: x mandatory;
scroll-snap-type: y proximity;
scroll-snap-type: both mandatory;

/* Global values */
scroll-snap-type: inherit;
scroll-snap-type: initial;
scroll-snap-type: revert;
scroll-snap-type: revert-layer;
scroll-snap-type: unset;
```

Values

This property is specified as one or two of the following keyword values:

`none`

When the visual viewport of this scroll container is scrolled, it must ignore snap points.

`x`

The scroll container snaps to snap positions in its horizontal axis only.

`y`

The scroll container snaps to snap positions in its vertical axis only.

`block`

The scroll container snaps to snap positions in its block axis only.

`inline`

The scroll container snaps to snap positions in its inline axis only.

`both`

The scroll container snaps to snap positions in both of its axes independently (potentially snapping to different elements in each axis).

`mandatory`

The visual viewport of this scroll container must snap to a snap position if it isn't currently scrolled.

`proximity`

The visual viewport of this scroll container may snap to a snap position if it isn't currently scrolled. The user agent decides if it snaps or not based on scroll parameters. This is the default snap strictness if any snap axis is specified.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scroll-snap-type =
  none                                          |
  [ x | y | block | inline | both | pair ] [ mandatory | proximity ]?
```

This syntax reflects the latest standard as per CSS Scroll Snap Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Snapping in different axes

HTML

```
<main>
  <section class="x mandatory-scroll-snapping" dir="ltr">
    <div>X Mand. LTR</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="x proximity-scroll-snapping" dir="ltr">
    <div>X Prox. LTR</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="y mandatory-scroll-snapping" dir="ltr">
    <div>Y Mand. LTR</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="y proximity-scroll-snapping" dir="ltr">
    <div>Y Prox. LTR</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="x mandatory-scroll-snapping" dir="rtl">
    <div>X Mand. RTL</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="x proximity-scroll-snapping" dir="rtl">
    <div>X Prox. RTL</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="y mandatory-scroll-snapping" dir="rtl">
    <div>Y Mand. RTL</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
  </section>
  <section class="y proximity-scroll-snapping" dir="rtl">
    <div>Y Prox. RTL</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
```

```
  </section>
</main>
```

CSS

```
main {
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-flow: column nowrap;
  font-family: monospace;
}
section {
  display: flex;
  margin: 1em auto;
  outline: 1px dashed lightgray;
  flex: none;
  overflow: auto;
}
.x {
  width: 100%;
  height: 128px;
  flex-flow: row nowrap;
  overflow-y: hidden;
}
.y {
  width: 256px;
  height: 256px;
  flex-flow: column nowrap;
  overflow-x: hidden;
}
```

```
/* scroll-snap */
.x.mandatory-scroll-snapping {
  scroll-snap-type: x mandatory;
}
.x.proximity-scroll-snapping {
  scroll-snap-type: x proximity;
}
.y.mandatory-scroll-snapping {
  scroll-snap-type: y mandatory;
}
.y.proximity-scroll-snapping {
  scroll-snap-type: y proximity;
}

div {
  text-align: center;
  scroll-snap-align: center;
  flex: none;
}
```

```
.x div {
  line-height: 128px;
  font-size: 64px;
  width: 100%;
  height: 128px;
}
.y div {
  line-height: 256px;
  font-size: 128px;
  width: 256px;
  height: 100%;
}

/* appearance fixes */
.y div:first-child {
  line-height: 1.3;
  font-size: 64px;
}

/* coloration */
div:nth-child(even) {
  background-color: #87ea87;
}
div:nth-child(odd) {
  background-color: #87ccea;
}
```

Results

See also

- Other scroll port properties: `scroll-margin`, `scroll-padding`
- Scroll-child properties: `scroll-snap-align`, `scroll-margin`, `scroll-snap-stop`
- Basic concepts of CSS scroll snap
- Using scroll snap events
- CSS scroll snap module
- Well-controlled scrolling with CSS scroll snap on web.dev (2018)

scroll-target-group CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`scroll-target-group`** CSS property specifies whether an element is a scroll marker group container.

Syntax

```
/* Keyword values */
scroll-target-group: none;
scroll-target-group: auto;

/* Global values */
scroll-target-group: inherit;
scroll-target-group: initial;
scroll-target-group: revert;
scroll-target-group: revert-layer;
scroll-target-group: unset;
```

The `scroll-target-group` property is specified as one of the following keyword values:

Values

`none`

The element is not a scroll marker group container.

`auto`

The element is a scroll marker group container.

Description

Setting `scroll-target-group: auto` on an element denotes it as a **scroll marker group container**. This groups together a set of navigation items that allow users to navigate between elements on a page (such as carousel items or article sections) and highlight which element is currently in view.

Any `<a>` elements with fragment identifiers inside the container are automatically set as scroll markers. The anchor element whose scroll target is currently in view can be styled via the `:target-current` pseudo-class.

Differences between <code>scroll-target-group</code> and <code>scroll-marker-group</code>

Scroll marker group containers created using `scroll-target-group` behave in a very similar way to those created using the `scroll-marker-group` property, with some differences:

- With `scroll-target-group`, you have to create your own markup to represent the scroll marker group container and scroll markers, whereas `scroll-marker-group` automatically creates a set of pseudo-elements to represent the container (`::scroll-marker-group`) and the markers (one or more instances of `::scroll-marker`). These automatically have the expected navigation associations with the scroll container they are generated on. Using `scroll-marker-group` provides a quicker setup because you don't need to use your own markup. However, creating your own markup and setting it as a scroll marker group container via `scroll-target-group` provides more control and flexibility.
- Links denoted as scroll markers via `scroll-target-group` have no special navigation behavior, whereas markers generated via `scroll-marker-group` automatically have `tablist`/`tab` semantics applied to them, meaning they behave like a single item in the tab order, and users can move between scroll markers with the arrow keys. Again, `scroll-marker-group` provides useful default behavior, but you have the flexibility of providing alternative semantics and behavior for markers specified using `scroll-target-group`.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scroll-target-group =
  none  |
  auto
```

This syntax reflects the latest standard as per CSS Overflow Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>scroll-target-group</code> usage

This example shows a page with a table of contents linking to different sections.

HTML

Our markup has a series of `<section>` elements containing content, and a table of contents created using an ordered list (`<ol>`/`<li>`) and `<a>` elements.

```
<nav id="toc">
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#ch1">Chapter 1</a></li>
    <li><a href="#ch2">Chapter 2</a></li>
    <li><a href="#ch3">Chapter 3</a></li>
    <li><a href="#ch4">Chapter 4</a></li>
  </ol>
</nav>
<section id="intro" class="chapter">
  <h1>Prose of the century</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</section>
<section id="ch1" class="chapter">
  <h2>Chapter 1</h2>

  <!-- ... -->
</section>
<section id="ch2" class="chapter">
  <h2>Chapter 2</h2>

  <!-- ... -->
```

```
</section>

<!-- ... -->
```

```
<nav id="toc">
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#ch1">Chapter 1</a></li>
    <li><a href="#ch2">Chapter 2</a></li>
    <li><a href="#ch3">Chapter 3</a></li>
    <li><a href="#ch4">Chapter 4</a></li>
  </ol>
</nav>
<section id="intro" class="chapter">
  <h1>My story</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</section>
<section id="ch1" class="chapter">
  <h2>Chapter 1</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
  <p>
    Nam vulputate diam nec tempor bibendum. Donec luctus augue eget malesuada
    ultrices. Phasellus turpis est, posuere sit amet dapibus ut, facilisis sed
    est. Nam id risus quis ante semper consectetur eget aliquam lorem. Vivamus
    tristique elit dolor, sed pretium metus suscipit vel. Mauris ultricies
    lectus sed lobortis finibus. Vivamus eu urna eget velit cursus viverra quis
    vestibulum sem. Aliquam tincidunt eget purus in interdum. Cum sociis natoque
    penatibus et magnis dis parturient montes, nascetur ridiculus mus.
  </p>
</section>
<section id="ch2" class="chapter">
  <h2>Chapter 2</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
<section id="ch3" class="chapter">
  <h2>Chapter 3</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
<section id="ch4" class="chapter">
  <h2>Chapter 4</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
```

CSS

We've hidden most of the styling for brevity. Most pertinent to this example, we've set `scroll-target-group: auto` on the `<ol>` to turn it into a scroll marker group container and trigger the browser's algorithm for calculating which `<a>` element is the `:target-current` at any given time (that is, which link's target is currently in view). We then style the `:target-current` pseudo-class with a `red` `color` so that it stands out clearly.

```
body {
  font: 1.2em / 1.5 system-ui;
  width: 50%;
  max-width: 700px;
  margin: 0 auto;
}

nav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  display: flex;
  justify-content: center;
}

section {
  padding-top: 60px;
}

a {
  color: black;
  text-decoration: none;
}

a:hover,
a:focus {
  text-decoration: underline;
}

ol {
  display: flex;
  width: 100%;
  justify-content: space-around;
  list-style-type: none;
  padding: 20px 0;
  margin: 0;
  background: white;
}
```

```
ol {
  scroll-target-group: auto;
}

:target-current {
  color: red;
}
```

Result

Try navigating by activating the links and by scrolling. You'll see that in each case, the red highlight moves between the links to match the section currently being shown.

CSS carousel with <code>scroll-target-group</code> scroll markers

This example shows how to create CSS carousel scroll markers using `scroll-target-group`. The code for this example is similar to our Carousel with single pages example; we'll just explain the differences below.

HTML

The markup has IDs set on the list items that define each page, and an ordered list added that we'll turn into a scroll marker group container using CSS.

```
<h1>CSS carousel single item per page</h1>
<ul>
  <li id="page1">
    <h2>Page 1</h2>
  </li>
  <li id="page2">
    <h2>Page 2</h2>
  </li>
  <li id="page3">
    <h2>Page 3</h2>
  </li>
  <li id="page4">
    <h2>Page 4</h2>
  </li>
</ul>
<ol>
  <li><a href="#page1">1</a></li>
  <li><a href="#page2">2</a></li>
  <li><a href="#page3">3</a></li>
  <li><a href="#page4">4</a></li>
</ol>
```

CSS

We create the scroll marker group container and scroll markers by setting `scroll-target-group` on the `<ol>` element. The rest of the code for styling these is very similar, except that we are targeting elements of our own choosing (`<ol>` and `<a>`) rather than the `::scroll-marker-group` and `::scroll-marker` pseudo-elements.

We complete the code by setting some styles on the `:target-current`, `a:hover`, and `a:focus` states to indicate which page is currently being shown and which links are being hovered/focused.

```
/* General styles */

* {
  box-sizing: border-box;
}
```

```css
body {
  margin: 0;
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  text-align: center;
}

button {
  background-color: white;
}

/* General styling of list as scrolling carousel */

ul {
  width: 100%;
  height: 400px;
  padding: 20px;
  gap: 4%;
  display: grid;
  grid-auto-flow: column;
  grid-auto-columns: 100%;
  overflow-x: scroll;
  scroll-snap-type: x mandatory;
}

ul li {
  list-style-type: none;
  background-color: #eeeeee;
  border: 1px solid #dddddd;
  padding: 20px;

  scroll-snap-align: center;
}

/* CSS scroll buttons */

/* Style the scroll buttons */

ul::scroll-button(*) {
  border: 0;
  font-size: 2rem;
  background: none;
  color: rgb(0 0 0 / 0.7);
  cursor: pointer;
}

ul::scroll-button(*):hover,
ul::scroll-button(*):focus {
  color: black;
}

ul::scroll-button(*):active {
  translate: 1px 1px;
}

ul::scroll-button(*):disabled {
  color: rgb(0 0 0 / 0.2);
}

ul::scroll-button(left) {
  content: "◄";
}

ul::scroll-button(right) {
  content: "►";
}

/* Position the scroll buttons */

ul {
  anchor-name: --my-carousel;
}

ul::scroll-button(*) {
  position: absolute;
  position-anchor: --my-carousel;
}

ul::scroll-button(left) {
  right: calc(anchor(left) - 70px);
  top: calc(anchor(bottom) - 80px);
}

ul::scroll-button(right) {
  left: calc(anchor(right) - 70px);
  top: calc(anchor(bottom) - 80px);
}
```

```css
ol {
  position: absolute;
  position-anchor: --my-carousel;
  top: calc(anchor(bottom) - 90px);
  justify-self: anchor-center;
  display: flex;
  justify-content: center;
  gap: 20px;

  list-style-type: none;
  padding: 0;
  scroll-target-group: auto;
}
```

```
ol a {
  width: 28px;
  height: 28px;
  display: inline-block;
  text-align: center;
  text-decoration: none;
  padding-top: 4px;
  color: black;
  background-color: transparent;
  border: 2px solid black;
  border-radius: 50%;
}

ol a:hover,
ol a:focus,
:target-current {
  background-color: black;
  color: white;
}
```

Result

Try navigating in each of these three ways: by activating the scroll marker links, by horizontal scrolling, or by pressing the scroll buttons on either side. You'll see that in each case, the highlight moves between the links to match the section currently being shown.

See also

- `scroll-marker-group`
- `::scroll-marker-group` and `::scroll-marker` pseudo-elements
- `:target-current`, `:target-before`, and `:target-after` pseudo-classes
- Creating CSS carousels
- CSS overflow module

scroll-timeline CSS property

The **`scroll-timeline`** CSS shorthand property is used to define a named scroll progress timeline, which is progressed through by scrolling a scrollable element (*scroller*) between top and bottom (or left and right).

Constituent properties

This property is a shorthand for the following CSS properties, in order:

- `scroll-timeline-name`
- `scroll-timeline-axis`

Syntax

```
/* One value */
scroll-timeline: none;
scroll-timeline: --custom_name_for_timeline;

/* Two values */
scroll-timeline: --custom_name_for_timeline block;
scroll-timeline: --custom_name_for_timeline x;
scroll-timeline: none inline;
scroll-timeline: none y;

/* Global values */
scroll-timeline: inherit;
scroll-timeline: initial;
scroll-timeline: revert;
scroll-timeline: revert-layer;
scroll-timeline: unset;
```

Values

`<scroll-timeline-name>`

A `<dashed-ident>` or the keyword `none`.

`<scroll-timeline-axis>`

An `<axis>` keyword. The default value is `block`.

Description

The `scroll-timeline` shorthand property can be applied to a container element to set both the `scroll-timeline-name` and `scroll-timeline-axis` properties. It is set on the scroller that will provide the timeline. If the container has no overflow to scroll or if the overflow is hidden or clipped, no timeline will be created.

The value for the `scroll-timeline-name`, if not set to `none`, must be a `<dashed-ident>`, which means it must start with `--`. This helps avoid name clashes with standard CSS keywords. The name can then be used as the value of an element's `animation-timeline` property to define the scrolling container element that will define its animation timeline, driving the animation progress as it scrolls.

The optional scroll `<axis>` value defines the `scroll-timeline-axis`, which defaults to `block` if omitted. If both a name and axis are specified, the order must be the `<scroll-timeline-name>` value followed by the `<axis>` value. If the `<axis>` is listed first, the declaration is invalid and will be ignored.

Formal definition

Initial value	as each of the properties of the shorthand: • `scroll-timeline-name`: `none` • `scroll-timeline-axis`: `block`
Applies to	scroll containers
Inherited	no

Computed value	as each of the properties of the shorthand: • `scroll-timeline-name`: `none` or an ordered list of identifiers • `scroll-timeline-axis`: as specified
Animation type	as each of the properties of the shorthand: • `scroll-timeline-name`: Not animatable • `scroll-timeline-axis`: Not animatable

Formal syntax

```
scroll-timeline =
  [ <'scroll-timeline-name'> <'scroll-timeline-axis'>? ]#

<scroll-timeline-name> =
  [ none | <dashed-ident> ]#

<scroll-timeline-axis> =
  [ block | inline | x | y ]#
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a named scroll progress timeline animation

In this example, a scroll timeline named `--square-timeline` is defined using the `scroll-timeline-name` property on the `#container` element. The timeline name is then applied to the animation on the `#square` element using `animation-timeline: --square-timeline`.

HTML

We include a container with two child `<div>` elements.

```
<div id="container">
  <div id="square"></div>
  <div id="stretcher"></div>
</div>
```

CSS

We set the container to `300px` tall to make it scroll vertically.

Using the `scroll-timeline` property, we define the container as the source of a scroll timeline named `--square-timeline`, explicitly setting the default vertical scrollbar as the timeline controller.

```
#container {
  height: 300px;
  overflow-y: scroll;

  scroll-timeline: --square-timeline y;
  /* Firefox supports the non-standard vertical/horizontal syntax */
  scroll-timeline: --square-timeline vertical;

  position: relative;
}
```

We provide basic styles for the square. We apply the `rotateAnimation` using the `animation-name` property. By default, this would use the time-based document timeline. By setting the `animation-timeline` property to the `--square-timeline` timeline named above, we set the square to rotate according to the scroll-based timeline, which progresses as the container scrolls.

```
#square {
  background-color: deeppink;
  width: 100px;
  height: 100px;
  position: absolute;
  bottom: 0;

  animation: rotateAnimation 1ms linear;
  animation-timeline: --square-timeline;
}
```

We ensure the container overflows by defining a stretcher that is wider than its parent. Without content that overflows the container, there would be no scrollbar, and hence no scroll timeline. We also define a CSS keyframe animation that rotates the elements on which it is applied by one full rotation.

```
#stretcher {
  height: 600px;
  background: #dedede;
}

@keyframes rotateAnimation {
  from {
    transform: rotate(0deg);
  }

  to {
    transform: rotate(360deg);
  }
}
```

```
@layer no-support {
  @supports not (scroll-timeline: --square-timeline) {
    body::before {
      content: "Your browser doesn't support the `scroll-timeline` property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
```

```
  }
}
```

Result

Scroll the vertical bar to see the square animate as you scroll.

The square animates as you scroll. When using `scroll-timeline`, the duration of the animation depends on how fast you scroll, not the value of the `animation-duration` property.

See also

- `animation-timeline`
- `scroll-timeline-axis`
- `scroll-timeline-name`
- Guide: Scroll-driven animation timelines
- CSS scroll-driven animations module
- CSS animations module

scroll-timeline-axis CSS property

The **`scroll-timeline-axis`** CSS property is used to specify the scrollbar direction that will be used to provide the timeline for a scroll driven animation, which is progressed through by scrolling a scrollable element (*scroller*).

Syntax

```
/* Logical property values */
scroll-timeline-axis: block;
scroll-timeline-axis: inline;

/* Physical property values */
scroll-timeline-axis: y;
scroll-timeline-axis: x;

/* Global values */
scroll-timeline-axis: inherit;
scroll-timeline-axis: initial;
scroll-timeline-axis: revert;
scroll-timeline-axis: revert-layer;
scroll-timeline-axis: unset;
```

Values

`<axis>`

> An `<axis>` keyword value describing the direction, or axis, of the scrollport that controls the scroll-driven animation. The default value is `block`.

Description

The `scroll-timeline-axis` property specifies which scrollbar will be used to provide the timeline for a scroll progress timeline animation. The value is the `<axis>` of the scrollbar. The `scroll-timeline` property is set on the scroller that will provide the timeline.

If the scroller element does not overflow its container in the axis dimension or if the overflow is hidden or clipped, no scroll progress timeline will be created.

The `scroll-timeline-axis` and `scroll-timeline-name` properties can also be set using the `scroll-timeline` shorthand property.

Formal definition

Initial value	`block`
Applies to	scroll containers
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
scroll-timeline-axis =
  [ block | inline | x | y ]#
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the axis of the scroll progress timeline

In this example, a scroll progress timeline named `--my-scroller` is defined using the `scroll-timeline-name` property on the `:root` element (`<html>`). This timeline is then applied to the animation on the element with the `animation` class using `animation-timeline: --my-scroller`.

To demonstrate the effect of `scroll-timeline-axis`, a horizontal (non-default) scrollbar is used in this example to drive the animation.

HTML

The HTML for the example is shown below.

```
<body>
  <div class="content"></div>
  <div class="box animation"></div>
</body>
```

CSS

The CSS for the container sets the `:root` as the source of a scroll progress timeline named `--my-scroller` using the `scroll-timeline-name` property. The scroll axis is set using `scroll-timeline-axis: x;` which causes the *horizontal scrollbar* position to determine the animation timeline. We also include `scroll-timeline-axis: horizontal;` for browsers that support the non-standard legacy `horizontal` and `vertical` values and not `x` and `y`.

The width of the `.content` element is set to a large value to make it overflow the `:root` element.

The `.animation` element has the animation applied to it using the `animation` shorthand, and the scroll timeline set using the `animation-timeline`.

```
:root {
  scroll-timeline-name: --my-scroller;

  scroll-timeline-axis: x;
  scroll-timeline-axis: horizontal;
}

body {
  margin: 0;
  overflow-y: hidden;
}

.content {
  height: 100vh;
  width: 2000px;
}

.box {
  width: 100px;
  height: 100px;
  border-radius: 10px;
  background-color: rebeccapurple;
  position: fixed;
  top: 25px;
  left: 25px;
}

.animation {
  animation: rotate-appear 1ms linear;
  animation-timeline: --my-scroller;
}

@keyframes rotate-appear {
  from {
    rotate: 0deg;
    top: 0%;
  }

  to {
    rotate: 720deg;
    top: 100%;
  }
}
```

```
@layer no-support {
  @supports not (scroll-timeline-axis: block) {
    body::before {
      content: "Your browser doesn't support the `scroll-timeline-axis` property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Scroll the horizontal bar at the bottom to see the square animate as you scroll.

See also

- `animation-timeline`
- `scroll-timeline`
- `scroll-timeline-name`
- Scroll-driven animation timelines
- CSS scroll-driven animations module

scroll-timeline-name CSS property

The **`scroll-timeline-name`** CSS property is used to define the name of a named scroll progress timeline.

Syntax

```
/* Keyword value */
scroll-timeline-name: none;

/* Named timeline */
scroll-timeline-name: --custom_name_for_timeline;

/* Global values */
scroll-timeline-name: inherit;
scroll-timeline-name: initial;
scroll-timeline-name: revert;
scroll-timeline-name: revert-layer;
scroll-timeline-name: unset;
```

Values

This property is specified as one of the following values:

`none`

The timeline has no name.

`<dashed-ident>`

An arbitrary custom identifier defining a name for a scroll progress timeline, which can then be referenced in an `animation-timeline` property.

Description

The `scroll-timeline-name` property is used to define the name of a *named scroll progress timeline*, which is progressed through by scrolling a scrollable element (*scroller*) between top and bottom (or left and right).

Set the `scroll-timeline-name` property on the scroller that provides the timeline. The name is then referenced in an `animation-timeline` declaration to indicate the container's element that is used to drive the progress of the animation through the scrolling action.

If the element does not overflow its container in the axis dimension or if the overflow is hidden or clipped, no timeline will be created.

The `scroll-timeline-name`, along with the `scroll-timeline-axis` property, can also be set using the `scroll-timeline` shorthand.

Formal definition

Initial value	none
Applies to	scroll containers
Inherited	no
Computed value	`none` or an ordered list of identifiers
Animation type	Not animatable

Formal syntax

```
scroll-timeline-name =
  [ none | <dashed-ident> ]#
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a named scroll progress timeline animation

In this example, a scroll timeline named `--square-timeline` is defined using the `scroll-timeline-name` property on the element with the ID `container`. This is then applied to the animation on the `#square` element using `animation-timeline: --square-timeline`.

HTML

The HTML for the example is shown below.

```
<div id="container">
  <div id="square"></div>
  <div id="stretcher"></div>
</div>
```

CSS

The CSS for the container sets it as the source of a scroll timeline named `--square-timeline` using the `scroll-timeline-name` property. No scrollbar axis is defined here because the vertical axis will be used by default.

The height of the container is set to `300px`, and the container is also set to create a vertical scrollbar if it overflows (the CSS `height` rule on the `stretcher` element below does make the content overflow its container).

```
#container {
  height: 300px;
  overflow-y: scroll;
  scroll-timeline-name: --square-timeline;
  position: relative;
}
```

The CSS below defines a square that rotates according to the timeline provided by the `animation-timeline` property, which is set to the `--square-timeline` timeline named above.

```
#square {
  background-color: deeppink;
  width: 100px;
  height: 100px;
  margin-top: 100px;
  animation-name: rotateAnimation;
  animation-duration: 1ms; /* Firefox requires this to apply the animation */
  animation-timeline: --square-timeline;
  position: absolute;
  bottom: 0;
}

#stretcher {
  height: 600px;
  background: #dedede;
}

@keyframes rotateAnimation {
  from {
    transform: rotate(0deg);
  }

  to {
    transform: rotate(360deg);
  }
}
```

The `stretcher` CSS rule sets the block height to `600px`, which creates content that overflows the container element, thereby creating scroll bars. Without this element, the content would not overflow the container, there would be no scrollbar, and hence no scroll timeline to associate with the animation timeline.

Result

Scroll the vertical bar to see the square animate as you scroll.

See also

- `animation-timeline`

- `scroll-timeline`, `scroll-timeline-axis`
- `timeline-scope`
- Scroll-driven animation timelines
- CSS scroll-driven animations

scrollbar-color CSS property

The **`scrollbar-color`** CSS property sets the color of the scrollbar track and thumb.

The **track** refers to the background of the scrollbar, which is generally fixed regardless of the scrolling position.

The **thumb** refers to the moving part of the scrollbar, which usually floats on top of the track.

When `scrollbar-color` value is set on the document's root element, the values are applied to the viewport scrollbars.

> **Note:** Browsers ignore the non-standard `::-webkit-scrollbar` family of pseudo-elements on any element whose *computed* `scrollbar-color` value is anything other than `auto`. Because `scrollbar-color` is inherited, this includes elements that don't set it themselves but inherit a non-`auto` value from an ancestor. Setting `scrollbar-color: auto` on such an element restores its `::-webkit-scrollbar-*` styling if it was previously removed by a `scrollbar-color` setting on an ancestor. See Adding a fallback for scrollbar styles.

Syntax

```
/* Keyword value */
scrollbar-color: auto;

/* <color> values */
scrollbar-color: rebeccapurple green; /* Two valid colors.
The first applies to the thumb of the scrollbar, the second to the track. */

/* Global values */
scrollbar-color: inherit;
scrollbar-color: initial;
scrollbar-color: revert;
scrollbar-color: revert-layer;
scrollbar-color: unset;
```

Values

`<scrollbar-color>`

Defines the color of the scrollbar.

`auto`	Default platform rendering for the track portion of the scrollbar, in the absence of any other related scrollbar color properties.
`<color> <color>`	Applies the first color to the scrollbar thumb, the second to the scrollbar track.

> **Note:** `@media (forced-colors: active)` sets `scrollbar-color` to `auto`.

Accessibility

When using `scrollbar-color` property with specific color values, authors should ensure the specified colors have enough contrast between them. For keyword values, UAs should ensure the colors they use have enough contrast. See Techniques for WCAG 2.0: G183: Using a contrast ratio of 3:1.

Formal definition

Initial value	`auto`
Applies to	scrolling boxes
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
scrollbar-color =
  auto        |
  <color>{2}
```

This syntax reflects the latest standard as per CSS Scrollbars Styling Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Coloring overflow scrollbars

CSS

```
.scroller {
  width: 300px;
  height: 100px;
  overflow-y: scroll;
  scrollbar-color: #000077 #bada55;
}
```

HTML

```
<div class="scroller">
  Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
  daikon amaranth tatsoi tomatillo melon azuki bean garlic. Gumbo beet greens
  corn soko endive gumbo gourd. Parsley shallot courgette tatsoi pea sprouts
  fava bean collard greens dandelion okra wakame tomato. Dandelion cucumber
  earthnut pea peanut soko zucchini.
</div>
```

Result

See also

- CSS overflow module
- CSS scrollbars styling module
- `overflow`
- `scrollbar-gutter`
- `scrollbar-width`

scrollbar-gutter CSS property

The **`scrollbar-gutter`** CSS property allows authors to reserve space for the scrollbar, preventing unwanted layout changes as the content grows while also avoiding unnecessary visuals when scrolling isn't needed.

An element's *scrollbar gutter* is the space between the inner border edge and the outer padding edge, where the browser may display a scrollbar. If no scrollbar is present, the gutter will be painted as an extension of the padding.

The browser determines whether *classic* scrollbars or *overlay* scrollbars are used:

- Classic scrollbars are always placed in a gutter, consuming space when present.
- Overlay scrollbars are placed over the content, not in a gutter, and are usually partially transparent.

Syntax

```
/* Initial value */
scrollbar-gutter: auto;

/* "stable" keyword, with optional modifier */
scrollbar-gutter: stable;
scrollbar-gutter: stable both-edges;

/* Global values */
scrollbar-gutter: inherit;
scrollbar-gutter: initial;
scrollbar-gutter: revert;
scrollbar-gutter: revert-layer;
scrollbar-gutter: unset;
```

Values

`auto`

The initial value. Classic scrollbars create a gutter when `overflow` is `scroll`, or when `overflow` is `auto` and the box is overflowing. Overlay scrollbars do not consume space.

`stable`

When using classic scrollbars, the gutter will be present if `overflow` is `auto`, `scroll`, or `hidden` even if the box is not overflowing. When using overlay scrollbars, the gutter will not be present.

`both-edges`

If a gutter would be present on one of the inline start/end edges of the box, another will be present on the opposite edge as well.

Formal definition

Initial value	`auto`
Applies to	scrolling boxes
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scrollbar-gutter =
  auto                  |
  stable && both-edges?
```

This syntax reflects the latest standard as per CSS Overflow Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

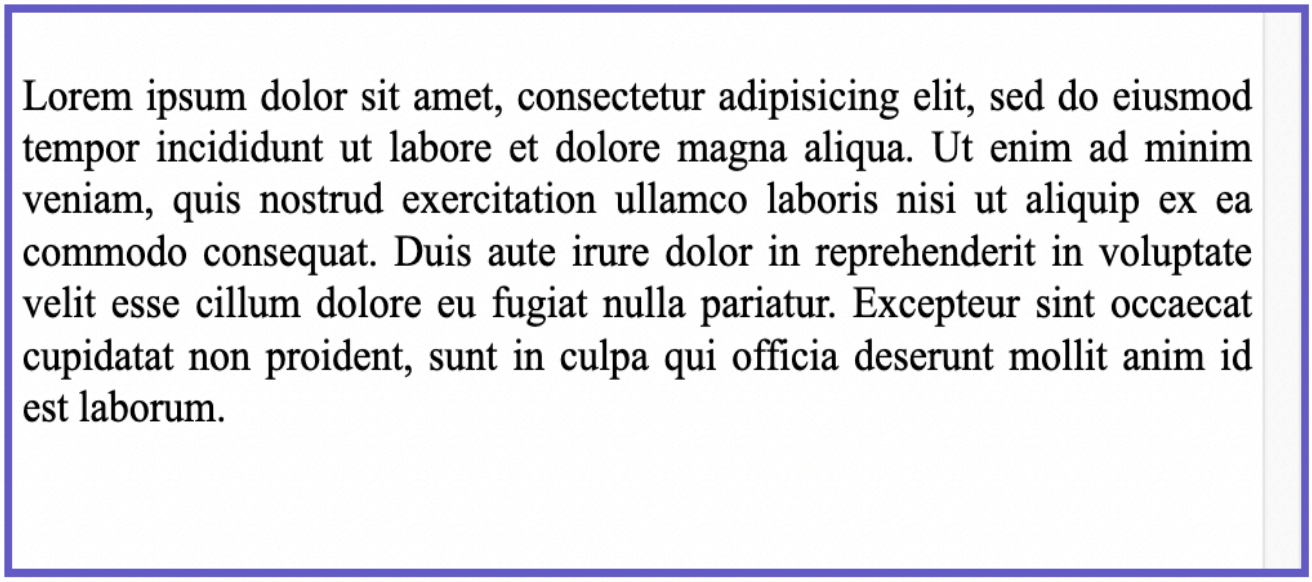
The examples below show how the different values for the `scrollbar-gutter` property would affect a scrollable `div` element (`.container`) with one or more paragraphs contained within.

> **Note:** In the images for the examples, the user's system settings are set to classic scrollbars (always shown).

Example 1

Prevent unneeded layout changes as the content growing or shrinking causes the scrollbar to appear/disappear, a space is reserved for it.

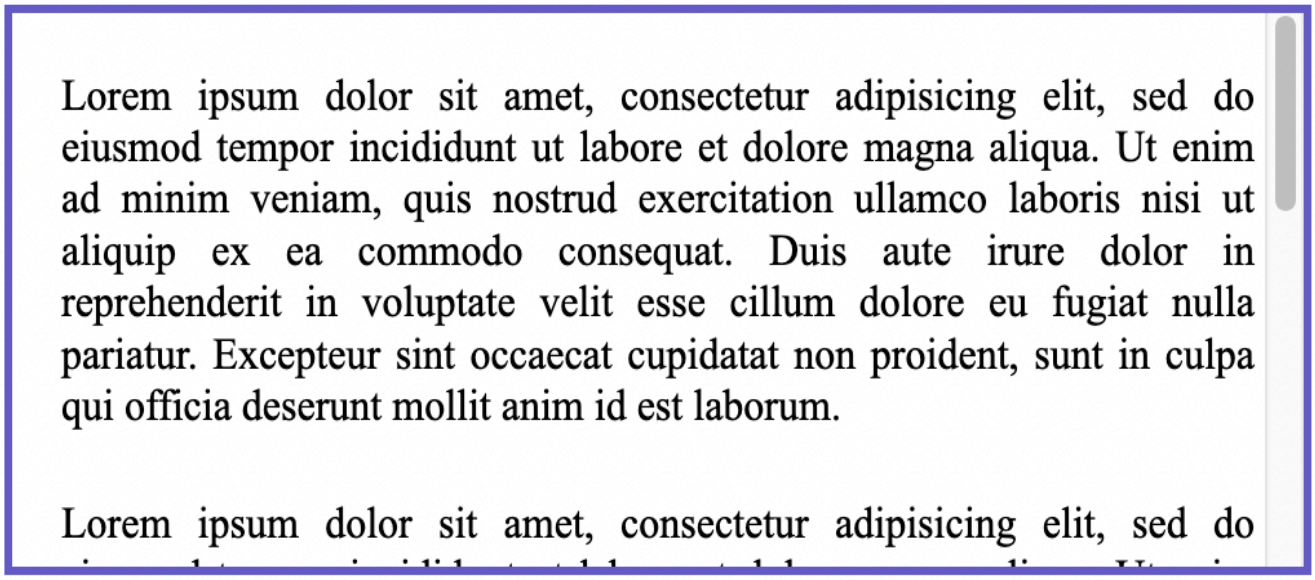
```
.container {
  scrollbar-gutter: stable;
}
```



Example 2

Add symmetric spacing to both sides of the box so the content is centered:

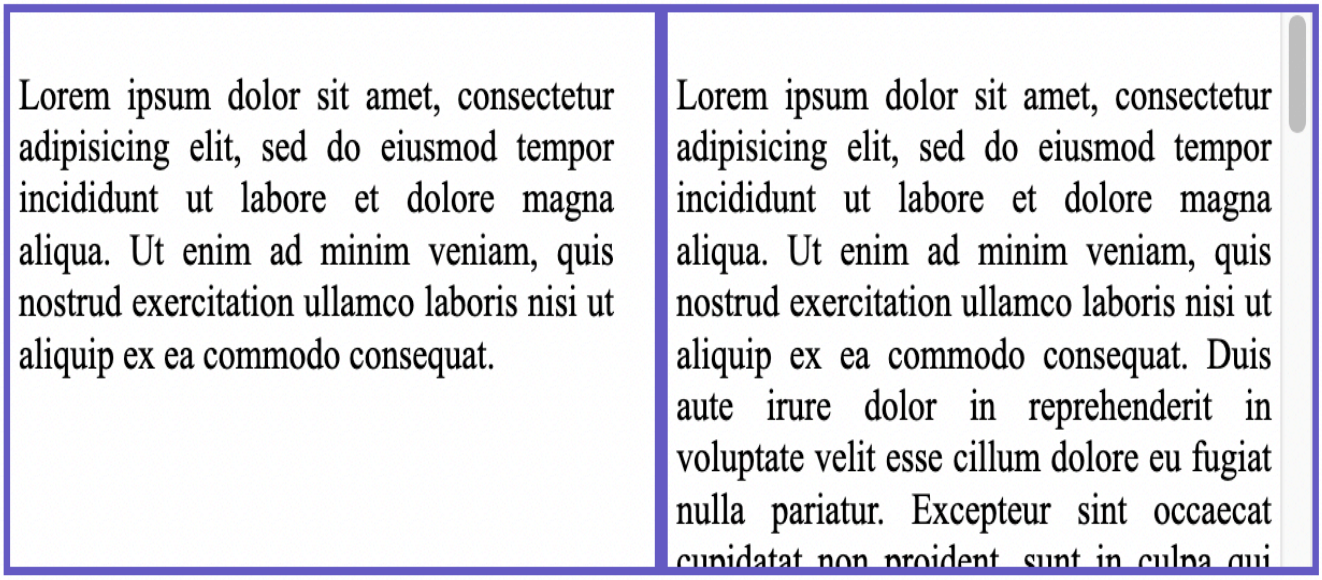
```
.container {
  scrollbar-gutter: stable both-edges;
}
```



Example 3

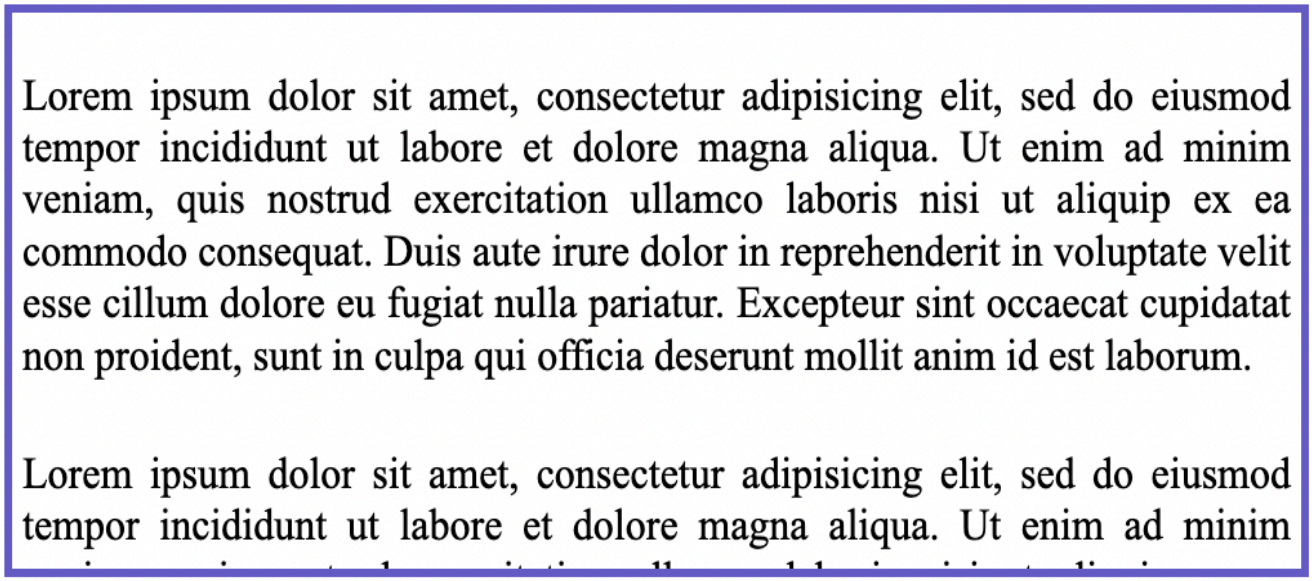
Align the contents of a non-scrolling element and a scrolling one adjacent to it: This example shows two divs side by side. The one on the left has no scroll, but the one on the right does. Both have `scrollbar-gutter` applied, which also reserves space for the div on the left which doesn't have scrollable content. This is a good technique to use to keep the width of content consistent.

```
.container1 {
  overflow: hidden;
  scrollbar-gutter: stable;
}

.container2 {
  scrollbar-gutter: stable;
}
```



Overlay scrollbars

For reference, this image shows the same div as above, but with the user's system settings set to overlay scrollbars. Note here the scrollbar will only show when the user is scrolling and on top of the content, so no space is reserved for it and the `scrollbar-gutter` property has no effect.



See also

- CSS overflow module
- CSS scrollbars styling module
- `overflow`
- `scrollbar-width`
- `scrollbar-color`

scrollbar-width CSS property

The **`scrollbar-width`** property allows the author to set the desired thickness of an element's scrollbars when they are shown.

The purpose of the `scrollbar-width` is to optimize the space occupied by the scrollbar on a page or element; the purpose is not related to scrollbar aesthetics. The `scrollbar-width` predefined keyword values indicate to the user agent whether a normal or smaller scrollbar should be rendered. Avoid using `none`, as hiding a scrollbar negatively impacts accessibility.

> **Note:** For elements that are scrollable only via programmatic means and not by direct user interaction, use the `overflow` property with a value of `hidden` rather than `scrollbar-width: none`.

> **Note:** Browsers ignore the non-standard `::-webkit-scrollbar` family of pseudo-elements on any element whose *computed* `scrollbar-width` value is anything other than `auto`. Unlike `scrollbar-color`, `scrollbar-width` is not inherited, so an ancestor's value doesn't affect its descendants. Setting `scrollbar-width: auto` on an element restores its `::-webkit-scrollbar-*` styling. See Adding a fallback for scrollbar styles.

Syntax

```
/* Keyword values */
scrollbar-width: auto;
scrollbar-width: thin;
scrollbar-width: none;

/* Global values */
scrollbar-width: inherit;
scrollbar-width: initial;
scrollbar-width: revert;
scrollbar-width: revert-layer;
scrollbar-width: unset;
```

Values

`auto`

The default scrollbar width for the platform.

`thin`

A thin scrollbar width variant on platforms that provide that option, or a thinner scrollbar than the default platform scrollbar width.

`none`

No scrollbar shown, however the element will still be scrollable.

> **Note:** User Agents must apply any `scrollbar-width` value set on the root element to the viewport.

Accessibility

Use this property with caution — setting `scrollbar-width` to `thin` or `none` can make content hard or impossible to scroll if the author does not provide an alternative scrolling mechanism. While swiping gestures or mouse wheels can enable scrolling on such content, some devices have no scroll alternative.

WCAG criterion 2.1.1 (Keyboard) has been in place for a long time to advise on basic keyboard accessibility, and this should include scrolling of content areas. And introduced in WCAG 2.1, criterion 2.5.5 (Target Size) advises that touch targets should be at least 44px in width and height (although the problem is compounded on high-resolution screens; thorough testing is advised).

- MDN Understanding WCAG, Guideline 2.1 explanations
- MDN Understanding WCAG, Guideline 2.5 explanations
- Understanding Success Criterion 2.1.1 | W3C Understanding WCAG 2.1
- Understanding Success Criterion 2.5.5 | W3C Understanding WCAG 2.1

Formal definition

Initial value	`auto`
Applies to	scrolling boxes
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
scrollbar-width =
  auto  |
  thin  |
  none
```

This syntax reflects the latest standard as per CSS Scrollbars Styling Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Sizing overflow scrollbars

CSS

```
.scroller {
  width: 300px;
  height: 100px;
  overflow-y: scroll;
  scrollbar-width: thin;
}
```

HTML

```
<div class="scroller">
  Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
  daikon amaranth tatsoi tomatillo melon azuki bean garlic. Gumbo beet greens
  corn soko endive gumbo gourd. Parsley shallot courgette tatsoi pea sprouts
  fava bean collard greens dandelion okra wakame tomato. Dandelion cucumber
  earthnut pea peanut soko zucchini.
</div>
```

Result

See also

- CSS overflow module
- CSS scrollbars styling module
- `overflow`
- `scrollbar-gutter`
- `scrollbar-color`

shape-image-threshold CSS property

The **`shape-image-threshold`** CSS property sets the alpha channel threshold used to extract the shape using an image as the value for `shape-outside`.

Any pixels whose alpha component's value is greater than the threshold are considered to be part of the shape for the purposes of determining its boundaries. For example, a value of `0.5` means that the shape will enclose all the pixels that are more than 50% opaque.

Try it

```
shape-outside: linear-gradient(
  50deg,
  rgb(77 26 103),
  transparent 80%,
  transparent
);
shape-image-threshold: 0.2;
```

```
shape-outside: linear-gradient(
  50deg,
  rgb(77 26 103),
  transparent 80%,
  transparent
);
shape-image-threshold: 0.4;
```

```
shape-outside: linear-gradient(
  50deg,
  rgb(77 26 103),
  transparent 80%,
  transparent
);
shape-image-threshold: 0.6;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element"></div>
    We had agreed, my companion and I, that I should call for him at his house,
    after dinner, not later than eleven o'clock. This athletic young Frenchman
    belongs to a small set of Parisian sportsmen, who have taken up "ballooning"
    as a pastime. After having exhausted all the sensations that are to be found
    in ordinary sports, even those of "automobiling" at a breakneck speed, the
    members of the "Aéro Club" now seek in the air, where they indulge in all
    kinds of daring feats, the nerve-racking excitement that they have ceased to
    find on earth.
  </div>
</section>
```

```
.example-container {
  text-align: left;
  padding: 20px;
}

#example-element {
  float: left;
  width: 150px;
  height: 150px;
  margin: 20px;
  background-image: linear-gradient(
    50deg,
    rgb(77 26 103),
    transparent 80%,
    transparent
  );
}
```

Syntax

```
/* <number> value */
shape-image-threshold: 0.7;

/* Global values */
shape-image-threshold: inherit;
shape-image-threshold: initial;
shape-image-threshold: revert;
shape-image-threshold: revert-layer;
shape-image-threshold: unset;
```

Values

`<alpha-value>`

> Sets the threshold used for extracting a shape from an image. The shape is defined by the pixels whose alpha value is greater than the threshold. Values outside the range 0.0 (fully transparent) to 1.0 (fully opaque) are clamped to this range.

Formal definition

Initial value	`0.0`
Applies to	floats
Inherited	no
Computed value	The same as the specified value after clipping the `<number>` to the range [0.0, 1.0].
Animation type	a number

Formal syntax

```
shape-image-threshold =
  <opacity-value>
```

```
<opacity-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Shapes Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Aligning text to a gradient

This example creates a `<div>` block with a gradient background image. The gradient is established as a CSS shape using `shape-outside`, so that pixels within the gradient which are at least 20% opaque (that is, those pixels with an alpha component greater than 0.2) are considered part of the shape.

HTML

```
<div id="gradient-shape"></div>

<p>
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. Vel at commodi
  voluptates enim, distinctio officia. Saepe optio accusamus doloribus sint
  facilis itaque ab nulla, dolor molestiae assumenda cum sit placeat adipisci,
  libero quae nihil porro debitis laboriosam inventore animi impedit nostrum
  nesciunt quisquam expedita! Dolores consectetur iure atque a mollitia dicta
  repudiandae illum exercitationem aliquam repellendus ipsum porro modi, id nemo
  eligendi, architecto ratione quibusdam iusto nisi soluta? Totam inventore ea
  eum sed velit et eligendi suscipit accusamus iusto dolore, at provident eius
  alias maxime pariatur non deleniti ipsum sequi rem eveniet laboriosam magni
  expedita?
</p>
```

CSS

```
#gradient-shape {
  width: 150px;
  height: 150px;
  float: left;
  background-image: linear-gradient(30deg, black, transparent 80%, transparent);
  shape-outside: linear-gradient(30deg, black, transparent 80%, transparent);
  shape-image-threshold: 0.2;
}
```

The shape is established here using `background-image` with a linear gradient rather than an image file. The same gradient is also used as the image from which the shape is derived for establishing the float area, using the `shape-outside` property.

The 20% opacity threshold for treating gradient pixels as part of the shape is then established using `shape-image-threshold` with a value of `0.2`.

Result

See also

- CSS Shapes
- Overview of CSS Shapes
- `<basic-shape>`
- `shape-outside`
- `shape-margin`

shape-margin CSS property

The **`shape-margin`** CSS property sets a margin for a CSS shape created using `shape-outside`.

The margin lets you adjust the distance between the edges of the shape (the **float element**) and the surrounding content.

Try it

```
shape-margin: 0;
```

```
shape-margin: 20px;
```

```
shape-margin: 1em;
```

```
shape-margin: 5%;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element"></div>
    We had agreed, my companion and I, that I should call for him at his house,
    after dinner, not later than eleven o'clock. This athletic young Frenchman
    belongs to a small set of Parisian sportsmen, who have taken up "ballooning"
    as a pastime. After having exhausted all the sensations that are to be found
    in ordinary sports, even those of "automobiling" at a breakneck speed, the
    members of the "Aéro Club" now seek in the air, where they indulge in all
    kinds of daring feats, the nerve-racking excitement that they have ceased to
    find on earth.
  </div>
</section>
```

```
.example-container {
  text-align: left;
  padding: 20px;
}

#example-element {
  float: left;
  margin: 20px;
  width: 180px;
  height: 180px;
  border-radius: 50%;
  background-color: rebeccapurple;
```

```
  shape-outside: circle(50%);
}
```

Syntax

```
/* <length> values */
shape-margin: 10px;
shape-margin: 20mm;

/* <percentage> value */
shape-margin: 60%;

/* Global values */
shape-margin: inherit;
shape-margin: initial;
shape-margin: revert;
shape-margin: revert-layer;
shape-margin: unset;
```

Values

`<length-percentage>`

Sets the margin of the shape to a `<length>` value or to a `<percentage>` of the width of the element's containing block.

Formal definition

Initial value	`0`
Applies to	floats
Inherited	no
Percentages	refer to the width of the containing block
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a length, percentage or calc();

Formal syntax

```
shape-margin =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Shapes Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adding a margin to a polygon

HTML

```
<section>
  <div class="shape"></div>
  We are not quite sure of any one thing in biology; our knowledge of geology is
  relatively very slight, and the economic laws of society are uncertain to
  every one except some individual who attempts to set them forth; but before
  the world was fashioned the square on the hypotenuse was equal to the sum of
  the squares on the other two sides of a right triangle, and it will be so
  after this world is dead; and the inhabitant of Mars, if one exists, probably
  knows its truth as we know it.
</section>
```

CSS

```
section {
  max-width: 400px;
}

.shape {
  float: left;
  width: 150px;
  height: 150px;
  background-color: maroon;
  clip-path: polygon(0 0, 150px 150px, 0 150px);
  shape-outside: polygon(0 0, 150px 150px, 0 150px);
  shape-margin: 20px;
}
```

Result

See also

- CSS Shapes
- Overview of CSS Shapes
- `shape-outside`
- `shape-image-threshold`
- `<basic-shape>`

shape-outside CSS property

The **`shape-outside`** CSS property defines a shape—which may be non-rectangular—around which adjacent inline content should wrap. By default, inline content wraps around its margin box; `shape-outside` provides a way to customize this wrapping, making it possible to wrap text around complex objects rather than rectangular boxes.

Try it

```
shape-outside: circle(50%);
```

```
shape-outside: ellipse(130px 140px at 20% 20%);
```

```
shape-outside: url("/shared-assets/images/examples/round-balloon.png");
```

```
shape-outside: polygon(50% 0, 100% 50%, 50% 100%, 0 50%);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <img
      class="transition-all"
      id="example-element"
      src="/shared-assets/images/examples/round-balloon.png"
      width="150" />
    We had agreed, my companion and I, that I should call for him at his house,
    after dinner, not later than eleven o'clock. This athletic young Frenchman
    belongs to a small set of Parisian sportsmen, who have taken up "ballooning"
    as a pastime. After having exhausted all the sensations that are to be found
    in ordinary sports, even those of "automobiling" at a breakneck speed, the
    members of the "Aéro Club" now seek in the air, where they indulge in all
    kinds of daring feats, the nerve-racking excitement that they have ceased to
    find on earth.
  </div>
</section>
```

```
.example-container {
  text-align: left;
  padding: 20px;
}

#example-element {
  float: left;
  width: 150px;
  margin: 20px;
}
```

Syntax

```
/* Keyword values */
shape-outside: none;
shape-outside: margin-box;
shape-outside: content-box;
shape-outside: border-box;
shape-outside: padding-box;

/* Function values */
shape-outside: circle();
shape-outside: ellipse();
shape-outside: inset(10px 10px 10px 10px);
shape-outside: polygon(10px 10px, 20px 20px, 30px 30px);

/* Shape box with basic shape */
shape-outside: circle() border-box;
shape-outside: margin-box ellipse();

/* <url> value */
shape-outside: url("image.png");

/* <gradient> value */
shape-outside: linear-gradient(45deg, white 150px, red 150px);

/* Global values */
shape-outside: inherit;
shape-outside: initial;
shape-outside: revert;
shape-outside: revert-layer;
shape-outside: unset;
```

The `shape-outside` property is specified using the values from the list below, which define the *float area* for *float elements*. The float area determines the shape around which inline content (float elements) wrap.

Values

`none`

The float area is unaffected. Inline content wraps around the element's margin box, like usual.

`<shape-box>`

The float area is computed according to the shape of a float element's edges (as defined by the CSS box model). This can be `margin-box`, `border-box`, `padding-box`, or `content-box`. The shape includes any curvature created by the `border-radius` property (behavior which is similar to `background-clip`).

`margin-box`

Defines the shape enclosed by the outside margin edge. The corner radii of this shape are determined by the corresponding `border-radius` and `margin` values. If the `border-radius / margin` ratio is `1` or more, then the margin box corner radius is `border-radius + margin`. If the ratio is less than `1`, then the margin box corner radius is `border-radius + (margin * (1 + (ratio - 1) ^ 3))`.

`border-box`

Defines the shape enclosed by the outside border edge. The shape follows the normal border radius shaping rules for the outside of the border.

`padding-box`

Defines the shape enclosed by the outside padding edge. The shape follows the normal border radius shaping rules for the inside of the border.

`content-box`

Defines the shape enclosed by the outside content edge. Each corner radius of this box is the larger of `0` or `border-radius - border-width - padding`.

`<basic-shape>`

The float area is computed based on the shape created by an `inset()`, `circle()`, `ellipse()`, or `polygon()` function; other `<basic-shape>` functions are invalid. If a `<shape-box>` is also supplied, it defines the reference box for the `<basic-shape>` function. Otherwise, the reference box defaults to `margin-box`.

`<image>`

The float area is extracted and computed based on the alpha channel of the specified `<image>` as defined by `shape-image-threshold`.

> **Note:** If the image is invalid, the effect is as if the keyword `none` had been specified. Additionally, the image must be served with CORS headers allowing its use.

Formal definition

Initial value	none
Applies to	floats
Inherited	no
Computed value	as defined for `<basic-shape>` (with `shape-box` following, if supplied), the `<image>` with its URI made absolute, otherwise as specified.
Animation type	yes, as specified for `<basic-shape>`, otherwise no

Formal syntax

```
shape-outside =
  none                              |
  [ <basic-shape> || <shape-box> ]  |
  <image>

<basic-shape> =
  <basic-shape-rect>  |
  <circle()>          |
  <ellipse()>         |
  <polygon()>         |
  <path()>            |
  <shape()>

<shape-box> =
  <visual-box>     |
  margin-box       |
  half-border-box

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<basic-shape-rect> =
  <inset()>  |
  <rect()>   |
  <xywh()>

<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<path()> =
  path( <'fill-rule'>? , <string> )

<shape()> =
  shape( <'fill-rule'>? from <position> , <shape-command># )

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>
```

```
<fill-rule> =
  nonzero  |
  evenodd

<length-percentage> =
  <length>      |
  <percentage>

<shape-command> =
  <move-command>             |
  <line-command>             |
  close                      |
  <horizontal-line-command>  |
  <vertical-line-command>    |
  <curve-command>            |
  <smooth-command>           |
  <arc-command>

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<move-command> =
  move <command-end-point>

<line-command> =
  line <command-end-point>

<horizontal-line-command> =
  hline [ to [ <length-percentage> | left | center | right | x-start | x-end ] | by <length-percentage> ]

<vertical-line-command> =
  vline [ to [ <length-percentage> | top | center | bottom | y-start | y-end ] | by <length-percentage> ]

<curve-command> =
  curve [ [ to <position> with <control-point> [ / <control-point> ]? ] | [ by <coordinate-pair> with <relative-control-point> [ / <relative-control-point> ]? ] ]

<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
```

```
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Box Model Module Level 4, CSS Fill and Stroke Module Level 3, CSS Images Module Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Funneling text

HTML

```
<div class="main">
  <div class="left"></div>
  <div class="right"></div>
  <p>
    Sometimes a web page's text content appears to be funneling your attention
    towards a spot on the page to drive you to follow a particular link.
    Sometimes you don't notice.
  </p>
</div>
```

CSS

```
.main {
  width: 530px;
}

.left,
.right {
  background-color: lightgray;
  height: 12ex;
  width: 40%;
}

.left {
  clip-path: polygon(0 0, 100% 100%, 0 100%);
  float: left;
  shape-outside: polygon(0 0, 100% 100%, 0 100%);
}

.right {
  clip-path: polygon(100% 0, 100% 100%, 0 100%);
  float: right;
  shape-outside: polygon(100% 0, 100% 100%, 0 100%);
}

p {
  text-align: center;
}
```

Result

See also

- CSS shapes
- Overview of shapes
- `<basic-shape>`
- `shape-margin`
- `shape-image-threshold`

shape-rendering CSS property

The **`shape-rendering`** CSS property provides hints to the renderer about what tradeoffs to make when rendering shapes like paths, circles, or rectangles. It only has an effect on the `<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements. If explicitly declared, the value of the CSS property overrides the any values of the element's `shape-rendering` attribute.

Syntax

```
shape-rendering: auto;
shape-rendering: crispEdges;
shape-rendering: geometricPrecision;
shape-rendering: optimizeSpeed;

/* Global values */
shape-rendering: inherit;
shape-rendering: initial;
shape-rendering: revert;
shape-rendering: revert-layer;
shape-rendering: unset;
```

Values

The `<length>` and `<percentage>` values denote the horizontal center of the circle or ellipse.

`auto`

> This value directs the user agents to make tradeoffs in order to balance speed, edge crispness, and geometric precision, with geometric precision given more importance than speed and edge crispness.

`crispEdges`

> This value directs the user agent to emphasize edge contrast over geometric precision or rendering speed. The final rendering is likely to skip techniques such as anti-aliasing. It may also adjust line positions and line widths in order to align edges with device pixels.

`geometricPrecision`

> This value directs the user agent to emphasize geometric precision over speed or crisp edges. The final rendering may involve techniques such as anti-aliasing.

`optimizeSpeed`

This value directs the user agent to emphasize rendering speed over geometric precision or edge crispness. The final rendering is likely to skip techniques such as anti-aliasing.

Formal definition

Initial value	`auto`
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
shape-rendering =
  auto               |
  optimizeSpeed      |
  crispEdges         |
  geometricPrecision
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

To show the different renderings, we create a set of four ellipses of equal size and shape.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 400 120">
  <ellipse cx="50" cy="60" rx="40" ry="60" />
  <ellipse cx="150" cy="60" rx="40" ry="60" />
  <ellipse cx="250" cy="60" rx="40" ry="60" />
  <ellipse cx="350" cy="60" rx="40" ry="60" />
</svg>
```

We then apply the four values of `shape-rendering`, one per ellipse.

```
ellipse:nth-of-type(1) {
  shape-rendering: crispEdges;
}
ellipse:nth-of-type(2) {
  shape-rendering: geometricPrecision;
}
ellipse:nth-of-type(3) {
  shape-rendering: optimizeSpeed;
}
ellipse:nth-of-type(4) {
  shape-rendering: auto;
}
```

The resulting SVG is shown here. The first and third ellipses (counting from left to right) are more likely to show jagged edges, whereas the second should have a smoother appearance. The fourth and last ellipse's appearance will be dictated by the specific tradeoffs made by the user agent you use to view the example.

See also

- SVG `shape-rendering` attribute

speak-as CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`speak-as`** CSS property is used to define how HTML content is spoken. The one to three enumerated key terms determine the manner by which elements and text get rendered by aural technologies, such as screen readers and digital assistants.

This property applies to all content, including pseudo-elements, with the exception `::marker` pseudo-elements constructed via a `@counter-style` with a defined `speak-as` descriptor, which takes precedence over any inherited `speak-as` property value.

Syntax

```
/* single value syntax */
speak-as: normal;
speak-as: spell-out;
speak-as: literal-punctuation;
speak-as: digits;
speak-as: no-punctuation;

/* multiple value syntax */
speak-as: spell-out literal-punctuation;
speak-as: spell-out no-punctuation;
speak-as: digits literal-punctuation;
speak-as: digits no-punctuation;
speak-as: spell-out digits literal-punctuation;
speak-as: spell-out digits no-punctuation;
```

Values

`normal`

Normal pronunciation rules with punctuation replaced by pauses. For example, "Hello, world!" would be pronounced as "Hello (pause) world (pause)". This is the default value.

`spell-out`

Content is spelled out letter by letter. For example, "role" would be pronounced as "r" "o" "l" "e".

`literal-punctuation`

Punctuation marks are spelled out literally. For example, "Hello, world!" would be pronounced as "Hello comma world exclamation mark."

`digits`

Numbers are pronounced as individual digits. For example, "31" would be pronounced as "three one".

`no-punctuation`

Content is pronounced normally without any punctuation. For example, "Hello, world!" would be pronounced as "Hello" "world".

> **Note:** Support of the `speak-as` property is limited and inconsistently implemented across different assistive technologies, such as screen readers or speech synthesizers. To ensure any pronunciation-dependent critical information remains user-friendly and accessible to a wide audience, do not rely solely on this CSS property to define how this information content is presented aurally.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	specified value
Animation type	discrete

Formal syntax

```
speak-as =
  normal                                          |
  spell-out || digits || [ literal-punctuation | no-punctuation ]
```

This syntax reflects the latest standard as per CSS Speech Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p class="normal">Hello, world! I'm 25.</p>
<p class="spell-out">Hello, world! I'm 25.</p>
<p class="literal-punctuation">Hello, world! I'm 25.</p>
<p class="no-punctuation">Hello, world! I'm 25.</p>
<p class="digits">Hello, world! I'm 25.</p>
<p class="multi">Hello, world! I'm 25.</p>
```

CSS

```
.normal {
  speak-as: normal;
}

.spell-out {
  speak-as: spell-out;
}

.literal-punctuation {
  speak-as: literal-punctuation;
}

.no-punctuation {
  speak-as: no-punctuation;
}

.digits {
  speak-as: digits;
}
.multi {
  speak-as: literal-punctuation digits;
}
```

See also

- `@counter-style` at-rule `speak-as` descriptor
- CSS counter styles module
- CSS lists and counters module
- Web Speech API

stop-color CSS property

The **`stop-color`** CSS property defines the color to use for an SVG `<stop>` element within a gradient. If present, it overrides the element's `stop-color` attribute.

> **Note:** The `stop-color` property only applies to `<stop>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* <color> values */
stop-color: red;
stop-color: hsl(120deg 75% 25% / 60%);
stop-color: currentColor;

/* Global values */
stop-color: inherit;
stop-color: initial;
stop-color: revert;
stop-color: revert-layer;
stop-color: unset;
```

Values

`<color>`

The color of the fill. This can be any valid CSS `<color>` value.

Formal definition

Initial value	`black`
Applies to	`<stop>` elements in `<svg>`
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
stop-color =
  <color>
```

Examples

Defining the color stops of SVG gradients

This example demonstrates the basic use case of `stop-color`, and how the CSS `stop-color` property takes precedence over the `stop-color` attribute.

HTML

We have an SVG with three `<rect>` squares and three `<linearGradient>` elements. Each gradient has four `<stop>` elements creating gradients that go from black to white and then white to gray; the only difference between them is the `id` value.

```
<svg viewBox="0 0 264 100" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="myGradient1">
      <stop offset="25%" stop-color="black" />
      <stop offset="40%" stop-color="white" />
      <stop offset="60%" stop-color="white" />
      <stop offset="75%" stop-color="#333333" />
    </linearGradient>
    <linearGradient id="myGradient2">
      <stop offset="25%" stop-color="black" />
      <stop offset="40%" stop-color="white" />
      <stop offset="60%" stop-color="white" />
      <stop offset="75%" stop-color="#333333" />
    </linearGradient>
    <linearGradient id="myGradient3">
      <stop offset="25%" stop-color="black" />
      <stop offset="40%" stop-color="white" />
      <stop offset="60%" stop-color="white" />
      <stop offset="75%" stop-color="#333333" />
    </linearGradient>
  </defs>
  <rect x="2" y="10" width="80" height="80" fill="url('#myGradient1')" />
  <rect x="92" y="10" width="80" height="80" fill="url('#myGradient2')" />
  <rect x="182" y="10" width="80" height="80" fill="url('#myGradient3')" />
</svg>
```

CSS

We include a `stroke` and `stroke-width` outlining the rectangle. We define the colors of the first and last stops in each gradient, overriding their `stop-color` attribute values, using the `stop-color` property. Various CSS `<color>` syntaxes are shown.

```
svg {
  border: 1px solid;
  height: calc(100vh - 20px);
  margin-bottom: 10px;
}
```

```
rect {
  stroke: #333333;
  stroke-width: 1px;
}

#myGradient1 {
  stop:first-of-type {
    stop-color: #66ccff;
  }
  stop:last-of-type {
    stop-color: #f4aab9;
  }
}
#myGradient2 {
  stop:first-of-type {
    stop-color: yellow;
  }
  stop:last-of-type {
    stop-color: purple;
  }
}
#myGradient3 {
  stop:first-of-type {
    stop-color: hsl(0deg 90% 50%);
  }
  stop:last-of-type {
    stop-color: hsl(20deg 60% 50%);
  }
}
```

Results

See also

- SVG `stop-color` attribute
- Presentation properties: `stop-color`, `clip-rule`, `color-interpolation-filters`, `fill-opacity`, `fill-rule`, `fill`, `marker-end`, `marker-mid`, `marker-start`, `shape-rendering`, `stop-opacity`, `stroke`, `stroke-dasharray`, `stroke-dashoffset`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `stroke-opacity`, `stroke-width`, `text-anchor`, and `vector-effect`
- `opacity`
- `background-color`
- `<color>`
- `<basic-shape>` data type

stop-opacity CSS property

The **`stop-opacity`** CSS property defines the opacity of a given color gradient stop in the SVG `<stop>` element within an SVG gradient. If present, it overrides the element's `stop-opacity` attribute.

The property value impacts the `stop-color`'s alpha channel; it can increase the transparency of a `<stop>`'s color but cannot make the color defined by the `stop-color` property more opaque.

> **Note:** The `stop-opacity` property only applies to `<stop>` elements nested in an `<svg>`. It doesn't apply to other SVG, HTML, or pseudo-elements.

Syntax

```
/* numeric and percentage values */
stop-opacity: 0.2;
stop-opacity: 20%;

/* Global values */
stop-opacity: inherit;
stop-opacity: initial;
stop-opacity: revert;
stop-opacity: revert-layer;
stop-opacity: unset;
```

Values

The `<opacity-value>` is a `<number>` or `<percentage>` denoting the opacity of the SVG gradient `<stop>` element.

`<number>`

A numeric value between `0` and `1`, inclusive.

`<percentage>`

A percentage value between `0%` and `100%`, inclusive.

With `0` or `0%` set, the stop is fully transparent. With `1` or `100%` set, the element is the full opacity of the `stop-color` value, which may or may not be partially opaque.

Formal definition

Initial value	`black`
Applies to	`<stop>` elements in `<svg>`
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
stop-opacity =
  <number>     |
  <percentage>
```

Examples

Defining the opacity of an SVG gradient color stop

This example demonstrates the basic use case of `stop-opacity`, and how the CSS `stop-opacity` property takes precedence over the `stop-opacity` attribute.

HTML

We have an SVG with a few `<polygon>` stars and three `<linearGradient>` elements: each has three `<stop>` elements defining three color-stops that create a gradient going from blue to white to pink; the only difference between them is the `id` value.

```
<svg viewBox="0 0 250 120" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="myGradient1">
      <stop offset="5%" stop-color="#66ccff" stop-opacity="1" />
      <stop offset="50%" stop-color="#fefefe" stop-opacity="1" />
      <stop offset="95%" stop-color="#f4aab9" stop-opacity="1" />
    </linearGradient>
    <linearGradient id="myGradient2">
      <stop offset="5%" stop-color="#66ccff" stop-opacity="1" />
      <stop offset="50%" stop-color="#fefefe" stop-opacity="1" />
      <stop offset="95%" stop-color="#f4aab9" stop-opacity="1" />
    </linearGradient>
    <linearGradient id="myGradient3">
      <stop offset="5%" stop-color="#66ccff" stop-opacity="1" />
      <stop offset="50%" stop-color="#fefefe" stop-opacity="1" />
      <stop offset="95%" stop-color="#f4aab9" stop-opacity="1" />
    </linearGradient>
  </defs>
  <polygon points="40,10 10,100 80,40 0,40 70,100" />
  <polygon points="125,10 95,100 165,40 85,40 155,100" />
  <polygon points="210,10 180,100 250,40 170,40 240,100" />
</svg>
```

CSS

We include a `stroke` and `stroke-width` making the polygon path line visible.

Each `polygon` has a gradient background set using the `fill` property; the gradient's `id` is the `url()` parameter. We set `magenta` as the fallback color.

We define the opacity of the stops of each gradient using the `stop-opacity` property.

The SVG has a striped background to make the transparency settings more apparent.

```
svg {
  border: 1px solid;
  height: calc(100vh - 20px);
  margin-bottom: 10px;
  background: repeating-linear-gradient(
    to top left,
    white 0 9px,
    black 9px 10px
  );
}
```

```
polygon {
  stroke: #333333;
  stroke-width: 1px;
}
polygon:nth-of-type(1) {
  fill: url("#myGradient1") magenta;
}
polygon:nth-of-type(2) {
  fill: url("#myGradient2") magenta;
}
polygon:nth-of-type(3) {
  fill: url("#myGradient3") magenta;
}

#myGradient1 stop {
  stop-opacity: 1;
}
#myGradient2 stop {
  stop-opacity: 0.8;
}
#myGradient3 stop {
  stop-opacity: 25%;
}
```

Results

The first star is fully opaque. The fill of the second star is 80% opaque because the color stops are slightly translucent; the `stop-opacity: 0.8;` overrode the `stop-opacity="1"` element attribute value. The fill of the last star is barely noticeable with color stops that are 25% opaque. Note the stroke is the same opaque dark gray in all cases.

> **Note:** Because we used the same `stop-opacity` value for all the sibling `<stop>` elements in the linear gradient, we could have instead used a single `<linearGradient>` with fully opaque stops, and set a value for each `<polygon>`s `fill-opacity` property instead.

See also

- SVG `stop-opacity` attribute
- Presentation properties: `stop-opacity`, `clip-rule`, `color-interpolation-filters`, `fill-opacity`, `fill-rule`, `fill`, `marker-end`, `marker-mid`, `marker-start`, `shape-rendering`, `stop-color`, `stroke`, `stroke-dasharray`, `stroke-dashoffset`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `stroke-opacity`, `stroke-width`, `text-anchor`, and `vector-effect`
- `opacity`
- `background-color`
- `<color>`
- `<basic-shape>` data type

stroke CSS property

The **`stroke`** CSS property defines the color or SVG paint server used to draw an element's stroke. As such, `stroke` only has an effect on elements that can be given a stroke (for example, `<rect>` or `<ellipse>`); see the page on the SVG `stroke` attribute for a complete list. When declared, the CSS value overrides any value of the element's `stroke` SVG attribute.

> **Note:** According to the 4 April 2017 draft of the CSS Fill and Stroke Module Level 3 specification, the `stroke` property is a shorthand for a number of other stroke properties. In practice, as of August 2024, browsers do not support the setting of other stroke-related values such as width or dash patterns via the `stroke` property, treating it instead as a direct analogue of the SVG `stroke` attribute.

Syntax

```
/* assorted color values */
stroke: rgb(153 51 102 / 1);
stroke: color-mix(in lch, var(--primaryColor) 35%, gray 15%);
stroke: dodgerblue;
stroke: currentColor;
stroke: transparent;
stroke: context-stroke;

/* Global values */
stroke: inherit;
stroke: initial;
stroke: revert;
stroke: revert-layer;
stroke: unset;
```

Values

`<color>`

Sets the painting of the stroke with any valid CSS color value.

`<image>`

Sets the painting of the stroke with what SVG calls a *paint server*, which in this context is an SVG gradient or pattern. CSS gradients cannot be used with the `stroke` property.

`context-stroke`

Causes an element to "inherit" its stroke definition from its *context element*. If there is no valid context element, then this value will result in no paint being used for the stroke.

Formal definition

Initial value	as each of the properties of the shorthand: • `stroke-dasharray`: `none` • `stroke-dashoffset`: `0` • `stroke-linecap`: `butt` • `stroke-linejoin`: `miter` • `stroke-miterlimit`: `4` • `stroke-opacity`: `1` • `stroke-width`: `1px`
Applies to	as each of the properties of the shorthand:
Inherited	yes
Computed value	as each of the properties of the shorthand:
Animation type	as each of the properties of the shorthand: • `stroke-dasharray`: a repeatable list • `stroke-dashoffset`: by computed value type • `stroke-linecap`: discrete • `stroke-linejoin`: discrete • `stroke-miterlimit`: by computed value type • `stroke-opacity`: by computed value type • `stroke-width`: by computed value type

Formal syntax

```
stroke =
  <paint>

<paint> =
  none         |
  <image>      |
  <svg-paint>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<svg-paint> =
  child              |
  child( <integer> )

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<integer> =
  <number-token>

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Specification, CSS Values and Units Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic color stroking

In this example, we create two different shapes, a circle and a rectangle, which are part of a `<g>` (group) that has a gray stroke color as a default for the two shapes.

```
<svg>
  <g fill="none" stroke="gray" stroke-width="10">
    <circle cx="100" cy="100" r="40" />
    <rect x="20" y="20" width="80" height="80" />
```

```
  </g>
</svg>
```

Via CSS, we then apply a dusky purple color to the rectangle and a red to the circle.

```
rect {
  stroke: rebeccapurple;
}
circle {
  stroke: red;
}
```

Pattern stroking

This example uses the same group and shapes (with the circle moved over a bit) as seen in the previous example, but also has an SVG pattern defined.

```
<svg>
  <g fill="none" stroke="gray" stroke-width="23">
    <circle cx="150" cy="90" r="40" />
    <rect x="20" y="20" width="80" height="80" />
  </g>
  <defs>
    <pattern id="star" viewBox="0,0,10,10" width="10%" height="10%">
      <polygon points="0,0 2,5 0,10 5,8 10,10 8,5 10,0 5,2" />
    </pattern>
  </defs>
</svg>
```

The pattern is then referenced in the CSS with a URL value pointing to the ID of the pattern. This pattern is applied to both shapes as a stroking paint, with the result shown.

```
rect,
circle {
  stroke: url("#star");
}
```

The pattern is drawn relative to the shapes' bounding boxes, which can lead to visual interference where they overlap, because parts of the pattern are transparent.

SVG versus CSS gradients

Here, we once again use the same group-and-shapes markup as the first example, but also add a definition for an SVG gradient.

```
<svg>
  <g fill="none" stroke="gray" stroke-width="10">
    <circle cx="100" cy="100" r="40" />
    <rect x="20" y="20" width="80" height="80" />
  </g>
  <defs>
    <linearGradient id="greenwhite">
      <stop offset="0%" stop-color="green" />
      <stop offset="100%" stop-color="white" />
    </linearGradient>
  </defs>
</svg>
```

In the CSS, we apply that SVG gradient to the rectangle using a URL value pointing to the ID of the linear gradient. To the circle, we apply a CSS linear gradient that is equivalent in intent to the SVG gradient.

```
rect {
  stroke: url("#greenwhite");
}
circle {
  stroke: linear-gradient(90deg, green, white);
}
```

Only the rectangle receives a gradient stroke, whereas the circle falls back to the gray stroke set by the group element. This happens because CSS gradients are not valid values for the `stroke` property, whereas URL references to SVG gradients are permitted.

Contextual stroking

In this case, we again start with a group element, inside of which two rectangle-shaped paths are defined. After that, a linear gradient and an SVG marker are defined.

```
<svg>
  <g fill="none" stroke="gray" stroke-width="4">
    <path d="M 20,20 l 180,0 0,100 -180,0 z" />
    <path d="M 100,40 l 180,0 0,100 -180,0 z" />
  </g>
  <defs>
    <linearGradient id="orangered">
      <stop offset="0%" stop-color="orange" />
      <stop offset="100%" stop-color="red" />
    </linearGradient>
    <marker
      id="circle"
      markerWidth="20"
      markerHeight="20"
      refX="10"
      refY="10"
      markerUnits="userSpaceOnUse">
      <circle
        cx="10"
        cy="10"
        r="8"
        stroke-width="4"
        stroke="none"
        fill="none" />
    </marker>
  </defs>
</svg>
```

We then write CSS to add a marker to both paths, and also to have a dusky purple stroke color. This is overridden for the second path, which is given a URL value to apply the orange-to-red gradient as its stroke. Finally, we set the circle element in the marker element to have a stroke value of `context-stroke`.

```
path {
  stroke: rebeccapurple;
  marker: url("#circle");
}
path:nth-of-type(2) {
  stroke: url("#orangered");
}
marker circle {
  stroke: context-stroke;
}
```

Because `stroke-context` is being applied to an element that descends from a `<marker>` element, the context element for every circle is the element that called the `<marker>` element; that is, the `<path>` elements. The result is that the markers on the first path use the color of the calling path, and are purple. The markers on the second path, by contrast, use the same orange-to-red SVG gradient the path uses. This latter case illustrates how SVG gradients are applied as a single gradient to the entire shape, rather than being applied independently to each individual part of the shape.

See also

- `stroke-dasharray`
- `stroke-dashoffset`
- `stroke-linecap`
- `stroke-linejoin`
- `stroke-miterlimit`
- `stroke-opacity`
- `stroke-width`
- `paint-order`
- SVG `stroke` attribute

stroke-dasharray CSS property

The **`stroke-dasharray`** CSS property defines a pattern of dashes and gaps used in the painting of the SVG shape's stroke. If present, it overrides the element's `stroke-dasharray` attribute.

This property applies to any SVG shape or text-content element (see `stroke-dasharray` for a full list), but as an inherited property, it may be applied to elements such as `<g>` and still have the intended effect on descendant elements' strokes.

Syntax

```
/* Keyword value */
stroke-dasharray: none;

/* Numeric, length, and percentage values */
stroke-dasharray: 2px, 5px;
stroke-dasharray: 20%, 50%;
stroke-dasharray: 2, 5;

/* The following two rules are equivalent */
stroke-dasharray: 2, 5, 3;
stroke-dasharray: 2, 5, 3, 2, 5, 3;

/* Global values */
stroke-dasharray: inherit;
stroke-dasharray: initial;
stroke-dasharray: revert;
stroke-dasharray: revert-layer;
stroke-dasharray: unset;
```

Values

The value is a list of comma and/or white space separated `<number>`, `<length>`, and / or `<percentage>` values that specify the lengths of alternating dashes and gaps, or the keyword `none`. If an odd number of values are given, the entire value will be repeated in order to set an even number of values.

`none`

The stroke will be drawn without any dashes. The default value.

`<number>`

A number of SVG units, the size of which are defined by the current unit space. Negative values are invalid.

`<length>`

Pixel units are handled the same as SVG units (see `<number>`, above) and font-based lengths such as `em` are calculated with respect to the element's SVG value for the text size; the effects of other length units may depend on the browser. Negative values are invalid.

`<percentage>`

Percentages refer to the normalized diagonal of the current SVG viewport, which is calculated as $\frac{\sqrt{\text{<width>}^2 + \text{<height>}^2}}{\sqrt{2}}$. Negative values are invalid.

Formal definition

Initial value	none
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Percentages	refer to the normalized diagonal measure of the current SVG viewport's applied `viewBox`, or of the viewport itself if no `viewBox` is specified
Computed value	A comma separated list of absolute lengths or percentages, numbers converted to absolute lengths first, or keyword specified
Animation type	a repeatable list

Formal syntax

```
stroke-dasharray =
  none                                |
  [ <length-percentage> | <number> ]+#
```

```
<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic dash array

This example demonstrates basic usage of the `stroke-dasharray` property using space-separated `<number>` values.

HTML

First, we set up a basic SVG rectangle shape. To this rectangle, a red stroke with a width of 2 is applied.

```
<svg viewBox="0 0 100 50" width="500" height="250">
  <rect
    x="10"
    y="10"
    width="80"
    height="30"
    fill="none"
    stroke="red"
    stroke-width="2" />
</svg>
```

CSS

We define a dash pattern for the stroke: ten units of dash, followed by five units of space. This means the gaps between dashes will be half the length as the dashes themselves.

```
rect {
  stroke-dasharray: 10 5;
}
```

Results

Where the stroke turns a corner, the pattern is carried along, as it were. At the top left corner, where the start and end of the stroke meet, the ten-unit-long starting dash appears to join with the part of the dash pattern seen at the end of the path, creating what looks like a longer-than-ten-units line bending around the corner.

Dash array repetition

This example includes an odd-number of comma-separated `<number>` values to demonstrates how the value is repeated if an odd number of values is given in order to set an even number of values.

HTML

In this case, we define two rectangles.

```
<svg viewBox="0 0 100 100" width="500" height="500">
  <rect
    x="10"
    y="10"
    width="80"
    height="30"
    fill="none"
    stroke="red"
    stroke-width="2" />
  <rect
    x="10"
    y="60"
    width="80"
    height="30"
    fill="none"
    stroke="red"
    stroke-width="2" />
</svg>
```

CSS

To the first rectangle, we define a dasharray of `5, 5, 1`, which calls for five units of dash, five of gap, and one unit of dash. However, because this is an odd number of numbers, the entire set of numbers is repeated, thus creating a value identical to that applied to the second rectangle.

```
rect:nth-of-type(1) {
  stroke-dasharray: 5, 5, 1;
}
rect:nth-of-type(2) {
  stroke-dasharray: 5, 5, 1, 5, 5, 1;
}
```

Result

The reason an even count of numbers is required is so that every dash array begins with a dash and ends with a gap. Thus, the pattern defined is a five-unit dash, a five-unit gap, a one-unit dash, a five-unit gap, a five-unit dash, and a one-unit gap. In the resulting stroke, every instance of a one-unit gap between two five-unit dashes indicates a place where the dash array starts over.

Percentage and pixel values

This example demonstrates the use of `<percentage>` and `<length>` values within the `stroke-dasharray` property value.

HTML

As in the previous example, we define two rectangles.

```
<svg viewBox="0 0 100 100" width="500" height="500">
  <rect
    x="10"
    y="10"
    width="80"
    height="30"
```

```
    fill="none"
    stroke="red"
    stroke-width="2" />
  <rect
    x="10"
    y="60"
    width="80"
    height="30"
    fill="none"
    stroke="red"
    stroke-width="2" />
</svg>
```

CSS

This time, rather than collections of bare numbers, we use pixel units and percentages.

```
rect:nth-of-type(1) {
  stroke-dasharray: 5px, 5px, 1px;
}
rect:nth-of-type(2) {
  stroke-dasharray: 5%, 5%, 1%;
}
```

Results

The results are essentially indistinguishable from the results in the previous example.

See also

- `stroke-dashoffset`
- `stroke-linecap`
- `stroke-linejoin`
- `stroke-miterlimit`
- `stroke-opacity`
- `stroke-width`
- `stroke`
- SVG `stroke-dasharray` attribute

stroke-dashoffset CSS property

The **`stroke-dashoffset`** CSS property defines an offset for the starting point of the rendering of an SVG element's associated dash array. If present, it overrides the element's `stroke-dashoffset` attribute.

This property applies to any SVG shape or text-content element (see `stroke-dashoffset` for a full list), but as an inherited property, it may be applied to elements such as `<g>` and still have the intended effect on descendant elements' strokes.

Syntax

```
/* Keyword value */
stroke-dashoffset: none;

/* Length and percentage values */
stroke-dashoffset: 2;
stroke-dashoffset: 2px;
stroke-dashoffset: 2%;

/* Global values */
stroke-dashoffset: inherit;
stroke-dashoffset: initial;
stroke-dashoffset: revert;
stroke-dashoffset: revert-layer;
stroke-dashoffset: unset;
```

Values

`<number>`

> A number of SVG units, the size of which defined by the current unit space. The value given, if other than `0`, moves the starting point from the start of the dash array to another point within it. Thus, positive values will appear to shift the dash-gap pattern *backwards*, and negative values will appear to shift the pattern *forwards*.

`<length>`

> Pixel units are handled the same as SVG units (see `<number>`, above) and font-based lengths such as `em` are calculated with respect to the element's SVG value for the text size; the effects of other length units may depend on the browser. The shifting effect for any value is the same as for `<number>` values (see above).

`<percentage>`

> Percentages refer to the normalized diagonal of the current SVG viewport, which is calculated as $\frac{\sqrt{\text{<width>}^2 + \text{<height>}^2}}{\sqrt{2}}$, *not* to the overall length of the stroke path. Negative values are invalid.

Formal definition

Initial value	`0`
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Percentages	refer to the normalized diagonal measure of the current SVG viewport's applied `viewBox`, or of the viewport itself if no `viewBox` is specified
Computed value	an absolute `<length>` or `<percentage>`, numbers converted to absolute lengths first
Animation type	by computed value type

Formal syntax

```
stroke-dashoffset =
  <length-percentage>  |
  <number>
```

```
<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Dash offsetting

To show how dashes can be offset, we first set up five identical paths, all of which are given a dash array of a 20-unit dash followed by a 3-unit gap via the SVG attribute `stroke-dasharray`. (This could also have been done with the CSS property `stroke-dasharray`.) The paths are then given individual dash offsets via CSS.

```
<svg viewBox="0 0 100 50" width="500" height="250">
  <rect x="10" y="5" width="80" height="30" fill="#eeeeee" />
  <g stroke="dodgerblue" stroke-width="2" stroke-dasharray="20,3">
    <path d="M 10,10 h 80" />
    <path d="M 10,15 h 80" />
    <path d="M 10,20 h 80" />
    <path d="M 10,25 h 80" />
    <path d="M 10,30 h 80" />
  </g>
</svg>
```

```
path:nth-of-type(1) {
  stroke-dashoffset: 0;
}
path:nth-of-type(2) {
  stroke-dashoffset: -5;
}
path:nth-of-type(3) {
  stroke-dashoffset: 5;
}
path:nth-of-type(4) {
  stroke-dashoffset: 5px;
}
path:nth-of-type(5) {
  stroke-dashoffset: 5%;
}
```

In order:

1. The first of the five paths is given a zero offset, which is the default behavior.
2. The second path is given an offset of `-5`, which shifts the start point of the array to five units before the zero-point. The visual effect is that the dash pattern is pushed forward five units; thus, we see, at the start of the path, the last two units of a dash and then a three-unit gap.
3. The third path has an offset of `5`, which means the starting point of the dashes is five units into the dash pattern. The visual effect is to push the dash pattern backward by five units; thus, we see, at the start of the path, the last fifteen units of a dash followed by a three-unit gap.
4. The fourth path has an offset of `5px`, which has the same effect as a value of `5` (see previous).
5. The fifth and last path has an offset of `5%`, which is very similar to the previous two examples, but is not quite the same. Percentages are calculated against the diagonal measure of the SVG viewport, and so can very depending on that viewport's size and aspect ratio.

See also

- SVG `stroke-dashoffset` attribute
- CSS `stroke-dasharray` property
- CSS `stroke` property

stroke-linecap CSS property

The **`stroke-linecap`** CSS property defines the shape to be used at the end of open subpaths of SVG elements' unclosed strokes. If present, it overrides the element's `stroke-linecap` attribute.

This property applies to any SVG shape that can have unclosed strokes and text-content element (see `stroke-linecap` for a full list), but as an inherited property, it may be applied to elements such as `<g>` and still have the intended effect on descendant elements' strokes.

Syntax

```
/* keyword values */
stroke-linecap: butt;
stroke-linecap: round;
stroke-linecap: square;

/* Global values */
stroke-linecap: inherit;
stroke-linecap: initial;
stroke-linecap: revert;
stroke-linecap: revert-layer;
stroke-linecap: unset;
```

Values

This property is specified as one of the following keyword values:

`butt`

> Indicates that the stroke for each subpath does not extend beyond its two endpoints. On a zero-length subpath, the path will not be rendered at all. This is the default value.

`round`

> Indicates that at the end of each subpath the stroke will be extended by a half circle with a diameter equal to the stroke width. On a zero-length subpath, the stroke consists of a full circle centered at the subpath's point.

`square`

> Indicates that at the end of each subpath the stroke will be extended by a rectangle with a width equal to half the width of the stroke and a height equal to the width of the stroke. On a zero-length subpath, the stroke consists of a square with its width equal to the stroke width, centered at the subpath's point.

Formal definition

Initial value	`butt`
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
stroke-linecap =
  butt    |
  round   |
  square
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Linecaps

This example demonstrates the property's three keyword values.

HTML

We first set up a light-gray rectangle. Then, in a group, three paths are defined whose length is exactly the same as the width of the rectangle, and all of which start at the left edge of the rectangle. They are all set to have a `dodgerblue` stroke with a width of seven.

```
<svg viewBox="0 0 100 50" width="500" height="250">
  <rect x="10" y="5" width="80" height="30" fill="#dddddd" />
  <g stroke="dodgerblue" stroke-width="7">
    <path d="M 10,10 h 80" />
    <path d="M 10,20 h 80" />
    <path d="M 10,30 h 80" />
  </g>
</svg>
```

CSS

We then apply a different linecap style to each path via CSS.

```
path:nth-of-type(1) {
  stroke-linecap: butt;
}
path:nth-of-type(2) {
  stroke-linecap: square;
}
path:nth-of-type(3) {
  stroke-linecap: round;
}
```

Results

The first path has `butt` linecaps, which essentially means the stroke runs exactly to the end points (both the start and the end) of the path, and no further. The second path has `square` linecaps, so the visible path extends out past the end points of the path, making the overall length of the path appear to be 87, since the path length is 80 and each of the two square caps is 3.5 wide. The third path has `circle` caps, so while it also appears to be 87 units long, the two caps are semicircular instead of square.

See also

- `stroke-dasharray`
- `stroke-dashoffset`
- `stroke-linejoin`
- `stroke-miterlimit`
- `stroke-opacity`
- `stroke-width`
- `stroke`
- SVG `stroke-linecap` attribute

stroke-linejoin CSS property

The **`stroke-linejoin`** CSS property defines the shape to be used at the corners of an SVG element's stroked paths. If present, it overrides the element's `stroke-linejoin` attribute.

This property applies to any SVG corner-generating shape or text-content element (see `stroke-linejoin` for a full list), but as an inherited property, it may be applied to elements such as `<g>` and still have the intended effect on descendant elements'

Syntax

```
/* keyword values */
stroke-linejoin: bevel;
stroke-linejoin: miter;
stroke-linejoin: round;

/* Global values */
stroke-linejoin: inherit;
stroke-linejoin: initial;
stroke-linejoin: revert;
stroke-linejoin: revert-layer;
stroke-linejoin: unset;
```

Values

`bevel`

> Indicates that a beveled corner is to be used to join path segments. The bevel is formed by truncating the corner by a line perpendicular to a line that bisects the difference in the subpath angles where they meet the join point.

`miter`

Indicates that a sharp corner is to be used to join path segments. The corner is formed by extending the outer edges of the stroke at the tangents of the path segments until they intersect. This is the default value.

`round`

Indicates that a round corner is to be used to join path segments. This is accomplished by cropping the join as per `bevel`, and then appending a filled arc tangent in order to round the corner.

The following values are defined, but not supported in any browser:

`arcs`

(Unsupported.) Indicates that an *arcs corner* is to be used to join path segments. The arc's shape is formed by extending the outer edges of the stroke at the join point with arcs that have the same curvature as the outer edges at the join point.

`crop`

(Unsupported.) Indicates the corner should extend past the join point by the minimum amount necessary to form a convex corner. This is functionally equivalent to `miter` (see above) with a `stroke-miterlimit` value of `1`.

`fallback`

(Unsupported; at risk.) behaves identically to `crop bevel` when the `stroke-miterlimit` value is exceeded.

Formal definition

Initial value	`miter`
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
stroke-linejoin =
  [ crop | arcs | miter ]      ||
  [ bevel | round | fallback ]
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Line-joining styles

This example demonstrates the three currently supported keyword values for `stroke-linejoin`.

HTML

We set up four identical paths, all of which have a black stroke with a width of one and no fill.

```
<svg viewBox="0 0 15 12" xmlns="http://www.w3.org/2000/svg">
  <g stroke="black" stroke-width="1" fill="none">
    <path d="M2,5  a2,2 0,0,0 2,-3 a3,3 0 0 1 2,3.5" />
    <path d="M8,5  a2,2 0,0,0 2,-3 a3,3 0 0 1 2,3.5" />
    <path d="M2,11 a2,2 0,0,0 2,-3 a3,3 0 0 1 2,3.5" />
    <path d="M8,11 a2,2 0,0,0 2,-3 a3,3 0 0 1 2,3.5" />
  </g>
</svg>
```

CSS

To each of the four paths, a supported line-joining value is applied. The first is beveled, the second rounded, the third mitered, and the fourth also mitered but with a `stroke-miterlimit` of `2`, which forces the corner to be beveled instead of mitered.

```
path:nth-child(1) {
  stroke-linejoin: bevel;
}
path:nth-child(2) {
  stroke-linejoin: round;
}
path:nth-child(3) {
  stroke-linejoin: miter;
}
path:nth-child(4) {
  stroke-linejoin: miter;
  stroke-miterlimit: 2;
}
```

Results

See also

- `stroke-dasharray`
- `stroke-dashoffset`
- `stroke-linecap`
- `stroke-miterlimit`
- `stroke-opacity`
- `stroke-width`
- `stroke`
- SVG `stroke-linejoin` attribute

stroke-miterlimit CSS property

The **`stroke-miterlimit`** CSS property defines a limit on the ratio of the miter length to the `stroke-width` when the shape to be used at the corners of an SVG element's stroked path is a mitered join. If the limit defined by this property is exceeded, the join is converted from `miter` to `bevel`, thus making the corner appear truncated.

This property applies to any SVG corner-generating shape or text-content element (see `stroke-miterlimit` for a full list), but as an inherited property, it may be applied to elements such as `<g>` and still have the intended effect on descendant elements' strokes. If present, it overrides the element's `stroke-miterlimit` attribute.

Description

When two line segments meet at a sharp angle and `miter` joins have been specified for `stroke-linejoin`, or if they default to that value, it is possible for the miter to extend far beyond the thickness of the line stroking the path. The `stroke-miterlimit` ratio is used to define a limit, past which the join is converted from a miter to a bevel.

The ratio of miter length (the distance between the outer tip and the inner corner of the miter) to `stroke-width` is directly related to the angle (theta) between the segments in user space by the formula:

$$\text{stroke-miterlimit} = \frac{\text{miterLength}}{\text{stroke-width}} = \frac{1}{\sin\left(\frac{\theta}{2}\right)}$$

For example, a miter limit of `1.414` converts miters to bevels for a theta value of less than 90 degrees, a limit of `4.0` converts them for a theta less than approximately 29 degrees, and a limit of `10.0` converts them for theta less than approximately 11.5 degrees.

Syntax

```
/* number values */
stroke-miterlimit: 1;
stroke-miterlimit: 3.1416;

/* Global values */
stroke-miterlimit: inherit;
stroke-miterlimit: initial;
stroke-miterlimit: revert;
stroke-miterlimit: revert-layer;
stroke-miterlimit: unset;
```

Values

`<number>`

Any real positive number equal to or greater than `1`; values below that are invalid. The initial value is `4`.

Formal definition

Initial value	`4`
Applies to	`<circle>`, `<ellipse>`, `<line>`, `<path>`, `<polygon>`, `<polyline>`, and `<rect>` elements in an `svg`
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
stroke-miterlimit =
  <number>
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Various miter limits

This example demonstrates the effect of different values for the `stroke-miterlimit` property.

HTML

We set up five multi-segment paths, all of which use a black stroke with a width of one, and no fill. Each path creates a series of mountain symbols, going from left (a shallow corner angle) to right (an extreme corner angle).

```
<svg viewBox="0 0 39 36" xmlns="http://www.w3.org/2000/svg">
  <g stroke="black" stroke-width="1" fill="none">
    <path
      d="M1,5 l7   ,-3 l7   ,3
         m2,0 l3.5 ,-3 l3.5 ,3
         m2,0 l2   ,-3 l2   ,3
         m2,0 l0.75,-3 l0.75,3
         m2,0 l0.5 ,-3 l0.5 ,3" />
    <path
      d="M1,12 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,19 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,26 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,33 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
  </g>
</svg>
```

CSS

We apply increasingly large values of `stroke-miterlimit` to the paths, such that for the first path, only the first (leftmost) subpath is mitered; for the second path, the first two subpaths are mitered; and so on until for the fifth path, all five subpaths are mitered.

```
path:nth-child(1) {
  stroke-miterlimit: 1.1;
}
path:nth-child(2) {
  stroke-miterlimit: 1.4;
}
path:nth-child(3) {
  stroke-miterlimit: 1.9;
}
path:nth-child(4) {
  stroke-miterlimit: 4.2;
}
```

```
path:nth-child(5) {
  stroke-miterlimit: 6.1;
}
```

See also

- stroke-dasharray
- stroke-dashoffset
- stroke-linecap
- stroke-linejoin
- stroke-opacity
- stroke-width
- stroke
- SVG stroke-miterlimit attribute

stroke-opacity CSS property

The **stroke-opacity** CSS property defines the opacity of an SVG shape's stroke. The effect is identical to that of opacity, except it is applied only to the stroke, not to the entire element. If present, it overrides the element's stroke-opacity attribute.

This property applies to SVG shapes and text-content elements (see stroke-opacity for a full list), but as an inherited property, it may be applied to elements such as <g> and still have the intended effect on descendant elements' strokes.

Note that a shape's stroke partially covers the fill of that shape, so a stroke with an opacity less than 1 will show the fill blended with the stroke where they overlap. To avoid this effect, it is possible to apply a global opacity with the opacity property or to put the stroke behind the fill with the paint-order attribute.

Syntax

```
/* numeric and percentage values */
stroke-opacity: 1;
stroke-opacity: 0.3;
stroke-opacity: 50%;

/* Global values */
stroke-opacity: inherit;
stroke-opacity: initial;
stroke-opacity: revert;
stroke-opacity: revert-layer;
stroke-opacity: unset;
```

Values

<number>

Any real number from 0 to 1, inclusive. A value of 0 makes the stroke completely transparent, and a value of 1 makes it completely opaque. Values outside the range 0 – 1 are clipped to the nearest end of that range; thus, negative values are clipped to 0.

<percentage>

The same as <number> (see above), except the allowed range is 0% to 100% and clipping is done with regard to that range.

Formal definition

Initial value	1
Applies to	<circle>, <ellipse>, <line>, <path>, <polygon>, <polyline>, and <rect> elements in an svg
Inherited	yes
Computed value	the specified value, clipped in the range [0,1]
Animation type	by computed value type

Formal syntax

```
stroke-opacity =
  <'opacity'>

<opacity> =
  <opacity-value>

<opacity-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Fill and Stroke Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Various stroke opacities

This example demonstrates basic usage of the stroke-opacity property and how, as a shape's stroke partially covers its fill, a stroke with an opacity less than 1 blends with the fill where they overlap.

HTML

First, we set up five multi-segment paths, all of which use a black stroke with a width of one, and a dodgerblue fill for the subpaths. Each path creates a series of mountain symbols, going from left (a shallow corner angle) to right (an extreme corner angle).

```
<svg viewBox="0 0 39 36" xmlns="http://www.w3.org/2000/svg">
  <g stroke="black" stroke-width="1" fill="dodgerblue">
    <path
      d="M1,5 l7   ,-3 l7   ,3
         m2,0 l3.5 ,-3 l3.5 ,3
         m2,0 l2   ,-3 l2   ,3
         m2,0 l0.75,-3 l0.75,3
         m2,0 l0.5 ,-3 l0.5 ,3" />
    <path
      d="M1,12 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,19 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
```

```
    <path
      d="M1,26 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,33 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
  </g>
</svg>
```

CSS

To these paths, we apply a successively higher stroke opacity value. For the first four paths, the fill can be seen through the inner half of the stroke path, though it may be difficult to discern for the fourth path. For the fifth and last path, the stroke is fully opaque and so the fill cannot be seen through the stroke.

```
g path:nth-child(1) {
  stroke-opacity: 0.2;
} /* equiv. 20% */
g path:nth-child(2) {
  stroke-opacity: 0.4;
} /* equiv. 40% */
g path:nth-child(3) {
  stroke-opacity: 0.6;
} /* equiv. 60% */
g path:nth-child(4) {
  stroke-opacity: 0.8;
} /* equiv. 80% */
g path:nth-child(5) {
  stroke-opacity: 1;
} /* equiv. 100% */
```

Results

See also

- opacity
- fill-opacity
- paint-order
- stroke
- stroke-dasharray
- stroke-dashoffset
- stroke-linecap
- stroke-linejoin
- stroke-miterlimit
- stroke-width
- SVG stroke-opacity attribute

stroke-width CSS property

The **stroke-width** CSS property defines the width of a stroke applied to the SVG shape. If present, it overrides the element's stroke-width attribute.

This property applies to any SVG shape or text-content element (see stroke-width for a full list), but as an inherited property, it may be applied to elements such as <g> and still have the intended effect on descendant elements' strokes. If the value evaluates to zero, the stroke is not drawn.

Syntax

```
/* Length and percentage values */
stroke-width: 0%;
stroke-width: 1.414px;

/* Global values */
stroke-width: inherit;
stroke-width: initial;
stroke-width: revert;
stroke-width: revert-layer;
stroke-width: unset;
```

Values

<length>

Pixel units are handled the same as SVG units (see <number>, above) and font-based lengths such as em are calculated with respect to the element's SVG value for the text size; the effects of other length units may depend on the browser.

<percentage>

Percentages refer to the normalized diagonal of the current SVG viewport, which is calculated as $\frac{\sqrt{\text{<width>}^2 + \text{<height>}^2}}{\sqrt{2}}$.

<number>

A number of SVG units, the size of which defined by the current unit space.

Formal definition

Initial value	1px
Applies to	<circle>, <ellipse>, <line>, <path>, <polygon>, <polyline>, and <rect> elements in an svg
Inherited	yes
Percentages	refer to the normalized diagonal measure of the current SVG viewport's applied viewBox, or of the viewport itself if no viewBox is specified
Computed value	an absolute <length> or <percentage>, numbers converted to absolute lengths first
Animation type	by computed value type

Formal syntax

```
stroke-width =
  [ <length-percentage> | <line-width> | <number> ]#

<length-percentage> =
```

```
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Fill and Stroke Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Various stroke widths

This example demonstrates various value syntaxes for the `stroke-width` property.

HTML

First, we set up five multi-segment paths, all of which use a black stroke with a width of one, and no fill. Each path creates a series of mountain symbols, going from left (a shallow corner angle) to right (an extreme corner angle).

```
<svg viewBox="0 0 39 30" xmlns="http://www.w3.org/2000/svg">
  <g stroke="black" stroke-width="1" fill="none">
    <path
      d="M1,5 l7   ,-3 l7   ,3
         m2,0 l3.5 ,-3 l3.5 ,3
         m2,0 l2   ,-3 l2   ,3
         m2,0 l0.75,-3 l0.75,3
         m2,0 l0.5 ,-3 l0.5 ,3" />
    <path
      d="M1,8 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,14 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,18 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
    <path
      d="M1,26 l7,-3 l7,3 m2,0 l3.5,-3 l3.5,3 m2,0 l2,-3 l2,3 m2,0 l0.75,-3 l0.75,3 m2,0 l0.5,-3 l0.5,3" />
  </g>
</svg>
```

CSS

To the first four paths, we apply stroke width values as pairs to show how bare number values and pixel values are functionally equivalent. For the first two paths, the values are `.25` and `.25px`. For the second two paths, the values are `1` and `1px`.

For the fifth and last path, a value of `5%` is applied, thus setting a stroke width that is five percent as wide as the SVG viewport's diagonal measure is long.

```
path:nth-child(1) {
  stroke-width: 0.25;
}
path:nth-child(2) {
  stroke-width: 0.25px;
}
path:nth-child(3) {
  stroke-width: 1;
}
path:nth-child(4) {
  stroke-width: 1px;
}
path:nth-child(5) {
  stroke-width: 5%;
}
```

Results

See also

- `stroke`
- `stroke-dasharray`
- `stroke-dashoffset`
- `stroke-linecap`
- `stroke-linejoin`
- `stroke-miterlimit`
- `stroke-opacity`
- `fill`
- SVG `stroke-width` attribute

tab-size CSS property

The **`tab-size`** CSS property is used to customize the width of tab characters (U+0009).

Try it

```
tab-size: 10px;
```

```
tab-size: 16px;
```

```
tab-size: 24px;
```

```
tab-size: 4;
```

```
<section id="default-example">
  <pre id="example-element">&#9;123</pre>
</section>
```

```
#example-element {
  border: 1px solid;
}
```

Syntax

```
/* <number> values */
tab-size: 4;
tab-size: 0;

/* <length> values */
tab-size: 10px;
tab-size: 2em;

/* Global values */
tab-size: inherit;
tab-size: initial;
tab-size: revert;
tab-size: revert-layer;
tab-size: unset;
```

Values

`<number>`

A multiple of the advance width of the space character (U+0020) to be used as the width of tabs. Must be non-negative. The advance width means the distance a cursor or a print head moves before printing the next character.

`<length>`

The width of tabs. Must be non-negative.

Formal definition

Initial value	`8`
Applies to	block containers
Inherited	yes
Computed value	the specified integer or an absolute length
Animation type	a length

Formal syntax

```
tab-size =
  <number [0,∞]>  |
  <length [0,∞]>
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Expanding by character count

```
pre {
  tab-size: 4; /* Set tab size to 4 characters wide */
}
```

Collapse tabs

```
pre {
  tab-size: 0; /* Remove indentation */
}
```

Default tab size vs custom sizes

This example compares a default tab size with a custom tab size. Note that `white-space` is set to `pre` to prevent the tabs from collapsing.

HTML

```
<p>no tab</p>
<p>&#0009;default tab size of 8 characters wide</p>
<p class="custom-number">&#0009;custom tab size of 3 characters wide</p>
<p>   3 spaces, equivalent to the custom tab size</p>
<p class="custom-length">&#0009;custom tab size of 50px wide</p>
```

CSS

```
body {
  border: 1px solid red;
  margin: 1rem;
}
```

```
p {
  white-space: pre;
}

.custom-number {
  tab-size: 3;
}

.custom-length {
  tab-size: 50px;
}
```

Result

See also

- white-space

table-layout CSS property

The **table-layout** CSS property sets the algorithm used to lay out <table> cells, rows, and columns.

Try it

```
table-layout: auto;
width: 150px;
```

```
table-layout: fixed;
width: 150px;
```

```
table-layout: auto;
width: 100%;
```

```
table-layout: fixed;
width: 100%;
```

```
<section class="default-example" id="default-example">
  <table class="transition-all" id="example-element">
    <tbody>
      <tr>
        <th>Name</th>
        <th>Location</th>
      </tr>
      <tr>
        <td>Lion</td>
        <td>Africa</td>
      </tr>
      <tr>
        <td>Norwegian Lemming</td>
        <td>Europe</td>
      </tr>
      <tr>
        <td>Seal</td>
        <td>Antarctica</td>
      </tr>
      <tr>
        <td>Tiger</td>
        <td>Asia</td>
      </tr>
    </tbody>
  </table>
</section>
```

```
table {
  border: 1px solid #113399;
}

th,
td {
  border: 2px solid #aa1199;
  padding: 0.25rem 0.5rem;
}
```

Syntax

```
/* Keyword values */
table-layout: auto;
table-layout: fixed;

/* Global values */
table-layout: inherit;
table-layout: initial;
table-layout: revert;
table-layout: revert-layer;
table-layout: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The automatic table layout algorithm is used. The widths of the table and its cells are adjusted to fit the content. Most browsers use this algorithm by default.

`fixed`

The fixed table layout algorithm is used. When using this keyword, the table's width *needs to be specified explicitly* using the `width` property. If the value of the `width` property is set to `auto` or is not specified, the browser uses the automatic table layout algorithm, in which case the `fixed` value has no effect.
The fixed table layout algorithm is faster than the automatic layout algorithm because the horizontal layout of the table depends only on the table's width, the width of the columns, and borders or cell spacing. The horizontal layout doesn't depend on the contents of the cells because it depends only on explicitly set widths.

In the fixed table layout algorithm, the width of each column is determined as follows:

- A column element with explicit width sets the width for that column.
- Otherwise, a cell in the first row with explicit width determines the width for that column.
- Otherwise, the column gets the width from the shared remaining horizontal space.

With this algorithm the entire table can be rendered once the first table row has been downloaded and analyzed. This can speed up rendering time over the "automatic" layout method, but subsequent cell content might not fit in the column widths provided. Cells use the `overflow` property to determine whether to clip any overflowing content, but only if the table has a known width; otherwise, they won't overflow the cells.

Formal definition

Initial value	`auto`
Applies to	`table` and `inline-table` elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
table-layout =
  auto   |
  fixed
```

This syntax reflects the latest standard as per CSS Table Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Fixed-width tables with text-overflow

This example uses a fixed table layout, combined with the `width` property, to restrict the table's width. The `text-overflow` property is used to apply an ellipsis to words that are too long to fit. If the table layout were `auto`, the table would grow to accommodate its contents, despite the specified `width`.

HTML

```
<table>
  <tbody>
    <tr>
      <td>Ed</td>
      <td>Wood</td>
    </tr>
    <tr>
      <td>Albert</td>
      <td>Schweitzer</td>
    </tr>
    <tr>
      <td>Jane</td>
      <td>Fonda</td>
    </tr>
    <tr>
      <td>William</td>
      <td>Shakespeare</td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  table-layout: fixed;
  width: 120px;
  border: 1px solid red;
}

td {
  border: 1px solid blue;
  overflow: hidden;
  white-space: nowrap;
  text-overflow: ellipsis;
}
```

Result

See also

- `<table>`
- CSS table module

text-align CSS property

The **`text-align`** CSS property sets the horizontal alignment of the inline-level content inside a block element or table-cell box. This means it works like `vertical-align` but in the horizontal direction.

Try it

```
text-align: start;
```

```
text-align: end;
```

```
text-align: center;
```

```
text-align: justify;
```

```
<section id="default-example">
  <div id="example-element">
    <p>
      Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
      veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea
      commodo consequat. Duis aute irure dolor in reprehenderit in voluptate
      velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat
      cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id
      est laborum.
    </p>
  </div>
</section>
```

```
section {
  font-size: 1.5em;
}

#default-example > div {
  width: 250px;
}
```

Syntax

```
/* Keyword values */
text-align: start;
text-align: end;
text-align: left;
text-align: right;
text-align: center;
text-align: justify;
text-align: match-parent;

/* Block alignment values (Non-standard syntax) */
text-align: -moz-center;
text-align: -webkit-center;

/* Global values */
text-align: inherit;
text-align: initial;
text-align: revert;
text-align: revert-layer;
text-align: unset;
```

Values

This property is specified as one of the following keyword values:

`start`

The same as `left` if direction is left-to-right and `right` if direction is right-to-left.

`end`

The same as `right` if direction is left-to-right and `left` if direction is right-to-left.

`left`

The inline contents are aligned to the left edge of the line box.

`right`

The inline contents are aligned to the right edge of the line box.

`center`

The inline contents are centered within the line box.

`justify`

The inline contents are justified. Spaces out the content to line up its left and right edges to the left and right edges of the line box, except for the last line.

`match-parent`

Similar to `inherit`, but the values `start` and `end` are calculated according to the parent's `direction` and are replaced by the appropriate `left` or `right` value.

Accessibility

The inconsistent spacing between words created by justified text can be problematic for people with cognitive concerns such as Dyslexia.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | Understanding WCAG 2.0

Formal definition

Initial value	`start`, or a nameless value that acts as `left` if `direction` is `ltr`, `right` if `direction` is `rtl` if `start` is not supported by the browser.
Applies to	block containers
Inherited	yes
Computed value	as specified, except for the `match-parent` value which is calculated against its parent's `direction` value and results in a computed value of either `left` or `right`
Animation type	discrete

Formal syntax

```
text-align =
  start         |
  end           |
  left          |
  right         |
  center        |
  <string>      |
  justify       |
  match-parent  |
  justify-all
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Start alignment

HTML

```
<p class="example">
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
```

```
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
.example {
  text-align: start;
  border: solid;
}
```

Result

Centered text

HTML

```
<p class="example">
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
.example {
  text-align: center;
  border: solid;
}
```

Result

Example using "justify"

HTML

```
<p class="example">
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
.example {
  text-align: justify;
  border: solid;
}
```

Result

Table alignment

This example demonstrates the use of `text-align` on `<table>` elements:

- The `<caption>` is set to right-aligned.
- The first two `<th>` elements inherit the left alignment from the `text-align: left` set on the `<thead>`, while the third is set to right-aligned.
- Inside the `<tbody>` element, the first row is set to right-aligned, the second is set to center-aligned, and the third uses the default (left) alignment.
- Within each row, some cells (c12, c31) are set to override the alignment of the row.

HTML

```
<table>
  <caption>
    Example table
  </caption>
  <thead>
    <tr>
      <th>Col 1</th>
      <th>Col 2</th>
      <th class="right">Col 3</th>
    </tr>
  </thead>
  <tbody>
    <tr class="right">
      <td>11</td>
      <td class="center">12</td>
      <td>13</td>
    </tr>
    <tr class="center">
      <td>21</td>
      <td>22</td>
      <td>23</td>
    </tr>
    <tr id="r3">
      <td class="right">31</td>
      <td>32</td>
      <td>33</td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  border-collapse: collapse;
  border: solid black 1px;
  width: 250px;
  height: 150px;
}
```

```
thead {
  text-align: left;
}

td,
th {
  border: solid 1px black;
}

.center {
  text-align: center;
}

.right,
caption {
  text-align: right;
}
```

Result

See also

- `margin: auto`, `margin-left: auto`, `vertical-align`

text-align-last CSS property

The **`text-align-last`** CSS property sets how the last line of a block or a line, right before a forced line break, is aligned.

Try it

```
text-align-last: right;
```

```
text-align-last: center;
```

```
text-align-last: left;
```

```
<section id="default-example">
  <div>
    <p id="example-element">
      Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut
      aliquip ex ea commodo consequat.
    </p>
  </div>
</section>
```

```
section {
  font-size: 1.5em;
}

#default-example > div {
  width: 250px;
}

#example-element {
  text-align: justify;
}
```

Syntax

```
/* Keyword values */
text-align-last: auto;
text-align-last: start;
text-align-last: end;
text-align-last: left;
text-align-last: right;
text-align-last: center;
text-align-last: justify;

/* Global values */
text-align-last: inherit;
text-align-last: initial;
text-align-last: revert;
text-align-last: revert-layer;
text-align-last: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

The affected line is aligned per the value of `text-align`, unless `text-align` is `justify`, in which case the effect is the same as setting `text-align-last` to `start`.

`start`

The same as `left` if direction is left-to-right and `right` if direction is right-to-left.

`end`

The same as `right` if direction is left-to-right and `left` if direction is right-to-left.

`left`

The inline contents are aligned to the left edge of the line box.

`right`

The inline contents are aligned to the right edge of the line box.

`center`

The inline contents are centered within the line box.

`justify`

The text is justified. Text should line up their left and right edges to the left and right content edges of the paragraph.

Formal definition

Initial value	`auto`
Applies to	block containers
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-align-last =
  auto          |
  start         |
  end           |
  left          |
  right         |
  center        |
  justify       |
  match-parent
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Justifying the last line

```
<p>
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
p {
  font-size: 1.4em;
  text-align: justify;
  text-align-last: center;
}
```

Results

See also

- `text-align`

text-anchor CSS property

The **`text-anchor`** CSS property aligns a box containing a string of text where the wrapping area is determined from the `inline-size` property, and the text is then placed relative to the anchor point of the element, which is defined using the `x` and `y` (or `dx` and `dy`) attributes. If present, the value of the CSS property overrides any value of the element's `text-anchor` attribute.

Each individual text fragment within an element is aligned independently; thus, a multi-line `<text>` element will have each line of text aligned as per the value of `text-anchor`. `text-anchor` values only have an effect on the `<text>`, `<textPath>`, and `<tspan>` SVG elements. `text-anchor` does not apply to automatically wrapped text; for that, see `text-align`.

Syntax

```
text-anchor: start;
text-anchor: middle;
text-anchor: end;

/* Global values */
text-anchor: inherit;
text-anchor: initial;
text-anchor: revert;
text-anchor: revert-layer;
text-anchor: unset;
```

Values

This property is specified as one of the following keyword values:

`start`

Aligns the text such that the inline start of the text string is aligned with the anchor point. This alignment is relative to the writing direction of the text; thus, for example, in right-to-left top-to-bottom writing, the text will be placed to the left of the anchor point. If the text's inline direction is vertical, as with many Asian languages, the top edge of the text is aligned with the anchor point.

`middle`

Aligns the text such that the center (middle) of the text string's inline box is aligned with the anchor point.

`end`

Aligns the text such that the inline end of the text string is aligned with the anchor point. This alignment is relative to the writing direction of the text; thus, for example, in right-to-left top-to-bottom writing, the text will be placed to the right of the anchor point. If the text's inline direction is vertical, as with many Asian languages, the bottom edge of the text is aligned with the anchor point.

Formal definition

Initial value	`start`
Applies to	`<text>`, `<textPath>`, and `<tspan>` elements in `<svg>`
Inherited	yes

Computed value	as specified
Animation type	discrete

Formal syntax

```
text-anchor =
  start   |
  middle  |
  end
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Example

Three `<text>` elements are given the same `x` position, but different values for `text-anchor`. A dashed red line is included to mark the `x`-axis position of all three anchor points.

```
<svg
  viewBox="0 0 200 150"
  height="150"
  width="200"
  xmlns="http://www.w3.org/2000/svg">
  <line
    x1="100"
    y1="0"
    x2="100"
    y2="150"
    stroke="red"
    stroke-dasharray="10,5" />
  <text x="100" y="40">Anchored</text>
  <text x="100" y="80">Anchored</text>
  <text x="100" y="120">Anchored</text>
</svg>
```

```
text {
  font-size: 24px;
}
```

```
text:nth-of-type(1) {
  text-anchor: start;
}
text:nth-of-type(2) {
  text-anchor: middle;
}
text:nth-of-type(3) {
  text-anchor: end;
}
```

See also

- `dominant-baseline`
- SVG `<text>` element
- SVG `text-anchor` attribute

text-autospace CSS property

The **`text-autospace`** CSS property allows you to specify the space applied between Chinese/Japanese/Korean (CJK) and non-CJK characters.

Syntax

```
text-autospace: normal;
text-autospace: no-autospace;
text-autospace: ideograph-alpha;
text-autospace: ideograph-numeric;
text-autospace: punctuation;
text-autospace: insert;
text-autospace: replace;
text-autospace: ideograph-alpha ideograph-numeric punctuation;
text-autospace: ideograph-alpha ideograph-numeric;
text-autospace: ideograph-alpha ideograph-numeric insert;
text-autospace: auto;

/* Global values */
text-autospace: inherit;
text-autospace: initial;
text-autospace: revert;
text-autospace: revert-layer;
text-autospace: unset;
```

Values

`normal`

> Creates the default behavior to automatically apply spacing between CJK and non-CJK characters and around punctuation. This value has the same effect as applying both `ideograph-alpha` and `ideograph-numeric`.

`<autospace>`

> Provides more control over spacing behaviors. It accepts the keyword `no-autospace`, or a combination of one or more of `ideograph-alpha`, `ideograph-numeric`, and `punctuation`, optionally followed by `insert` or `replace`.
>
> `no-autospace`
>
> > Disables automatic spacing between CJK and non-CJK characters.
>
> `ideograph-alpha`
>
> > Adds spacing only between ideographic characters (such as Katakana and Han) and non-ideographic letters (such as Latin). It does not add spacing between ideographic characters and non-ideographic numbers.

`ideograph-numeric`

Adds spacing only between ideographic characters (such as Katakana and Han) and non-ideographic numbers (such as Latin). It does not add spacing between ideographic characters and non-ideographic letters.

`punctuation`

Adds non-breaking spacing around punctuation as required by language-specific typographic conventions.

`insert`

Adds the specified spacing only if there are no existing spaces between the ideographic and non-ideographic scripts.

`replace`

Replaces existing spacing (such as U+0020) between ideographic and non-ideographic characters with the specified spacing.

`auto`

Lets the browser choose typographically appropriate spacing. The spacing may vary across browsers and platforms.

> **Note:** If neither `insert` nor `replace` are specified, the behavior is the same as `insert`.

> **Note:** This property is additive with the `word-spacing` and `letter-spacing` properties. The amount of spacing contributed by the `letter-spacing` setting is added to the spacing created by `text-autospace`. The same applies to `word-spacing`.

Formal definition

Initial value	`normal`
Applies to	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-autospace =
  normal       |
  <autospace>  |
  auto

<autospace> =
  no-autospace                                    |
  [ ideograph-alpha || ideograph-numeric || punctuation ] || [ insert | replace ]
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

This example shows the difference between various values of `text-autospace`. Try selecting a value from the drop-down box to see how it affects the spacing in the text.

```
<form>
  <label for="autospace">Choose value for <code>text-autospace:</code></label>
  <select name="autospace" id="autospace">
    <option selected value="no-autospace">no-autospace</option>
    <option value="normal">normal</option>
    <option value="ideograph-alpha">ideograph-alpha</option>
    <option value="ideograph-numeric">ideograph-numeric</option>
  </select>
</form>
<div class="unsupported">
  <code>text-autospace</code> is not supported in your browser.
</div>
```

```
<main>
  <figure class="no-autospace">
    <figcaption>
      <code>
        text-autospace: <span id="autospace-value">no-autospace</span>;
      </code>
    </figcaption>
    <div>
      <p>HTML超文本标记语言</p>
      <p>42四十二</p>
    </div>
  </figure>
</main>
```

```
.no-autospace {
  text-autospace: no-autospace;
}
.auto {
  text-autospace: auto;
}
.normal {
  text-autospace: normal;
}
.ideograph-alpha {
  text-autospace: ideograph-alpha;
}
.ideograph-numeric {
  text-autospace: ideograph-numeric;
}
```

```
figure {
  margin: 1rem;
  div {
    font-size: 2rem;
    padding: 1rem;
    font-family: sans-serif;
    border: tomato solid 1px;
  }
  p {
    margin: 0;
  }
}
main {
  max-width: max-content;
}
@supports not (text-autospace: normal) {
  form {
    display: none;
  }
  .unsupported {
    color: red;
  }
}
@supports (text-autospace: normal) {
  .unsupported {
    display: none;
  }
}
```

```
const chose = document.querySelector("#autospace");
const fig = document.querySelector("figure");
const codeValue = document.querySelector("#autospace-value");
chose.addEventListener("change", (e) => {
  fig.className = e.target.value;
  codeValue.innerText = e.target.value;
});
```

See also

- `text-spacing-trim`
- `ic` and `ric` units
- CSS Text module

text-box CSS property

The **`text-box`** CSS shorthand property corresponds to the `text-box-trim` and `text-box-edge` properties, which together specify the amount of space to trim from the block-start edge and block-end edge of a text element's block container.

Constituent properties

This property is a shorthand for the following CSS properties:

- `text-box-trim`
- `text-box-edge`

Syntax

```
/* Single keyword */
text-box: normal;

/* One text-box-edge keyword */
text-box: trim-start text;
text-box: trim-both text;

/* Two text-box-edge keywords */
text-box: trim-start cap alphabetic;
text-box: trim-both ex text;

/* Global values */
text-box: inherit;
text-box: initial;
text-box: revert;
text-box: revert-layer;
text-box: unset;
```

Values

The `text-box` value can be composed of a `text-box-trim` value and a `text-box-edge` value separated by a space. See those pages for value descriptions.

The `text-box` property can also take a keyword of `normal`, which is equivalent to `text-box: none auto;`

If `text-box-trim` is omitted, it is set to `trim-both`. If `text-box-edge` is omitted, it is set to `auto`.

Formal definition

Initial value	`normal`
Applies to	Block containers and inline boxes
Inherited	no
Computed value	the specified keyword
Animation type	discrete

Formal syntax

```
text-box =
  normal                                |
  <'text-box-trim'> || <'text-box-edge'>
```

```
<text-box-trim> =
  none        |
  trim-start  |
  trim-end    |
  trim-both

<text-box-edge> =
  auto        |
  <text-edge>

<text-edge> =
  [ text | ideographic | ideographic-ink ]            |
  [ text | ideographic | ideographic-ink | cap | ex ] [ text | ideographic | ideographic-ink | alphabetic ]
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Description

The height of text-only content is relative to the height of the font. In digital font files, the height contains all characters, including capital letters, ascenders, descenders, etc. Different fonts have different base line-heights, meaning that lines of text with the same `font-size` will produce line boxes of differing heights, affecting the appearance of spacing between lines.

The `text-box` properties enable trimming off extra spacing from the block-start edge and block-end edge of a text element's block container, which can include the leading at the text's block-start edge and block-end edges, and the spacing defined inside the font (as described above). This makes it much easier to control text spacing in the block direction.

Examples

Basic <code>text-box</code> usage

In the following example, we have two `<p>` elements with classes of `one` and `two`.

We apply a `text-box` value of `trim-end cap alphabetic` to the first paragraph. The `text-box-edge` value of `cap alphabetic` specifies trimming the over edge to the top of the capital letters and the under edge flush with the text baseline. Because the `text-box-trim` value is set to `trim-end`, only the under edge of the paragraph is trimmed.

We apply a `text-box` value of `trim-both ex alphabetic` to the second paragraph. The `text-box-edge` value of `ex alphabetic` specifies trimming the over edge to the font's x-height (the top edge of the short lower-case letters) and the under edge flush with the text baseline. Because the `text-box-trim` value is set to `trim-both`, both the over *and* under edge of the paragraph are trimmed.

```
<p class="one">This is .one</p>

<p class="two">This is .two</p>
```

```
html {
  font-family: sans-serif;
  height: 100%;
}

body {
  height: inherit;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 100px;
}

p {
  margin: 0;
  font-size: 6rem;
  font-weight: bold;
}
```

```
.one {
  text-box: trim-end cap alphabetic;
}

.two {
  text-box: trim-both ex alphabetic;
}

p {
  border-top: 5px solid magenta;
  border-bottom: 5px solid magenta;
}
```

Result

The output is as follows. Note how we've included a top and bottom border on each paragraph, so that you can see how the space has been trimmed in each case.

See also

- `text-box-edge`, `text-box-trim`
- `<text-edge>` data type
- CSS inline layout module
- CSS text-box-edge on developer.chrome.com (2025)

text-box-edge CSS property

The **`text-box-edge`** CSS property specifies an amount of space to trim from a text element's block container.

Syntax

```
/* Single keyword */
text-box-edge: auto;
text-box-edge: text;

/* Two <text-edge> values */
text-box-edge: text text;
text-box-edge: text alphabetic;
text-box-edge: cap alphabetic;
text-box-edge: ex text;

/* Global values */
text-box-edge: inherit;
```

```
text-box-edge: initial;
text-box-edge: revert;
text-box-edge: revert-layer;
text-box-edge: unset;
```

Value

The `text-box-edge` property value is specified as `auto` or a `<text-edge>` value:

`auto`

The default value. Equivalent to the `text-edge` value `text`.

`<text-edge>`

One or two separate keywords representing over and under edge positions to trim the text element's block container to.

- If two values are specified, the first value specifies the trimming behavior to apply to the block-start (over) edge of the text, and the second value specifies the trimming behavior to apply to the block-end (under) edge of the text.
  - Valid over edge trimming values: `text`, `cap`, and `ex`.
  - Valid under edge trimming values: `text` and `alphabetic`.
- If one value is specified, it specifies the over *and* under edge trimming behavior. At the time of writing, the only valid single value is `text`.

Description

The `text-box-edge` property specifies an amount of space to trim from a text element's block container.

Vertical spacing differs between fonts, making consistent typesetting historically challenging on the web. The `text-box-edge` property — along with its counterpart property `text-box-trim`, which specifies which edge(s) to trim space from — makes consistent typesetting easier to achieve. The `text-box-edge` property has no effect if `text-box-trim` is not set or is set to `none`.

The height of text-only content is relative to the height of the font. In digital font files, the height contains all characters, including capital letters, ascenders, descenders, etc. Different fonts have different base line-heights, meaning that lines of text with the same `font-size` will produce line boxes of differing heights, affecting the appearance of spacing between lines.

The `text-box-edge` property allows you to trim space from the start and/or end edge of the text's block container. This can include the leading at the text's block-start edge and block-end edges and the spacing defined inside the font (as described above). It does this by specifying a `<text-edge>` value that indicates the over edge and under edge to trim the space to.

Which edge(s) to trim space from is specified using the `text-box-trim` property. For example, you can choose to trim space from the over edge or the under edge of the text's block container, or both.

These properties make it much easier to control text spacing in the block direction.

The `text-box-edge` property, along with the `text-box-trim` property, can also be set by using the `text-box` shorthand.

Formal definition

Initial value	`auto`
Applies to	Block containers and inline boxes
Inherited	no
Computed value	the specified keyword
Animation type	discrete

Formal syntax

```
text-box-edge =
  auto         |
  <text-edge>

<text-edge> =
  [ text | ideographic | ideographic-ink ]            |
  [ text | ideographic | ideographic-ink | cap | ex ] [ text | ideographic | ideographic-ink | alphabetic ]
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>text-box-edge</code> usage

The most common `text-box-edge` values you'll use for horizontal `writing-mode` languages such as English or Arabic are `cap alphabetic` and `ex alphabetic`. The `cap` value trims the over edge of the text element's block container to the top of the capital letters, whereas `ex` trims the over edge to the font's x-height (the top edge of the short lower-case letters). In each case, `alphabetic` trims the under-edge flush with the text baseline.

In this example, we demonstrate the effect of both of these common values, on two `<p>` elements. Additionally, a `text-box-trim` value of `trim-both` has been set on both of them, so that their start *and* end edges are trimmed.

```
<p class="one">This is .one</p>

<p class="two">This is .two</p>
```

```
html {
  font-family: sans-serif;
  height: 100%;
}

body {
  height: inherit;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 100px;
}

p {
  margin: 0;
  font-size: 6rem;
  font-weight: bold;
}
```

```
p {
  text-box-trim: trim-both;
  border-top: 5px solid magenta;
  border-bottom: 5px solid magenta;
}

.one {
  text-box-edge: cap alphabetic;
}

.two {
  text-box-edge: ex alphabetic;
}
```

Result

The output is as follows. Note how we've included a top and bottom border on each paragraph, so that you can see how the space has been trimmed in each case.

Interactive <code>text-box-edge</code> value comparison

For a complete interactive `text-box-edge` example, see the `text-box-trim` page.

See also

- `text-box`, `text-box-trim`
- `<text-edge>` data type
- CSS inline layout module
- CSS text-box-edge on developer.chrome.com (2025)

text-box-trim CSS property

The **`text-box-trim`** CSS property specifies which of the over and under edges of text content to trim from a text element's block container.

Syntax

```
/* Keyword values */
text-box-trim: none;
text-box-trim: trim-both;
text-box-trim: trim-start;
text-box-trim: trim-end;

/* Global values */
text-box-trim: inherit;
text-box-trim: initial;
text-box-trim: revert;
text-box-trim: revert-layer;
text-box-trim: unset;
```

Value

The `text-box-trim` property value may be specified as one of the following keywords:

`none`

The default value. No space is trimmed from the text.

`trim-both`

The start (over) and end (under) edges are both trimmed.

`trim-start`

The start (over) edge is trimmed.

`trim-end`

The end (under) edge is trimmed.

Description

The `text-box-trim` property specifies which of the over and under edges of text content to trim from a text element's block container.

Vertical spacing differs between fonts, making consistent typesetting historically challenging on the web. The `text-box-trim` property — along with its counterpart property `text-box-edge`, which specifies how much space to trim — makes consistent vertical spacing of text easier to achieve.

The height of text-only content is relative to the height of the font. In digital font files, the height contains all characters, including capital letters, ascenders, descenders, etc. Different fonts have different base line-heights, meaning that lines of text with the same `font-size` will produce line boxes of differing heights, affecting the appearance of spacing between lines.

The `text-box-trim` property allows you to trim the over and under edge of the text's block container, making it easier to control text spacing in the block direction.

The actual amount of space trimmed is specified using the `text-box-edge` property. For example, you can choose to trim the over edge in line with a font's capital letters or lower-case letters, and the under edge flush with the font's baseline.

The `text-box-trim` property, along with the `text-box-edge` property, can also be set by using the `text-box` shorthand.

Formal definition

Initial value	none
Applies to	Block containers and inline boxes
Inherited	no
Computed value	the specified keyword
Animation type	discrete

Formal syntax

```
text-box-trim =
  none        |
  trim-start  |
```

```
  trim-end    |
  trim-both
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>text-box-trim</code> usage

In the following example we set `text-box-edge: cap alphabetic` on two paragraphs, which trims the over edge of the text elements' block containers to the top of the capital letters and the under edge flush with the text baseline.

We then set `text-box-trim` values of `trim-end` on the first one, and `trim-both` on the second one. This results in the first paragraph only having its under edge trimmed, whereas the second one has both the over *and* under edge trimmed.

```
<p class="one">This is .one</p>

<p class="two">This is .two</p>
```

```
html {
  font-family: sans-serif;
  height: 100%;
}

body {
  height: inherit;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 100px;
}

p {
  margin: 0;
  font-size: 6rem;
  font-weight: bold;
}
```

```
p {
  text-box-edge: cap alphabetic;
  border-top: 5px solid magenta;
  border-bottom: 5px solid magenta;
}

.one {
  text-box-trim: trim-end;
}

.two {
  text-box-trim: trim-both;
}
```

Result

The output is as follows. Note how we've included a top and bottom border on each paragraph, so that you can see how the space has been trimmed in each case.

Interactive <code>text-box-trim</code> and <code>text-box-edge</code> value comparison

In this example we provide a user interface that allows you to choose the `text-box-trim` and `text-box-edge` values applied to a paragraph of text.

HTML

In our HTML, we include three main items:

- Three `<select>` elements allowing you to set which edges of the paragraph should be trimmed (the `text-box-trim` value) and how much space to trim from the block-start and block-end edges of the paragraph (the `text-box-edge` value).
- A `<p>` element containing text, which the `text-box-*` values are applied to. This paragraph has `contenteditable` set on it so you can edit the text.
- An `<output>` element that displays the `text-box-*` declarations applied to the paragraph. This is updated when a selection is made.

We also import a font from the Google Fonts service to apply to our demo's text.

We have hidden the exact HTML code for brevity.

```
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link
  href="https://fonts.googleapis.com/css2?family=Roboto:ital,wght@0,100..900;1,100..900&display=swap"
  rel="stylesheet" />
<section>
  <div>
    <label for="box-trim">Select edge(s) to trim:</label>
    <select id="box-trim">
      <option>none</option>
      <option>trim-start</option>
      <option>trim-end</option>
      <option selected>trim-both</option>
    </select>
  </div>
  <div>
    <label for="trim-over">Select trim over (start) value:</label>
    <select id="trim-over">
      <option>text</option>
      <option selected>cap</option>
      <option>ex</option>
    </select>
  </div>
</section>
<p class="display" contenteditable>Holly Golightly</p>
<section>
  <div>
    <label for="trim-under">Select trim under (end) value:</label>
```

```
    <select id="trim-under">
      <option>text</option>
      <option selected>alphabetic</option>
    </select>
  </div>
</section>
<output></output>
```

CSS

In our CSS, we apply the imported font to the `<html>` element and lay out the UI using flexbox. We have hidden most of the CSS code for brevity, but below we show the rules styling the paragraph the `text-box-*` effects are applied to and the `<output>` that shows the `text-box-*` rules being applied:

```
html {
  font-family: "Roboto", sans-serif;
  height: 100%;
}

body {
  height: inherit;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 30px;
}

section {
  display: flex;
  justify-content: space-between;
}

section:nth-of-type(2) {
  justify-content: flex-end;
  padding-bottom: 30px;
}

select {
  width: 6rem;
}
```

```
p {
  margin: 0;
  font-size: 6rem;
  font-weight: bold;
  border-top: 5px solid magenta;
  border-bottom: 5px solid magenta;
}

output {
  border: 2px solid gray;
  border-radius: 10px;
  padding: 10px;
  margin: 0;
  width: fit-content;
}
```

Again, note how we've included a top and bottom border on the `.display` paragraph, so that you can see how the space being trimmed changes when different `text-box-*` values are selected.

JavaScript

In the JavaScript, we start by grabbing references to the three `<select>` elements and two `<p>` elements:

```
const boxTrimSelect = document.getElementById("box-trim");
const trimOverSelect = document.getElementById("trim-over");
const trimUnderSelect = document.getElementById("trim-under");

const displayElem = document.querySelector("p");
const codeElem = document.querySelector("output");
```

Next, we define a function called `setEdgeTrim()`. This applies a `text-box` value to the paragraph based on the values of the `<select>` elements, and also prints the applied declarations to the output (both the longhand and shorthand equivalents):

```
function setEdgeTrim() {
  const textBoxTrimValue = boxTrimSelect.value;
  const textBoxEdgeValue = `${trimOverSelect.value} ${trimUnderSelect.value}`;
  displayElem.style.textBox = `${textBoxTrimValue} ${textBoxEdgeValue}`;

  codeElem.innerHTML = `
    <span><code>text-box-trim: ${textBoxTrimValue}</code></span>
    <br>
    <span><code>text-box-edge: ${textBoxEdgeValue}</code></span>
    <br><br>
    <span>Shorthand equivalent:</span>
    <br><br>
    <span><code>text-box: ${textBoxTrimValue} ${textBoxEdgeValue}</code></span>
  `;
}
```

In the last part of the JavaScript we run the `setEdgeTrim()` function once to set an initial state for the UI. We then apply `change` event listeners to all of the `<select>` elements (via `addEventListener`) so that `setEdgeTrim()` is run whenever one of the `<select>` values changes to update the UI accordingly:

```
setEdgeTrim();

boxTrimSelect.addEventListener("change", setEdgeTrim);
trimOverSelect.addEventListener("change", setEdgeTrim);
trimUnderSelect.addEventListener("change", setEdgeTrim);
```

Result

The output is as follows:

`text-box-trim` is initially set to `trim-both`, meaning that the over *and* under edges of the paragraph are trimmed. `text-box-edge` is initially set to `cap alphabetic`, meaning that the text is trimmed flush with the top of capital letters at the start edge, and flush with the baseline at the end edge.

Try changing the `<select>` values to see the effect they have on the display text.

See also

- `text-box`, `text-box-edge`
- CSS inline layout module
- CSS text-box-trim on developer.chrome.com (2025)

text-combine-upright CSS property

The **`text-combine-upright`** CSS property sets the combination of characters into the space of a single character. If the combined text is wider than 1em, the user agent must fit the contents within 1em. The resulting composition is treated as a single upright glyph for layout and decoration. This property only has an effect in vertical writing modes.

This is used to produce an effect that is known as tate-chū-yoko "縦中横" in Japanese, or as "横向文字" in Chinese.

Try it

```
text-combine-upright: none;
```

```
text-combine-upright: all;
```

```
<section class="default-example" id="default-example">
  <div>
    <p>
      <span class="transition-all" id="example-element"
        >2022<span>年</span>12<span>月</span>8</span
      >日から楽しい
    </p>
  </div>
</section>
```

```
p {
  writing-mode: vertical-rl;
}
```

Syntax

```
/* Keyword values */
text-combine-upright: none;
text-combine-upright: all;

/* Global values */
text-combine-upright: inherit;
text-combine-upright: initial;
text-combine-upright: revert;
text-combine-upright: revert-layer;
text-combine-upright: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

There is no special processing.

`all`

Attempts to typeset all consecutive characters within the box horizontally, such that they take up the space of a single character within the vertical line of the box.

> **Note:** The CSS writing modes module defines a `digits <integer>` value for the `text-combine-upright` property to display two to four consecutive ASCII digits (U+0030–U+0039) such that it takes up the space of a single character within the vertical line box, however, this is not supported in any browsers.

Formal definition

Initial value	none
Applies to	non-replaced inline elements
Inherited	yes
Computed value	specified keyword, plus integer if 'digits'
Animation type	Not animatable

Formal syntax

```
text-combine-upright =
  none                          |
  all                           |
  [ digits <integer [2,4]>? ]

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Writing Modes Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using 'all' with horizontal text

The all value requires markup around every piece of horizontal text, but it is currently supported by more browsers than the digits value.

HTML

```
<p lang="zh-Hant">
  民國<span class="num">105</span>年<span class="num">4</span>月<span
    class="num"
    >29</span
  >日
</p>
```

CSS

```
html {
  writing-mode: vertical-rl;
  font: 24px serif;
}
.num {
  text-combine-upright: all;
}
```

Results

See also

- `writing-mode`, `text-orientation`

text-decoration CSS property

The **`text-decoration`** CSS shorthand property sets the appearance of decorative lines on text. It is a shorthand for `text-decoration-line`, `text-decoration-color`, `text-decoration-style`, and the newer `text-decoration-thickness` property.

Text decorations are drawn across descendant text elements. This means that if an element specifies a text decoration, then a child element can't remove the decoration. For example, in the markup `<p>This text has <em>some emphasized words</em> in it.</p>`, the style rule `p { text-decoration: underline; }` would cause the entire paragraph to be underlined. The style rule `em { text-decoration: none; }` would not cause any change; the entire paragraph would still be underlined. However, the rule `em { text-decoration: overline; }` would cause a second decoration to appear on "some emphasized words".

Try it

```
text-decoration: underline;
```

```
text-decoration: underline dotted;
```

```
text-decoration: underline dotted red;
```

```
text-decoration: green wavy underline;
```

```
text-decoration: underline overline #ff3028;
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `text-decoration-color`
- `text-decoration-line`
- `text-decoration-style`
- `text-decoration-thickness`

Syntax

```
text-decoration: underline;
text-decoration: overline red;
text-decoration: none;

/* Global values */
text-decoration: inherit;
text-decoration: initial;
text-decoration: revert;
text-decoration: revert-layer;
text-decoration: unset;
```

The `text-decoration` property is specified as one or more space-separated values representing the various longhand text-decoration properties.

Values

`text-decoration-line`

Sets the kind of decoration used, such as `underline` or `line-through`.

`text-decoration-color`

Sets the color of the decoration.

`text-decoration-style`

Sets the style of the line used for the decoration, such as `solid`, `wavy`, or `dashed`.

`text-decoration-thickness`

Sets the thickness of the line used for the decoration.

Formal definition

Initial value	as each of the properties of the shorthand: • `text-decoration-color`: `currentcolor` • `text-decoration-style`: `solid` • `text-decoration-line`: `none`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as each of the properties of the shorthand: • `text-decoration-line`: as specified • `text-decoration-style`: as specified • `text-decoration-color`: computed color • `text-decoration-thickness`: as specified
Animation type	as each of the properties of the shorthand: • `text-decoration-color`: a color • `text-decoration-style`: discrete • `text-decoration-line`: discrete • `text-decoration-thickness`: by computed value type

Formal syntax

```
text-decoration =
  <'text-decoration-line'>      ||
  <'text-decoration-thickness'> ||
  <'text-decoration-style'>     ||
  <'text-decoration-color'>

<text-decoration-line> =
  none                                          |
  [ underline || overline || line-through || blink ]  |
  spelling-error                                |
  grammar-error

<text-decoration-thickness> =
  auto                  |
  from-font             |
  <length-percentage>   |
  <line-width>

<text-decoration-style> =
  solid    |
  double   |
  dotted   |
  dashed   |
  wavy

<text-decoration-color> =
  <color>

<length-percentage> =
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Text Decoration Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Demonstration of text-decoration values

```
.under {
  text-decoration: underline red;
}

.over {
  text-decoration: wavy overline lime;
}

.line {
  text-decoration: line-through;
}

.plain {
  text-decoration: none;
}

.underover {
  text-decoration: dashed underline overline;
}
```

```
.thick {
  text-decoration: solid underline purple 4px;
}

.blink {
  text-decoration: blink;
}
```

```
<p class="under">This text has a line underneath it.</p>
<p class="over">This text has a line over it.</p>
<p class="line">This text has a line going through it.</p>
<p>
  This <a class="plain" href="#">link will not be underlined</a>, as links
  generally are by default. Be careful when removing the text decoration on
  anchors since users often depend on the underline to denote hyperlinks.
</p>
<p class="underover">This text has lines above <em>and</em> below it.</p>
<p class="thick">
  This text has a really thick purple underline in supporting browsers.
</p>
<p class="blink">
  This text might blink for you, depending on the browser you use.
</p>
```

Result

See also

- The individual text-decoration properties are `text-decoration-line`, `text-decoration-color`, `text-decoration-style`, and `text-decoration-thickness`.
- The `text-decoration-skip-ink`, `text-underline-offset`, and `text-underline-position` properties also affect text-decoration, but are not included in the shorthand.
- The `list-style` property controls the appearance of items in HTML `<ol>` and `<ul>` lists.
- SVG `text-decoration` attribute

text-decoration-color CSS property

The **`text-decoration-color`** CSS property sets the color of decorations added to text by `text-decoration-line`.

The color applies to decorations, such as underlines, overlines, strikethroughs, and wavy lines like those used to mark misspellings, in the scope of the property's value.

CSS does not provide a direct mechanism for specifying a unique color for each line type. This effect can nevertheless be achieved by nesting elements, applying a different line type to each element (with the `text-decoration-line` property), and specifying the line color (with `text-decoration-color`) on a per-element basis.

Try it

```
text-decoration-color: red;
```

```
text-decoration-color: #21ff21;
```

```
text-decoration-color: rgb(255 90 255);
```

```
text-decoration-color: hsl(70 100% 40%);
```

```
text-decoration-color: currentColor;
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}

#example-element {
  text-decoration-line: underline;
}
```

Syntax

```
/* <color> values */
text-decoration-color: currentColor;
text-decoration-color: red;
text-decoration-color: #00ff00;
text-decoration-color: rgb(255 128 128 / 50%);
text-decoration-color: transparent;

/* Global values */
text-decoration-color: inherit;
text-decoration-color: initial;
text-decoration-color: revert;
text-decoration-color: revert-layer;
text-decoration-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

The color of the line decoration.

Accessibility

It is important to ensure that the contrast ratio between the color of the text, the background the text is placed over, and the text decoration line is high enough that people experiencing low vision conditions will be able to read the content of the page. Color contrast ratio is determined by comparing the luminosity of the text and background color values.

Color alone should not be used to convey meaning. For example, change of text and text-decoration-color alone is not enough to indicate a link has focus.

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`currentcolor`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	computed color
Animation type	a color

Formal syntax

```
text-decoration-color =
  <color>
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
<p>
  This paragraph has <s>some erroneous text</s> inside it that I want to call
  attention to.
</p>
```

```
p {
  text-decoration-line: underline;
  text-decoration-color: cyan;
}

s {
  text-decoration-line: line-through;
  text-decoration-color: red;
  text-decoration-style: wavy;
}
```

See also

- When setting multiple line-decoration properties at once, it may be more convenient to use the `text-decoration` shorthand property instead.
- The `<color>` data type
- Other color-related properties: `background-color`, `border-color`, `outline-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`

text-decoration-inset CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`text-decoration-inset`** CSS property enables adjusting the start and end points of an element's text decoration so it can be shortened, lengthened, or have its position shifted with respect to the rendered text.

Try it

```
text-decoration-inset: 20px;
```

```
text-decoration-inset: -0.5em;
```

```
text-decoration-inset: 20px 1em;
```

```
text-decoration-inset: -0.5rem -1.5rem;
```

```
text-decoration-inset: -2ex 10vw;
```

```
<section id="default-example">
  <p id="example-element">Karmadrome</p>
</section>
```

```
#example-element {
  font: 2.5em sans-serif;
  text-decoration: underline 0.3em limegreen;
}
```

Syntax

```
/* auto keyword */
text-decoration-inset: auto;

/* One <length> value */
text-decoration-inset: 20px;
text-decoration-inset: -2rem;
text-decoration-inset: 5%;

/* Two <length> values */
```

```
text-decoration-inset: 20px 1em;
text-decoration-inset: -0.5rem -1.5rem;
text-decoration-inset: -2ex 1vw;
text-decoration-inset: -5% -5%;

/* Global values */
text-decoration-inset: inherit;
text-decoration-inset: initial;
text-decoration-inset: revert;
text-decoration-inset: revert-layer;
text-decoration-inset: unset;
```

Values

This property is specified as one or two of the following value or the keyword `auto`:

`<length-percentage>`

> Specifies the amount to adjust the text decoration position by. Positive values inset the text decoration (make it shorter) while negative values outset the text decoration (make it longer). If one value is specified, it applies to both the text decoration start and end points. If two values are specified, the first one applies to the text decoration start point and the second one applies to the text decoration end point. Percentage values either refer to the total inline size of the decorating box if `box-decoration-break` is set to `slice`, or to the inline size of each individual box fragment, if it is set to `clone`.

`auto`

> The browser chooses a start and end inset amount to ensure that, if two decorated text boxes appear side-by-side, the appearance of a gap is created between their text decorations so they do not appear to have a single text decoration.

Description

By default, an element's text decoration, as set by the `text-decoration` shorthand and associated longhand properties, is the same size as the rendered text.

The `text-decoration-inset` property allows you to adjust the start and/or end points of a text container's text decoration. This is useful for creating effects where you want the text decoration to be inset or outset from the text itself, or shifted in position. See Basic use cases for an example of each.

A single `<length-percentage>` value will set the inset (if positive) or outset (if negative) on the start and end positions of the text decoration. To set the start and end positions separately, you can use two `<length-percentage>` values — the first one applies to the start position of the text decoration and the second one applies to the end.

> **Note:** A percentage value depends on the value of `box-decoration-break`, if it is set to `slice` (the initial value) it refers to the inline size of the decorating box or if it is set to `clone` it refers to each individual box fragment.

The `text-decoration-inset` property can also take the `auto` keyword. This causes the browser to inset the text decoration start and end points to ensure that, if two decorated text boxes appear side-by-side, they do not appear to have a single text decoration. The `auto` value is particularly important when rendering Chinese text, where underlining is used to punctuate proper nouns, and adjacent proper nouns should have separate underlines. See Effect of the `auto` value for an example.

The `auto` value does not have the same effect as the initial value `0`. Setting `text-decoration-inset` to `0` causes there to be no space between decorations.

The `text-decoration-inset` property is not inherited, and it is not a constituent property of the `text-decoration` shorthand.

Formal definition

Initial value	`0`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the size of the box itself
Computed value	for percentage and length values, the absolute length, otherwise the keyword as specified
Animation type	by computed value

Formal syntax

```
text-decoration-inset =
  <length-percentage>{1,2}  |
  auto

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic use cases

In this example, we demonstrate the outset, inset, and "shifted" use cases.

HTML

We define an unordered list with three list items, each with a separate `id`.

```
<ul>
  <li id="one">Outset decoration</li>
  <li id="two">Inset decoration</li>
  <li id="three">Shifted decoration</li>
</ul>
```

CSS

We give each list item a different `text-decoration` and `text-decoration-inset`:

- The first one has a thick lime green underline, which is outset equally by `10px` on both sides.
- The second one has a medium thickness white strike-through, which is inset equally by `0.5em` on both sides.
- The third one has a thin wavy blue underline, which is moved to the right by `1em`.

```
li {
  font-family: sans-serif;
  font-size: 2em;
  margin-bottom: 20px;
}

@supports not (text-decoration-inset: auto) {
  body::before {
    content: "Your browser doesn't support the text-decoration-inset property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
#one {
  text-decoration: underline 0.3em limegreen;
  text-decoration-inset: -10px;
}

#two {
  text-decoration: line-through 5px white;
  text-decoration-inset: 0.5em;
}

#three {
  text-decoration: underline wavy 2px blue;
  text-decoration-inset: 1em -1em;
}
```

Result

This renders like so:

Effect of the <code>auto</code> value

This example demonstrates the effect of the `text-decoration-inset: auto` value.

HTML

We define two groups of side-by-side `<u>` elements:

```
<p lang="zh" id="one"><u>石井</u><u>艾俐俐</u></p>

<p lang="zh" id="two"><u>石井</u><u>艾俐俐</u></p>
```

CSS

Each `<u>` element has a `red` color and `3px` thickness set on its underline. The first group of `<u>` elements has a `text-decoration-inset` value of `auto` set on them, while the second set has the initial `text-decoration-inset` value of `0` explicitly set on them, for comparison purposes:

```
u {
  font-family: sans-serif;
  font-size: 2em;
}

@supports not (text-decoration-inset: auto) {
  body::before {
    content: "Your browser doesn't support the text-decoration-inset property.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
u {
  text-decoration-color: red;
  text-decoration-thickness: 3px;
}

#one u {
  text-decoration-inset: auto;
}

#two u {
  text-decoration-inset: 0;
}
```

Result

This renders like so:

Note how the `auto` value insets the text decoration subtly on both sides, creating a gap in between the underlines of the two elements (no space is added between the two elements themselves). The `0` value results in no gap.

See also

- `text-decoration`
- The CSS text decoration module

text-decoration-line CSS property

The **`text-decoration-line`** CSS property sets the kind of decoration that is used on text in an element, such as an underline or overline.

When setting multiple line-decoration properties at once, it may be more convenient to use the `text-decoration` shorthand property instead.

Try it

```
text-decoration-line: none;
```

```
text-decoration-line: underline;
```

```
text-decoration-line: overline;
```

```
text-decoration-line: line-through;
```

```
text-decoration-line: grammar-error;
```

```
text-decoration-line: spelling-error;
```

```
text-decoration-line: underline overline;
```

```
text-decoration-line: underline line-through;
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}
```

Syntax

```
/* Single keyword */
text-decoration-line: none;
text-decoration-line: underline;
text-decoration-line: overline;
text-decoration-line: line-through;
text-decoration-line: blink;
text-decoration-line: spelling-error;
text-decoration-line: grammar-error;

/* Multiple keywords */
text-decoration-line: underline overline; /* Two decoration lines */
text-decoration-line: overline underline line-through; /* Multiple decoration lines */

/* Global values */
text-decoration-line: inherit;
text-decoration-line: initial;
text-decoration-line: revert;
text-decoration-line: revert-layer;
text-decoration-line: unset;
```

Values

This property is specified either as `none` or a space-separated list of keyword values chosen from the list below:

`none`

Produces no text decoration.

`underline`

Each line of text has a decorative line beneath it.

`overline`

Each line of text has a decorative line above it.

`line-through`

Each line of text has a decorative line going through its middle.

`blink`

The text blinks (alternates between visible and invisible). Conforming user agents may not blink the text. This value is **deprecated** in favor of CSS animations.

`spelling-error`

Each line of text uses the user agents' method of highlighting spelling mistakes, which is a dotted red line in most browsers.

`grammar-error`

Each line of text uses the user agents' method of highlighting grammar mistakes, which is a dotted green line in most browsers.

> **Note:** When using `spelling-error` and `grammar-error` values, the browser disregards the other properties in the `text-decoration` shorthand (such as `text-underline-position`, `color`, or `stroke`).

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-decoration-line =
  none                                          |
  [ underline || overline || line-through || blink ]  |
  spelling-error                                |
  grammar-error
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
<p class="wavy">Here's some text with wavy red underline!</p>
<p class="both">This text has lines both above and below it.</p>
```

```
.wavy {
  text-decoration-line: underline;
  text-decoration-style: wavy;
  text-decoration-color: red;
}

.both {
  text-decoration-line: underline overline;
}
```

Errors example

In this example, the first paragraph contains a spelling mistake and uses the browser's styling for spelling errors on the misspelled word. The second paragraph uses the browser's styling for grammar errors. There is no styling change in browsers that do not support these `text-decoration-line` values.

```
<p>This text contains a <span class="spelling">speling</span> mistake.</p>
<p class="grammar">This text contain grammatical errors.</p>
```

```
.spelling {
  text-decoration-line: spelling-error;
}

.grammar {
  text-decoration-line: grammar-error;
}
```

See also

- When setting multiple line-decoration properties at once, it may be more convenient to use the `text-decoration` shorthand property instead, which also includes:
  - `text-decoration-style`
  - `text-decoration-color`
  - `text-decoration-thickness`
- `text-underline-offset`
- `::spelling-error`
- `::grammar-error`

text-decoration-skip CSS property

The **`text-decoration-skip`** CSS property sets what parts of an element's content any text decoration affecting the element must skip over. It controls all text decoration lines drawn by the element and also any text decoration lines drawn by its ancestors.

> **Note:** Most other browsers are converging on supporting the simpler `text-decoration-skip-ink` property.

Syntax

```
/* Keyword values */
text-decoration-skip: none;
text-decoration-skip: objects;
text-decoration-skip: spaces;
text-decoration-skip: edges;
text-decoration-skip: box-decoration;

/* Multiple keywords */
text-decoration-skip: objects spaces;
text-decoration-skip: leading-spaces trailing-spaces;
text-decoration-skip: objects edges box-decoration;

/* Global values */
text-decoration-skip: inherit;
text-decoration-skip: initial;
text-decoration-skip: revert;
text-decoration-skip: revert-layer;
text-decoration-skip: unset;
```

Values

`none`

Nothing is skipped. Thus, text decoration is drawn for all text content and across atomic inline-level boxes.

`objects`

The entire margin box of the element is skipped if it is an atomic inline such as an image or inline-block.

`spaces`

All spacing is skipped: all Unicode white space characters and all word separators, plus any adjacent `letter-spacing` or `word-spacing`.

`leading-spaces`

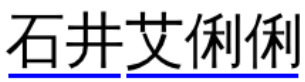
The same as `spaces`, except that only leading spaces are skipped.

`trailing-spaces`

The same as `spaces`, except that only trailing spaces are skipped.

`edges`

The start and end of the text decoration is inset slightly (e.g., by half of the line thickness) from the content edge of the decorating box. Thus, adjacent elements receive separate underlines. (This is important in Chinese, where underlining is a form of punctuation.)



`box-decoration`

The text decoration is skipped over the box's margin, border, and padding areas. This only has an effect on decorations imposed by an ancestor; a *decorating box* never draws over its own box decoration.

Formal definition

Initial value	`objects`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-decoration-skip =
  none  |
  auto
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Skipping edges

HTML

```
<p>Hey, grab a cup of <em>coffee!</em></p>
```

CSS

```
p {
  margin: 0;
  font-size: 3em;
  text-decoration: underline;
  text-decoration-skip: edges;
}
```

Result

See also

- `text-decoration-skip-ink`

text-decoration-skip-ink CSS property

The **`text-decoration-skip-ink`** CSS property specifies how overlines and underlines are drawn when they pass over glyph ascenders and descenders.

`text-decoration-skip-ink` is not part of the `text-decoration` shorthand.

Try it

```
text-decoration-skip-ink: auto;
```

```
text-decoration-skip-ink: none;
```

```
<section id="default-example">
  <p>
    <span class="transition-all" id="example-element">parapsychologists</span>
  </p>
</section>
```

```
p {
  font:
    1.9em "Georgia",
    serif;
  text-decoration: underline;
}
```

Syntax

```
/* Single keyword */
text-decoration-skip-ink: none;
text-decoration-skip-ink: auto;
text-decoration-skip-ink: all;

/* Global keywords */
text-decoration-skip-ink: inherit;
text-decoration-skip-ink: initial;
text-decoration-skip-ink: revert;
```

```
text-decoration-skip-ink: revert-layer;
text-decoration-skip-ink: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

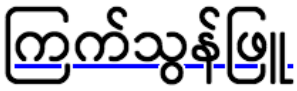
Underlines and overlines are drawn across the full length of the text content, including parts that cross over glyph descenders and ascenders.

`auto`

The default — the browser *may* interrupt underlines and overlines so that they do not touch or closely approach a glyph. That is, they are interrupted where they would otherwise cross over a glyph.

`all`

The browser *must* interrupt underlines and overlines so that they do not touch or closely approach a glyph. This can be helpful with certain Chinese, Japanese, or Korean (CJK) fonts, where the `auto` behavior might not create interruptions.



Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-decoration-skip-ink =
  auto  |
  none  |
  all
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p>You should go on a quest for a cup of coffee.</p>
<p class="no-skip-ink">Or maybe you'd prefer some tea?</p>
<p>この文は、 text-decoration-skip-ink: auto の使用例を示しています。</p>
<p class="skip-ink-all">
  この文は、 text-decoration-skip-ink: all の使用例を示しています。
</p>
```

CSS

```
p {
  font-size: 1.5em;
  text-decoration: underline blue;
  text-decoration-skip-ink: auto; /* this is the default anyway */
}

.no-skip-ink {
  text-decoration-skip-ink: none;
}

.skip-ink-all {
  text-decoration-skip-ink: all;
}
```

Result

See also

- `text-decoration`
- `text-decoration-skip`

text-decoration-style CSS property

The **`text-decoration-style`** CSS property sets the style of the lines specified by `text-decoration-line`. The style applies to all lines that are set with `text-decoration-line`.

Try it

```
text-decoration-style: solid;
```

```
text-decoration-style: double;
```

```
text-decoration-style: dotted;
```

```
text-decoration-style: dashed;
```

```
text-decoration-style: wavy;
```

```
<section id="default-example">
  <p>
```

```
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}

#example-element {
  text-decoration-line: underline;
}
```

If the specified decoration has a specific semantic meaning, like a line-through line meaning that some text has been deleted, authors are encouraged to denote this meaning using an HTML tag, like `<del>` or `<s>`. As browsers can disable styling in some cases, the semantic meaning won't disappear in such a situation.

When setting multiple line-decoration properties at once, it may be more convenient to use the `text-decoration` shorthand property instead.

Syntax

```
/* Keyword values */
text-decoration-style: solid;
text-decoration-style: double;
text-decoration-style: dotted;
text-decoration-style: dashed;
text-decoration-style: wavy;

/* Global values */
text-decoration-style: inherit;
text-decoration-style: initial;
text-decoration-style: revert;
text-decoration-style: revert-layer;
text-decoration-style: unset;
```

Values

solid

Draws a single line.

double

Draws a double line.

dotted

Draws a dotted line.

dashed

Draws a dashed line.

wavy

Draws a wavy line.

-moz-none

Draws no line. Use `text-decoration-line: none` instead.

Formal definition

Initial value	`solid`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-decoration-style =
  solid   |
  double  |
  dotted  |
  dashed  |
  wavy
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a wavy underline

The following creates a red wavy underline:

CSS

```
.wavy {
  text-decoration-line: underline;
  text-decoration-style: wavy;
  text-decoration-color: red;
}
```

HTML

```
<p class="wavy">This text has a wavy red line beneath it.</p>
```

Results

See also

- When setting multiple line-decoration properties at once, it may be more convenient to use the `text-decoration` shorthand property instead.
- `text-decoration-line`
- `text-decoration-color`
- `text-decoration-thickness`
- `text-underline-offset`

text-decoration-thickness CSS property

The **`text-decoration-thickness`** CSS property sets the stroke thickness of the decoration line that is used on text in an element, such as a line-through, underline, or overline.

Try it

```
text-decoration-line: underline;
text-decoration-thickness: 3px;
```

```
text-decoration-line: line-through;
text-decoration-thickness: 0.4rem;
```

```
text-decoration-line: underline overline;
text-decoration-thickness: from-font;
font-size: 2rem;
```

```
<section id="default-example">
  <p id="example-element">
    Confusion kissed me on the cheek, and left a taste so bittersweet
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
  text-decoration-color: red;
}
```

Syntax

```
/* Single keyword */
text-decoration-thickness: auto;
text-decoration-thickness: from-font;

/* length */
text-decoration-thickness: 0.1em;
text-decoration-thickness: 3px;

/* percentage */
text-decoration-thickness: 10%;

/* Global values */
text-decoration-thickness: inherit;
text-decoration-thickness: initial;
text-decoration-thickness: revert;
text-decoration-thickness: revert-layer;
text-decoration-thickness: unset;
```

Values

`auto`

The browser chooses an appropriate width for the text decoration line.

`from-font`

If the font file includes information about a preferred thickness, use that value. If the font file doesn't include this information, behave as if `auto` was set, with the browser choosing an appropriate thickness.

`<length>`

Specifies the thickness of the text decoration line as a `<length>`, overriding the font file suggestion or the browser default.

`<percentage>`

Specifies the thickness of the text decoration line as a `<percentage>` of **1em** in the current font. A percentage inherits as a relative value, and so therefore scales with changes in the font. The browser must use a minimum of 1 device pixel. For a given application of this property, the thickness is constant across the whole box it is applied to, even if there are child elements with a different font size.

Formal definition

Initial value	`auto`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Percentages	refer to the font size of the element itself
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
text-decoration-thickness =
  auto                  |
  from-font             |
  <length-percentage>   |
  <line-width>

<length-percentage> =
  <length>      |
  <percentage>
```

```
<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Text Decoration Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Varying thickness

HTML

```
<p class="thin">Here's some text with a 1px red underline.</p>
<p class="thick">This one has a 5px red underline.</p>
<p class="shorthand">This uses the equivalent shorthand.</p>
```

CSS

```
.thin {
  text-decoration-line: underline;
  text-decoration-style: solid;
  text-decoration-color: red;
  text-decoration-thickness: 1px;
}

.thick {
  text-decoration-line: underline;
  text-decoration-style: solid;
  text-decoration-color: red;
  text-decoration-thickness: 5px;
}

.shorthand {
  text-decoration: underline solid red 5px;
}
```

Results

> **Note:** The property used to be called `text-decoration-width`, but was updated in 2019 to `text-decoration-thickness`.

See also

- `text-decoration`
- `text-underline-offset`

text-emphasis CSS property

The **`text-emphasis`** CSS shorthand property applies emphasis marks to text (except spaces and control characters).

Try it

```
text-emphasis: none;
```

```
text-emphasis: filled red;
```

```
text-emphasis: "x";
```

```
text-emphasis: filled double-circle #ffb703;
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}
```

The `text-emphasis` property is quite different from `text-decoration`. The `text-decoration` property does not inherit, and the decoration specified is applied across the whole element. However, text-emphasis does inherit, which means it is possible to change emphasis marks for descendants.

The size of the emphasis symbol, like ruby symbols, is about 50% of the size of the font, and `text-emphasis` may affect line height when the current leading is not enough for the marks.

> **Note:** `text-emphasis` doesn't reset the value of `text-emphasis-position`. This is because if the style and the color of emphasis marks may vary in a text, it is extremely unlikely that their position will. In the very rare cases when this is needed, use the property `text-emphasis-position`.

Constituent properties

This property is a shorthand for the following CSS properties:

- `text-emphasis-color`
- `text-emphasis-style`

Syntax

```
/* Initial value */
text-emphasis: none; /* No emphasis marks */
```

```
/* <string> value */
text-emphasis: "x";
text-emphasis: "点";
text-emphasis: "\25B2";
text-emphasis: "*" #555555;
text-emphasis: "foo"; /* Should NOT use. It may be computed to or rendered as 'f' only */

/* Keywords value */
text-emphasis: filled;
text-emphasis: open;
text-emphasis: filled sesame;
text-emphasis: open sesame;

/* Keywords value combined with a color */
text-emphasis: filled sesame #555555;

/* Global values */
text-emphasis: inherit;
text-emphasis: initial;
text-emphasis: revert;
text-emphasis: revert-layer;
text-emphasis: unset;
```

Values

`none`

No emphasis marks.

`filled`

The shape is filled with solid color. If neither `filled` nor `open` is present, this is the default.

`open`

The shape is hollow.

`dot`

Display small circles as marks. The filled dot is `'•'` (U+2022), and the open dot is `'◦'` (U+25E6).

`circle`

Display large circles as marks. The filled circle is `'●'` (U+25CF), and the open circle is `'○'` (U+25CB). This is the default shape in horizontal writing modes when no other shape is given.

`double-circle`

Display double circles as marks. The filled double-circle is `'◉'` (U+25C9), and the open double-circle is `'◎'` (U+25CE).

`triangle`

Display triangles as marks. The filled triangle is `'▲'` (U+25B2), and the open triangle is `'△'` (U+25B3).

`sesame`

Display sesames as marks. The filled sesame is `'﹅'` (U+FE45), and the open sesame is `'﹆'` (U+FE46). This is the default shape in vertical writing modes when no other shape is given.

`<string>`

Display the given string as marks. Authors should not specify more than one *character* in `<string>`. The UA may truncate or ignore strings consisting of more than one grapheme cluster.

`<color>`

Defines the color of the mark. If no color is present, it defaults to `currentColor`.

Formal definition

Initial value	as each of the properties of the shorthand: • `text-emphasis-style`: `none` • `text-emphasis-color`: `currentcolor`
Applies to	all elements
Inherited	yes
Computed value	as each of the properties of the shorthand: • `text-emphasis-style`: as specified • `text-emphasis-color`: computed color
Animation type	as each of the properties of the shorthand: • `text-emphasis-color`: a color • `text-emphasis-style`: discrete

Formal syntax

```
text-emphasis =
  <'text-emphasis-style'>  ||
  <'text-emphasis-color'>

<text-emphasis-style> =
  none                                                |
  [ [ filled | open ] || [ dot | circle | double-circle | triangle | sesame ] ]  |
  <string>

<text-emphasis-color> =
  <color>
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A heading with emphasis shape and color

This example draws a heading with triangles used to emphasize each character.

CSS

```
h2 {
  text-emphasis: triangle #dd5555;
}
```

HTML

```
<h2>This is important!</h2>
```

Result

See also

- The longhand properties `text-emphasis-style`, `text-emphasis-color`.
- The `text-emphasis-position` property allowing you to define the position of the emphasis marks.

text-emphasis-color CSS property

The **`text-emphasis-color`** CSS property sets the color of emphasis marks. This value can also be set using the `text-emphasis` shorthand.

Try it

```
text-emphasis-color: currentColor;
```

```
text-emphasis-color: red;
```

```
text-emphasis-color: rgb(90 200 160 / 0.8);
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}

#example-element {
  text-emphasis: filled;
}
```

Syntax

```
/* Initial value */
text-emphasis-color: currentColor;

/* <color> */
text-emphasis-color: #555555;
text-emphasis-color: blue;
text-emphasis-color: rgb(90 200 160 / 80%);
text-emphasis-color: transparent;

/* Global values */
text-emphasis-color: inherit;
text-emphasis-color: initial;
text-emphasis-color: revert;
text-emphasis-color: revert-layer;
text-emphasis-color: unset;
```

Values

This property is specified as one `<color>` value:

`<color>`

Defines the color of the emphasis marks. If no color is present, it defaults to `currentColor`.

Formal definition

Initial value	`currentcolor`
Applies to	all elements
Inherited	yes
Computed value	computed color
Animation type	a color

Formal syntax

```
text-emphasis-color =
  <color>
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Emphasis with a color and custom character

CSS

```
em {
  text-emphasis-color: green;
  text-emphasis-style: "*";
}
```

HTML

```
<p>Here's an example:</p>

<em>This has emphasis marks!</em>
```

Result

See also

- The `<color>` data type
- The other emphasis mark related properties: `text-emphasis-style`, `text-emphasis`, and `text-emphasis-position`.
- Other color-related properties: `color`, `background-color`, `border-color`, `outline-color`, `text-shadow`, `caret-color`, and `column-rule-color`

text-emphasis-position CSS property

The **`text-emphasis-position`** CSS property sets where emphasis marks are drawn. Similar to the text rendered by the `<ruby>` HTML element, if there isn't enough room for emphasis marks, the line height is increased.

Try it

```
text-emphasis-position: auto;
```

```
text-emphasis-position: over right;
```

```
text-emphasis-position: under right;
```

```
text-emphasis-position: auto;
writing-mode: vertical-rl;
```

```
text-emphasis-position: over left;
writing-mode: vertical-rl;
```

```
text-emphasis-position: over right;
writing-mode: vertical-rl;
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}

#example-element {
  text-emphasis: filled double-circle #ffb703;
}
```

Syntax

```
/* Initial value */
text-emphasis-position: auto;

/* Keyword values */
text-emphasis-position: over;
text-emphasis-position: under;

text-emphasis-position: over right;
text-emphasis-position: over left;
text-emphasis-position: under right;
text-emphasis-position: under left;

text-emphasis-position: left over;
text-emphasis-position: right over;
text-emphasis-position: right under;
text-emphasis-position: left under;

/* Global values */
text-emphasis-position: inherit;
text-emphasis-position: initial;
text-emphasis-position: revert;
text-emphasis-position: revert-layer;
text-emphasis-position: unset;
```

Values

The property accepts one or two values:

- If only one value is provided, it can be `auto`, `over`, or `under`. When only `over` or `under` is used, `right` is assumed as the default position.
- If two values are provided, they must include one of `over` or `under` and one of `right` or `left`. Their order does not matter.

The values include:

`auto`

Draws marks over the text in horizontal writing mode and to the right of the text in vertical writing mode.

`over`

Draws marks over the text in horizontal writing mode.

`under`

Draws marks under the text in horizontal writing mode.

`right`

Draws marks to the right of the text in vertical writing mode.

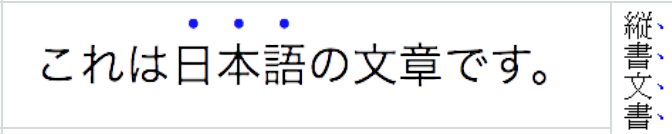
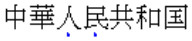
`left`

Draws marks to the left of the text in vertical writing mode.

Description

The preferred position of emphasis marks depends on the language. In Japanese for example, the preferred position is `over right`. In Chinese, on the other hand, the preferred position is `under right`. The informative table below summarizes the preferred emphasis mark positions for Chinese, Mongolian and Japanese:

Preferred emphasis mark and ruby position

Language	Preferred position		Illustration
	Horizontal	Vertical	
Japanese	over	right	
Korean			
Mongolian			
Chinese	under	right	

> **Note:** The `text-emphasis-position` cannot be set, and therefore are not reset either, using the `text-emphasis` shorthand property.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-emphasis-position =
  [ over | under ]  &&
  [ right | left ]?
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adding emphasis mark positions

Use the dropdown menu to change the position of the emphasis marks. This will change the class on the `<section>` element, which in turn, will update the position of the emphasis marks on the text.

HTML

```
<p class="unsupported">
  The <code>auto</code> value is not supported in your browser.
</p>
<label for="position">Emphasis position:</label>
<select id="position">
  <option value="auto">auto</option>
  <option value="over-right">over right</option>
  <option value="over-left">over left</option>
  <option value="under-right">under right</option>
  <option value="under-left">under left</option>
  <option value="preferred">preferred</option>
</select>
```

```
<section id="setting" class="auto">
  <p class="horizontal" lang="zh">你好世界</p>
  <!-- Hello World in Chinese -->
  <p class="vertical" lang="ja">世界、こんにちは。</p>
  <!-- Hello World in Japanese -->
</section>
```

CSS

```
.unsupported {
  color: red;
}
@supports (text-emphasis-position: auto) {
  .unsupported {
    display: none;
  }
}
.horizontal {
  writing-mode: horizontal-tb;
```

```
}
.vertical {
  writing-mode: vertical-rl;
}
section {
  display: flex;
  justify-content: space-around;
}
```

```
section p {
  text-emphasis: filled circle tomato;
  text-emphasis-position: auto;
}
.over-right p,
.preferred p [lang="ja"] {
  text-emphasis-position: over right;
}
.over-left p {
  text-emphasis-position: over left;
}
.under-right p,
.preferred p [lang="zh"] {
  text-emphasis-position: under right;
}
.under-left p {
  text-emphasis-position: under left;
}
.preferred p [lang="ja"] {
}
```

```
const position = document.querySelector("#position");
const setting = document.querySelector("#setting");
const updateClass = () => {
  const currentClass = setting.classList;
  setting.classList.replace(currentClass, position.value);
};
position.addEventListener("change", updateClass);
```

Result

Use the "Emphasis position" dropdown to choose the location of the emphasis marks. The `preferred` option in the dropdown uses the preferred positions, as explained in the Description section.

Preferring ruby over emphasis marks

Some editors prefer to hide emphasis marks when they conflict with ruby. In HTML, this can be done with the following style rule:

```
ruby {
  text-emphasis: none;
}
```

Preferring emphasis marks over ruby

Some other editors prefer to hide ruby when they conflict with emphasis marks. In HTML, this can be done with the following pattern:

```
em {
  text-emphasis: dot; /* Set text-emphasis for <em> elements */
}

em rt {
  display: none; /* Hide ruby inside <em> elements */
}
```

See also

- `text-underline-position`
- `text-emphasis-style`
- `text-emphasis-color`
- `text-emphasis` shorthand property
- `writing-mode`

text-emphasis-style CSS property

The **`text-emphasis-style`** CSS property sets the appearance of emphasis marks. It can also be set, and reset, using the `text-emphasis` shorthand.

Try it

```
text-emphasis-style: none;
```

```
text-emphasis-style: triangle;
```

```
text-emphasis-style: "x";
```

```
text-emphasis-style: filled double-circle;
```

```
<section id="default-example">
  <p>
    I'd far rather be
    <span class="transition-all" id="example-element">happy than right</span>
    any day.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}
```

Syntax

```
/* Initial value */
text-emphasis-style: none; /* No emphasis marks */

/* <string> values */
text-emphasis-style: "x";
text-emphasis-style: "\25B2";
text-emphasis-style: "*";

/* Keyword values */
text-emphasis-style: filled;
text-emphasis-style: open;
text-emphasis-style: dot;
text-emphasis-style: circle;
text-emphasis-style: double-circle;
text-emphasis-style: triangle;
text-emphasis-style: filled sesame;
text-emphasis-style: open sesame;

/* Global values */
text-emphasis-style: inherit;
text-emphasis-style: initial;
text-emphasis-style: revert;
text-emphasis-style: revert-layer;
text-emphasis-style: unset;
```

Values

`none`

No emphasis marks.

`filled`

The shape is filled with solid color. If neither `filled` nor `open` is present, this is the default.

`open`

The shape is hollow.

`dot`

Display small circles as marks. The filled dot is `'•'` (`U+2022`), and the open dot is `'◦'` (`U+25E6`).

`circle`

Display large circles as marks. The filled circle is `'●'` (`U+25CF`), and the open circle is `'○'` (`U+25CB`).

`double-circle`

Display double circles as marks. The filled double-circle is `'◉'` (`U+25C9`), and the open double-circle is `'◎'` (`U+25CE`).

`triangle`

Display triangles as marks. The filled triangle is `'▲'` (`U+25B2`), and the open triangle is `'△'` (`U+25B3`).

`sesame`

Display sesames as marks. The filled sesame is `'﹅'` (`U+FE45`), and the open sesame is `'﹆'` (`U+FE46`).

`<string>`

Display the given string as marks. Authors should not specify more than one *character* in `<string>`. The UA may truncate or ignore strings consisting of more than one grapheme cluster.

Formal definition

Initial value	none
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-emphasis-style =
  none                                          |
  [ [ filled | open ] || [ dot | circle | double-circle | triangle | sesame ] ]  |
  <string>
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
h2 {
  -webkit-text-emphasis-style: sesame;
  text-emphasis-style: sesame;
}
```

See also

- The related properties `text-emphasis-color`, `text-emphasis`.
- The `text-emphasis-position` property allowing you to define the position of the emphasis marks.

text-fit CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`text-fit`** CSS property can be used to scale the rendered font size of text nodes (and other inline content) so that they fit exactly within the inline dimension of their containing boxes, optionally limited by a maximum or minimum **scaling factor**.

Syntax

```
/* One keyword */
text-fit: none;
text-fit: grow;
text-fit: shrink;

/* Two keywords */
text-fit: grow consistent;
text-fit: shrink per-line-all;

/* Keywords and <percentage> */
text-fit: grow 300%;
text-fit: shrink 80%;
text-fit: grow per-line-all 300%;

/* Global values */
text-fit: inherit;
text-fit: initial;
text-fit: revert;
text-fit: revert-layer;
text-fit: unset;
```

Values

This property is specified as a space-separated list of one to three values from the following list:

`none`

The default value. No text scaling is applied.

`grow`

The text node's rendered font size is scaled up until it exactly fits inside its containing box's inline dimension.

`shrink`

The text node's rendered font size is scaled down until it exactly fits inside its containing box's inline dimension.

`consistent`

All of the lines of the text node are scaled using the same scaling factor. This keyword has no effect if `none` is specified as the first keyword. If a second keyword is not specified, `consistent` is assumed.

`per-line`

All of the lines of the text node are scaled with their own scaling factor. The last line of the text node, and any lines that end in a forced break (for example, due to a `<br>` element) have no text scaling applied. This keyword has no effect if `none` is specified as the first keyword.

`per-line-all`

All of the lines of the text node are scaled with their own scaling factor, including the last line and lines that end in a forced break. This keyword has no effect if `none` is specified as the first keyword.

`<percentage>`

Specifies the maximum (when `grow` is specified) or minimum (when `shrink` is specified) scaling factor. This must be `100%` or greater if `grow` is specified, or between `0%` and `100%` inclusive if `shrink` is specified, otherwise the percentage has no effect.

The mandatory first value can be `none`, `grow`, or `shrink`. The optional second value can be `consistent`, `per-line`, or `per-line-all`. The optional `<percentage>` value is declared last. The components must be specified in this order.

Description

A common web design challenge is getting headings and other text features to fit neatly within their containing boxes, regardless of layout or viewport size. The most typical use case is getting a horizontal text heading to fit the width of its containing box perfectly. Complex `font-size` calculations and JavaScript workarounds were historically used to achieve this.

The `text-fit` property provides a convenient CSS-only solution, adjusting the rendered font size of the text by a specific scaling factor to fit the space available, rather than justifying it like the `text-align` property's `justify` value does.

The basic form of `text-fit` uses a single keyword:

- You can specify `grow` to scale the font size up so that the text fits exactly inside its containing box. This works well for the use case described previously.
- You can specify `shrink` to scale the font size down so that the text fits exactly inside its containing box. This works well in cases where you have a line of text that is longer than its containing box, maybe due to a really long word, and you want to shrink it to avoid overflow.

Specifically, the parts of a text node that are affected by this scaling (the **scalable parts**) are the text itself, excluding trailing whitespace, and spacing whose inline size is proportional to the text's `font-size`, such as percentage-based `letter-spacing` and `word-spacing`, and `text-autospace`. Other parts, including inline `border`, `margin`, and `padding`, are not scaled.

The `text-fit` property does not affect a container's intrinsic size, which means that the text content cannot grow in cases where the container is sized by its content, for example using the `fit-content` keyword. Setting `text-fit` also doesn't change an element's computed `font-size`: the size adjustment is applied after the final rendering.

How is the scaling factor calculated?

The scaling factor is the ratio by which the scalable parts of a text line must be scaled in order for its inline content to fit exactly inside its containing box. The scaling factor for each line of a text node is calculated using a formula along these lines (the exact method for determining the scaling factor may differ between implementations):

```
(A + B) / A
```

where:

- `A` is the total inline size of the text line's scalable parts.

- `B` is the remaining space inside the text line, including any trailing whitespace, which may be negative if the text overflows.

The scaling factor for a text line with no scalable parts is `1`.

Specifying scaling factor limits

To limit the amount by which the text can grow, you can specify a `<percentage>` value after the `grow` or `shrink` keyword.

If `grow` is specified, the percentage has to be `100%` or greater, and acts as a maximum scaling factor. For example:

```
text-fit: grow 300%;
```

Any text this is applied to will grow to fit its containing box, but will not scale to a size greater than 300% of its original `font-size`.

If `shrink` is specified, the percentage has to be between `0%` and `100%` inclusive, and acts as a minimum scaling factor. For example:

```
text-fit: shrink 50%;
```

Any text this is applied to will shrink to fit its containing box, but will not scale to a size less than 50% of its original `font-size`.

> **Note:** In many situations, word breaking behavior results in `text-fit: shrink` having no effect — the words wrap onto new lines rather than shrinking. In these cases, you need to stop the breaking behavior using something like a `white-space` value of `nowrap` to see it working. See this in action in our basic example.

Specifying how scaling factor is applied to multiple text lines

By default, a text node with multiple lines scales all of the lines by the same scaling factor. Each line has its scaling factor calculated separately, and the smallest scaling factor is then applied to all of the lines. This is usually the behavior you'll want, to ensure that each line grows or shrinks by the same proportion.

If you want to adjust this behavior, you can specify a second value after the first keyword and before the percentage, if specified. This value can be specified as the default `consistent` keyword, which keeps the behavior described previously, but you can also specify `per-line` or `per-line-all`. Both of these cause the lines of the text node to be scaled with their own scaling factors. The difference is that, with `per-line`, the last line of the text node, and any lines that end in a forced break (for example, due to a `<br>` element) have no text scaling applied, whereas with `per-line-all` scaling is applied to these lines.

For example, this declaration will grow all lines of a text node by their own scaling factor to fit the containing box.

```
text-fit: grow per-line-all;
```

This declaration on the other hand will shrink all lines of a text node by their own scaling factor to fit the containing box, but not the last line or lines with forced breaks, and not below `50%` of the original `font-size`.

```
text-fit: shrink per-line 50%;
```

Accessibility

When using `text-fit`, designs must be tested carefully at different viewport sizes to make sure the rendered font size doesn't become too small (or too large). This can lead to content becoming illegible, especially for people with visual impairments or in low vision conditions.

In any case, text content should be resizable without loss of content or functionality; see WCAG Success Criterion 1.4.4 Resize Text.

Related guidance:

- MDN Understanding WCAG, Guideline 1.4: Make it easier for users to see and hear content including separating foreground from background

Formal definition

Initial value	none
Applies to	block container elements
Inherited	yes
Computed value	specified keyword, or computed `<percentage>`
Animation type	discrete

Formal syntax

```
text-fit =
  [ none | grow | shrink ] [ consistent | per-line | per-line-all ]? <percentage>?
```

This syntax reflects the latest standard as per CSS Text Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>text-fit</code> usage

This example demonstrates basic usage of `text-fit` to grow and shrink text nodes to fit their containers.

HTML

We include two text elements, an `<h1>` and an `<h2>`, nested inside a `<div>`.

```
<input
  type="range"
  min="0.4"
  max="1"
  value="1"
  step="0.01"
  aria-label="adjust the width of the page" />
```

```
<div>
  <h1>My heading</h1>

  <h2>My subtitle: a bit longer, may need shrinking</h2>
</div>
```

```
const divElem = document.querySelector("div");
const slider = document.querySelector("input");
```

```
let maxWidth;

function setMaxWidth() {
  maxWidth = document.documentElement.clientWidth;
}

function setWidth() {
  divElem.style.width = `${maxWidth * slider.value}px`;
}

slider.addEventListener("input", setWidth);
window.addEventListener("resize", () => {
  setMaxWidth();
  setWidth();
});

setMaxWidth();
setWidth();
```

We also include an `<input type="range">` element to enable adjusting the content width to emulate a changing viewport width. The range input, plus the JavaScript that powers it, have been hidden for brevity.

CSS

We give the container `<div>` a `padding` value of `20px` to give the text some space.

```
* {
  box-sizing: border-box;
}

html {
  font-family: sans-serif;
  background-color: white;
}

div {
  margin: 0 auto;
  background-color: coral;
}

input {
  width: 99%;
}
```

```
div {
  padding: 20px;
}
```

We give the `<h1>` a `text-fit` value of `grow`, so that it will grow to fill its container's available inline space. We give the longer `<h2>` a `white-space` value of `nowrap` so that ordinarily, it would all stay on a single line and overflow its container rather than wrapping, if it reaches the container edge. We then set `text-fit: shrink` on it so that, instead of overflowing, it will shrink to fit inside its container's available inline space.

```
h1 {
  text-fit: grow;
}

h2 {
  white-space: nowrap;
  text-fit: shrink;
}
```

Result

Adjust the slider and note how the `<h1>` grows automatically so that it always fills the available space inside its parent `<div>`. Also note how the `<h2>` shrinks so that it always fills the available space inside the `<div>` at narrower widths; at wider widths where it fits inside the `<div>`, it maintains its natural width.

Without `text-fit`, the `<h1>` wouldn't fill the available space, and the `<h2>` would overflow it at narrower widths.

Setting scaling factor percentage limits

This example is very similar to the previous one, except that in this case we limit the amount that our headings can grow or shrink using `<percentage>` values.

The HTML and JavaScript are identical to the previous example.

CSS

We give the `<h1>` a `text-fit` value of `grow 300%`, so that it will grow to fill its container's available inline space, up to 300% of its natural `font-size`:

```
h1 {
  text-fit: grow 300%;
}
```

We give the `<h2>` a `white-space` value of `nowrap` so that it will all stay on a single line and overflow its container rather than wrapping, when it touches the container edge. We then set `text-fit: shrink 80%` on it so that, instead of overflowing, it will shrink to fit inside its container's available inline space, down to `80%` of its natural `font-size`.

```
h2 {
  white-space: nowrap;
  text-fit: shrink 80%;
}
```

One issue that presents itself at this point is that, when the `<div>`'s `width` gets smaller than about `400px`, the `<h2>` will start to overflow its container. To avoid this, we want to set the text to wrap onto new lines normally at this point. To solve the problem, we first give the containing `<div>` a `container-type` of `inline-size`, so that we can use container queries to selectively apply CSS depending on its width.

```
div {
  padding: 20px;
}
```

```
div {
  container-type: inline-size;
}
```

We then use a container query to change the `<h2>`'s `white-space` value to `wrap` when the `<div>`'s `width` gets smaller than `400px`:

```
@container (width < 400px) {
  h2 {
    white-space: wrap;
  }
}
```

Result

Adjust the slider. Note how the `<h1>` grows automatically so that it always fills the available space inside its parent `<div>`, but only up to a certain width. Note how the `<h2>` shrinks so that it always fills the available space inside the `<div>` at narrower widths. When the `<div>` gets narrower than `400px`, the container query kicks in and we set `white-space: wrap`, meaning that the `<h2>` starts to wrap onto multiple lines.

Demonstrating multi-line scaling behavior keywords

This example demonstrates the difference in effect between setting the `consistent`, `per-line`, and `per-line-all` keywords in the `text-fit` property values of different paragraphs.

HTML

The HTML features three `<p>` elements containing the same filler text, each with different `id` values set on them:

```
<div>
  <p id="consistent">
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Iste vitae in id
    esse odit autem quisquam saepe repellendus.
  </p>

  <p id="per-line">
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Iste vitae in id
    esse odit autem quisquam saepe repellendus.
  </p>

  <p id="per-line-all">
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Iste vitae in id
    esse odit autem quisquam saepe repellendus.
  </p>
</div>
```

The HTML and JavaScript for the width adjustment slider are also included in this example.

CSS

We give each paragraph a `text-fit` value with the first keyword set to `grow`; the second keyword in each case is set to a different value — `consistent`, `per-line`, and `per-line-all`, respectively.

```
#consistent {
  text-fit: grow consistent;
}

#per-line {
  text-fit: grow per-line;
}

#per-line-all {
  text-fit: grow per-line-all;
}
```

```
* {
  box-sizing: border-box;
}

:root {
  overflow: hidden;
}

div {
  padding: 20px;
}

p {
  padding: 20px;
  background-color: rgb(0 0 0 / 0.2);
  position: relative;
  margin-bottom: 40px;
}

p::before {
  content: attr(id);
  background-color: black;
  color: white;
  padding: 5px 10px;
  position: absolute;
  top: -28px;
  left: 0px;
}
```

Result

Adjust the slider up and down the scale, taking careful note of the behavior of each paragraph. You should note that:

- The `consistent` paragraph's text lines always have the same `font-size` as one another throughout.
- The `per-line` paragraph's text lines vary in `font-size` somewhat, which becomes more noticeable at narrower widths. The last line's `font-size` is not scaled.
- The `per-line-all` paragraph's text lines vary in `font-size` somewhat, which includes the last line. This is very noticeable at widths that cause only one or two words to wrap onto the last line.

```
@supports not (text-fit: grow) {
  body::before {
    content: "Your browser does not support text-fit.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;
```

```
    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

See also

- CSS text module

text-indent CSS property

The **`text-indent`** CSS property sets the length of empty space (indentation) that is put before lines of text in a block.

Try it

```
text-indent: 0;
```

```
text-indent: 30%;
```

```
text-indent: -3em;
```

```
text-indent: 3em each-line;
```

```
text-indent: 3em hanging;
```

```
text-indent: 3em hanging each-line;
```

```
<section id="default-example">
  <div id="example-element">
    <p>
      This text is contained within a single paragraph. This paragraph is two
      sentences long.
    </p>
    <p>
      This is a new paragraph. There is a line break element
      <code>&lt;br&gt;</code> after this sentence.<br />There it is! Notice how
      it affects the indentation.
    </p>
  </div>
</section>
```

```
section {
  font-size: 1.25em;
  background-color: darkslateblue;
  align-items: start;
}

#example-element {
  text-align: left;
  margin-left: 3em;
  background-color: slateblue;
  color: white;
}
```

Syntax

```
/* <length-percentage> values */
text-indent: 3mm;
text-indent: 40px;
text-indent: 15%;

/* with keyword values */
text-indent: 5em each-line;
text-indent: 5vb hanging;
text-indent: 5% hanging each-line;

/* Global values */
text-indent: inherit;
text-indent: initial;
text-indent: revert;
text-indent: revert-layer;
text-indent: unset;
```

Values

`<length>`

Indentation is specified as an absolute `<length>`. Negative values are allowed. See `<length>` values for possible units.

`<percentage>`

Indentation is a `<percentage>`. The percentage is relative to the container's own inner inline size.

`each-line`

Indentation affects the first line of the block container as well as each line after a *forced line break*, but does not affect lines after a *soft wrap break*.

`hanging`

Inverts which lines are indented. All lines *except* the first line will be indented.

Description

The `text-indent` CSS property sets the length of empty space (indentation) that is put before lines of text in a block container. The indentation set by the property occurs at the inline-start edge of the content box. The value is a `<length-percentage>`, optionally with one or both of the keywords `each-line` and/or `hanging`. The initial value is `0`.

Percent values are relative to the block container's inline-axis inner size, which is the dimension of the content-area only, excluding the container's padding and border.

The `<length-percentage>` can be a positive or negative value. A negative value creates an outdent equal to the absolute value of the `<length-percentage>` equivalent positive value. A negative value effectively pushes the text the size of the value, but in the opposite direction. For example, `text-indent: 3%;` indents the first line of text, adding white space that is 3% of the container's inner-size before the text, pushing the first line of content toward the inline-end direction. Setting `text-indent: -3%` outdents the first line of text, pushing the start of the first line of text 3% of the container's inner-size past the inline-start of the content area, into the padding and border, possibly overflowing the container.

A negative value is different from adding the `hanging` keyword to a positive value. Continuing with the same example, `text-indent: 3% hanging` doesn't outdent the first line of text. Rather, it indents all the lines of text *except* the first line, by 3% of the container's inner-size.

Formal definition

Initial value	`0`
Applies to	block containers
Inherited	yes
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length, plus any keywords as specified
Animation type	a length, percentage or calc();

Formal syntax

```
text-indent =
  [ <length-percentage> ]  &&
  hanging?                 &&
  each-line?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Text Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the basic usage of the `text-indent` property.

HTML

We include two paragraphs of text.

```
<p>
  Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy
  nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.
</p>
<p>
  Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy
  nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.
</p>
```

CSS

We use the `text-indent` property to indent the first line of each paragraph by `5em`.

```
p {
  text-indent: 5em;
  background: powderblue;
}
```

Result

Percentage indent

Using the same HTML as in the previous example, here we demonstrate using percentage values and how percentage values are relative to the element's content area in the inline direction.

```
<p>
  Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy
  nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.
</p>
<p>
  Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy
  nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.
</p>
```

CSS

We set the `text-indent` to a percentage value. We also added padding and vertical stripes to better enable gauging the size of the indentation relative to the element's box model.

```
p {
  text-indent: 30%;

  padding: 30px;
  background-image: repeating-linear-gradient(
    to right,
    transparent 0 9.5%,
    #dedede 9.5% 10%
  );
  background-color: plum;
}
```

Result

Skipping indentation on the first paragraph

A common typographic practice when paragraph indentation is present is to skip the indentation for the first paragraph. As *The Chicago Manual of Style* puts it, "the first line of text following a subhead may begin flush left or be indented by the usual paragraph indention."

Treating first paragraphs differently from subsequent paragraphs can be done using the next-sibling combinator, as in the following example:

HTML

```
<h2>Lorem ipsum</h2>

<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse eu
  venenatis quam. Vivamus euismod eleifend metus vitae pharetra. In vel tempor
  metus. Donec dapibus feugiat euismod. Vivamus interdum tellus dolor. Vivamus
  blandit eros et imperdiet auctor. Mauris sapien nunc, condimentum a efficitur
  non, elementum ac sapien. Cras consequat turpis non augue ullamcorper, sit
  amet porttitor dui interdum.
</p>

<p>
  Sed laoreet luctus erat at rutrum. Proin velit metus, luctus in sapien in,
  tincidunt mattis ex. Praesent venenatis orci at sagittis eleifend. Nulla
  facilisi. In feugiat vehicula magna iaculis vehicula. Nulla suscipit tempor
  odio a semper. Donec vitae dapibus ipsum. Donec libero purus, convallis eu
  efficitur id, pulvinar elementum diam. Maecenas mollis blandit placerat. Ut
  gravida pellentesque nunc, in eleifend ante convallis sit amet.
</p>

<h2>Donec ullamcorper elit nisl</h2>

<p>
  Donec ullamcorper elit nisl, sagittis bibendum massa gravida in. Fusce tempor
  in ante gravida iaculis. Integer posuere tempor metus. Vestibulum lacinia,
  nunc et dictum viverra, urna massa aliquam tellus, id mollis sem velit
  vestibulum nulla. Pellentesque habitant morbi tristique senectus et netus et
  malesuada fames ac turpis egestas. Donec vulputate leo ut iaculis ultrices.
  Cras egestas rhoncus lorem. Nunc blandit tempus lectus, rutrum hendrerit orci
  eleifend id. Ut at quam velit.
</p>

<p>
  Aenean rutrum tempor ligula, at luctus ligula auctor vestibulum. Sed
  sollicitudin velit in leo fringilla sollicitudin. Proin eu gravida arcu. Nam
  iaculis malesuada massa, eget aliquet turpis sagittis sed. Sed mollis tellus
  ac dui ullamcorper, nec lobortis diam pellentesque. Quisque dapibus accumsan
  libero, sed euismod ipsum ullamcorper sed.
</p>
```

CSS

```
p {
  text-align: justify;
  margin: 1em 0 0 0;
}
p + p {
  text-indent: 2em;
  margin: 0;
}
```

Result

See also

- Related CSS properties:
  - `text-justify`
  - `text-orientation`
  - `text-overflow`
  - `text-rendering`
  - `text-transform`
  - `hanging-punctuation`
- CSS text decoration module
- CSS text module
- Learn to style HTML using CSS

text-justify CSS property

The **`text-justify`** CSS property sets what type of justification should be applied to text when `text-align: justify;` is set on an element.

Syntax

```
text-justify: none;
text-justify: auto;
text-justify: inter-word;
text-justify: inter-character;
text-justify: distribute; /* Deprecated value */

/* Global values */
text-justify: inherit;
text-justify: initial;
text-justify: revert;
text-justify: revert-layer;
text-justify: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

The text justification is turned off. This has the same effect as not setting `text-align` at all, although it is useful if you need to turn justification on and off for some reason.

`auto`

The browser chooses the best type of justification for the current situation based on a balance between performance and quality, but also on what is most appropriate for the language of the text (e.g., English, CJK languages, etc.). This is the default justification used if `text-justify` is not set at all.

`inter-word`

The text is justified by adding space between words (effectively varying `word-spacing`), which is most appropriate for languages that separate words using spaces, like English or Korean.

`inter-character`

The text is justified by adding space between characters (effectively varying `letter-spacing`), which is most appropriate for languages like Japanese.

`distribute`

Exhibits the same behavior as `inter-character`; this value is kept for backwards compatibility.

Formal definition

Initial value	`auto`
Applies to	inline-level and table-cell elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-justify =
  [ auto | none | inter-word | inter-character | ruby ]  ||
  no-compress
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Demonstration of the different values of text-justify

```
<p class="none">
  <code>text-justify: none</code> —<br />Lorem ipsum dolor sit amet, consectetur
  adipiscing elit. Nunc ornare maximus vehicula. Duis nisi velit, dictum id
  mauris vitae, lobortis pretium quam. Quisque sed nisi pulvinar, consequat
  justo id, feugiat leo. Cras eu elementum dui.
</p>
<p class="auto">
  <code>text-justify: auto</code> —<br />Lorem ipsum dolor sit amet, consectetur
  adipiscing elit. Nunc ornare maximus vehicula. Duis nisi velit, dictum id
  mauris vitae, lobortis pretium quam. Quisque sed nisi pulvinar, consequat
  justo id, feugiat leo. Cras eu elementum dui.
</p>
<p class="dist">
  <code>text-justify: distribute</code> —<br />Lorem ipsum dolor sit amet,
  consectetur adipiscing elit. Nunc ornare maximus vehicula. Duis nisi velit,
  dictum id mauris vitae, lobortis pretium quam. Quisque sed nisi pulvinar,
  consequat justo id, feugiat leo. Cras eu elementum dui.
</p>
<p class="word">
  <code>text-justify: inter-word</code> —<br />Lorem ipsum dolor sit amet,
  consectetur adipiscing elit. Nunc ornare maximus vehicula. Duis nisi velit,
  dictum id mauris vitae, lobortis pretium quam. Quisque sed nisi pulvinar,
  consequat justo id, feugiat leo. Cras eu elementum dui.
</p>
<p class="char">
  <code>text-justify: inter-character</code> —<br />Lorem ipsum dolor sit amet,
  consectetur adipiscing elit. Nunc ornare maximus vehicula. Duis nisi velit,
  dictum id mauris vitae, lobortis pretium quam. Quisque sed nisi pulvinar,
  consequat justo id, feugiat leo. Cras eu elementum dui.
</p>
```

```
p {
  font-size: 1.5em;
  border: 1px solid black;
  padding: 10px;
  width: 95%;
  margin: 10px auto;
  text-align: justify;
}

.none {
  text-justify: none;
}

.auto {
  text-justify: auto;
}

.dist {
  text-justify: distribute;
}

.word {
  text-justify: inter-word;
}

.char {
  text-justify: inter-character;
}
```

See also

- `text-align`

text-orientation CSS property

The **`text-orientation`** CSS property sets the orientation of the text characters in a line. It only affects text in vertical mode (when `writing-mode` is not `horizontal-tb`). It is useful for controlling the display of languages that use vertical script, and also for making vertical table headers.

Try it

```
writing-mode: vertical-rl;
text-orientation: mixed;
```

```
writing-mode: vertical-rl;
text-orientation: upright;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <p>
      In another moment down went Alice after it, never once considering how in
      the world she was to get out again.
    </p>
  </div>
</section>
```

Syntax

```
/* Keyword values */
text-orientation: mixed;
text-orientation: upright;
text-orientation: sideways-right;
text-orientation: sideways;
text-orientation: use-glyph-orientation;

/* Global values */
text-orientation: inherit;
text-orientation: initial;
text-orientation: revert;
text-orientation: revert-layer;
text-orientation: unset;
```

Values

This property is specified as one of the following keyword values:

`mixed`

> Rotates the characters of horizontal scripts 90° clockwise. Lays out the characters of vertical scripts naturally. Default value.

`upright`

> Lays out the characters of horizontal scripts naturally (upright), as well as the glyphs for vertical scripts. Note that this keyword causes all characters to be considered as left-to-right: the used value of `direction` is forced to be `ltr`.

`sideways`

> Causes characters to be laid out as they would be horizontally, but with the whole line rotated 90° clockwise.

`sideways-right`

> An alias to `sideways` that is kept for compatibility purposes.

`use-glyph-orientation`

> On SVG elements, this keyword leads to use the value of the deprecated SVG properties `glyph-orientation-vertical` and `glyph-orientation-horizontal`.

Formal definition

Initial value	`mixed`
Applies to	all elements, except table row groups, rows, column groups, and columns
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
text-orientation =
  mixed     |
  upright   |
  sideways
```

This syntax reflects the latest standard as per CSS Writing Modes Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p>Lorem ipsum dolet semper quisquam.</p>
```

CSS

```
p {
  writing-mode: vertical-rl;
  text-orientation: upright;
}
```

Result

See also

- The other vertical-script related CSS properties: `writing-mode`, `text-combine-upright`, and `unicode-bidi`.
- CSS Logical properties
- Styling vertical text (Chinese, Japanese, Korean and Mongolian)
- Extensive browsers support test results: https://w3c.github.io/i18n-tests/results/horizontal-in-vertical.html#text_orientation

text-overflow CSS property

The **`text-overflow`** CSS property sets how hidden overflow content is signaled to users.

Try it

```
text-overflow: clip;
```

```
text-overflow: ellipsis;
```

```
text-overflow: "-";
```

```
text-overflow: "";
```

```
<section id="default-example">
  <div id="example-element-container">
    <p id="example-element">"Is there any tea on this spaceship?" he asked.</p>
  </div>
</section>
```

```
#example-element-container {
  width: 100%;
  max-width: 18em;
}

#example-element {
  line-height: 50px;
  border: 1px solid #c5c5c5;
  overflow: hidden;
  white-space: nowrap;
  font-family: sans-serif;
  padding: 0 0.5em;
  text-align: left;
}
```

Syntax

```
text-overflow: clip;
text-overflow: ellipsis ellipsis;
text-overflow: ellipsis " [..]";

/* Global values */
text-overflow: inherit;
text-overflow: initial;
text-overflow: revert;
text-overflow: revert-layer;
text-overflow: unset;
```

Values

This property is specified as one or two space-separated values, including:

`clip`

> The default for this property. This keyword value will truncate the text at the limit of the content area, therefore the truncation can happen in the middle of a character. To clip at the transition between characters you can specify `text-overflow` as an empty string, if that is supported in your target browsers: `text-overflow: '';`.

`ellipsis`

> This keyword value will display an ellipsis (`'…'`, `U+2026 HORIZONTAL ELLIPSIS`) to represent clipped text. The ellipsis is displayed inside the content area, decreasing the amount of text displayed. If there is not enough space to display the ellipsis, it is clipped.

`<string>`

> The `<string>` to be used to represent clipped text. The string is displayed inside the content area, shortening the size of the displayed text. If there is not enough space to display the string itself, it is clipped.

Description

The `text-overflow` property sets how hidden overflow content is signaled to users. It can be clipped, display an ellipsis (…), or display a custom string.

The `text-overflow` property doesn't force an overflow to occur. To make text overflow its container, you have to set other CSS properties: `overflow` and `white-space`. For example:

```
overflow: hidden;
white-space: nowrap;
```

The `text-overflow` property only affects content that is overflowing a block container element in its *inline* progression direction (not text overflowing at the bottom of a box, for example).

The `text-overflow` property may be specified using one or two values. If one value is given, it specifies overflow behavior for the end of the line (the right end for left-to-right text, the left end for right-to-left text). If two values are given, the first specifies overflow behavior for the left end of the line, and the second specifies it for the right end of the line.

<code>ellipsis</code> behaving as <code>clip</code> for editable elements

When `text-overflow: ellipsis` is applied to editable elements such as textual `<input>`, `<textarea>`, and `contenteditable` elements, browsers generally treat it as `text-overflow: clip`, either all the time or when the element is focused. The exact behavior differs across browsers; see Browser compatibility.

This behavior is to ensure that the text can be moved through and edited, even when overflowing. If the overflowing text remained covered by an ellipsis, it would not be visible or editable.

You can see the behavior in the following live example, which has this CSS applied:

```
p[contenteditable],
input,
textarea {
  width: 150px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

```
<p contenteditable>
  contenteditable paragraph. The text in here is overflowing.
</p>

<hr />

<input type="text" value="Text input. The text in here is overflowing." />

<hr />

<textarea>Textarea. The text in here is overflowing.</textarea>
```

Focus the different elements and try editing their text content.

Formal definition

Initial value	`clip`
Applies to	block container elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-overflow =
  [ clip | ellipsis | <string> | fade | <fade()> ]{1,2}

<fade()> =
  fade( [ <length-percentage> ] )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Overflow Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

One-value syntax

This example shows different values for `text-overflow` applied to a paragraph, for left-to-right and right-to-left text.

HTML

```
<div class="ltr">
  <h2>Left to right text</h2>
  <pre>clip</pre>
  <p class="overflow-clip">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  </p>
  <pre>ellipsis</pre>
  <p class="overflow-ellipsis">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  </p>
  <pre>" [..]"</pre>
  <p class="overflow-string">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  </p>
</div>

<div class="rtl">
  <h2>Right to left text</h2>
  <pre>clip</pre>
  <p class="overflow-clip">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  </p>
  <pre>ellipsis</pre>
  <p class="overflow-ellipsis">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  </p>
  <pre>" [..]"</pre>
  <p class="overflow-string">
    Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  </p>
</div>
```

CSS

```
p {
  width: 200px;
  border: 1px solid;
  padding: 2px 5px;

  /* Both of the following are required for text-overflow */
  white-space: nowrap;
  overflow: hidden;
}

.overflow-clip {
```

```
  text-overflow: clip;
}

.overflow-ellipsis {
  text-overflow: ellipsis;
}

.overflow-string {
  text-overflow: " [..]";
}

body {
  display: flex;
  justify-content: space-around;
}

.ltr > p {
  direction: ltr;
}

.rtl > p {
  direction: rtl;
}
```

Result

Two-value syntax

This example shows the two-value syntax for `text-overflow`, where you can define different overflow behavior for the start and end of the text. To show the effect we have to scroll the line so the start of the line is also hidden.

HTML

```
<pre>clip clip</pre>
<p class="overflow-clip-clip">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit.
</p>
<pre>clip ellipsis</pre>
<p class="overflow-clip-ellipsis">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit.
</p>
<pre>ellipsis ellipsis</pre>
<p class="overflow-ellipsis-ellipsis">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit.
</p>
<pre>ellipsis " [..]"</pre>
<p class="overflow-ellipsis-string">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit.
</p>
```

CSS

```
p {
  width: 200px;
  border: 1px solid;
  padding: 2px 5px;

  /* Both of the following are required for text-overflow */
  white-space: nowrap;
  overflow: scroll;
}

.overflow-clip-clip {
  text-overflow: clip clip;
}

.overflow-clip-ellipsis {
  text-overflow: clip ellipsis;
}

.overflow-ellipsis-ellipsis {
  text-overflow: ellipsis ellipsis;
}

.overflow-ellipsis-string {
  text-overflow: ellipsis " [..]";
}
```

JavaScript

```
// Scroll each paragraph so the start is also hidden
const paras = document.querySelectorAll("p");

for (const para of paras) {
  para.scroll(100, 0);
}
```

Result

A previous version of this interface reached the *Candidate Recommendation* status. As some not-listed-at-risk features needed to be removed, the spec was demoted to the *Working Draft* level, explaining why browsers implemented this property unprefixed, though not at the CR state.

See also

- Related CSS properties: `overflow`, `white-space`
- CSS properties that control line breaks in words: `overflow-wrap`, `word-break`

text-rendering CSS property

The **`text-rendering`** CSS property provides information to the rendering engine about what to optimize for when rendering text.

The browser makes trade-offs among speed, legibility, and geometric precision.

> **Note:** The `text-rendering` property is an SVG property that is not defined in any CSS standard. However, Gecko and WebKit browsers let you apply this property to HTML and XML content on Windows, macOS, and Linux.

One very visible effect is `optimizeLegibility`, which enables ligatures (ff, fi, fl, etc.) in text smaller than 20px for some fonts (for example, Microsoft's *Calibri*, *Candara*, *Constantia*, and *Corbel*, or the *DejaVu* font family).

Syntax

```
/* Keyword values */
text-rendering: auto;
text-rendering: optimizeSpeed;
text-rendering: optimizeLegibility;
text-rendering: geometricPrecision;

/* Global values */
text-rendering: inherit;
text-rendering: initial;
text-rendering: revert;
text-rendering: revert-layer;
text-rendering: unset;
```

Values

`auto`

The browser makes educated guesses about when to optimize for speed, legibility, and geometric precision while drawing text. For differences in how this value is interpreted by the browser, see the compatibility table.

The `auto` value is a good default for balancing quality and performance, especially for extended bodies of plain text.

`optimizeSpeed`

The browser emphasizes rendering speed over legibility and geometric precision when drawing text. It disables kerning and ligatures.

The `optimizeSpeed` value is preferable in resource-constrained rendering scenarios, such as slow processors or low battery.

`optimizeLegibility`

The browser emphasizes legibility over rendering speed and geometric precision. This enables kerning and optional ligatures.

The `optimizeLegibility` value is preferable for texts that are large in size but short in content, such as headers or banners, to enhance their readability. It could also be used for high-quality professional typography such as published articles. It's not recommended for typical articles due to potential performance impacts.

`geometricPrecision`

The browser emphasizes geometric precision over rendering speed and legibility. Certain aspects of fonts — such as kerning — don't scale linearly. So this value can make text using those fonts look good.

In SVG, when text is scaled up or down, browsers calculate the final size of the text (which is determined by the specified font size and the applied scale) and request a font of that computed size from the platform's font system. But if you request a font size of, say, 9 with a scale of 140%, the resulting font size of 12.6 doesn't explicitly exist in the font system, so the browser rounds the font size to 12 instead. This results in stair-step scaling of text.

But the `geometricPrecision` property — when fully supported by the rendering engine — lets you scale your text fluidly. For large scale factors, you might see less-than-beautiful text rendering, but the size is what you would expect—neither rounded up nor down to the nearest font size supported by Windows or Linux.

The `geometricPrecision` value optimizes neither readability nor performance. It usually makes sense in SVG, where you want your graphic to scale faithfully without distorting the text dimensions.

> **Note:** WebKit precisely applies the specified value, but Gecko treats the value the same as `optimizeLegibility`.

Formal definition

Initial value	`auto`
Applies to	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-rendering =
  auto               |
  optimizeSpeed      |
  optimizeLegibility |
  geometricPrecision
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Automatic application of optimizeLegibility

This demonstrates how `optimizeLegibility` is used by browsers automatically when the `font-size` is smaller than `20px`.

HTML

```
<p class="small">LYoWAT - ff fi fl ffl</p>
<p class="big">LYoWAT - ff fi fl ffl</p>
```

CSS

```
.small {
  font:
    19.9px "Constantia",
    "Times New Roman",
```

```
    "Georgia",
    "Palatino",
    serif;
}
.big {
  font:
    20px "Constantia",
    "Times New Roman",
    "Georgia",
    "Palatino",
    serif;
}
```

Result

optimizeSpeed vs. optimizeLegibility

This example shows the difference between the appearance of `optimizeSpeed` and `optimizeLegibility` (in your browser; other browsers may vary).

HTML

```
<p class="speed">LYoWAT - ff fi fl ffl</p>
<p class="legibility">LYoWAT - ff fi fl ffl</p>
```

CSS

```
p {
  font:
    1.5em "Constantia",
    "Times New Roman",
    "Georgia",
    "Palatino",
    serif;
}

.speed {
  text-rendering: optimizeSpeed;
}
.legibility {
  text-rendering: optimizeLegibility;
}
```

Result

See also

- Drawing text in a `<canvas>`
- CSS Text Decoration CSS module
- Related CSS properties
  - `text-decoration` (and its longhand properties, such as `text-decoration-line`, `text-decoration-style`, and `text-decoration-thickness`)
  - `text-emphasis` (and its longhand properties, including `text-emphasis-color`, `text-emphasis-position`, and `text-emphasis-style`)
  - `text-shadow`
  - `text-transform`
- The SVG `text-rendering` attribute
- SVG and CSS

text-shadow CSS property

The **`text-shadow`** CSS property adds shadows to text. It accepts a comma-separated list of shadows to be applied to the text and any of its `text-decoration`. Each shadow is described by some combination of X and Y offsets from the element, blur radius, and color.

Try it

```
text-shadow: 1px 1px 2px pink;
```

```
text-shadow: #ffcc00 1px 0 10px;
```

```
text-shadow: 5px 5px #558abb;
```

```
text-shadow: red 2px 5px;
```

```
text-shadow: 5px 10px;
```

```
text-shadow:
  1px 1px 2px red,
  0 0 1em blue,
  0 0 0.2em blue;
```

```
<section id="default-example">
  <p id="example-element">
    Far out in the uncharted backwaters of the unfashionable end of the western
    spiral arm of the Galaxy...
  </p>
</section>
```

```
p {
  font:
    1.5em "Georgia",
    serif;
}
```

Syntax

```
/* offset-x | offset-y | blur-radius | color */
text-shadow: 1px 1px 2px black;

/* color | offset-x | offset-y | blur-radius */
text-shadow: #ffcc00 1px 0 10px;

/* offset-x | offset-y | color */
text-shadow: 5px 5px #558abb;

/* color | offset-x | offset-y */
text-shadow: white 2px 5px;

/* offset-x | offset-y
/* Use defaults for color and blur-radius */
text-shadow: 5px 10px;

/* Global values */
text-shadow: inherit;
text-shadow: initial;
text-shadow: revert;
text-shadow: revert-layer;
text-shadow: unset;
```

This property is specified as a comma-separated list of shadows.

Each shadow is specified as two or three `<length>` values, followed optionally by a `<color>` value. The first two `<length>` values are the `<offset-x>` and `<offset-y>` values. The third, optional, `<length>` value is the `<blur-radius>`. The `<color>` value is the shadow's color.

When more than one shadow is given, shadows are applied front-to-back, with the first-specified shadow on top.

This property applies to both `::first-line` and `::first-letter` pseudo-elements.

Values

`<color>`

> Optional. The color of the shadow. It can be specified either before or after the offset values. If unspecified, the color's value is left up to the user agent, so when consistency across browsers is desired you should define it explicitly.

`<offset-x> <offset-y>`

> Required. These `<length>` values specify the shadow's distance from the text. `<offset-x>` specifies the horizontal distance; a negative value places the shadow to the left of the text. `<offset-y>` specifies the vertical distance; a negative value places the shadow above the text. If both values are `0`, the shadow is placed directly behind the text, although it may be partly visible due to the effect of `<blur-radius>`.

`<blur-radius>`

> Optional. This is a `<length>` value. The higher the value, the bigger the blur; the shadow becomes wider and lighter. If not specified, it defaults to `0`.

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	a color plus three absolute lengths
Animation type	a shadow list

Formal syntax

```
text-shadow =
  none       |
  <shadow>#

<shadow> =
  <color>?                                  &&
  [ <length>{2} [ <length [0,∞]> <length>? ]? ]  &&
  inset?
```

This syntax reflects the latest standard as per CSS Backgrounds and Borders Module Level 3, CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic shadow

```
.red-text-shadow {
  text-shadow: red 0 -2px;
}
```

```
<p class="red-text-shadow">
  Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium
  doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.
</p>
```

Multiple shadows

```
.white-text-with-blue-shadow {
  text-shadow:
    1px 1px 2px black,
    0 0 1em blue,
    0 0 0.2em blue;
  color: white;
  font:
    1.5em "Georgia",
    serif;
}
```

```
<p class="white-text-with-blue-shadow">
  Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium
  doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.
</p>
```

See also

- Introduction to text shadows
- `<color>`
- `box-shadow`
- `drop-shadow()`
- CSS text decoration module

text-size-adjust CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`text-size-adjust`** CSS property controls the text inflation algorithm used on some smartphones and tablets. Other browsers will ignore this property.

Because many websites have not been developed with small devices in mind, mobile browsers differ from desktop browsers in the way they render web pages. Instead of laying out pages at the width of the device screen, they lay them out using a viewport that is much wider, usually 800 or 1000 pixels. To map the extra-wide layout back to the original device size, they either show only part of the whole render or scale the viewport down to fit.

Since text that has been scaled down to fit a mobile screen may be very small, many mobile browsers apply a text inflation algorithm to enlarge the text to make it more readable. When an element containing text uses 100% of the screen's width, the algorithm increases its text size, but without modifying the layout. The `text-size-adjust` property allows web authors to disable or modify this behavior, as web pages designed with small screens in mind do not need it.

Syntax

```
/* Keyword values */
text-size-adjust: none;
text-size-adjust: auto;

/* <percentage> value */
text-size-adjust: 80%;

/* Global values */
text-size-adjust: inherit;
text-size-adjust: initial;
text-size-adjust: revert;
text-size-adjust: revert-layer;
text-size-adjust: unset;
```

Values

This property is specified as one of the following values:

`none`

Disables the browser's inflation algorithm.

`auto`

Enables the browser's inflation algorithm. This value is used to cancel a `none` value previously set with CSS.

`<percentage>`

Enables the browser's inflation algorithm, specifying a percentage value with which to increase the font size.

Formal definition

Initial value	`auto` for smartphone browsers supporting inflation, `none` in other cases (and then not modifiable).
Applies to	all elements
Inherited	yes
Percentages	yes, refer to the corresponding size of the text font
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
text-size-adjust =
  auto               |
  none               |
  <percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Mobile Text Size Adjustment Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic disabling usage

As hinted at above, on a properly designed responsive site the `text-size-adjust` behavior is not needed, so developers can elect to turn it off by specifying a value of none:

```
p {
  -webkit-text-size-adjust: none;
  text-size-adjust: none;
}
```

See also

- Apple's documentation (2016)
- Google Chrome behavior description (2014)

- Gecko's behavior description, by L. David Baron (2011)

text-spacing-trim CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`text-spacing-trim`** CSS property controls the internal spacing set on Chinese/Japanese/Korean (CJK) punctuation characters between adjacent characters (kerning) and at the start or end of text lines.

Syntax

```
/* Keyword values */
text-spacing-trim: normal;
text-spacing-trim: space-all;
text-spacing-trim: space-first;
text-spacing-trim: trim-start;

/* Global values */
text-spacing-trim: inherit;
text-spacing-trim: initial;
text-spacing-trim: revert;
text-spacing-trim: revert-layer;
text-spacing-trim: unset;
```

Values

`<spacing-trim>`

Defines the different spacing trim options. Available values are:

`normal`

Sets CJK full-width opening punctuation characters to be full-width at the start of each line. Sets CJK full-width closing punctuation characters to be full-width at the end of each line, or half-width if they do not fit on the line before justification. Collapses spacing between punctuation characters.

`space-all`

All CJK full-width punctuation characters are set to be full-width.

`space-first`

Behaves as `normal`, except that CJK full-width opening punctuation characters are set to be full-width at the start of the first line of the text's block container, and the start of every subsequent line coming after an explicit line break such as a newline character.

`trim-start`

Behaves as `normal`, except that CJK full-width opening punctuation characters are set to be half-width at the start of each line.

> **Note:** The CSS Text module also defines `trim-both`, `trim-all`, and `auto` values. However, these are not currently implemented in any browser.

Description

The `text-spacing-trim` property applies spacing/kerning to CJK punctuation characters to produce visually pleasing typography as defined by the Requirements for Japanese text layout (JLREQ) and the Requirements for Chinese text layout (CLREQ).

Many CJK punctuation characters include glyph-internal spacing. For example, the CJK full stop and the CJK close parenthesis usually have glyph-internal spacing on their right-hand side, to give them a constant advance measure consistent with other ideographic characters. However, when they appear in a row, the glyph-internal spacing can become excessive.

`text-spacing-trim` can be used to adjust such excessive spacing between adjacent characters (kerning) and at the start or end of text lines. Generally speaking:

- If a full-width punctuation character is set to be full-width, it has internal spacing set on both sides and is the full width of an ideograph.
- If a full-width punctuation character is set to be half-width, it has internal spacing set on one side only, and its other side is flush to the start (in the case of opening punctuation characters) or end (in the case of closing punctuation characters). Half-width characters are typically half the width of an ideograph.

> **Note:** To avoid the risk of excessive kerning, fonts must have the OpenType Alternate Half Widths (`halt`) feature, the Contextual Half-width Spacing (`chws`) feature, or both. If the font doesn't have either feature, `text-spacing-trim` is disabled.

Full-width punctuation collapsing

When pairs of punctuation characters are adjacent to one another, the space between them is collapsed according to the following rules:

- Set a full-width opening punctuation character to half-width if the previous character is a fullwidth opening punctuation character, a fullwidth middle dot, an ideographic space (U+3000), a fullwidth closing punctuation character of an equivalent or larger font size, or a character belonging to Unicode general category "Open punctuation" Ps. Otherwise, set it to full-width.
- Set a full-width closing punctuation character to half-width if the next character is a fullwidth closing punctuation character, a fullwidth middle dot, an ideographic space (U+3000), a fullwidth opening punctuation character with larger font size, or a character belonging to Unicode general category "Close punctuation" (Pe). Otherwise, set it to full-width.

Formal definition

Initial value	`normal`
Applies to	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-spacing-trim =
  <spacing-trim>  |
  auto

<spacing-trim> =
  space-all    |
```

```
  normal       |
  space-first  |
  trim-start   |
  trim-both    |
  trim-all
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

<code>text-spacing-trim</code> value comparison

This example compares the effect of four different `text-spacing-trim` properties, applying them to four identical paragraphs so you can see the visual differences between each one.

HTML

```
<main>
  <div id="normal">
    <h2>normal</h2>
    <p>
      (東)、【「(東)」】。<br />
      「東」「東」「東」東東東「東」。
    </p>
  </div>
  <div id="space-all">
    <h2>space-all</h2>
    <p>
      (東)、【「(東)」】。<br />
      「東」「東」「東」東東東「東」。
    </p>
  </div>
  <div id="space-first">
    <h2>space-first</h2>
    <p>
      (東)、【「(東)」】。<br />
      「東」「東」「東」東東東「東」。
    </p>
  </div>
  <div id="trim-start">
    <h2>trim-start</h2>
    <p>
      (東)、【「(東)」】。<br />
      「東」「東」「東」東東東「東」。
    </p>
  </div>
</main>
```

CSS

```
main {
  font-family:
    "Yu Gothic", "YuGothic", "Noto Sans JP", "Hiragino Sans",
    "Hiragino Kaku Gothic ProN", sans-serif;
  display: grid;
  gap: 0.5em;
  grid-template-columns: 1fr 1fr;
  width: 70%;
  margin: 0 auto;
}
h2 {
  font-size: 80%;
  margin: 0;
}
p {
  font-size: 20px;
  border: 1px solid blue;
  margin: 0;
}
#normal {
  text-spacing-trim: normal;
  grid-column: 1;
  grid-row: 1;
}
#space-all {
  text-spacing-trim: space-all;
  grid-column: 2;
  grid-row: 1;
}
#space-first {
  text-spacing-trim: space-first;
  grid-column: 1;
  grid-row: 2;
}
#trim-start {
  text-spacing-trim: trim-start;
  grid-column: 2;
  grid-row: 2;
}
```

Result

See also

- `text-autospace`
- `ic` and `ric` units
- CSS text module

text-transform CSS property

The **`text-transform`** CSS property specifies how to capitalize an element's text. It can be used to make text appear in all-uppercase or all-lowercase, or with each word capitalized. It also can help improve legibility for ruby.

Try it

```
text-transform: capitalize;
```

```
text-transform: uppercase;
```

```
text-transform: lowercase;
```

```
text-transform: none;
```

```
text-transform: full-width;
```

```
text-transform: full-size-kana;
```

```
text-transform: math-auto;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element">
    <p>
      LONDON. Michaelmas term lately over, and the Lord Chancellor sitting in
      Lincoln's Inn Hall.
    </p>
    <p lang="el">
      Σ is a Greek letter and appears in ΟΔΥΣΣΕΥΣ. Θα πάμε στο "Θεϊκό φαΐ" ή στη
      "Νεράιδα"
    </p>
    <p lang="ja">アイウエ オカクケ</p>
  </div>
</section>
```

```
#example-element {
  font-size: 1.2em;
}
```

The `text-transform` property takes into account language-specific case mapping rules such as the following:

- In Turkic languages, like Turkish (`tr`), Azerbaijani (`az`), Crimean Tatar (`crh`), Volga Tatar (`tt`), and Bashkir (`ba`), there are two kinds of `i`, with and without the dot, and two case pairings: `i`/`İ` and `ı`/`I`.
- In German (`de`), the `ß` becomes `SS` in uppercase.
- In Dutch (`nl`), the `ij` digraph becomes `IJ`, even with `text-transform: capitalize`, which only puts the first letter of a word in uppercase.
- In Greek (`el`), vowels lose their accent when the whole word is in uppercase (`ά`/`Α`), except for the disjunctive eta (`ή`/`Η`). Also, diphthongs with an accent on the first vowel lose the accent and gain a diaeresis on the second vowel (`άι`/`ΑΪ`).
- In Greek (`el`), the lowercase sigma character has two forms: `σ` and `ς`. `ς` is used only when sigma terminates a word. When applying `text-transform: lowercase` to an uppercase sigma (`Σ`), the browser needs to choose the right lowercase form based on context.
- in Irish (`ga`), certain prefixed letters remain in lowercase when the base initial is capitalized, so for example `text-transform: uppercase` will change `ar aon tslí` to `AR AON tSLÍ` and not, as one might expect, `AR AON TSLÍ` (Firefox only). In some cases, a hyphen is also removed upon uppercasing: `an t-uisce` transforms to `AN tUISCE` (and the hyphen is correctly reinserted by `text-transform: lowercase`).

The language is defined by the `lang` HTML attribute or the `xml:lang` XML attribute.

> **Note:** Support for language-specific cases varies between browsers, so check the browser compatibility table.

Syntax

```
/* Keyword values */
text-transform: none;
text-transform: capitalize;
text-transform: uppercase;
text-transform: lowercase;
text-transform: full-width;
text-transform: full-size-kana;
text-transform: math-auto;

/* Global values */
text-transform: inherit;
text-transform: initial;
text-transform: revert;
text-transform: revert-layer;
text-transform: unset;
```

`capitalize`

Is a keyword that converts the first *letter* of each word to uppercase. Other characters remain unchanged (they retain their original case as written in the element's text). A letter is defined as a character that is part of Unicode's Letter or Number general categories; thus, any punctuation marks or symbols at the beginning of a word are ignored.

> **Warning:** In many languages, it is considered incorrect to capitalize every word in a sentence, even in titles and names. If you don't know the language of the text the style will be applied to, you should avoid using `capitalize`. Furthermore, `capitalize` does not create title casing, because it does not apply language-specific rules such as skipping articles in English.

> **Note:** The `capitalize` keyword was under-specified in CSS 1 and CSS 2.1. This resulted in differences between browsers in the way the first letter was calculated (Firefox considered `-` and `_` as letters, but other browsers did not. Both WebKit and Gecko incorrectly considered letter-based symbols like ⓐ to be real letters.) By precisely defining the correct behavior, CSS Text Level 3 cleans this mess up. The `capitalize` line in the browser compatibility table contains the version the different engines started to support this now precisely-defined behavior.

`uppercase`

Is a keyword that converts all characters to uppercase.

`lowercase`

Is a keyword that converts all characters to lowercase.

`none`

Is a keyword that prevents the case of all characters from being changed.

`full-width`

Is a keyword that forces the writing of a character — mainly ideograms and Latin scripts — inside a square, allowing them to be aligned in the usual East Asian scripts (like Chinese or Japanese).

`full-size-kana`

Generally used for `<ruby>` annotation text, the keyword converts all small Kana characters to the equivalent full-size Kana, to compensate for legibility issues at the small font sizes typically used in ruby.

`math-auto`

Used to automatically render text in math italic where appropriate. It transforms Latin and Greek letters, and a few other math-related symbols, to italic mathematical symbols but only if it's applied on a text node containing a single character. For example, "x" will become "x" (U+1D465), but "exp" will stay as "exp". It is primarily used to specify the behavior of `<mi>` elements in MathML. You should generally use MathML markup which automatically applies the right styling.

Accessibility

Large sections of text set with a `text-transform` value of `uppercase` may be difficult for people with cognitive concerns such as Dyslexia to read.

- MDN Understanding WCAG, Guideline 1.4 explanations
- W3C Understanding WCAG 2.2

Formal definition

Initial value	none
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-transform =
  none                                          |
  [ capitalize | uppercase | lowercase ] || full-width || full-size-kana  |
  math-auto
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Example using "none"

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit…</strong>
</p>
<p>
  text-transform: none
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit…</span
    ></strong
  >
</p>
```

```
span {
  text-transform: none;
}
strong {
  float: right;
}
```

This demonstrates no text transformation.

Example using "capitalize" (general)

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit…</strong>
</p>
<p>
  text-transform: capitalize
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit…</span
    ></strong
  >
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates text capitalization.

Example using "capitalize" (punctuation)

```
<p>
  Initial String
  <strong
    >(this) "is" [a] -short- -test- «for» *the* _css_ ¿capitalize?
    ?¡transform!</strong
  >
</p>
<p>
```

```
  text-transform: capitalize
  <strong
    ><span
      >(this) "is" [a] -short- -test- «for» *the* _css_ ¿capitalize?
      ?¡transform!</span
    ></strong
  >
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates how initial punctuations of a word are ignored. The keyword target the first letter, that is the first Unicode character part of the Letter or Number general category.

Example using "capitalize" (Symbols)

```
<p>
  Initial String
  <strong>ⓐⓑⓒ (ⓓⓔⓕ) —Ⓖⓗⓘ— ⓙkl</strong>
</p>
<p>
  text-transform: capitalize
  <strong><span>ⓐⓑⓒ (ⓓⓔⓕ) —Ⓖⓗⓘ— ⓙkl</span></strong>
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates how initial symbols are ignored. The keyword target the first letter, that is the first Unicode character part of the Letter or Number general category.

Example using "capitalize" (Dutch ij digraph)

```
<p>
  Initial String
  <strong lang="nl">The Dutch word: "ijsland" starts with a digraph.</strong>
</p>
<p>
  text-transform: capitalize
  <strong
    ><span lang="nl"
      >The Dutch word: "ijsland" starts with a digraph.</span
    ></strong
  >
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates how the Dutch *ij* digraph must be handled like one single letter.

Example using "uppercase" (general)

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit…</strong>
</p>
<p>
  text-transform: uppercase
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit…</span
    ></strong
  >
</p>
```

```
span {
  text-transform: uppercase;
}
strong {
  float: right;
}
```

This demonstrates transforming the text to uppercase.

Example using "uppercase" (Greek vowels)

```
<p>
  Initial String
  <strong>Θα πάμε στο "Θεϊκό φαΐ" ή στη "Νεράιδα"</strong>
</p>
<p>
  text-transform: uppercase
  <strong
    ><span lang="el">Θα πάμε στο "Θεϊκό φαΐ" ή στη "Νεράιδα"</span></strong
  >
</p>
```

```
span {
  text-transform: uppercase;
}
strong {
  float: right;
}
```

This demonstrates how Greek vowels except disjunctive *eta* should have no accent, and the accent on the first vowel of a vowel pair becomes a diaeresis on the second vowel.

Example using "lowercase" (general)

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit…</strong>
</p>
<p>
  text-transform: lowercase
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit…</span
    ></strong
  >
</p>
```

```
span {
  text-transform: lowercase;
}
strong {
  float: right;
}
```

This demonstrates transforming the text to lowercase.

Example using "lowercase" (Greek Σ)

```
<p>
  Initial String
  <strong>Σ IS A greek LETTER that appears SEVERAL TIMES IN ΟΔΥΣΣΕΥΣ.</strong>
</p>
<p>
  text-transform: lowercase
  <strong
    ><span
      >Σ IS A greek LETTER that appears SEVERAL TIMES IN ΟΔΥΣΣΕΥΣ.</span
    ></strong
  >
</p>
```

```
span {
  text-transform: lowercase;
}
strong {
  float: right;
}
```

This demonstrates how the Greek character sigma (Σ) is transformed into the regular lowercase sigma (σ) or the word-final variant (ς), according the context.

Example using "lowercase" (Lithuanian)

```
<p>
  Initial String
  <strong>Į is a Lithuanian LETTER as is j́</strong>
</p>
<p>
  text-transform: lowercase
  <strong><span lang="lt">Į is a Lithuanian LETTER as is j́</span></strong>
</p>
```

```
span {
  text-transform: lowercase;
}
strong {
  float: right;
}
```

This demonstrates how the Lithuanian letters į and j́ retain their dot when transformed to lowercase.

Example using "full-width" (general)

```
<p>
  Initial String
  <strong
    >0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&()*+,-./:;&lt;=&gt;?@{|}~</strong
  >
</p>
<p>
  text-transform: full-width
  <strong
    ><span
      >0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&()*+,-./:;&lt;=&gt;?@{|}~</span
    ></strong
  >
</p>
```

```
span {
  text-transform: full-width;
}
strong {
  width: 100%;
```

```
  float: right;
}
```

Some characters exist in two formats: normal width and a full-width, with different Unicode code points. The full-width version is used to mix them smoothly with Asian ideographic characters.

Example using "full-width" (Japanese half-width katakana)

```
<p>
  Initial String
  <strong>ｳｪﾌﾞ ﾌﾟﾛｸﾞﾗﾐﾝｸﾞ の勉強</strong>
</p>
<p>
  text-transform: full-width
  <strong><span>ｳｪﾌﾞ ﾌﾟﾛｸﾞﾗﾐﾝｸﾞ の勉強</span></strong>
</p>
```

```
span {
  text-transform: full-width;
}
strong {
  width: 100%;
  float: right;
}
```

The Japanese half-width katakana was used to represent katakana in 8-bit character codes. Unlike regular (full-width) katakana characters, a letter with dakuten (voiced sound mark) is represented as two code points, the body of letter and dakuten. The `full-width` combines these into a single code point when converting these characters into full-width.

Example using "full-size-kana"

```
<p>ｱｲｳｴ ｵｶｸｹ ｼｽｿ ﾇﾊﾋﾌ ﾍﾎﾑﾔ ﾕﾖﾗﾘ ﾙﾚﾛﾜ</p>
<p>ｱｲｳｴ ｵｶｸｹ ｼｽｿ ﾇﾊﾋﾌ ﾍﾎﾑﾔ ﾕﾖﾗﾘ ﾙﾚﾛﾜ</p>
```

```
p:nth-of-type(2) {
  text-transform: full-size-kana;
}
```

Example using "math-auto"

In this example, we use pure HTML markup to create a math formula:

```
<div>
  (<span class="math-id">sin</span> <span class="math-id">x</span>)<sup
    >2</sup
  >
  + (<span class="math-id">cos</span> <span class="math-id">x</span>)<sup
    >2</sup
  >
  = 1
</div>
```

We give every `.math-id` element `text-transform: math-auto`. However, note how only the `x` characters become italic, while the `sin` and `cos` remain unchanged.

```
.math-id {
  text-transform: math-auto;
}
```

However, you are encouraged to use MathML for mathematical formulas, as it provides a more robust and accessible way to represent mathematical content. Here's the same formula using MathML:

```
<math xmlns="http://www.w3.org/1998/Math/MathML" display="block">
  <semantics>
    <mrow>
      <mo stretchy="false">(</mo>
      <mo lspace="0em" rspace="0em">sin</mo>
      <mspace width="0.16666666666666666em"></mspace>
      <mi>x</mi>
      <msup>
        <mo stretchy="false">)</mo>
        <mn>2</mn>
      </msup>
      <mo>+</mo>
      <mo stretchy="false">(</mo>
      <mo lspace="0em" rspace="0em">cos</mo>
      <mspace width="0.16666666666666666em"></mspace>
      <mi>x</mi>
      <msup>
        <mo stretchy="false">)</mo>
        <mn>2</mn>
      </msup>
      <mo>=</mo>
      <mn>1</mn>
    </mrow>
    <annotation encoding="TeX">(\sin\,x)^2+(\cos\,x)^2=1</annotation>
  </semantics>
</math>
```

See also

- `font-variant`

text-underline-offset CSS property

The **`text-underline-offset`** CSS property sets the offset distance of an underline text decoration line (applied using `text-decoration`) from its original position.

`text-underline-offset` is not part of the `text-decoration` shorthand. While an element can have multiple `text-decoration` lines, `text-underline-offset` only impacts underlining, and **not** other possible line decoration options such as `overline` or `line-through`.

Try it

```
text-underline-offset: auto;
```

```
text-underline-offset: 8px;
```

```
text-underline-offset: -0.5rem;
```

```
<section id="default-example">
  <p id="example-element">And after all we are only ordinary</p>
</section>
```

```
p {
  font: 1.5em sans-serif;
  text-decoration-line: underline;
  text-decoration-color: red;
}
```

Syntax

```
/* Single keyword */
text-underline-offset: auto;

/* length */
text-underline-offset: 0.1em;
text-underline-offset: 3px;

/* percentage */
text-underline-offset: 20%;

/* Global values */
text-underline-offset: inherit;
text-underline-offset: initial;
text-underline-offset: revert;
text-underline-offset: revert-layer;
text-underline-offset: unset;
```

Values

This property is specified as one of the following values:

`auto`

The browser chooses the appropriate offset for underlines.

`<length>`

Specifies the offset of underlines as a `<length>`, overriding the font file suggestion and the browser default. It is recommended to use `em` units so the offset scales with the font size.

`<percentage>`

Specifies the offset of underlines as a `<percentage>` of **1 em** in the element's font. A percentage inherits as a relative value, and so therefore scales with changes in the font. For a given application of this property, the offset is constant across the whole box that the underline is applied to, even if there are child elements with different font sizes or vertical alignment.

Formal definition

Initial value	`auto`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Percentages	refer to the font size of the element itself
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
text-underline-offset =
  auto                |
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Demonstration of text-underline-offset

```
<p class="one-line">Here's some text with an offset wavy red underline!</p>
<br />
<p class="two-lines">
  This text has lines both above and below it. Only the bottom one is offset.
</p>
```

```
p {
  text-decoration: underline wavy red;
  text-underline-offset: 1em;
}

.two-lines {
  text-decoration-color: purple;
  text-decoration-line: underline overline;
}
```

See also

- `text-decoration`
- `text-decoration-thickness`

text-underline-position CSS property

The **`text-underline-position`** CSS property specifies the position of the underline which is set using the `text-decoration` property's `underline` value.

Try it

```
text-underline-position: auto;
```

```
text-underline-position: under;
```

```
<section id="default-example">
  <p>
    <span class="transition-all" id="example-element"
      >C<sub>8</sub>H<sub>10</sub>N<sub>4</sub>O<sub>2</sub></span
    >
    is the chemical formula for caffeine.
  </p>
</section>
```

```
p {
  font: 1.5em sans-serif;
}

#example-element {
  text-decoration-line: underline;
}
```

Syntax

```
/* Single keyword */
text-underline-position: auto;
text-underline-position: under;
text-underline-position: left;
text-underline-position: right;

/* Multiple keywords */
text-underline-position: under left;
text-underline-position: right under;

/* Global values */
text-underline-position: inherit;
text-underline-position: initial;
text-underline-position: revert;
text-underline-position: revert-layer;
text-underline-position: unset;
```

Values

This property is specified as one of the following keyword values:

`auto`

> The user agent uses its own algorithm to place the line at or under the alphabetic baseline.

`from-font`

> If the font file includes information about a preferred position, use that value. If the font file doesn't include this information, behave as if `auto` was set, with the browser choosing an appropriate position.

`under`

> Forces the line to be set below the alphabetic baseline, at a position where it won't cross any descenders. This is useful for ensuring legibility with chemical and mathematical formulas, which make a large use of subscripts.

`left`

> In vertical writing-modes, this keyword forces the line to be placed on the *left* side of the text. In horizontal writing-modes, it is a synonym of `auto`.

`right`

> In vertical writing-modes, this keyword forces the line to be placed on the *right* side of the text. In horizontal writing-modes, it is a synonym of `auto`.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-underline-position =
  auto                                |
  [ from-font | under ] || [ left | right ]
```

This syntax reflects the latest standard as per CSS Text Decoration Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A basic example

We create two example paragraphs:

```
<p class="horizontal">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam consectetur ac
  turpis vel laoreet. Nullam volutpat pharetra lorem, sit amet feugiat tortor
  volutpat quis. Nam eget sodales quam. Aliquam accumsan tellus ac erat posuere.
</p>

<p class="vertical">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam consectetur ac
  turpis vel laoreet. Nullam volutpat pharetra lorem, sit amet feugiat tortor
  volutpat quis. Nam eget sodales quam. Aliquam accumsan tellus ac erat posuere.
</p>
```

Our CSS looks like this:

```
p {
  font-size: 1.5rem;
  text-transform: capitalize;
  text-decoration: underline;
  text-decoration-thickness: 2px;
}

.horizontal {
  text-underline-position: under;
}

.vertical {
  writing-mode: vertical-rl;
  text-underline-position: left;
}
```

In this example we set both the paragraphs to have a thick underline. In the horizontal text we use `text-underline-position: under;` to put the underline below all the descenders.

In the text with a vertical `writing-mode` set, we can then use values of `left` or `right` to make the underline appear on the left or right of the text as required.

The live example looks like this:

Setting text-underline-position globally

Because the `text-underline-position` property inherits and is not reset by the `text-decoration` shorthand property, it may be appropriate to set its value at a global level. For example, the `under` value may be appropriate for a document with lots of chemical and mathematical formulas, which make a large use of subscripts.

```
:root {
  text-underline-position: under;
}
```

See also

- The `text-decoration` property is a shorthand for setting most text-decoration properties, including `text-decoration-line`, `text-decoration-color`, and `text-decoration-style`. However, it does not set `text-underline-position`.

text-wrap CSS property

The **`text-wrap`** CSS shorthand property controls how text inside an element is wrapped. The different values provide:

- Typographic improvements, for example more balanced line lengths across broken headings
- A way to turn text wrapping off completely.

Try it

```
text-wrap: wrap;
```

```
text-wrap: nowrap;
```

```
text-wrap: balance;
```

```
text-wrap: pretty;
```

```
text-wrap: stable;
```

```
<section class="default-example" id="default-example">
  <div class="whole-content-wrapper">
    <p>Edit the text in the box:</p>
    <div class="transition-all" id="example-element">
      <p contenteditable>
        Lorem ipsum dolor sit amet consectetur adipisicing elit. Voluptatem aut
        cum eum id quos est.
      </p>
    </div>
  </div>
</section>
```

```
.whole-content-wrapper {
  display: flex;
  flex-direction: column;
  align-items: center;
  width: 100%;
}

#example-element {
  border: 1px solid #c5c5c5;
  width: 250px;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `text-wrap-mode`
- `text-wrap-style`

Syntax

```
/* Keyword values */
text-wrap: wrap;
text-wrap: nowrap;
text-wrap: balance;
text-wrap: pretty;
text-wrap: stable;

/* Global values */
text-wrap: inherit;
text-wrap: initial;
text-wrap: revert;
text-wrap: revert-layer;
text-wrap: unset;
```

Values

This property is specified as one of the following keyword values:

`wrap`

Text is wrapped across lines at appropriate characters (for example spaces, in languages like English that use space separators) to minimize overflow. This is the default value.

`nowrap`

Text does not wrap across lines. It will overflow its containing element rather than breaking onto a new line.

`balance`

Text is wrapped in a way that best balances the number of characters on each line, enhancing layout quality and legibility. Because counting characters and balancing them across multiple lines is computationally expensive, this value is only supported for blocks of text spanning a limited number of lines (six or less for Chromium and ten or less for Firefox).

`pretty`

Results in the same behavior as `wrap`, except that the user agent will use a slower algorithm that favors better layout over speed. This is intended for body copy where good typography is favored over performance (for example, when the number of orphans should be kept to a minimum).

`stable`

Results in the same behavior as `wrap`, except that when the user is editing the content, the lines that come before the lines they are editing remain static rather than the whole block of text re-wrapping.

Description

There are 2 ways that text can flow across lines within a block of content, such as a paragraph (`<p>`) or headings (`<h1>`–`<h6>`). These are *forced line breaks*, that are controlled by the user, and *soft line breaks*, that are controlled by the browser. The `text-wrap` property can be used to prompt the browser how to control the *soft line breaks*.

The value you choose, for `text-wrap`, depends on how many lines of text you anticipate styling, whether the text is `contenteditable`, and whether you need to prioritize appearance or performance.

When the styled content will be limited to a short number of lines, such as headings, captions, and blockquotes, `text-wrap: balance` can be added to balance the number of characters on each line, enhancing layout quality and legibility. As browsers limit the number of lines impacted by this property, this value's impact on performance is negligible.

For longer sections of text, `text-wrap: pretty` can be used. Note that `pretty` has a negative effect on performance, so it should be only used for longer blocks of text when the layout is more important than speed.

The `stable` value improves user experience when used on content that is `contenteditable`. This value ensures that, as the user is editing text, the previous lines in the area being edited remain stable.

Formal definition

Initial value	`wrap`
Applies to	text and block containers
Inherited	yes
Percentages	as each of the properties of the shorthand: • `text-wrap-mode`: no • `text-wrap-style`: no
Computed value	as each of the properties of the shorthand: • `text-wrap-mode`: as specified • `text-wrap-style`: as specified
Animation type	as each of the properties of the shorthand: • `text-wrap-mode`: discrete • `text-wrap-style`: discrete

Formal syntax

```
text-wrap =
  <'text-wrap-mode'>   ||
  <'text-wrap-style'>

<text-wrap-mode> =
  wrap    |
  nowrap

<text-wrap-style> =
  auto                  |
  balance               |
  stable                |
  pretty                |
  avoid-short-last-line
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic text wrap value comparison

HTML

```
<h2 class="wrap" contenteditable="true">
  The default behavior; the text in the heading wraps "normally"
</h2>

<h2 class="nowrap" contenteditable="true">
  In this case the text in the heading doesn't wrap, and overflows the container
</h2>

<h2 class="balance" contenteditable="true">
  In this case the text in the heading is nicely balanced across lines
</h2>
```

CSS

```
.wrap {
  text-wrap: wrap;
}

.nowrap {
  text-wrap: nowrap;
}

.balance {
  text-wrap: balance;
}

h2 {
  font-size: 2rem;
  font-family: sans-serif;
}
```

Result

The text in the example is editable. Change the text, adding long words, to view how the different line and word lengths impact wrapping.

See also

- `white-space`
- `white-space-collapse`
- CSS text module
- CSS `text-wrap: balance` on developer.chrome.com (2023)
- CSS `text-wrap: pretty` on developer.chrome.com (2023)
- Balancing Japanese and Korean typography by Kelly Choyce-Dwan (2025)

text-wrap-mode CSS property

The **`text-wrap-mode`** CSS property controls whether the text inside an element is wrapped.

Try it

```
text-wrap-mode: wrap;
```

```
text-wrap-mode: nowrap;
```

```
<section class="default-example" id="default-example">
  <div class="whole-content-wrapper">
    <p>Edit the text in the box:</p>
    <div class="transition-all" id="example-element">
      <p contenteditable>
        You can edit this lorem ipsum dolor sit amet consectetur adipisicing
        elit. Voluptatem aut cum eum id quos est text.
      </p>
    </div>
  </div>
</section>
```

```
.whole-content-wrapper {
  display: flex;
  flex-direction: column;
  align-items: center;
  width: 100%;
}

#example-element {
  border: 1px solid #c5c5c5;
  width: 250px;
}
```

Syntax

```
/* Keyword values */
text-wrap-mode: wrap;
text-wrap-mode: nowrap;

/* Global values */
text-wrap-mode: inherit;
text-wrap-mode: initial;
text-wrap-mode: revert;
text-wrap-mode: revert-layer;
text-wrap-mode: unset;
```

Values

The `text-wrap-mode` property is specified as one of the following keyword values:

`wrap`

Text is wrapped across lines at appropriate characters (for example spaces, in languages like English that use space separators) to minimize overflow. This is the default value.

`nowrap`

Text does not wrap across lines. It will overflow its containing element rather than breaking onto a new line.

Description

The `text-wrap-mode` property can be used to control text wrapping inside an element. The different values provide alternate ways of wrapping the content of a block element, specifying whether lines may wrap at unforced soft wrap opportunities (`wrap`), or not (`nowrap`).

When set to `nowrap`, text will overflow rather than break onto multiple lines. As the `overflow` property defaults to `visible`, when content is larger than the container's inline-size, it will overflow in the inline direction. To enable scrolling in the inline direction, instead of overflow, set the `overflow-inline` property to `auto` or `scroll`. To ensure the container at least as wide as the longest word, set the `min-inline-size` to `min-content`.

When set to `wrap`, content flows onto new lines as necessary, wrapping at word break opportunities, such as `<wbr>` and hyphens, and other language-specific soft-wrap opportunities. The property does not prevent non-breakable pieces of content from overflowing in the inline direction if larger than the container's inline-size.

The `text-wrap-mode` is part of two shorthand properties. The `text-wrap-mode`, along with the `white-space-collapse` property, can be declared using the `white-space` shorthand. It can also be set and reset, along with the `text-wrap-style` property, using the `text-wrap` shorthand.

Formal definition

Initial value	`wrap`
Applies to	text and block containers
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-wrap-mode =
  wrap    |
  nowrap
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we compare the effect of the `text-wrap-mode` property's two values.

HTML

We include two `<div>` elements that are the same except for their class names.

```
<div class="wrap">CSS IS AWESOME</div>
<div class="nowrap">CSS IS AWESOME</div>
```

CSS

We apply the same styles to both containers, limiting their width.

```
div {
  font-family: "Arial", sans-serif;
  font-weight: bold;
  font-size: 4rem;
  box-sizing: border-box;
  border: 0.25rem solid black;
  padding: 0 4px;
  width: 14rem;
  margin: 1rem;
}
```

We give the examples different `text-wrap-mode` values:

```
.wrap {
  text-wrap-mode: wrap;
}
.nowrap {
  text-wrap-mode: nowrap;
}
```

Result

In the `wrap` example, the content flows over to the next line so that it fits in the box. "Awesome" is wider than the containing box, and contains no soft break opportunities, so it overflows.

In the `nowrap` example, the content does **not** flow to the next line as the content has been specifically set to not wrap, so it is all on one line, overflowing the container.

See also

- `text-wrap`
- `text-wrap-style`
- `hyphens`
- `white-space`
- `overflow`
- CSS text module
- CSS overflow module

text-wrap-style CSS property

The **`text-wrap-style`** CSS property controls how text inside an element is wrapped, providing alternate ways of determining where to create line breaks in order to fit the content within a block element.

Try it

```
text-wrap-style: auto;
```

```
text-wrap-style: balance;
```

```
text-wrap-style: pretty;
```

```
text-wrap-style: stable;
```

```
<section class="default-example" id="default-example">
  <div class="whole-content-wrapper">
    <p>Edit the text in the box:</p>
    <div class="transition-all" id="example-element">
      <p contenteditable>
        Lorem ipsum dolor sit amet consectetur adipisicing elit. Voluptatem aut
        cum eum id quos est.
      </p>
    </div>
  </div>
</section>
```

```
.whole-content-wrapper {
  display: flex;
  flex-direction: column;
  align-items: center;
  width: 100%;
}

#example-element {
  border: 1px solid #c5c5c5;
  width: 250px;
}
```

Syntax

```
/* Keyword values */
text-wrap-style: auto;
text-wrap-style: balance;
text-wrap-style: pretty;
text-wrap-style: stable;

/* Global values */
text-wrap-style: inherit;
text-wrap-style: initial;
text-wrap-style: revert;
text-wrap-style: revert-layer;
text-wrap-style: unset;
```

Values

The `text-wrap-style` property is specified as one of the following keyword values:

`auto`

> Text is wrapped in the most performant way for the browser and does not take into account the number of characters.

`balance`

> Text is wrapped in a way that best balances the number of characters on each line, enhancing layout quality and legibility. Because counting characters and balancing them across multiple lines is computationally expensive, this value is only supported for blocks of text spanning a limited number of lines (six or less for Chromium and ten or less for Firefox).

`pretty`

> Text is wrapped using a slower algorithm that favors better layout over speed. This is intended for body copy where good typography is favored over performance (for example, when the number of orphans should be kept to a minimum).

`stable`

> Text is wrapped such that when the user is editing the content, the lines that come before the lines they are editing remain static rather than the whole block of text re-wrapping.

The specification also defines an `avoid-orphans` value in which browsers consider more than one line when making break decisions to avoid excessively short last lines. This value is not yet supported in any browser.

Description

The `text-wrap-style` property can be used to provide a hint as to how the user agent should insert soft line breaks when the content is allowed to wrap. Each value defines a different approach to wrapping lines, balancing speed, quality, layout style, and stability, providing alternative ways to wrap the content of a block element.

When the content is allowed to wrap — the usual behavior because the default value of the `text-wrap-mode` property is `wrap` — the single keyword value of the `text-wrap-style` property specifies *how* the developer would like the text to be wrapped. The values have no effect on where a soft wrap opportunity exists, just how the browser should selects among them. If `text-wrap-mode` is set to `nowrap`, this property has no effect.

Selecting a value

The value you choose, for `text-wrap-style`, depends on how many lines of text you anticipate styling, whether the text is `contenteditable`, and whether you need to prioritize appearance or performance.

When the styled content will be limited to a short number of lines, such as headings, captions, and blockquotes, `text-wrap-style: balance` can be added to balance the number of characters on each line, enhancing layout quality and legibility. As browsers limit the number of lines impacted by this property, this value's impact on performance is negligible.

For longer sections of text, `text-wrap-style: pretty` can be used. Note that `pretty` has a negative effect on performance, so it should be only used for longer blocks of text when the layout is more important than speed.

The `stable` value improves user experience when used on content that is `contenteditable`. This value ensures that, as the user is editing text, the previous lines in the area being edited remain stable.

Formal definition

Initial value	`auto`
Applies to	text and block containers
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
text-wrap-style =
  auto                  |
  balance               |
  stable                |
  pretty                |
  avoid-short-last-line
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Balanced text

This example has two paragraphs, the first is the default `auto` and the second is `balance`.

HTML

```
<h2>Unbalanced</h2>
<p>
  Lorem ipsum dolor sit amet consectetur adipisicing elit. Velit, ad. Impedit
  adipisci rerum modi praesentium atque aperiam vitae nesciunt consectetur
  assumenda deleniti repudiandae perferendis sed odio doloremque, aliquid natus
  laboriosam?
</p>
<h2>Balanced</h2>
<p class="balanced">
  Lorem ipsum dolor sit amet consectetur adipisicing elit. Velit, ad. Impedit
  adipisci rerum modi praesentium atque aperiam vitae nesciunt consectetur
  assumenda deleniti repudiandae perferendis sed odio doloremque, aliquid natus
  laboriosam?
</p>
```

CSS

```
p {
  max-width: 60ch;
}
.balanced {
  text-wrap-style: balance;
}
```

Result

See also

- `text-wrap`
- `text-wrap-mode`
- `hyphens`
- `hyphenate-limit-chars`
- CSS text module

timeline-scope CSS property

The **`timeline-scope`** CSS property modifies the scope of a named animation timeline.

Syntax

```
/* Keyword values */
timeline-scope: all;
timeline-scope: none;

/* Custom name values */
timeline-scope: --custom_name_for_timeline;
timeline-scope: --timeline_name_one, --timeline_name_two;

/* Global values */
timeline-scope: inherit;
timeline-scope: initial;
timeline-scope: revert;
timeline-scope: revert-layer;
timeline-scope: unset;
```

Values

This property is specified as a keyword or a comma-separated list of custom timeline names:

`none`

Applies no change to the timeline scope. This is the default.

`all`

Sets the scope so that any timeline names set on the element or in its subtree are matched only by elements within the same subtree.

`<dashed-ident>`

Specifies one or more comma-separated timeline names that are declared using `scroll-timeline-name` or `view-timeline-name`. Sets the scope so that when the specified `scroll-timeline-name` or `view-timeline-name` values are set on the element or in its subtree, they are matched only by elements within the same subtree.

Description

The `timeline-scope` property modifies the scope of a named animation timeline. By default, a named timeline (i.e., declared using `scroll-timeline-name` or `view-timeline-name`) can only be set as the controlling timeline of a direct descendant element (i.e., by setting `animation-timeline` on it with the timeline name as its value). This is the default "scope" for the timeline.

The value of the `timeline-scope` is the name of a timeline defined on a descendant element; this changes the scope of the timeline to include the targeted element and its descendants. In other words, that element on which the `timeline-scope` property is defined, and all of its descendant elements, can be controlled using that timeline.

If no timeline (or more than one timeline) exists with the name given for the `timeline-scope` value, an inactive timeline with the specified name is created. The `timeline-scope` property only works with named timelines, and therefore cannot be used in conjunction with anonymous timelines created using the `view()` or `scroll()` animation timeline functions.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	none or an ordered list of identifiers
Animation type	Not animatable

Formal syntax

```
timeline-scope =
  none             |
  all              |
  <dashed-ident>#
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In this example, we animated an element in response to the scrolling of another element by increasing the timeline scope with the `timeline-scope` property.

HTML

The HTML includes an element to animate and an element to scroll:

```
<div class="content">
  <div class="box animation"></div>
</div>

<div class="scroller">
  <div class="long-element"></div>
</div>
```

CSS

A scroll timeline named `--my-scroller` is defined using the `scroll-timeline-name` property on a scrolling element. This scroll timeline name is used in two other places: it is applied as the `animation-timeline` on the element we want to animate, and as `timeline-scope` on an ancestor of both the scroller and the animated element, increasing the scope.

We set the `<body>`'s height to `100vh`, and lay its two child elements out as two equal columns using flexbox. To increase the timeline scope from the `<div class="scroller">` element to the whole `<body>`, we set `timeline-scope: --my-scroller` on it. By doing so, the `--my-scroller` timeline can be set as the controlling timeline for an animation set on the `<body>` or any element nested inside it.

```
body {
  margin: 0;
  height: 100vh;
  display: flex;

  timeline-scope: --my-scroller;
}

.content,
.scroller {
  flex: 1;
}
```

We set `--my-scroller` as the `scroll-timeline-name` on the scrolling element that should provide the scroll progress timeline for our animated element. We add `overflow` to enable scrolling, adding a background color to make its boundary visible. We set a large `height` on the contents of our scrolling element so the element actually scrolls.

```
.scroller {
  overflow: scroll;
  scroll-timeline-name: --my-scroller;
  background: deeppink;
}

.long-element {
  height: 2000px;
}
```

Next, we give the animated element some rudimentary styles and apply an animation to it using the `animation` shorthand property. We set the `animation-timeline` to the named scroll timeline: `--my-scroller`. To reiterate, animating the element based on the scroll progress of its cousin element is only possible because we set `timeline-scope` on a mutual ancestor; the animated element is **not** a descendant of the scrolling element.

```
.box {
  width: 100px;
  height: 100px;
  border-radius: 10px;
  background-color: rebeccapurple;
}

.animation {
  animation: rotate-appear 1ms linear;
  animation-timeline: --my-scroller;
}

@keyframes rotate-appear {
  from {
```

```
    rotate: 0deg;
    translate: 0;
  }

  to {
    rotate: 720deg;
    translate: 100%;
  }
}
```

```
@layer supports {
  @supports not (timeline-scope: none) {
    body::before {
      content: "Your browser does not support the 'timeline-scope' property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem;
    }
  }
}
```

Result

Scroll the vertical bar on the pink area to see the square animate.

The key point to note here is that the animated element is not a descendant of the scrolling element — to make this work, we increase the scope of the `--my-scroller` timeline by setting `timeline-scope: --my-scroller` on the `<body>`.

See also

- `animation-timeline`
- `scroll-timeline`, `scroll-timeline-name`
- `view-timeline`, `view-timeline-name`
- CSS scroll-driven animations

timeline-trigger CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger`** CSS shorthand property defines a scroll-triggered animation trigger on an element.

Constituent properties

This property is a shorthand for the following CSS properties:

- `timeline-trigger-name`
- `timeline-trigger-source`
- `timeline-trigger-activation-range`
- `timeline-trigger-active-range`

Syntax

```
/* Keyword */
timeline-trigger: none;

/* Name | source */
timeline-trigger: --t view();
timeline-trigger: --t --my-timeline;

/* Name | source | activation range */
timeline-trigger: --t view() contain;
timeline-trigger: --t --my-timeline entry exit 50%;

/* Name | source | activation range | active range */
timeline-trigger: --t view() contain / cover;
timeline-trigger: --t --my-timeline entry / entry exit 50%;

/* Multiple triggers */
timeline-trigger:
  --t view(),
  --other-trigger --my-timeline entry / entry 50% exit 50%;

/* Global values */
timeline-trigger: inherit;
timeline-trigger: initial;
timeline-trigger: revert;
timeline-trigger: revert-layer;
timeline-trigger: unset;
```

Values

This property is specified as the keyword `none` or a comma-separated list of `<timeline-trigger>` values:

`none`

Specifies that the element does not create a trigger, resetting all four longhand properties to their default values.

`<timeline-trigger>`

Specified as a space-separated list of the following values:

`<'timeline-trigger-name'>`

Specifies the `timeline-trigger-name` value representing the trigger's identifying name. Defaults to `none`.

`<'timeline-trigger-source'>`

Specifies the `timeline-trigger-source` value representing the trigger's timeline. Defaults to `auto`.

`<'timeline-trigger-activation-range'>` Optional

Specifies the `timeline-trigger-activation-range` value representing the trigger's activation range. Defaults to `normal`, which is equivalent to `cover 0% cover 100%` for a view progress timeline `timeline-trigger-source`, and `0% 100%` for a scroll progress timeline `timeline-trigger-source`.

`<'timeline-trigger-active-range'>` Optional

Preceded by a slash (`/`), specifies the `timeline-trigger-active-range` value representing the trigger's activation range. Defaults to `auto`, which sets the `<'timeline-trigger-active-range'>` to the same value as the `<'timeline-trigger-activation-range'>`.

Description

The `timeline-trigger` property can be used to set all the longhand properties used to create a CSS scroll-triggered animation trigger in a single declaration. Component properties not specified within the comma-separated list of `timeline-trigger` values are set to their default values.

Shorthand property order

Because some of the component properties share value types, the order of those component properties within the shorthand is important. The values must be given in the specified order.

- `timeline-trigger-name`
- `timeline-trigger-source`
- `timeline-trigger-activation-range`
- `timeline-trigger-active-range`, preceded by a forward slash.

The `timeline-trigger-active-range` value can only be included if the `timeline-trigger-activation-range` value is included; the two values are separated by a slash (`/`).

For example:

```
.trigger {
  timeline-trigger: --my-trigger view() entry / contain;
}
```

An element with this declaration set will have:

- An identifying `timeline-trigger-name` of `--my-trigger`.
- A `timeline-trigger-source` value of `view()`, which selects the element's nearest ancestor scrolling element to define its timeline trigger.
- An activation range of `entry`, meaning that the trigger will activate when its tracked element moves into the `entry` range. This is the range between the element's start edge crossing the scrollport's end edge and the element's end edge crossing the scrollport's end edge.
- An active range of `contain`, meaning that once activated, the trigger will stay active until its tracked element leaves the `contain` range: the range in which any part of the tracked element is visible in the scrollport.

To trigger an animated element via the previously described trigger, reference the `timeline-trigger-name` in the animated element's `animation-trigger` property. Set both the `timeline-trigger` and `animation-trigger` properties on the animated element to enable it to create its own trigger.

The <code>none</code> value

The `none` keyword specifies that the element does not create a scroll-triggered animation trigger. Setting `none` is equivalent to setting `none auto normal / normal`, which effectively resets all four equivalent longhand properties to their default values.

Formal definition

Initial value	as each of the properties of the shorthand:
	• `timeline-trigger-name`: `none`
	• `timeline-trigger-source`: `auto`
	• `timeline-trigger-activation-range`: as each of the properties of the shorthand:
	  ◦ `timeline-trigger-activation-range-start`: `normal`
	  ◦ `timeline-trigger-activation-range-end`: `normal`
	• `timeline-trigger-active-range`: as each of the properties of the shorthand:
	  ◦ `timeline-trigger-active-range-start`: `auto`
	  ◦ `timeline-trigger-active-range-end`: `auto`

Applies to	all elements
Inherited	no

Computed value	as each of the properties of the shorthand:
	• `timeline-trigger-name`: as specified
	• `timeline-trigger-source`: A list, with each item being either the keyword `none`, the keyword `auto`, a case-sensitive `<ident>`, a computed `scroll()` function, or a computed `view()` function.
	• `timeline-trigger-activation-range`: as each of the properties of the shorthand:
	  ◦ `timeline-trigger-activation-range-start`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
	  ◦ `timeline-trigger-activation-range-end`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
	• `timeline-trigger-active-range`: as each of the properties of the shorthand:
	  ◦ `timeline-trigger-active-range-start`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
	  ◦ `timeline-trigger-active-range-end`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage

Animation type	as each of the properties of the shorthand: • `timeline-trigger-name`: Not animatable • `timeline-trigger-source`: Not animatable • `timeline-trigger-activation-range`: Not animatable • `timeline-trigger-active-range`: Not animatable

Formal syntax

```
timeline-trigger =
  none                                          |
  [ <'timeline-trigger-name'> <'timeline-trigger-source'> [ <'timeline-trigger-range'> ]? ]#

<timeline-trigger-name> =
```

```
  none             |
  <dashed-ident>#

<timeline-trigger-source> =
  [ none | auto | [ <dashed-ident> | <scroll()> | <view()> ]+ ]#

<timeline-trigger-range> =
  [ <'timeline-trigger-activation-range'> [ / <'timeline-trigger-active-range'> ]? ]#

<scroll()> =
  scroll( [ <scroller> || <axis> ]? )

<view()> =
  view( [ <axis> || <'view-timeline-inset'> ]? )

<timeline-trigger-activation-range> =
  [ <'timeline-trigger-activation-range-start'> <'timeline-trigger-activation-range-end'>? ]#

<timeline-trigger-active-range> =
  [ <'timeline-trigger-active-range-start'> <'timeline-trigger-active-range-end'>? ]#

<scroller> =
  root     |
  nearest  |
  self

<axis> =
  block   |
  inline  |
  x       |
  y

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<timeline-trigger-activation-range-start> =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<timeline-trigger-activation-range-end> =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<timeline-trigger-active-range-start> =
  [ auto | normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<timeline-trigger-active-range-end> =
  [ auto | normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates using the `timeline-trigger` shorthand property to create a scroll-triggered animation.

HTML

We include two `<div>` elements, one to animate and one on which to create a trigger. The basic text content that causes the page to scroll has been hidden for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
```

```
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport to enable us to see when its animation starts and stops.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` to create a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, we apply the `rotate` animation to the `.animated` element. Without a trigger, animations start on page load. We include the `animation-trigger` property, which references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause`. This causes the animation to play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger using a `timeline-trigger` value of `--t view() entry / cover`. This specifies the following, all in a single declaration:

- A `timeline-trigger-name` value of `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` value of `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` of `entry`, which means that the trigger will activate when the tracked element's block start edge enters the scrollport.
- A `timeline-trigger-active-range` of `cover`, which means that, once activated, the trigger will stay active until the tracked element completely leaves the scrollport.

```
.trigger {
  timeline-trigger: --t view() entry / cover;
}
```

Result

Try scrolling the content. The rotation will start when the tracked element enters the `entry` range: when the `.trigger` element first enters the bottom of the scrollport. The animation won't stop until the `.trigger` element has completely exited the scrollport.

Multiple timeline-trigger values

This example builds on the previous one; it demonstrates how multiple `timeline-trigger` values can be set on the same element, creating multiple triggers that can be used to trigger multiple animations.

HTML

The markup is similar to the previous example, with an extra animated `<div>` element with a `class` of `animated2`. This example has two animated elements and one element on which to create triggers.

```
<div class="animated2">I am animated as well</div>
```

CSS

The animated elements are `fixed` in `position`, as in the previous example, with different `left` values so they don't overlap.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated,
.animated2 {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated,
.animated2 {
  position: fixed;
  top: 25px;
}

.animated {
  left: 25px;
}

.animated2 {
  left: 150px;
}
```

We define two sets of animation `@keyframes`:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}

@keyframes up-down {
  0% {
    translate: 0 0;
  }

  25% {
    translate: 0 25px;
  }

  50% {
    translate: 0 0;
  }

  75% {
    translate: 0 -25px;
  }

  100% {
    translate: 0 0;
  }
}
```

Each animated element has a different animation set, triggered by a separate timeline trigger, and different `<animation-action>` values applied. We apply the same `animation` to the `.animated` element as in the previous example, and a different `animation` to the `.animated2` element. Both have the `animation-trigger` property applied, but with different values. The first animation plays on activation and reverses on deactivation, whereas the second one plays on activation and pauses on deactivation.

```
.animated {
  animation: rotate 3s infinite linear both;
  animation-trigger: --t play-forwards play-backwards;
}

.animated2 {
  animation: up-down 1s infinite linear;
  animation-trigger: --t2 play pause;
}
```

We set a `timeline-trigger` value on `.trigger` that contains two values. Each value contains different `timeline-trigger-name`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range` values. As a result, the animated elements start and stop their animations at different offsets.

```
.trigger {
  timeline-trigger:
```

```
    --t view() entry / cover,
    --t2 view() contain;
}
```

```
@supports not (timeline-trigger: --t view() entry / cover) {
  body::before {
    content: "Your browser does not support the timeline-trigger property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content. The first animated element starts rotating when the tracked element enters the `entry` range down at the bottom of the scrollport, then rotates in reverse when the tracked element has completely exited the scrollport. The second animated element starts moving up and down when the tracked element has completely entered the scrollport, and stops when the tracked element begins exiting the scrollport.

See also

- `animation-trigger`
- `timeline-trigger-name`, `timeline-trigger-source`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range`
- `trigger-scope`
- `<animation-action>` type
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-activation-range CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-activation-range`** CSS shorthand property specifies the start and end of a scroll-triggered animation trigger's activation range.

Constituent properties

This property is a shorthand for the following CSS properties:

- `timeline-trigger-activation-range-start`
- `timeline-trigger-activation-range-end`

Syntax

```
/* Keyword */
timeline-trigger-activation-range: normal;

/* Range start only */
/* Offset only */
timeline-trigger-activation-range: 40%;
timeline-trigger-activation-range: 200px;
/* Named timeline only */
timeline-trigger-activation-range: contain;
timeline-trigger-activation-range: entry;
/* Named timeline and offset value */
timeline-trigger-activation-range: exit 50%;
timeline-trigger-activation-range: contain 150px;

/* Range start and end */
timeline-trigger-activation-range: 20% 80%;
timeline-trigger-activation-range: entry exit;
timeline-trigger-activation-range: normal 20%;
timeline-trigger-activation-range: 20% normal;
/* Offset on start only */
timeline-trigger-activation-range: entry 10% 90%;
/* Offset on end only */
timeline-trigger-activation-range: 200px exit 300px;
/* Named timeline and offset for both start and end */
timeline-trigger-activation-range: entry 0% exit 50%;
timeline-trigger-activation-range: contain 100px contain 90%;

/* Multiple ranges */
timeline-trigger-activation-range:
  contain,
  entry 0% exit 50%;

/* Global values */
timeline-trigger-activation-range: inherit;
timeline-trigger-activation-range: initial;
timeline-trigger-activation-range: revert;
timeline-trigger-activation-range: revert-layer;
timeline-trigger-activation-range: unset;
```

Values

This property is specified as a comma-separated list of animation ranges. Each animation range is specified as a `timeline-trigger-activation-range-start` value and, optionally, a `timeline-trigger-activation-range-end` value.

`<'timeline-trigger-activation-range-start'>`

> The keyword `normal`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name>` followed by a `<length-percentage>`, representing the `timeline-trigger-activation-range-start`. If a `<timeline-range-name>` is set without a `<length-percentage>`, the `<length-percentage>` defaults to `0%`.

<'timeline-trigger-activation-range-end'>

The keyword `normal`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name>` followed by a `<length-percentage>`, representing the `timeline-trigger-activation-range-end`. If a `<timeline-range-name>` is set without a `<length-percentage>`, the `<length-percentage>` defaults to `100%`.

Percentages are relative to the length of the named timeline range if one is specified, or the timeline represented by `normal` if not.

Description

The `timeline-trigger-activation-range` property can be used to explicitly specify the start or start and end of a trigger's activation range. The property sets both the `timeline-trigger-activation-range-start` and `timeline-trigger-activation-range-end` properties in one declaration, with each specified as a timeline range, offset, or both. Start and end offsets are both measured from the start of their ranges. If only the `timeline-trigger-activation-range-start` value is specified, the `timeline-trigger-activation-range-end` value defaults to `normal`, which is either `contain 100%` or `scroll 100%`, depending on the `timeline-trigger-source` value.

A trigger's activation range is the range along the associated scrollport within which a CSS scroll-triggered animation trigger will activate. Activation occurs when the tracked element enters the *activation range*, and deactivation occurs when it leaves the *active range*.

The default value is `normal`, which sets the activation range to the default named range. The default named range depends on the `timeline-trigger-source`: it is equivalent to `cover` for a view progress timeline and `scroll` for a scroll progress timeline. The default offset values are `0%` for activation start and `100%` for activation end. Therefore, `normal` resolves to either `cover 0% cover 100%` or `scroll 0% scroll 100%`.

Other `timeline-trigger-activation-range` values can be used to set:

Start and end offsets from the `normal` range

A `<length>` or `<percentage>` value specifies an offset from the beginning of the `normal` timeline, which again defaults to `cover` for a `view()` progress timeline source, and `scroll` for a `scroll()` progress timeline source. Negative values outset the start and end, resulting in a longer activation range. Positive values inset the start and end of the activation range, shortening it.

Specific named ranges

If a `<timeline-range-name>` values is set without including an offset, the offset defaults to `0%` for start and `100%` for end values. The named timeline ranges include `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, and `scroll`. See Understanding timeline range names.

Offsets from specific named ranges

When both a `<timeline-range-name>` and `<length>` or `<percentage>` value are specified for the start or end, the value is specified as a length or percentage offset from the start of the named range. Percentage values are relative to the full length of the named range specified. See Setting insets using percentages.

In each component of a `timeline-trigger-activation-range` value, the `<timeline-range-name>` value must come before the `<length>` or `<percentage>` offset. In the following example, you might think `timeline-trigger-activation-range-start` is set to `contain` and `timeline-trigger-activation-range-end` is set to `50%`, but that is not the case. Instead, `timeline-trigger-activation-range-start` is set to `contain 50%` while `timeline-trigger-activation-range-end` defaults to `normal`:

```
timeline-trigger-activation-range: contain 50%;
```

To set `timeline-trigger-activation-range-start` to `contain` and `timeline-trigger-activation-range-end` to `50%`, explicitly set `0%`, which is the default start offset:

```
timeline-trigger-activation-range: contain 0% 50%;
```

By default, the active range is the same as the activation range. To make the active range longer than the activation range, use the `timeline-trigger-active-range-start` and `timeline-trigger-active-range-end` properties, or the `timeline-trigger-active-range` shorthand. Making the active range longer than the activation range is useful when you want to trigger an animation in a small activation range but keep the trigger active over a larger range.

The `timeline-trigger-activation-range` property, along with the `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-active-range` properties, can also be set using the `timeline-trigger` shorthand.

Explicit and default values for <code>timeline-trigger-activation-range</code>

In terms of explicit and default values, `timeline-trigger-activation-range` works in exactly the same way as the `animation-range` property. See the following for more information:

- Explicitly defining both range start and range end with two values
- Defining range start and defaulting range end

Specifying multiple ranges

When multiple values are specified in a comma-separated `timeline-trigger-activation-range` declaration, each value applies to a timeline trigger in the order in which the names appear in the `timeline-trigger-name` property. When the number of triggers and `timeline-trigger-activation-range` property values do not match, they are applied in the same way as multiple animation property values:

- If the number of `timeline-trigger-activation-range` values exceeds the number of `timeline-trigger-name` values, the excess range values are discarded.
- If the number of trigger names is greater than the number of ranges, the `timeline-trigger-activation-range` values are cycled until every `timeline-trigger-name` value has a `timeline-trigger-activation-range` value set.
- If multiple `timeline-trigger-name` values are set, but only one `timeline-trigger-activation-range` value is set, the `timeline-trigger-activation-range` will apply to all the `timeline-trigger-name`s.

Formal definition

Initial value	as each of the properties of the shorthand: • `timeline-trigger-activation-range-start`: `normal` • `timeline-trigger-activation-range-end`: `normal`
Applies to	all elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `timeline-trigger-activation-range-start`: Relative to the specified named timeline range if specified, otherwise relative to the entire timeline • `timeline-trigger-activation-range-end`: Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	as each of the properties of the shorthand: • `timeline-trigger-activation-range-start`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage • `timeline-trigger-activation-range-end`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
timeline-trigger-activation-range =
  [ <'timeline-trigger-activation-range-start'> <'timeline-trigger-activation-range-end'>? ]#

<timeline-trigger-activation-range-start> =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<timeline-trigger-activation-range-end> =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we inset a scroll-triggered animation trigger's activation range by setting a custom `timeline-trigger-activation-range` value.

HTML

Our markup contains two `<div>` elements—one to animate and one to create a trigger on—and some text content to make the page scroll. We have hidden the text content for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport so we can see when its animation starts and stops.

```
body {
  width: 80%;
  margin: 0 auto;
```

```
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` element. Without an associated trigger, the element would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The value references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause` — which specify that the animation plays on activation and pauses on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` of `entry 50% exit 50%`. The `entry` range spans from when the trigger element first starts entering the scrollport to when it has completely entered the scrollport, while the `exit` range spans from when the trigger element first starts leaving the scrollport to when it has completely left the scrollport. This value sets the trigger's activation range to start at `50%` through the `entry` range and end `50%` through the `exit` range.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: entry 50% exit 50%;
}
```

Result

Try scrolling the content up and down. The animation starts playing when `50%` of the tracked `.trigger` element has entered the scrollport in either direction and pauses when `50%` of the trigger element has exited the scrollport at either edge.

Comparing multiple range values

This example is identical to the previous example, except that it allows selecting different activation ranges to compare their effects.

The markup is the same as the previous example except we've added a `<select>` element that can be used to change the `timeline-trigger-activation-range` value. When a new value is selected, it is applied to the trigger element using JavaScript. We have hidden the HTML and JavaScript for brevity.

```
<form>
  <label for="range-select">Select activation range</label>
  <select id="range-select">
    <optgroup label="Start value">
      <option>40%</option>
      <option>200px</option>
      <option>contain</option>
      <option selected>cover</option>
      <option>entry</option>
      <option>exit 50%</option>
      <option>contain 150px</option>
    </optgroup>
    <optgroup label="Start and end value">
      <option>20% 80%</option>
      <option>entry exit</option>
      <option>normal 20%</option>
      <option>20% normal</option>
      <option>contain contain 40%</option>
      <option>200px exit 300px</option>
      <option>entry 10% 90%</option>
      <option>entry 0% exit 50%</option>
      <option>contain 100px contain 90%</option>
    </optgroup>
  </select>
</form>
```

```
const selectElem = document.querySelector("select");
const triggerElem = document.querySelector(".trigger");

selectElem.addEventListener("change", () => {
  triggerElem.style.timelineTriggerActivationRange = selectElem.value;
});
```

CSS

The CSS is the same as for the previous example, except we've omitted the `timeline-trigger-activation-range` value. This means that until a range value is selected, the range will default to `normal`, which is `cover 0% cover 100%` in this case.

```
form {
  width: 250px;
  padding: 5px;
  border: 2px solid black;
  background: white;
  position: fixed;
  top: 0;
  right: 0;
}

label,
select {
  font-size: 1rem;
}

select {
  padding: 5px;
  margin-top: 5px;
  width: 100%;
}
```

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}

.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
```

```
@supports not (timeline-trigger-activation-range: entry 50% exit 50%) {
  body::before {
    content: "Your browser does not support the timeline-trigger-activation-range property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Select different range values then scroll the tracked element up and down the scrollport to see where the animated element starts and stops rotating.

See also

- `animation-trigger`
- `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand property
- `trigger-scope`
- `<animation-action>` type
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-activation-range-end CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-activation-range-end`** CSS property specifies the end of a scroll-triggered animation trigger's activation range.

Syntax

```
/* Keyword */
timeline-trigger-activation-range-end: normal;

/* <length-percentage> */
timeline-trigger-activation-range-end: 80%;
timeline-trigger-activation-range-end: 400px;

/* Named timeline range */
timeline-trigger-activation-range-end: contain;
timeline-trigger-activation-range-end: exit;

/* Named timeline with <length-percentage> */
timeline-trigger-activation-range-end: entry 110%;
timeline-trigger-activation-range-end: contain 600px;

/* Multiple range end values */
timeline-trigger-activation-range-end:
```

```
  contain,
  entry 100%;

/* Global values */
timeline-trigger-activation-range-end: inherit;
timeline-trigger-activation-range-end: initial;
timeline-trigger-activation-range-end: revert;
timeline-trigger-activation-range-end: revert-layer;
timeline-trigger-activation-range-end: unset;
```

Values

This property is specified as a comma-separated list of the following values:

`normal`

The default value. Equivalent to `cover 100%` for a view progress timeline `timeline-trigger-source`, and `scroll 100%` for a scroll progress timeline.

`<length-percentage>`

Specifies an offset as a `<length>` or `<percentage>`, measured from the beginning of the `normal` timeline. Percentages are relative to the length of the `normal` timeline range.

`<timeline-range-name>`

Specifies the end (`100%`) of the `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll` timeline range.

`<timeline-range-name> <length-percentage>`

Specifies a length or percentage offset measured from the beginning of the specified named timeline range. Percentages are relative to the length of the named timeline.

Description

The `timeline-trigger-activation-range-end` property can be used to explicitly specify the end of a trigger's activation range as a timeline range, offset, or both.

A trigger's activation range is the range along the associated scrollport within which a CSS scroll-triggered animation trigger will activate. Activation occurs when the tracked element enters the *activation range*, and deactivation occurs when it leaves the *active range*.

The `normal` value sets the end of the activation range to the end of the default named range. This is equivalent to `cover 100%` for a view progress timeline and `scroll 100%` for a scroll progress timeline.

Other `timeline-trigger-activation-range-end` values can be used to set:

An offset from the `normal` range

A `<length>` or `<percentage>` value specifies an offset from the beginning of the `normal` timeline, which again defaults to `cover 100%` for a view progress timeline source and `scroll 100%` for a scroll progress timeline source. Negative values outset the end, resulting in a longer activation range. Positive values inset the end of the activation range, making it shorter.

The end of a specific named range

A `<timeline-range-name>` value specifies a `100%` offset along the named timeline range, which is either `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll`. See Understanding timeline range names.

An offset from a specific named range

When both a `<timeline-range-name>` and `<length>` or `<percentage>` value are specified, the end is offset by the distance specified from the start of the named range. Percentage values are relative to the range specified. See Setting insets using percentages.

By default, the active range is the same as the activation range. You can use the `timeline-trigger-active-range-end` or `timeline-trigger-active-range` properties to extend the end of the active range and make the active range longer than the activation range. This is useful when you want to trigger an animation in a small activation range, but you want the trigger to stay active within a larger range.

The `timeline-trigger-activation-range-end` property, along with the `timeline-trigger-activation-range-start` property, can also be set using the `timeline-trigger-activation-range` shorthand, which in turn can be set using the `timeline-trigger` shorthand.

Specifying multiple range end values

When multiple values are specified in a comma-separated `timeline-trigger-activation-range-end` declaration, each value applies to a timeline trigger in the order in which the names appear in the `timeline-trigger-name` property. When the number of triggers and `timeline-trigger-activation-range-end` property values do not match, they are applied in the same way as multiple animation property values:

- If the number of `timeline-trigger-activation-range-end` values exceeds the number of `timeline-trigger-name` values, the excess range values are discarded.
- If the number of trigger names is greater than the number of ranges, the `timeline-trigger-activation-range-end` values are cycled until every `timeline-trigger-name` value has a `timeline-trigger-activation-range-end` value set.
- If multiple `timeline-trigger-name` values are set, but only one `timeline-trigger-activation-range-end` value is set, that `timeline-trigger-activation-range-end` value will apply to all the `timeline-trigger-name`s.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
timeline-trigger-activation-range-end =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we inset the end of a scroll-triggered animation trigger's activation range by setting a custom `timeline-trigger-activation-range-end` value.

HTML

Our markup contains two `<div>` elements — one to animate and one to create a trigger on — plus some basic text content to cause the page to scroll. We have hidden the text content for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport to enable us to see when its animation starts and stops.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
```

```
  left: 25px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` element. Without an associated trigger, the element would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The value references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause` — which specify that the animation will play on activation, and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range-end` of `contain 60%`. The `contain` range spans from when the trigger element has fully entered the scrollport to when it starts to leave. This value sets the trigger's activation range's start to `60%` through the `contain` range.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range-end: contain 60%;
}
```

When not explicitly set, the `timeline-trigger-activation-range-start` value defaults to `normal`, which in this case is `cover 0%`. This is the start of the `cover` range, meaning activation occurs when the tracked element starts to enter the scrollport's end edge.

```
@supports not (timeline-trigger-activation-range-end: contain 60%) {
  body::before {
    content: "Your browser does not support the timeline-trigger-activation-range-end property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up. The animation starts playing when the tracked `.trigger` element first becomes visible at the scrollport end edge and pauses when it has scrolled `60%` of the way up the timeline range. When you scroll down, the effect reverses — the animation restarts playing when the trigger element gets to `60%` of the way up, and pauses again when the trigger element reaches the end edge.

See also

- `timeline-trigger-activation-range-start`
- `timeline-trigger-activation-range` shorthand property
- `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand property
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-activation-range-start CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-activation-range-start`** CSS property specifies the start of a scroll-triggered animation trigger's activation range.

Syntax

```
/* Keyword */
timeline-trigger-activation-range-start: normal;

/* <length-percentage> */
timeline-trigger-activation-range-start: 20%;
timeline-trigger-activation-range-start: 350px;

/* Named timeline range */
timeline-trigger-activation-range-start: cover;
timeline-trigger-activation-range-start: exit;

/* Named timeline with <length-percentage> */
timeline-trigger-activation-range-start: entry 40%;
timeline-trigger-activation-range-start: contain -20px;

/* Multiple range start values */
timeline-trigger-activation-range-start:
  contain,
```

```
  entry -10%;

/* Global values */
timeline-trigger-activation-range-start: inherit;
timeline-trigger-activation-range-start: initial;
timeline-trigger-activation-range-start: revert;
timeline-trigger-activation-range-start: revert-layer;
timeline-trigger-activation-range-start: unset;
```

Values

This property is specified as a comma-separated list of the following values:

`normal`

The default value. Equivalent to `cover 0%` for a view progress timeline `timeline-trigger-source`, and `scroll 0%` for a scroll progress timeline.

`<length-percentage>`

Specifies an offset as a `<length>` or `<percentage>`, measured from the beginning of the `normal` timeline. Percentages are relative to the length of the `normal` timeline range.

`<timeline-range-name>`

Specifies the start (`0%`) of the `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll` timeline range.

`<timeline-range-name> <length-percentage>`

Specifies a length or percentage offset measured from the beginning of the specified named timeline range. Percentages are relative to the length of the named timeline.

Description

The `timeline-trigger-activation-range-start` property can be used to explicitly specify the start of a trigger's activation range. The value is specified as a timeline range, offset, or both.

A trigger's activation range is the range along the associated scrollport within which a CSS scroll-triggered animation trigger will activate. Activation occurs when the tracked element enters the *activation range*, and deactivation occurs when it leaves the *active range*.

The `normal` value sets the start of the activation range to the start of the default named range. This is equivalent to `cover 0%` for a view progress timeline and `scroll 0%` for a scroll progress timeline.

Other `timeline-trigger-activation-range-start` property can be used to set:

An offset from the `normal` range

A `<length>` or `<percentage>` value specifies an offset from the beginning of the `normal` timeline, which again defaults to `cover 0%` for a view progress timeline source, and `scroll 0%` for a scroll progress timeline source. Negative values outset the start, resulting in a longer activation range. Positive values inset the start, making the activation range shorter.

The start of a specific named range

A `<timeline-range-name>` value specifies a `0%` offset along the named timeline range, which is either `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll`. See Understanding timeline range names.

An offset from a specific named range

When both a `<timeline-range-name>` and `<length>` or `<percentage>` value are specified, the start is the beginning of the named range offset by the distance specified. Percentage values are relative to the length of the range specified. See Setting insets using percentages.

The `timeline-trigger-activation-range-start` property, along with the `timeline-trigger-activation-range-end` property, can also be set using the `timeline-trigger-activation-range` shorthand, which in turn can be set using the `timeline-trigger` shorthand.

Specifying multiple range start values

When multiple values are specified in a comma-separated `timeline-trigger-activation-range-start` declaration, each value applies to a timeline trigger in the order in which the names appear in the `timeline-trigger-name` property. When the number of triggers and `timeline-trigger-activation-range-start` property values do not match, they are applied in the same way as multiple animation property values:

- If the number of `timeline-trigger-activation-range-start` values exceeds the number of `timeline-trigger-name` values, the excess range values are discarded.
- If the number of trigger names is greater than the number of ranges, the `timeline-trigger-activation-range-start` values are cycled until every `timeline-trigger-name` value has a `timeline-trigger-activation-range-start` value set.
- If multiple `timeline-trigger-name` values are set, but only one `timeline-trigger-activation-range-start` value is set, that `timeline-trigger-activation-range-start` value will apply to all the `timeline-trigger-name`s.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
timeline-trigger-activation-range-start =
  [ normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

In this example, we inset the start of a scroll-triggered animation trigger's activation range by setting a custom `timeline-trigger-activation-range-start` value.

HTML

Our markup contains two `<div>` elements — one to animate and one to create a trigger on — plus some basic text content to cause the page to scroll. We have hidden the text content for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport to enable us to see when its animation starts and stops.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` element. Without an associated trigger, the element would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The value references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause` — which specify that the animation will play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range-start` of `entry 50%`. The `entry` range spans from when the trigger element first starts entering the scrollport to when it has completely entered the scrollport. This value sets the trigger's activation range's start to `50%` through the `entry` range, which occurs when `50%` of the tracked `.trigger` element has entered the scrollport via the scrollport's end edge.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range-start: entry 50%;
}
```

When not explicitly set, the `timeline-trigger-activation-range-end` value defaults to `normal`, which in this case is `cover 100%`. The `timeline-trigger-active-range-end` value defaults to `auto` — the same as `timeline-trigger-activation-range-end`. This means that deactivation occurs at the end of the `cover` range, when the tracked element exits the scrollport's start edge.

```
@supports not (timeline-trigger-activation-range-start: entry 50%) {
  body::before {
    content: "Your browser does not support the timeline-trigger-activation-range-start property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up. The animation starts playing when `50%` of the tracked `.trigger` element has entered the scrollport end edge and pauses when it has completely exited the scrollport at the opposite edge. When you scroll down, the effect reverses — the animation restarts playing when the trigger element starts to enter the top of the scrollport, and pauses again when `50%` of the trigger element has exited the bottom.

See also

- `timeline-trigger-activation-range-end`
- `timeline-trigger-activation-range` shorthand property
- `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand property
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-active-range CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-active-range`** CSS shorthand property specifies a scroll-triggered animation trigger's active range.

Constituent properties

This property is a shorthand for the following CSS properties:

- `timeline-trigger-active-range-start`
- `timeline-trigger-active-range-end`

Syntax

```
/* Keywords */
timeline-trigger-active-range: normal;
timeline-trigger-active-range: auto;

/* Range start only */
/* Offset only */
timeline-trigger-active-range: 10%;
timeline-trigger-active-range: 40px;
/* Named timeline only */
timeline-trigger-active-range: cover;
/* Named timeline and offset value */
timeline-trigger-active-range: exit -10%;
timeline-trigger-active-range: contain 50px;

/* Range start and end */
```

```
timeline-trigger-active-range: entry exit;
/* Offset on start only */
timeline-trigger-active-range: 10% normal;
timeline-trigger-active-range: 100px contain;
timeline-trigger-active-range: entry 10% contain;
/* Offset on end only */
timeline-trigger-active-range: normal exit-crossing 90%;
timeline-trigger-active-range: auto 10%;
timeline-trigger-active-range: contain contain 90%;
/* Offset for both start and end */
timeline-trigger-active-range: 5% 95%;
timeline-trigger-active-range: 200px exit 600px;
timeline-trigger-active-range: entry 10% 90%;
/* Named timeline and offset for both start and end */
timeline-trigger-active-range: entry 0% exit 50%;
timeline-trigger-active-range: contain 100px contain 90%;

/* Multiple ranges */
timeline-trigger-active-range:
  cover,
  entry -5% exit 50%;

/* Global values */
timeline-trigger-active-range: inherit;
timeline-trigger-active-range: initial;
timeline-trigger-active-range: revert;
timeline-trigger-active-range: revert-layer;
timeline-trigger-active-range: unset;
```

Values

This property is specified as a comma-separated list of animation ranges. Each animation range is specified as a `timeline-trigger-active-range-start` value, and, optionally, a `timeline-trigger-active-range-end` value.

`<'timeline-trigger-active-range-start'>`

> The keyword `normal`, the keyword `auto`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name>` followed by a `<length-percentage>`, representing the `timeline-trigger-active-range-start`. If a `<timeline-range-name>` is set without a `<length-percentage>`, the `<length-percentage>` defaults to `0%`.

`<'timeline-trigger-active-range-end'>`

> The keyword `normal`, the keyword `auto`, a `<length-percentage>`, a `<timeline-range-name>`, or a `<timeline-range-name>` followed by a `<length-percentage>`, representing the `timeline-trigger-active-range-end`. If a `<timeline-range-name>` is set without a `<length-percentage>`, the `<length-percentage>` defaults to `100%`.

Percentages are relative to the length of the named timeline range if one is specified, or the timeline represented by `normal` if not.

Description

The `timeline-trigger-active-range` property can be used to explicitly specify the start or both the start and end of a trigger's active range. The property sets both the `timeline-trigger-active-range-start` and `timeline-trigger-active-range-end` properties in one declaration, with each specified as a timeline range, offset, or both. Start and end offsets are both measured from the start of their ranges.

A trigger's *active range* is the range along the associated scrollport within which a CSS scroll-triggered animation trigger will stay active once activated. Activation occurs when the tracked element enters the *activation range*, and deactivation occurs when it leaves the *active range*. The default value is `auto`, which sets the `timeline-trigger-active-range` value to the same as the `timeline-trigger-activation-range`.

Making the active range longer than the activation range is useful when you want to trigger an animation in a small activation range, but you want the trigger to stay active within a larger range. The trigger will only deactivate when the tracked element leaves the active range.

A value of `normal` sets the active range to the default named range. The default named range depends on the `timeline-trigger-source`: it is equivalent to `cover` for a view progress timeline and `scroll` for a scroll progress timeline. The default offset values are `0%` and `100%`. Therefore, `normal` resolves to either `cover 0% cover 100%` or `scroll 0% scroll 100%`.

The `timeline-trigger-active-range` property can be used to set:

Start and end offsets from the `normal` range

> A `<length>` or `<percentage>` value specifies an offset from the beginning of the `normal` timeline, which again defaults to `cover` for a view progress timeline source, and `scroll` for a scroll progress timeline source. Negative values outset the start and end, resulting in a longer active range. Positive values inset the start and end of the active range, making it shorter.

Specific named ranges

> If a `<timeline-range-name>` value is set without including an offset, the offset defaults to `0%` for start and `100%` for the end values. The named timeline ranges include `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, and `scroll`. See Understanding timeline range names.

Offsets from specific named ranges

> When both a `<timeline-range-name>` and `<length>` or `<percentage>` value are specified, the start and end values are offset by the distances along the specified named ranges. Percentage values are relative to the range specified. See Setting insets using percentages.

In each component of a `timeline-trigger-active-range` value, the `<timeline-range-name>` value must come before the `<length>` or `<percentage>` offset. In the following example, you might think `timeline-trigger-active-range-start` is set to `contain`, and `timeline-trigger-active-range-end` is set to `50%`, but this is not the case. Instead, `timeline-trigger-active-range-start` is set to `contain 50%` while `timeline-trigger-active-range-end` defaults to `auto`:

```
timeline-trigger-active-range: contain 50%;
```

To set `timeline-trigger-active-range-start` to `contain` and `timeline-trigger-active-range-end` to `50%`, explicitly set `0%`, which is the default start offset:

```
timeline-trigger-active-range: contain 0% 50%;
```

A set active range must be equal to or larger than the activation range. Specifically, the `timeline-trigger-active-range-start` value must come before or at the same position as the `timeline-trigger-activation-range-start` value, and the `timeline-trigger-active-range-end` value must be the same as or come after the `timeline-trigger-activation-range-end` value. If either value is within the activation range, the value will have no effect, and the active range will be equal to the activation range.

The `timeline-trigger-active-range` property, along with the `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-activation-range` properties, can also be set using the `timeline-trigger` shorthand.

Explicit and default values of <code>timeline-trigger-active-range</code>

In terms of explicit and default values, `timeline-trigger-active-range` works in exactly the same way as the `animation-range` property. See the following for more information:

- Explicitly defining both range start and range end with two values

- Defining range start and defaulting range end

Specifying multiple ranges

When multiple values are specified in a comma-separated `timeline-trigger-active-range` declaration, each value applies to a timeline trigger in the order in which the names appear in the `timeline-trigger-name` property. When the number of triggers and `timeline-trigger-active-range` property values do not match, they are applied in the same way as multiple animation property values:

- If the number of `timeline-trigger-active-range` values exceeds the number of `timeline-trigger-name` values, the excess range values are discarded.
- If the number of trigger names is greater than the number of ranges, the `timeline-trigger-active-range` values are cycled until every `timeline-trigger-name` value has a `timeline-trigger-active-range` value set.
- If multiple `timeline-trigger-name` values are set, but only one `timeline-trigger-active-range` value is set, the `timeline-trigger-active-range` will apply to all the `timeline-trigger-name`s.

Formal definition

Initial value	as each of the properties of the shorthand: • `timeline-trigger-active-range-start`: `auto` • `timeline-trigger-active-range-end`: `auto`
Applies to	all elements
Inherited	no
Percentages	as each of the properties of the shorthand: • `timeline-trigger-active-range-start`: Relative to the specified named timeline range if specified, otherwise relative to the entire timeline • `timeline-trigger-active-range-end`: Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	as each of the properties of the shorthand: • `timeline-trigger-active-range-start`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage • `timeline-trigger-active-range-end`: A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
timeline-trigger-active-range =
  [ <'timeline-trigger-active-range-start'> <'timeline-trigger-active-range-end'>? ]#

<timeline-trigger-active-range-start> =
  [ auto | normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<timeline-trigger-active-range-end> =
  [ auto | normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the effect of extending a trigger's active range by comparing a triggered animation with a longer active range than its activation range against an identical triggered animation with no `timeline-trigger-active-range` property set.

HTML

The markup contains four `<div>` elements — two to animate and two to create a trigger on — plus some content that causes the page to scroll. We have hidden the extra content for brevity.

```
<div class="animated">I am animated</div>
<div class="animated longer">I am animated longer</div>

...
<section>
  <div class="trigger">I create the trigger</div>
  <div class="trigger longer">I create a longer trigger</div>
</section>
...
```

```
<div class="animated">I am animated</div>
<div class="animated longer">I am animated longer</div>
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
```

```
  tristique tellus, sed tincidunt velit.
</p>

<section>
  <div class="trigger">I create the trigger</div>
  <div class="trigger longer">I create a longer trigger</div>
</section>
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` elements' `position` is set to `fixed`, positioning them near the top-left of the scrollport to enable us to see when their animations start and stop.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
.animated.longer {
  left: 150px;
}
section {
  display: flex;
  gap: 20px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` elements. Without an associated trigger, the elements would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The values reference a `timeline-trigger-name` of `--t` and `--longer-t`, respectively, and define the same `<animation-action>` values on each — `play` and `pause` — which specify that the animations will play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
.animated.longer {
  animation-trigger: --longer-t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.

- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` of `contain 25% contain 75%`. The `contain` range spans from when the trigger element has completely entered the scrollport to when it starts to leave. This value sets the trigger's activation range to start `25%` of the way through the `contain` range and end `75%` of the way through the range. In other words, the activation range is the middle half of the scrollport.

The `.trigger.longer` element creates the `.animated.longer` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--longer-t` (overriding the `--t`), which is equal to the identifier referenced in the `.animated.longer` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-active-range` of `cover 0% cover 100%`. The `cover` range spans from when the trigger element starts to enter the scrollport to when it has completely left. In other words, the active range is when any part of the tracked element is in the scrollport.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: contain 25% contain 75%;
}
.trigger.longer {
  timeline-trigger-name: --longer-t;
  timeline-trigger-active-range: cover 0% cover 100%;
}
```

```
@supports not (timeline-trigger-active-range: contain 0%) {
  body::before {
    content: "Your browser does not support the timeline-trigger-active-range property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up. Both animations start playing when the tracked `.trigger` elements are about a quarter of the way up the scrollport. Continue scrolling. The first animation pauses when its trigger element gets to 75% through the scrollport, whereas the second animation doesn't pause until its trigger element has completely left the scrollport.

This is because the active range extends how long the trigger remains active, but doesn't change where activation occurs.

See also

- `timeline-trigger-active-range-end`, `timeline-trigger-active-range-start`
- `animation-trigger`
- `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-activation-range`
- `timeline-trigger` shorthand property
- `trigger-scope`
- `<animation-action>` type
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-active-range-end CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-active-range-end`** CSS property specifies the end of a scroll-triggered animation trigger's active range.

Syntax

```
/* Keywords */
timeline-trigger-active-range-end: auto;
timeline-trigger-active-range-end: normal;

/* <length-percentage> */
timeline-trigger-active-range-end: 80%;
timeline-trigger-active-range-end: 400px;

/* Named timeline range */
timeline-trigger-active-range-end: contain;
timeline-trigger-active-range-end: exit;

/* Named timeline with <length-percentage> */
timeline-trigger-active-range-end: exit -10px;
timeline-trigger-active-range-end: contain 110%;

/* Multiple range end values */
timeline-trigger-active-range-end:
  contain 110%,
  exit -10px;

/* Global values */
timeline-trigger-active-range-end: inherit;
timeline-trigger-active-range-end: initial;
timeline-trigger-active-range-end: revert;
timeline-trigger-active-range-end: revert-layer;
timeline-trigger-active-range-end: unset;
```

Values

This property is specified as a comma-separated list of the following values:

`auto`

Specifies the value of the `timeline-trigger-activation-range-end` property. This is the default value.

`normal`

Specifies the end, or `100%`, of the `normal` range. Equivalent to `cover 100%` for a view progress timeline `timeline-trigger-source`, and `scroll 100%` for a scroll progress timeline.

`<length-percentage>`

Specifies a length or percentage value measured from the beginning of the `normal` timeline. Percentages are relative to the length of the `normal` timeline range.

`<timeline-range-name>`

Specifies the end, or `100%`, of the `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll` timeline range.

`<timeline-range-name> <length-percentage>`

Specifies a length or percentage value measured from the beginning of the specified named timeline range. Percentages are relative to the length of the named range.

Description

The `timeline-trigger-active-range-end` property can be used to explicitly set the end of a trigger's active range to a value equal to or further along the timeline than the `timeline-trigger-activation-range-end`, which specifies the end of the trigger's activation range.

The *active range* is the range within which a trigger remains activated once activation has occurred. Activation occurs when the tracked element enters the *activation range* and deactivation occurs when it leaves the *active* range. By default, the active range ends where the activation range ends. This property creates a buffer zone and is used to prevent premature resetting when a user scrolls back and forth across the activation range's endpoint. Only when a tracked element moves out of the active range does the trigger become inactive.

The default value of `timeline-trigger-active-range-end` is `auto`, which sets the value to the same named range and offset as the `timeline-trigger-activation-range-end`. When specified as a timeline range, offset, or both, this property sets the end of the active range to a point that is independent of the `timeline-trigger-activation-range-end` value. If the value doesn't extend the end of the activation range, it has no effect.

The value of `normal` sets the end of the active range to the end of the default named range, resolving to either `cover 100%` for a view progress timeline (the `timeline-trigger-source` is set to a `view()` function) or `scroll 100%` for a scroll progress timeline (the `timeline-trigger-source` is set to a `scroll()` function).

Other values of the `timeline-trigger-active-range-end` property can be used to set:

An offset from the `normal` range

A `<length>` or `<percentage>` value specifies an offset from the beginning of the `normal` timeline, which again defaults to either `cover` or `scroll`. Negative values outset the end, resulting in a longer active range. Positive values inset the end of the active range, making it shorter.

The end of a specific named range

A `<timeline-range-name>` value specifies a `100%` offset from the start of the named timeline range, which is either `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll`. See Understanding timeline range names.

An offset from a specific named range

When both a `<timeline-range-name>` and `<length>` or `<percentage>` value are specified, the end is offset by the distance specified from the start of the named range. Percentage values are relative to the range specified. See Setting insets using percentages.

If the value is set to a point prior to the end of the activation range, the value of the `timeline-trigger-activation-range-end` is used, as if the value were set to `auto`.

The `timeline-trigger-active-range-end` property, along with the `timeline-trigger-active-range-start` property, can also be set using the `timeline-trigger-active-range` shorthand, which in turn can be set using the `timeline-trigger` shorthand.

Specifying multiple range end values

When you specify multiple comma-separated values in a single `timeline-trigger-active-range-end` declaration, they apply to the timeline triggers in the order in which they appear in the `timeline-trigger-name` property. When the number of triggers and `timeline-trigger-active-range-end` property values do not match, they are applied in the same way as multiple animation property values.

For example, if multiple `timeline-trigger-name` values are set, but only a single `timeline-trigger-active-range-end` value is set, the `timeline-trigger-active-range-end` will apply to all the `timeline-trigger-name`s. If two or more `timeline-trigger-active-range-end` values are set, they will cycle between the `timeline-trigger-name`s until every timeline trigger has a `timeline-trigger-active-range-end` value set.

Consider these declarations:

```
timeline-trigger-name: --my-trigger, --my-other-trigger, --another-trigger;
timeline-trigger-active-range-end:
  110%,
  exit 300px;
```

In this case, `--my-trigger` will use the `110%` range end and `--my-other-trigger` will use the `exit 300px` range end. As there are three names but only two range ends, the range ends are cycled, so the third trigger name, `--another-trigger`, will use the `110%` range end.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
timeline-trigger-active-range-end =
  [ auto | normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the effect of extending a trigger's active range. It compares two identical triggered animations, with the end of one animation trigger's active range outset using the `timeline-trigger-active-range-end` property.

HTML

The markup contains four `<div>` elements — two to animate and two to create a trigger on — plus some basic text content to cause the page to scroll. The text content is hidden for brevity.

```
<div class="animated">I am animated</div>
<div class="animated longer">I am animated longer</div>

...
<section>
  <div class="trigger">I create the trigger</div>
  <div class="trigger longer">I create a longer trigger</div>
</section>
...
```

```
<div class="animated">I am animated</div>
<div class="animated longer">I am animated longer</div>
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<section>
  <div class="trigger">I create the trigger</div>
  <div class="trigger longer">I create a longer trigger</div>
</section>
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` elements' `position` is set to `fixed`, positioning them near the top-left of the scrollport to enable us to see when their animations start and stop.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
```

```
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
.animated.longer {
  left: 150px;
}
section {
  display: flex;
  gap: 20px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` elements. Without an associated trigger, the elements would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The values reference a `timeline-trigger-name` of `--t` and `--longer-t`, respectively, and define two `<animation-action>` values — `play` and `pause` — which specify that the animation will play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
.animated.longer {
  animation-trigger: --longer-t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range-end` of `contain 50%`. The `contain` range spans from when the trigger element has completely entered the scrollport to when it starts to leave. This value sets the trigger's activation range to end `50%` through it, which occurs when it is vertically centered in the scrollport.

The `.trigger.longer` element creates the `.animated.longer` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--longer-t` (overriding the `--t`), which is equal to the identifier referenced in the `.animated.longer` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-active-range-end` of `cover 100%`. The `cover` range spans from when the trigger element first starts to enter the scrollport to when it has completely left it.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range-end: contain 50%;
}
.trigger.longer {
  timeline-trigger-name: --longer-t;
  timeline-trigger-active-range-end: cover 100%;
}
```

```
@supports not (timeline-trigger-active-range-end: cover 100%) {
  body::before {
    content: "Your browser does not support the timeline-trigger-active-range-end property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up. Both animations start playing when the tracked `.trigger` elements first enter into view. The animation of one element pauses when the trigger is at `50%` of the `contain` timeline. The other element only pauses when the trigger has fully exited the viewport.

When you scroll downward again, after both animations have paused, both animations will restart when the trigger elements reach the `50%` point. This is because the active range extends how long the trigger remains active, but does not change where activation occurs.

See also

- `timeline-trigger-active-range-start`
- `timeline-trigger-active-range` shorthand property
- `timeline-trigger-activation-range-start`
- `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-activation-range`
- `timeline-trigger` shorthand property
- Using CSS scroll-triggered animations

- CSS animation triggers module
- CSS animations module

timeline-trigger-active-range-start CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-active-range-start`** CSS property specifies the start of a scroll-triggered animation trigger's active range.

Syntax

```
/* Keywords */
timeline-trigger-active-range-start: auto;
timeline-trigger-active-range-start: normal;

/* <length-percentage> */
timeline-trigger-active-range-start: 10%;
timeline-trigger-active-range-start: 50px;

/* Named timeline range */
timeline-trigger-active-range-start: contain;
timeline-trigger-active-range-start: exit;

/* Named timeline with <length-percentage> */
timeline-trigger-active-range-start: entry -5%;
timeline-trigger-active-range-start: contain 100px;

/* Multiple range start values */
timeline-trigger-active-range-start:
  contain -10px,
  entry 5%;

/* Global values */
timeline-trigger-active-range-start: inherit;
timeline-trigger-active-range-start: initial;
timeline-trigger-active-range-start: revert;
timeline-trigger-active-range-start: revert-layer;
timeline-trigger-active-range-start: unset;
```

Values

This property is specified as a comma-separated list of the following values:

`auto`

Specifies the value of the `timeline-trigger-activation-range-start` property. This is the default value.

`normal`

Specifies the start, or `0%`, of the `normal` range. Equivalent to `cover 0%` for a view progress timeline `timeline-trigger-source`, and `scroll 0%` for a scroll progress timeline.

`<length-percentage>`

Specifies a length or percentage value measured from the beginning of the `normal` timeline. Percentages are relative to the length of the `normal` timeline range.

`<timeline-range-name>`

Specifies the start, or `0%`, of the `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll` timeline range.

`<timeline-range-name> <length-percentage>`

Specifies a length or percentage value measured from the beginning of the specified named timeline range. Percentages are relative to the length of the named range.

Description

The `timeline-trigger-active-range-start` property can be used to explicitly set the start of a trigger's active range to a value equal to or preceding the `timeline-trigger-activation-range-start`, which specifies the start of the trigger's activation range.

The *active range* is the range in which a trigger remains activated once activation has occurred. Activation occurs when the tracked element enters the *activation range*, and deactivation occurs when it leaves the *active range*. By default, the active range starts where the activation range starts. This property creates a buffer zone and is used to prevent premature resetting when a user scrolls back and forth across the activation's starting point. Only when a tracked element moves out of the *active range* does the trigger become inactive.

The default value of `timeline-trigger-active-range-start` is `auto`, which sets the value to the same named range and offset as the `timeline-trigger-activation-range-start`. When specified as a timeline range, offset, or both, this property sets the start of the active range to a point that is independent of the `timeline-trigger-activation-range-start` value. If the range start position doesn't extend the activation range, it has no effect.

The value of `normal` sets the start of the active range to the start of the default named range, resolving to either `cover 0%` for a view progress timeline (the `timeline-trigger-source` is set to a `view()` function) or `scroll 0%` for a scroll progress timeline (the `timeline-trigger-source` is set to a `scroll()` function).

Other values of the `timeline-trigger-active-range-start` property can be used to set:

An offset from the `normal` range

A `<length>` or `<percentage>` value specifies an offset from the beginning of the `normal` timeline of `cover` or `scroll`. Negative values outset the start, resulting in a longer active range. Positive values inset the start of the active range, making it shorter.

The start of a specific named range

A `<timeline-range-name>` value specifies a `0%` offset along the named timeline range, which is either `cover`, `contain`, `entry`, `exit`, `entry-crossing`, `exit-crossing`, or `scroll`. See Understanding timeline range names.

An offset from a specific named range

When both a `<timeline-range-name>` and `<length>` or `<percentage>` value are specified, the start is offset by the distance specified from the start of the named range. Percentage values are relative to the range specified. See Setting insets using percentages.

The `timeline-trigger-active-range-start` property, along with the `timeline-trigger-active-range-end` property, can also be set using the `timeline-trigger-active-range` shorthand, which in turn can be set using the `timeline-trigger` shorthand.

Specifying multiple range start values

When you specify multiple comma-separated values in a single `timeline-trigger-active-range-start` declaration, they apply to the timeline triggers in the order in which they appear in the `timeline-trigger-name` property. When the number of triggers and `timeline-trigger-active-range-start` property values do not match, they are applied in the same way as multiple animation property values.

For example, if multiple `timeline-trigger-name` values are set, but only a single `timeline-trigger-active-range-start` value is set, the `timeline-trigger-active-range-start` will apply to all the `timeline-trigger-names`. If two or more `timeline-trigger-active-range-start` values are set, they will cycle between the `timeline-trigger-names` until every timeline trigger has a `timeline-trigger-active-range-start` value set.

Consider these declarations:

```
timeline-trigger-name: --my-trigger, --my-other-trigger, --another-trigger;
timeline-trigger-active-range-start:
  contain,
  entry 5%;
```

In this case, `--my-trigger` will use the `contain` range start and `--my-other-trigger` will use the `entry 5%` range start. As there are three names but only two range starts, the range starts are cycled, so the third trigger name, `--another-trigger`, will use the `contain` range start.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Percentages	Relative to the specified named timeline range if specified, otherwise relative to the entire timeline
Computed value	A list where each item may be 'normal', a length percentage, or a timeline range name and a length percentage
Animation type	Not animatable

Formal syntax

```
timeline-trigger-active-range-start =
  [ auto | normal | <length-percentage> | <timeline-range-name> <length-percentage>? ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the effect of extending a trigger's active range. It compares two identical triggered animations, with the start of one animation trigger's active range outset using the `timeline-trigger-active-range-start` property.

HTML

The markup contains four `<div>` elements — two to animate and two to create a trigger on — plus some basic text content to cause the page to scroll. The text content is hidden for brevity.

```
<div class="animated">I am animated</div>
<div class="animated longer">I am animated longer</div>

...
<section>
  <div class="trigger">I create the trigger</div>
  <div class="trigger longer">I create a longer trigger</div>
</section>
...
```

```
<div class="animated">I am animated</div>
<div class="animated longer">I am animated longer</div>
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<section>
  <div class="trigger">I create the trigger</div>
  <div class="trigger longer">I create a longer trigger</div>
</section>
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
```

```
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The `.animated` elements' `position` is set to `fixed`, positioning them near the top-left of the scrollport to enable us to see when their animations start and stop.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
.animated.longer {
  left: 150px;
}
section {
  display: flex;
  gap: 20px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `rotate` animation is applied to the `.animated` elements. Without an associated trigger, the elements would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The values reference a `timeline-trigger-name` of `--t` and `--longer-t`, respectively, and define the same two `<animation-action>` values — `play` and `pause` — which specify that the animations will play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
.animated.longer {
  animation-trigger: --longer-t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` of `contain 50% contain 100%`. The `contain` range spans from when the trigger element has completely entered the scrollport to when it starts to leave. This value sets the trigger's activation range to start at `50%` through the `contain` range (when the tracked element's bottom edge is halfway through the scrollport), and end at `100%`, when the element's top edge first exits the scrollport.

The `.trigger.longer` element creates the `.animated.longer` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--longer-t` (overriding the `--t`), which is equal to the identifier referenced in the `.animated.longer` element's `animation-trigger` property value, associating the two together.

- A `timeline-trigger-active-range-start` of `contain 0%`, which occurs when the trigger's bottom edge is at the bottom edge of the scrollport.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: contain 50% contain 100%;
}
.trigger.longer {
  timeline-trigger-name: --longer-t;
  timeline-trigger-active-range-start: contain 0%;
}
```

```
@supports not (timeline-trigger-active-range-start: contain 0%) {
  body::before {
    content: "Your browser does not support the timeline-trigger-active-range-start property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up. Both animations start playing when the tracked `.trigger` elements get to the middle of the scrollport and stop playing when they start to leave the top edge of the scrollport.

If you then scroll downward, both animations start playing when they are fully in the scrollport, with the top edge of the tracked elements meeting the top edge of the scrollport. The first animation pauses when the tracked element exits the activation range, when it passes the center of the scrollport. The second animation doesn't pause until its trigger element starts to leave the scrollport at its bottom edge. This is because the active range on the second animation extends how long the trigger remains active.

See also

- `timeline-trigger-active-range-end`
- `timeline-trigger-active-range` shorthand property
- `timeline-trigger-activation-range-end`
- `timeline-trigger-name`, `timeline-trigger-source`, and `timeline-trigger-activation-range`
- `timeline-trigger` shorthand property
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-name CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-name`** CSS property specifies a scroll-triggered animation trigger's identifier(s).

Syntax

```
/* Keyword */
timeline-trigger-name: none;

/* Single dashed ident */
timeline-trigger-name: --my-trigger;
timeline-trigger-name: --my-other-trigger;

/* Multiple dashed idents */
timeline-trigger-name: --my-trigger, --my-other-name;

/* Global values */
timeline-trigger-name: inherit;
timeline-trigger-name: initial;
timeline-trigger-name: revert;
timeline-trigger-name: revert-layer;
timeline-trigger-name: unset;
```

Values

Specified as one or more `<dashed-ident>` values separated by commas, or the keyword `none`.

`none`

Specifies that the element does not define any scroll-triggered animation triggers.

`<dashed-ident>`

Specifies the name of the timeline trigger.

Description

The `timeline-trigger-name` property specifies one or more identifying names for a CSS scroll-triggered animation trigger. This identifier is used within the `animation-trigger` property value.

For example:

```
.trigger {
  timeline-trigger-name: --my-trigger;
  timeline-trigger-source: view();
}
```

An element with these declarations set will create a trigger with the name `--my-trigger`. The `timeline-trigger-source` declaration is needed to create a timeline to control triggering animations; in this case, the value of `view()` creates an anonymous view progress timeline.

The resulting `ViewTimeline` tracks the position of the `.trigger` element across the block-axis of the nearest ancestor scroller. The trigger is activated and deactivated when the tracked element is scrolled to specific positions inside the scrollport.

Any animated element with its `animation-trigger` property set to `--my-trigger` will have its animation controlled by the `--my-trigger` trigger:

```
.animated {
  animation: rotate 3s infinite linear both;
  animation-trigger: --my-trigger play-once;
}
```

Each `animation-trigger` value includes two or three components: the `<dashed-ident>` identifying the trigger, and one or two `<animation-action>` keywords specifying what should happen when the trigger is activated and, optionally, when it's deactivated. In this case, the animation will play once when activated.

The animated element can create its own trigger: to achieve this, set its `timeline-trigger-name` and `animation-trigger` properties to include the same `<dashed-ident>` value:

```
.animatedAndTrigger {
  animation: rotate 3s infinite linear both;
  animation-trigger: --my-trigger play-once;

  timeline-trigger-name: --my-trigger;
  timeline-trigger-source: view();
}
```

The `timeline-trigger-name` property, along with the `timeline-trigger-source`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range` properties, can also be set using the `timeline-trigger` shorthand property.

Multiple trigger names

You can specify multiple, comma-separated `<dashed-ident>` values in the same trigger's `timeline-trigger-name` value, meaning that it can be referenced by multiple identifiers. This can be useful in a situation where you have multiple components dropped into a page that you want to trigger animations on, but which use predefined `animation-trigger` values that you can't edit.

You could use multiple `timeline-trigger-name` values to create a trigger for all the animations:

```
.trigger {
  timeline-trigger-name: --animation-trigger-1, --animation-trigger-2;
  timeline-trigger-source: view();
}
```

If you specify the same `<dashed-ident>` multiple times in the same `timeline-trigger-name` list, only the last instance will define a trigger. The preceding ones will have no effect.

If multiple elements define triggers with the same trigger name, the trigger defined by the last element in the source order is used, unless scoped. See `trigger-scope` for more details.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
timeline-trigger-name =
  none             |
  <dashed-ident>#
```

This syntax reflects the latest standard as per Animation Triggers. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example shows how to create a basic scroll-triggered animation. We use the `timeline-trigger-name` property to name a trigger and reference that name within an animated element's `animation-trigger` property.

HTML

Our markup contains two `<div>` elements, one to animate and one on which to create a trigger, plus some basic text content to cause the page to scroll. We have hidden the text content for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
```

```
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The animated `<div>` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport to enable us to see when its animation starts and stops.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` for the `rotate` animation we will use later:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, we apply the `rotate` animation to the `.animated` element. Without an associated trigger, the element would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The value references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause` — which specify that the animation will play on activation, and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

The `.trigger` element creates the `.animated` element's trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the animated element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
```

Result

Try scrolling the content up. When any part of the tracked `.trigger` element appears in the scrollport, the animation plays; when it has completely left the scrollport at either edge, the animation pauses.

Making the animated element the trigger

This example demonstrates how an animated element can create its own trigger.

HTML

This time, our markup contains a single `<div>` element.

```
<div>I create my own trigger</div>
```

The text content that causes the page to scroll has been hidden for brevity.

```
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div>I create my own trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

We define an `invert-colors` keyframe animation that inverts the foreground and background colors:

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  background: orange;
  border: 5px solid black;
}
```

```
@keyframes invert-colors {
  from {
    background: orange;
    color: black;
```

```
  }

  to {
    background: black;
    color: orange;
  }
}
```

We set an `animation-trigger` value that references a `timeline-trigger-name` of `--t`. We also specify two `<animation-action>` values — `play-forwards` and `play-backwards` — which set the animation to play forwards on activation and in reverse on deactivation.

We then specify the following properties on the same `<div>`:

- A `timeline-trigger-name` with value `--t`. This specifies that the `<div>` creates the trigger for its own animation.
- A `timeline-trigger-source` with value `view()`. This sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` with value `contain`. This means the trigger will activate when the tracked element is fully inside the scrollport. Because `timeline-trigger-active-range` defaults to `auto`, the active range is the same as the activation range; therefore, the trigger will deactivate when it stops being fully inside the scrollport. This is in contrast to the default activation and active range, `cover`, which would cause the trigger to activate when any part of the element enters the scrollport and deactivate only when it has fully left the scrollport. This would make the reverse animation occur when the element is out of view.

```
div {
  animation: invert-colors 0.6s ease-in both;

  animation-trigger: --t play-forwards play-backwards;

  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: contain;
}
```

```
@supports not (timeline-trigger-name: --t) {
  body::before {
    content: "Your browser does not support the timeline-trigger-name property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Try scrolling the content up. The tracked element's animation plays after it has fully entered the scrollport, and pauses when it starts to leave the scrollport.

See also

- `animation-trigger`
- `timeline-trigger-source`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand property
- `trigger-scope`
- `<animation-action>` type
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

timeline-trigger-source CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`timeline-trigger-source`** CSS property specifies the timeline that will trigger a scroll-triggered animation.

Syntax

```
/* Keywords */
timeline-trigger-source: none;
timeline-trigger-source: auto;

/* Named timeline */
timeline-trigger-source: --my-timeline;

/* Anonymous scroll progress timeline */
timeline-trigger-source: scroll();
timeline-trigger-source: scroll(x root);

/* Anonymous view progress timeline */
timeline-trigger-source: view();
timeline-trigger-source: view(inline);
timeline-trigger-source: view(x 200px auto);

/* Multiple source values */
timeline-trigger-source: view(), none, --my-timeline;
timeline-trigger-source: scroll(x), auto, scroll(y root);

/* Global values */
timeline-trigger-source: inherit;
timeline-trigger-source: initial;
timeline-trigger-source: revert;
timeline-trigger-source: revert-layer;
timeline-trigger-source: unset;
```

Values

This property is specified as a comma-separated list of the following values:

`none`

The element's trigger does not have a source: it is not associated with any timeline and the animation does not happen.

`auto`

The element's trigger source is the document's default time-based `DocumentTimeline`. This is the default value.

`<dashed-ident>`

The element creates a scroll-triggered animation trigger as a named view progress timeline.

`scroll()`

The element creates a scroll-triggered animation trigger as an anonymous scroll progress timeline.

`view()`

The element creates a scroll-triggered animation trigger as an anonymous view progress timeline.

Description

The `timeline-trigger-source` property specifies the timeline trigger that will control a scroll-triggered animation.

For example:

```
.trigger {
  timeline-trigger-name: --my-trigger;
  timeline-trigger-source: view();
}
```

The resulting `ViewTimeline` tracks the position of the `.trigger` element across the block-axis of the nearest ancestor scroller. The trigger activates and deactivates when the tracked element is scrolled to certain positions inside the scrollport. By default, when `timeline-trigger-source` is set to `view()`, activation occurs when the tracked element starts to enter the scrollport, and deactivation occurs when the tracked element completely exits the scrollport.

An animated element can be triggered by the previously described trigger by referencing its `timeline-trigger-name` in its `animation-trigger` property. The `animation-trigger` value consists of a comma-separated list, each containing the name of a trigger and one or two `<animation-action>` keywords that specify what the animation should do when its trigger activates and deactivates.

For example:

```
.animated {
  animation: rotate 3s infinite linear both;
  animation-trigger: --my-trigger play-once;
}
```

The animated element and the element that creates the trigger can be the same element. In this case, the animated element creates its own trigger:

```
.animatedAndTrigger {
  animation: rotate 3s infinite linear both;
  animation-trigger: --my-trigger play-once;

  timeline-trigger-name: --my-trigger;
  timeline-trigger-source: view();
}
```

The `timeline-trigger-source` property, along with the `timeline-trigger-name`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range` properties, can also be set using the `timeline-trigger` shorthand property.

Trigger source types

To create a triggered animation, set the `timeline-trigger-source` property to one of three main value types:

- A `view()` function referencing an anonymous view progress timeline trigger. This is created on the nearest scrolling ancestor of the element that creates the trigger. As shown earlier, this allows you to create functionality whereby an element will start animating when it (or another element) reaches a certain scroll offset in the scrollport, and stop animating (or some other action) when it (or another element) reaches a different scroll offset. For example:

  ```
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  ```

- A `scroll()` function referencing an anonymous scroll progress timeline trigger. You can create this on the root element or the nearest scroller of the element that creates the trigger. This allows you to create functionality whereby an element will start animating when it (or another element) reaches an absolute scroll offset (for example, it scrolls upwards by `600px`), and stop animating (or some other action) when it (or another element) reaches a different offset. For example:

  ```
  timeline-trigger-name: --t;
  timeline-trigger-source: scroll();
  timeline-trigger-activation-range: 600px;
  ```

  > **Note:** See the `scroll()` example.

- A `<dashed-ident>` referencing a named view progress timeline or named scroll progress timeline. This involves setting a `view-timeline-name` or `scroll-timeline-name` on the element that creates the trigger, and then referring to that name in the value of the `timeline-trigger-source` property, for example:

  ```
  view-timeline-name: --my-timeline;
  timeline-trigger-source: --my-timeline;
  ```

Scroll progress timelines are arguably less useful for scroll-triggered animations than view progress timelines. You are more likely to want an animation to start at a scroll offset relative to the scrollport, not after an arbitrary amount of scrolling, where the animation may well be triggered offscreen on smaller screens.

Other values

It is also possible to set `timeline-trigger-source` to a keyword of `auto` or `none`. Both result in a non-scroll-triggered animation, but their effects differ.

- The default value, `auto`, sets the element's trigger source to the document's default time-based `DocumentTimeline`. This results in any animations applied to the element playing on page load.
- The `none` value results in the element's trigger having no source, which means that any animations applied to the element will not play at all.

Multiple sources

When you specify multiple comma-separated values on a single `timeline-trigger-source` property, they are applied to the timeline triggers in the order in which the `timeline-trigger-name`s appear. When the number of triggers and `timeline-trigger-source` property values do not match, they are applied in the same way as multiple animation property values.

For example, if multiple `timeline-trigger-name` values are set, but only a single `timeline-trigger-source` value is set, the `timeline-trigger-source` will apply to all the `timeline-trigger-name`s. If two `timeline-trigger-source` values are set, they will cycle between the `timeline-trigger-name`s until all of them have a `timeline-trigger-source` value set. And so on.

Consider these declarations:

```
timeline-trigger-name: --my-trigger, --my-other-trigger, --another-trigger;
timeline-trigger-source: view(), --my-source;
```

In this case, the first name will use the `view()` source, and the second name will use the `--my-source` source. The third name will cycle back to using the `view()` source again.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Computed value	A list, with each item being either the keyword `none`, the keyword `auto`, a case-sensitive `<ident>`, a computed `scroll()` function, or a computed `view()` function.
Animation type	Not animatable

Formal syntax

```
timeline-trigger-source =
  [ none | auto | [ <dashed-ident> | <scroll()> | <view()> ]+ ]#

<scroll()> =
  scroll( [ <scroller> || <axis> ]? )

<view()> =
  view( [ <axis> || <'view-timeline-inset'> ]? )

<scroller> =
  root     |
  nearest  |
  self

<axis> =
  block   |
  inline  |
  x       |
  y

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per Animation Triggers, CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic view progress timeline source usage

In this example, we create a basic scroll-triggered animation that uses an anonymous view progress timeline trigger source.

HTML

The markup contains two `<div>` elements: one to animate and one to create a trigger, plus basic text content to make the page scroll. The text is hidden for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
```

```
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

The animated `<div>` element's `position` is set to `fixed`, positioning it near the top-left of the scrollport to enable us to see when its animation starts and stops.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` for the `rotate` animation we will use later:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

Using the `animation` shorthand, the `.animated` element has the `rotate` animation applied. Without an associated trigger, the element would start animating when the page loads. The `animation-trigger` property makes it a triggered animation. The value references a `timeline-trigger-name` of `--t` and specifies two `<animation-action>` values — `play` and `pause` — which specify that the animation will play on activation and pause on deactivation.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

The `.trigger` element creates the animated `<div>`'s trigger via the following properties:

- A `timeline-trigger-name` with value `--t`, which is equal to the identifier referenced in the `.animated` element's `animation-trigger` property value, associating the two together.
- A `timeline-trigger-source` with value `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
}
```

Result

Try scrolling the content up. When any part of the `.trigger` appears in the scrollport, the animation will play; when it has completely left the scrollport at either edge, the animation will pause.

Basic scroll progress timeline source usage

This example is nearly identical to the previous one, except that this time we set `timeline-trigger-source` to an anonymous scroll progress timeline instead of an anonymous view progress timeline.

The HTML and CSS are nearly identical, except that this time we have set our `.trigger` element's `timeline-trigger-source` to `scroll()` instead of `view()`. This creates the trigger as an anonymous scroll progress timeline on the element's nearest scrolling ancestor.

We have also set a `timeline-trigger-activation-range` of `600px`, which means that the trigger will activate (meaning the animation will start playing) when the tracked element scrolls upwards by `600px`. If we didn't set this, the trigger would activate immediately on page load.

```
.animated {
  animation: rotate 3s infinite linear;
  animation-trigger: --t play pause;
}
```

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: scroll();
  timeline-trigger-activation-range: 600px;
}
```

```
@supports not (timeline-trigger-source: scroll()) {
  body::before {
    content: "Your browser does not support the timeline-trigger-source property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

The animation will start when the tracked element scrolls `600px` upwards.

See also

- `timeline-trigger-name`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand property
- `animation-trigger`
- `<animation-action>` type
- `trigger-scope`
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

top CSS property

The **`top`** CSS property sets the vertical position of a positioned element. This inset property has no effect on non-positioned elements.

Try it

```
top: 0;
```

```
top: 4em;
```

```
top: 10%;
```

```
top: 20px;
```

```
<section id="default-example">
  <div class="example-container">
    <div id="example-element">I am absolutely positioned.</div>
    <p>
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </p>
  </div>
</section>
```

```
.example-container {
  border: 0.75em solid;
  padding: 0.75em;
  text-align: left;
  position: relative;
  width: 100%;
  min-height: 200px;
}

#example-element {
  background-color: #264653;
  border: 4px solid #ffb500;
  color: white;
  position: absolute;
  width: 140px;
  height: 60px;
}
```

The effect of `top` depends on how the element is positioned (i.e., the value of the `position` property):

- When `position` is set to `absolute` or `fixed`, the `top` property specifies the distance between the element's outer margin of the top edge and the inner border of the top edge of its containing block, or, in the case of anchor positioned elements when the `anchor()` function is used within the value, relative to the specified `<anchor-side>` edge. The `top` property is compatible with the `top`, `bottom`, `start`, `end`, `self-start`, `self-end`, `center`, and `<percentage>` values.
- When `position` is set to `relative`, the `top` property specifies the distance the element's top edge is moved below its normal position.
- When `position` is set to `sticky`, the `top` property is used to compute the sticky-constraint rectangle.
- When `position` is set to `static`, the `top` property has *no effect*.

When both `top` and `bottom` values are specified, there are three different cases:

- If `position` is set to `absolute` or `fixed` and `height` is unspecified (either `auto` or `100%`), both the `top` and `bottom` values are respected.
- If `position` is set to `relative` or `height` is constrained, the `top` property takes precedence and the `bottom` property is ignored.
- If `position` is set to `sticky`, both `top` and `bottom` values are considered. This means that a sticky element can potentially move up and down within its containing block based on the values of these two properties as long as the element's position box remains contained within its containing block.

Syntax

```
/* <length> values */
top: 3px;
top: 2.4em;
top: anchor(bottom);
top: anchor-size(--my-anchor self-block, 10%);

/* <percentage>s of the height of the containing block */
top: 10%;

/* Keyword value */
top: auto;

/* Global values */
top: inherit;
top: initial;
top: revert;
top: revert-layer;
top: unset;
```

Values

`<length>`

A negative, null, or positive `<length>`:

- for *absolutely positioned elements*, it represents the distance to the top edge of the containing block.
- for *anchor-positioned elements*, the `anchor()` function resolves to a `<length>` value relative to the position of the associated *anchor element*'s top or bottom edge (see Using inset properties with `anchor()` function values), and the `anchor-size()` function resolves to a `<length>` value relative to the associated anchor element's width or height (see Setting element position based on anchor size).
- for *relatively positioned elements*, it represents the distance that the element is moved below its normal position.

`<percentage>`

A `<percentage>` of the containing block's height.

`auto`

Specifies that:

- for *absolutely positioned elements*, the position of the element is based on the `bottom` property, while `height: auto` is treated as a height based on the content; or if `bottom` is also `auto`, the element is positioned where it should vertically be positioned if it were a static element.
- for *relatively positioned elements*, the distance of the element from its normal position is based on the `bottom` property; or if `bottom` is also `auto`, the element is not moved vertically at all.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Percentages	refer to the height of the containing block
Computed value	if specified as a length, the corresponding absolute length; if specified as a percentage, the specified value; otherwise, `auto`
Animation type	a length, percentage or calc();

Formal syntax

```
top =
  auto                |
  <length-percentage>  |
  <anchor()>           |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor()> =
  anchor( <anchor-name>?                 &&
  <anchor-side> , <length-percentage>? )

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside        |
  outside       |
  top           |
  left          |
  right         |
```

```
  bottom        |
  start         |
  end           |
  self-start    |
  self-end      |
  <percentage>  |
  center

<anchor-size> =
  width         |
  height        |
  block         |
  inline        |
  self-block    |
  self-inline
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Positioned Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A positioned element set 10% from the top

```
body {
  background: beige;
}

div {
  position: absolute;
  top: 10%;
  right: 40%;
  bottom: 20%;
  left: 15%;
  background: gold;
  border: 1px solid blue;
}
```

```
<div>The size of this content is determined by the position of its edges.</div>
```

See also

- `bottom`, `left`, and `right`
- `inset` shorthand
- `inset-block-start`, `inset-block-end`, `inset-inline-start`, and `inset-inline-end`
- `inset-block` and `inset-inline` shorthands
- `position`
- CSS positioned layout module

touch-action CSS property

The **`touch-action`** CSS property sets how an element's region can be manipulated by a touchscreen user (for example, by zooming features built into the browser).

By default, panning (scrolling) and pinching gestures are handled exclusively by the browser. An application using Pointer events will receive a `pointercancel` event when the browser starts handling a touch gesture. By explicitly specifying which gestures should be handled by the browser, an application can supply its own behavior in `pointermove` and `pointerup` listeners for the remaining gestures. Applications using Touch events disable the browser handling of gestures by calling `preventDefault()`, but should also use `touch-action` to ensure the browser knows the intent of the application before any event listeners have been invoked.

When a gesture is started, the browser intersects the `touch-action` values of the touched element and its ancestors, up to the one that implements the gesture (in other words, the first containing scrolling element). This means that in practice, `touch-action` is typically applied only to top-level elements which have some custom behavior, without needing to specify `touch-action` explicitly on any of that element's descendants.

> **Note:** After a gesture starts, changes to `touch-action` will not have any impact on the behavior of the current gesture.

Syntax

```
/* Keyword values */
touch-action: auto;
touch-action: none;
touch-action: pan-x;
touch-action: pan-left;
touch-action: pan-right;
touch-action: pan-y;
touch-action: pan-up;
touch-action: pan-down;
touch-action: pinch-zoom;
touch-action: manipulation;

/* Global values */
touch-action: inherit;
touch-action: initial;
touch-action: revert;
touch-action: revert-layer;
touch-action: unset;
```

The `touch-action` property may be specified as either:

- One of the keywords `auto`, `none`, `manipulation`, *or*
- One of the keywords `pan-x`, `pan-left`, `pan-right`, and/or one of the keywords `pan-y`, `pan-up`, `pan-down`, plus optionally the keyword `pinch-zoom`.

Values

`auto`

Enable browser handling of all panning and zooming gestures.

`none`

Disable browser handling of all panning and zooming gestures.

`pan-x`

Enable single-finger horizontal panning gestures. May be combined with **pan-y**, **pan-up**, **pan-down** and/or **pinch-zoom**.

`pan-y`

Enable single-finger vertical panning gestures. May be combined with **pan-x**, **pan-left**, **pan-right** and/or **pinch-zoom**.

`manipulation`

Enable panning and pinch zoom gestures, but disable additional non-standard gestures such as double-tap to zoom. Disabling double-tap to zoom removes the need for browsers to delay the generation of **click** events when the user taps the screen. This is an alias for "**pan-x pan-y pinch-zoom**" (which, for compatibility, is itself still valid).

`pan-left, pan-right, pan-up, pan-down`

Enable single-finger gestures that begin by scrolling in the given direction(s). Once scrolling has started, the direction may still be reversed. Note that scrolling "up" (**pan-up**) means that the user is dragging their finger downward on the screen surface, and likewise **pan-left** means the user is dragging their finger to the right. Multiple directions may be combined except when there is a simpler representation (for example, **"pan-left pan-right**" is invalid since "**pan-x**" is simpler, but "**pan-left pan-down**" is valid).

`pinch-zoom`

Enable multi-finger panning and zooming of the page. This may be combined with any of the **pan-** values.

Accessibility

A declaration of `touch-action: none;` may inhibit operating a browser's zooming capabilities. This will prevent people experiencing low vision conditions from being able to read and understand page content.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4 | Understanding WCAG 2.0

Formal definition

Initial value	`auto`
Applies to	all elements except: non-replaced inline elements, table rows, row groups, table columns, and column groups
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
touch-action =
  auto                                          |
  none                                          |
  [ [ pan-x | pan-left | pan-right ] || [ pan-y | pan-up | pan-down ] || pinch-zoom ]  |
  manipulation
```

This syntax reflects the latest standard as per Compatibility. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Disabling all gestures

The most common usage is to disable all gestures on an element (and its non-scrollable descendants) that provides its own dragging and zooming behavior – such as a map or game surface.

HTML

```
<div id="map"></div>
```

CSS

```
#map {
  height: 150vh;
  width: 70vw;
  background: linear-gradient(blue, green);
  touch-action: none;
}
```

Result

See also

- `pointer-events`
- Pointer Events
- WebKit Blog More Responsive Tapping on iOS
- Google Developers Blog Making touch scrolling fast by default
- Scroll Snap

transform CSS property

The **`transform`** CSS property lets you rotate, scale, skew, or translate an element. It modifies the coordinate space of the CSS visual formatting model.

If the property has a value different from `none`, a stacking context will be created. In that case, the element will act as a containing block for any `position: fixed;` or `position: absolute;` elements that it contains.

You can also use the individual transform properties: `translate`, `rotate`, and `scale`. These properties are applied in the order: `translate`, `rotate`, `scale`, and finally `transform`.

> **Warning:** Only transformable elements can be `transform`ed. That is, all elements whose layout is governed by the CSS box model except for: non-replaced inline boxes, table-column boxes, and table-column-group boxes.

Try it

```
transform: matrix(1, 2, 3, 4, 5, 6);
```

```
transform: translate(120px, 50%);
```

```
transform: scale(2, 0.5);
```

```
transform: rotate(0.5turn);
```

```
transform: skew(30deg, 20deg);
```

```
transform: scale(0.5) translate(-100%, -100%);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
/* Keyword value */
transform: none;

/* Function values */
transform: matrix(1, 2, 3, 4, 5, 6);
transform: matrix3d(1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1);
transform: perspective(17px);
transform: rotate(0.5turn);
transform: rotate3d(1, 2, 3, 10deg);
transform: rotateX(10deg);
transform: rotateY(10deg);
transform: rotateZ(10deg);
transform: translate(12px, 50%);
transform: translate3d(12px, 50%, 3em);
transform: translateX(2em);
transform: translateY(3in);
transform: translateZ(2px);
transform: scale(2, 0.5);
transform: scale3d(2.5, 1.2, 0.3);
transform: scaleX(2);
transform: scaleY(0.5);
transform: scaleZ(0.3);
transform: skew(30deg, 20deg);
transform: skewX(30deg);
transform: skewY(1.07rad);

/* Multiple function values */
transform: translateX(10px) rotate(10deg) translateY(5px);
transform: perspective(500px) translate3d(10px, 0, 20px) rotateY(30deg);

/* Global values */
transform: inherit;
transform: initial;
transform: revert;
transform: revert-layer;
transform: unset;
```

Values

This property is specified as either the keyword value `none` or as a space-separated list of `<transform-function>` values:

`<transform-function>`

> One or more of the CSS transform functions to be applied. The transform functions are combined from left to right – each function establishes a new coordinate space for the next function and so on – so the visual result matches the written order of the functions. Alternatively, keeping the parent coordinate space fixed, the same transformation can be described as applying the functions in reverse order (right to left).

`none`

> Specifies that no transform should be applied.

Accessibility

Scaling/zooming animations are problematic for accessibility, as they are a common trigger for certain types of migraine. If you need to include such animations on your website, you should provide a control to allow users to turn off animations, preferably site-wide.

Also, consider making use of the `prefers-reduced-motion` media feature — use it to write a media query that will turn off animations if the user has reduced animation specified in their system preferences.

Find out more:

- MDN Understanding WCAG, Guideline 2.3 explanations
- Understanding Success Criterion 2.3.3 | W3C Understanding WCAG 2.1

Formal definition

Initial value	none
Applies to	transformable elements
Inherited	no
Percentages	refer to the size of bounding box
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a transform
Creates stacking context	yes

Formal syntax

```
transform =
  none              |
  <transform-list>

<transform-list> =
```

```
  <transform-function>+

<transform-function> =
  <scale3d()>      |
  <scale()>        |
  <scaleX()>       |
  <scaleY()>       |
  <scaleZ()>       |
  <translate3d()>  |
  <translate()>    |
  <translateX()>   |
  <translateY()>   |
  <translateZ()>   |
  <rotate3d()>     |
  <rotate()>       |
  <rotateX()>      |
  <rotateY()>      |
  <rotateZ()>      |
  <skew()>         |
  <skewX()>        |
  <skewY()>        |
  <matrix3d()>     |
  <matrix()>       |
  <perspective()>

<scale3d()> =
  scale3d( [ <number> | <percentage> ]#{3} )

<scale()> =
  scale( <number> , <number>? )

<scaleX()> =
  scaleX( <number> )

<scaleY()> =
  scaleY( <number> )

<scaleZ()> =
  scaleZ( [ <number> | <percentage> ] )

<translate3d()> =
  translate3d( <length-percentage> , <length-percentage> , <length> )

<translate()> =
  translate( <length-percentage> , <length-percentage>? )

<translateX()> =
  translateX( <length-percentage> )

<translateY()> =
  translateY( <length-percentage> )

<translateZ()> =
  translateZ( <length> )

<rotate3d()> =
  rotate3d( <number> , <number> , <number> , [ <angle> | <zero> ] )

<rotate()> =
  rotate( [ <angle> | <zero> ] )

<rotateX()> =
  rotateX( [ <angle> | <zero> ] )

<rotateY()> =
  rotateY( [ <angle> | <zero> ] )

<rotateZ()> =
  rotateZ( [ <angle> | <zero> ] )

<skew()> =
  skew( [ <angle> | <zero> ] , [ <angle> | <zero> ]? )

<skewX()> =
  skewX( [ <angle> | <zero> ] )

<skewY()> =
  skewY( [ <angle> | <zero> ] )

<matrix3d()> =
  matrix3d( <number>#{16} )

<matrix()> =
  matrix( <number>#{6} )

<perspective()> =
  perspective( [ <length [0,∞]> | none ] )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1, CSS Transforms Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Translating and rotating an element

HTML

```
<div>Transformed element</div>
```

CSS

```
div {
  border: solid red;
  transform: translate(30px, 20px) rotate(20deg);
```

```
  width: 140px;
  height: 60px;
}
```

Result

Comparing the order of transform functions

The order of transform functions matters.

In this example, two boxes are rotated and translated by the same values, but the functions are in the opposite order. The dotted lines mark the X-axis before and after rotation.

HTML

```
<div class="original"></div>
<div class="one">1</div>
<div class="two">2</div>
```

CSS

```
div {
  height: 200px;
  width: 200px;
  position: absolute;
  left: 200px;
  top: 50px;
  font-size: 4rem;
  line-height: 200px;
  text-align: center;
}
.original {
  border: 1px dashed;
}
.original::before,
.original::after {
  content: "";
  position: absolute;
  top: 100px;
  width: 500px;
  left: -150px;
  height: 1px;
  border-top: 2px dotted;
}
.original::after {
  transform: rotate(135deg);
}
.one {
  background-color: #cccccc;
}
.two {
  background-color: #d6bb72;
}
```

```
.one {
  transform: translateX(200px) rotate(135deg);
}
.two {
  transform: rotate(135deg) translateX(200px);
}
```

Result

- Box 1 (first `translateX()`, then `rotate()`): The coordinate space first shifts `200px` along the X axis, then rotates `135deg` within that shifted space, so the element ends up to the right of its original position, rotated.
- Box 2 (first `rotate()`, then `translateX()`): The coordinate space first rotates `135deg`, so the element then moves `200px` along the rotated axis, in the direction shown by the dotted lines. Please see Using CSS transforms and `<transform-function>` for more examples.

See also

- Using CSS transforms
- `<transform-function>` data type with all the transform functions explained.
- Individual CSS properties: `translate`, `rotate`, and `scale` (there is no `skew` property).
- SVG `transform` attribute
- Online tool to visualize CSS Transform functions: CSS Transform Playground

transform-box CSS property

The **`transform-box`** CSS property defines the layout box to which the `transform`, individual transform properties `translate`, `scale`, and `rotate`, and `transform-origin` properties relate.

Syntax

```
/* Keyword values */
transform-box: content-box;
transform-box: border-box;
transform-box: fill-box;
transform-box: stroke-box;
transform-box: view-box;

/* Global values */
transform-box: inherit;
transform-box: initial;
transform-box: revert;
transform-box: revert-layer;
transform-box: unset;
```

The `transform-box` property is specified as one of the keyword values listed below.

Values

`content-box`

The content box is used as the reference box. The reference box of a `<table>` is the border box of its table wrapper box, not its table box.

`border-box`

The border box is used as the reference box. The reference box of a `<table>` is the border box of its table wrapper box, not its table box.

`fill-box`

The object bounding box is used as the reference box. For elements with associated CSS layout box, acts as `content-box`.

`stroke-box`

The stroke bounding box is used as the reference box. For elements with associated CSS layout box, acts as `border-box`.

`view-box`

The nearest SVG viewport is used as the reference box. If a `viewBox` attribute is specified for the SVG viewport creating element, the reference box is positioned at the origin of the coordinate system established by the `viewBox` attribute, and the dimension of the reference box is set to the width and height values of the `viewBox` attribute. For elements with associated CSS layout box, acts as `border-box`.

Formal definition

Initial value	`view-box`
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
transform-box =
  content-box  |
  border-box   |
  fill-box     |
  stroke-box   |
  view-box
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

SVG transform-origin scoping

In this example we have an SVG:

```
<svg id="svg" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 50 50">
  <g>
    <circle id="center" fill="red" r="1" transform="translate(25 25)" />
    <circle id="boxcenter" fill="blue" r=".5" transform="translate(15 15)" />
    <rect
      id="box"
      x="10"
      y="10"
      width="10"
      height="10"
      rx="1"
      ry="1"
      stroke="black"
      fill="none" />
  </g>
</svg>
```

In the CSS we have an animation that uses a transform to rotate the rectangle infinitely. `transform-box: fill-box` is used to make the `transform-origin` the center of the bounding box, so the rectangle spins in place. Without it, the transform origin is the center of the SVG canvas, and so you get a very different effect.

```
svg {
  width: 80vh;
  border: 1px solid #d9d9d9;
  position: absolute;
  margin: auto;
  inset: 0;
}

#box {
  transform-origin: 50% 50%; /* anything other than `0 0` to see the effect */
  transform-box: fill-box;
  animation: rotate-box 3s linear infinite;
}

@keyframes rotate-box {
  to {
    transform: rotate(360deg);
  }
}
```

See also

- Using CSS transforms
- `transform`, `transform-origin`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`

transform-origin CSS property

The **`transform-origin`** CSS property sets the origin for an element's transformations.

Try it

```
transform-origin: center;
```

```
transform-origin: top left;
```

```
transform-origin: 50px 50px;
```

```
/* 3D rotation with z-axis origin */
transform-origin: bottom right 60px;
```

```
<section id="default-example">
  <div id="example-container">
    <div id="example-element">Rotate me!</div>
    <img
      alt=""
      id="crosshair"
      src="/shared-assets/images/examples/crosshair.svg"
      width="24px" />
    <div id="static-element"></div>
  </div>
</section>
```

```
@keyframes rotate {
  from {
    transform: rotate(0);
  }

  to {
    transform: rotate(30deg);
  }
}

@keyframes rotate3d {
  from {
    transform: rotate3d(0, 0, 0, 0);
  }

  to {
    transform: rotate3d(1, 2, 0, 60deg);
  }
}

#example-container {
  width: 160px;
  height: 160px;
  position: relative;
}

#example-element {
  width: 100%;
  height: 100%;
  display: flex;
  position: absolute;
  align-items: center;
  justify-content: center;
  background: #f7ebee;
  color: black;
  font-size: 1.2rem;
  text-transform: uppercase;
}

#example-element.rotate {
  animation: rotate 1s forwards;
}

#example-element.rotate3d {
  animation: rotate3d 1s forwards;
}

#crosshair {
  width: 24px;
  height: 24px;
  opacity: 0;
  position: absolute;
}

#static-element {
  width: 100%;
  height: 100%;
  position: absolute;
  border: dotted 3px #ff1100;
}
```

```
const crosshair = document.getElementById("crosshair");
const el = document.getElementById("example-element");

function update() {
  const selected = document.querySelector(".selected");

  /* Restart the animation
          https://developer.mozilla.org/en-US/docs/Web/API/Web_Animations_API/Tips */
  el.className = "";
  window.requestAnimationFrame(() => {
    window.requestAnimationFrame(() => {
      el.className =
```

```
        el.style.transformOrigin.split(" ")[2] === "60px"
          ? "rotate3d"
          : "rotate";
    });
  });

  const transformOrigin = getComputedStyle(el).transformOrigin;
  const pos = transformOrigin.split(/\s+/);
  crosshair.style.left = `calc(${pos[0]} - 12px)`;
  crosshair.style.top = `calc(${pos[1]} - 12px)`;
}

const observer = new MutationObserver(() => {
  update();
});

observer.observe(el, {
  attributes: true,
  attributeFilter: ["style"],
});

update();
crosshair.style.opacity = "1";
```

The transform origin is the point around which a transformation is applied. For example, the transform origin of the `rotate()` function is the center of rotation.

In effect, this property wraps a pair of translations around the element's other transformations. The first translation moves the transform origin to the true origin at $(0, 0)$. Then the other transformations are applied, and because the transform origin is at $(0, 0)$, those transformations act about the transform origin. Finally, the opposite translation is applied, moving the transform origin back to its original location. Consequently, this definition

```
transform-origin: -100% 50%;
transform: rotate(45deg);
```

results in the same transformation as

```
transform-origin: 0 0;
transform: translate(-100%, 50%) rotate(45deg) translate(100%, -50%);
```

Reading from right to left, `translate(100%, -50%)` is the translation to bring the transform origin to the true origin, `rotate(45deg)` is the original transformation, and `translate(-100%, 50%)` is the translation to restore the transform origin to its original location.

By default, the origin of a transform is `center`.

Syntax

```
/* One-value syntax */
transform-origin: 2px;
transform-origin: bottom;

/* x-offset | y-offset */
transform-origin: 3cm 2px;

/* x-offset-keyword | y-offset */
transform-origin: left 2px;

/* x-offset-keyword | y-offset-keyword */
transform-origin: right top;

/* y-offset-keyword | x-offset-keyword */
transform-origin: top right;

/* x-offset | y-offset | z-offset */
transform-origin: 2px 30% 10px;

/* x-offset-keyword | y-offset | z-offset */
transform-origin: left 5px -3px;

/* x-offset-keyword | y-offset-keyword | z-offset */
transform-origin: right bottom 2cm;

/* y-offset-keyword | x-offset-keyword | z-offset */
transform-origin: bottom right 2cm;

/* Global values */
transform-origin: inherit;
transform-origin: initial;
transform-origin: revert;
transform-origin: revert-layer;
transform-origin: unset;
```

The `transform-origin` property may be specified using one, two, or three values, where each value represents an offset. Offsets that are not explicitly defined are reset to their corresponding initial values.

If a single `<length>` or `<percentage>` value is defined, it represents the horizontal offset.

If two or more values are defined and either no value is a keyword, or the only used keyword is `center`, then the first value represents the horizontal offset and the second represents the vertical offset.

- One-value syntax:
  - The value must be a `<length>`, a `<percentage>`, or one of the keywords `left`, `center`, `right`, `top`, and `bottom`.
- Two-value syntax:
  - One value must be a `<length>`, a `<percentage>`, or one of the keywords `left`, `center`, and `right`.
  - The other value must be a `<length>`, a `<percentage>`, or one of the keywords `top`, `center`, and `bottom`.
- Three-value syntax:
  - The first two values are the same as for the two-value syntax.
  - The third value must be a `<length>`. It always represents the Z offset.

Values

x-offset

Is a `<length>` or a `<percentage>` describing how far from the left edge of the box the origin of the transform is set.

offset-keyword

Is one of the `left`, `right`, `top`, `bottom`, or `center` keyword describing the corresponding offset.

y-offset

Is a `<length>` or a `<percentage>` describing how far from the top edge of the box the origin of the transform is set.

x-offset-keyword

Is one of the `left`, `right`, or `center` keyword describing how far from the left edge of the box the origin of the transform is set.

y-offset-keyword

Is one of the `top`, `bottom`, or `center` keyword describing how far from the top edge of the box the origin of the transform is set.

z-offset

Is a `<length>` (and never a `<percentage>` which would make the statement invalid) describing how far from the user eye the z=0 origin is set.

The keywords are convenience shorthands and match the following `<percentage>` values:

Keyword	Value
`left`	`0%`
`center`	`50%`
`right`	`100%`
`top`	`0%`
`bottom`	`100%`

Formal definition

Initial value	`50% 50% 0`
Applies to	transformable elements
Inherited	no
Percentages	refer to the size of bounding box
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	simple list of length, percentage, or calc

> **Note:** The initial value of `transform-origin` is `0 0` for all SVG elements except for root `<svg>` elements and `<svg>` elements that are a direct child of a foreignObject, and whose `transform-origin` is `50% 50%`, like other CSS elements. See the SVG transform-origin attribute for more information.

Formal syntax

```
transform-origin =
  [ left | center | right | top | bottom | <length-percentage> ]  |
  [ left | center | right | <length-percentage> ] [ top | center | bottom | <length-percentage> ] <length>?  |
  [ [ center | left | right ] && [ center | top | bottom ] ] <length>?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

A demonstration of various transform values

This example shows the effect of choosing different `transform-origin` values for a variety of transformation functions.

```
<div class="container">
  <div class="example">
    <div class="box box1">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: none;
  </pre>

  <div class="example">
    <div class="box box2">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: rotate(30deg);
  </pre>

  <div class="example">
    <div class="box box3">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: rotate(30deg);
```

```
transform-origin: 0 0;
  </pre>

  <div class="example">
    <div class="box box4">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: rotate(30deg);
transform-origin: 100% 100%;
  </pre>

  <div class="example">
    <div class="box box5">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: rotate(30deg);
transform-origin: -1em -3em;
  </pre>

  <div class="example">
    <div class="box box6">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: scale(1.7);
  </pre>

  <div class="example">
    <div class="box box7">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: scale(1.7);
transform-origin: 0 0;
  </pre>

  <div class="example">
    <div class="box box8">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: scale(1.7);
transform-origin: 100% -30%;
  </pre>

  <div class="example">
    <div class="box box9">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: skewX(50deg);
transform-origin: 100% -30%;
  </pre>

  <div class="example">
    <div class="box box10">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: skewY(50deg);
transform-origin: 100% -30%;
  </pre>
</div>
```

```
.container {
  display: grid;
  grid-template-columns: 200px 100px;
  gap: 20px;
}

.example {
  position: relative;
  margin: 0 2em 4em 5em;
}

.box {
  display: inline-block;
  width: 3em;
  height: 3em;
  border: solid 1px;
  background-color: palegreen;
}

.original {
  position: absolute;
  left: 0;
  opacity: 20%;
}

.box1 {
  transform: none;
}

.box2 {
```

```
  transform: rotate(30deg);
}

.box3 {
  transform: rotate(30deg);
  transform-origin: 0 0;
}

.box4 {
  transform: rotate(30deg);
  transform-origin: 100% 100%;
}

.box5 {
  transform: rotate(30deg);
  transform-origin: -1em -3em;
}

.box6 {
  transform: scale(1.7);
}

.box7 {
  transform: scale(1.7);
  transform-origin: 0 0;
}

.box8 {
  transform: scale(1.7);
  transform-origin: 100% -30%;
}

.box9 {
  transform: skewX(50deg);
  transform-origin: 100% -30%;
}

.box10 {
  transform: skewY(50deg);
  transform-origin: 100% -30%;
}
```

See also

- Using CSS transforms
- SVG `transform-origin` attribute
- https://css-tricks.com/almanac/properties/t/transform-origin/

transform-style CSS property

The **`transform-style`** CSS property sets whether children of an element are positioned in the 3D space or are flattened in the plane of the element.

Try it

```
transform-style: flat;
```

```
transform-style: preserve-3d;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all layer" id="example-element">
    <p>Parent</p>
    <div class="numeral"><code>rotate3d(1, 1, 1, 45deg)</code></div>
  </div>
</section>
```

```
.layer {
  background: #623e3f;
  border-radius: 0.75rem;
  color: white;
  transform: perspective(200px) rotateY(30deg);
}

.numeral {
  background-color: #ffba08;
  border-radius: 0.2rem;
  color: black;
  margin: 1rem;
  padding: 0.2rem;
  transform: rotate3d(1, 1, 1, 45deg);
}
```

If flattened, the element's children will not exist on their own in the 3D-space.

As this property is not inherited, it must be set for all non-leaf descendants of the element.

Syntax

```
/* Keyword values */
transform-style: flat;
transform-style: preserve-3d;

/* Global values */
transform-style: inherit;
transform-style: initial;
transform-style: revert;
transform-style: revert-layer;
transform-style: unset;
```

Values

This property is specified as one of the following keyword values:

`flat`

Indicates that the children of the element are lying in the plane of the element itself.

`preserve-3d`

Indicates that the children of the element should be positioned in the 3D-space.

Description

The spec lists some grouping property values, which require the user agent to create a flattened representation of the descendant elements before they can be applied, and therefore force the element to have a used value of `transform-style: flat`, even when `preserve-3d` is specified. These property values include:

- `overflow`: any value other than `visible` or `clip`.
- `opacity`: any value less than `1`.
- `filter`: any value other than `none`.
- `clip`: any value other than `auto`.
- `clip-path`: any value other than `none`.
- `isolation`: used value of `isolate`.
- `mask-image`: any value other than `none`.
- `mask-border-source`: any value other than `none`.
- `mix-blend-mode`: any value other than `normal`.
- `contain`: `paint` and any other property/value combination that causes paint containment. This includes any property that affect the used value of the `contain` property, such as `content-visibility: hidden`.

Formal definition

Initial value	`flat`
Applies to	transformable elements
Inherited	no
Computed value	as specified
Animation type	discrete
Creates stacking context	yes

Formal syntax

```
transform-style =
  flat         |
  preserve-3d
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Transform style demonstration

In this example we have a 3D cube created using transforms. The parent container of the cube faces has `transform-style: preserve-3d` set on it by default, so it is transformed in the 3D space and you can see it as intended.

We also provide a checkbox allowing you to toggle between this, and `transform-style: flat`. In this alternative state, the cube faces are all flattened onto the plane of their parent, and you might not be able to see them at all, depending on the browser you are using.

HTML

```
<section id="example-element">
  <div class="face front">1</div>
  <div class="face back">2</div>
  <div class="face right">3</div>
  <div class="face left">4</div>
  <div class="face top">5</div>
  <div class="face bottom">6</div>
</section>

<div class="checkbox">
  <label for="preserve"><code>preserve-3d</code></label>
  <input type="checkbox" id="preserve" checked />
</div>
```

CSS

```
#example-element {
  margin: 50px;
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  transform: rotate3d(1, 1, 1, 30deg);
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 70%);
  transform: translateZ(50px);
```

```
}

.back {
  background: rgb(0 210 0 / 70%);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 70%);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 70%);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 70%);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 70%);
  transform: rotateX(-90deg) translateZ(50px);
}
```

JavaScript

```
const cube = document.getElementById("example-element");
const checkbox = document.getElementById("preserve");

checkbox.addEventListener("change", () => {
  cube.style.transformStyle = checkbox.checked ? "preserve-3d" : "flat";
});
```

Result

See also

- Using CSS transforms

transition CSS property

The **`transition`** CSS shorthand property for `transition-property`, `transition-duration`, `transition-timing-function`, `transition-delay`, and `transition-behavior`.

Transitions enable you to define the transition between two states of an element. Different states may be defined using pseudo-classes like `:hover` or `:active` or dynamically set using JavaScript.

Try it

```
transition: margin-right 2s;
```

```
transition: margin-right 2s 0.5s;
```

```
transition: margin-right 2s ease-in-out;
```

```
transition: margin-right 2s ease-in-out 0.5s;
```

```
transition:
  margin-right 2s,
  color 1s;
```

```
transition: all 1s ease-out;
```

```
<section id="default-example">
  <div id="example-element">Hover to see<br />the transition.</div>
</section>
```

```
#example-element {
  background-color: #e4f0f5;
  color: black;
  padding: 1rem;
  border-radius: 0.5rem;
  font: 1em monospace;
  width: 100%;
  transition: margin-right 2s;
}

#default-example:hover > #example-element {
  background-color: #990099;
  color: white;
  margin-right: 40%;
}
```

Constituent properties

This property is a shorthand for the following CSS properties:

- `transition-behavior`
- `transition-delay`
- `transition-duration`
- `transition-property`
- `transition-timing-function`

Syntax

```
/* Apply to 1 property */
/* property name | duration */
```

```
transition: margin-right 4s;

/* property name | duration | delay */
transition: margin-right 4s 1s;

/* property name | duration | easing function */
transition: margin-right 4s ease-in-out;

/* property name | duration | easing function | delay */
transition: margin-right 4s ease-in-out 1s;

/* property name | duration | behavior */
transition: display 4s allow-discrete;

/* Apply to 2 properties */
transition:
  margin-right 4s,
  color 1s;

/* Apply to all changed properties */
transition: all 0.5s ease-out allow-discrete;
transition: 200ms linear 50ms;

/* Global values */
transition: inherit;
transition: initial;
transition: revert;
transition: revert-layer;
transition: unset;
```

The `transition` property value is specified as one of the following:

- The special value `none`, which specifies that no transitions will occur on this element. This is the default value.
- One or more single-property transitions, separated by commas.

Each single-property transition describes the transition that should be applied to a single property or all properties. It includes:

- zero or one value representing the property or properties to which the transition should apply. This can be set as:
  - A `<custom-ident>` representing a single property.
  - The special value `all`, which specifies that the transition will be applied to all properties that change as the element changes state.
  - No value, in which case a value of `all` will be inferred and the specified transition will still apply to all changing properties.
- zero or one `<easing-function>` value representing the easing function to use
- zero, one, or two `<time>` values. The first value that can be parsed as a time is assigned to the `transition-duration`, and the second value that can be parsed as a time is assigned to `transition-delay`.
- zero or one value declaring whether to start transitions for properties whose animation behavior is discrete. The value, if present, is either the keyword `allow-discrete` or the keyword `normal`.

If you specify `all` as the transition property for one single-property transition, but then specify subsequent single-property transitions with `<custom-ident>` values, those subsequent transitions will override the first one. For example:

```
transition:
  all 200ms,
  opacity 400ms;
```

In this case, all the properties that change as the element changes state will transition with a duration of 200ms except for `opacity`, which will take 400ms to transition.

See how things are handled when lists of property values aren't the same length. In short, extra transition descriptions beyond the number of properties actually being animated are ignored.

Formal definition

Initial value	as each of the properties of the shorthand: • `transition-delay`: `0s` • `transition-duration`: `0s` • `transition-property`: `all` • `transition-timing-function`: `ease` • `transition-behavior`: `normal`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as each of the properties of the shorthand: • `transition-delay`: as specified • `transition-duration`: as specified • `transition-property`: as specified • `transition-timing-function`: as specified • `transition-behavior`: as specified
Animation type	Not animatable

Formal syntax

```
transition =
  <single-transition>#

<single-transition> =
  [ none | <single-transition-property> ]  ||
  <time>                                   ||
  <easing-function>                        ||
  <time>                                   ||
  <transition-behavior-value>

<single-transition-property> =
  all             |
  <custom-ident>

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
```

```
  <step-easing-function>

<transition-behavior-value> =
  normal          |
  allow-discrete

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

This syntax reflects the latest standard as per CSS Easing Functions Level 2, CSS Transitions Module Level 1, CSS Transitions Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

In this example, when the user hovers over the element, there is a half-second (`500ms`) delay before a two-second `background-color` transition occurs.

HTML

```
<a class="target">Hover over me</a>
```

CSS

We include two `<time>` values. In the `transition` shorthand, the first `<time>` value is the `transition-duration`. The second time value is the `transition-delay`. Both default to `0s` if omitted.

```
.target {
  font-size: 2rem;
  background-color: palegoldenrod;
  transition: background-color 2s 500ms;
}

.target:hover {
  background-color: darkorange;
}
```

See also

- CSS transitions module
- Using CSS transitions
- `TransitionEvent`

transition-behavior CSS property

The **`transition-behavior`** CSS property specifies whether transitions will be started for properties whose animation behavior is discrete.

Syntax

```
/* Keyword values */
transition-behavior: allow-discrete;
transition-behavior: normal;

/* Global values */
transition-behavior: inherit;
transition-behavior: initial;
transition-behavior: revert;
transition-behavior: revert-layer;
transition-behavior: unset;
```

Values

`allow-discrete`

Transitions will be started on the element for discrete animated properties.

`normal`

Transitions will *not* be started on the element for discrete animated properties.

Description

The `transition-behavior` property is only relevant when used in conjunction with other transition properties, notably `transition-property` and `transition-duration`, as no transition occurs if no properties are animated over a non-zero duration of time.

```
.card {
  transition-property: opacity, display;
  transition-duration: 0.25s;
  transition-behavior: allow-discrete;
}

.card.fade-out {
  opacity: 0;
  display: none;
}
```

The `transition-behavior` value can be included as part of a shorthand `transition` declaration. When included in the shorthand, when using or defaulting to all properties, the `allow-discrete` value has no impact on regular animatable properties. The following CSS is equivalent to the longhand declarations above:

```
.card {
  transition: all 0.25s;
  transition: all 0.25s allow-discrete;
}

.card.fade-out {
  opacity: 0;
  display: none;
}
```

In the above snippet we include the `transition` property twice. The first instance does not include the `allow-discrete` value — this provides cross-browser support, ensuring the card's other properties still transition in browsers that don't support `transition-behavior`.

Discrete animation behavior

Discrete-animated properties generally flip between two values 50% through animating between the two.

There is an exception, however, which is when animating to or from `display: none` or `content-visibility: hidden`. In this case, the browser will flip between the two values so that the transitioned content is shown for the entire animation duration.

So for example:

- When animating `display` from `none` to `block` (or another visible `display` value), the value will flip to `block` at `0%` of the animation duration so it is visible throughout.
- When animating `display` from `block` (or another visible `display` value) to `none`, the value will flip to `none` at `100%` of the animation duration so it is visible throughout.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
transition-behavior =
  <transition-behavior-value>#

<transition-behavior-value> =
  normal          |
  allow-discrete
```

This syntax reflects the latest standard as per CSS Transitions Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Transitioning a popover

In this example, a popover is animated as it transitions from hidden to shown and back again.

HTML

The HTML contains a `<div>` element declared as a popover using the popover attribute, and a `<button>` element designated as the popover's display control using its popovertarget attribute.

```
<button popovertarget="mypopover">Show the popover</button>
<div popover="auto" id="mypopover">I'm a Popover! I should animate.</div>
```

CSS

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

[popover] {
  font-size: 1.2rem;
  padding: 10px;
}
```

```
[popover]:popover-open {
  opacity: 1;
  transform: scaleX(1);
}

[popover] {
  /* Final state of the exit animation */
  opacity: 0;
  transform: scaleX(0);
```

```
  transition-property: opacity, transform, overlay, display;
  transition-duration: 0.7s;
  transition-behavior: allow-discrete;
  /* Using the shorthand transition property, we could write:
    transition:
      opacity 0.7s,
      transform 0.7s,
      overlay 0.7s allow-discrete,
      display 0.7s allow-discrete;

    or even:
    transition: all 0.7s allow-discrete;
  */
}

/* Needs to be included after the previous [popover]:popover-open
   rule to take effect, as the specificity is the same */
@starting-style {
  [popover]:popover-open {
    opacity: 0;
    transform: scaleX(0);
  }
}
```

The two properties we want to animate are `opacity` and `transform`: we want the popover to fade in and out while growing and shrinking in the horizontal direction. We set a starting state for these properties on the default hidden state of the popover element (selected via `[popover]`), and an end state on the open state of the popover (selected via the `:popover-open` pseudo-class). We then set a `transition` property to animate between the two.

Because the animated element is being promoted to the top layer when shown and removed from the top layer when hidden — which also means that its hidden state has `display: none` set on it — the following properties are added to the list of transitioned elements to get the animation working in both directions. In both cases, `transition-behavior: allow-discrete` is set in the shorthand to enable discrete animation.

- `display`: Required so that the animated element is visible (set to `display: block`) throughout both the entry and exit animation. Without this, the exit animation would not be visible; in effect, the popover would just disappear.
- `overlay`: Required to make sure that the removal of the element from the top layer is deferred until the animation has been completed. This doesn't make a huge difference for basic animations such as this one, but in more complex cases not doing this can result in the element being removed from the overlay too quickly, meaning the animation is not smooth or effective.

In addition, a starting state for the animation is set inside the `@starting-style` at-rule. This is needed to avoid unexpected behavior. By default transitions are not triggered on elements' first style updates, or when the `display` type changes from `none` to another type. `@starting-style` allows you to override that default in a specific controlled fashion. Without this, the entry animation would not occur and the popover would just appear.

Result

The code renders as follows:

> **Note:** Because popovers change from `display: none` to `display: block` each time they are shown, the popover transitions from its `@starting-style` styles to its `[popover]:popover-open` styles every time the entry transition occurs. When the popover closes, it transitions from its `[popover]:popover-open` state to the default `[popover]` state.
>
> It is possible for the style transition on entry and exit to be different in such cases. See our Demonstration of when starting styles are used example for a proof of this.

See also

- `overlay`
- `@starting-style`
- CSS transitions module
- Four new CSS features for smooth entry and exit animations on developer.chrome.com (2023)

transition-delay CSS property

The **`transition-delay`** CSS property specifies the duration to wait before starting a property's transition effect when its value changes.

Try it

```
transition-delay: 250ms;
transition-property: margin-right;
```

```
transition-delay: 1s;
transition-property: background-color;
```

```
transition-delay: 1s;
transition-property: margin-right, color;
```

```
transition-delay: 1s, 250ms;
transition-property: margin-right, color;
```

```
<section id="default-example">
  <div id="example-element">Hover to see<br />the transition.</div>
</section>
```

```
#example-element {
  background-color: #e4f0f5;
  color: black;
  padding: 1rem;
  border-radius: 0.5rem;
  font: 1em monospace;
  width: 100%;
  transition: margin-right 2s;
}

#default-example:hover > #example-element {
  background-color: #990099;
  color: white;
  margin-right: 40%;
}
```

The delay may be zero, positive, or negative:

- A value of `0s` (or `0ms`) will begin the transition effect immediately.
- A positive value will delay the start of the transition effect for the given length of time.
- A negative value will begin the transition effect immediately, and partway through the effect. In other words, the effect will be animated as if it had already been running for the given length of time.

You may specify multiple delays, which is useful when transitioning multiple properties. Each delay will be applied to the corresponding property as specified by the `transition-property` property, which acts as a master list. If there are fewer delays specified than in the master list, the list of delay values will be repeated until there are enough. If there are more delays, the list of delay values will be truncated to match the number of properties. In both cases, the CSS declaration remains valid.

Syntax

```
/* <time> values */
transition-delay: 3s;
transition-delay: 2s, 4ms;

/* Global values */
transition-delay: inherit;
transition-delay: initial;
transition-delay: revert;
transition-delay: revert-layer;
transition-delay: unset;
```

Values

`<time>`

Denotes the amount of time to wait between a property's value changing and the start of the transition effect.

Formal definition

Initial value	`0s`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
transition-delay =
  <time>#
```

This syntax reflects the latest standard as per CSS Transitions Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Example showing different delays

HTML

```
<div class="box delay-1">0.5 seconds</div>

<div class="box delay-2">2 seconds</div>

<div class="box delay-3">4 seconds</div>

<button id="change">Change</button>
```

CSS

```
.box {
  margin: 20px;
  padding: 10px;
  display: inline-block;
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 18px;
  transition-property: background-color, font-size, transform, color;
  transition-timing-function: ease-in-out;
  transition-duration: 3s;
}

.transformed-state {
  transform: rotate(270deg);
  background-color: blue;
  color: yellow;
  font-size: 12px;
  transition-property: background-color, font-size, transform, color;
  transition-timing-function: ease-in-out;
  transition-duration: 3s;
}

.delay-1 {
  transition-delay: 0.5s;
}

.delay-2 {
  transition-delay: 2s;
}

.delay-3 {
  transition-delay: 4s;
}
```

JavaScript

```
function change() {
  const elements = document.querySelectorAll("div.box");
  for (const element of elements) {
    element.classList.toggle("transformed-state");
  }
}

const changeButton = document.querySelector("#change");
changeButton.addEventListener("click", change);
```

Result

See also

- Using CSS transitions
- `TransitionEvent` API

transition-duration CSS property

The **`transition-duration`** CSS property sets the length of time a transition animation should take to complete. By default, the value is `0s`, meaning that no animation will occur.

You may specify multiple durations; each duration will be applied to the corresponding property as specified by the `transition-property` property, which acts as a master list. If the number of specified durations is less than in the master list, the user agent repeats the list of durations. If the number of specified durations is more than in the master list, the list is truncated to the right size. In both the cases, the CSS declaration stays valid.

Try it

```
transition-duration: 500ms;
transition-property: margin-right;
```

```
transition-duration: 2s;
transition-property: background-color;
```

```
transition-duration: 2s;
transition-property: margin-right, color;
```

```
transition-duration: 3s, 1s;
transition-property: margin-right, color;
```

```
<section id="default-example">
  <div id="example-element">Hover to see<br />the transition.</div>
</section>
```

```
#example-element {
  background-color: #e4f0f5;
  color: black;
  padding: 1rem;
  border-radius: 0.5rem;
  font: 1em monospace;
  width: 100%;
  transition: margin-right 2s;
}

#default-example:hover > #example-element {
  background-color: #990099;
  color: white;
  margin-right: 40%;
}
```

Syntax

```
/* <time> values */
transition-duration: 6s;
transition-duration: 120ms;
transition-duration: 1s, 15s;
transition-duration: 10s, 30s, 230ms;

/* Global values */
transition-duration: inherit;
transition-duration: initial;
transition-duration: revert;
transition-duration: revert-layer;
transition-duration: unset;
```

Values

`<time>`

Is a `<time>` denoting the amount of time the transition from the old value of a property to the new value should take. A time of `0s` indicates that no transition will happen, that is the switch between the two states will be instantaneous. A negative value for the time renders the declaration invalid.

Formal definition

Initial value	`0s`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
transition-duration =
  <time [0s,∞]>#
```

This syntax reflects the latest standard as per CSS Transitions Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Example showing different durations

HTML

```
<div class="box duration-1">0.5 seconds</div>

<div class="box duration-2">2 seconds</div>

<div class="box duration-3">4 seconds</div>

<button id="change">Change</button>
```

CSS

```
.box {
  margin: 20px;
  padding: 10px;
  display: inline-block;
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 18px;
  transition-property: background-color, font-size, transform, color;
  transition-timing-function: ease-in-out;
}

.transformed-state {
  transform: rotate(270deg);
  background-color: blue;
  color: yellow;
  font-size: 12px;
  transition-property: background-color, font-size, transform, color;
  transition-timing-function: ease-in-out;
}

.duration-1 {
  transition-duration: 0.5s;
}

.duration-2 {
  transition-duration: 2s;
}

.duration-3 {
  transition-duration: 4s;
}
```

JavaScript

```
function change() {
  const elements = document.querySelectorAll("div.box");
  for (const element of elements) {
    element.classList.toggle("transformed-state");
  }
}

const changeButton = document.querySelector("#change");
changeButton.addEventListener("click", change);
```

Result

See also

- Using CSS transitions
- `transition`
- `transition-property`
- `transition-timing-function`
- `transition-delay`
- `TransitionEvent`

transition-property CSS property

The **`transition-property`** CSS property sets the CSS properties to which a transition effect should be applied.

If you specify a shorthand property (e.g., `background`), all of its longhand sub-properties that can be animated will be.

Try it

```
transition-property: margin-right;
```

```
transition-property: margin-right, color;
```

```
transition-property: all;
```

```
transition-property: none;
```

```
<section id="default-example">
  <div id="example-element">Hover to see<br />the transition.</div>
</section>
```

```
#example-element {
  background-color: #e4f0f5;
  color: black;
  padding: 1rem;
```

```
  border-radius: 0.5rem;
  font: 1em monospace;
  width: 100%;
  transition: margin-right 2s;
}

#default-example:hover > #example-element {
  background-color: #990099;
  color: white;
  margin-right: 40%;
}
```

Syntax

```
/* Keyword values */
transition-property: none;
transition-property: all;

/* <custom-ident> values */
transition-property: test_05;
transition-property: -specific;
transition-property: sliding-vertically;

/* Multiple values */
transition-property: test1, animation4;
transition-property: all, height, color;
transition-property:
  all,
  -moz-specific,
  sliding;

/* Global values */
transition-property: inherit;
transition-property: initial;
transition-property: revert;
transition-property: revert-layer;
transition-property: unset;
```

Values

`none`

No properties will transition.

`all`

All properties that can transition will.

`<custom-ident>`

A string identifying the property to which a transition effect should be applied when its value changes.

Formal definition

Initial value	`all`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
transition-property =
  none                        |
  <single-transition-property>#

<single-transition-property> =
  all             |
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Transitions Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

When the button is hovered or focused, it undergoes a one-second color transition; the `transition-property` is `background-color`.

HTML

```
<button class="target">Focus me!</button>
```

CSS

```
html {
  height: 100vh;
}

button {
  font-size: 1.4rem;
  padding: 10px 20px;
  border: 1px solid #cccccc;
  border-radius: 10px;
  outline: none;
}
```

```
.target {
  transition-property: background-color;
  transition-duration: 1s;
```

```
  background-color: #cccccc;
}

.target:hover,
.target:focus {
  background-color: #eeeeee;
}
```

See our Using CSS transitions guide for more `transition-property` examples.

See also

- Using CSS transitions
- `transition`
- `transition-duration`
- `transition-timing-function`
- `transition-delay`
- `TransitionEvent`

transition-timing-function CSS property

The **`transition-timing-function`** CSS property sets how intermediate values are calculated for CSS properties being affected by a transition effect.

Try it

```
transition-timing-function: linear;
```

```
transition-timing-function: ease-in;
```

```
transition-timing-function: steps(6, end);
```

```
transition-timing-function: cubic-bezier(0.29, 1.01, 1, -0.68);
```

```
<section id="default-example">
  <div id="example-element">Hover to see<br />the transition.</div>
</section>
```

```
#example-element {
  background-color: #e4f0f5;
  color: black;
  padding: 1rem;
  border-radius: 0.5rem;
  font: 1em monospace;
  width: 100%;
  transition: margin-right 2s;
}

#default-example:hover > #example-element {
  background-color: #990099;
  color: white;
  margin-right: 40%;
}
```

This, in essence, lets you establish an acceleration curve so that the speed of the transition can vary over its duration.

This acceleration curve is defined using one `<easing-function>` for each property to be transitioned.

You may specify multiple easing functions; each one will be applied to the corresponding property as specified by the `transition-property` property, which acts as a `transition-property` list. If there are fewer easing functions specified than in the `transition-property` list, the user agent must calculate which value is used by repeating the list of values until there is one for each transition property. If there are more easing functions, the list is truncated to the right size. In both cases, the CSS declaration stays valid.

Syntax

```
/* Keyword values */
transition-timing-function: ease;
transition-timing-function: ease-in;
transition-timing-function: ease-out;
transition-timing-function: ease-in-out;
transition-timing-function: linear;
transition-timing-function: step-start;
transition-timing-function: step-end;

/* Function values */
transition-timing-function: steps(4, jump-end);
transition-timing-function: cubic-bezier(0.1, 0.7, 1, 0.1);

/* Steps Function keywords */
transition-timing-function: steps(4, jump-start);
transition-timing-function: steps(10, jump-end);
transition-timing-function: steps(20, jump-none);
transition-timing-function: steps(5, jump-both);
transition-timing-function: steps(6, start);
transition-timing-function: steps(8, end);

/* Multiple easing functions */
transition-timing-function: ease, step-start, cubic-bezier(0.1, 0.7, 1, 0.1);

/* Global values */
transition-timing-function: inherit;
transition-timing-function: initial;
transition-timing-function: revert;
transition-timing-function: revert-layer;
transition-timing-function: unset;
```

Values

`<easing-function>`

Each `<easing-function>` represents the easing function to link to the corresponding property to transition, as defined in `transition-property`.

The non-step keyword values (ease, linear, ease-in-out, etc.) each represent cubic Bézier curve with fixed four point values, with the cubic-bezier() function value allowing for a non-predefined value. The step easing functions divide the input time into a specified number of intervals that are equal in length. It is defined by a number of steps and a step position.

`ease`

Equal to `cubic-bezier(0.25, 0.1, 0.25, 1.0)`, the default value, increases in velocity towards the middle of the transition, slowing back down at the end.

`linear`

Equal to `cubic-bezier(0.0, 0.0, 1.0, 1.0)`, transitions at an even speed.

`ease-in`

Equal to `cubic-bezier(0.42, 0, 1.0, 1.0)`, starts off slowly, with the transition speed increasing until complete.

`ease-out`

Equal to `cubic-bezier(0, 0, 0.58, 1.0)`, starts transitioning quickly, slowing down as the transition continues.

`ease-in-out`

Equal to `cubic-bezier(0.42, 0, 0.58, 1.0)`, starts transitioning slowly, speeds up, and then slows down again.

`cubic-bezier(p1, p2, p3, p4)`

An author-defined cubic-Bezier curve, where the p1 and p3 values must be in the range of 0 to 1.

`steps(n, <jump-term>)`

Displays the transition along *n stops along the transition, displaying each stop for* equal lengths of time. For example, if *n* is 5, there are 5 steps. Whether the transition holds temporarily at 0%, 20%, 40%, 60% and 80%, on the 20%, 40%, 60%, 80% and 100%, or makes 5 stops between the 0% and 100% along the transition, or makes 5 stops including the 0% and 100% marks (on the 0%, 25%, 50%, 75%, and 100%) depends on which of the following jump terms is used:

`jump-start`

Denotes a left-continuous function, so that the first jump happens when the transition begins;

`jump-end`

Denotes a right-continuous function, so that the last jump happens when the animation ends;

`jump-none`

There is no jump on either end. Instead, holding at both the 0% mark and the 100% mark, each for 1/n of the duration

`jump-both`

Includes pauses at both the 0% and 100% marks, effectively adding a step during the transition time.

`start`

Same as `jump-start`.

`end`

Same as `jump-end`.

`step-start`

Equal to `steps(1, jump-start)`

`step-end`

Equal to `steps(1, jump-end)`

Accessibility

Some animations can be helpful such as to guide users to understand what actions are expected, to show relationships within the user interface, and to inform users as to what actions have occurred. Animations can help reduce cognitive load, prevent change blindness, and establish better recall in spatial relationships. However, some animations can be problematic for people with cognitive concerns such as Attention Deficit Hyperactivity Disorder (ADHD) and certain kinds of motion can be a trigger for Vestibular disorders, epilepsy, and migraine and Scotopic sensitivity.

Consider providing a mechanism for pausing or disabling animation, as well as using the Reduced Motion Media Query (or equivalent User Agent client hint `Sec-CH-Prefers-Reduced-Motion`) to create a complementary experience for users who have expressed a preference for less animation.

Formal definition

Initial value	`ease`
Applies to	all elements, `::before` and `::after` pseudo-elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
transition-timing-function =
  <easing-function>#

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
```

```
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

This syntax reflects the latest standard as per CSS Easing Functions Level 2, CSS Transitions Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Cubic-Bezier examples

```
<div class="parent">
  <div class="ease">ease</div>
  <div class="ease-in">ease-in</div>
  <div class="ease-out">ease-out</div>
  <div class="ease-in-out">ease-in-out</div>
  <div class="linear">linear</div>
  <div class="cb">cubic-bezier(0.2,-2,0.8,2)</div>
</div>
```

```
.parent {
}
.parent > div[class] {
  width: 12em;
  min-width: 12em;
  margin-bottom: 4px;
  background-color: black;
  border: 1px solid red;
  color: white;
  transition-property: all;
  transition-duration: 7s;
}
.parent > div.box1 {
  width: 90vw;
  min-width: 24em;
  background-color: magenta;
  color: yellow;
  border: 1px solid orange;
  transition-property: all;
  transition-duration: 2s;
}
```

```
function updateTransition() {
  const els = document.querySelectorAll(".parent > div[class]");
  for (const el of els) {
    el.classList.toggle("box1");
  }
}

const intervalID = setInterval(updateTransition, 10000);
```

```
.ease {
  transition-timing-function: ease;
}
.ease-in {
  transition-timing-function: ease-in;
}
.ease-out {
  transition-timing-function: ease-out;
}
.ease-in-out {
  transition-timing-function: ease-in-out;
}
.linear {
  transition-timing-function: linear;
}
.cb {
  transition-timing-function: cubic-bezier(0.2, -2, 0.8, 2);
}
```

Step examples

```
<div class="parent">
  <div class="jump-start">jump-start</div>
  <div class="jump-end">jump-end</div>
  <div class="jump-both">jump-both</div>
  <div class="jump-none">jump-none</div>
  <div class="step-start">step-start</div>
```

```
  <div class="step-end">step-end</div>
</div>
```

```
.parent {
}
.parent > div[class] {
  width: 12em;
  min-width: 12em;
  margin-bottom: 4px;
  background-color: black;
  border: 1px solid red;
  color: white;
  transition-property: all;
  transition-duration: 7s;
}
.parent > div.box1 {
  width: 90vw;
  min-width: 24em;
  background-color: magenta;
  color: yellow;
  border: 1px solid orange;
  transition-property: all;
  transition-duration: 2s;
}
```

```
function updateTransition() {
  const els = document.querySelectorAll(".parent > div[class]");
  for (const el of els) {
    el.classList.toggle("box1");
  }
}

const intervalID = setInterval(updateTransition, 10000);
```

```
.jump-start {
  transition-timing-function: steps(5, jump-start);
}
.jump-end {
  transition-timing-function: steps(5, jump-end);
}
.jump-none {
  transition-timing-function: steps(5, jump-none);
}
.jump-both {
  transition-timing-function: steps(5, jump-both);
}
.step-start {
  transition-timing-function: step-start;
}
.step-end {
  transition-timing-function: step-end;
}
```

See also

- Using CSS transitions
- `<easing-function>`
- CSS easing functions module
- `transition`
- `transition-property`
- `transition-duration`
- `transition-delay`
- `TransitionEvent`

translate CSS property

The **`translate`** CSS property allows you to specify translation transforms individually and independently of the `transform` property. This maps better to typical user interface usage, and saves having to remember the exact order of transform functions to specify in the `transform` value.

Try it

```
translate: none;
```

```
translate: 40px;
```

```
translate: 50% -40%;
```

```
translate: 20px 4rem;
```

```
translate: 20px 4rem 150px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
```

```
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Syntax

```
/* Keyword value */
translate: none;

/* Single values */
translate: 100px;
translate: 50%;

/* Two values */
translate: 100px 200px;
translate: 50% 105px;

/* Three values */
translate: 50% 105px 5rem;

/* Global values */
translate: inherit;
translate: initial;
translate: revert;
translate: revert-layer;
translate: unset;
```

Values

Single `<length-percentage>` value

A `<length>` or `<percentage>` that specifies a translation along the X-axis. Equivalent to a `translate()` (2D translation) function with a single value specified.

Two `<length-percentage>` values

Two `<length>` or `<percentage>` that specify the X and Y axis translation values (respectively) of a 2D translation. Equivalent to a `translate()` (2D translation) function with two values specified.

Three values

Two `<length-percentage>` and single `<length>` values that specify the X, Y, and Z axis translation values (respectively) of a 3D translation. Equivalent to a `translate3d()` (3D translation) function.

`none`

Specifies that no translation should be applied.

Formal definition

Initial value	none
Applies to	transformable elements
Inherited	no
Percentages	refer to the size of bounding box
Computed value	as specified, but with relative lengths converted into absolute lengths

Animation type	a transform
Creates stacking context	yes

Formal syntax

```
translate =
  none                                          |
  <length-percentage> [ <length-percentage> <length>? ]?

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Translating an element on hover

This example shows how to use the `translate` property to move an element in three axes. The first box is moved along the X axis and the second box is moved along the X and Y axes. The third box is moved along the X, Y and Z axes and has the appearance of moving toward the viewer because of the addition of `perspective` to the parent element.

HTML

```
<div class="wrapper">
  <div id="box1">translate X</div>
  <div id="box2">translate X,Y</div>
  <div id="box3">translate X,Y,Z</div>
</div>
```

CSS

```
.wrapper {
  perspective: 100px;
  display: inline-flex;
  gap: 1em;
}
.wrapper > div {
  width: 7em;
  line-height: 7em;
  text-align: center;
  transition: 0.5s ease-in-out;
  border: 3px dotted;
}
#box1:hover {
  translate: 20px;
}

#box2:hover {
  translate: 20px 20px;
}

#box3:hover {
  translate: 5px 5px 30px;
}
```

Result

See also

- `scale`
- `rotate`
- `transform`

Note: skew is not an independent transform value

trigger-scope CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`trigger-scope`** CSS property can be used to limit the scope of a scroll-triggered animation trigger name to a document subtree.

Syntax

```
/* Keywords */
trigger-scope: none;
trigger-scope: all;

/* <dashed-ident> */
trigger-scope: --my-trigger;

/* Multiple values */
trigger-scope: --my-trigger, --another-trigger;

/* Global values */
trigger-scope: inherit;
trigger-scope: initial;
trigger-scope: revert;
trigger-scope: revert-layer;
trigger-scope: unset;
```

Values

Specified as `none`, `all`, or a comma-separated list of `<dashed-ident>` values:

`none`

Specifies that no trigger scoping is set. This is the default value.

`all`

Sets the scope so that *any* `timeline-trigger-name` values set in the subtree can only be associated with animated elements in the same subtree.

`<dashed-ident>`

A trigger name. Sets the scope so that the specified `timeline-trigger-name` values, when set in the subtree, can only be associated with animated elements in the same subtree.

Description

The `trigger-scope` property is used to limit trigger scope to specific element subtrees in scroll-triggered animations.

Trigger names, defined with the `timeline-trigger-name` property, are global by default. When an animated element is associated with a trigger name via its `animation-trigger` property, the browser determines what its trigger element is as follows:

1. It walks up the animated element's ancestor tree until it finds an ancestor with a `timeline-trigger-name` set that is the same as the name referenced in its `animation-trigger` property value. If the animated element is also the trigger, it will be found instantly.
2. If it can't find a suitable ancestor trigger, it will use the *last* element in the HTML source order with that `timeline-trigger-name` value.
3. If it can't find an element anywhere in the DOM with that `timeline-trigger-name` value, the animated element won't be scroll-triggered; having no associated timeline, the animation will not happen.

If multiple elements define triggers with the same trigger name, only the last one in the document tree will be used as the trigger for animated elements referencing that trigger name in their `animation-trigger` properties. This is likely not the desired behavior.

The `trigger-scope` property can solve this problem by limiting the scope of a trigger name to a subtree of the document. This means the trigger is visible only to elements within the same subtree and has no effect on elements outside the subtree. When `trigger-scope` is set on an element, and that element or its descendants are defined as triggers, animated elements are associated with those triggers only if they are within the same subtree.

Which trigger names are included in the scope depends on the `trigger-scope` value set:

- `trigger-scope: all` means that all trigger names are included in the scope.
- `trigger-scope: --my-trigger, --another-trigger` means that only triggers with names of `--my-trigger` and/or `--another-trigger` are included in the scope.
- `trigger-scope: none` means that no trigger scoping is set on the element.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
trigger-scope =
  none            |
  all             |
  <dashed-ident>#
```

This syntax reflects the latest standard as per Animation Triggers. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates using the `trigger-scope` property to limit the scope of an `animation-trigger-name`.

HTML

We include three `<section>` elements, each containing two `<div>` elements: an `.animated` element and a `.trigger` element.

Most of the HTML, including a checkbox toggle that enables or disables the `trigger-scope` property, has been hidden for brevity.

```
<section id="one">
  <div class="animated"></div>
  ...
  <div class="trigger">Trigger for first animation</div>
  ...
</section>
<section id="two">
  <div class="animated"></div>
  ...
  <div class="trigger">Trigger for second animation</div>
  ...
</section>
<section id="three">
  <div class="animated"></div>
  ...
  <div class="trigger">Trigger for third animation</div>
  ...
</section>
```

```
<section id="one">
  <div class="animated"></div>

  <p>
    Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
    purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
    ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
    euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
    Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
    sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
    facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
    suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
    iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
    luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
    tristique tellus, sed tincidunt velit.
```

```
  </p>

  <div class="trigger">Trigger for first animation</div>

  <p>
    Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
    purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
    ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
    euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
    Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
    sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
    facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
    suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
    iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
    luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
    tristique tellus, sed tincidunt velit.
  </p>
</section>
<section id="two">
  <div class="animated"></div>
  <p>
    Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
    purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
    ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
    euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
    Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
    sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
    facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
    suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
    iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
    luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
    tristique tellus, sed tincidunt velit.
  </p>

  <div class="trigger">Trigger for second animation</div>

  <p>
    Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
    purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
    ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
    euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
    Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
    sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
    facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
    suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
    iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
    luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
    tristique tellus, sed tincidunt velit.
  </p>
</section>
<section id="three">
  <div class="animated"></div>

  <p>
    Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
    purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
    ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
    euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
    Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
    sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
    facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
    suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
    iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
    luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
    tristique tellus, sed tincidunt velit.
  </p>

  <div class="trigger">Trigger for third animation</div>

  <p>
    Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
    purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
    ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
    euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
    Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
    sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
    facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
    suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
    iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
    luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
    tristique tellus, sed tincidunt velit.
  </p>
</section>
<label for="trigger-scope"
  >Set <code>trigger-scope</code> to <code>none</code>
  <input id="trigger-scope" type="checkbox" />
</label>
```

CSS

We define three `@keyframes` animations. Each will be applied to a different `.animated` element.

```
@keyframes fade-in {
  from {
    opacity: 1;
  }

  to {
    opacity: 0;
  }
}

@keyframes color-cycle {
```

```
  from {
    background: red;
    scale: 1;
  }

  to {
    background: blue;
    scale: 2;
  }
}

@keyframes move-up-down {
  25% {
    translate: 0 -20px;
  }

  75% {
    translate: 0 20px;
  }
}
```

```
body {
  width: 60%;
  margin: 0 auto;
  font-size: 1.3rem;
  font-family: "Helvetica", "Arial";
}

p {
  line-height: 1.5;
}

section {
  background: #eeeeee;
  padding: 10px 20px;
  margin-top: 20px;
}

.animated {
  width: 50px;
  height: 50px;
  background: red;
  border: 5px solid black;
}

label {
  position: fixed;
  bottom: 2px;
  right: 2px;
  padding: 5px;
  border: 2px solid black;
  background: white;
}
```

The animated elements' `position` is set to `fixed`, positioning them near the top of the scrollport to keep them visible at all times.

Each animated element has the same `animation-trigger` value: their animations are triggered by a trigger with a `timeline-trigger-name` of `--t`, and the animations will play when their trigger activates and then reset when their trigger deactivates.

```
.animated {
  position: fixed;
  top: 10px;
  animation-trigger: --t play reset;
}
```

Using the `animation` shorthand, each `.animated` element is given a different `animation-name`. They each also have a different `left` value so that they are not positioned on top of one another.

```
#one .animated {
  animation: fade-in 1s infinite alternate ease-in;
  left: 10px;
}

#two .animated {
  animation: color-cycle 1s infinite alternate linear;
  left: 110px;
}

#three .animated {
  animation: move-up-down 2s infinite linear;
  left: 210px;
}
```

The `.trigger` elements are set as triggers for the `.animated` elements by giving them a `timeline-trigger-name` value that references the same identifier, `--t`, and a `timeline-trigger-source` of `view()`. We set the `timeline-trigger-activation-range` to `contain` and let the `timeline-trigger-active-range` default to the same value. As a result, activation and deactivation occur while the trigger is still visible. We also set some rudimentary styles to make them stand out from the rest of the text.

```
.trigger {
  timeline-trigger-name: --t;
  timeline-trigger-source: view();
  timeline-trigger-activation-range: contain;

  padding: 10px;
  border: 2px solid black;
  background: black;
  color: white;
}
```

Finally, we set the `trigger-scope` on the `<section>` element to `all`. This limits the effect of each trigger named `--t` to their `<section>` ancestor.

```
section {
  trigger-scope: all;
}
```

```
:has(:checked) section {
  trigger-scope: none;
}

@supports not (trigger-scope: all) {
  body::before {
    content: "Your browser does not support the trigger-scope property.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Scroll down the example. One square animates at a time. This is because each square animates only when the trigger element located in the same scope (the same `<section>`) is visible in the scrollport. Even though the three triggers share the same trigger name, each `.animated` element's animation is triggered by a different trigger.

Now check the checkbox to remove `trigger-scope: all` from the `<section>` elements. Scroll through the content again. None of the squares animate until the third `.trigger` is visible in the scrollport, at which point all of the squares start animating at the same time. Because scoping has been removed, each `.animated` element's animation is activated and deactivated by the last element with `--t` as a `timeline-trigger-name`.

See also

- `animation-trigger`
- `timeline-trigger-name`, `timeline-trigger-source`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range`
- `timeline-trigger` shorthand property
- `<animation-action>` type
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

unicode-bidi CSS property

The **`unicode-bidi`** CSS property, together with the `direction` property, determines how bidirectional text in a document is handled. For example, if a block of content contains both left-to-right and right-to-left text, the user-agent uses a complex Unicode algorithm to decide how to display the text. The `unicode-bidi` property overrides this algorithm and allows the developer to control the text embedding.

Try it

```
unicode-bidi: normal;
```

```
unicode-bidi: bidi-override;
```

```
unicode-bidi: plaintext;
```

```
unicode-bidi: isolate-override;
```

```
<section class="default-example" id="default-example">
  <p class="transition-all" id="example-element">
    בְּרֵאשִׁית בָּרָא אֱלֹהִים אֵת הַשָּׁמַיִם וְאֵת הָאָרֶץ.
  </p>
</section>
```

The `unicode-bidi` and `direction` properties are the only properties that are not affected by the `all` shorthand.

> **Warning:** This property is intended for Document Type Definition (DTD) designers. Web designers and similar authors **should not** override it.

Syntax

```
/* Keyword values */
unicode-bidi: normal;
unicode-bidi: embed;
unicode-bidi: isolate;
unicode-bidi: bidi-override;
unicode-bidi: isolate-override;
unicode-bidi: plaintext;

/* Global values */
unicode-bidi: inherit;
unicode-bidi: initial;
unicode-bidi: revert;
unicode-bidi: revert-layer;
unicode-bidi: unset;
```

Values

`normal`

The element does not offer an additional level of embedding with respect to the bidirectional algorithm. For inline elements, implicit reordering works across element boundaries.

`embed`

If the element is inline, this value opens an additional level of embedding with respect to the bidirectional algorithm. The direction of this embedding level is given by the `direction` property.

`bidi-override`

For inline elements this creates an override. For block container elements this creates an override for inline-level descendants not within another block container element. This means that inside the element, reordering is strictly in sequence according to the `direction` property; the implicit part of the bidirectional algorithm is ignored.

`isolate`

This keyword indicates that the element's container directionality should be calculated without considering the content of this element. The element is therefore *isolated* from its siblings. When applying its bidirectional-resolution algorithm, its container element treats it as one or several `U+FFFC Object Replacement Character`, i.e., like an image.

`isolate-override`

This keyword applies the isolation behavior of the `isolate` keyword to the surrounding content and the override behavior of the `bidi-override` keyword to the inner content.

`plaintext`

This keyword makes the elements directionality calculated without considering its parent bidirectional state or the value of the `direction` property. The directionality is calculated using the P2 and P3 rules of the Unicode Bidirectional Algorithm. This value allows the display of data that is already formatted using a tool following the Unicode Bidirectional Algorithm.

Formal definition

Initial value	`normal`
Applies to	all elements, though some values have no effect on non-inline elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
unicode-bidi =
  normal           |
  embed            |
  isolate          |
  bidi-override    |
  isolate-override |
  plaintext
```

This syntax reflects the latest standard as per CSS Writing Modes Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

CSS

```
.bible-quote {
  direction: rtl;
  unicode-bidi: embed;
}
```

HTML

```
<div class="bible-quote">A line of text</div>
<div>Another line of text</div>
```

Result

See also

- `direction`
- SVG `unicode-bidi` attribute
- Handling different text directions

user-modify CSS property

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`user-modify`** property has no effect in Firefox. It was originally planned to determine whether or not the content of an element can be edited by a user.

> **Warning:** This property has been replaced by the `contenteditable` attribute.

Syntax

```
/* Keyword values */
user-modify: read-only;
user-modify: read-write;
user-modify: write-only;

/* Global values */
user-modify: inherit;
user-modify: initial;
user-modify: revert;
user-modify: unset;
```

Values

This property is specified as one of the following keyword values:

`read-only`

Default value. Contents are read-only.

`read-write`

The user is able to read and write contents.

`read-write-plaintext-only`

Same as `read-write`, but rich text formatting will be lost.

`write-only`

The user is able to edit the content, but not to read it.

Formal syntax

```
user-modify =
  read-only                |
  read-write               |
  read-write-plaintext-only  |
  write-only
```

Examples

HTML

```
<div class="readwrite">The user is able to change this text.</div>
```

CSS

```
.readwrite {
  -moz-user-modify: read-write;
  -webkit-user-modify: read-write;
}
```

Specifications

Not part of any standard.

See also

- `-moz-user-focus`
- `-moz-user-input`
- `-moz-user-select`

user-select CSS property

The **`user-select`** CSS property controls whether the user can select text. This doesn't have any effect on content loaded as part of a browser's user interface (its chrome), except in textboxes.

Try it

```
user-select: none;
```

```
user-select: text;
```

```
user-select: all;
```

```
<section id="default-example">
  <p id="example-element">Try to select this text</p>
</section>
```

```
#example-element {
  font-size: 1.5rem;
}
```

Syntax

```
/* Keyword values */
user-select: none;
user-select: auto;
user-select: text;
user-select: all;

/* Global values */
user-select: inherit;
user-select: initial;
user-select: revert;
user-select: revert-layer;
user-select: unset;
```

> **Note:** `user-select` is not an inherited property, though the initial `auto` value makes it behave like it is inherited most of the time. WebKit/Chromium-based browsers *do* implement the property as inherited, which violates the behavior described in the spec, and this will bring some issues. Until now, Chromium has chosen to fix the issues to make the final behavior meet the specifications.

Values

`none`

The text of the element and its sub-elements is not selectable. Note that the `Selection` object can contain these elements.

`auto`

The used value of `auto` is determined as follows:

- On the `::before` and `::after` pseudo elements, the used value is `none`
- If the used value of `user-select` on the parent of this element is `none`, the used value is `none`
- Otherwise, if the used value of `user-select` on the parent of this element is `all`, the used value is `all`
- Otherwise, the used value is `text`

`text`

The text can be selected by the user.

`all`

The content of the element shall be selected atomically: If a selection would contain part of the element, then the selection must contain the entire element including all its descendants. If a double-click or context-click occurred in sub-elements, the highest ancestor with this value will be selected.

> **Note:** The CSS basic user interface module defines a `contain` value for the `user-select` property to enable selection to start within the element to be contained by the bounds of that element, however, this is not supported in any browsers.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
user-select =
  auto     |
  text     |
  none     |
  contain  |
  all
```

This syntax reflects the latest standard as per CSS Basic User Interface Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p>You should be able to select this text.</p>
<p class="unselectable">Hey, you can't select this text!</p>
<p class="all">Clicking once will select all of this text.</p>
```

CSS

```
.unselectable {
  -webkit-user-select: none; /* Safari */
  user-select: none;
}

.all {
  -webkit-user-select: all;
  user-select: all;
}
```

Result

See also

- `::selection` pseudo-element
- The JavaScript `Selection` object

vector-effect CSS property

The **`vector-effect`** CSS property suppresses specific transformation effects in SVG, thus permitting effects like a road on a map staying the same width no matter how the map is zoomed, or allowing a diagram key to retain its position and size regardless of other transforms. It can only be used with SVG elements that accept the `vector-effect` attribute. When used, the CSS value overrides any values of the element's `vector-effect` attribute.

Syntax

```
/* Keyword values */
vector-effect: none;
vector-effect: non-scaling-stroke;

/* Global values */
vector-effect: inherit;
vector-effect: initial;
vector-effect: revert;
vector-effect: revert-layer;
vector-effect: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

No vector effects are applied to the element, meaning it will be fully affected by transforms as normal.

`non-scaling-stroke`

The element's drawn stroke width will be physically equal in size to its defined stroke width, even if the element has been scaled up or down in size due to transforms of either itself or its coordinate system. This is the case whether the element is scaled with transforms or by physical resizing of the entire image.

> **Note:** The spec defines three other values, `non-scaling-size`, `non-rotation`, and `fixed-position`, but these have no implementations and are considered at-risk.

Formal syntax

```
vector-effect =
  none               |
  non-scaling-stroke |
  non-scaling-size   |
```

```
  non-rotation        |
  fixed-position
```

This syntax reflects the latest standard as per CSS Specification. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Preventing SVG stroke scaling with CSS

Here, we start with a 200x100 SVG image that contains two rectangles inside a group. The group is scaled up and rotated. The second of the two rectangles has a class of `thinned`.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 100">
  <g
    transform="scale(2.3) rotate(23)"
    transform-origin="100 50"
    stroke-width="3"
    stroke="orange"
    fill="#ddeeff88">
    <rect x=" 60" y="20" width="30" height="60" />
    <rect x="110" y="20" width="30" height="60" class="thinned" />
  </g>
</svg>
```

To this SVG image, we apply `width: 500px` to make it larger than its intrinsic size, and set the classed `<rect>` to have non-scaled strokes.

```
svg {
  width: 500px;
}
svg rect.thinned {
  vector-effect: non-scaling-stroke;
}
```

The result is that the first of the two rectangles has an apparent (visual) stroke width of approximately 17, whereas the second rectangle still has an apparent stroke width of 3 despite having been scaled up in the same way the first rectangle has.

Overriding SVG stroke scaling values with CSS

In this case, we start with a similar SVG image to the one used in the previous example. Here, the `<g>` element is rotated as before, but no scaling is applied to it. The `<rect>` elements are given a common origin for their transforms, and have their `vector-effect` SVG attributes set to a value of `none`.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 100">
  <g
    transform="rotate(23)"
    transform-origin="100 50"
    stroke-width="3"
    stroke="orange"
    fill="#ddeeff88">
    <rect
      x=" 60"
      y="20"
      width="30"
      height="60"
      transform-origin="100 50"
      vector-effect="none" />
    <rect
      x="110"
      y="20"
      width="30"
      height="60"
      transform-origin="100 50"
      vector-effect="none"
      class="thinned" />
  </g>
</svg>
```

As before, the SVG is made larger than its intrinsic size using CSS. This time, scaling is applied to the `<rect>` elements directly, and the second rectangle is set to have its strokes non-scaled.

```
svg {
  width: 500px;
}
svg rect {
  transform: scale(2.3);
}
svg rect.thinned {
  vector-effect: non-scaling-stroke;
}
```

The result is visually identical to that of the previous example. What we can see is that the attribute value of `none` is overridden by the CSS value `non-scaling-stroke`, and that the vector effects are honored even though the scaling was done directly on the `<rect>` rather than on its parent `<g>` element.

See also

- `stroke`
- `<basic-shape>` data type
- SVG `vector-effect` attribute

vertical-align CSS property

The **`vertical-align`** CSS shorthand property sets the vertical alignment of an inline, inline-block, or table-cell box.

Constituent properties

This property is a shorthand for the following CSS properties:

- `alignment-baseline`
- `baseline-source`
- `baseline-shift`

Try it

```
vertical-align: baseline;
```

```
vertical-align: top;
```

```
vertical-align: middle;
```

```
vertical-align: bottom;
```

```
vertical-align: sub;
```

```
vertical-align: text-top;
```

```
<section class="default-example" id="default-example">
  <p>
    Align the star:
    <img id="example-element" src="/shared-assets/images/examples/star2.png" />
  </p>
</section>
```

```
#default-example > p {
  line-height: 3em;
  font-family: monospace;
  font-size: 1.2em;
  text-decoration: underline overline;
}
```

The `vertical-align` property can be used in two contexts:

- To vertically align an inline-level element's box inside its containing line box. For example, it could be used to vertically position an image in a line of text.
- To vertically align the content of a cell in a table.

Use this shorthand (`vertical-align`) instead of its corresponding longhand properties, unless you need to cascade them independently or (on SVG elements) to support legacy SVG implementations.

Note that `vertical-align` only applies to inline, inline-block and table-cell elements: you can't use it to vertically align block-level elements.

Syntax

```
/* Keyword values */
vertical-align: baseline;
vertical-align: sub;
vertical-align: super;
vertical-align: text-top;
vertical-align: text-bottom;
vertical-align: middle;
vertical-align: top;
vertical-align: bottom;

/* <length> values */
vertical-align: 10em;
vertical-align: 4px;

/* <percentage> values */
vertical-align: 20%;

/* Global values */
vertical-align: inherit;
vertical-align: initial;
vertical-align: revert;
vertical-align: revert-layer;
vertical-align: unset;
```

The `vertical-align` property is specified as one of the values listed below.

Values for inline elements

Parent-relative values

These values vertically align the element relative to its parent element:

`baseline`

> Aligns the baseline of the element with the baseline of its parent. The baseline of some replaced elements, like `<textarea>`, is not specified by the HTML specification, meaning that their behavior with this keyword may vary between browsers.

`sub`

> Aligns the baseline of the element with the subscript-baseline of its parent.

`super`

> Aligns the baseline of the element with the superscript-baseline of its parent.

`text-top`

> Aligns the top of the element with the top of the parent element's font.

`text-bottom`

> Aligns the bottom of the element with the bottom of the parent element's font.

`middle`

> Aligns the middle of the element with the baseline plus half the x-height of the parent.

`<length>`

> Aligns the baseline of the element to the given length above the baseline of its parent. A negative value is allowed.

`<percentage>`

Aligns the baseline of the element to the given percentage above the baseline of its parent, with the value being a percentage of the `line-height` property. A negative value is allowed.

Line-relative values

The following values vertically align the element relative to the entire line:

`top`

Aligns the top of the element and its descendants with the top of the entire line.

`bottom`

Aligns the bottom of the element and its descendants with the bottom of the entire line.

For elements that do not have a baseline, the bottom margin edge is used instead.

Values for table cells

`baseline` (and `sub`, `super`, `text-top`, `text-bottom`, `<length>`, and `<percentage>`)

Aligns the baseline of the cell with the baseline of all other cells in the row that are baseline-aligned.

`top`

Aligns the top padding edge of the cell with the top of the row.

`middle`

Centers the padding box of the cell within the row.

`bottom`

Aligns the bottom padding edge of the cell with the bottom of the row.

Negative values are allowed.

Formal definition

Initial value	`baseline`
Applies to	as each of the properties of the shorthand:. It also applies to `::first-letter` and `::first-line`.
Inherited	no
Computed value	as each of the properties of the shorthand:
Animation type	discrete

Formal syntax

```
vertical-align =
  [ first | last ]       ||
  <'alignment-baseline'>  ||
  <'baseline-shift'>

<alignment-baseline> =
  baseline           |
  <baseline-metric>

<baseline-shift> =
  <length-percentage>  |
  sub                  |
  super                |
  top                  |
  center               |
  bottom

<baseline-metric> =
  text-bottom   |
  alphabetic    |
  ideographic   |
  middle        |
  central       |
  mathematical  |
  hanging       |
  text-top

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Inline Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

HTML

```
<div>
  An <img src="frame_image.svg" alt="link" width="32" height="32" /> image with
  a default alignment.
</div>
<div>
  An
  <img class="top" src="frame_image.svg" alt="link" width="32" height="32" />
  image with a text-top alignment.
</div>
<div>
  An
  <img class="bottom" src="frame_image.svg" alt="link" width="32" height="32" />
  image with a text-bottom alignment.
</div>
```

```
<div>
  An
  <img class="middle" src="frame_image.svg" alt="link" width="32" height="32" />
  image with a middle alignment.
</div>
```

CSS

```
img.top {
  vertical-align: text-top;
}
img.bottom {
  vertical-align: text-bottom;
}
img.middle {
  vertical-align: middle;
}
```

Result

Vertical alignment in a line box

HTML

```
<p>
top:         <img style="vertical-align: top" src="star.png" alt="star"/>
middle:      <img style="vertical-align: middle" src="star.png" alt="star"/>
bottom:      <img style="vertical-align: bottom" src="star.png" alt="star"/>
super:       <img style="vertical-align: super" src="star.png" alt="star"/>
sub:         <img style="vertical-align: sub" src="star.png" alt="star"/>
</p>

<p>
text-top:    <img style="vertical-align: text-top" src="star.png" alt="star"/>
text-bottom: <img style="vertical-align: text-bottom" src="star.png" alt="star"/>
0.2em:       <img style="vertical-align: 0.2em" src="star.png" alt="star"/>
-1em:        <img style="vertical-align: -1em" src="star.png" alt="star"/>
20%:         <img style="vertical-align: 20%" src="star.png" alt="star"/>
-100%:       <img style="vertical-align: -100%" src="star.png" alt="star"/>
</p>
```

```
#* {
  box-sizing: border-box;
}

img {
  margin-right: 0.5em;
}

p {
  height: 3em;
  padding: 0 0.5em;
  font-family: monospace;
  text-decoration: underline overline;
  margin-left: auto;
  margin-right: auto;
  width: 80%;
}
```

Result

Vertical alignment in a table cell

In this example, we have a table with a single row containing six cells. The row sets `vertical-align` to `bottom` as the default value.

- The first four cells each set their own `vertical-align` values, and these override the row's value.
- The fifth cell does not set any `vertical-align` value, so inherits the row's value.

The sixth cell is only used to ensure that the cells are tall enough to see the effect.

HTML

```
<table>
  <tbody>
    <tr class="bottom">
      <td class="baseline">baseline</td>
      <td class="top">top</td>
      <td class="middle">middle</td>
      <td>bottom</td>
      <td>Row's style</td>
      <td>
        Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse
        pretium felis eu sem mattis vulputate.
      </td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  margin-left: auto;
  margin-right: auto;
  width: 80%;
}

table,
th,
td {
  border: 1px solid black;
}
```

```
td {
  padding: 0.5em;
  font-family: monospace;
}

.bottom {
  vertical-align: bottom;
}

.baseline {
  vertical-align: baseline;
}

.top {
  vertical-align: top;
}

.middle {
  vertical-align: middle;
}
```

Result

See also

- Typical use cases of flexbox, section "Center item"
- `line-height`, `text-align`, `margin`
- Understanding `vertical-align`, or "How (Not) To Vertically Center Content"
- Vertical-Align: All You Need To Know

view-timeline CSS property

The **`view-timeline`** CSS shorthand property defines a named view progress timeline's name, direction, and inset values.

Constituent properties

This property is a shorthand for the following CSS properties:

- `view-timeline-axis`
- `view-timeline-inset`
- `view-timeline-name`

Syntax

```
/* One component */
view-timeline: none;
view-timeline: --custom_name_for_timeline;

/* Two components */
view-timeline: --custom_name_for_timeline block;
view-timeline: --custom_name_for_timeline y;
view-timeline: none inline;
view-timeline: none x;

/* Three components */
view-timeline: --custom_name_for_timeline block auto;
view-timeline: --custom_name_for_timeline block 20% 200px;

/* Global values */
view-timeline: inherit;
view-timeline: initial;
view-timeline: revert;
view-timeline: revert-layer;
view-timeline: unset;
```

Values

`<view-timeline-name>`

See `view-timeline-name`. The default value is `none`.

`<view-timeline-inset>`

See `view-timeline-inset`. The default value is `auto`.

`<view-timeline-axis>`

See `view-timeline-axis`. The default value is `block`.

Description

The `view-timeline` shorthand property defines a *named view progress timeline*, which progresses based on changes to the visibility of an element (the *subject*) within a scrollable element (*scroller*). The `view-timeline` property is set on the subject. If the scroller element does not overflow its container in the axis dimension or if the overflow is hidden or clipped, no scroll progress timeline will be created.

The subject's visibility inside the scroller is tracked — by default, the timeline is at `0%` when the subject is first visible at one edge of the scroller and `100%` when it reaches the opposite edge.

The `view-timeline` can contain three constituent values: a name for the named view progress timeline, a scroll axis value, and up to two timeline inset values. If only one value is declared, the value is the `view-timeline-name`. The name is then referenced in an `animation-timeline` declaration to indicate the element that will be animated as the timeline progresses. This can be the subject element, but it doesn't have to be — you can animate a different element as the subject moves through the scrolling area.

The `view-timeline` shorthand property can be applied to a container element to set a combination of the `<view-timeline-name>`, `<view-timeline-inset>`, and `<view-timeline-axis>` values. At least one value must be specified. If all values are specified, the order must be `<view-timeline-name>` followed by `<view-timeline-axis>` and/or `<view-timeline-inset>`.

The names specified in the `<view-timeline-name>` component must be a list of comma-separated `<dashed-ident>` values (which means they must start with `--`) or the keyword `none`.

Formal definition

Initial value	as each of the properties of the shorthand:

	• view-timeline-name: none • view-timeline-axis: block
Applies to	all elements
Inherited	no
Computed value	as each of the properties of the shorthand: • view-timeline-name: none or an ordered list of identifiers • view-timeline-axis: as specified
Animation type	as each of the properties of the shorthand: • view-timeline-name: Not animatable • view-timeline-axis: Not animatable

Formal syntax

```
view-timeline =
  [ <'view-timeline-name'> [ <'view-timeline-axis'> || <'view-timeline-inset'> ]? ]#

<view-timeline-name> =
  [ none | <dashed-ident> ]#

<view-timeline-axis> =
  [ block | inline | x | y ]#

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a named view progress timeline

A view progress timeline named `--subject-reveal` is defined using the `view-timeline` property on a subject element with a `class` of `animation`. This is then set as the timeline for the same element using `animation-timeline: --subject-reveal`. The result is that the subject element animates as it moves upwards through the document as it is scrolled.

HTML

The HTML for the example is shown below.

```
<div class="content">
  <h1>Content</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>

  <div class="subject animation"></div>

  <p>
    Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
    cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
    dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
    tellus orci ac auctor augue mauris. Risus quis varius quam quisque id diam
    vel quam elementum. Nibh praesent tristique magna sit amet purus gravida
    quis. Duis ultricies lacus sed turpis tincidunt id aliquet. In egestas erat
    imperdiet sed euismod nisi. Eget egestas purus viverra accumsan in nisl nisi
    scelerisque. Netus et malesuada fames ac.
  </p>
</div>
```

CSS

The `subject` element and its containing `content` element are styled minimally, and the text content is given some basic font settings:

```
.subject {
  width: 300px;
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p,
h1 {
```

```
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}
```

The `<div>` with the class of `subject` is also given a class of `animation` — this is where `view-timeline` is set to define a named view progress timeline. It is also given an `animation-timeline` name with the same value to declare that this will be the element animated as the view progress timeline progresses.

Lastly, an animation is specified on the element that animates its opacity and scale, causing it to fade in and grow as it moves up the scroller.

```
.animation {
  view-timeline: --subject-reveal block;
  animation-timeline: --subject-reveal;

  animation-name: appear;
  animation-fill-mode: both;
  animation-duration: 1ms; /* Firefox requires this to apply the animation */
}

@keyframes appear {
  from {
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    opacity: 1;
    transform: scaleX(1);
  }
}
```

```
@layer no-support {
  @supports not (view-timeline: none) {
    body::before {
      content: "Your browser doesn't support the `view-timeline` property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Scroll to see the subject element being animated.

See also

- `animation-timeline`
- `view-timeline-axis`, `view-timeline-inset`, `view-timeline-name`
- `view()`
- Guide: CSS scroll driven animation timelines
- CSS scroll-driven animations

view-timeline-axis CSS property

The **`view-timeline-axis`** CSS property specifies the scroll direction to be used for a named view progress timeline.

Syntax

```
/* Logical property values */
view-timeline-axis: block;
view-timeline-axis: inline;

/* Physical property values */
view-timeline-axis: y;
view-timeline-axis: x;

/* Global values */
view-timeline-axis: inherit;
view-timeline-axis: initial;
view-timeline-axis: revert;
view-timeline-axis: revert-layer;
view-timeline-axis: unset;
```

Values

`<axis>`

Specifies the scroll direction used by the view progress timeline. The value can be one of the `<axis>` keywords: `block`, `inline`, `x`, or `y`. The default value is `block`.

Description

The `view-timeline-axis` property specifies the direction, or `<axis>`, of named view progress timelines that are based on the element's box.

By default, CSS `@keyframes` animations progress along the time-based default timeline. When you set the animation progress via a view progress timeline instead, `view-timeline-axis` specifies the direction that controls the timeline progression.

For view progress timelines, progression of the animation along the timelines is based on the visibility of the element, or *subject*. The `view-timeline-axis` property is set on the subject.

The subject must be nested inside a scrollable element. If the scrollable element does not overflow its container in the axis dimension or if the overflow is hidden or clipped, no scroll progress timeline will be created.

The `view-timeline-axis`, along with the `view-timeline-inset` and `view-timeline-name` properties, is a component of the `view-timeline` shorthand property.

Formal definition

Initial value	`block`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
view-timeline-axis =
  [ block | inline | x | y ]#
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the axis of the view progress timeline

In this example, a view progress timeline named `--subject-reveal` is defined using the `view-timeline-name` property on a subject element with a class of "animation". This timeline is then applied to the animation on the same element, using `animation-timeline: --subject-reveal;`.

To demonstrate the effect of `view-timeline-axis`, a horizontal (non-default) scrollbar is used in this example to drive the animation.

HTML

The HTML for the example is shown below.

```
<div class="content">
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua.
  </p>

  <p>
    Risus quis varius quam quisque id. Et ligula ullamcorper malesuada proin
    libero nunc consequat interdum varius. Elit ullamcorper dignissim cras
    tincidunt lobortis feugiat vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras.
  </p>

  <p>
    A erat nam at lectus urna duis convallis convallis. Nibh ipsum consequat
    nisl vel pretium lectus.
  </p>

  <p>
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>

  <div class="subject animation"></div>

  <p>
    Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
    cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
    dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
    tellus orci ac auctor augue mauris.
  </p>
</div>
```

CSS

In the CSS, we set the `subject` element as the source of a view progress timeline named `--subject-reveal` using the `view-timeline-name` property. The scroll axis is set using `view-timeline-axis: x;`. We also include `view-timeline-axis: horizontal;` for browsers that support the non-standard legacy `horizontal` and `vertical` values, rather than `x` and `y`.

The `content` ancestor element is made to overflow horizontally by laying out its contents using `display: flex;` and `flex-flow: column wrap;`.

```
.subject {
  width: 300px;
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

.content {
  width: 50%;
  height: 400px;
  margin-top: 30px;
  display: flex;
  flex-flow: column wrap;
  gap: 10px;
}

p {
  font-family: "Helvetica", "Arial", sans-serif;
}

p {
  font-size: 1.3rem;
  line-height: 1.4;
}
```

```
.animation {
  view-timeline-name: --subject-reveal;
  view-timeline-axis: x;
  view-timeline-axis: horizontal;

  animation: appear 1ms linear both;
  animation-timeline: --subject-reveal;
}

@keyframes appear {
  from {
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    opacity: 1;
    transform: scaleX(1);
  }
}
```

```
@layer no-support {
  body::before {
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
  @supports not (view-timeline-axis: inherit) {
    body::before {
      content: "Your browser doesn't support the `view-timeline-axis` property.";
      background-color: wheat;
    }
  }
  @supports (view-timeline-axis: horizontal) {
    body::before {
      content: "Your browser supports legacy values for the `view-timeline-axis` property.";
      background-color: yellow;
    }
  }
}
```

Result

Scroll the horizontal bar at the bottom to see the subject element animate as you scroll.

See also

- `animation-timeline`
- `view-timeline`, `view-timeline-inset`, `view-timeline-name`
- `view()`
- Guide: Scroll-driven animation timelines
- CSS scroll-driven animations module
- CSS animations module

view-timeline-inset CSS property

The **`view-timeline-inset`** CSS property is used to specify one or two values representing an adjustment to the position of the scrollport (see Scroll container for more details) in which the subject element of a *named view progress timeline* animation is deemed to be visible. Put another way, this allows you to specify start and/or end inset (or outset) values that offset the position of the timeline.

This can be combined with or used instead of `animation-range` and its longhand properties, which can be used to set the attachment range of an animation along its timeline. See CSS scroll-driven animations for more details.

> **Note:** If the scroller element does not overflow its container in the axis dimension or if the overflow is hidden or clipped, no scroll progress timeline will be created.

The `view-timeline-inset`, `view-timeline-axis`, and `view-timeline-name` properties can also be set using the `view-timeline` shorthand property.

Syntax

```
/* Single value */
view-timeline-inset: auto;
view-timeline-inset: 200px;
view-timeline-inset: 20%;

/* Two values */
view-timeline-inset: 20% auto;
view-timeline-inset: auto 200px;
view-timeline-inset: 20% 200px;
```

Values

Allowed values for `view-timeline-inset` are:

`auto`

> If set, the corresponding `scroll-padding` (or equivalent longhand value) for that edge of the scrollport is used. If this is not set (or set to `auto`), the value will usually be 0, although some user agents may use heuristics to determine a different default value if appropriate.

`<length-percentage>`

> Any valid `<length-percentage>` value is accepted as an inset/outset value.
>
> - If the value is positive, the position of the animation's start/end will be moved inside the scrollport by the specified length or percentage.
> - If the value is negative, the position of the animation's start/end will be moved outside the scrollport by the specified length or percentage, i.e., it will start animating before it appears in the scrollport, or finish animating after it leaves the scrollport.

If two values are provided, the first value represents the start inset/outset in the relevant axis (where the animation begins) and the second value represents the end inset/outset (where the animation ends). If only one value is provided, the start and end inset/outset are both set to the same value.

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Percentages	Relative to the corresponding dimension of the relevant scrollport
Computed value	A list where each item may be either 'auto' or a length percentage
Animation type	by computed value type

Formal syntax

```
view-timeline-inset =
  [ [ auto | <length-percentage> ]{1,2} ]#

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a named view progress timeline with inset

A view progress timeline named `--subject-reveal` is defined using the `view-timeline` property on a subject element with a `class` of `animation`. This is then set as the timeline for the same element using `animation-timeline: --subject-reveal;`. The result is that the subject element animates as it moves upwards through the document as it is scrolled.

A `view-timeline-inset` declaration is also set to make the animation begin later than expected, and finish earlier.

HTML

The HTML for the example is shown below.

```
<div class="content">
  <h1>Content</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>

  <div class="subject animation"></div>

  <p>
    Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
    cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
    dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
    tellus orci ac auctor augue mauris. Risus quis varius quam quisque id diam
    vel quam elementum. Nibh praesent tristique magna sit amet purus gravida
    quis. Duis ultricies lacus sed turpis tincidunt id aliquet. In egestas erat
    imperdiet sed euismod nisi. Eget egestas purus viverra accumsan in nisl nisi
    scelerisque. Netus et malesuada fames ac.
  </p>
</div>
```

CSS

The `subject` element and its containing `content` element are styled minimally, and the text content is given some basic font settings:

```
.subject {
  width: 300px;
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p,
h1 {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}
```

The `<div>` with the class of `subject` is also given a class of `animation` — this is where `view-timeline` is set to define a named view progress timeline. We also give it a `view-timeline-inset` declaration to make the animation begin later than expected, and finish earlier. It is also given an `animation-timeline` name with the same value to declare that this will be the element animated as the view progress timeline is progressed.

Last, an animation is specified on the element that animates its opacity and scale, causing it to fade in and size up as it moves up the scroller.

```
.animation {
  view-timeline: --subject-reveal block;
  view-timeline-inset: 70% -100px;
  animation-timeline: --subject-reveal;

  animation-name: appear;
  animation-fill-mode: both;
  animation-duration: 1ms; /* Firefox requires this to apply the animation */
}

@keyframes appear {
  from {
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    opacity: 1;
    transform: scaleX(1);
  }
}
```

Result

Scroll to see the subject element being animated.

See also

- `animation-timeline`
- `timeline-scope`
- `view-timeline`, `view-timeline-axis`, `view-timeline-name`
- CSS scroll-driven animations

view-timeline-name CSS property

The **`view-timeline-name`** CSS property specifies the names of one or more named view progress timelines associated with the element.

Syntax

```
/* Keyword value */
view-timeline-name: none;

/* Custom identifier */
view-timeline-name: --custom_name_for_timeline;

/* Multiple identifiers */
view-timeline-name: --first_timeline_name, --another_timeline_name;

/* Global values */
view-timeline-name: inherit;
view-timeline-name: initial;
view-timeline-name: revert;
view-timeline-name: revert-layer;
view-timeline-name: unset;
```

Values

`none`

Indicates that the timeline has no name. This is the default value.

`<dashed-ident>`

Specifies a comma-separated list of `<dashed-ident>` custom name identifiers. All `<dashed-ident>` values must start with `--`, which helps avoid name clashes with standard CSS keywords.

Description

The `view-timeline-name` property is used to set the name of a view progress timeline. A view progress timeline is progressed through based on the change in visibility of an element, called the *subject*, inside a scrollable element, called the *scroller*. The `view-timeline-name` property is set on the subject. You can also set `view-timeline-name` by using the `view-timeline` shorthand property.

If the subject's named scroller element does not overflow its container in the axis dimension or if the overflow is hidden or clipped, no scroll progress timeline will be created.

Once created, the `<dashed-ident>` name values specified can be referenced in `animation-timeline` declarations to indicate the element that will be animated as the timeline progresses.

Formal definition

Initial value	`none`
Applies to	all elements
Inherited	no
Computed value	`none` or an ordered list of identifiers
Animation type	Not animatable

Formal syntax

```
view-timeline-name =
  [ none | <dashed-ident> ]#
```

This syntax reflects the latest standard as per Scroll-driven Animations. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating a named view progress timeline

A view progress timeline named `--subject-reveal` is defined using the `view-timeline-name` property on a subject element with a `class` of `animation`. This is then set as the timeline for the same element using `animation-timeline: --subject-reveal;`. The result is that the subject element animates as it moves upwards through the document as it is scrolled.

HTML

The HTML for the example is shown below.

```
<div class="content">
  <h1>Content</h1>

  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Risus quis varius quam
    quisque id. Et ligula ullamcorper malesuada proin libero nunc consequat
    interdum varius. Elit ullamcorper dignissim cras tincidunt lobortis feugiat
    vivamus at augue.
  </p>

  <p>
    Dolor sed viverra ipsum nunc aliquet. Sed sed risus pretium quam vulputate
    dignissim. Tortor aliquam nulla facilisi cras. A erat nam at lectus urna
    duis convallis convallis. Nibh ipsum consequat nisl vel pretium lectus.
    Sagittis aliquam malesuada bibendum arcu vitae elementum. Malesuada bibendum
    arcu vitae elementum curabitur vitae nunc sed velit.
  </p>

  <div class="subject animation"></div>

  <p>
    Adipiscing enim eu turpis egestas pretium aenean pharetra magna ac. Arcu
    cursus vitae congue mauris rhoncus aenean vel. Sit amet cursus sit amet
    dictum. Augue neque gravida in fermentum et. Gravida rutrum quisque non
    tellus orci ac auctor augue mauris. Risus quis varius quam quisque id diam
    vel quam elementum. Nibh praesent tristique magna sit amet purus gravida
    quis. Duis ultricies lacus sed turpis tincidunt id aliquet. In egestas erat
    imperdiet sed euismod nisi. Eget egestas purus viverra accumsan in nisl nisi
    scelerisque. Netus et malesuada fames ac.
  </p>
</div>
```

CSS

The `subject` element and its containing `content` element are styled minimally, and the text content is given some basic font settings:

```
.subject {
  width: 300px;
  height: 200px;
  margin: 0 auto;
  background-color: deeppink;
}

.content {
  width: 75%;
  max-width: 800px;
  margin: 0 auto;
}

p,
h1 {
  font-family: "Helvetica", "Arial", sans-serif;
}

h1 {
  font-size: 3rem;
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
}
```

The `<div>` with the class of `subject` is also given a class of `animation` — this is where `view-timeline-name` is set to define a named view progress timeline. It is also given an `animation-timeline` name with the same value to declare that this will be the element animated as the view progress timeline is progressed.

Last, an animation is specified on the element that animates its opacity and scale, causing it to fade in and size up as it moves up the scroller.

```
.animation {
  view-timeline-name: --subject-reveal;
  animation-timeline: --subject-reveal;

  animation-name: appear;
  animation-fill-mode: both;
  animation-duration: 1ms; /* Firefox requires this to apply the animation */
}

@keyframes appear {
  from {
    opacity: 0;
    transform: scaleX(0);
  }

  to {
    opacity: 1;
    transform: scaleX(1);
  }
}
```

```
@layer no-support {
  @supports not (view-timeline-name: none) {
    body::before {
      content: "Your browser doesn't support the `view-timeline-name` property.";
      background-color: wheat;
      display: block;
      text-align: center;
      padding: 1rem 0;
    }
  }
}
```

Result

Scroll to see the subject element being animated.

See also

- `animation-timeline`
- `view-timeline`
- Scroll-driven animation timelines
- CSS scroll-driven animations module
- CSS animations module

view-transition-class CSS property

The **`view-transition-class`** CSS property provides the selected elements with one or more identifying classes (`<custom-ident>`s), providing an additional method of styling the view transitions for those elements.

Syntax

```
/* <custom-ident> value examples */
view-transition-class: card;
view-transition-class: card fast-slide;

/* Keyword value */
view-transition-class: none;

/* Global values */
view-transition-class: inherit;
view-transition-class: initial;
view-transition-class: revert;
view-transition-class: revert-layer;
view-transition-class: unset;
```

Values

`<custom-ident>`

> An identifying name used to select view transition pseudo-elements for styling. Unlike `view-transition-name`, a class does not need to be unique and does not cause the element to participate in a separate view transition group.

`none`

> No class would apply to the named view transition pseudo-elements generated for this element.

Description

The `view-transition-class` value provides a styling hook, similar to a CSS class name, which can be used to apply the same styles to multiple view transition pseudo-elements. It does not mark an element for capturing. Each individual element still needs its own unique `view-transition-name`; the `view-transition-class` is only used as an additional way to style elements that already have a `view-transition-name`. Support for determining the `view-transition-name` automatically is being discussed in the CSS View Transitions Module Level 2 spec.

The `view-transition-class` applies styles using the view transition pseudo-elements, including `::view-transition-group()`, `::view-transition-image-pair()`, `::view-transition-old()`, and `::view-transition-new()`. This is different from the `view-transition-name`, which matches view transitions between the element in the old state with its corresponding element in the new state.

Until the `view-transition-class` property is fully supported in all browsers supporting view transitions, include a custom `::view-transition-group()` for each element.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
view-transition-class =
  none            |
  <custom-ident>+
```

This syntax reflects the latest standard as per CSS View Transitions Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Styling a shared class across multiple elements

In this example, three cards each have a unique `view-transition-name` (required for pairing old and new states), but they all share the same `view-transition-class`. This lets you write a single rule that styles all their transitions at once, rather than repeating styles for each name individually. Unlike `view-transition-name`, a `view-transition-class` does not need to be unique.

```
<div class="card" id="card1">Card 1</div>
<div class="card" id="card2">Card 2</div>
<div class="card" id="card3">Card 3</div>
```

```
/* Each element must have a unique view-transition-name */
#card1 {
  view-transition-name: card-1;
```

```
}

#card2 {
  view-transition-name: card-2;
}

#card3 {
  view-transition-name: card-3;
}

/* But they can all share the same view-transition-class */
.card {
  view-transition-class: card;
}

/* This single rule applies to all three cards' transitions */
::view-transition-group(.card) {
  animation-duration: 0.5s;
  animation-timing-function: ease-in-out;
}
```

Using multiple classes on a single element

A `view-transition-class` value can be a space-separated list of identifiers, letting you compose several "atomic" styles on the same element and target each one independently of your view transition pseudo-elements. In this example, both cards share the same two classes — `slide` controls the animation, and `fast-transition` controls its duration — while each card still has its own unique `view-transition-name`.

```
<div class="card" id="card1">Card 1</div>
<div class="card" id="card2">Card 2</div>
```

```
.card {
  view-transition-class: slide fast-transition;
}

#card1 {
  view-transition-name: card-1;
}

#card2 {
  view-transition-name: card-2;
}

/* The `slide` class drives which animation runs... */
::view-transition-new(.slide) {
  animation-name: slide-in;
}

::view-transition-old(.slide) {
  animation-name: slide-out;
}

/* ...while the `fast-transition` class drives how long it runs. */
::view-transition-group(.fast-transition) {
  animation-duration: 0.5s;
}
```

See also

- `view-transition-name`
- Using the View Transition API guide
- View Transition API
- Smooth transitions with the View Transition API

view-transition-name CSS property

The **`view-transition-name`** CSS property specifies the view transition snapshot that selected elements will participate in. This enables you to animate those elements separately from the rest of the page, which uses the default cross-fade animation during a view transition. You can then define custom animation styles for these elements.

Syntax

```
/* <custom-ident> value examples */
view-transition-name: header;
view-transition-name: figure-caption;

/* Keyword values */
view-transition-name: none;
view-transition-name: match-element;

/* Global values */
view-transition-name: inherit;
view-transition-name: initial;
view-transition-name: revert;
view-transition-name: revert-layer;
view-transition-name: unset;
```

Values

`<custom-ident>`

An identifying name that causes the selected element to participate in a separate snapshot from the root snapshot. The `<custom-ident>` cannot be `auto`, `match-element`, `none`, or a CSS-wide keyword value.

`match-element`

The browser automatically assigns a unique name to the selected element. This name is used to snapshot the element separately from all other elements on the page. (This name is internal and cannot be read from the DOM.)

`none`

The selected element will not participate in a separate snapshot, unless it has a parent element with a `view-transition-name` set, in which case it will be snapshotted as part of that element.

Description

By default, when a view transition is applied to a web app, all changes to the UI that occur during that transition are snapshotted and animated together. This is the default — or `root` — snapshot (see The view transition pseudo-element tree). By default, this animation is a smooth cross-fade, which can be seen in action in the View Transitions SPA demo.

If you want certain elements to be animated differently from the `root` snapshot during the view transition, you can do so by giving them a different `view-transition-name`, for example:

```
figcaption {
  view-transition-name: figure-caption;
}
```

You can then specify which animations you want for the before and after snapshots using the relevant view transition pseudo-elements — `::view-transition-old()` and `::view-transition-new()`. For example:

```
::view-transition-old(figure-caption) {
  animation: 0.25s linear both shrink-x;
}

::view-transition-new(figure-caption) {
  animation: 0.25s 0.25s linear both grow-x;
}
```

If you don't want an element to be snapshotted separately, you can specify a `view-transition-name` value of `none`:

```
.dont-animate-me {
  view-transition-name: none;
}
```

The `view-transition-name` `<custom-ident>` must be unique for each rendered element taking part in the view transition. If two rendered elements have the same `view-transition-name` at the same time, the `ViewTransition.ready` `Promise` will reject and the transition will be skipped.

Specifying <code>view-transition-name</code> values automatically

Sometimes you will want to animate multiple UI elements separately in a view transition. This is often the case when you have a list of elements on a page and want to rearrange them in some way:

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>

  <!-- ... -->

  <li>Item 99</li>
</ul>
```

Giving each one a unique name can be inconvenient, especially as the number of elements gets larger:

```
li:nth-child(1) {
  view-transition-name: item1;
}
li:nth-child(2) {
  view-transition-name: item2;
}
li:nth-child(3) {
  view-transition-name: item3;
}
li:nth-child(4) {
  view-transition-name: item4;
}

/* ... */

li:nth-child(99) {
  view-transition-name: item99;
}
```

To get around this problem, you can use the `match-element` value, which causes the browser to give each selected element a unique internal `view-transition-name`:

```
li {
  view-transition-name: match-element;
}
```

Because `match-element` assigns automatic `view-transition-name` values based on element identity, it can only be used for same-document view transitions. The auto-generated internal identifiers are not transferrable across different elements or documents.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
view-transition-name =
  none            |
  <custom-ident>
```

This syntax reflects the latest standard as per CSS View Transitions Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>view-transition-name</code> usage

This example comes from the View Transitions SPA demo, which is a basic image gallery. The Basic SPA view transition provides a more detailed explanation of how this demo works.

Most of the UI changes are animated using the `root` transition snapshot. However, the `<figcaption>` is given a `view-transition-name` of `figure-caption` to allow it to be animated differently from the rest of the page:

```
figcaption {
  view-transition-name: figure-caption;
}
```

The following code applies a custom animation just to the `<figcaption>`:

```
@keyframes grow-x {
  from {
    transform: scaleX(0);
  }
  to {
    transform: scaleX(1);
  }
}

@keyframes shrink-x {
  from {
    transform: scaleX(1);
  }
  to {
    transform: scaleX(0);
  }
}

::view-transition-group(figure-caption) {
  height: auto;
  right: 0;
  left: auto;
  transform-origin: right center;
}

::view-transition-old(figure-caption) {
  animation: 0.25s linear both shrink-x;
}

::view-transition-new(figure-caption) {
  animation: 0.25s 0.25s linear both grow-x;
}
```

We create a custom CSS animation and apply it to the `::view-transition-old(figure-caption)` and `::view-transition-new(figure-caption)` pseudo-elements. We also apply other styles to keep them both in the same place and to stop the default styling from interfering with our custom animations.

Using the <code>match-element</code> value

This example contains a list of technologies–HTML, CSS, SVG, and JS–that are displayed in a sidebar next to a main content area, which starts out empty. Clicking a technology's heading animates its content into the adjoining content area that shows more details.

HTML

The `<main>` element contains an unordered list and an `<article>` element. The multiple child `<li>` elements inside the list each contain an `<a>` element inside a heading.

```
<main class="match-element-applied">
  <ul>
    <li>
      <h2><a href="#">HTML</a></h2>
      <h3>HyperText Markup Language</h3>
      <p>
        HyperText Markup Language (HTML) is the most basic building block of the
        web. It defines the meaning and structure of web content. HTML provides
        the fundamental building blocks for structuring web documents and apps.
      </p>
    </li>
    <li>
      <h2><a href="#">CSS</a></h2>
      <h3>Cascading Style Sheets</h3>
      <p>
        Cascading Style Sheets (CSS) is a stylesheet language used to describe
        the presentation of a document written in HTML or XML (including XML
        dialects such as SVG, MathML or XHTML). CSS describes how elements
        should be rendered on screen, on paper, in speech, or on other media.
      </p>
    </li>
    <li>
      <h2><a href="#">SVG</a></h2>
      <h3>Scalable Vector Graphics</h3>
      <p>
        Scalable Vector Graphics (SVG) is an XML-based markup language for
        describing two-dimensional based vector graphics.
      </p>
    </li>
    <li>
      <h2><a href="#">JS</a></h2>
      <h3>JavaScript</h3>
      <p>
        JavaScript (JS) is the web's native programming language. JavaScript is
        a lightweight, interpreted (or just-in-time compiled) programming
        language with first-class functions. While it is most well-known as the
        scripting language for web pages, many non-browser environments, such as
        Node.js, also use it.
      </p>
    </li>
  </ul>
```

```
  <article></article>
</main>
```

```
<form>
  <label for="match-element-checkbox">
    Apply <code>match-element</code> to list items?
  </label>
  <input type="checkbox" id="match-element-checkbox" checked />
</form>
```

CSS

We use flexbox to lay out the `<li>` and the `<article>` side by side, and to make the list items share equal amount of space in the first column. The list takes up 35% of the container's width, while the `<article>` fills the remaining available horizontal space.

```
/* General styles and resets */
* {
  box-sizing: border-box;
  font-size: 0.9rem;
}

html {
  font-family: "Helvetica", "Arial", sans-serif;
  height: 100%;
}

body {
  margin: 0;
  height: inherit;
  display: flex;
  flex-direction: column;
  justify-content: space-around;
}
li h2 {
  margin: 0;
}

ul {
  padding: 0;
  margin: 0;
  list-style-type: none;
}

li {
  overflow: hidden;
  container-type: inline-size;
}

li p {
  display: none;
}

li.active-item p {
  display: block;
}

li:nth-child(1) {
  background-color: #cbc0d3;
  border: 20px solid #cbc0d3;
}

li:nth-child(2) {
  background-color: #efd3d7;
  border: 20px solid #efd3d7;
}

li:nth-child(3) {
  background-color: #feeafa;
  border: 20px solid #feeafa;
}

li:nth-child(4) {
  background-color: #dee2ff;
  border: 20px solid #dee2ff;
}

/* Links */

a {
  text-decoration: none;
  color: rgb(0 0 255 / 0.8);
}

a:hover,
a:focus {
  color: rgb(100 100 255);
}

/* Form and checkbox styles */
form {
  position: absolute;
  bottom: 0;
  right: 0;
  z-index: 2;
  background-color: white;
  padding: 10px;
  border: 1px solid black;
}
```

```
main {
  container-type: inline-size;
```

```
  width: 100%;
  height: 100%;
  display: flex;
  gap: 2cqw;
  position: relative;
}

ul {
  width: 35cqw;
  display: flex;
  flex-direction: column;
  gap: 1cqw;
}

article {
  flex: 1;
}

li {
  flex: 1;
}
```

We also define a rule that selects elements with the `active-item` class. When this class is applied to an element, the rule causes it to be positioned exactly over the top of the `<article>` element. This class will be applied to the list items via JavaScript when their links are clicked, which will initiate a view transition.

```
.active-item {
  position: absolute;
  z-index: 1;
  translate: 37cqw;
  width: calc(100% - 37cqw);
  height: 100%;
}
```

By default, all elements in a view transition are animated together in a single cross-fade. In this example, however, we don't want this — we want each list item to have its own movement animation. We can achieve this by applying `view-transition-name: match-element` to every list item:

```
.match-element-applied li {
  view-transition-name: match-element;
}
```

The `match-element-applied` class is applied to the `<main>` element by default, which is why the checkbox in the Result frame is initially selected. If you uncheck it, the class is removed and the default cross-fade animation comes into effect instead. You can toggle the checkbox to compare the default animation with the one applied when `view-transition-name: match-element` is used.

Next, we customize the animation by using the `::view-transition-group()` pseudo-element to apply an `animation-duration` to all the view transition groups (signified by the `*` identifier) and give all the old and new snapshots a `height` of `100%`. This works around differences in the aspect ratios of the old and new snapshots and makes the animations look smoother:

```
::view-transition-group(*) {
  animation-duration: 0.5s;
}

html::view-transition-old(*),
html::view-transition-new(*) {
  height: 100%;
}
```

JavaScript

In this example, the `active-item` class is applied to the list items when their links are clicked; this is achieved via the `updateActiveItem()` function:

```
const mainElem = document.querySelector("main");
let prevElem;
let checkboxElem = document.querySelector("input");

// View transition code
function updateActiveItem(event) {
  // Get the list item that contains the clicked link
  const clickedElem = event.target.parentElement.parentElement;

  // Set the active-item class on the list item
  clickedElem.className = "active-item";

  // Keep track of the previous item that was clicked, if any.
  // Remove the active-item class from the previous item so that only
  // one list item is placed over the <article> at any one time
  if (prevElem === clickedElem) {
    prevElem.className = "";
    prevElem = undefined;
  } else if (prevElem) {
    prevElem.className = "";
    prevElem = clickedElem;
  } else {
    prevElem = clickedElem;
  }
}

mainElem.addEventListener("click", (event) => {
  event.preventDefault(); // Prevent iframe from scrolling when clicked
  // Do nothing unless a link is clicked inside the <main> element
  if (event.target.tagName !== "A") {
    return;
  }

  // Run updateActiveItem() on its own if view transitions are not supported
  if (!document.startViewTransition) {
    updateActiveItem(event);
  } else {
    // Run updateActiveItem() via startViewTransition()
    const transition = document.startViewTransition(() =>
      updateActiveItem(event),
    );
```

```
  }
});

// Toggle the class on <main> to control whether or not match-element is applied

checkboxElem.addEventListener("change", () => {
  mainElem.classList.toggle("match-element-applied");
});
```

Running the `updateActiveItem()` function via the `startViewTransition()` function animates the display of technology details smoothly.

Result

Click a technology heading in the sidebar and notice the animation effect of its content into the main content area.

There is also a checkbox, which is selected by default, so `view-transition-name: match-element` is applied. Uncheck the checkbox and click a heading again to see how the view-transition works without `view-transition-name: match-element`.

See also

- `view-transition-class`
- `::view-transition-group()`
- `::view-transition-old()`
- `::view-transition-new()`
- View Transition API
- Smooth transitions with the View Transition API

view-transition-scope CSS property

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`view-transition-scope`** CSS property enables the discoverability of elements with `view-transition-name` values set on them (and therefore the creation of view transition snapshots) to be isolated to a specific element subtree.

Syntax

```
/* Keyword values */
view-transition-scope: none;
view-transition-scope: all;

/* Global values */
view-transition-scope: inherit;
view-transition-scope: initial;
view-transition-scope: revert;
view-transition-scope: revert-layer;
view-transition-scope: unset;
```

Values

This property is specified as one of the following keyword values:

`none`

The initial value. Discoverability of elements to snapshot during a view transition is not limited to any specific subtree.

`all`

Limits the discoverability of elements to snapshot during a view transition to the subtree of the element on which this property is set. Only elements with a non-`none` `view-transition-name` are considered.

Description

During the view transition process, the browser captures snapshots of elements that have a non-`none` `view-transition-name` set on them. These snapshots are then animated via CSS animations.

One issue that can arise during this process is naming collisions between elements involved in a view transition. You can't have the same `view-transition-name` set on multiple elements — if you do, the browser throws an `InvalidStateError` when the `Element.startViewTransition()` method is called to start the transition.

You could solve this problem by setting a `view-transition-name` of `match-element` on the elements to let the browser auto-assign internal unique names. However, this won't work if you are including multiple components from different sources that you don't control. A naming collision might still occur.

The `view-transition-scope` property allows view transitions to be self-contained. When `view-transition-scope: all` is set on an element, it limits the transition scope to that element and its descendants, which can be used to solve the above problem.

Whenever an element-scoped view transition is triggered, the browser automatically sets `view-transition-scope: all` on the transition root element, ensuring that only elements inside the transition scope are snapshotted and animated.

Formal definition

Initial value	none
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
view-transition-scope =
  none  |
  all
```

This syntax reflects the latest standard as per CSS View Transitions Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using <code>view-transition-scope</code> to isolate snapshots

This example demonstrates how to use `view-transition-scope` to isolate the scope of document-scoped view transitions, enabling the same `view-transition-name` to be used on multiple elements.

HTML

The HTML includes a `<button>` element to control updating the DOM, plus several components with the class `change-me`, some of which are nested, all wrapped in a `<section>` element.

```
<button>Update DOM</button>
<section>
  <div class="change-me"><span>I can change</span></div>
  <div class="change-me">
    <span>I can change</span>
    <div class="change-me"><span>I can change</span></div>
  </div>
  <div class="change-me"><span>I can change</span></div>
</section>
```

CSS

We start by setting the same `view-transition-name` on all components. We then set `view-transition-scope: all` on all of them, to isolate the view transition process for each one. Next, we set a longer `animation-duration` on all view transitions with this `view-transition-name` via the `::view-transition-group()` pseudo-element.

```
body {
  font: 1.2em / 1.5 sans-serif;
  width: 50%;
  max-width: 700px;
  margin: 0 auto;
}

section,
.change-me {
  border: 2px solid #666666;
  padding: 10px;
}

section {
  background-color: orange;
}
```

```
.change-me {
  background-color: white;
  view-transition-name: para-change;
  view-transition-scope: all;
}

::view-transition-group(para-change) {
  animation-duration: 1s;
}
```

JavaScript

The script starts by grabbing references to the button and the `<div>` elements (our components).

```
const btn = document.querySelector("button");
const divs = document.querySelectorAll("div");
```

Next, we define a function called `updateDivs()`, which toggles the text content of each component's nested `<span>` element between two values, and also toggles the component's foreground and background colors between two values.

```
function updateDivs() {
  divs.forEach((div) => {
    if (div.firstElementChild.textContent === "I can change") {
      div.firstElementChild.textContent = "I have changed";
      div.style.color = "white";
      div.style.backgroundColor = "black";
    } else {
      div.firstElementChild.textContent = "I can change";
      div.style.color = "black";
      div.style.backgroundColor = "white";
    }
  });
}
```

Finally, we add a `click` event listener to the `<button>` element. When the button is clicked, we first check whether `startViewTransition()` exists on the `document` object — if it does not, we run `updateDivs()` and then `return` out of the function. This first part allows browsers that don't support view transitions to still update the DOM without error. Next, we run `updateDivs()` inside a `startViewTransition()` callback to trigger the view transition as the DOM updates.

```
btn.addEventListener("click", handleClick);

function handleClick(e) {
  if (!document.startViewTransition) {
    updateDivs();
    return;
  }
  document.startViewTransition(() => {
    updateDivs();
  });
}
```

Result

Click the "Update DOM" button to see the view transition. Now try the following:

1. Inspect one of the `<div>` elements.
2. In the Styles panel in your browser's developer tools, uncheck the `view-transition-scope: all;` declaration to disable it.

3. Now switch to the JavaScript Console.
4. Click the "Update DOM" button again.

You should see that the view transition animation is not applied when the DOM changes, and an `InvalidStateError` is reported in the console.

See also

- `view-transition-name`
- View Transition API
- Using the View Transition API guide
- Using element-scoped view transitions

visibility CSS property

The **`visibility`** CSS property shows or hides an element without changing the layout of a document. The property can also hide rows or columns in a `<table>`.

To both hide an element *and remove it from the document layout*, set the `display` property to `none` instead of using `visibility`.

Try it

```
visibility: visible;
```

```
visibility: hidden;
```

```
visibility: collapse;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">Hide me</div>
    <div>Item 2</div>
    <div>Item 3</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  padding: 0.75em;
  width: 80%;
  max-height: 300px;
  display: flex;
}

.example-container > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex: 1;
}

#example-element {
  background-color: rgb(255 0 200 / 0.2);
  border: 3px solid rebeccapurple;
}
```

Syntax

```
/* Keyword values */
visibility: visible;
visibility: hidden;
visibility: collapse;

/* Global values */
visibility: inherit;
visibility: initial;
visibility: revert;
visibility: revert-layer;
visibility: unset;
```

The `visibility` property is specified as one of the keyword values listed below.

Values

`visible`

> The element box is visible.

`hidden`

> The element box is invisible (not drawn), but still affects layout as normal. Descendants of the element will be visible if they have `visibility` set to `visible`. The element cannot receive focus (such as when navigating through tab indexes).

`collapse`

> The `collapse` keyword has different effects for different elements:
>
> - For `<table>` rows, columns, column groups, and row groups, the row(s) or column(s) are hidden and the space they would have occupied is removed (as if `display: none` were applied to the column/row of the table). However, the size of other rows and columns is still calculated as though the cells in the collapsed row(s) or column(s) are present. This value allows for the fast removal of a row or column from a table without forcing the recalculation of widths and heights for the entire table.
> - Collapsed flex items and ruby annotations are hidden, and the space they would have occupied is removed.
> - For other elements, `collapse` is treated the same as `hidden`.

Accessibility

Using a `visibility` value of `hidden` on an element will remove it from the accessibility tree. This will cause the element and all its descendant elements to no longer be announced by screen reading technology.

Interpolation

When animated, visibility values are interpolated between *visible* and *not-visible*. One of the start or ending values must therefore be `visible` or no interpolation can happen. The value is interpolated as a discrete step, where values of the easing function between `0` and `1` map to `visible` and other values of the easing function (which occur only at the start/end of the transition or as a result of `cubic-bezier()` functions with y values outside of [0, 1]) map to the closer endpoint.

Notes

- Support for `visibility: collapse` is missing or partially incorrect in some modern browsers. It may not be correctly treated like `visibility: hidden` on elements other than table rows and columns.
- When applied to table rows, if the table contains cells (`<td>` and `<th>` elements) that span both visible and collapsed rows, the cell may render in unexpected ways. If the spanning cell is defined in a collapsed row, browsers do not render the table cell, as if the cells in subsequent rows were present with `visibility: collapse` applied. When the cell is defined in a visible row and spans a collapsed row, the cell contents are not reflowed, but the presentation of the cell itself varies by browser. Most browsers reduce the block size of the cell by the block size of the hidden row. This means the contents may be larger than the cell in the block-size direction. Depending on the browser, the overflowing contents are either cropped, as if `overflow: hidden` were set, while the content bleeds into the subsequent row in other browsers as if `overflow: visible` were set. In other browsers, the cell is rendered as if the row were not collapsed, with all the other cells in the row hidden as if `visibility: collapse` were set on individual cells rather than the row itself.
- `visibility: collapse` may change the layout of a table if the table has nested tables within the cells that are collapsed, unless `visibility: visible` is specified explicitly on nested tables.

Formal definition

Initial value	`visible`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
visibility =
  visible       |
  hidden        |
  force-hidden  |
  collapse
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

HTML

```
<p class="visible">The first paragraph is visible.</p>
<p class="not-visible">The second paragraph is NOT visible.</p>
<p class="visible">
  The third paragraph is visible. Notice the second paragraph is still occupying
  space.
</p>
```

CSS

```
.visible {
  visibility: visible;
}

.not-visible {
  visibility: hidden;
}
```

Table example

HTML

```
<table>
  <tbody>
    <tr>
      <td>1.1</td>
      <td class="collapse">1.2</td>
      <td>1.3</td>
    </tr>
    <tr class="collapse">
      <td>2.1</td>
      <td>2.2</td>
      <td>2.3</td>
    </tr>
    <tr>
      <td>3.1</td>
      <td>3.2</td>
      <td>3.3</td>
    </tr>
  </tbody>
</table>
```

CSS

```
.collapse {
  visibility: collapse;
}

table {
  border: 1px solid red;
}

td {
  border: 1px solid gray;
}
```

See also

- `display`
- SVG `visibility` attribute

white-space CSS property

The **`white-space`** CSS shorthand property sets how white space inside an element is handled.

Try it

```
white-space: normal;
```

```
white-space: pre;
```

```
white-space: pre-wrap;
```

```
white-space: pre-line;
```

```
white-space: wrap;
```

```
white-space: collapse;
```

```
white-space: preserve nowrap;
```

```
<section class="default-example" id="default-example">
  <div id="example-element">
    <p>
      But ere she from the church-door stepped She smiled and told us why: 'It
      was a wicked woman's curse,' Quoth she, 'and what care I?' She smiled, and
      smiled, and passed it off Ere from the door she stept—
    </p>
  </div>
</section>
```

```
#example-element {
  width: 16rem;
}

#example-element p {
  border: 1px solid #c5c5c5;
  padding: 0.75rem;
  text-align: left;
}
```

The property specifies two things:

- Whether and how white space is collapsed.
- Whether and how lines wrap.

> **Note:** To make words break *within themselves*, use `overflow-wrap`, `word-break`, or `hyphens` instead.

Constituent properties

This property is a shorthand for the following CSS properties:

- `white-space-collapse`
- `text-wrap-mode`

> **Note:** The spec defines a third constituent property: `white-space-trim`, which is not implemented in any browser yet.

Syntax

```
/* Single keyword values */
white-space: normal;
white-space: pre;
white-space: pre-wrap;
white-space: pre-line;

/* white-space-collapse and text-wrap-mode shorthand values */
white-space: nowrap;
white-space: wrap;
white-space: break-spaces;
white-space: collapse;
white-space: preserve nowrap;

/* Global values */
white-space: inherit;
white-space: initial;
white-space: revert;
white-space: revert-layer;
white-space: unset;
```

Values

The `white-space` property values can be specified as one or two keywords representing the values for the `white-space-collapse` and `text-wrap-mode` properties, or the following special keywords:

`normal`

Sequences of white space are collapsed. Newline characters in the source are handled the same as other white spaces. Lines are broken as necessary to fill line boxes. Equivalent to `collapse wrap`.

`pre`

Sequences of white space are preserved. Lines are only broken at newline characters in the source and at `<br>` elements. Equivalent to `preserve nowrap`.

`pre-wrap`

Sequences of white space are preserved. Lines are broken at newline characters, at `<br>`, and as necessary to fill line boxes. Equivalent to `preserve wrap`.

`pre-line`

Sequences of white space are collapsed. Lines are broken at newline characters, at `<br>`, and as necessary to fill line boxes. Equivalent to `preserve-breaks wrap`.

> **Note:** The `white-space` property as a shorthand is a relatively new feature (see browser compatibility). Originally, it had six keyword values; now, the value `nowrap` is instead interpreted as a value for `text-wrap-mode`, while the value `break-spaces` is interpreted as a value for `white-space-collapse`. The above four keywords are still specific to `white-space`, but they have longhand equivalents. The change to make `white-space` a shorthand expands acceptable values to even more keywords and combinations, such as `wrap` and `collapse`.

The following table summarizes the behavior of these four `white-space` keyword values:

	New lines	Spaces and tabs	Text wrapping	End-of-line spaces	End-of-line other space separators
`normal`	Collapse	Collapse	Wrap	Remove	Hang
`pre`	Preserve	Preserve	No wrap	Preserve	No wrap
`pre-wrap`	Preserve	Preserve	Wrap	Hang	Hang
`pre-line`	Preserve	Collapse	Wrap	Remove	Hang

A tab defaults to 8 spaces and can be configured using the `tab-size` property. In the case of `normal`, `nowrap`, and `pre-line` values, every tab is converted to a space (U+0020) character.

> **Note:** There is a distinction made between **spaces** and **other space separators**. These are defined as follows:
>
> spaces
>
> Spaces (U+0020), tabs (U+0009), and segment breaks (such as newlines).
>
> other space separators
>
> All other space separators defined in Unicode, other than those already defined as spaces.
>
> Where white space is said to *hang*, this can affect the size of the box when measured for intrinsic sizing.

Formal definition

Initial value	`normal`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
white-space =
  normal                                          |
  pre                                             |
  pre-wrap                                        |
  pre-line                                        |
  <'white-space-collapse'> || <'text-wrap-mode'> || <'white-space-trim'>

<white-space-collapse> =
  collapse          |
  discard           |
  preserve          |
  preserve-breaks   |
  preserve-spaces   |
  break-spaces

<text-wrap-mode> =
  wrap    |
  nowrap

<white-space-trim> =
  none                                          |
  discard-before || discard-after || discard-inner
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

```
code {
  white-space: pre;
}
```

Line breaks inside <pre> elements

```
pre {
  white-space: pre-wrap;
}
```

In action

```
<div id="css-code" class="box">
  p { white-space:
  <select>
    <option>normal</option>
    <option>nowrap</option>
    <option>pre</option>
```

```
    <option>pre-wrap</option>
    <option>pre-line</option>
    <option>break-spaces</option>
  </select>
  }
</div>
<div id="results" class="box">
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
    quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
    consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
    cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat
    non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
  </p>
</div>
```

```
.box {
  width: 350px;
  padding: 16px;
}

#css-code {
  background-color: gainsboro;
  font-size: 16px;
  font-family: monospace;
}

#css-code select {
  font-family: inherit;
  width: 100px;
}

#results {
  background-color: rgb(230 230 230);
  overflow-x: scroll;
  white-space: normal;
  font-size: 14px;
}
```

```
const select = document.querySelector("#css-code select");
const results = document.querySelector("#results p");
select.addEventListener("change", (e) => {
  results.style.setProperty("white-space", e.target.value);
});
```

Controlling line wrapping in tables

HTML

```
<table>
  <tbody>
    <tr>
      <td></td>
      <td>Very long content that splits</td>
      <td class="nw">Very long content that don't split</td>
    </tr>
    <tr>
      <td class="nw">white-space:</td>
      <td>normal</td>
      <td>nowrap</td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  border-collapse: collapse;
  border: solid black 1px;
  width: 250px;
  height: 150px;
}
td {
  border: solid 1px black;
  text-align: center;
}
.nw {
  white-space: nowrap;
}
```

Result

Multiple lines in SVG text element

The `white-space` CSS property can be used to create multiple lines in a `<text>` element, which does not wrap by default.

HTML

The text inside the `<text>` element needs to be split into multiple lines for the new lines to be detected. After the first line the rest need to have their whitespace removed.

```
<svg viewBox="0 0 320 150">
  <text y="20" x="10">Here is an English paragraph
that is broken into multiple lines
in the source code so that it can
be more easily read and edited
in a text editor.
  </text>
</svg>
```

CSS

```
text {
  white-space: break-spaces;
}
```

Result

See also

- Properties that define how words break *within themselves*: `overflow-wrap`, `word-break`, `hyphens`
- `tab-size`
- Handling whitespace in CSS

white-space-collapse CSS property

The **`white-space-collapse`** CSS property controls how white space inside an element is collapsed.

> **Note:** The `white-space-collapse` and `text-wrap-mode` properties can be declared together using the `white-space` shorthand property.

Syntax

```
/* Keyword values */
white-space-collapse: collapse;
white-space-collapse: preserve;
white-space-collapse: preserve-breaks;
white-space-collapse: preserve-spaces;
white-space-collapse: break-spaces;

/* Global values */
white-space-collapse: inherit;
white-space-collapse: initial;
white-space-collapse: revert;
white-space-collapse: revert-layer;
white-space-collapse: unset;
```

The `white-space-collapse` property is specified as a single keyword chosen from the list of values below.

Values

`collapse`

White space sequences are collapsed.

`preserve`

White space sequences and segment break characters are preserved.

`preserve-breaks`

White space sequences are collapsed, while segment break characters are preserved.

`preserve-spaces`

White space sequences are preserved, while tabs and segment break characters are converted to spaces.

`break-spaces`

The behavior is identical to `preserve`, except that:

- Any sequence of preserved white space always takes up space, including at the end of the line.
- A line-breaking opportunity exists after every preserved white space character, including between white space characters.
- Preserved spaces take up space and do not hang, thus affecting the box's intrinsic sizes (`min-content` size and `max-content` size).

> **Note:** *Segment break characters* are characters such as line feeds that cause text to break onto new lines.

> **Note:** The CSS text module defines a `discard` value for the `white-space-collapse` property to discard all white space in the element, however, this is not supported in any browsers.

Formal definition

Initial value	`collapse`
Applies to	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
white-space-collapse =
  collapse         |
  discard          |
  preserve         |
  preserve-breaks  |
  preserve-spaces  |
  break-spaces
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<h2 class="collapse">Default behavior;
  all   whitespace   is   collapsed
  in    the          heading       .</h2>

<h2 class="preserve">In this case
  all   whitespace   is   preserved
  in    the          heading       .</h2>

<h2 class="preserve-breaks">In this case only
  the   line breaks  are  preserved
  in    the          heading       .</h2>

<h2 class="preserve-spaces">In this case only
  the   spaces       are  preserved
  in    the          heading       .</h2>
```

CSS

```
.collapse {
  white-space-collapse: collapse;
}

.preserve {
  white-space-collapse: preserve;
}

.preserve-breaks {
  white-space-collapse: preserve-breaks;
}

.preserve-spaces {
  white-space-collapse: preserve-spaces;
}

h2 {
  font-size: 1.6rem;
  font-family: monospace;
  border-bottom: 1px dotted #cccccc;
}
```

Result

See also

- Shorthand for `white-space-collapse` and `text-wrap-mode`: The `white-space` property.
- CSS text module
- Handling whitespace in CSS

widows CSS property

The **`widows`** CSS property sets the minimum number of lines in a block container that must be shown at the *top* of a page, region, or column.

In typography, a *widow* is the last line of a paragraph that appears alone at the top of a page. (The paragraph is continued from a prior page.)

Syntax

```
/* <integer> values */
widows: 2;
widows: 3;

/* Global values */
widows: inherit;
widows: initial;
widows: revert;
widows: revert-layer;
widows: unset;
```

Values

This property is specified as one `<integer>`:

`<integer>`

The minimum number of lines that can stay by themselves at the top of a new fragment after a fragmentation break. The value must be positive.

Formal definition

Initial value	2
Applies to	block container elements
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Formal syntax

```
widows =
  <integer [1,∞]>

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Fragmentation Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Controlling column widows

This example uses a checkbox to switch `widows` between `2` and `1`. With `widows: 2`, at least two lines of a paragraph must appear at the top of a column. With `widows: 1`, the paragraph can be broken anywhere.

HTML

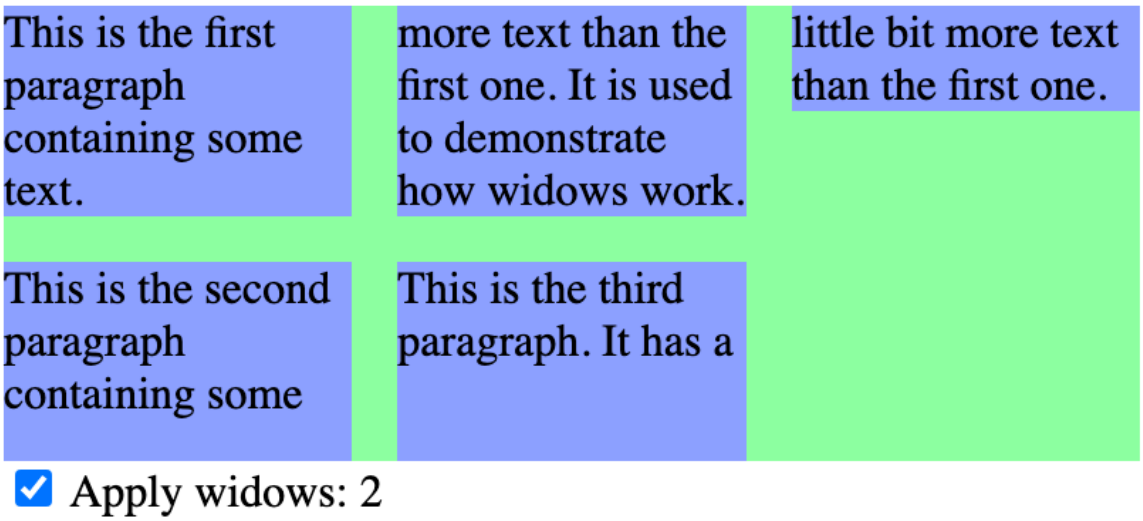
```
<div>
  <p>This is the first paragraph containing some text.</p>
  <p>
    This is the second paragraph containing some more text than the first one.
    It is used to demonstrate how widows work.
  </p>
  <p>
    This is the third paragraph. It has a little bit more text than the first
    one.
  </p>
</div>
<form>
  <input type="checkbox" id="widows" checked />
  <label for="widows">Apply widows: 2</label>
</form>
```

CSS

```
div {
  background-color: #8cffa0;
  width: 400px;
  height: 160px;
  columns: 3;
  column-fill: auto;
  orphans: 1;
  widows: 1;
}

div:has(~ form input:checked) {
  widows: 2;
}

p {
  background-color: #8ca0ff;
}

p:first-child {
  margin-top: 0;
}
```

Result

In Firefox, which does not support `widows`, the behavior in this example is essentially equivalent to `widows: 1`, regardless of the checkbox state. The following screenshot shows the result with `widows: 2` in a supporting browser:



See also

- `orphans`
- Paged media

width CSS property

The **`width`** CSS property sets an element's width. By default, it sets the width of the content area, but if `box-sizing` is set to `border-box`, it sets the width of the border area.

Try it

```
width: 150px;
```

```
width: 20em;
```

```
width: 75%;
```

```
width: auto;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the width.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: white;
}
```

The specified value of `width` applies to the content area so long as its value remains within the values defined by `min-width` and `max-width`.

- If the value for `width` is less than the value for `min-width`, then `min-width` overrides `width`.
- If the value for `width` is greater than the value for `max-width`, then `max-width` overrides `width`.

> **Note:** As a geometric property, `width` also applies to the `<svg>`, `<rect>`, `<image>`, and `<foreignObject>` SVG elements, with `auto` resolving to `100%` for `<svg>` and `0` for other elements, and percent values being relative to the SVG viewport width for `<rect>`. The CSS `width` property value overrides any SVG `width` attribute value set on the SVG element.

Syntax

```
/* <length> values */
width: 300px;
width: 25em;

/* <percentage> value */
width: 75%;

/* Keyword values */
width: max-content;
width: min-content;
width: fit-content;
width: auto;
width: stretch;

/* function values */
width: fit-content(20em);
width: anchor-size(width);
width: anchor-size(--my-anchor inline, 120%);
width: calc-size(max-content, size / 2);

/* Global values */
width: inherit;
width: initial;
width: revert;
width: revert-layer;
width: unset;
```

Values

`<length>`

Defines the width as a distance value.

`<percentage>`

Defines the width as a percentage of the containing block's width.

`auto`

The browser will calculate and select a width for the specified element.

`max-content`

The intrinsic preferred width.

`min-content`

The intrinsic minimum width.

`fit-content`

Use the available space, but not more than max-content, i.e., `min(max-content, max(min-content, stretch))`.

`anchor-size()`

Sets the width relative to a dimension of an anchor element. When defining the `width` of an anchor-positioned element, the first parameter defaults to the associated anchor's width. If applied to a non-anchor-positioned element, it sets the width to the fallback value. If no fallback is defined, the declaration is ignored.

`calc-size()`

Sets the width to a modified intrinsic size.

`fit-content(<length-percentage>)`

Uses the fit-content formula with the available space replaced by the specified argument, clamping the width according to the formula `min(maximum size, max(minimum size, <length-percentage>))`.

`stretch`

Sets the width of the element's margin box to the width of its containing block. It attempts to make the margin box fill the available space in the containing block, so in a way behaving similar to `100%` but applying the resulting size to the margin box rather than the box determined by box-sizing.

Accessibility

Ensure that elements set with a `width` aren't truncated and/or don't obscure other content when the page is zoomed to increase text size.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4 | W3C Understanding WCAG 2.0

Formal definition

Initial value	`auto`
Applies to	all elements but non-replaced inline elements, table rows, and row groups
Inherited	no
Percentages	refer to the width of the containing block
Computed value	a percentage or `auto` or the absolute length
Animation type	a length, percentage or calc();

Formal syntax

```
width =
  auto            |
  <box-size>      |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>               |
  stretch                                 |
  contain                                 |
  min-content                             |
  max-content                             |
  fit-content                             |
  stretch( <length-percentage [0,∞]> )    |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>     |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>       |
  <dimension>    |
  <percentage>   |
  <calc-keyword> |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Default width

This example demonstrates basic usage and the default `auto` value.

HTML

We include two paragraphs; one with a class name.

```
<p>The MDN community writes really great documentation.</p>
<p class="has-width">The MDN community writes really great documentation.</p>
```

CSS

We make all paragraphs have a gold background, explicitly setting the second paragraph's `width` to `auto`.

```
p {
  background: gold;
}
p.has-width {
  width: auto;
}
```

Results

As `width` defaults to `auto`, both paragraphs are the same width.

Using length units

This example demonstrates length width values.

HTML

We include two `<div>` elements with some text.

```
<div class="px_length">Width measured in px</div>
<div class="em_length">Width measured in em</div>
```

CSS

The `px_length` element is set to `200px` while the `em_length` element is set to be `20em` wide. Both elements also have different `background-color`, `color`, and `border` values to enable differentiating between the two when rendered.

```
.px_length {
  width: 200px;

  background-color: red;
  color: white;
  border: 1px solid black;
}

.em_length {
  width: 20em;

  background-color: white;
  color: red;
  border: 1px solid black;
}
```

Results

The `px_length` element will always be 200px wide. The rendered width of the `em_length` element depends on the font size.

Using percentages

This example demonstrates using percentage values.

HTML

We include one `<div>` element with some text.

```
<div class="percent">Width in percentage</div>
```

CSS

We set the `width` of the element to be `20%` of the width of its parent container.

```
.percent {
  width: 20%;

  background-color: silver;
  border: 1px solid red;
}
```

Results

Using intrinsic sizes

This example compares `max-content` and `min-content`, and introduces `calc-size`.

HTML

We include three paragraphs with the same content; just their class names differ.

```
<p class="max-green">The MDN community writes really great documentation.</p>
<p class="min-blue">The MDN community writes really great documentation.</p>
<p class="min-pink">The MDN community writes really great documentation.</p>
```

CSS

We set one paragraph's `width` to `max-content`, the second to `min-content`, and the third to be twice the size of the `min-content` by using the `calc-size()` function. Each is given a different `background-color` and `border-style` to enable differentiating between the two.

```
p.max-green {
  width: max-content;

  background-color: lightgreen;
  border-style: dotted;
}

p.min-blue {
```

```
  width: min-content;

  background-color: lightblue;
  border-style: dashed;
}

p.min-pink {
  width: calc-size(min-content, size * 2);

  background-color: pink;
  border-style: solid;
}
```

```
@supports not (width: calc-size(min-content, size * 2)) {
  body::before {
    content: "Your browser doesn't support the calc-size() function yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Results

The `max-content` example is as wide as the text. The `min-content` example is as wide as the widest word. The `calc-size()` example is set to be twice as wide as the `min-content`.

Using the stretch keyword

This example demonstrates the `stretch` value within a flex container.

HTML

We include a parent container with two child elements.

```
<div class="parent">
  <div class="child">text</div>
  <div class="child stretch">stretch</div>
</div>
```

CSS

We use the `display` property to make the parent a flex container, and set the second child's `width` the `stretch`.

```
.parent {
  border: solid;
  margin: 1rem;
  display: flex;
}

.child {
  background: #00999999;
  margin: 1rem;
}

.stretch {
  width: stretch;
}
```

```
@supports not (width: stretch) {
  body::before {
    content: "Your browser doesn't support the stretch value yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

By default, flex items are as wide as their content. The `stretch` value makes the element as wide as the available space allows, with the element's margin box otherwise clamped to the width of its containing block.

Using the anchor-size() function

This example demonstrates using the `anchor-size()` function to define the width of an anchor-positioned element; we defined its width as a multiple of its anchor's height.

HTML

We specify two `<div>` elements: one `anchor` element and one `infobox` element that we'll position relative to the anchor.

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>Infobox.</p>
</div>
```

CSS

We declare the `anchor` `<div>` as an anchor element by giving it an `anchor-name`. The positioned element has its `position` property set to `absolute`, and is associated with the anchor element via its `position-anchor` property. We also set absolute `height` and `width` dimensions on the anchor and define the width of the anchor-positioned element to be the width of the anchor using the `anchor-size()` function as the value of the `width` property. As a bonus, we also use the `anchor-size()` function to define the infobox's `left` position, making the gap between the anchor and the infobox one quarter of the height of the anchor.

```
.anchor {
  anchor-name: --my-anchor;
  width: 120px;
  height: 60px;
```

```
  font-size: 2rem;
  background-color: lightpink;
  text-align: center;
  align-content: center;
  outline: 1px solid black;
}

.infobox {
  position-anchor: --my-anchor;
  position: absolute;
  position-area: right;
  width: anchor-size(width);

  left: calc( anchor-size(height) / 4 )

  align-content: center;
  color: darkblue;
  background-color: azure;
  outline: 1px solid #dddddd;
}
```

```
body {
  padding: 5em;
}

@supports not (width: anchor-size(width)) {
  body::before {
    content: "Your browser doesn't support the anchor-size() function value.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Results

Note how the infobox's width is always the same as the anchor element's width.

See also

- `height`
- `box-sizing`
- `min-width`, `max-width`
- `block-size`, `inline-size`
- SVG `width` attribute
- Introduction to the CSS box model guide
- CSS box model module
- CSS anchor positioning module
- CSS values and units module

will-change CSS property

The **`will-change`** CSS property enables optimizing animations by providing a rendering hint to a browser about how an element is expected to change.

Syntax

```
/* Keyword values */
will-change: auto;
will-change: scroll-position;
will-change: contents;

/* <custom-ident> values */
will-change: transform;
will-change: opacity;

/* multiple values */
will-change: left, top;

/* Global values */
will-change: inherit;
will-change: initial;
will-change: revert;
will-change: revert-layer;
will-change: unset;
```

Values

The value is either `auto` or one or more comma separated `<animatable-feature>` values:

`auto`

Indicates that the browser applies whatever heuristics and optimizations it normally does. This is the default value.

`<animatable-feature>`

Represents one of the following:

`scroll-position`

Specifies that the scroll position of the element is expected to change in the near future, indicating the browser can optimize the rendering of overflowing content.

`contents`

Specifies that the element's contents, including all the elements in its subtree, are expected to change in the near future, indicating the browser can cache the element less aggressively.

`<custom-ident>`

Specifies the name of a CSS property, as an `<ident>`, whose value is animated or otherwise expected to change in the near future. If the specified `<ident>` represents a property shorthand, all of its longhand components are applied. The value cannot be `will-change`, `none`, `all`, `auto`, `scroll-position`, or `contents`.

Description

The `will-change` property provides a rendering hint to the browser indicating which properties are expected to be animated or otherwise changed. This enables browsers to create the necessary rendering optimizations to enable smoother changes and avoid jank.

The `will-change` property is aimed at improving rendering performance. This property can improve performance for elements with frequent repaints and reflows or for those that use properties such as `box-shadow` and `clip-path` to create complex visual effects.

Applying the property to an element applies the value to the element's entire subtree, indicating that any of the descendants can change. For this reason, applying a non-`auto` value on a large section, such as the `<body>`, can actually be bad for a page's performance. Instead, limit the use of this property to deeply nested elements, containing as little of the document as possible.

> **Warning:** Use the `will-change` property as a last resort to try to deal with existing performance problems. Don't use it to anticipate performance problems.

Proper usage of this property can be a bit tricky. Keep the following guidelines in mind:

- **Don't apply `will-change` to too many elements**: The browser already tries as hard as it can to optimize everything. Some of the stronger optimizations that are likely to be tied to `will-change` end up using a lot of a machine's resources. Overusing the property can cause the page to slow down instead of improving its performance.
- **Use sparingly**: The browser's normal behavior for optimizations is to remove the optimizations as soon as it can and revert to normal state. But adding `will-change` directly to a stylesheet implies that the targeted elements are always a few moments away from changing and the browser will keep the optimizations for a much longer time than it would have otherwise. So it is a good practice to switch `will-change` on and off using script code before and after the change occurs.
- **Don't apply will-change to elements to perform premature optimization**: If your page is performing well, don't add the `will-change` property to elements just to wring out a little more speed. `will-change` is intended to be used as a last resort to try to deal with existing performance problems. It should not be used to anticipate performance problems. Excessive use of `will-change` will result in excessive memory use and will cause more complex rendering to occur as the browser attempts to prepare for the possible change. This will lead to worse performance.
- **Give it sufficient time to work**: This property enables you to signal to the browser about properties that are likely to change, so the browser can choose to apply any ahead-of-time optimizations before the property change occurs. For this reason, you should give the browser some time to do the optimizations. Predict slightly ahead of time that something will change, and then set `will-change`.
- **Be aware that `will-change` may influence the visual appearance of elements**: When used with property values that create a stacking context (e.g., `will-change: opacity`), the stacking context is created up front.

Animations

If applying `will-change` to improve animations, add the property before the animation starts, not within the `@keyframes` animation definitions. Animated properties are treated as if they're already included in a `will-change`, so there's no reason to add them there.

Via stylesheet

It may be appropriate to include `will-change` in your style sheet for an application that does page flips on key presses like an album or a slide deck presentation where the pages are large and complex. This will let browser prepare the transition ahead of time and allow for snappy transitions between the pages as soon as the key is pressed. But use caution with the `will-change` property directly in stylesheets. It may cause the browser to keep the optimization in memory for much longer than it is needed.

```
.slide {
  will-change: transform;
}
```

Formal definition

Initial value	`auto`
Applies to	all elements
Inherited	no
Computed value	as specified
Animation type	discrete

Formal syntax

```
will-change =
  auto                  |
  <animateable-feature>#

<animateable-feature> =
  scroll-position  |
  contents         |
  <custom-ident>
```

This syntax reflects the latest standard as per CSS Will Change Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

This example demonstrates the basic CSS application of the `will-change` property.

CSS

We use CSS to apply the `will-change` property to the `#element`, providing a hint to the browser that the `transform` and `opacity` property values will be animated or otherwise change in the near future.

```
#element {
  will-change: transform, opacity;
}
```

Via script

This example shows how to apply the `will-change` property when needed and remove optimizations when done using JavaScript, which is generally how `will-change` should be applied.

JavaScript

We use JavaScript to add the `will-change` property to `#element` when the element is hovered over by using the `mouseenter` event. Setting `will-change` to `transform, opacity` tells the browser to optimize for changes to the `transform` and `opacity` properties. When the `animationend` event occurs, our script sets the value to `auto`.

```
const el = document.getElementById("element");

el.addEventListener("mouseenter", hintBrowser);
el.addEventListener("animationEnd", removeHint);

function hintBrowser() {
```

```
  this.style.willChange = "transform, opacity";
}

function removeHint() {
  this.style.willChange = "auto";
}
```

See also

- `transform`
- Individual transform properties: `translate`, `scale`, `rotate`
- `animation`
- CSS will change module

word-break CSS property

The **`word-break`** CSS property sets whether line breaks appear wherever the text would otherwise overflow its content box.

Try it

```
word-break: normal;
```

```
word-break: break-all;
```

```
word-break: keep-all;
```

```
word-break: break-word;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    Honorificabilitudinitatibus califragilisticexpialidocious
    Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
    グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
  </div>
</section>
```

```
#example-element {
  width: 80%;
  padding: 20px;
  text-align: start;
  border: solid 1px darkgray;
}
```

Syntax

```
/* Keyword values */
word-break: normal;
word-break: break-all;
word-break: keep-all;
word-break: auto-phrase; /* experimental */
word-break: break-word; /* deprecated */

/* Global values */
word-break: inherit;
word-break: initial;
word-break: revert;
word-break: revert-layer;
word-break: unset;
```

The `word-break` property is specified as a single keyword chosen from the list of values below.

Values

`normal`

Use the default line break rule.

`break-all`

To prevent overflow, word breaks should be inserted between any two characters (excluding Chinese/Japanese/Korean text).

`keep-all`

Word breaks should not be used for Chinese/Japanese/Korean (CJK) text. Non-CJK text behavior is the same as for `normal`.

`auto-phrase`

Has the same effect as `word-break: normal` except that language-specific analysis is performed to improve word breaks by not placing them in the middle of natural phrases.

`break-word`

Has the same effect as `overflow-wrap: anywhere` combined with `word-break: normal`, regardless of the actual value of the `overflow-wrap` property.

> **Note:** In contrast to `word-break: break-word` and `overflow-wrap: break-word` (see `overflow-wrap`), `word-break: break-all` will create a break at the exact place where text would otherwise overflow its container (even if putting an entire word on its own line would negate the need for a break).

The specification also lists an additional value, `manual`, which is not currently supported in any browsers. When implemented, `manual` will have the same effect as `word-break: normal` except that breaks won't be automatically inserted in Southeast Asian languages. This is needed because, in such languages, user agents frequently place breaks in suboptimal positions. `manual` will allow you to insert line breaks in optimal positions manually.

Formal definition

Initial value	`normal`
Applies to	all elements

Inherited	yes
Computed value	as specified
Animation type	discrete

Formal syntax

```
word-break =
  normal      |
  break-all   |
  keep-all    |
  manual      |
  auto-phrase |
  break-word
```

This syntax reflects the latest standard as per CSS Text Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p>1. <code>word-break: normal</code></p>
<p class="normal narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
</p>

<p>2. <code>word-break: break-all</code></p>
<p class="breakAll narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
</p>

<p>3. <code>word-break: keep-all</code></p>
<p class="keepAll narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
</p>

<p>4. <code>word-break: manual</code></p>
<p class="manual narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
</p>

<p>5. <code>word-break: auto-phrase</code></p>
<p class="autoPhrase narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
</p>

<p>6. <code>word-break: break-word</code></p>
<p class="breakWord narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
</p>
```

CSS

```
.narrow {
  padding: 10px;
  border: 1px solid;
  width: 500px;
  margin: 0 auto;
  font-size: 20px;
  line-height: 1.5;
  letter-spacing: 1px;
}

.normal {
  word-break: normal;
}

.breakAll {
  word-break: break-all;
}

.keepAll {
  word-break: keep-all;
}

.manual {
  word-break: manual;
}

.autoPhrase {
  word-break: auto-phrase;
}

.breakWord {
  word-break: break-word;
}
```

See also

- `overflow-wrap`

- white-space
- hyphens
- line-break
- Guide to wrapping and breaking text

word-spacing CSS property

The **word-spacing** CSS property sets the spacing between words and between tags.

Try it

```
word-spacing: normal;
```

```
word-spacing: 1rem;
```

```
word-spacing: 4px;
```

```
word-spacing: 50%;
```

```
word-spacing: -0.4ch;
```

```
<section id="default-example">
  <p id="example-element">
    As much mud in the streets as if the waters had but newly retired from the
    face of the earth, and it would not be wonderful to meet a Megalosaurus,
    forty feet long or so, waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.woff2");
  font-family: "Amstelvar";
  font-style: normal;
}

section {
  font-size: 1.2em;
  font-family: "Amstelvar", serif;
}
```

Syntax

```
/* Keyword value */
word-spacing: normal;

/* <length> values */
word-spacing: 3px;
word-spacing: 0.3em;
word-spacing: 65%;
word-spacing: -1ex;

/* Global values */
word-spacing: inherit;
word-spacing: initial;
word-spacing: revert;
word-spacing: revert-layer;
word-spacing: unset;
```

Values

This property is specified as the keyword `normal` or a `<length-percentage>`:

`normal`

The normal inter-word spacing, as defined by the current font and/or the browser.

`<length-percentage>`

Specifies extra spacing in addition to the intrinsic inter-word spacing defined by the font. Percentage values are calculated relative to the `font-size` of the text.

Accessibility

A large positive or negative `word-spacing` value will make the sentences the styling is applied to unreadable. For text styled with a very large positive value, the words will be so far apart that it will no longer appear to be a sentence. For text styled with a large negative value, the words can overlap each other to the point where the beginning and end of each word is unrecognizable.

Legible `word-spacing` must be determined on a case-by-case basis, as different font families have different character widths. There is no one value that can ensure all font families automatically maintain their legibility.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.8 | W3C Understanding WCAG 2.0

Examples

Basic usage

This example demonstrates basic usage of `word-spacing`.

HTML

Our HTML contains two paragraphs of text:

```
<p id="mozdiv1">Lorem ipsum dolor sit amet.</p>
<p id="mozdiv2">Lorem ipsum dolor sit amet.</p>
```

CSS

Our CSS applies a different `word-spacing` to each paragraph:

```
#mozdiv1 {
  word-spacing: 15px;
}

#mozdiv2 {
  word-spacing: 5em;
}
```

Result

The example renders like so:

Comparing word-spacing set with length and percentage

This example demonstrates that percentage `word-spacing` values are useful for responsive text sizing.

The code displays several paragraphs that have the same `word-spacing` set on text with increasing font size. We provide functionality to switch between a length and a percentage `word-spacing` value, so that you can observe the responsive qualities of using a percentage value.

HTML

The HTML contains several `<p>` elements containing text content, and an `<input type="checkbox">` that we'll use to toggle between a length `word-spacing` and a percentage `word-spacing` value.

```
<p class="x-small">X-small font-size (0.8em)</p>
<p class="small">Small font-size (1.3em)</p>
<p class="medium">Medium font-size (2em)</p>
<p class="large">Large font-size (3em)</p>
<p class="x-large">X-Large (3.5em)</p>

<form>
  <label for="ls-toggle">
    Toggle <code>word-spacing</code> (off: <code>10px</code>, on:
    <code>15%</code>)
  </label>
  <input type="checkbox" id="ls-toggle" />
</form>
```

CSS

Our CSS starts by applying increasing `font-size` values to each successive paragraph:

```
html {
  font-family: "Arial", sans-serif;
}
```

```
.x-small {
  font-size: 0.8em;
}

.small {
  font-size: 1.3em;
}

.medium {
  font-size: 2em;
}

.large {
  font-size: 3em;
}

.x-large {
  font-size: 3.5em;
}
```

We apply a `word-spacing` value of `10px` to all paragraphs by default. When the checkbox is checked, however, we change the `word-spacing` value to `15%`:

```
p {
  word-spacing: 10px;
}

p:has(~ form > input:checked) {
  word-spacing: 15%;
}
```

Result

The rendered result looks like this:

First, note how the initial length letter spacing value looks OK when applied to the larger font sizes, but it doesn't look good on the smaller font sizes. Now toggle the checkbox, and note how the percentage letter spacing looks appropriate on all lines, as it scales with the font size.

Formal definition

Initial value	`normal`
Applies to	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Percentages	refer to the width of the affected glyph
Computed value	absolute `<length>`
Animation type	a length

Formal syntax

```
word-spacing =
  normal                |
  <length-percentage>
```

```
<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Text Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- `letter-spacing`
- SVG `word-spacing` attribute

writing-mode CSS property

The **`writing-mode`** CSS property sets whether lines of text are laid out horizontally or vertically, as well as the direction in which text flows.

Try it

```
writing-mode: horizontal-tb;
```

```
writing-mode: vertical-lr;
```

```
writing-mode: vertical-rl;
```

```
writing-mode: sideways-rl;
```

```
writing-mode: sideways-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  padding: 0.75em;
  width: 80%;
  max-height: 300px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  margin: 10px;
}
```

Syntax

```
/* Keyword values */
writing-mode: horizontal-tb;
writing-mode: vertical-rl;
writing-mode: vertical-lr;
writing-mode: sideways-rl;
writing-mode: sideways-lr;

/* Global values */
writing-mode: inherit;
writing-mode: initial;
writing-mode: revert;
writing-mode: revert-layer;
writing-mode: unset;
```

Values

This property is specified as one of the following keywords:

`horizontal-tb`

> For `ltr` scripts, content flows horizontally from left to right. For `rtl` scripts, content flows horizontally from right to left. The next horizontal line is positioned below the previous line.

`vertical-rl`

> For `ltr` scripts, content flows vertically from top to bottom, and the next vertical line is positioned to the left of the previous line. For `rtl` scripts, content flows vertically from bottom to top, and the next vertical line is positioned to the left of the previous line.

`vertical-lr`

> For `ltr` scripts, content flows vertically from top to bottom, and the next vertical line is positioned to the right of the previous line. For `rtl` scripts, content flows vertically from bottom to top, and the next vertical line is positioned to the right of the previous line.

`sideways-rl`

> For `ltr` scripts, content flows vertically from top to bottom. For `rtl` scripts, content flows vertically from bottom to top. All the glyphs, even those in vertical scripts, are set sideways toward the right.

`sideways-lr`

> For `ltr` scripts, content flows vertically from bottom to top. For `rtl` scripts, content flows vertically from top to bottom. All the glyphs, even those in vertical scripts, are set sideways toward the left.

`lr`

> Deprecated except for SVG1 documents. For CSS, use `horizontal-tb` instead.

`lr-tb`

Deprecated except for SVG1 documents. For CSS, use `horizontal-tb` instead.

`rl`

Deprecated except for SVG1 documents. For CSS, use `horizontal-tb` instead.

`tb`

Deprecated except for SVG1 documents. For CSS, use `vertical-lr` instead.

`tb-lr`

Deprecated except for SVG1 documents. For CSS, use `vertical-lr` instead.

`tb-rl`

Deprecated except for SVG1 documents. For CSS, use `vertical-rl` instead.

Description

The `writing-mode` property sets whether lines of text are laid out horizontally or vertically. It specifies the *block flow direction*, which is the direction in which block-level containers are stacked, and the direction in which inline-level content flows within a block container. Thus, it also determines the ordering of block-level content.

The flow direction in horizontal scripts is also affected by the directionality of that script, either left-to-right (`ltr`, like English and most other languages) or right-to-left (`rtl`, like Hebrew or Arabic).

When set for an entire document, it should be set on the root element (`html` element for HTML documents).

Formal definition

Initial value	`horizontal-tb`
Applies to	all elements except table row groups, table column groups, table rows, and table columns
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Formal syntax

```
writing-mode =
  horizontal-tb  |
  vertical-rl    |
  vertical-lr    |
  sideways-rl    |
  sideways-lr
```

This syntax reflects the latest standard as per CSS Writing Modes Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using multiple writing modes

This example demonstrates all of the writing modes, showing each with text in various languages.

HTML

The HTML is a `<table>` with each writing mode in a row with a column showing text in various scripts using that writing mode.

```
<table>
  <caption>
    Using multiple writing modes
  </caption>
  <thead>
    <tr>
      <th>Value</th>
      <th>Vertical script</th>
      <th>Horizontal (LTR) script</th>
      <th>Horizontal (RTL) script</th>
      <th>Mixed script</th>
    </tr>
  </thead>
  <tbody>
    <tr class="text1">
      <th>horizontal-tb</th>
      <td>我家没有电脑。</td>
      <td>Example text</td>
      <td>חלל ארוך לדוגמא</td>
      <td>1994年に至っては</td>
    </tr>
    <tr class="text2">
      <th>vertical-lr</th>
      <td>我家没有电脑。</td>
      <td>Example text</td>
      <td>חלל ארוך לדוגמא</td>
      <td>1994年に至っては</td>
    </tr>
    <tr class="text3">
      <th>vertical-rl</th>
      <td>我家没有电脑。</td>
      <td>Example text</td>
      <td>חלל ארוך לדוגמא</td>
      <td>1994年に至っては</td>
    </tr>
    <tr class="experimental text4">
      <th>sideways-lr</th>
      <td>我家没有电脑。</td>
      <td>Example text</td>
      <td>חלל ארוך לדוגמא</td>
      <td>1994年に至っては</td>
    </tr>
    <tr class="experimental text5">
      <th>sideways-rl</th>
```

```
      <td>我家没有电脑。</td>
      <td>Example text</td>
      <td>מלל ארוך לדוגמא</td>
      <td>1994年に至っては</td>
    </tr>
  </tbody>
</table>
<p class="notice">
  Your browser does not support the <code>sideways-lr</code> or
  <code>sideways-rl</code> values.
</p>
```

CSS

```
table {
  border-collapse: collapse;
}
td,
th {
  border: 2px black solid;
  padding: 4px;
}
th {
  background-color: lightgray;
}

.experimental {
  display: none;
}

.notice {
  display: table-row;
  font-weight: bold;
  text-align: center;
}

@supports (writing-mode: sideways-lr) {
  .experimental {
    display: table-row;
  }
  .notice {
    display: none;
  }
}
```

The CSS that adjusts the directionality of the content looks like this:

```
.text1 td {
  writing-mode: horizontal-tb;
}

.text2 td {
  writing-mode: vertical-lr;
}

.text3 td {
  writing-mode: vertical-rl;
}

.text4 td {
  writing-mode: sideways-lr;
}

.text5 td {
  writing-mode: sideways-rl;
}
```

Result

Using writing-mode with transforms

If your browser doesn't support `sideways-lr`, a workaround is to use `transform` to achieve a similar effect depending on the script direction. The effect of `vertical-rl` is the same as with `sideways-lr`, so no transformation is required for left-to-right scripts. In some cases, rotating the text 180 degrees is sufficient to achieve the effect of `sideways-lr`, but font glyphs may not be designed to be rotated, so this may produce unexpected positioning or rendering.

HTML

```
<table>
  <caption>
    Using writing-mode with transforms
  </caption>
  <thead>
    <tr>
      <th>Vertical LR</th>
      <th>Vertical LR with transform</th>
      <th>Sideways LR</th>
      <th>Only rotate</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <span class="vertical-lr">我家没有电脑。</span>
        <span class="vertical-lr">Example text</span>
      </td>
      <td>
        <span class="vertical-lr rotated">我家没有电脑。</span>
        <span class="vertical-lr rotated">Example text</span>
      </td>
      <td>
        <span class="sideways-lr">我家没有电脑。</span>
```

```
        <span class="sideways-lr">Example text</span>
      </td>
      <td>
        <span class="only-rotate">我家没有电脑。</span>
        <span class="only-rotate">Example text</span>
      </td>
    </tr>
  </tbody>
</table>
```

CSS

```
.vertical-lr {
  writing-mode: vertical-lr;
}

.rotated {
  transform: rotate(180deg);
}

.sideways-lr {
  writing-mode: sideways-lr;
}

.only-rotate {
  inline-size: fit-content;
  transform: rotate(-90deg);
}
```

```
table {
  border-collapse: collapse;
}
td,
th {
  border: 2px black solid;
  padding: 4px;
}
th {
  background-color: lightgray;
}
span {
  display: inline-block;
  width: 1.5em;
  text-align: center;
}
```

Result

See also

- `direction`
- `unicode-bidi`
- `text-orientation`
- `text-combine-upright`
- CSS logical properties
- CSS writing modes module
- SVG `writing-mode` attribute
- Creating vertical form controls
- Handling different text directions
- Styling vertical text (Chinese, Japanese, Korean and Mongolian) on W3.org (2022)

x CSS property

The **x** CSS property defines the x-axis coordinate of the top left corner of the SVG `<rect>` shape, `<image>` image, `<foreignObject>` viewport or nested `<svg>` viewport relative to the nearest `<svg>` ancestor's user coordinate system. If present, it overrides the element's `x` attribute.

> **Note:** The `x` property only applies to `<rect>`, `<image>`, `<foreignObject>`, and `<svg>` elements nested in an `<svg>`. It has no effect on the outermost `<svg>` elements itself, and does not apply to other SVG elements nor to HTML elements or pseudo-elements.

Syntax

```
/* length and percentage values */
x: 40px;
x: 40%;

/* Global values */
x: inherit;
x: initial;
x: revert;
x: revert-layer;
x: unset;
```

Values

The `<length>` and `<percentage>` values denote the x-axis coordinate position of the top left corner of the SVG element container.

`<length>`

As an absolute or relative length, it can be expressed in any unit allowed by the CSS `<length>` data type.

`<percentage>`

Percentages refer to the width of the SVG `viewBox`, if declared, otherwise, the percentage refers to the width of the current SVG viewport.

Formal definition

Initial value	`0`
Applies to	`<svg>`, `<rect>`, `<image>`, and `<foreignObject>` elements in `<svg>`
Inherited	no
Percentages	refer to the width of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
x =
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the x-axis coordinates of SVG shapes

This example demonstrates the basic use case of `x`, and how the CSS `x` property takes precedence over the `x` attribute.

HTML

We include four identical SVG `<rect>` elements; their `x` and `y` attributes values are all `10`, meaning the four rectangles are all in the same location, `10px` from the top and left corner of the SVG viewport.

```
<svg xmlns="http://www.w3.org/2000/svg">
  <rect width="100" height="100" x="10" y="10" />
  <rect width="100" height="100" x="10" y="10" />
  <rect width="100" height="100" x="10" y="10" />
  <rect width="100" height="100" x="10" y="10" />
</svg>
```

CSS

We style all the rectangles to have a black border and be slightly transparent, so overlapping rectangles are visible. We provide each rectangle with different fill and `x` values.

```
svg {
  border: 1px solid;
  width: 300px;
}

rect {
  fill: none;
  stroke: black;
  opacity: 0.8;
}

rect:nth-of-type(2) {
  x: 3em;
  fill: red;
}

rect:nth-of-type(3) {
  x: 180px;
  fill: yellow;
}

rect:nth-of-type(4) {
  x: 50%;
  fill: orange;
}
```

Results

The left edges of the rectangles are at `10` (from the attribute), `3em`, `180px`, and `50%`, respectively. The SVG is `300px` wide, so the last rectangle's left side is at the `150px` mark.

See also

- SVG `x` attribute
- Geometry properties: `x`, `cx`, `cy`, `r`, `rx`, `ry`, `y`, `width`, `height`
- `fill`
- `stroke`
- `paint-order`
- `<basic-shape>` data type

y CSS property

The **`y`** CSS property defines the y-axis coordinate of the top left corner of the SVG `<rect>` shape, `<image>` image, `<foreignObject>` viewport and nested `<svg>` viewport relative to the nearest `<svg>` ancestor's user coordinate system. If present, it overrides the element's `y` attribute.

> **Note:** The `y` property only applies to `<rect>`, `<image>`, `<foreignObject>`, and `<svg>` elements nested in an `<svg>`. It has no effect on outermost `<svg>` elements and does not apply to other SVG elements nor to HTML elements or pseudo-elements.

Syntax

```
/* length and percentage values */
y: 10px;
y: 10%;
```

```
/* Global values */
y: inherit;
y: initial;
y: revert;
y: revert-layer;
y: unset;
```

Values

The `<length>` and `<percentage>` values denote the y-axis coordinate position of the top left corner of the SVG element.

`<length>`

As an absolute or relative length, it can be expressed in any unit allowed by the CSS `<length>` data type.

`<percentage>`

Percentages refer to the height of the SVG `viewBox`, if declared, otherwise, the percentage refers to the height of the current SVG viewport.

Formal definition

Initial value	`0`
Applies to	`<svg>`, `<rect>`, `<image>`, and `<foreignObject>` elements in `<svg>`
Inherited	no
Percentages	refer to the height of the current SVG viewport
Computed value	the percentage as specified or the absolute length
Animation type	by computed value type

Formal syntax

```
y =
  <length-percentage>

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Specification, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the y-axis coordinates of SVG shapes

This example demonstrates the basic use case of `y`, and how the CSS `y` property takes precedence over the `y` attribute.

HTML

We include four identical SVG `<rect>` elements; their `x` and `y` attributes values are all `10`, meaning the four rectangles are all in the same location, `10px` from the top and left corner of the SVG viewport.

```
<svg>
  <rect width="40" height="40" x="10" y="10" />
  <rect width="40" height="40" x="10" y="10" />
  <rect width="40" height="40" x="10" y="10" />
  <rect width="40" height="40" x="10" y="10" />
</svg>
```

CSS

We style all the rectangles to have a black border and be slightly transparent, so overlapping rectangles are visible. We provide the rectangle with different `fill` and `y` values.

```
svg {
  border: 1px solid;
}

rect {
  fill: none;
  stroke: black;
  opacity: 0.8;
}

rect:nth-of-type(2) {
  y: -20px;
  fill: red;
}

rect:nth-of-type(3) {
  y: 4em;
  fill: yellow;
}

rect:nth-of-type(4) {
  y: 60%;
  fill: orange;
}
```

Results

The top edges of the rectangles are at `10` (from the attribute), `-20px`, `4em`, and `60%`, respectively. The rectangle is `40px` tall, so the `-20px` places half the red rectangle outside the viewport. The SVG is `150px` tall, so the orange rectangle's top side is `90px` from the top of the SVG viewport.

See also

- SVG `y` attribute
- Geometry properties: `y`, `cx`, `cy`, `r`, `rx`, `ry`, `x`, `width`, `height`
- `fill`
- `stroke`
- `paint-order`

- `<basic-shape>` data type

z-index CSS property

The **`z-index`** CSS property sets the z-order of a positioned element and its descendants or flex and grid items. Overlapping elements with a larger z-index cover those with a smaller one.

Try it

```
z-index: auto;
```

```
z-index: 1;
```

```
z-index: 3;
```

```
z-index: 5;
```

```
z-index: 7;
```

```
<section class="default-example container" id="default-example">
  <div id="example-element">Change my z-index</div>
  <div class="block blue position1">z-index: 6</div>
  <div class="block blue position2">z-index: 4</div>
  <div class="block blue position3">z-index: 2</div>
  <div class="block red position4">z-index: auto</div>
  <div class="block red position5">z-index: auto</div>
  <div class="block red position6">z-index: auto</div>
</section>
```

```
#example-element {
  top: 15px;
  left: 15px;
  width: 180px;
  height: 230px;
  position: absolute;
  /* center the text so it is visible even when z-index is set to auto */
  line-height: 215px;
  font-family: monospace;
  background-color: #fcfbe5;
  border: solid 5px #e3e0a1;
  z-index: auto;
  color: black;
}

.container {
  display: inline-block;
  width: 250px;
  position: relative;
}

.block {
  width: 150px;
  height: 50px;
  position: absolute;
  font-family: monospace;
  color: black;
}

.blue {
  background-color: #e5e8fc;
  border: solid 5px #112382;
  /* move text to the bottom of the box */
  line-height: 55px;
}

.red {
  background-color: #fce5e7;
  border: solid 5px #e3a1a7;
}

.position1 {
  top: 0;
  left: 0;
  z-index: 6;
}

.position2 {
  top: 30px;
  left: 30px;
  z-index: 4;
}

.position3 {
  top: 60px;
  left: 60px;
  z-index: 2;
}

.position4 {
  top: 150px;
  left: 0;
  z-index: auto;
}

.position5 {
  top: 180px;
  left: 30px;
  z-index: auto;
}

.position6 {
```

```
  top: 210px;
  left: 60px;
  z-index: auto;
}
```

For a positioned box (that is, one with any `position` other than `static`), the `z-index` property specifies:

1. The stack level of the box in the current stacking context.
2. Whether the box establishes a local stacking context.

Syntax

```
/* Keyword value */
z-index: auto;

/* <integer> values */
z-index: 0;
z-index: 3;
z-index: 289;
z-index: -1; /* Negative values to lower the priority */

/* Global values */
z-index: inherit;
z-index: initial;
z-index: revert;
z-index: revert-layer;
z-index: unset;
```

Values

This property is specified as one of the following values:

`auto`

> The box does not establish a new local stacking context. The stack level of the generated box in the current stacking context is `0`.

`<integer>`

> This `<integer>` is the stack level of the generated box in the current stacking context. The box also establishes a local stacking context. This means that the z-indexes of descendants are not compared to the z-indexes of elements outside this element.

Formal definition

Initial value	`auto`
Applies to	positioned elements
Inherited	no
Computed value	as specified
Animation type	an integer
Creates stacking context	yes

Formal syntax

```
z-index =
  auto       |
  <integer>  |
  inherit

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, Cascading Style Sheets Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Visually layering elements

HTML

```
<div class="wrapper">
  <div class="dashed-box">Dashed box</div>
  <div class="gold-box">Gold box</div>
  <div class="green-box">Green box</div>
</div>
```

CSS

```
.wrapper {
  position: relative;
}

.dashed-box {
  position: relative;
  z-index: 1;
  border: dashed;
  height: 8em;
  margin-bottom: 1em;
  margin-top: 2em;
}
.gold-box {
  position: absolute;
  z-index: 3; /* put .gold-box above .green-box and .dashed-box */
  background: gold;
  width: 80%;
  left: 60px;
  top: 3em;
}
.green-box {
```

```
  position: absolute;
  z-index: 2; /* put .green-box above .dashed-box */
  background: lightgreen;
  width: 20%;
  left: 65%;
  top: -25px;
  height: 7em;
  opacity: 0.9;
}
```

Result

See also

- CSS `position` property
- Understanding CSS z-indexes

zoom CSS property

The **`zoom`** CSS property can be used to control the magnification level of an element. `transform: scale()` can be used as an alternative to this property.

The `zoom` CSS property scales the targeted element, which can affect the page layout. When scaling, the zoomed element scales from `top` and `center` when using the default `writing-mode`.

In contrast, an element scaled using `scale()` will not cause layout recalculation or move other elements on the page. If using `scale()` makes the contents larger than the containing element, then `overflow` comes into effect. Additionally, elements adjusted using `scale()` transform from the `center` by default; this can be changed with the `transform-origin` CSS property.

Syntax

```
/* <percentage> values */
zoom: 50%;
zoom: 200%;

/* <number> values */
zoom: 1.1;
zoom: 0.7;

/* Non-standard keyword values */
zoom: normal;
zoom: reset;

/* Global values */
zoom: inherit;
zoom: initial;
zoom: revert;
zoom: revert-layer;
zoom: unset;
```

Values

`<percentage>`

Zoom factor. `100%` is equivalent to `normal`. Values larger than `100%` zoom in. Values smaller than `100%` zoom out.

`<number>`

Zoom factor. Equivalent to the corresponding percentage (`1.0` = `100%` = `normal`). Values larger than `1.0` zoom in. Values smaller than `1.0` zoom out.

Two non-standard keyword values are not recommended. Check browser compatibility data:

`normal`

Render the element at its normal size; equal to `zoom: 1`. Use the global `unset` keyword value instead.

`reset`

Resets the value to `zoom: 1` and prevents the element from being (de)magnified if the user applies non-pinch-based zooming (e.g., by pressing `Ctrl` - - or `Ctrl` + + keyboard shortcuts) to the document.

Formal definition

Initial value	`1`
Applies to	all elements
Inherited	no
Percentages	Converted to `<number>`
Computed value	as specified, but with `<percentage>` converted to the equivalent `<number>`
Animation type	by computed value type

Formal syntax

```
zoom =
  <number [0,∞]>     |
  <percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Viewport Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Resizing paragraphs

In this example the paragraph elements are zoomed, on hovering a paragraph the `zoom` value is `unset`.

HTML

```
<p class="small">Small</p>
<p class="normal">Normal</p>
<p class="big">Big</p>
```

CSS

```
body {
  display: flex;
  align-items: center;
  justify-content: space-around;
  height: 100vh;
}
```

```
.small {
  zoom: 75%;
}
.normal {
  zoom: normal;
}
.big {
  zoom: 2.5;
}
p:hover {
  zoom: unset;
}
```

Result

Resizing elements

In this example the `div` elements are zoomed using the `normal`, `<percentage>`, and `<number>` values.

HTML

```
<div id="a" class="circle"></div>
<div id="b" class="circle"></div>
<div id="c" class="circle"></div>
```

CSS

```
div.circle {
  width: 25px;
  height: 25px;
  border-radius: 100%;
  vertical-align: middle;
  display: inline-block;
}
div#a {
  background-color: gold;
  zoom: normal; /* circle is 25px diameter */
}
div#b {
  background-color: green;
  zoom: 200%; /* circle is 50px diameter */
}
div#c {
  background-color: blue;
  zoom: 2.9; /* circle is 72.5px diameter */
}
```

Result

Creating a zoom control

In this example a `select` field is used to change the zoom level of the element.

HTML

In this first block, of HTML, a `select` field is defined with the different `zoom` values to be used.

```
<section class="controls">
  <label for="zoom"
    >Zoom level
    <select name="zoom" id="zoom">
      <option value="0.5">Extra Small</option>
      <option value="0.75">Small</option>
      <option value="normal" selected>Normal</option>
      <option value="1.5">Large</option>
      <option value="2">Extra Large</option>
    </select>
  </label>
</section>
```

In this second block a **not supported** message is added that will be hidden if the browser supports `zoom`.

```
<p class="zoom-notice">CSS zoom is not supported</p>
```

The final block just defines the content that will be zoomed.

```
<section class="content">
  <h1>This is the heading</h1>
  <p>
    Lorem ipsum dolor, sit amet consectetur adipisicing elit. Placeat inventore
    ea eveniet, fugiat in consequatur molestiae nostrum repellendus nam
    provident repellat officiis facilis alias facere obcaecati quos sunt
    voluptas! Iste.
  </p>
  <p>
    Lorem ipsum dolor, sit amet consectetur adipisicing elit. Placeat inventore
    ea eveniet, fugiat in consequatur molestiae nostrum repellendus nam
    provident repellat officiis facilis alias facere obcaecati quos sunt
    voluptas! Iste.
  </p>
</section>
```

CSS

In this first block, of CSS, we are setting the starting value for the `--zoom-level` using custom properties and then using that as the value for `zoom` on the content block.

```
html {
  --zoom-level: normal;
}
.content {
  max-width: 60ch;
  margin: auto;
  zoom: var(--zoom-level);
}
```

```
.controls,
.zoom-notice {
  display: flex;
  justify-content: space-around;
}
.zoom-notice {
  color: red;
}
```

In this final CSS block we are checking to see if the browser supports `zoom` and if so setting the **not supported** message to `display: none;`.

```
@supports (zoom: 1) {
  .zoom-notice {
    display: none;
  }
}
```

JavaScript

This JavaScript watches for a change in the select field and sets the new value for `--zoom-level` on the content `section`, e.g., `style="--zoom-level: 1.5;"`.

```
const zoomControl = document.querySelector("#zoom");
const content = document.querySelector(".content");
const updateZoom = () => {
  content.style = `--zoom-level: ${zoomControl.value}`;
};
zoomControl.addEventListener("change", updateZoom);
```

Result

See also

- `zoom` entry in CSS-Tricks' CSS Almanac
- `transform`
- `scale`
- `unset` keyword
- Legacy `zoom` property via CSS-Tricks (2013)

CSS selectors

CSS selectors are patterns used in CSS rules to target and select specific elements for styling.

> **Note:** This page is an index of all selectors in CSS. The CSS selectors page introduces the module that defines some, but not all, of these selectors.

For example, to style paragraphs, you will use the `p` type selector to select all `<p>` elements and apply a style to them:

```
/* Set font size on all <p> elements */
p {
  font-size: 12px;
  color: rebeccapurple;
}
```

Syntax

```
/* Select elements and apply styles */
selector {
  property: value;
}
```

Index of selectors

- & nesting selector
- Attribute selectors
- Class selectors
- ID selectors
- Keyframe selectors
- Namespace separator (|)
- Pseudo-class selectors
- Pseudo-element selectors
- Selector list
- Type selectors
- Universal selectors

Check the pseudo-classes and pseudo-elements pages for their respective specification tables.

See also

- CSS selectors module
- CSS selector structure
- CSS combinators
- Selector list

- Selectors and combinators
- CSS pseudo-elements module
- CSS nesting module
- Specificity

:-moz-broken CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-broken`** CSS pseudo-class is a Mozilla extension that matches elements representing broken image links.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-broken {
  /* ... */
}
```

Examples

HTML

```
<img src="broken.jpg" alt="This image is broken. :-(" />
```

CSS

```
:-moz-broken {
  background: bisque;
  padding: 8px;
}
```

Result

Specifications

Not part of any standard.

See also

- `:-moz-loading`, `:-moz-suppressed`, `:-moz-user-disabled`
- Firefox bug 11011

:-moz-drag-over CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-drag-over`** CSS pseudo-class is a Mozilla extension that matches an element when a `dragover` event is called on it.

Syntax

```
:-moz-drag-over {
  /* ... */
}
```

Examples

HTML

```
<div id="drop-target">
  <p>Drop target</p>
</div>

<div draggable="true">
  <p>Draggable</p>
</div>
```

JavaScript

Most elements are not valid places to drop data, so in order to allow a drop, you must prevent default behavior by canceling `dragenter` or `dragover` (or both) events. In this example, we only need to cancel the `dragenter` event, which is the first event fired when the browser evaluates if an element can be a drop target. For more information, see Drag operations: Specifying drop targets.

```
const target = document.getElementById("drop-target");
/* dragenter event fired on the drop target */
target.addEventListener("dragenter", (event) => {
  // prevent default to allow drop
  event.preventDefault();
});
```

CSS

```
body {
  font-family: "Arial";
}
div {
  display: inline-block;
  width: 150px;
  height: 150px;
  border: 2px dotted black;
  background-color: aquamarine;
```

```
  margin: 1rem;
}
p {
  padding: 1rem;
}
```

The following CSS changes the drop target color to red when the draggable element overlays the drop area.

```
#drop-target {
  background-color: cornflowerblue;
}
#drop-target:-moz-drag-over {
  background-color: red;
}
```

Result

Specifications

Not part of any standard.

See also

- Mozilla CSS extensions
- HTML drag and drop

:-moz-first-node CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:-moz-first-node`** CSS pseudo-class is a Mozilla extension that represents any element that is the first child node of some other element. It differs from `:first-child` because it does not match a first-child element with (non-whitespace) text before it.

> **Note:** Any whitespace at the start of an element is ignored for the determination of `:-moz-first-node`.

Syntax

```
:-moz-first-node {
  /* ... */
}
```

Examples

CSS

```
span:-moz-first-node {
  background-color: lime;
}
```

HTML

```
<p>
  <span>This matches!</span>
  <span>This doesn't match.</span>
</p>

<p>
  Blahblah.
  <span>This doesn't match because it's preceded by text.</span>
</p>
```

Result

Specifications

Not part of any standard.

See also

- `:-moz-last-node`
- `:first-child`

:-moz-handler-blocked CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-handler-blocked`** CSS pseudo-class is a Mozilla extension that matches elements that can't be displayed because their handlers have been blocked.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-handler-blocked {
  /* ... */
```

```
}
```

Specifications

Not part of any standard.

See also

- `:-moz-handler-crashed`
- `:-moz-handler-disabled`

:-moz-handler-crashed CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-handler-crashed`** CSS pseudo-class is a Mozilla extension that matches elements that can't be displayed because the plugin responsible for drawing them has crashed.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-handler-crashed {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `:-moz-handler-blocked`
- `:-moz-handler-disabled`

:-moz-handler-disabled CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-handler-disabled`** CSS pseudo-class is a Mozilla extension that matches elements that can't be displayed because their handlers have been disabled by the user.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-handler-disabled {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `:-moz-handler-blocked`
- `:-moz-handler-crashed`

:-moz-last-node CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:-moz-last-node`** CSS pseudo-class is a Mozilla extension that represents any element that is the last child node of some other element. It differs from `:last-child` because it does not match a last-child element with (non-whitespace) text after it.

> **Note:** Any whitespace at the end of an element is ignored for the determination of `:-moz-last-node`.

Syntax

```
:-moz-last-node {
  /* ... */
}
```

Examples

CSS

```
span:-moz-last-node {
  background-color: lime;
}
```

HTML

```
<p>
  <span>This does not match.</span>
  <span>This matches!</span>
</p>

<p>
  <span>This doesn't match because it's followed by text.</span>
  Blahblah.
</p>
```

Result

Specifications

Not part of any standard.

See also

- `:-moz-first-node`
- `:last-child`

:-moz-loading CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-loading`** CSS pseudo-class is a Mozilla extension that matches elements that can't be displayed because they have not started loading, such as images that haven't started to arrive yet. Note that images that are *in the process* of loading *are not* matched by this pseudo-class.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-loading {
  /* ... */
}
```

Examples

Setting a background for images that are loading

```
:-moz-loading {
  background: url("loading-animation.gif") center no-repeat;
}
```

Specifications

Not part of any standard.

See also

- `:-moz-broken`, `:-moz-suppressed`, `:-moz-user-disabled`

:-moz-locale-dir(ltr) CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-locale-dir(ltr)`** CSS pseudo-class is a Mozilla extension that matches an element if the user interface is being displayed left-to-right. This is determined by the preference `intl.uidirection.locale` (where `locale` is the current locale) being set to "ltr".

> **Note:** This selector is mainly used by extensions and themes to adapt the user interface based on the user's locale. (This can vary from window to window, and even from tab to tab.) It also allows extensions to work even when they don't support the user's default locale, since they can support both left-to-right and right-to-left layouts regardless of locale specifics.

> **Warning:** This selector does not work properly from HTML; it always matches, regardless of whether the UI locale is left-to-right or right-to-left.

Syntax

```
:-moz-locale-dir(ltr) {
  /* ... */
}
```

Examples

This example doesn't work if you're not using Firefox, and may not work even in Firefox due to an issue with the selector not working properly with HTML content. It was designed for use with XUL.

HTML

```
<p>If you're using a left-to-right interface, this should be red.</p>
```

CSS

```
p:-moz-locale-dir(ltr) {
  color: red;
}
```

Result

Specifications

Not part of any standard.

See also

- `:dir(…)`
- `:-moz-locale-dir(rtl)`

:-moz-locale-dir(rtl) CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-locale-dir(rtl)`** CSS pseudo-class is a Mozilla extension that matches an element if the user interface is being displayed right-to-left. This is determined by the preference `intl.uidirection.locale` (where `locale` is the current locale) being set to "rtl".

> **Note:** This selector is mainly used by extensions and themes to adapt the user interface based on the user's locale. (This can vary from window to window, and even from tab to tab.) It also allows extensions to work even when they don't support the user's default locale, since they can support both left-to-right and right-to-left layouts regardless of locale specifics.

> **Warning:** This selector does not work properly from HTML; it never matches, no matter whether the UI locale is left-to-right or right-to-left.

Syntax

```
:-moz-locale-dir(rtl) {
  /* ... */
}
```

Examples

This example will not work if you're not using Firefox, and might not work properly even in Firefox due to an issue with using this selector in HTML.

HTML

```
<p>If you're using a right-to-left interface, this should be red.</p>
```

CSS

```
p:-moz-locale-dir(rtl) {
  color: red;
}
```

Result

Specifications

Not part of any standard.

See also

- `:dir(…)`
- `:-moz-locale-dir(ltr)`

:-moz-only-whitespace CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** In Selectors Level 4, the `:empty` selector was changed to act like `:-moz-only-whitespace`, but no browser currently supports this yet.

The **`:-moz-only-whitespace`** CSS pseudo-class matches elements that only contain text nodes that only contain whitespace. (This includes elements with empty text nodes and elements with no child nodes.)

Syntax

```
:-moz-only-whitespace {
  /* ... */
}
```

Examples

Basic :-moz-only-whitespace example

HTML

```
<div> </div>
```

CSS

```
div {
  border: 4px solid red;
}

:-moz-only-whitespace {
  border-color: lime;
}
```

Result

Specifications

Briefly defined as `:blank` in Selectors Level 4, but then the functionality was merged into `:empty` and `:blank` redefined to mean empty `<input>`.

See also

- `:blank`
- `:empty`

:-moz-submit-invalid CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-submit-invalid`** CSS pseudo-class is a Mozilla extension that represents any submit `<button>` on forms whose contents aren't valid based on their validation constraints.

By default, no style is applied. You can use this pseudo-class to customize the appearance of the submit button when there are invalid form fields.

Syntax

```
:-moz-submit-invalid {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `:valid`
- `:invalid`
- `:required`
- `:optional`

:-moz-suppressed CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-suppressed`** CSS pseudo-class is a Mozilla extension that matches elements representing images that were suppressed because loading images from the specified site has been blocked.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-suppressed {
  /* ... */
}
```

Examples

Styling elements that have been blocked

```
:-moz-suppressed {
  background: yellow;
  padding: 8px;
}
```

Specifications

Not part of any standard.

See also

- `:-moz-broken`, `:-moz-loading`, `:-moz-user-disabled`

:-moz-user-disabled CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-user-disabled`** CSS pseudo-class is a Mozilla extension that matches elements representing images that were not loaded because images have been entirely disabled by the user's preferences.

> **Note:** This selector is mainly intended to be used by theme developers.

Syntax

```
:-moz-user-disabled {
  /* ... */
}
```

Examples

Styling user-disabled elements

```
:-moz-user-disabled {
  background-color: lightgray;
```

```
  padding: 8px;
}
```

Specifications

Not part of any standard.

See also

- `:-moz-broken`, `:-moz-loading`, `:-moz-suppressed`

:-moz-window-inactive CSS pseudo-class

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`:-moz-window-inactive`** CSS pseudo-class is a Mozilla extension that matches any element while it's in an inactive window.

`:-moz-window-inactive` works in content HTML documents, too.

> **Note:** Prior to the addition of this pseudo-class, giving different styles to background windows was achieved by setting an attribute (`active="true"`) on the top-level XUL chrome window. This attribute is no longer used.

Syntax

```
:-moz-window-inactive {
  /* ... */
}
```

Examples

This example alters the appearance of a box's background depending on whether its window is active or not.

HTML

```
<div id="my-box">
  <p>This is a box!</p>
</div>
```

CSS

```
#my-box {
  background: linear-gradient(to bottom, yellow, cyan);
  width: 200px;
  height: 200px;
}

#my-box:-moz-window-inactive {
  background: cyan;
}
```

Result

The result of this code is shown below. You can view this as a separate page as well.

Specifications

Not part of any standard.

See also

- Mozilla CSS extensions
- Related CSS selectors:
  - `:active`
- The `Document.activeElement` property for regular DOM, and the `ShadowRoot.activeElement` property for shadow DOM roots.

::-moz-color-swatch CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-moz-color-swatch`** CSS pseudo-element is a Mozilla extension that represents the color selected in an `<input>` of `type="color"`.

> **Note:** Using `::-moz-color-swatch` with anything but an `<input type="color">` doesn't match anything and has no effect.

Syntax

```
::-moz-color-swatch {
  /* ... */
}
```

Examples

HTML

```
<input type="color" value="#de2020" />
```

CSS

```
input[type="color"]::-moz-color-swatch {
  border-radius: 10px;
  border-style: none;
}
```

Result

Specifications

Not part of any standard.

See also

- Similar pseudo-elements used by other browsers:
  - `::-webkit-color-swatch`, pseudo-element supported by WebKit and Blink (Safari, Chrome, and Opera)

::-moz-focus-inner CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-moz-focus-inner`** CSS pseudo-element is a Mozilla extension that represents an inner focus ring of the `<button>` element as well as the button, submit, reset, and color types of the `<input>` element.

> **Note:** Using `::-moz-focus-inner` with anything than the buttons that support it doesn't match anything and has no effect.

Syntax

```
::-moz-focus-inner {
  /* ... */
}
```

Example

HTML

```
<input type="submit" value="Input" /> <button type="submit">Button</button>
```

CSS

```
button::-moz-focus-inner,
input[type="color"]::-moz-focus-inner,
input[type="reset"]::-moz-focus-inner,
input[type="button"]::-moz-focus-inner,
input[type="submit"]::-moz-focus-inner {
  padding-block-start: 0px;
  padding-inline-end: 2px;
  padding-block-end: 0px;
  padding-inline-start: 2px;
  border: 2px dotted red;
}
```

Result

Specifications

Not part of any standard.

See also

- Mozilla CSS extensions
- Related CSS properties:
  - `-moz-user-focus`
- Related CSS selectors:
  - `:focus`
  - `:focus-visible`
  - `:focus-within`

::-moz-list-bullet CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::-moz-list-bullet`** CSS pseudo-element is a Mozilla extension that represents the marker (typically a bullet) of a list item (`<li>`) in an unordered list (`<ul>`).

Syntax

```
li::-moz-list-bullet {
  /* ... */
}
```

Examples

Styling list item markers

HTML

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

CSS

```
::-moz-list-bullet {
  color: red;
  font-size: 1.5em;
}
```

Result

Specifications

Not part of any standard.

See also

- `::-moz-list-number`
- `::marker`

::-moz-list-number CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::-moz-list-number`** CSS pseudo-element is a Mozilla extension that represents the marker (typically a number) of a list item (`<li>`) in an ordered list (`<ol>`).

Syntax

```
li::-moz-list-number {
  /* ... */
}
```

Examples

HTML

```
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

CSS

```
li::-moz-list-number {
  font-style: italic;
  font-weight: bold;
}
```

Result

Specifications

Not part of any standard.

See also

- `::-moz-list-bullet`
- `::marker`

::-moz-meter-bar CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-moz-meter-bar`** CSS pseudo-element represents the meter gauge in a `<meter>` element. It is used for selecting and applying styles to the gauge inside a meter element.

> **Note:** By default, the `<meter>` element uses native styling. To apply your own styles, first set `appearance: none` on the `<meter>` element, and then style using `::-moz-meter-bar`.

Syntax

```
meter {
  appearance: none;
}
::-moz-meter-bar {
  /* ... */
}
```

Examples

HTML

```
Normal: <meter min="0" max="10" value="6">Score 6/10</meter>
<br />
Styled:   <meter class="styled" min="0" max="10" value="6">
  Score 6/10
</meter>
```

CSS

```
meter {
  height: 20px;
  width: 200px;
  vertical-align: -0.4rem;
}

.styled {
  appearance: none;
}

.styled::-moz-meter-bar {
  background: lime;
  box-shadow: 0 2px 3px grey inset;
  border-radius: 5px;
}
```

Result

Specifications

Not part of any standard.

See also

- `::-webkit-meter-bar`
- `appearance`
- `accent-color`

::-moz-progress-bar CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::-moz-progress-bar`** CSS pseudo-element is a Mozilla extension that represents the progress bar inside a `<progress>` element. (The bar represents the amount of progress that has been made.)

If you want to select the unfinished part of `<progress>` in Mozilla, please select the `<progress>` directly.

Syntax

```
::-moz-progress-bar {
  /* ... */
}
```

Examples

HTML

```
<progress value="30" max="100">30%</progress>
<progress max="100">Indeterminate</progress>
```

CSS

```
::-moz-progress-bar {
  background-color: red;
}

/* Force indeterminate bars to have zero width */
:indeterminate::-moz-progress-bar {
  width: 0;
}
```

Result

Specifications

Not part of any standard.

See also

- `<progress>`
- `::-webkit-progress-bar`
- `::-webkit-progress-value`
- `::-webkit-progress-inner-element`

::-moz-range-progress CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-moz-range-progress`** CSS pseudo-element is a Mozilla extension that represents the lower portion of the *track* (i.e., groove) in which the indicator slides in an `<input>` of `type="range"`. This portion corresponds to values lower than the value currently selected by the *thumb* (i.e., virtual knob).

> **Note:** Using `::-moz-range-progress` with anything but an `<input type="range">` doesn't match anything and has no effect.

Syntax

```
::-moz-range-progress {
  /* ... */
}
```

Examples

HTML

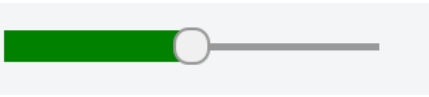
```
<input type="range" min="0" max="100" step="5" value="50" />
```

CSS

```
input[type="range"]::-moz-range-progress {
  background-color: green;
  height: 1em;
}
```

Result

A progress bar using this style might look something like this:



Specifications

Not part of any standard.

See also

- The pseudo-elements used by Gecko to style other parts of a range input:
  - `::-moz-range-thumb` represents the indicator that slides in the groove.
  - `::-moz-range-track` represents the groove in which the thumb slides.
- CSS-Tricks: Styling Cross-Browser Compatible Range Inputs with CSS
- QuirksMode: Styling and scripting sliders

::-moz-range-thumb CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-moz-range-thumb`** CSS pseudo-element is a Mozilla extension that represents the *thumb* (i.e., virtual knob) of an `<input>` of `type="range"`. The user can move the thumb along the input's track to alter its numerical value.

> **Note:** Using `::-moz-range-thumb` with anything but an `<input type="range">` doesn't match anything and has no effect.

Syntax

```
::-moz-range-thumb {
  /* ... */
}
```

Examples

HTML

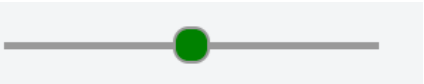
```
<input type="range" min="0" max="100" step="5" value="50" />
```

CSS

```
input[type="range"]::-moz-range-thumb {
  background-color: green;
}
```

Result

A progress bar using this style might look something like this:



Specifications

Not part of any standard.

See also

- The pseudo-elements used by Gecko to style other parts of a range input:

    - `::-moz-range-track` represents the groove in which the thumb slides.
    - `::-moz-range-progress` represents the lower portion of the track.
- Similar pseudo-elements used by other browsers:
    - `::-webkit-slider-thumb`, pseudo-element supported by WebKit and Blink (Safari, Chrome, and Opera)
- CSS-Tricks: Styling Cross-Browser Compatible Range Inputs with CSS
- QuirksMode: Styling and scripting sliders

::-moz-range-track CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-moz-range-track`** CSS pseudo-element is a Mozilla extension that represents the *track* (i.e., groove) in which the indicator slides in an `<input>` of `type="range"`.

> **Note:** Using `::-moz-range-track` with anything but an `<input type="range">` doesn't match anything and has no effect.

Syntax

```
::-moz-range-track {
  /* ... */
}
```

Examples

HTML

```
<input type="range" min="0" max="100" step="5" value="50" />
```

CSS

```
input[type="range"]::-moz-range-track {
  background-color: green;
}
```

Result

A range slider using this style might look something like this:

Specifications

Not part of any standard.

See also

- The pseudo-elements used by Gecko to style other parts of a range input:
    - `::-moz-range-thumb` represents the indicator that slides in the groove.
    - `::-moz-range-progress` represents the lower portion of the track.
- Similar pseudo-elements used by other browsers:
    - `::-webkit-slider-runnable-track`, pseudo-element supported by WebKit and Blink (Safari, Chrome, and Opera)
- CSS-Tricks: Styling Cross-Browser Compatible Range Inputs with CSS
- QuirksMode: Styling and scripting sliders

::-webkit-inner-spin-button CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-inner-spin-button`** CSS pseudo-element is used to style the inner part of the spinner button of number picker input elements.

Syntax

```
::-webkit-inner-spin-button {
  /* ... */
}
```

Examples

These examples work only in browsers based on WebKit and Blink.

Changing the cursor in the spin controls

In this example, the CSS `cursor` is changed to `pointer` whenever the cursor is positioned over the inner part of the input's spin controls.

HTML

```
<input type="number" />
```

CSS

```
input[type="number"]::-webkit-inner-spin-button {
  cursor: pointer;
}
```

Result

Specifications

Not part of any standard.

See also

- `::-webkit-textfield-decoration-container`

::-webkit-meter-bar CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-meter-bar`** CSS pseudo-element is a WebKit extension that represents the background of a `<meter>` element. It is used for selecting and applying styles to the container of a meter gauge.

Syntax

```
::-webkit-meter-bar {
  /* ... */
}
```

Examples

HTML

```
Normal: <meter min="0" max="10" value="6">Score 6/10</meter>
<br />
Styled:   <meter id="styled" min="0" max="10" value="6">
  Score 6/10
</meter>
```

CSS

```
meter {
  height: 30px;
  width: 200px;
  vertical-align: -0.8rem;
}
```

```
.safari meter {
  /* Reset the default appearance for Safari only */
  /* .safari class is added via JavaScript */
  -webkit-appearance: none;
}

#styled::-webkit-meter-bar {
  background: lime;
  box-shadow: 0 10px 20px grey inset;
  border-radius: 10px;
}
```

JavaScript

```
// Safari requires <meter> elements to have an `appearance` of `none` for custom styling
// using `::-webkit-meter-*` selectors, but `appearance: none` breaks rendering on Chrome.
// Therefore, we must check if the browser is Safari-based.

const isSafari =
  navigator.userAgent.includes("AppleWebKit/") &&
  !navigator.userAgent.includes("Chrome/");

if (isSafari) {
  document.body.classList.add("safari");
}
```

Result

Specifications

Not part of any standard.

See also

- The pseudo-elements used by WebKit/Blink to style other parts of a `<meter>` element are as follows:
  - `::-webkit-meter-inner-element`
  - `::-webkit-meter-even-less-good-value`
  - `::-webkit-meter-optimum-value`
  - `::-webkit-meter-suboptimum-value`
- WebKit CSS extensions

::-webkit-meter-even-less-good-value CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-meter-even-less-good-value`** styles the `<meter>` element when the value and the optimum attributes fall outside the low-high range, but in opposite zones. To illustrate, it applies when *value* < *low* < *high* < *optimum* or *value* > *high* > *low* > *optimum*.

Red is the default color.

Syntax

```
::-webkit-meter-even-less-good-value {
  /* ... */
}
```

Examples

HTML

```
Normal:
<meter min="0" max="10" low="3" high="7" optimum="8" value="2">
  Score 2/10
</meter>
<br />
Styled:
<meter id="styled" min="0" max="10" low="3" high="7" optimum="8" value="2">
  Score 2/10
</meter>
```

CSS

```
body {
  font-family: monospace;
}

.safari meter {
  /* Reset the default appearance for Safari only */
  /* .safari class is added via JavaScript */
  -webkit-appearance: none;
}

#styled::-webkit-meter-even-less-good-value {
  background: linear-gradient(
    to bottom,
    #ff7777,
    #990000 45%,
    #990000 55%,
    #ff7777
  );
  height: 100%;
  box-sizing: border-box;
}
```

JavaScript

```
// Safari requires <meter> elements to have an `appearance` of `none` for custom styling
// using `::-webkit-meter-*` selectors, but `appearance: none` breaks rendering on Chrome.
// Therefore, we must check if the browser is Safari-based.

const isSafari =
  navigator.userAgent.includes("AppleWebKit/") &&
  !navigator.userAgent.includes("Chrome/");

if (isSafari) {
  document.body.classList.add("safari");
}
```

Result

Specifications

Not part of any standard.

See also

The pseudo-elements used by WebKit/Blink to style other parts of a `<meter>` element are as follows:

- `::-webkit-meter-inner-element`
- `::-webkit-meter-bar`
- `::-webkit-meter-optimum-value`
- `::-webkit-meter-suboptimum-value`

::-webkit-meter-inner-element CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

`::-webkit-meter-inner-element` is a proprietary WebKit CSS pseudo-element for selecting and applying styles to the outer containing element of a `<meter>` element. Additional markup to render the meter element as read-only.

Syntax

```
::-webkit-meter-inner-element {
  /* ... */
}
```

Examples

This will only work in WebKit and Blink-based browsers, such as Safari, Chrome, and Chromium-based versions of Edge.

HTML

```
Normal: <meter min="0" max="10" value="6">Score 6/10</meter>
<br />
Styled: <meter id="styled" min="0" max="10" value="6">Score 6/10</meter>
```

CSS

```
body {
  font-family: monospace;
}

.safari meter {
  /* Reset the default appearance for Safari only */
  /* .safari class is added via JavaScript */
  -webkit-appearance: none;
}

#styled::-webkit-meter-inner-element {
  -webkit-appearance: inherit;
  box-sizing: inherit;
  border: 1px dashed #aaaaaa;
}
```

JavaScript

```
// Safari requires <meter> elements to have an `appearance` of `none` for custom styling
// using `::-webkit-meter-*` selectors, but `appearance: none` breaks rendering on Chrome.
// Therefore, we must check if the browser is Safari-based.

const isSafari =
  navigator.userAgent.includes("AppleWebKit/") &&
  !navigator.userAgent.includes("Chrome/");

if (isSafari) {
  document.body.classList.add("safari");
}
```

Result

Specifications

Not part of any standard.

See also

The pseudo-elements used by WebKit/Blink to style other parts of a `<meter>` element are as follows:

- `::-webkit-meter-bar`
- `::-webkit-meter-even-less-good-value`
- `::-webkit-meter-optimum-value`
- `::-webkit-meter-suboptimum-value`

::-webkit-meter-optimum-value CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-meter-optimum-value`** CSS pseudo-element styles the `<meter>` element when its value is inside the low-high range, or when the value is equivalent to the optimum value.

Green is the default color.

Syntax

```
::-webkit-meter-optimum-value {
  /* ... */
}
```

Examples

This example will only work in browsers based on WebKit or Blink.

HTML

```
Normal:
<meter min="0" max="10" low="3" high="7" optimum="6" value="6">
  Score 6/10
</meter>
<br />
Styled:
<meter id="styled" min="0" max="10" low="3" high="7" optimum="6" value="6">
  Score 6/10
</meter>
```

CSS

```
body {
  font-family: monospace;
}

.safari meter {
  /* Reset the default appearance for Safari only */
  /* .safari class is added via JavaScript */
  -webkit-appearance: none;
}

#styled::-webkit-meter-optimum-value {
  background: linear-gradient(
```

```
    to bottom,
    #77ff77,
    #009900 45%,
    #009900 55%,
    #77ff77
  );
  height: 100%;
  box-sizing: border-box;
}
```

JavaScript

```
// Safari requires <meter> elements to have an `appearance` of `none` for custom styling
// using `::-webkit-meter-*` selectors, but `appearance: none` breaks rendering on Chrome.
// Therefore, we must check if the browser is Safari-based.

const isSafari =
  navigator.userAgent.includes("AppleWebKit/") &&
  !navigator.userAgent.includes("Chrome/");

if (isSafari) {
  document.body.classList.add("safari");
}
```

Result

Specifications

Not part of any standard.

See also

The pseudo-elements used by WebKit/Blink to style other parts of a `<meter>` element are as follows:

- `::-webkit-meter-inner-element`
- `::-webkit-meter-bar`
- `::-webkit-meter-even-less-good-value`
- `::-webkit-meter-suboptimum-value`

::-webkit-meter-suboptimum-value CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-meter-suboptimum-value`** pseudo-element styles the `<meter>` element when the value attribute falls outside of the low-high range and is not equivalent to the optimum value.

Yellow is the default color.

Syntax

```
::-webkit-meter-suboptimum-value {
  /* ... */
}
```

Examples

This example will only work in browsers based on WebKit or Blink.

HTML

```
Normal:
<meter min="0" max="10" low="3" high="7" optimum="6" value="2">
  Score 2/10
</meter>
<br />
Styled:
<meter id="styled" min="0" max="10" low="3" high="7" optimum="6" value="2">
  Score 2/10
</meter>
```

CSS

```
body {
  font-family: monospace;
}

.safari meter {
  /* Reset the default appearance for Safari only */
  /* .safari class is added via JavaScript */
  -webkit-appearance: none;
}

#styled::-webkit-meter-suboptimum-value {
  background: linear-gradient(
    to bottom,
    #ffff77,
    #999900 45%,
    #999900 55%,
    #ffff77
  );
  height: 100%;
  box-sizing: border-box;
}
```

JavaScript

```
// Safari requires <meter> elements to have an `appearance` of `none` for custom styling
// using `::-webkit-meter-*` selectors, but `appearance: none` breaks rendering on Chrome.
```

```
// Therefore, we must check if the browser is Safari-based.

const isSafari =
  navigator.userAgent.includes("AppleWebKit/") &&
  !navigator.userAgent.includes("Chrome/");

if (isSafari) {
  document.body.classList.add("safari");
}
```

Result

Specifications

Not part of any standard.

See also

The pseudo-elements used by WebKit/Blink to style other parts of a `<meter>` element are as follows:

- `::-webkit-meter-inner-element`
- `::-webkit-meter-bar`
- `::-webkit-meter-even-less-good-value`
- `::-webkit-meter-optimum-value`

::-webkit-progress-bar CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-progress-bar`** CSS pseudo-element represents the entire bar of a `<progress>` element. Normally it's only visible as the unfilled portion of the bar, since by default it's rendered below the `::-webkit-progress-value` pseudo-element. It is a child of the `::-webkit-progress-inner-element` pseudo-element and the parent of the `::-webkit-progress-value` pseudo-element.

> **Note:** For `::-webkit-progress-value` to take effect, `appearance` needs to be set to `none` on the `<progress>` element.

Syntax

```
::-webkit-progress-bar {
  /* ... */
}
```

Examples

CSS

```
progress {
  -webkit-appearance: none;
}

::-webkit-progress-bar {
  background-color: orange;
}
```

HTML

```
<progress value="10" max="50"></progress>
```

Result

Result screenshot

The code above results in a progress bar that looks like this when using a WebKit or Blink browser:

Specifications

Not part of any standard.

See also

- The pseudo-elements used by WebKit/Blink to style other parts of a `<progress>` element:
  - `::-webkit-progress-value`
  - `::-webkit-progress-inner-element`
- `::-moz-progress-bar`

::-webkit-progress-inner-element CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-progress-inner-element`** CSS pseudo-element represents the outermost container of the `<progress>` element. It is the parent of the `::-webkit-progress-bar` pseudo-element.

> **Note:** In order to let `::-webkit-progress-value` take effect, `appearance` needs to be set to `none` on the `<progress>` element.

Syntax

```
::-webkit-progress-inner-element {
  /* ... */
}
```

Examples

These examples work only on Blink and WebKit.

Adding a black border around the progress bar

In this example, a 2px black border is added around the progress bar.

HTML

```
<progress value="10" max="50"></progress>
```

CSS

```
progress {
  -webkit-appearance: none;
}

::-webkit-progress-inner-element {
  border: 2px solid black;
}
```

Result

Result screenshot

If you're not using a Blink or WebKit browser, the above code results in a progress bar looking like this:

Specifications

Not part of any standard.

See also

- The pseudo-elements used by WebKit/Blink to style other parts of a `<progress>` element:
  - `::-webkit-progress-bar`
  - `::-webkit-progress-value`
- `::-moz-progress-bar`

::-webkit-progress-value CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-progress-value`** CSS pseudo-element represents the filled-in portion of the bar of a `<progress>` element. It is a child of the `::-webkit-progress-bar` pseudo-element.

> **Note:** In order to let `::-webkit-progress-value` take effect, `appearance` needs to be set to `none` on the `<progress>` element.

Syntax

```
::-webkit-progress-value {
  /* ... */
}
```

Examples

This example will only work in browsers based on Blink or WebKit.

HTML

```
<progress value="10" max="50"></progress>
```

CSS

```
progress {
  -webkit-appearance: none;
}

::-webkit-progress-value {
  background-color: orange;
}
```

Result

Result screenshot

A progress bar using the style above would look like this:

Specifications

Not part of any standard.

See also

- The pseudo-elements used by WebKit/Blink to style other parts of a `<progress>` element:
  - `::-webkit-progress-bar`
  - `::-webkit-progress-inner-element`
- `::-moz-progress-bar`

::-webkit-scrollbar CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The `::-webkit-scrollbar` CSS pseudo-element affects the style of an element's scrollbar when it has scrollable overflow.

The `scrollbar-color` and `scrollbar-width` standard properties may be used as alternatives for browsers that do not support this pseudo-element and the related `::-webkit-scrollbar-*` pseudo-elements (see Browser compatibility).

> **Note:** If an element's computed `scrollbar-color` and `scrollbar-width` values are anything other than `auto`, they will override `::-webkit-scrollbar-*` styling. Because `scrollbar-color` is inherited, an element is affected when an ancestor sets it, even if the element itself doesn't. Setting `scrollbar-color: auto` on the element restores its `::-webkit-scrollbar-*` styling if it was previously removed by a `scrollbar-color` setting on an ancestor. See Adding a fallback for scrollbar styles for more details.

CSS Scrollbar Selectors

You can use the following pseudo-elements to customize various parts of the scrollbar for WebKit browsers:

- `::-webkit-scrollbar` — the entire scrollbar.
- `::-webkit-scrollbar-button` — the buttons on the scrollbar (arrows pointing upwards and downwards that scroll one line at a time).
- `::-webkit-scrollbar:horizontal` — the horizontal scrollbar.
- `::-webkit-scrollbar-thumb` — the draggable scrolling handle.
- `::-webkit-scrollbar-track` — the track (progress bar) of the scrollbar, where there is a gray bar on top of a white bar.
- `::-webkit-scrollbar-track-piece` — the part of the track (progress bar) not covered by the handle.
- `::-webkit-scrollbar:vertical` — the vertical scrollbar.
- `::-webkit-scrollbar-corner` — the bottom corner of the scrollbar, where both horizontal and vertical scrollbars meet. This is often the bottom-right corner of the browser window.
- `::-webkit-resizer` — the draggable resizing handle that appears at the bottom corner of some elements.

Accessibility

Authors should avoid styling scrollbars, as changing the appearance of scrollbars away from the default breaks external consistency which negatively impacts usability. If styling scrollbars, ensure there is enough color contrast and touch targets are at least 44px wide and tall. See Techniques for WCAG 2.0: G183: Using a contrast ratio of 3:1 and Understanding WCAG 2.1 : Target Size.

Examples

Styling scrollbars using <code>-webkit-scrollbar</code>

CSS

```
.visible-scrollbar,
.invisible-scrollbar,
.mostly-customized-scrollbar {
  display: block;
  width: 10em;
  overflow: auto;
  height: 2em;
  padding: 1em;
  margin: 1em auto;
  outline: 2px dashed cornflowerblue;
}

.invisible-scrollbar::-webkit-scrollbar {
  display: none;
}

/* Demonstrate a "mostly customized" scrollbar
 * (won't be visible otherwise if width/height is specified) */
.mostly-customized-scrollbar::-webkit-scrollbar {
  width: 5px;
  height: 8px;
  background-color: #aaaaaa; /* or add it to the track */
}

/* Add a thumb */
.mostly-customized-scrollbar::-webkit-scrollbar-thumb {
  background: black;
}
```

HTML

```
<div class="visible-scrollbar">
  <h3>Visible scrollbar</h3>
  <p>
    Etiam sagittis sem sed lacus laoreet, eu fermentum eros auctor. Proin at
    nulla elementum, consectetur ex eget, commodo ante. Sed eros mi, bibendum ut
    dignissim et, maximus eget nibh. Phasellus blandit quam turpis, at mollis
    velit pretium ut. Nunc consequat efficitur ultrices. Nullam hendrerit
    posuere est. Nulla libero sapien, egestas ac felis porta, cursus ultricies
    quam. Vestibulum tincidunt accumsan sapien, a fringilla dui semper in.
    Vivamus consectetur ipsum a ornare blandit. Aenean tempus at lorem sit amet
    faucibus. Curabitur nibh justo, faucibus sed velit cursus, mattis cursus
    dolor. Pellentesque id pretium est. Quisque convallis nisi a diam malesuada
    mollis. Aliquam at enim ligula.
  </p>
```

```
</div>

<div class="invisible-scrollbar">
  <h3>Invisible scrollbar</h3>
  <p>
    Thisisaveeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeerylongword
  </p>
</div>

<div class="mostly-customized-scrollbar">
  <h3>Custom scrollbar</h3>
  <p>
    Thisisaveeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeerylongword<br />
    And pretty tall<br />
    thing with weird scrollbars.<br />
    Who thought scrollbars could be made weird?
  </p>
</div>
```

Result

Adding a fallback for scrollbar styles

You can use a `@supports` at-rule to detect if a browser supports the standard `scrollbar-color` and `scrollbar-width` properties, and otherwise use a fallback with `::-webkit-scrollbar-*` pseudo-elements. The following example shows how to apply colors to scrollbars using `scrollbar-color` if supported and `::-webkit-scrollbar-*` pseudo-elements if not.

HTML

```
<div class="scroll-box">
  <h1>Yoshi</h1>
  <p>
    Yoshi is a fictional dinosaur who appears in video games published by
    Nintendo. Yoshi debuted in Super Mario World (1990) on the SNES as Mario and
    Luigi's sidekick.
  </p>
  <p>
    Throughout the mainline Super Mario series, Yoshi typically serves as
    Mario's trusted steed.
  </p>
  <p>
    With a gluttonous appetite, Yoshi can gobble enemies with his long tongue,
    and lay eggs that doubly function as projectiles.
  </p>
</div>
```

CSS

```
.scroll-box {
  overflow: auto;
  width: 20rem;
  height: 5rem;
  border: 2px solid cornflowerblue;
  margin: 2rem auto;
  font-family: monospace;
}
```

```
/* For browsers that support `scrollbar-*` properties */
@supports (scrollbar-color: auto) {
  .scroll-box {
    scrollbar-color: aquamarine cornflowerblue;
  }
}

/* Otherwise, use `::-webkit-scrollbar-*` pseudo-elements */
@supports selector(::-webkit-scrollbar) {
  .scroll-box::-webkit-scrollbar {
    background: aquamarine;
  }
  .scroll-box::-webkit-scrollbar-thumb {
    background: cornflowerblue;
  }
}
```

Result

In the example below, you can scroll the bordered box vertically to see the effect of styling the scrollbar.

Specifications

Not part of any standard.

Browser compatibility

See also

- `scrollbar-width`
- `scrollbar-color`
- Don't use custom scrollbars (2023)
- Scrollbar styling on developer.chrome.com (2024)
- Styling Scrollbars on WebKit.org (2009)

::-webkit-search-cancel-button CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-search-cancel-button`** CSS pseudo-element represents a button (the "cancel button") at the edge of an `<input>` of `type="search"` which clears away the current value of the `<input>` element. This button and pseudo-element are non-standard, supported only in WebKit and Blink, hence the vendor prefix. The clear button is only shown on non-empty search `<input>` elements.

Syntax

```
selector::-webkit-search-cancel-button {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `::-webkit-search-results-button`

::-webkit-search-results-button CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-search-results-button`** CSS pseudo-element represents a button (the "search results button") at the left edge of an `<input>` of `type="search"` which when clicked displays a menu which allows the user to choose from previous recent search queries. This button and pseudo-element are non-standard, supported only in WebKit and Blink, hence the vendor prefix. The search results button is only shown on search `<input>` elements that have a `results` attribute.

Syntax

```
selector::-webkit-search-results-button {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `::-webkit-search-cancel-button`

::-webkit-slider-runnable-track CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-slider-runnable-track`** CSS pseudo-element represents the "track" (the groove in which the indicator slides) of an <input type="range">.

Syntax

```
::-webkit-slider-runnable-track {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `::-webkit-slider-thumb`
- Similar pseudo-elements used by other browsers:
  - `::-moz-range-track`
- CSS-Tricks: Styling Cross-Browser Compatible Range Inputs with CSS
- QuirksMode: Styling and scripting sliders

::-webkit-slider-thumb CSS pseudo-element

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`::-webkit-slider-thumb`** CSS pseudo-element represents the "thumb" that the user can move within the "groove" of an `<input>` of `type="range"` to alter its numerical value.

Syntax

```
::-webkit-slider-thumb {
  /* ... */
}
```

Specifications

Not part of any standard.

See also

- `::-webkit-slider-runnable-track`
- Similar pseudo-elements used by other browsers:
  - `::-moz-range-thumb`
- CSS-Tricks: Styling Cross-Browser Compatible Range Inputs with CSS
- QuirksMode: Styling and scripting sliders

- Couple of Gotchas to Watch-out For

::after CSS pseudo-element

In CSS, **`::after`** creates a pseudo-element that is the last child of the selected element. It is often used to add cosmetic content to an element with the `content` property. It is inline by default.

Try it

```
a::after {
  content: " (" attr(href) ")";
}

.dead-link {
  text-decoration: line-through;
}

.dead-link::after {
  content: url("/shared-assets/images/examples/warning.svg");
  display: inline-block;
  width: 12px;
  height: 12px;
}
```

```
<p>
  The sailfish is named for its sail-like dorsal fin and is widely considered
  the fastest fish in the ocean.
  <a href="https://en.wikipedia.org/wiki/Sailfish"
    >You can read more about it here</a
  >.
</p>

<p>
  The red lionfish is a predatory scorpionfish that lives on coral reefs of the
  Indo-Pacific Ocean and more recently in the western Atlantic.
  <a href="" class="dead-link">You can read more about it here</a>.
</p>
```

Syntax

```
::after {
  content: /* value */;
  /* properties */
}
```

Description

The `::after` pseudo-element is an inline box generated as an immediate child of the element it is associated with, or the "originating element". It is often used to add cosmetic content to an element via the `content` property, such as icons, quote marks, or other decoration.

`::after` pseudo-elements can't be applied to *replaced elements* such as `<img>`, whose contents are determined by external resources and not affected by the current document's styles.

An `::after` pseudo-element with a `display` value of `list-item` behaves like a list item, and can therefore generate a `::marker` pseudo-element just like an `<li>` element.

If the `content` property is not specified, has an invalid value, or has `normal` or `none` as a value, then the `::after` pseudo-element is not rendered. It behaves as if `display: none` is set.

> **Note:** The Selectors Level 3 specification introduced the double-colon notation `::after` to distinguish pseudo-classes from pseudo-elements. Browsers also accept single-colon notation `:after`, introduced in CSS2.

By default, the `::before` and `::after` pseudo-elements share the same stacking context as their parent. If no `z-index` is explicitly set, the `::after` pseudo-element's generated content will appear above the `::before` pseudo-element's generated content because `::after` is rendered later in the DOM flow.

Accessibility

Using an `::after` pseudo-element to add content is discouraged, as it is not reliably accessible to screen readers.

Examples

Basic usage

Let's create two classes: one for boring paragraphs and one for exciting ones. We can use these classes to add pseudo-elements to the end of paragraphs.

HTML

```
<p class="boring-text">Here is some plain old boring text.</p>
<p>Here is some normal text that is neither boring nor exciting.</p>
<p class="exciting-text">Contributing to MDN is fun.</p>
```

CSS

```
.exciting-text::after {
  content: " <- EXCITING!";
  color: darkgreen;
  font-weight: bolder;
}

.boring-text::after {
  content: " <- BORING";
  color: darkviolet;
  font-weight: bolder;
}
```

Result

Decorative example

We can style text or images in the `content` property almost any way we want.

HTML

```
<span class="ribbon">Look at the orange box after this text. </span>
```

CSS

```
.ribbon {
  background-color: #5bc8f7;
}

.ribbon::after {
  content: "This is a fancy orange box.";
  background-color: #ffba10;
  border-color: black;
  border-style: dotted;
}
```

Result

Tooltips

This example uses `::after`, in conjunction with the `attr()` CSS expression and a `data-description` custom data attribute, to create tooltips. No JavaScript is required!

We can also support keyboard users with this technique, by adding a `tabindex` of `0` to make each `span` keyboard focusable, and using a CSS `:focus` selector. This shows how flexible `::before` and `::after` can be, though for the most accessible experience a semantic disclosure widget created in some other way (such as with details and summary elements) is likely to be more appropriate.

HTML

```
<p>
  Here we have some
  <span tabindex="0" data-description="collection of words and punctuation">
    text
  </span>
  with a few
  <span tabindex="0" data-description="small popups that appear when hovering">
    tooltips</span
  >.
</p>
```

CSS

```
span[data-description] {
  position: relative;
  text-decoration: underline;
  color: blue;
  cursor: help;
}

span[data-description]:hover::after,
span[data-description]:focus::after {
  content: attr(data-description);
  position: absolute;
  left: 0;
  top: 24px;
  min-width: 200px;
  border: 1px #aaaaaa solid;
  border-radius: 10px;
  background-color: #ffffcc;
  padding: 12px;
  color: black;
  font-size: 14px;
  z-index: 1;
}
```

Result

<code>::after::marker</code> nested pseudo-elements

The `::after::marker` nested pseudo-element selects the list `::marker` of an `::after` pseudo-element that is itself a list item, that is, it has its `display` property set to `list-item`.

In this demo, we generate extra list items before and after a list navigation menu using `::before` and `::after` (setting them to `display: list-item` so they behave like list items). We then use `ul::before::marker` and `ul::after::marker` to give their list markers a different color.

HTML

```
<ul>
  <li><a href="#">Introduction</a></li>
  <li><a href="#">Getting started</a></li>
  <li><a href="#">Understanding the basics</a></li>
</ul>
```

CSS

```
ul {
  font-size: 1.5rem;
  font-family: "Helvetica", "Arial", sans-serif;
}

ul::before,
ul::after {
  display: list-item;
  color: orange;
}

ul::before {
  content: "Start";
}

ul::after {
  content: "End";
}
```

```
ul::before::marker,
ul::after::marker {
  color: red;
}
```

Result

While the list bullets of the three navigation items are generated because they are `<li>` elements, "Start" and "End" have been inserted via pseudo-elements and `::marker` is used to style their bullets.

See also

- `::before`
- `content`

::backdrop CSS pseudo-element

The **`::backdrop`** CSS pseudo-element is a box the size of the viewport, which is rendered immediately beneath any element being presented in the top layer.

Try it

```
button {
  font-size: 1.2rem;
  padding: 5px 15px;
}

dialog::backdrop {
  background-color: salmon;
}
```

```
<button id="showDialogBtn">Show a dialog</button>

<dialog id="favDialog">
  <form method="dialog">
    <p>The background shown outside of this dialog is a backdrop.</p>
    <button id="confirmBtn">Close the dialog</button>
  </form>
</dialog>
```

```
const showDialogBtn = document.getElementById("showDialogBtn");
const favDialog = document.getElementById("favDialog");

showDialogBtn.addEventListener("click", () => favDialog.showModal());
```

Syntax

```
::backdrop {
  /* ... */
}
```

Description

Backdrops appear in the following instances:

- Elements which have been placed in fullscreen mode using the Fullscreen API `Element.requestFullscreen()` method.
- `<dialog>` elements that have been shown in the top layer via a `HTMLDialogElement.showModal()` call.
- Popover elements that have been shown in the top layer via a `HTMLElement.showPopover()` call.

When multiple elements have been placed into the top layer, each one has its own `::backdrop` pseudo-element.

```
/* Backdrop is only displayed when dialog is opened with dialog.showModal() */
dialog::backdrop {
  background: rgb(255 0 0 / 25%);
}
```

Elements are placed in a last-in/first out (LIFO) stack in the top layer. The `::backdrop` pseudo-element makes it possible to obscure, style, or completely hide everything located below a top layer element.

`::backdrop` neither inherits from nor is inherited by any other elements. No restrictions are made on what properties apply to this pseudo-element.

Examples

Styling a modal dialog's backdrop

In this example, we use the `::backdrop` pseudo-element to style the backdrop that is used when a modal `<dialog>` is open.

HTML

We include a `<button>` that, when clicked, will open the included `<dialog>`. When the `<dialog>` is opened, we give focus to the button that closes the dialog:

```
<dialog>
  <button autofocus>Close</button>
  <p>This modal dialog has a beautiful backdrop!</p>
</dialog>
<button>Show the dialog</button>
```

CSS

We add a background to the backdrop, creating a colorful donut using CSS gradients:

```
::backdrop {
  background-image:
    radial-gradient(
      circle,
      white 0 5vw,
      transparent 5vw 20vw,
      white 20vw 22.5vw,
      #eeeeee 22.5vw
    ),
```

```
    conic-gradient(
      #272b66 0 50grad,
      #2d559f 50grad 100grad,
      #9ac147 100grad 150grad,
      #639b47 150grad 200grad,
      #e1e23b 200grad 250grad,
      #f7941e 250grad 300grad,
      #662a6c 300grad 350grad,
      #9a1d34 350grad 400grad,
      #43a1cd 100grad 150grad,
      #ba3e2e
    );
}
```

JavaScript

The dialog is opened modally using the `.showModal()` method and closed using the `.close()` method.

```
const dialog = document.querySelector("dialog");
const showButton = document.querySelector("dialog + button");
const closeButton = document.querySelector("dialog button");

// "Show the dialog" button opens the dialog modally
showButton.addEventListener("click", () => {
  dialog.showModal();
});

// "Close" button closes the dialog
closeButton.addEventListener("click", () => {
  dialog.close();
});
```

Results

See also

- `:fullscreen`
- `:xr-overlay`
- CSS positioned layout module
- CSS pseudo-elements module
- `<dialog>`
- `popover` HTML global attribute
- Fullscreen API
- Popover API

::before CSS pseudo-element

In CSS, **`::before`** creates a pseudo-element that is the first child of the selected element. It is often used to add cosmetic content to an element with the `content` property. It is inline by default.

Try it

```
a {
  color: blue;
  text-decoration: none;
}

a::before {
  content: "🔗";
}

.local-link::before {
  content: url("/shared-assets/images/examples/firefox-logo.svg");
  display: inline-block;
  width: 15px;
  height: 15px;
  margin-right: 5px;
}
```

```
<p>
  Learning resources for web developers can be found all over the internet. Try
  out
  <a href="https://web.dev/">web.dev</a>,
  <a href="https://www.w3schools.com/">w3schools.com</a> or our
  <a href="https://developer.mozilla.org/" class="local-link">MDN web docs</a>.
</p>
```

Syntax

```
::before {
  content: /* value */;
  /* properties */
}
```

Description

The `::before` pseudo-element is an inline box generated as an immediate child of the element it is associated with, or the "originating element". It is often used to add cosmetic content to an element via the `content` property, such as icons, quote marks, or other decoration.

`::before` pseudo-elements can't be applied to *replaced elements* such as `<img>`, whose contents are determined by external resources and not affected by the current document's styles.

A `::before` pseudo-element with a `display` value of `list-item` behaves like a list item, and can therefore generate a `::marker` pseudo-element just like an `<li>` element.

If the `content` property is not specified, has an invalid value, or has `normal` or `none` as a value, then the `::before` pseudo-element is not rendered. It behaves as if `display: none` is set.

> **Note:** The Selectors Level 3 specification introduced the double-colon notation `::before` to distinguish pseudo-classes from pseudo-elements. Browsers also accept single-colon notation `:before`, introduced in CSS2.

By default, the `::before` and `::after` pseudo-elements share the same stacking context as their parent. If no `z-index` is explicitly set, the `::after` pseudo-element's generated content will appear above the `::before` pseudo-element's generated content because `::after` is rendered later in the DOM flow.

Accessibility

Using a `::before` pseudo-element to add content is discouraged, as it is not reliably accessible to screen readers.

Examples

Quotation marks

One example of using `::before` pseudo-elements is to provide quotation marks. Here we use both `::before` and `::after` to insert quotation characters.

HTML

```
<q>Some quotes</q>, he said, <q>are better than none.</q>
```

CSS

```
q::before {
  content: "«";
  color: blue;
}

q::after {
  content: "»";
  color: red;
}
```

Result

Decorative example

We can style text or images in the `content` property almost any way we want.

HTML

```
<span class="ribbon">Notice where the orange box is.</span>
```

CSS

```
.ribbon {
  background-color: #5bc8f7;
}

.ribbon::before {
  content: "Look at this orange box.";
  background-color: #ffba10;
  border-color: black;
  border-style: dotted;
}
```

Result

To-do list

In this example we will create a to-do list using pseudo-elements. This method can often be used to add small touches to the UI and improve user experience.

HTML

```
<ul>
  <li>Buy milk</li>
  <li>Take the dog for a walk</li>
  <li>Exercise</li>
  <li>Write code</li>
  <li>Play music</li>
  <li>Relax</li>
</ul>
```

CSS

```
li {
  list-style-type: none;
  position: relative;
  margin: 2px;
  padding: 0.5em 0.5em 0.5em 2em;
  background: lightgrey;
  font-family: sans-serif;
}

li.done {
  background: #ccff99;
}

li.done::before {
  content: "";
  position: absolute;
  border-color: #009933;
  border-style: solid;
  border-width: 0 0.3em 0.25em 0;
  height: 1em;
  top: 1.3em;
  left: 0.6em;
  margin-top: -1em;
  transform: rotate(45deg);
  width: 0.5em;
}
```

JavaScript

```
const list = document.querySelector("ul");
list.addEventListener("click", (ev) => {
  if (ev.target.tagName === "LI") {
    ev.target.classList.toggle("done");
  }
});
```

Here is the above code example running live. Note that there are no icons used, and the check-mark is actually the `::before` that has been styled in CSS. Go ahead and get some stuff done.

Result

Unicode escape sequences

As generated content is CSS, not HTML, you **can't** use markup entities in content values. If you need to use a special character, and can't enter it literally into your CSS content string, use a unicode escape sequence. This consists of a backslash followed by the character's hexadecimal unicode value.

HTML

```
<ol>
  <li>Crack Eggs into bowl</li>
  <li>Add Milk</li>
  <li>Add Flour</li>
  <li aria-current="step">Mix thoroughly into a smooth batter</li>
  <li>Pour a ladleful of batter onto a hot, greased, flat frying pan</li>
  <li>Fry until the top of the pancake loses its gloss</li>
  <li>Flip it over and fry for a couple more minutes</li>
  <li>serve with your favorite topping</li>
</ol>
```

CSS

```
li {
  padding: 0.5em;
}

li[aria-current="step"] {
  font-weight: bold;
}

li[aria-current="step"]::before {
  content: "\21E8 "; /* Unicode escape sequence for "Rightwards White Arrow" */
  display: inline;
}
```

Result

<code>::before::marker</code> nested pseudo-elements

The `::before::marker` nested pseudo-element selects the list `::marker` of a `::before` pseudo-element that is itself a list item, that is, it has its `display` property set to `list-item`.

In this demo, we generate extra list items before and after a list navigation menu using `::before` and `::after` (setting them to `display: list-item` so they behave like list items). We then use `ul::before::marker` and `ul::after::marker` to give their list markers a different color.

HTML

```
<ul>
  <li><a href="#">Introduction</a></li>
  <li><a href="#">Getting started</a></li>
  <li><a href="#">Understanding the basics</a></li>
</ul>
```

CSS

```
ul {
  font-size: 1.5rem;
  font-family: "Helvetica", "Arial", sans-serif;
}

ul::before,
ul::after {
  display: list-item;
  color: orange;
}

ul::before {
  content: "Start";
}

ul::after {
  content: "End";
}

ul::before::marker,
ul::after::marker {
  color: red;
}
```

Result

While the list bullets of the three navigation items are generated because they are `<li>` elements, "Start" and "End" have been inserted via pseudo-elements and `::marker` is used to style their bullets.

See also

- `::after`
- `content`

::checkmark CSS pseudo-element

The **`::checkmark`** CSS pseudo-element targets the checkmark placed inside the currently-selected `<option>` element of a customizable select element. It can be used to provide a visual indication of which option is selected.

Try it

```
<label for="pet-select">
  Select pet:
</label>
<br />
<select id="pet-select">
  <option value="cat">Cat</option>
  <option value="dog">Dog</option>
  <option value="chicken">
    Chicken
  </option>
</select>
```

```
option::checkmark {
  color: orange;
  font-size: 1.5rem;
}

select,
::picker(select) {
  appearance: base-select;
  width: 200px;
}

select {
  border: 2px solid #dddddd;
  background: #eeeeee;
  padding: 10px;
}

::picker(select) {
  border: none;
}

option {
  border: 2px solid #dddddd;
  background: #eeeeee;
  padding: 10px;
}

option:first-of-type {
  border-radius: 8px 8px 0 0;
}

option:last-of-type {
  border-radius: 0 0 8px 8px;
}

option:nth-of-type(odd) {
  background: white;
}

option:not(option:last-of-type) {
  border-bottom: none;
}
```

Syntax

```
::checkmark {
  /* ... */
}
```

Description

The `::checkmark` pseudo-element targets the checkmark placed inside a customizable select element's currently-selected `<option>`.

It is only available to target when the originating element has a picker and has base appearance set on it via the `appearance` property `base-select` value. Its generated box appears before any boxes generated by the `::before` pseudo-element. The icon can be customized using the `content` property.

The `::checkmark` selector is useful for example if you want to hide the checkmark, use a custom icon, or adjust the checkmark's rendering position inside `<option>` elements.

> **Note:** The `::checkmark` pseudo-element is not included in the accessibility tree, so any generated `content` set on it will not be announced by assistive technologies. You should still make sure that any new icon you set visually makes sense for its intended purpose.

Examples

Customizing the checkmark

To opt-in to customizable select functionality, the `<select>` element and its picker both need to have an `appearance` value of `base-select` set on them:

```
select,
::picker(select) {
  appearance: base-select;
}
```

Assuming flexbox is being used to lay out the `<option>` elements (which is true in **current implementations** of customizable selects), you could then move the checkmark from the start of the row to the end by setting an `order` value on it greater than `0`, and aligning it to the end of the row using an `auto` `margin-left` value (see Alignment and auto margins).

The value of the `content` property could also be set to a different emoji to change the displayed icon.

```
option::checkmark {
  order: 1;
```

```
  margin-left: auto;
  content: "☑";
}
```

See Styling the current selection checkmark for a full example that uses this code, along with a live example rendering.

See also

- `<select>`, `<option>`, `<optgroup>`, `<label>`, `<button>`, `<selectedcontent>`
- `appearance`
- `::picker(select)`, `::picker-icon`
- `:open`, `:checked`
- Customizable select elements

::column CSS pseudo-element

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::column`** CSS pseudo-element represents the individual columns generated when a container is set to display its content in multiple columns via CSS multi-column layout. The `::column` pseudo-element enables applying styles that do not affect the layout to these generated fragments.

Syntax

```
::column {
  /* ... */
}
```

Description

When CSS multi-column layout is used to lay out a container's content in multiple columns (for example, using the `column-count` property), `::column` pseudo-elements are generated to contain each individual column.

The `::column` pseudo-element only accepts scroll snap properties that apply to elements inside a scroll container, including `scroll-margin`, `scroll-snap-align`, and `scroll-snap-stop`.

The `::column` pseudo-element can have a `::scroll-marker` pseudo-element. Other pseudo-elements like `::before` and `::after` are not generated on `::column`. Applying `::column::scroll-marker` creates a marker for every column of the originating scroll container, with the `::scroll-marker` pseudo-elements inheriting from the `::column` pseudo-element's originating element rather than the `::column` itself.

This is useful for CSS carousels — `::column` can be used to generate `::scroll-marker` pseudo-elements for each column, and set them as snap targets using CSS scroll snap.

While the styling that can be applied to `::column` is very limited, it may be expanded in the future. Any properties and values supported in the future will be limited to ones that do not affect layout.

Examples

Scrolling column layout

This demo creates a responsive container that snaps each "page" of content into place. It uses the `columns` property and the `::column` pseudo-element to create content columns that span the full width of their parent scroll container, which can be scrolled horizontally. Each column contains one or more list items, which vary in number depending on the viewport width.

HTML

The HTML consists of an unordered list, with each list item containing some sample content:

```
<ul>
...
  <li>
    <h2>Item 1</h2>
  </li>
...
</ul>
```

```
<ul>
  <li>
    <h2>Item 1</h2>
  </li>
  <li>
    <h2>Item 2</h2>
  </li>
  <li>
    <h2>Item 3</h2>
  </li>
  <li>
    <h2>Item 4</h2>
  </li>
  <li>
    <h2>Item 5</h2>
  </li>
  <li>
    <h2>Item 6</h2>
  </li>
  <li>
    <h2>Item 7</h2>
  </li>
  <li>
    <h2>Item 8</h2>
  </li>
  <li>
    <h2>Item 9</h2>
  </li>
  <li>
    <h2>Item 10</h2>
  </li>
  <li>
    <h2>Item 11</h2>
  </li>
  <li>
    <h2>Item 12</h2>
  </li>
```

```
  <li>
    <h2>Item 13</h2>
  </li>
  <li>
    <h2>Item 14</h2>
  </li>
  <li>
    <h2>Item 15</h2>
  </li>
</ul>
```

CSS

The list is given a fixed `height` and a `width` of `100vw` to make it span the full width of the viewport. An `overflow-x` value of `scroll` is then set so that the content will scroll horizontally, and CSS scroll snap is used to snap to each item or "page" — a `scroll-snap-type` value of `x mandatory` is used to make the list into a scroll snap container. Finally, a `columns` value of `1` is set to force the list contents to display as a single column. A `text-align` value of `center` is also applied, to align the content with the center of the list.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Helvetica", "Arial", sans-serif;
}
```

```
ul {
  width: 100vw;
  height: 300px;
  padding: 10px;

  overflow-x: scroll;
  scroll-snap-type: x mandatory;

  columns: 1;
  text-align: center;
}
```

The list items are then styled:

- A `display` value of `inline-block` is set to make the list items sit alongside one another and make the list scroll horizontally.
- A fixed `width` and `height` has been set on them.
- A `text-align` value of `left` is set on them to override the `text-align: center` set on the parent container, so the item content will be left-aligned.
- Every even-numbered list item is given a different background-color via `:nth-child()`, so that it is easier to see the scrolling effect.

```
li {
  list-style-type: none;

  display: inline-block;
  height: 100%;
  width: 200px;
  text-align: left;

  background-color: #eeeeee;
  outline: 1px solid #dddddd;
  padding: 0 20px;
  margin: 0 10px;
}

li:nth-child(even) {
  background-color: cyan;
}
```

The `scroll-snap-align` property is set on the `::column` pseudo-elements — which represent the content columns generated by the `columns` property — so that when scrolled, a column is snapped to be centered within the scroll container.

```
ul::column {
  scroll-snap-align: center;
}
```

Result

Column-based carousel with scroll markers

Expanding on the previous example, we will create scroll markers to enable direct navigation to different columns by applying a `scroll-marker-group` to the container and a `::scroll-marker` to each `::column` pseudo-element. The HTML remains unchanged.

CSS

We create a scroll marker group with the `scroll-marker-group` property, placing the group after all the content:

```
ul {
  scroll-marker-group: after;
}
```

We then apply styles to the `::scroll-marker-group` pseudo-element to lay out the scroll markers in the center of the row with a `0.4em` gap between each one:

```
::scroll-marker-group {
  display: flex;
  gap: 0.4em;
  place-content: center;
}
```

Finally, we use the `::scroll-marker` pseudo-element to create a round, transparent marker for each list item with a black border, then style the marker of the currently-scrolled element differently from the others, targeting the marker with the `:target-current` pseudo-class:

```
ul::column::scroll-marker {
  content: "";
  width: 16px;
  height: 16px;
  background-color: transparent;
  border: 2px solid black;
  border-radius: 10px;
}

ul::column::scroll-marker:target-current {
  background-color: black;
}
```

Result

Try pressing the scroll markers to jump straight to each page. Note how the current marker is highlighted so you can see where you are in the pagination. Also try tabbing to the scroll marker group, then use the cursor keys to cycle through each page.

See Creating CSS carousels for more carousel examples.

See also

- `columns`
- `::scroll-marker`
- `::scroll-marker-group`
- `:target-current`
- Creating CSS carousels
- CSS multi-column layout module
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

::cue CSS pseudo-element

The **`::cue`** CSS pseudo-element matches WebVTT cues within a selected element. This can be used to style captions and other cues in media with VTT tracks.

Try it

```
video {
  width: 100%;
}

video::cue {
  font-size: 1rem;
  color: yellow;
}

::cue(u) {
  color: red;
}
```

```
<video controls src="/shared-assets/videos/friday.mp4">
  <track
    default
    kind="captions"
    srclang="en"
    src="/shared-assets/misc/friday.vtt" />
  Sorry, your browser doesn't support embedded videos.
</video>
```

The properties are applied to the entire set of cues as if they were a single unit. The only exception is that `background` and its longhand properties apply to each cue individually, to avoid creating boxes and obscuring unexpectedly large areas of the media.

In the example above, the `::cue(u)` selector selects all the `<u>` elements inside the cue text.

Syntax

```
::cue | ::cue(<selector>) {
  /* ... */
}
```

Permitted properties

Rules whose selectors include this element may only use the following CSS properties:

- `background`
- `background-attachment`
- `background-clip`
- `background-color`
- `background-image`
- `background-origin`
- `background-position`
- `background-repeat`
- `background-size`
- `color`
- `font`
- `font-family`
- `font-size`
- `font-stretch`
- `font-style`
- `font-variant`
- `font-weight`
- `line-height`
- `opacity`
- `outline`
- `outline-color`

- `outline-style`
- `outline-width`
- `ruby-position`
- `text-combine-upright`
- `text-decoration`
- `text-decoration-color`
- `text-decoration-line`
- `text-decoration-style`
- `text-decoration-thickness`
- `text-shadow`
- `visibility`
- `white-space`

Examples

Styling WebVTT cues as white-on-black

The following CSS sets the cue style so that the text is white and the background is a translucent black box.

```
::cue {
  color: white;
  background-color: rgb(0 0 0 / 60%);
}
```

Styling WebVTT internal node objects

Cue text can include *internal node objects* as the tags (similar to HTML elements) `<c>`, `<i>`, `<b>`, `<u>`, `<ruby>`, `<rt>`, `<v>`, and `<lang>`. The `::cue()` selector can be used to apply styles to content inside these tags to customize how the WebVTT track is displayed. Consider the following cue text that uses the `<u>` tag to underline some text:

```
00:00:01.500 --> 00:00:02.999 line:80%
Tell me, is the <u>lord of the universe</u> in?
```

The following CSS rule customizes the text inside the `<u>` tag with a color and a text-decoration:

```
::cue(u) {
  color: red;
  text-decoration: wavy overline lime;
}
```

See also

- Web Video Tracks Format (WebVTT)
- `<track>`, `<video>`

::details-content CSS pseudo-element

The **`::details-content`** CSS pseudo-element represents the expandable/collapsible contents of a `<details>` element.

Try it

```
details[open]::details-content {
  color: dodgerblue;
  padding: 0.5em;
  border: thin solid grey;
}
```

```
<details open>
  <summary>Example summary</summary>
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
  <p>
    Architecto cupiditate ea optio modi quas sequi, esse libero asperiores
    debitis eveniet commodi hic ad.
  </p>
</details>
```

Syntax

```
selector::details-content
```

Examples

Basic example

This example demonstrates basic usage of the `::details-content` pseudo-element to style the content of a `<details>` element.

HTML

Our `<details>` element includes a `<summary>` element, whose contents will always be visible. The details content includes a `<p>` element.

```
<details>
  <summary>Click me</summary>
  <p>Here is some content</p>
</details>
```

CSS

We set a `background-color` on the `::details-content` pseudo-element:

```
details::details-content {
  background-color: #a29bfe;
}
```

Result

Click on the summary to view the detail contents.

Transition example

In this example the `::details-content` pseudo-element is used to set a transition on the content of the `<details>` element so that it smoothly fades into view when expanded, and fades out again when collapsed.

HTML

The HTML is the same as in the previous example.

```
<details>
  <summary>Click me</summary>
  <p>Here is some content</p>
</details>
```

CSS

To achieve our transition, we specify two separate transitions inside the `transition` shorthand property:

- The `opacity` property is given a basic transition over `600ms` to create the fade-in/fade-out effect.
- The `content-visibility` property (which is toggled between `hidden` and `visible` when the `<details>` content is expanded/collapsed) is given a `600ms` transition with the `transition-behavior` value `allow-discrete` specified. This opts the browser into having a transition started on `content-visibility`, the animation behavior of which is discrete. The effect is that the content is visible for the entire duration of the transition, allowing other transitions to be seen. If this transition was not included, the content would immediately disappear when the `<details>` content was collapsed — you wouldn't see the smooth fade-out.

```
details::details-content {
  opacity: 0;
  transition:
    opacity 600ms,
    content-visibility 600ms allow-discrete;
}

details[open]::details-content {
  opacity: 1;
}
```

Result

To see the animation, toggle the visibility of the detail contents by clicking on the summary.

See also

- `<details>`
- `<summary>`

::file-selector-button CSS pseudo-element

The **`::file-selector-button`** CSS pseudo-element represents the button of an `<input>` of `type="file"`.

Try it

```
input {
  margin-top: 1rem;
}

input::file-selector-button {
  font-weight: bold;
  color: dodgerblue;
  padding: 0.5em;
  border: thin solid grey;
  border-radius: 3px;
}
```

```
<label for="avatar">Choose a profile picture:</label><br />

<input id="avatar" type="file" name="avatar" accept="image/png, image/jpeg" />
```

Syntax

```
::file-selector-button {
  /* ... */
}
```

Examples

Basic example

HTML

```
<form>
  <label for="fileUpload">Upload file</label>
  <input type="file" id="fileUpload" />
</form>
```

CSS

```
form {
  display: flex;
  gap: 1em;
  align-items: center;
}
```

```
input[type="file"]::file-selector-button {
  border: 2px solid #6c5ce7;
  padding: 0.2em 0.4em;
  border-radius: 0.2em;
  background-color: #a29bfe;
  transition: 1s;
}
```

```
input[type="file"]::file-selector-button:hover {
  background-color: #81ecec;
  border: 2px solid #00cec9;
}
```

Result

Note that `::file-selector-button` is a whole element, and as such matches the rules from the UA stylesheet. In particular, fonts and colors won't necessarily inherit from the `input` element.

Fallback example

HTML

```
<form>
  <label for="fileUpload">Upload file</label>
  <input type="file" id="fileUpload" />
</form>
```

CSS

```
form {
  display: flex;
  gap: 1em;
  align-items: center;
}
```

```
input[type="file"]::file-selector-button {
  border: 2px solid #6c5ce7;
  padding: 0.2em 0.4em;
  border-radius: 0.2em;
  background-color: #a29bfe;
  transition: 1s;
}

input[type="file"]::-ms-browse:hover {
  background-color: #81ecec;
  border: 2px solid #00cec9;
}

input[type="file"]::-webkit-file-upload-button:hover {
  background-color: #81ecec;
  border: 2px solid #00cec9;
}

input[type="file"]::file-selector-button:hover {
  background-color: #81ecec;
  border: 2px solid #00cec9;
}
```

Result

See also

- WebKit CSS extensions
- File and Directory Entries API
- `<input type="file">`

::first-letter CSS pseudo-element

The **`::first-letter`** CSS pseudo-element applies styles to the first letter of the first line of a block container, but only when not preceded by other content (such as images or inline tables).

Try it

```
p::first-letter {
  font-size: 1.5rem;
  font-weight: bold;
  color: brown;
}
```

```
<p>
  Scientists exploring the depths of Monterey Bay unexpectedly encountered a
  rare and unique species of dragonfish. This species is the rarest of its
  species.
</p>

<p>
  When Robison and a team of researchers discovered this fish, they were aboard
  a week-long expedition.
</p>
```

The first letter of an element is not always trivial to identify:

- Punctuation that precedes or immediately follows the first letter is included in the match. Punctuation includes any Unicode character defined in the *open* (Ps), *close* (Pe), *initial quote* (Pi), *final quote* (Pf), and *other punctuation* (Po) classes.
- Some languages have digraphs that are always capitalized together, like the `IJ` in Dutch. In these cases, both letters of the digraph should be matched by the `::first-letter` pseudo-element.
- A combination of the `::before` pseudo-element and the `content` property may inject some text at the beginning of the element. In that case, `::first-letter` will match the first letter of this generated content.

> **Note:** CSS introduced the `::first-letter` notation (with two colons) to distinguish pseudo-classes from pseudo-elements. For backward compatibility, browsers also accept `:first-letter`, introduced earlier.
>
> Browser support for digraphs such as `IJ` in Dutch is poor. Check the compatibility table below to see the current state of support.

Allowable properties

Only a small subset of CSS properties can be used with the `::first-letter` pseudo-element:

- All font properties: `font`, `font-style`, `font-feature-settings`, `font-kerning`, `font-language-override`, `font-stretch`, `font-synthesis`, `font-variant`, `font-variant-alternates`, `font-variant-caps`, `font-variant-east-asian`, `font-variant-ligatures`, `font-variant-numeric`, `font-variant-position`, `font-weight`, `font-size`, `font-size-adjust`, `line-height` and `font-family`
- All background properties: `background`, `background-color`, `background-image`, `background-clip`, `background-origin`, `background-position`, `background-repeat`, `background-size`, `background-attachment`, and `background-blend-mode`
- All margin properties: `margin`, `margin-top`, `margin-right`, `margin-bottom`, `margin-left`
- All padding properties: `padding`, `padding-top`, `padding-right`, `padding-bottom`, `padding-left`
- All border properties: the shorthands `border`, `border-style`, `border-color`, `border-width`, `border-radius`, `border-image`, and the longhands properties
- The `color` property
- The `text-decoration`, `text-shadow`, `text-transform`, `letter-spacing`, `word-spacing` (when appropriate), `line-height`, `text-decoration-color`, `text-decoration-line`, `text-decoration-style`, `box-shadow`, `float`, `vertical-align` (only if `float` is `none`) CSS properties

Syntax

```
::first-letter {
  /* ... */
}
```

Examples

Basic drop cap

In this example we will use the `::first-letter` pseudo-element to create a drop cap effect on the first letter of the paragraph coming right after the `<h2>`.

HTML

```
<h2>My heading</h2>
<p>
  Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
  eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
  voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita
  kasd gubergren, no sea takimata sanctus est.
</p>
<p>
  Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie
  consequat.
</p>
```

CSS

```
p {
  width: 500px;
  line-height: 1.5;
}

h2 + p::first-letter {
  color: white;
  background-color: black;
  border-radius: 2px;
  box-shadow: 3px 3px 0 red;
  font-size: 250%;
  padding: 6px 3px;
  margin-right: 6px;
  float: left;
}
```

Result

Effect on special punctuation and non-Latin characters

This example illustrates the effect of `::first-letter` on special punctuation and non-Latin characters.

HTML

```
<p>
  Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie
  consequat.
</p>
<p>-The beginning of a special punctuation mark.</p>
<p>_The beginning of a special punctuation mark.</p>
<p>"The beginning of a special punctuation mark.</p>
<p>'The beginning of a special punctuation mark.</p>
<p>*The beginning of a special punctuation mark.</p>
<p>#The beginning of a special punctuation mark.</p>
<p>「特殊的汉字标点符号开头。</p>
<p>《特殊的汉字标点符号开头。</p>
<p>"特殊的汉字标点符号开头。</p>
```

CSS

```
p::first-letter {
  color: red;
  font-size: 150%;
}
```

Result

Styling first letter in SVG text element

In this example, we use the `::first-letter` pseudo-element to style the first letter of an SVG `<text>` element.

> **Note:** At time of writing this feature has limited support.

HTML

```
<svg viewBox="0 0 300 40">
  <text y="30">First letter in &lt;text&gt; SVG</text>
</svg>
```

CSS

```
text {
  font-family: sans-serif;
}

text::first-letter {
  font-family: serif;
  font-size: 2rem;
  font-weight: 600;
  fill: tomato;
  stroke: indigo;
}
```

Result

See also

- `::first-line`

::first-line CSS pseudo-element

The **`::first-line`** CSS pseudo-element applies styles to the first line of a block container.

Try it

```
p::first-line {
  font-size: 1.2rem;
  font-weight: bold;
  text-decoration: underline;
}
```

```
<p>
  In warm ocean waters around the world, you may see a strange sight: A fish
  leaping from the water and soaring dozens of meters before returning to the
  ocean's depths. Early Mediterranean sailors thought these flying fish returned
  to the shore at night to sleep, and therefore called this family of marine
  fish Exocoetidae.
</p>
```

The effects of `::first-line` are limited by the length and content of the first line of text in the element. The length of the first line depends on many factors, including the width of the element, the width of the document, and the font size of the text. `::first-line` has no effect when the first child of the element, which would be the first part of the first line, is an inline block-level element, such as an inline table.

> **Note:** Selectors Level 3 introduced the double-colon notation (`::`) to distinguish pseudo-elements from the single-colon (`:`) pseudo-classes. Browsers accept both `::first-line` and `:first-line`, which was introduced in CSS2.

For the purposes of CSS `background`, the `::first-line` pseudo-element is like an inline-level element meaning that in a left-justified first line, the background may not extend all the way to the right margin.

Allowable properties

Only a small subset of CSS properties can be used with the `::first-line` pseudo-element:

- All font-related properties: `font`, `font-kerning`, `font-style`, `font-variant`, `font-variant-numeric`, `font-variant-position`, `font-variant-east-asian`, `font-variant-caps`, `font-variant-alternates`, `font-variant-ligatures`, `font-synthesis`, `font-feature-settings`, `font-language-override`, `font-weight`, `font-size`, `font-size-adjust`, `font-stretch`, and `font-family`
- All background-related properties: `background-color`, `background-clip`, `background-image`, `background-origin`, `background-position`, `background-repeat`, `background-size`, `background-attachment`, and `background-blend-mode`
- The `color` property
- `word-spacing`, `letter-spacing`, `text-decoration`, `text-transform`, and `line-height`
- `text-shadow`, `text-decoration`, `text-decoration-color`, `text-decoration-line`, `text-decoration-style`, and `vertical-align`.

Syntax

```
::first-line {
  /* ... */
}
```

Examples

Styling first line of a paragraph

HTML

```
<p>
  Styles will only be applied to the first line of this paragraph. After that,
  all text will be styled like normal. See what I mean?
</p>

<span>
  The first line of this text will not receive special styling because it is not
  a block-level element.
</span>
```

CSS

```
* {
  font-size: 20px;
  font-family: sans-serif;
}
```

```
::first-line {
  color: blue;
  font-weight: bold;

  /* WARNING: DO NOT USE THESE */
  /* Many properties are invalid in ::first-line pseudo-elements */
  margin-left: 20px;
```

```
  text-indent: 20px;
}
```

Result

Styling the first line of an SVG text element

In this example, we style the first line of an SVG `<text>` element using the `::first-line` pseudo-element.

> **Note:** At time of writing this feature has limited support.

HTML

```
<svg viewBox="0 0 320 150">
  <text y="20">Here is an English paragraph
that is broken into multiple lines
in the source code so that it can
be more easily read and edited
in a text editor.
  </text>
</svg>
```

CSS

In order to make the SVG `<text>` element wrap to multiple lines, we use the `white-space` CSS property. We then select the first line using the `::first-line` pseudo-element.

```
text {
  font-size: 20px;
  font-family: sans-serif;
}
```

```
text {
  white-space: break-spaces;
}

text::first-line {
  fill: blue;
  font-weight: bold;
}
```

Result

See also

- `::first-letter`
- `white-space`

::grammar-error CSS pseudo-element

The **`::grammar-error`** CSS pseudo-element represents a text segment which the user agent has flagged as grammatically incorrect.

The `::grammar-error` pseudo-element follows a special inheritance model common to all highlight pseudo-elements. For more details on how this inheritance works, see the Highlight pseudo-elements inheritance section.

Allowable properties

Only a small subset of CSS properties can be used in a rule with `::grammar-error` in its selector:

- `color`
- `background-color`
- `cursor`
- `caret-color`
- `outline` and its longhands
- `text-decoration` and its associated properties
- `text-emphasis-color`
- `text-shadow`

Syntax

```
::grammar-error {
  /* ... */
}
```

Examples

Basic document grammar check

In this example, eventual supporting browsers should highlight any flagged grammatical errors with the styles shown.

HTML

```
<p contenteditable spellcheck="true">
  My friends is coming to the party tonight.
</p>
```

CSS

```
::grammar-error {
  text-decoration: underline red;
  color: red;
}
```

Result

See also

- `::spelling-error`
- `text-decoration-line`

::highlight() CSS pseudo-element

The **`::highlight()`** CSS pseudo-element applies styles to a custom highlight.

A custom highlight is a collection of `AbstractRange` objects and is registered on a webpage using the `HighlightRegistry`.

The `::highlight()` pseudo-element follows a special inheritance model common to all highlight pseudo-elements. For more details on how this inheritance works, see the Highlight pseudo-elements inheritance section.

Allowable properties

Only certain CSS properties can be used with `::highlight()`:

- `color`
- `background-color`
- `text-decoration` and its associated properties
- `text-shadow`
- `-webkit-text-stroke-color`, `-webkit-text-fill-color` and `-webkit-text-stroke-width`

In particular, `background-image` is ignored.

Syntax

```
::highlight(custom-highlight-name)
```

Examples

Highlighting characters

HTML

```
<p id="rainbow-text">CSS Custom Highlight API rainbow</p>
```

CSS

```
#rainbow-text {
  font-family: monospace;
  font-size: 1.5rem;
}

::highlight(rainbow-color-1) {
  color: violet;
  text-decoration: underline;
}
::highlight(rainbow-color-2) {
  color: purple;
  text-decoration: underline;
}
::highlight(rainbow-color-3) {
  color: blue;
  text-decoration: underline;
}
::highlight(rainbow-color-4) {
  color: green;
  text-decoration: underline;
}
::highlight(rainbow-color-5) {
  color: yellow;
  text-decoration: underline;
}
::highlight(rainbow-color-6) {
  color: orange;
  text-decoration: underline;
}
::highlight(rainbow-color-7) {
  color: red;
  text-decoration: underline;
}
```

JavaScript

```
const textNode = document.getElementById("rainbow-text").firstChild;

if (!CSS.highlights) {
  textNode.textContent =
    "The CSS Custom Highlight API is not supported in this browser!";
}

// Create and register highlights for each color in the rainbow.
const highlights = [];
for (let i = 0; i < 7; i++) {
  // Create a new highlight for this color.
  const colorHighlight = new Highlight();
  highlights.push(colorHighlight);

  // Register this highlight under a custom name.
  CSS.highlights.set(`rainbow-color-${i + 1}`, colorHighlight);
}

// Iterate over the text, character by character.
for (let i = 0; i < textNode.textContent.length; i++) {
  // Create a new range just for this character.
  const range = new Range();
  range.setStart(textNode, i);
  range.setEnd(textNode, i + 1);
```

```
  // Add the range to the next available highlight,
  // looping back to the first one once we've reached the 7th.
  highlights[i % 7].add(range);
}
```

Result

See also

- CSS custom highlight API module
- CSS custom highlight API
- CSS pseudo-elements module

::marker CSS pseudo-element

The **`::marker`** CSS pseudo-element selects the marker box of a list item, which typically contains a bullet or number. It works on any element or pseudo-element set to `display: list-item`, such as the `<li>` and `<summary>` elements.

Try it

```
li::marker {
  content: "† ";
  font-size: 1.2em;
}
```

```
<p>Group known as Mercury Seven:</p>
<ul>
  <li>Malcolm Scott Carpenter</li>
  <li>Leroy Gordon (Gordo) Cooper Jr.</li>
  <li>John Herschel Glenn Jr.</li>
  <li>Virgil Ivan (Gus) Grissom</li>
  <li>Walter Marty (Wally) Schirra Jr.</li>
  <li>Alan Bartlett Shepard Jr.</li>
  <li>Donald Kent (Deke) Slayton</li>
</ul>
```

Allowable properties

The `::marker` pseudo-element supports a limited number of CSS properties, including:

- All animation and transition properties
- All font properties
- `color`
- `content`
- `counter-increment`, `counter-reset`, and `counter-set`
- `quotes`
- `text-combine-upright`, `unicode-bidi`, and `direction`
- `white-space`

> **Note:** The specification states that additional CSS properties may be supported in the future.

Syntax

```
::marker {
  /* ... */
}
```

Examples

HTML

```
<ul>
  <li>Peaches</li>
  <li>Apples</li>
  <li>Plums</li>
</ul>
```

CSS

```
ul li::marker {
  color: red;
  font-size: 1.5em;
}
```

Result

See also

- HTML elements that have marker boxes by default: `<ol>`, `<li>`, `<summary>`
- CSS generated content module
- CSS lists and counters module
- CSS counter styles module

::part() CSS pseudo-element

The **`::part`** CSS pseudo-element represents any element within a shadow tree that has a matching `part` attribute.

```
custom-element::part(foo) {
  /* Styles to apply to the `foo` part */
}
```

Description

The global `part` attribute makes a shadow tree element visible to its parent DOM. The part names declared using the `part` attribute are used as the parameter of the `::part()` pseudo-element. In this way, you can apply CSS styles to elements in the shadow tree from outside of it.

Part names are similar to CSS classes: multiple elements can have the same part name, and a single element can have multiple part names. All part names used in `::part()` pseudo-element must be present in the `part` value declared on the shadow tree element but the order of the part names doesn't matter, i.e., the selectors `::part(tab active)` and `::part(active tab)` are the same.

The `::part()` pseudo-element is only visible to the parent DOM. This means that when a shadow tree is nested, the parts are not visible to any ancestors other than the direct parent. The `exportparts` attribute solves this limitation by explicitly exporting already defined `part` names, making them globally stylable.

Pseudo-classes (such as `::part(label):hover`) can be appended to the `::part()` selector, but structural pseudo-classes that match based on tree information, such as `:empty` and `:last-child`, cannot be appended.

Additional pseudo-elements, such as `::before`, can be appended to the `::part()` selector, but additional `::part()` element can't be appended. For example, `::part(confirm-button)::part(active)` never matches anything, i.e, it is not the same as `::part(confirm-button active)`. This is because doing so would expose more structural information than is intended.

Syntax

```
::part(<ident>+) {
  /* ... */
}
```

Examples

HTML

```
<template id="tabbed-custom-element">
  <style>
    *,
    ::before,
    ::after {
      box-sizing: border-box;
      padding: 1rem;
    }
    :host {
      display: flex;
    }
  </style>
  <div part="tab active">Tab A</div>
  <div part="tab">Tab B</div>
  <div part="tab">Tab C</div>
</template>

<tabbed-custom-element></tabbed-custom-element>
```

CSS

```
tabbed-custom-element::part(tab) {
  color: blue;
  border-bottom: transparent solid 4px;
}

tabbed-custom-element::part(tab):hover {
  background-color: black;
  color: white;
}

tabbed-custom-element::part(tab active) {
  border-color: blue !important;
}
```

JavaScript

```
const template = document.querySelector("#tabbed-custom-element");
globalThis.customElements.define(
  template.id,
  class extends HTMLElement {
    constructor() {
      super().attachShadow({ mode: "open" }).append(template.content);
    }
  },
);
```

Result

See also

- `part` attribute
- `:state()` pseudo-class function
- `exportparts` attribute
- CSS shadow parts module

::picker() CSS pseudo-element

The **`::picker()`** CSS pseudo-element targets the picker part of an element, for example the drop-down picker of a customizable select element.

Syntax

```
::picker(<ident>) {
  /* ... */
}
```

Parameters

`<ident>`

A string representing the element whose picker you want to target. The following values are available:

`select`

The drop-down picker of customizable select elements.

Description

The `::picker()` pseudo-element targets the picker part of a form control, that is, the pop-up part that appears to allow you to make a selection when you press the control button. It is only available to target when the originating element has a picker and has base appearance set on it via the `appearance` property `base-select` value.

The `::picker(select)` selector targets all descendants of customizable `<select>` element except for the first `<button>` child; these descendants are grouped together by the browser and rendered as the picker. The first `<button>` child represents the control button that opens the picker when pressed.

This allows you to target all of the picker contents as a single entity, for example if you want to customize its border, animate it when it appears and disappears, or position it somewhere different to the default position. Our customizable select elements guide shows many examples of `::picker(select)` usage.

Picker popover behavior

The `<select>` element and the picker have an implicit invoker/popover relationship assigned to them automatically, as specified by the Popover API. See Using the Popover API for more details of popover behavior, and see Animating the picker drop-down using popover states for a typical use case allowed by the implicit popover association.

Picker anchor positioning

A further side-effect of the implicit invoker/popover relationship mentioned above is that the `<select>` element and the picker also have an implicit anchor reference, meaning that the picker is automatically associated with the select via CSS anchor positioning. This has several advantages, most notably:

- The browser default styles position the picker relative to the button (the anchor) and you can customize this position as explained in Positioning elements relative to their anchor. For reference, the related default styles are as follows:

  ```
  inset: auto;
  margin: 0;
  min-inline-size: anchor-size(self-inline);
  min-block-size: 1lh;
  /* Go to the edge of the viewport, and add scrollbars if needed. */
  max-block-size: stretch;
  overflow: auto;
  /* Below and span-right, by default. */
  position-area: block-end span-inline-end;
  ```

- The browser default styles also define some position-try fallbacks that reposition the picker if it is in danger of overflowing the viewport. Position-try fallbacks are explained in the Fallback options and conditional hiding for overflow guide. For reference, the related default fallback styles are as follows:

  ```
  position-try-order: most-block-size;
  position-try-fallbacks:
    /* First try above and span-right, */
    /* then below but span-left, */
    /* then above and span-left. */
    block-start span-inline-end,
    block-end span-inline-start,
    block-start span-inline-start;
  ```

> **Note:** If you want to remove the implicit anchor reference to stop the picker from being anchored to the `<select>` element, you can do so by setting the `position-anchor` property of the picker to an anchor name that doesn't exist in the current document, such as `--not-an-anchor-name`. See also removing an anchor association.

Examples

Basic custom select usage

To opt-in to the custom select functionality and minimal browser base styles (and remove the OS-provided styling), the `<select>` element and its picker both need to have an `appearance` value of `base-select` set on them:

```
select,
::picker(select) {
  appearance: base-select;
}
```

You could then, for example, remove the picker's default black `border`:

```
::picker(select) {
  border: none;
}
```

See also

- `<select>`, `<option>`, `<optgroup>`, `<label>`, `<button>`, `<selectedcontent>`
- `appearance`
- `::picker-icon`, `::checkmark`
- `:open`, `:checked`
- Customizable select elements

::picker-icon CSS pseudo-element

The **`::picker-icon`** CSS pseudo-element targets the picker icon inside form controls that have an icon associated with them. In the case of a customizable select element, it selects the arrow icon shown on the `<select>` element that points down when it is closed.

Syntax

```
::picker-icon {
  /* ... */
}
```

Description

The `::picker-icon` pseudo-element targets the picker icon of form controls, that is, the icon shown on the control button. It is only available to target when the originating element has a picker and has base appearance set on it via the `appearance` property `base-select` value. Its generated box appears after any boxes generated by the `::after` pseudo-element, with the icon specified in default browser stylesheet; you

can customize it using the `content` property.

The `::picker-icon` selector can be used to select the down-facing arrow on the inline-end side of a customizable select element. This is useful for example if you want to customize the color or size of the icon, use a different icon (using `content` or SVG), or animate it when the picker is opened and closed.

Selecting customizable `<select>` picker icons is the only current use case for `::picker-icon`, but more may be added in the future.

> **Note:** The `::picker-icon` pseudo-element is not included in the accessibility tree, so any generated `content` set on it will not be announced by assistive technologies. You should still make sure that any new icon you set visually makes sense for its intended purpose.

Examples

Animating the picker icon

To opt-in to customizable select functionality, the `<select>` element and its picker both need to have an `appearance` value of `base-select` set on them:

```
select,
::picker(select) {
  appearance: base-select;
}
```

You could then, for example, target the `::picker-icon` and give it a custom `color` and a `transition` so that changes to its `rotate` property are smoothly animated:

```
select::picker-icon {
  color: #999999;
  transition: 0.4s rotate;
}
```

In the next rule, `::picker-icon` is combined with the `:open` pseudo-class — which targets the icon only when the picker is open — transitioning it to a `rotate` value of `180deg` when the `<select>` is opened.

```
select:open::picker-icon {
  rotate: 180deg;
}
```

See Styling the picker icon for a full example that uses this code, along with a live example rendering.

See also

- `<select>`, `<option>`, `<optgroup>`, `<label>`, `<button>`, `<selectedcontent>`
- `appearance`
- `::picker(select)`, `::checkmark`
- `:open`, `:checked`
- Customizable select elements

::placeholder CSS pseudo-element

The **`::placeholder`** CSS pseudo-element represents the placeholder text in an `<input>` or `<textarea>` element.

Try it

```
input {
  margin-top: 0.5rem;
}

input::placeholder {
  font-weight: bold;
  opacity: 0.5;
  color: red;
}
```

```
<label for="first-name">Your phone number:</label><br />

<input
  id="first-name"
  type="tel"
  name="phone"
  minlength="9"
  maxlength="9"
  placeholder="It must be 9 digits" />
```

Only the subset of CSS properties that apply to the `::first-line` pseudo-element can be used in a rule using `::placeholder` in its selector.

> **Note:** In most browsers, the appearance of placeholder text is a translucent or light gray color by default.

Syntax

```
::placeholder {
  /* ... */
}
```

Accessibility

Color contrast

Contrast Ratio

Placeholder text typically has a lighter color treatment to indicate that it is a suggestion for what kind of input will be valid, and is not actual input of any kind.

It is important to ensure that the contrast ratio between the color of the placeholder text and the background of the input is high enough that people experiencing low vision conditions will be able to read it while also making sure there is enough of a difference between the placeholder text and input text color that users do not mistake the placeholder for inputted data.

Color contrast ratio is determined by comparing the luminosity of the placeholder text and the input background color values. In order to meet current Web Content Accessibility Guidelines (WCAG), a ratio of 4.5:1 is required for text content and 3:1 for larger text such as headings. Large text is defined as 18.66px and bold or larger, or 24px or larger.

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Usability

Placeholder text with sufficient color contrast may be interpreted as entered input. Placeholder text will also disappear when a person enters content into an `<input>` element. Both of these circumstances can interfere with successful form completion, especially for people with cognitive concerns.

An alternate approach to providing placeholder information is to include it outside of the input in close visual proximity, then use `aria-describedby` to programmatically associate the `<input>` with its hint.

With this implementation, the hint content is available even if information is entered into the input field, and the input appears free of preexisting input when the page is loaded. Most screen reading technology will use `aria-describedby` to read the hint after the input's label text is announced, and the person using the screen reader can mute it if they find the extra information unnecessary.

```
<label for="user-email">Your email address</label>
<span id="user-email-hint" class="input-hint">Example: jane@sample.com</span>
<input
  id="user-email"
  aria-describedby="user-email-hint"
  name="email"
  type="email" />
```

- Placeholders in Form Fields Are Harmful — Nielsen Norman Group

Windows High Contrast Mode

Placeholder text will appear with the same styling as user-entered text content when rendered in Windows High Contrast Mode. This will make it difficult for some people to determine which content has been entered, and which content is placeholder text.

Labels

Placeholders are not a replacement for the `<label>` element. Without a label that has been programmatically associated with an input using a combination of the `for` and `id` attributes, assistive technology such as screen readers cannot parse `<input>` elements.

- Placeholders in Form Fields Are Harmful — Nielsen Norman Group

Examples

Change placeholder appearance

This example shows some of the adjustments that you can make to the styles of placeholder text.

HTML

```
<input placeholder="Type here" />
```

CSS

```
input::placeholder {
  color: red;
  font-size: 1.2em;
  font-style: italic;
  opacity: 0.5;
}
```

Result

Opaque text

Some browsers make placeholder text less opaque. If you want fully opaque text, then set the `color` property value explicitly. The `currentColor` value can be used to have the same color as the corresponding input element.

HTML

```
<input placeholder="Color set by browser" />
<input placeholder="Same color as input" class="explicit-color" />
<input placeholder="Semi-opaque text color" class="opacity-change" />
```

CSS

```
input {
  font-weight: bold;
  color: green;
}

.explicit-color::placeholder {
  /* use the same color as input element to avoid the browser set default color */
  color: currentColor;
}

.opacity-change::placeholder {
  /* less opaque text */
  color: color-mix(in srgb, currentColor 70%, transparent);
}
```

Result

> **Note:** Note that browsers use different default colors for placeholder text. For example, Firefox uses the input element's color with 54% opacity, and Chrome uses `darkgray` color. If you want consistent placeholder text color across the browsers, then set the `color` explicitly.

See also

- The `:placeholder-shown` pseudo-class styles an element that *has* an active placeholder.
- Related HTML elements: `<input>`, `<textarea>`
- HTML forms

::scroll-button() CSS pseudo-element

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::scroll-button()`** CSS pseudo-element represents a button for controlling the scrolling of a scroll container. They are generated on scroll containers when their `content` value is not `none`. The direction of the scrolling is determined by the parameter value.

Syntax

```
::scroll-button(<scroll-button-direction>) {
  /* ... */
}
```

Parameters

`<scroll-button-direction>`

A value representing which direction of scroll button you want to select. The following values are available:

`*`

Selects all the originating element's scroll buttons, allowing styles to be applied to each of them in a single rule.

`down`

Selects the button that will scroll the content downward.

`left`

Selects the button that will scroll the content left.

`right`

Selects the button that will scroll the content right.

`up`

Selects the button that will scroll the content upward.

`block-end`

Selects the button that will scroll the content in the block-end direction.

`block-start`

Selects the button that will scroll the content in the block-start direction.

`inline-end`

Selects the button that will scroll the content in the inline-end direction.

`inline-start`

Selects the button that will scroll the content in the inline-start direction.

The specification also defines two other values — `next` and `prev` — but these are not currently supported in any browser.

Description

The `::scroll-button()` pseudo-elements are generated inside a scroll container only when their `content` properties are set to a value other than `none`. They are generated as siblings of the scroll container's child DOM elements, immediately preceding them and any `::scroll-marker-group` generated on the container.

You can generate up to four scroll buttons per scroll container, which will scroll the content towards the start and end of the block and inline axes. The selector's argument specifies which scrolling direction is selected. You can also specify a value of `*` to target all of the `::scroll-button()` pseudo-elements, providing styles to all the buttons in a single rule.

The generated buttons behave just like regular `<button>` elements, including sharing their default browser styles. They are focusable, accessible, and can be activated like regular buttons. When a scroll button is pressed, the scroll container's content is scrolled in the specified direction by one "page," or approximately the dimension of the scroll container, similar to pressing `PgUp` and `PgDn` keys.

The recommendation is to set up CSS scroll snapping on the scroll container and set each separate item of content you want to scroll to as a snap target. This being the case, activating a scroll button will scroll the content to the snap target that is one "page" away. While the scroll buttons will work without scroll snapping, you might not get the desired effect.

When it is not possible to scroll any further in a particular scroll button's scrolling direction, the button is automatically disabled, otherwise it is enabled. You can style the scroll buttons in their enabled and disabled states using the `:enabled` and `:disabled` pseudo-classes.

Examples

See Creating CSS carousels for more carousel examples.

Creating scroll buttons

In this example, we demonstrate how to create scroll buttons on a CSS carousel.

HTML

We have a basic HTML `<ul>` list with several `<li>` list items.

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>
  <li>Item 5</li>
  <li>Item 6</li>
  <li>Item 7</li>
  <li>Item 8</li>
</ul>
```

CSS

Styling the carousel

We convert our `<ul>` into a carousel by setting the `display` to `flex`, creating a single, non-wrapping row of `<li>` elements. The `overflow-x` property is set to `auto`, meaning if the items overflow their container on the x-axis, the content will scroll horizontally. We then convert the `<ul>` into a scroll-snap container, ensuring that items always snap into place when the container is scrolled with a `scroll-snap-type` value of `mandatory`.

```
ul {
  display: flex;
  gap: 4vw;
  padding-left: 0;
  overflow-x: auto;
  overscroll-behavior-x: contain;
  scroll-snap-type: x mandatory;
}
```

Next, we style the `<li>` elements, using the `flex` property to make them 100% of the width of the container. The `scroll-snap-align` value of `start` causes the left-hand side of the left-most visible item to snap to the left edge of the container when the content is scrolled.

```
li {
  list-style-type: none;
  background-color: #eeeeee;
  flex: 0 0 100%;
  height: 100px;
  padding-top: 20px;
  scroll-snap-align: start;
  text-align: center;
}
```

Creating the scroll buttons

First, all scroll buttons are targeted with some rudimentary styles, as well as styling based on different states. It is important to set `:focus` styles for keyboard users. Also, as scroll buttons are automatically set to `disabled` when no more scrolling can occur in that direction, we use the `:disabled` pseudo-class to target this state.

```
ul::scroll-button(*) {
  border: 0;
  font-size: 2rem;
  background: none;
  color: black;
  opacity: 0.7;
  cursor: pointer;
}

ul::scroll-button(*):hover,
ul::scroll-button(*):focus {
  opacity: 1;
}

ul::scroll-button(*):active {
  translate: 1px 1px;
}

ul::scroll-button(*):disabled {
  opacity: 0.2;
  cursor: unset;
}
```

> **Note:** We also set a `cursor` value of `pointer` on the scroll buttons to make it more obvious that they can be interacted with (an improvement for both general UX and cognitive accessibility), unsetting it when the scroll buttons are `:disabled`.

Next, an appropriate icon is set on the left and right scroll buttons via the `content` property, which is also what causes the scroll buttons to be generated:

```
ul::scroll-button(left) {
  content: "◄";
}

ul::scroll-button(right) {
  content: "►";
}
```

We don't need to set alternative text for the icons on the `content` as the browser takes care of providing appropriate accessible names automatically.

Result

Note how the scroll buttons are created at the bottom left on the carousel. Try pressing them to see how they cause the content to be scrolled.

Positioning the scroll buttons

The previous example works, but the buttons are not ideally placed. In this section, we will add some CSS to position them using anchor positioning.

CSS

First of all, a reference `anchor-name` is set on the `<ul>` to define it as a named anchor. Next, each scroll button has its `position` set to `absolute` and its `position-anchor` property set to the list's `anchor-name`, to associate the two together.

```
ul {
  anchor-name: --my-carousel;
}

ul::scroll-button(*) {
  position: absolute;
  position-anchor: --my-carousel;
}
```

To actually position each scroll button, we first set an `align-self` value of `anchor-center` on both of them, to center them vertically on the carousel:

```
ul::scroll-button(*) {
  align-self: anchor-center;
}
```

We then set values on their inset properties to handle the horizontal positioning. We use `anchor()` functions to position the specified sides of the buttons relative to the sides of the carousel. In each case, the `calc()` function is used to add some space between the button edge and the carousel edge. For example, the right-hand edge of the left scroll button is positioned 45 pixels to the right of the carousel's left-hand edge.

```
ul::scroll-button(left) {
  right: calc(anchor(left) - 45px);
}

ul::scroll-button(right) {
  left: calc(anchor(right) - 45px);
}
```

Result

See also

- `scroll-marker-group`
- `::scroll-marker-group`
- `::scroll-marker`
- `::column`
- `:target-current`
- Creating CSS carousels
- CSS overflow module
- CSS anchor positioning module
- CSS Carousel Gallery via chrome.dev (2025)

::scroll-marker CSS pseudo-element

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::scroll-marker`** CSS pseudo-element can be generated inside any element and represents its scroll marker. All elements can have a `::scroll-marker` pseudo-element, which is placed into the `::scroll-marker-group` of the nearest scroll container ancestor. A scroll marker behaves like an anchor (`<a>` element) whose scroll target is the marker's originating element — and scrolls the scroll container to that element when activated.

Syntax

```
::scroll-marker {
  /* ... */
}
```

Description

A `::scroll-marker` is generated on an element when the `::scroll-marker`'s `content` property is set to a non-`none` value, and its has an ancestor scroll container with a non-`none` `scroll-marker-group` property value (meaning that it will generate a `::scroll-marker-group` pseudo-element).

The scroll container's `::scroll-marker-group` pseudo-element automatically contains any `::scroll-marker` pseudo-elements generated on the scroll container or its descendants. This allows them to be positioned and laid out as a group and is typically used when creating a CSS carousel to create a scroll position indicator. The individual scroll markers can be used to navigate to their associated content items.

> **Note:** Alternatively, a scroll marker group container can be created from an existing element container using `scroll-target-group`; any contained `<a>` elements with fragment identifiers linking to sections of the page automatically behave like scroll markers.

When a scroll marker group container is created on a scroll container using the `scroll-marker-group` property, the scroll container is rendered with `tablist`/`tab` semantics. You can `Tab` to it with the keyboard, then move between the different "pages" using the left and right (or up and down) cursor keys, which also changes the state of the associated scroll markers and scroll buttons as expected. The scroll markers can also be tabbed between normally, as expected.

Examples

See Creating CSS carousels for other examples that use the `::scroll-marker` pseudo-element.

Creating carousel scroll markers

In this example, we demonstrate how to create scroll markers on a CSS carousel.

HTML

We have a basic HTML `<ul>` list with several `<li>` list items.

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>
  <li>Item 5</li>
  <li>Item 6</li>
  <li>Item 7</li>
  <li>Item 8</li>
</ul>
```

CSS

We convert our `<ul>` into a scroll snapping overflow container by setting the `display` to `flex`, creating a single, non-wrapping row of `<li>` elements. The `overflow-x` property is set to `auto`, meaning if the items overflow their container on the x-axis, the content will scroll horizontally. We then convert the `<ul>` into a scroll-snap container, ensuring that items always snap into place when the container is scrolled with a `scroll-snap-type` value of `mandatory`.

We create a scroll marker group container with the `scroll-marker-group` property, placing it after all the content.

```
ul {
  display: flex;
  gap: 4vw;
  padding-left: 0;
  overflow-x: auto;
  overscroll-behavior-x: contain;
  scroll-snap-type: x mandatory;
  scroll-marker-group: after;
}
```

Next, we style the `<li>` elements, using the `flex` property to make them `33%` of the width of the container. The `scroll-snap-align` value of `start` causes the left-hand side of the left-most visible item to snap to the left edge of the container when the content is scrolled.

```
li {
  list-style-type: none;
  background-color: #eeeeee;
  flex: 0 0 33%;
  height: 100px;
  padding-top: 20px;
  scroll-snap-align: start;
  text-align: center;
}
```

We then use the `::scroll-marker` pseudo-element to create a square marker for each list item with a red border:

```
li::scroll-marker {
  content: "";
  border: 1px solid red;
  height: 1em;
  width: 1em;
}
```

We also apply styles to the `::scroll-marker-group` pseudo-element to lay out the scroll markers in the center of the row with a `0.4em` gap between each one:

```
::scroll-marker-group {
  display: flex;
  gap: 0.4em;
  place-content: center;
}
```

Finally, we style the marker of the currently-scrolled element differently from the others, targeting the marker with the `:target-current` pseudo-class.

```
::scroll-marker:target-current {
  background: red;
}
```

Result

Custom scroll marker numbering and style

This example is the same as the previous one, except that we have applied some different styling to the scroll markers, and used CSS counters to increment the number shown on each marker. The CSS differences are explained in the next section.

CSS

In this example, we set a name of a counter we want to increment on each `<li>` — `markers` — using the `counter-increment` property:

```
li {
  counter-increment: markers;
}
```

We then set the `::scroll-marker` pseudo-element's `content` property to the `counter()` function, passing it the `markers` counter name as an argument. This has the effect of inserting a number into each marker, which increments automatically. The rest of the styling is rudimentary, but it illustrates how the markers can be fully-styled.

```
li::scroll-marker {
  content: counter(markers);
  font-family: sans-serif;
  width: fit-content;
  height: 1em;
  padding: 5px;
  color: black;
  text-decoration: none;
  border: 2px solid rgb(0 0 0 / 0.15);
  border-radius: 0.5em;
  background-color: #eeeeee;
}
```

For another interesting customization, we include two rules to select the marker of the first and last list items by inserting `:first-child` and `:last-child` into the selector chain, respectively. We give the first marker text content of "First", and the last marker text content of "Last".

```
li:first-child::scroll-marker {
  content: "First";
}

li:last-child::scroll-marker {
  content: "Last";
}
```

To improve user experience, we set a different color on the markers on `:hover` and use the `:target-current` pseudo-class to set a different `color` and `background-color` on the currently-scrolled element's marker so users know which item is currently in view:

```
::scroll-marker:hover {
  background-color: #ddcccc;
}

::scroll-marker:target-current {
```

```
  background-color: purple;
  color: white;
}
```

Result

See also

- `scroll-marker-group`
- `scroll-target-group`
- `::scroll-button()`
- `::scroll-marker-group`
- `:target-current`
- `:target-before`
- `:target-after`
- Creating CSS carousels
- CSS lists and counters module
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

::scroll-marker-group CSS pseudo-element

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::scroll-marker-group`** CSS pseudo-element is generated inside a scroll container and contains any `::scroll-marker` pseudo-elements generated on descendants of the scroll container.

Syntax

```
::scroll-marker-group {
  /* ... */
}
```

Description

A scroll container's **`::scroll-marker-group`** pseudo-element represents a **scroll marker group container**. This is a container that automatically contains any `::scroll-marker` pseudo-elements generated on itself or its descendants. This allows them to be positioned and styled as a group. This pseudo-element is typically used when creating a CSS carousel to provide a scroll position indicator. The individual scroll markers can be used to navigate to their associated content items.

The scroll container must have its `scroll-marker-group` property set to a non-`none` value for the `::scroll-marker-group` pseudo-element to be generated. The `scroll-marker-group` value determines where the scroll marker group container appears in the carousel's tab order and layout box order (but not DOM structure) — `before` puts it at the start, while `after` puts it at the end.

> **Note:** Alternatively, a scroll marker group container can be created from an existing element containing a set of `<a>` elements using `scroll-target-group`.

As an accessibility best practice, match the visual rendering position of the scroll marker group container with the tab order. When positioning the group at the start of the content, put it at the beginning of the tab order using `before`. When positioning the group at the end of the content, put it at the end of the tab order using `after`.

When a scroll marker group container is created on a scroll container using the `scroll-marker-group` property, the scroll container is rendered with `tablist`/`tab` semantics. You can `Tab` to it with the keyboard, then move between the different "pages" using the left and right (or up and down) cursor keys, which also changes the state of the associated scroll markers and scroll buttons as expected. The scroll markers can also be tabbed between normally, as expected.

Examples

See Creating CSS carousels for other examples that use the `::scroll-marker` pseudo-element.

Creating carousel scroll markers

This demo is a carousel of single pages, with each item taking up the full page. We have included scroll markers to enable the user to navigate to any page with the click of a marker.

HTML

The HTML consists of an unordered list, with each list item containing some sample content:

```
<ul>
  <li>
    <h2>Page 1</h2>
  </li>
  <li>
    <h2>Page 2</h2>
  </li>
  <li>
    <h2>Page 3</h2>
  </li>
  <li>
    <h2>Page 4</h2>
  </li>
</ul>
```

CSS

We first convert our `<ul>` into a carousel by setting the `display` to `flex`, creating a single, non-wrapping row of `<li>` elements. The `overflow-x` property is set to `auto`, meaning if the items overflow their container on the x-axis, the content will scroll horizontally. We then convert the `<ul>` into a scroll-snap container, ensuring that items always snap into place when the container is scrolled with a `scroll-snap-type` value of `mandatory`.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Helvetica", "Arial", sans-serif;
}
```

```
ul {
  display: flex;
  gap: 4vw;
  padding-left: 0;
  overflow-x: auto;
  overscroll-behavior-x: contain;
  scroll-snap-type: x mandatory;
}
```

Next, we style the `<li>` elements, using the `flex` property to make them `100%` of the width of the container. The `scroll-snap-align` value of `start` causes the left-hand side of the left-most visible item to snap to the left edge of the container when the content is scrolled.

```
li {
  list-style-type: none;
  background-color: #eeeeee;
  flex: 0 0 100%;
  height: 200px;
  padding-top: 20px;
  scroll-snap-align: start;
  text-align: center;
}
```

Next, the list's `scroll-marker-group` property is set to `after`, so the `::scroll-marker-group` pseudo-element is placed after the list's DOM content in the tabbing order and layout box order; this means it comes after the scroll buttons:

```
ul {
  scroll-marker-group: after;
}
```

Next, the list's `::scroll-marker-group` pseudo-element is laid out using flexbox, with a `justify-content` value of `center` and a `gap` of `20px` so that its children (the `::scroll-marker` pseudo-elements) are centered inside the `::scroll-marker-group` with a gap between each one.

```
ul::scroll-marker-group {
  display: flex;
  justify-content: center;
  gap: 20px;
}
```

Next, the scroll markers themselves are styled. The look of each one is handled by setting `width`, `height`, `background-color`, `border`, and `border-radius`, but we also need to set a non-`none` value for the `content`, property so they are actually generated.

```
li::scroll-marker {
  width: 16px;
  height: 16px;
  background-color: transparent;
  border: 2px solid black;
  border-radius: 50%;
  content: "";
}
```

> **Note:** Generated content is inline by default; we can apply `width` and `height` to our scroll markers because they are being laid out as flex items.

Finally, the `:target-current` pseudo-class is used to select whichever scroll marker corresponds to the currently visible "page", highlighting how far the user has scrolled through the content. In this case, we set the `background-color` to `black` so it is styled as a filled-in circle.

```
li::scroll-marker:target-current {
  background-color: black;
}
```

Result

Positioning the scroll marker group container with anchor positioning

This example extends the previous one, demonstrating the use of CSS anchor positioning to position the scroll marker group container relative to the carousel.

CSS

The list's `::scroll-marker-group` pseudo-element is positioned relative to the carousel using CSS anchor positioning by giving the group a value for `position-anchor` that matches the `anchor-name` of the `<ul>`. We then position the group relative to the scroll container in the block direction by setting a `top` value that includes an `anchor()` function. We also add a `justify-self` value of `anchor-center`, which aligns the group to the center of the scroll container in the inline direction.

```
ul {
  anchor-name: --my-carousel;
}

ul::scroll-marker-group {
  /* From the previous example */
  display: flex;
  gap: 20px;

  position: absolute;
  position-anchor: --my-carousel;
  top: calc(anchor(bottom) - 70px);
  justify-self: anchor-center;
}
```

Result

See also

- `scroll-marker-group`
- `scroll-target-group`
- `::scroll-button()`
- `::scroll-marker`

- `::column`
- `:target-current`
- `:target-before`
- `:target-after`
- Creating CSS carousels
- CSS anchor positioning module
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

::search-text CSS pseudo-element

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`::search-text`** CSS pseudo-element applies styles to search results identified by the user agent's "Find" or "Find in page" text search feature.

Try it

```
p::search-text {
  color: crimson;
  background-color: wheat;
}
```

```
<p>
  Using your browser's "Find in page" functionality, find a word or phrase that
  appears in this sentence, and note how, in supporting browsers, each result is
  highlighted using the specified custom styles.
</p>
```

Syntax

```
::search-text {
  /* ... */
}
```

Description

Most browsers include some kind of in-page text search functionality, usually labeled "Find" or "Find in page". The `::search-text` pseudo-element, one of the highlight pseudo-elements, allows you to apply a limited set of styles to the text results highlighted by the browser search functionality.

Not all browsers and browser versions highlight search results using in-page highlights that are stylable with CSS. In such cases, `::search-text` may be unimplemented or just ignored.

Using `::search-text` as a selector on its own will style browser search results appearing *anywhere* on a page. If you want to only style browser search results appearing inside certain elements, you can combine `::search-text` with other selectors, for example `section::search-text`.

Additionally, `::search-text` can be combined with the `:current` pseudo-class to provide specific styles to the currently-focused search result, for example:

```
p::search-text {
  color: white;
  background-color: purple;
}

p::search-text:current {
  background-color: crimson;
}
```

Inheritance model

The `::search-text` pseudo-element follows a special inheritance model common to all highlight pseudo-elements, wherein styles are inherited from both their parent elements and the pseudo-elements of their parents. For more details on how this inheritance works, see the Highlight pseudo-elements inheritance section.

Allowable properties

A limited subset of CSS properties can be used with `::search-text`:

- `color`
- `background-color`
- The `text-decoration` shorthand and associated longhand properties:
  - `text-decoration-line`: the `grammar-error`, `spelling-error`, `line-through`, `none`, and `underline` values only.
  - `text-decoration-color`
  - `text-decoration-style`
  - `text-decoration-thickness`
  - `text-decoration-skip-ink`
- `text-underline-offset`
- `text-shadow`

Accessibility

Override text search result styles sparingly, especially when doing so for purely aesthetic reasons. For people experiencing cognitive concerns or who are less technologically literate, unexpected changes to these styles may hurt their understanding of the functionality.

A primary use case of `::search-text` is to increase color contrast compared to the default browser styling. When customizing highlighted text, it is important to ensure that the contrast ratio between the foreground and background colors is great enough for people to be able to perceive the content of the highlighted text.

Examples

Custom styles for text search results

This example shows how to use `::search-text` and `:current` to create custom styles for your browser's "Find in page" search results.

HTML

The HTML consists of a basic paragraph of text. We won't show the HTML source, both for brevity, and so that it is easier to navigate the search results in the rendered example.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec finibus est
  eget eros congue pellentesque. Etiam a augue accumsan, scelerisque nisl sit
  amet, lobortis nulla. Aliquam condimentum eu orci eu elementum. Donec
  porttitor quam et posuere commodo. Mauris rhoncus diam a scelerisque molestie.
  Integer sollicitudin risus dui, ut sagittis lorem laoreet eget. Duis eget
  pretium enim. Morbi tristique, diam sit amet gravida finibus, metus ex
  tincidunt nibh, ac volutpat urna purus et arcu. Donec risus risus, semper vel
  purus sit amet, gravida vestibulum est. Sed et tristique urna. Nam vel mi eget
  nisi consectetur elementum. Aenean faucibus aliquam cursus. Morbi posuere
  tincidunt velit, et sagittis quam sagittis in. Nam eget ante ultrices, auctor
  dui vel, euismod lacus. Vivamus tincidunt, sem ac sodales aliquet, tortor
  tortor consequat diam, nec tempor mi dui vel eros. Aliquam ac erat et metus
  egestas scelerisque.
</p>
```

CSS

In our CSS, we start by styling the `::search-text` pseudo-element. We give it custom `background-color`, `color`, and `text-shadow` styles.

```
html {
  font-family: "Helvetica", "Arial";
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
  width: 90%;
  margin: 0 auto;
}
@layer no-support {
  body::before {
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1em 0;
  }
  @supports not selector(:current) {
    body::before {
      content: "Your browser doesn't support the :current pseudo-class.";
    }
  }
  @supports not selector(::search-text) {
    body::before {
      content: "Your browser doesn't support the ::search-text pseudo-element.";
    }
  }
}
```

```
::search-text {
  background-color: purple;
  color: white;
  text-shadow: 1px 1px 1px black;
}
```

Finally, we style the currently-focused search result via `::search-text:current`, providing it with a different `background-color` and some `text-decoration` styles so that it is distinguishable from the rest of the results.

```
::search-text:current {
  background-color: crimson;
  text-decoration-line: underline;
  text-decoration-color: yellow;
  text-decoration-thickness: 3px;
}
```

Result

The example renders as follows:

Try using the browser's find in page interface to find a word that appears multiple times in the example text, such as "aliquam", "amet", or "tortor". Move between the previous and next results to check out the `:current` styling.

See also

- `:current`
- Highlight pseudo-elements
- CSS pseudo-elements module

::selection CSS pseudo-element

The **`::selection`** CSS pseudo-element applies styles to the part of a document that has been highlighted by the user (such as clicking and dragging the mouse across text).

The `::selection` pseudo-element follows a special inheritance model common to all highlight pseudo-elements. For more details on how this inheritance works, see the Highlight pseudo-elements inheritance section.

Try it

```
p::selection {
  color: red;
  background-color: yellow;
}
```

```
<p>
  Select a fragment of this paragraph, to see how its appearance is affected.
</p>
```

Allowable properties

Only certain CSS properties can be used with `::selection`:

- `color`
- `background-color`
- `text-decoration` and its associated properties
- `text-shadow`
- `-webkit-text-stroke-color`, `-webkit-text-fill-color` and `-webkit-text-stroke-width`

In particular, `background-image` is ignored.

Syntax

```
::selection {
  /* ... */
}
```

Accessibility

Don't override selected text styles for purely aesthetic reasons — users can customize them to suit their needs. For people experiencing cognitive concerns or who are less technologically literate, unexpected changes to selection styles may hurt their understanding of the functionality.

If overridden, it is important to ensure that the **contrast ratio** between the text and background colors of the selection is high enough that people experiencing low vision conditions can read it.

Color contrast ratio is found by comparing the luminosity of the selected text and the selected text background colors. To meet current Web Content Accessibility Guidelines (WCAG), text content must have a contrast ratio of **4.5:1**, or 3:1 for larger text such as headings. (WCAG defines large text as between `18.66px` and `24px` and bold, or `24px` or larger.)

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Examples

HTML

```
This text has special styles when you highlight it.
<p>Also try selecting text in this paragraph.</p>
```

CSS

```
::-moz-selection {
  color: gold;
  background-color: red;
}

p::-moz-selection {
  color: white;
  background-color: blue;
}
```

```
/* Make selected text gold on a red background */
::selection {
  color: gold;
  background-color: red;
}

/* Make selected text in a paragraph white on a blue background */
p::selection {
  color: white;
  background-color: blue;
}
```

Result

See also

- `pointer-events` - control which events are active on the element

::slotted() CSS pseudo-element

The **`::slotted()`** CSS pseudo-element represents any element that has been placed into a slot inside an HTML template (see Using templates and slots for more information).

This only works when used inside CSS placed within a shadow DOM. Note that this selector won't select a text node placed into a slot; it only targets actual elements.

Try it

```
/* This CSS is being applied inside the shadow DOM. */

::slotted(.content) {
  background-color: aqua;
}

h2 ::slotted(span) {
  background: silver;
}
```

```
<template id="card-template">
  <div>
    <h2><slot name="caption">title goes here</slot></h2>
    <slot name="content">content goes here</slot>
  </div>
</template>

<my-card>
  <span slot="caption">Error</span>
  <p class="content" slot="content">Build failed!</p>
</my-card>
```

```
customElements.define(
  "my-card",
  class extends HTMLElement {
    constructor() {
      super();

      const template = document.getElementById("card-template");
      const shadow = this.attachShadow({ mode: "open" });
      shadow.appendChild(document.importNode(template.content, true));

      const elementStyle = document.createElement("style");
      elementStyle.textContent = `
        div {
          width: 200px;
          border: 2px dotted red;
          border-radius: 4px;
        }`;
      shadow.appendChild(elementStyle);

      const cssTab = document.querySelector("#css-output");
      const editorStyle = document.createElement("style");
      editorStyle.textContent = cssTab.textContent;
      shadow.appendChild(editorStyle);
      cssTab.addEventListener("change", () => {
        editorStyle.textContent = cssTab.textContent;
      });
    }
  },
);
```

```
/* Selects any element placed inside a slot */
::slotted(*) {
  font-weight: bold;
}

/* Selects any <span> placed inside a slot */
::slotted(span) {
  font-weight: bold;
}
```

Syntax

```
::slotted(<compound-selector>) {
  /* ... */
}
```

Examples

Highlighting slotted elements

In this example, we use a template with three slots:

```
<template id="person-template">
  <div>
    <h2>Personal ID Card</h2>
    <slot name="person-name">NAME MISSING</slot>
    <ul>
      <li><slot name="person-age">AGE MISSING</slot></li>
      <li><slot name="person-occupation">OCCUPATION MISSING</slot></li>
    </ul>
  </div>
</template>
```

We define the `<person-details>` custom element. In this case, we add styles with JavaScript, though we could have added them in a `<style>` block within the `<template>` with the same effect:

```
customElements.define(
  "person-details",
  class extends HTMLElement {
    constructor() {
      super();
      const template = document.getElementById("person-template");

      const shadowRoot = this.attachShadow({ mode: "open" });

      const style = document.createElement("style");
      style.textContent =
        "div { padding: 10px; border: 1px solid gray; width: 200px; margin: 10px; }" +
        "h2 { margin: 0 0 10px; }" +
        "ul { margin: 0; }" +
        "p { margin: 10px 0; }" +
        "::slotted(*) { color: gray; font-family: sans-serif; } " +
        "::slotted(span) {text-decoration: underline;} ";

      shadowRoot.appendChild(style);
      shadowRoot.appendChild(document.importNode(template.content, true));
    }
  },
);
```

When filling the `style` element with content, you'll see that we select all slotted elements (`::slotted(*)`) and give them a different font and color. This differentiates them from the slots that haven't been filled. We styled all the slotted `<span>`s (`::slotted(span)`) to differentiate the `<span>`s from the `<p>`s.

Our markup includes three custom elements, including a custom element with an invalid slot name in a source order that differs from the `<template>`:

```
<person-details>
  <p slot="person-name">Wonder Woman</p>
  <span slot="person-age">Immortal</span>
  <span slot="person-occupation">Superhero</span>
</person-details>
```

```
<person-details>
  <p slot="person-name">Malala Yousafzai</p>
  <span slot="person-age">17</span>
  <span slot="person-occupation">Activist</span>
</person-details>

<person-details>
  <span slot="person-age">44</span>
  <span slot="not-a-slot-name">Time traveler</span>
  <p slot="person-name">Dr. Who</p>
</person-details>
```

Result

See also

- `:host`
- `:host()`
- `:host-context()`
- `:has-slotted`
- CSS scoping module
- HTML `slot` attribute
- HTML `<slot>` element
- HTML `<template>` element
- Web components

::spelling-error CSS pseudo-element

The **`::spelling-error`** CSS pseudo-element represents a text segment which the user agent has flagged as incorrectly spelled.

The `::spelling-error` pseudo-element follows a special inheritance model common to all highlight pseudo-elements. For more details on how this inheritance works, see the Highlight pseudo-elements inheritance section.

Allowable properties

Only a small subset of CSS properties can be used in a rule with `::spelling-error` in its selector:

- `color`
- `background-color`
- `cursor`
- `caret-color`
- `outline` and its longhands
- `text-decoration` and its associated properties
- `text-emphasis-color`
- `text-shadow`

Syntax

```
::spelling-error {
  /* ... */
}
```

Examples

Basic document spell check

In this example, eventual supporting browsers should highlight any flagged spelling errors with the styles shown.

HTML

```
<p contenteditable spellcheck="true">
  My friends are coegdfgfddffbgning to the party tonight.
</p>
```

CSS

```
::spelling-error {
  text-decoration: wavy red underline;
}
```

Result

See also

- `::grammar-error`
- `text-decoration-line`

::target-text CSS pseudo-element

The **`::target-text`** CSS pseudo-element represents the text that has been scrolled to if the browser supports text fragments. It allows authors to choose how to highlight that section of text.

The `::target-text` pseudo-element follows a special inheritance model common to all highlight pseudo-elements. For more details on how this inheritance works, see the Highlight pseudo-elements inheritance section.

Syntax

```
::target-text {
  /* ... */
}
```

Examples

Highlighting text fragments

```
::target-text {
  background-color: rebeccapurple;
```

```
  color: white;
}
```

To see this CSS in action follow the link to scroll-to-text demo.

See also

- Text fragments
- `:target` (for highlighting target elements)

::view-transition CSS pseudo-element

The **`::view-transition`** CSS pseudo-element represents the root of the view transitions overlay, which contains all view transition snapshot groups and sits over the top of all other page content.

During a view transition, `::view-transition` is included in the associated pseudo-element tree as explained in The view transition pseudo-element tree. It is the top-level node of this tree, and has one or more `::view-transition-group()`s as children.

`::view-transition` is given the following default styling in the UA stylesheet:

```
:root::view-transition {
  position: fixed;
  inset: 0;
}
```

All `::view-transition-group()` pseudo-elements are positioned relative to the view transition root.

Syntax

```
::view-transition {
  /* ... */
}
```

Examples

```
::view-transition {
  background-color: rgb(0 0 0 / 25%);
}
```

See also

- View Transition API
- Smooth transitions with the View Transition API

::view-transition-group() CSS pseudo-element

The **`::view-transition-group()`** CSS pseudo-element represents a single view transition snapshot group.

During a view transition, `::view-transition-group()` is included in the associated pseudo-element tree as explained in The view transition pseudo-element tree. It is only ever a child of `::view-transition`, and has a `::view-transition-image-pair()` as a child.

`::view-transition-group()` is given the following default styling in the UA stylesheet:

```
:root::view-transition-group(*) {
  position: absolute;
  top: 0;
  left: 0;

  animation-duration: 0.25s;
  animation-fill-mode: both;
}
```

By default, selected elements initially mirror the size and position of the `::view-transition-old()` pseudo-element representing the "old" view state, or the `::view-transition-new()` pseudo-element representing the "new" view state if there isn't an "old" view state.

If there's both an "old" and "new" view state, styles in the view transition style sheet animate this pseudo-element's `width` and `height` from the size of the "old" view state's border box to that of the "new" view state's border box.

> **Note:** View transition styles are dynamically generated during the view transition; see the specification setup transition pseudo-elements and update pseudo-element styles sections for more details.

In addition, the element's transform is animated from the "old" view state's screen space transform to the new view state's screen space transform. This style is generated dynamically since the values of animated properties are determined at the time that the transition begins.

Syntax

```
::view-transition-group([ <pt-name-selector> <pt-class-selector>? ] | <pt-class-selector>) {
  /* ... */
}
```

Parameters

`*`

The universal selector (`*`); selects all view transition groups on a page.

`root`

The `view-transition-name` applied to `:root` causes the pseudo-element to match the default `root` view transition group. This is the snapshot group created by the user agent to contain the view transition for the overall page. This group includes any element not assigned to its own specific view transition snapshot group via the `view-transition-name` property.

`<pt-name-selector>`

The `<custom-ident>` set as the value of the `view-transition-name` property.

`<pt-class-selector>`

The `<custom-ident>` set as the value of the `view-transition-class` property, preceded by a period (`.`).

The specificity of the named view transition pseudo-element is equal to the specificity of the type selector, unless the parameter used is the universal selector is used, in which case the specificity is zero.

Examples

```
::view-transition-group(embed-container) {
  animation-duration: 0.3s;
  animation-timing-function: ease;
  z-index: 1;
}

::view-transition-group(.card) {
  animation-duration: 1s;
}
```

See also

- View Transition API
- Smooth transitions with the View Transition API

::view-transition-image-pair() CSS pseudo-element

The **`::view-transition-image-pair()`** CSS pseudo-element represents a container for a view transition's "old" and "new" view states — before and after the transition.

During a view transition, `::view-transition-image-pair()` is included in the associated pseudo-element tree as explained in The view transition pseudo-element tree. It is only ever a child of a `::view-transition-group()`. In terms of children, it can have a `::view-transition-new()` or a `::view-transition-old()`, or both.

`::view-transition-image-pair()` is given the following default styling in the UA stylesheet:

```
:root::view-transition-image-pair(*) {
  position: absolute;
  inset: 0;

  animation-duration: inherit;
  animation-fill-mode: inherit;
  animation-delay: inherit;
}
```

During a view transition, `::view-transition-image-pair()` has `isolation: isolate` set on it in the view transition style sheet so that its children can be blended with non-normal blend modes without affecting other visual outputs.

Syntax

```
::view-transition-image-pair([ <pt-name-selector> <pt-class-selector>? ] | <pt-class-selector>) {
  /* ... */
}
```

Parameters

`*`

The universal selector (`*`); selects all view transition groups on a page.

`root`

Causes the pseudo-element to match the default `root` view transition snapshot group created by the user agent to contain the view transition for the overall page. This group includes any element not assigned to its own specific view transition snapshot group via the `view-transition-name` property.

`<pt-name-selector>`

The `<custom-ident>` set as the value of the `view-transition-name` property.

`<pt-class-selector>`

The `<custom-ident>` set as the value of the `view-transition-class` property preceded by a period (`.`).

Examples

```
::view-transition-image-pair(root) {
  isolation: auto;
}

::view-transition-image-pair(.card) {
  isolation: isolate;
}
```

See also

- View Transition API
- Smooth transitions with the View Transition API

::view-transition-new() CSS pseudo-element

The **`::view-transition-new()`** CSS pseudo-element represents the "new" view state of a view transition — a snapshot live representation of the state after the transition.

During a view transition, `::view-transition-new()` is included in the associated pseudo-element tree as explained in The view transition pseudo-element tree. It is only ever a child of a `::view-transition-image-pair()`, and never has any children.

It is a replaced element and therefore can be manipulated with properties such as `object-fit` and `object-position`. It has natural dimensions equal to the content's size.

The following default styling is included in the UA stylesheet:

```
:root::view-transition-old(*),
:root::view-transition-new(*) {
  position: absolute;
  inset-block-start: 0;
  inline-size: 100%;
  block-size: auto;

  animation-duration: inherit;
```

```
  animation-fill-mode: inherit;
  animation-delay: inherit;
}

/* Keyframes for blending when there are 2 images */
@keyframes -ua-mix-blend-mode-plus-lighter {
  from {
    mix-blend-mode: plus-lighter;
  }
  to {
    mix-blend-mode: plus-lighter;
  }
}

@keyframes -ua-view-transition-fade-in {
  from {
    opacity: 0;
  }
}
```

> **Note:** Additional view transition styles are also setup to animate `::view-transition-new()`. These are dynamically generated during the view transition; see the specification setup transition pseudo-elements and update pseudo-element styles sections for more details.

Syntax

```
::view-transition-new([ <pt-name-selector> <pt-class-selector>? ] | <pt-class-selector>) {
  /* ... */
}
```

Parameters

`*`

The universal selector (`*`); selects all view transition groups on a page.

`root`

Causes the pseudo-element to match the default `root` view transition snapshot group created by the user agent to contain the view transition for the overall page. This group includes any element not assigned to its own specific view transition snapshot group via the `view-transition-name` property.

`<pt-name-selector>`

The `<custom-ident>` set as the value of the `view-transition-name` property.

`<pt-class-selector>`

The `<custom-ident>` set as the value of the `view-transition-class` property preceded by a period (`.`).

Examples

```
figcaption {
  view-transition-name: figure-caption;
}

@keyframes grow-x {
  from {
    transform: scaleX(0);
  }
  to {
    transform: scaleX(1);
  }
}

@keyframes shrink-x {
  from {
    transform: scaleX(1);
  }
  to {
    transform: scaleX(0);
  }
}

::view-transition-old(figure-caption),
::view-transition-new(figure-caption) {
  height: auto;
  right: 0;
  left: auto;
  transform-origin: right center;
}

::view-transition-old(figure-caption) {
  animation: 0.25s linear both shrink-x;
}

::view-transition-new(figure-caption) {
  animation: 0.25s 0.25s linear both grow-x;
}
```

See also

- View Transition API
- Smooth transitions with the View Transition API

::view-transition-old() CSS pseudo-element

The **`::view-transition-old()`** CSS pseudo-element represents the "old" view state of a view transition — a static snapshot of the old view, before the transition.

During a view transition, `::view-transition-old()` is included in the associated pseudo-element tree as explained in The view transition pseudo-element tree, provided there's an "old" view state to represent. It is only ever a child of a `::view-transition-image-pair()`, and never has any children.

It is a replaced element and therefore can be manipulated with properties such as `object-fit` and `object-position`. It has natural dimensions equal to the content's size.

The following default styling is included in the UA stylesheet:

```
:root::view-transition-old(*),
:root::view-transition-new(*) {
  position: absolute;
  inset-block-start: 0;
  inline-size: 100%;
  block-size: auto;

  animation-duration: inherit;
  animation-fill-mode: inherit;
  animation-delay: inherit;
}

/* Keyframes for blending when there are 2 images */
@keyframes -ua-mix-blend-mode-plus-lighter {
  from {
    mix-blend-mode: plus-lighter;
  }
  to {
    mix-blend-mode: plus-lighter;
  }
}

@keyframes -ua-view-transition-fade-out {
  to {
    opacity: 0;
  }
}
```

> **Note:** Additional view transition styles are also setup to animate `::view-transition-old()`. These are dynamically generated during the view transition; see the specification setup transition pseudo-elements and update pseudo-element styles sections for more details.

Syntax

```
::view-transition-old([ <pt-name-selector> <pt-class-selector>? ] | <pt-class-selector>) {
  /* ... */
}
```

Parameters

`*`

The universal selector (`*`) selects all view transition groups on a page.

`root`

Causes the pseudo-element to match the default `root` view transition snapshot group created by the user agent to contain the view transition for the overall page. This group includes any element not assigned to its own specific view transition snapshot group via the `view-transition-name` property.

`<pt-name-selector>`

The `<custom-ident>` set as the value of the `view-transition-name` property.

`<pt-class-selector>`

The `<custom-ident>` set as the value of the `view-transition-class` property preceded by a period (`.`).

Examples

```
figcaption {
  view-transition-name: figure-caption;
}

@keyframes grow-x {
  from {
    transform: scaleX(0);
  }
  to {
    transform: scaleX(1);
  }
}

@keyframes shrink-x {
  from {
    transform: scaleX(1);
  }
  to {
    transform: scaleX(0);
  }
}

::view-transition-old(figure-caption),
::view-transition-new(figure-caption) {
  height: auto;
  right: 0;
  left: auto;
  transform-origin: right center;
}

::view-transition-old(figure-caption) {
  animation: 0.25s linear both shrink-x;
}

::view-transition-new(figure-caption) {
  animation: 0.25s 0.25s linear both grow-x;
}
```

See also

- View Transition API
- Smooth transitions with the View Transition API

:active CSS pseudo-class

The **`:active`** CSS pseudo-class represents an element (such as a button) that is being activated by the user. When using a mouse, "activation" typically starts when the user presses down the primary mouse button.

Try it

```
.joinBtn {
  width: 10em;
  height: 5ex;
  background-image: linear-gradient(135deg, #f34079 40%, #fc894d);
  border: none;
  border-radius: 5px;
  font-weight: bold;
  color: white;
  cursor: pointer;
}

.joinBtn:active {
  box-shadow: 2px 2px 5px #fc894d;
}
```

```
<p>Would you like to subscribe to our channel?</p>
<button class="joinBtn">Subscribe</button>
```

The `:active` pseudo-class is commonly used on `<a>` and `<button>` elements. Other common targets of this pseudo-class include elements that are *contained in* an activated element, and form elements that are being activated through their associated `<label>`.

Styles defined by the `:active` pseudo-class will be overridden by any subsequent link-related pseudo-class (`:link`, `:hover`, or `:visited`) that has at least equal specificity. To style links appropriately, put the `:active` rule after all other link-related rules, as defined by the *LVHA-order*: `:link` — `:visited` — `:hover` — `:active`.

> **Note:** On systems with multi-button mice, CSS specifies that the `:active` pseudo-class must only apply to the primary button; on right-handed mice, this is typically the leftmost button.

Syntax

```
:active {
  /* ... */
}
```

Examples

Active links

HTML

```
<p>
  This paragraph contains a link:
  <a href="#">This link will turn red while you click on it.</a>
  The paragraph will get a gray background while you click on it or the link.
</p>
```

CSS

```
/* Unvisited links */
a:link {
  color: blue;
}
/* Visited links */
a:visited {
  color: purple;
}
/* Hovered links */
a:hover {
  background: yellow;
}
/* Active links */
a:active {
  color: red;
}

/* Active paragraphs */
p:active {
  background: #eeeeee;
}
```

Result

Active form elements

HTML

```
<form>
  <label for="my-button">My button: </label>
  <button id="my-button" type="button">Try Clicking Me or My Label!</button>
</form>
```

CSS

```
form :active {
  color: red;
}
```

```
form button {
  background: white;
}
```

Result

See also

- Link-related pseudo-classes: `:link`, `:visited`, and `:hover`

:active-view-transition CSS pseudo-class

The **`:active-view-transition`** CSS pseudo-class matches the root element of a document when a view transition is in progress (*active*) and stops matching once the transition has completed.

Syntax

```
:root:active-view-transition ... {
  /* ... */
}
```

Examples

Styling an active view transition

This example extends the Basic view transition example on the `startViewTransition` page.

```
<main>
  <section class="color">
    <h2>Color is changing!</h2>
  </section>
  <button id="change-color">Change Color</button>
</main>
```

A `<h2>` element initially has a `display: none` style, and this is overwritten using the `:active-view-transition` pseudo-class setting the `<h2>` style to `display: block`. The button is hidden using `visibility: hidden`, when the view transition is in progress:

```
html {
  --bg: indigo;
}
main {
  display: flex;
  flex-direction: column;
  gap: 5px;
}
section {
  background-color: var(--bg);
  height: 60px;
  border-radius: 5px;
  display: flex;
  justify-content: center;
  align-items: center;
}
::view-transition-group(root) {
  animation-duration: 2s;
}
```

```
h2 {
  display: none;
  color: white;
}
:root:active-view-transition h2 {
  display: block;
}
:root:active-view-transition button {
  visibility: hidden;
}
```

```
const colors = ["darkred", "darkslateblue", "darkgreen"];
const colBlock = document.querySelector(".color");
let count = 0;
const updateColor = () => {
  colBlock.style = `--bg: ${colors[count]}`;
  count = count !== colors.length - 1 ? ++count : 0;
};
const changeColor = () => {
  if (!document.startViewTransition) {
    updateColor();
    return;
  }
  const transition = document.startViewTransition(() => {
    updateColor();
  });
};
const changeColorButton = document.querySelector("#change-color");
changeColorButton.addEventListener("click", changeColor);
changeColorButton.addEventListener("keypress", changeColor);
```

See also

- `:active-view-transition-type()` pseudo-class
- `startViewTransition()` method
- `Document.activeViewTransition`
- View Transition API
- Using the View Transition API
- Using view transition types

:active-view-transition-type() CSS pseudo-class

The functional **`:active-view-transition-type()`** CSS pseudo-class matches elements when a view transition with one or more specific types is in progress (is *active*) and stops matching once the view transition has completed.

```
html:active-view-transition-type(forwards, backwards) {
  /* ... */
}
```

Syntax

```
:active-view-transition-type(<custom-ident>#) {
  /* ... */
}
```

Parameters

`<custom-ident>#`

One or more comma-separated `<custom-ident>` values representing the choice of types that can be applied to the active view transition for this selector to match.

Description

View transition types provide a mechanism by which different **types** can be specified for active view transitions. Types can be set on view transitions in multiple ways:

- For same-document (SPA) view transitions, specify types in the `types` option of the `startViewTransition()` method.
- For cross-document view transitions, specify types in the `types` descriptor of the `@view-transition` at-rule.
- You can also modify the active view transition's types on-the-fly via the `types` property of the corresponding `ViewTransition` object:
  - For same-document view transitions, this is the `ViewTransition` object returned by the `startViewTransition()` method.
  - For cross-document view transitions, the `ViewTransition` object is available in the `viewTransition` property of the `pageswap` event object in the case of the outgoing page, and the `viewTransition` property of the `pagereveal` event object in the case of the incoming page.
  - You can also access the active `ViewTransition` via the `Document.activeViewTransition` property. This provides a consistent way to access the active view transition in any context.

Once the active view transition has one or more types set on it, the `:active-view-transition-type()` pseudo-class can be applied to the document root element to set custom styles for each type. The pseudo-class takes a comma-separated list of types as its argument to specify the types that can be set on the active view transition for the selector to be matched.

For example, you might want to apply different transition animations to an `<img>` element in an image gallery app as the displayed image changes, depending on whether you are moving forward or backward in the sequence, deleting an image, or adding an image into the sequence.

OR versus AND behavior

It is important to note that the comma-separated list of types specified inside the `:active-view-transition-type()` pseudo-class provides OR behavior — if one or more of these types are set on the active view transition, the selector will match.

For example, in this case, the selector will match if the active view transition has a type of `forwards`, `backwards`, or both:

```
html:active-view-transition-type(forwards, backwards) {
  /* ... */
}
```

If you want to specify AND behavior — that is, all the types must be set for the selector to match — you can do so by chaining multiple `:active-view-transition-type()` pseudo-classes together. In the following case, the selector will only match if the active view transition has types of `slide` *and* `forwards`:

```
html:active-view-transition-type(slide):active-view-transition-type(forwards) {
  /* ... */
}
```

Examples

See also Using view transition types for multiple complete examples.

Basic <code>:active-view-transition-type()</code> usage

This example includes a basic document that transitions between two different pieces of content via two different buttons, "Backwards" and "Forwards". We demonstrate how view transition types can be used to animate the transition differently depending on which button was pressed.

HTML

The markup includes a single `<p>` element to contain the content and two `<button>` elements to trigger the view transitions.

```
<p>This is my first piece of content. I hope you like it!</p>
<div>
  <button id="backwards">Backwards</button>
  <button id="forwards">Forwards</button>
</div>
```

JavaScript

In our script, we create references to both buttons and the content paragraph, and then store our two different pieces of content in two constants.

```
const backBtn = document.getElementById("backwards");
const fwdBtn = document.getElementById("forwards");
const content = document.querySelector("p");

const first = "This is my first piece of content. I hope you like it!";
const second =
  "This is my second piece of content. Is it better than the first?";
```

Next, we add `click` event listeners to the backwards and forwards buttons; when they are clicked, the `changeContent()` function is run.

```
backBtn.addEventListener("click", changeContent);
fwdBtn.addEventListener("click", changeContent);
```

Finally, we define the `changeContent()` function. We start by declaring a `type` value that will hold our view transition type value. If the event target is the "Backwards" button, we set `type` to `backwards`. If not, we set `type` to `forwards`. We then invoke the `startViewTransition()` method to update the content and start the transition:

- The `update` callback checks whether the paragraph `textContent` is equal to the `first` string. If so, we set it to the `second` string. If not, we set it to the `first` string.
- The `types` array is given one element: the `type` value we declared earlier.

```
function changeContent(e) {
  const type = e.target === backBtn ? "backwards" : "forwards";
  document.startViewTransition({
    update() {
      content.textContent === first
        ? (content.textContent = second)
        : (content.textContent = first);
    },
    types: [type],
  });
}
```

CSS

In our styles, we start off by creating a set of nested rules using the `:active-view-transition` pseudo-class. These styles will be applied whenever a view transition is active, regardless of the their types. We apply a `view-transition-name` of `none` to the `:root`, as we don't want any elements captured and animated on transition except for the `<p>` element, which is given a `view-transition-name` of `slide`.

```
html,
body {
  height: 100%;
}

body {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

div {
  display: flex;
  width: 60%;
  justify-content: space-between;
}

p {
  font-size: 1.7em;
  width: 60%;
  color: blue;
  margin-top: 0;
}
```

```
html:active-view-transition {
  :root {
    view-transition-name: none;
  }
  p {
    view-transition-name: slide;
  }
}
```

Next, we use the `:active-view-transition-type()` pseudo-class to create two blocks of nested styles, the first of which is only applied when the active view transition has a type of `forwards`, and the second of which is only applied when the active view transition has a type of `backwards`. In each block, we use the `::view-transition-old()` and `::view-transition-new()` pseudo-elements to apply custom `animation-name` values to the `slide` capture group's outgoing and incoming views.

As a result:

- When the transition type is `forwards`, the old content view slides out to the left, and the new content view slides in from the right.
- When the transition type is `backwards`, the old content view slides out to the right, and the new content view slides in from the left.

```
html:active-view-transition-type(forwards) {
  &::view-transition-old(slide) {
    animation-name: slide-out-to-left;
  }
  &::view-transition-new(slide) {
    animation-name: slide-in-from-right;
  }
}

html:active-view-transition-type(backwards) {
  &::view-transition-old(slide) {
    animation-name: slide-out-to-right;
  }
  &::view-transition-new(slide) {
    animation-name: slide-in-from-left;
  }
}
```

Finally, we use `@keyframes` animation blocks to define the animations referenced previously.

```
@keyframes slide-in-from-left {
  from {
    translate: -100vw 0;
  }
}
@keyframes slide-in-from-right {
  from {
    translate: 100vw 0;
  }
}
@keyframes slide-out-to-left {
  to {
    translate: -100vw 0;
  }
}
@keyframes slide-out-to-right {
```

```
  to {
    translate: 100vw 0;
  }
}
```

Result

The example is rendered like so:

Try clicking the "Backwards" and "Forwards" buttons, and note how, even though the same code is used to trigger the content update and view transition in each case, a different animation is used for the transition. This is because a different transition type is set depending on which button is pressed.

See also

- `Document.activeViewTransition`
- `:active-view-transition` pseudo-class
- View Transition API
- Using the View Transition API
- Using view transition types

:any-link CSS pseudo-class

The **`:any-link`** CSS pseudo-class selector represents an element that acts as the source anchor of a hyperlink, independent of whether it has been visited. In other words, it matches every `<a>` or `<area>` element that has an `href` attribute. Thus, it matches all elements that match `:link` or `:visited`.

Try it

```
p {
  font-weight: bold;
}

a:any-link {
  color: forestgreen;
  text-decoration-color: hotpink;
}
```

```
<p>Pages that you might have visited:</p>
<ul>
  <li>
    <a href="https://developer.mozilla.org">MDN Web Docs</a>
  </li>
  <li>
    <a href="https://www.youtube.com/YouTube">Google</a>
  </li>
</ul>
<p>Pages unlikely to be in your history:</p>
<ul>
  <li>
    <a href="https://developer.mozilla.org/missing-3">Random MDN page</a>
  </li>
  <li>
    <a href="https://example.com/missing-3">Random Example page</a>
  </li>
</ul>
```

Syntax

```
:any-link {
  /* ... */
}
```

Examples

HTML

```
<a href="https://example.com">External link</a><br />
<a href="#">Internal target link</a><br />
<a>Placeholder link (won't get styled)</a>
```

CSS

```
a:any-link {
  border: 1px solid blue;
  color: orange;
}

/* WebKit browsers */
a:-webkit-any-link {
  border: 1px solid blue;
  color: orange;
}
```

Result

See also

- Creating links
- Matches HTML elements: `<a>` and `<area>` with an `href` attribute
- Related CSS selectors:
  - `:visited`
  - `:link`

:autofill CSS pseudo-class

The **`:autofill`** CSS pseudo-class matches when an `<input>` element has its value autofilled by the browser. The class stops matching if the user edits the field.

Try it

```
input {
  border: 3px solid black;
}
input:autofill {
  outline: 5px solid magenta;
  border: 3px dashed yellow;
}
```

```
<form>
  <p>Click on the text box and choose any option suggested by your browser.</p>
  <p>
    <label for="name">Name</label>
    <input id="name" name="name" type="text" autocomplete="given-name" />
  </p>
  <p>
    <label for="email">Email Address</label>
    <input id="email" name="email" type="email" autocomplete="email" />
  </p>
  <p>
    <label for="country">Country</label>
    <input
      id="country"
      name="country"
      type="text"
      autocomplete="country-name" />
  </p>
</form>
```

Description

The `:autofill` pseudo-class selects `<input>` elements containing un-edited content that was auto-filled by the user-agent. Autocompletion may occur when browser settings are set to enable autocompletion or the `autocomplete` attribute is set on the element itself.

When the user uses the browser's autocompletion feature to autofill a form control, all the form controls that can be autofilled based on the selection get populated. These will all match the `:autofill` UI state. If the user edits a control, that control will no longer match `:autofill`, even if the value is the same as the autofilled value.

The specification includes the vendor-prefixed `:-webkit-autofill` pseudo-class as an alias. This also matches input elements autofilled by the user agent, and no longer matches if the user edits the autofilled field.

Note that the user agent style sheets of many browsers use `!important` in their style declarations, making some styles non-overridable. For example, Chrome has the following in its internal stylesheet:

```
input:-internal-autofill-selected {
  appearance: menulist-button;
  background-image: none !important;
  background-color:
    light-dark(rgb(232 240 254), rgb(70 90 126 / 0.4)) !important;
  color: fieldtext !important;
}
```

This means that you cannot override the default `background-color`, `background-image`, or `color` values on selected autofilled input elements in your own rules, but you can override the `appearance`, and you can override these properties for the autofilled elements that aren't currently focused.

Syntax

```
:autofill {
  /* ... */
}
```

Examples

The following example demonstrates the use of the `:autofill` pseudo-class to change the border of a text field that has been autocompleted by the browser.

HTML

We include three HTML `<input>` elements, each with an associated `<label>`. The `name` and `email` will likely autocomplete and match `:autofill`, while the `pet` value is unlikely to.

```
<form method="post" action="">
  <p>
    <label for="name">Name: </label>
    <input name="n" id="name" autocomplete="given-name" />
  </p>
  <p>
    <label for="email">Email: </label>
    <input type="email" name="email" id="email" autocomplete="email" />
  </p>
  <p>
    <label for="pet">Your pet's name: </label>
    <input name="pet" id="pet" />
  </p>
</form>
```

CSS

We apply a `border-radius`, `outline`, and `border-color` to the `<input>` elements when they match the `:autofill` state.

```
input {
  border-radius: 0px;
}

input:autofill {
  border-radius: 0.5lh;
  outline: 5px dashed magenta;
  border-color: yellow;
}
```

```
@supports not selector(:autofill) {
  body::before {
```

```
    content: "Your browser doesn't support the :autofill selector.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Results

If you allow the `name` to autocomplete, the `name` and `email` will both have a rounded border and a magenta outline, though likely the border will not change due to the `!important` flag set in the user agent style sheet. Try editing one of the fields: note how once you edit the value, even if you reset it to it's autocompletion value, the `:autofill` styles are no longer applied.

See also

- `<input>` pseudo-classes
- CSS selectors module

:blank CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

> **Note:** The `:blank` selector is considered at risk, as the CSSWG keeps changing it.
>
> See CSSWG issue #1967.

The **`:blank`** CSS pseudo-class selects empty user input elements (e.g., `<input>` or `<textarea>`).

Syntax

```
:blank {
  /* ... */
}
```

Examples

Basic :blank example

In eventual supporting browsers, the `:blank` pseudo-class will enable developers to highlight in some way input controls that are not required, but still have no content filled in, perhaps as a reminder to users.

HTML

```
<textarea></textarea>
```

CSS

```
textarea:blank {
  border: 3px solid red;
}
```

Result

Browser compatibility

Currently, no browsers support this feature.

See also

- `:empty`
- CSS selectors module

:buffering CSS pseudo-class

The **`:buffering`** CSS pseudo-class selector represents an element that is playable, such as `<audio>` or `<video>`, when the playable element is buffering a media resource.

An element is considered as buffering when that element cannot continue playing because it is trying to load media data but does not yet have enough data to begin or continue playback. For more information, see the Media buffering, seeking, and time ranges guide.

> **Note:** An element is still considered to be `:playing` when it is "buffering". If `:buffering` matches an element, `:playing` will also match that element.

Syntax

```
:buffering {
  /* ... */
}
```

Examples

CSS

```
:buffering {
  outline: 5px solid red;
}

video:buffering {
  outline: 5px solid blue;
}
```

See also

- `:muted`
- `:paused`

- `:playing`
- `:seeking`
- `:stalled`
- `:volume-locked`
- CSS selectors

:checked CSS pseudo-class

The **`:checked`** CSS pseudo-class selector represents any **radio** (`<input type="radio">`), **checkbox** (`<input type="checkbox">`), or **option** (`<option>` in a `<select>` element) that is checked or toggled to an `on` state.

Try it

```
label {
  display: block;
  margin-top: 0.5em;
}

fieldset {
  margin-top: 0.5em;
}

input:checked {
  border: none;
  outline: 2px solid deeppink;
}
```

```
<label for="name">Name (required):</label>
<input id="name" name="name" type="text" required />

<label for="birth">Date of birth:</label>
<input id="birth" name="birth" type="date" />

<label for="origin">How did you find out about us? (required)</label>
<select id="origin" name="origin" required>
  <option value="">Choose an option</option>
  <option value="google">Google</option>
  <option value="facebook">Facebook</option>
  <option value="advertisement">Advertisement</option>
</select>

<fieldset>
  <legend>Preferred contact method:</legend>
  <label>
    <input name="contact" type="radio" value="email" checked /> Email
  </label>
  <label><input name="contact" type="radio" value="phone" /> Phone</label>
</fieldset>

<label>
  <input name="newsletter" type="checkbox" checked /> I want to subscribe to a
  personalized newsletter.
</label>
<label>
  <input name="privacy" type="checkbox" required /> I have read and I agree to
  the Privacy Policy (required).
</label>
```

The user can engage this state by checking/selecting an element, or disengage it by unchecking/deselecting the element.

> **Note:** Because browsers often treat `<option>`s as replaced elements, the extent to which they can be styled with the `:checked` pseudo-class varies from browser to browser. Customizable select element functionality can be used to enable full customization of `<option>` elements just like any regular DOM element, in supporting browsers.

Syntax

```
:checked {
  /* ... */
}
```

Examples

Basic example

HTML

```
<div>
  <input type="radio" name="my-input" id="yes" value="yes" />
  <label for="yes">Yes</label>

  <input type="radio" name="my-input" id="no" value="no" />
  <label for="no">No</label>
</div>

<div>
  <input type="checkbox" name="my-checkbox" id="opt-in" />
  <label for="opt-in">Check me!</label>
</div>

<select name="my-select" id="fruit">
  <option value="opt1">Apples</option>
  <option value="opt2">Grapes</option>
  <option value="opt3">Pears</option>
</select>
```

CSS

```
div,
select {
  margin: 8px;
```

```
}

/* Labels for checked inputs */
input:checked + label {
  color: red;
}

/* Radio element, when checked */
input[type="radio"]:checked {
  box-shadow: 0 0 0 3px orange;
}

/* Checkbox element, when checked */
input[type="checkbox"]:checked {
  box-shadow: 0 0 0 3px hotpink;
}

/* Option elements, when selected */
option:checked {
  box-shadow: 0 0 0 3px lime;
  color: red;
}
```

Result

Toggling elements with a hidden checkbox

This example utilizes the `:checked` pseudo-class to let the user toggle content based on the state of a checkbox, all without using JavaScript.

HTML

```
<input type="checkbox" id="expand-toggle" />

<table>
  <thead>
    <tr>
      <th>Column #1</th>
      <th>Column #2</th>
      <th>Column #3</th>
    </tr>
  </thead>
  <tbody>
    <tr class="expandable">
      <td>[more text]</td>
      <td>[more text]</td>
      <td>[more text]</td>
    </tr>
    <tr>
      <td>[cell text]</td>
      <td>[cell text]</td>
      <td>[cell text]</td>
    </tr>
    <tr>
      <td>[cell text]</td>
      <td>[cell text]</td>
      <td>[cell text]</td>
    </tr>
    <tr class="expandable">
      <td>[more text]</td>
      <td>[more text]</td>
      <td>[more text]</td>
    </tr>
    <tr class="expandable">
      <td>[more text]</td>
      <td>[more text]</td>
      <td>[more text]</td>
    </tr>
  </tbody>
</table>

<label for="expand-toggle" id="expand-btn">Toggle hidden rows</label>
```

CSS

```
/* Hide the toggle checkbox */
#expand-toggle {
  display: none;
}

/* Hide expandable content by default */
.expandable {
  visibility: collapse;
  background: #dddddd;
}

/* Style the button */
#expand-btn {
  display: inline-block;
  margin-top: 12px;
  padding: 5px 11px;
  background-color: #ffff77;
  border: 1px solid;
  border-radius: 3px;
}

/* Show hidden content when the checkbox is checked */
#expand-toggle:checked ~ * .expandable {
  visibility: visible;
}

/* Style the button when the checkbox is checked */
#expand-toggle:checked ~ #expand-btn {
```

```
  background-color: #cccccc;
}
```

Result

See also

- Web forms — working with user data
- Styling web forms
- Related HTML elements: `<input type="checkbox">`, `<input type="radio">`, `<select>`, and `<option>`
- Replaced elements

:current CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:current`** CSS pseudo-class selector represents an element or the ancestor of an element that is currently being displayed or highlighted.

Syntax

```
:current {
  /* ... */
}

:current(<compound-selector-list>) {
  /* ... */
}
```

Description

The `:current` pseudo-class is used to represent the "currently displayed" element out of a series of elements.

This can mean "current" in terms of time: `:current` can be used to target the currently displayed captions or subtitles (represented using WebVTT) that are associated with a playing video.

It can also refer to the currently highlighted element in a series. For example, `:current` can be combined with the `::search-text` pseudo-element to provide specific styles to the currently-focused search result from the browser's "Find in page" text search feature.

For example:

```
p::search-text {
  color: white;
  background-color: purple;
}

p::search-text:current {
  background-color: crimson;
}
```

Examples

Custom styles for text search results

This example shows how to use `::search-text` and `:current` to create custom styles for your browser's "Find in page" search results.

HTML

The HTML consists of a basic paragraph of text. We won't show the HTML source, both for brevity, and so that it is easier to navigate the search results in the rendered example.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec finibus est
  eget eros congue pellentesque. Etiam a augue accumsan, scelerisque nisl sit
  amet, lobortis nulla. Aliquam condimentum eu orci eu elementum. Donec
  porttitor quam et posuere commodo. Mauris rhoncus diam a scelerisque molestie.
  Integer sollicitudin risus dui, ut sagittis lorem laoreet eget. Duis eget
  pretium enim. Morbi tristique, diam sit amet gravida finibus, metus ex
  tincidunt nibh, ac volutpat urna purus et arcu. Donec risus risus, semper vel
  purus sit amet, gravida vestibulum est. Sed et tristique urna. Nam vel mi eget
  nisi consectetur elementum. Aenean faucibus aliquam cursus. Morbi posuere
  tincidunt velit, et sagittis quam sagittis in. Nam eget ante ultrices, auctor
  dui vel, euismod lacus. Vivamus tincidunt, sem ac sodales aliquet, tortor
  tortor consequat diam, nec tempor mi dui vel eros. Aliquam ac erat et metus
  egestas scelerisque.
</p>
```

CSS

In our CSS, we start by styling the `::search-text` pseudo-element. We give it custom `background-color`, `color`, and `text-shadow` styles.

```
html {
  font-family: "Helvetica", "Arial";
}

p {
  font-size: 1.5rem;
  line-height: 1.5;
  width: 90%;
  margin: 0 auto;
}

@layer no-support {
  body::before {
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1em 0;
  }
  @supports not selector(::search-text) {
```

```
    body::before {
      content: "Your browser doesn't support the ::search-text pseudo-element.";
    }
  }
  @supports not selector(:current) {
    body::before {
      content: "Your browser doesn't support the :current pseudo-class.";
    }
  }
}
```

```
::search-text {
  background-color: purple;
  color: white;
  text-shadow: 1px 1px 1px black;
}
```

Finally, we style the currently-focused search result via `::search-text:current`, providing it with a different `background-color` and some `text-decoration` styles so that it is distinguishable from the rest of the results.

```
::search-text:current {
  background-color: crimson;
  text-decoration-line: underline;
  text-decoration-color: yellow;
  text-decoration-thickness: 3px;
}
```

Result

The example renders as follows:

Try using the browser's find in page interface to find a word that appears multiple times in the example text, such as "aliquam", "amet", or "tortor". Move between the previous and next results to check out the `:current` styling.

Styling currently-shown WebVTT subtitles

CSS

```
:current(p, span) {
  background-color: yellow;
}
```

HTML

```
<video controls preload="metadata">
  <source src="video.mp4" type="video/mp4" />
  <source src="video.webm" type="video/webm" />
  <track
    label="English"
    kind="subtitles"
    srclang="en"
    src="subtitles.vtt"
    default />
</video>
```

WebVTT

```
WEBVTT FILE

1
00:00:03.500 --> 00:00:05.000
This is the first caption

2
00:00:06.000 --> 00:00:09.000
This is the second caption

3
00:00:11.000 --> 00:00:19.000
This is the third caption
```

See also

- Web Video Text Tracks Format (WebVTT)
- `:past`
- `:future`
- CSS selectors module

:default CSS pseudo-class

The **`:default`** CSS pseudo-class selects form elements that are the default in a group of related elements.

Try it

```
label {
  display: block;
  margin-top: 0.5em;
}

fieldset {
  margin-top: 0.5em;
}

input:default {
  border: none;
  outline: 2px solid deeppink;
}
```

```
<label for="name">Name (required):</label>
<input id="name" name="name" type="text" required />

<label for="birth">Date of birth:</label>
<input id="birth" name="birth" type="date" />

<label for="origin">How did you find out about us? (required)</label>
<select id="origin" name="origin" required>
  <option value="">Choose an option</option>
  <option value="google">Google</option>
  <option value="facebook">Facebook</option>
  <option value="advertisement">Advertisement</option>
</select>

<fieldset>
  <legend>Preferred contact method:</legend>
  <label>
    <input name="contact" type="radio" value="email" checked /> Email
  </label>
  <label><input name="contact" type="radio" value="phone" /> Phone</label>
</fieldset>

<label>
  <input name="newsletter" type="checkbox" checked /> I want to subscribe to a
  personalized newsletter.
</label>
<label>
  <input name="privacy" type="checkbox" required /> I have read and I agree to
  the Privacy Policy (required).
</label>
```

What this selector matches is defined in HTML Standard §4.16.3 Pseudo-classes — it may match the `<button>`, `<input type="checkbox">`, `<input type="radio">`, and `<option>` elements:

- A default option element is the first one with the `selected` attribute, or the first enabled option in DOM order. `multiple` `<select>`s can have more than one `selected` option, so all will match `:default`.
- `<input type="checkbox">` and `<input type="radio">` match if they have the `checked` attribute.
- `<button>` matches if it is a `<form>`'s default submission button: the first `<button>` in DOM order that belongs to the form. This also applies to `<input>` types that submit forms, like `image` or `submit`.

Syntax

```
:default {
  /* ... */
}
```

Examples

HTML

```
<fieldset>
  <legend>Favorite season</legend>

  <input type="radio" name="season" id="spring" value="spring" />
  <label for="spring">Spring</label>

  <input type="radio" name="season" id="summer" value="summer" checked />
  <label for="summer">Summer</label>

  <input type="radio" name="season" id="fall" value="fall" />
  <label for="fall">Fall</label>

  <input type="radio" name="season" id="winter" value="winter" />
  <label for="winter">Winter</label>
</fieldset>
```

CSS

```
input:default {
  box-shadow: 0 0 2px 1px coral;
}

input:default + label {
  color: coral;
}
```

Result

See also

- Web forms — Working with user data
- Styling web forms
- Related HTML elements: `<button>`, `<input type="checkbox">`, `<input type="radio">`, and `<option>`

:defined CSS pseudo-class

The **`:defined`** CSS pseudo-class represents any element that has been defined. This includes any standard element built into the browser and custom elements that have been successfully defined (i.e., with the `CustomElementRegistry.define()` method).

```
/* Selects any defined element */
:defined {
  font-style: italic;
}

/* Selects any instance of a specific custom element */
custom-element:defined {
  display: block;
}
```

Syntax

```
:defined {
  /* ... */
}
```

Examples

Hiding elements until they are defined

In this demo, we define a basic custom element named `<custom-element>` and use the `:not(:defined)` and `:defined` selectors to style the element before and after it is defined. This is useful if you have a complex custom element that takes a while to load into the page — you might want to hide instances of the element until the definition is complete so that you don't end up with flashes of ugly unstyled elements on the page.

HTML

The following HTML code uses the custom element but the element hasn't been defined yet. We also include a `<button>` that will define the custom element when clicked, allowing you to see its state before and after definition.

```
<custom-element>
  <p>
    Loaded content: Lorem ipsum tel sed tellus eiusmod tellus. Aenean. Semper
    dolor sit nisi. Elit porttitor nisi sit vivamus.
  </p>
</custom-element>

<button id="btn">define the <code>&amp;lt;custom-element&amp;gt;</code></button>
```

CSS

```
custom-element {
  display: block;
  border: 5px dashed grey;
  border-radius: 1rem;
  height: 100px;
  width: 400px;
  padding: 1rem;
  position: relative;
  user-select: none;
}

code {
  background: #cccccc;
}

#btn {
  margin-top: 1rem;
  cursor: pointer;
}
```

In the following CSS, we use the `custom-element:not(:defined)` selector to select the element and color it gray while it is not defined and the `custom-element:defined` selector to select the element and color it black after it is defined.

```
custom-element:not(:defined) {
  border-color: grey;
  color: grey;
}

custom-element:defined {
  background-color: wheat;
  border-color: black;
  color: black;
}

/* show loading message */
custom-element:not(:defined)::before {
  content: "Loading...";
  position: absolute;
  inset: 0;
  align-content: center;
  text-align: center;
  font-size: 2rem;
  background-color: white;
  border-radius: 1rem;
}

/* remove the loading message */
custom-element:defined::before {
  content: "";
}
```

We have also used the `::before` pseudo-element to show a "Loading..." overlay message until the element is defined. After definition, it is removed by setting the `content` to an empty string.

The following JavaScript has been used to define the custom element. To allow you to see the state of the custom element before and after definition we run the `define()` method when the button is clicked.

```
const btn = document.querySelector("#btn");

btn.addEventListener("click", () => {
  customElements.define("custom-element", class extends HTMLElement {});
  btn.remove();
});
```

Result

See also

- Web components

:dir() CSS pseudo-class

The **`:dir()`** CSS pseudo-class matches elements based on the directionality of the text contained in them.

```
/* Selects any element with right-to-left text */
:dir(rtl) {
  background-color: red;
}
```

The `:dir()` pseudo-class uses only the *semantic* value of the directionality, i.e., the one defined in the document itself. It doesn't account for *styling* directionality, i.e., the directionality set by CSS properties such as `direction`.

> **Note:** Be aware that the behavior of the `:dir()` pseudo-class is not equivalent to the `[dir=…]` attribute selectors. The latter match the HTML `dir` attribute, and ignore elements that lack it — even if they inherit a direction from their parent. (Similarly, `[dir=rtl]` and `[dir=ltr]` won't match the `auto` value.) In contrast, `:dir()` will match the value calculated by the user agent, even if inherited.

> **Note:** In HTML, the direction is determined by the `dir` attribute. Other document types may have different methods.

Syntax

```
:dir([ltr | rtl]) {
  /* ... */
}
```

Parameters

The `:dir()` pseudo-class requires one parameter, representing the text directionality you want to target.

`ltr`

Target left-to-right elements.

`rtl`

Target right-to-left elements.

Examples

HTML

```
<div dir="rtl">
  <span>test1</span>
  <div dir="ltr">
    test2
    <div dir="auto">עִבְרִית</div>
  </div>
</div>
```

CSS

```
:dir(ltr) {
  background-color: yellow;
}

:dir(rtl) {
  background-color: powderblue;
}
```

Result

See also

- Language-related pseudo-classes: `:lang`
- HTML `lang` attribute
- HTML `translate` attribute

:disabled CSS pseudo-class

The **`:disabled`** CSS pseudo-class represents any disabled element. An element is disabled if it can't be activated (selected, clicked on, typed into, etc.) or accept focus. The element also has an enabled state, in which it can be activated or accept focus.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

*:disabled {
  background-color: dimgrey;
  color: linen;
  opacity: 1;
}
```

```
<form>
  <label for="name">Name:</label>
  <input id="name" name="name" type="text" />

  <label for="emp">Employed:</label>
  <select id="emp" name="emp" disabled>
    <option>No</option>
    <option>Yes</option>
  </select>

  <label for="empDate">Employment Date:</label>
  <input id="empDate" name="empDate" type="date" disabled />
```

```
  <label for="resume">Resume:</label>
  <input id="resume" name="resume" type="file" />
</form>
```

Syntax

```
:disabled {
  /* ... */
}
```

Examples

This example shows a basic shipping form. It uses the JavaScript `change` event to let the user enable/disable the billing fields.

HTML

```
<form action="#">
  <fieldset id="shipping">
    <legend>Shipping address</legend>
    <input type="text" placeholder="Name" />
    <input type="text" placeholder="Address" />
    <input type="text" placeholder="Zip Code" />
  </fieldset>
  <br />
  <fieldset id="billing">
    <legend>Billing address</legend>
    <label for="billing-checkbox">Same as shipping address:</label>
    <input type="checkbox" id="billing-checkbox" checked />
    <br />
    <input type="text" placeholder="Name" disabled />
    <input type="text" placeholder="Address" disabled />
    <input type="text" placeholder="Zip Code" disabled />
  </fieldset>
</form>
```

CSS

```
input[type="text"]:disabled {
  background: #cccccc;
}
```

JavaScript

Toggle the disabled input fields when the checkbox is clicked

```
const checkbox = document.querySelector("#billing-checkbox");
const billingItems = document.querySelectorAll('#billing input[type="text"]');

checkbox.addEventListener("change", () => {
  billingItems.forEach((item) => {
    item.disabled = !item.disabled;
  });
});
```

Result

Check/un-check the checkbox to change the styling on the billing fields.

See also

- `:enabled`

:empty CSS pseudo-class

The **`:empty`** CSS pseudo-class represents any element that has no children. Children can be either element nodes or text (including whitespace). Comments, processing instructions, and CSS `content` do not affect whether an element is considered empty.

Try it

```
div:empty {
  outline: 2px solid deeppink;
  height: 1em;
}
```

```
<p>Element with no content:</p>
<div></div>

<p>Element with comment:</p>
<div><!-- A comment --></div>

<p>Element with nested empty element:</p>
<div><p></p></div>
```

> **Note:** In Selectors Level 4, the `:empty` pseudo-class was changed to act like `:-moz-only-whitespace`, but no browser currently supports this yet.

Syntax

```
:empty {
  /* ... */
}
```

Accessibility

Assistive technology such as screen readers cannot parse interactive content that is empty. All interactive content must have an accessible name, which is created by providing a text value for the interactive control's parent element (anchors, buttons, etc.). Accessible names expose the interactive control to the accessibility tree, an API that communicates information useful for assistive technologies.

The text that provides the interactive control's accessible name can be hidden using a combination of properties that remove it visually from the screen but keep it parsable by assistive technology. This is commonly used for buttons that rely solely on an icon to convey purpose.

- What is an accessible name? | Vispero
- Hidden content for better a11y | Go Make Things
- MDN Understanding WCAG, Guideline 2.4 explanations
- Understanding Success Criterion 2.4.4 | W3C Understanding WCAG 2.0

Examples

HTML

```
<div class="box"><!-- I will be lime. --></div>
<div class="box">I will be pink.</div>
<div class="box">
  <!-- I will be pink in older browsers because of the whitespace around this comment. -->
</div>
<div class="box">
  <p>
    <!-- I will be pink in all browsers because of the non-collapsible whitespace and elements around this comment. -->
  </p>
</div>
```

CSS

```
body {
  display: flex;
  justify-content: space-around;
}
```

```
.box {
  background: pink;
  height: 80px;
  width: 80px;
}

.box:empty {
  background: lime;
}
```

Result

See also

- `:-moz-only-whitespace` – The prefixed implementation of the changes in Selectors Level 4
- `:blank`

:enabled CSS pseudo-class

The **`:enabled`** CSS pseudo-class represents any enabled element. An element is enabled if it can be activated (selected, clicked on, typed into, etc.) or accept focus. The element also has a disabled state, in which it can't be activated or accept focus.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

*:enabled {
  background-color: gold;
}
```

```
<form>
  <label for="name">Name:</label>
  <input id="name" name="name" type="text" />

  <label for="emp">Employed:</label>
  <select id="emp" name="emp" disabled>
    <option>No</option>
    <option>Yes</option>
  </select>

  <label for="empDate">Employment Date:</label>
  <input id="empDate" name="empDate" type="date" disabled />

  <label for="resume">Resume:</label>
  <input id="resume" name="resume" type="file" />
</form>
```

Syntax

```
:enabled {
  /* ... */
}
```

Examples

The following example makes the color of text and button `<input>`s green when enabled, and gray when disabled. This helps the user understand which elements can be interacted with.

HTML

```
<form action="url_of_form">
  <label for="FirstField">First field (enabled):</label>
  <input type="text" id="FirstField" value="Lorem" /><br />

  <label for="SecondField">Second field (disabled):</label>
  <input type="text" id="SecondField" value="Ipsum" disabled /><br />
```

```
  <input type="button" value="Submit" />
</form>
```

CSS

```
input:enabled {
  color: #22bb22;
}

input:disabled {
  color: #aaaaaa;
}
```

Result

See also

- `:disabled`

:first CSS pseudo-class

The **`:first`** CSS pseudo-class, used with the `@page` at-rule, represents the first page of a printed document. (See `:first-child` for general first element of a node.)

```
/* Selects the first page when printing */
@page :first {
  margin-left: 50%;
  margin-top: 50%;
}
```

> **Note:** You can't change all CSS properties with this pseudo-class. You can only change the margins, `orphans`, `widows`, and page breaks of the document. Furthermore, you may only use absolute-length units when defining the margins. All other properties will be ignored.

Syntax

```
:first {
  /* ... */
}
```

Examples

Using <code>:first</code> for page print styles

Press the "Print!" button to print the example. The words on the first page should be somewhere around the center, while other pages will have their contents at the default position:

```
<p>First Page.</p>
<p>Second Page.</p>
<button>Print!</button>
```

```
@page :first {
  size: 8.5in 11in;
  margin-left: 3in;
  margin-top: 5in;
}

p {
  page-break-after: always;
  font: 1.2em sans-serif;
}
```

```
document.querySelector("button").addEventListener("click", () => {
  window.print();
});
```

See also

- `@page`
- Other page-related pseudo-classes: `:left`, `:right`

:first-child CSS pseudo-class

The **`:first-child`** CSS pseudo-class represents the first element among a group of sibling elements.

Try it

```
p {
  font-weight: bold;
}

li:first-child {
  border: 2px solid orange;
}
```

```
<p>Track &amp;amp; field champions:</p>
<ul>
  <li>Adhemar da Silva</li>
  <li>Wang Junxia</li>
  <li>Wilma Rudolph</li>
  <li>Babe Didrikson-Zaharias</li>
  <li>Betty Cuthbert</li>
  <li>Fanny Blankers-Koen</li>
  <li>Florence Griffith-Joyner</li>
  <li>Irena Szewinska</li>
  <li>Jackie Joyner-Kersee</li>
  <li>Shirley Strickland</li>
  <li>Carl Lewis</li>
  <li>Emil Zatopek</li>
```

```
  <li>Haile Gebrselassie</li>
  <li>Jesse Owens</li>
  <li>Jim Thorpe</li>
  <li>Paavo Nurmi</li>
  <li>Sergei Bubka</li>
  <li>Usain Bolt</li>
</ul>
```

Syntax

```
:first-child {
  /* ... */
}
```

Examples

Basic example

HTML

```
<div>
  <p>This text is selected!</p>
  <p>This text isn't selected.</p>
</div>

<div>
  <h2>This text isn't selected: it's not a `p`.</h2>
  <p>This text isn't selected.</p>
</div>
```

CSS

```
p:first-child {
  color: lime;
  background-color: black;
  padding: 5px;
}
```

Result

Styling a list

HTML

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>
    Item 3
    <ul>
      <li>Item 3.1</li>
      <li>Item 3.2</li>
      <li>Item 3.3</li>
    </ul>
  </li>
</ul>
```

CSS

```
ul li {
  color: blue;
}

ul li:first-child {
  color: red;
  font-weight: bold;
}
```

Result

See also

- `:-moz-first-node`
- `:first-of-type`
- `:last-child`
- `:nth-child()`

:first-of-type CSS pseudo-class

The **`:first-of-type`** CSS pseudo-class represents the first element of its type (tag name) among a group of sibling elements.

Try it

```
dt {
  font-weight: bold;
}

dd {
  margin: 3px;
}

dd:first-of-type {
  border: 2px solid orange;
}
```

```
<dl>
  <dt>Vegetables:</dt>
  <dd>1. Tomatoes</dd>
  <dd>2. Cucumbers</dd>
```

```
  <dd>3. Mushrooms</dd>
  <dt>Fruits:</dt>
  <dd>4. Apples</dd>
  <dd>5. Mangos</dd>
  <dd>6. Pears</dd>
  <dd>7. Oranges</dd>
</dl>
```

Syntax

```
:first-of-type {
  /* ... */
}
```

Examples

Styling the first paragraph

HTML

```
<h2>Heading</h2>
<p>Paragraph 1</p>
<p>Paragraph 2</p>
```

CSS

```
p:first-of-type {
  color: red;
  font-style: italic;
}
```

Result

Nested elements

This example shows how nested elements can also be targeted. Note that the universal selector (*) is implied when no type selector is written.

HTML

```
<article>
  <div>This `div` is first!</div>
  <div>This <span>nested `span` is first</span>!</div>
  <div>
    This <em>nested `em` is first</em>, but this <em>nested `em` is last</em>!
  </div>
  <div>This <span>nested `span` gets styled</span>!</div>
  <p>This `p` qualifies!</p>
  <div>This is the final `div`.</div>
</article>
```

CSS

```
article :first-of-type {
  background-color: pink;
}
```

Result

See also

- `:first-child`, `:last-of-type`, `:nth-of-type`

:focus CSS pseudo-class

The **`:focus`** CSS pseudo-class represents an element (such as a form input) that has received focus. It is generally triggered when the user clicks or taps on an element or selects it with the keyboard's `Tab` key.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:focus {
  background-color: lightblue;
}

select:focus {
  background-color: ivory;
}
```

```
<p>Which flavor would you like to order?</p>
<label>Full Name: <input name="firstName" type="text" /></label>
<label
  >Flavor:
  <select name="flavor">
    <option>Cherry</option>
    <option>Green Tea</option>
    <option>Moose Tracks</option>
    <option>Mint Chip</option>
  </select>
</label>
```

> **Note:** This pseudo-class applies only to the focused element itself. Use `:focus-within` if you want to select an element that *contains* a focused element.

Syntax

```
:focus {
  /* ... */
}
```

Accessibility

Make sure the visual focus indicator can be seen by people with low vision. This will also benefit anyone use a screen in a brightly lit space (like outside in the sun). WCAG 2.1 SC 1.4.11 Non-Text Contrast requires that the visual focus indicator be at least 3 to 1.

- Accessible Visual Focus Indicators: Give Your Site Some Focus! Tips for Designing Useful and Usable Focus Indicators

<code>:focus { outline: none; }</code>

Never just remove the focus outline (visible focus indicator) without replacing it with a focus outline that will pass WCAG 2.1 SC 1.4.11 Non-Text Contrast.

- Quick Tip: Never remove CSS outlines

Examples

HTML

```
<div><input class="red-input" value="I'll be red when focused." /></div>
<div><input class="blue-input" value="I'll be blue when focused." /></div>
```

CSS

```
.red-input:focus {
  background: yellow;
  color: red;
}

.blue-input:focus {
  background: yellow;
  color: blue;
}
```

Result

See also

- `:focus-visible`
- `:focus-within`

:focus-visible CSS pseudo-class

The **`:focus-visible`** pseudo-class applies while an element matches the `:focus` pseudo-class and the UA (User Agent) determines via heuristics that the focus should be made evident on the element. (Many browsers show a "focus ring" by default in this case.)

This selector is useful to provide a different focus indicator based on the user's input modality (mouse vs. keyboard).

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:focus-visible {
  outline: 2px solid crimson;
  border-radius: 3px;
}

select:focus-visible {
  border: 2px dashed crimson;
  border-radius: 3px;
  outline: none;
}
```

```
<p>Which flavor would you like to order?</p>
<label>Full Name: <input name="firstName" type="text" /></label>
<label
  >Flavor:
  <select name="flavor">
    <option>Cherry</option>
    <option>Green Tea</option>
    <option>Moose Tracks</option>
    <option>Mint Chip</option>
  </select>
</label>
```

Syntax

```
:focus-visible {
  /* ... */
}
```

:focus vs :focus-visible

Originally, user-agent CSS set focus styles based only on the `:focus` pseudo-class, styling most focused elements with a focus ring outline. This meant all elements, including all links and buttons, had a focus ring applied when focused, which many found ugly. Because of the appearance, some authors removed the user-agent outline focus styles. Changing focus style can decrease usability, while removing focus styles makes keyboard navigation inaccessible for sighted users.

Browsers no longer visibly indicate focus (such as by drawing a "focus ring"), around each element when it has focus. Instead, they use a variety of heuristics to provide focus indicators only when it would be most helpful to the user. For instance, when a button is clicked using a pointing device, the focus is generally not visually indicated, but when a text box needing user input has focus, focus is indicated. While focus styles are always required when users are navigating the page with the keyboard or when focus is managed via scripts, focus styles are not required when the user knows where they are putting focus, such as when they use a pointing device such as a mouse or finger to physically set focus on an element, unless that element continues to need user attention.

The `:focus` pseudo-class always matches the currently-focused element. The `:focus-visible` pseudo-class also matches the focused element, but only if the user needs to be informed where the focus currently is. Because the `:focus-visible` pseudo-class matches the focused element when needed, using the `:focus-visible` (instead of the `:focus` pseudo-class) allows authors to change the appearance of the focus indicator without changing when the focus indicator appears.

When the `:focus` pseudo-class is used, it always targets the currently focused element. This means that when a user employs a pointing device, a visible focus ring appears around the focused element, which some consider obtrusive. The `:focus-visible` pseudo-class respects user agents' selective focus indication behavior while still allowing focus indicator customization.

Accessibility

Low vision

Make sure the visual focus indicator can be seen by people with low vision. This will also benefit anyone use a screen in a brightly lit space (like outside in the sun). WCAG 2.1 SC 1.4.11 Non-Text Contrast requires that the visual focus indicator be at least 3 to 1.

- Accessible Visual Focus Indicators: Give Your Site Some Focus! Tips for Designing Useful and Usable Focus Indicators

Cognition

It may not be obvious as to why the focus indicator is appearing and disappearing if a person is using mixed forms of input. For users with cognitive concerns, or who are less technologically literate, this lack of consistent behavior for interactive elements may be confusing.

Examples

Comparing :focus and :focus-visible

This example presents three pairs of controls. Each pair consists of a `text` input and a button.

- The first pair does not add any custom styles for focus states, and shows the default case.
- The second pair adds styles using the `:focus` pseudo-class.
- The third pair adds styles using the `:focus-visible` pseudo-class.

```
<input type="text" value="Default styles" /><br />
<button>Default styles</button><br />

<input class="focus-only" type="text" value=":focus" /><br />
<button class="focus-only">:focus</button><br />

<input class="focus-visible-only" type="text" value=":focus-visible" /><br />
<button class="focus-visible-only">:focus-visible</button>
```

```
input,
button {
  margin: 10px;
}

.focus-only:focus {
  outline: 2px solid black;
}

.focus-visible-only:focus-visible {
  outline: 4px dashed darkorange;
}
```

If you click each element in turn, you will see that when `:focus` is used to style the focus ring, the UA draws the focus ring when the user clicks the button. However, when `:focus-visible` is used to style the focus ring, the UA does not draw the focus ring when the user clicks the button, just like in the default case.

If you then tab through each element, you will see that in all three cases — default, `:focus`, and `:focus-visible` — the UA draws the focus ring around the button when the user navigates to it with the keyboard.

This shows how `:focus-visible` enables a designer to follow the browser's logic in determining when a focus ring should be shown.

See also

- `:focus`
- `:focus-within`

:focus-within CSS pseudo-class

The **`:focus-within`** CSS pseudo-class matches an element if the element or any of its descendants are focused. In other words, it represents an element that is itself matched by the `:focus` pseudo-class or has a descendant that is matched by `:focus`. (This includes descendants in shadow trees.)

This selector is useful, to take a common example, for highlighting an entire `<form>` container when the user focuses on one of its `<input>` fields.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

label:focus-within {
  font-weight: bold;
}
```

```
<p>Which flavor would you like to order?</p>
<label>Full Name: <input name="firstName" type="text" /></label>
<label
  >Flavor:
  <select name="flavor">
    <option>Cherry</option>
    <option>Green Tea</option>
    <option>Moose Tracks</option>
    <option>Mint Chip</option>
  </select>
</label>
```

Syntax

```
:focus-within {
  /* ... */
```

```
}
```

Examples

In this example, the container will receive special coloring styles when either text input receives focus.

HTML

```
<ul class="group">
  <li>
    <label for="given-name">Given Name:</label>
    <input id="given-name" type="text" />
  </li>
  <li>
    <label for="family-name">Family Name:</label>
    <input id="family-name" type="text" />
  </li>
</ul>
```

CSS

```
.group {
  list-style: none;
  border: 1px solid;
  color: gray;
  padding: 4px;
}

.group:focus-within {
  background: #ffff88;
  color: black;
}

input {
  margin: 4px;
}
```

Result

Try typing in the text fields.

See also

- `:focus`
- `:focus-visible`
- Grab your user's attention with the focus-within selector

:fullscreen CSS pseudo-class

The **`:fullscreen`** CSS pseudo-class matches every element that is currently in fullscreen mode. If multiple elements have been put into fullscreen mode, this selects them all.

Syntax

```
:fullscreen {
  /* ... */
}
```

Usage notes

The `:fullscreen` pseudo-class lets you configure your stylesheets to automatically adjust the size, style, or layout of content when elements switch back and forth between fullscreen and traditional presentations.

Examples

Styling a fullscreen element

This example applies a different background color to a `<div>` element, depending on whether or not it is in fullscreen mode. It includes a `<button>` to toggle fullscreen on and off.

```
<div class="element">
  <h1><code>:fullscreen</code> pseudo-class demo</h1>

  <p>
    This demo uses the <code>:fullscreen</code> pseudo-class to automatically
    change the background color of the <code>.element</code> div.
  </p>

  <p>
    Normally, the background is light yellow. In fullscreen mode, the background
    is light pink.
  </p>

  <button class="toggle">Toggle Fullscreen</button>
</div>
```

The `:fullscreen` pseudo-class is used to override the `background-color` of the `<div>` when it is in fullscreen mode.

```
.element {
  background-color: lightyellow;
}

.element:fullscreen {
  background-color: lightpink;
}
```

The following JavaScript provides an event handler function that toggles fullscreen when the `<button>` is clicked.

```
document.querySelector(".toggle").addEventListener("click", (event) => {
  if (document.fullscreenElement) {
    // If there is a fullscreen element, exit full screen.
    document.exitFullscreen();
```

```
    return;
  }
  // Make the .element div fullscreen.
  document.querySelector(".element").requestFullscreen();
});
```

```
.element {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  font-family: sans-serif;
  padding: 1.2em;
}
```

See also

- `::backdrop`
- `:xr-overlay`
- CSS selectors module
- Guide to the Fullscreen API
- Fullscreen API
- DOM API: `Element.requestFullscreen()`, `Document.exitFullscreen()`, `Document.fullscreenElement`
- `allowfullscreen` attribute

:future CSS pseudo-class

The **`:future`** CSS pseudo-class selector is a time-dimensional pseudo-class that will match for any element which appears entirely after an element that matches `:current`. For example in a video with captions which are being displayed by WebVTT.

```
:future(p, span) {
  display: none;
}
```

Syntax

```
:future {
  /* ... */
}
```

Examples

CSS

```
:future(p, span) {
  display: none;
}
```

HTML

```
<video controls preload="metadata">
  <source src="video.mp4" type="video/mp4" />
  <source src="video.webm" type="video/webm" />
  <track
    label="English"
    kind="subtitles"
    srclang="en"
    src="subtitles.vtt"
    default />
</video>
```

WebVTT

```
WEBVTT FILE

1
00:00:03.500 --> 00:00:05.000
This is the first caption

2
00:00:06.000 --> 00:00:09.000
This is the second caption

3
00:00:11.000 --> 00:00:19.000
This is the third caption
```

See also

- Web Video Text Tracks Format (WebVTT)
- `:current`
- `:past`

:has() CSS pseudo-class

The functional **`:has()`** CSS pseudo-class represents an element if any of the relative selectors that are passed as an argument match at least one element when anchored against this element. This pseudo-class presents a way of selecting a parent element or a previous sibling element with respect to a reference element by taking a relative selector list as an argument.

```
/* Selects an h1 heading with a
paragraph element that immediately follows
the h1 and applies the style to h1 */
h1:has(+ p) {
  margin-bottom: 0;
}
```

The `:has()` pseudo-class takes on the specificity of the most specific selector in its arguments the same way as `:is()` and `:not()` do.

Syntax

```
:has(<relative-selector-list>) {
  /* ... */
}
```

If the `:has()` pseudo-class itself is not supported in a browser, the entire selector block will fail unless `:has()` is in a forgiving selector list, such as in `:is()` and `:where()`.

The `:has()` pseudo-class cannot be nested within another `:has()`.

Pseudo-elements are also not valid selectors within `:has()` and pseudo-elements are not valid anchors for `:has()`. This is because many pseudo-elements exist conditionally based on the styling of their ancestors and allowing these to be queried by `:has()` can introduce cyclic querying.

Examples

Selecting a parent element

You may be looking for a "parent combinator", which allows you to go up the DOM tree and select the parent of a specific element. The `:has()` pseudo-class does that by using `parent:has(child)` (for any parent) or `parent:has(> child)` (for direct parent). This example shows how to style a `<section>` element when it contains a child with the `featured` class.

```
<section>
  <article class="featured">Featured content</article>
  <article>Regular content</article>
</section>
<section>
  <article>Regular content</article>
</section>
```

```
section:has(.featured) {
  border: 2px solid blue;
}
```

Result

With the sibling combinator

The `:has()` style declaration in the following example adjusts the spacing after `<h1>` headings if they are immediately followed by an `<h2>` heading.

HTML

```
<section>
  <article>
    <h1>Morning Times</h1>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </p>
  </article>
  <article>
    <h1>Morning Times</h1>
    <h2>Delivering you news every morning</h2>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </p>
  </article>
</section>
```

CSS

```
section {
  display: flex;
  align-items: start;
  justify-content: space-around;
}

article {
  display: inline-block;
  width: 40%;
}

h1,
h2 {
  font-size: 1.2em;
}

h2 {
  font-size: 1em;
  color: rgb(150 149 149);
}
```

```
h1,
h2 {
  margin: 0 0 1rem 0;
}

h1:has(+ h2) {
  margin: 0 0 0.25rem 0;
}
```

Result

This example shows two similar texts side-by-side for comparison – the left one with an `H1` heading followed by a paragraph and the right one with an `H1` heading followed by an `H2` heading and then a paragraph. In the example on the right, `:has()` helps to select the `H1` element that is immediately followed by an `H2` element (indicated by the next-sibling combinator `+`) and the CSS rule reduces the spacing after such an `H1` element. Without the `:has()` pseudo-class, you cannot use CSS selectors to select a preceding sibling of a different type or a parent element.

This usage of `:has()` is analogous to a lookahead assertion in regular expressions because they both allow you to select elements (or strings in regular expressions) based on what *follows* it, which is only possible if the processor is able to "rewind". In the regular expression `abc(?=xyz)`, the string `abc` is matched only if it is immediately followed by the string `xyz`. As it is a lookahead operation, the `xyz` is not included in the match. The analogous construct in CSS would be `.abc:has(+ .xyz)`: it selects the element `.abc` only if there is a next sibling `.xyz`. The part `:has(+ .xyz)` looks ahead for the next element `.xyz`, but then selects the

previous element `.abc` instead. Similarly, you can implement negative lookahead like `abc(?!xyz)` by negating the `:has()`: `.abc:not(:has(+ .xyz))` only matches `.abc` elements that are not followed by `.xyz` (either because the next sibling does not match `.xyz` or because it's the last child node).

With the :is() pseudo-class

This example builds on the previous example to show how to select multiple elements with `:has()`.

HTML

```
<section>
  <article>
    <h1>Morning Times</h1>
    <h2>Delivering you news every morning</h2>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </p>
  </article>
  <article>
    <h1>Morning Times</h1>
    <h2>Delivering you news every morning</h2>
    <h3>8:00 am</h3>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </p>
  </article>
</section>
```

CSS

```
section {
  display: flex;
  align-items: start;
  justify-content: space-around;
}

article {
  display: inline-block;
  width: 40%;
}

h1 {
  font-size: 1.2em;
}

h2 {
  font-size: 1em;
  color: rgb(150 149 149);
}

h3 {
  font-size: 0.9em;
  color: darkgrey;
}
```

```
h1,
h2,
h3 {
  margin: 0 0 1rem 0;
}

:is(h1, h2, h3):has(+ :is(h2, h3, h4)) {
  margin: 0 0 0.25rem 0;
}
```

Result

Here, the first `:is()` pseudo-class is used to select any of the heading elements in the list. The second `:is()` pseudo-class is used to pass a list of next-sibling selectors as an argument to `:has()`. The `:has()` pseudo-class helps to select any `H1`, `H2`, or `H3` element that is immediately followed by (indicated by `+`) an `H2`, `H3`, or `H4` element and the CSS rule reduces the spacing after such `H1`, `H2`, or `H3` elements.

This selector could have also been written as:

```
:is(h1, h2, h3):has(+ h2, + h3, + h4) {
  margin: 0 0 0.25rem 0;
}
```

Logical operations

The `:has()` relational selector can be used to check if one of the multiple features is true or if all the features are true.

By using comma-separated values inside the `:has()` relational selector, you are checking to see if any of the parameters exist. `x:has(a, b)` will style `x` if descendant `a` OR `b` exists.

By chaining together multiple `:has()` relational selectors together, you are checking to see if all of the parameters exist. `x:has(a):has(b)` will style `x` if descendant `a` AND `b` exist.

```
body:has(video, audio) {
  /* styles to apply if the content contains audio OR video */
}
body:has(video):has(audio) {
  /* styles to apply if the content contains both audio AND video */
}
```

Performance considerations

Certain uses of the `:has()` pseudo-class can significantly impact page performance, particularly during dynamic updates (DOM mutations). Browser engines must re-evaluate `:has()` selectors when the DOM changes, and complex or poorly constrained selectors can lead to expensive computations.

Avoid broad anchoring

The anchor selector (the `A` in `A:has(B)`) should not be an element that has too many children, like `body`, `:root`, or `*`. Anchoring `:has()` to very general selectors can degrade performance because any DOM change within the entire subtree of a broadly selected element requires the browser to re-check the `:has()` condition.

```
/* Avoid anchoring :has() to broad elements */
body:has(.sidebar) {
  /* styles */
}
:root:has(.content) {
  /* styles */
}
*:has(.item) {
  /* styles */
}
```

Instead, anchor `:has()` to specific elements like `.container` or `.gallery` to reduce the scope and improve performance.

```
/* Use specific containers to limit scope */
.container:has(.sidebar-expanded) {
  /* styles */
}
.content-wrapper:has(> article[data-priority="high"]) {
  /* styles */
}
.gallery:has(> img[data-loaded="false"]) {
  /* styles */
}
```

Minimize subtree traversals

The inner selector (the `B` in `A:has(B)`) should use combinators like `>` or `+` to limit traversal. When the selector inside `:has()` is not tightly constrained, the browser might need to traverse the entire subtree of the anchor element on every DOM mutation to check if the condition still holds.

In this example, any change within `.ancestor` requires checking all descendants for `.foo`:

```
/* May trigger full subtree traversal */
.ancestor:has(.foo) {
  /* styles */
}
```

Using child or sibling combinators limits the scope of the inner selector, reducing the performance cost of DOM mutations. In this example, the browser only needs to check direct children or a specific sibling's descendants:

```
/* More constrained - limits traversal */
.ancestor:has(> .foo) {
  /* direct child */
}
.ancestor:has(+ .sibling .foo) {
  /* descendant of adjacent sibling */
}
```

Certain inner selectors can force the browser to traverse up the ancestor chain for every DOM mutation, looking for potential anchors that might need updating. This happens when the structure implies a need to check ancestors of the mutated element.

In this example, any DOM change requires checking if the changed element is any element (`*`) that is a direct child of `.foo`, and if its parent (or further ancestors) is `.ancestor`.

```
/* Might trigger ancestor traversal */
.ancestor:has(.foo > *) {
  /* styles */
}
```

Constraining the inner selector with specific classes or direct child combinators (e.g., `.specific-child` in the next snippet) reduces expensive ancestor traversals by limiting the browser's check to a well-defined element, improving performance.

```
/* Constrain the inner selector to avoid ancestor traversals */
.ancestor:has(.foo > .specific-child) {
  /* styles */
}
```

> **Note:** These performance characteristics may improve as browsers optimize `:has()` implementations, but the fundamental constraints remain: `:has()` needs to traverse a whole subtree, so you need to minimize the subtree's size. In a selector like `A:has(B)`, make sure your `A` does not have too many children, and make sure your `B` is tightly constrained to avoid unnecessary traversal.

See also

- `:is()`, `:where()`, `:not()`
- CSS selectors and combinators
- CSS selector structure
- Selector list
- CSS selector module
- Selection and traversal on the DOM tree

:has-slotted CSS pseudo-class

The **`:has-slotted`** CSS pseudo-class matches when the content of a `<slot>` element is not empty or not using the default value (see Using templates and slots for more information).

> **Note:** Even a single whitespace text node is sufficient to make `:has-slotted` apply.

This only works when used inside CSS placed within a shadow DOM.

```
/* Selects the content of a <slot> element that has content that is not default  */
:has-slotted {
```

```
  color: green;
}

/* Selects the content of a <slot> element that has no content or default  */
:not(:has-slotted) {
  color: red;
}
```

Syntax

```
:has-slotted {
  /* ... */
}
```

Examples

This example has two `<slot>` elements, one of which has been assigned some content and the other has not.

HTML

```
<p>
  <template shadowrootmode="open">
    <style>
      :has-slotted {
        color: rebeccapurple;
      }
    </style>
    <slot name="one">Placeholder 1</slot>
    <slot name="two">Placeholder 2</slot>
  </template>
  <span slot="one">Slotted content</span>
</p>
```

Result

The `<slot>` element that has been assigned content matched the `:has-slotted` pseudo-class and has the `color` value of `rebeccapurple` applied.

See also

- HTML `<template>` element
- HTML `<slot>` element
- `::slotted`

:heading CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:heading`** CSS pseudo-class matches all heading elements in a document.

Syntax

```
:heading {
  /* ... */
}
```

Description

The `:heading` pseudo-class allows you to style all headings at once, rather than matching and styling them individually.

This pseudo-class matches only elements that by default are semantically recognized as headings (`<h1>` through `<h6>`). Elements with `role="heading"` are not matched; you can select those by using the `[role="heading"]` attribute selector.

The `:heading` pseudo-class has the same specificity as a class selector, that is, `0-1-0`. So `:heading` has a specificity of `0-1-0`, whereas `h1, h2, h3, h4, h5, h6` would have a specificity of `0-0-1` and `section:heading` would have a specificity of `0-1-1`.

Examples

Styling all headings

In this example, we use the `:heading` pseudo-class to style multiple levels of headings.

HTML

The document contains headings at three different levels, along with `<p>` paragraph elements.

```
<h1>Mastering CSS</h1>
<h2>Chapter 1: Selectors</h2>
<p>
  A CSS selector is the part of a CSS rule that describes what elements in a
  document the rule will match.
</p>
<h3>1.1 Pseudo-classes</h3>
<p>
  CSS pseudo-classes enable selecting elements based on information that lies
  outside of the document tree.
</p>
```

CSS

We set the heading elements to be italic and tomato red.

```
:heading {
  color: tomato;
  font-style: italic;
}
```

Results

The `:heading` pseudo-class applies the `color` and `font-style` to all the headings in the document, but not the paragraphs:

See also

- `:heading()`

:heading() CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:heading()`** CSS pseudo-class function represents all heading elements whose levels match a comma-separated list of integers. This allows you to style elements at specific heading levels at once, rather than matching and styling them individually.

> **Note:** The `:heading()` functional pseudo-class has the same specificity as a class selector, that is, `0-1-0`. For example, `section:heading()` has a specificity of `0-1-1`.

Syntax

```
:heading( <integer># ) {
  /* ... */
}
```

Parameters

The `:heading()` pseudo-class function takes a comma-separated list of `<integer>`s that represent the heading levels to be styled.

Usage notes

The `:heading()` functional pseudo-class matches only elements that are semantically recognized as headings. It does not match elements that use `role="heading"` or 'aria-level' attributes.

The heading level used by `:heading()` may be different from an element's type selector in cases where the browser computes a different exposed heading level. For example, `h1:heading(3)` will match any `<h1>` element that is exposed as a level 3 heading.

Examples

Selecting specific heading levels

In this example, a comma-separated list of values is used to target headings at odd-numbered levels (`<h1>` and `<h3>`) and even-numbered levels (`<h2>` and `<h4>`).

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
```

```
:heading(1, 3) {
  color: tomato;
}
:heading(2, 4) {
  color: slateblue;
}
```

See also

- `:heading`

:host CSS pseudo-class

The **`:host`** CSS pseudo-class selects the shadow host of the shadow DOM containing the CSS it is used inside — in other words, this allows you to select a custom element from inside its shadow DOM.

> **Note:** This has no effect when used outside a shadow DOM.

Try it

```
/* This CSS is being applied inside the shadow DOM. */

:host {
  background-color: aqua;
}
```

```
<h1 id="shadow-dom-host"></h1>
```

```
const shadowDom = init();

// add a <span> element in the shadow DOM
const span = document.createElement("span");
span.textContent = "Inside shadow DOM";
shadowDom.appendChild(span);

// attach shadow DOM to the #shadow-dom-host element
function init() {
  const host = document.getElementById("shadow-dom-host");
  const shadowDom = host.attachShadow({ mode: "open" });

  const cssTab = document.querySelector("#css-output");
  const shadowStyle = document.createElement("style");
  shadowStyle.textContent = cssTab.textContent;
  shadowDom.appendChild(shadowStyle);

  cssTab.addEventListener("change", () => {
    shadowStyle.textContent = cssTab.textContent;
  });
```

```
  return shadowDom;
}
```

```
/* Selects a shadow root host */
:host {
  font-weight: bold;
}
```

Syntax

```
:host {
  /* ... */
}
```

Examples

Styling the shadow host

The following snippets are taken from our host-selectors example (see it live also).

In this example we have a basic custom element — `<context-span>` — that you can wrap around text:

```
<h1>
  Host selectors <a href="#"><context-span>example</context-span></a>
</h1>
```

Inside the element's constructor, we create `style` and `span` elements, fill the `span` with the content of the custom element, and fill the `style` element with some CSS rules:

```
const style = document.createElement("style");
const span = document.createElement("span");
span.textContent = this.textContent;

const shadowRoot = this.attachShadow({ mode: "open" });
shadowRoot.appendChild(style);
shadowRoot.appendChild(span);

style.textContent =
  "span:hover { text-decoration: underline; }" +
  ":host-context(h1) { font-style: italic; }" +
  ':host-context(h1)::after { content: " - no links in headers!" }' +
  ":host-context(article, aside) { color: gray; }" +
  ":host(.footer) { color : red; }" +
  ":host { background: rgb(0 0 0 / 10%); padding: 2px 5px; }";
```

The `:host { background: rgb(0 0 0 / 10%); padding: 2px 5px; }` rule styles all instances of the `<context-span>` element (the shadow host in this instance) in the document.

See also

- Web components
- `:host()`
- `:host-context()`
- `::slotted`
- `:state()`
- CSS scoping module

:host() CSS pseudo-class

The **`:host()`** CSS pseudo-class function selects the shadow host of the shadow DOM containing the CSS it is used inside (so you can select a custom element from inside its shadow DOM) — but only if the selector given as the function's parameter matches the shadow host. **`:host()`** has no effect when used outside a shadow DOM.

The most obvious use of this is to put a class name only on certain custom element instances, and then include the relevant class selector as the function argument. You can't use this with a descendant selector expression to select only instances of the custom element that are inside a particular ancestor. That's the job of `:host-context()`.

> **Note:** While other functional pseudo-classes such as `:is()` and `:not()` accept a list of selectors as their parameters, `:host()` takes a single compound selector as its parameter. In addition, while `:is()` and `:not()` only take into account the specificity of their argument, the specificity of `:host()` is both the specificity of the pseudo-class **and** the specificity of its argument.

Try it

```
/* Following CSS is being applied inside the shadow DOM. */

:host(h1) {
  color: red;
}

:host(#shadow-dom-host) {
  border: 2px dashed blue;
}
```

```
<!-- elements outside shadow dom -->
<div id="container">
  <h1 id="shadow-dom-host"></h1>
</div>
```

```
const shadowDom = init();

// add a <span> element in the shadow DOM
const span = document.createElement("span");
span.textContent = "Inside shadow DOM";
shadowDom.appendChild(span);

// attach shadow DOM to the #shadow-dom-host element
function init() {
  const host = document.getElementById("shadow-dom-host");
  const shadowDom = host.attachShadow({ mode: "open" });
```

```
  const cssTab = document.querySelector("#css-output");
  const shadowStyle = document.createElement("style");
  shadowStyle.textContent = cssTab.textContent;
  shadowDom.appendChild(shadowStyle);

  cssTab.addEventListener("change", () => {
    shadowStyle.textContent = cssTab.textContent;
  });
  return shadowDom;
}
```

```
/* Selects a shadow root host, only if it is
   matched by the selector argument */
:host(.special-custom-element) {
  font-weight: bold;
}
```

Syntax

```
:host(<compound-selector>) {
  /* ... */
}
```

Examples

Selectively styling shadow hosts

The following snippets are taken from our host-selectors example (see it live also).

In this example we have a custom element — `<context-span>` — that you can wrap around text:

```
<h1>
  Host selectors <a href="#"><context-span>example</context-span></a>
</h1>
```

Inside the element's constructor, we create `style` and `span` elements, fill the `span` with the content of the custom element, and fill the `style` element with some CSS rules:

```
const style = document.createElement("style");
const span = document.createElement("span");
span.textContent = this.textContent;

const shadowRoot = this.attachShadow({ mode: "open" });
shadowRoot.appendChild(style);
shadowRoot.appendChild(span);

style.textContent =
  "span:hover { text-decoration: underline; }" +
  ":host-context(h1) { font-style: italic; }" +
  ':host-context(h1)::after { content: " - no links in headers!" }' +
  ":host-context(article, aside) { color: gray; }" +
  ":host(.footer) { color : red; }" +
  ":host { background: rgb(0 0 0 / 10%); padding: 2px 5px; }";
```

The `:host(.footer) { color : red; }` rule styles all instances of the `<context-span>` element (the shadow host in this instance) in the document that have the `footer` class set on them — we've used it to give instances of the element inside the `<footer>` a special color.

See also

- Web components
- `:host`
- `:host-context()`
- `:state()`

:host-context() CSS pseudo-class

The **`:host-context()`** CSS pseudo-class allows you to style elements within a shadow DOM differently based on the selector of the shadow host (the element that has the shadow root) and its DOM ancestors.

Normally, elements within a shadow DOM are isolated from the DOM outside of it. The `:host-context()` allows you to "peek outside" of this Shadow DOM and check if any of the element's ancestor elements match a certain CSS selector. For example, applying a different text color to elements within a shadow root when a `.dark-theme` class is applied to `<body>`.

Think of it like this: Imagine you have a `<greenhouse>` custom element, that has a `<chameleon>` living inside. Here, the `<greenhouse>` is the Shadow DOM host and the `<chameleon>` element is within the Shadow DOM. The `:host-context()` lets the `<chameleon>` change its appearance based on the `<greenhouse>`'s environment. If the `<greenhouse>` is in a sunny location (has a "sunny-theme" class), the `<chameleon>` turns yellow. If the `<greenhouse>` is in a shady spot (a "shady-theme" class applied instead), the `<chameleon>` turns blue.

This selector pierces through all shadow boundaries. It will look for the sunny or shady theme applied directly to the `<greenhouse>` or on any of the host's ancestors and ancestor DOMs all the way up until it reaches the document root.

To limit the selector to only the `<greenhouse>` host directly or limit the selection to host's DOM, use the `:host` or `:host()` pseudo-class instead.

> **Note:** This has no effect when used outside a shadow DOM.

The specificity of `:host-context()` is that of a pseudo-class, plus the specificity of the selector passed as the function's argument.

Try it

```
/* Following CSS is being applied inside the shadow DOM. */

:host-context(.container) {
  border: 5px dashed green;
}

:host-context(h1) {
  color: red;
}
```

```
<!-- elements outside shadow dom -->
<div class="container">
  <h1 id="shadow-dom-host"></h1>
</div>
```

```
const shadowDom = init();

// add a <span> element in the shadow DOM
const span = document.createElement("span");
span.textContent = "Inside shadow DOM";
shadowDom.appendChild(span);

// attach shadow DOM to the #shadow-dom-host element
function init() {
  const host = document.getElementById("shadow-dom-host");
  const shadowDom = host.attachShadow({ mode: "open" });

  const cssTab = document.querySelector("#css-output");
  const shadowStyle = document.createElement("style");
  shadowStyle.textContent = cssTab.textContent;
  shadowDom.appendChild(shadowStyle);

  cssTab.addEventListener("change", () => {
    shadowStyle.textContent = cssTab.textContent;
  });
  return shadowDom;
}
```

```
/* Selects a shadow root host, only if it is
   a descendant of the selector argument given */
:host-context(h1) {
  font-weight: bold;
}

/* Changes paragraph text color from black to white when
   a .dark-theme class is applied to the document body */
p {
  color: black;
}

:host-context(body.dark-theme) p {
  color: white;
}
```

Syntax

```
:host-context(<compound-selector>) {
  /* ... */
}
```

Examples

Selectively styling shadow hosts

The following snippets are taken from our host-selectors example (see it live also).

In this example we have a basic custom element — `<context-span>` — that you can wrap around text:

```
<h1>
  Host selectors <a href="#"><context-span>example</context-span></a>
</h1>
```

Inside the element's constructor, we create `style` and `span` elements, fill the `span` with the content of the custom element, and fill the `style` element with some CSS rules:

```
const style = document.createElement("style");
const span = document.createElement("span");
span.textContent = this.textContent;

const shadowRoot = this.attachShadow({ mode: "open" });
shadowRoot.appendChild(style);
shadowRoot.appendChild(span);

style.textContent =
  "span:hover { text-decoration: underline; }" +
  ":host-context(h1) { font-style: italic; }" +
  ':host-context(h1)::after { content: " - no links in headers!" }' +
  ":host(.footer) { color : red; }" +
  ":host { background: rgb(0 0 0 / 10%); padding: 2px 5px; }";
```

The `:host-context(h1) { font-style: italic; }` and `:host-context(h1)::after { content: " - no links in headers!" }` rules style the instance of the `<context-span>` element (the shadow host in this instance) inside the `<h1>`. We've used it to make it clear that the custom element shouldn't appear inside the `<h1>` in our design.

See also

- Web components
- CSS `:host` pseudo-class
- CSS `:host()` pseudo-class
- CSS `:state()` pseudo-class
- CSS `::slotted` pseudo-element
- HTML `<template>` element
- CSS scoping module

:hover CSS pseudo-class

The **`:hover`** CSS pseudo-class matches an element when a user interacts with it using a pointing device. The pseudo-class is generally triggered when the user moves the cursor (mouse pointer) over an element without pressing the mouse button.

Try it

```
.joinBtn {
  width: 10em;
  height: 5ex;
  background-color: gold;
  border: 2px solid firebrick;
  border-radius: 10px;
  font-weight: bold;
  color: black;
  cursor: pointer;
}

.joinBtn:hover {
  background-color: bisque;
}
```

```
<p>Would you like to join our quest?</p>
<button class="joinBtn">Confirm</button>
```

Styles defined by the `:hover` pseudo-class will be overridden by any subsequent link-related pseudo-class (`:link`, `:visited`, or `:active`) that has at least equal specificity. To style links appropriately, put the `:hover` rule after the `:link` and `:visited` rules but before the `:active` one, as defined by the *LVHA-order*: `:link` — `:visited` — `:hover` — `:active`.

> **Note:** The `:hover` pseudo-class is problematic on touchscreens. Depending on the browser, the `:hover` pseudo-class might never match, match only for a moment after touching an element, or continue to match even after the user has stopped touching and until the user touches another element. Web developers should make sure that content is accessible on devices with limited or non-existent hovering capabilities.

Syntax

```
:hover {
  /* ... */
}
```

Examples

Basic example

HTML

```
<a href="#">Try hovering over this link.</a>
```

CSS

```
a {
  background-color: powderblue;
  transition: background-color 0.5s;
}

a:hover {
  background-color: gold;
}
```

Result

See also

- Chromium bug #370155: Don't make `:hover` sticky on tap on sites that set a mobile viewport
- Chromium bug #306581: Immediately show hover and active states on touch when page isn't scrollable.

:in-range CSS pseudo-class

The **`:in-range`** CSS pseudo-class represents an `<input>` element whose current value is within the range limits specified by the `min` and `max` attributes.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:in-range {
  background-color: palegreen;
}
```

```
<label for="amount">How many tickets? (You can buy 2-6 tickets)</label>
<input id="amount" name="amount" type="number" min="2" max="6" value="4" />

<label for="dep">Departure Date: (Whole year 2022 is acceptable)</label>
<input
  id="dep"
  name="dep"
  type="date"
  min="2022-01-01"
  max="2022-12-31"
  value="2025-05-05" />

<label for="ret">Return Date: (Whole year 2022 is acceptable)</label>
<input id="ret" name="ret" type="date" min="2022-01-01" max="2022-12-31" />
```

This pseudo-class is useful for giving the user a visual indication that a field's current value is within the permitted limits.

> **Note:** This pseudo-class only applies to elements that have (and can take) a range limitation. In the absence of such a limitation, the element can neither be "in-range" nor "out-of-range."

Syntax

```
:in-range {
  /* ... */
}
```

Examples

HTML

```
<p>
  <label for="value1">How many toes? Values between 1 and 10 are valid.</label>
  <input
    id="value1"
    name="value1"
    type="number"
    placeholder="1 to 10"
    min="1"
    max="10"
    value="12"
    required />
  <span></span>
</p>
```

CSS

```
input {
  border: 1px solid black;
}

input:in-range {
  background-color: rgb(0 255 0 / 25%);
}

input:out-of-range {
  background-color: rgb(255 0 0 / 25%);
  border: 2px solid red;
}

input:in-range + span::after {
  content: "Your value is okay.";
}

input:out-of-range + span::after {
  content: "Your value is out of range!";
}
```

Result

> **Note:** An empty `<input>` does not count as out of range, and will not be selected using the `:out-of-range` pseudo-class selector. The `:blank` pseudo-class exists to select blank inputs, although at the time of writing this is experimental and not well-supported. You could also use the `required` attribute and the `:invalid` pseudo-class to provide more general logic and styling for making inputs mandatory (`:invalid` will style blank *and* out-of-range inputs).

See also

- `:out-of-range`
- Form data validation

:indeterminate CSS pseudo-class

The **`:indeterminate`** CSS pseudo-class represents any form element whose state is indeterminate, such as checkboxes that have been set to an `indeterminate` state with JavaScript, radio buttons which are members of a group in which all radio buttons are unchecked, and `<progress>` elements with no `value` attribute.

```
/* Selects any <input> whose state is indeterminate */
input:indeterminate {
  background: lime;
}
```

Elements targeted by this selector are:

- `<input type="checkbox">` elements with the `indeterminate` property set to `true`.
- `<input type="radio">` elements with the same `name` value and none of them `checked`.
- `<progress>` elements with no `value`, placing them in an indeterminate state.

Syntax

```
:indeterminate {
  /* ... */
}
```

Examples

Checkbox & radio button

This example applies special styles to the labels associated with indeterminate form fields.

HTML

```
<fieldset>
  <legend>Checkbox</legend>
  <div>
    <input type="checkbox" id="checkbox" />
    <label for="checkbox">This checkbox label starts out lime.</label>
  </div>
</fieldset>

<fieldset>
```

```
  <legend>Radio</legend>
  <div>
    <input type="radio" id="radio1" name="radioButton" value="val1" />
    <label for="radio1">First radio label starts out lime.</label>
  </div>
  <div>
    <input type="radio" id="radio2" name="radioButton" value="val2" />
    <label for="radio2">Second radio label also starts out lime.</label>
  </div>
</fieldset>
```

CSS

```
input:indeterminate + label {
  background: lime;
}
```

```
fieldset {
  padding: 1em 0.75em;
}

fieldset:first-of-type {
  margin-bottom: 1.5rem;
}

fieldset:not(:first-of-type) > div:not(:last-child) {
  margin-bottom: 0.5rem;
}
```

JavaScript

```
const inputs = document.getElementsByTagName("input");

for (const input of inputs) {
  input.indeterminate = true;
}
```

Result

Progress bar

HTML

```
<progress></progress>
```

CSS

```
progress {
  margin: 4px;
}

progress:indeterminate {
  width: 80vw;
  height: 20px;
}
```

Result

See also

- Web forms — Working with user data
- Styling web forms
- The `<input type="checkbox">` element's `indeterminate` property
- `<input>` and the `HTMLInputElement` interface which implements it.
- The `:checked` CSS selector lets you style checkboxes based on whether they're checked or not

:interest-source CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:interest-source`** CSS pseudo-class represents an interest invoker element when interest is shown on it.

Syntax

```
:interest-source {
  /* ... */
}
```

Examples

Selecting an interest invoker element

In this example, we demonstrate how `:interest-source` can be used to apply styles to an interest invoker element when interest is shown on it.

HTML

The markup includes a `<button>` and a `<p>`. We specify the `<button>` as an interest invoker by giving it the `interestfor` attribute whose value matches the `id` of the `<p>` element, making the paragraph the target element. The paragraph is turned into a popover by giving it the `popover` attribute, which hides it initially.

```
<button interestfor="mypopover">Button</button>
<p id="mypopover" popover>A hover tooltip</p>
```

CSS

In the CSS, we specify a rule with an `:interest-source` selector to apply a specific combination of `background-color` and `color` to the `<button>` when interest is shown on it. We also apply some other styles to the `<button>`, which are hidden for brevity.

```
button {
  margin: 10px;
  padding: 5px 10px;
  border: 1px solid #dddddd;
  border-radius: 5px;
  font-size: 1.5em;
  background-color: #eeeeee;

  cursor: pointer;
}
```

```
button:interest-source {
  background-color: hotpink;
  color: purple;
}
```

Result

This renders as follows:

Try showing interest in the button (for example, by hovering or focusing it), and note how the `hotpink` and `purple` color combination is applied only when interest is shown.

See also

- `:interest-target`
- Popover API
- Using interest invokers
- CSS selectors module

:interest-target CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:interest-target`** CSS pseudo-class represents the target element associated with an interest invoker when interest is shown on that invoker.

Syntax

```
:interest-target {
  /* ... */
}
```

Examples

Selecting the target element associated with an interest invoker

In this example, we demonstrate how `:interest-target` can be used to apply styles to an interest invoker element's target element when interest is shown on the invoker.

HTML

The markup includes a `<button>` and a `<p>`. We specify the `<button>` as an interest invoker by giving it the `interestfor` attribute whose value matches the `id` of the `<p>` element, making the paragraph the target element.

```
<button interestfor="mytarget">Button</button>
<p id="mytarget">A paragraph</p>
```

CSS

In the CSS, we specify a rule with an `:interest-target` selector, which will apply a specific set of styles to the `<p>` element when interest is shown in the `<button>`. We also apply some other styles to the `<button>`, which we've hidden for brevity.

```
body {
  margin: 10px;
}

button {
  padding: 5px 10px;
  border: 1px solid #dddddd;
  border-radius: 5px;
  background-color: #eeeeee;

  cursor: pointer;
}
```

```
p:interest-target {
  font-size: 1.5em;
  font-family: sans-serif;
  padding: 10px;
  background-color: hotpink;
  color: purple;
}
```

Result

This renders as follows:

Try showing interest in the button (for example, by hovering or focusing it), and note how this results in the styles shown earlier being applied to the paragraph.

See also

- `:interest-source`

- Popover API
- Using interest invokers
- CSS selectors module

:invalid CSS pseudo-class

The **`:invalid`** CSS pseudo-class represents any `<form>`, `<fieldset>`, `<input>` or other `<form>` element whose contents fail to validate.

This pseudo-class is useful for highlighting field errors for the user.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:invalid {
  background-color: ivory;
  border: none;
  outline: 2px solid red;
  border-radius: 5px;
}
```

```
<label for="email">Email Address:</label>
<input id="email" name="email" type="email" value="na@me@example.com" />

<label for="secret">Secret Code: (lower case letters)</label>
<input id="secret" name="secret" type="text" value="test" pattern="[a-z]+" />

<label for="age">Your age: (18+)</label>
<input id="age" name="age" type="number" value="5" min="18" />

<label
  ><input name="tos" type="checkbox" required checked /> - Do you agree to
  ToS?</label
>
```

Syntax

```
:invalid {
  /* ... */
}
```

Accessibility

The color red is commonly used to indicate invalid input. People who have certain types of color blindness will be unable to determine the input's state unless it is accompanied by an additional indicator that does not rely on color to convey meaning. Typically, descriptive text and/or an icon are used.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.1 | W3C Understanding WCAG 2.0

Examples

Coloring elements to show validation

HTML

```
<form>
  <div class="field">
    <label for="url_input">Enter a URL:</label>
    <input type="url" id="url_input" />
  </div>

  <div class="field">
    <label for="email_input">Enter an email address:</label>
    <input type="email" id="email_input" required />
  </div>
</form>
```

CSS

```
label {
  display: block;
  margin: 1px;
  padding: 1px;
}

.field {
  margin: 1px;
  padding: 1px;
}

input:invalid {
  background-color: #ffdddd;
}

form:invalid {
  border: 5px solid #ffdddd;
}

input:valid {
  background-color: #ddffdd;
}

form:valid {
  border: 5px solid #ddffdd;
}

input:required {
```

```
  border-color: maroon;
  border-width: 3px;
}

input:required:invalid {
  border-color: #c00000;
}
```

Result

Showing sections in stages

In this example we use `:invalid` along with ~, the subsequent-sibling combinator, to make a form appear in stages, so the form initially shows the first item to complete, and when the user completes each item the form displays the next one. When the whole form is complete the user can submit it.

HTML

```
<form>
  <fieldset>
    <legend>Personal Information</legend>
    <label for="form-name">Name</label><br />
    <input type="text" name="name" id="form-name" required />
  </fieldset>

  <fieldset>
    <legend>Contact Information</legend>
    <label for="form-email">Email Address</label><br />
    <input type="email" name="email" id="form-email" required />
  </fieldset>

  <fieldset>
    <legend>Message</legend>
    <label for="form-message">Message</label><br />
    <textarea name="message" id="form-message" required></textarea>
  </fieldset>

  <button type="submit" name="send">Submit</button>
</form>
```

```
document.querySelector("form").addEventListener("submit", (event) => {
  event.preventDefault();
});
```

CSS

```
/* Hide the fieldset after an invalid fieldset */
fieldset:invalid ~ fieldset {
  display: none;
}

/* Dim and disable the button while the form is invalid */
form:invalid button {
  opacity: 0.3;
  pointer-events: none;
}

input,
textarea {
  box-sizing: border-box;
  width: 100%;
  font-family: monospace;
  padding: 0.25em 0.5em;
}

button {
  width: 100%;
  border: thin solid darkgrey;
  font-size: 1.25em;
  background-color: darkgrey;
  color: white;
}
```

Result

Notes

Radio buttons

If any one of the radio buttons in a group is `required`, the `:invalid` pseudo-class is applied to all of them if none of the buttons in the group is selected. (Grouped radio buttons share the same value for their `name` attribute.)

Gecko defaults

By default, Gecko does not apply a style to the `:invalid` pseudo-class. However, it does apply a style (a red "glow" using the `box-shadow` property) to the `:user-invalid` pseudo-class, which applies in a subset of cases for `:invalid`.

See also

- Other validation-related pseudo-classes: `:required`, `:optional`, `:valid`
- Related Mozilla pseudo-classes: `:user-invalid`, `:-moz-submit-invalid`
- Form data validation
- Accessing the validity state from JavaScript

:is() CSS pseudo-class

The **`:is()`** CSS pseudo-class function takes a selector list as its argument, and selects any element that can be selected by one of the selectors in that list. This is useful for writing large selectors in a more compact form.

> **Note:** Originally named `:matches()` (and `:any()`), this selector was renamed to `:is()` in CSSWG issue #3258.

Try it

```
ol {
  list-style-type: upper-alpha;
  color: darkblue;
}

:is(ol, ul, menu:unsupported) :is(ol, ul) {
  color: green;
}

:is(ol, ul) :is(ol, ul) ol {
  list-style-type: lower-greek;
  color: chocolate;
}
```

```
<ol>
  <li>Saturn</li>
  <li>
    <ul>
      <li>Mimas</li>
      <li>Enceladus</li>
      <li>
        <ol>
          <li>Voyager</li>
          <li>Cassini</li>
        </ol>
      </li>
      <li>Tethys</li>
    </ul>
  </li>
  <li>Uranus</li>
  <li>
    <ol>
      <li>Titania</li>
      <li>Oberon</li>
    </ol>
  </li>
</ol>
```

Syntax

```
:is(<forgiving-selector-list>) {
  /* ... */
}
```

Parameters

The `:is()` pseudo-class requires a selector list, a comma-separated list of one or more selectors as its argument. The list must not contain a pseudo-element, but any other simple, compound, and complex selectors are allowed.

Difference between :is() and :where()

The difference between the two is that `:is()` counts towards the specificity of the overall selector (it takes the specificity of its most specific argument), whereas `:where()` has a specificity value of 0. This is demonstrated by the example on the `:where()` reference page.

Forgiving Selector Parsing

The specification defines `:is()` and `:where()` as accepting a forgiving selector list.

In CSS when using a selector list, if any of the selectors are invalid then the whole list is deemed invalid. When using `:is()` or `:where()` instead of the whole list of selectors being deemed invalid if one fails to parse, the incorrect or unsupported selector will be ignored and the others used.

```
:is(:valid, :unsupported) {
  /* … */
}
```

Will still parse correctly and match `:valid` even in browsers which don't support `:unsupported`, whereas:

```
:valid,
:unsupported {
  /* … */
}
```

Will be ignored in browsers which don't support `:unsupported` even if they support `:valid`.

Examples

Simplifying list selectors

The `:is()` pseudo-class can greatly simplify your CSS selectors. For example, take the following CSS:

```
/* 3-deep (or more) unordered lists use a square */
ol ol ul,
ol ul ul,
ol menu ul,
ol ol menu,
ol ul menu,
ol menu menu,
ul ol ul,
ul ul ul,
ul menu ul,
ul ol menu,
ul ul menu,
ul menu menu,
```

```
menu ol ul,
menu ul ul,
menu menu ul,
menu ol menu,
menu ul menu,
menu menu menu {
  list-style-type: square;
}
```

You can replace it with:

```
/* 3-deep (or more) unordered lists use a square */
:is(ol, ul, menu) :is(ol, ul, menu) :is(ul, menu) {
  list-style-type: square;
}
```

Simplifying section selectors

The `:is()` pseudo-class is particularly useful when dealing with HTML sections and headings. Since `<section>`, `<article>`, `<aside>`, and `<nav>` are commonly nested together, without `:is()`, styling them to match one another can be tricky.

For example, without `:is()`, styling all the h1 elements at different depths could be very complicated:

```
/* Level 0 */
h1 {
  font-size: 30px;
}

/* Level 1 */
section h1,
article h1,
aside h1,
nav h1 {
  font-size: 25px;
}

/* Level 2 */
section section h1,
section article h1,
section aside h1,
section nav h1,
article section h1,
article article h1,
article aside h1,
article nav h1,
aside section h1,
aside article h1,
aside aside h1,
aside nav h1,
nav section h1,
nav article h1,
nav aside h1,
nav nav h1 {
  font-size: 20px;
}

/* Level 3 */
/* don't even think about it! */
```

Using `:is()`, though, it's much easier:

```
/* Level 0 */
h1 {
  font-size: 30px;
}
/* Level 1 */
:is(section, article, aside, nav) h1 {
  font-size: 25px;
}
/* Level 2 */
:is(section, article, aside, nav) :is(section, article, aside, nav) h1 {
  font-size: 20px;
}
/* Level 3 */
:is(section, article, aside, nav)
  :is(section, article, aside, nav)
  :is(section, article, aside, nav)
  h1 {
  font-size: 15px;
}
```

:is() does not select pseudo-elements

The `:is()` pseudo-class does not match pseudo-elements. So rather than this:

```
some-element:is(::before, ::after) {
  display: block;
}
```

or this:

```
:is(some-element::before, some-element::after) {
  display: block;
}
```

instead do:

```
some-element::before,
some-element::after {
```

```
  display: block;
}
```

See also

- `:where()` - Like `:is()`, but with 0 specificity.
- Selector list
- Web components

:lang() CSS pseudo-class

The **`:lang()`** CSS pseudo-class matches elements based on the language they are determined to be in.

Try it

```
*:lang(en-US) {
  outline: 2px solid deeppink;
}
```

```
<p lang="en-US">
  Music during road trips and long commutes aren't a problem, but using
  headphones isn't the best thing to do while driving in your car.
</p>

<p lang="pl-PL">
  Gdy widzimy znak z narysowaną czaszką i napisem
  <strong lang="en-US">DANGER</strong> to lepiej nie wchodzić do środka.
</p>
```

> **Note:** In HTML, the language is determined by a combination of the `lang` attribute, the `<meta>` element, and possibly by information from the protocol (such as HTTP headers). For other document types there may be other document methods for determining the language.

Syntax

```
:lang(<language-code> [,<language-code> ]*) {
  /* ... */
}
```

Parameters

`<language-code>`

A comma-separated list of one or more `<string>`s that target elements based on the BCP 47 language tags set in their `lang` attributes. Matching by language range is case-insensitive.

Description

When selecting languages, there is implicit wildcard matching, so `:lang(de-DE)` will match `de-DE`, `de-DE-1996`, `de-Latn-DE`, `de-Latf-DE`, and `de-Latn-DE-1996`. Explicitly using wildcards must include a full match of a language subtag, so `:lang("*-F*")` is invalid but `:lang("*-Fr")` is valid.

Examples

Matching children of a given language

In this example, the `:lang()` pseudo-class is used to match the parents of quote elements (`<q>`) using child combinators. Note that this doesn't illustrate the only way to do this, and that the best method to use depends on the type of document. Also note that Unicode values are used to specify some of the special quote characters.

HTML

```
<div lang="en">
  <q>This English quote has a <q>nested</q> quote inside.</q>
</div>
<div lang="fr">
  <q>This French quote has a <q>nested</q> quote inside.</q>
</div>
<div lang="de">
  <q>This German quote has a <q>nested</q> quote inside.</q>
</div>
```

CSS

```
:lang(en) > q {
  quotes: "\201C" "\201D" "\2018" "\2019";
}
:lang(fr) > q {
  quotes: "« " " »";
}
:lang(de) > q {
  quotes: "»" "«" "\2039" "\203A";
}
```

Result

Matching multiple languages

The following example shows how to match multiple languages by providing a comma-separated list of language codes. It's also possible to use a wildcard to match languages in a given language range.

```
p {
  margin: 0;
}
```

CSS

```
/* Matches nl and de */
:lang("nl", "de") {
  color: green;
}
```

```
/* Omitting quotes & case-insensitive matching */
:lang(EN, FR) {
  color: blue;
}

/* Wildcard matching a language range */
:lang("*-Latn") {
  color: red;
}
```

HTML

```
<p lang="nl">Dit is een Nederlandse paragraaf.</p>
<p lang="de">Dies ist ein deutscher Satz.</p>
<p lang="en">This is an English sentence.</p>
<p lang="en-GB">Matching the language range of English.</p>
<p lang="fr">Ceci est un paragraphe français.</p>
<p lang="fr-Latn-FR">Ceci est un paragraphe français en latin.</p>
```

Result

See also

- The `:dir` pseudo-class that matches by directionality of text
- HTML `lang` attribute
- HTML `translate` attribute
- BCP 47 language tag

:last-child CSS pseudo-class

The **`:last-child`** CSS pseudo-class represents the last element among a group of sibling elements.

Try it

```
p {
  font-weight: bold;
}

li:last-child {
  border: 2px solid orange;
}
```

```
<p>Track &amp;amp; field champions:</p>
<ul>
  <li>Adhemar da Silva</li>
  <li>Wang Junxia</li>
  <li>Wilma Rudolph</li>
  <li>Babe Didrikson-Zaharias</li>
  <li>Betty Cuthbert</li>
  <li>Fanny Blankers-Koen</li>
  <li>Florence Griffith-Joyner</li>
  <li>Irena Szewinska</li>
  <li>Jackie Joyner-Kersee</li>
  <li>Shirley Strickland</li>
  <li>Carl Lewis</li>
  <li>Emil Zatopek</li>
  <li>Haile Gebrselassie</li>
  <li>Jesse Owens</li>
  <li>Jim Thorpe</li>
  <li>Paavo Nurmi</li>
  <li>Sergei Bubka</li>
  <li>Usain Bolt</li>
</ul>
```

Syntax

```
:last-child {
  /* ... */
}
```

Examples

Basic example

HTML

```
<div>
  <p>This text isn't selected.</p>
  <p>This text is selected!</p>
</div>

<div>
  <p>This text isn't selected.</p>
  <h2>This text isn't selected: it's not a `p`.</h2>
</div>
```

CSS

```
p:last-child {
  color: lime;
  background-color: black;
  padding: 5px;
}
```

Result

Styling a list

HTML

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>
    Item 3
    <ul>
      <li>Item 3.1</li>
      <li>Item 3.2</li>
      <li>Item 3.3</li>
    </ul>
  </li>
</ul>
```

CSS

```
ul li {
  color: blue;
}

ul li:last-child {
  border: 1px solid red;
  color: red;
}
```

Result

See also

- `:-moz-last-node`
- `:last-of-type`
- `:first-child`
- `:nth-child`

:last-of-type CSS pseudo-class

The **`:last-of-type`** CSS pseudo-class represents the last element of its type (tag name) among a group of sibling elements.

Try it

```
dt {
  font-weight: bold;
}

dd {
  margin: 3px;
}

dd:last-of-type {
  border: 2px solid orange;
}
```

```
<dl>
  <dt>Vegetables:</dt>
  <dd>1. Tomatoes</dd>
  <dd>2. Cucumbers</dd>
  <dd>3. Mushrooms</dd>
  <dt>Fruits:</dt>
  <dd>4. Apples</dd>
  <dd>5. Mangos</dd>
  <dd>6. Pears</dd>
  <dd>7. Oranges</dd>
</dl>
```

Syntax

```
:last-of-type {
  /* ... */
}
```

Examples

Styling the last paragraph

HTML

```
<h2>Heading</h2>
<p>Paragraph 1</p>
<p>Paragraph 2</p>
```

CSS

```
p:last-of-type {
  color: red;
  font-style: italic;
}
```

Result

Nested elements

This example shows how nested elements can also be targeted. Note that the universal selector (*) is implied when no simple selector is written.

HTML

```
<article>
  <div>This `div` is first.</div>
  <div>This <span>nested `span` is last</span>!</div>
  <div>
    This <em>nested `em` is first</em>, but this <em>nested `em` is last</em>!
  </div>
  <p>This `p` qualifies!</p>
  <div>This is the final `div`!</div>
</article>
```

CSS

```
article :last-of-type {
  background-color: pink;
}
```

Result

Multiple selectors elements

This HTML example contains nested elements of different types. The CSS contains both type selectors and class selectors.

HTML

```
<p>This `p` is not selected.</p>
<p>This `p` is not selected either.</p>
<p>
  This `p` is last `p` element of its parent e.g. `body` selected by `p` type
  selector.
</p>
<div class="container">
  <div class="item">This `div` is not selected.</div>
  <div class="item">This `div` is not selected either.</div>
  <div class="item">
    This `div` is last `div` element of its parent `div` selected by `.container
    .item` class selector.
  </div>
  <p class="item">
    This `p` is last `p` element of its parent `div` selected by `.container
    .item` class selector.
  </p>
</div>
```

CSS

```
p:last-of-type {
  background: skyblue;
}

.container .item:last-of-type {
  color: red;
  font-weight: bold;
}
```

Result

The last `<div>` and the last `<p>` are both red and bold as the `.item:last-of-type` selects the last of every type if that last element also has the `item` class.

See also

- `:last-child`, `:nth-last-of-type`

:left CSS pseudo-class

The **`:left`** CSS pseudo-class, used with the `@page` at-rule, represents all left-hand pages of a printed document.

```
/* Selects any left-hand pages when printing */
@page :left {
  margin: 2in 3in;
}
```

Whether a given page is "left" or "right" is determined by the major writing direction of the document. For example, if the first page has a major writing direction of left-to-right then it will be a `:right` page; if it has a major writing direction of right-to-left then it will be a `:left` page.

> **Note:** This pseudo-class can be used to change only the `margin`, `padding`, `border`, and `background` properties of the *page box*. All other properties will be ignored, and only the page box, not the document content on the page, will be affected.

Syntax

```
:left {
  /* ... */
}
```

Examples

Setting a margin for left-hand pages

```
@page :left {
  margin: 2in 3in;
}
```

See also

- `@page`
- Other page-related pseudo-classes: `:first`, `:right`

:link CSS pseudo-class

The **`:link`** CSS pseudo-class represents an element that has not yet been visited. It matches every unvisited `<a>` or `<area>` element that has an `href` attribute.

Try it

```
p {
  font-weight: bold;
}

a:link {
  color: forestgreen;
  text-decoration-color: hotpink;
}
```

```
<p>Pages that you might have visited:</p>
<ul>
  <li>
    <a href="https://developer.mozilla.org">MDN Web Docs</a>
  </li>
  <li>
    <a href="https://www.youtube.com/">YouTube</a>
  </li>
</ul>
<p>Pages unlikely to be in your history:</p>
<ul>
  <li>
    <a href="https://developer.mozilla.org/missing-2">Random MDN page</a>
  </li>
  <li>
    <a href="https://example.com/missing-2">Random Example page</a>
  </li>
</ul>
```

Styles defined by the `:link` and `:visited` pseudo-classes can be overridden by any subsequent user-action pseudo-classes (`:hover` or `:active`) that have at least equal specificity. To style links appropriately, put the `:link` rule before all other link-related rules, as defined by the *LVHA-order*: `:link` — `:visited` — `:hover` — `:active`. The `:visited` pseudo-class and `:link` pseudo-class are mutually exclusive.

> **Note:** Use `:any-link` to select an element independent of whether it has been visited or not.

Syntax

```
:link {
  /* ... */
}
```

Examples

By default, most browsers apply a special `color` value to visited links. Thus, the links in this example will probably have special font colors only before you visit them. (After that, you'll need to clear your browser history to see them again.) However, the `background-color` values are likely to remain, as most browsers do not set that property on visited links by default.

HTML

```
<a href="#ordinary-target">This is an ordinary link.</a><br />
<a href="">You've already visited this link.</a><br />
<a>Placeholder link (won't get styled)</a>
```

CSS

```
a:link {
  background-color: gold;
  color: green;
}
```

Result

See also

- Link-related pseudo-classes: `:visited`, `:hover`, `:active`

:local-link CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:local-link`** CSS pseudo-class represents a link to the same document. Therefore an element that is the source anchor of a hyperlink whose target's absolute URL matches the element's own document URL.

```
/* Selects any <a> that links to the current document */
a:local-link {
  color: green;
}
```

Syntax

```
:local-link {
  /* ... */
}
```

Examples

HTML

```
<a href="#target">This is a link on the current page.</a><br />
<a href="https://example.com">This is an external link</a><br />
```

CSS

```
a:local-link {
  color: green;
}
```

Result

Browser compatibility

Currently, no browsers support this feature.

See also

- Link-related pseudo-classes: `:link`, `:visited`, `:hover`, `:active`, `:any-link`
- CSS selectors module

:modal CSS pseudo-class

The **`:modal`** CSS pseudo-class matches an element that is in a state in which it excludes all interaction with elements outside it until the interaction has been dismissed. Multiple elements can be selected by the `:modal` pseudo-class at the same time, but only one of them will be active and able to receive input.

Try it

```
button {
  display: block;
  margin: auto;
  width: 10rem;
  height: 2rem;
}

:modal {
  background-color: beige;
  border: 2px solid burlywood;
  border-radius: 5px;
}

p {
  color: black;
}
```

```
<p>Would you like to see a new random number?</p>
<button id="showNumber">Show me</button>

<dialog id="favDialog">
  <form method="dialog">
    <p>Lucky number is: <strong id="number"></strong></p>
    <button>Close dialog</button>
  </form>
</dialog>
```

```
const showNumber = document.getElementById("showNumber");
const favDialog = document.getElementById("favDialog");
const number = document.getElementById("number");

showNumber.addEventListener("click", () => {
  number.innerText = Math.floor(Math.random() * 1000);
  favDialog.showModal();
});
```

Syntax

```
:modal {
  /* ... */
}
```

Usage notes

Examples of elements that will prevent user interaction with the rest of the page and will be selected by the `:modal` pseudo-class include:

- The `dialog` element opened with the `showModal()` API.
- The element selected by the `:fullscreen` pseudo-class when opened with the `requestFullscreen()` API.

Examples

Styling a modal dialog

This example styles a modal dialog that opens when the "Show the dialog" button is activated. This example is adapted from the `<dialog>` element example.

```
<!-- A modal dialog containing a form -->
<dialog id="favDialog">
  <form method="dialog">
    <p>
      <label>
        Favorite animal:
        <select>
          <option>Choose…</option>
          <option>Brine shrimp</option>
          <option>Red panda</option>
          <option>Spider monkey</option>
        </select>
      </label>
    </p>
    <div>
      <button>Cancel</button>
      <button>Confirm</button>
    </div>
  </form>
</dialog>
```

```
<p>
  <button id="showDialog">Show the dialog</button>
</p>
```

CSS

The `:modal` pseudo-class selects the dialog opened with `showModal()`, giving it a red border, a yellow background, and a box shadow.

```
:modal {
  border: 5px solid red;
  background-color: yellow;
  box-shadow: 3px 3px 10px rgb(0 0 0 / 50%);
}
```

```
const showButton = document.getElementById("showDialog");
const favDialog = document.getElementById("favDialog");

showButton.addEventListener("click", () => {
  favDialog.showModal();
});
```

Result

See also

- `dialog` element
- Other element display state pseudo-classes: `:fullscreen` and `:picture-in-picture`
- Complete list of pseudo-classes

:muted CSS pseudo-class

The **`:muted`** CSS pseudo-class selector represents an element that is capable of making sound, such as `<audio>` or `<video>`, but is muted (forced silent).

Muted is different from `:volume-locked` in that the page author has control over whether a media element can be muted or un-muted. User agents may set media `muted` value according to use preferences (e.g., remembering the last set value across sessions, on a per-site basis, or otherwise). An element that is `:volume-locked` cannot be muted, un-muted, or have its volume changed via JavaScript because of an operating system or user agent preference.

Syntax

```
:muted {
  /* ... */
}
```

Examples

HTML

The `muted` attribute is added to the `<video>` element so that the sound is muted, this works the same for `<audio>` elements.

```
<!-- 'Big Buck Bunny' licensed under CC 3.0 by the Blender foundation. Hosted by archive.org -->
<!-- Poster from peach.blender.org -->
<video
  controls
  muted
  src="https://archive.org/download/BigBuckBunny_124/Content/big_buck_bunny_720p_surround.mp4"
  poster="https://peach.blender.org/wp-content/uploads/title_anouncement.jpg?x11217"
  width="620">
  Sorry, your browser doesn't support embedded videos, but don't worry, you can
  <a href="https://archive.org/details/BigBuckBunny_124">download it</a>
  and watch it with your favorite video player!
</video>
```

CSS

If the `<video>` element is `muted` then a red outline is drawn around it, when it is not muted then it has a green outline. Try toggling the mute icon in the controls to see the state change.

```
video {
  margin: 5px;
}
```

```
video:muted {
  outline: 5px solid red;
}

video:not(:muted) {
  outline: 5px solid green;
}
```

See also

- `:buffering`
- `:paused`
- `:playing`
- `:seeking`
- `:stalled`
- `:volume-locked`
- CSS selectors
- `muted` property of `HTMLMediaElement` objects

:not() CSS pseudo-class

The **`:not()`** CSS pseudo-class represents elements that do not match a list of selectors. Since it prevents specific items from being selected, it is known as the *negation pseudo-class*.

The `:not()` pseudo-class has a number of quirks, tricks, and unexpected results that you should be aware of before using it.

Try it

```
p:not(.irrelevant) {
  font-weight: bold;
}

p > strong,
p > b.important {
  color: crimson;
}

p > :not(strong, b.important) {
  color: darkmagenta;
}
```

```
<p>
  <b>Mars</b> is one of the most Earth-like planets. <b>Mars</b> day is almost
  the same as an Earth day, only <strong>37 minutes</strong> longer.
</p>

<p class="irrelevant">
  <b class="important">NASA</b>'s Jet <del>Momentum</del> Propulsion Laboratory
  is designing mission concepts to survive the <b>Venus</b> extreme temperatures
  and atmospheric pressure.
</p>
```

Syntax

```
:not(<complex-selector-list>) {
  /* ... */
}
```

Parameters

The `:not()` pseudo-class requires a selector list, a comma-separated list of one or more selectors, as its argument. The list must not contain a pseudo-element, but any other simple, compound, and complex selectors are allowed.

Description

There are several unusual effects and outcomes when using `:not()` that you should keep in mind when using it:

- Useless selectors can be written using this pseudo-class. For example, `:not(*)` matches any element which is not an element, which is obviously nonsense, so the accompanying rule will never be applied.
- This pseudo-class can increase the specificity of a rule. For example, `#foo:not(#bar)` will match the same element as the simpler `#foo`, but has the higher specificity of two `id` selectors.
- The specificity of the `:not()` pseudo-class is replaced by the specificity of the most specific selector in its comma-separated argument of selectors; providing the same specificity as if it had been written `:not(:is(argument))`.
- `:not(.foo)` will match anything that isn't `.foo`, *including* `<html>` *and* `<body>`.
- This selector will match everything that is "not an X". This may be surprising when used with descendant combinators, since there are multiple paths to select a target element. For instance, `body :not(table) a` will still apply to links inside a `<table>`, since `<tr>`, `<tbody>`, `<th>`, `<td>`, `<caption>`, etc. can all match the `:not(table)` part of the selector. To avoid this, you can use `body a:not(table a)` instead, which will only apply to links that are not descendants of a table.
- You can negate several selectors at the same time. Example: `:not(.foo, .bar)` is equivalent to `:not(.foo):not(.bar)`.
- If any selector passed to the `:not()` pseudo-class is invalid or not supported by the browser, the whole rule will be invalidated. The effective way to overcome this behavior is to use `:is()` pseudo-class, which accepts a forgiving selector list. For example `:not(.foo, :invalid-pseudo-class)` will invalidate a whole rule, but `:not(:is(.foo, :invalid-pseudo-class))` will match any (*including* `<html>` *and* `<body>`) element that isn't `.foo`.

Examples

Using :not() with valid selectors

This example shows a few ways of using `:not()`.

HTML

```
<p>I am a paragraph.</p>
<p class="fancy">I am so very fancy!</p>
<div>I am NOT a paragraph.</div>
<h2>
  <span class="foo">foo inside h2</span>
  <span class="bar">bar inside h2</span>
</h2>
```

CSS

```
.fancy {
  text-shadow: 2px 2px 3px gold;
}

/* <p> elements that don't have a class `.fancy` */
p:not(.fancy) {
  color: green;
}

/* Elements that are not <p> elements */
body :not(p) {
  text-decoration: underline;
}

/* Elements that are not <div>s or `.fancy` */
body :not(div):not(.fancy) {
  font-weight: bold;
}

/* Elements that are not <div>s or `.fancy` */
body :not(div, .fancy) {
  text-decoration: overline underline;
}

/* Elements inside an <h2> that aren't a <span> with a class of `.foo` */
h2 :not(span.foo) {
```

```
  color: red;
}
```

Result

Using :not() with invalid selectors

This example shows the use of `:not()` with invalid selectors and how to prevent invalidation.

HTML

```
<p class="foo">I am a paragraph with .foo</p>
<p class="bar">I am a paragraph with .bar</p>
<div>I am a div without a class</div>
<div class="foo">I am a div with .foo</div>
<div class="bar">I am a div with .bar</div>
<div class="foo bar">I am a div with .foo and .bar</div>
```

CSS

```
/* Invalid rule, does nothing */
p:not(.foo, :invalid-pseudo-class) {
  color: red;
  font-style: italic;
}

/* Select all <p> elements without the `foo` class */
p:not(:is(.foo, :invalid-pseudo-class)) {
  color: green;
  border-top: dotted thin currentColor;
}

/* Select all <div> elements without the `foo` or the `bar` class */
div:not(.foo, .bar) {
  color: red;
  font-style: italic;
}

/* Select all <div> elements without the `foo` or the `bar` class */
div:not(:is(.foo, .bar)) {
  border-bottom: dotted thin currentColor;
}
```

Result

The `p:not(.foo, :invalid-pseudo-class)` rule is invalid because it contains an invalid selector. The `:is()` pseudo-class accepts a forgiving selector list, so the `:is(.foo, :invalid-pseudo-class)` rule is valid and equivalent to `:is(.foo)`. Thus, the `p:not(:is(.foo, :invalid-pseudo-class))` rule is valid and equivalent to `p:not(.foo)`.

If `:invalid-pseudo-class` was a valid selector, the first two rules above would still be equivalent (the last two rules showcase that). The use of `:is()` makes the rule more robust.

See also

- Pseudo-classes
- Learn: Pseudo-classes and pseudo-elements
- Other functional CSS pseudo-classes:
  - `:has()`
  - `:is()`
  - `:where()`
- How :not() chains multiple selectors on MDN blog (2023)

:nth-child() CSS pseudo-class

The **`:nth-child()`** CSS pseudo-class matches elements based on the indexes of the elements in the child list of their parents. In other words, the `:nth-child()` selector selects child elements according to their position among all the sibling elements within a parent element.

Try it

```
p {
  font-weight: bold;
}

li:nth-child(-n + 3) {
  border: 2px solid orange;
  margin-bottom: 1px;
}

li:nth-child(even) {
  background-color: lightyellow;
}
```

```
<p>Track &amp;amp; field champions:</p>
<ul>
  <li>Adhemar da Silva</li>
  <li>Wang Junxia</li>
  <li>Wilma Rudolph</li>
  <li>Babe Didrikson-Zaharias</li>
  <li>Betty Cuthbert</li>
  <li>Fanny Blankers-Koen</li>
  <li>Florence Griffith-Joyner</li>
  <li>Irena Szewinska</li>
  <li>Jackie Joyner-Kersee</li>
  <li>Shirley Strickland</li>
  <li>Carl Lewis</li>
  <li>Emil Zatopek</li>
  <li>Haile Gebrselassie</li>
  <li>Jesse Owens</li>
  <li>Jim Thorpe</li>
```

```
  <li>Paavo Nurmi</li>
  <li>Sergei Bubka</li>
  <li>Usain Bolt</li>
</ul>
```

> **Note:** In the `element:nth-child()` syntax, the child count includes sibling children of any element type; but it is considered a match only if the element *at that child position* matches the other components of the selector.

Syntax

```
:nth-child([ <An+B> | even | odd ] [of <complex-selector-list>]?) {
  /* ... */
}
```

Parameters

`:nth-child()` takes a single argument that describes a pattern for matching element indices in a list of siblings. Element indices are 1-based.

Keyword values

`odd`

Represents elements whose numeric position in a series of siblings is odd: 1, 3, 5, etc.

`even`

Represents elements whose numeric position in a series of siblings is even: 2, 4, 6, etc.

Functional notation

`<An+B>`

Represents elements whose numeric position in a series of siblings matches the pattern `An+B`, for every positive integer or zero value of `n`, where:

- `A` is an integer step size,
- `B` is an integer offset,
- `n` is all non-negative integers, starting from 0.

It can be read as the `An+B`-th element of a list. The `A` and `B` must both have `<integer>` values.

The `of <selector>` syntax

By passing a selector argument, we can select the **nth** element that matches that selector. For example, the following selector matches the first three list items which have a `class="important"` set.

```
:nth-child(-n + 3 of li.important) {
}
```

This is different from moving the selector outside of the function, like:

```
li.important:nth-child(-n + 3) {
}
```

This selector selects list items if they are among the first three children and match the selector `li.important`.

Examples

Example selectors

`tr:nth-child(odd)` or `tr:nth-child(2n+1)`

Represents the odd rows of an HTML table: 1, 3, 5, etc.

`tr:nth-child(even)` or `tr:nth-child(2n)`

Represents the even rows of an HTML table: 2, 4, 6, etc.

`:nth-child(7)`

Represents the seventh element.

`:nth-child(5n)`

Represents elements **5** [=5×1], **10** [=5×2], **15** [=5×3], **etc.** The first one to be returned as a result of the formula is **0** [=5x0], resulting in a no-match, since the elements are indexed from 1, whereas `n` starts from 0. This may seem weird at first, but it makes more sense when the `B` part of the formula is `>0`, like in the next example.

`:nth-child(n+7)`

Represents the seventh and all following elements: **7** [=0+7], **8** [=1+7], **9** [=2+7], **etc.**

`:nth-child(3n+4)`

Represents elements **4** [=(3×0)+4], **7** [=(3×1)+4], **10** [=(3×2)+4], **13** [=(3×3)+4], **etc.**

`:nth-child(-n+3)`

Represents the first three elements. [=-0+3, -1+3, -2+3]

`p:nth-child(n)`

Represents every `<p>` element in a group of siblings. This selects the same elements as a simple `p` selector (although with a higher specificity).

`p:nth-child(1)` or `p:nth-child(0n+1)`

Represents every `<p>` that is the first element in a group of siblings. This is the same as the `:first-child` selector (and has the same specificity).

`p:nth-child(n+8):nth-child(-n+15)`

Represents the eighth through the fifteenth `<p>` elements of a group of siblings.

Detailed example

HTML

```
<h3>
  <code>span:nth-child(2n+1)</code>, WITHOUT an <code>&lt;em&gt;</code> among
  the child elements.
</h3>
<p>Children 1, 3, 5, and 7 are selected.</p>
<div class="first">
  <span>Span 1!</span>
  <span>Span 2</span>
  <span>Span 3!</span>
  <span>Span 4</span>
  <span>Span 5!</span>
  <span>Span 6</span>
  <span>Span 7!</span>
</div>

<br />

<h3>
  <code>span:nth-child(2n+1)</code>, WITH an <code>&lt;em&gt;</code> among the
  child elements.
</h3>
<p>
  Children 1, 5, and 7 are selected.<br />
  3 is used in the counting because it is a child, but it isn't selected because
  it isn't a <code>&lt;span&gt;</code>.
</p>
<div class="second">
  <span>Span!</span>
  <span>Span</span>
  <em>This is an `em`.</em>
  <span>Span</span>
  <span>Span!</span>
  <span>Span</span>
  <span>Span!</span>
  <span>Span</span>
</div>

<br />

<h3>
  <code>span:nth-of-type(2n+1)</code>, WITH an <code>&lt;em&gt;</code> among the
  child elements.
</h3>
<p>
  Children 1, 4, 6, and 8 are selected.<br />
  3 isn't used in the counting or selected because it is an
  <code>&lt;em&gt;</code>, not a <code>&lt;span&gt;</code>, and
  <code>nth-of-type</code> only selects children of that type. The
  <code>&lt;em&gt;</code> is completely skipped over and ignored.
</p>
<div class="third">
  <span>Span!</span>
  <span>Span</span>
  <em>This is an `em`.</em>
  <span>Span!</span>
  <span>Span</span>
  <span>Span!</span>
  <span>Span</span>
  <span>Span!</span>
</div>
```

CSS

```
* {
  font-family: sans-serif;
}

span,
div em {
  padding: 5px;
  border: 1px solid tomato;
  display: inline-block;
  margin-bottom: 3px;
}
```

```
.first span:nth-child(2n + 1),
.second span:nth-child(2n + 1),
.third span:nth-of-type(2n + 1) {
  background-color: tomato;
}
```

Result

Using 'of <selector>'

In this example there is an unordered list of names, some of them have been marked as **noted** using `class="noted"`. These have been highlighted with a thick bottom border.

HTML

```
<ul>
  <li class="noted">Diego</li>
  <li>Shilpa</li>
  <li class="noted">Caterina</li>
  <li>Jayla</li>
  <li>Tyrone</li>
  <li>Ricardo</li>
  <li class="noted">Gila</li>
  <li>Sienna</li>
```

```
  <li>Titilayo</li>
  <li class="noted">Lexi</li>
  <li>Aylin</li>
  <li>Leo</li>
  <li>Leyla</li>
  <li class="noted">Bruce</li>
  <li>Aisha</li>
  <li>Veronica</li>
  <li class="noted">Kyouko</li>
  <li>Shireen</li>
  <li>Tanya</li>
  <li class="noted">Marlene</li>
</ul>
```

CSS

```
* {
  font-family: sans-serif;
}

ul {
  display: flex;
  flex-wrap: wrap;
  list-style: none;
  font-size: 1.2rem;
  padding-left: 0;
}

li {
  margin: 0.125rem;
  padding: 0.25rem;
}

li {
  border: 1px solid tomato;
}

.noted {
  border-bottom: 5px solid tomato;
}
```

In the following CSS we are targeting the **even** list items that are marked with `class="noted"`.

```
li:nth-child(even of .noted) {
  background-color: tomato;
  border-bottom-color: seagreen;
}
```

Result

Items with `class="noted"` have a thick bottom border and items 3, 10 and 17 have a solid background as they are the *even* list items with `class="noted"`.

of selector syntax vs selector nth-child

In this example, there are two unordered lists of names. The first list shows the effect of `li:nth-child(-n + 3 of .noted)` and the second list shows the effect of `li.noted:nth-child(-n + 3)`.

HTML

```
<ul class="one">
  <li class="noted">Diego</li>
  <li>Shilpa</li>
  <li class="noted">Caterina</li>
  <li>Jayla</li>
  <li>Tyrone</li>
  <li>Ricardo</li>
  <li class="noted">Gila</li>
  <li>Sienna</li>
  <li>Titilayo</li>
  <li class="noted">Lexi</li>
</ul>
<ul class="two">
  <li class="noted">Diego</li>
  <li>Shilpa</li>
  <li class="noted">Caterina</li>
  <li>Jayla</li>
  <li>Tyrone</li>
  <li>Ricardo</li>
  <li class="noted">Gila</li>
  <li>Sienna</li>
  <li>Titilayo</li>
  <li class="noted">Lexi</li>
</ul>
```

CSS

```
* {
  font-family: sans-serif;
}

ul {
  display: flex;
  flex-wrap: wrap;
  list-style: none;
  font-size: 1.2rem;
  padding-left: 0;
}

li {
  margin: 0.125rem;
  padding: 0.25rem;
}
```

```
li {
  border: 1px solid tomato;
}

.noted {
  border-bottom: 5px solid tomato;
}
```

```
ul.one > li:nth-child(-n + 3 of .noted) {
  background-color: tomato;
  border-bottom-color: seagreen;
}

ul.two > li.noted:nth-child(-n + 3) {
  background-color: tomato;
  border-bottom-color: seagreen;
}
```

Result

The first case applies a style to the first three list items with `class="noted"` whether or not they are the first three items in the list.

The second case applies a style to the items with `class="noted"` if they are within the first 3 items in the list.

Using of selector to fix striped tables

A common practice for tables is to use *zebra-stripes* which alternates between light and dark background colors for rows, making tables easier to read and more accessible. If a row is hidden, the stripes will appear merged and alter the desired effect. In this example, you can see two tables with a `hidden` row. The second table handles hidden rows using `of :not([hidden])`.

HTML

```
<table class="broken">
  <thead>
    <tr><th>Name</th><th>Age</th><th>Country</th></tr>
  </thead>
  <tbody>
    <tr><td>Mamitiana</td><td>23</td><td>Madagascar</td></tr>
    <tr><td>Yuki</td><td>48</td><td>Japan</td></tr>
    <tr hidden><td>Tlayolotl</td><td>36</td><td>Mexico</td></tr>
    <tr><td>Adilah</td><td>27</td><td>Morocco</td></tr>
    <tr><td>Vieno</td><td>55</td><td>Finland</td></tr>
    <tr><td>Ricardo</td><td>66</td><td>Brazil</td></tr>
  </tbody>
</table>
<table class="fixed">
  <thead>
    <tr><th>Name</th><th>Age</th><th>Country</th></tr>
  </thead>
  <tbody>
    <tr><td>Mamitiana</td><td>23</td><td>Madagascar</td></tr>
    <tr><td>Yuki</td><td>48</td><td>Japan</td></tr>
    <tr hidden><td>Tlayolotl</td><td>36</td><td>Mexico</td></tr>
    <tr><td>Adilah</td><td>27</td><td>Morocco</td></tr>
    <tr><td>Vieno</td><td>55</td><td>Finland</td></tr>
    <tr><td>Ricardo</td><td>66</td><td>Brazil</td></tr>
  </tbody>
</table>
```

CSS

```
body {
  display: flex;
  justify-content: space-around;
}
td {
  padding: 0.125rem 0.5rem;
}
```

```
.broken > tbody > tr:nth-child(even) {
  background-color: silver;
}
```

```
.fixed > tbody > tr:nth-child(even of :not([hidden])) {
  background-color: silver;
}
```

Result

In the first table this is just using `:nth-child(even)` the third row has the `hidden` attribute applied to it. So in this instance the 3rd row is not visible and the 2nd & 4th rows are counted as even, which technically they are but visually they are not.

In the second table the *of syntax* is used to target only the `tr`s that are **not** hidden using `:nth-child(even of :not([hidden]))`.

Styling a table column

To style a table column, you can't set the style on the `<col>` element as table cells are not children of it (as you can with the row element, `<tr>`). Pseudo-classes like `:nth-child()` are handy to select the column cells.

In this example, we set different styles for each of the column.

HTML

```
<table>
<caption>Student roster</caption>
<colgroup>
  <col/>
  <col/>
  <col/>
</colgroup>
  <thead>
```

```
    <tr><th>Name</th><th>Age</th><th>Country</th></tr>
  </thead>
  <tbody>
    <tr><td>Mamitiana</td><td>23</td><td>Madagascar</td></tr>
    <tr><td>Yuki</td><td>48</td><td>Japan</td></tr>
  </tbody>
</table>
```

CSS

```
td {
  padding: 0.125rem 0.5rem;
  height: 3rem;
  border: 1px solid black;
}

tr :nth-child(1) {
  text-align: left;
  vertical-align: bottom;
  background-color: silver;
}

tbody tr :nth-child(2) {
  text-align: center;
  vertical-align: middle;
}

tbody tr :nth-child(3) {
  text-align: right;
  vertical-align: top;
  background-color: tomato;
}
```

Result

See also

- `:nth-of-type()`
- `:nth-last-child()`
- `:has()`: pseudo-class for selecting parent element
- Tree-structural pseudo-classes
- CSS selectors module

:nth-last-child() CSS pseudo-class

The **`:nth-last-child()`** CSS pseudo-class matches elements based on their position among a group of siblings, counting from the end.

Try it

```
p {
  font-weight: bold;
}

li:nth-last-child(-n + 3) {
  border: 2px solid orange;
  margin-top: 1px;
}

li:nth-last-child(even) {
  background-color: lightyellow;
}
```

```
<p>Eight deadliest wildfires:</p>
<ol reversed>
  <li>Matheson Fire</li>
  <li>Miramichi Fire</li>
  <li>1997 Indonesian fires</li>
  <li>Thumb Fire</li>
  <li>Great Hinckley Fire</li>
  <li>Cloquet Fire</li>
  <li>Kursha-2 Fire</li>
  <li>Peshtigo Fire</li>
</ol>
```

Syntax

```
:nth-last-child(<nth> [of <complex-selector-list>]?) {
  /* ... */
}
```

Parameters

The `:nth-last-child()` pseudo-class is specified with a single argument, which represents the pattern for matching elements, counting from the end.

Keyword values

`odd`

Represents elements whose numeric position in a series of siblings is odd: 1, 3, 5, etc., counting from the end.

`even`

Represents elements whose numeric position in a series of siblings is even: 2, 4, 6, etc., counting from the end.

Functional notation

`<An+B>`

Represents elements whose numeric position in a series of siblings matches the pattern `An+B`, for every positive integer or zero value of `n`, where:

- `A` is an integer step size,
- `B` is an integer offset,
- `n` is all non-negative integers, starting from 0.

It can be read as the `An+B`-th element of a list. The index of the first element, counting from the end, is `1`. The `A` and `B` must both have `<integer>` values.

The `of <selector>` syntax

By passing a selector argument, we can select the **nth-last** element that matches that selector. For example, the following selector matches the last three *important* list items, which are assigned with `class="important"`.

```
:nth-last-child(-n + 3 of li.important) {
}
```

> **Note:** This is different from moving the selector outside of the function, like:

```
li.important:nth-last-child(-n + 3) {
}
```

This selector applies a style to list items if they are also within the last three children.

Examples

Example selectors

`tr:nth-last-child(odd)` or `tr:nth-last-child(2n+1)`

Represents the odd rows of an HTML table: 1, 3, 5, etc., counting from the end.

`tr:nth-last-child(even)` or `tr:nth-last-child(2n)`

Represents the even rows of an HTML table: 2, 4, 6, etc., counting from the end.

`:nth-last-child(7)`

Represents the seventh element, counting from the end.

`:nth-last-child(5n)`

Represents elements 5, 10, 15, etc., counting from the end.

`:nth-last-child(3n+4)`

Represents elements 4, 7, 10, 13, etc., counting from the end.

`:nth-last-child(-n+3)`

Represents the last three elements among a group of siblings.

`p:nth-last-child(n)` or `p:nth-last-child(n+1)`

Represents every `<p>` element among a group of siblings. This is the same as a simple `p` selector. (Since `n` starts at zero, while the last element begins at one, `n` and `n+1` will both select the same elements.)

`p:nth-last-child(1)` or `p:nth-last-child(0n+1)`

Represents every `<p>` that is the first element among a group of siblings, counting from the end. This is the same as the `:last-child` selector.

Table example

HTML

```
<table>
  <tbody>
    <tr>
      <td>First line</td>
    </tr>
    <tr>
      <td>Second line</td>
    </tr>
    <tr>
      <td>Third line</td>
    </tr>
    <tr>
      <td>Fourth line</td>
    </tr>
    <tr>
      <td>Fifth line</td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  border: 1px solid blue;
}

/* Selects the last three elements */
tr:nth-last-child(-n + 3) {
  background-color: pink;
}

/* Selects every element starting from the second to last item */
tr:nth-last-child(n + 2) {
  color: blue;
}

/* Select only the last second element */
tr:nth-last-child(2) {
```

```
  font-weight: 600;
}
```

Result

Quantity query

A *quantity query* styles elements depending on how many of them there are. In this example, list items turn red when there are at least three of them in a given list. This is accomplished by combining the capabilities of the `nth-last-child` pseudo-class and the subsequent-sibling combinator.

HTML

```
<h4>A list of four items (styled):</h4>
<ol>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
</ol>

<h4>A list of two items (unstyled):</h4>
<ol>
  <li>One</li>
  <li>Two</li>
</ol>
```

CSS

```
/* If there are at least three list items,
   style them all */
li:nth-last-child(n + 3),
li:nth-last-child(3) ~ li {
  color: red;
}
```

Result

<code>of <selector></code> syntax example

In this example, there is an unordered list of names. Some items have a `noted` class applied and are then highlighted with a thick bottom border.

HTML

```
<ul>
  <li class="noted">Diego</li>
  <li>Shilpa</li>
  <li class="noted">Caterina</li>
  <li>Jayla</li>
  <li>Tyrone</li>
  <li>Ricardo</li>
  <li class="noted">Gila</li>
  <li>Sienna</li>
  <li>Titilayo</li>
  <li class="noted">Lexi</li>
  <li>Aylin</li>
  <li>Leo</li>
  <li>Leyla</li>
  <li class="noted">Bruce</li>
  <li>Aisha</li>
  <li>Veronica</li>
  <li class="noted">Kyouko</li>
  <li>Shireen</li>
  <li>Tanya</li>
  <li class="noted">Marlene</li>
</ul>
```

CSS

```
* {
  font-family: sans-serif;
}

ul {
  display: flex;
  flex-wrap: wrap;
  list-style: none;
  font-size: 1.2rem;
  padding-left: 0;
}

li {
  margin: 0.125rem;
  padding: 0.25rem;
  border: 1px solid tomato;
}

.noted {
  border-bottom: 5px solid tomato;
}
```

In the following CSS we are targeting the **odd** list items that are marked with `class="noted"`.

```
li:nth-last-child(odd of .noted) {
  background-color: tomato;
  border-bottom-color: seagreen;
}
```

Result

Items with `class="noted"` have a thick bottom border and items 1, 7, 14, and 20 have a solid background as they are the *odd* list items with `class="noted"`.

See also

- `:nth-child`
- `:nth-last-of-type`
- Quantity Queries for CSS

:nth-last-of-type() CSS pseudo-class

The **`:nth-last-of-type()`** CSS pseudo-class matches elements based on their position among siblings of the same type (tag name), counting from the end.

Try it

```
dt {
  font-weight: bold;
}

dd {
  margin: 3px;
}

dd:nth-last-of-type(3n) {
  border: 2px solid orange;
}
```

```
<dl>
  <dt>Vegetables:</dt>
  <dd>1. Tomatoes</dd>
  <dd>2. Cucumbers</dd>
  <dd>3. Mushrooms</dd>
  <dt>Fruits:</dt>
  <dd>4. Apples</dd>
  <dd>5. Mangos</dd>
  <dd>6. Pears</dd>
  <dd>7. Oranges</dd>
</dl>
```

Syntax

```
:nth-last-of-type(<An+B> | even | odd) {
  /* ... */
}
```

Parameters

The `:nth-last-of-type()` pseudo-class is specified with a single argument, which represents the pattern for matching elements, counting from the end.

See `:nth-last-child` for a more detailed explanation of its syntax.

Examples

HTML

```
<div>
  <span>This is a span.</span>
  <span>This is another span.</span>
  <em>This is emphasized.</em>
  <span>Wow, this span gets limed!!!</span>
  <del>This is struck through.</del>
  <span>Here is one last span.</span>
</div>
```

CSS

```
span:nth-last-of-type(2) {
  background-color: lime;
}
```

Result

See also

- `:nth-last-child`, `:nth-of-type`

:nth-of-type() CSS pseudo-class

The **`:nth-of-type()`** CSS pseudo-class matches elements based on their position among siblings of the same type (tag name).

Try it

```
dt {
  font-weight: bold;
}

dd {
  margin: 3px;
}

dd:nth-of-type(even) {
  border: 2px solid orange;
}
```

```
<dl>
  <dt>Vegetables:</dt>
  <dd>1. Tomatoes</dd>
  <dd>2. Cucumbers</dd>
  <dd>3. Mushrooms</dd>
  <dt>Fruits:</dt>
  <dd>4. Apples</dd>
  <dd>5. Mangos</dd>
```

```
  <dd>6. Pears</dd>
  <dd>7. Oranges</dd>
</dl>
```

Syntax

```
:nth-of-type(<An+B> | even | odd) {
  /* ... */
}
```

Parameters

The `:nth-of-type()` pseudo-class is specified with a single argument, which represents the pattern for matching elements.

See `:nth-child` for a more detailed explanation of its syntax.

Examples

Basic example

HTML

```
<div>
  <div>This element isn't counted.</div>
  <p>1st paragraph.</p>
  <p class="fancy">2nd paragraph.</p>
  <div>This element isn't counted.</div>
  <p class="fancy">3rd paragraph.</p>
  <p>4th paragraph.</p>
</div>
```

CSS

```
/* Odd paragraphs */
p:nth-of-type(2n + 1) {
  color: red;
}

/* Even paragraphs */
p:nth-of-type(2n) {
  color: blue;
}

/* First paragraph */
p:nth-of-type(1) {
  font-weight: bold;
}

/* This will match the 3rd paragraph as it will match elements which are 2n+1 AND have a class of fancy.
The second paragraph has a class of fancy but is not matched as it is not :nth-of-type(2n+1) */
p.fancy:nth-of-type(2n + 1) {
  text-decoration: underline;
}
```

Result

> **Note:** There is no way to select the "nth-of-class" using this selector. The `:nth-of-type` selector looks at the type only when creating the list of matches. You can however use the `:nth-of-child` selector `of` syntax to achieve an "nth-of-class" behavior. You can also apply CSS to an element based on `:nth-of-type` location **and** a class, as shown in the previous example.

See also

- `:nth-child`, `:nth-last-of-type`

:only-child CSS pseudo-class

The **`:only-child`** CSS pseudo-class represents an element without any siblings. This is the same as `:first-child:last-child` or `:nth-child(1):nth-last-child(1)`, but with a lower specificity.

Try it

```
li:only-child {
  color: fuchsia;
}

b:only-child {
  text-decoration: underline;
}
```

```
<p>Stars expected to attend:</p>
<ol>
  <li>Robert Downey, Jr.</li>
</ol>

<p>Stars yet to confirm:</p>
<ol>
  <li>Scarlett Johansson</li>
  <li>Samuel L. Jackson</li>
  <li>Chris Pratt</li>
</ol>

<p>The ceremony is going to be held in <b>The Dolby Theatre</b>.</p>
```

Syntax

```
:only-child {
  /* ... */
}
```

Examples

Basic example

HTML

```
<div>
  <div>I am an only child.</div>
</div>

<div>
  <div>I am the 1st sibling.</div>
  <div>I am the 2nd sibling.</div>
  <div>
    I am the 3rd sibling,
    <div>but this is an only child.</div>
  </div>
</div>
```

CSS

```
div:only-child {
  color: red;
}

div {
  display: inline-block;
  margin: 6px;
  outline: 1px solid;
}
```

Result

A list example

HTML

```
<ol>
  <li>
    First
    <ul>
      <li>This list has just one element.</li>
    </ul>
  </li>
  <li>
    Second
    <ul>
      <li>This list has three elements.</li>
      <li>This list has three elements.</li>
      <li>This list has three elements.</li>
    </ul>
  </li>
</ol>
```

CSS

```
li li {
  list-style-type: disc;
}

li:only-child {
  color: red;
  list-style-type: square;
}
```

Result

See also

- `:only-of-type`
- `:first-child`
- `:last-child`
- `:nth-child`

:only-of-type CSS pseudo-class

The **`:only-of-type`** CSS pseudo-class represents an element that has no siblings of the same type (tag name).

Try it

```
a:only-of-type {
  color: fuchsia;
}

dd:only-of-type {
  background-color: bisque;
}
```

```
<p>
  To find out more about <b>QUIC</b>, check <a href="#">RFC 9000</a> and
  <a href="#">RFC 9114</a>.
</p>

<dl>
  <dt>Published</dt>
  <dd>2021</dd>
  <dd>2022</dd>
</dl>

<p>Details about <b>QPACK</b> can be found in <a href="#">RFC 9204</a>.</p>
```

```
<dl>
  <dt>Published</dt>
  <dd>2022</dd>
</dl>
```

Syntax

```
:only-of-type {
  /* ... */
}
```

Examples

Styling elements with no siblings of the same type

HTML

```
<main>
  <div>I am `div` #1.</div>
  <p>I am the only `p` among my siblings.</p>
  <div>I am `div` #2.</div>
  <div>
    I am `div` #3.
    <i>I am the only `i` child.</i>
    <em>I am `em` #1.</em>
    <em>I am `em` #2.</em>
  </div>
</main>
```

CSS

```
main :only-of-type {
  color: red;
}
```

Result

See also

- `:only-child`
- `:first-of-type`
- `:last-of-type`
- `:nth-of-type`

:open CSS pseudo-class

The **`:open`** CSS pseudo-class represents an element that has open and closed states, only when it is currently in the open state.

Syntax

```
:open {
  /* ... */
}
```

Description

The `:open` pseudo-class selects any element currently in the open state, which includes the following elements:

- `<details>` and `<dialog>` elements that are in an open state, that is, they have the `open` attribute set. This selection can also be done using an attribute selector: `details[open]`.
- `<input>` elements that display a picker interface for the user to choose a value from (for example `<input type="color">`), when the picker is displayed.
- `<select>` elements that display a drop-down picker for the user to choose a value from, when the picker is displayed. Note that when implementing customizable select elements, the picker itself can be selected using the `::picker(select)` pseudo-element.

Note that the open and closed states are semantic states, and don't necessary correlate with the visibility of the element in question. For example, a `<details>` element that is expanded to show its content is open, and will be selected by the `details:open` selector, even if it is hidden with a `visibility` value of `hidden`.

Popover elements (that is, elements with the `popover` attribute set on them) have distinct semantic states representing popovers that are showing or hidden, which can coexist alongside open and closed states. To target a popover element in a showing state, use the `:popover-open` pseudo-class instead.

Examples

Basic <code>:open</code> usage

This example demonstrates some of the HTML elements that have an open state.

CSS

```
details:open > summary {
  background-color: pink;
}

:is(select, input):open {
  background-color: pink;
}
```

```
@supports not selector(:open) {
  body::before {
    content: "Your browser doesn't support the :open selector.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

HTML

```
<details>
  <summary>Details</summary>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. In pulvinar dapibus
  lacus, sit amet finibus lectus mollis eu. Nullam quis orci dictum, porta lacus
  et, cursus nunc. Aenean pulvinar imperdiet neque fermentum facilisis. Nulla
  facilisi. Curabitur vitae sapien ut nunc pulvinar semper vitae vitae nisi.
</details>
<hr />

<label for="pet-select">Choose a pet:</label>
<select id="pet-select">
  <option value="dog">Dog</option>
  <option value="cat">Cat</option>
  <option value="hamster">Hamster</option>
</select>
<hr />

<label for="start">Start date:</label>
<input type="date" id="start" />
```

Result

Custom <code><select></code> styling with <code>:open</code>

In this example, we give a basic `<select>` element some custom styling. The `:open` pseudo-class is used to apply a styling enhancement to its open state — when the dropdown menu is displayed.

HTML

There is nothing special about our fruit selector.

```
<label>
  Choose your favorite fruit:
  <select name="fruit">
    <option>apple</option>
    <option>banana</option>
    <option>boysenberry</option>
    <option>cranberry</option>
    <option>fig</option>
    <option>grapefruit</option>
    <option>lemon</option>
    <option>orange</option>
    <option>papaya</option>
    <option>pomegranate</option>
    <option>tomato</option>
  </select>
</label>
```

> **Note:** We are not using a multi-line `<select>` (that is, one with the `multiple` attribute set) — those tend to render as a scrolling list box rather than a dropdown menu, so don't have an open state.

CSS

In the CSS, we set an `appearance` value of `none` on our `<select>` element to remove the default OS styling from the select box, and provide some basic styles of or own. Most notably, we set an SVG background image of a down arrow on the right-hand side — users tend to recognize `<select>` elements by the down arrow, so it is a good idea to include it.

We then set some `padding` on the surrounding `<label>` element, and a transparent border to keep the layout consistent when we later add a colored border to it.

```
select {
  appearance: none;
  width: 100%;
  display: block;
  font-family: inherit;
  font-size: 100%;
  padding: 5px;
  border: 1px solid black;
  background: url("data:image/svg+xml,%3Csvg width='20' height='20' viewBox='0 0 20 20' xmlns='http://www.w3.org/2000/svg'%3E%3Cpolygon points='5,5 15,5 10,15'/%3E%3C/svg%3E")
    no-repeat right 3px center / 1em 1em;
}

label {
  font-family: sans-serif;
  max-width: 20em;
  display: block;
  padding: 20px;
  border: 2px solid transparent;
}
```

When the `<select>` is opened, we use the `:open` pseudo-class to set a different background color and change the background image to an up arrow. We also set a different background color and border on the enclosing `<label>` element using a combination of the `:open` and `:has()` pseudo-classes to create a parent selector. We are literally saying "select the `<label>`, but only when its descendant `<select>` is open."

```
select:open {
  background-color: #f8f2dc;
  background-image: url("data:image/svg+xml,%3Csvg width='20' height='20' viewBox='0 0 20 20' xmlns='http://www.w3.org/2000/svg'%3E%3Cpolygon points='5,15 10,5 15,15'/%3E%3C/svg%3E");
}

label:has(select:open) {
  background-color: #81adc8;
  border-color: #cd4631;
}
```

Result

The result is as follows. Try opening the `<select>` dropdown to see the effect on the styling:

See also

- `<details>`, `<dialog>`, `<select>`, and `<input>` elements

- `:popover-open` pseudo-class
- `:modal`

:optional CSS pseudo-class

The **`:optional`** CSS pseudo-class represents any `<input>`, `<select>`, or `<textarea>` element that does not have the `required` attribute set on it.

Try it

```
label {
  display: block;
  margin-top: 0.5em;
}

fieldset {
  margin-top: 0.5em;
}

*:optional {
  background-color: palegreen;
}
```

```
<label for="name">Name (required):</label>
<input id="name" name="name" type="text" required />

<label for="birth">Date of birth:</label>
<input id="birth" name="birth" type="date" />

<label for="origin">How did you find out about us? (required)</label>
<select id="origin" name="origin" required>
  <option value="">Choose an option</option>
  <option value="google">Google</option>
  <option value="facebook">Facebook</option>
  <option value="advertisement">Advertisement</option>
</select>

<fieldset>
  <legend>Preferred contact method:</legend>
  <label>
    <input name="contact" type="radio" value="email" checked /> Email
  </label>
  <label><input name="contact" type="radio" value="phone" /> Phone</label>
</fieldset>

<label>
  <input name="newsletter" type="checkbox" checked /> I want to subscribe to a
  personalized newsletter.
</label>
<label>
  <input name="privacy" type="checkbox" required /> I have read and I agree to
  the Privacy Policy (required).
</label>
```

This pseudo-class is useful for styling fields that are not required to submit a form.

> **Note:** The `:required` pseudo-class selects *required* form fields.

Syntax

```
:optional {
  /* ... */
}
```

Accessibility

If a form contains optional `<input>`s, required inputs should be indicated using the `required` attribute. This will ensure that people navigating with the aid of assistive technology such as a screen reader will be able to understand which inputs need valid content to ensure a successful form submission.

Required inputs should also be indicated visually, using a treatment that does not rely solely on color to convey meaning. Typically, descriptive text and/or an icon are used.

- MDN Understanding WCAG, Guideline 3.3 explanations
- Understanding Success Criterion 3.3.2 | W3C Understanding WCAG 2.0

Examples

The optional field has a purple border

HTML

```
<form>
  <div class="field">
    <label for="url_input">Enter a URL:</label>
    <input type="url" id="url_input" />
  </div>

  <div class="field">
    <label for="email_input">Enter an email address:</label>
    <input type="email" id="email_input" required />
  </div>
</form>
```

CSS

```
label {
  display: block;
  margin: 1px;
  padding: 1px;
}
```

```
.field {
  margin: 1px;
  padding: 1px;
}

input:optional {
  border-color: rebeccapurple;
  border-width: 3px;
}
```

Result

See also

- Other validation-related pseudo-classes: `:required`, `:invalid`, `:valid`
- Form data validation

:out-of-range CSS pseudo-class

The **`:out-of-range`** CSS pseudo-class represents an `<input>` element whose current value is outside the range limits specified by the `min` and `max` attributes.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:out-of-range {
  background-color: orangered;
}
```

```
<label for="amount">How many tickets? (You can buy 2-6 tickets)</label>
<input id="amount" name="amount" type="number" min="2" max="6" value="4" />

<label for="dep">Departure Date: (Whole year 2022 is acceptable)</label>
<input
  id="dep"
  name="dep"
  type="date"
  min="2022-01-01"
  max="2022-12-31"
  value="2025-05-05" />

<label for="ret">Return Date: (Whole year 2022 is acceptable)</label>
<input id="ret" name="ret" type="date" min="2022-01-01" max="2022-12-31" />
```

This pseudo-class is useful for giving the user a visual indication that a field's current value is outside the permitted limits.

> **Note:** This pseudo-class only applies to elements that have (and can take) a range limitation. In the absence of such a limitation, the element can neither be "in-range" nor "out-of-range."

Syntax

```
:out-of-range {
  /* ... */
}
```

Examples

HTML

```
<p>Values between 1 and 10 are valid.</p>
<ul>
  <li>
    <input
      id="value1"
      name="value1"
      type="number"
      placeholder="1 to 10"
      min="1"
      max="10"
      value="12" />
    <label for="value1">Your value is </label>
  </li>
</ul>
```

CSS

```
li {
  list-style: none;
  margin-bottom: 1em;
}

input {
  border: 1px solid black;
}

input:in-range {
  background-color: rgb(0 255 0 / 25%);
}

input:out-of-range {
  background-color: rgb(255 0 0 / 25%);
  border: 2px solid red;
}

input:in-range + label::after {
```

```
  content: "okay.";
}

input:out-of-range + label::after {
  content: "out of range!";
}
```

Result

See also

- `:in-range`
- Form data validation

:past CSS pseudo-class

The **`:past`** CSS pseudo-class selector is a time-dimensional pseudo-class that will match for any element which appears entirely before an element that matches `:current`. For example in a video with captions which are being displayed by WebVTT.

```
:past(p, span) {
  display: none;
}
```

Syntax

```
:past {
  /* ... */
}
```

Examples

CSS

```
:past(p, span) {
  display: none;
}
```

HTML

```
<video controls preload="metadata">
  <source src="video.mp4" type="video/mp4" />
  <source src="video.webm" type="video/webm" />
  <track
    label="English"
    kind="subtitles"
    srclang="en"
    src="subtitles.vtt"
    default />
</video>
```

WebVTT

```
WEBVTT FILE

1
00:00:03.500 --> 00:00:05.000
This is the first caption

2
00:00:06.000 --> 00:00:09.000
This is the second caption

3
00:00:11.000 --> 00:00:19.000
This is the third caption
```

See also

- Web Video Text Tracks Format (WebVTT)
- `:current`
- `:future`

:paused CSS pseudo-class

The **`:paused`** CSS pseudo-class selector represents an element that is playable, such as `<audio>` or `<video>`, when that element is "paused" (i.e., not "playing").

A resource is paused if the user explicitly paused it, or if it is in a non-activated or other non-playing state, like "loaded, hasn't been activated yet". This is different from `:buffering` or `:stalled`, which are states that occur while the resource is considered "playing".

Syntax

```
:paused {
  /* ... */
}
```

Examples

CSS

```
:paused {
  border: 5px solid orange;
}
```

See also

- `:buffering`
- `:muted`

- `:playing`
- `:seeking`
- `:stalled`
- `:volume-locked`
- CSS selectors

:picture-in-picture CSS pseudo-class

The **`:picture-in-picture`** CSS pseudo-class matches the element which is currently in picture-in-picture mode.

Syntax

```
:picture-in-picture {
  /* ... */
}
```

Usage notes

The `:picture-in-picture` pseudo-class lets you configure your stylesheets to automatically adjust the size, style, or layout of content when a video switches back and forth between picture-in-picture and traditional presentation modes.

Examples

In this example, a video has a box shadow when it is displayed in the floating window.

HTML

The page's HTML looks like this:

```
<h1>MDN Web Docs Demo: :picture-in-picture pseudo-class</h1>

<p>
  This demo uses the <code>:picture-in-picture</code> pseudo-class to
  automatically change the style of a video entirely using CSS.
</p>

<video id="pip-video"></video>
```

The `<video>` with the ID `"pip-video"` will change between having a red box shadow or not, depending on whether or not it is displayed in the picture-in-picture floating window.

CSS

The magic happens in the CSS.

```
:picture-in-picture {
  box-shadow: 0 0 0 5px red;
}
```

See also

- Picture-in-picture API
- `HTMLVideoElement.requestPictureInPicture()`
- `HTMLVideoElement.disablePictureInPicture`
- `Document.pictureInPictureEnabled`
- `Document.exitPictureInPicture()`
- `Document.pictureInPictureElement`

:placeholder-shown CSS pseudo-class

The **`:placeholder-shown`** CSS pseudo-class represents any `<input>` or `<textarea>` element that is currently displaying placeholder text.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:placeholder-shown {
  background-color: ivory;
  border: 2px solid darkorange;
  border-radius: 5px;
}
```

```
<label for="name">Full Name:</label>
<input id="name" name="name" type="text" />

<label for="email">Email Address:</label>
<input id="email" name="email" type="email" placeholder="name@example.com" />

<label for="age">Your age:</label>
<input
  id="age"
  name="age"
  type="number"
  value="18"
  placeholder="You must be 18+" />
```

Syntax

```
:placeholder-shown {
  /* ... */
}
```

Examples

Basic example

This example applies special font and border styles when the placeholder is shown.

HTML

```
<input placeholder="Type something here!" />
```

CSS

```
input {
  border: 1px solid black;
  padding: 3px;
}

input:placeholder-shown {
  border-color: teal;
  color: purple;
  font-style: italic;
}
```

Result

Overflowing text

When form fields are too small, placeholder text can get cropped in an undesirable way. You can use the `text-overflow` property to alter the way overflowing text is displayed.

HTML

```
<input id="input1" placeholder="Name, Rank, and Serial Number" /> <br /><br />
<input id="input2" placeholder="Name, Rank, and Serial Number" />
```

CSS

```
#input2:placeholder-shown {
  text-overflow: ellipsis;
}
```

Result

Customized input field

The following example highlights the Student ID field with a custom style.

HTML

```
<p>
  <label for="name">Student Name:</label>
  <input id="name" placeholder="Student Name" />
</p>
<p>
  <label for="branch">Student Branch:</label>
  <input id="branch" placeholder="Student Branch" />
</p>
<p>
  <label for="sid">8-Digit Student ID:</label>
  <input
    pattern="[0-9]{8}"
    id="sid"
    class="student-id"
    placeholder="8 digit id" />
</p>
```

CSS

```
input {
  background-color: #e8e8e8;
  color: black;
}

input.student-id:placeholder-shown {
  background-color: yellow;
  color: red;
  font-style: italic;
}
```

Result

See also

- The `::placeholder` pseudo-element styles the placeholder *itself*.
- Related HTML elements: `<input>`, `<textarea>`
- HTML forms

:playing CSS pseudo-class

The **`:playing`** CSS pseudo-class selector represents the playback state of an element that is playable, such as `<audio>` or `<video>`, when that element is "playing". An element is considered to be playing if it is currently playing the media resource, or if it has temporarily stopped for reasons other than user intent (such as `:buffering` or `:stalled`).

Syntax

```
:playing {
  /* ... */
}
```

Examples

CSS

```
:playing {
  border: 5px solid green;
}
```

See also

- `:buffering`
- `:muted`
- `:paused`
- `:seeking`
- `:stalled`
- `:volume-locked`
- CSS selectors

:popover-open CSS pseudo-class

The **`:popover-open`** CSS pseudo-class represents a popover element (i.e., one with a `popover` attribute) that is in the showing state. You can use this to apply style to popover elements only when they are shown.

Syntax

```
:popover-open {
  /* ... */
}
```

Examples

By default, popovers appear in the middle of the viewport. The default styling is achieved like this in the UA stylesheet:

```
[popover] {
  position: fixed;
  inset: 0;
  width: fit-content;
  height: fit-content;
  margin: auto;
  border: solid;
  padding: 0.25em;
  overflow: auto;
  color: CanvasText;
  background-color: Canvas;
}
```

To override the default styles and get the popover to appear somewhere else on your viewport, you could override the above styles with something like this:

```
:popover-open {
  width: 200px;
  height: 100px;
  position: absolute;
  inset: unset;
  bottom: 5px;
  right: 5px;
  margin: 0;
}
```

See also

- Popover API
- `popover` HTML global attribute

:read-only CSS pseudo-class

The **`:read-only`** CSS pseudo-class selects elements (such as certain `<input>` types and `<textarea>`) that are not editable by the user. Elements on which the HTML attribute `readonly` doesn't have an effect (such as `<input type="radio">`, `<input type="checkbox">`, and all other non-form elements) are also selected by the `:read-only` pseudo-class. In fact, `:read-only` matches anything that `:read-write` doesn't match, making it equivalent to `:not(:read-write)`.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

*:read-only {
  font-weight: bold;
  color: indigo;
}
```

```
<p>Please enter your details:</p>

<label for="email">Email Address:</label>
<input id="email" name="email" type="email" value="test@example.com" readonly />

<label for="note">Short note about yourself:</label>
<textarea id="note" name="note">Don't be shy</textarea>

<label for="pic">Your picture:</label>
<input id="pic" name="pic" type="file" />
```

Syntax

```
:read-only {
  /* ... */
}
```

Examples

Confirming form information using read-only or read-write controls

One use of read-only form controls is to allow the user to check and verify information that they may have entered in an earlier form (for example, shipping details), while still being able to submit the information along with the rest of the form. This example shows how to style these controls.

The `:read-only` pseudo-class is used to remove all the styling that makes the inputs look like clickable fields, making them look more like read-only paragraphs. The `:read-write` pseudo-class on the other hand is used to provide some nicer styling to the editable `<textarea>`.

```
<fieldset>
  <legend>Check shipping details</legend>
  <div>
    <label for="name">Name: </label>
    <input id="name" name="name" type="text" value="Mr Soft" readonly />
  </div>
  <div>
    <label for="address">Address: </label>
    <textarea id="address" name="address" readonly>
23 Elastic Way,
Viscous,
Bright Ridge,
CA
</textarea>
  </div>
  <div>
    <label for="postal-code">Zip/postal code: </label>
    <input
      id="postal-code"
      name="postal-code"
      type="text"
      value="94708"
      readonly />
  </div>
</fieldset>

<fieldset>
  <legend>Final instructions</legend>
  <div>
    <label for="sms-confirm">Send confirmation by SMS?</label>
    <input id="sms-confirm" name="sms-confirm" type="checkbox" />
  </div>
  <div>
    <label for="instructions">Any special instructions?</label>
    <textarea id="instructions" name="instructions"></textarea>
  </div>
</fieldset>
```

```
body {
  font-family: "Josefin Sans", sans-serif;
  margin: 20px auto;
  max-width: 460px;
}

fieldset {
  padding: 10px 30px 0;
  margin-bottom: 20px;
}

legend {
  color: white;
  background: black;
  padding: 5px 10px;
}

fieldset > div {
  margin-bottom: 20px;
  display: flex;
  justify-content: space-between;
}

label,
input[type="text"],
textarea {
  display: block;
  font-family: inherit;
  font-size: 100%;
  margin: 0;
  box-sizing: border-box;
  padding: 5px;
  height: 30px;
}

input[type="text"],
textarea {
  width: 50%;
}

textarea {
  height: 110px;
  resize: none;
}

label {
  width: 40%;
}

input:hover,
input:focus,
textarea:hover,
textarea:focus {
```

```
  background-color: #eeeeee;
}
```

```
input:read-only,
textarea:read-only {
  border: 0;
  box-shadow: none;
  background-color: #dddddd;
}

textarea:read-write {
  outline: 1px dashed red;
  outline-offset: 2px;
  border-radius: 5px;
}
```

Styling read-only non-form controls

This selector doesn't just select `<input>`/`<textarea>` elements — it will select *any* element that cannot be edited by the user.

```
<p contenteditable>This paragraph is editable; it is read-write.</p>

<p>This paragraph is not editable; it is read-only.</p>
```

```
p {
  font-size: 150%;
  padding: 5px;
  border-radius: 5px;
}

p:read-only {
  background-color: red;
  color: white;
}

p:read-write {
  background-color: lime;
}
```

See also

- `:read-write`
- HTML `contenteditable` attribute

:read-write CSS pseudo-class

The **`:read-write`** CSS pseudo-class represents an element (such as `input` or `textarea`) that is editable by the user.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

*:read-write {
  background-color: ivory;
  border: 2px solid darkorange;
  border-radius: 5px;
}
```

```
<p>Please fill in your details:</p>

<label for="email">Email Address:</label>
<input id="email" name="email" type="email" value="test@example.com" />

<label for="note">Short note about yourself:</label>
<textarea id="note" name="note">Don't be shy</textarea>

<label for="pic">Your picture:</label>
<input id="pic" name="pic" type="file" />
```

Syntax

```
:read-write {
  /* ... */
}
```

Examples

Confirming form details using read-only controls

You can use `readonly` form controls when you want a user to verify information they entered earlier, which you want to submit with new data in read-write controls. In the example below, the `:read-only` pseudo-class is used to make the `<textarea>` (a user's address) look like a regular paragraph. The `:read-write` pseudo-class provides a way to highlight the editable `<textarea>` (the delivery instructions):

```
body {
  font-family: "Josefin Sans", sans-serif;
  margin: 10px auto;
}

legend {
  color: white;
  background: black;
  padding: 5px 10px;
}

fieldset > div {
  margin-bottom: 10px;
```

```
  display: flex;
  justify-content: space-between;
}

label,
textarea {
  display: block;
  font-family: inherit;
  font-size: 100%;
  margin: 0;
  box-sizing: border-box;
  padding: 5px;
  height: 30px;
}

textarea {
  width: 50%;
}

textarea {
  height: 70px;
  resize: none;
}

label {
  width: 40%;
}
```

```
textarea:read-only {
  border: 0;
}

textarea:read-write {
  box-shadow: inset 1px 1px 3px #cccccc;
  border-radius: 5px;
}
```

```
<fieldset>
  <legend>Confirm details</legend>
  <div>
    <label for="address">Address:</label>
    <textarea id="address" name="address" readonly>
123 Choco Mountain,
Awesome Ridge,
CA</textarea>
  </div>
  <div>
    <label for="instructions">Delivery instructions</label>
    <textarea id="instructions" name="instructions"></textarea>
  </div>
</fieldset>
```

Styling read-write non-form controls

This selector doesn't just select `<input>`/`<textarea>` elements — it will select *any* element that can be edited by the user, such as a `<p>` element with `contenteditable` set on it.

```
<p contenteditable>This paragraph is editable; it is read-write.</p>

<p>This paragraph is not editable; it is read-only.</p>
```

```
body {
  font-family: sans-serif;
}

p {
  font-size: 150%;
  padding: 5px;
  border-radius: 5px;
}

p:read-only {
  background-color: red;
  color: white;
}

p:read-write {
  background-color: lime;
}
```

See also

- `:read-only`
- HTML `contenteditable` attribute

:required CSS pseudo-class

The **`:required`** CSS pseudo-class represents any `<input>`, `<select>`, or `<textarea>` element that has the `required` attribute set on it.

Try it

```
label {
  display: block;
  margin-top: 0.5em;
}

fieldset {
  margin-top: 0.5em;
}

*:required {
```

```
  background-color: gold;
}
```

```
<label for="name">Name (required):</label>
<input id="name" name="name" type="text" required />

<label for="birth">Date of birth:</label>
<input id="birth" name="birth" type="date" />

<label for="origin">How did you find out about us? (required)</label>
<select id="origin" name="origin" required>
  <option value="">Choose an option</option>
  <option value="google">Google</option>
  <option value="facebook">Facebook</option>
  <option value="advertisement">Advertisement</option>
</select>

<fieldset>
  <legend>Preferred contact method:</legend>
  <label>
    <input name="contact" type="radio" value="email" checked /> Email
  </label>
  <label><input name="contact" type="radio" value="phone" /> Phone</label>
</fieldset>

<label>
  <input name="newsletter" type="checkbox" checked /> I want to subscribe to a
  personalized newsletter.
</label>
<label>
  <input name="privacy" type="checkbox" required /> I have read and I agree to
  the Privacy Policy (required).
</label>
```

This pseudo-class is useful for highlighting fields that must have valid data before a form can be submitted.

> **Note:** The `:optional` pseudo-class selects *optional* form fields.

Syntax

```
:required {
  /* ... */
}
```

Accessibility

Mandatory `<input>`s should have the `required` attribute applied to them. This will ensure that people navigating with the aid of assistive technology such as a screen reader will be able to understand which inputs need valid content to ensure a successful submission.

If the form also contains optional inputs, required inputs should be indicated visually using a treatment that does not rely solely on color to convey meaning. Typically, descriptive text and/or an icon are used.

- MDN Understanding WCAG, Guideline 3.3 explanations
- Understanding Success Criterion 3.3.2 | W3C Understanding WCAG 2.0

Examples

The required field has a red border

HTML

```
<form>
  <div class="field">
    <label for="url_input">Enter a URL:</label>
    <input type="url" id="url_input" />
  </div>

  <div class="field">
    <label for="email_input">Enter an email address:</label>
    <input type="email" id="email_input" required />
  </div>
</form>
```

CSS

```
label {
  display: block;
  margin: 1px;
  padding: 1px;
}

.field {
  margin: 1px;
  padding: 1px;
}

input:required {
  border-color: maroon;
  border-width: 3px;
}

input:required:invalid {
  border-color: #c00000;
}
```

Result

See also

- Other validation-related pseudo-classes: `:optional`, `:invalid`, `:valid`

- Form data validation

:right CSS pseudo-class

The **`:right`** CSS pseudo-class, used with the `@page` at-rule, represents all right-hand pages of a printed document.

```
/* Selects any right-hand pages when printing */
@page :right {
  margin: 2in 3in;
}
```

Whether a given page is "left" or "right" is determined by the major writing direction of the document. For example, if the first page has a major writing direction of left-to-right then it will be a `:right` page; if it has a major writing direction of right-to-left then it will be a `:left` page.

> **Note:** This pseudo-class can be used to change only the `margin`, `padding`, `border`, and `background` properties of the *page box*. All other properties will be ignored, and only the page box, not the document content on the page, will be affected.

Syntax

```
:right {
  /* ... */
}
```

Examples

Setting margins for right-hand pages

```
@page :right {
  margin: 2in 3in;
}
```

See also

- `@page`
- Other page-related pseudo-classes: `:first`, `:left`

:root CSS pseudo-class

The **`:root`** CSS pseudo-class matches the root element of a tree representing the document. In HTML, `:root` represents the `<html>` element and is identical to the selector `html`, except that its specificity (0-1-0) is higher than html's (0-0-1).

```
/* Selects the root element of the document:
   <html> in the case of HTML */
:root {
  background: yellow;
}
```

Syntax

```
:root {
  /* ... */
}
```

Examples

Declaring global CSS variables

`:root` can be useful for declaring global CSS variables:

```
:root {
  --main-color: hotpink;
  --pane-padding: 5px 42px;
}
```

See also

- `<html>`
- `Document.rootElement`
- `Node.getRootNode()`
- `Element.shadowRoot`
- `ShadowRoot`

:scope CSS pseudo-class

The **`:scope`** CSS pseudo-class represents elements that are a reference point, or scope, for selectors to match against.

```
/* Selects a scoped element */
:scope {
  background-color: lime;
}
```

Which element(s) `:scope` matches depends on the context in which it is used:

- When used at the root level of a stylesheet, `:scope` is equivalent to `:root`, which in a regular HTML document matches the `<html>` element.
- When used inside a `@scope` block, `:scope` matches the block's defined scope root. It provides a way to apply styles to the root of the scope from inside the `@scope` block itself.
- When used within a DOM API call — such as `querySelector()`, `querySelectorAll()`, `matches()`, or `Element.closest()` — `:scope` matches the element on which the method was called.

Syntax

```
:scope {
  /* ... */
}
```

Examples

Using <code>:scope</code> as an alternative to <code>:root</code>

This example shows that `:scope` is equivalent to `:root` when used at the root level of a stylesheet. In this case, the provided CSS colors the background of the `<html>` element orange.

```
:scope {
  background-color: orange;
}
```

Using <code>:scope</code> to style the scope root in a <code>@scope</code> block

In this example, we use two separate `@scope` blocks to match links inside elements with a `.light-scheme` and `.dark-scheme` class respectively. Note how `:scope` is used to select and provide styling to the scope roots themselves. In this example, the scope roots are the `<div>` elements that have the classes applied to them.

HTML

```
<div class="light-scheme">
  <p>
    MDN contains lots of information about
    <a href="/en-US/docs/Web/HTML">HTML</a>,
    <a href="/en-US/docs/Web/CSS">CSS</a>, and
    <a href="/en-US/docs/Web/JavaScript">JavaScript</a>.
  </p>
</div>

<div class="dark-scheme">
  <p>
    MDN contains lots of information about
    <a href="/en-US/docs/Web/HTML">HTML</a>,
    <a href="/en-US/docs/Web/CSS">CSS</a>, and
    <a href="/en-US/docs/Web/JavaScript">JavaScript</a>.
  </p>
</div>
```

CSS

```
div {
  padding: 10px;
}
```

```
@scope (.light-scheme) {
  :scope {
    background-color: plum;
  }

  a {
    color: indigo;
  }
}

@scope (.dark-scheme) {
  :scope {
    background-color: indigo;
    color: antiquewhite;
  }

  a {
    color: plum;
  }
}
```

Result

Using <code>:scope</code> in JavaScript

This example demonstrates using the `:scope` pseudo-class in JavaScript. This can be useful if you need to get a direct descendant of an already retrieved `Element`.

HTML

```
<div id="context">
  <div id="element-1">
    <div id="element-1-1"></div>
    <div id="element-1-2"></div>
  </div>
  <div id="element-2">
    <div id="element-2-1"></div>
  </div>
</div>
<p>
  Selected element ids :
  <span id="results"></span>
</p>
```

JavaScript

```
const context = document.getElementById("context");
const selected = context.querySelectorAll(":scope > div");

document.getElementById("results").textContent = [...selected]
  .map((element) => `#${element.id}`)
  .join(", ");
```

Result

The scope of `context` is the element with the `id` of `context`. The selected elements are the `<div>` elements that are direct children of that context — `element-1` and `element-2` — but not their descendants.

Sibling combinators to the right of <code>:scope</code> never match

The relationship defined by `:scope` is always ancestor-to-descendant from the scope root. Because of that, putting a sibling combinator to the right of `:scope` creates a selector that can never match, because no element can both be within the selector scope and also be a sibling of `:scope`.

```
:scope + p {
  color: red;
}

:scope ~ * {
  color: red;
}
```

Selectors that never match may trigger a warning in browser DevTools.

See also

- The `@scope` at-rule
- The `:root` pseudo-class
- Selection and traversal on the DOM tree
- `Element.querySelector()` and `Element.querySelectorAll()`
- `Document.querySelector()` and `Document.querySelectorAll()`
- `DocumentFragment.querySelector()` and `DocumentFragment.querySelectorAll()`

:seeking CSS pseudo-class

The **`:seeking`** CSS pseudo-class selector represents an element that is playable, such as `<audio>` or `<video>`, when the playable element is seeking a playback position in the media resource. A resource is considered to be seeking if the user has requested playback of a specific position in the media resource, but the media element has not yet reached that position.

Seeking is different from `:buffering` in that the media element is not currently loading data, but is instead skipping to a new position in the media resource. For more information, see the Media buffering, seeking, and time ranges guide.

Syntax

```
:seeking {
  /* ... */
}
```

Examples

CSS

```
:seeking {
  outline: 5px solid red;
}

video:seeking {
  outline: 5px solid blue;
}
```

See also

- `:buffering`
- `:muted`
- `:paused`
- `:playing`
- `:stalled`
- `:volume-locked`
- CSS selectors

:stalled CSS pseudo-class

The **`:stalled`** CSS pseudo-class selector represents an element that is playable, such as `<audio>` or `<video>`, when playback is stalled. A resource is considered to be stalled if the user has requested playback of a specific position in the media resource, but it has failed to receive any data for some amount of time. This is different from `:buffering` in that the media element is unexpectedly not loading data when stalled (e.g., due to a network error) for around 3 seconds (the exact time is user agent dependent).

> **Note:** Like with the `:buffering` pseudo-class, the element is still considered to be "playing" when it is "stalled". If `:stalled` matches an element, `:playing` will also match that element.

Syntax

```
:stalled {
  /* ... */
}
```

Examples

CSS

```
:stalled {
  outline: 5px solid red;
}

audio:stalled {
  background-color: red;
}
```

See also

- `:buffering`
- `:muted`
- `:paused`
- `:playing`
- `:seeking`

- `:volume-locked`
- CSS selectors
- `stalled` event

:state() CSS pseudo-class

The **`:state()`** CSS pseudo-class matches custom elements that have the specified custom state.

Syntax

```
:state(<custom identifier>) {
  /* ... */
}
```

Parameters

The `:state()` pseudo-class takes as its argument a custom identifier that represents the state of the custom element to match.

Description

Elements can transition between states due to user interaction and other factors. For instance, an element can be in the "hover" state when a user hovers over the element, or a link can be in the "visited" state after a user clicks on it. Elements provided by browsers can be styled based on these states using CSS pseudo-classes such as `:hover` and `:visited`. Similarly, autonomous custom elements (custom elements that are not derived from built-in elements) can expose their states, allowing pages that use the elements to style them using the CSS `:state()` pseudo-class.

The states of a custom element are represented by string values. These values are added to or removed from a `CustomStateSet` object associated with the element. The CSS `:state()` pseudo-class matches an element when the identifier, passed as an argument, is present in the `CustomStateSet` of the element.

The `:state()` pseudo-class can also be used to match custom states within the implementation of a custom element. This is achieved by using `:state()` within the `:host()` pseudo-class function, which matches a state only within the shadow DOM of the current custom element.

Additionally, the `::part()` pseudo-element followed by the `:state()` pseudo-class allows matching on the shadow parts of a custom element that are in a particular state. (**Shadow parts** are parts of a custom element's shadow tree that are explicitly exposed to a containing page for styling purposes.)

Examples

Matching a custom state

This CSS shows how to change the border of the autonomous custom element `<labeled-checkbox>` to `red` when it is in the "checked" state.

```
labeled-checkbox {
  border: dashed red;
}
labeled-checkbox:state(checked) {
  border: solid;
}
```

For a live example of this code in action, see the Matching the custom state of a custom checkbox element example on the `CustomStateSet` page.

Matching a custom state in a custom element's shadow DOM

This example shows how the `:state()` pseudo-class can be used within the `:host()` pseudo-class function to match custom states within the implementation of a custom element.

The following CSS injects a gray `[x]` before the element when it is in the "checked" state.

```
:host(:state(checked))::before {
  content: "[x]";
}
```

For a live example of this code in action, see the Matching the custom state of a custom checkbox element example on the `CustomStateSet` page.

Matching a custom state in a shadow part

This example shows how the `:state()` pseudo-class can be used to target the shadow parts of a custom element.

Shadow parts are defined and named using the `part` attribute. For example, consider a custom element named `<question-box>` that uses a `<labeled-checkbox>` custom element as a shadow part named `checkbox`:

```
shadowRoot.innerHTML = `<labeled-checkbox part='checkbox'>Yes</labeled-checkbox>`;
```

The CSS below shows how the `::part()` pseudo-element can be used to match against the `'checkbox'` shadow part. It then shows how the `::part()` pseudo-element followed by the `:state()` pseudo-class can be used to match against the same part when it is in the `checked` state.

```
question-box::part(checkbox) {
  color: red;
}

question-box::part(checkbox):state(checked) {
  color: green;
  outline: dashed 1px green;
}
```

For a live example of this code in action, see the Matching a custom state in a shadow part of a custom element example on the `CustomStateSet` page.

See also

- `CustomStateSet`
- Pseudo-classes
- Learn: Pseudo-classes and pseudo-elements
- Custom states and custom state pseudo-class CSS selectors in Using custom elements

:target CSS pseudo-class

The **`:target`** CSS pseudo-class selects the *target element of the document*. When the document is loaded, the target element is derived using the document's URL fragment identifier.

```
/* Selects document's target element */
:target {
```

```
  border: 2px solid black;
}
```

For example, the following URL has a fragment identifier (denoted by the # sign) that marks the element with the `id` of `setup` as the document's target element:

```
http://www.example.com/help/#setup
```

The following element would be selected by a `:target` selector when the current URL is equal to the above:

```
<section id="setup">Installation instructions</section>
```

Syntax

```
:target {
  /* ... */
}
```

Description

When an HTML document loads, the browser sets its target element. The element is identified using the URL fragment identifier. Without the URL fragment identifier, the document has no target element. The `:target` pseudo-class allows styling the document's target element. The element could be focused, highlighted, animated, etc.

The target element is set at document load and `history.back()`, `history.forward()`, and `history.go()` method calls. But it is *not* changed when `history.pushState()` and `history.replaceState()` methods are called.

> **Note:** Due to a possible bug in the CSS specification, `:target` doesn't work within a web component because the shadow root doesn't pass the target element down to the shadow tree.

Examples

A table of contents

The `:target` pseudo-class can be used to highlight the portion of a page that has been linked to from a table of contents.

HTML

```
<h3>Table of Contents</h3>
<ol>
  <li><a href="#p1">Jump to the first paragraph!</a></li>
  <li><a href="#p2">Jump to the second paragraph!</a></li>
  <li>
    <a href="#nowhere">
      This link goes nowhere, because the target doesn't exist.
    </a>
  </li>
</ol>

<h3>My Fun Article</h3>
<p id="p1">
  You can target <i>this paragraph</i> using a URL fragment. Click on the first
  link above to try out!
</p>
<p id="p2">
  This is <i>another paragraph</i>, also accessible from the second link above.
  Isn't that delightful?
</p>
```

CSS

```
p:target {
  background-color: gold;
}

/* Add a pseudo-element inside the target element */
p:target::before {
  font: 70% sans-serif;
  content: "►";
  color: limegreen;
  margin-right: 0.25em;
}

/* Style italic elements within the target element */
p:target i {
  color: red;
}
```

Result

See also

- Using the :target pseudo-class in selectors
- `::target-text` (for highlighting text fragments)

:target-after CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:target-after`** CSS pseudo-class selects scroll markers that come *after* the active scroll marker (the one that currently matches `:target-current`) within a scroll marker group. You can use this selector to style navigation items that come after the current navigation position within a scroll marker group.

> **Note:** The `:target-after` pseudo-class is only valid on `::scroll-marker` pseudo-elements and elements that have been designated as scroll markers via the `scroll-target-group` property.

Syntax

```
:target-after {
  /* ... */
}
```

Examples

Styling navigation items before and after the active scroll marker

In this example, we use the `:target-before` and `:target-after` pseudo-classes to highlight the scroll markers before and after the active one, indicating items the user has already viewed and those that are still to come.

HTML

The markup contains a table of contents created using an ordered list (`<ol>`/`<li>`) and `<a>` elements. This is followed by a series of `<section>` elements containing content.

```
<nav id="toc">
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#ch1">Chapter 1</a></li>
    <li><a href="#ch2">Chapter 2</a></li>
    <li><a href="#ch3">Chapter 3</a></li>
    <li><a href="#ch4">Chapter 4</a></li>
  </ol>
</nav>
<section id="intro" class="chapter">
  <h1>Prose of the century</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</section>
<section id="ch1" class="chapter">
  <h2>Chapter 1</h2>

  <!-- ... -->
</section>
<section id="ch2" class="chapter">
  <h2>Chapter 2</h2>

  <!-- ... -->
</section>

<!-- ... -->
```

```
<nav id="toc">
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#ch1">Chapter 1</a></li>
    <li><a href="#ch2">Chapter 2</a></li>
    <li><a href="#ch3">Chapter 3</a></li>
    <li><a href="#ch4">Chapter 4</a></li>
  </ol>
</nav>
<section id="intro" class="chapter">
  <h1>My story</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</section>
<section id="ch1" class="chapter">
  <h2>Chapter 1</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
  <p>
    Nam vulputate diam nec tempor bibendum. Donec luctus augue eget malesuada
    ultrices. Phasellus turpis est, posuere sit amet dapibus ut, facilisis sed
    est. Nam id risus quis ante semper consectetur eget aliquam lorem. Vivamus
    tristique elit dolor, sed pretium metus suscipit vel. Mauris ultricies
    lectus sed lobortis finibus. Vivamus eu urna eget velit cursus viverra quis
    vestibulum sem. Aliquam tincidunt eget purus in interdum. Cum sociis natoque
    penatibus et magnis dis parturient montes, nascetur ridiculus mus.
  </p>
</section>
<section id="ch2" class="chapter">
  <h2>Chapter 2</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
```

```
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
<section id="ch3" class="chapter">
  <h2>Chapter 3</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
<section id="ch4" class="chapter">
  <h2>Chapter 4</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
```

CSS

We've set `scroll-target-group: auto` on the `<ol>` to turn it into a scroll marker group container and let the browser determine which `<a>` element is the active scroll marker at any given time (that is, which link's target is currently in view). We then style the `:target-current` pseudo-class with a `red` `color` so that it stands out clearly.

```
body {
  font: 1.2em / 1.5 system-ui;
  width: 50%;
  max-width: 700px;
  margin: 0 auto;
}

nav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  display: flex;
  justify-content: center;
}

section {
  padding-top: 60px;
}

a {
  color: black;
  text-decoration: none;
}

a:hover,
a:focus {
  text-decoration: underline;
}

ol {
  display: flex;
  width: 100%;
  justify-content: space-around;
  list-style-type: none;
  padding: 20px 0;
  margin: 0;
  background: white;
}

@supports not selector(:target-after) {
  body::before {
    content: "Your browser does not support the :target-before and :target-after pseudo-classes.";
    color: black;
    background-color: #ffcd33;
    text-align: center;
    padding: 1rem;
    position: fixed;
    bottom: 0;
    left: 0;
    right: 0;
    z-index: 1;
  }
}
```

```
ol {
  scroll-target-group: auto;
}

:target-current {
  color: red;
}
```

Finally, we use the `:target-before` pseudo-class to style all `<a>` elements before the active scroll marker with a gray color and strike-through, to make them look completed/finished, and we use the `:target-after` pseudo-class to style all `<a>` elements after the active scroll marker with a bright underline.

```
a:target-before {
  color: gray;
  text-decoration: line-through;
}

a:target-after {
  text-decoration: underline orange 2px;
}
```

Result

Try navigating either by clicking the links or by scrolling. In both cases, you'll see that the red text color moves between the links to match the section currently in view. The links before and after the current red link also update to use the styles defined in the `a:target-before` and `a:target-after` rules.

See also

- `scroll-marker-group`
- `scroll-target-group`
- `:target-current`, `:target-before`
- `::scroll-marker`
- `::scroll-marker-group`
- Creating CSS carousels
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

:target-before CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:target-before`** CSS pseudo-class selects scroll markers that come *before* the active scroll marker (the one that currently matches `:target-current`) within a scroll marker group. You can use this selector to style navigation items that come before the current navigation position within a scroll marker group.

> **Note:** The `:target-before` pseudo-class is only valid on `::scroll-marker` pseudo-elements and elements that have been designated as scroll markers via the `scroll-target-group` property.

Syntax

```
:target-before {
  /* ... */
}
```

Examples

Styling navigation items before and after the active scroll marker

In this example, we use the `:target-before` and `:target-after` pseudo-classes to highlight the scroll markers before and after the active one, indicating items the user has already viewed and those that are still to come.

HTML

The markup contains a table of contents created using an ordered list (`<ol>`/`<li>`) and `<a>` elements. This is followed by a series of `<section>` elements containing content.

```
<nav id="toc">
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#ch1">Chapter 1</a></li>
    <li><a href="#ch2">Chapter 2</a></li>
    <li><a href="#ch3">Chapter 3</a></li>
    <li><a href="#ch4">Chapter 4</a></li>
  </ol>
</nav>
<section id="intro" class="chapter">
  <h1>Prose of the century</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</section>
<section id="ch1" class="chapter">
  <h2>Chapter 1</h2>

  <!-- ... -->
</section>
<section id="ch2" class="chapter">
  <h2>Chapter 2</h2>

  <!-- ... -->
</section>

<!-- ... -->
```

```
<nav id="toc">
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#ch1">Chapter 1</a></li>
    <li><a href="#ch2">Chapter 2</a></li>
    <li><a href="#ch3">Chapter 3</a></li>
    <li><a href="#ch4">Chapter 4</a></li>
  </ol>
</nav>
<section id="intro" class="chapter">
  <h1>My story</h1>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
</section>
<section id="ch1" class="chapter">
  <h2>Chapter 1</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
  <p>
    Nam vulputate diam nec tempor bibendum. Donec luctus augue eget malesuada
    ultrices. Phasellus turpis est, posuere sit amet dapibus ut, facilisis sed
    est. Nam id risus quis ante semper consectetur eget aliquam lorem. Vivamus
    tristique elit dolor, sed pretium metus suscipit vel. Mauris ultricies
    lectus sed lobortis finibus. Vivamus eu urna eget velit cursus viverra quis
    vestibulum sem. Aliquam tincidunt eget purus in interdum. Cum sociis natoque
    penatibus et magnis dis parturient montes, nascetur ridiculus mus.
  </p>
</section>
<section id="ch2" class="chapter">
  <h2>Chapter 2</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
<section id="ch3" class="chapter">
  <h2>Chapter 3</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
<section id="ch4" class="chapter">
  <h2>Chapter 4</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus
    aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci,
    pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at
    ultricies tellus laoreet sit amet.
  </p>
  <p>
    Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet
    orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare
    ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat.
    Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra
    congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus
    varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
  </p>
</section>
```

CSS

We've set `scroll-target-group: auto` on the `<ol>` to turn it into a scroll marker group container and let the browser determine which `<a>` element is the active scroll marker at any given time (that is, which link's target is currently in view). We then style the `:target-current` pseudo-class with a `red` `color` so that it stands out clearly.

```
body {
  font: 1.2em / 1.5 system-ui;
  width: 50%;
  max-width: 700px;
  margin: 0 auto;
}

nav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  display: flex;
  justify-content: center;
}

section {
  padding-top: 60px;
}

a {
  color: black;
  text-decoration: none;
}

a:hover,
a:focus {
  text-decoration: underline;
}

ol {
  display: flex;
  width: 100%;
  justify-content: space-around;
  list-style-type: none;
  padding: 20px 0;
  margin: 0;
  background: white;
}

@supports not selector(:target-before) {
  body::before {
    content: "Your browser does not support the :target-before and :target-after pseudo-classes.";
    color: black;
    background-color: #ffcd33;
    text-align: center;
    padding: 1rem;
    position: fixed;
    bottom: 0;
    left: 0;
    right: 0;
    z-index: 1;
  }
}
```

```
ol {
  scroll-target-group: auto;
}

:target-current {
  color: red;
}
```

Finally, we use the `:target-before` pseudo-class to style all `<a>` elements before the active scroll marker with a gray color and strike-through, to make them look completed/finished, and we use the `:target-after` pseudo-class to style all `<a>` elements after the active scroll marker with a bright underline.

```
a:target-before {
  color: gray;
  text-decoration: line-through;
}

a:target-after {
  text-decoration: underline orange 2px;
}
```

Result

Try navigating either by clicking the links or by scrolling. In both cases, you'll see that the red text color moves between the links to match the section currently in view. The links before and after the current red link also update to use the styles defined in the `a:target-before` and `a:target-after` rules.

See also

- `scroll-marker-group`
- `scroll-target-group`
- `:target-current`, `:target-after`
- `::scroll-marker`
- `::scroll-marker-group`
- Creating CSS carousels
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

:target-current CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:target-current`** CSS pseudo-class selects the **active** scroll marker — the scroll marker of a `scroll-marker-group` that is currently scrolled to. This selector can be used to style the active navigation position within a scroll marker group.

> **Note:** The `:target-current` pseudo-class is only valid on `::scroll-marker` pseudo-elements and elements that have been designated as scroll markers using the `scroll-target-group` property.

Syntax

```
:target-current {
  /* ... */
}
```

Examples

See Creating CSS carousels and `::scroll-marker` for complete examples that use the `:target-current` pseudo-class.

Basic usage

```
::scroll-marker {
  background-color: white;
}

::scroll-marker:target-current {
  background-color: black;
}
```

See also

- `scroll-marker-group`
- `scroll-target-group`
- `:target-before`, `:target-after`
- `::scroll-marker`
- `::scroll-marker-group`
- Creating CSS carousels
- CSS overflow module
- CSS Carousel Gallery via chrome.dev (2025)

:user-invalid CSS pseudo-class

The **`:user-invalid`** CSS pseudo-class represents any validated form element whose value isn't valid based on their validation constraints, after the user has interacted with it.

The `:user-invalid` pseudo-class must match an `:invalid`, `:out-of-range`, or blank-but `:required` element between the time the user has attempted to submit the form and before the user has interacted again with the form element.

Syntax

```
:user-invalid {
  /* ... */
}
```

Examples

Setting a color and symbol on :user-invalid

In the following example, the red border and ❌ only display once the user has interacted with the field. Try typing something other than an email address to see it in action.

```
<label for="email">Email *: </label>
<input id="email" name="email" type="email" required />
<span></span>
```

```
input:user-invalid {
  border: 2px solid red;
}

input:user-invalid + span::before {
  content: "✖";
  color: red;
}
```

See also

- `:valid`
- `:invalid`
- `:required`
- `:optional`
- `:user-valid`

:user-valid CSS pseudo-class

The **`:user-valid`** CSS pseudo-class represents any validated form element whose value validates correctly based on its validation constraints. However, unlike `:valid` it only matches once the user has interacted with it.

This pseudo-class is applied if the form control is valid and any of the following has occurred:

- The user made a change to the form control and committed the change such as by moving focus elsewhere.
- The user has attempted to submit the form, even if no change was made to the control.
- The value was invalid when it gained focus, and the user made a change making it valid, even if focus is still in the control.

Once this pseudo-class has been applied, the user-agent re-validates whether the control is valid at every keystroke when the control has focus.

- If the control has focus, and the value was invalid when it gained focus, re-validate on every keystroke.

The result is that if the control was valid when the user started interacting with it, the validity styling is changed only when the user shifts focus to another control. However, if the user is trying to correct a previously-flagged value, the control shows immediately when the value becomes valid. Required items are flagged as invalid only if the user changes them or attempts to submit an unchanged invalid value.

Syntax

```
:user-valid {
  /* ... */
}
```

Examples

Setting a color and symbol on :user-valid

In the following example, the green border and ✅ only display once the user has interacted with the field. Try changing the email address to another valid email to see it in action.

```
<label for="email">Email *: </label>
<input id="email" name="email" type="email" value="test@example.com" required />
<span></span>
```

```
input:user-valid {
  border: 2px solid green;
}

input:user-valid + span::before {
  content: "✓";
  color: green;
}
```

See also

- `:valid`
- `:invalid`
- `:required`
- `:optional`
- `:user-invalid`

:valid CSS pseudo-class

The **`:valid`** CSS pseudo-class represents any `<input>` or other `<form>` element whose contents validate successfully. This allows you to easily make valid fields adopt an appearance that helps the user confirm that their data is formatted properly.

Try it

```
label {
  display: block;
  margin-top: 1em;
}

input:valid {
  background-color: ivory;
  border: none;
  outline: 2px solid deepskyblue;
  border-radius: 5px;
  accent-color: gold;
}
```

```
<label for="email">Email Address:</label>
<input id="email" name="email" type="email" value="na@me@example.com" />

<label for="secret">Secret Code: (lower case letters)</label>
<input id="secret" name="secret" type="text" value="test" pattern="[a-z]+" />

<label for="age">Your age: (18+)</label>
<input id="age" name="age" type="number" value="5" min="18" />

<label
  ><input name="tos" type="checkbox" required checked /> - Do you agree to
  ToS?</label
>
```

Syntax

```
:valid {
  /* ... */
}
```

Accessibility

The color green is commonly used to indicate valid input. People who have certain types of color blindness will be unable to determine the input's state unless it is accompanied by an additional indicator that does not rely on color to convey meaning. Typically, descriptive text and/or an icon are used.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.1 | W3C Understanding WCAG 2.0

Examples

Indicating valid and invalid form fields

In this example, we include extra `<span>` elements to generate content that indicates valid or invalid data:

```
<fieldset>
  <legend>Contact information</legend>
  <p>Required fields are labeled with "required".</p>
  <div>
    <label for="fname">First name: </label>
    <input id="fname" name="fname" type="text" required />
    <span></span>
  </div>
  <div>
    <label for="lname">Last name: </label>
    <input id="lname" name="lname" type="text" required />
```

```
    <span></span>
  </div>
  <div>
    <label for="email"> Email address (include if you want a response): </label>
    <input id="email" name="email" type="email" />
    <span></span>
  </div>
</fieldset>
```

To provide these indicators, we use the following CSS:

```
body {
  font-family: sans-serif;
  margin: 20px auto;
  max-width: 460px;
}

fieldset {
  padding: 10px 30px 0;
}

legend {
  color: white;
  background: black;
  padding: 5px 10px;
}

fieldset > div {
  margin-bottom: 20px;
  display: flex;
  flex-flow: row wrap;
}

label,
input {
  display: block;
  font-family: inherit;
  margin: 0;
  box-sizing: border-box;
  width: 100%;
  padding: 5px;
  height: 30px;
}

input {
  box-shadow: inset 1px 1px 3px #cccccc;
  border-radius: 5px;
}

input:hover,
input:focus {
  background-color: #eeeeee;
}

input:required + span::after {
  font-size: 0.7rem;
  position: absolute;
  content: "required";
  color: white;
  background-color: black;
  padding: 5px 10px;
  top: -26px;
  left: -70px;
}
```

```
input + span {
  position: relative;
}

input + span::before {
  position: absolute;
  right: -20px;
  top: 5px;
}

input:invalid {
  border: 2px solid red;
}

input:invalid + span::before {
  content: "✖";
  color: red;
}

input:valid + span::before {
  content: "✓";
  color: green;
}
```

We set the `<span>`s to `position: relative` so that we can position the generated content relative to them. We then absolutely position different generated content depending on whether the form's data is valid or invalid — a green check or a red cross, respectively. To add a bit of extra urgency to the invalid data, we've also given the inputs a thick red border when invalid.

> **Note:** We've used `::before` to add these labels, as we were already using `::after` for the "required" labels.

You can try it below:

Notice how the required text inputs are invalid when empty, but valid when they have something filled in. The email input on the other hand is valid when empty, as it is not required, but invalid when it contains something that is not a proper email address.

See also

- Other validation-related pseudo-classes: `:required`, `:optional`, `:invalid`
- Form data validation
- Accessing the validity state from JavaScript

:visited CSS pseudo-class

The **`:visited`** CSS pseudo-class applies once the link has been visited by the user. For privacy reasons, the styles that can be modified using this selector are very limited. The `:visited` pseudo-class applies only to `<a>` and `<area>` elements that have an `href` attribute.

Styles defined by the `:visited` and unvisited `:link` pseudo-classes can be overridden by any subsequent user-action pseudo-classes (`:hover` or `:active`) that have at least equal specificity. To style links appropriately, put the `:visited` rule after the `:link` rule but before the `:hover` and `:active` rules, as defined by the *LVHA-order*: `:link` — `:visited` — `:hover` — `:active`. The `:visited` pseudo-class and `:link` pseudo-class are mutually exclusive.

Try it

```
p {
  font-weight: bold;
}

a:visited {
  color: forestgreen;
  text-decoration-color: hotpink;
}
```

```
<p>Pages that you might have visited:</p>
<ul>
  <li>
    <a href="https://developer.mozilla.org">MDN Web Docs</a>
  </li>
  <li>
    <a href="https://www.youtube.com/">YouTube</a>
  </li>
</ul>
<p>Pages unlikely to be in your history:</p>
<ul>
  <li>
    <a href="https://developer.mozilla.org/missing-1">Random MDN page</a>
  </li>
  <li>
    <a href="https://example.com/missing-1">Random Example page</a>
  </li>
</ul>
```

Privacy restrictions

For privacy reasons, browsers strictly limit which styles you can apply using this pseudo-class, and how they can be used:

- Allowable CSS properties are `color`, `background-color`, `border-color`, `border-bottom-color`, `border-left-color`, `border-right-color`, `border-top-color`, `column-rule-color`, `outline-color`, `text-decoration-color`, and `text-emphasis-color`.
- Allowable SVG attributes are `fill` and `stroke`.
- The alpha component of the allowed styles will be ignored. The alpha component of the element's non-`:visited` state will be used instead. In Firefox when the alpha component is `0`, the style set in `:visited` will be ignored entirely.
- Although these styles can change the appearance of colors to the end user, the `window.getComputedStyle` method will lie and always return the value of the non-`:visited` color.
- The `<link>` element is never matched by `:visited`.
- DOM methods that match elements via CSS selectors — such as `querySelector()` and `querySelectorAll()` — will always return an "empty" result even if there are visited links in a document. For the aforementioned methods, this will be `null` or an empty `NodeList`, respectively.

> **Note:** For more information on these limitations and the reasons behind them, see Privacy and the :visited selector.

Syntax

```
:visited {
  /* ... */
}
```

Examples

Properties that would otherwise have no color or be transparent cannot be modified with `:visited`. Of the properties that can be set with this pseudo-class, your browser probably has a default value for `color` and `column-rule-color` only. Thus, if you want to modify the other properties, you'll need to give them a base value outside the `:visited` selector.

HTML

```
<a href="#test-visited-link">Have you visited this link yet?</a><br />
<a href="">You've already visited this link.</a>
```

CSS

```
a {
  /* Specify non-transparent defaults to certain properties,
     allowing them to be styled with the :visited state */
  background-color: white;
  border: 1px solid white;
}

a:visited {
  background-color: yellow;
  border-color: hotpink;
  color: hotpink;
}
```

Result

See also

- Privacy and the :visited selector
- Link-related pseudo-classes: `:link`, `:active`, `:hover`

:volume-locked CSS pseudo-class

The **`:volume-locked`** CSS pseudo-class selector represents an element that is capable of making sound, such as `<audio>` or `<video>`, but the audio volume of the media element is currently "locked" by the user.

User agents may set media `muted` or `volume` values according to user preferences (e.g., remembering the last set value across sessions, on a per-site basis, or otherwise). An element that is `:volume-locked` cannot be muted, un-muted, or have its volume changed via JavaScript. The locked status is an operating system or user agent preference.

Syntax

```
:volume-locked {
  /* ... */
}
```

Examples

CSS

```
:volume-locked {
  border: 5px solid green;
}

video:volume-locked {
  border: 5px solid aqua;
}
```

See also

- `:buffering`
- `:muted`
- `:paused`
- `:playing`
- `:seeking`
- `:stalled`
- CSS selectors
- `volume` property of `HTMLMediaElement` objects

:where() CSS pseudo-class

The **`:where()`** CSS pseudo-class function takes a selector list as its argument, and selects any element that can be selected by one of the selectors in that list.

The difference between `:where()` and `:is()` is that `:where()` always has 0 specificity, whereas `:is()` takes on the specificity of the most specific selector in its arguments.

Try it

```
ol {
  list-style-type: upper-alpha;
  color: darkblue;
}

/* Not applied to ol, because of lower specificity */
:where(ol, ul, menu:unsupported) :where(ol, ul) {
  color: green;
}

:where(ol, ul) :where(ol, ul) ol {
  list-style-type: lower-greek;
  color: chocolate;
}
```

```
<ol>
  <li>Saturn</li>
  <li>
    <ul>
      <li>Mimas</li>
      <li>Enceladus</li>
      <li>
        <ol>
          <li>Voyager</li>
          <li>Cassini</li>
        </ol>
      </li>
      <li>Tethys</li>
    </ul>
  </li>
  <li>Uranus</li>
  <li>
    <ol>
      <li>Titania</li>
      <li>Oberon</li>
    </ol>
  </li>
</ol>
```

Syntax

```
:where(<complex-selector-list>) {
  /* ... */
}
```

Parameters

The `:where()` pseudo-class requires a selector list, a comma-separated list of one or more selectors, as its argument. The list must not contain a pseudo-element, but any other simple, compound, and complex selectors are allowed.

Forgiving Selector Parsing

The specification defines `:is()` and `:where()` as accepting a forgiving selector list.

In CSS when using a selector list, if any of the selectors are invalid then the whole list is deemed invalid. When using `:is()` or `:where()` instead of the whole list of selectors being deemed invalid if one fails to parse, the incorrect or unsupported selector will be ignored and the others used.

```
:where(:valid, :unsupported) {
  /* … */
}
```

Will still parse correctly and match `:valid` even in browsers which don't support `:unsupported`, whereas:

```
:valid,
:unsupported {
  /* … */
}
```

Will be ignored in browsers which don't support `:unsupported` even if they support `:valid`.

Examples

Comparing :where() and :is()

This example shows how `:where()` works, and also illustrates the difference between `:where()` and `:is()`.

Take the following HTML:

```
<article>
  <h2>:is()-styled links</h2>
  <section class="is-styling">
    <p>
      Here is my main content. This
      <a href="https://mozilla.org">contains a link</a>.
    </p>
  </section>

  <aside class="is-styling">
    <p>
      Here is my aside content. This
      <a href="https://developer.mozilla.org">also contains a link</a>.
    </p>
  </aside>

  <footer class="is-styling">
    <p>
      This is my footer, also containing
      <a href="https://github.com/mdn">a link</a>.
    </p>
  </footer>
</article>

<article>
  <h2>:where()-styled links</h2>
  <section class="where-styling">
    <p>
      Here is my main content. This
      <a href="https://mozilla.org">contains a link</a>.
    </p>
  </section>

  <aside class="where-styling">
    <p>
      Here is my aside content. This
      <a href="https://developer.mozilla.org">also contains a link</a>.
    </p>
  </aside>

  <footer class="where-styling">
    <p>
      This is my footer, also containing
      <a href="https://github.com/mdn">a link</a>.
    </p>
  </footer>
</article>
```

In this somewhat-contrived example, we have two articles that each contain a section, an aside, and a footer. They differ by the classes used to mark the child elements.

To group the selection of links, while keeping the `is-styling` and `where-styling` styles distinct, we *could* use `:is()` or `:where()`, in the following manner:

```
html {
  font-family: sans-serif;
  font-size: 150%;
}

:is(section.is-styling, aside.is-styling, footer.is-styling) a {
  color: red;
}

:where(section.where-styling, aside.where-styling, footer.where-styling) a {
  color: orange;
}
```

However, what if we later want to override the color of links in the footers using a compound selector made up of low-specificity type selectors?

```
footer a {
  color: blue;
}
```

This won't work for the red links, because the selectors inside `:is()` count towards the specificity of the overall selector, and class selectors have a higher specificity than element selectors.

However, selectors inside `:where()` have specificity 0, so the orange footer link will be overridden by our type-only compound selector.

> **Note:** You can also find this example on GitHub; see is-where.

See also

- `:is()`
- Selector list
- Web components
- Forgiving selector list via csswg.org.

:xr-overlay CSS pseudo-class

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`:xr-overlay`** CSS pseudo-class, defined in the WebXR DOM overlays module, matches the DOM overlay element when a web page is being viewed in an immersive AR or VR environment.

Syntax

```
:xr-overlay {
  /* ... */
}
```

Description

The `:xr-overlay` pseudo-class matches the overlay element for the duration of an immersive session using a DOM overlay.

The overlay element is a backdrop root. Any `backdrop-filter` effects on the DOM overlay element or its descendants do not modify the AR camera image (if applicable) or the rendered content drawn to the immersive session's `XRWebGLLayer`.

The overlay element itself is a stacking context due to its fixed `position`. The stacking contexts for ancestors of the overlay element, if any, do not paint to the immersive session's display.

> **Note:** On a multi-display system, the stacking contexts for ancestors or sibling trees of the overlay element may be displayed on separate displays.

Examples

Basic usage

In this example, we define the overlay to be semi-transparent black, enabling content behind the overlay to be partially visible. To ensure enough contrast between content and its background, we set the `color` to `white`.

```
:xr-overlay {
  background-color: rgb(0 0 0 / 0.5);
  color: white;
}
```

Targeting an overlay's contents

In this example, we use the `:xr-overlay` selector to style the `<button>` elements within an XR DOM overlay.

```
:xr-overlay button {
  background-color: white;
  color: black;
}
```

See also

- `:modal`
- `:fullscreen`
- `:picture-in-picture`
- CSS pseudo-classes
- WebXR DOM overlays module
- WebXR device API fundamentals
- `XRSession.domOverlayState`

Attribute selectors

The CSS **attribute selector** matches elements based on the element having a given attribute explicitly set, with options for defining an attribute value or substring value match.

Syntax

```
/* <a> elements with a title attribute */
a[title] {
  color: purple;
}

/* <a> elements with an href matching "https://example.org" */
a[href="https://example.org"] {
  color: green;
}

/* <a> elements with an href containing "example" */
a[href*="example"] {
  font-size: 2em;
}

/* <a> elements with an href ending ".org", case-insensitive */
a[href$=".org" i] {
  font-style: italic;
```

```
}

/* <a> elements whose class attribute contains the word "logo" */
a[class~="logo"] {
  padding: 2px;
}
```

`[attr]`

Represents elements with an attribute name of *attr*.

`[attr=value]`

Represents elements with an attribute name of *attr* whose value is exactly *value*.

`[attr~=value]`

Represents elements with an attribute name of *attr* whose value is a whitespace-separated list of words, one of which is exactly *value*.

`[attr|=value]`

Represents elements with an attribute name of *attr* whose value can be exactly *value* or can begin with *value* immediately followed by a hyphen, - (U+002D). It is often used for language subcode matches.

`[attr^=value]`

Represents elements with an attribute name of *attr* whose value is prefixed (preceded) by *value*.

`[attr$=value]`

Represents elements with an attribute name of *attr* whose value is suffixed (followed) by *value*.

`[attr*=value]`

Represents elements with an attribute name of *attr* whose value contains at least one occurrence of *value* within the string.

`[attr operator value i]`

Adding an `i` (or `I`) before the closing bracket causes the value to be compared case-insensitively (for characters within the ASCII range).

`[attr operator value s]`

Adding an `s` (or `S`) before the closing bracket causes the value to be compared case-sensitively (for characters within the ASCII range).

Values

`<attr>`

An `<ident>`, that is, the unquoted name of the attribute. This can be any valid language-specific attribute (SVG, HTML, XML, etc.), a `data-*` attribute, or an author-created attribute.

`<value>`

An `<ident>` or `<string>`, representing the attribute value. The value must be quoted if it contains spaces or special characters.

`s` or `i`

Case sensitivity or insensitivity flag. If included before the closing bracket (`]`), makes the value case sensitive or insensitive, irrespective of the markup language.

Description

The case sensitivity of attribute names and values depends on the document language. In HTML, attribute names are case-insensitive, as are spec-defined enumerated values. The case-insensitive HTML attribute values are listed in the HTML spec. For these attributes, the attribute value in the selector is case-insensitive, regardless of whether the value is invalid or the attribute for the element on which it is set is invalid.

If the attribute value is case-sensitive, like `class`, `id`, and `data-*` attributes, the attribute selector value match is case-sensitive. Attributes defined outside of the HTML specification, like `role` and `aria-*` attributes, are also case-sensitive. Case-sensitive attribute selectors can be made case-insensitive with the inclusion of the case-insensitive modifier (`i`).

Examples

Links

CSS

```
a {
  color: blue;
}

/* Internal links, beginning with "#" */
a[href^="#"] {
  background-color: gold;
}

/* Links with "example" anywhere in the URL */
a[href*="example"] {
  background-color: silver;
}

/* Links with "insensitive" anywhere in the URL,
   regardless of capitalization */
a[href*="insensitive" i] {
  color: cyan;
}

/* Links with "cAsE" anywhere in the URL,
with matching capitalization */
a[href*="cAsE" s] {
  color: pink;
}

/* Links that end in ".org" */
a[href$=".org"] {
  color: red;
}

/* Links that start with "https://" and end in ".org" */
```

```
a[href^="https://"][href$=".org"] {
  color: green;
}
```

HTML

```
<ul>
  <li><a href="#internal">Internal link</a></li>
  <li><a href="http://example.com">Example link</a></li>
  <li><a href="#InSensitive">Insensitive internal link</a></li>
  <li><a href="http://example.org">Example org link</a></li>
  <li><a href="https://example.org">Example https org link</a></li>
</ul>
```

Result

Languages

CSS

```
/* All divs with a `lang` attribute are bold. */
div[lang] {
  font-weight: bold;
}

/* All divs without a `lang` attribute are italicized. */
div:not([lang]) {
  font-style: italic;
}

/* All divs in US English are blue. */
div[lang~="en-us"] {
  color: blue;
}

/* All divs in Portuguese are green. */
div[lang="pt"] {
  color: green;
}

/* All divs in Chinese are red, whether
   simplified (zh-Hans-CN) or traditional (zh-Hant-TW). */
div[lang|="zh"] {
  color: red;
}

/* All divs with a Traditional Chinese
   `data-lang` are purple. */
/* Note: You could also use hyphenated attributes
   without double quotes */
div[data-lang="zh-Hant-TW"] {
  color: purple;
}
```

HTML

```
<div lang="en-us en-gb en-au en-nz">Hello World!</div>
<div lang="pt">Olá Mundo!</div>
<div lang="zh-Hans-CN">世界您好! </div>
<div lang="zh-Hant-TW">世界您好! </div>
<div data-lang="zh-Hant-TW">世界您好! </div>
```

Result

HTML ordered lists

The HTML specification requires the `type` attribute to be matched case-insensitively because it is primarily used in the `<input>` element. Note that if a modifier is not supported by the user agent, then the selector will not match.

CSS

```
/* Case-sensitivity depends on document language */
ol[type="a"]:first-child {
  list-style-type: lower-alpha;
  background: red;
}

ol[type="i" s] {
  list-style-type: lower-alpha;
  background: lime;
}

ol[type="I" s] {
  list-style-type: upper-alpha;
  background: grey;
}

ol[type="a" i] {
  list-style-type: upper-alpha;
  background: green;
}
```

HTML

```
<ol type="A">
  <li>
    Red background for case-insensitive matching (default for the type selector)
  </li>
</ol>
<ol type="i">
  <li>Lime background if `s` modifier is supported (case-sensitive match)</li>
```

```
</ol>
<ol type="I">
  <li>Grey background if `s` modifier is supported (case-sensitive match)</li>
</ol>
<ol type="A">
  <li>
    Green background if `i` modifier is supported (case-insensitive match)
  </li>
</ol>
```

Result

See also

- `attr()`
- Selecting a single element: `Document.querySelector()`, `DocumentFragment.querySelector()`, or `Element.querySelector()`
- Selecting all matching elements: `Document.querySelectorAll()`, `DocumentFragment.querySelectorAll()`, or `Element.querySelectorAll()`
- Case-insensitive attribute selector values on WHATWG

Child combinator

The **child combinator** (`>`) is placed between two CSS selectors. It matches only those elements matched by the second selector that are the direct children of elements matched by the first.

```
/* List items that are children of the "my-things" list */
ul.my-things > li {
  margin: 2em;
}
```

Elements matched by the second selector must be the immediate children of the elements matched by the first selector. This is stricter than the descendant combinator, which matches all elements matched by the second selector for which there exists an ancestor element matched by the first selector, regardless of the number of "hops" up the DOM.

Syntax

```
/* The white space around the > combinator is optional but recommended. */
selector1 > selector2 { /* style properties */ }
```

Examples

CSS

```
span {
  background-color: aqua;
}

div > span {
  background-color: yellow;
}
```

HTML

```
<div>
  <span>
    Span #1, in the div.
    <span>Span #2, in the span that's in the div.</span>
  </span>
</div>
<span>Span #3, not in the div at all.</span>
```

Result

See also

- Descendant combinator

Class selectors

The CSS **class selector** matches elements based on the contents of their `class` attribute.

```
/* All elements with class="spacious" */
.spacious {
  margin: 2em;
}

/* All <li> elements with class="spacious" */
li.spacious {
  margin: 2em;
}

/* All <li> elements with a class list that includes both "spacious" and "elegant" */
/* For example, class="elegant retro spacious" */
li.spacious.elegant {
  margin: 2em;
}
```

Syntax

```
.class_name {
  /* … */
}
```

Note that this is equivalent to the following attribute selector:

```
[class~="class_name"] {
  /* … */
}
```

The `class_name` value must be a valid CSS identifier. HTML `class` attributes which are not valid CSS identifiers must be escaped before they can be used in class selectors.

Examples

Valid class selectors

HTML

```
<p class="red">This paragraph has red text.</p>
<p class="red yellow-bg">
  This paragraph has red text and a yellow background.
</p>
<p class="red fancy">This paragraph has red text and "fancy" styling.</p>
<p>This is just a regular paragraph.</p>
```

```
<!-- The next two paragraphs have class attributes
that contain characters which must be escaped in CSS -->

<p class="item?one">This paragraph has a pink background.</p>
<p class="123item">This paragraph has a yellow background.</p>
```

CSS

```
.red {
  color: #ff3333;
}

.yellow-bg {
  background: #ffffaa;
}

.fancy {
  font-weight: bold;
  text-shadow: 4px 4px 3px #7777ff;
}
```

```
/* In the next two rules, the class attributes must be escaped */

.item\?one {
  background-color: pink;
}

.\00003123item {
  background-color: yellow;
}
```

Result

Invalid class selectors

The class selectors in the following rules are not valid CSS identifiers, and will be ignored.

```
.item?one {
  background-color: green;
}

.123item {
  background-color: green;
}
```

See also

- CSS Selectors
- Learn CSS: Basic selectors

Column combinator

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **column combinator** (||) is placed between two CSS selectors. It matches only those elements matched by the second selector that belong to the column elements matched by the first.

```
/* Table cells that belong to the "selected" column */
col.selected||td {
  background: gray;
}
```

Syntax

```
/* The white space around the || combinator is optional but recommended. */
column-selector || cell-selector {
  /* style properties */
}
```

Examples

HTML

```
<table border="1">
  <colgroup>
    <col span="2" />
    <col class="selected" />
  </colgroup>
  <tbody>
    <tr>
      <td>A</td>
      <td>B</td>
      <td>C</td>
    </tr>
```

```
    <tr>
      <td colspan="2">D</td>
      <td>E</td>
    </tr>
    <tr>
      <td>F</td>
      <td colspan="2">G</td>
    </tr>
  </tbody>
</table>
```

CSS

```
col.selected||td {
  background: gray;
  color: white;
  font-weight: bold;
}
```

Result

Browser compatibility

Currently, no browsers support this feature.

See also

- `<col>`
- `<colgroup>`
- `grid`
- `:nth-of-type`
- `:nth-last-of-type`
- CSS selectors module

CSS combinators

CSS **combinators** define relationships between selectors. They allow you to select elements based on their relationship to other elements in the document tree.

For example, to style only paragraph elements that are direct children of a `<div>`, you can use the child combinator (`>`):

```
/* Set top margin on <p> elements that are direct children of <div> */
div > p {
  margin-top: 0;
}
```

Syntax

```
/* Combine selectors to express relationships */
selector1 combinator selector2 {
  property: value;
}
```

Index of combinators

- Child combinator (`>`)
- Column combinator (`||`)
- Descendant combinator (" ")
- Next-sibling combinator (`+`)
- Subsequent-sibling combinator (`~`)

See also

- CSS selectors module
- Selectors and combinators

Descendant combinator

The **descendant combinator** — typically represented by a single space (" ") character — combines two selectors such that elements matched by the second selector are selected if they have an ancestor (parent, parent's parent, parent's parent's parent, etc.) element matching the first selector. Selectors that utilize a descendant combinator are called *descendant selectors*.

```
/* List items that are descendants of the "my-things" list */
ul.my-things li {
  margin: 2em;
}
```

The descendant combinator is technically one or more CSS white space characters — the space character and/or one of four control characters: carriage return, form feed, new line, and tab characters — between two selectors in the absence of another combinator. Additionally, the white space characters of which the combinator is comprised may contain any number of CSS comments.

Syntax

```
selector1 selector2 {
  /* property declarations */
}
```

Examples

CSS

```
li {
  list-style-type: disc;
}

li li {
  list-style-type: circle;
}
```

HTML

```
<ul>
  <li>
    <div>Item 1</div>
    <ul>
      <li>Subitem A</li>
      <li>Subitem B</li>
    </ul>
  </li>
  <li>
    <div>Item 2</div>
    <ul>
      <li>Subitem A</li>
      <li>Subitem B</li>
    </ul>
  </li>
</ul>
```

Result

See also

- Child combinator

ID selectors

The CSS **ID selector** matches an element based on the value of the element's `id` attribute. In order for the element to be selected, its `id` attribute must match exactly the value given in the selector.

```
/* The element with id="demo" */
#demo {
  border: red 2px solid;
}
```

Syntax

```
#id_value {
  /* … */
}
```

Note that syntactically (but not specificity-wise), this is equivalent to the following attribute selector:

```
[id="id_value"] {
  /* … */
}
```

The `id_value` value must be a valid CSS identifier. HTML `id` attributes which are not valid CSS identifiers must be escaped before they can be used in ID selectors.

Examples

Valid ID selectors

HTML

```
<p id="blue">This paragraph has a blue background.</p>
<p>This is just a regular paragraph.</p>
```

```
<!-- The next two paragraphs have id attributes
that contain characters which must be escaped in CSS -->

<p id="item?one">This paragraph has a pink background.</p>
<p id="123item">This paragraph has a yellow background.</p>
```

CSS

```
#blue {
  background-color: skyblue;
}
```

```
/* In the next two rules, the id attributes must be escaped */

#item\?one {
  background-color: pink;
}

#\00003123item {
  background-color: yellow;
}
```

Result

Invalid ID selectors

The ID selectors in the following rules are not valid CSS identifiers, and will be ignored.

```
#item?one {
  background-color: green;
}

#123item {
  background-color: green;
}
```

See also

- CSS Selectors
- Learn CSS: Basic selectors

Keyframe selectors

CSS **keyframe selectors** identify specific points in an animation timeline where keyframe styles should be applied. These selectors are used exclusively within the `@keyframes` at-rule.

Syntax

```
/* Keywords */
from {
}
to {
}

/* <percentage> values */
0% {
}
50% {
}
100% {
}

/* With <timeline-range-name> */
entry 20% {
}
exit 80% {
}

/* Selector lists */
0%,
50%,
100% {
}
from,
to {
}
```

Values

`from`

Represents the start of the animation sequence. It is equivalent to `0%`.

`to`

Represents the end of the animation sequence. It is equivalent to `100%`.

`<percentage>`

A `<percentage>` between `0%` and `100%`, inclusive, representing the total progression through the animation sequence.

`<timeline-range-name> <percentage>`

A `<timeline-range-name>` preceding a `<percentage>` component represents a specific progress point within the named timeline range.

Description

A `<keyframe-selector>` can be the `to` or `from` keyword, a percentage between `0%` and `100%`, inclusive, or a comma-separated list of these keywords and/or percentages. When the percentage is preceded by a `<timeline-range-name>`, it defines a timeline range if the animation timeline is a view progress timeline; otherwise, the selector is ignored. When a comma-separated `<keyframe-selector>` list is used, the style block that follows applies to all the specified progression points.

Valid percentage values

Percentage values must include the percent sign (`%`). Unitless values (such as `0` and `20`) and values outside the range `0%` to `100%` (such as `-10%` or `110%`) are invalid and will cause the keyframe block to be ignored.

Cascade, order, precedence, and importance

Property values set by animation `@keyframes` are more important than all normal styles, meaning no matter the specificity, an animated value will override values that do not have an `!important` flag set. Only important property values and values currently being transitioned take precedence over animated property values.

All keyframe selectors have the same specificity. The order of keyframe selectors within the selector list does not matter. The source order only matters if a duplicate keyframe selector declares a different value for an already declared property on the same selector.

The `!important` flag is not valid within an `@keyframes` definition.

Omitted start and end selectors

If no `0%` (or `from`) or `100%` (or `to`) keyframe is specified, the browser will use the element's computed styles for those states, allowing the animation to transition smoothly from or to the element's non-animated property values. In other words, if a property is specified in a middle-of-the-timeline keyframe without being set within a starting or ending keyframe selector block, the property will animate from its original value to that value.

For example, if an element has a `red` background color, and the following animation is applied:

```
@keyframes changeToPurple {
  25%,
  75% {
    background-color: purple;
  }
}
```

The background color will be `red` at the start of the animation, transitioning to `purple` a quarter of the way through the animation, remaining `purple` for half the animation, and transitioning back to `red`, the original background color, starting `75%` of the way through the animation timeline. See the example that omits the `to` and `from`.

Omitted property declarations

When creating an `@keyframes` animation to animate multiple properties, you don't have to declare all the properties in all the keyframe selector blocks.

For example, in the basic usage example, the `opacity` is declared in all the keyframe selector blocks, but only start and end `transform` values are set. In that case, the element will be fully opaque `50%` of the way through the animation timeline, but the point at which the element is transformed `25vw` to the right depends on the `animation-timing-function` — it will be `50%` in the case of `linear`, but not if `ease-in` is used (which it is in this case).

Cascade order

If multiple keyframe blocks use the same `<keyframe-selector>`, they cascade. This means that if the same property is defined in multiple blocks with the same selector, the last declaration in the rule takes precedence. If they define different properties, they are merged.

This animation repeats non-changing values in multiple selector blocks, which is not necessary:

```
@keyframes uglyAnimation {
  0% {
    transform: translateX(0);
    opacity: 0;
    background-color: purple;
  }
  50% {
    transform: translateX(0);
    opacity: 1;
    background-color: purple;
  }
  75% {
    transform: translateX(0);
    opacity: 0;
    background-color: green;
  }
  100% {
    transform: translateX(50vw);
    opacity: 0;
    background-color: purple;
  }
}
```

We can use the cascade to group values in one selector block, then override them as necessary. The following is equivalent to the previous animation, but with fewer lines of CSS:

```
@keyframes uglyAnimation {
  0%,
  50%,
  75%,
  100% {
    transform: translateX(0);
    opacity: 0;
    background-color: purple;
  }
  50% {
    opacity: 1;
  }
  75% {
    background-color: green;
  }
  100% {
    transform: translateX(50vw);
  }
}
```

Order is important! We can't ignore the cascade. If we reorder the above incorrectly, we lose the transitions. The following makes the element purple, fully opaque, and resets all the transforms during the entire time the animation is applied. The property values are applied and remain static because the last declaration overrides the declarations made in the previous keyframe selector blocks.

```
@keyframes makeItPurpleOnly {
  0% {
    background-color: yellow;
  }
  50% {
    opacity: 0;
  }
  75% {
    background-color: green;
  }
  100% {
    transform: translateX(50vw);
  }
  0%,
  50%,
  75%,
  100% {
    transform: translateX(0);
    opacity: 1;
    background-color: purple;
  }
}
```

With named timeline ranges

Originally defined in the CSS animation module, the CSS scroll-driven animations module expanded the keyframe selector to enable view progress timeline range information to be included directly in the `@keyframes` animation definition. A `<timeline-range-name>` can precede the `<percentage>` component of the selector to attach keyframes to specific progress points within the named timeline range. The `<timeline-range-name>` represents the selected predefined named timeline range, and the `<percentage>` after it represents the percentage progress between the start and end of that named timeline range.

```
@keyframes in-and-out {
  entry 0% {
    opacity: 0;
    transform: translateX(100%);
  }
  entry 100% {
    opacity: 1;
    transform: translateX(0);
  }
  exit 0% {
    opacity: 1;
    transform: translateX(0);
  }
  exit 100% {
    opacity: 0;
    transform: translateX(-100%);
```

```
  }
}
```

If the element's animation timeline does not have a corresponding named view timeline range, then any keyframes attached to points on that named timeline range are ignored. These attachment points may be outside the active interval of the animation. When this occurs, the automatic `from` (`0%`) and `to` (`100%`) keyframes are only generated for properties that don't have keyframes at or earlier than `0%` or at or after `100%`, respectively. See the timeline range name guide for more information.

Examples

Basic usage

This example demonstrates using keywords and percentages by creating a keyframe animation that uses these keyframe selector types.

HTML

We include an element that we will animate.

```
<div>I am animated</div>
```

CSS

We provide some basic styling to our box:

```
div {
  background-color: rebeccapurple;
  color: white;
  width: min-content;
  padding: 10px;
  font: 2rem sans-serif;
}
```

We create an `@keyframes` animation, applying styles to the `from` and `to` keywords and a middle percentage. We animate the `opacity` and the `transform` properties.

```
@keyframes slide-and-fade {
  from {
    transform: translateX(0);
    opacity: 0;
  }
  50% {
    opacity: 1;
  }
  to {
    transform: translateX(50vw);
    opacity: 0;
  }
}
```

We apply our animation to the element using the `animation` shorthand property:

```
div {
  animation: slide-and-fade 4s ease-in infinite;
}
```

Results

Selector lists

Using the same HTML and basic styling as in the previous example, this example demonstrates the use of comma-separated selectors to group keyframes, applying the same styles at multiple points in an animation.

```
<div>I am animated</div>
```

CSS

We create a `pulse` animation that changes the size of our element.

```
div {
  animation: pulse 4s linear infinite;
}

@keyframes pulse {
  0%,
  75% {
    transform: scale(1);
  }
  50% {
    transform: scale(1.2);
  }
  25%,
  100% {
    transform: scale(0.8);
  }
}
```

```
div {
  background-color: rebeccapurple;
  color: white;
  width: min-content;
  padding: 10px;
  font: 2rem sans-serif;
}
```

Result

By using a selector list with multiple comma-separated keyframe selectors, the animation "pauses" from the `25%` keyframe until it reaches the `75%` keyframe.

Omitting <code>to</code> and <code>from</code>

This example demonstrates how, when the `to` or `from` keyframe selectors are not included in a `@keyframes` animation definition, the animated properties will animate to and from the original, non-animated property values.

HTML

We include a few elements. We will be animating all of them.

```
<div>I am animated</div>
<div>I am animated</div>
<div>I am animated</div>
```

CSS

We provide basic styles to our elements, and give each one a different `outline-width` and `background-color`. We will animate these two properties.

```
div {
  background-color: magenta;
  outline: 10px dashed black;
  color: white;
  width: min-content;
  padding: 10px;
  font: 2rem sans-serif;
  margin: 35px auto;

  animation: changes 5s linear infinite;
}
div:first-of-type {
  background-color: blue;
  outline-width: 0;
}
div:last-of-type {
  background-color: green;
  outline-width: 20px;
}
```

We create an animation that sets an element's `background-color` and `outline-width` at `30%` and `40%` keyframes.

```
@keyframes changes {
  30%, 40% {
    background-color: black;
    outline-width: 15px;
}
```

Result

The `background-color` and `outline-width` properties are set in `30%` and `40%` keyframes. As a result, the elements' `background-color` values animate from `green`, `magenta`, and `blue` to `black` while their `outline-width` values animate from `0px`, `10px`, and `20px` to `15px`. They remain in that state for one-tenth of the animation — between `30%` and `40%` duration. After `40%` duration, these properties animate back to their initial values again.

See also

- `@keyframes`
- `animation`
- `animation-range`
- Understanding timeline range names
- Using CSS animations
- CSS animations module
- CSS scroll-driven animations

Namespace separator

The **namespace separator** (`|`) separates the selector from the namespace, identifying the namespace, or lack thereof, for a type selector.

```
/* Links in the namespace named myNameSpace */
myNameSpace|a {
  font-weight: bold;
}
/* paragraphs in any namespace (including no namespace) */
*|p {
  color: darkblue;
}
/* heading level 2 not in a namespace */
|h2 {
  margin-bottom: 0;
}
```

Type selectors and the universal selector allow an optional namespace component. When a namespace has been previously declared via `@namespace`, these selectors can be namespaced by prepending the selector with the name of the namespace separated by the namespace separator (`|`). This is useful when dealing with documents containing multiple namespaces such as HTML with inline SVG or MathML, or XML that mixes multiple vocabularies.

- `ns|h1` - matches `<h1>` elements in namespace `ns`
- `*|h1` - matches all `<h1>` elements
- `|h1` - matches all `<h1>` elements outside of any declared or implied namespace

Syntax

```
namespace|element { style properties }
```

Examples

By default, all elements in an HTML or SVG element have a namespace as the `http://www.w3.org/1999/xhtml` and `http://www.w3.org/2000/svg` namespace are implied. The `document.createElementNS` method, with an empty string for the namespace parameter, can be used to create elements with no namespace.

Named namespace example

In this example, all elements are in a namespace.

HTML

No namespaces are explicitly declared in the HTML or within the SVG.

```
<p>This paragraph <a href="#">has a link</a>.</p>

<svg width="400" viewBox="0 0 400 20">
  <a href="#">
    <text x="0" y="15">Link created in SVG</text>
  </a>
</svg>
```

CSS

The CSS declares two namespaces, then assigns styles to links globally (`a`), to links in no namespace (`|a`), to links in any namespace or no namespace (`*|a`), and then to two different named namespaces (`svgNamespace|a` and `htmlNameSpace|a`).

```
@namespace svgNamespace url("http://www.w3.org/2000/svg");
@namespace htmlNameSpace url("http://www.w3.org/1999/xhtml");
/* All `<a>`s in the default namespace, in this case, all `<a>`s */
a {
  font-size: 1.4rem;
}
/* no namespace */
|a {
  text-decoration: wavy overline lime;
  font-weight: bold;
}
/* all namespaces (including no namespace) */
*|a {
  color: red;
  fill: red;
  font-style: italic;
}
/* only the svgNamespace namespace, which is <svg> content */
svgNamespace|a {
  color: green;
  fill: green;
}
/* The htmlNameSpace namespace, which is the HTML document */
htmlNameSpace|a {
  text-decoration-line: line-through;
}
```

Result

The selector `|a`, a link not in a namespace, doesn't match any links. In HTML, the `http://www.w3.org/1999/xhtml` is implied, meaning all HTML is in a namespace, even if none is explicitly declared. In SVG, even if not explicitly set, the `http://www.w3.org/2000/svg` namespace is also implied. This means all the content is within at least one namespace.

Default namespace and no namespace

In this example, we use JavaScript to create an element without a namespace and append it to the document. We set the SVG namespace to be the default namespace by defining the unnamed namespace with `@namespace`.

> **Note:** If a default, or unnamed, namespace is defined, universal and type selectors apply only to elements in that namespace.

HTML

No namespaces are explicitly declared in the HTML or within the SVG.

```
<p><a href="#">A link</a> in the implied HTML namespace.</p>

<svg width="400" viewBox="0 0 400 20">
  <a href="#">
    <text x="0" y="15">Link created in SVG namespace</text>
  </a>
</svg>
```

JavaScript

With JavaScript, with `document.createElementNS`, we create an anchor link without a namespace, then append the link.

```
// create 'no namespace' anchor
const a = document.createElementNS("", "a");
a.href = "#";
a.appendChild(document.createTextNode("Link created without a namespace"));

document.body.appendChild(a);
```

CSS

We declare a namespace with `@namespace`. By omitting the name for the namespace, the `@namespace` declaration creates a default namespace.

```
/* By omitting a name, this sets SVG as the default namespace */
@namespace url("http://www.w3.org/2000/svg");

/* `<a>` in the default (set to SVG) namespace */
a {
  font-size: 1.4rem;
}

/* `<svg>` and `<p>` in the default (set to SVG) namespace */
svg,
p {
```

```
  border: 5px solid gold;
}

/* links outside of any namespace */
|a {
  text-decoration: wavy underline purple;
  font-weight: bold;
}

/* links with any namespace or no namespace */
*|a {
  font-style: italic;
  color: magenta;
  fill: magenta;
}
```

Result

The selector with no namespace separator, the `a`, matched only the SVG `<a>` elements, as SVG was set as the default namespace.

The selector with no namespace, the `|a`, matched the JavaScript defined and appended `<a>`, as that node is the only node that doesn't have a default namespace.

See also

- `@namespace`
- `Document.createElementNS()` method
- `Element.namespaceURI` property
- CSS type selector
- CSS universal selector
- CSS selector module

& nesting selector

The CSS **`&` nesting selector** explicitly states the relationship between parent and child rules when using CSS nesting. It makes the nested child rule selectors *relative to the parent element*. Without the `&` nesting selector, the child rule selector selects child elements. The child rule selectors have the same specificity weight as if they were within `:is()`.

> **Note:** *Child rule* does not mean *child element selector*. A child rule can target parent element or child elements depending on use of the `&` nesting selector.

If not used in nested style rule, the `&` nesting selector represents the scoping root.

Syntax

```
/* Nested directly — adds whitespace (descendant) */
parentRule {
  & childRule { }
}

/* Attached to parent — no whitespace (e.g., pseudo-class, compound selector) */
parentRule {
  &:pseudo-class { }
}

/* Reversed context — & placed after another selector */
parentRule {
  otherRule & { }
}
```

Description

<code>&</code> nesting selector and whitespace

Consider the following code where nesting is done *without* the `&` nesting selector.

```
.parent-rule {
  /* parent rule properties */
  .child-rule {
    /* child rule properties */
  }
}
```

When the browser parses the nested selectors, it automatically adds whitespace between the selectors to create a new CSS selector rule. The following code shows the equivalent non-nested rules:

```
.parent-rule {
  /* parent rule style properties */
}

.parent-rule .child-rule {
  /* style properties for .child-rule descendants for .parent-rule ancestors */
}
```

When the nested rule needs to be attached (with no whitespace) to the parent rule, such as when using a `pseudo-class` or creating compound selectors, the `&` nesting selector must be immediately prepended to achieve the desired effect.

Consider an example where we want to style an element, providing styles to be applied at all times, and also nesting some styles to be applied only on hover. If the `&` nesting selector is not included, whitespace is added and we end up with a ruleset that applies the nested styles to any *hovered descendant of the parent rule selector*. This is, however, not what we want.

```
.parent-rule {
  /* parent rule properties */
  :hover {
    /* child rule properties */
  }
}

/* the browser parses the above nested rules as shown below */
.parent-rule {
  /* parent rule properties */
}
```

```
.parent-rule *:hover {
  /* child rule properties */
}
```

With the `&` nesting selector added with no whitespace, the elements matched by the parent rule will be styled when hovered.

```
.parent-rule {
  /* parent rule properties */
  &:hover {
    /* child rule properties */
  }
}

/* the browser parses the above nested rules as shown below */
.parent-rule {
  /* parent rule properties */
}

.parent-rule:hover {
  /* child rule properties */
}
```

Appending the <code>&</code> nesting selector

The `&` nesting selector can also be appended to reverse the context of the rules.

```
.card {
  /* .card styles */
  .featured & {
    /* .featured .card styles */
  }
}

/* the browser parses above nested rules as */

.card {
  /* .card styles */
}

.featured .card {
  /* .featured .card styles */
}
```

The `&` nesting selector can be placed multiple times:

```
.card {
  /* .card styles */
  .featured & & & {
    /* .featured .card .card .card styles */
  }
}

/* the browser parses above nested rules as */

.card {
  /* .card styles */
}

.featured .card .card .card {
  /* .featured .card .card .card styles */
}
```

Cannot represent pseudo-elements

The `&` selector is equivalent to the `:is()` selector, and has the same limitation that it cannot represent pseudo-elements.

For example, with the following style rule, no generated content will be styled red, even when nested in `<div class="important">`, because `.important :is(.foo::before)` cannot match anything.

```
.foo::before {
  content: "Hello";

  .important & {
    color: red;
  }
}
```

This limitation also applies to nested at-rules, whose properties are implicitly wrapped in an `&` selector. For example, with the following rule, no generated content will be styled red, even on a small screen, because the `color: red` property is implicitly wrapped in an `&` selector, which in this case is `:is(.foo::before)`.

```
.foo::before {
  content: "Hello";

  @media (width < 600px) {
    color: red;
  }
}
```

Examples

Both of the following examples produce the same output. The first one uses normal CSS styles and the second one uses the `&` nesting selector.

Using normal CSS styles

This example uses normal CSS styling.

HTML

```
<p class="example">
  This paragraph <a href="#">contains a link</a>, try hovering or focusing it.
</p>
```

CSS

```
.example {
  font-family: system-ui;
  font-size: 1.2rem;
}

.example > a {
  color: tomato;
}

.example > a:hover,
.example > a:focus {
  color: ivory;
  background-color: tomato;
}
```

Result

Using <code>&</code> in nested CSS styles

This example uses nested CSS styling.

HTML

```
<p class="example">
  This paragraph <a href="#">contains a link</a>, try hovering or focusing it.
</p>
```

CSS

```
.example {
  font-family: system-ui;
  font-size: 1.2rem;
  & > a {
    color: tomato;
    &:hover,
    &:focus {
      color: ivory;
      background-color: tomato;
    }
  }
}
```

Result

Using <code>&</code> outside nested rule

If not used in a nested style rule, `&` represents the scoping root. In this case, all styles apply to the document.

```
<p>Hover over the output box to change document's background color.</p>
```

```
& {
  color: blue;
  font-weight: bold;
}

&:hover {
  background-color: wheat;
}
```

Result

See also

- Using CSS nesting
- CSS nesting module
- CSS selectors module

Next-sibling combinator

The **next-sibling combinator** (`+`) separates two selectors and matches the second element only if it *immediately* follows the first element, and both are children of the same parent `element`.

```
/* Paragraphs that come immediately after any image */
img + p {
  font-weight: bold;
}
```

Syntax

```
/* The white space around the + combinator is optional but recommended. */
former_element + target_element { style properties }
```

Examples

Basic usage

This example demonstrates how to select the next sibling if that next sibling is of a specific type.

CSS

We only style the `<li>` that comes immediately after an `<li>` that is the first of its type:

```
li:first-of-type + li {
  color: red;
  font-weight: bold;
}
```

HTML

```
<ul>
  <li>One</li>
  <li>Two!</li>
  <li>Three</li>
</ul>
```

Result

Selecting a previous sibling

The next-sibling combinator can be included within the `:has()` functional selector to select the previous sibling.

CSS

We only style the `<li>` with a next sibling that is an `<li>` that is the last of its type:

```
li:has(+ li:last-of-type) {
  color: red;
  font-weight: bold;
}
```

HTML

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three!</li>
  <li>Four</li>
</ul>
```

Result

See also

- Subsequent-sibling combinator

Pseudo-classes

A CSS ***pseudo-class*** is a keyword added to a selector that lets you select elements based on information that lies outside of the document tree, such as a specific state of the selected element(s). For example, the pseudo-class `:hover` can be used to style a button when a user's pointer hovers over it.

```
/* Any button over which the user's pointer is hovering */
button:hover {
  color: blue;
}
```

A pseudo-class consists of a colon (`:`) followed by the pseudo-class name (e.g., `:hover`). A functional pseudo-class also contains a pair of parentheses to define the arguments (e.g., `:dir()`). The element that a pseudo-class is attached to is defined as an *anchor element* (e.g., `button` in case `button:hover`).

Pseudo-classes let you apply a style to an element not only in relation to the content of the document tree, but also in relation to external factors like the history of the navigator (`:visited`, for example), the status of its content (like `:checked` on certain form elements), or the position of the mouse (like `:hover`, which lets you know if the mouse is over an element or not).

> **Note:** In contrast to pseudo-classes, pseudo-elements can be used to style a *specific part* of an element.

Elemental pseudo-classes

These pseudo-classes relate to the core identity of elements.

`:defined`

Matches any element that is defined.

`:heading`

Matches any heading element (`<h1>`-`<h6>`).

Element display state pseudo-classes

These pseudo-classes enable the selection of elements based on their display states.

`:open`

Matches an element that can either be open or closed that is currently open.

`:popover-open`

Matches a popover element that is currently in the showing state.

`:modal`

Matches an element that is in a state in which it excludes all interaction with elements outside it until the interaction has been dismissed.

`:fullscreen`

Matches an element that is currently in fullscreen mode.

`:picture-in-picture`

Matches an element that is currently in picture-in-picture mode.

`:xr-overlay`

Matches the DOM overlay element during an immersive AR or VR session.

Input pseudo-classes

These pseudo-classes relate to form elements, and enable selecting elements based on HTML attributes and the state that the field is in before and after interaction.

`:enabled`

Represents a user interface element that is in an enabled state.

`:disabled`

Represents a user interface element that is in a disabled state.

`:read-only`

Represents any element that cannot be changed by the user.

`:read-write`

Represents any element that is user-editable.

`:placeholder-shown`

Matches an input element that is displaying placeholder text. For example, it will match the `placeholder` attribute in the `<input>` and `<textarea>` elements.

`:autofill`

Matches when an `<input>` has been autofilled by the browser.

`:default`

Matches one or more UI elements that are the default among a set of elements.

`:checked`

Matches when elements such as checkboxes and radio buttons are toggled on.

`:indeterminate`

Matches UI elements when they are in an indeterminate state.

`:blank`

Matches a user-input element which is empty, containing an empty string or other null input.

`:valid`

Matches an element with valid contents. For example, an input element with the type 'email' that contains a validly formed email address or an empty value if the control is not required.

`:invalid`

Matches an element with invalid contents. For example, an input element with type 'email' with a name entered.

`:in-range`

Applies to elements with range limitations. For example, a slider control when the selected value is in the allowed range.

`:out-of-range`

Applies to elements with range limitations. For example, a slider control when the selected value is outside the allowed range.

`:required`

Matches when a form element is required.

`:optional`

Matches when a form element is optional.

`:user-valid`

Represents an element with correct input, but only when the user has interacted with it.

`:user-invalid`

Represents an element with incorrect input, but only when the user has interacted with it.

Linguistic pseudo-classes

These pseudo-classes reflect the document language and enable the selection of elements based on language or script direction.

`:dir()`

The directionality pseudo-class selects an element based on its directionality as determined by the document language.

`:lang()`

Select an element based on its content language.

Location pseudo-classes

These pseudo-classes relate to links, and to targeted elements within the current document.

`:any-link`

Matches an element if the element would match either `:link` or `:visited`.

`:link`

Matches links that have not yet been visited.

`:visited`

Matches links that have been visited.

`:local-link`

Matches links whose absolute URL is the same as the target URL. For example, anchor links to the same page.

`:target`

Matches the element which is the target of the document URL.

`:scope`

Represents elements that are a reference point for selectors to match against.

> **Note:** A `:target-within` pseudo-class, to match elements that are or have a descendant which is the target of the document URL, was defined but removed from the specification. Use `:has(:target)` for this purpose.

Resource state pseudo-classes

These pseudo-classes apply to media that is capable of being in a state where it would be described as playing, such as a video.

`:playing`

Represents a playable element that is playing.

`:paused`

Represents a playable element that is paused.

`:seeking`

Represents a playable element that is currently seeking a playback position in the media resource.

`:buffering`

Represents a playable element that is playing but is temporarily stalled because it is downloading the media resource.

`:stalled`

Represents a playable element that is playing but is stalled because it cannot download the media resource.

`:muted`

Represents a sound-producing element that is muted.

`:volume-locked`

Represents a sound-producing element that has its volume level locked by the browser.

Time-dimensional pseudo-classes

These pseudo-classes apply when viewing something which has timing, such as a WebVTT caption track.

`:current`

Represents the element or ancestor of the element that is being displayed.

`:past`

Represents an element that occurs entirely before the `:current` element.

`:future`

Represents an element that occurs entirely after the `:current` element.

Tree-structural pseudo-classes

These pseudo-classes relate to the location of an element within the document tree.

`:root`

Represents an element that is the root of the document. In HTML this is usually the `<html>` element.

`:empty`

Represents an element with no children other than white-space characters.

`:nth-child()`

Uses `An+B` notation to select elements from a list of sibling elements.

`:nth-last-child()`

Uses `An+B` notation to select elements from a list of sibling elements, counting backwards from the end of the list.

`:first-child`

Matches an element that is the first of its siblings.

`:last-child`

Matches an element that is the last of its siblings.

`:only-child`

Matches an element that has no siblings. For example, a list item with no other list items in that list.

`:heading()`

Uses `An+B` notation to select heading elements (`<h1>`-`<h6>`).

`:nth-of-type()`

Uses `An+B` notation to select elements from a list of sibling elements that match a certain type from a list of sibling elements.

`:nth-last-of-type()`

Uses `An+B` notation to select elements from a list of sibling elements that match a certain type from a list of sibling elements counting backwards from the end of the list.

`:first-of-type`

Matches an element that is the first of its siblings, and also matches a certain type selector.

`:last-of-type`

Matches an element that is the last of its siblings, and also matches a certain type selector.

`:only-of-type`

Matches an element that has no siblings of the chosen type selector.

Shadow-structural pseudo-classes

These pseudo-classes relate to the shadow DOM.

`:host`

Matches the shadow tree's shadow host.

`:host()`

Matches an element that matches `:host` and matches any of the selectors in the list provided.

`:host-context()`

Selects elements outside of the shadow tree in the context of the shadow host.

`:has-slotted`

Matches slot elements that have been assigned content.

User action pseudo-classes

These pseudo-classes require some interaction by the user in order for them to apply, such as holding a mouse pointer over an element.

`:hover`

Matches when a user designates an item with a pointing device, such as holding the mouse pointer over the item.

`:active`

Matches when an item is being activated by the user. For example, when the item is clicked on.

`:focus`

Matches when an element has focus.

`:focus-visible`

Matches when an element has focus and the user agent identifies that the element should be visibly focused.

`:focus-within`

Matches an element to which `:focus` applies, plus any element that has a descendant to which `:focus` applies.

`:target-current`

Matches the `::scroll-marker` pseudo-element of a `scroll-marker-group` that is currently scrolled to, in other words, the **active** scroll marker.

Top-layer ancestor matching boundary

When setting styles using a `:hover`, `:active`, or `:focus-within` pseudo-class selector, the selector matches a nested set of elements up the DOM tree. If the hierarchy includes an element in the top layer (for example, a popover or customizable `<select>` picker), the matching stops at that top-layer element.

For example, the following code features a customizable `<select>` element and a popover. We've set every element in the page to have a thick blue dashed `border` on hover.

```
<main>
  <select>
    <option>One</option>
    <option>Two</option>
    <option>Three</option>
  </select>

  <div>
    <button popovertarget="mypopover">Toggle popover</button>
    <section id="mypopover" popover>
      <p>I am a popover</p>
      <button>I am a popover button</button>
    </section>
  </div>
</main>
```

```
* {
  padding: 5px;
}
```

```
select,
::picker(select) {
  appearance: base-select;
}

:hover {
  border: 5px dashed blue;
}
```

Note that when you hover over the `<select>` element or the popover toggle button, all ancestors of those elements get the border. However, once the select picker or the popover is open, hovering over one of its descendants applies the border only up to the top-layer element itself (the select picker or the popover), that is, the matching stops at this element.

This behavior stops the pseudo-class matching from spilling out of a top-layer component into the surrounding page, which can otherwise cause a user interface to look broken.

Functional pseudo-classes

These pseudo-classes accept a selector list or forgiving selector list as a parameter.

`:is()`

The matches-any pseudo-class matches any element that matches any of the selectors in the list provided. The list is forgiving.

`:not()`

The negation, or matches-none, pseudo-class represents any element that is not represented by its argument.

`:where()`

The specificity-adjustment pseudo-class matches any element that matches any of the selectors in the list provided without adding any specificity weight. The list is forgiving.

`:has()`

The relational pseudo-class represents an element if any of the relative selectors match when anchored against the attached element.

Custom state pseudo-classes

These pseudo-classes apply to custom elements.

`:state()`

Matches custom elements that have the specified custom state.

Page pseudo-classes

These pseudo-classes relate to pages in a printed document and are used with the `@page` at-rule.

`:left`

Represents all left-hand pages of a printed document.

`:right`

Represents all right-hand pages of a printed document.

`:first`

Represents the first page of a printed document.

`:blank`

Represents a blank page in a printed document.

View transition pseudo-classes

These pseudo-classes relate to elements involved in a view transition.

`:active-view-transition`

Matches the root element of a document when a view transition is in progress (*active*) and stops matching once the transition has completed.

`:active-view-transition-type()`

Matches the root element of a document when a specified view transition is in progress (*active*) and stops matching once the transition has completed.

Syntax

```
selector:pseudo-class {
  property: value;
}
```

Like regular classes, you can chain together as many pseudo-classes as you want in a selector.

Alphabetical index

Pseudo-classes defined by a set of CSS specifications include the following:

A

- `:active`
- `:active-view-transition`
- `:active-view-transition-type()`
- `:any-link`
- `:autofill`

B

- `:blank` (input)
- `:blank` (page)
- `:buffering`

C

- `:checked`
- `:current`

D

- `:default`
- `:defined`
- `:dir()`
- `:disabled`

E

- :empty
- :enabled

F

- :first
- :first-child
- :first-of-type
- :focus
- :focus-visible
- :focus-within
- :fullscreen
- :future

H

- :has-slotted
- :has()
- :heading
- :heading()
- :host
- :host()
- :host-context()
- :hover

I

- :in-range
- :indeterminate
- :interest-source
- :interest-target
- :invalid
- :is()

L

- :lang()
- :last-child
- :last-of-type
- :left
- :link
- :local-link

M

- :modal
- :muted

N

- :not()
- :nth-child()
- :nth-last-child()
- :nth-last-of-type()
- :nth-of-type()

O

- :only-child
- :only-of-type
- :open
- :optional
- :out-of-range

P

- :past
- :paused
- :picture-in-picture
- :placeholder-shown
- :playing
- :popover-open

R

- :read-only
- :read-write
- :required
- :right
- :root

S

- :scope
- :seeking
- :stalled
- :state()

T

- :target
- :target-after
- :target-current

- `:target-before`

U

- `:user-invalid`
- `:user-valid`

V

- `:valid`
- `:visited`
- `:volume-locked`

W

- `:where()`

X

- `:xr-overlay`

Non-standard pseudo-classes

Non-standard vendor-prefixed pseudo-classes include:

`-moz-` prefix

- `:-moz-broken`
- `:-moz-drag-over`
- `:-moz-first-node`
- `:-moz-handler-blocked`
- `:-moz-handler-crashed`
- `:-moz-handler-disabled`
- `:-moz-last-node`
- `:-moz-loading`
- `:-moz-locale-dir(ltr)`
- `:-moz-locale-dir(rtl)`
- `:-moz-only-whitespace`
- `:-moz-submit-invalid`
- `:-moz-suppressed`
- `:-moz-user-disabled`
- `:-moz-window-inactive`

See also

- Pseudo-elements
- CSS selectors module

Pseudo-elements

A CSS **pseudo-element** is a keyword added to a selector that lets you style a specific part of the selected element(s).

> **Note:** This page is an index of all pseudo-elements in CSS. The CSS pseudo-elements page introduces the module that defines some, but not all, of these pseudo-elements.

Syntax

```
selector::pseudo-element {
  property: value;
}
```

For example, `::first-line` can be used to change the font of the first line of a paragraph.

```
/* The first line of every <p> element. */
p::first-line {
  color: blue;
  text-transform: uppercase;
}
```

Double colons (`::`) are used for pseudo-elements. This distinguishes pseudo-elements from pseudo-classes that use a single colon (`:`) in their notation. Note, browsers support single colon syntax for the original four pseudo-elements: `::before`, `::after`, `::first-line`, and `::first-letter`.

Pseudo-elements do not exist independently. The element of which a pseudo-element is a part is called its *originating element*. A pseudo-element must appear after all the other components in the complex or compound selector. The last element in the selector is the originating element of the pseudo-element. For example, you can select a paragraph's first line using `p::first-line` but not the first-line's children. So `p::first-line > *` is invalid.

A pseudo-element can be selected based on the current state of the originating element. For example, `p:hover::first-line` selects the first line (pseudo-element) of a paragraph when the paragraph itself is being hovered (pseudo-class).

The specifications allow some pseudo-elements to be selected based on the current state of the originating element. In these cases, the pseudo-element is selected only when it is in the same state. For example, `::picker(select):hover` selects the currently hovered descendant of a customizable `<select>` element, excluding the first `<button>` child, which is not selected by the `::picker()` pseudo-element. Note, however, that when a selector list contains an invalid selector, the entire style block is ignored, so make sure the state you're including is supported on the pseudo-element to which it is attached.

Typographic pseudo-elements

`::first-line`

The first line-box of the originating element.

`::first-letter`

The first letter, number, or symbol character on the first line of its originating element.

`::cue`

The WebVTT cues within a selected element. This can be used to style captions and other cues in media with VTT tracks. The CSS pseudo-elements module also defines the `::postfix` and `::prefix` sub-pseudo elements. These are not yet supported by any browser.

Highlight pseudo-elements

Selects document sections based on content and document status, enabling those areas to be styled differently to indicate that status to the user.

`::grammar-error`

A portion of text that the browser thinks is grammatically incorrect.

`::highlight()`

The elements in the highlight registry. It is used to create custom highlights.

`::search-text`

Search results identified by the user agent's "Find" or "Find in page" text search feature.

`::selection`

The portion of a document that has been selected.

`::spelling-error`

A portion of text that the browser thinks is misspelled.

`::target-text`

The document's target element. The target element is identified using the URL's fragment identifier.

Tree-Abiding pseudo-elements

These pseudo-elements behave like regular elements, fitting seamlessly within the box model. They act as a child element that can be styled directly within the originating element hierarchy.

`::before`

Creates a pseudo-element that is the first child of the selected element.

`::after`

Creates a pseudo-element that is the last child of the selected element.

`::column`

Each column fragment of a multi-column layout.

`::marker`

The automatically generated marker box of a list item.

`::backdrop`

The backdrop of the originating element rendered in the top layer.

`::scroll-button()`

Creates a button that can control the scrolling of the scroll container to which it is applied.

`::scroll-marker`

Creates a pseudo-element that is a scroll marker — a scroll target button for its originating element nested in a scroll-marker group.

`::scroll-marker-group`

Generates a container before or after a scroll container to contain the `::scroll-marker` pseudo-elements generated on the element or its descendants.

Element-backed pseudo-elements

These pseudo-elements are real elements that are not otherwise selectable.

`::details-content`

The expandable/collapsible contents of a `<details>` element.

`::part()`

Any element within a shadow tree that has a matching `part` attribute.

`::slotted()`

Any element placed into a slot inside an HTML template.

Form-related pseudo-elements

The pseudo-elements are related to form controls.

`::checkmark`

Targets the checkmark placed inside the currently-selected `<option>` element of a customizable select element to provide a visual indication of which one is selected.

`::file-selector-button`

The button of an `<input>` of `type="file"`.

`::picker()`

The picker part of an element, for example the drop-down picker of a customizable select element.

`::picker-icon`

The picker icon inside form controls that have an icon associated with them. In the case of a customizable select element, it selects the arrow that points down when the select is closed.

`::placeholder`

The placeholder text in an input field.

Alphabetical index

Pseudo-elements defined by a set of CSS specifications include the following:

A

- `::after`

B

- `::backdrop`
- `::before`

C

- `::column`
- `::checkmark`
- `::cue` (and `::cue()`)

D

- `::details-content`

F

- `::file-selector-button`
- `::first-letter`
- `::first-line`

G

- `::grammar-error`

H

- `::highlight()`

M

- `::marker`

P

- `::part()`
- `::picker()`
- `::picker-icon`
- `::placeholder`

S

- `::scroll-button()`
- `::scroll-marker`
- `::scroll-marker-group`
- `::selection`
- `::slotted()`
- `::spelling-error`

T

- `::target-text`

V

- `::view-transition`
- `::view-transition-image-pair()`
- `::view-transition-group()`
- `::view-transition-new()`
- `::view-transition-old()`

Non-standard pseudo-elements

Non-standard vendor-prefixed pseudo-elements include:

`-moz-` prefix

- `::-moz-color-swatch`
- `::-moz-focus-inner`
- `::-moz-list-bullet`
- `::-moz-list-number`
- `::-moz-meter-bar`
- `::-moz-progress-bar`
- `::-moz-range-progress`
- `::-moz-range-thumb`
- `::-moz-range-track`

`-webkit-` prefix

- `::-webkit-inner-spin-button`
- `::-webkit-meter-bar`
- `::-webkit-meter-even-less-good-value`
- `::-webkit-meter-inner-element`
- `::-webkit-meter-optimum-value`
- `::-webkit-meter-suboptimum-value`
- `::-webkit-progress-bar`
- `::-webkit-progress-inner-element`

- `::-webkit-progress-value`
- `::-webkit-scrollbar`
- `::-webkit-search-cancel-button`
- `::-webkit-search-results-button`
- `::-webkit-slider-runnable-track`
- `::-webkit-slider-thumb`

Nesting pseudo-elements

You can chain some pseudo-element selectors together to style pseudo-elements nested inside other pseudo-elements. The following nested pseudo-element combinations are supported:

- `::after`
  - `::after::marker`: Selects the `::marker` pseudo-element of an `::after` pseudo-element, when `::after` is styled as a list item, with `display: list-item`.
- `::before`
  - `::before::marker`: Selects the `::marker` pseudo-element of a `::before` pseudo-element, when `::before` is styled as a list item, with `display: list-item`.

Check out the individual pseudo-element reference pages for examples and browser compatibility information.

Highlight pseudo-elements inheritance

Highlight pseudo-elements, such as `::selection`, `::target-text`, `::highlight()`, `::spelling-error`, and `::grammar-error`, follow a consistent inheritance model that differs from regular element inheritance.

When you apply styles to highlight pseudo-elements, they inherit from both:

1. Their parent elements (following normal inheritance).
2. The highlight pseudo-elements of their parent elements (following highlight inheritance).

This means that if you style both a parent element's highlight pseudo-element and a child element's highlight pseudo-element, the child's highlighted text will combine properties from both sources.

Here is a concrete example.

First, we have some HTML that includes two nested `<div>` elements. Some of the included text content is contained directly inside the parent `<div>`, and some is nested inside the child `<div>`.

```
<div class="parent">
  Parent text
  <div class="child">Child text</div>
</div>
```

Next we include some CSS, which selects the parent and child `<div>` elements separately and gives them different `color` values, and selects the parent and child's selected text (`::selection`). This gives each `<div>` a different `background-color` and sets a different text `color` on the parent selection.

```
/* Style for the parent element */
.parent {
  color: blue;
}

/* Style for the parent's selected text */
.parent::selection {
  background-color: yellow;
  color: red;
}

/* Style for the child element */
.child {
  color: green;
}

/* Style for the child's selected text */
.child::selection {
  background-color: orange;
}
```

The example renders as follows:

Try selecting the text in both the parent and child elements. Notice that:

1. When you select the parent text, it uses the yellow background and red text color defined in `.parent::selection`.
2. When you select the child text, it uses:
   - The orange background from `.child::selection`.
   - The red text color inherited from the parent's `::selection` pseudo-element.

This demonstrates how the child's highlight pseudo-element inherits from both its parent element and the parent's highlight pseudo-element.

CSS custom properties (variables) in highlight pseudo-elements inherit from their originating element (the element they're being applied to), not through the highlight inheritance chain. For example:

```
:root {
  --selection-color: lightgreen;
}

::selection {
  color: var(--selection-color);
}

.blue {
  --selection-color: blue;
}
```

When using the universal selector with highlight pseudo-elements, it prevents highlight inheritance. For example:

```
/* This prevents highlight inheritance */
*::selection {
  color: lightgreen;
}

/* Prefer this to allow inheritance */
:root::selection {
  color: lightgreen;
}
```

See also

- CSS pseudo-elements module
- Pseudo-classes
- CSS selectors module
- Learn: Pseudo-classes and pseudo-elements
- Inheritance changes for CSS selection styling - Detailed explanation of the highlight pseudo-elements inheritance model changes in Chrome 134

Selector list

The CSS **selector list** (`,`) selects all the matching nodes. A selector list is a comma-separated list of selectors.

Description

When multiple selectors share the same declarations, they can be grouped together into a comma-separated list. Selector lists can also be passed as parameters to some functional CSS pseudo-classes. White space may appear before and/or after the comma.

The following three declarations are equivalent:

```
span {
  border: red 2px solid;
}
div {
  border: red 2px solid;
}
```

```
span,
div {
  border: red 2px solid;
}
```

```
:is(span, div) {
  border: red 2px solid;
}
```

Examples

When applying the same styles to elements matching different criteria, grouping the selectors in a comma-separated list can improve consistency while reducing the size of style sheets.

Single line grouping

This example shows grouping selectors in a single line using a comma-separated list.

```
h1, h2, h3, h4, h5, h6 {
  font-family: "Helvetica", "Arial";
}
```

Multi line grouping

This example shows grouping selectors in multiple lines using a comma-separated list.

```
#main,
.content,
article,
h1 + p {
  font-size: 1.1em;
}
```

Valid and invalid selector lists

An invalid selector represents, and therefore matches, nothing. When a selector list contains an invalid selector, the entire style block is ignored, except for the `:is()` and `:where()` pseudo-classes that accept forgiving selector lists.

Invalid selector list

A downside to using a selector list is that a single unsupported selector in the selector list invalidates the entire rule.

Consider the following two CSS rule sets:

```
h1 {
  font-family: sans-serif;
}
h2:invalid-pseudo {
  font-family: sans-serif;
}
h3 {
  font-family: sans-serif;
}
```

```
h1,
h2:invalid-pseudo,
h3 {
  font-family: sans-serif;
}
```

They are not equivalent. In the first rule set, styles will be applied on the `h1` and `h3` elements, but the `h2:invalid-pseudo` rule will not be parsed. In the second rule set, because one selector in the list is invalid, the entire rule will not be parsed. Because of this, no style will be applied to the `h1` and `h3` elements as when any selector in a list of selectors in invalid, the entire style block will be ignored.

Forgiving selector list

A way to remedy the invalid selector list problem is to use the `:is()` or the `:where()` pseudo-class, which accept a forgiving selector list. Each selector in a forgiving selector list is parsed individually. So any invalid selectors in the list are ignored and the valid ones are used.

Carrying on from the previous example, the following two CSS rule sets are now equivalent:

```
h1 {
  font-family: sans-serif;
```

```
}
h2:maybe-unsupported {
  font-family: sans-serif;
}
h3 {
  font-family: sans-serif;
}
```

```
:is(h1, h2:maybe-unsupported, h3) {
  font-family: sans-serif;
}
```

The difference between the two is that the specificity of `:is()` is its most specific argument, whereas the `:where()` selector and the forgiving selector list parameter do not add any specificity weight.

Relative selector list

A relative selector list is a comma-separated selector list parsed as relative selectors, which begin with an explicit or implied combinator.

```
h2:has(+ p, + ul.red) {
  font-style: italic;
}
```

In the above example, the italic style will be applied to any `h2` heading that is immediately followed by `<p>` or `<ul class="red">`. Note that pseudo-elements and the `:has()` selector are not valid within the `:has()` relative selector list.

See also

- The `:is()` and `:where()` pseudo-classes accept forgiving selector lists.
- The `:not()` pseudo-class accepts a regular selector list
- The `:has()` pseudo-class accepts a relative selector list.
- CSS selectors module
- Selectors and combinators

Subsequent-sibling combinator

The **subsequent-sibling combinator** (~, a tilde) separates two selectors and matches *all instances* of the second element that follow the first element (not necessarily immediately) and share the same parent element.

In the following example, the subsequent-sibling combinator (~) helps to select and style paragraphs that are both siblings of an image and appear after any image.

```
img ~ p {
  color: red;
}
```

Syntax

```
/* The white space around the ~ combinator is optional but recommended. */
former_element ~ target_element { style properties }
```

Examples

Using the combinator with simple selectors

This example shows the use of the ~ combinator when both the selectors are simple selectors (`p` and `span`).

```
<article>
  <span>This is not red because it appears before any paragraph.</span>
  <p>Here is a paragraph.</p>
  <code>Here is some code.</code>
  <span>
    This span is red because it appears after the paragraph, even though there
    are other nodes in between.
  </span>
  <p>Whatever it may be, keep smiling.</p>
  <h1>Dream big</h1>
  <span>
    Doesn't matter how many or what kind of nodes are in between, all spans from
    the same parent after a paragraph are red.
  </span>
</article>
<span>
  This span is not red because it doesn't share a parent with a paragraph.
</span>
```

```
p ~ span {
  color: red;
}
```

Using the combinator with complex selectors

This example contains two complex selectors, both using the subsequent-sibling combinator: `.foo p ~ span` and `.foo p ~ .foo span`.

- The first complex selector, `.foo p ~ span`, matches all spans that come after a paragraph *if* the span and paragraph share the same parent **and** that parent or an ancestor of that parent has the class `.foo`.
- The second complex selector, `.foo p ~ .foo span`, matches all spans that are a descendant of the element with class `.foo` *if* that element is a sibling of the previously mentioned paragraph.

The example below shows that the target element in the complex selector must share the same parent as the initial element in the complex selector.

```
<h1>Dream big</h1>
<span>And yet again this is a red span!</span>
<div class="foo">
  <p>Here is another paragraph.</p>
  <span>A blue span</span>
  <div class="foo">
    <span>A green span</span>
  </div>
</div>
```

```
.foo p ~ span {
  color: blue;
}

.foo p ~ .foo span {
  color: green;
}
```

In the above HTML, the two siblings of `.foo p` are `span` and `.foo`. The green `span` is a descendant of the `.foo` class, which is a sibling of `p`.

- When the target selector is `span`, the `span` element that is a sibling of `p` is selected. The `p` element is a descendant of `.foo`, so are its `span` siblings.
- In `.foo p ~ .foo span`, the target selector is `span` that is a descendant of `.foo`. In this case, the `span` element that's a descendant of `.foo` is selected if that `.foo` is a sibling of `p`; essentially, both are nested in an ancestor of `.foo`.

See also

- Next-sibling combinator

Type selectors

The CSS **type selector** matches elements by node name. In other words, it selects all elements of the given type within a document.

```
/* All <a> elements. */
a {
  color: red;
}
```

Type selectors can be namespaced when using `@namespace`. This is useful when dealing with documents containing multiple namespaces such as HTML with inline SVG or MathML, or XML that mixes multiple vocabularies.

- `ns|h1` - matches `<h1>` elements in namespace *ns*
- `*|h1` - matches all `<h1>` elements
- `|h1` - matches all `<h1>` elements without any declared namespace

Syntax

```
element { style properties }
```

Examples

CSS

```
span {
  background-color: skyblue;
}
```

HTML

```
<span>Here's a span with some text.</span>
<p>Here's a p with some text.</p>
<span>Here's a span with more text.</span>
```

Result

Namespaces

In this example the selector will only match `<h1>` elements in the example namespace.

```
@namespace example url("http://www.example.com/");
example|h1 {
  color: blue;
}
```

See also

- CSS Selectors
- Learn CSS: Basic selectors

Universal selectors

The CSS **universal selector** (`*`) matches elements of any type.

```
/* Selects all elements */
* {
  color: green;
}
```

The universal selector is a special type selector and can therefore be namespaced when using `@namespace`. This is useful when dealing with documents containing multiple namespaces such as HTML with inline SVG or MathML, or XML that mixes multiple vocabularies.

- `ns|*` - matches all elements in namespace *ns*
- `*|*` - matches all elements
- `|*` - matches all elements without any declared namespace

> **Note:** The universal selector (`*`) matches **elements only**. It does **not** directly match pseudo-elements by itself.
>
> To match all `::before` pseudo-elements on a page, for example, you would have to use a selector like `*::before`. This works because the `*` matches all elements, and the `::before` pseudo-element is available on all elements.

Syntax

```
* { style properties }
```

The asterisk is optional with simple selectors. For instance, `*.warning` and `.warning` are equivalent.

Examples

CSS

```
* [lang^="en"] {
  color: green;
}

*.warning {
  color: red;
}

*#maincontent {
  border: 1px solid blue;
}

.floating {
  float: left;
}

/* automatically clear the next sibling after a floating element */
.floating + * {
  clear: left;
}
```

HTML

```
<p class="warning">
  <span lang="en-us">A green span</span> in a red paragraph.
</p>
<p id="maincontent" lang="en-gb">
  <span class="warning">A red span</span> in a green paragraph.
</p>
```

Result

Namespaces

In this example the selector will only match elements in the example namespace.

```
@namespace example url("http://www.example.com/");
example|* {
  color: blue;
}
```

See also

- CSS selectors module
- Learn CSS: Basic selectors

CSS values

CSS values are the values that can be assigned to CSS properties. These include keywords, data types, and functions.

> **Note:** This page is an index of all value features in CSS. The CSS values and units page introduces the module that defines some, but not all, of these values.

Index of keywords

- `!important`
- `fit-content`
- `inherit`
- `initial`
- `max-content`
- `min-content`
- `revert`
- `revert-layer`
- `unset`

See also

- CSS data types
- CSS value functions
- CSS values and units module
- CSS syntax module

-moz-image-rect() CSS function

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`-moz-image-rect`** value for CSS `background-image` lets you use a portion of a larger image as a background.

Syntax

```
-moz-image-rect(url("my-url"), top, right, bottom, left);
```

Values

`<url>`

The URI of the image from which to take the sub-image.

`top`

The top edge, specified as an `<integer>` or `<percentage>`, of the sub-image within the specified image.

`right`

The right edge, specified as an `<integer>` or `<percentage>`, of the sub-image within the specified image.

`bottom`

The bottom edge, specified as an `<integer>` or `<percentage>`, of the sub-image within the specified image.

`left`

The left edge, specified as an `<integer>` or `<percentage>`, of the sub-image within the specified image.

Description

This property allows you to, for example, use different parts of one larger image as backgrounds in different parts of your content.

The syntax for the rectangle is similar to the `rect()` function generating a `<shape>` CSS type. All four values are relative to the upper left corner of the image.

Examples

This example loads an image and uses it in four segments to draw the Firefox logo in four `<div>` blocks. Clicking on their container causes the four segments to rotate around by swapping the `background-image` property values among the four `<div>` blocks.

CSS

The CSS defines one container style, then the styles for the four boxes that comprise the complete image.

The container looks like this:

```
#container {
  width: 267px;
  height: 272px;
  top: 100px;
  left: 100px;
  position: absolute;
  font-size: 16px;
  text-shadow: white 0px 0px 6px;
  text-align: center;
}
```

Then the four boxes defining the segments of the image are defined. Let's look at them one at a time.

```
#box1 {
  background-image: -moz-image-rect(url("firefox.png"), 0%, 50%, 50%, 0%);
  width: 133px;
  height: 136px;
  left: 0px;
  top: 0px;
  position: absolute;
}
```

This is the top-left corner of the image. It defines a rectangle containing the top-left quarter of the image in the file `firefox.jpg`.

```
#box2 {
  background-image: -moz-image-rect(url("firefox.png"), 0%, 100%, 50%, 50%);
  width: 133px;
  height: 136px;
  left: 133px;
  top: 0px;
  position: absolute;
}
```

This defines the top-right corner of the image.

The other corners follow a similar pattern:

```
#box3 {
  background-image: -moz-image-rect(url("firefox.png"), 50%, 50%, 100%, 0%);
  width: 133px;
  height: 136px;
  left: 0px;
  top: 136px;
  position: absolute;
}
#box4 {
  background-image: -moz-image-rect(url("firefox.png"), 50%, 100%, 100%, 50%);
  width: 133px;
  height: 136px;
  left: 133px;
  top: 136px;
  position: absolute;
}
```

HTML

We include a container with four boxes:

```
<div id="container">
  <div id="box1">Top left</div>
  <div id="box2">Top right</div>
  <div id="box3">Bottom left</div>
  <div id="box4">Bottom right</div>
</div>
```

This places the four segments of our image in a two-by-two box grid. These four segments are all contained within a larger `<div>` block whose primary purpose is to receive click events and dispatch them to our JavaScript code.

JavaScript

This code handles the click event when the container receives a mouse click.

```
function rotate() {
  let prevStyle = window
    .getComputedStyle(document.getElementById("box4"), null)
    .getPropertyValue("background-image");

  // Now that we've saved the last one, start rotating
  for (let i = 1; i <= 4; i++) {
    const curId = `box${i}`;

    // Shift the background images
    const curStyle = window
      .getComputedStyle(document.getElementById(curId), null)
      .getPropertyValue("background-image");
    document.getElementById(curId).style.backgroundImage = prevStyle;
    prevStyle = curStyle;
  }
}

document.getElementById("container").addEventListener("click", rotate);
```

This uses `window.getComputedStyle()` to fetch the style of each element, shifting it to the following element. Notice that before it begins doing so it saves a copy of the last box's style since it will be overwritten by the third element's style. By copying the values of the `background-image` property from one element to the next with each mouse click, we achieve the desired effect.

What it looks like

Specifications

Not part of any standard.

See also

- Mozilla CSS extensions
- CSS Backgrounds and Borders module

abs() CSS function

The **`abs()`** CSS function returns the absolute value of the argument, as the same type as the input.

Syntax

```
/* abs( <calc-sum> ) */
abs(20% - 100px)
abs(var(--gradientAngle))
```

Parameters

The `abs()` function accepts one parameter.

`<calc-sum>`

An expression or calculation that resolves to a `<number>`, a `<dimension>`, a `<percentage>` or a `<calc-keyword>`.

Return value

The absolute value of `<calc-sum>`.

- If `<calc-sum>`'s numeric value is positive or `0+`, the function returns `<calc-sum>`.
- Otherwise, it returns `-1 * <calc-sum>`.

Formal syntax

```
<abs()> =
  abs( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Positive variables

The `abs()` function can be used to ensure that a value is always positive. In the following example a CSS custom property `--font-size` is used as the value of `font-size`. Wrapping this custom property in `abs()` will convert a negative value to a positive one.

```
h1 {
  font-size: abs(var(--font-size));
}
```

Control gradient angle of direction

You can also control the gradient direction using `abs()` function. In the following example, with an angle of -45deg the gradient would start red and transition to blue. By using `abs()` to make the value positive, the gradient will start blue and transition to red.

```
div {
  --deg: -45deg;
  background-image: linear-gradient(abs(var(--deg)), blue, red);
}
```

Backwards compatible fallback

In browsers that lack support for CSS `abs()` function, you can use the CSS `max()` function to achieve the same result:

```
p {
  line-height: max(var(--lh), -1 * var(--lh));
}
```

We use the `max()` function to return the largest (most positive) value from a list of two values: `var(--lh)` or `-1 * var(--lh)`. Irrespective of whether `--lh` is positive or negative, the calculated return value will always be positive, that is, an absolute number.

See also

- `sign()`

<absolute-size> CSS type

The **`<absolute-size>`** CSS data type describes the absolute size keywords. This data type is used in the `font` shorthand and `font-size` properties.

The font size keywords are mapped to the deprecated HTML `size` attribute. See the HTML size attribute section below.

Syntax

```
<absolute-size> = xx-small | x-small | small | medium | large | x-large | xx-large | xxx-large
```

Values

The `<absolute-size>` data type is defined using a keyword value chosen from the list below.

`xx-small`

An absolute size 60% the size of `medium`. Mapped to the deprecated `size="1"`.

`x-small`

An absolute size 75% the size of `medium`.

`small`

An absolute size 89% the size of `medium`. Mapped to the deprecated `size="2"`.

`medium`

A user's preferred font size. This value is used as the reference middle value. Mapped to `size="3"`.

`large`

An absolute size 20% larger than `medium`. Mapped to the deprecated `size="4"`.

`x-large`

An absolute size 50% larger than `medium`. Mapped to the deprecated `size="5"`.

`xx-large`

An absolute size twice the size of `medium`. Mapped to the deprecated `size="6"`.

`xxx-large`

An absolute size three times the size of `medium`. Mapped to the deprecated `size="7"`.

Description

Each `<absolute-size>` keyword value is sized relative to the `medium` size and the individual device's characteristics, such as device resolution. User agents maintain a table of font sizes for each font, with the `<absolute-size>` keywords being the index.

In CSS1 (1996), the scaling factor between adjacent keyword value indexes was 1.5, which was too large. In CSS2 (1998), the scaling factor between adjacent keyword value indexes was 1.2, which created issues for the small values. As a single fixed ratio between adjacent absolute-size keywords was found to be problematic, there is no longer a fixed ratio recommendation. The only recommendation to preserve readability is that the smallest font size should not be less than `9px`.

For each `<absolute-size>` keyword value, the following table lists the scaling factor, mapping to `<h1>` to `<h6>` headings, and mapping to the deprecated HTML `size` attribute.

`<absolute-size>`	xx-small	x-small	small	medium	large	x-large	xx-large	xxx-large
scaling factor	3/5	3/4	8/9	1	6/5	3/2	2/1	3/1
HTML headings	h6		h5	h4	h3	h2	h1	
HTML `size` attribute	1		2	3	4	5	6	7

HTML size attribute

The `size` attribute to set a font's size in HTML is deprecated. The attribute value was either an integer between `1` and `7` or a relative value. Relative values were an integer preceded by `+` or `-` to increase or decrease the font size, respectively. A value of `+1` meant increasing the `size` by one and `-2` meant decreasing the size by two, with the computed value clamped at a minimum of `1` and a maximum computed value of `7`.

Examples

Comparing the keyword values

```
<ul>
  <li class="xx-small">font-size: xx-small;</li>
  <li class="x-small">font-size: x-small;</li>
  <li class="small">font-size: small;</li>
  <li class="medium">font-size: medium;</li>
  <li class="large">font-size: large;</li>
  <li class="x-large">font-size: x-large;</li>
  <li class="xx-large">font-size: xx-large;</li>
  <li class="xxx-large">font-size: xxx-large;</li>
</ul>
```

```
li {
  margin-bottom: 0.3em;
}
.xx-small {
  font-size: xx-small;
}
.x-small {
  font-size: x-small;
}
.small {
  font-size: small;
}
.medium {
  font-size: medium;
}
.large {
  font-size: large;
}
.x-large {
  font-size: x-large;
}
.xx-large {
  font-size: xx-large;
}
.xxx-large {
  font-size: xxx-large;
}
```

Result

See also

- CSS `<relative-size>` data type
- CSS `font` and `font-size` properties
- CSS fonts module

acos() CSS function

The **`acos()`** CSS function is a trigonometric function that returns the inverse cosine of a number between `-1` and `1`. The function contains a single calculation that returns an `<angle>` between `0deg` and `180deg`.

Syntax

```
/* Single <number> values */
transform: rotate(acos(-0.2));
transform: rotate(acos(2 * 0.125));

/* Other values */
transform: rotate(acos(pi / 5));
transform: rotate(acos(e / 3));
```

Parameters

The `acos(number)` function accepts only one value as its parameter.

`number`

A calculation which resolves to a `<number>` between `-1` and `1`.

Return value

The inverse cosine of a `number` will always return an `<angle>` between `0deg` and `180deg`.

- If `number` is less than `-1` or greater than `1`, the result is `NaN`.
- If `number` is exactly `1`, the result is `0`.

Formal syntax

```
<acos()> =
  acos( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
```

```
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotate elements

The `acos()` function can be used to `rotate` elements as it returns an `<angle>`.

HTML

```
<div class="box box-1"></div>
<div class="box box-2"></div>
<div class="box box-3"></div>
<div class="box box-4"></div>
<div class="box box-5"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

```
div.box {
  width: 100px;
  height: 100px;
  background: linear-gradient(orange, red);
}
div.box-1 {
  transform: rotate(acos(1));
}
div.box-2 {
  transform: rotate(acos(0.5));
}
div.box-3 {
  transform: rotate(acos(0));
}
div.box-4 {
  transform: rotate(acos(-0.5));
}
div.box-5 {
  transform: rotate(acos(-1));
}
```

Result

See also

- `sin()`
- `cos()`
- `tan()`
- `asin()`
- `atan()`
- `atan2()`

<alpha-value> CSS type

The **`<alpha-value>`** CSS data type represents a value that can be either a `<number>` or a `<percentage>`, specifying the **alpha channel** or **transparency** of a color.

Syntax

The value of an `<alpha-value>` is given as either a `<number>` or a `<percentage>`.

If given as a number, the useful range is 0 (fully transparent) to 1.0 (fully opaque), with decimal values in between; that is, 0.5 indicates that half of the foreground color is used and half of the background color is used. Values outside the range of 0 to 1 are permitted, but are clamped to lie within the range 0 to 1.

If the alpha value is given as a percentage, 0% corresponds to fully transparent while 100% indicates fully opaque.

Formal syntax

```
<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Interpolation

When animated, values of the `<alpha-value>` CSS data type are interpolated as real, floating-point numbers. The speed of the interpolation is determined by the easing function associated with the animation.

Examples

Setting text color opacity

The `rgb()` function accepts a fourth optional value to specify an alpha value. The following example shows how to apply a color with 60% opacity using the alpha value:

```
/* <rgb()> */
color: rgb(34 12 64 / 60%);
```

Setting shape image threshold

Here an alpha value is used to determine which parts of an image are considered part of a shape:

```
/* shape-image-threshold */
shape-image-threshold: 70%;
shape-image-threshold: 0.7;
```

See also

- Learn: Fundamental text and font styling
- CSS data types
- CSS Color
- `<color>`

anchor() CSS function

The **`anchor()`** CSS function can be used within an **anchor-positioned** element's inset property values, returning a length value relative to the position of the edges of its associated anchor element.

Syntax

```
/* side or percentage */
top: anchor(bottom);
top: anchor(50%);
top: calc(anchor(bottom) + 10px)
inset-block-end: anchor(start);

/* side of named anchor */
top: anchor(--my-anchor bottom);
inset-block-end: anchor(--my-anchor start);

/* side of named anchor with fallback */
top: anchor(--my-anchor bottom, 50%);
inset-block-end: anchor(--my-anchor start, 200px);
left: calc(anchor(--my-anchor right, 0%) + 10px);
```

Parameters

The `anchor()` function's syntax is as follows:

```
anchor(<anchor-name> <anchor-side>, <length-percentage>)
```

The parameters are:

`<anchor-name>` Optional

The `anchor-name` property value of an anchor element you want to position the element's side relative to. This is a `<dashed-ident>` value. If omitted, the element's **default anchor**, referenced in its `position-anchor` property, or associated with the element via the `anchor` HTML attribute, is used.

> **Note:** Specifying an `<anchor-name>` inside an `anchor()` function does not associate an element with an anchor; it only positions the element relative to that anchor. The `position-anchor` CSS property or the `anchor` HTML attribute is still needed to create the association.

`<anchor-side>`

Specifies the side of the anchor, or the relative distance from the `start` side, which the element is positioned relative to. If a physical or logical value is used that is not compatible with the inset property on which `anchor()` is set, the fallback value is used. Valid values include:

`top`

The top of the anchor element.

`right`

The right of the anchor element.

`bottom`

The bottom of the anchor element.

`left`

The left of the anchor element.

`inside`

The same side as the inset property.

`outside`

The opposite side of the inset property.

`start`

The logical start of the anchor element's containing block along the axis of the inset property on which the `anchor()` function is set.

`end`

The logical end of the anchor element's containing block along the axis of the inset property on which the `anchor()` function is set.

`self-start`

The logical start of the anchor element's content along the axis of the inset property on which the `anchor()` function is set.

`self-end`

The logical end of the anchor element's content along the axis of the inset property on which the `anchor()` function is set.

`center`

The center of the axis of the inset property on which the `anchor()` function is set.

`<percentage>`

Specifies the distance, as a percentage, from the start of the element's content along the axis of the inset property on which the `anchor()` function is set.

`<length-percentage>` Optional

Specifies a fallback value the function should resolve to if the `anchor()` function would otherwise not be valid.

Return value

Returns a `<length>` value.

Description

The `anchor()` function enables positioning an element relative to the edges of an anchor element. It is only valid within inset property values set on absolute or fixed position elements.

It returns a `<length>` value specifying the distance from the edge of the containing block corresponding to the inset property to the edge of the anchor element specified by the `<anchor-side>` value. As it returns a `<length>`, it can be used within other CSS functions that accept length values, including `calc()`, `clamp()`, etc.

If no anchor with the name specified by the `<anchor-name>` exists, or if the positioned element does not have an anchor associated with it (i.e., via the `position-anchor` property), the first parameter is considered invalid, and the fallback `<length-percentage>` value is used if one is available. For example, if `top: anchor(bottom, 50px)` were specified on the positioned element but no anchor was associated with it, the fallback value would be used, so `top` would get a computed value of `50px`.

For detailed information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

Properties that accept <code>anchor()</code> function values

The CSS inset properties that accept an `anchor()` function as a value component include:

- `top`
- `left`
- `bottom`
- `right`
- `inset` shorthand
- `inset-block-start`
- `inset-block-end`
- `inset-block` shorthand
- `inset-inline-start`
- `inset-inline-end`
- `inset-inline` shorthand

Compatibility of inset properties and <code><anchor-side></code> values

When using an `anchor()` function inside an inset property value, the `<anchor-side>` parameter specified inside the `anchor()` function has to be compatible with the axis on which the inset property resides.

This means that physical `<anchor-side>` values can be used within the values of physical inset properties if the property has the same axis direction as the `<anchor-side>`. In other words, the `top` and `bottom` sides are not valid within the `left` and `right` property values, and the `left` and `right` sides are not valid within `top` and `bottom` property values. For example, `top: anchor(bottom)` is fine, as they are both vertical values but `top: anchor(left)` is not valid, as `left` is a horizontal value. If `top: anchor(left, 50px)` were specified, the fallback value would be used, so `top` would get a computed value of `50px`. If no fallback is present, the inset property behaves as if it were set to `auto`.

You can use logical `<anchor-side>` values within both logical and physical inset properties as logical `<anchor-side>` values are relative to the inset property's relevant axis, whether the property is logical or relative. For example, `top: anchor(start)`, `top: anchor(self-end)`, `inset-block-start: anchor(end)` and `inset-inline-end: anchor(self-start)` all work fine.

The story gets more complicated when using physical `<anchor-side>` parameters within logical inset property values as the physical side has to match the axis the inset property is relevant to within the current writing mode. For example:

- In a horizontal writing mode, the block direction is top-to-bottom; therefore `inset-block-end: anchor(bottom)` will work, but `inset-block-end: anchor(left)` is incompatible. If `inset-block-end: anchor(left, 50px)` were set, the computed value would be `50px`, and the positioned element would be positioned `50px` from the block end (bottom) of its nearest positioned ancestor or the viewport, depending on the `position` value set.
- In a vertical writing mode, the block direction is right-to-left or left-to-right; therefore `inset-block-end: anchor(left)` will work, but `inset-block-end: anchor(top)` is incompatible. If `inset-block-end: anchor(top, 50px)` were set, the computed value would be `50px`, and the positioned element would be positioned `50px` from the block end (left or right depending on the writing mode) of its nearest positioned ancestor or the viewport, depending on the `position` value set.

To mitigate the potential for confusion with these values, you are advised to use logical inset properties with logical `<anchor-side>` values, and physical inset properties with physical `<anchor-side>` values. You should favor the use of logical values whenever possible because they are better for internationalization.

The `center` and `<percentage>` values are valid within the `anchor()` function within all logical and physical inset properties.

The below table lists the inset properties, and the `<anchor-side>` parameter values that are compatible with them. We have only listed the longhand inset properties; these comprise the shorthand inset property values.

Inset property	**Compatible `<anchor-side>` value**
All	`center`
All	`<percentage>`
`top` and `bottom`	`top`, `bottom`, `start`, `end`, `self-start`, `self-end`
`left` and `right`	`left`, `right`, `start`, `end`, `self-start`, `self-end`
`inset-block-start` and `inset-block-end`	`start`, `end`, `self-start`, and `self-end` `top` and `bottom` in horizontal writing modes `left` and `right` in vertical writing modes
`inset-inline-start` and `inset-inline-end`	`start`, `end`, `self-start`, and `self-end` `left` and `right` in horizontal writing modes `top` and `bottom` in vertical writing modes

Using <code>anchor()</code> to position popovers

When using `anchor()` to position popovers, be aware that the default styles for popovers may conflict with the position you're trying to achieve. The usual culprits are the default styles for `margin` and `inset`, so it's advisable to reset those:

```
.positionedPopover {
  margin: 0;
  inset: auto;
}
```

The CSS working group is looking at ways to avoid requiring this workaround.

Using <code>anchor()</code> inside <code>calc()</code>

When the `anchor()` function refers to a side of the default anchor, you can include a `margin` to create spacing between the edges of the anchor and the positioned element as needed. Alternatively, you can include the `anchor()` function within a `calc()` function to add spacing.

This example positions the right edge of the positioned element flush to the anchor element's left edge, then adds margin to make some space between the edges:

```
.positionedElement {
  right: anchor(left);
  margin-right: 10px;
}
```

This example positions the positioned element's logical block end edge `10px` from the anchor element's logical block start edge:

```
.positionedElement {
  inset-block-end: calc(anchor(start) + 10px);
}
```

Positioning an element relative to multiple anchors

You can position an element relative to multiple anchors by specifying different `<anchor-name>` values inside the `anchor()` function of different inset properties on the same element (see Element positioned relative to multiple anchors below). This can be used to create useful functionality such as drag handles at the corners of a positioned element that can be used to resize it.

While a positioned element can be positioned relative to more than one anchor element, it is only ever associated with the single anchor defined via its `position-anchor` property (or the `anchor` HTML attribute). This is the anchor the element will scroll with when the page scrolls; it can also be used to control when the element is conditionally hidden.

Formal syntax

```
<anchor()> =
  anchor( <anchor-name>?                 &&
  <anchor-side> , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-side> =
  inside        |
  outside       |
  top           |
  left          |
  right         |
  bottom        |
  start         |
  end           |
  self-start    |
  self-end      |
  <percentage>  |
  center

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Common usage

In this example, the `anchor()` function is used to set the height of an anchor-positioned element to the height of its anchor by setting the bottom and top edges to the bottom and top edges of the anchor. The `anchor()` function within a `calc()` function is then used to offset the anchor-positioned element from its anchor.

HTML

We include a `<div>` element, which we'll set as our anchor, and a `<p>` element that we will position relative to that anchor:

```
<div class="anchor">⚓</div>

<p class="positionedElement">This is a positioned element.</p>
```

CSS

We set the anchor element's `anchor-name` value as the value of the positioned element's `position-anchor` property to associate the elements, then set three inset properties on the anchor-positioned element. The first two position the element's top edge flush with the top edge of the anchor and the bottom edge flush with the bottom edge of the anchor. In the third inset property, the `anchor()` function is used within a `calc()` function to position the element's left edge `10px` to the right edge of the anchor.

```
.anchor {
  anchor-name: --infobox;
  background: palegoldenrod;
  font-size: 3em;
  width: fit-content;
  border: 1px solid goldenrod;
}

.positionedElement {
  position: absolute;
  position-anchor: --infobox;
  margin: 0;
  top: anchor(top);
  left: calc(anchor(right) + 10px);
  bottom: anchor(bottom);
  background-color: olive;
  border: 1px solid darkolivegreen;
}
```

Results

Comparison of different anchor-side values

This example shows an element positioned relative to an anchor via its `top` and `left` properties, which are defined using `anchor()` functions. It also includes two drop-down menus that allow you to vary the `<anchor-side>` values inside those `anchor()` functions, so you can see what effect they have.

HTML

We specify two `<div>` elements, one with a class of `anchor` and one with a class of `infobox`. These are intended to be the anchor element and the positioned element we will associate with it, respectively.

We also include some filler text around the two `<div>` elements to make the `<body>` taller so that it will scroll. This example also includes two `<select>` elements to create the drop-down menus enabling the selection of different `<anchor-side>` values to place the positioned element with. We've hidden the filler text and the `<select>` elements for brevity.

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
```

```
<div class="anchor">⚓</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

```
<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>

<form>
  <div>
    <label for="top-anchor-side">
      Choose a vertical <code>anchor()</code> value:
    </label>
    <select id="top-anchor-side" name="top-anchor-side">
      <option value="top">top: anchor(top)</option>
      <option value="bottom" selected>top: anchor(bottom)</option>
      <option value="start">top: anchor(start)</option>
      <option value="end">top: anchor(end)</option>
      <option value="center">top: anchor(center)</option>
      <option value="0%">top: anchor(0%)</option>
      <option value="25%">top: anchor(25%)</option>
      <option value="50%">top: anchor(50%)</option>
      <option value="75%">top: anchor(75%)</option>
      <option value="100%">top: anchor(100%)</option>
    </select>
  </div>
  <div>
    <label for="left-anchor-side">
      Choose a horizontal <code>anchor()</code> value:
    </label>
    <select id="left-anchor-side" name="left-anchor-side">
      <option value="left">left: anchor(left)</option>
      <option value="right" selected>left: anchor(right)</option>
      <option value="self-start">left: anchor(self-start)</option>
      <option value="self-end">left: anchor(self-end)</option>
      <option value="center">left: anchor(center)</option>
      <option value="0%">left: anchor(0%)</option>
      <option value="25%">left: anchor(25%)</option>
      <option value="50%">left: anchor(50%)</option>
      <option value="75%">left: anchor(75%)</option>
      <option value="100%">left: anchor(100%)</option>
    </select>
  </div>
</form>
```

CSS

We declare the `anchor` `<div>` as an anchor element by setting an anchor name on it via the `anchor-name` property. We then associate it with the positioned element by setting the same value for its `position-anchor` property. `top: anchor(--my-anchor bottom)` positions the infobox's top edge flush to the bottom edge of its anchor, while `left: anchor(right)` positions the infobox's left edge flush to the right edge of its anchor. This provides an initial position that will be overwritten when different values are selected from the drop-down menus.

```
.anchor {
  font-size: 2rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: 100px;
  height: 100px;
  text-align: center;
  line-height: 100px;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
}

body {
  width: 80%;
  margin: 0 auto;
}

form {
  background: white;
  border: 1px solid black;
```

```
  padding: 5px;
  position: fixed;
  top: 0;
  right: 2px;
}

select {
  display: block;
  margin-top: 5px;
}

form div:last-child {
  margin-top: 10px;
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

```
.anchor {
  anchor-name: --my-anchor;
}

.infobox {
  position: fixed;
  position-anchor: --my-anchor;
  top: anchor(--my-anchor bottom);
  left: anchor(right);
}
```

JavaScript

We listen for the `change` event that occurs when a new `<anchor-side>` value is selected, and set the selected value as the `<anchor-side>` in the `anchor()` function within the infobox's relevant inset property (`top` or `left`) value.

```
const infobox = document.querySelector(".infobox");
const topSelect = document.querySelector("#top-anchor-side");
const leftSelect = document.querySelector("#left-anchor-side");

topSelect.addEventListener("change", (e) => {
  const anchorSide = e.target.value;
  infobox.style.top = `anchor(--my-anchor ${anchorSide})`;
});

leftSelect.addEventListener("change", (e) => {
  const anchorSide = e.target.value;
  infobox.style.left = `anchor(${anchorSide})`;
});
```

Result

Select different values from the drop-down menus to see how they affect the positioning of the infobox.

Element positioned relative to multiple anchors

This example positions an element relative to two different anchors, which are used to set the position of the top-left and bottom-right corners of the anchor-positioned element. The anchors can be moved via keyboard controls or dragged, resizing the positioned element.

HTML

We specify three `<div>` elements in total. The first two have a class of `anchor` and will be defined as anchors; each one has an individual `id` that will be used to provide them with different positioning information. The last `<div>` has a class of `infobox` and will be defined as the positioned element. We include the `tabindex` attribute to enable them to receive keyboard focus.

```
<div id="anchor1" class="anchor" tabindex="0">1.1</div>

<div id="anchor2" class="anchor" tabindex="0">1.2</div>

<div class="infobox">
  <p>This is an information box.</p>
</div>
```

CSS

```
body {
  width: 150vw;
  height: 150vh;
}

.anchor {
  font-size: 1rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  width: fit-content;
  border-radius: 10px;
  border: 1px solid black;
  padding: 3px;
  &:focus {
    background-color: hsl(60 100% 75%);
  }
}

.infobox {
  color: darkblue;
  background-color: azure;
  border: 1px solid #dddddd;
```

```
  padding: 10px;
  border-radius: 10px;
  font-size: 1rem;
}
```

The anchors are each given a different `anchor-name` value, a `position` value of `absolute`, and different inset values to position the anchors in a rectangular formation.

```
.anchor {
  position: absolute;
}

#anchor1 {
  anchor-name: --my-anchor1;
  top: 50px;
  left: 100px;
}

#anchor2 {
  anchor-name: --my-anchor2;
  top: 200px;
  left: 350px;
}
```

The anchor-positioned element, with its `position` set to `fixed`, is associated with one anchor via its `position-anchor` property. It is positioned relative to two anchors by including two different `<anchor-name>` values with the `anchor()` functions set on its inset properties. In this case, we used `<percentage>` values for the `<anchor-side>` parameter, specifying the distance from the start of the axis of the inset property on which the function is set.

```
.infobox {
  position-anchor: --my-anchor1;
  position: fixed;
  top: anchor(--my-anchor1 100%);
  left: anchor(--my-anchor1 100%);
  bottom: anchor(--my-anchor2 0%);
  right: anchor(--my-anchor2 0%);
}
```

```
// grab all the anchors and make each one draggable
const anchors = document.querySelectorAll(".anchor");
anchors.forEach((anchor) => makeDraggable(anchor));

function makeDraggable(elem) {
  let pos1, pos2, pos3, pos4;

  elem.onmousedown = dragMouseDown;
  elem.addEventListener("keyup", (e) => {
    switch (e.key) {
      case "d":
        elem.style.left = `${elem.offsetLeft + 5}px`;
        break;
      case "a":
        elem.style.left = `${elem.offsetLeft - 5}px`;
        break;
      case "w":
        elem.style.top = `${elem.offsetTop - 5}px`;
        break;
      case "s":
        elem.style.top = `${elem.offsetTop + 5}px`;
        break;
    }
    e.preventDefault();
  });

  function elementMove(e) {
    console.dir(e);
    // calculate the new cursor position:
    pos1 = pos3 - e.offsetLeft;
    pos2 = pos4 - e.offsetTop;
    pos3 = e.offsetLeft;
    pos4 = e.offsetTop;
    // set the element's new position:
    elem.style.top = `${elem.offsetTop - pos2}px`;
    elem.style.left = `${elem.offsetLeft - pos1}px`;
  }

  function dragMouseDown(e) {
    // get the mouse cursor position at startup:
    pos3 = e.clientX;
    pos4 = e.clientY;
    document.onmouseup = closeDragElement;
    // call a function whenever the mouse moves:
    document.onmousemove = elementDrag;
  }

  function elementDrag(e) {
    // calculate the new cursor position:
    pos1 = pos3 - e.clientX;
    pos2 = pos4 - e.clientY;
    pos3 = e.clientX;
    pos4 = e.clientY;
    // set the element's new position:
    elem.style.top = `${elem.offsetTop - pos2}px`;
    elem.style.left = `${elem.offsetLeft - pos1}px`;
  }

  function closeDragElement() {
    // stop moving when mouse button is released:
    document.onmouseup = null;
    document.onmousemove = null;
  }
}
```

Result

The positioned element is positioned relative to both anchor elements. Drag them with the mouse or tab to them and use the `W`, `A`, `S`, and `D` keys to move them up, down, left, and right. See how this changes their position, and as a consequence, the area of the positioned element. Scroll to see how the positions of all the elements are maintained.

> **Note:** This example is a proof-of-concept and not intended to be used in production code. Among its shortcomings, the example breaks if you try to move the anchors past each other horizontally or vertically.

See also

- `position-anchor`
- `position-area`
- `anchor-size()` function
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide
- CSS anchor positioning module

anchor-size() CSS function

The **`anchor-size()`** CSS function enables setting anchor-positioned element's size, position, and margins relative to the dimensions of anchor elements. It returns the `<length>` of a specified side of the target anchor element. `anchor-size()` is only valid when used within the value of anchor-positioned elements' sizing, inset, and margin properties.

For detailed information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

Syntax

```
/* without parameters */
anchor-size()

/* anchor size parameter */
anchor-size(width);
anchor-size(block);
anchor-size(self-inline);

/* anchor-name and anchor-side */
width: anchor-size(--my-anchor width);
block-size: anchor-size(--my-anchor block);

/* sizing relative to named anchor's side with fallback */
width: anchor-size(--my-anchor width, 50%);
block-size: anchor-size(--my-anchor block, 200px);

/* positioning relative to anchor side */
left: anchor-size(width);
inset-inline-end: anchor-size(--my-anchor height, 100px);

/* setting margin relative to anchor side */
margin-left: calc(anchor-size(width) / 4);
margin-block-start: anchor-size(--my-anchor self-block, 20px);
```

Parameters

The `anchor-size()` function's syntax is as follows:

```
anchor-size(<anchor-name> <anchor-size>, <length-percentage>)
```

The parameters are:

`<anchor-name>` Optional

The `anchor-name` property value of an anchor element you want to set the element's size, position, or margins relative to. This is a `<dashed-ident>` value. If omitted, the element's default anchor is used.

> **Note:** Specifying an `<anchor-name>` inside an `anchor-size()` function neither associates nor tethers an element to an anchor; it only defines which anchor the element's property values should be set relative to.

`<anchor-size>` Optional

Specifies the dimension of the anchor element that the positioned element's property values will be set relative to. Valid values include:

`width`

The width of the anchor element.

`height`

The height of the anchor element.

`block`

The length of the anchor element's containing block in the block direction.

`inline`

The length of the anchor element's containing block in the inline direction.

`self-block`

The length of the anchor element in the block direction.

`self-inline`

The length of the anchor element in the inline direction.

> **Note:** If this parameter is omitted, the dimension defaults to the `<anchor-size>` keyterm that matches the axis of the property in which the function is included. For example, `width: anchor-size();` is equivalent to `width: anchor-size(width);`.

`<length-percentage>` Optional

Specifies the size to use as a fallback value if the element is not absolutely or fixed positioned, or the anchor element doesn't exist. If this parameter is omitted in a case when the fallback would otherwise be used, the declaration is invalid.

> **Note:** The anchor dimension you set the positioned element's property values relative to does not have to be along the same axis as the sizing value being set. For example, `width: anchor-size(height)` is valid.

Return value

Returns a `<length>` value.

Description

The `anchor-size()` function enables a positioned element's sizing, position, and margin values to be expressed in terms of an anchor element's dimensions; it returns a `<length>` value representing the dimension of a specific anchor element the positioned element's property values are set relative to. It is a valid value for sizing, inset, and margin properties set on anchor-positioned elements.

The length returned is the vertical or horizontal size of an anchor element or its containing block. The dimension used is defined by the `<anchor-size>` parameter. If that parameter is omitted, the dimension used will match the axis of the sizing, position, or margin property is it set on. So for example:

- `width: anchor-size()` is equivalent to `width: anchor-size(width)`.
- `top: anchor-size()` is equivalent to `top: anchor-size(height)`.
- `margin-inline-end: anchor-size()` is equivalent to `margin-inline-end: anchor-size(self-inline)`. `margin-inline-end: anchor-size()` is also equivalent to `margin-inline-end: anchor-size(width)` in horizontal writing modes, or `margin-inline-end: anchor-size(height)` in vertical writing modes.

The anchor element used as the basis for the dimension length is the element with the `anchor-name` specified in the `<anchor-name>` parameter. If more than one element has the same anchor name, the last element with that anchor name in the DOM order is used.

If no `<anchor-name>` parameter is included in the function call, the element's **default anchor**, referenced in its `position-anchor` property, or associated with the element via the `anchor` HTML attribute, is used.

If an `<anchor-name>` parameter is included and there are no elements matching that anchor name, the fallback value is used. If no fallback was included, the declaration is ignored. For example, if `width: anchor-size(--foo width, 50px); height: anchor-size(--foo width);` were specified on the positioned element but no anchor named `--foo` exists in the DOM, the `width` would be `50px` and the `height` declaration would have no effect.

If an element has sizing, position, or margin properties with `anchor-size()` values set on them, but it is not an anchor-positioned element (it does not have its `position` property set to `absolute` or `fixed` or does not have an anchor associated with it via its `position-anchor` property), the fallback value will be used if one is available. If no fallback is available, the declaration is ignored.

For example, if `width: anchor-size(width, 50px);` were specified on the positioned element but no anchor was associated with it, the fallback value would be used, so `width` would get a computed value of `50px`.

For detailed information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

Properties that accept <code>anchor-size()</code> function values

The properties that accept an `anchor-size()` function as a value include:

- Sizing properties:
  - `width`
  - `height`
  - `min-width`
  - `min-height`
  - `max-width`
  - `max-height`
  - `block-size`
  - `inline-size`
  - `min-block-size`
  - `min-inline-size`
  - `max-block-size`
  - `max-inline-size`
- Inset properties:
  - `bottom`
  - `left`
  - `right`
  - `top`
  - `inset` shorthand
  - `inset-block` shorthand
  - `inset-block-end`
  - `inset-block-start`
  - `inset-inline` shorthand
  - `inset-inline-end`
  - `inset-inline-start`
- Margin properties:
  - `margin` shorthand
  - `margin-bottom`
  - `margin-left`
  - `margin-right`
  - `margin-top`
  - `margin-block` shorthand
  - `margin-block-end`
  - `margin-block-start`
  - `margin-inline` shorthand
  - `margin-inline-end`
  - `margin-inline-start`

Using <code>anchor-size()</code> inside <code>calc()</code>

The most common `anchor-size()` functions you'll use will just refer to a dimension of the default anchor. Alternative, include the `anchor-size()` function inside a `calc()` function to modify the size applied to the positioned element.

For example, this rule sizes the positioned element's width equal to the default anchor element's width:

```
.positionedElem {
  width: anchor-size(width);
}
```

This rule sizes the positioned element's inline size to 4 times the anchor element's inline size, with the multiplication being done inside a `calc()` function:

```
.positionedElem {
  inline-size: calc(anchor-size(self-inline) * 4);
}
```

Formal syntax

```
<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline

<length-percentage> =
  <length>     |
  <percentage>
```

This syntax reflects the latest standard as per CSS Anchor Positioning Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>anchor-size()</code> usage

This example shows two elements positioned relative to an anchor and sized using `anchor-size()` functions.

HTML

We specify three `<div>` elements, one `anchor` element and the two `infobox` elements we'll position relative to the anchor. We also include filler text to make the `<body>` tall enough to require scrolling, but this has been hidden for the sake of brevity.

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>
```

```
<div class="anchor">⚓</div>

<div class="infobox" id="infobox1">
  <p>This is the first infobox.</p>
</div>

<div class="infobox" id="infobox2">
  <p>This is the second infobox.</p>
</div>
```

```
<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

CSS

We declare the `anchor` `<div>` as an anchor element by giving it an `anchor-name`. The positioned elements, with their `position` properties set to `fixed`, are associated with the anchor element via their `position-anchor` properties. We also set absolute `height` and `width` dimensions on the anchor to provide a reference point when checking the positioned element dimensions, for example, with browser developer tools:

```
.anchor {
  font-size: 2rem;
  color: white;
  text-shadow: 1px 1px 1px black;
  background-color: hsl(240 100% 75%);
  text-align: center;
  align-content: center;
  outline: 1px solid black;
}

body {
  width: 80%;
  margin: 0 auto;
}

.infobox {
  align-content: center;
  color: darkblue;
  background-color: azure;
  outline: 1px solid #dddddd;
  font-size: 1rem;
  text-align: center;
}
```

```
.anchor {
  anchor-name: --my-anchor;
  width: 100px;
  height: 50px;
}

.infobox {
  position-anchor: --my-anchor;
  position: fixed;
}
```

We set some distinct property values on the positioned elements:

- The positioned elements are tethered to the anchor with different `position-area` values that position the elements in different places around the anchor element.
- The `height` of the first infobox is set to the same height as the anchor element: `anchor-size(height)` returns the anchor element's height. The element's `width` is set to double the anchor element's width using the `anchor-size()` function within a `calc()` function: `anchor-size(width)` retrieves the anchor element's width, which is then multiplied by two.
- The `height` of the second infobox is set to 80% of the anchor element's height, using a similar technique.
- Margin values are included to provide some separation from the anchor element.

```
#infobox1 {
  position-area: right;
  height: anchor-size(height);
  width: calc(anchor-size(width) * 2);
  margin-left: 5px;
}

#infobox2 {
  position-area: top span-right;
  height: calc(anchor-size(height) / 1.25);
  margin-bottom: 5px;
}
```

Result

Use your browser tools to inspect the anchor-positioned elements. The first infobox will be `50px` tall and `200px` wide, while the second infobox will be `40px` tall, with the `width` defaulting to `max-content`.

Position and margin example

See `anchor-size()` position and margin example.

See also

- `anchor-name`
- `position-anchor`
- `anchor()` function
- Using CSS anchor positioning guide
- CSS anchor positioning module

<angle> CSS type

The **`<angle>`** CSS data type represents an angle value expressed in degrees, gradians, radians, or turns. It is used, for example, in `<gradient>`s and in some `transform` functions.

Try it

```
transform: rotate(45deg);
```

```
transform: rotate(3.1416rad);
```

```
transform: rotate(-50grad);
```

```
transform: rotate(1.75turn);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This box can rotate to different angles.
  </div>
</section>
```

```
#example-element {
  background-color: #0118f3;
  padding: 0.75em;
  width: 180px;
  height: 120px;
  color: white;
}
```

Syntax

The `<angle>` data type consists of a `<number>` followed by one of the units listed below. As with all dimensions, there is no space between the unit literal and the number. While some legacy uses accept a unitless `0`, you should always include a unit for zero values, such as: `0deg`, `0grad`, `0rad`, or `0turn`.

Optionally, it may be preceded by a single `+` or `-` sign. Positive numbers represent clockwise angles, while negative numbers represent counterclockwise angles. For static properties of a given unit, any angle can be represented by various equivalent values. For example, `90deg` equals `-270deg`, and `1turn` equals `4turn`. For dynamic properties, like when applying an `animation` or `transition`, the effect will nevertheless be different.

Units

`deg`

Represents an angle in degrees. One full circle is `360deg`. Examples: `0deg`, `90deg`, `14.23deg`.

`grad`

Represents an angle in gradians. One full circle is `400grad`. Examples: `0grad`, `100grad`, `38.8grad`.

`rad`

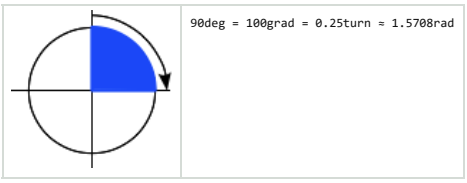
Represents an angle in radians. One full circle is 2π radians which approximates to `6.2832rad`. `1rad` is 180/π degrees. Examples: `0rad`, `1.0708rad`, `6.2832rad`.

`turn`

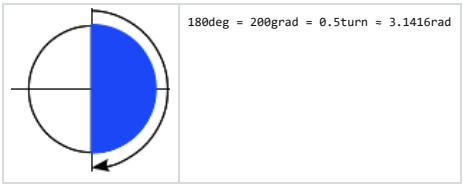
Represents an angle in a number of turns. One full circle is `1turn`. Examples: `0turn`, `0.25turn`, `1.2turn`.

Examples

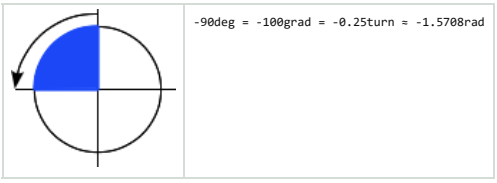
Setting a clockwise right angle



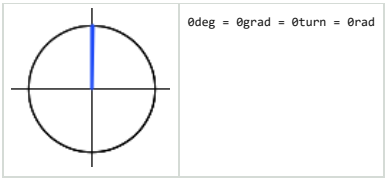
Setting a flat angle



Setting a counterclockwise right angle



Setting a null angle



See also

- CSS data types
- The `<gradient>` type
- CSS rotation transforms: `rotate()`, `rotate3d()`, `rotateX()`, `rotateY()`, and `rotateZ()`
- CSS transforms
- Using CSS transforms
- Using CSS gradients

<angle-percentage> CSS type

The **`<angle-percentage>`** CSS data type represents a value that can be either an `<angle>` or a `<percentage>`.

Where an `<angle-percentage>` is specified as an allowable type, this means that the percentage resolves to an angle and therefore can be used in a `calc()` expression.

Syntax

Refer to the documentation for `<angle>` and `<percentage>` for details of the individual syntaxes allowed by this type.

Formal syntax

```
<angle-percentage> =
  <angle>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- CSS data types
- Using CSS gradients
- `conic-gradient()` and `repeating-conic-gradient()`

<animation-action>

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`<animation-action>`** enumerated data type represents keyword values that specify how an animation should behave in certain circumstances — for example, how a triggered animation should behave when its trigger is activated and deactivated.

The `<animation-action>` keyword values are used in the following properties:

- `animation-trigger`

Syntax

The `<animation-action>` enumerated type is specified using one of the following values:

`none`

No action is specified for the animation.

`play`

The animation will play, resume (if paused), or restart (if currently finished) in the same direction as its current direction of play.

`play-forwards`

As `play`, except that the animation's `playbackRate` is adjusted if required (flipping it to positive if negative) so that the animation will play forwards.

`play-backwards`

As `play`, except that the animation's `playbackRate` is adjusted if required (flipping it to negative if positive) so that the animation will play in reverse.

`play-once`

As `play`, except that once the animation has played through all its iterations, it won't be triggered again. Like `play`, if an animation is paused, `play-once` will resume playing it; unlike `play`, it won't replay a finished animation.

`pause`

The animation will pause.

`replay`

As `play`, except that the animation is put back to the start.

`reset`

As `pause`, except that the animation is put back to the start.

Description

The `<animation-action>` type specifies how an animation behaves when certain things happen. For example, when setting an `animation-trigger` value on an animated element to specify the animation as a triggered animation, the value can include one or two `<animation-action>` values, separated by a space. The first value specifies the animation behavior when its trigger activates, while the optional second value specifies the animation behavior when its trigger deactivates. If you only specify a single value, the animation doesn't change its behavior when its trigger deactivates; it continues with the activation behavior. It has the same effect as setting `none` as the second value.

There are some common patterns to observe:

- `play-forwards play-backwards` is very common when you want a UI element to "animate in" when it scrolls into view, and then "animate out" again when it scrolls out of view.
- `play pause` is common for animating an element as it scrolls into view, then pausing the animation as it scrolls out of view.
- `play-once` is often used on its own, when you want an animation to play only once when it scrolls into view.

The eight `<animation-action>` values provide different animation behaviors. It's important to understand how they behave on their own and the effects that can created by changing trigger activation and deactivation values.

Specifying no action

To specify that no action should occur, use the `none` value.

Playing the animation

The `play`, `play-forwards`, `play-backwards`, and `play-once` keyword values all cause the animation to play, but each one specifies a different precise behavior.

`play`

Setting `play` will play the animation through all its iterations, as defined by the `animation-iteration-count` property.

If only `play` is set, the animation will play on activation but never deactivate, as a deactivation action is not specified.

```
animation-trigger: --t play;
```

If `play` is combined with `pause`, `replay` or `reset`, the animation will play on activation, and then `pause`, `replay` or `reset` on deactivation. On subsequent activation, the animation will play again.

```
animation-trigger: --t play reset;
```

When combining `play` with `play-backwards`, the animation will play on activation, then play backward through all the iterations it previously played forwards through on deactivation:

```
animation-trigger: --t play play-backwards;
```

While valid, combining `play` with `play-once` is unnecessary, as it behaves the same as `play`. Similarly, combining `play` with `play-forwards` is unnecessary, because `play-forwards` plays the animation in the same direction as `play`, even when `animation-direction` is set to `reverse` or `alternate`.

`play-forwards` and `play-backwards`

Setting `play-forwards` and `play-backwards` will play the animation through all its iterations, except that the direction of play will change to forwards or backward, respectively. This is achieved by adjusting the animation's `playbackRate`; the `animation-direction` value is not affected.

Specifying an activation action of `play-forwards` only has the same effect as specifying `play` only:

```
animation-trigger: --t play-forwards;
```

Combining `play-forwards` with `play-backwards` causes the animation to play forwards on activation, then play backward through all the iterations it previously played forwards through on deactivation. On subsequent activations, the animation starts playing forwards again.

```
animation-trigger: --t play-forwards play-backwards;
```

Combining `play-forwards` with `pause`, `replay` or `reset` produces the same effect as `play`: the animation will play on activation, and then `pause`, `replay` or `reset` on deactivation. On subsequent activations, the animation will play again.

```
animation-trigger: --t play-forwards pause;
```

It doesn't make sense to combine `play-forwards` with `play` or `play-once`, because all of these actions effectively play the animation forwards; doing the same thing on deactivation as on activation has no discernible effect.

Note that using `play-backwards` as an activation action has no effect if the animation is already at the start of its iterations. With the following example, the animation does not play because it is already at the start:

```
animation-trigger: --t play-backwards;
```

`play-once`

Setting `play-once` will play the animation through all its iterations, but only once. If `animation-iteration-count` is set to `infinite`, there is not much difference in effect between `play-once` and `play`/`play-forwards`. However, with `animation-iteration-count` set to a finite number, you will observe the following behavior.

When combining `play-once` with `pause`, the animation will play on activation and then `pause` on deactivation. However, once the animation has played through all its iterations, the animation will not play again on subsequent activations.

```
animation-trigger: --t play-once pause;
```

If you combine `play-once` with `replay`, the animation will play on activation, then play again from the beginning on deactivation. It won't exceed its iteration count on any playthrough, but it will play again on subsequent deactivations because we reset the animation to the start each time. On subsequent activations, however, the animation will not play again.

```
animation-trigger: --t play-once replay;
```

If `play-once` is combined with `reset`, the animation plays on activation, then resets to the beginning on deactivation. On subsequent activation, the animation will play again.

```
animation-trigger: --t play-once reset;
```

If `play-once` is combined with `play-backwards`, the animation plays on activation, then plays backward through all the iterations on deactivation. On subsequent activation, the animation will not play again, but the animation will play backward again on subsequent deactivations.

```
animation-trigger: --t play-once play-backwards;
```

Pausing the animation

The `pause` value pauses the animation at whatever point it reached in its playback upon activation/deactivation. Using this in combination with other values was discussed earlier, although one case not mentioned was using `pause` as the activation action, for example:

```
animation-trigger: --t pause play;
```

This has an interesting effect of no playback on activation, but playback on subsequent deactivation: useful if you want an animation to play only when the subject leaves the scrollport.

It doesn't make sense to combine `pause` with `reset` because both effectively pause the animation, except that `reset` puts the animation back to the start. If the animation has not been played, `reset` has no discernible effect.

Resetting the animation

The `replay` and `reset` values are similar to `pause`, except that:

- `reset` pauses the animation but also puts it back to the start.
- `replay` puts the animation back to the start and then starts playing it again.

We previously discussed using these values in combination with other values. However, one case was not mentioned: using them as the activation action.

For example:

```
animation-trigger: --t replay pause;
```

This has an interesting effect: it plays on activation (the same as an action like `play`) and pauses on deactivation. However, on subsequent activation, it will play from the start again, regardless of the previous play state. This is useful if you want an animation to play when the subject enters the scrollport, pause when it leaves the scrollport, but then play from the start on each subsequent entry.

Another interesting example is as follows:

```
animation-trigger: --t reset play;
```

This has an interesting effect: it doesn't play on activation but plays on deactivation. On subsequent activation, it resets to progress `0`, regardless of the previous play state. This is useful if you want an animation to play when the subject leaves the scrollport, and then reset to the beginning on each subsequent entry.

Equivalence with the Web Animations API

The behavior specified by the various `<animation-action>` keywords is equivalent to calling various methods of the Web Animations API on the specified animation, as follows:

`play`

: Equivalent to calling `Animation.play()` on the animation.

`play-forwards`

: Equivalent to setting the `Animation.playbackRate` of a playing animation to its absolute positive value.

`play-backwards`

: Equivalent to setting the `Animation.playbackRate` of a playing animation to its absolute positive value multiplied by `-1`.

`play-once`

: Equivalent to calling `Animation.play()` on the animation except that it only plays once.

`pause`

: Equivalent to calling `Animation.pause()` on the animation.

`replay`

Equivalent to setting the `Animation.overallProgress` of a playing animation to `0`.

`reset`

Equivalent to setting the `Animation.overallProgress` of a paused animation to `0`.

Formal syntax

```
<animation-action> =
  none            |
  pause           |
  play            |
  play-backwards  |
  play-forwards   |
  play-once       |
  replay          |
  reset
```

Examples

Basic usage

This example shows how to create a basic scroll-triggered animation that plays forwards on trigger activation and backward on trigger deactivation.

HTML

Our markup contains two `<div>` elements, one to animate and one to create a trigger, plus some basic text content to scroll the page. We have hidden the text content for brevity.

```
<div class="animated">I am animated</div>

...

<div class="trigger">I create the trigger</div>

...
```

```
<div class="animated">I am animated</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<div class="trigger">I create the trigger</div>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

We give the `.animated` element a `position` of `fixed`, positioning it near the top-left of the scrollport so we can see when its animation starts and stops.

```
body {
  width: 80%;
```

```
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  height: 100px;
  border: 5px solid black;
}

.animated {
  width: 100px;
  background: orange;
}

.trigger {
  background: wheat;
}
```

```
.animated {
  position: fixed;
  top: 25px;
  left: 25px;
}
```

Next, we define the `@keyframes` for a `rotate` animation:

```
@keyframes rotate {
  from {
    rotate: 0deg;
  }

  to {
    rotate: 360deg;
  }
}
```

The `.animated` element has the `rotate` animation applied. We then give it an `animation-trigger` value that references a `timeline-trigger-name` of `--t` with two `<animation-action>` values, `play-forwards` and `play-backwards`. These specify that the animation will play on activation and play in reverse on deactivation.

```
.animated {
  animation: rotate 1.5s infinite linear both;
  animation-trigger: --t play-forwards play-backwards;
}
```

The `.trigger` element creates the animated `<div>`'s trigger using a `timeline-trigger` value of `--t view()`. This value includes the identifier referenced in the animated `<div>`'s `animation-trigger` property value (the `timeline-trigger-name`), associating the two together. It also includes:

- A `timeline-trigger-source` value of `view()`, which sets the timeline trigger as a view progress timeline, and the element providing the timeline trigger as the nearest scrolling ancestor element.
- A `timeline-trigger-activation-range` value of `contain`, which means that the trigger will activate when the `.trigger` element is fully inside the scrollport. Because `timeline-trigger-active-range` defaults to `auto`, its value is the same as the activation range; the trigger therefore deactivates when it stops being fully inside the scrollport.

```
.trigger {
  timeline-trigger: --t view() contain;
}
```

Result

Try scrolling the content up. When the tracked `<div>` fully appears in the scrollport, the animation plays; when it starts to leave the scrollport at either edge, the animation plays in reverse.

Comparing the <code><animation-action></code> values

This example provides a comparison of the various `<animation-action>` values. By applying the same rotation animation to identical side-by-side elements and varying the `animation-trigger` values, you can compare and contrast the effects of the different actions.

HTML

We include a `<section>` element containing five `<div>` elements, each with a number inside. We also include text content to make the page scroll, which we've hidden for brevity.

```
<section>
  <div class="one">1</div>
  <div class="two">2</div>
  <div class="three">3</div>
  <div class="four">4</div>
  <div class="five">5</div>
</section>
```

```
<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
```

```
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<section>
  <div class="one">1</div>
  <div class="two">2</div>
  <div class="three">3</div>
  <div class="four">4</div>
  <div class="five">5</div>
</section>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>

<p>
  Fusce dictum ex quis ipsum consectetur placerat. Cras sed lectus ex. Quisque
  purus dolor, vulputate ac mi eget, commodo varius odio. Suspendisse faucibus
  ipsum vel libero finibus, in placerat nibh congue. Sed iaculis, metus et
  euismod posuere, mi diam vestibulum felis, ac vulputate eros ipsum id justo.
  Etiam a tincidunt purus. Maecenas semper sed enim at blandit. Aenean ut
  sagittis lorem, eget gravida purus. Phasellus eleifend, lectus nec pulvinar
  facilisis, dui dolor feugiat odio, iaculis tempor felis est non tortor. In
  suscipit lorem efficitur molestie tempus. Integer sit amet neque et risus
  iaculis sodales sed eget diam. Quisque sodales nunc sapien, vitae lacinia ex
  luctus quis. Maecenas scelerisque scelerisque elit eu consequat. Etiam ac
  tristique tellus, sed tincidunt velit.
</p>
```

CSS

We apply the same `animation` to each `<div>` element—we play the `rotate` animation infinitely, with each iteration lasting two seconds. We also style each `<div>` to be a `50px` diameter colored circle.

```
body {
  width: 80%;
  margin: 0 auto;
  font-family: "Helvetica", "Arial";
  font-size: 1.3rem;
}

div {
  display: flex;
  justify-content: center;
}

section {
  display: flex;
  justify-content: space-between;
}
```

```
div {
  animation: rotate 2s infinite linear both;
  height: 50px;
  width: 50px;
  border: 5px solid black;
  border-radius: 50%;
  background-color: orange;
}
```

Next, we set the `<section>` element to create an animation trigger, with a `timeline-trigger` value of `--t view() contain 20% contain 80%`. There is nothing unusual here, except that we've set a `timeline-trigger-activation-range` value and defaulted the `timeline-trigger-active-range` to `contain 20% contain 80%`. This means the trigger activates when the `<section>` element has scrolled around `20%` of the way up the scrollport, and deactivates when it has scrolled around `80%` of the way up the scrollport. This allows you to see the `<animation-action>` effects more clearly than if the activation range covered the entire scrollport.

```
section {
  timeline-trigger: --t view() contain 20% contain 80%;
}
```

Next, we set a different `animation-trigger` property value on each `<div>` element. Each one references the `<section>` element's `timeline-trigger-name`, but each one has a different set of `<animation-action>` values applied. The last `<div>` additionally has a new `animation` property value applied, overriding the one we set earlier. It is the same as the original `animation` property, except that the iteration count is set to `1` rather than `infinite`. It is easier to demonstrate the effect of `play-once` when `animation-iteration-count` is not `infinite` (if this were the case, it would play forever, regardless).

```
.one {
  animation-trigger: --t play-forwards play-backwards;
}

.two {
  animation-trigger: --t play;
}

.three {
  animation-trigger: --t play replay;
}

.four {
  animation-trigger: --t pause play;
}
```

```
.five {
  animation: rotate 2s 1 linear both;
  animation-trigger: --t play-once reset;
}
```

```
@supports not (animation-trigger: --t play-once reset) {
  body::before {
    content: "Your browser does not support scroll-triggered animations.";
    background-color: wheat;
    text-align: center;
    padding: 1rem 0;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Scroll down to the point where the `<section>` and `<div>` elements enter the scrollport. Move them through the start and end of the trigger activation range, concentrating on a different `<div>` each time, to visualize the effects of each set of `<animation-action>`s.

The effects are:

1. The first `<div>` (far left) has `play-forwards play-backwards` set. When the tracked element enters the activation range, the animation plays forward. When it leaves the activation range (at the top or bottom of the scrollport), it starts playing backward.
2. The second `<div>` has a single `<animation-action>` set — `play`. When the tracked element enters the activation range, the animation starts to play. However, because no deactivation action is set to change its behavior, the animation continues to play forever until the page reloads.
3. The third `<div>` has `play replay` set. When the tracked element enters the activation range, the animation starts playing forward. When it leaves the activation range, the animation resets to progress `0` and then starts playing again.
4. The fourth `<div>` has `pause play` set. When the tracked element enters the activation range, the animation continues not to play due to the paused state. However, when it leaves the activation range, the animation starts to play. From then on, the animation pauses when the tracked element is inside the activation range and plays when it is outside it.
5. The fifth `<div>` (far right, with an iteration count of `1`) has `play-once reset` set. When the tracked element enters the activation range, the animation plays once. When it leaves the activation range, the animation resets to progress `0` and pauses. From then on, the animation plays once whenever the tracked element enters the activation range and resets when it leaves the activation range.

See also

- `animation-trigger`
- `timeline-trigger` shorthand property
- `timeline-trigger-name`, `timeline-trigger-source`, `timeline-trigger-activation-range`, and `timeline-trigger-active-range`
- `trigger-scope`
- Using CSS scroll-triggered animations
- CSS animation triggers module
- CSS animations module

asin() CSS function

The **`asin()`** CSS function is a trigonometric function that returns the inverse sine of a number between `-1` and `1`. The function contains a single calculation that returns an `<angle>` between `-90deg` and `90deg`.

Syntax

```
/* Single <number> values */
transform: rotate(asin(-0.2));
transform: rotate(asin(2 * 0.125));

/* Other values */
transform: rotate(asin(pi / 5));
transform: rotate(asin(e / 3));
```

Parameters

The `asin(number)` function accepts only one value as its parameter.

`number`

A calculation which resolves to a `<number>` between `-1` and `1`.

Return value

The inverse sine of a `number` will always return an `<angle>` between `-90deg` and `90deg`.

- If `number` is less than `-1` or greater than `1`, the result is `NaN`.
- If `number` is `0⁻`, the result is `0⁻`.

Formal syntax

```
<asin()> =
  asin( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotate elements

The `asin()` function can be used to `rotate` elements as it returns an `<angle>`.

HTML

```
<div class="box box-1"></div>
<div class="box box-2"></div>
<div class="box box-3"></div>
<div class="box box-4"></div>
<div class="box box-5"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

```
div.box {
  width: 100px;
  height: 100px;
  background: linear-gradient(orange, red);
}
div.box-1 {
  transform: rotate(asin(1));
}
div.box-2 {
  transform: rotate(asin(0.5));
}
div.box-3 {
  transform: rotate(asin(0));
}
div.box-4 {
  transform: rotate(asin(-0.5));
}
div.box-5 {
  transform: rotate(asin(-1));
}
```

Result

See also

- `sin()`
- `cos()`
- `tan()`
- `acos()`
- `atan()`
- `atan2()`

atan() CSS function

The **`atan()`** CSS function is a trigonometric function that returns the inverse tangent of a number between `-∞` and `+∞`. The function contains a single calculation that returns an `<angle>` between `-90deg` and `90deg`.

Syntax

```
/* Single <number> values */
transform: rotate(atan(1));
transform: rotate(atan(4 * 50));

/* Other values */
transform: rotate(atan(pi / 2));
transform: rotate(atan(e * 3));
```

Parameters

The `atan(number)` function accepts only one value as its parameter.

`number`

A calculation which resolves to a `<number>` between `-∞` and `+∞`.

Return value

The inverse tangent of a `number` will always return an `<angle>` between `-90deg` and `90deg`.

- If `number` is `0⁻`, the result is `0⁻`.
- If `number` is `+∞` the result is `90deg`.
- If `number` is `-∞` the result is `-90deg`.

That is:

- `atan(-infinity)` representing `-90deg`.
- `atan(-1)` representing `-45deg`
- `atan(0)` representing `0deg`
- `atan(1)` representing `45deg`
- `atan(infinity)` representing `90deg`.

Formal syntax

```
<atan()> =
  atan( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotate elements

The `atan()` function can be used to `rotate` elements as it returns an `<angle>`.

HTML

```
<div class="box box-1"></div>
<div class="box box-2"></div>
<div class="box box-3"></div>
<div class="box box-4"></div>
<div class="box box-5"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

```
div.box {
  width: 100px;
  height: 100px;
  background: linear-gradient(orange, red);
}
div.box-1 {
  transform: rotate(atan(-99999));
}
div.box-2 {
  transform: rotate(atan(-1));
}
div.box-3 {
  transform: rotate(atan(0));
}
div.box-4 {
  transform: rotate(atan(1));
}
div.box-5 {
  transform: rotate(atan(99999));
}
```

Result

See also

- `sin()`
- `cos()`
- `tan()`
- `asin()`
- `acos()`
- `atan2()`

atan2() CSS function

The **`atan2()`** CSS function is a trigonometric function that returns the inverse tangent of two values between `-infinity` and `infinity`. The function accepts two arguments and returns an `<angle>` between `-180deg` and `180deg`, without implying any specific unit such as radians.

Syntax

```
/* Two <number> values */
transform: rotate(atan2(3, 2));

/* Two <dimension> values */
transform: rotate(atan2(1rem, -0.5rem));

/* Two <percentage> values */
transform: rotate(atan2(20%, -30%));

/* Other values */
```

```
transform: rotate(atan2(pi, 45));
transform: rotate(atan2(e, 30));
```

Parameters

The `atan2(y, x)` function accepts two comma-separated values as its parameters. Each value can be a `<number>`, a `<dimension>`, or a `<percentage>`. Both values must be of the same type, although if they are `<dimension>` they can be of different units (example: `atan2(100px, 5vw)` is valid).

`y`

The y-coordinate of the point. A calculation which resolves to a `<number>`, a `<dimension>`, or a `<percentage>`.

`x`

The x-coordinate of the point. A calculation which resolves to a `<number>`, a `<dimension>`, or a `<percentage>`.

Return value

Given two values `x` and `y`, the function `atan2(y, x)` calculates and returns the `<angle>` between the positive x-axis and the ray from the origin to the point `(x, y)`.

Formal syntax

```
<atan2()> =
  atan2( <calc-sum> , <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotate elements

The `atan2()` function can be used to `rotate` elements as it returns an `<angle>`.

HTML

```
<div class="box box-1"></div>
<div class="box box-2"></div>
<div class="box box-3"></div>
<div class="box box-4"></div>
<div class="box box-5"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

```
div.box {
  width: 100px;
  height: 100px;
  background: linear-gradient(orange, red);
}
div.box-1 {
  transform: rotate(atan2(3, 2));
}
div.box-2 {
  transform: rotate(atan2(3%, -2%));
}
div.box-3 {
  transform: rotate(atan2(-1, 0.5));
}
div.box-4 {
  transform: rotate(atan2(1, 0.5));
}
div.box-5 {
  transform: rotate(atan2(1rem, -0.5rem));
}
```

Result

See also

- `sin()`
- `cos()`
- `tan()`
- `asin()`
- `acos()`
- `atan()`

- Using CSS typed arithmetic

attr() CSS function

> **Note:** The `attr()` function can be used with any CSS property, but support for properties other than `content` is experimental.

The **`attr()`** CSS function is used to retrieve the value of an attribute of the selected element and use it in a property value, similar to how the `var()` function substitutes a custom property value. It can also be used with pseudo-elements, in which case the attribute's value on the pseudo-element's originating element is returned.

Try it

```
blockquote {
  margin: 1em 0;
}

blockquote::after {
  display: block;
  content: " (source: " attr(cite) ") ";
  color: hotpink;
}
```

```
<blockquote cite="https://mozilla.org/en-US/about/">
  Mozilla is working to put control of the internet back in the hands of the
  people using it.
</blockquote>

<blockquote cite="https://web.dev/about/">
  Build beautiful, accessible, fast, and secure websites that work
  cross-browser.
</blockquote>
```

Syntax

```
/* Basic usage */
attr(data-count)
attr(href)

/* With type */
attr(data-width px)
attr(data-size rem)
attr(data-name raw-string)
attr(id type(<custom-ident>))
attr(data-count type(<number>))
attr(data-size type(<length> | <percentage>))

/* With fallback */
attr(data-count type(<number>), 0)
attr(data-width px, inherit)
attr(data-something, "default")

/* With namespace */
attr(color|myAttr type(*), red)
```

Parameters

The `attr()` function's syntax is as follows:

```
attr(<attr-name> <attr-type>? , <fallback-value>?)
```

The parameters are:

`<attr-name>`

The attribute name whose value should be retrieved from the selected HTML element(s).

Namespaces

The attribute name can contain a `namespace` which allows the targeting elements of XML-based markup languages such as SVG or MathML.

```
@namespace svg url("http://www.w3.org/2000/svg");
a {
  fill: attr(svg|myAttr type(*), green);
}
```

> **Note:** If no namespace is specified (just an identifier is given, like `attr(foo)`), the null namespace is implied. This is usually what's desired, as namespaced attributes are rare. As with attribute selectors, the case-sensitivity of `<attr-name>` depends on the document language.

`<attr-type>`

Specifies how the attribute value is parsed into a CSS value. This can be the `raw-string` keyword, a `type()` function, or a CSS dimension unit (specified using an `<attr-unit>` identifier). When omitted, it defaults to `raw-string`.

`raw-string`

The `raw-string` keyword causes the attribute's literal value to be treated as the value of a CSS string, with no CSS parsing performed (including CSS escapes, whitespace removal, comments, etc.). The `<fallback-value>` is only used if the attribute is omitted; specifying an empty value doesn't trigger the fallback.

```
attr(data-name raw-string, "stranger")
```

> **Note:** This keyword was originally named and supported in Chromium browsers as `string`. Both keywords will be supported briefly, for backward compatibility purposes.

`type()`

The `type()` function takes a `<syntax>` as its argument that specifies what data type to parse the value into.

> **Note:** For security reasons `<url>` is not allowed as an `attr()` data type.

`<attr-unit>`

The `<attr-unit>` identifier specifies the unit a numeric value should have (if any). It can be the `%` character (percentage) or a CSS distance unit such as `px`, `rem`, `deg`, `s`, etc.

```
attr(data-size rem)
attr(data-width px, inherit)
attr(data-rotation deg)
```

`<fallback-value>`

The value to be used if the specified attribute is missing or contains an invalid value.

Return value

The return value of `attr()` is the value of the HTML attribute whose name is `<attr-name>` parsed as the given `<attr-type>` or parsed as a CSS string.

When an `<attr-type>` is set, `attr()` will try to parse the attribute into that specified `<attr-type>` and return it. If the attribute cannot be parsed into the given `<attr-type>`, the `<fallback-value>` will be returned instead. When no `<attr-type>` is set, the attribute will be parsed into a CSS string.

If no `<fallback-value>` is set, the return value will default to an empty string when no `<attr-type>` is set or the guaranteed-invalid value when an `<attr-type>` is set.

Description

Limitations and security

The `attr()` function can reference attributes that were never intended for styling use and might contain sensitive information (for example, a security token used by scripts on the page). In general, this is fine, but it can become a security threat when used in URLs.

For this reason, you can't use `attr()` to dynamically construct URLs:

```
<!-- This won't work! -->
<span data-icon="https://example.org/icons/question-mark.svg">help</span>
```

```
span[data-icon] {
  background-image: url(attr(data-icon));
}
```

This restriction also applies to any context that could potentially result in a `<url>` value. Values that use `attr()` get marked as *`attr()`-tainted*, and using them as or in a `<url>` makes a declaration become "invalid at computed value time" (IACVT).

So, for example, functions like `image-set()` that take values that resolve to `<url>` also won't work:

```
span[data-icon] {
  background: image-set(attr(data-icon));
}
```

Backwards compatibility

Generally speaking, the modern `attr()` syntax is backward-compatible because the old way of using it — without specifying an `<attr-type>` — behaves the same as before. Having `attr(data-attr)` in your code is the same as writing `attr(data-attr type(<string>))` or the simpler `attr(data-attr string)`).

However, there are two edge cases where the modern `attr()` syntax behaves differently from the old syntax.

In the following snippet, browsers that don't support the modern `attr()` syntax will discard the second declaration because they cannot parse it. The result in those browsers is `"Hello World"`.

```
<div text="Hello"></div>
```

```
div::before {
  content: attr(text) " World";
}
div::before {
  content: attr(text) 1px;
}
```

In browsers with support for the modern syntax, the output will be … nothing. Those browsers will successfully parse the second declaration, but because it is invalid content for the `content` property, the declaration becomes "invalid at computed value time" or IACVT for short.

To prevent this kind of situation, feature detection is recommended.

A second edge case is the following:

```
<div id="parent"><div id="child" data-attr="foo"></div></div>
```

```
#parent {
  --x: attr(data-attr);
}
#child::before {
  content: var(--x);
}
```

Browsers without support for modern syntax display the text `"foo"`. In browsers with modern `attr()` support there is no output.

This is because `attr()` — similar to custom properties that use the `var()` function — gets substituted at computed value time. With the modern behavior, `--x` first tries to read the `data-attr` attribute from the `#parent` element, which results in an empty string because there is no such attribute on `#parent`. That empty string then gets inherited by the `#child` element, resulting in a `content: ;` declaration being set.

To prevent this sort of situation, don't pass inherited `attr()` values onto children unless you explicitly want to.

Feature detection

You can feature detect support for modern `attr()` syntax using the `@supports` at-rule. In the test, try to assign an advanced `attr()` to a (non-custom) CSS property.

For example:

```
@supports (x: attr(x type(*))) {
  /* Browser has modern attr() support */
}

@supports not (x: attr(x type(*))) {
  /* Browser does not have modern attr() support */
}
```

We can perform the same check in JavaScript with `CSS.supports()`:

```
if (CSS.supports("x: attr(x type(*))")) {
  /* Browser has modern attr() support */
}

if (!CSS.supports("x: attr(x type(*))")) {
  /* Browser does not have modern attr() support */
}
```

Formal syntax

```
<attr()> =
  attr( <attr-name> <attr-type>? , <declaration-value>? )

<attr-name> =
  [ <ident-token>? '|' ]? <ident-token>

<attr-type> =
  type( <syntax> )  |
  raw-string        |
  number            |
  <attr-unit>

<syntax> =
  '*'                                              |
  <syntax-component> [ <syntax-combinator> <syntax-component> ]*  |
  <syntax-string>

<attr-unit> =
  <custom-ident>

<syntax-component> =
  <syntax-single-component> <syntax-multiplier>?  |
  '<' transform-list '>'

<syntax-combinator> =
  '|'

<syntax-string> =
  <string>

<syntax-single-component> =
  '<' <syntax-type-name> '>'  |
  <ident>

<syntax-multiplier> =
  '#'  |
  '+'

<syntax-type-name> =
  angle               |
  color               |
  custom-ident        |
  image               |
  integer             |
  length              |
  length-percentage   |
  number              |
  percentage          |
  resolution          |
  string              |
  time                |
  url                 |
  transform-function
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

content property

In this example, we prepend the value of the `data-foo` `data-*` global attribute to the contents of the `<p>` element.

HTML

```
<p data-foo="hello">world</p>
```

CSS

```
[data-foo]::before {
  content: attr(data-foo) " ";
}
```

Result

Using a fallback value

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

In this example, we append the value of `data-browser` `data-*` global attribute to the `<p>` element. If the `data-browser` attribute is missing from the `<p>` element, we append the *fallback* value of "**Unknown**".

HTML

```
<p data-browser="Firefox">My favorite browser is:</p>
<p>Your favorite browser is:</p>
```

CSS

```
p::after {
  content: " " attr(data-browser, "Unknown");
  color: tomato;
}
```

Result

color value

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

In this example, we set the CSS value of `background-color` to the value of the `data-background` `data-*` global attribute assigned to the `<div>` element.

HTML

```
<div class="background" data-background="lime">
  background expected to be red if your browser does not support advanced usage
  of attr()
</div>
```

CSS

```
.background {
  height: 100vh;
}
```

```
.background {
  background-color: red;
}

.background[data-background] {
  background-color: attr(data-background type(<color>), red);
}
```

Result

Using dimension units

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

In this example the `data-rotation` attribute is parsed into a `deg` unit, which specifies the element's rotation.

HTML

```
<div data-rotation="-3">I am rotated by -3 degrees</div>
<div data-rotation="2">And I by 2 degrees</div>
<div>And so am I, using the fallback value of 1.5deg</div>
```

CSS

```
body {
  min-height: 100svh;
  display: grid;
  place-content: center;
  gap: 1em;
}
div {
  margin: 0 auto;
  border: 1px solid;
  border-radius: 0.25em;
  padding: 0.5em;
}
```

```
div {
  width: fit-content;
  transform-origin: 50% 50%;
  rotate: attr(data-rotation deg, 1.5deg);
}
```

Result

Parsing <code>attr()</code> values as <code><custom-ident></code>s

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

In this example, the values for the `view-transition-name` property are derived from the `id` attribute of the element. The attribute gets parsed into a `<custom-ident>`, which is what `view-transition-name` accepts as a value.

The resulting values for `view-transition-name` are `card-1`, `card-2`, `card-3`, etc.

HTML

The HTML contains four cards with different `id` attributes and a "Shuffle cards" `<button>`, which shuffles the cards.

```
<div class="cards">
  <div class="card" id="card-1">1</div>
  <div class="card" id="card-2">2</div>
  <div class="card" id="card-3">3</div>
  <div class="card" id="card-4">4</div>
</div>
<button>Shuffle cards</button>
```

```
<div class="warning">
  <p>
    Your browser does not support advanced <code>attr()</code>. As a result,
    this demo won't do anything.
  </p>
</div>
```

CSS

The cards are laid out in a flex container:

```
.cards {
  display: flex;
  flex-direction: row;
  gap: 1em;
  padding: 1em;
}
```

```
:root {
  view-transition-name: none;
}
::view-transition {
  pointer-events: none;
}

@supports (x: attr(x type(*))) {
  .warning {
    display: none;
  }
}

@layer layout {
  .card {
    border-radius: 0.25em;
    width: 20vw;
    max-width: 5em;
    aspect-ratio: 1 / 1.6;
    background: lightgrey;

    display: grid;
    place-content: center;
    font-size: 2em;
  }

  * {
    box-sizing: border-box;
  }

  body {
    min-height: 100svh;
    display: grid;
    place-content: center;
  }

  button {
    justify-self: center;
  }
}

@layer warning {
  .warning {
    padding: 1em;
    border: 1px solid #cccccc;
    background: rgb(255 255 205 / 0.8);
    text-align: center;
    order: -1;
    margin: 1em;
  }

  .warning > :first-child {
    margin-top: 0;
  }
  .warning > :last-child {
    margin-bottom: 0;
  }
}
```

On each card, the `attr()` function gets the `id` attribute and parses it into a `<custom-ident>`, which is used as the value for the `view-transition-name` property. When there is no `id` set on a card, the fallback value `none` is used instead.

```
.card {
  view-transition-name: attr(id type(<custom-ident>), none);
  view-transition-class: card;
}
```

JavaScript

When the `<button>` is pressed, the cards are shuffled. This is done by randomizing the order of an array containing references to all the cards and then updating the `order` property of each card to its new array index position.

To animate each card to its new position, View Transitions are used. This is done by wrapping the `order` update in a call to `document.startViewTransition`.

```
const shuffle = (array) => {
  for (let i = array.length - 1; i >= 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [array[i], array[j]] = [array[j], array[i]];
  }
};

document.querySelector("button").addEventListener("click", (e) => {
  const $cards = Array.from(document.querySelectorAll(".card"));
  shuffle($cards);
  document.startViewTransition(() => {
    $cards.forEach(($card, i) => {
      $card.style.setProperty("order", i);
    });
  });
});
```

Result

See also

- Attribute selectors
- HTML `data-*` attributes
- SVG `data-*` attributes

<axis> CSS type

The **`<axis>`** enumerated data type specifies the scrolling direction of the scroll container controlling a scroll timeline.

The `<axis>` keyword values are used in the following CSS functions and properties:

- `scroll-timeline-axis`
- `scroll-timeline` shorthand
- `scroll()`
- `view-timeline-axis`
- `view-timeline` shorthand
- `view()`

Syntax

Valid `<axis>` values:

`block`

> The block axis of the scroller element, which is the axis in the direction perpendicular to the flow of text within a line. For horizontal writing modes, such as standard English, this is the same as `y`, while for vertical writing modes, it is the same as `x`.

`inline`

> The inline axis of the scroller element, which is the axis in the direction parallel to the flow of text in a line. For horizontal writing modes, this is the same as `x`, while for vertical writing modes, this is the same as `y`.

`x`

> The horizontal axis of the scroller element.

`y`

> The vertical axis of the scroller element.

Formal syntax

```
<axis> =
  block   |
  inline  |
  x       |
  y
```

See also

- `animation-timeline`
- Scroll-driven animation timelines guide
- CSS scroll-driven animations module

<baseline-position> CSS type

The **`<baseline-position>`** enumerated value type represents the `baseline` keyword values and `first` and `last` modifiers, used for the `align-content`, `align-items`, `align-self`, `justify-items` and `justify-self` properties as well as the `place-content`, `place-items`, and `place-self` shorthand properties.

The `first` and `last` values give a box a baseline alignment preference, defaulting to `first` if the modifier is omitted.

Syntax

```
<baseline-position> = [ first | last ]? && baseline
```

Values

The `<baseline-position>` enumerated value type is specified using an optional `first` or `last` modifier with the `baseline` value. If a box does not belong to a shared alignment context, then the fallback alignment is used. The fallback alignment is also used to align the baseline-sharing group within its alignment container.

`baseline`

> Computes to `first baseline`, as defined below.

`first baseline`

> Aligns the alignment baseline of the box's first baseline set with the corresponding baseline of its baseline-sharing group. The fallback alignment is `safe self-start` for self-alignment or `safe start` for content distribution.

`last baseline`

Aligns the alignment baseline of the box's last baseline set with the corresponding baseline of its baseline-sharing group. The fallback alignment is `safe self-end` for self-alignment or `safe end` for content distribution.

See also

- Properties that use this data type: `align-content`, `align-items`, `align-self`, `justify-items`, `justify-self`, `place-content`, `place-items`, and `place-self`
- Other box alignment data types: `<content-distribution>`, `<content-position>`, `<overflow-position>`, and `<self-position>`
- CSS box alignment module
- CSS flexible box layout module
- CSS grid layout module

<basic-shape> CSS type

The **`<basic-shape>`** CSS data type represents a shape used in the `border-shape`, `clip-path`, `offset-path`, and `shape-outside` properties.

Try it

```
clip-path: inset(22% 12% 15px 35px);
```

```
clip-path: circle(6rem at 12rem 8rem);
```

```
clip-path: ellipse(115px 55px at 50% 40%);
```

```
clip-path: polygon(
  50% 2.4%,
  34.5% 33.8%,
  0% 38.8%,
  25% 63.1%,
  19.1% 97.6%,
  50% 81.3%,
  80.9% 97.6%,
  75% 63.1%,
  100% 38.8%,
  65.5% 33.8%
);
```

```
clip-path: path("M 50,245 A 160,160 0,0,1 360,120 z");
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#default-example {
  background: #ffee99;
}

#example-element {
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
  width: 100%;
  height: 100%;
}
```

Syntax

The `<basic-shape>` data type is used to create basic shapes including rectangles by container inset, by coordinate distance, or by set dimensions, circles, ellipses, polygons, paths, and author created shapes. These basic shapes are defined using one `<basic_shape>` CSS functions, with each value requiring a parameter that follows the shape's function-specific syntax.

Common parameters

The parameters common across the syntax of some basic shape functions include:

`round <'border-radius'>`

Defines rounded corners for rectangles by container insets, rectangles by distance, and rectangles with dimensions using the same syntax as the CSS `border-radius` shorthand property.

`<shape-radius>`

Defines the radius for a circle or an ellipse. Valid values include `<length>`, `<percentage>`, `closest-side` (the default), `farthest-side`, `closest-corner`, and `farthest-corner`. Negative values are invalid.

The `closest-side` keyword value uses the length from the center of the shape to the closest side of the reference box to create the radius length. The `farthest-side` keyword value uses the length from the center of the shape to the farthest side of the reference box. Similarly, the `closest-corner` and `farthest-corner` use the length from the center of the shape to the closest and farthest corners, respectively.

`<position>`

Defines the center `<position>` of a circle or an ellipse. It defaults to `center` if omitted.

`<fill-rule>`

Sets the `fill-rule` that is used to determine how the interior of the shape defined by the basic shapes polygon, path, and shape is to be filled. Possible values are `nonzero` (the default) and `evenodd`.

> **Note:** `<fill-rule>` is not supported in `offset-path` and using it invalidates the property.

Syntax for rectangles: <code><basic-shape-rect></code>

The `<basic-shape-rect>` type, a subset of the `<basic-shape>` type, represents the basic-shape functions limited to creating rectangles, including `inset()`, `rect()`, and `xywh()`.

The `polygon()`, `path()`, and `shape()` functions can also be used to create rectangles, but are not limited to only four-sided, right-angled shapes.

Syntax for rectangles by container insets

The `inset()` function creates an inset rectangle, with its size defined by the offset distance of each of the four sides of its container and, optionally, rounded corners.

```
inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )
```

When all of the first four arguments are supplied, they represent the top, right, bottom, and left offsets from the reference box inward that define the position of the edges of the inset rectangle. These arguments follow the syntax of the `margin` shorthand, which lets you set all four insets with one, two, three, or four values.

If a pair of insets for a dimension adds up to more than 100% of that dimension, both values are proportionally reduced so their sum equals 100%. For example, the value `inset(90% 10% 60% 10%)` has a top inset of `90%` and a bottom inset of `60%`. These values are reduced proportionally to `inset(60% 10% 40% 10%)`. Shapes such as this, that enclose no area and have no `shape-margin`, do not affect wrapping.

Syntax for rectangles by distance

The `rect()` function defines a rectangle using the specified distances from the top and left edges of the reference box, with optional rounded corners.

```
rect( [ <length-percentage> | auto ]{4} [ round <'border-radius'> ]? )
```

When using the `rect()` function, you do not define the width and height of the rectangle. Instead, you specify four values to create the rectangle, with its dimensions determined by the size of the reference box and the four offset values. Each value can be either a `<length>`, a `<percentage>`, or the keyword `auto`. The `auto` keyword is interpreted as `0%` for the top and left values and as `100%` for the bottom and right values.

Syntax for rectangles with dimensions

The `xywh()` function defines a rectangle located at the specified distances from the left (`x`) and top (`y`) edges of the reference box and sized by the specified width (`w`) and height (`h`) of the rectangle, in that order, with optional rounded corners.

```
xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )
```

Syntax for circles

The `circle()` function defines a circle using a radius and a position.

```
circle( <shape-radius>? [ at <position> ]? )
```

The `<shape-radius>` argument represents the radius of the circle defined as either a `<length>` or a `<percentage>`. A percentage value here is resolved from the used width and height of the reference box as `sqrt(width^2+height^2)/sqrt(2)`. If omitted, the radius is defined by `closest-side`.

Syntax for ellipses

The `ellipse()` function defines an ellipse using two radii and a position.

```
ellipse( [ <shape-radius>{2} ]? [ at <position> ]? )
```

The `<shape-radius>` arguments represent *rx* and *ry*, the x-axis and y-axis radii of the ellipse, in that order. These values are specified as either a `<length>` or a `<percentage>`. Percentage values here are resolved against the used width (for the rx value) and the used height (for the ry value) of the reference box. If only one radius value is provided, the `ellipse()` shape function is invalid. If no value is provided, `50% 50%` is used.

Syntax for polygons

The `polygon()` function defines a polygon using a set of comma-separated coordinate pairs, optionally preceded by an SVG `fill-rule`, a corner-rounding parameter, or both.

```
polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )
```

The optional first parameter can contain one or both of the following, separated by a space:

- An SVG `fill-rule` value, either `nonzero` or `evenodd`, specifying the algorithm used to fill the polygon shape.
- The keyword `round` followed by a `<length>` value, separated by a space. This specifies that the polygon should have rounded corners, with the `<length>` specifying the corner radius.

Each coordinate pair consists of two space-separated `<length-percentage>` values. These values represent the x- and y-axis coordinates of a polygon vertex.

Syntax for paths

The `path()` function defines a shape using an SVG `fill-rule` and an SVG path definition.

```
path( <'fill-rule'>? , <string> )
```

The required `<string>` is an SVG path as a quoted string. The `path()` function is not a valid `shape-outside` property value.

Syntax for shapes

The `shape()` function defines a shape using an initial starting point and a series of shape commands.

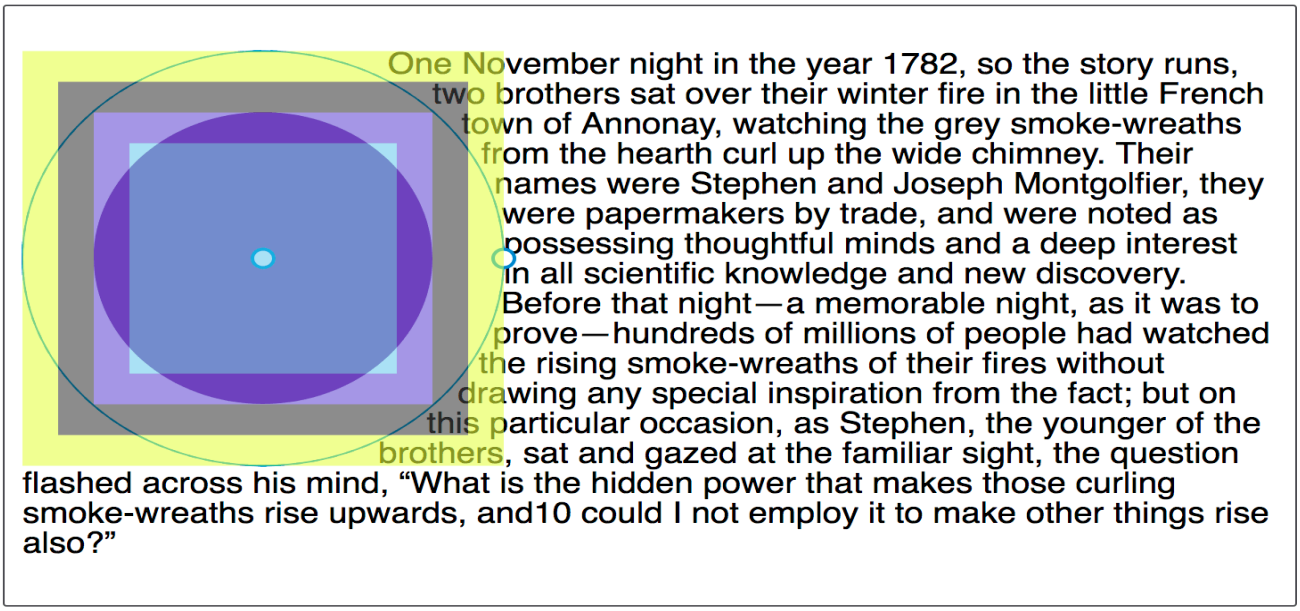
```
shape( <'fill-rule'>? from <coordinate-pair> , <shape-command># )
```

The `from <coordinate-pair>` parameter represents the starting point for the first shape command, and `<shape-command>` defines one or more shape commands, which are similar to the SVG path commands. The `shape()` function is not a valid `shape-outside` property value.

Description

When creating a shape, the reference box is defined by the property that uses `<basic-shape>` values. The coordinate system for the shape has its origin at the top-left corner of the element's margin box by default, with the x-axis running to the right and the y-axis running downwards. All the lengths expressed in percentages are resolved from the dimensions of the reference box.

The default reference box is the `margin-box`, as demonstrated in the image below. The image shows a circle created using `shape-outside: circle(50%)`, highlighting the different parts of the box model as seen in a browser's Developer Tools. The shape here is defined with reference to the margin-box.



Computed values of basic shapes

The values in a `<basic-shape>` function are computed as specified, with the following additional considerations:

- For any omitted values, their defaults are used.
- A `<position>` value in `circle()` or `ellipse()` is computed as a pair of offsets from the top left corner of the reference box: the first offset is horizontal, and the second is vertical. Each offset is specified as a `<length-percentage>` value.
- A `<border-radius>` value in `inset()` is expanded into a list of eight values, each either a `<length>` or a `<percentage>`.
- `inset()`, `rect()`, and `xywh()` functions compute to the equivalent `inset()` function.

Interpolation of basic shapes

When animating between two `<basic-shape>` functions, the interpolation rules listed below are followed. The parameter values of each `<basic-shape>` function form a list. For interpolation to occur between two shapes, both shapes must use the same reference box and the number and type of values in both `<basic-shape>` lists must match.

Each value in the lists of the two `<basic-shape>` functions is interpolated based on its computed value as a `<number>`, `<length>`, `<percentage>`, `<angle>`, or `calc()` where possible. Interpolation can still occur if the values are not one of those data types but are identical between the two interpolating basic shape functions, such as `nonzero`.

- **Both shapes are of type `ellipse()` or type `circle()`**: Interpolation is applied between each corresponding value if their radii are specified as either a `<length>` or a `<percentage>` (rather than keywords such as `closest-side` or `farthest-side`).
- **Both shapes are of type `inset()`**: Interpolation is applied between each corresponding value.
- **Both shapes are of type `polygon()`**: Interpolation is applied between each corresponding value if they use the same `<fill-rule>` and have the same number of comma-separated coordinate pairs.
- **Both shapes are of type `path()`**: Interpolation is applied to each parameter as a `<number>` if the path strings in both the shapes match the number, type, and sequence of path data commands.
- **Both shapes are of type `shape()`**: Interpolation is applied between each corresponding value if they have the identical command keyword and use the same `<by-to>` keyword. If `shape()` is used in the `clip-path` property, the two shapes interpolate if they also have the same `<fill-rule>`.
    - If they use the `<curve-command>` or the `<smooth-command>`, the number of control points must match for interpolation.
    - If they use the `<arc-command>` with different `<arc-sweep>` directions, the interpolated result goes clockwise (`cw`). If they use different `<arc-size>` keywords, the size is interpolated using the `large` value.
- **One shape is of type `path()` and the other is of type `shape()`**: Interpolation is applied between each corresponding value if the list of path data commands is identical in number as well as sequence. The interpolated shape is a `shape()` function, maintaining the same list of path data commands.

In all other cases, no interpolation occurs and the animation is discrete.

Examples

Animated polygon

In this example, we use the @keyframes at-rule to animate a clip path between two polygons. Note that both polygons have the same number of vertices, which is necessary for this type of animation to work.

HTML

```
<div></div>
```

CSS

```
div {
  width: 300px;
  height: 300px;
  background: repeating-linear-gradient(red, orange 50px);
  clip-path: polygon(
    50% 0%,
    60% 40%,
    100% 50%,
    60% 60%,
    50% 100%,
    40% 60%,
    0% 50%,
    40% 40%
  );
  animation: 4s poly infinite alternate ease-in-out;
```

```
  margin: 10px auto;
}

@keyframes poly {
  from {
    clip-path: polygon(
      50% 0%,
      60% 40%,
      100% 50%,
      60% 60%,
      50% 100%,
      40% 60%,
      0% 50%,
      40% 40%
    );
  }

  to {
    clip-path: polygon(
      50% 30%,
      100% 0%,
      70% 50%,
      100% 100%,
      50% 70%,
      0% 100%,
      30% 50%,
      0% 0%
    );
  }
}
```

Result

See also

- Properties that use this data type: `border-shape`, `clip-path`, `offset-path`, `shape-outside`
- SVG shape elements: `<circle>`, `<ellipse>`, `<line>`, `<polygon>`, `<polyline>`, `<rect>`
- Overview of CSS shapes
- CSS shapes module
- Introduction to CSS masking
- CSS masking module
- Edit shape paths in Firefox developer tools

circle() CSS function

The **`circle()`** CSS function defines a circle using a radius and a position. It is one of the `<basic-shape>` data types.

Try it

```
clip-path: circle(50px);
```

```
clip-path: circle(6rem at right center);
```

```
clip-path: circle(10% at 2rem 90%);
```

```
clip-path: circle(closest-side at 5rem 6rem);
```

```
clip-path: circle(farthest-side);
```

```
clip-path: circle(closest-corner at 70% 70%);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#default-example {
  background: #ffee99;
}

#example-element {
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
  width: 100%;
  height: 100%;
}
```

Syntax

```
shape-outside: circle(50%);
clip-path: circle(6rem at 12rem 8rem);
border-shape: circle(60%);
```

Values

`<shape-radius>`

This may be a `<length>`, a `<percentage>`, or any of the following keyword values:

`closest-side`

The radius is the length from the center of the circle to the closest side of the reference box, so that the circle's boundary just touches that side.

`farthest-side`

The radius is the length from the center of the circle to the farthest side of the reference box, so that the circle's boundary just touches that side.

`closest-corner`

The radius is the length from the center of the circle to the closest corner of the reference box, so that the circle's boundary passes through that corner.

`farthest-corner`

The radius is the length from the center of the circle to the farthest corner of the reference box, so that the circle's boundary passes through that corner.

`<position>`

Moves the center of the circle. May be a `<length>`, or a `<percentage>`, or a value such as `left`. The `<position>` value defaults to center if omitted.

Formal syntax

```
<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}
```

This syntax reflects the latest standard as per CSS Images Module Level 3, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic circle

In the example below, the `shape-outside` property has a value of `circle(50%)`, which defines a circle on a floated element for the text to flow round.

```
<div class="box">
  <img
    alt="A hot air balloon"
    src="https://mdn.github.io/shared-assets/images/examples/round-balloon.png" />
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
img {
  float: left;
  shape-outside: circle(50%);
}
```

Radius keyword interactive example

This example allows you to test the effect of moving the center point of the circle when using each of the four `<shape-radius>` keywords.

HTML

The HTML first defines controls for selecting a chosen radius keyword and setting the position where the circle is to be displayed, followed by a `<pre>` element for displaying the `clip-path` that the control values select. After this a `#support-note` element is provided for indicating when particular radius keywords are not supported.

```
<div class="controls">
  <div class="controls-row">
    <label>
      Shape radius:
      <select id="radius-keyword">
        <option value="closest-side">closest-side</option>
        <option value="closest-corner">closest-corner</option>
        <option value="farthest-side">farthest-side</option>
        <option value="farthest-corner" selected>farthest-corner</option>
      </select>
    </label>
  </div>
  <div class="controls-row">
    <label>
      Position X:
      <input type="range" id="pos-x" min="0" max="200" value="120" />
    </label>
    <label>
      Position Y:
      <input type="range" id="pos-y" min="0" max="120" value="40" />
    </label>
  </div>
</div>

<pre id="declaration"></pre>

<p id="support-note"></p>
```

The HTML then defines a number of `<div>` elements that are used to render the circle, reference box and center marker. The most important elements are the reference box (`#refbox`) and the `.fill` element it contains, which define the gradient that is drawn (as discussed in the following CSS section). A `clipPath` is set on the reference box in JavaScript to clip that gradient to a circular shape.

```
<div class="canvas">
  <div class="refbox" id="refbox">
    <div class="fill"></div>
  </div>
  <div class="outline"></div>
  <div class="center-marker" id="center-marker"></div>
</div>
```

Note that the `.outline` and `.center-marker` are provided to make it easier to see the reference box and the center of the clipping circle, which would otherwise be invisible. The `.outline` element must be placed after `#refbox` in the markup: since it isn't clipped itself, painting it after the reference box means its dashed border always renders on top of the `.fill` gradient, even when the clipping circle extends beyond the reference box.

CSS

The CSS for the canvas, reference box, and fill elements is shown below. Note that the `.fill` defines a gradient that overflows the reference box, filling most of the canvas. This is the gradient that we will clip using the `circle()` (we do this by setting the CSS clip path dynamically in our JavaScript code).

```
.canvas {
  position: relative;
  width: 640px;
  height: 480px;
  overflow: hidden;
  border: 1px solid #888888;
}

.refbox {
  position: absolute;
  top: 180px;
  left: 220px;
  width: 200px;
  height: 120px;
  box-sizing: border-box;
}

.fill {
  position: absolute;
  inset: -240px;
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
}
```

The CSS for the controls and other elements is not shown, as it isn't needed to understand the `circle()` method. Interested readers can view it in the example playground.

```
body {
  display: flex;
  flex-direction: column;
  gap: 20px;
  align-items: flex-start;
}

.controls {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.controls-row {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  align-items: center;
}
```

```
.outline {
  position: absolute;
  top: 180px;
  left: 220px;
  width: 200px;
  height: 120px;
```

```
  box-sizing: border-box;
}

.outline {
  border: 3px dashed #e6007a;
  pointer-events: none;
}

.center-marker {
  position: absolute;
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: white;
  border: 2px solid black;
  transform: translate(-50%, -50%);
  pointer-events: none;
}

#declaration {
  margin: 0;
}

#support-note {
  margin: 0;
  color: #b3001b;
}
```

JavaScript

First we get handles to each of the elements used by the example.

```
const select = document.getElementById("radius-keyword");
const posX = document.getElementById("pos-x");
const posY = document.getElementById("pos-y");
const refbox = document.getElementById("refbox");
const declaration = document.getElementById("declaration");
const marker = document.getElementById("center-marker");
const supportNote = document.getElementById("support-note");
```

Then we define the `checkSupport()` function used to feature-test whether each radius keyword is supported and enable/disable the associated selection options. This uses the `CSS.supports()` method to check for keyword support.

```
function checkSupport() {
  const unsupported = [];
  for (const option of select.options) {
    if (!CSS.supports("clip-path", `circle(${option.value} at 0px 0px)`)) {
      option.disabled = true;
      option.textContent += " (not supported)";
      unsupported.push(option.value);
    }
  }
  if (unsupported.length > 0) {
    supportNote.textContent = `Your browser doesn't support: ${unsupported.join(", ")}.`;
  }
  if (select.selectedOptions[0]?.disabled) {
    const firstSupported = [...select.options].find(
      (option) => !option.disabled,
    );
    if (firstSupported) {
      select.value = firstSupported.value;
    }
  }
}
```

Next we define an `update()` function that is called whenever the input controls are changed. This sets the clip path on the reference box based on the selected keyword and position, and moves the center marker to match.

```
function update() {
  const keyword = select.value;
  const x = Number(posX.value);
  const y = Number(posY.value);

  // Build the clip-path value and apply it to the reference box
  const value = `circle(${keyword} at ${x}px ${y}px)`;
  refbox.style.clipPath = value;

  // Update the displayed declaration text and marker position
  declaration.textContent = `clip-path: ${value};`;
  marker.style.left = `${refbox.offsetLeft + x}px`;
  marker.style.top = `${refbox.offsetTop + y}px`;
}

select.addEventListener("change", update);
posX.addEventListener("input", update);
posY.addEventListener("input", update);
checkSupport();
update();
```

Result

Change the position of the circle center and the keyword used to see their relative effects. Note that the reference box, together with the selected radius keyword and position, is used to calculate the clipping circle. This circle can extend beyond the reference box (for example, with `closest-corner` or `farthest-corner`); the `.fill` gradient is deliberately sized larger than the reference box so that it always fully covers the clipped circle, however far it extends.

See also

- Properties that use this data type: `border-shape`, `clip-path`, `shape-outside`
- Guide to Basic Shapes

ellipse() CSS function

The **`ellipse()`** CSS function is one of the `<basic-shape>` data types.

Try it

```
clip-path: ellipse(20px 50px);
```

```
clip-path: ellipse(4rem 50% at right center);
```

```
clip-path: ellipse(closest-side closest-side at 5rem 6rem);
```

```
clip-path: ellipse(closest-side farthest-side);
```

```
clip-path: ellipse(closest-corner closest-corner at 25% 25%);
```

```
clip-path: ellipse(closest-side closest-corner at 25% 25%);
```

```
clip-path: ellipse(closest-side farthest-corner at 25% 25%);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#default-example {
  background: #ffee99;
}

#example-element {
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
  width: 100%;
  height: 100%;
}
```

Syntax

```
shape-outside: ellipse(40% 50% at left);
shape-outside: ellipse(closest-side farthest-side at 30%);
border-shape: ellipse(50% 40%);
```

An ellipse is essentially a squashed circle and so `ellipse()` acts in a very similar way to `circle()` except that we have to specify two radii x and y.

Values

`<shape-radius>`

Two radii, x and y, in that order. These may be a `<length>`, a `<percentage>`, or any of the following keyword values:

`closest-side`

The radius is the length from the center of the ellipse to the closest side of the reference box in the given radius direction, so that the ellipse's boundary just touches that side.

`farthest-side`

The radius is the length from the center of the ellipse to the farthest side of the reference box in the given radius direction, so that the ellipse's boundary just touches that side.

`closest-corner`

The radius is the length from the center of the ellipse to the closest corner of the reference box in the given radius direction.

`farthest-corner`

The radius is the length from the center of the ellipse to the farthest corner of the reference box in the given radius direction.

`<position>`

Moves the center of the ellipse. May be a `<length>`, or a `<percentage>`, or a value such as `left`. The `<position>` value defaults to center if omitted.

Formal syntax

```
<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
```

```
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}
```

This syntax reflects the latest standard as per CSS Images Module Level 3, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic ellipse() example

This example shows an ellipse that is floated left that has a horizontal radius of 40%, a vertical radius of 50%, and a left position. This means that the center of the ellipse is on the left edge of the box giving us a half ellipse shape to wrap our text around. Click "Play" in the code blocks to change these values to see how the ellipse changes:

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.shape {
  float: left;
  shape-outside: ellipse(40% 50% at left);
  margin: 20px;
  width: 100px;
  height: 200px;
}
```

Using closest-side / farthest-side values

The keyword values of `closest-side` and `farthest-side` are useful to create a quick ellipse based on the size of the floated element reference box.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
body {
  font: 1.2em / 1.5 sans-serif;
}
.shape {
  float: left;
  shape-outside: ellipse(closest-side farthest-side at 30%);
  margin: 20px;
  width: 100px;
  height: 140px;
}
```

Radius keyword interactive example

This example allows you to test the effect of moving the center point of the ellipse when using each of the four `<shape-radius>` keywords — chosen independently for the x-radius and the y-radius.

Because `closest-corner` and `farthest-corner` are a newer addition to `ellipse()`, the script feature-tests each keyword with `CSS.supports()` before enabling it. Any keyword your browser doesn't yet support is disabled in both dropdowns (and labeled "not supported").

HTML

The HTML first defines controls for selecting the x-radius and y-radius keywords and setting the position where the ellipse is to be displayed, followed by a `<pre>` element for displaying the `clip-path` that the control values select. After this a `#support-note` element is provided for indicating when particular radius keywords are not supported.

```
<div class="controls">
  <div class="controls-row">
    <label>
      X radius:
```

```
      <select id="radius-x-keyword">
        <option value="closest-side">closest-side</option>
        <option value="closest-corner">closest-corner</option>
        <option value="farthest-side">farthest-side</option>
        <option value="farthest-corner" selected>farthest-corner</option>
      </select>
    </label>
    <label>
      Y radius:
      <select id="radius-y-keyword">
        <option value="closest-side">closest-side</option>
        <option value="closest-corner">closest-corner</option>
        <option value="farthest-side" selected>farthest-side</option>
        <option value="farthest-corner">farthest-corner</option>
      </select>
    </label>
  </div>
  <div class="controls-row">
    <label>
      Position X:
      <input type="range" id="pos-x" min="0" max="200" value="120" />
    </label>
    <label>
      Position Y:
      <input type="range" id="pos-y" min="0" max="120" value="40" />
    </label>
  </div>
</div>

<pre id="declaration"></pre>

<p id="support-note"></p>
```

The HTML then defines several `<div>` elements used to render the ellipse, reference box, and center marker. The most important elements are the reference box (`#refbox`) and the `.fill` element it contains, which define the gradient that is drawn (as discussed in the following CSS section). A `clipPath` is set on the reference box in JavaScript to clip that gradient to an elliptical shape.

```
<div class="canvas">
  <div class="refbox" id="refbox">
    <div class="fill"></div>
  </div>
  <div class="outline"></div>
  <div class="center-marker" id="center-marker"></div>
</div>
```

Note that the `.outline` and `.center-marker` are provided to make it easier to see the reference box and the center of the clipping ellipse, which would otherwise be invisible. The `.outline` element must be placed after `#refbox` in the markup: since it isn't clipped itself, painting it after the reference box means its dashed border always renders on top of the `.fill` gradient, even when the clipping ellipse extends beyond the reference box.

CSS

The CSS for the canvas, reference box, and fill elements is shown below. Note that the `.fill` defines a gradient that overflows the reference box, filling most of the canvas. This is the gradient that we will clip using the `ellipse()` function (we do this by setting the CSS clip path dynamically in our JavaScript code).

```
.canvas {
  position: relative;
  width: 640px;
  height: 480px;
  overflow: hidden;
  border: 1px solid #888888;
}

.refbox {
  position: absolute;
  top: 180px;
  left: 220px;
  width: 200px;
  height: 120px;
  box-sizing: border-box;
}

.fill {
  position: absolute;
  inset: -240px;
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
}
```

The CSS for the controls and other elements is not shown, as it isn't needed to understand the `ellipse()` method. Interested readers can view it in the example playground.

```
body {
  display: flex;
  flex-direction: column;
  gap: 20px;
  align-items: flex-start;
}

.controls {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.controls-row {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  align-items: center;
}
```

```
.outline {
  position: absolute;
```

```
  top: 180px;
  left: 220px;
  width: 200px;
  height: 120px;
  box-sizing: border-box;
}

.outline {
  border: 3px dashed #e6007a;
  pointer-events: none;
}

.center-marker {
  position: absolute;
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: white;
  border: 2px solid black;
  transform: translate(-50%, -50%);
  pointer-events: none;
}

#declaration {
  margin: 0;
}

#support-note {
  margin: 0;
  color: #b3001b;
}
```

JavaScript

First we get handles to each of the elements used by the example.

```
const selectX = document.getElementById("radius-x-keyword");
const selectY = document.getElementById("radius-y-keyword");
const posX = document.getElementById("pos-x");
const posY = document.getElementById("pos-y");
const refbox = document.getElementById("refbox");
const declaration = document.getElementById("declaration");
const marker = document.getElementById("center-marker");
const supportNote = document.getElementById("support-note");
```

Then we define a `checkSupport()` function to feature-test whether each radius keyword is supported and enable/disable the associated options in both dropdowns. This uses the `CSS.supports()` method to check for keyword support.

```
function checkSupport() {
  const unsupported = new Set();
  for (const option of selectX.options) {
    const test = `ellipse(${option.value} ${option.value} at 0px 0px)`;
    if (!CSS.supports("clip-path", test)) {
      unsupported.add(option.value);
    }
  }
  for (const select of [selectX, selectY]) {
    for (const option of select.options) {
      if (unsupported.has(option.value)) {
        option.disabled = true;
        option.textContent += " (not supported)";
      }
    }
    if (select.selectedOptions[0]?.disabled) {
      const firstSupported = [...select.options].find(
        (option) => !option.disabled,
      );
      if (firstSupported) {
        select.value = firstSupported.value;
      }
    }
  }
  if (unsupported.size > 0) {
    supportNote.textContent = `Your browser doesn't support: ${[...unsupported].join(", ")}.`;
  }
}
```

Next we define an `update()` function that is called whenever the input controls are changed. This sets the clip path on the reference box based on the selected x-radius and y-radius keywords and position, and moves the center marker to match.

```
function update() {
  const xKeyword = selectX.value;
  const yKeyword = selectY.value;
  const x = Number(posX.value);
  const y = Number(posY.value);

  // Build the clip-path value and apply it to the reference box
  const value = `ellipse(${xKeyword} ${yKeyword} at ${x}px ${y}px)`;
  refbox.style.clipPath = value;

  // Update the displayed declaration text and marker position
  declaration.textContent = `clip-path: ${value};`;
  marker.style.left = `${refbox.offsetLeft + x}px`;
  marker.style.top = `${refbox.offsetTop + y}px`;
}

selectX.addEventListener("change", update);
selectY.addEventListener("change", update);
posX.addEventListener("input", update);
posY.addEventListener("input", update);
```

```
checkSupport();
update();
```

Result

Change the position of the center and the keyword for each radius to see their relative effects. Note that the reference box, together with the selected x-radius and y-radius keywords and position, is used to calculate the clipping ellipse. This ellipse can extend beyond the reference box (for example, with `closest-corner` or `farthest-corner`); the `.fill` gradient is deliberately sized larger than the reference box so that it always fully covers the clipped ellipse, however far it extends.

See also

- Properties that use this data type: `border-shape`, `clip-path`, `shape-outside`
- Guide to Basic Shapes

inset() CSS function

The **`inset()`** CSS function defines a rectangle at the specified inset distances from each side of the reference box. It is a basic shape function used to define one of the `<basic-shape>` data types.

Try it

```
clip-path: inset(30px);
```

```
clip-path: inset(1rem 2rem 3rem 4rem);
```

```
clip-path: inset(20% 30% round 20px);
```

```
clip-path: inset(4rem 20% round 1rem 2rem 3rem 4rem);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#default-example {
  background: #ffee99;
}

#example-element {
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
  width: 100%;
  height: 100%;
}
```

Syntax

```
shape-outside: inset(20px 50px 10px 0 round 50px);
border-shape: inset(10px 20px 10px 20px round 20px);
```

Values

`<length-percentage>{1,4}`

> When all of the four arguments are supplied they represent the top, right, bottom, and left offsets from the reference box inward that define the positions of the edges of the inset rectangle. These arguments follow the syntax of the margin shorthand, which let you set all four insets with one, two, or four values.
>
> If a pair of insets for a dimension adds up to more than 100% of that dimension, both values are proportionally reduced so their sum equals 100%. For example, the value `inset(90% 10% 60% 10%)` has a top inset of `90%` and a bottom inset of `60%`. These values are reduced proportionally to `inset(60% 10% 40% 10%)`. Shapes such as this, that enclose no area and have no `shape-margin`, do not affect wrapping.

`<border-radius>`

> The optional `<border-radius>` argument(s) define rounded corners for the inset rectangle using the border-radius shorthand syntax.

Formal syntax

```
<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<length-percentage> =
  <length>      |
  <percentage>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Shapes Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic inset example

In the example below we have an `inset()` shape used to pull content over the floated element. Change the offset values to see how the shape changes.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
.box {
  width: 400px;
  margin: 0 auto;
}

.shape {
  float: left;
  width: 150px;
  height: 100px;
  clip-path: inset(45px 50px 15px 0 round 50px);
  shape-outside: inset(40px 40px 10px 0 round 50px);
  background-color: coral;
  border-radius: 20px;
  margin: 0;
  padding: 20px;
}
```

See also

- Properties that use this data type: `border-shape`, `clip-path`, `shape-outside`
- CSS shapes module
- Guide to basic shapes

path() CSS function

The **`path()`** CSS function accepts an SVG path string, and is used in the CSS shapes and CSS motion path modules to enable a shape to be drawn. The `path()` function is a `<basic-shape>` data type value. It can be used in the CSS `offset-path` and `clip-path` properties and in the SVG `d` attribute.

There are some limitations to using the `path()` function. The path has to be defined as a single string, so a custom path can't be created using variables (`var()` functions). Also, all the lengths in the path are implicitly defined in pixel (`px`) units; other units can't be used. The `shape()` function offers more flexibility than the `path()` function.

Try it

```
clip-path: path(
  "M  20  240 \
 L  20  80 L 160  80 \
 L 160  20 L 280 100 \
 L 160 180 L 160 120 \
 L  60 120 L  60 240 Z"
);
```

```
clip-path: path(
  "M 20 240 \
 C 20 0 300 0 300 240 Z"
);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#default-example {
  background: #ffee99;
}

#example-element {
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
  width: 100%;
  height: 100%;
}
```

Syntax

```
path("M 10 80 C 40 10, 65 10, 95 80 S 150 150, 180 80")

/* When used in clip-path only */
path(evenodd,"M 10 80 C 40 10, 65 10, 95 80 S 150 150, 180 80")
```

Parameters

`<fill-rule>` Optional

Defines what parts of the path are inside the shape. The possible values include:

- `nonzero`: A point is considered inside the shape if a ray drawn from the point crosses more left-to-right than right-to-left path segments, resulting in a non-zero count. This is the default value when `<fill-rule>` is omitted.
- `evenodd`: A point is considered to be inside the shape if a ray drawn from the point crosses an odd number of path segments. This means that for each time the ray enters the shape, it has not exited an equal number of times, indicating an odd count of entries without corresponding exits.

> **Warning:** `<fill-rule>` is not supported in `offset-path` and using it invalidates the property.

`<string>`

A data string, contained in quotes, which defines an SVG path. The SVG path data string contains path commands that implicitly use pixel units. An empty path is considered invalid.

Return value

Returns a `<basic-shape>` value.

Formal syntax

```
<path()> =
  path( <'fill-rule'>? , <string> )

<fill-rule> =
  nonzero  |
  evenodd
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Shapes Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using a <code>path()</code> function as an <code>offset-path</code> value

A `path()` function has been provided as an `offset-path` value in the following example to create an elliptical path for a ball to move along.

```
<div id="path">
  <div id="ball"></div>
</div>
<button>animate</button>
```

```
#path {
  margin: 40px;
  width: 400px;
  height: 200px;

  /* draw the gray ramp */
  background: radial-gradient(at 50% 0%, transparent 70%, grey 70%, grey 100%);
}

#ball {
  width: 30px;
  height: 30px;
  background-color: red;
  border-radius: 50%;

  /* mark the elliptical path */
  offset-path: path("M 15 15 A 6 5.5 10 0 0 385 15");
}
```

```
const btn = document.querySelector("button");
const ball = document.getElementById("ball");

btn.addEventListener("click", () => {
  btn.setAttribute("disabled", true);
  setTimeout(() => btn.removeAttribute("disabled"), 6000);

  ball.animate(
    // animate the offset path
    { offsetDistance: [0, "100%"] },
    {
      duration: 1500,
      iterations: 4,
      easing: "cubic-bezier(.667,0.01,.333,.99)",
      direction: "alternate",
    },
  );
});
```

Modify the value of the SVG path d attribute

The `path()` can be used to modify the value of the SVG `d` attribute, which can also be set to `none` in your CSS.

The "V" symbol will flip vertically when you hover over it, if `d` is supported as a CSS property.

CSS

```
html,
body,
svg {
  height: 100%;
}

/* This path is displayed on hover */
#svg_css_ex1:hover path {
  d: path("M20,80 L50,20 L80,80");
}
```

HTML

```
<svg id="svg_css_ex1" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <path fill="none" stroke="red" d="M20,20 L50,80 L80,20" />
</svg>
```

Result

See also

- `border-shape`, `shape-outside`
- CSS shapes module
- Overview of CSS shapes
- The SVG `path` syntax: an illustrated guide via CSS-tricks (2021)

polygon() CSS function

The **`polygon()`** CSS function is one of the `<basic-shape>` data types. It's used to draw a polygon by providing one or more pairs of coordinates, each of which represents a vertex of the shape.

Try it

```
clip-path: polygon(
  0% 20%,
  60% 20%,
  60% 0%,
  100% 50%,
  60% 100%,
  60% 80%,
```

```
  0% 80%
);
```

```
clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#default-example {
  background: #ffee99;
}

#example-element {
  background: linear-gradient(to bottom right, #ff5522, #0055ff);
  width: 100%;
  height: 100%;
}
```

Syntax

```
/* Coordinate list */
polygon(50% 2.4%, 34.5% 33.8%, 0% 38.8%, 25% 63.1%, 19.1% 97.6%)
polygon(0px 0px, 200px 100px, 0px 200px)
polygon(0% 0px, 100% 100px, 0px 100%)
polygon(0 0, 50% 1rem, 100% 2vw, calc(100% - 20px) 100%, 0 100%)

/* Coordinate list with fill-rule and/or round value */
polygon(nonzero, 0% 0%, 50% 50%, 0% 100%)
polygon(round 20px, 0% 0%, 50% 50%, 0% 100%)
polygon(evenodd round 2em, 0% 0%, 50% 50%, 0% 100%)
```

Parameters

The `polygon()` function accepts an optional first parameter containing values that modify the polygon's appearance — a `fill-rule` keyword, the `round` keyword followed by a `<length>` value, or both. The first parameter's components are separated by spaces. The other parameters are space-separated x/y coordinate pairs of `<length-percentage>` values.

`<fill-rule>` Optional

A keyword equal to `nonzero` (the default) or `evenodd`, which specifies the algorithm used to fill the polygon shape.

`round <length>` Optional

The `round` keyword specifies that the polygon should have rounded corners, and the accompanying `<length>` value specifies the radius of those corners.

`<length-percentage>`

Each vertex, or point, of the polygon is represented by a space-separated pair of `<length-percentage>` values defining the x/y coordinates of the vertex relative to the shape's reference box.

Return value

A `<basic-shape>` value.

Description

You can create almost any shape with the `polygon()` function by specifying the x/y coordinates of its vertices, or points, as comma-separated pairs of `<length-percentage>` values:

`polygon(x1 y1, x2 y2, x3 y3, x4 y4, xn yn)`

Although only a single point is required to create a valid `polygon()` function value, at least 3 points are required to create a shape (a triangle). There is no upper limit on the number of points that can be specified. The shape is drawn through the specified points in the order they appear in the function, with a final line automatically being drawn between the last and first points to close the shape.

We could define the coordinates of a triangle shape like this:

axis	point 1	point 2	point 3
x	0%	100%	100%
y	0%	0%	100%

We can apply those coordinates to the CSS `clip-path` property in a `polygon()` function as follows:

```
clip-path: polygon(0% 0%, 100% 0%, 100% 100%);
```

This creates a triangle shape that covers half the area of its parent container by specifying the coordinates of three of its four corners: top-left (`0% 0%`), top-right (`100% 0%`), and bottom-right (`100% 100%`). If we assume a 200x200px container with a green background:

```
<div class="box"></div>
```

```
.box {
  width: 200px;
  height: 200px;
  background-color: green;
  clip-path: polygon(0% 0%, 100% 0%, 100% 100%);
}
```

The effect of point order

The order in which you define the points can result in different shapes. For example, the following two `clip-path` declarations both use a `polygon()` function with X/Y coordinate pairs for the container's four corners, but in a different order.

```
clip-path: polygon(0% 0%, 100% 0%, 100% 100%, 0% 100%);
clip-path: polygon(0% 0%, 100% 0%, 0% 100%, 100% 100%);
```

```
<div class="box"></div>
<div class="box box2"></div>
```

```
body {
  display: flex;
  gap: 20px;
}

.box {
  width: 200px;
  height: 200px;
  background-color: purple;
  clip-path: polygon(0% 0%, 100% 0%, 100% 100%, 0% 100%);
}

.box2 {
  clip-path: polygon(0% 0%, 100% 0%, 0% 100%, 100% 100%);
}
```

The first function draws a square, while the second draws an hourglass shape.

Specifying polygon modifiers

The `polygon()` function accepts an optional first parameter modifying the rendering of the resulting shape. The parameter value can include either or both of the following, separated by spaces:

- A `<fill-rule>` keyword equal to `nonzero` (the default) or `evenodd`, which specifies the algorithm used to fill the polygon shape. This only has an effect when the lines drawn between coordinate values overlap.
- The `round` keyword followed by a `<length>` value. This specifies that the polygon should have rounded corners, with the `<length>` value specifying the radius of those corners.

For example, we could expand upon the previous triangle example and add rounded corners:

```
clip-path: polygon(round 20px, 0% 0%, 100% 0%, 100% 100%);
```

```
<div class="box"></div>
```

```
.box {
  width: 200px;
  height: 200px;
  background-color: green;
  clip-path: polygon(round 20px, 0% 0%, 100% 0%, 100% 100%);
}
```

This results in the same triangle shape, but with `20px` radius rounded corners:

> **Note:** In each case, the corner radius is clamped to ensure it is never larger than half of any line segment length. The maximum corner radius is clamped to the smaller of `tan(corner-angle/2) * (segment-length / 2)` evaluated against both line segments forming the corner. The specification contains further details for those interested.

Formal syntax

```
<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<fill-rule> =
  nonzero  |
  evenodd

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Shapes Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Create a triangle

In this example, a triangle is formed by defining the coordinates of its three points.

HTML

```
<div class="triangle"></div>
```

CSS

```
.triangle {
  width: 400px;
  height: 400px;
  background-color: magenta;
  clip-path: polygon(100% 0%, 50% 50%, 100% 100%);
}
```

Result

The coordinates for the triangle are the top-right corner (`100% 0%`), the center point (`50% 50%`), and the bottom-right corner (`100% 100%`) of the container.

Create a rounded star

In this example, we create a star shape and use the `round` keyword to round its corners.

HTML

```
<div class="star"></div>
```

CSS

```
.star {
  width: 400px;
  height: 400px;
  background-color: hotpink;
  clip-path: polygon(
```

```
    round 20px,
    50% 5%,
    60.85% 27.48%,
    85.22% 21.99%,
    74.38% 44.44%,
    93.88% 60.01%,
    69.57% 65.56%,
    69.53% 90.55%,
    50% 75%,
    30.47% 90.55%,
    30.43% 65.56%,
    6.12% 60.01%,
    25.62% 44.44%,
    14.78% 21.99%,
    39.15% 27.48%
  );
}
```

```
@supports not (clip-path: polygon(round 20px, 0% 0%, 100% 0%, 100% 100%)) {
  body::before {
    font-family: sans-serif;
    content: "Your browser does not support the polygon() function's round keyword.";
    background-color: wheat;
    padding: 1rem 0;
    text-align: center;

    z-index: 1;
    position: fixed;
    inset: 40% 0 auto;
  }
}
```

Result

Setting a polygon for shape-outside

In this example a shape is created for text to follow using the `shape-outside` property.

```
<div class="box">
  <div class="shape"></div>
  <p>
    One November night in the year 1782, so the story runs, two brothers sat
    over their winter fire in the little French town of Annonay, watching the
    grey smoke-wreaths from the hearth curl up the wide chimney. Their names
    were Stephen and Joseph Montgolfier, they were papermakers by trade, and
    were noted as possessing thoughtful minds and a deep interest in all
    scientific knowledge and new discovery. Before that night—a memorable night,
    as it was to prove—hundreds of millions of people had watched the rising
    smoke-wreaths of their fires without drawing any special inspiration from
    the fact.
  </p>
</div>
```

```
.box {
  width: 250px;
}

.shape {
  float: left;
  shape-outside: polygon(
    0 5%,
    15% 12%,
    30% 15%,
    40% 26%,
    45% 35%,
    45% 45%,
    40% 55%,
    10% 90%,
    10% 98%,
    8% 100%,
    0 100%
  );
  width: 300px;
  height: 320px;
}

p {
  font-size: 0.9rem;
}
```

See also

- Properties that use this data type: `border-shape`, `clip-path`, `shape-outside`
- Guide to Basic Shapes

rect() CSS function

The **`rect()`** CSS function creates a rectangle at the specified distance from the top and left edges of the containing block. It is a basic shape function of the `<basic-shape>` data type. You can use the `rect()` function in CSS properties such as `offset-path` to create the rectangular path along which an element moves, in `clip-path` to define the shape of the clipping region, and in `border-shape` to define the shape of an element's border.

Syntax

```
offset-path: rect(0 1% auto 3% round 0 1px);
clip-path: rect(50px 70px 80% 20%);
border-shape: rect(10px 500px 130px 20px round 20px);
```

Values

The inset rectangle is defined by specifying four offset values, starting with the top edge offset and going clockwise, and an optional `round` keyword with the `border-radius` parameter to add rounded corners to the rectangle. Each offset value can be either a `<length>`, a `<percentage>`, or the keyword `auto`.

`<length-percentage>`

> Specifies the `<length-percentage>` value of the distance of the top, right, bottom, or left edge of the rectangle from the top or left edge of the containing block. The first (top) and third (bottom) values are distances from the top edge of the containing block, and the second (right) and fourth (left) values are distances from the left edge of the containing block. The second (right) and third (bottom) values are clamped by the fourth (left) and first (top) values, respectively, to prevent the bottom edge from crossing over the top edge and right edge from crossing over the left edge. For example, `rect(10px 0 0 20px)` is clamped to `rect(10px 20px 10px 20px)`.

`auto`

> Makes the edge for which this value is used to coincide with the corresponding edge of the containing block. If `auto` is used for the first (top) or fourth (left) value, the value of `auto` is `0`, and if used for the second (right) or third (bottom) value, the value of `auto` is `100%`.

`round <'border-radius'>`

> Specifies the radius of the rounded corners of the rectangle using the same syntax as the CSS `border-radius` shorthand property. This parameter is optional.

Formal syntax

```
<rect()> =
  rect( [ <length-percentage> | auto ]{4} [ round <'border-radius'> ]? )

<length-percentage> =
  <length>      |
  <percentage>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Shapes Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating offset-path using rect()

In this example, the `offset-path` property uses the `rect()` function to define the shape of the path on which the element, a red box in this case, moves. Three different scenarios are shown, each using different values for the `rect()` function. The arrow inside the boxes points to the right edge of the box.

```
<div class="container">
  Rectangular path 1
  <div class="path rect-path-1">→</div>
</div>
<div class="container">
  Rectangular path 2
  <div class="path rect-path-2">→</div>
</div>
<div class="container">
  Rectangular path 3
  <div class="path rect-path-3">→</div>
</div>
```

```
.container {
  position: relative;
  display: inline-block;
  width: 200px;
  height: 200px;
  border: 1px solid black;
  margin: 15px;
  text-align: center;
}

.path {
  width: 40px;
  height: 40px;
  background-color: red;
  position: absolute;
  animation: move 10s linear infinite;
}

.rect-path-1 {
  offset-path: rect(50px 150px 200px 50px round 20%);
}

.rect-path-2 {
  offset-path: rect(50px auto 200px 50px round 20%);
}

.rect-path-3 {
  offset-path: rect(50px auto 200px auto);
}

@keyframes move {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}
```

Result

- The path 1 rectangle specifies the distances of the four edges (top, right, bottom, and left) from the containing block. The top and bottom values are distances from the top edge of the containing block. The right and left values are distances from the left edge of the containing block. In addition, the corner of the rectangle is rounded at `20%`, making the red box element follow the rounded corners as it moves along this path. Notice how the arrow inside the box follows curve at the rectangular path corners.

- The path 2 rectangle is similar to the path 1 rectangle, except that the right value is `auto`, which is equal to the value `100%`. This causes the right edge of the rectangle to match the right edge of the containing block, creating a wider rectangle than path 1.
- The path 3 rectangle specifies both the left and right edge parameters as `auto` and omits the `round <'border-radius'>` parameter. This creates a rectangle that has the width of the containing block and rectangular corners instead of rounded corners like in path 1 and path 2 rectangles. Notice the movement of the arrow inside this box at the corners.

See also

- `inset()` function
- `xywh()` function
- `border-shape` property
- `clip-path` property
- `offset-path` property
- `<basic-shape>` data type
- CSS shapes module
- Guide to basic shapes

shape() CSS function

The **`shape()`** CSS function is used to define a shape for the `border-shape`, `clip-path`, and `offset-path` properties. It combines an initial starting point with a series of shape commands that define the path of the shape. The `shape()` function is a member of the `<basic-shape>` data type.

Syntax

```
/* <fill-rule> */
clip-path: shape(nonzero from 0 0, line to 10px 10px);

/* <move-command>, <line-command>, and close */
offset-path: shape(from 10px 10px, move by 10px 5px, line by 20px 40%, close);

/* <hvline-command> */
offset-path: shape(from 10px 10px, hline by 50px, vline to 5rem);

/* <curve-command> */
offset-path: shape(
  from 10px 10px,
  curve to 80px 80px with 160px 1px / 20% 16px
);

/* <smooth-command> */
offset-path: shape(from 10px 10px, smooth to 100px 50pt);

/* <arc-command> */
offset-path: shape(
  from 5% 0.5rem,
  arc to 80px 1pt of 10% ccw large rotate 25deg
);

/* Using a CSS math function */
offset-path: shape(
  from 5px -5%,
  hline to 50px,
  vline by calc(0% + 160px),
  hline by 70.5px,
  close,
  vline by 60px
);

clip-path: shape(
  evenodd from 10px 10px,
  curve to 60px 20% with 40px 0,
  smooth to 90px 0,
  curve by -20px 60% with 10% 40px / 20% 20px,
  smooth by -40% -10px with -10px 70px,
  close
);
```

Parameters

`<fill-rule>` Optional

Specifies how overlapping regions of a shape should be filled. The possible values include:

- `nonzero`: A point is considered inside the shape if a ray drawn from the point crosses more left-to-right than right-to-left path segments, resulting in a non-zero count. This is the default value when `<fill-rule>` is omitted.
- `evenodd`: A point is considered to be inside the shape if a ray drawn from the point crosses an odd number of path segments. This means that for each time the ray enters the shape, it has not exited an equal number of times, indicating an odd count of entries without corresponding exits.

> **Warning:** `<fill-rule>` is not supported in `offset-path` and using it invalidates the property.

`from <coordinate-pair>`

Defines the starting point of the first `<shape-command>` as a pair of coordinates that are measured from the top-left corner of the reference box. The coordinates are specified as space-separated `<x>` `<y>` `<length-percentage>` values representing the left offset and top offset, respectively. Percentage values are relative to the width and height of the element's reference box, respectively. Add a comma after this parameter.

`<shape-command>`

Specifies a list of one or more comma-separated commands that define the shape, using syntax similar to SVG path commands. Commands include `<move-command>`, `<line-command>`, `<hv-line-command>`, `<curve-command>`, `<smooth-command>`, `<arc-command>`, and `close`. Each command's starting point is the previous command's ending point, with the first point of the shape defined by the `from <coordinate-pair>` parameter.

Shape commands

The syntax of most shape commands is a keyword that provides a directive, such as `move` or `line`, followed by the `by` or `to` keyword, and a set of coordinates.

- `by`: Indicates that the `<coordinate-pair>` is relative to the command's starting point (a "relative" value).

- `to`: Indicates that the `<coordinate-pair>` is relative to the top-left corner of the reference box (an "absolute" value).

> **Note:** If a coordinate in a `<coordinate-pair>` is specified as a percentage, the value is calculated relative to the respective width or height of the reference box.

The following `<shape-command>`s can be specified:

`<move-command>`

Specified as `move [by | to] <coordinate-pair>`. This command adds a MoveTo command to the list of shape commands. It draws nothing; instead, it specifies the starting position for the next command. The `by` or `to` keyword specifies whether the `<coordinate-pair>` point is relative or absolute, respectively. If the `<move-command>` follows the `close` command, it identifies the starting point of the next shape or subpath.

`<line-command>`

Specified as `line [by | to] <coordinate-pair>`. This command adds a LineTo command to the list of shape commands. It draws a straight line from the command's starting point to its ending point. The `by` or `to` keyword specifies whether the ending point specified by `<coordinate-pair>` is relative or absolute, respectively.

`<hv-line-command>`

Specified as `[hline | vline] [by | to] <length-percentage>`. This command adds a horizontal (`hline`) or vertical (`vline`) LineTo command to the list of shape commands. With `hline`, a horizontal line is drawn from the command's starting point `to` or `by` the `x` position defined by `<length-percentage>`. With `vline`, a vertical line is drawn from the command's starting point `to` or `by` the `y` position defined by `<length-percentage>`. The `by` or `to` keyword determines the relative or absolute ending point, respectively. This command is equivalent to `<line-command>` with one coordinate value set by the single `<length-percentage>` and the other coordinate value remaining unchanged from its starting command.

`<curve-command>`

Specified as `curve [by | to] <end-point> with <control-point> [/ <control-point>]`. This command adds a Bézier curve command to the list of shape commands. The `by` or `to` keyword determines whether the ending point of the curve, specified by the `<end-point>`, is relative or absolute.

The `with` keyword specifies the control points of the Bézier curve as follows.

- If only a single `<control-point>` is provided, the command draws a quadratic Bézier curve, which is defined by three points (the start point, control point, and end point).
- If two `<control-point>` values are provided, the command draws a cubic Bézier curve, which is defined by four points (the start point, two control points, and the end point).

Valid values for `<end-point>` include:

`<position>` keywords or a `<coordinate-value-pair>`

Can be used if the curve end point is absolute (specified using `to`).

`<coordinate-value-pair>`

Can be used if the curve end point is relative (specified using `by`).

Valid values for `<control-point>` include:

`<position>`

Specifies a position keyword. This value is valid only when the curve end point is absolute (specified using `to`).

`<coordinate-value-pair>`

Specifies a pair of `<length-percentage>` values that define coordinates.

`<relative-control-point>`

Defines a `<coordinate-value-pair>` followed by the `from` keyword and one of the following keywords:

`start`

Indicates that the control point is relative to the start point of the current command.

`end`

Indicates that the control point is relative to the end point of the current command.

`origin`

Indicates that the control point is relative to the top-left (origin) point of the container the shape is being drawn inside.

> **Note:** If the `<relative-control-point>` keywords are not specified, making the `<control-point>` a regular `<coordinate-value-pair>`, the coordinates are relative to the start of the curve. In other words, `start` is the default setting.

`<smooth-command>`

Specified as `smooth [by | to] <end-point> [with <control-point>]`. This command adds a smooth Bézier curve command to the list of shape commands. The `by` or `to` keyword determines whether the ending point of the curve, specified by the `<end-point>`, is relative or absolute.

The `with` keyword specifies an optional control point for the Bézier curve:

- If `with <control-point>` is omitted, the command draws a smooth quadratic Bézier curve, which uses the previous control point and the current endpoint to define the curve.
- If the optional `with` keyword is included, it specifies a control points of the curve through `<control-point>`, drawing a smooth cubic Bézier curve defined by the previous control point, the current control point, and the current endpoint.

Smooth curves ensure a continuous transition from the shape, while quadratic curves do not. Smooth quadratic curves maintain a seamless transition using a single control point, whereas smooth cubic curves provide a more refined transition using two control points.

Valid values for the `<end-point>` and `<control-point>` components are the same as those for `<curve-command>`.

`<arc-command>`

Specified as `arc [by | to] <coordinate-pair> of <length-percentage> [<length-percentage>] [<arc-sweep> | <arc-size> | rotate <angle>]`. This command adds an elliptical arc curve command to the list of shape commands. It draws an elliptical arc between a starting point and an ending point. The `by` or `to` keyword determines whether the ending point of the curve, specified by the first `<coordinate-pair>`, is relative or absolute, respectively.

The elliptical arc curve command defines two possible ellipses, which intersect both the starting and ending points, and each can be traced clockwise or counterclockwise, resulting in four possible arcs depending on the arc size, direction, and angle. The `of` keyword specifies the size of the ellipse from which the arc is taken: the first `<length-percentage>` provides the horizontal radius of the ellipse, and the

second `<length-percentage>` provides the vertical radius.

Specify the following parameters to choose which of the four arcs to use:

- `<arc-sweep>`: Indicates whether the desired arc is the one traced around the ellipse clockwise (`cw`) or counterclockwise (`ccw`). If omitted, this defaults to `ccw`.
- `<arc-size>`: Indicates whether the desired arc is the larger (`large`) or smaller (`small`) of the two arcs. If omitted, this defaults to `small`.
- `<angle>`: Specifies the angle, in degrees, by which to rotate the ellipse relative to the x-axis. A positive angle rotates the ellipse clockwise, and a negative angle rotates it counterclockwise. If omitted, this defaults to `0deg`.

Special situations are handled as follows:

- If only one `<length-percentage>` is provided, the same value is used for both the horizontal and vertical radii, effectively creating a circle. In this case, `<arc-size>` and `<angle>` have no effect.
- If either radius is zero, the command is equivalent to a `<line-command>` to the ending point.
- If either radius is negative, its absolute value is used instead.
- If the horizontal and vertical radii don't describe an ellipse large enough to intersect both the starting point and ending points (after rotation by the specified `<angle>`), the radii are scaled up uniformly until the ellipse is just large enough to intersect both points.
- If the starting and ending points of the arc lie on exactly opposite sides of the ellipse, there is only one possible ellipse and two possible arcs. In this case, `<arc-sweep>` specifies the arc to choose, and `<arc-size>` has no effect.

`close`

Adds a ClosePath command to the list of shape commands, drawing a straight line from the current position (end of the last command) to the first point in the path defined in the `from <coordinate-pair>` parameter. To close the shape without drawing a line, include a `<move-command>` with the originating coordinates before the close command. If the `close` command is immediately followed by a `<move-command>`, it defines the starting point of the next shape or subpath.

Description

The `shape()` function allows you to define complex shapes. It is similar to the `path()` shape function in several ways:

- The `<fill-rule>` parameter in the `shape()` function works exactly like the same parameter in the `path()` function.
- The `shape()` function requires specifying one or more `<shape-command>`s, where each command uses an underlying path command, such as MoveTo, LineTo, and ClosePath.

However, `shape()` offers several advantages over using `path()`:

- `shape()` uses standard CSS syntax, making it easier to create and modify shapes directly in your stylesheet. In comparison, `path()` uses the SVG path syntax, which is less intuitive for those not familiar with SVG.
- `shape()` supports a variety of CSS units, including percentages, `rem`, and `em`. `path()`, on the other hand, defines shapes as a single string and limits units to `px`.
- `shape()` also allows the use of CSS math functions, like `calc()`, `max()`, and `abs()`, providing more versatility when defining shapes.

Formal syntax

```
&lt;shape()&gt; =
  shape( &lt;'fill-rule'&gt;? from &lt;position&gt; , &lt;shape-command&gt;# )

&lt;fill-rule&gt; =
  nonzero  |
  evenodd

&lt;position&gt; =
  &lt;position-one&gt;   |
  &lt;position-two&gt;   |
  &lt;position-four&gt;

&lt;shape-command&gt; =
  &lt;move-command&gt;             |
  &lt;line-command&gt;             |
  close                       |
  &lt;horizontal-line-command&gt;  |
  &lt;vertical-line-command&gt;    |
  &lt;curve-command&gt;            |
  &lt;smooth-command&gt;           |
  &lt;arc-command&gt;

&lt;position-one&gt; =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  &lt;length-percentage&gt;

&lt;position-two&gt; =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | &lt;length-percentage&gt; ] [ top | center | bottom | y-start | y-end | &lt;length-percentage&gt; ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

&lt;position-four&gt; =
  [ [ left | right | x-start | x-end ] &lt;length-percentage&gt; ] && [ [ top | bottom | y-start | y-end ] &lt;length-percentage&gt; ]  |
  [ [ block-start | block-end ] &lt;length-percentage&gt; ] && [ [ inline-start | inline-end ] &lt;length-percentage&gt; ]  |
  [ [ start | end ] &lt;length-percentage&gt; ]{2}

&lt;move-command&gt; =
  move &lt;command-end-point&gt;

&lt;line-command&gt; =
  line &lt;command-end-point&gt;

&lt;horizontal-line-command&gt; =
  hline [ to [ &lt;length-percentage&gt; | left | center | right | x-start | x-end ] | by &lt;length-percentage&gt; ]

&lt;vertical-line-command&gt; =
  vline [ to [ &lt;length-percentage&gt; | top | center | bottom | y-start | y-end ] | by &lt;length-percentage&gt; ]

&lt;curve-command&gt; =
  curve [ [ to &lt;position&gt; with &lt;control-point&gt; [ / &lt;control-point&gt; ]? ] | [ by &lt;coordinate-pair&gt; with &lt;relative-control-point&gt; [ / &lt;relative-control-point&gt; ]? ] ]
```

```
<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<length-percentage> =
  <length>      |
  <percentage>

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

This syntax reflects the latest standard as per CSS Fill and Stroke Module Level 3, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using <code>shape()</code> to define a path

This example demonstrates how the `shape()` function can be used in the `offset-path` property to define the shape of the path an element can follow.

The first shape, `shape1`, follows a cubic Bézier curved path defined by the `curve to` command. Next, the `close` command draws a straight line from the curve's end point back to the initial point defined in the `from` command. Finally, `shape1` moves to its new position at `0px 150px` and then proceeds along a horizontal line.

The second shape, `shape2`, initially follows a horizontal line, then moves back to its starting position at `50px 90px`. Next, it follows a vertical line before closing the path back to the initial point.

Both shapes start with their original colors and gradually transition to `hotpink` by the end of the `move` animation, reverting to their initial color as the animation restarts. This cyclic color change provides you a visual cue about the animation's progression and restart.

```
<div class="container">
  Using <code>&lt;curve-command&gt;</code>
  <div class="shape shape1">>></div>
</div>

<div class="container">
  Using <code>&lt;move-command&gt;</code> and
  <code>&lt;hvline-command&gt;</code>
  <div class="shape shape2">>></div>
</div>
```

```
body {
  align-items: center;
  justify-content: center;
  display: flex;
}

.container {
  position: relative;
  display: inline-block;
  width: 250px;
  height: 250px;
  border: 2px dotted green;
  margin: 20px;
}

@supports not (offset-path: shape(from 0 0, move to 0 0)) {
  body::after {
    content: "Your browser doesn't support the `shape()` function yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
.shape {
  width: 50px;
  height: 50px;
  background: #2bc4a2;
  position: absolute;
  text-align: center;
  line-height: 50px;
  animation: move 6s infinite linear;
}

.shape1 {
  offset-path: shape(
    from 30% 60px,
    curve to 180px 180px with 90px 190px,
    close,
    move by 0px 150px,
    hline by 40%
```

```
  );
}

.shape2 {
  offset-path: shape(
    from 50px 90px,
    hline to 8em,
    move to 50px 90px,
    vline by 20%,
    close
  );
}

@keyframes move {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
    background-color: hotpink;
  }
}
```

Result

Using <code>shape()</code> to define the visible part of an element

This example demonstrates how the `shape()` function can be used in the `clip-path` property to create different shapes for the clipping region. The first shape (`shape1`) uses a triangle defined by straight lines. The second shape (`shape2`) includes curves and smooth transitions; it also illustrates the use of the `<move-command>` after the `close` command, which adds a rectangular shape to the clipping region.

```
<div class="container">
  <div class="shape shape1"></div>
</div>

<div class="container">
  <div class="shape shape2"></div>
</div>
```

```
body {
  align-items: center;
  justify-content: center;
  display: flex;
}

.container {
  position: relative;
  display: inline-block;
  width: 200px;
  height: 200px;
  margin: 20px;
  background-color: lightgray;
}

@supports not (clip-path: shape(from 0 0, move to 0 0)) {
  body::after {
    content: "Your browser doesn't support the `shape()` function yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
.shape {
  width: 100%;
  height: 100%;
  background: #2bc4a2;
  position: absolute;
  text-align: center;
  line-height: 50px;
}

/* Triangular clipping region */
.shape1 {
  clip-path: shape(from 0% 0%, line to 100% 0%, line to 50% 100%, close);
}

/* A Heart clipping region using curve and arc transitions
   and a box using hline and vline transitions */
.shape2 {
  clip-path: shape(
    from 20px 70px,
    arc to 100px 70px of 1% cw,
    arc to 180px 70px of 1% cw,
    curve to 100px 190px with 180px 130px,
    curve to 20px 70px with 20px 130px,
    close,
    move to 150px 150px,
    hline by 40px,
    vline by 40px,
    hline by -40px,
    close
  );
}
```

Result

Using <code>shape()</code> to draw curves with relative control points

Like previous examples, this example also uses `clip-path` to create different shapes for the clipping regions of the elements. The shapes are specified using a combination of `<curve-command>` and `<smooth-command>`, and the control points are specified using `<relative-control-point>` values.

The first shape (`shape1`) draws two cubic Bézier curves.

- The first curve starts from the center of the left edge of the box and is drawn to a point `200px` along the x-axis — the center of the box's right edge. It uses one control point relative to the start of the curve and one control point relative to the origin (top-left of the box).
- The second curve starts from the center right of the box and is drawn `-200px` along the x-axis — the center of the box's left edge. It uses one control point relative to the origin and one control point relative to the start of the curve.

```
<div class="container">
  <div id="shape1"></div>
  <div id="shape2"></div>
  <div id="shape3"></div>
</div>
```

```
.container {
  display: flex;
  justify-content: center;
  gap: 20px;
}

@supports not (
  clip-path: shape(
      from center left,
      curve by 200px 0 with 50% -50% from start / 50% 0 from origin,
      curve by -200px 0 with 50% 100% from origin / -50% 50% from start,
      close
    )
) {
  body::after {
    content: "Your browser doesn't support `shape()` relative control points.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
#shape1 {
  width: 200px;
  height: 200px;
  background: green;
  clip-path: shape(
    from center left,
    curve by 200px 0 with 50% -50% from start / 50% 0 from origin,
    curve by -200px 0 with 50% 100% from origin / -50% 50% from start,
    close
  );
}
```

The second shape (`shape2`) draws one quadratic Bézier curve and one cubic Bézier curve.

- The first curve starts from the center of the left edge of the box and is drawn to an absolute point `200px` from the origin along the x-axis and `100px` from the origin along the y-axis. It uses one control point relative to the start of the curve.
- The second curve starts from the previous curve's end point and is drawn to the center left of the box. It uses one control point relative to the start of the curve and one control point relative to the end.

```
#shape2 {
  width: 200px;
  height: 200px;
  background: purple;
  clip-path: shape(
    from center left,
    curve to 200px 100px with 50% -80% from start,
    curve to center left with 0% 70% from start / 20% 0% from end,
    close
  );
}
```

The third shape (`shape3`) draws one quadratic Bézier curve and one cubic Bézier curve using a `smooth` command.

- The first curve starts from the center of the left edge of the box and is drawn to a point `200px` along the x-axis. It uses one control point relative to the start of the curve.
- The second curve starts from the previous curve's end point and is drawn to the center of the box. It uses one control point relative to the start of the curve (the last control point of the previous curve) and one control point relative to the origin.

```
#shape3 {
  width: 200px;
  height: 200px;
  background: orangered;
  clip-path: shape(
    from center left,
    curve by 200px 0px with 50% -80% from start,
    smooth to center with 50% 100% from origin,
    close
  );
}
```

Result

See also

- `border-shape`
- `clip-path`
- `offset-path`
- CSS shapes module
- Overview of shapes guide
- Basic shapes guide

xywh() CSS function

The **`xywh()`** CSS function creates a rectangle using the specified distances from the left (x) and top (y) edges of the containing block and the specified width (w) and height (h) of the rectangle. It is a basic shape function of the `<basic-shape>` data type. You can use the `xywh()` function in CSS properties such as `offset-path` to create the rectangular path along which an element moves, in `clip-path` to define the shape of the clipping region, and in `border-shape` to define the shape of an element's border.

Syntax

```
offset-path: xywh(0 1% 2px 3% round 0 1px 2% 3px);
clip-path: xywh(1px 2% 3px 4em round 0 1% 2px 3em);
border-shape: xywh(5% 5% 90% 90% round 20px);
```

Values

`<length-percentage>`

Specifies the `<length-percentage>` values for the x and y coordinates of the rectangle.

`<length-percentage [0,∞]>`

Specifies non-negative `<length-percentage>` values for the width and height of the rectangle. The minimum value can be zero, and the maximum value has no limit.

`round <'border-radius'>`

Specifies the radius of the rounded corners of the rectangle using the same syntax as the CSS `border-radius` shorthand property. This parameter is optional.

Formal syntax

```
<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<length-percentage> =
  <length>      |
  <percentage>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4, CSS Shapes Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Creating offset-path using xywh()

In the example below, the `offset-path` property uses the `xywh()` function to define the shape of the path on which the element, a magenta box in this case, moves. Two different scenarios are shown, each with different values for the `xywh()` function. The arrow inside the boxes points to the right edge of the box.

```
<div class="container">
  Rectangular path 1
  <div class="path xywh-path-1">→</div>
</div>
<div class="container">
  Rectangular path 2
  <div class="path xywh-path-2">→</div>
</div>
```

```
.container {
  position: relative;
  display: inline-block;
  width: 200px;
  height: 200px;
  border: 1px solid black;
  margin: 30px;
  text-align: center;
}

.path {
  width: 50px;
  height: 50px;
  position: absolute;
  background-color: magenta;
  animation: move 10s linear infinite;
}

.xywh-path-1 {
  offset-path: xywh(20px 20px 100% 100% round 10%);
}

.xywh-path-2 {
  offset-path: xywh(20px 30% 150% 200%);
}

@keyframes move {
  0% {
    offset-distance: 0%;
  }
  100% {
    offset-distance: 100%;
  }
}
```

Result

- The path 1 rectangle is offset by `20px` from the left and top edges of the containing block. This path rectangle has the same dimension as the containing block, that is, the width is `100%` of the width of the containing block, and the height is `100%` of the height of the containing block. Notice how the arrow inside the box follows the `10%` curve (defined by `round 10%`) at the rectangular path corners.
- As the upper limit of both width and height in `xywh()` is infinity, setting the height to `200%` in the path 2 rectangle makes the generated rectangle twice as tall as the containing block. Notice how the arrow inside the box behaves at the corners when no `round <'border-radius'>` is specified.

See also

- `inset()` function
- `rect()` function
- `border-shape` property
- `clip-path` property
- `offset-path` property
- `<basic-shape>` data type
- CSS shapes module
- Guide to basic shapes

<blend-mode> CSS type

The **`<blend-mode>`** CSS data type describes how colors should appear when elements overlap. It is used in the `background-blend-mode` and `mix-blend-mode` properties.

Syntax

The `<blend-mode>` data type is defined using a keyword value chosen from the list below.

Values

`normal`

The final color is the top color, regardless of what the bottom color is. The effect is like two opaque pieces of paper overlapping.

`multiply`

The final color is the result of multiplying the top and bottom colors. A black layer leads to a black final layer, and a white layer leads to no change. The effect is like two images printed on transparent film overlapping.

`screen`

The final color is the result of inverting the colors, multiplying them, and inverting that value. A black layer leads to no change, and a white layer leads to a white final layer. The effect is like two images shining onto a projection screen.

`overlay`

The final color is the result of `multiply` if the bottom color is darker, or `screen` if the bottom color is lighter. This blend mode is equivalent to `hard-light` but with the layers swapped.

`darken`

The final color is composed of the darkest values of each color channel.

`lighten`

The final color is composed of the lightest values of each color channel.

`color-dodge`

The final color is the result of dividing the bottom color by the inverse of the top color. A black foreground leads to no change. A foreground with the inverse color of the backdrop leads to a fully lit color. This blend mode is similar to `screen`, but the foreground only needs to be as light as the inverse of the backdrop to create a fully lit color.

`color-burn`

The final color is the result of inverting the bottom color, dividing the value by the top color, and inverting that value. A white foreground leads to no change. A foreground with the inverse color of the backdrop leads to a black final image. This blend mode is similar to `multiply`, but the foreground only needs to be as dark as the inverse of the backdrop to make the final image black.

`hard-light`

The final color is the result of `multiply` if the top color is darker, or `screen` if the top color is lighter. This blend mode is equivalent to `overlay` but with the layers swapped. The effect is similar to shining a *harsh* spotlight on the backdrop.

`soft-light`

The final color is similar to `hard-light`, but softer. This blend mode behaves similar to `hard-light`. The effect is similar to shining a *diffused* spotlight on the backdrop.

`difference`

The final color is the result of subtracting the darker of the two colors from the lighter one. A black layer has no effect, while a white layer inverts the other layer's color.

`exclusion`

The final color is similar to `difference`, but with less contrast. As with `difference`, a black layer has no effect, while a white layer inverts the other layer's color.

`hue`

The final color has the *hue* of the top color, while using the *saturation* and *luminosity* of the bottom color.

`saturation`

The final color has the *saturation* of the top color, while using the *hue* and *luminosity* of the bottom color. A pure gray backdrop, having no saturation, will have no effect.

`color`

The final color has the *hue* and *saturation* of the top color, while using the *luminosity* of the bottom color. The effect preserves gray levels and can be used to colorize the foreground.

`luminosity`

The final color has the *luminosity* of the top color, while using the *hue* and *saturation* of the bottom color. This blend mode is equivalent to `color`, but with the layers swapped.

Description

For each pixel among the layers to which it is applied, a blend mode takes the colors of the foreground and the background, performs a calculation on them, and returns a new color value.

Changes between blend modes are not interpolated. Any change occurs immediately.

Formal syntax

```
<blend-mode> =
  normal       |
  darken       |
  multiply     |
```

```
  color-burn   |
  lighten      |
  screen       |
  color-dodge  |
  overlay      |
  soft-light   |
  hard-light   |
  difference   |
  exclusion    |
  hue          |
  saturation   |
  color        |
  luminosity
```

This syntax reflects the latest standard as per Compositing and Blending Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Example using "normal"

```
<div class="container"></div>
```

```
.container {
  width: 150px;
  height: 150px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: normal;
}
```

Set other values for `background-blend-mode`, you will get different results.

Comparison between different values with <code>background-blend-mode</code>

```
<div class="container"></div>
```

```
.container {
  width: 720px;
  height: 760px;
  display: grid;
  grid: auto-flow 190px / repeat(4, 180px);
  border-top: 1px solid #d8d8d8;
  border-left: 1px solid #d8d8d8;
}

.container > div {
  border-right: 1px solid #d8d8d8;
  border-bottom: 1px solid #d8d8d8;
}

.container div div {
  margin-left: 15px;
  width: 150px;
  height: 150px;
  background: url("br.png"), url("tr.png");
}

.container div p {
  line-height: 30px;
  margin: 0;
  color: #a33333;
  text-align: center;
}
```

```
const list = [
  "normal",
  "multiply",
  "screen",
  "overlay",
  "darken",
  "lighten",
  "color-dodge",
  "color-burn",
  "hard-light",
  "soft-light",
  "difference",
  "exclusion",
  "hue",
  "saturation",
  "color",
  "luminosity",
];

const containerElem = document.querySelector(".container");

list.forEach((item) => {
  const innerElem = document.createElement("div");
  innerElem.style.backgroundBlendMode = item;

  const textElem = document.createElement("p");
  textElem.innerText = item;

  const outerElem = document.createElement("div");
  outerElem.appendChild(textElem);
  outerElem.appendChild(innerElem);

  containerElem.appendChild(outerElem);
});
```

Create multiple `div` elements by traversing a list and set different `backgroundBlendMode` value for each.

Comparison between different values with <code>mix-blend-mode</code>

```
<div class="container"></div>
```

```
.container {
  width: 640px;
  height: 800px;
  display: grid;
  grid: auto-flow 160px / repeat(4, 160px);
  border-top: 1px solid #d8d8d8;
  border-left: 1px solid #d8d8d8;
}

.container > div {
  border-right: 1px solid #d8d8d8;
  border-bottom: 1px solid #d8d8d8;
}

.container > div > div {
  position: relative;
  margin-left: 20px;
  width: 120px;
  height: 120px;
}

.container div p {
  margin: 0;
  line-height: 30px;
  color: #a33333;
  text-align: center;
}
.circle {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  mix-blend-mode: screen;
  position: absolute;
}

.circle-1 {
  background: red;
}

.circle-2 {
  background: lightgreen;
  left: 40px;
}

.circle-3 {
  background: blue;
  left: 20px;
  top: 40px;
}

.isolate {
  isolation: isolate; /* Without isolation, the background color will be taken into account */
  position: relative;
}
```

```
const list = [
  "normal",
  "multiply",
  "screen",
  "overlay",
  "darken",
  "lighten",
  "color-dodge",
  "color-burn",
  "hard-light",
  "soft-light",
  "difference",
  "exclusion",
  "hue",
  "saturation",
  "color",
  "luminosity",
  "plus-darker",
  "plus-lighter",
];

const containerElem = document.querySelector(".container");

list.forEach((item) => {
  const innerElem = document.createElement("div");
  innerElem.innerHTML = `
    <div class="circle circle-1"></div>
    <div class="circle circle-2"></div>
    <div class="circle circle-3"></div>
  `;

  innerElem.querySelectorAll(".circle").forEach((circle) => {
    circle.style.mixBlendMode = item;
  });

  const textElem = document.createElement("p");
  textElem.innerText = item;

  const outerElem = document.createElement("div");
  outerElem.appendChild(textElem);
  outerElem.appendChild(innerElem);
```

```
  containerElem.appendChild(outerElem);
});
```

In the following example, we create multiple `<div>` elements by traversing a list and set different `mixBlendMode` values for each.

Blend mode comparison

In the following example, we have a `<div>` with two background images set on it — a Firefox logo on top of a linear gradient. Below it, we have a provided a `<select>` menu that allows you to change the `background-blend-mode` applied to the `<div>`, allowing you to compare the different blend mode effects.

HTML

```
<div></div>
<p>Choose a blend-mode:</p>
<select>
  <option selected>normal</option>
  <option>multiply</option>
  <option>screen</option>
  <option>overlay</option>
  <option>darken</option>
  <option>lighten</option>
  <option>color-dodge</option>
  <option>color-burn</option>
  <option>hard-light</option>
  <option>soft-light</option>
  <option>difference</option>
  <option>exclusion</option>
  <option>hue</option>
  <option>saturation</option>
  <option>color</option>
  <option>luminosity</option>
</select>
```

CSS

```
div {
  width: 300px;
  height: 300px;
  background:
    url("https://mdn.dev/archives/media/attachments/2020/07/29/17350/3b4892b7e820122ac6dd7678891d4507/firefox.png")
      no-repeat center,
    linear-gradient(to bottom, blue, orange);
}
```

JavaScript

```
const selectElem = document.querySelector("select");
const divElem = document.querySelector("div");

selectElem.addEventListener("change", () => {
  divElem.style.backgroundBlendMode = selectElem.value;
});
```

Result

See also

- The CSS compositing and blending module that defines the `<blend-mode>` values.
- Properties that use this data type: `background-blend-mode`, `mix-blend-mode`

Description to various blend modes on other website:

- Blend modes on Wikipedia
- Blending modes in Adobe Photoshop by Adobe

<box-edge> CSS type

The **`<box-edge>`** value types represent a box edge keyword, such as `content-box` and `border-box`. The box-edge keywords are used to define different aspects of an element's box model and how elements are positioned and rendered on screen.

The box-edge keywords are the components of, but not limited to, the data types `<visual-box>`, `<layout-box>`, `<paint-box>`, `<coord-box>`, and `<geometry-box>`. These types are applied to properties such as `transform-box` and `background-clip`.

Syntax

```
<visual-box> = content-box | padding-box | border-box /* the three <box> values */
<layout-box> or <shape-box> = <visual-box> | margin-box
<paint-box> = <visual-box> | fill-box | stroke-box
<coord-box> = <paint-box> | fill-box | stroke-box | view-box
<geometry-box> = <shape-box> | fill-box | stroke-box | view-box
```

Values

A `<box-edge>` can be of the type `<visual-box>`, `<layout-box>`, `<paint-box>`, `<coord-box>`, or `<geometry-box>`.

`<visual-box>`

> Refers to the rectangular box generated for an element as seen by a user on a web page. It includes the element's content, padding, and border. Also referred to as `<box>`, this value excludes the margin area. This value type is used for the `background-clip` and `overflow-clip-margin` properties.

`<layout-box>`

> Refers to the space occupied by an element, including its content, padding, border, and margin. This value type is used for layout and positioning purposes. Also referred to as `<shape-box>`, this value type is used for the `shape-outside` property.

`<paint-box>`

> Refers to the area within the layout box that is used to visually render the content. This includes the area where the element's background and borders are painted. As an element's paintable area does not include its margins, this value excludes `margin-box`.

`<coord-box>`

Refers to the coordinate box used for positioning and sizing an element within its parent container. It is used to control how content flows around the edges of the box. It excludes the margin area. This value type is used for the `offset-path` property.

`<geometry-box>`

Defines the reference box for a basic shape, or if specified by itself, causes the edges of the specified box, including any corner shaping (such as a `border-radius`), to be the clipping path. This value type is used for the `clip-path`, `mask-clip`, and `mask-origin` properties and the SVG `clip-path` attribute; it is also used as part of the `border-shape` property values.

Keywords

The `<box-edge>` keywords are defined as follows:

`content-box`

Refers to the outer edge of the box's content area. The content box is the innermost box. The content area contains the actual content, such as text, images, or other HTML elements. In SVG, this value is treated as `fill-box`.

`padding-box`

Refers to the outer edge of the padding of the box. If there is no padding on a side, then the value is the same as `content-box`. In SVG, `padding-box` is treated as `fill-box`. The padding area surrounds the content area, starting at the outer edge of the content box.

`border-box`

Refers to the outer edge of the border of the box. If there is no border on a side, then the value is the same as `padding-box`. In SVG, `border-box` is treated as `stroke-box`. The border area surrounds the padding area, starting at the outer edge of the padding box.

`half-border-box`

Refers to a box shape that traverses the center of the border region; the inner half of the border region is inside the shape, and the outer half of the border region is outside the shape.

`margin-box`

Refers to the outer edge of the margin of the box. If there is no margin on a side, then the value is the same as `border-box`. In SVG, `margin-box` is treated as `stroke-box`.

`fill-box`

Refers to the object bounding box in SVG. In CSS, `fill-box` is treated as `content-box`. It is used to wrap the content around the edges defined by the `coord-box` values.

`stroke-box`

Refers to the stroke bounding box in SVG. In CSS, `stroke-box` is treated as `border-box`. It is used to define the shape of the element when strokes are applied.

`view-box`

Refers to the nearest SVG viewport element's origin box. The origin box is a rectangle with the width and height of the initial SVG user coordinate system established by the `viewBox` attribute for that element. The origin box is positioned such that its top left corner is anchored at the coordinate system origin. In CSS, `view-box` is treated as `border-box`.

> **Note:** When the SVG viewport is not anchored at the origin, the origin box does not correspond to the SVG viewport.

See also

- CSS box model module

calc() CSS function

The **`calc()`** CSS function lets you perform calculations when specifying CSS property values. It can be used with `<length>`, `<frequency>`, `<angle>`, `<time>`, `<percentage>`, `<number>`, `<integer>`, and `<color-function>` values.

Try it

```
width: calc(10px + 100px);
```

```
width: calc(100% - 30px);
```

```
width: calc(2em * 5);
```

```
width: calc(var(--variable-width) + 20px);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">Change my width.</div>
</section>
```

```
:root {
  --variable-width: 100px;
}

#example-element {
  border: 10px solid black;
  padding: 10px;
}
```

Syntax

```
/* calc(expression) */
calc(100% - 80px)

/* Expression with a CSS function */
calc(100px * sin(pi / 2))

/* Expression containing a variable */
calc(var(--hue) + 180)

/* Expression with color channels in relative colors */
lch(from aquamarine l c calc(h + 180))
```

The `calc()` function takes a single expression as its parameter, and the expression's result is used as the value for a CSS property. In this expression, the operands can be combined using the operators listed below. When the expression contains multiple operands, `calc()` uses the standard operator precedence rules:

`+`

Adds the specified operands.

`-`

Subtracts the second operand from the first operand.

`*`

Multiplies the specified operands.

`/`

Divides the left-side operand (dividend) by the right-side operand (divisor).

All operands, except those of type `<number>`, must be suffixed with an appropriate unit string, such as `px`, `em`, or `%`. You can use a different unit with each operand in your expression. You may also use parentheses to establish computation order when needed.

Description

There are a few points to note about `calc()`, which are detailed in the sections below.

Resulting values

The `calc()` function must stand in place of a full CSS value of one of the following types:

- `<length>`
- `<frequency>`
- `<angle>`
- `<time>`
- `<flex>`
- `<resolution>`
- `<percentage>`
- `<number>`
- `<integer>`
- One of the mixed types like `<length-percentage>`

`calc()` cannot only replace the numeric part of percentage values, length values, etc., without also replacing the unit after it. For example, `calc(100 / 4)%` is invalid, while `calc(100% / 4)` is valid.

The resulting value of `calc()` must be compatible with the context in which it is used. For example, `margin: calc(1px + 2px)` is valid, but `margin: calc(1 + 2)` is not: it is equivalent to specifying `margin: 3`, which results in the property being ignored.

When an `<integer>` is expected, the `calc()` expression can also evaluate to a `<number>`, which gets rounded to the nearest integer. So, `calc(1.4)` will result in a value of `1`. If the fractional part of the value is exactly `0.5`, the value is rounded towards positive infinity. For example, `calc(1.5)` will result in a value of `2`, while `calc(-1.5)` will round to `-1`.

`calc()` performs floating point math following the IEEE-754 standard, which results in some considerations concerning the `infinity` and `NaN` values. For more details on how constants are serialized, see the `<calc-keyword>` page.

Input considerations

- `calc()` cannot perform calculations on intrinsic size values such as `auto` and `fit-content`. Use the `calc-size()` function instead.
- The `*` and `/` operators do not require whitespace, but adding it for consistency is recommended.
- It is permitted to nest `calc()` functions, in which case, the inner ones are treated as simple parentheses.
- Math expressions involving percentages for widths and heights on table columns, table column groups, table rows, table row groups, and table cells in both auto and fixed layout tables *may* be treated as if `auto` is specified.
- See `<calc-sum>` for more information on the syntax of `+` and `-` expressions.

CSS typed arithmetic

When using `calc()` to multiply values, using the `*` operator, only one value can contain a unit. Calculations like `200px * 4px` are not supported as 800px^2 is meaningless in CSS.

Inversely, `200px / 4px` resolves to `50`, which does make sense in CSS. Therefore, when using the `calc()` function to divide numbers, using the `/` operator, supporting browsers allow units on both sides of the operand, provided they are of the same data type. For example, `100vw / 1px` is valid and results in a unitless value.

The quotient can then be used in the values of properties or functions that accept a `<number>` as a value or parameter, or converted into a different data type by multiplying it by a typed value.

For a full explanation of typed arithmetic in CSS, along with examples, see Using CSS typed arithmetic.

Support for computing color channels in relative colors

The `calc()` function can be used to manipulate color channels directly within the context of relative colors. This allows for dynamic adjustments of color channels in color models such as `rgb()`, `hsl()`, and `lch()`.

The relative color syntax defines several color-channel keywords, each of which represents the value of the color channel as a `<number>` (see Channel values resolve to `<number>` values for more information). The `calc()` function can use these color-channel keywords to perform dynamic adjustments on the color channels, for example, `calc(r + 10)`.

Formal syntax

```
<calc()> =
  calc( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
```

```
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

When `calc()` is used for controlling text size, be sure that one of the values includes a relative length unit, for example:

```
h1 {
  font-size: calc(1.5rem + 3vw);
}
```

This ensures that text size will scale if the page is zoomed.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4: Resize Text | WAI | W3C

Examples

Positioning an object on screen with a margin

`calc()` enables positioning an object with a set margin. In this example, the CSS creates a banner that stretches across the window, with a 40-pixel gap between both sides of the banner and the edges of the window:

```
.banner {
  position: absolute;
  left: 40px;
  width: calc(100% - 80px);
  border: solid black 1px;
  box-shadow: 1px 2px;
  background-color: yellow;
  padding: 6px;
  text-align: center;
  box-sizing: border-box;
}
```

```
<div class="banner">This is a banner!</div>
```

Automatically sizing form fields to fit their container

Another use case for `calc()` is to help ensure that form fields fit in the available space, without extruding past the edge of their container while maintaining an appropriate margin.

Let's look at some CSS:

```
input {
  padding: 2px;
  display: block;
  width: calc(100% - 1em);
}

#form-box {
  width: calc(100% / 6);
  border: 1px solid black;
  padding: 4px;
}
```

Here, the container is established to use 1/6 of the available window width. Then, to ensure that input fields retain an appropriate size, we use `calc()` again to establish that they should be the width of their container minus 1em. Then, the following HTML makes use of this CSS:

```
<div id="form-box">
  <label for="misc">Type something:</label>
  <input type="text" id="misc" name="misc" />
</div>
```

Nesting with CSS variables

You can use `calc()` with CSS variables. Consider the following code:

```
.foo {
  --width-a: 100px;
  --width-b: calc(var(--width-a) / 2);
  --width-c: calc(var(--width-b) / 2);
  width: var(--width-c);
}
```

After all variables are expanded, `--width-c`'s value will be `calc(calc(100px / 2) / 2)`. When it's assigned to `.foo`'s width property, all inner `calc()` functions (no matter how deeply nested) will be flattened to just parentheses. Therefore, the `width` property's value will eventually be `calc((100px / 2) / 2)`, which equals `25px`. In short, a `calc()` inside of a `calc()` is identical to using parentheses.

Adjusting color channels in relative colors

The `calc()` function can be used to adjust individual color channels in relative colors without the need for storing color channel values as variables.

In the example below, the first paragraph uses a `<named-color>`. In the paragraphs that follow, `calc()` is used with the `rgb()` and `hsl()` functions to adjust the values of each color channel relative to the original named color.

```
<p class="original">Original text color in rebeccapurple</p>
<p class="increase-hue">Hue increased by 80</p>
<p class="increase-lightness">Lightness increased by 20</p>
<p class="decrease-lightness">Lightness decreased by 10</p>
```

```
p {
  font-family: monospace;
  font-size: 16px;
}
```

```
.original {
  color: rebeccapurple;
}

.increase-hue {
  color: lch(from rebeccapurple l c calc(h + 80));
}

.increase-lightness {
  color: lch(from rebeccapurple calc(l + 20) c h);
}

.decrease-lightness {
  color: lch(from rebeccapurple calc(l - 10) c h);
}
```

For another example of using the `calc()` function to derive relative colors, see the Using math functions section on the *Using relative colors* page.

See also

- `<calc-sum>`
- `<calc-keyword>`
- CSS functions
- A Complete Guide to calc() in CSS (CSS-Tricks)

<calc-keyword> CSS type

The **`<calc-keyword>`** CSS data type represents well-defined constants such as `e` and `pi`. Rather than require authors to manually type out several digits of these mathematical constants or calculate them, a few of them are provided directly by CSS for convenience.

Syntax

The `<calc-keyword>` type defines numeric constants that can be used in CSS math functions.

Values

`e`

The base of the natural logarithm, approximately equal to `2.7182818284590452354`.

`pi`

The ratio of a circle's circumference to its diameter, approximately equal to `3.1415926535897932`.

`infinity` & `-infinity`

An infinite value, used to indicate the largest/smallest possible value.

`NaN`

A value representing "Not a Number" canonical casing.

Notes

Serializing the arguments inside `calc()` follows the IEEE-754 standard for floating point math which means there's a few cases to be aware of regarding constants like `infinity` and `NaN`:

- Dividing by zero will return positive or negative `infinity` depending on the sign of the numerator.
- Adding, subtracting or multiplying `infinity` to anything will return `infinity` unless it produces `NaN` (see below).
- Any operation with at least one `NaN` argument will return `NaN`. This means `0 / 0`, `infinity / infinity`, `0 * infinity`, `infinity + (-infinity)`, and `infinity - infinity` will all return `NaN`.
- Positive and negative zero are possible values (`0`⁺ and `0`⁻). This has the following effects:
  - Multiplication or division that produces zero with exactly one negative argument (`-5 * 0` or `1 / (-infinity)`) or negative result from combinations in the other math functions will return `0`⁻.
  - `0`⁻ `+ 0`⁻ or `0`⁻ `- 0` will return `0`⁻. All other additions or subtractions that would produce a zero will return `0`⁺.
  - Multiplying or dividing `0`⁻ with a positive number (including `0`⁺) will return a negative result (either `0`⁻ or `-infinity`), while multiplying or dividing `0`⁻ with a negative number will return a positive result.

Examples of how these rules apply are shown in the Infinity, NaN, and division by zero section.

> **Note:** It's rare to need to use `infinity` as an argument in `calc()`, but it can be used to avoid hardcoded "magic numbers" or making sure a certain value is always larger than another value. It may be useful if you need to make it obvious that a property has "the largest possible value" for that data type.

Formal syntax

```
<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Description

Mathematical constants can only be used inside CSS math functions for calculations. Math constants are not CSS keywords, but if they are used outside of a calculation, they're treated like any other keyword. For example:

- `animation-name: pi;` refers to an animation named "pi", not the `pi` numeric constant.
- `line-height: e;` is invalid, but `line-height: calc(e);` is valid.
- `rotate(1rad * pi);` won't work because `rotate()` is not a math function. Use `rotate(calc(1rad * pi));`

In math functions, `<calc-keyword>` values are evaluated as `<number>` values, therefore `e` and `pi` act as numeric constants.

Both `infinity` and `NaN` are slightly different, they are considered as degenerate numeric constants. While not technically numbers, they act as `<number>` values, so to get an infinite `<length>`, for example, requires an expression like `calc(infinity * 1px)`.

The `infinity` and `NaN` values are included mostly to make serialization simpler and more obvious, but can be used to indicate a "largest possible value", since an infinite value is clamped to the allowed range. It's rare for this to be reasonable, but when using infinity it's much simpler than just putting an enormous number in a stylesheet or hardcoding magic numbers.

All constants are case-insensitive except for `NaN`, which makes `calc(Pi)`, `calc(E)` and `calc(InFiNiTy)` valid:

```
e
-e
E
pi
-pi
Pi
infinity
-infinity
InFiNiTy
NaN
```

The following are all invalid:

```
nan
Nan
NAN
```

Examples

Using e and pi in <code>calc()</code>

The following example shows how to use `e` inside `calc()` to rotate an element with an exponentially-increasing angle. The second box shows how to use `pi` inside a `sin()` function.

```
#wrapper {
  display: flex;
  flex-direction: row;
  justify-content: space-evenly;
}

.container {
  display: flex;
  flex-direction: column;
  align-items: start;
  width: 200px;
}
.container > div {
  width: 100px;
  height: 100px;
  margin: 10px;
}

span {
  font-family: monospace;
  font-size: 0.8em;
}

#e {
  background-color: blue;
}

#pi {
  background-color: blue;
}
```

```
<div id="wrapper">
  <div class="container">
    <div id="e"></div>
    <input type="range" min="0" max="7" step="0.01" value="0" id="e-slider" />
    <label for="e-slider">e:</label>
    <span id="e-value"></span>
  </div>
  <div class="container">
    <div id="pi"></div>
    <input type="range" min="0" max="1" step="0.01" value="0" id="pi-slider" />
    <label for="pi-slider">pi:</label>
    <span id="pi-value"></span>
  </div>
</div>
```

```
// sliders
const eInput = document.querySelector("#e-slider");
const piInput = document.querySelector("#pi-slider");
// spans for displaying values
const eValue = document.querySelector("#e-value");
const piValue = document.querySelector("#pi-value");

eInput.addEventListener("input", function () {
  e.style.transform = `rotate(calc(1deg * pow(${this.value}, e)))`;
  eValue.textContent = e.style.transform;
});

piInput.addEventListener("input", function () {
  pi.style.rotate = `calc(sin(${this.value} * pi) * 100deg)`;
  piValue.textContent = pi.style.rotate;
});
```

Infinity, NaN, and division by zero

The following example shows the computed value of the `width` property when dividing by zero, followed by how serialization with different `calc()` constants look when viewed in the console:

```
<div></div>
```

```
div {
  height: 50px;
```

```
  background-color: red;
  width: calc(1px / 0);
}
```

```
const div = document.querySelector("div");
console.log(div.offsetWidth); // 17895698 (infinity clamped to largest value for width)

function logSerializedWidth(value) {
  div.style.width = value;
  console.log(div.style.width);
}

logSerializedWidth("calc(1px / 0)"); // calc(infinity * 1px)
logSerializedWidth("calc(1px / -0)"); // calc(-infinity * 1px)

logSerializedWidth("calc(1px * -infinity * -infinity)"); // calc(infinity * 1px)
logSerializedWidth("calc(1px * -infinity * infinity)"); // calc(-infinity * 1px)

logSerializedWidth("calc(1px * (NaN + 1))"); // calc(NaN * 1px)
```

See also

- `<calc-sum>`
- `<calc-product>`
- `<calc-value>`

calc-size() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`calc-size()`** CSS function allows you to perform calculations on intrinsic size values such as `auto`, `fit-content`, and `max-content`; this is not supported by the regular `calc()` function.

`calc-size()` return values can also be interpolated, enabling size keyword values to be used in animations and transitions. In effect, including `calc-size()` in a property value automatically applies `interpolate-size: allow-keywords` to the selection.

Note however that `interpolate-size` is inherited, therefore applying it to an element enables interpolation of intrinsic size keywords for every property applied to that element and its children. As a result, `interpolate-size` is the preferred solution for enabling intrinsic size animations. You should only use `calc-size()` to enable intrinsic size animations if they also require calculations.

Syntax

```
/* Pass a value through calc-size() */
calc-size(auto, size)
calc-size(fit-content, size)

/* Perform a calculation */
calc-size(min-content, size + 100px)
calc-size(fit-content, size / 2)

/* Calculation including a function */
calc-size(auto, round(up, size, 50px))
```

Parameters

The `calc-size()` function's syntax is as follows:

```
calc-size(<calc-size-basis>, <calc-sum>)
```

The parameters are:

`<calc-size-basis>`

The value (most commonly an intrinsic size) that you want to perform a calculation on.

`<calc-sum>`

An expression that defines the calculation to be performed on the `<calc-size-basis>`.

Return value

Returns a value equal to the `<calc-size-basis>` modified by the `<calc-sum>` expression. As the `<calc-size-basis>` value is an intrinsic size value, the return value is a modified intrinsic size value that behaves like the intrinsic size value input into the function.

Description

Certain browser layout algorithms have special behaviors for intrinsic sizing keywords. The `calc-size()` function is explicitly defined to represent an intrinsic size rather than a `<length-percentage>`, thereby enforcing correctness. `calc-size()` enables calculations to be performed on intrinsic size values in a safe, well-defined manner.

Valid values for the first argument (<code><calc-size-basis></code>)

The first `calc-size()` argument can be one of the following intrinsic values:

- `auto`
- `min-content`
- `max-content`
- `fit-content`
- `content` (for containers sized using `flex-basis`).

There are also a few special values that this argument can take:

- A nested `calc-size()` value. This isn't something you'd be likely to do very often, but it is available ensuring using a CSS variable as the `<calc-size-basis>` will always work, provided the variable is a valid value for the property `calc-size()` is being set on. So for example, this will work:

  ```
  section {
    height: calc-size(calc-size(max-content, size), size + 2rem);
  }
  ```

As will this:

```
:root {
  --intrinsic-size: calc-size(max-content, size);
}

section {
  height: calc-size(var(--intrinsic-size), size + 2rem);
}
```

- Another `<calc-sum>`, with the same restrictions as the `<calc-sum>` specified for the second argument, except that the `size` keyword cannot be included. You likely will not do this, as you are no longer doing a calculation on an intrinsic size value, but if a custom property value is a `<calc-sum>`, the function will still work. For example, this will work directly or if you use a custom property with a value of `300px + 2rem`:

  ```
  section {
    height: calc-size(300px + 2rem, size / 2);
  }
  ```

- The keyword `any`, which represents an unspecified definite size. In this case, the `size` keyword cannot be included in the second argument, and the `calc-size()` returns the result of the second argument calculation. For example:

  ```
  section {
    height: calc-size(any, 300px * 1.5); /* Returns 450px */
  }
  ```

Mixing different intrinsic sizes together in the same calculation doesn't work. For example, `max-content - min-content` doesn't make sense. `calc-size()` only allows a single intrinsic size value in each calculation, avoiding this problem.

Valid values for the second argument (<code><calc-sum></code>)

The second `calc-size()` argument is a `<calc-sum>` expression.

In this expression:

- The keyword `size` represents the `<calc-size-basis>` specified as the first argument.
- Operands can include `size`, and any value types that make sense in the context.
- The `+`, `-`, `*`, and `/` operators can be included.
- Other mathematical functions can be included such as `round()`, `max()`, or even a nested `calc-size()`.
- The overall expression must match `<length-percentage>`, and resolve to a `<length>`.

Enabling animation of intrinsic size values

`calc-size()` return values can be interpolated, enabling animations between a `<length-percentage>` value and a `calc-size()` intrinsic size return value.

> **Note:** You should avoid animating box model properties if possible, to cut down on layout events and mitigate the resulting impact on performance (see Critical rendering path > Layout).

For example, you could use a transition to animate a container `width` between `0` and `auto` like so:

```
section {
  width: 0;
  transition: width ease 1s;
}

section:hover,
section:focus {
  width: calc-size(auto, size);
}
```

In the above case, we are not calculating anything — we are putting `auto` into `calc-size()` and returning it unchanged. The `interpolate-size` property makes animations like the above simpler to implement in most cases, especially when there are multiple animations to consider. It is inherited and therefore only needs to be declared once on an ancestor property, meaning we could have transitioned between `0` and `auto` without using `calc-size()`.

The `calc-size()` function should only be used to enable intrinsic size animations if they also require calculations. For example, in the following case we are animating the `width` *and* applying a calculation to the intrinsic size end state:

```
section {
  width: 0;
  transition: width ease 1s;
}

section:hover,
section:focus {
  width: calc-size(auto, size + 2rem);
}
```

One case in which `calc-size()` is useful is when you want to animate between an intrinsic size and a modified version of the same intrinsic size. This is not possible with `interpolate-size` and `calc()`. For example, the following `@keyframes` definition animates a container `width` between `fit-content` and 70% of the `fit-content`.

```
@keyframes narrower {
  from {
    width: fit-content;
  }

  to {
    width: calc-size(fit-content, size * 0.7);
  }
}
```

> **Note:** Note that `calc-size()` does not enable animating between two different intrinsic size values.

Formal syntax

```
<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>calc-size</code> usage

This example shows basic dimension sizing of a container using `calc-size()`

HTML

The HTML contains a single `<section>` element that contains some child content.

```
<section>
  <h2>Favorite quote</h2>

  <p>
    Fashion is something so ugly it has to be changed every fifteen minutes.
  </p>
</section>
```

CSS

```
* {
  box-sizing: border-box;
}

section {
  font-family: "Helvetica", "Arial", sans-serif;
  border: 1px solid black;
}

h2 {
  margin: 0;
  font-weight: normal;
  font-size: 1.1rem;
  text-align: center;
  letter-spacing: 1px;
}

p {
  font-size: 0.8rem;
}
```

In the CSS, we use flexbox to center the child elements inside the `<section>`, and set the `width` and `height` of the `<section>` to `calc-size()` functions. The `width` is set equal to `fit-content` plus `6rem`. The `height` is set to `auto` multiplied by two.

```
section {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;

  width: calc-size(fit-content, size + 6rem);
  height: calc-size(auto, size * 2);
}
```

The rest of the CSS has been hidden for brevity.

Result

We've created some horizontal and vertical space for the text to be centered in, without the use of padding.

Basic <code>calc-size</code> animations

This example demonstrates how to use `calc-size()` to animate between a specific size and an intrinsic size. The demo features a character badge/"name tag", which can be hovered or focused to reveal information about the character. The reveal is handled by a `height` transition between a set length and `max-content`.

HTML

The HTML contains a single `<section>` element with `tabindex="0"` set on it so it can receive keyboard focus. The `<section>` contains `<header>` and `<main>` elements, each with their own child content.

```
<section tabindex="0">
  <header>
    <h2>Chris Mills</h2>
  </header>
  <main>
    <p>Chris is the silent phantom of MDN.</p>
    <ul>
      <li><strong>Height</strong>: 3.03m</li>
      <li><strong>Weight</strong>: 160kg</li>
      <li><strong>Tech Fu</strong>: 7</li>
      <li><strong>Bad Jokes</strong>: 9</li>
    </ul>
  </main>
</section>
```

CSS

```
* {
  box-sizing: border-box;
}

section {
  font-family: "Helvetica", "Arial", sans-serif;
  width: 175px;
  border-radius: 5px;
  background: #eeeeee;
  box-shadow:
    inset 1px 1px 4px rgb(255 255 255 / 0.5),
    inset -1px -1px 4px rgb(0 0 0 / 0.5);
}

header {
  padding: 10px;
  border-bottom: 2px solid #cccccc;
}

main {
  padding: 0.7rem;
}

h2 {
  margin: 0;
  font-weight: normal;
  font-size: 1.1rem;
  text-align: center;
  letter-spacing: 1px;
}

p,
li {
  font-size: 0.8rem;
  line-height: 1.5;
}

p {
  margin-top: 0;
}
```

In the CSS, we set the `<section>`'s `height` to `2.5rem` and `overflow` to `hidden` so only the `<header>` is shown by default, then specify a `transition` that animates the `<section>` `height` over 1 second during state changes. Finally, we set the `<section>` `height` to a `calc-size()` function call on `:hover` and `:focus`. The function return value is the equivalent of `max-content + 2rem`.

```
section {
  height: 2.5rem;
  overflow: hidden;
  transition: height ease 1s;
}

section:hover,
section:focus {
  height: calc-size(max-content, size + 2rem);
}
```

The rest of the CSS has been hidden for brevity.

Result

Try hovering over the `<section>` or focusing it via the keyboard — it will animate to its full height + 2rem, revealing all the content with 2rems of extra space at the bottom.

Adjusting reading width based on <code>fit-content</code>

This example shows a container with text inside it, and a button that can be clicked to make the container width narrower or wider depending on reading preference.

HTML

The HTML contains a single `<section>` element containing child text content, plus a `<button>` to change the `<section>` width.

```
<section class="easy-reader">
  <h2>Easy reader</h2>

  <p>
    Eius velit aperiam ipsa. Deleniti eum excepturi ut magni maxime maxime
    beatae. Dicta aperiam est laudantium ut illum facere qui officiis. Sunt
    deleniti quam id. Quis sunt voluptatem praesentium minima dolorum autem
    consequatur velit.
  </p>

  <p>
    Vitae ab incidunt velit aspernatur deleniti distinctio rerum. Et natus sed
    et quos mollitia quia quod. Quae officia ex ea. Ducimus ut voluptatem et et
    debitis. Quidem provident laboriosam exercitationem similique deleniti.
    Temporibus vel veniam mollitia magni unde a nostrum.
```

```
  </p>

  <button class="width-adjust">Narrower</button>
</section>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  width: 600px;
  margin: 0 auto;
}

section {
  margin-top: 20px;
  font-family: "Helvetica", "Arial", sans-serif;
  background: #eeeeee;
  border: 2px solid #cccccc;
  padding: 0 20px;
  position: relative;
}

p,
li {
  font-size: 0.8rem;
  line-height: 1.5;
}

button {
  position: absolute;
  top: 2px;
  right: 2px;
}
```

In the CSS, we set the `<section>`'s `width` to a default of `fit-content`. We then define two sets of `@keyframes`, `narrower`, which animates from `fit-content` to 70% of `fit-content` (calculated using `calc-size()`), and `wider`, which animates the same values but in the opposite direction. Finally, we attach those animations to two classes — `.narrower` and `.wider`. Each animation is defined to last one second and to keep the final state applied once finished.

```
section {
  width: fit-content;
}

@keyframes narrower {
  from {
    width: fit-content;
  }

  to {
    width: calc-size(fit-content, size * 0.7);
  }
}

@keyframes wider {
  from {
    width: calc-size(fit-content, size * 0.7);
  }

  to {
    width: fit-content;
  }
}

.narrower {
  animation: narrower 1s ease forwards;
}

.wider {
  animation: wider 1s ease forwards;
}
```

The rest of the CSS has been hidden for brevity.

JavaScript

The JavaScript provides a narrower/wider toggle that applies the relevant class to the `<section>` when the button is clicked:

```
const widthAdjustBtn = document.querySelector(".width-adjust");
const easyReader = document.querySelector(".easy-reader");

widthAdjustBtn.addEventListener("click", () => {
  if (easyReader.classList.length === 1) {
    easyReader.classList.add("narrower");
    widthAdjustBtn.textContent = "Wider";
  } else if (easyReader.classList.contains("wider")) {
    easyReader.classList.replace("wider", "narrower");
    widthAdjustBtn.textContent = "Wider";
  } else if (easyReader.classList.contains("narrower")) {
    easyReader.classList.replace("narrower", "wider");
    widthAdjustBtn.textContent = "Narrower";
  }
});
```

Result

Try clicking the `<button>` a few times to adjust the `<section>` between the wide and narrow reading width, achieved by manipulating the `width` based on the `fit-content` value.

Using a function inside the <code>calc-size()</code> function

As mentioned earlier, it is possible to use another function inside `calc-size()`. This example sets `field-sizing: content` on `<input>` elements to make them as wide as the entered content, and then uses a `max()` function inside `calc-size()` to ensure that the `<input>`s are at least a minimum size, and only start to grow when the entered text becomes wider than that size — by being set to `fit-content` plus `20px`.

HTML

The HTML contains a `<form>` element containing three textual `<input>` types. Each `<input>` has a `<label>` associated with it to make the form accessible, and a `maxlength` applied to stop entered values getting long enough to break the form layout.

```
<form>
  <div>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" maxlength="48" />
  </div>
  <div>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" maxlength="48" />
  </div>
  <div>
    <label for="address">Address:</label>
    <input type="text" id="address" name="address" maxlength="60" />
  </div>
</form>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  width: 600px;
  margin: 0 auto;
}

form {
  margin-top: 20px;
  padding: 20px;
  font-family: "Helvetica", "Arial", sans-serif;
  background: #eeeeee;
  border: 2px solid #cccccc;
}

div {
  display: flex;
  align-items: center;
}

div:not(div:last-child) {
  margin-bottom: 20px;
}
```

In the CSS, we set the `width` of the `<label>` elements to `100px`. We set `field-sizing: content` on the `<input>` elements to make them as wide as the entered content — by default they would no width because nothing would be entered into them. To counteract this, we set their `width` values to `calc-size(fit-content, max(100px, size + 20px))`. This means that they are a minimum of `100px` wide, even with no value entered. When an entered value becomes wider than `100px`, their `width` changes to `fit-content` plus `20px`, which means they start to grow with the content size but keep a `20px` gap on the right-hand side.

```
label {
  width: 100px;
}

input {
  field-sizing: content;
  width: calc-size(fit-content, max(100px, size + 20px));
}
```

The rest of the CSS has been hidden for brevity.

Result

Try entering some text inside the form inputs, and see how they grow when the values start to become as wide as the minimum width enforced by the `max()` function.

See also

- `interpolate-size`
- `calc()`
- `round()`
- Animate to height: auto; (and other intrinsic sizing keywords) in CSS on developer.chrome.com (2024)

<calc-sum> CSS type

The **`<calc-sum>`** CSS data type represents an expression which performs a calculation in any CSS math function. The expression executes a basic arithmetic operation of addition and subtraction between two values.

Syntax

The `<calc-sum>` type defines two numeric values and one of the following arithmetic operators between them.

`+`

Adds two numbers together.

`-`

Subtracts the right number from the left.

Formal syntax

```
<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*
```

```
<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Description

The operands in the expression may be any `<length>` syntax value. You can use `<length>`, `<frequency>`, `<angle>`, `<time>`, `<percentage>`, `<number>`, or `<integer>`.

The two operands' types must be consistent. For lengths, you can't use `0` to mean `0px` (or another length unit). Instead, you must add an explicit unit: `margin-top: calc(0px + 20px);` is valid, while `margin-top: calc(0 + 20px);` is invalid. Percentage value types are resolved based on the context. For example, `margin-top: calc(50% + 20px);` is valid because `margin-top` resolves percentages to lengths.

Including CSS variables in `calc-sum` expressions is also allowed. The following code `calc(10px + var(--variable))`, is a valid expression.

The `+` and `-` operators **must be surrounded by whitespace**. For instance, `calc(50% -8px)` will be parsed as "a percentage followed by a negative length" — which is an invalid expression — while `calc(50% - 8px)` is "a percentage followed by a subtraction operator and a length". Likewise, `calc(8px + -50%)` is treated as "a length followed by an addition operator and a negative percentage".

See also

- `<calc-product>`
- `<calc-value>`
- `<calc-keyword>`

clamp() CSS function

The **`clamp()`** CSS function clamps a value within a range of values between a defined minimum bound and a maximum bound. The function takes three parameters: a minimum value, a preferred value, and a maximum allowed value.

Try it

```
font-size: clamp(1rem, 2.5vw, 2rem);
```

```
font-size: clamp(1.5rem, 2.5vw, 4rem);
```

```
font-size: clamp(1rem, 10vw, 2rem);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    The font-size of this text varies depending on the base font of the page,
    and the size of the viewport.
  </div>
</section>
```

Note that using `clamp()` for font sizes, as in these examples, allows you to set a font-size that grows with the size of the viewport, but doesn't go below a minimum font-size or above a maximum font-size. It has the same effect as the code in Fluid Typography but in one line, and without the use of media queries.

Syntax

```
/* Static values */
width: clamp(200px, 40%, 400px);
width: clamp(20rem, 30vw, 70rem);
width: clamp(10vw, 20em, 100vw);

/* Calculated values */
width: clamp(min(10vw, 20rem), 300px, max(90vw, 55rem));
width: clamp(100px, calc(30% / 2rem + 10px), 900px);
```

Parameters

The `clamp(min, val, max)` function accepts three comma-separated expressions as its parameters.

`min`

The minimum value is the smallest (most negative) value. This is the lower bound in the range of allowed values. If the preferred value is less than this value, the minimum value will be used.

`val`

The preferred value is the expression whose value will be used as long as the result is between the minimum and maximum values.

`max`

The maximum value is the largest (most positive) expression value to which the value of the property will be assigned if the preferred value is greater than this upper bound.

The expressions can be math functions (see `calc()` for more information), literal values, other expressions that evaluate to a valid argument type (like `<length>`), or nested `min()` and `max()` functions. For math expressions, you can use addition, subtraction, multiplication, and division without using the `calc()` function itself. You may also use parentheses to establish computation order when needed.

You can use different units for each value in your expressions and different units in any math function making up any of the arguments.

Keep the following aspects in mind while working with the function:

- Math expressions involving percentages for widths and heights on table columns, table column groups, table rows, table row groups, and table cells in both auto and fixed layout tables *may* be treated as if `auto` had been specified.
- It is permitted to nest `max()` and `min()` functions as expression values, in which case the inner ones are treated as basic parentheses. The expressions are full math expressions, so you can use direct addition, subtraction, multiplication and division without using the `calc()` function itself.

- The expression can be values combining the addition (+), subtraction (-), multiplication (*), and division (/) operators, using standard operator precedence rules. Make sure to put a space on each side of the + and - operands. You can use different units for each value in your expression. You may also use parentheses to establish computation order when needed.
- Oftentimes you will want to use `min()` and `max()` within a `clamp()` function.

Return value

`clamp(MIN, VAL, MAX)` is resolved as `max(MIN, min(VAL, MAX))`.

Based on the provided parameters, the function returns `<length>`, `<frequency>`, `<angle>`, `<time>`, `<percentage>`, `<number>`, or `<integer>`.

Formal syntax

```
<clamp()> =
  clamp( [ <calc-sum> | none ] , <calc-sum> , [ <calc-sum> | none ] )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

min(), max(), and clamp() comparison

In this example we have a web page that uses `min()`, `max()`, and `clamp()` to set sizes responsively.

The example adjusts the sizes of page elements in three ways:

- the lengths of the lines of text
- the font size of paragraph text
- the font size of heading text

In all three cases, the page uses a combination of a viewport-relative units (`vw` and `<percentage>`), to set a size that varies with the viewport width, and a value that is not viewport relative (`rem` and `px`) to implement minimum and/or maximum sizes.

The example is at https://mdn.github.io/css-examples/min-max-clamp/. Open it in a new window and try adjusting the window width.

The **line length** (controlled by the `width` of the `<body>` element) will increase as the window width increases, but only up to a certain point (`1000px`), and beyond that point, it won't increase anymore. We're using `min()` to set a **maximum line length**: it can go under `1000px`, but won't go over. This is helpful because long lines are harder to read, so we often want to limit how long a line can be. To achieve this we use `min(1000px, calc(70% + 100px))`: when the result of the percentage-based calculation goes above `1000px`, we switch to the fixed `1000px` value.

The **size of the paragraph text**, controlled by the `font-size` of the `<p>` element, decreases as the window gets narrower, but only up to a certain point, and beyond that point (the point where `1.2vw` is less than `1.2rem`) it doesn't get any smaller: it stays at `1.2rem`. We're using `max()` to set a **minimum font size**: the font can grow above `1.2rem` but will never go below it. This is helpful because really small text is hard to read. To achieve this we use `max(1.2rem, 1.2vw)`. This means that the `font-size` will be set at `1.2rem`, unless the computed value of `1.2vw` is greater than that of `1.2rem`, in which case it will be set to `1.2vw` instead.

The **size of the heading text**, controlled by the `font-size` of the `<h1>` element, has a viewport-relative value with both a maximum and a minimum threshold. To achieve this we use `clamp(1.8rem, 2.5vw, 2.8rem)`. The viewport-relative value is `2.5vw` but it is clamped between `1.8rem` and `2.8rem`, so:

- if the calculated value of `2.5vw` is less than `1.8rem`, then `1.8rem` will be used
- if the calculated value of `2.5vw` is greater than `2.8rem`, then `2.8rem` will be used.

This prevents the heading text from getting too small in a very narrow window, or too big in a very wide window.

HTML

```
<h1>Basic responsive test</h1>
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. In orci orci,
  eleifend id risus nec, mattis rutrum velit. Suspendisse fringilla egestas erat
  eu convallis. Phasellus eu velit ut magna dapibus elementum cursus at ligula.
  Ut tempus varius nibh, nec auctor sapien iaculis sit amet. Fusce iaculis,
  libero quis elementum viverra, nulla ante accumsan lectus, sit amet convallis
  lacus ipsum vel est. Curabitur et urna non est consectetur pulvinar vel id
  risus. Ut vestibulum, sem in semper aliquet, felis arcu euismod sapien, ac
  imperdiet massa nisl quis sem. Vestibulum ac elementum felis, in tempor velit.
  Pellentesque purus ex, mattis at ornare quis, porta condimentum mi. Donec
  vestibulum ligula vel nulla blandit, quis euismod nulla vestibulum.
  Suspendisse potenti. Nunc neque mauris, tempor sed facilisis at, ultrices eget
  nulla. Pellentesque convallis ante nec augue porttitor, id tempus ante luctus.
</p>

<p>
  Integer rutrum sollicitudin tellus, quis cursus nulla scelerisque nec. Nunc eu
  facilisis lorem. Maecenas faucibus sapien eleifend, semper tellus at, pharetra
  quam. Cras feugiat vulputate tortor at rhoncus. Class aptent taciti sociosqu
  ad litora torquent per conubia nostra, per inceptos himenaeos. Nam non felis
  quis sem lobortis sodales vel id libero. Phasellus sit amet placerat lorem.
</p>
```

CSS

```
html {
  font-family: sans-serif;
}
```

```
body {
  margin: 0 auto;
  width: min(1000px, calc(70% + 100px));
}

h1 {
  letter-spacing: 2px;
  font-size: clamp(1.8rem, 2.5vw, 2.8rem);
}

p {
  line-height: 1.5;
  font-size: max(1.2rem, 1.2vw);
}
```

Accessibility

When `clamp()` is used for controlling text size, make sure that the maximum allowed value is a relative length unit that is no less than twice the minimum allowed value, for example:

```
font-size: clamp(1rem, 2.5vw, 2rem);
```

This helps to ensure that text size can scale to at least 200% if the page is zoomed.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4: Resize Text | WAI | W3C

See also

- `calc()`
- `max()`
- `min()`
- Learn: CSS Values and units

<color> CSS type

The **`<color>`** CSS data type represents a color. A `<color>` may also include an alpha-channel *transparency value*, indicating how the color should composite with its background.

> **Note:** Although `<color>` values are precisely defined, their actual appearance may vary (sometimes significantly) from device to device. This is because most devices are not calibrated, and some browsers do not support output devices' color profiles.

Syntax

```
/* Named colors */
rebeccapurple
aliceblue

/* RGB Hexadecimal */
#f09
#ff0099

/* RGB (Red, Green, Blue) */
rgb(255 0 153)
rgb(255 0 153 / 80%)

/* HSL (Hue, Saturation, Lightness) */
hsl(150 30% 60%)
hsl(150 30% 60% / 80%)

/* HWB (Hue, Whiteness, Blackness) */
hwb(12 50% 0%)
hwb(194 0% 0% / 0.5)

/* Lab (Lightness, A-axis, B-axis) */
lab(50% 40 59.5)
lab(50% 40 59.5 / 0.5)

/* LCH (Lightness, Chroma, Hue) */
lch(52.2% 72.2 50)
lch(52.2% 72.2 50 / 0.5)

/* Oklab (Lightness, A-axis, B-axis) */
oklab(59% 0.1 0.1)
oklab(59% 0.1 0.1 / 0.5)

/* OkLCh (Lightness, Chroma, Hue) */
oklch(60% 0.15 50)
oklch(60% 0.15 50 / 0.5)

/* Relative CSS colors */
/* HSL hue change */
hsl(from red 240deg s l)
/* HWB alpha channel change */
hwb(from green h w b / 0.5)
/* LCH lightness change */
lch(from blue calc(l + 20) c h)

/* light-dark */
light-dark(white, black)
light-dark(rgb(255 255 255), rgb(0 0 0))
```

A `<color>` value can be specified using one of the methods listed below:

- By keywords: `<named-color>` (such as `blue` or `pink`), `<system-color>`, and `currentColor`.
- By hexadecimal notations: `<hex-color>` (such as `#ff0000`).
- By `<color-function>`, with parameters in a color space using functional notations:
  - sRGB color space: `hsl()`, `hwb()`, and `rgb()`.
  - CIELAB color space: `lab()` and `lch()`.

    - Oklab color space: `oklab()` and `oklch()`.
    - Other color spaces: `color()`, `device-cmyk()`.
- By using relative color syntax to output a new color based on an existing color. Any of the above color functions can take an **origin color** preceded by the `from` keyword and followed by definitions of the channel values for the new **output color**.
- By mixing two colors: `color-mix()`.
- By specifying a color that you want a contrasting color returned for: `contrast-color()`.
- By specifying two colors, using the first for light color-schemes and the second for dark color-schemes: `light-dark()`.

<code>currentColor</code> keyword

The `currentColor` keyword represents the value of an element's `color` property. This lets you use the `color` value on properties that do not receive it by default.

If `currentColor` is used as the value of the `color` property, it instead takes its value from the inherited value of the `color` property.

```
<div class="container">
  The color of this text is blue.
  <div class="child"></div>
  This block is surrounded by a blue border.
</div>
```

```
.container {
  color: blue;
  border: 1px dashed currentColor;
}
.child {
  background: currentColor;
  height: 9px;
}
```

Missing color components

Each component of any CSS color functions - except for those using the legacy comma-separated syntax - can be specified as the keyword `none` to be a missing component.

Explicitly specifying missing components in color interpolation is useful for cases where you would like to interpolate some color components but not others. For all other purposes, a missing component will effectively have a zero value in an appropriate unit: `0`, `0%`, or `0deg`. For example, the following colors are equivalent when used outside of interpolation:

```
/* These are equivalent */
color: oklab(50% none -0.25);
color: oklab(50% 0 -0.25);

/* These are equivalent */
background-color: hsl(none 100% 50%);
background-color: hsl(0deg 100% 50%);
```

Interpolation

Color interpolation happens with gradients, transitions, and animations.

When interpolating `<color>` values, they are first converted to a given color space, and then each component of the computed values are interpolated linearly, with interpolation's speed being determined by the easing function in transitions and animations. The interpolation color space defaults to Oklab, but can be overridden through `<color-interpolation-method>` in some color-related functional notations.

Interpolation with missing components

Interpolating colors in the same space

When interpolating colors that are exactly in the interpolation color space, missing components from one color are replaced with existing values of the same components from the other color. For example, the following two expressions are equivalent:

```
color-mix(in oklch, oklch(none 0.2 10), oklch(60% none 30))
color-mix(in oklch, oklch(60% 0.2 10), oklch(60% 0.2 30))
```

> **Note:** If a component is missing from both colors, this component will be missing after the interpolation.

Interpolating colors from different spaces: analogous components

If any color to be interpolated is not in the interpolation color space, its missing components are transferred into the converted color based on **analogous components** of the same category as described in the following table:

Category	Analogous components
Reds	`R`, `X`
Greens	`G`, `Y`
Blues	`B`, `Z`
Lightness	`L`
Colorfulness	`C`, `S`
Hue	`H`
a	`a`
b	`b`

For example:

- `X` (`0.2`) in `color(xyz 0.2 0.1 0.6)` is analogous to `R` (`50%`) in `rgb(50% 70% 30%)`.
- `H` (`0deg`) in `hsl(0deg 100% 80%)` is analogous to `H` (`140`) in `oklch(80% 0.1 140)`.

Using OkLCh as the interpolation color space and the two colors below as an example:

```
lch(80% 30 none)
color(display-p3 0.7 0.5 none)
```

The preprocessing procedure is:

1. Replace the missing components in both colors with a zero value:

```
lch(80% 30 0)
color(display-p3 0.7 0.5 0)
```

2. Convert both colors into the interpolation color space:

```
oklch(83.915% 0.0902 0.28)
oklch(63.612% 0.1522 78.748)
```

3. If any component of the converted colors is analogous to a missing component in the corresponding original color, reset it as a missing component:

```
oklch(83.915% 0.0902 none)
oklch(63.612% 0.1522 78.748)
```

4. Replace any missing component with the same component from the other converted color:

```
oklch(83.915% 0.0902 78.748)
oklch(63.612% 0.1522 78.748)
```

Accessibility

Some people have difficulty distinguishing colors. The WCAG 2.2 recommendation strongly advises against using color as the only means of conveying a specific message, action, or result. See color and color contrast for more information.

Formal syntax

```
<color> =
  <color-base>        |
  currentColor        |
  <system-color>      |
  <contrast-color()>  |
  <device-cmyk()>     |
  <light-dark-color>

<color-base> =
  <hex-color>       |
  <color-function>  |
  <named-color>     |
  <color-mix()>     |
  transparent

<system-color> =
  AccentColor         |
  AccentColorText     |
  ActiveText          |
  ButtonBorder        |
  ButtonFace          |
  ButtonText          |
  Canvas              |
  CanvasText          |
  Field               |
  FieldText           |
  GrayText            |
  Highlight           |
  HighlightText       |
  LinkText            |
  Mark                |
  MarkText            |
  SelectedItem        |
  SelectedItemText    |
  VisitedText         |
  <deprecated-color>

<contrast-color()> =
  contrast-color( [ [ <color> && [ tbd-fg | tbd-bg ] && <target-contrast>? ] | [ <color> && [ tbd-fg | tbd-bg ] && <target-contrast> , <color># ] ] )

<device-cmyk()> =
  <legacy-device-cmyk-syntax>  |
  <modern-device-cmyk-syntax>

<light-dark-color> =
  light-dark( <color> , <color> )

<color-function> =
  <rgb()>        |
  <rgba()>       |
  <hsl()>        |
  <hsla()>       |
  <hwb()>        |
  <lab()>        |
  <lch()>        |
  <oklab()>      |
  <oklch()>      |
  <ictcp()>      |
  <jzazbz()>     |
  <jzczhz()>     |
  <alpha()>      |
  <color()>      |
  <hdr-color()>

<named-color> =
  aliceblue             |
  antiquewhite          |
  aqua                  |
  aquamarine            |
  azure                 |
  beige                 |
  bisque                |
  black                 |
  blanchedalmond        |
  blue                  |
  blueviolet            |
  brown                 |
```

burlywood
cadetblue
chartreuse
chocolate
coral
cornflowerblue
cornsilk
crimson
cyan
darkblue
darkcyan
darkgoldenrod
darkgray
darkgreen
darkgrey
darkkhaki
darkmagenta
darkolivegreen
darkorange
darkorchid
darkred
darksalmon
darkseagreen
darkslateblue
darkslategray
darkslategrey
darkturquoise
darkviolet
deeppink
deepskyblue
dimgray
dimgrey
dodgerblue
firebrick
floralwhite
forestgreen
fuchsia
gainsboro
ghostwhite
gold
goldenrod
gray
green
greenyellow
grey
honeydew
hotpink
indianred
indigo
ivory
khaki
lavender
lavenderblush
lawngreen
lemonchiffon
lightblue
lightcoral
lightcyan
lightgoldenrodyellow
lightgray
lightgreen
lightgrey
lightpink
lightsalmon
lightseagreen
lightskyblue
lightslategray
lightslategrey
lightsteelblue
lightyellow
lime
limegreen
linen
magenta
maroon
mediumaquamarine
mediumblue
mediumorchid
mediumpurple
mediumseagreen
mediumslateblue
mediumspringgreen
mediumturquoise
mediumvioletred
midnightblue
mintcream
mistyrose
moccasin
navajowhite
navy
oldlace
olive
olivedrab
orange
orangered
orchid
palegoldenrod
palegreen
paleturquoise
palevioletred
papayawhip
peachpuff
peru
pink
plum
powderblue
purple
rebeccapurple
red

```
  rosybrown             |
  royalblue             |
  saddlebrown           |
  salmon                |
  sandybrown            |
  seagreen              |
  seashell              |
  sienna                |
  silver                |
  skyblue               |
  slateblue             |
  slategray             |
  slategrey             |
  snow                  |
  springgreen           |
  steelblue             |
  tan                   |
  teal                  |
  thistle               |
  tomato                |
  turquoise             |
  violet                |
  wheat                 |
  white                 |
  whitesmoke            |
  yellow                |
  yellowgreen           |
  transparent

<color-mix()> =
  color-mix( <color-interpolation-method>? , [ <color> && <percentage [0,100]>? ]# )

<deprecated-color> =
  ActiveBorder         |
  ActiveCaption        |
  AppWorkspace         |
  Background           |
  ButtonHighlight      |
  ButtonShadow         |
  CaptionText          |
  InactiveBorder       |
  InactiveCaption      |
  InactiveCaptionText  |
  InfoBackground       |
  InfoText             |
  Menu                 |
  MenuText             |
  Scrollbar            |
  ThreeDDarkShadow     |
  ThreeDFace           |
  ThreeDHighlight      |
  ThreeDLightShadow    |
  ThreeDShadow         |
  Window               |
  WindowFrame          |
  WindowText

<target-contrast> =
  <wcag2>

<legacy-device-cmyk-syntax> =
  device-cmyk( <number>#{4} )

<modern-device-cmyk-syntax> =
  device-cmyk( <cmyk-component>{4} [ / [ <alpha-value> | none ] ]? )

<rgb()> =
  <legacy-rgb-syntax>  |
  <modern-rgb-syntax>

<rgba()> =
  <legacy-rgba-syntax>  |
  <modern-rgba-syntax>

<hsl()> =
  <legacy-hsl-syntax>  |
  <modern-hsl-syntax>

<hsla()> =
  <legacy-hsla-syntax>  |
  <modern-hsla-syntax>

<hwb()> =
  hwb( [ from <color> ]? [ <hue> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<lab()> =
  lab( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<lch()> =
  lch( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <hue> | none ] [ / [ <alpha-value> | none ] ]? )

<oklab()> =
  oklab( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<oklch()> =
  oklch( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <hue> | none ] [ / [ <alpha-value> | none ] ]? )

<ictcp()> =
  ictcp( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<jzazbz()> =
  jzazbz( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<jzczhz()> =
  jzczhz( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <hue> | none ] [ / [ <alpha-value> | none ] ]? )

<alpha()> =
  alpha( [ from <color> ] [ / [ <alpha-value> | none ] ]? )
```

```
<color()> =
  color( [ from <color> ]? <colorspace-params> [ / [ <alpha-value> | none ] ]? )

<hdr-color()> =
  color-hdr( [ <color> && <number [0,∞]>? ]#{2} )

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<wcag2> =
  wcag2                                          |
  wcag2( [ <number> | [ aa | aaa ] && large? ] )

<cmyk-component> =
  <number>      |
  <percentage>  |
  none

<alpha-value> =
  <number>      |
  <percentage>

<legacy-rgb-syntax> =
  rgb( <percentage>#{3} , <alpha-value>? )  |
  rgb( <number>#{3} , <alpha-value>? )

<modern-rgb-syntax> =
  rgb( [ from <color> ]? [ <number> | <percentage> | none ]{3} [ / [ <alpha-value> | none ] ]? )

<legacy-rgba-syntax> =
  rgba( <percentage>#{3} , <alpha-value>? )  |
  rgba( <number>#{3} , <alpha-value>? )

<modern-rgba-syntax> =
  rgba( [ from <color> ]? [ <number> | <percentage> | none ]{3} [ / [ <alpha-value> | none ] ]? )

<legacy-hsl-syntax> =
  hsl( <hue> , <percentage> , <percentage> , <alpha-value>? )

<modern-hsl-syntax> =
  hsl( [ from <color> ]? [ <hue> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<legacy-hsla-syntax> =
  hsla( <hue> , <percentage> , <percentage> , <alpha-value>? )

<modern-hsla-syntax> =
  hsla( [ from <color> ]? [ <hue> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<hue> =
  <number>  |
  <angle>

<colorspace-params> =
  <custom-params>          |
  <predefined-rgb-params>  |
  <xyz-params>

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<custom-params> =
  <dashed-ident> [ <number> | <percentage> | none ]+

<predefined-rgb-params> =
  <predefined-rgb> [ <number> | <percentage> | none ]{3}

<xyz-params> =
  <xyz-space> [ <number> | <percentage> | none ]{3}

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<predefined-rgb> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  rec2100-pq         |
  rec2100-hlg        |
  rec2100-linear
```

This syntax reflects the latest standard as per CSS Color HDR Module Level 1, CSS Color Module Level 4, CSS Color Module Level 5, CSS Color Module Level 6. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Exploring color values

In this example, we provide a `<div>` and a text input. Entering a valid color into the input causes the `<div>` to adopt that color, allowing you to test our color values.

HTML

```
<div></div>
<hr />
<label for="color">Enter a valid color value:</label>
<input type="text" id="color" />
```

```
div {
  height: 200px;
  width: 200px;
}
```

```
const inputElem = document.querySelector("input");
const divElem = document.querySelector("div");

function validTextColor(stringToTest) {
  if (stringToTest === "inherit" || stringToTest === "transparent") {
    return false;
  }

  const div = document.createElement("div");
  div.style.color = stringToTest;
  return !!div.style.color;
}

inputElem.addEventListener("input", () => {
  if (validTextColor(inputElem.value)) {
    divElem.style.backgroundColor = inputElem.value;
    divElem.textContent = "";
  } else {
    divElem.removeAttribute("style");
    divElem.textContent = "Invalid color value";
  }
});
```

Result

Generating fully saturated sRGB colors

This example shows fully saturated sRGB colors in the sRGB color space.

HTML

```
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
}
div {
  height: 80px;
  margin: 10px;
  width: 80px;
}
```

```
div:nth-child(1) {
  background-color: hsl(0 100% 50%);
}
div:nth-child(2) {
  background-color: hsl(30 100% 50%);
}
div:nth-child(3) {
  background-color: hsl(60 100% 50%);
}
div:nth-child(4) {
  background-color: hsl(90 100% 50%);
}
div:nth-child(5) {
  background-color: hsl(120 100% 50%);
}
div:nth-child(6) {
  background-color: hsl(150 100% 50%);
}
div:nth-child(7) {
  background-color: hsl(180 100% 50%);
}
div:nth-child(8) {
  background-color: hsl(210 100% 50%);
}
div:nth-child(9) {
  background-color: hsl(240 100% 50%);
}
```

```
div:nth-child(10) {
  background-color: hsl(270 100% 50%);
}
div:nth-child(11) {
  background-color: hsl(300 100% 50%);
}
div:nth-child(12) {
  background-color: hsl(330 100% 50%);
}
```

Result

Creating different shades of red

This example shows reds of different shades in the sRGB color space.

HTML

```
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
}
div {
  box-sizing: border-box;
  height: 80px;
  margin: 10px;
  width: 80px;
}
```

```
div:nth-child(1) {
  background-color: hsl(0 100% 0%);
}
div:nth-child(2) {
  background-color: hsl(0 100% 20%);
}
div:nth-child(3) {
  background-color: hsl(0 100% 40%);
}
div:nth-child(4) {
  background-color: hsl(0 100% 60%);
}
div:nth-child(5) {
  background-color: hsl(0 100% 80%);
}
div:nth-child(6) {
  background-color: hsl(0 100% 100%);
  border: solid;
}
```

Result

Creating reds of different saturation

This example shows reds of different saturations in the sRGB color space.

HTML

```
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
}
div {
  height: 80px;
  margin: 10px;
  width: 80px;
}
```

```
div:nth-child(1) {
  background-color: hsl(0 0% 50%);
}
div:nth-child(2) {
  background-color: hsl(0 20% 50%);
}
div:nth-child(3) {
  background-color: hsl(0 40% 50%);
}
div:nth-child(4) {
  background-color: hsl(0 60% 50%);
}
div:nth-child(5) {
  background-color: hsl(0 80% 50%);
}
div:nth-child(6) {
```

```
  background-color: hsl(0 100% 50%);
}
```

Result

See also

- `opacity`: the property defining transparency at the element level
- `<hue>`: the data type representing the hue angle of a color
- `color`, `background-color`, `border-color`, `box-shadow`, `outline-color`, `text-shadow`: common properties that use `<color>`
- `color()` function
- Applying color to HTML elements using CSS
- Using relative colors
- New functions, gradients, and hues in CSS colors (Level 4) on MDN blog (2023)

alpha() CSS function

The **`alpha()`** CSS function takes an origin `color` and returns that same color with a modified alpha (transparency) channel. The color components of the origin color are preserved unchanged. The result is in the same color space as the origin color.

Syntax

```
/* Replace alpha with a fixed value */
alpha(from red / 50%)
alpha(from var(--my-color) / 80%)

/* Derive alpha relative to the origin color's alpha */
alpha(from var(--my-color) / calc(alpha * 0.5))
```

Parameters

`from <color>`

> The relative color syntax keyword `from` followed by any valid `<color>` value, defining the **origin color**. The color components of the origin color are carried through to the result unchanged; only the alpha channel is affected.

`/ <alpha-value>`

> An `<alpha-value>` specifying the alpha of the output color. This can be a `<number>` between `0` and `1`, a `<percentage>` between `0%` and `100%`, or a `calc()` expression. Within this value, the keyword **`alpha`** may be used to refer to the alpha channel of the origin color as a `<number>` (where `1.0` is equivalent to `100%`).

Return value

A color value in the same color space as the origin color, with identical color components and a modified alpha channel value as specified.

Description

The `alpha()` function is a relative color function that lets you adjust the transparency of any color without rewriting its other components. This is particularly useful when you need a semi-transparent variant of a design token or custom property color, and you want to derive it automatically from the original rather than maintaining a separate value.

Within the alpha value argument, the component keyword `alpha` resolves to the alpha channel of the origin color as a `<number>` in the range `[0, 1]`. This allows you to express the output alpha relative to the input, for example halving it with `calc(alpha * 0.5)`, or clamping it with `clamp(0.2, alpha, 0.8)`.

Unlike general relative color syntax (e.g., `oklch(from ...)`), the `alpha()` function does not expose the individual color channel keywords of the origin color space. It is solely concerned with the alpha channel, keeping the rest of the color intact.

The return value is always in the same color space as the origin color. For example, if the origin color is an `oklch()` color, the result will also be resolved in OKLCh, with the same lightness, chroma, and hue.

Formal syntax

```
&lt;alpha()&gt; =
  alpha( [ from &lt;color&gt; ] [ / [ &lt;alpha-value&gt; | none ] ]? )

&lt;alpha-value&gt; =
  &lt;number&gt;      |
  &lt;percentage&gt;
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Replacing the alpha of a color

In this example, we specify two colors. The second color is defined by passing the first color into the `alpha()` function, setting its alpha channel to a fixed value of `80%` in the process. The two colors are set as the `background-color` of two `<div>` elements to demonstrate the difference.

```
<div class="box1">Original</div>
<div class="box2">Fixed 80% alpha</div>
```

```
:root {
  --my-color: oklch(60% 0.25 315 / 0.3);

  /* Same color, but with alpha set to 80% */
  --my-color-80: alpha(from var(--my-color) / 80%);
}

.box1 {
  background-color: var(--my-color);
}

.box2 {
  background-color: var(--my-color-80);
}
```

```
body {
  font-family: sans-serif;
  display: flex;
  gap: 10%;
  justify-content: center;
```

```
}

div {
  width: 40%;
  height: 100px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Deriving a semi-transparent variant

This example is very similar to the previous one, except that this time the alpha of the second color is computed as half of the first color's alpha, using the `alpha` component keyword inside a `calc()` expression.

```
<div class="box1">Original</div>
<div class="box2">Derived 50% alpha</div>
```

```
:root {
  --my-color: oklch(60% 0.25 315 / 0.8);

  /* Half the opacity of --my-color */
  --my-color-half-opacity: alpha(from var(--my-color) / calc(alpha * 0.5));
}

.box1 {
  background-color: var(--my-color);
}

.box2 {
  background-color: var(--my-color-half-opacity);
}
```

Fading a color on hover

In this example, an element's background fades to `40%` opacity on `:hover`, while the color itself stays the same.

```
<button class="card">Hover over me</button>
```

```
:root {
  --brand: oklch(55% 0.22 270);
}

.card {
  background-color: var(--brand);
  color: white;
  padding: 1rem 2rem;
  border: none;
  border-radius: 0.5rem;
  font-size: 1.1rem;
  cursor: pointer;
  transition: background-color 0.3s;
}

.card:hover {
  background-color: alpha(from var(--brand) / 40%);
}
```

```
body {
  padding: 2rem;
}
```

```
@supports not (color: alpha(from red / 50%)) {
  body::before {
    font-family: sans-serif;
    content: "Your browser does not support the CSS alpha() function.";
    background-color: wheat;
    padding: 1rem 0;
    text-align: center;

    z-index: 1;
    position: fixed;
    inset: 30% 0 auto;
  }
}
```

See also

- `<color>` data type
- `<alpha-value>` data type
- Relative color syntax
- CSS colors module
- CSS custom properties and `var()`

color() CSS function

The **`color()`** functional notation allows a color to be specified in a particular, specified color space rather than the implicit sRGB color space that most of the other color functions operate in.

Support for a particular color space can be detected with the `color-gamut` CSS media feature.

Syntax

```
/* Absolute values */
color(display-p3 1 0.5 0);
color(display-p3 1 0.5 0 / .5);

/* Relative values */
color(from green srgb r g b / 0.5)
color(from #123456 xyz calc(x + 0.75) y calc(z - 0.35))
```

Values

Below are descriptions of the allowed values for both absolute and relative colors.

Absolute value syntax

```
color(colorspace c1 c2 c3[ / A])
```

The parameters are as follows:

`colorspace`

An `<ident>` denoting one of the predefined color spaces: `srgb`, `srgb-linear`, `display-p3`, `display-p3-linear`, `a98-rgb`, `prophoto-rgb`, `rec2020`, `xyz`, `xyz-d50`, or `xyz-d65`.

`c1`, `c2`, `c3`

Each value can be written as a `<number>`, a `<percentage>`, or the keyword `none` (equivalent to `0` in this case). These values represent the component values for the colorspace. When using a `<number>` value, generally, `0` to `1` represents the bounds of the color space. Values outside of that range are permitted but will be out of gamut for the given color space. When using a percentage value, `100%` represents `1` and `0%` represents `0`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

Relative value syntax

```
color(from <color> colorspace c1 c2 c3[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**. This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`colorspace`

An `<ident>` denoting the color space of the output color, generally one of the predefined color spaces: `srgb`, `srgb-linear`, `display-p3`, `display-p3-linear`, `a98-rgb`, `prophoto-rgb`, `rec2020`, `xyz`, `xyz-d50`, or `xyz-d65`.

`c1`, `c2`, `c3`

Each value can be written as a `<number>`, a `<percentage>`, or the keyword `none` (equivalent to `0` in this case). These values represent the component values for the output color. When using a `<number>` value, generally `0` to `1` represents the bounds of the color space. Values outside of that range are permitted but will be out of gamut for the given color space. Generally, when using a percentage value, `100%` represents `1` and `0%` represents `0`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

Defining relative color output channel components

When using relative color syntax inside a `color()` function, the browser converts the origin color into an equivalent color in the specified color space (if it is not already specified as such). The color is defined as three distinct color channel values plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The three color channel values of the origin color are resolved to a `<number>`. For predefined color spaces, depending on which is specified, these values will be one of the following:
  - `r`, `g`, and `b`: Color channel values for the RGB-based color spaces `srgb`, `srgb-linear`, `display-p3`, `display-p3-linear`, `a98-rgb`, `prophoto-rgb`, and `rec2020`.
  - `x`, `y`, and `z`: Color channel values for the CIE XYZ-based color spaces `xyz`, `xyz-d50`, and `xyz-d65`.

    > **Note:** Each of these values is usually between `0` and `1` but, as explained above, they may be outside these bounds.

    > **Note:** Referencing `r`, `g`, and `b` values inside a `color()` function with an XYZ-based colorspace, `x`, `y`, and `z` values inside a `color()` function with an RGB-based colorspace, or any other characters, is invalid. The origin color channel values available inside the function must match the specified type of colorspace.
- `alpha`: The color's transparency value, resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `color()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `red`). While you are unlikely to ever write the following functions because they output the same color as the origin color, this demonstrates how to use the origin color channel values as the output channel values:

```
color(from hsl(0 100% 50%) srgb r g b)
color(from hsl(0 100% 50%) xyz x y z)
```

These functions' output colors are `color(srgb 1 0 0)` and `color(xyz-d65 0.412426 0.212648 0.0193173)`, respectively.

The next functions use absolute values for the output color channel values, outputting completely different colors not based on the origin color:

```
color(from hsl(0 100% 50%) srgb 0.749938 0 0.609579)
/* Computed output color: color(srgb 0.749938 0 0.609579) */

color(from hsl(0 100% 50%) xyz 0.75 0.6554 0.1)
/* Computed output color: color(xyz-d65 0.75 0.6554 0.1 */
```

The following functions use two of the origin color channel values for the output color channel values (`r` and `b`, and `x` and `y`, respectively), but use a new value for the other output channel value (`g` and `z`, respectively), creating a relative color based on the origin color in each case:

```
color(from hsl(0 100% 50%) srgb r 1 b)
/* Computed output color: color(srgb 1 1 0) */

color(from hsl(0 100% 50%) xyz x y 0.5)
/* Computed output color: color(xyz-d65 0.412426 0.212648 0.5) */
```

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels). For example, the `hsl()` color `hsl(0 100% 50%)` is converted to `color(srgb 1 0 0)` in the first case above and `color(xyz 0.412426 0.212648 0.5)` in the second case.

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
color(from hsl(0 100% 50% / 0.8) srgb r g b / alpha)
/* Computed output color: color(srgb 1 0 0 / 0.8) */

color(from hsl(0 100% 50% / 0.8) xyz x y z / 0.5)
/* Computed output color: color(xyz-d65 0.412426 0.212648 0.0193173 / 0.5) */
```

The following examples use `calc()` functions to calculate new channel values for the output colors that are relative to the origin color channel values:

```
color(from hsl(0 100% 50%) srgb calc(r - 0.4) calc(g + 0.1) calc(b + 0.6) / calc(alpha - 0.1))
/* Computed output color: color(srgb 0.6 0.1 0.6 / 0.9)  */

color(from hsl(0 100% 50%) xyz calc(x - 0.3) calc(y + 0.3) calc(z + 0.3) / calc(alpha - 0.1))
/* Computed output color: color(xyz-d65 0.112426 0.512648 0.319317 / 0.9) */
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<color()> =
  color( [ from <color> ]? <colorspace-params> [ / [ <alpha-value> | none ] ]? )

<colorspace-params> =
  <custom-params>          |
  <predefined-rgb-params>  |
  <xyz-params>

<alpha-value> =
  <number>      |
  <percentage>

<custom-params> =
  <dashed-ident> [ <number> | <percentage> | none ]+

<predefined-rgb-params> =
  <predefined-rgb> [ <number> | <percentage> | none ]{3}

<xyz-params> =
  <xyz-space> [ <number> | <percentage> | none ]{3}

<predefined-rgb> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  rec2100-pq         |
  rec2100-hlg        |
  rec2100-linear

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65
```

This syntax reflects the latest standard as per CSS Color HDR Module Level 1, CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using predefined color spaces with color()

The following example shows the effect of varying the lightness, a-axis, and b-axis values of the `color()` function.

HTML

```
<div data-color="red-a98-rgb"></div>
<div data-color="red-prophoto-rgb"></div>
<div data-color="green-srgb-linear"></div>
<div data-color="green-display-p3"></div>
<div data-color="green-display-p3-linear"></div>
<div data-color="blue-rec2020"></div>
<div data-color="blue-srgb"></div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
```

```
[data-color="red-a98-rgb"] {
  background-color: color(a98-rgb 1 0 0);
}
[data-color="red-prophoto-rgb"] {
  background-color: color(prophoto-rgb 1 0 0);
}
/* sRGB gamut, linear-light — full brightness (green channel 100%) */
[data-color="green-srgb-linear"] {
  background-color: color(srgb-linear 0 1 0);
}
/* P3 gamut (wider), gamma-encoded — low brightness (green channel ~21% light output) */
[data-color="green-display-p3"] {
  background-color: color(display-p3 0 0.5 0);
}
/* P3 gamut (wider), linear-light — half brightness (green channel at 50% light output) */
[data-color="green-display-p3-linear"] {
  background-color: color(display-p3-linear 0 0.5 0);
}
[data-color="blue-rec2020"] {
  background-color: color(rec2020 0 0 1);
}
[data-color="blue-srgb"] {
  background-color: color(srgb 0 0 1);
}
```

Result

Using the xyz color space with color()

The following example shows how to use the `xyz` color space to specify a color.

HTML

```
<div data-color="red"></div>
<div data-color="green"></div>
<div data-color="blue"></div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
```

```
[data-color="red"] {
  background-color: color(xyz 45 20 0);
}

[data-color="green"] {
  background-color: color(xyz-d50 0.3 80 0.3);
}

[data-color="blue"] {
  background-color: color(xyz-d65 5 0 50);
}
```

Result

Using color-gamut media queries with color()

This example shows how to use the `color-gamut` media query to detect support for a particular color space and use that color space to specify a color.

HTML

```
<div></div>
<div></div>
<div></div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
```

```
@media (color-gamut: p3) {
  div {
    background-color: color(display-p3 1 0 0);
  }
}

@media (color-gamut: srgb) {
```

```
  div:nth-child(2) {
    background-color: color(srgb 1 0 0);
  }
}

@media (color-gamut: rec2020) {
  div:nth-child(3) {
    background-color: color(rec2020 1 0 0);
  }
}
```

Result

Using relative colors with color()

This example styles three `<div>` elements with different background colors. The middle one is given the unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into a `color()` function, and the output colors have their `g` and `b` channels modified to achieve the desired effect via `calc()` functions. The lightened color has 15% added to those channels, and the darkened color has 15% subtracted from those channels.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

#one {
  background-color: color(
    from var(--base-color) display-p3 r calc(g + 0.15) calc(b + 0.15)
  );
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: color(
    from var(--base-color) display-p3 r calc(g - 0.15) calc(b - 0.15)
  );
}

/* Use @supports to add in support old syntax that requires r g b values
   to be specified as percentages (with units) in calculations.
   This is required for Safari 16.4+ */
@supports (color: color(from red display-p3 r g calc(b + 30%))) {
  #one {
    background-color: color(
      from var(--base-color) display-p3 r calc(g + 15%) calc(b + 15%)
    );
  }

  #three {
    background-color: color(
      from var(--base-color) display-p3 r calc(g - 15%) calc(b - 15%)
    );
  }
}
```

Result

The output is as follows:

See also

- `color` property
- The `<color>` data type for a list of all color notations
- Using relative colors
- Color_format_converter tool
- CSS colors module
- `color-gamut` media feature
- Wide Gamut Color in CSS with Display-p3

color-mix() CSS function

The **`color-mix()`** functional notation takes one or more `<color>` values and returns the result of mixing them in a given colorspace by a given amount.

Syntax

```
/* Polar color space */
color-mix(in hsl, hsl(200 50 80), coral)
color-mix(in hsl, hsl(200 50 80) 20%, coral 80%)

/* Rectangular color space */
color-mix(in srgb, plum, #123456)
color-mix(in lab, plum 60%, #123456 50%)

/* With hue interpolation method */
color-mix(in lch increasing hue, hsl(200deg 50% 80%), coral)
color-mix(in lch longer hue, hsl(200deg 50% 80%) 44%, coral 16%)

/* With a color argument list */
color-mix(in oklab, teal)
color-mix(in oklab, teal 20%, olive 30%, blue 50%)
color-mix(in oklab, teal, olive, blue, purple)
```

Parameters

The `color-mix( <color-interpolation-method>? , [ <color> && <percentage [0,100]>? ]#)` accepts the following parameters:

`<color-interpolation-method>` Optional

Specifies what interpolation method should be used to mix the colors. It consists of the `in` keyword followed by a color space (one of the color spaces listed in the formal syntax, defaulting to `oklab`), and, optionally, a `<hue-interpolation-method>`, which defaults to `shorter hue`.

`<color>`

A color to mix; can be any valid `<color>` value.

`<percentage>` Optional

A percentage value specifying the amount of the corresponding color to mix; can be any `<percentage>` value between `0%` and `100%`, inclusive.

Return value

A `<color>`; the result of mixing the colors, in the given `<color-space>`, in the specified amounts and hue direction.

Description

The `color-mix()` function enables mixing one or more `<color>` values of any type, in a specific ratio, in a given colorspace, using either a shorter or longer hue interpolation method. Browsers support a plethora of color spaces; the `color-mix()` function enables a wide range of colors to be mixed, not limited to the sRGB color space.

This demo allows you to select two colors, `color-one` and `color-two`, and mix them, optionally setting each color's percentage, the color space in which the colors are mixed, and the interpolation method. The source colors are shown on the outside, and the mixed color is shown in the middle. You can change colors by clicking on them and choosing a new color using the resulting color picker. Change the percentage values of each color using the sliders. Change the color space via the drop-down menu.

Choosing a color space

Choosing the correct color space is important for producing desired results. Given the same colors to mix, different color spaces may be more appropriate depending on the interpolation use case.

- If the result of physically mixing colored lights is desired, the CIE XYZ or srgb-linear color space is appropriate, because they are linear in light intensity.
- If colors need to be evenly spaced perceptually (such as in a gradient), the Oklab (and older Lab) color spaces are appropriate, because they are designed to be perceptually uniform.
- If avoiding graying out in color mixing is desired, i.e., maximizing chroma throughout the transition, the Oklch (and older LCH) color spaces work well.
- Only use sRGB if you need to match the behavior of a specific device or software that uses sRGB. The sRGB color space is neither linear-light nor perceptually uniform, and produces poorer results such as overly dark or grayish mixes.

Color interpolation method

The `<color-interpolation-method>` specifies what interpolation method should be used to mix the colors. It consists of the `in` keyword and the color space the colors should be mixed in. The color space must be one of the available color spaces listed in the formal syntax. Depending on the color space used, you can optionally direct the hue to be mixed along a longer or shorter path.

The `<rectangular-color-space>` category includes `srgb`, `srgb-linear`, `display-p3`, `a98-rgb`, `prophoto-rgb`, `rec2020`, `lab`, `oklab`, `xyz`, `xyz-d50`, and `xyz-d65`.

The `<polar-color-space>` category includes `hsl`, `hwb`, `lch`, and `oklch`. With these you can optionally follow the color space name with a `<hue-interpolation-method>`. This value defaults to `shorter hue`, but can also be set to `longer hue`, `increasing hue`, or `decreasing hue`.

Default color space and interpolation method

When mixing colors without specifying a color space, the `oklab` color space is used.

The following two declarations are equivalent:

```
background-color: color-mix(red, blue);
background-color: color-mix(in oklab, red, blue);
```

When using a polar color space such as `oklch`, the hue interpolation method defaults to `shorter hue`. The following two declarations are equivalent:

```
background-color: color-mix(in oklch, red, blue);
background-color: color-mix(in oklch shorter hue, red, blue);
```

Color percentages

Each color can be declared with a `<percentage>` value between `0%` and `100%`, which specifies the amount of the corresponding color to mix. The percentages are normalized if the total value of the declared percentages does not equal `100%`.

When two colors are being mixed, the two color percentages (we'll refer to them as `p1` and `p2`) are normalized as follows:

- If both `p1` and `p2` are omitted, then `p1 = p2 = 50%`.
- If `p1` is omitted, then `p1 = 100% - p2`.
- If `p2` is omitted, then `p2 = 100% - p1`.
- If `p1 = p2 = 0%`, the function is invalid.
- If `p1 + p2 ≠ 100%`, then `p1' = p1 / (p1 + p2)` and `p2' = p2 / (p1 + p2)`, where `p1'` and `p2'` are the normalization results.
    - If `p1 + p2 < 100%`, then an alpha multiplier of `p1 + p2` is applied to the resulting color. This is similar to mixing in `transparent`, with percentage `pt = 100% - p1 - p2`.

Formal syntax

```
<color-mix()> =
  color-mix( <color-interpolation-method>? , [ <color> && <percentage [0,100]>? ]# )

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<rectangular-color-space> =
  srgb              |
  srgb-linear       |
  display-p3        |
  display-p3-linear |
  a98-rgb           |
  prophoto-rgb      |
  rec2020           |
  lab               |
  oklab             |
  <xyz-space>

<polar-color-space> =
  hsl   |
  hwb   |
  lch   |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<xyz-space> =
  xyz     |
  xyz-d50 |
  xyz-d65
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Mixing two colors

This example demonstrates mixing two colors, red `#a71e14` at different percentages and white with no percentage given. The higher the percentage of `#a71e14` is mixed, the more red and less white the output color is.

HTML

```
<ul>
  <li>0%</li>
  <li>25%</li>
  <li>50%</li>
  <li>75%</li>
  <li>100%</li>
  <li></li>
</ul>
```

CSS

The `color-mix()` function is used to add increasing percentages of red, up to 100%. The 6th `<li>` doesn't include a percentage for either color.

```
ul {
  display: flex;
  list-style-type: none;
  font-size: 150%;
  gap: 10px;
  border: 2px solid;
  padding: 10px;
}

li {
  padding: 10px;
  flex: 1;
  box-sizing: border-box;
  font-family: monospace;
  outline: 3px solid #a71e14;
  text-align: center;
}
```

```
li:nth-child(1) {
  background-color: color-mix(in oklab, #a71e14 0%, white);
}

li:nth-child(2) {
  background-color: color-mix(in oklab, #a71e14 25%, white);
}

li:nth-child(3) {
  background-color: color-mix(in oklab, #a71e14 50%, white);
}

li:nth-child(4) {
  background-color: color-mix(in oklab, #a71e14 75%, white);
}

li:nth-child(5) {
  background-color: color-mix(in oklab, #a71e14 100%, white);
}

li:nth-child(6) {
  background-color: color-mix(in oklab, #a71e14, white);
}
```

Result

The total value of both colors in a `color-mix()` function is 100%, even if the values set by the developer don't total 100%. In this example, as only one color has a percentage assigned, the other color is implicitly given a percentage value so that the combined total equals 100%. In the last `<li>`, where neither color is assigned a percentage, both default to 50%.

Mixing a list of colors

This example demonstrates passing a color argument list to `color-mix()`. The function accepts any number of colors, not just two, and each of them can optionally be given a percentage.

HTML

```
<ul>
  <li>1 color</li>
  <li>3 colors, with percentages</li>
  <li>4 colors, no percentages</li>
</ul>
```

CSS

The first `<li>` mixes a single color, which resolves to that color. The second mixes three colors whose percentages total 100%. The third mixes four colors with no percentage, so each one gets an equal share.

```
ul {
  display: flex;
  list-style-type: none;
  font-size: 150%;
  gap: 10px;
  border: 2px solid;
  padding: 10px;
}

li {
  padding: 10px;
  flex: 1;
  box-sizing: border-box;
  font-family: monospace;
  text-align: center;
}
```

```
li:nth-child(1) {
  background-color: color-mix(in oklab, teal);
}

li:nth-child(2) {
  background-color: color-mix(in oklab, teal 20%, olive 30%, blue 50%);
}

li:nth-child(3) {
  background-color: color-mix(in oklab, teal, olive, blue, purple);
}
```

```
@supports not (color: color-mix(in oklab, red, white, blue)) {
  body::before {
    content: "Your browser doesn't support color lists in the color-mix() function.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Adding transparency

This example demonstrates using the `color-mix()` function to add transparency to a color by mixing any color with `transparent`.

HTML

```
<ul>
  <li>0%</li>
  <li>25%</li>
  <li>50%</li>
  <li>75%</li>
  <li>100%</li>
  <li></li>
</ul>
```

CSS

The `color-mix()` function is used to add increasing percentages of `red`, which is declared using a custom property named `--base`, defined on the `:root`. The 6th `<li>` doesn't include a percentage, creating an output color that is half as opaque as the `--base` color. We include a striped background on the `<ul>` to make the transparency visible.

```
ul {
  display: flex;
  list-style-type: none;
  font-size: 150%;
  gap: 10px;
  border: 2px solid;
  padding: 10px;
}

li {
  padding: 10px;
  flex: 1;
  box-sizing: border-box;
  font-family: monospace;
  outline: 1px solid var(--base);
  text-align: center;
}
```

```
:root {
  --base: red;
}

ul {
  background: repeating-linear-gradient(
    45deg,
    chocolate 0px 2px,
    white 2px 12px
  );
}

li:nth-child(1) {
  background-color: color-mix(in srgb, var(--base) 0%, transparent);
}

li:nth-child(2) {
  background-color: color-mix(in srgb, var(--base) 25%, transparent);
}

li:nth-child(3) {
  background-color: color-mix(in srgb, var(--base) 50%, transparent);
}

li:nth-child(4) {
  background-color: color-mix(in srgb, var(--base) 75%, transparent);
}

li:nth-child(5) {
  background-color: color-mix(in srgb, var(--base) 100%, transparent);
}

li:nth-child(6) {
  background-color: color-mix(in srgb, var(--base), transparent);
}
```

Result

In this way, the `color-mix()` function can be used to add transparency to any color, even if the color is already non-opaque (with an alpha channel value < 1). However, `color-mix()` can't be used to make a semi-transparent color fully opaque. For this, use a relative color with a CSS color function. Relative colors can alter the value of any color channel, including increasing a color's alpha channel to render the color fully opaque.

Using hue interpolation in color-mix()

This example demonstrates the hue interpolation methods available to the `color-mix()` function. When using hue interpolation, the resulting hue is between the hue values of the colors being mixed. The value will be different based on which route is taken around the color wheel.

For more information, see `<hue-interpolation-method>`.

```
<p>longer</p>
<ul>
  <li>100%</li>
  <li>80%</li>
  <li>60%</li>
  <li>40%</li>
  <li>20%</li>
  <li>0%</li>
</ul>
<p>shorter</p>
<ul>
  <li>100%</li>
  <li>80%</li>
  <li>60%</li>
  <li>40%</li>
  <li>20%</li>
  <li>0%</li>
</ul>
<p>increasing</p>
<ul>
  <li>100%</li>
  <li>80%</li>
  <li>60%</li>
  <li>40%</li>
  <li>20%</li>
  <li>0%</li>
</ul>
<p>decreasing</p>
<ul>
  <li>100%</li>
  <li>80%</li>
  <li>60%</li>
  <li>40%</li>
  <li>20%</li>
  <li>0%</li>
</ul>
```

CSS

The `shorter hue` interpolation method takes the shorter route around the color wheel, whereas the `longer hue` interpolation method takes the longer route. With `increasing hue`, the route starts with increasing values. With `decreasing hue` the value decreases. We mix two `<named-color>` values to create a series of `lch()` intermediary colors that differ based on which route is taken around the color wheel. The mixed colors include `red`, `blue`, and `yellow` with LCH hue values of approximately 41deg, 301deg, and 100deg, respectively.

To reduce code redundancy, we used CSS custom properties for both colors and for the interpolation method, setting different values on each `<ul>`.

```
body {
  font-family: monospace;
}
ul {
  display: flex;
  list-style-type: none;
  font-size: 150%;
```

```
  gap: 10px;
  padding: 10px;
  margin: 0;
}

li {
  padding: 10px;
  flex: 1;
  outline: 1px solid var(--base);
  text-align: center;
}
```

```
ul:nth-of-type(1) {
  --distance: longer; /* 52 degree hue increments */
  --base: red;
  --mixin: blue;
}
ul:nth-of-type(2) {
  /* 20 degree hue decrements */
  --distance: shorter;
  --base: red;
  --mixin: blue;
}
ul:nth-of-type(3) {
  /* 40 degree hue increments */
  --distance: increasing;
  --base: yellow;
  --mixin: blue;
}
ul:nth-of-type(4) {
  /* 32 degree hue decrements */
  --distance: decreasing;
  --base: yellow;
  --mixin: blue;
}

li:nth-child(1) {
  background-color: color-mix(
    in lch var(--distance) hue,
    var(--base) 100%,
    var(--mixin)
  );
}

li:nth-child(2) {
  background-color: color-mix(
    in lch var(--distance) hue,
    var(--base) 80%,
    var(--mixin)
  );
}

li:nth-child(3) {
  background-color: color-mix(
    in lch var(--distance) hue,
    var(--base) 60%,
    var(--mixin)
  );
}

li:nth-child(4) {
  background-color: color-mix(
    in lch var(--distance) hue,
    var(--base) 40%,
    var(--mixin)
  );
}

li:nth-child(5) {
  background-color: color-mix(
    in lch var(--distance) hue,
    var(--base) 20%,
    var(--mixin)
  );
}

li:nth-child(6) {
  background-color: color-mix(
    in lch var(--distance) hue,
    var(--base) 0%,
    var(--mixin)
  );
}
```

Result

With `longer hue` the increments or decrements between colors will always be the same or larger than when using `shorter hue`. Use `increasing hue` or `decreasing hue` when the direction of the hue value change is more important than the length between values.

See also

- `<color>`
- `<color-interpolation-method>`
- `<hue>`
- CSS relative colors

contrast-color() CSS function

The **`contrast-color()`** CSS function takes a `color` value and returns a contrasting color. The function commonly ensures the WCAG AA minimum contrast. The browsers may use different and better algorithms.

The `contrast-color()` function enables specifying a text color and automatically generating a contrasting background color, or vice versa. It avoids the need to maintain background-text color pairs.

Syntax

```
contrast-color(red)
contrast-color(var(--backgroundColor))
```

Parameters

`color`

Any valid `<color>` value.

Return value

A `<named-color>` of either `white` or `black`.

Description

The `contrast-color()` function returns a value of `white` or `black`, depending on which value has the greatest contrast with the input color. If both `white` and `black` have the same contrast with the input color, `white` is returned.

> **Warning:** WCAG AA (4.5:1) contrast is not capable of producing clearly readable text in all cases. Mid-tone background colors generally don't provide enough contrast with either black or white foreground. For example, `contrast-color()` on a royal bluish (`#2277d3`) background produces black text, which is not readable for small text. It is recommended, therefore, to use light or dark colors with the `contrast-color()` function.

Examples

Contrasting text for a button

In the following example, the browser automatically applies the selected color to the background of the "Button" `<button>`, and the contrasting color (black or white) to its text.

```
<label>
  Select background color of the button:
  <input type="color" id="colorPicker" value="#660066" />
</label>
<br />
<button>Button</button>
```

```
:root {
  --button-color: lightblue;
}

button {
  background-color: var(--button-color);

  /* Set contrasting text color automatically */
  color: contrast-color(var(--button-color));
}
```

```
body {
  padding: 1rem;
}

button {
  margin: 3rem;
  padding: 1rem;
  width: 150px;
  font-size: 2rem;
  border-radius: 1rem;
}

@supports not (color: contrast-color(red)) {
  body::before {
    content: "Your browser doesn't support the contrast-color() function.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

```
const colorPicker = document.getElementById("colorPicker");
const root = document.documentElement;

function updateColor(color) {
  root.style.setProperty("--button-color", colorPicker.value);
}

colorPicker.addEventListener("input", updateColor);
updateColor();
```

Light and dark mode usage

In the following example, the `prefers-color-scheme` media query is used to set a background color based on operating system or browser color scheme settings. The `contrast-color()` function is used to set the text color automatically.

Try changing the browser or OS dark mode setting to see the effect.

```
<pre>
    Q: How does CSS transform light into energy?
  Ans: Using <a href="/en-US/docs/Web/CSS/Reference/Properties/font-synthesis">font-synthesis</a>.
</pre>
```

```
:root {
  --background-color: navy;
}

@media (prefers-color-scheme: light) {
```

```
  :root {
    --background-color: wheat;
  }
}

body,
a {
  background-color: var(--background-color);
  color: contrast-color(var(--background-color));
}
```

```
body {
  padding: 2rem;
  font-size: 1.2rem;
}

pre {
  margin-top: 3rem;
}

@supports not (color: contrast-color(red)) {
  body::before {
    content: "Your browser doesn't support the contrast-color() function.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

See also

- `contrast()`
- CSS colors module
- CSS custom properties and `var()`
- `prefers-contrast` and `prefers-color-scheme` `@media` features
- WCAG: color contrast
- How to have the browser pick a contrasting color in CSS on webkit.org (2025)
- WebAIM Contrast Checker on webaim.org (2025)

device-cmyk() CSS function

The **`device-cmyk()`** functional notation is used to express CMYK colors in a device dependent way, specifying the cyan, magenta, yellow, and black components.

This approach to color is useful when creating material to be output to a particular printer, when the output for particular ink combinations is known. CSS processors may attempt to approximate the color for other media; however, the end result is likely to be different from the printed result without knowing the precise output colorimetry. A `@color-profile` declaration for `device-cmyk` can specify the exact color profile for conversion.

Syntax

```
device-cmyk(0 81% 81% 30%);
device-cmyk(none 0.81 0.81 0.3);
device-cmyk(0 81% 81% 30% / .5);
```

Values

Functional notation: `device-cmyk(C M Y K[ / A])`

`C`, `M`, `Y`, `K`

Each a `<number>` between `0` and `1`, a `<percentage>` between `0%` and `100%`, or the keyword `none` providing the cyan, magenta, yellow, and black components of CMYK color.

> **Note:** See Missing color components for more information on the effect of `none`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel.

Formal syntax

```
<device-cmyk()> =
  <legacy-device-cmyk-syntax>  |
  <modern-device-cmyk-syntax>

<legacy-device-cmyk-syntax> =
  device-cmyk( <number>#{4} )

<modern-device-cmyk-syntax> =
  device-cmyk( <cmyk-component>{4} [ / [ <alpha-value> | none ] ]? )

<cmyk-component> =
  <number>      |
  <percentage>  |
  none

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Browser compatibility

Currently, no browsers support this feature.

See also

- CSS colors module

- `@page`

hsl() CSS function

> **Note:** The `hsla()` functional notation is an alias for `hsl()`. They are exactly equivalent. It is recommended to use `hsl()`.

The **`hsl()`** functional notation expresses a color in the sRGB color space according to its *hue*, *saturation*, and *lightness* components. An optional *alpha* component represents the color's transparency.

Defining *complementary colors* with `hsl()` can be done by adding or subtracting 180 degrees from the hue value, as they are positioned on the same diameter of the color wheel. For example, if the hue angle of a color is `10deg`, its complementary has `190deg` as its hue angle.

Try it

```
background: hsl(50 80% 40%);
```

```
background: hsl(150deg 30% 60%);
```

```
background: hsl(0.3turn 60% 45% / 0.7);
```

```
background: hsl(0 80% 50% / 25%);
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Syntax

```
/* Absolute values */
hsl(120deg 75% 25%)
hsl(120 75 25) /* deg and % units are optional */
hsl(120deg 75% 25% / 60%)
hsl(none 75% 25%)

/* Relative values */
hsl(from green h s l / 0.5)
hsl(from #123456 h s calc(l + 20))
hsl(from rgb(200 0 0) calc(h + 30) s calc(l + 30))

/* Legacy 'hsla()' alias */
hsla(120deg 75% 25% / 60%)

/* Legacy format */
hsl(120, 75%, 25%)
hsl(120deg, 75%, 25%, 0.8)
```

> **Note:** `hsl()`/`hsla()` can also be written in a legacy form in which all values are separated with commas, for example `hsl(120, 75%, 25%)` or `hsla(120deg, 75%, 25%, 0.8)`. The `none` value is not permitted in the comma-separated legacy syntax, the `deg` on the hue value is optional, and the `%` units are required for the saturation and lightness values.

Values

Below are descriptions of the allowed values for both absolute and relative colors.

Absolute value syntax

```
hsl(H S L[ / A])
```

The parameters are as follows:

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg` in this case) representing the color's `<hue>` angle.

> **Note:** The angles corresponding to particular hues differ across the sRGB (used by `hsl()` and `hwb()`), CIELAB (used by `lch()`), and Oklab (used by `oklch()`) color spaces. See the `<hue>` reference page for more detail and examples.

`S`

A `<percentage>` or the keyword `none` (equivalent to `0%` in this case). This value represents the color's saturation. Here `100%` is completely saturated, while `0%` is completely unsaturated (gray).

`L`

A `<percentage>` or the keyword `none` (equivalent to `0%` in this case). This value represents the color's lightness. Here `100%` is white, `0%` is black, and `50%` is "normal".

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

> **Note:** Absolute `hsl()` colors are serialized to `rgb()` values. The values of the red, green, and blue components may be rounded in serialization.

Relative value syntax

```
hsl(from <color> H S L[ / A])
```

The parameters are as follows:

`from <color>`

> The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**. This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`H`

> A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg` in this case) representing the output color's `<hue>` angle.

`S`

> A `<percentage>` or the keyword `none` (equivalent to `0%` in this case). This represents the saturation of the output color. Here `100%` is completely saturated, while `0%` is completely unsaturated (gray).

`L`

> A `<percentage>` or the keyword `none` (equivalent to `0%` in this case). This represents the lightness of the output color. Here `100%` is white, `0%` is black, and `50%` is "normal".

`A` Optional

> An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

> **Note:** To fully enable the representation of the full spectrum of visible colors, the output of relative `hsl()` color functions is serialized to `color(srgb)`. That means that querying the output color value via the `HTMLElement.style` property or the `CSSStyleDeclaration.getPropertyValue()` method returns the output color as a `color(srgb ...)` value.

Defining relative color output channel components

When using relative color syntax inside an `hsl()` function, the browser converts the origin color into an equivalent HSL color (if it is not already specified as such). The color is defined as three distinct color channel values — `h` (hue), `s` (saturation), and `l` (lightness) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `h` value is resolved to a `<number>` between `0` and `360`, inclusive, that represents the origin color's `<hue>` degree value.
- The `s` and `l` values are each resolved to a `<number>` between `0` and `100`, inclusive, where `100` is equivalent to `100%`.
- The `alpha` value is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `hsl()` syntax.

Let's start with an origin color of `rgb(255 0 0)` (equivalent to `hsl(0 100% 50%)`). The following function outputs the same color as the origin color — it uses the origin color's `h`, `s`, and `l` channel values (`0`, `100%`, and `50%`) as the output channel values:

```
hsl(from rgb(255 0 0) h s l)
```

This function's output color is the sRGB `color()` equivalent of `hsl(0 100% 50%)`: `color(srgb 1 0 0)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
hsl(from rgb(255 0 0) 240 60% 70%)
```

In the above case, the output color is the sRGB `color()` equivalent of `hsl(240 60% 70%)`: `color(srgb 0.52 0.52 0.88)`.

The following function creates a relative color based on the origin color:

```
hsl(from rgb(255 0 0) h 30% 60%)
```

This example:

- Converts the origin color (`rgb(255 0 0)`) into an `hsl()` equivalent (`hsl(0 100% 50%)`).
- Sets the `H` channel value for the output color to those of the origin color `hsl()` equivalent's `H` channel value — `0`.
- Sets the output color's `S` and `L` channel values to new values not based on the origin color: `30%` and `60%`, respectively.

The final output color is the equivalent of `hsl(0 30% 60%)` in the sRGB color space — `color(srgb 0.72 0.48 0.48)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
hsl(from rgb(255 0 0 / 0.8) h s l / alpha)
/* Computed output color: color(srgb 1 0 0 / 0.8) */

hsl(from rgb(255 0 0 / 0.8) h s l / 0.5)
/* Computed output color: color(srgb 1 0 0 / 0.5) */
```

In the following example, the `rgb()` origin color is again converted into an `hsl()` representation — `hsl(0 100% 50% / 0.8)`. `calc()` calculations are applied to the `H`, `S`, `L`, and `A` values, and the final output color is the equivalent of `hsl(60 80% 30% / 0.7)` in the sRGB color space: `color(srgb 0.72 0.72 0.08 / 0.7)`.

```
hsl(from rgb(255 0 0 / 0.8) calc(h + 60) calc(s - 20) calc(l - 10) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<hsl()> =
  <legacy-hsl-syntax>  |
  <modern-hsl-syntax>

<legacy-hsl-syntax> =
  hsl( <hue> , <percentage> , <percentage> , <alpha-value>? )

<modern-hsl-syntax> =
  hsl( [ from <color> ]? [ <hue> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<hue> =
  <number>  |
  <angle>

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using hsl() with conic-gradient()

The `hsl()` function works well with `conic-gradient()` as both deal with angles.

```
<div></div>
```

CSS

```
div {
  width: 100px;
  height: 100px;
  background: conic-gradient(
    hsl(360 100% 50%),
    hsl(315 100% 50%),
    hsl(270 100% 50%),
    hsl(225 100% 50%),
    hsl(180 100% 50%),
    hsl(135 100% 50%),
    hsl(90 100% 50%),
    hsl(45 100% 50%),
    hsl(0 100% 50%)
  );
  clip-path: circle(closest-side);
}
```

Result

Using relative colors with hsl()

This example styles three `<div>` elements with different background colors. The middle one is given the unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into an `hsl()` function, and the output color has its lightness channel modified to achieve the desired effect via a `calc()` function, while the hue and saturation are left unchanged. The lightened color has 20% added to the lightness channel, and the darkened color has 20% subtracted from it.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

#one {
  background-color: hsl(from var(--base-color) h s calc(l + 20));
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: hsl(from var(--base-color) h s calc(l - 20));
}

/* Use @supports to add in support for old syntax that requires % units to
```

```
   be specified in lightness calculations */
@supports (color: hsl(from red h s calc(l - 20%))) {
  #one {
    background-color: hsl(from var(--base-color) h s calc(l + 20%));
  }

  #three {
    background-color: hsl(from var(--base-color) h s calc(l - 20%));
  }
}
```

Result

The output is as follows:

Legacy syntax: comma-separated values

For legacy reasons, the `hsl()` function accepts a form in which all values are separated using commas.

HTML

```
<div class="space-separated"></div>
<div class="comma-separated"></div>
```

CSS

```
div {
  width: 100px;
  height: 50px;
  margin: 1rem;
}

div.space-separated {
  background-color: hsl(0 100% 50% / 50%);
}

div.comma-separated {
  background-color: hsl(0, 100%, 50%, 0.5);
}
```

Result

Legacy versus modern syntax

The example demonstrates how the `hsla()` syntax is an alias for `hsl()`; both are supported using both modern and legacy (comma-separated) syntaxes.

HTML

```
<div class="modern">HSL</div>
<div class="legacy">HSL</div>
<div class="modernWithAlpha">HSL</div>
<div class="modernHSLA">HSLA</div>
<div class="legacyHSLA">HSLA</div>
```

CSS

```
div {
  width: 100px;
  min-height: 50px;
  font-family: sans-serif;
  display: flex;
  align-items: center;
  justify-content: center;
}
body {
  display: flex;
  gap: 20px;
}
```

```
div.modern {
  background-color: hsl(90 80% 50%);
}

div.legacy {
  background-color: hsl(90, 80%, 50%);
}

div.modernWithAlpha {
  background-color: hsl(90 80% 50% / 50%);
}

div.modernHSLA {
  background-color: hsla(90 80% 50% / 50%);
}

div.legacyHSLA {
  background-color: hsla(90, 80%, 50%, 0.5);
}
```

Result

See also

- `<hue>` data type
- `lch()` and `hwb()` color functions
- Hue interpolation in `color-mix()`
- List of all color notations
- Color format converter tool
- Using relative colors
- CSS colors module

- Color picker tool by Lea Verou

hwb() CSS function

The **`hwb()`** functional notation expresses a color in the sRGB color space according to its hue, whiteness, and blackness. An optional alpha component represents the color's transparency.

Try it

```
background: hwb(12 50% 0%);
```

```
background: hwb(50deg 30% 40%);
```

```
background: hwb(0.5turn 10% 0% / 0.5);
```

```
background: hwb(0 100% 0% / 50%);
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Syntax

```
/* Absolute values */
hwb(194 0% 0%)
hwb(194 0% 0% / .5)

/* Relative values */
hwb(from green h w b / 0.5)
hwb(from #123456 h calc(w + 30) b)
hwb(from lch(40% 70 240deg) h w calc(b - 30))
```

Description

This color function in the sRGB color space is defined by a `<hue>` angle value, a whiteness value, a blackness value, and, optionally, an alpha value representing the color's transparency.

The angles corresponding to particular hues differ across the sRGB (used by `hsl()` and `hwb()`), CIELAB (used by `lch()`), and Oklab (used by `oklch()`) color spaces. `hwb()` is in the same color space as `hsl()`, and therefore has the same hue color angles. See the `<hue>` reference page for more detail and examples, or try changing the hues on the color picker to see it in action.

An `hwb()` color is fully saturated when its whiteness (`W`) and blackness (`B`) values are both `0`. For any hue value `H`, `hwb(H 0% 0%)` is the same color as `hsl(H 100% 50%)`. Increasing the whiteness value lightens the color. Increasing the blackness darkens the color.

When both the blackness and whiteness are greater than 0, the color gets muted, tending towards gray. When the amount of whiteness and blackness added in are equal to or greater than 100% — in other words, if `W + B >= 100%`, the color function defines a shade of gray. When the sum of both values is greater than 100% (`W + B > 100%`), the whiteness and blackness values of the gray color are effectively normalized as `W / (W + B)` and `B / (W + B)`, respectively.

Values

Below are descriptions of the allowed values for both absolute and relative colors.

Absolute value syntax

```
hwb(H W B[ / A])
```

The parameters are as follows:

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg` in this case) representing the color's `<hue>` angle.

`W`

A `<percentage>` representing the color's whiteness or the keyword `none` (equivalent to `0%` in this case) to mix in. `0%` represents no whiteness. `100%` represents full whiteness if `B` is `0`, otherwise both the `W` and `B` values are normalized.

`B`

A `<percentage>` representing the color's blackness or the keyword `none` (equivalent to `0%` in this case) to mix in. `0%` represents no blackness. `100%` represents full blackness if `W` is `0`, otherwise both the `W` and `B` values are normalized.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

> **Note:** Absolute `hwb()` colors are serialized to `rgb()` values. The values of the red, green, and blue components may be rounded in serialization.

Relative value syntax

```
hwb(from <color> H W B[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**. This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg` in this case) representing the output color's `<hue>` angle.

`W`

A `<percentage>` representing the color's whiteness or the keyword `none` (equivalent to `0%` in this case) to mix in. `0%` represents no whiteness. `100%` represents full whiteness if `B` is `0`, otherwise both the `W` and `B` values are normalized.

`B`

A `<percentage>` representing the color's blackness or the keyword `none` (equivalent to `0%` in this case) to mix in. `0%` represents no blackness. `100%` represents full blackness if `W` is `0`, otherwise both the `W` and `B` values are normalized.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

> **Note:** To fully enable the representation of the full spectrum of visible colors, the output of relative `hwb()` color functions is serialized to `color(srgb)`. That means that querying the output color value via the `HTMLElement.style` property or the `CSSStyleDeclaration.getPropertyValue()` method returns the output color as a `color(srgb ...)` value.

Defining relative color output channel components

When using relative color syntax inside an `hwb()` function, the browser converts the origin color into an equivalent HWB color (if it is not already specified as such). The color is defined as three distinct color channel values — `h` (hue), `w` (white), and `b` (black) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `h` channel value is resolved to a `<number>` between `0` and `360`, inclusive.
- The `w` and `b` channels are each resolved to a `<number>` between `0` and `100`, inclusive.
- The `alpha` channel is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `hwb()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `hwb(0 0% 0%)`). The following function outputs the same color as the origin color — it uses the origin color's `h`, `w`, and `b` channel values (`0`, `0%`, and `0%`) as the output channel values:

```
hwb(from hsl(0 100% 50%) h w b)
```

This function's output color is the sRGB `color()` equivalent of `hwb(0 0% 0%)`: `color(srgb 1 0 0)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
hwb(from hsl(0 100% 50%) 240 52% 12%)
```

In the above case, the output color is the sRGB `color()` equivalent of `hwb(240 52% 12%)`: `color(srgb 0.52 0.52 0.88)`.

The following function creates a relative color based on the origin color:

```
hwb(from hsl(0 100% 50%) h 30% b)
```

This example:

- Converts the origin color (`hsl(0 100% 50%)`) into an `hwb()` equivalent (`hwb(0 0% 0%)`).
- Sets the `H` and `B` channel values for the output color to those of the origin color `hwb()` equivalent's `H` and `B` channel values — those values are `0` and `0%`, respectively.
- Sets the output color's `W` channel value to a new value not based on the origin color: `30%`.

The final output color is the equivalent of `hwb(0 30% 0%)` in the sRGB color space — `color(srgb 1 0.3 0.3)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model or space as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
hwb(from hsl(0 100% 50% / 0.8) h w b / alpha)
/* Computed output color: color(srgb 1 0 0 / 0.8) */

hwb(from hsl(0 100% 50% / 0.8) h w b / 0.5)
/* Computed output color: color(srgb 1 0 0 / 0.5) */
```

In the following example, the `hsl()` origin color is again converted into an `hwb()` representation — `hwb(0 0% 0%)`. `calc()` calculations are applied to the `H`, `W`, `B`, and `A` values, and the final output color is the equivalent of `hwb(120 25% 10% / 0.9)` in the sRGB color space: `color(srgb 0.25 0.9 0.25 / 0.9)`.

```
hwb(from hsl(0 100% 50%) calc(h + 120) calc(w + 25) calc(b + 10) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<hwb()> =
  hwb( [ from <color> ]? [ <hue> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )
```

```
<hue> =
  <number>  |
  <angle>

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using relative colors with hwb()

This example styles three `<div>` elements with different background colors. The middle one is given the unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into an `hwb()` function, and the output colors have their white and black channels modified to achieve the desired effect via a `calc()` function. The lightened color has 30% added to the white channel, and the darkened color has 30% added to the black channel.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

/* As per the spec, w and b values should resolve to a number between 0-100
   However, Chrome 121+ incorrectly resolves them to numbers between 0-1
   hence currently using calculations like w + 0.3 instead of w + 30 */

#one {
  background-color: hwb(from var(--base-color) h calc(w + 0.3) b);
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: hwb(from var(--base-color) h w calc(b + 0.3));
}

/* Use @supports to add in support for old syntax that requires % units to
   be specified in w and b calculations. This is required for Safari 16.4+. */
@supports (color: hwb(from red h w calc(b + 30%))) {
  #one {
    background-color: hwb(from var(--base-color) h calc(w + 30%) b);
  }

  #three {
    background-color: hwb(from var(--base-color) h w calc(b + 30%));
  }
}
```

Result

The output is as follows:

See also

- `<color>`: For a list of all color notations
- Color format converter tool
- Using relative colors
- CSS colors module
- `<hue>`: the data type representing a hue angle of a color

lab() CSS function

The **`lab()`** functional notation expresses a given color in the CIE L*a*b* color space.

Lab represents the entire range of colors that humans can see by specifying the color's lightness, a red/green axis value, a blue/yellow axis value, and an optional alpha transparency value.

Syntax

```
/* Absolute values */
lab(29.2345% 39.3825 20.0664);
lab(52.2345% 40.1645 59.9971);
lab(52.2345% 40.1645 59.9971 / .5);

/* Relative values */
lab(from green l a b / 0.5)
```

```
lab(from #123456 calc(l + 10) a b)
lab(from hsl(180 100% 50%) calc(l - 10) a b)
```

Values

Below are descriptions of the allowed values for both absolute and relative colors.

Absolute value syntax

```
lab(L a b[ / A])
```

The parameters are as follows:

`L`

A `<number>` between `0` and `100`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's lightness. Here the number `0` corresponds to `0%` (black) and the number `100` corresponds to `100%` (white).

`a`

A `<number>` between `-125` and `125`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's distance along the `a` axis, which defines how green (moving towards `-125`) or red (moving towards `+125`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±125` (`±100%`) limits. In practice, values cannot exceed `±160`.

`b`

A `<number>` between `-125` and `125`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's distance along the `b` axis, which defines how blue (moving towards `-125`) or yellow (moving towards `+125`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±125` (`±100%`) limits. In practice, values cannot exceed `±160`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

Relative value syntax

```
lab(from <color> L a b[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**. This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`L`

A `<number>` between `0` and `100`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case) This value represents the lightness of the output color. Here the number `0` corresponds to `0%` (black) and the number `100` corresponds to `100%` (white).

`a`

A `<number>` between `-125` and `125`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value represents the output color's distance along the `a` axis, which defines how green (moving towards `-125`) or red (moving towards `+125`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±125` (`±100%`) limits. In practice, values cannot exceed `±160`.

`b`

A `<number>` between `-125` and `125`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value represents the output color's distance along the `b` axis, which defines how blue (moving towards `-125`) or yellow (moving towards `+125`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±125` (`±100%`) limits. In practice, values cannot exceed `±160`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

> **Note:** Usually when percentage values have a numeric equivalent in CSS, `100%` is equal to the number `1`. This is not always the case for Lab's lightness and `a` and `b` axes, as mentioned above. With `L`, the range is from 0 to 100, with `100%` equal to `100`. The `a` and `b` values support both negative and positive values, with `100%` being equal to `125` and `-100%` being equal to `-125`.

Defining relative color output channel components

When using relative color syntax inside a `lab()` function, the browser converts the origin color into an equivalent Lab color (if it is not already specified as such). The color is defined as three distinct color channel values — `l` (lightness), `a` (green/red axis), and `b` (blue/yellow axis) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `l` channel value is resolved to a `<number>` between `0` and `100`, inclusive.
- The `a` and `b` channels are each resolved to a `<number>` between `-125` and `125`, inclusive.
- The `alpha` channel is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `lab()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `red`). The following function outputs the same color as the origin color — it uses the origin color's `l`, `a`, and `b` channel values (`54.29`, `80.8198`, and `69.8997`) as the output channel values:

```
lab(from hsl(0 100% 50%) l a b)
```

This function's output color is `lab(54.29 80.8198 69.8997)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
lab(from hsl(0 100% 50%) 29.692% 44.89% -29.034%)
```

In the above case, the output color is `lab(29.692 56.1125 -36.2925)`.

The following function creates a relative color based on the origin color:

```
lab(from hsl(0 100% 50%) l -100 b)
```

This example:

- Converts the `hsl()` origin color to an equivalent `lab()` color — `lab(54.29 80.8198 69.8997)`.
- Sets the `l` and `b` channel values for the output color to those of the origin `lab()` equivalent's `L` and `b` channel values — those values are `54.29` and `69.8997`, respectively.
- Sets the output color's `a` channel value to a new value not based on the origin color: `-100`.

The final output color is `lab(54.29 -100 69.8997)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
lab(from hsl(0 100% 50% / 0.8) l a b / alpha)
/* Computed output color: lab(54.29 80.8198 69.8997 / 0.8) */

lab(from hsl(0 100% 50% / 0.8) l a b / 0.5)
/* Computed output color: lab(54.29 80.8198 69.8997 / 0.5) */
```

In the following example, the `hsl()` origin color is again converted to the `lab()` equivalent — `lab(54.29 80.8198 69.8997)`. `calc()` calculations are applied to the `L`, `a`, `b`, and `A` values, resulting in an output color of `lab(74.29 60.8198 29.8997 / 0.9)`:

```
lab(from hsl(0 100% 50%) calc(l + 20) calc(a - 20) calc(b - 40) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<lab()> =
  lab( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adjusting lightness

The following example shows the effect of varying the lightness value of the `lab()` function.

HTML

```
<div data-color="red-dark"></div>
<div data-color="red"></div>
<div data-color="red-light"></div>

<div data-color="green-dark"></div>
<div data-color="green"></div>
<div data-color="green-light"></div>

<div data-color="blue-dark"></div>
<div data-color="blue"></div>
<div data-color="blue-light"></div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
div {
  height: 50px;
  flex: 0 0 28%;
  border: 1px solid black;
}
```

```
[data-color="red-dark"] {
  background-color: lab(5 125 71);
}
[data-color="red"] {
  background-color: lab(40 125 71);
}
[data-color="red-light"] {
```

```
  background-color: lab(95 125 71);
}

[data-color="green-dark"] {
  background-color: lab(10% -120 125);
}
[data-color="green"] {
  background-color: lab(50% -120 125);
}
[data-color="green-light"] {
  background-color: lab(90% -120 125);
}

[data-color="blue-dark"] {
  background-color: lab(10 -120 -120);
}
[data-color="blue"] {
  background-color: lab(50 -120 -120);
}
[data-color="blue-light"] {
  background-color: lab(90 -120 -120);
}
```

Result

Adjusting color axes

This example demonstrates the effects of setting the `a` and `b` values of the `lab()` function to the ends and midpoints of the a-axis, which goes from green (-125) to red (125) and the b-axis, which goes from yellow (-125) to blue (125).

HTML

```
<div data-color="red-yellow"></div>
<div data-color="red-zero"></div>
<div data-color="red-blue"></div>

<div data-color="zero-yellow"></div>
<div data-color="zero-zero"></div>
<div data-color="zero-blue"></div>

<div data-color="green-yellow"></div>
<div data-color="green-zero"></div>
<div data-color="green-blue"></div>
```

CSS

Using the CSS `background-color` property, we vary the `a` and `b` values of the `lab()` color function along the a-axis and b-axis, showing the effects of maximum, midpoint, and minimum values in each case.

```
body {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
div {
  height: 50px;
  flex: 0 0 28%;
  border: 1px solid black;
}
```

```
/* a-axis max, variable b-axis */
[data-color="red-yellow"] {
  background-color: lab(50 125 125);
}
[data-color="red-zero"] {
  background-color: lab(50 125 0);
}
[data-color="red-blue"] {
  background-color: lab(50 125 -125);
}

/* a-axis center, variable b-axis */
[data-color="zero-yellow"] {
  background-color: lab(50 0 125);
}
[data-color="zero-zero"] {
  background-color: lab(50 0 0);
}
[data-color="zero-blue"] {
  background-color: lab(50 0 -125);
}

/* a-axis min, variable b-axis */
[data-color="green-yellow"] {
  background-color: lab(50 -125 125);
}
[data-color="green-zero"] {
  background-color: lab(50 -125 0);
}
[data-color="green-blue"] {
  background-color: lab(50 -125 -125);
}
```

Result

The left column is at the yellow end (-125) of the b-axis and the right column is at the blue end (125). The top row displays colors at the red end of the a-axis (-125) and the bottom row is at the green end (125). The middle column and row are at the midpoints (0) of each axis, with the middle cell being gray; it contains no red, green, yellow, or blue, with a `0` value for both axes.

Linear gradients along the a-axis and b-axis

This example includes linear gradients to demonstrate the progression of values of the `lab()` function along the a-axis (from red to green) and along the b-axis (from yellow to blue). In each gradient image, one axis remains static while the other axis progresses from the low end to the high end of the axis values.

```
<div data-color="red-to-green-yellow">
  <span>red</span><span>`b`= -125 (yellow)</span><span>green</span>
</div>
<div data-color="red-to-green-zero">
  <span>red</span><span>no yellow or blue</span><span>green</span>
</div>
<div data-color="red-to-green-blue">
  <span>red</span><span>`b`= 125 (blue)</span><span>green</span>
</div>

<div data-color="yellow-to-blue-red">
  <span>yellow</span><span>`a` = -125 (red)</span><span>blue</span>
</div>
<div data-color="yellow-to-blue-zero">
  <span>yellow</span><span>no red or green</span><span>blue</span>
</div>
<div data-color="yellow-to-blue-green">
  <span>yellow</span><span>`a` = 125 (green)</span><span>blue</span>
</div>
```

CSS

```
div {
  height: 50px;
  padding: 5px;
  margin: 5px;
  border: 1px solid black;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
span {
  background-color: #ffffffcc;
  padding: 3px;
}
```

```
/* a-axis gradients */
[data-color="red-to-green-yellow"] {
  background-image: linear-gradient(to right, lab(50 125 125), lab(50 -125 125));
}
[data-color="red-to-green-zero"] {
  background-image: linear-gradient(to right, lab(50 125 0), lab(50 -125 0));
}
[data-color="red-to-green-blue"] {
  background-image: linear-gradient(to right, lab(50 125 -125), lab(50 -125 -125));
}

/* b-axis gradients */
[data-color="yellow-to-blue-red"] {
  background-image: linear-gradient(to right, lab(50 125 125), lab(50 125 -125));
}
[data-color="yellow-to-blue-zero"] {
  background-image: linear-gradient(to right, lab(50 0 125), lab(50 0 -125));
}
[data-color="yellow-to-blue-green"] {
  background-image: linear-gradient(to right, lab(50 -125 125),lab(50 -125 -125));
}
```

Result

Adjusting opacity

The following example shows the effect of varying the `A` (alpha) value of the `lab()` functional notation. The `red` and `red-alpha` elements overlap the `#background-div` element to demonstrate the effect of opacity. Giving `A` a value of `0.4` makes the color 40% opaque.

HTML

```
<div id="background-div">
  <div data-color="red"></div>
  <div data-color="red-alpha"></div>
</div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
#background-div {
  background-color: lightblue;
  width: 150px;
  height: 40px;
}
```

```
[data-color="red"] {
  background-color: lab(80 125 125);
}
[data-color="red-alpha"] {
  background-color: lab(80 125 125 / 0.4);
}
```

Result

Using relative colors with lab()

This example styles three `<div>` elements with different background colors. The middle one is given the unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into a `lab()` function, and the output colors have their lightness channel modified to achieve the desired effect via a `calc()` function. The lightened color has 15% added to the lightness channel, and the darkened color has 15% subtracted from the lightness channel.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
  /* equivalent of lab(75 24 79) */
}

#one {
  background-color: lab(from var(--base-color) calc(l + 15) a b);
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: lab(from var(--base-color) calc(l - 15) a b);
}
```

Result

The output is as follows:

See also

- `<color>` data type
- `<color-function>` data type
- Using relative colors
- CSS colors module
- LCH colors in CSS: what, why, and how? by Lea Verou (2020)
- Safari Technology Preview 122 release notes: includes `lab()` and `lch()` colors

lch() CSS function

The **`lch()`** functional notation expresses a given color using the LCH color space, which represents lightness, chroma, and hue. It uses the same `L` axis as the `lab()` color function of the CIELab color space, but it uses the polar coordinates `C` (Chroma) and `H` (Hue).

Syntax

```
/* Absolute values */
lch(29.2345% 44.2 27);
lch(52.2345% 72.2 56.2);
lch(52.2345% 72.2 56.2 / .5);

/* Relative values */
lch(from green l c h / 0.5)
lch(from #123456 calc(l + 10) c h)
lch(from hsl(180 100% 50%) calc(l - 10) c h)
lch(from var(--color-value) l c h / calc(alpha - 0.1))
```

Values

Below are descriptions of the allowed values for both absolute and relative colors.

> **Note:** Usually when percentage values have a numeric equivalent in CSS, `100%` is equal to the number `1`. This is not the case for `lch()`. Here `100%` is equal to the number `100` for the `L` value and `150` for the `C` value.

Absolute value syntax

```
lch(L C H[ / A])
```

The parameters are as follows:

`L`

A `<number>` between `0` and `100`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%`). The number `0` corresponds to `0%` (black), and the number `100` corresponds to `100%` (white). This value specifies the color's brightness in the CIELab color space.

> **Note:** The `L` in `lch()` is the perceived lightness, which refers to the "brightness" we visually perceive with our eyes. This is different from the `L` in `hsl()`, where it represents lightness as compared to other colors.

`C`

A `<number>`, a `<percentage>`, or the keyword `none` (equivalent to `0%` in this case). This value is a measure of the color's chroma (roughly representing the "amount of color"). Its minimum useful value is `0%`, or `0`, while its maximum is theoretically unbounded (but in practice does not exceed `230`), with `100%` being equivalent to `150`.

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg`) representing the color's `<hue>` angle.

> **Note:** The angles corresponding to particular hues differ across the sRGB (used by `hsl()` and `hwb()`), CIELAB (used by `lch()`), and Oklab (used by `oklch()`) color spaces. See the hues in LCH example below and the `<hue>` reference page for more detail and examples.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

Relative value syntax

```
lch(from <color> L C H[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**. This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`L`

A `<number>` between `0` and `100`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%`). The number `0` corresponds to `0%` (black), and the number `100` corresponds to `100%` (white). This value specifies the color's brightness in the CIELab color space.

`C`

A `<number>`, a `<percentage>`, or the keyword `none` (equivalent to `0%` in this case). This value represents the output color's chroma value (roughly representing the "amount of color"). Its minimum useful value is `0%`, or `0`, while its maximum is theoretically unbounded (but in practice does not exceed `230`), with `100%` being equivalent to `150`.

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg`) representing the output color's `<hue>` angle. See the hue example below.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

Defining relative color output channel components

When using relative color syntax inside an `lch()` function, the browser converts the origin color into an equivalent Lch color (if it is not already specified as such) The color is defined as three distinct color channel values — `l` (lightness), `c` (chroma), and `h` (hue) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `l` channel value is resolved to a `<number>` between `0` and `100`, inclusive.
- The `c` channel value is resolved to a `<number>` between `0` and `150`, inclusive.
- The `h` channel value is resolved to a `<number>` between `0` and `360`, inclusive.
- The `alpha` channel is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `lch()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `red`). The following function outputs the same color as the origin color — it uses the origin color's `l`, `c`, and `h` channel values (`54.29`, `106.854`, and `40.856`) as the output channel values:

```
lch(from hsl(0 100% 50%) l c h)
```

This function's output color is `lch(54.29 106.854 40.856)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
lch(from hsl(0 100% 50%) 29.6871% 66.83 327.109)
```

In the above case, the output color is `lch(29.6871 66.83 327.109)`.

The following function creates a relative color based on the origin color:

```
lch(from hsl(0 100% 50%) 70 150 h)
```

This example:

- Converts the `hsl()` origin color to an equivalent `lch()` color — `lch(54.29 106.854 40.856)`.
- Sets the `H` channel value for the output color to that of the origin `lch()` equivalent's `H` channel value — `40.856`.
- Sets the output color's `L` and `C` channel values to new values not based on the origin color: `70` and `150` respectively.

The final output color is `lch(70 150 40.856)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
lch(from hsl(0 100% 50% / 0.8) l c h / alpha)
/* Computed output color: lch(54.29 106.854 40.856 / 0.8) */

lch(from hsl(0 100% 50% / 0.8) l c h / 0.5)
/* Computed output color: lch(54.29 106.854 40.856 / 0.5) */
```

In the following example, the `hsl()` origin color is again converted to the `lch()` equivalent — `lch(54.29 106.854 40.856)`. `calc()` calculations are applied to the `L`, `C`, `H`, and `A` values, resulting in an output color of `lch(74.29 86.8541 0.856018 / 0.9)`:

```
lch(from hsl(0 100% 50%) calc(l + 20) calc(c - 20) calc(h - 40) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<lch()> =
  lch( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <hue> | none ] [ / [ <alpha-value> | none ] ]? )

<hue> =
  <number>  |
  <angle>

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adjusting the brightness of a color

This example shows the effect of varying the `L` (lightness) value of the `lch()` functional notation.

HTML

```
<div data-color="blue-dark"></div>
<div data-color="blue"></div>
<div data-color="blue-light"></div>

<div data-color="red-dark"></div>
<div data-color="red"></div>
<div data-color="red-light"></div>

<div data-color="green-dark"></div>
<div data-color="green"></div>
<div data-color="green-light"></div>
```

CSS

```
body {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
div {
  height: 50px;
  border: 1px solid black;
}
```

```
[data-color="blue-dark"] {
  background-color: lch(10% 100 240);
}
[data-color="blue"] {
  background-color: lch(50% 100 240);
}
[data-color="blue-light"] {
  background-color: lch(90% 100 240);
}

[data-color="red-dark"] {
  background-color: lch(10% 130 20);
}
[data-color="red"] {
  background-color: lch(50% 130 20);
}
[data-color="red-light"] {
  background-color: lch(90% 130 20);
}

[data-color="green-dark"] {
  background-color: lch(10% 132 130);
}
[data-color="green"] {
```

```
  background-color: lch(50% 132 130);
}
[data-color="green-light"] {
  background-color: lch(90% 132 130);
}
```

Result

Adjusting color intensity via chroma

The following example shows the effect of varying the `C` (chroma) value of the `lch()` functional notation, with colors decreasing in intensity as the `C` value decreases from fully saturated to almost gray.

HTML

```
<div data-color="blue"></div>
<div data-color="blue-chroma1"></div>
<div data-color="blue-chroma2"></div>
<div data-color="blue-chroma3"></div>

<div data-color="red"></div>
<div data-color="red-chroma1"></div>
<div data-color="red-chroma2"></div>
<div data-color="red-chroma3"></div>

<div data-color="green"></div>
<div data-color="green-chroma1"></div>
<div data-color="green-chroma2"></div>
<div data-color="green-chroma3"></div>
```

CSS

With the initial starting colors blue, red, and green, we declare progressively smaller values for chroma on them: starting from full color saturation at the highest value of `150` (equivalent to `100%`) down to `3` (equivalent to `2%`), which is almost gray for all the colors.

```
body {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr;
  gap: 20px;
}
div {
  height: 50px;
  border: 1px solid black;
}
```

```
[data-color="blue"] {
  background-color: lch(50% 150 240);
}
[data-color="blue-chroma1"] {
  background-color: lch(50% 105 240);
}
[data-color="blue-chroma2"] {
  background-color: lch(50% 54 240);
}
[data-color="blue-chroma3"] {
  background-color: lch(50% 3 240);
}

[data-color="red"] {
  background-color: lch(50% 100% 20deg);
}
[data-color="red-chroma1"] {
  background-color: lch(50% 70% 20deg);
}
[data-color="red-chroma2"] {
  background-color: lch(50% 36% 20deg);
}
[data-color="red-chroma3"] {
  background-color: lch(50% 2% 20deg);
}

[data-color="green"] {
  background-color: lch(50% 150 130);
}
[data-color="green-chroma1"] {
  background-color: lch(50% 105 130);
}
[data-color="green-chroma2"] {
  background-color: lch(50% 54 130);
}
[data-color="green-chroma3"] {
  background-color: lch(50% 3 130);
}
```

Result

If we had used `0` instead of `3` and `2%`, with the same lightness values, the colors would have all been the same shade of gray. In this example, they are almost gray.

Hues in LCH

The following example shows swatches with different `H` (hue) values of the `lch()` functional notation.

HTML

```
<div data-color="0">0deg</div>
<div data-color="20">20deg</div>
<div data-color="40">40deg</div>
<div data-color="60">60deg</div>
```

and so on.

```
<div data-color="80">80deg</div>
<div data-color="100">100deg</div>
<div data-color="120">120deg</div>
<div data-color="140">140deg</div>
<div data-color="160">160deg</div>
<div data-color="180">180deg</div>
<div data-color="200">200deg</div>
<div data-color="220">220deg</div>
<div data-color="240">240deg</div>
<div data-color="260">260deg</div>
<div data-color="280">280deg</div>
<div data-color="300">300deg</div>
<div data-color="320">320deg</div>
<div data-color="340">340deg</div>
<div data-color="360">360deg</div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
  gap: 3px;
}
div {
  flex: 0 0 4em;
  text-align: center;
  line-height: 4em;
  display: inline-block;
  border: 1px solid black;
  color: white;
  font-family: monospace;
}
```

```
[data-color="0"] {
  background-color: lch(50% 150 0deg);
}
[data-color="20"] {
  background-color: lch(50% 150 20deg);
}
[data-color="40"] {
  background-color: lch(50% 150 40deg);
}
[data-color="60"] {
  background-color: lch(50% 150 60deg);
}
```

and so on.

```
[data-color="80"] {
  background-color: lch(50% 150 80deg);
}
[data-color="100"] {
  background-color: lch(50% 150 100deg);
}
[data-color="120"] {
  background-color: lch(50% 150 120deg);
}
[data-color="140"] {
  background-color: lch(50% 150 140deg);
}
[data-color="160"] {
  background-color: lch(50% 150 160deg);
}
[data-color="180"] {
  background-color: lch(50% 150 180deg);
}
[data-color="200"] {
  background-color: lch(50% 150 200deg);
}
[data-color="220"] {
  background-color: lch(50% 150 220deg);
}
[data-color="240"] {
  background-color: lch(50% 150 240deg);
}
[data-color="260"] {
  background-color: lch(50% 150 260deg);
}
[data-color="280"] {
  background-color: lch(50% 150 280deg);
}
[data-color="300"] {
  background-color: lch(50% 150 300deg);
}
[data-color="320"] {
  background-color: lch(50% 150 320deg);
}
[data-color="340"] {
  background-color: lch(50% 150 340deg);
}
[data-color="360"] {
  background-color: lch(50% 150 360deg);
}
```

Result

The hue angles in `lch()` are different from those in `hsl()`. See `<hue>` for more information. In `hsl()`, the sRGB color `0deg` represents red. However, in the CIELab color space, `0deg` corresponds to magenta, while red is approximately `41deg`.

Adjusting opacity with lch()

The following example shows the effect of varying the `A` (alpha) value of the `lch()` functional notation. The `red` and `red-alpha` elements overlap the `#background-div` element to demonstrate the effect of opacity. Giving `A` a value of `0.4` makes the color 40% opaque.

HTML

```
<div id="background-div">
  <div data-color="red"></div>
  <div data-color="red-alpha"></div>
</div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
#background-div {
  background-color: lch(100% 100 240);
  width: 150px;
  height: 40px;
}
```

```
[data-color="red"] {
  background-color: lch(50% 130 20);
}
[data-color="red-alpha"] {
  background-color: lch(50% 130 20 / 0.4);
}
```

Result

Using relative colors with lch()

This example styles three `<div>` elements with different background colors, demonstrating the use of relative colors to change a color's brightness using the `lch()` color function. The middle `<div>` retains the original `--base-color`, while the left and right `<div>`s are given lightened and darkened variants of the `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into an `lch()` function, and the output colors have their lightness channel modified to achieve the desired effect via a `calc()` function. The lightened color has 15% added to the lightness channel, and the darkened color has 15% subtracted from the lightness channel.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

#one {
  background-color: lch(from var(--base-color) calc(l + 15) c h);
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: lch(from var(--base-color) calc(l - 15) c h);
}
```

Result

The output is as follows:

See also

- List of all color notations
- Using relative colors
- CSS colors module
- `<hue>` data type
- LCH colors in CSS: what, why, and how? by Lea Verou (2020)

light-dark() CSS function

The **`light-dark()`** CSS function accepts two colors or two images and returns a color or an image based on the active color scheme, without needing a `prefers-color-scheme` media feature.

Syntax

```
/* Named color values */
light-dark(
  black,
  white
);

/* RGB color values */
light-dark(
  rgb(0 0 0),
  rgb(255 255 255)
);

/* image url values */
light-dark(
  url("light-icon.png"),
  url("dark-icon.png")
);

/* linear-gradient values */
light-dark(
  linear-gradient(135deg, ghostwhite 20%, tomato),
  linear-gradient(45deg, darkslategray 20%, gold)
);

/* Custom properties */
light-dark(
  var(--light),
  var(--dark)
);
```

Values

The light-dark() function has two forms:

- Accepts two `<color>` values:

  `<color>` (light)

  > The `<color>` value used when the `color-scheme` is `light` or no preference is set.

  `<color>` (dark)

  > The `<color>` value used when the `color-scheme` is `dark`.

- Accepts two `<image>` values:

  `<image>` (light)

  > The `<image>` value used when the `color-scheme` is `light` or no preference is set.

  `<image>` (dark)

  > The `<image>` value used when the `color-scheme` is `dark`.

  `none`

  > The `none` keyword produces a fully transparent image with no natural size.

Description

Users can indicate their color scheme preference through their operating system settings (e.g., light or dark mode) or their user agent settings. The `light-dark()` function enables providing either two color values where any `<color>` value is accepted or two image values where any `<image>` value is accepted. The `light-dark()` function returns the first value if the used color scheme is `light` or if no preference is set and the second value if the used color scheme is `dark`.

To enable support for the `light-dark()` color function, the `color-scheme` must have a value of `light dark`, usually set on the `:root` pseudo-class.

```
:root {
  color-scheme: light dark;
}
body {
  color: light-dark(#333b3c, #efefec);
  background-color: light-dark(#efedea, #223a2c);
}
```

Formal syntax

```
<light-dark()> =
  <light-dark-color>  |
  <light-dark-image>

<light-dark-color> =
  light-dark( <color> , <color> )

<light-dark-image> =
  light-dark( [ <image> | none ] , [ <image> | none ] )

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )
```

```
<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>      |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Color Module Level 5, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting colors based on a color scheme

By default, in supporting browsers, the color returned by the `light-dark()` color function depends on the user preference set through an operating system's settings (e.g., light or dark mode) or from a user agent setting. You can also change this setting in the browser's developer tools.

HTML

We include three sections to enable targeting light colors, dark colors, and the colors selected based on the user's preferred color scheme.

```
<h1><code>light-dark()</code> CSS function</h1>
<section>
  <h2>Automatic</h2>
  <p>This section will react to the users system or user agent setting.</p>
</section>
<section class="light">
  <h2>Light</h2>
  <p>
    This section will be light due to the <code>color-scheme: light;</code>.
  </p>
</section>
<section class="dark">
  <h2>Dark</h2>
  <p>This section will be dark due to the <code>color-scheme: dark;</code>.</p>
</section>
```

CSS

We include colors for both light and dark themes. We also define `color-scheme` for the document on the `:root` to enable the `light-dark()` color function for the entire document.

```
:root {
  /* this has to be set to switch between light or dark */
  color-scheme: light dark;

  --light-bg: ghostwhite;
  --light-color: darkslategray;
  --light-code: tomato;

  --dark-bg: darkslategray;
  --dark-color: ghostwhite;
  --dark-code: gold;
}
* {
  background-color: light-dark(var(--light-bg), var(--dark-bg));
  color: light-dark(var(--light-color), var(--dark-color));
}
code {
  color: light-dark(var(--light-code), var(--dark-code));
}
```

In addition to enabling the `light-dark()` function, the `color-scheme` property enables overriding a user's color scheme for document sections. Forcing a page section to only use a light or dark color scheme can be done by setting the `color-scheme` property to `light` or `dark`.

> **Note:** Generally this should not be done, we are using it here for demonstration purposes. If the user has made a preference, you generally should not override their preferences.

```
.light {
  /* forces light color-scheme */
  color-scheme: light;
}
.dark {
  /* forces dark color-scheme */
  color-scheme: dark;
}
```

```
section {
  padding: 0.8rem;
}
```

Result

Setting gradient background based on a color scheme

This example uses the same HTML code as the previous example but includes a `<div>` instead of a `<p>`.

```
<h1><code>light-dark()</code> CSS function with images</h1>
<p class="supports">
  Your browser does not support <code>light-dark()</code> with images.
</p>
<div class="wrapper">
  <section>
    <h2>Automatic</h2>
    <div></div>
  </section>
  <section class="light">
    <h2>Light</h2>
    <div></div>
  </section>
  <section class="dark">
    <h2>Dark</h2>
    <div></div>
  </section>
</div>
```

CSS

```
:root {
  /* this has to be set to switch between light or dark */
  color-scheme: light dark;

  --light-bg: ghostwhite;
  --light-color: darkslategray;
  --light-code: tomato;

  --dark-bg: darkslategray;
  --dark-color: ghostwhite;
  --dark-code: gold;
}
* {
  background-color: light-dark(var(--light-bg), var(--dark-bg));
  color: light-dark(var(--light-color), var(--dark-color));
}
.wrapper {
  display: flex;
  justify-content: space-around;
  padding: 0.8rem;
}
.light {
  /* forces light color-scheme */
  color-scheme: light;
}
.dark {
  /* forces dark color-scheme */
  color-scheme: dark;
}
section {
  width: 25%;
  padding: 5px;
  color: light-dark(
    var(--light-code),
    var(--dark-code)
  );
  border: 2px solid light-dark(var(--light-code), var(--dark-code));
}
@supports (background-image: light-dark(url("light.png"), url("dark.png"))) {
  .supports {display:none;}
}
```

Firstly, we define light and dark `linear-gradient()` values as custom properties.

```
:root {
  /*  light dark gradients  */
  --light-grad: linear-gradient(135deg, var(--light-bg) 20%, var(--light-code));
  --dark-grad: linear-gradient(45deg, var(--dark-bg) 30%, var(--dark-code));
}
```

```
section div {
  width: 80%;
  aspect-ratio: 1/1;
  margin: auto;
  border: 1px solid light-dark(var(--light-code), var(--dark-code));
}
@supports not (
  background-image: light-dark(url("light.png"), url("dark.png"))
) {
  section div {
    width: 60%;
  }
}
```

Then we use the custom properties with `light-dark()` to set the `background-image` property based on the active color scheme.

```
section div {
  background-image: light-dark(
    var(--light-grad),
    var(--dark-grad)
  );
}
```

Result

See also

- `color-scheme`
- `<color>`

- `<image>`
- CSS colors module
- `prefers-contrast` `@media` feature
- `contrast()`
- WCAG: color contrast
- CSS custom properties and `var()`

oklab() CSS function

The **`oklab()`** functional notation expresses a given color in the Oklab color space, which attempts to mimic how color is perceived by the human eye.

Oklab is a perceptual color space and is useful to:

- Transform an image to grayscale, without changing its lightness.
- Modify the saturation of colors, while keeping user perception of hue and lightness
- Create smooth and uniform gradients of colors (when interpolated manually, for example, in a `<canvas>` element).

`oklab()` works with a Cartesian coordinate system on the Oklab color space — a- and b-axes. It can represent a wider range of colors than RGB, including wide-gamut and P3 colors. If you want a polar color system, chroma and hue, use `oklch()`.

Syntax

```
/* Absolute values */
oklab(40.1% 0.1143 0.045);
oklab(59.69% 0.1007 0.1191);
oklab(59.69% 0.1007 0.1191 / 0.5);

/* Relative values */
oklab(from green l a b / 0.5)
oklab(from #123456 calc(l + 0.1) a b / calc(alpha * 0.9))
oklab(from hsl(180 100% 50%) calc(l - 0.1) a b)
```

Values

Below are descriptions of the allowed values for both absolute and relative colors.

Absolute value syntax

```
oklab(L a b[ / A])
```

The parameters are as follows:

`L`

A `<number>` between `0` and `1`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's perceived lightness. The number `0` corresponds to `0%` (black) and the number `1` corresponds to `100%` (white).

`a`

A `<number>` between `-0.4` and `0.4`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's distance along the `a` axis in the Oklab color space, which defines how green (moving towards `-0.4`) or red (moving towards `+0.4`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±0.4` (`±100%`) limits. In practice, values cannot exceed `±0.5`.

`b`

A `<number>` between `-0.4` and `0.4`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value specifies the color's distance along the `b` axis in the Oklab color space, which defines how blue (moving towards `-0.4`) or yellow (moving towards `+0.4`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±0.4` (`±100%`) limits. In practice, values cannot exceed `±0.5`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

Relative value syntax

```
oklab(from <color> L a b[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**. This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`L`

A `<number>` between `0` and `1`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value represents the lightness of the output color. The number `0` corresponds to `0%` (black) and the number `1` corresponds to `100%` (white).

`a`

A `<number>` between `-0.4` and `0.4`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value represents the output color's distance along the `a` axis in the Oklab color space, which defines how green (moving towards `-0.4`) or red (moving towards `+0.4`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±0.4` (`±100%`) limits. In practice, values cannot exceed `±0.5`.

`b`

A `<number>` between `-0.4` and `0.4`, a `<percentage>` between `-100%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This value represents the output color's distance along the `b` axis in the Oklab color space, which defines how blue (moving towards `-0.4`) or yellow (moving towards `+0.4`) the color is. Note that these values are signed (allowing both positive and negative values) and theoretically unbounded, meaning that you can set values outside the `±0.4` (`±100%`) limits. In practice, values cannot exceed `±0.5`.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

Defining relative color output channel components

When using relative color syntax inside an `oklab()` function, the browser converts the origin color into an equivalent Oklab color (if it is not already specified as such). The color defined as three distinct color channel values — `l` (lightness), `a` (green/red axis), and `b` (blue/yellow axis) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `l` channel value is resolved to a `<number>` between `0` and `1`, inclusive.
- The `a` and `b` channels are each resolved to a `<number>` between `-0.4` and `0.4`, inclusive.
- The `alpha` channel is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `oklab()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `red`). The following function outputs the same color as the origin color — it uses the origin color's `l`, `a`, and `b` channel values (`0.627966`, `0.22488`, and `0.125859`) as the output channel values:

```
oklab(from hsl(0 100% 50%) l a b)
```

This function's output color is `oklab(0.627966 0.22488 0.125859)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
oklab(from hsl(0 100% 50%) 42.1% 0.165 -0.101)
```

In the above case, the output color is `oklab(0.421 0.165 -0.101)`.

The following function creates a relative color based on the origin color:

```
oklab(from hsl(0 100% 50%) l -0.3 b)
```

This example:

- Converts the `hsl()` origin color to an equivalent `oklab()` color — `oklab(0.627966 0.22488 0.125859)`.
- Sets the `L` and `b` channel values for the output color to those of the origin `oklab()` equivalent's `L` and `b` channel values — those values are `0.627966` and `0.125859`, respectively.
- Sets the output color's `a` channel value to a new value not based on the origin color: `-0.3`.

The final output color is `oklab(0.627966 -0.3 0.125859)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
oklab(from hsl(0 100% 50% / 0.8) l a b / alpha)
/* Computed output color: oklab(0.627966 0.22488 0.125859 / 0.8) */

oklab(from hsl(0 100% 50% / 0.8) l a b / 0.5)
/* Computed output color: oklab(0.627966 0.22488 0.125859 / 0.5) */
```

In the following example, the `hsl()` origin color is again converted to the `oklab()` equivalent — `oklab(0.627966 0.22488 0.125859)`. `calc()` calculations are applied to the `L`, `a`, `b`, and `A` values, resulting in an output color of `oklab(0.827966 0.14488 -0.0741406 / 0.9)`:

```
oklab(from hsl(0 100% 50%) calc(l + 0.2) calc(a - 0.08) calc(b - 0.2) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<oklab()> =
  oklab( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ / [ <alpha-value> | none ] ]? )

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adjusting lightness

The following example shows the effect of varying the lightness, a-axis, and b-axis values of the `oklab()` function.

HTML

```
<div data-color="red-dark"></div>
<div data-color="red"></div>
<div data-color="red-light"></div>

<div data-color="green-dark"></div>
<div data-color="green"></div>
```

```
<div data-color="green-light"></div>

<div data-color="blue-dark"></div>
<div data-color="blue"></div>
<div data-color="blue-light"></div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
div {
  height: 50px;
  flex: 0 0 28%;
  border: 1px solid black;
}
```

```
[data-color="red-dark"] {
  background-color: oklab(0.05 0.4 0.4);
}
[data-color="red"] {
  background-color: oklab(0.5 0.4 0.4);
}
[data-color="red-light"] {
  background-color: oklab(0.95 0.4 0.4);
}

[data-color="green-dark"] {
  background-color: oklab(5% -100% 0.4);
}
[data-color="green"] {
  background-color: oklab(50% -100% 0.4);
}
[data-color="green-light"] {
  background-color: oklab(95% -100% 0.4);
}

[data-color="blue-dark"] {
  background-color: oklab(0.05 -0.3 -0.4);
}
[data-color="blue"] {
  background-color: oklab(0.5 -0.3 -0.4);
}
[data-color="blue-light"] {
  background-color: oklab(0.95 -0.3 -0.4);
}
```

Result

Adjusting opacity

The following example shows the effect of varying the `A` (alpha) value of the `oklab()` function. The `red` and `red-alpha` elements overlap the `#background-div` element to demonstrate the effect of opacity. Giving the `red-alpha` element an opacity of `0.4` makes it appear more transparent than the `red` element.

HTML

```
<div id="background-div">
  <div data-color="red"></div>
  <div data-color="red-alpha"></div>
</div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
#background-div {
  background-color: lightblue;
  width: 150px;
  height: 40px;
}
```

```
[data-color="red"] {
  background-color: oklab(0.628 0.225 0.126);
}
[data-color="red-alpha"] {
  background-color: oklab(0.628 0.225 0.126 / 0.4);
}
```

Result

Adjusting color axes

This example demonstrates the effects of setting the `a` and `b` values of the `oklab()` function to the ends and midpoints of the a-axis and b-axis, respectively. The a-axis goes from green (`-0.4`) to red (`0.4`) and the b-axis goes from yellow (`-0.4`) to blue (`0.4`).

HTML

```
<div data-color="red-yellow"></div>
<div data-color="red-zero"></div>
<div data-color="red-blue"></div>

<div data-color="zero-yellow"></div>
<div data-color="zero-zero"></div>
```

```
<div data-color="zero-blue"></div>

<div data-color="green-yellow"></div>
<div data-color="green-zero"></div>
<div data-color="green-blue"></div>
```

CSS

Using the CSS `background-color` property, we vary the `a` and `b` values of the `oklab()` color function along the a-axis and b-axis, showing the effects of maximum, midpoint, and minimum values in each case.

```
body {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
div {
  height: 50px;
  flex: 0 0 28%;
  border: 1px solid black;
}
```

```
/* a-axis max, variable b-axis */
[data-color="red-yellow"] {
  background-color: oklab(0.5 0.4 0.4);
}
[data-color="red-zero"] {
  background-color: oklab(0.5 0.4 0);
}
[data-color="red-blue"] {
  background-color: oklab(0.5 0.4 -0.4);
}

/* a-axis center, variable b-axis */
[data-color="zero-yellow"] {
  background-color: oklab(0.5 0 0.4);
}
[data-color="zero-zero"] {
  background-color: oklab(0.5 0 0);
}
[data-color="zero-blue"] {
  background-color: oklab(0.5 0 -0.4);
}

/* a-axis min, variable b-axis */
[data-color="green-yellow"] {
  background-color: oklab(0.5 -0.4 0.4);
}
[data-color="green-zero"] {
  background-color: oklab(0.5 -0.4 0);
}
[data-color="green-blue"] {
  background-color: oklab(0.5 -0.4 -0.4);
}
```

Result

The left column is at the yellow end (`-0.4`) of the b-axis and the right column is at the blue end (`0.4`). The top row displays colors at the red end of the a-axis (`-0.4`) and the bottom row is at the green end (`0.4`). The middle column and row are at the midpoints of each axis, with the middle cell being gray; it contains no red, green, yellow, or blue, with a `0` value for both axes.

Linear gradients along the a-axis and b-axis

This example includes linear gradients to demonstrate the progression of values of the `oklab()` function along the a-axis (from red to green) and along the b-axis (from yellow to blue). In each gradient image, one axis remains static while the other axis progresses from low to high values.

```
<div data-color="red-to-green-yellow">
  <span>red</span><span>`b`= -0.4 (yellow)</span><span>green</span>
</div>
<div data-color="red-to-green-zero">
  <span>red</span><span>no yellow or blue</span><span>green</span>
</div>
<div data-color="red-to-green-blue">
  <span>red</span><span>`b`= 0.4 (blue)</span><span>green</span>
</div>

<div data-color="yellow-to-blue-red">
  <span>yellow</span><span>`a` = -0.4 (red)</span><span>blue</span>
</div>
<div data-color="yellow-to-blue-zero">
  <span>yellow</span><span>no red or green</span><span>blue</span>
</div>
<div data-color="yellow-to-blue-green">
  <span>yellow</span><span>`a` = 0.4 (green)</span><span>blue</span>
</div>
```

CSS

```
div {
  height: 50px;
  padding: 5px;
  margin: 5px;
  border: 1px solid black;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
span {
  background-color: #ffffffcc;
  padding: 3px;
}
```

```
/* a-axis gradients */
[data-color="red-to-green-yellow"] {
  background-image: linear-gradient(to right, oklab(50% 0.4 0.4), oklab(50% -0.4 0.4));
}
[data-color="red-to-green-zero"] {
  background-image: linear-gradient(to right, oklab(50% 0.4 0), oklab(50% -0.4 0));
}
[data-color="red-to-green-blue"] {
  background-image: linear-gradient(to right, oklab(50% 0.4 -0.4), oklab(50% -0.4 -0.4));
}

/* b-axis gradients */
[data-color="yellow-to-blue-red"] {
  background-image: linear-gradient(to right, oklab(50% 0.4 0.4), oklab(50% 0.4 -0.4));
}
[data-color="yellow-to-blue-zero"] {
  background-image: linear-gradient(to right, oklab(50% 0 0.4), oklab(50% 0 -0.4));
}
[data-color="yellow-to-blue-green"] {
  background-image: linear-gradient(to right, oklab(50% -0.4 0.4),oklab(50% -0.4 -0.4));
}
```

Result

Using relative colors with oklab()

This example styles three `<div>` elements with different background colors. The middle one is given the unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into an `oklab()` function, and the output colors have their lightness channel modified to achieve the desired effect via a `calc()` function. The lightened color has `0.15` (15%) added to the lightness channel, and the darkened color has `0.15` (15%) subtracted from the lightness channel.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

#one {
  background-color: oklab(from var(--base-color) calc(l + 0.15) a b);
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: oklab(from var(--base-color) calc(l - 0.15) a b);
}
```

Result

The output is as follows:

See also

- The `<color>` data type for a list of all color notations
- `lab()` and `oklch()` color functions
- Using relative colors
- CSS colors module
- A perceptual color space for image processing on bottosson.github.io (2023)
- OKLAB color wheel on observablehq.com

oklch() CSS function

The **`oklch()`** functional notation expresses a given color in the Oklab color space. `oklch()` is the cylindrical form of `oklab()`, using the same `L` axis, but with polar Chroma (`C`) and Hue (`H`) coordinates.

Syntax

```
/* Absolute values */
oklch(40.1% 0.123 21.57)
oklch(59.69% 0.156 49.77)
oklch(59.69% 0.156 49.77 / .5)

/* Relative values */
oklch(from green l c h / 0.5)
oklch(from #123456 calc(l + 0.1) c h)
oklch(from hsl(180 100% 50%) calc(l - 0.1) c h)
oklch(from var(--color) l c h / calc(alpha - 0.1))
```

Values

Below are descriptions of the allowed values for both absolute and relative colors.

> **Note:** Usually when percentage values have a numeric equivalent in CSS, `100%` is equal to the number `1`. This is not the case for all of the `oklch()` component values. Here, `100%` is equal to `0.4` for the `C` value.

Absolute value syntax

```
oklch(L C H[ / A])
```

The parameters are as follows:

`L`

A `<number>` between `0` and `1`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). In this case, the number `0` corresponds to `0%` (black) and the number `1` corresponds to `100%` (white). This value specifies the color's perceived lightness, or "brightness".

> **Note:** The `L` in `oklch()` is the perceived lightness, which refers to the "brightness" we visually perceive with our eyes. This is different from the `L` in `hsl()`, where it represents lightness as compared to other colors.

`C`

A `<number>`, a `<percentage>`, or the keyword `none` (equivalent to `0%` in this case). This value is a measure of the color's chroma (roughly representing the "amount of color"). Its minimum useful value is `0`, while the maximum is theoretically unbounded (but in practice does not exceed `0.5`). In this case, `0%` is `0` and `100%` is the number `0.4`.

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg` in this case) representing the color's `<hue>` angle.

> **Note:** The angles corresponding to particular hues differ across the sRGB (used by `hsl()` and `hwb()`), CIELAB (used by `lch()`), and Oklab (used by `oklch()`) color spaces. See the Hues in `oklch()` example below and the `<hue>` reference page for more details and examples.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

Relative value syntax

```
oklch(from &lt;color&gt; L C H[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**: This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`L`

A `<number>` between `0` and `1`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). This represents the lightness value of the output color. Here the number `0` corresponds to `0%` (black) and the number `1` corresponds to `100%` (white).

`C`

A `<number>`, a `<percentage>`, or the keyword `none` (equivalent to `0%` in this case). This value represents the output color's chroma value (roughly representing the "amount of color"). Its minimum useful value is `0`, while its maximum is theoretically unbounded (but in practice does not exceed `0.5`). In this case, `0%` is `0` and `100%` is the number `0.4`.

`H`

A `<number>`, an `<angle>`, or the keyword `none` (equivalent to `0deg` in this case) representing the output color's `<hue>` angle. See a sample of different hues in the Examples section below.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

Defining relative color output channel components

When using relative color syntax inside an `oklch()` function, the browser converts the origin color into an equivalent OkLCh color (if it is not already specified as such). The color is defined as three distinct color channel values — `l` (lightness), `c` (chroma), and `h` (hue) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `l` channel value is resolved to a `<number>` between `0` and `1`, inclusive.
- The `c` channel value is resolved to a `<number>` between `0` and `0.4`, inclusive.
- The `h` channel value is resolved to a `<number>` between `0` and `360`, inclusive.
- The `alpha` channel is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `oklch()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `red`). The following function outputs the same color as the origin color — it uses the origin color's `l`, `c`, and `h` channel values (`0.627966`, `0.257704`, and `29.2346`) as the output channel values:

```
oklch(from hsl(0 100% 50%) l c h)
```

This function's output color is `oklch(0.627966 0.257704 29.2346)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
oklch(from hsl(0 100% 50%) 42.1% 0.25 328.363)
```

In the above case, the output color is `oklch(0.421 0.25 328.363)`.

The following function creates a relative color based on the origin color:

```
oklch(from hsl(0 100% 50%) 0.8 0.4 h)
```

This example:

- Converts the `hsl()` origin color to an equivalent `oklch()` color — `oklch(0.627966 0.257704 29.2346)`.
- Sets the `H` channel value for the output color to that of the origin `oklch()` equivalent's `H` channel value — `29.2346`.
- Sets the output color's `L` and `C` channel values to new values not based on the origin color: `0.8` and `0.4` respectively.

The final output color is `oklch(0.8 0.4 29.2346)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
oklch(from hsl(0 100% 50% / 0.8) l c h / alpha)
/* Computed output color: oklch(0.627966 0.257704 29.2346 / 0.8) */

oklch(from hsl(0 100% 50% / 0.8) l c h / 0.5)
/* Computed output color: oklch(0.627966 0.257704 29.2346 / 0.5) */
```

In the following example, the `hsl()` origin color is again converted to the `oklch()` equivalent — `oklch(0.627966 0.257704 29.2346)`. `calc()` calculations are applied to the `L`, `C`, `H`, and `A` values, resulting in an output color of `oklch(0.827966 0.357704 9.23462 / 0.9)`:

```
oklch(from hsl(0 100% 50%) calc(l + 0.2) calc(c + 0.1) calc(h - 20) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<oklch()> =
  oklch( [ from <color> ]? [ <percentage> | <number> | none ] [ <percentage> | <number> | none ] [ <hue> | none ] [ / [ <alpha-value> | none ] ]? )

<hue> =
  <number>  |
  <angle>

<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Adjusting the brightness of a color

This example shows the effect of varying the `L` (lightness) value of the `oklch()` functional notation.

HTML

```
<div data-color="blue-dark"></div>
<div data-color="blue"></div>
<div data-color="blue-light"></div>

<div data-color="red-dark"></div>
<div data-color="red"></div>
<div data-color="red-light"></div>

<div data-color="green-dark"></div>
<div data-color="green"></div>
<div data-color="green-light"></div>
```

CSS

```
body {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
div {
  height: 50px;
  border: 1px solid black;
}
```

```
[data-color="blue-dark"] {
  background-color: oklch(10% 0.4 240);
}
[data-color="blue"] {
```

```
  background-color: oklch(50% 0.4 240);
}
[data-color="blue-light"] {
  background-color: oklch(90% 0.4 240);
}

[data-color="red-dark"] {
  background-color: oklch(10% 0.4 20);
}
[data-color="red"] {
  background-color: oklch(50% 0.4 20);
}
[data-color="red-light"] {
  background-color: oklch(90% 0.4 20);
}

[data-color="green-dark"] {
  background-color: oklch(10% 0.4 130);
}
[data-color="green"] {
  background-color: oklch(50% 0.4 130);
}
[data-color="green-light"] {
  background-color: oklch(90% 0.4 130);
}
```

Result

Adjusting color intensity via chroma

The following example shows the effect of varying the `C` (chroma) value of the `oklch()` functional notation, with colors decreasing in intensity as the `C` value decreases from fully saturated to almost gray.

HTML

```
<div data-color="blue"></div>
<div data-color="blue-chroma1"></div>
<div data-color="blue-chroma2"></div>
<div data-color="blue-chroma3"></div>

<div data-color="red"></div>
<div data-color="red-chroma1"></div>
<div data-color="red-chroma2"></div>
<div data-color="red-chroma3"></div>

<div data-color="green"></div>
<div data-color="green-chroma1"></div>
<div data-color="green-chroma2"></div>
<div data-color="green-chroma3"></div>
```

CSS

With the initial starting colors blue, red, and green, we declare progressively smaller values for chroma on them: starting from full color saturation at the high value of `0.4` (equivalent to `100%`) down to `0.01` (equivalent to `2%`), which is almost gray for all the colors.

```
body {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr;
  gap: 20px;
}
div {
  height: 50px;
  border: 1px solid black;
}
```

```
[data-color="blue"] {
  background-color: oklch(50% 0.4 240);
}
[data-color="blue-chroma1"] {
  background-color: oklch(50% 0.2 240);
}
[data-color="blue-chroma2"] {
  background-color: oklch(50% 0.1 240);
}
[data-color="blue-chroma3"] {
  background-color: oklch(50% 0.01 240);
}

[data-color="red"] {
  background-color: oklch(50% 100% 20deg);
}
[data-color="red-chroma1"] {
  background-color: oklch(50% 50% 20deg);
}
[data-color="red-chroma2"] {
  background-color: oklch(50% 25% 20deg);
}
[data-color="red-chroma3"] {
  background-color: oklch(50% 2% 20deg);
}

[data-color="green"] {
  background-color: oklch(50% 0.4 130);
}
[data-color="green-chroma1"] {
  background-color: oklch(50% 0.2 130);
}
[data-color="green-chroma2"] {
  background-color: oklch(50% 0.1 130);
}
[data-color="green-chroma3"] {
```

```
  background-color: oklch(50% 0.01 130);
}
```

Result

If we had used `0` instead of `0.01` and `2%`, with the same lightness values, the colors would have all been the same shade of gray. In this example, they are almost gray.

Hues in OkLCh

The following example shows swatches with different `H` (hue) values of the `oklch()` functional notation.

HTML

```
<div data-color="0">0deg</div>
<div data-color="20">20deg</div>
<div data-color="40">40deg</div>
<div data-color="60">60deg</div>
```

and so on.

```
<div data-color="80">80deg</div>
<div data-color="100">100deg</div>
<div data-color="120">120deg</div>
<div data-color="140">140deg</div>
<div data-color="160">160deg</div>
<div data-color="180">180deg</div>
<div data-color="200">200deg</div>
<div data-color="220">220deg</div>
<div data-color="240">240deg</div>
<div data-color="260">260deg</div>
<div data-color="280">280deg</div>
<div data-color="300">300deg</div>
<div data-color="320">320deg</div>
<div data-color="340">340deg</div>
<div data-color="360">360deg</div>
```

CSS

```
body {
  display: flex;
  flex-wrap: wrap;
  gap: 3px;
}
div {
  flex: 0 0 4em;
  text-align: center;
  line-height: 4em;
  display: inline-block;
  border: 1px solid black;
  color: white;
  font-family: monospace;
}
```

```
[data-color="0"] {
  background-color: oklch(50% 0.4 0deg);
}
[data-color="20"] {
  background-color: oklch(50% 0.4 20deg);
}
[data-color="40"] {
  background-color: oklch(50% 0.4 40deg);
}
[data-color="60"] {
  background-color: oklch(50% 0.4 60deg);
}
```

and so on.

```
[data-color="80"] {
  background-color: oklch(50% 0.4 80deg);
}
[data-color="100"] {
  background-color: oklch(50% 0.4 100deg);
}
[data-color="120"] {
  background-color: oklch(50% 0.4 120deg);
}
[data-color="140"] {
  background-color: oklch(50% 0.4 140deg);
}
[data-color="160"] {
  background-color: oklch(50% 0.4 160deg);
}
[data-color="180"] {
  background-color: oklch(50% 0.4 180deg);
}
[data-color="200"] {
  background-color: oklch(50% 0.4 200deg);
}
[data-color="220"] {
  background-color: oklch(50% 0.4 220deg);
}
[data-color="240"] {
  background-color: oklch(50% 0.4 240deg);
}
[data-color="260"] {
  background-color: oklch(50% 0.4 260deg);
}
[data-color="280"] {
  background-color: oklch(50% 0.4 280deg);
}
```

```
[data-color="300"] {
  background-color: oklch(50% 0.4 300deg);
}
[data-color="320"] {
  background-color: oklch(50% 0.4 320deg);
}
[data-color="340"] {
  background-color: oklch(50% 0.4 340deg);
}
[data-color="360"] {
  background-color: oklch(50% 0.4 360deg);
}
```

Result

The hue angles in `oklch()` are different from those in `hsl()`. See `<hue>` for more information. In `hsl()`, the sRGB color `0deg` represents red. However, in the CIELab color space, `0deg` corresponds to magenta, while red is approximately `41deg`.

Adjusting the alpha value of a color

The following example shows the effect of varying the `A` (alpha) value of the `oklch()` color function. The `red` and `red-alpha` elements overlap the `#background-div` element to demonstrate the effect of opacity. Giving `A` a value of `0.4` makes the color 40% opaque.

HTML

```
<div id="background-div">
  <div data-color="red"></div>
  <div data-color="red-alpha"></div>
</div>
```

CSS

```
div {
  width: 50px;
  height: 50px;
  padding: 5px;
  margin: 5px;
  display: inline-block;
  border: 1px solid black;
}
#background-div {
  background-color: oklch(100% 0.57 217);
  width: 150px;
  height: 40px;
}
```

```
[data-color="red"] {
  background-color: oklch(50% 0.5 20);
}
[data-color="red-alpha"] {
  background-color: oklch(50% 0.5 20 / 0.4);
}
```

Result

Using relative colors with oklch()

This example styles three `<div>` elements with different background colors. The middle one is given the unmodified `--base-color`, while the left and right ones are given lightened and darkened variants of that `--base-color`.

These variants are defined using relative colors — the `--base-color` custom property is passed into an `oklch()` function, and the output colors have their lightness channel modified to achieve the desired effect via a `calc()` function. The lightened color has `0.15` (15%) added to the lightness channel, and the darkened color has `0.15` (15%) subtracted from the lightness channel.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
}

#one {
  background-color: oklch(from var(--base-color) calc(l + 0.15) c h);
}

#two {
  background-color: var(--base-color);
}

#three {
  background-color: oklch(from var(--base-color) calc(l - 0.15) c h);
}
```

Result

The output is as follows:

See also

- List of all color notations
- Using relative colors
- CSS colors module
- `<hue>` data type
- `lch()` and `oklab()` color functions
- Interactive post on OkLCh color space (2024)
- OKLCH in CSS: why we moved from RGB and HSL (2024)
- A perceptual color space for image processing (2020)

rgb() CSS function

> **Note:** The `rgba()` functional notation is an alias for `rgb()`. They are exactly equivalent. It is recommended to use `rgb()`.

The **`rgb()`** functional notation expresses a color in the sRGB color space according to its red, green, and blue components. An optional alpha component represents the color's transparency.

Try it

```
background: rgb(31 120 50);
```

```
background: rgb(30% 20% 50%);
```

```
background: rgb(255 122 127 / 80%);
```

```
background: rgb(255 122 127 / 0.2);
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Syntax

```
/* Absolute values */
rgb(255 255 255)
rgb(255 255 255 / 50%)

/* Relative values */
rgb(from green r g b / 0.5)
rgb(from #123456 calc(r + 40) calc(g + 40) b)
rgb(from hwb(120deg 10% 20%) r g calc(b + 200))

/* Legacy 'rgba()' alias */
rgba(0 255 255)

/* Legacy format */
rgb(0, 255, 255)
rgb(0, 255, 255, 50%)
```

> **Note:** For compatibility reasons, Web API-serialized color values are expressed as `rgb()` colors if the alpha channel value is exactly 1, and `rgba()` colors otherwise. In both cases, legacy syntax is used, with commas as separators (for example `rgb(255, 0, 0)`).

Values

Below are descriptions of the allowed values for both absolute and relative colors.

Absolute value syntax

```
rgb(R G B[ / A])
```

The parameters are as follows:

`R, G, B`

Each value can be represented as a `<number>` between `0` and `255`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). These values represent the red, green, and blue channels, respectively.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to 100%. If included, the value is preceded by a slash (`/`).

> **Note:** See Missing color components for more information on the effect of `none`.

Relative value syntax

```
rgb(from &lt;color&gt; R G B[ / A])
```

The parameters are as follows:

`from <color>`

The keyword `from` is always included when defining a relative color, followed by a `<color>` value representing the **origin color**: This is the original color that the relative color is based on. The origin color can be *any* valid `<color>` syntax, including another relative color.

`R`, `G`, `B`

Each value can be represented as a `<number>` between `0` and `255`, a `<percentage>` between `0%` and `100%`, or the keyword `none` (equivalent to `0%` in this case). These values represent the red, green, and blue channel values of the output color, respectively.

`A` Optional

An `<alpha-value>` representing the alpha channel value of the output color, where the number `0` corresponds to `0%` (fully transparent) and `1` corresponds to `100%` (fully opaque). Additionally, the keyword `none` can be used to explicitly specify no alpha channel. If the `A` channel value is not explicitly specified, it defaults to the alpha channel value of the origin color. If included, the value is preceded by a slash (`/`).

> **Note:** To fully enable the representation of the full spectrum of visible colors, the output of relative `rgb()` color functions is serialized to `color(srgb)`. That means that querying the output color value via the `HTMLElement.style` property or the `CSSStyleDeclaration.getPropertyValue()` method returns the output color as a `color(srgb ...)` value.

Defining relative color output channel components

When using relative color syntax inside an `rgb()` function, the browser converts the origin color into an equivalent RGB color (if it is not already specified as such). The color is defined as three distinct color channel values — `r` (red), `g` (green), and `b` (blue) — plus an alpha channel value (`alpha`). These channel values are made available inside the function to be used when defining the output color channel values:

- The `r`, `g` and `b` values are each resolved to `<number>`s between `0` and `255`, inclusive.
- The `alpha` channel is resolved to a `<number>` between `0` and `1`, inclusive.

When defining a relative color, the different channels of the output color can be expressed in several different ways. Below, we'll study some examples to illustrate these.

In the first two examples below, we are using relative color syntax. However, the first one outputs the same color as the origin color and the second one outputs a color not based on the origin color at all. They don't really create relative colors! You'd be unlikely to ever use these in a real codebase, and would probably just use an absolute color value instead. We included these examples as a starting point for learning about relative `rgb()` syntax.

Let's start with an origin color of `hsl(0 100% 50%)` (equivalent to `rgb(255 0 0)`). The following function outputs the same color as the origin color — it uses the origin color's `r`, `g`, and `b` channel values (`255`, `0`, and `0`) as the output channel values:

```
rgb(from hsl(0 100% 50%) r g b)
```

This function's output color is the sRGB `color()` equivalent of `rgb(255 0 0)`: `color(srgb 1 0 0)`.

The next function uses absolute values for the output color's channel values, outputting a completely different color not based on the origin color:

```
rgb(from hsl(0 100% 50%) 132 132 224)
```

In the above case, the output color is the sRGB `color()` equivalent of `rgb(132 132 224)`: `color(srgb 0.517647 0.517647 0.878431)`.

The following function creates a relative color based on the origin color:

```
rgb(from hsl(0 100% 50%) r 80 80)
```

This example:

- Converts the origin color (`hsl(0 100% 50%)`) into an `rgb()` equivalent (`rgb(255 0 0)`).
- Sets the `R` channel value for the output color to the origin color `rgb()` equivalent's `R` channel value — `255`.
- Sets the output color's `G` and `B` channel values to new values not based on the origin color: `80` and `80`, respectively.

The final output color is the equivalent of `rgb(255 80 80)` in the sRGB color space — `color(srgb 1 0.313726 0.313726)`.

> **Note:** As mentioned above, if the output color is using a different color model to the origin color, the origin color is converted to the same model or space as the output color in the background so that it can be represented in a way that is compatible (i.e., using the same channels).

In the examples we've seen so far in this section, the alpha channels have not been explicitly specified for either the origin or output colors. When the output color alpha channel is not specified, it defaults to the same value as the origin color alpha channel. When the origin color alpha channel is not specified (and it is not a relative color), it defaults to `1`. Therefore, the origin and output alpha channel values are `1` for the above examples.

Let's look at some examples that specify origin and output alpha channel values. The first one specifies the output alpha channel value as being the same as the origin alpha channel value, whereas the second one specifies a different output alpha channel value, unrelated to the origin alpha channel value.

```
rgb(from hsl(0 100% 50% / 0.8) r g b / alpha)
/* Computed output color: color(srgb 1 0 0 / 0.8) */

rgb(from hsl(0 100% 50% / 0.8) r g b / 0.5)
/* Computed output color: color(srgb 1 0 0 / 0.5) */
```

In the following example, the `hsl()` origin color is again converted into an `rgb()` representation — `rgb(255 0 0)`. `calc()` calculations are applied to the `R`, `G`, `B`, and `A` values. After calculating, the R, G, B and A values are `127.5`, `25`, `175`, and `0.9` respectively. The final output color is the equivalent of `rgb(127.5 25 175 / 0.9)` in the sRGB color space: `color(srgb 0.5 0.0980392 0.686275 / 0.9)`.

```
rgb(from hsl(0 100% 50%) calc(r/2) calc(g + 25) calc(b + 175) / calc(alpha - 0.1))
```

> **Note:** Because the origin color channel values are resolved to `<number>` values, you have to add numbers to them when using them in calculations, even in cases where a channel would normally accept `<percentage>`, `<angle>`, or other value types. Adding a `<percentage>` to a `<number>`, for example, doesn't work.

Formal syntax

```
<rgb()> =
  <legacy-rgb-syntax>  |
  <modern-rgb-syntax>

<legacy-rgb-syntax> =
  rgb( <percentage>#{3} , <alpha-value>? )  |
  rgb( <number>#{3} , <alpha-value>? )

<modern-rgb-syntax> =
  rgb( [ from <color> ]? [ <number> | <percentage> | none ]{3} [ / [ <alpha-value> | none ] ]? )
```

```
<alpha-value> =
  <number>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic syntax

In this example, we have three `<div>` elements with different background colors displayed over a striped background.

HTML

```
<div>
  <div id="integer"></div>
  <div id="percent"></div>
  <div id="alpha"></div>
</div>
```

CSS

The background colors are set using the `rgb()` color function. The three colors are the same. The third is semi-transparent, so we included a `repeating-linear-gradient()` on the `<body>` to better demonstrate the transparency of alpha channels.

```
div {
  display: flex;
  gap: 20px;
  height: 50px;
  flex: 1;
}
body {
  height: calc(100vh - 20px);
}
```

```
body {
  background: repeating-linear-gradient(-45deg, #eeeeee 0 2px, white 2px 6px);
  padding: 10px;
}

#integer {
  background-color: rgb(189 85 218);
}

#percent {
  background-color: rgb(74% 33% 85%);
}

#alpha {
  background-color: rgb(189 85 218 / 0.25);
}
```

Result

Using relative colors with rgb()

This example styles three `<div>` elements with different background colors. The left-hand one is given the unmodified `--base-color`, while the middle and right ones are given variants of that `--base-color` that successively remove more from the red channel and add more to the blue channel.

These variants are defined using relative colors — the `--base-color` custom property is passed into an `rgb()` function, and the output color has its red and blue channels modified to achieve the desired effect via `calc()` functions, while the green channel is left unchanged.

```
<div id="container">
  <div class="item" id="one"></div>
  <div class="item" id="two"></div>
  <div class="item" id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  width: 100vw;
  height: 100vh;
  box-sizing: border-box;
}

.item {
  flex: 1;
  margin: 20px;
}
```

```
:root {
  --base-color: orange;
  /* equal to rgb(255 165 0) */
}

#one {
  background-color: var(--base-color);
}

#two {
  background-color: rgb(from var(--base-color) calc(r - 76.5) g calc(b + 76.5));
  /* 76.5 is 30% of 255 */
}

#three {
  background-color: rgb(from var(--base-color) calc(r - 153) g calc(b + 153));
  /* 153 is 60% of 255 */
}
```

```
/* Use @supports to add in support for old syntax that requires r g b values to
   be specified as percentages (with units) in calculations. This is required
   for Safari 16.4+. */
@supports (color: rgb(from red r g calc(b + 30%))) {
  #two {
    background-color: rgb(from var(--base-color) calc(r - 30%) g calc(b + 30%));
  }

  #three {
    background-color: rgb(from var(--base-color) calc(r - 60%) g calc(b + 60%));
  }
}
```

Result

Legacy syntax: comma-separated values

For legacy reasons, the `rgb()` function accepts a form in which all values are separated using commas.

HTML

```
<div class="space-separated"></div>
<div class="comma-separated"></div>
```

CSS

```
div {
  width: 100px;
  height: 50px;
  margin: 1rem;
}

div.space-separated {
  background-color: rgb(255 0 0 / 50%);
}

div.comma-separated {
  background-color: rgb(255, 0, 0, 0.5);
}
```

Result

See also

- The `<color>` data type for a list of all color notations
- Color format converter tool
- Using relative colors
- CSS colors module

<color-interpolation-method> CSS type

The **`<color-interpolation-method>`** CSS data type represents the color space used for interpolation between `<color>` values. It can be used to override the default interpolation color space for color-related functional notations such as `color-mix()` and `linear-gradient()`.

When interpolating `<color>` values, the interpolation color space defaults to Oklab.

Syntax

The `<color-interpolation-method>` specifies whether interpolation should use a rectangular color space or a polar color space with an optional hue interpolation method:

```
in <rectangular-color-space>
// or
in <polar-color-space>[ <hue-interpolation method>]
```

Values

`<rectangular-color-space>`

One of the keywords `srgb`, `srgb-linear`, `display-p3`, `a98-rgb`, `prophoto-rgb`, `rec2020`, `lab`, `oklab`, `xyz`, `xyz-d50`, or `xyz-d65`.

`<polar-color-space>`

One of the keywords `hsl`, `hwb`, `lch`, or `oklch`.

`<hue-interpolation-method>` Optional

The algorithm for hue interpolation. It defaults to `shorter hue`.

`<custom-color-space>`

A `<dashed-ident>` referring to a custom @color profile.

Formal syntax

```
<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
```

```
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing interpolation color spaces using gradients

The following example shows the effect of using different interpolation color spaces for `linear-gradient()`.

HTML

```
<div>sRGB:</div>
<div class="gradient srgb"></div>
<div>Oklab:</div>
<div class="gradient oklab"></div>
<div>Oklch (with <code>longer hue</code>):</div>
<div class="gradient oklch-longer"></div>
```

CSS

```
.gradient {
  height: 50px;
  width: 100%;
}
.srgb {
  background-image: linear-gradient(in srgb to right, blue, red);
}
.oklab {
  background-image: linear-gradient(in oklab to right, blue, red);
}
.oklch-longer {
  background-image: linear-gradient(in oklch longer hue to right, blue, red);
}
```

Result

Color interpolation in repeating gradients

The following example shows how to specify a color space when interpolating colors in repeating gradients. Three boxes show different types of repeating gradients using the `repeating-conic-gradient()`, `repeating-linear-gradient()`, and `repeating-radial-gradient()` functions. The first box uses the Lab color space to interpolate between two color values. The second and third boxes use OkLCh and additionally provide a `<hue-interpolation-method>` to specify how to interpolate between hue values.

HTML

```
<div class="gradient conic">conic</div>
<div class="gradient linear">linear</div>
<div class="gradient radial">radial</div>
```

CSS

We used the same two colors in each gradient to demonstrate the different effects of `<hue-interpolation-method>` and color space on color interpolation in gradients.

```
.gradient {
  height: 200px;
  width: 200px;
  display: inline-block;
  font-family: monospace;
  margin: 10px;
  font-size: 16px;
}
```

```
.conic {
  background-image: repeating-conic-gradient(
    in lab,
    burlywood,
    blueviolet 120deg
  );
}

.linear {
  background-image: repeating-linear-gradient(
    in oklch,
    burlywood,
    blueviolet 75px
  );
}

.radial {
  background-image: repeating-radial-gradient(
    in oklch longer hue,
    blueviolet 50px,
    burlywood 100px
  );
}
```

Result

Comparing the first and second boxes demonstrates the difference of interpolating between two colors in differing color spaces. Comparing the second and third boxes shows the difference between `<hue-interpolation-method>`s, with the linear gradient using the shorter method (default) and the radial gradient using the longer method.

Browser compatibility

See also

- `<color>`, `<gradient>`
- `<hue-interpolation-method>`

<content-distribution> CSS type

The **`<content-distribution>`** enumerated value type is used by `justify-content` and `align-content` properties, and the `place-content` shorthand, to distribute a container's extra space among its alignment subjects.

Syntax

```
<content-distribution> = space-between | space-around | space-evenly | stretch
```

Values

The following keyword values are represented by the `<content-distribution>` grammar term:

`space-between`

> Evenly distributes the alignment subject within the alignment container. The first item is placed flush with the start edge of the alignment container, the last item subject is placed flush with the end edge of the alignment container, and the remaining items are evenly distributed so that the spacing between any two adjacent items is the same. The default fallback alignment for `space-between` is `safe flex-start` for flex layout, and `start` otherwise. If there is only one item, the item will be flush with the start edge.

`space-around`

> The items are evenly distributed in the container, with a half-size space on either end. The spacing between any two adjacent items is the same, and the spacing before the first and after the last items is half the size of the other spacing. The default fallback alignment for `space-around` is `safe center`. If the container has only one child, the item will be centered.

`space-evenly`

> The items are evenly distributed in the container, with a full-size space on either end. The spacing between any two adjacent items, before the first item, and after the last item, are all the same. The default fallback alignment for `space-evenly` is `safe center`. If the container has only one child, the item will be centered.

`stretch`

> If the combined size of the items is less than the size of the container, any items that can grow will have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`, `max-width`, or equivalent functionality, so that the combined size of the items exactly fills the container. The default fallback alignment for `stretch` is `flex-start` in flexbox, and `start` in other layout modes. If there is only one item, and that item can grow, it will grow to fill the container.

See also

- Properties that use this data type: `align-content`, `justify-content`, `place-content`
- Other box alignment data types: `<baseline-position>`, `<content-position>`, `<overflow-position>`, and `<self-position>`
- CSS box alignment module
- CSS flexible box layout module
- CSS grid layout module

<content-position> CSS type

The **`<content-position>`** enumerated value type is used by `justify-content` and `align-content` properties, and the `place-content` shorthand, to align the box's contents within itself.

Syntax

```
<content-position> = center | start | end | flex-start | flex-end
```

Values

The `<content-position>` enumerated value type is specified using one of the following key terms.

`center`

> Centers the alignment subject within its alignment container.

`start`

> Aligns the alignment subject flush with the alignment container's start edge.

`end`

> Aligns the alignment subject flush with the alignment container's end edge.

`flex-start`

> In flex layout, aligns the alignment subject flush with the edge of the alignment container corresponding to the flex container's main-start or cross-start side, as appropriate. It is identical to `start` for layout modes other than flex layout.

`flex-end`

> In flex layout, aligns the alignment subject flush with the edge of the alignment container corresponding to the flex container's main-end or cross-end side, as appropriate. Identical to `end` for layout modes other than flex layout.

> **Note:** The `left` and `right` keywords are excluded from `<content-position>`, despite being valid positional alignment values for the `justify-*` properties (`justify-content`, `justify-self`, and `justify-items`), because they are not allowed in the `align-*` properties (`align-content`, `align-self`, and `align-items`). They are instead explicitly included in the `justify-*` properties' grammars.

See also

- Properties that use this data type: `align-content`, `justify-content`, `place-content`
- Other box alignment data types: `<baseline-position>`, `<content-distribution>`, `content-position`, `<overflow-position>`, and `<self-position>`
- CSS box alignment module
- CSS flexible box layout module

- CSS grid layout module

<corner-shape-value> CSS type

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`<corner-shape-value>`** CSS data type describes the shape of a container corner. It is used by the `corner-shape` shorthand property and its constituent properties to specify the shape to apply to affected container corners.

Syntax

The `<corner-shape-value>` data type can take a `superellipse()` function that defines a custom shape, or one of six keyword values that describe common `superellipse()` values.

Values

`superellipse()`

Defines a custom corner superellipse shape. A negative parameter creates an inward, or concave, curve while a positive parameter creates an outward, or convex, curve.

Keywords

The available keyword values are as follows:

`bevel`

Defines a straight, diagonal corner, which is neither convex nor concave. The `bevel` keyword is equivalent to `superellipse(0)`.

`notch`

Defines a 90-degree concave square corner. The `notch` keyword is equivalent to `superellipse(-infinity)`.

`round`

Defines a convex ordinary ellipse, which is the standard rounded corner created by `border-radius` without a `corner-shape` applied. The `round` keyword is equivalent to `superellipse(1)`. This is the default (initial) value for all `corner-shape` properties.

`scoop`

Defines a concave ordinary ellipse. The `scoop` keyword is equivalent to `superellipse(-1)`.

`square`

Defines a 90-degree convex square corner, which is the default corner shape when no `border-radius` (or `border-radius: 0`) is applied. The `square` keyword is equivalent to `superellipse(infinity)`.

`squircle`

Defines a "squircle", which is a convex curve in between `round` and `square`. The `squircle` keyword is equivalent to `superellipse(2)`.

> **Note:** You can smoothly animate between different `superellipse()` values, and between different corner shape keywords, as the animation interpolates between their `superellipse()` equivalents.

Formal syntax

```
<corner-shape-value> =
  round             |
  scoop             |
  bevel             |
  notch             |
  square            |
  squircle          |
  <superellipse()>

<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

<code><corner-shape-value></code> value comparison

In this example, we provide a drop-down menu allowing you to select different `<corner-shape-value>` values and a slider that updates the container's `border-radius`. This enables visualizing the effect of the different keywords and `superellipse()` parameter values.

The `corner-shape` property defines the shape of the box's corners while the region the shape is applied to is specified by the `border-radius` property. The code is hidden for brevity, but you can find a full explanation of `corner-shape` values along with other related examples on the `corner-shape` reference page.

```
<form>
  <div>
    <label for="corner-shape-choice">Choose a corner-shape value:</label>
    <select id="corner-shape-choice">
      <optgroup label="Keywords">
        <option value="square">square | superellipse(infinity)</option>
        <option selected value="squircle">squircle | superellipse(2)</option>
        <option value="round">round | superellipse(1)</option>
        <option value="bevel">bevel | superellipse(0)</option>
        <option value="scoop">scoop | superellipse(-1)</option>
        <option value="notch">notch | superellipse(-infinity)</option>
      </optgroup>
      <optgroup label="Functions">
        <option>superellipse(3)</option>
        <option>superellipse(1.5)</option>
        <option>superellipse(0.5)</option>
        <option>superellipse(-0.5)</option>
        <option>superellipse(-1.5)</option>
        <option>superellipse(-3)</option>
      </optgroup>
```

```
    </select>
  </div>
  <div>
    <label for="radius-slider">Choose a border-radius value:</label>
    <input
      type="range"
      id="radius-slider"
      min="0"
      value="45"
      max="90"
      step="1" />
  </div>
</form>
<section></section>
```

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

body {
  width: fit-content;
  margin: 20px auto;
}

section {
  display: flex;
  justify-content: center;
  align-items: center;
  margin-top: 20px;
}

select {
  padding: 3px 5px;
}

form div:nth-of-type(2) {
  margin-top: 5px;
  display: flex;
}

section {
  width: 100%;
  height: 180px;
  background-color: orange;
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}

section {
  box-shadow: 1px 1px 3px gray;
}
```

```
const rectangle = document.querySelector("section");
const select = document.querySelector("select");
const range = document.getElementById("radius-slider");

function setCorners() {
  rectangle.style.cornerShape = select.value;
  const brValue = `${range.value}px`;
  rectangle.style.borderRadius = brValue;
  rectangle.innerHTML = `<div><code>corner-shape: ${select.value};</code><br><code>border-radius: ${brValue};</code></div>`;
}

select.addEventListener("change", setCorners);
range.addEventListener("input", setCorners);
setCorners();
```

> **Note:** See also the `superellipse()` function value comparison example.

See also

- `corner-shape`
- `border-radius`
- `superellipse()`
- CSS borders and box decorations module

cos() CSS function

The **`cos()`** CSS function is a trigonometric function that returns the cosine of a number, which is a value between `-1` and `1`. The function contains a single calculation that must resolve to either a `<number>` or an `<angle>` by interpreting the result of the argument as radians. That is, `cos(45deg)`, `cos(0.125turn)`, and `cos(3.14159 / 4)` all represent the same value, approximately `0.707`.

Try it

```
transform: translateX(calc(cos(0deg) * 140px))
  translateY(calc(sin(0deg) * -140px));
```

```
transform: translateX(calc(cos(90deg) * 140px))
  translateY(calc(sin(90deg) * -140px));
```

```
transform: translateX(calc(cos(135deg) * 140px))
  translateY(calc(sin(135deg) * -140px));
```

```
transform: translateX(calc(cos(180deg) * 140px))
  translateY(calc(sin(180deg) * -140px));
```

```
transform: translateX(calc(cos(-45deg) * 140px))
  translateY(calc(sin(-45deg) * -140px));
```

```
<div class="circle">
  <span class="dot" id="example-element"></span>
</div>
```

```
:root {
  --radius: 140px;
  --dot-size: 10px;
}
.circle {
  display: grid;
  place-content: center;
  margin: 0 auto;
  width: calc(var(--radius) * 2);
  aspect-ratio: 1;
  border-radius: 50%;
  border: 2px solid #666666;
  background-image:
    radial-gradient(black var(--dot-size), transparent var(--dot-size)),
    linear-gradient(135deg, blue, deepskyblue, lightgreen, lavender, honeydew);
}
.dot {
  display: block;
  width: var(--dot-size);
  aspect-ratio: 1;
  border-radius: 50%;
  border: 2px solid #666666;
  background-color: #ff6666;
  transform: translateX(calc(cos(0deg) * var(--radius)))
    translateY(calc(sin(0deg) * var(--radius) * -1));
}
```

Syntax

```
/* Single <angle> values */
width: calc(100px * cos(45deg));
width: calc(100px * cos(0.125turn));
width: calc(100px * cos(0.785398163rad));

/* Single <number> values */
width: calc(100px * cos(63.673));
width: calc(100px * cos(2 * 0.125));

/* Other values */
width: calc(100px * cos(pi));
width: calc(100px * cos(e / 2));
```

Parameters

The `cos(angle)` function accepts only one value as its parameter.

`angle`

A calculation which resolves to a `<number>` or an `<angle>`. When specifying unitless numbers they are interpreted as a number of radians, representing an `<angle>`.

Return value

The cosine of an `angle` will always return a number between `-1` and `1`.

- If `angle` is `infinity`, `-infinity`, or `NaN`, the result is `NaN`.

Formal syntax

```
<cos()> =
  cos( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Keep the size of a rotated box

The `cos()` function can be used to keep the size of a rotated box.

When the element is rotated using `rotate()`, it goes beyond its initial size. To fix this, we will use `cos()` to update the element size.

For example, if you rotate a `100px`/`100px` square `45deg`, the diamond it creates will be wider and taller than the original square. To shrink the diamond into the box allotted for the original square, you would have to scale down the diamond using this formula: `width = height = 100px * cos(45deg) = 100px * 0.707 = 70.7px`. You need to also adjust the `transform-origin` and add `translate()` to correct the position:

HTML

```
<div class="original-square"></div>
<div class="rotated-diamond"></div>
<div class="rotated-scaled-diamond"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

```
div.original-square {
  width: 100px;
  height: 100px;
  background-color: blue;
}
div.rotated-diamond {
  width: 100px;
  height: 100px;
  background-color: red;
  transform: rotate(45deg);
}
div.rotated-scaled-diamond {
  width: calc(100px * cos(45deg));
  height: calc(100px * cos(45deg));
  margin: calc(100px / 4 * cos(45deg));
  transform: rotate(45deg);
  transform-origin: center;
  background-color: green;
}
```

Result

See also

- `sin()`
- `tan()`
- `asin()`
- `acos()`
- `atan()`
- `atan2()`

counter() CSS function

The **`counter()`** CSS function returns a string representing the current value of the named counter, if there is one.

The `counter()` function is generally used within pseudo-element through the `content` property but, theoretically, it can be used wherever a `<string>` value is supported.

Try it

```
.double-list {
  counter-reset: count -1;
}

.double-list li {
  counter-increment: count 2;
}

.double-list li::marker {
  content: counter(count, decimal) ") ";
}
```

```
<p>Best Dynamic Duos in Sports:</p>
<ol class="double-list">
  <li>Simone Biles + Jonathan Owens</li>
  <li>Serena Williams + Venus Williams</li>
  <li>Aaron Judge + Giancarlo Stanton</li>
  <li>LeBron James + Dwyane Wade</li>
  <li>Xavi Hernandez + Andres Iniesta</li>
</ol>
```

Syntax

```
/* Basic usage */
counter(counter-name);

/* changing the counter display */
counter(counter-name, upper-roman)
```

Counters have no visible effect by themselves. The `counter()` and `counters()` functions are what make counters useful by returning developer-defined strings (or images).

Values

The `counter()` function accepts up to two parameters. The first parameter is the `<counter-name>`. The optional second parameter is the `<counter-style>`.

`<counter-name>`

> A `<custom-ident>` identifying the counter, which is the same case-sensitive name used with the `counter-reset` and `counter-increment` property values. The counter name cannot start with two dashes and can't be `none`, `unset`, `initial`, or `inherit`.

`<counter-style>`

> A `list-style-type` name, `counter-style-name` or `symbols()` function, where a counter style name is a `numeric`, `alphabetic`, or `symbolic` predefined counter style, a complex longhand east Asian or Ethiopic predefined counter style, or other predefined counter style. If omitted, the counter-style defaults to `decimal`.

> **Note:** To join the counter values when nesting counters, use the `counters()` function, which provides an additional `<string>` parameter.

Formal syntax

```
<counter()> =
  counter( <counter-name> , <counter-style>? )

<counter-name> =
  <custom-ident>

<counter-style> =
  <counter-style-name>  |
  <symbols()>

<counter-style-name> =
  <custom-ident>

<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

lower-roman compared to lower-alpha

In this example, we display a counter using `lower-roman` and `lower-alpha` list styles.

HTML

```
<ol>
  <li></li>
  <li></li>
  <li></li>
</ol>
```

CSS

```
ol {
  counter-reset: count;
}
li {
  counter-increment: count;
}
li::after {
  content:
    "[" counter(count, lower-roman) "] == ["
    counter(count, lower-alpha) "]";
}
```

Result

Displaying a counter using three styles

In this example, we use the `counter()` function three times.

HTML

```
<ol>
  <li></li>
  <li></li>
  <li></li>
</ol>
```

CSS

We include the `counter()` function with three different counter styles, including the default decimal value. We've added padding to the list to provide space for the long `::marker` string.

```
ol {
  counter-reset: listCounter;
  padding-left: 5em;
}
li {
  counter-increment: listCounter;
}
li::marker {
  content:
    "Item #" counter(listCounter) " is: ";
}
li::after {
  content:
    "[" counter(listCounter, decimal-leading-zero) "] == ["
    counter(listCounter, upper-roman) "]";
}
```

Result

See also

- Using CSS Counters
- `counter-reset`
- `counter-set`
- `counter-increment`
- `@counter-style`
- CSS `counters()` function
- CSS lists and counters module
- CSS counter styles module
- CSS generated content module

counters() CSS function

The **`counters()`** CSS function enables combining markers when nesting counters. The function returns a string that concatenates the current values of the named and nested counters, if any are present, with the string provided. The third, optional parameter enables defining the list style.

The `counters()` function is generally used within pseudo-element through the `content` property, but theoretically, it can be used wherever a `<string>` value is supported.

The `counters()` function has two forms: `counters(<name>, <string>)` and `counters(<name>, <string>, <style>)`. The generated text is the value of all counters with the given `<name>`, arranged from the outermost to the innermost, and separated by the specified `<string>`. The counters are rendered in the `<style>` indicated, defaulting to `decimal` if no `<style>` is specified.

Try it

```
ol {
  counter-reset: index;
  list-style-type: none;
}

li::before {
  counter-increment: index;
  content: counters(index, ".", decimal) " ";
}
```

```
<ol>
  <li>Mars</li>
  <li>
    Saturn
    <ol>
      <li>Mimas</li>
      <li>Enceladus</li>
      <li>
        <ol>
          <li>Voyager</li>
          <li>Cassini</li>
        </ol>
      </li>
      <li>Tethys</li>
    </ol>
  </li>
  <li>
    Uranus
    <ol>
      <li>Titania</li>
    </ol>
  </li>
</ol>
```

Syntax

```
/* Basic usage  - style defaults to decimal */
counters(counter-name, '.');

/* changing the counter display */
counters(counter-name, '-', upper-roman)
```

A counter has no visible effect by itself. The `counters()` function (and `counter()` function) is what makes it useful by returning developer-defined content.

Values

The `counters()` function accepts two or three parameters. The first parameter is the `<counter-name>`. The second parameter is the concatenator `<string>`. The optional third parameter is the `<counter-style>`.

`<counter-name>`

> A `<custom-ident>` identifying the counters, which is the same case-sensitive name used for the `counter-reset` and `counter-increment` properties. The name cannot start with two dashes and can't be `none`, `unset`, `initial`, or `inherit`. Alternatively, for inline, single-use counters, the `symbols()` function can be used instead of a named counter in browsers that support `symbols()`.

`<string>`

> Any number of text characters. Non-Latin characters must be encoded using their Unicode escape sequences: for example, `\000A9` represents the copyright symbol.

`<counter-style>`

> A counter style name or a `symbols()` function. The counter style name can be a predefined style such as numeric, alphabetic, or symbolic, a complex longhand predefined style such as East Asian or Ethiopic, or another predefined counter style. If omitted, the counter-style defaults to decimal.

The return value is a string containing all the values of all the counters in the element's CSS counters set named `<counter-name>` in the counter style defined by `<counter-style>` (or decimal, if omitted). The return string is sorted in outermost-first to innermost-last order, joined by the `<string>` specified.

> **Note:** For information about non-concatenated counters, see the `counter()` function, which omits the `<string>` as a parameter.

Formal syntax

```
<counters()> =
  counters( <counter-name> , <string> , <counter-style>? )

<counter-name> =
  <custom-ident>

<counter-style> =
  <counter-style-name>  |
  <symbols()>

<counter-style-name> =
  <custom-ident>

<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing default counter value to uppercase roman numerals

This example includes two `counters()` functions: one with `<counter-style>` set and the other defaulting to `decimal`.

HTML

```
<ol>
  <li>
    <ol>
      <li></li>
      <li></li>
      <li></li>
    </ol>
```

```
  </li>
  <li></li>
  <li></li>
  <li>
    <ol>
      <li></li>
      <li>
        <ol>
          <li></li>
          <li></li>
          <li></li>
        </ol>
      </li>
    </ol>
  </li>
</ol>
```

CSS

```
ol {
  counter-reset: listCounter;
}
li {
  counter-increment: listCounter;
}
li::marker {
  content:
    counters(listCounter, ".", upper-roman) ") ";
}
li::before {
  content:
    counters(listCounter, ".") " == "
    counters(listCounter, ".", lower-roman);
}
```

Result

Comparing decimal-leading-zero counter value to lowercase letters

This example includes three `counters()` functions, each with different `<string>` and `<counter-style>` values.

HTML

```
<ol>
  <li>
    <ol>
      <li></li>
      <li></li>
      <li></li>
    </ol>
  </li>
  <li></li>
  <li></li>
  <li>
    <ol>
      <li></li>
      <li>
        <ol>
          <li></li>
          <li></li>
          <li></li>
        </ol>
      </li>
    </ol>
  </li>
</ol>
```

CSS

```
ol {
  counter-reset: count;
}
li {
  counter-increment: count;
}
li::marker {
  content:
    counters(count, "-", decimal-leading-zero) ") ";
}
li::before {
  content:
    counters(count, "~", upper-alpha) " == "
    counters(count,  "*", lower-alpha);
}
```

Result

See also

- Using CSS Counters
- `counter-set` property
- `counter-reset` property
- `counter-increment` property
- `@counter-style` at-rule
- CSS `counter()` function
- `::marker` pseudo-element
- CSS lists and counters module
- CSS counter styles module
- CSS generated content module

cross-fade() CSS function

The **`cross-fade()`** CSS function can be used to blend two or more images at a defined transparency. It can be used for many basic image manipulations, such as tinting an image with a solid color or highlighting a particular area of the page by combining an image with a radial gradient.

Syntax

> **Warning:** The specification and current implementations have different syntaxes. The specification syntax is explained first.

Specification syntax

The `cross-fade()` function takes a list of images with a percentage defining how much of each image is retained in terms of opacity when it is blended with the other images. The percent value must be coded without quotes, must contain the `'%'` symbol, and its value must be between 0% and 100%.

The function can be used in CSS anywhere an ordinary image reference can be used.

Cross-fade percentages

Think of the percentage as an opacity value for each image. This means a value of 0% means the image is fully transparent while a value of 100% makes the image fully opaque.

```
cross-fade(url("white.png") 0%, url("black.png") 100%); /* fully black */
cross-fade(url("white.png") 25%, url("black.png") 75%); /* 25% white, 75% black */
cross-fade(url("white.png") 50%, url("black.png") 50%); /* 50% white, 50% black */
cross-fade(url("white.png") 75%, url("black.png") 25%); /* 75% white, 25% black */
cross-fade(url("white.png") 100%, url("black.png") 0%); /* fully white */
cross-fade(url("green.png") 75%, url("red.png") 75%); /* both green and red at 75% */
```

If any percentages are omitted, all the specified percentages are summed together and subtracted from `100%`. If the result is greater than 0%, the result is then divided equally between all images with omitted percentages.

In the simplest case, two images are faded between each other. To do that, only one of the images needs to have a percentage, the other one will be faded accordingly. For example, a value of 0% defined for the first image yields only the second image, while 100% yields only the first. A 25% value renders the first image at 25% and the second at 75%. The 75% value is the inverse, showing the first image at 75% and the second at 25%.

The above could also have been written as:

```
cross-fade(url("white.png") 0%, url("black.png")); /* fully black */
cross-fade(url("white.png") 25%, url("black.png")); /* 25% white, 75% black */
cross-fade(url("white.png"), url("black.png")); /* 50% white, 50% black */
cross-fade(url("white.png") 75%, url("black.png")); /* 75% white, 25% black */
cross-fade(url("white.png") 100%, url("black.png")); /* fully white */
cross-fade(url("green.png") 75%, url("red.png") 75%); /* both green and red at 75% */
```

If no percentages are declared, both the images will be 50% opaque, with a cross-fade rendering as an even merge of both images. The 50%/50% example seen above did not need to have the percentages listed, as when a percentage value is omitted, the included percentages are added together and subtracted from 100%. The result, if greater than 0, is then divided equally between all images with omitted percentages.

In the last example, the sum of both percentages is not 100%, and therefore both images include their respective opacities.

If no percentages are declared and three images are included, each image will be 33.33% opaque. The two following are lines (almost) identical:

```
cross-fade(url("red.png"), url("yellow.png"), url("blue.png")); /* all three will be 33.3333% opaque */
cross-fade(url("red.png") 33.33%, url("yellow.png") 33.33%, url("blue.png") 33.33%);
```

Older, implemented syntax

```
cross-fade( <image>, <image>, <percentage> )
```

The specification for the `cross-fade()` function allows for multiple images and for each image to have transparency values independent of the other values. This was not always the case. The original syntax, which has been implemented in some browsers, only allowed for two images, with the sum of the transparency of those two images being exactly 100%. The original syntax is supported in Safari and supported with the `-webkit-` prefix in Chrome, Opera, and other blink-based browsers.

```
cross-fade(url("white.png"), url("black.png"), 0%);   /* fully black */
cross-fade(url("white.png"), url("black.png"), 25%);  /* 25% white, 75% black */
cross-fade(url("white.png"), url("black.png"), 50%);  /* 50% white, 50% black */
cross-fade(url("white.png"), url("black.png"), 75%);  /* 75% white, 25% black */
cross-fade(url("white.png"), url("black.png"), 100%); /* fully white */
```

In the implemented syntax, the two comma-separated images are declared first, followed by a comma and required percent value. Omitting the comma or percent invalidates the value. The percent is the opacity of the first declared image. The included percentage is subtracted from 100%, with the difference being the opacity of the second image.

The green/red example (with the percentages totaling 150%) and the yellow/red/blue example (with three images) from the specification syntax section, are not possible in this implementation.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document. When using background images, make sure the contrast in color is great enough that any text is legible over the image as well as if the images are missing.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Formal syntax

```
<cross-fade()> =
  cross-fade( <cf-image>#? )

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
```

```
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Older syntax for cross-fade

HTML

```
<div class="cross-fade"></div>
```

CSS

```
.cross-fade {
  width: 300px;
  height: 300px;
  background-image: -webkit-cross-fade(url("br.png"), url("tr.png"), 75%);
  background-image: cross-fade(url("br.png"), url("tr.png"), 75%);
}
```

Result

See also

- `<image>`
- `<url>`
- `image()`
- `image-set()`
- `element()`
- Using CSS gradients
- Gradient functions: `linear-gradient()`, `radial-gradient()`, `repeating-linear-gradient()`, `repeating-radial-gradient()`, `conic-gradient()`, `repeating-conic-gradient()`

<custom-ident> CSS type

The **`<custom-ident>`** CSS data type denotes an arbitrary user-defined string used as an identifier. It is case-sensitive, and certain values are forbidden in various contexts to prevent ambiguity.

Syntax

The syntax of `<custom-ident>` is similar to CSS identifiers (such as property names), except that it is case-sensitive. It consists of one or more characters, where characters can be any of the following:

- any alphabetical character (`A` to `Z`, or `a` to `z`),
- any decimal digit (`0` to `9`),
- a hyphen (`-`),
- an underscore (`_`),
- an escaped character (preceded by a backslash, `\`),
- a Unicode character (in the format of a backslash, `\`, followed by one to six hexadecimal digits, representing its Unicode code point)

Note that `id1`, `Id1`, `iD1`, and `ID1` are all different identifiers as they are case-sensitive.

Escaping characters

Any Unicode code point can be included as part of a `<custom-ident>` or quoted `<string>` by escaping it.

In CSS, there are several ways to escape a character. Escape sequences start with a backslash (`\`), and continue with:

- One to six hex (`ABCDEF0123456789`) digits. The hex digits can optionally be followed by white space. The hex escape sequence gets replaced by the Unicode code point whose value is given by those digits. The whitespace allows the sequences to be followed by actual hex digits (versus replaced ones).
- Any Unicode code point that is not a hex digit or a newline character.

Examples:

- "&B" can be written as `\26 B` or `\000026B`.
- "hi.there" can be written as `hi\.there` or `hi\002Ethere`.
- "toto?" can be written as `toto\?`, `toto\3F`, or `toto\00003F`.

To include actual white space after an escape sequence, include two white spaces in the escape sequence.

Forbidden values

A `<custom-ident>` must not be placed between single or double quotes as this would be identical to a `<string>`. Moreover, the first character must not be a decimal digit, nor a hyphen (`-`) followed by a decimal digit.

To prevent ambiguity, each property that uses `<custom-ident>` forbids the use of specific values:

`animation-name`

Forbids the global CSS values (`unset`, `initial`, and `inherit`), as well as `none`.

`counter-reset`, `counter-increment`

Forbids the global CSS values (`unset`, `initial`, and `inherit`), as well as `none`.

`@counter-style`, `list-style-type`

Forbids the global CSS values (`unset`, `initial`, and `inherit`), as well as the values:

- `none`
- `inline`
- `outside`

Also, quite a few predefined values are implemented by the different browsers:

- `disc`
- `circle`
- `square`
- `decimal`
- `cjk-decimal`
- `decimal-leading-zero`
- `lower-roman`
- `upper-roman`
- `lower-greek`
- `lower-alpha`
- `lower-latin`
- `upper-alpha`
- `upper-latin`
- `arabic-indic`
- `armenian`
- `bengali`
- `cambodian`
- `cjk-earthly-branch`
- `cjk-heavenly-stem`
- `cjk-ideographic`
- `devanagari`
- `ethiopic-numeric`
- `georgian`
- `gujarati`
- `gurmukhi`
- `hebrew`
- `hiragana`
- `hiragana-iroha`
- `japanese-formal`
- `japanese-informal`
- `kannada`
- `katakana`
- `katakana-iroha`
- `khmer`
- `korean-hangul-formal`
- `korean-hanja-formal`
- `korean-hanja-informal`
- `lao`
- `lower-armenian`
- `malayalam`
- `mongolian`
- `myanmar`
- `oriya`
- `persian`
- `simp-chinese-formal`
- `simp-chinese-informal`
- `tamil`
- `telugu`
- `thai`
- `tibetan`
- `trad-chinese-formal`
- `trad-chinese-informal`
- `upper-armenian`
- `disclosure-open`
- `disclosure-close`

`grid-row-start`, `grid-row-end`, `grid-column-start`, `grid-column-end`, `grid-template-rows`, `grid-template-columns`

Forbids the `span` and `auto` values.

`view-transition-name`

Forbids the global CSS values (`unset`, `initial`, and `inherit`), as well as `none`.

`will-change`

Forbids the global CSS values (`unset`, `initial`, and `inherit`), as well as the values `will-change`, `auto`, `scroll-position`, and `contents`.

Examples

Valid identifiers

```
nono79            A mix of alphanumeric characters and numbers
ground-level      A mix of alphanumeric characters and a dash
-test             A dash followed by alphanumeric characters
_internal         An underscore followed by alphanumeric characters
```

```
\22 toto          A Unicode character followed by a sequence of alphanumeric characters
scooby\.doo       A correctly escaped period
```

Invalid identifiers

```
34rem             It must not start with a decimal digit.
-12rad            It must not start with a dash followed by a decimal digit.
scooby.doo        Only alphanumeric characters, _, and - needn't be escaped.
'scoobyDoo'       This would be a <string>.
"scoobyDoo"       This would be a <string>.
```

Browser compatibility

As this type is not a real type but a convenience type used to simplify the description of allowed values, there is no browser compatibility information as such.

See also

- <ident>
- <dashed-ident>

<dashed-function> CSS type

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`<dashed-function>`** CSS data type represents the syntax used to call CSS custom functions, which are defined using the `@function` at-rule.

Syntax

A `<dashed-function>` value consists of the `--function-name`, followed by parentheses containing the function's arguments (for example, `--my-function(30px, 3)`).

You can include `<dashed-function>` values in place of regular CSS property values or property value components, in cases where you want to dynamically calculate the values based on function logic rather than providing static values.

In cases where you want to pass comma-containing values as arguments, you can do so by wrapping them in curly braces (`{ }`).

Examples

For more examples, see our Using CSS custom functions guide.

Basic <code><dashed-function></code> usage

This example shows a basic function that returns a semi-transparent version of the color passed to it.

HTML

The markup features a `<p>` containing some text content:

```
<p>Some content</p>
```

CSS

In our styles, we first define the CSS custom function. The function is called `--transparent` and accepts two parameters: a color and a numeric alpha channel value. Inside the function body, we use relative color syntax to transform the passed color into an `oklch()` color with an alpha channel equal to the passed alpha value; this becomes the function's `result`:

```
@function --transparent(--color <color>, --alpha <number>) {
  result: oklch(from var(--color) l c h / var(--alpha));
}
```

Next, we define a `--base-color` custom property with a value of `#faa6ff` on the `:root` element. We assign that property to be the value of the `<p>` element's `border` color, and then set its `background-color` value to equal a transparent version of the same color. This is done by setting the value to equal the `<dashed-function>` syntax, specifying the `--transparent()` function and passing it arguments of `var(--base-color)` and `0.8`.

```
html,
body {
  height: 100%;
}

body {
  margin: 0;
  display: grid;
  place-items: center;
  font-family: system-ui;
  background-image: repeating-linear-gradient(
    -45deg,
    transparent 0 20px,
    lightgrey 20px 40px
  );
}
```

```
:root {
  --base-color: #faa6ff;
}

p {
  width: 50%;
  padding: 30px;
  border-radius: 20px;
  border: 3px solid var(--base-color);
  background-color: --transparent(var(--base-color), 0.8);
}
```

Result

See also

- CSS custom properties

- `@function` at-rule
- `type()` function
- Using CSS custom functions
- CSS custom functions and mixins module

<dashed-ident> CSS type

The **`<dashed-ident>`** CSS data type is a case-sensitive `<custom-ident>` starting with two dashes that denotes an arbitrary string used as an identifier.

Syntax

The syntax of `<dashed-ident>` is similar to CSS identifiers (such as property names), except that it is case-sensitive. It is a user-defined identifier preceded by two dashes (`--`).

The double dash at the beginning makes them easily identifiable when reading through a CSS code block, and helps to avoid name clashes with standard CSS keywords.

Just like `<custom-ident>`s, `<dashed-ident>`s are defined by the user. However, some `<custom-ident>`s are defined by the CSS language itself; `<dashed-ident>`s will never be defined in CSS.

Examples

Using with CSS custom properties

When a `<dashed-ident>` is used as a CSS custom property, the property is declared first, and then the `<dashed-ident>` is used within a CSS `var()` function.

```
html {
  --primary-color: red;
  --secondary-color: blue;
  --tertiary-color: green;
}

h1,
h4 {
  color: var(--primary-color);
}

h2,
h5 {
  color: var(--secondary-color);
}

h3,
h6 {
  color: var(--tertiary-color);
}
```

Using with @color-profile

When a `<dashed-ident>` is used with the `@color-profile` at-rule, the at-rule is declared first, and then the `<dashed-ident>` is used within a CSS `color()` function.

```
@color-profile --my-color-profile {
  src: url("https://example.org/SWOP2006_Coated5v2.icc");
}

.header {
  background-color: color(--my-color-profile 0% 70% 20% 0%);
}
```

Using with @font-palette-values

When a `<dashed-ident>` is used with the `@font-palette-values` at-rule, the at-rule is declared first, and then the `<dashed-ident>` is used as the value for the `font-palette` property.

```
@font-palette-values --my-palette {
  font-family: "Bixa";
  base-palette: 1;
  override-colors: 0 red;
}

h1,
h2,
h3,
h4 {
  font-palette: --my-palette;
}
```

Using with env() and param()

When a `<dashed-dent>` is used in an external resource in an `env()` CSS function, it can be updated using the `param()` CSS function.

```
<svg viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <path fill="env(--color, black)" d="..." />
</svg>
```

```
path:hover {
  link-parameters: param(--color, tomato);
}
```

Browser compatibility

As this type is not a real type but a convenience type used to simplify the definition of other CSS syntax, there is no browser compatibility information as such.

See also

- `<ident>`
- `<custom-ident>`

CSS data types

CSS data types define typical values (including keywords and units) accepted by CSS properties and functions. They are a special kind of component value type.

The most commonly-used types are defined in the CSS Values and Units module. This module also defines functional notations, which allow for more complex types or processing. Other types are defined in the specifications to which they apply.

Below you will find a reference to the types that you are most likely to come across, however it is not a comprehensive reference for all types defined in every CSS specification.

Syntax

In formal CSS syntax, data types are denoted by a keyword placed between the angle brackets `<` and `>`. They don't correspond to any tangible CSS code entity.

Textual data types

These types include keywords and identifiers as well as strings, and URLs.

Pre-defined keywords

Keywords with a pre-defined meaning, for example, the value of `collapse` for the `border-collapse` property.

CSS-wide keywords

All properties, including custom properties, accept the CSS-wide keywords:

`initial`

The value specified as the property's initial value.

`inherit`

The computed value of the property on the element's parent.

`revert`

Rolls back the cascade to the value of the earlier origin.

`revert-layer`

Rolls back the cascade to the value of the earlier cascade layer.

`revert-rule`

Rolls back the cascade to the value of an earlier matching style rule.

`unset`

Acts as `inherit` or `initial` depending on whether the property is inherited or not.

`<custom-ident>`

A user-defined identifier, for example the name assigned using the `grid-area` property.

`<dashed-ident>`

A `<custom-ident>` with the additional restriction that it must start with two dashes, for example, with CSS Custom Properties.

`<string>`

A quoted string, such as used for a value of the `content` property.

`<url>`

A pointer to a resource, for example as the value of `background-image`.

Numeric data types

These data types are used to indicate quantities, indexes, and positions. The majority of these are defined in the CSS values and units module, however additional types are described in other modules where they are specific to that specification alone — for example the `fr` unit in the CSS grid layout module.

`<integer>`

One or more decimal units 0 through 9, optionally preceded by `-` or `+`.

`<number>`

Real numbers which may also have a fractional component, for example `1` or `1.34`.

`<dimension>`

A number with a unit attached to it, for example `90deg`, `50ms`, or `15em`. This type includes distances (`<length>`), durations (`<time>`), frequencies (`<frequency>`), resolutions (`<resolution>`), and other quantities.

`<percentage>`

A number with a percentage sign attached to it, for example `10%`.

`<ratio>`

A ratio, written with the syntax `<number> / <number>`.

`<flex>`

A flexible length introduced for CSS grid layout, written as a `<number>` with the `fr` unit attached and used for grid track sizing.

Quantities

These `<dimension>` types are used to specify distance and other quantities.

`<length>`

Lengths are a `<dimension>` and refer to distances.

`<angle>`

Angles are used in properties such as `linear-gradient()` and are a `<dimension>` with one of `deg`, `grad`, `rad`, or `turn` units attached.

`<time>`

Duration units are a `<dimension>` with an `s` or `ms` unit.

`<frequency>`

Frequencies are a `<dimension>` with a `Hz` or `kHz` unit attached.

`<resolution>`

Is a `<dimension>` with a unit identifier of `dpi`, `dpcm`, `dppx`, or `x`.

Combinations of types

Some CSS properties can take a `<dimension>` or a `<percentage>` value. In this case the percentage value will be resolved to a quantity that matches the allowable dimension. Properties which can accept a percentage in addition to a dimension will use one of the types listed below.

`<length-percentage>`

A type that can accept a length or a percentage as a value.

`<frequency-percentage>`

A type that can accept a frequency or a percentage as a value.

`<angle-percentage>`

A type that can accept an angle or a percentage as a value.

`<time-percentage>`

A type that can accept a time or a percentage as a value.

Color

The CSS color module defines the `<color>` data type, and other types which relate to color in CSS.

`<color>`

Specified as a keyword or a numerical color value.

`<alpha-value>`

Specifies the transparency of a color. May be a `<number>`, in which case 0 is fully transparent and 1 is fully opaque, or a `<percentage>`, in which case 0% is fully transparent and 100% fully opaque.

`<hue>`

Specifies the `<angle>`, with a unit identifier of `deg`, `grad`, `rad`, or `turn`, or unitless `<number>` interpreted as `deg`, of the color wheel specific to the `<absolute-color-functions>` of which it is a component.

Images

The CSS images module defines the data types which deal with images, including gradients.

`<image>`

A URL reference to an image or a color gradient.

`<color-stop-list>`

A list of two or more color stops with optional transition information using a color hint.

`<linear-color-stop>`

A `<color>` and a `<length-percentage>` to indicate the color stop for this part of the gradient.

`<linear-color-hint>`

A `<length-percentage>` to indicate how the color interpolates.

`<ending-shape>`

Used for radial gradients; can have a keyword value of `circle` or `ellipse`.

`<size>`

Determines the size of the radial gradient's ending shape. This accepts a value of a keyword or a `<length>` but not a percentage.

2D positioning

The `<position>` data type is interpreted as defined for the `background-position` property.

`<position>`

Defines the position of an object area. Accepts a keyword value such as `top` or `left`, or a `<length-percentage>`.

Calculation data types

These data types are used in CSS math function calculations.

`<calc-sum>`

A calculation which is a sequence of calculation values interspersed with addition (`+`) and subtraction (`-`) operators. This data type requires both values to have units.

`<calc-product>`

A calculation which is a sequence of calculation values interspersed with multiplication (`*`) and division (`/`) operators. When multiplying, one value must be unitless. When dividing, the second value must be unitless.

`<calc-value>`

Defines accepted values for calculations, values such as `<number>`, `<dimension>`, `<percentage>`, `<calc-keyword>` or nested `<calc-sum>` calculations.

`<calc-keyword>`

Defines a number of CSS keywords representing numeric constants such as `e` and π, that can be used in CSS math functions.

Shape data types

The CSS shapes and CSS borders and box decorations modules define shape data types:

`<basic-shape>`

Describes shape functions used in the `clip-path`, `shape-outside`, and `offset-path` properties.

`<corner-shape-value>`

Describes the shape of a container corner. It is used by the `corner-shape` shorthand property and its constituent properties to specify the shape applied to affected container corners.

Alphabetical index of data types

- `<absolute-size>`
- `<alpha-value>`
- `<angle>`
- `<angle-percentage>`
- `<axis>`
- `<baseline-position>`
- `<basic-shape>`
- `<blend-mode>`
- `<box-edge>`
- `<calc-keyword>`
- `<calc-sum>`
- `<color>`
- `<color-interpolation-method>`
- `<content-distribution>`
- `<content-position>`
- `<corner-shape-value>`
- `<custom-ident>`
- `<dashed-function>`
- `<dashed-ident>`
- `<dimension>`
- `<display-box>`
- `<display-inside>`
- `<display-internal>`
- `<display-legacy>`
- `<display-listitem>`
- `<display-outside>`
- `<easing-function>`
- `<filter-function>`
- `<flex>`
- `<frequency>`
- `<frequency-percentage>`
- `<generic-family>`
- `<gradient>`
- `<hex-color>`
- `<hue>`
- `<hue-interpolation-method>`
- `<ident>`
- `<image>`
- `<integer>`
- `<length>`
- `<length-percentage>`
- `<line-style>`
- `<line-width>`
- `<named-color>`
- `<number>`
- `<overflow>`
- `<overflow-position>`
- `<percentage>`
- `<position>`
- `<position-area>`
- `<ratio>`
- `<relative-size>`
- `<resolution>`
- `<self-position>`
- `<shape>`
- `<string>`
- `<system-color>`
- `<text-edge>`
- `<time>`
- `<timeline-range-name>`
- `<time-percentage>`
- `<transform-function>`
- `<url>`

See also

- CSS values and units module
- Learn: Values and units
- CSS functional notation

`<dimension>` CSS type

The **`<dimension>`** CSS data type represents a `<number>` with a unit attached to it, for example `10px`.

CSS uses dimensions to specify distances (`<length>`), durations (`<time>`), frequencies (`<frequency>`), resolutions (`<resolution>`), and other quantities.

Syntax

The syntax of `<dimension>` is a `<number>` immediately followed by a unit which is an identifier. Unit identifiers are case insensitive.

Examples

Valid dimensions

```
12px      12 pixels
1rem      1 rem
1.2pt     1.2 points
2200ms    2200 milliseconds
5s        5 seconds
200hz     200 Hertz
200Hz     200 Hertz (values are case insensitive)
```

Invalid dimensions

```
12 px       The unit must come immediately after the number.
12"px"      Units are identifiers and therefore unquoted.
3sec        The seconds unit is abbreviated "s" not "sec".
```

See also

- CSS data types
- Learn to style HTML using CSS
- CSS distances (`<length>`), durations (`<time>`), frequencies (`<frequency>`), and resolutions (`<resolution>`)

<display-box> CSS type

These keywords define whether an element generates display boxes at all.

Syntax

Valid `<display-box>` values:

`contents`

> These elements don't produce a specific box by themselves. They are replaced by their pseudo-box and their child boxes. Please note that the CSS Display Level 3 spec defines how the `contents` value should affect "unusual elements" — elements that aren't rendered purely by CSS box concepts such as replaced elements. See Appendix B: Effects of display: contents on Unusual Elements for more details.
>
> *Due to a bug in browsers this will currently remove the element from the accessibility tree — screen readers will not look at what's inside. See the Accessibility section below for more details.*

`none`

> Turns off the display of an element so that it has no effect on layout (the document is rendered as though the element did not exist). All descendant elements also have their display turned off. To have an element take up the space that it would normally take, but without actually rendering anything, use the `visibility` property instead.

Accessibility

Current implementations in most browsers will remove from the accessibility tree any element with a `display` value of `contents`. This will cause the element — and in some browser versions, its descendant elements — to no longer be announced by screen reading technology. This is incorrect behavior according to the CSSWG specification.

- More accessible markup with display: contents | Hidde de Vries
- Display: Contents Is Not a CSS Reset | Adrian Roselli

Formal syntax

```
<display-box> =
  contents  |
  none
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In this first example, the paragraph with a class of secret is set to `display: none`; the box and any content is now not rendered.

display: none

HTML

```
<p>Visible text</p>
<p class="secret">Invisible text</p>
```

CSS

```
p.secret {
  display: none;
}
```

Result

display: contents

In this example the outer `<div>` has a 2-pixel red border and a width of 300px. However it also has `display: contents` specified therefore this `<div>` will not be rendered, the border and width will no longer apply, and the child element will be displayed as if the parent had never existed.

HTML

```
<div class="outer">
  <div>Inner div.</div>
</div>
```

CSS

```
.outer {
  border: 2px solid red;
  width: 300px;
  display: contents;
```

```
}

.outer > div {
  border: 1px solid green;
}
```

Result

See also

- `display`
  - `<display-outside>`
  - `<display-inside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-legacy>`
- Display: Contents Is Not a CSS Reset | Adrian Roselli
- More accessible markup with display: contents by Hidde de Vries (2018)

`<display-inside>` CSS type

These keywords specify the element's inner `display` type, which defines the type of formatting context that lays out its contents (assuming it is a non-replaced element). These keywords are used as values of the `display` property, and can be used for legacy purposes as a single keyword, or as defined in the Level 3 specification alongside a value from the `<display-outside>` keywords.

Syntax

Valid `<display-inside>` values:

`flow`

The element lays out its contents using flow layout (block-and-inline layout).

If its outer display type is `inline`, and it is participating in a block or inline formatting context, then it generates an inline box. Otherwise it generates a block container box.

Depending on the value of other properties (such as `position`, `float`, or `overflow`) and whether it is itself participating in a block or inline formatting context, it either establishes a new block formatting context (BFC) for its contents or integrates its contents into its parent formatting context.

`flow-root`

The element generates a block element box that establishes a new block formatting context, defining where the formatting root lies.

`table`

These elements behave like HTML `<table>` elements. It defines a block-level box.

`flex`

The element behaves like a block element and lays out its content according to the flexbox model.

`grid`

The element behaves like a block element and lays out its content according to the grid model.

`ruby`

The element behaves like an inline element and lays out its content according to the ruby formatting model. It behaves like the corresponding HTML `<ruby>` elements.

> **Note:** Browsers that support the two value syntax, on finding the inner value only, such as when `display: flex` or `display: grid` is specified, will set their outer value to `block`. This will result in expected behavior; for example if you specify an element to be `display: grid`, you would expect that the box created on the grid container would be a block level box.

Formal syntax

```
<display-inside> =
  flow       |
  flow-root  |
  table      |
  flex       |
  grid       |
  ruby
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In this example the parent box has been given `display: flow-root` and so establishes a new BFC and contains the floated item.

HTML

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>I am content inside the container.</p>
</div>
```

CSS

```
.box {
  background-color: rgb(224 206 247);
  border: 5px solid rebeccapurple;
  display: flow-root;
}

.float {
  float: left;
  width: 200px;
  height: 150px;
  background-color: white;
  border: 1px solid black;
```

```
  padding: 10px;
}
```

Result

Browser compatibility

See also

- `display`
  - `<display-outside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-box>`
  - `<display-legacy>`
- Basic concepts of flexbox
- Basic Concepts of grid layout

<display-internal> CSS type

Some layout models such as `table` and `ruby` have a complex internal structure, with several different roles that their children and descendants can fill. This page defines those "internal" display values, which only have meaning within that particular layout mode.

Syntax

Valid `<display-internal>` values:

`table-row-group`

These elements behave like `<tbody>` HTML elements.

`table-header-group`

These elements behave like `<thead>` HTML elements.

`table-footer-group`

These elements behave like `<tfoot>` HTML elements.

`table-row`

These elements behave like `<tr>` HTML elements.

`table-cell`

These elements behave like `<td>` HTML elements.

`table-column-group`

These elements behave like `<colgroup>` HTML elements.

`table-column`

These elements behave like `<col>` HTML elements.

`table-caption`

These elements behave like `<caption>` HTML elements.

`ruby-base`

These elements behave like `<rb>` HTML elements.

`ruby-text`

These elements behave like `<rt>` HTML elements.

`ruby-base-container`

These elements are generated as anonymous boxes.

`ruby-text-container`

These elements behave like `<rtc>` HTML elements.

Formal syntax

```
<display-internal> =
  table-row-group      |
  table-header-group   |
  table-footer-group   |
  table-row            |
  table-cell           |
  table-column-group   |
  table-column         |
  table-caption        |
  ruby-base            |
  ruby-text            |
  ruby-base-container  |
  ruby-text-container
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

CSS tables example

The following example demonstrates laying out a form using CSS table layout.

HTML

```
<main>
  <div>
    <label for="name">Name</label>
    <input type="text" id="name" name="name" />
  </div>
  <div>
    <label for="age">Age</label>
    <input type="text" id="age" name="age" />
  </div>
</main>
```

CSS

```
main {
  display: table;
}

div {
  display: table-row;
}

label,
input {
  display: table-cell;
  margin: 5px;
}
```

Result

Browser compatibility

See also

- display
  - <display-outside>
  - <display-inside>
  - <display-listitem>
  - <display-box>
  - <display-legacy>

<display-legacy> CSS type

CSS 2 used a single-keyword syntax for the display property, requiring separate keywords for block-level and inline-level variants of the same layout mode. This page details those values.

Syntax

Valid <display-legacy> values:

inline-block

The element generates a block element box that will be flowed with surrounding content as if it were a single inline box (behaving much like a replaced element would).

It is equivalent to inline flow-root.

inline-table

The inline-table value does not have a direct mapping in HTML. It behaves like an HTML <table> element, but as an inline box, rather than a block-level box. Inside the table box is a block-level context.

It is equivalent to inline table.

inline-flex

The element behaves like an inline element and lays out its content according to the flexbox model.

It is equivalent to inline flex.

inline-grid

The element behaves like an inline element and lays out its content according to the grid model.

It is equivalent to inline grid.

Formal syntax

```
<display-legacy> =
  inline-block  |
  inline-table  |
  inline-flex   |
  inline-grid
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In the below example, we are creating an inline flex container with the legacy keyword inline-flex.

HTML

```
<div class="container">
  <div>Flex Item</div>
  <div>Flex Item</div>
</div>

Not a flex item
```

CSS

```
.container {
  display: inline-flex;
```

```
}
```

Result

In the new syntax the inline flex container would be created using two values, inline for the outer display type, and flex for the inner display type.

```
.container {
  display: inline flex;
}
```

Browser compatibility

See also

- `display`
  - `<display-outside>`
  - `<display-inside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-box>`

<display-listitem> CSS type

The `list-item` keyword causes the element to generate a `::marker` pseudo-element with the content specified by its `list-style` properties (for example a bullet point) together with a principal box of the specified type for its own contents.

Syntax

A single value of `list-item` will cause the element to behave like a list item. This can be used together with `list-style-type` and `list-style-position`.

`list-item` can also be combined with any `<display-outside>` keyword and the `flow` or `flow-root` `<display-inside>` keywords.

> **Note:** In browsers that support the two-value syntax, if no inner value is specified it will default to `flow`. If no outer value is specified, the principal box will have an outer display type of `block`.

Formal syntax

```
<display-listitem> =
  <display-outside>?     &&
  [ flow | flow-root ]?  &&
  list-item

<display-outside> =
  block   |
  inline  |
  run-in
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div class="fake-list">I will display as a list item</div>
```

CSS

```
.fake-list {
  display: list-item;
  list-style-position: inside;
}
```

Result

See also

- `display`
  - `<display-outside>`
  - `<display-inside>`
  - `<display-internal>`
  - `<display-box>`
  - `<display-legacy>`

<display-outside> CSS type

The `<display-outside>` keywords specify the element's outer `display` type, which is essentially its role in flow layout. These keywords are used as values of the `display` property, and can be used for legacy purposes as a single keyword, or as defined in the Level 3 specification alongside a value from the `<display-inside>` keywords.

Syntax

Valid `<display-outside>` values:

`block`

The element generates a block element box, generating line breaks both before and after the element when in the normal flow.

`inline`

The element generates one or more inline element boxes that do not generate line breaks before or after themselves. In normal flow, the next element will be on the same line if there is space.

> **Note:** When browsers encounter a display property with only an **outer** `display` value (e.g., `display: block` or `display: inline`), the inner value defaults to `flow` (e.g., `display: block flow` and `display: inline flow`). This is backwards-compatible with single-keyword syntax.

Formal syntax

```
<display-outside> =
  block   |
  inline  |
  run-in
```

This syntax reflects the latest standard as per CSS Display Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In the following example, span elements (normally displayed as inline elements) are set to `display: block` and so break onto new lines and expand to fill their container in the inline dimension.

HTML

```
<span>span 1</span> <span>span 2</span>
```

CSS

```
span {
  display: block;
  border: 1px solid rebeccapurple;
}
```

Result

Browser compatibility

See also

- `display`
  - `<display-inside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-box>`
  - `<display-legacy>`
- Block and inline layout in normal flow
- Introduction to formatting contexts

dynamic-range-limit-mix() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`dynamic-range-limit-mix()`** CSS function creates a custom maximum luminance limit by mixing different `dynamic-range-limit` keywords in specified amounts.

Syntax

```
dynamic-range-limit-mix(standard 70%, no-limit 30%);
dynamic-range-limit-mix(no-limit 10%, constrained 20%);
dynamic-range-limit-mix(no-limit 30%, constrained 30%, standard 30%);
dynamic-range-limit-mix(
    no-limit 20%,
    dynamic-range-limit-mix(standard 25%, constrained 75%) 20%
)
```

Parameters

`dynamic-range-limit` `<percentage>`

A pair consisting of a `dynamic-range-limit` value (which can be another `dynamic-range-limit-mix()` function) and a `<percentage>` between `0%` and `100%` (inclusive). The `<percentage>` specifies the proportion of a `dynamic-range-limit` keyword value in the custom limit. The `dynamic-range-limit-mix()` function can take two or more of these pairs as parameters.

Return value

A custom maximum luminance limit, expressed as a number of photographic stops higher than the HDR reference white. For privacy reasons, the actual calculated result is not exposed.

Description

The `dynamic-range-limit` property allows you to control the brightness of High Dynamic Range (HDR) content. The `dynamic-range-limit-mix()` function can be provided as a value of `dynamic-range-limit`, and enables you to create custom brightness limits by mixing together percentages of the `dynamic-range-limit` keyword values.

Calculating the percentages

When the given percentages add up to `100%`, the result is obvious:

```
/* standard 70%, no-limit 30% */
dynamic-range-limit-mix(standard 70%, no-limit 30%);
```

When the given percentages do not add up to `100%`, the resulting percentages are equal to the given percentages expressed proportionately to one another so the total equals `100%`:

```
/* no-limit 40%, constrained 60% */
dynamic-range-limit-mix(no-limit 20%, constrained 30%);

/* no-limit 20%, constrained 40%, standard 40% */
dynamic-range-limit-mix(no-limit 40%, constrained 80%, standard 80%);
```

If a `dynamic-range-limit` keyword value is used more than once, the percentages for that keyword value are added together to get the total percentage:

```
/* constrained 70%, standard 30% */
dynamic-range-limit-mix(constrained 40%, standard 30%, constrained 30%);

/* no-limit 40%, constrained 60% */
dynamic-range-limit-mix(no-limit 10%, constrained 30%, no-limit 10%);
```

If a specified percentage is less than `0%` or greater than `100%`, the `dynamic-range-limit-mix()` function — and therefore the associated `dynamic-range-limit` property value — is invalid. If a keyword is used more than once and the cumulative percentage is more than `100%`, the value is valid, and the proportion rules described above come into play.

Nesting <code>dynamic-range-limit-mix()</code> functions

You can nest `dynamic-range-limit-mix()` functions inside one another. When doing so, the same rules explained earlier apply, and each set of percentages is calculated separately and then added. In the following example:

```
dynamic-range-limit-mix(
    no-limit 10%,
    dynamic-range-limit-mix(standard 25%, constrained 75%) 20%,
    dynamic-range-limit-mix(constrained 10%, no-limit 30%) 20%
)
```

- The first line gives us `no-limit 10%`.
- Since `25%` and `75%` add up to `100%`, the second line gives us `standard 5%` (`25%` of `20%`) and `constrained 15%` (`75%` of `20%`).
- In the third line, because `10%` and `30%` add up to only `40%`, not `100%`, we normalize both as proportions of `40%`: 10/40 = `25%` and 30/40 = `75%`. This gives us `constrained 5%` (`25%` of `20%`) and `no-limit 15%` (`75%` of `20%`).

Adding these up to get the raw percentages gives us:

```
dynamic-range-limit-mix(standard 5%, constrained 20%, no-limit 25%)
```

The above percentages add up to `50%`, so they need to be doubled to get the final percentages. The computed value is therefore:

```
dynamic-range-limit-mix(standard 10%, constrained 40%, no-limit 50%)
```

Formal syntax

```
<dynamic-range-limit-mix()> =
  dynamic-range-limit-mix( [ <'dynamic-range-limit'> && <percentage [0,100]> ]# )

<dynamic-range-limit> =
  standard                    |
  no-limit                    |
  constrained                 |
  <dynamic-range-limit-mix()>
```

This syntax reflects the latest standard as per CSS Color HDR Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic usage

Consider an `<img>` element being used to embed an HDR image on a web page:

```
<img src="my-hdr-image.jpg" alt="my image" />
```

On HDR displays, the brightest areas of the image could prove to be jarring and uncomfortable to look at. To solve this problem, we could set the image's `dynamic-range-limit` property to `dynamic-range-limit-mix(standard 70%, no-limit 30%)`, which gives it a maximum luminance limit of only slightly brighter than HDR reference white:

```
img {
  dynamic-range-limit: dynamic-range-limit-mix(standard 70%, no-limit 30%);
}
```

You can see the `dynamic-range-limit` property in action in our dynamic-range-limit property demo, which includes an HDR image that can be hovered and focused to transition the `dynamic-range-limit` value. View the example live in a display capable of displaying HDR colors, and try it out.

See also

- `dynamic-range-limit`

<easing-function> CSS type

The **`<easing-function>`** CSS data type represents a mathematical function that describes the rate at which a value changes.

This transition between two values may be applied in different situations. It may be used to describe how fast values change during animations. This lets you vary the animation's speed over the course of its duration. You can specify an easing function for CSS transition and animation properties.

Syntax

```
/* Keyword linear easing function */
linear                /* linear(0, 1) */

/* Custom linear easing functions */
linear(0, 0.25, 1)
linear(0, 0.25 75%, 1)

/* Keyword cubic Bézier easing functions */
ease                  /* cubic-bezier(0.25, 0.1, 0.25, 1) */
ease-in               /* cubic-bezier(0.42, 0, 1, 1) */
ease-out              /* cubic-bezier(0, 0, 0.58, 1) */
ease-in-out           /* cubic-bezier(0.42, 0, 0.58, 1) */

/* Custom cubic Bézier easing function */
cubic-bezier(0.25, 0.1, 0.25, 1)

/* Keyword step easing functions */
step-start            /* steps(1, jump-start) */
step-end              /* steps(1, jump-end) */

/* Custom step easing functions */
steps(4, end)
steps(10, jump-both)
```

Values

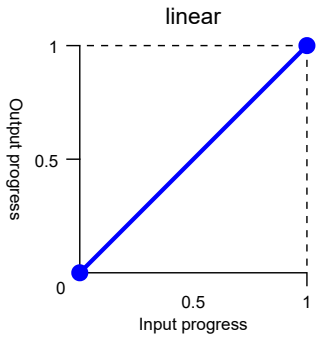
An `<easing-function>` can be one of the following types:

`<linear-easing-function>`

Creates transitions that progress at a constant rate. This function can be specified using one of the following:

`linear`

Specifies a constant rate of interpolation, with no change in the rate of progress throughout the duration (that is, no acceleration or deceleration). This keyword value is equivalent to `linear(0, 1)`. It can also be represented as `cubic-bezier(0, 0, 1, 1)`.



> **Note:** The `linear` keyword is always interpreted as `linear(0, 1)`, whereas the function `linear(0, 1)` is interpreted as `linear(0 0%, 1 100%)`.

`linear()`

Defines multiple points of progress using `<number>` values, with optional `<percentage>` values to control their timing.

`<cubic-bezier-easing-function>`

Creates smooth transitions with variable rates of change. This function can be specified using one of the following:

`ease`

Represents the easing function `cubic-bezier(0.25, 0.1, 0.25, 1)`. It indicates that the interpolation starts slowly, accelerates sharply, and then slows gradually towards the end. It is similar to the `ease-in-out` keyword, though it accelerates more sharply at the beginning.

`ease-in`

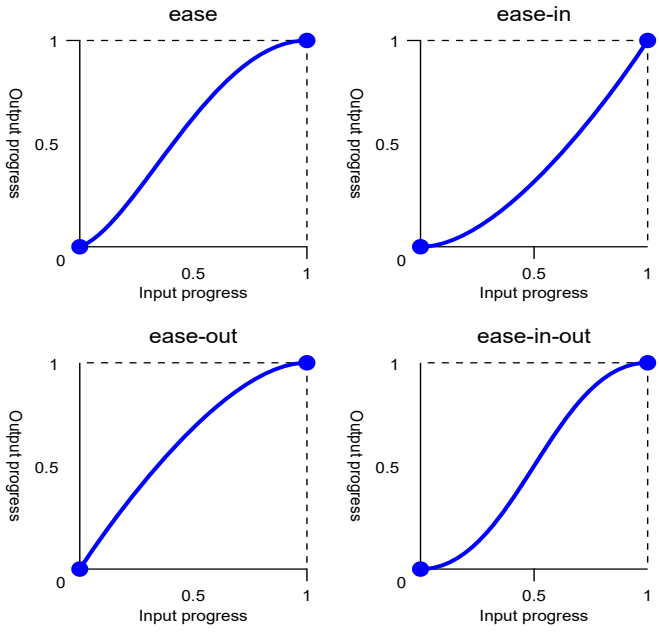
Represents the easing function `cubic-bezier(0.42, 0, 1, 1)`. It indicates that the interpolation starts slowly, then progressively speeds up until the end, at which point it stops abruptly.

`ease-out`

Represents the easing function `cubic-bezier(0, 0, 0.58, 1)`. It indicates that the interpolation starts abruptly and then progressively slows down towards the end.

`ease-in-out`

Represents the easing function `cubic-bezier(0.42, 0, 0.58, 1)`. It indicates that the interpolation starts slowly, speeds up, and then slows down towards the end. At the beginning, it behaves like the `ease-in` keyword; at the end, it is like the `ease-out` keyword.



`cubic-bezier()`

Defines a custom curve using four `<number>` values that specify the coordinates of two control points. The x-coordinates must be in the range `[0, 1]`.

`<step-easing-function>`

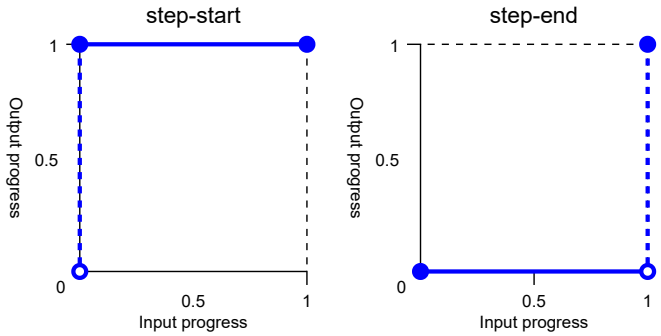
Creates stepped transitions that divides the animation into a set number of equal-length intervals, causing the animation to jump from one step to the next rather than transitioning smoothly. This function can be specified using one of the following:

`step-start`

Represents the easing function `steps(1, jump-start)` or `steps(1, start)`. It indicates that the interpolation jumps immediately to its final state, where it stays until the end.

`step-end`

Represents the easing function `steps(1, jump-end)` or `steps(1, end)`. It indicates that the interpolation stays in its initial state until the end, at which point it jumps directly to its final state.



`steps()`

Creates a stair-shaped curve using an `<integer>` to specify the number of intervals and an optional keyword to control the timing of jumps.

Formal syntax

```
<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

This syntax reflects the latest standard as per CSS Easing Functions Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing the easing functions

This example provides a comparison between the different easing functions using an animation. From the drop-down menu, you can select an easing function – there are a couple of keywords and some `cubic-bezier()` and `steps()` options. After selecting an option, you can start and stop the animation using the provided button.

HTML

```
<div>
  <div></div>
</div>
<ul>
  <li>
    <button class="animation-button">Start animation</button>
  </li>
  <li>
    <label for="easing-select">Choose an easing function:</label>
    <select id="easing-select">
      <option selected>linear</option>
      <option>linear(0, 0.5 50%, 1)</option>
      <option>ease</option>
      <option>ease-in</option>
      <option>ease-in-out</option>
      <option>ease-out</option>
```

```
      <option>cubic-bezier(0.1, -0.6, 0.2, 0)</option>
      <option>cubic-bezier(0, 1.1, 0.8, 4)</option>
      <option>steps(5, end)</option>
      <option>steps(3, start)</option>
      <option>steps(4)</option>
    </select>
  </li>
</ul>
```

CSS

```
body > div {
  position: relative;
  height: 100px;
}

div > div {
  position: absolute;
  width: 50px;
  height: 50px;
  background-color: blue;
  background-image: radial-gradient(
    circle at 10px 10px,
    rgb(25 255 255 / 80%),
    rgb(25 255 255 / 40%)
  );
  border-radius: 50%;
  top: 25px;
  animation: 1.5s infinite alternate;
}

@keyframes move-right {
  from {
    left: 10%;
  }

  to {
    left: 90%;
  }
}

li {
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 20px;
}
```

JavaScript

```
const selectElem = document.querySelector("select");
const startBtn = document.querySelector("button");
const divElem = document.querySelector("div > div");

startBtn.addEventListener("click", () => {
  if (startBtn.textContent === "Start animation") {
    divElem.style.animationName = "move-right";
    startBtn.textContent = "Stop animation";
    divElem.style.animationTimingFunction = selectElem.value;
  } else {
    divElem.style.animationName = "unset";
    startBtn.textContent = "Start animation";
  }
});

selectElem.addEventListener("change", () => {
  divElem.style.animationTimingFunction = selectElem.value;
});
```

Result

See also

- CSS easing functions module
- CSS animations module
- CSS transitions module
- `linear()` easing generator by Jake Archibald
- cubic-bezier.com by Lea Verou

cubic-bezier() CSS function

The **`cubic-bezier()`** CSS function creates a smooth transition using a cubic Bézier curve. As an `<easing-function>`, it can be used to smooth out the start and end of the interpolation.

Syntax

```
cubic-bezier(0.25, 0.1, 0.25, 1)
cubic-bezier(0.1, -0.6, 0.2, 0)
cubic-bezier(0, 0, 1, 1)
```

Parameters

The function accepts the following four parameters, which represent the coordinates of two control points:

`<x1>`

A `<number>` representing the x-axis coordinate of the first control point. It must be in the `[0, 1]` range.

`<y1>`

A `<number>` representing the y-axis coordinate of the first control point.

`<x2>`

A `<number>` representing the x-axis coordinate of the second control point. It must be in the `[0, 1]` range.

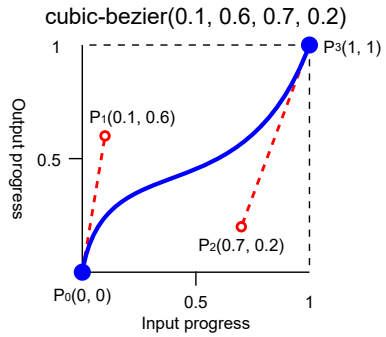
`<y2>`

A `<number>` representing the y-axis coordinate of the second control point.

Description

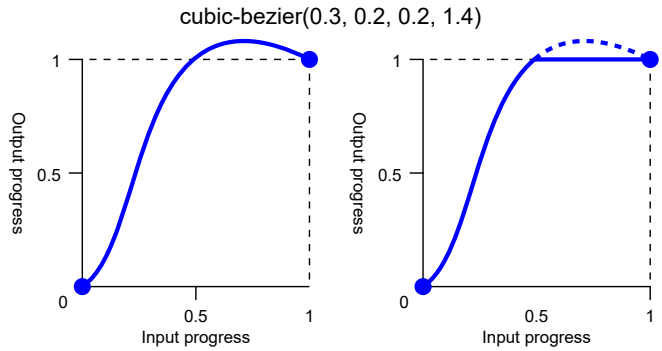
The cubic Bézier functions, often called "smooth" easing functions, correlate an input progress to an output progress, both expressed as `<number>`s, where `0.0` represents the initial state and `1.0` represents the final state. If the cubic Bézier curve is invalid, CSS ignores the whole property.

A cubic Bézier curve is defined by four points: P0, P1, P2, and P3. The points P0 and P3 represent the start and the end of the curve. In CSS, the start point P0 is fixed at `(0, 0)` and the end point P3 is fixed at `(1, 1)`, while intermediate points P1 and P2 are defined by the function parameters `(<x1>, <y1>)` and `(<x2>, <y2>)` respectively. The x-axis represents input progress and the y-axis represents output progress.



Not all cubic Bézier curves are suitable as easing functions because not all are mathematical functions; i.e., curves that for a given x-axis coordinate have zero or one value. With P0 and P3 fixed as defined by CSS, a cubic Bézier curve is a function, and is valid when the x-axis coordinates of P1 and P2 are both in the [0, 1] range.

Cubic Bézier curves with the P1 or P2 ordinate outside the `[0, 1]` range can cause the value to go farther than the final state and then return. In animations, this creates a kind of "bouncing" effect.



However, certain properties will restrict the output if it goes outside an allowable range. For example, a color component greater than `255` or smaller than `0` in `rgb()` will be clipped to the closest allowed value (`255` and `0`, respectively). Some `cubic-bezier()` values exhibit this property.

Formal syntax

```
<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )
```

This syntax reflects the latest standard as per CSS Easing Functions Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Bouncing effect

In this example, the red ball bounces out of the box when transitioned from its original position. This is because one of the P2 values, `2.3`, goes beyond the `[0, 1]` range.

```
<div tabindex="0">
  <span></span>
</div>
```

```
div {
  margin: 8px auto;
  padding: 8px;
  width: 256px;
  border-radius: 40px;
  background-color: wheat;
}

span {
  display: block;
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: tomato;
}

div:hover span,
div:focus span {
  translate: 192px 0;
}
```

```
span {
  transition: translate 0.3s cubic-bezier(0.3, 0.8, 0.3, 2.3);
}
```

Using the cubic-bezier() function

These cubic Bézier curves are valid for use in CSS:

```
/* The canonical Bézier curve with four <number> in the [0,1] range */
cubic-bezier(0.1, 0.7, 1.0, 0.1)

/* Using <integer> is valid because any <integer> is also a <number> */
cubic-bezier(0, 0, 1, 1)

/* Negative values for ordinates are valid, leading to bouncing effects */
cubic-bezier(0.1, -0.6, 0.2, 0)

/* Values greater than 1.0 for ordinates are also valid */
cubic-bezier(0, 1.1, 0.8, 4)
```

These cubic Bézier curve definitions are invalid:

```
/* Parameters must be numbers */
cubic-bezier(0.1, red, 1.0, green)

/* X coordinates must be in the [0, 1] range */
cubic-bezier(2.45, 0.6, 4, 0.1)

/* There must be exactly four parameters */
cubic-bezier(0.3, 2.1)

/* X coordinates must be in the [0, 1] range */
cubic-bezier(-1.9, 0.3, -0.2, 2.1)
```

See also

- Other easing functions: `linear()` and `steps()`
- CSS easing functions module
- cubic-bezier.com by Lea Verou

linear() CSS function

The **`linear()`** CSS function creates a transition curve that progresses uniformly between points. As an `<easing-function>`, it creates transitions where the interpolation occurs at a constant rate from beginning to end.

Syntax

```
linear(0, 1)
linear(0, 0.25, 1)
linear(0, 0.25 75%, 1)
linear(0, 0.5 25% 75%, 1)
```

Parameters

The function accepts two or more of the following values, which represent progress points in the animation timeline:

`<number>`

> Represents a point in time along the duration of the animation or transition. At least two values must be specified. The value `0` represents the start of the transition, and `1` represents the end. Values outside the `0` to `1` range are also allowed.

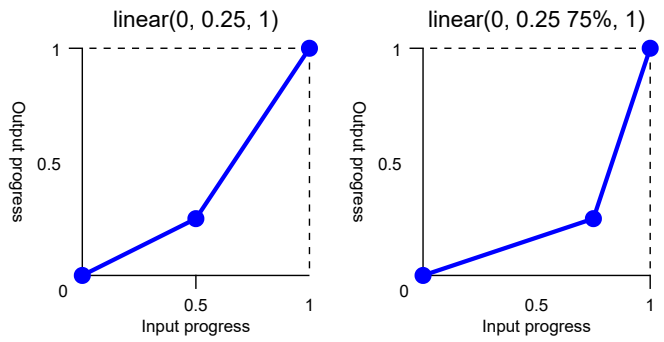
`<percentage>` Optional

> Indicates when the progress `<number>` is reached during the animation timeline. It can be specified after any `<number>` value except the first and last and can take up to two values. If two percentage values are specified, they define the length of the stop: the first percentage indicates the starting point and the second percentage indicates the ending point for that segment in the animation or transition. If no `<percentage>` value is specified, the progress values are distributed evenly along the timeline.

Description

The `linear()` function allows the approximation of complex animations and transitions by interpolating linearly between the specified points. A typical use of the `linear()` function is to provide many points to approximate any curve.

The `linear()` function creates transitions where progress occurs at a constant rate between specified points. For example, `linear(0, 0.25, 1)` has linear stops of `0`, `0.25`, and `1`. The animation starts at point `0`, moves linearly to `0.25`, and then continues linearly to point `1`. Since no percentage is specified, the same duration (`50%`) is used for each segment, that is, from `0` to `0.25` and from `0.25` to `1`.



By default, the stops are equidistant. For example, if there are five stops, they will occur at 0%, 25%, 50%, 75%, and 100% of the duration. You can use optional percentage values to provide finer control, defining when each progress value should occur and allowing for a more controlled progression of the transition.

Consider an animation with a duration time of 100 seconds and a change of 100 pixels. Let's look at a scenario where the easing of the animation is specified as `linear(0, 0.25 75%, 1)`. In this case, the animation progresses to 25 pixels (25% of its total change) in the first 75 seconds (75% of the duration). The last 75 pixels are applied in the remaining 25 seconds of the animation.

For the same animation, suppose the easing function is specified as `linear(0, 0.5 25% 75%, 1)`. Here, the animation reaches 50 pixels (50% of its total change) in 25 seconds (25% of the duration) and remains there for 50 seconds (75% - 25% of the duration). Then the last 50 pixels are applied in the remaining 25 seconds of the duration. Note that `linear(0, 0.5 25% 75%, 1)` is equivalent to `linear(0, 0.5 25%, 0.5 75%, 1)`.

Formal syntax

```
<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )
```

This syntax reflects the latest standard as per CSS Easing Functions Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using the linear() function

The following `linear()` functions are valid for use in CSS:

```
/* Three evenly distributed progress points */
linear(0, 0.25, 1)

/* Custom timing with percentage values */
linear(0, 0.5 25% 75%, 1)
```

The following `linear()` definitions are invalid:

```
/* At least two parameters are required */
linear(0.5)

/* Percentages must be in ascending order */
linear(0, 0.25 80%, 0.5 60%, 1)

/* Values must be numbers */
linear(start, middle, end)
```

See also

- Other easing functions: `cubic-bezier()` and `steps()`
- CSS easing functions module
- `linear()` easing generator by Jake Archibald

steps() CSS function

The **`steps()`** CSS function defines a transition that divides the input time into a specified number of intervals that are equal in length. This subclass of step functions are sometimes also called *staircase functions*.

Syntax

```
/* Different intervals */
steps(2, end)
steps(4, jump-end)
steps(12, end)

/* Different jump positions */
steps(3, jump-start)
steps(3, jump-end)
steps(3, jump-none)
steps(3, jump-both)
```

Parameters

The function accepts the following parameters:

`<integer>`

Represents the number of equidistant intervals or 'steps'. It must be a positive integer greater than `0` unless the second parameter is `jump-none`, in which case, it must be a positive integer greater than `1`.

`<step-position>`

Specifies when the jump between values occurs. If omitted, it defaults to `end`. The possible keyword values include:

`jump-start` or `start`

Indicates that the first step happens when the animation begins.

`jump-end` or `end`

Indicates that the last step happens when the animation ends.

`jump-none`

Indicates neither early nor late jumps happen.

`jump-both`

Indicates both early and late jumps happen.

Description

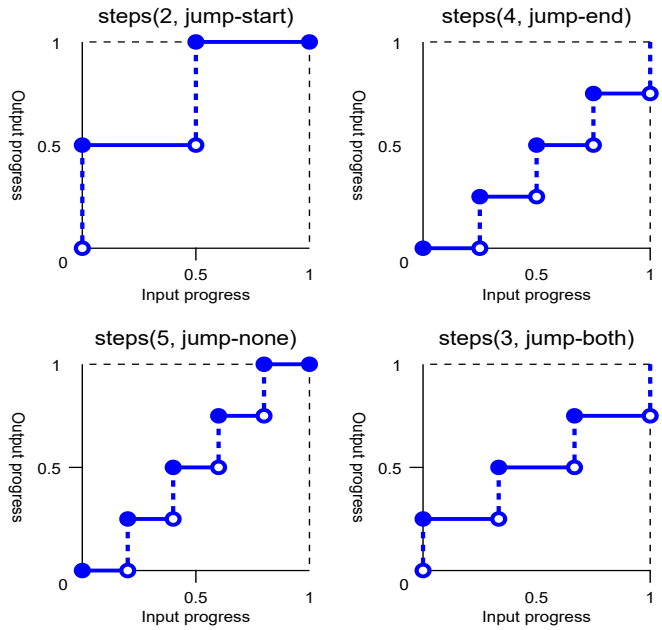
The `steps()` function divides the animation duration into equal intervals. For example, `steps(4, end)` divides the animation into four equal intervals, with values changing at the end of each interval except the last change which occurs at the animation's end.

If an animation contains multiple segments, the specified number of steps applies to each segment. For example, if an animation has three segments and uses `steps(2)`, there will be six steps in total, with two steps per segment.

The following image shows the effect of different `<step-position>` values when the jumps occur:

```
steps(2, jump-start)  /* Or steps(2, start) */
steps(4, jump-end)    /* Or steps(4, end) */
```

```
steps(5, jump-none)
steps(3, jump-both)
```



Formal syntax

```
<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

This syntax reflects the latest standard as per CSS Easing Functions Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using the steps() function

The following `steps()` functions are valid:

```
/* Five steps with jump at the end */
steps(5, end)

/* Two steps with jump at the start */
steps(2, start)

/* Using default second parameter */
steps(2)
```

The following `steps()` functions are invalid:

```
/* First parameter must be an <integer>, not a real value */
steps(2.0, jump-end)

/* Number of steps must be positive */
steps(-3, start)

/* Number of steps must be at least 1 */
steps(0, jump-none)
```

See also

- Other easing functions: `cubic-bezier()` and `linear()`
- CSS easing functions module
- Step function on Wikipedia

element() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`element()`** CSS function defines an `<image>` value generated from an arbitrary HTML element. This image is live, meaning that if the HTML element is changed, the CSS properties using the resulting value are automatically updated.

A particularly useful scenario for using this would be to render an image in an HTML `<canvas>` element, then use that as a background.

On Gecko browsers, you can use the non-standard `document.mozSetImageElement()` method to change the element being used as the background for a given CSS background element.

Syntax

```
element(id)
```

where:

id

The ID of an element to use as the background, specified using the HTML attribute *#id* on the element.

Formal syntax

```
<element()> =
  element( <id-selector> )

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

These examples work in builds of Firefox that support `-moz-element()`.

A somewhat realistic example

This example uses a hidden `<div>` as a background. The background element uses a gradient, but also includes text that is rendered as part of the background.

```
<div id="target-box">
  <p>This box uses the element with the #my-background ID as its background!</p>
</div>

<div id="background-container">
  <div id="my-background">
    <p>This text is part of the background. Cool, huh?</p>
  </div>
</div>
```

```
#target-box {
  width: 400px;
  height: 400px;
  background: -moz-element(#my-background) no-repeat;
}

#background-container {
  overflow: hidden;
  height: 0;
}

#my-background {
  width: 1024px;
  height: 1024px;
  background-image: linear-gradient(to right, red, orange, yellow, white);
}

#my-background p {
  transform-origin: 0 0;
  rotate: 45deg;
  color: white;
}
```

The `<div>` element with the ID "my-background" is used as the background for the content including the paragraph "This box uses the element with the #my-background ID as its background!".

Page Preview

This example based on Vincent De Oliveira's creates a preview of the `<div id="css-source">` inside `<div id="css-result">`.

HTML

```
<div id="css-source">
  <h1>Page Preview</h1>
</div>
<div id="css-result"></div>
```

CSS

```
#css-result {
  background: -moz-element(#css-source) no-repeat;
  width: 256px;
  height: 32px;
  background-size: 80%;
  border: dashed;
}
```

Result

See also

- `image()`
- `image-set()`
- `<image>`
- `<gradient>`
- `cross-fade()`
- `document.mozSetImageElement()`

env() CSS function

The **`env()`** CSS function can be used to insert the value of a user-agent defined environment variable into your CSS. Alternatively `env()` can be used to make dynamic values in external SVG files which updated using the `link-parameters` CSS property.

Syntax

```
/* Without a fallback value */
env(safe-area-inset-top);
env(titlebar-area-width);
env(viewport-segment-right 0 0);

/* With a fallback value */
env(safe-area-inset-right, 1em);
env(titlebar-area-y, 40px);
env(viewport-segment-width 0 0, 40%);
```

```
<svg viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <path fill="env(--color, black)" d="..." />
</svg>
```

Parameters

The `env( <environment-variable> | <dashed-ident>, <fallback> | <declaration-value> )` function accepts the following parameters:

`<environment-variable>`

A `<custom-ident>` specifying the name of the environment variable to be inserted. If the name provided represents an array-like environment variable, the name is followed by `<integer>` values identifying the specific instance the name is referencing. The case-sensitive environment variable name can be one of the following:

`safe-area-inset-top`, `safe-area-inset-right`, `safe-area-inset-bottom`, `safe-area-inset-left`

The safe distance from the top, right, bottom, or left inset edge of the viewport, defining where it is safe to place content into without risking it being cut off by the shape of a non-rectangular display. The four values form a rectangle, inside which all content is visible. The values are `0` if the viewport is a rectangle and no features — such as toolbars or dynamic keyboards — are occupying viewport space; otherwise, it is a `px` value greater than `0`.

`safe-area-max-inset-top`, `safe-area-max-inset-right`, `safe-area-max-inset-bottom`, `safe-area-max-inset-left`

The static maximum values of their dynamic `safe-area-inset-*` variable counterparts when all dynamic user interface features are retracted. While the `safe-area-inset-*` values change as the currently-visible content area changes, the `safe-area-max-inset-*` values are constants.

`titlebar-area-x`, `titlebar-area-y`, `titlebar-area-width`, `titlebar-area-height`

The dimensions of a visible `titlebar-area-*` area. These variables are available when using the `window-controls-overlay` `display_override` manifest field. The variables' values can be used to ensure content doesn't overlap window control buttons (that is, minimize, maximize, and close) with progressive web apps (PWA) installed on desktop devices.

`keyboard-inset-top`, `keyboard-inset-right`, `keyboard-inset-bottom`, `keyboard-inset-left`, `keyboard-inset-width`, `keyboard-inset-height`

The insets from the edge of the viewport and dimensions of the device's on-screen virtual keyboard. Defined in the VirtualKeyboard API.

`preferred-text-scale`

The user's preferred font scaling factor, a number set in browser or OS-level preferences. This can be used to size content proportionally to browser or OS-set font sizes.

`viewport-segment-width`, `viewport-segment-height`, `viewport-segment-top`, `viewport-segment-right`, `viewport-segment-bottom`, `viewport-segment-left`

The dimensions and offset positions of specific viewport segments. The `viewport-segment-*` keyword is followed by two space-separated `<integer>` values that indicate the segment's horizontal and vertical position, or indices. The viewport-segment keywords are only defined when the viewport is made up of two or more segments, as with foldable or hinged devices.

`<dashed-ident>`

A `<dashed-ident>` is a user-defined variable that can be used as an identifier in the `param()` CSS function to update the value.

`<fallback>` Optional

A fallback value to be inserted if the environment variable referenced in the first argument does not exist. Everything after the first comma is deemed to be the fallback value. This can be a single value, another `env()` function, or a comma-separated list of values.

`<declaration_value>` Optional

A `<declaration_value>` is the default value of the SVG attribute being set dynamically. If the `<declaration-value>` is omitted, it represents an empty value.

Description

The `env()` function is used to insert the value of a globally-scoped, user-agent-defined environment variable into your CSS. The `env()` function can be used as a property value or in place of any part of a property value or descriptor (for example, in Media query rules).

The function accepts an `<environment-variable>` as its first argument. This is a case-sensitive `<custom-ident>` equal to the name of the environment variable to be substituted, but it can also include additional space-separated values if required. For example, `env(viewport-segment-width 0 0)` would return the width of the top or left segment in the case of a device with multiple viewport segments.

The second argument, if provided, is the fallback value, which is used if the environment variable referenced in the first argument is not supported or doesn't exist. The fallback can be another environment variable, even with its own fallback.

The syntax of the fallback is similar to the fallback syntax of the `var()` function used to insert CSS custom properties in that it allows for multiple commas. Anything between the first comma and the end of the function is considered the fallback value. However, if the `env()` function is used within a property value or descriptor that doesn't include commas, a fallback value that includes commas will not be valid.

A property or descriptor containing a syntactically valid `env()` function is assumed to be valid at parse time, when the browser first reads and interprets the downloaded CSS text. It is only syntax-checked at compute time, after each `env()` function has been substituted with its browser-provided value (or the fallback value if the environment variable passed as the first parameter is not a recognized environment variable name). If the value is invalid and no fallback is provided, the property or descriptor containing the `env()` function is invalid at computed-value time.

When an `env()` substitution is invalid, and an invalid fallback is included, or the fallback is omitted, the declaration is not ignored. Instead, the initial or inherited value of the property is used. The property is set to a new value, but it may not be the expected one.

Use cases

Originally provided by the iOS browser to allow developers to place their content in a safe area of the viewport, and not be obscured by device notches or rounded corners, the `safe-area-inset-*` values can be used to help ensure content is visible to viewers. This feature was later expanded beyond its initial purpose to enable use cases such as stopping device notifications from covering up some of the app user interface.

Another use case for `env()` variables is for desktop Progressive web apps (PWAs) that use the Window Controls Overlay feature to take advantage of the full application window surface area. Using the `titlebar-area-*` values, developers can position elements where the title bar would have been and ensure content is not obscured by window control buttons.

The `viewport-segment-*` variable names can be used to set your containers to fit neatly into the available segments of a multi-viewport-segment device such as a hinged or foldable device. The integers following the `viewport-segment-*` name indicate which segment of the multiple segments the environment variable is referencing.

The `preferred-text-scale` variable can be used to size website text or other UI features proportionally to browser or OS-set font sizes. For example, you could set the body font size to be a percentage based on the user-defined text scale:

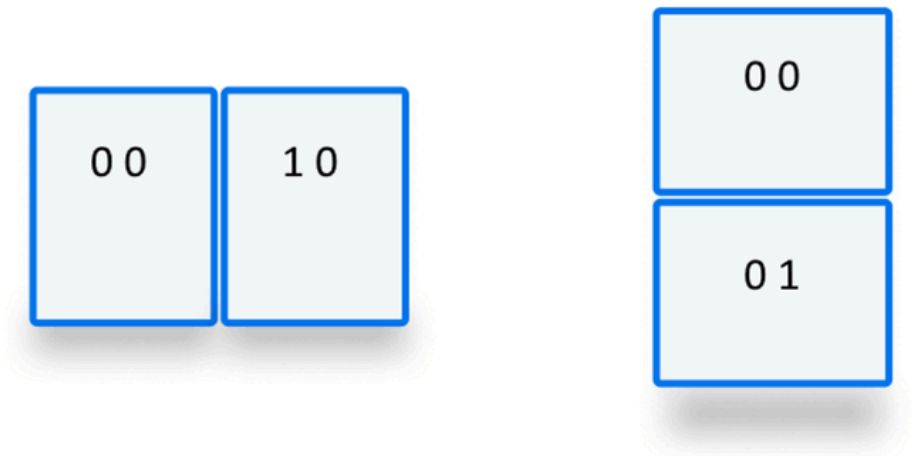
```
body {
  font-size: calc(100% * env(preferred-text-scale));
}
```

Sizes can also be set to be proportional to browser or OS-level font size by including `<meta name="text-scale" content="scale">` inside the document `<head>`. The `<meta>` tag should be used in favor of the `env(preferred-text-scale)` when possible, as the `<meta>` tag is supported across a wider range of platforms and is also simpler to use.

> **Warning:** Be careful using `env(preferred-text-scale)` when `<meta name="text-scale" content="scale">` is set as this will result in text scaling being applied twice when combined with relative sizes, such as `em` and `rem`. For example, when the `<meta>` is set, a declaration such as `font-size: calc(2rem * env(preferred-text-scale))` will cause small font sizes to be made even smaller and large font sizes larger.

Names followed by integers

When the environment variable is array-like, meaning the name may reference more than once value, such as is the case with devices with multiple viewport segments, the `<environment-variable>` parameter includes both the name of the variable and the indices of the specific instance of the variable the function is referencing. For example, in the case of the `viewport-segment-*` variables, the variable names are passed to the `env()` function along with two integers indicating the indices of the segment to return the value for. These values are both integers of `0` or greater. The first integer represents the horizontal index of the segment, with `0` being the left-most segment, and the second value represents the vertical index of the segment, with `0` representing the bottom-most segment:



- In a horizontal side-by-side layout, the left segment is represented by `0 0`, and the right segment is represented by `1 0`.
- In a vertical top-to-bottom layout, the top segment is represented by `0 0`, and the bottom segment is represented by `0 1`.
- In devices with more than two segments, the numbers may be greater. For example, a device with three horizontal segments may have the center segment represented by `1 0`, and the right-hand segment represented by `2 0`.

For example, the following returns the width of the right-hand segment on a two-segment foldable device where the segments are oriented horizontally:

```
env(viewport-segment-width 1 0)
```

See the Viewport segment API demo for a full working demo (source code). Also check out Using the Viewport Segments API for a full demo explanation.

Formal syntax

```
<env()> =
  env( <custom-ident> <integer [0,∞]>* , <declaration-value>? )

<integer> =
  <number-token>
```

This syntax reflects the latest standard as per CSS Environment Variables Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using env() to ensure buttons are not obscured by device UI

In the following example, `env()` is used to ensure that fixed app toolbar buttons are not obscured by device notifications appearing at the bottom of the screen. On the desktop `safe-area-inset-bottom` is `0`. However, in devices that display notifications at the bottom of the screen, such as iOS, it contains a value that leaves space for the notification to display. This can then be used in the value for `padding-bottom` to create a gap that appears natural on that device.

HTML

We have a `<main>` section containing a fake application and a `<footer>` containing two `<button>` elements:

```
<main>Main content of app here</main>
<footer>
  <button>Go here</button>
  <button>Or here</button>
</footer>
```

CSS

Using CSS flexible box layout, we create a footer that is only as tall as it needs to be, while the main section containing the application fills up the rest of the viewport:

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  font: 1em system-ui;
}
```

```
main {
  flex: 1;
  background-color: #eeeeee;
  padding: 1em;
}

footer {
  flex: none;
  display: flex;
  gap: 1em;
  justify-content: space-evenly;
  background: black;
}

button {
  padding: 1em;
  background: white;
  color: black;
  margin: 0;
  width: 100%;
  border: none;
  font: 1em system-ui;
}
```

We set `position: sticky` to stick the footer to the bottom of the viewport. We then use the `padding` shorthand to add padding to the footer. We include the value of the `safe-area-inset-bottom` environment value to an initial `1em` of bottom padding. A larger black area will display on devices that have a positive value for this variable, ensuring the buttons in the footer are never obscured.

```
footer {
  position: sticky;
  bottom: 0;

  padding: 1em 1em calc(1em + env(safe-area-inset-bottom));
}
```

Results

Using a fallback value

This example makes use of the optional second parameter of `env()`, which provides a fallback value in case the environment variable is not available.

HTML

We include a paragraph of text:

```
<p>
  If the <code>env()</code> function is supported in your browser, this
  paragraph's text will have 50px of padding between it and the left border —
  but not the top, right and bottom. This is because the accompanying CSS is the
  equivalent of <code>padding: 0 0 0 50px</code>, because, unlike other CSS
  properties, user agent property names are case-sensitive.
</p>
```

CSS

We set a `width` of `300px` and a `border`. We then add `padding`, using the `env()` function with a fallback for the size of the padding on each side. We intentionally set an invalid value for the left padding (remember, environment variable names are case-sensitive), to demonstrate the use of the fallback value.

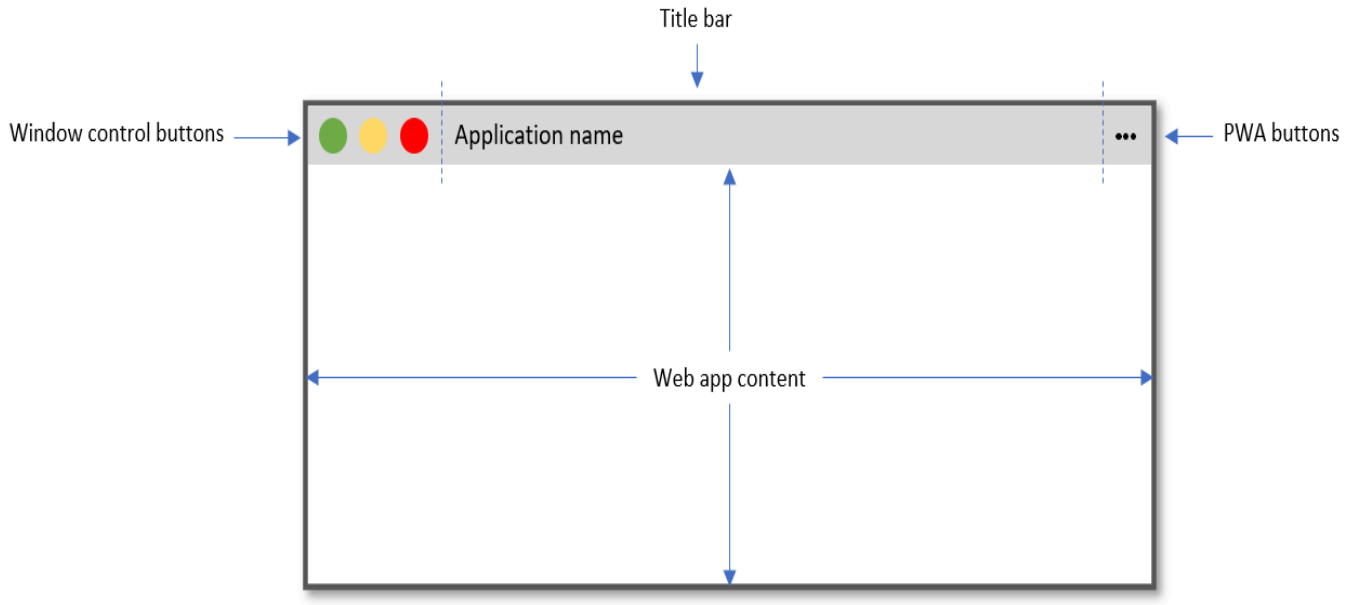
```
p {
  width: 300px;
  border: 2px solid red;
  padding: env(safe-area-inset-top, 50px) env(safe-area-inset-right, 50px)
    env(safe-area-inset-bottom, 50px) env(SAFE-AREA-INSET-LEFT, 50px);
}
```

Results

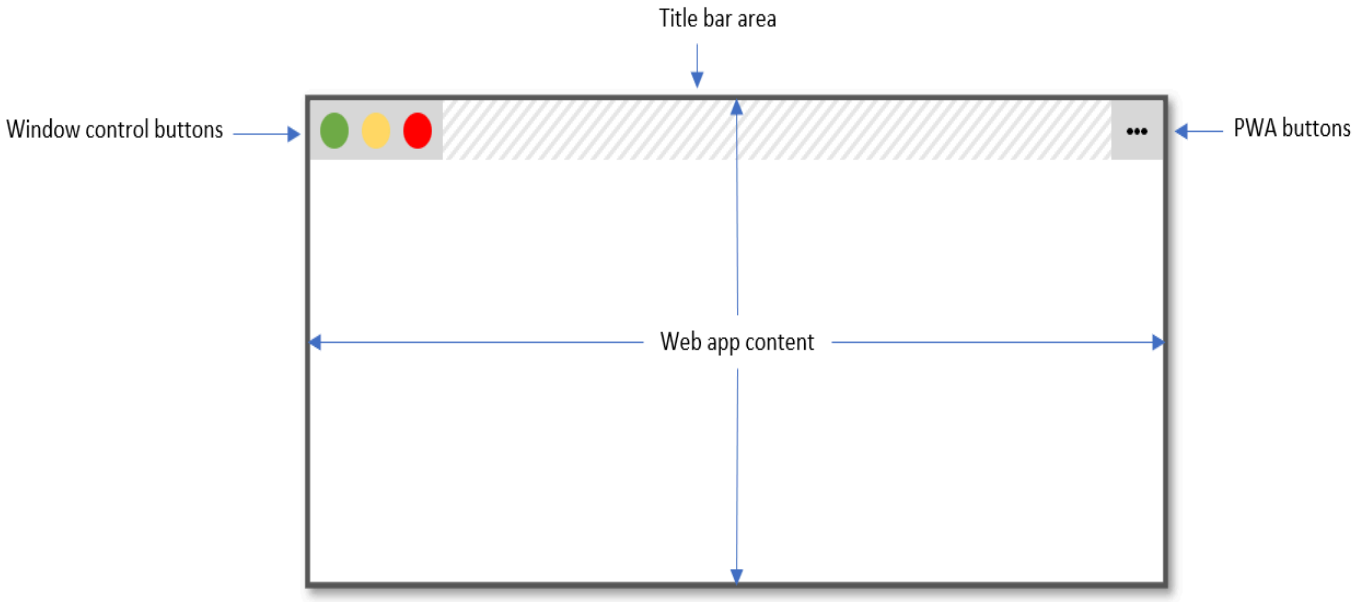
Using env() to ensure content is not obscured by window control buttons in desktop PWAs

In the following example, `env()` ensures that content displayed in a desktop Progressive Web App that uses the Window Controls Overlay API is not obscured by the operating system's window control buttons. The `titlebar-area-*` values define a rectangle where the title bar would normally have been displayed. On devices that do not support the Window Controls Overlay feature, such as mobile devices, the fallback values are used.

Here is what a PWA installed on a desktop device normally looks like:



With the Window Controls Overlay feature, the web content covers the whole app window surface area, with the window controls and PWA buttons displayed as overlays:




```
<header>Title of the app here</header>
<main>Main content of app here</main>
```

```
header {
  position: fixed;
  left: env(titlebar-area-x);
  top: env(titlebar-area-y);
  width: env(titlebar-area-width);
  height: env(titlebar-area-height);
}

main {
  margin-top: env(titlebar-area-height);
}
```

> **Note:** Using `position:fixed` makes sure the header does not scroll with the rest of the content, and instead stays aligned with the window control buttons, even on device/browsers that support elastic overscroll (also known as rubber banding).

Viewport segments

The Viewport segment API demo and Using the Viewport Segments API guide provides a demonstration and explanation of using the `env()` function with the `viewport-segments-*` environment variables.

See also

- Using environment variables
- CSS environment variables module
- `var()`
- CSS custom properties for cascading variables module
- Custom properties (`--*`): CSS variables

- `<meta name="text-scale">`
- Using CSS custom properties (variables)
- Viewport Segments API
- Customize the window controls overlay of your PWA's title bar
- Display content in the title bar
- Breaking Out of the Box

exp() CSS function

The **`exp()`** CSS function is an exponential function that takes a number as an argument and returns the mathematical constant `e` raised to the power of the given number.

The mathematical constant `e` is the base of natural logarithms, and is approximately `2.718281828459045`.

The `exp(number)` function contains a calculation which returns the same value as `pow(e, number)`.

Syntax

```
/* A <number> value */
width: calc(100px * exp(-1)); /* 100px * 0.367879441171442 = 36.8px */
width: calc(100px * exp(0)); /* 100px * 1 = 100px */
width: calc(100px * exp(1)); /* 100px * 2.718281828459045 = 271.8px */
```

Parameters

The `exp(number)` function accepts only one value as its parameter.

`number`

A calculation which resolves to a `<number>`. Representing the value to be raised by a power of `e`.

Return value

Returns a non-negative `<number>` representing e^{number}, which is the result of calculating `e` raised to the power of `number`.

- If `number` is `-Infinity`, the result is `0`.
- If `number` is `0`, the result is `1`.
- If `number` is `1`, the result is `e` (i.e., `2.718281828459045`).
- If `number` is `Infinity`, the result is `Infinity`.

Formal syntax

```
<exp()> =
  exp( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>       |
  <dimension>    |
  <percentage>   |
  <calc-keyword> |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotate elements

The `exp()` function can be used to `rotate` elements as it returns a `<number>`.

HTML

```
<div class="box box-1"></div>
<div class="box box-2"></div>
<div class="box box-3"></div>
<div class="box box-4"></div>
<div class="box box-5"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

```
div.box {
  width: 100px;
  height: 100px;
  background: linear-gradient(orange, red);
}
div.box-1 {
  transform: rotate(calc(1turn * exp(-1))); /* 0.3678794411714423turn */
}
div.box-2 {
  transform: rotate(calc(1turn * exp(-0.75))); /* 0.4723665527410147turn */
}
```

```
div.box-3 {
  transform: rotate(calc(1turn * exp(-0.5))); /* 0.6065306597126334turn */
}
div.box-4 {
  transform: rotate(calc(1turn * exp(-0.25))); /* 0.7788007830714049turn */
}
div.box-5 {
  transform: rotate(calc(1turn * exp(0))); /* 1turn */
}
```

Result

Scale headings by fixed ratio

The `exp()` function can be useful for strategies like CSS modular scale, which relates all the font-sizes on a page to each other by a fixed ratio.

HTML

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

CSS

```
h1 {
  font-size: calc(1rem * exp(1.25)); /* 3.4903429574618414rem */
}
h2 {
  font-size: calc(1rem * exp(1)); /* 2.718281828459045rem */
}
h3 {
  font-size: calc(1rem * exp(0.75)); /* 2.117000016612675rem */
}
h4 {
  font-size: calc(1rem * exp(0.5)); /* 1.6487212707001282rem */
}
h5 {
  font-size: calc(1rem * exp(0.25)); /* 1.2840254166877414rem */
}
h6 {
  font-size: calc(1rem * exp(0)); /* 1rem */
}
```

Result

See also

- `pow()`
- `sqrt()`
- `hypot()`
- `log()`

<filter-function> CSS type

The **`<filter-function>`** CSS data type represents a graphical effect that can change the appearance of an input image. It is used in the `filter` and `backdrop-filter` properties.

Syntax

The `<filter-function>` data type is specified using one of the filter functions listed below. Each function requires an argument which, if invalid, results in no filter being applied.

`blur()`

Blurs the image.

`brightness()`

Makes the image brighter or darker.

`contrast()`

Increases or decreases the image's contrast.

`drop-shadow()`

Applies a drop shadow behind the image.

`grayscale()`

Converts the image to grayscale.

`hue-rotate()`

Changes the overall hue of the image.

`invert()`

Inverts the colors of the image.

`opacity()`

Makes the image transparent.

`saturate()`

Super-saturates or desaturates the input image.

`sepia()`

Converts the image to sepia.

Formal syntax

```
<filter-function> =
  <blur()>         |
  <brightness()>   |
  <contrast()>     |
  <drop-shadow()>  |
  <grayscale()>    |
  <hue-rotate()>   |
  <invert()>       |
  <opacity()>      |
  <sepia()>        |
  <saturate()>

<blur()> =
  blur( <length>? )

<brightness()> =
  brightness( [ <number> | <percentage> ]? )

<contrast()> =
  contrast( [ <number> | <percentage> ]? )

<drop-shadow()> =
  drop-shadow( [ <color>? && <length>{2,3} ] )

<grayscale()> =
  grayscale( [ <number> | <percentage> ]? )

<hue-rotate()> =
  hue-rotate( [ <angle> | <zero> ]? )

<invert()> =
  invert( [ <number> | <percentage> ]? )

<opacity()> =
  opacity( [ <number> | <percentage> ]? )

<sepia()> =
  sepia( [ <number> | <percentage> ]? )

<saturate()> =
  saturate( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Filter function comparison

This example provides a graphic, a select menu to allow you to choose between the different types of filter function, and a slider to allow you to vary the values used inside the filter function. Updating the controls updates the filter effect in real time, allowing you to investigate the effects of different filters.

The dropdown selects the function name, and the slider sets the parameter value for that function. For `drop-shadow`, the value is used for both the horizontal and vertical offsets, and the radius is set to half the value.

```
<div></div>
<ul>
  <li>
    <label for="filter-select">Choose a filter function:</label>
    <select id="filter-select">
      <option selected>blur</option>
      <option>brightness</option>
      <option>contrast</option>
      <option>drop-shadow</option>
      <option>grayscale</option>
      <option>hue-rotate</option>
      <option>invert</option>
      <option>opacity</option>
      <option>saturate</option>
      <option>sepia</option>
    </select>
  </li>
  <li><input type="range" /><output></output></li>
  <li>
    <p>Current value: <code></code></p>
  </li>
</ul>
```

```
div {
  width: 100%;
  height: 512px;
  background-image: url("https://mdn.github.io/shared-assets/images/examples/fx-nightly-512.png");
  background-repeat: no-repeat;
  background-position: center center;
}

li {
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 20px;
}

input {
  width: 60%;
}

output {
  width: 5%;
  text-align: center;
}

select {
```

```
  width: 40%;
  margin-left: 2px;
}
```

```
const selectElem = document.querySelector("select");
const divElem = document.querySelector("div");
const slider = document.querySelector("input");
const output = document.querySelector("output");
const curValue = document.querySelector("p code");

selectElem.addEventListener("change", () => {
  setSlider(selectElem.value);
  setDiv(selectElem.value);
});

slider.addEventListener("input", () => {
  setDiv(selectElem.value);
});

function setSlider(filter) {
  switch (filter) {
    case "blur":
      slider.value = 0;
      slider.min = 0;
      slider.max = 30;
      slider.step = 1;
      slider.setAttribute("data-unit", "px");
      break;
    case "brightness":
    case "contrast":
    case "saturate":
      slider.value = 1;
      slider.min = 0;
      slider.max = 4;
      slider.step = 0.05;
      slider.setAttribute("data-unit", "");
      break;
    case "drop-shadow":
      slider.value = 0;
      slider.min = -20;
      slider.max = 40;
      slider.step = 1;
      slider.setAttribute("data-unit", "px");
      break;
    case "opacity":
      slider.value = 1;
      slider.min = 0;
      slider.max = 1;
      slider.step = 0.01;
      slider.setAttribute("data-unit", "");
      break;
    case "grayscale":
    case "invert":
    case "sepia":
      slider.value = 0;
      slider.min = 0;
      slider.max = 1;
      slider.step = 0.01;
      slider.setAttribute("data-unit", "");
      break;
    case "hue-rotate":
      slider.value = 0;
      slider.min = 0;
      slider.max = 360;
      slider.step = 1;
      slider.setAttribute("data-unit", "deg");
      break;
    default:
      console.error("Unknown filter set");
  }
}

function setDiv(filter) {
  const unit = slider.getAttribute("data-unit");
  const offset = `${Math.round(slider.value)}${unit}`;
  const radius = `${Math.round(Math.abs(slider.value / 2))}${unit}`;
  divElem.style.filter =
    filter === "drop-shadow"
      ? `${selectElem.value}(${offset} ${offset} ${radius})`
      : `${selectElem.value}(${slider.value}${unit})`;

  updateOutput();
  updateCurValue();
}

function updateOutput() {
  output.textContent = slider.value;
}

function updateCurValue() {
  curValue.textContent = `filter: ${divElem.style.filter}`;
}

setSlider(selectElem.value);
setDiv(selectElem.value);
```

See also

- Properties that use this data type: `filter` and `backdrop-filter`

blur() CSS function

The **`blur()`** CSS function applies a Gaussian blur to the input image. Its result is a `<filter-function>`.

Try it

```
filter: blur(0);
```

```
filter: blur(4px);
```

```
filter: blur(1.5rem);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
blur()         /* No effect */
blur(0)        /* No effect */

blur(8px)      /* Blur with 8px radius */
blur(1.17rem)  /* Blur with 1.17rem radius */
```

Parameters

`<length>` Optional

> Specifies the radius of the blur. It defines the value of the standard deviation to the Gaussian function, that is, how many pixels on the screen blend into each other. Therefore, a larger value will create more blur. A value of `0` leaves the input unchanged. The initial value for interpolation is `0`. Percentage values are invalid. The default value is `0`.

SVG filter

The SVG `<feGaussianBlur>` filter element can also be used to blur content. The filter's `stdDeviation` attribute accepts up to two values enabling creating more complex blur values. To create an equivalent blur, we include one value for `stdDeviation`. This SVG effect can then be referenced by ID:

```
<svg role="none">
  <filter id="blur11">
    <feGaussianBlur stdDeviation="1.1" edgeMode="duplicate" />
  </filter>
</svg>
```

The following declarations produce the same effect:

```
filter: blur(1.1px);
filter: url("#blur11"); /* with embedded SVG */
filter: url("folder/fileName.svg#blur11"); /* external svg filter definition */
```

Formal syntax

```
<blur()> =
  blur( <length>? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

This example shows three images: the image with a `blur()` filter function applied, the image with the equivalent SVG blur function applied, and the original images for comparison:

```
.filter {
  filter: blur(3.5px);
}
```

```
<svg role="img" aria-label="Flag">
  <filter id="blur">
    <feGaussianBlur stdDeviation="3.5" edgeMode="duplicate" />
  </filter>
  <image
    href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    xlink:href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
    filter="url('#blur')" />
</svg>
```

```
svg:not([height]) {
  display: none;
}

th,
td {
  padding: 5px;
}
```

```
<table>
  <thead>
    <tr>
      <th>Live example</th>
      <th>SVG Equivalent</th>
      <th>Original image</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
```

```
        <img
          class="filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <svg id="svg" height="220" width="220" overflow="visible">
          <filter id="svgBlur">
            <feGaussianBlur stdDeviation="3.5" />
          </filter>
          <image
            href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
            xlink:href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
            filter="url('#svgBlur')" />
        </svg>
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

See also

- CSS filter effects module
- The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:
  - `brightness()`
  - `contrast()`
  - `drop-shadow()`
  - `grayscale()`
  - `hue-rotate()`
  - `invert()`
  - `opacity()`
  - `saturate()`
  - `sepia()`

brightness() CSS function

The **`brightness()`** CSS `<filter-function>` applies a linear multiplier value on an element or an input image, making the image appear brighter or darker.

Try it

```
filter: brightness(1);
```

```
filter: brightness(1.75);
```

```
filter: brightness(50%);
```

```
filter: brightness(0);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
brightness(amount)
```

Values

`amount` Optional

> Brightness specified as a `<number>` or a `<percentage>`. A value less than `100%` darkens the input image or element, while a value over `100%` brightens it. A value of `0%` creates a completely black image or element, while a value of `100%` leaves the input unchanged. Other values between `0%` to `100%` have a linear multiplier effect. Values greater than `100%` are allowed, providing brighter results. The initial value for interpolation is `1`. Negative values are not allowed. The default value is `1`.

The following are pairs of equivalent values:

```
brightness(0)   /* Brightness is reduced to zero, so input turns black */
brightness(0%)

brightness(0.4) /* Brightness of input is reduced to 40%, so input is 60% darker */
brightness(40%)

brightness()     /* Brightness of input is not changed */
brightness(1)
brightness(100%)

brightness(2)   /* Brightness of input is doubled */
brightness(200%)
```

Formal syntax

```
<brightness()> =
  brightness( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Applying brightness using the backdrop-filter property

This example shows how to apply the `brightness()` filter to a paragraph via the `backdrop-filter` CSS property.

CSS

```
.container {
  background: url("be_fierce.jpg") no-repeat right / contain #d4d5b2;
}
p {
  backdrop-filter: brightness(150%);
  text-shadow: 2px 2px white;
}
```

```
.container {
  padding: 5rem 3rem 1rem;
  width: 30rem;
}
p {
  padding: 0.5rem;
  color: black;
  font-size: 2rem;
  font-family: sans-serif;
}
```

```
<div class="container">
  <p>
    Text on images can be illegible and inaccessible even with a drop shadow.
  </p>
</div>
```

Result

In this example, the colors in the area behind the `<p>` element shift linearly. If the `backdrop-filter` property was set to `brightness(0%)`, the `<div>` area with the `<p>` element would have been black and hidden the image behind. At `brightness(100%)`, the `<div>` area color would be the same as the input `#d4d5b2`, and the image behind would be completely transparent. With the brightness set to `150%` as in this example, the colors in the image behind are getting hidden by the brightness of the `<div>` element.

Applying brightness using the filter property

In this example, a `brightness()` filter is applied to the entire element, including content, border, and background image via the `filter` CSS property. The result shows three variations of different brightness values.

```
p:first-of-type {
  filter: brightness(50%);
}
p:last-of-type {
  filter: brightness(200%);
}
```

```
p {
  text-shadow: 2px 2px blue;
  background-color: magenta;
  color: palegoldenrod;
  border: 1em solid rebeccapurple;
  box-shadow:
    inset -5px -5px red,
    5px 5px yellow;
  padding: 0.25rem;
  font-size: 1.25rem;
  font-family: sans-serif;
  width: 85vw;
}
```

```
<p>This paragraph has reduced brightness.</p>
<p>This paragraph has normal brightness.</p>
<p>This paragraph has increased brightness.</p>
```

Applying brightness using the url() SVG brightness filter

The SVG `<filter>` element is used to define custom filter effects that can then be referenced by `id`. The `<filter>` element's `<feComponentTransfer>` primitive enables pixel-level color remapping.

In this example, to create a filter that darkens the content on which it is applied by 25% (i.e., 75% of the original brightness), the `slope` attribute is set to `0.75`. We can then reference the filter by `id`.

Given the following:

```
<svg role="none">
  <filter id="darken25" color-interpolation-filters="sRGB">
    <feComponentTransfer>
      <feFuncR type="linear" slope="0.75" />
      <feFuncG type="linear" slope="0.75" />
      <feFuncB type="linear" slope="0.75" />
    </feComponentTransfer>
  </filter>
</svg>
```

The following declarations produce similar effects:

```
filter: brightness(75%);
filter: url("#darken25"); /* with embedded SVG */
filter: url("folder/fileName.svg#darken25"); /* external svg filter definition */
```

In the images below, the first one has a `brightness()` filter function applied, the second one has a similar SVG brightness function applied, and the third is the original image for comparison.

```
<table>
  <thead>
    <tr>
```

```
      <th>Live example</th>
      <th>SVG Equivalent</th>
      <th>Original image</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <img
          class="css-filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="darkened pride flag" />
      </td>
      <td>
        <img
          class="svg-filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="darkened pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

```
.css-filter {
  filter: brightness(0.75);
}
.svg-filter {
  filter: url("#darken25");
}

th,
td {
  padding: 5px;
}
svg:not(:root) {
  display: none;
}
```

See also

- CSS filter effects module
- The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:
  - `blur()`
  - `contrast()`
  - `drop-shadow()`
  - `grayscale()`
  - `hue-rotate()`
  - `invert()`
  - `opacity()`
  - `saturate()`
  - `sepia()`

contrast() CSS function

The **`contrast()`** CSS function adjusts the contrast of the input image. Its result is a `<filter-function>`.

Try it

```
filter: contrast(1);
```

```
filter: contrast(1.75);
```

```
filter: contrast(50%);
```

```
filter: contrast(0);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
contrast(amount)
```

Values

`amount` Optional

> The contrast of the result, specified as a `<number>` or a `<percentage>`. A value under `100%` decreases the contrast, while a value over `100%` increases it. A value of `0` or `0%` will create an image that is completely gray, while a value of `1` or `100%` leaves the input unchanged. Negative values are not allowed. The initial value for interpolation is `1`. The default value is `1`.

The following are pairs of equivalent values:

```
contrast(0)    /* Completely gray */
contrast(0%)

contrast(0.65) /* 65% contrast */
```

```
contrast(65%)

contrast()     /* No effect */
contrast(1)
contrast(100%)

contrast(2)    /* Double contrast */
contrast(200%)
```

Formal syntax

```
<contrast()> =
  contrast( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

With the backdrop-filter property

This example applies a `contrast()` filter via the `backdrop-filter` CSS property to the paragraph and monospaced text, color shifting to the area behind the `<p>` and `<code>`.

```
.container {
  background: url("unity_for_the_people.jpg") no-repeat center / contain #333399;
}
p {
  backdrop-filter: contrast(0.5);
}
code {
  backdrop-filter: contrast(0.15);
}
```

```
.container {
  padding: 3rem;
  width: 30rem;
}
p {
  padding: 0.5rem;
  color: white;
  font-family: sans-serif;
}
```

```
<div class="container">
  <p>
    Always ensure there is enough contrast between text and all background
    colors. If you think your text may land on top of a background image,
    include a <code>backdrop-filter</code>. Reducing the contrast of background
    colors with the <code>contrast()</code> filter may improve legibility but
    does not guarantee accessibility.
  </p>
</div>
```

With the filter property

This example applies a `contrast()` filter via the `filter` CSS property, changing contrast by shifting colors of the entire element, including content, border, background, and shadows.

```
p:first-of-type {
  filter: contrast(30%);
}
p:last-of-type {
  filter: contrast(300%);
}
```

```
p {
  text-shadow: 2px 2px blue;
  background-color: magenta;
  color: palegoldenrod;
  border: 1em solid rebeccapurple;
  box-shadow:
    inset -5px -5px red,
    5px 5px yellow;
  padding: 0.25rem;
  font-size: 1.25rem;
  font-family: sans-serif;
  width: 85vw;
}
```

```
<p>This paragraph has reduced contrast.</p>
<p>This paragraph has normal contrast.</p>
<p>This paragraph has increased contrast.</p>
```

With url() and the SVG contrast filter

The SVG `<filter>` element is used to define custom filter effects that can then be referenced by `id`. The `<filter>`'s `<feComponentTransfer>` primitive enables pixel-level color remapping. Given the following:

```
<svg
  xmlns="http://www.w3.org/2000/svg"
  id="svg"
  viewBox="0 0 240 151"
  height="0"
  width="0"
  overflow="visible"
  color-interpolation-filters="sRGB">
  <filter id="contrast">
    <feComponentTransfer>
      <feFuncR type="linear" slope="2" intercept="-0.5" />
      <feFuncG type="linear" slope="2" intercept="-0.5" />
      <feFuncB type="linear" slope="2" intercept="-0.5" />
    </feComponentTransfer>
```

```
  </filter>
</svg>
```

These values produce the same results:

```
filter: contrast(200%);
filter: url("#contrast"); /* with embedded SVG */
filter: url("folder/fileName.svg#contrast"); /* external svg filter definition */
```

This example shows three images: the image with a `contrast()` filter function applied, the image with an equivalent `url()` filter applied, and the original images for comparison:

```
<table>
  <thead>
    <tr>
      <th><code>contrast()</code></th>
      <th><code>url()</code></th>
      <th>Original image</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <img
          class="css-filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          class="svg-filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

```
.css-filter {
  filter: contrast(200%);
}
.svg-filter {
  filter: url("#contrast");
}

th,
td {
  padding: 5px;
}
svg:not(:root) {
  display: none;
}
```

See also

- CSS filter effects module
- The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:
  - `blur()`
  - `brightness()`
  - `drop-shadow()`
  - `grayscale()`
  - `hue-rotate()`
  - `invert()`
  - `opacity()`
  - `saturate()`
  - `sepia()`

drop-shadow() CSS function

The **`drop-shadow()`** CSS function applies a drop shadow effect to the input image. Its result is a `<filter-function>`.

Try it

```
filter: drop-shadow(30px 10px 4px #4444dd);
```

```
filter: drop-shadow(0 -6mm 4mm rgb(160 0 210));
```

```
filter: drop-shadow(0 0 0.75rem crimson);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

A drop shadow is effectively a blurred, offset version of the input image's alpha mask, drawn in a specific color and composited below the image.

> **Note:** This function is somewhat similar to the `box-shadow` property. The `box-shadow` property creates a rectangular shadow behind an element's *entire box*, while the `drop-shadow()` filter function creates a

> shadow that conforms to the shape (alpha channel) of the *image itself*.

Syntax

```
/* Two length values */
/* drop-shadow( <length> <length> ) */
drop-shadow(5px 5px)

/* Three length values */
/* drop-shadow( <length> <length> <length> ) */
drop-shadow(5px 5px 15px)

/* Two length values and a color */
/* drop-shadow( <length> <length> <color> ) */
drop-shadow(5px 5px red)

/* Three length values and a color */
/* drop-shadow( <length> <length> <length> <color> ) */
drop-shadow(5px 5px 15px red)

/* The order of color and length values can be changed */
/* drop-shadow( <color> <length> <length> <length> ) */
drop-shadow(#ee2233 0.5rem 0.5rem 1rem)

/* Pass multiple drop-shadows to a filter to stack them */
drop-shadow(10px 10px red) drop-shadow(-5px -5px yellow)
```

The `drop-shadow()` function accepts a parameter of type `<shadow>` (defined in the `box-shadow` property), with the exception that the `inset` keyword and `spread` parameters are not allowed.

Parameters

`<color>` Optional

> Specifies the color for the shadow. If not specified, the value of the `color` property defined in the parent element is used.

`<length>`

> Specifies the offset length of the shadow. This parameter accepts two or three values. If two values are specified, they are interpreted as `<offset-x>` (horizontal offset) and `<offset-y>` (vertical offset) values. Negative `<offset-x>` value places the shadow to the left of the element. Negative `<offset-y>` value places the shadow above the element. If a third value is specified, it is interpreted as `<standard-deviation>`, which is the value of the standard deviation to the Gaussian blur function. A larger `<standard-deviation>` value creates a larger and more blurred shadow. Negative values for `<standard-deviation>` are not allowed.

Formal syntax

```
<drop-shadow()> =
  drop-shadow( [ <color>? && <length>{2,3} ] )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Setting a drop shadow

```
<div>drop-shadow(16px 16px)</div>
<div>drop-shadow(16px 16px red)</div>
<div>drop-shadow(red 1rem 1rem 10px)</div>
<div>drop-shadow(-16px -16px red)</div>
<div>
  drop-shadow(1px 1px red) drop-shadow(1px -1px red) drop-shadow(-1px 1px red)
  drop-shadow(-1px -1px red)
</div>
```

```
div {
  display: inline-block;
  margin: 0 0.5rem 2rem 1rem;
  padding: 0.5rem;
  height: 100px;
  width: 190px;
  vertical-align: top;
  background-color: #222222;

  color: lime;
}

div:nth-child(1) {
  filter: drop-shadow(16px 16px);
}

div:nth-child(2) {
  filter: drop-shadow(16px 16px red);
}

div:nth-child(3) {
  filter: drop-shadow(red 1rem 1rem 10px);
}

div:nth-child(4) {
  filter: drop-shadow(-16px -16px red);
}

div:nth-child(5) {
  filter: drop-shadow(1px 1px red) drop-shadow(1px -1px red)
    drop-shadow(-1px 1px red) drop-shadow(-1px -1px red);
}
```

In the absence of a `<color>` value in the `drop-shadow()` function in the first box, the shadow uses the value of the `color` property from the element (`lime`). The second and third shadows illustrate that the length and color values can be specified in any order. The third shadow shows the blurring effect when a third `<length>` value is specified. The fourth shadow uses negative offsets, which shift the shadow to the left and top. The fifth example shows how to use multiple drop-shadows on a single element.

See also

- `<filter-function>` functions:
  - `blur()`
  - `brightness()`
  - `contrast()`
  - `grayscale()`
  - `hue-rotate()`
  - `invert()`
  - `opacity()`
  - `saturate()`
  - `sepia()`
- `filter`
- `backdrop-filter`
- `box-shadow`
- `text-shadow`
- Introduction to text shadows
- CSS filter effects module

grayscale() CSS function

The **`grayscale()`** CSS function converts the input image to grayscale. Its result is a `<filter-function>`.

Try it

```
filter: grayscale(0);
```

```
filter: grayscale(0.2);
```

```
filter: grayscale(60%);
```

```
filter: grayscale(1);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
grayscale(amount)
```

Parameters

`amount` Optional

Amount of the input image that is converted to grayscale. It is specified as a `<number>` or a `<percentage>`. A value of `100%` changes the input completely to grayscale, while a value of `0%` leaves the input unchanged. Values between `0%` and `100%` have linear multipliers on the effect. The initial value used for interpolation is `0`. The default value is `1`.

Formal syntax

```
<grayscale()> =
  grayscale( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Examples of correct values for grayscale()

```
grayscale(0)     /* No effect */
grayscale(.7)    /* 70% grayscale */

grayscale()      /* Completely grayscale */
grayscale(1)
grayscale(100%)
```

See also

The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:

- `blur()`
- `brightness()`
- `contrast()`
- `drop-shadow()`
- `hue-rotate()`
- `invert()`
- `opacity()`
- `saturate()`
- `sepia()`

hue-rotate() CSS function

The **`hue-rotate()`** CSS function rotates the hue of an element and its contents. Its result is a `<filter-function>`.

> **Note:** `hue-rotate()` is specified as a matrix operation on the RGB color. It does not actually convert the color to the HSL model, which is a non-linear operation. Therefore, it may not preserve the saturation or lightness of the original color, especially for saturated colors.

Try it

```
filter: hue-rotate(0);
```

```
filter: hue-rotate(90deg);
```

```
filter: hue-rotate(-0.25turn);
```

```
filter: hue-rotate(3.142rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
hue-rotate(angle)
```

Values

`angle` Optional

> The relative change in hue of the input sample, specified as an `<angle>`. A value of `0deg` leaves the input unchanged. A positive hue rotation increases the hue value, while a negative rotation decreases the hue value. The initial value for interpolation is `0`. There is no minimum or maximum value. The effect of values above `360deg` are, given `hue-rotate(Ndeg)`, evaluates to `N` modulo 360. The default value is `0deg`.

The `<angle>` CSS data type represents an angle value expressed in degrees, gradians, radians, or turns. The following are equivalent:

```
hue-rotate(-180deg)
hue-rotate(540deg)
hue-rotate(200grad)
hue-rotate(3.14159rad)
hue-rotate(0.5turn)
```

Formal syntax

```
<hue-rotate()> =
  hue-rotate( [ <angle> | <zero> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

With the backdrop-filter property

This example applies a `hue-rotate()` filter via the `backdrop-filter` CSS property to the paragraph, color shifting to the area behind the `<p>`.

```
.container {
  background: url("/shared-assets/images/examples/listen_to_black_women.jpg")
    no-repeat left / contain #011296;
}
p {
  backdrop-filter: hue-rotate(240deg);
  text-shadow: 2px 2px #011296;
}
```

```
.container {
  padding: 3rem;
  width: 30rem;
}
p {
  padding: 0.5rem;
  color: white;
  font-size: 2rem;
  font-family: sans-serif;
}
```

```
<div class="container">
  <p>
    Text on images can be illegible and inaccessible even with a drop shadow.
  </p>
</div>
```

With the filter property

This example applies a `hue-rotate()` filter via the `filter` CSS property adding the color shift to the entire element, including content, border, and background image.

```
p {
  filter: hue-rotate(-60deg);
  text-shadow: 2px 2px blue;
  background-color: magenta;
  color: goldenrod;
  border: 1em solid rebeccapurple;
  box-shadow:
    inset -5px -5px red,
    5px 5px yellow;
}
```

```
p {
  padding: 0.5rem;
  font-size: 2rem;
  font-family: sans-serif;
  width: 85vw;
}
```

```
<p>The person who wrote this example is not a designer, fortunately.</p>
```

With url() and the SVG hue-rotate filter

The SVG `<filter>` element is used to define custom filter effects that can then be referenced by `id`. The `<filter>`'s `<feColorMatrix>` primitive `hueRotate` type provides the same effect. Given the following:

```
<svg
  xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 220 220"
  color-interpolation-filters="sRGB"
  height="220"
  width="220">
  <filter id="hue-rotate">
    <feColorMatrix type="hueRotate" values="90" />
  </filter>
</svg>
```

These values produce the same results:

```
filter: hue-rotate(90deg); /* 90deg rotation */
filter: url("#hue-rotate"); /* with embedded SVG */
filter: url("folder/fileName.svg#hue-rotate"); /* external svg filter definition */
```

This example shows three images: the image with a `hue-rotate()` filter function applied, the image with an equivalent `url()` filter applied, and the original images for comparison:

```
<table>
  <thead>
    <tr>
      <th><code>hue-rotate()</code></th>
      <th><code>url()</code></th>
      <th>Original image</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <img
          class="css-filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag with rotated colors" />
      </td>
      <td>
        <img
          class="svg-filter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag with rotated colors" />
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

```
.css-filter {
  filter: hue-rotate(90deg);
}
.svg-filter {
  filter: url("#hue-rotate");
}

th,
td {
  padding: 5px;
}
svg:not(:root) {
  display: none;
}
```

hue-rotate() does not preserve saturation or lightness

The diagram below compares two color gradients starting with red: the first is generated using `hue-rotate()`, and the second uses actual HSL color values. Note how the `hue-rotate()` gradient shows obvious differences in saturation and lightness in the middle.

```
<div>
  <p>Using <code>hue-rotate()</code></p>
  <div id="hue-rotate"></div>
</div>
<div>
  <p>Using <code>hsl()</code></p>
  <div id="hsl"></div>
</div>
```

```
#hue-rotate,
#hsl {
  display: flex;
  margin: 1em 0;
}

#hue-rotate div,
#hsl div {
  width: 2px;
  height: 100px;
}
```

```
const hueRotate = document.getElementById("hue-rotate");
const hsl = document.getElementById("hsl");

for (let i = 0; i < 360; i++) {
  const div1 = document.createElement("div");
  div1.style.backgroundColor = `hsl(${i}, 100%, 50%)`;
  hsl.appendChild(div1);

  const div2 = document.createElement("div");
  div2.style.backgroundColor = "red";
  div2.style.filter = `hue-rotate(${i}deg)`;
  hueRotate.appendChild(div2);
}
```

See also

- CSS filter effects module
- The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:
  - `blur()`
  - `brightness()`
  - `contrast()`
  - `drop-shadow()`
  - `grayscale()`
  - `invert()`
  - `opacity()`
  - `saturate()`
  - `sepia()`

invert() CSS function

The **`invert()`** CSS function inverts the color samples in the input image. Its result is a `<filter-function>`.

Try it

```
filter: invert(0);
```

```
filter: invert(0.3);
```

```
filter: invert(50%);
```

```
filter: invert(70%);
```

```
filter: invert(1);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
/* No inversion */
invert(0)

/* Complete inversion */
invert()
invert(1)
invert(100%)

/* 60% inversion */
invert(.6)
invert(60%)
```

Parameters

`<number>` or `<percentage>` Optional

Specifies the amount of the conversion. A value of `100%` is completely inverted, while a value of `0%` leaves the input unchanged. Values between `0%` and `100%` are linear multipliers on the effect. The initial value for interpolation is `0`. The default value is `1`.

Formal syntax

```
<invert()> =
  invert( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

SVG filter

The SVG `<feComponentTransfer>` filter element can also be used to invert content by creating equivalent inversion on nested `<feFuncR>`, `<feFuncG>`, and `<feFuncB>` table elements. We set the same `tableValue` for the red, green, and blue filter primitives to the same value, then we can reference the SVG effect by the ID of the `<filter>`:

```
<svg role="none">
  <filter id="invert90">
    <feComponentTransfer>
      <feFuncR type="table" tableValues="0.1 0" />
      <feFuncG type="table" tableValues="0.1 0" />
      <feFuncB type="table" tableValues="0.1 0" />
    </feComponentTransfer>
  </filter>
</svg>
```

The following declarations produce the same effect:

```
filter: invert(0.9);
filter: invert(90%);
filter: url("#invert90"); /* with embedded SVG */
filter: url("fileName.svg#invert90"); /* external SVG */
```

Examples

This example shows three images for comparison: an image with an `invert()` filter function applied, an image with the equivalent SVG function applied, and the original image:

SVG

We create an SVG filter that inverts the content on which it is applied by 70%:

```
<svg height="0">
  <filter id="invert">
    <feComponentTransfer>
      <feFuncR type="table" tableValues="0.3 0" />
      <feFuncG type="table" tableValues="0.3 0" />
      <feFuncB type="table" tableValues="0.3 0" />
    </feComponentTransfer>
  </filter>
</svg>
```

CSS

We include CSS that will invert elements based on their `filter` or `svgFilter` class:

```
.filter {
  filter: invert(70%);
}

.svgFilter {
  filter: url("#invert");
}
```

```
th,
td {
  padding: 5px;
}
```

```
<table>
  <thead>
    <tr>
      <th><code>invert()</code> filter</th>
      <th>SVG filter equivalent</th>
      <th>Original image</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        <img
          class="svgFilter"
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
      <td>
        <svg id="svg" height="220" width="220" overflow="visible">
          <filter id="svgInvert">
            <feComponentTransfer>
              <feFuncR type="table" tableValues="0.3 0" />
              <feFuncG type="table" tableValues="0.3 0" />
              <feFuncB type="table" tableValues="0.3 0" />
            </feComponentTransfer>
          </filter>
          <image
            href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
            xlink:href="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
            filter="url('#svgInvert')" />
        </svg>
      </td>
      <td>
        <img
          src="https://mdn.github.io/shared-assets/images/examples/progress-pride-flag.jpg"
          alt="Pride flag" />
      </td>
    </tr>
  </tbody>
</table>
```

See also

The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:

- `blur()`
- `brightness()`
- `contrast()`
- `drop-shadow()`
- `grayscale()`
- `hue-rotate()`
- `opacity()`
- `saturate()`
- `sepia()`

opacity() CSS function

The **`opacity()`** CSS function applies transparency to the samples in the input image. Its result is a `<filter-function>`.

Try it

```
filter: opacity(1);
```

```
filter: opacity(80%);
```

```
filter: opacity(50%);
```

```
filter: opacity(0.2);
```

```
filter: opacity(0);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

> **Note:** This function is similar to the more established `opacity` property. The difference is that with filters, some browsers provide hardware acceleration for better performance.

Syntax

```
opacity(amount)
```

Parameters

`amount` Optional

The amount of the conversion, specified as a `<number>` or a `<percentage>`. A value of `0%` is completely transparent, while a value of `100%` leaves the input unchanged. Values between `0%` and `100%` are linear multipliers on the effect. The initial value for interpolation is `1`. The default value is `1`.

Formal syntax

```
<opacity()> =
  opacity( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Examples of correct values for opacity()

```
opacity(0%)   /* Completely transparent */
opacity(50%)  /* 50% transparent */

opacity()     /* No effect */
opacity(1)
```

See also

- The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:
  - `blur()`
  - `brightness()`
  - `contrast()`
  - `drop-shadow()`
  - `grayscale()`
  - `hue-rotate()`
  - `invert()`
  - `saturate()`
  - `sepia()`
- The CSS `opacity` property

saturate() CSS function

The **`saturate()`** CSS function super-saturates or desaturates the input image. Its result is a `<filter-function>`.

> **Note:** `saturate()` is specified as a matrix operation on the RGB color. It does not actually convert the color to the HSL model, which is a non-linear operation. Therefore, it may not preserve the hue or lightness of the original color.

Try it

```
filter: saturate(1);
```

```
filter: saturate(4);
```

```
filter: saturate(50%);
```

```
filter: saturate(0);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
```

```
    width="200" />
</section>
```

Syntax

```
saturate(amount)
```

Parameters

`amount` Optional

> The amount of the conversion, specified as a `<number>` or a `<percentage>`. A value under `100%` desaturates the image, while a value over `100%` super-saturates it. A value of `0%` is completely unsaturated, while a value of `100%` leaves the input unchanged. The initial value for interpolation is `1`. The default value is `1`.

Formal syntax

```
<saturate()> =
  saturate( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Examples of correct values for saturate()

```
saturate(0)     /* Completely unsaturated */
saturate(.4)    /* 40% saturated */
saturate()      /* No effect */
saturate(100%)  /* No effect */
saturate(200%)  /* Double saturation */
```

saturate() does not preserve hue or lightness

The diagram below compares two color gradients with `hsl(0 50% 50%)` as the mid-point: the first is generated using `saturate()`, and the second uses actual HSL color values. Note how the `saturate()` gradient shows differences in hue and lightness towards the two ends.

```
<div>
  <p>Using <code>saturate()</code></p>
  <div id="saturate"></div>
</div>
<div>
  <p>Using <code>hsl()</code></p>
  <div id="hsl"></div>
</div>
```

```
#saturate,
#hsl {
  display: flex;
  margin: 1em 0;
}

#saturate div,
#hsl div {
  width: 2px;
  height: 100px;
}
```

```
const saturate = document.getElementById("saturate");
const hsl = document.getElementById("hsl");

for (let i = 0; i <= 200; i++) {
  const div1 = document.createElement("div");
  div1.style.backgroundColor = `hsl(0 ${i / 2}% 50%)`;
  hsl.appendChild(div1);

  const div2 = document.createElement("div");
  div2.style.backgroundColor = "hsl(0 50% 50%)";
  div2.style.filter = `saturate(${i}%)`;
  saturate.appendChild(div2);
}
```

See also

The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:

- `blur()`
- `brightness()`
- `contrast()`
- `drop-shadow()`
- `grayscale()`
- `hue-rotate()`
- `invert()`
- `opacity()`
- `sepia()`

sepia() CSS function

The **`sepia()`** CSS function converts the input image to sepia, giving it a warmer, more yellow/brown appearance. Its result is a `<filter-function>`.

Try it

```
filter: sepia(0);
```

```
filter: sepia(0.2);
```

```
filter: sepia(60%);
```

```
filter: sepia(1);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
sepia(amount)
```

Parameters

`amount` Optional

> The amount of the conversion, specified as a `<number>` or a `<percentage>`. A value of `100%` is completely sepia, while a value of `0%` leaves the input unchanged. Values between `0%` and `100%` are linear multipliers on the effect. The initial value for interpolation is `0`. The default value is `1`.

Formal syntax

```
<sepia()> =
  sepia( [ <number> | <percentage> ]? )
```

This syntax reflects the latest standard as per Filter Effects Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Examples of correct values for sepia()

```
sepia(0)     /* No effect */
sepia(.65)   /* 65% sepia */

sepia()      /* Completely sepia */
sepia(100%)
```

See also

The other `<filter-function>` functions available to be used in values of the `filter` and `backdrop-filter` properties include:

- `blur()`
- `brightness()`
- `contrast()`
- `drop-shadow()`
- `grayscale()`
- `hue-rotate()`
- `invert()`
- `opacity()`
- `saturate()`

fit-content CSS keyword

The `fit-content` sizing keyword represents an element size that adapts to its content while staying within the limits of its container. The keyword ensures that the element is never smaller than its minimum intrinsic size (`min-content`) or larger than its maximum intrinsic size (`max-content`).

> **Note:** This keyword is different from the `fit-content()` function. The function is used for grid track sizing (for example in `grid-template-columns` and `grid-auto-rows`) and for laid-out box sizing for properties such as `width`, `height`, `min-width`, and `max-height`.

Syntax

```
/* Used in sizing properties */
width: fit-content;
height: fit-content;
inline-size: fit-content;
block-size: fit-content;
```

Description

This keyword is used with sizing properties such as `width`, `height`, `block-size`, `inline-size`, `min-width`, and `max-width`. When used on these properties, the calculated size refers to the element's content box.

When `fit-content` is set, the element grows or shrinks to fit its content, but stops expanding after the relevant dimension reaches the size limit of its container.

The `fit-content` size is calculated using the following formula:

```
min(max-content, max(min-content, stretch))
```

where, `stretch` matches the element's margin box to the width of its containing block. The keyword is essentially equivalent to `fit-content(stretch)`.

You can enable animations to and from `fit-content` using the `interpolate-size` property and the `calc-size()` function.

Examples

Sizing boxes with fit-content

HTML

```
<div class="container">
  <div class="item">Item</div>
  <div class="item">Item with more text in it.</div>
  <div class="item">
    Item with more text in it, hopefully we have added enough text so the text
    will start to wrap.
```

```
  </div>
</div>
```

CSS

```
.container {
  border: 2px solid #cccccc;
  padding: 10px;
  width: 20em;
}

.item {
  width: fit-content;
  background-color: #8ca0ff;
  padding: 5px;
  margin-bottom: 1em;
}
```

Result

See also

- Related sizing keywords: `min-content`, `max-content`
- CSS box sizing module

fit-content() CSS function

The **`fit-content()`** CSS function clamps a given size to an available size according to the formula `min(maximum size, max(minimum size, argument))`.

It is distinct from the `fit-content` keyword, which takes no argument and sizes a box based on its content within the available space. Only `fit-content()` is valid in grid track sizing properties such as `grid-template-columns`.

Try it

```
grid-template-columns: fit-content(8ch) fit-content(8ch) 1fr;
```

```
grid-template-columns: fit-content(100px) fit-content(100px) 1fr;
```

```
grid-template-columns: fit-content(40%) fit-content(40%) 1fr;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One. This column has more text in it.</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-gap: 10px;
  width: 250px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  text-align: left;
}
```

The function can be used as a track size in CSS grid properties, where the maximum size is defined by `max-content` and the minimum size by `auto`, which is calculated similar to `auto` (i.e., `minmax(auto, max-content)`), except that the track size is clamped at *argument* if it is greater than the `auto` minimum.

See the `grid-template-columns` page for more information on the `max-content` and `auto` keywords.

The `fit-content()` function can also be used as laid out box size for `width`, `height`, `min-width`, `min-height`, `max-width` and `max-height`, where the maximum and minimum sizes refer to the content size.

Syntax

```
/* <length> values */
fit-content(200px)
fit-content(5cm)
fit-content(30vw)
fit-content(100ch)

/* <percentage> value */
fit-content(40%)
```

Values

`<length>`

An absolute length.

`<percentage>`

A percentage relative to the available space in the given axis.

In grid properties it is relative to the inline size of the grid container in column tracks and to the block size of the grid container for row tracks. Otherwise it is relative to the available inline size or block size of the laid out box depending on the writing mode.

Formal syntax

```
<fit-content()> =
  fit-content( <length-percentage> )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Sizing grid columns with fit-content

HTML

```
<div id="container">
  <div>Item as wide as the content.</div>
  <div>
    Item with more text in it. Because the contents of it are wider than the
    maximum width, it is clamped at 300 pixels.
  </div>
  <div>Flexible item</div>
</div>
```

CSS

```
#container {
  display: grid;
  grid-template-columns: fit-content(300px) fit-content(300px) 1fr;
  grid-gap: 5px;
  box-sizing: border-box;
  height: 200px;
  width: 100%;
  background-color: #8cffa0;
  padding: 10px;
}

#container > div {
  background-color: #8ca0ff;
  padding: 5px;
}
```

Result

Browser compatibility

See also

- `min-content` keyterm
- `max-content` keyterm
- CSS box sizing module
- `grid-template`
- `grid-template-rows`
- `grid-template-columns`
- `grid-template-areas`
- `grid-auto-columns`
- `grid-auto-rows`
- `grid-auto-flow`
- `fit-content` keyterm
- Line-based placement with CSS grid
- Grid template areas: grid definition shorthands

<flex> CSS type

The **`<flex>`** CSS data type denotes a flexible length within a grid container. It is used in `grid-template-columns`, `grid-template-rows` and other related properties.

Syntax

The `<flex>` data type is specified as a `<number>` followed by the unit `fr`. The `fr` unit represents a fraction of the leftover space in the grid container. As with all CSS dimensions, there is no space between the unit and the number.

Examples

Examples of correct values for the fr data type

```
1fr    /* Using an integer value */
2.5fr  /* Using a float value */
```

Example of use in a track listing for CSS grid layout

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 2.5fr 1.5fr;
}
```

See also

- CSS grid layout

<frequency> CSS type

The **`<frequency>`** CSS data type represents a frequency dimension, such as the pitch of a speaking voice. It is not currently used in any CSS properties.

Syntax

The `<frequency>` data type consists of a `<number>` followed by one of the units listed below. As with all CSS dimensions, there is no space between the unit literal and the number.

Units

`Hz`

Represents a frequency in hertz. Examples: `0Hz`, `1500Hz`, `10000Hz`.

`kHz`

Represents a frequency in kilohertz. Examples: `0kHz`, `1.5kHz`, `10kHz`.

> **Note:** Although the number `0` is always the same regardless of unit, the unit may not be omitted. In other words, `0` is invalid and does not represent `0Hz` or `0kHz`. Though the units are case-insensitive, it is good practice to use a capital "H" for `Hz` and `kHz`, as specified in the SI.

Examples

Valid frequency values:

```
12Hz     Positive integer
4.3Hz    Non-integer
14KhZ    The unit is case-insensitive, though non-SI capitalization is not recommended.
+0Hz     Zero, with a leading + and a unit
-0kHz    Zero, with a leading - and a unit
```

Invalid frequency values:

```
12.0     This is a <number>, not an <frequency>, because it is missing a unit.
7 Hz     No space is allowed between the number and the unit.
0        Although unitless zero is an allowable <length>, it's an invalid <frequency>.
```

Browser compatibility

Currently, no browsers support this feature.

See also

- `<frequency-percentage>`
- CSS values and units module

<frequency-percentage> CSS type

The **`<frequency-percentage>`** CSS data type represents a value that can be either a `<frequency>` or a `<percentage>`. Frequency values, e.g., the pitch of a speaking voice, are not currently used in any CSS properties.

Syntax

The value of a `<frequency-percentage>` is either a `<frequency>` or a `<percentage>`; see their individual reference pages for details about their syntaxes.

Description

Use in calc()

Where a `<frequency-percentage>` is specified as an allowable type, this means that the percentage resolves to a frequency and therefore can be used in a `calc()` expression.

Formal syntax

```
<frequency-percentage> =
  <frequency>   |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Valid percentage values

```
90% Positive percentage
+90% Positive percentage with leading +
-90% Negative percentage — not valid for all properties that use percentages
```

Invalid percentage values

```
90 % No space is allowed between the number and the unit
```

Valid frequency values

```
12Hz     Positive integer
4.3Hz    Non-integer
14KhZ    The unit is case-insensitive, though non-SI capitalization is not recommended.
+0Hz     Zero, with a leading + and a unit
-0kHz    Zero, with a leading - and a unit
```

Invalid frequency values

```
12.0     This is a <number>, not an <frequency>, because it is missing a unit.
7 Hz     No space is allowed between the number and the unit.
0        Although unitless zero is an allowable <length>, it's an invalid <frequency>.
```

Browser compatibility

Currently, no browsers support this feature.

See also

- CSS data types

- CSS values and units module
- Related CSS data types:
  - `<frequency>`
  - `<percentage>`

CSS value functions

CSS value functions are statements that invoke special data processing or calculations to return a CSS value for a CSS property. CSS value functions represent more complex data types and they may take some input arguments to calculate the return value.

Syntax

```
selector {
  property: function([argument]? [, argument]!);
}
```

The value syntax starts with the **name of the function**, followed by a left parenthesis `(`. Next up are the argument(s), and the function is finished off with a closing parenthesis `)`.

Functions can take multiple arguments, which are formatted similarly to CSS property values. Whitespace is allowed, but they are optional inside the parentheses. In some functional notations multiple arguments are separated by commas, while others use spaces.

> **Note:** The CSS value functions are used as property values and should not be confused with pseudo-classes. The functional pseudo-classes, linguistic pseudo-classes, and several tree-structural pseudo-classes require parameter values, but they're not value functions. The conditional at-rules are also not value functions; the parentheses are used for groupings.

Transform functions

The `<transform-function>` CSS data type represent appearance transformation. It is used as a value of `transform` property.

Translate functions

`translateX()`

Translates an element horizontally.

`translateY()`

Translates an element vertically.

`translateZ()`

Translates an element along the z-axis.

`translate()`

Translates an element on the 2D plane.

`translate3d()`

Translates an element in 3D space.

Rotation functions

`rotateX()`

Rotates an element around the horizontal axis.

`rotateY()`

Rotates an element around the vertical axis.

`rotateZ()`

Rotates an element around the z-axis.

`rotate()`

Rotates an element around a fixed point on the 2D plane.

`rotate3d()`

Rotates an element around a fixed axis in 3D space.

Scaling functions

`scaleX()`

Scales an element up or down horizontally.

`scaleY()`

Scales an element up or down vertically.

`scaleZ()`

Scales an element up or down along the z-axis.

`scale()`

Scales an element up or down on the 2D plane.

`scale3d()`

Scales an element up or down in 3D space.

Skew functions

`skewX()`

Skews an element in the horizontal direction.

`skewY()`

Skews an element in the vertical direction.

`skew()`

Skews an element on the 2D plane.

Matrix functions

`matrix()`

Describes a homogeneous 2D transformation matrix.

`matrix3d()`

Describes a 3D transformation as a 4×4 homogeneous matrix.

Perspective functions

`perspective()`

Sets the distance between the user and the z=0 plane.

Math functions

The math functions allow CSS numeric values to be written as mathematical expressions.

Each of the pages below contains detailed information about a math function's syntax, browser compatibility data, examples, and more. For a holistic introduction to CSS math functions, see Using CSS math functions.

Basic arithmetic

`calc()`

Performs basic arithmetic calculations on numerical values.

`calc-size()`

Perform calculations on intrinsic size values such as `auto`, `fit-content`, and `max-content`, which are not supported by the `calc()` function.

Comparison functions

`min()`

Calculates the smallest of a list of values.

`max()`

Calculates the largest of a list of values.

`clamp()`

Calculates the central of a minimum, central, and maximum values.

Stepped value functions

`round()`

Calculates a rounded number based on a rounding strategy.

`mod()`

Calculates a modulus (with the same sign as the divisor) when dividing one number by another.

`progress()`

Calculates the position of a value between two other values — a start value and an end value. The result represents the progress between the start and end values, where 0 is the start, and 1 is the end.

`rem()`

Calculates a remainder (with the same sign as the dividend) when dividing one number by another.

Trigonometric functions

`sin()`

Calculates the trigonometric sine of a number.

`cos()`

Calculates the trigonometric cosine of a number.

`tan()`

Calculates the trigonometric tangent of a number.

`asin()`

Calculates the trigonometric inverse sine of a number.

`acos()`

Calculates the trigonometric inverse cosine of a number.

`atan()`

Calculates the trigonometric inverse tangent of a number.

`atan2()`

Calculates the trigonometric inverse tangent of two-numbers in a plane.

Exponential functions

`pow()`

Calculates the base raised to the power of a number.

`sqrt()`

Calculates the square root of a number.

`hypot()`

Calculates the square root of the sum of the squares of its arguments.

`log()`

Calculates the logarithm of a number.

`exp()`

Calculates `e` raised to the power of a number.

Random value functions

`random()`

Generates a random value between a minimum and maximum value.

Sign-related functions

`abs()`

Calculates the absolute value of a number.

`sign()`

Calculates the sign (positive or negative) of the number.

Filter functions

The `<filter-function>` CSS data type represents a graphical effect that can change the appearance of an input image. It is used in the `filter` and `backdrop-filter` properties.

`blur()`

Increases the image gaussian blur.

`brightness()`

Brightens or darkens an image.

`contrast()`

Increases or decreases the image contrast.

`drop-shadow()`

Applies a drop shadow behind an image.

`grayscale()`

Converts an image to grayscale.

`hue-rotate()`

Changes the overall hue of an image.

`invert()`

Inverts the colors of an image.

`opacity()`

Adds transparency to an image.

`saturate()`

Changes the overall saturation of an image.

`sepia()`

Increases the sepia of an image.

Color functions

The `<color>` CSS data type specifies different color representations.

`rgb()`

Defines a given color according to its red, green, blue and alpha (transparency) components.

`hsl()`

Defines a given color according to its hue, saturation, lightness and alpha (transparency) components.

`hwb()`

Defines a given color according to its hue, whiteness and blackness components.

`lch()`

Defines a given color according to its lightness, chroma and hue components.

`oklch()`

Defines a given color according to its lightness, chroma, hue and alpha (transparency) components.

`lab()`

Defines a given color according to its lightness, a-axis distance and b-axis distance in the Lab colorspace.

`oklab()`

Defines a given color according to its lightness, a-axis distance, b-axis distance in the Lab colorspace and alpha (transparency).

`color()`

Specifies a particular, specified colorspace rather than the implicit sRGB colorspace.

`color-mix()`

Mixes two color values in a given colorspace by a given amount.

`contrast-color()`

Returns a color with maximum color contrast for a given color.

`device-cmyk()`

Defines CMYK colors in a device-dependent way.

`alpha()`

Returns a color with a modified alpha (transparency) channel.

`light-dark()`

Returns one of two provided colors based on the current color scheme.

`dynamic-range-limit-mix()`

Creates a custom maximum luminance limit that is a mix of different `dynamic-range-limit` keywords in specified percentages.

Image functions

The `<image>` CSS data type provides graphical representation of images or gradients.

Gradient functions

`linear-gradient()`

Linear gradients transition colors progressively along an imaginary line.

`radial-gradient()`

Radial gradients transition colors progressively from a center point (origin).

`conic-gradient()`

Conic gradients transition colors progressively around a circle.

`repeating-linear-gradient()`

Is similar to `linear-gradient()` and takes the same arguments, but it repeats the color stops infinitely in all directions so as to cover its entire container.

`repeating-radial-gradient()`

Is similar to `radial-gradient()` and takes the same arguments, but it repeats the color stops infinitely in all directions so as to cover its entire container.

`repeating-conic-gradient()`

Is similar to `conic-gradient()` and takes the same arguments, but it repeats the color stops infinitely in all directions so as to cover its entire container.

Image functions

`image()`

Defines an `<image>` in a similar fashion to the `<url>` type, but with added functionality including specifying the image's directionality and fallback images for when the preferred image is not supported.

`image-set()`

Picks the most appropriate CSS image from a given set, primarily for high pixel density screens.

`cross-fade()`

Blends two or more images at a defined transparency.

`element()`

Defines an `<image>` value generated from an arbitrary HTML element.

`paint()`

Defines an `<image>` value generated with a PaintWorklet.

Counter functions

CSS counter functions are generally used with the `content` property, although in theory, they may be used wherever a `<string>` is supported.

`counter()`

Returns a string representing the current value of the named counter if there is one.

`counters()`

Enables nested counters, returning a concatenated string representing the current values of the named counters, if there are any.

`symbols()`

Defines the counter styles inline, directly as the value of a property.

Shape functions

Basic shapes

The `<basic-shape>` CSS data type represents a graphical shape. It is used in the `clip-path`, `offset-path`, and `shape-outside` properties.

`circle()`

Defines a circle shape.

`ellipse()`

Defines an ellipse shape.

`inset()`

Defines an inset rectangle shape.

`rect()`

Defines a rectangle shape using the distances from the top and left edges of the reference box.

`xywh()`

Defines a rectangle shape using the specified distances from the top and left edges of the reference box and the rectangle width and height.

`polygon()`

Defines a polygon shape.

`path()`

Accepts an SVG path string to enable a shape to be drawn.

`shape()`

Accepts a comma-separated list of commands defining the shape to be drawn.

Other shape functions

`ray()`

Valid with `offset-path`; defines the line segment an animated element can follow.

`superellipse()`

Defines the curvature of an ellipse; can be used to specify a `<corner-shape-value>`, which is used with `corner-shape` and its constituent and related properties.

Reference functions

The following functions are used as a value of properties to reference a value defined elsewhere:

`attr()`

Uses the attributes defined on HTML element.

`env()`

Uses the user-agent defined as environment variable.

`if()`

Conditionally sets a property value depending on the result of a style query, media query, or feature query.

`url()`

Uses a file from the specified URL.

`var()`

Uses the custom property value instead of any part of a value of another property.

Grid functions

The following functions are used to define a CSS grid:

`fit-content()`

Clamps a given size to an available size according to the formula `min(maximum size, max(minimum size, argument))`.

`minmax()`

Defines a size range greater-than or equal-to *min* and less-than or equal-to *max*.

`repeat()`

Represents a repeated fragment of the track list, allowing a large number of columns or rows that exhibit a recurring pattern.

Font functions

CSS font functions are used with the `font-variant-alternates` property to control the use of alternate glyphs.

`stylistic()`

Enables stylistic alternates for individual characters. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `salt`, like `salt 2`.

`styleset()`

Enables stylistic alternatives for sets of characters. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `ssXY`, such as `ss02`.

`character-variant()`

Enables specific stylistic alternatives for characters. It is similar to `styleset()`, but doesn't create coherent glyphs for a set of characters; individual characters will have independent and not necessarily coherent styles. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `cvXY`, such as `cv02`.

`swash()`

Enables swash glyphs. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType values `swsh` and `cswh`, such as `swsh 2` and `cswh 2`.

`ornaments()`

Enables ornaments such as fleurons and other dingbat glyphs. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `ornm`, such as `ornm 2`.

`annotation()`

Enables annotations such as circled digits or inverted characters. The parameter is a font-specific name mapped to a number. It corresponds to the OpenType value `nalt`, such as `nalt 2`.

Easing functions

The `<easing-function>` CSS data type represents a mathematical function. It is used in transition and animation properties:

`linear()`

Easing function that interpolates linearly between its points.

`cubic-bezier()`

Easing function that defines a cubic Bézier curve.

`steps()`

Iteration along a specified number of stops along the transition, displaying each stop for equal lengths of time.

Animation functions

The following functions are used as a value of different `animation-timeline` properties:

`scroll()`

Sets the `animation-timeline` of an element to an *anonymous scroll progress timeline*.

`view()`

Sets the `animation-timeline` of an element to an *anonymous view progress timeline*.

Anchor positioning functions

The anchor positioning functions are used when positioning and sizing anchor-positioned elements relative to the location and size of their associated anchor elements.

`anchor()`

Returns a length relative to the position of the edges of an anchor-positioned element's associated anchor element.

`anchor-size()`

Returns a length relative to the size of the associated anchor element.

Tree counting functions

The following functions return an integer value based on the DOM tree, rather than the flat tree like most CSS values do:

`sibling-index()`

Returns an integer reflecting the position of the selected element among its siblings.

`sibling-count()`

Returns an integer reflecting the total number of siblings, including the selected element.

Alphabetical index of functions

- `-moz-image-rect()`
- `abs()`
- `acos()`
- `alpha()`
- `anchor()`
- `anchor-size()`
- `asin()`
- `atan()`
- `atan2()`
- `attr()`
- `blur()`
- `brightness()`
- `calc()`
- `calc-size()`
- `circle()`
- `clamp()`
- `color()`
- `color-mix()`
- `conic-gradient()`
- `contrast()`
- `contrast-color()`
- `cos()`
- `counter()`
- `counters()`
- `cross-fade()`
- `cubic-bezier()`
- `device-cmyk()`
- `drop-shadow()`
- `dynamic-range-limit-mix()`
- `element()`
- `ellipse()`
- `env()`
- `exp()`
- `fit-content()`
- `grayscale()`
- `hsl()`

- `hue-rotate()`
- `hwb()`
- `hypot()`
- `if()`
- `image()`
- `image-set()`
- `inset()`
- `invert()`
- `lab()`
- `layer()`
- `lch()`
- `light-dark()`
- `linear()`
- `linear-gradient()`
- `log()`
- `matrix()`
- `matrix3d()`
- `max()`
- `min()`
- `minmax()`
- `mod()`
- `oklab()`
- `oklch()`
- `opacity()`
- `paint()`
- `palette-mix()`
- `path()`
- `perspective()`
- `polygon()`
- `pow()`
- `progress()`
- `radial-gradient()`
- `random()`
- `ray()`
- `rect()`
- `rem()`
- `repeat()`
- `repeating-conic-gradient()`
- `repeating-linear-gradient()`
- `repeating-radial-gradient()`
- `rgb()`
- `rotate()`
- `rotate3d()`
- `rotateX()`
- `rotateY()`
- `rotateZ()`
- `round()`
- `saturate()`
- `scale()`
- `scale3d()`
- `scaleX()`
- `scaleY()`
- `scaleZ()`
- `scroll()`
- `sepia()`
- `shape()`
- `sibling-count()`
- `sibling-index()`
- `sign()`
- `sin()`
- `skew()`
- `skewx()`
- `skewy()`
- `sqrt()`
- `steps()`
- `superellipse()`
- `symbols()`
- `tan()`
- `translate()`
- `translate3d()`
- `translateX()`
- `translateY()`
- `translateZ()`
- `type()`
- `url()`
- `var()`
- `view()`
- `xywh()`

See also

- CSS values and units module
- Learn: Values and units

<generic-family> CSS type

The **`<generic-family>`** CSS data type represents the keyword values for generic font families used in the `font` shorthand and `font-family` longhand properties. The `<generic-family>` represents one or more locally-installed fonts belonging to that category of fonts.

Syntax

Values

The `<generic-family>` enumerated type is specified using one of the values listed below:

`serif`

A serif is a small line or stroke attached to the end of a larger stroke in a letter. In serif fonts, glyphs have finishing strokes, flared or tapering ends. Examples include Lucida Bright, Lucida Fax, Palatino, Palatino Linotype, Palladio, and URW Palladio.

`sans-serif`

A font without serifs; glyphs have plain stroke endings, without ornamentation. Example sans-serif fonts include Open Sans, Fira Sans, Lucida Sans, Lucida Sans Unicode, Trebuchet MS, Liberation Sans, and Nimbus Sans L.

`monospace`

All glyphs have the same fixed width. Example monospace fonts include Fira Mono, DejaVu Sans Mono, Menlo, Consolas, Liberation Mono, Monaco, and Lucida Console.

`cursive`

Glyphs in cursive fonts generally use a cursive script or other handwriting style, and the result looks more like handwritten pen or brush writing than printed typesetting. CSS uses the term "cursive" to apply to a font for any script, including those that do not have joining strokes. Example cursive fonts include Brush Script MT, Brush Script Std, Lucida Calligraphy, Lucida Handwriting, and Apple Chancery.

`fantasy`

Fantasy fonts are primarily decorative fonts that contain playful representations of characters. Example fantasy fonts include Papyrus, Herculanum, Party LET, Curlz MT, Harrington, and Comic Sans MS.

`system-ui`

Glyphs are taken from the default user interface font on a given platform. Because typographic traditions vary widely across the world, this generic family is provided for typefaces that don't map cleanly into the others.

> **Note:** As the name implies, `system-ui` is intended to make UI elements look like native apps, and not for typesetting large paragraphs of text. It may cause the displayed typeface to be undesirable for some users—for example, the default Windows CJK font may render Latin scripts poorly, and the `lang` attribute may not affect the displayed font. Some operating systems do not allow customizing `system-ui`, while browsers generally allow customizing the `sans-serif` font family. For large paragraphs, use `sans-serif` or some other non-UI font family instead.

`ui-serif`

The default user interface serif font. See the definition of `serif` above.

`ui-sans-serif`

The default user interface sans-serif font. See the definition of `sans-serif` above.

`ui-monospace`

The default user interface monospace font. See the definition of `monospace` above.

`ui-rounded`

The default user interface font that has rounded features.

`math`

Fonts for displaying mathematical expressions, for example superscript and subscript, brackets that cross several lines, nesting expressions, and double-struck glyphs with distinct meanings.

`fangsong`

A particular style of Chinese characters that are between serif-style Song and cursive-style Kai forms. This style is often used for government documents.

Formal syntax

```
<generic-family> =
  serif          |
  sans-serif     |
  monospace      |
  cursive        |
  fantasy        |
  system-ui      |
  ui-serif       |
  ui-sans-serif  |
  ui-monospace   |
  ui-rounded     |
  math           |
  fangsong
```

Examples

This example demos several of the `<generic-family>` enumerated values for the `font-family` property.

HTML

```
<ul>
  <li class="serif">serif</li>
  <li class="sans-serif">sans-serif</li>
  <li class="monospace">monospace</li>
  <li class="cursive">cursive</li>
  <li class="fantasy">fantasy</li>
  <li class="system-ui">system-ui</li>
</ul>
```

CSS

```
ul {
  font-size: 1.5rem;
  line-height: 2;
}
.serif {
  font-family: serif;
}
.sans-serif {
  font-family: sans-serif;
}
.monospace {
  font-family: monospace;
}
.cursive {
  font-family: cursive;
}
.fantasy {
  font-family: fantasy;
}
.system-ui {
  font-family: system-ui;
}
```

Result

See also

- Properties that use this data type: `font-family` and `font`
- CSS fonts module

<gradient> CSS type

The **`<gradient>`** CSS data type is a special type of `<image>` that consists of a progressive transition between two or more colors.

A CSS gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element to which it applies.

Try it

```
background: linear-gradient(#f69d3c, #3f87a6);
```

```
background: radial-gradient(#f69d3c, #3f87a6);
```

```
background: repeating-linear-gradient(#f69d3c, #3f87a6 50px);
```

```
background: repeating-radial-gradient(#f69d3c, #3f87a6 50px);
```

```
background: conic-gradient(#f69d3c, #3f87a6);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

Syntax

The `<gradient>` data type is defined with one of the function types listed below.

Linear gradient

Linear gradients transition colors progressively along an imaginary line. They are generated with the `linear-gradient()` function.

Radial gradient

Radial gradients transition colors progressively from a center point (origin). They are generated with the `radial-gradient()` function.

Conic gradient

Conic gradients transition colors progressively around a circle. They are generated with the `conic-gradient()` function.

Repeating gradient

Repeating gradients duplicate a gradient as much as necessary to fill a given area. They are generated with the `repeating-linear-gradient()`, `repeating-radial-gradient()`, and `repeating-conic-gradient()` functions.

Interpolation

As with any interpolation involving colors, gradients are calculated in the alpha-premultiplied color space. This prevents unexpected shades of gray from appearing when both the color and the opacity are changing. (Be aware that older browsers may not use this behavior when using the transparent keyword.)

Formal syntax

```
<gradient> =
  <linear-gradient()>           |
  <repeating-linear-gradient()> |
  <radial-gradient()>           |
  <repeating-radial-gradient()> |
  <conic-gradient()>            |
  <repeating-conic-gradient()>

<linear-gradient()> =
  linear-gradient( [ <linear-gradient-syntax> ] )

<repeating-linear-gradient()> =
  repeating-linear-gradient( [ <linear-gradient-syntax> ] )
```

```
<radial-gradient()> =
  radial-gradient( [ <radial-gradient-syntax> ] )

<repeating-radial-gradient()> =
  repeating-radial-gradient( [ <radial-gradient-syntax> ] )

<conic-gradient()> =
  conic-gradient( [ <conic-gradient-syntax> ] )

<repeating-conic-gradient()> =
  repeating-conic-gradient( [ <conic-gradient-syntax> ] )

<linear-gradient-syntax> =
  [ [ <angle> | <zero> | to <side-or-corner> ] || <color-interpolation-method> ]? , <color-stop-list>

<radial-gradient-syntax> =
  [ [ [ <radial-shape> || <radial-size> ]? [ at <position> ]? ] || <color-interpolation-method> ]? , <color-stop-list>

<conic-gradient-syntax> =
  [ [ [ from [ <angle> | <zero> ] ]? [ at <position> ]? ] || <color-interpolation-method> ]? , <angular-color-stop-list>

<side-or-corner> =
  [ left | right ]  ||
  [ top | bottom ]

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<color-stop-list> =
  <linear-color-stop> , [ <linear-color-hint>? , <linear-color-stop> ]#?

<radial-shape> =
  circle   |
  ellipse

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<angular-color-stop-list> =
  <angular-color-stop> , [ <angular-color-hint>? , <angular-color-stop> ]#?

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<linear-color-stop> =
  <color> <color-stop-length>?

<linear-color-hint> =
  <length-percentage>

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                 |
  center               |
  right                |
  top                  |
  bottom               |
  x-start              |
  x-end                |
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
```

```
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<angular-color-stop> =
  <color> <color-stop-angle>?

<angular-color-hint> =
  <angle-percentage>  |
  <zero>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-length> =
  <length-percentage>{1,2}

<color-stop-angle> =
  [ <angle-percentage> | <zero> ]{1,2}

<angle-percentage> =
  <angle>       |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 3, CSS Images Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Linear gradient example

A linear gradient.

```
<div class="linear-gradient">Linear gradient</div>
```

```
div {
  width: 240px;
  height: 80px;
}
```

```
.linear-gradient {
  background: linear-gradient(
    to right,
    red,
    orange,
    yellow,
    green,
    blue,
    indigo,
    violet
  );
}
```

Radial gradient example

A radial gradient.

```
<div class="radial-gradient">Radial gradient</div>
```

```
div {
  width: 240px;
  height: 80px;
}
```

```
.radial-gradient {
  background: radial-gradient(red, yellow, dodgerblue);
}
```

Conic gradient example

A conic gradient example.

```
<div class="conic-gradient">Conic gradient</div>
```

```
div {
  width: 200px;
  height: 200px;
}
```

```
.conic-gradient {
  background: conic-gradient(pink, coral, lime);
}
```

Repeating gradient examples

Repeating linear and radial gradient examples.

```
<div class="linear-repeat"></div>
<span>Repeating linear gradient</span>
<hr />
<div class="radial-repeat"></div>
<span>Repeating radial gradient</span>
<hr />
```

```
<div class="conic-repeat"></div>
<span>Repeating conic gradient</span>
```

```
div {
  display: inline-block;
  width: 240px;
  height: 80px;
}

span {
  font-weight: bold;
  vertical-align: top;
}
```

```
.linear-repeat {
  background: repeating-linear-gradient(
    to top left,
    pink,
    pink 5px,
    white 5px,
    white 10px
  );
}

.radial-repeat {
  background: repeating-radial-gradient(
    lime,
    lime 15px,
    white 15px,
    white 30px
  );
}

.conic-repeat {
  background: repeating-conic-gradient(lime, pink 30deg);
}
```

See also

- Using CSS gradients
- Gradient functions: `linear-gradient()`, `repeating-linear-gradient()`, `radial-gradient()`, `repeating-radial-gradient()`, `conic-gradient()`, `repeating-conic-gradient()`
- CSS Basic Data Types
- CSS values and units module
- Learn: Values and Units

conic-gradient() CSS function

The **`conic-gradient()`** CSS function creates an image consisting of a gradient with color transitions rotated around a center point (rather than radiating from the center). Example conic gradients include pie charts and color wheels. The result of the `conic-gradient()` function is an object of the `<gradient>` data type, which is a special kind of `<image>`.

Try it

```
background: conic-gradient(red, orange, yellow, green, blue);
```

```
background: conic-gradient(
  from 0.25turn at 50% 30%,
  #f69d3c,
  10deg,
  #3f87a6,
  350deg,
  #ebf8e1
);
```

```
background: conic-gradient(from 3.1416rad at 10% 50%, #e66465, #9198e5);
```

```
background: conic-gradient(
  red 6deg,
  orange 6deg 18deg,
  yellow 18deg 45deg,
  green 45deg 110deg,
  blue 110deg 200deg,
  purple 200deg
);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

Syntax

```
/* A gradient with a single color of red */
conic-gradient(red)

/* A conic gradient rotated 45 degrees,
   starting blue and finishing red */
conic-gradient(from 45deg, blue, red)

/* A bluish purple box: the gradient goes from blue to red,
   but only the bottom right quadrant is visible, as the
   center of the conic gradient is at the top left corner */
conic-gradient(from 90deg at 0 0, blue, red)

/* Interpolation in polar color space
```

```
  with longer hue interpolation method */
conic-gradient(in hsl longer hue, red, blue, green, red)
```

Values

`<angle>`

Preceded by the `from` keyterm, and taking an angle as its value, defines the gradient rotation in clockwise direction.

`<position>`

Using the same length, order, and keyterm values as the `background-position` property, the `position` value defines center of the gradient. If not specified, the value used for `position` by default is `center`, meaning the gradient will be centered.

`<angular-color-stop>`

A color-stop's `<color>` value, followed by one or two optional stop positions, (an `<angle>` along the gradient's circumference axis).

`<color-hint>`

An interpolation hint defining how the gradient progresses between adjacent color stops. The length defines at which point between two color stops the gradient color should reach the midpoint of the color transition. If omitted, the midpoint of the color transition is the midpoint between two color stops.

> **Note:** Rendering of color stops in conic gradients follows the same rules as color stops in linear gradients.

Description

As with any gradient, a conic gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element it applies to, or the size of the `<image>` if it is set to something other than the element size.

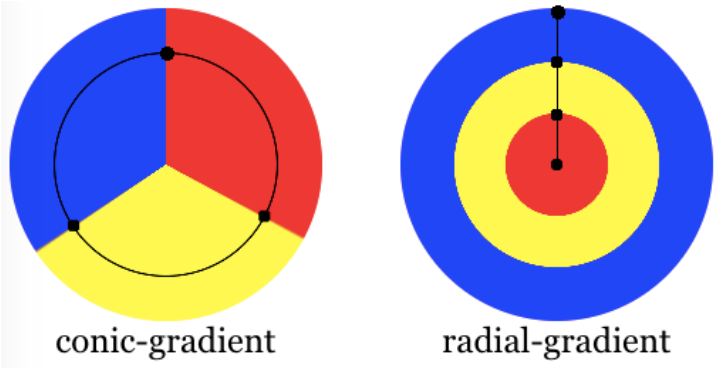
To create a conic gradient that repeats so as to fill a 360 degree rotation, use the `repeating-conic-gradient()` function instead.

Because `<gradient>`s belong to the `<image>` data type, they can only be used where `<image>`s can be used. For this reason, `conic-gradient()` won't work on `background-color` and other properties that use the `<color>` data type.

> **Note:** Why is it called a "conic" gradient? If the color stops are much lighter on one side than the other, it can look like a cone from above.

Composition of a conic gradient

The conic-gradient syntax is similar to the radial-gradient syntax, but the color-stops are placed around a gradient arc, the circumference of a circle, rather than on the gradient line emerging from the center of the gradient. With conic gradients, the colors transition as if spun around the center of a circle, starting at the top and going clockwise. In a radial gradient, the colors transition from the center of an ellipse, outward, in all directions.



A conic gradient is specified by indicating a rotation angle, the center of the gradient, and then specifying a list of color-stops. Unlike linear and radial gradients, whose color-stops are placed by specifying a `<length>`, the color-stops of a conic gradient are specified with an angle. Units include `deg` for degrees, `grad` for gradians, `rad` for radians, and `turn` for turns. There are 360 degrees, 400 gradians, 2π radians, and 1 turn in a circle. Browsers supporting conic gradients also accept percent values, with 100% equaling 360 degrees, but this is not in the specification.

Similar to radial gradients, the conic gradient syntax provides for positioning the center of the gradient anywhere within, or even outside, the image. The values for the position are similar to the syntax for 2-value background-position.

The gradient arc is the circumference of the gradient. The *starting point* of the gradient or arc is north, or 12:00pm. The gradient is then rotated by the *from* angle. The colors of the gradient are determined by the angled color stops, their starting points, ending points, and, in between, and optional angled color-stop points. The transitions between colors can be altered with color hints between adjacent colors' color stops.

Customizing gradients

By adding more angled color-stop points on the gradient arc, you can create a highly customized transition between multiple colors. A color-stop's position can be explicitly defined by using an `<angle>`. If you don't specify the location of a color stop, it is placed halfway between the one that precedes it and the one that follows it. If you don't specify an angle for the first or last color stop, their values are 0deg and 360deg respectively. The following two gradients are equivalent:

```
conic-gradient(red, orange, yellow, green, blue);
conic-gradient(red 0deg, orange 90deg, yellow 180deg, green 270deg, blue 360deg);
```

By default, colors transition smoothly from the color at one color stop to the color at the subsequent color stop, with the midpoint between the colors being the half way point between the color transition. You can move this color transition midpoint to any point between two color stops by adding a color hint, indicating where the middle of the color transition should be. The following is solid red from the start to the 10% mark, transitions from red to blue over 80% of the turn, with the final 10% being solid blue. The midpoint of the red to blue gradient change, however, is at the 20% mark rather than the 50% mark as would have happened without the 80grad, or 20%, color hint.

```
conic-gradient(red 40grad, 80grad, blue 360grad);
```

If two or more color stops are at the same location, the transition will be a hard line between the first and last colors declared at that location. To use conic gradients to create pie charts — which is NOT the correct way to create pie charts as background images are not accessible — use hard color stops, where the color stop angles for two adjacent color stops are the same. The easiest way to do this is to use multiple position colors stops. The following two declarations are equivalent:

```
conic-gradient(white 0.09turn, #bbbbbb 0.09turn, #bbbbbb 0.27turn, #666666 0.27turn, #666666 0.54turn, black 0.54turn);
conic-gradient(white 0turn 0.09turn, #bbbbbb 0.09turn 0.27turn, #666666 0.27turn 0.54turn, black 0.54turn 1turn);
```

Color stops should be listed in ascending order. Subsequent color stops of lower value will override the value of the previous color stop creating a hard transition. The following changes from red to yellow at the 30% mark, and then transitions from yellow to blue over 35% of the gradient:

```
conic-gradient(red .8rad, yellow .6rad, blue 1.3rad);
```

There are other effects you can create with conic gradients. Oddly, a checkerboard is one of them. By creating quadrants with an upper left and lower right white quadrant and lower left and upper right black quadrants, then repeating the gradient 16 times (four times across and four times down) you can make a checkerboard.

```
conic-gradient(white 90deg, black 0.25turn 0.5turn, white calc(pi * 1rad) calc(pi * 1.5rad), black 300grad);
background-size: 25% 25%;
```

And, yes, you can mix and match different angle units, but don't. The above is hard to read.

Formal syntax

```
<conic-gradient()> =
  conic-gradient( [ <conic-gradient-syntax> ] )

<conic-gradient-syntax> =
  [ [ [ from [ <angle> | <zero> ] ]? [ at <position> ]? ] || <color-interpolation-method> ]? , <angular-color-stop-list>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<angular-color-stop-list> =
  <angular-color-stop> , [ <angular-color-hint>? , <angular-color-stop> ]#?

<position-one> =
  left                 |
  center               |
  right                |
  top                  |
  bottom               |
  x-start              |
  x-end                |
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<angular-color-stop> =
  <color> <color-stop-angle>?

<angular-color-hint> =
  <angle-percentage>  |
  <zero>

<length-percentage> =
  <length>      |
  <percentage>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-angle> =
  [ <angle-percentage> | <zero> ]{1,2}

<angle-percentage> =
```

```
  <angle>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. While it is possible to create pie charts, checkerboards, and other effects with conic gradients, CSS images provide no native way to assign alternative text, and therefore the image represented by the conic gradient will not be accessible to screen reader users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Examples

Gradient at 40-degrees

```
div {
  width: 100px;
  height: 100px;
}
```

```
<div></div>
```

```
div {
  background-image: conic-gradient(from 40deg, white, black);
}
```

Off-centered gradient

```
div {
  width: 100px;
  height: 100px;
}
```

```
<div></div>
```

```
div {
  background: conic-gradient(from 0deg at 0% 25%, blue, green, yellow 180deg);
}
```

Gradient pie-chart

This example uses multi-position color stops, with adjacent colors having the same color stop value, creating a striped effect.

```
div {
  width: 100px;
  height: 100px;
}
```

```
<div></div>
```

```
div {
  background: conic-gradient(red 36deg, orange 36deg 170deg, yellow 170deg);
  border-radius: 50%;
}
```

Checkerboard

```
div {
  width: 100px;
  height: 100px;
}
```

```
<div></div>
```

```
div {
  background: conic-gradient(
      white 0.25turn,
      black 0.25turn 0.5turn,
      white 0.5turn 0.75turn,
      black 0.75turn
    )
    top left / 25% 25% repeat;
  border: 1px solid;
}
```

Interpolating with hue

```
<div class="shorter"></div>
<div class="longer"></div>
```

```
div {
  display: inline-block;
  margin-top: 1rem;
  width: 45vw;
  height: 80vh;
}

.shorter::before {
  content: "shorter hue";
  display: block;
  margin-top: -1rem;
```

```
}

.longer::before {
  content: "longer hue";
  display: block;
  margin-top: -1rem;
}
```

In this example for interpolation hsl color system is being used and hue is being interpolated.

```
.shorter {
  background-image: conic-gradient(in hsl shorter hue, red, blue);
}

.longer {
  background-image: conic-gradient(in hsl longer hue, red, blue);
}
```

The box on the left uses shorter interpolation, meaning color goes straight from red to blue using the shorter arc on the color wheel. The box on the right uses longer interpolation, meaning the color goes from red to blue using the longer arc, traversing through greens, yellows, and oranges.

Color Wheel

```
<div></div>
```

```
div {
  width: 100px;
  height: 100px;
  border-radius: 50px;
}
```

```
div {
  background-image: conic-gradient(
    in hsl longer hue,
    hsl(360 100% 50%),
    hsl(0 100% 50%)
  );
}
```

More conic-gradient examples

Please see Using CSS gradients for more examples.

See also

- Using CSS gradients
- Other gradient functions: `repeating-conic-gradient()`, `linear-gradient()`, `repeating-linear-gradient()`, `radial-gradient()`, `repeating-radial-gradient()`
- `<hue-interpolation-method>`
- `<color-interpolation-method>`
- `<image>`
- `image()`
- `element()`
- `image-set()`
- `cross-fade()`

linear-gradient() CSS function

The **`linear-gradient()`** CSS function creates an image consisting of a progressive transition between two or more colors along a straight line. Its result is an object of the `<gradient>` data type, which is a special kind of `<image>`.

Try it

```
background: linear-gradient(#e66465, #9198e5);
```

```
background: linear-gradient(0.25turn, #3f87a6, #ebf8e1, #f69d3c);
```

```
background: linear-gradient(
  to left,
  #333333,
  #333333 50%,
  #eeeeee 75%,
  #333333 75%
);
```

```
background:
  linear-gradient(217deg, rgb(255 0 0 / 0.8), transparent 70.71%),
  linear-gradient(127deg, rgb(0 255 0 / 0.8), transparent 70.71%),
  linear-gradient(336deg, rgb(0 0 255 / 0.8), transparent 70.71%);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

Syntax

```
/* A gradient with a single color of red */
linear-gradient(red)

/* A gradient tilted 45 degrees,
   starting blue and finishing red */
linear-gradient(45deg, blue, red)
```

```
/* A gradient going from the bottom right to the top left corner,
   starting blue and finishing red */
linear-gradient(to left top, blue, red)

/* Interpolation in rectangular color space */
linear-gradient(in oklab, blue, red)

/* Interpolation in polar color space */
linear-gradient(in hsl, blue, red)

/* Interpolation in polar color space
  with longer hue interpolation method */
linear-gradient(in hsl longer hue, blue, red)

/* Color stop: A gradient going from the bottom to top,
   starting blue, turning green at 40% of its length,
   and finishing red */
linear-gradient(0deg, blue, green 40%, red)

/* Color hint: A gradient going from the left to right,
   starting red, getting to the midpoint color
   10% of the way across the length of the gradient,
   taking the rest of the 90% of the length to change to blue */
linear-gradient(.25turn, red, 10%, blue)

/* Multi-position color stop: A gradient tilted 45 degrees,
   with a red bottom-left half and a blue top-right half,
   with a hard line where the gradient changes from red to blue */
linear-gradient(45deg, red 0 50%, blue 50% 100%)
```

Values

`<side-or-corner>`

The position of the gradient line's starting point. If specified, it consists of the word `to` and up to two keywords: one indicates the horizontal side (`left` or `right`), and the other the vertical side (`top` or `bottom`). The order of the side keywords does not matter. If unspecified, it defaults to `to bottom`.

The values `to top`, `to bottom`, `to left`, and `to right` are equivalent to the angles `0deg`, `180deg`, `270deg`, and `90deg`, respectively. The other values are translated into an angle.

`<angle>`

The gradient line's angle of direction. A value of `0deg` is equivalent to `to top`; increasing values rotate clockwise from there.

`<linear-color-stop>`

A color-stop's `<color>` value, followed by one or two optional stop positions, (each being either a `<percentage>` or a `<length>` along the gradient's axis).

`<color-hint>`

An interpolation hint defining how the gradient progresses between adjacent color stops. The length defines at which point between two color stops the gradient color should reach the midpoint of the color transition. If omitted, the midpoint of the color transition is the midpoint between two color stops.

> **Note:** Rendering of color stops in CSS gradients follows the same rules as color stops in SVG gradients.

Description

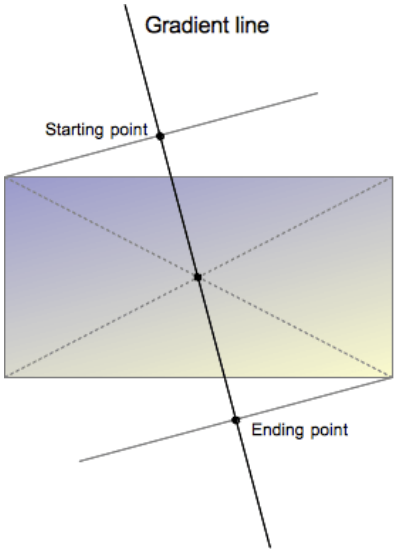
As with any gradient, a linear gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element it applies to.

To create a linear gradient that repeats to fill its container, use the `repeating-linear-gradient()` function instead.

Because `<gradient>`s belong to the `<image>` data type, they can only be used where `<image>`s can be used. For this reason, `linear-gradient()` won't work on `background-color` and other properties that use the `<color>` data type.

Composition of a linear gradient

A linear gradient is defined by an axis—the *gradient line*—and two or more *color-stop points*. Each point on the axis is a distinct color; to create a smooth gradient, the `linear-gradient()` function draws a series of colored lines perpendicular to the gradient line, each one matching the color of the point where it intersects the gradient line.



The gradient line is defined by the center of the box containing the gradient image and by an angle. The colors of the gradient are determined by two or more points: the starting point, the ending point, and, in between, optional color-stop points.

The *starting point* is the location on the gradient line where the first color begins. The *ending point* is the point where the last color ends. Each of these two points is defined by the intersection of the gradient line with a perpendicular line passing from the box corner which is in the same quadrant. The ending point can be understood as the symmetrical point of the starting point. These somewhat complex definitions lead to an interesting effect sometimes called *magic corners*: the corners nearest to the starting and ending points have the same color as their respective starting or ending points.

Customizing Gradients

By adding more color-stop points on the gradient line, you can create a highly customized transition between multiple colors. A color-stop's position can be explicitly defined by using a `<length>` or a `<percentage>`. If you don't specify the location of a color, it is placed halfway between the one that precedes it and the one that follows it. The following two gradients are equivalent.

```
linear-gradient(red, orange, yellow, green, blue);
linear-gradient(red 0%, orange 25%, yellow 50%, green 75%, blue 100%);
```

By default, colors transition smoothly from the color at one color-stop to the color at the subsequent color-stop, with the midpoint between the colors being the halfway point between the color transition. You can move this midpoint to any position between two color-stops by adding an unlabeled % color hint between the two colors to indicate where the middle of the color transition should be. The following example is solid red from the start to the 10% mark and solid blue from 90% to the end. Between 10% and 90% the color transitions from red to blue, however, the midpoint of the transition is at the 30% mark rather than 50% as would have happened without the 30% color hint.

```
linear-gradient(red 10%, 30%, blue 90%);
```

If two or more color-stops are at the same location, the transition will be a hard line between the first and last colors declared at that location.

Color-stops should be listed in ascending order. Subsequent color-stops of lower value will override the value of the previous color-stop creating a hard transition. The following changes from red to yellow at the 40% mark, and then transitions from yellow to blue over 25% of the gradient:

```
linear-gradient(red 40%, yellow 30%, blue 65%);
```

Multi-position color-stops are allowed. A color can be declared as two adjacent color-stops by including both positions in the CSS declaration. The following three gradients are equivalent:

```
linear-gradient(red 0%, orange 10%, orange 30%, yellow 50%, yellow 70%, green 90%, green 100%);
linear-gradient(red, orange 10% 30%, yellow 50% 70%, green 90%);
linear-gradient(red 0%, orange 10% 30%, yellow 50% 70%, green 90% 100%);
```

By default, if there is no color with a `0%` stop, the first color declared will be at that point. Similarly, the last color will continue to the `100%` mark, or be at the `100%` mark if no length has been declared on that last stop.

Formal syntax

```
<linear-gradient()> =
  linear-gradient( [ <linear-gradient-syntax> ] )

<linear-gradient-syntax> =
  [ [ <angle> | <zero> | to <side-or-corner> ] || <color-interpolation-method> ]? , <color-stop-list>

<side-or-corner> =
  [ left | right ]  ||
  [ top | bottom ]

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<color-stop-list> =
  <linear-color-stop> , [ <linear-color-hint>? , <linear-color-stop> ]#?

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<linear-color-stop> =
  <color> <color-stop-length>?

<linear-color-hint> =
  <length-percentage>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-length> =
  <length-percentage>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 3, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Gradient at a 45-degree angle

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background: linear-gradient(45deg, red, blue);
}
```

Gradient that starts at 60% of the gradient line

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background: linear-gradient(135deg, orange 60%, cyan);
}
```

Interpolation in rectangular color space

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background: linear-gradient(90deg in oklab, blue, red);
}
```

Interpolating with hue

```
<div class="shorter">shorter hue</div>
<div class="longer">longer hue</div>
```

```
div {
  height: 50vh;
  color: white;
  font-weight: bolder;
}
```

In this example for interpolation, hsl color system is being used and hue is being interpolated.

```
.shorter {
  background: linear-gradient(90deg in hsl shorter hue, red, blue);
}

.longer {
  background: linear-gradient(90deg in hsl longer hue, red, blue);
}
```

The box on the top uses shorter interpolation, meaning color goes straight from red to blue using the shorter arc on color wheel. The box on the bottom uses longer interpolation, meaning the color goes from red to blue using the longer arc, traversing through greens, yellows, and oranges.

Gradient with multi-position color-stops

This example uses multi-position color-stops, with adjacent colors having the same color-stop value, creating a striped effect.

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background: linear-gradient(
    to right,
    red 20%,
    orange 20% 40%,
    yellow 40% 60%,
    green 60% 80%,
    blue 80%
  );
}
```

More linear-gradient examples

Please see using CSS gradients for more examples.

See also

- Using CSS gradients
- Other gradient functions: `repeating-linear-gradient()`, `radial-gradient()`, `repeating-radial-gradient()`, `conic-gradient()`, `repeating-conic-gradient()`
- `<hue-interpolation-method>`
- `<color-interpolation-method>`
- `<image>`
- CSS images module

radial-gradient() CSS function

The **`radial-gradient()`** CSS function creates an image consisting of a progressive transition between two or more colors that radiate from an origin. Its shape may be a circle or an ellipse. The function's result is an object of the `<gradient>` data type, which is a special kind of `<image>`.

Try it

```
background: radial-gradient(#e66465, #9198e5);
```

```
background: radial-gradient(closest-side, #3f87a6, #ebf8e1, #f69d3c);
```

```
background: radial-gradient(
  circle at 100%,
  #333333,
  #333333 50%,
  #eeeeee 75%,
  #333333 75%
);
```

```
background:
  radial-gradient(ellipse at top, #e66465, transparent),
  radial-gradient(ellipse at bottom, #4d9f0c, transparent);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

Syntax

```
/* A gradient with a single color of red */
radial-gradient(red)

/* A gradient at the center of its container,
   starting red, changing to blue, and finishing green */
radial-gradient(circle at center, red 0, blue, green 100%)

/* hsl color space with longer hue interpolation */
radial-gradient(circle at center in hsl longer hue, red 0, blue, green 100%)
```

A radial gradient is specified by indicating the center of the gradient (where the 0% ellipse will be) and the size and shape of the *ending shape* (the 100% ellipse).

Values

`<position>`

The position of the gradient, interpreted in the same way as `background-position` or `transform-origin`. If unspecified, it defaults to `center`.

`<ending-shape>`

The gradient's ending-shape. The value can be `circle` (meaning that the gradient's shape is a circle with a constant radius) or `ellipse` (meaning that the shape is an axis-aligned ellipse). If unspecified, it defaults to `ellipse`.

`<size>`

Determines the size of the gradient's ending shape. If omitted it defaults to farthest-corner. It can be given explicitly or by keyword. For the purpose of the keyword definitions, consider the gradient box edges as extending infinitely in both directions, rather than being finite line segments.

Both circle and ellipse gradients accept the following keywords for their `<size>`:

Keyword	Description
`closest-side`	The gradient's ending shape meets the side of the box closest to its center (for circles) or meets both the vertical and horizontal sides closest to the center (for ellipses).
`closest-corner`	The gradient's ending shape is sized so that it exactly meets the closest corner of the box from its center.
`farthest-side`	Similar to `closest-side`, except the ending shape is sized to meet the side of the box farthest from its center (or vertical and horizontal sides).
`farthest-corner`	The default value, the gradient's ending shape is sized so that it exactly meets the farthest corner of the box from its center.

If `<ending-shape>` is specified as `circle`, the size may be given explicitly as a `<length>`, which provides an explicit circle radius. Negative values are invalid.

If `<ending-shape>` is specified as `ellipse`, the size may be given as a `<length-percentage>` with two values to provide an explicit ellipse size. The first value represents the horizontal radius and the second is the vertical radius. Percentage values are relative to the corresponding dimension of the gradient box. Negative values are invalid.

When the `<ending-shape>` keyword is omitted, the gradient shape is determined by the size given. One `<length>` value provides a circle, while two values in `<length-percentage>` units provide an ellipse. A single `<percentage>` value is not valid.

`<linear-color-stop>`

A color-stop's `<color>` value, followed by one or two optional stop positions (either a `<percentage>` or a `<length>` along the gradient's axis). A percentage of `0%`, or a length of `0`, represents the center of the gradient; the value `100%` represents the intersection of the ending shape with the virtual gradient ray. Percentage values in between are linearly positioned on the gradient ray. Including two stop positions is equivalent to declaring two color stops with the same color at the two positions.

`<color-hint>`

The color-hint is an interpolation hint defining how the gradient progresses between adjacent color stops. The length defines at which point between two color stops the gradient color should reach the midpoint of the color transition. If omitted, the midpoint of the color transition is the midpoint between two color stops.

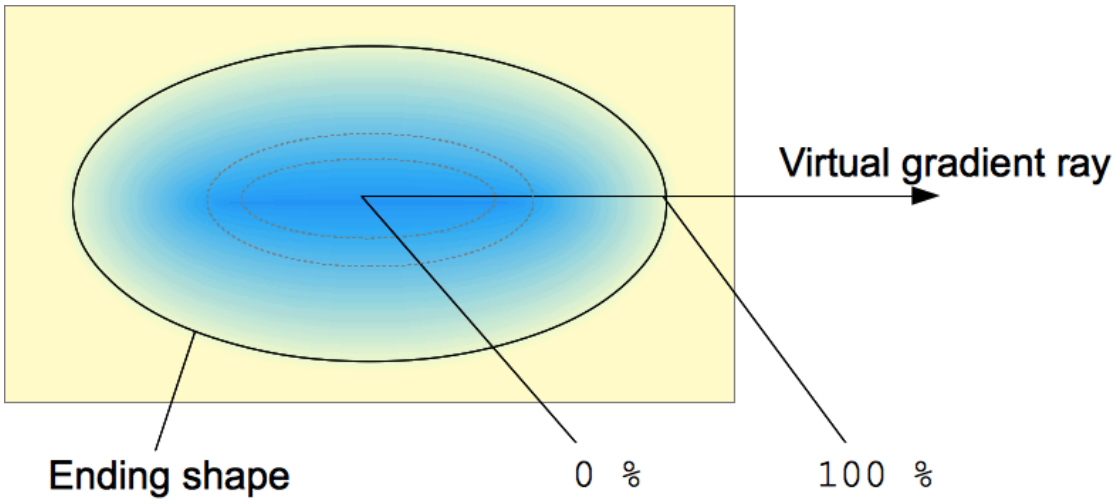
Description

As with any gradient, a radial gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element it applies to.

To create a radial gradient that repeats so as to fill its container, use the `repeating-radial-gradient()` function instead.

Because `<gradient>`s belong to the `<image>` data type, they can only be used where `<image>`s can be used. For this reason, `radial-gradient()` won't work on `background-color` and other properties that use the `<color>` data type.

Composition of a radial gradient



A radial gradient is defined by a *center point*, an *ending shape*, and two or more *color-stop points*.

To create a smooth gradient, the `radial-gradient()` function draws a series of concentric shapes radiating out from the center to the *ending shape* (and potentially beyond). The ending shape may be either a circle or an ellipse.

Color-stop points are positioned on a *virtual gradient ray* that extends horizontally from the center towards the right. Percentage-based color-stop positions are relative to the intersection between the ending shape and this gradient ray, which represents `100%`. Each shape is a single color determined by the color on the gradient ray it intersects.

Formal syntax

```
<radial-gradient()> =
  radial-gradient( [ <radial-gradient-syntax> ] )

<radial-gradient-syntax> =
  [ [ [ <radial-shape> || <radial-size> ]? [ at <position> ]? ] || <color-interpolation-method> ]? , <color-stop-list>

<radial-shape> =
  circle   |
  ellipse

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<color-stop-list> =
  <linear-color-stop> , [ <linear-color-hint>? , <linear-color-stop> ]#?

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
```

```
  a98-rgb           |
  prophoto-rgb      |
  rec2020           |
  lab               |
  oklab             |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<linear-color-stop> =
  <color> <color-stop-length>?

<linear-color-hint> =
  <length-percentage>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-length> =
  <length-percentage>{1,2}
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 3, CSS Images Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic gradient

```
<div class="radial-gradient"></div>
```

```
.radial-gradient {
  width: 240px;
  height: 120px;
}
```

```
.radial-gradient {
  background-image: radial-gradient(cyan 0%, transparent 20%, salmon 40%);
}
```

Non-centered gradient

```
<div class="radial-gradient"></div>
```

```
.radial-gradient {
  width: 240px;
  height: 120px;
}
```

```
.radial-gradient {
  background-image: radial-gradient(
    farthest-corner at 40px 40px,
    #ff3355 0%,
    #4433ee 100%
  );
}
```

Interpolating with hue

```
<div class="shorter"></div>
<div class="longer"></div>
```

```
div {
  display: inline-block;
  margin-top: 1rem;
  width: 45vw;
  height: 80vh;
}

.shorter::before {
  content: "shorter hue";
  display: block;
  margin-top: -1rem;
}

.longer::before {
  content: "longer hue";
  display: block;
  margin-top: -1rem;
}
```

In this example for interpolation, hsl color system is being used and hue is being interpolated.

```
.shorter {
  background-image: radial-gradient(
    circle at center in hsl shorter hue,
    red,
    blue
  );
}
```

```
.longer {
  background-image: radial-gradient(
    circle at center in hsl longer hue,
    red,
    blue
  );
}
```

The box on the left uses shorter interpolation, meaning the color goes straight from red to blue using the shorter arc on color wheel. The box on the right uses longer interpolation, meaning the color goes from red to blue using the longer arc, traversing through greens, yellows, and oranges.

More radial-gradient examples

Please see Using CSS gradients for more examples.

See also

- Using CSS gradients
- Other gradient functions: `repeating-radial-gradient()`, `linear-gradient()`, `repeating-linear-gradient()`, `conic-gradient()`, `repeating-conic-gradient()`
- `<hue-interpolation-method>`
- `<color-interpolation-method>`
- `<image>`
- `image()`
- `element()`
- `image-set()`
- `cross-fade()`

repeating-conic-gradient() CSS function

The **`repeating-conic-gradient()`** CSS function creates an image consisting of a repeating gradient (rather than a single gradient) with color transitions rotated around a center point (rather than radiating from the center).

Try it

```
background: repeating-conic-gradient(red 0%, yellow 15%, red 33%);
```

```
background: repeating-conic-gradient(
  from 45deg at 10% 50%,
  brown 0deg 10deg,
  darkgoldenrod 10deg 20deg,
  chocolate 20deg 30deg
);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

Syntax

```
/* Starburst: a blue on blue starburst: the gradient
   is a starburst of lighter and darker blue,
   centered in the upper left quadrant,
   offset by 3degrees so there is no up/down straight line */
repeating-conic-gradient(
  from 3deg at 25% 25%,
  hsl(200 100% 50%) 0deg 15deg,
  hsl(200 100% 60%) 10deg 30deg
)

/* Interpolation in polar color space
  with shorter hue interpolation method */
repeating-conic-gradient(in hsl shorter hue, red, blue 90deg, green 180deg)
```

Values

`<angle>`

> Preceded by the `from` keyterm, and taking an angle as its value, defines the gradient rotation in clockwise direction.

`<position>`

> Using the same length, order and keyterm values as the background-position property, the position defines center of the gradient. If omitted, the default value is `center`, meaning the gradient will be centered.

`<angular-color-stop>`

> A color-stop's `<color>` value, followed by one or two optional stop positions, (an `<angle>` along the gradient's circumference axis). The last color stop minus the first color-stop angle defines the size of the repeating gradient.

`<color-hint>`

> An interpolation hint defining how the gradient progresses between adjacent color stops. The length defines at which point between two color stops the gradient color should reach the midpoint of the color transition. If omitted, the midpoint of the color transition is the midpoint between two color stops.

> **Note:** Rendering of color stops in repeating conic gradients follows the same rules as color stops in linear gradients.

Description

Example repeating conic gradients include starbursts. The result of the `repeating-conic-gradient()` function is an object of the `<gradient>` data type, which is a special kind of `<image>`.

If neither the first nor the last color stops include a color stop angle greater than 0deg or less than 360 degrees respectively, the conic-gradient will not repeat.

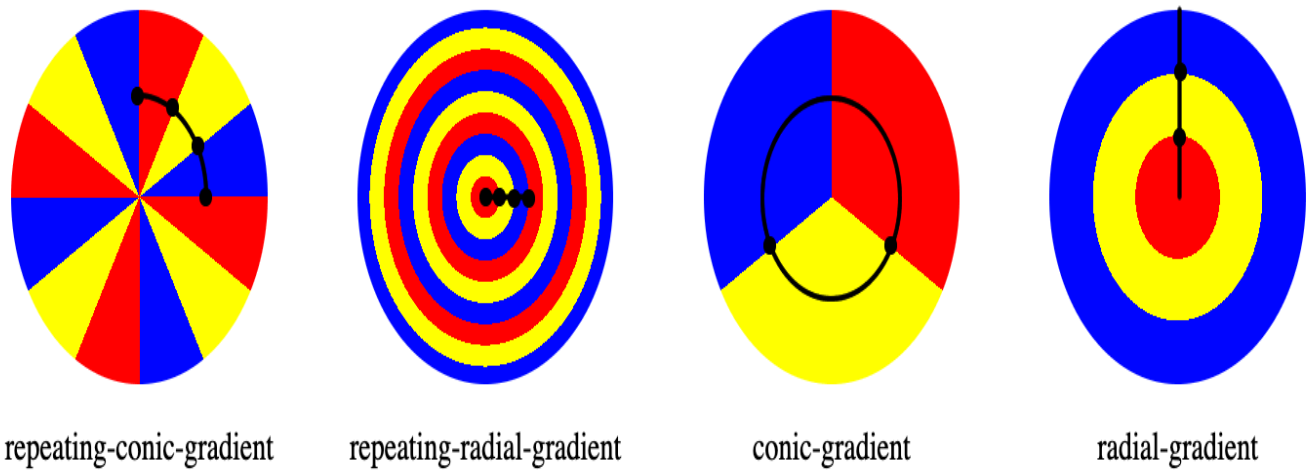
As with any gradient, a repeating-conic gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element it applies to, or the size the `<image>` is set to if it's set to something other than the element's size.

Because `<gradient>`s belong to the `<image>` data type, they can only be used where `<image>`s can be used. For this reason, `repeating-conic-gradient()` won't work on `background-color` and other properties that use the `<color>` data type.

> **Note:** To create a conic gradient that does not repeat, make the gradient a full 360 degree rotation, or use the `conic-gradient()` function instead.

Understanding repeating conic gradients

The repeating-conic-gradient syntax is similar to the `conic-gradient()` and `repeating-radial-gradient()` syntax. Like the non-repeating conic-gradient, the color-stops are placed around a gradient arc. Like the repeating radial gradient, the size of the repeating section is the first color stop subtracted from the angle of the last color stop.



The above gradients are defined as being one third blue, one third red, and one third yellow.

```
repeating-conic-gradient(from 0deg, red 0deg 30deg, yellow 30deg 60deg, blue 60deg 90deg);

repeating-radial-gradient(red 0 8%, yellow 8% 16%, blue 16% 24%);

conic-gradient(red 120deg, yellow 120deg 240deg, blue 240deg);

radial-gradient(red 33%, yellow 33% 66%, blue 66%);
```

For a repeating gradient to repeat we define the first and last color stops. Like in non-repeating gradients, the first and last color stops are assumed to be 0 and either 100% or 360deg if not explicitly declared. When defaulting to these values, the repeating arc is 360 degrees, and therefore doesn't repeat.

Like the non-repeating conic gradient, the color-stops are placed around a gradient arc — the circumference of a circle, rather than on the gradient line emerging from the center of the gradient. The colors transition as if spun around the center of a circle, starting at the top if no `from <angle>` is declared, and going clockwise for the size of the angle that is the different between the largest and smallest color angle, then repeating.

A repeating conic gradient is specified by indicating a rotation angle, the center of the gradient, and then specifying a list of color-stops. Like non-repeating conic gradients, the color-stops of a repeating conic gradient are specified with an `<angle>`. Units include `deg` for degrees, `grad` for gradients, `rad` for radians, and `turn` for turns. There are 360 degrees, 400 gradians, 2π radians, and 1 turn in a circle. Browsers supporting repeating conic gradients also accept percent values, with 100% equaling 360 degrees, but this is not in the specification.

Radial and conic gradient syntax provides for positioning the center of the gradient anywhere within, or even outside, the image. The values for the position are similar to the syntax for 2-value `background-position`.

The gradient arc is part of the circumference of the gradient. 0 degrees is north, or 12:00pm. The colors of the gradient are determined by the angled color stops, their starting points, ending points, and, in between, and optional angled color-stop points. The transitions between colors can be altered with color hints between adjacent colors' color stops.

Customizing gradients

By adding more angled color-stop points on the gradient arc, you can create a highly customized transition between multiple colors. A color-stop's position can be explicitly defined by using an `<angle>`. If you don't specify the location of a color stop, it is placed halfway between the one that precedes it and the one that follows it. Like the non-repeating gradient counterpart, if you don't specify an angle for the first or last color stop, the values will be 0deg and 360deg. If you don't declare an angle for either, you'll get a non-repeating conic gradient. If you declare a non 0 or 360degree for the first or last respectively, the gradient will repeat based on that value. For example, if you don't declare an angle for the first color, and declare 10% on the last color stop, the arc will repeat 10 times. Rather, the starting point is the first color stop declared, and the last color stop is the last color stop angle declared. The following two gradients are equivalent:

```
repeating-conic-gradient(red, orange, yellow, green, blue 50%);
repeating-conic-gradient(from -45deg, red 45deg, orange, yellow, green, blue 225deg)
```

By default, colors transition smoothly from the color at one color stop to the color at the subsequent color stop, with the midpoint between the colors being the half way point between the color transition. You can move this color transition midpoint to any point between two color stops by adding a color hint, indicating where the middle of the color transition should be.

If two or more color stops are at the same location, the transition will be a hard line between the first and last colors declared at that location.

While you can mix and match different angle units, don't. It makes CSS hard to read.

Formal syntax

```
<repeating-conic-gradient()> =
  repeating-conic-gradient( [ <conic-gradient-syntax> ] )

<conic-gradient-syntax> =
  [ [ [ from [ <angle> | <zero> ] ]? [ at <position> ]? ] || <color-interpolation-method> ]? , <angular-color-stop-list>

<position> =
  <position-one>    |
  <position-two>    |
  <position-four>
```

```
<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<angular-color-stop-list> =
  <angular-color-stop> , [ <angular-color-hint>? , <angular-color-stop> ]#?

<position-one> =
  left                 |
  center               |
  right                |
  top                  |
  bottom               |
  x-start              |
  x-end                |
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<angular-color-stop> =
  <color> <color-stop-angle>?

<angular-color-hint> =
  <angle-percentage>  |
  <zero>

<length-percentage> =
  <length>      |
  <percentage>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-angle> =
  [ <angle-percentage> | <zero> ]{1,2}

<angle-percentage> =
  <angle>       |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. While it is possible to create pie charts, checkerboards, and other effects with conic gradients, CSS images provide no native way to assign alternative text, and therefore the image represented by the conic gradient will not be accessible to screen reader users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Examples

Black and white starburst

```
div {
  width: 200px;
  height: 200px;
}
```

```
<div></div>
```

```
div {
  background-image: repeating-conic-gradient(white 0 9deg, black 9deg 18deg);
}
```

Off-centered gradient

This gradient repeats 18 times, but since we only see the right half, we only see 9 repeats.

```
div {
  width: 200px;
  height: 200px;
}
```

```
<div></div>
```

```
div {
  background: repeating-conic-gradient(
    from 3deg at 25% 25%,
    green,
    blue 2deg 5deg,
    green,
    yellow 15deg 18deg,
    green 20deg
  );
}
```

Interpolating with hue

```
<div class="shorter"></div>
<div class="longer"></div>
```

```
div {
  display: inline-block;
  margin-top: 1rem;
  width: 45vw;
  height: 80vh;
}

.shorter::before {
  content: "shorter hue";
  display: block;
  margin-top: -1rem;
}

.longer::before {
  content: "longer hue";
  display: block;
  margin-top: -1rem;
}
```

In this example for interpolation, hsl color system is being used and hue is being interpolated.

```
.shorter {
  background-image: repeating-conic-gradient(
    in hsl shorter hue,
    red,
    blue 180deg
  );
}

.longer {
  background-image: repeating-conic-gradient(
    in hsl longer hue,
    red,
    blue 180deg
  );
}
```

The box on the left uses shorter interpolation, meaning color goes straight from red to blue using the shorter arc on the color wheel. The box on the right uses longer interpolation, meaning the color goes from red to blue using the longer arc, traversing through greens, yellows, and oranges.

More repeating-conic-gradient examples

Please see Using CSS gradients for more examples.

See also

- Using CSS gradients
- Other gradient functions: `conic-gradient()`, `linear-gradient()`, `repeating-linear-gradient()`, `radial-gradient()`, `repeating-radial-gradient()`
- `<hue-interpolation-method>`
- `<color-interpolation-method>`
- `<image>`
- `image()`
- `element()`
- `image-set()`
- `cross-fade()`

repeating-linear-gradient() CSS function

The **`repeating-linear-gradient()`** CSS function creates an image consisting of repeating linear gradients. It is similar to `linear-gradient()` and takes the same arguments, but it repeats the color stops infinitely in all directions so as to cover its entire container. The function's result is an object of the `<gradient>` data type, which is a special kind of `<image>`.

Try it

```
background: repeating-linear-gradient(
  #e66465,
```

```
  #e66465 20px,
  #9198e5 20px,
  #9198e5 25px
);
```

```
background: repeating-linear-gradient(45deg, #3f87a6, #ebf8e1 15%, #f69d3c 20%);
```

```
background:
  repeating-linear-gradient(transparent, #4d9f0c 40px),
  repeating-linear-gradient(0.25turn, transparent, #3f87a6 20px);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

The length of the gradient that repeats is the distance between the first and last color stop. If the first color does not have a color-stop-length, the color-stop-length defaults to 0. With each repetition, the positions of the color stops are shifted by a multiple of the length of the basic linear gradient. Thus, the position of each ending color stop coincides with a starting color stop; if the color values are different, this will result in a sharp visual transition. This can be altered with repeating the first color again as the last color.

As with any gradient, a repeating linear gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element it applies to.

Because `<gradient>`s belong to the `<image>` data type, they can only be used where `<image>`s can be used. For this reason, `repeating-linear-gradient()` won't work on `background-color` and other properties that use the `<color>` data type.

Syntax

```
/* A repeating gradient tilted 45 degrees,
   starting blue and finishing red, repeating 3 times */
repeating-linear-gradient(45deg, blue, red 33.3%)

/* A repeating gradient going from the bottom right to the top left,
   starting blue and finishing red, repeating every 20px */
repeating-linear-gradient(to left top, blue, red 20px)

/* A gradient going from the bottom to top,
   starting blue, turning green after 40%,
   and finishing red. This gradient doesn't repeat because
   the last color stop defaults to 100% */
repeating-linear-gradient(0deg, blue, green 40%, red)

/* A gradient repeating five times, going from the left to right,
   starting red, turning green, and back to red */
repeating-linear-gradient(to right, red 0%, green 10%, red 20%)

/* Interpolation in rectangular color space */
repeating-linear-gradient(in oklab, blue, red 50px)

/* Interpolation in polar color space */
repeating-linear-gradient(in hsl, blue, red 50px)

/* Interpolation in polar color space
  with longer hue interpolation method */
repeating-linear-gradient(in hsl longer hue, blue, red 50px)
```

Values

`<side-or-corner>`

The position of the gradient line's starting point. If specified, it consists of the word `to` and up to two keywords: one indicates the horizontal side (`left` or `right`), and the other the vertical side (`top` or `bottom`). The order of the side keywords does not matter. If unspecified, it defaults to `to bottom`.

The values `to top`, `to bottom`, `to left`, and `to right` are equivalent to the angles `0deg`, `180deg`, `270deg`, and `90deg` respectively. The other values are translated into an angle.

`<angle>`

The gradient line's angle of direction. A value of `0deg` is equivalent to `to top`; increasing values rotate clockwise from there.

`<linear-color-stop>`

A color-stop's `<color>` value, followed by one or two optional stop positions, (each being either a `<percentage>` or a `<length>` along the gradient's axis). A percentage of `0%`, or a length of `0`, represents the start of the gradient; the value `100%` is 100% of the image size, meaning the gradient will not repeat.

`<color-hint>`

The color-hint is an interpolation hint defining how the gradient progresses between adjacent color stops. The length defines at which point between two color stops the gradient color should reach the midpoint of the color transition. If omitted, the midpoint of the color transition is the midpoint between two color stops.

> **Note:** Rendering of color stops in repeating linear gradients follows the same rules as color stops in linear gradients.

Formal syntax

```
<repeating-linear-gradient()> =
  repeating-linear-gradient( [ <linear-gradient-syntax> ] )

<linear-gradient-syntax> =
  [ [ <angle> | <zero> | to <side-or-corner> ] || <color-interpolation-method> ]? , <color-stop-list>

<side-or-corner> =
  [ left | right ]  ||
  [ top | bottom ]

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<color-stop-list> =
```

```
  <linear-color-stop> , [ <linear-color-hint>? , <linear-color-stop> ]#?

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<linear-color-stop> =
  <color> <color-stop-length>?

<linear-color-hint> =
  <length-percentage>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-length> =
  <length-percentage>{1,2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 4, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Zebra stripes

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background-image: repeating-linear-gradient(
    -45deg,
    transparent,
    transparent 20px,
    black 20px,
    black 40px
  );
  /* with multiple color stop lengths */
  background-image: repeating-linear-gradient(
    -45deg,
    transparent 0 20px,
    black 20px 40px
  );
}
```

Ten repeating horizontal bars

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background-image: repeating-linear-gradient(
    to bottom,
    rgb(26 198 204),
    rgb(26 198 204) 7%,
    rgb(100 100 100) 10%
  );
}
```

Because the last color stop is 10% and the gradient is vertical, each gradient in the repeated gradient is 10% of the height, fitting 10 horizontal bars.

Interpolation in rectangular color space

```
body {
  width: 100vw;
  height: 100vh;
}
```

```
body {
  background: repeating-linear-gradient(90deg in oklab, blue, red 100px);
}
```

Interpolating with hue

```
<div class="shorter">shorter hue</div>
<div class="longer">longer hue</div>
```

```
div {
  height: 50vh;
  color: #333300;
  font-weight: bolder;
  padding-left: 1.5rem;
}
```

In this example for interpolation, hsl color system is being used and hue is being interpolated.

```
.shorter {
  background: repeating-linear-gradient(
    90deg in hsl shorter hue,
    red,
    blue 300px
  );
}

.longer {
  background: repeating-linear-gradient(
    90deg in hsl longer hue,
    red,
    blue 300px
  );
}
```

The box on the top uses shorter interpolation, meaning the color goes from red to blue using the shorter arc on the color wheel. The box on the bottom uses longer interpolation, meaning the color goes from red to blue using the longer arc, traversing through greens, yellows, and oranges.

> **Note:** Please see Using CSS gradients for more examples.

See also

- Using CSS gradients
- Other gradient functions: `linear-gradient()`, `radial-gradient()`, `repeating-radial-gradient()`, `conic-gradient()`, `repeating-conic-gradient()`
- `<hue-interpolation-method>`
- `<color-interpolation-method>`
- `<image>`
- `image()`
- `element()`
- `image-set()`
- `cross-fade()`

repeating-radial-gradient() CSS function

The **`repeating-radial-gradient()`** CSS function creates an image consisting of repeating gradients that radiate from an origin. It is similar to `radial-gradient()` and takes the same arguments, but it repeats the color stops infinitely in all directions so as to cover its entire container, similar to `repeating-linear-gradient()`. The function's result is an object of the `<gradient>` data type, which is a special kind of `<image>`.

Try it

```
background: repeating-radial-gradient(#e66465, #9198e5 20%);
```

```
background: repeating-radial-gradient(closest-side, #3f87a6, #ebf8e1, #f69d3c);
```

```
background: repeating-radial-gradient(
  circle at 100%,
  #333333,
  #333333 10px,
  #eeeeee 10px,
  #eeeeee 20px
);
```

```
<section class="display-block" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  min-height: 100%;
}
```

With each repetition, the positions of the color stops are shifted by a multiple of the dimensions of the basic radial gradient (the distance between the last color stop and the first). Thus, the position of each ending color stop coincides with a starting color stop; if the color values are different, this will result in a sharp visual transition, which can be mitigated by repeating the first color as the last color.

As with any gradient, a repeating radial gradient has no intrinsic dimensions; i.e., it has no natural or preferred size, nor a preferred ratio. Its concrete size will match the size of the element it applies to.

Because `<gradient>`s belong to the `<image>` data type, they can only be used where `<image>`s can be used. For this reason, `repeating-radial-gradient()` won't work on `background-color` and other properties that use the `<color>` data type.

Syntax

```
/* A gradient at the center of its container,
   starting red, changing to blue, and finishing green,
   with the colors repeating every 30px */
repeating-radial-gradient(circle at center, red 0, blue, green 30px)

/* An elliptical gradient near the top left of its container,
   starting red, changing to green and back again,
   repeating five times between the center and the bottom right corner,
   and only once between the center and the top left corner */
repeating-radial-gradient(farthest-corner at 20% 20%, red 0, green, red 20%)
```

Values

`<position>`

The position of the gradient, interpreted in the same way as `background-position` or `transform-origin`. If unspecified, it defaults to `center`.

`<shape>`

The gradient's shape. The value can be `circle` (meaning that the gradient's shape is a circle with constant radius) or `ellipse` (meaning that the shape is an axis-aligned ellipse). If unspecified, it defaults to `ellipse`.

`<extent-keyword>`

A keyword describing how big the ending shape must be. The possible values are:

Keyword	Description
`closest-side`	The gradient's ending shape meets the side of the box closest to its center (for circles) or meets both the vertical and horizontal sides closest to the center (for ellipses).
`closest-corner`	The gradient's ending shape is sized so that it exactly meets the closest corner of the box from its center.
`farthest-side`	Similar to `closest-side`, except the ending shape is sized to meet the side of the box farthest from its center (or vertical and horizontal sides).
`farthest-corner`	The gradient's ending shape is sized so that it exactly meets the farthest corner of the box from its center.

> **Note:** Early implementations of this function included other keywords (`cover` and `contain`) as synonyms of the standard `farthest-corner` and `closest-side`, respectively. Use the standard keywords only, as some implementations have already dropped those older variants.

`<color-stop>`

A color-stop's `<color>` value, followed by an optional stop position (either a `<percentage>` or a `<length>` along the gradient's axis). A percentage of `0%`, or a length of `0`, represents the center of the gradient; the value `100%` represents the intersection of the ending shape with the virtual gradient ray. Percentage values in between are linearly positioned on the virtual gradient ray.

Formal syntax

```
<repeating-radial-gradient()> =
  repeating-radial-gradient( [ <radial-gradient-syntax> ] )

<radial-gradient-syntax> =
  [ [ [ <radial-shape> || <radial-size> ]? [ at <position> ]? ] || <color-interpolation-method> ]? , <color-stop-list>

<radial-shape> =
  circle   |
  ellipse

<radial-size> =
  <radial-extent>              |
  <length [0,∞]>               |
  <length-percentage [0,∞]>{2}

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<color-interpolation-method> =
  in [ <rectangular-color-space> | <polar-color-space> <hue-interpolation-method>? | <custom-color-space> ]

<color-stop-list> =
  <linear-color-stop> , [ <linear-color-hint>? , <linear-color-stop> ]#?

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<rectangular-color-space> =
  srgb               |
  srgb-linear        |
  display-p3         |
  display-p3-linear  |
```

```
  a98-rgb            |
  prophoto-rgb       |
  rec2020            |
  lab                |
  oklab              |
  <xyz-space>

<polar-color-space> =
  hsl    |
  hwb    |
  lch    |
  oklch

<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue

<custom-color-space> =
  <dashed-ident>

<linear-color-stop> =
  <color> <color-stop-length>?

<linear-color-hint> =
  <length-percentage>

<xyz-space> =
  xyz      |
  xyz-d50  |
  xyz-d65

<color-stop-length> =
  <length-percentage>{1,2}
```

This syntax reflects the latest standard as per CSS Color Module Level 4, CSS Color Module Level 5, CSS Images Module Level 3, CSS Images Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Black and white gradient

```
<div class="radial-gradient"></div>
```

```
.radial-gradient {
  width: 120px;
  height: 120px;
}
```

```
.radial-gradient {
  background: repeating-radial-gradient(
    black,
    black 5px,
    white 5px,
    white 10px
  );
}
```

Farthest-corner

```
<div class="radial-gradient"></div>
```

```
.radial-gradient {
  width: 240px;
  height: 120px;
}
```

```
.radial-gradient {
  background: repeating-radial-gradient(
    ellipse farthest-corner at 20% 20%,
    red,
    black 5%,
    blue 5%,
    green 10%
  );
  background: repeating-radial-gradient(
    ellipse farthest-corner at 20% 20%,
    red 0 5%,
    green 5% 10%
  );
}
```

The elliptical gradient will be centered 20% from the top left, and will repeat 10 times between the center and the farthest corner (the bottom right corner). Browsers supporting multi position color stops will display a red and green striped ellipse. Browsers not supporting the syntax yet will see a gradient that goes from red to black and then from blue to green.

Interpolating with hue

```
<div class="shorter"></div>
<div class="longer"></div>
```

```
div {
  display: inline-block;
  margin-top: 1rem;
  width: 45vw;
  height: 80vh;
}

.shorter::before {
  content: "shorter hue";
  display: block;
  margin-top: -1rem;
}
```

```
.longer::before {
  content: "longer hue";
  display: block;
  margin-top: -1rem;
}
```

In this example for interpolation, hsl color system is being used and hue is being interpolated.

```
.shorter {
  background-image: repeating-radial-gradient(
    circle at center in hsl shorter hue,
    red 30px,
    blue 60px
  );
}

.longer {
  background-image: repeating-radial-gradient(
    circle at center in hsl longer hue,
    red 30px,
    blue 60px
  );
}
```

The box on the left uses shorter interpolation, meaning the color goes from red to blue using the shorter arc on color wheel. The box on the right uses longer interpolation, meaning the color goes from red to blue using the longer arc, traversing through greens, yellows, and oranges.

> **Note:** Please see Using CSS gradients for more examples.

See also

- Using CSS gradients
- Other gradient functions: `radial-gradient()`, `linear-gradient()`, `repeating-linear-gradient()`, `conic-gradient()`, `repeating-conic-gradient()`
- `<hue-interpolation-method>`
- `<color-interpolation-method>`
- `<image>`
- `image()`
- `element()`
- `image-set()`
- `cross-fade()`

<hex-color> CSS type

The **`<hex-color>`** CSS data type is a notation for describing the *hexadecimal color syntax* of an sRGB color using its primary color components (red, green, blue) written as hexadecimal numbers, as well as its transparency.

A `<hex-color>` value can be used everywhere where a `<color>` can be used.

Syntax

```
#RGB        // The three-value syntax
#RGBA       // The four-value syntax
#RRGGBB     // The six-value syntax
#RRGGBBAA   // The eight-value syntax
```

Value

`R` or `RR`

The *red* component of the color, as a case-insensitive hexadecimal number between `0` and `ff` (255). If there is only one number, it is duplicated: `1` means `11`.

`G` or `GG`

The *green* component of the color, as a case-insensitive hexadecimal number between `0` and `ff` (255). If there is only one number, it is duplicated: `c` means `cc`.

`B` or `BB`

The *blue* component of the color, as a case-insensitive hexadecimal number between `0` and `ff` (255). If there is only one number, it is duplicated: `9` means `99`.

`A` or `AA` Optional

The *alpha* component of the color, indicating its transparency, as a case-insensitive hexadecimal number between `0` and `ff` (255). If there is only one number, it is duplicated: `e` means `ee`. `0`, or `00`, represents a fully transparent color, and `f`, or `ff`, a fully opaque one.

> **Note:** The syntax is case-insensitive: `#00ff00` is the same as `#00FF00`.

Examples

Hexadecimal hot pink

This example includes four hot pink squares, with fully opaque or semi-transparent backgrounds created using four different-length case-insensitive hex-color syntaxes.

HTML

```
<div>
  #F09
  <div class="c1"></div>
</div>
<div>
  #f09a
  <div class="c2"></div>
</div>
<div>
  #ff0099
  <div class="c3"></div>
```

```
</div>
<div>
  #FF0099AA
  <div class="c4"></div>
</div>
```

CSS

The hot pink background colors are created using the three-, four-, six-, and eight-value hex notations, using both uppercase and lowercase letters.

```
body {
  display: flex;
  justify-content: space-evenly;
  font-family: monospace;
}
div {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}
```

```
[class] {
  width: 40px;
  height: 40px;
}
.c1 {
  background: #f09;
}
.c2 {
  background: #f09a;
}
.c3 {
  background: #ff0099;
}
.c4 {
  background: #ff0099aa;
}
```

Result

See also

- `<color>` data type
- `<named-color>` data-type
- `rgb()` color function
- CSS color module

<hue> CSS type

The **`<hue>`** CSS data type represents the hue angle of a color. It is used in the color functions that accept hue expressed as a single value, specifically `hsl()`, `hwb()`, `lch()`, and `oklch()` functional notations.

Syntax

A `<hue>` can be either an `<angle>` or a `<number>`.

Values

`<angle>`

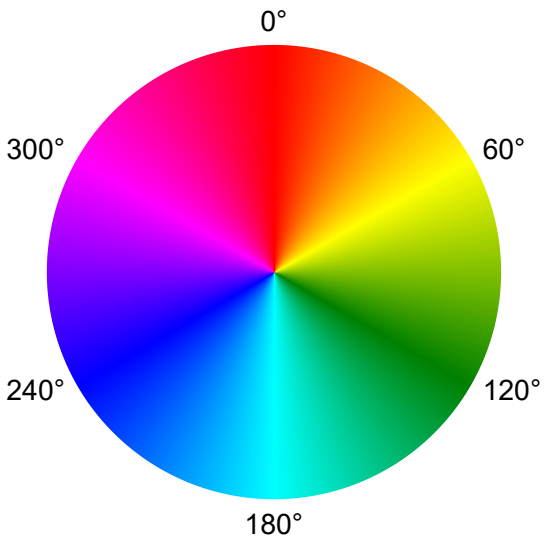
An angle expressed in degrees, gradians, radians, or turns using the `deg`, `grad`, `rad`, or `turn`, respectively.

`<number>`

A real number, representing degrees of the hue angle.

As an `<angle>` is periodic, `<hue>` is normalized to the range `[0deg, 360deg)`. It implicitly wraps around such that `480deg` is the same as `120deg`, `-120deg` is the same as `240deg`, `-1turn` is the same as `1turn`, and so on.

Description



The color wheel above shows hues at all angles in the sRGB color space. In particular, *red* is at `0deg`, *yellow* is at `60deg`, *lime* is at `120deg`, *cyan* is at `180deg`, *blue* is at `240deg`, and *magenta* is at `300deg`.

The angles corresponding to particular hues differ depending on the color space. For example, the hue angle of sRGB green is `120deg` in the sRGB color space, but `134.39deg` in the CIELAB color space.

The following table lists typical colors at various angles in the sRGB (used by `hsl()` and `hwb()`), CIELAB (used by `lch()`), and Oklab (used by `oklch()`) color spaces:

	0°	60°	120°	180°	240°	300°
sRGB						
CIELAB						
Oklab						

Interpolation of <code><hue></code> values

`<hue>` values are interpolated as `<angle>` values, and the default interpolation algorithm is `shorter`. In some color-related CSS functions, this can be overridden by the `<hue-interpolation-method>` component.

Formal syntax

```
<hue> =
  <number>  |
  <angle>
```

This syntax reflects the latest standard as per CSS Color Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing the hue of a color using a slider

The following example shows the effect of changing the `hue` value of the `hsl()` functional notation on a color.

HTML

```
<input type="range" min="0" max="360" value="0" id="hue-slider" />
<p>Hue: <span id="hue-value">0deg</span></p>
<div id="box"></div>
```

CSS

```
div {
  width: 100px;
  height: 100px;
  margin: 10px;
  border: 1px solid black;
}
p {
  font-family: sans-serif;
}
span {
  font-family: monospace;
  background: rgb(0 0 0 / 10%);
  padding: 3px;
}
#hue-slider {
  width: 90%;
}
```

```
#box {
  background-color: hsl(0 100% 50%);
}
```

JavaScript

```
const hue = document.querySelector("#hue-slider");
const box = document.querySelector("#box");
hue.addEventListener("input", () => {
  box.style.backgroundColor = `hsl(${hue.value} 100% 50%)`;
  document.querySelector("#hue-value").textContent = `${hue.value}deg`;
});
```

Result

Approximating red hues in different color spaces

The following example shows a similar red color in different color spaces. The values in the `lch()` and `oklch()` functions are rounded for readability.

HTML

```
<div data-color="hsl-red">hsl()</div>
<div data-color="hwb-red">hwb()</div>
<div data-color="lch-red">lch()</div>
<div data-color="oklch-red">oklch()</div>
```

CSS

```
[data-color="hsl-red"] {
  /* hsl(<hue> <saturation> <lightness>) */
  background-color: hsl(0 100% 50%);
}
[data-color="hwb-red"] {
  /* hwb(<hue> <whiteness> <blackness>) */
  background-color: hwb(0 0% 0%);
}
[data-color="lch-red"] {
  /* lch(<lightness> <chroma> <hue>) */
  background-color: lch(50 150 40);
}
[data-color="oklch-red"] {
  /* oklch(<lightness> <chroma> <hue>) */
  background-color: oklch(0.6 0.4 20);
}
```

```
div {
  font-family: monospace;
  width: 100px;
  height: 100px;
  margin: 10px;
  border: 1px solid black;
  display: inline-block;
}
```

Result

Browser compatibility

See also

- `<color>`
- `<hue-interpolation-method>`

`<hue-interpolation-method>` CSS type

The **`<hue-interpolation-method>`** CSS data type represents the algorithm used for interpolation between `<hue>` values. The interpolation method specifies how to find a midpoint between two hue values based on a color wheel. It is used as a component of the `<color-interpolation-method>` data type.

When interpolating `<hue>` values, the hue interpolation algorithm defaults to `shorter`.

Syntax

A `<hue-interpolation-method>` value consists of the name of a hue interpolation algorithm followed by a literal token `hue`:

```
shorter hue
longer hue
increasing hue
decreasing hue
```

Values

Any pair of hue angles correspond to two radii on the color wheel, which cut the circumference into two possible arcs for interpolation. Both arcs start at the first radius and end at the second radius, but one goes clockwise and the other goes counterclockwise.

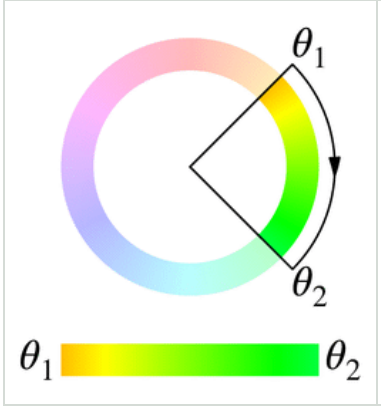
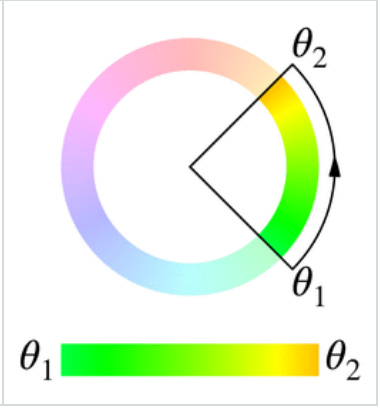
> **Note:** The following descriptions and illustrations are based on color wheels in which hue angles increase in a clockwise direction. Be aware that there are color wheels where an increase in angles will be a counterclockwise operation.

For a pair of hue angles `θ1` and `θ2` normalized to the range `[0deg, 360deg)`, there are four algorithms to determine which arc is used when interpolating from `θ1` to `θ2`:

`shorter`

Use the shorter arc. When the two radii coincide, the arc degenerates to a single point. When both arcs have the same lengths:

- If `θ1 < θ2`, use the clockwise arc;
- If `θ1 > θ2`, use the counterclockwise arc.

θ1 = 45deg, θ2 = 135deg	**θ1 = 135deg, θ2 = 45deg**
	

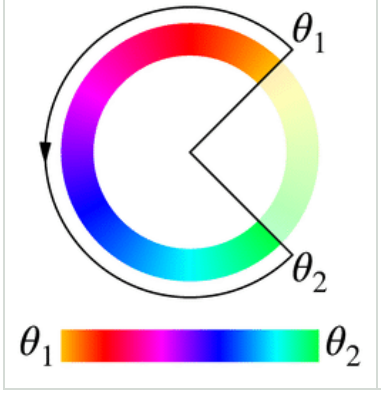
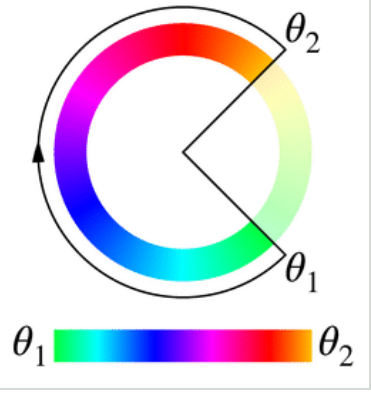
`longer`

Use the longer arc. When the two radii coincide:

- If `θ1 ≤ θ2`, the arc becomes the full circumference with a clockwise orientation.
- If `θ1 > θ2`, the arc becomes the full circumference with a counterclockwise orientation.

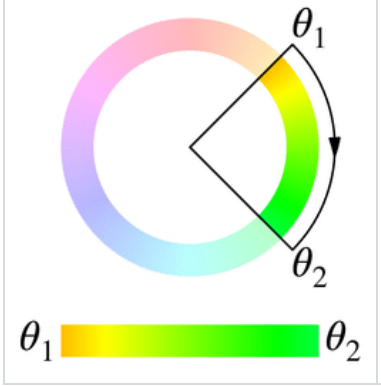
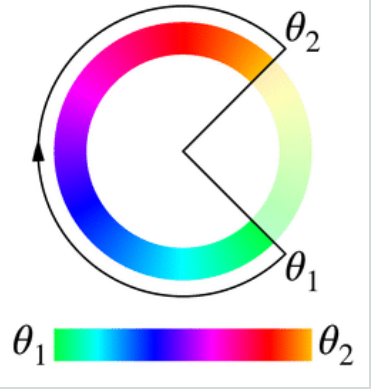
When both arcs have the same lengths:

- If `θ1 < θ2`, use the clockwise arc;
- If `θ1 > θ2`, use the counterclockwise arc.

θ1 = 45deg, θ2 = 135deg	θ1 = 135deg, θ2 = 45deg
	

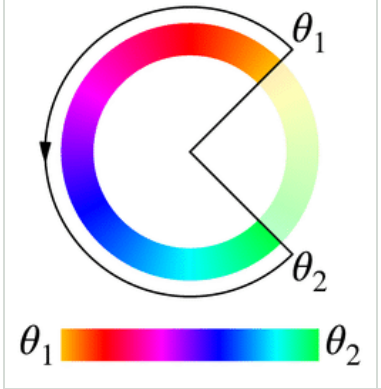
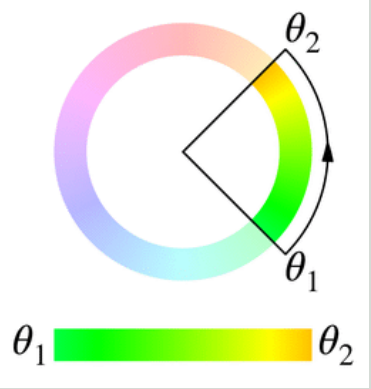
`increasing`

Use the clockwise arc. When the two radii coincide, the arc degenerates to a single point.

θ1 = 45deg, θ2 = 135deg	θ1 = 135deg, θ2 = 45deg
	

`decreasing`

Use the counterclockwise arc. When the two radii coincide, the arc degenerates to a single point.

θ1 = 45deg, θ2 = 135deg	θ1 = 135deg, θ2 = 45deg
	

As there are only two arcs to choose from, these algorithms are pairwise equivalent under certain circumstances. Specifically:

- If `0deg < θ2 - θ1 < 180deg` or `θ2 - θ1 < -180deg`, `shorter` and `increasing` are equivalent, whereas `longer` and `decreasing` are equivalent.
- If `-180deg < θ2 - θ1 < 0deg` or `θ2 - θ1 > 180deg`, `shorter` and `decreasing` are equivalent, whereas `longer` and `increasing` are equivalent.

A notable feature of `increasing` and `decreasing` is that when the hue angle difference passes through `180deg` during transition or animation, the arc will not flip to the other side like `shorter` and `longer` do.

Formal syntax

```
<hue-interpolation-method> =
  [ shorter | longer | increasing | decreasing ] hue
```

This syntax reflects the latest standard as per CSS Color Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Comparing hue interpolation algorithms

The following example shows the effect of using different hue interpolation algorithms in a `linear-gradient()`.

HTML

```
<div class="hsl">
  <p>HSL</p>
</div>
<div class="hsl-increasing">
```

```
  <p>HSL increasing</p>
</div>
<div class="hsl-decreasing">
  <p>HSL decreasing</p>
</div>
<div class="hsl-shorter">
  <p>HSL shorter</p>
</div>
<div class="hsl-longer">
  <p>HSL longer</p>
</div>
<div class="hsl-named">
  <p>HSL named</p>
</div>
<div class="hsl-named-longer">
  <p>HSL named (longer)</p>
</div>
```

CSS

```
div {
  border: 1px solid black;
  height: 50px;
  margin: 10px;
  width: 90%;
}
p {
  color: white;
  margin: 6px;
}

/* Fallback styles */
.hsl,
.hsl-shorter,
.hsl-named {
  background: linear-gradient(
    to right,
    hsl(39 100% 50%),
    hsl(46 100% 50%),
    hsl(53 100% 50%),
    hsl(60 100% 50%)
  );
}
.hsl-increasing {
  background: linear-gradient(
    to right,
    hsl(190 100% 50%),
    hsl(225 100% 50%),
    hsl(260 100% 50%),
    hsl(295 100% 50%),
    hsl(330 100% 50%),
    hsl(365 100% 50%),
    hsl(400 100% 50%),
    hsl(435 100% 50%),
    hsl(470 100% 50%),
    hsl(505 100% 50%),
    hsl(540 100% 50%)
  );
}
.hsl-decreasing,
.hsl-longer,
.hsl-named-longer {
  background: linear-gradient(
    to right,
    hsl(399 100% 50%),
    hsl(368 100% 50%),
    hsl(337 100% 50%),
    hsl(307 100% 50%),
    hsl(276 100% 50%),
    hsl(245 100% 50%),
    hsl(214 100% 50%),
    hsl(183 100% 50%),
    hsl(152 100% 50%),
    hsl(122 100% 50%),
    hsl(91 100% 50%),
    hsl(60 100% 50%)
  );
}
```

```
.hsl {
  background: linear-gradient(
    to right in hsl,
    hsl(39deg 100% 50%),
    hsl(60deg 100% 50%)
  );
}
.hsl-increasing {
  background: linear-gradient(
    to right in hsl increasing hue,
    hsl(190deg 100% 50%),
    hsl(180deg 100% 50%)
  );
}
.hsl-decreasing {
  background: linear-gradient(
    to right in hsl decreasing hue,
    hsl(39deg 100% 50%),
    hsl(60deg 100% 50%)
  );
}
.hsl-shorter {
  background: linear-gradient(
```

```
    to right in hsl shorter hue,
    hsl(39deg 100% 50%),
    hsl(60deg 100% 50%)
  );
}
.hsl-longer {
  background: linear-gradient(
    to right in hsl longer hue,
    hsl(39deg 100% 50%),
    hsl(60deg 100% 50%)
  );
}
.hsl-named {
  background: linear-gradient(to right in hsl, orange, yellow);
}
.hsl-named-longer {
  background: linear-gradient(to right in hsl longer hue, orange, yellow);
}
```

Result

See also

- `<color-interpolation-method>`
- `<hue>` data type

hypot() CSS function

The **`hypot()`** CSS function is an exponential function that returns the square root of the sum of squares of its parameters.

While `pow()` and `sqrt()` only work on unitless numbers, `hypot()` accepts values with units, but they all must have the same type.

Syntax

```
/* A <number> value */
width: hypot(2em); /* 2em */
width: hypot(3em, 4em); /* 5em */
width: hypot(30px, 40px); /* 50px */
width: hypot(48px, 64px); /* 80px */
width: hypot(3px, 4px, 5px); /* 7.0710678118654755px */
```

Parameters

The `hypot(x [, ...]#)` function accepts one or more comma-separated calculations as its parameters.

`x, x2, ..., xN`

A calculation that resolves to a `<number>`, `<dimension>`, or `<percentage>`.

Return value

Returns a `<number>`, `<dimension>`, or `<percentage>` (based on the inputs), which is the square root of the sum of squares of its parameters.

- If any of the inputs is `infinite`, the result is `+∞`.
- If a single parameter is provided, the result is the absolute value of its input. `hypot(2em)` and `hypot(-2em)` both resolve to `2em`.

Formal syntax

```
<hypot()> =
  hypot( <calc-sum># )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>       |
  <dimension>    |
  <percentage>   |
  <calc-keyword> |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Sizes based on hypot function

This example shows how you can use the `hypot()` function to calculate sizes.

HTML

```
<div class="boxes">
  <div class="box">100px</div>
  <div class="box one">100px</div>
  <div class="box two">141.42px</div>
  <div class="box three">250px</div>
</div>
```

CSS

Here we are using CSS custom properties to define the sizes to be used. First we declare the first size (`--size-0`) which is then used to calculate the other sizes.

- `--size-1` is calculated with the hypotenuse of `--size-0` (100px). This takes the square value and, as there is no other value, returns the square root of the value, which results in 100px.
- `--size-2` is calculated with the hypotenuse of `--size-0` (100px), twice. This takes the square of the value (100px * 100px = 10000px^2) and adds it to the square of `--size-0` again (10000px^2 + 10000px^2 = 20000px^2) and returns the square root of the sum (√(20000px^2)), which results in 141.42px.
- `--size-3` is calculated with the hypotenuse `--size-0` * 1.5 (150px) and `--size-0` * 2 (200px). The result is the square root of the sum of their squares: The values are squared (22500px^2 and 40000px^2) and added together (62500px^2), with the sum square-rooted (√(62500px^2)) being 250px.

```
:root {
  --size-0: 100px;
  --size-1: hypot(var(--size-0)); /*  100px */
  --size-2: hypot(var(--size-0), var(--size-0)); /*  141.42px */
  --size-3: hypot(
    calc(var(--size-0) * 1.5),
    calc(var(--size-0) * 2)
  ); /*  250px */
}
```

```
.boxes {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-around;
}

.box {
  width: var(--size-0);
  height: var(--size-0);
  background-color: tomato;
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

The sizes are then applied as the `width` and `height` values of the selectors.

```
.one {
  width: var(--size-1);
  height: var(--size-1);
}
.two {
  width: var(--size-2);
  height: var(--size-2);
}
.three {
  width: var(--size-3);
  height: var(--size-3);
}
```

Result

See also

- `pow()`
- `sqrt()`
- `log()`
- `exp()`

<ident> CSS type

The **`<ident>`** CSS data type denotes an arbitrary unquoted string of characters used as an identifier.

Syntax

A CSS identifier consists of one or more characters, which can be any of the following:

- any ASCII character in the ranges `A-Z` and `a-z`
- any decimal digit (`0` to `9`)
- a hyphen (`-`)
- an underscore (`_`)
- any other Unicode character `U+00A0` and higher (that is, any other non-ASCII Unicode character)
- an escaped character

Additionally, an identifier must not start with an unescaped digit, and must not start with an unescaped hyphen followed by an unescaped digit.

Note that `id1`, `Id1`, `iD1` and `ID1` are all different identifiers because they are case-sensitive. On the other hand, since there are several ways to escape a character, `toto\?` and `toto\3F` are the same identifiers.

Escaping characters

Escaping a character means representing it in a way that changes how it is interpreted by a software system. In CSS, you can escape a character by adding a backslash (`\`) in front of the character. Any character, except the hexadecimal digits `0-9`, `a-f`, and `A-F`, can be escaped in this way. For example, `&` can be escaped as `\&`.

You can also escape any character with a backslash followed by the character's Unicode code point represented by one to six hexadecimal digits. For example, `&` can be escaped as `\26`. In this usage, if the escaped character is followed by a hexadecimal digit, do one of the following:

- Place a space or other whitespace character after the Unicode code point.
- Provide the full six-digit Unicode code point of the character being escaped.

For example, the string `&123` can be escaped as `\26 123` (with a whitespace) or `\000026123` (with the six-digit Unicode code point for `&`) to ensure that `123` is not considered as part of the escape pattern.

Examples

Valid identifiers

```
nono79        /* A mix of alphanumeric characters and numbers */
ground-level  /* A mix of alphanumeric characters and a dash */
-test         /* A hyphen followed by alphanumeric characters */
--toto        /* A custom-property like identifier */
_internal     /* An underscore followed by alphanumeric characters */
```

```
\22 toto      /* An escaped character followed by alphanumeric characters */
\000022toto   /* Same as the previous example */
scooby\.doo   /* A correctly escaped period */
🔥123         /* A non-ASCII character followed by numbers */
```

Invalid identifiers

```
34rem     /* Must not start with a decimal digit */
-12rad    /* Must not start with a dash followed by a decimal digit */
scooby.doo  /* ASCII characters apart from alphanumerics must be escaped */
'scoobyDoo' /* Treated as a string */
"scoobyDoo" /* Treated as a string */
```

Browser compatibility

As this type is not a real type but a convenience type used to simplify the definition of other CSS syntax, there is no browser compatibility information as such.

See also

- <custom-ident>
- <dashed-ident>

if() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`if()`** CSS function allows different values to be set for a property depending on the result of a conditional test. The test can be based on a style query, a media query, or a feature query.

Syntax

```
/* Single <if-test> */
if(style(--scheme: dark): #eeeeee;)
if(media(print): black;)
if(media(width > 700px): 0 auto;)
if(supports(color: lch(7.1% 60.23 300.16)): lch(7.1% 60.23 300.16);)

/* <if-test> with else */
if(style(--size: "2xl"): 1em; else: 0.25em;)
if(media(print): white; else: black;)
if(media(width < 700px): 0 auto; else: 20px auto)
if(
  supports(color: lch(7.1% 60.23 300.16)): lch(7.1% 60.23 300.16);
  else: #03045e;
)
if(
  supports(color: lch(75% 0 0)): 3px solid lch(75% 0 0);
  else: 3px solid silver;
)

/* Multiple <if-test>s */
if(
  style(--scheme: ice): linear-gradient(#caf0f8, white, #caf0f8);
  style(--scheme: fire): linear-gradient(#ffc971, white, #ffc971);
  else: none;
)

/* <if-test> within a shorthand */
3px yellow if(
  style(--color: green): dashed;
  style(--color: yellow): inset;
  else: solid;
)
```

Parameters

The parameter is a semi-colon–separated list of `<if-branch>`es. Each `<if-branch>` is an `<if-condition>` followed by a colon and a `<value>`:

```
<if-branch> = <if-condition> : <value>;
```

`<if-condition>`

An `<if-test>` or the `else` keyword.

`<if-test>`

A style query, media query, or feature query.

`else`

A keyword representing an `<if-condition>` that always evaluates to true.

`<value>`

A property value.

Return value

A value or guaranteed-invalid.

Description

The CSS `if()` function provides conditional logic to CSS property values, and works in a similar fashion to JavaScript `if...else` statements.

The `if()` function can be used inside the value of any property, and can contain zero or more semi-colon-separated `<if-condition>`s. Each `<if-condition>` is either an `<if-test> : <value>` pair or an `else : <value>` pair. The semi-colon after the last `<if-condition>` is optional.

The return value is calculated as follows:

1. The `<if-condition>` expressions are evaluated, in the order they appear in the function.
2. The first `<if-condition>` that evaluates to `true` has its associated `<value>` returned.
3. If no `<if-condition>` evaluates to `true`, the function returns a <guaranteed-invalid>:. This behaves as invalid or `false` if the `if()` function is used in a value statement that has a fallback, such as a custom property or an `anchor()` function.

For example:

```
div {
  background-image: if(
    style(--scheme: ice): linear-gradient(#caf0f8, white, #caf0f8);
    style(--scheme: fire): linear-gradient(#ffc971, white, #ffc971);
    else: none;
  );
}
```

In this case, we are setting a different `linear-gradient()` as the `background-image` on `<div>` elements, depending on whether a `--scheme` custom property is set to `ice` or `fire`. If `--scheme` doesn't exist, or it exists and is set to any other value, the `else` value comes into play, and the `background-image` property is set to `none`.

> **Note:** Each condition must be separated from its associated value with a colon, and each `<if-condition> : <value>` pair must be separated with a semi-colon. The semi-colon is optional for the last `<if-condition> : <value>` pair.

> **Warning:** There must be no space between the `if` and the opening parenthesis (`(`). If there is, the whole declaration is invalid.

If a single `<if-condition>` or `<value>` is invalid, it does not invalidate the entire `if()` function; instead, the parser moves on to the next `<if-condition> : <value>` pair. If no `<if-condition>` nor `<value>` is valid, the function returns guaranteed-invalid.

Frequency and position of <code>else : <value></code> pairs

You can include multiple `else : <value>` pairs inside an `if()` function, in any position. However, in most cases, a single `else : <value>` pair at the end of the semi-colon-separated list is used to provide the default value that is always returned if none of the `<if-test>`s evaluate to true.

If you include an `else : <value>` pair before any `<if-test> : <value>` pairs, the conditions that follow it are not evaluated because `else` always evaluates to `true`. The following `if()` therefore always returns `none`, and the two `<if-test> : <value>` pairs are never evaluated:

```
div {
  background-image: if(
    else: none;
    style(--scheme: ice): linear-gradient(#caf0f8, white, #caf0f8);
    style(--scheme: fire): linear-gradient(#ffc971, white, #ffc971)
  );
}
```

Debugging a value that is not behaving as expected is one case where you might want to put an `else : <value>` in a position other than the end of the value list. In the following example, we are trying to work out whether the first `<if-test> : <value>` pair is working properly. If it isn't, the `else : <value>` pair returns a value of `url("debug.png")` to display an image indicating that the first `<if-test> : <value>` pair needs fixing. The last two `<if-test> : <value>` pairs are again never evaluated.

```
div {
  background-image: if(
    style(--scheme: ice): linear-gradient(#caf0f8, white, #caf0f8);
    else: url("debug.png");
    style(--scheme: fire): linear-gradient(#ffc971, white, #ffc971);
    else: none;
  );
}
```

Note that an `if()` function is still valid if it contains only an `else : <value>` pair, or nothing at all. The following property values are both valid:

```
background-color: if(else: yellow);
background-image: if();
```

These functions are not useful. They were included to demonstrate their validity. In this case, the `background-color` value will always be set to `yellow` and the `background-image` will be set to its initial value. You would be better off setting the `background-color` directly to `yellow`, and the `background-image` to `initial` or `none`.

Types of if-tests

An `<if-test>` accepts one of three query types. This section looks at each one in detail.

Style queries

A style query `<if-test>` allows you to test whether a particular property value is set on an element, and apply a value to a different property as a result. We walked through several style query examples earlier on; let's look at another example:

```
background-image: if(
  style(--scheme: ice): linear-gradient(#caf0f8, white, #caf0f8);
  else: none;
);
```

If the `--scheme` custom property is set to a value of `ice` on the same element, the provided `linear-gradient()` value is returned. If not, then `none` is returned.

Using style queries inside `if()` statements has an advantage over `@container` queries — you can target an element with styles directly, based on whether a custom property is set on it, rather than having to check set styles on a container parent element.

You can also use `and`, `or`, or `not` logic inside style queries. For example:

```
background-color: if(
  style((--scheme: dark) or (--scheme: very-dark)): black;
);

background-color: if(
  style((--scheme: dark) and (--contrast: hi)): black;
);

background-color: if(
```

```
  not style(--scheme: light): black;
);
```

A `@container` query does have some advantages — you can only set single property values at a time with `if()` style queries, whereas `@container` queries can be used to conditionally apply whole sets of rules. The two approaches are complementary, and have different uses.

Note that container style queries currently don't support regular CSS properties, just CSS custom properties. For example, the following won't work:

```
if(
  background-color: if(style(color: white): black;);
)
```

Media queries

A media query `<if-test>` can be used to set a value for a property depending on whether a media query test returns true.

You can use media types. For example, the following `<if-test> : <value>` pair returns a value of `white` on print media, while the `else` clause causes `#eeeeee` to be returned on non-print media.

```
background-color: if(
  media(print): white;
  else: #eeeeee;
)
```

You can also use media features — the following returns a value of `0 auto` if the current viewport width is less than `700px`, or `20px auto` if this is not the case:

```
margin: if(
  media(width < 700px): 0 auto;
  else: 20px auto;
)
```

This is really useful when you need to vary a single property value based on a media query result.

You can also use `and`, `or`, or `not` logic inside media queries. For example:

```
border-color: if(
  media((width > 700px) and (width < 1000px)): blue;
);

border-color: if(
  media((width < 500px) or (orientation: landscape)): blue;
);

background-color: if(
  not media(width < 500px): blue;
  else: red
);
```

When you want to set multiple declarations or rules based on one media query, a regular `@media` construct is needed. The two approaches are complementary, and have different uses.

Feature queries

A feature query `<if-test>` can be used to set a value for a property depending on whether the browser supports a particular property value.

For example, the following returns an `lch()` color if `lch()` colors are supported, or an `rgb()` color if not:

```
color: if(
  supports(color: lch(75% 0 0)): lch(75% 0 0);
  else: rgb(185 185 185);
)
```

Selector support queries also work. The following will return a value of `1em` if the browser supports the `:buffering` pseudo-class, or `initial` if not:

```
margin-top: if(
  supports(selector(:buffering)): 1em;
  else: initial;
)
```

You can also use `and`, `or`, or `not` logic inside feature queries. For example:

```
margin-top: if(
  supports((selector(:buffering)) and (color: blue)): 1em;
);

margin-top: if(
  supports((selector(:buffering)) or (color: not-a-color)): 1em;
);

margin-top: if(
  supports(not selector(:buffering)): 1em;
);
```

Feature queries are really useful inside `if()` statements when you need to vary a single property value based on support for a particular value or a separate property. When you want to set multiple declarations or rules based on one feature query, a regular `@supports` construct is better. The two approaches are complementary, and have different uses.

Providing fallback values

The `if()` statement does not gracefully degrade; an explicit fallback needs to be provided for non-supporting browsers.

For example, in this case we provide a static `padding` value for browsers that don't support `if()`. Browsers that support `if()` will override the first declaration with the second one, which sets different padding values depending on whether or not the `--size: "2xl"` custom property is set.

```
padding: 1em;
padding: if(style(--size: "2xl"): 1em; else: 0.25em);
```

> **Note:** Remember to include the `else` condition. In `if()`-supporting browsers, if no `else` value were included and `--size` was not equal to `"2xl"`, the padding would be set to `initial`.

Whole and partial values

An `if()` function can be set as the value of any CSS property, but it can also be used to determine parts of property values. For example, the following sets a different `border-color` inside a `border` shorthand property, depending on whether `lch()` colors are supported:

```
border: if(
  supports(color: lch(75% 0 0)): 3px solid lch(75% 0 0);
  else: 3px solid silver;
);
```

However, we could use the `if()` function to determine the `border-color` component only:

```
border: 3px solid
  if(
    supports(color: lch(75% 0 0)): lch(75% 0 0); else: silver;
  );
```

Nesting if() functions

Because an `if()` function can take the place of whole property values or individual components, it is possible to nest `if()` functions within other `if()` functions, and inside other functions such as `calc()`.

For example, in this declaration we are using `if()` to set a `color` property value depending on various conditions. We have an outer `if()` function that returns a particular value depending on whether the `--scheme` custom property is set to `ice` or `fire` (with an `else` value of `black` that is returned if neither of the conditions return true).

However, the two `<value>`s are also `if()` functions. These inner `if()` functions return a light color value if the user prefers a dark color scheme (determined using the `prefers-color-scheme` media query), and a dark color value otherwise.

```
color: if(
  style(--scheme: ice):
    if(
      media(prefers-color-scheme: dark): #caf0f8;
      else: #03045e;
    );
  style(--scheme: fire):
    if(
      media(prefers-color-scheme: dark): #ffc971;
      else: #621708;
    );
  else: black
);
```

In the next example, we set the `width` property equal to a `calc()` function that subtracts `50px` from a percentage of the parent element's width. The percentage is represented by an `if()` function, which test whether the `--scheme: wide` custom property is set. If so, the percentage is `70%`, so the outer function resolves to `calc(70% - 50px)`. If not, the percentage is `50%`, so the outer function resolves to `calc(50% - 50px)`.

```
width: calc(if(
    style(--scheme: wide): 70%;
    else: 50%;
  ) - 50px);
```

Formal syntax

```
<if()> =
  if( [ <if-branch> ; ]* <if-branch> ;? )

<if-branch> =
  <if-condition> : <declaration-value>?

<if-condition> =
  <if-test>]>  |
  else

<if-test> =
  supports( [ <ident> : <declaration-value> ]  |
  <supports-condition> )                       |
  media( <media-feature>                       |
  <media-condition> )                          |
  style( <style-query> )

<supports-condition> =
  not <supports-in-parens>                           |
  <supports-in-parens> [ and <supports-in-parens> ]*  |
  <supports-in-parens> [ or <supports-in-parens> ]*

<media-feature> =
  <mf-plain>    |
  <mf-boolean>  |
  <mf-range>

<media-condition> =
  <media-not>                                  |
  <media-in-parens> [ <media-and>* | <media-or>* ]

<style-query> =
  not <style-in-parens>                              |
  <style-in-parens> [ [ and <style-in-parens> ]* | [ or <style-in-parens> ]* ]  |
  <style-feature>

<supports-in-parens> =
  ( <supports-condition> )  |
  <supports-feature>        |
  <general-enclosed>

<mf-plain> =
  <mf-name> : <mf-value>

<mf-boolean> =
  <mf-name>

<mf-range> =
  <mf-name> <mf-comparison> <mf-value>             |
  <mf-value> <mf-comparison> <mf-name>             |
  <mf-value> <mf-lt> <mf-name> <mf-lt> <mf-value>  |
```

```
  <mf-value> <mf-gt> <mf-name> <mf-gt> <mf-value>

<media-not> =
  not <media-in-parens>

<media-in-parens> =
  ( <media-condition> )  |
  ( <media-feature> )    |
  <general-enclosed>

<media-and> =
  and <media-in-parens>

<media-or> =
  or <media-in-parens>

<style-in-parens> =
  ( <style-query> )    |
  ( <style-feature> )  |
  <general-enclosed>

<style-feature> =
  <style-feature-plain>    |
  <style-feature-boolean>  |
  <style-range>

<supports-feature> =
  <supports-selector-fn>       |
  <supports-font-tech-fn>      |
  <supports-font-format-fn>    |
  <supports-at-rule-fn>        |
  <supports-named-feature-fn>  |
  <supports-env-fn>            |
  <supports-decl>

<general-enclosed> =
  [ <function-token> <any-value>? ) ]  |
  [ ( <any-value>? ) ]

<mf-name> =
  <ident>

<mf-value> =
  <number>     |
  <dimension>  |
  <ident>      |
  <ratio>

<mf-comparison> =
  <mf-lt>  |
  <mf-gt>  |
  <mf-eq>

<mf-lt> =
  '<' '='?

<mf-gt> =
  '>' '='?

<style-feature-plain> =
  <style-feature-name> : <style-feature-value>

<style-feature-boolean> =
  <style-feature-name>

<style-range> =
  <style-range-value> <mf-comparison> <style-range-value>  |
  <style-range-value> <mf-lt> <style-range-value> <mf-lt> <style-range-value>  |
  <style-range-value> <mf-gt> <style-range-value> <mf-gt> <style-range-value>

<supports-selector-fn> =
  selector( <complex-selector> )

<supports-font-tech-fn> =
  font-tech( <font-tech> )

<supports-font-format-fn> =
  font-format( <font-format> )

<supports-at-rule-fn> =
  at-rule( <at-keyword-token> )

<supports-named-feature-fn> =
  named-feature( <ident> )

<supports-env-fn> =
  env( <ident> )

<supports-decl> =
  ( [ <declaration> | <supports-condition-name> ] )

<ratio> =
  <number [0,∞]> [ / <number [0,∞]> ]?

<mf-eq> =
  '='

<style-range-value> =
  <custom-property-name>  |
  <style-feature-value>

<complex-selector> =
  <complex-selector-unit> [ <combinator>? <complex-selector-unit> ]*

<font-tech> =
  <font-features-tech>  |
  <color-font-tech>     |
  variations            |
  palettes              |
```

```
  incremental

<font-format> =
  <string>           |
  collection         |
  embedded-opentype  |
  opentype           |
  svg                |
  truetype           |
  woff               |
  woff2

<complex-selector-unit> =
  [ <compound-selector>? <pseudo-compound-selector>* ]!

<combinator> =
  '>'                |
  '+'                |
  '~'                |
  [ '|' '|' ]        |
  [ / <wq-name> / ]

<font-features-tech> =
  features-opentype  |
  features-aat       |
  features-graphite

<color-font-tech> =
  color-COLRv0  |
  color-COLRv1  |
  color-SVG     |
  color-sbix    |
  color-CBDT

<compound-selector> =
  [ <type-selector>? <subclass-selector>* ]!

<pseudo-compound-selector> =
  <pseudo-element-selector> <pseudo-class-selector>*

<wq-name> =
  <ns-prefix>? <ident-token>

<type-selector> =
  <wq-name>         |
  <ns-prefix>? '*'

<subclass-selector> =
  <id-selector>            |
  <class-selector>         |
  <attribute-selector>     |
  <pseudo-class-selector>

<pseudo-element-selector> =
  : <pseudo-class-selector>         |
  <legacy-pseudo-element-selector>

<pseudo-class-selector> =
  : <ident-token>                   |
  : <function-token> <any-value> )

<ns-prefix> =
  [ <ident-token> | '*' ]? '|'

<id-selector> =
  <hash-token>

<class-selector> =
  '.' <ident-token>

<attribute-selector> =
  '[' <wq-name> ']'                                  |
  '[' <wq-name> <attr-matcher> [ <string-token> | <ident-token> ] <attr-modifier>? ']'

<legacy-pseudo-element-selector> =
  : [ before | after | first-line | first-letter ]

<attr-matcher> =
  [ '~' | '|' | '^' | '$' | '*' ]? '='

<attr-modifier> =
  i  |
  s
```

This syntax reflects the latest standard as per CSS Conditional Rules Module Level 3, CSS Conditional Rules Module Level 4, CSS Conditional Rules Module Level 5, CSS Fonts Module Level 5, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Media Queries Level 5, Selectors Level 4, Selectors Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>if()</code> usage

In this example, we'll show basic usage of each of the three types of `<if-test>`.

HTML

Our HTML features a `<section>` element with two `<article>` elements inside it, containing `<h2>` headings. The `<section>` has a custom property set on it inside its `style` attribute — `--show-apple:true` — which we use later on to conditionally set a property value.

```
<section style="--show-apple:true">
  <article><h2>First article</h2></article>
  <article><h2>Second article</h2></article>
</section>
```

CSS

In our CSS, we first target the `<section>` element, laying it out with flexbox and setting a `gap` between the two child `<article>` elements. We then use an `if()` function with an `orientation` media query `<if-test>` to set the value of the `flex-direction` property to `row` if the document is in landscape orientation, or `column` if it is in portrait orientation. This lays out the `article` elements side-by-side in wide screens, and top-to-bottom in narrow screens.

```
html {
  height: 100%;
  font-family: sans-serif;
}

body,
section {
  height: inherit;
}

h2 {
  text-align: center;
}

article {
  background-color: cyan;
  border: 3px solid gray;
  flex: 1;
}
```

```
section {
  display: flex;
  gap: 16px;
  flex-direction: if(
    media(orientation: landscape): row;
    else: column;
  )
}
```

Next, we target the `<h2>` element's `::before` pseudo-element, setting its `content` property to an apple emoji, but only if `--show-apple: true` is set (we did this earlier with an inline `<style>` in our HTML). We achieve this using an `if()` function with a style query `<if-test>`:

```
h2::before {
  content: if(
    style(--show-apple: true): "🍎 ";
  );
}
```

Finally, we target the `<h2>` element itself. We use a feature query `<if-test>` to test whether the browser supports `lch()` colors, and set the `color` property to an `lch()` color if so, or a hex equivalent if not.

```
h2 {
  color: if(
    supports(color: lch(29.57% 43.25 344.44)): lch(29.57% 43.25 344.44);
    else: #792359;
  )
}
```

Result

Note how the styling is applied. Test out the conditional styling for the first two `if()` queries by modifying the rendered demo using your browser's devtools:

- Remove the `<section>` element's `style` attribute and note how the apple emojis are no longer rendered.
- Change the `height` attribute of the embedding `<iframe>` to `1200px`. This will change the orientation from landscape to portrait. Note how the layout changes as a result.

Controlling a color scheme with <code>if()</code>

This demo shows how you can start to have some real fun with CSS `if()` functions. Among other things, we use `if()` functions to conditionally set the values of some custom properties, enabling us to control the entire color scheme!

HTML

Our HTML contains an `<article>` element with some content inside it — a top-level heading, a couple of `<p>` elements, and an `<aside>`. We also include a `<form>` containing a `<select>` drop-down that enables selecting a color scheme.

```
<article>
  <h1>Main heading</h1>
  <p>
    Lorem ipsum dolor sit amet consectetur adipiscing elit.
    Quisque faucibus ex sapien vitae pellentesque sem placerat.
    In id cursus mi pretium tellus duis convallis.
  </p>
  <aside>
    <h2>An aside</h2>
    <p>
      Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus
      fringilla lacus nec metus bibendum egestas.
    </p>
  </aside>
  <p>
    Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut
    hendrerit semper vel class aptent taciti sociosqu. Ad litora
    torquent per conubia nostra inceptos himenaeos.
  </p>
</article>
<form>
  <label for="scheme">Choose color scheme:</label>
  <select id="scheme">
    <option value="">Default</option>
    <option value="ice">Ice</option>
    <option value="fire">Fire</option>
  </select>
</form>
```

JavaScript

Our JavaScript adds a `change` event listener to the `<select>` element. When a new value is selected, our script sets the `<article>` element's `class` attribute to equal that value.

```
const articleElem = document.querySelector("article");
const selectElem = document.querySelector("select");

selectElem.addEventListener("change", () => {
  articleElem.className = selectElem.value;
});
```

CSS

In our CSS, we give the `<body>` element a `max-width` of `700px` and center it using `auto` `margin` values. However, we use an `if()` function with a media query `<if-test>` to set the `margin-top` component inside the `margin` shorthand to `0` if the viewport's width is less than `700px`, and `20px` if it is wider. This means that on wide screens we get a bit of margin at the top of the content, but this is removed on narrow screens, where it looks a bit weird.

```
* {
  box-sizing: border-box;
}

html {
  font-family: "Helvetica", "Arial", sans-serif;
}

p {
  line-height: 1.5;
}

form {
  padding-left: 20px;
  margin-top: 20px;
}

article h2 {
  margin: 0;
  font-size: 1.8rem;
}
```

```
body {
  max-width: 700px;
  margin: if(
    media(width < 700px): 0;
    else: 20px;
  ) auto 0;
}
```

We then set the `--scheme` custom property to match the `<article>` element's `class` name. The class will be set by our JavaScript when a new value is selected in our `<select>` element. You'll see the significance of the custom element value in the next CSS block.

```
.ice {
  --scheme: ice;
}

.fire {
  --scheme: fire;
}
```

We can see the real power of CSS `if()` functions when we combine them with custom properties. Here we use `if()` functions to set our `--color1` and `--color2` custom properties to different color values depending on the value of the `--scheme` custom property. We then use the `--color1` and `--color2` values in our `<article>` element's `color`, `border`, and `background-image` properties, and our `<aside>` element's `color` and `background-color` properties.

We are controlling our entire color scheme via custom properties, with different values set via `if()` functions.

```
article {
  padding: 20px;
  --color1: if(
    style(--scheme: ice): #03045e;
    style(--scheme: fire): #621708;
    else: black;
  );
  --color2: if(
    style(--scheme: ice): #caf0f8;
    style(--scheme: fire): #ffc971;
    else: white;
  );

  color: var(--color1);
  border: 3px solid var(--color1);
  background-image: linear-gradient(
    to left,
    var(--color2),
    white,
    var(--color2)
  );
}

aside {
  color: var(--color2);
  background-color: var(--color1);
  padding: 20px;
}
```

Finally, we use `if()` functions in a couple more places:

- We set our `<h1>` element's `font-size` to be `calc(3rem + 2vw)` if the viewport is wider than `700px`, and `3rem` otherwise. This means the font size updates dynamically with viewport width changes on wider screens, but stays the same on narrow screens.
- We set a suitable emoji as the `content` of our `<h1>` element's `::before` pseudo-class, depending on the value of the `--scheme` custom property.

```
h1 {
  margin: 0;
  font-size: if(
    media(width > 700px): calc(3rem + 2vw);
    else: 3rem;
  );
}

h1::before {
  content: if(
    style(--scheme: ice): "❄️ ";
    style(--scheme: fire): "🔥 ";
    else: "";
  );
}
```

Result

This demo renders as follows:

Try selecting different color scheme values to see the effect on the look and feel.

See also

- Container style queries
- Media queries
- Feature queries
- CSS values and units module

<image> CSS type

The **`<image>`** CSS data type represents a two-dimensional image.

Syntax

The `<image>` data type can be represented with any of the following:

- An image denoted by the `<url>` data type
- A `<gradient>` data type
- A part of the webpage, defined by the `element()` function
- An image, image fragment or solid patch of color, defined by the `image()` function
- A blending of two or more images defined by the `cross-fade()` function.
- A selection of images chosen based on resolution defined by the `image-set()` function.
- Generated by a paint worklet with the `paint()` function.

Description

CSS can handle the following kinds of images:

- Images with *intrinsic dimensions* (a natural size), like a JPEG, PNG, or other raster format.
- Images with *multiple intrinsic dimensions*, existing in multiple versions inside a single file, like some .ico formats. (In this case, the intrinsic dimensions will be those of the image largest in area and the aspect ratio most similar to the containing box.)
- Images with no intrinsic dimensions but with *an intrinsic aspect ratio* between its width and height, like an SVG or other vector format.
- Images with *neither intrinsic dimensions, nor an intrinsic aspect ratio*, like a CSS gradient.

Concrete size

CSS determines an object's *concrete size* using (1) its *intrinsic dimensions*; (2) its *specified size*, defined by CSS properties like `width`, `height`, or `background-size`; and (3) its *default size*, determined by the kind of property the image is used with:

Kind of Object (CSS Property)	Default object size
`background-image`	The size of the element's background positioning area
`list-style-image`	The size of a `1em` character
`border-image-source`	The size of the element's border image area
`cursor`	The browser-defined size matching the usual cursor size on the client's system
`mask-image`	?
`shape-outside`	?
`mask-border-source`	?
`symbols()` for @counter-style	At risk feature. If supported, the browser-defined size matching the usual cursor size on the client's system
`content` for a pseudo-element (`::after`/`::before`)	A 300px × 150px rectangle

The concrete object size is calculated using the following algorithm:

- If the specified size defines *both the width and the height*, these values are used as the concrete object size.
- If the specified size defines *only the width or only the height*, the missing value is determined using the intrinsic ratio, if there is any, the intrinsic dimensions if the specified value matches, or the default object size for that missing value.
- If the specified size defines *neither the width nor the height*, the concrete object size is calculated so that it matches the intrinsic aspect ratio of the image but without exceeding the default object size in any dimension. If the image has no intrinsic aspect ratio, the intrinsic aspect ratio of the object it applies to is used; if this object has none, the missing width or height are taken from the default object size.

> **Note:** Not all browsers support every type of image on every property. See the browser compatibility section for details.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Formal syntax

```
<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Valid images

```
url("test.jpg")               /* A <url>, as long as test.jpg is an actual image */
linear-gradient(blue, red)  /* A <gradient> */
element(#real-id)            /* A part of the webpage, referenced with the element() function,
                                if "real-id" is an existing ID on the page */
image(ltr "arrow.png#xywh=0,0,16,16", red)
                            /* A section 16x16 section of <url>, starting from the top, left of the original
                               image as long as arrow.png is a supported image, otherwise a solid
                               red swatch. If language is rtl, the image will be horizontally flipped. */
cross-fade(20% url("twenty.png"), url("eighty.png"))
                            /* cross faded images, with twenty being 20% opaque
                               and eighty being 80% opaque. */
image-set("test.jpg' 1x, 'test-2x.jpg" 2x)
                            /* a selection of images with varying resolutions */
```

Invalid images

```
"no-url.jpg"           /* An image file must be defined using the url() function. */
url("report.pdf")      /* A file pointed to by the url() function must be an image. */
element(#fakeid)     /* An element ID must be an existing ID on the page. */
image(z.jpg#xy=0,0)  /* The spatial fragment must be written in the format of xywh=#,#,#,# */
image-set("cat.jpg" 1x, "dog.jpg" 1x) /* every image in an image set must have a different resolution */
```

See also

- `<gradient>`
- `element()`
- `image()`
- `image-set()`
- `cross-fade()`

image() CSS function

The **`image()`** CSS function defines an `<image>` in a similar fashion to the `url()` function, but with added functionality including specifying the image's directionality, displaying just a part of that image defined by a media fragment, and specifying a solid color as a fallback in case none of the specified images are able to be rendered.

> **Note:** The CSS `image()` function should not be confused with `Image()`, the `HTMLImageElement` constructor.

Syntax

```
/* Basic usage */
image("image1.jpg");
image(url("image2.jpg"));

/* Bidi-sensitive Images */
image(ltr "image1.jpg");
image(rtl "image1.jpg");

/* Image Fallbacks */
image("image1.jpg", black);

/* Image Fragments */
image("image1.jpg#xywh=40,0,20,20");
```

```
/* Solid-color Images */
image(rgb(0 0 255 / 0.5)), url("bg-image.png");
```

Values

`image-tags` Optional

The directionality of the image, either `ltr` for left-to-right or `rtl` for right-to-left.

`image-src` Optional

Zero or more `<url>`s or `<string>`s specifying the image sources, with optional image fragment identifiers.

`color` Optional

A color, specifying a solid background color to use as a fallback if no `image-src` is found, supported, or declared.

Bi-directional awareness

The first, optional parameter of the `image()` notation is the directionality of the image. If included, and the image is used on an element with opposite directionality, the image will be flipped horizontally in horizontal writing modes. If the directionality is omitted, the image won't be flipped if the language direction is changed.

Image fragments

One key difference between `url()` and `image()` is the ability to add a media fragment identifier — a starting point along the x and y axis, along with a width and height — onto the image source to display only a section of the source image. The section of the image defined in the parameter becomes a standalone image. The syntax looks like so:

```
background-image: image("my-image.webp#xywh=0,20,40,60");
```

The background image of the element will be the portion of the image *myImage.webp* that starts at the coordinate 0px, 20px (the top left-hand corner) and is 40px wide and 60px tall.

The `#xywh=#,#,#,#` media fragment syntax takes four comma separated numeric values. The first two represent the X and Y coordinates for the starting point of the box that will be created. The third value is the width of the box, and the last value is the height. By default, these are pixel values. The spatial dimension definition in the media specification indicates that percentages will be supported as well:

```
xywh=160,120,320,240        /* results in a 320x240 image at x=160 and y=120 */
xywh=pixel:160,120,320,240  /* results in a 320x240 image at x=160 and y=120 */
xywh=percent:25,25,50,50    /* results in a 50%x50% image at x=25% and y=25% */
```

The image fragments can be used in `url()` notation as well. The `#xywh=#,#,#,#` media fragment syntax is 'backwards compatible' in that a media fragment will be ignored if not understood, and won't break the source call when used with `url()`. If the browser doesn't understand the media fragments notation, it ignores the fragment, displaying the entire image.

Browsers that understand `image()` also understand the fragment notation. Therefore, if the fragment is not understood within `image()`, the image will be considered invalid.

Color fallback

If a color is specified in `image()` along with your image sources, it acts as a fallback if the images are invalid and do not appear. In such cases, the `image()` function renders as if no image were included, generating a solid-color image. As a use case, consider a dark image being used as a background for some white text. A dark background color may be needed for foreground text to be legible, if the image does not render.

Omitting image sources while including a color is valid and creates a color swatch. Unlike declaring a `background-color`, which is placed under or behind all the background images, this can be used to put (generally semi-transparent) colors over other images.

The size of the color swatch can be set with the `background-size` property. This is different from the `background-color`, which sets a color to cover the entire element. Both `image(color)` and `background-color` placements are impacted by the `background-clip` and `background-origin` properties.

Formal syntax

```
<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>
```

This syntax reflects the latest standard as per CSS Images Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

This feature can help improve accessibility by providing a fallback color when an image fails to load. While this can and should be done by including a background-color on every background image, the CSS `image()` function allows adding allows only including background colors should an image fail to load, which means you can add a fallback color should a transparent PNG/GIF/WebP not load.

Examples

Directionally-sensitive images

```
<ul>
  <li dir="ltr">Bullet is a right facing arrow on the left</li>
  <li dir="rtl">Bullet is the same arrow, flipped to point left.</li>
</ul>
```

```
ul {
  list-style-image: image(ltr "rightarrow.png");
}
```

In the left-to-right list items — those with `dir="ltr"` set on the element itself or inheriting the directionality from an ancestor or default value for the page — the image will be used as-is. List items with `dir="rtl"` set on the `<li>` or inheriting the right-to-left directionality from an ancestor, such as documents set to Arabic or Hebrew, will have the bullet display on the right, horizontally flipped, as if `transform: scaleX(-1)` had been set. The text will also be displayed left-to-right.

Displaying a section of the background image

```
<div class="box">Hover over me. What cursor do you see?</div>
```

```
.box:hover {
  cursor: image("sprite.png#xywh=32,64,16,16"), auto;
}
```

When the user hovers over the box, the cursor will change to display the 16x16 px section of the sprite image, starting at x=32 and y=64.

Putting color on top of a background image

```
.quarter-logo {
  height: 200px;
  width: 200px;
  border: 1px solid;
}
```

```
.quarter-logo {
  background-image: image(rgb(0 0 0 / 25%)), url("firefox.png");
  background-size: 25%;
  background-repeat: no-repeat;
}
```

```
<div class="quarter-logo">
  If supported, a quarter of this div has a darkened logo
</div>
```

The above will put a semi-transparent black mask over the Firefox logo background image. Had we used the `background-color` property instead, the color would have appeared behind the logo image instead of on top of it. Additionally, the entire container would have had the same background color. Because we used `image()` along with the `background-size` property (and prevented the image from repeating with the `background-repeat` property), the color swatch will only cover a quarter of the container.

Browser compatibility

Currently, no browsers support this feature.

See also

- `<image>`
- `element()`
- `<url>`
- `clip-path`
- `<gradient>`
- `image-set()`
- `cross-fade()`
- CSS images module

image-set() CSS function

The **`image-set()`** CSS functional notation is a method of letting the browser pick the most appropriate CSS image from a given set, primarily for high pixel density screens.

Resolution and bandwidth differ by device and network access. The `image-set()` function delivers the most appropriate image resolution for a user's device, providing a set of image options — each with an associated resolution declaration — from which the browser picks the most appropriate for the device and settings. Resolution can be used as a proxy for filesize — a user agent on a slow mobile connection with a high-resolution screen may prefer to receive lower-resolution images rather than waiting for a higher resolution image to load.

`image-set()` allows the author to provide options rather than determining what each individual user needs.

Syntax

```
/* Select image based on resolution */
image-set(
  "image1.jpg" 1x,
  "image2.jpg" 2x
);

image-set(
  url("image1.jpg") 1x,
  url("image2.jpg") 2x
);

/* Select gradient based on resolution */
image-set(
  linear-gradient(blue, white) 1x,
  linear-gradient(blue, green) 2x
);

/* Select image based on supported formats */
image-set(
  url("image1.avif") type("image/avif"),
  url("image2.jpg") type("image/jpeg")
);
```

Values

`<image>`

The `<image>` can be any image type except for an image set. The `image-set()` function may not be nested inside another `image-set()` function.

`<string>`

A URL to an image.

`<resolution>` Optional

`<resolution>` units include `x` or `dppx`, for dots per pixel unit, `dpi`, for dots per inch, and `dpcm` for dots per centimeter. Every image within an `image-set()` must have a unique resolution.

`type(<string>)` Optional

A valid MIME type string, for example "image/jpeg".

Formal syntax

```
<image-set()> =
  image-set( <image-set-option># )

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Browsers do not provide any special information on background images to assistive technology. This is important primarily for screen readers, as a screen reader will not announce its presence and therefore convey nothing to its users. If the image contains information critical to understanding the page's overall purpose, it is better to describe it semantically in the document.

- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

Examples

Using image-set() to provide alternative background-image options

This example shows how to use `image-set()` to provide two alternative `background-image` options, chosen depending on the resolution needed: a normal version and a high-resolution version.

```
<div class="box"></div>
```

```
.box {
  width: 400px;
  height: 200px;
  background-repeat: no-repeat;
  background-size: cover;

  background-image: image-set(
    url("https://mdn.github.io/shared-assets/images/examples/balloons-small.jpg")
      1x,
    url("https://mdn.github.io/shared-assets/images/examples/balloons-landscape.jpg")
      2x
  );
}
```

Using image-set() to provide alternative image formats

In the next example the `type()` function is used to serve the image in AVIF and JPEG formats. If the browser supports avif, it will choose that version. Otherwise it will use the jpeg version.

```
<div class="box"></div>
```

```
.box {
  width: 400px;
  height: 200px;
  background-repeat: no-repeat;
  background-size: cover;

  background-image: image-set(
    "https://mdn.github.io/shared-assets/images/examples/balloons-landscape.avif"
      type("image/avif"),
    "https://mdn.github.io/shared-assets/images/examples/balloons-landscape.jpg"
      type("image/jpeg")
  );
}
```

See also

- `<image>`
- `image()`
- `element()`
- `<url>`

- `<gradient>`
- `cross-fade()`

paint() CSS function

The **`paint()`** CSS function defines an `<image>` value generated with a PaintWorklet.

Syntax

```
paint(workletName, ...parameters)
```

where:

workletName

The name of the registered worklet.

parameters Optional

Optional additional parameters to pass to the paintWorklet

Formal syntax

```
<paint()> =
  paint( <ident> , <declaration-value>? )
```

This syntax reflects the latest standard as per CSS Painting API Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic CSS paint() usage

Given the following HTML:

```
<ul>
  <li>item 1</li>
  <li>item 2</li>
  <li>item 3</li>
  <li>item 4</li>
  <li>item 5</li>
  <li>item 6</li>
  <li>item 7</li>
  <li>item 8</li>
  <li>item 9</li>
  <li>item 10</li>
  <li>item N</li>
</ul>
```

In JavaScript, we register the paint worklet:

```
CSS.paintWorklet.addModule(
  "https://mdn.github.io/houdini-examples/cssPaint/intro/worklets/boxbg.js",
);
```

In the CSS, we define the `background-image` as a `paint()` type with the worklet name, `boxbg`, along with any variables (ex. `--box-color` and `--width-subtractor`) the worklet will use:

```
body {
  font: 1.2em / 1.2 sans-serif;
}
li {
  background-image: paint(boxbg);
  --box-color: hsl(55 90% 60%);
}

li:nth-of-type(3n) {
  --box-color: hsl(155 90% 60%);
  --width-subtractor: 20;
}

li:nth-of-type(3n + 1) {
  --box-color: hsl(255 90% 60%);
  --width-subtractor: 40;
}
```

CSS paint() with parameters

You can pass optional arguments in the CSS `paint()` function. In this example, we passed two arguments that control whether the `background-image` on a group of list items is `filled` or has a `stroke` outline, and the `width` of that outline:

```
<ul>
  <li>item 1</li>
  <li>item 2</li>
  <li>item 3</li>
  <li>item 4</li>
  <li>item 5</li>
  <li>item 6</li>
  <li>item 7</li>
  <li>item 8</li>
  <li>item 9</li>
  <li>item 10</li>
  <li>item N</li>
</ul>
```

```
CSS.paintWorklet.addModule(
  "https://mdn.github.io/houdini-examples/cssPaint/intro/worklets/hilite.js",
);
```

```
body {
  font: 1.2em / 1.2 sans-serif;
```

```
}

li {
  --box-color: hsl(55 90% 60% / 100%);
  background-image: paint(hollow-highlights, stroke, 2px);
}

li:nth-of-type(3n) {
  --box-color: hsl(155 90% 60% / 100%);
  background-image: paint(hollow-highlights, filled, 3px);
}

li:nth-of-type(3n + 1) {
  --box-color: hsl(255 90% 60% / 100%);
  background-image: paint(hollow-highlights, stroke, 1px);
}
```

We've included a custom property in the selector block defining a boxColor. Custom properties are accessible to the PaintWorklet.

See also

- `PaintWorkletGlobalScope`
- CSS Painting API
- Using the CSS Painting API
- `<image>`
- Canvas API

!important CSS keyword

A `!` delimiter followed by the `important` keyword marks the declaration as important. The `!important` flag alters the rules selecting declarations inside the cascade. A declaration that is not *important* is called *normal*.

To mark a declaration important, add the *important flag* (`!important`) after the value in the declaration. While white space is allowed between the delimiter and the keyword, the flag is generally written as `!important` without any white space.

```
selector {
  property: value; /* normal declaration */
  property: value !important; /* important declaration (preferred) */
  property: value ! important; /* important declaration (not preferred) */
}
```

The `!important` comes after the value of the property value pair declaration, preceded by zero or more spaces. The important flag must be the last token in the declaration. In other words, there can be white space and comments between the flag and the declaration's ending semicolon, but nothing else.

Impact on the cascade

When it comes to important declarations, the cascade origin and layer orders are reversed. Without the important flag, declarations in the author's style sheets override declarations in a user's style sheet, which override declarations in the user-agent's default style sheet.

When a declaration is important, the order of precedence is reversed. Declarations marked as important in the user-agent style sheets override all important declarations in the user style sheets. Similarly, all important declarations in the user style sheets override all important declarations in the author's style sheets. Finally, all important declarations take precedence over all animations.

> **Note:** All important declarations take precedence over all animations. `!important` is not valid within @keyframes animation declarations.

Reversing the precedence order for important declarations ensures users with special needs, such as personalized color schemes or large fonts, can override author styles when needed by marking some declarations in their user's style sheet as important. It also guarantees malicious extensions can't override important user-agent styles, which might break functionality or negatively impact security.

Does anything have precedence over important declarations? Yes, transitions. CSS transitions are a way to control the speed at which the property changes from one value to another. While transitioning from one value to another, a property will not match a specific important declaration.

```
a {
  color: red !important;
  background-color: yellow;
  transition: all 2s linear;
}
a:hover {
  color: blue !important;
  background-color: orange !important;
}
```

In this example, the `color` and `background-color` properties will transition to the hovered state over two seconds. Even though default states are normal declarations and hover states are `!important` declarations, the transition does happen.

Cascade layers

Within each of the three origins for style sheets – author, user, and user-agent – normal declarations in unlayered styles override layered style declarations, with the last declared having precedence over the layers declared before it. Important declarations reverse the order of precedence: important declarations in the first layer take precedence over important declarations in the next layer, and so on. Also, all the important declarations have precedence over important declarations made outside any layer.

Inline styles

Inline styles are styles defined using the `style` attributes. They can also be normal or important. Inline *normal* styles take precedence over all *normal* declarations, no matter the origin. Inline *important* styles take precedence over all other *important* author styles, no matter the layer, but important styles from user's or user-agent's style sheets and transitions override them.

!important and specificity

While `!important` is not part of determining specificity, it is related. Important declarations override all other declarations from the same origin and cascade layer.

```
#myElement#myElement#myElement .myClass.myClass p:hover {
  color: blue;
}

p {
  color: red !important;
}
```

This example displays a case of over-specifying a selector. No matter how high the selector specificity matches a normal declaration, an important declaration from the same source and cascade layer will always have precedence. In this case, the paragraph will always be red.

When two important declarations from the same origin and layer apply to the same element, browsers select and use the declaration with the highest specificity.

```
#myElement p {
  color: green !important;
}

p {
  color: purple !important;
}
```

In this case, the selector specificity matters. Only if the selectors had the same specificity would source order matter.

Impact on shorthand properties

Declaring a shorthand property with `!important` sets all of sub-properties as important. The two following selector style blocks are equivalent:

```
p {
  background: blue !important;
}

p {
  background-image: none !important;
  background-position: 0 0 !important;
  background-size: auto auto !important;
  background-repeat: repeat !important;
  background-origin: padding-box !important;
  background-clip: border-box !important;
  background-attachment: scroll !important;
  background-color: blue !important;
}
```

This example shows one of the several reasons avoiding the important flag is generally recommended.

Impact on custom properties

When the `!important` flag is added to a custom property value declaration, it makes the value assignment important. The `!important` flag is then stripped from the custom property value. The `!important` flag is not passed as part of the custom property value to the `var()` function.

```
:root {
  --my-color: red !important;
  --my-color: blue;
}
p {
  color: var(--my-color);
}
blockquote {
  color: var(--my-color);
  color: purple;
}
```

```
<p>This is a paragraph</p>
<blockquote>This is a blockquote</blockquote>
```

In this example, the paragraph will be red, not blue, as the custom property value assignment is important. The blockquote will be purple, because the purple normal declaration comes after the normal red declaration.

Best practices

Avoid using `!important` to override specificity. When intentionally creating important declarations for UI requirements, comment in your CSS code to explain to maintainers why they should not override that feature.

Even when working to override high-specificity styles not under your control, such as styles in a third-party plugin declared with an id selector, you don't need to use `!important`. Consider instead importing the third-party stylesheet script into a named or anonymous layer as your first cascade layer, instead of using `!important`. As long as the external styles do not include important declarations, your styles will take precedence over the widget styles, no matter the specificity.

If you need to override an external stylesheet containing important declarations, create a cascade layer containing the needed overrides, and declare that layer first.

Accessibility concerns

Important styles from a user stylesheet take precedence over the author style sheet's important declarations, meaning adding an `!important` flag to a site's styles will not prevent individual users with special requirements, such as large fonts, from being able to override your styles by adding important styles in their own user's style sheet.

Browser compatibility

This feature is supported in all browsers.

See also

- CSS Specificity
- CSS Cascade

inherit CSS keyword

The **`inherit`** CSS keyword causes the element to take the computed value of the property from its parent element. It can be applied to any CSS property, including the CSS shorthand property `all`.

For inherited properties, this reinforces the default behavior, and is only needed to override another rule.

> **Note:** Inheritance is always from the parent element in the document tree, even when the parent element is not the containing block.

Examples

Basic usage

In this example, we demonstrate the effect of the `inherit` keyword by comparing two paragraphs with nested elements: the inline elements in one use their browser default styles, while those in the other inherit individual property values from their parent.

HTML

We include two identical paragraphs with several inline elements.

```
<p>
  This paragraph has <em>emphasized text</em>, <strong>strong text</strong>, and
  <a href="#">a link to top</a>.
</p>
<p>
  This paragraph has <em>emphasized text</em>, <strong>strong text</strong>, and
  <a href="#">a link to top</a>.
</p>
```

CSS

We don't style the inline elements in the first paragraph, so they use their default browser styles. In the second paragraph, we set properties on each inline element to `inherit`, so they get the computed styles from the parent `<p>` element.

```
p:nth-of-type(2) {
  a {
    text-decoration: inherit;
    color: inherit;
  }
  em {
    font-style: inherit;
  }
  strong {
    font-weight: inherit;
  }
}
```

Result

Inheriting all property values

In this example, we use the same HTML as in the previous example to demonstrate the issues that can occur when the `inherit` keyword is applied to all properties.

```
<p>
  This paragraph has <em>emphasized text</em>, <strong>strong text</strong>, and
  <a href="#">a link to top</a>.
</p>
<p>
  This paragraph has <em>emphasized text</em>, <strong>strong text</strong>, and
  <a href="#">a link to top</a>.
</p>
```

CSS

In the second paragraph, instead of setting individual properties to `inherit`, we set the `all` property on all inline elements to `inherit`, so they all get computed styles from their parent `<p>`.

```
p:nth-of-type(2) > * {
  all: inherit;
}
```

Result

Notice how the inline element inherit all the properties of the parent `<p>`, including the paragraph's block-level `display` value. This is likely not the effect you would want.

Excluding selected elements from a rule

This example demonstrates how the `inherit` keyword can be used to exclude selected elements from a color rule, allowing them to use their parent's color instead.

HTML

We include some semantically structured content.

```
<header>
  <h1>This is my blog</h1>
  <h2>This is the subtitle of my blog in the HEADER</h2>
  <p>My blog is not very interesting</p>
</header>
<main>
  <h2>This first blog post in MAIN</h2>
  <p>I really have nothing to say.</p>
  <section>
    <h2>This second blog post is in a SECTION within MAIN</h2>
    <p>
      It is in a section because it is important even though it isn't the least
      bit interesting.
    </p>
  </section>
</main>
<footer>
  <h2>Contact in FOOTER</h2>
  <p>Find me on Mastodon</p>
  <section>
    <h2>Copyright in SECTION within FOOTER</h2>
    <p>1996</p>
  </section>
</footer>
```

CSS

We set the text color of the `<main>` element to `blue` and all `<h2>` elements to `green` in `monospace` font. The `<h2>` elements inside a `<section>` are set to `inherit` their parent's text color.

```
main {
  color: blue;
}

h2 {
  color: green;
  font-family: monospace;
}

section h2 {
  color: inherit;
}
```

Result

The `<h2>` elements are all `green`. However, if they are nested in a `<section>` element, they inherit their color from their parent, which is `blue` within `<main>`. The default text color is black otherwise.

See also

- `initial`, `revert`, `revert-layer`, `revert-rule`, and `unset` keywords
- Inheritance

initial CSS keyword

The **`initial`** CSS keyword applies the initial (or default) value of a property to an element. It can be applied to any CSS property, including the CSS shorthand property `all`. With `all` set to `initial`, all CSS properties can be restored to their respective initial values in one go instead of restoring each one separately.

On inherited properties, the initial value may be unexpected. You should consider using the `inherit`, `unset`, `revert`, `revert-layer`, or `revert-rule` keywords instead.

Examples

Basic usage

In this example, we use the `initial` keyword to reset an element's `color` and `font-size` property values.

HTML

```
<p>
  This text is red and large.
  <em
    >This text is in the initial color (typically black) and initial size
    (typically 16px).</em
  >
  This is red and large again.
</p>
```

CSS

We set `color` and `font-size` on the `<p>` element, then set those properties to `initial` on the `<em>` element to reset them.

```
p {
  color: red;
  font-size: 2rem;
}

em {
  color: initial;
  font-size: initial;
}
```

Result

With the `initial` keyword in this example, the `color` and `font-size` values on the `em` element are restored to the initial values for `color` and `font-size`, respectively.

See also

- `inherit`, `revert`, `revert-layer`, `revert-rule`, and `unset` keywords
- `all`
- Inheritance

<integer> CSS type

The **`<integer>`** CSS data type is a special type of `<number>` that represents a positive or negative whole number. Integers can be used in numerous CSS properties and descriptors, such as the `column-count`, `counter-increment`, `grid-column`, `grid-row`, and `z-index` properties and the `range` descriptor.

Syntax

The `<integer>` data type consists of one or several decimal digits, 0 through 9 inclusive, optionally preceded by a single `+` or `-` sign. There is no unit associated with integers.

> **Note:** There is no official range of valid `<integer>` values, and the specifications do not specify a range.

Interpolation

When animated, values of the `<integer>` data type are interpolated using discrete, whole steps. The calculation is done as if they were real, floating-point numbers; the discrete value is obtained using the floor function. The speed of the interpolation is determined by the easing function associated with the animation.

Examples

Valid integers

```
12          Positive integer (without a leading + sign)
+123        Positive integer (with a leading + sign)
-456        Negative integer
```

```
0          Zero
+0         Zero, with a leading +
-0         Zero, with a leading -
```

Invalid integers

```
12.0       This is a <number>, not an <integer>, though it represents an integer.
12.        Decimal points are not allowed.
+---12     Only one leading +/- is allowed.
ten        Letters are not allowed.
_5         Special characters are not allowed.
\35        Escaped Unicode characters are not allowed, even if they are an integer (here: 5).
\4E94      Non-arabic numerals are not allowed, even when escaped (here: the Japanese 5, 五).
3e4        Scientific notation is not allowed.
```

See also

- `<number>`

<length> CSS type

The **`<length>`** CSS data type represents a distance value. Lengths can be used in numerous CSS properties, such as `width`, `height`, `margin`, `padding`, `border-width`, `font-size`, and `text-shadow`.

> **Note:** Although `<percentage>` values are usable in some of the same properties that accept `<length>` values, they are not themselves `<length>` values. See `<length-percentage>`.

Syntax

The `<length>` data type consists of a `<number>` followed by one of the units listed below. As with all CSS dimensions, there is no space between the number and the unit literal. Specifying the length unit is optional if the number is `0`.

> **Note:** Some properties allow negative `<length>` values, while others do not.

Specified vs. computed values

The specified value of a length (*specified length*) is represented by its quantity and unit. The computed value of a length (*computed length*) is the specified length resolved to an absolute length, and its unit is not distinguished.

For some properties, such as `border-width`, `outline-width`, `column-rule-width`, and `outline-offset`, the computed `<length>` values are rounded to an integer number of device pixels to ensure reasonable visual display:

- A non-zero value less than 1 device pixel is rounded up.
- A value greater than 1 device pixel is rounded down to the nearest whole device pixel.

For example, on a screen with a `devicePixelRatio` of 3, `border-width: 1.5px` computes to approximately `1.33px` (rounded down from 4.5 to 4 device pixels), and `outline-width: 0.2px` computes to approximately `0.33px` (rounded up from 0.6 to 1 device pixel).

Relative vs. absolute lengths

The `<length>` units can be relative or absolute. Relative lengths represent a measurement in terms of some other distance. Depending on the unit, this distance can be the size of a specific character, the line height, or the size of the viewport. Style sheets that use relative length units can more easily scale from one output environment to another.

> **Note:** Child elements do not inherit the relative values as specified for their parent; they inherit the computed values.

Relative length units

CSS relative length units are based on font, container, or viewport sizes.

Relative length units based on font

Font lengths define the `<length>` value in terms of the size of a particular character or font attribute in the font currently in effect in an element or its parent.

> **Note:** These units, especially `em` and the root relative `rem`, are often used to create scalable layouts, which maintain the vertical rhythm of the page even when the user changes the font size.

`cap`

: Equal to the "cap height" (nominal height of capital letters) of the element's `font`.

`ch`

: Represents the width or, more precisely, the advance measure of the glyph `0` (zero, the Unicode character U+0030) in the element's `font`. In cases where determining the measure of the `0` glyph is impossible or impractical, it must be assumed to be `0.5em` wide by `1em` tall.

`em`

: Represents the calculated `font-size` of the element. If used on the `font-size` property itself, it represents the *inherited* font-size of the element.

`ex`

: Equal to the x-height of the element's `font`. In fonts with the `x` letter, this is generally the height of lowercase letters in the font; `1ex ≈ 0.5em` in many fonts.

`ic`

: Represents the used advance measure of the "水" glyph (CJK water ideograph, U+6C34), found in the font used to render it.

`lh`

: Equal to the computed value of the `line-height` property of the element on which it is used, converted to an absolute length. This unit enables length calculations based on the theoretical size of an ideal empty line. However, the size of actual line boxes may differ based on their content.

Relative length units based on root element's font

Root element font relative length units define the `<length>` value in terms of the size of a particular character or font attribute of the root element:

`rcap`

Equal to the "cap height" (nominal height of capital letters) of the root element's `font`.

`rch`

Equal to the width or the advance measure of the glyph `0` (zero, the Unicode character U+0030) in the root element's `font`.

`rem`

Represents the `font-size` of the root element (typically `<html>`). When used within the root element `font-size`, it represents its initial value. A common browser default is `16px`, but user-defined preferences may modify this.

`rex`

Equal to the x-height of the root element's `font`.

`ric`

Equal to the value of `ic` unit on the root element's font.

`rlh`

Equal to the value of `lh` unit on the root element's font. This unit enables length calculations based on the theoretical size of an ideal empty line. However, the size of actual line boxes may differ based on their content.

Relative length units based on viewport

The **viewport-percentage length units** are based on four different viewport sizes: small, large, dynamic, and default. The allowance for the different viewport sizes is in response to browser interfaces expanding and retracting dynamically and hiding and showing the content underneath.

Small viewport units

When you want the smallest possible viewport in response to browser interfaces expanding dynamically, you should use the small viewport size. The small viewport size allows the content you design to fill the entire viewport when browser interfaces are expanded. Choosing this size might also possibly leave empty spaces when browser interfaces retract.

For example, an element that is sized using viewport-percentage units based on the small viewport size, the element will fill the screen perfectly without any of its content being obscured when all the dynamic browser interfaces are shown. When those browser interfaces are hidden, however, there might be extra space visible around the element. Therefore, the small viewport-percentage units are "safer" to use in general, but might not produce the most attractive layout after a user starts interacting with the page.

The small viewport size is represented by the `sv` prefix and results in the `sv*` viewport-percentage length units. The sizes of the small viewport-percentage units are fixed, and therefore stable, unless the viewport itself is resized.

Large viewport units

When you want the largest possible viewport in response to browser interfaces retracting dynamically, you should use the large viewport size. The large viewport size allows the content you design to fill the entire viewport when browser interfaces are retracting. You need to be aware that the content might get hidden when browser interfaces expand.

For example, on mobile phones where screen real-estate is at a premium, browsers often hide part or all of the title and address bar after a user starts scrolling the page. When an element is sized using a viewport-percentage unit based on the large viewport size, the content of the element will fill the entire visible page when these browser interfaces are hidden. However, when these retractable browser interfaces are shown, they can hide the content that is sized or positioned using the *large* viewport-percentage units.

The large viewport unit is represented by the `lv` prefix and results in the `lv*` viewport-percentage units. The sizes of the large viewport-percentage units are fixed and therefore stable, unless the viewport itself is resized.

Dynamic viewport units

When you want the viewport to be automatically sized in response to browser interfaces dynamically expanding or retracting, you can use the dynamic viewport size. The dynamic viewport size allows the content you design to fit exactly within the viewport, irrespective of the presence of dynamic browser interfaces.

The dynamic viewport unit is represented by the `dv` prefix and results in the `dv*` viewport-percentage units. The sizes of the dynamic viewport-percentage units are not stable, even when the viewport itself is unchanged.

> **Note:** While the dynamic viewport size can give you more control and flexibility, using viewport-percentage units based on the dynamic viewport size can cause the content to resize while a user is scrolling a page. This can lead to degradation of the user interface and cause a performance hit.

Default viewport units

The default viewport size is defined by the browser. The behavior of the resulting viewport-percentage unit could be equivalent to the viewport-percentage unit based on the small viewport size, the large viewport size, an intermediate size between the two, or the dynamic viewport size.

> **Note:** For example, a browser might implement the default viewport-percentage unit for height (`vh`) that is equivalent to the large viewport-percentage height unit (`lvh`). If so, this could obscure content on a full-page display while the browser interface is expanded. Currently, all default viewport units (`vh`, `vw`, etc.) are equivalent to their large viewport counterparts (`lvh`, `lvw`, etc.).

Viewport-percentage lengths define `<length>` values in percentage relative to the size of the initial containing block, which in turn is based on either the size of the viewport or the page area, i.e., the visible portion of the document. When the height or width of the initial containing block is changed, the elements that are sized based on them are scaled accordingly. There is a viewport-percentage length unit variant corresponding to each of the viewport sizes, as described below.

> **Note:** Viewport lengths are invalid in `@page` declaration blocks.

`vh`

Represents a percentage of the height of the viewport's initial containing block. `1vh` is 1% of the viewport height. For example, if the viewport height is `300px`, then a value of `70vh` on a property will be `210px`.

The respective viewport-percentage units for small, large, and dynamic viewport sizes are `svh`, `lvh`, and `dvh`. `vh` is equivalent to `lvh`, representing the viewport-percentage length unit based on the large viewport size.

`vw`

Represents a percentage of the width of the viewport's initial containing block. `1vw` is 1% of the viewport width. For example, if the viewport width is `800px`, then a value of `50vw` on a property will be `400px`.

For small, large, and dynamic viewport sizes, the respective viewport-percentage units are `svw`, `lvw`, and `dvw`. `vw` is equivalent to `lvw`, representing the viewport-percentage length unit based on the large viewport size.

`vmax`

Represents in percentage the largest of `vw` and `vh`.

For small, large, and dynamic viewport sizes, the respective viewport-percentage units are `svmax`, `lvmax`, and `dvmax`. `vmax` is equivalent to `lvmax`, representing the viewport-percentage length unit based on the large viewport size.

`vmin`

Represents in percentage the smallest of `vw` and `vh`.

For small, large, and dynamic viewport sizes, the respective viewport-percentage units are `svmin`, `lvmin`, and `dvmin`. `vmin` is equivalent to `lvmin`, representing the viewport-percentage length unit based on the large viewport size.

`vb`

Represents the percentage of the size of the initial containing block, in the direction of the root element's block axis.

For small, large, and dynamic viewport sizes, the respective viewport-percentage units are `svb`, `lvb`, and `dvb`, respectively. `vb` is equivalent to `lvb`, representing the viewport-percentage length unit based on the large viewport size.

`vi`

Represents a percentage of the size of the initial containing block, in the direction of the root element's inline axis.

For small, large, and dynamic viewport sizes, the respective viewport-percentage units are `svi`, `lvi`, and `dvi`. `vi` is equivalent to `lvi`, representing the viewport-percentage length unit based on the large viewport size.

Container query length units

When applying styles to a container using container queries, you can use container query length units. These units specify a length relative to the dimensions of a query container. Components that use units of length relative to their container are more flexible to use in different containers without having to recalculate concrete length values.

If no eligible container is available for the query, the container query length unit defaults to the small viewport unit for that axis (`sv*`).

For more information, see Container queries.

`cqw`

Represents a percentage of the width of the query container. `1cqw` is 1% of the query container's width. For example, if the query container's width is `800px`, then a value of `50cqw` on a property will be `400px`.

`cqh`

Represents a percentage of the height of the query container. `1cqh` is 1% of the query container's height. For example, if the query container's height is `300px`, then a value of `10cqh` on a property will be `30px`.

`cqi`

Represents a percentage of the inline size of the query container. `1cqi` is 1% of the query container's inline size. For example, if the query container's inline size is `800px`, then a value of `50cqi` on a property will be `400px`.

`cqb`

Represents a percentage of the block size of the query container. `1cqb` is 1% of the query container's block size. For example, if the query container's block size is `300px`, then a value of `10cqb` on a property will be `30px`.

`cqmin`

Represents a percentage of the smaller value of either the query container's inline size or block size. `1cqmin` is 1% of the smaller value of either the query container's inline size or block size. For example, if the query container's inline size is `800px` and its block size is `300px`, then a value of `50cqmin` on a property will be `150px`.

`cqmax`

Represents a percentage of the larger value of either the query container's inline size or block size. `1cqmax` is 1% of the larger value of either the query container's inline size or block size. For example, if the query container's inline size is `800px` and its block size is `300px`, then a value of `50cqmax` on a property will be `400px`.

Absolute length units

Absolute length units represent a physical measurement when the physical properties of the output medium are known, such as for print layout. This is done by anchoring one of the units to a **physical unit** or the **visual angle unit** and then defining the others relative to it. Physical units include `cm`, `in`, `mm`, `pc`, `pt`, `px`, and `Q`.The anchoring is done differently for low-resolution devices, such as screens, versus high-resolution devices, such as printers.

For low-dpi devices, the unit `px` represents the physical *reference pixel*; other units are defined relative to it. Thus, `1in` is defined as `96px`, which equals `72pt`. The consequence of this definition is that on such devices, dimensions described in inches (`in`), centimeters (`cm`), or millimeters (`mm`) don't necessarily match the size of the physical unit with the same name.

For high-dpi devices, inches (`in`), centimeters (`cm`), and millimeters (`mm`) are the same as their physical counterparts. Therefore, the `px` unit is defined relative to them (1/96 of `1in`).

> **Note:** Many users increase their user agent's default font size to make text more legible. Absolute lengths can cause accessibility problems because they are fixed and do not scale according to user settings. For this reason, prefer relative lengths (such as `em` or `rem`) when setting `font-size`.

`px`

One pixel. For screen displays, it traditionally represents one device pixel (dot). However, for *printers* and *high-resolution screens*, one CSS pixel implies multiple device pixels. `1px = 1in / 96`.

`cm`

One centimeter. `1cm = 96px / 2.54`.

`mm`

One millimeter. `1mm = 1cm / 10`.

`Q`

One quarter of a millimeter. `1Q = 1cm / 40`.

`in`

One inch. `1in = 2.54cm = 96px`.

`pc`

One pica. `1pc = 12pt = 1in / 6`.

`pt`

One point. `1pt = 1in / 72`.

Interpolation

When animated, values of the `<length>` data type are interpolated as real, floating-point numbers. The interpolation happens on the calculated value. The speed of the interpolation is determined by the easing function associated with the animation.

Examples

Comparing different length units

The following example provides you with an input field in which you can enter a `<length>` value (e.g., `300px`, `50%`, `30vw`) to set the width of a result bar that will appear below it once you've pressed the `Enter` or the `Return` key.

This allows you to compare and contrast the effects of different length units.

HTML

```
<div class="outer">
  <div class="input-container">
    <label for="length">Enter width:</label>
    <input type="text" id="length" />
  </div>
  <div class="inner"></div>
</div>
<div class="results"></div>
```

CSS

```
html {
  font-family: sans-serif;
  font-weight: bold;
  box-sizing: border-box;
}

.outer {
  width: 100%;
  height: 50px;
  background-color: #eeeeee;
  position: relative;
}

.inner {
  height: 50px;
  background-color: #999999;
  box-shadow:
    inset 3px 3px 5px rgb(255 255 255 / 50%),
    inset -3px -3px 5px rgb(0 0 0 / 50%);
}

.result {
  height: 20px;
  box-shadow:
    inset 3px 3px 5px rgb(255 255 255 / 50%),
    inset -3px -3px 5px rgb(0 0 0 / 50%);
  background-color: orange;
  display: flex;
  align-items: center;
  margin-top: 10px;
}

.result code {
  position: absolute;
  margin-left: 20px;
}

.results {
  margin-top: 10px;
}

.input-container {
  position: absolute;
  display: flex;
  justify-content: flex-start;
  align-items: center;
  height: 50px;
}

label {
  margin: 0 10px 0 20px;
}
```

JavaScript

```
const inputDiv = document.querySelector(".inner");
const inputElem = document.querySelector("input");
const resultsDiv = document.querySelector(".results");

inputElem.addEventListener("change", () => {
  inputDiv.style.width = inputElem.value;

  const result = document.createElement("div");
  result.className = "result";
  result.style.width = inputElem.value;
  const code = document.createElement("code");
  code.textContent = `width: ${inputElem.value}`;
  result.appendChild(code);
  resultsDiv.appendChild(result);

  inputElem.value = "";
```

```
  inputElem.focus();
});
```

Result

See also

- Learn: Values and units
- CSS values & units module
- Box Model

<length-percentage> CSS type

The **`<length-percentage>`** CSS data type represents a value that can be either a `<length>` or a `<percentage>`.

Syntax

Refer to the documentation for `<length>` and `<percentage>` for details of the individual syntaxes allowed by this type.

Formal syntax

```
<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

length-percentage examples

This example demonstrates several properties that use `<length-percentage>` values.

HTML

```
<p>You can use percentages and lengths in so many places.</p>
```

CSS

```
p {
  /* length-percentage examples */
  width: 75%;
  height: 200px;
  margin: 3rem;
  padding: 1%;
  border-radius: 10px 10%;
  font-size: 250%;
  line-height: 1.5em;

  /* length examples */
  text-shadow: 1px 1px 1px red;
  border: 5px solid red;
  letter-spacing: 3px;

  /* percentage example */
  text-size-adjust: 20%;
}
```

Result

Use in calc()

Where a `<length-percentage>` is specified as an allowable type, this means that the percentage resolves to a length and therefore can be used in a `calc()` expression. Therefore, all of the following values are acceptable for `width`:

```
width: 200px;
width: 20%;
width: calc(100% - 200px);
```

See also

- `<percentage>`
- `<length>`
- CSS values and units module

<line-style> CSS type

The **`<line-style>`** enumerated value type represents keyword values that define the style of a line, or the lack of a line. The `<line-style>` keyword values are used in the following longhand and shorthand border and gap decoration properties:

- `border`, `border-style`
- `border-block`, `border-block-style`
- `border-block-end`, `border-block-end-style`
- `border-block-start`, `border-block-start-style`
- `border-bottom`, `border-bottom-style`
- `border-inline`, `border-inline-style`
- `border-inline-end`, `border-inline-end-style`
- `border-inline-start`, `border-inline-start-style`
- `border-left`, `border-left-style`
- `border-right`, `border-right-style`
- `border-top`, `border-top-style`
- `column-rule`, `column-rule-style`
- `row-rule`, `row-rule-style`

Syntax

Values

The `<line-style>` enumerated type is specified using one of the values listed below:

`none`

Displays no line. The used value of the line width is `0` even if a width value is specified. In the case of table cell and border collapsing, the `none` value has the *lowest* priority. If any other conflicting border is set, it will be displayed. The `none` value is similar to `hidden`.

`hidden`

Displays no line. The used value of the line width is `0` even if a width value is specified. In the case of table cell and border collapsing, the `hidden` value has the *highest* priority. If any other conflicting border is set, it won't be displayed. The `hidden` value is similar to `none`, but `hidden` is not a valid value for outline styles.

`dotted`

Displays a series of round dots. The radius of the dots is half the computed value of the line's width. The spacing of the dots is not defined by the specification and is implementation-specific.

`dashed`

Displays a series of short square-ended dashes or line segments. The exact size and length of the segments are not defined by the specification and are implementation-specific.

`solid`

Displays a single, straight solid line.

`double`

Displays two straight lines with some space between them. The length of the lines adds up to the pixel size defined by the line's width.

`groove`

Displays a border with a carved appearance. This value is the opposite of `ridge`.

`ridge`

Displays a border with an extruded appearance. This value is the opposite of `groove`.

`inset`

Displays a border that makes the element appear embedded. This value is the opposite of `outset`. When applied to a table cell border and `border-collapse` is set to `collapsed`, this value behaves like `groove`.

`outset`

Displays a border that makes the element appear embossed. This value is the opposite of `inset`. When applied to a table cell with `border-collapse` set to `collapsed`, this value behaves like `ridge`.

> **Note:** When `<outline-style>` is used as the value type for `outline` and `outline-style` properties, it is similar to `<line-style>`, but does not support `hidden` and includes the `auto` value. When `auto` is set, the user-agent defined `<line-style>` value is used.

Formal syntax

```
<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

Examples

The first example demonstrates all the `<line-style>` keyword values. The second example demonstrates how some line style colors may display in unexpected ways.

Defining line styles

This example shows all the `<line-style>` values as values for the CSS `border-style` and `column-rule-style` properties.

HTML

This example uses multiple `<div>` elements, each with a class representing the `<line-style>` value that is being demonstrated.

```
<div class="{line-style}">
  <p>{line-style}</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
```

```
<div class="none">
  <p>none</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="hidden">
  <p>hidden</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="dotted">
  <p>dotted</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="dashed">
  <p>dashed</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="solid">
  <p>solid</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
```

```
</div>
<div class="double">
  <p>double</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="groove">
  <p>groove</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="ridge">
  <p>ridge</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="inset">
  <p>inset</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
<div class="outset">
  <p>outset</p>
  <p>a b c d e f g h i j k l m n o p q r s t u v w x y z</p>
</div>
```

CSS

In the CSS for this example, the border and the column-rule for all the `<p>` elements is defined to have a width of `7px` and the style value of `double`. For each paragraph, the `double` value is then overridden by specifying a different `<line-style>` value for the `border-style` and `column-rule-style` properties.

```
div {
  display: flex;
  gap: 1em;
  list-style: none;
  align-items: center;
}
div:first-of-type {
  display: none;
}

p:first-of-type {
  text-align: center;
  line-height: 5em;
}
p {
  width: 10em;
  height: 5em;
  background-color: palegoldenrod;
}
```

```
p {
  padding: 5px;
  border: double 7px #bada55;
}
p + p {
  columns: 3;
  column-gap: 20px;
  column-rule: double 7px;
  border-color: black;
}
.none p {
  border-style: none;
  column-rule-style: none;
}
.hidden p {
  border-style: hidden;
  column-rule-style: hidden;
}
.dotted p {
  border-style: dotted;
  column-rule-style: dotted;
}
.dashed p {
  border-style: dashed;
  column-rule-style: dashed;
}
.solid p {
  border-style: solid;
  column-rule-style: solid;
}
.double p {
  border-style: double;
  column-rule-style: double;
}
.groove p {
  border-style: groove;
  column-rule-style: groove;
}
.ridge p {
  border-style: ridge;
  column-rule-style: ridge;
}
.inset p {
  border-style: inset;
  column-rule-style: inset;
}
.outset p {
  border-style: outset;
  column-rule-style: outset;
}
```

Result

Notice that the black border is not always black.

Defining line styles and colors

This example demonstrates line-style and color choice. With some `<line-style>` keyword values, the color of the line may not be what you expect. To create the required "3D" effect of `groove`, `ridge`, `inset`, and `outset` styles when displaying these values in black or white, user agents use different color calculations than any other color-line combinations.

CSS

The four sides of each `<div>` have a different `<line-style>` value, and each list item has a different `<color>` value. We use generated content to display the CSS declared inline.

```
body {
  display: flex;
  flex-wrap: wrap;
  gap: 1em;
  text-transform: uppercase;
  font-family: monospace;
}
```

```
div {
  border-width: 10px;
  border-style: inset groove ridge outset;
  padding: 5px;
}
```

JavaScript

The JavaScript dynamically creates `<div>` elements, each with a different `border-color` set.

```
// prettier-ignore
const colors = [
  "#000000", "#000001", "#ffffff",
  "#ff00ff", "#ffff00", "#00ffff",
  "#cc33cc", "#cccc33", "#33cccc",
  "#ff0000", "#00ff00", "#0000ff",
  "#cc3333", "#33cc33", "#3333cc",
  "#993333", "#339933", "#333399",
];

for (const c of colors) {
  const div = document.createElement("div");
  div.style.borderColor = c;
  div.textContent = c;
  document.body.appendChild(div);
}
```

Result

Notice that the almost-black color of `#000001` may be different from the actual black, and the contrast between the dark and light edges is more noticeable when using lighter colors.

See also

- `<line-width>` data type
- CSS backgrounds and borders module
- CSS basic user interface module
- CSS gaps module
- CSS multi-column layout module

<line-width> CSS type

The **`<line-width>`** enumerated value type represents the length or keyword value that defines the width of a line, or the lack of a line. The `<line-width>` keyword values are used in the following longhand and shorthand properties:

- `border`, `border-width`
- `border-block`, `border-block-width`
- `border-block-end`, `border-block-end-width`
- `border-block-start`, `border-block-start-width`
- `border-bottom`, `border-bottom-width`
- `border-inline`, `border-inline-width`
- `border-inline-end`, `border-inline-end-width`
- `border-inline-start`, `border-inline-start-width`
- `border-left`, `border-left-width`
- `border-right`, `border-right-width`
- `border-top`, `border-top-width`
- `column-rule`, `column-rule-width`
- `row-rule`, `row-rule-width`
- `rule`, `rule-width`
- `outline`, `outline-width`

Syntax

Values

The `<line-width>` type is specified using either `<length>` or a keyword:

`<length>`

A non-negative `<length>`.

`hairline`

Represents a "just visible" line that is less than or equal to `1px`.

`thin`

Same as `1px`.

`medium`

Same as `3px`.

`thick`

Same as `5px`.

Formal syntax

```
<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

Examples

Defining a line width

This example demonstrates all the `<line-width>` keyword values, along with a `<length>` value.

HTML

We include five boxes within a `<section>` element.

```
<section>
  <div data-width="hairline">hairline</div>
  <div data-width="thin">thin</div>
  <div data-width="medium">medium</div>
  <div data-width="thick">thick</div>
  <div data-width="10px">10px</div>
</section>
```

CSS

We style all the elements to be identical, then set the `border-width` based on the element's `data-width` attribute.

```
section {
  display: flex;
  gap: 10px;
}
div {
  flex: 0 0 15%;
  border-style: solid;
  border-color: purple;
  text-align: center;
}
[data-width="hairline"] {
  border-width: hairline;
}

[data-width="thin"] {
  border-width: thin;
}

[data-width="medium"] {
  border-width: medium;
}

[data-width="thick"] {
  border-width: thick;
}

[data-width="10px"] {
  border-width: 10px;
}
```

```
@supports not (border-width: hairline) {
  body::before {
    content: "Your browser does not support the 'hairline' value.";
    color: black;
    background-color: #ffcd33;
    display: block;
    width: 100%;
    text-align: center;
  }
}
```

Result

See also

- `<line-style>` data type
- CSS backgrounds and borders module
- CSS basic user interface module
- CSS gaps module

log() CSS function

The **`log()`** CSS function is an exponential function that returns the logarithm of a number.

Logarithm is the inverse of exponentiation. It is the number that a fixed base has to be raised to in order to yield the number passed as the first parameter.

In CSS, when a single parameter is passed, the natural logarithm `e`, or approximately `2.7182818`, is used, though the base can be set to any value with an optional second parameter.

Syntax

```
/* A <number> value */
width: calc(100px * log(7.389)); /* 200px */
```

```
width: calc(100px * log(8, 2)); /* 300px */
width: calc(100px * log(625, 5)); /* 400px */
```

Parameters

The `log(value [, base]?)` function accepts two comma-separated values as its parameters.

`value`

A calculation which resolves to a `<number>` greater than or equal to 0. Representing the value to be taken the log of.

`base`

Optional. A calculation which resolves to a `<number>` greater than or equal to 0. Representing the base of the logarithm. If not defined, the default logarithmic base `e` is used.

Return value

The logarithm of `value`, when `base` is defined.

The natural logarithm (base `e`) of `value`, when `base` is not defined.

Formal syntax

```
<log()> =
  log( <calc-sum> , <calc-sum>? )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>       |
  <dimension>    |
  <percentage>   |
  <calc-keyword> |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using the <code>log()</code> function on a logarithmic scale

This example illustrates how the `log()` function can be used to visualize data values by using a logarithmic scale. The width of each bar in this example is relative to its data value on a logarithmic scale with base 10. On each element, its value is assigned to a CSS custom property named `--value`, which is then used by the `.bar` class to calculate its width.

HTML

```
<div class="bar" style="--value: 50">50</div>
<div class="bar" style="--value: 100">100</div>
<div class="bar" style="--value: 500">500</div>
<div class="bar" style="--value: 10000">10,000</div>
<div class="bar" style="--value: 2000000">2,000,000</div>
```

CSS

```
.bar {
  width: calc(log(var(--value), 10) * 2em);
}
```

```
.bar {
  height: 2em;
  background-color: tomato;
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  margin: 0.25em 0;
}
```

Result

See also

- `pow()`
- `sqrt()`
- `hypot()`
- `exp()`

max() CSS function

The **`max()`** CSS function lets you set the largest (most positive) value from a list of comma-separated expressions as the value of a CSS property value. The `max()` function can be used anywhere a `<length>`, `<frequency>`, `<angle>`, `<time>`, `<percentage>`, `<number>`, or `<integer>` is allowed.

Try it

```
width: max(20vw, 400px);
```

```
width: max(20vw, 100px);
```

```
width: max(5vw, 100px);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <img
      alt="Firefox logo"
      class="logo"
      src="/shared-assets/images/examples/firefox-logo.svg" />
  </div>
</section>
```

In the first example shown above, the width will be at least 400px, but will be wider if the viewport is more than 2000px wide (in which case 1vw would be 20px, so 20vw would be 400px). This technique uses an absolute unit to specify a fixed minimum value for the property, and a relative unit to allow the value to grow to suit larger viewports.

Syntax

```
max(1, 2, 3)
max(1px, 2px, 3px)
```

Parameters

The `max()` function takes one or more comma-separated expressions as its parameter, with the largest (most positive) expression value used as the value of the property to which it is assigned.

The expressions can be math expressions (using arithmetic operators), literal values, or other expressions, such as `attr()`, that evaluate to a valid argument type (like `<length>`), or nested `min()` and `max()` functions.

You can use different units for each value in your expression. You may also use parentheses to establish computation order when needed.

Notes

- Math expressions involving percentages for widths and heights on table columns, table column groups, table rows, table row groups, and table cells in both auto and fixed layout tables *may* be treated as if `auto` had been specified.
- It is permitted to nest `min()` and other `max()` functions as expression values. The expressions are full math expressions, so you can use direct addition, subtraction, multiplication and division without using the `calc()` function itself.
- The expression can be values combining the addition (`+`), subtraction (`-`), multiplication (`*`), and division (`/`) operators, using standard operator precedence rules. Make sure to put a space on each side of the `+` and `-` operands.
- You can (and often need to) combine `min()` and `max()` values, or use `max()` within a `clamp()` or `calc()` function.

Formal syntax

```
<max()> =
  max( <calc-sum># )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

When `max()` is used for controlling text size, make sure the text is always large enough to read. A suggestion is to use the `min()` function nested within a `max()` that has as its second value a relative length unit that is always large enough to read. For example:

```
small {
  font-size: max(min(0.5vw, 0.5em), 1rem);
}
```

This ensures a minimum size of *1rem*, with a text size that scales if the page is zoomed.

- MDN Understanding WCAG, Guideline 1.4 explanations Understanding Success Criterion 1.4.4: Resize Text | WAI | W3C

Examples

Setting a minimum size for a font

Another use case for `max()` is to allow a font size to grow while ensuring it is at least a minimum size, enabling responsive font sizes while ensuring legibility.

Let's look at some CSS:

```
h1 {
  font-size: 2rem;
}
h1.responsive {
  font-size: max(4vw, 2em, 2rem);
}
```

The font-size will at minimum be 2rems, or twice the default size of font for the page. This ensures that it is legible and accessible.

```
<h1>This text is always legible, but doesn't change size</h1>
<h1 class="responsive">
  This text is always legible, and is responsive, to a point
</h1>
```

Think of the `max()` function as finding the minimum value allowed for a property.

See also

- `calc()`
- `clamp()`
- `min()`
- Learn: Values and units

max-content CSS keyword

The `max-content` sizing keyword represents the maximum intrinsic size of an element. The keyword expands the element to the largest size needed to display its content without any soft wraps. For text content, this keyword does not wrap the content at all, even if it causes overflow.

The `interpolate-size` property and `calc-size()` function can be used to enable animations to and from `max-content`.

Syntax

```
/* Used as a length value */
width: max-content;
inline-size: max-content;
height: max-content;
block-size: max-content;

/* Used in grid tracks */
grid-template-columns: 200px 1fr max-content;
```

Examples

Sizing boxes with max-content

HTML

```
<div id="container">
  <div class="item">Item</div>
  <div class="item">
    Item with more text in it which will overflow the fixed width box.
  </div>
</div>
```

CSS

```
#container {
  background-color: #8cffa0;
  padding: 10px;
  width: 200px;
}

.item {
  width: max-content;
  background-color: #8ca0ff;
  padding: 5px;
  margin-bottom: 1em;
}
```

Result

Sizing grid columns with max-content

HTML

```
<div id="container">
  <div>Item</div>
  <div>Item with more text in it.</div>
  <div>Flexible item</div>
</div>
```

CSS

```
#container {
  display: grid;
  grid-template-columns: max-content max-content 1fr;
  grid-gap: 5px;
  box-sizing: border-box;
  height: 200px;
  width: 100%;
  background-color: #8cffa0;
  padding: 10px;
}

#container > div {
  background-color: #8ca0ff;
  padding: 5px;
}
```

Result

See also

- Related sizing keywords: `min-content`, `fit-content`
- CSS box sizing module

min() CSS function

The **`min()`** CSS function lets you set the smallest (most negative) value from a list of comma-separated expressions as the value of a CSS property value. The `min()` function can be used anywhere a `<length>`, `<frequency>`, `<angle>`, `<time>`, `<percentage>`, `<number>`, or `<integer>` is allowed.

Try it

```
width: min(50vw, 200px);
```

```
width: min(100vw, 4000px);
```

```
width: min(150vw, 100px);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <img
      alt="Firefox logo"
      class="logo"
      src="/shared-assets/images/examples/firefox-logo.svg" />
  </div>
</section>
```

In the first above example, the width will be at most 200px, but will be smaller if the viewport is less than 400px wide (in which case 1vw would be 4px, so 50vw would be 200px). This technique uses an absolute unit to specify a fixed maximum value for the property, and a relative unit to allow the value to shrink to suit smaller viewports.

Syntax

```
min(1, 2, 3)
min(1px, 2px, 3px)
```

Parameters

The `min()` function takes one or more comma-separated expressions as its parameter, with the smallest (most negative) expression value result used as the value.

The expressions can be math expressions (using arithmetic operators), literal values, or other expressions, such as `attr()`, that evaluate to a valid argument type (like `<length>`).

You can use different units for each value in your expression, if you wish. You may also use parentheses to establish computation order when needed.

Notes

- Math expressions involving percentages for widths and heights on table columns, table column groups, table rows, table row groups, and table cells in both auto and fixed layout tables *may* be treated as if `auto` had been specified.
- It is permitted to nest `max()` and other `min()` functions as expression values. The expressions are full math expressions, so you can use direct addition, subtraction, multiplication and division without using the `calc()` function itself.
- The expression can be values combining the addition (+), subtraction (-), multiplication (*), and division (/) operators, using standard operator precedence rules. Make sure to put a space on each side of the + and - operands.
- You can (and often need to) combine `min()` and `max()` values, or use `min()` within a `clamp()` or `calc()` function.
- You can provide more than two arguments, if you have multiple constraints to apply.

Formal syntax

```
<min()> =
  min( <calc-sum># )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

When using `min()` to set a maximum font size, ensure that the font can still be scaled at least 200% for readability (without assistive technology like a zoom function).

- MDN Understanding WCAG, Guideline 1.4 explanations Understanding Success Criterion 1.4.4: Resize Text | WAI | W3C

Examples

Setting a maximum size for a label and input

Another use case for `min()` is to set a maximum size on responsive form controls: enabling the width of labels and inputs to shrink as the width of their container shrinks.

Let's look at some CSS:

```
input,
label {
  padding: 2px;
  box-sizing: border-box;
  display: inline-block;
  width: min(40%, 400px);
  background-color: pink;
}

.container {
  margin: 4px;
  border: 1px solid black;
  padding: 4px;
}
```

Here, the container itself, along with the margin, border, and padding, will be 100% of its parent's width. We declare the input and label to be the lesser of 40% of the container width up to the padding or 400px wide, whichever is smaller. In other words, the widest that the label and input can be is 400px. The narrowest they will be is 40% of the container's width, which on a smartwatch's screen is very small.

```
<div class="container">
  <label for="misc">Type something:</label>
  <input type="text" id="misc" name="misc" />
</div>
```

See also

- `calc()`
- `clamp()`
- `max()`
- Learn: Values and units

min-content CSS keyword

The `min-content` sizing keyword represents the minimum intrinsic size of an element. The keyword shrinks the element to the smallest possible size without causing avoidable overflow of its content. For text content, this keyword causes the content to wrap at every opportunity (such as spaces between words), and the element will be only as wide as the longest word.

The `interpolate-size` property and `calc-size()` function can be used to enable animations to and from `min-content`.

Syntax

```
/* Used as a length value */
width: min-content;
inline-size: min-content;
height: min-content;
block-size: min-content;

/* Used in grid tracks */
grid-template-columns: 200px 1fr min-content;
```

Examples

Sizing boxes with min-content

HTML

```
<div class="item">Item</div>
<div class="item">Item with more text in it.</div>
```

CSS

```
.item {
  width: min-content;
  background-color: #8ca0ff;
  padding: 5px;
  margin-bottom: 1em;
}
```

Result

Sizing grid columns with min-content

HTML

```
<div id="container">
  <div>Item</div>
  <div>Item with more text in it.</div>
  <div>Flexible item</div>
</div>
```

CSS

```
#container {
  display: grid;
  grid-template-columns: min-content min-content 1fr;
  grid-gap: 5px;
  box-sizing: border-box;
  height: 200px;
  width: 100%;
  background-color: #8cffa0;
  padding: 10px;
}

#container > div {
  background-color: #8ca0ff;
  padding: 5px;
}
```

Result

See also

- Related sizing keywords: `max-content`, `fit-content`
- CSS box sizing module

minmax() CSS function

The **`minmax()`** CSS function defines a size range greater than or equal to *min* and less than or equal to *max*. It is used with CSS grids.

Try it

```
grid-template-columns: minmax(20px, auto) 1fr 1fr;
```

```
grid-template-columns: minmax(0, 1fr) minmax(0, 1fr) minmax(0, 1fr);
```

```
grid-template-columns: minmax(2ch, 10ch) 1fr 1fr;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One. This column has more text in it.</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-gap: 10px;
  width: 250px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
  text-align: left;
}
```

Syntax

```
/* <inflexible-breadth>, <track-breadth> values */
minmax(200px, 1fr)
minmax(400px, 50%)
minmax(30%, 300px)
minmax(100px, max-content)
minmax(min-content, 400px)
minmax(max-content, auto)
minmax(auto, 300px)
minmax(min-content, auto)

/* <fixed-breadth>, <track-breadth> values */
minmax(200px, 1fr)
minmax(30%, 300px)
minmax(400px, 50%)
minmax(50%, min-content)
minmax(300px, max-content)
minmax(200px, auto)

/* <inflexible-breadth>, <fixed-breadth> values */
minmax(400px, 50%)
minmax(30%, 300px)
minmax(min-content, 200px)
minmax(max-content, 200px)
minmax(auto, 300px)
```

A function taking two parameters, *min* and *max*.

Each parameter can be a `<length>`, a `<percentage>` or one of the keyword values `max-content`, `min-content`, or `auto`.

The second parameter *max* also accepts `<flex>` values. (this `fr` units can only be used for *max*, and are invalid for *min*.)

If *max* < *min*, then *max* is ignored and `minmax(min,max)` is treated as *min*.

Values

`<length>`

A non-negative length.

`<percentage>`

A non-negative percentage relative to the inline size of the grid container in column grid tracks, and the block size of the grid container in row grid tracks. If the size of the grid container depends on the size of its tracks, then the `<percentage>` must be treated as `auto`. The user agent may adjust the intrinsic size contributions of the track to the size of the grid container and increase the final size of the track by the minimum amount that would result in honoring the percentage.

`<flex>`

A non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor.

`max-content`

Represents the largest max-content contribution of the grid items occupying the grid track.

`min-content`

Represents the largest min-content contribution of the grid items occupying the grid track.

`auto`

As `min`, it represents the largest minimum size (as specified by `min-width`/`min-height`) of the grid items occupying the grid track. As `max`, it is identical to `max-content`. However, unlike `max-content`, it allows expansion of the track by the `align-content` and `justify-content` property values like `normal` and `stretch`.

Formal syntax

```
<minmax()> =
  minmax( min , max )
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

CSS properties

`minmax()` function can be used within:

- `grid-template-columns`

- grid-template-rows
- grid-auto-columns
- grid-auto-rows

Examples

CSS

```
#container {
  display: grid;
  grid-template-columns: minmax(min-content, 300px) minmax(200px, 1fr) 150px;
  grid-gap: 5px;
  box-sizing: border-box;
  height: 200px;
  width: 100%;
  background-color: #8cffa0;
  padding: 10px;
}

#container > div {
  background-color: #8ca0ff;
  padding: 5px;
}
```

HTML

```
<div id="container">
  <div>Item as wide as the content, but at most 300 pixels.</div>
  <div>Item with flexible width but a minimum of 200 pixels.</div>
  <div>Inflexible item of 150 pixels width.</div>
</div>
```

Result

See also

- Basic concepts of grid layout: track sizing with minmax()
- CSS grids, logical values and writing modes
- Video: Introducing minmax()

mod() CSS function

The **`mod()`** CSS function returns a modulus left over when the first parameter is divided by the second parameter, similar to the JavaScript remainder operator (`%`). The modulus is the value left over when one operand, the dividend, is divided by a second operand, the divisor. It always takes the sign of the divisor.

The calculation is `a - (Math.floor(a / b) * b)`. For example, the CSS `mod(21, -4)` function returns the remainder of `-3`. The full calculation is `21 - (Math.floor(21 / -4) * -4)`. When dividing `21` by `-4`, the result is `-5.25`. This is floored to `-6`. Multiplying `-6` by `-4` is `24`. Subtracting this `24` from the original `21`, the remainder is -3.

Syntax

```
/* Unitless <number> */
line-height: mod(7, 2); /* 1 */
line-height: mod(14, 5); /* 4 */
line-height: mod(3.5, 2); /* 1.5 */

/* Unit based <percentage> and <dimension> */
margin: mod(15%, 2%); /* 1% */
margin: mod(18px, 4px); /* 2px */
margin: mod(19rem, 5rem); /* 4rem */
margin: mod(29vmin, 6vmin); /* 5vmin */
margin: mod(1000px, 29rem); /* 72px - if root font-size is 16px */

/* Negative/positive values */
rotate: mod(100deg, 30deg); /* 10deg */
rotate: mod(135deg, -90deg); /* -45deg */
rotate: mod(-70deg, 20deg); /* 10deg */
rotate: mod(-70deg, -15deg); /* -10deg */

/* Calculations */
scale: mod(10 * 2, 1.7); /* 1.3 */
rotate: mod(10turn, 18turn / 3); /* 4turn */
transition-duration: mod(20s / 2, 3000ms * 2); /* 4s */
```

Parameters

The `mod(dividend, divisor)` function accepts two comma-separated values as its parameters. Both parameters must have the same type, number, dimension, or `<percentage>`, for the function to be valid. While the units in the two parameters don't need to be the same, they do need of the same dimension type, such as `<length>`, `<angle>`, `<time>`, or `<frequency>` to be valid.

`dividend`

A calculation that resolves to a `<number>`, `<dimension>`, or `<percentage>` representing the dividend.

`divisor`

A calculation that resolves to a `<number>`, `<dimension>`, or `<percentage>` representing the divisor.

Return value

Returns a `<number>`, `<dimension>`, or `<percentage>` (corresponds to the parameters' type) representing the modulus, that is, the operation left over.

- If `divisor` is `0`, the result is `NaN`.
- If `dividend` is `infinite`, the result is `NaN`.
- If `divisor` is positive the result is positive (`0+`), and if `divisor` is negative the result is negative (`0-`).

Formal syntax

```
<mod()> =
  mod( <calc-sum> , <calc-sum> )

<calc-sum> =
```

```
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- round()
- rem()

<named-color> CSS type

The **<named-color>** CSS data type is the name of a color, such as red, blue, black, or lightseagreen. Syntactically, a <named-color> is an <ident>.

A <named-color> value can be used anywhere a <color> can be used.

Syntax

```
color: red;
color: orange;
color: tan;
color: rebeccapurple;
color: transparent;
```

Value

Named colors consists of standard colors, the transparent and currentColor keywords.

Standard colors

Basic colors have standard, memorable names:

Keyword	RGB hex value	Sample
black	#000000	
silver	#c0c0c0	
gray	#808080	
white	#ffffff	
maroon	#800000	
red	#ff0000	
purple	#800080	
fuchsia	#ff00ff	
green	#008000	
lime	#00ff00	
olive	#808000	
yellow	#ffff00	
navy	#000080	
blue	#0000ff	
teal	#008080	
aqua	#00ffff	

In addition to these 16 colors, about 150 other colors have a keyword associated to them:

Keyword	RGB hex value	Sample
aliceblue	#f0f8ff	
antiquewhite	#faebd7	
aqua	#00ffff	
aquamarine	#7fffd4	
azure	#f0ffff	
beige	#f5f5dc	
bisque	#ffe4c4	
black	#000000	
blanchedalmond	#ffebcd	
blue	#0000ff	
blueviolet	#8a2be2	
brown	#a52a2a	

Keyword	RGB hex value	Sample
burlywood	#deb887	
cadetblue	#5f9ea0	
chartreuse	#7fff00	
chocolate	#d2691e	
coral	#ff7f50	
cornflowerblue	#6495ed	
cornsilk	#fff8dc	
crimson	#dc143c	
cyan	#00ffff (synonym of aqua)	
darkblue	#00008b	
darkcyan	#008b8b	
darkgoldenrod	#b8860b	
darkgray	#a9a9a9	
darkgreen	#006400	
darkgrey	#a9a9a9	
darkkhaki	#bdb76b	
darkmagenta	#8b008b	
darkolivegreen	#556b2f	
darkorange	#ff8c00	
darkorchid	#9932cc	
darkred	#8b0000	
darksalmon	#e9967a	
darkseagreen	#8fbc8f	
darkslateblue	#483d8b	
darkslategray	#2f4f4f	
darkslategrey	#2f4f4f	
darkturquoise	#00ced1	
darkviolet	#9400d3	
deeppink	#ff1493	
deepskyblue	#00bfff	
dimgray	#696969	
dimgrey	#696969	
dodgerblue	#1e90ff	
firebrick	#b22222	
floralwhite	#fffaf0	
forestgreen	#228b22	
fuchsia	#ff00ff	
gainsboro	#dcdcdc	
ghostwhite	#f8f8ff	
gold	#ffd700	
goldenrod	#daa520	
gray	#808080	
green	#008000	
greenyellow	#adff2f	
grey	#808080 (synonym of gray)	
honeydew	#f0fff0	
hotpink	#ff69b4	
indianred	#cd5c5c	
indigo	#4b0082	
ivory	#fffff0	
khaki	#f0e68c	
lavender	#e6e6fa	
lavenderblush	#fff0f5	
lawngreen	#7cfc00	
lemonchiffon	#fffacd	
lightblue	#add8e6	
lightcoral	#f08080	
lightcyan	#e0ffff	
lightgoldenrodyellow	#fafad2	

Keyword	RGB hex value	Sample
lightgray	#d3d3d3	
lightgreen	#90ee90	
lightgrey	#d3d3d3	
lightpink	#ffb6c1	
lightsalmon	#ffa07a	
lightseagreen	#20b2aa	
lightskyblue	#87cefa	
lightslategray	#778899	
lightslategrey	#778899	
lightsteelblue	#b0c4de	
lightyellow	#ffffe0	
lime	#00ff00	
limegreen	#32cd32	
linen	#faf0e6	
magenta	#ff00ff (synonym of fuchsia)	
maroon	#800000	
mediumaquamarine	#66cdaa	
mediumblue	#0000cd	
mediumorchid	#ba55d3	
mediumpurple	#9370db	
mediumseagreen	#3cb371	
mediumslateblue	#7b68ee	
mediumspringgreen	#00fa9a	
mediumturquoise	#48d1cc	
mediumvioletred	#c71585	
midnightblue	#191970	
mintcream	#f5fffa	
mistyrose	#ffe4e1	
moccasin	#ffe4b5	
navajowhite	#ffdead	
navy	#000080	
oldlace	#fdf5e6	
olive	#808000	
olivedrab	#6b8e23	
orange	#ffa500	
orangered	#ff4500	
orchid	#da70d6	
palegoldenrod	#eee8aa	
palegreen	#98fb98	
paleturquoise	#afeeee	
palevioletred	#db7093	
papayawhip	#ffefd5	
peachpuff	#ffdab9	
peru	#cd853f	
pink	#ffc0cb	
plum	#dda0dd	
powderblue	#b0e0e6	
purple	#800080	
rebeccapurple	#663399	
red	#ff0000	
rosybrown	#bc8f8f	
royalblue	#4169e1	
saddlebrown	#8b4513	
salmon	#fa8072	
sandybrown	#f4a460	
seagreen	#2e8b57	
seashell	#fff5ee	
sienna	#a0522d	
silver	#c0c0c0	

Keyword	RGB hex value	Sample
skyblue	#87ceeb	
slateblue	#6a5acd	
slategray	#708090	
slategrey	#708090	
snow	#fffafa	
springgreen	#00ff7f	
steelblue	#4682b4	
tan	#d2b48c	
teal	#008080	
thistle	#d8bfd8	
tomato	#ff6347	
transparent	See transparent.	
turquoise	#40e0d0	
violet	#ee82ee	
wheat	#f5deb3	
white	#ffffff	
whitesmoke	#f5f5f5	
yellow	#ffff00	
yellowgreen	#9acd32	

Initially, in CSS Level 1, only 16 basic colors were defined, with `orange` added in CSS Level 2. Web designers found this list too short, and browser vendors added numerous names for colors based on the X11 color names. In SVG 1, then in CSS Colors Level 3, these names got standardized, formally defined, and made uniform (some had different spellings that are now aliases). They are called *extended color keywords*, *X11 colors*, or *SVG colors*.

In CSS Colors Level 4, an additional color, `rebeccapurple` was added to honor web pioneer Eric Meyer.

transparent

The `transparent` keyword represents a fully transparent color. This makes the background behind the colored item completely visible. Technically, `transparent` is a shortcut for `rgb(0 0 0 / 0%)`.

To prevent unexpected behavior, such as in a `<gradient>`, the current CSS spec states that `transparent` should be calculated in the alpha-premultiplied color space. However, be aware that older browsers may treat it as black with an alpha value of `0`.

The `transparent` keyword wasn't a true color in CSS Level 2 (Revision 1). It was a special keyword that could be used instead of a regular `<color>` value on two CSS properties: `background` and `border`. It was essentially added to allow developers to override an inherited solid color. With the advent of alpha channels in CSS Colors Level 3, `transparent` was redefined as a true color. It can now be used wherever a `<color>` value can be used.

Description

All names specify a color in the sRGB color space. Although the names more or less describe their respective colors, they are essentially artificial, without a strict rationale behind the terms used.

The color keywords all represent plain, solid colors without transparency.

Several keywords are aliases for each other:

- `aqua` / `cyan`
- `fuchsia` / `magenta`
- `darkgray` / `darkgrey`
- `darkslategray` / `darkslategrey`
- `dimgray` / `dimgrey`
- `lightgray` / `lightgrey`
- `lightslategray` / `lightslategrey`
- `gray` / `grey`
- `slategray` / `slategrey`

Though many keywords have been adapted from X11, their RGB values may differ from the corresponding color on X11 systems since manufacturers sometimes tailor X11 colors to their specific hardware.

Examples

Using named colors

HTML

```
<div id="container">
  <div id="one"></div>
  <div id="two"></div>
  <div id="three"></div>
</div>
```

CSS

```
#container {
  display: flex;
  justify-content: space-around;
  background-color: darkslateblue;
  padding: 20px;
}

#container > div {
  height: 100px;
  width: 100px;
  margin: 3px;
  border: 2px solid black;
}
```

```
#one {
  background-color: red;
}

#two {
  background-color: lavender;
}

#three {
  background-color: transparent;
}
```

Result

See also

- `<color>`: the data type of whose definition `<named-color>` is a constituent part.

<number> CSS type

The **`<number>`** CSS data type represents a number, being either an integer, a number with a fractional component, or a base-ten exponent in scientific notation.

Syntax

The syntax of `<number>` extends the syntax of `<integer>`. A fractional value is represented by a `.` followed by one or more decimal digits, and may be appended to an integer. A `<number>` can also end with the letter `e` or `E` followed by an integer, which indicates a base-ten exponent in scientific notation. There is no unit associated with numbers.

As with integers, the first character of the number can be immediately preceded by `-` or `+` to indicate the number's sign: whether the number is positive or negative.

Interpolation

When animated, values of the `<number>` CSS data type are interpolated as real, floating-point numbers. The speed of the interpolation is determined by the easing function associated with the animation.

Examples

Valid numbers

```
12          A raw <integer> is also a <number>.
4.01        Positive fraction
-456.8      Negative fraction
0.0         Zero
+0.0        Zero, with a leading +
-0.0        Zero, with a leading -
.60         Fractional number without a leading zero
10e3        Scientific notation
-3.4e-2     Complicated scientific notation
```

Invalid numbers

```
12.         Decimal points must be followed by at least one digit.
+-12.2      Only one leading +/- is allowed.
12.1.1      Only one decimal point is allowed.
```

See also

- `<integer>`
- `<ratio>`
- CSS values and units module

<overflow> CSS type

The **`<overflow>`** enumerated value type represents the keyword values for the `overflow-block`, `overflow-inline`, `overflow-x`, and `overflow-y` longhand properties and the `overflow` shorthand property. These properties apply to block containers, flex containers, and grid containers.

Syntax

```
<overflow> = visible | hidden | clip | scroll | auto
```

Values

The `<overflow>` enumerated value type is specified using one of the values listed below.

`visible`

> Overflow content is not clipped and may be visible outside the element's padding box. The element box is not a scroll container. This is the default value for all the properties that have the `<overflow>` enumerated value type.

`hidden`

> Overflow content is clipped at the element's padding box. There are no scroll bars, and the clipped content is not visible (i.e., clipped content is hidden), but the content still exists. User agents do not add scrollbars and also do not allow users to view the content outside the clipped region by actions such as dragging on a touch screen or using the scroll wheel on a mouse. The content *can* be scrolled programmatically (for example, by setting the value of the `scrollLeft` property or the `scrollTo()` method). The content can also be scrolled via keyboard interaction; arrows enable scrolling through the content and tabbing to a focusable element within the hidden content enables scrolling the focused element into view. The element box on which this value is set is a scroll container.

`clip`

> Overflow content is clipped at the element's *overflow clip edge* that is defined using the `overflow-clip-margin` property. As a result, content overflows the element's padding box by the `<length>` value of `overflow-clip-margin` or by `0px` if not set. Overflow content outside the clipped region is not visible, user agents do not add a scrollbar, and programmatic scrolling is also not supported. No new formatting context is created.

`scroll`

> Overflow content is clipped at the element's padding box, and overflow content can be scrolled into view using scrollbars. User agents display scrollbars in both horizontal and vertical directions if only one value is set, whether or not any content is overflowing or clipped. The use of this keyword value, therefore, can prevent scrollbars from appearing and disappearing as content changes. Printers may still print overflowing content. The element box on which this value is set is a scroll container.

`auto`

Overflow content is clipped at the element's padding box, and overflow content can be scrolled into view. Unlike `scroll`, user agents display scrollbars *only if* the content is overflowing and hide scrollbars by default. If content fits inside the element's padding box, it looks the same as with `visible` but still establishes a new formatting context. The element box on which this value is set is a scroll container.

> **Note:** The keyword value `overlay` is a legacy value alias for `auto`. With `overlay`, the scroll bars are drawn on top of the content instead of taking up space.

Examples

This example demos all the `<overflow>` enumerated values for the `overflow` property.

HTML

The HTML in this example contains some lyrics within the `<pre>` element. The HTML also contains a link text to enable testing the effects of keyboard focus on overflow and scroll behaviors. The same HTML code is repeated multiple times to show the effect of each `<overflow>` enumerated value.

```
<pre> 
Oh, Rubber Duckie, you're the one
You make bath time lots of fun
Rubber Duckie, I'm awfully fond of you

Rubber Duckie, joy of joys
When I squeeze you, you make noise
Rubber Duckie, you're my very best friend, it's true

Oh, every day when I make my way to the tubby
I find a little fella who's cute and yellow and chubby
Rub-a-dub-dubby

<a href="#">Rubber Duckie</a>, you're so fine
And I'm lucky that you're mine
Rubber Duckie, I'm awfully fond of you
</pre>
```

```
<pre> 
Oh, Rubber Duckie, you're the one
You make bath time lots of fun
Rubber Duckie, I'm awfully fond of you

Rubber Duckie, joy of joys
When I squeeze you, you make noise
Rubber Duckie, you're my very best friend, it's true

Oh, every day when I make my way to the tubby
I find a little fella who's cute and yellow and chubby
Rub-a-dub-dubby

<a href="#">Rubber Duckie</a>, you're so fine
And I'm lucky that you're mine
Rubber Duckie, I'm awfully fond of you
</pre>

<pre> 
Oh, Rubber Duckie, you're the one
You make bath time lots of fun
Rubber Duckie, I'm awfully fond of you

Rubber Duckie, joy of joys
When I squeeze you, you make noise
Rubber Duckie, you're my very best friend, it's true

Oh, every day when I make my way to the tubby
I find a little fella who's cute and yellow and chubby
Rub-a-dub-dubby

<a href="#">Rubber Duckie</a>, you're so fine
And I'm lucky that you're mine
Rubber Duckie, I'm awfully fond of you
</pre>

<pre> 
Oh, Rubber Duckie, you're the one
You make bath time lots of fun
Rubber Duckie, I'm awfully fond of you

Rubber Duckie, joy of joys
When I squeeze you, you make noise
Rubber Duckie, you're my very best friend, it's true

Oh, every day when I make my way to the tubby
I find a little fella who's cute and yellow and chubby
Rub-a-dub-dubby

<a href="#">Rubber Duckie</a>, you're so fine
And I'm lucky that you're mine
Rubber Duckie, I'm awfully fond of you
</pre>

<pre> 
Oh, Rubber Duckie, you're the one
You make bath time lots of fun
Rubber Duckie, I'm awfully fond of you

Rubber Duckie, joy of joys
When I squeeze you, you make noise
Rubber Duckie, you're my very best friend, it's true

Oh, every day when I make my way to the tubby
I find a little fella who's cute and yellow and chubby
Rub-a-dub-dubby

<a href="#">Rubber Duckie</a>, you're so fine
```

```
And I'm lucky that you're mine
Rubber Duckie, I'm awfully fond of you
</pre>

<pre> 
Oh, Rubber Duckie, you're the one
You make bath time lots of fun
Rubber Duckie, I'm awfully fond of you

Rubber Duckie, joy of joys
When I squeeze you, you make noise
Rubber Duckie, you're my very best friend, it's true

Oh, every day when I make my way to the tubby
I find a little fella who's cute and yellow and chubby
Rub-a-dub-dubby

<a href="#">Rubber Duckie</a>, you're so fine
And I'm lucky that you're mine
Rubber Duckie, I'm awfully fond of you
</pre>
```

CSS

For the purpose of demonstration, the size of the `<pre>` element box has been defined to ensure that the content overflows its container in both the inline and block directions. A different `<overflow>` value is set for each of the repeating `<pre>` elements. For the `clip` value demonstration, an `overflow-clip-margin` has been added.

```
pre {
  border: 2px dashed red;
  margin-bottom: 3em;
}

::before {
  font-weight: bold;
  color: white;
  background: crimson;
  display: inline-block;
  width: 100%;
  padding: 3px 5px;
  box-sizing: border-box;
}
```

```
pre {
  block-size: 100px;
  inline-size: 295px;
}

pre:nth-of-type(1) {
  overflow: hidden;
}
pre:nth-of-type(1)::before {
  content: "hidden: ";
}

pre:nth-of-type(2) {
  overflow: clip;
  overflow-clip-margin: 1em;
}
pre:nth-of-type(2)::before {
  content: "clip: ";
}

pre:nth-of-type(3) {
  overflow: scroll;
}
pre:nth-of-type(3)::before {
  content: "scroll: ";
}

pre:nth-of-type(4) {
  overflow: auto;
}
pre:nth-of-type(4)::before {
  content: "auto: ";
}

pre:nth-of-type(5) {
  overflow: clip;
  overflow: overlay;
  overflow-clip-margin: 3em;
}
pre:nth-of-type(5)::before {
  content: "overlay (or clip if not supported): ";
}

pre:nth-of-type(6) {
  overflow: visible;
}
pre:nth-of-type(6)::before {
  content: "visible: ";
}
```

Result

To see the effect of keyboard focus on overflow and scroll behaviors, try tabbing through all the links in the example. Notice that the `clip` box does not create a scroll container, and the link does not come into view when the link is focused. The `visible` value, which has the link always in view, is also not a scroll container.

See also

- Properties that use this data type: `overflow-x`, `overflow-y`, `overflow-inline`, `overflow-block` and `overflow`
- CSS overflow module

<overflow-position> CSS type

The **`<overflow-position>`** enumerated value type defines how an alignment subject that is larger than its alignment container will overflow that container. For example, if centered items are wider than their container, the overflow may be displayed beyond the viewport's start edge, which can't be scrolled to. The `<overflow-position>` value defines whether the alignment mode should be overridden to ensure the content is visible (`safe`) or if the alignment mode must be adhered to (`unsafe`).

This data type is valid for the `align-content`, `align-items`, `align-self`, `justify-items` and `justify-self` properties as well as the `place-content`, `place-items`, and `place-self` shorthand properties. If omitted, the default overflow alignment is a blend of `safe` and `unsafe`.

Syntax

```
<overflow-position> = unsafe | safe
```

Values

The following keyword values are represented by the `<overflow-position>` grammar term:

`safe`

If the size of the alignment subject overflows the alignment container, the alignment subject is instead aligned as if the alignment mode were `start`.

`unsafe`

Regardless of the relative sizes of the alignment subject and alignment container, the given alignment value is honored.

See also

- Properties that use this data type: `align-content`, `align-items`, `align-self`, `justify-content` `justify-items`, `justify-self`, `place-content`, `place-items`, and `place-self`
- Other box alignment data types: `<content-distribution>`, `<content-position>`, `<baseline-position>`, and `<self-position>`
- CSS box alignment module
- CSS flexible box layout module
- CSS grid layout module

param() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`param()`** CSS function is used to set link parameters. This can be done using the `link-parameters` CSS function, in the fragment URL of an external resource, or in the `<url-modifier>` of the `url()` CSS function.

Syntax

```
/* a single value */
param(--color, red);

/* multiple values */
param(--color1, red),
param(--color2, blue),
param(--color3, green);
```

Values

`<dashed-ident>`

A `<dashed-ident>` is a user-defined variable that is used as an identifier in the `env()` CSS function to update the value.

`<declaration_value>` Optional

A `<declaration_value>` is the value of the attribute being updated. If the `<declaration-value>` is omitted, it represents an empty value.

Formal syntax

```
<param()> =
  param( <param-spec> , <declaration-value>? )

<param-spec> =
  color                          |
  accent-color                   |
  [ <dashed-ident> <css-type>? ]

<css-type> =
  <syntax-component>  |
  <type()>

<syntax-component> =
  <syntax-single-component> <syntax-multiplier>?  |
  '<' transform-list '>'

<type()> =
  type( <string> )

<syntax-single-component> =
  '<' <syntax-type-name> '>'  |
  <ident>

<syntax-multiplier> =
  '#'  |
  '+'

<syntax-type-name> =
  angle              |
  color              |
  custom-ident       |
  image              |
  integer            |
  length             |
  length-percentage  |
  number             |
  percentage         |
  resolution         |
  string             |
```

```
  time               |
  url                |
  transform-function
```

This syntax reflects the latest standard as per CSS Functions and Mixins Module, CSS Images Module Level 4, CSS Linked Parameters Module Level 1, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

All of the following examples use the same SVG file, which has attributes set with `env()` CSS function.

```
<!-- example of the code in the external SVG file -->
<svg viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <rect
    width="100"
    height="100"
    stroke="env(--color1, chartreuse)"
    stroke-width="10"
    fill="env(--color2, darkgreen)"
   />
</svg>
```

Using <code>link-parameters</code> property

In this example the SVG attributes are updated with the `link-parameters` CSS property and `param()` function.

```
<div class="squares">
  <img
    class="original"
    src="square.svg"
    alt="A square with a chartreuse border and a dark green fill." />
  <img
    class="greyscale"
    src="square.svg"
    alt="A square with a slate grey border and a light grey fill." />
  <img
    class="high-contrast"
    src="square.svg"
    alt="A square with a fuchsia border and a yellow fill." />
</div>
```

```
.squares {
  height: 200px;
  display: flex;
  flex-direction: row;
  justify-content: space-between;
}
img {
  height: 100%;
}
```

```
.greyscale {
  link-parameters:
    param(--color1, slategrey),
    param(--color2, lightgrey);
  &:hover {
    link-parameters:
      param(--color1, lightgrey),
      param(--color2, slategrey);
  }
}
.high-contrast {
  link-parameters:
    param(--color1, fuchsia),
    param(--color2, yellow);
  &:hover {
    link-parameters:
      param(--color1, yellow),
      param(--color2, fuchsia);
  }
}
```

Passing <code>param()</code> into URL modifier

In this example the SVG attributes are updated by passing the `param()` function into the URL fragment of the `src` attribute of the `<img>` HTML element.

```
<img
  src="square.svg#param(--color1, slategrey)&param(--color2, lightgrey)"
  alt="A square with a slate grey border and a light grey fill."
/>
```

Using <code>param()</code> with <code>background-image</code> property

In this example the SVG attributes are updated by passing the `param()` function into the `url()` data type of the `background-image` CSS property.

```
.foo {
  background-image: url(
    "square.svg"
    param(--color1, slategrey)
    param(--color2, lightgrey)
  );
}
```

See also

- `link-parameters`
- `env()`

<percentage> CSS type

The **`<percentage>`** CSS data type represents a percentage value. It is often used to define a size as relative to an element's parent object. Numerous properties can use percentages, such as `width`, `height`, `margin`, `padding`, and `font-size`.

> **Note:** Only calculated values can be inherited. Thus, even if a percentage value is used on the parent property, a real value (such as a width in pixels for a `<length>` value) will be accessible on the inherited property, not the percentage value.

Syntax

The `<percentage>` data type consists of a `<number>` followed by the percentage sign (`%`). Optionally, it may be preceded by a single `+` or `-` sign, although negative values are not valid for all properties. As with all CSS dimensions, there is no space between the symbol and the number.

Interpolation

When animated, values of the `<percentage>` data type are interpolated as real, floating-point numbers. The speed of the interpolation is determined by the easing function associated with the animation.

Examples

Width and margin-left

```
<div class="container">
  <div class="box1">Width: 50%, Left margin: 20%</div>
  <div class="box2">Width: 30%, Left margin: 60%</div>
</div>
```

```
.container {
  background-color: navy;
}

.box1 {
  width: 50%;
  margin-left: 20%;
  background-color: chartreuse;
}

.box2 {
  width: 30%;
  margin-left: 60%;
  background-color: pink;
}
```

Font-size

```
<div class="container">
  <p>Full-size text (18px)</p>
  <p><span class="half">50% (9px)</span></p>
  <p><span class="double">200% (36px)</span></p>
</div>
```

```
.container {
  font-size: 18px;
}

.half {
  font-size: 50%;
}

.double {
  font-size: 200%;
}
```

See also

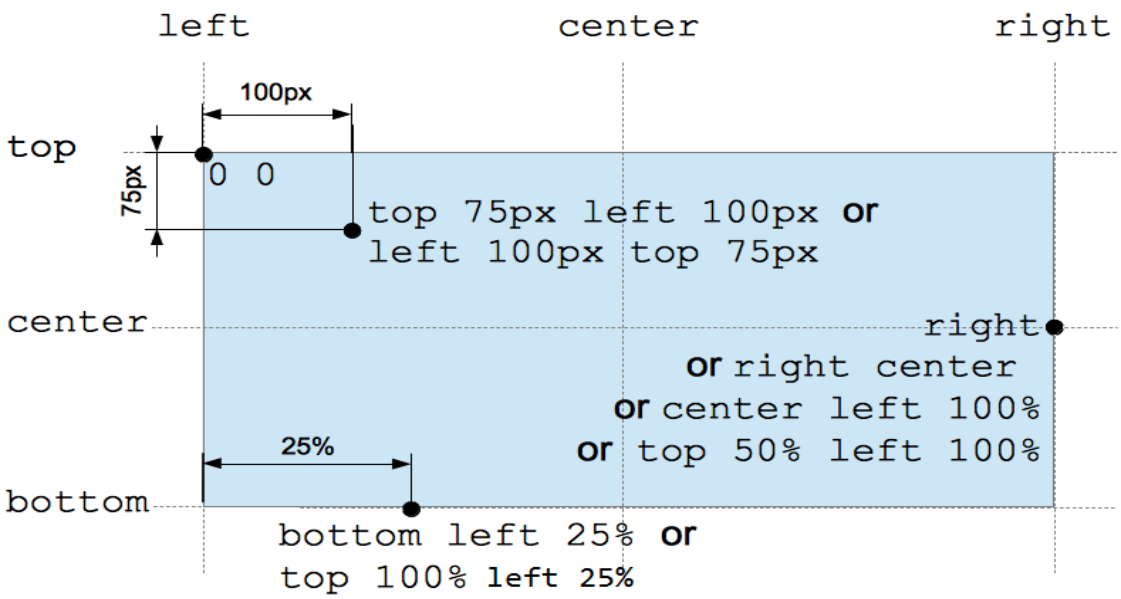
- `<length-percentage>`
- CSS values and units module

<position> CSS type

The **`<position>`** CSS data type denotes a two-dimensional coordinate used to set a location relative to an element box. It is used in the `background-position`, `object-position`, `mask-position` `offset-position`, `offset-anchor` and `transform-origin` properties.

> **Note:** The final position described by the `<position>` value does not need to be inside the element's box.

Syntax



The `<position>` data type is specified with one or two keywords, with optional offsets.

The keyword values are `center`, `top`, `right`, `bottom`, and `left`. Each keyword represents either an edge of the element's box or the center line between two edges. Depending on the context, `center` represents either the center between the left and right edges, or the center between the top and bottom edges.

If specified, an offset can be either a relative `<percentage>` value or an absolute `<length>` value. Positive values are offset towards the right or the bottom, whichever is appropriate. Negative values are offset in the opposite directions.

If only a single offset value is specified, it defines the x-coordinate, with the value for the other axis defaulting to `center`.

```
/* 1-value syntax */
keyword                 /* Either the horizontal or vertical position; the other axis defaults to center */
value                   /* The position on the x-axis; the y-axis defaults to 50% */

/* 2-value syntax */
keyword keyword         /* A keyword for each direction (the order is irrelevant) */
keyword value           /* A keyword for horizontal position, value for vertical position */
value keyword           /* A value for horizontal position, keyword for vertical position */
value value             /* A value for each direction (horizontal then vertical) */

/* 4-value syntax */
keyword value keyword value /* Each value is an offset from the keyword that precedes it */
```

> **Note:** The `background-position` property also accepts a three-value syntax. This is not allowed in other properties that use `<position>`.

Interpolation

When animated, a point's abscissa and ordinate values are interpolated independently. However, because the speed of the interpolation is determined by a single easing function for both coordinates, the point will move in a straight line.

Formal syntax

```
<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-one> =
  left             |
  center           |
  right            |
  top              |
  bottom           |
  x-start          |
  x-end            |
  y-start          |
  y-end            |
  block-start      |
  block-end        |
  inline-start     |
  inline-end       |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Valid positions

```
center
left
center top

right 8.5%
bottom 12vmin right -6px

10% 20%
8rem 14px
```

Invalid positions

```
left right
bottom top
10px 15px 20px 15px
```

See also

- CSS values and units module
- Learn: CSS Values and units
- `background-position`
- `radial-gradient()`
- `conic-gradient()`

<position-area> CSS type

The **`<position-area>`** CSS data type defines the cell or spanned cells of a **position-area grid**, a 3x3 grid whose center cell is an anchor element.

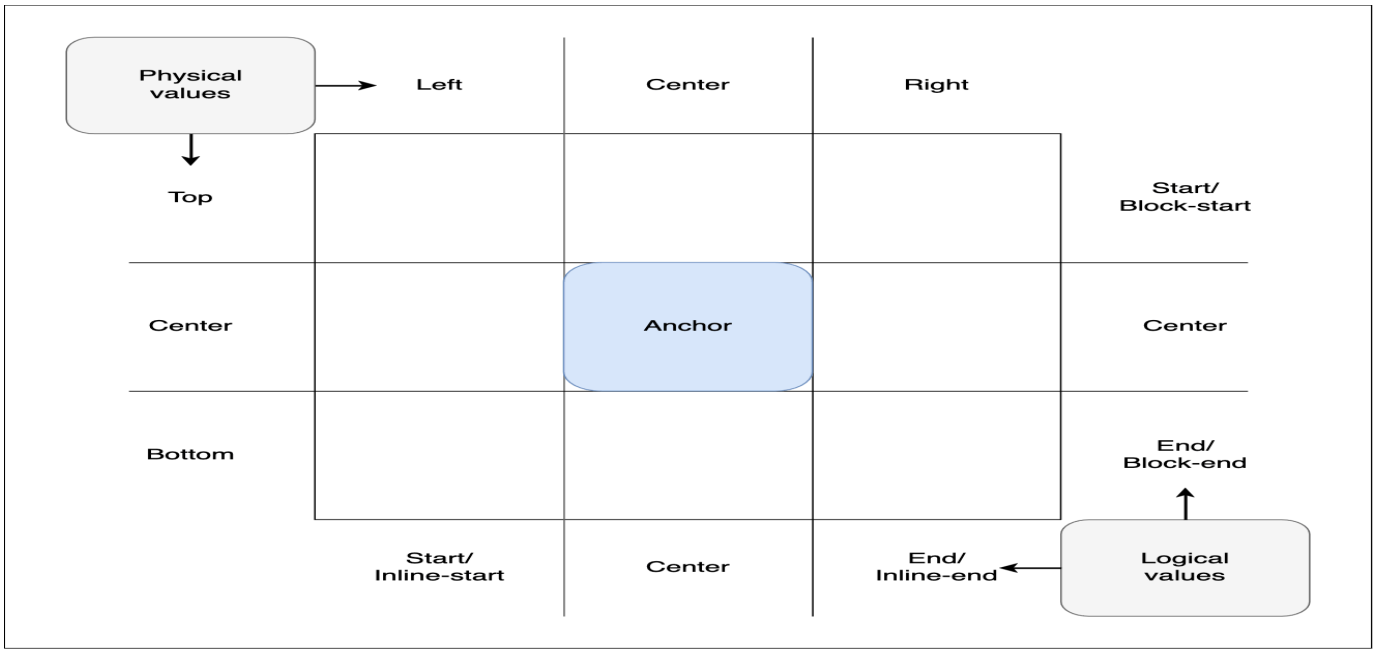
The `<position-area>` keyword values can be set as the value of the `position-area` property to place an anchor-positioned element in a specific location relative to its associated anchor element.

Syntax

```
<position-area> = [
  [ left | center | right | span-left | span-right | x-start | x-end | span-x-start | span-x-end | self-x-start | self-x-end | span-self-x-start | span-self-x-end | span-all ]
||
  [ top | center | bottom | span-top | span-bottom | y-start | y-end | span-y-start | span-y-end | self-y-start | self-y-end | span-self-y-start | span-self-y-end | span-all ]
|
  [ block-start | center | block-end | span-block-start | span-block-end | span-all ]
||
  [ inline-start | center | inline-end | span-inline-start | span-inline-end | span-all ]
|
  [ self-block-start | self-block-end | span-self-block-start | span-self-block-end | span-all ]
||
  [ self-inline-start | self-inline-end | span-self-inline-start | span-self-inline-end | span-all ]
|
  [ start | center | end | span-start | span-end | span-all ]{1,2}
|
  [ self-start | center | self-end | span-self-start | span-self-end | span-all ]{1,2}
]
```

Description

Position areas work on the concept of a **position-area grid**, a 3x3 grid of tiles composed of four grid lines, two on each axis, with an anchor element being the center tile:



When used as the value of a positioned element's `position-area` property, the dimensions of the center tile are defined by the containing block of the element's default anchor element. The dimensions of the grid's outer edge are defined by the positioned element's containing block. Logical keyterms are generally based on the writing mode and direction of the containing block, except for the `self-*` keyterms, which

are calculated from the writing mode of the anchor-positioned element.

The grid tiles are broken up into rows and columns:

- The three rows are represented by the physical values `top`, `center`, and `bottom`. They also have logical equivalents such as `block-start`, `center`, and `block-end`, and coordinate equivalents — `y-start`, `center`, and `y-end`.
- The three columns are represented by the physical values `left`, `center`, and `right`. They also have logical equivalents such as `inline-start`, `center`, and `inline-end`, and coordinate equivalents — `x-start`, `center`, and `x-end`.

`<position-area>` values contain one or two keywords defining a specific area of the position-area grid. Setting a `position-area` value on a positioned element places its containing block in the specified grid area:

```
/* Examples: Two keywords to place the element in a single specific tile */
position-area: top left;
position-area: bottom right;
position-area: start end;
position-area: center end;
position-area: block-start center;
position-area: inline-start block-end;
position-area: x-start y-end;
position-area: center self-y-end;

/* Examples: Two keywords to span the element across two tiles */
position-area: top span-left;
position-area: span-bottom right;
position-area: center span-start;
position-area: inline-start span-block-end;
position-area: y-start span-x-end;

/* Examples: Two keywords to span the element across three tiles */
position-area: top span-all;
position-area: block-end span-all;
position-area: self-x-start span-all;

/* Examples: One keyword with an implicit second keyword  */
position-area: top; /* equiv: top span-all */
position-area: inline-start; /* equiv: inline-start span-all */
position-area: center; /* equiv: center center */
position-area: span-all; /* equiv: span-all span-all */
position-area: start; /* equiv: start start */
position-area: end; /* equiv: end end */
```

The different types of keywords that can be used include:

- Physical grid keywords
- Generic logical row and column keywords
- Explicit inline and block logical keywords
- Coordinate grid keywords

> **Note:** Generally, you can't mix different types in one value, e.g., physical and logical. To do so results in invalid values. For example, `position-area: bottom inline-end` is not a valid value because it mixes physical and logical keywords.

Physical grid keywords

The physical grid keywords specify a cell or section of the `position-area` grid using physical values. These values are not affected by `writing-mode` or `direction` settings.

With physical row and column keywords, you can specify one keyword from each of the two lists below to select a single specific grid tile:

- `top`, `center`, or `bottom`: The top, center, or bottom row of the grid.
- `left`, `center`, or `right`: The left-hand, center, or right-hand column of the grid.

For example, `top left` selects the top left tile, while `center right` selects the center tile of the right-hand column.

Physical spanning grid keywords

The physical spanning keywords — when combined with a physical row or column keyword — specify a second grid tile for the position area to expand into. When such a combination is set as a `position-area` property value, a selected element is initially placed in the center of the specified row or column; it then spans in the direction specified in the spanning keyword, spanning two grid tiles:

`span-left`

Span the center column and the left-hand column of the grid.

`span-right`

Span the center column and the right-hand column of the grid.

`span-top`

Span the center row and the top row of the grid.

`span-bottom`

Span the center row and the bottom row of the grid.

`span-all`

Valid with all the keyword types, spans the cell listed as well as the adjacent cells in the same row or column. See `span-all` below.

For example, `top span-left` spans the top-center and top-left grid cells.

> **Note:** Trying to pair a row or column keyword with an inappropriate spanning keyword will result in an invalid value. For example, `right span-right` is invalid — you can't select the center-right grid tile and then try to span further to the right.

Physical grid keyword defaults

If only a single physical keyword is specified in the `position-area` value, the other value is implied as follows:

`left`, `right`, `top`, or `bottom`

The other value defaults to `span-all`, causing the element to span all three tiles of the column or row it was initially placed in. For example, `left` is equivalent to `left span-all`.

`span-left`, `span-right`, `span-top`, or `span-bottom`

The other value defaults to `span-all`. For example, `span-left` is equivalent to `span-left span-all`.

`center` or `span-all`

The other value defaults to the same as the first value. For example, `center` is equivalent to `center center` and `span-all` is equivalent to `span-all span-all`.

Logical grid keywords

The logical grid keywords specify an area of the position area grid using logical values. With these values, the position and direction are affected by `writing-mode` and `direction` settings on either the element's containing block or, in the case of the `self` keywords, the positioned element itself. There are two types of logical keywords; generic and explicit.

Generic logical row and column keywords

The generic logical keywords use the same terms for the inline and block directions, with the direction determined by the position of the keyterm within a pair of `<position-area>` values. The first value defines the block direction position and the second value defines the inline value. You can specify one or two keyterms from the following list. Specifying two from this list defines a single specific grid tile. The keyword position or direction is:

`start`

The start of the grid's block or inline direction, calculated from the containing block's writing mode.

`end`

The end of the grid's block or inline direction, calculated from the containing block's writing mode.

`self-start`

The start of the grid's block or inline direction, calculated from the element's own writing mode.

`self-end`

The end of the grid's block or inline direction, calculated from the element's own writing mode.

`center`

The center of the grid's block direction (if this keyword is specified first) or inline direction (if this keyword is specified second).

For example, `start end` and `self-start self-end` both describe the position at the start of the block direction and the end of the inline direction. With `writing-mode: horizontal-tb` set, this is the top right of the anchor element, whereas with `writing-mode: vertical-rl` it is the bottom right of the anchor.

Generic logical spanning row and column keywords

The generic logical spanning keywords — when combined with a logical row or column keyword — specify a second grid tile for the position area to expand into. When such a combination is set as a `position-area` property value, a selected element is initially placed in the center of the specified row or column, and it then spans in the direction specified in the spanning keyword, spanning two grid tiles:

`span-start`

Span the center tile and the start tile of the grid row/column, with the direction referring to the writing mode of the element's containing block.

`span-end`

Span the center tile and the end tile of the grid row/column, with the direction referring to the writing mode of the element's containing block.

`span-self-start`

Span the center tile and the start tile of the grid row/column for the positioned element's own writing mode.

`span-self-end`

Span the center tile and the end tile of the grid row/column, calculated from the element's own writing mode.

For example, `start span-end` and `self-start span-self-end` both specify a grid position area that starts in the center of the start block row, and spans across the tiles in that row that sit in the inline center and end columns. With `writing-mode: horizontal-tb` set, this would span over the top center and top right of the anchor, whereas with `writing-mode: vertical-rl` set it would span the element over the right center and bottom right.

Explicit inline and block logical keywords

The explicit inline and block logical row and column keywords refer explicitly to a block (row) or inline (column) position. You can specify one keyword for the block direction and one for the inline direction to select a single specific grid tile. Unlike with generic logical keyword values, the keyword order doesn't matter. However, declaring two keywords in the same axis invalidates the value.

`block-start`

The start of the grid's block direction calculated from the containing block's writing mode.

`block-end`

The end of the grid's block direction calculated from the containing block's writing mode.

`inline-start`

The start of the grid's inline direction calculated from the containing block's writing mode.

`inline-end`

The end of the grid's inline direction calculated from the containing block's writing mode.

For example, `block-start inline-end` specifies the tile at the start of the block direction and the end of the inline direction. With `writing-mode: horizontal-tb` set, this would be the tile at the top-right of the anchor, whereas with `writing-mode: vertical-rl` set this would be the tile at the bottom-right.

> **Note:** The specification defines `self` equivalents of these keywords — `block-self-start`, `block-self-end`, `inline-self-start`, and `inline-self-end`. However, these are not currently supported in any browser.

Explicit inline and block logical spanning keywords

The explicit logical spanning keywords — when combined with a logical row or column keyword — specify a second grid tile for the position area to expand into. When such a combination is set as a `position-area` property value, a selected element is initially placed in the center of the specified row or column, based on the containing block's writing mode, and it then spans in the direction specified in the spanning keyword, spanning two grid tiles:

`span-block-start`

Span the center tile and the block-start tile of the specified inline column.

`span-block-end`

Span the center tile and the block-end tile of the specified inline column.

`span-inline-start`

Span the center tile and the inline-start tile of the specified block row.

`span-inline-end`

Span the center tile and the inline-end tile of the specified block row.

For example, `block-end span-inline-start` selects the center tile of the end block row and spans across the tiles in that row that sit in the inline center and start columns. With `writing-mode: horizontal-tb` set, this would span the bottom-center and bottom-left grid tiles, whereas with `writing-mode: vertical-rl` set it would span the left-center and top-left grid tiles.

> **Note:** The specification defines `self` equivalents of these keywords, for example — `span-self-block-start`, `span-self-block-end`, `span-self-inline-start`, and `span-self-inline-end`. However, these are not currently supported in any browser.

> **Note:** Trying to pair a row or column keyword with an inappropriate spanning keyword will result in an invalid property value. For example, `block-end span-block-end` is invalid — you can't select the center block-end row and then try to span one tile further past the block end direction.

Logical grid keyword defaults

If only a single logical `<position-area>` keyword is specified, the other value is implied as follows:

`start`, `end`, `self-start`, or `self-end`

The other value defaults to the same as the first value, selecting the grid cell at the start row and column, or the end row and column.

`span-start`, `span-self-start`, `span-end`, `span-self-end`

The other value defaults to the same as the first value. For example, `span-start` is equivalent to `span-start span-start`.

`block-start`, `block-end`, `inline-start`, `inline-end`

The other value defaults to `span-all`, spanning all three tiles of the column or row set. For example, `block-start` is equivalent to `block-start span-all`.

`span-block-start`, `span-block-end`, `span-inline-start`, `span-inline-end`

The other value defaults to `span-all`. For example, `span-inline-start` is equivalent to `span-inline-start span-all`.

Coordinate grid keywords

These keywords specify the cells of the `position-area` grid using x- and y-coordinate values. Its position/direction will be affected by `writing-mode` and/or `direction` settings on either an element's containing block or, in the case of the `self` keywords, the element itself.

However, the grid cells are defined according to physical axes rather than block/inline directions:

- For `writing-mode: horizontal-tb` and `vertical-lr`, the x-axis runs left-to-right and the y-axis runs top-to-bottom.
- For `writing-mode: horizontal-tb; direction: rtl` and `writing-mode: vertical-rl`, the x-axis runs right-to-left and the y-axis runs top-to-bottom.

With coordinate row and column keywords, you can specify one keyword from the x-axis and one from the y-axis to define a single specific grid tile.

The x-axis keywords include:

`x-start`

The start tile along the grid's x-axis, calculated from the containing block's writing mode.

`x-end`

The end tile along the grid's x-axis, calculated from the containing block's writing mode.

`self-x-start`

The start tile along the grid's x-axis, calculated from the element's own writing mode.

`self-x-end`

The end tile along the grid's x-axis, calculated from the element's own writing mode.

`center`

The center of the grid's x-axis, calculated from the element's own writing mode.

The y-axis keywords include:

`y-start`

The start tile along the grid's y-axis, calculated from the containing block's writing mode.

`y-end`

The end tile along the grid's y-axis, calculated from the containing block's writing mode.

`self-y-start`

The start tile along the grid's y-axis, calculated from the element's own writing mode.

`self-y-end`

The end tile along the grid's y-axis, calculated from the element's own writing mode.

`center`

The center of the grid's y-axis, calculated from the element's own writing mode.

For example, `x-end y-start` and `self-x-end self-y-start` both select the grid cell at end of the x-axis and the start of the y-axis. With `writing-mode: horizontal-tb` set, this would be the cell to the top right of the anchor, whereas with `writing-mode: vertical-rl` is at the top left.

Coordinate spanning keywords

When combined with a coordinate row or column keyword, the coordinate-spanning keywords specify a second grid tile for the position area to expand into. When such a combination is set as a `position-area` property value, a selected element is initially placed in the center of the specified row or column, and it then spans in the direction specified in the spanning keyword, spanning two grid tiles:

`span-x-start`

: Span the center tile and the x-start tile of the specified y-axis row.

`span-x-end`

: Span the center tile and the x-end tile of the specified y-axis row.

`span-y-start`

: Span the center tile and the y-start tile of the specified x-axis column.

`span-y-end`

: Span the center tile and the y-end tile of the specified x-axis column.

For example, `y-end span-x-end` selects the tile at the center of the end y-row, and spans across the tiles in that row that sit in the x-center and x-end columns. With `writing-mode: horizontal-tb` set, the position grid area would span the grid tiles at the bottom-center and bottom-right, whereas with `writing-mode: vertical-rl` set it would span the bottom-center and bottom-left tiles.

> **Note:** The specification doesn't define separate coordinate `self` spanning keywords, but these are not needed — the spanning keywords can be used with both coordinate row and column keywords.

Coordinate grid keyword defaults

If only a single coordinate grid `<position-area>` keyword is specified, the other value is implied as follows:

`x-start`, `self-x-start`, `x-end`, `self-x-end`, `y-start`, `self-y-start`, `y-end`, or `self-y-end`

: The other value defaults to `span-all`, selecting the grid tiles spanning all three tiles of the column or row it was initially placed in. For example, `x-start` is equivalent to `x-start span-all`.

`span-x-start`, `span-x-end`, `span-y-start`, `span-y-end`, `span-self-x-start`, `span-self-x-end`, `span-self-y-end`, or `span-self-y-start`

: The other value defaults to `span-all`. For example, `span-x-start` is equivalent to `span-x-start span-all`.

<code>span-all</code>

`span-all` is a special keyword usable with all of the row and column keywords listed in the sections above. When you specify two values — a row/column keyword and `span-all`, the element is placed in the specified row or column, and it then spans all of the tiles in that row or column.

Examples

See the `position-area` property page.

For detailed information on anchor features and usage, see the CSS anchor positioning module and the Using CSS anchor positioning guide.

See also

- `position-area`
- `anchor-name`
- `position-anchor`
- `anchor()` function
- Using CSS anchor positioning guide
- Fallback options and conditional hiding for overflow guide
- CSS anchor positioning module

pow() CSS function

The **`pow()`** CSS function is an exponential function that returns the value of a base raised to the power of a number.

The `exp()` function is a special case of `pow()` where the value of the base is the mathematical constant e.

Syntax

```
/* A <number> value */
width: calc(10px * pow(5, 2)); /* 10px * 25 = 250px */
width: calc(10px * pow(5, 3)); /* 10px * 125 = 1250px */
width: calc(10px * pow(2, 10)); /* 10px * 1024 = 10240px */
```

Parameters

The `pow(base, number)` function accepts two comma-separated values as its parameters.

`base`

: A calculation that resolves to a `<number>`, representing the base.

`number`

: A calculation that resolves to a `<number>`, representing the exponent.

Return value

Returns a `<number>` representing $base^{number}$, that is, `base` raised to the power of `number`.

Formal syntax

```
<pow()> =
  pow( <calc-sum> , <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*
```

```
<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Scale headings by fixed ratio

The `pow()` function can be useful for strategies like CSS Modular Scale, which relates all the font-sizes on a page to each other by a fixed ratio.

HTML

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

CSS

```
h1 {
  font-size: calc(1rem * pow(1.5, 4));
}
h2 {
  font-size: calc(1rem * pow(1.5, 3));
}
h3 {
  font-size: calc(1rem * pow(1.5, 2));
}
h4 {
  font-size: calc(1rem * pow(1.5, 1));
}
h5 {
  font-size: calc(1rem * pow(1.5, 0));
}
h6 {
  font-size: calc(1rem * pow(1.5, -1));
}
```

Result

See also

- `sqrt()`
- `hypot()`
- `exp()`
- `log()`

progress() CSS function

The **`progress()`** CSS function returns a `<number>` value representing the position of one value (the progress value) relative to two other values (the progress start and end values).

Syntax

```
/* With fixed progress value */
progress(300, 0, 1000)
progress(50px, 0px, 100px)
progress(50%, 30%, 80%)

/* Unclamped return value */
progress(no-clamp 300, 0, 100)
progress(no-clamp 50px, 0px, 100px)

/* With custom property */
progress(var(--container-width), 320, 1200)

/* Inside math function */
calc((progress(var(--container-width), 20%, 80%) / 2) + 0.5)

/* Inside non-math function */
rgb(
  calc(255 * progress(var(--container-width), 320px, 1200px))
  calc(255 * progress(var(--container-width), 320px, 1200px)) 255 / 0.5
);

/* Math function inside progress() */
progress(calc(20 + 30), 0, 100)
```

Parameters

The `progress()` function takes an optional keyword followed by three comma-separated `<calc-sum>` expressions as parameters:

```
progress(no-clamp? <calc-sum>, <calc-sum>, <calc-sum>)
```

These are:

`no-clamp` Optional

A keyword that stops the return value from being clamped to the range `0` to `1`. It is written before the progress value, and is not followed by a comma. See Clamping the return value.

Progress

The value to calculate the position of relative to the other two values.

Progress start

The lower progress bound.

Progress end

The upper progress bound.

Return value

A `<number>` representing the position of the progress value relative to the other two values. This is calculated as follows:

```
(progress - progress start) / (progress end - progress start)
```

If the progress value is in between the progress start and progress end values, the return value will be between `0` and `1`, representing a percentage.

If the `no-clamp` keyword is included, then the return value can be any number. Otherwise, the return value is clamped to the range `0` to `1`.

Description

The CSS `progress()` function provides a way to calculate a progress ratio, which is useful for creating use cases such as progress bar animations, or boxes that fade in as they become wider to reveal their contents.

The simplest possible use might look like this:

```
opacity: progress(5, 0, 10);
```

In this case, the computed value of `opacity` would be `0.5`, as 5 is mid-way between `0` and `10`.

Clamping the return value

By default, the return value is clamped to the range `0` to `1`, so a progress value outside the progress start and end bounds returns the nearest bound:

```
/* Computes to 1, not 1.5 */
scale: progress(15, 0, 10);

/* Computes to 0, not -0.5 */
scale: progress(-5, 0, 10);
```

Including the `no-clamp` keyword before the progress value removes this restriction, so the function extrapolates beyond the bounds instead of stopping at them:

```
/* Computes to 1.5 */
scale: progress(no-clamp 15, 0, 10);

/* Computes to -0.5 */
scale: progress(no-clamp -5, 0, 10);
```

Note that `no-clamp` is a keyword rather than a separate parameter, so it is not followed by a comma.

Permitted unit types

The parameters of a `progress()` function can be mathematical expressions or simple values. The values (or expression results) can be any `<number>`, `<dimension>`, or `<percentage>` value. They can have different units, but they must all be of the same type, or else the function is invalid.

The example we saw previously is valid — all of its parameters are unitless `<number>` values:

```
progress(5, 0, 10)
```

The next example is also valid — all of its parameters have `<length>` units. Behind the scenes, the computed values are used for the calculation. Provided the `font-size` is `16px` at the point of calculation, `3em` will resolve to `48px`, which is `48%` of the way between `0px` and `100px`, so the return value will be `0.48`.

```
progress(3em, 0px, 100px)
```

The final couple of examples in this section is not valid, however. The types do not match, so the resulting calculations do not make sense.

```
progress(3s, 0px, 100px)
progress(3em, 0, 100)
```

Creating unitless values

The `progress()` function outputs unitless values, therefore it can be used for the purpose of removing units from values in the same manner as the `tan(atan2())` hack. Note however that, due to the updates in behavior around CSS typed arithmetic, this can also be achieved via simple division.

Combining <code>progress()</code> with other functions and custom properties

Because `progress()` only ever returns a unitless value, it is common to combine it with another math function such as `calc()` to output the value and units you want. You can also use CSS custom properties inside `progress()` functions — this makes sense, as you'll often want to set the same values in multiple places, and/or base them on custom properties set via JavaScript.

The following example calculates what percentage the viewport width is between a minimum width of `320px` and a maximum width of `1200px`. The `calc()` function is used to multiply the `progress()` return value by `600px` to convert it into a pixel value that will half of the viewport width's progress value between `320px` and `1200px`.

```
width: calc(progress(100vw, 320px, 1200px) * 600px);
```

For example, if the viewport width is `700px`, the progress value will be calculated as `((700 - 320) / (1200 - 320))` = `0.431818`. The width will then be calculated as `0.431818 * 600px`, which equals `259.1px`.

The next example is an update of the previous one, in which we've used custom properties for the progress, progress start, and progress end values.

```
width: calc(
  progress(
      var(--container-width),
      var(--custom-min-width),
      var(--custom-max-width)
    ) *
    var(--custom-max-width)
);
```

It is possible to use `progress()` functions to calculate individual values inside other functions and component values inside shorthand property values, provided your functions return valid types for those values.

This can result in some complex expressions. For example, here we are calculating the first two channels of an `rgb()` color proportionate to the same width proportion as before:

```
background-color: rgb(
  calc(
      255 *
        progress(
          var(--container-width),
          var(--custom-min-width),
          var(--custom-max-width)
        )
    )
    calc(
      255 *
        progress(
          var(--container-width),
          var(--custom-min-width),
          var(--custom-max-width)
        )
    )
    255 / 0.5
);
```

Formal syntax

```
<progress()> =
  progress( no-clamp? <calc-sum> , <calc-sum> , <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic <code>progress()</code> usage

In this example, we'll show basic usage of the `progress()` function to set a progress bar `width` as a percentage equal to the progress ratio of its parent's `width` between its `min-width` and `max-width`.

HTML

Our HTML features a `<section>` element representing our content, and a `<div>` element representing the width progress bar.

```
<section>
  <div class="progress"></div>
</section>
```

CSS

In our CSS, we first set some custom properties on our `<section>` element to represent its `min-width`, `max-width`, and `width`. We then set those properties to the appropriate custom property values, then give our `<section>` a solid `background-color` so it is visible.

```
html {
  height: 100%;
  font-family: sans-serif;
}

body,
section {
  height: inherit;
}
```

```
section {
  --custom-min-width: 300px;
  --custom-max-width: 700px;
  --custom-width: 600px;

  min-width: var(--custom-min-width);
  max-width: var(--custom-max-width);
  width: var(--custom-width);

  background-color: cyan;
}
```

Now onto our `<div>` — we first give it a `height` and a dark `background-color` so it stands out against our `<section>` element. We then calculate its `width` by using a `progress()` function to calculate the progress ratio of the width between the minimum and maximum width, then using a `calc()` function to multiply the `progress()` return value by `100%` to return a percentage.

```
.progress {
  height: 4px;
  background-color: red;

  width: calc(
    progress(
        var(--custom-width),
        var(--custom-min-width),
        var(--custom-max-width)
      ) *
      100%
  );
}
```

Result

This demo will render as follows:

The width of the `<div>` is `75%` of the `<section>` width, as the `min-width` is `400px`, the `max-width` is `700px`, and the `width` is `600px`, which is `75%` of the distance between the previous two values.

Resize effects on a container

This example shows off some more involved uses of the `progress()` function, resulting in some fun effects as the browser window is resized.

This example works much better when rendered full-size in a desktop browser tab. Therefore, we have not rendered it in an embedded live sample in this page. Instead, you can find it running live at CSS `progress()` function demo (also see the source code).

Open the live example in a separate tab, and try increasing and decreasing the browser window width to see the effect. Keep this open so you can refer back to it as you read the explanation below.

HTML

Our HTML features an `<article>` element that contains the rest of our content, and two `<section>` elements — one to hang a background image off, and the other one to contain our content. The `<section class="content">` also contains a `<div class="progress">` representing a width progress bar, the same as the one in our previous demo. We have omitted the rest of the content for brevity.

```
<article>
  <section class="background"></section>
  <section class="content">
    <div class="progress"></div>
    <!-- Content here -->
  </section>
</article>
```

JavaScript

In our script, we first grab a reference to our `<article>` element. We then define a function called `setContainerWidth()`, which grabs the `<article>`'s client width via `Element.getBoundingClientRect()`, and sets a custom property on it called `--container-width`, which is equal to the client width rounded down, with `px` appended to it.

We then set a `resize` event listener on the `Window` object, which runs `setContainerWidth()` when the browser window is resized. We also run it once to set the `--container-width` custom property on the `<article>` element once the page loads.

```
const articleElem = document.querySelector("article");

function setContainerWidth() {
  const clientWidth = articleElem.getBoundingClientRect().width;
  articleElem.style.setProperty(
    "--container-width",
    `${Math.floor(clientWidth)}px`,
  );
}

window.addEventListener("resize", setContainerWidth);

setContainerWidth();
```

With this set up, we can now set some property values based on the `--container-width`, so that parts of our design will dynamically change as the window resizes.

CSS

The following section explains just the CSS relevant to how we've used the `progress()` function in the demo. For the full CSS, see the CSS source.

We first center the `<article>` inside the `<body>` using flexbox, then set some custom properties on it to represent the `min-width` and `max-width` values we will use elsewhere. We then target the `<article>` element, giving it `min-width` and `max-width` values equal to the custom properties we set earlier. We set its `position` to `relative` so we can position its content relative to it, then give it a percentage `width`, fixed `height`, and `border`.

```
body {
  height: inherit;
  display: flex;
  justify-content: center;
  align-items: center;
  --custom-min-width: 320px;
  --custom-max-width: 1200px;
}

article {
  min-width: var(--custom-min-width);
  max-width: var(--custom-max-width);
  position: relative;
  width: 70%;
  height: 600px;
  border: 3px solid black;
}
```

Now on to our `progress` `<div>`. We set its `width` equal to a percentage based on the progress ratio of the `--container-width` custom property set on the `<article>` element via JavaScript between its `min-width` and `max-width` (we use the same custom properties here for the second and third `progress()` parameters as we did for the `<article>` `min-width` and `max-width`).

We also give it a `height` and `background-color`, then absolutely position it at the top and left corner of the `<article>`.

```
.progress {
  width: calc(
    progress(
        var(--container-width),
        var(--custom-min-width),
        var(--custom-max-width)
      ) *
      100%
  );
  height: 4px;
  background-color: red;
  position: absolute;
  top: 0;
  left: 0;
}
```

Next, we'll look at our `background` `<section>`. We absolutely position it relative to our `<article>`, setting `inset: 0` on it so that it adopts the same size and sits over the top of it. We then set a rather wide `background-image` on it, and position the background image by giving the `background-position-x` property the same value as we gave the progress bar's `width` property. This has the effect that, when you increase the browser window width, the background image is moved over to the left, creating a nice image scrolling effect.

```
.background {
  position: absolute;
  inset: 0;
  background-image: url("https://mdn.github.io/shared-assets/images/examples/wide-background.jpg");
  background-position-x: calc(
    progress(
        var(--container-width),
        var(--custom-min-width),
        var(--custom-max-width)
      ) *
      100%
  );
}
```

We position the `content` `<section>` absolutely so it sits over the top of the `background` `<section>`, then give it some `padding`. We then vary two property values as the browser window is resized, using the same progress ratio as before:

- We vary the R and G components of the `background-color`, multiplying the progress ratio by 255 in each case to get a proportionate channel value. As the window gets wider, the background color gets less blue and more white, making the scene look as it is going from night to day (the color value has an opacity of `0.5`, so it acts like a tint for the underlying image).
- We vary the `opacity` so that the content fades in a bit when the window gets wider.

```
.content {
  position: absolute;
  inset: 0;
  padding: 20px;
  background-color: rgb(
    calc(
        255 *
          progress(
            var(--container-width),
            var(--custom-min-width),
            var(--custom-max-width)
          )
      )
      calc(
        255 *
          progress(
            var(--container-width),
            var(--custom-min-width),
            var(--custom-max-width)
          )
      )
      255 / 0.5
  );
  opacity: calc(
    (
        progress(
            var(--container-width),
            var(--custom-min-width),
            var(--custom-max-width)
          ) /
          2
      ) +
      0.5
  );
}
```

See also

- CSS values and units module
- Container style queries
- Media queries
- Feature queries

random() CSS function

The **`random()`** CSS function generates a random value within a specified range, optionally limiting the possible values to step size intervals between those limits. It can be used when specifying a `<length>`, `<frequency>`, `<angle>`, `<time>`, `<resolution>`, `<percentage>`, `<number>`, or `<integer>` within a property value.

Try it

```
<div class="box"></div>
```

```
.box {
  rotate: random(property-scoped, 0deg, 360deg);
  width: random(property-scoped, 50px, 300px);
  background-color: hsl(random(property-scoped, 0, 360) 50% 50%);
  height: random(property-scoped, 50px, 300px);
```

```
}

@supports not (order: random(1, 2)) {
  body::before {
    content: "Your browser doesn't support the random() function.";
  }
}
```

Syntax

```
/* Basic usage */
random(0, 100)
random(10px, 500px)
random(0deg, 360deg)

/* With step interval */
random(0, 100, 10)
random(0rad, 1turn, 30deg)

/* With <random-key> */
random(auto, 0, 360)
random(element-scoped, 0s, 5s)
random(property-scoped, 0s, 5s)
random(property-index-scoped, 0s, 5s)
random(--unique-base, 400px, 100px)
random(fixed 0.5, 1em, 40vw)
random(--unique-base property-scoped, 100dpi, 300dpi)

/* With <random-key> and step interval */
random(property-scoped, 0deg, 360deg, 45deg)
random(--my-base, 1em, 3rem, 2px)
```

Parameters

The `random(seed, min, max, step)` function accepts two to four comma-separated expressions as its parameters.

`<random-key>` Optional

Controls which `random()` functions in the document will share a random base value, or *seed*, and which will get distinct values. Specified as one of the following values:

`auto`

The `auto` value is the default and is what is applied if the `<random-key>` is excluded. This random function generates independent random values. The random cache name, and thus the result, varies across every `random()` instance in a multi-component value, across different properties, and across different elements. This is equivalent to specifying `element-scoped property-index-scoped`.

`element-scoped`

Adds an element-specific identifier to the random cache name, so if applied over multiple properties (e.g., `width` and `height`) both properties use the same value, but each element (e.g., `<div>`) gets a different random values.

`property-scoped`

Adds the property name to the random cache name, so if applied over multiple properties (e.g., `width` and `height`) both properties gets a different random value, but each element (e.g., `<div>`) uses the same random values.

`property-index-scoped`

Adds the property name and the index of the `random()` function among all random functions used in the same property value to the random cache name, so multiple instances in the same declaration each get different random values.

`<dashed-ident>`

A custom name for the random cache key (e.g., `--my-random-key`). When used alone, the same random base value is shared across all elements and properties that use the same identifier. When combined with a `*-scoped` keyword, the sharing behavior is determined by the keyword.

`fixed <number>`

Bypasses the random cache name and uses the `<number>` — between `0` and `1`, inclusive of 0, exclusive of 1 — as the seed's value.

`<calc-sum>, <calc-sum>`

Specified as `<number>`, `<dimension>`, or `<percentage>` values, or calculations resolving to one of these types, defining the minimum and maximum values, respectively. Both values must be resolvable to the same data type. If the maximum is less than the minimum, the function will return the first `<calc-sum>` value.

`<calc-sum>` Optional

Specifies the step interval. If present and of the same data type as the minimum and maximum `<calc-sum>` values, it defines the return value to be the minimum value or at increments of the step value from the minimum value, up to the maximum value.

Return value

Returns a random `<number>`, `<dimension>`, or `<percentage>` between the minimum and maximum range values, inclusive, in the same type as the `<calc-sum>` parameters.

Description

The `random(SEED, MIN, MAX, STEP)` function specifies the minimum and maximum values and optional step increments, starting at the minimum value. The function generates a random result within the range specified. The seed, an optional `<random-key>` parameter, enables sharing or varying random base values across different properties and elements.

The minimum, maximum and step values specified must be of the same data type for the function to be valid. While the units in the two to three `<calc-sum>` parameters don't need to be the same, they do need to be of the same data type, such as `<number>`, `<percentage>`, `<length>`, `<angle>`, `<time>`, or `<frequency>`, to be valid.

Random base value

The random base value works like a seed for randomness. It's a starting number that is used to generate the final random result. When two `random()` functions share the same base value, their results vary together in a predictable pattern. When they have different base values, their results are completely independent of each other.

The optional first `<random-key>` parameter controls how the random base value is shared. It can be `auto`, a scope keyword (`element-scoped`, `property-scoped`, or `property-index-scoped`), a custom `<dashed-ident>`, `fixed <number>`, or a `<dashed-ident>` paired with a scope keyword.

Scope keywords

Used on their own, the scope keywords control sharing without a custom name:

- `property-scoped` shares a base value across all elements, per property.
- `element-scoped` gives each element its own base value, per property.
- `property-index-scoped` works like `property-scoped`, but also separates `random()` calls by position within a shorthand.

The `property-scoped` keyword makes `.a`, `.b`, and `.c` identical rectangles, since every element gets the same `width` value and the same `height` value:

```
.a,
.b,
.c {
  width: random(property-scoped, 10px, 200px);
  height: random(property-scoped, 10px, 200px);
}
```

The `element-scoped` values would give each element its own independently sized `width` value and `height` value.

The `property-index-scoped` keyword is useful in shorthands, where each position needs its own shared value:

```
.a,
.b,
.c {
  margin: random(property-index-scoped, 5px, 40px)
    random(property-index-scoped, 5px, 40px)
    random(property-index-scoped, 5px, 40px)
    random(property-index-scoped, 5px, 40px);
}
```

All three elements get the same top margin, the same right margin, and so on, but the four margin values differ from each other.

Custom names

A `<dashed-ident>` (e.g., `--custom-name`) used alone shares its base value globally: any `random()` call anywhere in the document with the same identifier gets the same result. This makes `.a`, `.b`, and `.c` identical squares, since every `width` and `height` resolves to the same value:

```
.a,
.b,
.c {
  width: random(--custom-name, 10px, 200px);
  height: random(--custom-name, 10px, 200px);
}
```

Pair a `<dashed-ident>` with a scope keyword to narrow that sharing. `property-scoped` keeps it global across elements but separates it by property:

```
.a,
.b,
.c {
  width: random(--custom-name property-scoped, 10px, 200px);
  height: random(--custom-name property-scoped, 10px, 200px);
}
```

`element-scoped` instead keeps `width` and `height` linked within each element, but gives each element its own value, so `.a`, `.b`, and `.c` become squares of different sizes:

```
.a,
.b,
.c {
  width: random(--custom-name element-shared, 10px, 200px);
  height: random(--custom-name element-shared, 10px, 200px);
}
```

Automatic behavior

When the first parameter is omitted, or explicitly set to `auto`, an ident is auto-generated from the property name and position. This behavior can cause some unexpected random base value sharing.

```
.foo {
  width: random(100px, 200px);
}
.foo:hover {
  width: random(100px, 200px);
}
.bar {
  margin: random(1px, 100px) random(1px, 100px);
}
.bar:hover {
  margin: random(1px, 100px) random(1px, 100px) random(1px, 100px)
    random(1px, 100px);
}
```

When the `<random-key>` defaults or is explicitly set to `auto`, the user agent auto-generates a seed name, or *generated value sharing identifier*, following consistent rules based on property name and order. Because of this, `random()` functions can end up with the same seed name and, therefore, the same random base value. In this example, the generated value sharing identifier for the `random()` function in the `width` property value is the same for `.foo` as for `.foo:hover`, so the value won't change between states. Similarly, the first two `random()` functions in both `margin` declarations have the same generated value sharing identifier, meaning the first two values in the `margin` shorthand will be unchanged when hovered; on hover, `bar`'s top and right margins will remain the same, but the bottom and left margins will get independent random values. To get an independent value for each `random()` function, provide a unique `<dashed-ident>`.

Custom properties

As with all CSS functions, when a `random()` function is included within a custom property value, the value remains a function; behaving like a text replacement mechanism, not storing a single return value.

```
--random-size: random(1px, 100px);
```

In this example, the `--random-size` custom property does not "store" the randomly generated result. When `var(--random-size)` is parsed it is effectively replaced with `random(1px, 100px)`, meaning each use creates a new `random()` function call with its own base value depending on the context in which its used.

This is not true in the case of using `random()` when registering a custom property with `@property`. Registered custom properties compute random values and store them.

In this example, as `--defaultSize` is registered, `.a`, `.b`, and `.c` will be squares of equal size, but their colors will be random, as `--random-angle` was not registered:

```
@property --defaultSize {
  syntax: "<length> | <percentage>";
  inherits: true;
  initial-value: random(100px, 200px);
}
:root {
  --random-angle: random(0deg, 360deg);
}
.a,
.b,
.c {
  background-color: hsl(var(--random-angle) 100% 50%);
  height: var(--defaultSize);
  width: var(--defaultSize);
}
```

Accessibility

Because `random()` can generate an unknown value within a range, you don't have full control over what you get. This can lead to inaccessible results. For example, if you use `random()` to generate text color, you could end up with a value that has low contrast with its background. It is important to be mindful of the context in which `random()` is used, ensuring your results are always accessible.

Formal syntax

```
<random()> =
  random( <random-key>? , <calc-sum> , <calc-sum> , <calc-sum>? )

<random-key> =
  auto                  |
  <random-cache-key>    |
  fixed <number [0,1]>

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<random-cache-key> =
  <dashed-ident>                                  ||
  element-scoped                                  ||
  [ property-scoped | property-index-scoped | <random-ua-ident> ]

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<random-ua-ident> =
  <custom-ident>

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic Usage

In this example, we'll generate random colors for some circular badges to demonstrate the basic usage of the `random()` function.

HTML

We include five badges, one using the `desaturated` class and two using the `unique` class.

```
<div class="badge"></div>
<div class="badge"></div>
<div class="badge desaturated"></div>
<div class="badge unique"></div>
<div class="badge unique"></div>
```

CSS

We render the five badges as circles. We use the `random()` function within an `hsl()` color function to define the `<angle>` of the `<hue>`. We set `property-scoped` to share the random base value between the default `badge` and the `desaturated` one, so it is a less saturated version of the same `<hue>`. We then override the `unique` badges to have a truly random `hue` by letting the base value sharing parameter default to `auto`.

```
.badge {
  display: inline-block;
  width: 5em;
  aspect-ratio: 1/1;
  border-radius: 50%;
  background: hsl(random(property-scoped, 0, 360) 50% 50%);
}
.badge.desaturated {
  background: hsl(random(property-scoped, 0, 360) 10% 50%);
}
.badge.unique {
  background: hsl(random(0, 360) 50% 50%);
}
```

```
@supports not (order: random(1, 2)) {
  body::before {
    content: "Your browser doesn't support the random() function.";
```

```
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

Random value sharing between properties

In this example, we create a starry backdrop to demonstrate combining a `<dashed-ident>` with `element-scoped` to share a random base value between properties within each individual element, without sharing it across elements.

HTML

We include five particles, all sharing the same class name.

```
<div class="particle"></div>
<div class="particle"></div>
<div class="particle"></div>
<div class="particle"></div>
<div class="particle"></div>
```

CSS

Every particle has the same styles. We use the `random()` function for the `height`, `width`, `top`, and `left` values to randomly size and position each particle. We pair a `<dashed-ident>` with `element-scoped` for `height` and `width`, so each particle is a circle (its height matches its width) but sized independently of the others. `top` and `left` are left at the default `auto`, so each axis is positioned independently.

```
body {
  background: black;
}

.particle {
  border-radius: 50%;
  background: white;
  position: fixed;
  width: random(--particle-size element-scoped, 0.25em, 1em);
  height: random(--particle-size element-scoped, 0.25em, 1em);
  top: random(0%, 100%);
  left: random(0%, 100%);
  animation: move 1s alternate-reverse infinite;
}
```

```
@supports not (order: random(1, 2)) {
  body::before {
    content: "Your browser doesn't support the random() function.";
    color: white;
    display: block;
    text-align: center;
    padding: 1rem 0;
  }
}
```

Result

See also

- `calc()`
- CSS units and values module
- `Math.random()`
- Rolling the Dice with CSS random() via webkit.org (2025)
- CSS Almanac: random() via CSS-Tricks.com

<ratio> CSS type

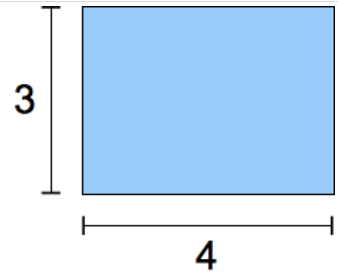
The **`<ratio>`** CSS data type describes the proportional relationship between two values. It mostly represents the aspect ratio, which relates width to height. For example, the `<ratio>` is used as a value for the `aspect-ratio` media feature in `@media` media queries, the `aspect-ratio` size feature in `@container` container queries, and as a value for the CSS `aspect-ratio` property.

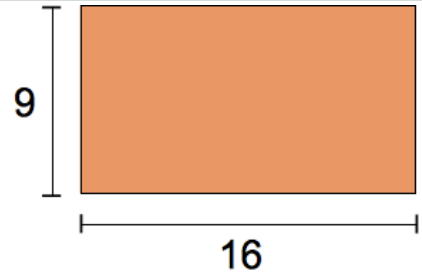
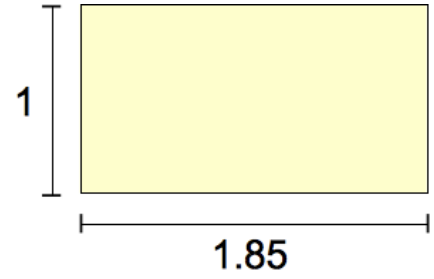
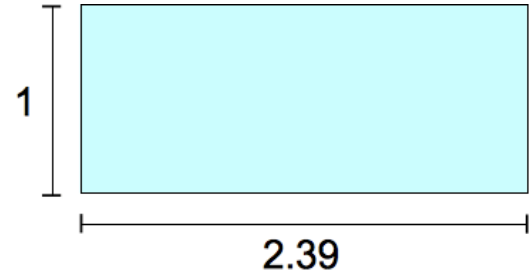
Syntax

The `<ratio>` data type is a `<number>` followed by a forward slash ('/', Unicode `U+002F SOLIDUS`) and a second `<number>`. Both numbers must be positive. Spaces before and after the slash are optional. The first number represents the width, while the second represents the height. In addition a single `<number>` as a value is allowable.

Two ratios are compared using the quotients' numeric values. For example, 16/16 is less than 16/9 because it resolves to 1 while the second resolves to 1.7. This means a tall screen's aspect ratio is smaller than a wide screen's, and portrait images have smaller aspect ratios than landscape images.

Common aspect ratios

Ratio		Usage
`4/3` or `1.33333`		Traditional TV format in the twentieth century.

Ratio		Usage
`16/9` or `1.7777778`		Modern "widescreen" TV format.
`185/100` or `1.85`		The most common movie format since the 1960s.
`239/100` or `2.39`		"Widescreen," anamorphic movie format.

Formal syntax

```
<ratio> =
  <number [0,∞]> [ / <number [0,∞]> ]?
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Use in a media query

```
@media screen and (aspect-ratio >= 16/9) {
  /* … */
}
```

Use in a @container size query

```
@container (aspect-ratio > 1) and (width < 20em) {
  /* … */
}
```

Use as a CSS property value

```
.square {
  aspect-ratio: 1 / 1;
}
.circle {
  aspect-ratio: 1;
  border-radius: 50%;
}
.portrait {
  aspect-ratio: 5 / 7;
}
```

See also

- `aspect-ratio` media descriptor
- Understanding aspect ratios
- CSS container queries guide
- Using container size and style queries guide
- CSS media queries module
- CSS containment module
- CSS box sizing module
- CSS values and units module

ray() CSS function

The **`ray()`** CSS function defines the `offset-path` line segment that an animated element can follow. The line segment is referred to as "ray". The ray begins from an `offset-position` and extends in the direction of the specified angle. The length of a ray can be constrained by specifying a size and using the `contain` keyword.

Syntax

```
/* all parameters specified */
offset-path: ray(50deg closest-corner contain at 100px 20px);

/* two parameters specified, order does not matter */
offset-path: ray(contain 200deg);

/* only one parameter specified */
offset-path: ray(45deg);
```

Parameters

The parameters can be specified in any order.

`<angle>`

Specifies the direction in which the line segment extends from the offset starting position. The angle `0deg` lies on the y-axis pointing up, and positive angles increase in the clockwise direction.

`<size>`

Specifies the length of the line segment, which is the distance between `offset-distance` `0%` and `100%`, relative to the containing box. This is an optional parameter (`closest-side` is used if no `<size>` is specified). It accepts one of the following keyword values:

`closest-side`: Distance between the ray's starting point and the closest side of the containing block of the element. If the ray's starting point lies on an edge of the containing block, the length of the line segment is zero. If the ray's starting point is outside the containing block, the edge of the containing block is considered to extend to infinity. This is the default value.

`closest-corner`: Distance between the ray's starting point and the closest corner in the element's containing block. If the ray's starting point lies on a corner of the containing block, the length of the line segment is zero.

`farthest-side`: Distance between the ray's starting point and the farthest side of the containing block of the element. If the ray's starting point is outside the containing block, the edge of the containing block is considered to extend to infinity.

`farthest-corner`: Distance between the ray's starting point and the farthest corner in the element's containing block.

`sides`: Distance between the ray's starting point and the point where the line segment intersects the containing block's boundary. If the starting point is on or outside the containing block's boundary, the length of the line segment is zero.

`contain`

Reduces the length of the line segment so that the element stays within the containing block even at `offset-distance: 100%`. Specifically, the segment's length is reduced by half the width or half the height of the element's border box, whichever is greater, and never less than zero. This is an optional parameter.

`at <position>`

Specifies the point where the ray begins and where the element is placed in its containing block. This is an optional parameter. If included, the `<position>` value must be preceded by the `at` keyword. If omitted, the value used is the `offset-position` value of the element. If omitted and the element doesn't have an `offset-position` value, the value used for the ray's starting position is `offset-position: normal`, which places the element at the center (or `50% 50%`) of the containing block.

Description

The `ray()` function positions an element along a path by specifying its location in a two-dimensional space through an angle and a distance from a reference point (polar coordinates). This feature makes the `ray()` function useful for creating 2D spatial transitions. For comparison, this approach differs from the method of specifying a point by its horizontal and vertical distances from a fixed origin (rectangular coordinates), which is used by the `translate()` function, and from moving an element along a defined path through animation.

As `ray()` works in 2D space, it's important to consider both the initial position and orientation of the element. When the `ray()` function is applied as the `offset-path` value on an element, here's how you can control these aspects:

- The element is initially positioned by moving the element's `offset-anchor` point to the element's offset starting position. By default, the ray's starting position is determined by the `offset-position` value. If `offset-position` is explicitly specified as `normal` (or omitted and allowed to default to `normal`), the element is positioned at the `center` (or `50% 50%`) of its containing block. Specifying `offset-position: auto` sets the starting position at the `top left` corner (or `0 0`) of the element's position.
- The element is initially rotated such that its inline axis — its direction of text flow — aligns with the angle specified by `ray()`. For example, with the `ray()` angle of `0deg`, which lies on the y-axis pointing up, the element's inline axis is rotated to be vertical to match the ray's angle. The element maintains this rotation throughout its path. To customize this behavior, use the `offset-rotate` property, which allows you to specify a different rotation angle or direction for the element, enabling more precise control over its appearance as it follows the path. For example, setting `offset-rotate: 0deg` will remove any rotation applied by `ray()`, aligning back the element's inline axis with the direction of text flow.

Formal syntax

```
<ray()> =
  ray( <angle>          &&
  <ray-size>?           &&
  contain?              &&
  [ at <position> ]? )

<ray-size> =
  <radial-extent>  |
  sides

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<position-one> =
  left               |
  center             |
  right              |
  top                |
  bottom             |
  x-start            |
  x-end              |
  y-start            |
  y-end              |
  block-start        |
  block-end          |
  inline-start       |
```

```
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Images Module Level 3, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Motion Path Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Defining the angle and starting position for a ray

This example shows how to work with an element's starting position and how the element's orientation is impacted by the specified ray angle.

CSS

```
body {
  width: fit-content;
  height: fit-content;
}

.container {
  width: 80vw;
  height: 100px;
  border: 1px dashed black;
  margin: 0 0.5em 2em 2em;
  text-align: center;
}

pre {
  font-size: 1em;
  text-align: right;
  padding-right: 10px;
  line-height: 1em;
}

.box {
  display: flex;
  justify-content: center;
  align-items: center;
  width: 50px;
  height: 50px;
}

.box + .box {
  opacity: 1;
}
```

```
.box {
  background-color: palegreen;
  border-top: 4px solid black;
  opacity: 20%;
}

.box:first-of-type {
  position: absolute;
}

.box1 {
  offset-path: ray(0deg);
}

.box2 {
  offset-path: ray(150deg);
}

.box3 {
  offset-rotate: 0deg;
  offset-position: 20% 40%;
  offset-path: ray(150deg);
}

.box4 {
  offset-position: 0 0;
  offset-path: ray(0deg);
}

.box5 {
  offset-path: ray(60deg closest-side at bottom right);
}
```

```
<pre>/* original */</pre>
<div class="container">
  <div class="box">0</div>
  <div class="box box0">0</div>
</div>

<pre>
  offset-path: ray(0deg);
  /* Default offset starting position is 50% 50% */
```

```
</pre>
<div class="container">
  <div class="box">0</div>
  <div class="box box1">1</div>
</div>

<pre>
  offset-path: ray(150deg);
</pre>
<div class="container">
  <div class="box">0</div>
  <div class="box box2">2</div>
</div>

<pre>
  offset-rotate: 0deg;
  offset-position: 20% 40%;
  offset-path: ray(150deg);
</pre>
<div class="container">
  <div class="box">0</div>
  <div class="box box3">3</div>
</div>

<pre>
  offset-position: 0 0;
  offset-path: ray(0deg);
</pre>
<div class="container">
  <div class="box">0</div>
  <div class="box box4">4</div>
</div>

<pre>
  offset-path: ray(60deg closest-side at bottom right);
</pre>
<div class="container">
  <div class="box">0</div>
  <div class="box box5">5</div>
</div>
```

Similar to `transform-origin`, the default anchor point is at the center of an element. This anchor point can be modified using the `offset-anchor` property.

In this example, various `offset-path: ray()` values are applied to the boxes numbered `1` to `5`. The "containing block" of each box is depicted with a dashed border. A faded box in the upper left corner shows each box's default position without any `offset-position` or `offset-path` applied, allowing for a side-by-side comparison. The top of each box is highlighted with a `solid` border to illustrate variations in ray starting points and orientations. After positioning at the ray's starting point, a box aligns with the direction of the specified ray angle. If `offset-position` is not specified, the default offset starting position of the ray is the center (or `50% 50%`) of the box's containing block.

Result

- `box1` gets initially positioned such that its anchor point (its center) is at the default offset starting position (`50% 50%` of the containing block). `box1` is also rotated to orient it towards the `0deg` angle of the ray. This will now be the starting point of the path. You can observe the change in position and rotation of the box by comparing it to the faded `box0` on the left. The box is rotated to match the `0deg` angle along y-axis, pointing up. The box rotation is evident from the orientation of the number inside the box.
- In `box2`, a greater positive angle of `150deg` is applied to the ray to show how the ray angle works. Starting from the top-left corner, the box is rotated in a clockwise direction to reach the specified angle of `150deg`.
- `box2` and `box3` have the same `offset-path` values. In `box3`, an `offset-rotate` of `0deg` is also applied to the element. As a result, the element will remain rotated at this specific angle all along the ray's path, and the element will not rotate in the direction of the path. Notice in `box3` that the ray path is at `150deg`, but the box orientation will not change along the path because of `offset-rotate`. Also note that the `offset-path` property of `box3` does not specify a starting `<position>`, so the ray's starting position is derived from the element's `offset-position`, which in this case is `top 20% left 40%`.
- The `offset-position` of `box4` is set to top-left corner (`0 0`) of the containing block, and as a result, the element's anchor point and the offset starting position coincide. The ray angle of `0deg` is applied to the element at this starting point.
- In `box5`, the `offset-path` property specifies the `at <position>` value, which places the box at the `bottom` and `right` edge of the element's containing block and `60deg` is applied to the ray's angle.

Animating an element along the ray

In this example, the first shape is shown as a reference for its position and orientation. A ray motion path is applied on the other shapes.

CSS

```
body {
  display: grid;
  grid-template-columns: 200px 100px;
  gap: 40px;
  margin-left: 40px;
}

.container {
  transform-style: preserve-3d;
  width: 150px;
  height: 100px;
  border: 2px dotted green;
}

.shape {
  width: 40px;
  height: 40px;
  background: #2bc4a2;
  margin: 5px;
  text-align: center;
  line-height: 40px;
  clip-path: polygon(0% 0%, 70% 0%, 100% 50%, 70% 100%, 0% 100%, 30% 50%);
  animation: move 5000ms infinite alternate ease-in-out;
}

.shape2 {
  offset-path: ray(120deg sides contain);
}
```

```
.shape3 {
  offset-rotate: 0deg;
  offset-path: ray(120deg sides contain);
}

.shape4 {
  offset-position: auto;
  offset-path: ray(120deg closest-corner);
}

.shape5 {
  offset-position: auto;
  offset-path: ray(120deg farthest-corner);
}

@keyframes move {
  0%,
  20% {
    offset-distance: 0%;
  }
  80%,
  100% {
    offset-distance: 100%;
  }
}
```

```
<div>
  <div class="container">
    <div class="shape shape1">&mdash;</div>
  </div>
</div>

<pre>/* no offset-path applied */</pre>

<div>
  <div class="container">
    <div class="shape shape2">&mdash;</div>
  </div>
</div>

<pre>offset-path: ray(120deg sides contain);</pre>

<div>
  <div class="container">
    <div class="shape shape3">&mdash;</div>
  </div>
</div>

<pre>
offset-path: ray(120deg sides contain);
offset-rotate: 0deg;
</pre>

<div>
  <div class="container">
    <div class="shape shape4">&mdash;</div>
  </div>
</div>

<pre>
  offset-position: auto;
  offset-path: ray(120deg closest-corner);
</pre>

<div>
  <div class="container">
    <div class="shape shape5">&mdash;</div>
  </div>
</div>

<pre>
  offset-position: auto;
  offset-path: ray(120deg farthest-corner);
</pre>
```

Result

In the first two samples where `offset-path` is applied, notice the orientation of the shape without `offset-rotate` and with `offset-rotate`. Both these samples use the default `offset-position` value `normal`, and therefore, the path motion starts from `50% 50%`. The last two `offset-path` samples show the impact of corner `<size>` values: `closest-corner` and `farthest-corner`. The `closest-corner` value creates a very short offset-path because the shape is already at the corner (`offset-position: auto`). The `farthest-corner` value creates the longest offset-path, going from the top-left corner of the containing block to the bottom-right corner.

See also

- `offset-distance`
- `offset-path`
- `offset-rotate`

<relative-size> CSS type

The **`<relative-size>`** CSS data type describes relative size keywords. The `<relative-size>` keywords define a size relative to the computed size of the parent element. This data type is used in the `font` shorthand and `font-size` properties.

Syntax

```
<relative-size> = smaller | larger
```

Values

The `<relative-size>` data type is defined using a keyword value chosen from the list below.

`smaller`

A relative size one size smaller than the inherited size.

`larger`

A relative size one size larger than the inherited size.

Description

The `<relative-size>` keywords are relative to the current size of the element. If the inherited size is defined using an `<absolute-size>` keyword, the `<relative-size>` value equates to the adjacent size in the `<absolute-size>` table. Otherwise, the relative increase or decrease in size is between 120% and 150%.

Examples

Comparing the keyword values

```
<ul>
  <li class="smaller">font-size: smaller;</li>
  <li>font-size is not specified</li>
  <li class="larger">font-size: larger;</li>
</ul>
```

```
li {
  margin-bottom: 0.3em;
}
.smaller {
  font-size: smaller;
}
.larger {
  font-size: larger;
}
```

Result

See also

- CSS `<absolute-size>` data type
- CSS `font` and `font-size` properties
- CSS fonts module

rem() CSS function

The **`rem()`** CSS function returns a remainder left over when the first parameter is divided by the second parameter, similar to the JavaScript remainder operator (%). The remainder is the value left over when one operand, the dividend, is divided by a second operand, the divisor. It always takes the sign of the dividend.

> **Note:** To read about the unit `rem`, see the `<length>` page.

For example, the CSS `rem(27, 5)` function returns the remainder of `2`. When dividing 27 by 5, the result is 5 with a remainder of 2. The full calculation is `27 / 5 = 5 * 5 + 2`.

Syntax

```
/* Unitless <number> */
line-height: rem(21, 2); /* 1 */
line-height: rem(14, 5); /* 4 */
line-height: rem(5.5, 2); /* 1.5 */

/* Unit based <percentage> and <dimension> */
margin: rem(14%, 3%); /* 2% */
margin: rem(18px, 5px); /* 3px */
margin: rem(10rem, 6rem); /* 4rem */
margin: rem(26vmin, 7vmin); /* 5vmin */
margin: rem(1000px, 29rem); /* 72px - if root font-size is 16px */

/* Negative/positive values */
rotate: rem(200deg, 30deg); /* 20deg */
rotate: rem(140deg, -90deg); /* 50deg */
rotate: rem(-90deg, 20deg); /* -10deg */
rotate: rem(-55deg, -15deg); /* -10deg */

/* Calculations */
scale: rem(10 * 2, 1.7); /* 1.3 */
rotate: rem(10turn, 18turn / 3); /* 4turn */
transition-duration: rem(20s / 2, 3000ms * 2); /* 4s */
```

Parameters

The `rem(dividend, divisor)` function accepts two comma-separated values as its parameters. Both parameters must have the same type, number, dimension, or `<percentage>`, for the function to be valid. While the units in the two parameters don't need to be the same, they do need of the same dimension type, such as `<length>`, `<angle>`, `<time>`, or `<frequency>` to be valid.

`dividend`

A calculation that resolves to a `<number>`, `<dimension>`, or `<percentage>` representing the dividend.

`divisor`

A calculation that resolves to a `<number>`, `<dimension>`, or `<percentage>` representing the divisor.

Return value

Returns a `<number>`, `<dimension>`, or `<percentage>` (corresponds to the parameters' type) representing the remainder, that is, the operation left over.

- If `divisor` is `0`, the result is `NaN`.
- If `dividend` is `infinite`, the result is `NaN`.
- If `dividend` is positive the result is positive (`0+`), and if `dividend` is negative the result is negative (`0-`).

Formal syntax

```
<rem()> =
  rem( <calc-sum> , <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>       |
  <dimension>    |
  <percentage>   |
  <calc-keyword> |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

See also

- round()
- mod()
- <length>

repeat() CSS function

The **repeat()** CSS function represents a repeated fragment of the track list or rule-line list. It allows repeated patterns of columns, rows, and their gap decorations, to be written in a compact form.

Try it

```
grid-template-columns: repeat(2, 60px);
rule-color: repeat(2, green, orange), yellow;
```

```
grid-template-columns: 1fr repeat(2, 60px);
rule-color: repeat(2, green), repeat(2, yellow);
```

```
grid-template-columns: repeat(2, 20px 1fr);
rule-color: repeat(auto, green), repeat(2, yellow);
```

```
grid-template-columns: repeat(auto-fill, 40px);
rule-color: repeat(2, orange, yellow), repeat(auto, green);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-auto-rows: 40px;
  grid-gap: 10px;
  width: 220px;
  rule: solid 4px;
}

#example-element > div {
  background-color: rgb(0 0 255 / 0.2);
  border: 3px solid blue;
}
```

This function can be used with the following features:

CSS grid properties:

- grid-template-columns
- grid-template-rows

CSS gap properties:

- column-rule-color
- row-rule-color
- rule-color shorthand
- column-rule-style
- row-rule-style

- `rule-style` shorthand
- `column-rule-width`
- `row-rule-width`
- `rule-width` shorthand
- `column-rule` shorthand
- `row-rule` shorthand
- `rule` shorthand

Syntax

```
/* <track-repeat> values */
repeat(4, 1fr)
repeat(4, [col-start] 250px [col-end])
repeat(4, [col-start] 60% [col-end])
repeat(4, [col-start] 1fr [col-end])
repeat(4, [col-start] min-content [col-end])
repeat(4, [col-start] max-content [col-end])
repeat(4, [col-start] auto [col-end])
repeat(4, [col-start] minmax(100px, 1fr) [col-end])
repeat(4, [col-start] fit-content(200px) [col-end])
repeat(4, 10px [col-start] 30% [col-middle] auto [col-end])
repeat(4, [col-start] min-content [col-middle] max-content [col-end])

/* <auto-repeat> values */
repeat(auto-fill, 250px)
repeat(auto-fit, 250px)
repeat(auto-fill, [col-start] 250px [col-end])
repeat(auto-fit, [col-start] 250px [col-end])
repeat(auto-fill, [col-start] minmax(100px, 1fr) [col-end])
repeat(auto-fill, 10px [col-start] 30% [col-middle] 400px [col-end])

/* <fixed-repeat> values */
repeat(4, 250px)
repeat(4, [col-start] 250px [col-end])
repeat(4, [col-start] 60% [col-end])
repeat(4, [col-start] minmax(100px, 1fr) [col-end])
repeat(4, [col-start] fit-content(200px) [col-end])
repeat(4, 10px [col-start] 30% [col-middle] 400px [col-end])

/* <name-repeat> values */
repeat(5, [footer])
repeat(auto-fill, [header])
repeat(2, [header footer])

/* <repeat-line> */
repeat(4, dashed)
repeat(2, solid, dotted, inset)
repeat(3, medium)
repeat(4, red)
repeat(3, green medium outset)

/* <auto-repeat-line> */
repeat(auto, dotted)
repeat(auto, 10px)
repeat(auto, currentcolor)
repeat(auto, red, blue, green)
repeat(auto, var(--lineColor) var(--lineSize) solid)
```

Values

The `repeat()` function takes two arguments: The first argument specifies **repeat count**, and the second specifies the **repeated values list** (the list of values to be repeated).

Repeat count values

`<integer>`

A positive integer greater than or equal to `1` specifying the number of times the feature list in the second parameter will be repeated.

`auto-fill`

Specifies an auto repeater that repeats to fill a space. Resolves to the largest number of repetitions that does not cause overflow of a constrained (has a maximum size) content box. Otherwise, if the content box has a minimum size, the fewest number of repetitions that meets the minimum size. If there is neither a minimum nor a maximum size, resolves to `1`. When used with subgrid, the second parameter must be a list of line names.

`auto-fit`

Behaves as `auto-fill`, except that after placing grid items, any empty repeated tracks are collapsed.

`auto`

Specifies an auto repeater. Once the values from other components of the property value have been applied, the values from the second parameter are repeated as many times as needed, filling any missing values.

Repeated values list

A list from one of the following track list types. Whether the components of the list are comma- or space-separated, depends on which separator is expected for that property value list type.

`<line-names>`

To add line names to subgrids, each `<line-names>` is zero or more `<custom-ident>` values, space-separated and enclosed in square brackets, such as `[start header-start]`.

`<track-size>`

Each `<track-size>` is a positive `<length-percentage>`, a `minmax()` function with the first parameter being the keyword `min-content`, `max-content`, or `auto`, or a positive `<length-percentage>`, or a `fit-content()` function.

`<fixed-size>`

Each `<fixed-size>` is either a positive `<length-percentage>`, or a `minmax()` function whose first parameter is a positive `<length-percentage>` and whose second parameter is a non-negative dimension with the unit `fr` specifying the track's flex factor or the keyword `min-content`, `max-content`, or `auto`, or a `minmax()` function whose first parameter is a positive `<length-percentage>` or the keyword `min-content`, `max-content`, or `auto`, and whose second parameter is a positive `<length-percentage>`.

`<value>`

For defining gap rules, each `<value>` is a value that would be accepted by the property in which the `repeat()` appears, such as a `<line-style>`, `<line-width>`, `<color>`, or all three as a `<gap-rule>`.

Description

The `repeat()` function represents a repeated fragment within a comma- or space-separated values list, that allows a recurring pattern to be written in a more compact form that can be repeated a specified number of times or can be auto-repeated.

The generic form of the `repeat()` syntax is, approximately,

```
repeat( <repeat-count>, <values-list> )
```

The first argument, the **repeat count**, specifies the number of repetitions. It defines the number of times the list of values defined in the second parameter should be repeated. It is specified either as an integer value of `1` or more or as the keyword `auto-fill`, `auto-fit`, or `auto`.

The second argument is a comma- or space-separated list of values that are valid for the property in which the `repeat()` appears. It specifies the **repeated value type**, which is generally the *tracks* or *rule feature(s)* to be repeated.

tracks:

Specifies the set of tracks that will be repeated. Fundamentally this consists of one or more space-separated values, where each value represents the size of that track. Each size is specified using either a `<track-size>` value or a `<fixed-size>` value. You can also specify one or more line names before or after each track, by providing `<line-names>` values before and/or after the track size.

rule feature:

Specifies the rule feature that will be repeated. This is either a comma separated list of `<color>`, `<line-width>`, or `<line-style>` values, or a comma-separated list of shorthand declarations setting the line's color, width, and style.

Syntax forms

The syntax of the `repeat()` function has several forms:

`<track-repeat> = repeat( <integer>, [ <line-names>? <track-size> ]+ <line-names>? )`

The `<integer>` sets the repeat count.

The `<track-size>` values set the track size. A `<track-size>` is either a `<length-percentage>`, a `<flex>` value (a positive `fr` unit value), or the keyword `min-content`, `max-content`, or `auto`, a `fit-content()` function with a `<length-percentage>` parameter, or a `minmax()` function. For `minmax()` track values, the `min` is either a `<length-percentage>`, or the `min-content`, `max-content`, or `auto` keyword while the `max` can be any of those, or a `<flex>` value.

Each `<track-size>` is optionally preceded by `<line-names>`, which are zero or more space-separated `<custom-ident>` values enclosed in square brackets.

An ending `<line-names>` is optional.

`<auto-repeat> = repeat( auto-fill | auto-fit, [ <line-names>? <fixed-size> ]+ <line-names>? )`

The `auto-fill` or `auto-fit` keyword sets the repeat count

The `<fixed-size>` sets the track size. A `<fixed-size>` is either a `<length-percentage>` value, or a `minmax()` function with either the `min` or the `max` being a `<length-percentage>` and the other value also being a `<length-percentage>`, or set to the keyword `min-content`, `max-content`, or `auto`.

Each `<fixed-size>` is optionally preceded by `<line-names>`.

The ending `<line-names>` is optional.

`<fixed-repeat> = repeat( <integer>, [ <line-names>? <fixed-size> ]+ <line-names>? )`

Same as `<auto-repeat>`, except the `<integer>` sets the repeat count, so there is no auto-repeating.

`<name-repeat> = repeat( [ <integer> | auto-fill ], <line-names>+)`

The integer or `auto-fill` keyword sets the repeat count

The `<line-names>` are one or more space-separated `<custom-ident>` values enclosed in square brackets.

These line names (rather than track sizes) are relevant for `subgrid`.

`<repeat-value-type> = repeat( <integer> , <value># )`

The `<integer>` sets the repeat count.

The `<value>` is one or more comma-separated valid values for the property on which the `repeat()` function is applied.

Multiple `repeat()` functions of the `<repeat-value-type>` may be used within a single value declaration.

`<auto-repeat-value-type> = repeat( auto , <value># )`

The `auto` keyword specifies an auto repeater. An auto repeater will be used to fill in values for gaps that would not otherwise receive values from other parts of the list.

The `<value>` is one or more comma-separated valid values for the property on which the `repeat()` function is applied.

At most one `repeat()` in a given list of values may be an auto repeater.

Auto-fit versus auto-fill

When the first parameter of the `repeat()` function is a keyword, either `auto-fill`, `auto-fit`, or `auto`, rather than an integer, it creates an auto-repeater. An auto-repeating `repeat()` function repeats the values provided as the second parameter as many times as necessary.

With `auto-fill`, if the container has a definite or maximum size set in the relevant axis, the number of repetitions is the largest possible positive integer that does not cause the content to overflow its container. Treating each track as its maximal track sizing function (each independent value used to define `grid-template-rows` or `grid-template-columns`), if that is definite. Otherwise, if there is no definite or maximum size defined, the `repeat()` function is a minimum track sizing function. If any number of repetitions would overflow, the repetition is `1`. Otherwise, if the grid container has a definite minimal size in the relevant axis, the number of repetitions is the smallest possible positive integer that fulfills that minimum requirement. Otherwise, the specified track list repeats only once.

The `auto-fit` value behaves the same as `auto-fill`, except that after placing the grid items any empty repeated tracks are collapsed. An empty track is one with no in-flow grid items placed into or spanning across it. (This can result in all tracks being collapsed, if they're all empty.)

A collapsed track is treated as having a single fixed track sizing function of `0px`, and the gutters on either side of it collapse.

For the purpose of finding the number of auto-repeated tracks, the user agent floors the track size to a user agent specified value (e.g., `1px`), to avoid dividing by zero.

Use exceptions and rules

There are some rules and restrictions when using `repeat()`:

A property value can contain multiple `repeat()` functions.

```
.wrapper {
  grid-template-columns:
    repeat(2, 10px)
    repeat(2, minmax(min-content, max-content));
}
```

A property value can contain, at most, one auto-repeat `repeat()` function.

```
.wrapper {
  row-rule:
    repeat(auto, yellow 3px solid, red 1px dashed),
    repeat(auto, green 5px dotted);
}
```

If a property declaration uses `<auto-repeat>`, it is only allowed to use `<fixed-repeat>` for any additional `repeat()` calls. Automatic repetitions (`auto-fill` or `auto-fit`) cannot be combined with fully intrinsic or flexible sizes. For example, this is invalid, because it combines the `<auto-repeat>` form with the `<track-repeat>` form:

```
.wrapper {
  grid-template-columns:
    repeat(auto-fill, 10px)
    repeat(2, minmax(min-content, max-content));
}
```

The `repeat()` notation cannot be nested within another `repeat()` function.

```
.wrapper {
  grid-template-columns: repeat(
    2,
    minmax(min-content, max-content),
    repeat(3, 10px)
  );
  column-rule: repeat(
    auto,
    yellow 3px solid,
    repeat(5, green 5px dotted),
    red 1px dashed
  );
}
```

Formal syntax

```
<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<integer> =
  <number-token>

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                              |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<fixed-size> =
  <fixed-breadth>                              |
  minmax( <fixed-breadth> , <track-breadth> )       |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

This syntax reflects the latest standard as per CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Specifying grid columns using repeat()

HTML

```
<div id="container">
  <div>This item is 50 pixels wide.</div>
  <div>Item with flexible width.</div>
  <div>This item is 50 pixels wide.</div>
  <div>Item with flexible width.</div>
  <div>Inflexible item of 100 pixels width.</div>
</div>
```

CSS

```
#container {
  display: grid;
  grid-template-columns: repeat(2, 50px 1fr) 100px;
  grid-gap: 5px;
  box-sizing: border-box;
  height: 200px;
  width: 100%;
  background-color: #8cffa0;
  padding: 10px;
}

#container > div {
  background-color: #8ca0ff;
  padding: 5px;
}
```

Result

Browser compatibility

See also

- `grid-template`
- `grid-template-rows`
- `grid-template-columns`
- `grid-template-areas`
- `grid-auto-columns`
- `grid-auto-rows`
- `grid-auto-flow`
- Line-based placement with CSS grid
- Grid template areas: grid definition shorthands
- CSS grid layout module
- CSS gaps module

<resolution> CSS type

The **`<resolution>`** CSS data type, used for describing resolutions in media queries, denotes the pixel density of an output device, i.e., its resolution.

On screens, the units are related to *CSS* inches, centimeters, or pixels, not physical values.

Syntax

The `<resolution>` data type consists of a strictly positive `<number>` followed by one of the units listed below. As with all CSS dimensions, there is no space between the unit literal and the number.

Units

`dpi`

Represents the number of dots per inch. Screens typically contains 72 or 96 dots per inch, but the dpi for printed documents is usually much greater. As 1 inch is 2.54 cm, `1dpi = 0.39dpcm`.

`dpcm`

Represents the number of dots per centimeter. As 1 inch is 2.54 cm, `1dpcm = 2.54dpi`.

`dppx`

Represents the number of dots per `px` unit. Due to the 1:96 fixed ratio of CSS `in` to CSS `px`, `1dppx` is equivalent to `96dpi`, which corresponds to the default resolution of images displayed in CSS as defined by `image-resolution`.

`x`

Alias for `dppx`.

> **Note:** Although the number `0` is always the same regardless of unit, the unit may not be omitted. In other words, `0` is invalid and does not represent `0dpi`, `0dpcm`, or `0dppx`.

Examples

Use in a media query

```
@media print and (resolution >= 300dpi) {
  /* … */
}

@media (resolution: 120dpcm) {
  /* … */
}

@media (resolution >= 2dppx) {
  /* … */
}

@media (resolution: 1x) {
```

```
  /* … */
}
```

Valid resolutions

```
96dpi
50.82dpcm
3dppx
```

Invalid resolutions

```
72 dpi     Spaces are not allowed between the number and the unit.
ten dpi    The number must use digits only.
0          The unit is required.
```

See also

- resolution media feature
- The `image-resolution` property
- Using @media queries

revert CSS keyword

The **`revert`** CSS keyword reverts the cascaded value of the property from its current value to the value the property would have had if no changes had been made by the current **style origin** to the current element. Thus, it resets the property either to user agent set value, to user set value, to its inherited value (if it is inheritable), or to initial value. It can be applied to any CSS property, including the CSS shorthand property `all`.

This keyword removes from the cascade all of the styles that have been overridden until the style being rolled back to is reached.

- If used by a site's own styles (the author origin), `revert` rolls back the property's cascaded value to the user's custom style, if one exists; otherwise, it rolls the style back to the user agent's default style.
- If used in a user's custom stylesheet, or if the style was applied by the user (the user origin), `revert` rolls back the cascaded value to the user agent's default style.
- If used within the user agent's default styles, this keyword is functionally equivalent to `unset`.

The `revert` keyword works exactly the same as `unset` in many cases. The only difference is for properties that have values set by the browser or by custom stylesheets created by users (set on the browser side).

Revert will not affect rules applied to children of an element you reset (but will remove effects of a parent rule on a child). So if you have a `color: green` for all sections and `all: revert` on a specific section, the color of the section will be black. But if you have a rule to make all paragraphs red, then all paragraphs will still be red in all sections.

> **Note:** Revert is just a value. It is still possible to override the `revert` value using specificity.

> **Note:** The `revert` keyword is different from and should not be confused with the `initial` keyword, which uses the initial value defined on a per-property basis by the CSS specifications. In contrast, user-agent stylesheets set default values on the basis of CSS selectors.
>
> For example, the initial value for the `display` property is `inline`, whereas a normal user-agent stylesheet sets the default `display` value of `<div>`s to `block`, of `<table>`s to `table`, etc.

Examples

Revert vs. unset

Although `revert` and `unset` are similar, they differ for some properties for some elements.

So in the below example, we set custom `font-weight`, but then try to `revert` and `unset` it inline in the HTML document. The `revert` keyword will revert the text to bold because that is the default value for headers in most browsers. The `unset` keyword will keep the text normal because, as an inherited property, the `font-weight` would then inherit its value from the body.

HTML

```
<h3 style="font-weight: revert; color: revert;">
  This should have its original font-weight (bold) and color: black
</h3>
<p>Just some text</p>
<h3 style="font-weight: unset; color: unset;">
  This will still have font-weight: normal, but color: black
</h3>
<p>Just some text</p>
```

CSS

```
h3 {
  font-weight: normal;
  color: blue;
}
```

Result

Revert all

Reverting all values is useful in a situation where you've made several style changes and then you want to revert to the browser default values. So in the above example, instead of reverting `font-weight` and `color` separately, you could just revert all of them at once - by applying the `revert` keyword on `all`.

HTML

```
<h3>This will have custom styles</h3>
<p>Just some text</p>
<h3 style="all: revert">This should be reverted to browser/user defaults.</h3>
<p>Just some text</p>
```

CSS

```
h3 {
  font-weight: normal;
  color: blue;
  border-bottom: 1px solid grey;
}
```

Result

Revert on a parent

Reverting effectively removes the value for the element you select with some rule and this happens only for that element. To illustrate this, we will set a green color on a section and red color on a paragraph.

HTML

```
<main>
  <section>
    <h3>This h3 will be dark green</h3>
    <p>Text in paragraph will be red.</p>
    This stray text will also be dark green.
  </section>
  <section class="with-revert">
    <h3>This h3 will be steelblue</h3>
    <p>Text in paragraph will be red.</p>
    This stray text will also be steelblue.
  </section>
</main>
```

CSS

```
main {
  border: 3px solid steelblue;
}

section {
  margin: 0.5rem;
  border: 2px dashed darkgreen;
}
```

```
main {
  color: steelblue;
}
section {
  color: darkgreen;
}
p {
  color: red;
}
section.with-revert {
  color: revert;
}
```

Result

Notice how the paragraph is still red even though a `color` property for the section was reverted. Also, note that both the header and plain text node are `steelblue`. The result of reverting makes it as if `section { color: darkgreen; }` did not exist for the section with `color: revert` applied.

Also, if neither the user agent nor the user override the `<h3>` or `<section>` color values, then the `steelblue` color is inherited from `<main>`, as the `color` property is an inherited property.

See also

- Use the `initial` keyword to set a property to its initial value.
- Use the `inherit` keyword to make an element's property the same as its parent.
- Use the `revert-layer` keyword to reset a property to the value established in a previous cascade layer.
- Use the `revert-rule` keyword to reset a property to the value of an earlier matching style rule.
- Use the `unset` keyword to set a property to its inherited value if it inherits or to its initial value if not.
- The `all` property lets you reset all properties to their initial, inherited, reverted, or unset state at once.

revert-layer CSS keyword

The **`revert-layer`** CSS-wide keyword rolls back the value of a property in a cascade layer to the value of the property in a CSS rule matching the element in a previous cascade layer. The value of a property with this keyword is recalculated as if no rules were specified on the target element in the current cascade layer.

If there is no other cascade layer to revert to for the matching CSS rule, the property value rolls back to the computed value derived from the current layer. Furthermore, if there is no matching CSS rule in the current layer, the property value for the element rolls back to the style defined in a previous style origin.

This keyword can be applied to any CSS property, including the CSS shorthand property `all`.

Revert-layer vs. revert

The `revert-layer` keyword lets you rollback styles to those specified in previous cascade layers within the author origin. The `revert` keyword, in comparison, lets you roll back styles applied in the author origin to those specified in the user origin or user-agent origin.

The `revert-layer` keyword is ideally meant to be applied to properties within a cascade layer. However, if applied to properties outside a cascade layer, it rolls back property values to any values set by presentational hints (such as `width` and `height` attributes or the `<s>` element in HTML), defaulting to the values established by the user agent's stylesheet or user styles. Unlike the `revert` keyword, which considers presentational hints as part of the author origin and reverts them as well, the `revert-layer` keyword ignores presentational hints outside the cascade layer, so it does not revert them.

Examples

Default cascade layer behavior

In the example below, two cascade layers are defined in the CSS, `base` and `special`. By default, rules in the `special` layer will override competing rules in the `base` layer because `special` is listed after `base` in the `@layer` declaration statement.

HTML

```
<p>This example contains a list.</p>

<ul>
  <li class="item feature">Item one</li>
  <li class="item">Item two</li>
  <li class="item">Item three</li>
</ul>
```

CSS

```
@layer base, special;

@layer special {
  .item {
    color: red;
  }
}

@layer base {
  .item {
    color: blue;
  }
  .feature {
    color: green;
  }
}
```

Result

All the `<li>` elements match the `item` rule in the `special` layer and are red. This is the default cascade layer behavior, where rules in the `special` layer take precedence over rules in the `base` layer.

Revert to style in previous cascade layer

Let's examine how the `revert-layer` keyword changes the default cascade layer behavior. For this example, the `special` layer contains an additional `feature` rule targeting the first `<li>` element. The `color` property in this rule is set to `revert-layer`.

HTML

```
<p>This example contains a list.</p>

<ul>
  <li class="item feature">Item one</li>
  <li class="item">Item two</li>
  <li class="item">Item three</li>
</ul>
```

CSS

```
@layer base, special;

@layer special {
  .item {
    color: red;
  }
  .feature {
    color: revert-layer;
  }
}

@layer base {
  .item {
    color: blue;
  }
  .feature {
    color: green;
  }
}
```

Result

With `color` set to `revert-layer`, the `color` property value rolls back to the value in the matching `feature` rule in the previous layer `base`, and so 'Item one' is now green.

Revert to style in previous origin

This example shows the `revert-layer` keyword behavior when there is no cascade layer to revert to *and* there is no matching CSS rule in the current layer to inherit the property value.

HTML

```
<p>This example contains a list.</p>

<ul>
  <li class="item feature">Item one</li>
  <li class="item">Item two</li>
  <li class="item">Item three</li>
</ul>
```

CSS

```
@layer base {
  .item {
    color: revert-layer;
  }
}
```

Result

The style for all `<li>` elements rolls back to the defaults in the user-agent origin.

See also

- `initial`
- `inherit`
- `revert`
- `revert-rule`
- `unset`
- `all`
- CSS cascading and inheritance module

revert-rule CSS keyword

The **`revert-rule`** CSS-wide keyword rolls back the cascaded value of a property to the value it would have had if the current style rule had not been present. The cascade then determines the value from the remaining declarations — this could be another rule in the same cascade layer, a rule in a different layer, a different style origin, or a default value (`inherited` or `initial`).

When used inside a CSS animation (the animation origin), the `revert-rule` keyword behaves like `revert-layer`.

This keyword can be applied to any CSS property, including the CSS shorthand property `all`.

Revert-rule vs. revert-layer vs. revert

The `revert-rule`, `revert-layer`, and `revert` keywords each roll back the cascade, but at different levels of granularity:

- `revert` removes all declarations from the current style origin, rolling back to the previous origin (for example, from author styles to user-agent styles).
- `revert-layer` removes all declarations from the current cascade layer, rolling back to the previous layer within the same origin.
- `revert-rule` removes only the declarations from the current style rule. Other rules in the same cascade layer still apply.

This makes `revert-rule` useful for conditionally ignoring specific declarations within a rule while still respecting declarations from other rules in the same layer.

Examples

Rolling back to the previous rule

In this example, two rules target the same element. The second rule uses `revert-rule` on the `color` property, which causes the cascade to determine the value as if the `p.special` rule were not present, falling back to the value established by the first rule.

HTML

```
<p class="special">This paragraph has special styling.</p>
```

CSS

```
body {
  font-family: system-ui;
}

@supports not (color: revert-rule) {
  body::before {
    content: "Your browser doesn't support the revert-rule keyword yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1em;
  }
}
```

```
p {
  color: blue;
  font-weight: bold;
}

p.special {
  color: revert-rule;
  border: 1px solid currentColor;
}
```

Result

The paragraph text is blue from the `p` rule because `color: revert-rule` causes the `color` declaration in `p.special` to be ignored. The `font-weight` and `border` declarations are unaffected.

Reverting from a style attribute

When `revert-rule` is used in a style attribute, it causes the cascade to act as if the style attribute were not present. This works because the style attribute is treated as its own style rule.

HTML

```
<p style="color: revert-rule">This text uses the stylesheet color.</p>
```

CSS

```
body {
  font-family: system-ui;
}

@supports not (color: revert-rule) {
  body::before {
    content: "Your browser doesn't support the revert-rule keyword yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1em;
  }
}
```

```
p {
  color: green;
}
```

Result

The paragraph text is green because `revert-rule` causes the cascade to ignore the style attribute's declaration, and the `p` rule takes effect.

Chaining multiple <code>revert-rule</code> values

If multiple rules use `revert-rule` for the same property, the cascade ignores each of them in turn, continuing back through earlier rules until it finds a concrete value.

HTML

```
<p class="a b">This text is styled by a chain of revert-rule values.</p>
```

CSS

```
body {
  font-family: system-ui;
}

@supports not (color: revert-rule) {
  body::before {
    content: "Your browser doesn't support the revert-rule keyword yet.";
    background-color: wheat;
    display: block;
    text-align: center;
    padding: 1em;
  }
}
```

```
p {
  color: red;
}
p.a {
  color: revert-rule;
}
p.b {
  color: revert-rule;
}
```

Result

Both the `p.b` and `p.a` rules are ignored by `revert-rule`. The cascade falls through to the `p` rule, so the text is red.

See also

- `initial`
- `inherit`
- `revert`
- `revert-layer`
- `unset`
- `all`
- CSS cascading and inheritance module

round() CSS function

The **`round()`** CSS function returns a rounded number based on a selected rounding strategy.

Authors should use a custom CSS property (e.g., `--my-property`) for the rounding value, interval, or both; using the `round()` function is redundant if these have known values.

Syntax

```
width: round(var(--width), 50px);
width: round(up, 101px, var(--interval));
width: round(down, var(--height), var(--interval));
margin: round(to-zero, -105px, 10px);
```

Parameters

The `round(<rounding-strategy>, valueToRound, roundingInterval)` function specifies an optional rounding strategy, a value (or mathematical expression) to be rounded and a rounding interval (or mathematical expression). The `valueToRound` is rounded according to the rounding strategy, to the nearest integer multiple of `roundingInterval`.

`<rounding-strategy>`

The rounding strategy. This may be one of the following values:

`up`

Round `valueToRound` up to the nearest integer multiple of `roundingInterval` (if the value is negative, it will become "more positive"). This is equivalent to the JavaScript `Math.ceil()` method.

`down`

Round `valueToRound` down to the nearest integer multiple of `roundingInterval` (if the value is negative, it will become "more negative"). This is equivalent to the JavaScript `Math.floor()` method.

`nearest` (default)

Round `valueToRound` to the nearest integer multiple of `roundingInterval`, which may be either above or below the value. If the `valueToRound` is half way between the rounding targets above and below (neither is "nearest"), it will be rounded up. Equivalent to JavaScript `Math.round()`.

`to-zero`

Round `valueToRound` to the nearest integer multiple of `roundingInterval` closer to/towards zero (a positive number will decrease, while a negative value will become "less negative"). This is equivalent to the JavaScript `Math.trunc()` method.

`valueToRound`

The value to be rounded. This must be a `<number>`, `<dimension>`, or `<percentage>`, or a mathematical expression that resolves to one of those values.

`roundingInterval`

The rounding interval. This is a `<number>`, `<dimension>`, or `<percentage>`, or a mathematical expression that resolves to one of those values. If `valueToRound` is a `<number>`, `roundingInterval` may be omitted and defaults to `1`. Otherwise, omitting it results in an invalid expression.

Return value

The value of `valueToRound`, rounded to the nearest lower or higher integer multiple of `roundingInterval`, depending on the `rounding strategy`.

- If `roundingInterval` is 0, the result is `NaN`.
- If `valueToRound` and `roundingInterval` are both `infinite`, the result is `NaN`.
- If `valueToRound` is infinite but `roundingInterval` is finite, the result is the same `infinity`.
- If `valueToRound` is finite but `roundingInterval` is infinite, the result depends on the rounding strategy and the sign of `A`:

    - `up` - If `valueToRound` is positive (not zero), return `+∞`. If `valueToRound` is `0+`, return `0+`. Otherwise, return `0-`.
    - `down` - If `valueToRound` is negative (not zero), return `-∞`. If `valueToRound` is `0-`, return `0-`. Otherwise, return `0+`.
    - `nearest`, `to-zero` - If `valueToRound` is positive or `0+`, return `0+`. Otherwise, return `0-`.
- The argument calculations can resolve to `<number>`, `<dimension>`, or `<percentage>`, but must have the same type, or else the function is invalid; the result will have the same type as the arguments.
- If `valueToRound` is exactly equal to an integer multiple of `roundingInterval`, `round()` resolves to `valueToRound` exactly (preserving whether `valueToRound` is `0-` or `0+`, if relevant). Otherwise, there are two integer multiples of `roundingInterval` that are potentially "closest" to `valueToRound`, lower `roundingInterval` which is closer to `-∞` and upper `roundingInterval` which is closer to `+∞`.

Formal syntax

```
<round()> =
  round( <rounding-strategy>? , <calc-sum> , <calc-sum>? )

<rounding-strategy> =
  nearest     |
  up          |
  down        |
  to-zero     |
  line-width

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Round positive values

This example demonstrates how the `round()` function's rounding strategies work for positive values.

Of the five boxes below, the `round()` function is used to set the height of the last four. The value to be rounded is between 100 px and 125 px in each case, and the rounding value is 25px in all cases. The height of the boxes is therefore either rounded up to 125 px or down to 100 px.

HTML

The HTML defines 5 `div` elements that will be rendered as boxes by the CSS. The elements contain text indicating the rounding strategy, initial value, and expected final height of the box (in parentheses).

```
<div class="box box-1">height: 100px</div>
<div class="box box-2">up 101px (125px)</div>
<div class="box box-3">down 122px (100px)</div>
<div class="box box-4">to-zero 120px (100px)</div>
<div class="box box-5">nearest 117px (125px)</div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 50px;
}
```

The CSS that is applied to all boxes is shown below. Note that we apply a custom CSS property named `--rounding-interval`, that we will use for the rounding interval.

```
div.box {
  width: 100px;
  height: 100px;
  background: lightblue;
  --rounding-interval: 25px;
}
```

The first `div` from the left isn't targeted with specific CSS rules, so it will have a default height of 100px. The CSS for `div` two, three, and four is shown below, which round, up, down, and to-zero, respectively.

```
div.box-2 {
  height: round(up, 101px, var(--rounding-interval));
}
div.box-3 {
  height: round(down, 122px, var(--rounding-interval));
}
div.box-4 {
  height: round(to-zero, 120px, var(--rounding-interval));
}
```

Notice how above we indicate the rounding interval using `var()` and the custom CSS property `--rounding-interval`.

The last box is set without specifying a rounding strategy, and hence defaults to `nearest`. In this case, the nearest interval to 117 px is 125px, so it will round up. Just for contrast, here we specified hardcoded values for both the rounding value and interval. While this is allowed, you wouldn't do this normally because there is no point rounding a number when you already know what the result must be.

```
div.box-5 {
  height: round(117px, 25px);
```

```
}
```

Result

If the browser supports the CSS `round()` function, you should see five columns with heights that are rounded as indicated by their contained text.

See also

- `mod()`
- `rem()`

<rule-list> CSS type

The **`<rule-list>`** CSS data type represents a sequence of **zero or more CSS rules**. It is used to define places in CSS where multiple rules may appear, such as the top level of a stylesheet or inside grouping at-rules like `@media` or `@supports`.

A `<rule-list>` is not written directly. Instead, it describes how the CSS parser collects and interprets rules inside a block or stylesheet.

Syntax

A `<rule-list>` is defined as a sequence of **zero or more**:

- **Style rules** (e.g., `p { color: red; }`).
- **Qualified at-rules** (e.g., `@media (width < 600px) { ... }`).
- **Ignored statements** (e.g., parse errors handled by the parser, which the parser automatically skips).

All whitespace, comments, and invalid or malformed constructs are handled according to the CSS parser rules.

Description

The `<rule-list>` type appears in the specification anywhere CSS is defined to contain a "list of rules."

Examples include:

- The **top level** of a stylesheet.
- The body of **grouping at-rules** like `@media`, `@custom-media`, `@supports`, `@layer`, `@container`.
- The contents of **nesting selectors** (CSS Nesting).

Although authors cannot explicitly write `<rule-list>`, understanding it is crucial when interpreting how CSS parses nested structures, how invalid rules are dropped, and how the cascade forms within conditional blocks.

Examples

A <code><rule-list></code> in a stylesheet

The following stylesheet is treated as a `<rule-list>` containing two style rules and an at-rule.

```
p {
  margin: 0;
}

h1 {
  font-size: 2rem;
}

@media (width < 600px) {
  body {
    background: lightgray;
  }
}
```

A <code><rule-list></code> inside <code>@media</code> at-rule

The block contained inside an `@media` at-rule is a `<rule-list>` containing one or more style rules. The following example contains two style rules.

```
@media (prefers-color-scheme: dark) {
  main {
    background: black;
    color: white;
  }

  a {
    color: skyblue;
  }
}
```

Invalid rules inside a <code><rule-list></code>

The invalid token sequence (`!invalid-rule`) is ignored by the parser. The rest of the rules form a valid `<rule-list>`.

```
body {
  color: black;
}

!invalid-rule

@supports (display: grid) {
  section {
    display: grid;
  }
}
```

See also

- CSS `@supports` at-rule
- CSS `@media` at-rule
- CSS `@custom-media` at-rule

<self-position> CSS type

The **`<self-position>`** enumerated value data type is used by the `justify-self` and `align-self` properties, and the `place-self` shorthand, to align the box within its alignment container. It is also used by the `justify-items` and `align-items` properties, and the `place-items` shorthand, to specify default values for `justify-self` and `align-self`.

Syntax

```
<self-position> = center | start | end | self-start | self-end | flex-start | flex-end
```

Values

The following keyword values are represented by the `<self-position>` grammar term:

`center`

Centers the alignment subject within its alignment container.

`start`

Aligns the alignment subject flush with the alignment container's start edge.

`end`

Aligns the alignment subject flush with the alignment container's end edge.

`self-start`

Aligns the alignment subject flush with the edge of the alignment container corresponding to the alignment subject's start side.

`self-end`

Aligns the alignment subject flush with the edge of the alignment container corresponding to the alignment subject's end side.

`flex-start`

In flex layout, aligns the alignment subject flush with the edge of the alignment container corresponding to the flex container's main-start or cross-start side, as appropriate. It is identical to `start` for layout modes other than flex layout.

`flex-end`

In flex layout, aligns the alignment subject flush with the edge of the alignment container corresponding to the flex container's main-end or cross-end side, as appropriate. Identical to `end` for layout modes other than flex layout.

> **Note:** The `left` and `right` keywords are excluded from `<self-position>`, despite being valid positional alignment values for the `justify-*` properties (`justify-content`, `justify-self`, and `justify-items`), because they are not allowed in the `align-*` properties (`align-content`, `align-self`, and `align-items`). They are instead explicitly included in the `justify-*` properties' grammars.

See also

- Properties that use this data type: `align-self`, `justify-self`, `place-self`, `align-items`, `justify-items`, `place-items`
- Other box alignment data types: `<baseline-position>`, `<content-distribution>`, `<overflow-position>`, and `<content-position>`
- CSS box alignment module
- CSS flexible box layout module
- CSS grid layout module

<shape> CSS type

The **`<shape>`** CSS data type defines the specific form (shape) of a region. The region represents the part of an element to which the `clip` property applies.

> **Note:** `<shape>` and `rect()` work in conjunction with `clip`, which has been deprecated in favor of `clip-path`. When possible, use `clip-path` and the `<basic-shape>` data type instead.

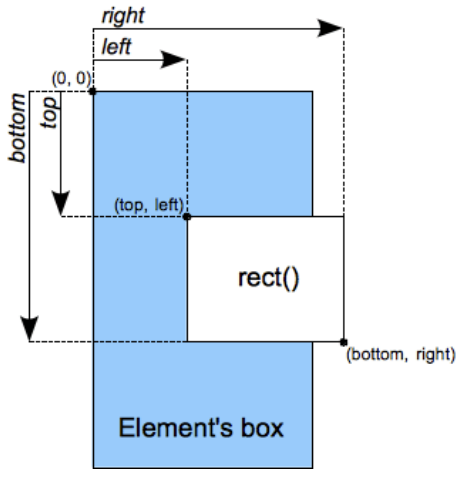
Syntax

The `<shape>` data type is specified using the `rect()` function, which produces a region in the form of a rectangle.

`rect()`

```
rect(top, right, bottom, left)
```

Values



top

Is a `<length>` representing the offset for the top of the rectangle relative to the top border of the element's box.

right

Is a `<length>` representing the offset for the right of the rectangle relative to the left border of the element's box.

bottom

Is a `<length>` representing the offset for the bottom of the rectangle relative to the top border of the element's box.

left

Is a `<length>` representing the offset for the left of the rectangle relative to the left border of the element's box.

Interpolation

When animated, values of the `<shape>` data type are interpolated over their `top`, `right`, `bottom`, and `left` components, each treated as a real, floating-point number. The speed of the interpolation is determined by the easing function associated with the animation.

Example

Example demonstrating correct use of the rect() function

```
img.clip04 {
  clip: rect(10px, 20px, 20px, 10px);
}
```

See also

- Related CSS property: `clip`

sibling-count() CSS function

The **`sibling-count()`** CSS function returns an integer representing the total number of sibling DOM elements (direct children of the parent) of the element on which it is used, including itself.

> **Note:** The `counter()` function provides a similar result but it returns a `<string>` (which is more suitable for generated content, while `sibling-count()` returns an `<integer>` (which can be used for calculations).

Syntax

```
sibling-count()
```

Parameters

The `sibling-count()` function doesn't accept parameters.

Return value

An integer; the total number of sibling DOM elements including the element itself.

Examples

Dynamic column count

This example demonstrates setting the width of each item in a list based on the number of items, enabling placing each child in its own column.

HTML

We include a `<ul>` container and several `<li>` elements children.

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
</ul>
```

CSS

We divide the `width` of each list item by the number of direct children the list contains. We also set every odd element to have a `background-color` to better demonstrate the resulting effect.

```
ul {
  list-style-type: none;
  padding: 0;
  margin: 0;
  text-align: center;
  display: flex;
}

li {
  width: calc(100% / sibling-count());
}

li:nth-of-type(odd) {
  background-color: rgb(0 0 0 / 0.05);
}
```

Results

See also

- `sibling-index()`
- `calc()`

sibling-index() CSS function

The **`sibling-index()`** CSS function returns an integer representing the position of the current element in the DOM tree relative to all its sibling elements. The returned value is the index number of the contextual child's position among all the sibling elements within a parent element, with the first child returning `1` and the last child, returning the `length` of `Element.children`.

> **Note:** Like the `:nth-child()` pseudo-class, `sibling-index()` starts from 1, not 0.

> **Note:** The `counter()` function provides a similar result but it returns a `<string>` (which is more suitable for generated content, while `sibling-index()` returns an `<integer>` (which can be used for calculations).

Try it

```
--width: calc(sibling-index() * 30px);
```

```
--width: calc(sibling-index() * 20px);
```

```
--width: calc(sibling-index() * 10px);
```

```
--width: 100px;
```

```
<ul id="example-element">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
  <li>9</li>
  <li>10</li>
</ul>
```

```
#example-element {
  list-style-type: none;
  padding: 0;
  margin: 0 auto;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
}

#example-element > li {
  text-align: center;
  padding: 2px;
  border-radius: 8px;
  width: var(--width, calc(sibling-index() * 30px));
  color: white;
  background-color: hsl(
    calc(360deg / sibling-count() * sibling-index()) 50% 50%
  );
}
```

Syntax

```
sibling-index()
```

Parameters

The `sibling-index()` function doesn't accept parameters.

Return value

An integer; the position of the current element in the DOM tree's sibling order.

Examples

Dynamic list width

This example demonstrates how to dynamically increase the width of each `<li>` item in the `<ul>` by `50px`.

HTML

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
</ul>
```

CSS

```
li {
  width: calc(sibling-index() * 50px);
  background-color: #ffaaaa;
}
```

Results

Sequential animations

Combining `sibling-index()` with CSS animations opens new possibilities. In this example, the opacity of elements in sequential order by setting an `animation-delay` based on their order in the DOM.

HTML

We include a container element with four children:

```
<ul>
  <li>One</li>
  <li>Two</li>
```

```
  <li>Three</li>
  <li>Four</li>
</ul>
```

CSS

We apply the `fade-in` animation to each element. We use the `sibling-index()` function within a `calc()` function to set the duration of the `animation-delay` based on the source element's position in the source order. The `animation-fill-mode` applies the animation's `0%` keyframe until the `animation-duration` expires.

```
ul {
  list-style-type: none;
  padding: 0;
  margin: 0;
}

li {
  animation-name: fade-in;
  animation-duration: 1s;
  animation-iteration-count: 1;
  animation-timing-function: linear;
  animation-fill-mode: backwards;
  animation-delay: calc(1s * sibling-index());
}

@keyframes fade-in {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}
```

Results

See also

- `sibling-count()`
- `counter()`

sign() CSS function

The **`sign()`** CSS function contains one calculation, and returns `-1` if the numeric value of the argument is negative, `+1` if the numeric value of the argument is positive, `0`⁺ if the numeric value of the argument is 0^+, and `0`⁻ if the numeric value of the argument is 0^-.

> **Note:** While `abs()` returns the absolute value of the argument, `sign()` returns the sign of the argument.

Syntax

```
/* property: sign( expression ) */
top: sign(20vh - 100px);
```

Parameters

The `sign(x)` function accepts only one value as its parameter.

`x`

A calculation which resolves to a number.

Return value

A number representing the sign of `A`:

- If `x` is positive, returns `1`.
- If `x` is negative, returns `-1`.
- If `x` is positive zero, returns `0`.
- If `x` is negative zero, returns `-0`.
- Otherwise, returns `NaN`.

Formal syntax

```
<sign()> =
  sign( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Background image position

For example, in `background-position` positive percentages resolve to a negative length, and vice versa, if the background image is larger than the background area. Thus `sign(10%)` might return `1` or `-1`, depending on how the percentage is resolved! (Or even `0`, if it's resolved against a zero length.)

```
div {
  background-position: calc(sign(10%) * 1px);
}
```

Position direction

Another use case is to control the `position` of the element. Either a positive or a negative value.

```
div {
  position: absolute;
  top: calc(100px * sign(var(--value)));
}
```

See also

- `abs()`
- Using CSS typed arithmetic

sin() CSS function

The **`sin()`** CSS function is a trigonometric function that returns the sine of a number, which is a value between `-1` and `1`. The function contains a single calculation that must resolve to either a `<number>` or an `<angle>` by interpreting the result of the argument as radians. That is, `sin(45deg)`, `sin(0.125turn)`, and `sin(3.14159 / 4)` all represent the same value, approximately `0.707`.

Try it

```
transform: translateX(calc(cos(0deg) * 140px))
  translateY(calc(sin(0deg) * -140px));
```

```
transform: translateX(calc(cos(90deg) * 140px))
  translateY(calc(sin(90deg) * -140px));
```

```
transform: translateX(calc(cos(135deg) * 140px))
  translateY(calc(sin(135deg) * -140px));
```

```
transform: translateX(calc(cos(180deg) * 140px))
  translateY(calc(sin(180deg) * -140px));
```

```
transform: translateX(calc(cos(-45deg) * 140px))
  translateY(calc(sin(-45deg) * -140px));
```

```
<div class="circle">
  <span class="dot" id="example-element"></span>
</div>
```

```
:root {
  --radius: 140px;
  --dot-size: 10px;
}
.circle {
  display: grid;
  place-content: center;
  margin: 0 auto;
  width: calc(var(--radius) * 2);
  aspect-ratio: 1;
  border-radius: 50%;
  border: 2px solid #666666;
  background-image:
    radial-gradient(black var(--dot-size), transparent var(--dot-size)),
    linear-gradient(135deg, blue, deepskyblue, lightgreen, lavender, honeydew);
}
.dot {
  display: block;
  width: var(--dot-size);
  aspect-ratio: 1;
  border-radius: 50%;
  border: 2px solid #666666;
  background-color: #ff6666;
  transform: translateX(calc(cos(0deg) * var(--radius)))
    translateY(calc(sin(0deg) * var(--radius) * -1));
}
```

Syntax

```
/* Single <angle> values */
width: calc(100px * sin(45deg));
width: calc(100px * sin(0.25turn));
width: calc(100px * sin(1.0471967rad));

/* Single <number> values */
width: calc(100px * sin(63.673));
width: calc(100px * sin(2 * 0.125));

/* Other values */
width: calc(100px * sin(pi / 2));
width: calc(100px * sin(e / 4));
```

Parameters

The `sin(angle)` function accepts only one value as its parameter.

`angle`

A calculation which resolves to a `<number>` or an `<angle>`. When specifying unitless numbers they are interpreted as a number of radians, representing an `<angle>`

Return value

The sine of an `angle` will always return a number between `-1` and `1`.

- If `angle` is `infinity`, `-infinity`, or `NaN`, the result is `NaN`.
- If `angle` is `0`⁻, the result is `0`⁻.

Formal syntax

```
<sin()> =
  sin( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Changing box sizes

In this example, `sin(30deg)` will return `0.5`, making the box have a `50px` width and a `50px` height.

```
div {
  background-color: red;
  width: calc(sin(30deg) * 100px);
  height: calc(sin(30deg) * 100px);
}
```

Controlling animation duration

Another use case is to control the `animation-duration`, reducing the duration based on the sine value. In this case, the animation duration will be `1s`.

```
div {
  animation-name: myAnimation;
  animation-duration: calc(sin(0.25turn) * 1s);
}
```

See also

- `cos()`
- `tan()`
- `asin()`
- `acos()`
- `atan()`
- `atan2()`

sqrt() CSS function

The **`sqrt()`** CSS function is an exponential function that returns the square root of a number.

The function `pow(x, 0.5)` is equivalent to `sqrt(x)`.

Syntax

```
/* A <number> value */
width: calc(100px * sqrt(9)); /*  300px */
width: calc(100px * sqrt(25)); /*  500px */
width: calc(100px * sqrt(100)); /* 1000px */
```

Parameters

The `sqrt(x)` function accepts only one value as its parameter.

`x`

A calculation which resolves to a `<number>` greater than or equal to 0.

Return value

Returns a `<number>` which is the square root of `x`.

- if `x` is `+∞`, the result is `+∞`.
- If `x` is `0`⁻, the result is `0`⁻.
- If `x` is less than `0`, the result is `NaN`.

Formal syntax

```
<sqrt()> =
  sqrt( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Scale sizes based on square root

This example shows how you can use the `sqrt()` function to calculate sizes.

HTML

```
<div class="boxes">
  <div class="box">50px</div>
  <div class="box one">100px</div>
  <div class="box two">150px</div>
  <div class="box three">200px</div>
</div>
```

CSS

Here we are using CSS custom properties to define the sizes to be used. First, we declare the first size (`--size-0`), which is then used to calculate the other sizes.

- `--size-1` is calculated by multiplying the value of `--size-0` (50px) by the square root of 4 (2), which results in 100px.
- `--size-2` is calculated by multiplying the value of `--size-0` (50px) by the square root of 9 (3), which results in 150px.
- `--size-3` is calculated by multiplying the value of `--size-0` (50px) by the square root of 16 (4), which results in 200px.

```
:root {
  --size-0: 50px;
  --size-1: calc(var(--size-0) * sqrt(4)); /*  100px */
  --size-2: calc(var(--size-0) * sqrt(9)); /*  150px */
  --size-3: calc(var(--size-0) * sqrt(16)); /*  200px */
}
```

```
.boxes {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-around;
}

.box {
  width: var(--size-0);
  height: var(--size-0);
  background-color: tomato;
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

The sizes are then applied as the `width` and `height` values of the selectors.

```
.one {
  width: var(--size-1);
  height: var(--size-1);
}
.two {
  width: var(--size-2);
  height: var(--size-2);
}
.three {
  width: var(--size-3);
  height: var(--size-3);
}
```

Result

See also

- `pow()`
- `hypot()`
- `log()`
- `exp()`

<string> CSS type

The **`<string>`** CSS data type represents a sequence of characters. Strings are used in numerous CSS properties, such as `content`, `font-family`, and `quotes`.

Syntax

The `<string>` data type is composed of any number of Unicode characters surrounded by either double (`"`) or single (`'`) quotes.

Most characters can be represented literally. All characters can also be represented with their respective Unicode code points in hexadecimal, in which case they are preceded by a backslash (`\`). For example, `\22` represents a double quote, `\27` a single quote (`'`), and `\A9` the copyright symbol (©).

Importantly, certain characters which would otherwise be invalid can be escaped with a backslash. These include double quotes when used inside a double-quoted string, single quotes when used inside a single-quoted string, and the backslash itself. For example, `\\` will create a single backslash.

To output new lines, you must escape them with a line feed character such as `\A` or `\00000A`. In your code, however, strings can span multiple lines, in which case each new line must be escaped with a `\` as the last character of the line.

However, to get new lines, you must also set the `white-space` property to appropriate value.

> **Note:** Character references (such as ` ` or `—`) cannot be used in a CSS `<string>`.

Examples

Examples of valid strings

```
/* Basic strings */
"This string is demarcated by double quotes."
'This string is demarcated by single quotes.'

/* Character escaping */
"This is a string with \" an escaped double quote."
"This string also has \22 an escaped double quote."
'This is a string with \' an escaped single quote.'
'This string also has \27 an escaped single quote.'
"This is a string with \\ an escaped backslash."

/* New line in a string */
"This string has a \Aline break in it."

/* String spanning two lines of code (these two strings will have identical output) */
"A really long \
awesome string"
"A really long awesome string"
```

See also

- CSS values and units module
- CSS basic data types
- Introduction to CSS: Values and units

superellipse() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`superellipse()`** CSS function defines the curvature of an ellipse, and is used to specify corner shapes either directly, or via `<corner-shape-value>` keywords.

Syntax

```
superellipse(infinity)
superellipse(4)
superellipse(1.7)
superellipse(0)
superellipse(-2.8)
superellipse(-3)
superellipse(-infinity)
```

Parameters

`<number>`

A number in the range of `-infinity` to `infinity`, inclusive.

Return value

A superellipse shape.

Description

The `superellipse()` function returns a superellipse shape, which is used to specify `corner-shape` values. A superellipse is a closed curve symmetric shape between a rectangle and an ellipse. It resembles an ellipse that retains the geometric features of its two axes.

The superellipse shape is calculated using a modified version of an ellipse. The following equation defines an ellipse centered at the origin:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

The `a` and `b` variables refer to the radii of the ellipse, while the `x` and `y` coordinates are points on the ellipse's circumference.

A circle is an ellipse where the radii, the `a` and `b` in the previous equation, are the same length. With `a` and `b` both equal to `r`, the equation for a circle can be written as:

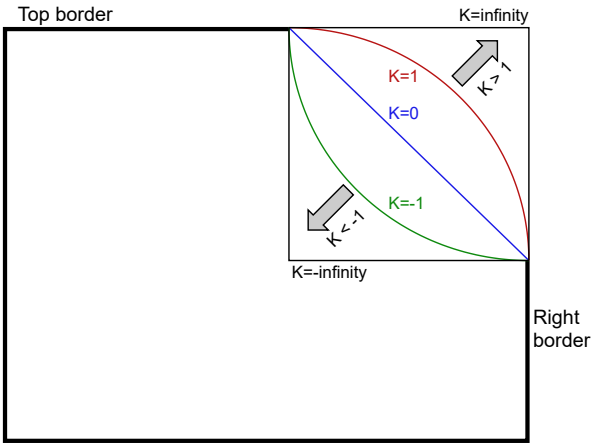
$$x^2 + y^2 = r^2$$

In this equation, the `x` and `y` are coordinates of points on the circle's circumference, and the `r` is the radius of the circle, with the center of the circle being `(0, 0)`. The ellipse is produced by scaling a circle shape along the `x` and/or `y` axis.

A superellipse shape is created by replacing the `2` exponent in each case with 2^K, `K` being the argument passed to the `superellipse()` function, which modifies the curvature of the ellipse:

$$x^{2^K} + y^{2^K} = 1$$

The following diagram illustrates different `superellipse()` values for the top right corner of a container: `infinity`, `1`, `0`, `-1`, and `-infinity`:



- A `κ` value of `0` creates a straight line. This value can be used to create beveled corners and corresponds to the `<corner-shape-value>` `bevel` keyword.
- A `κ` value of `1` creates an ordinary ellipse, corresponding to the `round` keyword.
- A `κ` value of `>1` makes the ellipse shape more square; `2` corresponds to the `squircle` keyword.
- A `κ` value of `infinity` creates a perfect square (corresponding to the `square` keyword), although `κ` values of `10` or more are virtually indistinguishable from a square.
- Negative `κ` values result in a concave curve, resulting in corner shapes that are curved inward, or "scooped out". A `κ` value of `-1` corresponds to the `scoop` keyword and `-infinity` corresponds to the `notch` keyword.

A negative or positive superellipse would appear symmetrical to a superellipse with its inverse value.

> **Note:** For any `κ` parameter value passed, the `superellipse()` function's return value will always be the same for that `κ` value. When that value is applied to two elements, the appearance of the corner shape may differ if the box size or `border-radius` values differ.

Formal syntax

```
<superellipse()> =
  superellipse( <number>  |
  infinity                |
  -infinity )
```

This syntax reflects the latest standard as per CSS Borders and Box Decorations Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

<code>superellipse()</code> value comparison

In this example, two `<input type="range">` sliders allow you to cycle through many different `corner-shape` `superellipse()` values and `border-radius` values enabling you to compare the effects of each on a container. The code is hidden for brevity, but the full explanation of the superellipse value comparison is provided on the `corner-shape` reference page.

```
<form>
  <div>
    <label for="superellipse-slider">Choose a superellipse() value:</label>
    <input
      type="range"
      id="superellipse-slider"
      min="-5"
      value="0"
      max="5"
      step="0.1" />
  </div>
  <div>
    <label for="radius-slider">Choose a border-radius value:</label>
    <input type="range" id="radius-slider" min="0" value="45" max="90" />
  </div>
</form>
<section></section>
```

```
html {
  font-family: "Helvetica", "Arial", sans-serif;
}

body {
  width: fit-content;
  margin: 20px auto;
}

section {
  display: flex;
  justify-content: center;
  align-items: center;
  flex: 100%;
  margin-top: 20px;
}

form div {
  margin-top: 5px;
  display: flex;
}

section {
  width: 100%;
  height: 180px;
  background-color: orange;
```

```
  background-image: linear-gradient(
    to bottom,
    rgb(255 255 255 / 0),
    rgb(255 255 255 / 0.5)
  );
}

section {
  box-shadow: 1px 1px 3px gray;
}
```

```
const rectangle = document.querySelector("section");
const superEllipseRange = document.getElementById("superellipse-slider");
const borderRadiusRange = document.getElementById("radius-slider");

function setCorners() {
  const seValue = `superellipse(${superEllipseRange.value})`;
  rectangle.style.cornerShape = seValue;
  const brValue = `${borderRadiusRange.value}px`;
  rectangle.style.borderRadius = brValue;
  rectangle.innerHTML = `<div><code>corner-shape: ${seValue};</code><br><code>border-radius: ${brValue};</code></div>`;
}

superEllipseRange.addEventListener("input", setCorners);
borderRadiusRange.addEventListener("input", setCorners);
setCorners();
```

> **Note:** See also the `<corner-shape-value>` value comparison example.

See also

- `corner-shape`
- `<corner-shape-value>`
- CSS borders and box decorations module

symbols() CSS function

The **`symbols()`** CSS function enables defining counter styles inline, directly as a value of properties such as `list-style`, providing a less powerful but simpler alternative to the `@counter-style` method of defining a counter style.

Unlike `@counter-style`, which defines a reusable counter style, `symbols()` is *anonymous* (i.e., it can only be used once). This function accepts strings and images as values. In comparison, the `@counter-style`'s `symbols` descriptor also accepts identifiers.

Syntax

```
symbols() = symbols( <symbols-type>? [ <string> | <image> ]+ );
```

`<symbols-type>` can be one of the following:

- `cyclic`: The system cycles through the given values in the order of their definition, and returns to the start when it reaches the end.
- `numeric`: The system interprets the given values as the successive units of a place-value numbering system.
- `alphabetic`: The system interprets the given values as the digits of an alphabetic numbering system, like a place-value numbering system but without `0`.
- `symbolic`: The system cycles through the values, printing them an additional time at each cycle (one time for the first cycle, two times for the second, etc.).
- `fixed`: The system cycles through the given values once, then falls back to Arabic numerals.

Formal syntax

```
<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
```

```
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

This syntax reflects the latest standard as per CSS Counter Styles Level 3, CSS Images Module Level 4, CSS Images Module Level 5, Selectors Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<ol>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
  <li>Four</li>
  <li>Five</li>
</ol>
```

CSS

```
ol {
  list-style: symbols(cyclic "*" "†" "‡");
}
```

Result

See also

- `@counter-style` descriptors: `system`, `symbols`, `additive-symbols`, `prefix`, `suffix`, `range`, `pad`, `speak-as`, `fallback`
- List style properties: `list-style`, `list-style-type`
- CSS counter styles module
- CSS lists and counters module

<system-color> CSS type

The **`<system-color>`** CSS data type usually reflects the default color choices used for the different parts of a web page.

However, user agents can provide an accessibility feature called *forced colors mode*, in which colors are restricted into a user- and user agent-defined palette, overriding the author's choice of colors in certain properties. In forced colors mode, `<system-color>` exposes the chosen colors, so that the rest of the page can integrate with them. An example of forced colors mode is high contrast mode on Windows.

In forced colors mode, authors should use colors from the `<system-color>` type for all properties that are *not* in the set of properties whose colors are overridden. This ensures that the page consistently uses the same color palette across all properties.

Authors can detect forced colors mode using the `forced-colors` media feature.

A `<system-color>` value can be used anywhere a `<color>` can be used.

Syntax

Note that these keywords are *case insensitive*, but are listed here with mixed case for readability.

Depending on your settings, the sample colors displayed in the table may change. You can also view this page with different browsers, operating systems, and system settings, to check the differences.

Keyword	Description	Sample
`AccentColor`	Background of accented user interface controls.	
`AccentColorText`	Text of accented user interface controls.	
`ActiveText`	Text of active links.	
`ButtonBorder`	Base border color of controls.	
`ButtonFace`	Background color of controls.	
`ButtonText`	Text color of controls.	
`Canvas`	Background of application content or documents.	
`CanvasText`	Text color in application content or documents.	
`Field`	Background of input fields.	
`FieldText`	Text in input fields.	
`GrayText`	Text color for disabled items (for example, a disabled control).	
`Highlight`	Background of selected items.	
`HighlightText`	Text color of selected items.	
`LinkText`	Text of non-active, non-visited links.	
`Mark`	Background of text that has been specially marked (such as by the HTML `mark` element).	
`MarkText`	Text that has been specially marked (such as by the HTML `mark` element).	
`SelectedItem`	Background of selected items, for example, a selected checkbox.	
`SelectedItemText`	Text of selected items.	
`VisitedText`	Text of visited links.	

> **Note:** To reduce the risk of fingerprinting, some browsers return a fixed value for `AccentColor` and `AccentColorText` unless they are used in certain restricted circumstances. See browser compatibility for details.

Deprecated system color keywords

The following keywords were defined in earlier versions of the CSS Color Module. They are now deprecated for use on public web pages.

Keyword	Description	Replacement	Sample
`ActiveBorder`	Active window border	`ButtonBorder`	
`ActiveCaption`	Active window caption. Should be used with `CaptionText` as foreground color.	`Canvas`	
`AppWorkspace`	Background color of multiple document interface.	`Canvas`	
`Background`	Desktop background.	`Canvas`	
`ButtonHighlight`	The color of the border facing the light source for 3-D elements that appear 3-D due to that layer of surrounding border.	`ButtonFace`	
`ButtonShadow`	The color of the border away from the light source for 3-D elements that appear 3-D due to that layer of surrounding border.	`ButtonFace`	
`CaptionText`	Text in caption, size box, and scrollbar arrow box. Should be used with the `ActiveCaption` background color.	`CanvasText`	
`InactiveBorder`	Inactive window border.	`ButtonBorder`	
`InactiveCaption`	Inactive window caption. Should be used with the `InactiveCaptionText` foreground color.	`Canvas`	
`InactiveCaptionText`	Color of text in an inactive caption. Should be used with the `InactiveCaption` background color.	`GrayText`	
`InfoBackground`	Background color for tooltip controls. Should be used with the `InfoText` foreground color.	`Canvas`	
`InfoText`	Text color for tooltip controls. Should be used with the `InfoBackground` background color.	`CanvasText`	
`Menu`	Menu background. Should be used with the `MenuText` or `-moz-MenuBarText` foreground color.	`Canvas`	
`MenuText`	Text in menus. Should be used with the `Menu` background color.	`CanvasText`	
`Scrollbar`	Background color of scroll bars.	`Canvas`	
`ThreeDDarkShadow`	The color of the darker (generally outer) of the two borders away from the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.	`ButtonBorder`	
`ThreeDFace`	The face background color for 3-D elements that appear 3-D due to two concentric layers of surrounding border. Should be used with the `ButtonText` foreground color.	`ButtonFace`	
`ThreeDHighlight`	The color of the lighter (generally outer) of the two borders facing the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.	`ButtonBorder`	
`ThreeDLightShadow`	The color of the darker (generally inner) of the two borders facing the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.	`ButtonBorder`	
`ThreeDShadow`	The color of the lighter (generally inner) of the two borders away from the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.	`ButtonBorder`	
`Window`	Window background. Should be used with the `WindowText` foreground color.	`Canvas`	
`WindowFrame`	Window frame.	`ButtonBorder`	
`WindowText`	Text in windows. Should be used with the `Window` background color.	`CanvasText`	

Examples

Using system colors

In this example we have a button that normally gets its contrast using the `box-shadow` property. In forced colors mode, `box-shadow` is forced to `none`, so the example uses the `forced-colors` media feature to ensure there is a border of the appropriate color (`ButtonBorder` in this case).

HTML

```
<button class="button">Press me!</button>
```

CSS

```
.button {
  border: 0;
  padding: 10px;
  box-shadow:
    -2px -2px 5px gray,
    2px 2px 5px gray;
}

@media (forced-colors: active) {
  .button {
    /* Use a border instead, since box-shadow
    is forced to 'none' in forced-colors mode */
    border: 2px ButtonBorder solid;
  }
}
```

Result

See also

- `<color>`: the data type these keywords belong to

tan() CSS function

The **`tan()`** CSS function is a trigonometric function that returns the tangent of a number, which is a value between `-infinity` and `infinity`. The function contains a single calculation that must resolve to either a `<number>` or an `<angle>` by interpreting the result of the argument as radians.

Syntax

```
/* Single <angle> values */
width: calc(100px * tan(45deg));
width: calc(100px * tan(0.125turn));
width: calc(100px * tan(0.785398163rad));

/* Single <number> values */
width: calc(100px * tan(0.5773502));
width: calc(100px * tan(1.732 - 1));

/* Other values */
```

```
width: calc(100px * tan(pi / 3));
width: calc(100px * tan(e));
```

Parameters

The `tan(angle)` function accepts only one value as its parameter.

`angle`

A calculation which resolves to a `<number>` or an `<angle>`. When specifying unitless numbers they are interpreted as a number of radians, representing an `<angle>`.

Return value

The tangent of an `angle` will always return a number between −∞ and +∞.

- If `angle` is `infinity`, `-infinity`, or `NaN`, the result is `NaN`.
- If `angle` is `0`−, the result is `0`−.
- If `angle` is one of the asymptote values (such as `90deg`, `270deg`, etc.), the result is *explicitly undefined*. Authors *must not* rely on `tan()` returning any particular value for these inputs.

Formal syntax

```
<tan()> =
  tan( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Drawing parallelograms

The `tan()` function can be used to draw a parallelogram with a given bounding box.

HTML

```
<div class="parallelogram"></div>
```

CSS

```
body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

```
.parallelogram {
  --w: 400;
  --h: 200;
  --angle: 30deg;
  position: relative;
  width: calc(1px * var(--w));
  height: calc(1px * var(--h));
}
.parallelogram::before {
  content: "";
  position: absolute;
  width: calc(100% - 100% * var(--h) / var(--w) * tan(var(--angle)));
  height: 100%;
  transform-origin: 0 100%;
  transform: skewX(calc(0deg - var(--angle)));
  background-color: red;
}
```

Result

See also

- `sin()`
- `cos()`
- `asin()`
- `acos()`
- `atan()`
- `atan2()`
- Using CSS typed arithmetic

<text-edge> CSS type

The **`<text-edge>`** enumerated data type defines keywords that specify font metrics representing specific regions on a font's block-start edge and block-end edge. Each keyword specifies a position of a font's over and/or under edge.

The `<text-edge>` values are used in the `text-box-edge` property to specify an amount of space to trim from the block-start and block-end edge of a text element's block container.

Syntax

The `<text-edge>` data type is composed of one or two keywords representing specific regions on a font's block-start (over) edge and/or block-end (under) edge:

- When one value is specified, the position of the font's over edge and under edge are specified using that same keyword.
- When two values are specified, the first value specifies the position of the font's over edge, and the second value specifies the position of the font's under edge.

Values

Single keyword values

`text`

The font's over and under edges are its text-over baseline/text-under baseline: this includes the font's ascenders and descenders but excludes the half-leading set on the text.

> **Note:** The amount of half-leading included on a text element can be controlled using the `line-height` property.

> **Note:** The `ideographic` and `ideographic-ink` keywords are intended to specify over and under edge positions specific to CJK language characters. Currently their exact behavior is being debated and they are not supported by any browser.

Two keyword values

`alphabetic`

The font's under edge is its alphabetic baseline, which is the bottom of its short lower-case letters (for example, "m", "n", and "o") or capital letters.

`cap`

The font's over edge is its cap-height baseline, which is the top of its capital letters.

`ex`

The font's over edge is its x-height baseline, which is the top of its short lower-case letters.

`text`

The font's over edge is its text-over baseline (includes the font's ascenders but excludes the over edge half-leading), or its under edge is its text-under baseline (includes the font's descenders but excludes the under edge half-leading), depending on which edge the value is set for.

Formal syntax

```
<text-edge> =
  [ text | ideographic | ideographic-ink ]           |
  [ text | ideographic | ideographic-ink | cap | ex ] [ text | ideographic | ideographic-ink | alphabetic ]
```

Examples

See `text-box-edge` examples

See also

- `text-box`, `text-box-edge`, `text-box-trim`
- CSS inline layout module

<time> CSS type

The **`<time>`** CSS data type represents a time value expressed in seconds or milliseconds. It is used in `animation`, `transition`, and related properties.

Syntax

The `<time>` data type consists of a `<number>` followed by one of the units listed below. Optionally, it may be preceded by a single `+` or `-` sign. As with all dimensions, there is no space between the unit literal and the number.

> **Note:** Although the number `0` is always the same regardless of unit, the unit may not be omitted. In other words, `0` is invalid and does not represent `0s` or `0ms`.

Units

`s`

Represents a time in seconds. Examples: `0s`, `1.5s`, `-60s`.

`ms`

Represents a time in milliseconds. Examples: `0ms`, `150.25ms`, `-60000ms`.

> **Note:** Conversion between `s` and `ms` follows the logical `1s = 1000ms`.

Examples

Valid times

```
12s         Positive integer
-456ms      Negative integer
4.3ms       Non-integer
14mS        The unit is case-insensitive, although capital letters are not recommended.
+0s         Zero with a leading + and a unit
-0ms        Zero with a leading - and a unit
```

Invalid times

```
0           Although unitless zero is allowed for <length>s, it's invalid for <time>s.
12.0        This is a <number>, not a <time>, because it's missing a unit.
7 ms        No space is allowed between the number and the unit.
```

See also

- `<time-percentage>`
- CSS values and units module

<time-percentage> CSS type

The **`<time-percentage>`** CSS data type represents a value that can be either a `<time>` or a `<percentage>`.

Syntax

Refer to the documentation for `<time>` and `<percentage>` for details of the individual syntaxes allowed by this type.

Formal syntax

```
<time-percentage> =
  <time>       |
  <percentage>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Use in calc()

Where a `<time-percentage>` is specified as an allowable type, this means that the percentage resolves to a time and therefore can be used in a `calc()` expression.

Valid percentages

```
50%
+50%       Optional plus sign
-50%       Negative percentages are not valid for all properties that accept percentages
```

Invalid percentages

```
50 %       Space not allowed between the number and the percentage sign
```

Valid times

```
12s        Positive integer
-456ms     Negative integer
4.3ms      Non-integer
14mS       The unit is case-insensitive, although capital letters are not recommended.
+0s        Zero with a leading + and a unit
-0ms       Zero with a leading - and a unit
```

Invalid times

```
0          Although unitless zero is allowed for <length>s, it's invalid for <time>s.
12.0       This is a <number>, not a <time>, because it's missing a unit.
7 ms       No space is allowed between the number and the unit.
```

See also

- `<percentage>`
- `<time>`
- CSS values and units module

<timeline-range-name> CSS type

The **`<timeline-range-name>`** enumerated data type is a CSS identifier representing one of the predefined named timeline ranges inside a view progress timeline.

The `<timeline-range-name>` keyword values are used in keyframe selectors and the following longhand and shorthand properties:

- `animation-range-end`
- `animation-range-start`
- `animation-range` shorthand
- `timeline-trigger-activation-range-end`
- `timeline-trigger-activation-range-start`
- `timeline-trigger-activation-range` shorthand
- `timeline-trigger-active-range-end`
- `timeline-trigger-active-range-start`
- `timeline-trigger-active-range` shorthand

Syntax

Valid `<timeline-range-name>` values:

`cover`

> Represents the full range of a view progress timeline, from the point where the subject element's start border edge first enters the scrollport's view progress visibility range (`0%` progress) to the point where the end border edge has completely left it (`100%` progress). This is the default range for view progress timelines.

`contain`

> Represents the range of a view progress timeline where the subject element is fully contained by, or fully contains, the view progress visibility range within the scrollport
>
> - If the subject element is smaller than the scrollport, it ranges from the point where the subject element is first completely contained by the scroll port (`0%` progress), to the point where it is no longer completely contained by the scroll port (`100%` progress).
> - If the subject element is larger than the scrollport, it ranges from the point where the subject element first completely covers the scroll port (`0%` progress), to the point where it no longer completely covers the scrollport (`100%` progress).

`entry`

> Represents the range of a view progress timeline from the point where the subject element first starts to enter the scroll port, to the point where it has completely entered the scrollport. `0%` is equivalent to `0%` of the `cover` range. `100%` is equivalent to `0%` of the `contain` range.

`exit`

Represents the range of a view progress timeline from the point where the subject element first starts to exit the scroll port, to the point where it has completely exited the scrollport. `0%` is equivalent to `100%` of the `contain` range. `100%` is equivalent to `100%` of the `cover` range.

`entry-crossing`

Represents the range during which the principal box crosses the end border edge. The start (0% progress) of the range occurs when the start border edge of the element's principal box coincides with the end edge of its view progress visibility range. The end (100%) of the range is the point at which the end border edge of the element's principal box coincides with the end edge of its view progress visibility range. The size of the range is the size of the element's principal box in the scroll direction.

`exit-crossing`

Represents the range where the principal box crosses the start border edge. The range start (0% progress) occurs when the start border edge of the element's principal box coincides with the start edge of its view progress visibility range. The range end (100% progress) is the point at which the end border edge of the element's principal box coincides with the start edge of its view progress visibility range. The size of the range is the size of the element's principal box in the scroll direction.

`scroll`

Represents the full range of the scroll container on which the view progress timeline is defined. The range start (0% progress) and end (100% progress) occur at the very start and end positions of the scroll container underlying the view progress timeline. This is the default range for scroll progress timelines.

Formal syntax

```
<timeline-range-name> =
  cover           |
  contain         |
  entry           |
  exit            |
  entry-crossing  |
  exit-crossing   |
  scroll
```

Examples

See the View timeline range visualizer.

See also

- `animation-range-start`, `animation-range-end`, `animation-range`
- `animation-timeline`
- `scroll-timeline`
- `view-timeline-inset`
- `scroll()`, `view()`
- Understanding timeline range names
- Scroll-driven animation timelines
- CSS scroll-driven animations module
- CSS animation triggers module
- View timeline range visualizer

<transform-function> CSS type

The **`<transform-function>`** CSS data type represents a transformation that affects an element's appearance. Transformation functions can rotate, resize, distort, or move an element in 2D or 3D space. It is used in the `transform` property.

Syntax

The `<transform-function>` data type is specified using one of the transformation functions listed below. Each function applies a geometric operation in either 2D or 3D.

Matrix transformation

`matrix()`

Describes a homogeneous 2D transformation matrix.

`matrix3d()`

Describes a 3D transformation as a 4×4 homogeneous matrix.

Perspective

`perspective()`

Sets the distance between the user and the z=0 plane.

Rotation

`rotate()`

Rotates an element around a fixed point on the 2D plane.

`rotate3d()`

Rotates an element around a fixed axis in 3D space.

`rotateX()`

Rotates an element around the horizontal axis.

`rotateY()`

Rotates an element around the vertical axis.

`rotateZ()`

Rotates an element around the z-axis.

Scaling (resizing)

`scale()`

Scales an element up or down on the 2D plane.

`scale3d()`

Scales an element up or down in 3D space.

`scaleX()`

Scales an element up or down horizontally.

`scaleY()`

Scales an element up or down vertically.

`scaleZ()`

Scales an element up or down along the z-axis.

Skewing (distortion)

`skew()`

Skews an element on the 2D plane.

`skewX()`

Skews an element in the horizontal direction.

`skewY()`

Skews an element in the vertical direction.

Translation (moving)

`translate()`

Translates an element on the 2D plane.

`translate3d()`

Translates an element in 3D space.

`translateX()`

Translates an element horizontally.

`translateY()`

Translates an element vertically.

`translateZ()`

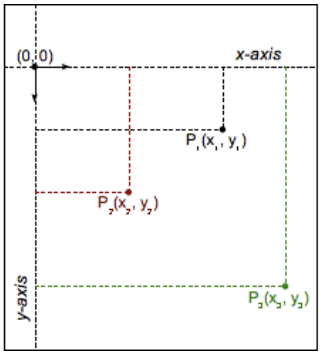
Translates an element along the z-axis.

Description

Various coordinate models can be used to describe an HTML element's size and shape, as well as any transformations applied to it. The most common is the Cartesian coordinate system, although homogeneous coordinates are also sometimes used.

Cartesian coordinates

In the Cartesian coordinate system, a two-dimensional point is described using two values: an x coordinate (abscissa) and a y coordinate (ordinate). This is represented by the vector notation `(x, y)`.



In CSS (and most computer graphics), the origin `(0, 0)` represents the *top-left* corner of any element. Positive coordinates are down and to the right of the origin, while negative ones are up and to the left. Thus, a point that's 2 units to the right and 5 units down would be `(2, 5)`, while a point 3 units to the left and 12 units up would be `(-3, -12)`.

Transformation functions

Transformation functions alter the appearance of an element by manipulating the values of its coordinates. A linear transformation function is described using a 2×2 matrix, like this:

$$\begin{pmatrix} a & c \\ b & d \end{pmatrix}$$

The function is applied to an element by using matrix multiplication. Thus, each coordinate changes based on the values in the matrix:

$$\begin{pmatrix} a & c \\ b & d \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax + cy \\ bx + dy \end{pmatrix}$$

It is even possible to apply several transformations in a row:

$$\begin{pmatrix} a_1 & c_1 \\ b_1 & d_1 \end{pmatrix}\begin{pmatrix} a_2 & c_2 \\ b_2 & d_2 \end{pmatrix} = \begin{pmatrix} a_1a_2 + c_1b_2 & a_1c_2 + c_1d_2 \\ b_1a_2 + d_1b_2 & b_1c_2 + d_1d_2 \end{pmatrix}$$

With this notation, it is possible to describe, and therefore compose, most common transformations: rotations, scaling, or skewing. (In fact, all transformations that are linear functions can be described.) Composite transformations are effectively applied in order from right to left.

However, one major transformation is not linear, and therefore must be special-cased when using this notation: translation. The translation vector `(tx, ty)` must be expressed separately, as two additional parameters.

> **Note:** Though trickier than Cartesian coordinates, homogeneous coordinates in projective geometry lead to 3×3 transformation matrices, and can express translations as linear functions.

> **Note:** Transform functions are used with the `transform` property but not with individual transform properties-`translate`, `scale`, and `rotate`.

Examples

Transform function comparison

The following example provides a 3D cube created from DOM elements and transforms, and a select menu allowing you to choose different transform functions to transform the cube with, so you can compare the effects of the different types.

Choose one, and the transform is applied to the cube; after 2 seconds, the cube reverts to its starting state. The cube's starting state is slightly rotated using `transform3d()`, to allow you to see the effect of all the transforms.

HTML

```
<main>
  <section id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </section>

  <div class="select-form">
    <label for="transfunction">Select a transform function</label>
    <select id="transfunction">
      <option selected>Choose a function</option>
      <option>rotate(360deg)</option>
      <option>rotateX(360deg)</option>
      <option>rotateY(360deg)</option>
      <option>rotateZ(360deg)</option>
      <option>rotate3d(1, 1, 1, 90deg)</option>
      <option>scale(1.5)</option>
      <option>scaleX(1.5)</option>
      <option>scaleY(1.5)</option>
      <option>scaleZ(1.5)</option>
      <option>scale3d(1, 1.5, 1.5)</option>
      <option>skew(17deg, 13deg)</option>
      <option>skewX(17deg)</option>
      <option>skewY(17deg)</option>
      <option>translate(100px, 100px)</option>
      <option>translateX(100px)</option>
      <option>translateY(100px)</option>
      <option>translateZ(100px)</option>
      <option>translate3d(50px, 50px, 50px)</option>
      <option>perspective(200px)</option>
      <option>matrix(1, 2, -1, 1, 80, 80)</option>
      <option>matrix3d(1,0,0,0,0,1,3,0,0,0,1,0,50,100,0,1.1)</option>
    </select>
  </div>
</main>
```

CSS

```
main {
  width: 400px;
  height: 200px;
  padding: 50px;
  background-image: linear-gradient(135deg, white, cyan, white);
}

#example-element {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  transition: transform 1.5s;
  transform: rotate3d(1, 1, 1, 30deg);
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 70%);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 70%);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 70%);
```

```
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 70%);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 70%);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 70%);
  transform: rotateX(-90deg) translateZ(50px);
}

.select-form {
  margin-top: 50px;
}
```

JavaScript

```
const selectElem = document.querySelector("select");
const example = document.querySelector("#example-element");

selectElem.addEventListener("change", () => {
  if (selectElem.value === "Choose a function") {
    return;
  }
  example.style.transform = `rotate3d(1, 1, 1, 30deg) ${selectElem.value}`;
  setTimeout(() => {
    example.style.transform = "rotate3d(1, 1, 1, 30deg)";
  }, 2000);
});
```

Result

See also

- CSS `transform` property
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`

matrix() CSS function

The **`matrix()`** CSS function defines a homogeneous 2D transformation matrix. Its result is a `<transform-function>` data type.

> **Note:** The `matrix(a, b, c, d, tx, ty)` function is a shorthand for `matrix3d(a, b, 0, 0, c, d, 0, 0, 0, 0, 1, 0, tx, ty, 0, 1)`.

Try it

```
transform: matrix(1.2, 0.2, -1, 0.9, 0, 20);
```

```
transform: matrix(0.4, 0, 0.5, 1.2, 60, 10);
```

```
transform: matrix(0, 1, 1, 0, 0, 0);
```

```
transform: matrix(0.1, 1, -0.3, 1, 0, 0);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
matrix(a, b, c, d, tx, ty)
```

Values

The `matrix()` function is specified with six values. The constant values are implied and not passed as parameters; the other parameters are described in the column-major order.

a b c d

Are `<number>`s describing the linear transformation.

tx ty

Are `<number>`s describing the translation to apply.

Cartesian coordinates on ℝ^2	Homogeneous coordinates on ℙ^2	Cartesian coordinates on ℝ^3	Homogeneous coordinates on ℙ^3
$\begin{pmatrix} a & c \\ b & d \end{pmatrix}$	$\begin{pmatrix} a & c & \text{tx} \\ b & d & \text{ty} \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} a & c & \text{tx} \\ b & d & \text{ty} \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} a & c & 0 & \text{tx} \\ b & d & 0 & \text{ty} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
	`[a b c d tx ty]`		

The values represent the following functions: `matrix(scaleX(), skewY(), skewX(), scaleY(), translateX(), translateY())`.

Formal syntax

```
<matrix()> =
  matrix( <number>#{6} )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="changed">Changed</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.changed {
  transform: matrix(1, 2, -1, 1, 80, 80);
  background-color: pink;
}
```

Result

See also

- `transform`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
- `<transform-function>`
- `matrix3d()`

matrix3d() CSS function

The **`matrix3d()`** CSS function defines a 3D transformation as a 4x4 homogeneous matrix. Its result is a `<transform-function>` data type.

Try it

```
transform: matrix3d(
  -0.6,
  1.34788,
  0,
  0,
  -2.34788,
  -0.6,
  0,
  0,
  0,
  0,
  1,
  0,
  0,
  0,
  10,
  1
);
```

```
transform: matrix3d(
  0.5,
  0,
  -0.866025,
  0,
  0.595877,
  1.2,
  -1.03209,
  0,
  0.866025,
  0,
  0.5,
  0,
  25.9808,
  0,
  15,
  1
);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Syntax

```
matrix3d(a1, b1, c1, d1, a2, b2, c2, d2, a3, b3, c3, d3, a4, b4, c4, d4)
```

Values

The `matrix3d()` function is specified with 16 values. They are described in the column-major order.

a1 b1 c1 d1 a2 b2 c2 d2 a3 b3 c3 d3

Are `<number>`s describing the linear transformation.

a4 b4 c4 d4

Are `<number>`s describing the translation to apply.

Cartesian coordinates on ℝ^2	**Homogeneous coordinates on ℝP^2**	**Cartesian coordinates on ℝ^3**	**Homogeneous coordinates on ℝP^3**
This transformation applies to the 3D space and can't be represented on the plane.		A generic 3D affine transformation can't be represented using a Cartesian-coordinate matrix, as translations are not linear transformations.	$\begin{pmatrix} a1 & a2 & a3 & a4 \\ b1 & b2 & b3 & b4 \\ c1 & c2 & c3 & c4 \\ d1 & d2 & d3 & d4 \end{pmatrix}$

Formal syntax

```
<matrix3d()> =
  matrix3d( <number>#{16} )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Cube squashing example

The following example shows a 3D cube created from DOM elements and transforms, which can be hovered/focused to apply a `matrix3d()` transform to it.

HTML

```
<section id="example-element" tabindex="0">
  <div class="face front">1</div>
  <div class="face back">2</div>
  <div class="face right">3</div>
  <div class="face left">4</div>
  <div class="face top">5</div>
  <div class="face bottom">6</div>
</section>
```

CSS

```
#example-element {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  transition: transform 1.5s;
  transform: rotate3d(1, 1, 1, 30deg);
  margin: 50px auto;
}

#example-element:hover,
#example-element:focus {
  transform: rotate3d(1, 1, 1, 30deg)
    matrix3d(1, 0, 0, 0, 0, 1, 6, 0, 0, 0, 1, 0, 50, 100, 0, 1.1);
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 70%);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 70%);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 70%);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 70%);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 70%);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 70%);
  transform: rotateX(-90deg) translateZ(50px);
}
```

Result

Matrix translation and scale example

Another `transform3d()` example, which implements an animated combined translate and scale.

HTML

```
<div class="foo">
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. Quos quaerat sit
  soluta, quisquam exercitationem delectus qui unde in facere necessitatibus aut
  quia porro dolorem nesciunt enim, at consequuntur aliquam esse?
</div>
```

CSS

```
html {
  width: 100%;
}
body {
  height: 100vh;
  /* Centering content */
  display: flex;
  flex-flow: row wrap;
  justify-content: center;
  align-content: center;
}
.foo {
  width: 50%;
  padding: 1em;
  color: white;
  background: #ff8c66;
  border: 2px dashed black;
  text-align: center;
  font-family: system-ui, sans-serif;
  font-size: 14px;
  /* Setting up animation for better demonstration */
  animation: MotionScale 2s alternate linear infinite;
}

@keyframes MotionScale {
  0% {
    /*
      Identity matrix is used as basis here.
      The matrix below describes the
      following transformations:
        Translates every X point by -50px
        Translates every Y point by -100px
        Translates every Z point by 0
        Scales down by 10%
    */
    /* prettier-ignore */
    transform: matrix3d(
      1, 0, 0, 0,
      0, 1, 0, 0,
      0, 0, 1, 0,
      -50, -100, 0, 1.1
    );
  }
  50% {
    /* prettier-ignore */
    transform: matrix3d(
      1, 0, 0, 0,
      0, 1, 0, 0,
      0, 0, 1, 0,
      0, 0, 0, 0.9
    );
  }
  100% {
    /* prettier-ignore */
    transform: matrix3d(
      1, 0, 0, 0,
      0, 1, 0, 0,
      0, 0, 1, 0,
      50, 100, 0, 1.1
    )
  }
}
```

Result

See also

- `transform`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
- `<transform-function>`

perspective() CSS function

The **`perspective()`** CSS function defines a transformation that sets the distance between the user and the z=0 plane, the perspective from which the viewer would be if the 2-dimensional interface were 3-dimensional. Its result is a `<transform-function>` data type.

Try it

```
transform: perspective(0);
```

```
transform: perspective(none);
```

```
transform: perspective(800px);
```

```
transform: perspective(23rem);
```

```
transform: perspective(6.5cm);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

The `perspective()` transform function is part of the `transform` value applied on the element being transformed. This differs from the `perspective` and `perspective-origin` properties which are attached to the parent of a child transformed in 3-dimensional space.

Syntax

```
perspective(d)
```

Values

d

Is a `<length>` representing the distance from the user to the z=0 plane. The z=0 plane is the plane where everything appears in a 2-dimensional view, or the screen. Values smaller than `1px` (including zero) are clamped to `1px`. Negative values are syntax errors.

Values other than `none` cause elements with positive z positions to appear larger, and elements with negative z positions to appear smaller. Elements with z positions equal to or larger than the perspective value disappear as though they are behind the user. Large values of perspective represent a small transformation; small values of `perspective()` represent a large transformation; `perspective(none)` represents perspective from infinite distance and no transformation.

Cartesian coordinates on $\mathbb{R}^2$	**Homogeneous coordinates on $\mathbb{RP}^2$**	**Cartesian coordinates on $\mathbb{R}^3$**	**Homogeneous coordinates on $\mathbb{RP}^3$**
This transformation applies to the 3D space and can't be represented on the plane.		This transformation is not a linear transformation in $\mathbb{R}^3$, and can't be represented using a Cartesian-coordinate matrix.	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 1 \end{pmatrix}$

Formal syntax

```
<perspective()> =
  perspective( [ <length [0,∞]> | none ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<p>Without perspective:</p>
<div class="no-perspective-box">
  <div class="face front">A</div>
  <div class="face top">B</div>
  <div class="face left">C</div>
</div>

<p>With perspective (9cm):</p>
<div class="perspective-box-far">
  <div class="face front">A</div>
  <div class="face top">B</div>
  <div class="face left">C</div>
</div>

<p>With perspective (4cm):</p>
<div class="perspective-box-closer">
  <div class="face front">A</div>
  <div class="face top">B</div>
  <div class="face left">C</div>
</div>
```

CSS

```
.face {
  position: absolute;
  width: 100px;
  height: 100px;
  line-height: 100px;
  font-size: 100px;
  text-align: center;
}

p + div {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  margin-left: 100px;
}
.no-perspective-box {
  transform: rotateX(-15deg) rotateY(30deg);
}

.perspective-box-far {
  transform: perspective(9cm) rotateX(-15deg) rotateY(30deg);
}

.perspective-box-closer {
  transform: perspective(4cm) rotateX(-15deg) rotateY(30deg);
}

.top {
  background-color: skyblue;
  transform: rotateX(90deg) translate3d(0, 0, 50px);
}

.left {
  background-color: pink;
  transform: rotateY(-90deg) translate3d(0, 0, 50px);
}

.front {
  background-color: limegreen;
  transform: translate3d(0, 0, 50px);
}
```

Result

See also

- `transform`
- `<transform-function>`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`

rotate() CSS function

The **`rotate()`** CSS function defines a transformation that rotates an element around a fixed point on the 2D plane, without deforming it. Its result is a `<transform-function>` data type.

Try it

```
transform: rotate(0);
```

```
transform: rotate(90deg);
```

```
transform: rotate(-0.25turn);
```

```
transform: rotate(3.142rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

The fixed point that the element rotates around — mentioned above — is also known as the **transform origin**. This defaults to the center of the element, but you can set your own custom transform origin using the `transform-origin` property.

> **Note:** The SVG `transform` attribute has its own grammar, in which `rotate()` accepts two optional extra values giving the center of rotation, as in `rotate(45, 50, 50)`. That form is only valid in the attribute, not in CSS: the CSS `rotate()` function takes an angle alone, and the center is set with `transform-origin`.

Syntax

```
rotate(a)
```

Values

a

Is an `<angle>` representing the angle of the rotation. The direction of rotation depends on the writing direction. In a left-to-right context, a positive angle denotes a clockwise rotation, a negative angle a counter-clockwise one. In a right-to-left context, a positive angle denotes a counter-clockwise rotation, a negative angle a clockwise one. A rotation by 180° is called *point reflection*.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
$\begin{pmatrix} \cos(a) & -\sin(a) \\ \sin(a) & \cos(a) \end{pmatrix}$	$\begin{pmatrix} \cos(a) & -\sin(a) & 0 \\ \sin(a) & \cos(a) & 0 \\ 0 & 0 & 1 \end{pmatrix}$ `[cos(a) sin(a) -sin(a) cos(a) 0 0]`	$\begin{pmatrix} \cos(a) & -\sin(a) & 0 \\ \sin(a) & \cos(a) & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} \cos(a) & -\sin(a) & 0 & 0 \\ \sin(a) & \cos(a) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<rotate()> =
  rotate( [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Basic example

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotate(45deg); /* Equal to rotateZ(45deg) */
  background-color: pink;
}
```

Result

Combining rotation with another transformation

If you want to apply multiple transformations to an element, be careful about the order in which you specify your transformations. For example, if you rotate before translating, the translation will be along the new axis of rotation!

HTML

```
<div>Normal</div>
<div class="rotate">Rotated</div>
<div class="rotate-translate">Rotated + Translated</div>
<div class="translate-rotate">Translated + Rotated</div>
```

CSS

```
div {
  position: absolute;
  left: 40px;
  top: 40px;
  width: 100px;
  height: 100px;
  background-color: lightgray;
}

.rotate {
  background-color: transparent;
  outline: 2px dashed;
  transform: rotate(45deg);
}

.rotate-translate {
  background-color: pink;
```

```
  transform: rotate(45deg) translateX(180px);
}

.translate-rotate {
  background-color: gold;
  transform: translateX(180px) rotate(45deg);
}
```

Result

See also

- `transform` property
- `rotate` property
- `<transform-function>`
- `rotate3d()`

rotate3d() CSS function

The **`rotate3d()`** CSS function defines a transformation that rotates an element around a fixed axis in 3D space, without deforming it. Its result is a `<transform-function>` data type.

Try it

```
transform: rotate3d(0, 0, 0, 0);
```

```
transform: rotate3d(1, 1, 1, 45deg);
```

```
transform: rotate3d(2, -1, -1, -0.2turn);
```

```
transform: rotate3d(0, 1, 0.5, 3.142rad);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 550px;
}

#example-element {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

In 3D space, rotations have three degrees of freedom, which together describe a single axis of rotation. The axis of rotation is defined by an [x, y, z] vector and pass by the origin (as defined by the `transform-origin` property). If, as specified, the vector is not *normalized* (i.e., if the sum of the square of its three coordinates is not 1), the user agent will normalize it internally. A non-normalizable vector, such as the null vector, [0, 0, 0], will cause the rotation to be ignored, but without invalidating the whole CSS property.

> **Note:** Unlike rotations in the 2D plane, the composition of 3D rotations is usually not commutative. In other words, the order in which the rotations are applied impacts the result.

Syntax

```
rotate3d(x, y, z, a)
```

Values

`x`

Is a `<number>` describing the x-coordinate of the vector denoting the axis of rotation which can be a positive or negative number.

`y`

Is a `<number>` describing the y-coordinate of the vector denoting the axis of rotation which can be a positive or negative number.

`z`

Is a `<number>` describing the z-coordinate of the vector denoting the axis of rotation which can be a positive or negative number.

`a`

Is an `<angle>` representing the angle of the rotation. A positive angle denotes a clockwise rotation, a negative angle a counter-clockwise one.

Cartesian coordinates on $\mathbb{R}^2$	This transformation applies to the 3D space and can't be represented on the plane.
Homogeneous coordinates on $\mathbb{RP}^2$	
Cartesian coordinates on $\mathbb{R}^3$	$\begin{pmatrix} 1+(1-\cos(a))(x^2-1) & z\cdot\sin(a)+xy(1-\cos(a)) & -y\cdot\sin(a)+xz(1-\cos(a)) \\ -z\cdot\sin(a)+xy(1-\cos(a)) & 1+(1-\cos(a))(y^2-1) & x\cdot\sin(a)+yz(1-\cos(a)) \\ y\cdot\sin(a)+xz(1-\cos(a)) & -x\cdot\sin(a)+yz(1-\cos(a)) & 1+(1-\cos(a))(z^2-1) \end{pmatrix}$
Homogeneous coordinates on $\mathbb{RP}^3$	$\begin{pmatrix} 1+(1-\cos(a))(x^2-1) & z\cdot\sin(a)+xy(1-\cos(a)) & -y\cdot\sin(a)+xz(1-\cos(a)) & 0 \\ -z\cdot\sin(a)+xy(1-\cos(a)) & 1+(1-\cos(a))(y^2-1) & x\cdot\sin(a)+yz(1-\cos(a)) & 0 \\ y\cdot\sin(a)+xz(1-\cos(a)) & -x\cdot\sin(a)+yz(1-\cos(a)) & 1+(1-\cos(a))(z^2-1) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<rotate3d()> =
  rotate3d( <number> , <number> , <number> , [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Rotating on the y-axis

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
body {
  perspective: 800px;
}

div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotate3d(0, 1, 0, 60deg);
  background-color: pink;
}
```

Result

Rotating on a custom axis

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
body {
  perspective: 800px;
}

div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotate3d(1, 2, -1, 192deg);
  background-color: pink;
}
```

Result

See also

- `transform` property
- `rotate` property
- `<transform-function>`

rotateX() CSS function

The **`rotateX()`** CSS function defines a transformation that rotates an element around the x-axis (horizontal) without deforming it. Its result is a `<transform-function>` data type.

Try it

```
transform: rotateX(0);
```

```
transform: rotateX(45deg);
```

```
transform: rotateX(-0.2turn);
```

```
transform: rotateX(3.142rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

The axis of rotation passes through an origin, defined by the `transform-origin` CSS property.

> **Note:** `rotateX(a)` is equivalent to `rotate3d(1, 0, 0, a)`.

> **Note:** Unlike rotations in the 2D plane, the composition of 3D rotations is usually not commutative. In other words, the order in which the rotations are applied impacts the result.

Syntax

```
rotateX(a)
```

Values

`a`

Is an `<angle>` representing the angle of the rotation. A positive angle denotes a clockwise rotation, a negative angle a counter-clockwise one.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
This transformation applies to the 3D space and can't be represented on the plane.		$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(a) & -\sin(a) \\ 0 & \sin(a) & \cos(a) \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(a) & -\sin(a) & 0 \\ 0 & \sin(a) & \cos(a) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<rotateX()> =
  rotateX( [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotateX(45deg);
  background-color: pink;
}
```

Result

See also

- `transform` property
- `rotate` property
- `<transform-function>`

rotateY() CSS function

The **`rotateY()`** CSS function defines a transformation that rotates an element around the y-axis (vertical) without deforming it. Its result is a `<transform-function>` data type.

Try it

```
transform: rotateY(0);
```

```
transform: rotateY(45deg);
```

```
transform: rotateY(-0.2turn);
```

```
transform: rotateY(3.142rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

The axis of rotation passes through an origin, defined by the `transform-origin` CSS property.

> **Note:** `rotateY(a)` is equivalent to `rotate3d(0, 1, 0, a)`.

> **Note:** Unlike rotations in the 2D plane, the composition of 3D rotations is usually not commutative. In other words, the order in which the rotations are applied impacts the result.

Syntax

```
rotateY(a)
```

Values

`a`

Is an `<angle>` representing the angle of the rotation. A positive angle denotes a clockwise rotation, a negative angle a counter-clockwise one.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
This transformation applies to the 3D space and can't be represented on the plane.		$\begin{pmatrix} \cos(a) & 0 & \sin(a) \\ 0 & 1 & 0 \\ -\sin(a) & 0 & \cos(a) \end{pmatrix}$	$\begin{pmatrix} \cos(a) & 0 & \sin(a) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(a) & 0 & \cos(a) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<rotateY()> =
  rotateY( [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotateY(60deg);
  background-color: pink;
}
```

Result

See also

- `transform` property
- `rotate` property
- `<transform-function>`

rotateZ() CSS function

The **`rotateZ()`** CSS function defines a transformation that rotates an element around the z-axis without deforming it. Its result is a `<transform-function>` data type.

Try it

```
transform: rotateZ(0);
```

```
transform: rotateZ(90deg);
```

```
transform: rotateZ(-0.25turn);
```

```
transform: rotateZ(3.142rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

The axis of rotation passes through an origin, defined by the `transform-origin` CSS property.

> **Note:** `rotateZ(a)` is equivalent to `rotate(a)` or `rotate3d(0, 0, 1, a)`.

> **Note:** Unlike rotations in the 2D plane, the composition of 3D rotations is usually not commutative. In other words, the order in which the rotations are applied impacts the result.

Syntax

```
rotateZ(a)
```

Values

`a`

Is an `<angle>` representing the angle of the rotation. A positive angle denotes a clockwise rotation, a negative angle a counter-clockwise one.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
This transformation applies to the 3D space and can't be represented on the plane.		$\begin{pmatrix} \cos(a) & -\sin(a) & 0 \\ \sin(a) & \cos(a) & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} \cos(a) & -\sin(a) & 0 & 0 \\ \sin(a) & \cos(a) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<rotateZ()> =
  rotateZ( [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotateZ(45deg);
  background-color: pink;
}
```

Result

See also

- `transform` property
- `rotate` property
- `<transform-function>`

scale() CSS function

The **`scale()`** CSS function defines a transformation that resizes an element on the 2D plane. Because the amount of scaling is defined by a vector [sx, sy], it can resize the horizontal and vertical dimensions at different scales. Its result is a `<transform-function>` data type.

Try it

```
transform: scale(1);
```

```
transform: scale(0.7);
```

```
transform: scale(1.3, 0.4);
```

```
transform: scale(-0.5, 1);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

This scaling transformation is characterized by a two-dimensional vector. Its coordinates define how much scaling is done in each direction. If both coordinates are equal, the scaling is uniform (*isotropic*) and the aspect ratio of the element is preserved (this is a homothetic transformation).

When a coordinate value is outside the [-1, 1] range, the element grows along that dimension; when inside, it shrinks. A negative value results in a point reflection in that dimension. The value `1` has no effect.

> **Note:** The `scale()` function only scales in 2D. To scale in 3D, use `scale3d()` instead.

Syntax

```
scale(sx)

scale(sx, sy)
```

Values

`sx`

A `<number>` or `<percentage>` representing the abscissa (horizontal, x-component) of the scaling vector.

`sy` Optional

A `<number>` or `<percentage>` representing the ordinate (vertical, y-component) of the scaling vector. If not defined, its default value is `sx`, resulting in a uniform scaling that preserves the element's aspect ratio.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
$\begin{pmatrix} sx & 0 \\ 0 & sy \end{pmatrix}$	$\begin{pmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
	`[sx 0 0 sy 0 0]`		

Formal syntax

```
<scale()> =
  scale( <number> , <number>? )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Accessibility

Scaling/zooming animations are problematic for accessibility, as they are a common trigger for certain types of migraine. If you need to include such animations on your website, you should provide a control to allow users to turn off animations, preferably site-wide.

Also, consider making use of the `prefers-reduced-motion` media feature — use it to write a media query that will turn off animations if the user has reduced animation specified in their system preferences.

Find out more:

- MDN Understanding WCAG, Guideline 2.3 explanations
- Understanding Success Criterion 2.3.3 | W3C Understanding WCAG 2.1

Examples

Scaling the X and Y dimensions together

HTML

```
<div>Normal</div>
<div class="scaled">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: scale(0.7); /* Equal to scaleX(0.7) scaleY(0.7) */
  background-color: pink;
}
```

Result

Scaling X and Y dimensions separately, and translating the origin

HTML

```
<div>Normal</div>
<div class="scaled">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: scale(2, 0.5); /* Equal to scaleX(2) scaleY(0.5) */
  transform-origin: left;
  background-color: pink;
}
```

Result

See also

- `transform`

- `scale`
- `zoom`
- `<transform-function>`
- `scale3d()`
- Other individual transform properties `translate` and `rotate`

scale3d() CSS function

The **`scale3d()`** CSS function defines a transformation that resizes an element in 3D space. Because the amount of scaling is defined by a vector [sx, sy, sz], it can resize different dimensions at different scales. Its result is a `<transform-function>` data type.

Try it

```
transform: scale3d(1, 1, 1);
```

```
transform: scale3d(1.3, 1.3, 1.3);
```

```
transform: scale3d(0.5, 1, 1.7);
```

```
transform: scale3d(-1.4, 0.4, 0.7);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

This scaling transformation is characterized by a three-dimensional vector. Its coordinates define how much scaling is done in each direction. If all three coordinates are equal, the scaling is uniform (*isotropic*) and the aspect ratio of the element is preserved (this is a homothetic transformation).

When a coordinate value is outside the [-1, 1] range, the element grows along that dimension; when inside, it shrinks. If it is negative, the result is a point reflection in that dimension. A value of 1 has no effect.

Syntax

```
scale3d(sx, sy, sz)
```

Values

`sx`

Is a `<number>` representing the abscissa (horizontal, x-component) of the scaling vector.

`sy`

Is a `<number>` representing the ordinate (vertical, y-component) of the scaling vector.

`sz`

Is a `<number>` representing the z-component of the scaling vector.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
This transformation applies to the 3D space and can't be represented on the plane.		$\begin{pmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & sz \end{pmatrix}$	$\begin{pmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<scale3d()> =
  scale3d( [ <number> | <percentage> ]#{3} )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Without changing the origin

HTML

```
<div>Normal</div>
<div class="scaled">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: perspective(500px) scale3d(2, 0.7, 0.2) translateZ(100px);
  background-color: pink;
}
```

Result

Translating the origin of the transformation

HTML

```
<div>Normal</div>
<div class="scaled">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: perspective(500px) scale3d(2, 0.7, 0.2) translateZ(100px);
  transform-origin: left;
  background-color: pink;
}
```

Result

See also

- `transform`
- `<transform-function>`
- `scaleZ()`
- `translate3d()`
- `rotate3d()`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`

scaleX() CSS function

The **`scaleX()`** CSS function defines a transformation that resizes an element along the x-axis (horizontally). Its result is a `<transform-function>` data type.

Try it

```
transform: scaleX(1);
```

```
transform: scaleX(0.7);
```

```
transform: scaleX(1.3);
```

```
transform: scaleX(-0.5);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

It modifies the abscissa (horizontal, x-coordinate) of each element point by a constant factor, except when the scale factor is 1, in which case the function is the identity transform. The scaling is not isotropic, and the angles of the element are generally not conserved, except for multiples of 90 degrees. `scaleX(-1)` defines an axial symmetry, with a vertical axis passing through the origin (as specified by the `transform-origin` property).

> **Note:** `scaleX(sx)` is equivalent to `scale(sx, 1)` or `scale3d(sx, 1, 1)`.

Syntax

```
scaleX(s)
```

Values

`s`

Is a `<number>` representing the scaling factor to apply on the abscissa (horizontal, x-coordinate) of each point of the element.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
$\begin{pmatrix} s & 0 \\ 0 & 1 \end{pmatrix}$	$\begin{pmatrix} s & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ `[s 0 0 1 0 0]`	$\begin{pmatrix} s & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} s & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<scaleX()> =
  scaleX( <number> )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="scaled">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: scaleX(0.6);
  background-color: pink;
}
```

Result

See also

- `scaleY()`
- `scaleZ()`
- `transform`
- `scale`
- `<transform-function>`
- `transform-origin`
- Other individual transform properties:
  - `translate`
  - `rotate`
  - Note: there is no `skew` property

scaleY() CSS function

The **`scaleY()`** CSS function defines a transformation that resizes an element along the y-axis (vertically). Its result is a `<transform-function>` data type.

Try it

```
transform: scaleY(1);
```

```
transform: scaleY(0.7);
```

```
transform: scaleY(1.3);
```

```
transform: scaleY(-0.5);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

It modifies the ordinate (vertical, y-coordinate) of each element point by a constant factor, except when the scale factor is 1, in which case the function is the identity transform. The scaling is not isotropic, and the angles of the element are not conserved. `scaleY(-1)` defines an axial symmetry, with a horizontal axis passing through the origin (as specified by the `transform-origin` property).

> **Note:** `scaleY(sy)` is equivalent to `scale(1, sy)` or `scale3d(1, sy, 1)`.
>
> `transform: rotateX(180deg); === transform: scaleY(-1);`

Syntax

```
scaleY(s)
```

Values

`s`

Is a `<number>` representing the scaling factor to apply on the ordinate (vertical, y-coordinate) of each point of the element.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
$\begin{pmatrix} 1 & 0 \\ 0 & s \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
	`[1 0 0 s 0 0]`		

Formal syntax

```
<scaleY()> =
  scaleY( <number> )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="scaled">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: scaleY(0.6);
  background-color: pink;
}
```

Result

See also

- `scaleX()`
- `scaleZ()`
- `transform`
- `<transform-function>`
- `transform-origin`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
  - Note: there is no `skew` property

scaleZ() CSS function

The **`scaleZ()`** CSS function defines a transformation that resizes an element along the z-axis. Its result is a `<transform-function>` data type.

Try it

```
transform: scaleZ(1);
```

```
transform: scaleZ(1.4);
```

```
transform: scaleZ(0.5);
```

```
transform: scaleZ(-1.4);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
```

```
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

This scaling transformation modifies the z-coordinate of each element point by a constant factor, except when the scale factor is 1, in which case the function is the identity transform. The scaling is not isotropic, and the angles of the element are not conserved. `scaleZ(-1)` defines an axial symmetry, with the z-axis passing through the origin (as specified by the `transform-origin` property).

In the above interactive examples, `perspective: 550px;` (to create a 3D space) and `transform-style: preserve-3d;` (so the children, the 6 sides of the cube, are also positioned in the 3D space), have been set on the cube.

> **Note:** `scaleZ(sz)` is equivalent to `scale3d(1, 1, sz)`.

Syntax

```
scaleZ(s)
```

Values

`s`

Is a `<number>` representing the scaling factor to apply on the z-coordinate of each point of the element.

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
This transformation applies to the 3D space and can't be represented on the plane.		$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & s \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<scaleZ()> =
  scaleZ( [ <number> | <percentage> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="perspective">Translated</div>
<div class="scaled-translated">Scaled</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.perspective {
  /* Includes a perspective to create a 3D space */
  transform: perspective(400px) translateZ(-100px);
  background-color: limegreen;
}

.scaled-translated {
  /* Includes a perspective to create a 3D space */
  transform: perspective(400px) scaleZ(2) translateZ(-100px);
  background-color: pink;
}
```

Result

See also

- `scaleX()`
- `scaleY()`
- `transform`
- `<transform-function>`
- `transform-origin`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
  - Note: there is no `skew` property

skew() CSS function

The **`skew()`** CSS function defines a transformation that skews an element on the 2D plane. Its result is a `<transform-function>` data type.

Try it

```
transform: skew(0);
```

```
transform: skew(15deg, 15deg);
```

```
transform: skew(-0.06turn, 18deg);
```

```
transform: skew(0.312rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

This transformation is a shear mapping (transvection) that distorts each point within an element by a certain angle in the horizontal and vertical directions. The effect is as if you grabbed each corner of the element and pulled them along a certain angle.

The coordinates of each point are modified by a value proportionate to the specified angle and the distance to the origin. Thus, the farther from the origin a point is, the greater the value added to it.

Syntax

```
skew(ax)

skew(ax, ay)
```

Values

`ax`

Is an `<angle>` representing the angle to use to distort the element along the x-axis.

`ay` Optional

Is an `<angle>` representing the angle to use to distort the element along the y-axis. If not defined, its default value is `0`, resulting in a purely horizontal skewing.

Cartesian coordinates on $\mathbb{R}^2$	**Homogeneous coordinates** on $\mathbb{RP}^2$	**Cartesian coordinates** on $\mathbb{R}^3$	**Homogeneous coordinates** on $\mathbb{RP}^3$
$\begin{pmatrix} 1 & \tan(ax) \\ \tan(ay) & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & \tan(ax) & 0 \\ \tan(ay) & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & \tan(ax) & 0 \\ \tan(ay) & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & \tan(ax) & 0 & 0 \\ \tan(ay) & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
	`[1 tan(ay) tan(ax) 1 0 0]`		

Formal syntax

```
<skew()> =
  skew( [ <angle> | <zero> ] , [ <angle> | <zero> ]? )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Skewing on the x-axis only

HTML

```
<div>Normal</div>
<div class="skewed">Skewed</div>
```

CSS

```
body {
  margin: 20px;
}

div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.skewed {
  transform: skew(10deg); /* Equal to skewX(10deg) */
  background-color: pink;
}
```

Result

Skewing on both axes

HTML

```
<div>Normal</div>
<div class="skewed">Skewed</div>
```

CSS

```
body {
  margin: 20px;
}

div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.skewed {
  transform: skew(10deg, 10deg);
  background-color: pink;
}
```

Result

See also

- `transform`
- `<transform-function>`
- skewX()
- skewY()
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
  - Note: there is no `skew` property

skewX() CSS function

The **`skewX()`** CSS function defines a transformation that skews an element in the horizontal direction on the 2D plane. Its result is a `<transform-function>` data type.

Try it

```
transform: skewX(0);
```

```
transform: skewX(35deg);
```

```
transform: skewX(-0.06turn);
```

```
transform: skewX(0.352rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

This transformation is a shear mapping (transvection) that distorts each point within an element by a certain angle in the horizontal direction. The abscissa (horizontal, x-coordinate) of each point is modified by a value proportionate to the specified angle and the distance to the origin; thus, the farther from the origin a point is, the greater will be the value added it.

> **Note:** `skewX(a)` is equivalent to `skew(a)`.

Syntax

```
skewX(a)
```

Values

`a`

Is an `<angle>` representing the angle to use to distort the element along the abscissa (horizontal, x-coordinate).

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
$\begin{pmatrix} 1 & \tan(a) \\ 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & \tan(a) & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & \tan(a) & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & \tan(a) & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
	`[1 0 tan(a) 1 0 0]`		

Formal syntax

```
<skewX()> =
  skewX( [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="skewed">Skewed</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.skewed {
  transform: skewX(10deg); /* Equal to skew(10deg) */
  background-color: pink;
}
```

Result

See also

- `transform`
- `<transform-function>`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
  - Note: there is no `skew` property

skewY() CSS function

The **`skewY()`** CSS function defines a transformation that skews an element in the vertical direction on the 2D plane. Its result is a `<transform-function>` data type.

Try it

```
transform: skewY(0);
```

```
transform: skewY(35deg);
```

```
transform: skewY(-0.06turn);
```

```
transform: skewY(0.352rad);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

This transformation is a shear mapping (transvection) that distorts each point within an element by a certain angle in the vertical direction. The ordinate (vertical, y-coordinate) of each point is modified by a value proportionate to the specified angle and the distance to the origin; thus, the farther from the origin a point is, the greater will be the value added it.

Syntax

```
skewY(a)
```

Values

`a`

Is an `<angle>` representing the angle to use to distort the element along the ordinate (vertical, y-coordinate).

Cartesian coordinates on $\mathbb{R}^2$	Homogeneous coordinates on $\mathbb{RP}^2$	Cartesian coordinates on $\mathbb{R}^3$	Homogeneous coordinates on $\mathbb{RP}^3$
$\begin{pmatrix} 1 & 0 \\ \tan(a) & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \\ \tan(a) & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ `[1 tan(a) 0 1 0 0]`	$\begin{pmatrix} 1 & 0 & 0 \\ \tan(a) & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ \tan(a) & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<skewY()> =
  skewY( [ <angle> | <zero> ] )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Normal</div>
<div class="skewed">Skewed</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.skewed {
  transform: skewY(40deg);
  background-color: pink;
}
```

Result

See also

- `transform`
- `<transform-function>`
- Individual transform properties:
  - `translate`
  - `scale`
  - `rotate`
  - Note: there is no `skew` property

translate() CSS function

The **`translate()`** CSS function repositions an element in the horizontal and/or vertical directions. Its result is a `<transform-function>` data type.

Try it

```
transform: translate(0);
```

```
transform: translate(42px, 18px);
```

```
transform: translate(-2.1rem, -2ex);
```

```
transform: translate(3ch, 3mm);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="static-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

```
#static-element {
  opacity: 0.4;
  position: absolute;
}

#example-element {
  position: absolute;
}
```

This transformation is characterized by a two-dimensional vector [tx, ty]. Its coordinates define how much the element moves in each direction.

Syntax

```
/* Single <length-percentage> values */
transform: translate(200px);
transform: translate(50%);

/* Double <length-percentage> values */
transform: translate(100px, 200px);
transform: translate(100px, 50%);
transform: translate(30%, 200px);
transform: translate(30%, 50%);
```

Values

Single `<length-percentage>` values

> This value is a `<length>` or `<percentage>` representing the abscissa (horizontal, x-component) of the translating vector [tx, 0]. The ordinate (vertical, y-component) of the translating vector will be set to `0`. For example, `translate(2px)` is equivalent to `translate(2px, 0)`. A percentage value refers to the width of the reference box defined by the `transform-box` property.

Double `<length-percentage>` values

> This value describes two `<length>` or `<percentage>` values representing both the abscissa (horizontal, x-component) and the ordinate (vertical, y-component) of the translating vector [tx, ty]. A percentage as first value refers to the width, as second part to the height of the reference box defined by the `transform-box` property.

Cartesian coordinates on ℝ^2	**Homogeneous coordinates on ℝP^2**	**Cartesian coordinates on ℝ^3**	**Homogeneous coordinates on ℝP^3**
A translation is not a linear transformation in ℝ^2 and can't be represented using a Cartesian-coordinate matrix.	$\begin{pmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{pmatrix}$ `[1 0 0 1 tx ty]`	$\begin{pmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<translate()> =
  translate( <length-percentage> , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using a single-axis translation

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
<div>Static</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  /* Equal to: translateX(10px) or translate(10px, 0) */
  transform: translate(10px);
  background-color: pink;
}
```

Result

Combining y-axis and x-axis translation

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
<div>Static</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  transform: translate(10px, 10px);
  background-color: pink;
}
```

Result

See also

- `transform`
- `<transform-function>`
- `translate`

translate3d() CSS function

The **`translate3d()`** CSS function repositions an element in 3D space. Its result is a `<transform-function>` data type.

Try it

```
transform: translate3d(0, 0, 0);
```

```
transform: translate3d(42px, -62px, -135px);
```

```
transform: translate3d(-2.7rem, 0, 1rem);
```

```
transform: translate3d(5ch, 0.4in, 5em);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

This transformation is characterized by a three-dimensional vector [tx, ty, tz]. Its coordinates define how much the element moves in each direction.

Syntax

```
translate3d(tx, ty, tz)
```

Values

`tx`

Is a `<length>` or `<percentage>` representing the abscissa (horizontal, x-component) of the translating vector [tx, ty, tz].

`ty`

Is a `<length>` or `<percentage>` representing the ordinate (vertical, y-component)of the translating vector [tx, ty, tz].

`tz`

Is a `<length>` representing the z-component of the translating vector. It can't be a `<percentage>` value; in that case the property containing the transform is considered invalid [tx, ty, tz].

Cartesian coordinates on $\mathbb{R}^2$	**Homogeneous coordinates on $\mathbb{RP}^2$**	**Cartesian coordinates on $\mathbb{R}^3$**	**Homogeneous coordinates on $\mathbb{RP}^3$**
This transformation applies to the 3D space and can't be represented on the plane.		A translation is not a linear transformation in $\mathbb{R}^3$ and can't be represented using a Cartesian-coordinate matrix.	$\begin{pmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<translate3d()> =
  translate3d( <length-percentage> , <length-percentage> , <length> )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using a single axis translation

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
<div>Static</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  /* Equivalent to perspective(500px) translateX(10px) */
  transform: perspective(500px) translate3d(10px, 0, 0px);
  background-color: pink;
}
```

Result

Combining z-axis and x-axis translation

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
<div>Static</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  transform: perspective(500px) translate3d(10px, 0, 100px);
  background-color: pink;
}
```

Result

See also

- transform
- <transform-function>
- translate

translateX() CSS function

The **translateX()** CSS function repositions an element horizontally on the 2D plane. Its result is a <transform-function> data type.

Try it

```
transform: translateX(0);
```

```
transform: translateX(42px);
```

```
transform: translateX(-2.1rem);
```

```
transform: translateX(3ch);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="static-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

```
#static-element {
  opacity: 0.4;
  position: absolute;
}
```

```
#example-element {
  position: absolute;
}
```

> **Note:** `translateX(tx)` is equivalent to `translate(tx, 0)` or `translate3d(tx, 0, 0)`.

Syntax

```
/* <length-percentage> values */
transform: translateX(200px);
transform: translateX(50%);
```

Values

`<length-percentage>`

Is a `<length>` or `<percentage>` representing the abscissa (horizontal, x-component) of the translating vector [tx, 0]. In Cartesian coordinate system it represents shift along x-axis. A percentage value refers to the width of the reference box defined by the `transform-box` property.

Cartesian coordinates on $\mathbb{R}^2$	**Homogeneous coordinates on $\mathbb{RP}^2$**	**Cartesian coordinates on $\mathbb{R}^3$**	**Homogeneous coordinates on $\mathbb{RP}^3$**
A translation is not a linear transformation in $\mathbb{R}^2$ and can't be represented using a Cartesian-coordinate matrix.	$\begin{pmatrix} 1 & 0 & t \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ `[1 0 0 1 t 0]`	$\begin{pmatrix} 1 & 0 & t \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & t \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<translateX()> =
  translateX( <length-percentage> )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
<div>Static</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  transform: translateX(10px); /* Equal to translate(10px) */
  background-color: pink;
}
```

Result

See also

- `translate()`
- `translateY()`
- `transform`
- `<transform-function>`
- `translate`

translateY() CSS function

The **`translateY()`** CSS function repositions an element vertically on the 2D plane. Its result is a `<transform-function>` data type.

Try it

```
transform: translateY(0);
```

```
transform: translateY(42px);
```

```
transform: translateY(-2.1rem);
```

```
transform: translateY(3ch);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="static-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
```

```
    width="200" />
</section>
```

```
#static-element {
  opacity: 0.4;
  position: absolute;
}

#example-element {
  position: absolute;
}
```

> **Note:** `translateY(ty)` is equivalent to `translate(0, ty)` or `translate3d(0, ty, 0)`.

Syntax

```
/* <length-percentage> values */
transform: translateY(200px);
transform: translateY(50%);
```

Values

`<length-percentage>`

The value is a `<length>` or `<percentage>` representing the ordinate (vertical, y-coordinate) of the translating vector [0, ty]. In Cartesian coordinate system it represents shift along y-axis. A percentage value refers to the height of the reference box defined by the `transform-box` property.

Cartesian coordinates on ℝ^2	**Homogeneous coordinates on ℙ^2**	**Cartesian coordinates on ℝ^3**	**Homogeneous coordinates on ℙ^3**
A translation is not a linear transformation in ℝ^2 and can't be represented using a Cartesian-coordinate matrix.	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & t \\ 0 & 0 & 1 \end{pmatrix}$ `[1 0 0 1 0 t]`	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & t \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<translateY()> =
  translateY( <length-percentage> )

<length-percentage> =
  <length>      |
  <percentage>
```

This syntax reflects the latest standard as per CSS Transforms Module Level 1, CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
<div>Static</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  transform: translateY(10px);
  background-color: pink;
}
```

Result

See also

- `transform`
- `<transform-function>`
- `translate`

translateZ() CSS function

The **`translateZ()`** CSS function repositions an element along the z-axis in 3D space, i.e., closer to or farther away from the viewer. Its result is a `<transform-function>` data type.

Try it

```
transform: translateZ(0);
```

```
transform: translateZ(42px);
```

```
transform: translateZ(-9.7rem);
```

```
transform: translateZ(-3ch);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div class="face front">1</div>
    <div class="face back">2</div>
```

```
    <div class="face right">3</div>
    <div class="face left">4</div>
    <div class="face top">5</div>
    <div class="face bottom">6</div>
  </div>
</section>
```

```
#default-example {
  background: linear-gradient(skyblue, khaki);
  perspective: 800px;
  perspective-origin: 150% 150%;
}

#example-element {
  width: 100px;
  height: 100px;
  perspective: 550px;
  transform-style: preserve-3d;
}

.face {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  position: absolute;
  backface-visibility: inherit;
  font-size: 60px;
  color: white;
}

.front {
  background: rgb(90 90 90 / 0.7);
  transform: translateZ(50px);
}

.back {
  background: rgb(0 210 0 / 0.7);
  transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgb(210 0 0 / 0.7);
  transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgb(0 0 210 / 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgb(210 210 0 / 0.7);
  transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgb(210 0 210 / 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
```

This transformation is defined by a `<length>` which specifies how far inward or outward the element or elements move.

In the above interactive examples, `perspective: 550px;` (to create a 3D space) and `transform-style: preserve-3d;` (so the children, the 6 sides of the cube, are also positioned in the 3D space), have been set on the cube.

> **Note:** `translateZ(tz)` is equivalent to `translate3d(0, 0, tz)`.

Syntax

```
translateZ(tz)
```

Values

`tz`

A `<length>` representing the z-component of the translating vector [0, 0, tz]. In Cartesian coordinate system it represents shift along z-axis. A positive value moves the element towards the viewer, and a negative value farther away.

Cartesian coordinates on $\mathbb{R}^2$	**Homogeneous coordinates on $\mathbb{RP}^2$**	**Cartesian coordinates on $\mathbb{R}^3$**	**Homogeneous coordinates on $\mathbb{RP}^3$**
This transformation applies to the 3D space and can't be represented on the plane.		A translation is not a linear transformation in $\mathbb{R}^3$ and can't be represented using a Cartesian-coordinate matrix.	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & t \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Formal syntax

```
<translateZ()> =
  translateZ( <length> )
```

This syntax reflects the latest standard as per CSS Transforms Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

In this example, two boxes are created. One is positioned normally on the page, without being translated at all. The second is altered by applying perspective to create a 3D space, then moved towards the user.

HTML

```
<div>Static</div>
<div class="moved">Moved</div>
```

CSS

```
div {
  position: relative;
  width: 60px;
  height: 60px;
  left: 100px;
  background-color: skyblue;
}

.moved {
  transform: perspective(500px) translateZ(200px);
  background-color: pink;
}
```

What really matters here is the class "moved"; let's take a look at what it does. First, the `perspective()` function positions the viewer relative to the plane that lies where z=0 (in essence, the surface of the screen). A value of `500px` means the user is 500 pixels "in front of" the imagery located at z=0.

Then, the `translateZ()` function moves the element 200 pixels "outward" from the screen, toward the user. This has the effect of making the element appear larger when viewed on a 2D display, or closer when viewed using a VR headset or other 3D display device.

Note if the `perspective()` value is less than the `translateZ()` value, such as `transform: perspective(200px) translateZ(300px);` the transformed element will not be visible as it is further than the user's viewport. The smaller the difference between the perspective and translateZ values, the closer the user is to the element and the larger the translated element will seem.

> **Note:** As the composition of transforms isn't commutative, the order you write the different functions is significant. In particular, in general, you want `perspective()` to be placed before `translateZ()`.

Result

See also

- `transform`
- `<transform-function>`
- `translate`

type() CSS function

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`type()`** CSS function allows you to specify a particular data type, and is used by the following CSS features:

- The `attr()` function, to specify the data type that an attribute value should be parsed into.
- The `@function` at-rule, to specify the permitted data types for CSS custom function parameters and results.

> **Note:** The `@property` at-rule `syntax` descriptor value uses the same `<syntax>` to define the permitted data types for registered custom properties, however, this always takes the form of a string.

Syntax

```
/* Single value */
type(<color>)
type(auto)

/* "|" combinator for multiple types */
type(<length> | <percentage>)

/* Space-separated list of values */
type(<color>+)

/* Comma-separated list of values */
type(<length>#)

/* Multiple keywords */
type(red | blue | green)

/* Combination of data type and keyword */
type(<percentage> | auto)

/* Universal syntax value */
type(*)
```

Parameters

The `type()` function's syntax is as follows:

```
type(<syntax>)
```

The `<syntax>` parameter is an expression defining the data type. This can take the following forms:

`<ident>`

A CSS keyword value, written without angle brackets.

`<syntax-type>`

A type name, written in angle brackets, representing a CSS data type. The following data types are supported:

`<angle>`

Accepts any valid `<angle>` value.

`<color>`

Accepts any valid `<color>` value.

`<custom-ident>`

Accepts any valid `<custom-ident>` value.

`<image>`

Accepts any valid `<image>` value.

`<integer>`

Accepts any valid `<integer>` value.

`<length>`

Accepts any valid `<length>` value.

`<length-percentage>`

Accepts any valid `<length>` or `<percentage>` value and any valid `calc()` expression combining `<length>` and `<percentage>` values.

`<number>`

Accepts any valid `<number>` value.

`<percentage>`

Accepts any valid `<percentage>` value.

`<resolution>`

Accepts any valid `<resolution>` value.

`<string>`

Accepts any valid `<string>` value.

`<time>`

Accepts any valid `<time>` value.

`<transform-function>`

Accepts any valid `<transform-function>` value.

`<transform-list>`

Accepts a list of valid `<transform-function>` values. It is equivalent to "`<transform-function>+`", and may not be followed by a `+` or `#` token.

`<url>`

Accepts any valid `<url>` value.

`*`

The universal syntax.

You can combine `<syntax-type>` values using the following tokens:

`+`

A space-separated list of values is expected.

`#`

A comma-separated list of values is expected.

In addition, the `|` token can be used as a separator when specifying multiple values or combining `<ident>` and `<syntax-type>` values for the expected syntax.

Return value

A data type definition.

Description

The `type()` function is used when you need to define a data type. It can be considered as a subset of the overall value definition syntax used to define the set of valid values for every CSS property and function.

Most commonly, you'll see `type()` used to specify a single data type. The next example uses the `attr()` function to set the `background-color` property equal to the value of a user-defined `data-background` function. The required data type for the value has been specified as a `<color>`.

```
background-color: attr(data-background type(<color>), red);
```

You could also specify an exact keyword requirement (for example, `type(blue)`), but this would be overly limiting.

Specifying `type(*)` permits any valid CSS value. This is the universal syntax, which cannot be multiplied or combined with other syntax components.

Specifying multiple permitted types

You can use the `|` token as a separator when specifying a range of permitted data types, keywords, or a combination of both. For example:

- `type(<length> | <percentage>)`
- `type(red | green)`
- `type(<length> | auto)`

The following example shows how a `@function` at-rule can be used to define a custom function that takes two color parameters, and returns the first one unless the viewport width is less than `700px`, in which case it returns the second one. The first one is allowed to be `red` or `green`, while the second one has to be `blue`.

```
@function --color-choice(--color1 type(red | green), --color2 blue) {
  result: var(--color1);
  @media (width < 700px) {
    result: var(--color2);
  }
}
```

> **Note:** In the case of `@function` data types, you can omit the `type()` function and just include the value in cases where only one data type or keyword is specified. This is the case with the `blue` type definition in the previous custom function. This doesn't work with the `attr()` function.

Specifying lists of types

The `+` and `#` tokens can be appended to a `<syntax-type>` to specify that you are expecting a space-separated list or a comma-separated list, respectively. For example:

- A `<color>+` parameter expects a space-separated list of `<color>` values, for example `red blue #a60000 rgb(234 45 100)`.
- A `<length>#` parameter expects a comma-separated lists of `<length>` values, for example `30px, 1em, 15vw`.

You can combine multiple tokens using `|` as a separator. For example, `<color># | <integer>#` would expect a comma-separated list of `<color>` values or a comma-separated list of `<integer>` values.

Formal syntax

```
<type()> =
  type( <string> )
```

This syntax reflects the latest standard as per CSS Images Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

You can find several other examples in the `attr()` and `@function` documentation.

Basic <code>@function</code> data type definition

This example defines a CSS custom function that combines multiple strings.

HTML

The HTML contains a single `<section>` element with some text content.

```
<section>What you gonna say?</section>
```

CSS

In the CSS, we start by specifying a `@function` called `--combine-strings`. This has a parameter called `--strings`, the data type of which is specified as one or more space-separated `<string>` values. It returns the string values with a space and a heart emoji appended onto the end.

```
@function --combine-strings(--strings type(<string>+)) {
  result: var(--strings) " ❤️";
}
```

We then specify some basic styles for the `<section>` element and use the `--combine-strings()` function to specify the value of its `content` property, including two space-separated strings as the argument.

```
section {
  font-family: system-ui;
  background-color: lime;
  padding: 20px;
}

section::after {
  content: --combine-strings("hello" "goodbye");
}
```

Result

See also

- `@function` at-rule
- `attr()` function
- Using CSS custom functions

unset CSS keyword

The **`unset`** CSS keyword resets a property to its inherited value if the property naturally inherits from its parent, and to its initial value if not. In other words, it behaves like the `inherit` keyword in the first case, when the property is an inherited property, and like the `initial` keyword in the second case, when the property is a non-inherited property.

`unset` can be applied to any CSS property, including the CSS shorthand property `all`.

Examples

Color

`color` is an inherited property.

HTML

```
<p>This text is red.</p>
<div class="foo">
  <p>This text is also red.</p>
</div>
<div class="bar">
  <p>This text is green (default inherited value).</p>
</div>
```

CSS

```
.foo {
  color: blue;
}
```

```
.bar {
  color: green;
}

p {
  color: red;
}

.bar p {
  color: unset;
}
```

Result

Border

`border` is a non-inherited property.

HTML

```
<p>This text has a red border.</p>
<div>
  <p>This text has a red border.</p>
</div>
<div class="bar">
  <p>This text has a black border (initial default, not inherited).</p>
</div>
```

CSS

```
div {
  border: 1px solid green;
}

p {
  border: 1px solid red;
}

.bar p {
  border-color: unset;
}
```

Result

See also

- Use the `initial` keyword to set a property to its initial value.
- Use the `inherit` keyword to make an element's property the same as its parent.
- Use the `revert` keyword to reset a property to the value established by the user-agent stylesheet (or by user styles, if any exist).
- Use the `revert-layer` keyword to reset a property to the value established in a previous cascade layer.
- Use the `revert-rule` keyword to reset a property to the value of an earlier matching style rule.
- The `all` property lets you reset all properties to their initial, inherited, reverted, or unset state at once.

url() CSS function

The **`url()`** CSS function is used to include a file. The parameter is an absolute URL, a relative URL, a blob URL, or a data URL. The **`url()`** function can be passed as a parameter of another CSS function, like the `attr()` function. Depending on the property for which it is a value, the resource sought can be an image, font, or a stylesheet. The `url()` functional notation is the value for the `<url>` data type.

> **Note:** There is a difference between a URI and a URL. A URI identifies a resource. A URL is a type of URI, and describes the *location* of a resource. A URI can be either a URL or a name (URN) of a resource.
>
> In CSS Level 1, the `url()` functional notation described only true URLs. In CSS Level 2, the definition of `url()` was extended to describe any URI, whether a URL or a URN. Confusingly, this meant that `url()` could be used to create a `<uri>` CSS data type. This change was not only awkward but, debatably, unnecessary, since URNs are rarely used in actual CSS. To alleviate the confusion, CSS Level 3 returned to the narrower, initial definition. Now, `url()` denotes only true `<url>`s.

Relative URLs are resolved relative to the stylesheet URL, not the web page URL. However, when a `url()` is used within a custom property (e.g., `--my-image: url(foo.jpg)`), the relative URL is not resolved immediately. It is resolved when the custom property is evaluated via `var()` substitution in a standard property. The base URL at that point is the stylesheet where the `var()` reference appears, not where the custom property was defined.

The **`url()`** function can be included as a value for `background`, `background-image`, `border-image`, `border-image-source`, `content`, `cursor`, `filter`, `list-style`, `list-style-image`, `mask`, `mask-image`, `offset-path`, `clip-path`, `src` as part of a `@font-face` block, and `@counter-style/symbol`

Syntax

```
/* Basic usage */
url("https://example.com/images/myImg.jpg");
url('https://example.com/images/myImg.jpg');
url(https://example.com/images/myImg.jpg);
url("…");
url(myImg.jpg);
url(#IDofSVGpath);

/* associated properties */
background-image: url("star.gif");
list-style-image: url('../images/bullet.jpg');
content: url("my-icon.jpg");
cursor: url(my-cursor.cur);
border-image-source: url(/media/diamonds.png);
src: url('fantastic-font.woff');
offset-path: url(#path);
mask-image: url("masks.svg#mask1");

/* Properties with fallbacks */
cursor: url(pointer.cur), pointer;

/* Associated short-hand properties */
background: url('star.gif') bottom right repeat-x blue;
```

```
border-image: url("/media/diamonds.png") 30 fill / 30px / 30px space;

/* As a parameter in another CSS function */
background-image: cross-fade(20% url(first.png), url(second.png));
mask-image: image(url(mask.png), skyblue, linear-gradient(black, transparent));

/* as part of a non-shorthand multiple value */
content: url(star.svg) url(star.svg) url(star.svg) url(star.svg) url(star.svg);

/* at-rules */
@document url("https://www.example.com/") { /* … */ }
@import url("https://www.example.com/style.css");
@namespace url(http://www.w3.org/1999/xhtml);
```

Values

`<string>`

A string specifying a URL, which is a relative or absolute address, or pointer, to the web resource to be included, or a data URL. You can also use a hash URL to reference the ID of an SVG shape or an SVG filter.

The quotes are generally optional—they are required if the URL includes parentheses, whitespace, or quotes (unless these characters are escaped), or if the address includes control characters above 0x7e. Normal string syntax rules apply: double quotes cannot occur inside double quotes and single quotes cannot occur inside single quotes unless escaped.

`<url-modifier>`

In the future, the `url()` function may support specifying a modifier, an identifier or a functional notation, which alters the meaning of the URL string. This is not supported and not fully defined in the specification.

Formal syntax

```
<url()> =
  url( <string> <url-modifier>* )  |
  <url-token>
```

This syntax reflects the latest standard as per CSS Values and Units Module Level 4. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

As the background property value

```
body {
  background: url("https://mdn.github.io/shared-assets/images/examples/leopard.jpg")
    #0000dd no-repeat fixed;
}
```

For setting an image as a list bullet

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
</ul>
```

```
ul {
  font-size: 3rem;
  margin: 0;
}
```

```
ul {
  list-style: outside
    url("https://mdn.github.io/shared-assets/images/examples/firefox-logo.svg");
}
```

Usage in the content property

HTML

```
<ul>
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
</ul>
```

CSS

```
li::after {
  content: " - "
    url("https://mdn.github.io/shared-assets/images/examples/star-white_16x16.png");
}
```

Result

Using a data URL

CSS

```
body {
  background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='90' height='45'%3E%3Cpath d='M10 10h60' stroke='%2300F' stroke-width='5'/%3E%3Cpath d='M10 20h60'
stroke='%230F0' stroke-width='5'/%3E%3Cpath d='M10 30h60' stroke='red' stroke-width='5'/%3E%3C/svg%3E");
}
```

Usage in filters

When a URL is used as a path for a filter, the URL must be one of the following:

1. The path to an SVG file with the ID of the filter appended.
2. the ID of the filter, if the SVG already exists on the page.

```
.blur {
  filter: url("my-file.svg#svg-blur"); /* the URL of an SVG file used as a filter */
}

.inline-blur {
  filter: url("#svg-blur"); /* the ID of an SVG that is embedded in the HTML page */
}
```

See also

- `<gradient>`
- `element()`
- `image()`
- `image-set()`
- `cross-fade()`

`<url>` CSS type

The **`<url>`** CSS data type is a pointer to a resource.

Syntax

```
<url> = url()
```

Values

The value may be an absolute or relative URL.

`<url()>`

The `url()` function accepts a URL, which may be written as a quoted string or as an unquoted URL token.

> **Note:** The CSS values and units module also introduces the `src()` function as a `<url>` value. Currently, no browser supports this feature.

Description

The `<url>` data type, written with the `url()` function, represents a `URL`, which is a pointer to an internal or external resource. The resource can be an image, a video, a CSS file, a font file, an SVG feature, etc. The URL may be absolute or relative.

```
/* Relative URL */
url("styles.css")
url("assets/icon.svg")
url("../assets/image.png")

/* Absolute URL */
url("http://example.com/fonts/myFont.woff2")
url("https://example.com/images/background.jpg")

/* Data URL */
url("data:image/svg+xml,%3Csvg'%3E%3Cpath d='M10 10h60' stroke='%2300F' stroke-width='5'/%3E%3Cpath d='M10 20h60' stroke='%230F0' stroke-width='5'/%3E%3C/svg%3E")
url("...")
```

External resources and CORS

The ability to import external resources via the `<url>` value is implementation-defined and often restricted for security reasons.

Depending on the CSS property on which a `<url>` referencing external resources is applied, the resource may be subject to Cross-Origin Resource Sharing (CORS) restrictions.

Some CSS properties, including `mask-image`, `filter`, as well as `clip-path` and a few others when referring to `<svg>` image elements, may require successful CORS validation when they cause external, cross-origin resources to be fetched in CORS mode. If CORS validation fails, the resource may be blocked and therefore not used for rendering.

Note that the `<url>` value type does not enforce CORS validation itself, but individual CSS properties do.

When opening an HTML file directly with `file://`, resources may fail because CORS rules apply locally. Modern browsers treat `file://` as a unique origin, meaning cross-file resources can get blocked. In this case, an HTTP server may be hosted to avoid CORS errors. The security behavior of `file://` URLs also varies depending on the browser and the operating system's file permissions.

Examples

Relative URL

```
body {
  background-image: url("images/background.jpg");
}
```

Absolute URL

```
body {
  background-image: url("https://example.com/images/background.jpg");
}
```

See also

- `url()`

var() CSS function

The **`var()`** CSS function can be used to insert the value of a custom property (sometimes called a "CSS variable") instead of any part of a value of another property.

The `var()` function cannot be used in property names, selectors or anything else besides property values. (Doing so usually produces invalid syntax, or else a value whose meaning has no connection to the variable.)

Try it

```
border-color: var(--color-a);
```

```
border-color: var(--color-b);
```

```
border-color: var(--color-c);
```

```
<section class="default-example" id="default-example">
  <div id="example-element">
    Three color options have been set on the :root use these to change the
    border color.
  </div>
</section>
```

```
:root {
  --color-a: pink;
  --color-b: green;
  --color-c: rebeccapurple;
}

#example-element {
  border: 10px solid black;
  padding: 10px;
}
```

Syntax

```
/* Basic usage */
var(--custom-prop);

/* With fallback */
var(--custom-prop,);  /* empty value as fallback */
var(--custom-prop, initial); /* initial value of the property as fallback */
var(--custom-prop, red);
var(--custom-prop, var(--default-value));
var(--custom-prop, var(--default-value, red));
```

The first argument to the function is the name of the custom property to be substituted. An optional second argument to the function serves as a fallback value. The fallback is used when the referenced custom property doesn't have a usable value — what the specification calls the guaranteed-invalid value. In practice, this means the fallback is used when:

- The custom property hasn't been declared on a rule that applies to the element, and it isn't registered (with `@property`) with an `initial-value` descriptor.
- The custom property is set to the `initial` keyword and isn't a registered custom property with an `initial-value`. Setting a non-registered custom property to `initial` resets it to the guaranteed-invalid value.
- The custom property's declared value is invalid at computed-value time — for example, because of a cyclic dependency between custom properties — and the property is non-registered or is registered with the universal `*` syntax.

If the custom property is registered with `@property` using a non-universal `syntax` and an `initial-value`, that initial value is substituted when no other declaration applies. The fallback is *not* used in that case.

The other CSS-wide keywords — `inherit`, `unset`, `revert`, `revert-layer`, and `revert-rule` — behave in the same way when set as a custom property value as they do on any other property. They can resolve the custom property to an inherited or previously-cascaded value rather than to the guaranteed-invalid value, so they don't necessarily trigger the fallback. For example, if `--foo` is set to `revert-layer` in one cascade layer and to a color in another layer, `var(--foo)` resolves to the color value instead of using the fallback.

The syntax of the fallback, like that of custom properties, allows commas. For example, `var(--foo, red, blue)` defines a fallback of `red, blue`; that is, anything between the first comma and the end of the function is considered a fallback value.

Values

`<custom-property-name>`

> A custom property's name represented by an identifier that starts with two dashes. Custom properties are solely for use by authors and users; CSS will never give them a meaning beyond what is presented here.

`<declaration-value>`

> A fallback value for the custom property, used when the referenced custom property has the guaranteed-invalid value (see the Syntax section above for the conditions under which this happens). This value may contain any character except some characters with special meaning like newlines, unmatched closing brackets, i.e., `)`, `]`, or `}`, top-level semicolons, or exclamation marks. The fallback value can itself be a custom property using the `var()` syntax. If the fallback value is omitted, and the custom property has the guaranteed-invalid value, the `var()` function resolves to an invalid value.

> **Note:** `var(--a,)` is valid, specifying that if the `--a` custom property has the guaranteed-invalid value, the `var()` should be replaced with nothing.

Formal syntax

```
<var()> =
  var( <custom-property-name> , <declaration-value>? )
```

This syntax reflects the latest standard as per CSS Custom Properties for Cascading Variables Module Level 2. Not all browsers may have implemented every part. See Browser compatibility for support information.

Examples

Using a custom property set on :root

CSS

```
:root {
  --main-bg-color: pink;
}

body {
  background-color: var(--main-bg-color);
}
```

Result

Here, the value of the `background-color` property has been set via the custom property `--main-bg-color`. So the background color of the HTML body will be pink.

Using a custom property before it is set

CSS

```
body {
  background-color: var(--main-bg-color);
}

:root {
  --main-bg-color: pink;
}
```

Result

In this example, the background color of the HTML body will be pink even though the custom property is set later.

Using a custom property set in another file

HTML

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="1.css" />
    <link rel="stylesheet" href="2.css" />
  </head>
  <body></body>
</html>
```

CSS

```
/* 1.css */
body {
  background-color: var(--main-bg-color);
}
```

```
/* 2.css */
:root {
  --main-bg-color: pink;
}
```

Result

The background color of the HTML body will be pink in this case even though the custom property is declared in another file.

Custom properties with fallbacks for use when the property has not been set

HTML

```
<div class="component">
  <h1 class="header">Header</h1>
  <p class="text">Text</p>
</div>
```

CSS

```
/* In the component's style: */
.component .header {
  /* header-color isn't set, and so remains blue, the fallback value */
  color: var(--header-color, blue);
}

.component .text {
  color: var(--text-color, black);
}

/* In the larger application's style: */
.component {
  --text-color: #008800;
}
```

Result

Since `--header-color` isn't set, the text "Header" will be blue, the fallback value.

Using a custom property as a fallback

CSS

```
:root {
  --backup-bg-color: teal;
}

body {
  background-color: var(--main-bg-color, var(--backup-bg-color, white));
}
```

Result

Since `--main-bg-color` isn't set, the body's `background-color` will fall back to `--backup-bg-color`, which is teal.

Invalid values

`var()` functions can resolve to invalid values if:

- The custom property is not defined and no fallback value is provided.
- The custom property is defined but its value is an invalid value for the property it is used in.

When this happens, the property is treated as if it has value `unset`. This is because variables can't "fail early" like other syntax errors can, so by the time the user agent realizes a property value is invalid, it has already thrown away the other cascaded values.

For example:

HTML

```
<p class="p1">Undefined variable</p>
<p class="p2">Invalid variable</p>
<p class="p3">Invalid literal color</p>
```

CSS

```
p {
  color: red;
}

.p1 {
  color: var(--invalid-color);
}

.p2 {
  --invalid-color: 20px;
  color: var(--invalid-color);
}

.p3 {
  color: 20px;
}
```

Result

Note how the paragraphs using `var()` are reset to the default black, but the paragraph with an invalid literal color is still red, because the `color: 20px` declaration is simply ignored.

See also

- `env(…)` – read-only environment variables controlled by the user-agent.
- Using CSS custom properties (variables)
- `@property` at-rule
- Registering CSS custom properties
- CSS custom properties for cascading variables module

WebKit (-webkit-) vendor-prefixed CSS extensions

A vendor prefix is used to indicate that a feature is specific to a certain browser. User agents based on WebKit or Blink (such as Safari and Chrome) support several extensions to CSS, which are prefixed with `-webkit-`.

-webkit-prefixed properties without standard equivalents

> **Note:** These properties work in WebKit- or Blink-based browsers except where support notes say otherwise. You should avoid using them on production websites.

A-C

- `-webkit-app-region` : No longer supported in Safari.
- `-webkit-border-horizontal-spacing`
- `-webkit-border-vertical-spacing`
- `-webkit-box-reflect`: Supported with `-webkit-` by every browser, for compatibility reasons.
- `-webkit-column-axis`: Not supported in Chrome.
- `-webkit-column-progression`: Not supported in Chrome.
- `-webkit-cursor-visibility`: Not supported in Chrome.

D-L

- `-webkit-font-smoothing`: See `font-smooth`.
- `-webkit-hyphenate-limit-after`: Not supported in Chrome.
- `-webkit-hyphenate-limit-before`: Not supported in Chrome.
- `-webkit-hyphenate-limit-lines`: Not supported in Chrome.
- `-webkit-line-align`: Not supported in Chrome.
- `-webkit-line-box-contain`: Not supported in Chrome.
- `-webkit-line-grid`: Not supported in Chrome.
- `-webkit-line-snap`: Not supported in Chrome.
- `-webkit-locale`
- `-webkit-logical-height`
- `-webkit-logical-width`

M

- `-webkit-mask-box-image`: See `mask-border` and `border-image`.
- `-webkit-mask-box-image-outset`: See `mask-border` and `border-image`.
- `-webkit-mask-box-image-repeat`: See `mask-border` and `border-image`.
- `-webkit-mask-box-image-slice`: See `mask-border` and `border-image`.
- `-webkit-mask-box-image-source`: See `mask-border` and `border-image`.
- `-webkit-mask-box-image-width`: See `mask-border` and `border-image`.
- `-webkit-mask-composite`: See `mask-border` and `border-image`.
- `-webkit-mask-position-x`: Supported with `-webkit-` by every browser for compatibility reasons.
- `-webkit-mask-position-y`: Supported with `-webkit-` by every browser for compatibility reasons.
- `-webkit-mask-repeat-x` : No longer supported; see `mask-repeat`.
- `-webkit-mask-repeat-y` : No longer supported; see `mask-repeat`.
- `-webkit-mask-source-type`: Not supported in Chrome.
- `-webkit-max-logical-height`
- `-webkit-max-logical-width`
- `-webkit-min-logical-height`
- `-webkit-min-logical-width`

N-Z

- `-webkit-nbsp-mode`: Not supported in Chrome.
- `-webkit-perspective-origin-x`
- `-webkit-perspective-origin-y`
- `-webkit-rtl-ordering`
- `-webkit-tap-highlight-color`: Only supported in Safari on iOS.
- `-webkit-text-decoration-skip`: Not supported in Chrome.
- `-webkit-text-decorations-in-effect`
- `-webkit-text-fill-color`
- `-webkit-text-security`
- `-webkit-text-stroke`
- `-webkit-text-stroke-color`
- `-webkit-text-stroke-width`
- `-webkit-text-zoom`: Not supported in Chrome.
- `-webkit-touch-callout` : Only supported in Safari on iOS.
- `-webkit-transform-origin-x`
- `-webkit-transform-origin-y`
- `-webkit-transform-origin-z`
- `-webkit-user-drag`
- `-webkit-user-modify`

-webkit-prefixed properties with standard equivalents

Several `-webkit-` prefixed properties have standard equivalents. Even if the name and syntax may be different, they shouldn't be used any more. For each of the properties below, use the standard equivalents.

A-B

- `-webkit-border-after`: Use `border-block-end`.
- `-webkit-border-after-color`: Use `border-block-end-color`.
- `-webkit-border-after-style`: Use `border-block-end-style`.
- `-webkit-border-after-width`: Use `border-block-end-width`.
- `-webkit-border-before`: Use `border-block-start`.
- `-webkit-border-before-color`: Use `border-block-start-color`.
- `-webkit-border-before-style`: Use `border-block-start-style`.
- `-webkit-border-before-width`: Use `border-block-start-width`.
- `-webkit-border-end`: Use `border-inline-end`.
- `-webkit-border-end-color`: Use `border-inline-end-color`.
- `-webkit-border-end-style`: Use `border-inline-end-style`.
- `-webkit-border-end-width`: Use `border-inline-end-width`.
- `-webkit-border-start`: Use `border-inline-start`.
- `-webkit-border-start-color`: Use `border-inline-start-color`
- `-webkit-border-start-style`: Use `border-inline-start-style`.
- `-webkit-border-start-width`: Use `border-inline-start-width`.
- `-webkit-box-align`: Use CSS flexbox with `align-items`.
- `-webkit-box-direction`: Use CSS flexbox with `flex-direction`.
- `-webkit-box-flex-group`: Use CSS flexbox with `flex-basis`, `flex-grow`, and `flex-shrink`.
- `-webkit-box-flex`: Use CSS flexbox with `flex-grow`.
- `-webkit-box-lines`: Use CSS flexbox with `flex-flow`.
- `-webkit-box-ordinal-group`: Use CSS flexbox with `order`.
- `-webkit-box-orient`: Use CSS flexbox with `flex-direction`.
- `-webkit-box-pack`: Use CSS flexbox with `justify-content`.
- `-webkit-box-reflect`: Use the CSS `element()` function.

C-I

- `-webkit-column-break-after`: Use CSS multicolumn layout with `break-after`.
- `-webkit-column-break-before`: Use CSS multicolumn layout with `break-before`.
- `-webkit-column-break-inside`: Use CSS multicolumn layout with `break-inside`.
- `-webkit-font-feature-settings`: Use `font-feature-settings` (the prefixed version not supported in Safari).
- `-webkit-hyphenate-character`: Use `hyphenate-character`.
- `-webkit-initial-letter`: Use `initial-letter`.

J-Z

- `-webkit-line-clamp`: Use `line-clamp`.
- `-webkit-margin-after`: Use `margin-block-end`.
- `-webkit-margin-before`: Use `margin-block-start`.
- `-webkit-margin-end`: Use `margin-inline-end`.
- `-webkit-margin-start`: Use `margin-inline-start`.
- `-webkit-padding-after`: Use `padding-block-end`.
- `-webkit-padding-before`: Use `padding-block-start`.
- `-webkit-padding-end`: Use `padding-inline-end`.
- `-webkit-padding-start`: Use `padding-inline-start`.

-webkit-prefixed property values

`-webkit-fill-available`

Used with sizing properties like `width` and `height`, to allow elements to take up all available space within their parent container. The `stretch` value provides a standard replacement, but `-webkit-fill-available` is supported as an alias by browsers for backwards-compatibility reasons.

Pseudo-classes

> **Note:** If there is an invalid pseudo-class within in a chain or group of selectors, the whole selector list is invalid.

- `:-webkit-any()`: Use `:is`
- `:-webkit-any-link`: Use `:any-link`
- `:-webkit-autofill`: Use `:autofill`

- `:-webkit-autofill-strong-password`: Use `:autofill`
- `:-webkit-drag`
- `:-webkit-full-page-media`: Use `:fullscreen`
- `:-webkit-full-screen`: Use `:fullscreen`
- `:-webkit-full-screen-ancestor`: Use `:fullscreen`
- `:-webkit-full-screen-document`: Use `:fullscreen`
- `:-webkit-full-screen-controls-hidden`: Use `:fullscreen`

Pseudo-elements

For web-compatibility reasons, Blink, WebKit, and Gecko browsers treat all pseudo-elements starting with `::-webkit-` as valid. If there is an invalid pseudo-element or pseudo-class within in a chain or group of selectors, the whole selector list is invalid. If a pseudo-element (but not pseudo-class) has a `-webkit-` prefix, Blink, WebKit and Gecko browsers assume it is valid, not invalidating the selector list.

- `::-webkit-file-upload-button`: Use `::file-selector-button`
- `::-webkit-inner-spin-button`
- `::-webkit-input-placeholder`: Use `::placeholder`
- `::-webkit-meter-bar`
- `::-webkit-meter-even-less-good-value`
- `::-webkit-meter-inner-element`
- `::-webkit-meter-optimum-value`
- `::-webkit-meter-suboptimum-value`
- `::-webkit-progress-bar`
- `::-webkit-progress-inner-element`
- `::-webkit-progress-value`
- `::-webkit-search-cancel-button`
- `::-webkit-search-results-button`
- `::-webkit-slider-runnable-track`
- `::-webkit-slider-thumb`

Media features

- `-webkit-animation`
- `-webkit-device-pixel-ratio`: Supported cross-browser
- `-webkit-transform-2d`
- `-webkit-transform-3d`: Supported cross-browser
- `-webkit-transition`

See also

- Mozilla (-moz-) vendor-prefixed CSS extensions
- Vendor Prefix glossary entry

CSS Tutorials

Learning CSS may be a daunting task. In order to help you, we have written numerous **tutorials about CSS**. Some are aimed at complete beginners, while others present complex features to be used by more experienced users.

This page lists them all, with a short description. They are grouped by complexity so that you can choose the most appropriate for your level.

Beginner-level CSS tutorials

CSS basics

> This guide is aimed at complete beginners: You haven't written one single line of CSS? — this is for you. It explains the fundamental concepts of the language and guides you in writing basic stylesheets.

Using multiple backgrounds

> Backgrounds are fundamental for nice styling: CSS allows you to set several of them on each box. This tutorial explains how they interact and how to achieve nice effects.

Resizing background images

> CSS allows you to resize images used as an element's background. This tutorial describes how to achieve this.

Using media queries

> The size of the screens, or the kind of devices like touchscreens or printed sheets vary greatly nowadays. Media queries are the fundamental building blocks in building websites that render everywhere in their best quality.

Understanding CSS z-index

> Controlling superposition of boxes is a basic feature that is very quickly needed by Web developers. Though not that difficult, it requires a basic knowledge of CSS.

Intermediate-level CSS tutorials

After the release of CSS 2 (Level 1), new features have been added to CSS. Some of them are *fancy* and are pretty self-contained. They are easy to use for anybody with a fair knowledge of basic concepts.

CSS Counter Styles

> Counting items and pages is an easy task in CSS. Learn to use `counter-reset`, `counter-increment`, `counters()`, and `counter()`.

CSS Animations

> CSS Animations allow you to define configurations of style, as keyframes, and to transition between them defining an animation.

CSS Transitions

> CSS Transitions allow you to define an animation between several styles and to control the way this transition happens.

CSS Transforms

> Transforms allow you to change the position of elements by modifying their coordinate space: it allows for translating, rotating, and deforming them in the 2D or 3D spaces.

CSS Gradients

> Gradients are images that transition smoothly from one color to another. There are several types of gradients allowed in CSS: linear or radial, repeating or not. This tutorial describes how to use them.

Advanced-level CSS tutorials

CSS also got new features allowing you to create complex layouts. Though the simplest way to achieve such layout, they are more complex to use for people without too much experience.

CSS multi-column layouts

> CSS3 introduces a new layout allowing you to easily define multiple columns in an element. Though multi-column text is not that common on devices like screens, this is particularly useful on printed pages, or for indexes.

CSS flexible boxes layouts

> This new layout allow you to give boxes flexibility, allowing them to be resized smoothly. It is a powerful way to describe complex interfaces.